🐧 সহজ বাংলায় লিনাক্স — শূন্য থেকে এক্সপার্ট (A to Z)

> একদম নতুন (fresher) মানুষের জন্য লেখা একটি পূর্ণাঙ্গ লিনাক্স বই। শুধু "কোন কমান্ড" না — **কেন**, **কীভাবে**, আর **ভেতরে কী ঘটে** — সবকিছু সহজ বাংলায়।

এই বই কার জন্য?

- যে আগে কখনো লিনাক্স ব্যবহার করেনি
- যে Windows থেকে লিনাক্সে আসতে চায়
- যে SysAdmin, DevOps, Cloud, Cyber Security বা Backend Developer হতে চায়
- যে কমান্ড মুখস্থ না করে **আর্কিটেকচার বুঝে** লিনাক্স শিখতে চায়

কীভাবে পড়বে?

1. **ক্রম অনুযায়ী পড়ো।** প্রতিটি অধ্যায় আগের অধ্যায়ের উপর দাঁড়িয়ে আছে।
2. **পড়ার সাথে সাথে টার্মিনালে টাইপ করো।** শুধু পড়ে লিনাক্স শেখা যায় না, হাতে-কলমে করতে হয়।
3. **প্রতিটি অধ্যায়ের শেষে "হাতে-কলমে অনুশীলন" আর "নিজেকে যাচাই করো" অংশ** অবশ্যই করবে।
4. **একটা Virtual Machine-এ অনুশীলন করো** (অধ্যায় ৩ দেখো), যাতে ভুল হলেও আসল কম্পিউটারের কোনো ক্ষতি না হয়।
5. শর্টকাটগুলো এক জায়গায় পেতে পরিশিষ্ট ক প্রিন্ট করে পাশে রাখো।

চিহ্নগুলোর মানে

চিহ্ন	মানে
`$ command`	সাধারণ ইউজার হিসেবে চালানো কমান্ড (`$` টাইপ করবে না)
`# command`	root/sudo হিসেবে চালানো কমান্ড
> ⚠️ সতর্কতা	ভুল করলে ক্ষতি হতে পারে
> 💡 টিপস	কাজ সহজ করার কৌশল
> 📝 মনে রাখো	জরুরি কথা

📚 সূচিপত্র

ভাগ ১ — শুরুর কথা

#	অধ্যায়
১	লিনাক্স কী এবং কেন শিখবে
২	লিনাক্স আর্কিটেকচার — ভেতরের গঠন
৩	ডিস্ট্রিবিউশন বাছাই ও ইনস্টলেশন
৪	ডেস্কটপ পরিবেশ ও GUI পরিচিতি
৫	টার্মিনাল ও শেল পরিচিতি
৬	টার্মিনাল শর্টকাট ও কমান্ড হিস্ট্রি

ভাগ ২ — ফাইল সিস্টেম

#	অধ্যায়
৭	ফাইল সিস্টেম হায়ারার্কি (FHS)
৮	ফাইল সিস্টেমে চলাফেরা
৯	ফাইল ও ডিরেক্টরি ম্যানেজমেন্ট
১০	ফাইলের ভেতরের লেখা দেখা
১১	inode, হার্ড লিংক ও সফট লিংক
১২	ফাইল ও কমান্ড খোঁজা
১৩	nano টেক্সট এডিটর
১৪	Vim — সম্পূর্ণ গাইড

ভাগ ৩ — ইউজার, পারমিশন ও নিরাপত্তার ভিত্তি

#	অধ্যায়
১৫	ইউজার ও গ্রুপ ম্যানেজমেন্ট
১৬	ফাইল পারমিশন
১৭	বিশেষ পারমিশন, ACL ও ফাইল অ্যাট্রিবিউট
১৮	root, sudo ও প্রশাসনিক ক্ষমতা

ভাগ ৪ — টেক্সট প্রসেসিং (লিনাক্সের আসল শক্তি)

#	অধ্যায়
১৯	ইনপুট/আউটপুট রিডাইরেকশন ও পাইপ
২০	grep ও রেগুলার এক্সপ্রেশন
২১	sed — স্ট্রিম এডিটর
২২	awk — টেক্সট প্রসেসিং ভাষা
২৩	আরও টেক্সট টুল — cut, sort, uniq, tr, xargs

ভাগ ৫ — প্রসেস ও সিস্টেম পরিচালনা

#	অধ্যায়
২৪	প্রসেস ম্যানেজমেন্ট
২৫	জব কন্ট্রোল, nohup, screen ও tmux
২৬	systemd ও সার্ভিস ম্যানেজমেন্ট
২৭	লিনাক্স বুট প্রসেস
২৮	লগ ও সিস্টেম লগ বিশ্লেষণ
২৯	কাজ শিডিউল করা — cron, at ও systemd timers

ভাগ ৬ — সফটওয়্যার

#	অধ্যায়
৩০	প্যাকেজ ম্যানেজমেন্ট (apt, dnf, pacman, snap, flatpak)
৩১	সোর্স কোড থেকে সফটওয়্যার বিল্ড করা

ভাগ ৭ — ডিস্ক ও স্টোরেজ

#	অধ্যায়
৩২	ডিস্ক, পার্টিশন, ফাইলসিস্টেম ও মাউন্ট
৩৩	LVM ও RAID
৩৪	আর্কাইভ ও কম্প্রেশন
৩৫	ব্যাকআপ ও সিঙ্ক — rsync, dd

ভাগ ৮ — নেটওয়ার্কিং ও সার্ভার

#	অধ্যায়
৩৬	নেটওয়ার্কিং-এর মূলনীতি
৩৭	নেটওয়ার্ক কমান্ড ও ট্রাবলশুটিং
৩৮	SSH ও রিমোট অ্যাক্সেস
৩৯	ফায়ারওয়াল — ufw, firewalld, iptables, nftables
৪০	সার্ভার সেটআপ — ওয়েব সার্ভার ও সার্ভিস

ভাগ ৯ — শেল স্ক্রিপ্টিং ও অটোমেশন

#	অধ্যায়
৪১	শেল স্ক্রিপ্টিং — পর্ব ১ (ভিত্তি)
৪২	শেল স্ক্রিপ্টিং — পর্ব ২ (লুপ, ফাংশন, অ্যারে)
৪৩	শেল স্ক্রিপ্টিং — পর্ব ৩ (অ্যাডভান্সড ও বাস্তব প্রজেক্ট)
৪৪	এনভায়রনমেন্ট, .bashrc, alias ও কাস্টমাইজেশন

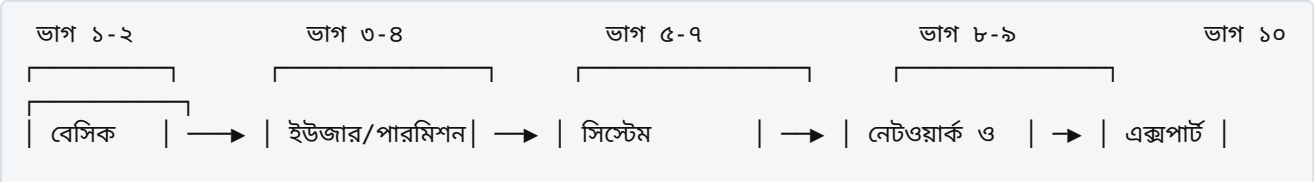
ভাগ ১০ — এক্সপার্ট লেভেল

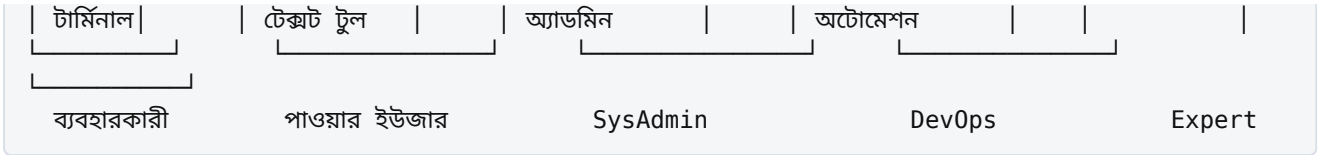
#	অধ্যায়
৪৫	লিনাক্স নিরাপত্তা ও হার্ডেনিং
৪৬	পারফরম্যান্স মনিটরিং ও ট্রাবলশুটিং
৪৭	কার্নেল, মডিউল, /proc ও /sys
৪৮	কন্টেইনার ও ভার্চুয়ালাইজেশন

পরিশিষ্ট

ক	সব শর্টকাটের মহা-চিটশিট
খ	কমান্ড রেফারেন্স A-Z, শেখার রোডম্যাপ ও ইন্টারভিউ প্রস্তুতি

🗺️ শেখার পথ এক নজরে





শুভ শেখা! 🚀 — প্রথম অধ্যায় থেকে শুরু করো: লিনাক্স কী এবং কেন শিখবে →

পরিশিষ্ট ক: সব শর্টকাটের মহা-চিটশিট

> এই পরিশিষ্টটা পড়ার জন্য না, **পাশে রেখে কাজ করার** জন্য। পুরো বইয়ে যত কীবোর্ড শর্টকাট এসেছে — ডেস্কটপ, টার্মিনাল, Bash, এডিটর, tmux, সিস্টেম-লেভেলের জরুরি কী — সব এখানে ক্যাটাগরি অনুযায়ী টেবিলে সাজানো। প্রিন্ট করে টেবিলের পাশে রাখো, বা আলাদা ট্যাবে খুলে রাখো।

এই পরিশিষ্টে যা শিখবে

- শর্টকাট লেখার নোটেশন (`Ctrl+X` , `C-x` , `^X` , `M-x` , `Super` , `Meta`) কীভাবে পড়তে হয়
- একটা কী চাপলে সেটা কীবোর্ড থেকে প্রোগ্রাম পর্যন্ত কোন পথে যায় — কেন একই `Ctrl+C` কোথাও "কপি", কোথাও "প্রোগ্রাম থামাও"
- GNOME ও KDE Plasma ডেস্কটপের শর্টকাট, ফাইল ম্যানেজারের শর্টকাট
- টার্মিনাল emulator (GNOME Terminal, Konsole) এর শর্টকাট
- Bash readline-এর **সম্পূর্ণ** শর্টকাট — কার্সর চলা, এডিট, kill/yank, হিস্ট্রি, কমপ্লিশন
- History expansion (`!!` , `!$` , `^old^new` ইত্যাদি)
- `less` / `man` , `nano` , `vim` (বড় টেবিল), `tmux` , `screen` , `top` , `htop` এর কী
- TTY (ভার্চুয়াল কনসোল) বদলানো — `Ctrl+Alt+F1` … `F6`
- Magic SysRq এবং **REISUB** — সিস্টেম জমে গেলে নিরাপদে রিবুট
- SSH escape sequence (`~.` ইত্যাদি) আর GRUB মেনুর কী
- কীভাবে এই চিটশিটটা প্রিন্ট/PDF করবে

শর্টকাট নোটেশন — কীভাবে পড়বে

বিভিন্ন ডকুমেন্টেশন একই জিনিস বিভিন্নভাবে লেখে। আগে এগুলো চিনে নাও, না হলে `man` পেজ পড়তে গিয়ে ধাঁধায় পড়বে।

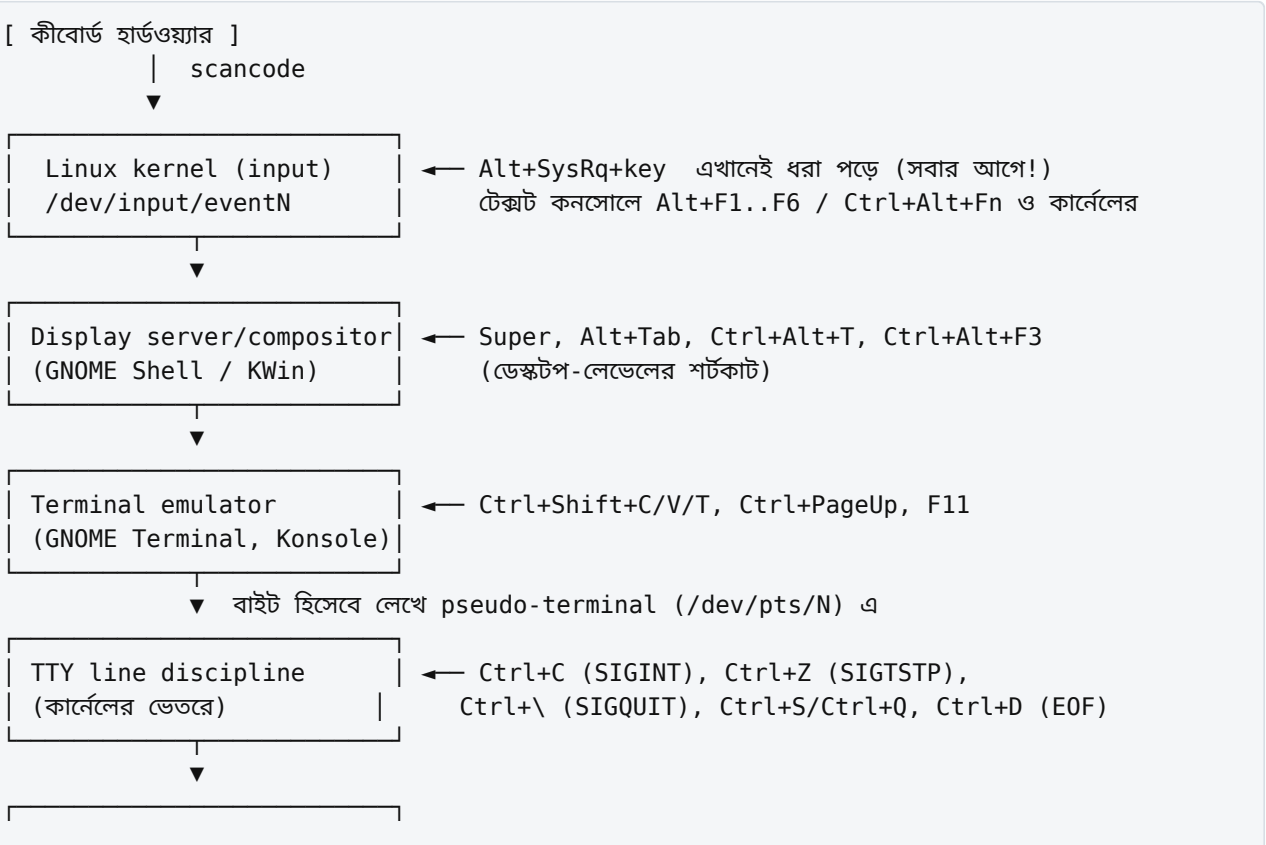
নোটেশন	মানে	কোথায় দেখবে
`Ctrl+X`	Ctrl চেপে ধরে X চাপো	এই বই, GUI ডকুমেন্টেশন
`C-x`	একই — Ctrl+X	Emacs, Bash `man bash` , tmux ডক
`^X`	একই — Ctrl+X (ক্যারেট = Ctrl)	nano-র নিচের মেনু, `stty -a` , টার্মিনালে দেখানো `^C`
`M-x`	Meta+X — লিনাক্সে সাধারণত **Alt+X**, অথবা Esc চেপে ছেড়ে তারপর X	Bash readline, nano, Emacs

নোটেশন	মানে	কোথায় দেখবে
`Super` / `Meta` (KDE) / `Win`	উইন্ডোজ লোগো কী	GNOME, KDE
`Ctrl+b c`	প্রথমে Ctrl+b চেপে **ছাড়ো**, তারপর c চাপো (prefix + key)	tmux, screen
`Ctrl+X Ctrl+E`	Ctrl+X, তারপর Ctrl+E — পরপর দুটো কম্বিনেশন	Bash readline
`<Esc>` , `<CR>`	Esc কী, Enter (Carriage Return)	Vim ডকুমেন্টেশন
`Alt+SysRq+B`	Alt আর SysRq (সাধারণত PrintScreen কী) চেপে ধরে B	কার্নেল ডক

> 📝 **মনে রাখো:** `Ctrl+Shift+C` আর `Ctrl+C` সম্পূর্ণ আলাদা কম্বিনেশন। টার্মিনালে কপি করতে `Ctrl+Shift+C` লাগে, কারণ শুধু `Ctrl+C` এর মানে টার্মিনালের ভেতরে "চলমান প্রোগ্রামকে থামাও"। কেন — নিচে দেখো।

ভেতরে কী হয়: একটা কী চাপলে সেটা কোথায় যায়?

শর্টকাট মুখস্থ করার আগে এটা বোঝা জরুরি, কারণ এতে বুঝবে **কোন শর্টকাট কে ধরে ফেলে**। ধরো একটা অফিস ভবন: কীবোর্ড হলো রিসেপশন। একটা চিঠি (কী-প্রেস) এলে রিসেপশন সেটা ভবনের ভেতরে পাঠায়। পথে প্রতিটা ডেস্ক (layer) চিঠিটা দেখে — যদি চিঠিটা ওর জন্য হয়, ও রেখে দেয়, আর সামনে যায় না।



```
  │ চলমান প্রোগ্রাম              │ ◄── Bash readline: Ctrl+A, Ctrl+R, Alt+. ...
  │ (bash / vim / less / tmux)│     অথবা vim/less/tmux নিজের কী
  └──────────────────────────┘
```

এখান থেকে কয়েকটা গুরুত্বপূর্ণ সিদ্ধান্ত:

1. **উপরের লেয়ার আগে ধরে।** GNOME যদি `Ctrl+Alt+T` কে "টার্মিনাল খোলো" হিসেবে রাখে, তাহলে টার্মিনালের ভেতরের প্রোগ্রাম কখনো `Ctrl+Alt+T` পাবে না।
2. **`Ctrl+C` কপি না কেন?** টার্মিনাল emulator যদি `Ctrl+C` নিজে ধরে কপি করত, তাহলে প্রোগ্রাম থামানোর উপায় থাকত না। তাই টার্মিনালগুলো কপি/পেস্টে `Shift` যোগ করে।
3. **Ctrl আসলে একটা গাণিতিক কাজ।** টার্মিনালে `Ctrl+অক্ষর` চাপলে অক্ষরটার ASCII কোডের নিচের ৫টা বিট রেখে বাকি মুছে দেওয়া হয় (`কোড AND 0x1F`)। ফলে:

তুমি চাপো	পাঠানো বাইট	ASCII নাম	তাই এর প্রভাব
`Ctrl+@` / `Ctrl+Space`	`0x00`	NUL	—
`Ctrl+C`	`0x03`	ETX	line discipline → SIGINT
`Ctrl+D`	`0x04`	EOT	লাইন খালি থাকলে EOF
`Ctrl+G`	`0x07`	BEL	বেল/বাতিল (readline-এ abort)
`Ctrl+H`	`0x08`	BS	Backspace এর মতো
`Ctrl+I`	`0x09`	TAB	**Tab কী-এর সমান**
`Ctrl+J`	`0x0A`	LF	**Enter এর মতো** (newline)
`Ctrl+M`	`0x0D`	CR	**Enter এর সমান**
`Ctrl+[`	`0x1B`	ESC	**Esc কী-এর সমান** (vim-এ খুব কাজের!)
`Ctrl+\`	`0x1C`	FS	SIGQUIT (core dump সহ থামানো)
`Ctrl+?`	`0x7F`	DEL	অনেক টার্মিনালে Backspace এটাই পাঠায়

4. **Alt = Esc + কী।** বেশিরভাগ টার্মিনাল `Alt+F` চাপলে আসলে দুটো বাইট পাঠায়: `ESC` তারপর `f`। এজন্যই readline-এ `Alt+F` এর বদলে `Esc` চেপে ছেড়ে `F` চাপলেও একই কাজ হয় — macOS-এর টার্মিনাল বা যেখানে Alt কাজ করে না, সেখানে এই কৌশল বাঁচিয়ে দেয়।

তুমি নিজে দেখতে পারো কোন কী কী বাইট পাঠায়:

```
$ cat -v
^[f          ← Alt+F চাপলে (^[ মানে ESC)
^[[A         ← উপরের অ্যারো কী
^C           ← Ctrl+C (আর cat বন্ধ হয়ে যাবে)
```

TTY লেয়ার কোন কন্ট্রোল ক্যারেক্টার কোন কাজে লাগায়, সেটা দেখো:

```
$ stty -a | grep -o 'intr = [^;]*;\|eof = [^;]*;\|susp = [^;]*;\|quit = [^;]*;'
intr = ^C;
```

```
quit = ^\;
eof = ^D;
susp = ^Z;
```

> 💡 **টিপস:** `showkey -a` (kbd প্যাকেজ) চালালেও প্রতিটা কী-প্রেসের বাইট দেখায়। বের হতে ১০ সেকেন্ড কিছু না চাপলেই হয়, বা `Ctrl+D`।

বিস্তারিত: টার্মিনাল ও শেলের সম্পর্ক অধ্যায় ৫-এ, সিগন্যাল অধ্যায় ২৪-এ।

---

১. GNOME ডেস্কটপ শর্টকাট (Ubuntu 24.04 / Fedora Workstation)

GNOME-এর পরিচিতি অধ্যায় ৪-এ। এখানে শুধু কী। `Super` = উইন্ডোজ লোগো কী।

১.১ সাধারণ ও অ্যাপ

শর্টকাট	কাজ
`Super`	Activities overview (সব উইন্ডো + সার্চ) খোলা/বন্ধ
`Super+A`	অ্যাপ্লিকেশন গ্রিড (সব অ্যাপের তালিকা)
`Super+1` ... `Super+9`	ড্যাশ/ডকের ১ম...৯ম অ্যাপ চালু বা সেটায় যাওয়া
`Alt+F2`	"Run a command" ডায়ালগ (একটা কমান্ড লিখে চালাও; `r` লিখলে X11-এ GNOME Shell রিস্টার্ট)
`Ctrl+Alt+T`	নতুন টার্মিনাল (Ubuntu-তে ডিফল্ট; Fedora-তে নিজে সেট করতে হয়)
`Super+V` বা `Super+M`	নোটিফিকেশন লিস্ট ও ক্যালেন্ডার
`Super+S`	Quick Settings মেনু (Wi-Fi, সাউন্ড, পাওয়ার)
`Super+L`	স্ক্রিন লক
`Ctrl+Alt+Delete`	লগআউট ডায়ালগ
`Super+Space`	কীবোর্ড লেআউট/ইনপুট সোর্স বদলানো (যেমন English ↔ বাংলা)

১.২ উইন্ডো ম্যানেজমেন্ট

শর্টকাট	কাজ
`Alt+Tab` / `Super+Tab`	অ্যাপ বদলানো (Shift সহ উল্টো দিকে)
`Alt+`` ` (Tab-এর উপরের কী)	একই অ্যাপের উইন্ডোগুলোর মধ্যে বদলানো
`Alt+Esc`	সরাসরি পরের উইন্ডোতে (তালিকা না দেখিয়ে)

শর্টকাট	কাজ
`Alt+F4`	উইন্ডো বন্ধ
`Super+Up`	ম্যাক্সিমাইজ
`Super+Down`	রিস্টোর / আন-ম্যাক্সিমাইজ
`Super+Left` / `Super+Right`	উইন্ডো স্ক্রিনের বাম/ডান অর্ধেকে টাইল
`Super+H`	উইন্ডো লুকানো (minimize)
`Super+D`	সব উইন্ডো লুকিয়ে ডেস্কটপ দেখানো (Ubuntu-তে ডিফল্ট; ভ্যানিলা GNOME-এ Settings থেকে সেট করতে হয়)
`Alt+Space`	উইন্ডো মেনু
`Alt+F7`	কীবোর্ড দিয়ে উইন্ডো সরানো
`Alt+F8`	কীবোর্ড দিয়ে উইন্ডো রিসাইজ
`Alt+F10`	ম্যাক্সিমাইজ টগল
`Super+Shift+Left/Right`	উইন্ডো অন্য মনিটরে পাঠানো

১.৩ ওয়ার্কস্পেস (ভার্চুয়াল ডেস্কটপ)

শর্টকাট	কাজ
`Super+Page Up` / `Super+Page Down`	আগের/পরের ওয়ার্কস্পেস
`Ctrl+Alt+Left` / `Ctrl+Alt+Right`	আগের/পরের ওয়ার্কস্পেস (GNOME-এর অনুভূমিক লেআউটে)
`Super+Shift+Page Up/Down`	বর্তমান উইন্ডো নিয়ে আগের/পরের ওয়ার্কস্পেসে
`Super+Home` / `Super+End`	প্রথম / শেষ ওয়ার্কস্পেস

১.৪ স্ক্রিনশট ও স্ক্রিন রেকর্ড (GNOME 42 এবং পরের)

শর্টকাট	কাজ
`Print`	স্ক্রিনশট টুল (এলাকা/স্ক্রিন/উইন্ডো বেছে নাও, ভিডিও মোডও আছে)
`Shift+Print`	সরাসরি পুরো স্ক্রিনের স্ক্রিনশট
`Alt+Print`	সরাসরি বর্তমান উইন্ডোর স্ক্রিনশট
`Ctrl+Alt+Shift+R`	স্ক্রিন রেকর্ডিং শুরু/বন্ধ

> 💡 **টিপস:** স্ক্রিনশট `~/Pictures/Screenshots/` এ জমা হয় এবং ক্লিপবোর্ডেও কপি হয়। নিজের শর্টকাট বানাতে বা দেখতে: **Settings → Keyboard → View and Customize Shortcuts**। কমান্ড লাইন থেকে দেখতে: `gsettings list-recursively org.gnome.desktop.wm.keybindings`।

১.৫ Files (Nautilus) ফাইল ম্যানেজার

শর্টকাট	কাজ
`Ctrl+H`	লুকানো ফাইল (`.` দিয়ে শুরু) দেখানো/লুকানো
`Ctrl+L`	লোকেশন বারে পাথ টাইপ করা (যেমন `/etc`)
`Ctrl+Shift+N`	নতুন ফোল্ডার
`F2`	রিনেম
`Delete`	ট্র্যাশে পাঠানো
`Shift+Delete`	স্থায়ীভাবে মুছে ফেলা (ট্র্যাশ ছাড়া) ⚠️
`Ctrl+T` / `Ctrl+W`	নতুন ট্যাব / ট্যাব বন্ধ
`Alt+Up` / `Alt+Left` / `Alt+Home`	প্যারেন্ট ফোল্ডার / পেছনে / হোম
`Ctrl+1` / `Ctrl+2`	লিস্ট ভিউ / গ্রিড ভিউ
`Ctrl+F`	সার্চ
`Ctrl+I` বা `Alt+Enter`	প্রপার্টিজ

---

২. KDE Plasma শর্টকাট (Plasma 6 — Kubuntu, Fedora KDE, openSUSE)

KDE-তে উইন্ডোজ কী-কে বলে **Meta**। সব শর্টকাট বদলানো যায়: **System Settings → Keyboard → Shortcuts**।

শর্টকাট	কাজ
`Meta`	অ্যাপ্লিকেশন লঞ্চার (স্টার্ট মেনুর মতো)
`Alt+Space` বা `Alt+F2`	KRunner — অ্যাপ, ফাইল, হিসাব, কমান্ড সব খোঁজার বার
`Meta+W`	Overview (সব উইন্ডো ও ডেস্কটপ)
`Meta+G`	Grid view (সব ভার্চুয়াল ডেস্কটপ একসাথে)
`Meta+D`	ডেস্কটপ উঁকি দেওয়া (সব উইন্ডো সরিয়ে)
`Alt+Tab` / `Alt+Shift+Tab`	উইন্ডো বদলানো
`Meta+Left/Right/Up/Down`	Quick tile — অর্ধেক বা কোণায় বসানো
`Meta+Page Up` / `Meta+Page Down`	ম্যাক্সিমাইজ / মিনিমাইজ
`Alt+F3`	উইন্ডো মেনু
`Alt+F4`	উইন্ডো বন্ধ

শর্টকাট	কাজ
`Ctrl+Alt+Esc`	"Kill window" — কার্সর মাথার খুলি হবে, জমে থাকা উইন্ডোতে ক্লিক করলে সেটা জোর করে বন্ধ
`Ctrl+F1` … `Ctrl+F4`	১ম…৪র্থ ভার্চুয়াল ডেস্কটপে যাওয়া
`Meta+L`	স্ক্রিন লক
`Ctrl+Alt+Delete`	লগআউট/শাটডাউন মেনু
`Meta+E`	Dolphin ফাইল ম্যানেজার
`Meta+V`	ক্লিপবোর্ড হিস্ট্রি (Klipper)
`Print`	Spectacle স্ক্রিনশট টুল
`Meta+Shift+S`	আয়তাকার এলাকার স্ক্রিনশট
`Meta+Print`	সক্রিয় উইন্ডোর স্ক্রিনশট
`Ctrl+Alt+T`	Konsole (অনেক distro-তে যেমন Kubuntu-তে সেট করা থাকে)

Dolphin ফাইল ম্যানেজার: `F4` = বিল্ট-ইন টার্মিনাল প্যানেল (বর্তমান ফোল্ডারেই খোলে — খুব কাজের), `Alt+.` বা `Ctrl+H` = লুকানো ফাইল, `F2` = রিনেম, `F3` = স্প্লিট ভিউ, `Ctrl+L` = পাথ এডিট, `F7` বা `Ctrl+Shift+N` = নতুন ফোল্ডার।

---

৩. টার্মিনাল emulator শর্টকাট

এগুলো **টার্মিনাল প্রোগ্রাম** ধরে, শেল না। GNOME Terminal (Ubuntu 24.04-এ ডিফল্ট; Fedora 41+-এ Ptyxis ডিফল্ট, যার শর্টকাট প্রায় একই) এবং Konsole।

কাজ	GNOME Terminal / Ptyxis	Konsole
কপি	`Ctrl+Shift+C`	`Ctrl+Shift+C`
পেস্ট	`Ctrl+Shift+V`	`Ctrl+Shift+V`
নতুন ট্যাব	`Ctrl+Shift+T`	`Ctrl+Shift+T`
নতুন উইন্ডো	`Ctrl+Shift+N`	`Ctrl+Shift+N`
ট্যাব বন্ধ	`Ctrl+Shift+W`	`Ctrl+Shift+W`
উইন্ডো বন্ধ	`Ctrl+Shift+Q`	`Ctrl+Shift+Q`
পরের/আগের ট্যাব	`Ctrl+Page Down` / `Ctrl+Page Up`	`Shift+Right` / `Shift+Left`
ট্যাব সরানো	`Ctrl+Shift+Page Down/Up`	`Ctrl+Shift+Right/Left`
n-তম ট্যাবে যাওয়া	`Alt+1` … `Alt+9`	—
টেক্সট খোঁজা	`Ctrl+Shift+F`	`Ctrl+Shift+F`
জুম বাড়ানো/কমানো	`Ctrl++` / `Ctrl+-`	`Ctrl++` / `Ctrl+-`

কাজ	GNOME Terminal / Ptyxis	Konsole
জুম রিসেট	`Ctrl+0`	`Ctrl+0`
ফুলস্ক্রিন	`F11`	`Ctrl+Shift+F11`
স্ক্রোল এক পেজ	`Shift+Page Up/Down`	`Shift+Page Up/Down`
স্প্লিট ভিউ	— (নেই; tmux ব্যবহার করো)	`Ctrl+(` বাম/ডান, `Ctrl+)` উপর/নিচ

মাউস দিয়ে (X11 ঐতিহ্য, Wayland-এ-ও বেশিরভাগ জায়গায় চলে):

কাজ	কী
টেক্সট সিলেক্ট করলেই "primary selection"-এ কপি	শুধু মাউস দিয়ে সিলেক্ট
primary selection পেস্ট	মাউসের **মাঝের বাটন** (চাকা) চাপো
একটা শব্দ সিলেক্ট / পুরো লাইন সিলেক্ট	ডাবল-ক্লিক / ট্রিপল-ক্লিক
লিংক খোলা	`Ctrl+ক্লিক`

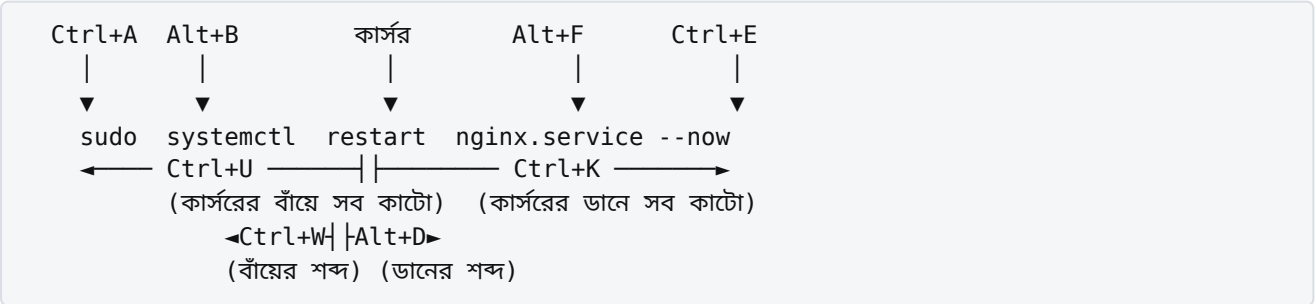
> ⚠️ **সতর্কতা:** ইন্টারনেট থেকে কমান্ড কপি করে টার্মিনালে পেস্ট করার আগে দেখে নাও — ওয়েবপেজে লুকানো টেক্সট থাকতে পারে, পেস্ট করা লেখার শেষে newline থাকলে Enter চাপার আগেই কমান্ড চলে যেতে পারে। আধুনিক Bash (5.1+) এ "bracketed paste" ডিফল্ট চালু থাকে, তাই পেস্ট করা লেখা আগে লাইনে বসে, তুমি Enter না চাপা পর্যন্ত চলে না।

---

৪. Bash readline — সম্পূর্ণ তালিকা

Bash নিজে কী-বোর্ড এডিটিং করে না, করে **GNU Readline** লাইব্রেরি। এটা ডিফল্টভাবে **Emacs মোডে** থাকে (নিচের সব শর্টকাট সেই মোডের)। একই লাইব্রেরি `python3` REPL, `mysql`, `psql`, `gdb`, `bc` ইত্যাদিতেও ব্যবহৃত হয় — তাই এখানে যা শিখবে, সেখানেও কাজ করবে। বিস্তারিত ব্যাখ্যা ও অনুশীলন অধ্যায় ৬-এ।

একটা কল্পনা করো: কমান্ড লাইন হলো একটা ছোট্ট টেক্সট এডিটর, যেটার এক লাইন মাত্র।



৪.১ কার্সর চলাফেরা

শর্টকাট	কাজ	মনে রাখার কৌশল
`Ctrl+A`	লাইনের শুরুতে	**A** = বর্ণমালার শুরু
`Ctrl+E`	লাইনের শেষে	**E** = End
`Ctrl+F`	এক অক্ষর সামনে (→)	**F**orward
`Ctrl+B`	এক অক্ষর পেছনে (←)	**B**ack
`Alt+F`	এক শব্দ সামনে	
`Alt+B`	এক শব্দ পেছনে	
`Ctrl+X Ctrl+X`	লাইনের শুরু আর বর্তমান অবস্থানের মধ্যে লাফালাফি (mark ↔ point)	
`Ctrl+]` তারপর একটা অক্ষর	সামনে ওই অক্ষরের পরের উপস্থিতিতে লাফ	
`Alt+Ctrl+]` তারপর অক্ষর	পেছনে ওই অক্ষরে লাফ	
`Home` / `End`	শুরু / শেষ (বেশিরভাগ টার্মিনালে)	
`Ctrl+Left` / `Ctrl+Right`	শব্দ ধরে চলা (বেশিরভাগ টার্মিনালে)	

৪.২ মুছে ফেলা, কাটা (kill) ও পেস্ট (yank)

Readline-এ "cut" কে বলে **kill** আর "paste" কে **yank**। কাটা লেখা যায় একটা **kill ring**-এ — একটা ঘূর্ণায়মান ক্লিপবোর্ড, যেখানে একাধিক কাটা অংশ জমা থাকে। এটা সিস্টেম ক্লিপবোর্ড **না** — শুধু ওই শেলের ভেতরে।

শর্টকাট	কাজ
`Ctrl+D`	কার্সরের নিচের অক্ষর মোছা (লাইন **খালি** থাকলে EOF → শেল থেকে বের হয়ে যায়!)
`Ctrl+H` / `Backspace`	কার্সরের বাঁয়ের অক্ষর মোছা
`Ctrl+K`	কার্সর থেকে লাইনের শেষ পর্যন্ত কাটা
`Ctrl+U`	কার্সর থেকে লাইনের শুরু পর্যন্ত কাটা (পাসওয়ার্ড ভুল টাইপ করলে সব মুছতে দারুণ)
`Ctrl+W`	কার্সরের বাঁয়ের শব্দ কাটা (স্পেস পর্যন্ত — `/usr/local/bin` পুরোটা একটা শব্দ)
`Alt+Backspace`	বাঁয়ের শব্দ কাটা, কিন্তু `/`, `.` ইত্যাদিতে থামে (`/usr/local/bin` থেকে শুধু `bin`)
`Alt+D`	কার্সর থেকে ডানের শব্দের শেষ পর্যন্ত কাটা
`Alt+\`	কার্সরের চারপাশের সব স্পেস/ট্যাব মুছে ফেলা
`Ctrl+Y`	সবশেষে কাটা লেখা পেস্ট (yank)
`Alt+Y`	`Ctrl+Y` এর **ঠিক পরে** চাপলে kill ring ঘুরিয়ে আগের কাটা অংশ আনে

৪.৩ লেখা পরিবর্তন

শর্টকাট	কাজ
`Ctrl+T`	কার্সরের আগের দুটো অক্ষর অদলবদল (`sl` → `ls` টাইপো ঠিক করা)
`Alt+T`	দুটো শব্দ অদলবদল
`Alt+U`	কার্সর থেকে শব্দের শেষ পর্যন্ত বড় হাতের (UPPERCASE)
`Alt+L`	শব্দ ছোট হাতের (lowercase)
`Alt+C`	শব্দের প্রথম অক্ষর বড় হাতের (Capitalize)
`Ctrl+_` বা `Ctrl+X Ctrl+U` বা `Ctrl+/` (অনেক টার্মিনালে)	Undo — শেষ পরিবর্তন বাতিল (বারবার চাপা যায়)
`Alt+R`	পুরো লাইনের সব পরিবর্তন বাতিল (হিস্ট্রি থেকে আনা লাইন আসল অবস্থায়)
`Ctrl+V` তারপর কী	পরের কী-কে আক্ষরিকভাবে বসানো (যেমন `Ctrl+V Tab` = আসল Tab ক্যারেক্টার)
`Alt+#`	লাইনের শুরুতে `#` বসিয়ে Enter — কমান্ডটা না চালিয়ে হিস্ট্রিতে সেভ
`Ctrl+X Ctrl+E`	বর্তমান লাইন `$EDITOR` (nano/vim) এ খোলা — বড় কমান্ড আরাম করে এডিট, সেভ করে বের হলে চলবে

৪.৪ হিস্ট্রি

শর্টকাট	কাজ
`Ctrl+P` / `Up`	আগের কমান্ড
`Ctrl+N` / `Down`	পরের কমান্ড
`Alt+<`	হিস্ট্রির একদম প্রথম কমান্ড
`Alt+>`	হিস্ট্রির শেষে (বর্তমান লাইনে)
`Ctrl+R`	পেছনের দিকে incremental search — টাইপ করতে থাকো, মিল দেখাবে; আবার `Ctrl+R` = আরও আগের মিল
`Ctrl+S`	সামনের দিকে incremental search (আগে `stty -ixon` লাগবে, না হলে টার্মিনাল ফ্রিজ!)
`Ctrl+G`	সার্চ বাতিল করে আগের লাইনে ফেরা
`Enter` (সার্চের মধ্যে)	পাওয়া কমান্ড সরাসরি চালানো
`Esc` / `Ctrl+J` / অ্যারো (সার্চের মধ্যে)	পাওয়া কমান্ড লাইনে রেখে এডিট করা
`Alt+P` / `Alt+N`	non-incremental search (লিখে Enter চাপো)
`Ctrl+O`	বর্তমান কমান্ড চালাও এবং হিস্ট্রির পরের কমান্ডটা লাইনে আনো — পরপর কয়েকটা পুরোনো কমান্ড আবার চালানোর জন্য
`Alt+.` বা `Alt+_`	আগের কমান্ডের **শেষ argument** বসানো; বারবার চাপলে আরও আগের কমান্ডের

শর্টকাট	কাজ
`Alt+Ctrl+Y`	আগের কমান্ডের **প্রথম argument** বসানো
`Alt+^`	লাইনে থাকা `!!`, `!$` ইত্যাদি history expansion তখনই খুলে দেখানো
`Alt+Ctrl+E`	পুরো লাইন expand করা (alias, variable, history সব)

৪.৫ কমপ্লিশন (Tab)

শর্টকাট	কাজ
`Tab`	কমান্ড/ফাইল/অপশন পূর্ণ করা
`Tab Tab`	একাধিক মিল থাকলে সব সম্ভাবনা দেখানো
`Alt+?`	সম্ভাব্য সব কমপ্লিশন দেখানো (`Tab Tab` এর মতো)
`Alt+*`	সব সম্ভাব্য কমপ্লিশন একসাথে লাইনে বসিয়ে দেওয়া
`Alt+/`	ফাইলনেম হিসেবে কমপ্লিট করা
`Alt+~`	ইউজারনেম কমপ্লিট (`~ro` → `~root`)
`Alt+$`	ভ্যারিয়েবল কমপ্লিট (`$HO` → `$HOME`)
`Alt+@`	হোস্টনেম কমপ্লিট (`/etc/hosts` থেকে)
`Alt+!`	কমান্ডের নাম কমপ্লিট
`Ctrl+X /`, `Ctrl+X ~`, `Ctrl+X $`, `Ctrl+X @`, `Ctrl+X !`	উপরের পাঁচটার শুধু তালিকা দেখানো (বসানো ছাড়া)

> 💡 **টিপস:** `apt install bash-completion` (Debian/Ubuntu-তে সাধারণত আগে থেকেই থাকে; Fedora/RHEL-এ `dnf install bash-completion`) করলে `Tab` শুধু ফাইল না — `git che<Tab>`, `systemctl res<Tab>`, `ssh <Tab>` (হোস্ট) পর্যন্ত বুদ্ধিমানের মতো পূর্ণ করে।

৪.৬ স্ক্রিন, সিগন্যাল ও টার্মিনাল কন্ট্রোল

এর মধ্যে কয়েকটা readline-এর না, **TTY line discipline**-এর (উপরের ডায়াগ্রাম দেখো) — তাই bash ছাড়াও যেকোনো প্রোগ্রামে কাজ করে।

শর্টকাট	কাজ	কে ধরে
`Ctrl+L`	স্ক্রিন পরিষ্কার (বর্তমান লাইন রেখে) — `clear` এর মতো	readline
`Ctrl+C`	চলমান প্রোগ্রামকে SIGINT পাঠানো (থামাও); prompt-এ থাকলে লাইন বাতিল	TTY
`Ctrl+Z`	প্রোগ্রামকে SIGTSTP — সাময়িক থামিয়ে ব্যাকগ্রাউন্ডে (ফেরাতে `fg`)	TTY
`Ctrl+\`	SIGQUIT — `Ctrl+C` না মানলে আরও কড়া থামানো (core dump হতে পারে)	TTY
`Ctrl+D`	ইনপুট শেষ (EOF); খালি prompt-এ = `exit`	TTY

শর্টকাট	কাজ	কে ধরে
`Ctrl+S`	টার্মিনাল আউটপুট থামানো (XOFF) — "টার্মিনাল জমে গেছে!" এর সবচেয়ে কমন কারণ	TTY
`Ctrl+Q`	আউটপুট আবার চালু (XON)	TTY
`Ctrl+J` / `Ctrl+M`	Enter	উভয়
`Ctrl+X Ctrl+R`	`~/.inputrc` আবার পড়া	readline
`Ctrl+X Ctrl+V`	Bash ভার্সন দেখানো	readline
`Alt+0` … `Alt+9` তারপর কমান্ড	numeric argument — যেমন `Alt+3 Ctrl+D` = ৩টা অক্ষর মোছা; `Alt+-` = উল্টো দিক	readline

> ⚠️ **সতর্কতা:** টার্মিনাল হঠাৎ কিছু টাইপ নিচ্ছে না? প্রথমে `Ctrl+Q` চাপো (সম্ভবত ভুলে `Ctrl+S` চেপেছিলে)। স্ক্রিনে আজব অক্ষর (বাইনারি ফাইল `cat` করার পর)? অন্ধভাবে `reset` লিখে Enter চাপো।

৪.৭ Vi মোড ও কাস্টমাইজেশন

Vim ভক্ত হলে `set -o vi` লেখো (স্থায়ী করতে `~/.bashrc`-এ)। তখন লাইন Insert মোডে শুরু হয়, `Esc` চাপলে Normal মোড, আর `h l w b 0 $ dd cw /pattern` সব vim-এর মতো। `set -o emacs` দিয়ে ফেরত। নিজের readline বাইন্ডিং `~/.inputrc` এ রাখা যায়:

```
# ~/.inputrc
"\e[A": history-search-backward     # Up: যা লিখেছ তা দিয়ে শুরু হওয়া আগের কমান্ড
"\e[B": history-search-forward
set completion-ignore-case on        # Tab কমপ্লিশনে বড়-ছোট হাতের পার্থক্য নেই
set show-all-if-ambiguous on         # একবার Tab-এই সব সম্ভাবনা দেখাও
```

সব বর্তমান বাইন্ডিং দেখতে: `bind -P` (ফাংশনসহ) বা `bind -p`। বিস্তারিত অধ্যায় ৪৪।

---

৫. History expansion (`!` দিয়ে জাদু)

এটা কীবোর্ড শর্টকাট না, কিন্তু টাইপিং বাঁচানোর সবচেয়ে বড় অস্ত্র। Bash লাইন চালানোর **আগে** `!` দিয়ে শুরু হওয়া অংশগুলো হিস্ট্রি থেকে বদলে দেয়।

৫.১ Event designator — কোন কমান্ড?

এক্সপ্রেশন	মানে	উদাহরণ
`!!`	ঠিক আগের পুরো কমান্ড	`sudo !!` — আগের কমান্ড sudo দিয়ে

এক্সপ্রেশন	মানে	উদাহরণ
`!n`	হিস্ট্রির n নম্বর কমান্ড	`!42`
`!-n`	n টা আগের কমান্ড	`!-2` = দুটো আগের
`!string`	`string` দিয়ে শুরু হওয়া সর্বশেষ কমান্ড	`!ssh`
`!?string?`	`string` আছে এমন সর্বশেষ কমান্ড	`!?nginx?`
`^old^new`	আগের কমান্ডে প্রথম `old` কে `new` দিয়ে বদলে চালাও	`^stop^start`
`!#`	এখন পর্যন্ত টাইপ করা বর্তমান লাইন	`cp file.txt !#:1.bak` → `cp file.txt file.txt.bak`

৫.২ Word designator — কোন শব্দ? (`:` দিয়ে)

শব্দ গোনা শুরু ০ থেকে: `0` = কমান্ডের নাম, `1` = প্রথম argument।

এক্সপ্রেশন	মানে
`!$`	আগের কমান্ডের শেষ argument (`!!:$` এর সংক্ষেপ)
`!^`	আগের কমান্ডের প্রথম argument (`!!:1`)
`!*`	আগের কমান্ডের সব argument (`!!:1-$`)
`!:0`	আগের কমান্ডের নাম
`!:n`	আগের কমান্ডের n-তম শব্দ
`!:n-m`	n থেকে m পর্যন্ত শব্দ
`!:n*` বা `!:n-$`	n থেকে শেষ পর্যন্ত
`!vim:$`	`vim` দিয়ে শুরু হওয়া শেষ কমান্ডের শেষ argument

৫.৩ Modifier — শব্দটাকে বদলানো (আবার `:` দিয়ে)

Modifier	কাজ	`!$` = `/var/log/nginx/access.log` হলে
`:h`	head — শেষ অংশ বাদ (ডিরেক্টরি)	`/var/log/nginx`
`:t`	tail — শুধু শেষ অংশ (ফাইলনেম)	`access.log`
`:r`	extension বাদ	`/var/log/nginx/access`
`:e`	শুধু extension	`.log`
`:p`	**চালাবে না**, শুধু দেখাবে (নিরাপদ পরীক্ষা)	—
`:s/old/new/`	প্রথম মিল বদলানো	—

Modifier	কাজ	`!$` = `/var/log/nginx/access.log` হলে
`:gs/old/new/` বা `:as/old/new/`	সব মিল বদলানো	—
`:q`	quote করা	—
`:&`	আগের substitution আবার	—

```
$ ls /etc/ssh/sshd_config
/etc/ssh/sshd_config
$ sudo vim !$
sudo vim /etc/ssh/sshd_config
$ cd !$:h
cd /etc/ssh
$ systemctl status nginx
$ ^status^restart
systemctl restart nginx
$ !!:gs/nginx/apache2/:p
systemctl restart apache2        ← শুধু দেখাল, চালাল না
```

সম্পর্কিত	কাজ
`$_`	শেলের ভ্যারিয়েবল: আগের কমান্ডের শেষ argument (স্ক্রিপ্টেও চলে) — `mkdir newdir && cd $_`
`history` / `history 20`	হিস্ট্রি তালিকা
`history -c`	বর্তমান সেশনের হিস্ট্রি মোছা
`fc`	আগের কমান্ড এডিটরে খুলে এডিট করে চালানো
`fc -l -10`	শেষ ১০টা কমান্ড দেখানো
`HISTCONTROL=ignoreboth`	স্পেস দিয়ে শুরু কমান্ড আর ডুপ্লিকেট হিস্ট্রিতে না রাখা
`shopt -s histverify`	`!` expansion চালানোর আগে লাইনে দেখিয়ে থামবে — নবীনদের জন্য নিরাপদ

> 💡 **টিপস:** পাসওয়ার্ড বা টোকেন আছে এমন কমান্ডের আগে একটা **স্পেস** দাও (যদি `HISTCONTROL`-এ `ignorespace` বা `ignoreboth` থাকে — Ubuntu-র ডিফল্ট `.bashrc`-এ আছে) — হিস্ট্রিতে জমা হবে না।

---

৬. `less` ও `man` পেজার

`man` পেজ আসলে `less` দিয়েই দেখানো হয়, তাই দুটোর কী একই। বিস্তারিত অধ্যায় ১০।

কী	কাজ
`Space` / `f` / `Page Down` / `Ctrl+F`	পরের পেজ
`b` / `Page Up` / `Ctrl+B`	আগের পেজ
`d` / `u`	অর্ধেক পেজ নিচে / উপরে
`j` / `Enter` / `Down` / `e`	এক লাইন নিচে
`k` / `Up` / `y`	এক লাইন উপরে
`g` / `<` / `Home`	ফাইলের শুরুতে
`G` / `>` / `End`	ফাইলের শেষে
`50g`	৫০ নম্বর লাইনে
`50p` বা `50%`	ফাইলের ৫০% জায়গায়
`/pattern`	সামনের দিকে খোঁজা (regex চলে)
`?pattern`	পেছনের দিকে খোঁজা
`n` / `N`	পরের / আগের মিল
`&pattern`	শুধু মিল থাকা লাইন দেখাও (ফিল্টার); `&` তারপর Enter = ফিল্টার বাদ
`Esc u`	সার্চ হাইলাইট টগল
`-i` (less চলার সময়)	সার্চে বড়-ছোট হাতের পার্থক্য উপেক্ষা টগল
`-N`	লাইন নম্বর টগল
`-S`	লম্বা লাইন কাটা/মোড়ানো টগল (লগ দেখতে কাজের)
`ma`	বর্তমান অবস্থানে `a` নামে mark
`'a`	mark `a` তে ফেরা
`F`	follow mode — `tail -f` এর মতো; বের হতে `Ctrl+C`
`=` বা `Ctrl+G`	ফাইলের নাম, লাইন, শতাংশ দেখানো
`v`	ফাইলটা `$EDITOR` এ খোলা
`:n` / `:p`	একাধিক ফাইলে পরের / আগের ফাইল
`R`	স্ক্রিন রিফ্রেশ (ফাইল নতুন করে পড়া)
`!command`	শেল কমান্ড চালানো
`h`	সাহায্য (সব কী)
`q`	বের হওয়া

`man` -এর নিজের কমান্ড-লাইন কৌশল:

কমান্ড	কাজ
`man ls`	ls-এর ম্যানুয়াল

কমান্ড	কাজ
`man 5 passwd`	সেকশন ৫ (ফাইল ফরম্যাট) এর passwd — `man passwd` দিলে সেকশন ১ এর কমান্ড আসে
`man -k keyword` বা `apropos keyword`	কীওয়ার্ড দিয়ে ম্যানুয়াল খোঁজা
`man -f ls` বা `whatis ls`	এক লাইনের বিবরণ
`man man`	সেকশন নম্বরের তালিকা (1=কমান্ড, 5=ফাইল, 8=অ্যাডমিন...)

> 💡 **টিপস:** `journalctl`, `git log`, `systemctl status` — এরাও আউটপুট `less` দিয়ে দেখায়। তাই এই টেবিল শিখলে সব জায়গায় কাজে লাগবে।

---

৭. nano এডিটর

nano-র নিচে দুই লাইনে শর্টকাট লেখা থাকে: `^` = Ctrl, `M-` = Alt। বিস্তারিত অধ্যায় ১৩। (নিচের টেবিল nano 7.x — Ubuntu 24.04, Debian 12, Fedora — অনুযায়ী।)

শর্টকাট	কাজ
`Ctrl+G`	সাহায্য
`Ctrl+X`	বের হওয়া (সেভ না হলে জিজ্ঞেস করবে: `Y` / `N` / `Ctrl+C`)
`Ctrl+O`	Write Out — ফাইলের নাম জিজ্ঞেস করে সেভ (Save As-এর মতো)
`Ctrl+S`	সরাসরি সেভ (কিছু জিজ্ঞেস না করে)
`Ctrl+R`	অন্য ফাইলের লেখা কার্সরের জায়গায় ঢোকানো
`Ctrl+W`	খোঁজা (Where is)
`Alt+W` / `Alt+Q`	পরের / আগের মিল
`Ctrl+\` বা `Alt+R`	খুঁজে বদলানো (Replace)
`Ctrl+K`	বর্তমান লাইন (বা সিলেক্ট করা অংশ) কাটা
`Ctrl+U`	কাটা লেখা পেস্ট
`Alt+6`	লাইন (বা সিলেকশন) কপি
`Alt+A` বা `Ctrl+6`	মার্ক সেট — সিলেকশন শুরু; তারপর অ্যারো দিয়ে সিলেক্ট
`Alt+U`	Undo
`Alt+E`	Redo
`Ctrl+_` বা `Alt+G`	নির্দিষ্ট লাইন (ও কলাম) নম্বরে যাওয়া
`Ctrl+C`	কার্সরের অবস্থান (লাইন/কলাম) দেখানো

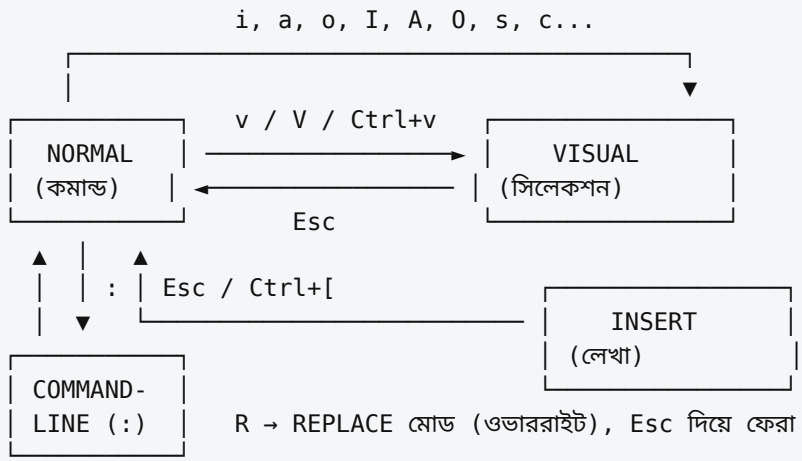
শর্টকাট	কাজ
`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষ
`Ctrl+Y` / `Ctrl+V`	এক পেজ উপরে / নিচে
`Alt+\` / `Alt+/`	ফাইলের শুরু / শেষ
`Ctrl+Left` / `Ctrl+Right`	শব্দ ধরে চলা
`Alt+}` / `Alt+{`	ইনডেন্ট বাড়ানো / কমানো
`Alt+3`	লাইন কমেন্ট/আনকমেন্ট (সিনট্যাক্স অনুযায়ী `#`, `//`)
`Alt+N`	লাইন নম্বর দেখানো টগল
`Alt+P`	হোয়াইটস্পেস দেখানো টগল
`Alt+S`	soft wrap টগল
`Alt+X`	নিচের হেল্প লাইন লুকানো/দেখানো
`Ctrl+J`	অনুচ্ছেদ justify
`Ctrl+T`	Execute — কমান্ড চালিয়ে আউটপুট ঢোকানো / স্পেল চেক

> 💡 **টিপস:** `nano -l file` (লাইন নম্বরসহ), `nano +25 file` (২৫ নম্বর লাইনে খোলা)। কনফিগ ফাইল এডিট করতে `sudoedit /etc/hosts` — নিরাপদ পদ্ধতি (অধ্যায় ১৮)।

---

৮. Vim — বড় টেবিল

Vim একটা **modal** এডিটর: একই কী বিভিন্ন মোডে বিভিন্ন কাজ করে। পিয়ানোর প্যাডেলের মতো ভাবো — প্যাডেল চাপা অবস্থায় একই কী অন্যরকম শোনায়। পুরো ব্যাখ্যা অধ্যায় ১৪-এ।



> 📝 **মনে রাখো — Vim-এর ব্যাকরণ:** `[সংখ্যা] ক্রিয়া [সংখ্যা] গতি/বস্তু`। `d` (delete) + `w` (word) = `dw`; `3dw` = তিনটা শব্দ মোছা; `c` (change) + `i"` (inside quotes) = `ci"`। ক্রিয়া আর গতি আলাদা শিখলে হাজার কম্বিনেশন নিজে থেকেই পারবে।

৮.১ বাঁচার কী (প্রথম দিনেই)

কী	কাজ
`Esc` বা `Ctrl+[`	Normal মোডে ফেরা (সন্দেহ হলে দু'বার চাপো)
`i`	Insert মোড (কার্সরের আগে)
`:w`	সেভ
`:q`	বের হওয়া
`:wq` / `:x` / `ZZ`	সেভ করে বের হওয়া (`:x` আর `ZZ` শুধু পরিবর্তন থাকলে লেখে)
`:q!` / `ZQ`	সেভ না করে বের হওয়া
`u`	Undo
`Ctrl+R`	Redo
`:help` / `:help dw`	সাহায্য (বের হতে `:q`)
`vimtutor` (শেল থেকে)	৩০ মিনিটের ইন্টারঅ্যাক্টিভ টিউটোরিয়াল

৮.২ চলাফেরা (Normal মোড)

কী	কাজ
`h` `j` `k` `l`	বাঁয়ে, নিচে, উপরে, ডানে
`w` / `W`	পরের শব্দের শুরু (`W` = শুধু স্পেস দিয়ে আলাদা "WORD")
`b` / `B`	আগের শব্দের শুরু
`e` / `E`	শব্দের শেষ
`ge`	আগের শব্দের শেষ
`0`	লাইনের একদম শুরু (কলাম ০)
`^`	লাইনের প্রথম non-space অক্ষর
`$`	লাইনের শেষ
`gg`	ফাইলের প্রথম লাইন
`G`	ফাইলের শেষ লাইন
`42G` বা `:42`	৪২ নম্বর লাইন
`H` / `M` / `L`	স্ক্রিনের উপর / মাঝ / নিচের লাইন

কী	কাজ
`Ctrl+F` / `Ctrl+B`	পুরো পেজ নিচে / উপরে
`Ctrl+D` / `Ctrl+U`	অর্ধেক পেজ নিচে / উপরে
`Ctrl+E` / `Ctrl+Y`	কার্সর না সরিয়ে স্ক্রিন এক লাইন স্ক্রোল
`zz` / `zt` / `zb`	বর্তমান লাইন স্ক্রিনের মাঝে / উপরে / নিচে আনা
`%`	মিলে যাওয়া বন্ধনী `( ) [ ] { }` তে লাফ
`{` / `}`	আগের / পরের অনুচ্ছেদ (ফাঁকা লাইন)
`fx` / `Fx`	লাইনে সামনে / পেছনে `x` অক্ষরে
`tx` / `Tx`	`x` এর ঠিক আগে / পরে পর্যন্ত
`;` / `,`	শেষ `f` / `t` আবার একই দিকে / উল্টো দিকে
`*` / `#`	কার্সরের নিচের শব্দ সামনে / পেছনে খোঁজা
`Ctrl+O` / `Ctrl+I`	jump list-এ পেছনে / সামনে (আগে যেখানে ছিলে)
`` `` (দুটো backtick)	শেষ লাফের আগের জায়গায়
`gd`	কার্সরের নিচের ভ্যারিয়েবলের local ডেফিনিশনে
`gf`	কার্সরের নিচের পাথের ফাইল খোলা
`Ctrl+G`	ফাইলের নাম ও অবস্থান দেখানো

৮.৩ Insert মোডে ঢোকার উপায়

কী	কোথায় লেখা শুরু
`i` / `I`	কার্সরের আগে / লাইনের প্রথম non-space অক্ষরের আগে
`a` / `A`	কার্সরের পরে / লাইনের শেষে
`o` / `O`	নিচে / উপরে নতুন লাইন খুলে
`s`	কার্সরের অক্ষর মুছে
`S` বা `cc`	পুরো লাইন মুছে
`C`	কার্সর থেকে লাইনের শেষ পর্যন্ত মুছে
`gi`	শেষবার যেখানে Insert মোড ছেড়েছিলে, সেখানে

Insert মোডের ভেতরে:

কী	কাজ
`Ctrl+W`	আগের শব্দ মোছা
`Ctrl+U`	লাইনের শুরু পর্যন্ত মোছা
`Ctrl+H`	Backspace

কী	কাজ
`Ctrl+N` / `Ctrl+P`	ফাইলে থাকা শব্দ থেকে অটো-কমপ্লিট (পরের / আগের)
`Ctrl+O`	একটা Normal কমান্ড চালিয়ে আবার Insert মোডে
`Ctrl+R a`	রেজিস্টার `a` এর লেখা বসানো (`Ctrl+R "` = শেষ কপি)
`Ctrl+T` / `Ctrl+D`	লাইনের ইনডেন্ট বাড়ানো / কমানো
`Ctrl+V` তারপর কী	আক্ষরিক অক্ষর (যেমন আসল Tab)

৮.৪ এডিটিং (Normal মোড)

কী	কাজ
`x` / `X`	কার্সরের / আগের অক্ষর মোছা
`dd`	লাইন কাটা (মোছা + রেজিস্টারে)
`5dd`	৫টা লাইন কাটা
`dw` / `db`	শব্দের সামনের / পেছনের অংশ কাটা
`D` বা `d$`	লাইনের শেষ পর্যন্ত কাটা
`d0`	লাইনের শুরু পর্যন্ত কাটা
`dG` / `dgg`	ফাইলের শেষ / শুরু পর্যন্ত কাটা
`yy` বা `Y`	লাইন কপি (yank)
`yw` / `y$`	শব্দ / লাইনের শেষ পর্যন্ত কপি
`p` / `P`	কার্সরের পরে / আগে পেস্ট
`r x`	কার্সরের অক্ষর `x` দিয়ে বদলানো
`R`	Replace মোড (ওভাররাইট)
`cw` / `ce`	শব্দ বদলানো (মুছে Insert মোড)
`~`	কার্সরের অক্ষরের case উল্টানো
`gUiw` / `guiw`	শব্দ UPPER / lower case
`J`	নিচের লাইন এই লাইনের সাথে জোড়া
`>>` / `<<`	লাইন ইনডেন্ট / আন-ইনডেন্ট
`=` + গতি, `gg=G`	স্বয়ংক্রিয় ইনডেন্ট (পুরো ফাইল)
`.`	শেষ পরিবর্তন আবার করা (Vim-এর সবচেয়ে শক্তিশালী কী!)
`Ctrl+A` / `Ctrl+X`	কার্সরের সংখ্যা ১ বাড়ানো / কমানো (`10 Ctrl+A` = ১০ বাড়ানো)
`xp`	দুটো অক্ষর অদলবদল
`ddp`	দুটো লাইন অদলবদল
`U`	বর্তমান লাইনের সব পরিবর্তন বাতিল

৮.৫ Text objects (ক্রিয়ার পরে বসে: `d`, `c`, `y`, `v`)

`i` = inner (ভেতরে, ঘের ছাড়া), `a` = a/around (ঘের সহ)।

বস্তু	মানে	উদাহরণ
`iw` / `aw`	শব্দ / শব্দ + পাশের স্পেস	`diw`, `daw`
`iW` / `aW`	WORD	`ciW`
`is` / `as`	বাক্য	`das`
`ip` / `ap`	অনুচ্ছেদ	`yap`
`i"` / `a"`	`"..."` এর ভেতর / সহ	`ci"` — কোটের ভেতরের লেখা বদলাও
`i'` / `i``	`'...'` / `` `...` `` এর ভেতর	`di'`
`i(` বা `ib` / `a(`	`( )` এর ভেতর / সহ	`ci(`
`i{` বা `iB` / `a{`	`{ }` এর ভেতর / সহ	`di{`
`i[` / `a[`	`[ ]`	`yi[`
`i<` / `a<`	`< >`	`ci<`
`it` / `at`	HTML/XML ট্যাগের ভেতর / সহ	`cit`

৮.৬ Visual মোড

কী	কাজ
`v`	অক্ষর ধরে সিলেকশন
`V`	লাইন ধরে সিলেকশন
`Ctrl+V`	ব্লক (কলাম) সিলেকশন
`gv`	শেষ সিলেকশন আবার
`o`	সিলেকশনের অন্য প্রান্তে কার্সর
`d` / `y` / `c` / `>` / `<` / `~` / `U` / `u`	সিলেকশনে কাটা / কপি / বদল / ইনডেন্ট / case
`Ctrl+V` → `j` (কয়েকবার) → `I#` → `Esc`	কয়েকটা লাইনের শুরুতে একসাথে `#` বসানো (মাল্টি-লাইন কমেন্ট)
`:` (সিলেকশন থাকা অবস্থায়)	`:'<,'>` — শুধু সিলেকশনে কমান্ড

৮.৭ খোঁজা ও বদলানো

কমান্ড	কাজ
`/pattern` / `?pattern`	সামনে / পেছনে খোঁজা
`n` / `N`	পরের / আগের মিল
`:noh`	সার্চ হাইলাইট সরানো

কমান্ড	কাজ
`:s/old/new/`	বর্তমান লাইনে প্রথম মিল বদলানো
`:s/old/new/g`	বর্তমান লাইনে সব মিল
`:%s/old/new/g`	পুরো ফাইলে সব মিল
`:%s/old/new/gc`	প্রতিটায় জিজ্ঞেস করে (`y` / `n` / `a` সব/ `q`)
`:%s/old/new/gi`	case-insensitive
`:10,20s/old/new/g`	১০–২০ নম্বর লাইনে
`:g/pattern/d`	pattern আছে এমন সব লাইন মোছা
`:v/pattern/d` বা `:g!/pattern/d`	pattern **নেই** এমন সব লাইন মোছা
`:set ic` / `:set noic`	সার্চে case উপেক্ষা চালু / বন্ধ
`:set hlsearch` / `:set incsearch`	মিল হাইলাইট / টাইপ করতে করতে খোঁজা

৮.৮ ফাইল, বাফার, উইন্ডো, ট্যাব

কমান্ড	কাজ
`:e file`	অন্য ফাইল খোলা
`:e!`	ডিস্ক থেকে ফাইল আবার পড়া (সব পরিবর্তন বাদ)
`:w newname`	অন্য নামে লেখা (বর্তমান ফাইল খোলাই থাকে)
`:saveas newname`	অন্য নামে সেভ করে সেটাতে সুইচ
`:r file`	অন্য ফাইলের লেখা ঢোকানো
`:r !date`	কমান্ডের আউটপুট ঢোকানো
`:!ls`	শেল কমান্ড চালানো
`:w !sudo tee %`	sudo ছাড়া খুলে ফেললে root হিসেবে সেভ করার পুরোনো কৌশল (Neovim-এ কাজ করে না; বদলে `sudoedit` ব্যবহার করো)
`:ls`	খোলা বাফারের তালিকা
`:bn` / `:bp` / `:b3` / `:bd`	পরের / আগের / ৩ নম্বর বাফার / বাফার বন্ধ
`:sp file` / `:vsp file`	অনুভূমিক / উল্লম্ব split
`Ctrl+W s` / `Ctrl+W v`	বর্তমান উইন্ডো split
`Ctrl+W w`	পরের উইন্ডো
`Ctrl+W h/j/k/l`	বাঁ / নিচ / উপর / ডানের উইন্ডো
`Ctrl+W q` / `Ctrl+W o`	এই উইন্ডো বন্ধ / বাকি সব বন্ধ
`Ctrl+W =`	সব উইন্ডো সমান সাইজ
`Ctrl+W +` / `Ctrl+W -`	উচ্চতা বাড়ানো / কমানো

কমান্ড	কাজ
`:tabnew file`	নতুন ট্যাবে খোলা
`gt` / `gT` / `3gt`	পরের / আগের / ৩ নম্বর ট্যাব
`:tabclose`	ট্যাব বন্ধ
`vim -o a b` / `vim -O a b` / `vim -p a b`	শুরু থেকেই split / vsplit / ট্যাবে

৮.৯ Mark, register, macro

কী	কাজ
`ma`	বর্তমান অবস্থানে mark `a` (ছোট হাতের = এই ফাইলে; বড় হাতের `A` = যেকোনো ফাইল থেকে)
`'a` / `` `a ``	mark `a` এর লাইনে / ঠিক অবস্থানে
`:marks`	সব mark
`"ayy`	লাইনটা রেজিস্টার `a` তে কপি
`"ap`	রেজিস্টার `a` থেকে পেস্ট
`"+y` / `"+p`	সিস্টেম ক্লিপবোর্ডে কপি / থেকে পেস্ট (`vim --version` এ `+clipboard` লাগবে; Ubuntu-তে `vim-gtk3` প্যাকেজে আছে)
`"0p`	শেষ yank (মোছা না) পেস্ট
`:reg`	সব রেজিস্টারের লেখা
`qa` … `q`	macro `a` রেকর্ড শুরু … শেষ
`@a`	macro `a` চালানো
`@@`	শেষ চালানো macro আবার
`10@a`	macro ১০ বার

৮.১০ ফোল্ডিং, সেটিং ও অন্যান্য

কমান্ড	কাজ
`zf` + গতি / `zo` / `zc` / `za`	fold বানানো / খোলা / বন্ধ / টগল
`zR` / `zM`	সব fold খোলা / বন্ধ
`:set number` / `:set nu!`	লাইন নম্বর / টগল
`:set relativenumber`	আপেক্ষিক লাইন নম্বর (`5j` এর হিসাব সহজ)
`:set paste` / `:set nopaste`	বাইরে থেকে পেস্টের সময় অটো-ইনডেন্ট বন্ধ (Vim 8+ টার্মিনালে bracketed paste থাকলে সাধারণত দরকার নেই)
`:set list`	Tab ও লাইনের শেষ দেখানো
`:syntax on`	সিনট্যাক্স হাইলাইট

কমান্ড	কাজ
`:set expandtab tabstop=4 shiftwidth=4`	Tab = ৪ স্পেস
`K`	কার্সরের শব্দের `man` পেজ
`ga`	কার্সরের অক্ষরের ASCII/Unicode মান
`g Ctrl+G`	শব্দ/অক্ষর/লাইন গণনা
`:sort` / `:sort u`	লাইন সাজানো / ডুপ্লিকেট বাদ দিয়ে সাজানো
`:%!jq .`	পুরো ফাইল বাইরের কমান্ডের ভেতর দিয়ে পাঠিয়ে ফলাফল বসানো

> 💡 **টিপস:** স্থায়ী সেটিং `~/.vimrc` তে রাখো (`:` ছাড়া), যেমন `set number`, `syntax on`। ভুলে sudo ছাড়া `/etc` এর ফাইল খুলেছ? সেভ করার আগে `:w /tmp/x` করে নিরাপদে রাখো।

---

৯. tmux

tmux একটা **terminal multiplexer** — এক টার্মিনালের ভেতরে অনেক সেশন, উইন্ডো, প্যান; আর SSH কেটে গেলেও কাজ চলতে থাকে। ব্যাখ্যা অধ্যায় ২৫-এ।

```
tmux server (ব্যাকগ্রাউন্ড প্রসেস — SSH কাটলেও বেঁচে থাকে)
 └── session "work"            ← tmux attach -t work
      ├── window 0: "editor"   ← স্ট্যাটাস বারে 0:editor
      │     ├── pane 0 (vim)
      │     └── pane 1 (bash)  ← Prefix % দিয়ে বানানো
      └── window 1: "logs"
            └── pane 0 (journalctl -f)
```

Prefix = `Ctrl+B` (চেপে ছেড়ে দাও, তারপর কী)। নিচের সব কী prefix এর পরে।

৯.১ সেশন

কী	কাজ
`d`	detach — সেশন ব্যাকগ্রাউন্ডে চলতে থাকবে
`s`	সেশনের তালিকা (tree) থেকে বেছে নেওয়া
`$`	সেশন রিনেম
`(` / `)`	আগের / পরের সেশন
`L`	শেষ ব্যবহৃত সেশনে
`D`	কোন client detach করবে বেছে নেওয়া

৯.২ উইন্ডো

কী	কাজ
`c`	নতুন উইন্ডো
`,`	উইন্ডো রিনেম
`n` / `p`	পরের / আগের উইন্ডো
`0` … `9`	ওই নম্বরের উইন্ডো
`l`	শেষ ব্যবহৃত উইন্ডো
`w`	সব উইন্ডোর তালিকা থেকে বেছে নেওয়া
`'`	উইন্ডো নম্বর লিখে যাওয়া
`f`	উইন্ডোতে লেখা খোঁজা
`&`	উইন্ডো বন্ধ (নিশ্চিত করতে `y`)
`.`	উইন্ডোর নম্বর বদলানো (move)

৯.৩ প্যান

কী	কাজ
`%`	বাম-ডানে ভাগ (দুই পাশাপাশি প্যান)
`"`	উপর-নিচে ভাগ
`Left/Right/Up/Down`	ওই দিকের প্যানে যাওয়া
`o`	পরের প্যান
`;`	শেষ সক্রিয় প্যান
`q`	প্যান নম্বর দেখানো (দেখানো অবস্থায় নম্বর চাপলে সেখানে যায়)
`z`	প্যান zoom টগল (ফুলস্ক্রিন / আগের মতো)
`x`	প্যান বন্ধ (নিশ্চিত করতে `y`)
`!`	প্যানটাকে আলাদা উইন্ডো বানানো
`{` / `}`	প্যান আগের / পরের প্যানের সাথে অদলবদল
`Space`	পরের প্রি-সেট লেআউট
`Alt+1` … `Alt+5`	নির্দিষ্ট লেআউট (even-horizontal, even-vertical, main-horizontal, main-vertical, tiled)
`Ctrl+Left/Right/Up/Down`	প্যান ১ ঘর করে রিসাইজ
`Alt+Left/Right/Up/Down`	প্যান ৫ ঘর করে রিসাইজ

৯.৪ কপি মোড ও বিবিধ

কী	কাজ
`[`	কপি মোড — স্ক্রোল করে পুরোনো আউটপুট দেখা (মাউস ছাড়া)
`Page Up`	কপি মোডে ঢুকে এক পেজ উপরে
`]`	tmux বাফারে কপি করা লেখা পেস্ট
`:`	tmux কমান্ড প্রম্পট (যেমন `:setw synchronize-panes on` — সব প্যানে একসাথে টাইপ)
`?`	সব কী-বাইন্ডিং তালিকা
`t`	বড় ঘড়ি
`~`	tmux-এর মেসেজ লগ
`r`	client রিফ্রেশ
`Ctrl+B` (prefix দু'বার)	ভেতরের প্রোগ্রামে আক্ষরিক `Ctrl+B` পাঠানো

কপি মোডের ভেতরে: tmux `mode-keys` সেটিং অনুযায়ী দুই রকম। `EDITOR` / `VISUAL` এ `vi` থাকলে ডিফল্ট vi, না হলে emacs।

কাজ	vi মোড	emacs মোড
সিলেকশন শুরু	`Space`	`Ctrl+Space`
কপি করে বের হওয়া	`Enter`	`Alt+W`
খোঁজা (পেছনে / সামনে)	`?` / `/`	`Ctrl+R` / `Ctrl+S`
বের হওয়া	`q`	`q` বা `Esc`

কমান্ড লাইন থেকে:

কমান্ড	কাজ
`tmux` / `tmux new -s work`	নতুন সেশন / নাম দিয়ে
`tmux ls`	সেশনের তালিকা
`tmux attach -t work` (বা `tmux a -t work`)	সেশনে ফেরা
`tmux kill-session -t work`	সেশন বন্ধ
`tmux kill-server`	সব সেশন বন্ধ

> 💡 **টিপস:** অনেকে prefix `Ctrl+A` তে বদলায় (`~/.tmux.conf` এ `set -g prefix C-a`, `unbind C-b`, `bind C-a send-prefix`), আর মাউস চালু করে `set -g mouse on`। কনফিগ আবার লোড: `tmux source-file ~/.tmux.conf`।

---

১০. GNU screen

screen হলো tmux-এর বড় ভাই (পুরোনো, কিন্তু অনেক সার্ভারে এখনো আছে; RHEL 8 থেকে Red Hat এটা বাদ দিয়ে tmux রেখেছে — দরকারে EPEL থেকে ইনস্টল)। **Prefix = `Ctrl+A`**।

কী (`Ctrl+A` এর পরে)	কাজ
`c`	নতুন উইন্ডো
`n` / `p` (বা `Space` / `Backspace`)	পরের / আগের উইন্ডো
`Ctrl+A`	শেষ ব্যবহৃত উইন্ডোতে টগল
`0` … `9`	ওই নম্বরের উইন্ডো
`"`	উইন্ডো তালিকা থেকে বেছে নেওয়া
`w`	উইন্ডো তালিকা নিচে এক লাইনে
`A`	উইন্ডোর নাম (title) দেওয়া
`k`	উইন্ডো বন্ধ (kill; নিশ্চিত করতে `y`)
`d`	detach
`S`	উপর-নিচে split (region)
`|`	বাম-ডানে split (screen 4.1+)
`Tab`	পরের region-এ
`X`	বর্তমান region বন্ধ
`Q`	বর্তমানটা ছাড়া সব region বন্ধ
`[` বা `Esc`	কপি/স্ক্রোলব্যাক মোড (`Space` দিয়ে শুরু ও শেষ চিহ্ন, `Esc` দিয়ে বের)
`]`	পেস্ট
`H`	উইন্ডোর আউটপুট ফাইলে লগ করা টগল (`screenlog.N`)
`M`	এই উইন্ডোতে activity হলে জানানো
`_`	৩০ সেকেন্ড চুপ থাকলে জানানো (লম্বা কাজ শেষ হলো কিনা)
`x`	স্ক্রিন লক
`a`	আক্ষরিক `Ctrl+A` পাঠানো (bash-এ লাইনের শুরুতে যেতে: `Ctrl+A a`)
`:`	screen কমান্ড
`?`	সব কী
`\`	সব উইন্ডো বন্ধ করে screen থেকে বের হওয়া ⚠️

কমান্ড	কাজ
`screen -S name`	নাম দিয়ে নতুন সেশন
`screen -ls`	সেশনের তালিকা

কমান্ড	কাজ
`screen -r name`	ফেরা
`screen -d -r name`	অন্য জায়গায় attach থাকলে সেখান থেকে detach করে এখানে আনা
`screen -x name`	একই সেশনে একাধিক জায়গা থেকে (শেয়ার করা)

---

১১. `top` ও `htop`

প্রসেস দেখার লাইভ টুল। ব্যাখ্যা অধ্যায় ২৪ ও অধ্যায় ৪৬।

১১.১ `top` (procps-ng)

কী	কাজ
`h` বা `?`	সাহায্য
`q`	বের হওয়া
`Space`	এখনই রিফ্রেশ
`d` বা `s`	রিফ্রেশ বিরতি বদলানো (সেকেন্ডে)
`P`	CPU% অনুযায়ী সাজানো (ডিফল্ট)
`M`	মেমরি% অনুযায়ী
`T`	মোট CPU সময় অনুযায়ী
`N`	PID অনুযায়ী
`<` / `>`	সাজানোর কলাম বাঁয়ে / ডানে সরানো
`R`	সাজানো উল্টানো
`x` / `y`	সাজানোর কলাম / চলমান টাস্ক হাইলাইট
`1`	সব CPU core আলাদা দেখানো টগল
`t`	CPU সামারি লাইনের ধরন বদলানো (বার/গ্রাফ)
`m`	মেমরি সামারি লাইনের ধরন বদলানো
`l`	load average লাইন টগল
`E` / `e`	সামারি / প্রসেস তালিকায় মেমরির একক (KiB → MiB → GiB…)
`c`	পুরো কমান্ড লাইন দেখানো টগল
`V`	forest view — প্যারেন্ট-চাইল্ড গাছ
`H`	থ্রেড আলাদা দেখানো টগল
`i`	idle প্রসেস লুকানো টগল
`u` বা `U`	একটা ইউজারের প্রসেস শুধু

কী	কাজ
`o` / `O`	ফিল্টার (যেমন `COMMAND=nginx` বা `%CPU>5.0`); `=` চাপলে ফিল্টার মুছে যায়
`L`	লেখা খোঁজা
`n` বা `#`	কতগুলো প্রসেস দেখাবে
`f`	কোন কলাম দেখাবে বেছে নেওয়া
`z`	রঙিন টগল
`k`	প্রসেস kill (PID ও সিগন্যাল জিজ্ঞেস করবে; ডিফল্ট 15)
`r`	renice (nice মান বদলানো)
`W`	বর্তমান সেটিং `~/.config/procps/toprc` এ সেভ

১১.২ `htop`

htop নিচে F-কী-গুলো দেখায়। প্রায় সবগুলোর একটা অক্ষর-বিকল্পও আছে (ল্যাপটপে F-কী ঝামেলা করলে কাজের)।

কী	বিকল্প	কাজ
`F1`	`h` / `?`	সাহায্য
`F2`	`S`	সেটআপ (মিটার, কলাম, রঙ)
`F3`	`/`	খোঁজা (নামে লাফ)
`F4`	`\`	ফিল্টার (শুধু মিল থাকা প্রসেস)
`F5`	`t`	tree view টগল
`F6`	`<` / `>`	কোন কলামে সাজাবে
`F7`	`]`	nice কমানো = priority বাড়ানো (root লাগে)
`F8`	`[`	nice বাড়ানো = priority কমানো
`F9`	`k`	সিগন্যাল পাঠানো (kill) — তালিকা থেকে সিগন্যাল বেছে নাও
`F10`	`q`	বের হওয়া
`Space`		প্রসেস ট্যাগ (চিহ্নিত) করা — একসাথে অনেকগুলো kill করতে
`c`		প্রসেস ও তার সব চাইল্ড ট্যাগ
`U`		সব ট্যাগ বাতিল
`u`		একটা ইউজারের প্রসেস শুধু
`M` / `P` / `T`		মেমরি / CPU / সময় অনুযায়ী সাজানো
`I`		সাজানো উল্টানো
`H`		ইউজার থ্রেড লুকানো/দেখানো
`K`		কার্নেল থ্রেড লুকানো/দেখানো

কী	বিকল্প	কাজ
`p`		প্রোগ্রামের পুরো পাথ দেখানো টগল
`+` / `-`		tree view-তে শাখা খোলা / বন্ধ
`F`		কার্সর নির্দিষ্ট প্রসেসকে অনুসরণ করবে
`l`		প্রসেসের খোলা ফাইল (`lsof` লাগবে)
`s`		প্রসেসে `strace` লাগানো
`e`		প্রসেসের environment দেখানো
`i`		I/O priority (ionice)
`a`		CPU affinity (কোন core-এ চলবে)
`Ctrl+L`		স্ক্রিন রিফ্রেশ

---

১২. TTY (ভার্চুয়াল কনসোল) বদলানো

লিনাক্স এক মনিটরেই কয়েকটা আলাদা "ভার্চুয়াল টার্মিনাল" (VT) রাখে — যেন একটা টিভিতে কয়েকটা চ্যানেল। GUI নিজেও একটা চ্যানেলে চলে। GUI জমে গেলে অন্য চ্যানেলে টেক্সট কনসোলে লগইন করে সমস্যা ঠিক করা যায়। বুট ও TTY-র ভেতরের কাজ অধ্যায় ২৭-এ।

শর্টকাট	কাজ
`Ctrl+Alt+F1` ... `Ctrl+Alt+F6`	GUI থেকে VT ১–৬ এ যাওয়া
`Alt+F1` ... `Alt+F6`	**টেক্সট কনসোল থেকে** অন্য VT তে (সেখানে Ctrl লাগে না)
`Alt+Left` / `Alt+Right`	টেক্সট কনসোলে আগের / পরের VT
`chvt 3` (root)	কমান্ড দিয়ে VT ৩ এ
`Ctrl+Alt+Backspace`	X11 সার্ভার মেরে ফেলা — আধুনিক distro-তে **ডিফল্ট বন্ধ**, Wayland-এ প্রযোজ্য না

Ubuntu 24.04 / Fedora / RHEL 9 (GDM সহ) সাধারণ বিন্যাস:

```
VT1  (Ctrl+Alt+F1) → GDM লগইন স্ক্রিন
VT2  (Ctrl+Alt+F2) → তোমার লগইন করা গ্রাফিক্যাল সেশন (সাধারণত)
VT3–VT6            → টেক্সট লগইন (getty) — জরুরি অবস্থার জন্য
```

> 📝 **মনে রাখো:** GUI-তে কোন VT সেটা distro ও display manager অনুযায়ী বদলায় (পুরোনো সিস্টেমে VT7)। হারিয়ে গেলে `Ctrl+Alt+F1` থেকে `F7` পর্যন্ত একটা একটা করে চেষ্টা করো। বর্তমানে কোন VT: `fgconsole` (root লাগতে পারে) বা `loginctl` দেখো।

> 💡 **টিপস:** লিনাক্স কার্নেল ৫.৯ থেকে টেক্সট কনসোলের (fbcon/vgacon) সফটওয়্যার স্ক্রোলব্যাক সরিয়ে দেওয়া হয়েছে, তাই আধুনিক সিস্টেমে `Shift+Page Up` সাধারণত আর কাজ করে না। লম্বা আউটপুট দেখতে কনসোলে `| less` ব্যবহার করো বা `tmux` চালাও।

---

১৩. Magic SysRq এবং REISUB

সিস্টেম পুরো জমে গেছে — মাউস নড়ে না, `Ctrl+Alt+F3` কাজ করে না। পাওয়ার বাটন চেপে ধরলে ডিস্কে লেখা অর্ধেক থেকে যেতে পারে, ফাইলসিস্টেম নষ্ট হতে পারে। শেষ ভরসা **Magic SysRq**: কার্নেলের সরাসরি "জরুরি হটলাইন"। উপরের ডায়াগ্রামে দেখেছ, এই কী-কম্বিনেশন **কার্নেলের ইনপুট লেয়ারেই** ধরা পড়ে — GUI, শেল, কিছুই লাগে না। কার্নেল নিজে বেঁচে থাকলেই কাজ করে।

কীভাবে চাপবে: `Alt` চেপে ধরো + `SysRq` চেপে ধরো (সাধারণত `PrintScreen` কী-তেই লেখা থাকে; ল্যাপটপে `Fn` লাগতে পারে) + তারপর কমান্ড কী। কিছু কীবোর্ডে সুবিধার জন্য: `Alt` + `SysRq` চেপে ধরো, `SysRq` ছেড়ে দাও (Alt ধরে রেখে), তারপর কমান্ড কী।

১৩.১ REISUB — নিরাপদ রিবুট

ধীরে, প্রতিটা অক্ষরের মাঝে ১–৩ সেকেন্ড (বিশেষ করে `S` এর পরে কয়েক সেকেন্ড) অপেক্ষা করে চাপো। মনে রাখার বাক্য: **"Reboot Even If System Utterly Broken"** (ক্রমটা উল্টো করে পড়লে BUSIER — আরেকটা মনে রাখার উপায়)।

ক্রম	কী	কার্নেলের কাজ	কেন
১	`R`	unRaw — কীবোর্ডকে raw মোড থেকে স্বাভাবিক (XLATE) মোডে আনা	জমে যাওয়া GUI কীবোর্ড দখল করে রাখতে পারে
২	`E`	tErminate — init (PID 1) ছাড়া সব প্রসেসকে SIGTERM	প্রোগ্রামগুলোকে ভদ্রভাবে ডেটা সেভ করে বন্ধ হওয়ার সুযোগ
৩	`I`	kIll — init ছাড়া সব প্রসেসকে SIGKILL	যারা কথা শোনেনি, তাদের জোর করে বন্ধ
৪	`S`	Sync — সব মাউন্ট করা ফাইলসিস্টেমের ক্যাশ ডিস্কে লেখা	মেমরিতে আটকে থাকা ডেটা যাতে না হারায়
৫	`U`	Unmount — সব ফাইলসিস্টেম read-only হিসেবে আবার মাউন্ট	যাতে আর কিছু লেখা না হয়; পরের বুটে fsck লাগবে না
৬	`B`	reBoot — সঙ্গে সঙ্গে রিবুট (sync/unmount ছাড়া, তাই আগের ধাপগুলো জরুরি)	

বন্ধ করতে চাইলে শেষে `B` এর বদলে `O` (power **O**ff) — **REISUO**।

১৩.২ সব SysRq কী

কী	কাজ
`b`	তাৎক্ষণিক রিবুট
`o`	তাৎক্ষণিক পাওয়ার অফ
`s`	sync
`u`	সব ফাইলসিস্টেম read-only রিমাউন্ট
`e`	init বাদে সবাইকে SIGTERM
`i`	init বাদে সবাইকে SIGKILL
`f`	OOM killer চালানো — সবচেয়ে বেশি মেমরি খাওয়া প্রসেস মারা (মেমরি শেষ হয়ে সিস্টেম জমে গেলে প্রথমে এটা চেষ্টা করো!)
`k`	SAK (Secure Access Key) — বর্তমান কনসোলের সব প্রসেস মারা
`r`	কীবোর্ড raw মোড বন্ধ
`n`	real-time প্রসেসগুলোকে স্বাভাবিক priority তে আনা
`h`	সাহায্য (কনসোল/`dmesg` এ লেখে)
`m`	মেমরির তথ্য কনসোলে/লগে
`t`	সব টাস্কের তালিকা ও স্ট্যাক
`w`	আটকে থাকা (uninterruptible, D state) টাস্কগুলো
`l`	সব CPU-র backtrace
`p`	বর্তমান রেজিস্টার ও ফ্ল্যাগ
`0` … `9`	কনসোল log level বদলানো
`c`	ইচ্ছাকৃতভাবে কার্নেল crash (kdump পরীক্ষার জন্য) ⚠️ ডেটা হারাবে

১৩.৩ চালু আছে কিনা — `kernel.sysrq` বিটমাস্ক

নিরাপত্তার জন্য distro গুলো সব SysRq ফাংশন চালু রাখে না। মান দেখো:

```
$ cat /proc/sys/kernel/sysrq
176
```

মান (বিট)	অনুমতি দেয়
`0`	সব বন্ধ
`1`	সব চালু
`2`	কনসোল log level
`4`	কীবোর্ড কন্ট্রোল (`r`, `k`)

মান (বিট)	অনুমতি দেয়
`8`	ডিবাগ ডাম্প (`t` , `m` , `w` , `l` , `p` ...)
`16`	sync (`s`)
`32`	read-only remount (`u`)
`64`	প্রসেসে সিগন্যাল (`e` , `i` , `f`)
`128`	রিবুট / পাওয়ার অফ (`b` , `o`)
`256`	real-time nice (`n`)

ডিফল্ট মান distro ভেদে আলাদা:

Distro	ডিফল্ট	মানে
Ubuntu 24.04	`176` = 16+32+128	শুধু `s` , `u` , `b` / `o` — তাই পূর্ণ REISUB-এর `R` , `E` , `I` কাজ করে না, কিন্তু `S U B` করবে
Fedora / RHEL 9	`16` (systemd-এর ডিফল্ট)	শুধু sync
Debian 12	`438` = 2+4+16+32+128+256	`e` / `i` / `f` (64) ছাড়া প্রায় সব

পুরোপুরি চালু করতে:

```
$ sudo sysctl -w kernel.sysrq=1                                  # এখনই (রিবুটে হারাবে)
$ echo 'kernel.sysrq = 1' | sudo tee /etc/sysctl.d/90-sysrq.conf  # স্থায়ী
```

কীবোর্ড ছাড়াও (যেমন SSH দিয়ে) root হিসেবে সরাসরি কার্নেলকে বলা যায় — এটা বিটমাস্ক মানে না:

```
$ echo s | sudo tee /proc/sysrq-trigger      # sync
$ echo m | sudo tee /proc/sysrq-trigger && sudo dmesg | tail
```

> ⚠️ **সতর্কতা:** `echo b > /proc/sysrq-trigger` সঙ্গে সঙ্গে রিবুট করে, কিছু সেভ না করে। প্রোডাকশন সার্ভারে শুধু শেষ উপায় হিসেবে, আর আগে `s` ও `u` দিয়ে। `/proc` সম্পর্কে বিস্তারিত অধ্যায় ৪৭।

---

১৪. SSH escape sequence

SSH সেশন জমে গেছে (নেটওয়ার্ক কেটে গেছে, `Ctrl+C` কাজ করছে না)? ssh ক্লায়েন্ট একটা গোপন "escape character" শোনে — ডিফল্ট `~` — কিন্তু **শুধু লাইনের একদম শুরুতে**। তাই আগে `Enter` চাপো, তারপর `~` , তারপর কমান্ড অক্ষর। এগুলো রিমোট সার্ভারে যায় না, তোমার নিজের ssh ক্লায়েন্ট ধরে। SSH-এর পুরো আলোচনা অধ্যায় ৩৮-এ।

ক্রম	কাজ
`Enter` `~` `.`	সংযোগ তাৎক্ষণিক বন্ধ (জমে যাওয়া সেশন থেকে বের হওয়ার উপায়)
`Enter` `~` `Ctrl+Z`	ssh-কে সাসপেন্ড করে লোকাল শেলে (ফেরত `fg`)
`Enter` `~` `#`	চলমান port forward / সংযোগের তালিকা
`Enter` `~` `&`	ssh-কে ব্যাকগ্রাউন্ডে পাঠানো (forward করা সংযোগ বন্ধের অপেক্ষার সময়)
`Enter` `~` `C`	ssh কমান্ড লাইন খুলে চলতি সেশনেই port forward যোগ (`-L 8080:localhost:80`) — OpenSSH 9.2 থেকে ডিফল্ট বন্ধ; `-o EnableEscapeCommandline=yes` লাগবে
`Enter` `~` `R`	rekey (নতুন এনক্রিপশন কী)
`Enter` `~` `v` / `~` `V`	লগ verbosity বাড়ানো / কমানো
`Enter` `~` `B`	রিমোটে BREAK পাঠানো
`Enter` `~` `?`	সব escape-এর তালিকা
`Enter` `~` `~`	আক্ষরিক `~` পাঠানো

> 💡 **টিপস:** ssh-এর ভেতরে ssh থাকলে (jump করে ঢুকলে), ভেতরের সংযোগ বন্ধ করতে `~~.` (দুটো tilde) — প্রথম `~` বাইরের ssh খেয়ে নেয়। escape character বদলাতে `ssh -e '^]' host`, বা বন্ধ করতে `-e none` (বাইনারি ডেটা পাঠানোর সময় দরকারি)।

---

১৫. GRUB মেনুর কী

GRUB হলো বুটলোডার — পাওয়ার বাটন আর লিনাক্স কার্নেলের মাঝের দারোয়ান (অধ্যায় ২৭)। Ubuntu ও Fedora একটাই OS থাকলে মেনু লুকিয়ে রাখে।

কখন / কী	কাজ
বুটের সময় `Shift` চেপে ধরা	লুকানো মেনু দেখানো — BIOS/Legacy মোডে (Ubuntu)
বুটের সময় `Esc` চাপা (একবার, বারবার না)	লুকানো মেনু দেখানো — UEFI মোডে (Ubuntu); বারবার চাপলে GRUB প্রম্পটে চলে যেতে পারে
`Esc`, `F8` বা `Shift`	Fedora/RHEL-এ লুকানো মেনু (menu auto-hide) দেখানো
`Up` / `Down`	এন্ট্রি বাছাই
`Enter`	বাছাই করা এন্ট্রি বুট
`e`	এন্ট্রি এডিট (এক বুটের জন্য কার্নেল প্যারামিটার বদলানো)
`c`	GRUB কমান্ড লাইন (`ls`, `set`, `linux`, `boot`…)
`Esc`	এডিট/কমান্ড লাইন থেকে মেনুতে ফেরা (পরিবর্তন বাদ)

এডিট মোডে (`e` চাপার পর):

কী	কাজ
`Ctrl+X` বা `F10`	এই পরিবর্তন নিয়ে বুট (স্থায়ী হয় না)
`Ctrl+C` বা `F2`	GRUB কমান্ড লাইনে যাওয়া
`Esc`	বাতিল, মেনুতে ফেরা
`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষ
`Ctrl+K`	লাইনের শেষ পর্যন্ত মোছা
`Tab` (কমান্ড লাইনে)	কমান্ড/ডিভাইস কমপ্লিশন

কাজের উদাহরণ — `linux` দিয়ে শুরু হওয়া লাইনের শেষে যোগ করো:

যোগ করো	ফল
`systemd.unit=rescue.target` (বা শুধু `single`)	রেসকিউ মোড — root পাসওয়ার্ড লাগে
`systemd.unit=emergency.target`	ইমার্জেন্সি মোড — আরও কম সার্ভিস
`nomodeset`	গ্রাফিক্স ড্রাইভার সমস্যায় কালো স্ক্রিন এড়ানো
`rd.break` (Fedora/RHEL)	initramfs-এ থামা — root পাসওয়ার্ড রিসেটের পদ্ধতি
`init=/bin/bash`	শুধু একটা root শেল (Debian/Ubuntu-তে পাসওয়ার্ড রিসেটের পুরোনো কৌশল; ফাইলসিস্টেম read-only থাকে, `mount -o remount,rw /` লাগবে)

> ⚠️ **সতর্কতা:** GRUB-এ `e` দিয়ে যে কেউ root শেল পেতে পারে — শারীরিক অ্যাক্সেস মানেই প্রায় পূর্ণ অ্যাক্সেস। গুরুত্বপূর্ণ মেশিনে GRUB পাসওয়ার্ড ও ডিস্ক এনক্রিপশন (LUKS) দরকার — অধ্যায় ৪৫। মেনু স্থায়ীভাবে দেখাতে Ubuntu-তে `/etc/default/grub` এ `GRUB_TIMEOUT_STYLE=menu` আর `GRUB_TIMEOUT=5` দিয়ে `sudo update-grub`; Fedora/RHEL-এ `sudo grub2-editenv - unset menu_auto_hide`।

---

১৬. "জরুরি অবস্থায় কী চাপব?" — সমস্যা ধরে শর্টকাট

শর্টকাট মুখস্থ করার চেয়ে কাজে লাগে "কোন বিপদে কোনটা" জানা। নিচের টেবিল উপর থেকে নিচে — হালকা থেকে কড়া — সাজানো।

সমস্যা	প্রথমে চেষ্টা করো	তারপর
টার্মিনাল টাইপ নিচ্ছে না	`Ctrl+Q` (XOFF ছাড়ানো)	`reset` লিখে Enter

সমস্যা	প্রথমে চেষ্টা করো	তারপর
কমান্ড চলছে, থামছে না	`Ctrl+C`	`Ctrl+\`, তারপর অন্য টার্মিনাল থেকে `kill -9 PID`
ভুলে একটা প্রোগ্রাম খুলে ফেলেছি, বের হব কীভাবে	`q` (less/man/top), `Esc :q!` (vim), `Ctrl+X` (nano)	`Ctrl+C`, `Ctrl+D`
vim থেকে বের হতে পারছি না	`Esc` `Esc` `:q!` `Enter`	`Ctrl+Z` দিয়ে সাসপেন্ড করে `kill %1`
SSH সেশন জমে গেছে	`Enter` `~` `.`	টার্মিনাল ট্যাব বন্ধ
একটা GUI উইন্ডো জমে গেছে	`Alt+F4`	KDE: `Ctrl+Alt+Esc`; GNOME: "Force Quit" ডায়ালগ বা `xkill` / `pkill`
পুরো GUI জমে গেছে	`Ctrl+Alt+F3` → লগইন → `systemctl restart gdm` (বা `sddm`) ⚠️ খোলা কাজ হারাবে	
মেমরি শেষ, সব জমে গেছে	`Alt+SysRq+F` (OOM killer — যদি চালু থাকে)	
কিছুই কাজ করছে না	`Alt+SysRq` + `R E I S U B`	পাওয়ার বাটন ৫–১০ সেকেন্ড (শেষ উপায়)
বুট হচ্ছে না	GRUB মেনু (`Shift` / `Esc`) → পুরোনো কার্নেল বা Advanced → recovery	`e` → `systemd.unit=rescue.target`

---

১৭. প্রিন্ট ও PDF করার নির্দেশনা

এই পরিশিষ্টের টেবিলগুলো প্রিন্ট-বান্ধব রাখা হয়েছে (রঙ বা ছবির উপর নির্ভর নেই)। দুটো সহজ উপায়:

১) ব্রাউজার দিয়ে: GitHub/GitLab বা VS Code-এর Markdown preview-তে খোলো → `Ctrl+P` → "Save as PDF"। Landscape (আড়াআড়ি) বেছে নিলে চওড়া টেবিল ভালো আসে।

২) pandoc দিয়ে (বাংলা ফন্ট সহ):

```
$ sudo apt install pandoc texlive-xetex fonts-noto-core      # Debian/Ubuntu
$ sudo dnf install pandoc texlive-xetex google-noto-sans-bengali-fonts   # Fedora
$ pandoc appendix-a-shortcuts-cheatsheet.md -o cheatsheet.pdf \
    --pdf-engine=xelatex \
    -V mainfont="Noto Sans Bengali" -V monofont="DejaVu Sans Mono" \
    -V geometry:landscape -V geometry:margin=1.5cm
```

> 💡 **টিপস:** বাংলা যুক্তাক্ষর ঠিকমতো দেখাতে xelatex-এর বদলে `--pdf-engine=lualatex` ভালো কাজ করতে পারে (`texlive-luatex` প্যাকেজ)। পুরো চিটশিট একবারে প্রিন্ট না করে সবচেয়ে বেশি ব্যবহৃত অংশগুলো (Bash readline, vim, tmux) আলাদা পাতায় নিয়ে মনিটরের পাশে লাগাও। আর ফন্ট

> প্যাকেজের নাম distro ভার্সনে বদলাতে পারে — না পেলে `apt search noto | grep -i bengali` বা `dnf search bengali` দিয়ে খোঁজো।

হাতে-কলমে অনুশীলন

শর্টকাট পড়ে শেখা যায় না, **আঙুলে** শিখতে হয়। প্রতিদিন ১০ মিনিট করে এক সপ্তাহ এই ল্যাবগুলো করো।

ল্যাব ১: কী আসলে কী পাঠায় (৫ মিনিট)

1. টার্মিনালে `cat -v` চালাও।
2. একে একে চাপো: `Alt+F`, উপরের অ্যারো, `Ctrl+[` তারপর `[A` টাইপ করো, `Tab`, `Ctrl+I`।
3. লক্ষ্য করো: `Alt+F` দেখায় `^[f`, আর অ্যারো দেখায় `^[[A` — অর্থাৎ অ্যারো কী আসলে Esc + `[A`! `Tab` আর `Ctrl+I` স্ক্রিনে একই ফাঁকা জায়গা দেয়।
4. `Ctrl+D` (খালি লাইনে) দিয়ে বের হও। এবার `stty -a | head -4` চালিয়ে দেখো `intr = ^C` কোথায় লেখা।

ল্যাব ২: Readline ড্রিল (১০ মিনিট)

নিচের লাইনটা টাইপ করো, **Enter চাপবে না**:

```
$ echo one two three four /usr/local/share/doc
```

এবার শুধু কীবোর্ড দিয়ে:

1. `Ctrl+A` → শুরুতে যাও; `Ctrl+E` → শেষে।
2. `Alt+B` তিনবার চাপো — কোথায় থামল? (শব্দ গণনায় `/` ও আলাদা করে)
3. `Ctrl+E` তারপর `Alt+Backspace` — শুধু `doc` কাটল। এবার `Ctrl+W` — বাকি পুরো পাথ কাটল। পার্থক্যটা বোঝো।
4. `Ctrl+Y` — শেষ কাটা অংশ ফিরে এল। সঙ্গে সঙ্গে `Alt+Y` — kill ring ঘুরে আগেরটা এল।
5. `Ctrl+A`, তারপর `Alt+F` দু'বার (কার্সর `one` এর শেষে), তারপর `Alt+T` — `one` আর `two` এর জায়গা বদলাল? (`Alt+T` কার্সরের আগের শব্দকে পরের শব্দের পার করে দেয়।)
6. `Ctrl+_` কয়েকবার চেপে undo করো। শেষে `Ctrl+U` দিয়ে পুরো লাইন মুছে ফেলো।
7. `Ctrl+X Ctrl+E` চাপো — এডিটর খুলবে। একটা লম্বা `for` লুপ লিখে সেভ করে বের হও, দেখো চলে কিনা।

ল্যাব ৩: হিস্ট্রি জাদু (১০ মিনিট)

```
$ mkdir -p /tmp/lab/a/b/c
$ cd !$
$ pwd
```

```
/tmp/lab/a/b/c
$ touch report.txt.bak
$ echo !$:r
echo report.txt
report.txt
$ cat /etc/hostname
$ ^hostname^os-release
cat /etc/os-release
...
$ ls /etc/ssh /etc/apt
$ echo !*:p
echo /etc/ssh /etc/apt
```

তারপর `Ctrl+R` চাপো, `host` লেখো — `cat /etc/hostname` খুঁজে পাবে। `Enter` না চেপে `Right` চাপো যাতে এডিট করতে পারো। (Fedora-তে `/etc/apt` নেই — `ls` এরর দেখাবে, কিন্তু অনুশীলনে সমস্যা নেই।)

ল্যাব ৪: less ও vim একসাথে (১৫ মিনিট)

1. `man bash` খোলো। `/READLINE` লিখে Enter → `n` চেপে ৩–৪ বার এগোও → `&bind` দিয়ে ফিল্টার করো → `&` Enter দিয়ে ফিল্টার সরাও → `G`, `g`, `q`।
2. `cp /etc/services /tmp/lab/s.txt && vim /tmp/lab/s.txt`
3. Vim-এ: `:set nu` → `/ssh` → `n` → `dd` → `u` → `Ctrl+R` → `:%s/tcp/TCP/gc` (কয়েকটায় `y`, বাকিগুলোতে `q`) → `gg` → `qa0i# <Esc>jq` (macro রেকর্ড) → `5@a` → `:wq`।
4. `head /tmp/lab/s.txt` দিয়ে ফল দেখো — প্রথম কয়েকটা লাইনের শুরুতে `#` বসেছে তো?

ল্যাব ৫: tmux সার্ভাইভাল (১৫ মিনিট)

1. `tmux new -s lab`
2. `Ctrl+B %` (বাম-ডান), তারপর ডান প্যানে `Ctrl+B "` (উপর-নিচ)।
3. এক প্যানে `top`, এক প্যানে `ping -c 100 127.0.0.1`, এক প্যানে খালি শেল।
4. `Ctrl+B` + অ্যারো দিয়ে ঘোরো; `Ctrl+B z` দিয়ে একটা প্যান zoom করো, আবার `Ctrl+B z`।
5. `Ctrl+B d` দিয়ে detach করো। টার্মিনালটাই বন্ধ করে দাও!
6. নতুন টার্মিনাল খুলে `tmux ls`, তারপর `tmux attach -t lab` — সব আগের মতো চলছে।
7. `Ctrl+B [` দিয়ে ping-এর পুরোনো আউটপুট স্ক্রোল করো, `q` দিয়ে বের হও। শেষে `tmux kill-session -t lab`।

ল্যাব ৬: TTY ও SysRq (নিরাপদ অংশ, ১০ মিনিট)

1. `Ctrl+Alt+F3` → টেক্সট কনসোলে লগইন করো → `who` চালাও → `Alt+F4` বা `Alt+Right` দিয়ে আরেক কনসোলে যাও → `exit` দিয়ে লগআউট → আবার GUI-তে ফেরো (সাধারণত `Ctrl+Alt+F2` বা `F1`)।
2. `cat /proc/sys/kernel/sysrq` দিয়ে তোমার distro-র মান দেখো এবং উপরের টেবিল মিলিয়ে বের করো কোন কী অনুমোদিত।

3. নিরাপদ পরীক্ষা: `echo h | sudo tee /proc/sysrq-trigger`, তারপর `sudo dmesg | tail -3` — কার্নেল সাহায্য বার্তা লিখেছে।
4. ⚠️ `b`, `o`, `c`, `e`, `i` **পরীক্ষা করো না** — চলমান সব কাজ হারাবে। শুধু ভার্চুয়াল মেশিনে (VM) চর্চা করো (অধ্যায় ৪৮)।

ল্যাব ৭: SSH escape (৫ মিনিট, নিজের মেশিনেই)

1. `ssh localhost` (`openssh-server` চালু থাকলে)।
2. `Enter` চাপো, তারপর `~?` — escape তালিকা দেখো।
3. `Enter` `~` `Ctrl+Z` → লোকাল শেলে ফিরলে। `jobs` দেখো, তারপর `fg` দিয়ে ফেরো।
4. `Enter` `~` `.` — সংযোগ তৎক্ষণাৎ বন্ধ।

সারসংক্ষেপ

পুরো পরিশিষ্ট থেকে "যদি মাত্র ৪০টা শিখি" — দ্রুত রেফারেন্স:

ক্যাটাগরি	শর্টকাট	কাজ
ডেস্কটপ	`Super` / `Super+A`	Activities / অ্যাপ গ্রিড (GNOME)
ডেস্কটপ	`Alt+Tab`, `Super+Left/Right/Up`	অ্যাপ বদলানো, টাইল/ম্যাক্সিমাইজ
ডেস্কটপ	`Ctrl+Alt+T`, `Super+L`	টার্মিনাল, লক
টার্মিনাল	`Ctrl+Shift+C` / `Ctrl+Shift+V`	কপি / পেস্ট
টার্মিনাল	`Ctrl+Shift+T`, `Ctrl+Page Up/Down`	নতুন ট্যাব, ট্যাব বদলানো
Readline	`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষ
Readline	`Alt+B` / `Alt+F`	শব্দ ধরে পেছনে / সামনে
Readline	`Ctrl+U` / `Ctrl+K` / `Ctrl+W`	শুরু পর্যন্ত / শেষ পর্যন্ত / আগের শব্দ কাটা
Readline	`Ctrl+Y` / `Ctrl+_`	পেস্ট / undo
Readline	`Ctrl+R` / `Ctrl+G`	হিস্ট্রি সার্চ / বাতিল
Readline	`Alt+.`	আগের কমান্ডের শেষ argument
Readline	`Tab Tab`	কমপ্লিশনের সব সম্ভাবনা
Readline	`Ctrl+X Ctrl+E`	লাইন এডিটরে খোলা
TTY	`Ctrl+C` / `Ctrl+Z` / `Ctrl+D`	থামাও / সাসপেন্ড / EOF
TTY	`Ctrl+S` / `Ctrl+Q`	আউটপুট থামাও / চালু করো
TTY	`Ctrl+L`	স্ক্রিন পরিষ্কার
History	`!!`, `sudo !!`	আগের কমান্ড / sudo দিয়ে
History	`!$`, `!*`, `^old^new`	শেষ arg / সব arg / বদলে চালানো

ক্যাটাগরি	শর্টকাট	কাজ
less/man	`/` `n` `N` `g` `G` `q`	খোঁজা, পরের, আগের, শুরু, শেষ, বের
nano	`Ctrl+O` / `Ctrl+X` / `Ctrl+W` / `Ctrl+K` / `Ctrl+U`	সেভ / বের / খোঁজা / কাটা / পেস্ট
vim	`i`, `Esc`, `:wq`, `:q!`	Insert, Normal, সেভ-বের, না সেভ করে বের
vim	`dd` `yy` `p` `u` `Ctrl+R` `.`	কাটা, কপি, পেস্ট, undo, redo, পুনরাবৃত্তি
vim	`gg` `G` `/x` `:%s/a/b/g`	শুরু, শেষ, খোঁজা, সব বদলানো
vim	`ciw`, `ci"`, `V`, `Ctrl+V`	শব্দ বদল, কোটের ভেতর বদল, লাইন/ব্লক সিলেকশন
tmux	`Ctrl+B` `c` / `n` / `p`	নতুন / পরের / আগের উইন্ডো
tmux	`Ctrl+B` `%` / `"` / `z`	বাম-ডান / উপর-নিচ ভাগ / zoom
tmux	`Ctrl+B` `d`, `tmux attach -t name`	detach, ফেরা
tmux	`Ctrl+B` `[`	স্ক্রোল/কপি মোড
screen	`Ctrl+A` `c` / `d` / `"`	নতুন / detach / তালিকা
top	`P` `M` `1` `k` `q`	CPU, মেমরি, প্রতি-core, kill, বের
htop	`F3` `F4` `F5` `F6` `F9`	খোঁজা, ফিল্টার, tree, sort, kill
TTY	`Ctrl+Alt+F3`, `Alt+F1..F6`	টেক্সট কনসোলে / কনসোলের মাঝে
SysRq	`Alt+SysRq+R E I S U B`	নিরাপদ জরুরি রিবুট
SysRq	`Alt+SysRq+F`	OOM killer
SSH	`Enter ~ .`	জমে যাওয়া সেশন বন্ধ
SSH	`Enter ~ Ctrl+Z`, `Enter ~ ?`	সাসপেন্ড, সাহায্য
GRUB	`Shift` / `Esc` (বুটে)	লুকানো মেনু
GRUB	`e`, `Ctrl+X`, `c`	এডিট, এডিট করে বুট, কমান্ড লাইন

নিজেকে যাচাই করো

1. টার্মিনালে কপি করতে `Ctrl+C` এর বদলে `Ctrl+Shift+C` কেন লাগে? কোন লেয়ার `Ctrl+C` ধরে?
2. `Ctrl+W` আর `Alt+Backspace` — দুটোই আগের শব্দ মোছে। `cd /var/log/nginx` লাইনের শেষে থেকে চাপলে দুটোর ফল কী আলাদা হবে?
3. টার্মিনাল হঠাৎ আর কিছু দেখাচ্ছে না, টাইপও হচ্ছে না বলে মনে হচ্ছে। সবচেয়ে সম্ভাব্য কারণ কী, আর কোন কী দিয়ে ঠিক করবে?
4. আগের কমান্ড ছিল `vim /etc/nginx/sites-available/default`। এখন শুধু `/etc/nginx/sites-available` ডিরেক্টরিতে যেতে সবচেয়ে ছোট কমান্ড কী?

5. Vim-এ `"hello world"` এর ভেতরে কার্সর আছে। কোট রেখে ভেতরের পুরো লেখা বদলাতে কোন কী? কোটসহ মুছতে কোনটা?
6. tmux-এ ডিফল্ট prefix কী, আর screen-এ কী? screen চালু অবস্থায় bash-এ লাইনের শুরুতে যেতে কী চাপবে?
7. REISUB-এ `S` আর `U` কেন `B` এর আগে? `B` সরাসরি চাপলে কী সমস্যা?
8. Ubuntu 24.04-এ `cat /proc/sys/kernel/sysrq` দেখাল `176`। `Alt+SysRq+E` কাজ করবে কি? কেন?
9. SSH সেশন জমে গেছে। `~.` চাপলে কিছু হচ্ছে না। সম্ভাব্য দুটো কারণ কী?
10. GRUB মেনুতে একটা এন্ট্রি এডিট করে `nomodeset` যোগ করলে। পরিবর্তন নিয়ে বুট করার কী কোনটা, আর এটা কি পরের বুটেও থাকবে?

▼ **উত্তর দেখো**

1. কারণ `Ctrl+C` বাইট `0x03` পাঠায়, যেটা কার্নেলের **TTY line discipline** ধরে চলমান প্রোগ্রামকে SIGINT পাঠায়। টার্মিনাল emulator যদি `Ctrl+C` কে কপি হিসেবে নিত, তাহলে প্রোগ্রাম থামানোর উপায় থাকত না — তাই কপিতে Shift যোগ হয়।
2. `Ctrl+W` স্পেস পর্যন্ত মোছে → পুরো `/var/log/nginx` মুছে শুধু `cd ` থাকে। `Alt+Backspace` `/` কে শব্দের সীমা ধরে → শুধু `nginx` মোছে, `cd /var/log/` থাকে।
3. সম্ভবত ভুলে `Ctrl+S` চাপা হয়েছে (XOFF — আউটপুট থেমে আছে, তুমি যা টাইপ করছ তা জমা হচ্ছে)। `Ctrl+Q` চাপো। আজব অক্ষর দেখালে `reset`।
4. `cd !$:h` — `!$` = শেষ argument, `:h` = head অর্থাৎ শেষ অংশ (`default`) বাদ। কীবোর্ড দিয়ে বিকল্প: `cd ` লিখে `Alt+.` চাপো, তারপর `Alt+Backspace` দিয়ে `default` মুছে Enter।
5. `ci"` — কোটের ভেতরের লেখা মুছে Insert মোডে। কোটসহ মুছতে `da"`।
6. tmux: `Ctrl+B`; screen: `Ctrl+A`। screen-এর ভেতরে bash-এর `Ctrl+A` পাঠাতে `Ctrl+A a` চাপতে হয়।
7. `S` (sync) মেমরির ক্যাশে থাকা অলিখিত ডেটা ডিস্কে লেখে, `U` ফাইলসিস্টেমকে read-only করে যাতে আর কিছু অর্ধেক লেখা না হয়। `B` নিজে কিছু sync বা unmount করে না — সরাসরি রিসেটের মতো; ফলে ডেটা হারানো ও ফাইলসিস্টেম ক্ষতির ঝুঁকি।
8. না। `176` = 16 (sync) + 32 (remount) + 128 (reboot/poweroff)। `E` প্রসেসে সিগন্যাল পাঠানোর ফাংশন, যার বিট 64 — সেটা এই মানে নেই। তবে `S`, `U`, `B` কাজ করবে। (root হিসেবে `/proc/sysrq-trigger` এ লিখলে বিটমাস্ক মানা হয় না।)
9. (ক) `~` লাইনের **শুরুতে** চাপা হয়নি — আগে `Enter` চাপতে হয়। (খ) ssh-এর ভেতরে আরেকটা ssh আছে — তখন বাইরেরটা `~` খেয়ে নেয়, ভেতরেরটার জন্য `~~.` লাগে। (escape character `-e none` দিয়ে বন্ধ করা থাকলেও কাজ করবে না।)
10. `Ctrl+X` বা `F10`। না — GRUB-এ `e` দিয়ে করা পরিবর্তন শুধু ওই একবারের বুটের জন্য। স্থায়ী করতে `/etc/default/grub` এর `GRUB_CMDLINE_LINUX_DEFAULT` এ যোগ করে `sudo update-grub` (Debian/Ubuntu) বা `sudo grubby --update-kernel=ALL --args=nomodeset` (Fedora/RHEL)।

অধ্যায় ১: লিনাক্স কী এবং কেন শিখবে

> "Linux is only free if your time has no value." — একটা পুরোনো রসিকতা। এই বইয়ের লক্ষ্য হলো রসিকতাটাকে ভুল প্রমাণ করা: তোমার সময় যাতে বাঁচে, সেভাবে লিনাক্স শেখানো।

এই অধ্যায়ে যা শিখবে

- **অপারেটিং সিস্টেম (Operating System)** আসলে কী, কেন দরকার, আর কম্পিউটারের ভেতরে সে কী কাজ করে
- **Unix**-এর জন্মকথা: Bell Labs, Ken Thompson, Dennis Ritchie আর C ভাষা
- **Unix দর্শন (Unix Philosophy)** — যে চিন্তাধারা আজও লিনাক্সের প্রতিটি কমান্ডে দেখা যায়
- **Richard Stallman ও GNU প্রজেক্ট**, Free Software Foundation (FSF) এবং "free" শব্দের আসল মানে
- **Linus Torvalds** কীভাবে একটা "শখের প্রজেক্ট" থেকে Linux kernel বানালেন
- **"Linux" নাকি "GNU/Linux"** — নামকরণ বিতর্ক আসলে কী নিয়ে
- **ওপেন সোর্স ও লাইসেন্স** — GPL, LGPL, MIT, Apache, BSD; copyleft বনাম permissive
- **লিনাক্স কোথায় কোথায় চলে** — সার্ভার, ক্লাউড, Android, সুপারকম্পিউটার, IoT, গাড়ি, এমনকি মহাকাশে
- **Windows ও macOS-এর সাথে লিনাক্সের তুলনা**
- **লিনাক্স জেনে কী কী ক্যারিয়ার** গড়া যায় — SysAdmin, DevOps, Cloud, Security, SRE, Embedded
- **এই বইটি কীভাবে পড়বে**, কীভাবে অনুশীলন করবে, আর বইয়ের সংকেতগুলোর মানে
- জীবনের প্রথম কয়েকটি লিনাক্স কমান্ড: `uname`, `cat /etc/os-release`, `hostnamectl`, `whoami`

১.১ প্রথমে একটা গল্প: কম্পিউটারের "ম্যানেজার"

ধরো, তুমি একটা বড় রেস্টুরেন্টে গেলে। সেখানে আছে রান্নাঘর (চুলা, ফ্রিজ, ছুরি), আছে বাবুর্চি, আছে অনেক টেবিলে অনেক কাস্টমার। কাস্টমার কি সরাসরি রান্নাঘরে ঢুকে চুলা জ্বালায়? না। সে **ওয়েটারকে** অর্ডার দেয়। ওয়েটার অর্ডারটা রান্নাঘরে পৌঁছে দেয়, রান্নাঘরের **ম্যানেজার** ঠিক করে কোন বাবুর্চি কোন চুলায় কোন রান্না আগে করবে, কোন উপকরণ কতটুকু খরচ হবে, তারপর খাবার আবার ওয়েটারের হাত দিয়ে টেবিলে আসে।

কম্পিউটারও ঠিক এমন:

রেস্টুরেন্ট	কম্পিউটার
রান্নাঘরের যন্ত্রপাতি (চুলা, ফ্রিজ)	**Hardware** — CPU, RAM, ডিস্ক, নেটওয়ার্ক কার্ড
রান্নাঘরের ম্যানেজার	**Kernel** (কার্নেল) — অপারেটিং সিস্টেমের মূল অংশ
ওয়েটার	**Shell** (শেল) / GUI — তোমার কথা কার্নেলের কাছে পৌঁছায়
কাস্টমার	**User** (ব্যবহারকারী) ও **Application** (প্রোগ্রাম)
অর্ডারের নিয়মকানুন	**System call** — প্রোগ্রাম কার্নেলের কাছে যেভাবে "অনুরোধ" করে

এই "ম্যানেজার + নিয়মকানুন + সহায়ক টুল" — সব মিলিয়ে যা দাঁড়ায়, তাকেই বলে **অপারেটিং সিস্টেম**।

১.২ অপারেটিং সিস্টেম (Operating System) কী?

অপারেটিং সিস্টেম (OS) হলো এমন একটি সফটওয়্যার, যা কম্পিউটারের হার্ডওয়্যার পরিচালনা করে এবং অন্য সব প্রোগ্রামকে সেই হার্ডওয়্যার নিরাপদে ও ন্যায্যভাবে ব্যবহার করার সুযোগ দেয়।

OS না থাকলে প্রতিটি প্রোগ্রামকে (ধরো একটা গান বাজানোর অ্যাপ) নিজেই জানতে হতো তোমার সাউন্ড কার্ডের চিপ কোন কোম্পানির, ডিস্কের কোন সেক্টরে গানের ফাইল আছে, আর অন্য প্রোগ্রাম একই সময়ে সাউন্ড কার্ড চাইলে কী হবে। এটা অসম্ভব রকম জটিল হতো। OS এই জটিলতা লুকিয়ে রাখে — এটাকে বলে **abstraction** (বিমূর্তকরণ, মানে জটিল জিনিসকে সহজ একটা "মুখোশ" দিয়ে ঢেকে দেওয়া)।

OS-এর প্রধান কাজগুলো

কাজ	ইংরেজি নাম	সহজ ব্যাখ্যা
প্রসেস ম্যানেজমেন্ট	Process management	কোন প্রোগ্রাম কখন CPU পাবে, কতক্ষণ পাবে — ঠিক করা (অধ্যায় ২৪)
মেমরি ম্যানেজমেন্ট	Memory management	কোন প্রোগ্রাম RAM-এর কোন অংশ পাবে, যাতে একজন আরেকজনের মেমরিতে হাত না দেয়
ফাইল সিস্টেম	File system	ডিস্কে ফাইল-ফোল্ডার আকারে ডেটা সাজিয়ে রাখা (অধ্যায় ৭, অধ্যায় ৩২)
ডিভাইস ম্যানেজমেন্ট	Device management	কীবোর্ড, মাউস, প্রিন্টার, USB — এদের driver দিয়ে চালানো
নেটওয়ার্কিং	Networking	ইন্টারনেট ও লোকাল নেটওয়ার্কে ডেটা আদান-প্রদান (অধ্যায় ৩৬)
নিরাপত্তা ও অনুমতি	Security & permissions	কে কোন ফাইল পড়তে/লিখতে পারবে (অধ্যায় ১৬)
ইউজার ইন্টারফেস	User interface	কমান্ড লাইন (CLI) বা গ্রাফিক্যাল (GUI) মাধ্যমে মানুষের সাথে যোগাযোগ

OS-এর স্তরগুলো (একটা সরল ছবি)

```
+-----------------------------------------------------------+
|   তুমি (User)                                             |
```

```
+-------------------------------------------------------------+
|   Applications: Firefox, VS Code, LibreOffice, nginx ...    |
+-------------------------------------------------------------+
|   Shell (bash, zsh)  /  Desktop (GNOME, KDE)                |
+-------------------------------------------------------------+
|   System Libraries (glibc ইত্যাদি)                          |
+=============================================================+  <-- System Call সীমানা
|   KERNEL  (Linux kernel)                                    |
|   - Process scheduler  - Memory manager  - VFS (file system)|
|   - Network stack      - Device drivers                     |
+=============================================================+
|   HARDWARE: CPU, RAM, Disk, NIC, GPU, USB ...               |
+-------------------------------------------------------------+
```

উপরের দুই লাইনের `=====` সীমানাটা খুব গুরুত্বপূর্ণ। এর ওপরে যা চলে তাকে বলে **user space** (ব্যবহারকারীর এলাকা), আর নিচে কার্নেল চলে **kernel space**-এ। user space-এর প্রোগ্রাম সরাসরি হার্ডওয়্যার ছুঁতে পারে না; তাকে **system call** (যেমন `open()`, `read()`, `write()`, `fork()`) দিয়ে কার্নেলকে অনুরোধ করতে হয়। CPU নিজেই এই সীমানা পাহারা দেয় (x86-এ একে বলে privilege ring — কার্নেল চলে ring 0-তে, সাধারণ প্রোগ্রাম ring 3-তে)।

> 📝 **মনে রাখো:** "Linux" শব্দটা কঠোর অর্থে শুধু **kernel**-কে বোঝায়। আমরা দৈনন্দিন জীবনে যাকে "লিনাক্স" বলি (Ubuntu, Fedora ইত্যাদি), সেটা হলো kernel + GNU টুল + লাইব্রেরি + অন্যান্য সফটওয়্যারের একটা পূর্ণ প্যাকেজ — যাকে বলে **distribution** বা **distro**। এই পার্থক্য পরের অধ্যায়গুলোতে বারবার কাজে লাগবে। গভীর আর্কিটেকচার আছে অধ্যায় ২-এ।

Kernel-এর ধরন (একনজরে)

ধরন	মানে	উদাহরণ
Monolithic kernel	প্রায় সব সার্ভিস (driver, file system, network) কার্নেলের ভেতরেই একসাথে চলে	**Linux**, পুরোনো Unix
Microkernel	কার্নেল খুব ছোট; driver ও সার্ভিস আলাদা user-space প্রসেস হিসেবে চলে	MINIX 3, QNX, seL4
Hybrid kernel	দুইয়ের মিশ্রণ	Windows NT, macOS-এর XNU

লিনাক্স monolithic হলেও **loadable kernel module** সমর্থন করে — মানে চলন্ত অবস্থায় driver যোগ বা সরানো যায় (বিস্তারিত অধ্যায় ৪৭)।

১.৩ Unix-এর ইতিহাস — যেখান থেকে সব শুরু

লিনাক্স বুঝতে হলে আগে Unix বুঝতে হবে, কারণ লিনাক্স হলো Unix-এর "আত্মিক উত্তরসূরি" — Unix-এর একটা লাইন কোডও লিনাক্সে নেই, কিন্তু চিন্তাধারা, কমান্ড আর ডিজাইন প্রায় একই।

Multics থেকে Unix

১৯৬০-এর দশকে MIT, General Electric আর AT&T-এর **Bell Labs** মিলে **Multics** নামে একটা উচ্চাভিলাষী OS বানাচ্ছিল। প্রজেক্টটা অত্যন্ত জটিল ও ধীর হয়ে পড়ায় ১৯৬৯ সালে Bell Labs সরে আসে।

Bell Labs-এর দুই প্রোগ্রামার **Ken Thompson** ও **Dennis Ritchie** Multics-এর ভালো ধারণাগুলো রেখে অনেক সহজ একটা সিস্টেম বানাতে চাইলেন। ১৯৬৯ সালে Thompson একটা পুরোনো **PDP-7** কম্পিউটারে প্রথম সংস্করণ লেখেন। নামটা এসেছে Multics নিয়ে একটা শব্দ-খেলা থেকে: "Multi" এর জায়গায় "Uni" — প্রথমে **UNICS**, পরে **Unix**।

C ভাষা — Unix-এর বিপ্লব

প্রথম দিকে Unix লেখা হয়েছিল **assembly language**-এ, যা নির্দিষ্ট একটা যন্ত্রের সাথে বাঁধা। Dennis Ritchie তখন **C** প্রোগ্রামিং ভাষা তৈরি করেন, আর **১৯৭৩ সালে** Unix-এর কার্নেল প্রায় পুরোটাই C-তে নতুন করে লেখা হয়। এর ফল ছিল বিপ্লবী: Unix এখন নতুন কোনো কম্পিউটারে নিয়ে যেতে শুধু C compiler বানালেই চলত, পুরো OS আবার লিখতে হতো না। একে বলে **portability** (বহনযোগ্যতা)।

> 💡 **টিপস:** আজও লিনাক্স কার্নেলের বেশিরভাগ কোড C-তে লেখা। ২০২২ সালের শেষ দিকে (kernel 6.1) থেকে কিছু অংশে **Rust** ভাষাও যোগ হওয়া শুরু হয়েছে।

Unix ছড়িয়ে পড়ে — BSD আর বাণিজ্যিক Unix

আইনি কারণে AT&T তখন কম্পিউটার ব্যবসা করতে পারত না, তাই তারা খুব কম খরচে Unix-এর সোর্স কোড বিশ্ববিদ্যালয়গুলোকে দিত। **University of California, Berkeley** এতে অনেক উন্নতি করে বানায় **BSD (Berkeley Software Distribution)** — যেখানে TCP/IP নেটওয়ার্কিং, `vi` এডিটর, C shell ইত্যাদি এসেছে। আজকের **FreeBSD, OpenBSD, NetBSD** — এবং আংশিকভাবে **macOS** — BSD-র বংশধর।

পরে AT&T-এর ওপর থেকে নিষেধাজ্ঞা উঠলে Unix বাণিজ্যিক পণ্য হয়ে যায় এবং দামি লাইসেন্স লাগতে শুরু করে। বিভিন্ন কোম্পানি নিজস্ব Unix বানায়: Sun-এর **SunOS/Solaris**, IBM-এর **AIX**, HP-এর **HP-UX** ইত্যাদি। প্রত্যেকটা একটু আলাদা — এই সময়টাকে বলে "**Unix wars**"। এই বিশৃঙ্খলা কমাতে আসে **POSIX** স্ট্যান্ডার্ড (১৯৮৮ থেকে) — যা বলে দেয় একটা Unix-সদৃশ সিস্টেমে কোন কমান্ড ও system call কেমন আচরণ করবে। লিনাক্স মোটামুটি POSIX মেনে চলে, তাই Unix-এ শেখা জ্ঞান লিনাক্সে খাটে, আর উল্টোটাও।

টাইমলাইন (একনজরে)

```
1969  Ken Thompson ও Dennis Ritchie — Bell Labs-এ Unix শুরু (PDP-7)
1972  C ভাষা (Dennis Ritchie)
1973  Unix কার্নেল C-তে নতুন করে লেখা
1977  BSD-র প্রথম রিলিজ (Berkeley)
1983  Richard Stallman — GNU প্রজেক্ট ঘোষণা (২৭ সেপ্টেম্বর)
1985  Free Software Foundation (FSF) প্রতিষ্ঠা
1987  Andrew Tanenbaum — MINIX (শিক্ষার জন্য Unix-সদৃশ OS)
1988  POSIX স্ট্যান্ডার্ড
```

```
1989  GNU GPL version 1
1991  Linus Torvalds — Linux ঘোষণা (২৫ আগস্ট), 0.01 রিলিজ (সেপ্টেম্বর)
      GNU GPL version 2
1992  Linux GPLv2-এর অধীনে আসে
1993  Slackware ও Debian — প্রথম দিকের distro
1994  Linux 1.0
1996  Tux পেঙ্গুইন মাসকট
1998  "Open Source" শব্দের জন্ম, Open Source Initiative (OSI)
2004  Ubuntu-র প্রথম রিলিজ
2005  Git — Linus Torvalds নিজেই বানালেন
2007  GNU GPL version 3
2008  Android-এর প্রথম ফোন (Linux kernel-এর ওপর)
2011  Linux 3.0 (২০ বছর পূর্তি)
2017  বিশ্বের TOP500 সুপারকম্পিউটারের ৫০০টিই লিনাক্সে
2022  Linux 6.x সিরিজ; কার্নেলে Rust-এর প্রবেশ
```

১.৪ Unix দর্শন (Unix Philosophy)

Unix শুধু একটা OS না, একটা চিন্তাধারা। Doug McIlroy (Unix pipe-এর উদ্ভাবক) এটাকে এভাবে বলেছিলেন:

1. **একটা প্রোগ্রাম একটাই কাজ করবে, এবং সেটা খুব ভালোভাবে করবে।**
2. **প্রোগ্রামগুলো এমনভাবে লেখো যাতে তারা একসাথে কাজ করতে পারে।**
3. **টেক্সট স্ট্রিম ব্যবহার করো, কারণ সেটাই সর্বজনীন ইন্টারফেস।**

এর সাথে আরেকটা বিখ্যাত নীতি: **"Everything is a file"** — লিনাক্সে সাধারণ ফাইল তো ফাইলই, কিন্তু হার্ডডিস্ক (`/dev/sda`), টার্মিনাল (`/dev/tty`), এমনকি চলন্ত প্রসেসের তথ্যও (`/proc/1234/`) ফাইলের মতো করে পড়া যায়।

উপমা দিই: লেগো ব্লক। প্রতিটা ব্লক ছোট আর সরল, কিন্তু জোড়া লাগিয়ে তুমি বাড়ি, গাড়ি, রকেট — যা খুশি বানাতে পারো। লিনাক্স কমান্ডও তাই:

```
$ cat access.log | grep "404" | wc -l
137
```

এখানে তিনটা ছোট প্রোগ্রাম `|` (pipe) দিয়ে জোড়া লাগানো হয়েছে: `cat` ফাইল পড়ে, `grep` শুধু "404" থাকা লাইন বাছে, `wc -l` লাইন গোনে। কেউ "৪০৪ এরর গোনার প্রোগ্রাম" আলাদা করে লেখেনি — আমরা লেগো জোড়া দিয়েছি। Pipe-এর পুরো কারিগরি শিখবে অধ্যায় ১৯-এ।

> 📝 **মনে রাখো:** লিনাক্স শেখা মানে শত শত কমান্ড মুখস্থ করা না। মানে হলো অল্প কিছু ছোট টুল ভালো করে চেনা আর সেগুলো জোড়া লাগানোর কৌশল শেখা।

১.৫ Richard Stallman ও GNU প্রজেক্ট

"Free" মানে "বিনামূল্যে" না

১৯৮০-র দশকের শুরুতে MIT-র প্রোগ্রামার **Richard M. Stallman (RMS)** দেখলেন সফটওয়্যার ক্রমশ "বন্ধ" হয়ে যাচ্ছে — সোর্স কোড দেখা যায় না, বদলানো যায় না, শেয়ার করা বেআইনি। একটা বিখ্যাত ঘটনা: MIT ল্যাবের একটা প্রিন্টার বারবার জ্যাম হতো, কিন্তু নতুন ড্রাইভারের সোর্স কোড না পাওয়ায় সমস্যাটা ঠিক করা যাচ্ছিল না।

এর প্রতিক্রিয়ায় **২৭ সেপ্টেম্বর ১৯৮৩** সালে Stallman ঘোষণা দিলেন **GNU প্রজেক্টের** — লক্ষ্য একটা সম্পূর্ণ **free** Unix-সদৃশ অপারেটিং সিস্টেম বানানো। **GNU** নামটা একটা recursive acronym (নিজের মধ্যেই নিজেকে ধারণ করা সংক্ষিপ্ত রূপ): **"GNU's Not Unix"** — অর্থাৎ এটা Unix-এর মতো, কিন্তু Unix-এর কোনো কোড নেই।

১৯৮৫ সালে তিনি প্রতিষ্ঠা করেন **Free Software Foundation (FSF)**।

Stallman-এর বিখ্যাত কথা: *"Free as in free speech, not as in free beer."* — মানে "free" শব্দটা **স্বাধীনতা** বোঝায়, **দাম** না। free software বিক্রিও করা যায়।

ফ্রি সফটওয়্যারের চারটি স্বাধীনতা

FSF-এর সংজ্ঞায় (গোনা শুরু শূন্য থেকে — প্রোগ্রামারদের অভ্যাস!):

স্বাধীনতা	মানে
Freedom 0	যেকোনো উদ্দেশ্যে প্রোগ্রাম চালানোর স্বাধীনতা
Freedom 1	প্রোগ্রাম কীভাবে কাজ করে তা পড়া এবং নিজের মতো বদলানোর স্বাধীনতা (এজন্য সোর্স কোড পাওয়া জরুরি)
Freedom 2	কপি বানিয়ে অন্যকে বিতরণের স্বাধীনতা
Freedom 3	নিজের বদলানো সংস্করণ অন্যদের বিতরণের স্বাধীনতা

GNU কী কী বানাল

১৯৯০ সালের মধ্যে GNU প্রজেক্ট একটা OS-এর প্রায় সব অংশ বানিয়ে ফেলেছিল:

GNU টুল	কাজ
GCC (GNU Compiler Collection)	C/C++ ইত্যাদি কোড compile করা
glibc (GNU C Library)	প্রোগ্রাম আর কার্নেলের মাঝের মূল লাইব্রেরি
Bash (Bourne Again SHell)	কমান্ড লাইন শেল — এই বইয়ের প্রধান হাতিয়ার
GNU Coreutils	`ls`, `cp`, `mv`, `rm`, `cat`, `mkdir`, `chmod` ...
GNU Emacs	টেক্সট এডিটর
grep, sed, gawk, tar, gzip, make, gdb	টেক্সট প্রসেসিং, আর্কাইভ, বিল্ড, ডিবাগ

কিন্তু একটা জিনিস বাকি ছিল — **kernel**। GNU-র নিজস্ব কার্নেল **GNU Hurd** (microkernel ডিজাইন) তৈরির কাজ ১৯৯০ সালে শুরু হলেও তা অত্যন্ত ধীরে এগোয় এবং আজও সাধারণ ব্যবহারের উপযোগী হয়নি। GNU-র হাতে ছিল পুরো গাড়ি — শুধু ইঞ্জিন নেই।

১.৬ Linus Torvalds ও লিনাক্সের জন্ম

একজন ছাত্রের "শখের প্রজেক্ট"

Linus Torvalds তখন ফিনল্যান্ডের University of Helsinki-র ২১ বছরের ছাত্র। তিনি একটা নতুন **Intel 80386** PC কিনেছিলেন এবং **MINIX** ব্যবহার করছিলেন — অধ্যাপক Andrew Tanenbaum-এর লেখা শিক্ষামূলক Unix-সদৃশ OS। কিন্তু MINIX-এর লাইসেন্স ও সীমাবদ্ধতায় তিনি সন্তুষ্ট ছিলেন না। তাই নিজেই 386-এর বৈশিষ্ট্য কাজে লাগিয়ে একটা terminal emulator লিখতে শুরু করলেন, যা ধীরে ধীরে একটা কার্নেলে পরিণত হলো।

২৫ আগস্ট ১৯৯১ সালে তিনি `comp.os.minix` নিউজগ্রুপে সেই বিখ্যাত পোস্ট দিলেন:

> "I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones."

যে মানুষটা বলেছিলেন "বড় আর প্রফেশনাল হবে না", তাঁর সেই কার্নেল আজ পৃথিবীর বেশিরভাগ সার্ভার, সব সুপারকম্পিউটার আর কোটি কোটি ফোন চালায়।

সেপ্টেম্বর ১৯৯১-এ প্রথম কোড (**version 0.01**) প্রকাশ পায়। Linus প্রথমে এর নাম রাখতে চেয়েছিলেন "Freax" (free + freak + x); কিন্তু FTP সার্ভারের অ্যাডমিন Ari Lemmke ফোল্ডারের নাম দিয়ে দেন **"linux"** — নামটাই থেকে যায়।

লাইসেন্স বদল — সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত

প্রথমে লিনাক্সের লাইসেন্স ছিল নিজস্ব, যাতে বাণিজ্যিক বিতরণ নিষিদ্ধ ছিল। ১৯৯২ সালে (version 0.12) Linus লিনাক্সকে **GNU GPL version 2**-এর অধীনে নিয়ে আসেন। পরে তিনি বলেছিলেন, এটাই ছিল তাঁর নেওয়া সবচেয়ে ভালো সিদ্ধান্তগুলোর একটা। কারণ:

1. GPL নিশ্চিত করল কেউ লিনাক্স নিয়ে বন্ধ (closed) সংস্করণ বানিয়ে উন্নতিগুলো লুকিয়ে রাখতে পারবে না।
2. GNU টুলগুলোর সাথে লিনাক্স মিলে প্রথমবারের মতো একটা **সম্পূর্ণ free অপারেটিং সিস্টেম** তৈরি হলো।

Tanenbaum–Torvalds বিতর্ক

১৯৯২ সালের শুরুতে Tanenbaum পোস্ট করলেন "LINUX is obsolete" — তাঁর যুক্তি ছিল monolithic kernel পুরোনো ধারণা, microkernel-ই ভবিষ্যৎ। Linus বাস্তবসম্মত দিক থেকে monolithic ডিজাইনের পক্ষে

যুক্তি দেন। এই বিতর্ক অপারেটিং সিস্টেম ডিজাইনের ইতিহাসে বিখ্যাত হয়ে আছে — আর বাস্তবে monolithic লিনাক্স ব্যাপক সফল হয়েছে।

লিনাক্স কীভাবে তৈরি হয় আজ

```
হাজার হাজার ডেভেলপার (Intel, Google, Red Hat, Meta, AMD, স্বেচ্ছাসেবী ...)
              |   প্যাচ (patch) পাঠায় mailing list-এ
              v
Subsystem maintainers  (networking, filesystems, drivers, mm ...)
              |   রিভিউ করে নিজেদের git tree-তে নেয়
              v
Linus Torvalds  (mainline tree)  ---- merge window (~২ সপ্তাহ)
              |                          তারপর rc1, rc2 ... rc7/rc8 (সাপ্তাহিক)
              v
নতুন কার্নেল রিলিজ – মোটামুটি প্রতি ৯–১০ সপ্তাহে
              |
              v
Stable / LTS maintainers  ---> Distro-রা (Ubuntu, Fedora, RHEL ...)
```

- এখনো Linus Torvalds-ই চূড়ান্ত সিদ্ধান্তদাতা — তাঁকে মজা করে বলা হয় **BDFL** ("Benevolent Dictator For Life")।
- প্রকল্পটির আর্থিক ও প্রশাসনিক সহায়তা দেয় **The Linux Foundation**।
- লিনাক্সের সোর্স কোড পরিচালনার জন্য ২০০৫ সালে Linus নিজেই বানান **Git** — যা আজ পৃথিবীর প্রায় সব সফটওয়্যার প্রজেক্টে ব্যবহার হয়।
- মাসকট **Tux** পেঙ্গুইন, আঁকেন Larry Ewing (১৯৯৬)।

১.৭ "Linux" নাকি "GNU/Linux"? — নামকরণ বিতর্ক

একটা সাধারণ Ubuntu বা Debian সিস্টেমে তুমি যা ব্যবহার করো:

```
+-----------------------------------------------------------+
|  Desktop / Apps:  GNOME, Firefox, LibreOffice  (অন্যান্য) |
+-----------------------------------------------------------+
|  Shell ও টুল:  bash, coreutils, grep, sed, tar  (GNU)     |
+-----------------------------------------------------------+
|  C Library:  glibc  (GNU)                                 |
+-----------------------------------------------------------+
|  Kernel:  Linux  (Linus Torvalds ও সম্প্রদায়)            |
+-----------------------------------------------------------+
```

FSF ও Stallman-এর যুক্তি: যেহেতু পুরো সিস্টেমের একটা বড় অংশ GNU প্রজেক্টের, তাই একে **"GNU/Linux"** বলা উচিত — নইলে GNU-র অবদান ও free software-এর আদর্শ আড়ালে পড়ে যায়।

অন্যদিকে বেশিরভাগ মানুষ সহজতার জন্য শুধু **"Linux"** বলে। আর কিছু লিনাক্স সিস্টেম আছে যেখানে GNU প্রায় নেই-ই:

- **Android** — Linux kernel, কিন্তু glibc-এর বদলে Google-এর **Bionic** লাইব্রেরি, আর GNU টুল প্রায় নেই।
- **Alpine Linux** — glibc-এর বদলে **musl**, আর coreutils-এর বদলে **BusyBox** (Docker image-এ খুব জনপ্রিয়)।

> 💡 **টিপস:** Debian প্রজেক্ট আনুষ্ঠানিকভাবে নিজেকে "Debian GNU/Linux" বলে। এই বইয়ে আমরা সুবিধার জন্য "লিনাক্স" লিখব — কিন্তু তুমি জানবে যে পর্দার আড়ালে GNU-র বিশাল অবদান আছে।

১.৮ ওপেন সোর্স ও সফটওয়্যার লাইসেন্স

Free Software বনাম Open Source

১৯৯৮ সালে Netscape তাদের ব্রাউজারের কোড উন্মুক্ত করার পর কিছু মানুষ (Eric S. Raymond, Bruce Perens প্রমুখ) ভাবলেন "free" শব্দটা ব্যবসায়ীদের কাছে বিভ্রান্তিকর ("ফ্রি মানে কি টাকা নেই?")। তাঁরা নতুন শব্দ আনলেন **"Open Source"** এবং প্রতিষ্ঠা করলেন **Open Source Initiative (OSI)**।

দিক	Free Software (FSF)	Open Source (OSI)
মূল জোর	ব্যবহারকারীর **নৈতিক স্বাধীনতা**	**বাস্তব সুবিধা** — ভালো মানের কোড, দ্রুত উন্নয়ন
ভাষা	"এটা অধিকার"	"এটা ভালো ডেভেলপমেন্ট মডেল"
বাস্তবে	প্রায় সব free software-ই open source, এবং প্রায় সব open source-ই free software	

দুই মতকে একসাথে প্রায়ই বলা হয় **FOSS** (Free and Open Source Software) বা **FLOSS**।

ক্লোজড সোর্স (Proprietary) বনাম ওপেন সোর্স

উপমা: **রান্নার রেসিপি**।

- **Proprietary software** = রেস্টুরেন্টের গোপন রেসিপি। খাবার কিনে খেতে পারবে, কিন্তু কীভাবে রান্না হয়েছে জানবে না, বদলাতেও পারবে না।
- **Open source software** = রেসিপি বইয়ে ছাপা। পড়তে পারবে, নিজের মতো লবণ কমিয়ে রাঁধতে পারবে, বন্ধুকে কপি দিতে পারবে।

প্রধান লাইসেন্সগুলো

লাইসেন্স দুই বড় পরিবারে ভাগ হয়:

১. Copyleft — "নিলে, ফেরত দিতে হবে"। তুমি কোড বদলে **বিতরণ** করলে, তোমার বদলানো কোডও একই লাইসেন্সে সোর্সসহ দিতে হবে। ("Copyleft" শব্দটা "copyright"-এর ওপর একটা শব্দ-খেলা।)

২. Permissive — "যা খুশি করো, শুধু কৃতিত্ব দিও"। বদলানো কোড বন্ধ (closed) করে বিক্রি করলেও সমস্যা নেই।

লাইসেন্স	ধরন	মূল শর্ত	উদাহরণ
GPL v2	Strong copyleft	বিতরণ করলে সোর্স দিতে হবে, একই লাইসেন্সে	**Linux kernel** (GPLv2-only)
GPL v3	Strong copyleft	GPLv2 + পেটেন্ট সুরক্ষা + "tivoization" বিরোধী শর্ত (ডিভাইসে বদলানো সফটওয়্যার চালাতে দিতে হবে)	Bash, GCC, অনেক GNU টুল
LGPL	Weak copyleft	লাইব্রেরি বদলালে তা শেয়ার করতে হবে, কিন্তু লাইব্রেরি *ব্যবহারকারী* প্রোগ্রাম closed হতে পারে	glibc
AGPL v3	Network copyleft	ওয়েবে সার্ভিস হিসেবে চালালেও ব্যবহারকারীদের সোর্স দিতে হবে	কিছু ডেটাবেস ও ওয়েব অ্যাপ
MIT	Permissive	শুধু কপিরাইট নোটিশ রাখতে হবে	Node.js, jQuery, X11-এর বড় অংশ
BSD (2/3-clause)	Permissive	নোটিশ রাখো; 3-clause-এ লেখকের নাম প্রচারে ব্যবহার নিষেধ	FreeBSD, OpenBSD-এর বড় অংশ
Apache 2.0	Permissive	নোটিশ + স্পষ্ট পেটেন্ট অনুমতি	Kubernetes, Android-এর user space-এর বড় অংশ
MPL 2.0	File-level copyleft	বদলানো *ফাইলগুলো* উন্মুক্ত রাখতে হবে	Firefox

> ⚠️ **সতর্কতা:** GPL-এর শর্ত চালু হয় **বিতরণ (distribution)** করলে। তুমি নিজের কম্পিউটারে GPL কোড বদলে চালালে কাউকে সোর্স দেওয়ার বাধ্যবাধকতা নেই। কিন্তু অফিসের প্রোডাক্টে GPL কোড ঢুকিয়ে গ্রাহককে বিক্রি করলে সোর্স দিতেই হবে — এটা না জেনে অনেক কোম্পানি আইনি ঝামেলায় পড়েছে। চাকরিতে লাইসেন্স নিয়ে সন্দেহ হলে আইন বিভাগকে জিজ্ঞেস করো।

> 📝 **মনে রাখো:** Linux kernel **GPLv2-only** — "or any later version" লেখা নেই। তাই GPLv3 আসার পরও কার্নেল v2-তেই আছে। Linus GPLv3-এর কিছু শর্তের (বিশেষত হার্ডওয়্যার লক নিয়ে) বিরোধী ছিলেন।

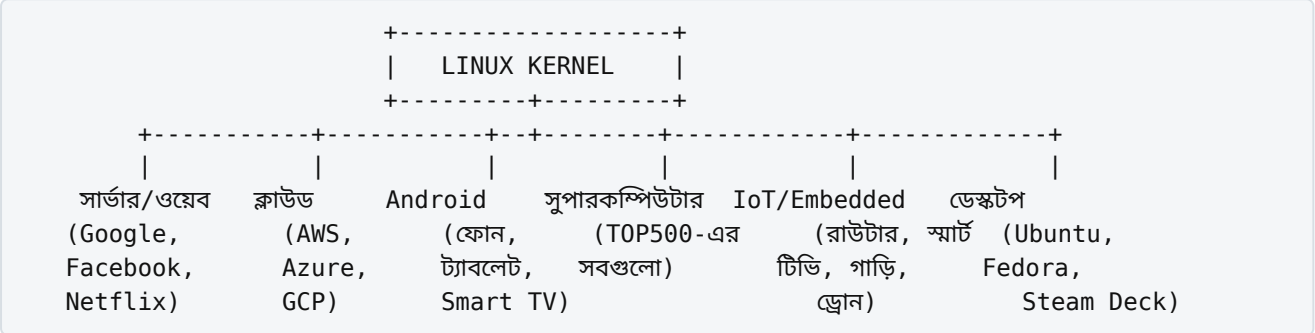
ওপেন সোর্স কেন জিতল?

- **"Given enough eyeballs, all bugs are shallow"** (Linus's Law, Eric Raymond-এর দেওয়া নাম) — অনেক মানুষ কোড দেখলে বাগ দ্রুত ধরা পড়ে।
- কোম্পানিগুলো একই কাজ বারবার না করে একসাথে একটা ভালো জিনিস বানায় (Intel, AMD, Google, Microsoft সবাই লিনাক্সে কোড দেয়)।

- কোনো একটা কোম্পানির ওপর নির্ভরতা (**vendor lock-in**) থাকে না।
- নিরাপত্তা: কোড দেখা যায়, তাই পেছনের দরজা (backdoor) লুকানো কঠিন — যদিও অসম্ভব না।

১.৯ লিনাক্স কোথায় কোথায় ব্যবহার হয়?

তুমি হয়তো ভাবছ "আমার পরিচিত কেউ তো লিনাক্স চালায় না!" আসলে তুমি প্রতিদিন হয়তো শতবার লিনাক্স ব্যবহার করো — শুধু টের পাও না।



সার্ভার ও ইন্টারনেট

তুমি যখন Google-এ সার্চ করো, YouTube দেখো, Facebook স্ক্রল করো, Wikipedia পড়ো — অপর প্রান্তের সার্ভারগুলোর বেশিরভাগই লিনাক্স চালায়। বিশ্বের ওয়েব সার্ভারের সিংহভাগই Unix-সদৃশ OS, যার মধ্যে লিনাক্সই প্রধান। এর কারণ: স্থিতিশীলতা (মাসের পর মাস রিবুট ছাড়া চলে), লাইসেন্স খরচ নেই, রিমোট থেকে কমান্ড লাইনে সহজে নিয়ন্ত্রণ, আর স্বয়ংক্রিয় করা (automation) সহজ। সার্ভার সেটআপ শিখবে অধ্যায় ৪০-এ।

ক্লাউড কম্পিউটিং

Amazon Web Services (AWS), Microsoft Azure, Google Cloud — সবখানে ভার্চুয়াল মেশিনের বড় অংশ লিনাক্স চালায়। Microsoft-এর Azure-এও লিনাক্স VM-এর সংখ্যা Windows-কে ছাড়িয়ে গেছে বলে Microsoft নিজেই জানিয়েছে। আর আধুনিক ক্লাউডের ভিত্তি **container** প্রযুক্তি (Docker, Kubernetes) সরাসরি লিনাক্স কার্নেলের **namespaces** ও **cgroups** বৈশিষ্ট্যের ওপর দাঁড়িয়ে (অধ্যায় ৪৮)।

Android

পৃথিবীর সবচেয়ে বেশি ব্যবহৃত মোবাইল OS **Android** লিনাক্স কার্নেলের ওপর চলে। ফলে তোমার পকেটের ফোনটাই সম্ভবত তোমার জীবনের প্রথম লিনাক্স ডিভাইস! (তবে Android-এর user space সাধারণ GNU/Linux-এর মতো না — আগেই বলেছি।) ChromeOS-ও লিনাক্স-ভিত্তিক।

সুপারকম্পিউটার

বিশ্বের দ্রুততম ৫০০টি সুপারকম্পিউটারের তালিকা **TOP500**-এ **নভেম্বর ২০১৭** থেকে **৫০০টির সবগুলোই** লিনাক্স চালায়। আবহাওয়ার পূর্বাভাস, ওষুধ আবিষ্কার, পারমাণবিক গবেষণা, AI মডেল ট্রেনিং — সবই লিনাক্সে।

IoT ও Embedded সিস্টেম

Embedded system মানে কোনো যন্ত্রের ভেতরে লুকানো ছোট কম্পিউটার। Wi-Fi রাউটার (OpenWrt), স্মার্ট টিভি, সেট-টপ বক্স, গাড়ির ইনফোটেইনমেন্ট সিস্টেম (Automotive Grade Linux), স্মার্ট ঘড়ি, শিল্প-কারখানার যন্ত্র, **Raspberry Pi** — সবখানে লিনাক্স। ছোট ডিভাইসের জন্য কাস্টম লিনাক্স বানাতে **Yocto Project** ও **Buildroot** ব্যবহৃত হয়।

মহাকাশ ও বিজ্ঞান

SpaceX-এর রকেট ও Dragon মহাকাশযানের ফ্লাইট কম্পিউটারে লিনাক্স ব্যবহারের কথা জানা যায়; NASA-র মঙ্গল হেলিকপ্টার **Ingenuity** লিনাক্সে চলত। CERN-এর Large Hadron Collider-এর কম্পিউটিংও লিনাক্স-নির্ভর।

ডেস্কটপ ও গেমিং

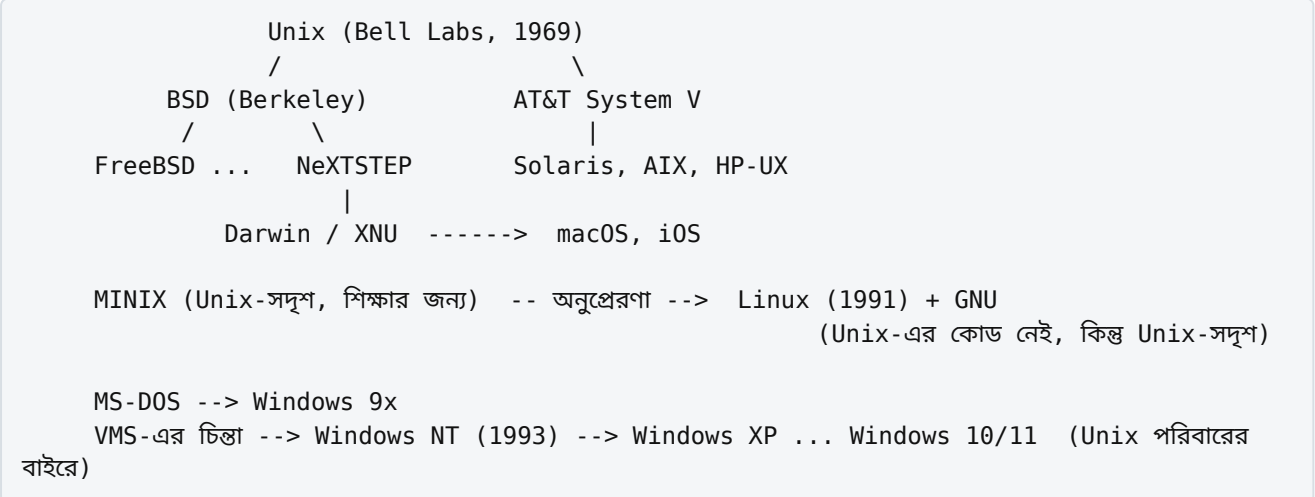
ডেস্কটপে লিনাক্সের অংশ এখনো কয়েক শতাংশ, তবে বাড়ছে। Valve-এর **Steam Deck** গেমিং ডিভাইস লিনাক্স-ভিত্তিক **SteamOS** চালায়, আর **Proton** দিয়ে অনেক Windows গেম লিনাক্সে চলে। প্রোগ্রামার ও ডেভেলপারদের মধ্যে লিনাক্স ডেস্কটপ খুব জনপ্রিয়। Windows-এও এখন **WSL2** (Windows Subsystem for Linux) দিয়ে আসল লিনাক্স কার্নেল চালানো যায়।

বাংলাদেশ ও ভারতে

ব্যাংক, টেলিকম কোম্পানি, ISP, সরকারি ডেটা সেন্টার, সফটওয়্যার কোম্পানি — সবখানে সার্ভার মানেই সাধারণত লিনাক্স। দেশি সফটওয়্যার কোম্পানির ব্যাকএন্ড, হোস্টিং কোম্পানির শেয়ার্ড হোস্টিং (cPanel ইত্যাদি) — প্রায় সবই লিনাক্সে।

১.১০ Windows ও macOS-এর সাথে তুলনা

পারিবারিক বৃক্ষ



- **macOS** একটা আনুষ্ঠানিকভাবে সার্টিফাইড **UNIX** (The Open Group-এর UNIX 03 সার্টিফিকেশন আছে) — কিন্তু এটা **লিনাক্স না**। এর কার্নেল **XNU** (Mach + BSD অংশ)।
- **Linux** সার্টিফাইড "UNIX" না (সার্টিফিকেশন ব্যয়বহুল ও এর দরকারও পড়েনি), কিন্তু আচরণে Unix-সদৃশ (**Unix-like**)।
- **Windows** সম্পূর্ণ আলাদা বংশের — NT kernel।

এই কারণে macOS-এর Terminal-এ শেখা `ls`, `cd`, `grep` লিনাক্সেও প্রায় একইভাবে চলে (যদিও macOS-এ অনেক টুল BSD সংস্করণের, তাই কিছু অপশন আলাদা), কিন্তু Windows-এর CMD-তে চলে না।

বিস্তারিত তুলনা

বৈশিষ্ট্য	Linux	Windows	macOS
কার্নেল	Linux (monolithic, modular)	Windows NT (hybrid)	XNU (hybrid)
সোর্স কোড	উন্মুক্ত (GPLv2)	বন্ধ (proprietary)	বেশিরভাগ বন্ধ; Darwin অংশ উন্মুক্ত
দাম	সাধারণত বিনামূল্যে; এন্টারপ্রাইজে সাপোর্টের জন্য সাবস্ক্রিপশন (RHEL, SUSE, Ubuntu Pro)	লাইসেন্স কিনতে হয়	Apple হার্ডওয়্যারের সাথে আসে
হার্ডওয়্যার	প্রায় সব কিছুতে: x86, ARM, RISC-V, PowerPC, IBM Z মেইনফ্রেম	মূলত x86-64 ও ARM	শুধু Apple-এর Mac
ফাইল সিস্টেম	ext4, XFS, Btrfs ইত্যাদি	NTFS, ReFS	APFS
পথ (path) বিভাজক	`/` (যেমন `/home/rahim`)	`\` (যেমন `C:\Users\Rahim`)	`/`
ড্রাইভ লেটার	নেই — একটাই গাছ, শুরু `/` থেকে	`C:`, `D:` ...	নেই
নামের বড়-ছোট হাতের অক্ষর	**case-sensitive** (`File.txt` ≠ `file.txt`)	সাধারণত case-insensitive	ডিফল্টে case-insensitive
সফটওয়্যার ইনস্টল	প্যাকেজ ম্যানেজার (`apt`, `dnf`) — এক কমান্ডে	`.exe` / `.msi`, Microsoft Store, `winget`	App Store, `.dmg`, Homebrew
আপডেট	পুরো সিস্টেম + সব অ্যাপ একসাথে; বেশিরভাগ ক্ষেত্রে রিবুট ছাড়া	প্রায়ই রিবুট দরকার	মাঝেমধ্যে রিবুট
শেল	Bash (ডিফল্ট বেশিরভাগ distro-তে), zsh, fish	PowerShell, CMD	zsh (Catalina থেকে ডিফল্ট)
GUI	বদলানো যায় — GNOME, KDE Plasma, XFCE ... (অধ্যায় ৪)	একটাই	একটাই
ভাইরাস/ম্যালওয়্যার	ডেস্কটপে তুলনামূলক কম; তবে সার্ভারে আক্রমণ অনেক	সবচেয়ে বেশি লক্ষ্যবস্তু	মাঝামাঝি
সার্ভার বাজার	প্রধান	উল্লেখযোগ্য (বিশেষত Active Directory, .NET)	প্রায় নেই
গেমিং	বাড়ছে (Steam/Proton)	সবচেয়ে ভালো	সীমিত

বৈশিষ্ট্য	Linux	Windows	macOS
বাণিজ্যিক সফটওয়্যার	কিছু নেই (যেমন MS Office, Adobe Photoshop-এর নেটিভ সংস্করণ)	সবচেয়ে বেশি	ভালো
নিয়ন্ত্রণ	তোমার হাতে প্রায় সবকিছু	সীমিত	সীমিত

লিনাক্সে প্রথম দিন যা অবাক লাগতে পারে

Windows-এর অভ্যাস	লিনাক্সে কী
`C:\Program Files`	প্রোগ্রাম থাকে `/usr/bin`, `/usr/lib` ইত্যাদিতে (অধ্যায় ৭)
ফাইলের শেষে `.exe` দেখে বোঝা যায় প্রোগ্রাম	এক্সটেনশন বাধ্যতামূলক না; **execute permission** ঠিক করে কোনটা চালানো যাবে (অধ্যায় ১৬)
Administrator	**root** ব্যবহারকারী ও `sudo` (অধ্যায় ১৮)
Task Manager	`top`, `htop`, `ps`, `kill` (অধ্যায় ২৪)
Services.msc	`systemctl` (অধ্যায় ২৬)
Event Viewer	`journalctl`, `/var/log` (অধ্যায় ২৮)
Control Panel → Programs	`apt`, `dnf` (অধ্যায় ৩০)
Disk Management	`lsblk`, `fdisk`, `parted`, `mount` (অধ্যায় ৩২)
Task Scheduler	`cron`, `systemd timers` (অধ্যায় ২৯)
Remote Desktop	`ssh` (অধ্যায় ৩৮)
লুকানো ফাইল = attribute	নামের শুরুতে `.` থাকলেই লুকানো (যেমন `.bashrc`)

> ⚠️ **সতর্কতা:** লিনাক্সে কোনো "Recycle Bin" ছাড়াই কমান্ড লাইনের `rm` ফাইল মুছে দেয় — ফেরত আনার সহজ রাস্তা নেই। আর `root` হিসেবে সিস্টেম কোনো প্রশ্ন না করেই তোমার কথা মেনে নেয়, ভুল হলেও। "লিনাক্স ধরে নেয় তুমি জানো তুমি কী করছ।" তাই এই বইয়ের প্রতিটা ⚠️ মনোযোগ দিয়ে পড়বে।

লিনাক্সের সীমাবদ্ধতা — সৎভাবে বলি

- কিছু বাণিজ্যিক সফটওয়্যার (MS Office ডেস্কটপ, Adobe Creative Cloud, কিছু ইঞ্জিনিয়ারিং টুল) নেটিভভাবে নেই — বিকল্প (LibreOffice, GIMP, Inkscape, Kdenlive, Blender) বা ওয়েব সংস্করণ ব্যবহার করতে হয়।
- কিছু নতুন বা বিশেষ হার্ডওয়্যারের (কিছু Wi-Fi চিপ, ফিঙ্গারপ্রিন্ট রিডার) driver পেতে সমস্যা হতে পারে।
- অনেক distro থাকায় শুরুতে বিভ্রান্তি — অধ্যায় ৩ এই সমস্যা মেটাবে।
- শেখার শুরুটা একটু খাড়া — যেটা এই বই সহজ করতে চায়।

১.১১ জীবনের প্রথম কয়েকটা লিনাক্স কমান্ড

এখনো লিনাক্স ইনস্টল না করে থাকলে চিন্তা নেই — অধ্যায় ৩-এ ইনস্টলেশন আছে। আপাতত তুমি এগুলো পরখ করতে পারো:

- Windows 10/11-এ PowerShell (Administrator) খুলে `wsl --install` দিয়ে Ubuntu বসিয়ে
- VirtualBox/VMware-এ ভার্চুয়াল মেশিনে
- Ubuntu-র Live USB থেকে (ইনস্টল না করেই)

টার্মিনাল খোলার পর তুমি এমন কিছু দেখবে:

```
rahim@laptop:~$
```

এটাকে বলে **prompt** (প্রম্পট) — শেল তোমার আদেশের অপেক্ষায় আছে। `rahim` = ব্যবহারকারীর নাম, `laptop` = কম্পিউটারের নাম (hostname), `~` = তুমি এখন নিজের home ফোল্ডারে, `$` = সাধারণ ব্যবহারকারী (root হলে `#` দেখায়)। এই বইয়ে সংক্ষেপে শুধু `$` লিখব। `$` -এর পরের অংশটা তুমি টাইপ করবে (`$` নিজে টাইপ করবে না), পরের লাইনগুলো হলো output। টার্মিনাল ও শেলের বিস্তারিত অধ্যায় ৫-এ।

> 💡 **টিপস:** তোমার কম্পিউটারের output হুবহু বইয়ের মতো হবে না — কার্নেল সংস্করণ, নাম, তারিখ আলাদা হবে। এটা স্বাভাবিক।

`whoami` — আমি কে?

কাজ: বর্তমান ব্যবহারকারীর নাম দেখায়।

```
$ whoami
rahim
```

`uname` — কোন কার্নেল চলছে?

কাজ: কার্নেল ও সিস্টেমের তথ্য দেখায় (uname = "Unix name")। **Syntax:** `uname [OPTION]...`

অপশন	লম্বা রূপ	কী দেখায়
`-s`	`--kernel-name`	কার্নেলের নাম (কোনো অপশন না দিলে এটাই দেখায়)
`-n`	`--nodename`	নেটওয়ার্কে কম্পিউটারের নাম (hostname)
`-r`	`--kernel-release`	কার্নেল রিলিজ সংস্করণ
`-v`	`--kernel-version`	কার্নেল বিল্ডের তথ্য ও তারিখ
`-m`	`--machine`	হার্ডওয়্যার আর্কিটেকচার (যেমন `x86_64`, `aarch64`)
`-o`	`--operating-system`	অপারেটিং সিস্টেম (সাধারণত `GNU/Linux`)

অপশন	লম্বা রূপ	কী দেখায়
`-a`	`--all`	সব তথ্য একসাথে

```
$ uname
Linux

$ uname -r
6.8.0-45-generic

$ uname -m
x86_64

$ uname -o
GNU/Linux

$ uname -a
Linux laptop 6.8.0-45-generic #45-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug 30 12:02:04 UTC
2024 x86_64 x86_64 x86_64 GNU/Linux
```

লক্ষ করো: `uname -o` নিজেই বলছে **`GNU/Linux`** — নামকরণ বিতর্কের ছোট্ট প্রমাণ!

কার্নেল সংস্করণ নম্বর পড়া:

```
   6 . 8 . 0 - 45 - generic
   |   |   |    |      |
   |   |   |    |      +-- distro-র কার্নেল ফ্লেভার (Ubuntu-র সাধারণ কার্নেল)
   |   |   |    +---------- distro-র নিজস্ব বিল্ড/প্যাচ নম্বর
   |   |   +--------------- upstream patch/stable নম্বর
   |   +------------------- minor সংস্করণ
   +----------------------- major সংস্করণ
```

Fedora-তে এমন দেখাতে পারে: `6.11.4-301.fc41.x86_64`, RHEL 9-এ: `5.14.0-427.13.1.el9_4.x86_64`। এন্টারপ্রাইজ distro (RHEL) পুরোনো major সংস্করণ রেখে তাতে নিরাপত্তা প্যাচ যোগ করে যায় (একে বলে **backporting**) — তাই RHEL 9-এ সংখ্যা "5.14" দেখে ভেবো না সেটা পুরোনো ও অনিরাপদ।

`cat /etc/os-release` — কোন distro চলছে?

কাজ: `cat` ফাইলের লেখা দেখায় (অধ্যায় ১০), আর `/etc/os-release` ফাইলে distro-র পরিচয় থাকে। এটা systemd যুগের সব আধুনিক distro-তে আছে, তাই সবচেয়ে নির্ভরযোগ্য উপায়।

```
$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.1 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
```

```
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

`ID_LIKE=debian` বলছে Ubuntu হলো Debian-পরিবারের। RHEL 9-এ পাবে `ID="rhel"` আর `ID_LIKE="fedora"`।

`hostnamectl` — সব একসাথে

কাজ: systemd-চালিত সিস্টেমে hostname, OS, কার্নেল, আর্কিটেকচার এক জায়গায় দেখায় (hostname বদলাতেও পারে — পরে শিখবে)।

```
$ hostnamectl
 Static hostname: laptop
       Icon name: computer-laptop
         Chassis: laptop 💻
      Machine ID: 3f1c9e0a7b2d4c51a8e6f0b9d2c7e413
         Boot ID: 9a8b7c6d5e4f40312a1b0c9d8e7f6a5b
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
    Architecture: x86-64
 Hardware Vendor: Lenovo
  Hardware Model: ThinkPad E14
```

(সংস্করণভেদে কিছু লাইন — যেমন Firmware Version — বেশি-কম দেখা যেতে পারে।)

> 📝 **মনে রাখো:** WSL বা কিছু Docker container-এ systemd চালু না থাকলে `hostnamectl` কাজ নাও করতে পারে। তখন `cat /etc/os-release` আর `uname -a` ব্যবহার করো।

`lsb_release` — পুরোনো কিন্তু প্রচলিত উপায়

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.1 LTS
Release:        24.04
Codename:       noble
```

Ubuntu/Debian-এ সাধারণত থাকে; Fedora/RHEL-এ অনেক সময় ইনস্টল করা থাকে না। তাই `/etc/os-release`-ই ভরসা।

সাহায্য কোথায় পাবে — শুরু থেকেই অভ্যাস করো

কমান্ড	কাজ
`man uname`	পূর্ণাঙ্গ ম্যানুয়াল পেজ (বের হতে `q`)
`uname --help`	সংক্ষিপ্ত সাহায্য (GNU টুলে প্রায় সবখানে আছে)
`whatis uname`	এক লাইনে বর্ণনা
`apropos kernel`	"kernel" শব্দ আছে এমন ম্যানুয়াল খোঁজা
`info coreutils`	GNU-র বিস্তারিত info ডকুমেন্টেশন

```
$ whatis uname
uname (1)            - print system information
```

`(1)` মানে ম্যানুয়ালের সেকশন ১ (সাধারণ ব্যবহারকারীর কমান্ড)। `man` নিয়ে আরও আছে অধ্যায় ৫ ও অধ্যায় ১২-এ।

১.১২ প্রথম দিনের কীবোর্ড শর্টকাট

পূর্ণ তালিকা আছে অধ্যায় ৬ ও পরিশিষ্ট ক-এ। আপাতত এগুলো জানলেই প্রথম দিন চলে যাবে:

শর্টকাট	কোথায়	কাজ
`Ctrl` + `Alt` + `T`	Ubuntu (GNOME), Linux Mint	নতুন টার্মিনাল খোলা (Fedora-তে ডিফল্টে নেই, Settings → Keyboard থেকে যোগ করা যায়)
`Super` (Windows কী)	GNOME	Activities — অ্যাপ খোঁজা
`Tab`	টার্মিনাল	কমান্ড/ফাইলের নাম auto-complete
`Tab` `Tab`	টার্মিনাল	সম্ভাব্য সব মিল দেখানো
`↑` / `↓`	টার্মিনাল	আগের/পরের কমান্ড
`Ctrl` + `C`	টার্মিনাল	চলন্ত কমান্ড থামানো (interrupt)
`Ctrl` + `L`	টার্মিনাল	স্ক্রিন পরিষ্কার (`clear`-এর মতো)
`Ctrl` + `D`	টার্মিনাল	ইনপুট শেষ / শেল থেকে বের হওয়া (`exit`-এর মতো)
`Ctrl` + `Shift` + `C` / `V`	টার্মিনাল	কপি / পেস্ট (টার্মিনালে শুধু `Ctrl+C` কপি করে না — কমান্ড থামায়!)
`q`	`man`, `less`	বের হওয়া
`Ctrl` + `Alt` + `F3` … `F6`	বেশিরভাগ distro	গ্রাফিক্যাল পরিবেশের বাইরে টেক্সট-মোড কনসোল (TTY)

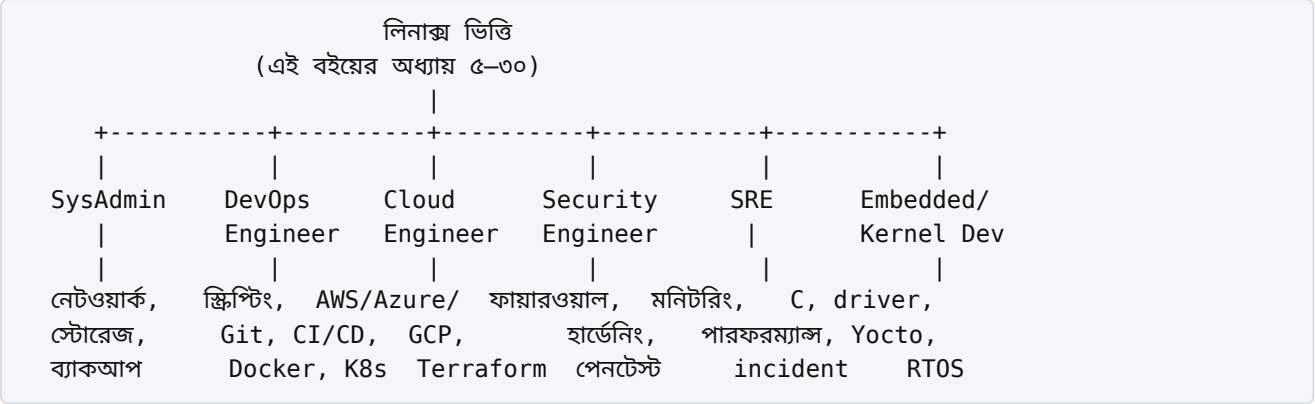
শর্টকাট	কোথায়	কাজ
`Ctrl` + `Alt` + `F1` বা `F2`	Ubuntu/Fedora (GDM)	সাধারণত আবার গ্রাফিক্যাল লগইন/ডেস্কটপে ফেরা (distro-ভেদে নম্বর ভিন্ন)

> ⚠️ **সতর্কতা:** Windows-এর অভ্যাসে টার্মিনালে `Ctrl+C` চাপলে কপি হবে না, বরং চলমান কাজ থেমে যাবে। কপির জন্য `Ctrl+Shift+C`, অথবা মাউস দিয়ে সিলেক্ট করে মাঝের বোতাম (middle-click) দিয়ে পেস্ট।

১.১৩ লিনাক্স কেন শিখবে? — ক্যারিয়ার

লিনাক্স জ্ঞান কেন চাকরির বাজারে মূল্যবান

যেহেতু সার্ভার, ক্লাউড আর কন্টেইনারের ভিত্তি লিনাক্স, তাই আইটি-র প্রায় সব "পেছনের দিকের" (backend/infra) চাকরিতে লিনাক্স জানা প্রায় বাধ্যতামূলক। এমনকি সাধারণ ওয়েব ডেভেলপারকেও ডেপ্লয় করার সময় লিনাক্স সার্ভারে SSH করে ঢুকতে হয়।



প্রধান ক্যারিয়ার পথ

পদ	কী করে	এই বইয়ের কোন অংশ সবচেয়ে জরুরি	পরে কী শিখবে
Linux System Administrator (SysAdmin)	সার্ভার ইনস্টল, ইউজার, ব্যাকআপ, আপডেট, সমস্যা সমাধান	অধ্যায় ১৫–১৮, ২৪–৩৫, ৪৫–৪৬	LDAP, ভার্চুয়ালাইজেশন, Ansible
DevOps Engineer	কোড থেকে প্রোডাকশন পর্যন্ত পুরো পাইপলাইন স্বয়ংক্রিয় করা	অধ্যায় ১৯–২৩, ৪১–৪৪, ৪৮	Git, CI/CD (GitHub Actions, Jenkins, GitLab CI), Docker, Kubernetes, Terraform
Cloud Engineer	AWS/Azure/GCP-তে অবকাঠামো ডিজাইন ও পরিচালনা	অধ্যায় ৩৬–৪০, ৪৮	ক্লাউড প্ল্যাটফর্মের নিজস্ব সার্টিফিকেশন, IaC

পদ	কী করে	এই বইয়ের কোন অংশ সবচেয়ে জরুরি	পরে কী শিখবে
Site Reliability Engineer (SRE)	বড় সিস্টেম যাতে সবসময় চালু ও দ্রুত থাকে	অধ্যায় ২৪, ২৬–২৮, ৪৬–৪৭	Prometheus, Grafana, প্রোগ্রামিং (Go/Python)
Security Engineer / Pentester	সিস্টেম সুরক্ষিত করা / দুর্বলতা খোঁজা	অধ্যায় ১৬–১৮, ৩৭–৩৯, ৪৫	Kali Linux, নেটওয়ার্ক সিকিউরিটি, SELinux/AppArmor গভীরে
Network Engineer	রাউটিং, ফায়ারওয়াল, VPN	অধ্যায় ৩৬–৩৯	CCNA-ধরনের নেটওয়ার্কিং জ্ঞান
Embedded / Kernel Developer	ডিভাইস, driver, কার্নেলে কোড	অধ্যায় ২, ২৭, ৩১, ৪৭	C, কম্পিউটার আর্কিটেকচার, Yocto/Buildroot
Backend / Data Engineer	সার্ভারে অ্যাপ ও ডেটা পাইপলাইন চালানো	অধ্যায় ১৯–২৫, ৩৮, ৪১–৪৪	ডেটাবেস, প্রোগ্রামিং

জনপ্রিয় সার্টিফিকেশন

সার্টিফিকেশন	প্রতিষ্ঠান	ধরন	কাদের জন্য
LPI Linux Essentials	Linux Professional Institute	MCQ	একদম শুরু
LPIC-1	Linux Professional Institute	MCQ (২টি পরীক্ষা)	জুনিয়র অ্যাডমিন, distro-নিরপেক্ষ
CompTIA Linux+	CompTIA	MCQ + পারফরম্যান্স-ভিত্তিক	জুনিয়র অ্যাডমিন
LFCS	Linux Foundation	হাতে-কলমে (performance-based)	অ্যাডমিন
RHCSA	Red Hat	হাতে-কলমে, RHEL-এ	এন্টারপ্রাইজ অ্যাডমিন — খুব সম্মানিত
RHCE	Red Hat	হাতে-কলমে, Ansible অটোমেশন	RHCSA-র পরের ধাপ
CKA / CKAD	CNCF / Linux Foundation	হাতে-কলমে Kubernetes	DevOps/Cloud

> 💡 **টিপস:** সার্টিফিকেট চাকরির দরজা খোলে, কিন্তু ইন্টারভিউতে টেকায় **হাতে-কলমে দক্ষতা**। তাই এই বইয়ের প্রতিটা "হাতে-কলমে অনুশীলন" নিজে টাইপ করে করবে। ইন্টারভিউ প্রস্তুতির প্রশ্ন আছে পরিশিষ্ট খ-এ।

ক্যারিয়ার ছাড়াও লিনাক্স শেখার কারণ

- **কম্পিউটার আসলে কীভাবে কাজ করে** তা বুঝবে — লিনাক্স কিছু লুকায় না।
- পুরোনো, কম শক্তিশালী কম্পিউটারেও হালকা distro (যেমন Xubuntu, Lubuntu, Linux Mint XFCE) চালিয়ে কাজ চালানো যায়।
- **গোপনীয়তা (privacy)** ও নিয়ন্ত্রণ — কী ডেটা কোথায় যাচ্ছে তুমি নিয়ন্ত্রণ করো।

- **প্রোগ্রামিং** শেখার জন্য সেরা পরিবেশ — compiler, interpreter, ডেটাবেস সব এক কমান্ডে ইনস্টল।
- **অটোমেশন**: যে কাজ রোজ ৩০ মিনিট লাগে, একটা ছোট স্ক্রিপ্ট দিয়ে তা ৩ সেকেন্ডে (অধ্যায় ৪১)।
- **ওপেন সোর্সে অবদান** রেখে আন্তর্জাতিক পরিচিতি ও পোর্টফোলিও তৈরি।

১.১৪ এই বই কীভাবে পড়বে

বইয়ের মানচিত্র

```
পর্ব ১: ভিত্তি              অধ্যায় ১–৪    লিনাক্স কী, আর্কিটেকচার, ইনস্টল, ডেস্কটপ
পর্ব ২: কমান্ড লাইন         অধ্যায় ৫–১৪   টার্মিনাল, শর্টকাট, ফাইল সিস্টেম, ফাইল নিয়ে কাজ, এডিটর
পর্ব ৩: ইউজার ও নিরাপত্তা   অধ্যায় ১৫–১৮  ইউজার, গ্রুপ, পারমিশন, sudo
পর্ব ৪: টেক্সট প্রসেসিং     অধ্যায় ১৯–২৩  pipe, grep, sed, awk, অন্যান্য টুল
পর্ব ৫: সিস্টেম চালানো      অধ্যায় ২৪–৩১  প্রসেস, tmux, systemd, বুট, লগ, cron, প্যাকেজ
পর্ব ৬: স্টোরেজ             অধ্যায় ৩২–৩৫  ডিস্ক, LVM, RAID, আর্কাইভ, ব্যাকআপ
পর্ব ৭: নেটওয়ার্ক ও সার্ভার  অধ্যায় ৩৬–৪০  নেটওয়ার্কিং, SSH, ফায়ারওয়াল, ওয়েব সার্ভার
পর্ব ৮: অটোমেশন            অধ্যায় ৪১–৪৪  শেল স্ক্রিপ্টিং ১–৩, পরিবেশ কাস্টমাইজেশন
পর্ব ৯: এক্সপার্ট স্তর      অধ্যায় ৪৫–৪৮  নিরাপত্তা, পারফরম্যান্স, কার্নেল, কন্টেইনার
পরিশিষ্ট                    ক, খ          শর্টকাট চিটশিট, কমান্ড রেফারেন্স, রোডম্যাপ, ইন্টারভিউ
```

কোন পাঠক কোন পথে

তুমি কে	প্রস্তাবিত পথ
একদম নতুন (fresher)	অধ্যায় ১ থেকে ক্রমানুসারে। অধ্যায় ১–২০ খুব ভালো করে পড়ো — বাকি সব এর ওপর দাঁড়ানো
ওয়েব/সফটওয়্যার ডেভেলপার	৫–১২, ১৬, ১৯–২৪, ৩০, ৩৮, ৪১–৪৪, ৪৮
সিস্টেম অ্যাডমিন হতে চাও	পুরো বই; বিশেষ জোর ১৫–৪০, ৪৫–৪৬
DevOps/Cloud লক্ষ্য	১–৩০ পুরো, তারপর ৩৬–৪৮
আগে থেকে কিছুটা জানো	প্রতিটা অধ্যায়ের শেষে "নিজেকে যাচাই করো" আগে দেখো; সব পারলে পরের অধ্যায়ে যাও

বইয়ের সংকেত ও নিয়ম

সংকেত	মানে
`$ command`	সাধারণ ব্যবহারকারী হিসেবে চালাও (`$` টাইপ করবে না)
`# command`	root হিসেবে চালাতে হবে (বা `sudo command`) — **code block-এর ভেতরে** `#` থাকলে খেয়াল করো, এটা কমেন্টও হতে পারে; প্রসঙ্গ দেখে বুঝবে
`<file>` বা `FILE`	তোমার নিজের মান বসাবে (যেমন `<file>` এর জায়গায় `notes.txt`)
`[OPTION]`	ঐচ্ছিক — দিলেও চলে, না দিলেও চলে
`...`	একাধিক দেওয়া যায়

সংকেত	মানে
> ⚠️ **সতর্কতা**	ভুল করলে ডেটা হারানো বা সিস্টেম নষ্ট হতে পারে
> 💡 **টিপস**	কাজ সহজ/দ্রুত করার কৌশল
> 📝 **মনে রাখো**	ধারণাগত জরুরি কথা, পরীক্ষা/ইন্টারভিউতে আসে

কার্যকর পড়ার ৭টি নিয়ম

1. **পড়ো, তারপর টাইপ করো।** কপি-পেস্ট না — নিজের আঙুলে টাইপ করো। আঙুলের স্মৃতি (muscle memory) তৈরি হওয়া জরুরি।
2. **ভাঙতে ভয় পেও না — তবে নিরাপদ জায়গায়।** ভার্চুয়াল মেশিনে (VirtualBox) অনুশীলন করো, snapshot রাখো। ভুল হলে snapshot-এ ফিরে যাও।
3. **"কেন" জিজ্ঞেস করো।** প্রতিটা কমান্ডের পেছনে ভেতরে কী হচ্ছে — বইয়ে "ভেতরে কীভাবে কাজ করে" অংশগুলো বাদ দিও না।
4. `man` **পড়ার অভ্যাস করো।** বই একদিন শেষ হবে, `man` সবসময় থাকবে।
5. **প্রতিদিন অল্প অল্প।** সপ্তাহে একদিন ৭ ঘণ্টার চেয়ে প্রতিদিন ১ ঘণ্টা অনেক কার্যকর।
6. **নিজের নোট রাখো।** একটা `~/linux-notes.md` ফাইলে নতুন শেখা কমান্ড লিখে রাখো (কীভাবে, তা অধ্যায় ১৩-এ শিখবে)।
7. **একটা ছোট প্রজেক্ট ধরো।** যেমন: পুরোনো ল্যাপটপে লিনাক্স বসিয়ে নিজের ওয়েবসাইট হোস্ট করা। প্রজেক্টই আসল শিক্ষক।

> 💡 **টিপস:** আটকে গেলে এরর মেসেজটা পুরোটা মন দিয়ে পড়ো — লিনাক্সের এরর মেসেজ সাধারণত সমস্যাটা ঠিকঠাক বলে দেয়। তারপর `man`, distro-র অফিসিয়াল ডকুমেন্টেশন (Ubuntu Docs, Arch Wiki, Red Hat Docs), এবং Ask Ubuntu / Unix & Linux Stack Exchange দেখো। **Arch Wiki** যেকোনো distro-র ব্যবহারকারীর জন্যই অসাধারণ।

> ⚠️ **সতর্কতা:** ইন্টারনেটে পাওয়া কোনো কমান্ড — বিশেষত `sudo`, `rm -rf`, `dd`, `curl ... | bash` থাকলে — না বুঝে কখনো চালাবে না। কিছু "মজার" পোস্ট ইচ্ছাকৃতভাবে সিস্টেম নষ্টকারী কমান্ড দেয়।

হাতে-কলমে অনুশীলন

> এই ল্যাবের জন্য দরকার যেকোনো লিনাক্স টার্মিনাল (ইনস্টল করা লিনাক্স, VM, WSL, বা Live USB)। কোনো কমান্ডই সিস্টেমে কিছু বদলায় না — নিশ্চিন্তে চালাও।

ল্যাব ১: নিজের সিস্টেমকে চেনো

1. টার্মিনাল খোলো (`Ctrl+Alt+T`, অথবা `Super` চেপে "Terminal" লিখে Enter)।
2. প্রম্পটটা দেখো। ব্যবহারকারীর নাম আর hostname কী? খাতায় লেখো।
3. যাচাই করো:

   ```
   $ whoami
   $ hostname
   ```

4. কার্নেল সংস্করণ বের করো:

   ```
   $ uname -r
   ```

   সংখ্যাটার major ও minor অংশ আলাদা করে লেখো।
5. আর্কিটেকচার দেখো:

   ```
   $ uname -m
   ```

   `x86_64` হলে সাধারণ ৬৪-বিট Intel/AMD; `aarch64` হলে ৬৪-বিট ARM (যেমন Raspberry Pi 4/5, Apple Silicon-এ চালানো VM)।
6. distro বের করো:

   ```
   $ cat /etc/os-release
   ```

   `NAME`, `VERSION_ID`, আর `ID_LIKE` কী — লেখো। `ID_LIKE` দেখে বলো তোমার distro কোন পরিবারের।
7. systemd থাকলে:

   ```
   $ hostnamectl
   ```

ল্যাব ২: GNU-র ছাপ খোঁজো

1. `uname -o` চালাও — output-এ "GNU" আছে কি?
2. Bash নিজের পরিচয় দিক:

   ```
   $ bash --version
   GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
   Copyright (C) 2022 Free Software Foundation, Inc.
   License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
   ...
   ```

   লক্ষ করো: কপিরাইট **Free Software Foundation**-এর, লাইসেন্স **GPLv3+**।
3. `ls` কোন প্রজেক্টের?

```
$ ls --version
ls (GNU coreutils) 9.4
...
```

4. C লাইব্রেরি:

   ```
   $ ldd --version
   ldd (Ubuntu GLIBC 2.39-0ubuntu8.3) 2.39
   ...
   ```

   (Alpine-এ এখানে `musl` দেখাবে।)
5. **ভাবো:** তুমি যে "লিনাক্স" ব্যবহার করছ, তার কোন অংশ Linus-এর আর কোন অংশ GNU-র? একটা ছোট তালিকা বানাও।

ল্যাব ৩: লাইসেন্স খুঁজে দেখো

Debian/Ubuntu-তে প্রতিটা প্যাকেজের লাইসেন্স একটা নির্দিষ্ট জায়গায় থাকে:

```
$ ls /usr/share/doc/bash/
$ less /usr/share/doc/bash/copyright
```

(`less` থেকে বের হতে `q` ; উপরে-নিচে যেতে তীর-চিহ্ন।) লাইসেন্সের নাম কী লেখা আছে? এবার `coreutils` -এর জন্যও দেখো। Fedora/RHEL-এ লাইসেন্স জানতে:

```
$ rpm -q --queryformat '%{NAME}: %{LICENSE}\n' bash
bash: GPL-3.0-or-later
```

(পুরোনো রিলিজে লাইসেন্স `GPLv3+` আকারে দেখাতে পারে।)

ল্যাব ৪: সাহায্য নেওয়ার অভ্যাস

1. `man uname` খোলো, `-r` অপশনের বর্ণনা খুঁজে বের করো। ভেতরে খুঁজতে `/` চেপে `release` লিখে Enter; পরের মিলে যেতে `n` । বের হতে `q` ।
2. `whatis whoami hostname uname` চালাও।
3. `apropos "system information"` চালিয়ে দেখো আর কোন কমান্ড আছে।

ল্যাব ৫ (চিন্তার অনুশীলন)

তোমার চারপাশের ১০টা ডিভাইস বা সার্ভিসের তালিকা বানাও (ফোন, রাউটার, টিভি, প্রিয় ওয়েবসাইট, ব্যাংকের অ্যাপ...)। প্রতিটার পাশে লেখো সেটা লিনাক্স চালায় বলে তোমার ধারণা কিনা, এবং কেন। (রাউটারের ক্ষেত্রে অনেক সময় অ্যাডমিন পেজে বা ম্যানুয়ালে "GPL source code" ডাউনলোডের লিংক থাকে — সেটা লিনাক্স ব্যবহারের ভালো ইঙ্গিত!)

সারসংক্ষেপ

মূল ধারণা

ধারণা	এক লাইনে
Operating System	হার্ডওয়্যার পরিচালনা করে প্রোগ্রামকে নিরাপদে ব্যবহারের সুযোগ দেয় এমন সফটওয়্যার
Kernel	OS-এর কেন্দ্র; হার্ডওয়্যারের সাথে সরাসরি কথা বলে; Linux আসলে একটা kernel
User space / Kernel space	সাধারণ প্রোগ্রামের এলাকা / কার্নেলের সুরক্ষিত এলাকা; মাঝে system call
Unix	১৯৬৯, Bell Labs, Thompson ও Ritchie; ১৯৭৩-এ C-তে লেখা
Unix দর্শন	একটা টুল একটা কাজ, টুল জোড়া লাগাও, টেক্সট স্ট্রিম
GNU	১৯৮৩, Richard Stallman; "GNU's Not Unix"; free OS-এর টুল
Linux	১৯৯১, Linus Torvalds; GPLv2-only kernel
Distribution (distro)	kernel + GNU টুল + অন্য সফটওয়্যার + প্যাকেজ ম্যানেজার = সম্পূর্ণ OS
Copyleft	বদলানো কোড বিতরণ করলে একই লাইসেন্সে সোর্স দিতে হবে (GPL)
Permissive	কৃতিত্ব রাখলেই হলো, closed করা যায় (MIT, BSD, Apache)
POSIX	Unix-সদৃশ সিস্টেমের স্ট্যান্ডার্ড; লিনাক্স মোটামুটি মেনে চলে

এই অধ্যায়ের কমান্ড

কমান্ড	কাজ
`whoami`	বর্তমান ব্যবহারকারীর নাম
`hostname`	কম্পিউটারের নাম
`uname -a`	কার্নেল ও সিস্টেমের সব তথ্য
`uname -r`	কার্নেল রিলিজ সংস্করণ
`uname -m`	হার্ডওয়্যার আর্কিটেকচার
`uname -o`	OS নাম (`GNU/Linux`)
`cat /etc/os-release`	distro-র নাম ও সংস্করণ (সবচেয়ে নির্ভরযোগ্য)
`hostnamectl`	hostname, OS, কার্নেল একসাথে (systemd)
`lsb_release -a`	distro তথ্য (মূলত Debian/Ubuntu)
`bash --version` , `ls --version`	টুলের সংস্করণ ও লাইসেন্স
`man <cmd>` / `<cmd> --help` / `whatis <cmd>` / `apropos <word>`	সাহায্য

এই অধ্যায়ের শর্টকাট

শর্টকাট	কাজ
`Ctrl+Alt+T`	টার্মিনাল খোলা (Ubuntu/Mint)
`Tab` / `Tab Tab`	auto-complete / সব মিল দেখানো
`↑` `↓`	কমান্ড হিস্ট্রি
`Ctrl+C`	চলন্ত কমান্ড থামানো
`Ctrl+Shift+C` / `Ctrl+Shift+V`	টার্মিনালে কপি / পেস্ট
`Ctrl+L`	স্ক্রিন পরিষ্কার
`Ctrl+D`	শেল থেকে বের হওয়া
`q`	`man` / `less` থেকে বের হওয়া
`Ctrl+Alt+F3` … `F6`	টেক্সট কনসোল (TTY)

নিজেকে যাচাই করো

1. অপারেটিং সিস্টেমের পাঁচটা প্রধান কাজ বলো।
2. "Linux" শব্দটা কঠোর অর্থে কী বোঝায়, আর দৈনন্দিন কথায় কী বোঝায়?
3. Unix কোথায়, কবে, কারা তৈরি করেন? C ভাষায় নতুন করে লেখা কেন বিপ্লবী ছিল?
4. Unix দর্শনের তিনটা মূল নীতি কী? একটা pipe-এর উদাহরণ দাও।
5. "Free as in free speech, not as in free beer" — এর মানে কী? ফ্রি সফটওয়্যারের চার স্বাধীনতা বলো।
6. GNU প্রজেক্টের কোন অংশটা ১৯৯১ সালে অসম্পূর্ণ ছিল, আর লিনাক্স কীভাবে সেই ফাঁক পূরণ করল?
7. Linux kernel কোন লাইসেন্সে? Copyleft ও permissive লাইসেন্সের পার্থক্য উদাহরণসহ বলো।
8. macOS কি লিনাক্স? macOS আর লিনাক্সের সম্পর্ক কী?
9. তোমার সিস্টেমের distro ও কার্নেল সংস্করণ জানতে কোন কোন কমান্ড ব্যবহার করবে? `hostnamectl` কাজ না করলে বিকল্প কী?
10. টার্মিনালে `Ctrl+C` চাপলে কী হয়? কপি করতে কী চাপবে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. প্রসেস ম্যানেজমেন্ট (CPU ভাগ করা), মেমরি ম্যানেজমেন্ট, ফাইল সিস্টেম ম্যানেজমেন্ট, ডিভাইস ম্যানেজমেন্ট (driver), নেটওয়ার্কিং, নিরাপত্তা/অনুমতি, ইউজার ইন্টারফেস — এর যেকোনো পাঁচটা।

2. কঠোর অর্থে **Linux** শুধু **kernel** — Linus Torvalds শুরু করা অপারেটিং সিস্টেমের কেন্দ্রীয় অংশ। দৈনন্দিন কথায় "লিনাক্স" মানে পুরো **distribution** — kernel + GNU টুল (bash, coreutils, glibc) + ডেস্কটপ + প্যাকেজ ম্যানেজার ইত্যাদি (যেমন Ubuntu, Fedora)।
</details>

3. **১৯৬৯ সালে AT&T-এর Bell Labs-এ Ken Thompson ও Dennis Ritchie** তৈরি করেন। ১৯৭৩-এ কার্নেল **C**-তে নতুন করে লেখা হয়। assembly নির্দিষ্ট যন্ত্রে বাঁধা থাকে, কিন্তু C-তে লেখায় শুধু নতুন যন্ত্রের জন্য C compiler বানিয়েই Unix সেখানে নেওয়া যেত — একে বলে **portability**। এতেই Unix নানা ধরনের কম্পিউটারে ছড়িয়ে পড়ে।

4. (ক) একটা প্রোগ্রাম একটা কাজ ভালোভাবে করবে; (খ) প্রোগ্রামগুলো একসাথে কাজ করবে; (গ) টেক্সট স্ট্রিম সর্বজনীন ইন্টারফেস। উদাহরণ: `cat access.log | grep "404" | wc -l` — ফাইল পড়ে, "404" থাকা লাইন বেছে, লাইন গোনে।

5. "Free" মানে **স্বাধীনতা**, **বিনামূল্যে** না — free software বিক্রিও করা যায়। চার স্বাধীনতা: **0** — যেকোনো উদ্দেশ্যে চালানো; **1** — সোর্স পড়ে বোঝা ও বদলানো; **2** — কপি বিতরণ; **3** — বদলানো সংস্করণ বিতরণ।

6. **Kernel** অসম্পূর্ণ ছিল — GNU-র নিজস্ব কার্নেল **GNU Hurd** ব্যবহারযোগ্য হয়নি। ১৯৯১-এ Linus Linux kernel বানান এবং ১৯৯২-এ GPLv2-এর অধীনে আনেন। ফলে GNU টুল + Linux kernel মিলে প্রথম সম্পূর্ণ free OS তৈরি হয় — এ কারণেই FSF একে "GNU/Linux" বলতে বলে।

7. **GPLv2-only**। **Copyleft** (যেমন GPL): বদলানো কোড বিতরণ করলে সেটাও একই লাইসেন্সে সোর্সসহ দিতে হবে — যেমন কেউ লিনাক্স কার্নেল বদলে রাউটারে বিক্রি করলে সেই বদলানো সোর্স দিতে হবে। **Permissive** (যেমন MIT, BSD, Apache 2.0): কপিরাইট নোটিশ রাখলেই হলো, বদলানো কোড বন্ধ (closed) রেখে বিক্রি করা যায়।

8. **না।** macOS-এর কার্নেল **XNU** (Mach + BSD অংশ), আর macOS সার্টিফাইড **UNIX** (UNIX 03)। লিনাক্স Unix-এর কোড ছাড়াই লেখা **Unix-like** সিস্টেম। দুজনেই Unix পরিবারের আত্মীয় এবং POSIX-ঘেঁষা, তাই বেশিরভাগ কমান্ড ও ধারণা একই — তবে macOS-এ অনেক টুল BSD সংস্করণের, তাই কিছু অপশন আলাদা।

9. Distro: `cat /etc/os-release` (সবচেয়ে নির্ভরযোগ্য), `hostnamectl`, `lsb_release -a`। কার্নেল: `uname -r` বা `uname -a`। `hostnamectl` systemd-এর ওপর নির্ভর করে — WSL বা কন্টেইনারে systemd না থাকলে কাজ না-ও করতে পারে; তখন `cat /etc/os-release` আর `uname -a`।

10. `Ctrl+C` চলন্ত কমান্ডকে **interrupt** (SIGINT সংকেত পাঠিয়ে) করে থামায় — কপি করে না। টার্মিনালে কপি `Ctrl+Shift+C`, পেস্ট `Ctrl+Shift+V` (অথবা সিলেক্ট করে মাউসের মাঝের বোতামে পেস্ট)। সিগন্যাল নিয়ে বিস্তারিত অধ্যায় ২৪-এ।

অধ্যায় ২: লিনাক্স আর্কিটেকচার — ভেতরের গঠন

> "একটা গাড়ি চালাতে ইঞ্জিনের ভেতরটা জানতে হয় না। কিন্তু গাড়ি নষ্ট হলে ঠিক করতে চাইলে, বা রেসিং ড্রাইভার হতে চাইলে — ইঞ্জিন বুঝতেই হবে।"

আগের অধ্যায়ে (অধ্যায় ১) আমরা জেনেছি লিনাক্স কী আর কেন শিখব। এই অধ্যায়ে আমরা লিনাক্সের "ইঞ্জিনের ঢাকনা" খুলে ভেতরে তাকাব। এখানে যা শিখবে, তা বাকি পুরো বইয়ের ভিত্তি। পরে যখন প্রসেস, পারমিশন, ফাইলসিস্টেম, নেটওয়ার্ক শিখবে, তখন বারবার মনে হবে — "ও আচ্ছা, এটা তো সেই kernel-এর কাজ!"

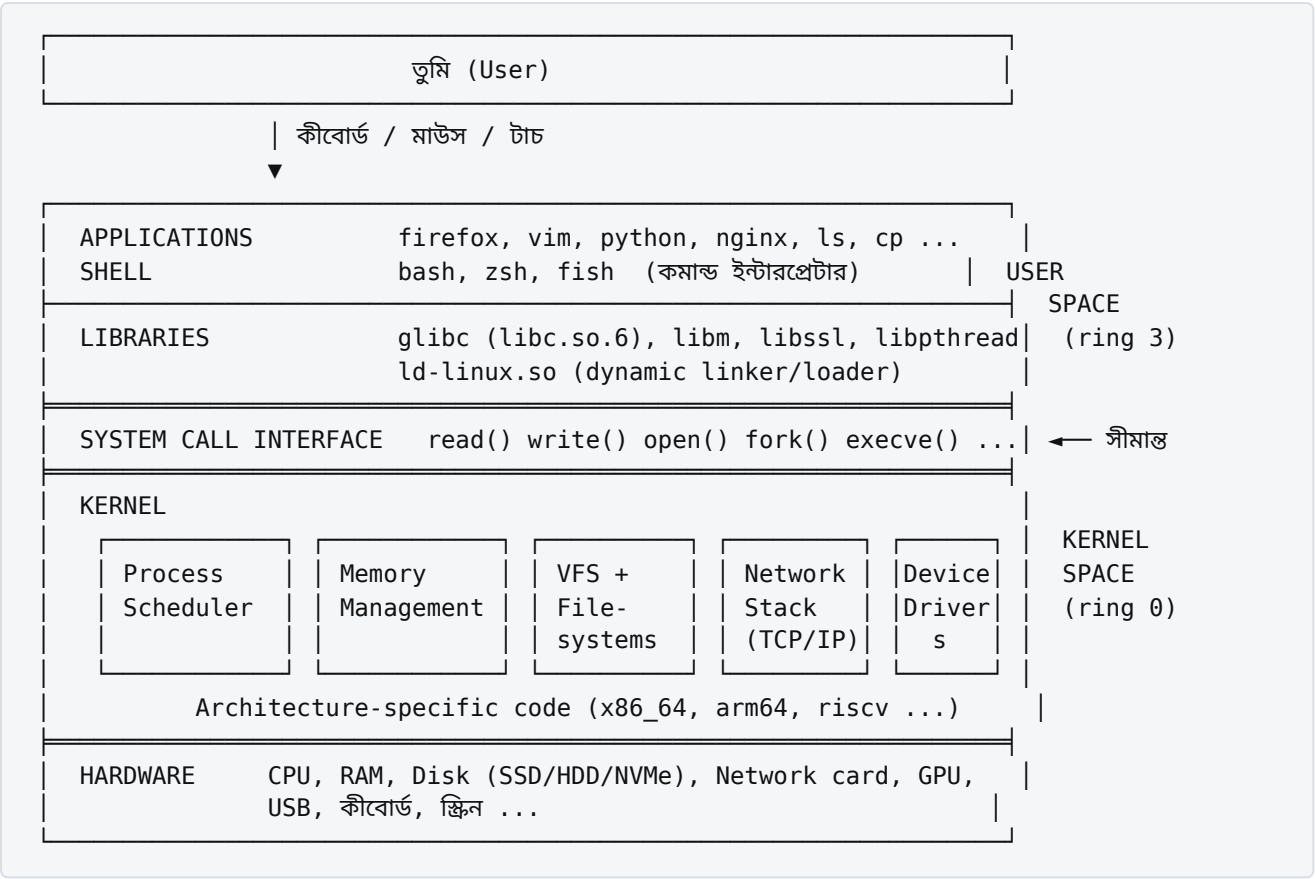
ভয় পেয়ো না। এই অধ্যায়ে কিছু নতুন শব্দ আসবে, কিন্তু প্রতিটি শব্দ আমরা উপমা দিয়ে, ধাপে ধাপে বুঝব।

এই অধ্যায়ে যা শিখবে

- লিনাক্স সিস্টেম কয়টা স্তরে (layer) সাজানো — hardware থেকে application পর্যন্ত
- **kernel** আসলে কী, আর "লিনাক্স" বলতে আসলে কী বোঝায়
- **kernel space** আর **user space** কী, কেন এই ভাগাভাগি দরকার
- CPU-এর **privilege ring** (ring 0 / ring 3) কীভাবে নিরাপত্তা দেয়
- **monolithic kernel**, **microkernel** ও **hybrid kernel**-এর পার্থক্য
- **loadable kernel module** কী — `lsmod`, `modinfo`, `modprobe`
- kernel-এর পাঁচটি প্রধান কাজ: process scheduling, memory management, VFS, device drivers, networking
- **system call** কী, কীভাবে কাজ করে, আর `strace` দিয়ে কীভাবে তা চোখে দেখা যায়
- **glibc** ও অন্যান্য library-র ভূমিকা — `ldd` দিয়ে দেখা
- **shell** কোথায় বসে, কী করে
- লিনাক্সের বিখ্যাত দর্শন: **"Everything is a file"**
- টার্মিনালে `ls` লিখে Enter চাপলে ভেতরে ঠিক কী কী ঘটে — **fork** → **exec** → **wait** ধাপে ধাপে
- architecture-সম্পর্কিত কীবোর্ড শর্টকাট (signal, virtual console, Magic SysRq)

২.১ বড় ছবি: লিনাক্স সিস্টেমের স্তরগুলো

একটা লিনাক্স কম্পিউটারকে একটা বহুতল ভবন হিসেবে কল্পনা করো। সবার নিচে মাটি আর ভিত্তি (hardware), তার ওপর ইঞ্জিনিয়ারিং কন্ট্রোল রুম (kernel), তার ওপর রিসেপশন ডেস্ক (system call ইন্টারফেস), তার ওপর বিভিন্ন সার্ভিস অফিস (libraries), আর একদম ওপরের তলায় সাধারণ মানুষ তাদের কাজ করে (applications ও shell)।



প্রতিটি স্তর শুধু তার ঠিক নিচের স্তরের সাথে কথা বলে (মোটামুটি)। Firefox কখনো সরাসরি নেটওয়ার্ক কার্ডে বিদ্যুৎ সংকেত পাঠায় না। সে glibc-র একটা ফাংশন ডাকে, glibc একটা system call করে, kernel সেটা গ্রহণ করে network stack আর driver-এর মাধ্যমে hardware-কে নির্দেশ দেয়।

প্রতিটি স্তর সংক্ষেপে

স্তর	কী	উদাহরণ	উপমা
Hardware	ভৌত যন্ত্রাংশ	CPU, RAM, SSD, NIC	ভবনের মাটি, ইট, বিদ্যুৎ লাইন
Kernel	OS-এর কেন্দ্র; hardware নিয়ন্ত্রণ করে	Linux kernel 6.x	ভবনের ম্যানেজার ও কন্ট্রোল রুম
System call	user প্রোগ্রাম kernel-কে অনুরোধ করার একমাত্র দরজা	`read`, `write`, `openat`	রিসেপশন ডেস্কের ফর্ম
Libraries	প্রোগ্রামারদের জন্য সহজ ফাংশন; ভেতরে system call করে	glibc, OpenSSL	ফর্ম পূরণে সাহায্যকারী কেরানি
Shell	তোমার লেখা কমান্ড বুঝে প্রোগ্রাম চালায়	bash, zsh	দোভাষী
Applications	আসল কাজের প্রোগ্রাম	Firefox, LibreOffice, `ls`	ভবনের বাসিন্দা

> 📝 **মনে রাখো:** "Linux" শব্দটা কঠোর অর্থে শুধু **kernel**-কে বোঝায়, যেটা ১৯৯১ সালে Linus Torvalds লেখা শুরু করেন। আমরা যে পুরো অপারেটিং সিস্টেম ব্যবহার করি (kernel + GNU টুল + library + shell + desktop), তাকে অনেকে **GNU/Linux** বলেন। প্রতিদিনের কথায় পুরোটাকেই "লিনাক্স" বলা হয় — এই বইতেও তাই বলব, তবে পার্থক্যটা জেনে রাখো।

২.২ Hardware — সবার নিচের স্তর

kernel-এর কাজ বুঝতে hardware সম্পর্কে অল্প কিছু জানা দরকার:

- **CPU (Central Processing Unit):** নির্দেশ (instruction) একটার পর একটা চালায়। আধুনিক CPU-তে একাধিক **core** থাকে; প্রতিটি core একসাথে একটা করে কাজ চালাতে পারে (hyper-threading থাকলে একটা core দুটো hardware thread চালায়)।
- **RAM (Memory):** দ্রুত কিন্তু অস্থায়ী (বিদ্যুৎ গেলে মুছে যায়) জায়গা, যেখানে চলমান প্রোগ্রাম ও তার ডেটা থাকে।
- **Storage (HDD/SSD/NVMe):** ধীর কিন্তু স্থায়ী জায়গা।
- **I/O devices:** কীবোর্ড, মাউস, স্ক্রিন, নেটওয়ার্ক কার্ড (NIC), USB ইত্যাদি।
- **MMU (Memory Management Unit):** CPU-র ভেতরের একটা অংশ, যেটা "virtual address"-কে "physical address"-এ অনুবাদ করে। (এটা পরে memory management-এ কাজে লাগবে।)
- **Interrupt:** hardware যখন CPU-র মনোযোগ চায় (যেমন কীবোর্ডে কী চাপা হলো, বা নেটওয়ার্কে প্যাকেট এলো), তখন সে একটা interrupt সংকেত পাঠায়। CPU যা করছিল থামিয়ে kernel-এর একটা নির্দিষ্ট কোড (interrupt handler) চালায়।

তোমার মেশিনের hardware দেখতে কয়েকটা কমান্ড:

```
$ lscpu | head -8
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Address sizes:          39 bits physical, 48 bits virtual
  Byte Order:             Little Endian
CPU(s):                   8
  On-line CPU(s) list:    0-7
Vendor ID:                GenuineIntel
  Model name:             Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz

$ free -h
               total        used        free      shared  buff/cache   available
Mem:            15Gi       4.2Gi       6.1Gi       512Mi       5.3Gi        10Gi
Swap:          2.0Gi          0B       2.0Gi
```

(তোমার মেশিনে সংখ্যাগুলো আলাদা হবে।) এগুলো বিস্তারিত আমরা অধ্যায় ৪৬-এ দেখব।

২.৩ Kernel — অপারেটিং সিস্টেমের হৃদয়

Kernel (কার্নেল, শাব্দিক অর্থ "শস্যদানার শাঁস" বা "কেন্দ্র") হলো সেই প্রোগ্রাম, যেটা কম্পিউটার চালু হওয়ার পর সবার আগে মেমোরিতে লোড হয় এবং বন্ধ হওয়া পর্যন্ত সবসময় চলতে থাকে। এটা hardware আর বাকি সব প্রোগ্রামের মাঝখানে বসে থাকা **মধ্যস্থতাকারী**।

উপমা: রেস্তোরাঁর রান্নাঘর

একটা ব্যস্ত রেস্তোরাঁর কথা ভাবো:

- **খদ্দের** (applications) রান্নাঘরে ঢুকতে পারে না। তারা শুধু মেনু থেকে অর্ডার দেয়।
- **মেনু** হলো system call-এর তালিকা — কী কী চাওয়া যাবে, তা নির্দিষ্ট।
- **ওয়েটার** (library/glibc) অর্ডারটা সঠিক ফরম্যাটে লিখে রান্নাঘরে পৌঁছে দেয়।
- **হেড শেফ** (kernel) ঠিক করে কোন রাঁধুনি কখন কোন চুলা ব্যবহার করবে (scheduling), কোন তাকে কোন উপকরণ থাকবে (memory), ফ্রিজ-গুদাম কীভাবে সাজানো (filesystem)।
- **চুলা, ছুরি, ফ্রিজ** হলো hardware। প্রতিটি যন্ত্র চালানোর আলাদা কায়দা আছে — সেই কায়দা জানে **device driver**।

খদ্দেররা যদি সরাসরি রান্নাঘরে ঢুকে যে যার মতো চুলা দখল করত, তাহলে বিশৃঙ্খলা, আগুন, চুরি — সব হতো। kernel ঠিক এটাই আটকায়।

Kernel-এর মূল দায়িত্ব এক নজরে

1. **Process management** — কোন প্রোগ্রাম কখন CPU পাবে
2. **Memory management** — কে কতটুকু RAM পাবে, কেউ যেন অন্যের মেমোরিতে হাত না দেয়
3. **Filesystem (VFS)** — ডিস্কের ডেটা ফাইল-ফোল্ডার আকারে দেখানো
4. **Device drivers** — hardware-এর সাথে কথা বলা
5. **Networking** — TCP/IP দিয়ে ডেটা পাঠানো-আনা
6. **Security ও access control** — কে কী করতে পারবে (permissions, capabilities, SELinux/AppArmor)
7. **System call interface** — user প্রোগ্রামকে এসব সেবা দেওয়ার দরজা

তোমার kernel-এর তথ্য দেখো

```
$ uname -r
6.8.0-45-generic

$ uname -a
Linux mypc 6.8.0-45-generic #45-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug 30 12:02:04 UTC 2024
x86_64 x86_64 x86_64 GNU/Linux
```

`uname` অপশন	কী দেখায়
`-s`	kernel-এর নাম (`Linux`)

`uname` অপশন	কী দেখায়
`-n`	hostname (মেশিনের নাম)
`-r`	kernel release (ভার্সন)
`-v`	kernel build-এর তথ্য
`-m`	hardware architecture (`x86_64` , `aarch64`)
`-o`	অপারেটিং সিস্টেম (`GNU/Linux`)
`-a`	সব একসাথে

kernel ভার্সন নম্বর পড়ার নিয়ম: `6.8.0-45-generic` মানে mainline kernel 6.8, তার ওপর distro-র (এখানে Ubuntu) নিজস্ব patch লেভেল `0-45` , আর `generic` হলো kernel flavour। Ubuntu 24.04 kernel 6.8 দিয়ে আসে, Debian 12 আসে 6.1 দিয়ে, RHEL 9 আসে 5.14 দিয়ে (Red Hat অনেক নতুন ফিচার backport করে), Fedora সাধারণত সবচেয়ে নতুন stable kernel রাখে।

kernel-এর ফাইলটা নিজে থাকে `/boot` -এ:

```
$ ls /boot
config-6.8.0-45-generic      initrd.img-6.8.0-45-generic  vmlinuz-6.8.0-45-generic
System.map-6.8.0-45-generic  grub                          ...
```

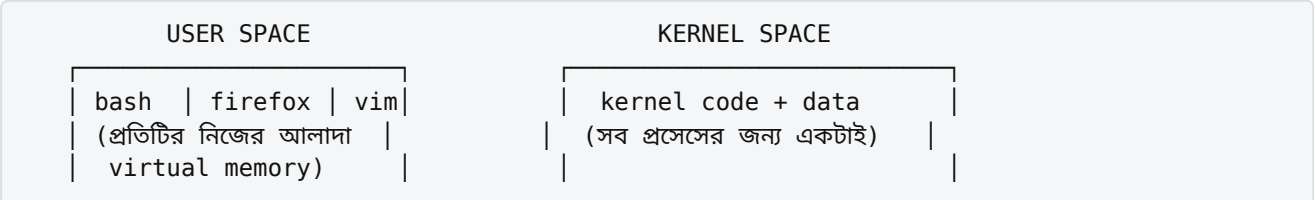
`vmlinuz` হলো compressed kernel image ("vm" = virtual memory সাপোর্ট, "z" = compressed)। `initrd.img` / `initramfs` হলো বুটের সময় ব্যবহৃত ছোট একটা অস্থায়ী root filesystem। এগুলো কীভাবে লোড হয়, তা অধ্যায় ২৭-এ বিস্তারিত।

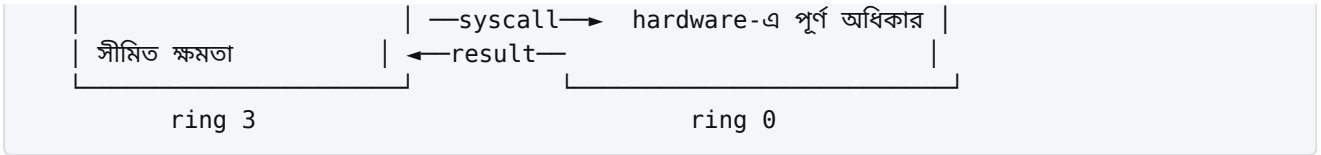
২.৪ Kernel Space বনাম User Space

এটা এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ ধারণাগুলোর একটা।

মেমোরি আর CPU-র ক্ষমতাকে লিনাক্স দুটো জগতে ভাগ করে:

- **Kernel space (কার্নেল জগৎ):** এখানে শুধু kernel-এর কোড চলে। এখানে কোড hardware-এ সরাসরি হাত দিতে পারে, যেকোনো মেমোরি পড়তে-লিখতে পারে, যেকোনো CPU instruction চালাতে পারে। পূর্ণ ক্ষমতা।
- **User space (ব্যবহারকারী জগৎ):** তোমার সব সাধারণ প্রোগ্রাম — bash, Firefox, Python, এমনকি root হিসেবে চলা প্রোগ্রামও — এখানে চলে। এখানে ক্ষমতা সীমিত: নিজের মেমোরি ছাড়া অন্য কিছুতে হাত দেওয়া যায় না, hardware সরাসরি ছোঁয়া যায় না।



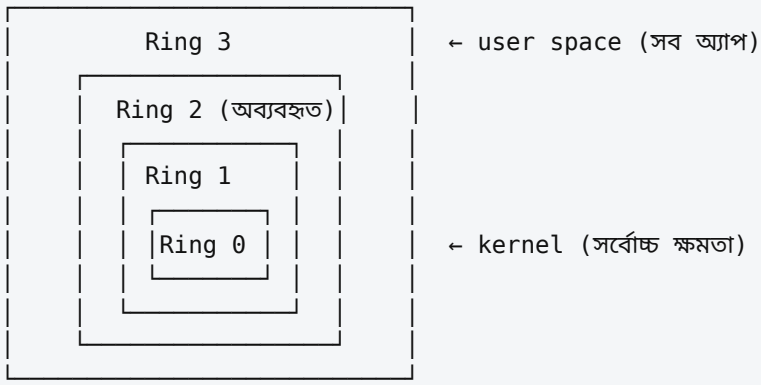


কেন এই ভাগাভাগি?

1. **স্থিতিশীলতা (Stability):** একটা প্রোগ্রাম crash করলে শুধু সেই প্রোগ্রামটাই মরে, পুরো সিস্টেম না। Firefox হ্যাং করলে তুমি তাকে মেরে ফেলো, কম্পিউটার চলতে থাকে।
2. **নিরাপত্তা (Security):** একটা ক্ষতিকর বা বাগযুক্ত প্রোগ্রাম অন্য প্রোগ্রামের মেমোরি (যেমন তোমার ব্রাউজারের পাসওয়ার্ড) পড়তে পারে না।
3. **ন্যায্য বণ্টন (Fairness):** কোনো প্রোগ্রাম CPU বা ডিস্ক একা দখল করে রাখতে পারে না, কারণ বণ্টন করে kernel।
4. **বিমূর্তকরণ (Abstraction):** প্রোগ্রামারকে জানতে হয় না ডিস্কটা SATA না NVMe — সে শুধু `write()` ডাকে।

CPU Privilege Ring — hardware-স্তরের পাহারা

এই ভাগাভাগি শুধু "নিয়ম" না — CPU নিজেই এটা জোর করে মানায়। x86 CPU-তে চারটা **privilege level** বা **ring** আছে (0 থেকে 3)। লিনাক্স ব্যবহার করে মাত্র দুটো:



ARM64 CPU-তে একই ধারণাকে বলে **Exception Level** — EL0 (user), EL1 (kernel), EL2 (hypervisor)।

Ring 3-এ চলা কোড যদি কোনো "বিশেষাধিকারপ্রাপ্ত" instruction চালাতে চায় (যেমন interrupt বন্ধ করা, বা page table বদলানো), CPU নিজেই একটা exception তোলে, kernel সেটা ধরে, আর প্রোগ্রামটাকে সাধারণত একটা signal (যেমন `SIGSEGV` বা `SIGILL`) দিয়ে মেরে ফেলে।

> 💡 **টিপস:** "Segmentation fault (core dumped)" মেসেজটা দেখেছ? এর মানে একটা প্রোগ্রাম এমন মেমোরিতে হাত দিতে চেয়েছিল যেটা তার না। MMU সেটা ধরেছে, kernel প্রোগ্রামটাকে `SIGSEGV` signal পাঠিয়ে থামিয়েছে। সিস্টেমের বাকি সবকিছু নিরাপদ।

> ⚠️ **সতর্কতা: root user** আর **kernel space** এক জিনিস না! root হিসেবে চলা প্রোগ্রামও user space-এ (ring 3) চলে। root-এর বিশেষত্ব হলো kernel তার অনুরোধগুলো বেশি অনুমোদন করে (যেমন অন্যের ফাইল পড়া)। কিন্তু root-এর প্রোগ্রামও hardware-এ সরাসরি না, system call-এর মাধ্যমেই কাজ করে। root ও sudo নিয়ে বিস্তারিত অধ্যায় ১৮-এ।

Mode switch — দুই জগতের মধ্যে যাতায়াত

একটা প্রোগ্রাম সারাজীবন user mode-এ থাকে না। তিনটা উপায়ে CPU user mode থেকে kernel mode-এ যায়:

উপায়	কখন হয়	উদাহরণ
System call	প্রোগ্রাম নিজে ইচ্ছা করে kernel-এর সেবা চায়	ফাইল পড়া, প্রিন্ট করা
Exception / fault	প্রোগ্রাম এমন কিছু করেছে যা kernel-এর সামলানো দরকার	page fault, শূন্য দিয়ে ভাগ
Hardware interrupt	কোনো device মনোযোগ চাইছে	কীবোর্ডে কী চাপা, timer tick

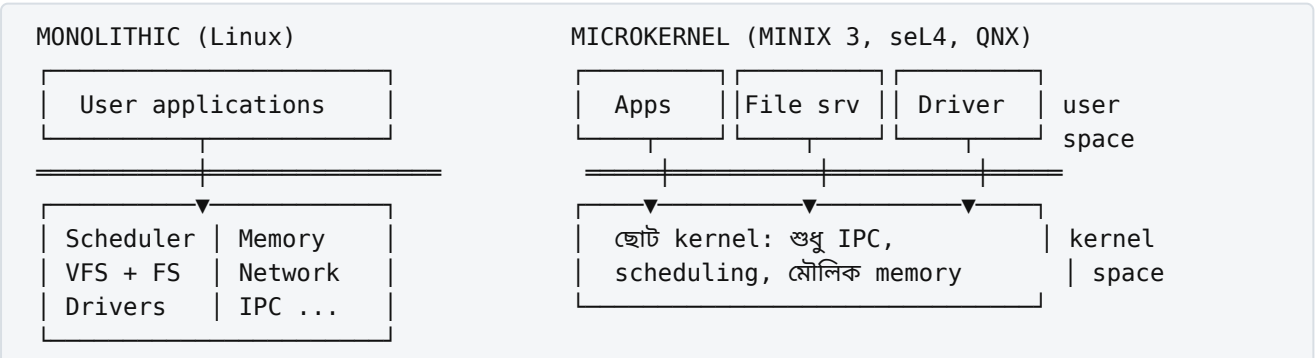
প্রতিবার এই সীমানা পার হতে অল্প হলেও সময় লাগে (CPU-র অবস্থা সংরক্ষণ করা, নিরাপত্তা যাচাই)। তাই দক্ষ প্রোগ্রাম অপ্রয়োজনীয় system call কমায় — যেমন এক বাইট এক বাইট করে না পড়ে একসাথে ৪ KB বা ৬৪ KB পড়ে।

২.৫ Kernel-এর ডিজাইন: Monolithic, Microkernel ও Hybrid

অপারেটিং সিস্টেমের kernel বানানোর কয়েকটা দর্শন আছে।

Monolithic kernel (একশিলা kernel)

সব প্রধান সেবা — scheduler, memory manager, filesystem, driver, network stack — **একটাই বড় প্রোগ্রামে**, একই kernel space-এ, একই address space-এ চলে।



- **সুবিধা:** দ্রুত। এক অংশ আরেক অংশকে সরাসরি function call করে; message পাঠাতে হয় না।

- **অসুবিধা:** একটা driver-এ বাগ থাকলে পুরো kernel crash করতে পারে (একে বলে **kernel panic**)। কোড বড় হয়।

Microkernel (ক্ষুদ্র kernel)

kernel-এ শুধু একেবারে মৌলিক জিনিস (IPC বা inter-process communication, মৌলিক scheduling, মৌলিক memory) থাকে। driver, filesystem ইত্যাদি user space-এ আলাদা "server" প্রসেস হিসেবে চলে, message পাঠিয়ে কথা বলে।

- **সুবিধা:** একটা driver crash করলে শুধু সেটা restart করলেই হয়; বেশি নিরাপদ, যাচাই করা সহজ।
- **অসুবিধা:** message আদান-প্রদানের কারণে ঐতিহাসিকভাবে ধীর।

Hybrid kernel

দুটোর মিশ্রণ। Windows NT kernel ও macOS-এর XNU এই শ্রেণিতে পড়ে বলে সাধারণত ধরা হয়।

> 📝 **ইতিহাস:** ১৯৯২ সালে MINIX-এর স্রষ্টা অধ্যাপক Andrew Tanenbaum Usenet-এ লেখেন "LINUX is obsolete", কারণ এটা monolithic। Linus Torvalds জবাব দেন — শুরু হয় বিখ্যাত **Tanenbaum–Torvalds বিতর্ক**। বাস্তবে লিনাক্স monolithic থেকেও ব্যাপক সফল হয়েছে, কারণ নিচের "loadable module" কৌশল দিয়ে সে monolithic ডিজাইনের সবচেয়ে বড় দুর্বলতা — নমনীয়তার অভাব — অনেকটাই কাটিয়ে উঠেছে।

বৈশিষ্ট্য	Monolithic (Linux)	Microkernel	Hybrid (Windows, macOS)
driver কোথায় চলে	kernel space	user space	মূলত kernel space
গতি	খুব দ্রুত	তুলনামূলক ধীর	দ্রুত
একটা driver বাগের প্রভাব	পুরো সিস্টেম পড়ে যেতে পারে	শুধু ঐ server	প্রায়ই পুরো সিস্টেম
উদাহরণ	Linux, FreeBSD	MINIX 3, seL4, QNX	Windows NT, XNU

২.৬ Loadable Kernel Modules (LKM) — দরকারমতো যোগ-বিয়োগ

ভাবো, পৃথিবীর হাজার হাজার রকম hardware-এর সব driver যদি kernel-এর মূল ফাইলে ঢোকানো থাকত, তাহলে kernel হতো বিশাল, আর নতুন একটা Wi-Fi কার্ডের driver যোগ করতে পুরো kernel আবার build করে reboot করতে হতো।

লিনাক্সের সমাধান: **loadable kernel module** (`.ko` ফাইল, "kernel object")। এগুলো kernel-এর টুকরো, যা **চলন্ত অবস্থায়** লোড বা আনলোড করা যায়, reboot ছাড়াই। লোড হওয়ার পর module পুরোপুরি kernel space-এ, kernel-এর অংশ হিসেবেই চলে — তাই লিনাক্স এখনও monolithic, কিন্তু "মডুলার monolithic"।

উপমা: monolithic kernel একটা সুইস আর্মি ছুরির মতো। module হলো এমন ব্যবস্থা যেখানে দরকারি ব্লেড আলাদা করে লাগানো-খোলা যায়, কিন্তু লাগানোর পর সেটা ছুরির অংশই।

Module কোথায় থাকে

```
$ ls /lib/modules/$(uname -r)/kernel/
arch  block  crypto  drivers  fs  kernel  lib  mm  net  samples  sound  ubuntu  virt
```

(সংস্করণ ও distro অনুযায়ী তালিকা একটু ভিন্ন হতে পারে। আধুনিক distro-তে `/lib` আসলে `/usr/lib`-এর symlink।) ফাইলগুলো প্রায়ই compressed থাকে — যেমন `.ko.zst` (Ubuntu 24.04, Fedora) বা `.ko.xz`।

Module সংক্রান্ত কমান্ড

কমান্ড	কাজ
`lsmod`	এখন লোড হওয়া module-এর তালিকা (আসলে `/proc/modules` সুন্দর করে দেখায়)
`modinfo <নাম>`	একটা module-এর বিবরণ: ফাইল, লাইসেন্স, লেখক, parameter, dependency
`sudo modprobe <নাম>`	module লোড করে, dependency সহ (প্রস্তাবিত উপায়)
`sudo modprobe -r <নাম>`	module আনলোড করে (অব্যবহৃত dependency সহ)
`sudo insmod /path/x.ko`	নির্দিষ্ট ফাইল সরাসরি লোড (dependency দেখে না)
`sudo rmmod <নাম>`	module সরাসরি আনলোড
`sudo depmod -a`	module dependency ডাটাবেস (`modules.dep`) আবার তৈরি করে

উদাহরণ:

```
$ lsmod | head -5
Module                  Size  Used by
snd_hda_codec_hdmi     94208  1
iwlmvm                806912  0
mac80211             1720320  1 iwlmvm
iwlwifi               598016  1 iwlmvm

$ modinfo loop | head -6
filename:       /lib/modules/6.8.0-45-generic/kernel/drivers/block/loop.ko.zst
license:        GPL
alias:          devname:loop-control
alias:          char-major-10-237
alias:          block-major-7-*
...
```

`lsmod`-এর কলাম: **Module** = নাম, **Size** = মেমোরিতে আকার (বাইট), **Used by** = কতজন এটা ব্যবহার করছে ও কারা। উপরে দেখো `mac80211` ব্যবহার করছে `iwlmvm` (Intel Wi-Fi driver) — এটাই dependency।

> 💡 **টিপস:** কিছু driver module না হয়ে kernel-এর ভেতরেই "built-in" থাকে। সেগুলো `lsmod`-এ দেখাবে না, কিন্তু `/lib/modules/$(uname -r)/modules.builtin` ফাইলে তালিকা থাকে। তাই "`lsmod`-এ নেই মানে driver নেই" — এই সিদ্ধান্ত সবসময় ঠিক না।

> ⚠️ **সতর্কতা:** ব্যবহৃত module জোর করে আনলোড করা বা অজানা উৎসের `.ko` লোড করা সিস্টেম crash করাতে পারে — কারণ module kernel space-এ পূর্ণ ক্ষমতা নিয়ে চলে। Secure Boot চালু থাকলে অনেক distro unsigned module লোড করতেই দেয় না। module, `/proc`, `/sys` নিয়ে গভীর আলোচনা অধ্যায় ৪৭-এ।

বুটের সময় স্বয়ংক্রিয়ভাবে module লোড করতে `/etc/modules-load.d/*.conf`, আর কোনো module ব্লক (blacklist) করতে বা option দিতে `/etc/modprobe.d/*.conf` ব্যবহার হয়। hardware সংযোগ হলে সঠিক module বেশিরভাগ ক্ষেত্রে **udev** স্বয়ংক্রিয়ভাবে লোড করে।

২.৭ Kernel-এর প্রধান কাজগুলো — একটু গভীরে

২.৭.১ Process Scheduling — CPU-র সময় ভাগ করা

Process (প্রসেস) হলো একটা চলমান প্রোগ্রাম। ডিস্কে থাকা `/usr/bin/ls` ফাইলটা একটা *program*; যখন সেটা চালানো হয় আর মেমোরিতে লোড হয়ে কাজ করতে থাকে, তখন সেটা একটা *process*। একই প্রোগ্রাম থেকে একসাথে অনেক প্রসেস চলতে পারে (যেমন তিনটা আলাদা টার্মিনালে তিনটা bash)।

তোমার কম্পিউটারে হয়তো ৮টা CPU core, কিন্তু চলছে ৩০০টা প্রসেস। সবাই একসাথে চলছে বলে মনে হয় কীভাবে?

উপমা: দাবার গ্র্যান্ডমাস্টারের simultaneous exhibition। একজন গ্র্যান্ডমাস্টার ৩০ জনের সাথে একসাথে খেলেন — এক বোর্ডে একটা চাল দিয়ে পরের বোর্ডে যান। প্রত্যেকে ভাবে তিনি তার সাথেই খেলছেন। kernel-এর **scheduler** ঠিক এটাই করে — প্রতিটি প্রসেসকে কয়েক মিলিসেকেন্ড করে CPU দেয়, তারপর পরেরটায় যায়। এত দ্রুত যে মনে হয় সবাই একসাথে চলছে। একে বলে **multitasking**, আর প্রসেস বদলানোকে বলে **context switch**।

```
CPU core 0 এর টাইমলাইন (মিলিসেকেন্ডে):
 |--bash--|--firefox--|--vim--|--firefox--|--sshd--|--bash--| ...
 0        3           7       9           13       14       17
          ▲ context switch: আগের প্রসেসের register অবস্থা সংরক্ষণ,
            পরেরটার অবস্থা ফিরিয়ে আনা
```

লিনাক্স **preemptive** multitasking করে: প্রসেস স্বেচ্ছায় CPU ছাড়ুক বা না ছাড়ুক, kernel timer interrupt দিয়ে তাকে থামিয়ে অন্যকে সুযোগ দিতে পারে। ফলে একটা অসীম loop-এ আটকে থাকা প্রোগ্রাম পুরো সিস্টেম জমিয়ে দিতে পারে না।

লিনাক্সের scheduler-এর ইতিহাস:

সময়কাল	Scheduler	সংক্ষেপে
2.6.0–2.6.22	O(1) scheduler	অগ্রাধিকার-ভিত্তিক queue
2.6.23 (2007)–6.5	**CFS** (Completely Fair Scheduler)	প্রতিটি task-এর "virtual runtime" হিসাব করে, যে সবচেয়ে কম CPU পেয়েছে তাকে আগে দেয় (red-black tree ব্যবহার করে)
6.6 (2023) থেকে	**EEVDF** (Earliest Eligible Virtual Deadline First)	CFS-এর জায়গা নিয়েছে; ন্যায্যতার সাথে latency আরও ভালোভাবে সামলায়

তাই Debian 12 (kernel 6.1) ও RHEL 9 (5.14) CFS ব্যবহার করে, আর Ubuntu 24.04 (kernel 6.8) ও নতুন Fedora EEVDF ব্যবহার করে। সাধারণ ব্যবহারকারীর কাছে পার্থক্য প্রায় অদৃশ্য।

প্রতিটি প্রসেসের একটা **nice value** থাকে (−20 থেকে 19; কম মান = বেশি অগ্রাধিকার)। এছাড়া real-time কাজের জন্য আলাদা নীতি (`SCHED_FIFO` , `SCHED_RR`) আছে। প্রসেস, PID, state, signal, `nice` , `ps` , `top` — এসব বিস্তারিত অধ্যায় ২৪-এ।

> 📝 **মনে রাখো:** লিনাক্স kernel-এর চোখে process আর thread দুটোই আসলে **task** — একই ডেটা স্ট্রাকচার (`task_struct`)। thread হলো এমন task যারা একে অন্যের সাথে মেমোরি শেয়ার করে। দুটোই তৈরি হয় `clone()` system call দিয়ে, শুধু কোন জিনিস শেয়ার হবে তার flag আলাদা।

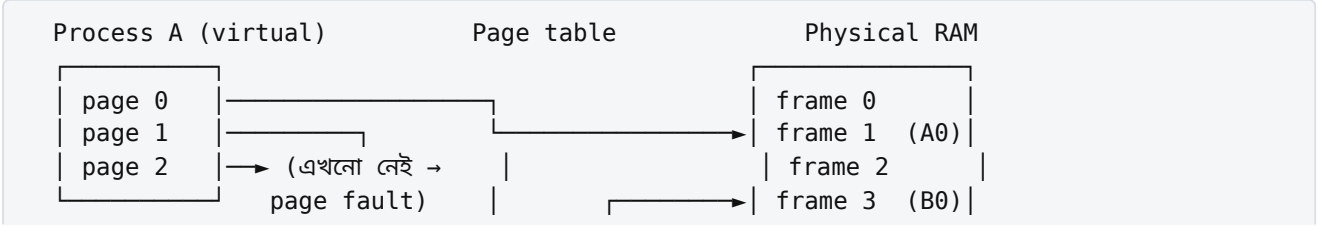
২.৭.২ Memory Management — RAM-এর হিসাবরক্ষক

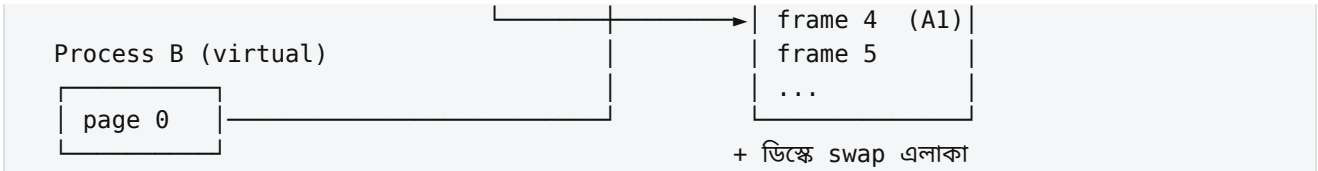
Virtual memory লিনাক্সের অন্যতম চমৎকার ধারণা।

উপমা: হোটেলের রুম নম্বর। একটা চেইন হোটেলে প্রতিটি অতিথিকে বলা হয় "তোমার রুম ১০১ থেকে ১১০"। প্রত্যেক অতিথিই ভাবে সে ১০১–১১০ পেয়েছে। কিন্তু রিসেপশনের খাতায় লেখা আছে — অতিথি A-র "১০১" আসলে ৫ম তলার ৫১২ নম্বর রুম, অতিথি B-র "১০১" আসলে ২য় তলার ২০৩। অতিথিরা কখনো একে অপরের রুমে ঢুকতে পারে না, কারণ তাদের নম্বরগুলো আলাদা আসল রুমে যায়।

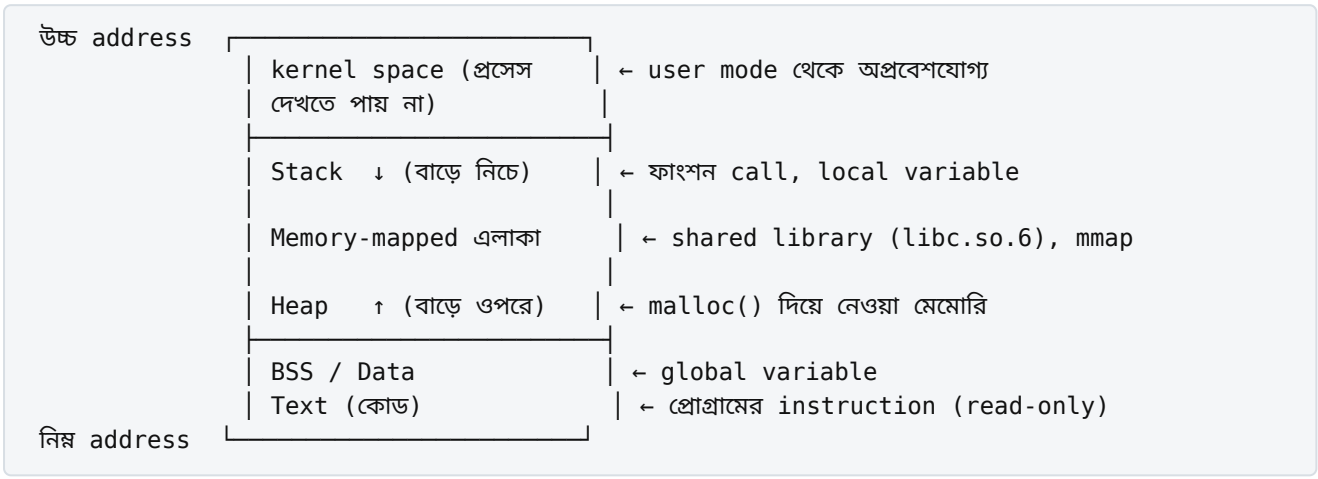
ঠিক এভাবে:

- প্রতিটি প্রসেস নিজের **virtual address space** পায়, যেন পুরো মেমোরি তারই।
- **page table** নামের "খাতা" (kernel পরিচালনা করে, MMU পড়ে) বলে দেয় কোন virtual address আসলে কোন physical RAM-এ।
- মেমোরি ভাগ করা হয় **page**-এ, সাধারণত ৪ KB করে (`getconf PAGESIZE` দিয়ে দেখো)।





একটা প্রসেসের virtual memory সাধারণত এভাবে সাজানো থাকে (ওপরে বেশি address):



নিজে দেখো — `cat` প্রসেসের নিজের memory map:

```
$ cat /proc/self/maps | head -5
5d6f1a2b0000-5d6f1a2b2000 r--p 00000000 103:02 1835092   /usr/bin/cat
5d6f1a2b2000-5d6f1a2b7000 r-xp 00002000 103:02 1835092   /usr/bin/cat
...
7f3c8e400000-7f3c8e428000 r--p 00000000 103:02 1840190   /usr/lib/x86_64-linux-
gnu/libc.so.6
```

(`/proc/self` সবসময় "যে প্রসেস এটা পড়ছে" তাকে নির্দেশ করে। address প্রতিবার আলাদা হবে — নিরাপত্তার জন্য ASLR, Address Space Layout Randomization।)

Memory management-এর গুরুত্বপূর্ণ ধারণাগুলো:

ধারণা	ব্যাখ্যা
Demand paging	প্রোগ্রাম চালু হলেই সব মেমোরিতে আনা হয় না; যখন যে page প্রথম দরকার হয় তখন আনা হয় (**page fault** হয়ে)
Copy-on-Write (CoW)	`fork()`-এর পর parent ও child একই page শেয়ার করে; কেউ লিখতে চাইলে তখনই কপি হয় (নিচে fork অংশে আবার দেখব)
Page cache	ডিস্ক থেকে পড়া ডেটা RAM-এ রেখে দেওয়া, যাতে দ্বিতীয়বার দ্রুত পাওয়া যায়। `free`-এর `buff/cache` কলাম এটাই
Swap	RAM ভরে গেলে কম-ব্যবহৃত page ডিস্কে (swap partition/file) সরিয়ে রাখা
OOM killer	RAM আর swap দুটোই শেষ হলে kernel একটা প্রসেস (সাধারণত সবচেয়ে বেশি মেমোরি খাওয়া) মেরে সিস্টেম বাঁচায়

ধারণা	ব্যাখ্যা
Shared memory	একাধিক প্রসেস একই physical page ব্যবহার করে — যেমন `libc.so.6` RAM-এ একবারই থাকে, সব প্রসেস শেয়ার করে

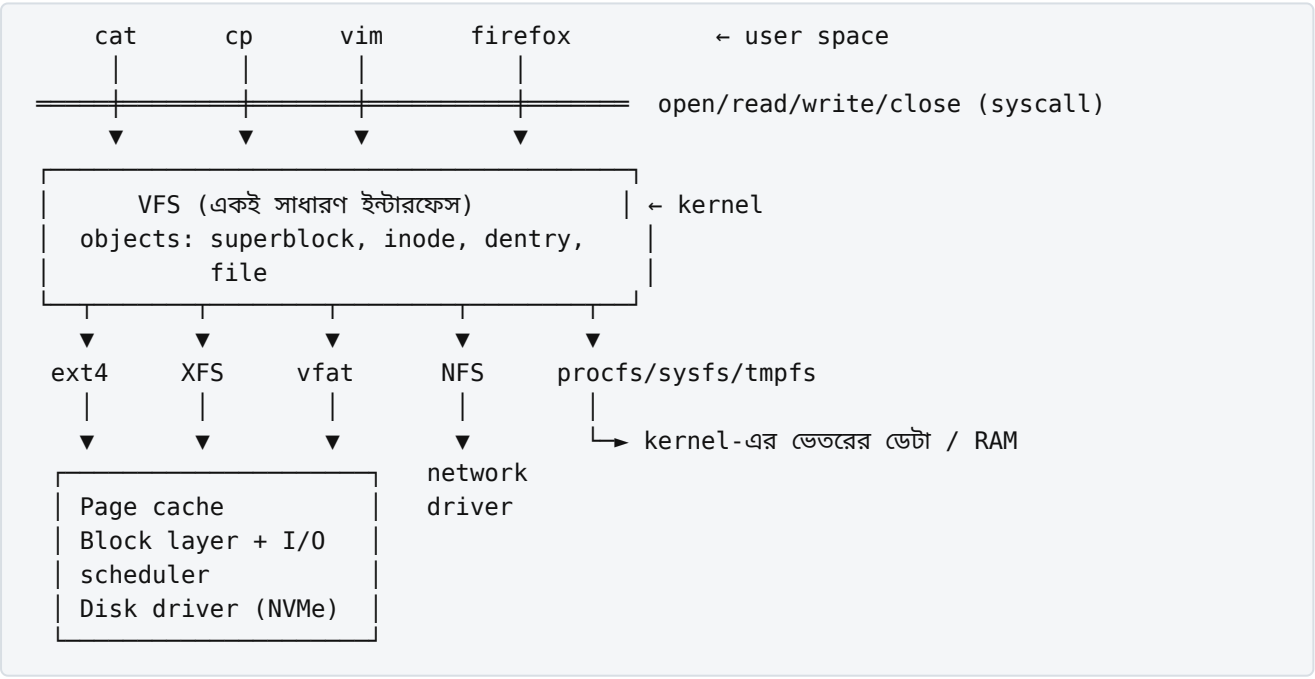
> 💡 **টিপস:** নতুনরা `free` দেখে ভয় পায়: "আমার ১৬ GB-র মধ্যে free মাত্র ১ GB!" ভয় পেয়ো না — লিনাক্স অব্যবহৃত RAM-কে page cache হিসেবে কাজে লাগায় ("unused RAM is wasted RAM")। আসল সূচক হলো **`available`** কলাম — প্রয়োজনে কতটা মেমোরি নতুন প্রোগ্রামকে দেওয়া যাবে।

২.৭.৩ VFS — Virtual File System

লিনাক্স ডজনখানেক ফাইলসিস্টেম বোঝে: ext4, XFS, Btrfs, FAT32/vfat, exFAT, NTFS, NFS (নেটওয়ার্ক), tmpfs (RAM-এ), procfs, sysfs... প্রতিটির ভেতরের ফরম্যাট সম্পূর্ণ আলাদা।

তাহলে `cat` প্রোগ্রাম কীভাবে জানে USB ড্রাইভের FAT ফাইল আর SSD-র ext4 ফাইল কীভাবে পড়তে হয়? উত্তর: **সে জানে না, জানার দরকারও নেই।** `cat` শুধু `open()`, `read()` ডাকে। kernel-এর **VFS (Virtual File System)** স্তর সেই অনুরোধ সঠিক ফাইলসিস্টেম driver-এর কাছে পাঠায়।

উপমা: সার্বজনীন বিদ্যুৎ অ্যাডাপ্টার। তোমার ল্যাপটপের চার্জার (application) একটাই প্লাগ চেনে। অ্যাডাপ্টার (VFS) বিভিন্ন দেশের সকেট (ext4, NTFS, NFS) এর সাথে মানিয়ে নেয়।



VFS-এর চারটা প্রধান object:

Object	কী প্রতিনিধিত্ব করে
superblock	একটা মাউন্ট করা পুরো ফাইলসিস্টেম (তার আকার, ধরন ইত্যাদি)

Object	কী প্রতিনিধিত্ব করে
inode	একটা ফাইলের metadata — মালিক, পারমিশন, আকার, ডেটা কোথায় (নাম বাদে!) — দেখো অধ্যায় ১১
dentry (directory entry)	একটা নাম ↔ inode সংযোগ; path খোঁজা দ্রুত করতে cache হয়
file	একটা প্রসেসের খোলা ফাইল — কোথায় পর্যন্ত পড়া হয়েছে (offset), কোন mode-এ খোলা

এই VFS-ই "everything is a file" দর্শনকে বাস্তব করে (নিচে ২.১০ দেখো)। ফাইলসিস্টেম হায়ারার্কি অধ্যায় ৭-এ, আর ডিস্ক-পার্টিশন-মাউন্ট অধ্যায় ৩২-এ।

২.৭.৪ Device Drivers — hardware-এর দোভাষী

প্রতিটি hardware-এর নিজস্ব "ভাষা" আছে (নির্দিষ্ট register, command, protocol)। **device driver** হলো kernel-এর সেই কোড, যেটা ঐ ভাষা জানে আর kernel-এর বাকি অংশকে একটা সাধারণ ইন্টারফেস দেয়। লিনাক্স kernel-এর সোর্স কোডের সবচেয়ে বড় অংশই হলো driver (`drivers/` ডিরেক্টরি)।

User space থেকে device-গুলো দেখা যায় `/dev` ডিরেক্টরিতে **device file** হিসেবে। দুই প্রধান ধরন:

ধরন	`ls -l`-এ প্রথম অক্ষর	বৈশিষ্ট্য	উদাহরণ
Character device	`c`	ডেটা বাইট-স্রোত হিসেবে, ক্রমানুসারে	`/dev/tty`, `/dev/null`, `/dev/random`, সিরিয়াল পোর্ট
Block device	`b`	নির্দিষ্ট আকারের block-এ, যেকোনো জায়গা থেকে পড়া-লেখা যায়	`/dev/sda`, `/dev/nvme0n1`, `/dev/loop0`

```
$ ls -l /dev/sda /dev/null /dev/tty
brw-rw---- 1 root disk 8, 0 Nov 12 09:10 /dev/sda
crw-rw-rw- 1 root root 1, 3 Nov 12 09:10 /dev/null
crw-rw-rw- 1 root tty  5, 0 Nov 12 09:15 /dev/tty
```

সাধারণ ফাইলের আকারের জায়গায় এখানে দুটো সংখ্যা: **major number** (কোন driver — যেমন 8 = SCSI/SATA disk driver) আর **minor number** (সেই driver-এর কোন device — 0 = প্রথম ডিস্ক)। kernel এই সংখ্যা দেখেই বোঝে অনুরোধ কোন driver-এর কাছে যাবে।

আধুনিক লিনাক্সে `/dev` একটা `devtmpfs` ফাইলসিস্টেম, যেখানে kernel নিজেই device node তৈরি করে, আর **udev** (systemd-udevd) daemon নিয়ম অনুযায়ী পারমিশন ঠিক করে, বন্ধুসুলভ symlink বানায় (যেমন `/dev/disk/by-uuid/...`), প্রয়োজনীয় module লোড করে। তুমি pendrive লাগালে কী ঘটে:

```
USB লাগালে → hardware interrupt → kernel-এর USB driver device চিনল
  → kernel একটা "uevent" পাঠাল → udev ধরল → module লোড, /dev/sdb তৈরি
  → desktop (udisks) জানতে পেল → ফাইল ম্যানেজারে আইকন হাজির
```

hardware ও driver দেখার কয়েকটা কমান্ড:

```
$ lspci -k | grep -A 2 -i network
02:00.0 Network controller: Intel Corporation Wi-Fi 6 AX201
        Subsystem: Intel Corporation Device 0070
        Kernel driver in use: iwlwifi

$ lsusb
Bus 001 Device 003: ID 046d:c52b Logitech, Inc. Unifying Receiver
...
```

`lspci -k`-এর "Kernel driver in use" লাইনটা বলে দেয় কোন driver ঐ device চালাচ্ছে — Wi-Fi কাজ না করলে troubleshooting-এর প্রথম ধাপ। (`lspci` ও `lsusb` যথাক্রমে `pciutils` ও `usbutils` প্যাকেজে থাকে, minimal সার্ভারে আলাদা ইনস্টল লাগতে পারে।)

২.৭.৫ Networking — Network Stack

kernel-এর ভেতরে একটা সম্পূর্ণ **TCP/IP network stack** আছে। যখন Firefox একটা ওয়েবসাইট খোলে:

```
Firefox:  socket() → connect() → send() / recv()       (user space)
═══════════════════════════════════════════════════════
Socket layer                                           (kernel)
  ↓
TCP (নির্ভরযোগ্যতা, ক্রম, পুনঃপ্রেরণ)  / UDP
  ↓
IP (ঠিকানা, routing — কোন পথে যাবে)
  ↓
Netfilter (firewall নিয়ম: nftables/iptables) — পথে পথে পরীক্ষা
  ↓
Network device layer + driver (e1000e, iwlwifi ...)
═══════════════════════════════════════════════════════
NIC (network card) → তার/বাতাস → ইন্টারনেট             (hardware)
```

লক্ষ করো: TCP, IP, firewall — সবকিছু kernel-এর ভেতরে। Firefox শুধু "socket"-এ লেখে-পড়ে, ঠিক যেমন ফাইলে লেখে-পড়ে। socket-ও একটা file descriptor! নেটওয়ার্কের তত্ত্ব অধ্যায় ৩৬-এ, কমান্ড অধ্যায় ৩৭-এ, আর firewall অধ্যায় ৩৯-এ।

২.৭.৬ অন্যান্য গুরুত্বপূর্ণ kernel উপব্যবস্থা

সংক্ষেপে আরও কিছু, যেগুলো পরের অধ্যায়ে কাজে লাগবে:

- **IPC (Inter-Process Communication):** প্রসেসদের মধ্যে কথা বলার উপায় — pipe, signal, Unix domain socket, shared memory, message queue।
- **Security:** user/group ভিত্তিক পারমিশন, **capabilities** (root-এর ক্ষমতাকে ছোট ছোট টুকরোয় ভাগ করা), **LSM** (Linux Security Modules) — SELinux (Fedora/RHEL-এ ডিফল্ট), AppArmor (Ubuntu/Debian-এ ডিফল্ট)। দেখো অধ্যায় ৪৫।
- **namespaces ও cgroups:** প্রসেসকে আলাদা "জগৎ" দেখানো আর তার resource সীমিত করা — এই দুটো দিয়েই Docker/Podman container তৈরি হয়। দেখো অধ্যায় ৪৮।
- **Timekeeping:** সময় রাখা, timer চালানো।

২.৮ System Call — user space থেকে kernel-এর দরজা

System call কী?

System call (syscall) হলো kernel-এর কাছে একটা প্রসেসের আনুষ্ঠানিক অনুরোধ — "আমার হয়ে এই কাজটা করে দাও"। user space-এর প্রোগ্রাম hardware বা অন্যের ডেটায় হাত দিতে পারে না, তাই যা-ই করতে চাক — ফাইল খোলা, স্ক্রিনে লেখা, নেটওয়ার্কে পাঠানো, নতুন প্রসেস তৈরি, এমনকি নিজেকে বন্ধ করা — সব system call দিয়ে।

উপমা: ব্যাংকের কাউন্টার। তুমি ব্যাংকের ভল্টে ঢুকে নিজে টাকা তুলতে পারো না। তুমি কাউন্টারে একটা নির্দিষ্ট ফর্ম (system call নম্বর + তথ্য) জমা দাও। ক্যাশিয়ার (kernel) যাচাই করে তোমার অ্যাকাউন্টে টাকা আছে কিনা (পারমিশন), তারপর ভল্ট থেকে এনে দেয় — অথবা "অনুমতি নেই" বলে ফিরিয়ে দেয় (error code)।

গুরুত্বপূর্ণ কিছু system call

ক্যাটাগরি	System call	কাজ
ফাইল	`openat`, `read`, `write`, `close`	ফাইল খোলা, পড়া, লেখা, বন্ধ
ফাইল তথ্য	`stat` / `newfstatat` / `statx`, `lseek`	metadata জানা, অবস্থান সরানো
ডিরেক্টরি	`getdents64`, `mkdir`, `unlink`, `rename`	ডিরেক্টরির তালিকা, তৈরি, মোছা
প্রসেস	`clone` (`fork`), `execve`, `exit_group`, `wait4`	প্রসেস তৈরি, প্রোগ্রাম চালানো, শেষ, অপেক্ষা
মেমোরি	`mmap`, `munmap`, `brk`, `mprotect`	মেমোরি নেওয়া-ছাড়া-সুরক্ষা
নেটওয়ার্ক	`socket`, `connect`, `bind`, `listen`, `accept`, `sendto`, `recvfrom`	নেটওয়ার্ক যোগাযোগ
signal	`kill`, `rt_sigaction`	signal পাঠানো ও handler বসানো
অন্যান্য	`ioctl`, `getpid`, `uname`	device নিয়ন্ত্রণ, তথ্য

x86_64 লিনাক্সে system call-এর সংখ্যা ৩০০-এর বেশি (প্রতিটির একটা নম্বর আছে; যেমন x86_64-এ `read` = 0, `write` = 1)। পূর্ণ তালিকা: `man 2 syscalls`।

> 💡 **টিপস:** `man` পেজের section নম্বর মনে রাখো: `man 1` = সাধারণ কমান্ড, `man 2` = system call, `man 3` = library ফাংশন। তাই `man 2 write` system call-টা দেখায়, আর `man 1 write` দেখায় অন্য user-কে মেসেজ পাঠানোর `write` কমান্ড! (`man`-এর সব section অধ্যায় ৫-এ।)

ভেতরে কীভাবে কাজ করে

ধরো একটা C প্রোগ্রাম `printf("Hello\n")` লিখল। ধাপগুলো (x86_64):

```
① printf("Hello\n")            ← তোমার কোড (user space)
       │
② glibc: লেখাটা format করে, buffer-এ রাখে, পরে write() wrapper ডাকে
       │
③ glibc-র write() wrapper:
       rax = 1        (syscall নম্বর: write)
       rdi = 1        (fd 1 = stdout)
       rsi = buffer-এর address
       rdx = 6        (কত বাইট)
       `syscall` instruction  ──────────┐
═══════════════════════════════════════╪═══  ring 3 → ring 0
       ┌──────────────────────────────────┘
④ kernel entry: register সংরক্ষণ, rax দেখে syscall table থেকে
   sys_write খুঁজে বের করে
       │
⑤ sys_write: fd 1 কোন "file"? → একটা terminal (tty) → tty driver
   → টার্মিনাল এমুলেটর → স্ক্রিনে "Hello"
       │
⑥ ফলাফল (লেখা বাইট সংখ্যা = 6, বা ঋণাত্মক error) rax-এ রেখে
   `sysret` দিয়ে ফেরত                  ring 0 → ring 3
       │
⑦ glibc: ফলাফল ঋণাত্মক হলে errno সেট করে −1 ফেরত দেয়
```

ARM64-এ একই কাজ `svc #0` instruction দিয়ে হয়, syscall নম্বর থাকে `x8` register-এ।

errno: system call ব্যর্থ হলে একটা error code দেয় — যেমন `ENOENT` (No such file or directory), `EACCES` (Permission denied), `EEXIST` (File exists)। টার্মিনালে যে বাংলা-ইংরেজি error মেসেজ দেখো, সেগুলোর বেশিরভাগই এই errno থেকে আসে।

vDSO — syscall ছাড়া syscall

কিছু অনুরোধ এত ঘন ঘন হয় (যেমন "এখন কয়টা বাজে?" — `clock_gettime`, `gettimeofday`) যে প্রতিবার ring বদলানো অপচয়। তাই kernel প্রতিটি প্রসেসের মেমোরিতে একটা ছোট shared library বসিয়ে দেয় — **vDSO** (virtual dynamic shared object)। এই ফাংশনগুলো user space-এ বসেই উত্তর দেয়। `ldd`-এর output-এ `linux-vdso.so.1` দেখলে বুঝবে এটাই সেই জিনিস।

strace — system call চোখে দেখা

`strace` একটা অসাধারণ টুল। এটা একটা প্রোগ্রাম চালায় আর সে যত system call করে, সব একটা একটা করে দেখায় — প্রোগ্রাম আর kernel-এর মধ্যকার কথোপকথনের রেকর্ডিং! (ইনস্টল: `sudo apt install strace` বা `sudo dnf install strace`।)

Syntax:

```
strace [অপশন] কমান্ড [আর্গুমেন্ট...]
strace [অপশন] -p PID
```

অপশন	কাজ
`-c`	প্রতিটি call-এর বদলে শেষে সারসংক্ষেপ টেবিল (কোন syscall কতবার, কত সময়)
`-e trace=openat,read`	শুধু নির্দিষ্ট syscall দেখাও
`-e trace=%file`	ফাইল-নাম নেয় এমন সব syscall (এছাড়া `%process`, `%network`, `%memory` ইত্যাদি)
`-f`	child প্রসেস/thread-ও অনুসরণ করো
`-p PID`	আগে থেকে চলা প্রসেসে যুক্ত হও
`-o ফাইল`	output ফাইলে লেখো
`-t` / `-tt`	প্রতিটি লাইনে সময় / মাইক্রোসেকেন্ড সহ সময়
`-T`	প্রতিটি syscall-এ কত সময় লেগেছে
`-s N`	string-এর সর্বোচ্চ N অক্ষর দেখাও (ডিফল্ট 32)
`-y`	file descriptor-এর পাশে তার path দেখাও

উদাহরণ ১ — `echo` কীভাবে লেখে:

```
$ strace -e trace=write /bin/echo হ্যালো
write(1, "\340\246\271\340\247\215\340\246\257\340\246\276\340\246\262\340\247\213\n",
19হ্যালো
) = 19
+++ exited with 0 +++
```

দেখো: `echo` আসলে `write(1, ..., 19)` system call করেছে — fd 1 (stdout)-এ ১৯ বাইট (বাংলা অক্ষর UTF-8-এ প্রতিটি ৩ বাইট, ৬টা code point = ১৮ বাইট + newline)। strace ঐ বাইটগুলো octal escape হিসেবে দেখায়, আর আসল লেখাটা একই লাইনে স্ক্রিনে চলে আসে, কারণ strace নিজের output stderr-এ দেয়।

উদাহরণ ২ — একটা পুরো প্রোগ্রামের কথোপকথন (সংক্ষিপ্ত):

```
$ strace cat /etc/hostname
execve("/usr/bin/cat", ["cat", "/etc/hostname"], 0x7ffd5c8e1a38 /* 31 vars */) = 0
brk(NULL)                               = 0x5612d4e3a000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f1b2c6f4000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
...
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0"..., 832) = 832
...
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=5, ...}) = 0
read(3, "mypc\n", 131072)               = 5
write(1, "mypc\n", 5mypc
)                   = 5
```

```
read(3, "", 131072)                     = 0
close(3)                                = 0
close(1)                                = 0
close(2)                                = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

এটা পড়তে শেখো, অনেক রহস্য পরিষ্কার হবে:

1. `execve(...)` — প্রোগ্রাম `/usr/bin/cat` লোড হলো।
2. `brk`, `mmap` — মেমোরি নেওয়া হচ্ছে।
3. `access("/etc/ld.so.preload")` → `ENOENT` — dynamic loader খুঁজল কোনো preload library আছে কিনা; নেই, স্বাভাবিক।
4. `openat("/etc/ld.so.cache")`, `openat(".../libc.so.6")` — dynamic loader glibc খুঁজে লোড করছে। `\177ELF` হলো লিনাক্স executable ফাইলের "magic number" (ELF = Executable and Linkable Format)।
5. `openat("/etc/hostname") = 3` — আসল কাজ শুরু। ফাইল খুলে **fd 3** পাওয়া গেল (0, 1, 2 আগে থেকেই stdin/stdout/stderr)।
6. `read(3, ...) = 5` — ৫ বাইট পড়ল; `write(1, ...) = 5` — stdout-এ ৫ বাইট লিখল।
7. `read(3, "", ...) = 0` — 0 মানে ফাইল শেষ (EOF)।
8. `close`, `exit_group(0)` — সব বন্ধ, exit status 0 (সফল)।

(তোমার সিস্টেমে লাইবেরির path, address, buffer আকার একটু আলাদা হতে পারে; Fedora/RHEL-এ libc থাকে `/lib64/libc.so.6`-এ।)

উদাহরণ ৩ — সারসংক্ষেপ (`-c`):

```
$ strace -c ls > /dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 23.41    0.000110           5        21           mmap
 14.04    0.000066           7         9           openat
 ...
  2.13    0.000010           5         2         2 statfs
------ ----------- ----------- --------- --------- ----------------
100.00    0.000470           3       127        10 total
```

একটা সাধারণ `ls`-ও ১০০-র বেশি system call করে! (সংখ্যা সিস্টেম অনুযায়ী আলাদা হবে।)

বাস্তব troubleshooting কৌশল: একটা প্রোগ্রাম বলছে "config file not found" কিন্তু কোন path-এ খুঁজছে বলছে না? —

```
$ strace -f -e trace=%file -o trace.txt ./myapp
$ grep ENOENT trace.txt
openat(AT_FDCWD, "/etc/myapp/config.yml", O_RDONLY) = -1 ENOENT (No such file or
directory)
```

এখন তুমি জানো ঠিক কোন ফাইলটা দরকার। এটা লিনাক্স এক্সপার্টদের প্রিয় কৌশল।

> ⚠️ **সতর্কতা:** `strace` প্রোগ্রামকে অনেক ধীর করে দেয় — ব্যস্ত production সার্ভারে সাবধানে ব্যবহার করো। অন্য user-এর প্রসেসে `-p` দিয়ে যুক্ত হতে root লাগবে; Ubuntu-তে Yama নিরাপত্তা নিয়মের (`/proc/sys/kernel/yama/ptrace_scope` = 1) কারণে নিজের চলমান প্রসেসে attach করতেও সাধারণত `sudo` লাগে।

> 💡 **টিপস:** `ltrace` নামে একটা সহোদর টুল আছে, যেটা system call না, **library call** (যেমন `printf`, `malloc`) দেখায়। সব distro-তে ডিফল্ট ইনস্টল থাকে না।

> 📝 **মনে রাখো:** উদাহরণ ১-এ `echo` না লিখে `/bin/echo` লিখেছি। কারণ `echo` bash-এর একটা **builtin** — bash নিজেই সেটা চালায়, আলাদা প্রোগ্রাম চালায় না। `strace` শুধু আলাদা প্রোগ্রাম চালাতে পারে, তাই বাইরের `/bin/echo` ফাইলটা দিয়েছি। builtin বনাম বাইরের কমান্ড নিয়ে নিচে ২.১২-এ আরও বলব।

২.৯ Libraries — glibc ও বন্ধুরা

Library কী?

Library হলো আগে থেকে লেখা, বারবার ব্যবহারযোগ্য ফাংশনের সংগ্রহ। প্রতিটি প্রোগ্রামারকে যদি স্ক্রিনে লেখা, স্ট্রিং তুলনা, মেমোরি নেওয়ার কোড নিজে লিখতে হতো — আর সেটাও প্রতিটি CPU-র syscall নিয়ম মেনে assembly-তে — জীবন দুর্বিষহ হতো।

লিনাক্সে সবচেয়ে গুরুত্বপূর্ণ library হলো **C standard library**। বেশিরভাগ distro-তে এটা **glibc** (GNU C Library), ফাইলের নাম `libc.so.6`। এটা দুটো কাজ করে:

1. **system call wrapper:** `open()`, `read()`, `fork()` ইত্যাদি — register-এ মান বসিয়ে `syscall` instruction চালানোর ঝামেলা লুকিয়ে রাখে।
2. **উচ্চস্তরের সুবিধা:** `printf()` (format করা + buffer করা), `malloc()` (ছোট ছোট মেমোরি টুকরো হিসাব রাখা; ভেতরে `brk` / `mmap` ব্যবহার করে), `strlen()`, `qsort()` — এগুলোর অনেকগুলো কোনো system call-ই করে না।

```
তোমার প্রোগ্রাম:   printf()   malloc()   strlen()   fopen()
                       |          |          |          |
glibc:           format+buffer  heap      সাধারণ     FILE* তৈরি
                       |       manager    লুপ (কোনো      |
```

```
                        ▼          ▼       syscall নেই)   ▼
  syscall:          write()   brk()/mmap()              openat()
```

অন্যান্য C library-ও আছে: **musl** (ছোট, Alpine Linux-এ ব্যবহার হয় — Docker image-এ জনপ্রিয়), Android-এ **Bionic**।

> 💡 **টিপস:** Python, Java, Node.js, Go — এরা কোথায়? Python ও Node-এর interpreter নিজে C/C++-এ লেখা, glibc ব্যবহার করে। Go একটু ব্যতিক্রম — Go-র runtime অনেক সময় glibc বাদ দিয়ে নিজেই সরাসরি syscall করে। কিন্তু যে ভাষাই হোক, শেষ পর্যন্ত kernel-এর দরজা একটাই: **system call**।

Static বনাম Dynamic linking

	Static linking	Dynamic linking
library কোথায়	প্রোগ্রামের ফাইলের ভেতরেই কপি করা	আলাদা `.so` ফাইলে, চালানোর সময় লোড
ফাইলের আকার	বড়	ছোট
RAM ব্যবহার	প্রতিটি প্রোগ্রামে আলাদা কপি	সব প্রোগ্রাম এক কপি শেয়ার করে
নিরাপত্তা আপডেট	প্রতিটি প্রোগ্রাম আবার build করতে হয়	শুধু library আপডেট করলেই সবাই পায়
এক্সটেনশন	`.a` (archive)	`.so` (shared object)

লিনাক্সের বেশিরভাগ প্রোগ্রাম **dynamic**। চালু হওয়ার সময় **dynamic linker/loader** (`ld-linux-x86-64.so.2`, অর্থাৎ `ld.so`) প্রয়োজনীয় `.so` ফাইল খুঁজে মেমোরিতে map করে। খোঁজে `/etc/ld.so.cache` (যেটা `ldconfig` কমান্ড তৈরি করে), `LD_LIBRARY_PATH` environment variable ইত্যাদি থেকে।

`ldd` — একটা প্রোগ্রাম কোন কোন library চায়

```
$ ldd /usr/bin/ls
        linux-vdso.so.1 (0x00007ffd3a7e6000)
        libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f6b1c8a0000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f6b1c600000)
        libpcre2-8.so.0 => /lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x00007f6b1c560000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f6b1c900000)
```

- `linux-vdso.so.1` — উপরে বলা vDSO; কোনো ফাইল নেই, kernel মেমোরিতে বসিয়ে দেয়।
- `libc.so.6` — glibc।
- `/lib64/ld-linux-x86-64.so.2` — dynamic loader নিজেই।

glibc-র ভার্সন দেখতে:

```
$ ldd --version | head -1
ldd (Ubuntu GLIBC 2.39-0ubuntu8.3) 2.39
```

(Ubuntu 24.04-এ glibc 2.39, Debian 12-এ 2.36, RHEL 9-এ 2.34।)

> ⚠️ **সতর্কতা:** অবিশ্বস্ত বা অজানা প্রোগ্রামে `ldd` চালাবে না। কিছু ক্ষেত্রে `ldd` প্রোগ্রামটাকে আসলে আংশিক চালিয়ে ফেলতে পারে (man পেজেও সতর্ক করা আছে)। অজানা ফাইলের জন্য `objdump -p ফাইল | grep NEEDED` নিরাপদ।

> ⚠️ **সতর্কতা:** কখনো glibc হাত দিয়ে মুছবে না বা অন্য ভার্সন দিয়ে জোর করে বদলাবে না। প্রায় সব প্রোগ্রাম — `ls`, `bash`, এমনকি `sudo` — এর ওপর নির্ভরশীল। glibc ভাঙলে সিস্টেম কার্যত অচল হয়ে যায়, রিকভারির জন্য live USB লাগে।

library কম্পাইল ও লিংক করার ব্যবহারিক দিক অধ্যায় ৩১-এ।

২.১০ Shell — তোমার আর সিস্টেমের মাঝে দোভাষী

Shell (খোলস) হলো একটা সাধারণ user-space প্রোগ্রাম, যেটা তোমার লেখা কমান্ড পড়ে, ব্যাখ্যা করে, আর প্রয়োজনীয় প্রোগ্রাম চালায়। নাম "shell" কারণ এটা kernel-কে ঘিরে থাকা বাইরের খোলস — তুমি খোলসের সাথে কথা বলো, খোলস ভেতরের kernel-এর সাথে।

গুরুত্বপূর্ণ কথা: **shell-এর কোনো বিশেষ ক্ষমতা নেই।** সে Firefox বা vim-এর মতোই একটা সাধারণ প্রোগ্রাম, শুধু তার কাজ অন্য প্রোগ্রাম চালানো। তাই অনেক রকম shell আছে, আর তুমি চাইলে বদলাতে পারো:

Shell	বৈশিষ্ট্য
`bash`	Bourne Again Shell; Ubuntu, Debian, Fedora, RHEL-এ user-দের ডিফল্ট
`sh`	POSIX shell; Debian/Ubuntu-তে `/bin/sh` আসলে `dash` (ছোট, দ্রুত), Fedora/RHEL-এ `bash`
`zsh`	অনেক ফিচার; macOS-এর ডিফল্ট; Oh My Zsh জনপ্রিয়
`fish`	নতুনদের জন্য বন্ধুসুলভ, স্বয়ংক্রিয় সাজেশন

```
$ echo $SHELL
/bin/bash
$ cat /etc/shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/dash
...
```

Terminal বনাম Shell: terminal (যেমন GNOME Terminal, Konsole) হলো জানালা — কীবোর্ড ইনপুট নেয়, লেখা দেখায়। shell হলো তার ভেতরে চলা প্রোগ্রাম, যে কমান্ড বোঝে। দুটোর মাঝে kernel-এর একটা **pseudo-terminal (pty)** থাকে। বিস্তারিত অধ্যায় ৫-এ।

```
তুমি → [Terminal emulator] ⇄ /dev/pts/0 (kernel-এর pty + tty driver) ⇄ [bash] → প্রোগ্রাম
        (GUI জানালা)                                                        (কমান্ড ব্যাখ্যা)
```

২.১১ "Everything is a File" — লিনাক্সের দর্শন

Unix (এবং তার উত্তরসূরি লিনাক্স)-এর সবচেয়ে বিখ্যাত নীতি: **"সবকিছুই ফাইল"**। আরও সঠিকভাবে বললে — **"সবকিছুই file descriptor"**: যতটা সম্ভব সবকিছুকে একই ইন্টারফেস (`open` , `read` , `write` , `close`) দিয়ে ব্যবহারযোগ্য করা।

উপমা: সার্বজনীন রিমোট কন্ট্রোল। একটা রিমোটের চারটা বোতাম (open/read/write/close) দিয়ে তুমি টিভি, এসি, ফ্যান, লাইট — সব চালাতে পারো। প্রতিটি যন্ত্রের জন্য আলাদা রিমোট শিখতে হয় না।

এর ফলে কী সুবিধা? `cat` , `grep` , `echo` , `>` রিডাইরেকশন — যে টুল সাধারণ ফাইলে কাজ করে, সেগুলো hardware, kernel-এর তথ্য, প্রসেস — সবকিছুতেই কাজ করে!

"ফাইল"	আসলে কী	উদাহরণ ব্যবহার
`/home/rahim/note.txt`	সাধারণ ফাইল	`cat note.txt`
`/dev/sda`	পুরো হার্ডডিস্ক	`sudo dd if=/dev/sda ...` (সাবধান!)
`/dev/null`	"কালো গর্ত" — যা লেখো সব হারিয়ে যায়, পড়লে খালি	`command > /dev/null`
`/dev/zero`	অসীম শূন্য বাইটের উৎস	ফাঁকা ফাইল বানাতে
`/dev/urandom`	অসীম random বাইট	`head -c 16 /dev/urandom | base64`
`/dev/tty`	তোমার বর্তমান টার্মিনাল	`echo hi > /dev/tty`
`/proc/cpuinfo`	CPU-র তথ্য (kernel তৎক্ষণাৎ তৈরি করে)	`cat /proc/cpuinfo`
`/proc/<PID>/`	একটা প্রসেসের সব তথ্য	`ls /proc/1/`
`/sys/class/...`	device ও driver-এর তথ্য ও সেটিং	ব্যাটারি, ব্রাইটনেস
pipe (`|`)	দুই প্রসেসের মাঝে ডেটা চ্যানেল	`ls | grep txt`
socket	নেটওয়ার্ক বা লোকাল সংযোগ	`/run/...sock`

নিজে চেষ্টা করো:

```
$ cat /proc/version
Linux version 6.8.0-45-generic (buildd@lcy02-amd64-075) (x86_64-linux-gnu-gcc-13 ...)
#45-Ubuntu SMP PREEMPT_DYNAMIC ...

$ cat /proc/uptime
12345.67 98765.43
```

```
$ cat /sys/class/power_supply/BAT0/capacity
87
```

(`BAT0` শুধু ল্যাপটপে থাকবে, নাম `BAT1` বা অন্যরকমও হতে পারে।) `/proc/uptime`-এর প্রথম সংখ্যা = সিস্টেম কত সেকেন্ড চালু, দ্বিতীয় = সব CPU মিলিয়ে কত সেকেন্ড অলস ছিল।

মজার ব্যাপার: `/proc` আর `/sys`-এর ফাইলগুলো **ডিস্কে নেই**! এগুলো **virtual filesystem** (procfs, sysfs)। তুমি `cat` করলে kernel সেই মুহূর্তে উত্তরটা তৈরি করে দেয়। এজন্য `ls -l` এগুলোর আকার প্রায়ই 0 দেখায়, অথচ `cat` করলে লেখা আসে।

File Descriptor — খোলা ফাইলের টোকেন

প্রসেস যখন একটা ফাইল খোলে, kernel তাকে একটা ছোট পূর্ণসংখ্যা দেয় — **file descriptor (fd)**। উপমা: দর্জির দোকানে কাপড় জমা দিলে একটা টোকেন নম্বর দেয়; পরে শুধু টোকেন দেখালেই চলে। প্রতিটি প্রসেস তিনটা fd নিয়ে জন্মায়:

fd	নাম	ডিফল্ট সংযোগ
0	stdin (standard input)	কীবোর্ড (টার্মিনাল)
1	stdout (standard output)	স্ক্রিন (টার্মিনাল)
2	stderr (standard error)	স্ক্রিন (টার্মিনাল)

```
$ ls -l /proc/$$/fd
lrwx------ 1 rahim rahim 64 Nov 12 10:00 0 -> /dev/pts/0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 1 -> /dev/pts/0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 2 -> /dev/pts/0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 255 -> /dev/pts/0
```

(`$$` = বর্তমান shell-এর PID। bash নিজের ব্যবহারের জন্য fd 255 খোলা রাখে।) তিনটাই একই টার্মিনাল `/dev/pts/0`-এর দিকে নির্দেশ করছে। রিডাইরেকশন (`>`, `<`, `2>`) আর পাইপ মানে আসলে এই fd-গুলোকে অন্য কোথাও জুড়ে দেওয়া — অধ্যায় ১৯-এ পুরোটা।

> 📝 **মনে রাখো:** "everything is a file" একটা দর্শন, পরম সত্য না। যেমন নেটওয়ার্ক ইন্টারফেস (`eth0`, `wlan0`) `/dev`-এ কোনো ফাইল হিসেবে থাকে না (যদিও `/sys/class/net/`-এ তাদের তথ্য আছে), আর socket তৈরি হয় `socket()` দিয়ে, `open()` দিয়ে না। তবে একবার তৈরি হলে socket-ও একটা fd, তাতে `read`/`write` চলে।

২.১২ একটা কমান্ড চালালে ভেতরে কী হয় — fork, exec, wait

এখন সব জ্ঞান একসাথে করি। তুমি টার্মিনালে লিখলে:

```
$ ls -l /home
```

আর Enter চাপলে। এক সেকেন্ডের ভগ্নাংশে যা ঘটে:

ধাপ ১: কীবোর্ড থেকে shell পর্যন্ত

তুমি প্রতিটি কী চাপলে → কীবোর্ড interrupt (বা GUI-তে display server-এর মাধ্যমে terminal emulator ঘটনা পায়) → terminal emulator অক্ষরগুলো pty-তে লেখে → bash `read()` system call দিয়ে সেগুলো পড়ে। (bash লাইন এডিটিং নিজে করে, readline library দিয়ে — তাই Tab completion, Ctrl+A ইত্যাদি কাজ করে; দেখো অধ্যায় ৬।) Enter চাপলে bash পুরো লাইনটা হাতে পায়।

ধাপ ২: Parsing — লাইনটা টুকরো করা

bash লাইনটা শব্দে ভাঙে: `ls`, `-l`, `/home`। তারপর expansion করে: variable (`$HOME`), wildcard (`*.txt`), `~`, ইত্যাদি। কোনো alias থাকলে বদলায় (Ubuntu-তে `ls` আসলে প্রায়ই `ls --color=auto`-এর alias)।

ধাপ ৩: কমান্ড কোথায়? — খোঁজার ক্রম

bash এই ক্রমে খোঁজে:

1. **alias** আছে কি?
2. **keyword** (`if`, `for`...) বা **function**?
3. **builtin** — bash নিজেই জানে এমন কমান্ড (`cd`, `echo`, `pwd`, `export`, `exit`)? থাকলে bash নিজেই চালায়, **কোনো নতুন প্রসেস হয় না**।
4. নাহলে বাইরের প্রোগ্রাম — `$PATH` variable-এর ডিরেক্টরিগুলোতে বাঁ থেকে ডানে খোঁজে। (খুঁজে পেলে মনে রাখে — `hash` টেবিলে।)

```
$ type ls cd
ls is aliased to `ls --color=auto'
cd is a shell builtin

$ type -a ls
ls is aliased to `ls --color=auto'
ls is /usr/bin/ls
ls is /bin/ls

$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
```

(`/bin` আধুনিক distro-তে `/usr/bin`-এর symlink, তাই একই ফাইল দুইবার দেখাচ্ছে।)

> 💡 **টিপস:** `cd` কেন builtin হতেই হবে? কারণ "বর্তমান ডিরেক্টরি" প্রতিটি প্রসেসের নিজস্ব সম্পত্তি। যদি `cd` আলাদা প্রসেস হতো, সে নিজের ডিরেক্টরি বদলে মরে যেত — তোমার shell যেখানে ছিল সেখানেই থাকত! এজন্য shell-এর নিজের অবস্থা বদলানো কমান্ড (`cd` , `export` , `exit` , `source`) builtin।

ধাপ ৪: fork() — নিজের একটা কপি বানানো

লিনাক্সে নতুন প্রসেস তৈরির মূল নিয়ম অদ্ভুত শোনাতে পারে: **নতুন প্রোগ্রাম সরাসরি "জন্ম" দেওয়া হয় না। আগে চলমান প্রসেস নিজের একটা কপি বানায় (fork), তারপর কপিটা নিজেকে নতুন প্রোগ্রাম দিয়ে বদলে ফেলে (exec)।**

bash `fork()` ডাকে (glibc-তে এটা আসলে `clone()` system call)। kernel:

- নতুন একটা `task_struct` তৈরি করে, নতুন **PID** দেয়;
- parent-এর প্রায় সবকিছু কপি করে: খোলা fd (তাই child-এরও stdin/stdout/stderr = একই টার্মিনাল), বর্তমান ডিরেক্টরি, environment variable, user ID;
- মেমোরি **কপি করে না** — **Copy-on-Write**: parent ও child একই physical page শেয়ার করে, page-গুলো read-only চিহ্নিত হয়। কেউ লিখতে চাইলে শুধু তখনই ঐ page-টার কপি হয়। এতে fork প্রায় তাৎক্ষণিক।

`fork()` একবার ডাকা হয়, কিন্তু **দুইবার ফেরে**: parent-এ ফেরত দেয় child-এর PID, আর child-এ ফেরত দেয় 0। এভাবে কোড জানে সে parent না child।

উপমা: জেরক্স করা রেসিপি বই। মা (bash) রান্নাঘরে কাজ করছেন। মেয়ে এলো (fork) — শুরুতে সে মায়ের হুবহু কপি, একই রান্নাঘর, একই চাবির গোছা (fd)। তারপর মেয়ে তার হাতের বইটা ফেলে সম্পূর্ণ নতুন বই (ls) খুলে সেই অনুযায়ী কাজ শুরু করল (exec)। মা অপেক্ষা করছেন মেয়ের কাজ শেষ হওয়ার (wait)।

ধাপ ৫: execve() — নতুন প্রোগ্রামে রূপান্তর

child প্রসেস `execve("/usr/bin/ls", ["ls", "--color=auto", "-l", "/home"], envp)` ডাকে। kernel:

- পুরনো মেমোরি (bash-এর কোড ও ডেটার কপি) ফেলে দেয়;
- `/usr/bin/ls` ফাইলটা খোলে, ELF header পড়ে, পারমিশন (execute bit) যাচাই করে;
- নতুন মেমোরি তৈরি করে, dynamic loader (`ld-linux-x86-64.so.2`) map করে;
- **PID একই থাকে**, খোলা fd-গুলোও থাকে (যেগুলো close-on-exec চিহ্নিত না) — এজন্যই `ls` -এর output তোমার টার্মিনালে আসে।

execve সফল হলে কখনো ফেরে না — কারণ যে কোড ডেকেছিল, সে আর নেই! এরপর dynamic loader glibc লোড করে, আর `ls` -এর `main()` চলতে শুরু করে। এই পর্যায়টাই উপরে strace উদাহরণে `execve` → `openat(libc.so.6)` হিসেবে দেখেছ।

ধাপ ৬: ls তার কাজ করে

`ls -l /home` করে: `openat("/home")` → `getdents64` (ডিরেক্টরির এন্ট্রি পড়া) → প্রতিটি এন্ট্রির জন্য `statx` / `newfstatat` / `lstat` (মালিক, পারমিশন, আকার জানতে; ঠিক কোনটা তা glibc ভার্সনের ওপর নির্ভর করে) → user ID-কে নাম বানাতে `/etc/passwd` পড়া → `write(1, ...)` দিয়ে ফলাফল লেখা। VFS, ফাইলসিস্টেম driver, page cache, ডিস্ক driver — সবাই এখানে কাজ করছে।

ধাপ ৭: exit ও wait — শেষকৃত্য

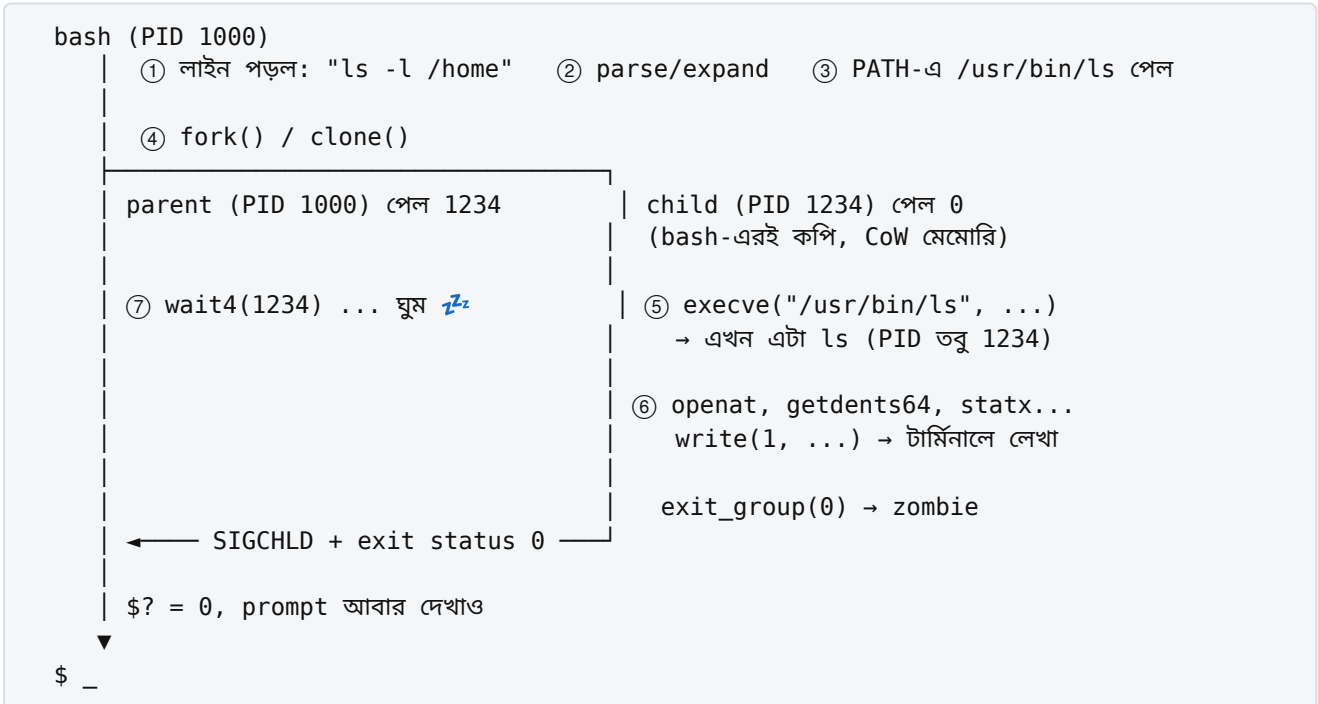
`ls` শেষে `exit_group(0)` ডাকে। kernel তার মেমোরি, fd সব মুক্ত করে, কিন্তু প্রসেসের একটা ছোট্ট রেকর্ড (PID ও **exit status**) রেখে দেয়, যতক্ষণ না parent সেটা সংগ্রহ করে। এই মৃত-কিন্তু-সংগৃহীত-না অবস্থাকে বলে **zombie** (`ps` -এ state `Z`)।

এদিকে bash `wait4()` (glibc-র `waitpid()`) দিয়ে ঘুমিয়ে অপেক্ষা করছিল। child মরলে kernel bash-কে `SIGCHLD` signal দেয় ও জাগায়; bash exit status নেয় (zombie দূর হয়), `$?` variable-এ রাখে, তারপর আবার prompt দেখায়:

```
$ ls -l /home
total 4
drwxr-x--- 25 rahim rahim 4096 Nov 12 09:30 rahim
$ echo $?
0
```

`$?` = শেষ কমান্ডের exit status। 0 = সফল, অন্য কিছু = কোনো সমস্যা।

পুরো ছবি একসাথে



নিজে প্রমাণ দেখো — strace দিয়ে bash-এর fork/exec:

```
$ strace -f -e trace=clone,clone3,execve,wait4 bash -c 'ls /tmp > /dev/null; true'
execve("/usr/bin/bash", ["bash", "-c", "ls /tmp > /dev/null; true"], 0x7ffc... /* 31
vars */) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f...) = 5678
strace: Process 5678 attached
[pid  5678] execve("/usr/bin/ls", ["ls", "/tmp"], 0x55... /* 31 vars */) = 0
[pid  5678] +++ exited with 0 +++
wait4(-1, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 5678
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=5678, si_uid=1000,
si_status=0, si_utime=0, si_stime=0} ---
+++ exited with 0 +++
```

(শেষে `; true` দিয়েছি, কারণ `bash -c 'ls /tmp'` এর মতো একটাই কমান্ড হলে bash অনেক সময় fork না করে নিজেই সরাসরি `exec` করে ফেলে — একটা অপটিমাইজেশন। PID, address, ক্রম তোমার সিস্টেমে আলাদা হতে পারে; কিছু সিস্টেমে `wait4` আর `SIGCHLD` লাইনের ক্রম উল্টো দেখাতে পারে।)

> 📝 **মনে রাখো:** লিনাক্সের প্রতিটি প্রসেসের একটা parent আছে। সবার আদি পিতা হলো **PID 1** — আধুনিক distro-তে `systemd` — যেটা kernel বুটের শেষে নিজে চালু করে। কোনো প্রসেসের parent আগে মরে গেলে সেই "এতিম" প্রসেসকে PID 1 (বা একটা subreaper) দত্তক নেয়। `pstree` কমান্ড দিয়ে এই পারিবারিক গাছ দেখতে পারো। systemd অধ্যায় ২৬-এ।

```
$ pstree -p | head -5
systemd(1)─┬─ModemManager(812)─┬─{ModemManager}(830)
           ├─NetworkManager(790)─┬─{NetworkManager}(815)
           ├─gnome-terminal-(2210)─┬─bash(2230)───pstree(2301)
           ...
```

২.১৩ বুট থেকে prompt — architecture-এর টাইমলাইন (সংক্ষেপে)

পুরো আর্কিটেকচার কোন ক্রমে "জেগে ওঠে", এক নজরে (বিস্তারিত অধ্যায় ২৭-এ):

```
পাওয়ার অন → UEFI/BIOS firmware (hardware পরীক্ষা)
          → Bootloader (GRUB / systemd-boot)
          → Kernel (vmlinuz) + initramfs লোড → kernel hardware চেনে, driver/module লোড
          → আসল root filesystem মাউন্ট
          → PID 1 = systemd (প্রথম user-space প্রসেস)
          → সার্ভিস চালু (নেটওয়ার্ক, লগ, ssh...)
          → লগইন স্ক্রিন (GDM ইত্যাদি) বা টেক্সট login
          → তোমার shell / desktop
```

kernel-এর বুট মেসেজ দেখতে পারো:

```
$ sudo dmesg | head -3
[    0.000000] Linux version 6.8.0-45-generic (buildd@lcy02-amd64-075) ...
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-45-generic root=UUID=... ro
quiet splash
[    0.000000] KERNEL supported cpus:
```

(অনেক distro-তে সাধারণ user-এর জন্য `dmesg` বন্ধ থাকে — `kernel.dmesg_restrict` — তাই `sudo`; বিকল্প `journalctl -k`। লগ অধ্যায় ২৮-এ।)

২.১৪ Architecture-সম্পর্কিত কীবোর্ড শর্টকাট

এই শর্টকাটগুলো আসলে kernel-এর কাজ — tty driver কী চাপা দেখে প্রসেসকে **signal** পাঠায়, বা kernel নিজে সরাসরি সাড়া দেয়।

টার্মিনালের signal-শর্টকাট (tty driver পাঠায়)

শর্টকাট	কী ঘটে (ভেতরে)	ফলাফল
`Ctrl + C`	foreground প্রসেসে `SIGINT`	প্রোগ্রাম থামে (সাধারণত)
`Ctrl + Z`	`SIGTSTP`	প্রোগ্রাম সাময়িক স্থগিত (`fg` দিয়ে ফেরত — অধ্যায় ২৫)
`Ctrl + \`	`SIGQUIT`	প্রোগ্রাম থামে, অনুমতি থাকলে core dump
`Ctrl + D`	কোনো signal না; ইনপুট লাইনের শুরুতে চাপলে `read()` 0 ফেরত দেয় = **EOF**	`cat`-এর ইনপুট শেষ; খালি prompt-এ চাপলে shell বন্ধ
`Ctrl + S`	tty-র output থামায় (XOFF flow control)	স্ক্রিন "জমে গেছে" মনে হয়
`Ctrl + Q`	output আবার চালু (XON)	`Ctrl+S`-এর পর জমে গেলে এটা চাপো

```
$ stty -a | grep -o 'intr = [^;]*\|quit = [^;]*\|eof = [^;]*\|susp = [^;]*'
intr = ^C
quit = ^\
eof = ^D
susp = ^Z
```

`stty -a` দিয়ে দেখা যায় tty driver কোন কী-কে কোন কাজে লাগাচ্ছে — প্রমাণ যে এটা kernel-এর tty স্তরের সেটিং, bash-এর না।

Virtual console ও সিস্টেম শর্টকাট

শর্টকাট	কাজ
`Ctrl + Alt + F1` … `F6`	kernel-এর virtual console (টেক্সট টার্মিনাল, tty1–tty6) এ যাওয়া। কোনটায় GUI আছে তা distro অনুযায়ী আলাদা — Ubuntu-তে GDM সাধারণত tty1, তোমার desktop session প্রায়ই tty2-এ
`Alt + F1` … `F6` / `Alt + ←/→`	টেক্সট console থেকে আরেক console-এ (ইতিমধ্যে টেক্সট মোডে থাকলে)
`Ctrl + Alt + Delete`	টেক্সট console-এ systemd-কে reboot সংকেত দেয় (`ctrl-alt-del.target`); GNOME-এ সাধারণত log-out ডায়ালগ
`Alt + SysRq + <কী>`	**Magic SysRq** — kernel সরাসরি সাড়া দেয়, user space জমে গেলেও

Magic SysRq — শেষ ভরসা

kernel-এর ভেতরে বসানো একটা জরুরি কীবোর্ড। সিস্টেম পুরো জমে গেলে (মাউস নড়ছে না, কিছু কাজ করছে না) ডিস্ক নষ্ট না করে নিরাপদে reboot করার জন্য বিখ্যাত ক্রম **REISUB** (মনে রাখার বাক্য: "**R**eboot **E**ven **I**f **S**ystem **U**tterly **B**roken"), প্রতিটির মাঝে কয়েক সেকেন্ড বিরতি দিয়ে, `Alt + SysRq` চেপে ধরে:

কী	কাজ
`R`	কীবোর্ড raw mode থেকে ফেরত নেয়
`E`	init (PID 1) বাদে সব প্রসেসে `SIGTERM`
`I`	init বাদে সব প্রসেসে `SIGKILL`
`S`	সব ফাইলসিস্টেম sync (RAM-এর ডেটা ডিস্কে লেখা)
`U`	সব ফাইলসিস্টেম read-only হিসেবে remount
`B`	তৎক্ষণাৎ reboot

```
$ cat /proc/sys/kernel/sysrq
176
```

এই মান একটা bitmask — কোন SysRq ফাংশন অনুমোদিত। `1` = সব চালু, `0` = সব বন্ধ। Ubuntu-র ডিফল্ট 176 (= 128 + 32 + 16: reboot, remount read-only, sync অনুমোদিত, কিন্তু `E` / `I` এর মতো signal পাঠানো বন্ধ)। Fedora/RHEL-এ ডিফল্ট সাধারণত 16 (শুধু sync)। অনেক ল্যাপটপে SysRq কী হলো `PrtSc`, কখনো `Fn` সহ চাপতে হয়।

> ⚠️ **সতর্কতা:** Magic SysRq সত্যিকারের জরুরি অবস্থার জন্য। `B` সরাসরি reboot করে — `S` আর `U` না করলে না-সংরক্ষিত ডেটা হারাবে, ফাইলসিস্টেম ক্ষতিগ্রস্ত হতে পারে। মজা করে চাপবে না, বিশেষ করে সার্ভারে।

সব শর্টকাটের পূর্ণ তালিকা: পরিশিষ্ট ক।

২.১৫ সাধারণ ভুল ধারণা

ভুল ধারণা	সঠিক
"লিনাক্স = Ubuntu"	Ubuntu একটা distribution; লিনাক্স হলো kernel, যার ওপর শত শত distro তৈরি (অধ্যায় ৩)
"root হলে kernel mode-এ চলে"	না; root-ও user space-এ, শুধু বেশি অনুমতি পায়
"shell = terminal"	terminal জানালা, shell তার ভেতরে চলা প্রোগ্রাম
"free মেমোরি কম মানে RAM শেষ"	page cache দেখো; আসল সূচক `available`
"`/proc` এর ফাইল ডিস্কে থাকে"	এগুলো virtual, kernel তৎক্ষণাৎ তৈরি করে
"monolithic মানে পুরনো, অনমনীয়"	লিনাক্স module দিয়ে চলন্ত অবস্থায় বদলানো যায়
"প্রতিটি কমান্ড আলাদা প্রসেস"	builtin (`cd`, `echo` ...) bash নিজেই চালায়

হাতে-কলমে অনুশীলন

যেকোনো লিনাক্স মেশিন, VM বা WSL-এ (WSL2-ও আসল লিনাক্স kernel চালায়) চেষ্টা করো। `strace` না থাকলে আগে ইনস্টল করো:

```
$ sudo apt install strace       # Ubuntu/Debian
$ sudo dnf install strace       # Fedora/RHEL
```

ল্যাব ১: তোমার সিস্টেমের পরিচয়পত্র

```
$ uname -srm
$ cat /etc/os-release | head -3
$ lscpu | grep 'Model name'
$ free -h
$ ldd --version | head -1
```

একটা খাতায় লেখো: kernel ভার্সন, architecture, distro, CPU, RAM, glibc ভার্সন। প্রশ্ন: তোমার kernel কি CFS না EEVDF scheduler ব্যবহার করে? (ইঙ্গিত: ভার্সন 6.6 এর আগে না পরে?)

ল্যাব ২: Kernel module অনুসন্ধান

```
$ lsmod | wc -l                 # কতগুলো module লোড আছে (হেডারসহ)
$ lsmod | sort -k2 -n -r | head -5   # সবচেয়ে বড় ৫টা module
$ modinfo -F description ext4   # ext4 module-এর বর্ণনা
$ lspci -k | grep -A3 -i vga     # তোমার গ্রাফিক্স কার্ড কোন driver ব্যবহার করে
```

(`ext4` তোমার সিস্টেমে built-in হলে `modinfo` তবুও তথ্য দেখাতে পারে, বা "not found" বলতে পারে — দুটোই স্বাভাবিক।)

ল্যাব ৩: System call গোয়েন্দাগিরি

```
$ strace -c ls /etc > /dev/null
$ strace -e trace=openat cat /etc/hostname
$ strace -e trace=%file ls /nonexistent 2>&1 | grep ENOENT
```

প্রশ্ন: `strace -c` অনুযায়ী `ls` সবচেয়ে বেশি কোন system call করেছে? তৃতীয় কমান্ডে `ls` কোন system call দিয়ে বুঝল ফাইলটা নেই?

ল্যাব ৪: Everything is a file

```
$ cat /proc/cpuinfo | grep 'model name' | head -1
$ cat /proc/meminfo | head -3
$ echo "আমি সরাসরি টার্মিনাল ফাইলে লিখছি" > /dev/tty
$ tty                            # তোমার টার্মিনাল ফাইলের নাম
$ ls -l /proc/$$/fd
$ head -c 12 /dev/urandom | base64
$ echo "হারিয়ে যাবে" > /dev/null ; echo $?
```

দ্বিতীয় একটা টার্মিনাল খোলো, সেখানে `tty` চালাও (ধরো `/dev/pts/1`)। এখন প্রথম টার্মিনাল থেকে `echo "হ্যালো পাশের জানালা" > /dev/pts/1` লেখো — দ্বিতীয় জানালায় লেখাটা হাজির হবে!

ল্যাব ৫: fork/exec চোখে দেখা

```
$ echo "আমার shell-এর PID: $$"
$ bash                           # একটা child shell চালাও
$ echo "নতুন shell-এর PID: $$, parent: $PPID"
$ pstree -p $PPID
$ exit
$ type cd ls pwd echo
$ strace -f -e trace=clone,clone3,execve bash -c 'date; true' 2>&1 | grep -E
'clone|execve'
```

লক্ষ করো: child shell-এর `$PPID` তোমার প্রথম shell-এর `$$`-এর সমান। `strace` output-এ দুটো `execve` দেখবে — একটা `bash`-এর, একটা `date`-এর — আর মাঝে একটা `clone`।

ল্যাব ৬: builtin বনাম বাইরের কমান্ড

```
$ type -a echo pwd
$ strace -c -f bash -c 'for i in 1 2 3; do echo hi; done' 2>&1 | grep -E
'clone|execve|total'
$ strace -c -f bash -c 'for i in 1 2 3; do /bin/echo hi; done' 2>&1 | grep -E
'clone|execve|total'
```

দ্বিতীয় ক্ষেত্রে `clone` আর `execve` অনেক বেশি হবে — কারণ প্রতিবার নতুন প্রসেস। এজন্যই বড় script-এ builtin ব্যবহার দ্রুততর।

সারসংক্ষেপ

মূল ধারণা:

- লিনাক্সের স্তর: Hardware → Kernel → System call interface → Libraries (glibc) → Shell → Applications।
- Kernel ring 0-তে (kernel space) পূর্ণ ক্ষমতায়, সব user প্রোগ্রাম (root-এরটাও) ring 3-তে (user space) সীমিত ক্ষমতায় চলে।
- লিনাক্স monolithic kernel, কিন্তু loadable module দিয়ে নমনীয়।
- Kernel-এর কাজ: scheduling (CFS → EEVDF), memory (virtual memory, paging, CoW, page cache, swap, OOM), VFS, device drivers (`/dev` , char/block, udev), networking (TCP/IP stack, netfilter)।
- System call হলো user space থেকে kernel-এ যাওয়ার একমাত্র আনুষ্ঠানিক দরজা; `strace` দিয়ে দেখা যায়।
- "Everything is a file" — একই `open/read/write/close` দিয়ে ফাইল, device, kernel তথ্য, pipe, socket।
- কমান্ড চালানো = parse → খোঁজা (alias/builtin/PATH) → `fork` (clone, CoW) → `execve` → চলা → `exit` → parent-এর `wait` → `$?` ।

দ্রুত রেফারেন্স:

কমান্ড / শর্টকাট	কাজ
`uname -a` / `uname -r`	kernel-এর সব তথ্য / ভার্সন
`cat /proc/version`	kernel ভার্সন ও build তথ্য
`lscpu` , `free -h`	CPU ও মেমোরি তথ্য
`lsmod`	লোড করা kernel module
`modinfo <mod>`	module-এর বিবরণ
`sudo modprobe <mod>` / `-r`	module লোড / আনলোড (dependency সহ)
`lspci -k` , `lsusb`	PCI/USB device ও তাদের driver
`strace <cmd>`	system call দেখা
`strace -c` , `-f` , `-e trace=` , `-p`	সারসংক্ষেপ, child অনুসরণ, ফিল্টার, চলমান প্রসেস
`ldd <prog>`	প্রোগ্রামের shared library
`ldd --version`	glibc ভার্সন
`type -a <cmd>`	কমান্ডটা alias/builtin/ফাইল — কোনটা

কমান্ড / শর্টকাট	কাজ
`echo $PATH`, `echo $?`, `echo $$`	PATH, শেষ exit status, shell-এর PID
`ls -l /proc/$$/fd`	shell-এর খোলা file descriptor
`pstree -p`	প্রসেসের পারিবারিক গাছ
`sudo dmesg` / `journalctl -k`	kernel মেসেজ
`getconf PAGESIZE`	মেমোরি page-এর আকার
`Ctrl+C` / `Ctrl+Z` / `Ctrl+\`	SIGINT / SIGTSTP / SIGQUIT
`Ctrl+D`	EOF (ইনপুট শেষ)
`Ctrl+S` / `Ctrl+Q`	output থামানো / চালু
`Ctrl+Alt+F1…F6`	virtual console বদলানো
`Alt+SysRq+R,E,I,S,U,B`	জমে যাওয়া সিস্টেম নিরাপদে reboot

নিজেকে যাচাই করো

1. লিনাক্স সিস্টেমের ছয়টা স্তর নিচ থেকে ওপরে ক্রমানুসারে লেখো।
2. kernel space আর user space কেন আলাদা রাখা হয়? অন্তত তিনটা কারণ বলো।
3. root user হিসেবে চলা প্রোগ্রাম কি kernel space-এ চলে? ব্যাখ্যা করো।
4. monolithic kernel আর microkernel-এর মূল পার্থক্য কী? লিনাক্স monolithic হয়েও কীভাবে নমনীয়?
5. একটা `.ko` ফাইল কী? `modprobe` আর `insmod`-এর পার্থক্য কী?
6. `printf("hi\n")` থেকে স্ক্রিনে লেখা পর্যন্ত যাত্রায় glibc ও kernel কে কী করে?
7. `strace` output-এ `openat(...) = -1 ENOENT` মানে কী? এটা troubleshooting-এ কীভাবে কাজে লাগে?
8. `/proc/cpuinfo`-এর আকার `ls -l`-এ 0 দেখায়, অথচ `cat` করলে অনেক লেখা আসে। কেন?
9. `fork()` কেন দ্রুত, যদিও সে পুরো প্রসেসের "কপি" বানায়? `fork()` কয়বার ফেরে, কী ফেরত দেয়?
10. `cd` কেন একটা আলাদা প্রোগ্রাম হতে পারে না?

▼ উত্তর দেখো

1. Hardware → Kernel → System call interface → Libraries (glibc) → Shell → Applications।

2. (ক) **স্থিতিশীলতা** — একটা প্রোগ্রাম crash করলে পুরো সিস্টেম পড়ে না; (খ) **নিরাপত্তা** — এক প্রোগ্রাম অন্যের মেমোরি বা hardware-এ হাত দিতে পারে না; (গ) **ন্যায্য বণ্টন** — CPU/RAM/ডিস্ক kernel ভাগ করে, কেউ একা দখল করতে পারে না; (ঘ) **বিমূর্তকরণ** — প্রোগ্রামারকে hardware-এর খুঁটিনাটি জানতে হয় না।

3. না। root-এর প্রোগ্রামও user space-এ (ring 3) চলে এবং system call দিয়েই kernel-এর সেবা নেয়। পার্থক্য হলো, kernel root-এর অনুরোধ বেশি অনুমোদন করে (যেমন যেকোনো ফাইল পড়া, module লোড করা)।

4. monolithic kernel-এ scheduler, memory, filesystem, driver, network সব একই kernel space-এ একটা প্রোগ্রাম হিসেবে চলে (দ্রুত, কিন্তু একটা driver বাগ পুরো সিস্টেম ফেলে দিতে পারে)। microkernel-এ kernel-এ শুধু মৌলিক জিনিস থাকে, বাকিগুলো user space-এ আলাদা server হিসেবে চলে, message দিয়ে কথা বলে। লিনাক্স **loadable kernel module** দিয়ে চলন্ত অবস্থায় driver ইত্যাদি যোগ-বিয়োগ করতে পারে, reboot ছাড়াই।

5. `.ko` (kernel object) হলো একটা loadable kernel module ফাইল — kernel-এর একটা টুকরো যা চলন্ত kernel-এ লোড করা যায়। `modprobe` নাম দিয়ে module খোঁজে (`/lib/modules/$(uname -r)` থেকে) এবং `modules.dep` দেখে dependency-ও লোড করে; `insmod` শুধু নির্দিষ্ট path-এর একটা ফাইল লোড করে, dependency দেখে না।

6. glibc-র `printf` লেখাটা format করে buffer-এ রাখে, তারপর `write()` wrapper ডাকে, যেটা register-এ syscall নম্বর (x86_64-এ write = 1), fd (1), buffer address, দৈর্ঘ্য বসিয়ে `syscall` instruction চালায়। CPU ring 0-তে যায়; kernel syscall table থেকে `sys_write` চালায়, fd 1 যে টার্মিনাল (pty/tty) তার driver-এর কাছে ডেটা পাঠায়, ফলাফল ফেরত দিয়ে ring 3-তে ফেরে। glibc ফলাফল দেখে (error হলে `errno` সেট করে)।

7. প্রোগ্রামটা একটা ফাইল খুলতে চেয়েছিল, কিন্তু kernel "No such file or directory" error দিয়েছে। `strace -f -e trace=%file` চালিয়ে `ENOENT` খুঁজলে বোঝা যায় প্রোগ্রাম ঠিক কোন path-এ কোন ফাইল (config, library) খুঁজছে — যা প্রোগ্রামের নিজের error মেসেজ প্রায়ই বলে না।

8. `/proc` একটা virtual filesystem (procfs) — ফাইলগুলো ডিস্কে নেই। পড়ার মুহূর্তে kernel লেখাটা তৈরি করে দেয়, তাই আগে থেকে "আকার" জানা থাকে না, 0 দেখায়।

9. কারণ **Copy-on-Write**: kernel মেমোরি আসলে কপি করে না; parent ও child একই physical page শেয়ার করে, শুধু কেউ লিখতে গেলে ঐ page-টা কপি হয়। আর বেশিরভাগ child তো সাথে সাথেই `execve` করে পুরো মেমোরি বদলে ফেলে। `fork()` একবার ডাকা হয় কিন্তু দুইবার ফেরে: parent-এ child-এর PID, child-এ 0 (ব্যর্থ হলে parent-এ −1)।

10. বর্তমান ডিরেক্টরি প্রতিটি প্রসেসের নিজস্ব সম্পত্তি। `cd` যদি আলাদা প্রোগ্রাম হতো, তাহলে fork হওয়া child প্রসেস নিজের ডিরেক্টরি বদলে exit করত — parent shell-এর ডিরেক্টরি অপরিবর্তিত থাকত। তাই shell-এর নিজের অবস্থা বদলাতে `cd` অবশ্যই shell-এর builtin হতে হবে।

অধ্যায় ৩: ডিস্ট্রিবিউশন বাছাই ও ইনস্টলেশন

এই অধ্যায়ে যা শিখবে

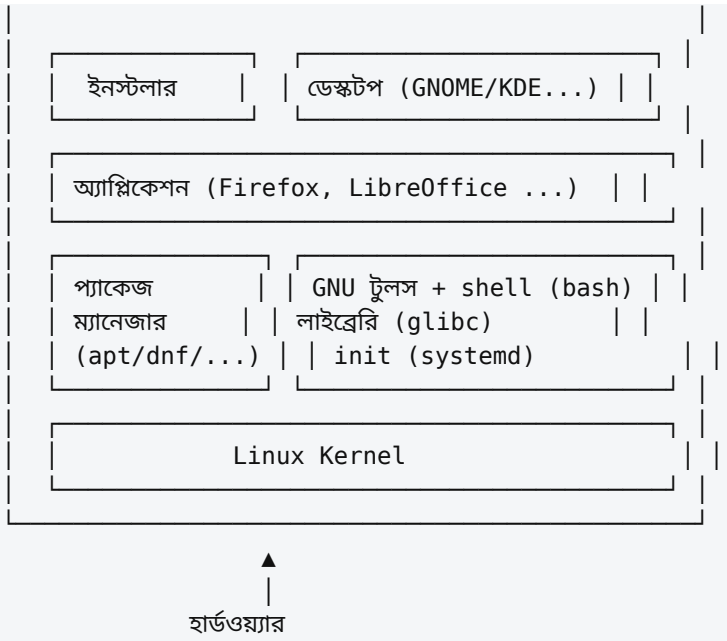
- **distro** (distribution) আসলে কী, আর এত distro কেন আছে
- বড় বড় distro পরিবার: Debian/Ubuntu, RHEL/Fedora/Rocky/Alma, Arch, SUSE, কোনটার কী বৈশিষ্ট্য
- **LTS** বনাম **rolling release** মডেল: কোনটা কখন ভালো
- একজন fresher-এর জন্য কোন distro দিয়ে শুরু করা উচিত, আর কেন
- ইনস্টল করার চারটি পদ্ধতি:
  - VirtualBox / VMware-এ **ভার্চুয়াল মেশিন (VM)**, ধাপে ধাপে
  - **WSL2**: Windows-এর ভেতরেই লিনাক্স
  - **Live USB** বানানো (Rufus, balenaEtcher, `dd`)
  - **Dual boot**: একই কম্পিউটারে Windows + Linux
- ISO ফাইল যাচাই করা (`sha256sum`)
- পার্টিশনিং-এর বেসিক: MBR/GPT, BIOS/UEFI, ESP, `/`, `/home`, swap
- প্রথম বুটের পরে কী কী করবে: আপডেট, ড্রাইভার, ব্যাকআপ
- সংশ্লিষ্ট কীবোর্ড শর্টকাট (BIOS/boot menu, VirtualBox, VMware, GRUB)

৩.১ Distro কী? আগে একটা উপমা

অধ্যায় ২-এ আমরা দেখেছি, "Linux" শব্দটা আসলে শুধু **kernel** (সিস্টেমের কেন্দ্রীয় অংশ, যা হার্ডওয়্যার আর সফটওয়্যারের মাঝে দোভাষীর কাজ করে)-এর নাম। কিন্তু শুধু kernel দিয়ে তুমি কিছুই করতে পারবে না। তোমার লাগবে shell, কমান্ড (`ls`, `cp`...), লাইব্রেরি (glibc), init system (systemd), প্যাকেজ ম্যানেজার, ডেস্কটপ, ব্রাউজার, ইনস্টলার...

Distribution (distro) = Linux kernel + GNU টুলস + অন্যান্য সফটওয়্যার + প্যাকেজ ম্যানেজার + ইনস্টলার + কনফিগারেশন + সাপোর্ট নীতি, সব একসাথে প্যাকেট করা একটা সম্পূর্ণ অপারেটিং সিস্টেম।

> 💡 **উপমা: গাড়ির ইঞ্জিন আর গাড়ি** Linux kernel হলো একটা গাড়ির **ইঞ্জিন**। Toyota, Honda, Tata সবাই ইঞ্জিনের চারপাশে বডি, সিট, চাকা, ড্যাশবোর্ড লাগিয়ে আলাদা আলাদা গাড়ি বানায়। ঠিক তেমনি Ubuntu, Fedora, Arch একই (বা প্রায় একই) kernel নিয়ে নিজেদের মতো করে "গাড়ি" বানায়। চালানোর মূল নিয়ম (স্টিয়ারিং, ব্রেক) সব গাড়িতেই এক, তাই একটা শিখলে বাকিগুলো সহজ।



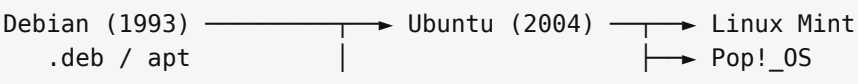
distro-গুলো একে অপরের থেকে আসলে কোথায় আলাদা?

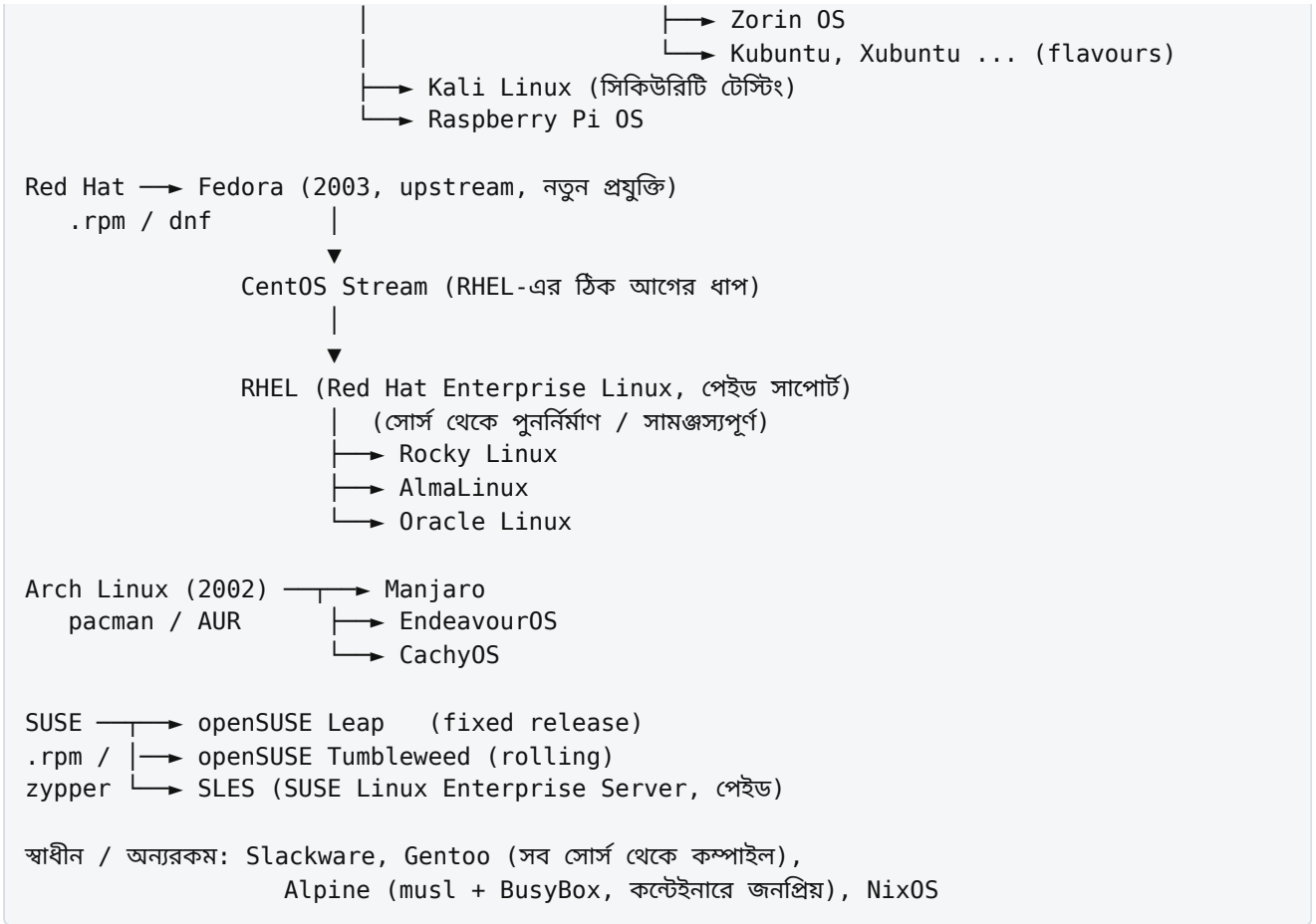
বিষয়	উদাহরণ
প্যাকেজ ফরম্যাট ও ম্যানেজার	`.deb` + `apt` (Debian/Ubuntu), `.rpm` + `dnf` (Fedora/RHEL), `pacman` (Arch), `.rpm` + `zypper` (SUSE)
রিলিজ মডেল	নির্দিষ্ট সময় পরপর ভার্সন (fixed/LTS), নাকি সবসময় আপডেট (rolling)
ডিফল্ট ডেস্কটপ	GNOME, KDE Plasma, Cinnamon, XFCE (অধ্যায় ৪)
ডিফল্ট কনফিগারেশন	ফায়ারওয়াল, SELinux/AppArmor, ফাইলসিস্টেম (ext4/Btrfs/XFS)
সাপোর্ট ও লক্ষ্য	ডেস্কটপ, সার্ভার, এন্টারপ্রাইজ, শেখা, পুরোনো হার্ডওয়্যার
কে বানায়	কমিউনিটি (Debian, Arch) নাকি কোম্পানি (Canonical→Ubuntu, Red Hat→RHEL, SUSE→SLES)

> 📝 **মনে রাখো:** distro বদলালে বেশিরভাগ কমান্ড (`ls`, `cd`, `grep`, `systemctl` ...) একই থাকে। মূলত বদলায় প্যাকেজ ইনস্টলের কমান্ড আর কিছু কনফিগ ফাইলের জায়গা। তাই এই বইয়ের ৯০% জিনিস যেকোনো distro-তে কাজ করবে।

৩.২ Distro পরিবার: বংশতালিকা

হাজারের বেশি distro আছে, কিন্তু বেশিরভাগই কয়েকটা "মূল" distro থেকে এসেছে। পরিবারগুলো চিনলে যেকোনো নতুন distro দেখেই বুঝবে সেটা কীভাবে চালাতে হবে।





৩.২.১ Debian / Ubuntu পরিবার

Debian সবচেয়ে পুরোনো ও সম্মানিত কমিউনিটি distro-গুলোর একটা। কোনো কোম্পানি এর মালিক না; হাজারো স্বেচ্ছাসেবক মিলে বানায়। এর মূলমন্ত্র **স্থিতিশীলতা (stability)**। Debian-এর তিনটি শাখা:

শাখা	মানে
stable	প্রকাশিত, পরীক্ষিত ভার্সন। Debian 12 "Bookworm" (২০২৩), Debian 13 "Trixie" (আগস্ট ২০২৫)
testing	পরের stable হওয়ার প্রস্তুতি
unstable ("sid")	সবচেয়ে নতুন প্যাকেজ, ডেভেলপারদের জন্য

Debian-এর ভার্সনের নাম Toy Story সিনেমার চরিত্র থেকে নেওয়া (Buster, Bullseye, Bookworm, Trixie...)।

Ubuntu বানায় Canonical কোম্পানি, Debian-এর ভিত্তির ওপর। Ubuntu-র লক্ষ্য ছিল "সাধারণ মানুষের জন্য লিনাক্স"। তাই ইনস্টল সহজ, ড্রাইভার সাপোর্ট ভালো, আর ইন্টারনেটে সবচেয়ে বেশি টিউটোরিয়াল Ubuntu নিয়েই। ভার্সন নম্বর = **বছর.মাস**: `24.04` মানে ২০২৪ সালের এপ্রিল।

- প্রতি ৬ মাসে একটা রিলিজ (এপ্রিল ও অক্টোবর)
- প্রতি ২ বছরে এপ্রিলের রিলিজটা **LTS** (24.04, 26.04...)
- Ubuntu-র নিজস্ব জিনিস: **Snap** প্যাকেজ, **Netplan** (নেটওয়ার্ক কনফিগ), **AppArmor**

Linux Mint: Ubuntu-ভিত্তিক, Cinnamon ডেস্কটপ, Windows ব্যবহারকারীদের কাছে খুব পরিচিত লাগে।

৩.২.২ Red Hat পরিবার: RHEL / Fedora / CentOS Stream / Rocky / Alma

এন্টারপ্রাইজ সার্ভারের দুনিয়ায় এই পরিবারের রাজত্ব। ব্যাংক, টেলিকম, সরকারি প্রতিষ্ঠানে প্রচুর RHEL চলে।

- **Fedora**: Red Hat-এর স্পনসরে কমিউনিটি distro। নতুন প্রযুক্তি (Wayland, PipeWire, Btrfs ডিফল্ট) সবার আগে Fedora-তে আসে। প্রায় ৬ মাস পরপর রিলিজ, প্রতিটার সাপোর্ট প্রায় ১৩ মাস। ডেভেলপারদের ডেস্কটপ হিসেবে খুব জনপ্রিয়।
- **CentOS Stream**: Fedora থেকে জিনিস পরিপক্ক হয়ে আসে এখানে, তারপর RHEL-এ। এটা RHEL-এর "পরের মাইনর ভার্সনের প্রিভিউ"।
- **RHEL**: পেইড সাবস্ক্রিপশন, ১০ বছরের লাইফসাইকেল, অফিশিয়াল সাপোর্ট। RHEL 9 (২০২২), RHEL 10 (২০২৫)। শেখার জন্য Red Hat-এর **ফ্রি Developer Subscription** দিয়ে বিনামূল্যে ব্যবহার করা যায়।
- **Rocky Linux** ও **AlmaLinux**: পুরোনো CentOS Linux বন্ধ হওয়ার পর (CentOS 8 শেষ হয় ডিসেম্বর ২০২১-এ, CentOS 7 শেষ হয় জুন ২০২৪-এ) এরা এসেছে RHEL-এর সঙ্গে সামঞ্জস্যপূর্ণ (compatible), ফ্রি বিকল্প হিসেবে। RHEL শিখতে চাইলে এগুলো দারুণ।

```
নতুন প্রযুক্তির প্রবাহ:

 Fedora  ──────►  CentOS Stream  ──────►  RHEL  ──────►  Rocky / Alma
(পরীক্ষাগার)      (প্রিভিউ)              (এন্টারপ্রাইজ)   (ফ্রি সামঞ্জস্যপূর্ণ)
```

> 💡 **টিপস:** তুমি যদি **RHCSA** (Red Hat Certified System Administrator) সার্টিফিকেশন দিতে চাও, তাহলে Rocky Linux / AlmaLinux / RHEL দিয়ে প্র্যাকটিস করো।

৩.২.৩ Arch পরিবার

Arch Linux এর দর্শন: **KISS** (Keep It Simple, Stupid) আর "ব্যবহারকারীই সব ঠিক করবে"। ডিফল্টে কোনো ডেস্কটপ নেই। তুমি নিজে কমান্ড দিয়ে সব ইনস্টল করবে (এখন `archinstall` নামে একটা গাইডেড স্ক্রিপ্টও আছে)।

- **Rolling release**: একবার ইনস্টল করো, সারাজীবন `pacman -Syu` দিয়ে আপডেট
- **AUR** (Arch User Repository): কমিউনিটির বানানো প্যাকেজ রেসিপির বিশাল ভান্ডার
- **ArchWiki**: লিনাক্সের অন্যতম সেরা ডকুমেন্টেশন। অন্য distro ব্যবহারকারীরাও এটা পড়ে

Manjaro, **EndeavourOS**: Arch-কে সহজ করে ইনস্টলার ও ডেস্কটপসহ দেয়।

> ⚠️ **সতর্কতা:** একদম প্রথম দিন Arch দিয়ে শুরু করলে হতাশ হওয়ার সম্ভাবনা বেশি। আগে Ubuntu/Mint/Fedora-তে বেসিক শিখে নাও, তারপর Arch চ্যালেঞ্জ হিসেবে নিও। তখন অনেক কিছু শিখবে।

৩.২.৪ SUSE পরিবার

জার্মান কোম্পানি SUSE। ইউরোপে এন্টারপ্রাইজে জনপ্রিয়।

- **openSUSE Leap**: fixed release, SLES-এর সঙ্গে কোড শেয়ার করে, স্থিতিশীল
- **openSUSE Tumbleweed**: rolling, কিন্তু **openQA** নামের স্বয়ংক্রিয় টেস্টিং পার হয়ে প্যাকেজ আসে, তাই rolling হয়েও তুলনামূলক নির্ভরযোগ্য
- **SLES**: পেইড এন্টারপ্রাইজ
- প্যাকেজ ম্যানেজার `zypper`। ঐতিহ্যগতভাবে **YaST** নামে একটা সর্বগ্রাসী সেটিংস টুল ছিল (নতুন Leap 16-এ ইনস্টলার হিসেবে Agama আর ম্যানেজমেন্টে Cockpit-এর দিকে সরে যাচ্ছে)।
- Btrfs + **Snapper** ডিফল্ট: আপডেটের আগে স্বয়ংক্রিয় স্ন্যাপশট, গোলমাল হলে আগের অবস্থায় ফিরে যাওয়া যায়।

৩.২.৫ পরিবারগুলোর তুলনা এক নজরে

পরিবার	প্যাকেজ ফরম্যাট	ম্যানেজার	আপডেট কমান্ড	রিলিজ মডেল	মূল ব্যবহার
Debian	`.deb`	`apt`	`sudo apt update && sudo apt upgrade`	fixed (~২ বছর)	সার্ভার, স্থিতিশীল ডেস্কটপ
Ubuntu	`.deb` (+snap)	`apt`	একই	৬ মাস / LTS ২ বছর	ডেস্কটপ, ক্লাউড, সার্ভার
Fedora	`.rpm`	`dnf`	`sudo dnf upgrade --refresh`	~৬ মাস	ডেভেলপার ডেস্কটপ
RHEL/Rocky/Alma	`.rpm`	`dnf`	`sudo dnf update`	মেজর ~৩ বছর, ১০ বছর সাপোর্ট	এন্টারপ্রাইজ সার্ভার
Arch	`.pkg.tar.zst`	`pacman`	`sudo pacman -Syu`	rolling	উন্নত ব্যবহারকারী
openSUSE	`.rpm`	`zypper`	Leap: `sudo zypper up`, Tumbleweed: `sudo zypper dup`	Leap fixed / TW rolling	ডেস্কটপ, সার্ভার

প্যাকেজ ম্যানেজমেন্ট বিস্তারিত দেখবে অধ্যায় ৩০-এ।

৩.৩ LTS বনাম Rolling Release

Fixed / Point release এবং LTS

Fixed release মানে distro নির্দিষ্ট সময় পরপর একটা "ভার্সন" বের করে (Ubuntu 24.04, Debian 12)। ওই ভার্সনের ভেতরে প্যাকেজের **মূল ভার্সন জমিয়ে রাখা হয় (freeze)**; শুধু নিরাপত্তা ও বাগ ফিক্স আসে

(backport)।

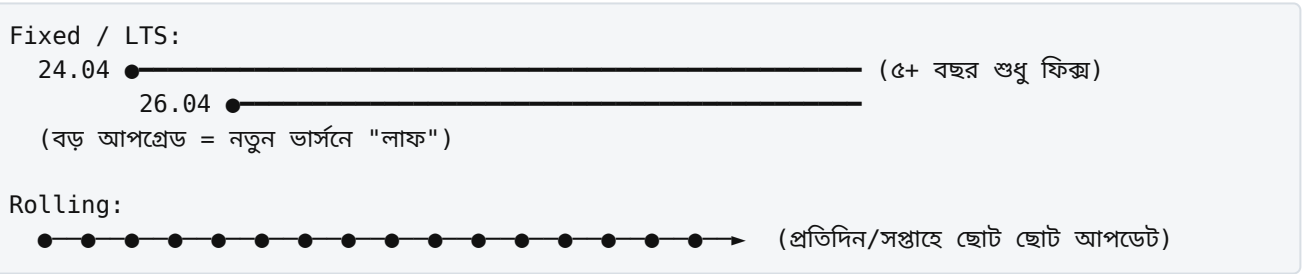
LTS (Long Term Support) = দীর্ঘমেয়াদি সাপোর্টের ভার্সন। যেমন:

Distro	LTS/সাপোর্ট সময়কাল
Ubuntu LTS (যেমন 24.04)	৫ বছর স্ট্যান্ডার্ড সাপোর্ট; **Ubuntu Pro** (ব্যক্তিগত ব্যবহারে ৫টি মেশিন পর্যন্ত ফ্রি) দিয়ে মোট ১০ বছর ESM, Legacy add-on সহ আরও বেশি
Ubuntu interim (যেমন 24.10, 25.04)	মাত্র ৯ মাস
Debian stable	~৩ বছর নিয়মিত + LTS টিম মিলিয়ে মোট ~৫ বছর
RHEL / Rocky / Alma	~১০ বছর

Rolling release

কোনো "ভার্সন" নেই। প্যাকেজের নতুন ভার্সন আসার পর (কিছু টেস্টের পরে) সরাসরি তোমার কাছে আসে। Arch, openSUSE Tumbleweed, Debian sid এই মডেলে চলে।

> 💡 **উপমা:** Fixed release হলো **বছরে একবার ছাপা বই**: প্রতিটা সংস্করণ যত্ন করে সম্পাদিত, ভুল কম, কিন্তু নতুন খবর পেতে পরের সংস্করণের অপেক্ষা। Rolling release হলো **প্রতিদিনের খবরের কাগজ**: সবসময় সর্বশেষ খবর, কিন্তু মাঝে মাঝে ছাপার ভুল থাকে।

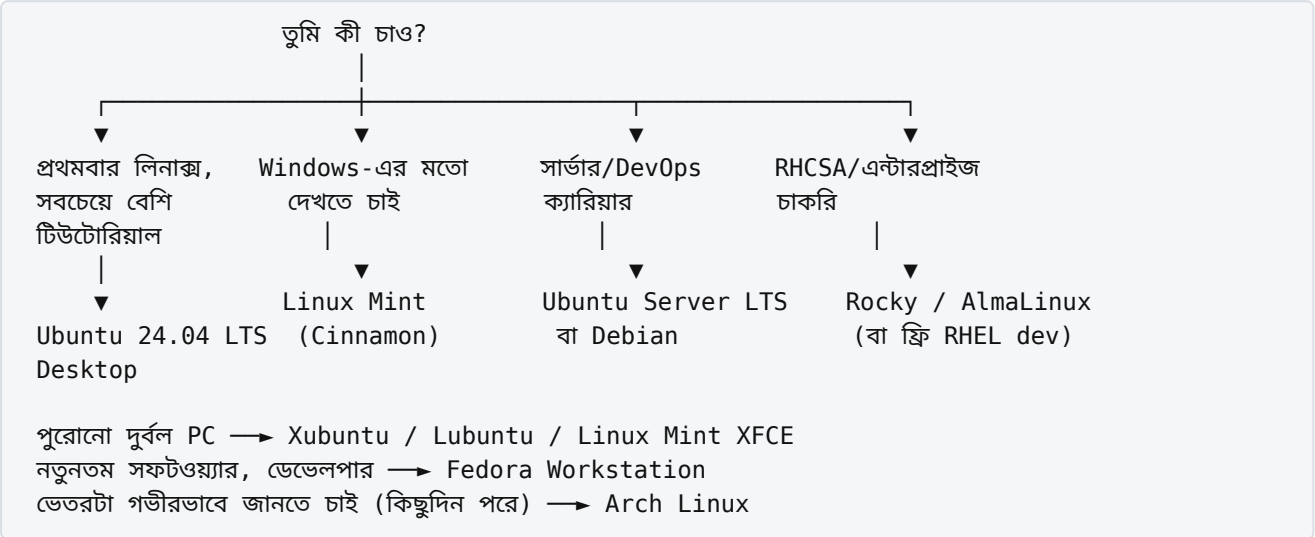


দিক	LTS / Fixed	Rolling
স্থিতিশীলতা	অনেক বেশি	কম (মাঝে মাঝে কিছু ভাঙতে পারে)
সফটওয়্যারের নতুনত্ব	পুরোনো (কিন্তু পরীক্ষিত)	সবসময় সর্বশেষ
নতুন হার্ডওয়্যার সাপোর্ট	কিছুটা দেরিতে (Ubuntu HWE kernel দিয়ে আসে)	দ্রুত
রক্ষণাবেক্ষণ	কম মনোযোগ লাগে	নিয়মিত খবর রাখতে হয় (Arch news পড়া)
সার্ভারের জন্য	✅ আদর্শ	সাধারণত ❌
শেখার জন্য	✅ fresher-এর জন্য ভালো	অভিজ্ঞ হলে

> 📝 **মনে রাখো:** প্রোডাকশন সার্ভারে প্রায় সবসময় LTS/এন্টারপ্রাইজ distro চলে (Ubuntu LTS, Debian, RHEL/Rocky/Alma)। কারণ সার্ভারে "নতুন ফিচার" না, দরকার "কখনো না ভাঙা"।

৩.৪ Fresher-এর জন্য কোনটা?

সিদ্ধান্তের ফ্লোচার্ট



আমাদের পরামর্শ

এই বইয়ের জন্য Ubuntu 24.04 LTS (Desktop) ব্যবহার করো, VirtualBox-এ বা WSL2-তে। কারণ:

1. সবচেয়ে বেশি গাইড, Stack Overflow উত্তর, YouTube ভিডিও
2. LTS, তাই বছরের পর বছর স্থিতিশীল
3. ক্লাউডে (AWS, Azure, GCP) সবচেয়ে বেশি ব্যবহৃত সার্ভার OS-গুলোর একটা, তাই শেখা সরাসরি চাকরিতে কাজে লাগবে
4. পরে চাইলে পাশাপাশি একটা Rocky Linux VM-ও বানাও, যাতে `dnf`-ভিত্তিক দুনিয়াও চেনো

সিস্টেম রিকোয়ারমেন্ট (মোটামুটি)

Distro	RAM	ডিস্ক	মন্তব্য
Ubuntu 24.04 Desktop	৪ GB+ (৮ GB আরামদায়ক)	২৫ GB+	GNOME ভারী
Linux Mint (Cinnamon/XFCE)	২–৪ GB	২০ GB+	XFCE হালকা
Fedora Workstation	৪ GB+	২০ GB+	
Ubuntu Server / Debian (GUI ছাড়া)	১–২ GB	১০ GB+	শুধু টার্মিনাল
Lubuntu / Xubuntu	১–২ GB	১০–২০ GB	পুরোনো PC

> 💡 **টিপস:** VM-এ চালালে তোমার আসল কম্পিউটারের RAM ভাগ হবে। হোস্টে ৮ GB RAM থাকলে VM-কে ৩–৪ GB দাও; ১৬ GB থাকলে ৪–৬ GB।

৩.৫ ইনস্টলেশনের চারটি পথ: তুলনা

পদ্ধতি	ঝুঁকি	পারফরম্যান্স	আসল হার্ডওয়্যার অভিজ্ঞতা	কার জন্য
VM (VirtualBox/VMware)	প্রায় শূন্য	মাঝারি	না	✅ সব fresher, প্রথম ধাপ
WSL2	প্রায় শূন্য	ভালো (CLI)	না, GUI সীমিত	Windows ব্যবহারকারী, ডেভেলপার
Live USB	শূন্য (ইনস্টল না করলে)	USB-নির্ভর	হ্যাঁ	চেষ্টা করে দেখা, রেসকিউ
Dual boot	মাঝারি (ভুলে ডেটা যেতে পারে)	সর্বোচ্চ	হ্যাঁ	যারা নিয়মিত লিনাক্স চালাবে
পুরো ডিস্কে লিনাক্স	Windows মুছে যাবে	সর্বোচ্চ	হ্যাঁ	পূর্ণ লিনাক্স ব্যবহারকারী

সবচেয়ে নিরাপদ শুরু: **VM**। এবার প্রতিটা ধাপে ধাপে দেখি। তার আগে, সব পদ্ধতিতে যেটা লাগবে: **ISO ফাইল**।

৩.৬ ISO ফাইল ডাউনলোড ও যাচাই

ISO হলো পুরো ইনস্টলেশন ডিস্কের একটা হুবহু ইমেজ ফাইল (`.iso`)। সবসময় **অফিশিয়াল সাইট** থেকে নামাবে:

Distro	সাইট
Ubuntu	`https://ubuntu.com/download`
Linux Mint	`https://linuxmint.com/download.php`
Fedora	`https://fedoraproject.org`
Debian	`https://www.debian.org/distrib/`
Rocky Linux	`https://rockylinux.org/download`
AlmaLinux	`https://almalinux.org`
Arch	`https://archlinux.org/download/`

কেন checksum যাচাই করবে?

ডাউনলোডের সময় ফাইল নষ্ট (corrupt) হতে পারে, বা কোনো ক্ষতিকর মিরর পরিবর্তিত ফাইল দিতে পারে। **SHA256 checksum** হলো ফাইলের একটা "আঙুলের ছাপ": ফাইলের একটা বিটও বদলালে ছাপ সম্পূর্ণ বদলে যায়।

```
ubuntu-24.04.x-desktop-amd64.iso  ──►  [SHA-256 অ্যালগরিদম]  ──►  a1b2c3...৬৪ অক্ষর
                                                                          │
                    distro-র সাইটে দেওয়া SHA256SUMS ফাইলের মান ◄── মিলছে? ✅/❌
```

লিনাক্স/macOS-এ:

```
$ sha256sum ubuntu-24.04.3-desktop-amd64.iso
<64-অক্ষরের-হ্যাশ>  ubuntu-24.04.3-desktop-amd64.iso
```

SHA256SUMS ফাইলটাও একই ফোল্ডারে নামিয়ে থাকলে স্বয়ংক্রিয় মিলিয়ে দেখো:

```
$ sha256sum -c SHA256SUMS --ignore-missing
ubuntu-24.04.3-desktop-amd64.iso: OK
```

Windows-এ (PowerShell):

```
PS> Get-FileHash .\ubuntu-24.04.3-desktop-amd64.iso -Algorithm SHA256
```

`sha256sum` অপশন	কাজ
`-c FILE`	FILE-এ লেখা হ্যাশের সঙ্গে মিলিয়ে দেখে
`--ignore-missing`	`-c` এর সময় যে ফাইল ডিস্কে নেই, সেটা এড়িয়ে যায়
`--quiet`	শুধু ব্যর্থ ফলাফল দেখায়

> 💡 **টিপস (অ্যাডভান্সড):** আরও নিরাপদ হতে চাইলে `SHA256SUMS.gpg` সিগনেচার `gpg --verify SHA256SUMS.gpg SHA256SUMS` দিয়ে যাচাই করো (আগে distro-র পাবলিক কী ইমপোর্ট করতে হয়)। এতে নিশ্চিত হও যে checksum ফাইলটাও আসল।

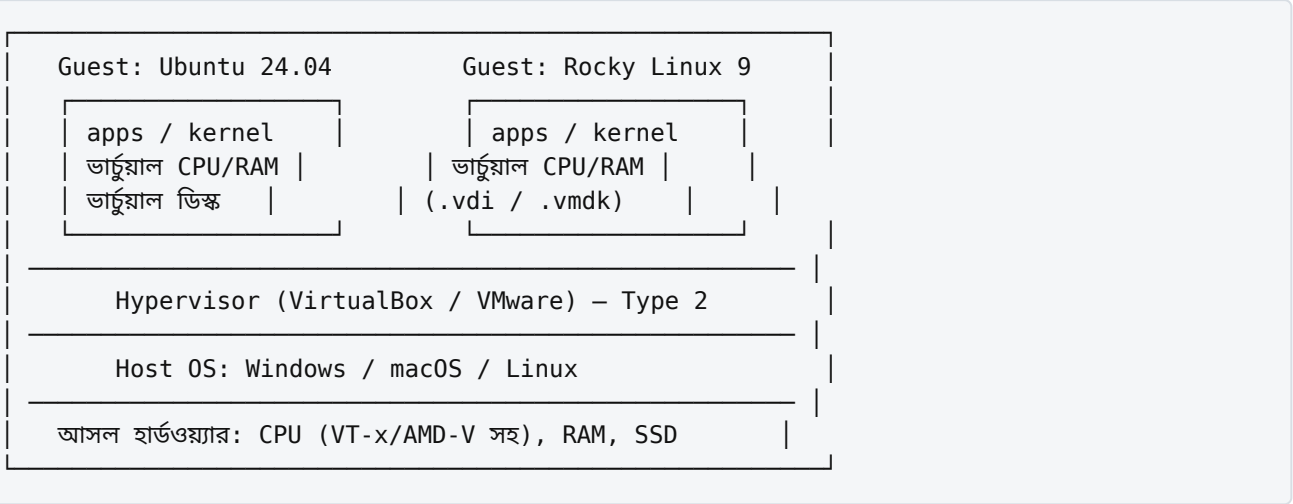
> 📝 ISO ফাইলের নামে ভার্সন নম্বর (যেমন `24.04.3`) সময়ের সঙ্গে বদলায় (point release)। তুমি সাইটে যেটা পাবে সেটাই নাও।

৩.৭ পদ্ধতি ১: VirtualBox-এ ভার্চুয়াল মেশিন (ধাপে ধাপে)

VM ভেতরে কীভাবে কাজ করে?

Virtual Machine (VM) = সফটওয়্যার দিয়ে বানানো একটা নকল কম্পিউটার। তোমার আসল কম্পিউটার (**host**) তার CPU, RAM, ডিস্কের একটা অংশ ধার দেয় একটা "অতিথি" OS-কে (**guest**)। মাঝখানে থাকে **hypervisor** (VirtualBox, VMware), যে guest-কে বিশ্বাস করায় যে সে আসল হার্ডওয়্যারে চলছে।

> 💡 **উপমা:** তোমার বাড়ি (host কম্পিউটার)-এর ভেতরে একটা আলাদা ঘর (VM) ভাড়া দিলে। ভাড়াটে (guest OS) মনে করে এটা তার পুরো বাড়ি। ঘরে আগুন লাগলেও (guest ক্র্যাশ) মূল বাড়ির কিছু হয় না, আর ঘরটা যেকোনো সময় ভেঙে নতুন করে বানানো যায়।



- **Type 2 hypervisor** (VirtualBox, VMware Workstation): একটা সাধারণ OS-এর ওপর অ্যাপ হিসেবে চলে।
- **Type 1 hypervisor** (KVM, VMware ESXi, Hyper-V, Xen): সরাসরি হার্ডওয়্যারের ওপর বা kernel-এর অংশ হিসেবে চলে, সার্ভারে ব্যবহৃত।
- আধুনিক CPU-তে **হার্ডওয়্যার ভার্চুয়ালাইজেশন** (Intel **VT-x**, AMD **AMD-V/SVM**) থাকে, যা VM-কে প্রায় নেটিভ গতিতে চালায়। এটা BIOS/UEFI-তে **চালু** থাকতে হবে।

বিস্তারিত ভার্চুয়ালাইজেশন ও কন্টেইনার: অধ্যায় ৪৮।

ধাপ ০: প্রস্তুতি

1. **ভার্চুয়ালাইজেশন চালু আছে কিনা দেখো।** Windows-এ: Task Manager → Performance → CPU → নিচে **"Virtualization: Enabled"**। Disabled হলে BIOS/UEFI-তে ঢুকে (বুটের সময় `F2` / `Del`) **Intel Virtualization Technology / VT-x** বা **SVM Mode** চালু করো।
2. লিনাক্স host হলে যাচাই:

   ```
   $ grep -Ec '(vmx|svm)' /proc/cpuinfo
   8
   ```

   ০-এর বেশি হলে CPU সাপোর্ট করে।
3. **VirtualBox** নামাও: `https://www.virtualbox.org` (Windows/macOS-Intel/Linux সবার জন্য ফ্রি; Apple Silicon Mac-এ ARM ভার্সনের সাপোর্ট সীমিত, সেখানে UTM বা VMware Fusion সহজ)। সঙ্গে একই ভার্সনের **Extension Pack** চাইলে নিতে পারো (USB 3 পাসথ্রু ইত্যাদির জন্য; শেখার জন্য বাধ্যতামূলক না)।
4. Ubuntu ISO নামিয়ে checksum যাচাই করো (৩.৬)।

> ⚠️ **সতর্কতা:** Windows-এ **Hyper-V**, **WSL2**, বা **Memory Integrity (Core Isolation)** চালু থাকলে VirtualBox ধীর হতে পারে (VM-এর নিচের কোণায় সবুজ কচ্ছপ 🐢 আইকন দেখায়)। নতুন VirtualBox ভার্সনগুলো Hyper-V-এর ওপর দিয়ে চলতে পারে, কিন্তু গতি কম। বেশি সমস্যা হলে VMware Workstation চেষ্টা করো, যেটা Hyper-V-এর সঙ্গে ভালো মানিয়ে চলে।

ধাপ ১: নতুন VM তৈরি

VirtualBox খুলে **New** (বা `Ctrl+N`)-এ ক্লিক করো:

ঘর	কী দেবে
Name	`ubuntu-lab`
Folder	যেখানে VM ফাইল থাকবে (পর্যাপ্ত জায়গা আছে এমন ড্রাইভ)
ISO Image	নামানো `ubuntu-24.04.x-desktop-amd64.iso`
Type / Version	ISO দিলে নিজেই Linux / Ubuntu (64-bit) বসিয়ে নেয়
Skip Unattended Installation	✅ টিক দাও (নিজের হাতে ইনস্টল করে শেখার জন্য)

> 📝 **Unattended Installation** চালু থাকলে VirtualBox নিজেই ইউজারনেম/পাসওয়ার্ড দিয়ে চুপচাপ ইনস্টল করে দেয়। সুবিধাজনক, কিন্তু তুমি ইনস্টলারের ধাপগুলো দেখতে পাবে না। প্রথমবার নিজে করো।

ধাপ ২: হার্ডওয়্যার বরাদ্দ

সেটিং	প্রস্তাবিত মান	কেন
Base Memory	৪০৯৬ MB (৪ GB)	GNOME ডেস্কটপ আরামে চলবে
Processors	২ (হোস্টে যত কোর, তার অর্ধেকের বেশি না)	host যেন ধীর না হয়
Enable EFI	✅ (ঐচ্ছিক)	আধুনিক UEFI বুট শেখার জন্য

ধাপ ৩: ভার্চুয়াল হার্ড ডিস্ক

সেটিং	মান
Create a Virtual Hard Disk Now	নির্বাচন করো
Size	২৫–৪০ GB
Format	VDI (VirtualBox Disk Image)
Pre-allocate Full Size	❌ (dynamically allocated: যতটুকু ব্যবহার, ততটুকু জায়গা নেয়)

> 💡 **dynamically allocated** ডিস্ক শুরুতে ছোট থাকে, ডেটা লিখলে বাড়ে। **fixed size** একটু দ্রুত, কিন্তু শুরুতেই পুরো জায়গা দখল করে।

Finish চাপো। এবার VM সিলেক্ট করে **Settings**-এ কয়েকটা জিনিস ঠিক করো:

- **Display** → **Video Memory**: সর্বোচ্চ (১২৮ MB), Graphics Controller: **VMSVGA**
- **Network** → **Adapter 1**: **NAT** (ডিফল্ট, ইন্টারনেট পাবে)। পরে নেটওয়ার্কিং/SSH শিখতে **Bridged Adapter** বা **Host-only** লাগবে (অধ্যায় ৩৮)।
- **General** → **Advanced** → **Shared Clipboard**: Bidirectional (Guest Additions ইনস্টলের পরে কাজ করবে)

নেটওয়ার্ক মোড	কী হয়
NAT	VM ইন্টারনেট পায়, কিন্তু বাইরের কেউ VM-এ ঢুকতে পারে না (port forwarding ছাড়া)
Bridged	VM তোমার রাউটার থেকে নিজস্ব IP পায়, LAN-এর আরেকটা কম্পিউটারের মতো
Host-only	শুধু host আর VM-গুলোর মধ্যে নেটওয়ার্ক, ইন্টারনেট নেই
NAT Network	একাধিক VM এক NAT-এর পেছনে একে অপরকে দেখতে পায়
Internal Network	শুধু VM-গুলোর মধ্যে, host-ও দেখে না

ধাপ ৪: VM চালু ও Ubuntu ইনস্টলার

Start চাপো। GRUB মেনু আসবে: **"Try or Install Ubuntu"** বেছে Enter। কিছুক্ষণ পর Ubuntu-র ইনস্টলার (Flutter-এ বানানো নতুন ইনস্টলার) খুলবে:

1. **Language**: English (বাংলাও আছে, কিন্তু টার্মিনালের এরর মেসেজ ইংরেজিতে রাখলে গুগল করা সহজ)
2. **Accessibility**: দরকার হলে
3. **Keyboard layout**: English (US)
4. **Network**: VM-এ NAT থাকলে স্বয়ংক্রিয় সংযোগ
5. **Try or Install**: **Install Ubuntu**
6. **Type of installation**: **Interactive installation**
7. **Applications**: **Default selection** (হালকা) বা **Extended selection** (অফিস ইত্যাদি সহ)
8. **Optimise your computer**: ✅ *Install third-party software for graphics and Wi-Fi hardware* এবং ✅ *Download and install support for additional media formats* (আসল হার্ডওয়্যারে বিশেষ জরুরি)
9. **Disk setup**: **Erase disk and install Ubuntu**। চিন্তা নেই, এটা শুধু **ভার্চুয়াল ডিস্ক** মুছবে, তোমার আসল ডিস্ক না। (Advanced features-এ LVM/এনক্রিপশনের অপশন আছে; অধ্যায় ৩৩-এ LVM শিখবে)
10. **Create your account**: নাম, কম্পিউটারের নাম (hostname, যেমন `ubuntu-lab`), ইউজারনেম (ছোট হাতের, স্পেস ছাড়া, যেমন `rahim`), শক্তিশালী পাসওয়ার্ড
11. **Timezone**: Asia/Dhaka
12. **Review** → **Install**

ইনস্টল শেষে **Restart now**। "Please remove the installation medium, then press ENTER" এলে Enter চাপো (VirtualBox সাধারণত ISO নিজেই খুলে নেয়)।

```
ইনস্টলার ভেতরে কী করে (সংক্ষেপে):

 [ডিস্ক পার্টিশন] → [ফাইলসিস্টেম তৈরি (ext4)] → [বেস সিস্টেম কপি]
        → [ইউজার তৈরি, /etc কনফিগ] → [bootloader (GRUB) ইনস্টল]
        → [initramfs তৈরি] → রিবুট
```

ধাপ ৫: Guest Additions (VirtualBox)

Guest Additions = guest-এর ভেতরে চলা কিছু ড্রাইভার/সার্ভিস, যাতে পাওয়া যায়: স্বয়ংক্রিয় স্ক্রিন রিসাইজ, শেয়ার্ড ক্লিপবোর্ড, ড্র্যাগ-ড্রপ, শেয়ার্ড ফোল্ডার, ভালো মাউস।

Ubuntu-তে লগইন করে টার্মিনাল খোলো (`Ctrl+Alt+T`), তারপর:

```
$ sudo apt update
$ sudo apt install -y build-essential dkms linux-headers-$(uname -r) bzip2
```

এখন VirtualBox মেনু থেকে **Devices → Insert Guest Additions CD image...**। তারপর:

```
$ cd /media/$USER/VBox_GAs_*
$ sudo ./VBoxLinuxAdditions.run
Verifying archive integrity...  All good.
...
VirtualBox Guest Additions: Building the modules for kernel 6.8.0-xx-generic.
...
$ sudo reboot
```

> 💡 **সহজ বিকল্প:** অনেক সময় distro-র রিপোজিটরিতেই guest প্যাকেজ থাকে (Ubuntu-তে `virtualbox-guest-x11` ইত্যাদি)। তবে VirtualBox-এর সঙ্গে আসা ISO-টাই ভার্সন মেলানোর জন্য সবচেয়ে নির্ভরযোগ্য।

শেয়ার্ড ফোল্ডার ব্যবহার করতে নিজেকে `vboxsf` গ্রুপে যোগ করতে হয় (গ্রুপ নিয়ে অধ্যায় ১৫):

```
$ sudo usermod -aG vboxsf $USER
# তারপর লগআউট-লগইন
```

ধাপ ৬: Snapshot: তোমার "Undo" বোতাম

VM-এর সবচেয়ে বড় সুবিধা **snapshot**: VM-এর কোনো মুহূর্তের পুরো অবস্থা সংরক্ষণ। পরে কিছু ভেঙে ফেললে এক ক্লিকে ওই অবস্থায় ফিরে যাবে।

VirtualBox Manager → VM সিলেক্ট → Snapshots → **Take** (চলন্ত VM-এ `Host+T`)। নাম দাও: `fresh-install-updated`।

> 💡 **টিপস:** এই বইয়ের প্রতিটা "বিপজ্জনক" অধ্যায়ের (পার্টিশন, LVM, ফায়ারওয়াল, বুট প্রসেস) আগে একটা snapshot নিও। নির্ভয়ে পরীক্ষা করো: ভাঙলে restore!

VMware Workstation দিয়ে করলে

২০২৪ থেকে **VMware Workstation Pro** (Windows/Linux) ও **Fusion Pro** (macOS) ব্যক্তিগত ও বাণিজ্যিক উভয় ব্যবহারে ফ্রি করা হয়েছে; Broadcom-এর সাপোর্ট পোর্টালে অ্যাকাউন্ট খুলে নামাতে হয়। ধাপ প্রায় একই:

1. **Create a New Virtual Machine** → **Typical** → ISO দেখাও
2. Linux / Ubuntu 64-bit, ডিস্ক ২৫–৪০ GB (*Store virtual disk as a single file* বা *split*, যেকোনো একটা)
3. **Customize Hardware**: Memory ৪ GB, Processors ২
4. ইনস্টল একই রকম। Guest tools হিসেবে **open-vm-tools** (Ubuntu ইনস্টলার সাধারণত নিজেই বসায়; না থাকলে):

```
$ sudo apt install -y open-vm-tools open-vm-tools-desktop
```

বিষয়	VirtualBox	VMware Workstation Pro
দাম	ফ্রি, ওপেন সোর্স (মূল অংশ)	ফ্রি (ক্লোজড সোর্স)
ডিস্ক ফরম্যাট	`.vdi` (`.vmdk` ও পারে)	`.vmdk`
Guest টুল	Guest Additions	open-vm-tools
3D/গ্রাফিক্স	মোটামুটি	সাধারণত ভালো
Hyper-V-এর সঙ্গে	ধীর হতে পারে	ভালো চলে

> 📝 **লিনাক্স host হলে:** সবচেয়ে ভালো পারফরম্যান্স **KVM/QEMU** (kernel-এর ভেতরের hypervisor) + **virt-manager** GUI দিয়ে। অধ্যায় ৪৮-এ দেখবে।

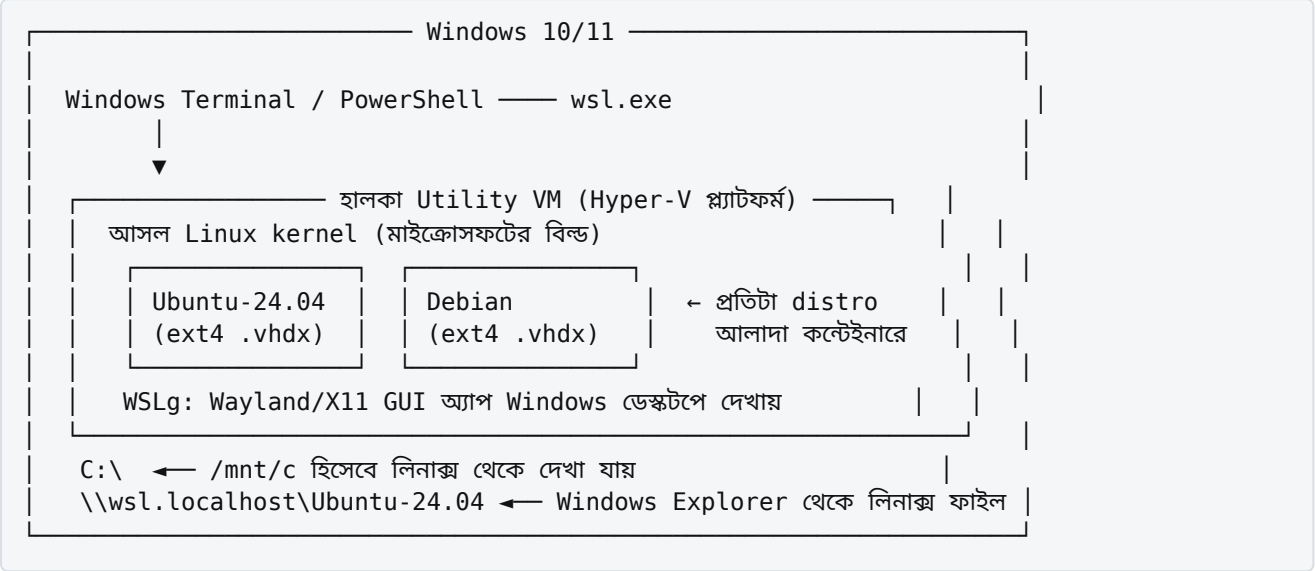
৩.৮ পদ্ধতি ২: WSL2: Windows-এর ভেতরে লিনাক্স

WSL2 কী এবং ভেতরে কীভাবে কাজ করে?

WSL (Windows Subsystem for Linux) মাইক্রোসফটের একটা ফিচার, যা দিয়ে Windows-এর ভেতরেই আসল লিনাক্স চালানো যায়, আলাদা VM সফটওয়্যার ছাড়া।

- **WSL1** লিনাক্স system call-গুলোকে Windows kernel-এর call-এ "অনুবাদ" করত। কোনো আসল Linux kernel ছিল না।

- **WSL2** একটা হালকা, বিশেষভাবে বানানো **utility VM**-এর ভেতরে মাইক্রোসফটের রক্ষণাবেক্ষণ করা **আসল Linux kernel** চালায় (Hyper-V প্রযুক্তি দিয়ে)। তাই সামঞ্জস্য প্রায় ১০০%, Docker-ও চলে। চালু হতে মাত্র ১–২ সেকেন্ড লাগে।



ইনস্টল (Windows 11, বা Windows 10 ভার্সন 2004 / build 19041 বা তার পরে)

Administrator হিসেবে PowerShell বা Terminal খোলো (Start → "PowerShell" → Run as administrator):

```
PS> wsl --install
```

এই একটা কমান্ডই প্রয়োজনীয় Windows ফিচার (Virtual Machine Platform, WSL) চালু করে, kernel নামায় আর ডিফল্ট distro (Ubuntu) বসায়। শেষে **রিস্টার্ট** করো। রিস্টার্টের পর Ubuntu উইন্ডো খুলে ইউজারনেম আর পাসওয়ার্ড চাইবে (এই পাসওয়ার্ড Windows-এর পাসওয়ার্ড না, লিনাক্সের `sudo`-র জন্য)।

নির্দিষ্ট distro চাইলে:

```
PS> wsl --list --online
The following is a list of valid distributions that can be installed.
NAME                   FRIENDLY NAME
Ubuntu                 Ubuntu
Ubuntu-24.04           Ubuntu 24.04 LTS
Debian                 Debian GNU/Linux
...
PS> wsl --install -d Ubuntu-24.04
```

গুরুত্বপূর্ণ `wsl` কমান্ড

কমান্ড	কাজ
`wsl --install`	WSL + ডিফল্ট distro ইনস্টল

কমান্ড	কাজ
`wsl --install -d <Distro>`	নির্দিষ্ট distro ইনস্টল
`wsl -l -o` / `wsl --list --online`	ইনস্টলযোগ্য distro-র তালিকা
`wsl -l -v` / `wsl --list --verbose`	ইনস্টল করা distro, অবস্থা (Running/Stopped) ও WSL ভার্সন
`wsl`	ডিফল্ট distro-তে ঢোকা
`wsl -d <Distro>`	নির্দিষ্ট distro-তে ঢোকা
`wsl --set-default <Distro>`	ডিফল্ট distro বদলানো
`wsl --set-version <Distro> 2`	কোনো distro-কে WSL2-তে রূপান্তর
`wsl --set-default-version 2`	নতুন distro সবসময় WSL2
`wsl --shutdown`	সব distro ও utility VM বন্ধ
`wsl --terminate <Distro>`	একটা distro বন্ধ
`wsl --update`	WSL ও kernel আপডেট
`wsl --status` / `wsl --version`	অবস্থা ও ভার্সন দেখা
`wsl --export <Distro> file.tar`	distro-র ব্যাকআপ
`wsl --import <Name> <Folder> file.tar`	ব্যাকআপ থেকে ফেরত/কপি
`wsl --unregister <Distro>`	⚠️ distro ও এর **সব ডেটা মুছে ফেলা**

```
PS> wsl -l -v
  NAME            STATE           VERSION
* Ubuntu-24.04    Running         2
```

WSL2 ব্যবহারের টিপস

- **ফাইল কোথায় রাখবে:** লিনাক্সের কাজ লিনাক্স ফাইলসিস্টেমে রাখো (`~/projects`), `/mnt/c/...`-তে না। Windows ড্রাইভ জুড়ে ফাইল অ্যাক্সেস অনেক ধীর।
- লিনাক্স থেকে বর্তমান ফোল্ডার Windows Explorer-এ খুলতে: `explorer.exe .`
- **systemd:** নতুন Ubuntu WSL ইনস্টলে systemd ডিফল্টে চালু থাকে। না থাকলে `/etc/wsl.conf`-এ লেখো:

  ```
  [boot]
  systemd=true
  ```

  তারপর PowerShell-এ `wsl --shutdown` দিয়ে আবার চালু করো। (systemd নিয়ে অধ্যায় ২৬)
- **GUI অ্যাপ:** Windows 11 (ও আপডেটেড Windows 10)-এ **WSLg** দিয়ে লিনাক্স GUI অ্যাপ (যেমন `gedit`) সরাসরি চলে।
- **RAM/CPU সীমা:** Windows-এ `%UserProfile%\.wslconfig` ফাইলে:

  ```
  [wsl2]
  memory=4GB
  ```

```
processors=2
```

> ⚠️ **সতর্কতা:** WSL2-তে বুট প্রসেস (GRUB, BIOS), ডিস্ক পার্টিশন, আসল নেটওয়ার্ক ইন্টারফেস ইত্যাদি পুরোপুরি শেখা যায় না, কারণ সেগুলো মাইক্রোসফট সামলায়। অধ্যায় ২৭, ৩২, ৩৩ চর্চার জন্য একটা পূর্ণ VM রাখো।

৩.৯ পদ্ধতি ৩: Live USB তৈরি

Live USB কী?

Live USB = এমন একটা পেনড্রাইভ, যেটা থেকে কম্পিউটার বুট করলে লিনাক্স **হার্ড ডিস্কে কিছু না লিখেই** সরাসরি RAM থেকে চলে। কাজ:

1. ইনস্টলের আগে "চেখে দেখা": Wi-Fi, সাউন্ড, টাচপ্যাড চলে কিনা
2. আসল কম্পিউটারে ইনস্টল করা (dual boot বা পুরো ডিস্ক)
3. **রেসকিউ:** নষ্ট সিস্টেম থেকে ফাইল উদ্ধার, পাসওয়ার্ড রিসেট, GRUB মেরামত

```
Live USB বুট হলে ভেতরে কী হয়:

 USB ──► UEFI/BIOS ──► bootloader (GRUB) ──► kernel + initramfs RAM-এ লোড
                                                        │
                    কম্প্রেসড ফাইলসিস্টেম (squashfs) ◄─┘ শুধু-পড়া হিসেবে mount
                                                        │
                    ওপরে RAM-এ লেখার স্তর (overlay) ──► তুমি যা বদলাও তা RAM-এ,
                                                          রিবুট দিলে মুছে যায়
```

লাগবে: কমপক্ষে ৮ GB পেনড্রাইভ (এর **সব ডেটা মুছে যাবে**), যাচাই করা ISO।

টুল তুলনা

টুল	প্ল্যাটফর্ম	বৈশিষ্ট্য
Rufus	Windows	দ্রুত, ছোট; GPT/MBR নিজে বেছে দিতে দেয়
balenaEtcher	Windows, macOS, Linux	সবচেয়ে সহজ, ৩ ক্লিক, লেখার পর যাচাই করে
Ventoy	Windows, Linux	একবার USB-তে বসালে শুধু ISO ফাইল কপি করলেই হয়; একটা USB-তে অনেক ISO
Fedora Media Writer	সব	Fedora-র অফিশিয়াল
GNOME Disks / Startup Disk Creator	Linux	GUI, Ubuntu-তে বিল্ট-ইন
`dd`	Linux/macOS	কমান্ড লাইন, শক্তিশালী কিন্তু বিপজ্জনক

Rufus দিয়ে (Windows)

1. `https://rufus.ie` থেকে নামিয়ে চালাও (portable ভার্সনে ইনস্টল লাগে না)
2. **Device**: তোমার USB (নাম ও সাইজ দেখে নিশ্চিত হও!)
3. **Boot selection**: **SELECT** → ISO
4. **Partition scheme**: আধুনিক কম্পিউটার (২০১২-এর পরের) হলে **GPT**, target system **UEFI (non CSM)**; খুব পুরোনো BIOS কম্পিউটার হলে **MBR**
5. **File system**: ডিফল্ট (FAT32) রাখো
6. **START**। কোনো কোনো ISO-র ক্ষেত্রে মোড জিজ্ঞেস করবে: **ISO Image mode** (প্রস্তাবিত) বা **DD Image mode**। ISO mode-এ সমস্যা হলে DD mode চেষ্টা করো।
7. "সব ডেটা মুছে যাবে" সতর্কতায় OK

balenaEtcher দিয়ে

1. `https://etcher.balena.io` থেকে নামাও
2. **Flash from file** → ISO
3. **Select target** → USB
4. **Flash!** → লেখা শেষে নিজেই **Validating** করে

`dd` দিয়ে (লিনাক্স)

`dd` একটা লো-লেভেল কপি টুল: বাইট-বাই-বাইট হুবহু কপি করে। বিস্তারিত অধ্যায় ৩৫-এ।

ধাপ ১: USB-র ডিভাইস নাম খুঁজে বের করো (USB লাগানোর আগে ও পরে চালিয়ে তুলনা করো):

```
$ lsblk -o NAME,SIZE,TYPE,TRAN,MODEL,MOUNTPOINTS
NAME          SIZE TYPE TRAN   MODEL              MOUNTPOINTS
nvme0n1     476.9G disk nvme   Samsung SSD 980
├─nvme0n1p1   512M part                           /boot/efi
└─nvme0n1p2 476.4G part                           /
sdb          14.6G disk usb    SanDisk Ultra
└─sdb1       14.6G part                           /media/rahim/USB
```

এখানে `TRAN` = `usb` আর সাইজ ১৪.৬G দেখে বোঝা যাচ্ছে USB হলো **`/dev/sdb`**।

ধাপ ২: USB-র পার্টিশন unmount করো:

```
$ sudo umount /dev/sdb1
```

ধাপ ৩: ISO লেখো (পার্টিশন `sdb1` না, পুরো ডিস্ক **`sdb`**):

```
$ sudo dd if=ubuntu-24.04.3-desktop-amd64.iso of=/dev/sdb bs=4M status=progress
oflag=sync
6234832896 bytes (6.2 GB, 5.8 GiB) copied, 412 s, 15.1 MB/s
1486+1 records in
1486+1 records out
```

```
6237151232 bytes (6.2 GB, 5.8 GiB) copied, 413.5 s, 15.1 MB/s
$ sync
```

`dd` অপশন	মানে
`if=FILE`	input file: কোথা থেকে পড়বে
`of=FILE`	output file: কোথায় লিখবে (এখানে ডিভাইস)
`bs=4M`	block size: একবারে ৪ MB করে পড়া-লেখা (দ্রুততর)
`status=progress`	চলাকালীন অগ্রগতি দেখায়
`oflag=sync`	প্রতিটা ব্লক সত্যিই ডিস্কে লেখা নিশ্চিত করে
`conv=fsync`	শেষে একবার সব ডেটা ডিস্কে flush করে (বিকল্প)

> ⚠️ **ভয়ংকর সতর্কতা:** `dd`-কে লোকে মজা করে **"disk destroyer"** বলে। `of=` এ ভুল ডিভাইস (যেমন `/dev/nvme0n1` বা `/dev/sda`) দিলে কোনো প্রশ্ন না করেই **তোমার মূল ডিস্ক মুছে যাবে**, কোনো undo নেই। কমান্ড চালানোর আগে `lsblk` দিয়ে **দুইবার** দেখো।

> 📝 **কেন পার্টিশন (`sdb1`) না, পুরো ডিস্ক (`sdb`)?** আধুনিক লিনাক্স ISO-গুলো **hybrid ISO**: এর ভেতরে নিজস্ব পার্টিশন টেবিল ও bootloader আছে। তাই পুরো ডিস্কের শুরু থেকে হুবহু বসাতে হয়।

USB থেকে বুট করা

1. USB লাগিয়ে কম্পিউটার রিস্টার্ট দাও
2. বুটের শুরুতে **Boot Menu key** বারবার চাপো (নিচের টেবিল)
3. তালিকা থেকে USB বেছে নাও (UEFI হলে নামের আগে "UEFI:" লেখা থাকে, সেটা বাছাই করো)

সাধারণ BIOS/UEFI ও Boot Menu কী (মডেল ভেদে বদলায়; স্ক্রিনে লেখা দেখে নিশ্চিত হও):

ব্র্যান্ড	Boot Menu	BIOS/UEFI সেটআপ
Dell	`F12`	`F2`
HP	`F9` (বা `Esc` → মেনু)	`F10` (বা `Esc`)
Lenovo (ThinkPad/IdeaPad)	`F12` (কিছুতে Novo বাটন)	`F1` / `F2`
ASUS	`F8` বা `Esc`	`F2` / `Del`
Acer	`F12` (BIOS-এ চালু করতে হতে পারে)	`F2`
MSI	`F11`	`Del`
Gigabyte	`F12`	`Del`
Apple Intel Mac	`Option (⌥)` চেপে ধরে রাখো	—

> 💡 **টিপস:** Windows থেকেও UEFI-তে যাওয়া যায়: Settings → System → Recovery → **Advanced startup** → **Restart now** → Troubleshoot → Advanced options → **UEFI Firmware Settings**। অথবা `Shift` চেপে ধরে Restart।

৩.১০ পার্টিশনিং-এর বেসিক (ইনস্টলের আগে জানা জরুরি)

পার্টিশন, ফাইলসিস্টেম, mount পুরো বিস্তারিত পাবে অধ্যায় ৩২-এ। এখানে শুধু ইনস্টলের জন্য যতটুকু দরকার।

> 💡 **উপমা:** একটা হার্ড ডিস্ক একটা বড় জমির মতো। **পার্টিশন** হলো জমিকে দেয়াল দিয়ে কয়েকটা প্লটে ভাগ করা। **ফাইলসিস্টেম** (ext4, NTFS) হলো প্রতিটা প্লটে কীভাবে ঘর-তাক বানিয়ে জিনিস সাজানো হবে তার নকশা। **Partition table** (MBR/GPT) হলো জমির দলিল, যেখানে লেখা কোন প্লট কোথা থেকে কোথা পর্যন্ত।

BIOS বনাম UEFI

	BIOS (Legacy)	UEFI
যুগ	১৯৮০-এর দশক থেকে	~২০১০ থেকে আধুনিক সব PC
বুট কীভাবে	ডিস্কের প্রথম সেক্টর (MBR)-এর ছোট কোড চালায়	**ESP** পার্টিশনের `.efi` ফাইল চালায়
পার্টিশন টেবিল	সাধারণত MBR	GPT
Secure Boot	নেই	আছে

MBR বনাম GPT

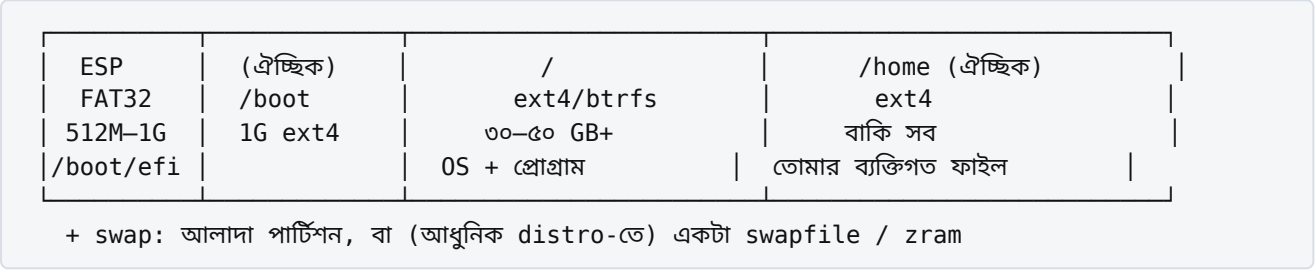
	MBR	GPT
সর্বোচ্চ ডিস্ক সাইজ	২ TiB (৫১২-বাইট সেক্টরে)	কার্যত অসীম (৯.৪ ZB)
পার্টিশন সংখ্যা	৪টি primary (বা ৩ primary + ১ extended, তার ভেতরে logical)	সাধারণত ১২৮টি
ব্যাকআপ টেবিল	নেই	ডিস্কের শেষে ব্যাকআপ কপি থাকে
ব্যবহার	পুরোনো BIOS সিস্টেম	✅ আধুনিক UEFI সিস্টেম

লিনাক্সে ডিস্ক ও পার্টিশনের নাম

নাম	মানে
`/dev/sda`, `/dev/sdb`	প্রথম, দ্বিতীয় SATA/SCSI/USB ডিস্ক
`/dev/sda1`, `/dev/sda2`	`sda`-র প্রথম, দ্বিতীয় পার্টিশন

নাম	মানে
`/dev/nvme0n1`	প্রথম NVMe SSD (controller 0, namespace 1)
`/dev/nvme0n1p1`	NVMe-র প্রথম পার্টিশন (`p` যোগ হয়)
`/dev/vda`	KVM VM-এর virtio ডিস্ক
`/dev/mmcblk0p1`	SD কার্ড/eMMC-র পার্টিশন

লিনাক্স ইনস্টলের জন্য সাধারণ পার্টিশন বিন্যাস (UEFI + GPT)



মাউন্ট পয়েন্ট	ফাইলসিস্টেম	সাইজ (প্রস্তাবিত)	কাজ
`/boot/efi` (**ESP**)	FAT32	৫১২ MB – ১ GB	UEFI bootloader ফাইল। dual boot-এ Windows-এর ESP-টাই শেয়ার করা যায়
`/boot`	ext4	~১ GB (ঐচ্ছিক)	kernel ও initramfs; LVM/এনক্রিপশন থাকলে আলাদা লাগে
`/` (root)	ext4 / Btrfs / XFS	৩০–৫০ GB+	পুরো OS
`/home`	ext4	বাকি জায়গা (ঐচ্ছিক)	ইউজারের ফাইল; আলাদা রাখলে OS পুনরায় ইনস্টলেও ফাইল থাকে
swap	swap	নিচে দেখো	RAM ভরে গেলে অতিরিক্ত জায়গা; **hibernate**-এর জন্য RAM-এর সমান বা বেশি

Swap কতটুকু? আধুনিক নিয়ম মোটামুটি:

RAM	swap (hibernate ছাড়া)	hibernate চাইলে
≤ ২ GB	RAM-এর ২ গুণ	RAM-এর ৩ গুণ
২–৮ GB	RAM-এর সমান	RAM-এর ২ গুণ
৮–৬৪ GB	কমপক্ষে ৪ GB	RAM-এর ১.৫ গুণ

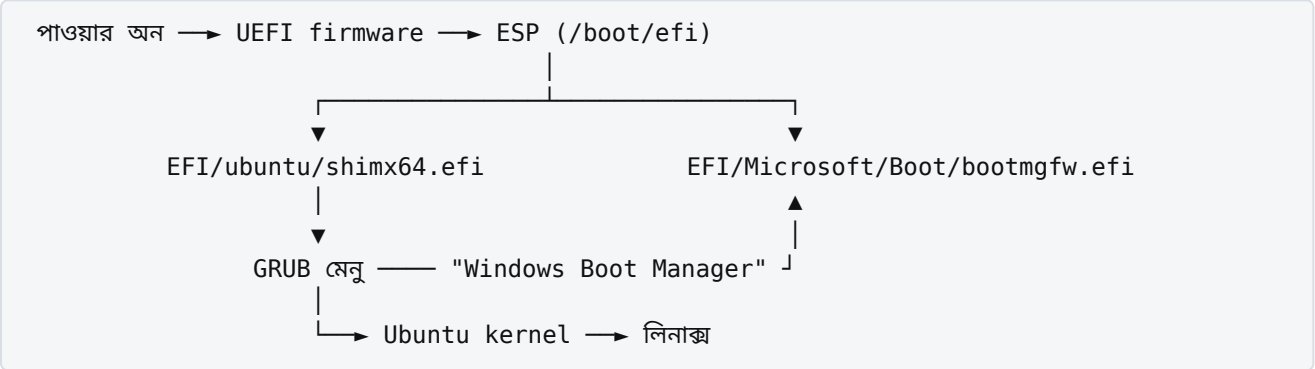
> 📝 **distro-ভেদে ডিফল্ট:**
>
> - **Ubuntu**: ESP + একটা বড় `/` (ext4) পার্টিশন; swap হিসেবে `/swap.img` **swapfile**।
> - **Fedora Workstation**: **Btrfs** ফাইলসিস্টেম, `root` ও `home` আলাদা **subvolume** (একই পার্টিশনে); swap হিসেবে **zram** (RAM-এর ভেতরে কম্প্রেসড swap)।
> - **RHEL/Rocky/Alma**: **XFS** ফাইলসিস্টেম + **LVM** ডিফল্ট।

- **openSUSE**: Btrfs + Snapper স্ন্যাপশট।

> 💡 **fresher-এর পরামর্শ:** VM-এ বা পুরো ডিস্কে ইনস্টল করলে ইনস্টলারের **স্বয়ংক্রিয় পার্টিশনিং** (Erase disk) নাও। ম্যানুয়াল পার্টিশনিং অভ্যাসের জন্য আলাদা একটা VM-এ চেষ্টা করো।

৩.১১ পদ্ধতি ৪: Dual Boot (Windows + Linux)

Dual boot মানে একই কম্পিউটারের ডিস্কে দুটো OS। চালু করার সময় **GRUB** মেনু জিজ্ঞেস করবে কোনটায় যাবে।



বুট প্রসেসের গভীরে যাবো অধ্যায় ২৭-এ।

ধাপ ০: নিরাপত্তার প্রস্তুতি (বাদ দিও না!)

1. **পুরো ব্যাকআপ নাও।** পার্টিশনে ভুল হলে ডেটা যেতে পারে।
2. **BitLocker রিকভারি কী** সংগ্রহ করো (Windows 11-এ অনেক ল্যাপটপে ডিভাইস এনক্রিপশন ডিফল্টে চালু)। কী পাবে `https://account.microsoft.com/devices/recoverykey`-এ, বা Command Prompt (Admin)-এ `manage-bde -status` দিয়ে অবস্থা দেখো। প্রয়োজনে ইনস্টলের সময় BitLocker সাময়িক **Suspend** করো। নাহলে বুট বদলানোর পর Windows রিকভারি কী চাইবে।
3. **Fast Startup বন্ধ করো:** Control Panel → Power Options → *Choose what the power buttons do* → *Change settings that are currently unavailable* → ❌ *Turn on fast startup*। কারণ Fast Startup আসলে Windows-কে আধা-hibernate অবস্থায় রাখে; NTFS পার্টিশন "লক" থাকে, লিনাক্স থেকে লিখলে ডেটা নষ্ট হতে পারে।
4. **ডিস্ক মোড:** কিছু Intel ল্যাপটপে স্টোরেজ মোড **RST/RAID/VMD**-এ থাকে, তখন লিনাক্স ইনস্টলার ডিস্ক দেখতে পায় না। সমাধান সাধারণত UEFI-তে **AHCI** মোডে নেওয়া, কিন্তু তার আগে Windows-কে Safe Mode দিয়ে প্রস্তুত না করলে Windows বুট হবে না। এটা ঝামেলার; প্রয়োজন হলে ভালো করে গাইড পড়ে করো।
5. **Secure Boot**: Ubuntu, Fedora, Debian, RHEL-পরিবার, openSUSE সাইন করা **shim** bootloader দিয়ে Secure Boot চালু রেখেই বুট করে। বন্ধ করার দরকার নেই। (NVIDIA-র মতো বাইরের kernel

মডিউল বসালে **MOK** কী enroll করতে বলতে পারে, স্ক্রিনের নির্দেশ মানো।)

ধাপ ১: Windows থেকে জায়গা খালি করা

1. `Win+X` → **Disk Management**
2. C: ড্রাইভে রাইট-ক্লিক → **Shrink Volume**
3. কত MB কমাবে লেখো (যেমন ৮০ GB = `81920` MB) → Shrink
4. এখন **Unallocated** কালো অংশ দেখা যাবে। এখানে কোনো পার্টিশন **বানাবে না**, লিনাক্স ইনস্টলার নিজেই ব্যবহার করবে।

ধাপ ২: Live USB থেকে বুট ও ইনস্টল

ইনস্টলারে Disk setup ধাপে:

- **"Install Ubuntu alongside Windows Boot Manager"** অপশন দেখালে সেটাই সবচেয়ে সহজ: খালি জায়গায় নিজেই পার্টিশন বানায়, আর বিদ্যমান ESP-তে GRUB বসায়।
- না দেখালে **Manual installation**:
  - Unallocated জায়গা সিলেক্ট → `+` → সাইজ সব, ফরম্যাট **ext4**, mount `/`
  - বিদ্যমান ESP (FAT32, ~১০০–৫০০ MB) কে **ফরম্যাট না করে** `/boot/efi` হিসেবে নির্বাচন (অনেক সময় ইনস্টলার নিজেই করে)
  - **Device for boot loader installation**: পুরো ডিস্ক (যেমন `/dev/nvme0n1`)

> ⚠️ **সতর্কতা: "Erase disk and install Ubuntu"** বেছে নিলে **Windows সহ পুরো ডিস্ক মুছে যাবে**। dual boot-এ এটা কখনোই না। আর Windows-এর ESP **কখনো ফরম্যাট করবে না**, করলে Windows বুট হবে না।

ধাপ ৩: রিবুট ও GRUB

রিবুটে GRUB মেনুতে Ubuntu আর Windows Boot Manager দুটোই থাকা উচিত। Windows না দেখালে, Ubuntu-তে ঢুকে:

```
$ sudo nano /etc/default/grub
# এই লাইনটা যোগ করো বা কমেন্ট (#) সরাও:
GRUB_DISABLE_OS_PROBER=false
$ sudo apt install os-prober
$ sudo update-grub
...
Found Windows Boot Manager on /dev/nvme0n1p1@/EFI/Microsoft/Boot/bootmgfw.efi
done
```

> 📝 Fedora/RHEL-পরিবারে `update-grub` নেই; সেখানে `sudo grub2-mkconfig -o`

> `/boot/grub2/grub.cfg`।

Dual boot-এর সাধারণ সমস্যা

সমস্যা	কারণ	সমাধান
লিনাক্সে গেলে Windows-এর ঘড়ি ভুল	Windows হার্ডওয়্যার ঘড়িকে local time ধরে, লিনাক্স UTC ধরে	লিনাক্সে `timedatectl set-local-rtc 1` (বা Windows-কে UTC-তে নাও)
GRUB না এসে সরাসরি Windows চালু হয়	UEFI boot order-এ Windows প্রথমে	UEFI সেটআপে "ubuntu" প্রথমে রাখো
Windows আপডেটের পর GRUB উধাও	Windows boot order বদলেছে	UEFI-তে আবার ঠিক করো, বা Live USB থেকে GRUB মেরামত (Boot-Repair টুল)
ইনস্টলার ডিস্ক দেখে না	RST/VMD মোড	ধাপ ০-এর ৪ নম্বর
BitLocker রিকভারি কী চাইছে	বুট চেইন বদলেছে	সংরক্ষিত কী দাও

৩.১২ প্রথম বুটের পরে: প্রথম দিনের চেকলিস্ট

ইনস্টল শেষ, লগইন করেছো। এবার টার্মিনাল খোলো (`Ctrl+Alt+T` Ubuntu-তে; টার্মিনাল নিয়ে বিস্তারিত অধ্যায় ৫)।

১. কোন সিস্টেমে আছো, জানো

```
$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.3 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
...
```

`ID_LIKE=debian` দেখে বোঝা যায় এটা Debian পরিবারের, তাই `apt` চলবে। Rocky-তে দেখবে `ID_LIKE="rhel centos fedora"`।

```
$ hostnamectl
 Static hostname: ubuntu-lab
       Icon name: computer-vm
         Chassis: vm 🖴
Operating System: Ubuntu 24.04.3 LTS
          Kernel: Linux 6.8.0-xx-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
```

```
  Hardware Model: VirtualBox
...
$ uname -r
6.8.0-xx-generic
```

কমান্ড	কী দেখায়
`cat /etc/os-release`	distro-র নাম, ভার্সন, পরিবার (সব distro-তে থাকে)
`lsb_release -a`	distro তথ্য (প্যাকেজ `lsb-release` থাকলে)
`hostnamectl`	hostname, OS, kernel, হার্ডওয়্যার/VM কিনা
`uname -r`	kernel ভার্সন
`uname -a`	kernel-এর সব তথ্য

২. সিস্টেম আপডেট: সবচেয়ে জরুরি কাজ

ISO বানানোর পর থেকে অনেক নিরাপত্তা আপডেট এসেছে। তাই **প্রথম কাজ আপডেট।**

কেন দুই ধাপে (Debian/Ubuntu)?

```
sudo apt update   ⟶ ইন্টারনেটের রিপোজিটরি থেকে "কোন প্যাকেজের কোন ভার্সন আছে"
                     তার তালিকা নামায় (দোকানের নতুন ক্যাটালগ আনা)
                     কিছু ইনস্টল করে না!
sudo apt upgrade  ⟶ তালিকা মিলিয়ে ইনস্টল করা প্যাকেজগুলো নতুন ভার্সনে আপগ্রেড
                     (ক্যাটালগ দেখে আসলে জিনিস কেনা)
```

Ubuntu / Debian / Mint:

```
$ sudo apt update
[sudo] password for rahim:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
...
87 packages can be upgraded. Run 'apt list --upgradable' to see them.

$ sudo apt upgrade -y
...
$ sudo apt full-upgrade -y      # দরকার হলে প্যাকেজ সরিয়ে/নতুন যোগ করে আপগ্রেড (kernel ইত্যাদি)
$ sudo apt autoremove -y        # অপ্রয়োজনীয় পুরোনো প্যাকেজ সরানো
```

কমান্ড	কাজ
`sudo apt update`	প্যাকেজ তালিকা রিফ্রেশ
`sudo apt upgrade`	প্যাকেজ আপগ্রেড (কিছু মুছবে না)
`sudo apt full-upgrade`	প্রয়োজনে প্যাকেজ মুছে/যোগ করে পূর্ণ আপগ্রেড
`sudo apt autoremove`	অনাথ নির্ভরতা প্যাকেজ মুছে ফেলা
`apt list --upgradable`	কোনগুলো আপগ্রেড হবে দেখা

কমান্ড	কাজ
`-y`	সব প্রশ্নে স্বয়ংক্রিয় "yes"

অন্য distro:

Distro	আপডেট কমান্ড
Fedora	`sudo dnf upgrade --refresh`
RHEL	প্রথমে `sudo subscription-manager register`, তারপর `sudo dnf update`
Rocky / Alma / CentOS Stream	`sudo dnf update` (`dnf upgrade`-এর সমার্থক)
Arch / Manjaro	`sudo pacman -Syu`
openSUSE Tumbleweed	`sudo zypper refresh && sudo zypper dup`
openSUSE Leap	`sudo zypper refresh && sudo zypper update`

> 📝 Ubuntu-তে Snap অ্যাপগুলো আলাদা আপডেট হয় (স্বয়ংক্রিয়ভাবে দিনে কয়েকবার); হাতে করতে চাইলে `sudo snap refresh`। Flatpak থাকলে `flatpak update`।

রিবুট দরকার কিনা? kernel বা গুরুত্বপূর্ণ লাইব্রেরি আপডেট হলে রিবুট লাগে:

```
$ ls /var/run/reboot-required 2>/dev/null && cat /var/run/reboot-required
/var/run/reboot-required
*** System restart required ***
$ sudo reboot
```

(Fedora/RHEL-এ `sudo dnf needs-restarting -r`; এটা `dnf-plugins-core` / `dnf-utils` প্যাকেজের অংশ।)

৩. ড্রাইভার ও ফার্মওয়্যার (আসল হার্ডওয়্যারে)

```
$ ubuntu-drivers devices          # কোন হার্ডওয়্যারের জন্য কোন ড্রাইভার প্রস্তাবিত
$ sudo ubuntu-drivers install     # প্রস্তাবিত ড্রাইভার (যেমন NVIDIA) ইনস্টল
```

GUI-তে: **Software & Updates → Additional Drivers**। Fedora-তে NVIDIA ড্রাইভারের জন্য **RPM Fusion** রিপোজিটরি লাগে।

ফার্মওয়্যার (BIOS, SSD, টাচপ্যাড) আপডেট, সমর্থিত ডিভাইসে:

```
$ fwupdmgr refresh
$ fwupdmgr get-updates
$ fwupdmgr update
```

৪. প্রয়োজনীয় টুল বসানো

```
# Ubuntu/Debian
$ sudo apt install -y curl wget git vim htop tree build-essential
# Fedora/RHEL-পরিবার
$ sudo dnf install -y curl wget git vim htop tree
$ sudo dnf groupinstall -y "Development Tools"   # gcc, make ইত্যাদি
```

> 📝 RHEL/Rocky/Alma-তে `htop` ইত্যাদির জন্য আগে **EPEL** রিপোজিটরি লাগে: `sudo dnf install -y epel-release`।

৫. স্বয়ংক্রিয় নিরাপত্তা আপডেট (Ubuntu)

Ubuntu-তে **unattended-upgrades** সাধারণত ডিফল্টে চালু থাকে এবং নিরাপত্তা আপডেট নিজে বসায়। যাচাই/চালু করতে:

```
$ sudo apt install -y unattended-upgrades
$ sudo dpkg-reconfigure -plow unattended-upgrades
```

Fedora/RHEL-পরিবারে এর সমতুল্য `dnf-automatic`। অধ্যায় ৪৫-এ নিরাপত্তা নিয়ে বিস্তারিত।

৬. ফায়ারওয়াল ও ব্যাকআপ

- Ubuntu: `sudo ufw enable` (ডেস্কটপে সাধারণত ঝুঁকি কম, তবে চালু রাখা ভালো অভ্যাস); বিস্তারিত অধ্যায় ৩৯
- **Timeshift** (Mint-এ ডিফল্ট, Ubuntu-তে `sudo apt install timeshift`) দিয়ে সিস্টেম স্ন্যাপশট; VM-এ VirtualBox snapshot

প্রথম দিনের চেকলিস্ট (সংক্ষেপে)

```
[ ] cat /etc/os-release দিয়ে distro চেনা
[ ] sudo apt update && sudo apt upgrade  (বা dnf/pacman/zypper)
[ ] প্রয়োজনে রিবুট
[ ] VM হলে Guest Additions / open-vm-tools
[ ] আসল হার্ডওয়্যারে ড্রাইভার + fwupd
[ ] বেসিক টুল (curl, git, vim, htop)
[ ] ফায়ারওয়াল চালু
[ ] snapshot / Timeshift ব্যাকআপ
```

৩.১৩ সংশ্লিষ্ট কীবোর্ড শর্টকাট

VirtualBox (Host key = ডিফল্টে **ডান** `Ctrl`)

শর্টকাট	কাজ
`Host` (ডান Ctrl) একবার	VM-এর আটকে থাকা মাউস/কীবোর্ড ছাড়ানো
`Host + F`	পূর্ণ স্ক্রিন (Full-screen) চালু/বন্ধ
`Host + L`	Seamless mode (guest-এর উইন্ডো host ডেস্কটপে মিশে যায়)
`Host + C`	Scaled mode
`Host + Home`	(Full-screen/Seamless-এ) VM মেনু দেখানো
`Host + Del`	guest-এ `Ctrl+Alt+Del` পাঠানো
`Host + Backspace`	guest-এ `Ctrl+Alt+Backspace` পাঠানো
`Host + T`	Snapshot নেওয়া
`Host + E`	Screenshot নেওয়া
`Host + P`	VM pause / resume
`Host + R`	VM রিসেট (হঠাৎ রিস্টার্ট, সাবধান)
`Host + H`	ACPI shutdown সিগন্যাল (ভদ্রভাবে বন্ধ)
`Host + Q`	VM বন্ধ করার ডায়ালগ
`Ctrl + N` (Manager-এ)	নতুন VM

> 📝 guest-এর ভেতরে `Ctrl+Alt+F3` ইত্যাদি (virtual console) পাঠাতে VirtualBox-এ `Host + F3` চাপো, কারণ সরাসরি চাপলে host OS ধরে ফেলে। Virtual console নিয়ে অধ্যায় ৫।

VMware Workstation

শর্টকাট	কাজ
`Ctrl + Alt`	মাউস/কীবোর্ড guest থেকে ছাড়ানো
`Ctrl + G`	guest-এ ইনপুট ধরানো (grab)
`Ctrl + Alt + Enter`	পূর্ণ স্ক্রিন চালু/বন্ধ
`Ctrl + Alt + Insert`	guest-এ `Ctrl+Alt+Del` পাঠানো

বুট সময়ের শর্টকাট

শর্টকাট	কাজ
`F2` / `Del` / `F10` / `F1`	BIOS/UEFI সেটআপ (মডেল ভেদে; ৩.৯-এর টেবিল)
`F12` / `F11` / `F9` / `F8` / `Esc`	Boot menu (মডেল ভেদে)
`Shift` চেপে রাখা (BIOS বুট) / `Esc` চাপা (UEFI বুট)	লুকানো GRUB মেনু দেখানো (Ubuntu-তে একমাত্র OS হলে মেনু লুকানো থাকে)
`↑` / `↓`	GRUB-এ এন্ট্রি বাছাই
`e`	নির্বাচিত GRUB এন্ট্রি সাময়িক এডিট (kernel প্যারামিটার)
`Ctrl + X` বা `F10`	এডিট করা এন্ট্রি দিয়ে বুট
`c`	GRUB কমান্ড লাইন
`Esc`	এডিট বাতিল/আগের মেনু
Windows-এ `Shift` + Restart	Advanced startup (UEFI-তে যাওয়ার পথ)

Ubuntu ইনস্টলের পর কাজে লাগবে

শর্টকাট	কাজ
`Ctrl + Alt + T`	টার্মিনাল খোলা (Ubuntu)
`Super` (Windows key)	Activities / অ্যাপ সার্চ
`Super + L`	স্ক্রিন লক

Windows (WSL প্রসঙ্গে)

শর্টকাট	কাজ
`Win + X`	Power user মেনু (Disk Management, Terminal (Admin))
`Win + R` → `wsl`	সরাসরি ডিফল্ট লিনাক্সে ঢোকা
`Ctrl + Shift + T` (Windows Terminal)	নতুন ট্যাব

হাতে-কলমে অনুশীলন

ল্যাব ১: distro পরিচিতি (১০ মিনিট)

1. `https://distrowatch.com` খুলে প্রথম ১০টি distro দেখো। প্রত্যেকটার "Based on" ঘর থেকে লেখো কোন পরিবারের।
2. নিচের টেবিল পূরণ করো:

Distro	পরিবার	প্যাকেজ ম্যানেজার	LTS/fixed না rolling
Linux Mint			
Fedora			
EndeavourOS			
AlmaLinux			
openSUSE Tumbleweed			

ল্যাব ২: ISO নামিয়ে যাচাই (১৫ মিনিট)

1. Ubuntu 24.04 LTS Desktop ISO আর একই পেজের `SHA256SUMS` ফাইল নামাও।
2. লিনাক্স/WSL/macOS-এ: `sha256sum -c SHA256SUMS --ignore-missing` চালাও। `OK` দেখা উচিত।
3. Windows-এ: `Get-FileHash` দিয়ে হ্যাশ বের করে `SHA256SUMS`-এর লাইনের সঙ্গে চোখে মেলাও।
4. **পরীক্ষা:** ISO-র একটা কপি বানিয়ে (`cp ubuntu.iso test.iso`) তার শেষে একটা বাইট যোগ করো: `echo x >> test.iso`। এবার `sha256sum test.iso` চালিয়ে দেখো হ্যাশ সম্পূর্ণ আলাদা। শেষে `rm test.iso`।

ল্যাব ৩: প্রথম VM (৪৫–৬০ মিনিট)

1. ৩.৭-এর ধাপ মেনে VirtualBox-এ `ubuntu-lab` VM বানাও: ৪ GB RAM, ২ CPU, ৩০ GB ডিস্ক।
2. ইনস্টল শেষে টার্মিনালে:

   ```
   $ cat /etc/os-release
   $ hostnamectl
   $ uname -r
   $ lsblk
   ```

   `lsblk`-এর output দেখে লেখো: ডিস্কের নাম কী (`sda`), কয়টা পার্টিশন, কোনটা কোথায় mount।
3. `sudo apt update && sudo apt upgrade -y` চালাও। প্রয়োজনে রিবুট।
4. Guest Additions বসাও, উইন্ডো রিসাইজ করে দেখো স্ক্রিন বদলায় কিনা।
5. `fresh-install-updated` নামে snapshot নাও।

ল্যাব ৪: snapshot-এর জাদু (১০ মিনিট)

1. VM-এ একটা ফাইল বানাও: `echo "আমি থাকব না" > ~/test.txt`
2. VM বন্ধ করে snapshot `fresh-install-updated` **Restore** করো।
3. চালু করে `ls ~/test.txt` দেখো। ফাইলটা নেই! এটাই নিরাপদ পরীক্ষার ভিত্তি।

ল্যাব ৫ (Windows ব্যবহারকারীদের জন্য): WSL2 (২০ মিনিট)

1. Admin PowerShell-এ `wsl --install -d Ubuntu-24.04`, রিস্টার্ট, ইউজার বানাও।

2. `wsl -l -v` দিয়ে দেখো VERSION `2` কিনা।
3. লিনাক্সে ঢুকে `cat /etc/os-release`, `uname -r` (দেখবে kernel-এর নামে `microsoft-standard-WSL2` লেখা)।
4. `cd ~ && explorer.exe .` দিয়ে লিনাক্স হোম Windows Explorer-এ খোলো।
5. `ls /mnt/c` দিয়ে C: ড্রাইভ দেখো।
6. PowerShell-এ `wsl --shutdown`, তারপর আবার `wsl -l -v`। STATE `Stopped` দেখাবে।

ল্যাব ৬ (ঐচ্ছিক, সাবধানে): Live USB (২০ মিনিট)

1. একটা খালি পেনড্রাইভ দিয়ে balenaEtcher বা Rufus দিয়ে Ubuntu Live USB বানাও।
2. লিনাক্স থাকলে `dd` দিয়ে করার আগে `lsblk` দিয়ে USB **নিশ্চিতভাবে** চিনে নাও।
3. Boot menu key দিয়ে USB থেকে বুট করে **"Try Ubuntu"** মোডে Wi-Fi, সাউন্ড, টাচপ্যাড পরীক্ষা করো। **Install চাপবে না।**

ল্যাব ৭ (চ্যালেঞ্জ): দ্বিতীয় পরিবার

একটা **Rocky Linux** বা **AlmaLinux** VM বানাও (Minimal ISO-তে GUI নেই, শুধু টার্মিনাল)। ইনস্টলার (Anaconda) কীভাবে Ubuntu-র থেকে আলাদা লক্ষ করো। ইনস্টলের পর `sudo dnf update` চালাও আর `cat /etc/os-release`-এর `ID_LIKE` লক্ষ করো। ডিফল্ট ফাইলসিস্টেম দেখতে `lsblk -f` চালাও (XFS আর LVM দেখবে)।

সারসংক্ষেপ

মূল ধারণা:

- **distro** = kernel + টুলস + প্যাকেজ ম্যানেজার + ইনস্টলার + নীতি
- চার বড় পরিবার: **Debian/Ubuntu** (`apt`, `.deb`), **Red Hat** (`dnf`, `.rpm`), **Arch** (`pacman`), **SUSE** (`zypper`, `.rpm`)
- **LTS/fixed** = স্থিতিশীল, সার্ভারের জন্য; **rolling** = সবসময় নতুন, অভিজ্ঞদের জন্য
- fresher-এর জন্য: **Ubuntu 24.04 LTS**, প্রথমে **VM**-এ
- পদ্ধতি: VM (সবচেয়ে নিরাপদ) → WSL2 (Windows-এ CLI) → Live USB (চেখে দেখা/রেসকিউ) → Dual boot (পূর্ণ গতি)
- আধুনিক PC = **UEFI + GPT**, লাগে **ESP** (`/boot/efi`, FAT32)
- ইনস্টলের পর প্রথম কাজ: **আপডেট**

কমান্ড রেফারেন্স:

কমান্ড	কাজ
`sha256sum FILE` / `sha256sum -c SHA256SUMS --ignore-missing`	ISO-র হ্যাশ / যাচাই

কমান্ড	কাজ
`lsblk -o NAME,SIZE,TYPE,TRAN,MODEL`	ডিস্ক ও USB চেনা
`sudo dd if=X.iso of=/dev/sdX bs=4M status=progress oflag=sync`	Live USB লেখা (⚠️)
`wsl --install [-d Distro]`	WSL2 ইনস্টল
`wsl -l -v` / `wsl -l -o`	ইনস্টল করা / উপলব্ধ distro
`wsl --shutdown`	WSL বন্ধ
`cat /etc/os-release`	distro চেনা
`hostnamectl` / `uname -r`	সিস্টেম ও kernel তথ্য
`sudo apt update && sudo apt upgrade`	Debian/Ubuntu আপডেট
`sudo dnf upgrade --refresh`	Fedora/RHEL-পরিবার আপডেট
`sudo pacman -Syu`	Arch আপডেট
`sudo zypper dup` / `sudo zypper up`	Tumbleweed / Leap আপডেট
`sudo ubuntu-drivers install`	Ubuntu-তে প্রস্তাবিত ড্রাইভার
`fwupdmgr get-updates` / `update`	ফার্মওয়্যার আপডেট
`sudo update-grub`	GRUB মেনু পুনর্নির্মাণ (Debian/Ubuntu)

শর্টকাট রেফারেন্স:

শর্টকাট	কাজ
ডান `Ctrl` (VirtualBox Host)	মাউস ছাড়ানো
`Host+F` / `Host+T` / `Host+Del`	পূর্ণ স্ক্রিন / snapshot / Ctrl+Alt+Del
`Ctrl+Alt` (VMware)	মাউস ছাড়ানো
`F12` / `F11` / `F9` / `F8` / `Esc`	Boot menu (মডেল ভেদে)
`F2` / `Del`	BIOS/UEFI সেটআপ
`Shift` / `Esc` (বুটে)	GRUB মেনু দেখানো
`e` তারপর `Ctrl+X` (GRUB)	এন্ট্রি এডিট করে বুট
`Ctrl+Alt+T`	Ubuntu-তে টার্মিনাল

নিজেকে যাচাই করো

1. "Linux" আর "Linux distribution"-এর মধ্যে পার্থক্য কী?

2. Ubuntu, Linux Mint, Rocky Linux, Manjaro: প্রত্যেকটা কোন পরিবারের, এবং প্যাকেজ ইনস্টলের কমান্ড কী দিয়ে শুরু হয়?
3. LTS আর rolling release-এর একটা করে সুবিধা ও অসুবিধা বলো। প্রোডাকশন সার্ভারে কোনটা আর কেন?
4. Ubuntu `24.04`-এর `24` আর `04` কী বোঝায়? এটা LTS কিনা কীভাবে বুঝবে?
5. ISO নামানোর পর কেন `sha256sum` দিয়ে যাচাই করা উচিত?
6. `dd` দিয়ে Live USB বানানোর সময় `of=/dev/sdb1` না দিয়ে `of=/dev/sdb` কেন দিই? আর সবচেয়ে বড় বিপদ কী?
7. WSL1 আর WSL2-এর মূল প্রযুক্তিগত পার্থক্য কী?
8. UEFI সিস্টেমে ESP কী, কোন ফাইলসিস্টেমে থাকে, আর লিনাক্সে কোথায় mount হয়?
9. Dual boot করার আগে Windows-এ কোন তিনটি কাজ অবশ্যই করবে?
10. `sudo apt update` আর `sudo apt upgrade`-এর মধ্যে পার্থক্য কী?

▼ উত্তর দেখো

1. **Linux** কঠোর অর্থে শুধু kernel। **Distribution** হলো kernel-এর সঙ্গে GNU টুলস, shell, লাইব্রেরি, init system, প্যাকেজ ম্যানেজার, ইনস্টলার, ডেস্কটপ ও অ্যাপ মিলিয়ে বানানো সম্পূর্ণ, ব্যবহারযোগ্য অপারেটিং সিস্টেম।
2. Ubuntu ও Linux Mint → Debian পরিবার → `apt`। Rocky Linux → Red Hat পরিবার → `dnf`। Manjaro → Arch পরিবার → `pacman`।
3. **LTS:** সুবিধা স্থিতিশীলতা ও দীর্ঘ সাপোর্ট; অসুবিধা সফটওয়্যার পুরোনো। **Rolling:** সুবিধা সবসময় সর্বশেষ সফটওয়্যার ও হার্ডওয়্যার সাপোর্ট; অসুবিধা আপডেটে মাঝে মাঝে কিছু ভাঙতে পারে, নিয়মিত মনোযোগ লাগে। প্রোডাকশন সার্ভারে **LTS/এন্টারপ্রাইজ**, কারণ সেখানে অপরিবর্তনীয়তা ও নির্ভরযোগ্যতা সবচেয়ে জরুরি।
4. `24` = ২০২৪ সাল, `04` = এপ্রিল মাস। Ubuntu-তে জোড় বছরের এপ্রিল রিলিজ (`.04`) হলো LTS; তাই 24.04 LTS। নিশ্চিত হতে `cat /etc/os-release`-এর VERSION-এ "LTS" দেখো।
5. ডাউনলোডের সময় ফাইল নষ্ট হতে পারে, বা কেউ ক্ষতিকর পরিবর্তিত ISO দিতে পারে। হ্যাশ অফিশিয়াল মানের সঙ্গে মিললে নিশ্চিত হওয়া যায় ফাইল হুবহু আসল। (GPG দিয়ে SHA256SUMS ফাইলের সিগনেচার যাচাই করলে নিরাপত্তা আরও বাড়ে।)
6. আধুনিক ISO **hybrid**: এর ভেতরে নিজস্ব পার্টিশন টেবিল ও bootloader আছে, তাই পুরো ডিস্কের শুরু থেকে লিখতে হয়। সবচেয়ে বড় বিপদ: `of=`-এ ভুল ডিস্ক দিলে কোনো সতর্কতা ছাড়াই সেই ডিস্কের সব ডেটা চিরতরে মুছে যায়। তাই আগে `lsblk` দিয়ে দুবার মিলিয়ে নিতে হয়।
7. **WSL1** লিনাক্স system call-কে Windows kernel-এর call-এ অনুবাদ করত, আসল Linux kernel ছিল না। **WSL2** একটা হালকা utility VM-এর ভেতরে আসল Linux kernel চালায়, তাই সামঞ্জস্য প্রায় পূর্ণ (Docker, systemd চলে), যদিও `/mnt/c`-এর মতো Windows ফাইল অ্যাক্সেস তুলনামূলক ধীর।
8. **ESP (EFI System Partition)** একটা ছোট পার্টিশন (~৫১২ MB–১ GB), যেখানে UEFI firmware যে `.efi` bootloader ফাইল চালায় সেগুলো থাকে। ফাইলসিস্টেম **FAT32** (vfat), লিনাক্সে mount হয় `/boot/efi`-তে।
9. (ক) পুরো ডেটার **ব্যাকআপ**; (খ) **BitLocker রিকভারি কী** সংরক্ষণ (বা সাময়িক suspend); (গ) **Fast Startup বন্ধ**। এরপর Disk Management দিয়ে **Shrink Volume** করে খালি জায়গা বানানো।

10. `apt update` শুধু রিপোজিটরি থেকে কোন প্যাকেজের কোন ভার্সন পাওয়া যাচ্ছে তার **তালিকা রিফ্রেশ** করে, কিছু ইনস্টল করে না। `apt upgrade` সেই তালিকা দেখে ইনস্টল করা প্যাকেজগুলোকে আসলে **নতুন ভার্সনে আপগ্রেড** করে। তাই আগে `update`, তারপর `upgrade`।

অধ্যায় ৪: ডেস্কটপ পরিবেশ ও GUI পরিচিতি

লিনাক্স ইনস্টল করে প্রথমবার যখন কম্পিউটার চালু করো, তখন যা চোখে পড়ে তা হলো লগইন স্ক্রিন, তারপর একটা সুন্দর ডেস্কটপ, প্যানেল, আইকন আর উইন্ডো। এগুলো একসাথে মিলে **GUI** (Graphical User Interface, মানে ছবি আর মাউস দিয়ে কম্পিউটার চালানোর ব্যবস্থা)। Windows বা macOS-এ এই GUI অপারেটিং সিস্টেমের সাথে একদম জোড়া লাগানো। লিনাক্সে ব্যাপারটা আলাদা। এখানে GUI কয়েকটা আলাদা সফটওয়্যারের স্তর দিয়ে তৈরি, আর প্রতিটা স্তর চাইলে বদলে ফেলা যায়।

এই অধ্যায়ে আমরা এই স্তরগুলো খুলে দেখব। কোনটা কী কাজ করে, কেন এভাবে বানানো, আর দৈনন্দিন কাজে কোন কী-বোর্ড শর্টকাটগুলো তোমার সময় বাঁচাবে, সব এখানে আছে।

এই অধ্যায়ে যা শিখবে

- লিনাক্স GUI-এর স্তরগুলো: kernel, display server, compositor, window manager, display manager ও desktop environment, এবং এদের পার্থক্য
- X11 আর Wayland কী, কেন Wayland আসছে, আর তোমার সিস্টেম কোনটা চালাচ্ছে তা কীভাবে বুঝবে
- জনপ্রিয় desktop environment: GNOME, KDE Plasma, XFCE, Cinnamon (এবং আরও কিছু)
- ফাইল ম্যানেজার (Nautilus, Dolphin, Thunar, Nemo) ব্যবহার ও শর্টকাট
- সফটওয়্যার সেন্টার (GNOME Software, Ubuntu App Center, KDE Discover), Flatpak ও Snap-এর পরিচয়
- সেটিংস অ্যাপ, আর কমান্ড লাইন থেকে `gsettings` দিয়ে সেটিংস বদলানো
- workspace / virtual desktop দিয়ে কাজ গুছিয়ে রাখা
- GNOME ও KDE Plasma-র সম্পূর্ণ কী-বোর্ড শর্টকাট টেবিল
- টার্মিনাল খোলার সব উপায়, যা পরের অধ্যায়ের দরজা খুলে দেবে

৪.১ GUI আসলে কী, আর লিনাক্সে এটা আলাদা কেন

কম্পিউটারের আসল কাজ চালায় **kernel** (অপারেটিং সিস্টেমের মূল অংশ, যে হার্ডওয়্যার আর প্রোগ্রামের মাঝে মধ্যস্থতা করে; বিস্তারিত অধ্যায় ২-এ আছে)। kernel নিজে কোনো উইন্ডো, বাটন বা মাউস পয়েন্টার আঁকে না। সে শুধু জানে কীভাবে গ্রাফিক্স কার্ডে পিক্সেল পাঠাতে হয় আর কী-বোর্ড-মাউসের ইনপুট পড়তে হয়।

উইন্ডো আঁকা, কোন উইন্ডো কার উপরে থাকবে ঠিক করা, প্যানেল-মেনু দেখানো, এসব কাজ করে **user space**-এর (kernel-এর বাইরের সাধারণ প্রোগ্রামের জগৎ) কিছু প্রোগ্রাম। লিনাক্সে এই প্রোগ্রামগুলো আলাদা আলাদা প্রজেক্ট, আলাদা দল বানায়। তাই:

- একই Ubuntu-তে তুমি GNOME, KDE বা XFCE, যেকোনোটা চালাতে পারো।
- সার্ভারে কোনো GUI-ই রাখতে হয় না। এতে RAM আর CPU বাঁচে, নিরাপত্তার ঝুঁকিও কমে।

- GUI ক্র্যাশ করলেও kernel আর বাকি সিস্টেম সাধারণত চলতেই থাকে।

> 💡 **উপমা:** একটা রেস্টুরেন্টের কথা ভাবো। রান্নাঘর হলো kernel। খাবার পরিবেশনের জন্য টেবিল-চেয়ার (display server), কোন টেবিলে কে বসবে ঠিক করা ম্যানেজার (window manager), দরজায় দাঁড়ানো রিসেপশনিস্ট (display manager), আর পুরো ডেকোরেশন, মেনুকার্ড, লাইটিং (desktop environment)। Windows-এ পুরো রেস্টুরেন্ট একটা কোম্পানির প্যাকেজ। লিনাক্সে তুমি রিসেপশনিস্ট বা ডেকোরেশন আলাদাভাবে বদলাতে পারো।

৪.২ GUI-এর স্তরগুলো: কে কী করে

এই অংশটা এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ জায়গা। নামগুলো একরকম শোনায়, তাই নতুনরা প্রায়ই গুলিয়ে ফেলে।

৪.২.১ Display Server (ডিসপ্লে সার্ভার)

Display server হলো সেই প্রোগ্রাম যে GUI প্রোগ্রাম আর হার্ডওয়্যারের মাঝখানে বসে থাকে। তার কাজ:

1. কী-বোর্ড, মাউস, টাচপ্যাডের ইনপুট kernel থেকে নিয়ে সঠিক উইন্ডোতে পৌঁছে দেওয়া।
2. অ্যাপগুলো যা আঁকতে চায় তা স্ক্রিনে দেখানোর ব্যবস্থা করা।

লিনাক্সে দুটো display প্রযুক্তি চলে: পুরোনো **X11** (X Window System, যার প্রধান বাস্তবায়ন **Xorg**) আর নতুন **Wayland**। এ নিয়ে ৪.৩ অংশে বিস্তারিত বলছি।

> 📝 **মনে রাখো:** "সার্ভার" শব্দ শুনে ইন্টারনেট সার্ভার ভেবো না। এখানে সার্ভার মানে "সেবা দেওয়া প্রোগ্রাম"। অ্যাপগুলো ক্লায়েন্ট, তারা display server-এর কাছে অনুরোধ পাঠায়: "আমার উইন্ডোটা দেখাও।"

৪.২.২ Window Manager (উইন্ডো ম্যানেজার)

Window manager (WM) ঠিক করে:

- উইন্ডো কোথায় বসবে, কত বড় হবে
- উইন্ডোর টাইটেল বার, minimize/maximize/close বাটন (এগুলোকে বলে *decoration*)
- কোন উইন্ডো focus পাবে (মানে কী-বোর্ডে টাইপ করলে কোন উইন্ডোতে যাবে)
- উইন্ডো সরানো, resize, পাশাপাশি সাজানো (tiling)

WM দুই রকম:

ধরন	কীভাবে কাজ করে	উদাহরণ
Stacking (floating)	উইন্ডো একটার উপর আরেকটা কাগজের মতো স্তূপ হয়ে থাকে	Mutter, KWin, Xfwm4, Muffin, Openbox

ধরন	কীভাবে কাজ করে	উদাহরণ
Tiling	উইন্ডোগুলো নিজে থেকে স্ক্রিন ভাগ করে টাইলের মতো বসে, ওভারল্যাপ হয় না	i3, Sway, Hyprland, bspwm, dwm

৪.২.৩ Compositor (কম্পোজিটর)

আগে প্রতিটা অ্যাপ সরাসরি স্ক্রিনে আঁকত। **Compositor** আসার পর প্রতিটা উইন্ডো আগে নিজের জন্য আলাদা মেমরিতে (off-screen buffer) আঁকে, তারপর compositor সব উইন্ডোর ছবি একসাথে জুড়ে (composite করে) চূড়ান্ত স্ক্রিন বানায়। এতেই সম্ভব হয়েছে স্বচ্ছতা (transparency), ছায়া, অ্যানিমেশন, আর tearing ছাড়া মসৃণ ছবি।

X11-এ compositor একটা আলাদা ঐচ্ছিক জিনিস ছিল (যেমন `picom`)। Wayland-এ compositor-ই display server, আর সাধারণত সে-ই window manager। GNOME-এর **Mutter** আর KDE-র **KWin** একাই তিনটা কাজ করে: display server (Wayland-এ), compositor, window manager।

৪.২.৪ Display Manager (ডিসপ্লে ম্যানেজার / লগইন ম্যানেজার)

বুট শেষে যে গ্রাফিক্যাল লগইন স্ক্রিন আসে, ইউজারের নাম বাছো, পাসওয়ার্ড দাও, সেটাই **display manager (DM)**। নাম "display manager" হলেও এটা আসলে **লগইন ম্যানেজার**। এর কাজ:

- ইউজারের পাসওয়ার্ড যাচাই করা (PAM-এর মাধ্যমে, অধ্যায় ৪৫ দেখো)
- কোন session (GNOME, Plasma, XFCE...) চালু হবে তা বাছতে দেওয়া
- সেই session চালু করে দেওয়া

Display Manager	সাধারণত যার সাথে আসে
GDM (GNOME Display Manager; Debian/Ubuntu-তে প্যাকেজ নাম `gdm3`)	GNOME: Ubuntu, Fedora Workstation, RHEL, Debian GNOME
SDDM (Simple Desktop Display Manager)	KDE Plasma: Kubuntu, Fedora KDE
LightDM	XFCE, Cinnamon, MATE: Xubuntu, Linux Mint
plasma-login-manager / greetd ইত্যাদি	নতুন বা হালকা বিকল্প

display manager নিজে একটা systemd service। এটা কীভাবে বুটের সময় চালু হয় তা অধ্যায় ২৬ আর অধ্যায় ২৭-এ দেখবে।

৪.২.৫ Desktop Environment (ডেস্কটপ এনভায়রনমেন্ট)

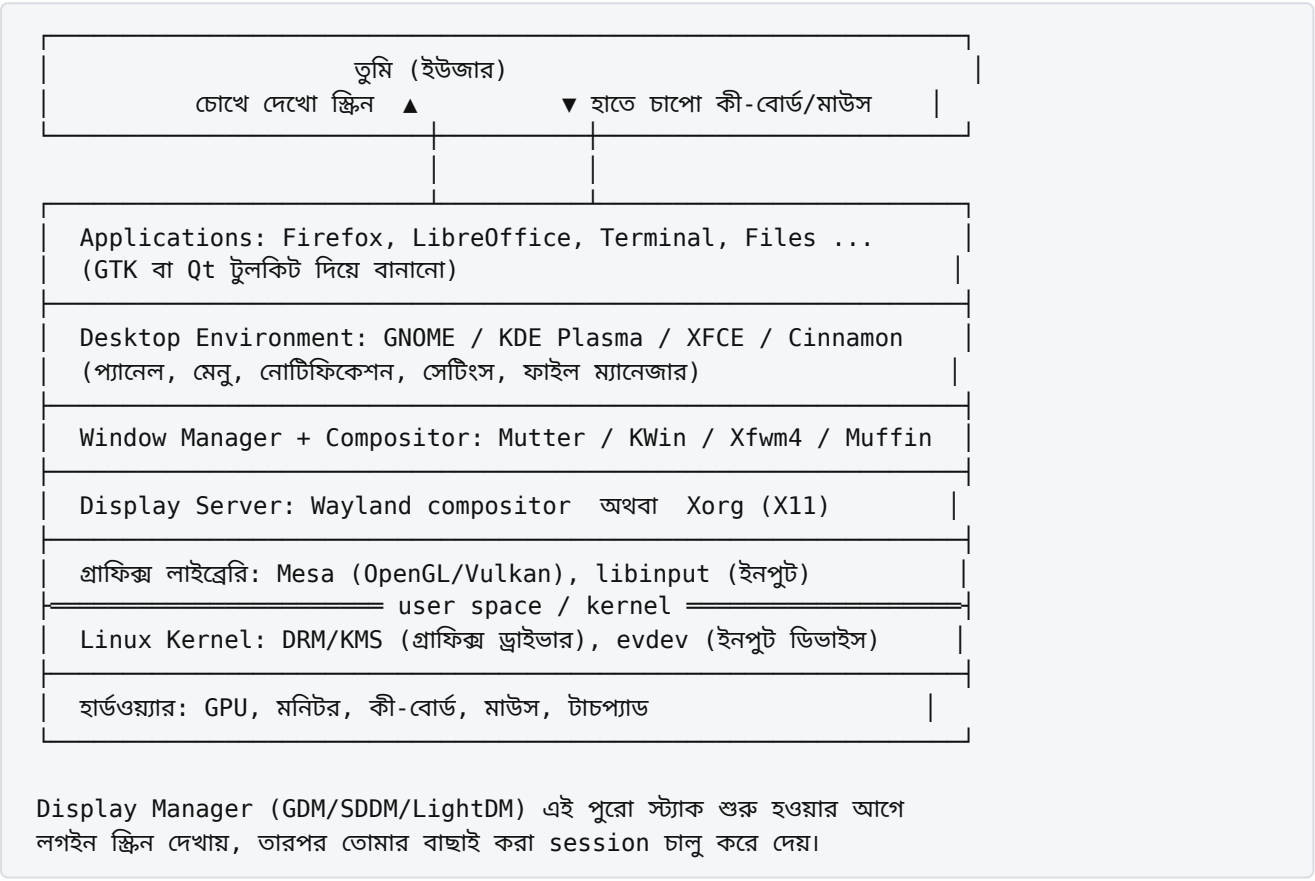
Desktop environment (DE) একটা পুরো প্যাকেজ। এর ভেতরে থাকে:

- একটা window manager/compositor
- প্যানেল, টাস্কবার, অ্যাপ মেনু বা লঞ্চার
- নোটিফিকেশন সিস্টেম
- ফাইল ম্যানেজার

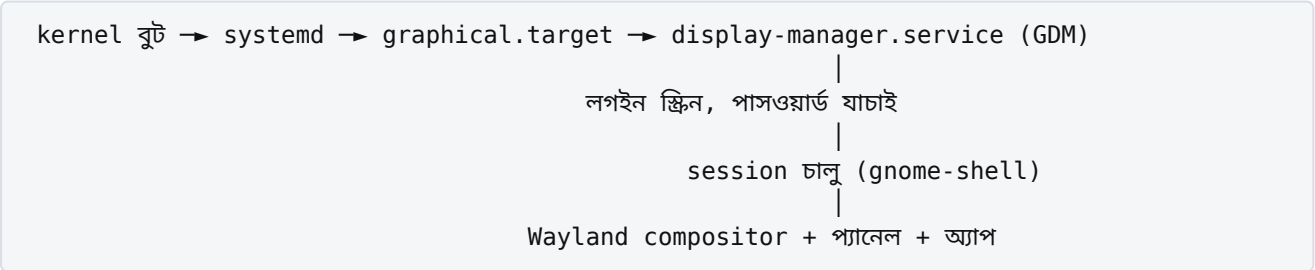
- সেটিংস অ্যাপ
- টার্মিনাল, টেক্সট এডিটর, ইমেজ ভিউয়ারের মতো নিজস্ব অ্যাপ
- একই রকম দেখতে থিম, আইকন, ফন্ট

মানে window manager শুধু উইন্ডো সামলায়, আর desktop environment তোমাকে পুরো "কম্পিউটার চালানোর অভিজ্ঞতা" দেয়।

৪.২.৬ সব স্তর একসাথে: ডায়াগ্রাম



বুট থেকে ডেস্কটপ পর্যন্ত ক্রমটা এরকম:



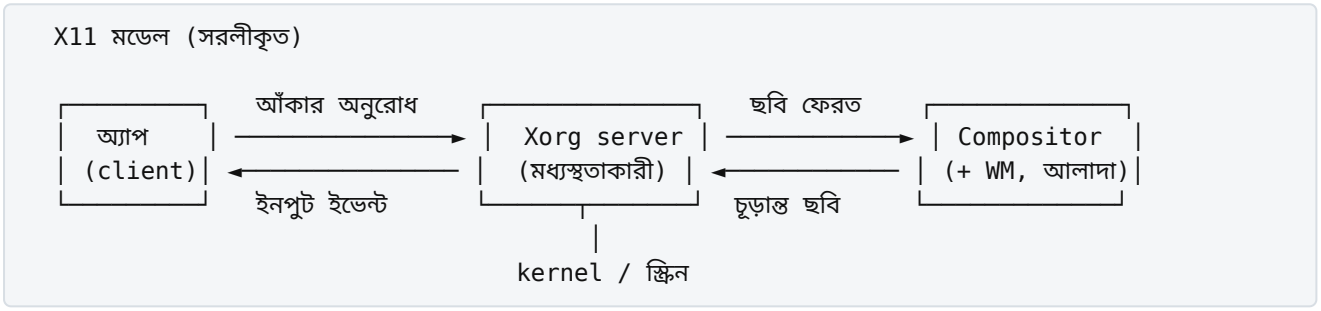
> 💡 **টিপস: GTK** আর **Qt** হলো দুটো "টুলকিট", মানে বাটন, মেনু, টেক্সটবক্স বানানোর লাইব্রেরি। GNOME, XFCE, Cinnamon GTK ব্যবহার করে, আর KDE Plasma ব্যবহার করে Qt। যেকোনো DE-তে যেকোনো টুলকিটের অ্যাপ চলে, শুধু দেখতে একটু বেমানান লাগতে পারে।

৪.৩ X11 বনাম Wayland

৪.৩.১ X11 (X Window System): পুরোনো সৈনিক

X11 এর জন্ম ১৯৮৪ সালে MIT-তে। এর সংস্করণ ১১ (X11) ১৯৮৭ থেকে চলছে। লিনাক্সে এর বাস্তবায়ন **Xorg server**।

X11-এর ডিজাইন **নেটওয়ার্ক-কেন্দ্রিক**। তখনকার দিনে একটা বড় কম্পিউটারে প্রোগ্রাম চলত, আর অনেক দুর্বল টার্মিনাল নেটওয়ার্ক দিয়ে তার ছবি দেখত। তাই X-এ:

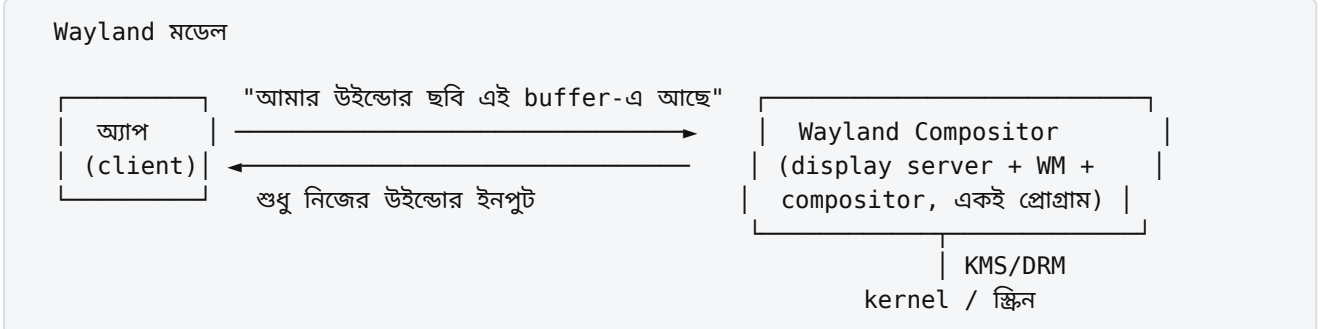


সমস্যাগুলো:

- **নিরাপত্তা:** X11-এ যেকোনো অ্যাপ অন্য যেকোনো অ্যাপের কী-বোর্ড ইনপুট পড়তে পারে, স্ক্রিনশট নিতে পারে। একটা দুষ্ট প্রোগ্রাম তোমার ব্রাউজারে টাইপ করা পাসওয়ার্ড শুনে ফেলতে পারে (keylogging)।
- **জটিলতা:** কয়েক দশকের পুরোনো কোড আর এমন অনেক ফিচার যা এখন কেউ ব্যবহার করে না।
- **অতিরিক্ত ধাপ:** ছবি app → X server → compositor → X server → স্ক্রিন, এভাবে ঘুরে যায়। এতে বিলম্ব আর tearing হয়।
- আধুনিক জিনিস যেমন বিভিন্ন মনিটরে বিভিন্ন স্কেলিং (একটায় 100%, আরেকটায় 150%), HDR, variable refresh rate এসবে দুর্বল।

৪.৩.২ Wayland: নতুন প্রজন্ম

Wayland ২০০৮ সালে শুরু হয়। এটা কোনো একক প্রোগ্রাম নয়, একটা **protocol** (অ্যাপ আর compositor কীভাবে কথা বলবে তার নিয়মকানুন)। প্রতিটা DE নিজের compositor-এ এই protocol বাস্তবায়ন করে: GNOME-এর Mutter, KDE-র KWin, Sway, Hyprland ইত্যাদি।



সুবিধা:

- **নিরাপত্তা:** একটা অ্যাপ শুধু নিজের উইন্ডোর ইনপুট পায়। স্ক্রিনশট বা স্ক্রিন শেয়ার করতে হলে **xdg-desktop-portal** আর **PipeWire**-এর মাধ্যমে অনুমতি চাইতে হয়। এজন্যই Zoom/Meet-এ স্ক্রিন শেয়ার করলে Wayland-এ একটা "কোন স্ক্রিন শেয়ার করবে?" ডায়ালগ আসে।
- **মসৃণতা:** "every frame is perfect", মানে tearing হয় না।
- **আধুনিক ফিচার:** প্রতি মনিটরে আলাদা fractional scaling, HDR (নতুন GNOME/Plasma-তে), touchpad gesture।

XWayland: যেসব পুরোনো অ্যাপ এখনো শুধু X11 বোঝে, তাদের জন্য Wayland session-এর ভেতরে একটা ছোট X server চলে, নাম **XWayland**। তাই পুরোনো অ্যাপও Wayland-এ সাধারণত ঠিকঠাক চলে।

৪.৩.৩ পাশাপাশি তুলনা

বিষয়	X11 (Xorg)	Wayland
শুরু	১৯৮৪ (X11: ১৯৮৭)	২০০৮
গঠন	আলাদা X server + আলাদা WM + আলাদা compositor	compositor-ই server, WM সাধারণত একই প্রোগ্রামে
নিরাপত্তা	যেকোনো অ্যাপ অন্যের ইনপুট/স্ক্রিন দেখতে পারে	অ্যাপগুলো একে অপরের থেকে আলাদা
স্ক্রিন শেয়ার/স্ক্রিনশট	সরাসরি	portal + PipeWire দিয়ে, অনুমতি নিয়ে
Tearing	হতে পারে	মূলত হয় না
Fractional / মিশ্র স্কেলিং	দুর্বল	ভালো
নেটওয়ার্কে GUI (`ssh -X`)	বিল্ট-ইন	নেই; বিকল্প `waypipe` , RDP
পুরোনো অ্যাপ	সব চলে	XWayland দিয়ে চলে
`xdotool` , `xrandr` , `xkill` জাতীয় টুল	কাজ করে	অনেকগুলো কাজ করে না (Wayland-নিজস্ব বিকল্প লাগে)

৪.৩.৪ Distro-ভেদে বর্তমান অবস্থা

Distro	ডিফল্ট
Ubuntu 24.04 LTS (GNOME 46)	Wayland। লগইন স্ক্রিনে "Ubuntu on Xorg" বিকল্প এখনো আছে
Debian 12 (GNOME 43)	Wayland। কিছু পুরোনো NVIDIA ড্রাইভারে Xorg-এ ফিরে যায়
Fedora Workstation (GNOME)	Wayland। Fedora 43 থেকে GNOME-এর X11 session আর নেই
Fedora KDE (Plasma 6)	Wayland only (Fedora 40 থেকে X11 session নেই)
RHEL 9	Wayland ডিফল্ট, Xorg session এখনো আছে (deprecated)
RHEL 10	Xorg server সরানো হয়েছে, শুধু Wayland (পুরোনো অ্যাপের জন্য XWayland)
Linux Mint (Cinnamon)	X11 ডিফল্ট, Wayland পরীক্ষামূলক
Xubuntu / XFCE	X11 ডিফল্ট, XFCE 4.20 থেকে Wayland আংশিক/পরীক্ষামূলক

> ⚠️ **সতর্কতা:** পুরোনো টিউটোরিয়ালে প্রায়ই `xrandr`, `xdotool`, `xkill` এর মতো কমান্ড পাবে। এগুলো X11-এর জন্য বানানো। Wayland session-এ এরা হয় কাজ করে না, নয়তো শুধু XWayland-এ চলা অ্যাপের উপর কাজ করে। কোনো কমান্ড "কিছু হচ্ছে না" মনে হলে আগে দেখো তুমি কোন session-এ আছো।

৪.৩.৫ তুমি কোন session-এ আছো, কীভাবে বুঝবে

টার্মিনাল খোলো (কীভাবে খুলবে তা ৪.১০ অংশে আছে; শর্টকাটে Ubuntu-তে `Ctrl+Alt+T`) আর লেখো:

```
$ echo $XDG_SESSION_TYPE
wayland
```

`x11` দেখালে তুমি X11-এ আছো, `tty` দেখালে টেক্সট কনসোলে।

কোন desktop environment চলছে:

```
$ echo $XDG_CURRENT_DESKTOP
ubuntu:GNOME
```

Fedora-তে শুধু `GNOME`, KDE-তে `KDE`, XFCE-তে `XFCE`, Mint-এ `X-Cinnamon` দেখাবে।

আরেকটু বিস্তারিত দেখতে `loginctl` ব্যবহার করো (এটা systemd-এর লগইন ম্যানেজার `systemd-logind`-এর টুল):

```
$ loginctl
SESSION  UID USER  SEAT  LEAVER TTY  STATE  IDLE SINCE
      2 1000 rahim seat0 -      tty2 active no   -

1 sessions listed.

$ loginctl show-session 2 -p Type -p Desktop -p Service
Type=wayland
Desktop=ubuntu
Service=gdm-password
```

> 📝 **মনে রাখো:** `loginctl`-এর আউটপুটের কলাম systemd-এর সংস্করণভেদে একটু আলাদা হতে পারে। তোমার session নম্বর `2` না হয়ে অন্য কিছু হতে পারে; প্রথম কমান্ডের আউটপুট থেকে নম্বরটা নিয়ে দ্বিতীয় কমান্ডে বসাও।

Wayland-এ থাকলে দুটো environment variable থাকে:

```
$ echo $WAYLAND_DISPLAY
wayland-0
$ echo $DISPLAY
:0
```

`DISPLAY` থাকে কারণ XWayland চালু থাকে পুরোনো অ্যাপের জন্য। Environment variable জিনিসটা কী, তা অধ্যায় ৪৪-এ শিখবে।

৪.৩.৬ Session বদলানো

1. Log out করো।
2. লগইন স্ক্রিনে তোমার নামে ক্লিক করো।
3. GDM-এ নিচের ডান কোণে একটা গিয়ার (⚙) আইকন আসবে; SDDM-এ নিচের বাম দিকে session-এর ড্রপডাউন; LightDM-এ সাধারণত উপরের প্যানেলে আইকন।
4. সেখান থেকে "Ubuntu" (Wayland) বা "Ubuntu on Xorg" বা অন্য DE বেছে নাও, তারপর পাসওয়ার্ড দাও।

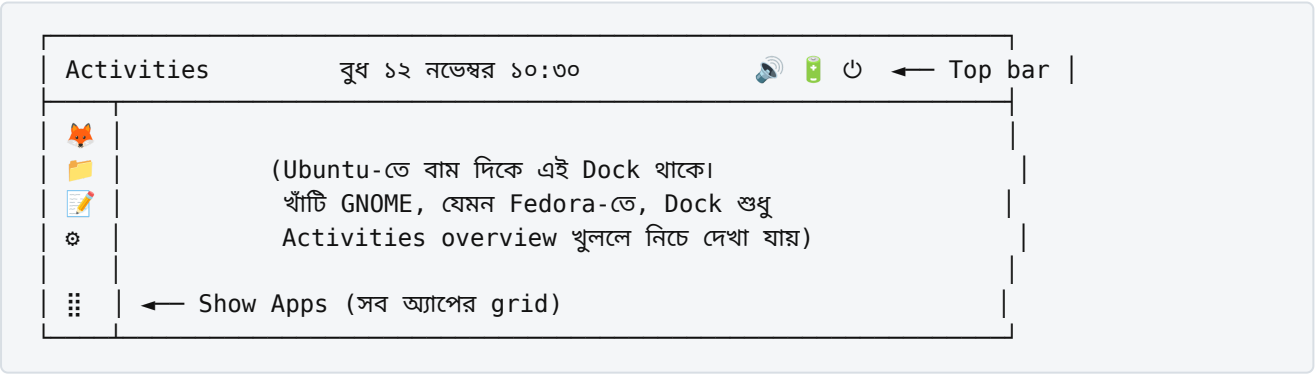
> 💡 **টিপস:** নতুন হিসেবে ডিফল্ট Wayland-ই রাখো। শুধু যদি কোনো নির্দিষ্ট অ্যাপ (পুরোনো স্ক্রিন রেকর্ডার, রিমোট সাপোর্ট টুল, কিছু গেম) ঠিকমতো না চলে, তখন Xorg session চেষ্টা করো।

৪.৪ জনপ্রিয় Desktop Environment

৪.৪.১ GNOME

Ubuntu, Fedora Workstation, Debian (ডিফল্ট), RHEL-এর ডিফল্ট DE। ২০১১ সালের GNOME 3 থেকে এর ডিজাইন বেশ আলাদা: কোনো ক্লাসিক স্টার্ট মেনু নেই, কোনো টাস্কবার নেই।

মূল অংশগুলো:



- **Top bar:** বাম দিকে "Activities" বা workspace নির্দেশক, মাঝে ঘড়ি (ক্লিক করলে ক্যালেন্ডার আর নোটিফিকেশন), ডানে **Quick Settings** (Wi-Fi, ব্লুটুথ, শব্দ, ব্যাটারি, ডার্ক মোড, পাওয়ার)।
- **Activities Overview:** `Super` কী (কী-বোর্ডের Windows লোগো কী) চাপলে খোলে। সব খোলা উইন্ডো, workspace আর একটা সার্চ বক্স একসাথে দেখা যায়।
- **Search:** Overview খুলে সরাসরি টাইপ করো, অ্যাপ, ফাইল, সেটিংস, এমনকি হিসাব (`25*4`) পর্যন্ত খুঁজে দেয়।
- **Dash/Dock:** প্রিয় অ্যাপগুলোর সারি।

- **Extensions:** GNOME-এর চেহারা আর আচরণ বদলানো যায় extension দিয়ে। Ubuntu-র Dock আর ডেস্কটপ আইকন আসলে Ubuntu-র যোগ করা extension।

শক্তি: পরিষ্কার, কম বিভ্রান্তিকর, টাচপ্যাড gesture চমৎকার (তিন আঙুলে উপরে সোয়াইপ করলে overview, পাশে করলে workspace বদল)। দুর্বলতা: অনেক জিনিস কাস্টমাইজ করতে extension বা **GNOME Tweaks** (`gnome-tweaks`) লাগে। RAM বেশি লাগে (খালি অবস্থায় মোটামুটি ১–১.৫ GB)।

৪.৪.২ KDE Plasma

Kubuntu, Fedora KDE, openSUSE, KDE neon-এর ডিফল্ট। বর্তমানে **Plasma 6** চলছে। Qt টুলকিটে বানানো।

- Windows-এর মতো চেনা লেআউট: নিচে প্যানেল (টাস্কবার), বাম কোণে **Application Launcher** (Kickoff), ডানে **System Tray** আর ঘড়ি।
- **KRunner:** `Alt+Space` বা `Alt+F2` চাপলে উপরে একটা সার্চ বার আসে; অ্যাপ চালানো, হিসাব, ইউনিট কনভার্শন, ফাইল খোঁজা সব এখান থেকে।
- **Widgets:** ডেস্কটপ আর প্যানেলে ঘড়ি, সিস্টেম মনিটর, নোট ইত্যাদি বসানো যায়।
- **Activities:** virtual desktop-এর চেয়েও বড় ধারণা। "অফিস" আর "বাড়ি" নামে দুটো Activity বানালে প্রতিটার আলাদা ওয়ালপেপার, উইজেট আর খোলা অ্যাপ থাকে।
- **Overview:** `Meta+W` চাপলে সব উইন্ডো আর ডেস্কটপ একসাথে দেখা যায় (KDE-তে Windows কী-কে বলে **Meta**)।

শক্তি: প্রায় সবকিছু কাস্টমাইজ করা যায়, নিজস্ব অ্যাপগুলো শক্তিশালী (Dolphin, Konsole, Kate, Spectacle, Okular)। আধুনিক Plasma বেশ হালকাও। দুর্বলতা: অপশন এত বেশি যে নতুনদের মাথা ঘুরতে পারে।

৪.৪.৩ XFCE

Xubuntu-র ডিফল্ট; Debian, Fedora, Mint-এ ঐচ্ছিক সংস্করণ আছে। GTK-ভিত্তিক।

- হালকা আর স্থির: খালি অবস্থায় সাধারণত ৫০০ MB-এর কম RAM-এ চলে।
- পুরোনো ধাঁচের পরিচিত লেআউট: প্যানেল, Whisker মেনু, ডেস্কটপ আইকন।
- উইন্ডো ম্যানেজার **Xfwm4**, ফাইল ম্যানেজার **Thunar**।
- পুরোনো ল্যাপটপ বা কম RAM-এর মেশিনের জন্য আদর্শ।
- দুর্বলতা: এখনো মূলত X11-নির্ভর; চেহারা কিছুটা পুরোনো ধাঁচের।

৪.৪.৪ Cinnamon

Linux Mint-এর বানানো, Mint-এর সবচেয়ে জনপ্রিয় সংস্করণ। GNOME থেকে জন্ম নিলেও দেখতে Windows 7/10-এর মতো:

- নিচে প্যানেল, বাম কোণে মেনু, ডানে system tray।
- উইন্ডো ম্যানেজার **Muffin** (Mutter-এর fork), ফাইল ম্যানেজার **Nemo**।
- Windows থেকে আসা নতুনদের জন্য সবচেয়ে কম ধাক্কা দেয়।

- Wayland সাপোর্ট এখনো পরীক্ষামূলক।

৪.৪.৫ আরও কিছু DE ও WM

নাম	ধরন	কাদের জন্য
MATE	পুরোনো GNOME 2-এর ধারাবাহিকতা, হালকা	ক্লাসিক লুক পছন্দ যাদের
LXQt	খুব হালকা, Qt-ভিত্তিক	খুব পুরোনো হার্ডওয়্যার (Lubuntu)
Budgie	আধুনিক, সরল	Ubuntu Budgie
COSMIC	System76-এর Rust-এ লেখা নতুন DE, Wayland-only	Pop!_OS
i3 / Sway	tiling WM (Sway হলো Wayland-এ i3-এর সমতুল্য)	কী-বোর্ড প্রেমী পাওয়ার ইউজার
Hyprland	অ্যানিমেশনসমৃদ্ধ tiling Wayland compositor	কাস্টমাইজেশন প্রেমী

৪.৪.৬ কোনটা বেছে নেবে: তুলনা টেবিল

বৈশিষ্ট্য	GNOME	KDE Plasma	XFCE	Cinnamon
টুলকিট	GTK	Qt	GTK	GTK
WM/Compositor	Mutter	KWin	Xfwm4	Muffin
Display manager (সাধারণত)	GDM	SDDM	LightDM	LightDM
ফাইল ম্যানেজার	Files (Nautilus)	Dolphin	Thunar	Nemo
টার্মিনাল	GNOME Terminal / Ptyxis	Konsole	xfce4-terminal	GNOME Terminal
সেটিংস অ্যাপ	Settings (`gnome-control-center`)	System Settings (`systemsettings`)	`xfce4-settings-manager`	`cinnamon-settings`
সফটওয়্যার সেন্টার	GNOME Software / Ubuntu App Center	Discover	(distro অনুযায়ী)	Mint Software Manager
Wayland	পূর্ণ, ডিফল্ট	পূর্ণ, ডিফল্ট	পরীক্ষামূলক	পরীক্ষামূলক
RAM (আনুমানিক, খালি অবস্থায়)	বেশি	মাঝারি	কম	মাঝারি
কাস্টমাইজেশন	extension দিয়ে	সর্বোচ্চ	অনেক	ভালো
কার জন্য	সরল, আধুনিক কাজ	পাওয়ার ইউজার, Windows-অভ্যস্ত	পুরোনো PC	Windows থেকে আসা নতুনরা

> 📝 **মনে রাখো:** RAM-এর সংখ্যাগুলো আনুমানিক। distro, সংস্করণ আর কোন অ্যাপ background-এ চলছে তার উপর অনেক বদলায়। নিজের মেশিনে মাপতে `free -h` ব্যবহার করো (অধ্যায় ৪৬)।

৪.৪.৭ একাধিক DE ইনস্টল করা

একই সিস্টেমে একাধিক DE রাখা যায়, লগইন স্ক্রিন থেকে বেছে নেওয়া যায়। প্যাকেজ ম্যানেজার নিয়ে বিস্তারিত অধ্যায় ৩০-এ; এখানে শুধু উদাহরণ:

```
# Ubuntu-তে KDE Plasma
$ sudo apt install kubuntu-desktop

# Ubuntu-তে XFCE
$ sudo apt install xubuntu-desktop

# Debian-এ (tasksel-এর task প্যাকেজ)
$ sudo apt install task-kde-desktop

# Fedora-তে KDE Plasma
$ sudo dnf install @kde-desktop-environment
```

ইনস্টলের সময় Ubuntu/Debian জিজ্ঞেস করতে পারে কোন display manager ডিফল্ট হবে (`gdm3` না `sddm`)। পরে বদলাতে:

```
$ sudo dpkg-reconfigure gdm3
```

এটা একটা মেনু খুলবে, সেখানে ইনস্টল থাকা display manager-গুলো থেকে বেছে নাও। Debian/Ubuntu-তে বর্তমান ডিফল্টটা এই ফাইলে লেখা থাকে:

```
$ cat /etc/X11/default-display-manager
/usr/sbin/gdm3
```

Fedora/RHEL-এ display manager হলো `display-manager.service` নামের একটা symlink। কোনটা চলছে দেখতে:

```
$ systemctl status display-manager
● gdm.service - GNOME Display Manager
     Loaded: loaded (/usr/lib/systemd/system/gdm.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-11-12 09:02:11 +06; 1h 28min ago
...
```

> ⚠️ **সতর্কতা:** "kubuntu-desktop" বা "xubuntu-desktop" এর মতো meta-package অনেক অ্যাপ নিয়ে আসে, ডাউনলোড ১–২ GB হতে পারে, আর মেনুতে প্রতিটা কাজের দুটো করে অ্যাপ দেখা যাবে। শুধু চেখে দেখতে চাইলে আগে Live USB বা virtual machine-এ (অধ্যায় ৪৮) চেষ্টা করো।

৪.৫ ফাইল ম্যানেজার

ফাইল ম্যানেজার হলো Windows-এর File Explorer বা macOS-এর Finder-এর সমতুল্য। ভেতরে ভেতরে এটা ঠিক সেই কাজগুলোই করে যা তুমি টার্মিনালে `ls`, `cp`, `mv`, `rm` দিয়ে করবে (অধ্যায় ৮, অধ্যায় ৯)। পার্থক্য শুধু মাউস দিয়ে করা।

DE	ফাইল ম্যানেজার	কমান্ড
GNOME	Files (আগের নাম Nautilus)	`nautilus`
KDE Plasma	Dolphin	`dolphin`
XFCE	Thunar	`thunar`
Cinnamon	Nemo	`nemo`

৪.৫.১ সাধারণ ধারণা

- **Home folder:** প্রতিটা ইউজারের নিজস্ব ফোল্ডার, `/home/তোমার-নাম`, সংক্ষেপে `~`। ফাইল ম্যানেজার খুললে সাধারণত এখানেই শুরু হয়। ভেতরে থাকে `Desktop`, `Documents`, `Downloads`, `Music`, `Pictures`, `Videos` ইত্যাদি। এসব ফোল্ডারের নাম `~/.config/user-dirs.dirs` ফাইলে সংরক্ষিত থাকে।
- **Hidden file:** যে ফাইল বা ফোল্ডারের নাম ডট (`.`) দিয়ে শুরু, যেমন `.bashrc` বা `.config`, সেটা লুকানো থাকে। এটা কোনো নিরাপত্তা ব্যবস্থা নয়, শুধু চোখের আরামের জন্য। `Ctrl+H` চাপলে দেখা যায়।
- **Trash (রিসাইকেল বিন):** ফাইল ম্যানেজারে Delete চাপলে ফাইল আসলে মোছে না, `~/.local/share/Trash/files/`-এ সরে যায়। কিন্তু টার্মিনালের `rm` কমান্ড সরাসরি মুছে ফেলে, Trash-এ যায় না।
- **Other Locations / Devices:** USB ড্রাইভ, অন্য পার্টিশন, নেটওয়ার্ক শেয়ার এখানে দেখা যায়। USB ঢোকালে ফাইল ম্যানেজার নিজেই সাধারণত `/media/তোমার-নাম/ড্রাইভের-নাম`-এ (Fedora/RHEL-এ `/run/media/তোমার-নাম/...`) mount করে। Mount জিনিসটা কী তা অধ্যায় ৩২-এ।
- **সার্ভারে সংযোগ:** অ্যাড্রেস বারে `sftp://user@192.168.1.10/` বা `smb://server/share` লিখে রিমোট ফোল্ডার খোলা যায় (GNOME-এ এটা **GVfs** নামের স্তর সামলায়, KDE-তে **KIO**)।

> ⚠️ **সতর্কতা:** `Shift+Delete` চাপলে ফাইল Trash-এ না গিয়ে সরাসরি মুছে যায়। টার্মিনালে `rm` করলেও তাই। এর কোনো "undo" নেই।

৪.৫.২ ফাইল ম্যানেজার থেকে টার্মিনালে যাওয়া

- **Nautilus:** ফোল্ডারের খালি জায়গায় রাইট-ক্লিক → "Open in Terminal" (Ubuntu-তে বিল্ট-ইন; অন্যান্য distro-তে `nautilus-extension-gnome-terminal` বা distro-ভেদে সমতুল্য প্যাকেজ লাগতে পারে)।

- **Dolphin:** `F4` চাপলে নিচে একটা ছোট টার্মিনাল প্যানেল আসে, যেটা ফোল্ডার বদলালে নিজেও সেই ফোল্ডারে যায়। `Shift+F4` আলাদা Konsole উইন্ডো খোলে।
- **Thunar / Nemo:** রাইট-ক্লিক → "Open Terminal Here"।

উল্টো দিকে, টার্মিনাল থেকে বর্তমান ফোল্ডার ফাইল ম্যানেজারে খুলতে:

```
$ xdg-open .
```

`xdg-open` যেকোনো ফাইল তার ডিফল্ট অ্যাপে খোলে: `xdg-open report.pdf` PDF ভিউয়ারে, `xdg-open https://kernel.org` ব্রাউজারে। এটা DE-নিরপেক্ষ, সব DE-তে চলে।

৪.৫.৩ ফাইল ম্যানেজারের কী-বোর্ড শর্টকাট

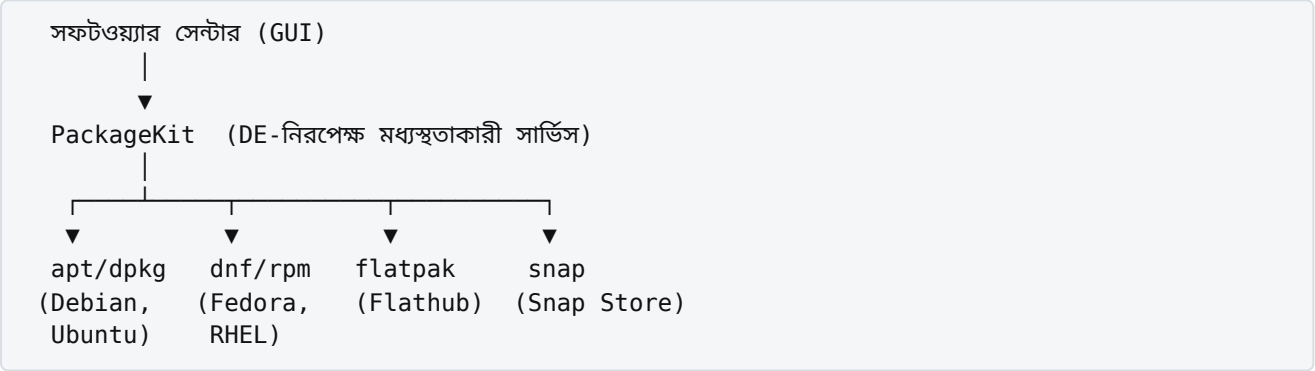
কাজ	Nautilus (GNOME)	Dolphin (KDE)	Thunar / Nemo
লুকানো ফাইল দেখাও/লুকাও	`Ctrl+H`	`Alt+.` (অথবা `Ctrl+H`)	`Ctrl+H`
অ্যাড্রেস বারে পাথ লেখা	`Ctrl+L`	`Ctrl+L`	`Ctrl+L`
নতুন ফোল্ডার	`Ctrl+Shift+N`	`F10`	`Ctrl+Shift+N`
নাম বদলানো	`F2`	`F2`	`F2`
Trash-এ পাঠানো	`Delete`	`Delete`	`Delete`
স্থায়ীভাবে মোছা	`Shift+Delete`	`Shift+Delete`	`Shift+Delete`
কপি / কাট / পেস্ট	`Ctrl+C` / `Ctrl+X` / `Ctrl+V`	একই	একই
সব নির্বাচন	`Ctrl+A`	`Ctrl+A`	`Ctrl+A`
নতুন ট্যাব	`Ctrl+T`	`Ctrl+T`	`Ctrl+T`
নতুন উইন্ডো	`Ctrl+N`	`Ctrl+N`	`Ctrl+N`
উপরের (parent) ফোল্ডার	`Alt+↑`	`Alt+↑` বা `Backspace`	`Alt+↑`
পিছনে / সামনে	`Alt+←` / `Alt+→`	`Alt+←` / `Alt+→`	`Alt+←` / `Alt+→`
Home ফোল্ডার	`Alt+Home`	`Alt+Home`	`Alt+Home`
ফাইল খোঁজা	`Ctrl+F` (বা সরাসরি টাইপ)	`Ctrl+F`	`Ctrl+F`
Properties	`Alt+Enter`	`Alt+Enter`	`Alt+Enter`
রিফ্রেশ	`F5` বা `Ctrl+R`	`F5`	`F5` বা `Ctrl+R`
Undo	`Ctrl+Z`	`Ctrl+Z`	`Ctrl+Z`
Split view (দুই প্যানে)	নেই	`F3`	`F3`
সাইডবার দেখাও/লুকাও	`F9`	`F9` (Places প্যানেল)	`Ctrl+B` (Thunar), `F9` (Nemo)

কাজ	Nautilus (GNOME)	Dolphin (KDE)	Thunar / Nemo
টার্মিনাল প্যানেল	নেই	`F4`	নেই
বড়/ছোট আইকন	`Ctrl+=` / `Ctrl+-`	`Ctrl+=` / `Ctrl+-`	`Ctrl+=` / `Ctrl+-`

> 📝 **মনে রাখো:** Dolphin-এর বেশির ভাগ শর্টকাট Settings → Configure Keyboard Shortcuts থেকে বদলানো যায়, আর সংস্করণভেদে দু-একটা ডিফল্ট বদলে যেতে পারে। মেনুতে প্রতিটা অপশনের পাশে তার বর্তমান শর্টকাট লেখা থাকে। সন্দেহ হলে মেনু খুলে দেখে নাও।

৪.৬ সফটওয়্যার সেন্টার

সফটওয়্যার সেন্টার হলো লিনাক্সের "অ্যাপ স্টোর"। পার্থক্য হলো, এখানের প্রায় সব সফটওয়্যার বিনামূল্যে, আর ভেতরে এটা আসলে **প্যাকেজ ম্যানেজারের** একটা গ্রাফিক্যাল মুখ।



DE / Distro	সফটওয়্যার সেন্টার	কী কী সামলায়
Ubuntu 24.04	**App Center** (`snap-store` snap প্যাকেজ)	Snap ডিফল্ট, `.deb` ফাইলও
Fedora, Debian GNOME	**GNOME Software** (`gnome-software`)	rpm/deb + Flatpak
KDE Plasma	**Discover** (`plasma-discover`)	distro প্যাকেজ + Flatpak (+ Snap, যদি প্লাগইন থাকে)
Linux Mint	**Software Manager** (`mintinstall`)	deb + Flatpak

৪.৬.১ তিন ধরনের প্যাকেজ: সংক্ষিপ্ত পরিচয়

- **Distro প্যাকেজ (`.deb` , `.rpm`):** distro-র নিজস্ব repository থেকে আসে। সিস্টেমের লাইব্রেরি শেয়ার করে, তাই ছোট; কিন্তু LTS distro-তে সংস্করণ কিছুটা পুরোনো হতে পারে।
- **Flatpak:** distro-নিরপেক্ষ। প্রতিটা অ্যাপ একটা স্যান্ডবক্সে চলে, নিজের "runtime" নিয়ে আসে। প্রধান ভাণ্ডার **Flathub**। Fedora, Mint, Debian-এ জনপ্রিয়।

- **Snap:** Canonical-এর (Ubuntu-র কোম্পানি) বানানো, ধারণা Flatpak-এর মতো। Ubuntu-তে ডিফল্ট; Firefox-ও Ubuntu-তে snap হিসেবে আসে।

এদের ভেতরের কাজ, কমান্ড লাইনে `apt`, `dnf`, `flatpak`, `snap` ব্যবহার — সব অধ্যায় ৩০-এ বিস্তারিত।

> 💡 **টিপস:** সফটওয়্যার সেন্টারে একই অ্যাপ (যেমন VLC) দুই-তিনবার দেখা যেতে পারে। অ্যাপের পাতায় "Source" বা ড্রপডাউনে দেখা যায় কোনটা Snap, কোনটা Flatpak, কোনটা distro-র deb/rpm। একটাই ইনস্টল করো, নইলে মেনুতে দুটো আইকন পাবে।

> ⚠️ **সতর্কতা:** ইন্টারনেট থেকে এলোমেলো `.deb` / `.rpm` ফাইল বা স্ক্রিপ্ট ডাউনলোড করে ইনস্টল কোরো না। সফটওয়্যার সেন্টার বা অফিশিয়াল repository-র সফটওয়্যার স্বাক্ষরযুক্ত (signed) আর যাচাই করা।

৪.৬.২ সিস্টেম আপডেট

GUI-তেও আপডেট খুব সহজ:

- **Ubuntu:** "Software Updater" অ্যাপ নিজে থেকেই নোটিফিকেশন দেয়।
- **Fedora (GNOME Software):** Updates ট্যাব। Fedora সিস্টেম আপডেট ডাউনলোড করে রিস্টার্টের সময় ইনস্টল করে ("offline update"), যাতে চলন্ত প্রোগ্রামের ফাইল বদলাতে না হয়।
- **KDE Discover:** Updates অংশ, প্যানেলেও আপডেট আইকন আসে।

৪.৭ সেটিংস অ্যাপ

প্রতিটা DE-র নিজস্ব সেটিংস অ্যাপ আছে। সাধারণ যা যা পাবে:

অংশ	কী বদলানো যায়
Network / Wi-Fi	Wi-Fi, ইথারনেট, VPN, প্রক্সি (ভেতরে NetworkManager, দেখো অধ্যায় ৩৬)
Bluetooth	ডিভাইস জোড়া লাগানো
Displays	রেজোলিউশন, স্কেলিং, একাধিক মনিটর, Night Light
Sound	আউটপুট/ইনপুট ডিভাইস, ভলিউম (ভেতরে PipeWire)
Power	স্ক্রিন বন্ধ হওয়ার সময়, সাসপেন্ড, পাওয়ার মোড
Appearance	লাইট/ডার্ক থিম, ওয়ালপেপার, অ্যাকসেন্ট রং
Keyboard	লেআউট/ইনপুট সোর্স (যেমন বাংলা), শর্টকাট
Mouse & Touchpad	গতি, natural scrolling, tap-to-click
Users	ইউজার যোগ করা, পাসওয়ার্ড বদলানো (অধ্যায় ১৫)

অংশ	কী বদলানো যায়
Region & Language	ভাষা, তারিখ/সংখ্যার ফরম্যাট
Date & Time	টাইমজোন, স্বয়ংক্রিয় সময়
Privacy / Security	লক স্ক্রিন, লোকেশন, ফাইল হিস্ট্রি
Default Apps	কোন ব্রাউজার, মেইল, মিউজিক প্লেয়ার ডিফল্ট
About / System	হার্ডওয়্যার তথ্য, OS সংস্করণ

টার্মিনাল থেকে সেটিংস খোলা:

```
$ gnome-control-center            # GNOME Settings
$ gnome-control-center display    # সরাসরি Displays প্যানেল
$ systemsettings                  # KDE System Settings
$ xfce4-settings-manager          # XFCE
$ cinnamon-settings               # Cinnamon
```

৪.৭.১ বাংলায় লেখার জন্য কী-বোর্ড যোগ করা

GNOME-এ: Settings → Keyboard → Input Sources → "+" → Bangla (Bengali) → পছন্দের লেআউট (যেমন Bangla (Probhat))। এরপর `Super+Space` চাপলে ইংরেজি আর বাংলার মধ্যে বদল হবে। Avro-র মতো ফোনেটিক টাইপিং চাইলে **IBus** বা **Fcitx5** input method framework-এর সাথে `ibus-m17n` বা OpenBangla Keyboard ব্যবহার করা হয়। KDE-তে System Settings → Keyboard → Layouts, বা Input Method থেকে।

৪.৭.২ ভেতরে কীভাবে কাজ করে: GSettings ও dconf

GNOME-এর প্রায় সব সেটিংস সংরক্ষিত থাকে একটা কী-ভ্যালু ডাটাবেজে, নাম **dconf**, ফাইল `~/.config/dconf/user`। এটা binary ফাইল, হাতে এডিট করা যায় না। অ্যাপগুলো এটা পড়ে **GSettings** API দিয়ে। টার্মিনাল থেকে একই কাজ করে `gsettings` কমান্ড।

```
gsettings get <schema> <key>
gsettings set <schema> <key> <value>
gsettings reset <schema> <key>
gsettings list-keys <schema>
gsettings list-recursively [schema]
```

সাব-কমান্ড	কাজ
`get`	একটা সেটিং-এর বর্তমান মান দেখায়
`set`	মান বদলায় (তৎক্ষণাৎ কার্যকর)
`reset`	ডিফল্টে ফিরিয়ে দেয়
`list-schemas`	সব schema-র নাম
`list-keys`	একটা schema-র সব key

সাব-কমান্ড	কাজ
`list-recursively`	schema-র সব key আর মান একসাথে
`range`	একটা key কোন কোন মান নিতে পারে
`monitor`	কোনো সেটিং বদলালে লাইভ দেখায়

উদাহরণ: ডার্ক মোড চালু করা

```
$ gsettings get org.gnome.desktop.interface color-scheme
'default'
$ gsettings set org.gnome.desktop.interface color-scheme 'prefer-dark'
$ gsettings get org.gnome.desktop.interface color-scheme
'prefer-dark'
```

কোন মান বৈধ তা দেখতে:

```
$ gsettings range org.gnome.desktop.interface color-scheme
enum
'default'
'prefer-dark'
'prefer-light'
```

আরও কিছু কাজের উদাহরণ:

```
# ঘড়িতে সেকেন্ড দেখানো
$ gsettings set org.gnome.desktop.interface clock-show-seconds true

# ব্যাটারি শতাংশ দেখানো
$ gsettings set org.gnome.desktop.interface show-battery-percentage true

# টাইটেল বারে minimize, maximize, close তিনটা বাটন
$ gsettings set org.gnome.desktop.wm.preferences button-layout
':minimize,maximize,close'

# Settings অ্যাপে কোনো কিছু বদলালে কোন key বদলাল তা লাইভ দেখা
$ gsettings monitor org.gnome.desktop.interface
color-scheme: 'prefer-dark'
```

> 💡 **টিপস:** `gsettings monitor` চালু রেখে Settings অ্যাপে কোনো অপশন বদলাও। টার্মিনালে দেখবে ঠিক কোন key বদলাল। এভাবে যেকোনো GUI সেটিং-এর কমান্ড খুঁজে বের করে স্ক্রিপ্টে ব্যবহার করতে পারবে। গ্রাফিক্যালভাবে সব key ঘাঁটতে চাইলে `dconf-editor` অ্যাপ আছে।

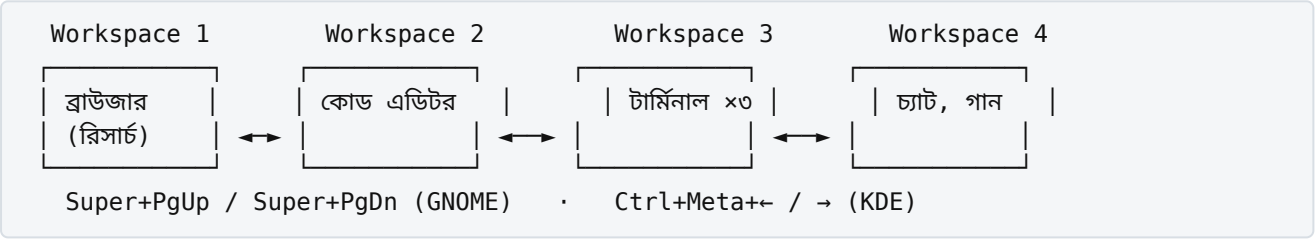
> ⚠️ **সতর্কতা:** `dconf reset -f /` তোমার সব GNOME সেটিংস মুছে ডিফল্ট করে দেয়। না বুঝে চালিও না।

KDE Plasma-র সেটিংস থাকে `~/.config/` ফোল্ডারে সাধারণ টেক্সট ফাইলে (যেমন `kdeglobals`, `kwinrc`)। কমান্ড লাইন থেকে বদলাতে Plasma 6-এ `kwriteconfig6` / `kreadconfig6` ব্যবহার হয়। XFCE-র সেটিংস সামলায় `xfconf`, কমান্ড `xfconf-query`।

৪.৮ Workspace / Virtual Desktop

ধরো একসাথে ১৫টা উইন্ডো খোলা: ব্রাউজার, কোড এডিটর, তিনটা টার্মিনাল, মিউজিক প্লেয়ার, চ্যাট। Alt+Tab করে খুঁজতে খুঁজতে মাথা খারাপ। **Workspace** (KDE-তে **Virtual Desktop**) এই সমস্যার সমাধান: একই মনিটরে একাধিক "ভার্চুয়াল স্ক্রিন"।

> 💡 **উপমা:** তোমার পড়ার টেবিলে চারটা ড্রয়ার আছে। একটায় অঙ্কের খাতা, একটায় ইংরেজি, একটায় গল্পের বই। যখন যেটা পড়বে সেই ড্রয়ার খুলে টেবিলে রাখো। বাকিগুলো হারায়নি, শুধু চোখের সামনে নেই।



৪.৮.১ GNOME-এ workspace

- ডিফল্টভাবে **dynamic**: একটা workspace-এ কোনো উইন্ডো রাখলে ডান পাশে স্বয়ংক্রিয়ভাবে একটা নতুন খালি workspace তৈরি হয়; খালি হলে মুছে যায়। সবসময় শেষে একটা খালি থাকে।
- workspace-গুলো পাশাপাশি (horizontal) সাজানো।
- Overview-তে উইন্ডো টেনে অন্য workspace-এ ফেলা যায়।
- টাচপ্যাডে তিন আঙুলে বামে/ডানে সোয়াইপ করলে workspace বদলায়।
- একাধিক মনিটরে ডিফল্টভাবে workspace বদলায় শুধু প্রধান মনিটরে; বাকি মনিটর স্থির থাকে (Settings → Multitasking থেকে বদলানো যায়)।

নির্দিষ্ট সংখ্যক workspace চাইলে (Settings → Multitasking-এও আছে):

```
$ gsettings set org.gnome.mutter dynamic-workspaces false
$ gsettings set org.gnome.desktop.wm.preferences num-workspaces 4
```

৪.৮.২ KDE Plasma-য় virtual desktop

- System Settings → Window Management → Virtual Desktops থেকে সংখ্যা, নাম আর কয় সারিতে সাজানো হবে ঠিক করো।
- `Meta+G` (Grid view) সব desktop একসাথে দেখায়; উইন্ডো টেনে সরানো যায়।

- `Ctrl+F1` … `Ctrl+F4` সরাসরি নির্দিষ্ট desktop-এ নিয়ে যায়।
- উইন্ডোর টাইটেল বারে রাইট-ক্লিক → "Move to Desktop" দিয়ে সরাও।
- এর উপরে **Activities** (`Meta+Tab` দিয়ে বদল), যা ৪.৪.২-এ বলেছি।

৪.৮.৩ Workspace ব্যবহারের ভালো অভ্যাস

1. প্রতিটা workspace-এর একটা নির্দিষ্ট কাজ রাখো (১ = ব্রাউজার, ২ = কোড, ৩ = টার্মিনাল, ৪ = যোগাযোগ)।
2. মাউসের বদলে শর্টকাট দিয়ে বদলাও। কয়েক দিনে হাতের অভ্যাস হয়ে যাবে।
3. পরে অধ্যায় ২৫-এ শিখবে `tmux`, যেটা টার্মিনালের ভেতরে একই রকম "workspace" দেয়, এমনকি SSH দিয়ে সার্ভারেও।

৪.৯ GNOME কী-বোর্ড শর্টকাট (সম্পূর্ণ টেবিল)

`Super` = কী-বোর্ডের Windows/লোগো কী। নিচের শর্টকাটগুলো GNOME 46–48 (Ubuntu 24.04, Fedora 40–42, Debian 13) অনুযায়ী; Debian 12-এর GNOME 43-এও প্রায় সবই এক। কোনো শর্টকাট শুধু Ubuntu-তে থাকলে তা উল্লেখ করেছি। Settings → Keyboard → View and Customize Shortcuts থেকে সব দেখা ও বদলানো যায়।

৪.৯.১ সিস্টেম ও নেভিগেশন

শর্টকাট	কাজ
`Super`	Activities Overview খোলা/বন্ধ (সার্চ সহ)
`Super+A`	সব অ্যাপের grid (Show Apps)
`Super+S`	Quick Settings মেনু
`Super+V` বা `Super+M`	নোটিফিকেশন তালিকা ও ক্যালেন্ডার
`Super+N`	নোটিফিকেশনে focus
`Alt+F2`	কমান্ড চালানোর ছোট বক্স (Run a command)
`Super+L`	স্ক্রিন লক
`Ctrl+Alt+Delete`	Log out ডায়ালগ
`Super+Space`	ইনপুট সোর্স/কী-বোর্ড লেআউট বদল (যেমন English ⇄ বাংলা)
`Ctrl+Alt+Tab`	সিস্টেম কন্ট্রোলে (top bar, dock) কী-বোর্ড focus
`Ctrl+Alt+T`	টার্মিনাল খোলা (Ubuntu-তে ডিফল্ট; Fedora/Debian-এ নিজে সেট করতে হয়)
`Super+D`	সব উইন্ডো লুকিয়ে ডেস্কটপ দেখানো (Ubuntu-তে ডিফল্ট; upstream GNOME-এ বন্ধ)
`Super+E`	Home ফোল্ডার ফাইল ম্যানেজারে (Ubuntu-তে)

৪.৯.২ অ্যাপ ও উইন্ডো বদলানো

শর্টকাট	কাজ
`Alt+Tab` / `Super+Tab`	অ্যাপের মধ্যে বদল (একই অ্যাপের সব উইন্ডো একসাথে গোনা হয়)
`Alt+Shift+Tab`	উল্টো দিকে অ্যাপ বদল
`Alt+`` / `Super+``	একই অ্যাপের উইন্ডোগুলোর মধ্যে বদল (`` হলো Tab-এর উপরের কী)
`Alt+Esc`	কোনো তালিকা না দেখিয়ে সরাসরি উইন্ডো বদল
`Super+1` … `Super+9`	Dock/Dash-এর ১ম … ৯ম অ্যাপ খোলা বা তাতে যাওয়া

৪.৯.৩ উইন্ডো ম্যানেজমেন্ট

শর্টকাট	কাজ
`Super+↑`	উইন্ডো maximize
`Super+↓`	maximize থেকে আগের আকারে / restore
`Super+←` / `Super+→`	স্ক্রিনের বাম/ডান অর্ধেকে বসানো (tile)
`Super+H`	উইন্ডো minimize (hide)
`Alt+F4`	উইন্ডো বন্ধ
`Alt+F10`	maximize টগল
`Alt+F7`	কী-বোর্ড দিয়ে উইন্ডো সরানো (তারপর তীর কী, শেষে Enter)
`Alt+F8`	কী-বোর্ড দিয়ে উইন্ডো resize
`Alt+Space`	উইন্ডো মেনু (Move, Resize, Always on Top…)
`Super+Shift+←` / `→`	উইন্ডো পাশের মনিটরে সরানো
`Super` চেপে মাউস দিয়ে টানা	উইন্ডোর যেকোনো জায়গা ধরে সরানো

৪.৯.৪ Workspace

শর্টকাট	কাজ
`Super+Page Up` / `Super+Page Down`	আগের/পরের workspace
`Ctrl+Alt+←` / `Ctrl+Alt+→`	আগের/পরের workspace (GNOME 40+)
`Super+Shift+Page Up` / `Page Down`	বর্তমান উইন্ডো আগের/পরের workspace-এ নিয়ে যাওয়া
`Ctrl+Alt+Shift+←` / `→`	একই কাজ, তীর দিয়ে
`Super+Home` / `Super+End`	প্রথম/শেষ workspace
`Super+Shift+Home` / `End`	উইন্ডো প্রথম/শেষ workspace-এ

৪.৯.৫ স্ক্রিনশট ও রেকর্ডিং (GNOME 42+)

শর্টকাট	কাজ
`Print`	স্ক্রিনশট টুল খোলে (অংশ/পুরো স্ক্রিন/উইন্ডো বাছো, রেকর্ডিং মোডও আছে)
`Shift+Print`	সঙ্গে সঙ্গে পুরো স্ক্রিনের স্ক্রিনশট
`Alt+Print`	সঙ্গে সঙ্গে বর্তমান উইন্ডোর স্ক্রিনশট
`Ctrl+Shift+Alt+R`	স্ক্রিন রেকর্ডিং শুরু/বন্ধ

স্ক্রিনশট `~/Pictures/Screenshots/`-এ সেভ হয় আর ক্লিপবোর্ডেও কপি হয়; রেকর্ডিং যায় `~/Videos/Screencasts/`-এ।

৪.৯.৬ Accessibility

শর্টকাট	কাজ
`Super+Alt+8`	Zoom চালু/বন্ধ
`Super+Alt+=` / `Super+Alt+-`	Zoom in / out
`Super+Alt+S`	Screen reader (Orca) চালু/বন্ধ

৪.৯.৭ টাচপ্যাড gesture (GNOME, Wayland)

Gesture	কাজ
তিন আঙুলে উপরে সোয়াইপ	Overview; আবার উপরে = App grid
তিন আঙুলে নিচে সোয়াইপ	ফিরে আসা
তিন আঙুলে বামে/ডানে	workspace বদল
দুই আঙুলে স্ক্রল	স্ক্রল (natural scrolling Settings-এ)

৪.১০ KDE Plasma কী-বোর্ড শর্টকাট (সম্পূর্ণ টেবিল)

`Meta` = Windows/লোগো কী। নিচের তালিকা Plasma 6 অনুযায়ী (Fedora KDE, Kubuntu 24.10+, KDE neon)। Kubuntu 24.04-এ Plasma 5.27 আছে; পার্থক্যগুলো টেবিলে বলেছি। System Settings → Keyboard → Shortcuts থেকে সব দেখা ও বদলানো যায়।

৪.১০.১ সিস্টেম ও লঞ্চার

শর্টকাট	কাজ
`Meta`	Application Launcher (Kickoff মেনু)

শর্টকাট	কাজ
`Alt+Space` বা `Alt+F2`	KRunner (সার্চ ও কমান্ড চালানো)
`Meta+L` বা `Ctrl+Alt+L`	স্ক্রিন লক
`Ctrl+Alt+Delete`	Log out / Shut down স্ক্রিন
`Ctrl+Alt+T`	Konsole টার্মিনাল
`Meta+E`	Dolphin ফাইল ম্যানেজার
`Ctrl+Esc`	System Monitor (প্রসেস তালিকা)
`Meta+V`	ক্লিপবোর্ড হিস্ট্রি (Klipper)
`Meta+.`	ইমোজি নির্বাচক
`Meta+P`	ডিসপ্লে/প্রজেক্টর কনফিগারেশন
`Meta+Alt+K`	পরের কী-বোর্ড লেআউটে বদল

> 📝 **মনে রাখো:** KDE-তে লেআউট বদলের শর্টকাট System Settings → Keyboard → Layouts-এ নিজে সেট করতে হতে পারে; distro-ভেদে ডিফল্ট আলাদা।

৪.১০.২ উইন্ডো বদল ও ওভারভিউ

শর্টকাট	কাজ
`Alt+Tab` / `Alt+Shift+Tab`	উইন্ডোর মধ্যে বদল (সামনে/পিছনে)
`Alt+`` `	একই অ্যাপের উইন্ডোর মধ্যে বদল
`Meta+W`	Overview (সব উইন্ডো + desktop; Plasma 6)
`Meta+G`	Grid view (সব virtual desktop; Plasma 6)
`Meta+1` … `Meta+9`	টাস্ক ম্যানেজারের ১ম … ৯ম অ্যাপে যাওয়া/খোলা
`Meta+D`	ডেস্কটপ উঁকি দেওয়া (Plasma 6; Plasma 5-এ `Ctrl+F12`)
`Meta+Tab`	Activity বদল

৪.১০.৩ উইন্ডো ম্যানেজমেন্ট

শর্টকাট	কাজ
`Meta+Page Up`	maximize
`Meta+Page Down`	minimize
`Meta+←` / `Meta+→`	বাম/ডান অর্ধেকে quick tile

শর্টকাট	কাজ
`Meta+↑` / `Meta+↓`	উপরের/নিচের অর্ধেকে quick tile (কোণে বসাতে পাশের তীরের সাথে মেলাও)
`Meta+T`	Tile editor (Plasma 6, নিজের লেআউট বানানো)
`Alt+F4`	উইন্ডো বন্ধ
`Alt+F3`	উইন্ডো অপারেশন মেনু
`Ctrl+Alt+Esc`	উইন্ডো "kill" মোড: পয়েন্টার খুলির মতো হয়, যে উইন্ডোতে ক্লিক করবে সেটা জোর করে বন্ধ (Esc চাপলে বাতিল)
`Meta+Shift+←` / `→`	উইন্ডো পাশের মনিটরে সরানো
`Meta` + মাউস টানা	উইন্ডোর যেকোনো জায়গা ধরে সরানো (Plasma 6; Plasma 5-এ modifier ছিল `Alt`)

৪.১০.৪ Virtual Desktop

শর্টকাট	কাজ
`Ctrl+F1` … `Ctrl+F4`	১ … ৪ নম্বর desktop-এ যাওয়া
`Ctrl+Meta+←` / `Ctrl+Meta+→`	আগের/পরের desktop
`Ctrl+Meta+↑` / `↓`	উপরের/নিচের সারির desktop (গ্রিডে সাজানো থাকলে)
`Meta+G`	সব desktop গ্রিডে দেখা

উইন্ডোকে নির্দিষ্ট desktop-এ পাঠানোর শর্টকাট ("Window to Desktop 2" ইত্যাদি) ডিফল্টভাবে খালি; শর্টকাট সেটিংসে নিজে দিয়ে নাও।

৪.১০.৫ স্ক্রিনশট (Spectacle)

শর্টকাট	কাজ
`Print`	Spectacle খোলা
`Meta+Print`	সক্রিয় উইন্ডোর স্ক্রিনশট
`Meta+Shift+Print` বা `Meta+Shift+S`	নির্বাচিত অংশের স্ক্রিনশট
`Shift+Print`	পুরো স্ক্রিনের স্ক্রিনশট

৪.১০.৬ GNOME বনাম KDE: একই কাজ, আলাদা শর্টকাট

কাজ	GNOME	KDE Plasma
অ্যাপ খোঁজা/চালানো	`Super`, টাইপ	`Meta` বা `Alt+Space`
সব উইন্ডো দেখা	`Super`	`Meta+W`
স্ক্রিন লক	`Super+L`	`Meta+L`

কাজ	GNOME	KDE Plasma
টার্মিনাল	`Ctrl+Alt+T` (Ubuntu)	`Ctrl+Alt+T`
ফাইল ম্যানেজার	`Super+E` (Ubuntu)	`Meta+E`
maximize	`Super+↑`	`Meta+Page Up`
অর্ধেক স্ক্রিনে tile	`Super+←/→`	`Meta+←/→`
পরের workspace	`Super+Page Down`	`Ctrl+Meta+→`
ক্লিপবোর্ড হিস্ট্রি	নেই (extension লাগে)	`Meta+V`
স্ক্রিনশট	`Print`	`Print`

৪.১০.৭ XFCE ও Cinnamon-এর দরকারি শর্টকাট

কাজ	XFCE (Xubuntu)	Cinnamon (Mint)
মেনু	`Super` (Whisker menu)	`Super`
টার্মিনাল	`Ctrl+Alt+T`	`Ctrl+Alt+T`
কমান্ড চালানো	`Alt+F2`	`Alt+F2`
স্ক্রিন লক	`Ctrl+Alt+L`	`Ctrl+Alt+L`
workspace বদল	`Ctrl+Alt+←/→`	`Ctrl+Alt+←/→`
উইন্ডো অন্য workspace-এ	`Ctrl+Alt+Shift+←/→`	`Ctrl+Alt+Shift+←/→`
সব workspace দেখা	নেই (ডিফল্টে)	`Ctrl+Alt+↑` (Expo)
সব উইন্ডো দেখা	নেই (ডিফল্টে)	`Ctrl+Alt+↓` (Scale)
ডেস্কটপ দেখানো	`Ctrl+Alt+D`	`Super+D`
tile	`Super+←/→` (Xubuntu-তে সংস্করণভেদে)	`Super+←/→`

৪.১০.৮ সব DE-তে প্রায় একই সর্বজনীন শর্টকাট

শর্টকাট	কাজ
`Ctrl+C` / `Ctrl+X` / `Ctrl+V`	কপি / কাট / পেস্ট (GUI অ্যাপে; টার্মিনালে আলাদা, নিচে দেখো)
`Ctrl+Z` / `Ctrl+Shift+Z` বা `Ctrl+Y`	Undo / Redo
`Ctrl+S`	সেভ
`Ctrl+Q`	অ্যাপ পুরো বন্ধ
`Ctrl+W`	ট্যাব/ডকুমেন্ট বন্ধ
`Ctrl+F`	খোঁজা
`F11`	ফুলস্ক্রিন
`Alt+F4`	উইন্ডো বন্ধ

শর্টকাট	কাজ
`Alt+Tab`	উইন্ডো বদল
মাউসের মাঝের বাটন (চাকা) ক্লিক	নির্বাচিত লেখা পেস্ট (লিনাক্সের "PRIMARY selection", নিচে দেখো)
`Ctrl+Alt+F3` ... `Ctrl+Alt+F6`	টেক্সট কনসোল (TTY)-এ যাওয়া

> 💡 **লিনাক্সের দুটো ক্লিপবোর্ড:** লিনাক্সে কোনো লেখা মাউস দিয়ে শুধু **সিলেক্ট** করলেই সেটা একটা আলাদা ক্লিপবোর্ডে (PRIMARY selection) চলে যায়। তারপর যেকোনো জায়গায় মাউসের **মাঝের বাটন** চাপলে পেস্ট হয়। `Ctrl+C` / `Ctrl+V`-এর সাধারণ ক্লিপবোর্ড (CLIPBOARD) আলাদা। দুটো একে অপরকে বদলায় না। এটা X11 যুগের ঐতিহ্য, Wayland-এও বেশির ভাগ DE রেখেছে।

> ⚠️ **সতর্কতা:** টার্মিনালের ভেতরে `Ctrl+C` মানে কপি না, চলন্ত প্রোগ্রাম থামানো! টার্মিনালে কপি-পেস্ট হলো `Ctrl+Shift+C` / `Ctrl+Shift+V`। বিস্তারিত অধ্যায় ৬-এ।

৪.১১ টার্মিনাল খোলা

এই বইয়ের বাকি অংশের বেশির ভাগ কাজ টার্মিনালে। তাই টার্মিনাল খোলার সব উপায় জেনে রাখো। টার্মিনাল আসলে কী, শেল কী, তা পরের অধ্যায়ে (অধ্যায় ৫) বিস্তারিত।

DE / Distro	ডিফল্ট টার্মিনাল অ্যাপ	কমান্ড
Ubuntu 24.04, Debian 12 (GNOME)	GNOME Terminal	`gnome-terminal`
Fedora 41+ Workstation	Ptyxis	`ptyxis`
কিছু GNOME distro	Console	`kgx`
KDE Plasma	Konsole	`konsole`
XFCE	Xfce Terminal	`xfce4-terminal`
Cinnamon (Mint)	GNOME Terminal	`gnome-terminal`

৪.১১.১ টার্মিনাল খোলার উপায়

1. **শর্টকাট:** `Ctrl+Alt+T` (Ubuntu, Mint, Kubuntu, Xubuntu-তে ডিফল্ট)।
2. **সার্চ করে:** GNOME-এ `Super` চেপে "terminal" লেখো, Enter। KDE-তে `Meta` চেপে "konsole" বা "terminal"।
3. **অ্যাপ মেনু থেকে:** GNOME-এ `Super+A` → Terminal; KDE/XFCE/Cinnamon-এ মেনু → System → Terminal।
4. **ফাইল ম্যানেজার থেকে:** রাইট-ক্লিক → "Open in Terminal" (৪.৫.২ দেখো)।
5. **Run dialog থেকে:** `Alt+F2` চেপে `gnome-terminal` (বা `konsole`, `ptyxis`) লিখে Enter।

6. **ডেস্কটপে রাইট-ক্লিক:** Ubuntu-তে ডেস্কটপের খালি জায়গায় রাইট-ক্লিক → "Open in Terminal"।

যদি `Ctrl+Alt+T` কাজ না করে (যেমন Fedora বা Debian GNOME-এ), নিজে বানাও: Settings → Keyboard → View and Customize Shortcuts → Custom Shortcuts → "+" → Name: `Terminal`, Command: `ptyxis` (বা `gnome-terminal`), Shortcut: `Ctrl+Alt+T`।

> 💡 **টিপস:** টার্মিনাল Dock/প্যানেলে pin করে রাখো (আইকনে রাইট-ক্লিক → "Pin to Dash" বা "Add to Favorites")। তাহলে `Super+1`-এর মতো শর্টকাটেও খুলবে।

৪.১১.২ GUI ছাড়া টেক্সট কনসোল (TTY)

GUI জমে গেলে বা ক্র্যাশ করলে `Ctrl+Alt+F3` চাপো। একটা পুরো কালো টেক্সট স্ক্রিন আসবে, যার নাম **TTY** (virtual console)। এখানে ইউজারনেম আর পাসওয়ার্ড দিয়ে লগইন করে কমান্ড চালানো যায়, যেমন জমে থাকা প্রোগ্রাম বন্ধ করা (অধ্যায় ২৪)।

```
Ubuntu 24.04.1 LTS rahim-pc tty3

rahim-pc login: rahim
Password:
rahim@rahim-pc:~$
```

GUI-তে ফিরতে: Ubuntu/Fedora-তে GDM সাধারণত `Ctrl+Alt+F1` (লগইন স্ক্রিন) আর তোমার session `Ctrl+Alt+F2`-এ থাকে। কোনটায় আছে তা distro আর display manager অনুযায়ী আলাদা; `loginctl`-এর `TTY` কলাম দেখে নিশ্চিত হতে পারো।

> 📝 **মনে রাখো:** TTY-তে পাসওয়ার্ড টাইপ করলে কিছুই দেখা যায় না, এমনকি তারকা চিহ্নও না। এটা স্বাভাবিক নিরাপত্তা ব্যবস্থা। টাইপ করে Enter চাপো।

৪.১১.৩ GUI চালু থাকবে কি না: systemd target

সিস্টেম বুট হয়ে GUI-তে যাবে নাকি শুধু টেক্সট মোডে থাকবে, তা ঠিক করে systemd-এর **default target**:

```
$ systemctl get-default
graphical.target
```

সার্ভারে সাধারণত `multi-user.target` থাকে (GUI ছাড়া)। বদলানো এবং target-এর বিস্তারিত অধ্যায় ২৬ ও অধ্যায় ২৭-এ।

৪.১২ GUI-সংক্রান্ত দরকারি কমান্ড ও সমস্যা সমাধান

কমান্ড	কাজ
`echo $XDG_SESSION_TYPE`	wayland / x11 / tty
`echo $XDG_CURRENT_DESKTOP`	কোন DE চলছে
`loginctl` / `loginctl show-session N`	session-এর তথ্য
`systemctl status display-manager`	কোন display manager চলছে
`xdg-open FILE_OR_URL`	ডিফল্ট অ্যাপে খোলা
`xdg-mime query default application/pdf`	PDF-এর ডিফল্ট অ্যাপ কোনটা
`gsettings get/set ...`	GNOME সেটিং পড়া/বদলানো
`gnome-extensions list`	ইনস্টল করা GNOME extension-এর তালিকা
`gnome-extensions disable UUID`	কোনো extension বন্ধ করা
`gnome-shell --version`	GNOME সংস্করণ
`plasmashell --version`	Plasma সংস্করণ
`journalctl -b --user`	তোমার session-এর লগ (অধ্যায় ২৮)

```
$ xdg-mime query default application/pdf
org.gnome.Evince.desktop

$ gnome-shell --version
GNOME Shell 46.0
```

৪.১২.১ ভেতরে কী: `.desktop` ফাইল

অ্যাপ মেনুতে যে আইকন আর নাম দেখো, প্রতিটার পেছনে একটা ছোট টেক্সট ফাইল আছে, যার এক্সটেনশন `.desktop`। থাকে `/usr/share/applications/` (সিস্টেমজুড়ে), `~/.local/share/applications/` (শুধু তোমার), আর Flatpak অ্যাপের জন্য `/var/lib/flatpak/exports/share/applications/`।

```
$ cat /usr/share/applications/org.gnome.Terminal.desktop
[Desktop Entry]
Name=Terminal
Comment=Use the command line
Exec=gnome-terminal --window
Icon=org.gnome.Terminal
Type=Application
Categories=GNOME;GTK;System;TerminalEmulator;
...
```

নিজের স্ক্রিপ্টের জন্য মেনুতে আইকন চাইলে `~/.local/share/applications/`-এ এমন একটা ফাইল বানালেই হয়। এটা freedesktop.org-এর একটা মানদণ্ড, তাই সব DE বোঝে। একইভাবে লগইনের সময় স্বয়ংক্রিয়ভাবে কোনো অ্যাপ চালাতে `.desktop` ফাইল রাখা হয় `~/.config/autostart/`-এ।

৪.১২.২ সাধারণ সমস্যা ও সমাধান

সমস্যা	সম্ভাব্য কারণ ও সমাধান
স্ক্রিন শেয়ারে কালো স্ক্রিন	Wayland-এ অ্যাপটা portal সাপোর্ট করে না। অ্যাপ আপডেট করো, `xdg-desktop-portal` চলছে কি না দেখো, শেষ উপায় Xorg session
একটা উইন্ডো জমে গেছে	`Alt+F4` ; না হলে GNOME নিজেই "Force Quit" জিজ্ঞেস করবে; KDE-তে `Ctrl+Alt+Esc`
পুরো GUI জমে গেছে	`Ctrl+Alt+F3` → TTY-তে লগইন → সমস্যার প্রসেস `kill` (অধ্যায় ২৪)
GNOME extension-এর পর অদ্ভুত আচরণ	`gnome-extensions disable` দিয়ে বন্ধ করে দেখো
লগইনের পর আবার লগইন স্ক্রিনে ফেরত (login loop)	প্রায়ই home ফোল্ডারের ফাইলের মালিকানা বা ডিস্ক ভর্তি। TTY থেকে `df -h` আর `ls -la ~` দেখো (অধ্যায় ১৬)
বিভিন্ন মনিটরে আলাদা স্কেলিং ঠিক হচ্ছে না	Wayland session ব্যবহার করো

> 💡 **টিপস:** পুরোনো X11 যুগে GNOME জমে গেলে `Alt+F2` → `r` দিয়ে GNOME Shell রিস্টার্ট করা যেত। Wayland-এ এটা কাজ করে না, কারণ GNOME Shell নিজেই display server; ওটা বন্ধ হলে পুরো session বন্ধ হয়ে যায়। তাই Wayland-এ log out/log in করাই উপায়।

হাতে-কলমে অনুশীলন

ল্যাব ১: তোমার GUI স্ট্যাক চেনো

1. `Ctrl+Alt+T` (বা ৪.১১.১-এর যেকোনো উপায়ে) টার্মিনাল খোলো।
2. নিচের কমান্ডগুলো চালাও আর উত্তরগুলো খাতায় লেখো:

```
$ echo $XDG_SESSION_TYPE
$ echo $XDG_CURRENT_DESKTOP
$ systemctl status display-manager --no-pager | head -3
$ gnome-shell --version     # অথবা KDE হলে: plasmashell --version
```

3. এবার নিচের টেবিলটা পূরণ করো:

স্তর	আমার সিস্টেমে
Display server / protocol	?
Desktop environment ও সংস্করণ	?
Display manager	?

স্তর	আমার সিস্টেমে
Window manager / compositor (৪.৪.৬-এর টেবিল দেখে)	?

ল্যাব ২: শর্টকাট দিয়ে একটা দিন

মাউস একপাশে সরিয়ে রাখো, শুধু কী-বোর্ড দিয়ে:

1. `Super` (KDE: `Meta`) চেপে "files" লিখে ফাইল ম্যানেজার খোলো।
2. `Ctrl+H` দিয়ে লুকানো ফাইল দেখো। `.bashrc` আর `.config` খুঁজে বের করো। আবার `Ctrl+H`।
3. `Ctrl+Shift+N` (Dolphin-এ `F10`) দিয়ে `linux-practice` নামে ফোল্ডার বানাও।
4. `Ctrl+L` চেপে অ্যাড্রেস বারে `/etc` লিখে Enter। এটা সিস্টেম কনফিগারেশনের ফোল্ডার (অধ্যায় ৭)।
5. `Alt+Home` দিয়ে Home-এ ফেরো।
6. `Super+←` দিয়ে ফাইল ম্যানেজার বাম অর্ধেকে রাখো, তারপর টার্মিনাল খুলে `Super+→` দিয়ে ডান অর্ধেকে (KDE-তে `Meta+←/→`)।
7. `Super+Page Down` (KDE: `Ctrl+Meta+→`) দিয়ে পরের workspace-এ যাও, সেখানে ব্রাউজার খোলো।
8. `Super+Page Up` দিয়ে ফেরো। `Super` চেপে overview-তে দেখো দুটো workspace-এ কী কী আছে।
9. `Super+L` দিয়ে স্ক্রিন লক করো, আবার আনলক করো।

ল্যাব ৩: GUI আর টার্মিনালের যোগাযোগ

1. টার্মিনালে `gsettings monitor org.gnome.desktop.interface` চালাও।
2. Settings → Appearance-এ গিয়ে Light/Dark বদলাও। টার্মিনালে কী দেখা গেল?
3. `Ctrl+C` দিয়ে monitor থামাও।
4. এবার কমান্ড দিয়ে ডার্ক মোড চালু করো:

```
$ gsettings set org.gnome.desktop.interface color-scheme 'prefer-dark'
```

5. আগের অবস্থায় ফেরাতে:

```
$ gsettings reset org.gnome.desktop.interface color-scheme
```

6. `mkdir -p ~/linux-practice && cd ~/linux-practice && xdg-open .` চালাও। ফাইল ম্যানেজারে ফোল্ডারটা খুলে গেল কি?

(KDE ব্যবহারকারী হলে ধাপ ১–৫ বাদ দিয়ে System Settings → Colors & Themes থেকে থিম বদলাও, আর `cat ~/.config/kdeglobals | head` দিয়ে দেখো সেটিং কোথায় লেখা হলো।)

ল্যাব ৪: TTY থেকে ঘুরে আসা

1. সব কাজ সেভ করো।
2. `Ctrl+Alt+F3` চাপো। ইউজারনেম আর পাসওয়ার্ড দিয়ে লগইন করো।

3. `loginctl` চালাও। এখন দুটো session দেখা যাওয়ার কথা: একটা GUI-র, একটা এই TTY-র।
4. `exit` লিখে লগআউট করো।
5. `Ctrl+Alt+F2` (না হলে `F1`, `F7`) চেপে GUI-তে ফেরো।

ল্যাব ৫: মেনুতে নিজের লঞ্চার (ঐচ্ছিক)

1. `~/.local/share/applications/hello-terminal.desktop` নামে একটা ফাইল বানাও (এডিটর হিসেবে আপাতত GUI টেক্সট এডিটর বা অধ্যায় ১৩-এর `nano` ব্যবহার করো):

   ```
   [Desktop Entry]
   Type=Application
   Name=Hello Terminal
   Comment=আমার প্রথম লঞ্চার
   Exec=gnome-terminal
   Icon=utilities-terminal
   Terminal=false
   Categories=Utility;
   ```

   (KDE-তে `Exec=konsole`, Fedora 41+-এ `Exec=ptyxis` লেখো।)
2. অ্যাপ মেনু খুলে "Hello Terminal" খোঁজো। পাওয়া গেলে ক্লিক করো।
3. কাজ শেষে ফাইলটা মুছে ফেলো।

সারসংক্ষেপ

মূল ধারণা:

পরিভাষা	এক লাইনে	উদাহরণ
Display server	অ্যাপ, ইনপুট আর স্ক্রিনের মধ্যস্থতাকারী	Xorg, Wayland compositor
Compositor	সব উইন্ডোর ছবি জুড়ে চূড়ান্ত স্ক্রিন বানায়	Mutter, KWin, picom
Window manager	উইন্ডোর অবস্থান, আকার, focus, decoration	Mutter, KWin, Xfwm4, i3
Display manager	গ্রাফিক্যাল লগইন স্ক্রিন, session চালু করে	GDM, SDDM, LightDM
Desktop environment	পুরো প্যাকেজ: WM + প্যানেল + অ্যাপ + সেটিংস	GNOME, KDE Plasma, XFCE, Cinnamon
XWayland	Wayland-এ পুরোনো X11 অ্যাপ চালানোর সেতু	

দ্রুত রেফারেন্স কমান্ড:

কমান্ড	কাজ
`echo $XDG_SESSION_TYPE`	Wayland না X11
`echo $XDG_CURRENT_DESKTOP`	কোন DE
`loginctl`	session তালিকা

কমান্ড	কাজ
`systemctl status display-manager`	display manager
`xdg-open .`	বর্তমান ফোল্ডার ফাইল ম্যানেজারে
`gsettings get/set/reset/monitor`	GNOME সেটিংস
`gnome-control-center` / `systemsettings`	সেটিংস অ্যাপ
`gnome-extensions list`	GNOME extension

সবচেয়ে দরকারি শর্টকাট:

কাজ	GNOME	KDE Plasma
লঞ্চার/সার্চ	`Super`	`Meta` , `Alt+Space`
সব উইন্ডো	`Super`	`Meta+W`
টার্মিনাল	`Ctrl+Alt+T` (Ubuntu)	`Ctrl+Alt+T`
অ্যাপ বদল	`Alt+Tab`	`Alt+Tab`
অর্ধেক স্ক্রিন	`Super+←/→`	`Meta+←/→`
maximize	`Super+↑`	`Meta+Page Up`
workspace বদল	`Super+Page Up/Down`	`Ctrl+Meta+←/→`
উইন্ডো অন্য workspace-এ	`Super+Shift+Page Up/Down`	(নিজে সেট করতে হয়)
লক	`Super+L`	`Meta+L`
স্ক্রিনশট	`Print`	`Print`
লুকানো ফাইল (ফাইল ম্যানেজার)	`Ctrl+H`	`Alt+.` / `Ctrl+H`
TTY	`Ctrl+Alt+F3`	`Ctrl+Alt+F3`

নিজেকে যাচাই করো

1. Window manager আর desktop environment-এর মধ্যে পার্থক্য কী? একটা করে উদাহরণ দাও।
2. Display manager কী কাজ করে? Ubuntu আর Kubuntu-তে ডিফল্ট display manager কোনটা?
3. X11-এর সবচেয়ে বড় নিরাপত্তা সমস্যা কী, আর Wayland কীভাবে তা সমাধান করে?
4. তুমি Wayland-এ আছো না X11-এ, কোন কমান্ড দিয়ে জানবে?
5. XWayland কী এবং কেন দরকার?
6. ফাইল ম্যানেজারে `Delete` আর `Shift+Delete`-এর পার্থক্য কী? আর টার্মিনালের `rm` কোনটার মতো আচরণ করে?
7. GNOME-এ টার্মিনাল থেকে ডার্ক মোড চালু করার কমান্ড কী? কোনো GUI সেটিং-এর key খুঁজে বের করার সহজ উপায় কী?

8. GNOME-এ বর্তমান উইন্ডোটাকে পরের workspace-এ পাঠানোর শর্টকাট কী? KDE-তে সব virtual desktop গ্রিডে দেখার শর্টকাট কী?
9. পুরো GUI জমে গেলে কী করবে?
10. অ্যাপ মেনুর আইকনগুলোর পেছনে কোন ধরনের ফাইল থাকে, আর সেগুলো কোথায় থাকে?

▼ **উত্তর দেখো**

1. Window manager শুধু উইন্ডোর অবস্থান, আকার, focus আর decoration সামলায় (যেমন i3, Openbox, Xfwm4)। Desktop environment একটা পুরো প্যাকেজ, যার ভেতরে একটা window manager ছাড়াও প্যানেল, মেনু, নোটিফিকেশন, ফাইল ম্যানেজার, সেটিংস আর নিজস্ব অ্যাপ থাকে (যেমন GNOME, KDE Plasma)।
2. Display manager গ্রাফিক্যাল লগইন স্ক্রিন দেখায়, পাসওয়ার্ড যাচাই করায় (PAM দিয়ে), কোন session চলবে তা বাছতে দেয় এবং সেটা চালু করে। Ubuntu-তে GDM (প্যাকেজ `gdm3`), Kubuntu-তে SDDM।
3. X11-এ যেকোনো অ্যাপ অন্য অ্যাপের কী-বোর্ড ইনপুট পড়তে আর স্ক্রিন দেখতে পারে, তাই keylogging সহজ। Wayland-এ প্রতিটা অ্যাপ শুধু নিজের উইন্ডোর ইনপুট পায়; স্ক্রিনশট/স্ক্রিন শেয়ারের জন্য xdg-desktop-portal আর PipeWire-এর মাধ্যমে ইউজারের অনুমতি লাগে।
4. `echo $XDG_SESSION_TYPE` — `wayland` বা `x11` দেখাবে। বিকল্প: `loginctl show-session <নম্বর> -p Type`।
5. XWayland হলো Wayland session-এর ভেতরে চলা একটা X server, যাতে যেসব পুরোনো অ্যাপ শুধু X11 বোঝে সেগুলোও চলতে পারে।
6. `Delete` ফাইলকে Trash-এ (`~/.local/share/Trash/`) পাঠায়, পরে ফেরত আনা যায়। `Shift+Delete` সরাসরি স্থায়ীভাবে মুছে ফেলে। টার্মিনালের `rm` `Shift+Delete`-এর মতো, Trash-এ যায় না।
7. `gsettings set org.gnome.desktop.interface color-scheme 'prefer-dark'`। key খোঁজার উপায়: `gsettings monitor <schema>` (বা `dconf watch /`) চালু রেখে Settings অ্যাপে অপশন বদলানো; অথবা `dconf-editor` দিয়ে ঘাঁটা।
8. GNOME: `Super+Shift+Page Down` (বা `Ctrl+Alt+Shift+→`)। KDE Plasma 6: `Meta+G`।
9. `Ctrl+Alt+F3` চেপে TTY-তে যাও, লগইন করো, জমে থাকা প্রোগ্রাম খুঁজে `kill` করো (অধ্যায় ২৪); দরকার হলে সেখান থেকে রিবুট। KDE-তে একটা উইন্ডো জমলে `Ctrl+Alt+Esc` দিয়ে সেটা বন্ধ করা যায়।
10. `.desktop` ফাইল (freedesktop.org মানদণ্ড)। থাকে `/usr/share/applications/` (সিস্টেমজুড়ে), `~/.local/share/applications/` (ব্যক্তিগত) এবং Flatpak-এর জন্য `/var/lib/flatpak/exports/share/applications/`।

অধ্যায় ৫: টার্মিনাল ও শেল পরিচিতি

> "GUI দিয়ে তুমি কম্পিউটারকে ইশারায় কথা বলো; টার্মিনালে তুমি তার সাথে পুরো বাক্যে কথা বলো।"

এতদিন তুমি মাউস দিয়ে আইকনে ক্লিক করে কাজ করেছ (অধ্যায় ৪)। এবার আমরা লিনাক্সের আসল শক্তির দরজায় পা রাখছি: **টার্মিনাল** আর **শেল**। লিনাক্স এক্সপার্টরা দিনের বেশিরভাগ সময় এখানেই কাটান। সার্ভারে সাধারণত কোনো GUI থাকেই না, তাই টার্মিনাল না জানলে সার্ভার চালানো যায় না।

ভয় পেও না। কালো স্ক্রিনে সাদা লেখা দেখতে কঠিন লাগে, কিন্তু নিয়মগুলো খুব সরল। এই অধ্যায় শেষে তুমি নিজে কমান্ড লিখতে পারবে, আর কোনো কমান্ড না জানলে নিজেই তার সাহায্য খুঁজে নিতে পারবে। এটাই সবচেয়ে বড় দক্ষতা।

এই অধ্যায়ে যা শিখবে

- terminal emulator, shell, console আর TTY আলাদা জিনিস কেন, আর এরা একসাথে কীভাবে কাজ করে
- `Ctrl+Alt+F1` – `F6` দিয়ে virtual console-এ যাওয়া ও ফিরে আসা
- কোন কোন shell আছে (`sh`, `bash`, `zsh`, `fish`), কোনটা কখন ব্যবহার হয়
- prompt (`user@host:~$` বনাম `#`) পড়তে শেখা
- একটা কমান্ডের গঠন: command, option, argument
- short option (`-l`) আর long option (`--all`)-এর পার্থক্য
- সাহায্য নেওয়ার সব উপায়: `man` (section 1–8 সহ), `--help`, `info`, `tldr`, `apropos`, `whatis`
- `type`, `which`, `help` দিয়ে কমান্ডের পরিচয় বের করা
- built-in কমান্ড বনাম external কমান্ড, এবং ভেতরে `fork()` / `exec()` কীভাবে কাজ করে
- প্রথম কাজের কমান্ড: `echo`, `clear`, `date`, `cal`, `whoami`, `uname`
- টার্মিনাল উইন্ডো ও `man` পেজের কীবোর্ড শর্টকাট

৫.১ কেন টার্মিনাল শিখব?

একটা উপমা দিয়ে শুরু করি। ধরো তুমি একটা রেস্তোরাঁয় গেছ।

- **GUI** হলো ছবিওয়ালা মেনু। ছবিতে আঙুল রাখলেই খাবার আসে। সহজ, কিন্তু মেনুতে যা আছে তার বাইরে কিছু চাইতে পারবে না।
- **টার্মিনাল** হলো সরাসরি বাবুর্চির সাথে কথা বলা: "ঝাল কম, পেঁয়াজ বেশি, ১০টা প্লেট, প্রতিটা আলাদা বক্সে।" প্রথমে বাক্যটা বানানো শিখতে হয়, কিন্তু তারপর যা খুশি অর্ডার করা যায়।

টার্মিনালের সুবিধাগুলো:

সুবিধা	ব্যাখ্যা
গতি	১০০০টা ফাইলের নাম বদলানো GUI-তে ঘণ্টার কাজ, টার্মিনালে এক লাইন
অটোমেশন	কমান্ডগুলো ফাইলে লিখে রাখলেই স্ক্রিপ্ট (অধ্যায় ৪১)
রিমোট কাজ	হাজার মাইল দূরের সার্ভারে SSH দিয়ে ঢুকে কাজ (অধ্যায় ৩৮)
কম রিসোর্স	GUI ছাড়া সার্ভার কম RAM/CPU খায়
পুনরাবৃত্তি	কী করেছ তার লিখিত রেকর্ড থাকে (history), আবার চালানো যায়
সব জায়গায় একই	Ubuntu, Fedora, Debian, এমনকি Android/macOS-এও প্রায় একই কমান্ড

৫.২ চারটা শব্দের জট: Terminal, Console, TTY, Shell

নতুনরা সবচেয়ে বেশি গুলিয়ে ফেলে এই চারটা শব্দ। এরা আসলে চারটা আলাদা স্তর।

৫.২.১ একটু ইতিহাস: আসল "টার্মিনাল"

১৯৬০–৭০-এর দশকে কম্পিউটার ছিল একটা বিশাল ঘর-ভর্তি যন্ত্র। মানুষ তার সাথে কথা বলত **টার্মিনাল** নামের আলাদা যন্ত্র দিয়ে: একটা কীবোর্ড আর একটা প্রিন্টার বা পর্দা, তার দিয়ে কম্পিউটারের সাথে যুক্ত। সবচেয়ে বিখ্যাত ছিল **Teletype** (টেলিটাইপ) মেশিন। সেখান থেকেই নাম **TTY** (TeleTYpewriter)।

আর **console** মানে ছিল কম্পিউটারের গায়ে সরাসরি লাগানো প্রধান টার্মিনাল, যেটা দিয়ে অপারেটর মেশিন নিয়ন্ত্রণ করত।

আজ সেই যন্ত্রগুলো নেই, কিন্তু লিনাক্স (UNIX-এর উত্তরসূরি হিসেবে) এখনো সেই ধারণাগুলো **সফটওয়্যারে** ধরে রেখেছে।

৫.২.২ আধুনিক অর্থ

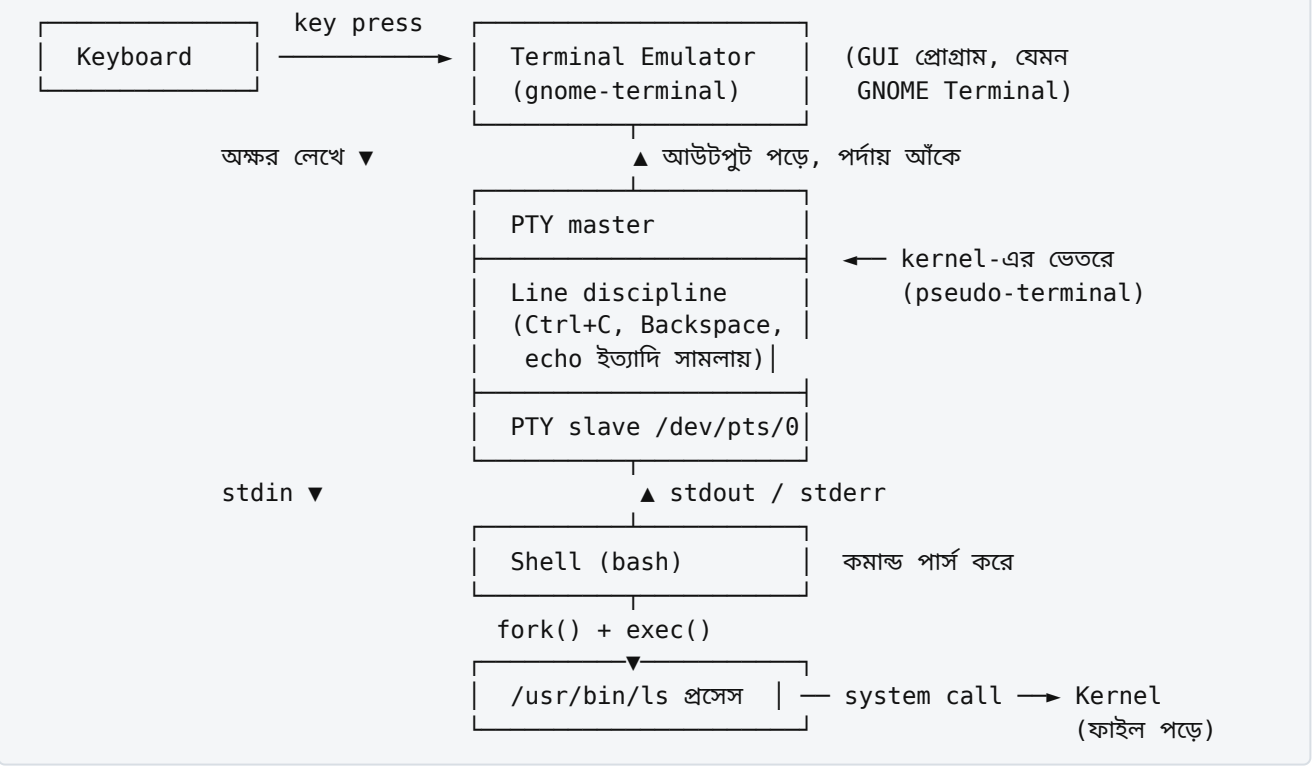
শব্দ	আজকের অর্থ	উদাহরণ
TTY	kernel-এর ভেতরের একটা "টার্মিনাল ডিভাইস"। যেকোনো টেক্সট ইনপুট/আউটপুট চ্যানেল	`/dev/tty1`, `/dev/pts/0`
Console / Virtual Console	GUI ছাড়া, সরাসরি মনিটরে kernel-এর দেওয়া পূর্ণ-পর্দা টেক্সট টার্মিনাল	`Ctrl+Alt+F3` চাপলে যা আসে
Terminal Emulator	GUI-র ভেতরে একটা প্রোগ্রাম (উইন্ডো), যেটা পুরনো টার্মিনাল যন্ত্রের **নকল** (emulate) করে	GNOME Terminal, Konsole, Ptyxis, xterm, Alacritty, kitty
Shell	যে প্রোগ্রাম তোমার লেখা কমান্ড **বোঝে ও চালায়**	`bash`, `zsh`, `fish`, `dash`

> 📝 **মনে রাখো:** Terminal হলো **জানালা** (কোথায় লেখো আর দেখো)। Shell হলো **দোভাষী** (তোমার লেখা বুঝে kernel-কে দিয়ে কাজ করায়)। টার্মিনাল নিজে কোনো কমান্ড বোঝে না, সে শুধু অক্ষর আনা-নেওয়া করে।

উপমা: টার্মিনাল হলো **টেলিফোন সেট**, শেল হলো **ফোনের ওপাশের অপারেটর**। টেলিফোন বদলালে (GNOME Terminal থেকে Konsole) অপারেটর একই থাকতে পারে (bash)। আবার একই ফোনে অন্য অপারেটরকেও ডাকা যায় (bash থেকে zsh)।

৫.২.৩ ভেতরে কীভাবে কাজ করে (Under the hood)

তুমি GNOME Terminal-এ `ls` লিখে Enter চাপলে আসলে যা ঘটে:



ধাপে ধাপে:

1. টার্মিনাল এমুলেটর চালু হওয়ার সময় kernel-এর কাছে একজোড়া **pseudo-terminal (PTY)** চায়। এক মাথা (master) থাকে টার্মিনালের হাতে, অন্য মাথা (slave, যেমন `/dev/pts/0`) দেওয়া হয় shell-কে।
2. তুমি `l`, `s` চাপলে টার্মিনাল অক্ষরগুলো master-এ লেখে। kernel-এর **line discipline** অংশ সেগুলো জমিয়ে রাখে, আর পর্দায় ফেরত দেখায় (echo), যাতে তুমি নিজের লেখা দেখতে পাও। Backspace চাপলে এখানেই অক্ষর মোছে।
3. Enter চাপলে পুরো লাইন `ls\n` shell-এর কাছে যায় (shell-এর stdin)।
4. shell লাইনটা ভেঙে বোঝে, তারপর `fork()` করে নিজের একটা কপি বানায়, আর সেই কপি `exec()` দিয়ে নিজেকে `/usr/bin/ls` প্রোগ্রামে বদলে ফেলে।

5. `ls` kernel-কে system call দিয়ে ডিরেক্টরির তালিকা চায়, তারপর ফলাফল stdout-এ লেখে, যেটা PTY হয়ে টার্মিনালে ফেরে, আর টার্মিনাল সেটা পর্দায় আঁকে।
6. `ls` শেষ হলে shell আবার prompt দেখায়।

`Ctrl+C` চাপলে কী হয়? line discipline সেটাকে অক্ষর হিসেবে পাঠায় না, বরং সামনের প্রসেসকে `SIGINT` **signal** পাঠায়, তাই প্রোগ্রাম থেমে যায়। সিগন্যাল নিয়ে বিস্তারিত অধ্যায় ২৪-এ।

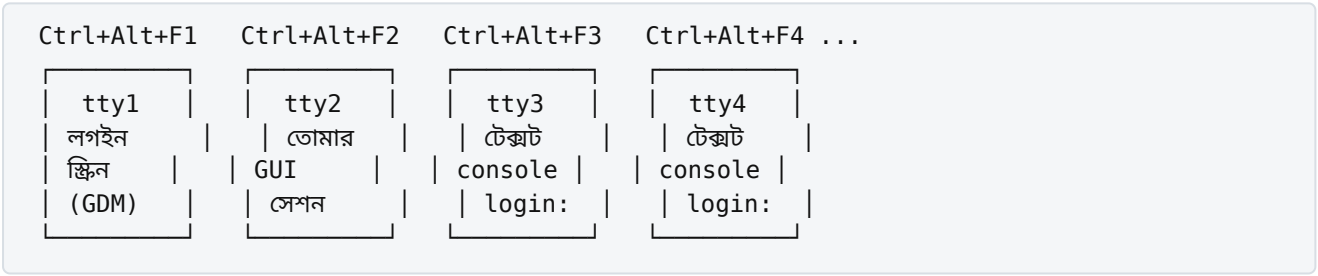
নিজের টার্মিনাল ডিভাইস দেখতে `tty` কমান্ড চালাও:

```
$ tty
/dev/pts/0
```

GUI টার্মিনালে `/dev/pts/N` (pts = pseudo terminal slave), virtual console-এ গেলে `/dev/tty3` এর মতো দেখাবে।

৫.২.৪ Virtual Console: `Ctrl+Alt+F1` – `F6`

লিনাক্স একই সাথে কয়েকটা **virtual console** চালায়, যেন একই মনিটরে কয়েকটা আলাদা টার্মিনাল যন্ত্র লাগানো। `Ctrl+Alt+F1` থেকে `Ctrl+Alt+F6` (কখনো `F12` পর্যন্ত) দিয়ে এদের মধ্যে যাওয়া যায়।



আধুনিক distro-তে বাস্তবে যা দেখবে:

Distro	GUI কোথায় থাকে	টেক্সট console
Ubuntu 24.04 / Fedora / RHEL 9 (GNOME + GDM)	সাধারণত tty1-এ লগইন স্ক্রিন, লগইনের পর তোমার সেশন tty2-এ	`Ctrl+Alt+F3` – `F6`
কিছু পুরনো সিস্টেম / অন্য display manager	প্রায়ই tty7-এ	`Ctrl+Alt+F1` – `F6`
সার্ভার (GUI নেই)	নেই	সব tty টেক্সট

ফিরে আসতে: GUI যে tty-তে আছে সেই নম্বর চাপো (Ubuntu/Fedora-তে সাধারণত `Ctrl+Alt+F2`, না হলে `F1` বা `F7` চেষ্টা করো)। টেক্সট console-এর ভেতর থেকে একটা থেকে আরেকটায় যেতে `Alt+F2`, `Alt+F3` বা `Alt+←` / `Alt+→`-ও কাজ করে।

> 💡 **টিপস:** GUI হ্যাং হয়ে গেলে কম্পিউটার রিস্টার্ট না করে `Ctrl+Alt+F3` চাপো, ইউজারনেম-পাসওয়ার্ড

দিয়ে লগইন করো, তারপর সমস্যার প্রোগ্রামটা বন্ধ করো (অধ্যায় ২৪)। এক্সপার্টদের এটা পুরনো কৌশল।

> ⚠️ **সতর্কতা:** console-এ পাসওয়ার্ড টাইপ করার সময় পর্দায় কিছুই দেখাবে না (`*` চিহ্নও না)। এটা স্বাভাবিক, নিরাপত্তার জন্য। টাইপ করে Enter চাপো।

কমান্ড দিয়েও console বদলানো যায় (root লাগে):

```
$ sudo chvt 3     # tty3-এ চলে যাবে
```

ভার্চুয়াল মেশিনে (VirtualBox ইত্যাদি) `Ctrl+Alt` অনেক সময় host ধরে ফেলে। VirtualBox-এ `Host+F3` (Host key সাধারণত ডান `Ctrl`) ব্যবহার করো।

৫.২.৫ টার্মিনাল এমুলেটর খোলা

পরিবেশ	কীভাবে খুলবে	ডিফল্ট টার্মিনাল
Ubuntu 24.04 (GNOME)	`Ctrl+Alt+T` , অথবা Activities-এ "Terminal" লিখে	GNOME Terminal
Fedora Workstation	Activities → "Terminal"	নতুন ভার্সনে Ptyxis, পুরনোতে GNOME Terminal
KDE Plasma (Kubuntu, Fedora KDE)	Application Launcher → Konsole (অনেক সেটআপে `Ctrl+Alt+T`)	Konsole
যেকোনো ফাইল ম্যানেজার	ফোল্ডারের ভেতরে right-click → "Open in Terminal"	—

টার্মিনাল উইন্ডো (GNOME Terminal ধরে) এর শর্টকাট:

শর্টকাট	কাজ
`Ctrl+Alt+T`	নতুন টার্মিনাল খোলা (Ubuntu)
`Ctrl+Shift+C`	কপি (সাধারণ `Ctrl+C` এখানে প্রোগ্রাম থামায়!)
`Ctrl+Shift+V`	পেস্ট
`Shift+Insert`	পেস্ট (বিকল্প)
মাউস দিয়ে সিলেক্ট + মাঝের বাটন ক্লিক	X11/Wayland-এর "primary selection" পেস্ট
`Ctrl+Shift+T`	নতুন ট্যাব
`Ctrl+Shift+N`	নতুন উইন্ডো
`Ctrl+Shift+W`	বর্তমান ট্যাব বন্ধ
`Ctrl+Shift+Q`	পুরো উইন্ডো বন্ধ
`Ctrl+PageUp` / `Ctrl+PageDown`	আগের/পরের ট্যাব

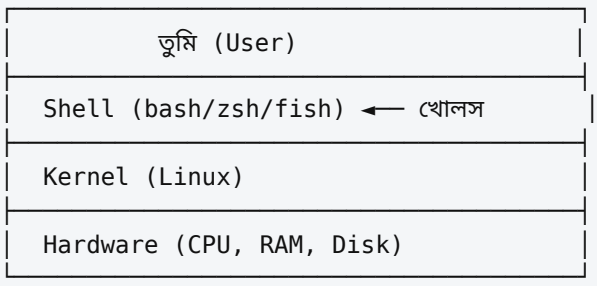
শর্টকাট	কাজ
`Alt+1` … `Alt+9`	নির্দিষ্ট নম্বরের ট্যাবে যাওয়া
`Ctrl+Shift+F`	আউটপুটের মধ্যে খোঁজা
`Ctrl++` / `Ctrl+-` / `Ctrl+0`	লেখা বড় / ছোট / আগের মাপে
`F11`	ফুল স্ক্রিন
`Shift+PageUp` / `Shift+PageDown`	স্ক্রলব্যাক (পুরনো আউটপুট দেখা)

> ⚠️ **সতর্কতা:** টার্মিনালে `Ctrl+C` মানে **কপি নয়**, মানে চলমান প্রোগ্রাম থামানো। কপি করতে `Ctrl+Shift+C`। এই ভুল সবাই প্রথম সপ্তাহে করে। (Konsole-এও একই: `Ctrl+Shift+C` / `Ctrl+Shift+V`।)

শেলের ভেতরের এডিটিং শর্টকাট (`Ctrl+A`, `Ctrl+R`, `Tab` ইত্যাদি) আলাদা করে পুরো একটা অধ্যায়ে আছে: অধ্যায় ৬।

৫.৩ Shell: তোমার দোভাষী

Shell (শেল, মানে "খোলস") হলো একটা সাধারণ প্রোগ্রাম, যেটা kernel-কে চারপাশ থেকে ঘিরে থাকে। তুমি সরাসরি kernel-এর সাথে কথা বলতে পারো না; shell তোমার মানুষের ভাষার মতো কমান্ড নিয়ে kernel-এর জন্য উপযুক্ত system call-এ রূপান্তর করে। (kernel ও তার স্তরগুলো অধ্যায় ২-এ দেখেছ।)



shell-এর কাজ:

- কমান্ড পড়া ও ভাঙা (parsing)
- `*`, `~`, `$HOME` এর মতো বিশেষ চিহ্ন বিস্তার (expansion) করা
- প্রোগ্রাম খুঁজে চালানো
- redirection ও pipe সামলানো (অধ্যায় ১৯)
- variable, history, alias রাখা
- একটা পূর্ণ প্রোগ্রামিং ভাষা হিসেবে কাজ করা (if, loop, function)

৫.৩.১ প্রধান shell-গুলো

Shell	পুরো নাম	বৈশিষ্ট্য	কোথায় ডিফল্ট
`sh`	Bourne shell (১৯৭৯) / POSIX shell	পুরনো ও সবচেয়ে সাধারণ মান। আজ `/bin/sh` সাধারণত অন্য shell-এর দিকে link	Debian/Ubuntu-তে `/bin/sh` → `dash`; Fedora/RHEL-এ `/bin/sh` → `bash`
`dash`	Debian Almquist shell	খুব ছোট ও দ্রুত, শুধু POSIX ফিচার	Debian/Ubuntu-র সিস্টেম স্ক্রিপ্টের `/bin/sh`
`bash`	Bourne Again SHell (GNU)	সবচেয়ে প্রচলিত; history, tab completion, array, স্ক্রিপ্টিং	Ubuntu, Debian, Fedora, RHEL-এ লগইন shell
`zsh`	Z shell	bash-এর মতো, সাথে আরও শক্তিশালী completion, theme (Oh My Zsh)	macOS, Kali Linux
`fish`	Friendly Interactive SHell	লেখার সময়ই সাজেশন, রঙিন, সহজ; কিন্তু POSIX/bash-এর সাথে সিনট্যাক্স মেলে না	কোথাও ডিফল্ট না, আলাদা ইনস্টল করতে হয়
`ksh`, `tcsh`	Korn shell, C shell	পুরনো UNIX পরিবেশে দেখা যায়	কিছু বাণিজ্যিক UNIX

> 💡 **টিপস:** এই বইয়ে সব উদাহরণ **bash** দিয়ে। কারণ প্রায় সব লিনাক্স সার্ভারে bash থাকে, আর ইন্টারভিউতেও bash-ই ধরা হয়। bash শিখলে zsh প্রায় সরাসরি চালাতে পারবে।

৫.৩.২ আমি কোন shell চালাচ্ছি?

```
$ echo $SHELL
/bin/bash
```

`$SHELL` হলো তোমার **লগইন shell**, অর্থাৎ অ্যাকাউন্টের জন্য নির্ধারিত shell। কিন্তু এই মুহূর্তে আসলে কোনটা চলছে দেখতে:

```
$ echo $0
bash
$ ps -p $$
    PID TTY          TIME CMD
   4127 pts/0    00:00:00 bash
```

(`$$` মানে বর্তমান shell-এর process ID।) লগইন shell হলে `echo $0`-এর আউটপুট `-bash` এর মতো সামনে হাইফেনসহ দেখাতে পারে।

সিস্টেমে কোন কোন shell ইনস্টল আছে:

```
$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
```

```
/usr/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/usr/bin/dash
```

`/bin/sh` আসলে কী, দেখো (Ubuntu-তে):

```
$ ls -l /bin/sh
lrwxrwxrwx 1 root root 4 Mar 31  2024 /bin/sh -> dash
```

৫.৩.৩ অন্য shell চেষ্টা করা ও ডিফল্ট বদলানো

```
$ sudo apt install zsh fish      # Ubuntu/Debian
$ sudo dnf install zsh fish      # Fedora/RHEL (RHEL-এ fish EPEL রিপো থেকে)
```

শুধু একবার চালিয়ে দেখতে, নাম লিখলেই নতুন shell শুরু হবে; `exit` দিলে আগেরটায় ফিরে আসবে:

```
$ zsh
nishat@laptop ~ % exit
$ fish
Welcome to fish, the friendly interactive shell
nishat@laptop ~> exit
$
```

স্থায়ীভাবে লগইন shell বদলাতে `chsh` (change shell):

```
$ chsh -s /usr/bin/zsh
Password:
```

পরিবর্তন কাজ করবে পরের লগইন থেকে। (তথ্যটা `/etc/passwd` ফাইলের শেষ কলামে লেখা থাকে, অধ্যায় ১৫ দেখো।)

> ⚠️ **সতর্কতা:** `chsh`-এ এমন path দিও না যেটা `/etc/shells`-এ নেই বা ভুল বানান। তাহলে লগইন করতে পারবে না। সার্ভারে বদলানোর আগে একটা আলাদা সেশন খোলা রেখো।

৫.৪ Prompt বোঝা

টার্মিনাল খুললে যে লেখাটা দেখো, যেটার পরে কার্সর জ্বলছে, সেটাই **prompt**। shell বলছে, "আমি তৈরি, বলো কী করব।"

```
nishat@laptop:~/Documents$
  |      |     |        |
  |      |     |        └── $ = সাধারণ ইউজার   (# হলে root)
  |      |     └─────────── বর্তমান ডিরেক্টরি (~ মানে home)
  |      └───────────────── hostname (কম্পিউটারের নাম)
  └──────────────────────── username (কে লগইন করে আছে)
```

অংশ	মানে
`nishat`	ইউজারের নাম
`@`	"at", শুধু ভাগ করার চিহ্ন
`laptop`	মেশিনের নাম (hostname)
`:`	ভাগ করার চিহ্ন
`~`	তোমার home ডিরেক্টরি, যেমন `/home/nishat`
`~/Documents`	home-এর ভেতরের Documents ফোল্ডার
`$`	সাধারণ (unprivileged) ইউজার
`#`	**root** (সর্বোচ্চ ক্ষমতার অ্যাডমিন)

root হলে prompt এমন দেখায়:

```
root@laptop:~#
```

> ⚠️ **সতর্কতা:** prompt-এর শেষে `#` দেখলে সাবধান! root যেকোনো ফাইল মুছতে পারে, সিস্টেম নষ্ট করতে পারে, কেউ বাধা দেবে না। root নিয়ে বিস্তারিত অধ্যায় ১৮-এ।

> 📝 **মনে রাখো (বইয়ের নিয়ম):** এই বইয়ের উদাহরণে `$` দিয়ে শুরু লাইন মানে তুমি টাইপ করবে, কিন্তু **`$` নিজে টাইপ করবে না**। `#` দিয়ে শুরু হলে সেটা root হিসেবে চালাতে হবে। prompt ছাড়া লাইনগুলো আউটপুট। আর কমান্ডের পরে `# ...` লেখা থাকলে সেটা মন্তব্য (comment); shell `#` এর পরের অংশ উপেক্ষা করে।

৫.৪.১ Prompt কোথা থেকে আসে: `PS1`

prompt-এর চেহারা একটা variable-এ রাখা থাকে, নাম `PS1` (Prompt String 1):

```
$ echo "$PS1"
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\
[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
```

জটিল লাগছে? মূল অংশগুলো: `\u` = user, `\h` = host, `\w` = বর্তমান ডিরেক্টরি, `\$` = সাধারণ ইউজারের জন্য `$`, root-এর জন্য `#`। বাকিগুলো রঙের কোড। সাময়িকভাবে বদলে দেখো:

```
$ PS1='[\t] \u> '
[14:32:10] nishat>
```

নতুন টার্মিনাল খুললেই আগের চেহারা ফিরবে। স্থায়ীভাবে বদলানো ও সাজানো শিখবে অধ্যায় ৪৪-এ।

Fedora/RHEL-এ prompt একটু ভিন্ন দেখায়: `[nishat@fedora ~]$`। অর্থ একই।

৫.৫ কমান্ডের গঠন

প্রায় সব লিনাক্স কমান্ড একটা ব্যাকরণ মেনে চলে:

```
 command   -options   arguments
 ──────┐   ──────┐   ──────┐
       │         │         └── কীসের উপর কাজ (ফাইল, ফোল্ডার, লেখা)
       │         └──────────── কীভাবে কাজ (আচরণ বদলায়)
       └────────────────────── কী কাজ (ক্রিয়া)
```

বাংলা বাক্যের মতো ভাবো: "**দেখাও** (command) **বিস্তারিতভাবে** (option) **Documents ফোল্ডার** (argument)।"

```
$ ls -l /etc/hostname
-rw-r--r-- 1 root root 7 Jan 10 09:15 /etc/hostname
```

- `ls` → command (list করো)
- `-l` → option (long format-এ, বিস্তারিত)
- `/etc/hostname` → argument (এই ফাইলটার)

৫.৫.১ মৌলিক নিয়ম

নিয়ম	উদাহরণ
অংশগুলোর মাঝে **space** লাগবে	`ls -l` ঠিক, `ls-l` ভুল
লিনাক্স **case-sensitive**	`ls` চলবে, `LS` বা `Ls` চলবে না; `File.txt` আর `file.txt` আলাদা ফাইল
option আর argument ঐচ্ছিক হতে পারে	শুধু `ls` দিলে বর্তমান ফোল্ডার দেখায়
একাধিক argument দেওয়া যায়	`ls /etc /tmp`
নামে space থাকলে quote দাও	`ls "My Documents"` অথবা `ls My\ Documents`
এক লাইনে একাধিক কমান্ড `;` দিয়ে	`date; whoami`

৫.৫.২ Short option বনাম Long option

ধরন	চেহারা	উদাহরণ	বৈশিষ্ট্য
Short option	এক হাইফেন + এক অক্ষর	`-l`, `-a`, `-h`	লেখা ছোট; একসাথে জোড়া দেওয়া যায়
Long option	দুই হাইফেন + পুরো শব্দ	`--all`, `--human-readable`	পড়তে সহজ, স্ক্রিপ্টে ভালো

এই চারটা কমান্ড একই কাজ করে:

```
$ ls -l -a -h
$ ls -lah
$ ls -alh
$ ls -l --all --human-readable
```

short option জোড়া লাগানো যায় (`-lah`), long option যায় না।

মান (value) সহ option: কিছু option-এর সাথে একটা মান দিতে হয়।

```
$ head -n 5 /etc/passwd          # short: space দিয়ে মান
$ head -n5 /etc/passwd           # short: গায়ে লাগিয়েও চলে
$ head --lines=5 /etc/passwd     # long: = দিয়ে মান
$ head --lines 5 /etc/passwd     # long: space দিয়েও সাধারণত চলে
```

বিশেষ `--`: দুটো হাইফেন একা বসালে মানে "option শেষ, এরপর যা আসবে সব argument"। ধরো একটা ফাইলের নাম `-f`:

```
$ rm -- -f        # "-f" নামের ফাইল মুছবে, option হিসেবে ধরবে না
```

> 📝 **মনে রাখো:** এই হাইফেন-নিয়ম বেশিরভাগ GNU কমান্ড মানে, কিন্তু সবাই না। পুরনো ধাঁচের কিছু কমান্ডে হাইফেন ছাড়াই option হয় (যেমন `ps aux`, `tar xvf`), আর কিছু কমান্ডে long option এক হাইফেনে (যেমন `find -name`, `java -version`)। সন্দেহ হলে সবসময় `man` দেখো।

৫.৫.৩ প্রথম কমান্ড চালানো ও ভুল হলে

```
$ whoami
nishat
$ Whoami
Command 'Whoami' not found, did you mean:
  command 'whoami' from deb coreutils (9.4-3ubuntu6)
Try: sudo apt install <deb name>
```

Ubuntu এভাবে ভুল ধরিয়ে দেয় (এটা `command-not-found` প্যাকেজের কাজ)। অন্য distro-তে শুধু দেখাবে:

```
bash: Whoami: command not found
```

exit status: প্রতিটা কমান্ড শেষে shell-কে একটা সংখ্যা ফেরত দেয়। `0` মানে সফল, অন্য যেকোনো সংখ্যা মানে কোনো সমস্যা। শেষ কমান্ডের স্ট্যাটাস দেখতে `$?` :

```
$ ls /etc/hostname
/etc/hostname
$ echo $?
0
$ ls /nai-emon-folder
ls: cannot access '/nai-emon-folder': No such file or directory
$ echo $?
2
```

স্ক্রিপ্টিং-এ এটা খুব কাজে লাগবে (অধ্যায় ৪১)।

৫.৬ Built-in বনাম External কমান্ড

তুমি যে "কমান্ড" লেখো সেটা আসলে পাঁচ রকম জিনিসের একটা হতে পারে:

ধরন	কী	উদাহরণ
alias	অন্য কমান্ডের ডাকনাম	`ll` (Ubuntu-তে `ls -alF`)
keyword	shell-এর ভাষার সংরক্ষিত শব্দ	`if`, `for`, `while`, `[[`
function	shell-এ সংজ্ঞায়িত ছোট প্রোগ্রাম	নিজের বানানো ফাংশন
builtin	shell-এর **ভেতরেই** লেখা কমান্ড	`cd`, `echo`, `pwd`, `type`, `exit`, `export`, `history`
file (external)	ডিস্কে থাকা আলাদা প্রোগ্রাম ফাইল	`/usr/bin/ls`, `/usr/bin/date`, `/usr/bin/cal`

৫.৬.১ shell কমান্ড খোঁজে কোন ক্রমে

তুমি `ls` লিখলে bash এই ক্রমে খোঁজে:

```
"ls" লেখা হলো
     │
     ▼
১. alias আছে?        ── হ্যাঁ ──► alias বিস্তার করে আবার শুরু
     │ না
     ▼
২. keyword?           ── হ্যাঁ ──► ভাষার অংশ হিসেবে চালাও
     │ না
     ▼
৩. function?          ── হ্যাঁ ──► function চালাও
     │ না
     ▼
৪. builtin?           ── হ্যাঁ ──► shell নিজেই চালায় (নতুন প্রসেস নেই)
     │ না
     ▼
৫. hash টেবিলে বা $PATH-এর ফোল্ডারগুলোতে ফাইল খোঁজো
     │ পেলে                              │ না পেলে
```

```
      ▼                                ▼
  fork() + exec() দিয়ে চালাও      "command not found"
```

৫.৬.২ `$PATH`: প্রোগ্রাম খোঁজার ঠিকানার তালিকা

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
```

`:` দিয়ে আলাদা করা ফোল্ডারগুলোতে shell বাম থেকে ডানে খোঁজে; যেখানে প্রথম পায় সেটা চালায়। উপমা: ডাকপিয়ন একটা নাম নিয়ে তালিকার প্রথম মহল্লা থেকে খুঁজতে খুঁজতে যায়। (`$PATH` বদলানো অধ্যায় ৪৪-এ।)

> 💡 **টিপস:** একবার কোনো external কমান্ড পেলে bash তার ঠিকানা মনে রাখে (hash টেবিল), যাতে পরের বার আবার খুঁজতে না হয়। `hash` কমান্ডে তালিকা দেখা যায়, `hash -r` দিয়ে মুছে ফেলা যায়।

৫.৬.৩ কেন কিছু কমান্ড builtin হতেই হবে

external প্রোগ্রাম চালানো মানে নতুন **child process**। আর child process তার বাবা-মা (shell)-এর অবস্থা বদলাতে পারে না। ধরো `cd` যদি `/usr/bin/cd` নামে আলাদা প্রোগ্রাম হতো:

```
bash (cwd=/home/nishat)
  └─ fork → cd প্রসেস (cwd বদলে /etc করল) → শেষ হয়ে মরে গেল
bash (cwd এখনো /home/nishat!)   ← কিছুই বদলায়নি
```

তাই `cd`, `export`, `exit`, `source`, `alias` এগুলো shell-এর ভেতরেই থাকতে হয়। আর `echo`, `pwd`, `printf` builtin রাখা হয় **গতির** জন্য, কারণ নতুন প্রসেস বানানোর খরচ বাঁচে।

> 📝 **মনে রাখো:** কিছু কমান্ড দুইভাবেই আছে। যেমন `echo` builtin হিসেবেও আছে, আবার `/usr/bin/echo` ফাইল হিসেবেও। shell builtin-টাকেই আগে বেছে নেয়।

৫.৬.৪ `type`: কমান্ডের পরিচয়পত্র

`type` (নিজেও builtin) বলে দেয় একটা নাম কোন ধরনের:

```
$ type cd
cd is a shell builtin
$ type ls
ls is aliased to `ls --color=auto'
$ type date
date is /usr/bin/date
$ type if
if is a shell keyword
```

```
$ type ll
ll is aliased to `ls -alF'
```

অপশন	কাজ
`-a`	সব রূপ দেখাও (alias, builtin, সব path)
`-t`	শুধু এক শব্দে ধরন: `alias`, `keyword`, `function`, `builtin`, `file`
`-p`	শুধু ফাইলের path (যদি external হয়)
`-P`	alias/builtin উপেক্ষা করে `$PATH`-এ খুঁজে path দেখাও

```
$ type -a echo
echo is a shell builtin
echo is /usr/bin/echo
echo is /bin/echo
$ type -t pwd
builtin
```

(Ubuntu/Fedora-তে `/bin` আসলে `/usr/bin`-এর link, তাই একই ফাইল দুই ঠিকানায় দেখায়; অধ্যায় ৭।)

৫.৬.৫ `which` ও `command -v`

```
$ which date
/usr/bin/date
$ which cd
$                  # কিছু দেখায় না, কারণ cd ফাইল নয়
$ command -v cd
cd
```

`which` একটা external প্রোগ্রাম, শুধু `$PATH`-এ ফাইল খোঁজে; তাই alias ও builtin-এর কথা জানে না। স্ক্রিপ্টে কোনো কমান্ড আছে কিনা যাচাইয়ে POSIX-সম্মত `command -v` ভালো। ফাইল খোঁজার আরও টুল (`whereis`, `locate`) অধ্যায় ১২-এ।

৫.৬.৬ alias বা function এড়িয়ে আসল কমান্ড চালানো

```
$ \ls            # সামনে ব্যাকস্ল্যাশ: alias উপেক্ষা
$ command ls     # alias ও function উপেক্ষা
$ /usr/bin/ls    # পুরো path: সরাসরি সেই ফাইল
```

৫.৭ সাহায্য নেওয়া: এক্সপার্টের আসল অস্ত্র

কোনো লিনাক্স এক্সপার্ট সব কমান্ডের সব অপশন মুখস্থ রাখেন না। তাঁরা জানেন **কোথায় দেখতে হয়**। নিচের টুলগুলো সবচেয়ে সংক্ষিপ্ত থেকে সবচেয়ে বিস্তারিত ক্রমে:

টুল	কতটা বিস্তারিত	কখন ব্যবহার করবে
`whatis`	এক লাইন	"এই কমান্ডটা কী?"
`tldr`	কয়েকটা উদাহরণ	"সাধারণভাবে কীভাবে ব্যবহার করি?"
`--help`	অপশনের তালিকা	"ঐ অপশনটার নাম কী ছিল?"
`help`	builtin-এর সাহায্য	`cd`, `type` এর মতো builtin-এর জন্য
`man`	পূর্ণ সরকারি ম্যানুয়াল	সব খুঁটিনাটি
`info`	বই-ধাঁচের গাইড	GNU টুলের গভীর ব্যাখ্যা
`apropos`	কীওয়ার্ড দিয়ে খোঁজা	"কোন কমান্ড দিয়ে X করা যায়?"

৫.৭.১ `--help`: সবচেয়ে দ্রুত

বেশিরভাগ GNU কমান্ড `--help` পেলে সংক্ষিপ্ত সাহায্য দেখায়:

```
$ date --help
Usage: date [OPTION]... [+FORMAT]
  or:  date [-u|--utc|--universal] [MMDDhhmm[[CC]YY][.ss]]
Display date and time in the given FORMAT.
With -s, or with [MMDDhhmm[[CC]YY][.ss]], set the date and time.

Mandatory arguments to long options are mandatory for short options too.
  -d, --date=STRING          display time described by STRING, not 'now'
...
```

Usage লাইন পড়তে শেখো, এটা সব ডকুমেন্টেশনের সাধারণ ভাষা:

চিহ্ন	মানে
`[ ]`	ঐচ্ছিক, না দিলেও চলবে
`...`	একাধিকবার দেওয়া যায়
`<ফাইল>` বা `FILE` (বড় হাতের)	এখানে তোমার আসল মান বসাবে
`a|b`	a অথবা b, যেকোনো একটা
`{a,b}`	এর মধ্যে একটা বাধ্যতামূলক

আউটপুট লম্বা হলে পাতা পাতা দেখো:

```
$ ls --help | less     # Space দিয়ে সামনে, q দিয়ে বের হও
```

> ⚠️ **সতর্কতা:** সব প্রোগ্রাম `--help` বোঝে না। কিছু `-h` চায়, আর কিছুতে `-h` মানে অন্য কিছু (যেমন `ls -h` মানে human-readable সাইজ)। builtin-এর জন্য `--help` প্রায়ই কাজ করে না; সেখানে `help` ব্যবহার করো।

৫.৭.২ `help` : builtin-এর সাহায্য

```
$ help cd
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.
...
$ help            # সব builtin-এর তালিকা
$ help -d type    # এক লাইনের বর্ণনা
type - Display information about command type.
```

> 💡 **টিপস:** `man cd` চালালে অনেক সময় "No manual entry" বা পুরো bash-এর বিশাল ম্যানুয়াল আসে। builtin হলে আগে `type` দিয়ে নিশ্চিত হও, তারপর `help` ব্যবহার করো।

৫.৭.৩ `man` : সরকারি ম্যানুয়াল

man (manual) হলো লিনাক্সের অন্তর্নির্মিত রেফারেন্স লাইব্রেরি। প্রায় প্রতিটা কমান্ড, কনফিগ ফাইল, system call-এর জন্য একটা "man page" আছে।

```
$ man ls
```

একটা man page সাধারণত এই অংশগুলোয় ভাগ করা:

অংশ	কী থাকে
`NAME`	নাম ও এক লাইনের বর্ণনা
`SYNOPSIS`	ব্যবহারবিধি (usage)
`DESCRIPTION`	বিস্তারিত বিবরণ ও অপশনের তালিকা
`OPTIONS`	(কখনো আলাদা) অপশনগুলো
`EXAMPLES`	উদাহরণ (সব পেজে থাকে না)
`EXIT STATUS`	কোন exit code-এর কী মানে
`FILES`	সংশ্লিষ্ট ফাইল
`ENVIRONMENT`	প্রভাব ফেলে এমন environment variable
`BUGS`	জানা সমস্যা
`AUTHOR`	লেখক
`SEE ALSO`	সম্পর্কিত পেজ, যেমন `dir(1)` , `stat(1)`

man-এর section (অধ্যায়) 1–8

পুরো ম্যানুয়াল কয়েকটা **section**-এ ভাগ করা। একই নামে একাধিক section-এ পেজ থাকতে পারে, তাই এটা জানা জরুরি।

Section	বিষয়	উদাহরণ
1	সাধারণ ইউজারের কমান্ড	`ls(1)`, `passwd(1)`, `date(1)`
2	System call (kernel-এর ফাংশন)	`open(2)`, `fork(2)`, `read(2)`
3	Library function (মূলত C লাইব্রেরি)	`printf(3)`, `malloc(3)`
4	Special file / device	`null(4)` (`/dev/null`), `tty(4)`
5	ফাইল ফরম্যাট ও কনফিগারেশন ফাইল	`passwd(5)` (`/etc/passwd`), `fstab(5)`, `crontab(5)`
6	গেম ও মজার প্রোগ্রাম	`sl(6)` (ইনস্টল থাকলে)
7	বিবিধ: ধারণা, নিয়ম, প্রোটোকল	`signal(7)`, `regex(7)`, `hier(7)`, `man-pages(7)`
8	System administration (সাধারণত root-এর) কমান্ড	`mount(8)`, `useradd(8)`, `fdisk(8)`

(কিছু সিস্টেমে আরও আছে, যেমন section `9` kernel routine, আর `1p`, `3p` এর মতো POSIX সংস্করণ।)

`passwd(5)` লেখার মানে "section 5-এর passwd পেজ"। উদাহরণ:

```
$ man passwd       # প্রথম যেটা পায়: section 1, passwd কমান্ড
$ man 5 passwd     # /etc/passwd ফাইলের ফরম্যাট
$ man 1 printf     # shell-এর printf কমান্ড
$ man 3 printf     # C ভাষার printf ফাংশন
$ man -a passwd    # সব section-এর passwd পেজ একটার পর একটা
```

> 💡 **টিপস:** section-গুলো মনে রাখার কৌশল: **১** ইউজার, **৮** অ্যাডমিন, **৫** কনফিগ ফাইল। শুরুতে এই তিনটাই সবচেয়ে বেশি লাগবে। man-এর নিজের পেজেও তালিকাটা আছে: `man man`।

man-এর ভেতরে চলাফেরা

man পেজ সাধারণত `less` নামের **pager** দিয়ে দেখানো হয়। তাই শর্টকাটগুলো less-এর:

কী	কাজ
`Space` / `f` / `PageDown`	এক পাতা সামনে
`b` / `PageUp`	এক পাতা পেছনে
`↓` / `j` / `Enter`	এক লাইন নিচে
`↑` / `k`	এক লাইন উপরে
`d` / `u`	আধা পাতা নিচে / উপরে
`g` / `<`	একদম শুরুতে
`G` / `>`	একদম শেষে

কী	কাজ
`/শব্দ` তারপর `Enter`	নিচের দিকে খোঁজো
`?শব্দ` তারপর `Enter`	উপরের দিকে খোঁজো
`n`	পরের মিল
`N`	আগের মিল
`&শব্দ`	শুধু মিল থাকা লাইন দেখাও
`h`	less-এর সব শর্টকাটের সাহায্য
`q`	বের হয়ে আসো

> 💡 **টিপস:** কোনো অপশন খুঁজতে `/` দিয়ে খোঁজার সময় সামনে space দিয়ে হাইফেন লিখো, যেমন `/ -r` (space-হাইফেন-r)। এতে শব্দের মাঝের "r" না ধরে অপশনের লাইনটাই পাবে।

man কাজ করে কীভাবে (Under the hood)

```
$ man ls
  │
  ├─► MANPATH / /etc/manpath.config (Debian-ধাঁচে) দেখে ঠিক করে কোথায় খুঁজবে
  │       সাধারণত /usr/share/man/man1/ls.1.gz
  ├─► .gz ফাইলটা খুলে (decompress)
  ├─► groff দিয়ে roff মার্কআপকে টার্মিনালের লেখায় সাজায় (format)
  └─► $MANPAGER বা $PAGER (ডিফল্ট less) দিয়ে দেখায়
```

```
$ man -w ls          # পেজের ফাইল কোথায়
/usr/share/man/man1/ls.1.gz
```

man-এর আরও কিছু অপশন:

অপশন	কাজ
`man N name`	নির্দিষ্ট section N
`-a`	সব section-এর মিল দেখাও
`-f name`	`whatis`-এর সমান
`-k keyword`	`apropos`-এর সমান
`-w`	পেজ ফাইলের path
`-K text`	সব পেজের পুরো লেখার ভেতরে খোঁজো (ধীর)

> ⚠️ **সতর্কতা:** Ubuntu Server বা Docker-এর মতো **minimized** ইমেজে জায়গা বাঁচাতে man পেজ বাদ দেওয়া থাকে। তখন `man ls` কোনো পেজের বদলে একটা বার্তা দেখায়। Ubuntu-তে `sudo unminimize`

> চালালে (ও `man-db` প্যাকেজ থাকলে) পেজ ফিরে আসে। Fedora/RHEL-এর container ইমেজেও ডকুমেন্টেশন বাদ থাকতে পারে, তখন `man-db` ও `man-pages` প্যাকেজ ইনস্টল করো।

৫.৭.৪ `whatis` : এক লাইনে পরিচয়

```
$ whatis ls
ls (1)               - list directory contents
$ whatis passwd
passwd (1ssl)        - OpenSSL application commands
passwd (1)           - change user password
passwd (5)           - the password file
```

একসাথে দেখিয়ে দিল `passwd` নামে কোন কোন section-এ পেজ আছে। `man -f passwd` একই ফল দেয়।

৫.৭.৫ `apropos` : কাজ জানো, কমান্ডের নাম জানো না

ধরো তুমি জানতে চাও, ডিস্কের খালি জায়গা দেখার কমান্ড কী? `apropos` সব man পেজের NAME অংশে কীওয়ার্ড খোঁজে:

```
$ apropos "disk space"
df (1)               - report file system space usage
$ apropos -s 8 partition
fdisk (8)            - manipulate disk partition table
parted (8)           - a partition manipulation program
...
```

অপশন	কাজ
`-s N`	শুধু section N-এ খোঁজো
`-a`	সব কীওয়ার্ড মিলতে হবে (AND)
`-e`	হুবহু শব্দ মিলতে হবে
`-r` / `-w`	regex / wildcard দিয়ে খোঁজো

`whatis` / `apropos` কাজ না করলে: `mandb`

```
$ apropos copy
copy: nothing appropriate.
```

এর কারণ: `whatis` ও `apropos` প্রতিবার সব পেজ পড়ে না, বরং একটা **ইনডেক্স ডাটাবেস** খোঁজে। নতুন ইনস্টল করা সিস্টেমে ডাটাবেস এখনো তৈরি না-ও হতে পারে। একটা systemd timer/cron সাধারণত এটা নিয়মিত আপডেট করে; চাইলে নিজেই চালাও:

```
$ sudo mandb
Processing manual pages under /usr/share/man...
```

```
...
```

৫.৭.৬ `info`: GNU-র বই-ধাঁচের ডকুমেন্টেশন

GNU প্রজেক্টের অনেক টুল (coreutils, bash, grep, sed) তাদের **পূর্ণ** ডকুমেন্টেশন রাখে Texinfo/`info` ফরম্যাটে। man পেজ সেখানে প্রায়ই সংক্ষিপ্ত সারাংশ মাত্র; প্রায়ই শেষে লেখা থাকে "The full documentation … is maintained as a Texinfo manual"।

```
$ info coreutils        # সব মূল কমান্ডের সম্পূর্ণ বই
$ info date             # date-এর অংশে সরাসরি যাবে
$ info                  # সব info বইয়ের সূচি (directory)
```

info-র পেজগুলো **node** নামে ভাগ করা, আর node-গুলো লিংক দিয়ে জোড়া, অনেকটা ওয়েবসাইটের মতো।

কী	কাজ
`Space` / `Backspace`	পাতা সামনে / পেছনে (node পেরিয়েও)
`n`	পরের node
`p`	আগের node
`u`	এক ধাপ উপরের node (parent)
`Tab`	পরের লিংকে যাও
`Enter`	লিংকে ঢোকো
`l`	আগের দেখা node-এ ফেরো (ব্রাউজারের Back)
`t`	বইয়ের শুরুর node (Top)
`d`	সব info বইয়ের সূচি
`s` বা `/`	খোঁজো
`H`	সাহায্য
`q`	বের হও

> 📝 **মনে রাখো:** কোনো কমান্ডের info পেজ না থাকলে `info` সাধারণত তার man পেজটাই দেখায়। বাস্তবে মানুষ বেশি ব্যবহার করে `man` ও `--help`; info লাগে গভীর ব্যাখ্যা চাইলে।

৫.৭.৭ `tldr`: উদাহরণে ভরা সংক্ষিপ্ত সাহায্য

man পেজ বিস্তারিত, কিন্তু নতুনদের জন্য লম্বা। **tldr** ("Too Long; Didn't Read") একটা কমিউনিটি-চালিত প্রজেক্ট, যেখানে প্রতিটা কমান্ডের সবচেয়ে বেশি ব্যবহৃত ৫–৮টা উদাহরণ থাকে। এটা ডিফল্টে ইনস্টল থাকে না।

```
$ sudo apt install tldr       # Ubuntu/Debian
$ sudo dnf install tldr       # Fedora
```

```
$ tldr --update                 # প্রথমবার পেজগুলো ডাউনলোড করো (ইন্টারনেট লাগে)
```

> 📝 **মনে রাখো:** tldr-এর কয়েকটা আলাদা client আছে (Python, Rust-এ লেখা `tealdeer` ইত্যাদি), distro ভেদে প্যাকেজ আলাদা হতে পারে, আর আপডেটের অপশনও একটু ভিন্ন হতে পারে (কোনোটায় `tldr -u`)। কোনটা ইনস্টল হলো `tldr --help` দিয়ে দেখে নাও। `pip` / `npm` দিয়েও ইনস্টল করা যায়।

```
$ tldr tar

  tar

  Archiving utility.
  Often combined with a compression method, such as gzip or bzip2.

  - Create an archive and write it to a file:
    tar cf path/to/target.tar path/to/file1 path/to/file2 ...

  - Extract a (compressed) archive file into the current directory verbosely:
    tar xvf path/to/source.tar[.gz|.bz2|.xz]
...
```

(আউটপুটের লেখা tldr-এর পেজ আপডেটের সাথে বদলায়।)

> 💡 **টিপস:** শেখার সেরা কৌশল: প্রথমে `tldr` দিয়ে সাধারণ ব্যবহার দেখো, কোনো অপশন বুঝতে না পারলে `man` খুলে `/` দিয়ে সেই অপশন খোঁজো।

৫.৭.৮ অনলাইন ও অন্যান্য উৎস

- `/usr/share/doc/<প্যাকেজ>/` : প্যাকেজের README, changelog, উদাহরণ কনফিগ
- distro-র অফিসিয়াল ডকুমেন্টেশন: Ubuntu Server docs, Fedora Docs, Red Hat Documentation, ArchWiki (অন্য distro-র জন্যও খুব কাজের)
- কোনো এরর বার্তা হুবহু কপি করে সার্চ করা

> ⚠️ **সতর্কতা:** ইন্টারনেট থেকে পাওয়া কমান্ড, বিশেষ করে `sudo`, `rm -rf`, `curl ... | bash` থাকলে, না বুঝে কখনো চালাবে না। আগে `man` দিয়ে প্রতিটা অংশ বুঝে নাও।

৫.৭.৯ "কোথায় সাহায্য পাব" সিদ্ধান্ত-চিত্র

```
কমান্ডের নাম জানো?
  ├─ না ──► apropos <কাজের শব্দ>   (যেমন: apropos compress)
  └─ হ্যাঁ
       │
       type <নাম> চালাও
       ├─ builtin/keyword ──► help <নাম>
```

```
        └─ file (external)
             ├─ দ্রুত উদাহরণ চাই ──► tldr <নাম>
             ├─ অপশনের নাম ভুলে গেছি ──► <নাম> --help
             └─ সব খুঁটিনাটি ──► man <নাম>  (আরও গভীরে: info <নাম>)
```

৫.৮ প্রথম কাজের কমান্ডগুলো

এবার কয়েকটা সরল কিন্তু রোজ লাগে এমন কমান্ড শিখি।

৫.৮.১ `echo` : লেখা প্রিন্ট করা

কী করে: যা দাও তা stdout-এ (পর্দায়) লিখে দেয়। bash-এ এটা builtin।

Syntax: `echo [OPTION]... [STRING]...`

অপশন	কাজ
`-n`	শেষে নতুন লাইন (newline) দেবে না
`-e`	`\n`, `\t` এর মতো escape sequence চালু করো
`-E`	escape sequence বন্ধ (bash-এ এটাই ডিফল্ট)

`-e` দিলে ব্যবহারযোগ্য escape:

Escape	মানে
`\n`	নতুন লাইন
`\t`	ট্যাব
`\\`	ব্যাকস্ল্যাশ নিজেই
`\a`	বেল (বিপ শব্দ, টার্মিনাল সমর্থন করলে)
`\e` বা `\033`	escape অক্ষর (রঙের কোডে লাগে)
`\c`	এর পরের আউটপুট আর দেখাবে না

উদাহরণ:

```
$ echo Hello Linux
Hello Linux
$ echo "আমি লিনাক্স শিখছি"
আমি লিনাক্স শিখছি
$ echo -e "নাম:\tনিশাত\nশহর:\tঢাকা"
নাম:    নিশাত
শহর:    ঢাকা
$ echo -n "লাইন শেষ হয়নি"; echo " ...এখন হলো"
লাইন শেষ হয়নি ...এখন হলো
$ echo "আমার home হলো $HOME"
আমার home হলো /home/nishat
```

```
$ echo 'আমার home হলো $HOME'
আমার home হলো $HOME
$ echo -e "\e[1;32mসবুজ মোটা লেখা\e[0m"
সবুজ মোটা লেখা       (পর্দায় সবুজ রঙে দেখাবে)
```

ডাবল কোট বনাম সিঙ্গেল কোট: `"..."`-এর ভেতরে `$HOME` এর মতো variable-এর মান বসে যায়; `'...'`-এর ভেতরে সব কিছু হুবহু থাকে। quoting-এর পূর্ণ নিয়ম অধ্যায় ৪১-এ।

```
$ echo      ফাঁকা      অনেক
ফাঁকা অনেক
$ echo "     ফাঁকা      অনেক"
     ফাঁকা      অনেক
```

quote ছাড়া shell শব্দগুলো আলাদা argument হিসেবে ভাঙে, আর `echo` সেগুলো একটা করে space দিয়ে জোড়ে। তাই অতিরিক্ত space হারিয়ে যায়।

> 💡 **টিপস:** `echo`-র আচরণ shell ও সিস্টেমভেদে একটু আলাদা (যেমন `dash`-এর `echo` `-e` ছাড়াই escape বোঝে)। স্ক্রিপ্টে নির্ভরযোগ্য ফরম্যাটেড আউটপুটের জন্য `printf` ভালো: `printf "নাম: %s\n" "নিশাত"`।

৫.৮.২ `clear`: পর্দা পরিষ্কার করা

```
$ clear
```

পর্দা মুছে prompt উপরে চলে যায়। শর্টকাট: `Ctrl+L` (একই কাজ, তবে চলমান লেখা নষ্ট করে না)।

কমান্ড/শর্টকাট	কাজ
`clear`	পর্দা ও (বেশিরভাগ টার্মিনালে) scrollback মুছে ফেলে
`clear -x`	শুধু দৃশ্যমান পর্দা মোছে, scrollback রাখে
`Ctrl+L`	পর্দা পরিষ্কার; টাইপ করা অর্ধেক কমান্ড থেকে যায়
`reset`	টার্মিনাল পুরো রিসেট (নিচে দেখো)

> 💡 **টিপস:** ভুল করে কোনো বাইনারি ফাইল `cat` করলে টার্মিনালে আবোলতাবোল চিহ্ন আসে, লেখা অদ্ভুত হয়ে যায়। তখন অন্ধভাবে `reset` টাইপ করে Enter চাপো, ঠিক হয়ে যাবে।

> 📝 **মনে রাখো:** `clear` শুধু পর্দা মোছে; কমান্ড history মোছে না। history নিয়ে অধ্যায় ৬।

৫.৮.৩ `date` : তারিখ ও সময়

কী করে: সিস্টেমের বর্তমান তারিখ-সময় দেখায়, যেকোনো ফরম্যাটে সাজায়, বা অন্য তারিখ হিসাব করে।

Syntax: `date [OPTION]... [+FORMAT]`

```
$ date
Thu Nov 12 14:35:07 +06 2026
```

(locale ও timezone অনুযায়ী ফরম্যাট আলাদা দেখাতে পারে।)

অপশন	কাজ
`+FORMAT`	নিজের ফরম্যাটে দেখাও
`-u` , `--utc`	UTC সময় দেখাও
`-d STRING` , `--date=STRING`	"এখন" না, অন্য সময় দেখাও
`-I` , `--iso-8601[=FMT]`	ISO 8601 ফরম্যাট (`date` , `hours` , `minutes` , `seconds` , `ns`)
`-R` , `--rfc-email`	ইমেইলের মানক ফরম্যাট
`-r FILE`	ফাইলটা শেষ কবে পরিবর্তন হয়েছিল
`-s STRING`	সিস্টেমের সময় সেট করো (root লাগে)

ফরম্যাট কোড:

কোড	মানে	উদাহরণ
`%Y`	চার অঙ্কের বছর	`2026`
`%y`	দুই অঙ্কের বছর	`26`
`%m`	মাস (01–12)	`11`
`%B` / `%b`	মাসের পুরো / ছোট নাম	`November` / `Nov`
`%d`	দিন (01–31)	`12`
`%A` / `%a`	বারের পুরো / ছোট নাম	`Thursday` / `Thu`
`%H`	ঘণ্টা (00–23)	`14`
`%I`	ঘণ্টা (01–12)	`02`
`%p`	AM / PM	`PM`
`%M`	মিনিট	`35`
`%S`	সেকেন্ড	`07`
`%Z`	timezone-এর নাম	`+06` বা `UTC`
`%z`	UTC থেকে পার্থক্য	`+0600`
`%s`	১৯৭০-০১-০১ থেকে মোট সেকেন্ড (Unix epoch)	`1794472507`

কোড	মানে	উদাহরণ
`%j`	বছরের কততম দিন	`316`
`%F`	`%Y-%m-%d` এর সংক্ষেপ	`2026-11-12`
`%T`	`%H:%M:%S` এর সংক্ষেপ	`14:35:07`
`%%`	`%` চিহ্ন নিজে	`%`

উদাহরণ:

```
$ date +%F
2026-11-12
$ date "+আজ %A, %d %B %Y, সময় %I:%M %p"
আজ Thursday, 12 November 2026, সময় 02:35 PM
$ date -u
Thu Nov 12 08:35:07 UTC 2026
$ date -d tomorrow +%F
2026-11-13
$ date -d "next friday" +%F
2026-11-13
$ date -d "2026-12-16 + 10 days" +%F
2026-12-26
$ date -d @0
Thu Jan  1 06:00:00 +06 1970
$ date -I
2026-11-12
$ date -Iseconds
2026-11-12T14:35:07+06:00
$ date +%s
1794472507
```

(`-d` দিয়ে "tomorrow", "yesterday", "2 weeks ago", "next monday" এর মতো ইংরেজি বাক্য বোঝা GNU `date`-এর বিশেষত্ব; macOS/BSD-এর `date`-এ এটা নেই।)

ফাইলের নামে তারিখ বসানো, ব্যাকআপে খুব কাজের:

```
$ echo "backup-$(date +%F_%H%M).tar.gz"
backup-2026-11-12_1435.tar.gz
```

এখানে `$(...)` মানে **command substitution**: ভেতরের কমান্ডের আউটপুট ঐ জায়গায় বসে যায়।

> 💡 **টিপস:** সিস্টেমের সময়, timezone, NTP sync দেখতে ও বদলাতে আধুনিক টুল হলো `timedatectl`:
>
> ```
> $ timedatectl
> $ sudo timedatectl set-timezone Asia/Dhaka
> ```
>
> `date -s` দিয়ে সময় বদলালে NTP চালু থাকলে আবার ঠিক করে দেবে। systemd-এর বিস্তারিত অধ্যায় ২৬-এ।

৫.৮.৪ `cal`: ক্যালেন্ডার

কী করে: টার্মিনালে ক্যালেন্ডার দেখায়।

```
$ cal
   November 2026
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
```

(আজকের তারিখ সাধারণত উল্টো রঙে হাইলাইট করা থাকে।)

> ⚠️ **সতর্কতা (distro পার্থক্য):** Fedora/RHEL-এ `cal` আসে `util-linux` প্যাকেজ থেকে, ডিফল্টে থাকে। Ubuntu/Debian-এ এটা `ncal` প্যাকেজে (BSD সংস্করণ), যেটা অনেক সময় ডিফল্টে ইনস্টল থাকে না:
>
> ```
> $ sudo apt install ncal
> ```
>
> দুই সংস্করণের কিছু অপশনের মানে আলাদা। যেমন util-linux-এর `cal -m` মানে "সপ্তাহ সোমবার থেকে", কিন্তু ncal-এর `cal -m 5` মানে "মে মাস"। নিচের অপশনগুলো দুটোতেই কাজ করে।

ব্যবহার / অপশন	কাজ
`cal`	চলতি মাস
`cal 2027`	পুরো বছরের ক্যালেন্ডার
`cal 12 2026`	নির্দিষ্ট মাস ও বছর (ডিসেম্বর ২০২৬)
`cal -3`	আগের, চলতি ও পরের মাস
`cal -y`	চলতি বছর পুরোটা
`cal -j`	Julian day: বছরের কততম দিন সেই সংখ্যায় দেখাও

```
$ cal 2 2028
   February 2028
Su Mo Tu We Th Fr Sa
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29
```

> ⚠️ **সতর্কতা:** `cal 26` মানে ২০২৬ সাল **না**, মানে খ্রিস্টাব্দ ২৬ সাল! সবসময় চার অঙ্কে বছর লেখো।

> 💡 **টিপস (মজার তথ্য):** `cal 9 1752` চালিয়ে দেখো, ৩ থেকে ১৩ সেপ্টেম্বর নেই! ঐ মাসে ব্রিটেন ও তার উপনিবেশগুলো Julian থেকে Gregorian ক্যালেন্ডারে গিয়েছিল, `cal` সেটা মনে রাখে।

৫.৮.৫ `whoami`: আমি কে?

```
$ whoami
nishat
$ sudo whoami
root
```

কাজ: বর্তমানে কার **effective user ID** দিয়ে কমান্ড চলছে, তার নাম দেখায়। `sudo` দিয়ে চালালে root দেখায়, কারণ তখন কমান্ডটা root হিসেবে চলে। এটা বিশেষ করে অনেক সার্ভারে `su` বা `sudo -i` করার পর কে হয়ে আছ নিশ্চিত হতে কাজে লাগে।

আরও বিস্তারিত পরিচয় দেখতে `id` (বিস্তারিত অধ্যায় ১৫-এ):

```
$ id
uid=1000(nishat) gid=1000(nishat)
groups=1000(nishat),4(adm),24(cdrom),27(sudo),46(plugdev)
```

সম্পর্কিত কমান্ড:

কমান্ড	কাজ
`whoami`	বর্তমান effective ইউজারের নাম
`id`	UID, GID ও সব গ্রুপ
`who`	কে কে লগইন করে আছে, কোন tty-তে
`w`	কে লগইন, আর তারা কী করছে, সাথে uptime
`logname`	মূল লগইন নাম (sudo-র পরেও আসল ইউজার; কিছু GUI টার্মিনালে কাজ না-ও করতে পারে)

```
$ who
nishat   seat0        2026-11-12 09:02 (login screen)
nishat   tty2         2026-11-12 09:02 (tty2)
```

(GDM থাকলে এমন দেখায়; সেটআপ ভেদে আউটপুট আলাদা।)

৫.৮.৬ `uname`: সিস্টেম ও kernel-এর তথ্য

কী করে: চলমান kernel ও মেশিনের তথ্য দেখায় (uname = Unix name)। ভেতরে এটা `uname()` system call দিয়ে সরাসরি kernel থেকে তথ্য নেয়।

```
$ uname
Linux
```

```
$ uname -a
Linux laptop 6.8.0-48-generic #48-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep 27 14:04:52 UTC
2024 x86_64 x86_64 x86_64 GNU/Linux
```

অপশন	Long option	কী দেখায়	উদাহরণ
`-s`	`--kernel-name`	kernel-এর নাম (অপশন না দিলেও এটাই)	`Linux`
`-n`	`--nodename`	নেটওয়ার্কে মেশিনের নাম (hostname)	`laptop`
`-r`	`--kernel-release`	kernel ভার্সন	`6.8.0-48-generic`
`-v`	`--kernel-version`	kernel বিল্ডের তারিখ/তথ্য	`#48-Ubuntu SMP ...`
`-m`	`--machine`	হার্ডওয়্যার আর্কিটেকচার	`x86_64`, `aarch64`
`-p`	`--processor`	প্রসেসর টাইপ (প্রায়ই `unknown`)	`x86_64`
`-i`	`--hardware-platform`	প্ল্যাটফর্ম (প্রায়ই `unknown`)	`x86_64`
`-o`	`--operating-system`	অপারেটিং সিস্টেম	`GNU/Linux`
`-a`	`--all`	সব একসাথে	উপরের আউটপুট

সবচেয়ে বেশি লাগে `uname -r` (ড্রাইভার/মডিউল ইনস্টলের সময় kernel ভার্সন জানতে) আর `uname -m` (কোন আর্কিটেকচারের প্যাকেজ নামাবে):

```
$ uname -r
6.8.0-48-generic
$ uname -m
x86_64
```

kernel ভার্সন সংখ্যার মানে:

```
  6 . 8 . 0 - 48 - generic
  │   │   │   │     └── distro-র kernel flavor (generic, lowlatency ...)
  │   │   │   └──────── distro-র নিজস্ব বিল্ড/প্যাচ নম্বর
  │   │   └──────────── patch level
  │   └──────────────── minor version
  └──────────────────── major version
```

(Fedora-তে দেখতে এমন: `6.11.4-301.fc41.x86_64`; RHEL 9-এ `5.14.0-...el9...x86_64`।)

> 📝 **মনে রাখো:** `uname` বলে **kernel**-এর কথা, distro-র কথা বলে না। "আমি Ubuntu-র কোন ভার্সন চালাচ্ছি?" জানতে:
>
> ```
> $ cat /etc/os-release
> PRETTY_NAME="Ubuntu 24.04.1 LTS"
> NAME="Ubuntu"
> VERSION_ID="24.04"
> ...
> ```

```
$ hostnamectl          # hostname, OS, kernel, architecture সব একসাথে
$ lsb_release -a       # Debian/Ubuntu-তে সাধারণত থাকে
```

kernel, `/proc`, `/sys` নিয়ে গভীর আলোচনা অধ্যায় ৪৭-এ।

৫.৮.৭ শেল থেকে বের হওয়া

উপায়	কাজ
`exit`	বর্তমান shell বন্ধ (টার্মিনাল উইন্ডোও বন্ধ হতে পারে)
`logout`	লগইন shell থেকে বের হওয়া (console/SSH)
`Ctrl+D`	ফাঁকা লাইনে চাপলে "ইনপুট শেষ" (EOF) সংকেত, shell বন্ধ হয়

৫.৯ কয়েকটা সাধারণ ভুল ও সমাধান

সমস্যা	কারণ	সমাধান
`command not found`	বানান ভুল, বড়-ছোট হাতের অক্ষর, বা প্রোগ্রাম ইনস্টল নেই	বানান দেখো; `apropos` দিয়ে খোঁজো; প্যাকেজ ইনস্টল করো (অধ্যায় ৩০)
prompt-এর বদলে `>` দেখাচ্ছে	একটা quote (`"` বা `'`) খোলা রয়ে গেছে, shell বাকি অংশের অপেক্ষায়	quote বন্ধ করো, অথবা `Ctrl+C` চেপে বাতিল
কমান্ড চলছে তো চলছেই, prompt ফিরছে না	প্রোগ্রাম ইনপুটের অপেক্ষায় (যেমন শুধু `cat` লিখেছ)	`Ctrl+C` বা `Ctrl+D`
`Ctrl+C` দিয়ে কপি হচ্ছে না	টার্মিনালে `Ctrl+C` = থামাও	`Ctrl+Shift+C`
টার্মিনাল জমে গেছে, কিছু টাইপ হচ্ছে না	ভুলে `Ctrl+S` চেপেছ (আউটপুট থামিয়ে দেয়)	`Ctrl+Q` চাপো
পর্দায় আবোলতাবোল চিহ্ন	বাইনারি ফাইল প্রিন্ট হয়েছে	`reset`
`Permission denied`	এই কাজে তোমার অনুমতি নেই	পারমিশন বোঝো (অধ্যায় ১৬), প্রয়োজনে `sudo` (অধ্যায় ১৮)
`man`-এ আটকে গেছি	less pager খোলা	`q`

হাতে-কলমে অনুশীলন

ল্যাব ১: টার্মিনাল ও console চেনা

1. `Ctrl+Alt+T` (Ubuntu) বা Activities → Terminal দিয়ে টার্মিনাল খোলো।
2. `tty` চালাও। আউটপুট লেখো (যেমন `/dev/pts/0`)।

3. `Ctrl+Shift+T` দিয়ে নতুন ট্যাব খুলে আবার `tty` চালাও। সংখ্যা বেড়েছে? কেন? (প্রতিটা ট্যাব আলাদা PTY।)
4. `Ctrl+Alt+F3` চাপো। ইউজারনেম ও পাসওয়ার্ড দিয়ে লগইন করো।
5. এখানে `tty` চালাও, এবার `/dev/tty3` দেখাবে।
6. `who` চালাও। তুমি দুই জায়গায় লগইন দেখাচ্ছে?
7. `exit` দিয়ে লগআউট করো, তারপর `Ctrl+Alt+F2` (বা `F1` / `F7`) চেপে GUI-তে ফেরো।

ল্যাব ২: shell চেনা

```
$ echo $SHELL
$ echo $0
$ ps -p $$
$ cat /etc/shells
$ ls -l /bin/sh
```

প্রশ্ন: তোমার `/bin/sh` কোন shell-এর দিকে নির্দেশ করে? (Ubuntu/Debian হলে `dash`, Fedora/RHEL হলে `bash`।)

ঐচ্ছিক: `zsh` ইনস্টল করে চালাও, prompt কেমন বদলায় দেখো, তারপর `exit` দাও। (প্রথমবার zsh একটা কনফিগারেশন মেনু দেখাতে পারে; `q` চাপলে বেরিয়ে যাবে।)

ল্যাব ৩: prompt নিয়ে খেলা

```
$ echo "$PS1"
$ PS1='\u আছে \w-তে \$ '
$ cd /etc
$ PS1='[\A] \h:\W\$ '
```

(`\A` = ২৪ ঘণ্টার HH:MM, `\W` = শুধু বর্তমান ফোল্ডারের নাম।) টার্মিনাল বন্ধ করে নতুন খোলো; আগের prompt ফিরেছে কিনা দেখো।

ল্যাব ৪: কমান্ডের গঠন ও অপশন

```
$ ls
$ ls -l
$ ls -a
$ ls -la
$ ls --all -l --human-readable
$ ls -l /etc/hostname /etc/os-release
$ LS
$ echo $?
```

শেষ দুই লাইনে exit status কত এলো? (`command not found` হলে সাধারণত `127`।)

ল্যাব ৫: কমান্ডের পরিচয়

নিচের প্রতিটার জন্য `type -a` চালাও এবং খাতায় ধরন লেখো (alias / builtin / keyword / file):

```
$ type -a cd
$ type -a echo
$ type -a ls
$ type -a for
$ type -a date
$ type -a pwd
```

তারপর `which cd` আর `command -v cd` চালিয়ে পার্থক্যটা দেখো।

ল্যাব ৬: সাহায্যের গোয়েন্দা

1. `whatis ls cal uname` চালাও।
2. `apropos "user"` চালিয়ে দেখো কতগুলো ফল আসে। এবার `apropos -s 8 user` চালাও; কমলো?
3. `man 5 passwd` খোলো। `/` দিয়ে `shell` শব্দটা খোঁজো, `n` চেপে পরের মিলে যাও, `G` দিয়ে শেষে, `g` দিয়ে শুরুতে যাও, `q` দিয়ে বের হও।
4. `man man` খুলে section-গুলোর তালিকা খুঁজে বের করো।
5. `help cd` আর `cd --help` দুটোই চালাও। পার্থক্য কী?
6. `info coreutils` খোলো। `Tab` দিয়ে একটা লিংকে গিয়ে `Enter`, তারপর `l` দিয়ে ফেরো, `q` দিয়ে বের হও।
7. (ইন্টারনেট থাকলে) `tldr` ইনস্টল ও আপডেট করে `tldr date` চালাও।
8. `apropos` কিছু না দেখালে `sudo mandb` চালিয়ে আবার চেষ্টা করো।

ল্যাব ৭: প্রথম কাজের কমান্ড

```
$ whoami
$ id
$ uname -a
$ uname -r
$ uname -m
$ cat /etc/os-release
$ date
$ date "+%d/%m/%Y %H:%M"
$ date -d "30 days" +%F
$ date -d "2000-01-01" +%A
$ cal
$ cal -3
$ cal 12 2026
$ echo -e "আমি:\t$(whoami)\nkernel:\t$(uname -r)\nআজ:\t$(date +%F)"
$ clear
```

চ্যালেঞ্জ: এক লাইনের একটা কমান্ড লেখো যেটা প্রিন্ট করবে: `হ্যালো <তোমার নাম>! আজ <বার>, তুমি <hostname> মেশিনে Linux <kernel ভার্সন> চালাচ্ছ।`

▼ সম্ভাব্য সমাধান

```
$ echo "হ্যালো $(whoami)! আজ $(date +%A), তুমি $(uname -n) মেশিনে Linux $(uname -r) চালাচ্ছ।"
হ্যালো nishat! আজ Thursday, তুমি laptop মেশিনে Linux 6.8.0-48-generic চালাচ্ছ।
```

সারসংক্ষেপ

এই অধ্যায়ের মূল কথা:

- **Terminal** = জানালা (ইনপুট/আউটপুট), **Shell** = দোভাষী (কমান্ড বোঝে ও চালায়), **TTY** = kernel-এর টার্মিনাল ডিভাইস, **Console** = সরাসরি মনিটরের টেক্সট টার্মিনাল।
- GUI টার্মিনাল ও shell একজোড়া **PTY** (`/dev/pts/N`) দিয়ে কথা বলে।
- prompt-এর শেষে `$` = সাধারণ ইউজার, `#` = root।
- কমান্ড = `command -options arguments` ; লিনাক্স case-sensitive।
- builtin shell-এর ভেতরে চলে; external চলে `fork()` + `exec()` দিয়ে নতুন প্রসেসে, `$PATH` থেকে খুঁজে।
- সাহায্যের ক্রম: `whatis` → `tldr` → `--help` / `help` → `man` → `info` ; নাম না জানলে `apropos` ।

দ্রুত রেফারেন্স: কমান্ড

কমান্ড	কাজ
`tty`	বর্তমান টার্মিনাল ডিভাইস দেখাও
`echo $SHELL` / `echo $0`	লগইন shell / চলমান shell
`cat /etc/shells`	ইনস্টল করা বৈধ shell-এর তালিকা
`chsh -s /path/to/shell`	ডিফল্ট লগইন shell বদলাও
`echo $?`	শেষ কমান্ডের exit status
`type -a name`	কমান্ডের ধরন ও সব অবস্থান
`which name` / `command -v name`	external কমান্ডের path
`help name`	builtin-এর সাহায্য
`name --help`	কমান্ডের সংক্ষিপ্ত সাহায্য
`man name` / `man N name`	ম্যানুয়াল পেজ / নির্দিষ্ট section
`man -k word` = `apropos word`	কীওয়ার্ড দিয়ে কমান্ড খোঁজা
`man -f name` = `whatis name`	এক লাইনের বর্ণনা
`sudo mandb`	man-এর ইনডেক্স আপডেট
`info name`	GNU info ডকুমেন্টেশন

কমান্ড	কাজ
`tldr name`	উদাহরণভিত্তিক সংক্ষিপ্ত সাহায্য
`echo [-n] [-e] text`	লেখা প্রিন্ট
`clear` / `reset`	পর্দা পরিষ্কার / টার্মিনাল রিসেট
`date [+FORMAT]`, `date -d STRING`	তারিখ-সময় দেখাও / হিসাব করো
`cal`, `cal -3`, `cal M YYYY`	ক্যালেন্ডার
`whoami`, `id`, `who`, `w`	আমি কে / কে কে লগইন
`uname -a`, `-r`, `-m`	kernel ও আর্কিটেকচারের তথ্য
`cat /etc/os-release`, `hostnamectl`	distro-র নাম ও ভার্সন
`exit`, `Ctrl+D`	shell থেকে বের হওয়া

দ্রুত রেফারেন্স: শর্টকাট

শর্টকাট	কোথায়	কাজ
`Ctrl+Alt+T`	Ubuntu GNOME	টার্মিনাল খোলো
`Ctrl+Alt+F3` … `F6`	যেকোনো জায়গা	টেক্সট virtual console
`Ctrl+Alt+F2` (বা `F1` / `F7`)	console	GUI-তে ফেরো
`Ctrl+Shift+C` / `Ctrl+Shift+V`	টার্মিনাল	কপি / পেস্ট
`Ctrl+Shift+T` / `Ctrl+Shift+W`	টার্মিনাল	ট্যাব খোলো / বন্ধ
`Ctrl+PageUp` / `Ctrl+PageDown`	টার্মিনাল	ট্যাব বদলাও
`Shift+PageUp` / `Shift+PageDown`	টার্মিনাল	আগের আউটপুটে স্ক্রল
`Ctrl++` / `Ctrl+-` / `Ctrl+0`	টার্মিনাল	জুম
`Ctrl+C`	shell	চলমান কমান্ড থামাও
`Ctrl+L`	shell	পর্দা পরিষ্কার
`Ctrl+D`	shell	EOF / shell বন্ধ
`Ctrl+S` / `Ctrl+Q`	shell	আউটপুট থামাও / আবার চালু
`Space` / `b`	man/less	পাতা সামনে / পেছনে
`/word`, `n`, `N`	man/less	খোঁজো, পরের, আগের
`g` / `G`	man/less	শুরু / শেষ
`q`	man/less/info	বের হও
`n` / `p` / `u` / `l`	info	পরের / আগের / উপরের node / ফিরে যাও

(shell-এর ভেতরের লাইন-এডিটিং ও history শর্টকাটের পূর্ণ তালিকা: অধ্যায় ৬; সব শর্টকাট একসাথে: পরিশিষ্ট ক।)

নিজেকে যাচাই করো

1. Terminal emulator আর shell-এর মধ্যে মূল পার্থক্য কী? একটা উপমা দিয়ে বোঝাও।
2. GNOME Terminal-এ `tty` চালালে `/dev/pts/0` আর `Ctrl+Alt+F3`-এ চালালে `/dev/tty3` আসে কেন?
3. prompt `root@server:/var/log#` থেকে কী কী তথ্য পাওয়া যায়?
4. `ls -la` আর `ls --all -l` কি একই কাজ করে? `--all-l` লিখলে কী হবে?
5. `cd` কেন external প্রোগ্রাম হতে পারে না?
6. `/etc/crontab` ফাইলের ফরম্যাট জানতে `man`-এর কোন section দেখবে, কমান্ডটা কী?
7. ফাইল কম্প্রেস করার কমান্ডের নাম মনে নেই। কীভাবে খুঁজবে? আর সেটা কিছু না দেখালে কী করবে?
8. `echo 'Path: $PATH'` আর `echo "Path: $PATH"`-এর আউটপুটে পার্থক্য কী হবে?
9. আজ থেকে ১০০ দিন পরের তারিখ `YYYY-MM-DD` ফরম্যাটে দেখানোর কমান্ড লেখো।
10. `uname -r` আর `cat /etc/os-release` দুটো কী আলাদা তথ্য দেয়?

▼ উত্তর দেখো

1. Terminal emulator শুধু একটা জানালা: কীবোর্ড থেকে অক্ষর নেয় আর আউটপুট পর্দায় আঁকে। Shell হলো সেই প্রোগ্রাম যেটা কমান্ড পার্স করে চালায়। উপমা: টার্মিনাল = টেলিফোন সেট, shell = ফোনের ওপাশের অপারেটর।
2. GUI টার্মিনাল kernel থেকে একটা pseudo-terminal (PTY) নেয়, যার slave প্রান্ত `/dev/pts/N`। আর `Ctrl+Alt+F3` হলো kernel-এর আসল virtual console, যার ডিভাইস `/dev/tty3`।
3. ইউজার `root`, মেশিনের নাম `server`, বর্তমান ডিরেক্টরি `/var/log`, আর `#` মানে root (সর্বোচ্চ ক্ষমতা), তাই সাবধানে কাজ করতে হবে।
4. হ্যাঁ, দুটো একই (`-a` = `--all`, `-l` long format)। কিন্তু long option জোড়া দেওয়া যায় না; `--all-l` লিখলে `ls` এটাকে অচেনা option ধরে এরর দেবে (`unrecognized option '--all-l'`)।
5. external প্রোগ্রাম নতুন child process-এ চলে, আর child process তার parent (shell)-এর working directory বদলাতে পারে না। তাই `cd` shell-এর ভেতরে (builtin) থাকতে হয়, নইলে child-এর ডিরেক্টরি বদলে সে মরে যেত, shell আগের জায়গাতেই থাকত।
6. section 5 (ফাইল ফরম্যাট): `man 5 crontab`। (`man crontab` দিলে section 1-এর `crontab` কমান্ড আসবে।)
7. `apropos compress` (বা `man -k compress`) চালাবে। কিছু না দেখালে ইনডেক্স তৈরি হয়নি, তখন `sudo mandb` চালিয়ে আবার চেষ্টা করবে।
8. সিঙ্গেল কোটে হুবহু `Path: $PATH` প্রিন্ট হবে; ডাবল কোটে `$PATH`-এর আসল মান (যেমন `/usr/local/sbin:/usr/local/bin:...`) বসে যাবে।
9. `date -d "100 days" +%F` (অথবা `date -d "+100 days" +%Y-%m-%d`)।

10. `uname -r` চলমান **kernel**-এর ভার্সন দেয় (যেমন `6.8.0-48-generic`); `/etc/os-release` দেয় **distribution**-এর নাম ও ভার্সন (যেমন Ubuntu 24.04.1 LTS)।

অধ্যায় ৬: টার্মিনাল শর্টকাট ও কমান্ড হিস্ট্রি — দ্রুত কাজের চাবিকাঠি

> একজন নতুন ইউজার আর একজন লিনাক্স এক্সপার্টের মধ্যে সবচেয়ে চোখে পড়ার মতো পার্থক্য কী জানো? এক্সপার্ট কম টাইপ করে। সে জানে কীভাবে আগের কমান্ড ফিরিয়ে আনতে হয়, কীভাবে এক চাপে পুরো লাইন মুছতে হয়, কীভাবে লম্বা ফাইলের নাম Tab চেপে পূরণ করতে হয়। এই অধ্যায়ে আমরা ঠিক সেই "জাদু"গুলো শিখব, আর দেখব ভেতরে আসলে কী ঘটে।

এই অধ্যায়ে যা শিখবে

- কীবোর্ডে একটা কী চাপলে সেটা টার্মিনাল থেকে শেল পর্যন্ত কীভাবে পৌঁছায় (terminal emulator → pty → line discipline → readline → bash)
- **GNU Readline** কী, আর কেন bash-এর প্রায় সব এডিটিং শর্টকাট আসলে readline-এর
- কার্সর চলাফেরার শর্টকাট: `Ctrl+A`, `Ctrl+E`, `Alt+B`, `Alt+F` ইত্যাদি
- লেখা মোছা, কাটা আর পেস্ট করা (kill ring): `Ctrl+U`, `Ctrl+K`, `Ctrl+W`, `Alt+D`, `Ctrl+Y`, `Alt+Y`
- বড়/ছোট হাতের অক্ষর বদলানো ও অদলবদল: `Alt+U`, `Alt+L`, `Alt+C`, `Ctrl+T`, `Alt+T`
- কন্ট্রোল শর্টকাট: `Ctrl+C`, `Ctrl+D`, `Ctrl+Z`, `Ctrl+L`, `Ctrl+S` / `Ctrl+Q` — এবং এগুলো আসলে signal নাকি readline-এর কাজ
- Tab completion, double Tab, আর `bash-completion` প্যাকেজ
- `history` কমান্ড, `~/.bash_history` ফাইল, আর history কীভাবে সেভ হয়
- History expansion: `!!`, `!n`, `!-n`, `!string`, `!?string?`, `!$`, `!^`, `!*`, `^old^new`, modifiers
- `Ctrl+R` reverse search — একদম খুঁটিনাটি
- `HISTSIZE`, `HISTFILESIZE`, `HISTCONTROL`, `HISTIGNORE`, `HISTTIMEFORMAT`, `histappend`
- `fc` কমান্ড দিয়ে আগের কমান্ড এডিটরে খুলে ঠিক করা
- Terminal emulator-এর নিজস্ব শর্টকাট (`Ctrl+Shift+C/V/T/N` ইত্যাদি)
- emacs mode বনাম vi mode (`set -o vi`)
- `~/.inputrc` দিয়ে readline কাস্টমাইজ করা, আর `bind` কমান্ড

৬.১ কেন শর্টকাট শিখব?

ধরো তুমি এই লম্বা কমান্ডটা টাইপ করলে:

```
$ sudo systemctl restart nginx.servcie
```

Enter চাপার পর দেখলে বানান ভুল — `servcie`। নতুন ইউজার কী করে? পুরো লাইন আবার টাইপ করে। এক্সপার্ট কী করে?

```
$ ^servcie^service
sudo systemctl restart nginx.service
```

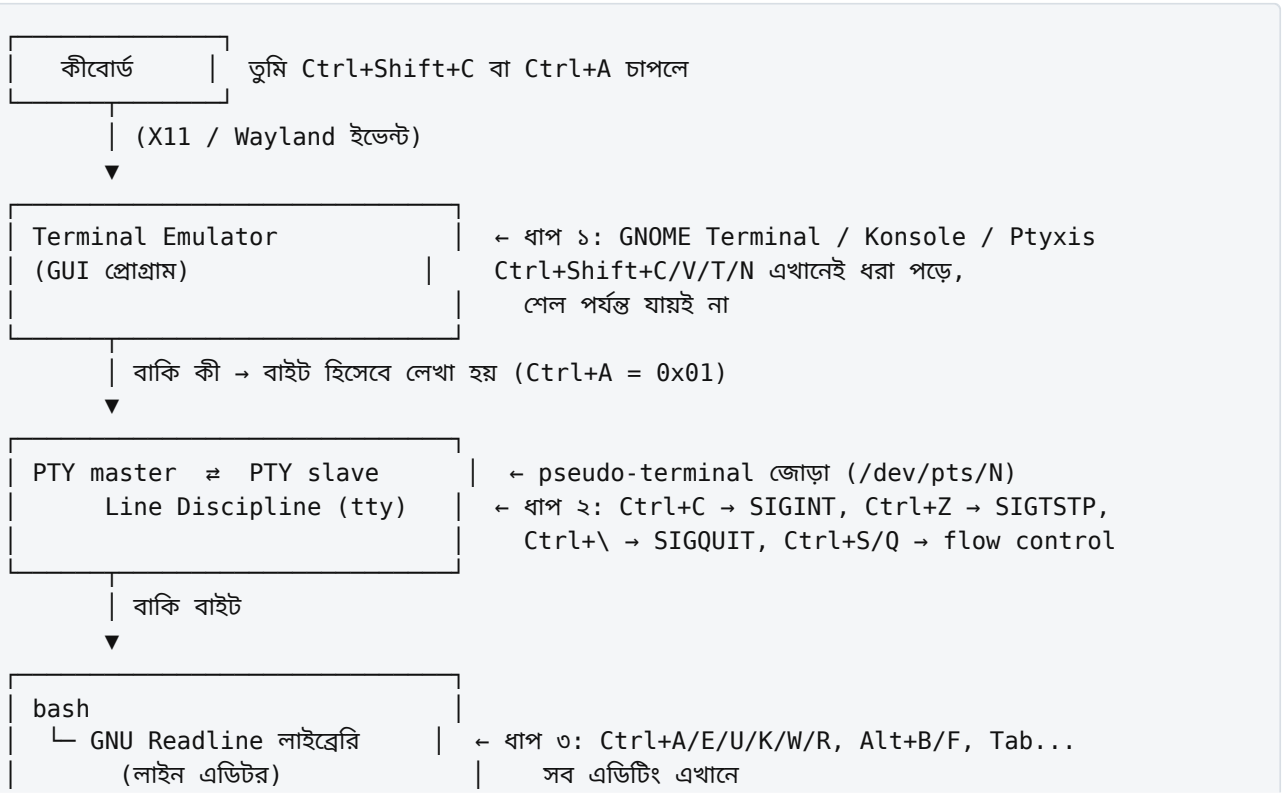
মাত্র ১৭টা অক্ষর! আবার ধরো ২০ মিনিট আগে একটা লম্বা `rsync` কমান্ড চালিয়েছিলে। এক্সপার্ট `Ctrl+R` চেপে `rsync` লিখবে — কমান্ডটা সামনে চলে আসবে।

দিনে হাজারবার কমান্ড লিখলে প্রতিবার ৫ সেকেন্ড বাঁচানো মানে দিনে প্রায় দেড় ঘণ্টা! তাছাড়া কম টাইপ মানে কম ভুল। সার্ভারে কাজ করার সময় একটা ভুল অক্ষর অনেক বড় বিপদ ডেকে আনতে পারে।

> 💡 **টিপস:** এই অধ্যায়ের সব শর্টকাট একসাথে মুখস্থ করার চেষ্টা কোরো না। প্রথম সপ্তাহে শুধু `Ctrl+A`, `Ctrl+E`, `Ctrl+U`, `Ctrl+R`, `Tab` আর `!!` ব্যবহার করো। আঙুলে বসে গেলে পরের সপ্তাহে আরও ৫টা যোগ করো।

৬.২ ভেতরে কী ঘটে — কীবোর্ড থেকে শেল পর্যন্ত

শর্টকাটগুলো কেন এভাবে আচরণ করে, সেটা বুঝতে হলে আগে জানতে হবে একটা কী চাপলে সেটা কোথায় কোথায় যায়। অধ্যায় ৫-এ আমরা terminal, shell আর console-এর পার্থক্য দেখেছি। এখন একটু গভীরে যাই।



```
│  └─ History expansion          │  ← ধাপ ৪: Enter চাপার পর !!, !$, ^a^b
└──────────────────────────────┘
```

তাহলে শর্টকাট মোটামুটি তিন জায়গায় ধরা পড়ে:

স্তর	কারা ধরে	উদাহরণ
Terminal emulator	GUI অ্যাপ নিজে	`Ctrl+Shift+C`, `Ctrl+Shift+T`, `F11`
Kernel-এর tty line discipline	কার্নেল	`Ctrl+C`, `Ctrl+Z`, `Ctrl+\`, `Ctrl+S`, `Ctrl+Q`
Readline (bash-এর ভেতরে)	bash প্রসেস	`Ctrl+A`, `Ctrl+R`, `Alt+.`, `Tab`

উপমা: রেস্টুরেন্টের অর্ডার

ধরো তুমি রেস্টুরেন্টে অর্ডার দিচ্ছ।

- **ওয়েটার** (terminal emulator) কিছু কথা নিজেই সামলায় — "পানি দিন", "টেবিল মুছে দিন"। এগুলো রান্নাঘরে যায়ই না (যেমন `Ctrl+Shift+T` নতুন ট্যাব)।
- **ম্যানেজার** (tty line discipline) কিছু জরুরি নির্দেশ সামলায় — "অর্ডার বাতিল!" — সে সরাসরি রান্নাঘরে গিয়ে রাঁধুনিকে থামিয়ে দেয় (যেমন `Ctrl+C` = SIGINT)।
- **অর্ডার-প্যাড** (readline) — তুমি অর্ডার লিখছ, কাটছ, ঠিক করছ। চূড়ান্ত না হওয়া পর্যন্ত (Enter) রাঁধুনি (bash) কিছুই রাঁধে না।

নিজে দেখো: `stty -a`

কোন Ctrl-কী কোন tty কাজে বাঁধা, তা দেখতে:

```
$ stty -a
speed 38400 baud; rows 40; columns 120; line = 0;
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = <undef>;
eol2 = <undef>; swtch = <undef>; start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R;
werase = ^W; lnext = ^V; discard = ^O; min = 1; time = 0;
...
```

এখানে `^C` মানে `Ctrl+C`। `intr = ^C` মানে Ctrl+C চাপলে interrupt signal (SIGINT) যাবে। `susp = ^Z` মানে suspend।

> 📝 **মনে রাখো:** Readline লাইন এডিট করার সময় tty-কে প্রায় "raw" মোডে রাখে, তাই `^U`, `^W`-এর মতো কী readline নিজেই সামলায় (`stty`-র সেটিং দেখে)। কিন্তু `Ctrl+C`, `Ctrl+Z` সবসময় কার্নেলই signal-এ রূপান্তর করে — তাই এগুলো যেকোনো প্রোগ্রামে কাজ করে, শুধু bash-এ নয়।

GNU Readline কী?

Readline হলো একটা লাইব্রেরি (অনেক প্রোগ্রাম মিলে ব্যবহার করা কোডের সংগ্রহ), যেটা "একটা লাইন ইনপুট নাও, আর ইউজারকে সেটা আরামে এডিট করতে দাও" — এই কাজটা করে। bash ছাড়াও `python3` REPL, `mysql`, `psql`, `gdb`, `sqlite3`-এর মতো অনেক প্রোগ্রাম readline ব্যবহার করে। তাই এই অধ্যায়ে যা শিখবে, সেগুলো ওই প্রোগ্রামগুলোতেও প্রায় হুবহু কাজ করবে!

Readline-এর ডিফল্ট কী-বাইন্ডিং এসেছে **Emacs** এডিটর থেকে। তাই একে বলে **emacs mode**। চাইলে **vi mode**-ও চালু করা যায় (৬.১১ অংশে দেখব)।

"Meta" কী আর Alt

Readline-এর ডকুমেন্টেশনে `M-f`, `M-b` লেখা থাকে। **M** মানে **Meta**। আজকের কীবোর্ডে Meta = `Alt`। আর `C-a` মানে `Ctrl+A`।

ডকুমেন্টেশনে লেখা	কীবোর্ডে চাপো
`C-a`	`Ctrl+A`
`M-f`	`Alt+F` (অথবা `Esc` ছেড়ে তারপর `F`)
`C-x C-e`	`Ctrl+X` চেপে ছাড়ো, তারপর `Ctrl+E`
`M-C-e`	`Ctrl+Alt+E`

> ⚠️ **সতর্কতা:** কিছু terminal emulator-এ `Alt+F` চাপলে "File" মেনু খুলে যায়, শেলে পৌঁছায় না। সমাধান: (১) টার্মিনালের Preferences থেকে "Enable menu access keys" বন্ধ করো, অথবা (২) `Alt`-এর বদলে `Esc` চেপে ছেড়ে দিয়ে তারপর `F` চাপো — readline এটাকে `Alt+F` হিসেবেই ধরে। macOS Terminal-এ "Use Option as Meta key" চালু করতে হয়।

৬.৩ কার্সর চলাফেরা (Movement)

ধরো এই লাইন টাইপ করেছ, কার্সর একদম শেষে:

```
$ cp /var/log/nginx/access.log /home/rahim/backup/
```

এখন শুরুতে `sudo` বসাতে চাও। ←-কী চেপে ৫০ বার যাবে? না! `Ctrl+A` চাপো, কার্সর একলাফে শুরুতে।

শর্টকাট	Readline ফাংশন	কাজ
`Ctrl+A`	`beginning-of-line`	লাইনের **শুরুতে** যাও (A = alphabet-এর শুরু)
`Ctrl+E`	`end-of-line`	লাইনের **শেষে** যাও (E = End)

শর্টকাট	Readline ফাংশন	কাজ
`Ctrl+B`	`backward-char`	এক অক্ষর পেছনে (← এর মতো) (B = Back)
`Ctrl+F`	`forward-char`	এক অক্ষর সামনে (→ এর মতো) (F = Forward)
`Alt+B`	`backward-word`	এক **শব্দ** পেছনে
`Alt+F`	`forward-word`	এক **শব্দ** সামনে
`Ctrl+←` / `Ctrl+→`	(বেশিরভাগ distro-তে `/etc/inputrc`-এ বাঁধা)	শব্দ ধরে পেছনে/সামনে
`Home` / `End`		লাইনের শুরু/শেষ
`Ctrl+X Ctrl+X`	`exchange-point-and-mark`	কার্সর আর "mark"-এর মধ্যে লাফ দাও (দুই প্রান্তে আসা-যাওয়ার জন্য দারুণ)
`Ctrl+]` তারপর একটা অক্ষর	`character-search`	সামনে ওই অক্ষরের পরের উপস্থিতিতে লাফ দাও
`Ctrl+Alt+]` তারপর অক্ষর	`character-search-backward`	পেছনে খোঁজো

উদাহরণ — `Ctrl+]` চেপে `/` চাপলে কার্সর পরের `/`-তে চলে যাবে।

> 📝 **মনে রাখো — "শব্দ" কী?** `Alt+B` / `Alt+F`-এর কাছে শব্দ মানে শুধু অক্ষর আর সংখ্যা (alphanumeric)। তাই `/var/log/nginx`-এ `var`, `log`, `nginx` আলাদা তিনটা শব্দ। কিন্তু `Ctrl+W`-এর কাছে শব্দ মানে স্পেস দিয়ে আলাদা করা অংশ — এই পার্থক্যটা পরের অংশে কাজে লাগবে।

৬.৪ মোছা, কাটা আর পেস্ট করা — Kill Ring

Readline-এ "মুছে ফেলা" দুই রকম:

1. **Delete** — অক্ষর একদম হারিয়ে যায় (`Backspace`, `Ctrl+D`, `Delete`)।
2. **Kill** — লেখাটা কাটা হয় এবং একটা বিশেষ ক্লিপবোর্ডে জমা থাকে, যার নাম **kill ring**। পরে `Ctrl+Y` (yank) দিয়ে ফেরত আনা যায়।

Kill ring — উপমা

Kill ring হলো একটা গোল ঘূর্ণায়মান তাক। প্রতিবার কিছু kill করলে সেটা তাকের সামনে রাখা হয়। `Ctrl+Y` সামনের জিনিসটা বের করে। `Ctrl+Y`-এর ঠিক পরে `Alt+Y` চাপলে তাক ঘুরে আগেরটা আসে।

```
    Kill ring (ঘূর্ণায়মান)
  ┌──────────────────────────┐
  │ [৩] "nginx"     ← সর্বশেষ │  Ctrl+Y → "nginx"
  │ [২] "--force "           │  তারপর Alt+Y → "--force "
```

```
  [১] "sudo "                  আবার Alt+Y → "sudo "
```

> 📝 **মনে রাখো:** এই kill ring শুধু ওই শেলের ভেতরে। এটা তোমার ডেস্কটপের ক্লিপবোর্ড (`Ctrl+Shift+C/V`) থেকে সম্পূর্ণ আলাদা!

Kill ও Delete শর্টকাট

শর্টকাট	Readline ফাংশন	কাজ
`Ctrl+U`	`unix-line-discard`	কার্সর থেকে **লাইনের শুরু পর্যন্ত** কেটে ফেলো
`Ctrl+K`	`kill-line`	কার্সর থেকে **লাইনের শেষ পর্যন্ত** কেটে ফেলো
`Ctrl+W`	`unix-word-rubout`	কার্সরের **আগের শব্দ** কাটো (স্পেস পর্যন্ত)
`Alt+Backspace`	`backward-kill-word`	আগের শব্দ কাটো (alphanumeric শব্দ — `/` এ থামে)
`Alt+D`	`kill-word`	কার্সর থেকে **সামনের শব্দের শেষ** পর্যন্ত কাটো
`Ctrl+Y`	`yank`	সর্বশেষ কাটা লেখা পেস্ট করো
`Alt+Y`	`yank-pop`	ঠিক আগে `Ctrl+Y` করে থাকলে — kill ring ঘুরিয়ে আগের কাটা লেখা আনো
`Ctrl+D`	`delete-char`	কার্সরের নিচের অক্ষর মোছো (লাইন খালি হলে = EOF, নিচে দেখো)
`Backspace`	`backward-delete-char`	কার্সরের আগের অক্ষর মোছো
`Alt+\`	`delete-horizontal-space`	কার্সরের চারপাশের সব স্পেস/ট্যাব মুছে ফেলো
`Ctrl+_` বা `Ctrl+X Ctrl+U`	`undo`	শেষ এডিট **undo** করো (বারবার চাপা যায়)
`Alt+R`	`revert-line`	লাইনে করা সব পরিবর্তন বাতিল করো

`Ctrl+W` বনাম `Alt+Backspace` — হাতে-কলমে

```
$ ls /var/log/nginx
```

- `Ctrl+W` চাপলে: পুরো `/var/log/nginx` মুছে যাবে (কারণ স্পেস পর্যন্ত এক শব্দ)।
- `Alt+Backspace` চাপলে: শুধু `nginx` মুছবে, থাকবে `ls /var/log/`।

পাথের শেষ অংশ বদলাতে `Alt+Backspace` আদর্শ।

বাস্তব উদাহরণ ১: ভুলে যাওয়া `sudo`

```
$ apt install htop
E: Could not open lock file /var/lib/dpkg/lock-frontend - open (13: Permission denied)
```

সবচেয়ে সহজ:

```
$ sudo !!
sudo apt install htop
```

`!!` সম্পর্কে ৬.৮ অংশে বিস্তারিত।

বাস্তব উদাহরণ ২: অর্ধেক লেখা কমান্ড "পার্ক" করা

তুমি একটা লম্বা কমান্ড লিখছ, হঠাৎ মনে হলো আগে দেখা দরকার ফোল্ডারটা আছে কিনা।

```
$ tar -czvf backup-2026.tar.gz /home/rahim/projects --exclude=node_modules
```

1. `Ctrl+E` (শেষে যাও, যদি না থাকো), তারপর `Ctrl+U` — পুরো লাইন kill ring-এ চলে গেল।
2. `ls /home/rahim/projects` লিখে Enter।
3. `Ctrl+Y` — আগের পুরো কমান্ড ফেরত!

> 💡 **টিপস:** এর চেয়েও সুন্দর একটা উপায় আছে: লাইনের শুরুতে `#` বসিয়ে (`Ctrl+A`, তারপর `#`) Enter চাপো, অথবা সরাসরি `Alt+#` চাপো (`insert-comment`)। কমান্ডটা কমেন্ট হিসেবে history-তে সেভ হবে, চলবে না। পরে `↑` চেপে এনে `#` মুছে চালাও।

৬.৫ বড়-ছোট হাতের অক্ষর ও অদলবদল

শর্টকাট	Readline ফাংশন	কাজ
`Alt+U`	`upcase-word`	কার্সর থেকে শব্দের শেষ পর্যন্ত **UPPERCASE** করো
`Alt+L`	`downcase-word`	কার্সর থেকে শব্দের শেষ পর্যন্ত **lowercase** করো
`Alt+C`	`capitalize-word`	শব্দের প্রথম অক্ষর বড়, বাকি ছোট (**Capitalize**)
`Ctrl+T`	`transpose-chars`	কার্সরের আগের অক্ষর আর কার্সরের নিচের অক্ষর অদলবদল
`Alt+T`	`transpose-words`	আগের শব্দ আর বর্তমান শব্দ অদলবদল

উদাহরণ: টাইপো ঠিক করা

```
$ sl -la        # ভুল: sl লিখতে চেয়েছিলে ls
```

`Ctrl+A` চেপে শুরুতে যাও, `Ctrl+F` একবার (কার্সর `l`-এর ওপর), তারপর `Ctrl+T`:

```
$ ls -la
```

> 💡 **টিপস:** লাইনের একদম শেষে `Ctrl+T` চাপলে শেষ দুটো অক্ষর অদলবদল হয়। যেমন `grpe` লিখে ফেলেছ (কার্সর শেষে) — সাথে সাথে `Ctrl+T` চাপো, হয়ে যাবে `grep`। মূল কথা: `Ctrl+T` কার্সরের **ঠিক আগের** অক্ষরকে এক ঘর সামনে টেনে আনে। টাইপ করার সাথে সাথে শেষ দুটো অক্ষর উল্টো হলে (যেমন `sl` → `ls`) এটা সবচেয়ে কাজের।

উদাহরণ: environment variable বড় হাতে

```
$ echo $home
```

`Alt+B` চেপে `home`-এর শুরুতে যাও, তারপর `Alt+U`:

```
$ echo $HOME
/home/rahim
```

৬.৬ কন্ট্রোল শর্টকাট: Ctrl+C, D, Z, L, S, Q

এগুলো লাইন এডিট করে না — প্রসেস বা টার্মিনালকে নিয়ন্ত্রণ করে। ভেতরে কে সামলায় সেটা খেয়াল করো।

শর্টকাট	কে সামলায়	কাজ
`Ctrl+C`	tty → **SIGINT**	চলমান প্রোগ্রামকে থামতে বলো (interrupt)। প্রম্পটে চাপলে বর্তমান লাইন বাতিল হয়ে নতুন প্রম্পট আসে
`Ctrl+Z`	tty → **SIGTSTP**	চলমান প্রোগ্রাম **সাময়িক থামাও** (suspend), শেলে ফিরে আসো
`Ctrl+\`	tty → **SIGQUIT**	আরও কড়া থামানো; প্রোগ্রাম core dump তৈরি করতে পারে
`Ctrl+D`	tty/readline → **EOF**	খালি লাইনে: "ইনপুট শেষ" — শেল/প্রোগ্রাম বন্ধ হয়। লেখা থাকলে: কার্সরের নিচের অক্ষর মোছে
`Ctrl+L`	readline (`clear-screen`)	স্ক্রিন পরিষ্কার, বর্তমান লাইন অক্ষত থাকে
`Ctrl+Alt+L`	readline (`clear-display`, bash 5.1+)	স্ক্রিন **এবং** scrollback দুটোই পরিষ্কার (টার্মিনাল সাপোর্ট করলে)
`Ctrl+S`	tty (**XOFF**)	টার্মিনাল আউটপুট **জমিয়ে দেয়** (freeze)
`Ctrl+Q`	tty (**XON**)	জমানো আউটপুট আবার চালু করে
`Ctrl+V` তারপর কী	readline (`quoted-insert`)	পরের কী-টা আক্ষরিক অর্থে বসাও (যেমন আসল Tab অক্ষর)

শর্টকাট	কে সামলায়	কাজ
`Ctrl+J` বা `Ctrl+M`	readline	Enter-এর সমান (newline / carriage return)
`Ctrl+O`	readline (`operate-and-get-next`)	বর্তমান লাইন চালাও, আর history-র পরের লাইনটা প্রস্তুত করে রাখো

`Ctrl+C` — কেন এটা "Copy" নয়?

উইন্ডোজে অভ্যস্তরা সবচেয়ে বেশি এখানে ধাক্কা খায়। টার্মিনালে `Ctrl+C` চাপলে কার্নেলের tty driver বাইট `0x03` দেখে চলমান **foreground process group**-কে `SIGINT` signal পাঠায়। বেশিরভাগ প্রোগ্রাম এতে বন্ধ হয়ে যায়।

```
$ ping google.com
PING google.com (142.250.193.14) 56(84) bytes of data.
64 bytes from ...: icmp_seq=1 ttl=117 time=24.1 ms
64 bytes from ...: icmp_seq=2 ttl=117 time=23.8 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
```

টার্মিনালে কপি করতে `Ctrl+Shift+C` লাগে (৬.১০ অংশ)। Signal সম্পর্কে বিস্তারিত পড়বে অধ্যায় ২৪-এ।

> 📝 **মনে রাখো:** `Ctrl+C` একটা **অনুরোধ**। প্রোগ্রাম চাইলে SIGINT উপেক্ষা করতে পারে। আটকে গেলে `Ctrl+\` চেষ্টা করো, তাতেও না হলে অন্য টার্মিনাল থেকে `kill -9 <PID>`।

`Ctrl+Z` — থামাও, কিন্তু মেরো না

```
$ vim notes.txt
# vim-এ কাজ করছিলে, Ctrl+Z চাপলে:
[1]+  Stopped                 vim notes.txt
$ ls          # অন্য কাজ করো
$ fg          # vim ফিরে আসবে, যেখানে ছিলে ঠিক সেখানে
```

> ⚠️ **সতর্কতা:** অনেকে ভাবে `Ctrl+Z` প্রোগ্রাম বন্ধ করে। না! প্রোগ্রাম মেমোরিতে থেকে যায়। শেল থেকে `exit` করতে গেলে bash সাবধান করবে: `There are stopped jobs.` `jobs`, `fg`, `bg` নিয়ে বিস্তারিত অধ্যায় ২৫-এ।

`Ctrl+D` — EOF, অর্থাৎ "ইনপুট শেষ"

EOF মানে End Of File। খালি লাইনে `Ctrl+D` চাপা মানে শেলকে বলা "আমার আর কিছু বলার নেই" — শেল বেরিয়ে যায় (টার্মিনাল ট্যাব বন্ধ হয়, বা SSH সেশন শেষ হয়)।

```
$ cat > list.txt
আলু
পেঁয়াজ
^D          ← খালি লাইনে Ctrl+D, cat থামল, ফাইল সেভ হলো
$ cat list.txt
আলু
পেঁয়াজ
```

> ⚠️ **সতর্কতা:** ভুল করে `Ctrl+D` চেপে গুরুত্বপূর্ণ root শেল বা SSH সেশন বন্ধ হয়ে যাওয়া খুব কমন। প্রতিরোধ:
>
> ```
> $ set -o ignoreeof        # এখন খালি লাইনে Ctrl+D-তে বেরোবে না
> $ ^D
> Use "exit" to leave the shell.
> ```
>
> অথবা `IGNOREEOF=3` সেট করলে পরপর ৩ বার `Ctrl+D` চাপলে তবেই বেরোবে।

`Ctrl+L` বনাম `clear` বনাম `reset`

উপায়	কী করে
`Ctrl+L`	স্ক্রিন পরিষ্কার, অর্ধেক লেখা কমান্ড হারায় না
`clear`	স্ক্রিন পরিষ্কার (সাধারণত scrollback-ও মুছে দেয়)
`reset`	টার্মিনাল পুরো রিসেট করে — যখন কোনো binary ফাইল `cat` করে টার্মিনাল আজব অক্ষরে ভরে যায়

`Ctrl+S` — "টার্মিনাল জমে গেছে!" রহস্য

নতুনরা প্রায়ই ভুলে `Ctrl+S` চাপে (অন্য অ্যাপে "Save" এর অভ্যাস থেকে) এবং মনে করে টার্মিনাল হ্যাং হয়ে গেছে — কিছু টাইপ করলে দেখা যায় না। আসলে এটা ১৯৭০-এর দশকের **XON/XOFF flow control** — ধীর প্রিন্টার-টার্মিনালকে "একটু থামো" বলার ব্যবস্থা।

সমাধান: `Ctrl+Q` চাপো। সবকিছু আবার চলবে (এর মধ্যে যা টাইপ করেছিলে তা-ও চলে আসবে — সাবধান!)

সমস্যা হলো, readline-এ `Ctrl+S` আসলে **forward history search**-এর জন্য বাঁধা, কিন্তু tty আগেই ধরে ফেলে বলে কখনো readline পর্যন্ত পৌঁছায় না। flow control বন্ধ করে দিলে `Ctrl+S` readline-এর কাজে লাগবে:

```
$ stty -ixon       # এই সেশনের জন্য; স্থায়ী করতে ~/.bashrc-এ লেখো
```

৬.৭ Tab Completion — অর্ধেক লেখো, বাকিটা শেল লিখবে

Tab completion হলো লিনাক্স টার্মিনালের সবচেয়ে বড় সময়-বাঁচানো ফিচার। কোনো কিছুর শুরুটা লিখে `Tab` চাপো:

- **একটাই মিল থাকলে** → শেল পুরোটা লিখে দেয়।
- **একাধিক মিল থাকলে** → যতটুকু সবার মধ্যে কমন, ততটুকু লিখে থামে, একটা বিপ (bell) দিতে পারে।
- **তখন আবার `Tab` চাপলে (double Tab)** → সব সম্ভাব্য অপশনের তালিকা দেখায়।

```
$ cd /usr/sh<Tab>
$ cd /usr/share/

$ sys<Tab><Tab>
sysctl                    systemd-analyze           systemd-run
systemctl                 systemd-ask-password      systemd-tmpfiles
systemd                   systemd-cat               ...
$ sys
```

অনেক বেশি মিল থাকলে (ডিফল্ট ১০০-র বেশি) জিজ্ঞেস করে:

```
$ <Tab><Tab>
Display all 2381 possibilities? (y or n)
```

`n` চাপো, তারপর আরও কয়েকটা অক্ষর লিখে আবার Tab।

কী কী complete হয়?

কী লিখছ	কী complete হয়
লাইনের প্রথম শব্দ	কমান্ড (PATH-এর প্রোগ্রাম, alias, function, builtin)
পরের শব্দ	ফাইল ও ডিরেক্টরির নাম
`$` দিয়ে শুরু	ভেরিয়েবলের নাম (`$HO<Tab>` → `$HOME`)
`~` দিয়ে শুরু	ইউজারনেম (`~ro<Tab>` → `~root/`)
`@` দিয়ে শুরু	হোস্টনেম (`/etc/hosts` থেকে)

Programmable completion ও `bash-completion` প্যাকেজ

Ubuntu/Debian/Fedora-তে সাধারণত `bash-completion` প্যাকেজ ইনস্টল থাকে। এটা থাকলে শেল কমান্ড বুঝে স্মার্ট completion দেয়:

```
$ git che<Tab>
$ git checkout
$ systemctl rest<Tab>
$ systemctl restart
$ sudo apt ins<Tab>
```

```
$ sudo apt install
$ ssh <Tab><Tab>          # ~/.ssh/config আর known_hosts থেকে সার্ভারের নাম
```

এটা কাজ করে `complete` নামে bash-এর একটা builtin দিয়ে, যেখানে প্রতিটা কমান্ডের জন্য "কী কী সাজেস্ট করতে হবে" তার নিয়ম বাঁধা থাকে।

```
$ complete -p git
complete -o bashdefault -o default -o nospace -F __git_wrap__git_main git
```

না থাকলে ইনস্টল করো (প্যাকেজ ইনস্টল নিয়ে বিস্তারিত অধ্যায় ৩০-এ):

```
$ sudo apt install bash-completion      # Ubuntu/Debian
$ sudo dnf install bash-completion      # Fedora/RHEL
```

নিজের কমান্ডের জন্য সহজ completion বানাতে পারো:

```
$ complete -W "start stop restart status" myservice
$ myservice st<Tab><Tab>
start   status  stop
```

Completion সংক্রান্ত অন্যান্য শর্টকাট

শর্টকাট	Readline ফাংশন	কাজ
`Tab`	`complete`	Complete করো
`Tab Tab`	—	সম্ভাব্য সব দেখাও
`Alt+?`	`possible-completions`	সম্ভাব্য সব দেখাও (Tab Tab-এর মতো)
`Alt+*`	`insert-completions`	সব মিল একসাথে লাইনে বসিয়ে দাও
`Alt+/`	`complete-filename`	জোর করে ফাইলনেম complete করো
`Alt+$`	`complete-variable`	ভেরিয়েবল complete করো
`Alt+~`	`complete-username`	ইউজারনেম complete করো
`Alt+@`	`complete-hostname`	হোস্টনেম complete করো
`Alt+!`	`complete-command`	কমান্ডের নাম complete করো

উদাহরণ — `Alt+*`:

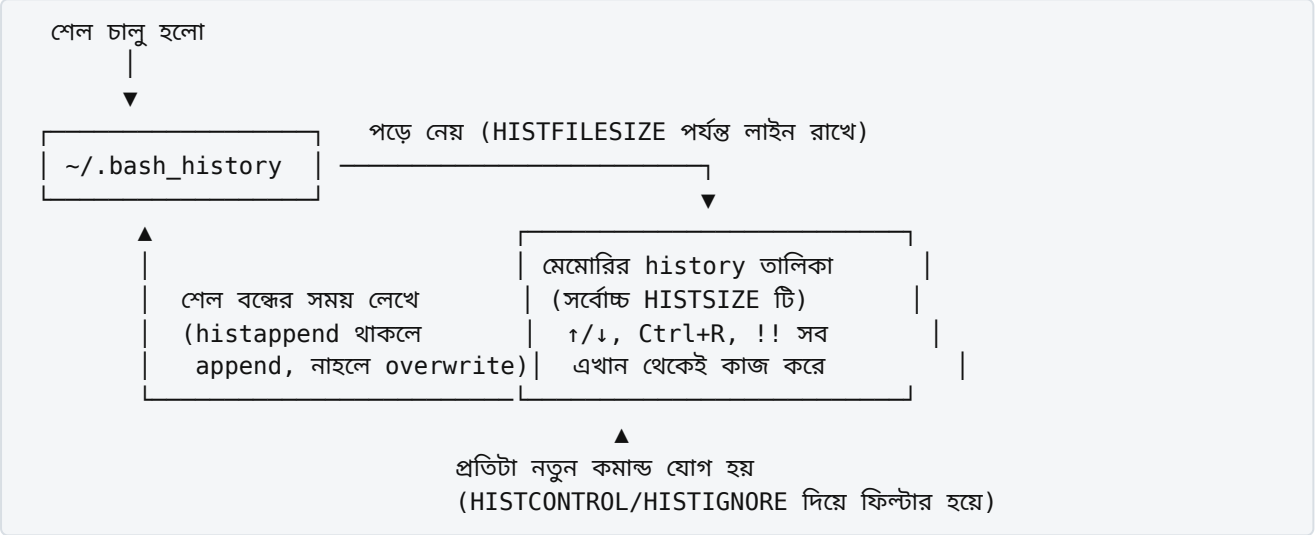
```
$ rm *.log<Alt+*>
$ rm access.log error.log debug.log
```

মোছার আগে ঠিক কোন ফাইলগুলো যাবে, তা দেখে নেওয়ার চমৎকার উপায়!

> 💡 **টিপস:** Tab-কে "বানান যাচাইকারী" হিসেবেও ব্যবহার করো। ফাইলের নাম লিখে Tab চাপলে যদি complete না হয়, তার মানে নাম ভুল বা ফাইল নেই — Enter চাপার আগেই বুঝে যাবে।

৬.৮ কমান্ড হিস্ট্রি — শেলের স্মৃতি

তুমি যত কমান্ড চালাও, bash সব মনে রাখে। প্রথমে **মেমোরিতে** (একটা তালিকায়), আর শেল বন্ধ হওয়ার সময় সেগুলো লেখে `~/.bash_history` ফাইলে।



> 📝 **মনে রাখো:** একটা টার্মিনালে চালানো কমান্ড অন্য খোলা টার্মিনালে সাথে সাথে `↑` চেপে পাবে না — কারণ প্রতিটা শেলের নিজস্ব মেমোরি-তালিকা আছে, আর ফাইলে লেখা হয় শেল বন্ধের সময়। এর সমাধান ৬.৯ অংশে।

History-তে ঘোরাফেরা

শর্টকাট	Readline ফাংশন	কাজ
`↑` বা `Ctrl+P`	`previous-history`	আগের কমান্ড (P = Previous)
`↓` বা `Ctrl+N`	`next-history`	পরের কমান্ড (N = Next)
`Alt+<`	`beginning-of-history`	history-র একদম প্রথম কমান্ড
`Alt+>`	`end-of-history`	বর্তমান (খালি) লাইনে ফেরো
`Ctrl+R`	`reverse-search-history`	পেছনের দিকে খোঁজো (incremental)
`Ctrl+S`	`forward-search-history`	সামনের দিকে খোঁজো (`stty -ixon` দরকার)

শর্টকাট	Readline ফাংশন	কাজ
`Alt+P`	`non-incremental-reverse-search-history`	পুরো শব্দ লিখে Enter, তারপর খোঁজে
`Alt+N`	`non-incremental-forward-search-history`	ওপরেরটার সামনের দিকের সংস্করণ
`Alt+.` বা `Alt+_`	`yank-last-arg`	আগের কমান্ডের **শেষ argument** বসাও; বারবার চাপলে আরও আগের কমান্ডে যায়
`Ctrl+Alt+Y`	`yank-nth-arg`	আগের কমান্ডের **প্রথম argument** বসাও
`Alt+^`	`history-expand-line`	লাইনের `!!`, `!$` ইত্যাদি এখনই খুলে দেখাও (না চালিয়ে)
`Ctrl+Alt+E`	`shell-expand-line`	alias, ভেরিয়েবল, history সব expand করে দেখাও
`Ctrl+X Ctrl+E`	`edit-and-execute-command`	বর্তমান লাইন `$EDITOR`-এ খোলো; সেভ করে বেরোলে চলবে

`Alt+.` — সবচেয়ে ভালোবাসার শর্টকাট

```
$ mkdir -p /home/rahim/projects/website/assets/images
$ cd <Alt+.>
$ cd /home/rahim/projects/website/assets/images
```

বারবার `Alt+.` চাপলে আরও পেছনের কমান্ডগুলোর শেষ argument ঘুরে ঘুরে আসে।

সংখ্যা সহ (numeric argument): `Alt` চেপে ধরে একটা সংখ্যা চাপো, তারপর `Alt+.` — তখন আগের কমান্ডের **ওই নম্বরের** শব্দ আসে (০ = কমান্ডের নাম)।

```
$ cp report.pdf /mnt/usb/
$ ls -l <Alt+1><Alt+.>
$ ls -l report.pdf
```

> 💡 **টিপস:** Readline-এর প্রায় সব ফাংশনের আগে `Alt+সংখ্যা` দিয়ে "কতবার" বলে দেওয়া যায় (`digit-argument`)। যেমন `Alt+3 Ctrl+W` = আগের ৩টা শব্দ মুছে ফেলো। `Alt+-` দিয়ে উল্টো দিক।

`Ctrl+X Ctrl+E` — লম্বা কমান্ডের জন্য এডিটর

একটা জটিল one-liner লিখছ, পাঁচটা পাইপ দিয়ে — টার্মিনালের এক লাইনে এডিট করা কষ্টের। `Ctrl+X Ctrl+E` চাপলে লাইনটা তোমার এডিটরে (`$VISUAL`, না থাকলে `$EDITOR`, না থাকলে ডিফল্ট — সাধারণত Debian/Ubuntu-তে `editor` → nano) খুলবে। সেভ করে বেরোলে কমান্ডটা চলবে।

```
$ export EDITOR=nano     # বা vim — nano শিখবে অধ্যায় ১৩-এ, vim অধ্যায় ১৪-এ
```

> ⚠️ **সতর্কতা:** এডিটর থেকে সেভ করে বেরোলে কমান্ড **সাথে সাথে চলে**। চালাতে না চাইলে এডিটরে সব লেখা মুছে সেভ করো।

৬.৯ `history` কমান্ড

Syntax

```
history [n]
history -c
history -d offset
history -d start-end
history -a | -n | -r | -w [filename]
history -s arg [arg...]
```

গুরুত্বপূর্ণ অপশন

অপশন	কাজ
(কিছু না)	পুরো history তালিকা, নম্বর সহ
`n` (একটা সংখ্যা)	শেষ `n` টা কমান্ড দেখাও
`-c`	মেমোরির পুরো history মুছে দাও (clear)
`-d offset`	নির্দিষ্ট নম্বরের এন্ট্রি মোছো
`-d start-end`	একটা রেঞ্জ মোছো (bash 5.0+)
`-a`	এই সেশনের নতুন লাইনগুলো ফাইলে **append** করো
`-n`	ফাইলে থাকা কিন্তু এখনো না-পড়া লাইনগুলো মেমোরিতে আনো
`-r`	পুরো ফাইল পড়ে মেমোরির তালিকায় যোগ করো
`-w`	মেমোরির পুরো তালিকা ফাইলে লেখো (overwrite)
`-s arg`	কমান্ড না চালিয়েই history-তে একটা এন্ট্রি যোগ করো
`-p arg`	history expansion করে ফল দেখাও, চালায় না এবং সেভ করে না

উদাহরণ

```
$ history 5
  496  cd /etc/nginx
  497  sudo nano nginx.conf
  498  sudo nginx -t
  499  sudo systemctl reload nginx
  500  history 5
```

```
$ history | grep ssh          # পাইপ আর grep শিখবে অধ্যায় ১৯ ও ২০-এ
  212  ssh rahim@192.168.1.50
  340  ssh -p 2222 deploy@example.com
```

ভুল করে পাসওয়ার্ড কমান্ডে লিখে ফেলেছ? মুছে দাও:

```
$ mysql -u root -pMySecret123        # ওহ না!
$ history 2
  612  mysql -u root -pMySecret123
  613  history 2
$ history -d 612
$ history -w                         # ফাইলেও হালনাগাদ করো
```

> ⚠️ **সতর্কতা:** `history -c` শুধু মেমোরি মোছে। ফাইল `~/.bash_history` থেকে মুছতে এরপর `history -w` চালাও। আর মনে রেখো, ব্যাকআপ বা অন্য খোলা শেলে কপি থেকে যেতে পারে। পাসওয়ার্ড কখনো কমান্ড লাইনে লিখবে না — `ps` দিয়েও অন্য ইউজার সেটা দেখতে পারে।

`fc` — Fix Command

`fc` একটা পুরনো (POSIX) কিন্তু দারুণ builtin।

কমান্ড	কাজ
`fc -l`	শেষ ১৬টা কমান্ডের তালিকা
`fc -l -5`	শেষ ৫টা
`fc -l 100 110`	১০০ থেকে ১১০ নম্বর
`fc`	শেষ কমান্ড এডিটরে খোলো, সেভ করলে চলবে
`fc 498`	৪৯৮ নম্বর কমান্ড এডিটরে খোলো
`fc 495 499`	৪৯৫–৪৯৯ সবগুলো এডিটরে, সেভ করলে পরপর চলবে
`fc -s`	শেষ কমান্ড আবার চালাও (`!!`-এর মতো)
`fc -s old=new`	শেষ কমান্ডে old কে new বানিয়ে চালাও (সব জায়গায়)
`fc -s ssh`	`ssh` দিয়ে শুরু হওয়া শেষ কমান্ড চালাও
`fc -e nano`	নির্দিষ্ট এডিটর দিয়ে খোলো

```
$ echo dev dev dev
dev dev dev
$ fc -s dev=prod
echo prod prod prod
prod prod prod
```

৬.১০ History Expansion — `!` এর জাদু

History expansion হলো একটা "টেক্সট-রিপ্লেস" সুবিধা: Enter চাপার পর, কমান্ড চালানোর **আগে**, bash `!` দিয়ে শুরু হওয়া চিহ্নগুলো history থেকে আসল লেখা দিয়ে বদলে দেয় — আর বদলানো কমান্ডটা স্ক্রিনে প্রিন্ট করে দেখায়।

এর তিনটা অংশ: **event designator** (কোন কমান্ড), **word designator** (সেই কমান্ডের কোন শব্দ), **modifier** (শব্দটাকে কীভাবে বদলাবে)। ফরম্যাট: `!event:word:modifier`

ধাপ ১: Event designator — কোন কমান্ড?

চিহ্ন	মানে	উদাহরণ
`!!`	ঠিক আগের কমান্ড	`sudo !!`
`!n`	history-র `n` নম্বর কমান্ড	`!498`
`!-n`	`n` ধাপ আগের কমান্ড	`!-2` = দুই কমান্ড আগে
`!string`	`string` দিয়ে **শুরু** হওয়া সর্বশেষ কমান্ড	`!ssh`
`!?string?`	`string` **আছে** এমন সর্বশেষ কমান্ড	`!?nginx?`
`^old^new`	আগের কমান্ডে `old`-এর **প্রথম** উপস্থিতি `new` দিয়ে বদলে চালাও	`^8080^9090`
`!#`	এখন পর্যন্ত লেখা বর্তমান লাইন	`cp file.txt !#:1.bak`

```
$ cat /etc/ssh/sshd_config
cat: /etc/ssh/sshd_config: Permission denied
$ sudo !!
sudo cat /etc/ssh/sshd_config
# ... ফাইলের লেখা ...
```

```
$ curl http://localhost:8080/health
curl: (7) Failed to connect to localhost port 8080
$ ^8080^9090
curl http://localhost:9090/health
OK
```

ধাপ ২: Word designator — কোন শব্দ?

একটা কমান্ডের শব্দগুলো ০ থেকে গোনা হয়:

```
  cp   -r   photos   /mnt/backup
  ↑    ↑    ↑        ↑
  0    1    2        3 ($)
       ^ (প্রথম argument)
```

চিহ্ন	মানে
`!$`	আগের কমান্ডের **শেষ** শব্দ (`!!:$` -এর সংক্ষেপ)
`!^`	আগের কমান্ডের **প্রথম argument** (শব্দ ১)
`!*`	আগের কমান্ডের **সব argument** (১ থেকে শেষ, কমান্ডের নাম বাদে)
`!:0`	আগের কমান্ডের নাম
`!:2`	আগের কমান্ডের ২ নম্বর শব্দ
`!:1-3`	১ থেকে ৩ নম্বর শব্দ
`!:2*`	২ থেকে শেষ পর্যন্ত (`!:2-$`)
`!:2-`	২ থেকে শেষের আগের শব্দ পর্যন্ত
`!ssh:$`	`ssh` দিয়ে শুরু হওয়া শেষ কমান্ডের শেষ শব্দ

```
$ mkdir /var/www/mysite
$ cd !$
cd /var/www/mysite

$ touch a.txt b.txt c.txt
$ ls -l !*
ls -l a.txt b.txt c.txt

$ cp -r photos /mnt/backup
$ ls !:3
ls /mnt/backup
```

> 💡 **টিপস:** `!$` আর `Alt+.` প্রায় একই কাজ করে। পার্থক্য: `Alt+.` লেখাটা **সাথে সাথে** লাইনে বসায়, তুমি দেখে-শুনে Enter চাপো। `!$` দেখা যায় Enter চাপার পর। নিরাপত্তার জন্য `Alt+.` ভালো।

ধাপ ৩: Modifier — শব্দটা বদলাও

Modifier	কাজ	উদাহরণ (শব্দ = `/home/rahim/song.mp3`)
`:h`	head — শেষ পাথ-অংশ বাদ (dirname)	`/home/rahim`
`:t`	tail — শুধু শেষ অংশ (basename)	`song.mp3`
`:r`	remove extension	`/home/rahim/song`
`:e`	শুধু extension	`.mp3`
`:p`	**শুধু প্রিন্ট করো, চালিও না**	—
`:s/old/new/`	প্রথম `old` কে `new` বানাও	—
`:gs/old/new/`	সব `old` কে `new` বানাও	—
`:&`	আগের substitution আবার করো	—
`:q`	quote করো	`'/home/rahim/song.mp3'`

```
$ ls /home/rahim/song.mp3
/home/rahim/song.mp3
$ cd !$:h
cd /home/rahim
$ ffmpeg -i !-2:$ !-2:$:r.ogg
ffmpeg -i /home/rahim/song.mp3 /home/rahim/song.ogg
```

`^old^new` শুধু **প্রথম** উপস্থিতি বদলায়। সব বদলাতে:

```
$ echo cat cat cat
cat cat cat
$ !!:gs/cat/dog/
echo dog dog dog
dog dog dog
```

নিরাপদে history expansion: `:p` আর `histverify`

`rm` জাতীয় বিপজ্জনক কমান্ডে `!` ব্যবহার করার আগে দেখে নাও:

```
$ !rm:p
rm -rf ./build/*
```

এটা শুধু প্রিন্ট করে (আর history-তে যোগ করে, যাতে `↑` চেপে চালাতে পারো)। আরও ভালো উপায় — `histverify` চালু করো:

```
$ shopt -s histverify
$ sudo !!
$ sudo apt update          ← চলল না! expand হয়ে লাইনে বসল, দেখে Enter চাপো
```

> ⚠️ **সতর্কতা:** `!` বিশেষ চিহ্ন বলে ডাবল কোটের ভেতরে ঝামেলা করে:
>
> ```
> $ echo "Hello World!"            # bash 4.3+ এ ঠিক আছে, কারণ ! এর পরে " আছে
> $ echo "Wow!great"
> bash: !great: event not found
> ```
>
> সমাধান: সিঙ্গেল কোট ব্যবহার করো (`'Wow!great'`) অথবা `\!` লেখো। স্ক্রিপ্টের ভেতরে (non-interactive শেলে) history expansion ডিফল্টভাবে বন্ধ থাকে, তাই সেখানে এই সমস্যা নেই। চাইলে interactive শেলেও বন্ধ করতে পারো: `set +H`।

৬.১১ History কনফিগার করা: HIST* ভেরিয়েবল

এই ভেরিয়েবলগুলো সাধারণত `~/.bashrc`-এ সেট করা হয় (`.bashrc` নিয়ে বিস্তারিত অধ্যায় ৪৪-এ)। Ubuntu-র ডিফল্ট `~/.bashrc`-এ আগে থেকেই কয়েকটা আছে:

```
$ grep -i hist ~/.bashrc
# don't put duplicate lines or lines starting with space in the history.
HISTCONTROL=ignoreboth
# append to the history file, don't overwrite it
shopt -s histappend
# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000
```

ভেরিয়েবলের তালিকা

ভেরিয়েবল	কাজ	ডিফল্ট
`HISTFILE`	history ফাইলের পাথ	`~/.bash_history`
`HISTSIZE`	**মেমোরিতে** কতগুলো কমান্ড থাকবে	৫০০ (Ubuntu-র `.bashrc`-এ ১০০০)
`HISTFILESIZE`	**ফাইলে** সর্বোচ্চ কত লাইন থাকবে	`HISTSIZE`-এর মান (Ubuntu-তে ২০০০)
`HISTCONTROL`	কোন কমান্ড সেভ হবে না	সেট নেই (Ubuntu-তে `ignoreboth`)
`HISTIGNORE`	কোন প্যাটার্নের কমান্ড বাদ যাবে	সেট নেই
`HISTTIMEFORMAT`	সময় দেখানোর ফরম্যাট (সেট থাকলে timestamp সেভ হয়)	সেট নেই

> 📝 **মনে রাখো:** `HISTSIZE` বা `HISTFILESIZE`-এ ঋণাত্মক মান (যেমন `-1`) দিলে **সীমাহীন** (bash 4.3+)। আর `HISTSIZE=0` দিলে history পুরোপুরি বন্ধ।

`HISTCONTROL`-এর মানগুলো

মান	কাজ
`ignorespace`	**স্পেস দিয়ে শুরু** হওয়া কমান্ড সেভ হবে না
`ignoredups`	ঠিক আগের কমান্ডের সাথে হুবহু এক হলে সেভ হবে না
`ignoreboth`	ওপরের দুটোই
`erasedups`	নতুন কমান্ড সেভের আগে history থেকে তার সব পুরনো কপি মুছে ফেলো

একাধিক মান কোলন দিয়ে: `HISTCONTROL=ignoreboth:erasedups`

`ignorespace` চালু থাকলে গোপন কমান্ড এভাবে চালাও (শুরুতে একটা স্পেস):

```
$  export API_TOKEN=abc123secret
$ history 2
  701  history 2          ← export কমান্ডটা নেই!
```

`HISTIGNORE` — প্যাটার্ন দিয়ে বাদ দাও

কোলন দিয়ে আলাদা করা glob প্যাটার্ন। পুরো কমান্ড লাইনটা প্যাটার্নের সাথে মিলতে হবে।

```
HISTIGNORE="ls:ll:cd:pwd:exit:clear:history*"
```

এখন শুধু `ls` বা `cd` সেভ হবে না, কিন্তু `ls -la /etc` সেভ হবে (কারণ পুরো লাইন `ls`-এর সাথে মেলে না)। `ls *` লিখলে `ls` দিয়ে শুরু হওয়া সব বাদ।

`HISTTIMEFORMAT` — কখন কোন কমান্ড চালিয়েছিলাম?

সার্ভারে সমস্যা হলে প্রায়ই প্রশ্ন ওঠে "কে, কখন এটা চালিয়েছিল?" `HISTTIMEFORMAT` সেট করলে bash প্রতিটা কমান্ডের সাথে সময় (Unix timestamp, `#` দিয়ে শুরু লাইনে) ফাইলে সেভ করে, আর `history` সেটা দেখায়। ফরম্যাট `strftime`-এর মতো (`date` কমান্ডের `%` কোডের মতো)।

```
$ export HISTTIMEFORMAT="%F %T  "
$ history 3
  702  2026-11-12 10:15:32  sudo systemctl restart nginx
  703  2026-11-12 10:16:01  tail -f /var/log/nginx/error.log
  704  2026-11-12 10:17:45  history 3
```

কোড	মানে	উদাহরণ
`%F`	তারিখ (`%Y-%m-%d`)	`2026-11-12`
`%T`	সময় (`%H:%M:%S`)	`10:15:32`
`%d/%m/%y`	দিন/মাস/বছর	`12/11/26`

> 📝 **মনে রাখো:** `HISTTIMEFORMAT` সেট করার **আগে** চালানো কমান্ডের timestamp ছিল না, তাই সেগুলোতে ফাইলটা প্রথম পড়ার সময়টাই দেখাবে — ভুল তথ্য মনে কোরো না।

একাধিক টার্মিনালে history শেয়ার করা

ডিফল্টভাবে শেল বন্ধ হলে history ফাইলে লেখা হয়, আর `histappend` না থাকলে শেষ যে শেল বন্ধ হয় সে ফাইলটা **overwrite** করে — অন্য টার্মিনালের কমান্ড হারিয়ে যায়। সমাধান:

```
# ~/.bashrc
shopt -s histappend                 # overwrite না করে append করো
PROMPT_COMMAND="history -a${PROMPT_COMMAND:+; $PROMPT_COMMAND}"
```

`PROMPT_COMMAND` প্রতিবার প্রম্পট দেখানোর আগে চলে। `history -a` প্রতিটা কমান্ডের পরপরই ফাইলে লিখে দেয় — ক্র্যাশ হলেও হারায় না। অন্য টার্মিনালের কমান্ড এই টার্মিনালে আনতে চাইলে `history -n` চালাও (অথবা

`PROMPT_COMMAND`-এ `history -a; history -n` দাও, তবে তাতে ↑ চাপলে অন্য টার্মিনালের কমান্ড মিশে যায় — কারো ভালো লাগে, কারো না)।

একটা ভালো সেটআপ (~/.bashrc)

```
# ===== History সেটিং =====
HISTSIZE=50000
HISTFILESIZE=100000
HISTCONTROL=ignoreboth:erasedups
HISTIGNORE="ls:ll:pwd:exit:clear:history"
HISTTIMEFORMAT="%F %T  "
shopt -s histappend      # append করো
shopt -s cmdhist         # মাল্টি-লাইন কমান্ড এক এন্ট্রিতে সেভ করো (ডিফল্ট চালু)
shopt -s histverify      # !! ইত্যাদি expand করে দেখাও, সাথে সাথে চালিও না
PROMPT_COMMAND="history -a${PROMPT_COMMAND:+; $PROMPT_COMMAND}"
stty -ixon 2>/dev/null   # Ctrl+S দিয়ে forward search চালু
```

পরিবর্তন সক্রিয় করতে: `source ~/.bashrc`

> ⚠️ **সতর্কতা:** root ইউজারের আলাদা history থাকে (`/root/.bash_history`)। `sudo -i` বা `sudo su` দিয়ে root হলে যা চালাবে, তা তোমার নিজের history-তে যাবে না।

৬.১২ `Ctrl+R` — Reverse Search বিস্তারিত

এটা টার্মিনালের সবচেয়ে শক্তিশালী শর্টকাট। "আমার মনে আছে কমান্ডে `docker` আর `postgres` ছিল, কিন্তু পুরোটা মনে নেই" — `Ctrl+R` এর জন্যই।

কীভাবে কাজ করে

1. `Ctrl+R` চাপো। প্রম্পট বদলে যাবে:

   ```
   (reverse-i-search)`':
   ```

2. টাইপ করা শুরু করো। প্রতিটা অক্ষরের সাথে সাথে (incremental — তাই নামে "i") সর্বশেষ মিলে যাওয়া কমান্ড দেখাবে:

   ```
   (reverse-i-search)`post': docker run -d --name db -e POSTGRES_PASSWORD=x
   postgres:16
   ```

3. মিলে যাওয়া অংশ শব্দের **যেকোনো জায়গায়** থাকতে পারে, শুরুতে হওয়া জরুরি না।

Search চলাকালীন কী কী চাপা যায়

কী	কাজ
আরও অক্ষর টাইপ	খোঁজা আরও নির্দিষ্ট করো
`Ctrl+R` আবার	একই শব্দের **আরও পুরনো** মিল দেখাও
`Ctrl+S`	উল্টো দিকে — **নতুনতর** মিল (`stty -ixon` লাগবে)
`Backspace`	খোঁজার শব্দ থেকে শেষ অক্ষর মোছো
`Enter`	পাওয়া কমান্ড **সাথে সাথে চালাও**
`Esc` , `←` , `→` , `Ctrl+A` , `Ctrl+E` ইত্যাদি	খোঁজা থামাও, কমান্ডটা লাইনে রেখে **এডিট করার সুযোগ দাও**
`Ctrl+G`	খোঁজা বাতিল, আগের (খালি) লাইনে ফেরো
`Ctrl+C`	খোঁজা ও লাইন দুটোই বাতিল

```
তুমি টাইপ করছ:   Ctrl+R  →  "ssh"
                        |
    history (নতুন থেকে পুরনো দিকে খোঁজে)
    ┌──────────────────────────────────────┐
    | 905  ls                              |
    | 904  ssh deploy@10.0.0.7   ◄── ১ম মিল |
    | 903  vim app.py                      |
    | 870  ssh -p 2222 rahim@vps ◄── Ctrl+R আবার: ২য় মিল |
    | ...                                  |
    └──────────────────────────────────────┘
```

খুব কাজের কৌশল

1. **কমান্ডে "ট্যাগ" দাও:** লম্বা কমান্ডের শেষে কমেন্ট লিখে রাখো:

   ```
   $ rsync -avz --delete ~/site/ deploy@vps:/var/www/site/  # deploy-site
   ```

   পরে `Ctrl+R` চেপে `deploy-site` লিখলেই পেয়ে যাবে।

2. **পেয়ে এডিট করো, তারপর চালাও:** `Enter` না চেপে `→` বা `Ctrl+E` চাপো, বদলাও, তারপর Enter।

3. **`Ctrl+O` দিয়ে সিরিজ চালাও:** `Ctrl+R` দিয়ে কোনো পুরনো কমান্ড খুঁজে `Ctrl+O` চাপলে সেটা চলবে **এবং** history-তে তার পরের কমান্ডটা লাইনে চলে আসবে। আবার `Ctrl+O` — তার পরেরটা। একটা পুরনো কাজের ধারাবাহিকতা হুবহু আবার চালাতে চমৎকার।

4. **ইতিহাসে prefix-ভিত্তিক খোঁজা Up-arrow দিয়ে** (অনেকের প্রিয়) — `~/.inputrc` -এ:

   ```
   "\e[A": history-search-backward
   "\e[B": history-search-forward
   ```

   এখন `git` লিখে `↑` চাপলে শুধু `git` দিয়ে **শুরু** হওয়া কমান্ডগুলোই ঘুরবে।

> 💡 **টিপস:** আরও আধুনিক অভিজ্ঞতা চাইলে `fzf` টুল (Ubuntu/Debian/Fedora-র রিপোজিটরিতে আছে) ইনস্টল করে তার bash ইন্টিগ্রেশন চালু করতে পারো — তখন `Ctrl+R` fuzzy খোঁজার একটা তালিকা দেখায়। তবে আগে বিল্ট-ইন `Ctrl+R` ভালোভাবে শেখো, কারণ যেকোনো সার্ভারে এটাই পাবে।

৬.১৩ Terminal Emulator-এর শর্টকাট

এগুলো শেলের নয়, GUI টার্মিনাল অ্যাপের নিজস্ব। ডেস্কটপ পরিবেশ নিয়ে বিস্তারিত অধ্যায় ৪-এ।

কেন `Ctrl+Shift`?

কারণ সাধারণ `Ctrl+অক্ষর` কম্বিনেশনগুলো (`Ctrl+C`, `Ctrl+V`, `Ctrl+W`...) শেল ও প্রোগ্রামের জন্য সংরক্ষিত — ৬.২ অংশের ডায়াগ্রাম মনে করো। টার্মিনাল যদি `Ctrl+C`-কে "কপি" বানাত, তাহলে কোনো প্রোগ্রাম থামানো যেত না। তাই টার্মিনাল অ্যাপগুলো `Shift` যোগ করে নিজের কাজ করে।

GNOME Terminal / Ptyxis (Ubuntu, Fedora Workstation) — সাধারণ ডিফল্ট

শর্টকাট	কাজ
`Ctrl+Alt+T`	নতুন টার্মিনাল খোলো (Ubuntu ডেস্কটপের সিস্টেম শর্টকাট)
`Ctrl+Shift+C`	কপি
`Ctrl+Shift+V`	পেস্ট
`Shift+Insert`	পেস্ট (প্রায় সব টার্মিনালে)
`Ctrl+Shift+T`	নতুন ট্যাব
`Ctrl+Shift+N`	নতুন উইন্ডো
`Ctrl+Shift+W`	বর্তমান ট্যাব বন্ধ
`Ctrl+Shift+Q`	পুরো উইন্ডো বন্ধ
`Ctrl+Page Up` / `Ctrl+Page Down`	আগের/পরের ট্যাবে যাও
`Ctrl+Shift+Page Up` / `Page Down`	ট্যাবকে বামে/ডানে সরাও
`Alt+1` ... `Alt+9`	নির্দিষ্ট নম্বরের ট্যাবে যাও (GNOME Terminal)
`Ctrl+Shift+F`	টার্মিনালের লেখার মধ্যে খোঁজো (Find)
`Ctrl++` (`Ctrl+Shift+=`)	লেখা বড় করো (zoom in)
`Ctrl+-`	লেখা ছোট করো
`Ctrl+0`	স্বাভাবিক সাইজ
`F11`	ফুলস্ক্রিন

শর্টকাট	কাজ
`Shift+Page Up` / `Shift+Page Down`	scrollback-এ ওপরে/নিচে স্ক্রল

Konsole (KDE) — কিছু পার্থক্য

শর্টকাট	কাজ
`Ctrl+Shift+T`	নতুন ট্যাব
`Ctrl+Shift+N`	নতুন উইন্ডো
`Shift+←` / `Shift+→`	আগের/পরের ট্যাব
`Ctrl+(` / `Ctrl+)`	উইন্ডো উপরে-নিচে/পাশাপাশি ভাগ (split view)
`Ctrl+Shift+F`	খোঁজো

> 📝 **মনে রাখো:** শর্টকাটগুলো অ্যাপ ও ভার্সনভেদে একটু আলাদা হতে পারে। সঠিক তালিকা সবসময় টার্মিনালের **Preferences** → **Shortcuts** (বা Keyboard Shortcuts) মেনুতে দেখা যায়, আর সেখান থেকে বদলানোও যায়। Ubuntu 25.10 ও সাম্প্রতিক Fedora-তে ডিফল্ট টার্মিনাল **Ptyxis**; Ubuntu 24.04-এ এখনো GNOME Terminal।

লিনাক্সের "দুটো ক্লিপবোর্ড"

লিনাক্সে (X11 ও বেশিরভাগ Wayland ডেস্কটপে) কপি-পেস্টের দুটো আলাদা ব্যবস্থা আছে:

ব্যবস্থা	কপি	পেস্ট
CLIPBOARD (সাধারণ)	`Ctrl+Shift+C` (টার্মিনালে) / `Ctrl+C` (অন্য অ্যাপে)	`Ctrl+Shift+V` / `Ctrl+V`
PRIMARY selection	শুধু মাউস দিয়ে **সিলেক্ট** করলেই কপি	মাউসের **মাঝের বাটন** (চাকা) ক্লিক

> 💡 **টিপস:** টার্মিনালে কোনো শব্দে **ডাবল-ক্লিক** করলে শব্দটা সিলেক্ট হয়, **ট্রিপল-ক্লিক** করলে পুরো লাইন। তারপর মাঝের বাটনে ক্লিক — পেস্ট! কোনো কী চাপতেই হলো না।

> ⚠️ **সতর্কতা:** ওয়েবসাইট থেকে কমান্ড কপি করে সরাসরি টার্মিনালে পেস্ট করা বিপজ্জনক — পেজে লুকানো লেখা (যেমন একটা newline সহ খারাপ কমান্ড) থাকতে পারে যা পেস্ট হওয়া মাত্র চলে যায়। আধুনিক bash (5.1+) ও readline ডিফল্টভাবে **bracketed paste** চালু রাখে, যাতে পেস্ট করা লেখা Enter না চাপা পর্যন্ত চলে না। তবুও অপরিচিত উৎসের কমান্ড আগে কোনো টেক্সট এডিটরে পেস্ট করে পড়ে নাও।

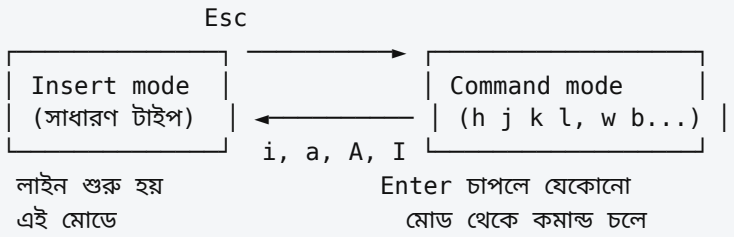
৬.১৪ Emacs Mode বনাম Vi Mode

এতক্ষণ যা শিখলে সব **emacs mode**-এর শর্টকাট — bash-এর ডিফল্ট। যারা Vim এডিটরে (অধ্যায় ১৪) অভ্যস্ত, তারা কমান্ড লাইনেও Vim-এর মতো এডিট করতে পারে।

```
$ set -o vi        # vi mode চালু
$ set -o emacs     # আবার emacs mode-এ ফেরো
$ set -o | grep -E '^(emacs|vi) '
emacs           off
vi              on
```

Vi mode কীভাবে কাজ করে

Vi mode-এ দুটো অবস্থা (mode) আছে:



Vi command mode-এর প্রধান কী

কী	কাজ
`Esc`	command mode-এ যাও
`i` / `a`	কার্সরের আগে / পরে insert
`I` / `A`	লাইনের শুরুতে / শেষে insert
`h` / `l`	এক অক্ষর বামে / ডানে
`w` / `b` / `e`	পরের শব্দ / আগের শব্দ / শব্দের শেষ
`0` / `^` / `$`	লাইনের শুরু / প্রথম অক্ষর / শেষ
`k` / `j`	history-তে আগের / পরের কমান্ড
`/text` তারপর Enter	history-তে পেছনের দিকে খোঁজো
`n` / `N`	পরের / উল্টো দিকের মিল
`x`	কার্সরের নিচের অক্ষর মোছো
`dw` / `db` / `dd` / `D`	শব্দ মোছো / আগের শব্দ / পুরো লাইন / শেষ পর্যন্ত
`cw` / `cc` / `C`	শব্দ / লাইন / শেষ পর্যন্ত বদলাও (মুছে insert)
`r` তারপর অক্ষর	একটা অক্ষর প্রতিস্থাপন
`~`	বড়/ছোট হাতের অক্ষর উল্টাও

কী	কাজ
`u`	undo
`p` / `P`	কার্সরের পরে / আগে পেস্ট
`.`	শেষ পরিবর্তন আবার করো
`v`	বর্তমান লাইন এডিটরে খোলো (emacs mode-এর `Ctrl+X Ctrl+E`-এর মতো)
`fX` / `FX`	সামনে / পেছনে অক্ষর `X`-এ লাফ দাও

> 📝 **মনে রাখো:** vi mode-এ `Ctrl+A`, `Ctrl+E`, `Ctrl+R`-এর মতো emacs শর্টকাট ডিফল্টভাবে **কাজ করে না** (insert mode-এ কয়েকটা — যেমন `Ctrl+W`, `Ctrl+U` — tty-র কারণে কাজ করে)। তাই অনেকে vi mode চালু করে কিছু emacs শর্টকাট `~/.inputrc`-এ যোগ করে নেয় (নিচে দেখো)। তবে `Ctrl+C`, `Ctrl+Z`, `Ctrl+D` কার্নেল/tty-র বলে দুই মোডেই একই।

কোনটা বেছে নেবে?

দিক	Emacs mode	Vi mode
ডিফল্ট?	হ্যাঁ, সব সার্ভারে	না, চালু করতে হয়
শেখা	সহজ — টাইপ করো আর Ctrl চাপো	Vim জানলে স্বাভাবিক, না জানলে কঠিন
সার্ভারে কাজ	অন্যের মেশিনেও একই	প্রতিবার `set -o vi` লাগবে
পরামর্শ	**নতুনদের জন্য এটাই**	Vim দক্ষতা আসার পর চেষ্টা করো

স্থায়ীভাবে vi mode চাইলে `~/.bashrc`-এ `set -o vi`, অথবা `~/.inputrc`-এ `set editing-mode vi` (দ্বিতীয়টা readline-ব্যবহারকারী সব প্রোগ্রামে প্রযোজ্য)।

৬.১৫ `~/.inputrc` — Readline-কে নিজের মতো সাজাও

Readline চালু হওয়ার সময় একটা কনফিগারেশন ফাইল পড়ে:

1. `$INPUTRC` ভেরিয়েবলে দেওয়া ফাইল, না থাকলে
2. `~/.inputrc`, না থাকলে
3. `/etc/inputrc` (সিস্টেম-ব্যাপী)

> ⚠️ **সতর্কতা:** তুমি `~/.inputrc` বানালে readline আর `/etc/inputrc` পড়বে **না**। ফলে distro-র দেওয়া সুবিধা (যেমন `Ctrl+←/→` দিয়ে শব্দ লাফানো, `Home` / `End` কী) হারিয়ে যেতে পারে। তাই নিজের ফাইলের প্রথম লাইনে সবসময় লেখো:

```
$include /etc/inputrc
```

ফাইলের গঠন

তিন রকম লাইন থাকে:

```
# ১. কমেন্ট — # দিয়ে শুরু

# ২. ভেরিয়েবল সেট:  set নাম মান
set completion-ignore-case on

# ৩. কী-বাইন্ডিং:  "কী-সিকোয়েন্স": ফাংশন-নাম   অথবা   "কী": "ম্যাক্রো লেখা"
"\C-x\C-r": re-read-init-file
"\e[A": history-search-backward
```

কী লেখার নিয়ম: `\C-` = Ctrl, `\M-` বা `\e` = Meta/Alt/Esc। যেমন `"\C-a"`, `"\M-f"`, `"\e."`।

একটা কাজের `~/.inputrc` উদাহরণ

```
$include /etc/inputrc

# ---- Completion ----
set completion-ignore-case on        # Tab-এ বড়-ছোট হাতের পার্থক্য উপেক্ষা: doc<Tab> → Documents/
set completion-map-case on           # - আর _ কে সমান ধরো
set show-all-if-ambiguous on         # একবার Tab-এই সব অপশন দেখাও (double Tab লাগবে না)
set colored-stats on                 # completion তালিকা ls-এর মতো রঙিন
set visible-stats on                 # ফাইলের ধরন চিহ্ন (/ * @) দেখাও
set mark-symlinked-directories on    # symlink ডিরেক্টরির পরে / বসাও
set colored-completion-prefix on     # মিলে যাওয়া অংশ রঙিন করো
set completion-query-items 200       # কতগুলোর বেশি হলে "Display all?" জিজ্ঞেস করবে

# ---- অন্যান্য ----
set bell-style none                  # বিপ বন্ধ (visible দিলে স্ক্রিন ঝলকাবে)
set enable-bracketed-paste on        # পেস্ট নিরাপদ রাখো

# ---- History খোঁজা ↑/↓ দিয়ে (লেখা শুরুর সাথে মিলিয়ে) ----
"\e[A": history-search-backward
"\e[B": history-search-forward

# ---- শুধু bash-এর জন্য ----
$if Bash
  # Alt+S চাপলে লাইনের শুরুতে "sudo " বসাও (ম্যাক্রো)
  "\es": "\C-asudo \C-e"
$endif
```

শেষের `$if Bash ... $endif` অংশটা শুধু bash-এ কাজ করবে, `python3` বা `mysql`-এ না। `$if mode=vi` দিয়ে শুধু vi mode-এর জন্যও বাইন্ডিং দেওয়া যায়।

Vi mode ব্যবহারকারীদের জন্য উপকারী সংযোজন:

```
set editing-mode vi
set show-mode-in-prompt on          # প্রম্পটের আগে (ins)/(cmd) দেখাও
$if mode=vi
  set keymap vi-insert
  "\C-l": clear-screen
  "\C-a": beginning-of-line
  "\C-e": end-of-line
$endif
```

পরিবর্তন কার্যকর করা

- নতুন টার্মিনাল খোলো, **অথবা**
- বর্তমান শেলে `Ctrl+X Ctrl+R` চাপো (`re-read-init-file`), **অথবা**
- `bind -f ~/.inputrc`

`bind` কমান্ড — বর্তমান বাইন্ডিং দেখা ও বদলানো

`bind` হলো bash-এর builtin যা দিয়ে readline-এর সেটিং দেখা ও বদলানো যায়।

কমান্ড	কাজ
`bind -P`	সব ফাংশন ও তাদের কী-বাইন্ডিং পড়ার মতো করে দেখাও
`bind -p`	ওই তালিকা `.inputrc` ফরম্যাটে দেখাও
`bind -l`	সব readline ফাংশনের নাম
`bind -v`	সব ভেরিয়েবল ও তাদের মান
`bind -q ফাংশন`	ফাংশনটা কোন কী-তে বাঁধা
`bind -u ফাংশন`	ফাংশনের সব বাইন্ডিং সরাও
`bind -r "কী"`	একটা কী-এর বাইন্ডিং সরাও
`bind '"কী": ফাংশন'`	নতুন বাইন্ডিং (এই সেশনে)
`bind -x '"কী": শেল-কমান্ড'`	কী চাপলে একটা শেল কমান্ড চালাও
`bind -f ফাইল`	একটা inputrc ফাইল পড়ো

```
$ bind -q yank-last-arg
yank-last-arg can be invoked via "\e.", "\e_".

$ bind -P | grep -w kill-line
kill-line can be invoked via "\C-k".

$ bind -x '"\C-xt": date'      # Ctrl+X তারপর T চাপলে তারিখ দেখাবে
$ <Ctrl+X t>
Thu Nov 12 10:30:00 AM +06 2026
```

একটা কী কোন সিকোয়েন্স পাঠায় জানব কীভাবে?

`Ctrl+V` চেপে তারপর কী-টা চাপো (`quoted-insert`), বা `cat` চালিয়ে কী চাপো:

```
$ cat
^[[A        ← ↑ কী (^[ মানে Esc, তাই "\e[A")
^[[1;5C     ← Ctrl+→
^C
```

এই সিকোয়েন্সই `.inputrc` -এ লেখো: `"\e[1;5C": forward-word` ।

হাতে-কলমে অনুশীলন

একটা টার্মিনাল খোলো (`Ctrl+Alt+T`)। প্রতিটা ল্যাব নিজে টাইপ করে করো — পড়ে শেখা যায় না, আঙুলে শিখতে হয়।

ল্যাব ১: কার্সর আর মোছা

1. টাইপ করো (Enter চাপবে না):

   ```
   echo one two three four five
   ```

2. `Ctrl+A` — শুরুতে। `Ctrl+E` — শেষে।
3. `Alt+B` তিনবার চাপো — কার্সর `three` -এর শুরুতে।
4. `Alt+D` — `three` মুছে গেল। `Ctrl+Y` — ফিরে এল।
5. `Ctrl+E` , তারপর `Ctrl+W` — `five` গেল। `Ctrl+W` আবার — `four` গেল।
6. `Ctrl+Y` — দেখো কী আসে (শুধু শেষের kill, নাকি দুটো একসাথে?)। পরপর kill করলে readline সেগুলো kill ring-এর একই এন্ট্রিতে জুড়ে দেয়, তাই `four five` একসাথে ফেরত আসবে।
7. `Ctrl+_` কয়েকবার — undo।
8. `Ctrl+U` — পুরো লাইন গেল। `Ctrl+Y` — ফিরে এল। Enter।

ল্যাব ২: অক্ষর বদল

1. টাইপ করো: `echo hello world`
2. `Alt+B` (world-এর শুরু), `Alt+U` → `WORLD` ।
3. `Alt+B` দুবার (hello), `Alt+C` → `Hello` ।
4. `Ctrl+E` , তারপর `Alt+T` → শেষ দুটো শব্দ অদলবদল। ফলাফল দেখো, Enter।

ল্যাব ৩: History expansion

```
$ mkdir -p ~/lab6/a/b/c
$ cd !$
$ pwd
$ touch file1.txt file2.txt file3.txt
$ ls -l !*
$ echo ~/lab6/report.final.pdf
$ echo !$:t
$ echo !-2:$:r
$ echo red green blue
$ ^green^yellow
$ !!:gs/e/E/
$ !mkdir:p
$ history 12
```

প্রতিটা লাইনের পর bash কী expand করে প্রিন্ট করল, খেয়াল করো। শেষ লাইনে `history 12`-এ নম্বরগুলো দেখে `!<নম্বর>` দিয়ে একটা কমান্ড আবার চালাও।

ল্যাব ৪: Ctrl+R

1. এই কমান্ডটা চালাও: `echo "my secret project location" # proj-tag`
2. কয়েকটা অন্য কমান্ড চালাও: `ls`, `date`, `whoami`।
3. `Ctrl+R` চাপো, `proj-tag` লেখো। কমান্ডটা দেখা দেবে।
4. `→` চাপো (চালাবে না) — `secret` শব্দটা বদলে `public` করো, Enter।
5. আবার `Ctrl+R`, `echo` লেখো, তারপর `Ctrl+R` বারবার চাপো — পুরনো echo-গুলো ঘুরে দেখো। `Ctrl+G` দিয়ে বাতিল করো।

ল্যাব ৫: History কনফিগার

```
$ cp ~/.bashrc ~/.bashrc.backup-ch6        # আগে ব্যাকআপ!
$ nano ~/.bashrc                           # একদম শেষে ৬.১১-এর "ভালো সেটআপ" অংশ যোগ
করো
$ source ~/.bashrc
$  echo "this has a leading space"
$ ls
$ ls
$ history 5                                # leading-space কমান্ড নেই, ls একবারই
```

এখন দুটো টার্মিনাল খোলো। প্রথমটায় `echo from-terminal-1` চালাও। দ্বিতীয়টায় `history -n` চালিয়ে তারপর `Ctrl+R` দিয়ে `terminal-1` খোঁজো — পেয়ে যাবে।

ল্যাব ৬: Tab completion ও inputrc

1. `ls /usr/share/doc/` লিখে Tab Tab — তালিকা দেখো।
2. `cd ~/doc` লিখে Tab চাপো — `Documents` বড় হাতের `D`, তাই কিছু হবে না।

3. `~/.inputrc` বানাও:

```
$ printf '$include /etc/inputrc\nset completion-ignore-case on\nset show-all-if-ambiguous on\n' > ~/.inputrc
```

4. `Ctrl+X Ctrl+R` চাপো। এখন আবার `cd ~/doc<Tab>` — `Documents/` complete হবে!
5. `bind -v | grep completion-ignore-case` চালিয়ে যাচাই করো।

ল্যাব ৭: Vi mode এক্সপেরিমেন্ট

```
$ set -o vi
$ echo vi mode is fun
```

1. `↑` চেপে কমান্ডটা আনো। `Esc` চাপো।
2. `0` (শুরু), `w` দুবার (`mode`), `cw` চেপে `editing` লেখো, `Esc`।
3. `$` (শেষে), `A` চেপে `indeed` লেখো, Enter।
4. `Esc`, তারপর `/vi` লিখে Enter — history খোঁজা।
5. শেষে `set -o emacs` দিয়ে ফেরো।

ল্যাব ৮: ফ্রিজ রহস্য

1. `Ctrl+S` চাপো। এখন `ls` লিখে Enter চাপো — কিছু দেখা যাচ্ছে না!
2. `Ctrl+Q` চাপো — `ls` চলে আউটপুট দেখা দেবে।
3. `stty -ixon` চালাও, তারপর `Ctrl+R` দিয়ে কিছু খুঁজে `Ctrl+S` চাপো — এবার forward search কাজ করছে।

পরিষ্কার করা: `rm -r ~/lab6`

সারসংক্ষেপ

সবচেয়ে জরুরি ২০টা

শর্টকাট / কমান্ড	কাজ
`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষ
`Alt+B` / `Alt+F`	এক শব্দ পেছনে / সামনে
`Ctrl+U` / `Ctrl+K`	শুরু পর্যন্ত / শেষ পর্যন্ত কাটো
`Ctrl+W` / `Alt+D`	আগের শব্দ / পরের শব্দ কাটো
`Alt+Backspace`	আগের শব্দ কাটো (`/`-এ থামে)
`Ctrl+Y` / `Alt+Y`	পেস্ট / kill ring ঘোরাও

শর্টকাট / কমান্ড	কাজ
`Ctrl+_`	undo
`Ctrl+T` / `Alt+T`	অক্ষর / শব্দ অদলবদল
`Alt+U` / `Alt+L` / `Alt+C`	UPPER / lower / Capitalize
`Ctrl+L`	স্ক্রিন পরিষ্কার
`Ctrl+C`	প্রোগ্রাম থামাও (SIGINT) / লাইন বাতিল
`Ctrl+D`	EOF — খালি লাইনে শেল থেকে বেরোও
`Ctrl+Z`	প্রোগ্রাম suspend (`fg` দিয়ে ফেরত)
`Ctrl+S` / `Ctrl+Q`	আউটপুট ফ্রিজ / চালু
`Tab` / `Tab Tab`	complete / সব অপশন দেখাও
`Ctrl+R`	history-তে খোঁজো
`Ctrl+G`	খোঁজা বাতিল
`Alt+.`	আগের কমান্ডের শেষ argument
`Ctrl+X Ctrl+E`	লাইন এডিটরে খোলো
`Alt+#`	লাইন কমেন্ট করে history-তে রাখো

History expansion

চিহ্ন	মানে
`!!`	আগের কমান্ড
`!n` / `!-n`	n নম্বর / n ধাপ আগের কমান্ড
`!str` / `!?str?`	str দিয়ে শুরু / str আছে এমন শেষ কমান্ড
`!$` / `!^` / `!*`	শেষ / প্রথম / সব argument
`!:n` / `!:n-m`	n নম্বর শব্দ / রেঞ্জ
`^old^new`	প্রথম old → new করে আগের কমান্ড চালাও
`!!:gs/old/new/`	সব old → new
`:h :t :r :e :p`	dirname / basename / extension বাদ / শুধু extension / শুধু প্রিন্ট

কমান্ড ও সেটিং

কমান্ড / ভেরিয়েবল	কাজ
`history [n]`, `history -c/-d/-a/-n/-w`	history দেখা ও ব্যবস্থাপনা
`fc -l`, `fc`, `fc -s old=new`	তালিকা / এডিটরে খোলো / বদলে চালাও
`HISTSIZE`, `HISTFILESIZE`	মেমোরি / ফাইলে কত কমান্ড

কমান্ড / ভেরিয়েবল	কাজ
`HISTCONTROL=ignoreboth:erasedups`	স্পেস-শুরু ও ডুপ্লিকেট বাদ
`HISTIGNORE`, `HISTTIMEFORMAT`	প্যাটার্ন বাদ / timestamp
`shopt -s histappend histverify`	append করো / expand করে যাচাই
`set -o vi` / `set -o emacs`	এডিটিং মোড বদল
`~/.inputrc` + `$include /etc/inputrc`	readline কনফিগ
`bind -P`, `bind -q`, `bind -x`	বাইন্ডিং দেখো / খোঁজো / কমান্ড বাঁধো
`stty -a`, `stty -ixon`	tty কী দেখো / Ctrl+S ফ্রিজ বন্ধ
`Ctrl+Shift+C/V/T/N/W`	টার্মিনাল: কপি / পেস্ট / ট্যাব / উইন্ডো / ট্যাব বন্ধ

নিজেকে যাচাই করো

1. টার্মিনালে `Ctrl+C` চাপলে কেন লেখা কপি হয় না? কপি করতে কী চাপবে?
2. `Ctrl+W` আর `Alt+Backspace`-এর মধ্যে পার্থক্য কী? `cd /etc/nginx/sites-available` লাইনের শেষে দুটো চাপলে কী কী মুছবে?
3. তুমি `apt update` চালালে, "Permission denied" এলো। সবচেয়ে কম টাইপে কীভাবে sudo দিয়ে আবার চালাবে?
4. হঠাৎ টার্মিনালে কিছু টাইপ করলে দেখা যাচ্ছে না। সম্ভাব্য কারণ কী আর সমাধান কী?
5. `HISTSIZE` আর `HISTFILESIZE`-এর পার্থক্য কী?
6. একটা কমান্ড চালাতে চাও যেটা history-তে সেভ হবে না। কোন সেটিং থাকলে কীভাবে করবে?
7. `Ctrl+R` দিয়ে কমান্ড পেলে, সেটা না চালিয়ে এডিট করতে কী চাপবে? আর খোঁজা পুরো বাতিল করতে?
8. `~/.inputrc` বানানোর পর `Ctrl+←` কাজ করা বন্ধ করে দিল। কেন, আর কীভাবে ঠিক করবে?
9. `!$` আর `Alt+.`-এর মধ্যে মূল পার্থক্য কী? কোনটা বেশি নিরাপদ?
10. `ls /tmp/report.tar.gz` চালানোর ঠিক পরে `echo !$:r` চালালে কী দেখাবে? আর তার ঠিক পরে `echo !$:t` চালালে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. টার্মিনালে `Ctrl+C` বাইট `0x03` পাঠায়, যা কার্নেলের tty line discipline ধরে চলমান প্রোগ্রামকে `SIGINT` signal পাঠায় — প্রোগ্রাম থামানোর জন্য এটা সংরক্ষিত। কপি করতে `Ctrl+Shift+C` (পেস্ট `Ctrl+Shift+V`), অথবা মাউসে সিলেক্ট করে মাঝের বাটনে পেস্ট।
2. `Ctrl+W` স্পেস পর্যন্ত মোছে, তাই পুরো `/etc/nginx/sites-available` মুছে যাবে। `Alt+Backspace` শুধু alphanumeric শব্দ মোছে — প্রথমবার `available` মুছবে (`-` এ থামবে), থাকবে `cd /etc/nginx/sites-`।
3. `sudo !!` (`histverify` চালু থাকলে একবার বাড়তি Enter লাগবে)। বিকল্প: `↑`, `Ctrl+A`, `sudo` লিখে Enter।
</details>

4. সম্ভবত ভুলে `Ctrl+S` চাপা হয়েছে, যা XON/XOFF flow control দিয়ে আউটপুট ফ্রিজ করে। `Ctrl+Q` চাপলে ঠিক হবে। স্থায়ীভাবে এড়াতে `~/.bashrc`-এ `stty -ixon`।
5. `HISTSIZE` = মেমোরিতে (বর্তমান সেশনে) কতগুলো কমান্ড থাকবে; `HISTFILESIZE` = `~/.bash_history` ফাইলে সর্বোচ্চ কত লাইন রাখা হবে।
6. `HISTCONTROL`-এ `ignorespace` বা `ignoreboth` থাকলে কমান্ডের আগে একটা স্পেস দিয়ে লেখো। (ভুলে সেভ হয়ে গেলে `history -d <নম্বর>` তারপর `history -w`।)
7. `Esc`, `→`, `←`, `Ctrl+E` বা `Ctrl+A`-এর মতো যেকোনো movement কী চাপলে কমান্ড লাইনে থেকে যাবে এডিটের জন্য। পুরো বাতিল করতে `Ctrl+G`।
8. `~/.inputrc` থাকলে readline `/etc/inputrc` আর পড়ে না, আর distro-র `Ctrl+←/→` বাইন্ডিং সেখানেই ছিল। সমাধান: `~/.inputrc`-এর প্রথম লাইনে `$include /etc/inputrc` লিখে `Ctrl+X Ctrl+R` চাপো।
9. `Alt+.` লেখাটা সাথে সাথে লাইনে বসায়, তাই Enter চাপার আগে দেখে নেওয়া যায়; `!$` Enter চাপার পর expand হয়ে সরাসরি চলে (যদি না `histverify` চালু থাকে)। তাই `Alt+.` বেশি নিরাপদ।
10. প্রথমটা: `!$` = `/tmp/report.tar.gz`, আর `:r` শুধু **শেষ** extension সরায় → `/tmp/report.tar`। দ্বিতীয়টা: এখন ঠিক আগের কমান্ড হলো `echo /tmp/report.tar`, তাই `!$` = `/tmp/report.tar`, আর `:t` শুধু শেষ অংশ রাখে → `report.tar`। শিক্ষা: `!$` সবসময় **ঠিক আগের** কমান্ড দেখে — মাঝে অন্য কমান্ড চালালে ফল বদলে যায়।

অধ্যায় ৭: ফাইল সিস্টেম হায়ারার্কি (FHS)

> "লিনাক্সে সবকিছুই ফাইল" — এই একটা বাক্য বুঝলে লিনাক্সের অর্ধেক রহস্য খুলে যায়। এই অধ্যায়ে আমরা দেখব সেই ফাইলগুলো কোথায় থাকে, কেন ঠিক ওখানেই থাকে, আর ভেতরে কীভাবে সাজানো।

এই অধ্যায়ে যা শিখবে

- লিনাক্সের ফাইল সিস্টেম কেন একটা মাত্র **উল্টো গাছের (inverted tree)** মতো, যার গোড়া `/`
- **FHS (Filesystem Hierarchy Standard)** কী, কে বানিয়েছে, কেন দরকার
- `/bin`, `/sbin`, `/usr`, `/usr/local`, `/etc`, `/home`, `/root`, `/var`, `/tmp`, `/opt`, `/boot`, `/dev`, `/proc`, `/sys`, `/run`, `/mnt`, `/media`, `/lib`, `/srv` — প্রতিটি ডিরেক্টরির কাজ ও তার পেছনের যুক্তি
- **usr-merge** কী, কেন `/bin` আসলে `/usr/bin`-এর একটা লিংক
- Windows-এর `C:\`, `D:\` ড্রাইভ লেটারের সাথে লিনাক্সের **mount** ধারণার পার্থক্য
- **Case sensitivity** — কেন `File.txt` আর `file.txt` দুটো আলাদা ফাইল
- **Hidden file** (ডট দিয়ে শুরু নাম) কী, কীভাবে দেখতে হয়
- সাত ধরনের **ফাইল টাইপ** — regular, directory, link, block device, character device, socket, pipe — আর `ls -l` আউটপুটে সেগুলো চেনা
- `file` ও `stat` কমান্ড দিয়ে ফাইলের আসল পরিচয় বের করা
- **ext4, XFS, Btrfs** — জনপ্রিয় ফাইলসিস্টেমগুলোর পরিচিতি ও পার্থক্য

৭.১ ফাইল সিস্টেম কী? — একটা লাইব্রেরির গল্প

ধরো তোমার এলাকায় একটা বিশাল লাইব্রেরি আছে। সেখানে লাখ লাখ বই। যদি বইগুলো এলোমেলোভাবে মেঝেতে ফেলে রাখা হতো, তাহলে একটা নির্দিষ্ট বই খুঁজে পেতে কয়েক দিন লেগে যেত। তাই লাইব্রেরিতে আছে:

- **তলা (floor)** → বিভাগ (বিজ্ঞান, সাহিত্য, ইতিহাস)
- **তাক (shelf)** → উপবিভাগ
- **বই** → আসল তথ্য
- **ক্যাটালগ কার্ড** → কোন বই কোথায় আছে তার রেকর্ড

কম্পিউটারের **file system (ফাইল সিস্টেম)** ঠিক এই লাইব্রেরির মতো। হার্ডডিস্ক বা SSD আসলে শুধু কোটি কোটি বাইটের একটা লম্বা সারি। ফাইল সিস্টেম সেই কাঁচা বাইটগুলোকে **ফাইল** আর **ডিরেক্টরি (directory — ফোল্ডারের লিনাক্সীয় নাম)**-তে সাজিয়ে রাখে এবং মনে রাখে কোন ফাইল ডিস্কের কোন জায়গায় আছে।

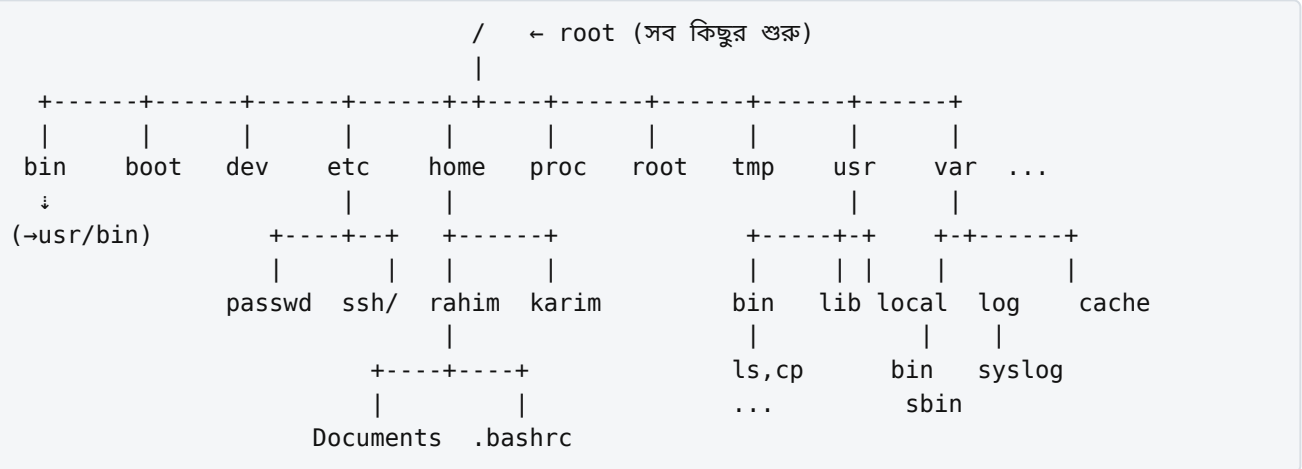
"ফাইল সিস্টেম" শব্দটা লিনাক্সে দুটো আলাদা অর্থে ব্যবহার হয় — এটা শুরুতেই পরিষ্কার করে নেওয়া ভালো:

অর্থ	মানে	উদাহরণ
১. ডিরেক্টরির সাজানো কাঠামো (hierarchy)	কোন ফোল্ডার কোথায়, কী রাখা হয় — এই অধ্যায়ের মূল বিষয়	`/etc`, `/home`, `/var`
২. ডিস্কে ডেটা লেখার ফরম্যাট (format)	ডিস্কের বাইটগুলো কীভাবে সাজানো থাকবে তার নিয়ম	ext4, XFS, Btrfs, NTFS, FAT32

প্রথম অর্থটা হলো লাইব্রেরির **তলা-তাকের নকশা**, দ্বিতীয়টা হলো **তাক বানানোর কারিগরি পদ্ধতি**। এই অধ্যায়ের বেশিরভাগ অংশ প্রথম অর্থ নিয়ে, শেষে দ্বিতীয়টার পরিচিতি দেওয়া হবে।

৭.২ একটাই গাছ, একটাই গোড়া: `/` (root)

লিনাক্সে সব ফাইল আর ডিরেক্টরি মিলে **একটা মাত্র গাছ** তৈরি করে। গাছটা উল্টো — গোড়া ওপরে, ডালপালা নিচে। এই গোড়ার নাম **root directory**, লেখা হয় একটা মাত্র স্ল্যাশ দিয়ে: `/`



এখানে খেয়াল করো:

- `/` সবার ওপরে। এর ওপরে আর কিছু নেই।
- `/home/rahim/Documents` মানে: root → home → rahim → Documents। এটাকে বলে **path (পাথ)** — ফাইলে পৌঁছানোর রাস্তা।
- পাথে ডিরেক্টরিগুলো আলাদা করা হয় `/` দিয়ে (Windows-এ যেটা `\`)।

> 📝 **মনে রাখো:** `/` চিহ্নটা দুটো কাজে লাগে। পাথের **শুরুতে** থাকলে মানে "root directory"; পাথের **মাঝে** থাকলে মানে "এর ভেতরে"। `/home/rahim` = "root-এর ভেতরে home, তার ভেতরে rahim"।

> ⚠️ **সতর্কতা:** "root" শব্দটা লিনাক্সে তিনটা আলাদা জিনিস বোঝায় — (১) `/` root directory, (২) `root` নামের সুপার-ইউজার (অ্যাডমিন), (৩) `/root` — সেই root ইউজারের হোম ডিরেক্টরি। প্রসঙ্গ দেখে বুঝে

> নিতে হবে। root ইউজার নিয়ে বিস্তারিত অধ্যায় ১৮-এ।

Absolute path আর relative path — এক নজরে

- **Absolute path** সবসময় `/` দিয়ে শুরু হয় — পুরো ঠিকানা (যেমন "ঢাকা, মিরপুর-১০, রোড ৫, বাড়ি ১২")।
- **Relative path** বর্তমান অবস্থান থেকে শুরু হয় — (যেমন "এখান থেকে দুই বাড়ি পরে")।

`.` (বর্তমান ডিরেক্টরি), `..` (প্যারেন্ট ডিরেক্টরি), `~` (হোম) আর `cd`, `pwd` দিয়ে চলাফেরার পুরো কৌশল আসবে পরের অধ্যায়ে — অধ্যায় ৮।

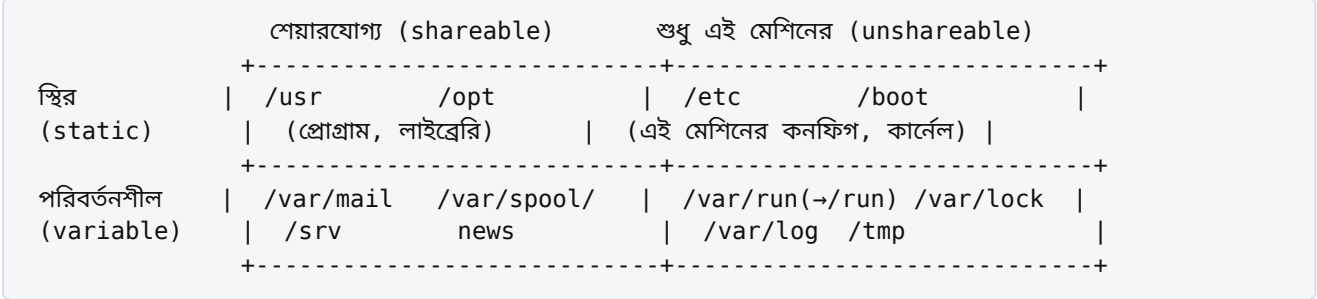
৭.৩ FHS কী এবং কেন দরকার?

FHS (Filesystem Hierarchy Standard) হলো একটা নিয়মের বই, যেটা বলে দেয় লিনাক্স সিস্টেমে কোন ধরনের ফাইল কোন ডিরেক্টরিতে থাকবে। এটা মূলত Linux Foundation রক্ষণাবেক্ষণ করে; সর্বশেষ বড় সংস্করণ **FHS 3.0 (২০১৫)**।

কেন দরকার? কল্পনা করো দেশের প্রতিটা শহরে পোস্ট অফিস, থানা, হাসপাতাল একেবারে আলাদা আলাদা নিয়মে বসানো। নতুন শহরে গেলে তুমি কিছুই খুঁজে পাবে না। FHS হলো সব শহরের জন্য একই "নগর-পরিকল্পনা"। এর সুবিধা:

1. **প্রোগ্রামাররা জানে** কনফিগারেশন ফাইল `/etc`-তে খুঁজতে হবে, লগ লিখতে হবে `/var/log`-এ।
2. **অ্যাডমিনরা** Ubuntu থেকে Fedora বা RHEL-এ গেলেও প্রায় একই জায়গায় সব পায়।
3. **ব্যাকআপ সহজ** — তোমার নিজের ডেটা `/home`-এ, সিস্টেম কনফিগ `/etc`-এ, পরিবর্তনশীল ডেটা `/var`-এ। কোনটা ব্যাকআপ নিতে হবে আর কোনটা আবার ইনস্টল করে পাওয়া যাবে, তা পরিষ্কার।
4. **নিরাপত্তা ও শেয়ারিং** — যেমন `/usr` রিড-অনলি করে রাখা যায় বা নেটওয়ার্কে শেয়ার করা যায়, কারণ সেখানে পরিবর্তনশীল কিছু থাকার কথা না।

FHS দুটো প্রশ্ন দিয়ে ডেটা ভাগ করে চিন্তা করে:



> 💡 **টিপস:** তোমার নিজের সিস্টেমেই এর অফিসিয়াল বিবরণ আছে। টার্মিনালে লিখো `man hier` (ক্লাসিক বিবরণ) অথবা `man file-hierarchy` (systemd-এর আধুনিক বিবরণ)। `q` চাপলে বেরিয়ে আসবে।

> man পেজ পড়ার কৌশল অধ্যায় ৫-এ আছে।

৭.৪ নিজের চোখে দেখো: `ls /`

টার্মিনাল খুলে লিখো:

```
$ ls /
bin                boot   dev  home  lib64              media  opt   root  sbin
srv  tmp  var
bin.usr-is-merged  cdrom  etc  lib   lib.usr-is-merged  mnt    proc  run   sbin.usr-is-
merged  snap  sys  usr
```

(এটা Ubuntu 24.04-এর আউটপুট। `snap`, `cdrom`, `*.usr-is-merged` Ubuntu-নির্দিষ্ট; Fedora-তে এগুলো দেখবে না, বরং `afs` জাতীয় কিছু থাকতে পারে।)

আরো বিস্তারিত দেখতে `-l` (long format) যোগ করো:

```
$ ls -l /
lrwxrwxrwx   1 root root          7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x   4 root root       4096 Oct 30 09:12 boot
drwxr-xr-x  20 root root       4840 Nov 12 08:01 dev
drwxr-xr-x 140 root root      12288 Nov 11 22:40 etc
drwxr-xr-x   3 root root       4096 Apr 25  2024 home
lrwxrwxrwx   1 root root          7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx   1 root root          9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x   2 root root       4096 Aug 27  2024 media
drwxr-xr-x   2 root root       4096 Aug 27  2024 mnt
drwxr-xr-x   3 root root       4096 Sep  3 10:20 opt
dr-xr-xr-x 412 root root          0 Nov 12 08:00 proc
drwx------   6 root root       4096 Oct  2 14:55 root
drwxr-xr-x  38 root root       1080 Nov 12 08:03 run
lrwxrwxrwx   1 root root          8 Apr 22  2024 sbin -> usr/sbin
drwxr-xr-x   2 root root       4096 Aug 27  2024 srv
dr-xr-xr-x  13 root root          0 Nov 12 08:00 sys
drwxrwxrwt  18 root root       4096 Nov 12 09:30 tmp
drwxr-xr-x  12 root root       4096 Aug 27  2024 usr
drwxr-xr-x  14 root root       4096 Aug 27  2024 var
```

খেয়াল করো:

- `bin -> usr/bin` — এটা একটা **symbolic link**, প্রথম অক্ষর `l`। এর মানে নিচে usr-merge অংশে।
- `proc` আর `sys`-এর সাইজ `0` — কারণ এগুলো আসল ডিস্কে নেই, **virtual** (৭.৬ অংশে দেখবে)।
- `tmp`-এর পারমিশনের শেষে `t` — এটা **sticky bit** (অধ্যায় ১৭)।
- `root`-এর পারমিশন `drwx------` — শুধু root ইউজার ঢুকতে পারে।

> 💡 **টিপস:** `tree` কমান্ড গাছের মতো করে দেখায়। ইনস্টল না থাকলে `sudo apt install tree` (Ubuntu/Debian) বা `sudo dnf install tree` (Fedora/RHEL)। তারপর `tree -L 1 /` (শুধু প্রথম স্তর) বা `tree -d -L 2 /usr` (শুধু ডিরেক্টরি, দুই স্তর পর্যন্ত)।

৭.৫ প্রতিটি ডিরেক্টরি — কী আছে, কেন আছে

এবার একে একে প্রতিটা ডিরেক্টরিতে ঢুকব। প্রতিটার জন্য তিনটা প্রশ্ন: **কী থাকে? কেন আলাদা? কী উদাহরণ?**

৭.৫.১ `/bin` — অপরিহার্য ইউজার কমান্ড (essential user binaries)

- **কী থাকে:** সাধারণ ইউজারের দরকারি মৌলিক প্রোগ্রাম — `ls`, `cp`, `mv`, `cat`, `bash`, `mkdir`, `rm` ইত্যাদি। "binary" মানে কম্পাইল করা চালানোযোগ্য প্রোগ্রাম।
- **ঐতিহাসিক কারণ:** আগে বুট হওয়ার সময় `/usr` আলাদা পার্টিশনে থাকত, হয়তো তখনও মাউন্ট হয়নি। তাই বুটের জন্য একেবারে জরুরি কমান্ডগুলো `/bin`-এ রাখা হতো।
- **আধুনিক বাস্তবতা:** Ubuntu, Debian, Fedora, RHEL — সবখানে এখন `/bin` শুধু `usr/bin`-এর দিকে নির্দেশ করা একটা লিংক (দেখো ৭.৭ usr-merge)।

```
$ ls /bin | head -5
'['
aa-enabled
aa-exec
aa-features-abi
acpi_listen
```

(প্রথম নামটা `[` — হ্যাঁ, এটা আসলে একটা প্রোগ্রাম! `test` কমান্ডের আরেক নাম, শেল স্ক্রিপ্টে ব্যবহার হয় — অধ্যায় ৪১।)

৭.৫.২ `/sbin` — সিস্টেম অ্যাডমিন কমান্ড (system binaries)

- **কী থাকে:** সিস্টেম পরিচালনার কমান্ড, যেগুলো সাধারণত root চালায় — `fdisk`, `mkfs`, `reboot`, `ip`, `iptables`, `fsck`।
- **কেন আলাদা:** "s" = system। সাধারণ ইউজারের PATH-এ এগুলো রাখার দরকার ছিল না, তাই আলাদা।
- **আধুনিক বাস্তবতা:** `/sbin` → `usr/sbin` লিংক। আরো এক ধাপ এগিয়ে **Fedora 42 থেকে `/usr/sbin`-কেও `/usr/bin`-এর সাথে একীভূত** করা হয়েছে (Arch Linux-ও তাই করে)। Ubuntu 24.04 / Debian 12 / RHEL 9-এ এখনো `/usr/sbin` আলাদা ডিরেক্টরি।

> 📝 **মনে রাখো:** সাধারণ ইউজার হিসেবে `/sbin`-এর কিছু কমান্ড চালাতে পারো (যেমন `ip addr`), কিন্তু পরিবর্তন করার মতো কাজ (যেমন `fdisk /dev/sda`) করতে `sudo` লাগবে।

৭.৫.৩ `/usr` — ইউজার প্রোগ্রাম ও ডেটার বিশাল ভাণ্ডার

নামটা দেখে মনে হয় "user", কিন্তু এখানে ইউজারের ব্যক্তিগত ফাইল থাকে **না**। অনেকে এখন একে **"Unix System Resources"** বলে ব্যাখ্যা করে (যদিও ঐতিহাসিকভাবে নামটা user থেকেই এসেছিল — প্রাচীন Unix-এ ইউজারদের হোম এখানেই ছিল)। এটা সিস্টেমের সবচেয়ে বড় ডিরেক্টরিগুলোর একটা — প্যাকেজ ম্যানেজার দিয়ে ইনস্টল করা প্রায় সব সফটওয়্যার এখানে।

```
/usr
├── bin/       ← প্রায় সব ইউজার কমান্ড (firefox, python3, git, ls...)
├── sbin/      ← অ্যাডমিন কমান্ড
├── lib/       ← লাইব্রেরি (.so ফাইল), কার্নেল মডিউল (/usr/lib/modules)
├── lib64/     ← 64-bit লাইব্রেরি (Fedora/RHEL-এ বেশি ব্যবহার হয়)
├── include/   ← C/C++ হেডার ফাইল (.h) — প্রোগ্রাম কম্পাইলের জন্য
├── share/     ← আর্কিটেকচার-নিরপেক্ষ ডেটা: man পেজ, আইকন, ডকুমেন্টেশন, ফন্ট
│   ├── man/
│   ├── doc/
│   └── icons/
├── src/       ← সোর্স কোড (যেমন কার্নেল হেডার)
└── local/     ← ⭐ লোকালি ইনস্টল করা সফটওয়্যার (পরের অংশ)
```

- **মূলনীতি:** `/usr` হলো **শেয়ারযোগ্য ও রিড-অনলি** ডেটা। প্যাকেজ ম্যানেজার (`apt`, `dnf`) এটার মালিক। নিজে হাতে এখানে ফাইল বদলানো উচিত না — পরের আপডেটে মুছে যেতে পারে।

৭.৫.৪ `/usr/local` — তোমার নিজের হাতে ইনস্টল করা সফটওয়্যার

- **কী থাকে:** যে সফটওয়্যার তুমি প্যাকেজ ম্যানেজার ছাড়া, নিজে কম্পাইল করে বা স্ক্রিপ্ট দিয়ে ইনস্টল করেছ। এর ভেতরেও `bin`, `lib`, `share`, `include`, `sbin`, `etc` — একই কাঠামো।
- **কেন আলাদা:** প্যাকেজ ম্যানেজার `/usr/local`-এ হাত দেয় না। তাই সিস্টেম আপডেট করলেও তোমার নিজের ইনস্টল করা জিনিস নষ্ট হয় না।
- **উদাহরণ:** সোর্স থেকে বিল্ড করে `make install` দিলে ডিফল্টভাবে সব `/usr/local`-এ যায় (অধ্যায় ৩১)।

> 💡 **টিপস:** PATH-এ সাধারণত `/usr/local/bin` আগে থাকে `/usr/bin`-এর চেয়ে। তাই একই নামের দুটো প্রোগ্রাম থাকলে লোকালটা আগে চলে। `echo $PATH` দিয়ে দেখো।

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
```

৭.৫.৫ `/etc` — কনফিগারেশনের রাজধানী

- **কী থাকে:** পুরো সিস্টেমের **কনফিগারেশন ফাইল** — সাধারণ টেক্সট ফাইল, যেগুলো এডিটর দিয়ে খুলে বদলানো যায়। নামটা এসেছে "et cetera" (ইত্যাদি) থেকে, কারণ প্রথম দিকে যে জিনিস কোথাও ফিট করত না সেগুলো এখানে রাখা হতো। এখন অনেকে মজা করে বলে "Editable Text Configuration"।

- **কেন আলাদা:** এগুলো **এই নির্দিষ্ট মেশিনের** সেটিং। `/usr`-এর প্রোগ্রাম সব মেশিনে এক, কিন্তু `/etc`-এর সেটিং তোমার মেশিনের নিজস্ব। ব্যাকআপে `/etc` অবশ্যই রাখতে হয়।
- **নিয়ম:** `/etc`-এ কোনো binary (চালানোযোগ্য প্রোগ্রাম) রাখার কথা না।

গুরুত্বপূর্ণ কিছু ফাইল:

ফাইল/ডিরেক্টরি	কী আছে	বিস্তারিত কোথায়
`/etc/passwd`	ইউজারদের তালিকা (পাসওয়ার্ড নয়!)	অধ্যায় ১৫
`/etc/shadow`	এনক্রিপ্ট (hash) করা পাসওয়ার্ড — শুধু root পড়তে পারে	অধ্যায় ১৫
`/etc/group`	গ্রুপের তালিকা	অধ্যায় ১৫
`/etc/sudoers`	কে sudo চালাতে পারবে	অধ্যায় ১৮
`/etc/fstab`	বুটের সময় কোন ডিস্ক কোথায় মাউন্ট হবে	অধ্যায় ৩২
`/etc/hostname`	কম্পিউটারের নাম	অধ্যায় ৩৬
`/etc/hosts`	লোকাল নাম → IP ম্যাপিং	অধ্যায় ৩৬
`/etc/os-release`	কোন ডিস্ট্রো, কোন ভার্সন	নিচে উদাহরণ
`/etc/ssh/`	SSH সার্ভারের কনফিগ	অধ্যায় ৩৮
`/etc/systemd/`	systemd সার্ভিসের লোকাল কনফিগ	অধ্যায় ২৬
`/etc/apt/` অথবা `/etc/yum.repos.d/`	প্যাকেজ রিপোজিটরি	অধ্যায় ৩০
`/etc/crontab`	সিস্টেম-ব্যাপী শিডিউল	অধ্যায় ২৯

```
$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.1 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
...
```

> 💡 **টিপস:** অনেক কনফিগের এখন `.d` ডিরেক্টরি থাকে — যেমন `/etc/sudoers.d/`, `/etc/sysctl.d/`, `/etc/apt/sources.list.d/`। মূল ফাইল না বদলে এখানে নতুন ছোট ফাইল যোগ করা নিরাপদ পদ্ধতি — আপডেটে মূল ফাইল বদলালেও তোমার সেটিং টিকে থাকে।

> ⚠️ **সতর্কতা:** `/etc`-এর কোনো ফাইল বদলানোর আগে কপি রেখে দাও: `sudo cp /etc/fstab /etc/fstab.bak`। ভুল `/etc/fstab` বা `/etc/sudoers` সিস্টেমকে বুট-অযোগ্য বা অ্যাডমিন-অযোগ্য করে দিতে পারে।

৭.৫.৬ `/home` — সাধারণ ইউজারদের বাড়ি

- **কী থাকে:** প্রতিটা সাধারণ ইউজারের নিজস্ব ডিরেক্টরি — `/home/rahim`, `/home/karim`। এখানে তোমার ডকুমেন্ট, ছবি, ডাউনলোড, আর অ্যাপের ব্যক্তিগত সেটিং (hidden file হিসেবে) থাকে।
- **কেন আলাদা:** ইউজারের ডেটা সিস্টেম থেকে আলাদা রাখলে: (১) এক ইউজার আরেকজনের ফাইল দেখতে পারে না (পারমিশন দিয়ে), (২) `/home`-কে আলাদা পার্টিশনে রাখলে OS নতুন করে ইনস্টল করলেও তোমার ডেটা থেকে যায়।
- **শর্টকাট:** শেলে `~` মানে তোমার হোম ডিরেক্টরি। `~rahim` মানে rahim-এর হোম।

```
$ echo ~
/home/rahim
$ ls ~
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

৭.৫.৭ `/root` — root ইউজারের বাড়ি

- **কী থাকে:** সুপার-ইউজার `root`-এর হোম ডিরেক্টরি।
- **কেন `/home/root` না?** কারণ `/home` অনেক সময় আলাদা পার্টিশন বা নেটওয়ার্ক ড্রাইভে থাকে। সেটা মাউন্ট না হলে বা নষ্ট হলেও root যেন লগইন করে সিস্টেম ঠিক করতে পারে — তাই root-এর বাড়ি মূল `/` পার্টিশনেই।

```
$ ls /root
ls: cannot open directory '/root': Permission denied
$ sudo ls /root
snap
```

৭.৫.৮ `/var` — পরিবর্তনশীল ডেটা (variable data)

- **কী থাকে:** যে ডেটা সিস্টেম চলার সময় **ক্রমাগত বাড়ে বা বদলায়** — লগ, ক্যাশ, ডাটাবেস, মেইল কিউ, প্রিন্ট কিউ।
- **কেন আলাদা:** `/usr` রিড-অনলি রাখতে চাইলে বদলাতে-থাকা ডেটার আলাদা জায়গা দরকার। সার্ভারে অনেক সময় `/var`-কে আলাদা পার্টিশনে রাখা হয়, যাতে লগ ফুলে ডিস্ক ভরে গেলেও `/` পার্টিশন ভরে সিস্টেম না থামে।

```
/var
├── log/        ← লগ ফাইল (syslog, auth.log, nginx/...) — [অধ্যায় ২৮]
├── cache/      ← ক্যাশ; মুছে ফেললেও ক্ষতি নেই, আবার তৈরি হবে (apt-এর .deb ক্যাশ)
├── lib/        ← প্রোগ্রামের স্থায়ী অবস্থা/ডেটা: MySQL ডাটাবেস (/var/lib/mysql),
```

```
│                  Docker (/var/lib/docker), প্যাকেজ ডাটাবেস (/var/lib/dpkg)
├── spool/      ← কিউ: প্রিন্ট জব, মেইল, cron জব
├── tmp/        ← অস্থায়ী, কিন্তু রিবুটের পরও টিকে থাকে
├── www/        ← অনেক ডিস্ট্রোতে ওয়েব সার্ভারের ডিফল্ট ফাইল (/var/www/html)
├── mail/       ← ইউজারদের মেইলবক্স
└── run → /run  ← পুরনো পথ, এখন লিংক
```

⚠️ **সতর্কতা:** "ডিস্ক ভরে গেছে" সমস্যার সবচেয়ে কমন কারণ `/var/log` বা `/var/lib/docker`। খুঁজতে: `sudo du -sh /var/* 2>/dev/null | sort -h` (অধ্যায় ৩২, অধ্যায় ৪৬)। কিন্তু `/var/lib`-এর কিছু অন্ধভাবে মুছবে না — সেখানে ডাটাবেস থাকে!

৭.৫.৯ `/tmp` — অস্থায়ী ফাইল

- **কী থাকে:** প্রোগ্রাম আর ইউজারদের সাময়িক ফাইল। যে কেউ এখানে লিখতে পারে।
- **নিরাপত্তা:** পারমিশন `drwxrwxrwt` — শেষের `t` (sticky bit) মানে সবাই ফাইল বানাতে পারবে, কিন্তু **শুধু নিজের ফাইল মুছতে পারবে**, অন্যেরটা না।
- **কখন মোছে (distro-ভেদে):**

ডিস্ট্রো	`/tmp` কোথায়	কখন খালি হয়
Ubuntu 24.04	ডিস্কে	প্রতি বুটে খালি হয় (systemd-tmpfiles)
Debian 12	ডিস্কে	প্রতি বুটে খালি হয়
Debian 13	RAM-এ (tmpfs)	রিবুটে স্বাভাবিকভাবেই মুছে যায়; পুরনো ফাইলও সময় অনুযায়ী মোছে
Fedora / RHEL 9	Fedora-তে RAM (tmpfs); RHEL 9-এ ডিফল্টভাবে ডিস্কে	রিবুটে / সময়ভিত্তিক পরিষ্কার

- **`/tmp` বনাম `/var/tmp`:** `/var/tmp`-এর ফাইল রিবুটের পরও থাকে (সাধারণত ৩০ দিন পুরনো হলে মোছে)। যে অস্থায়ী ফাইল রিবুটের পরও লাগবে, সেটা `/var/tmp`-এ।

📝 **মনে রাখো:** `/tmp`-এ কখনো গুরুত্বপূর্ণ কিছু রেখো না। রিবুট দিলেই হারিয়ে যেতে পারে।

৭.৫.১০ `/opt` — অতিরিক্ত (optional) তৃতীয়-পক্ষ সফটওয়্যার

- **কী থাকে:** বড়, স্বয়ংসম্পূর্ণ তৃতীয়-পক্ষ সফটওয়্যার, যেগুলো নিজের সব ফাইল এক ফোল্ডারে রাখে — যেমন `/opt/google/chrome`, `/opt/zoom`, বিভিন্ন কোম্পানির প্রোপ্রাইটারি টুল।
- **`/opt` বনাম `/usr/local`:** `/usr/local` হলো FHS কাঠামো মেনে ছড়ানো (`bin`, `lib`, `share` আলাদা); `/opt` হলো "এক প্যাকেজ = এক ফোল্ডার" (`/opt/<package>/` এর ভেতরে সব)। আনইনস্টল করা সহজ — ফোল্ডারটা মুছে দিলেই হলো।

৭.৫.১১ `/boot` — বুটের জন্য দরকারি ফাইল

- **কী থাকে:** কার্নেল, initramfs আর বুটলোডার (GRUB)-এর ফাইল — কম্পিউটার চালু হওয়ার একদম প্রথম ধাপে যা লাগে।

```
$ ls /boot
config-6.8.0-45-generic      initrd.img.old                  vmlinuz
efi                          System.map-6.8.0-45-generic     vmlinuz-6.8.0-45-generic
grub                         initrd.img                      vmlinuz.old
initrd.img-6.8.0-45-generic
```

ফাইল	কী
`vmlinuz-*`	কম্প্রেস করা লিনাক্স **kernel** (z = compressed)
`initrd.img-*` (Ubuntu/Debian) বা `initramfs-*.img` (Fedora/RHEL)	ছোট একটা অস্থায়ী root ফাইল সিস্টেম, যেটা আসল ডিস্ক মাউন্ট করার ড্রাইভার লোড করে
`config-*`	কার্নেল কোন অপশনে কম্পাইল হয়েছে
`System.map-*`	কার্নেলের ফাংশন-ঠিকানার ম্যাপ (ডিবাগিং-এ লাগে)
`grub/` (Fedora/RHEL-এ `grub2/`)	বুটলোডারের কনফিগ ও মডিউল
`efi/`	UEFI সিস্টেমে **EFI System Partition (ESP)** এখানে মাউন্ট হয় (FAT32 ফরম্যাটে)

> ⚠️ **সতর্কতা:** `/boot`-এর কিছু হাতে মুছবে না। পুরনো কার্নেল সরাতে প্যাকেজ ম্যানেজার ব্যবহার করো (`sudo apt autoremove`)। পাওয়ার বাটন থেকে লগইন পর্যন্ত পুরো যাত্রা অধ্যায় ২৭-এ।

৭.৫.১২ `/dev` — ডিভাইস ফাইল

এখানেই "**লিনাক্সে সবকিছু ফাইল**" দর্শনটা সবচেয়ে স্পষ্ট। হার্ডডিস্ক, কিবোর্ড, টার্মিনাল, এমনকি "শূন্যতা" — সবকিছু `/dev`-এ একটা ফাইল হিসেবে দেখা যায়। প্রোগ্রাম এই ফাইলে পড়লে/লিখলে আসলে kernel-এর **device driver**-কে নির্দেশ দেওয়া হয়।

```
তোমার প্রোগ্রাম                (user space)
     │  write("/dev/sda", ...)
     ▼
┌──────────────────────────────────────┐
│  Kernel: VFS → device driver (sd)    │   (kernel space)
└──────────────────────────────────────┘
     │
     ▼
আসল হার্ডওয়্যার: SATA ডিস্ক
```

ডিভাইস ফাইল	কী
`/dev/sda`, `/dev/sdb`	প্রথম, দ্বিতীয় SATA/SCSI/USB ডিস্ক

ডিভাইস ফাইল	কী
`/dev/sda1`, `/dev/sda2`	sda-এর প্রথম, দ্বিতীয় পার্টিশন
`/dev/nvme0n1`, `/dev/nvme0n1p1`	NVMe SSD ও তার পার্টিশন
`/dev/vda`	ভার্চুয়াল মেশিনের (virtio) ডিস্ক
`/dev/tty`, `/dev/pts/0`	টার্মিনাল
`/dev/null`	"কৃষ্ণগহ্বর" — যা লিখবে হারিয়ে যাবে; পড়লে কিছু পাবে না
`/dev/zero`	পড়লে অসীম শূন্য বাইট দেয়
`/dev/random`, `/dev/urandom`	র‍্যান্ডম বাইট
`/dev/sr0`	CD/DVD ড্রাইভ

```
$ echo "এটা হারিয়ে যাবে" > /dev/null
$ head -c 8 /dev/urandom | od -An -tx1
 3f a1 07 c9 5e 22 8b d0
```

ভেতরে কীভাবে তৈরি হয়? `/dev` আসলে ডিস্কে থাকে না। বুটের সময় kernel **devtmpfs** নামের একটা RAM-ভিত্তিক ফাইল সিস্টেম মাউন্ট করে এবং নিজে ডিভাইস নোড বানায়। তারপর **udev** (systemd-udevd) নামের সার্ভিস নিয়ম মেনে পারমিশন ঠিক করে আর সুন্দর নামের লিংক বানায় — যেমন `/dev/disk/by-uuid/...`। পেনড্রাইভ লাগালে সাথে সাথে `/dev/sdb` আবির্ভূত হয় এভাবেই।

> ⚠️ **সতর্কতা:** `/dev/sda`-তে সরাসরি লেখা মানে পুরো ডিস্কে লেখা। `dd if=... of=/dev/sda` ভুল ডিস্কে দিলে সব ডেটা চিরতরে শেষ (অধ্যায় ৩৫)।

৭.৫.১৩ `/proc` — চলমান প্রসেস ও kernel-এর জানালা

- **কী:** একটা **virtual file system (procfs)**। এখানকার ফাইলগুলো ডিস্কে নেই — তুমি যখন পড়ো, kernel তখনই তাৎক্ষণিকভাবে তথ্য বানিয়ে দেয়। তাই `ls -l`-এ বেশিরভাগ ফাইলের সাইজ `0` দেখায়, অথচ `cat` করলে লেখা আসে।
- **কেন:** kernel-এর ভেতরের অবস্থা জানতে আলাদা বিশেষ টুল না বানিয়ে সাধারণ `cat` দিয়েই পড়া যায় — "সবকিছু ফাইল" দর্শন।

```
$ cat /proc/cpuinfo | grep "model name" | head -1
model name      : Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz
$ cat /proc/meminfo | head -3
MemTotal:       16244368 kB
MemFree:         2318840 kB
MemAvailable:   9876544 kB
$ cat /proc/version
Linux version 6.8.0-45-generic (buildd@lcy02-amd64-075) ...
$ ls /proc | head
1
2
```

```
3
...
```

সংখ্যা নামের প্রতিটা ডিরেক্টরি (`/proc/1` , `/proc/2345`) হলো একটা চলমান **process**-এর তথ্য — তার PID (process ID) অনুযায়ী। `/proc/self` সবসময় যে প্রসেস পড়ছে তাকেই নির্দেশ করে। প্রসেস নিয়ে অধ্যায় ২৪, আর `/proc` ও `/sys` -এর গভীরে অধ্যায় ৪৭।

৭.৫.১৪ `/sys` — kernel-এর ডিভাইস-মডেলের জানালা

- **কী:** আরেকটা virtual file system (**sysfs**)। এখানে kernel তার চেনা সব ডিভাইস, ড্রাইভার, বাস (USB, PCI) সুশৃঙ্খল গাছ আকারে দেখায়।
- **`/proc` বনাম `/sys` :** `/proc` মূলত প্রসেস ও সাধারণ kernel তথ্য (ঐতিহাসিকভাবে সব কিছুই এখানে ঢুকে গিয়েছিল); `/sys` নতুন ও পরিচ্ছন্ন — প্রতি ফাইলে সাধারণত একটাই মান, মূলত হার্ডওয়্যার/ড্রাইভার নিয়ে।

```
$ cat /sys/class/power_supply/BAT0/capacity
87
$ cat /sys/class/net/*/address
00:00:00:00:00:00
a4:5e:60:e1:22:9b
```

> 💡 **টিপস:** কিছু `/sys` ফাইলে লিখে সেটিং বদলানো যায় (যেমন স্ক্রিনের উজ্জ্বলতা `/sys/class/backlight/*/brightness`)। কিন্তু রিবুটে সেটা হারিয়ে যায়, কারণ পুরোটাই RAM-এ।

৭.৫.১৫ `/run` — রানটাইম ডেটা (বুট থেকে এখন পর্যন্ত)

- **কী থাকে:** চলমান সিস্টেমের ক্ষণস্থায়ী তথ্য — **PID ফাইল** (কোন সার্ভিসের প্রসেস নম্বর কত), **socket** ফাইল, লক ফাইল, লগইন সেশনের তথ্য (`/run/user/1000`)।
- **কেন আলাদা:** এটা **tmpfs** (RAM-এ)। বুটের একেবারে শুরুতেই লেখার যোগ্য থাকে, আর রিবুটে নিজে থেকেই সব মুছে যায় — পুরনো "stale" PID ফাইলের সমস্যা থাকে না।
- পুরনো পথ `/var/run` আর `/var/lock` এখন `/run` আর `/run/lock` -এর লিংক।

```
$ ls -ld /var/run
lrwxrwxrwx 1 root root 4 Aug 27  2024 /var/run -> /run
$ df -h /run
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G  2.4M  1.6G   1% /run
```

৭.৫.১৬ `/mnt` — অস্থায়ী, হাতে করা মাউন্ট

- **কী:** অ্যাডমিন যখন সাময়িকভাবে নিজে হাতে কোনো ফাইল সিস্টেম মাউন্ট করে (যেমন উদ্ধার কাজের সময় অন্য ডিস্ক), তার জন্য একটা খালি জায়গা।

- উদাহরণ: `sudo mount /dev/sdb1 /mnt` — তারপর `/mnt`-এ গেলে sdb1-এর ফাইল দেখা যাবে। (অধ্যায় ৩২)

৭.৫.১৭ `/media` — পেনড্রাইভ, CD ইত্যাদি অপসারণযোগ্য মিডিয়া

- **কী:** USB পেনড্রাইভ, এক্সটার্নাল হার্ডডিস্ক, CD/DVD প্লাগ করলে ডেস্কটপ পরিবেশ (udisks2 সার্ভিসের মাধ্যমে) স্বয়ংক্রিয়ভাবে মাউন্ট করে।
- **distro-ভেদে পার্থক্য:** Ubuntu/Debian-এ `/media/<ইউজারনেম>/<ডিস্কের লেবেল>`; Fedora/RHEL-এ `/run/media/<ইউজারনেম>/<লেবেল>`।

```
$ ls /media/rahim/
MY_PENDRIVE
```

	`/mnt`	`/media`
কে মাউন্ট করে	অ্যাডমিন, হাতে	সিস্টেম, স্বয়ংক্রিয়ভাবে
কী জন্য	সাময়িক কাজ	অপসারণযোগ্য মিডিয়া

৭.৫.১৮ `/lib`, `/lib64` — শেয়ার্ড লাইব্রেরি ও kernel মডিউল

- **কী থাকে:** প্রোগ্রামগুলো যে **shared library (`.so` ফাইল — Windows-এর `.dll`-এর মতো)** ব্যবহার করে, সেগুলো। যেমন C লাইব্রেরি `libc.so.6`, আর ডাইনামিক লিংকার `ld-linux-x86-64.so.2`। kernel মডিউল থাকে `/lib/modules/<kernel-version>/`-এ।
- **কেন:** প্রতিটা প্রোগ্রাম একই কোড আলাদাভাবে না রেখে এক কপি লাইব্রেরি শেয়ার করে — ডিস্ক ও RAM দুটোই বাঁচে। (উপমা: পাড়ার সবাই একটা কমন লাইব্রেরি থেকে বই নেয়, প্রত্যেকে আলাদা কেনে না।)
- **আধুনিক বাস্তবতা:** `/lib` → `usr/lib`, `/lib64` → `usr/lib64` লিংক।
- **distro-ভেদে:** Debian/Ubuntu "multiarch" ব্যবহার করে — 64-bit লাইব্রেরি `/usr/lib/x86_64-linux-gnu/`-এ। Fedora/RHEL 64-bit লাইব্রেরি রাখে `/usr/lib64/`-এ।

```
$ ldd /bin/ls
    linux-vdso.so.1 (0x00007ffd4f3e5000)
    libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f2a1c2b4000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2a1c000000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f2a1c31c000)
```

`ldd` দেখায় কোনো প্রোগ্রাম কোন কোন লাইব্রেরির ওপর নির্ভর করে।

৭.৫.১৯ `/srv` — সার্ভিসের পরিবেশন করা ডেটা

- **কী:** এই মেশিন যে সার্ভিস দিচ্ছে, তার ডেটা — যেমন ওয়েবসাইট (`/srv/www`), FTP ফাইল (`/srv/ftp`), git রিপো (`/srv/git`)।
- **বাস্তবে:** অনেক ডিস্ট্রোতে এটা খালি থাকে, কারণ ওয়েব সার্ভারের ডিফল্ট `/var/www`। কিন্তু নিজে সার্ভার সাজালে `/srv` ব্যবহার করা FHS-সম্মত ও পরিচ্ছন্ন (অধ্যায় ৪০)।

৭.৫.২০ অন্যান্য যা দেখতে পারো

ডিরেক্টরি	কী
`/lost+found`	ext4 পার্টিশনের রুটে থাকে। `fsck` ডিস্ক মেরামতের সময় হারানো টুকরো ফাইল এখানে রাখে
`/snap`	Ubuntu-তে snap প্যাকেজ এখানে মাউন্ট হয়
`/cdrom`	Ubuntu ইনস্টলারের রেখে যাওয়া খালি ডিরেক্টরি
`/swap.img` বা `/swapfile`	swap ফাইল (RAM ভরে গেলে ব্যবহারের জায়গা)
`/*.usr-is-merged`	Ubuntu/Debian-এ usr-merge সম্পন্ন হয়েছে তা চিহ্নিত করার মার্কার

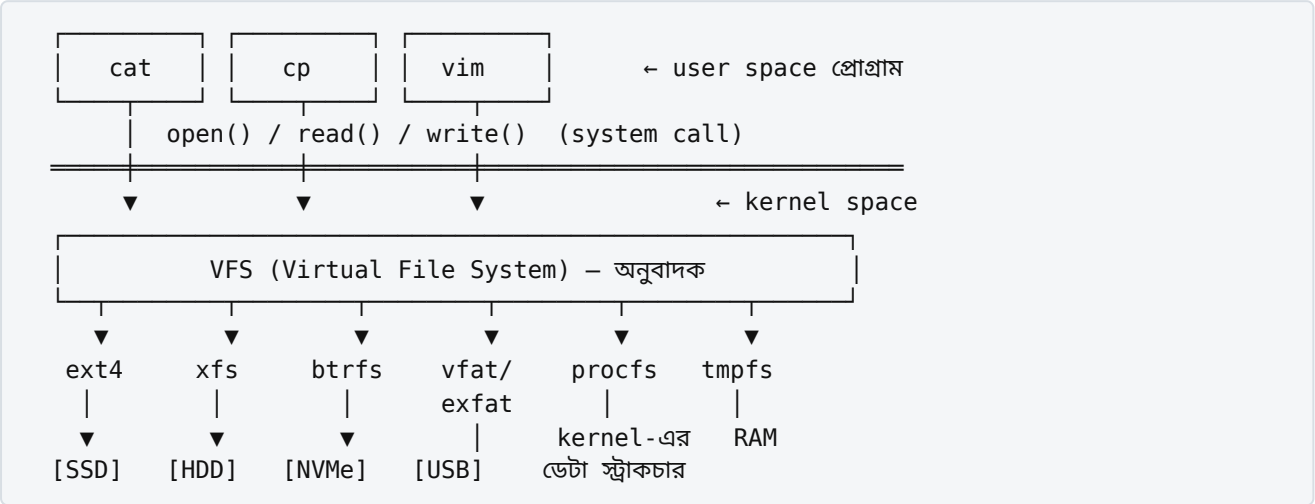
৭.৫.২১ এক নজরে পুরো মানচিত্র

ডিরেক্টরি	এক লাইনে	ডিস্কে?	উপমা
`/`	সব কিছুর গোড়া	হ্যাঁ	দেশের মানচিত্র
`/bin` → `/usr/bin`	ইউজার কমান্ড	হ্যাঁ	সাধারণ যন্ত্রপাতির বাক্স
`/sbin` → `/usr/sbin`	অ্যাডমিন কমান্ড	হ্যাঁ	মিস্ত্রির বিশেষ যন্ত্র
`/usr`	ইনস্টল করা সফটওয়্যার	হ্যাঁ	শহরের বাজার
`/usr/local`	নিজে ইনস্টল করা সফটওয়্যার	হ্যাঁ	নিজের দোকান
`/opt`	তৃতীয়-পক্ষ প্যাকেজ	হ্যাঁ	ভাড়া দেওয়া আলাদা দোকান
`/etc`	কনফিগারেশন	হ্যাঁ	অফিসের নিয়মের খাতা
`/home`	ইউজারদের বাড়ি	হ্যাঁ	আবাসিক এলাকা
`/root`	root-এর বাড়ি	হ্যাঁ	প্রধানের বাংলো
`/var`	বদলাতে-থাকা ডেটা	হ্যাঁ	হিসাবের খাতা, ডায়েরি
`/tmp`	অস্থায়ী	distro-ভেদে	রাফ খাতা
`/boot`	বুট ফাইল	হ্যাঁ	গাড়ির চাবি ও স্টার্টার
`/dev`	ডিভাইস	না (devtmpfs)	যন্ত্রের রিমোট কন্ট্রোল
`/proc`	প্রসেস/kernel তথ্য	না (procfs)	লাইভ স্কোরবোর্ড
`/sys`	হার্ডওয়্যার/ড্রাইভার	না (sysfs)	যন্ত্রের ড্যাশবোর্ড
`/run`	রানটাইম তথ্য	না (tmpfs)	আজকের নোটিশ বোর্ড
`/mnt`	হাতে মাউন্ট	হ্যাঁ (খালি)	অস্থায়ী পার্কিং
`/media`	পেনড্রাইভ	হ্যাঁ (খালি)	অতিথি পার্কিং
`/lib` → `/usr/lib`	লাইব্রেরি	হ্যাঁ	পাড়ার কমন লাইব্রেরি
`/srv`	সার্ভিসের ডেটা	হ্যাঁ	দোকানের শো-রুম

৭.৬ ভার্চুয়াল ফাইল সিস্টেম — ভেতরে কীভাবে কাজ করে (VFS)

তুমি হয়তো ভাবছ — `/home` আছে SSD-তে ext4 ফরম্যাটে, পেনড্রাইভ FAT32/exFAT-এ, `/proc` কোথাও নেই — অথচ `cat` সবগুলোকে একই ভাবে পড়ে কীভাবে?

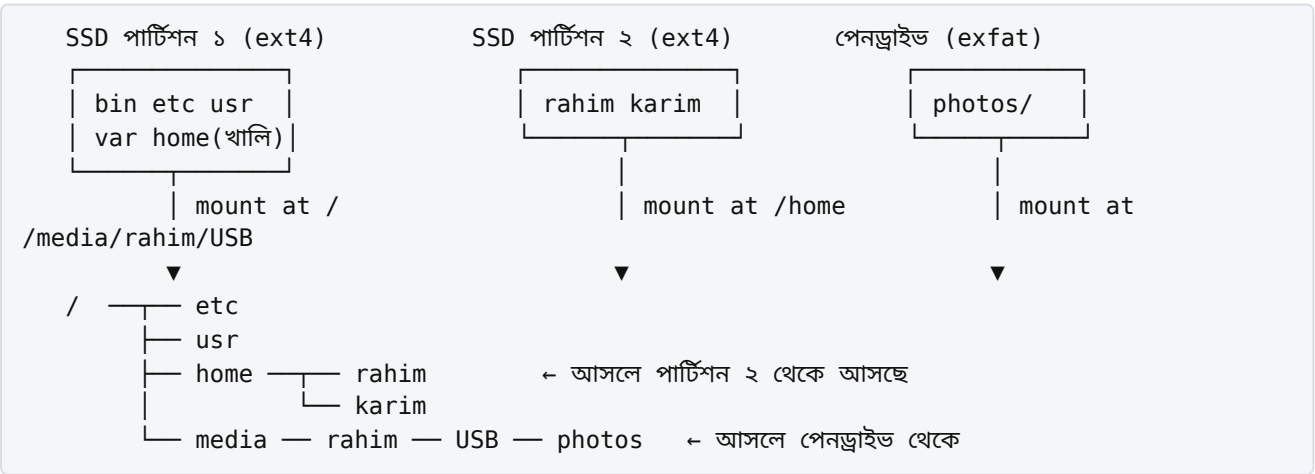
উত্তর: kernel-এর **VFS (Virtual File System)** স্তর। এটা একটা "অনুবাদক" — সব প্রোগ্রামকে একটাই ভাষা (open, read, write, close) দেয়, আর পেছনে প্রতিটা ফাইল সিস্টেমের নিজস্ব ড্রাইভারের সাথে কথা বলে।



উপমা: VFS হলো একটা বহুভাষী দোভাষী। তুমি বাংলায় বলো "এই ফাইলটা পড়ো" — দোভাষী ext4-কে তার ভাষায়, procfs-কে তার ভাষায় বুঝিয়ে দেয়, আর ফলাফল তোমাকে বাংলায় ফেরত দেয়।

Mount: গাছে ডাল জোড়া লাগানো

Mount (মাউন্ট) মানে একটা ফাইল সিস্টেমকে (পার্টিশন, পেনড্রাইভ, বা virtual fs) মূল গাছের কোনো একটা ডিরেক্টরিতে "জুড়ে দেওয়া"। যে ডিরেক্টরিতে জোড়া হয় তাকে বলে **mount point**।



ইউজারের চোখে সব একটা গাছ — কোন ফাইল কোন ডিস্কে, তা জানার দরকারই নেই।

মাউন্ট দেখার কমান্ড:

```
$ findmnt --real
TARGET              SOURCE          FSTYPE OPTIONS
/                   /dev/nvme0n1p2  ext4   rw,relatime
├─/boot/efi         /dev/nvme0n1p1  vfat   rw,relatime,fmask=0077,...
└─/media/rahim/USB  /dev/sda1       exfat  rw,nosuid,nodev,relatime,...

$ df -hT
Filesystem     Type   Size  Used Avail Use% Mounted on
tmpfs          tmpfs  1.6G  2.4M  1.6G   1% /run
/dev/nvme0n1p2 ext4   468G  112G  333G  26% /
tmpfs          tmpfs  7.8G   12M  7.8G   1% /dev/shm
/dev/nvme0n1p1 vfat   1.1G  6.2M  1.1G   1% /boot/efi
/dev/sda1      exfat   29G  4.1G   25G  15% /media/rahim/USB

$ lsblk -f
NAME        FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE%
MOUNTPOINTS
sda
└─sda1      exfat  1.0   USB   6A1B-2C3D                            24.9G    15%
/media/rahim/USB
nvme0n1
├─nvme0n1p1 vfat   FAT32       1A2B-3C4D                             1.1G     1%
/boot/efi
└─nvme0n1p2 ext4   1.0         8f3e2d1c-...                        332.8G    24% /
```

কমান্ড	কাজ	দরকারি অপশন
`findmnt`	মাউন্টগুলো গাছ আকারে	`--real` (শুধু আসল fs), `-t ext4` (টাইপ অনুযায়ী), `/home` (নির্দিষ্ট পাথ কোন fs-এ)
`df`	কোন fs-এ কত জায়গা	`-h` (মানুষের পড়ার মতো সাইজ), `-T` (fs টাইপ), `-i` (inode ব্যবহার)
`lsblk`	ব্লক ডিভাইস ও পার্টিশন	`-f` (fs, UUID, লেবেল), `-o NAME,SIZE,TYPE,MOUNTPOINTS`
`mount`	মাউন্ট করা / তালিকা	(বিস্তারিত অধ্যায় ৩২)

৭.৭ usr-merge — `/bin` কেন `/usr/bin`-এর লিংক?

সমস্যার ইতিহাস

১৯৭১ সাল নাগাদ Unix-এর আবিষ্কারকদের (Ken Thompson, Dennis Ritchie) প্রথম ডিস্কটা ভরে গিয়েছিল। তাই তারা দ্বিতীয় ডিস্ক লাগিয়ে তার নাম দিলেন `/usr` (তখন ইউজারদের হোম সেখানে থাকত), আর প্রথম ডিস্কের কিছু প্রোগ্রাম সেখানে সরালেন। প্রথম ডিস্কে শুধু বুটের জন্য একেবারে দরকারি জিনিস (`/bin` , `/sbin` , `/lib`) রাখা হলো। এই বিভাজনটা ছিল **হার্ডওয়্যারের সীমাবদ্ধতা** — কোনো সুচিন্তিত নকশা না। কিন্তু অভ্যাসটা ৫০ বছর টিকে গেল।

আজ কেন এর দরকার নেই

- এখন initramfs (৭.৫.১১) বুটের শুরুতেই `/usr` মাউন্ট করে ফেলে, তাই `/` আর `/usr`-এর আলাদা "জরুরি" ভাগ অর্থহীন।
- কোন প্রোগ্রাম `/bin`-এ আর কোনটা `/usr/bin`-এ — এই নিয়ে ডিস্ট্রোগুলোর মধ্যে বিভ্রান্তি ছিল; স্ক্রিপ্টে `#!/bin/bash` বনাম `#!/usr/bin/bash` নিয়ে ঝামেলা হতো।
- সব অপারেটিং-সিস্টেম কোড এক জায়গায় (`/usr`) থাকলে স্ন্যাপশট নেওয়া, রিড-অনলি রাখা, শেয়ার করা অনেক সহজ।

সমাধান: usr-merge

সব কিছু `/usr`-এ সরিয়ে পুরনো জায়গায় **symbolic link** রেখে দেওয়া হয়েছে — যাতে পুরনো পাথ ব্যবহার করা স্ক্রিপ্টও কাজ করে।

```
  usr-merge-এর আগে                    usr-merge-এর পরে
  ──────────────────                   ──────────────────
  /bin/ls       (আসল ফাইল)            /bin   ──→ usr/bin   (লিংক)
  /usr/bin/git  (আসল ফাইল)            /sbin  ──→ usr/sbin  (লিংক)
  /lib/libc...  (আসল ফাইল)            /lib   ──→ usr/lib   (লিংক)
                                       /lib64 ──→ usr/lib64 (লিংক)
                                       /usr/bin/ls, /usr/bin/git  (সব আসল ফাইল এক
জায়গায়)
```

ডিস্ট্রো	usr-merge কবে থেকে
Fedora	১৭ (২০১২) — প্রথম বড় ডিস্ট্রো
RHEL / CentOS	৭ থেকে
Arch Linux	২০১৩
Ubuntu	নতুন ইনস্টলে ১৯.০৪ থেকে; ২৪.০৪-এ সম্পূর্ণ বাধ্যতামূলক
Debian	নতুন ইনস্টলে ১০ থেকে; ১২ (bookworm)-এ বাধ্যতামূলক

```
$ ls -ld /bin /sbin /lib
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /lib -> usr/lib
lrwxrwxrwx 1 root root 8 Apr 22  2024 /sbin -> usr/sbin
$ readlink -f /bin/ls
/usr/bin/ls
```

> 📝 **মনে রাখো:** `/bin/bash` আর `/usr/bin/bash` আধুনিক সিস্টেমে **একই ফাইল**। তাই `#!/bin/bash` লিখলেও চলবে। পোর্টেবল স্ক্রিপ্টের জন্য অনেকে `#!/usr/bin/env bash` লেখে (অধ্যায় ৪১)।

> 💡 **টিপস:** লিংক কী, `readlink` কী — সব অধ্যায় ১১-এ গভীরভাবে।

৭.৮ Windows-এর ড্রাইভ লেটার বনাম লিনাক্সের একক গাছ

Windows থেকে আসা মানুষের সবচেয়ে বড় প্রশ্ন: "আমার C: আর D: ড্রাইভ কোথায়?"

```
   Windows: অনেকগুলো আলাদা গাছ                    Linux: একটাই গাছ
   ──────────────────────────────                   ──────────────────

   C:\                  D:\         E:\             /
   ├─ Windows           ├─ Movies   ├─ (USB)         ├─ usr, etc, ...   (SSD পার্টিশন ১)
   ├─ Program Files     └─ Songs    └─ photos        ├─ home            (চাইলে পার্টিশন ২)
   └─ Users                                          ├─ data            (D-এর সমতুল্য — যেখানে
খুশি)
      └─ Rahim                                       └─ media/rahim/USB (USB)
```

বিষয়	Windows	Linux
গাছের সংখ্যা	প্রতি ড্রাইভে আলাদা (`C:\`, `D:\`)	একটাই, গোড়া `/`
নতুন ডিস্ক কোথায় দেখায়	নতুন অক্ষর পায় (`E:`)	যেকোনো ডিরেক্টরিতে মাউন্ট হয়
পাথ বিভাজক	`\` (ব্যাকস্ল্যাশ)	`/` (ফরোয়ার্ড স্ল্যাশ)
বড়/ছোট হাতের অক্ষর	সাধারণত case-**insensitive** (NTFS case সংরক্ষণ করে কিন্তু খোঁজে না)	case-**sensitive**
প্রোগ্রাম কোথায়	`C:\Program Files\<অ্যাপ>\` (সব এক ফোল্ডারে)	ভাগ করে: binary `/usr/bin`, লাইব্রেরি `/usr/lib`, কনফিগ `/etc`, ডেটা `/usr/share`
ইউজারের ফোল্ডার	`C:\Users\Rahim`	`/home/rahim`
সিস্টেম সেটিং	Registry (বাইনারি ডাটাবেস)	`/etc`-এর টেক্সট ফাইল
লুকানো ফাইল	ফাইলের "hidden" অ্যাট্রিবিউট	নামের শুরুতে `.`
ফাইল টাইপ চেনা	এক্সটেনশন (`.exe`, `.docx`) দিয়ে	কনটেন্ট ও পারমিশন দিয়ে; এক্সটেনশন শুধু রীতি

লিনাক্স পদ্ধতির সুবিধা:

- ডিস্ক বদলালে পাথ বদলায় না। `/home`-কে নতুন বড় ডিস্কে সরালেও সব প্রোগ্রাম `/home/rahim` দিয়েই খুঁজবে।
- নেটওয়ার্ক ড্রাইভ, RAM ডিস্ক, পেনড্রাইভ — সবই একইভাবে গাছের অংশ।

> 💡 **টিপস:** লিনাক্স থেকে Windows পার্টিশন পড়তে চাইলে সেটাও মাউন্ট হয় — যেমন `/media/rahim/Windows` বা `/mnt/c`। (WSL-এ Windows-এর `C:` ড্রাইভ `/mnt/c`-তে দেখা যায়।)

৭.৯ Case sensitivity — বড় হাত, ছোট হাত আলাদা

লিনাক্সে `Report.txt`, `report.txt`, `REPORT.TXT` — **তিনটা সম্পূর্ণ আলাদা ফাইল**। একই ডিরেক্টরিতে তিনটাই একসাথে থাকতে পারে।

```
$ touch report.txt Report.txt REPORT.TXT
$ ls
REPORT.TXT  Report.txt  report.txt
$ cat REPORT.txt
cat: REPORT.txt: No such file or directory
```

কমান্ডও case-sensitive:

```
$ LS
LS: command not found
$ ls
REPORT.TXT  Report.txt  report.txt
```

কেন এমন? লিনাক্সের ext4-এর মতো ফাইল সিস্টেম ফাইলের নামকে শুধু **বাইটের সারি** হিসেবে দেখে। `R` (বাইট 0x52) আর `r` (বাইট 0x72) আলাদা বাইট, তাই আলাদা নাম। এতে kernel-কে কোনো ভাষার "বড় হাত-ছোট হাত" নিয়ম জানতে হয় না — দ্রুত ও সহজ।

ফাইল নামের নিয়ম:

- নামে `/` আর null বাইট (`\0`) ছাড়া প্রায় সব অক্ষর চলে — বাংলা নামও! (`নোট.txt`)
- সাধারণত সর্বোচ্চ ২৫৫ বাইট নাম (UTF-8-এ বাংলা অক্ষর ৩ বাইট করে নেয়, তাই বাংলায় প্রায় ৮৫টা অক্ষর)।
- পূর্ণ পাথের সীমা সাধারণত ৪০৯৬ বাইট।

> ⚠️ **সতর্কতা:** নামে স্পেস, `*`, `?`, `$`, `!`, `&`, `;` বা শুরুতে `-` দিলে টার্মিনালে ঝামেলা হয়। যেমন `my file.txt`-কে `rm my file.txt` লিখলে শেল দুটো আলাদা ফাইল ভাববে: `my` আর `file.txt`! সঠিক: `rm "my file.txt"` বা `rm my\ file.txt`। ভালো অভ্যাস: নামে ছোট হাতের ইংরেজি, সংখ্যা, `-`, `_`, `.` ব্যবহার করো — `my-file.txt`।

> 💡 **টিপস (অ্যাডভান্সড):** kernel 5.2 থেকে ext4 আর পরে F2FS-এ নির্দিষ্ট ডিরেক্টরির জন্য **casefold** (case-insensitive) সুবিধা চালু করা যায় (`chattr +F`, তবে ফাইল সিস্টেম `-O casefold` দিয়ে তৈরি

> হতে হবে)। মূলত Android ও Windows গেম (Steam/Proton) চালানোর সুবিধার জন্য। সাধারণ ব্যবহারে দরকার পড়ে না। আর FAT32/exFAT পেনড্রাইভ লিনাক্সেও case-insensitive আচরণ করে, কারণ সেটা ওই ফাইল সিস্টেমের নিজের নিয়ম।

৭.১০ Hidden files — ডট দিয়ে শুরু হওয়া ফাইল

লিনাক্সে যে ফাইল বা ডিরেক্টরির নাম `.` (ডট) দিয়ে শুরু, সেটা **hidden (লুকানো)**। কোনো বিশেষ অ্যাট্রিবিউট নেই — শুধু নামের রীতি। `ls` আর ফাইল ম্যানেজার ডিফল্টভাবে এগুলো দেখায় না।

কেন? ইউজারের হোমে শত শত অ্যাপ নিজের সেটিং রাখে — `.bashrc`, `.ssh/`, `.config/`, `.cache/`। এগুলো সবসময় চোখের সামনে থাকলে আসল ফাইল খুঁজে পাওয়া কঠিন। (মজার ইতিহাস: এটা আসলে প্রথম দিকের `ls`-এর একটা "বাগ" ছিল, যেটা `.` আর `..` লুকাতে গিয়ে ডট দিয়ে শুরু সব নাম লুকিয়ে ফেলত। পরে সেটাই রীতি হয়ে গেল।)

```
$ ls ~
Desktop  Documents  Downloads  Music  Pictures

$ ls -a ~
.   .bash_history  .bashrc  .config  .local    .ssh     Documents  Music
..  .bash_logout   .cache   .gnupg   .profile  Desktop  Downloads  Pictures

$ ls -A ~
.bash_history  .bashrc  .config  .local    .ssh     Documents  Music
.bash_logout   .cache   .gnupg   .profile  Desktop  Downloads  Pictures
```

অপশন	কাজ
`ls -a`	সব দেখায়, `.` আর `..` সহ (all)
`ls -A`	সব দেখায়, কিন্তু `.` আর `..` বাদে (almost all)
`ls -d .*`	শুধু hidden গুলো (ডিরেক্টরির ভেতরে না ঢুকে)

গুরুত্বপূর্ণ hidden ফাইল/ডিরেক্টরি:

নাম	কী	বিস্তারিত
`~/.bashrc`	bash শেলের ব্যক্তিগত সেটিং	অধ্যায় ৪৪
`~/.profile` / `~/.bash_profile`	লগইন শেলের সেটিং	অধ্যায় ৪৪
`~/.bash_history`	তোমার চালানো কমান্ডের ইতিহাস	অধ্যায় ৬
`~/.ssh/`	SSH কী — অত্যন্ত গোপনীয়	অধ্যায় ৩৮
`~/.config/`	আধুনিক অ্যাপের সেটিং (XDG স্ট্যান্ডার্ড)	—
`~/.cache/`	অ্যাপের ক্যাশ; মুছলে ক্ষতি নেই	—

নাম	কী	বিস্তারিত
`~/.local/share/`	অ্যাপের ব্যক্তিগত ডেটা; `~/.local/bin` ইউজারের নিজের স্ক্রিপ্ট	—

> 📝 **মনে রাখো:** ফাইল লুকাতে চাইলে নাম বদলে সামনে ডট দাও: `mv notes.txt .notes.txt`। কিন্তু এটা **নিরাপত্তা না** — যে কেউ `ls -a` দিয়ে দেখতে পারে। গোপনীয়তার জন্য পারমিশন লাগে (অধ্যায় ১৬)।

> ⚠️ **সতর্কতা:** `rm -rf .*` কখনো লিখবে না! পুরনো শেলে `.*` প্যাটার্ন `..`-কেও ধরতে পারত, অর্থাৎ প্যারেন্ট ডিরেক্টরিও। (আধুনিক bash 5.2+ ডিফল্টভাবে `.` ও `..` বাদ দেয়, আর `rm` নিজেও `.` / `..` মুছতে অস্বীকার করে — তবু অভ্যাসটাই বিপজ্জনক।)

৭.১১ সাত ধরনের ফাইল — `ls -l`-এ চেনা

লিনাক্সে "ফাইল" মানে শুধু ডকুমেন্ট না। মোট **সাত ধরনের** ফাইল আছে, আর `ls -l` আউটপুটের **প্রথম অক্ষর** বলে দেয় কোনটা কী।

```
-rw-r--r-- 1 rahim rahim 1234 Nov 12 10:00 notes.txt
│└──┬──┘ │  │     │     │       │           │
│   │    │  │     │     │       │           └── নাম
│   │    │  │     │     │       └── শেষ পরিবর্তনের সময়
│   │    │  │     │     └── সাইজ (বাইট)
│   │    │  │     └── গ্রুপ
│   │    │  └── মালিক (owner)
│   │    └── হার্ড লিংক সংখ্যা ([অধ্যায় ১১])
│   └── পারমিশন ([অধ্যায় ১৬])
└── ⭐ ফাইল টাইপ (এই অংশের বিষয়)
```

চিহ্ন	টাইপ	কী	উদাহরণ
`-`	**Regular file**	সাধারণ ফাইল — টেক্সট, ছবি, প্রোগ্রাম, ZIP	`/etc/passwd`, `/usr/bin/ls`
`d`	**Directory**	অন্য ফাইলের তালিকা ধারণকারী	`/home`, `/etc`
`l`	**Symbolic link**	অন্য ফাইলের দিকে নির্দেশক (শর্টকাটের মতো)	`/bin -> usr/bin`
`b`	**Block device**	ব্লক আকারে (একসাথে অনেক বাইট) ডেটা আদান-প্রদান, যেকোনো জায়গায় লাফানো যায়	`/dev/sda`, `/dev/nvme0n1`
`c`	**Character device**	বাইট-বাই-বাইট স্ট্রিম	`/dev/null`, `/dev/tty`, `/dev/urandom`
`s`	**Socket**	প্রসেসগুলোর মধ্যে দুই-দিকের যোগাযোগের প্রান্ত	`/run/dbus/system_bus_socket`

চিহ্ন	টাইপ	কী	উদাহরণ
`p`	**Named pipe (FIFO)**	এক প্রসেস লেখে, আরেক প্রসেস পড়ে — একমুখী পাইপ	`mkfifo` দিয়ে বানানো

প্রতিটার বাস্তব উদাহরণ

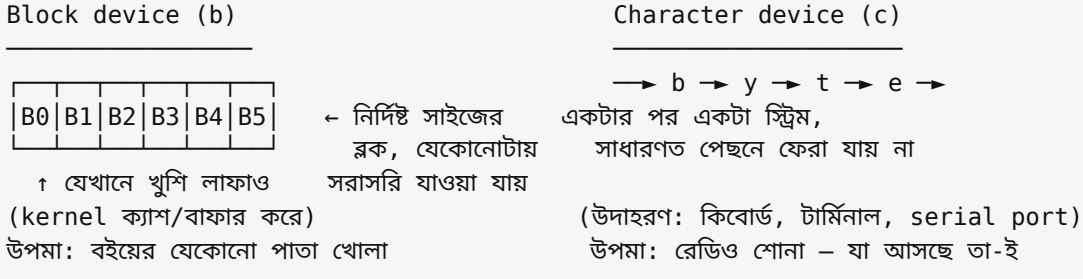
```
$ ls -l /etc/hostname
-rw-r--r-- 1 root root 7 Apr 25  2024 /etc/hostname

$ ls -ld /etc
drwxr-xr-x 140 root root 12288 Nov 11 22:40 /etc

$ ls -l /bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin

$ ls -l /dev/sda /dev/nvme0n1
brw-rw---- 1 root disk 8, 0 Nov 12 08:00 /dev/sda
brw-rw---- 1 root disk 259, 0 Nov 12 08:00 /dev/nvme0n1

$ ls -l /dev/null /dev/tty
crw-rw-rw- 1 root root 1, 3 Nov 12 08:00 /dev/null
crw-rw-rw- 1 root tty  5, 0 Nov 12 09:15 /dev/tty

$ ls -l /run/dbus/system_bus_socket
srw-rw-rw- 1 root root 0 Nov 12 08:00 /run/dbus/system_bus_socket

$ mkfifo /tmp/mypipe
$ ls -l /tmp/mypipe
prw-r--r-- 1 rahim rahim 0 Nov 12 10:05 /tmp/mypipe
```

খেয়াল করো, device ফাইলে সাইজের জায়গায় `8, 0` বা `1, 3` — এগুলো **major, minor নম্বর**:

- **Major নম্বর** বলে কোন **driver** (যেমন 8 = SCSI/SATA ডিস্ক ড্রাইভার, 1 = মেমরি ডিভাইস)।
- **Minor নম্বর** বলে সেই driver-এর কোন নির্দিষ্ট ডিভাইস (sda = 0, sda1 = 1, sdb = 16...)।

উপমা: major নম্বর হলো "কোন হাসপাতাল", minor নম্বর "কোন বেড"।

Block বনাম Character — পার্থক্যটা কোথায়?



Pipe-এর জাদু — হাতে-কলমে

একটা টার্মিনালে:

```
$ cat /tmp/mypipe
```

(এটা অপেক্ষা করবে — কেউ লেখেনি এখনো।)

আরেকটা টার্মিনালে:

```
$ echo "হ্যালো পাইপ!" > /tmp/mypipe
```

প্রথম টার্মিনালে সাথে সাথে দেখাবে:

```
হ্যালো পাইপ!
```

ডেটা আসলে ডিস্কে লেখা হয়নি — kernel-এর মেমরি দিয়ে সরাসরি এক প্রসেস থেকে আরেক প্রসেসে গেছে। (সাধারণ `|` পাইপ এর নামহীন সংস্করণ — অধ্যায় ১৯।)

> 📝 **মনে রাখো:** Socket আর pipe ফাইলের সাইজ সবসময় `0`, কারণ এগুলো ডেটা জমিয়ে রাখে না — শুধু যোগাযোগের "দরজা"।

`ls -F` আর রঙ দিয়ে চেনা

```
$ ls -F /etc | head -5
adduser.conf
alternatives/
apparmor/
apparmor.d/
apport/
```

`ls -F` চিহ্ন	মানে
`/`	ডিরেক্টরি
`*`	চালানোযোগ্য (executable) ফাইল
`@`	symbolic link
`|`	named pipe (FIFO)
`=`	socket
(কিছু না)	সাধারণ ফাইল

Ubuntu-তে `ls` ডিফল্টভাবে রঙিন (alias `ls --color=auto`): সাধারণত **নীল = ডিরেক্টরি, সায়ান/আকাশি = লিংক, সবুজ = executable, হলুদ = device, ম্যাজেন্টা/গোলাপি = socket/ছবি, লাল =**

আর্কাইভ বা ভাঙা লিংক। রঙ `LS_COLORS` ভেরিয়েবল দিয়ে বদলানো যায়, তাই সব সিস্টেমে একই নাও হতে পারে।

`stat` — একটা ফাইলের পূর্ণ পরিচয়পত্র

```
$ stat /etc/hostname
  File: /etc/hostname
  Size: 7               Blocks: 8          IO Block: 4096   regular file
Device: 259,2   Inode: 1310742     Links: 1
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-11-12 08:00:41.123456789 +0600
Modify: 2024-04-25 11:20:03.000000000 +0600
Change: 2024-04-25 11:20:03.000000000 +0600
 Birth: 2024-04-25 11:20:03.000000000 +0600

$ stat -c '%n -> %F' /etc /bin /dev/sda /dev/null
/etc -> directory
/bin -> symbolic link
/dev/sda -> block special file
/dev/null -> character special file
```

`stat` অপশন	কাজ
`-c FORMAT`	নিজের ফরম্যাটে আউটপুট: `%n` নাম, `%F` টাইপ, `%s` সাইজ, `%i` inode, `%a` পারমিশন (সংখ্যায়), `%U` মালিক
`-L`	লিংক অনুসরণ করে আসল ফাইলের তথ্য
`-f`	ফাইলের বদলে ফাইল সিস্টেমের তথ্য

inode আর Access/Modify/Change সময়ের গভীর মানে অধ্যায় ১১-এ।

৭.১২ `file` কমান্ড — ফাইলের আসল পরিচয় বের করা

Windows-এ ফাইলের টাইপ ঠিক হয় এক্সটেনশন দিয়ে। লিনাক্সে **এক্সটেনশন শুধু একটা রীতি** — `photo.jpg` নামের ফাইল আসলে টেক্সট হতে পারে, আর এক্সটেনশন ছাড়া ফাইলও প্রোগ্রাম হতে পারে। তাই আসল টাইপ জানতে `file` কমান্ড।

`file` কীভাবে কাজ করে (under the hood)

`file` তিন ধাপে পরীক্ষা করে, এবং যে ধাপে প্রথম উত্তর পায় সেটাই দেখায়:

```
১. Filesystem test   → stat() দিয়ে: directory? link? device? socket? খালি?
        │ (regular file হলে পরের ধাপে)
        ▼
২. Magic test        → ফাইলের প্রথম কিছু বাইট "magic number"-এর ডাটাবেসের সাথে মেলায়
        │              (যেমন ELF প্রোগ্রাম শুরু হয় 7F 45 4C 46 = "\x7FELF" দিয়ে,
```

```
          │              PNG শুরু হয় 89 50 4E 47, PDF শুরু হয় "%PDF")
          ▼
  ৩. Language test     → টেক্সট হলে কোন এনকোডিং (ASCII/UTF-8), কোন ভাষার স্ক্রিপ্ট
```

উপমা: ডাক্তার রোগীর নাম (এক্সটেনশন) দেখে রোগ নির্ণয় করে না — রক্ত পরীক্ষা (magic bytes) করে।

Syntax

```
file [অপশন] ফাইল...
```

গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-b`	brief — ফাইলের নাম ছাড়া শুধু টাইপ
`-i`	MIME টাইপ দেখায় (যেমন `text/plain; charset=utf-8`)
`--mime-type`	শুধু MIME টাইপ, charset ছাড়া
`-L`	symbolic link অনুসরণ করে আসল ফাইলের টাইপ
`-h`	লিংক অনুসরণ করে না (ডিফল্ট, যদি `POSIXLY_CORRECT` সেট না থাকে)
`-s`	block/char device-এর ভেতরের ডেটা পড়ে (যেমন পার্টিশনে কোন fs) — root লাগে
`-z`	কম্প্রেস করা ফাইলের ভেতরে দেখে
`-f FILE`	FILE-এ লেখা তালিকা থেকে ফাইলের নাম নেয়

উদাহরণ

```
$ file /etc/passwd
/etc/passwd: ASCII text

$ file /usr/bin/ls
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=..., for GNU/Linux
3.2.0, stripped

$ file /bin
/bin: symbolic link to usr/bin

$ file -L /bin
/bin: directory

$ file /dev/sda /dev/null
/dev/sda:  block special (8/0)
/dev/null: character special (1/3)

$ file /run/dbus/system_bus_socket
/run/dbus/system_bus_socket: socket

$ file /tmp/mypipe
```

```
/tmp/mypipe: fifo (named pipe)

$ file /usr/bin/zgrep
/usr/bin/zgrep: POSIX shell script, ASCII text executable

$ file -i ~/নোট.txt
/home/rahim/নোট.txt: text/plain; charset=utf-8

$ sudo file -s /dev/nvme0n1p2
/dev/nvme0n1p2: Linux rev 1.0 ext4 filesystem data, UUID=8f3e2d1c-..., (extents)
(64bit) (large files) (huge files)
```

এক্সটেনশনের ধোঁকা ধরো

```
$ echo "আমি আসলে টেক্সট" > fake.jpg
$ file fake.jpg
fake.jpg: Unicode text, UTF-8 text
```

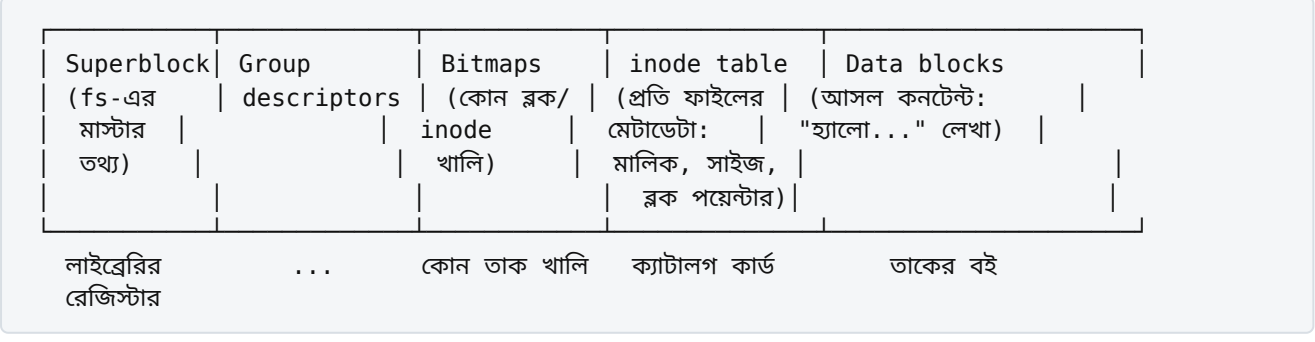
> 💡 **টিপস:** ইন্টারনেট থেকে নামানো সন্দেহজনক ফাইল খোলার আগে `file` দিয়ে দেখে নাও। "invoice.pdf" যদি `ELF executable` বা `shell script` দেখায় — সাবধান!

> 📝 **মনে রাখো:** `file` টেক্সটের "টাইপ" অনুমান করে — শতভাগ নিশ্চিত না। magic ডাটাবেস থাকে `/usr/share/misc/magic` বা `/usr/share/file/magic*` -এ।

৭.১৩ ডিস্ক ফরম্যাট: ext4, XFS, Btrfs পরিচিতি

এবার "ফাইল সিস্টেম"-এর দ্বিতীয় অর্থ — ডিস্কে ডেটা সাজানোর পদ্ধতি। ফরম্যাট করা (`mkfs`), পার্টিশন বানানো, মাউন্ট করার পুরো হাতে-কলমে কাজ অধ্যায় ৩২-এ; এখানে শুধু চেনা-জানা।

ভেতরে একটা ফাইল সিস্টেম দেখতে কেমন (সরলীকৃত ext4)



- **Superblock**: ফাইল সিস্টেমের সাইজ, ব্লক সাইজ, অবস্থা — মাস্টার রেজিস্টার (নিরাপত্তার জন্য কয়েকটা কপি থাকে)।

- **inode**: প্রতি ফাইলের "ক্যাটালগ কার্ড" — মালিক, পারমিশন, সাইজ, সময়, ডেটা কোথায় — **নাম বাদে সব** (অধ্যায় ১১)।
- **Directory**: আসলে একটা বিশেষ ফাইল, যার ভেতরে "নাম → inode নম্বর"-এর তালিকা।
- **Journal**: বড় পরিবর্তনের আগে "কী করতে যাচ্ছি" লিখে রাখার খাতা। হঠাৎ বিদ্যুৎ গেলে পরের বুটে journal দেখে ফাইল সিস্টেম দ্রুত সামঞ্জস্যপূর্ণ অবস্থায় ফেরে — পুরো ডিস্ক স্ক্যান করতে হয় না।

তিনটি প্রধান লিনাক্স ফাইল সিস্টেম

বিষয়	ext4	XFS	Btrfs
পূর্ণ নাম	Fourth Extended Filesystem	(SGI-র তৈরি) XFS	B-tree File System ("বাটার এফএস" উচ্চারণ)
জন্ম	২০০৮ (ext2/ext3-এর উত্তরসূরি)	১৯৯৩ (SGI IRIX), লিনাক্সে ২০০১	২০০৯ থেকে উন্নয়ন
ডিফল্ট কোথায়	**Ubuntu, Debian**	**RHEL 9, Rocky, AlmaLinux**	**Fedora Workstation (৩৩ থেকে)**, openSUSE
মূল কৌশল	journaling, extents	journaling, বিশাল ফাইলে দক্ষ, সমান্তরাল I/O	**Copy-on-Write (CoW)**
সবচেয়ে ভালো	সাধারণ ডেস্কটপ/সার্ভার — নির্ভরযোগ্য, পরীক্ষিত	বড় সার্ভার, বিশাল ফাইল, ডাটাবেস	স্ন্যাপশট, রোলব্যাক, কম্প্রেশন
বড় করা (grow)	হ্যাঁ (মাউন্ট অবস্থায়ও)	হ্যাঁ (মাউন্ট অবস্থায়)	হ্যাঁ
ছোট করা (shrink)	হ্যাঁ (আনমাউন্ট করে)	**না**	হ্যাঁ
স্ন্যাপশট	না (LVM দিয়ে করা যায়)	না (LVM দিয়ে)	**হ্যাঁ, বিল্ট-ইন**
স্বচ্ছ কম্প্রেশন	না	না	হ্যাঁ (zstd, lzo, zlib)
ডেটা checksum	শুধু মেটাডেটা	শুধু মেটাডেটা	**ডেটা + মেটাডেটা দুটোই**
মেরামতের টুল	`e2fsck` (`fsck.ext4`)	`xfs_repair`	`btrfs check`, `btrfs scrub`

Copy-on-Write (CoW) — Btrfs-এর জাদু

সাধারণ ফাইল সিস্টেম কোনো ফাইল বদলালে পুরনো ব্লকের ওপরেই নতুন ডেটা লেখে। Btrfs **নতুন জায়গায়** লেখে, তারপর পয়েন্টারটা নতুন ব্লকের দিকে ঘুরিয়ে দেয়:

```
পরিবর্তনের আগে:      ফাইল ──► [A][B][C]
B বদলালাম:              ফাইল ──► [A][B'][C]      (B' নতুন জায়গায় লেখা)
                                      ↑
স্ন্যাপশট ──────────► [A][B][C]   (পুরনো B এখনো আছে, স্ন্যাপশট সেটা দেখায়)
```

উপমা: খাতায় ভুল কেটে ওপরে না লিখে নতুন পাতায় লেখা, আর পুরনো পাতা রেখে দেওয়া। তাই স্ন্যাপশট তাৎক্ষণিক আর প্রায় বিনা খরচে হয় — আপডেটে সমস্যা হলে আগের অবস্থায় ফিরে যাওয়া যায়।

অন্যান্য ফাইল সিস্টেম যেগুলো দেখবে

ফাইল সিস্টেম	কোথায়
vfat (FAT32)	পুরনো পেনড্রাইভ, **EFI System Partition (`/boot/efi`)**; একক ফাইল সর্বোচ্চ ৪ GB-এর একটু কম
exFAT	আধুনিক পেনড্রাইভ, SD কার্ড; kernel 5.4 থেকে নেটিভ সাপোর্ট
NTFS	Windows পার্টিশন; kernel 5.15 থেকে `ntfs3` ড্রাইভার, আগে থেকে `ntfs-3g` (FUSE)
tmpfs	RAM-ভিত্তিক (`/run`, `/dev/shm`, কখনো `/tmp`)
proc, sysfs, devtmpfs	virtual — আগে দেখেছি
swap	আসলে ফাইল সিস্টেম না, RAM-এর বাড়তি জায়গা
ZFS	অত্যন্ত শক্তিশালী CoW fs; লাইসেন্সের কারণে মূল kernel-এ নেই, Ubuntu-তে আলাদা মডিউল হিসেবে পাওয়া যায়

তোমার সিস্টেমে কোনটা ব্যবহার হচ্ছে দেখো:

```
$ df -T /
Filesystem     Type 1K-blocks     Used Available Use% Mounted on
/dev/nvme0n1p2 ext4 490617784 117489020 348117888  26% /

$ findmnt -n -o FSTYPE /
ext4

$ cat /proc/filesystems | head -8
nodev   sysfs
nodev   tmpfs
nodev   bdev
nodev   proc
nodev   cgroup
nodev   cgroup2
nodev   cpuset
nodev   devtmpfs
```

`/proc/filesystems` দেখায় kernel বর্তমানে কোন কোন ফাইল সিস্টেম চেনে; `nodev` মানে এর জন্য কোনো ডিস্ক লাগে না (virtual)।

> 💡 **টিপস:** কোনটা বাছবে? নতুন হলে ডিস্ট্রোর ডিফল্টটাই নাও। ডেস্কটপে স্ন্যাপশট-রোলব্যাক চাইলে Btrfs; সাধারণ সার্ভারে ext4 বা XFS; RHEL-ভিত্তিক এন্টারপ্রাইজে XFS। মনে রেখো XFS ছোট করা যায় না।

৭.১৪ সম্পর্কিত কীবোর্ড শর্টকাট

টার্মিনালে (bash)

শর্টকাট	কাজ
`Tab`	পাথ/ফাইলের নাম অটো-কমপ্লিট (`/u` + Tab → `/usr/`)
`Tab Tab`	একাধিক মিল থাকলে সব সম্ভাব্য নাম দেখায় (`ls /e` + Tab Tab)
`Alt + .`	আগের কমান্ডের শেষ আর্গুমেন্ট (প্রায়ই একটা পাথ) বসায়
`Ctrl + C`	চলমান কমান্ড থামায় (যেমন আটকে থাকা `cat /tmp/mypipe`)
`Ctrl + L`	স্ক্রিন পরিষ্কার
`~` (টাইপ)	হোম ডিরেক্টরির শর্টহ্যান্ড

(পুরো তালিকা: অধ্যায় ৬)

`less` / `man` -এ ফাইল পড়ার সময়

শর্টকাট	কাজ
`Space` / `b`	এক পাতা সামনে / পেছনে
`/শব্দ`	খোঁজা; `n` পরেরটা, `N` আগেরটা
`g` / `G`	শুরুতে / শেষে
`q`	বেরিয়ে আসা

GNOME Files (Nautilus) ফাইল ম্যানেজারে

শর্টকাট	কাজ
`Ctrl + H`	hidden ফাইল দেখানো/লুকানো
`Ctrl + L`	লোকেশন বারে পাথ টাইপ করা (যেমন `/etc` লিখে Enter)
`/`	সরাসরি root থেকে পাথ টাইপ শুরু
`~`	হোম থেকে পাথ টাইপ শুরু
`Alt + ↑`	প্যারেন্ট ডিরেক্টরি
`Alt + ←` / `Alt + →`	পেছনে / সামনে
`Alt + Home`	হোম ডিরেক্টরিতে
`Ctrl + Shift + N`	নতুন ফোল্ডার
`Ctrl + I` বা `Alt + Enter`	Properties (টাইপ, সাইজ, পারমিশন)
`F2`	নাম বদল

> 💡 **টিপস:** KDE-র Dolphin-এও hidden ফাইলের জন্য `Ctrl + H` বা `Alt + .`, আর লোকেশন বারের জন্য `Ctrl + L` কাজ করে। ডেস্কটপ পরিবেশ নিয়ে অধ্যায় ৪।

হাতে-কলমে অনুশীলন

এই ল্যাবগুলো সাধারণ ইউজার হিসেবে করো। যেখানে `sudo` আছে, সেখানে শুধু **পড়া** হচ্ছে — কিছু বদলানো হচ্ছে না।

ল্যাব ১: গাছটা ঘুরে দেখো

```
$ ls /
$ ls -l /
$ tree -L 1 /          # tree না থাকলে: sudo apt install tree
$ tree -d -L 1 /usr
```

প্রশ্ন: তোমার সিস্টেমে `/` এর নিচে কোনগুলো symbolic link? (`ls -l /` আউটপুটে প্রথম অক্ষর `l` খোঁজো।)

ল্যাব ২: usr-merge প্রমাণ করো

```
$ ls -ld /bin /sbin /lib
$ readlink -f /bin/bash
$ ls -i /bin/bash /usr/bin/bash
```

শেষ কমান্ডে দুটো পাথের পাশে **একই inode নম্বর** দেখবে — অর্থাৎ একই ফাইল।

ল্যাব ৩: কনফিগ ফাইল পড়ো

```
$ cat /etc/os-release
$ cat /etc/hostname
$ head -3 /etc/passwd
$ ls /etc | wc -l
$ cat /etc/shadow
```

শেষেরটায় `Permission denied` আসবে — কেন? (উত্তর: এখানে পাসওয়ার্ড hash আছে, শুধু root পড়তে পারে।)

ল্যাব ৪: virtual ফাইল সিস্টেম অনুভব করো

```
$ ls -l /proc/cpuinfo        # সাইজ 0!
$ wc -c /proc/cpuinfo        # অথচ ভেতরে হাজার হাজার বাইট
$ cat /proc/uptime           # কত সেকেন্ড ধরে চালু
$ ls /proc/self/             # এই মুহূর্তের ls প্রসেসের তথ্য
```

```
$ cat /proc/self/status | head -5
$ ls /sys/class/net/         # তোমার নেটওয়ার্ক ইন্টারফেস
$ df -h /run /dev /proc 2>/dev/null
```

ল্যাব ৫: সাত ধরনের ফাইল নিজে খুঁজে বের করো

```
$ cd /tmp
$ mkdir lab7 && cd lab7
$ echo "hello" > regular.txt          # -  regular
$ mkdir mydir                        # d  directory
$ ln -s regular.txt mylink           # l  symbolic link
$ mkfifo mypipe                      # p  named pipe
$ ls -l
$ ls -l /dev/sda /dev/nvme0n1 2>/dev/null   # b  block (যেটা আছে)
$ ls -l /dev/null                    # c  character
$ ls -l /run/dbus/system_bus_socket  # s  socket
$ ls -F
$ stat -c '%-12n %F' *
```

প্রত্যাশিত (আংশিক):

```
$ stat -c '%-12n %F' *
mydir        directory
mylink       symbolic link
mypipe       fifo
regular.txt  regular file
```

> 💡 socket আরো খুঁজতে: `find /run -type s 2>/dev/null | head` (`find` নিয়ে অধ্যায় ১২)।

ল্যাব ৬: `file` দিয়ে গোয়েন্দাগিরি

```
$ file regular.txt mydir mylink mypipe
$ file -L mylink
$ file /usr/bin/ls /usr/bin/zgrep
$ cp "$(ls /usr/share/pixmaps/*.png | head -1)" ./pic; file pic   # এক্সটেনশন ছাড়া ছবি
$ echo "not an image" > fake.png && file fake.png
$ file -i regular.txt
$ file -b --mime-type /usr/bin/ls
```

ল্যাব ৭: case sensitivity আর hidden file

```
$ touch Data.txt data.txt DATA.txt
$ ls
$ touch .secret
$ ls
$ ls -a
$ ls -A
```

GUI ফাইল ম্যানেজারে `/tmp/lab7` খুলে `Ctrl + H` চাপো — `.secret` দেখা যাবে।

ল্যাব ৮: পাইপের জাদু (দুটো টার্মিনাল)

টার্মিনাল ১:

```
$ cat /tmp/lab7/mypipe
```

টার্মিনাল ২:

```
$ date > /tmp/lab7/mypipe
```

টার্মিনাল ১-এ তারিখ চলে আসবে।

ল্যাব ৯: তোমার ডিস্ক আর মাউন্ট

```
$ lsblk -f
$ findmnt --real
$ df -hT
$ cat /proc/filesystems | grep -v nodev
```

প্রশ্ন: তোমার `/` কোন ফাইল সিস্টেমে (ext4/xfs/btrfs)? `/boot/efi` আছে কি (অর্থাৎ UEFI সিস্টেম কি)?

ল্যাব ১০: পরিষ্কার করো

```
$ cd ~
$ rm -r /tmp/lab7
```

সারসংক্ষেপ

ডিরেক্টরি দ্রুত রেফারেন্স

ডিরেক্টরি	এক কথায়
`/`	সব কিছুর গোড়া
`/bin` , `/sbin` , `/lib` , `/lib64`	আধুনিক সিস্টেমে `/usr/...` -এর লিংক (usr-merge)
`/usr`	প্যাকেজ ম্যানেজারের ইনস্টল করা প্রোগ্রাম, লাইব্রেরি, ডকুমেন্টেশন
`/usr/local`	নিজে কম্পাইল/ইনস্টল করা সফটওয়্যার
`/opt`	স্বয়ংসম্পূর্ণ তৃতীয়-পক্ষ প্যাকেজ
`/etc`	সিস্টেম কনফিগারেশন (টেক্সট)

ডিরেক্টরি	এক কথায়
`/home/<user>`	সাধারণ ইউজারের ফাইল (`~`)
`/root`	root ইউজারের হোম
`/var`	লগ, ক্যাশ, ডাটাবেস, কিউ — বদলাতে-থাকা ডেটা
`/tmp` , `/var/tmp`	অস্থায়ী (প্রথমটা রিবুটে মোছে, দ্বিতীয়টা টিকে থাকে)
`/boot`	kernel, initramfs, GRUB, `/boot/efi`
`/dev`	ডিভাইস ফাইল (devtmpfs + udev)
`/proc`	প্রসেস ও kernel তথ্য (virtual)
`/sys`	হার্ডওয়্যার/ড্রাইভার তথ্য (virtual)
`/run`	রানটাইম PID/socket/lock (tmpfs)
`/mnt`	হাতে অস্থায়ী মাউন্ট
`/media`	পেনড্রাইভের স্বয়ংক্রিয় মাউন্ট (Fedora-তে `/run/media`)
`/srv`	সার্ভিসের পরিবেশিত ডেটা

ফাইল টাইপ

`ls -l` অক্ষর	`ls -F` চিহ্ন	টাইপ
`-`	(নেই) বা `*`	regular file
`d`	`/`	directory
`l`	`@`	symbolic link
`b`	—	block device
`c`	—	character device
`s`	`=`	socket
`p`	`|`	named pipe (FIFO)

কমান্ড

কমান্ড	কাজ
`ls /` , `ls -l /`	root-এর সব ডিরেক্টরি
`ls -a` / `ls -A`	hidden সহ সব
`ls -F`	টাইপ চিহ্ন সহ
`ls -ld DIR`	ডিরেক্টরির নিজের তথ্য (ভেতরের নয়)
`tree -L N DIR`	গাছ আকারে N স্তর
`file FILE`	কনটেন্ট দেখে আসল টাইপ

কমান্ড	কাজ
`file -i` / `-L` / `-s`	MIME / লিংক অনুসরণ / device-এর ভেতর
`stat FILE` / `stat -c '%F'`	পূর্ণ মেটাডেটা / শুধু টাইপ
`readlink -f PATH`	লিংকের চূড়ান্ত আসল পথ
`mkfifo NAME`	named pipe তৈরি
`ldd PROGRAM`	প্রোগ্রামের লাইব্রেরি নির্ভরতা
`df -hT`	মাউন্ট, fs টাইপ, খালি জায়গা
`lsblk -f`	ডিস্ক, পার্টিশন, fs
`findmnt`	মাউন্টের গাছ
`man hier`, `man file-hierarchy`	অফিসিয়াল FHS বিবরণ

শর্টকাট

শর্টকাট	কোথায়	কাজ
`Tab` / `Tab Tab`	টার্মিনাল	পাথ অটো-কমপ্লিট / সব বিকল্প
`Alt + .`	টার্মিনাল	আগের কমান্ডের শেষ আর্গুমেন্ট
`Ctrl + H`	ফাইল ম্যানেজার	hidden ফাইল দেখাও/লুকাও
`Ctrl + L`	ফাইল ম্যানেজার	পাথ টাইপ করো
`Alt + ↑`	ফাইল ম্যানেজার	প্যারেন্ট ডিরেক্টরি
`Alt + Home`	ফাইল ম্যানেজার	হোম

নিজেকে যাচাই করো

1. লিনাক্সে কয়টা "ডিরেক্টরি গাছ" থাকে? Windows-এর `D:` ড্রাইভের সমতুল্য জিনিস লিনাক্সে কীভাবে দেখা যায়?
2. Apache/Nginx-এর কনফিগারেশন ফাইল কোথায় খুঁজবে, আর তাদের লগ কোথায়?
3. `/tmp` আর `/var/tmp`-এর পার্থক্য কী?
4. আধুনিক Ubuntu-তে `/bin/ls` আর `/usr/bin/ls` কি আলাদা ফাইল? কেন?
5. `ls -l` আউটপুটে `crw-rw-rw- 1 root root 1, 3 ... /dev/null` — প্রথম অক্ষর `c` আর `1, 3` মানে কী?
6. `/proc/cpuinfo`-এর সাইজ `0` দেখায় অথচ `cat` করলে অনেক লেখা আসে — কেন?
7. একটা ফাইলের নাম `invoice.pdf`, কিন্তু তুমি সন্দেহ করছ এটা আসলে প্রোগ্রাম। কোন কমান্ডে যাচাই করবে?
8. নিজে সোর্স থেকে কম্পাইল করা সফটওয়্যার `/usr/bin`-এ না রেখে `/usr/local/bin`-এ রাখা উচিত কেন?

9. root ইউজারের হোম `/home/root` না হয়ে `/root` কেন?
10. RHEL 9 সার্ভারের ডিফল্ট ফাইল সিস্টেম কোনটা, এবং সেটার একটা সীমাবদ্ধতা কী? Fedora Workstation-এর ডিফল্টের বিশেষ সুবিধা কী?

<details>
<summary>▼ উত্তর দেখো</summary>

1. **একটাই** গাছ, গোড়া `/`। অন্য ডিস্ক বা পার্টিশন গাছের কোনো ডিরেক্টরিতে **mount** হয় — যেমন `/data`, `/home`, বা পেনড্রাইভ হলে `/media/<user>/<label>`। আলাদা ড্রাইভ লেটার নেই।
2. কনফিগ `/etc`-এ (যেমন `/etc/nginx/`, `/etc/apache2/` বা RHEL-এ `/etc/httpd/`); লগ `/var/log`-এ (যেমন `/var/log/nginx/`)। systemd জার্নালেও থাকতে পারে (অধ্যায় ২৮)।
3. `/tmp`-এর ফাইল সাধারণত রিবুটে মুছে যায় (অনেক ডিস্ট্রোতে এটা RAM-ভিত্তিক tmpfs); `/var/tmp`-এর ফাইল রিবুটের পরও থাকে, সাধারণত ৩০ দিন পুরনো হলে মোছে।
4. না, **একই ফাইল**। usr-merge-এর পর `/bin` হলো `usr/bin`-এর দিকে symbolic link, তাই `/bin/ls` আসলে `/usr/bin/ls`-এ পৌঁছায় (`ls -i` দিলে একই inode)।
5. `c` = **character device** (বাইট-বাই-বাইট স্ট্রিম)। `1` = major নম্বর (কোন driver — memory devices), `3` = minor নম্বর (সেই driver-এর কোন ডিভাইস — null)।
6. `/proc` একটা **virtual file system (procfs)** — ডিস্কে কিছু জমা নেই। পড়ার মুহূর্তে kernel তাৎক্ষণিকভাবে লেখা তৈরি করে দেয়, তাই আগে থেকে সাইজ জানা যায় না।
7. `file invoice.pdf` — এটা এক্সটেনশন না দেখে ফাইলের শুরুর বাইট (magic number) পরীক্ষা করে। আসল PDF হলে `PDF document`, প্রোগ্রাম হলে `ELF ... executable` বা `shell script` দেখাবে।
8. `/usr/bin` প্যাকেজ ম্যানেজারের এলাকা — আপডেটে তোমার ফাইল ওভাররাইট বা মুছে যেতে পারে, আর কোন ফাইল কার তা গুলিয়ে যায়। `/usr/local`-এ প্যাকেজ ম্যানেজার হাত দেয় না, আর PATH-এ এটা আগে থাকায় তোমার ভার্সনই চলবে।
9. `/home` প্রায়ই আলাদা পার্টিশন বা নেটওয়ার্ক মাউন্ট হয়। সেটা মাউন্ট না হলে বা নষ্ট হলেও root যেন লগইন করে সিস্টেম মেরামত করতে পারে, তাই root-এর হোম মূল `/` পার্টিশনে।
10. RHEL 9-এর ডিফল্ট **XFS** — এটাকে বড় করা যায় কিন্তু **ছোট (shrink) করা যায় না**। Fedora Workstation-এর ডিফল্ট **Btrfs** — Copy-on-Write-এর কারণে বিল্ট-ইন তাৎক্ষণিক **স্ন্যাপশট**, স্বচ্ছ কম্প্রেশন আর ডেটা checksum দেয়।

</details>

অধ্যায় ৮: ফাইল সিস্টেমে চলাফেরা

আগের অধ্যায়ে (অধ্যায় ৭) তুমি দেখেছ লিনাক্সের ফাইল সিস্টেম একটা উল্টো গাছের মতো সাজানো। গোড়ায় `/` (root), তার নিচে `/home`, `/etc`, `/var` ইত্যাদি। এখন প্রশ্ন হলো, এই গাছের ভেতরে ঘুরব কীভাবে? এক ফোল্ডার থেকে আরেক ফোল্ডারে যাব কীভাবে, আর কোন ফোল্ডারে কী আছে দেখব কীভাবে?

GUI-তে তুমি ফোল্ডারে ডাবল-ক্লিক করো। টার্মিনালে সেই কাজ হয় কয়েকটা ছোট কমান্ড দিয়ে: `pwd`, `cd`, `ls`। এগুলো তুমি দিনে শত শত বার চালাবে। তাই এই অধ্যায়টা ভালোভাবে শিখলে বাকি বই অনেক সহজ লাগবে।

এই অধ্যায়ে যা শিখবে

- "Current working directory" কী, আর shell কীভাবে মনে রাখে তুমি কোথায় আছ
- `pwd` দিয়ে নিজের অবস্থান জানা (`-L` বনাম `-P`)
- `cd` এর সব রূপ: `cd`, `cd ~`, `cd -`, `cd ..`, `cd /`, `cd ~user`
- Absolute path আর relative path এর পার্থক্য
- `.` আর `..` আসলে কী (এগুলো ফাইল সিস্টেমের সত্যিকারের এন্ট্রি)
- `~` (tilde) expansion কীভাবে কাজ করে
- `ls` এর গুরুত্বপূর্ণ অপশন: `-l -a -A -h -t -r -S -R -d -i -F -1 --color` সহ আরও
- `ls -l` আউটপুটের প্রতিটি কলামের মানে
- `tree` দিয়ে গাছের মতো করে ডিরেক্টরি দেখা
- `pushd`, `popd`, `dirs` দিয়ে ডিরেক্টরি stack ব্যবহার
- Wildcard/globbing (`*`, `?`, `[]`) আর brace expansion (`{}`)
- Tab completion এর কৌশল আর নেভিগেশন সংক্রান্ত সব শর্টকাট

৮.১ তুমি এখন কোথায়? — Current Working Directory

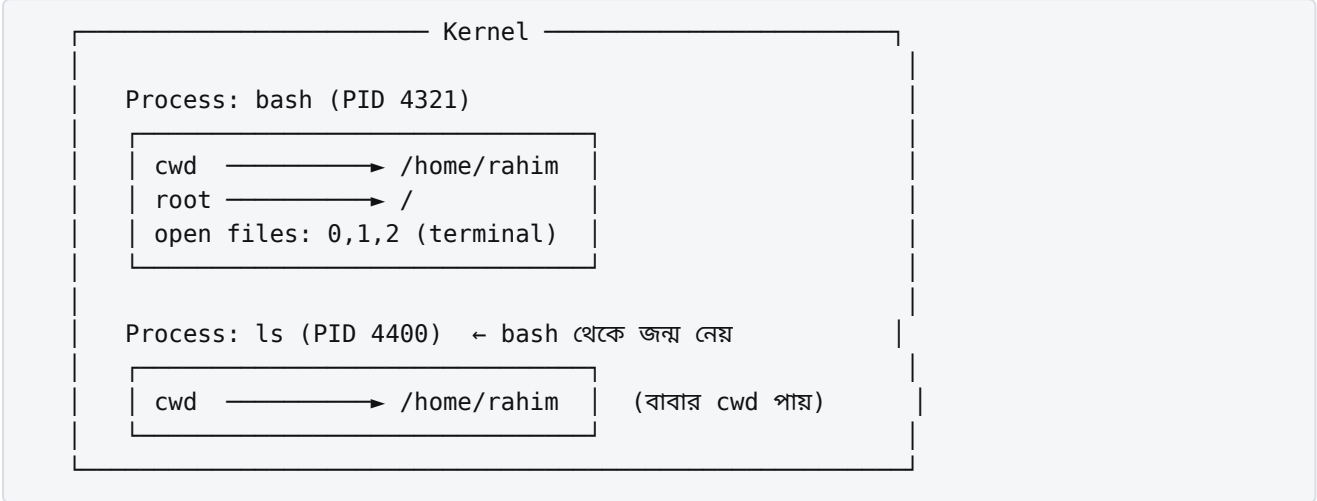
ধারণাটা কী

ধরো তুমি একটা বিশাল শপিং মলে আছ। কোনো দোকান খুঁজতে হলে আগে জানতে হয় "আমি এখন কোথায়"। মলের ম্যাপে লাল একটা বিন্দু থাকে, "You are here"। লিনাক্সে এই বিন্দুর নাম **current working directory** (বর্তমান কাজের ডিরেক্টরি), সংক্ষেপে **cwd**।

টার্মিনালে তুমি যে কমান্ডই চালাও, সেটা ধরে নেয় তুমি এই ডিরেক্টরিতে আছ। যেমন `ls` লিখলে সে cwd-র ভেতরের জিনিস দেখায়। `cat notes.txt` লিখলে সে cwd-তেই `notes.txt` খোঁজে।

ভেতরে কীভাবে কাজ করে (under the hood)

লিনাক্সে প্রতিটা **process** (চলমান প্রোগ্রাম, বিস্তারিত অধ্যায় ২৪-এ) এর নিজস্ব একটা cwd থাকে। kernel প্রতিটা process এর তথ্যের মধ্যে এটা রেখে দেয়। তোমার shell (যেমন `bash`) নিজেও একটা process, তাই তারও একটা cwd আছে।



গুরুত্বপূর্ণ কয়েকটা কথা:

1. যখন shell কোনো নতুন প্রোগ্রাম চালায় (যেমন `ls`), নতুন process টা বাবা (shell) এর cwd **উত্তরাধিকার** হিসেবে পায়। এজন্যই `ls` জানে কোন ফোল্ডারের জিনিস দেখাতে হবে।
2. cwd বদলানোর জন্য kernel এর একটা **system call** আছে: `chdir()`। `cd` কমান্ড ভেতরে ভেতরে এই `chdir()` ডাকে।
3. তুমি চাইলে যেকোনো process এর cwd দেখতে পারো `/proc/<PID>/cwd` থেকে (এটা একটা symbolic link, `/proc` নিয়ে বিস্তারিত অধ্যায় ৪৭-এ):

```
$ ls -l /proc/$$/cwd
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:15 /proc/4321/cwd -> /home/rahim
```

এখানে `$$` হলো বর্তমান shell এর PID।

৮.২ `pwd` — Print Working Directory

কী করে

`pwd` তোমার বর্তমান ডিরেক্টরির পুরো ঠিকানা (absolute path) প্রিন্ট করে।

Syntax

```
pwd [OPTION]
```

উদাহরণ

```
$ pwd
/home/rahim
```

```
$ cd /var/log
$ pwd
/var/log
```

অপশন

অপশন	মানে
`-L`	**Logical** path দেখায়। symlink দিয়ে ঢুকলে symlink এর নামসহ path দেখায়। bash-এ এটাই ডিফল্ট।
`-P`	**Physical** path দেখায়। সব symlink resolve করে আসল ডিরেক্টরির path দেখায়।

`-L` বনাম `-P` উদাহরণ

ধরো `/home/rahim/proj` আসলে `/srv/projects/website` এর একটা symlink (symlink নিয়ে বিস্তারিত অধ্যায় ১১-এ)।

```
$ cd ~/proj
$ pwd
/home/rahim/proj
$ pwd -L
/home/rahim/proj
$ pwd -P
/srv/projects/website
```

ভেতরের কথা: shell builtin বনাম `/usr/bin/pwd`

`pwd` দুই রকম আছে:

```
$ type -a pwd
pwd is a shell builtin
pwd is /usr/bin/pwd
```

- **builtin `pwd`** : bash নিজেই চালায়, নতুন process তৈরি হয় না। bash একটা variable `PWD` তে logical path মনে রাখে, সেখান থেকেই দেখায়।
- **`/usr/bin/pwd`** : GNU coreutils এর আলাদা প্রোগ্রাম। এটা ডিফল্টে `-P` এর মতো আচরণ করে (আসল path দেখায়)।

```
$ echo $PWD
/home/rahim/proj
```

```
$ echo $OLDPWD
/home/rahim
```

> 📝 **মনে রাখো:** `PWD` variable এ থাকে বর্তমান ডিরেক্টরি, আর `OLDPWD` এ থাকে ঠিক আগের ডিরেক্টরি। `cd` প্রতিবার এই দুটো আপডেট করে।

> 💡 **টিপস:** বেশিরভাগ distro-তে prompt নিজেই বর্তমান ডিরেক্টরি দেখায়, যেমন `rahim@laptop:~/Documents$`। তাই প্রায়ই `pwd` লাগে না। কিন্তু স্ক্রিপ্টের ভেতরে বা লম্বা path হলে `pwd` খুব কাজের।

৮.৩ Path — ঠিকানা লেখার দুই নিয়ম

ফাইল বা ডিরেক্টরির ঠিকানাকে বলে **path**। Path লেখার দুটো উপায় আছে।

Absolute path (পূর্ণ ঠিকানা)

- সবসময় `/` দিয়ে শুরু হয়।
- root থেকে শুরু করে পুরো রাস্তা বলে দেয়।
- তুমি যেখানেই থাকো, একই absolute path সবসময় একই জায়গা বোঝায়।

উপমা: চিঠির পূর্ণ ঠিকানা — "বাংলাদেশ, ঢাকা, মিরপুর, রোড ১০, বাড়ি ৫"। পৃথিবীর যেকোনো জায়গা থেকে পাঠালে চিঠি একই বাড়িতে পৌঁছাবে।

```
/home/rahim/Documents/report.txt
/etc/ssh/sshd_config
/var/log/syslog
```

Relative path (আপেক্ষিক ঠিকানা)

- `/` দিয়ে শুরু **হয় না**।
- বর্তমান ডিরেক্টরি (cwd) থেকে হিসাব করা হয়।
- তুমি কোথায় আছ তার ওপর নির্ভর করে একই relative path ভিন্ন জায়গা বোঝাতে পারে।

উপমা: কাউকে বলা "এখান থেকে ডানে দুই বাড়ি পরে"। কোথা থেকে বলছ তার ওপর পুরো অর্থ নির্ভর করে।

```
Documents/report.txt
../shared/data.csv
./script.sh
```

একটা গাছ দিয়ে বুঝি

```
/
├── etc/
│   └── hosts
└── home/
    ├── rahim/          ← তুমি এখানে (cwd)
    │   ├── Documents/
    │   │   └── report.txt
    │   └── Music/
    └── karim/
        └── notes.txt
```

তুমি `/home/rahim` এ আছ। একই ফাইলগুলো দুইভাবে লেখা:

লক্ষ্য	Absolute path	Relative path (cwd = `/home/rahim`)
report.txt	`/home/rahim/Documents/report.txt`	`Documents/report.txt`
Music ফোল্ডার	`/home/rahim/Music`	`Music` বা `./Music`
karim এর notes	`/home/karim/notes.txt`	`../karim/notes.txt`
hosts ফাইল	`/etc/hosts`	`../../etc/hosts`

ভেতরে kernel কীভাবে path খোঁজে (path resolution)

kernel যখন একটা path পায়, সে সেটাকে `/` চিহ্ন দিয়ে টুকরো টুকরো করে এক এক ধাপ এগোয়:

```
Path: /home/rahim/Documents/report.txt

শুরু: '/' দিয়ে শুরু? হ্যাঁ ──► শুরু করো root ডিরেক্টরি থেকে
  ধাপ ১: root এর ভেতরে "home" খোঁজো      ──► পাওয়া গেল (ডিরেক্টরি)
  ধাপ ২: home এর ভেতরে "rahim" খোঁজো      ──► পাওয়া গেল
  ধাপ ৩: rahim এর ভেতরে "Documents" খোঁজো ──► পাওয়া গেল
  ধাপ ৪: Documents এর ভেতরে "report.txt"  ──► পাওয়া গেল (ফাইল) ✔

Path: Documents/report.txt
শুরু: '/' দিয়ে শুরু? না ──► শুরু করো process এর cwd থেকে
  বাকিটা একই রকম…
```

প্রতিটা ধাপে kernel পরীক্ষা করে ওই ডিরেক্টরিতে ঢোকার (execute, `x`) permission তোমার আছে কিনা। না থাকলে `Permission denied` আসে। Permission নিয়ে বিস্তারিত অধ্যায় ১৬-এ।

> 💡 **টিপস:** স্ক্রিপ্ট আর cron job-এ সবসময় absolute path ব্যবহার করো। কারণ ওগুলো কোন cwd থেকে চলবে তার নিশ্চয়তা নেই। ইন্টারঅ্যাক্টিভ কাজে relative path টাইপ করা দ্রুত।

৮.৪ `.` আর `..` — দুটি বিশেষ ডিরেক্টরি

প্রতিটা ডিরেক্টরির ভেতরে দুটো লুকানো এন্ট্রি সবসময় থাকে:

নাম	মানে	উদাহরণ
`.`	এই ডিরেক্টরি নিজেই	`./script.sh` = এই ফোল্ডারের script.sh
`..`	এর ঠিক উপরের (parent) ডিরেক্টরি	`cd ..` = এক ধাপ উপরে যাও

```
$ ls -a
.  ..  Documents  Music  .bashrc
```

এগুলো কি shell এর কারসাজি? না!

অনেকে ভাবে `.` আর `..` shell বানায়। আসলে না। এগুলো ফাইল সিস্টেমে ডিরেক্টরির ভেতরের সত্যিকারের এন্ট্রি (ext4 এর মতো ফাইলসিস্টেমে)। `.` নিজের inode কে নির্দেশ করে, আর `..` parent এর inode কে। inode নম্বর মিলিয়ে দেখো:

```
$ cd /home/rahim
$ ls -id . .. /home /home/rahim
1048578 .   1048577 ..   1048577 /home   1048578 /home/rahim
```

দেখো, `.` আর `/home/rahim` এর inode একই, আর `..` আর `/home` এর inode একই। (inode নিয়ে বিস্তারিত অধ্যায় ১১-এ।)

> 📝 **মনে রাখো:** root `/` এর `..` হলো `/` নিজেই। তাই `/` এ থেকে `cd ..` দিলে তুমি `/` তেই থাকবে। গাছের গোড়ার উপরে আর কিছু নেই।

কেন `./script.sh` লিখতে হয়?

তুমি যখন শুধু `script.sh` লেখো, shell সেটাকে কমান্ড ভেবে `PATH` variable এর ডিরেক্টরিগুলোতে (যেমন `/usr/bin`) খোঁজে, cwd-তে না। নিরাপত্তার কারণে cwd ডিফল্টে `PATH` এ থাকে না। তাই নিজের ফোল্ডারের স্ক্রিপ্ট চালাতে বলতে হয় "এই ফোল্ডারের" — `./script.sh`। (`PATH` নিয়ে বিস্তারিত অধ্যায় ৪৪-এ।)

```
$ script.sh
script.sh: command not found
$ ./script.sh
Hello!
```

৮.৫ `~` (Tilde) Expansion — বাড়ির শর্টকাট

`~` হলো তোমার **home directory** এর সংক্ষিপ্ত রূপ। home directory হলো প্রতিটা ইউজারের নিজস্ব ঘর, যেখানে তার ফাইল, সেটিংস থাকে (সাধারণত `/home/<username>`, আর root ইউজারের জন্য `/root`)।

কে `~` কে বদলায়?

Shell বদলায়, kernel না। তুমি Enter চাপার পর, কমান্ড চালানোর আগে, bash `~` কে আসল path দিয়ে বদলে দেয়। এই প্রক্রিয়ার নাম **tilde expansion**। প্রোগ্রাম কখনো `~` দেখে না, সে পায় পুরো path।

```
$ echo ~
/home/rahim
$ echo ~/Documents
/home/rahim/Documents
```

Tilde এর সব রূপ

রূপ	কীসে বদলায়	উদাহরণ output
`~`	নিজের home (`$HOME`)	`/home/rahim`
`~/dir`	home এর ভেতরের dir	`/home/rahim/dir`
`~karim`	ইউজার karim এর home	`/home/karim`
`~root`	root ইউজারের home	`/root`
`~+`	বর্তমান ডিরেক্টরি (`$PWD`)	`/var/log`
`~-`	আগের ডিরেক্টরি (`$OLDPWD`)	`/home/rahim`

```
$ echo ~root
/root
$ cd /var/log
$ echo ~+ ~-
/var/log /home/rahim
```

কখন tilde expand হয় না

```
$ echo "~"
~
$ echo '~/Documents'
~/Documents
$ echo a~b
a~b
```

- উদ্ধৃতি চিহ্নের (`"..."` বা `'...'`) ভেতরে `~` expand হয় না।
- শব্দের শুরুতে না থাকলে expand হয় না (তবে `VAR=~/x` এর মতো assignment এ `=` এর পরে হয়)।

> ⚠️ **সতর্কতা:** স্ক্রিপ্টে `"~/backup"` লিখলে সেটা আক্ষরিক `~` নামের ফোল্ডার বোঝাবে! উদ্ধৃতির ভেতরে home বোঝাতে `"$HOME/backup"` লেখো।

৮.৬ `cd` — Change Directory

কী করে

`cd` তোমার shell এর বর্তমান ডিরেক্টরি বদলায়।

Syntax

```
cd [-L|-P] [DIRECTORY]
```

কেন `cd` একটা shell builtin? (খুব গুরুত্বপূর্ণ ধারণা)

```
$ type cd
cd is a shell builtin
```

`cd` কে **অবশ্যই** shell এর ভেতরেই থাকতে হয়। কেন?

মনে করো `cd` একটা আলাদা প্রোগ্রাম হতো (`/usr/bin/cd`)। তাহলে shell একটা নতুন child process বানাত, সেই child নিজের cwd বদলাত, তারপর শেষ হয়ে যেত। আর বাবা shell এর cwd যেমন ছিল তেমনই থাকত! কারণ একটা process অন্য process এর cwd বদলাতে পারে না।

```
যদি cd আলাদা প্রোগ্রাম হতো (কাজ করত না):

  bash (cwd=/home/rahim)
     │ fork + exec
     ▼
  /usr/bin/cd /tmp  ──► chdir("/tmp") ──► নিজের cwd=/tmp ──► exit
     │
     ▼
  bash (cwd=/home/rahim)   ✘ কিছুই বদলায়নি!

আসলে যা হয় (builtin):

  bash (cwd=/home/rahim) ──► নিজেই chdir("/tmp") ──► bash (cwd=/tmp) ✔
```

উপমা: তুমি তোমার ছোট ভাইকে পাঠালে অন্য ঘরে যেতে। সে গেল, কিন্তু তাতে তুমি নিজে ঘর বদলাওনি। নিজে যেতে হলে নিজেকেই হাঁটতে হবে। `cd` হলো shell এর নিজে হাঁটা।

`cd` এর সব রূপ

কমান্ড	কোথায় যায়
`cd`	নিজের home এ (`$HOME`)
`cd ~`	নিজের home এ
`cd ~karim`	karim এর home এ (permission থাকলে)
`cd /`	root ডিরেক্টরিতে
`cd ..`	এক ধাপ উপরে (parent)
`cd ../..`	দুই ধাপ উপরে
`cd -`	ঠিক আগের ডিরেক্টরিতে (`$OLDPWD`), আর নতুন path প্রিন্ট করে
`cd .`	একই জায়গায় থাকে (কিছু বদলায় না)
`cd /etc/ssh`	absolute path এ
`cd Documents`	cwd এর ভেতরের Documents এ (relative)

উদাহরণ — একটা ছোট ভ্রমণ

```
$ pwd
/home/rahim
$ cd /var/log
$ pwd
/var/log
$ cd ..
$ pwd
/var
$ cd -
/var/log
$ cd -
/var
$ cd
$ pwd
/home/rahim
$ cd /
$ pwd
/
$ cd ~/Documents
$ pwd
/home/rahim/Documents
```

> 💡 **টিপস:** `cd -` হলো টিভির রিমোটের "Previous Channel" বাটনের মতো। দুটো ডিরেক্টরির মধ্যে বারবার আসা-যাওয়ার জন্য অসাধারণ। যেমন `/etc/nginx` আর `/var/log/nginx` এর মধ্যে।

স্পেস থাকা নাম

ফোল্ডারের নামে স্পেস থাকলে shell সেটাকে দুটো আলাদা শব্দ ভাবে। তাই উদ্ধৃতি বা backslash দিতে হয়:

```
$ cd My Documents
bash: cd: too many arguments
$ cd "My Documents"
$ cd My\ Documents
$ cd 'My Documents'
```

Tab completion ব্যবহার করলে shell নিজেই `\` বসিয়ে দেয়।

`cd -L` বনাম `cd -P` (symlink এর সাথে)

```
$ cd -P ~/proj      # symlink resolve করে আসল ডিরেক্টরিতে যায়
$ pwd
/srv/projects/website
```

ডিফল্ট `-L`। এর একটা মজার দিক আছে: logical মোডে `cd ..` symlink এর **নামের** parent এ যায়, আসল ডিরেক্টরির parent এ না।

```
$ cd ~/proj          # (symlink → /srv/projects/website)
$ cd ..
$ pwd
/home/rahim          # logical parent

$ cd -P ~/proj
$ cd ..
$ pwd
/srv/projects        # physical parent
```

`CDPATH` — cd এর জন্য খোঁজার তালিকা

`CDPATH` variable এ কিছু ডিরেক্টরি দিলে, relative নাম দিয়ে `cd` করলে bash ওই জায়গাগুলোতেও খুঁজবে।

```
$ export CDPATH=.:~/projects
$ cd website
/home/rahim/projects/website
```

`CDPATH` থেকে কোনো ডিরেক্টরি পাওয়া গেলে bash নতুন path প্রিন্ট করে দেয়। শুরুতে `.` রাখা জরুরি, যাতে cwd আগে খোঁজা হয়।

> ⚠️ **সতর্কতা:** `CDPATH` কে `export` করলে কিছু স্ক্রিপ্ট অপ্রত্যাশিত ডিরেক্টরিতে চলে যেতে পারে। নিরাপদ উপায় হলো `~/.bashrc` তে `export` ছাড়া শুধু `CDPATH=...` লেখা।

`shopt -s autocd` — শুধু নাম লিখেই ঢোকা

bash এর এই অপশন চালু থাকলে শুধু ডিরেক্টরির নাম লিখলেই `cd` হয়ে যায়:

```
$ shopt -s autocd
$ /etc
cd -- /etc
$ pwd
/etc
```

আর `shopt -s cdspell` চালু করলে `cd` এ ছোটখাটো বানান ভুল (একটা অক্ষর বাদ/বেশি/অদলবদল) bash নিজেই ঠিক করে নেয়।

সাধারণ error

```
$ cd /root
bash: cd: /root: Permission denied
$ cd notes.txt
bash: cd: notes.txt: Not a directory
$ cd /nonexistent
bash: cd: /nonexistent: No such file or directory
```

৮.৭ `ls` — List Directory Contents

কী করে

`ls` একটা ডিরেক্টরির ভেতরে কী কী ফাইল/ফোল্ডার আছে তার তালিকা দেখায়। কোনো argument না দিলে cwd দেখায়।

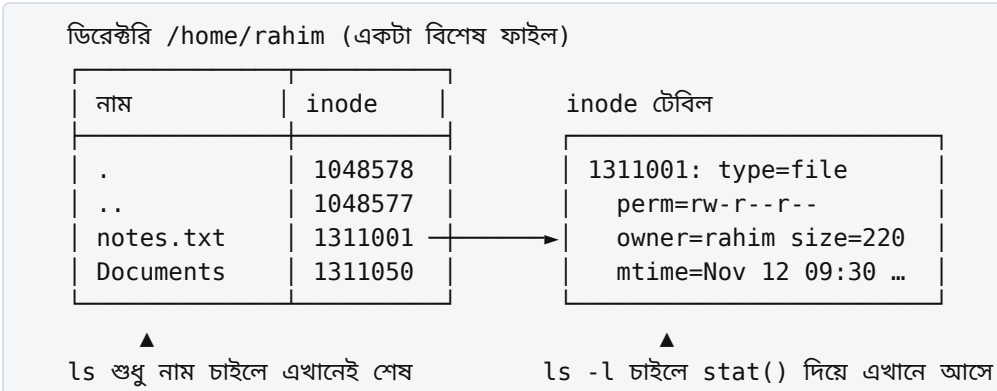
Syntax

```
ls [OPTION]... [FILE]...
```

ভেতরে কীভাবে কাজ করে

ডিরেক্টরি আসলে একটা বিশেষ ফাইল, যার ভেতরে লেখা থাকে "নাম → inode নম্বর" এর একটা তালিকা। `ls`:

1. `opendir()` / `getdents64()` system call দিয়ে নামগুলো পড়ে।
2. `-l` এর মতো বিস্তারিত চাইলে প্রতিটা ফাইলের জন্য `stat()` (আধুনিক glibc তে `statx()` বা `lstat()`) ডেকে inode থেকে size, owner, permission, সময় ইত্যাদি আনে।
3. ডিফল্টে নামের বর্ণানুক্রমে sort করে (locale অনুযায়ী) তারপর প্রিন্ট করে।



এজন্যই হাজার হাজার ফাইলের ফোল্ডারে `ls` দ্রুত, কিন্তু `ls -l` কিছুটা ধীর (প্রতিটা ফাইলে আলাদা stat লাগে)।

সাধারণ ব্যবহার

```
$ ls
Desktop  Documents  Downloads  Music  Pictures  notes.txt
$ ls /etc/ssh
moduli  ssh_config  ssh_config.d  sshd_config  sshd_config.d  ssh_host_ed25519_key ...
$ ls Documents Music
Documents:
report.txt  resume.pdf

Music:
song.mp3
```

গুরুত্বপূর্ণ অপশনের টেবিল

অপশন	পুরো নাম	কী করে
`-l`	long format	বিস্তারিত তথ্য: permission, link, owner, group, size, সময়, নাম
`-a`	`--all`	লুকানো ফাইলসহ (`.` দিয়ে শুরু) সব দেখায়, `.` আর `..` সহ
`-A`	`--almost-all`	লুকানো ফাইল দেখায়, কিন্তু `.` আর `..` বাদে
`-h`	`--human-readable`	`-l` এর সাথে size কে K, M, G আকারে দেখায়
`-t`		শেষ পরিবর্তনের সময় (mtime) অনুযায়ী sort, নতুনটা আগে
`-r`	`--reverse`	sort এর ক্রম উল্টে দেয়
`-S`		size অনুযায়ী sort, বড়টা আগে
`-R`	`--recursive`	ভেতরের সব সাব-ডিরেক্টরিও দেখায়
`-d`	`--directory`	ডিরেক্টরির ভেতরের জিনিস না, ডিরেক্টরিটাকেই দেখায়
`-i`	`--inode`	প্রতিটা ফাইলের inode নম্বর দেখায়

অপশন	পুরো নাম	কী করে
`-F`	`--classify`	নামের শেষে টাইপ চিহ্ন: `/` ডিরেক্টরি, `*` executable, `@` symlink, `
`-1`		প্রতি লাইনে একটা করে নাম
`-n`	`--numeric-uid-gid`	`-l` এর মতো, কিন্তু owner/group নামের বদলে UID/GID সংখ্যা
`-X`		extension অনুযায়ী sort
`-v`		স্বাভাবিক সংখ্যা ক্রমে sort (file2 আগে, file10 পরে)
`-u`		`-lt` এর সাথে: access time (atime) অনুযায়ী
`-c`		`-lt` এর সাথে: inode change time (ctime) অনুযায়ী
`-s`	`--size`	প্রতিটা ফাইলের ডিস্কে বরাদ্দকৃত block size
`-m`		কমা দিয়ে আলাদা করা তালিকা
`--color=auto`		টার্মিনাল হলে রঙ দিয়ে টাইপ বোঝায় (`always` / `never` ও আছে)
`--group-directories-first`		ডিরেক্টরিগুলো আগে, তারপর ফাইল
`--time-style=STYLE`		সময়ের ফরম্যাট: `full-iso` , `long-iso` , `iso` , `locale` , বা `+FORMAT`
`--full-time`		`-l --time-style=full-iso` এর সমান
`-Z`	`--context`	SELinux security context দেখায় (Fedora/RHEL এ কাজের)

একাধিক ছোট অপশন একসাথে লেখা যায়: `ls -l -a -h` = `ls -lah`।

প্রতিটা অপশনের উদাহরণ

`-a` আর `-A` — লুকানো ফাইল

লিনাক্সে যে ফাইলের নাম `.` দিয়ে শুরু, সেটা **hidden** (লুকানো)। এটা কোনো বিশেষ attribute না, শুধু একটা নিয়ম: `ls` আর ফাইল ম্যানেজার এগুলো ডিফল্টে দেখায় না। সাধারণত কনফিগারেশন ফাইলগুলো এভাবে রাখা হয়, যাতে home ফোল্ডার পরিষ্কার দেখায়।

```
$ ls
Documents  Music  notes.txt
$ ls -a
.  ..  .bash_history  .bashrc  .cache  .config  .profile  Documents  Music  notes.txt
$ ls -A
.bash_history  .bashrc  .cache  .config  .profile  Documents  Music  notes.txt
```

`-l` আর `-h` — বিস্তারিত ও পড়ার মতো size

```
$ ls -l
total 2056
drwxr-xr-x 2 rahim rahim    4096 Nov 10 14:22 Documents
-rw-r--r-- 1 rahim rahim 2097152 Nov 12 09:30 backup.tar
-rw-r--r-- 1 rahim rahim     220 Nov  1 08:00 notes.txt
$ ls -lh
total 2.1M
drwxr-xr-x 2 rahim rahim 4.0K Nov 10 14:22 Documents
-rw-r--r-- 1 rahim rahim 2.0M Nov 12 09:30 backup.tar
-rw-r--r-- 1 rahim rahim  220 Nov  1 08:00 notes.txt
```

`-t`, `-r`, `-S` — sort করা

```
$ ls -lt          # নতুন পরিবর্তিত আগে
$ ls -ltr         # পুরনো আগে, নতুনটা একদম নিচে (লগ দেখতে দারুণ)
$ ls -lS          # সবচেয়ে বড় আগে
$ ls -lSr         # সবচেয়ে ছোট আগে
$ ls -lhS /var/log | head -5
total 48M
-rw-r----- 1 syslog adm 21M Nov 12 10:01 syslog
-rw-r----- 1 syslog adm 15M Nov  9 00:00 syslog.1
-rw-r--r-- 1 root   root 4.1M Nov 12 09:55 dpkg.log
-rw-r----- 1 syslog adm 2.3M Nov 12 10:01 kern.log
```

> 💡 **টিপস:** `ls -ltr` সিস্টেম অ্যাডমিনদের প্রিয় কমান্ড। কোনো ফোল্ডারে "এইমাত্র কোন ফাইল বদলেছে" জানতে চাইলে এটা চালাও, সবচেয়ে নতুনটা স্ক্রিনের একদম নিচে, চোখের সামনে থাকবে।

`-R` — recursive

```
$ ls -R project
project:
README.md  src

project/src:
main.c  utils

project/src/utils:
helper.c
```

> ⚠️ **সতর্কতা:** `ls -R /` চালালে পুরো সিস্টেমের লাখ লাখ ফাইল স্ক্রিনে বয়ে যাবে। আটকাতে `Ctrl+C` চাপো।

`-d` — ডিরেক্টরিকে নিজেকেই দেখা

```
$ ls -l /etc        # /etc এর ভেতরের শত শত ফাইল দেখাবে
$ ls -ld /etc       # শুধু /etc ডিরেক্টরিটার তথ্য
drwxr-xr-x 139 root root 12288 Nov 12 08:10 /etc
```

```
$ ls -d */          # cwd তে শুধু ডিরেক্টরিগুলো
Documents/  Music/  Pictures/
```

`-i` — inode নম্বর

```
$ ls -i
1311050 Documents  1311001 notes.txt
```

`-F` — টাইপ চিহ্ন

```
$ ls -F
Documents/  backup.sh*  link-to-notes@  notes.txt
```

`--color` — রঙ

Ubuntu/Debian-এ `~/.bashrc` তে `alias ls='ls --color=auto'` আগে থেকেই থাকে, Fedora/RHEL-এ `/etc/profile.d/colorls.sh` এটা করে। সাধারণ রঙের মানে (ডিফল্ট থিমে):

রঙ	সাধারণত মানে
নীল (bold)	ডিরেক্টরি
সবুজ (bold)	executable ফাইল
সায়ান (হালকা নীল)	symbolic link
লাল (bold)	archive (`.tar` , `.zip` , `.gz`)
ম্যাজেন্টা	ছবি/ভিডিও
হলুদ, কালো পটভূমি	device ফাইল (`/dev` এ)
লাল পটভূমিতে লেখা	ভাঙা symlink (orphan)

রঙগুলো আসে `LS_COLORS` variable থেকে, যেটা `dircolors` কমান্ড সেট করে। `dircolors -p` দিয়ে পুরো ডিফল্ট সেটিং দেখতে পারো।

> 📝 **মনে রাখো:** `--color=auto` এর মানে হলো output টার্মিনালে গেলে রঙ, আর pipe বা ফাইলে গেলে রঙ ছাড়া। `--color=always` দিয়ে pipe এও রঙ পাঠালে `less -R` দিয়ে দেখতে হয়।

Ubuntu এর তৈরি alias গুলো

Ubuntu/Debian এর ডিফল্ট `~/.bashrc` তে এগুলো থাকে:

```
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'
```

তাই Ubuntu তে `ll` লিখলেই `ls -alF` চলে। Fedora/RHEL এ `ll` হলো `ls -l` (সাথে color)। যাচাই করতে:

```
$ type ll
ll is aliased to `ls -alF'
```

৮.৮ `ls -l` আউটপুটের প্রতিটা কলাম — বিস্তারিত

এই আউটপুটটা পড়তে পারা লিনাক্সের অন্যতম মৌলিক দক্ষতা। একটা লাইন ভেঙে দেখি:

```
-rw-r--r-- 1 rahim developers 2097152 Nov 12 09:30 backup.tar
│└──┬──┘ │   │       │         │          │          │
│   │    │   │       │         │          │          └─ ⑦ নাম
│   │    │   │       │         │          └─ ⑥ শেষ পরিবর্তনের সময় (mtime)
│   │    │   │       │         └─ ⑤ size (bytes এ)
│   │    │   │       └─ ④ group
│   │    │   └─ ③ owner (মালিক ইউজার)
│   │    └─ ② hard link সংখ্যা
│   └─ ① (খ) permission (৯ অক্ষর)
└─ ① (ক) ফাইলের টাইপ (১ অক্ষর)
```

① ফাইলের টাইপ আর permission

প্রথম অক্ষর টাইপ বোঝায়:

অক্ষর	টাইপ	কোথায় দেখা যায়
`-`	সাধারণ ফাইল (regular file)	টেক্সট, ছবি, প্রোগ্রাম
`d`	ডিরেক্টরি	`/home`, `Documents`
`l`	symbolic link	`/bin -> usr/bin`
`c`	character device	`/dev/tty`, `/dev/null`
`b`	block device	`/dev/sda`, `/dev/nvme0n1`
`p`	named pipe (FIFO)	`mkfifo` দিয়ে বানানো
`s`	socket	`/run/systemd/...`

পরের ৯টা অক্ষর তিনটা দলে permission:

```
 r w -   r - -   r - -
 └─┬─┘   └─┬─┘   └─┬─┘
 owner   group   others
```

`r` = পড়া, `w` = লেখা, `x` = চালানো (ডিরেক্টরির জন্য ঢোকা), `-` = অনুমতি নেই। কখনো কখনো `s`, `t` দেখবে (বিশেষ permission, অধ্যায় ১৭), আর শেষে `+` থাকলে ACL আছে, `.` থাকলে SELinux context আছে

(Fedora/RHEL এ প্রায় সব ফাইলে)। পুরো permission সিস্টেম অধ্যায় ১৬-এ।

② Hard link সংখ্যা

ফাইলের জন্য: এই inode কে কতগুলো নাম নির্দেশ করছে। সাধারণত `1`।

ডিরেক্টরির জন্য: সাধারণত `2 + সাব-ডিরেক্টরির সংখ্যা`। কেন? ডিরেক্টরির নিজের নাম (parent এ) একটা link, ভেতরের `.` আরেকটা, আর প্রতিটা সাব-ডিরেক্টরির `..` একটা করে। (btrfs এর মতো কিছু ফাইলসিস্টেম এই নিয়ম মানে না, সেখানে ডিরেক্টরিতে সবসময় `1` দেখায়।) বিস্তারিত অধ্যায় ১১।

```
$ ls -ld /home
drwxr-xr-x 4 root root 4096 Nov  1 08:00 /home   # ২ + ২টা ইউজার ফোল্ডার
```

③ Owner, ④ Group

কোন ইউজার ফাইলটার মালিক আর কোন group এর। kernel আসলে নাম রাখে না, রাখে সংখ্যা (UID, GID)। `ls` সেগুলো `/etc/passwd` আর `/etc/group` থেকে নামে রূপান্তর করে। `ls -n` দিলে কাঁচা সংখ্যা দেখবে:

```
$ ls -ln notes.txt
-rw-r--r-- 1 1000 1000 220 Nov  1 08:00 notes.txt
```

ইউজার মুছে ফেলা হলে নামের জায়গায় শুধু সংখ্যা দেখা যায়। ইউজার/গ্রুপ নিয়ে অধ্যায় ১৫।

⑤ Size

ফাইলের আকার byte এ। `-h` দিলে K/M/G (1024 ভিত্তিক)। `--si` দিলে 1000 ভিত্তিক।

> 📝 **মনে রাখো:** ডিরেক্টরির size (যেমন `4096`) মানে ভেতরের ফাইলগুলোর মোট আকার **না**। এটা হলো ডিরেক্টরির নিজের "নামের তালিকা" রাখতে কতটা জায়গা লাগছে। ভেতরের মোট আকার জানতে `du -sh dirname` লাগে (অধ্যায় ৩২)।

Device ফাইলে size এর জায়গায় দুটো সংখ্যা থাকে: **major, minor** নম্বর (কোন driver আর কোন device):

```
$ ls -l /dev/sda
brw-rw---- 1 root disk 8, 0 Nov 12 08:00 /dev/sda
```

⑥ সময়

ডিফল্টে **mtime** (modification time): ফাইলের ভেতরের লেখা শেষ কখন বদলেছে। ছয় মাসের বেশি পুরনো (বা ভবিষ্যতের) হলে ঘণ্টা-মিনিটের বদলে বছর দেখায়:

```
-rw-r--r-- 1 rahim rahim 512 Mar  3  2024 old.txt
```

```
-rw-r--r-- 1 rahim rahim 220 Nov  1 08:00 notes.txt
```

তিন ধরনের সময় আছে:

সময়	মানে	`ls` এ দেখার উপায়
mtime	কন্টেন্ট বদলানোর সময়	`ls -l` (ডিফল্ট)
atime	শেষ পড়ার সময়	`ls -lu`
ctime	inode (permission, owner, নাম ইত্যাদি) বদলানোর সময়	`ls -lc`

সঠিক সেকেন্ড পর্যন্ত দেখতে:

```
$ ls -l --full-time notes.txt
-rw-r--r-- 1 rahim rahim 220 2026-11-01 08:00:13.482193201 +0600 notes.txt
```

⑦ নাম

symlink হলে নামের পরে `->` দিয়ে লক্ষ্য দেখায়:

```
$ ls -l /bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
```

প্রথম লাইনের `total` কী?

```
$ ls -l
total 2056
```

এটা তালিকার সব ফাইলের ডিস্কে **বরাদ্দকৃত** block এর যোগফল, GNU `ls` এ ডিফল্টে 1 KiB একক হিসেবে। এটা byte size এর যোগফল না। `-h` দিলে `total 2.1M` এর মতো দেখায়।

৮.৯ `tree` — গাছের মতো করে দেখা

কী করে

`tree` একটা ডিরেক্টরির ভেতরের সব কিছু গাছের ডালপালার আকারে দেখায়। কোন ফাইল কোন ফোল্ডারের ভেতরে, এক নজরে বোঝা যায়।

ইনস্টল

`tree` বেশিরভাগ distro-তে ডিফল্টে থাকে না:

```
$ sudo apt install tree        # Ubuntu / Debian
$ sudo dnf install tree        # Fedora / RHEL 9
```

(প্যাকেজ ইনস্টল নিয়ে বিস্তারিত অধ্যায় ৩০-এ।)

উদাহরণ

```
$ tree project
project
├── README.md
├── docs
│   └── guide.md
└── src
    ├── main.c
    └── utils
        └── helper.c

4 directories, 4 files
```

গুরুত্বপূর্ণ অপশন

অপশন	কী করে
`-L N`	সর্বোচ্চ N স্তর গভীর পর্যন্ত দেখায়
`-d`	শুধু ডিরেক্টরি
`-a`	লুকানো ফাইলসহ
`-f`	প্রতিটার পুরো path দেখায়
`-h`	size মানুষের পড়ার মতো করে
`-p`	permission দেখায়
`-u`, `-g`	owner, group দেখায়
`-D`	শেষ পরিবর্তনের সময়
`-I PATTERN`	pattern মিলে যাওয়া জিনিস বাদ দেয় (যেমন `-I 'node_modules`
`-P PATTERN`	শুধু pattern মিলে যাওয়া ফাইল দেখায়
`--dirsfirst`	ডিরেক্টরি আগে
`-C`	রঙ জোর করে চালু
`-J` / `-X`	JSON / XML আকারে output

```
$ tree -L 1 /
/
├── bin -> usr/bin
├── boot
├── dev
├── etc
├── home
```

```
...
$ tree -d -L 2 ~
$ tree -h --dirsfirst -I 'node_modules|.git' myapp
```

> 💡 **টিপস:** `tree` না থাকলে কাছাকাছি কাজ `find . -maxdepth 2` দিয়ে করা যায় (`find` নিয়ে অধ্যায় ১২)।

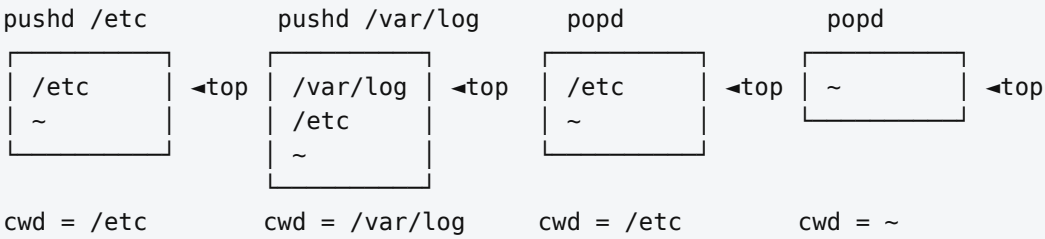
৮.১০ `pushd`, `popd`, `dirs` — ডিরেক্টরি স্ট্যাক

সমস্যাটা কী

`cd -` শুধু একটা আগের জায়গা মনে রাখে। কিন্তু তুমি যদি তিন-চারটা ডিরেক্টরিতে ঘুরে আবার ক্রম অনুযায়ী ফিরে আসতে চাও?

সমাধান: stack

Stack মানে প্লেটের স্তূপের মতো। নতুন প্লেট রাখো একদম উপরে (push), তুলতেও হয় উপর থেকে (pop)। একে বলে LIFO, Last In First Out।



Stack এর উপরের জিনিসটা সবসময় বর্তমান ডিরেক্টরি।

কমান্ডগুলো

কমান্ড	কী করে
`pushd DIR`	বর্তমান ডিরেক্টরি stack এ রেখে DIR এ যায়
`pushd`	উপরের দুটো এন্ট্রি অদলবদল করে (দুই জায়গায় টগল)
`pushd +N`	stack ঘুরিয়ে বাম দিক থেকে N নম্বর এন্ট্রিকে উপরে আনে (০ থেকে গোনা)
`pushd -n DIR`	ডিরেক্টরি না বদলে শুধু stack এ যোগ করে
`popd`	উপরের এন্ট্রি সরিয়ে নতুন উপরের ডিরেক্টরিতে যায়
`popd +N`	N নম্বর এন্ট্রি সরিয়ে দেয়
`dirs`	stack দেখায় (এক লাইনে)

কমান্ড	কী করে
`dirs -v`	নম্বরসহ প্রতি লাইনে একটা করে দেখায়
`dirs -p`	নম্বর ছাড়া প্রতি লাইনে একটা
`dirs -l`	`~` এর বদলে পুরো path
`dirs -c`	পুরো stack মুছে দেয়

উদাহরণ

```
$ pwd
/home/rahim
$ pushd /etc
/etc ~
$ pushd /var/log
/var/log /etc ~
$ pushd /usr/share
/usr/share /var/log /etc ~
$ dirs -v
 0  /usr/share
 1  /var/log
 2  /etc
 3  ~
$ pushd +2
/etc ~ /usr/share /var/log
$ popd
~ /usr/share /var/log
$ pwd
/home/rahim
```

`pushd` আর `popd` প্রতিবার পুরো stack প্রিন্ট করে। চুপ রাখতে চাইলে `pushd /etc > /dev/null`।

`~N` দিয়ে stack এর এন্ট্রি সরাসরি ব্যবহার করা যায় (tilde expansion এর অংশ):

```
$ dirs -v
 0  /usr/share
 1  /var/log
$ cp ~1/syslog .        # ~1 = /var/log
```

> 💡 **টিপস:** স্ক্রিপ্টে কোনো ফোল্ডারে গিয়ে কাজ করে আবার আগের জায়গায় ফিরতে `pushd dir > /dev/null; ...; popd > /dev/null` খুব পরিষ্কার পদ্ধতি। আরেকটা উপায় হলো subshell: `( cd dir && make )`, এতে বাইরের shell এর cwd বদলায়ই না।

৮.১১ Wildcards / Globbing — অনেক ফাইল একসাথে ধরা

কী এবং কেন

ধরো একটা ফোল্ডারে ৫০০টা ফাইল, তুমি শুধু `.jpg` গুলো দেখতে চাও। প্রতিটার নাম টাইপ করা অসম্ভব। এখানে কাজে আসে **wildcard**, বিশেষ চিহ্ন যা নামের একটা "ছাঁচ" (pattern) বোঝায়। এই pattern কে ফাইলের নামে মিলিয়ে তালিকা বানানোর প্রক্রিয়ার নাম **globbing** (বা pathname expansion)।

উপমা: তাসের খেলায় জোকার যেকোনো তাসের জায়গা নিতে পারে। `*` হলো সেই জোকার।

ভেতরে কে কাজ করে? — Shell!

এটা বোঝা খুব জরুরি: globbing করে **shell**, কমান্ড না। `ls *.txt` লিখলে `ls` কখনো `*` দেখে না।

```
তুমি টাইপ করলে:      ls *.txt
                           │
              bash cwd পড়ে নাম মেলায়
                           │
                           ▼
আসলে যা চলে:          ls a.txt b.txt notes.txt
```

প্রমাণ দেখতে `echo` ব্যবহার করো, কারণ `echo` শুধু যা পায় তাই প্রিন্ট করে:

```
$ echo *.txt
a.txt b.txt notes.txt
```

> 💡 **টিপস:** কোনো ঝুঁকিপূর্ণ কমান্ডের আগে (যেমন `rm *.log`) আগে `echo *.log` চালিয়ে দেখে নাও কোন ফাইলগুলো ধরা পড়বে।

Wildcard এর চিহ্নগুলো

চিহ্ন	মেলে	উদাহরণ	মিলবে
`*`	যেকোনো সংখ্যক অক্ষর (শূন্যসহ)	`*.txt`	`a.txt`, `notes.txt`, `.txt` না (hidden)
`?`	ঠিক একটা অক্ষর	`file?.txt`	`file1.txt`, `fileA.txt`; `file10.txt` না
`[abc]`	বন্ধনীর যেকোনো একটা অক্ষর	`[abc]*.txt`	`a.txt`, `b1.txt`, `cat.txt`
`[a-z]`	পরিসরের একটা অক্ষর	`file[0-9].txt`	`file0.txt` … `file9.txt`
`[!abc]` বা `[^abc]`	বন্ধনীর বাইরের একটা অক্ষর	`[!0-9]*`	যা সংখ্যা দিয়ে শুরু হয়নি

চিহ্ন	মেলে	উদাহরণ	মিলবে
`[[:digit:]]`	character class	`*[[:digit:]]`	যা সংখ্যা দিয়ে শেষ

Character class গুলো: `[:alpha:]` (অক্ষর), `[:digit:]` (সংখ্যা), `[:alnum:]` (অক্ষর+সংখ্যা), `[:upper:]`, `[:lower:]`, `[:space:]`, `[:punct:]`। এগুলো অবশ্যই আরেক জোড়া বন্ধনীর ভেতরে: `[[:upper:]]`।

> ⚠️ **সতর্কতা:** `[a-z]` এর মানে locale এর ওপর নির্ভর করতে পারে (কিছু locale এ বড় হাতের অক্ষরও মিলে যেত)। নিশ্চিতভাবে ছোট হাতের অক্ষর চাইলে `[[:lower:]]` ব্যবহার করো।

উদাহরণ

ধরো ফোল্ডারে আছে:

```
$ ls
data1.csv  data2.csv  data10.csv  photo.jpg  Photo.PNG  report.txt  .hidden
```

```
$ ls *.csv
data10.csv  data1.csv  data2.csv
$ ls data?.csv
data1.csv  data2.csv
$ ls data??.csv
data10.csv
$ ls [pP]hoto.*
photo.jpg  Photo.PNG
$ ls [!d]*
photo.jpg  Photo.PNG  report.txt
$ ls *
data10.csv  data1.csv  data2.csv  photo.jpg  Photo.PNG  report.txt
```

লক্ষ্য করো, `*` লুকানো ফাইল `.hidden` ধরেনি।

নিয়মগুলো যা মনে রাখতে হবে

1. **লুকানো ফাইল:** `*` আর `?` নামের শুরুর `.` এর সাথে মেলে না। লুকানো ফাইল ধরতে `.*` লিখতে হয়, অথবা `shopt -s dotglob` চালু করতে হয়।
2. **`/` এর সাথে মেলে না:** `*` কখনো `/` পার হয় না। তাই `*.txt` সাব-ফোল্ডারের ভেতরে খোঁজে না। `*/*.txt` লিখলে এক স্তর নিচে খোঁজে।
3. **কিছু না মিললে:** bash ডিফল্টে pattern টাকে অক্ষত রেখে দেয়:

```
$ ls *.pdf
ls: cannot access '*.pdf': No such file or directory
```

এখানে কোনো `.pdf` নেই, তাই bash আক্ষরিক `*.pdf` পাঠিয়েছে, আর `ls` সেই নামের ফাইল খুঁজে পায়নি।

4. **উদ্ধৃতির ভেতরে globbing হয় না:** `ls "*.txt"` আক্ষরিক `*.txt` নামের ফাইল খুঁজবে। এটাই `find . -name "*.txt"` এর মতো কমান্ডে উদ্ধৃতি দেওয়ার কারণ, যাতে shell না, `find` নিজে pattern পায়।
5. **sort:** ফলাফল locale অনুযায়ী বর্ণানুক্রমে সাজানো থাকে।

`shopt` দিয়ে globbing এর আচরণ বদলানো (bash)

অপশন	কী করে
`shopt -s dotglob`	`*` লুকানো ফাইলও ধরে (`.` আর `..` ছাড়া)
`shopt -s nullglob`	কিছু না মিললে pattern টা খালি হয়ে যায় (স্ক্রিপ্টে লুপের জন্য উপকারী)
`shopt -s failglob`	কিছু না মিললে error দেয়, কমান্ড চালায় না
`shopt -s nocaseglob`	বড়/ছোট হাতের পার্থক্য উপেক্ষা করে
`shopt -s globstar`	`**` দিয়ে সব স্তরের সাব-ডিরেক্টরি ধরা যায়
`shopt -s extglob`	বাড়তি pattern চালু করে (নিচে দেখো)
`shopt`	সব অপশনের বর্তমান অবস্থা দেখায়

`-s` = set (চালু), `-u` = unset (বন্ধ)।

globstar (`**`):

```
$ shopt -s globstar
$ ls **/*.c
src/main.c  src/utils/helper.c
```

extglob (Ubuntu তে ইন্টারঅ্যাক্টিভ shell এ সাধারণত আগে থেকেই চালু থাকে, bash-completion এর কারণে):

Pattern	মানে
`?(pat)`	শূন্য বা একবার
`*(pat)`	শূন্য বা যতবার খুশি
`+(pat)`	এক বা তার বেশি বার
`@(pat1|pat2)`	ঠিক একবার, যেকোনো একটা
`!(pat)`	pattern ছাড়া সব কিছু

```
$ shopt -s extglob
$ ls !(*.csv)
photo.jpg  Photo.PNG  report.txt
$ ls *.@(jpg|PNG)
photo.jpg  Photo.PNG
```

> 💡 **টিপস:** `rm !(important.txt)` দিয়ে একটা ফাইল রেখে বাকি সব মোছা যায়। কিন্তু আগে অবশ্যই `echo !(important.txt)` চালিয়ে দেখে নাও।

> 📝 **মনে রাখো:** Globbing এর pattern আর regular expression (অধ্যায় ২০) এক জিনিস না। glob এ `*` মানে "যেকোনো অক্ষর", regex এ `*` মানে "আগের জিনিসটা শূন্য বা বেশি বার"। glob এর `?` = regex এর `.`।

৮.১২ Brace Expansion `{}` — নাম তৈরির কারখানা

Glob থেকে পার্থক্য

Brace expansion **দেখতে** wildcard এর মতো, কিন্তু কাজ সম্পূর্ণ আলাদা:

	Globbing (`*`, `?`, `[]`)	Brace expansion (`{}`)
কাজ	**আছে এমন** ফাইলের নাম খোঁজে	নতুন শব্দ **তৈরি** করে
ফাইল থাকতে হবে?	হ্যাঁ	না, ফাইল দেখেই না
কখন হয়	পরে	সব expansion এর আগে, সবার প্রথমে

উপমা: globbing হলো রেজিস্টার খাতা থেকে "যাদের নাম 'রা' দিয়ে শুরু" খুঁজে বের করা। Brace expansion হলো ছাপাখানা, যা তালিকা অনুযায়ী নতুন নামের কার্ড ছেপে দেয়।

তালিকা রূপ `{a,b,c}`

```
$ echo file{1,2,3}.txt
file1.txt file2.txt file3.txt
$ echo {red,green,blue}-shirt
red-shirt green-shirt blue-shirt
$ mkdir -p project/{src,docs,tests}
```

শেষেরটা একটা কমান্ডেই তিনটা ফোল্ডার বানায়। (`mkdir` নিয়ে অধ্যায় ৯।)

পরিসর রূপ `{start..end}` ও `{start..end..step}`

```
$ echo {1..5}
1 2 3 4 5
$ echo {a..e}
a b c d e
$ echo {5..1}
5 4 3 2 1
$ echo {01..10}
```

```
01 02 03 04 05 06 07 08 09 10
$ echo {0..20..5}
0 5 10 15 20
$ echo {a..z..2}
a c e g i k m o q s u w y
```

শুরুতে `0` দিলে (`{01..10}`) সব সংখ্যা একই প্রস্থে শূন্য বসিয়ে দেখায়।

নেস্টেড আর গুণ (combination)

```
$ echo {A,B}{1,2}
A1 A2 B1 B2
$ echo img_{2025,2026}_{01..03}.jpg
img_2025_01.jpg img_2025_02.jpg img_2025_03.jpg img_2026_01.jpg img_2026_02.jpg
img_2026_03.jpg
$ echo {a,b{1,2},c}
a b1 b2 c
```

সবচেয়ে কাজের কৌশল: ব্যাকআপ কপি

```
$ cp config.yml{,.bak}
```

এটা হয়ে যায় `cp config.yml config.yml.bak`। কারণ `{,.bak}` এর প্রথম অংশ খালি, দ্বিতীয়টা `.bak`।

```
$ echo config.yml{,.bak}
config.yml config.yml.bak
$ mv report.{txt,md}          # report.txt কে report.md নামে
```

নিয়মগুলো

- কমা বা `..` না থাকলে expansion হয় না: `echo {a}` → `{a}`।
- কমার পাশে স্পেস দেওয়া যাবে না: `{a, b}` কাজ করবে না (স্পেস শব্দ ভেঙে দেয়)।
- উদ্ধৃতির ভেতরে হয় না: `echo "{1..3}"` → `{1..3}`।
- Range এ variable কাজ করে না, কারণ brace expansion variable expansion এর **আগে** হয়: `n=5; echo {1..$n}` → `{1..5}`। এর বদলে `seq 1 $n` ব্যবহার করো।
- এটা bash/zsh এর ফিচার; POSIX `sh` (যেমন Debian/Ubuntu এর `dash`) এ নেই।

Shell expansion এর ক্রম

bash একটা কমান্ড লাইন কোন ক্রমে বদলায়, সেটা জানলে অনেক রহস্য পরিষ্কার হয়:

```
১. Brace expansion          {a,b}  {1..5}
২. Tilde expansion          ~  ~user
৩. Parameter/variable       $HOME  ${var}
   Arithmetic               $((1+2))
   Command substitution     $(date)
৪. Word splitting           স্পেস দিয়ে শব্দ ভাগ
```

```
৫. Pathname expansion       *  ?  [ ]   (globbing)
৬. Quote removal            উদ্ধৃতি চিহ্ন সরানো
                 |
                 ▼
          কমান্ড চলে
```

(২ ও ৩ নম্বরের ধাপগুলো আসলে বাম থেকে ডানে একসাথে হয়।) এই ক্রমের কারণেই `{1..$n}` কাজ করে না, আর `~/{a,b}` কাজ করে। Shell expansion এর বিস্তারিত অধ্যায় ৪১-এ।

৮.১৩ Tab Completion — টাইপ কম, কাজ বেশি

কী

নাম বা কমান্ডের প্রথম কয়েকটা অক্ষর লিখে `Tab` চাপলে shell বাকিটা নিজেই পূরণ করে দেয়। এটা লিনাক্স ব্যবহারকারীর সবচেয়ে বড় গতি-বর্ধক।

কীভাবে কাজ করে

```
টাইপ: cd Doc[Tab]
          |
   bash দেখে: কোন শব্দ পূরণ করতে হবে? → "Doc"
   এটা প্রথম শব্দ? → না, তাই ফাইল/ডিরেক্টরি খোঁজো
   (cd এর জন্য নিয়ম থাকলে শুধু ডিরেক্টরি)
          |
   cwd তে "Doc" দিয়ে শুরু:
     ┌─ একটাই মিলেছে (Documents) → পুরো নাম বসাও + "/"
     ├─ কয়েকটা মিলেছে (Documents, Doctor) → সাধারণ অংশ পর্যন্ত বসাও,
     │                                        বিপ বা কিছু না; আবার Tab চাপলে তালিকা
     └─ কিছুই মেলেনি → কিছু হয় না (বা বিপ)
```

- লাইনের **প্রথম শব্দে** Tab দিলে bash কমান্ডের নাম পূরণ করে (`PATH` এর প্রোগ্রাম, alias, builtin, function)।
- পরের শব্দগুলোতে সাধারণত ফাইলের নাম।
- `$` দিয়ে শুরু হলে variable এর নাম, `~` দিয়ে শুরু হলে ইউজারের নাম পূরণ করে।

Programmable completion (`bash-completion`)

`bash-completion` প্যাকেজ (Ubuntu, Debian, Fedora তে সাধারণত আগে থেকেই ইনস্টল থাকে) প্রতিটা কমান্ডের জন্য আলাদা বুদ্ধিমান নিয়ম যোগ করে:

```
$ git che[Tab]          → git checkout
$ systemctl sta[Tab][Tab]
start   status
$ ssh us[Tab]           → ~/.ssh/config বা known_hosts থেকে host এর নাম
```

```
$ sudo apt install ngi[Tab]   → nginx (প্যাকেজের নাম)
$ ls --col[Tab]         → ls --color=
```

নিয়মগুলো থাকে `/usr/share/bash-completion/completions/` এ। না থাকলে `sudo apt install bash-completion` বা `sudo dnf install bash-completion`। `complete -p cd` দিয়ে দেখতে পারো কোনো কমান্ডের জন্য কোন নিয়ম আছে।

Tab completion এর কৌশল

1. **সবসময় Tab চাপো।** শুধু দ্রুততার জন্য না, এটা বানান যাচাইও। Tab কাজ না করলে বুঝবে নাম ভুল বা ফাইল নেই।
2. **দুইবার Tab** (`Tab Tab`) চাপলে সব সম্ভাব্য বিকল্প দেখায়।
3. **অনেক বিকল্প থাকলে** bash জিজ্ঞেস করে: `Display all 2154 possibilities? (y or n)`। `n` চাপো, তারপর আরও কিছু অক্ষর লেখো।
4. **স্পেস থাকা নাম:** Tab নিজেই `\` বসিয়ে দেয় (`My\ Documents/`)।
5. **লম্বা path এক এক ধাপে:** `cd /u[Tab]sh[Tab]do[Tab]` → `cd /usr/share/doc/`।
6. **খালি লাইনে** `Tab Tab` সব কমান্ডের তালিকা দেখায় (হাজারখানেক!)।

Readline কনফিগারেশন (`~/.inputrc`) দিয়ে Tab কে আরও ভালো করা

bash এর লাইন-এডিটিং আসে **Readline** লাইব্রেরি থেকে। এর সেটিং থাকে `~/.inputrc` এ (system-wide `/etc/inputrc`):

```
# ~/.inputrc
$include /etc/inputrc
set completion-ignore-case on      # বড়/ছোট হাতের পার্থক্য উপেক্ষা
set show-all-if-ambiguous on       # একবার Tab এই তালিকা দেখাও
set mark-symlinked-directories on  # symlink করা ডিরেক্টরির শেষে / বসাও
set colored-stats on               # তালিকায় ls এর মতো রঙ
"\e[A": history-search-backward    # ↑ = যা লিখেছ তা দিয়ে শুরু হওয়া পুরনো কমান্ড
"\e[B": history-search-forward
```

নতুন টার্মিনাল খুললে বা `bind -f ~/.inputrc` চালালে কার্যকর হয়। (প্রথম লাইনটা system এর ডিফল্ট সেটিং হারানো ঠেকায়।) শেল কাস্টমাইজেশন নিয়ে বিস্তারিত অধ্যায় ৪৪-এ।

> 💡 **টিপস:** zsh বা fish shell এ Tab completion আরও উন্নত (মেনু, বর্ণনাসহ)। shell এর পার্থক্য অধ্যায় ৫-এ আছে।

৮.১৪ নেভিগেশন সংক্রান্ত কীবোর্ড শর্টকাট

এগুলো bash এর ডিফল্ট (emacs মোড) শর্টকাট। পুরো শর্টকাট-তালিকা অধ্যায় ৬ আর পরিশিষ্ট ক-এ।

শর্টকাট	কাজ
`Tab`	নাম/কমান্ড পূরণ (completion)
`Tab Tab`	সব সম্ভাব্য বিকল্প দেখাও
`Alt+?`	সম্ভাব্য completion এর তালিকা দেখাও (`Tab Tab` এর মতো)
`Alt+*`	সব সম্ভাব্য completion লাইনে বসিয়ে দাও
`Alt+/`	ফাইলের নাম হিসেবে completion জোর করো
`Alt+.`	আগের কমান্ডের শেষ argument বসাও (যেমন `mkdir newdir` তারপর `cd Alt+.`)
`Ctrl+L`	স্ক্রিন পরিষ্কার (`clear` এর মতো)
`Ctrl+C`	চলমান কমান্ড থামাও / লাইন বাতিল
`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষে যাও
`Ctrl+W`	কার্সরের আগের একটা শব্দ মোছো (path এর ভুল অংশ মুছতে কাজের)
`Ctrl+U`	কার্সরের আগের পুরো লাইন মোছো
`Ctrl+R`	পুরনো কমান্ড খোঁজো (যেমন আগের লম্বা `cd` path)
`Shift+PageUp` / `PageDown`	টার্মিনালের আউটপুট স্ক্রল (বেশিরভাগ টার্মিনাল এমুলেটরে)

কমান্ড-ভিত্তিক "শর্টকাট":

লেখো	মানে
`cd`	home এ যাও
`cd -`	আগের ডিরেক্টরিতে
`cd ..`	উপরে
`~`	home
`~-` / `~+`	আগের / বর্তমান ডিরেক্টরি
`!$`	আগের কমান্ডের শেষ argument (history expansion)

```
$ mkdir -p ~/work/2026/reports
$ cd !$
cd ~/work/2026/reports
```

৮.১৫ সব মিলিয়ে একটা বাস্তব দৃশ্য

একজন অ্যাডমিনের সকাল:

```
$ cd /var/log                    # লগ ফোল্ডারে
$ ls -ltr | tail -5              # সবচেয়ে নতুন ৫টা ফাইল
$ ls -lhS | head                 # কোনটা সবচেয়ে বড়?
```

```
$ pushd /etc/nginx > /dev/null   # কনফিগ দেখতে যাই, লগের জায়গা stack এ থাকুক
$ ls sites-enabled/
default  mysite.conf
$ cp sites-enabled/mysite.conf{,.bak}   # ব্যাকআপ
$ popd > /dev/null               # লগে ফেরত
$ pwd
/var/log
$ ls nginx/*.log
nginx/access.log  nginx/error.log
$ cd                             # বাড়ি
```

হাতে-কলমে অনুশীলন

নিচের ল্যাবগুলো নিজের home ডিরেক্টরিতে নিরাপদে করা যাবে। (`mkdir`, `touch` এর বিস্তারিত পরের অধ্যায়ে, এখানে শুধু অনুশীলনের জায়গা বানাতে ব্যবহার করছি।)

ল্যাব ১: পরিবেশ তৈরি

```
$ cd
$ mkdir -p lab8/{docs,src/{utils,tests},music,.secret}
$ touch lab8/docs/{report,notes,todo}.txt
$ touch lab8/src/main.c lab8/src/utils/helper.{c,h}
$ touch lab8/music/song{1..12}.mp3
$ touch lab8/.config lab8/README.md
$ tree -a lab8
```

প্রত্যাশিত (আংশিক):

```
lab8
├── .config
├── .secret
├── README.md
├── docs
│   ├── notes.txt
│   ├── report.txt
│   └── todo.txt
├── music
│   ├── song1.mp3
...
```

(`tree` না থাকলে `ls -R lab8` ব্যবহার করো।)

ল্যাব ২: pwd ও cd

1. `cd ~/lab8/src/utils` চালাও, তারপর `pwd`।
2. `cd ../..` দিয়ে কোথায় পৌঁছলে? `pwd` দিয়ে যাচাই করো (উত্তর: `~/lab8`)।
3. `cd /etc` তারপর `cd -` — কোথায় ফিরলে?

4. `cd` (কিছু না দিয়ে) — এখন কোথায়?
5. `/` এ গিয়ে `cd ..` চালাও, তারপর `pwd`। কী দেখলে এবং কেন?

ল্যাব ৩: absolute vs relative

`~/lab8/docs` এ থেকে নিচের প্রতিটা ফাইলে **দুইভাবে** `ls` চালাও (absolute ও relative):

- `~/lab8/src/main.c`
- `~/lab8/music/song5.mp3`
- `/etc/hostname`

উদাহরণ: `ls ../src/main.c` আর `ls /home/<তোমার-নাম>/lab8/src/main.c`।

ল্যাব ৪: ls এর অপশন

```
$ cd ~/lab8
$ ls
$ ls -a
$ ls -A
$ ls -lh
$ ls -F
$ ls -ld docs
$ ls -i docs
$ ls -lt music | head -3
$ ls -v music
$ ls -R
```

প্রশ্ন: `ls music` আর `ls -v music` এর ক্রমে পার্থক্য কী? (ইঙ্গিত: `song10` আর `song2` কোথায় আছে দেখো।)

ল্যাব ৫: `ls -l` পড়া

```
$ ls -l ~/lab8 /etc/passwd /dev/null
```

প্রতিটা লাইনের জন্য খাতায় লেখো: টাইপ, owner এর permission, link count, owner, group, size। `/dev/null` এর size এর জায়গায় কী আছে আর কেন?

ল্যাব ৬: globbing

```
$ cd ~/lab8/music
$ echo song?.mp3
$ echo song??.mp3
$ echo song[1-3].mp3
$ echo song1*.mp3
$ echo song[!1]*.mp3
$ echo *.wav
$ shopt -s nullglob; echo *.wav; shopt -u nullglob
```

```
$ cd ..
$ echo *
$ echo .*
$ shopt -s dotglob; echo *; shopt -u dotglob
$ shopt -s globstar; echo **/*.c; shopt -u globstar
```

প্রতিটার output লিখে নিজের কাছে ব্যাখ্যা করো কেন এটা এসেছে।

ল্যাব ৭: brace expansion

```
$ echo backup-{mon,tue,wed}.tar.gz
$ echo {10..1..3}
$ cp ~/lab8/README.md{,.bak}
$ ls ~/lab8/README*
```

ল্যাব ৮: directory stack

```
$ cd ~/lab8
$ pushd docs
$ pushd ../src/utils
$ pushd /etc
$ dirs -v
$ pushd +2
$ popd
$ popd
$ dirs -v
```

প্রতিটা ধাপের পর `pwd` দিয়ে দেখো কোথায় আছ।

ল্যাব ৯: Tab completion

1. `cd ~/la` লিখে `Tab` চাপো।
2. `ls ~/lab8/music/song1` লিখে `Tab Tab` চাপো, কয়টা বিকল্প এলো?
3. `sys` লিখে `Tab Tab` চাপো, কোন কোন কমান্ড দেখাল?
4. `mkdir ~/lab8/temp` চালাও, তারপর `cd` লিখে `Alt+.` চাপো।

পরিষ্কার

```
$ cd
$ rm -r ~/lab8
```

> ⚠️ **সতর্কতা:** `rm -r` এর আগে path টা আবার দেখে নাও। এটা ফাইল চিরতরে মুছে দেয়, Trash এ যায় না (অধ্যায় ৯)।

সারসংক্ষেপ

কমান্ড / চিহ্ন	কাজ
`pwd`	বর্তমান ডিরেক্টরি দেখাও
`pwd -P`	symlink resolve করে আসল path
`cd DIR`	DIR এ যাও
`cd` / `cd ~`	home এ যাও
`cd -`	আগের ডিরেক্টরিতে
`cd ..`	parent এ
`cd /`	root এ
`/a/b`	absolute path (root থেকে)
`a/b`	relative path (cwd থেকে)
`.` / `..`	বর্তমান / parent ডিরেক্টরি
`~`, `~user`, `~+`, `~-`	home, অন্যের home, `$PWD`, `$OLDPWD`
`ls -l`	বিস্তারিত তালিকা
`ls -a` / `ls -A`	লুকানো ফাইলসহ (`.` `..` সহ / ছাড়া)
`ls -lh`	পড়ার মতো size
`ls -ltr`	পুরনো থেকে নতুন (নতুনটা নিচে)
`ls -lS`	বড় থেকে ছোট
`ls -R`	recursive
`ls -ld DIR`	ডিরেক্টরির নিজের তথ্য
`ls -i`	inode নম্বর
`ls -F`	টাইপ চিহ্ন (`/ * @`)
`tree -L 2`	২ স্তর পর্যন্ত গাছ
`pushd DIR` / `popd` / `dirs -v`	ডিরেক্টরি stack
`*` `?` `[abc]` `[!abc]`	globbing
`**`	সব স্তর (`shopt -s globstar`)
`{a,b}` `{1..10}` `{1..10..2}`	brace expansion
`Tab` / `Tab Tab`	completion / বিকল্প তালিকা
`Alt+.`	আগের কমান্ডের শেষ argument
`Ctrl+L`	স্ক্রিন পরিষ্কার

নিজেকে যাচাই করো

1. `cd` কেন একটা shell builtin, আলাদা প্রোগ্রাম হতে পারে না?
2. তুমি `/home/rahim/Documents` এ আছ। `/home/karim/notes.txt` এর relative path কী?
3. `ls -a` আর `ls -A` এর পার্থক্য কী?
4. `ls -l` এর একটা লাইন: `drwxr-x--- 5 rahim dev 4096 Nov 12 09:30 proj`। এখানে টাইপ কী, group এর permission কী, আর `5` সংখ্যাটা কী বোঝায়?
5. `ls -l /etc` আর `ls -ld /etc` এর পার্থক্য কী?
6. `echo *.txt` চালালে `*` কে expand করে কে — `echo` নাকি shell? কিছু না মিললে bash ডিফল্টে কী করে?
7. `echo {1..3}` আর `echo [1-3]` এর মধ্যে মূল পার্থক্য কী?
8. `cp app.conf{,.orig}` কমান্ডটা আসলে কী চালায়?
9. `pushd /etc`, `pushd /var`, `popd` চালানোর পর তুমি কোথায় থাকবে (শুরু `~` থেকে)?
10. স্ক্রিপ্টে `"~/backup"` লিখলে কী সমস্যা হবে, আর ঠিক উপায় কী?

▼ উত্তর দেখো

1. cwd প্রতিটা process এর নিজস্ব। আলাদা প্রোগ্রাম হলে child process নিজের cwd বদলে বেরিয়ে যেত, আর shell এর cwd অপরিবর্তিত থাকত। তাই shell কে নিজেই `chdir()` ডাকতে হয়।
2. `../../karim/notes.txt` (Documents → rahim → home, তারপর karim)।
3. `-a` সব লুকানো এন্ট্রিসহ `.` আর `..` দেখায়; `-A` লুকানো ফাইল দেখায় কিন্তু `.` আর `..` বাদ দেয়।
4. `d` = ডিরেক্টরি; group এর permission `r-x` (পড়া ও ঢোকা যাবে, লেখা যাবে না); `5` হলো hard link সংখ্যা, ডিরেক্টরির ক্ষেত্রে সাধারণত ২ + ৩টা সাব-ডিরেক্টরি।
5. `ls -l /etc` /etc এর ভেতরের সব ফাইলের তালিকা দেখায়; `ls -ld /etc` শুধু /etc ডিরেক্টরিটার নিজের তথ্য দেখায়।
6. Shell expand করে; `echo` আগেই তৈরি তালিকা পায়। কিছু না মিললে bash pattern টাকে অক্ষত (`*.txt`) রেখে দেয় (যদি না `nullglob` / `failglob` চালু থাকে)।
7. `{1..3}` ফাইল থাকুক না থাকুক সবসময় `1 2 3` তৈরি করে (brace expansion)। `[1-3]` হলো glob, শুধু cwd-তে `1`, `2` বা `3` নামের ফাইল থাকলে সেগুলোর সাথে মেলে, না থাকলে আক্ষরিক `[1-3]` থাকে।
8. `cp app.conf app.conf.orig` — অর্থাৎ ফাইলের একটা ব্যাকআপ কপি।
9. `/etc` এ। stack ছিল `/var /etc ~`; `popd` উপরের `/var` সরিয়ে `/etc` এ নিয়ে যায়।
10. উদ্ধৃতির ভেতরে tilde expand হয় না, তাই আক্ষরিক `~` নামের ফোল্ডার বোঝাবে। ঠিক উপায়: `"$HOME/backup"`।

অধ্যায় ৯: ফাইল ও ডিরেক্টরি ম্যানেজমেন্ট

আগের অধ্যায়ে তুমি শিখেছ কীভাবে ফাইল সিস্টেমে ঘুরে বেড়াতে হয় (`pwd` , `cd` , `ls`)। এবার সময় হাতে কাজ করার: নতুন ফাইল আর ফোল্ডার বানানো, কপি করা, সরানো, নাম বদলানো, মুছে ফেলা। লিনাক্স ব্যবহারকারী দিনে সবচেয়ে বেশি যে কমান্ডগুলো চালায়, এগুলো তার মধ্যে পড়ে। তাই এই অধ্যায়টা মন দিয়ে পড়ো। এখানে একটা ভুল কমান্ড, বিশেষ করে `rm` , তোমার কয়েক মাসের কাজ মুহূর্তে মুছে দিতে পারে।

এই অধ্যায়ে যা শিখবে

- `touch` দিয়ে খালি ফাইল বানানো এবং ফাইলের তিন রকম timestamp (atime, mtime, ctime) কী, কেন আছে
- `mkdir` ও `mkdir -p` দিয়ে এক কমান্ডে পুরো ফোল্ডার-গাছ বানানো
- `cp` দিয়ে কপি: `-r` , `-i` , `-v` , `-p` , `-a` , `-u` সহ গুরুত্বপূর্ণ সব অপশন
- `mv` দিয়ে নাম বদলানো ও সরানো, এবং `-i` , `-n` দিয়ে ভুল করে ওভাররাইট ঠেকানো
- `rm` ও `rmdir` দিয়ে মোছা, `rm -rf /` কেন ভয়ংকর, আর `trash-cli` দিয়ে নিরাপদে মোছা
- `stat` দিয়ে ফাইলের পুরো "জন্মকুণ্ডলী" দেখা
- নামের মধ্যে space, `-` , `*` , `$` ইত্যাদি বিশেষ অক্ষর থাকলে কীভাবে quoting করতে হয়
- ফাইল কপি, মুভ বা ডিলিট করলে ভেতরে inode আর data block-এর কী হয়
- `basename` , `dirname` , `realpath` দিয়ে পাথ নিয়ে কাজ
- ওয়াইল্ডকার্ড (glob) আর brace expansion দিয়ে একসাথে অনেক ফাইলে কাজ
- এসব কাজ দ্রুত করার কীবোর্ড শর্টকাট

৯.১ শুরুর আগে: ফাইল আসলে কী? (inode-এর এক ঝলক)

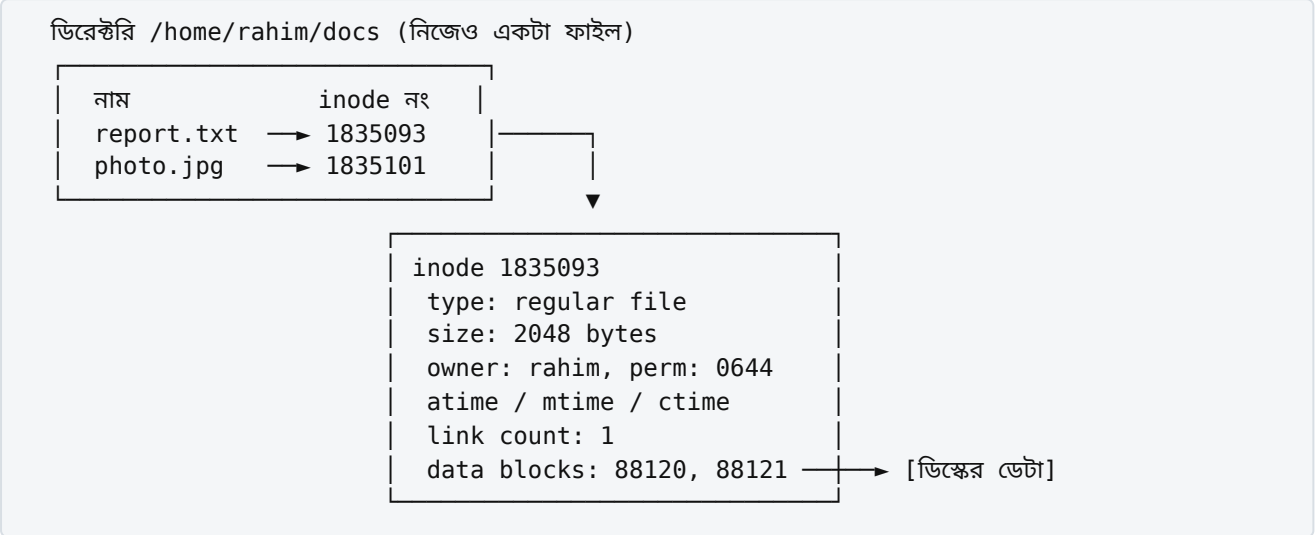
কমান্ড শেখার আগে একটা জিনিস মাথায় ঢুকিয়ে নাও। এটা বুঝলে বাকি সব কমান্ডের আচরণ নিজে থেকেই পরিষ্কার হয়ে যাবে।

লিনাক্সে একটা "ফাইল" তিনটা আলাদা অংশ দিয়ে তৈরি:

1. **Directory entry (নাম)**: ডিরেক্টরি আসলে একটা বিশেষ ফাইল, যার ভেতরে একটা তালিকা থাকে: "এই নাম → এই inode নম্বর"।
2. **inode (index node)**: ফাইলের metadata বা তথ্যপত্র। এতে থাকে ফাইলের সাইজ, মালিক (owner), পারমিশন, timestamp, link count, আর ডেটা ডিস্কের কোন কোন block-এ আছে তার ঠিকানা। **ফাইলের নাম inode-এ থাকে না।**
3. **Data blocks**: ডিস্কের যেখানে ফাইলের আসল কনটেন্ট থাকে।

একটা লাইব্রেরির কথা ভাবো:

- ক্যাটালগ কার্ডে লেখা থাকে বইয়ের নাম আর একটা নম্বর। এটা **directory entry**।
- নম্বর মিলিয়ে রেজিস্টার খাতায় পাবে বইয়ের সব তথ্য: লেখক, পৃষ্ঠাসংখ্যা, কোন তাকে আছে। এটা **inode**।
- তাকের উপর রাখা আসল বইটা হলো **data blocks**।



এই ছবি থেকে কয়েকটা জিনিস বোঝা যায়, যা পরে কাজে লাগবে:

- **নাম বদলানো (`mv`, একই ফাইলসিস্টেমে)** মানে শুধু ক্যাটালগ কার্ডে নাম পাল্টানো। বই সরে না, তাই কাজটা প্রায় সঙ্গে সঙ্গে শেষ হয়।
- **কপি (`cp`)** মানে নতুন inode, নতুন data block, পুরো ডেটার আরেকটা প্রতিলিপি। বড় ফাইলে তাই সময় লাগে।
- **মোছা (`rm`)** মানে ক্যাটালগ কার্ড ছিঁড়ে ফেলা। কোনো কার্ড আর ওই বইকে না দেখালে (link count 0) এবং কেউ বইটা পড়ছে না থাকলে, তবেই তাকের জায়গা খালি হিসেবে চিহ্নিত হয়।

inode আর লিংকের বিস্তারিত আলোচনা আছে অধ্যায় ১১-এ। এখানে শুধু ততটুকু নিচ্ছি যতটুকু এই অধ্যায়ের জন্য দরকার।

> 📝 **মনে রাখো:** লিনাক্সে "ফাইলের নাম" আর "ফাইল" এক জিনিস না। নাম হলো ডিরেক্টরির ভেতরের একটা এন্ট্রি, যেটা একটা inode-কে দেখায়।

৯.২ `touch`: খালি ফাইল বানানো ও timestamp বদলানো

কী করে

`touch` এর মূল কাজ আসলে ফাইলের **timestamp আপডেট করা**। ফাইল না থাকলে সেটা একটা খালি (০ বাইট) ফাইল বানায়। এই দ্বিতীয় আচরণটার জন্যই বেশিরভাগ মানুষ `touch` চেনে।

Syntax

```
touch [OPTION]... FILE...
```

সাধারণ ব্যবহার

```
$ touch notes.txt
$ ls -l notes.txt
-rw-r--r-- 1 rahim rahim 0 Mar 12 10:15 notes.txt
```

একসাথে অনেকগুলো:

```
$ touch a.txt b.txt c.txt
$ touch file{1..5}.log        # brace expansion: file1.log ... file5.log
$ ls
a.txt  b.txt  c.txt  file1.log  file2.log  file3.log  file4.log  file5.log
```

ফাইল আগে থেকেই থাকলে `touch` তার কনটেন্টে হাত দেয় না। শুধু সময়টা "এখন" করে দেয়।

তিন রকম timestamp: atime, mtime, ctime

প্রতিটা inode-এ অন্তত তিনটা সময় লেখা থাকে:

Timestamp	পুরো নাম	কখন বদলায়	উপমা
atime	access time	ফাইলের **কনটেন্ট পড়লে** (যেমন `cat`)	বইটা শেষ কবে কেউ খুলে পড়েছে
mtime	modification time	ফাইলের **কনটেন্ট বদলালে**	বইয়ের লেখা শেষ কবে বদলানো হয়েছে
ctime	change time	**inode বদলালে**: কনটেন্ট, পারমিশন, মালিক, নাম, link count	রেজিস্টার খাতার এন্ট্রি শেষ কবে বদলানো হয়েছে
btime	birth time	ফাইল তৈরির সময় (সব ফাইলসিস্টেমে থাকে না)	বইটা লাইব্রেরিতে কবে এসেছে

> ⚠️ **সতর্কতা:** অনেকে ভাবে ctime মানে "creation time"। ভুল। ctime হলো **change** time, মানে metadata শেষ কবে বদলেছে। তৈরির সময়কে বলে birth time (btime)। ext4, XFS, Btrfs-এর মতো আধুনিক ফাইলসিস্টেম এটা রাখে, আর নতুন coreutils-এর `stat` (Ubuntu 24.04, Debian 12, Fedora, RHEL 9) এটা `Birth:` লাইনে দেখায়।

কিছু নিয়ম মনে রাখো:

- mtime বদলালে ctime-ও বদলায়, কারণ কনটেন্ট বদলালে inode-এর সাইজ ও mtime ফিল্ডও বদলায়।
- `chmod`, `chown`, `mv` (নাম বদলানো), `ln` (হার্ড লিংক) শুধু ctime বদলায়, mtime না।

- **ctime তুমি ইচ্ছামতো সেট করতে পারো না।** `touch` দিয়েও না। inode-এ যেকোনো পরিবর্তন হলে কার্নেল নিজেই ctime বর্তমান সময়ে সেট করে দেয়। ফরেনসিক তদন্তে ctime এজন্যই এত দরকারি।

atime আর `relatime`: পারফরম্যান্সের গল্প

প্রতিবার ফাইল পড়লেই atime আপডেট করতে হলে প্রতিটা **read** অপারেশন একটা ডিস্ক **write** তৈরি করত। এটা ধীর, আর SSD-র আয়ু কমায়। তাই আধুনিক লিনাক্সে ডিফল্ট mount অপশন হলো **`relatime`**। এতে atime আপডেট হয় কেবল তখন, যখন:

- বর্তমান atime, mtime বা ctime-এর চেয়ে পুরনো, অথবা
- শেষ আপডেটের পর ২৪ ঘণ্টার বেশি পার হয়েছে।

এজন্য তুমি `cat` করার পরেও atime না বদলাতে দেখলে অবাক হয়ো না। `findmnt /` চালালে OPTIONS কলামে সাধারণত `relatime` দেখবে। আরও আছে `noatime` (কখনো atime আপডেট হবে না) এবং `strictatime` (প্রতিবার হবে)। mount অপশন নিয়ে বিস্তারিত আছে অধ্যায় ৩২-এ।

`touch`-এর গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-a`	শুধু atime বদলাও
`-m`	শুধু mtime বদলাও
`-c`, `--no-create`	ফাইল না থাকলে নতুন বানিও না
`-d STRING`, `--date=STRING`	নির্দিষ্ট সময় দাও (মানুষের ভাষায়, যেমন `"2 days ago"`, `"2024-01-15 14:30"`)
`-t STAMP`	`[[CC]YY]MMDDhhmm[.ss]` ফরম্যাটে সময় দাও
`-r FILE`, `--reference=FILE`	অন্য একটা ফাইলের timestamp কপি করো
`-h`, `--no-dereference`	symlink হলে লিংকের নিজের timestamp বদলাও, যে ফাইলকে দেখায় তার না

উদাহরণ

```
# নির্দিষ্ট তারিখ বসানো
$ touch -d "2024-01-15 14:30" old-report.txt
$ ls -l old-report.txt
-rw-r--r-- 1 rahim rahim 0 Jan 15  2024 old-report.txt

# -t ফরম্যাট: 2023 সালের 25 ডিসেম্বর রাত 11:59
$ touch -t 202312252359 xmas.txt

# শুধু mtime "২ দিন আগে" করে দাও
$ touch -m -d "2 days ago" build.log

# a.txt-এর সময় b.txt-এ বসাও
$ touch -r a.txt b.txt
```

```
# ফাইল থাকলে আপডেট, না থাকলে কিছু কোরো না
$ touch -c maybe.txt
```

> 💡 **টিপস:** `make` এর মতো build tool mtime দেখে ঠিক করে কোন ফাইল আবার compile করতে হবে। `touch source.c` চালালে `make` ভাববে ফাইলটা বদলেছে এবং আবার build করবে। অধ্যায় ৩১-এ এটা কাজে লাগবে।

> 📝 **মনে রাখো:** ফাইলে কনটেন্ট সহ নতুন ফাইল বানাতে চাইলে `touch` লাগে না। `echo "hello" > file.txt` বা এডিটর (অধ্যায় ১৩) ব্যবহার করো। `>` রিডাইরেকশন নিয়ে বিস্তারিত আছে অধ্যায় ১৯-এ।

৯.৩ `stat` : ফাইলের পুরো তথ্যপত্র

`ls -l` তোমাকে ফাইলের কিছু তথ্য দেখায়। `stat` সরাসরি inode খুলে **সব** তথ্য দেখায়।

Syntax

```
stat [OPTION]... FILE...
```

উদাহরণ ও আউটপুট

```
$ echo "hello linux" > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 1835093     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   rahim)   Gid: ( 1000/   rahim)
Access: 2025-03-12 10:20:41.118034512 +0600
Modify: 2025-03-12 10:20:41.118034512 +0600
Change: 2025-03-12 10:20:41.118034512 +0600
 Birth: 2025-03-12 10:20:41.118034512 +0600
```

লাইন ধরে মানে:

ফিল্ড	মানে
`Size: 12`	ফাইলে ১২ বাইট ("hello linux" = ১১ অক্ষর + ১টা newline)
`Blocks: 8`	ডিস্কে কতগুলো ৫১২-বাইট ইউনিট দখল করেছে (৮ × ৫১২ = ৪০৯৬, মানে একটা 8KB ব্লক)
`IO Block: 4096`	ফাইলসিস্টেমের পছন্দের I/O সাইজ
`regular file`	ফাইলের ধরন (directory, symbolic link ইত্যাদিও হতে পারে)

ফিল্ড	মানে
`Device: 8,2`	কোন ডিভাইসে আছে (major,minor নম্বর)
`Inode: 1835093`	inode নম্বর
`Links: 1`	কয়টা নাম (হার্ড লিংক) এই inode-কে দেখাচ্ছে
`Access: (0644/-rw-r--r--)`	পারমিশন, সংখ্যায় ও অক্ষরে (অধ্যায় ১৬)
`Uid` / `Gid`	মালিক ইউজার ও গ্রুপ
`Access/Modify/Change/Birth`	atime / mtime / ctime / btime

> 📝 **মনে রাখো:** ১২ বাইটের ফাইলও ডিস্কে ৪০৯৬ বাইট দখল করে, কারণ ফাইলসিস্টেম জায়গা দেয় পুরো block হিসেবে। `ls -l` দেখায় আসল সাইজ, `du` দেখায় দখল করা জায়গা।

timestamp বদলানো হাতে-কলমে দেখো

```
$ cat notes.txt > /dev/null        # পড়লাম → atime (relatime নিয়ম মেনে)
$ echo "more" >> notes.txt         # লিখলাম → mtime + ctime
$ chmod 600 notes.txt              # পারমিশন বদলালাম → শুধু ctime
$ stat -c 'atime=%x%nmtime=%y%nctime=%z' notes.txt
atime=2025-03-12 10:20:41.118034512 +0600
mtime=2025-03-12 10:22:05.402118833 +0600
ctime=2025-03-12 10:23:10.771003410 +0600
```

`stat`-এর গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-c FORMAT`, `--format=FORMAT`	নিজের মতো ফরম্যাটে আউটপুট (স্ক্রিপ্টে খুব কাজের)
`--printf=FORMAT`	`-c`-এর মতো, তবে `\n` এর মতো escape বোঝে আর শেষে নিজে থেকে newline দেয় না
`-L`, `--dereference`	symlink হলে লিংক যে ফাইলকে দেখায় তার তথ্য দেখাও
`-f`, `--file-system`	ফাইলের বদলে ফাইলসিস্টেমের তথ্য দেখাও
`-t`, `--terse`	সংক্ষিপ্ত, এক লাইনের আউটপুট

দরকারি format specifier

Specifier	মানে
`%n`	ফাইলের নাম
`%s`	সাইজ (বাইট)

Specifier	মানে
`%i`	inode নম্বর
`%h`	হার্ড লিংক সংখ্যা
`%U` / `%G`	মালিক ইউজার / গ্রুপের নাম
`%a` / `%A`	পারমিশন সংখ্যায় (644) / অক্ষরে (-rw-r--r--)
`%F`	ফাইলের ধরন
`%x` / `%y` / `%z` / `%w`	atime / mtime / ctime / btime (পড়ার মতো ফরম্যাটে)
`%X` / `%Y` / `%Z` / `%W`	একই সময়, Unix epoch সেকেন্ডে

```
$ stat -c '%n: %s bytes, inode %i, perm %a, owner %U' notes.txt
notes.txt: 17 bytes, inode 1835093, perm 600, owner rahim

$ stat -c '%F' /etc /etc/passwd /dev/null
directory
regular file
character special file
```

৯.৪ `mkdir`: ডিরেক্টরি বানানো

Syntax

```
mkdir [OPTION]... DIRECTORY...
```

সাধারণ ব্যবহার

```
$ mkdir projects
$ mkdir music videos photos         # একসাথে তিনটা
```

সমস্যা: মাঝের ফোল্ডার না থাকলে

```
$ mkdir projects/website/css
mkdir: cannot create directory 'projects/website/css': No such file or directory
```

`projects/website` নেই, তাই তার ভেতরে `css` বানানো যাচ্ছে না। এর সমাধান `-p`।

`mkdir -p`: পুরো পথ একবারে

```
$ mkdir -p projects/website/css
$ mkdir -pv projects/website/js/lib
```

```
mkdir: created directory 'projects/website/js'
mkdir: created directory 'projects/website/js/lib'
```

`-p` (parents) দুটো কাজ করে:

1. পথের মাঝে যা নেই সব বানিয়ে দেয়।
2. ডিরেক্টরি আগে থেকে থাকলে **error দেয় না**। এজন্য স্ক্রিপ্টে `mkdir -p` লেখাই নিরাপদ অভ্যাস, কারণ বারবার চালালেও ভাঙে না। এই গুণকে বলে *idempotent*।

brace expansion দিয়ে পুরো প্রজেক্টের কাঠামো

```
$ mkdir -p myapp/{src/{models,views,controllers},tests,docs,config}
$ tree myapp
myapp
├── config
├── docs
├── src
│   ├── controllers
│   ├── models
│   └── views
└── tests

7 directories, 0 files
```

(`tree` আলাদা প্যাকেজ: `sudo apt install tree` বা `sudo dnf install tree`।)

> 💡 **টিপস:** `{a,b,c}` brace expansion কাজটা করে **শেল**, `mkdir` না। কমান্ড চালানোর আগেই bash এটাকে `myapp/src/models myapp/src/views ...` এ ভেঙে ফেলে। কী হবে আগে দেখতে চাইলে সামনে `echo` বসাও: `echo mkdir -p myapp/{a,b}`। `{` আর `,`-এর আশেপাশে space দিলে কিন্তু expansion হবে না।

`mkdir`-এর অপশন

অপশন	কাজ
`-p`, `--parents`	মাঝের ডিরেক্টরিও বানাও; থাকলে error দিও না
`-v`, `--verbose`	প্রতিটা তৈরি ডিরেক্টরির নাম দেখাও
`-m MODE`, `--mode=MODE`	তৈরির সময়েই পারমিশন সেট করো (যেমন `-m 700`)

```
$ mkdir -m 700 private      # শুধু আমি ঢুকতে পারব
$ ls -ld private
drwx------ 2 rahim rahim 4096 Mar 12 10:40 private
```

> 📝 **মনে রাখো:** নতুন ডিরেক্টরির link count কমপক্ষে ২ হয়। একটা হলো প্যারেন্টে তার নাম, আরেকটা হলো তার নিজের ভেতরের `.`। প্রতিটা সাব-ডিরেক্টরি এর সাথে আরও ১ যোগ করে, কারণ সাব-ডিরেক্টরির `..` এটাকে দেখায়।

৯.৫ নামে space ও বিশেষ অক্ষর: quoting

এই অংশটা `cp`, `mv`, `rm` শেখার আগেই জানা দরকার, কারণ এখানে ভুল করলে ভুল ফাইল মুছে যেতে পারে।

সমস্যাটা কী

শেল space দেখে কমান্ডকে আলাদা আলাদা "শব্দে" (argument) ভাগ করে। ধরো তোমার একটা ফাইল আছে `my notes.txt`:

```
$ rm my notes.txt
rm: cannot remove 'my': No such file or directory
rm: cannot remove 'notes.txt': No such file or directory
```

শেল ভেবেছে তুমি দুটো ফাইল মুছতে বলেছ: `my` আর `notes.txt`। কপাল খারাপ হলে `notes.txt` নামে সত্যিই একটা ফাইল থাকতে পারত। তাহলে সেটা মুছে যেত।

সমাধান: তিন রকম quoting

পদ্ধতি	উদাহরণ	আচরণ
Single quote `'...'`	`'my notes.txt'`	ভেতরের **সবকিছু** হুবহু থাকে। `$`, `*`, `\` এর কোনো বিশেষ মানে থাকে না
Double quote `"..."`	`"my notes.txt"`	space রক্ষা করে, কিন্তু `$VAR`, `$(cmd)`, `` ` ``, `\` কাজ করে
Backslash `\`	`my\ notes.txt`	ঠিক পরের একটা অক্ষরের বিশেষ মানে বাতিল করে

```
$ touch 'my notes.txt'
$ ls -l "my notes.txt"
-rw-r--r-- 1 rahim rahim 0 Mar 12 10:45 'my notes.txt'
$ mv my\ notes.txt my_notes.txt
```

লক্ষ করো, `ls` নিজেই space-ওয়ালা নাম single quote দিয়ে দেখাচ্ছে। coreutils 8.25-এর পর থেকে টার্মিনালে এটাই ডিফল্ট আচরণ।

variable সবসময় double quote-এ রাখো

```
$ f="my notes.txt"
$ rm $f          # ভুল! ভেঙে দুটো argument হয়ে যায়
$ rm "$f"        # ঠিক
```

শেল স্ক্রিপ্টে এটাই সবচেয়ে বেশি বাগ তৈরি করে (অধ্যায় ৪১)।

বিশেষ অক্ষরের তালিকা

এই অক্ষরগুলো নামে থাকলে quote ছাড়া চলবে না:

অক্ষর	শেলের কাছে মানে
space, tab	argument আলাদা করে
`*` `?` `[ ]`	wildcard (glob)
`$`	variable
`'` `"` `` ` ``	quoting / command substitution
`\`	escape
`&` `;` `|`	কমান্ড আলাদা করা / ব্যাকগ্রাউন্ড / pipe
`<` `>`	রিডাইরেকশন
`(` `)` `{` `}`	subshell / brace expansion
`!`	history expansion (ইন্টার‍্যাক্টিভ bash-এ)
`#`	শব্দের শুরুতে থাকলে comment
`~`	শব্দের শুরুতে থাকলে home ডিরেক্টরি

```
$ touch 'price$100.txt' 'what?.txt' 'rock&roll.mp3'
$ ls
'price$100.txt'  'rock&roll.mp3'  'what?.txt'
```

নাম যদি `-` দিয়ে শুরু হয়

`-file.txt` নামের ফাইল থাকলে quote দিয়েও কাজ হয় না, কারণ quote শুধু শেলের জন্য। কমান্ড নিজে `-f...` দেখে সেটাকে অপশন ভাবে:

```
$ rm -file.txt
rm: invalid option -- 'l'
Try 'rm ./-file.txt' to get rid of the file '-file.txt'.
Try 'rm --help' for more information.
```

দুটো সমাধান আছে:

```
$ rm -- -file.txt      # "--" মানে: এখানে অপশন শেষ, এর পরে সব ফাইলের নাম
$ rm ./-file.txt       # path দিলে আর '-' দিয়ে শুরু হয় না
```

`--` এর নিয়মটা প্রায় সব GNU কমান্ডে কাজ করে।

> 💡 **টিপস:** সবচেয়ে সহজ উপায় **Tab completion**। `rm my` টাইপ করে `Tab` চাপো। শেল নিজেই `rm my\ notes.txt` বানিয়ে দেবে, escape সহ। এতে টাইপো হয় না, আর quoting নিয়ে মাথাও ঘামাতে হয় না।

> ⚠️ **সতর্কতা:** লিনাক্সে ফাইলের নামে `/` আর NUL (`\0`) ছাড়া যেকোনো অক্ষর বৈধ, এমনকি newline-ও। তবে নিজের ফাইলের নামে space ও বিশেষ অক্ষর এড়িয়ে চলো। `my_notes.txt` বা `my-notes.txt` লেখো। পরে নিজেরই সুবিধা হবে।

৯.৬ ওয়াইল্ডকার্ড (glob): একসাথে অনেক ফাইল

`cp` , `mv` , `rm` প্রায়ই একসাথে অনেক ফাইলে চালাতে হয়। এজন্য শেল দেয় **globbing**:

প্যাটার্ন	মেলে	উদাহরণ
`*`	যেকোনো সংখ্যক যেকোনো অক্ষর (শূন্যও)	`*.txt` → সব .txt
`?`	ঠিক একটা যেকোনো অক্ষর	`file?.log` → file1.log, fileA.log
`[abc]`	বন্ধনীর যেকোনো একটা অক্ষর	`[abc]*.txt`
`[0-9]`	পরিসরের একটা অক্ষর	`img[0-9].png`
`[!0-9]` বা `[^0-9]`	বন্ধনীর বাইরের একটা অক্ষর	`[!.]*`

```
$ ls
a.txt  b.txt  c.log  img1.png  img2.png  img10.png
$ ls *.txt
a.txt  b.txt
$ ls img?.png
img1.png  img2.png
$ ls img[0-9]*.png
img10.png  img1.png  img2.png
```

ভেতরে কী হয়: glob শেল নিজে expand করে, কমান্ড চলার **আগেই**। `rm *.txt` চালালে `rm` আসলে পায় `rm a.txt b.txt` । `rm` কখনো `*` দেখতেই পায় না।

```
তুমি টাইপ করো:   rm *.txt
        │
        ▼
bash: ডিরেক্টরি পড়ে প্যাটার্ন মেলায়
        │
        ▼
আসলে চলে:        rm a.txt b.txt     ← rm শুধু এটাই দেখে
```

> 📝 **মনে রাখো:** `*` ডিফল্টে hidden ফাইল (`.` দিয়ে শুরু) ধরে না। কোনো ফাইল না মিললে bash প্যাটার্নটা যেমন আছে তেমনই কমান্ডকে দিয়ে দেয়, যেমন `ls *.xyz` তখন error দেয় `cannot access '*.xyz'`।

> 💡 **টিপস:** বিপজ্জনক কাজের আগে সবসময় একই glob দিয়ে `ls` বা `echo` চালাও: `echo rm *.log`। কী মুছবে আগে দেখে নাও, তারপর `echo` বাদ দিয়ে চালাও।

৯.৭ `cp`: কপি করা

Syntax

```
cp [OPTION]... SOURCE DEST               # একটা ফাইল কপি / নতুন নামে কপি
cp [OPTION]... SOURCE... DIRECTORY       # অনেক ফাইল একটা ডিরেক্টরিতে
cp [OPTION]... -t DIRECTORY SOURCE...    # target আগে দিয়ে
```

মৌলিক ব্যবহার

```
$ cp report.txt report-backup.txt        # একই জায়গায় নতুন নামে
$ cp report.txt ~/Documents/             # অন্য ডিরেক্টরিতে, একই নামে
$ cp report.txt ~/Documents/final.txt    # অন্য ডিরেক্টরিতে, নতুন নামে
$ cp a.txt b.txt c.txt backup/           # অনেক ফাইল একসাথে
$ cp *.jpg ~/Pictures/                   # glob দিয়ে
```

শেষ argument যদি একটা existing ডিরেক্টরি হয়, ফাইল তার **ভেতরে** যায়। না হলে শেষ argument-টা নতুন ফাইলের নাম হয়।

> ⚠️ **সতর্কতা:** `cp` ডিফল্টে **কোনো প্রশ্ন না করেই** গন্তব্যের ফাইল ওভাররাইট করে দেয়। `cp new.txt important.txt` চালালে `important.txt`-এর পুরনো কনটেন্ট চিরতরে হারিয়ে যায়।

ডিরেক্টরি কপি: `-r`

```
$ cp projects backup
cp: -r not specified; omitting directory 'projects'

$ cp -r projects backup
```

`-r` (বা `-R`) মানে recursive: ভেতরের সব ফাইল আর সাব-ডিরেক্টরি সহ কপি।

একটা সূক্ষ্ম নিয়ম আছে যেটা নতুনদের বিভ্রান্ত করে:

- `backup` **না থাকলে**: `cp -r projects backup` চালালে `backup` তৈরি হয়, আর তার ভেতরে থাকে `projects`-এর কনটেন্ট।
- `backup` **আগে থেকে থাকলে**: একই কমান্ড `backup/projects` তৈরি করে।

তাই একই কমান্ড দ্বিতীয়বার চালালে ভিন্ন ফল আসে! শুধু কনটেন্ট কপি করতে চাইলে লেখো:

```
$ cp -r projects/. backup/      # projects-এর ভেতরের সব (hidden সহ) backup-এ
```

(`rsync`-এ trailing slash-এর মানে আবার আলাদা। সেটা আছে অধ্যায় ৩৫-এ।)

`-i`: ওভাররাইটের আগে জিজ্ঞেস করো

```
$ cp -i new.txt important.txt
cp: overwrite 'important.txt'? n
```

`y` দিলে ওভাররাইট হবে, `n` বা অন্য কিছু দিলে বাদ যাবে।

`-v`: কী হচ্ছে দেখাও

```
$ cp -v *.txt backup/
'a.txt' -> 'backup/a.txt'
'b.txt' -> 'backup/b.txt'
```

`-p`: মূল তথ্য ধরে রাখো

সাধারণ `cp`-তে নতুন ফাইলের মালিক হও তুমি, mtime হয় "এখন", আর পারমিশন হয় umask অনুযায়ী (বিস্তারিত অধ্যায় ১৬-এ)। `-p` দিলে মূল ফাইলের **mode (পারমিশন), ownership আর timestamps** ধরে রাখা হয়।

```
$ ls -l old.txt
-rw-r----- 1 rahim rahim 120 Jan 15  2024 old.txt
$ cp old.txt c1.txt
$ cp -p old.txt c2.txt
$ ls -l c1.txt c2.txt
-rw-r----- 1 rahim rahim 120 Mar 12 11:02 c1.txt
-rw-r----- 1 rahim rahim 120 Jan 15  2024 c2.txt
```

> 📝 **মনে রাখো:** ownership ধরে রাখতে পারে শুধু root। সাধারণ ইউজার অন্যের মালিকানার ফাইল `-p` দিয়ে কপি করলে মালিক নিজেই হয়ে যায়।

`-a` : আর্কাইভ মোড (ব্যাকআপের জন্য সেরা)

`-a` = `-dR --preserve=all`। সহজ ভাষায়:

- recursive (`-R`)
- symlink-কে symlink হিসেবেই কপি করে, যে ফাইলকে দেখায় সেটা না (`-d` = `--no-dereference --preserve=links`)
- mode, ownership, timestamps, হার্ড লিংক, এবং সম্ভব হলে context, xattr ও ACL সব ধরে রাখে

```
$ sudo cp -a /etc /backup/etc-2025-03-12
```

ফোল্ডারের "হুবহু" কপি চাইলে `cp -a` ব্যবহার করো।

`-u` : শুধু নতুন হলে কপি করো

```
$ cp -u *.txt backup/
```

গন্তব্যে ফাইল না থাকলে, অথবা উৎসের mtime গন্তব্যের চেয়ে নতুন হলে, তবেই কপি হয়। এটা সহজ "সিঙ্ক" এর মতো কাজ করে। আসল সিঙ্কের জন্য `rsync` ভালো।

`cp` -এর সম্পূর্ণ অপশন টেবিল

অপশন	কাজ
`-r` , `-R` , `--recursive`	ডিরেক্টরি সহ ভেতরের সব কপি
`-i` , `--interactive`	ওভাররাইটের আগে জিজ্ঞেস
`-n` , `--no-clobber`	existing ফাইল কখনো ওভাররাইট করবে না
`-f` , `--force`	গন্তব্য খোলা না গেলে মুছে আবার চেষ্টা
`-v` , `--verbose`	প্রতিটা কপির বিবরণ দেখাও
`-p`	mode, ownership, timestamps ধরে রাখো
`--preserve=LIST`	নির্দিষ্ট গুণ ধরে রাখো (`mode,ownership,timestamps,links,context,xattr,all`)
`-a` , `--archive`	`-dR --preserve=all` : হুবহু কপি
`-u` , `--update`	উৎস নতুন হলে বা গন্তব্য না থাকলেই কপি
`-l` , `--link`	কপির বদলে হার্ড লিংক বানাও
`-s` , `--symbolic-link`	কপির বদলে symlink বানাও
`-L` , `--dereference`	symlink-এর লক্ষ্য ফাইল কপি করো
`-P` , `--no-dereference`	symlink-কে symlink হিসেবেই কপি করো
`-b` , `--backup`	ওভাররাইটের আগে পুরনোটার ব্যাকআপ রাখো (`file~`)

অপশন	কাজ
`-t DIR`, `--target-directory=DIR`	গন্তব্য ডিরেক্টরি আগে দাও
`-T`, `--no-target-directory`	DEST-কে সবসময় ফাইল হিসেবে ধরো
`-x`, `--one-file-system`	অন্য ফাইলসিস্টেমের mount point-এ ঢুকবে না
`--reflink[=WHEN]`	Btrfs/XFS-এ copy-on-write ক্লোন (তাৎক্ষণিক, জায়গা বাঁচায়)
`--sparse=WHEN`	sparse ফাইলের ফাঁকা অংশ সামলানো (`auto`, `always`, `never`)

> 📝 **মনে রাখো:** coreutils 9.x-এ `-n` এর আচরণ কয়েকবার বদলেছে (ফাইল বাদ পড়লে exit status কী হবে, সেটা নিয়ে)। স্ক্রিপ্টে নিশ্চিত আচরণ চাইলে নতুন coreutils-এ `--update=none` ব্যবহার করতে পারো। মানে "গন্তব্যে থাকলে বাদ দাও, error ছাড়া"।

```
$ cp -b notes.txt backup/notes.txt
$ ls backup/
notes.txt  notes.txt~

$ cp -t backup/ a.txt b.txt c.txt         # xargs/find এর সাথে খুব কাজের
```

কপি করলে ভেতরে কী হয় (inode দৃষ্টিকোণ)

```
$ ls -i report.txt
1835093 report.txt
$ cp report.txt copy.txt
$ ls -i report.txt copy.txt
1835093 report.txt
1835240 copy.txt
```

inode নম্বর আলাদা। মানে `cp` সম্পূর্ণ নতুন একটা ফাইল তৈরি করেছে। ধাপে ধাপে:

```
  cp report.txt copy.txt
  ──────────────────────────────────────────────────────────────
  1. open("report.txt", O_RDONLY)        ← উৎস পড়ার জন্য খোলো
  2. open("copy.txt", O_WRONLY|O_CREAT)  ← কার্নেল নতুন inode দেয় (1835240)
                                           ডিরেক্টরিতে "copy.txt → 1835240" এন্ট্রি বসে
  3. loop: read() → write()              ← ডেটা নতুন data block-এ লেখা হয়
     (আধুনিক coreutils দ্রুততার জন্য copy_file_range() বা
      Btrfs/XFS-এ reflink ব্যবহার করতে পারে)
  4. -p/-a থাকলে: chmod/chown/utimensat দিয়ে metadata বসাও
  5. close() দুটোই
  ──────────────────────────────────────────────────────────────

   report.txt ──► inode 1835093 ──► [blocks A B C]
   copy.txt   ──► inode 1835240 ──► [blocks X Y Z]  ← সম্পূর্ণ আলাদা কপি
```

ফলাফল:

- একটা বদলালে অন্যটায় কোনো প্রভাব পড়ে না।
- ডিস্কে দ্বিগুণ জায়গা লাগে (reflink ছাড়া)।
- নতুন ফাইলের সময় "এখন" হয়, যদি না `-p` / `-a` দাও।

`cp -l` দিলে নতুন inode তৈরি হয় না। একই inode-এর জন্য আরেকটা নাম (হার্ড লিংক) তৈরি হয়। এ নিয়ে বিস্তারিত অধ্যায় ১১-এ।

> 💡 **টিপস:** Btrfs বা XFS ফাইলসিস্টেমে (Fedora-র ডিফল্ট Btrfs) coreutils 9.0 থেকে `cp` ডিফল্টে `--reflink=auto` ব্যবহার করে। এতে ১০ GB ফাইলের কপিও মুহূর্তে হয়ে যায়: শুরুতে দুটো inode একই block শেয়ার করে, আর কেউ কিছু বদলালে তখনই নতুন block লেখা হয় (copy-on-write)। ext4 এটা সাপোর্ট করে না।

৯.৮ `mv`: সরানো ও নাম বদলানো

লিনাক্সে আলাদা কোনো "rename" কমান্ড লাগে না। নাম বদলানো আর সরানো আসলে একই কাজ: ফাইলের ঠিকানা (path) বদলানো। দুটোই করে `mv` (move)।

Syntax

```
mv [OPTION]... SOURCE DEST
mv [OPTION]... SOURCE... DIRECTORY
mv [OPTION]... -t DIRECTORY SOURCE...
```

উদাহরণ

```
$ mv draft.txt final.txt                # নাম বদলানো
$ mv final.txt ~/Documents/             # সরানো (নাম একই)
$ mv final.txt ~/Documents/report.txt   # সরানো + নাম বদলানো
$ mv *.mp3 ~/Music/                     # অনেক ফাইল সরানো
$ mv old_project new_project            # ডিরেক্টরির নাম বদল (-r লাগে না!)
```

> 📝 **মনে রাখো:** `cp` আর `rm`-এর মতো `mv`-তে ডিরেক্টরির জন্য `-r` লাগে না। ডিরেক্টরি সরানো মানে শুধু তার একটা এন্ট্রি সরানো, ভেতরের ফাইলগুলোয় হাত দিতে হয় না (একই ফাইলসিস্টেমে হলে)।

`-i`, `-n`, `-f`: ওভাররাইট নিয়ন্ত্রণ

`cp`-এর মতোই `mv` ডিফল্টে প্রশ্ন না করে গন্তব্যের ফাইল ওভাররাইট করে। ফলে গন্তব্যের পুরনো ফাইল চিরতরে হারিয়ে যায়।

```
$ mv -i a.txt b.txt
mv: overwrite 'b.txt'? n

$ mv -n a.txt b.txt        # b.txt থাকলে চুপচাপ বাদ দেবে, কিছুই জিজ্ঞেস করবে না
$ mv -f a.txt b.txt        # কোনো প্রশ্ন ছাড়াই ওভাররাইট (-i alias থাকলেও)
```

`-i`, `-n`, `-f` একসাথে দিলে **শেষে যেটা দেওয়া হয়েছে সেটাই কার্যকর** হয়।

`mv` -এর অপশন টেবিল

অপশন	কাজ
`-i`, `--interactive`	ওভাররাইটের আগে জিজ্ঞেস
`-n`, `--no-clobber`	existing ফাইল কখনো ওভাররাইট করবে না
`-f`, `--force`	প্রশ্ন ছাড়া ওভাররাইট
`-v`, `--verbose`	কী হচ্ছে দেখাও
`-u`, `--update`	উৎস নতুন হলে বা গন্তব্য না থাকলেই সরাও
`-b`, `--backup`	ওভাররাইট হওয়া ফাইলের ব্যাকআপ রাখো (`file~`)
`-t DIR`, `--target-directory=DIR`	গন্তব্য ডিরেক্টরি আগে দাও
`-T`, `--no-target-directory`	DEST-কে ডিরেক্টরি হিসেবে না ধরে নাম হিসেবে ধরো

```
$ mv -v report.txt archive/
renamed 'report.txt' -> 'archive/report.txt'
```

ভেতরে কী হয়: একই vs ভিন্ন ফাইলসিস্টেম

এই অংশটা খুব গুরুত্বপূর্ণ। এটা বুঝলেই পরিষ্কার হবে কেন ৫০ GB ফাইল এক জায়গায় মুহূর্তে সরে যায়, আর অন্য জায়গায় ১০ মিনিট লাগে।

কেস ১: একই ফাইলসিস্টেমে (যেমন `/home/rahim/a` থেকে `/home/rahim/b`)

`mv` কার্নেলের `rename()` system call ডাকে। কার্নেল শুধু:

1. পুরনো ডিরেক্টরি থেকে নামের এন্ট্রিটা সরায়,
2. নতুন ডিরেক্টরিতে একই inode নম্বর দিয়ে নতুন এন্ট্রি বসায়।

ডেটা একটুও নড়ে না। **inode নম্বর একই থাকে**, আর কাজটা মুহূর্তে শেষ হয়, ফাইল যত বড়ই হোক।

```
$ ls -i movie.mkv
2097321 movie.mkv
$ mv movie.mkv ~/Videos/
$ ls -i ~/Videos/movie.mkv
2097321 /home/rahim/Videos/movie.mkv      ← একই inode!
```

```
আগে:  ~/            ──► "movie.mkv" → inode 2097321 ──► [data]
       ~/Videos/     ──► (খালি)

rename()

পরে:  ~/            ──► (এন্ট্রি নেই)
       ~/Videos/     ──► "movie.mkv" → inode 2097321 ──► [data]  (একই ডেটা)
```

`rename()` **atomic**। মানে অন্য কোনো প্রোগ্রাম কখনো এমন অবস্থা দেখবে না যেখানে ফাইলটা দুই জায়গাতেই আছে বা কোথাও নেই। এজন্য অনেক প্রোগ্রাম ফাইল নিরাপদে আপডেট করে এভাবে: আগে `file.tmp`-এ লেখে, তারপর `mv file.tmp file`।

কেস ২: ভিন্ন ফাইলসিস্টেমে (যেমন `/home` থেকে USB ড্রাইভ `/media/usb`)

inode নম্বর শুধু নিজের ফাইলসিস্টেমের ভেতরে অর্থ বহন করে। তাই `rename()` এখানে `EXDEV` error দেয় ("cross-device link")। তখন `mv` নিজেই:

1. ফাইল **কপি** করে (নতুন ফাইলসিস্টেমে নতুন inode, `cp -a`-এর মতো metadata সহ),
2. কপি সফল হলে উৎসটা **মুছে** দেয়।

তাই এটা ধীর, আর inode নম্বর বদলে যায়।

```
/home (ext4)                    /media/usb (exFAT)
inode 2097321 ──copy data──►   নতুন inode 15
     │
     └── কপি সফল হলে unlink (মুছে ফেলা)
```

> ⚠️ **সতর্কতা:** অন্য ফাইলসিস্টেমে বড় ডিরেক্টরি `mv` করার সময় মাঝপথে থামিয়ে দিলে (Ctrl+C, পাওয়ার চলে যাওয়া) কিছু ফাইল এক জায়গায় আর কিছু আরেক জায়গায় থেকে যেতে পারে। গুরুত্বপূর্ণ ডেটার ক্ষেত্রে আগে `cp -a` বা `rsync` দিয়ে কপি করো, যাচাই করো, তারপর উৎস মোছো।

একসাথে অনেক ফাইলের নাম বদলানো

`mv` একবারে একটা ফাইলের নাম বদলায়। অনেকগুলোর জন্য শেল লুপ ব্যবহার করো:

```
# সব .txt কে .md বানাও
$ for f in *.txt; do mv -- "$f" "${f%.txt}.md"; done
```

`${f%.txt}` মানে variable থেকে শেষের `.txt` কেটে ফেলা (অধ্যায় ৪২)। এছাড়া `rename` নামে একটা টুল আছে, তবে distro-ভেদে তার দুটো আলাদা সংস্করণ:

Distro	কমান্ড	উদাহরণ
Debian/Ubuntu (`rename` প্যাকেজ, Perl-ভিত্তিক)	`rename`	`rename 's/\.txt$/.md/' *.txt`

Distro	কমান্ড	উদাহরণ
Fedora/RHEL (util-linux)	`rename`	`rename .txt .md *.txt`

দুটোরই `-n` অপশন আছে, যেটা আসলে কিছু না বদলে শুধু দেখায় কী হবে।

৯.৯ `rm`: মোছা

সবার আগে একটা কঠিন সত্য

> ⚠️ **সতর্কতা:** টার্মিনালের `rm` দিয়ে মোছা ফাইল **Trash/Recycle Bin-এ যায় না।** কোনো "Undo" নেই। একবার মুছলে সাধারণত চিরতরে চলে যায়।

Syntax

```
rm [OPTION]... FILE...
```

মৌলিক ব্যবহার

```
$ rm old.txt
$ rm a.txt b.txt c.txt
$ rm *.tmp
```

write-protected ফাইল হলে `rm` প্রশ্ন করে:

```
$ chmod 444 locked.txt
$ rm locked.txt
rm: remove write-protected regular file 'locked.txt'? y
```

ডিরেক্টরি মোছা: `-r`

```
$ rm myfolder
rm: cannot remove 'myfolder': Is a directory
$ rm -r myfolder
```

`-r` ভেতরের সবকিছু সহ ডিরেক্টরি মুছে দেয়।

`-f`, `-i`, `-I`

```
$ rm -i *.txt
rm: remove regular file 'a.txt'? y
```

```
rm: remove regular empty file 'b.txt'? n

$ rm -I *.log           # ৩টার বেশি ফাইল বা -r হলে একবারই জিজ্ঞেস করে
rm: remove 5 arguments? y

$ rm -f nofile.txt      # ফাইল না থাকলেও error দেবে না, কিছু জিজ্ঞেসও করবে না
```

`rm`-এর অপশন টেবিল

অপশন	কাজ
`-r`, `-R`, `--recursive`	ডিরেক্টরি ও তার ভেতরের সব মোছো
`-f`, `--force`	কোনো প্রশ্ন নয়; না থাকা ফাইলের জন্য error নয়
`-i`	প্রতিটা ফাইলের আগে জিজ্ঞেস
`-I`	৩টার বেশি ফাইল বা recursive হলে একবার জিজ্ঞেস (`-i`-এর চেয়ে কম বিরক্তিকর)
`-d`, `--dir`	খালি ডিরেক্টরি মোছো
`-v`, `--verbose`	কী মুছছে দেখাও
`--preserve-root`	`/` মুছতে দেবে না (**ডিফল্টে চালু**)
`--no-preserve-root`	`/`-এর সুরক্ষা বন্ধ (**কখনো ব্যবহার কোরো না**)
`--one-file-system`	recursive মোছার সময় অন্য ফাইলসিস্টেমের mount point-এ ঢুকবে না

```
$ rm -rv project/
removed 'project/main.c'
removed 'project/README'
removed directory 'project/'
```

ভেতরে কী হয়: `unlink()`

`rm` আসলে ডেটা "মোছে" না। এটা কার্নেলের `unlink()` (ডিরেক্টরির জন্য `rmdir()`) ডাকে:

```
rm report.txt
──────────────────────────────────────────────────────────
1. ডিরেক্টরি থেকে "report.txt → 1835093" এন্ট্রি সরাও
2. inode 1835093-এর link count এক কমাও (1 → 0)
3. link count == 0 এবং কোনো প্রসেস ফাইলটা খোলা রাখেনি?
      হ্যাঁ → inode ও data block "খালি" হিসেবে চিহ্নিত
      না  → প্রসেস ফাইল বন্ধ না করা পর্যন্ত ডেটা থেকে যায়
──────────────────────────────────────────────────────────
```

এর তিনটা বাস্তব ফল আছে:

1. **ফাইল মোছার জন্য দরকার ডিরেক্টরির write পারমিশন, ফাইলের না।** কারণ তুমি আসলে ডিরেক্টরির তালিকা বদলাচ্ছ (অধ্যায় ১৬)।
2. **ডিস্ক ফুল, বড় লগ ফাইল মুছলাম, তবু জায়গা খালি হলো না!** কারণ কোনো প্রসেস (যেমন চালু থাকা সার্ভিস) ফাইলটা এখনো খোলা রেখেছে। `lsof +L1` দিয়ে এমন "deleted কিন্তু খোলা" ফাইল খুঁজে পাবে,

আর সার্ভিস রিস্টার্ট করলে জায়গা ফেরত আসবে। এর বিস্তারিত অধ্যায় ৪৬-এ।

3. **ডেটা তাৎক্ষণিক ওভাররাইট হয় না।** তাই বিশেষ টুল দিয়ে কখনো কখনো উদ্ধার করা যায় (নিশ্চয়তা নেই, SSD-তে TRIM থাকলে প্রায় অসম্ভব)। উল্টো দিকে, গোপন ফাইল নিশ্চিতভাবে মুছতে `shred -u file` আছে, তবে journaling ফাইলসিস্টেম আর SSD-তে এটাও পুরোপুরি নির্ভরযোগ্য না।

`rm -rf /` এর বিপদ

`rm -rf /` মানে: "রুট থেকে শুরু করে **সবকিছু**, কোনো প্রশ্ন ছাড়া মুছে ফেলো।" এতে পুরো অপারেটিং সিস্টেম, তোমার সব ডেটা, আর mount করা সব ডিস্ক (USB, নেটওয়ার্ক ড্রাইভ) মুছে যেত।

আধুনিক GNU `rm`-এ `--preserve-root` ডিফল্টে চালু থাকে, তাই সরাসরি চালালে আটকে দেয়:

```
$ sudo rm -rf /
rm: it is dangerous to operate recursively on '/'
rm: use --no-preserve-root to override this failsafe
```

কিন্তু এই সুরক্ষা খুব সীমিত। নিচের কমান্ডগুলো আটকায় না:

```
sudo rm -rf /*          # glob expand হয়ে /bin /boot /etc ... হয়ে যায়, "/" আর থাকে না!
sudo rm -rf / home      # space-এর ভুল: আসলে "/" আর "home" দুটো
rm -rf ~ /Downloads     # space-এর ভুল: পুরো home ডিরেক্টরি চলে যাবে
```

স্ক্রিপ্টে সবচেয়ে কুখ্যাত বাগ হলো খালি variable:

```
# ভয়ংকর:
rm -rf "$BUILD_DIR"/*
# BUILD_DIR খালি বা unset হলে এটা হয়ে যায়:  rm -rf /*
```

নিরাপদ বিকল্প:

```
rm -rf "${BUILD_DIR:?BUILD_DIR is not set}"/*   # খালি হলে স্ক্রিপ্ট error দিয়ে থেমে যায়
```

অথবা স্ক্রিপ্টের শুরুতে `set -u` দাও (অধ্যায় ৪৩)।

> ⚠️ **সতর্কতা — `rm`-এর সোনার নিয়ম:**
>
> 1. `rm -rf` টাইপ করার আগে **একবার থামো**। কোন ডিরেক্টরিতে আছ (`pwd`) আর কী মুছছ, পড়ে দেখো।
> 2. glob থাকলে আগে `ls` দিয়ে দেখো: `ls *.log` → ঠিক থাকলে `rm *.log`।
> 3. `sudo rm -rf` হলো সবচেয়ে বিপজ্জনক সংমিশ্রণ। root-এর কোনো পারমিশন বাধা নেই।
> 4. ইন্টারনেট থেকে কপি করা কমান্ড বুঝে না চালালে কখনো চালাবে না। "`sudo rm -rf / --no-preserve-root` চালাও, তোমার কম্পিউটার দ্রুত হবে" ধরনের মজা অনলাইনে ঘোরে।
> 5. নিয়মিত ব্যাকআপ রাখো (অধ্যায় ৩৫)। এটাই শেষ সুরক্ষা।

`rm -i` alias: distro-ভেদে পার্থক্য

Fedora/RHEL-এ **root** ইউজারের `~/.bashrc`-এ ডিফল্টে এগুলো থাকে:

```
alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'
```

Ubuntu/Debian-এ এমন alias ডিফল্টে থাকে না। নিজের জন্য যোগ করতে চাইলে `~/.bashrc`-এ `alias rm='rm -I'` লিখতে পারো (অধ্যায় ৪৪)। তবে alias-এর উপর পুরো ভরসা কোরো না: অন্য মেশিনে গেলে, `sudo` দিয়ে চালালে, বা স্ক্রিপ্টে চালালে alias নাও থাকতে পারে। alias এড়িয়ে আসল কমান্ড চালাতে লেখো `\rm` বা `command rm`।

নিরাপদ বিকল্প: `trash-cli`

GUI ফাইল ম্যানেজারে Delete চাপলে ফাইল Trash-এ যায়। টার্মিনালে সেই সুবিধা দেয় `trash-cli`। এটা FreeDesktop Trash স্পেসিফিকেশন মেনে চলে, তাই GNOME/KDE-র Trash-এর সাথেই শেয়ার হয়।

```
# ইনস্টল
$ sudo apt install trash-cli          # Ubuntu/Debian
$ sudo dnf install trash-cli          # Fedora (RHEL-এ EPEL লাগতে পারে বা pipx install
trash-cli)
```

কমান্ড	কাজ
`trash-put FILE...` (বা `trash FILE`)	ফাইল/ডিরেক্টরি Trash-এ পাঠাও
`trash-list`	Trash-এ কী আছে দেখাও
`trash-restore`	ইন্টার‍্যাক্টিভভাবে ফাইল ফিরিয়ে আনো
`trash-rm PATTERN`	Trash থেকে নির্দিষ্ট ফাইল চিরতরে মোছো
`trash-empty`	পুরো Trash খালি করো
`trash-empty 30`	৩০ দিনের পুরনো জিনিস মোছো

```
$ trash-put report.txt old_project/
$ trash-list
2025-03-12 11:30:02 /home/rahim/report.txt
2025-03-12 11:30:02 /home/rahim/old_project
$ trash-restore
   0 2025-03-12 11:30:02 /home/rahim/report.txt
   1 2025-03-12 11:30:02 /home/rahim/old_project
What file to restore [0..1]: 0
```

Trash আসলে থাকে `~/.local/share/Trash/`-এ (`files/` আর `info/` সাব-ডিরেক্টরিতে)। GNOME সিস্টেমে বাড়তি কিছু ইনস্টল না করেই `gio trash FILE` দিয়েও একই কাজ করা যায়।

> 💡 **টিপস:** অনেকে `alias rm='echo "Use trash-put or \rm"'` লিখে রাখে, যাতে অভ্যাসবশত `rm` টাইপ করলেও থামতে হয়। নিজের প্রতিদিনের কাজে `trash-put` ব্যবহারের অভ্যাস করো, আর `rm` রাখো স্ক্রিপ্ট ও নিশ্চিত কাজের জন্য।

৯.১০ `rmdir`: খালি ডিরেক্টরি মোছা

`rmdir` শুধু **খালি** ডিরেক্টরি মোছে। শুনতে সীমাবদ্ধ মনে হলেও এটাই এর সুবিধা: ভুল করে ভরা ডিরেক্টরি মোছার কোনো সম্ভাবনা নেই।

```
$ rmdir empty_folder
$ rmdir photos
rmdir: failed to remove 'photos': Directory not empty
```

অপশন	কাজ
`-p`, `--parents`	ডিরেক্টরি আর তার খালি প্যারেন্টগুলোও মোছো
`-v`, `--verbose`	কী মুছছে দেখাও
`--ignore-fail-on-non-empty`	খালি না হওয়ার কারণে ব্যর্থ হলে error দেখাবে না

```
$ mkdir -p a/b/c
$ rmdir -pv a/b/c
rmdir: removing directory, 'a/b/c'
rmdir: removing directory, 'a/b'
rmdir: removing directory, 'a'
```

একটা ট্রি-র সব খালি ডিরেক্টরি খুঁজে মুছতে: `find . -type d -empty -delete` (অধ্যায় ১২)।

> 📝 **মনে রাখো:** hidden ফাইল (যেমন `.gitkeep`) থাকলেও ডিরেক্টরি "খালি" না। `ls -A` দিয়ে দেখো।

৯.১১ পাথ নিয়ে কাজ: `basename`, `dirname`, `realpath`

এই তিনটা কমান্ড ফাইল তৈরি বা মোছে না। এরা শুধু **path string** কেটে-ছেঁটে দেয়। স্ক্রিপ্টে প্রায়ই লাগে।

```
        /home/rahim/projects/report.final.pdf
        └──────── dirname ────────┘└─ basename ─┘
```

`basename`: শেষ অংশ (নাম)

```
$ basename /home/rahim/projects/report.pdf
report.pdf
$ basename /home/rahim/projects/report.pdf .pdf      # suffix কেটে দাও
report
$ basename -s .pdf a.pdf b.pdf                        # অনেকগুলো একসাথে (-s নিজেই -a বোঝায়)
a
b
$ basename /home/rahim/projects/
projects
```

অপশন	কাজ
`-a`, `--multiple`	একাধিক path নাও
`-s SUFFIX`, `--suffix=SUFFIX`	শেষের suffix কাটো (`-a` বোঝায়)
`-z`, `--zero`	newline-এর বদলে NUL দিয়ে আলাদা করো

`dirname`: ডিরেক্টরি অংশ

```
$ dirname /home/rahim/projects/report.pdf
/home/rahim/projects
$ dirname report.pdf
.
$ dirname /etc/
/
```

একটা গুরুত্বপূর্ণ কথা

`basename` আর `dirname` **শুধু লেখা নিয়ে কাজ করে**। ফাইলটা আসলে আছে কি না, সেটা দেখে না:

```
$ basename /this/does/not/exist.txt
exist.txt
```

শেলেই এর দ্রুত বিকল্প আছে (নতুন প্রসেস চালাতে হয় না): `${path##*/}` দেয় basename, আর `${path%/*}` দেয় dirname।

`realpath`: আসল, পূর্ণ (absolute) path

`realpath` relative path, `..`, `.` আর symlink সব সমাধান করে চূড়ান্ত absolute path দেয়।

```
$ pwd
/home/rahim/projects
$ realpath ../Documents/./notes.txt
/home/rahim/Documents/notes.txt
```

```
$ ln -s /etc/os-release myos
$ realpath myos
/usr/lib/os-release          # (Ubuntu/Fedora-তে /etc/os-release নিজেই একটা symlink)
```

অপশন	কাজ
`-e`, `--canonicalize-existing`	path-এর সব অংশ অবশ্যই থাকতে হবে, না থাকলে error
`-m`, `--canonicalize-missing`	কিছুই না থাকলেও চলবে
`-s`, `--strip`, `--no-symlinks`	symlink resolve কোরো না, শুধু `.` / `..` ঠিক করো
`--relative-to=DIR`	DIR থেকে relative path দেখাও
`-q`, `--quiet`	বেশিরভাগ error message চেপে যাও

```
$ realpath --relative-to=/home/rahim /home/rahim/projects/app/main.c
projects/app/main.c
```

`readlink -f FILE` প্রায় একই কাজ করে। স্ক্রিপ্ট নিজের ফোল্ডার খুঁজে পেতে এই ধরনটা বহুল ব্যবহৃত:

```
SCRIPT_DIR="$(dirname "$(realpath "$0")")"
```

৯.১২ আরও কয়েকটা দরকারি সহায়ক কমান্ড

কমান্ড	কাজ	উদাহরণ
`install`	কপি + পারমিশন/মালিক একসাথে সেট (ইনস্টল স্ক্রিপ্টে)	`sudo install -m 755 app /usr/local/bin/`
`install -d`	ডিরেক্টরি বানাও (mode সহ)	`install -d -m 700 ~/.secret`
`mktemp`	নিরাপদ, ইউনিক temp ফাইল বানাও	`tmp=$(mktemp)`
`mktemp -d`	ইউনিক temp ডিরেক্টরি	`dir=$(mktemp -d)`
`truncate -s SIZE`	ফাইলের সাইজ বদলাও / খালি করো	`truncate -s 0 big.log`
`file`	ফাইলের ধরন চেনো	`file photo.jpg` (অধ্যায় ১০)
`ln`	লিংক বানাও	অধ্যায় ১১

> 💡 **টিপস:** চালু থাকা সার্ভিসের বড় লগ ফাইল খালি করতে `rm` না করে `truncate -s 0 app.log` (বা `: > app.log`) করো। এতে inode একই থাকে, প্রসেস লিখতে থাকে, আর জায়গা সাথে সাথে খালি হয়।

৯.১৩ কীবোর্ড শর্টকাট

টার্মিনালে (bash)

শর্টকাট	কাজ
`Tab`	ফাইল/ডিরেক্টরির নাম সম্পূর্ণ করো (space escape সহ)
`Tab Tab`	একাধিক মিল থাকলে সব দেখাও
`Alt+.` বা `Esc` তারপর `.`	আগের কমান্ডের শেষ argument বসাও (যেমন `mkdir newdir` এরপর `cd` + `Alt+.`)
`Ctrl+C`	চলমান কমান্ড থামাও (ভুল `cp -r` বা `rm -r` আটকাতে)
`Ctrl+U`	কার্সরের আগের পুরো লাইন মুছে দাও (বিপজ্জনক কমান্ড বাতিল করতে)
`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষে যাও
`Ctrl+W`	কার্সরের আগের একটা শব্দ মোছো
`Ctrl+R`	পুরনো কমান্ড খোঁজো
`!$`	আগের কমান্ডের শেষ argument (`mkdir proj && cd !$`)
`Ctrl+X` তারপর `*`	কার্সরের নিচের glob শব্দটাকে লাইনেই আসল ফাইলের নামে expand করো (চালানোর আগে যাচাই)
`Ctrl+X` তারপর `g`	glob কোন কোন ফাইল ধরবে তার তালিকা দেখাও, লাইন না বদলে

এসব শর্টকাটের পূর্ণ তালিকা আছে অধ্যায় ৬-এ।

> 💡 **টিপস:** `rm *.log` লিখে Enter চাপার আগে কার্সর `*.log`-এর উপর রেখে `Ctrl+X` তারপর `g` চাপো। দেখবে আসলে কোন কোন ফাইল মুছতে যাচ্ছ। `Ctrl+X` তারপর `*` চাপলে লাইনেই নামগুলো বসে যাবে।

GUI ফাইল ম্যানেজারে (GNOME Files/Nautilus, KDE Dolphin প্রায় একই)

শর্টকাট	কাজ
`Ctrl+C` / `Ctrl+X` / `Ctrl+V`	কপি / কাট / পেস্ট
`F2`	নাম বদলাও
`Delete`	Trash-এ পাঠাও
`Shift+Delete`	চিরতরে মুছে ফেলো (Trash ছাড়া)
`Ctrl+Shift+N`	নতুন ফোল্ডার
`Ctrl+Z`	শেষ কাজ undo করো
`Ctrl+H`	hidden ফাইল দেখাও/লুকাও

শর্টকাট	কাজ
`Ctrl+L`	path টাইপ করার বার
`Ctrl+A`	সব সিলেক্ট করো
`Alt+Enter` (Dolphin) / `Ctrl+I` (Nautilus)	Properties

ডেস্কটপ পরিবেশ নিয়ে বিস্তারিত আছে অধ্যায় ৪-এ।

হাতে-কলমে অনুশীলন

নিজের home ডিরেক্টরিতে একটা নিরাপদ "খেলার মাঠ" বানাও। এই ল্যাবে `sudo` লাগবে না।

ধাপ ১: খেলার মাঠ তৈরি

```
$ mkdir -p ~/lab09 && cd ~/lab09
$ mkdir -pv shop/{products/{phones,laptops},orders,archive}
mkdir: created directory 'shop'
mkdir: created directory 'shop/products'
mkdir: created directory 'shop/products/phones'
mkdir: created directory 'shop/products/laptops'
mkdir: created directory 'shop/orders'
mkdir: created directory 'shop/archive'
```

ধাপ ২: ফাইল তৈরি ও timestamp

```
$ touch shop/orders/order{1..5}.txt
$ echo "Samsung A55" > shop/products/phones/list.txt
$ touch -d "2024-06-01 09:00" shop/orders/order1.txt
$ ls -l shop/orders/
$ stat -c '%n  mtime=%y' shop/orders/order1.txt
```

খেয়াল করো `order1.txt`-এর তারিখ ২০২৪ দেখাচ্ছে।

ধাপ ৩: ctime বদলাতে দেখো

```
$ stat -c 'm=%y | c=%z' shop/products/phones/list.txt
$ chmod 600 shop/products/phones/list.txt
$ stat -c 'm=%y | c=%z' shop/products/phones/list.txt
```

mtime একই থাকবে, ctime বদলে যাবে। কেন? (উত্তর: পারমিশন inode-এর metadata, কনটেন্ট না।)

ধাপ ৪: কপি আর inode

```
$ cd shop/products/phones
$ cp list.txt list-copy.txt
```

```
$ cp -p list.txt list-preserve.txt
$ ls -li
```

তিনটা ফাইলের inode নম্বর আলাদা, আর `list-preserve.txt`-এর পারমিশন ও সময় মূলটার মতো। লক্ষ করো।

ধাপ ৫: mv আর inode

```
$ ls -i list-copy.txt
$ mv -v list-copy.txt ../laptops/moved.txt
$ ls -i ../laptops/moved.txt
```

inode একই! কারণ একই ফাইলসিস্টেমে `rename()` হয়েছে।

ধাপ ৬: ওভাররাইট থেকে বাঁচা

```
$ echo "old" > a.txt; echo "new" > b.txt
$ cp -i b.txt a.txt          # 'n' দাও
$ cat a.txt                  # old
$ cp -b b.txt a.txt
$ ls                         # a.txt~ ব্যাকআপ দেখবে
$ cat a.txt~                 # old
```

ধাপ ৭: অদ্ভুত নামের ফাইল

```
$ cd ~/lab09
$ touch 'my file.txt' -- '-weird.txt' 'cost$5.txt'
$ ls
$ rm 'my file.txt'
$ rm -- -weird.txt
$ rm cost\$5.txt
$ ls
```

(`touch -- '-weird.txt'`-এর `--` ছাড়া `touch` ভাবত `-w` একটা অপশন।)

ধাপ ৮: ডিরেক্টরি কপির ফাঁদ

```
$ cp -r shop shop-backup        # shop-backup ছিল না → কনটেন্ট সরাসরি ভেতরে
$ cp -r shop shop-backup        # এবার আছে → shop-backup/shop তৈরি হবে
$ ls shop-backup
```

দুবার চালিয়ে ফলের পার্থক্য দেখো।

ধাপ ৯: path টুল

```
$ f=~/lab09/shop/products/phones/list.txt
$ basename "$f"; basename "$f" .txt; dirname "$f"
$ cd shop/products
```

```
$ realpath ../../shop/./orders
$ realpath --relative-to="$HOME" "$f"
```

ধাপ ১০: নিরাপদ মোছা

```
$ sudo apt install trash-cli      # বা: sudo dnf install trash-cli (এই ধাপে sudo লাগবে)
$ trash-put ~/lab09/shop-backup
$ trash-list
$ trash-restore                   # ফিরিয়ে আনো
```

ধাপ ১১: পরিষ্কার করা

```
$ cd ~
$ rmdir lab09                     # ব্যর্থ হবে, কারণ খালি না
$ ls lab09                        # কী মুছবে আগে দেখে নাও
$ rm -rI lab09
rm: remove 1 argument recursively? y
```

সারসংক্ষেপ

কাজ	কমান্ড
খালি ফাইল / timestamp আপডেট	`touch file`
নির্দিষ্ট সময় বসানো	`touch -d "2024-01-15 14:30" file`
অন্য ফাইলের সময় কপি	`touch -r ref file`
inode-এর সব তথ্য	`stat file`
নিজের ফরম্যাটে তথ্য	`stat -c '%n %s %i %a' file`
ডিরেক্টরি (মাঝেরগুলো সহ)	`mkdir -p a/b/c`
পারমিশন সহ ডিরেক্টরি	`mkdir -m 700 dir`
ফাইল কপি	`cp src dst`
ডিরেক্টরি কপি	`cp -r src/ dst/`
হুবহু কপি (ব্যাকআপ)	`cp -a src dst`
জিজ্ঞেস করে / দেখিয়ে কপি	`cp -iv src dst`
শুধু নতুন হলে কপি	`cp -u src dst`
নাম বদল / সরানো	`mv old new`
ওভাররাইট না করে সরানো	`mv -n src dst`
ফাইল মোছা	`rm file`
ডিরেক্টরি সহ মোছা	`rm -r dir`
একবার নিশ্চিত হয়ে মোছা	`rm -rI dir`

কাজ	কমান্ড
`-` দিয়ে শুরু হওয়া নাম	`rm -- -file` বা `rm ./-file`
Trash-এ পাঠানো	`trash-put file`
Trash থেকে ফেরত	`trash-restore`
খালি ডিরেক্টরি মোছা	`rmdir dir` / `rmdir -p a/b/c`
নাম অংশ	`basename path [suffix]`
ডিরেক্টরি অংশ	`dirname path`
পূর্ণ absolute path	`realpath path`
শেষ argument পুনরায় ব্যবহার	`Alt+.` / `!$`
glob কোন ফাইল ধরবে দেখা	`Ctrl+X g` / `Ctrl+X *`

নিজেকে যাচাই করো

1. atime, mtime আর ctime-এর পার্থক্য কী? `chmod` চালালে কোনটা বদলায়?
2. `touch` দিয়ে কি ctime ইচ্ছামতো সেট করা যায়? কেন?
3. `mkdir -p` সাধারণ `mkdir`-এর চেয়ে দুটো কোন দিক থেকে আলাদা?
4. `backup` ডিরেক্টরি আগে থেকে থাকলে `cp -r projects backup` কী তৈরি করে? না থাকলে কী করে?
5. `cp -p` আর `cp -a`-এর পার্থক্য কী?
6. একই ফাইলসিস্টেমে ১০ GB ফাইল `mv` করতে মুহূর্ত লাগে, কিন্তু USB ড্রাইভে সরাতে অনেক সময় লাগে। কেন? দুই ক্ষেত্রে inode নম্বরের কী হয়?
7. বড় লগ ফাইল `rm` করার পরেও `df` তে জায়গা খালি হলো না। সম্ভাব্য কারণ কী, আর কীভাবে খুঁজবে?
8. `sudo rm -rf /` আধুনিক সিস্টেমে আটকে যায়, কিন্তু `sudo rm -rf /*` আটকায় না। কেন?
9. `-notes.txt` নামের ফাইল মোছার দুটো উপায় লেখো।
10. স্ক্রিপ্টে `rm -rf "$DIR"/*` কেন বিপজ্জনক, আর এটাকে নিরাপদ করবে কীভাবে?

▼ উত্তর দেখো

1. **atime**: কনটেন্ট শেষ কবে পড়া হয়েছে। **mtime**: কনটেন্ট শেষ কবে বদলেছে। **ctime**: inode (metadata বা কনটেন্ট) শেষ কবে বদলেছে। `chmod` শুধু **ctime** বদলায়।
2. না। inode-এ যেকোনো পরিবর্তন হলে (`touch` নিজে atime/mtime বদলালেও) কার্নেল ctime বর্তমান সময়ে সেট করে দেয়। সাধারণ টুল দিয়ে ctime পেছনে নেওয়া যায় না।
3. (ক) পথের মাঝে না থাকা প্যারেন্ট ডিরেক্টরিগুলো বানিয়ে দেয়; (খ) ডিরেক্টরি আগে থেকে থাকলে error দেয় না।
4. থাকলে তৈরি হয় `backup/projects/...`। না থাকলে `backup` নিজেই তৈরি হয় এবং `projects`-এর কনটেন্ট তার ভেতরে যায়। শুধু কনটেন্ট চাইলে লেখো `cp -r projects/. backup/`।

5. `-p` ধরে রাখে mode, ownership আর timestamps। `-a` = `-dR --preserve=all`: recursive, symlink-কে symlink রাখে, হার্ড লিংক, xattr, ACL, SELinux context সহ সব ধরে রাখে।
6. একই ফাইলসিস্টেমে `mv` শুধু `rename()` ডাকে, ডিরেক্টরি এন্ট্রি বদলায়, ডেটা নড়ে না। তাই inode **একই থাকে**। ভিন্ন ফাইলসিস্টেমে `rename()` ব্যর্থ হয় (EXDEV), তাই `mv` পুরো ডেটা কপি করে তারপর উৎস মোছে। তাই **নতুন inode** তৈরি হয় আর সময় লাগে।
7. কোনো চালু প্রসেস ফাইলটা এখনো খোলা রেখেছে, তাই link count 0 হলেও ডেটা মুক্ত হয়নি। `lsof +L1` দিয়ে খুঁজে প্রসেস/সার্ভিস রিস্টার্ট করো। ভবিষ্যতে এমন ক্ষেত্রে `truncate -s 0 file` ব্যবহার করো।
8. `--preserve-root` শুধু আক্ষরিক `/` argument আটকায়। `/*` শেল expand করে `/bin /boot /etc ...` বানিয়ে দেয়, তাই `rm` আর `/` দেখতে পায় না।
9. `rm -- -notes.txt` অথবা `rm ./-notes.txt`।
10. `DIR` খালি বা unset হলে কমান্ডটা হয়ে যায় `rm -rf /*`। নিরাপদ রূপ: `rm -rf "${DIR:?}"/*` (খালি হলে স্ক্রিপ্ট থেমে যায়), অথবা স্ক্রিপ্টে `set -u` দাও এবং আগে `[ -d "$DIR" ]` যাচাই করো।

অধ্যায় ১০: ফাইলের ভেতরের লেখা দেখা

এই অধ্যায়ে যা শিখবে

- `cat` দিয়ে ফাইল দেখা, লাইন নম্বর দেওয়া (`-n` , `-b`) আর লুকানো অক্ষর দেখা (`-A`)
- `tac` দিয়ে ফাইল উল্টো ক্রমে দেখা
- `less` — লিনাক্সের সবচেয়ে গুরুত্বপূর্ণ "pager", এর সব নেভিগেশন শর্টকাট (`space` , `b` , `g` , `G` , `/` , `?` , `n` , `N` , `q` , `F` এবং আরও অনেক)
- পুরোনো pager `more` আর এটা কেন এখন কম ব্যবহার হয়
- `head` আর `tail` দিয়ে ফাইলের শুরু ও শেষ দেখা (`-n` , `-c` , `-f` , `-F`)
- `tail -f` / `tail -F` দিয়ে লাইভ লগ দেখা, আর log rotation হলে কী হয়
- `nl` দিয়ে লাইন নম্বর দেওয়া, `wc` দিয়ে লাইন/শব্দ/বাইট গোনা
- binary ফাইল আর text ফাইলের আসল পার্থক্য — ভেতরে আসলে কী থাকে
- `od` , `hexdump` , `xxd` দিয়ে ফাইলের বাইট-বাই-বাইট চেহারা দেখা
- `strings` দিয়ে binary ফাইল থেকে পড়ার মতো লেখা বের করা
- character encoding (ASCII, UTF-8), BOM, আর line ending (LF বনাম CRLF) — `file` , `iconv` , `dos2unix`
- `column` দিয়ে এলোমেলো ডেটা সুন্দর টেবিলে সাজানো

১০.১ শুরুর আগে: "ফাইল দেখা" মানে আসলে কী?

আগের অধ্যায়ে (অধ্যায় ৯) তুমি ফাইল বানানো, কপি, সরানো, মোছা শিখেছ। কিন্তু ফাইলের **ভেতরে** কী লেখা আছে, সেটা দেখবে কীভাবে? Windows-এ তুমি ডাবল-ক্লিক করে Notepad-এ খোলো। লিনাক্সে — বিশেষ করে সার্ভারে, যেখানে কোনো GUI নেই — তোমাকে টার্মিনাল থেকেই ফাইল পড়তে হবে।

একটা মজার কথা দিয়ে শুরু করি: **লিনাক্সের কাছে প্রতিটা ফাইল আসলে শুধু কিছু বাইটের (byte) সারি।** একটা বাইট মানে ০ থেকে ২৫৫ পর্যন্ত একটা সংখ্যা। "Hello" লেখাটা ডিস্কে আসলে এই পাঁচটা সংখ্যা হিসেবে থাকে:

```
H    e    l    l    o
72   101  108  108  111     (দশমিকে)
48   65   6c   6c   6f      (hex/ষোড়শিকে)
```

kernel (লিনাক্সের মূল অংশ, অধ্যায় ২ দেখো) জানে না ফাইলটা "লেখা" নাকি "ছবি"। সে শুধু বাইট দেয়। সেই বাইটগুলোকে **কীভাবে দেখাবে** — অক্ষর হিসেবে, সংখ্যা হিসেবে, নাকি hex হিসেবে — সেটা ঠিক করে যে প্রোগ্রাম দিয়ে তুমি ফাইলটা খুলছ।

> 📝 **মনে রাখো:** একই ফাইল `cat` দিয়ে দেখলে অক্ষর দেখবে, `xxd` দিয়ে দেখলে hex সংখ্যা দেখবে। ফাইল বদলায়নি — শুধু "চশমা" বদলেছে।

ভেতরে কী ঘটে: `cat file.txt` চালালে

```
তুমি                shell (bash)          kernel              ডিস্ক
 |  cat file.txt       |                     |                   |
 |-------------------->| fork + exec("cat")  |                   |
 |                     |-------------------->|                   |
 |                     |         cat: open("file.txt")  ---------->| inode খুঁজে
 |                     |         cat: read(fd, buf, 128KB) <-------| বাইট পড়ে
 |                     |         cat: write(1, buf)  (stdout-এ)    |
 |   স্ক্রিনে লেখা   <---------- terminal ওই বাইটগুলোকে            |
 |                     |         অক্ষর হিসেবে আঁকে (UTF-8 ধরে)     |
```

খেয়াল করো: `cat` নিজে কিছু "বোঝে" না। সে শুধু বাইট পড়ে আর stdout-এ (standard output, মানে সাধারণত তোমার টার্মিনাল) লিখে দেয়। বাইটগুলোকে অক্ষরে রূপ দেয় তোমার **terminal emulator**। এজন্যই binary ফাইল `cat` করলে টার্মিনালে আবোল-তাবোল চিহ্ন আসে — terminal ওই এলোমেলো বাইটগুলোকেও অক্ষর হিসেবে আঁকার চেষ্টা করে। (stdin/stdout নিয়ে বিস্তারিত আছে অধ্যায় ১৯-এ।)

অনুশীলনের জন্য একটা ছোট ফাইল বানিয়ে নাও, এই অধ্যায়ের বেশিরভাগ উদাহরণে এটা ব্যবহার করব:

```
$ mkdir -p ~/ch10 && cd ~/ch10
$ printf 'আম\nজাম\n\nকাঁঠাল\nলিচু\tপেয়ারা\n' > fruits.txt
$ seq 1 100 > numbers.txt
```

`seq 1 100` এক থেকে একশো পর্যন্ত প্রতিটা সংখ্যা আলাদা লাইনে লেখে — ১০০ লাইনের একটা ফাইল পেয়ে গেলে।

১০.২ `cat` — ফাইল জোড়া লাগানো আর দেখানো

কী করে

`cat` নামটা এসেছে **concatenate** (জোড়া লাগানো) থেকে। এর আসল কাজ হলো এক বা একাধিক ফাইল পরপর পড়ে stdout-এ লিখে দেওয়া। একটা ফাইল দিলে সেটাই দেখায় — এজন্য সবাই একে "ফাইল দেখার" কমান্ড হিসেবে চেনে।

উপমা: `cat` যেন একটা ফটোকপি মেশিন যেটা কাগজ বের না করে সরাসরি দেয়ালে প্রজেক্ট করে। যত পাতাই দাও, সব একটানা দেখিয়ে দেয় — থামে না।

Syntax

```
cat [OPTION]... [FILE]...
```

গুরুত্বপূর্ণ অপশন

অপশন	পুরো নাম	কাজ
`-n`	`--number`	সব লাইনে নম্বর দেয় (ফাঁকা লাইনসহ)
`-b`	`--number-nonblank`	শুধু ফাঁকা-নয় এমন লাইনে নম্বর দেয় (`-n` -কে override করে)
`-s`	`--squeeze-blank`	পরপর একাধিক ফাঁকা লাইনকে একটা ফাঁকা লাইনে নামিয়ে আনে
`-E`	`--show-ends`	প্রতিটা লাইনের শেষে `$` দেখায়
`-T`	`--show-tabs`	Tab অক্ষরকে `^I` হিসেবে দেখায়
`-v`	`--show-nonprinting`	অদৃশ্য control অক্ষর `^` বা `M-` চিহ্নে দেখায় (Tab আর newline বাদে)
`-A`	`--show-all`	`-vET` একসাথে — সব লুকানো জিনিস দেখায়
`-e`		`-vE` এর সমান
`-t`		`-vT` এর সমান

উদাহরণ

সাধারণভাবে দেখা:

```
$ cat fruits.txt
আম
জাম

কাঁঠাল
লিচু	পেয়ারা
```

লাইন নম্বরসহ:

```
$ cat -n fruits.txt
     1	আম
     2	জাম
     3
     4	কাঁঠাল
     5	লিচু	পেয়ারা
```

শুধু ভরা লাইনে নম্বর:

```
$ cat -b fruits.txt
     1	আম
     2	জাম
```

```
     3  কাঁঠাল
     4  লিচু        পেয়ারা
```

`cat -A` — লুকানো অক্ষরের এক্স-রে

এটা এই অধ্যায়ের সবচেয়ে কাজের ট্রিকগুলোর একটা। অনেক সময় একটা ফাইল দেখতে ঠিক আছে কিন্তু স্ক্রিপ্ট চলছে না — কারণ ভেতরে লুকানো Tab, শেষে অতিরিক্ত space, বা Windows-এর `\r` আছে। `-A` সব ফাঁস করে দেয়:

```
$ cat -A fruits.txt
আম$
জাম$
$
কাঁঠাল$
লিচু^Iপেয়ারা$
```

- `$` = এখানে লাইন শেষ (newline, `\n`)
- `^I` = Tab অক্ষর
- `^M` = Carriage Return (`\r`) — Windows-এর ফাইলে প্রতিটা লাইনের শেষে `^M$` দেখবে (১০.১৩ অংশে বিস্তারিত)

> 💡 **টিপস:** লাইনের শেষে অদৃশ্য space আছে কিনা বুঝতে `cat -A` চালাও। `hello $` দেখলে বুঝবে `$`-এর আগে তিনটা space লুকিয়ে আছে।

> 📝 **মনে রাখো:** বাংলা অক্ষরে `cat -A` চালালে GNU coreutils-এর নতুন সংস্করণগুলো (Ubuntu 24.04 / Debian 12 / Fedora) সাধারণত বাংলা ঠিকমতো দেখায়, কারণ তারা UTF-8 locale বোঝে। খুব পুরোনো সংস্করণে বা `LC_ALL=C` দিয়ে চালালে non-ASCII বাইটগুলো `M-` দিয়ে শুরু হওয়া চিহ্নে (যেমন `M-` M-&M-^F`) দেখাতে পারে — ভয় পেও না, ওটা শুধু UTF-8 বাইটের প্রতিনিধিত্ব।

একাধিক ফাইল জোড়া লাগানো (আসল কাজ)

```
$ printf 'part1\n' > a.txt; printf 'part2\n' > b.txt
$ cat a.txt b.txt
part1
part2
$ cat a.txt b.txt > joined.txt     # জোড়া লাগিয়ে নতুন ফাইলে
$ cat a.txt >> b.txt               # a.txt-কে b.txt-এর শেষে যোগ
```

`-` মানে stdin (কীবোর্ড বা পাইপ থেকে আসা ইনপুট):

```
$ echo "মাঝখানে" | cat a.txt - b.txt
```

`cat` দিয়ে ছোট ফাইল লেখা (heredoc)

```
$ cat > note.txt << 'EOF'
প্রথম লাইন
দ্বিতীয় লাইন
EOF
```

বা শুধু `cat > note.txt` লিখে টাইপ করো, শেষে নতুন লাইনে `Ctrl+D` চাপো (Ctrl+D মানে "ইনপুট শেষ", End-Of-File)।

> ⚠️ **সতর্কতা:** `cat > important.txt` চালালে `important.txt`-এর পুরোনো সব লেখা **সাথে সাথে মুছে যায়** — তুমি কিছু টাইপ করার আগেই। `>` আর `>>` গুলিয়ে ফেলো না।

কখন `cat` ব্যবহার করবে না

- **বড় ফাইলে** — ১০ লাখ লাইনের লগ `cat` করলে সব স্ক্রিন দিয়ে দৌড়ে চলে যাবে, তুমি শুধু শেষ ৩০ লাইন দেখবে। এর জন্য `less`।
- **binary ফাইলে** — টার্মিনাল এলোমেলো হয়ে যেতে পারে (১০.১১ দেখো)।
- **"Useless use of cat"** — `cat file | grep x` না লিখে `grep x file` লেখো। ফল একই, কিন্তু একটা অতিরিক্ত process আর pipe বাঁচে।

১০.৩ `tac` — উল্টো `cat`

`tac` হলো `cat` উল্টো করে লেখা — আর কাজও উল্টো। শেষ লাইন আগে, প্রথম লাইন পরে দেখায়।

```
$ seq 1 5 | tac
5
4
3
2
1
```

কোথায় কাজে লাগে: লগ ফাইলে নতুন এন্ট্রি নিচে যোগ হয়। সর্বশেষ ঘটনা আগে দেখতে চাইলে:

```
$ tac /var/log/dpkg.log | less        # Debian/Ubuntu: সর্বশেষ প্যাকেজ ইনস্টল আগে
```

অপশন	কাজ
`-s STRING`	newline-এর বদলে অন্য কিছুকে "লাইন বিভাজক" ধরে
`-b`	বিভাজককে রেকর্ডের শেষে না, শুরুতে ধরে
`-r`	বিভাজককে regular expression হিসেবে ধরে

```
$ printf 'a,b,c,' | tac -s ','
c,b,a,
```

> 📝 **মনে রাখো:** `tac` লাইনের ভেতরের অক্ষর উল্টায় না। অক্ষর উল্টাতে চাইলে `rev`: `echo hello | rev` → `olleh`।

১০.৪ Pager কী, আর কেন দরকার

Pager হলো এমন প্রোগ্রাম যেটা লম্বা লেখাকে এক স্ক্রিন করে দেখায় আর তোমাকে সামনে-পেছনে যেতে দেয়। **উপমা:** `cat` হলো একটা বই ছুড়ে মারা — সব পাতা একসাথে উড়ে যায়। Pager হলো বইটা হাতে নিয়ে পাতা উল্টানো।

লিনাক্সে তিনটা pager বিখ্যাত:

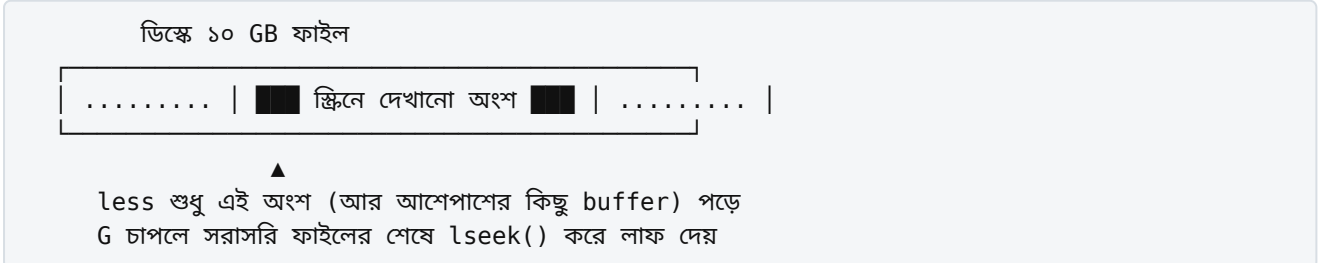
```
more (১৯৭৮, BSD)  ──►  less (১৯৮৩–৮৫, Mark Nudelman)  ──►  most (কম প্রচলিত)
শুধু সামনে যায়          সামনে-পেছনে, সার্চ, follow          রং, একাধিক উইন্ডো
```

নামের মজা: `more`-এর উন্নত সংস্করণের নাম রাখা হয়েছে `less` — কারণ "less is more"। আজকের দিনে `man` পেজ, `git log`, `journalctl`, `systemctl status` — সবই ডিফল্টভাবে `less` দিয়ে লেখা দেখায়। তাই `less`-এর শর্টকাট শিখলে অনেক টুলে একসাথে দক্ষ হয়ে যাবে।

১০.৫ `less` — সবচেয়ে গুরুত্বপূর্ণ ফাইল-দেখার টুল

কেন `less` এত দ্রুত (under the hood)

`less` পুরো ফাইল মেমোরিতে লোড করে না। সে শুধু যতটুকু স্ক্রিনে দেখাতে হবে তার আশেপাশের অংশ পড়ে (buffer করে)। এজন্য ১০ GB লগ ফাইলও `less` দিয়ে প্রায় সাথে সাথে খুলে যায়, যেখানে একটা সাধারণ GUI editor হয়তো আটকে যাবে।



> 💡 **টিপস:** `G` (শেষে যাও) চাপলে খুব বড় ফাইলে `less` নিচে "Calculating line numbers... (interrupt to abort)" লিখতে পারে — কারণ লাইন নম্বর বের করতে পুরো ফাইল গুনতে হয়। `Ctrl+C` চাপলে গোনা

> বাদ দিয়ে সাথে সাথে চলে যাবে। `-n` অপশন দিয়ে খুললে লাইন নম্বর গোনাই বন্ধ থাকে।

Syntax

```
less [OPTIONS] [FILE]...
command | less
```

```
$ less /etc/services
$ ls -la /usr/bin | less
$ less +G /var/log/syslog        # খুলেই শেষে চলে যাও
$ less +/ssh /etc/services       # খুলেই "ssh" সার্চ করো
$ less +F app.log                # খুলেই follow মোডে (tail -f এর মতো)
```

`+` দিয়ে শুরু হওয়া যেকোনো জিনিস `less` খোলার পর প্রথম কমান্ড হিসেবে চালায়।

`less` এর ভেতরের নেভিগেশন শর্টকাট — সম্পূর্ণ তালিকা

`less` খোলার পর নিচের কী-গুলো চাপো (টার্মিনালে আর কিছু টাইপ করতে হবে না)।

চলাফেরা (Movement):

কী	কাজ
`Space` বা `f` বা `Ctrl+F` বা `Page Down`	এক স্ক্রিন সামনে
`b` বা `Ctrl+B` বা `Page Up`	এক স্ক্রিন পেছনে
`Enter` বা `j` বা `e` বা `Ctrl+N` বা `↓`	এক লাইন সামনে
`k` বা `y` বা `Ctrl+P` বা `↑`	এক লাইন পেছনে
`d` বা `Ctrl+D`	আধা স্ক্রিন সামনে
`u` বা `Ctrl+U`	আধা স্ক্রিন পেছনে
`g` বা `<` বা `Home`	ফাইলের একদম শুরুতে
`G` বা `>` বা `End`	ফাইলের একদম শেষে
`50g` (সংখ্যা + g)	৫০ নম্বর লাইনে যাও
`50%` বা `50p`	ফাইলের ৫০% জায়গায় যাও
`→` / `←`	ডানে/বামে আধা স্ক্রিন সরাও (লম্বা লাইন কাটা থাকলে, `-S` দেখো)
`r` বা `Ctrl+L`	স্ক্রিন আবার আঁকো (repaint)

সার্চ (Searching):

কী	কাজ
`/pattern` তারপর `Enter`	সামনের দিকে `pattern` খোঁজো

কী	কাজ
`?pattern` তারপর `Enter`	পেছনের দিকে খোঁজো
`n`	একই দিকে পরের মিল
`N`	উল্টো দিকে পরের মিল
`&pattern`	শুধু যেসব লাইনে `pattern` আছে সেগুলো দেখাও (filter); `&` তারপর ফাঁকা `Enter` দিলে filter বাতিল
`/!pattern`	যেসব লাইনে `pattern` **নেই** সেগুলো খোঁজো
`Esc` তারপর `u` (`Esc u`)	সার্চের highlight বন্ধ/চালু

সার্চের pattern হলো regular expression (অধ্যায় ২০)। ডিফল্টে সার্চ case-sensitive; `-i` অপশন দিয়ে খুললে (বা `less`-এর ভেতরে `-i` টাইপ করে `Enter` চাপলে) ছোট-বড় হাতের অক্ষর এক ধরা হয় — যদি pattern-এ বড় হাতের অক্ষর না থাকে।

follow ও বাকি কাজ:

কী	কাজ
`F`	follow মোড — ফাইলের শেষে গিয়ে নতুন লাইন আসার অপেক্ষা করো (`tail -f` এর মতো)
`Ctrl+C`	follow মোড থেকে বেরিয়ে আবার স্বাভাবিক নেভিগেশনে
`q` বা `Q` বা `:q` বা `ZZ`	`less` থেকে বের হও
`h` বা `H`	সাহায্য (help) পেজ — সব শর্টকাট
`=` বা `Ctrl+G` বা `:f`	ফাইলের নাম, বর্তমান লাইন, মোট বাইট, শতাংশ দেখাও
`v`	বর্তমান ফাইলটা editor-এ (`$VISUAL` বা `$EDITOR`, না থাকলে `vi`) খোলো
`!command`	`less` না ছেড়েই একটা shell কমান্ড চালাও
`ma`	বর্তমান জায়গায় `a` নামে একটা mark (বুকমার্ক) রাখো (a–z যেকোনো অক্ষর)
`'a`	`a` mark-এ ফিরে যাও
`''`	আগের জায়গায় ফিরে যাও (শেষ বড় লাফের আগে যেখানে ছিলে)
`-N` তারপর `Enter` (ভেতরে টাইপ)	লাইন নম্বর চালু/বন্ধ (যেকোনো অপশন এভাবে ভেতর থেকে টগল করা যায়)
`-S` তারপর `Enter`	লম্বা লাইন কাটা/মোড়ানো টগল

একাধিক ফাইল: `less a.txt b.txt c.txt` দিয়ে খুললে:

কী	কাজ
`:n`	পরের ফাইল

কী	কাজ
`:p`	আগের ফাইল
`:e filename`	নতুন ফাইল খোলো
`:d`	বর্তমান ফাইল তালিকা থেকে সরাও

follow মোড (`F`) — `tail -f` এর চেয়ে ভালো কেন

```
$ less +F /var/log/nginx/access.log
Waiting for data... (interrupt to abort)
```

এখন নতুন লাইন এলে নিচে দেখা যাবে। একটা সন্দেহজনক লাইন দেখলে `Ctrl+C` চাপো — follow থামবে কিন্তু `less` বন্ধ হবে না। এখন উপরে স্ক্রল করো, `/error` দিয়ে সার্চ করো, তারপর আবার `F` চেপে follow-এ ফেরো। `tail -f`-এ এটা সম্ভব না — সেখানে `Ctrl+C` মানে পুরো কমান্ডই বন্ধ।

কমান্ড-লাইন অপশন

অপশন	কাজ
`-N`	প্রতিটা লাইনের বামে লাইন নম্বর
`-n`	লাইন নম্বর গোনা বন্ধ (বিশাল ফাইলে দ্রুত)
`-S`	লম্বা লাইন মোড়াবে না, কেটে দেবে (`→` / `←` দিয়ে পাশে সরো) — CSV/লগের জন্য দারুণ
`-i`	সার্চে case উপেক্ষা (pattern-এ বড় হাতের অক্ষর থাকলে case মানবে)
`-I`	সবসময় case উপেক্ষা
`-R`	ANSI color code-কে রং হিসেবে দেখাও (`ls --color=always | less -R`)
`-F`	পুরো লেখা এক স্ক্রিনে ধরলে `less` না খুলে সরাসরি দেখিয়ে বের হয়ে যাও
`-X`	বের হওয়ার পর স্ক্রিন পরিষ্কার করবে না (লেখা টার্মিনালে থেকে যাবে)
`-M`	নিচের prompt-এ বিস্তারিত তথ্য (লাইন নম্বর, শতাংশ)
`-J`	বাম পাশে একটা status কলাম, সার্চের মিল চিহ্নিত করে
`-x4`	Tab-কে ৪ ঘর হিসেবে দেখাও (ডিফল্ট ৮)
`+F`	follow মোডে শুরু
`--mouse`	(নতুন সংস্করণে, ৫৫১+) মাউসের চাকা দিয়ে স্ক্রল

`less` ডিফল্ট অপশন `LESS` environment variable থেকে পড়ে। অনেকে `~/.bashrc`-এ রাখে:

```
export LESS='-R -i -M -S'
```

(environment variable আর `.bashrc` নিয়ে অধ্যায় ৪৪।) `git` নিজে `LESS` না থাকলে `FRX` সেট করে — এজন্যই ছোট `git log` সরাসরি প্রিন্ট হয়ে বের হয়ে যায়।

`less` দিয়ে compressed আর বিশেষ ফাইল দেখা — `lesspipe`

Ubuntu/Debian-এ `lesspipe` নামে একটা সাহায্যকারী আছে যেটা `less`-কে `.gz`, `.tar`, `.deb`, `.zip` ইত্যাদির ভেতরটা দেখাতে শেখায়। Ubuntu-র ডিফল্ট `~/.bashrc`-এ সাধারণত এই লাইন থাকে:

```
[ -x /usr/bin/lesspipe ] && eval "$(SHELL=/bin/sh lesspipe)"
```

এটা চালু থাকলে `less archive.tar.gz` লিখলে ভেতরের ফাইলের তালিকা দেখাবে। Fedora/RHEL-এ একই কাজ করে `/usr/bin/lesspipe.sh`, যেটা `LESSOPEN` variable দিয়ে সাধারণত আগে থেকেই চালু থাকে। আলাদাভাবে compressed ফাইলের জন্য `zless`, `zcat`, `zgrep` তো আছেই (বিস্তারিত অধ্যায় ৩৪)।

```
$ zless /var/log/syslog.2.gz
$ zcat /usr/share/doc/bash/changelog.Debian.gz | head
```

> ⚠️ **সতর্কতা:** অপরিচিত বা ক্ষতিকর হতে পারে এমন ফাইল `less` দিয়ে খোলার সময় মনে রাখো `LESSOPEN` প্রিপ্রসেসর ফাইলটা অন্য প্রোগ্রাম দিয়ে পার্স করে। সন্দেহজনক ফাইল কাঁচা অবস্থায় দেখতে `less -L file` (LESSOPEN উপেক্ষা) ব্যবহার করো।

১০.৬ `more` — পুরোনো pager

`more` প্রায় সব ইউনিক্স-জাতীয় সিস্টেমে থাকে — এমনকি খুব ছোট সিস্টেমে বা rescue mode-এ যেখানে `less` নেই। লিনাক্সে `more` আসে `util-linux` প্যাকেজ থেকে।

```
$ more /etc/services
```

কী	কাজ
`Space`	পরের স্ক্রিন
`Enter`	পরের লাইন
`b` বা `Ctrl+B`	এক স্ক্রিন পেছনে (শুধু ফাইলে কাজ করে, পাইপে না)
`/pattern`	সামনে সার্চ
`n`	পরের মিল
`=`	বর্তমান লাইন নম্বর
`v`	editor-এ খোলো
`h` বা `?`	সাহায্য
`q`	বের হও

অপশন	কাজ
`-d`	নিচে "[Press space to continue, 'q' to quit.]" জাতীয় সাহায্য দেখাও
`-s`	পরপর ফাঁকা লাইন এক করো
`+NUM`	NUM নম্বর লাইন থেকে শুরু
`+/pattern`	pattern-এর জায়গা থেকে শুরু

`more` বনাম `less`:

বৈশিষ্ট্য	`more`	`less`
পেছনে যাওয়া	সীমিত (পাইপে না)	পুরোপুরি
পেছনে সার্চ (`?`)	না	হ্যাঁ
follow মোড	না	হ্যাঁ (`F`)
শেষে পৌঁছলে	নিজে থেকে বের হয়ে যায়	থেকে যায়, `q` লাগে
বড় ফাইল	ঠিক আছে	আরও দক্ষ

> 💡 **টিপস:** দৈনন্দিন কাজে সবসময় `less` ব্যবহার করো। `more` শুধু তখন জানা দরকার যখন কোনো minimal সিস্টেমে `less` নেই।

১০.৭ `head` — ফাইলের শুরুটা দেখা

উপমা: খবরের কাগজের শিরোনাম পড়া — পুরো পত্রিকা না পড়ে শুধু উপরের অংশ।

Syntax

```
head [OPTION]... [FILE]...
```

ডিফল্টে প্রথম **১০ লাইন** দেখায়।

অপশন	কাজ
`-n N` / `--lines=N`	প্রথম N লাইন
`-n -N`	শেষের N লাইন **বাদে** বাকি সব (GNU)
`-c N` / `--bytes=N`	প্রথম N বাইট (`K` , `M` suffix চলে: `-c 1K` = ১০২৪ বাইট)
`-c -N`	শেষের N বাইট বাদে সব
`-q`	একাধিক ফাইলে নামের header দেখাবে না
`-v`	একটা ফাইলেও নামের header দেখাও

অপশন	কাজ
`-z`	newline-এর বদলে NUL অক্ষরকে লাইন-বিভাজক ধরো

উদাহরণ

```
$ head -n 3 numbers.txt
1
2
3

$ head -3 numbers.txt          # পুরোনো কিন্তু এখনো চলা ছোট রূপ
1
2
3

$ head -n -97 numbers.txt      # শেষের ৯৭টা বাদ → প্রথম ৩টা
1
2
3

$ head -c 5 numbers.txt | od -c
0000000   1  \n   2  \n   3
0000005
```

একাধিক ফাইলে header আসে:

```
$ head -n 2 numbers.txt fruits.txt
==> numbers.txt <==
1
2

==> fruits.txt <==
আম
জাম
```

> ⚠️ **সতর্কতা:** `head -c` বাইট গোনে, অক্ষর না। বাংলা অক্ষর UTF-8-এ সাধারণত ৩ বাইট। `head -c 4 fruits.txt` দিলে "আ" (৩ বাইট) আর "ম"-এর প্রথম বাইট আসবে — একটা ভাঙা অক্ষর, স্ক্রিনে `�` দেখাতে পারে।

বাস্তব ব্যবহার:

```
$ head -n 1 data.csv                # CSV-র কলামের নাম দেখো
$ ls -t ~/Downloads | head -n 5     # সর্বশেষ ৫টা ডাউনলোড
$ head -c 16 /dev/urandom | xxd     # ১৬ বাইট random ডেটা
```

১০.৮ `tail` — ফাইলের শেষটা দেখা

ডিফল্টে শেষের **১০ লাইন**।

অপশন	কাজ
`-n N`	শেষের N লাইন
`-n +N`	N নম্বর লাইন **থেকে শুরু করে** শেষ পর্যন্ত
`-c N`	শেষের N বাইট
`-c +N`	N নম্বর বাইট থেকে শেষ পর্যন্ত
`-f` / `--follow`	শেষ দেখিয়ে থামবে না, নতুন লাইন আসতে থাকলে দেখাতে থাকবে (file descriptor ধরে রাখে)
`-F`	`--follow=name --retry` এর সমান — ফাইল মুছে/বদলে গেলেও নাম ধরে আবার খোলে
`--retry`	ফাইল না পেলে বা পড়া না গেলে বারবার চেষ্টা করো
`--pid=PID`	`-f` এর সাথে: ওই process মরে গেলে tail-ও থামবে
`-s N`	`-f` এর সাথে: প্রতি N সেকেন্ড পরপর চেক (inotify না থাকলে কাজে লাগে)
`-q` / `-v`	header দেখাবে না / সবসময় দেখাবে

উদাহরণ

```
$ tail -n 3 numbers.txt
98
99
100

$ tail -n +98 numbers.txt       # ৯৮ নম্বর লাইন থেকে শেষ পর্যন্ত
98
99
100

$ tail -n +2 data.csv           # header লাইন বাদ দিয়ে বাকি সব — খুব কাজের!
```

`head` আর `tail` মিলিয়ে মাঝখানের লাইন

২০ থেকে ২৫ নম্বর লাইন দেখতে:

```
$ head -n 25 numbers.txt | tail -n 6
20
21
22
23
24
25
```

বা `sed -n '20,25p' numbers.txt` (অধ্যায় ২১)।

```
numbers.txt (১০০ লাইন)
┌──────┐
│  1   │ ┐
│ ...  │ │ head -n 25  → প্রথম ২৫ লাইন
│ 20   │ │                    │
│ ...  │ │                    ▼ পাইপ
│ 25   │ ┘           tail -n 6 → ওই ২৫-এর শেষ ৬টা = ২০–২৫
│ ...  │
│ 100  │
└──────┘
```

১০.৯ `tail -f` আর `tail -F` দিয়ে লাইভ লগ দেখা

সার্ভার অ্যাডমিনের সবচেয়ে প্রিয় কমান্ডগুলোর একটা। ওয়েব সার্ভারে কেউ পেজ খুললে কী লগ হচ্ছে, অ্যাপে error আসছে কিনা — রিয়েল-টাইমে দেখা যায়।

```
$ tail -f /var/log/nginx/access.log
192.168.1.10 - - [12/Nov/2026:10:15:01 +0600] "GET / HTTP/1.1" 200 612 "-" "curl/8.5.0"
192.168.1.10 - - [12/Nov/2026:10:15:04 +0600] "GET /about HTTP/1.1" 404 153 "-"
"curl/8.5.0"
▉   ← কার্সর এখানে অপেক্ষা করছে, নতুন লাইন এলে দেখাবে
```

থামাতে `Ctrl+C`।

ভেতরে কীভাবে কাজ করে

GNU `tail -f` লিনাক্সে **inotify** ব্যবহার করে — এটা kernel-এর একটা সুবিধা যেটা কোনো ফাইল বদলালে প্রোগ্রামকে খবর দেয়। ফলে `tail` বারবার ফাইল চেক করে CPU নষ্ট করে না; kernel "ফাইল বদলেছে" বললে তবেই নতুন অংশ পড়ে। (inotify ব্যবহার করা না গেলে, যেমন কিছু নেটওয়ার্ক ফাইলসিস্টেমে, প্রতি ১ সেকেন্ডে চেক করে — `-s` দিয়ে বদলানো যায়।)

`-f` বনাম `-F`: log rotation সমস্যা

লগ ফাইল অনন্তকাল বড় হতে পারে না। `logrotate` (অধ্যায় ২৮) নিয়মিত পুরোনো লগকে সরিয়ে নতুন ফাঁকা ফাইল বানায়:

```
 রাত ১২টার আগে:                          রাত ১২টার পরে (rotation):
 app.log  (inode 5001) ◄── tail -f      app.log.1 (inode 5001) ◄── tail -f এখনো এটা ধরে
আছে!
                                          app.log   (inode 5002) ◄── নতুন লেখা এখানে যাচ্ছে
```

- `tail -f` (= `--follow=descriptor`) খোলা **file descriptor** ধরে থাকে — মানে নির্দিষ্ট inode-কে (অধ্যায় ১১ দেখো)। ফাইল rename হলেও সে পুরোনো ফাইলটাই দেখতে থাকে, নতুন লেখা আর আসে না — অথচ তুমি ভাবছ "কোনো নতুন লগ নেই"!

- `tail -F` (= `--follow=name --retry`) **নাম** ধরে থাকে। নিয়মিত চেক করে `app.log` নামে এখন কোন ফাইল আছে; বদলে গেলে নতুনটা খোলে আর জানিয়ে দেয়:

```
$ tail -F app.log
...
tail: 'app.log' has been replaced;  following new file
```

> 💡 **টিপস:** লগ দেখার সময় অভ্যাস করো `tail -F` লিখতে, `tail -f` না। ঝামেলা কম।

আরও কাজের কৌশল

```
# একসাথে কয়েকটা লগ
$ tail -F /var/log/nginx/access.log /var/log/nginx/error.log

# শুধু error লাইনগুলো লাইভ দেখো
$ tail -F app.log | grep --line-buffered -i error

# শেষ ১০০ লাইন দেখিয়ে তারপর follow
$ tail -n 100 -F app.log

# একটা process চলা পর্যন্ত follow, সে থামলে tail-ও থামবে
$ tail -f build.log --pid=$(pgrep -n make)
```

`--line-buffered` জরুরি: না দিলে `grep` আউটপুট জমিয়ে রাখে (buffer করে) আর লাইন দেরিতে আসে।

systemd-এর যুগে: `journalctl -f`

আধুনিক distro-তে অনেক লগ টেক্সট ফাইলে নয়, systemd journal-এ (binary ফরম্যাটে) থাকে। বিশেষ করে Debian 12 আর Fedora-তে ডিফল্টে `rsyslog` নেই, তাই `/var/log/syslog` বা `/var/log/messages` ফাইলই নাও থাকতে পারে। Ubuntu 24.04 আর RHEL 9-এ সাধারণত rsyslog থাকে (Ubuntu-তে `/var/log/syslog`, RHEL-এ `/var/log/messages`)। journal-এর জন্য `tail -f`-এর সমতুল্য:

```
$ journalctl -f                 # সব লগ লাইভ
$ journalctl -u nginx -f        # শুধু nginx সার্ভিস
$ journalctl -n 50              # শেষ ৫০টা এন্ট্রি
```

বিস্তারিত অধ্যায় ২৮-এ।

> ⚠️ **সতর্কতা:** `/var/log`-এর অনেক ফাইল শুধু root বা `adm` (Debian/Ubuntu) গ্রুপ পড়তে পারে। "Permission denied" এলে `sudo tail -F ...` লাগবে (অধ্যায় ১৮)।

১০.১০ `nl` আর `wc` — নম্বর দেওয়া আর গোনা

`nl` — number lines

`cat -n`-এর চেয়ে বেশি নিয়ন্ত্রণ দেয়। ডিফল্টে **শুধু ফাঁকা-নয় এমন লাইনে** নম্বর দেয় (`cat -b`-এর মতো)।

```
$ nl fruits.txt
     1  আম
     2  জাম

     3  কাঁঠাল
     4  লিচু       পেয়ারা
```

অপশন	কাজ
`-b a`	সব লাইনে নম্বর (body numbering: all)
`-b t`	শুধু ফাঁকা-নয় লাইনে (ডিফল্ট)
`-b n`	কোনো লাইনে না
`-b pREGEX`	শুধু যেসব লাইন REGEX মেলে
`-n ln`	নম্বর বামে সাঁটা (left, no zeros)
`-n rn`	ডানে সাঁটা (ডিফল্ট)
`-n rz`	ডানে সাঁটা, সামনে শূন্য দিয়ে ভরা (`000001`)
`-w N`	নম্বরের জায়গা N ঘর চওড়া (ডিফল্ট ৬)
`-s STR`	নম্বর আর লেখার মাঝে STR (ডিফল্ট Tab)
`-v N`	N থেকে গোনা শুরু (ডিফল্ট ১)
`-i N`	প্রতি লাইনে N করে বাড়াও (ডিফল্ট ১)

```
$ nl -b a -n rz -w 3 -s ': ' fruits.txt
001: আম
002: জাম
003:
004: কাঁঠাল
005: লিচু         পেয়ারা

$ nl -b 'p^ক' fruits.txt     # শুধু "ক" দিয়ে শুরু হওয়া লাইনে নম্বর
```

`wc` — word count

অপশন	কাজ
`-l`	লাইন সংখ্যা (আসলে newline অক্ষরের সংখ্যা)
`-w`	শব্দ সংখ্যা (space/Tab/newline দিয়ে আলাদা করা টুকরো)
`-c`	বাইট সংখ্যা

অপশন	কাজ
`-m`	অক্ষর (character) সংখ্যা — locale মেনে, UTF-8-এ বাংলা ঠিকমতো গোনে
`-L`	সবচেয়ে লম্বা লাইনের দৈর্ঘ্য (display width)

কোনো অপশন না দিলে ক্রমে লাইন, শব্দ, বাইট দেখায়:

```
$ wc numbers.txt
100 100 292 numbers.txt

$ wc -l numbers.txt
100 numbers.txt

$ wc -l < numbers.txt          # শুধু সংখ্যা, ফাইলের নাম ছাড়া (স্ক্রিপ্টে কাজের)
100

$ printf 'আম' | wc -c -m
      2       6
```

শেষ উদাহরণে দেখো: "আম" দুই অক্ষর (`-m` = 2) কিন্তু ছয় বাইট (`-c` = 6), কারণ প্রতিটা বাংলা অক্ষর UTF-8-এ ৩ বাইট। (`wc` সবসময় lines, words, chars, bytes ক্রমে দেখায়, অপশন যে ক্রমেই দাও।)

> ⚠️ **সতর্কতা:** `wc -l` newline গোনে। ফাইলের শেষ লাইনের পরে newline না থাকলে সেই লাইনটা গোনা হয় না! `printf 'a\nb' | wc -l` → `1`, যদিও লাইন দুটো। এজন্যই লিনাক্সে "ফাইলের শেষে newline রাখো" নিয়মটা গুরুত্বপূর্ণ।

```
$ ls /etc | wc -l              # /etc-তে কতগুলো জিনিস
$ grep -c "ERROR" app.log      # কতগুলো লাইনে ERROR (grep নিজেই গোনে)
$ wc -l *.py                   # প্রতিটা ফাইল আর শেষে total
```

`wc` আর অন্যান্য টেক্সট টুলের গভীর ব্যবহার অধ্যায় ২৩-এ।

১০.১১ Binary বনাম Text — আসল পার্থক্য কী

সংজ্ঞা

- **Text ফাইল:** যার সব বাইট কোনো character encoding (যেমন ASCII বা UTF-8) অনুযায়ী পড়ার মতো অক্ষর, আর লাইনগুলো newline দিয়ে আলাদা। উদাহরণ: `.txt`, `.conf`, `.sh`, `.py`, `.html`, `.csv`, `/etc/passwd`।
- **Binary ফাইল:** যার বাইটগুলো অক্ষর হিসেবে পড়ার জন্য না, কোনো প্রোগ্রামের নিজস্ব ফরম্যাট। উদাহরণ: প্রোগ্রাম (`/usr/bin/ls`), ছবি (`.png`), `.zip`, `.pdf`, ডেটাবেস ফাইল।

ভেতরে দুটোই বাইট — পার্থক্য শুধু বাইটগুলোর **অর্থ** কীভাবে করা হবে। একটা মোটা দাগের নিয়ম: যে ফাইলে NUL বাইট (`0x00`) আছে, প্রায় নিশ্চিত সেটা binary। `grep`-ও এভাবেই চেনে, আর তখন বলে "binary file matches"।

উপমা: text ফাইল হলো চিঠি — যেকেউ পড়তে পারে। binary ফাইল হলো ক্যাসেট টেপ — হাতে নিয়ে তাকালে কিছু বোঝা যায় না, প্লেয়ার (নির্দিষ্ট প্রোগ্রাম) লাগে।

লিনাক্সে extension দিয়ে কিছু ঠিক হয় না — `file` কমান্ড

Windows extension দেখে ফাইলের ধরন ঠিক করে। লিনাক্সে extension শুধু নামের অংশ। আসল ধরন জানতে `file` কমান্ড ফাইলের ভেতরের প্রথম কিছু বাইট (**magic number**) দেখে:

```
$ file fruits.txt
fruits.txt: Unicode text, UTF-8 text

$ file /usr/bin/ls
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, ... stripped

$ cp /usr/bin/ls fake.txt
$ file fake.txt
fake.txt: ELF 64-bit LSB pie executable, x86-64, ...
```

`.txt` নাম দিলেও `file` ধরে ফেলেছে এটা প্রোগ্রাম।

ফাইলের ধরন	শুরুর বাইট (magic)
ELF প্রোগ্রাম	`7f 45 4c 46` (`\x7fELF`)
PNG ছবি	`89 50 4e 47 0d 0a 1a 0a`
PDF	`25 50 44 46` (`%PDF`)
ZIP (এবং .docx, .jar)	`50 4b 03 04` (`PK..`)
gzip	`1f 8b`
Shell script	`23 21` (`#!`)

`file` অপশন	কাজ
`-i` / `--mime`	MIME type আর charset দেখাও
`-b`	ফাইলের নাম ছাড়া শুধু ফল
`-L`	symlink অনুসরণ করো
`-z`	compressed ফাইলের ভেতরটা দেখো

```
$ file -i fruits.txt
fruits.txt: text/plain; charset=utf-8
```

binary ফাইল `cat` করলে কী হয়

```
$ cat /usr/bin/ls
ELF>@...@@@8@@@@@@...  (এলোমেলো চিহ্ন, বিপ শব্দ হতে পারে)
```

কিছু বাইট terminal-এর কাছে **escape sequence** — নির্দেশ (যেমন "রং বদলাও", "character set বদলাও")। ফলে টার্মিনাল এলোমেলো হয়ে যেতে পারে: অক্ষর অদ্ভুত দেখায়, prompt গায়েব হয়।

> 💡 **টিপস:** টার্মিনাল নষ্ট হয়ে গেলে (যা টাইপ করছ দেখা যায় না বা আবোল-তাবোল) অন্ধের মতো `reset` টাইপ করে `Enter` চাপো। না হলে `stty sane` চালাও। এটা টার্মিনালকে প্রাথমিক অবস্থায় ফেরায়।

binary ফাইলের ভেতরে দেখার সঠিক টুল হলো `od`, `hexdump`, `xxd` আর `strings`।

১০.১২ hex dump — `od`, `hexdump`, `xxd`

hex কেন?

একটা বাইট মানে ৮টা bit, মান ০–২৫৫। hexadecimal (ষোড়শিক, ০–৯ আর a–f) দিয়ে লিখলে প্রতিটা বাইট ঠিক **দুই অঙ্কে** ধরে (`00` – `ff`)। পড়তে সহজ, সারিতে সুন্দর সাজে। তাই "hex dump" — ফাইলের প্রতিটা বাইট hex-এ দেখানো।

```
একটা hex dump লাইনের গঠন (xxd / hexdump -C):

00000000: 4865 6c6c 6f0a                           Hello.
└──┬───┘ └──────────┬──────────┘                  └──┬──┘
  offset          বাইটগুলো hex-এ                ASCII রূপ
(ফাইলের শুরু থেকে   (প্রতি লাইনে ১৬ বাইট)       (না-পড়ার-মতো
 কত নম্বর বাইট,                                   বাইট = '.')
 hex-এ)
```

উদাহরণের জন্য একটা ফাইল:

```
$ printf 'Hello\n' > hello.txt
```

`od` — octal dump (সবচেয়ে পুরোনো, সব জায়গায় পাওয়া যায়)

`od` GNU coreutils-এর অংশ, তাই প্রায় সব সিস্টেমে থাকে। ডিফল্টে octal (অষ্টক) দেখায় — যেটা আজকাল কম কাজের। তাই অপশন দিয়ে চালাও:

অপশন	কাজ
`-c`	প্রতিটা বাইট অক্ষর হিসেবে, বিশেষ অক্ষর escape (`\n`, `\t`, `\r`, `\0`) দিয়ে

অপশন	কাজ
`-t x1`	প্রতিটা বাইট আলাদা hex
`-t d1` / `-t u1`	signed / unsigned দশমিক
`-t x1z`	hex, সাথে ডানে পাশে অক্ষর রূপ (`>...<`)
`-A x` / `-A d` / `-A n`	offset hex-এ / দশমিকে / দেখাবে না
`-x`	২-বাইট করে hex (byte order-এর কারণে বিভ্রান্তিকর হতে পারে)
`-N N`	শুধু প্রথম N বাইট
`-j N`	প্রথম N বাইট বাদ দিয়ে শুরু
`-v`	একই রকম লাইন `*` দিয়ে সংক্ষেপ করবে না

```
$ od -c hello.txt
0000000   H   e   l   l   o  \n
0000006

$ od -A x -t x1z hello.txt
000000 48 65 6c 6c 6f 0a                                >Hello.<
000006
```

> 💡 **টিপস:** `od -c` হলো লুকানো অক্ষর খোঁজার আরেক দারুণ উপায় — `\r`, `\t`, `\0` স্পষ্ট দেখা যায়।

`hexdump` — `-C` হলো সেরা

`hexdump` আসে util-linux থেকে (Debian 11+ / Ubuntu 22.04+-এ এটা `bsdextrautils` প্যাকেজে আছে, সাধারণত আগে থেকেই ইনস্টল থাকে)। এর `-C` ("canonical") মোডই সবাই ব্যবহার করে:

```
$ hexdump -C hello.txt
00000000  48 65 6c 6c 6f 0a                                 |Hello.|
00000006
```

অপশন	কাজ
`-C`	canonical: offset + ১৬টা hex বাইট + `
`-n N`	শুধু N বাইট
`-s N`	N বাইট বাদ দিয়ে শুরু
`-v`	পুনরাবৃত্ত লাইন `*` দিয়ে সংক্ষেপ করবে না

`hd` হলো `hexdump -C`-এর একটা ছোট নাম (Debian/Ubuntu-তে থাকে; সব distro-তে না)।

একটা PNG ছবির শুরু:

```
$ hexdump -C -n 16 photo.png
00000000  89 50 4e 47 0d 0a 1a 0a  00 00 00 0d 49 48 44 52  |.PNG........IHDR|
00000010
```

ঠিক উপরের টেবিলের magic number!

`xxd` — hex দেখা এবং উল্টো করে ফাইল বানানো

`xxd` আসলে Vim-এর সাথে আসে (Debian/Ubuntu-তে `xxd` বা `vim-common` প্যাকেজ; Fedora-তে নতুন সংস্করণে আলাদা `xxd` প্যাকেজ)। এর বিশেষ ক্ষমতা: hex থেকে আবার binary বানাতে পারে (`-r`)।

```
$ xxd hello.txt
00000000: 4865 6c6c 6f0a                           Hello.
```

অপশন	কাজ
`-l N`	শুধু N বাইট
`-s N`	N বাইট offset থেকে শুরু (`-s -16` = শেষের ১৬ বাইট)
`-c N`	প্রতি লাইনে N বাইট (ডিফল্ট ১৬)
`-g N`	N বাইট করে দল বাঁধো (ডিফল্ট ২; `-g 1` = আলাদা বাইট)
`-b`	hex না, binary (০/১) দেখাও
`-u`	বড় হাতের hex (`A-F`)
`-p`	plain: শুধু hex একটানা, offset/ASCII ছাড়া
`-i`	C ভাষার array হিসেবে আউটপুট
`-r`	উল্টো: hex dump থেকে binary বানাও (`-p` সহ plain hex থেকেও)

```
$ xxd -g 1 hello.txt
00000000: 48 65 6c 6c 6f 0a                              Hello.

$ xxd -b -l 2 hello.txt
00000000: 01001000 01100101                                    He

$ echo -n 'Hi' | xxd -p
4869

$ echo '4869210a' | xxd -r -p
Hi!
```

বাস্তব কৌশল — Vim দিয়ে binary edit:

```
$ xxd file.bin > file.hex      # hex-এ রূপান্তর
$ nano file.hex                # hex এডিট করো (বাম পাশের hex অংশ)
$ xxd -r file.hex > file.bin   # আবার binary
```

Vim-এর ভেতরে `:%!xxd` আর `:%!xxd -r` দিয়েও এটা করা যায় (অধ্যায় ১৪)।

বাংলা অক্ষর বাইটে দেখো

```
$ printf 'আ\n' | xxd
00000000: e0a6 860a                                ....
```

"আ" = `e0 a6 86` (৩ বাইট, UTF-8), তারপর `0a` = newline। ASCII কলামে `...` কারণ ওই বাইটগুলো ASCII-র বাইরে।

তিনটার তুলনা

বৈশিষ্ট্য	`od`	`hexdump`	`xxd`
কোন প্যাকেজ	coreutils (সবখানে)	util-linux / bsdextrautils	vim-common / xxd
সেরা ব্যবহার	`od -c` (escape অক্ষর)	`hexdump -C`	`xxd`, `xxd -r`
hex → binary	না	না	হ্যাঁ (`-r`)
POSIX মান	হ্যাঁ	না	না

১০.১৩ `strings` — binary থেকে পড়ার মতো লেখা বের করা

একটা binary ফাইলে অনেক সময় মানুষের পড়ার মতো লেখা লুকিয়ে থাকে — error message, ভার্সন নম্বর, URL, ফাইলের পথ। `strings` সেগুলো খুঁজে বের করে: সে এমন অক্ষরের সারি খোঁজে যেখানে পরপর অন্তত **৪টা** (ডিফল্ট) printable অক্ষর আছে।

`strings` আসে `binutils` প্যাকেজ থেকে। Ubuntu/Debian-এ না থাকলে `sudo apt install binutils`, Fedora/RHEL-এ `sudo dnf install binutils`।

অপশন	কাজ
`-n N` / `--bytes=N`	কমপক্ষে N অক্ষরের সারি (ডিফল্ট ৪)
`-t x` / `-t d` / `-t o`	প্রতিটা স্ট্রিং-এর offset hex/দশমিক/octal-এ দেখাও
`-a`	পুরো ফাইল স্ক্যান করো (নতুন binutils-এ এটাই ডিফল্ট)
`-e l`	16-bit little-endian (যেমন Windows-এর UTF-16 স্ট্রিং) খোঁজো
`-f`	প্রতিটা লাইনের আগে ফাইলের নাম

```
$ strings /usr/bin/ls | head -n 5
/lib64/ld-linux-x86-64.so.2
...

$ strings -n 10 /usr/bin/ls | grep -i usage
Usage: %s [OPTION]... [FILE]...

$ strings -t x /usr/bin/ls | grep -i version | head -n 2
```

বাস্তব ব্যবহার:

- অজানা প্রোগ্রাম কী করে আন্দাজ করা (কোন URL-এ যোগাযোগ করে, কোন ফাইল পড়ে)
- মুছে যাওয়া/নষ্ট ফাইল থেকে লেখা উদ্ধার
- একটা binary কোন ভার্সনের তা দেখা: `strings app | grep -i 'version'`
- নিরাপত্তা বিশ্লেষণ — সন্দেহজনক ফাইলে হার্ডকোড করা পাসওয়ার্ড বা IP খোঁজা

> ⚠️ **সতর্কতা:** `strings` বাংলা বা অন্য multibyte UTF-8 লেখা ডিফল্টে ঠিকমতো দেখায় না, কারণ সে শুধু ASCII-র printable অক্ষর খোঁজে। নতুন binutils-এ `--unicode=show` অপশন আছে, কিন্তু সব সংস্করণে না।

১০.১৪ Character encoding — অক্ষর কীভাবে বাইট হয়

ASCII থেকে UTF-8 পর্যন্ত

কম্পিউটার শুধু সংখ্যা বোঝে। কোন সংখ্যা কোন অক্ষর — এই চুক্তিকে বলে **character encoding**। **উপমা:** একটা গোপন কোড-বই — "৬৫ মানে A, ৬৬ মানে B"। লেখক আর পাঠক একই কোড-বই না ব্যবহার করলে লেখা আবোল-তাবোল হয়ে যায় (একে বলে **mojibake**)।

```
ASCII (১৯৬৩)        : ০–১২৭, ইংরেজি অক্ষর + সংখ্যা + চিহ্ন, ১ বাইট
Latin-1 (ISO-8859-1): ০–২৫৫, পশ্চিম ইউরোপীয় অক্ষর, ১ বাইট
Unicode             : পৃথিবীর সব অক্ষরের জন্য একটা নম্বর (code point)
                      যেমন 'আ' = U+0986
UTF-8               : Unicode নম্বরকে ১–৪ বাইটে লেখার পদ্ধতি
                      ASCII অক্ষর = ১ বাইট (ASCII-র সাথে হুবহু মেলে!)
                      বাংলা অক্ষর = ৩ বাইট
                      ইমোজি = ৪ বাইট
UTF-16              : Windows-এর ভেতরে প্রচলিত, প্রতি অক্ষর ২ বা ৪ বাইট
```

আধুনিক লিনাক্সে সবকিছু ডিফল্টে **UTF-8**। তোমার locale দেখতে:

```
$ locale | head -n 3
LANG=en_US.UTF-8
LANGUAGE=
LC_CTYPE="en_US.UTF-8"
```

ফাইলের encoding চেনা আর বদলানো

```
$ file -i report.txt
report.txt: text/plain; charset=iso-8859-1
```

encoding বদলাতে `iconv`:

```
$ iconv -f ISO-8859-1 -t UTF-8 report.txt > report-utf8.txt
$ iconv -f UTF-16 -t UTF-8 windows-export.csv > fixed.csv
$ iconv -l | less                          # সমর্থিত সব encoding এর তালিকা
```

`iconv` অপশন	কাজ
`-f ENC`	উৎস (from) encoding
`-t ENC`	লক্ষ্য (to) encoding; `-t ASCII//TRANSLIT` দিলে মেলে না এমন অক্ষর কাছাকাছি ASCII দিয়ে বদলায়
`-c`	রূপান্তর করা যায় না এমন অক্ষর বাদ দাও
`-o FILE`	আউটপুট ফাইল
`-l`	সব encoding এর তালিকা

> 📝 **মনে রাখো:** `file -i` encoding **আন্দাজ** করে, নিশ্চিত জানে না। শুধু ASCII অক্ষরের ফাইলকে সে `us-ascii` বলবে — যেটা UTF-8-এরও বৈধ রূপ।

BOM (Byte Order Mark)

কিছু Windows প্রোগ্রাম (যেমন পুরোনো Notepad, Excel-এর "CSV UTF-8") UTF-8 ফাইলের শুরুতে তিনটা অদৃশ্য বাইট `ef bb bf` বসায় — একে বলে BOM। লিনাক্সে এটা ঝামেলা করে: shell script-এর প্রথম লাইন `#!/bin/bash` আর চেনা যায় না, CSV-র প্রথম কলামের নামে অদৃশ্য অক্ষর ঢুকে যায়।

```
$ head -c 3 data.csv | xxd
00000000: efbb bf                                  ...
$ file data.csv
data.csv: CSV text, Unicode text, UTF-8 (with BOM) text

$ sed -i '1s/^\xEF\xBB\xBF//' data.csv      # BOM মুছে ফেলো (GNU sed)
```

১০.১৫ Line ending — LF বনাম CRLF আর `dos2unix`

ইতিহাস: টাইপরাইটার থেকে

পুরোনো টাইপরাইটারে নতুন লাইনে যেতে দুটো কাজ লাগত: ক্যারেজকে বামে ফেরানো (**Carriage Return**, CR, `\r`, `0x0d`) আর কাগজ এক লাইন উপরে তোলা (**Line Feed**, LF, `\n`, `0x0a`)। বিভিন্ন অপারেটিং সিস্টেম বিভিন্ন নিয়ম বেছে নিল:

সিস্টেম	লাইনের শেষ	বাইট
Linux / macOS / Unix	LF	`0a`
Windows / DOS	CR LF	`0d 0a`

সিস্টেম	লাইনের শেষ	বাইট
পুরোনো Mac OS (৯ পর্যন্ত)	CR	`0d`

সমস্যা কোথায় হয়

Windows-এ লেখা একটা script লিনাক্সে চালালে:

```
$ ./deploy.sh
-bash: ./deploy.sh: /bin/bash^M: bad interpreter: No such file or directory
```

kernel shebang লাইনে `/bin/bash\r` নামে প্রোগ্রাম খুঁজছে — `\r` সহ! এমন কোনো ফাইল নেই। একই কারণে script-এর ভেতরের variable-এর শেষে `\r` লেগে থাকে আর তুলনা মেলে না।

খুঁজে বের করা

```
$ file deploy.sh
deploy.sh: Bourne-Again shell script, ASCII text executable, with CRLF line terminators

$ cat -A deploy.sh | head -n 2
#!/bin/bash^M$
echo "hi"^M$

$ od -c deploy.sh | head -n 2
0000000   #   !   /   b   i   n   /   b   a   s   h  \r  \n   e   c   h
```

`^M$` বা `\r \n` দেখলেই বুঝবে এটা CRLF ফাইল।

ঠিক করা: `dos2unix` আর `unix2dos`

এগুলো সাধারণত আগে থেকে ইনস্টল থাকে না:

```
$ sudo apt install dos2unix        # Ubuntu/Debian
$ sudo dnf install dos2unix        # Fedora/RHEL
```

```
$ dos2unix deploy.sh               # জায়গাতেই CRLF → LF (in-place)
dos2unix: converting file deploy.sh to Unix format...

$ unix2dos notes.txt               # LF → CRLF (Windows-এর জন্য)
$ dos2unix -n in.txt out.txt       # মূল ফাইল ঠিক রেখে নতুন ফাইলে
$ dos2unix -b file.txt             # BOM থাকলে রেখে দাও (keep-bom)
$ dos2unix -r file.txt             # BOM মুছে ফেলো
$ dos2unix -i *.txt                # শুধু তথ্য দেখাও: কোন ফাইলে কতগুলো CRLF/LF আছে
```

অপশন	কাজ
`-n IN OUT`	নতুন ফাইলে লেখো (new file mode)
`-k`	ফাইলের পুরোনো তারিখ (timestamp) রেখে দাও

অপশন	কাজ
`-b` / `-r`	BOM রাখো / সরাও
`-i`	কিছু না বদলে শুধু লাইন-ending গোনা দেখাও
`-c mac`	পুরোনো Mac (শুধু CR) রূপান্তর; এর জন্য `mac2unix` কমান্ডও আছে

`dos2unix` না থাকলে বিকল্প:

```
$ sed -i 's/\r$//' deploy.sh              # GNU sed
$ tr -d '\r' < in.txt > out.txt           # সব \r মুছে ফেলো
```

> 💡 **টিপস:** Git ব্যবহার করলে `.gitattributes`-এ `*.sh text eol=lf` লিখে রাখো — Windows-এ কেউ edit করলেও repository-তে LF থাকবে।

১০.১৬ `column` — ডেটাকে টেবিলে সাজানো

এলোমেলো চওড়ার কলামকে সুন্দর সারিবদ্ধ টেবিলে দেখাতে `column`। লিনাক্সে এটা util-linux-এর (Debian/Ubuntu-তে `bsdextrautils` প্যাকেজের) অংশ।

অপশন	কাজ
`-t` / `--table`	ইনপুটের কলাম চিনে টেবিল বানাও
`-s SEP`	ইনপুটের কলাম-বিভাজক (ডিফল্ট whitespace)
`-o SEP`	আউটপুটে কলামের মাঝে কী বসবে (ডিফল্ট দুটো space)
`-N names`	কলামের নাম দাও (কমা দিয়ে)
`-R cols`	কোন কলাম ডানে সাঁটা হবে
`-J`	টেবিলকে JSON হিসেবে আউটপুট (`-N` লাগবে)
`-c N`	আউটপুটের চওড়া N অক্ষর ধরো
`-x`	`-t` ছাড়া: আগে সারি ধরে ভরো, তারপর কলাম

```
$ cat scores.csv
name,dept,score
Rahim,IT,92
Karim,Finance,8
Sultana,HR,100

$ column -t -s ',' scores.csv
name     dept     score
Rahim    IT       92
Karim    Finance  8
Sultana  HR       100
```

```
$ column -t -s ',' -o ' | ' scores.csv
name    | dept    | score
Rahim   | IT      | 92
Karim   | Finance | 8
Sultana | HR      | 100
```

`/etc/passwd`-কে পড়ার মতো করে দেখো:

```
$ head -n 3 /etc/passwd | column -t -s ':'
root    x  0  0  root    /root      /bin/bash
daemon  x  1  1  daemon  /usr/sbin  /usr/sbin/nologin
bin     x  2  2  bin     /bin       /usr/sbin/nologin
```

`mount` বা অন্য কমান্ডের আউটপুট সাজাতে:

```
$ mount | column -t | less -S
```

`column` ছাড়া একটা শব্দের তালিকা দিলে (`-t` ছাড়া) সেটা `ls`-এর মতো কয়েকটা কলামে ভাগ করে দেখায়:

```
$ seq 1 12 | column -c 40
1       4       7       10
2       5       8       11
3       6       9       12
```

> ⚠️ **সতর্কতা:** `column -t` ফাঁকা ঘরকে ঠিকমতো সামলায় না পুরোনো সংস্করণে — `a,,c` থাকলে কলাম সরে যেতে পারে। আর বাংলা অক্ষরের দৃশ্যমান চওড়া হিসাবে কখনো সারি সামান্য বাঁকা দেখাতে পারে।

১০.১৭ আরও কিছু কাজের টুল (এক নজরে)

টুল	কাজ
`bat`	রং আর লাইন নম্বরসহ `cat` (syntax highlighting)। Ubuntu/Debian-এ কমান্ডের নাম `batcat` (নাম-সংঘাত এড়াতে); Fedora-তে `bat`
`zcat`, `zless`, `zgrep`	`.gz` ফাইল না খুলে দেখা
`xzcat`, `bzcat`, `zstdcat`	`.xz`, `.bz2`, `.zst` ফাইলের জন্য
`rev`	প্রতিটা লাইনের অক্ষর উল্টো
`fold -w 40`	লম্বা লাইন ৪০ অক্ষরে ভাঙো
`expand` / `unexpand`	Tab ↔ space রূপান্তর

টুল	কাজ
`pr`	ছাপার উপযোগী পাতা বানাও (header, কলাম)
`diff a b`	দুই ফাইলের পার্থক্য
`cmp a b`	দুই binary ফাইল বাইট-বাই-বাইট তুলনা, প্রথম অমিল দেখায়
`split`	বড় ফাইল টুকরো করো
`jq .`	JSON সুন্দর করে দেখাও (`sudo apt install jq`)

এগুলোর বেশ কয়েকটার বিস্তারিত অধ্যায় ২৩-এ।

কোন কাজে কোন টুল — সিদ্ধান্তের গাছ

```
ফাইলের ভেতরে দেখতে চাও?
│
├── এটা text না binary জানো না? ──────────────► file
│
├── text ফাইল
│     ├── ছোট (এক স্ক্রিনে ধরে) ──────────────► cat  (লুকানো অক্ষর? cat -A)
│     ├── বড়, ঘুরে ঘুরে পড়বে ──────────────► less
│     ├── শুধু শুরু / শেষ ──────────────► head / tail
│     ├── লাইভ বাড়ছে (লগ) ──────────────► tail -F  বা  less +F
│     ├── লাইন নম্বর চাই ──────────────► cat -n / nl / less -N
│     ├── গুনতে চাই ──────────────► wc
│     └── টেবিল আকারে ──────────────► column -t
│
└── binary ফাইল
      ├── বাইটগুলো দেখতে চাও ──────────────► xxd / hexdump -C / od -c
      └── ভেতরের লেখা খুঁজতে চাও ──────────► strings
```

কীবোর্ড শর্টকাট — এই অধ্যায়ের সব

প্রসঙ্গ	শর্টকাট	কাজ
`less`	`Space` / `f` / `PgDn`	এক পাতা সামনে
`less`	`b` / `PgUp`	এক পাতা পেছনে
`less`	`j` / `Enter` / `↓` — `k` / `↑`	এক লাইন সামনে / পেছনে
`less`	`d` / `u`	আধা পাতা সামনে / পেছনে
`less`	`g` / `<` — `G` / `>`	শুরু / শেষ
`less`	`NUMg` / `NUM%`	নির্দিষ্ট লাইন / শতাংশে
`less`	`/pat` — `?pat`	সামনে / পেছনে সার্চ
`less`	`n` / `N`	পরের / আগের মিল

প্রসঙ্গ	শর্টকাট	কাজ
`less`	`&pat`	শুধু মেলা লাইন দেখাও
`less`	`F` → `Ctrl+C`	follow শুরু → থামাও
`less`	`ma` / `'a`	mark রাখো / mark-এ যাও
`less`	`=`	ফাইলের তথ্য
`less`	`v`	editor-এ খোলো
`less`	`:n` / `:p`	পরের / আগের ফাইল
`less`	`-N` + `Enter`	লাইন নম্বর টগল
`less`	`h`	সাহায্য
`less`	`q`	বের হও
`more`	`Space` / `Enter` / `b` / `/` / `q`	পাতা / লাইন / পেছনে / সার্চ / বের হও
`tail -f`	`Ctrl+C`	থামাও
`cat >`	`Ctrl+D`	ইনপুট শেষ (EOF)
টার্মিনাল	`Ctrl+L`	স্ক্রিন পরিষ্কার
টার্মিনাল	`Ctrl+S` / `Ctrl+Q`	আউটপুট থামাও / আবার চালাও (দ্রুত চলা `cat`-এ কাজের, ভুলে `Ctrl+S` চাপলে টার্মিনাল "জমে" গেলে `Ctrl+Q`)
টার্মিনাল	`Shift+PgUp` / `Shift+PgDn`	অনেক terminal emulator-এ scrollback দেখা
টার্মিনাল নষ্ট	`reset` + `Enter`	টার্মিনাল ঠিক করো

টার্মিনালের অন্য শর্টকাটগুলো অধ্যায় ৬-এ।

হাতে-কলমে অনুশীলন

ল্যাব ১: `cat` আর লুকানো অক্ষর

```
$ mkdir -p ~/ch10 && cd ~/ch10
$ printf 'name   \nage\t25\n\n\n\ncity\r\n' > messy.txt
$ cat messy.txt
$ cat -A messy.txt
$ cat -s -n messy.txt
```

প্রশ্ন: `cat -A`-এ কোন লাইনে অতিরিক্ত space আছে? কোথায় Tab? কোথায় `\r`? `-s` দেওয়ার পর কতগুলো ফাঁকা লাইন থাকল?

ল্যাব ২: `less`-এ দক্ষতা

```
$ less -N /etc/services
```

ভেতরে এই কাজগুলো করো:

1. `G` চেপে শেষে যাও, `g` চেপে শুরুতে ফেরো।
2. `/ssh` লিখে `Enter`, তারপর `n` কয়েকবার, তারপর `N`।
3. `100g` দিয়ে ১০০ নম্বর লাইনে যাও, `ma` চেপে mark রাখো।
4. `G` দিয়ে শেষে যাও, তারপর `'a` দিয়ে ১০০ নম্বর লাইনে ফেরো।
5. `&tcp` লিখে শুধু tcp লাইনগুলো দেখো, তারপর `&` + `Enter` দিয়ে filter বাতিল করো।
6. `=` চেপে দেখো তুমি ফাইলের কত শতাংশে আছ।
7. `q` দিয়ে বের হও।

ল্যাব ৩: লাইভ লগ আর log rotation

দুটো টার্মিনাল খোলো (বা `tmux`, অধ্যায় ২৫)।

টার্মিনাল ১:

```
$ cd ~/ch10
$ while true; do date >> app.log; sleep 1; done
```

টার্মিনাল ২:

```
$ cd ~/ch10
$ tail -f app.log
```

প্রতি সেকেন্ডে নতুন লাইন আসছে দেখবে। এখন **তৃতীয়** টার্মিনালে rotation নকল করো:

```
$ cd ~/ch10 && mv app.log app.log.1
```

টার্মিনাল ২-এ কী হলো? নতুন লাইন আসা বন্ধ (টার্মিনাল ১-এর loop নতুন `app.log` বানিয়ে লিখছে, কিন্তু `tail -f` পুরোনোটা ধরে আছে)। `Ctrl+C` চেপে এবার চালাও:

```
$ tail -F app.log
```

তারপর আবার `mv app.log app.log.2` করো — দেখবে `tail -F` বলছে "has been replaced; following new file" (বা কাছাকাছি বার্তা) আর লেখা আসতে থাকছে। শেষে টার্মিনাল ১-এ `Ctrl+C`।

এবার `less +F app.log` দিয়ে খুলে `Ctrl+C`, `/`-সার্চ, আবার `F` চেষ্টা করো।

ল্যাব ৪: `head`, `tail`, `wc`, `nl`

```
$ seq 1 1000 > big.txt
$ head -n 5 big.txt
$ tail -n 5 big.txt
$ tail -n +996 big.txt
$ head -n 510 big.txt | tail -n 11      # ৫০০ থেকে ৫১০
$ wc -l big.txt
$ wc -l < big.txt
$ nl -b a -n rz -w 4 fruits.txt
$ printf 'বাংলা' | wc -c -m
```

প্রশ্ন: "বাংলা"-তে কত অক্ষর আর কত বাইট? কেন আলাদা? (ইঙ্গিত: "বাংলা" লিখতে Unicode-এ ব, া, ং, ল, া — ৫টা code point লাগে।)

ল্যাব ৫: binary-র ভেতরে উঁকি

```
$ file /usr/bin/ls /etc/passwd ~/ch10/fruits.txt
$ xxd -l 16 /usr/bin/ls
$ hexdump -C -n 16 /usr/bin/ls
$ od -c -N 4 /usr/bin/ls
$ strings -n 8 /usr/bin/ls | grep -i usage
$ printf 'আমি' | xxd -g 1
```

প্রশ্ন: `/usr/bin/ls`-এর প্রথম চারটা বাইট কী? এগুলো কী নির্দেশ করে?

ল্যাব ৬: hex থেকে ফাইল বানাও

```
$ echo '4c696e7578206969732066756e210a' | xxd -r -p
$ echo '4c696e7578206969732066756e210a' | xxd -r -p > secret.txt
$ file secret.txt && cat secret.txt
```

ল্যাব ৭: Windows ফাইল মেরামত

```
$ printf '#!/bin/bash\r\necho "Hello from Windows"\r\n' > win.sh
$ chmod +x win.sh
$ ./win.sh                  # error দেখো
$ file win.sh
$ cat -A win.sh
$ sudo apt install dos2unix   # বা: sudo dnf install dos2unix
$ dos2unix win.sh
$ file win.sh
$ ./win.sh
Hello from Windows
```

(`chmod` নিয়ে অধ্যায় ১৬।)

ল্যাব ৮: encoding রূপান্তর

```
$ printf 'caf\xe9\n' > latin.txt         # Latin-1-এ "café"
$ cat latin.txt                           # ভাঙা অক্ষর দেখাবে
$ file -i latin.txt
$ iconv -f ISO-8859-1 -t UTF-8 latin.txt > utf8.txt
$ cat utf8.txt
café
$ xxd latin.txt; xxd utf8.txt            # 'é' কীভাবে ১ বাইট থেকে ২ বাইট হলো দেখো
```

ল্যাব ৯: `column`

```
$ printf 'name,dept,score\nRahim,IT,92\nKarim,Finance,8\nSultana,HR,100\n' > scores.csv
$ column -t -s ',' scores.csv
$ column -t -s ',' -o ' | ' scores.csv
$ head -n 5 /etc/passwd | column -t -s ':'
$ df -h | column -t
```

সারসংক্ষেপ

কমান্ড	মূল কাজ	সবচেয়ে কাজের রূপ
`cat`	ফাইল দেখাও / জোড়া লাগাও	`cat -n f`, `cat -A f`, `cat a b > c`
`tac`	লাইন উল্টো ক্রমে	`tac log | less`
`less`	পাতায় পাতায় দেখা, সার্চ	`less -N f`, `less +F log`, `less -S f.csv`
`more`	পুরোনো সহজ pager	`more f`
`head`	শুরুর লাইন/বাইট	`head -n 20 f`, `head -c 100 f`, `head -n -5 f`
`tail`	শেষের লাইন/বাইট, লাইভ	`tail -n 50 f`, `tail -n +2 f`, `tail -F log`
`nl`	নিয়ন্ত্রিত লাইন নম্বর	`nl -b a f`
`wc`	লাইন/শব্দ/বাইট/অক্ষর গোনা	`wc -l f`, `wc -l < f`, `wc -m f`
`file`	আসল ফাইল-ধরন আর encoding	`file f`, `file -i f`
`od`	octal/hex/অক্ষর dump	`od -c f`, `od -A x -t x1z f`
`hexdump`	hex dump	`hexdump -C f`
`xxd`	hex dump ও উল্টো	`xxd f`, `xxd -p`, `xxd -r -p`
`strings`	binary থেকে লেখা	`strings -n 8 f`
`iconv`	encoding রূপান্তর	`iconv -f UTF-16 -t UTF-8 in > out`
`dos2unix` / `unix2dos`	CRLF ↔ LF	`dos2unix f`
`column`	টেবিলে সাজাও	`column -t -s ',' f.csv`

কমান্ড	মূল কাজ	সবচেয়ে কাজের রূপ
`reset`	নষ্ট টার্মিনাল ঠিক করো	`reset`
`journalctl -f`	systemd journal লাইভ	`journalctl -u nginx -f`

নিজেকে যাচাই করো

1. `cat -n` আর `cat -b`-এর পার্থক্য কী?
2. একটা script চালালে `/bin/bash^M: bad interpreter` আসছে। সমস্যা কী, কীভাবে নিশ্চিত হবে আর কীভাবে ঠিক করবে?
3. `less`-এ ফাইলের শেষে যাওয়া, শুরুতে যাওয়া, পেছনের দিকে সার্চ করা আর follow মোড চালু করার শর্টকাট কী কী?
4. `tail -f` আর `tail -F`-এর পার্থক্য কী? লগ দেখার জন্য কোনটা ভালো, কেন?
5. একটা CSV ফাইলের header লাইন বাদ দিয়ে বাকি সব কীভাবে দেখবে?
6. ১০০০ লাইনের একটা ফাইলের ৪০০ থেকে ৪১০ নম্বর লাইন `head` আর `tail` দিয়ে কীভাবে দেখবে?
7. `printf 'আম' | wc -c` আর `printf 'আম' | wc -m` আলাদা ফল দেয় কেন?
8. extension ছাড়া একটা ফাইল পেয়েছ। এটা ছবি, প্রোগ্রাম নাকি লেখা — কীভাবে জানবে? `file` কীসের ভিত্তিতে বলে?
9. hex লেখা `48656c6c6f` থেকে আসল লেখা কীভাবে বের করবে?
10. binary ফাইল `cat` করার পর টার্মিনাল আবোল-তাবোল দেখাচ্ছে। কী করবে?

▼ উত্তর দেখো

1. `cat -n` সব লাইনে (ফাঁকা লাইনসহ) নম্বর দেয়; `cat -b` শুধু ফাঁকা-নয় এমন লাইনে নম্বর দেয়। দুটো একসাথে দিলে `-b` জেতে।

2. ফাইলটা Windows-এ লেখা, তাই প্রতি লাইনের শেষে CRLF (`\r\n`) আছে। kernel `/bin/bash\r` নামে interpreter খুঁজছে। নিশ্চিত হতে `file script.sh` ("with CRLF line terminators") বা `cat -A script.sh` (লাইনের শেষে `^M$`)। ঠিক করতে `dos2unix script.sh` বা `sed -i 's/\r$//' script.sh`।

3. শেষে: `G` (বা `>`); শুরুতে: `g` (বা `<`); পেছনের দিকে সার্চ: `?pattern` তারপর `Enter` (পরের মিলের জন্য `n`, উল্টো দিকে `N`); follow মোড: `F` (থামাতে `Ctrl+C`)। ফাইল খোলার সময়ই follow চাইলে `less +F file`।

4. `tail -f` খোলা file descriptor (নির্দিষ্ট inode) ধরে রাখে; log rotation-এ ফাইল rename হলে সে পুরোনো ফাইলেই আটকে থাকে, নতুন লেখা দেখায় না। `tail -F` (= `--follow=name --retry`) ফাইলের নাম ধরে রাখে আর ফাইল বদলে গেলে বা মুছে আবার তৈরি হলে নতুনটা খোলে। লগের জন্য `tail -F` ভালো।

5. `tail -n +2 data.csv` — দ্বিতীয় লাইন থেকে শেষ পর্যন্ত।

6. `head -n 410 file | tail -n 11` (৪০০ থেকে ৪১০ = ১১টা লাইন)। বিকল্প: `sed -n '400,410p' file`।

7. `-c` বাইট গোনে, `-m` অক্ষর গোনে। UTF-8-এ প্রতিটা বাংলা অক্ষর ৩ বাইট, তাই "আম" = ২ অক্ষর কিন্তু ৬ বাইট। (`-m` সঠিক গুনতে UTF-8 locale লাগে।)

8. `file filename` চালাও। `file` extension দেখে না; ফাইলের শুরুর বাইটগুলো (magic number, যেমন ELF-এর `7f 45 4c 46`, PNG-এর `89 50 4e 47`) আর ভেতরের বিষয়বস্তু পরীক্ষা করে ধরন বলে। চাইলে `xxd -l 16 filename` দিয়ে নিজেই শুরুর বাইট দেখতে পারো।

9. `echo 48656c6c6f | xxd -r -p` → `Hello`। (`-p` মানে plain hex, `-r` মানে উল্টো রূপান্তর।)

10. `reset` টাইপ করে `Enter` চাপো (লেখা দেখা না গেলেও)। কাজ না করলে `stty sane`। ভবিষ্যতে binary ফাইল দেখতে `xxd`, `hexdump -C` বা `strings` ব্যবহার করো, আর আগে `file` দিয়ে ধরন দেখে নাও।

অধ্যায় ১১: inode, হার্ড লিংক ও সফট লিংক

এতদিন আমরা ফাইলকে দেখেছি তার **নাম** দিয়ে: `notes.txt`, `/etc/passwd`, `photo.jpg`। কিন্তু লিনাক্সের ভেতরে ফাইলের আসল পরিচয় তার নাম নয়, একটা **নম্বর**। সেই নম্বরের নাম **inode number**। এই অধ্যায়ের বিষয়টা শুরুতে একটু তাত্ত্বিক মনে হতে পারে। তবু এটা একবার বুঝলে অনেক রহস্য খুলে যাবে। যেমন: `mv` এত দ্রুত কেন, ফাইল মুছলেও ডিস্ক খালি হয় না কেন, `df -h`-এ জায়গা থাকা সত্ত্বেও "No space left on device" কেন আসে, `/bin` আসলে কোথায়, আর `python3` টাইপ করলে কোন ভার্সন চলে।

এই অধ্যায়ে যা শিখবে

- inode কী, এর ভেতরে কী কী তথ্য থাকে আর কী থাকে **না**
- ডিরেক্টরি আসলে কী: নাম থেকে inode নম্বরে যাওয়ার একটা টেবিল
- `ls -i` আর `stat` দিয়ে inode দেখা
- হার্ড লিংক (`ln`) এবং link count
- সিম্বলিক বা সফট লিংক (`ln -s`)
- হার্ড লিংক ও সফট লিংকের পার্থক্য
- ভাঙা লিংক (broken/dangling link) খুঁজে বের করা ও ঠিক করা
- `readlink` আর `realpath` দিয়ে লিংকের আসল গন্তব্য বের করা
- `rm` চালালে ভেতরে আসলে কী ঘটে, আর মুছে ফেলা কিন্তু এখনো খোলা ফাইল
- `df -i` এবং inode শেষ হয়ে যাওয়ার সমস্যা
- বাস্তব ব্যবহার: `/etc/alternatives`, ভার্সনযুক্ত লাইব্রেরি, `/bin → /usr/bin`, ডিপ্লয়মেন্টের `current` লিংক

১১.১ একটা উপমা দিয়ে শুরু করি: লাইব্রেরি

ধরো একটা বড় লাইব্রেরিতে গেলে। বই খুঁজতে তুমি প্রথমে **ক্যাটালগ কার্ডে** যাও:

- কার্ডে লেখা থাকে: *"পথের পাঁচালী" → শেলফ নম্বর 4521*
- শেলফ 4521-এ বইয়ের একটা **রেজিস্টার এন্ট্রি** আছে: লেখক কে, কত পৃষ্ঠা, কবে কেনা, কে ধার নিতে পারবে, বইটা কোন তাকে রাখা
- আর আছে **বইটা নিজে**, মানে আসল পাতাগুলো

লিনাক্সের সাথে মিলিয়ে দেখো:

লাইব্রেরি	লিনাক্স
ক্যাটালগ কার্ড (নাম → নম্বর)	**ডিরেক্টরি এন্ট্রি** (filename → inode number)

লাইব্রেরি	লিনাক্স
রেজিস্টার এন্ট্রি (মেটাডেটা)	**inode**
বইয়ের পাতা	**data blocks** (ডিস্কে আসল কনটেন্ট)

একই বইয়ের জন্য দুটো ক্যাটালগ কার্ড থাকতে পারে, একটা "পথের পাঁচালী" নামে আর একটা "Pather Panchali" নামে। দুটোই একই শেলফ নম্বর দেখায়। এটাই **হার্ড লিংক**।

আবার এমন একটা কার্ড থাকতে পারে যেটায় শুধু লেখা: *"পথের পাঁচালী কার্ডটা দেখো"*। এটা বইটাকে সরাসরি দেখায় না, আরেকটা **নাম** দেখায়। এটাই **সফট লিংক** (symbolic link)।

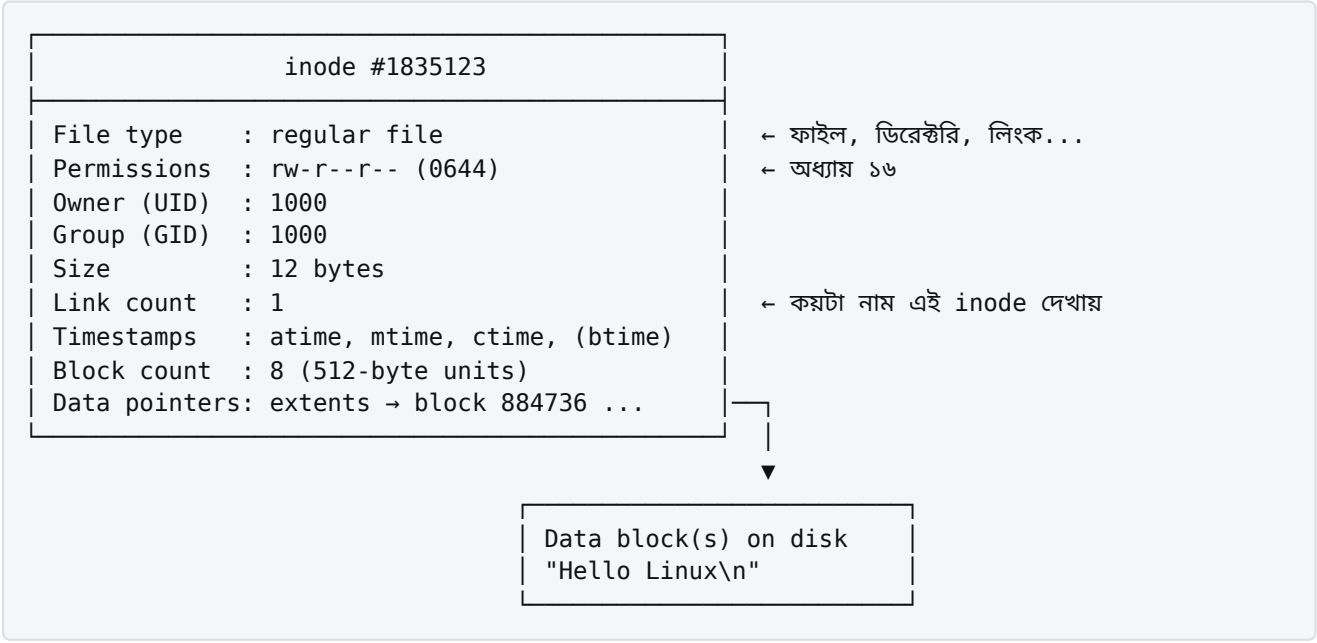
> 📝 **মনে রাখো:** লিনাক্সে ফাইলের **নাম** আর ফাইল **নিজে** দুটো আলাদা জিনিস। নাম থাকে ডিরেক্টরিতে, ফাইলের পরিচয় থাকে inode-এ, আর কনটেন্ট থাকে data block-এ।

১১.২ inode কী?

inode (index node, অর্থাৎ সূচক নোড) হলো ফাইলসিস্টেমের ভেতরের একটা ছোট ডেটা স্ট্রাকচার। প্রতিটা ফাইল, ডিরেক্টরি, সিম্বলিক লিংক বা ডিভাইস ফাইলের একটা করে inode থাকে। প্রতিটা inode-এর একটা নম্বর থাকে, যেটা **ওই ফাইলসিস্টেমের ভেতরে** ইউনিক।

ext4-এর মতো প্রচলিত লিনাক্স ফাইলসিস্টেমে একটা inode সাধারণত 256 বাইটের হয়।

inode-এ কী কী থাকে



বিস্তারিত:

তথ্য	ব্যাখ্যা
File type	regular file, directory, symbolic link, block/character device, FIFO (named pipe), socket
Mode / permissions	rwx পারমিশন + setuid/setgid/sticky bit (অধ্যায় ১৬, অধ্যায় ১৭)
UID / GID	মালিক ইউজার ও গ্রুপের **নম্বর** (নাম নয়)
Size	বাইটে ফাইলের আকার
Link count	কয়টা ডিরেক্টরি এন্ট্রি (নাম) এই inode-কে দেখায়
atime	access time: শেষ কবে **পড়া** হয়েছে
mtime	modify time: শেষ কবে **কনটেন্ট** বদলেছে
ctime	change time: শেষ কবে **inode** বদলেছে (পারমিশন, মালিক, link count, কনটেন্ট)
btime / crtime	birth time: কবে তৈরি হয়েছে (ext4, XFS, btrfs সাপোর্ট করে)
Block pointers / extents	ডিস্কে ডেটা কোথায় আছে তার ঠিকানা

inode-এ কী থাকে না

> ⚠️ **সতর্কতা:** inode-এ ফাইলের **নাম** থাকে না! নাম থাকে ডিরেক্টরিতে। এই একটা তথ্য বুঝলেই হার্ড লিংক, `mv` আর `rm` সব পরিষ্কার হয়ে যায়।

> 💡 **টিপস:** `ctime`-কে অনেকে "creation time" ভাবে। এটা ভুল। `ctime` মানে **change** time। তৈরির সময় হলো `btime` (Birth)।

১১.৩ ডিরেক্টরি আসলে কী?

তুমি হয়তো ভাবো ডিরেক্টরি হলো একটা "বাক্স" যার ভেতরে ফাইল রাখা থাকে। আসলে ডিরেক্টরিও একটা **বিশেষ ফাইল**। এর নিজের inode আছে, আর এর কনটেন্ট হলো একটা **টেবিল**: কোন নাম কোন inode নম্বরে যায়।

```
ডিরেক্টরি /home/rahim/project   (নিজের inode: 1835100)
┌──────────────────┬──────────────┐
│ Name             │ Inode number │
├──────────────────┼──────────────┤
│ .                │ 1835100      │ ← নিজে
│ ..               │ 1835001      │ ← প্যারেন্ট (/home/rahim)
│ notes.txt        │ 1835123      │
│ report.pdf       │ 1835124      │
│ src              │ 1835130      │ ← সাব-ডিরেক্টরি
└──────────────────┴──────────────┘
```

তাহলে কার্নেল যখন `/home/rahim/project/notes.txt` খোলে, তখন ভেতরে এই কাজগুলো হয় (একে বলে **path resolution** বা path lookup):

```
"/"           → root ডিরেক্টরি, inode 2 (ext4-এ root সবসময় 2)
  └─ "home"   → / এর টেবিলে খোঁজো       → inode 1572865
      └─ "rahim"   → home এর টেবিলে খোঁজো → inode 1835001
          └─ "project" → ...              → inode 1835100
              └─ "notes.txt" → ...        → inode 1835123
                                              │
                                              ▼
                                  inode পড়ো → পারমিশন চেক → data block পড়ো
```

> 💡 **টিপস:** বারবার ডিস্ক থেকে এই টেবিল না পড়তে কার্নেল একটা ক্যাশ রাখে। এর নাম **dentry cache** (directory entry cache)। এজন্য একই path দ্বিতীয়বার অনেক দ্রুত খোলে।

এই মডেল থেকে কয়েকটা জিনিস বোঝা যায়:

1. **ফাইল rename (`mv`) একই ফাইলসিস্টেমে প্রায় তাৎক্ষণিক।** 10 GB ফাইল সরালেও ডেটা নড়ে না, শুধু ডিরেক্টরির টেবিলে একটা লাইন বদলায়। inode নম্বর একই থাকে।
2. **ভিন্ন ফাইলসিস্টেমে `mv` আসলে copy + delete।** নতুন ফাইলসিস্টেমে নতুন inode তৈরি হয়, তাই সময় লাগে।
3. **ফাইল তৈরি বা মোছার জন্য ডিরেক্টরিতে write পারমিশন লাগে**, ফাইলে না। কারণ বদলাচ্ছে ডিরেক্টরির টেবিল। এটা অধ্যায় ১৬-এ আবার দেখবে।
4. **ডিরেক্টরির `x` পারমিশন** মানে টেবিলে নাম ধরে inode খুঁজে ভেতরে ঢোকার অনুমতি। **`r` পারমিশন** মানে নামগুলোর তালিকা দেখার অনুমতি।

১১.৪ `ls -i`: inode নম্বর দেখা

`ls`-এর `-i` (`--inode`) অপশন প্রতিটা ফাইলের আগে তার inode নম্বর দেখায়।

Syntax:

```
ls -i [OPTION]... [FILE]...
```

উদাহরণ:

```
$ mkdir ~/inode-lab && cd ~/inode-lab
$ echo "Hello Linux" > notes.txt
$ echo "second" > other.txt
$ ls -i
1835123 notes.txt  1835124 other.txt
```

`-l`-এর সাথে মিলিয়ে দেখলে আরও বেশি তথ্য পাওয়া যায়:

```
$ ls -li
total 8
1835123 -rw-r--r-- 1 rahim rahim 12 Nov 12 10:15 notes.txt
1835124 -rw-r--r-- 1 rahim rahim  7 Nov 12 10:15 other.txt
   |         |      |
   |         |      └── link count
   |         └───────── type + permissions
   └─────────────────── inode number
```

অপশন	কাজ
`-i`	inode নম্বর দেখাও
`-l`	লম্বা ফরম্যাট (link count সহ)
`-d`	ডিরেক্টরির ভেতরের জিনিস না দেখিয়ে ডিরেক্টরিটাকেই দেখাও
`-a`	`.` আর `..` সহ লুকানো ফাইলও দেখাও
`-L`	সিম্বলিক লিংক হলে লিংকের বদলে টার্গেটের তথ্য দেখাও

```
$ ls -lia
total 16
1835100 drwxr-xr-x  2 rahim rahim 4096 Nov 12 10:15 .
1835001 drwxr-x--- 25 rahim rahim 4096 Nov 12 10:14 ..
1835123 -rw-r--r--  1 rahim rahim   12 Nov 12 10:15 notes.txt
1835124 -rw-r--r--  1 rahim rahim    7 Nov 12 10:15 other.txt
```

> 📝 **মনে রাখো:** তোমার সিস্টেমে inode নম্বরগুলো ভিন্ন হবে। এখানের নম্বরগুলো শুধু উদাহরণ।

১১.৫ `stat` : inode-এর পুরো তথ্য

`ls -l` কিছু তথ্য দেখায়, কিন্তু inode-এর **সব** তথ্য দেখতে চাইলে `stat` লাগবে।

Syntax:

```
stat [OPTION]... FILE...
```

```
$ stat notes.txt
  File: notes.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 1835123     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   rahim)   Gid: ( 1000/   rahim)
Access: 2026-11-12 10:15:42.118204511 +0600
Modify: 2026-11-12 10:15:42.118204511 +0600
Change: 2026-11-12 10:15:42.118204511 +0600
 Birth: 2026-11-12 10:15:42.118204511 +0600
```

প্রতিটা লাইনের মানে:

ফিল্ড	মানে
`Size`	বাইটে আকার
`Blocks`	কতগুলো 512-বাইট ব্লক বরাদ্দ হয়েছে (12 বাইটের জন্যও পুরো 4 KB ব্লক নেয়, মানে 8 × 512)
`IO Block`	ফাইলসিস্টেমের পছন্দের I/O ব্লকের আকার
`Device`	কোন ডিভাইসে আছে (major,minor নম্বর)
`Inode`	inode নম্বর
`Links`	link count
`Access: (0644/...)`	পারমিশন, অক্টাল ও rwx দুই রূপেই
`Uid` / `Gid`	মালিকের নম্বর ও নাম
`Access` / `Modify` / `Change` / `Birth`	চারটা টাইমস্ট্যাম্প

> 💡 **টিপস:** পুরনো coreutils-এ `Device` লাইনটা `Device: 802h/2050d` রূপে দেখায় (hex আর decimal)। মানে একই, শুধু ফরম্যাট আলাদা। Ubuntu 24.04-এর coreutils 9.x `8,2` রূপে দেখায়।

`stat`-এর গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-c FORMAT` / `--format=FORMAT`	নিজের মতো ফরম্যাটে নির্দিষ্ট তথ্য দেখাও
`-L` / `--dereference`	সিম্বলিক লিংক হলে টার্গেটের তথ্য দেখাও
`-f` / `--file-system`	ফাইলের বদলে ফাইলসিস্টেমের তথ্য দেখাও (মোট/ফ্রি inode সহ)
`-t` / `--terse`	সংক্ষিপ্ত এক-লাইনের আউটপুট (স্ক্রিপ্টের জন্য)

`-c` ফরম্যাটের দরকারি চিহ্ন

চিহ্ন	মানে
`%n`	ফাইলের নাম
`%i`	inode নম্বর
`%h`	হার্ড লিংক সংখ্যা
`%s`	আকার (বাইট)
`%F`	ফাইলের ধরন (লেখায়)
`%a` / `%A`	পারমিশন অক্টাল / rwx রূপে

চিহ্ন	মানে
`%U` / `%G`	মালিক / গ্রুপের নাম
`%d`	ডিভাইস নম্বর (decimal)
`%x` `%y` `%z` `%w`	atime, mtime, ctime, birth time
`%N`	উদ্ধৃত নাম; লিংক হলে `-> টার্গেট` সহ

```
$ stat -c '%n  inode=%i  links=%h  type=%F' notes.txt other.txt
notes.txt  inode=1835123  links=1  type=regular file
other.txt  inode=1835124  links=1  type=regular file
```

ফাইলসিস্টেমের inode পরিসংখ্যান দেখতে:

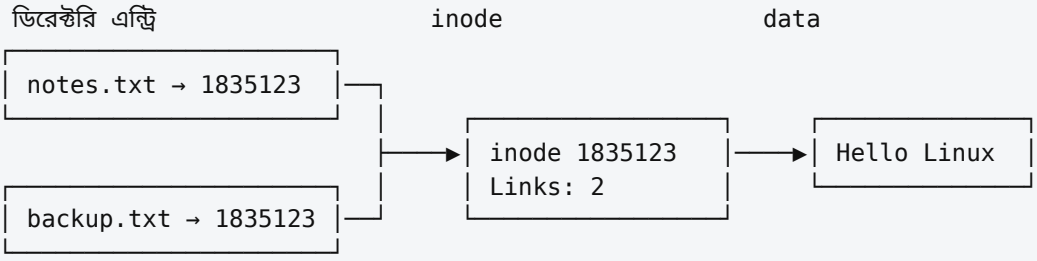
```
$ stat -f /
  File: "/"
    ID: 1a2b3c4d5e6f7a8b Namelen: 255     Type: ext2/ext3
Block size: 4096       Fundamental block size: 4096
Blocks: Total: 61202592   Free: 40123456   Available: 37000000
Inodes: Total: 15630336   Free: 14812001
```

> 📝 **মনে রাখো:** ext4-ও `stat -f`-এ `ext2/ext3` হিসেবে দেখায়, কারণ তিনটার magic number একই। ফাইলসিস্টেমের আসল ধরন জানতে `df -T` বা `findmnt` ব্যবহার করো (অধ্যায় ৩২)।

১১.৬ হার্ড লিংক (Hard Link)

ধারণা

হার্ড লিংক মানে একই inode-এর জন্য আরেকটা **নাম**, অর্থাৎ ডিরেক্টরিতে আরেকটা এন্ট্রি। "আসল" আর "কপি" বলে কিছু নেই। সব নাম সমান মর্যাদার। প্রথম যে নামে ফাইল তৈরি হয়েছিল, সেটাও আসলে একটা হার্ড লিংক।



`ln` কমান্ড

Syntax:

```
ln [OPTION]... TARGET LINK_NAME        # একটা লিংক বানাও
ln [OPTION]... TARGET... DIRECTORY     # ডিরেক্টরির ভেতরে একই নামে লিংক বানাও
```

> 📝 **মনে রাখো:** ক্রম `cp`-এর মতো: আগে **যেটা আছে**, পরে **নতুন নাম**। মনে রাখার কৌশল: "ln **থেকে কী**"।

```
$ ln notes.txt backup.txt
$ ls -li
total 12
1835123 -rw-r--r-- 2 rahim rahim 12 Nov 12 10:15 backup.txt
1835123 -rw-r--r-- 2 rahim rahim 12 Nov 12 10:15 notes.txt
1835124 -rw-r--r-- 1 rahim rahim  7 Nov 12 10:15 other.txt
```

খেয়াল করো:

- দুটো নামের inode নম্বর **একই** (1835123)
- link count এখন **2**
- আকার, পারমিশন, সময় সবকিছু হুবহু এক, কারণ এগুলো inode-এ থাকে, আর inode একটাই

একটা নাম দিয়ে বদলালে অন্য নামেও বদল দেখা যাবে:

```
$ echo "added line" >> backup.txt
$ cat notes.txt
Hello Linux
added line
```

একটা নাম মুছলে অন্যটা টিকে থাকে:

```
$ rm notes.txt
$ ls -li backup.txt
1835123 -rw-r--r-- 1 rahim rahim 23 Nov 12 10:20 backup.txt
$ cat backup.txt
Hello Linux
added line
```

link count আবার 1 হয়ে গেছে, ডেটা অক্ষত।

`ln`-এর গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-s`, `--symbolic`	হার্ড লিংকের বদলে সিম্বলিক লিংক বানাও
`-f`, `--force`	একই নামে আগে কিছু থাকলে সেটা মুছে নতুন লিংক বানাও
`-i`, `--interactive`	আগের ফাইল মোছার আগে জিজ্ঞেস করো
`-n`, `--no-dereference`	LINK_NAME যদি ডিরেক্টরির দিকে সিমলিংক হয়, তাকে সাধারণ ফাইলের মতো ধরো (`-f`-এর সাথে ব্যবহার হয়)

অপশন	কাজ
`-T`, `--no-target-directory`	LINK_NAME-কে সবসময় ফাইলের নাম ধরো, ডিরেক্টরি নয়
`-r`, `--relative`	সিম্বলিক লিংকে relative path বসাও (`-s`-এর সাথে)
`-v`, `--verbose`	কী বানানো হলো দেখাও
`-b`, `--backup`	আগের ফাইল মুছে না দিয়ে `~` যোগ করে ব্যাকআপ রাখো
`-t DIR`, `--target-directory=DIR`	কোন ডিরেক্টরিতে লিংক বানাবে আগে বলে দাও
`-L` / `-P`	হার্ড লিংক করার সময় TARGET সিমলিংক হলে তার টার্গেটে (`-L`) নাকি সিমলিংকে নিজেই (`-P`) লিংক করবে

```
$ ln -v backup.txt copy2.txt
'copy2.txt' => 'backup.txt'
```

link count-এর রহস্য: ডিরেক্টরি

```
$ mkdir demo
$ ls -ld demo
drwxr-xr-x 2 rahim rahim 4096 Nov 12 10:25 demo
```

খালি ডিরেক্টরির link count **2** কেন? কারণ দুটো নাম এই inode-কে দেখায়:

1. প্যারেন্ট ডিরেক্টরিতে `demo` এন্ট্রি
2. `demo`-র ভেতরে `.` এন্ট্রি

এবার একটা সাব-ডিরেক্টরি বানাও:

```
$ mkdir demo/sub
$ ls -ld demo
drwxr-xr-x 3 rahim rahim 4096 Nov 12 10:26 demo
```

3 হলো, কারণ `demo/sub/..`-ও `demo`-কে দেখায়। সূত্রটা তাহলে:

```
ডিরেক্টরির link count = 2 + (সরাসরি সাব-ডিরেক্টরির সংখ্যা)
```

> 💡 **টিপস:** এটা ext4 ও XFS-এর আচরণ। btrfs-এ ডিরেক্টরির link count সবসময় 1 দেখায়, তাই সব ফাইলসিস্টেমে এই হিসাবের উপর নির্ভর কোরো না।

হার্ড লিংকের সীমাবদ্ধতা

১. ডিরেক্টরির হার্ড লিংক করা যায় না:

```
$ ln demo demo-link
ln: demo: hard link not allowed for directory
```

কেন? ডিরেক্টরির হার্ড লিংক অনুমতি পেলে ফাইলসিস্টেম ট্রিতে **লুপ** তৈরি হতে পারত (A-র ভেতরে B, B-র ভেতরে A)। তখন `find` বা `du` অনন্তকাল ঘুরত, আর `..` কোন প্যারেন্টকে বোঝাবে সেটাও অস্পষ্ট হতো। `.` আর `..` শুধু কার্নেল নিজে বানায়।

২. ফাইলসিস্টেমের সীমা পার হওয়া যায় না:

```
$ ln ~/inode-lab/backup.txt /mnt/usb/backup.txt
ln: failed to create hard link '/mnt/usb/backup.txt' => '/home/rahim/inode-
lab/backup.txt': Invalid cross-device link
```

কেন? inode নম্বর শুধু **একটা ফাইলসিস্টেমের ভেতরে** ইউনিক। USB ড্রাইভের ডিরেক্টরি টেবিলে "1835123" লিখলে সেটা USB-এর নিজের 1835123 নম্বর inode-কে বোঝাবে, যেটা সম্পূর্ণ অন্য ফাইল।

> 📝 **মনে রাখো:** দুটো ফাইল একই কিনা নিশ্চিত হতে inode নম্বরের সাথে device নম্বরও মিলাতে হয়: `stat -c '%d:%i' FILE`।

৩. নিরাপত্তা সীমা (protected_hardlinks): আধুনিক distro-তে `fs.protected_hardlinks = 1` চালু থাকে। এর মানে সাধারণ ইউজার এমন ফাইলের হার্ড লিংক বানাতে পারে না যেটার মালিক সে নয় এবং যেটা পড়া/লেখার অনুমতিও তার নেই। এতে `/tmp`-এ লিংক বানিয়ে প্রিভিলেজড প্রোগ্রামকে ধোঁকা দেওয়ার আক্রমণ বন্ধ হয়।

```
$ sysctl fs.protected_hardlinks fs.protected_symlinks
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
```

হার্ড লিংক খোঁজা

একই inode-এর সব নাম খুঁজতে:

```
$ find ~/inode-lab -samefile backup.txt
/home/rahim/inode-lab/backup.txt
/home/rahim/inode-lab/copy2.txt

$ find ~/inode-lab -inum 1835123
/home/rahim/inode-lab/backup.txt
/home/rahim/inode-lab/copy2.txt

$ find ~/inode-lab -type f -links +1     # একাধিক নাম আছে এমন ফাইল
```

`find` নিয়ে বিস্তারিত আছে অধ্যায় ১২-এ।

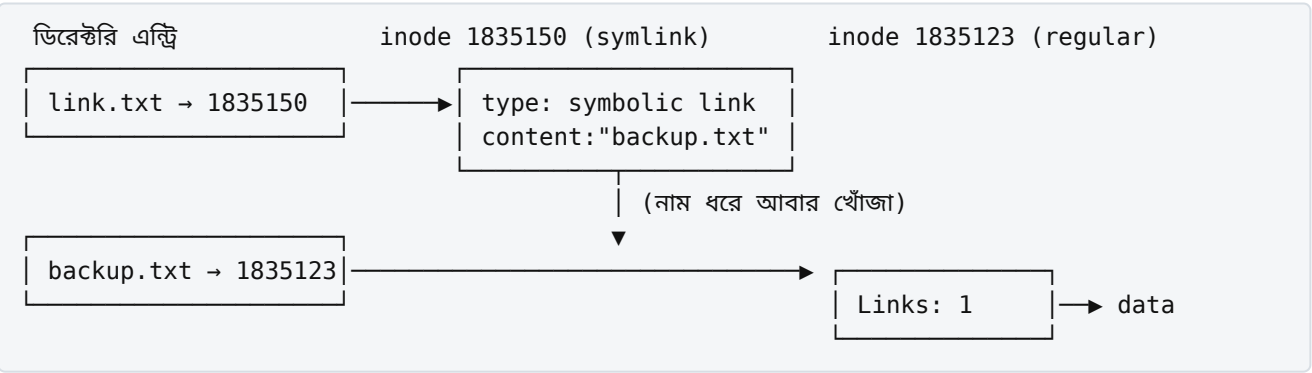
> ⚠️ **সতর্কতা:** কিছু টুল ফাইল সেভ করার সময় **নতুন inode** তৈরি করে পুরনোটার জায়গায় rename করে বসায়। যেমন `sed -i` এবং অনেক এডিটরের "safe save"। এতে হার্ড লিংক ভেঙে যায়: তুমি যে নাম দিয়ে এডিট করেছ সেটা নতুন inode পায়, আর অন্য নামটা পুরনো কনটেন্ট নিয়ে পড়ে থাকে। `echo >>` বা `>`-এর মতো in-place লেখা হার্ড লিংক রক্ষা করে।

```
$ ls -i backup.txt copy2.txt
1835123 backup.txt  1835123 copy2.txt
$ sed -i 's/Hello/Hi/' backup.txt
$ ls -i backup.txt copy2.txt
1835140 backup.txt  1835123 copy2.txt      # লিংক ভেঙে গেছে!
```

১১.৭ সিম্বলিক লিংক / সফট লিংক (Symbolic Link)

ধারণা

সিম্বলিক লিংক (symlink বা soft link) একটা **আলাদা ফাইল**। এর নিজের inode আছে, আর এর কনটেন্ট শুধু একটা **path লেখা**। কার্নেল এই ফাইলের মধ্য দিয়ে যাওয়ার সময় সেই লেখা path-টা অনুসরণ করে। Windows-এর "shortcut"-এর মতো, তবে অনেক বেশি স্বচ্ছভাবে কাজ করে: প্রায় সব প্রোগ্রাম এটাকে আসল ফাইলের মতোই ব্যবহার করে।



খেয়াল করো: symlink টার্গেটের inode-কে চেনে না, শুধু **নামটা** জানে। এজন্য টার্গেটের link count বাড়ে না।

তৈরি করা: `ln -s`

```
ln -s TARGET LINK_NAME
```

```
$ ln -s backup.txt link.txt
$ ls -li
total 8
1835123 -rw-r--r-- 2 rahim rahim 23 Nov 12 10:20 backup.txt
1835123 -rw-r--r-- 2 rahim rahim 23 Nov 12 10:20 copy2.txt
1835150 lrwxrwxrwx 1 rahim rahim 10 Nov 12 10:30 link.txt -> backup.txt
```

আউটপুটটা পড়ো:

- প্রথম অক্ষর `l` মানে সিম্বলিক লিংক
- পারমিশন সবসময় `rwxrwxrwx`। লিনাক্স symlink-এর নিজের পারমিশন **উপেক্ষা করে**, আসল পারমিশন চেক হয় টার্গেটের।
- আকার **10**, কারণ `backup.txt` লেখাটায় 10টা অক্ষর!
- `-> backup.txt` দেখায় কোথায় নির্দেশ করছে
- টার্গেটের link count এখনো 2 (`backup.txt` + `copy2.txt`), symlink-এর জন্য বাড়েনি

> 💡 **টিপস:** ext4-এ টার্গেট path 60 বাইটের কম হলে সেটা আলাদা data block-এ না রেখে inode-এর ভেতরেই রাখা হয়। একে **fast symlink** বলে। তাই ছোট symlink-এর `stat`-এ `Blocks: 0` দেখতে পারো।

```
$ cat link.txt
Hello Linux
added line
```

ডিরেক্টরির সিম্বলিক লিংক

হার্ড লিংকে যা নিষেধ, symlink-এ তা চলে:

```
$ ln -s /var/log logs
$ ls -l logs
lrwxrwxrwx 1 rahim rahim 8 Nov 12 10:32 logs -> /var/log
$ cd logs && pwd
/home/rahim/inode-lab/logs
$ pwd -P
/var/log
```

`pwd` তোমাকে "লজিক্যাল" path দেখায় (যে পথে এসেছ), আর `pwd -P` দেখায় "ফিজিক্যাল" আসল path (অধ্যায় ৮)।

Absolute বনাম Relative symlink

এটা অনেক নতুনদের বিভ্রান্ত করে, তাই মন দিয়ে পড়ো:

> 📝 **মনে রাখো:** relative symlink-এর টার্গেট path হিসাব হয় **লিংকটা যে ডিরেক্টরিতে আছে সেখান থেকে**, তুমি এখন যেখানে দাঁড়িয়ে আছ সেখান থেকে নয়।

```
$ cd ~
$ ln -s inode-lab/backup.txt inode-lab/wrong-link
```

```
$ cat inode-lab/wrong-link
cat: inode-lab/wrong-link: No such file or directory
```

কেন ভাঙল? লিংকটা আছে `~/inode-lab/`-এ, আর তার ভেতরে লেখা `inode-lab/backup.txt`। কার্নেল খুঁজছে `~/inode-lab/inode-lab/backup.txt`, যেটা নেই!

সমাধান তিনটা:

```
$ ln -sf backup.txt inode-lab/wrong-link              # টার্গেট লিংকের ডিরেক্টরি থেকে হিসাব করে লেখো
$ ln -sf ~/inode-lab/backup.txt inode-lab/wrong-link  # absolute path দাও
$ ln -srf inode-lab/backup.txt inode-lab/wrong-link   # -r দিলে ln নিজেই relative বানিয়ে নেয়
```

ধরন	উদাহরণ	সুবিধা	অসুবিধা
Absolute	`-> /home/rahim/data/file`	লিংক যেখানেই সরাও, কাজ করবে	টার্গেটের ফোল্ডার অন্য জায়গায় সরালে বা অন্য মেশিনে কপি করলে ভাঙবে
Relative	`-> ../data/file`	পুরো প্রজেক্ট ফোল্ডার একসাথে সরালে/কপি করলে কাজ করে	শুধু লিংকটা আলাদা করে সরালে ভাঙবে

> 💡 **টিপস:** প্রজেক্ট, git রিপো বা ব্যাকআপের ভেতরের লিংকে relative path ব্যবহার করো। সিস্টেম-ব্যাপী জায়গা দেখাতে (যেমন `/opt/app/current`) absolute ভালো।

symlink-এর উপর কমান্ডের আচরণ

কোন কমান্ড লিংক "অনুসরণ" (dereference) করে আর কোনটা লিংককে নিজেকে নিয়েই কাজ করে, এটা জানা জরুরি:

কমান্ড	ডিফল্ট আচরণ	উল্টোটা করতে
`cat`, `less`, `vim`, `cd`	টার্গেট অনুসরণ করে	(প্রযোজ্য নয়)
`ls -l`	লিংক নিজেকে দেখায়	`ls -lL` (টার্গেট দেখাও)
`stat`	লিংক নিজেকে দেখায়	`stat -L`
`rm`	**লিংক** মোছে, টার্গেট নয়	(প্রযোজ্য নয়)
`chmod`	**টার্গেটের** পারমিশন বদলায়	লিংকের পারমিশন বদলানোর অর্থ নেই
`chown`	টার্গেটের মালিক বদলায়	`chown -h` দিলে লিংকের মালিক বদলায়
`cp` (একটা ফাইল)	টার্গেটের কনটেন্ট কপি করে	`cp -P` দিলে লিংকটাই কপি করে
`cp -r` / `cp -a`	ভেতরের লিংককে লিংক হিসেবেই কপি করে	`cp -rL` দিলে অনুসরণ করে
`find`	লিংক অনুসরণ করে না	`find -L`

কমান্ড	ডিফল্ট আচরণ	উল্টোটা করতে
`tar`	লিংককে লিংক হিসেবে রাখে	`tar -h` (`--dereference`)

symlink-এর লুপ

symlink ডিরেক্টরি দেখাতে পারে, তাই লুপ তৈরি সম্ভব:

```
$ ln -s loopB loopA
$ ln -s loopA loopB
$ cat loopA
cat: loopA: Too many levels of symbolic links
```

কার্নেল একটা path resolve করতে সর্বোচ্চ 40টা symlink অনুসরণ করে, তারপর `ELOOP` এরর দেয়। এভাবে অনন্ত লুপ থেকে সিস্টেম রক্ষা পায়।

১১.৮ হার্ড লিংক বনাম সফট লিংক: পার্থক্যের টেবিল

বৈশিষ্ট্য	হার্ড লিংক	সিম্বলিক (সফট) লিংক
কমান্ড	`ln target name`	`ln -s target name`
আসলে কী	একই inode-এর আরেকটা নাম	আলাদা ফাইল, যার কনটেন্ট একটা path
নিজের inode	না, টার্গেটের সাথে **একই**	হ্যাঁ, **আলাদা**
`ls -l`-এ চেনার উপায়	সাধারণ ফাইলের মতো, link count > 1	`l` টাইপ, `-> target`
টার্গেটের link count বাড়ে?	হ্যাঁ	না
ডিরেক্টরির লিংক	না	হ্যাঁ
ভিন্ন ফাইলসিস্টেম/পার্টিশন	না	হ্যাঁ
টার্গেট মুছলে	ডেটা টিকে থাকে	লিংক **ভেঙে** যায় (dangling)
টার্গেট rename/move করলে	কোনো সমস্যা নেই	লিংক ভেঙে যায়
অস্তিত্বহীন জিনিসের দিকে বানানো	সম্ভব নয়	সম্ভব
আকার	টার্গেটের সমান (একই inode)	টার্গেট path-এর অক্ষরসংখ্যা
পারমিশন	টার্গেটের সাথে এক (একই inode)	সবসময় `rwxrwxrwx`, উপেক্ষিত
গতি	একটু দ্রুত (সরাসরি inode)	একটা অতিরিক্ত lookup লাগে
সাধারণ ব্যবহার	ডিডুপ্লিকেশন, স্ন্যাপশট ব্যাকআপ	শর্টকাট, ভার্সন সুইচ, কনফিগ, লাইব্রেরি

> 💡 **টিপস:** দৈনন্দিন কাজে ৯৫% সময় তোমার দরকার হবে **সফট লিংক**। হার্ড লিংক বিশেষ কাজের জন্য, যেমন `rsync --link-dest` দিয়ে জায়গা-বাঁচানো ব্যাকআপ (অধ্যায় ৩৫)।

১১.৯ ভাঙা লিংক (Broken / Dangling Link)

symlink-এর টার্গেট মুছে বা সরিয়ে ফেললে লিংকটা কোথাও নির্দেশ করে না। একে **broken** বা **dangling** লিংক বলে।

```
$ echo data > target.txt
$ ln -s target.txt pointer
$ rm target.txt
$ ls -l pointer
lrwxrwxrwx 1 rahim rahim 10 Nov 12 10:40 pointer -> target.txt
$ cat pointer
cat: pointer: No such file or directory
```

মজার ব্যাপার হলো `ls` বলছে `pointer` আছে, কিন্তু `cat` বলছে "No such file"। কারণ লিংকটা আছে, তার টার্গেট নেই। রঙিন টার্মিনালে `ls` সাধারণত ভাঙা লিংক **লাল** (কখনো জ্বলা-নেভা) রঙে দেখায়, আর ভালো লিংক সায়ান/আকাশি রঙে।

চেনার উপায়

```
$ file pointer
pointer: broken symbolic link to target.txt

$ test -e pointer && echo "ok" || echo "broken or missing"
broken or missing
$ test -L pointer && echo "it is a symlink"
it is a symlink
```

`test -e` (বা `[ -e ]`) লিংক অনুসরণ করে, আর `test -L` (বা `-h`) শুধু দেখে লিংকটা আছে কিনা। দুটো মিলিয়ে বোঝা যায় লিংক ভাঙা কিনা। এই টেস্টগুলো শেল স্ক্রিপ্টিংয়ে কাজে লাগবে (অধ্যায় ৪১)।

সব ভাঙা লিংক খুঁজে বের করা

```
$ find ~/inode-lab -xtype l
/home/rahim/inode-lab/pointer

$ find -L ~/inode-lab -type l          # একই কাজের আরেক উপায়
```

`-xtype l` মানে "লিংক অনুসরণ করার পরেও যেটা এখনো লিংক", অর্থাৎ যার টার্গেট খুঁজে পাওয়া যায়নি।

ঠিক করা বা মোছা

```
$ ln -sfn /new/place/target.txt pointer     # নতুন টার্গেটে নির্দেশ করাও
$ find ~/inode-lab -xtype l -delete          # সব ভাঙা লিংক মুছে দাও (আগে -delete ছাড়া চালিয়ে দেখে নাও!)
```

> 💡 **টিপস:** ভাঙা লিংক সবসময় ভুল নয়। কিছু প্রোগ্রাম ইচ্ছাকৃতভাবে বানায়, যেমন Firefox প্রোফাইলের `lock` ফাইল একটা symlink যার "টার্গেট" আসলে IP আর PID লেখা একটা স্ট্রিং। মোছার আগে দেখে নাও।

১১.১০ `readlink` ও `realpath`: লিংকের গন্তব্য বের করা

`readlink`

Syntax:

```
readlink [OPTION]... FILE...
```

অপশন ছাড়া চালালে symlink-এর ভেতরে **যা লেখা আছে ঠিক তাই** দেখায়, এক ধাপ, কোনো resolve ছাড়া:

```
$ ln -s backup.txt link.txt
$ ln -s link.txt link2.txt
$ readlink link2.txt
link.txt
$ readlink -f link2.txt
/home/rahim/inode-lab/backup.txt
```

অপশন	কাজ
(কিছু না)	এক ধাপ: লিংকের ভেতরের লেখা দেখাও। ফাইলটা লিংক না হলে কিছু না দেখিয়ে exit code 1 দেয়
`-f`, `--canonicalize`	সব লিংক অনুসরণ করে পুরো absolute path দাও; **শেষ অংশটা** ছাড়া বাকি সব path-এর অস্তিত্ব থাকতে হবে
`-e`, `--canonicalize-existing`	`-f`-এর মতো, তবে **সব** অংশের অস্তিত্ব থাকতে হবে
`-m`, `--canonicalize-missing`	`-f`-এর মতো, তবে কোনো অংশের অস্তিত্ব না থাকলেও চলবে
`-n`, `--no-newline`	শেষে newline দিও না
`-v`, `--verbose`	এরর মেসেজ দেখাও

`realpath`

`realpath` হলো `readlink -f`-এর আধুনিক ও বেশি সুবিধার সংস্করণ:

```
$ realpath link2.txt
/home/rahim/inode-lab/backup.txt
$ realpath -s link2.txt                          # symlink resolve কোরো না, শুধু path পরিষ্কার করো
```

```
/home/rahim/inode-lab/link2.txt
$ realpath --relative-to=/home/rahim /var/log
../../var/log
```

বাস্তব উদাহরণ: সিস্টেমে কোন `python3` আসলে চলে?

```
$ which python3
/usr/bin/python3
$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Apr  2  2024 /usr/bin/python3 -> python3.12
$ readlink -f "$(which python3)"
/usr/bin/python3.12
```

> 💡 **টিপস:** স্ক্রিপ্টের ভেতরে স্ক্রিপ্ট নিজে কোন ফোল্ডারে আছে জানতে এই কৌশলটা খুব প্রচলিত: `SCRIPT_DIR="$(dirname "$(realpath "$0")")"`। স্ক্রিপ্টটা symlink দিয়ে চালালেও এটা সঠিক উত্তর দেয়।

`namei` কমান্ড path-এর প্রতিটা ধাপ ভেঙে দেখায়, তাই কোথায় লিংক আছে বোঝা সহজ হয় (`util-linux` প্যাকেজে আছে):

```
$ namei /usr/bin/python3
f: /usr/bin/python3
 d /
 d usr
 d bin
 l python3 -> python3.12
   - python3.12
```

`namei -l` দিলে প্রতিটা ধাপের পারমিশনও দেখায়, যেটা "Permission denied" কোথা থেকে আসছে খুঁজতে দারুণ কাজে লাগে।

১১.১১ ফাইল মুছলে আসলে কী হয়?

`rm` = unlink

লিনাক্সে "ফাইল মোছা" বলে আলাদা কোনো system call নেই! `rm` আসলে `unlink()` (বা `unlinkat()`) system call চালায়। এটা শুধু:

1. ডিরেক্টরির টেবিল থেকে **নামটা** সরিয়ে দেয়
2. inode-এর **link count এক কমায়**

```
rm notes.txt
     |
     ▼
```

```
ডিরেক্টরি থেকে "notes.txt" এন্ট্রি মোছো
      |
      ▼
inode link count -= 1
      |
      ▼
link count == 0 ? ──No──▶ শেষ (অন্য নাম দিয়ে ফাইল এখনো আছে)
      |Yes
      ▼
কোনো প্রসেস কি ফাইলটা খোলা রেখেছে? ──Yes──▶ অপেক্ষা: শেষ প্রসেস close করলে মুক্ত হবে
      |No
      ▼
inode ও data block "ফ্রি" হিসেবে চিহ্নিত করো
```

📝 **মনে রাখো:** একটা ফাইলের ডেটা মুক্ত হয় তখনই, যখন **দুটো** শর্ত পূরণ হয়: (১) link count = 0, এবং (২) কোনো প্রসেস ফাইলটা খোলা রাখেনি।

⚠️ **সতর্কতা:** `rm` ডেটা "শূন্য" দিয়ে মুছে দেয় না, শুধু ব্লকগুলো ফ্রি চিহ্নিত করে। নতুন ডেটা ওপরে লেখা না হওয়া পর্যন্ত বিশেষ টুল দিয়ে উদ্ধার সম্ভব হতে পারে। আবার উল্টোদিকে, লিনাক্সে কোনো **Recycle Bin নেই**। `rm` করা ফাইল সাধারণ উপায়ে ফিরে আসে না। (GUI ফাইল ম্যানেজারের "Trash" আলাদা ব্যাপার: সেটা আসলে `~/.local/share/Trash`-এ `mv` করে।)

`unlink` নামে একটা কমান্ডও আছে, যেটা ঠিক একটা ফাইলের জন্য এই system call চালায়:

```
$ unlink pointer
```

মুছে ফেলা কিন্তু খোলা ফাইল: "ডিস্ক ভরা, কিন্তু ফাইল তো নেই!"

সার্ভারে এটা খুব পরিচিত সমস্যা। একটা 20 GB লগ ফাইল `rm` করলে, কিন্তু `df -h` এখনো বলছে ডিস্ক ভরা। কারণ যে প্রসেস লগটা লিখছিল (যেমন nginx বা কোনো অ্যাপ), সে এখনো ফাইলটা খোলা রেখেছে। link count 0, কিন্তু open file descriptor আছে, তাই ডেটা মুক্ত হয়নি।

হাতে-কলমে দেখো:

```
$ dd if=/dev/zero of=big.bin bs=1M count=200 status=none
$ df -h . | tail -1
/dev/sda2       234G   80G  142G  37% /
$ tail -f big.bin > /dev/null &
[1] 45210
$ rm big.bin
$ df -h . | tail -1
/dev/sda2       234G   80G  142G  37% /       ← জায়গা ফেরত আসেনি!
```

খুঁজে বের করো কোন প্রসেস মুছে ফেলা ফাইল ধরে আছে:

```
$ lsof +L1
COMMAND   PID  USER   FD   TYPE DEVICE  SIZE/OFF NLINK    NODE NAME
tail    45210 rahim    3r   REG    8,2 209715200     0 1835160 /home/rahim/inode-
lab/big.bin (deleted)
```

`+L1` মানে "link count 1-এর কম", অর্থাৎ 0। `NLINK 0` আর `(deleted)` দেখো। (`lsof` ইনস্টল না থাকলে `sudo apt install lsof` বা `sudo dnf install lsof`।)

`/proc` দিয়েও দেখা যায় (অধ্যায় ৪৭):

```
$ ls -l /proc/45210/fd
lr-x------ 1 rahim rahim 64 Nov 12 10:50 3 -> '/home/rahim/inode-lab/big.bin (deleted)'
```

সমাধানের তিনটা উপায়:

```
$ kill 45210                       # ১. প্রসেস বন্ধ করো (বা সার্ভিস restart: অধ্যায় ২৬)
$ : > /proc/45210/fd/3             # ২. প্রসেস না থামিয়ে ফাইলটা truncate করো (খালি করো)
$ cp /proc/45210/fd/3 ~/rescued.bin   # ৩. ভুল করে মুছেছ? এখনো উদ্ধার করা যায়!
```

> 💡 **টিপস:** ভবিষ্যতে বড় লগ খালি করতে `rm` না করে `truncate -s 0 app.log` বা `: > app.log` ব্যবহার করো। এতে inode একই থাকে, প্রসেস লিখে যেতে পারে, আর জায়গা সাথে সাথে ফেরত আসে। লগ রোটেশনের জন্য `logrotate` আছে (অধ্যায় ২৮)।

এই আচরণ কেন ভালো?

এটা আসলে লিনাক্সের একটা শক্তিশালী ডিজাইন। প্যাকেজ আপডেট করার সময় চলমান প্রোগ্রামের binary ফাইল বদলানো হয়। নতুন ফাইল নতুন inode পায় আর পুরনো নাম তার জায়গা নেয়। পুরনো চলমান প্রসেস তার পুরনো inode নিয়ে নির্বিঘ্নে চলতে থাকে, ক্র্যাশ করে না। এজন্যই লিনাক্সে বেশিরভাগ আপডেটের পর রিবুট লাগে না (শুধু সার্ভিস restart করলেই নতুন ভার্সন চালু হয়)। Windows-এ চলমান ফাইল সাধারণত লক থাকে, তাই সেখানে প্রায়ই রিবুট লাগে।

অদ্ভুত নামের ফাইল inode দিয়ে মোছা

কখনো কখনো এমন ফাইলের নাম পাবে যা টাইপ করা কঠিন (যেমন `-rf` নামে একটা ফাইল, বা অদৃশ্য অক্ষরসহ নাম)। inode নম্বর দিয়ে মুছে ফেলো:

```
$ ls -li
1835170 -rw-r--r-- 1 rahim rahim 0 Nov 12 11:00 ?????
$ find . -maxdepth 1 -inum 1835170 -delete
```

(বিকল্প: `rm -- '-rf'` বা `rm ./-rf`।)

১১.১২ inode শেষ হয়ে যাওয়া: `df -i`

সমস্যাটা কী

ext4-এ inode-এর মোট সংখ্যা **ফাইলসিস্টেম তৈরির (`mkfs`) সময়েই নির্দিষ্ট হয়ে যায়**। ডিফল্টে প্রতি 16 KB জায়গার জন্য একটা inode বানানো হয়। লক্ষ লক্ষ ছোট ফাইল (সেশন ফাইল, ক্যাশ, মেইল কিউ, ছোট থাম্বনেইল) থাকলে ডিস্কে প্রচুর জায়গা খালি থাকা সত্ত্বেও **inode শেষ হয়ে যেতে পারে**। তখন নতুন ফাইল তৈরি করা যায় না:

```
$ touch newfile
touch: cannot touch 'newfile': No space left on device
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       234G   61G  162G  28% /          ← জায়গা তো আছে!?
```

উপমা: একটা পার্কিং লটে প্রচুর খালি জমি আছে, কিন্তু **পার্কিং টিকিট** শেষ। টিকিট ছাড়া কেউ গাড়ি ঢোকাতে পারবে না।

`df -i` দিয়ে দেখা

```
$ df -i
Filesystem       Inodes   IUsed    IFree IUse% Mounted on
/dev/sda2      15630336 15630336        0  100% /
tmpfs           2041216    1205  2040011    1% /run
/dev/sda1             0       0        0     - /boot/efi
```

কলাম	মানে
`Inodes`	মোট inode
`IUsed`	ব্যবহৃত
`IFree`	বাকি
`IUse%`	ব্যবহারের শতাংশ

> 📝 **মনে রাখো:** "No space left on device" এলে সবসময় **দুটোই** চেক করো: `df -h` (জায়গা) আর `df -i` (inode)।

`/boot/efi`-তে `0` আর `-` দেখাচ্ছে কারণ FAT32-এ inode ধারণাটাই নেই। আর XFS বা btrfs-এ inode প্রয়োজন অনুযায়ী তৈরি হয় (dynamic), তাই সেখানে এই সমস্যা প্রায় হয় না। RHEL/Fedora সার্ভারে ডিফল্ট ফাইলসিস্টেম XFS, আর Fedora Workstation-এ btrfs। Ubuntu/Debian ডিফল্টে ext4 ব্যবহার করে।

কোথায় এত ফাইল, খুঁজে বের করো

```
$ sudo du --inodes -x / 2>/dev/null | sort -n | tail -5
182311  /usr
```

```
240155  /var/lib
1210887 /var/spool/postfix/maildrop
1215643 /var/spool
15630011 /
```

`du --inodes` জায়গার বদলে inode (ফাইল সংখ্যা) গোনে, আর `-x` অন্য ফাইলসিস্টেমে ঢোকে না। আরেকটা পুরনো কিন্তু কাজের উপায়:

```
$ sudo find /var -xdev -type f | cut -d/ -f2,3 | sort | uniq -c | sort -n | tail
```

সমাধান

1. অপ্রয়োজনীয় ছোট ফাইল মুছে ফেলো (পুরনো সেশন, ক্যাশ, আটকে থাকা মেইল কিউ)। অনেক ফাইল একসাথে মুছতে `find DIR -type f -delete` ব্যবহার করো, কারণ `rm *` দিলে "Argument list too long" আসতে পারে।
2. সমস্যার মূল ঠিক করো: কোন প্রোগ্রাম এত ফাইল বানাচ্ছে? (cron job, PHP session cleanup ইত্যাদি)
3. নতুন ফাইলসিস্টেম বানানোর সময় বেশি inode রাখো:

```
$ sudo mkfs.ext4 -i 4096 /dev/sdb1       # প্রতি 4 KB-তে একটা inode (বেশি inode)
$ sudo mkfs.ext4 -N 50000000 /dev/sdb1   # সরাসরি মোট inode সংখ্যা বলে দাও
$ sudo mkfs.ext4 -T small /dev/sdb1      # ছোট ফাইলের জন্য প্রিসেট
```

> ⚠️ **সতর্কতা:** `mkfs` চালালে পার্টিশনের **সব ডেটা মুছে যায়**। ext4-এ ইতিমধ্যে তৈরি ফাইলসিস্টেমের inode সংখ্যা পরে বাড়ানো যায় না। ডিস্ক নিয়ে বিস্তারিত অধ্যায় ৩২-এ।

ext4-এ inode-এর তথ্য দেখতে `tune2fs` ব্যবহার করা যায়:

```
$ sudo tune2fs -l /dev/sda2 | grep -i inode
Inode count:              15630336
Free inodes:              14812001
Inodes per group:         8192
Inode size:               256
...
```

> 💡 **টিপস (অ্যাডভান্সড):** `debugfs` দিয়ে ext4-এর ভেতরে সরাসরি উঁকি দেওয়া যায়, যেমন `sudo debugfs -R 'stat <1835123>' /dev/sda2`। এটা শুধু শেখার জন্য read-only ভাবে ব্যবহার করো। ভুল কমান্ডে ফাইলসিস্টেম নষ্ট হতে পারে।

১১.১৩ বাস্তব জীবনে লিংকের ব্যবহার

১. `/bin` → `/usr/bin` (usrmerge)

আধুনিক Ubuntu, Debian 12, Fedora ও RHEL 9-এ পুরনো ডিরেক্টরিগুলো আসলে symlink:

```
$ ls -ld /bin /sbin /lib /lib64
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
lrwxrwxrwx 1 root root 8 Apr 22  2024 /sbin -> usr/sbin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /lib -> usr/lib
lrwxrwxrwx 1 root root 9 Apr 22  2024 /lib64 -> usr/lib64
```

এতে পুরনো স্ক্রিপ্টে লেখা `#!/bin/bash` এখনো কাজ করে, অথচ সব binary এক জায়গায় (`/usr/bin`) থাকে। FHS নিয়ে বিস্তারিত আছে অধ্যায় ৭-এ।

২. `/etc/alternatives`: একই কাজের একাধিক প্রোগ্রাম

সিস্টেমে হয়তো `nano`, `vim` দুটোই ইনস্টল আছে। `editor` কমান্ড চালালে কোনটা খুলবে? Debian/Ubuntu এটা দুই স্তরের symlink দিয়ে সমাধান করে:

```
$ ls -l /usr/bin/editor
lrwxrwxrwx 1 root root 24 Apr 22  2024 /usr/bin/editor -> /etc/alternatives/editor
$ ls -l /etc/alternatives/editor
lrwxrwxrwx 1 root root 9 Apr 22  2024 /etc/alternatives/editor -> /bin/nano
```

```
/usr/bin/editor ──▶ /etc/alternatives/editor ──▶ /bin/nano
   (স্থির নাম)          (সুইচবোর্ড)                (আসল প্রোগ্রাম)
                                       └── বদলে ──▶ /usr/bin/vim.basic
```

কেন দুই স্তর? `/usr/bin`-এর ফাইল প্যাকেজ ম্যানেজারের এলাকা, আর `/etc` হলো কনফিগারেশনের এলাকা। অ্যাডমিন শুধু `/etc/alternatives`-এর লিংক বদলায়।

এটা হাতে না বদলে `update-alternatives` টুল ব্যবহার করো:

```
$ update-alternatives --display editor
editor - auto mode
  link best version is /bin/nano
  link currently points to /bin/nano
  link editor is /usr/bin/editor
/bin/nano - priority 40
/usr/bin/vim.basic - priority 30

$ sudo update-alternatives --config editor
There are 2 choices for the alternative editor (providing /usr/bin/editor).

  Selection    Path                Priority   Status
------------------------------------------------------------
* 0            /bin/nano            40        auto mode
  1            /bin/nano            40        manual mode
```

```
  2            /usr/bin/vim.basic   30        manual mode

Press <enter> to keep the current choice[*], or type selection number: 2
```

কমান্ড	কাজ
`update-alternatives --display NAME`	বর্তমান অবস্থা ও সব বিকল্প দেখাও
`update-alternatives --list NAME`	শুধু বিকল্পগুলোর path দেখাও
`sudo update-alternatives --config NAME`	মেনু থেকে বেছে নাও
`sudo update-alternatives --set NAME PATH`	মেনু ছাড়াই সরাসরি সেট করো (স্ক্রিপ্টের জন্য)
`sudo update-alternatives --auto NAME`	আবার স্বয়ংক্রিয় (সর্বোচ্চ priority) মোডে ফেরো
`sudo update-alternatives --install LINK NAME PATH PRIORITY`	নতুন বিকল্প যোগ করো

> 💡 **টিপস:** Fedora/RHEL-এ একই সিস্টেম আছে, তবে কমান্ডের নাম `alternatives` (যেমন `sudo alternatives --config java`)। Java, Python বা `editor`-এর মতো জায়গায় এটা খুব কাজে লাগে।

৩. ভার্সনযুক্ত শেয়ার্ড লাইব্রেরি (`.so` ফাইল)

```
$ ls -l /usr/lib/x86_64-linux-gnu/libz.so*
lrwxrwxrwx 1 root root     13 ... libz.so -> libz.so.1.3
lrwxrwxrwx 1 root root     13 ... libz.so.1 -> libz.so.1.3
-rw-r--r-- 1 root root 113000 ... libz.so.1.3
```

(সঠিক ভার্সন নম্বর তোমার distro-তে আলাদা হতে পারে। আর `libz.so` লিংকটা শুধু `-dev` / `-devel` প্যাকেজ ইনস্টল থাকলে থাকে।)

তিন স্তরের নামকরণ:

```
libz.so          ⟶ libz.so.1     ⟶ libz.so.1.3
(linker name)       (soname)         (real name: আসল ফাইল)
compile-এর সময়     চলার সময়        আসল কোড
gcc -lz খোঁজে      প্রোগ্রাম খোঁজে
```

- **real name** (`libz.so.1.3`): আসল লাইব্রেরি ফাইল, পুরো ভার্সন সহ
- **soname** (`libz.so.1`): major ভার্সন। একই major ভার্সনের ভেতরে ABI সামঞ্জস্যপূর্ণ থাকে। কম্পাইল করা প্রোগ্রামের ভেতরে এই নামটাই লেখা থাকে।
- **linker name** (`libz.so`): শুধু কম্পাইল করার সময় `gcc -lz` এটা খোঁজে

কেন এভাবে? zlib-এ সিকিউরিটি প্যাচ এলে `libz.so.1.3.1` ইনস্টল হয় আর `libz.so.1` লিংকটা নতুন ফাইলে নির্দেশ করে। তোমার কোনো প্রোগ্রাম আবার কম্পাইল করতে হয় না, সব প্রোগ্রাম সাথে সাথে প্যাচড লাইব্রেরি

পায়। আর একসাথে `libfoo.so.1` ও `libfoo.so.2` থাকতে পারে, তাই পুরনো ও নতুন দুই ধরনের প্রোগ্রামই চলে।

soname লিংকগুলো `ldconfig` স্বয়ংক্রিয়ভাবে বানায়/আপডেট করে, আর `ldd` দেখায় কোনো প্রোগ্রাম কোন লাইব্রেরি ব্যবহার করে:

```
$ ldd /usr/bin/gzip
    linux-vdso.so.1 (0x00007ffd...)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f...)
    /lib64/ld-linux-x86-64.so.2 (0x00007f...)
```

সোর্স থেকে বিল্ড করার সময় এগুলো আবার আসবে (অধ্যায় ৩১)।

৪. ডিপ্লয়মেন্টে `current` লিংক (zero-downtime switch)

ওয়েব অ্যাপ ডিপ্লয়ের একটা জনপ্রিয় কৌশল:

```
/var/www/app/
├── releases/
│   ├── 2026-11-10/
│   ├── 2026-11-11/
│   └── 2026-11-12/      ← নতুন রিলিজ
└── current -> releases/2026-11-11     (ওয়েব সার্ভার এখানে দেখে)
```

নতুন রিলিজ প্রস্তুত হলে শুধু লিংকটা ঘুরিয়ে দাও। সমস্যা হলে আগেরটায় ফিরিয়ে দাও (rollback), কয়েক মিলিসেকেন্ডে।

```
$ ln -sfn releases/2026-11-12 /var/www/app/current
```

`-n` এখানে জরুরি। এটা না দিলে `current` যেহেতু একটা ডিরেক্টরির দিকে লিংক, `ln` নতুন লিংকটা **`current`-এর ভেতরে** (মানে পুরনো রিলিজ ফোল্ডারের ভেতরে) বানিয়ে ফেলবে!

> 💡 **টিপস (অ্যাডভান্সড):** `ln -sfn` ভেতরে আগে পুরনো লিংক মোছে, তারপর নতুন বানায়। এই দুই ধাপের মাঝে খুব অল্প সময়ের জন্য `current` থাকে না। পুরোপুরি atomic সুইচ চাইলে এভাবে করো (rename system call atomic):
>
> ```
> ln -s releases/2026-11-12 /var/www/app/current.tmp
> mv -T /var/www/app/current.tmp /var/www/app/current
> ```

৫. আরও কিছু জায়গা যেখানে লিংক দেখবে

জায়গা	কী
`/etc/systemd/system/*.wants/`	`systemctl enable` আসলে এখানে symlink বানায় (অধ্যায় ২৬)
`/etc/nginx/sites-enabled/`	`sites-available`-এর ফাইলের দিকে symlink (Debian/Ubuntu) (অধ্যায় ৪০)
`/etc/localtime`	`/usr/share/zoneinfo/Asia/Dhaka`-এর দিকে symlink
`/dev/disk/by-uuid/`	আসল ডিভাইস (`../../sda2`)-এর দিকে symlink
`/proc/self`	যে প্রসেস পড়ছে তার নিজের `/proc/PID`-এর দিকে symlink
`/etc/resolv.conf`	Ubuntu-তে `../run/systemd/resolve/stub-resolv.conf`-এর দিকে symlink
`~/.bashrc` ইত্যাদি dotfiles	অনেকে git রিপো থেকে symlink করে সব মেশিনে একই সেটআপ রাখে (অধ্যায় ৪৪)
হার্ড লিংক ব্যাকআপ	`rsync --link-dest` আর `cp -al`: অপরিবর্তিত ফাইল আবার কপি না করে হার্ড লিংক করে, তাই প্রতিটা স্ন্যাপশট "পূর্ণ" দেখায় কিন্তু জায়গা প্রায় নেয় না

১১.১৪ কীবোর্ড শর্টকাট

এই টপিকের কাজ মূলত কমান্ড দিয়ে, তবে কিছু শর্টকাট সময় বাঁচায়:

শর্টকাট	কোথায়	কাজ
`Tab`	টার্মিনাল	ফাইল/লিংকের নাম auto-complete (লিংক ডিরেক্টরি হলে শেষে `/` যোগ হয়)
`Alt + .`	Bash	আগের কমান্ডের শেষ argument বসাও (যেমন `ln -s target link` চালিয়ে তারপর `ls -l` + `Alt+.`)
`Ctrl + M`	Nautilus (GNOME Files)	নির্বাচিত ফাইলের লিংক বানাও (Make Link)
`Ctrl + Shift` + ড্র্যাগ	Nautilus	ছেড়ে দেওয়ার জায়গায় লিংক তৈরি হবে
`Ctrl + X`, তারপর `L`	Midnight Commander (`mc`)	হার্ড লিংক বানাও
`Ctrl + X`, তারপর `S`	Midnight Commander	সিম্বলিক লিংক বানাও
`Ctrl + X`, তারপর `Ctrl + S`	Midnight Commander	বিদ্যমান symlink এডিট করো

টার্মিনালের অন্য সব শর্টকাট পাবে অধ্যায় ৬ আর পরিশিষ্ট ক-এ।

হাতে-কলমে অনুশীলন

একটা নিরাপদ খেলার জায়গা বানিয়ে নাও। সব কাজ এর ভেতরে করো:

```
$ mkdir -p ~/link-lab && cd ~/link-lab
```

ল্যাব ১: inode আর নাম আলাদা

```
$ echo "original" > a.txt
$ ls -i a.txt                       # inode নম্বর লিখে রাখো
$ mv a.txt b.txt
$ ls -i b.txt                       # নম্বর কি বদলাল? (না)
$ cp b.txt c.txt
$ ls -i b.txt c.txt                 # cp নতুন inode বানায়
```

প্রশ্ন: `mv` আর `cp`-এর পর inode নম্বরে কী পার্থক্য দেখলে, কেন?

ল্যাব ২: হার্ড লিংক ও link count

```
$ ln b.txt hard1
$ ln b.txt hard2
$ stat -c '%n %i %h' b.txt hard1 hard2
$ echo "changed" >> hard2
$ cat b.txt                         # পরিবর্তন দেখা যাচ্ছে?
$ rm b.txt hard1
$ stat -c '%n %i %h' hard2          # link count এখন কত?
$ cat hard2                         # ডেটা আছে তো?
```

ল্যাব ৩: সিম্বলিক লিংক, relative বনাম absolute

```
$ mkdir -p data && echo "secret" > data/info.txt
$ ln -s data/info.txt rel-link
$ ln -s "$PWD/data/info.txt" abs-link
$ ls -l rel-link abs-link
$ mkdir moved && mv rel-link abs-link moved/
$ cat moved/rel-link                # ভাঙল কেন?
$ cat moved/abs-link                # কাজ করল কেন?
$ ln -srf data/info.txt moved/rel-link
$ cat moved/rel-link                # এখন ঠিক
$ readlink moved/rel-link           # ../data/info.txt
```

ল্যাব ৪: ভাঙা লিংক শিকার

```
$ for i in 1 2 3; do echo $i > f$i; ln -s f$i l$i; done
$ rm f2
$ find . -xtype l                    # ./l2 দেখাবে
$ file l2
$ find . -xtype l -delete
$ ls -l
```

ল্যাব ৫: মুছে ফেলা কিন্তু খোলা ফাইল

```
$ echo "important data" > keep.txt
$ sleep 600 < keep.txt &             # একটা প্রসেস ফাইলটা খোলা রাখল
$ PID=$!
$ rm keep.txt
$ ls -l /proc/$PID/fd/0              # (deleted) দেখো
$ cat /proc/$PID/fd/0                # ডেটা এখনো পড়া যাচ্ছে!
$ cp /proc/$PID/fd/0 recovered.txt
$ kill $PID
$ cat recovered.txt
```

ল্যাব ৬: inode শেষ করার পরীক্ষা (নিরাপদভাবে, RAM-এ)

`tmpfs`-এ একটা ছোট ফাইলসিস্টেম বানিয়ে ইচ্ছে করে inode শেষ করো। এতে তোমার আসল ডিস্কের কোনো ক্ষতি হবে না:

```
$ mkdir -p ~/tinyfs
$ sudo mount -t tmpfs -o size=10M,nr_inodes=100 tmpfs ~/tinyfs
$ sudo chown "$USER" ~/tinyfs
$ df -i ~/tinyfs
$ for i in $(seq 1 200); do touch ~/tinyfs/f$i || break; done
touch: cannot touch '/home/rahim/tinyfs/f100': No space left on device
$ df -h ~/tinyfs                     # জায়গা প্রায় পুরোটা খালি!
$ df -i ~/tinyfs                     # কিন্তু IUse% 100%
$ sudo umount ~/tinyfs
```

(যে ফাইলে গিয়ে থামবে তার নম্বর একটু আলাদা হতে পারে, কারণ root ডিরেক্টরি নিজেও একটা inode নেয়।)

ল্যাব ৭: alternatives অনুসন্ধান (Debian/Ubuntu)

```
$ ls -l /etc/alternatives | head
$ readlink -f /usr/bin/editor
$ update-alternatives --display editor
$ ls -l /usr/lib/x86_64-linux-gnu/libc.so*
```

ল্যাব ৮: atomic ডিপ্লয় সিমুলেশন

```
$ mkdir -p app/releases/v1 app/releases/v2
$ echo "v1" > app/releases/v1/index.html
$ echo "v2" > app/releases/v2/index.html
$ ln -s releases/v1 app/current
$ cat app/current/index.html                        # v1
$ ln -s releases/v2 app/current.tmp && mv -T app/current.tmp app/current
$ cat app/current/index.html                        # v2
$ ln -sfn releases/v1 app/current                   # rollback
$ cat app/current/index.html                        # v1
```

শেষে পরিষ্কার করো:

```
$ cd ~ && rm -rf ~/link-lab ~/tinyfs
```

সারসংক্ষেপ

কমান্ড / শর্টকাট	কাজ
`ls -i` / `ls -li`	inode নম্বর দেখাও
`stat FILE`	inode-এর পুরো তথ্য
`stat -c '%i %h %n' FILE`	নির্দিষ্ট তথ্য (inode, link count, নাম)
`stat -L LINK`	লিংকের টার্গেটের তথ্য
`stat -f /`	ফাইলসিস্টেমের তথ্য (inode সহ)
`ln TARGET NAME`	হার্ড লিংক বানাও
`ln -s TARGET NAME`	সিম্বলিক লিংক বানাও
`ln -sr TARGET NAME`	relative সিম্বলিক লিংক
`ln -sfn TARGET NAME`	বিদ্যমান লিংক (ডিরেক্টরির দিকেও) বদলাও
`mv -T tmp link`	atomic ভাবে লিংক প্রতিস্থাপন
`readlink LINK`	লিংকের ভেতরের লেখা (এক ধাপ)
`readlink -f` / `realpath`	চূড়ান্ত absolute path
`namei -l PATH`	path-এর প্রতিটা ধাপ ও পারমিশন
`find . -samefile F` / `-inum N`	একই inode-এর সব নাম
`find . -xtype l`	ভাঙা লিংক খোঁজো
`file LINK`	লিংক ভাঙা কিনা বলে
`rm LINK` / `unlink LINK`	লিংক মোছো (টার্গেট নয়, শেষে `/` দিও না)
`lsof +L1`	মুছে ফেলা কিন্তু খোলা ফাইল

কমান্ড / শর্টকাট	কাজ
`: > FILE` / `truncate -s 0 FILE`	inode না বদলে ফাইল খালি করো
`df -i`	inode ব্যবহার দেখো
`du --inodes -x DIR`	কোথায় বেশি ফাইল
`update-alternatives --config NAME`	বিকল্প প্রোগ্রাম বাছাই (Fedora/RHEL: `alternatives`)
`ldd PROGRAM`	প্রোগ্রাম কোন লাইব্রেরি ব্যবহার করে
`Ctrl+M` (Nautilus)	GUI-তে লিংক বানাও
`Ctrl+X L` / `Ctrl+X S` (mc)	হার্ড / সিম্বলিক লিংক

> ⚠️ **সতর্কতা:** ডিরেক্টরির দিকে নির্দেশ করা symlink মোছার সময় কখনো শেষে `/` দিও না। `rm mylink` ঠিক আছে। কিন্তু `rm -rf mylink/` লিংকের বদলে **টার্গেট ডিরেক্টরির ভেতরের সবকিছু** মুছে দিতে পারে!

নিজেকে যাচাই করো

1. inode-এ কোন কোন তথ্য থাকে? কোন গুরুত্বপূর্ণ তথ্যটা থাকে **না**, আর সেটা কোথায় থাকে?
2. একই ফাইলসিস্টেমে 50 GB ফাইল `mv` করতে এক মুহূর্ত লাগে, কিন্তু অন্য ডিস্কে সরাতে অনেক সময় লাগে। কেন?
3. একটা নতুন খালি ডিরেক্টরির link count 2 কেন? তার ভেতরে 3টা সাব-ডিরেক্টরি বানালে কত হবে (ext4-এ)?
4. হার্ড লিংক দিয়ে কোন দুটো কাজ করা যায় না, আর কেন?
5. `ln -s ../config/app.conf settings` চালানো হলো `/home/rahim/project/bin/` ডিরেক্টরিতে দাঁড়িয়ে, আর লিংকটাও সেখানেই তৈরি হলো। লিংকটা কোন absolute path-এ নির্দেশ করবে?
6. `readlink link` আর `readlink -f link`-এর পার্থক্য কী?
7. একটা সার্ভারে 30 GB লগ ফাইল `rm` করার পরও `df -h` জায়গা ফেরত দেখাচ্ছে না। কারণ কী, আর কীভাবে খুঁজে বের করবে ও ঠিক করবে?
8. `touch` বলছে "No space left on device", অথচ `df -h` দেখাচ্ছে 60% খালি। প্রথমে কোন কমান্ড চালাবে, আর সমস্যাটা কী হতে পারে?
9. `libssl.so.3` কী ধরনের নাম, আর লাইব্রেরি এভাবে symlink দিয়ে সাজানোর সুবিধা কী?
10. symlink-এর পারমিশন সবসময় `lrwxrwxrwx` দেখায়। এর মানে কি যে কেউ লিংক দিয়ে টার্গেট ফাইল এডিট করতে পারবে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. inode-এ থাকে ফাইলের ধরন, পারমিশন, মালিক (UID), গ্রুপ (GID), আকার, link count, টাইমস্ট্যাম্প (atime, mtime, ctime, birth time) এবং ডেটা ব্লকের ঠিকানা। **ফাইলের নাম থাকে না।** নাম থাকে প্যারেন্ট ডিরেক্টরির টেবিলে (name → inode number)।
</details>

2. একই ফাইলসিস্টেমে `mv` শুধু ডিরেক্টরি এন্ট্রি বদলায় (rename)। inode আর ডেটা নড়ে না। অন্য ফাইলসিস্টেমে inode নম্বরের কোনো অর্থ নেই, তাই সেখানে পুরো ডেটা কপি করে নতুন inode বানাতে হয়, তারপর পুরনোটা মুছতে হয়।

3. কারণ দুটো নাম এটাকে দেখায়: প্যারেন্টে থাকা এর নাম, আর এর নিজের ভেতরের `.`। প্রতিটা সাব-ডিরেক্টরির `..` আরেকটা করে যোগ করে, তাই 3টা সাব-ডিরেক্টরি থাকলে 2 + 3 = **5**।

4. (ক) ডিরেক্টরির হার্ড লিংক বানানো যায় না, কারণ এতে ট্রিতে লুপ তৈরি হতে পারে আর `..`-এর মানে অস্পষ্ট হয়ে যায়। (খ) ভিন্ন ফাইলসিস্টেমে হার্ড লিংক বানানো যায় না, কারণ inode নম্বর শুধু এক ফাইলসিস্টেমের ভেতরে ইউনিক ("Invalid cross-device link")।

5. relative টার্গেট হিসাব হয় লিংকের ডিরেক্টরি (`/home/rahim/project/bin/`) থেকে, তাই নির্দেশ করবে `/home/rahim/project/config/app.conf`-এ।

6. `readlink link` লিংকের ভেতরে যা লেখা আছে শুধু সেটাই দেখায় (এক ধাপ, relative হলে relative-ই)। `readlink -f link` সব লিংকের চেইন অনুসরণ করে চূড়ান্ত **absolute canonical path** দেখায় (`realpath`-এর মতো)।

7. কোনো প্রসেস (লগ লেখা সার্ভিস) এখনো ফাইলটা খোলা রেখেছে। link count 0 হলেও open file descriptor থাকায় ডেটা মুক্ত হয়নি। খুঁজতে `sudo lsof +L1` চালাও বা `/proc/*/fd`-এ `(deleted)` খোঁজো। ঠিক করতে সার্ভিস restart করো, অথবা `: > /proc/PID/fd/N` দিয়ে truncate করো। ভবিষ্যতে `rm`-এর বদলে `truncate -s 0` ব্যবহার করো।

8. `df -i`। সম্ভবত inode শেষ হয়ে গেছে (IUse% 100%), কারণ লক্ষ লক্ষ ছোট ফাইল আছে। `du --inodes -x /` দিয়ে কোথায় সেগুলো খুঁজে বের করো আর অপ্রয়োজনীয়গুলো মুছে ফেলো।

9. এটা **soname**: লাইব্রেরির নাম আর তার major ভার্সন (3)। কম্পাইল করা প্রোগ্রামের ভেতরে এই নামটাই লেখা থাকে। বেশিরভাগ লাইব্রেরিতে soname একটা symlink, যেটা পূর্ণ ভার্সনযুক্ত আসল ফাইলের দিকে নির্দেশ করে, আর `ldconfig` এই লিংক সাজায়। (কিছু লাইব্রেরিতে, যেমন অনেক distro-র OpenSSL 3-এ, আসল ফাইলের নামই soname, তখন আলাদা লিংক থাকে না।) সুবিধা: প্যাচ বা minor আপডেটে শুধু লিংক বদলালেই সব প্রোগ্রাম নতুন লাইব্রেরি পায়, আবার কম্পাইল করতে হয় না। আর ভিন্ন major ভার্সন পাশাপাশি থাকতে পারে।

10. না। লিনাক্স symlink-এর নিজের পারমিশন উপেক্ষা করে। লিংক দিয়ে ফাইল খুললে **টার্গেটের** পারমিশন চেক হয়। টার্গেট `rw-r--r--` আর মালিক root হলে সাধারণ ইউজার লিংক দিয়েও শুধু পড়তে পারবে, লিখতে পারবে না।

অধ্যায় ১২: ফাইল ও কমান্ড খোঁজা

> "কম্পিউটারে ফাইল হারায় না, শুধু আমরা ভুলে যাই কোথায় রেখেছি।" এই অধ্যায় শেষ করলে কোনো ফাইল বা কমান্ড খুঁজে পেতে আর বেশি সময় লাগবে না।

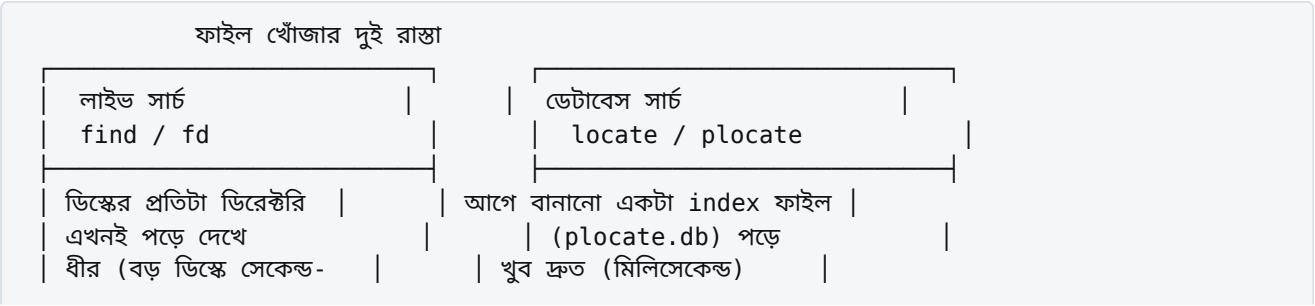
এই অধ্যায়ে যা শিখবে

- লিনাক্সে "খোঁজা" কেন দুই রকম: **লাইভ সার্চ** (`find`, `fd`) আর **ডেটাবেস সার্চ** (`locate` / `plocate`)
- `find` কমান্ড শুরু থেকে শেষ পর্যন্ত: `-name`, `-iname`, `-type`, `-size`, `-mtime`, `-mmin`, `-user`, `-perm`, `-maxdepth`, `-empty` ইত্যাদি
- `find`-এর ভেতরের কাজ: directory tree কীভাবে ঘোরে আর expression কীভাবে বাম থেকে ডানে মূল্যায়ন হয়
- লজিক্যাল অপারেটর (`-a`, `-o`, `!`, `\( \)`) আর `-prune`
- খুঁজে পাওয়া ফাইলের ওপর কাজ চালানো: `-exec {} \;`, `-exec {} +`, `-execdir`, `-ok`, `-delete`
- নিরাপদ পাইপলাইন: `-print0` আর `xargs -0`
- `locate` / `plocate` আর `updatedb`: কেন এত দ্রুত, আর কখন ভুল ফল দেয়
- কমান্ড খোঁজা: `which`, `whereis`, `type`, `command -v`, `hash`, `apropos`
- আধুনিক বিকল্প `fd`
- রোজকার কাজে লাগে এমন অনেকগুলো রেসিপি

১২.১ খোঁজার দুই পদ্ধতি: লাইভ বনাম ডেটাবেস

একটা লাইব্রেরিতে বই খোঁজার কথা ভাবো। দুইভাবে খোঁজা যায়:

1. **প্রতিটা তাক ঘুরে দেখা**: সময় লাগে, কিন্তু যা পাবে তা এই মুহূর্তে তাকে আছে। এটা `find`-এর পদ্ধতি।
2. **ক্যাটালগ খাতা দেখা**: মুহূর্তে উত্তর পাবে, কিন্তু খাতাটা গত রাতে লেখা। আজ সকালে যে বই এসেছে সেটা খাতায় নেই, আর যে বই আজ কেউ নিয়ে গেছে সেটা খাতায় এখনও আছে। এটা `locate`-এর পদ্ধতি।



```
| মিনিট)                          |      |                                  |
| সবসময় হালনাগাদ                 |      | শেষ updatedb পর্যন্ত হালনাগাদ|
| নাম, সাইজ, সময়, মালিক,         |      | শুধু নাম/পাথ দিয়ে খোঁজে      |
| পারমিশন সব দিয়ে খোঁজা যায়      |      |                                  |
└──────────────────────────────────┘      └──────────────────────────────────┘
```

কমান্ড খোঁজা (যেমন `ls` প্রোগ্রামটা ডিস্কে কোথায় আছে) আলাদা জিনিস, সেটা শেষের দিকে `which`, `type` ইত্যাদি দিয়ে দেখব।

> 📝 **মনে রাখো:** ফাইলের *নাম* দিয়ে খুঁজতে চাইলে `find` বা `locate`। ফাইলের *ভেতরের লেখা* দিয়ে খুঁজতে চাইলে `grep -r`, যেটা অধ্যায় ২০-এ আছে।

১২.২ `find` পরিচিতি

`find` লিনাক্সের সবচেয়ে শক্তিশালী খোঁজার টুল। এটা GNU findutils প্যাকেজের অংশ, তাই সব বড় distro-তে (Ubuntu, Debian, Fedora, RHEL) আগে থেকেই ইনস্টল থাকে।

সিনট্যাক্স

```
find [অপশন] [কোথা থেকে শুরু...] [expression]
```

- **কোথা থেকে শুরু (starting point)**: এক বা একাধিক ডিরেক্টরি। না দিলে বর্তমান ডিরেক্টরি (`.`) ধরে নেয়।
- **expression**: কী খুঁজবে আর পেলে কী করবে। এর ভেতরে তিন রকম জিনিস থাকে:
  - **test** (পরীক্ষা): `-name`, `-type`, `-size` ইত্যাদি। প্রতিটা ফাইলের জন্য true বা false দেয়।
  - **action** (কাজ): `-print`, `-delete`, `-exec` ইত্যাদি।
  - **option**: `-maxdepth`, `-mindepth`, `-xdev` ইত্যাদি, যা পুরো সার্চের আচরণ বদলায়।
  - **operator**: `-a`, `-o`, `!`, `\( \)`, যা test আর action জোড়া লাগায়।

সবচেয়ে সহজ ব্যবহার:

```
$ find
.
./notes.txt
./project
./project/main.c
./project/README.md
```

কোনো test বা action না দিলে `find` সব ফাইল দেখায়, কারণ default action হলো `-print`।

ভেতরে কীভাবে কাজ করে (under the hood)

`find` আসলে একটা **depth-first tree walker**। প্রতিটা ডিরেক্টরিতে ঢুকে kernel-কে system call দিয়ে জিজ্ঞেস করে "এখানে কী কী আছে?" (`getdents64`), তারপর দরকার হলে প্রতিটা ফাইলের metadata চায় (`lstat` / `statx`): সাইজ, সময়, মালিক, পারমিশন। এই metadata আসে ফাইলের inode থেকে (অধ্যায় ১১)।

```
find /home/rahim -name "*.txt"

/home/rahim                          ← শুরু
├── docs/        ──► ভেতরে ঢোকো (depth-first)
│   ├── a.txt    ──► test: -name "*.txt" → TRUE  → -print
│   └── b.pdf    ──► test → FALSE → বাদ
├── music/       ──► ভেতরে ঢোকো
│   └── song.mp3 ──► FALSE
└── todo.txt     ──► TRUE → -print

প্রতিটা ফাইলের জন্য expression বাম → ডান মূল্যায়ন হয়।
কোনো test FALSE হলে বাকি অংশ আর চেক হয় না (short-circuit)।
```

short-circuit মানে: `find . -name "*.log" -size +10M` লিখলে আগে নাম মেলানো হয়। নাম না মিললে সাইজ আর দেখা হয় না। নাম মেলানোর জন্য শুধু ডিরেক্টরি এন্ট্রি পড়লেই চলে, কিন্তু সাইজ জানতে `stat` লাগে। তাই সস্তা test আগে রাখলে `find` দ্রুত চলে।

> 💡 **টিপস:** বড় ডিরেক্টরিতে `find` চালানোর সময় `-name` / `-type` আগে আর `-size` / `-mtime` / `-exec` পরে রাখো।

"Permission denied" নিয়ন্ত্রণ

সাধারণ ইউজার হিসেবে `/` থেকে খুঁজলে অনেক ডিরেক্টরিতে ঢোকার অনুমতি থাকে না:

```
$ find / -name "sshd_config"
find: '/root': Permission denied
find: '/proc/1/task/1/fd': Permission denied
...
/etc/ssh/sshd_config
```

এরর মেসেজগুলো stderr-এ যায়, তাই ফেলে দেওয়া যায় (অধ্যায় ১৯):

```
$ find / -name "sshd_config" 2>/dev/null
/etc/ssh/sshd_config
```

অথবা `sudo find ...` দিয়ে সব জায়গায় খুঁজতে পারো।

১২.৩ নাম দিয়ে খোঁজা: `-name`, `-iname`, `-path`, `-regex`

`-name` আর `-iname`

```
$ find ~/Documents -name "report.pdf"
/home/rahim/Documents/2024/report.pdf

$ find ~/Documents -name "*.pdf"
/home/rahim/Documents/2024/report.pdf
/home/rahim/Documents/cv.pdf

$ find ~/Documents -iname "*.PDF"      # বড়-ছোট হাতের অক্ষর উপেক্ষা
/home/rahim/Documents/2024/report.pdf
/home/rahim/Documents/cv.pdf
/home/rahim/Documents/SCAN.PDF
```

`-name` shell-এর মতো wildcard (glob) বোঝে:

প্যাটার্ন	মানে	উদাহরণ
`*`	যেকোনো সংখ্যক অক্ষর (শূন্যও)	`"*.txt"`
`?`	ঠিক একটা অক্ষর	`"file?.log"` → `file1.log`
`[abc]`	এর যেকোনো একটা	`"[Rr]eadme*"`
`[0-9]`	রেঞ্জ	`"img[0-9][0-9].jpg"`
`[!a]`	a ছাড়া যেকোনো	`"[!.]*"`

> ⚠️ **সতর্কতা: প্যাটার্ন সবসময় কোটেশনে রাখো।** `find . -name *.txt` লিখলে shell আগেই `*.txt`-কে বর্তমান ডিরেক্টরির ফাইলের নাম দিয়ে বদলে দেয় (globbing)। বর্তমান ডিরেক্টরিতে `a.txt` আর `b.txt` থাকলে কমান্ডটা হয়ে যায় `find . -name a.txt b.txt`, ফলে `find: paths must precede expression` এরর আসে, বা ভুল ফল আসে। সঠিক: `find . -name "*.txt"`।

📝 `-name` শুধু ফাইলের **শেষ অংশ** (basename) মেলায়, পুরো পাথ না। `-name "docs/*.txt"` কখনো মিলবে না।

`-path` আর `-ipath`: পুরো পাথ মেলানো

```
$ find . -path "*/docs/*.txt"
./project/docs/intro.txt
./old/docs/notes.txt
```

এখানে `*` ঘোরাফেরা করে `/`-সহ পুরো পাথে। (`-wholename` হলো `-path`-এর পুরোনো নাম।)

`-regex` আর `-iregex`

রেগুলার এক্সপ্রেশন দিয়ে **পুরো পাথ** মেলায় (অধ্যায় ২০)। GNU find-এর default regex টাইপ emacs-style; extended regex চাইলে `-regextype posix-extended` দাও:

```
$ find . -regextype posix-extended -regex '.*/img_[0-9]{4}\.(jpg|png)'
./photos/img_2024.jpg
./photos/img_0001.png
```

> 💡 regex পুরো পাথের সাথে মেলে, তাই শুরুতে `.*/` দিতে ভুলো না।

১২.৪ ফাইলের ধরন দিয়ে: `-type`

লিনাক্সে "সব কিছুই ফাইল", কিন্তু ফাইলের ধরন আলাদা আলাদা।

অক্ষর	ধরন	উদাহরণ
`f`	সাধারণ ফাইল (regular file)	`/etc/hostname`
`d`	ডিরেক্টরি	`/home`
`l`	সিম্বলিক লিংক	`/bin` (অনেক distro-তে `/usr/bin`-এর লিংক)
`b`	block device	`/dev/sda`
`c`	character device	`/dev/null`
`p`	named pipe (FIFO)	
`s`	socket	`/run/systemd/notify`

```
$ find ~/project -type d
/home/rahim/project
/home/rahim/project/src
/home/rahim/project/docs

$ find /etc -maxdepth 1 -type l
/etc/localtime
/etc/os-release
/etc/mtab
```

GNU find-এ একসাথে একাধিক টাইপ কমা দিয়ে দেওয়া যায়: `-type f,l`।

ভাঙা লিংক খোঁজা: `-xtype l` লিংক অনুসরণ করে দেখে, গন্তব্যও যদি লিংকের মতো দেখায় (মানে গন্তব্য নেই) তবে true।

```
$ find ~ -xtype l
```

```
/home/rahim/.config/old-link
```

১২.৫ সাইজ দিয়ে: `-size` আর `-empty`

```
find . -size [+|-]n[একক]
```

একক	মানে
`c`	বাইট
`k`	কিবিবাইট (1024 বাইট)
`M`	মেবিবাইট (1024 k)
`G`	গিবিবাইট (1024 M)
`b`	512-বাইটের ব্লক (কোনো একক না দিলে এটাই default)
`w`	2-বাইটের word

লেখা	মানে
`-size +100M`	100 MiB-এর **বেশি**
`-size -10k`	10 KiB-এর **কম**
`-size 4k`	ঠিক 4 KiB (রাউন্ড করার পরে)

```
$ find ~ -type f -size +500M 2>/dev/null
/home/rahim/Downloads/ubuntu-24.04-desktop-amd64.iso
/home/rahim/Videos/wedding.mp4

$ find /var/log -type f -size +10M -size -100M     # 10M থেকে 100M-এর মাঝে
```

> ⚠️ **সতর্কতা: রাউন্ডিং-এর ফাঁদ।** `find` সাইজকে আগে একক অনুযায়ী **উপরের দিকে রাউন্ড** করে তারপর তুলনা করে। তাই `-size -1M` মানে "রাউন্ড করার পর 1 MiB-এর কম", অর্থাৎ শুধু 0 বাইটের ফাইল! 1 বাইটের ফাইলও রাউন্ড হয়ে 1M হয়ে যায়। 1 MiB-এর ছোট ফাইল চাইলে বাইটে লেখো: `-size -1048576c`, অথবা `-size -1024k`।

`-empty`

খালি ফাইল (0 বাইট) বা খালি ডিরেক্টরি খোঁজে:

```
$ find ~/project -empty
/home/rahim/project/build
/home/rahim/project/src/placeholder.txt
```

```
$ find ~/project -type d -empty      # শুধু খালি ডিরেক্টরি
/home/rahim/project/build
```

১২.৬ সময় দিয়ে: `-mtime`, `-mmin` ও আত্মীয়রা

প্রতিটা inode-এ তিনটা প্রধান সময় থাকে:

সময়	মানে	কখন বদলায়	find test
mtime (modify)	কনটেন্ট শেষ কবে বদলেছে	ফাইলে লিখলে	`-mtime`, `-mmin`
atime (access)	শেষ কবে পড়া হয়েছে	পড়লে (তবে নিচের নোট দেখো)	`-atime`, `-amin`
ctime (change)	inode/metadata শেষ কবে বদলেছে	chmod, chown, rename, লেখা	`-ctime`, `-cmin`

> 📝 **মনে রাখো:** ctime মানে "creation time" **না**। লিনাক্সের অনেক ফাইলসিস্টেম (ext4, btrfs, xfs) birth time রাখে, যা `stat` কমান্ডে `Birth:` হিসেবে দেখা যায়, আর GNU find-এ (ফাইলসিস্টেম সমর্থন করলে) `-newerBt` দিয়ে তুলনা করা যায়। কিন্তু `-ctime` সবসময় change time।
>
> আধুনিক distro-তে ফাইলসিস্টেম সাধারণত `relatime` অপশনে মাউন্ট হয়, তাই প্রতিবার পড়লেই atime বদলায় না। তাই `-atime`-এর ওপর পুরোপুরি ভরসা কোরো না।

সংখ্যার মানে: `+n`, `-n`, `n`

`-mtime` দিন হিসেবে কাজ করে (১ দিন = ২৪ ঘণ্টা), `-mmin` মিনিট হিসেবে।

```
এখন ◄──────────────────────────────────────── অতীত
    |  0 দিন  |  1 দিন  |  2 দিন  |  3 দিন  | ...
    | (0-24ঘ) |(24-48ঘ) |(48-72ঘ) |

-mtime 0   → শেষ 24 ঘণ্টার মধ্যে বদলেছে
-mtime 1   → 24 থেকে 48 ঘণ্টা আগে
-mtime -3  → 3 দিনের কম আগে (0, 1, 2 নম্বর ঘর)
-mtime +3  → 3 নম্বর ঘরের পরে, মানে কমপক্ষে 4×24 = 96 ঘণ্টা আগে
```

> ⚠️ **সতর্কতা:** `find` বয়সকে ২৪ ঘণ্টা দিয়ে ভাগ করে **ভগ্নাংশ ফেলে দেয়**। তাই `-mtime +1` মানে "২ দিন বা তার বেশি পুরোনো", "১ দিনের বেশি" না। ৩০ ঘণ্টা পুরোনো ফাইল `-mtime +1`-এ মিলবে না। নিখুঁত হিসাব চাইলে মিনিট ব্যবহার করো: `-mmin +1440`।

```
$ find ~ -type f -mmin -30           # শেষ ৩০ মিনিটে বদলানো ফাইল
/home/rahim/.bash_history
/home/rahim/project/main.c

$ find /tmp -type f -mtime +7        # ৭ দিনের বেশি (কমপক্ষে ৮ দিন) পুরোনো
```

```
/tmp/old-report.csv

$ find /var/log -name "*.log" -mtime -1    # আজকের (শেষ ২৪ ঘণ্টা) লগ
```

অন্য ফাইল বা তারিখের সাথে তুলনা: `-newer`, `-newermt`

```
$ touch -d "2026-11-01 00:00" /tmp/mark
$ find ~/project -newer /tmp/mark          # mark-এর পরে বদলানো
```

GNU find-এ সরাসরি তারিখ দেওয়া যায়:

```
$ find ~/project -type f -newermt "2026-11-01"
$ find ~/project -type f -newermt "2026-11-01" ! -newermt "2026-11-08"   # ১ থেকে ৭ নভেম্বর
$ find . -newermt "2 hours ago"
```

`-newerXY`-এ X আর Y হতে পারে `a` (atime), `m` (mtime), `c` (ctime), `B` (birth), আর Y-তে `t` মানে "লেখা তারিখ"।

`-daystart` অপশন দিলে সময় হিসাব হয় আজকের দিনের শুরু (রাত ১২টা) থেকে, ২৪ ঘণ্টা আগে থেকে না:

```
$ find . -daystart -mtime 0      # আজ (ক্যালেন্ডার দিন) বদলানো
```

১২.৭ মালিক ও গ্রুপ দিয়ে: `-user`, `-group`, `-nouser`

```
$ find /home -user rahim -name "*.sh"
$ find /var/www -group www-data -type f
$ find / -uid 1001 2>/dev/null       # UID দিয়ে
$ sudo find / -nouser -o -nogroup 2>/dev/null   # যে ফাইলের মালিক আর নেই
```

`-nouser` খুব কাজের: কোনো ইউজার ডিলিট করার পর তার ফাইলগুলো "অনাথ" হয়ে যায় (মালিকের জায়গায় শুধু সংখ্যা দেখায়)। ইউজার ও গ্রুপ নিয়ে বিস্তারিত অধ্যায় ১৫-এ।

১২.৮ পারমিশন দিয়ে: `-perm`

পারমিশন কী, তা অধ্যায় ১৬ আর অধ্যায় ১৭-এ আছে। এখানে শুধু খোঁজার নিয়ম। `-perm`-এর তিনটা রূপ আছে, আর এটাই অনেকে গুলিয়ে ফেলে:

রূপ	মানে	উপমা
`-perm 644`	পারমিশন **হুবহু** 644	চাবির খাঁজ হুবহু মিলতে হবে
`-perm -644`	644-এর **সবগুলো** বিট সেট আছে (বেশিও থাকতে পারে)	"অন্তত এগুলো তো থাকতেই হবে"

রূপ	মানে	উপমা
`-perm /022`	উল্লেখ করা বিটের **যেকোনো একটা** সেট আছে	"এর কোনো একটা থাকলেই হলো"

```
$ find ~ -type f -perm 777            # হুবহু rwxrwxrwx
$ find ~ -type f -perm -o=w           # others লিখতে পারে (symbolic রূপ)
$ find . -type f -perm /u=x,g=x,o=x   # কারো না কারো execute পারমিশন আছে
$ sudo find / -xdev -perm -4000 -type f 2>/dev/null   # SUID প্রোগ্রাম
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
...
```

> 📝 **মনে রাখো:** পুরোনো `-perm +mode` রূপ এখন আর চলে না; তার বদলে `/mode`।

শুধু তোমার জন্য অনুমতি পরীক্ষা করতে `-readable`, `-writable`, `-executable` আছে (এগুলো আসল access চেক করে, ACL-সহ):

```
$ find /etc -maxdepth 1 -writable    # তুমি লিখতে পারো এমন
```

১২.৯ গভীরতা নিয়ন্ত্রণ: `-maxdepth`, `-mindepth`, `-xdev`

```
 depth 0:  /etc
 depth 1:  /etc/ssh
 depth 2:  /etc/ssh/sshd_config
```

```
$ find /etc -maxdepth 1 -name "*.conf"      # শুধু /etc-এর সরাসরি ফাইল
/etc/host.conf
/etc/resolv.conf
/etc/sysctl.conf
...

$ find /etc -mindepth 2 -maxdepth 2 -name "*.conf"   # ঠিক এক লেভেল নিচে
```

`-xdev` (বা `-mount`) অন্য ফাইলসিস্টেমে ঢোকে না। `/` থেকে খুঁজলে এটা দিলে `/proc`, `/sys`, নেটওয়ার্ক মাউন্ট, USB ড্রাইভ বাদ পড়ে, ফলে সার্চ অনেক দ্রুত আর পরিষ্কার হয় (অধ্যায় ৩২)।

> ⚠️ **সতর্কতা:** `-maxdepth`, `-mindepth`, `-xdev` হলো global option, তাই এগুলো starting point-এর ঠিক পরে, অন্য test-এর **আগে** লেখো। পরে লিখলে GNU find warning দেয়: `find: warning: you have specified the global option -maxdepth after the argument -name...`।

সিম্বলিক লিংক অনুসরণ

default-এ `find` লিংকের ভেতরে ঢোকে না (`-P` আচরণ)। `-L` দিলে লিংক অনুসরণ করে; `-H` শুধু কমান্ড লাইনে দেওয়া পাথের লিংক অনুসরণ করে। এগুলো starting point-এর **আগে** বসে:

```
$ find -L /opt/app -name "*.jar"
```

১২.১০ লজিক্যাল অপারেটর

অপারেটর	বিকল্প লেখা	মানে
`expr1 expr2`	`expr1 -a expr2`, `-and`	AND (দুটোই সত্য) — কিছু না লিখলে এটাই
`expr1 -o expr2`	`-or`	OR (যেকোনো একটা)
`! expr`	`-not expr`	NOT (উল্টো)
`\( expr \)`		গ্রুপ করা (বন্ধনী)
`expr1 , expr2`		দুটোই চালাও, শেষেরটার ফল নাও

```
$ find . -name "*.jpg" -o -name "*.png"            # jpg অথবা png
$ find . -type f ! -name "*.txt"                   # txt বাদে সব ফাইল
$ find . -type f \( -name "*.jpg" -o -name "*.png" \) -size +1M
```

অগ্রাধিকার (precedence) এর ফাঁদ

গণিতে যেমন গুণ যোগের আগে হয়, তেমনি `find`-এ **AND, OR-এর আগে** হয়। ক্রম: `\( \)` > `!` > `-a` > `-o`।

```
$ find . -name "*.jpg" -o -name "*.png" -print
```

এটা আসলে মানে:

```
  ( -name "*.jpg" )  -o  ( -name "*.png" -a -print )
```

ফলে শুধু png ফাইলগুলো print হয়! jpg মিললেও print হয় না, কারণ `-print` শুধু ডান দিকের অংশের সাথে AND হয়েছে। সমাধান বন্ধনী:

```
$ find . \( -name "*.jpg" -o -name "*.png" \) -print
```

> 💡 **টিপস:** `-o` ব্যবহার করলে সবসময় বন্ধনী দাও। বন্ধনীর আগে `\` দিতে হয় (বা `'('` কোটেশনে), নাহলে shell সেটাকে নিজের subshell চিহ্ন ভাববে। বন্ধনীর দুপাশে স্পেস লাগবে।

> 📝 যখন expression-এ `-prune` ছাড়া আর কোনো action (`-print`, `-exec`, `-delete`, `-ls` ইত্যাদি) থাকে না, `find` পুরো expression-কে `\( ... \) -print` হিসেবে ধরে। তাই `find . -name "*.jpg" -o -name "*.png"` (কোনো `-print` ছাড়া) ঠিকঠাক দুটোই দেখায়।

`-prune`: কোনো ডিরেক্টরি পুরোপুরি বাদ দেওয়া

`! -path "./node_modules/*"` লিখলে ফলাফল থেকে বাদ যায়, কিন্তু `find` তবুও ভেতরে ঢুকে সব ফাইল দেখে (সময় নষ্ট)। `-prune` বলে "এই ডিরেক্টরির ভেতরে ঢুকবেই না":

```
$ find . -path ./node_modules -prune -o -name "*.js" -print
```

পড়ার নিয়ম: "পাথটা `./node_modules` হলে prune করো (ঢুকো না), **নাহলে** নাম `*.js` হলে print করো।"

একাধিক ডিরেক্টরি বাদ:

```
$ find . \( -name .git -o -name node_modules -o -name vendor \) -prune -o -type f -name
"*.php" -print
```

> ⚠️ **সতর্কতা:** `-prune`-এর সাথে শেষে স্পষ্ট `-print` দিতেই হবে, নাহলে বাদ দেওয়া ডিরেক্টরির নামটাও আউটপুটে চলে আসবে।

১২.১১ আউটপুট সাজানো: `-print`, `-ls`, `-printf`

action	কাজ
`-print`	পাথ, তারপর newline (default)
`-print0`	পাথ, তারপর NUL অক্ষর (`\0`) — নিরাপদ পাইপের জন্য
`-ls`	`ls -dils`-এর মতো বিস্তারিত
`-printf FORMAT`	নিজের মতো ফরম্যাট (GNU)
`-fprint FILE`	ফলাফল সরাসরি ফাইলে
`-quit`	প্রথমটা পেলেই থামো

```
$ find ~/project -name "*.c" -ls
  1835123   4 -rw-r--r--   1 rahim  rahim    1240 Nov 10 14:02
/home/rahim/project/main.c
```

`-printf`-এর দরকারি ফরম্যাট:

কোড	মানে
`%p`	পুরো পাথ
`%f`	শুধু ফাইলের নাম
`%h`	পাথের ডিরেক্টরি অংশ
`%s`	সাইজ (বাইট)
`%u` / `%g`	মালিক / গ্রুপের নাম
`%m`	পারমিশন (অক্টাল)
`%M`	পারমিশন (`-rw-r--r--` রূপে)
`%TY-%Tm-%Td %TH:%TM`	mtime তারিখ-সময়
`%T@`	mtime, epoch থেকে সেকেন্ড
`%d`	গভীরতা
`%i`	inode নম্বর
`\n`	নতুন লাইন

```
$ find ~/project -type f -printf "%s\t%p\n" | sort -n | tail -3
2048    /home/rahim/project/README.md
10240   /home/rahim/project/data.csv
524288  /home/rahim/project/logo.png

$ find . -maxdepth 1 -type f -printf "%M %u %TY-%Tm-%Td %f\n"
-rw-r--r-- rahim 2026-11-10 notes.txt
```

প্রথম মিলেই থামা (বড় ডিস্কে সময় বাঁচায়):

```
$ find / -name "nginx.conf" -print -quit 2>/dev/null
/etc/nginx/nginx.conf
```

১২.১২ খুঁজে পাওয়া ফাইলে কাজ চালানো: `-exec`

`find` শুধু খুঁজেই থামে না; পাওয়া প্রতিটা ফাইলের ওপর অন্য কমান্ড চালাতে পারে।

`-exec কমান্ড {} \;`

```
$ find . -name "*.txt" -exec wc -l {} \;
12 ./notes.txt
40 ./docs/intro.txt
```

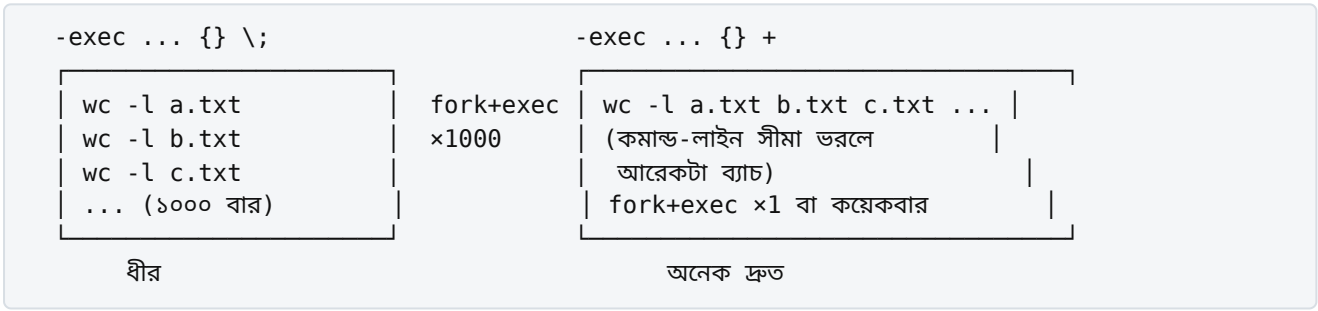
- `{}` জায়গায় বসে পাওয়া ফাইলের পাথ।
- `\;` বলে কমান্ড এখানে শেষ। `;` shell-এর বিশেষ অক্ষর, তাই `\` দিয়ে বা `';'` লিখে escape করতে হয়।

- **প্রতিটা ফাইলের জন্য একবার** কমান্ড চলে।

`-exec কমান্ড {} +`

```
$ find . -name "*.txt" -exec wc -l {} +
12 ./notes.txt
40 ./docs/intro.txt
52 total
```

এখানে `find` অনেকগুলো ফাইলের নাম জড়ো করে **একবারে** কমান্ড চালায় (`wc -l ./notes.txt ./docs/intro.txt`)। তাই আউটপুটে `total` এসেছে।



ভেতরের কারণ: প্রতিটা কমান্ড চালাতে kernel-কে নতুন process বানাতে হয় (`fork` + `exec`, অধ্যায় ২৪)। ১০,০০০ ফাইলে ১০,০০০টা process বনাম হাতে-গোনা কয়েকটা, পার্থক্য বিশাল।

> 📝 **মনে রাখো:** `+` রূপে `{}` অবশ্যই `+`-এর ঠিক আগে, শেষে থাকতে হবে। `-exec cp {} /backup/ +` চলবে না। এমন ক্ষেত্রে কমান্ডের "গন্তব্য আগে" অপশন ব্যবহার করো: `-exec cp -t /backup/ {} +` (GNU cp-এর `-t`)।

`-execdir`: নিরাপদ বিকল্প

`-execdir` কমান্ডটা চালায় ফাইলটা যে ডিরেক্টরিতে আছে **সেখানে গিয়ে**, আর `{}`-এ দেয় `./ফাইলের_নাম`। এতে race condition কমে, আর কিছু কমান্ডে (যেমন একই ফোল্ডারে rename) সুবিধা হয়:

```
$ find . -name "*.jpeg" -execdir mv {} {}.bak \;
```

`-ok`: জিজ্ঞেস করে চালাও

`-exec`-এর মতো, কিন্তু প্রতিবার নিশ্চিত হতে চায়:

```
$ find . -name "*.tmp" -ok rm {} \;
< rm ... ./a.tmp > ? y
< rm ... ./b.tmp > ? n
```

শিখতে থাকার সময় ঝুঁকিপূর্ণ কাজে `-ok` (বা `-okdir`) ব্যবহার করো।

`-exec` দিয়ে shell-এর জটিল কাজ

একটা কমান্ডে পাইপ বা একাধিক ধাপ লাগলে `sh -c` ব্যবহার করো:

```
$ find . -name "*.log" -exec sh -c 'gzip -9 "$1" && echo "compressed: $1"' _ {} \;
```

`_` হলো `$0`-এর জায়গা, আর `{}` যায় `$1`-এ। ফাইলের নাম কখনো সরাসরি `sh -c '... {} ...'`-এর স্ট্রিং-এর ভেতরে বসিও না: অদ্ভুত নামের ফাইল (`; rm -rf ~`) তখন কোড হিসেবে চলে যেতে পারে।

`-exec`-ও একটা test: কমান্ড exit status 0 দিলে true। তাই ফিল্টার হিসেবেও ব্যবহার করা যায়:

```
$ find . -name "*.txt" -exec grep -q "TODO" {} \; -print   # যে txt-তে TODO আছে
```

১২.১৩ `-delete`: খুঁজে মুছে ফেলা

```
$ find /tmp/myapp -type f -name "*.tmp" -delete
$ find ~/project -type d -empty -delete     # খালি ডিরেক্টরি মুছো
```

`-delete` নিজে থেকেই `-depth` চালু করে, মানে ডিরেক্টরির ভেতরের জিনিস আগে প্রসেস হয়, ডিরেক্টরি নিজে পরে। তাই খালি ডিরেক্টরির শিকল (a/b/c সব খালি) একবারেই মুছে যায়। তবে `-delete` খালি না এমন ডিরেক্টরি মুছতে পারে না।

> ⚠️ **সতর্কতা (খুব গুরুত্বপূর্ণ):**
>
> 1. `-delete` সবসময় **শেষে** রাখো। `find . -delete -name "*.tmp"` লিখলে বাম থেকে ডানে প্রথমেই `-delete` চলে, তাই **সব ফাইল** মুছে যায়, তারপর নাম চেক হয়!
> 2. আগে `-delete` ছাড়া (`-print` দিয়ে) চালিয়ে তালিকা দেখো, তারপর একই কমান্ডের শেষে `-delete` যোগ করো।
> 3. `-delete` দিয়ে মুছলে Trash-এ যায় না, ফেরত আনা যায় না।
> 4. `-delete` আর `-prune` একসাথে কাজ করে না (`-delete` `-depth` চালু করায় `-prune` অকার্যকর হয়)।

১২.১৪ `-print0` আর `xargs`

`xargs` কী করে

`xargs` stdin থেকে লাইন/শব্দ পড়ে সেগুলোকে অন্য কমান্ডের **আর্গুমেন্ট** বানিয়ে দেয়। অনেক কমান্ড (যেমন `rm`, `cp`) stdin পড়ে না, আর্গুমেন্ট চায়; `xargs` সেই সেতু।

```
  find ... ──(পাথের তালিকা)──► xargs ──► rm a b c d ...
            stdout → stdin              আর্গুমেন্ট বানায়
```

```
$ find . -name "*.bak" | xargs rm
```

সমস্যা: ফাঁকা জায়গাওয়ালা নাম

ধরো ফাইলের নাম `My Song.mp3`। সাধারণ `xargs` স্পেস দেখলে আলাদা শব্দ ভাবে, তাই `rm My` আর `rm Song.mp3` চালাতে চায়: ভুল ফাইল মুছতে পারে বা এরর দেয়। লিনাক্সে ফাইলের নামে newline-ও থাকতে পারে! একমাত্র যে অক্ষর পাথে থাকতে পারে না তা হলো **NUL (`\0`)**।

সমাধান: `find` NUL দিয়ে আলাদা করবে, `xargs` NUL দিয়ে পড়বে।

```
$ find . -name "*.mp3" -print0 | xargs -0 ls -l
-rw-r--r-- 1 rahim rahim 4123456 Nov  2 10:00 './My Song.mp3'
```

> 💡 **টিপস:** `find` থেকে `xargs`-এ পাঠালে **সবসময়** `-print0 | xargs -0` ব্যবহার করো। এটাকে অভ্যাস বানাও।

`xargs`-এর দরকারি অপশন

অপশন	কাজ
`-0`	ইনপুট NUL দিয়ে আলাদা
`-n N`	একবারে সর্বোচ্চ N টা আর্গুমেন্ট
`-I {}`	প্রতিটা ইনপুট আইটেমের জন্য একবার চালাও, `{}` জায়গায় বসাও
`-P N`	একসাথে N টা process সমান্তরালে (parallel) চালাও
`-r`	ইনপুট খালি হলে কমান্ড চালিও না (GNU-তে `--no-run-if-empty`)
`-t`	চালানোর আগে কমান্ডটা stderr-এ দেখাও
`-p`	প্রতিবার জিজ্ঞেস করো
`-d DELIM`	নিজের পছন্দের বিভাজক (GNU)

```
$ find . -name "*.png" -print0 | xargs -0 -r -P 4 -n 10 optipng -o2
# ৪টা CPU core-এ সমান্তরালে, প্রতিবার ১০টা করে ছবি

$ find . -name "*.txt" -print0 | xargs -0 -I {} cp {} /backup/
```

`-exec +` না `xargs`?

বিষয়	`-exec {} +`	`-print0 | xargs -0`
অদ্ভুত নামে নিরাপদ	হ্যাঁ	হ্যাঁ (`-0` দিলে)
ব্যাচিং	হ্যাঁ	হ্যাঁ
সমান্তরাল চালানো	না	হ্যাঁ (`-P`)
ব্যাচ সাইজ নিয়ন্ত্রণ	না	হ্যাঁ (`-n`)
বাড়তি process	না	হ্যাঁ (`xargs` নিজে)

সাধারণ কাজে `-exec ... {} +` সবচেয়ে সহজ। সমান্তরাল বা বিশেষ নিয়ন্ত্রণ দরকার হলে `xargs`। `xargs` নিয়ে আরও বিস্তারিত অধ্যায় ২৩-এ।

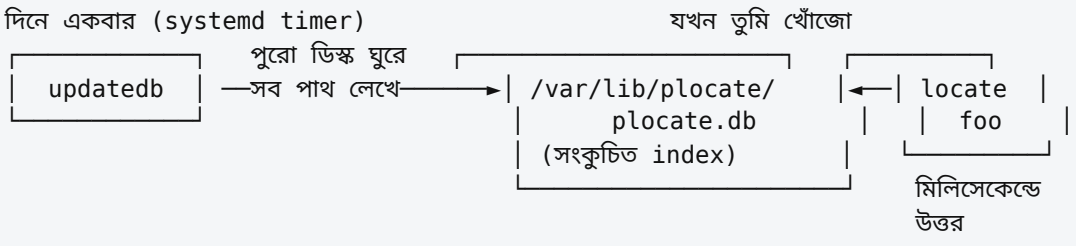
shell লুপ দিয়ে (নিরাপদ পদ্ধতি)

```
find . -name "*.jpg" -print0 | while IFS= read -r -d '' f; do
    echo "প্রসেস করছি: $f"
done
```

লুপ ও `read` অধ্যায় ৪২-এ বিস্তারিত।

১২.১৫ `locate` / `plocate` ও `updatedb`

কীভাবে কাজ করে



`updatedb` পুরো ফাইলসিস্টেম একবার ঘুরে সব পাথ একটা ডেটাবেসে লিখে রাখে। `locate` শুধু সেই ডেটাবেসে খোঁজে, তাই ডিস্ক ঘোরা লাগে না। `plocate` হলো আধুনিক সংস্করণ: সংকুচিত trigram index ব্যবহার করে, তাই লাখ লাখ ফাইলেও প্রায় সঙ্গে সঙ্গে উত্তর দেয়।

distro-ভেদে

Distro	প্যাকেজ	ইনস্টল
Ubuntu 24.04 / Debian 12	`plocate`	`sudo apt install plocate`
Fedora (৩৬ থেকে)	`plocate` (আগে `mlocate`)	`sudo dnf install plocate`
RHEL 9 / Rocky / Alma 9	`mlocate`	`sudo dnf install mlocate`

সব ক্ষেত্রে কমান্ডের নাম `locate` আর `updatedb` (plocate প্যাকেজ `plocate` নামেও কমান্ড দেয়)। ইনস্টলের পর ডেটাবেস আপডেট হয় systemd timer দিয়ে (Ubuntu/Debian/Fedora-তে `plocate-updatedb.timer`, RHEL 9-এর mlocate-এ `mlocate-updatedb.timer`), দিনে একবার। timer নিয়ে অধ্যায় ২৯।

ইনস্টলের ঠিক পরে ডেটাবেস থাকে না, তাই প্রথমবার হাতে চালাও:

```
$ sudo updatedb
$ locate sshd_config
/etc/ssh/sshd_config
/usr/share/doc/openssh-server/examples/sshd_config
```

দরকারি অপশন

অপশন	কাজ
`-i`	বড়-ছোট অক্ষর উপেক্ষা
`-c`	শুধু কয়টা মিলেছে সেই সংখ্যা
`-l N` / `-n N`	প্রথম N টা ফল
`-b`	শুধু basename-এ মেলাও (পুরো পাথে না)
`-r REGEX`	regex দিয়ে খোঁজো
`-0`	NUL দিয়ে আলাদা আউটপুট
`-S`	ডেটাবেসের পরিসংখ্যান (mlocate)

```
$ locate -i readme.md | head -3
$ locate -c "*.conf"
842
$ locate -b '\passwd'          # ঠিক "passwd" নামের ফাইল (\ দিলে * যোগ হয় না)
/etc/passwd
/usr/bin/passwd
$ locate -r '\.ya\?ml$' | head
```

📝 প্যাটার্নে wildcard না থাকলে `locate` ধরে নেয় `*প্যাটার্ন*`, মানে পাথের যেকোনো জায়গায় মিললেই হলো। তাই `locate passwd` অনেক ফল দেয়।

সীমাবদ্ধতা

- **হালনাগাদ না:** আজ বানানো ফাইল `sudo updatedb` না চালানো পর্যন্ত পাবে না। আর ডিলিট হওয়া ফাইল তালিকায় থেকে যেতে পারে।
- শুধু নাম/পাথ দিয়ে খোঁজে; সাইজ, সময়, মালিক দিয়ে খোঁজা যায় না।
- কিছু পাথ ইচ্ছা করে বাদ দেওয়া হয়: `/etc/updatedb.conf`-এ `PRUNEPATHS` (যেমন `/tmp`, `/var/spool`), `PRUNEFS` (যেমন `nfs`, `proc`), `PRUNENAMES` (যেমন `.git`)।
- **নিরাপত্তা:** ডেটাবেসে সব ফাইলের নাম থাকে, কিন্তু plocate/mlocate তোমাকে শুধু সেই ফাইলগুলোই দেখায় যেগুলোর ডিরেক্টরিতে তোমার অনুমতি আছে।

```
$ grep PRUNE /etc/updatedb.conf
PRUNE_BIND_MOUNTS="yes"
PRUNENAMES=".git .bzr .hg .svn"
PRUNEPATHS="/tmp /var/spool /media /var/lib/os-prober /var/lib/ceph /home/.ecryptfs
/var/lib/schroot"
PRUNEFS="NFS afs autofs binfmt_misc ceph cgroup ... proc ... tmpfs ..."
```

(তোমার সিস্টেমে মান কিছুটা আলাদা হতে পারে।)

> 💡 **কখন কোনটা:** "এই নামের কোনো ফাইল সিস্টেমে কোথায় আছে?" → `locate` (দ্রুত)। "গত এক ঘণ্টায় বদলানো 10MB-এর বড় ফাইল" → `find`।

১২.১৬ কমান্ড খোঁজা: প্রোগ্রামটা আসলে কোথায়?

তুমি টার্মিনালে `ls` লিখলে shell কীভাবে জানে কোন প্রোগ্রাম চালাবে? shell একটা নির্দিষ্ট ক্রমে খোঁজে:

```
তুমি লিখলে:  ls
      │
      ▼
1. alias?          (alias ls='ls --color=auto')   ──হ্যাঁ──► সেটা বিস্তার করো
      │না
      ▼
2. shell keyword?  (if, for, while ...)
      │না
      ▼
3. function?       (তুমি বানানো shell ফাংশন)
      │না
      ▼
4. builtin?        (cd, echo, type, export ...)
      │না
      ▼
5. hash table?     (আগে খুঁজে রাখা পাথ মনে আছে?)
      │না
      ▼
6. $PATH-এর প্রতিটা ডিরেক্টরি বাম থেকে ডানে:
   /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:...
```

```
      প্রথম যেখানে executable 'ls' পাওয়া গেল → চালাও
         │কোথাও নেই
         ▼
      "ls: command not found"
```

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
```

`PATH` নিয়ে বিস্তারিত অধ্যায় ৪৪-এ।

`which`

`$PATH`-এ খুঁজে executable ফাইলের পাথ দেখায়:

```
$ which python3
/usr/bin/python3

$ which -a python3        # PATH-এ যতগুলো আছে সব
/usr/bin/python3
/bin/python3
```

(Ubuntu/Debian-এ `/bin` হলো `/usr/bin`-এর লিংক, তাই একই প্রোগ্রাম দুবার দেখা যায়। এটাকে "usrmerge" বলে; অধ্যায় ৭ দেখো।)

> ⚠️ **সতর্কতা:** `which` একটা বাইরের প্রোগ্রাম; সে shell-এর alias, function বা builtin চেনে না। `which cd` হয় কিছুই দেখায় না, নয়তো ভুল কিছু দেখায়, অথচ `cd` আসলে builtin। আসল উত্তর চাইলে `type` ব্যবহার করো। (Fedora/RHEL-এ `which` একটা alias হিসেবে সেট করা থাকতে পারে যা alias-ও দেখায়।)

`type` (bash builtin)

shell আসলে কোনটা চালাবে, তার সবচেয়ে সঠিক উত্তর:

```
$ type ls
ls is aliased to `ls --color=auto'

$ type cd
cd is a shell builtin

$ type if
if is a shell keyword

$ type python3
python3 is /usr/bin/python3

$ type -a echo
echo is a shell builtin
echo is /usr/bin/echo
```

```
echo is /bin/echo

$ type -t ls
alias
```

অপশন	কাজ
`-a`	সব সম্ভাব্য জায়গা (alias, builtin, ফাইল)
`-t`	শুধু ধরন: `alias`, `keyword`, `function`, `builtin`, `file`
`-p`	ডিস্কের ফাইল হলে শুধু পাথ
`-P`	alias/function উপেক্ষা করে PATH-এ খোঁজো

`command -v`

POSIX মানসম্মত, সব shell-এ (bash, dash, zsh) চলে। স্ক্রিপ্টে কোনো প্রোগ্রাম ইনস্টল আছে কিনা দেখার **সবচেয়ে ভালো উপায়**:

```
$ command -v git
/usr/bin/git
$ command -v ll
alias ll='ls -alF'

if command -v docker >/dev/null 2>&1; then
    echo "docker আছে"
else
    echo "docker ইনস্টল করো"
fi
```

`command`-এর আরেকটা কাজ: alias/function এড়িয়ে আসল কমান্ড চালানো। `command ls` লিখলে alias ছাড়া `ls` চলে (`\ls` লিখলেও alias এড়ানো যায়)।

`whereis`

শুধু executable না, সাথে **man page আর source**-ও খোঁজে, নির্দিষ্ট কিছু স্ট্যান্ডার্ড ডিরেক্টরিতে (PATH-এর ওপর পুরোপুরি নির্ভর করে না):

```
$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz

$ whereis -b bash     # শুধু binary
bash: /usr/bin/bash

$ whereis -m passwd   # শুধু man page
passwd: /usr/share/man/man1/passwd.1ssl.gz /usr/share/man/man5/passwd.5.gz
/usr/share/man/man1/passwd.1.gz
```

অপশন	কাজ
`-b`	শুধু binary
`-m`	শুধু manual
`-s`	শুধু source
`-u`	যেগুলোর কোনো একটা অংশ পাওয়া যায়নি

`hash`: shell-এর মনে রাখা খাতা

bash প্রতিবার পুরো PATH ঘোরে না; একবার পেলে পাথটা একটা hash table-এ মনে রাখে।

```
$ hash
hits    command
   3    /usr/bin/git
   1    /usr/bin/ls

$ hash -r     # খাতা মুছে ফেলো
```

📝 কোনো প্রোগ্রাম এক জায়গা থেকে অন্য জায়গায় সরালে (যেমন `/usr/local/bin`-এ নতুন ভার্সন বসালে) bash তবুও পুরোনো পাথ চালাতে চাইতে পারে এবং `No such file or directory` দেখাতে পারে। তখন `hash -r` চালাও।

নাম জানা নেই, কাজ জানা আছে: `apropos` / `man -k`

```
$ apropos "copy files"
cp (1)               - copy files and directories
...
$ man -k partition
fdisk (8)            - manipulate disk partition table
parted (8)           - a partition manipulation program
...
```

এগুলো man page-এর ছোট বিবরণে খোঁজে (ডেটাবেস `mandb` দিয়ে বানানো)। কোনো কমান্ড ইনস্টল নেই কিন্তু কোন প্যাকেজে আছে জানতে চাইলে Ubuntu-তে `command-not-found` সাহায্য করে, আর `apt-file search` / `dnf provides` (অধ্যায় ৩০)।

তুলনা টেবিল

টুল	ধরন	alias/function/builtin চেনে	man page দেখায়	স্ক্রিপ্টে উপযুক্ত
`which`	বাইরের প্রোগ্রাম	না	না	মোটামুটি
`type`	bash builtin	হ্যাঁ	না	bash-এ হ্যাঁ
`command -v`	POSIX builtin	হ্যাঁ	না	**সবচেয়ে ভালো**
`whereis`	বাইরের প্রোগ্রাম	না	হ্যাঁ	না

১২.১৭ `fd`: আধুনিক, দ্রুত, সহজ

`fd` হলো Rust-এ লেখা `find`-এর একটা আধুনিক বিকল্প। ব্যবহারে সহজ, রঙিন আউটপুট, আর একাধিক থ্রেডে সমান্তরালে খোঁজে।

ইনস্টল

Distro	কমান্ড	কমান্ডের নাম
Ubuntu / Debian	`sudo apt install fd-find`	`fdfind` (নাম-সংঘাত এড়াতে)
Fedora	`sudo dnf install fd-find`	`fd`
RHEL 9	EPEL রিপোজিটরি থেকে `fd-find`	`fd`

Ubuntu/Debian-এ `fd` নামে চালাতে:

```
$ mkdir -p ~/.local/bin
$ ln -s $(which fdfind) ~/.local/bin/fd
```

(নিশ্চিত হও `~/.local/bin` তোমার `PATH`-এ আছে।)

`find` থেকে কী আলাদা

- প্যাটার্ন default-এ **regex**, আর পুরো পাথ না, নামের যেকোনো অংশে মেলে।
- **smart case**: প্যাটার্ন পুরোটা ছোট হাতের হলে case-insensitive, কোনো বড় হাতের অক্ষর থাকলে case-sensitive।
- default-এ **লুকানো ফাইল** (`.` দিয়ে শুরু) আর **`.gitignore`-এ থাকা ফাইল** বাদ দেয়। প্রজেক্ট ফোল্ডারে এটা খুব আরামদায়ক (`node_modules`, `vendor` আপনিই বাদ)।

```
$ fd config
src/config.rs
docs/configuration.md

$ fd -e md                  # এক্সটেনশন
README.md
docs/configuration.md

$ fd -t d src               # শুধু ডিরেক্টরি
$ fd -H bashrc ~            # লুকানো ফাইলসহ
/home/rahim/.bashrc
```

দরকারি অপশন

অপশন	কাজ	find-এর সমতুল্য
`-e EXT`	এক্সটেনশন	`-name "*.EXT"`

অপশন	কাজ	find-এর সমতুল্য
`-t f/d/l/x/e`	ফাইল/ডিরেক্টরি/লিংক/executable/empty	`-type`, `-empty`
`-H`	লুকানো ফাইলও	(default)
`-I`	`.gitignore` মানবে না	(default)
`-s` / `-i`	case-sensitive / insensitive	`-name` / `-iname`
`-g`	regex-এর বদলে glob প্যাটার্ন	`-name`
`-p`	পুরো পাথে মেলাও	`-path` / `-regex`
`-d N`	সর্বোচ্চ গভীরতা	`-maxdepth N`
`-E PATTERN`	বাদ দাও	`-prune`
`-S +1m`	সাইজ	`-size`
`--changed-within 2d`	সাম্প্রতিক বদল	`-mtime -2`
`--changed-before 1w`	পুরোনো	`-mtime +7` (মোটামুটি)
`-x CMD`	প্রতিটার জন্য কমান্ড	`-exec {} \;`
`-X CMD`	সবগুলো একবারে	`-exec {} +`
`-0`	NUL বিভাজক	`-print0`
`-l`	`ls -l` ধরনের তালিকা	`-ls`

```
$ fd -e jpg -x convert {} {.}.png     # প্রতিটা jpg → png
$ fd -e log -X rm                      # সব log একবারে মুছো
$ fd -t f -S +100m . ~                 # 100 MB-এর বড় ফাইল
$ fd -E node_modules -e js
```

`-x`-এ বিশেষ placeholder: `{}` পুরো পাথ, `{/}` basename, `{//}` parent ডিরেক্টরি, `{.}` এক্সটেনশন ছাড়া পাথ, `{/.}` এক্সটেনশন ছাড়া basename।

> 💡 **টিপস:** রোজকার প্রজেক্টের ভেতরে `fd` দারুণ। কিন্তু সার্ভারে `fd` ইনস্টল না-ও থাকতে পারে, আর স্ক্রিপ্টে portability দরকার। তাই `find` ভালোভাবে জানা বাধ্যতামূলক, `fd` বোনাস।

১২.১৮ ব্যবহারিক রেসিপি

```
# ১. ডিস্ক ভরে গেছে: সবচেয়ে বড় ২০টা ফাইল
sudo find / -xdev -type f -size +100M -printf "%s\t%p\n" 2>/dev/null | sort -rn | head
-20 | numfmt --field=1 --to=iec

# ২. শেষ ১০ মিনিটে কোন কনফিগ ফাইল বদলেছে
sudo find /etc -type f -mmin -10
```

```
# ৩. ৩০ দিনের পুরোনো লগ কম্প্রেস
sudo find /var/log/myapp -name "*.log" -mtime +30 -exec gzip {} +

# ৪. ৯০ দিনের পুরোনো .gz লগ মুছো (আগে -print দিয়ে দেখে নাও!)
sudo find /var/log/myapp -name "*.gz" -mtime +90 -print
sudo find /var/log/myapp -name "*.gz" -mtime +90 -delete

# ৫. ওয়েব রুটে ডিরেক্টরি 755, ফাইল 644
sudo find /var/www/html -type d -exec chmod 755 {} +
sudo find /var/www/html -type f -exec chmod 644 {} +

# ৬. সবাই লিখতে পারে এমন ফাইল (নিরাপত্তা ঝুঁকি)
sudo find / -xdev -type f -perm -o=w 2>/dev/null

# ৭. SUID/SGID প্রোগ্রামের তালিকা (নিরাপত্তা অডিট, অধ্যায় ৪৫)
sudo find / -xdev \( -perm -4000 -o -perm -2000 \) -type f -printf "%M %u %p\n"
2>/dev/null

# ৮. ভাঙা সিম্বলিক লিংক খুঁজে মুছো
find ~ -xtype l -print
find ~ -xtype l -delete

# ৯. ডুপ্লিকেট ফাইল: md5 হ্যাশ মিলিয়ে একই কনটেন্টের ফাইল
find ~/Pictures -type f -print0 | xargs -0 md5sum | sort | uniq -w32 -D

# ১০. প্রজেক্টে কয় লাইন PHP কোড (vendor বাদে)
find . -path ./vendor -prune -o -name "*.php" -print0 | xargs -0 cat | wc -l

# ১১. কোন ফাইলে "password" লেখা আছে (নাম + কনটেন্ট মিলিয়ে)
find . -type f -name "*.env*" -exec grep -l "password" {} +

# ১২. inode নম্বর দিয়ে ফাইল (অদ্ভুত নামের ফাইল মুছতে, অধ্যায় ১১)
ls -i
find . -maxdepth 1 -inum 1835123 -delete

# ১৩. কোনো ফাইলের সব হার্ড লিংক
find / -xdev -samefile /path/to/file 2>/dev/null

# ১৪. অনাথ ফাইল (মালিক ইউজার আর নেই)
sudo find / -xdev \( -nouser -o -nogroup \) 2>/dev/null

# ১৫. আজ বদলানো সব ফাইলের ব্যাকআপ আর্কাইভ (অধ্যায় ৩৪)
find ~/Documents -type f -daystart -mtime 0 -print0 | tar --null -czf today.tar.gz -T -

# ১৬. ফোল্ডারে কতগুলো ফাইল (টাইপ অনুযায়ী)
find . -type f | sed 's/.*\.//' | sort | uniq -c | sort -rn | head

# ১৭. সবচেয়ে সম্প্রতি বদলানো ১০টা ফাইল
find . -type f -printf "%T@ %p\n" | sort -rn | head -10 | cut -d' ' -f2-

# ১৮. খালি ফোল্ডার পরিষ্কার
find ~/Downloads -mindepth 1 -type d -empty -delete
```

> 💡 **টিপস:** রেসিপি ১-এর `numfmt --to=iec` বাইটকে `1.2G` ধরনের পড়ার মতো রূপে দেখায়। ডিরেক্টরি

> অনুযায়ী জায়গা দেখতে `du -sh *` বা `ncdu` (অধ্যায় ৩২)।

> ⚠️ **সতর্কতা:** রেসিপি ১১-এর মতো কনটেন্ট খোঁজায় ছোট প্রজেক্টে `grep -rl "password" --include="*.env*" .` আরও সহজ (অধ্যায় ২০)।

১২.১৯ খোঁজার সাথে সম্পর্কিত কীবোর্ড শর্টকাট

শর্টকাট	কোথায়	কাজ
`Ctrl + C`	টার্মিনাল	চলমান `find` থামাও
`Ctrl + Z`	টার্মিনাল	`find` সাময়িক থামাও (`bg` / `fg` দিয়ে ফেরাও, অধ্যায় ২৫)
`Ctrl + S` / `Ctrl + Q`	টার্মিনাল	আউটপুট স্ক্রল থামাও / আবার চালাও
`Ctrl + R`	bash	আগে চালানো `find` কমান্ড হিস্ট্রিতে খোঁজো (অধ্যায় ৬)
`Tab` / `Tab Tab`	bash	পাথ বা কমান্ডের নাম সম্পূর্ণ করো / সব বিকল্প দেখাও
`Alt + .`	bash	আগের কমান্ডের শেষ আর্গুমেন্ট বসাও (একই পাথে আবার খুঁজতে)
`Ctrl + Shift + F`	GNOME Terminal	টার্মিনালের লেখার ভেতরে খোঁজো
`/লেখা`, `n`, `N`	`less`	অনেক লম্বা ফলাফল `find ... | less` দিয়ে দেখার সময় খোঁজো, পরের/আগের মিল
`q`	`less`	বের হও
`Ctrl + F` বা সরাসরি টাইপ করা	Files (Nautilus)	GUI-তে ফাইল খোঁজো (অধ্যায় ৪)
`Super` তারপর টাইপ	GNOME	অ্যাপ ও ফাইল খোঁজো

হাতে-কলমে অনুশীলন

একটা নিরাপদ খেলার মাঠ বানিয়ে নাও, যাতে ভুল হলেও আসল ফাইলের ক্ষতি না হয়।

ধাপ ১: খেলার মাঠ তৈরি

```
$ mkdir -p ~/findlab/{docs,photos,logs,code/node_modules,empty}
$ cd ~/findlab
$ touch docs/{a,b,c}.txt docs/Report.PDF "docs/my notes.txt"
$ touch photos/img_{0001..0005}.jpg photos/logo.png
$ for i in 1 2 3; do echo "line $i" > logs/app$i.log; done
$ touch code/main.js code/node_modules/lib.js
$ dd if=/dev/zero of=photos/big.raw bs=1M count=5 status=none
```

```
$ touch -d "40 days ago" logs/app1.log
$ chmod 777 code/main.js
$ ln -s /nowhere broken-link
$ tree
```

ধাপ ২: নাম দিয়ে

```
$ find . -name "*.txt"
$ find . -iname "*.pdf"
$ find . -name "img_000[1-3].jpg"
```

লক্ষ করো, `my notes.txt` ঠিকঠাক এসেছে কিনা।

ধাপ ৩: টাইপ, সাইজ, empty

```
$ find . -type d
$ find . -type f -size +1M          # big.raw আসবে
$ find . -empty                      # খালি ফাইল আর empty/ ডিরেক্টরি
$ find . -type f -size -1M | wc -l   # কতগুলো? কেন? (রাউন্ডিং মনে করো)
```

ধাপ ৪: সময় আর পারমিশন

```
$ find . -mtime +30                  # app1.log
$ find . -mmin -5                    # সদ্য তৈরি সব
$ find . -perm 777                   # main.js
$ find . -xtype l                    # broken-link
```

ধাপ ৫: লজিক আর prune

```
$ find . -name "*.jpg" -o -name "*.png" -print      # কী এলো? কেন শুধু png?
$ find . \( -name "*.jpg" -o -name "*.png" \) -print
$ find . -name "*.js"
$ find . -path ./code/node_modules -prune -o -name "*.js" -print
```

ধাপ ৬: exec ও xargs

```
$ find logs -name "*.log" -exec wc -l {} \;
$ find logs -name "*.log" -exec wc -l {} +
$ find . -name "*.txt" -print0 | xargs -0 ls -l
$ find . -name "*.txt" | xargs ls -l      # "my notes.txt"-এ কী এরর আসে?
```

ধাপ ৭: নিরাপদে মোছা

```
$ find logs -mtime +30 -print       # আগে দেখো
$ find logs -mtime +30 -delete      # তারপর মুছো
$ find . -type d -empty -print
```

ধাপ ৮: locate পরীক্ষা

```
$ locate findlab/docs/a.txt          # সম্ভবত কিছু আসবে না, কেন?
$ sudo updatedb
$ locate findlab/docs/a.txt
```

ধাপ ৯: কমান্ড খোঁজা

```
$ type ls cd if find
$ which -a ls
$ command -v find
$ whereis find
$ apropos "search for files"
```

ধাপ ১০ (ঐচ্ছিক): fd

```
$ fdfind -e jpg            # Fedora-তে: fd -e jpg
$ fdfind js                # node_modules কি আসে? (gitignore না থাকলে আসবে)
```

শেষে পরিষ্কার: `rm -rf ~/findlab`

সারসংক্ষেপ

কাজ	কমান্ড
নাম দিয়ে	`find DIR -name "*.txt"`
case উপেক্ষা	`find DIR -iname "*.jpg"`
পুরো পাথ	`find DIR -path "*/docs/*"`
ধরন	`-type f` / `d` / `l`
সাইজ	`-size +100M`, `-size -10k`
খালি	`-empty`
বদলের সময়	`-mtime -7`, `-mmin -30`, `-newermt "2026-11-01"`
মালিক	`-user rahim`, `-group www-data`, `-nouser`
পারমিশন	`-perm 644`, `-perm -4000`, `-perm /o=w`
গভীরতা	`-maxdepth 1`, `-mindepth 2`, `-xdev`
AND / OR / NOT	(খালি) বা `-a`, `-o`, `!`, `\( \)`
ডিরেক্টরি বাদ	`-path ./x -prune -o ... -print`
ফরম্যাট	`-ls`, `-printf "%s %p\n"`
প্রতিটায় কমান্ড	`-exec cmd {} \;`
একবারে কমান্ড	`-exec cmd {} +`

কাজ	কমান্ড
জিজ্ঞেস করে	`-ok cmd {} \;`
মোছা	`... -delete` (শেষে!)
নিরাপদ পাইপ	`-print0 | xargs -0 cmd`
প্রথমটাতেই থামো	`-print -quit`
ডেটাবেস খোঁজা	`locate -i NAME`, `sudo updatedb`
executable কোথায়	`which -a`, `command -v`
shell কী চালাবে	`type -a NAME`
binary + man	`whereis NAME`
কাজ দিয়ে কমান্ড	`apropos KEYWORD`, `man -k`
PATH ক্যাশ মোছা	`hash -r`
আধুনিক বিকল্প	`fd -e EXT`, `fd -t d`, `fd -H`, `fd -x`

নিজেকে যাচাই করো

1. `find` আর `locate`-এর মূল পার্থক্য কী? কোন পরিস্থিতিতে `locate` ভুল উত্তর দেয়?
2. `find . -name *.txt` (কোটেশন ছাড়া) কেন বিপজ্জনক?
3. `-size -1M` দিয়ে খুঁজলে শুধু খালি ফাইল আসে কেন? 1 MiB-এর ছোট সব ফাইল কীভাবে খুঁজবে?
4. `-mtime +1` ঠিক কোন ফাইলগুলো মেলায়?
5. `-perm 755`, `-perm -755` আর `-perm /755`-এর পার্থক্য কী?
6. `find . -name "*.jpg" -o -name "*.png" -print` কেন jpg দেখায় না? ঠিক করো।
7. `-exec rm {} \;` আর `-exec rm {} +`-এর পার্থক্য কী, আর কোনটা দ্রুত কেন?
8. `find . -delete -name "*.tmp"` চালালে কী হবে?
9. `xargs`-এর সাথে `-print0` আর `-0` কেন দরকার?
10. `which cd` আর `type cd`-এর উত্তর আলাদা কেন? স্ক্রিপ্টে কোনো প্রোগ্রাম আছে কিনা দেখতে কোনটা ব্যবহার করবে?

▼ উত্তর দেখো

1. `find` এই মুহূর্তে ডিস্ক ঘুরে খোঁজে (ধীর, কিন্তু সবসময় সঠিক, আর সাইজ/সময়/মালিক দিয়েও খোঁজা যায়)। `locate` আগে বানানো ডেটাবেসে খোঁজে (খুব দ্রুত, কিন্তু শুধু নাম দিয়ে)। শেষ `updatedb`-এর পরে তৈরি হওয়া ফাইল `locate` দেখায় না, আর ডিলিট হওয়া ফাইল দেখাতে পারে। `PRUNEPATHS`-এ থাকা জায়গাও (যেমন `/tmp`) দেখায় না।
2. shell আগেই `*.txt`-কে বর্তমান ডিরেক্টরির মিলে যাওয়া ফাইলের নাম দিয়ে বদলে দেয়। একটা মিললে শুধু সেই নামেই খোঁজা হয়, একাধিক মিললে `paths must precede expression` এরর আসে।

3. `find` সাইজকে একক অনুযায়ী উপরের দিকে রাউন্ড করে। 1 বাইটের ফাইলও 1M হয়ে যায়, তাই "1M-এর কম" মানে শুধু 0। সমাধান: `-size -1048576c` বা `-size -1024k`।
4. বয়স ÷ ২৪ ঘণ্টা (ভগ্নাংশ বাদ) 1-এর বেশি, মানে কমপক্ষে 48 ঘণ্টা পুরোনো ফাইল।
5. `755` = হুবহু rwxr-xr-x। `-755` = এই সব বিট সেট আছে (আরও বেশি থাকতে পারে, যেমন 777-ও মিলবে)। `/755` = এর মধ্যে যেকোনো একটা বিট সেট থাকলেই হলো।
6. AND-এর অগ্রাধিকার OR-এর চেয়ে বেশি, তাই `-print` শুধু `-name "*.png"`-এর সাথে জুড়েছে। ঠিক রূপ: `find . \( -name "*.jpg" -o -name "*.png" \) -print`।
7. `\;` প্রতিটা ফাইলের জন্য আলাদা `rm` process চালায়; `+` অনেক নাম জড়ো করে অল্প কয়েকবার চালায়। process তৈরি (fork+exec) ব্যয়বহুল, তাই `+` অনেক দ্রুত।
8. expression বাম থেকে ডানে চলে, তাই `-delete` আগে চলে, বর্তমান ডিরেক্টরির নিচের (মুছতে পারা) সব ফাইল ও খালি ডিরেক্টরি মুছে যায়। `-delete` সবসময় শেষে।
9. ফাইলের নামে স্পেস, ট্যাব, এমনকি newline থাকতে পারে, যা সাধারণ `xargs` ভেঙে আলাদা আর্গুমেন্ট বানায়। NUL অক্ষর পাথে থাকতে পারে না, তাই সেটাই একমাত্র নিরাপদ বিভাজক।
10. `which` বাইরের প্রোগ্রাম, শুধু PATH-এ ফাইল খোঁজে, shell builtin চেনে না। `type` হলো bash-এর নিজের কমান্ড, তাই জানে `cd` একটা builtin। স্ক্রিপ্টে `command -v NAME >/dev/null 2>&1` সবচেয়ে ভালো, কারণ এটা POSIX এবং সব shell-এ চলে।

অধ্যায় ১৩: nano টেক্সট এডিটর

> "লিনাক্সে সবকিছুই ফাইল, আর প্রায় সব কনফিগারেশনই সাধারণ টেক্সট ফাইল। তাই যে টেক্সট এডিট করতে পারে, সে লিনাক্সকে নিয়ন্ত্রণ করতে পারে।"

এই অধ্যায়ে যা শিখবে

- কেন গ্রাফিক্যাল এডিটর থাকার পরেও টার্মিনাল-ভিত্তিক এডিটর (terminal text editor) শেখা জরুরি
- nano কী, কোথা থেকে এল, আর ভেতরে কীভাবে কাজ করে (buffer, terminal, keystroke)
- nano দিয়ে ফাইল খোলা, লেখা, সেভ করা আর বের হওয়া
- `^` আর `M-` চিহ্নের মানে কী, আর কোন কী (key) চাপলে কী হয়
- চলাফেরা (navigation), কাট-কপি-পেস্ট, undo/redo
- খোঁজা (search) ও প্রতিস্থাপন (replace), রেগুলার এক্সপ্রেশন দিয়েও
- লাইন নম্বর, syntax highlighting, soft wrap, auto-indent
- একসাথে একাধিক ফাইল (multiple buffer) নিয়ে কাজ
- `~/.nanorc` দিয়ে nano নিজের মতো সাজানো
- `sudo` আর `sudoedit` দিয়ে নিরাপদে সিস্টেম ফাইল এডিট করা
- `EDITOR` ও `VISUAL` ভেরিয়েবল দিয়ে ডিফল্ট এডিটর ঠিক করা
- GUI বিকল্প: GNOME Text Editor / gedit, Kate, Mousepad ইত্যাদি
- nano-র **সম্পূর্ণ শর্টকাট টেবিল**

১৩.১ কেন টার্মিনাল এডিটর লাগে?

ধরো তুমি একটা নতুন বাসায় উঠেছ। বাসার সব সুইচ, পানির লাইন, গ্যাসের লাইন ঠিকঠাক চালানোর নির্দেশনা লেখা আছে দেয়ালে সাঁটানো কিছু কাগজে। কিছু বদলাতে হলে তোমাকে শুধু কাগজের লেখাটা বদলাতে হবে। লিনাক্স ঠিক এরকম: নেটওয়ার্ক, ইউজার, সার্ভিস, বুট, ওয়েব সার্ভার, প্রায় সব কিছুর সেটিং থাকে `/etc` ডিরেক্টরির সাধারণ **টেক্সট ফাইলে** (অধ্যায় ৭ দেখো)। Windows-এর মতো রেজিস্ট্রি বা বাইনারি সেটিং ডাটাবেস এখানে মূল জিনিস না।

তাহলে প্রশ্ন আসে, Notepad-এর মতো গ্রাফিক্যাল এডিটর থাকতে টার্মিনালে এডিটর কেন?

পরিস্থিতি	কেন GUI এডিটর কাজ করবে না
রিমোট সার্ভার (SSH দিয়ে ঢুকেছ, অধ্যায় ৩৮)	সার্ভারে সাধারণত কোনো ডেস্কটপই নেই, শুধু টেক্সট
সিস্টেম বুট হচ্ছে না, rescue/recovery mode	গ্রাফিক্স চালুই হয়নি, আছে শুধু কনসোল

পরিস্থিতি	কেন GUI এডিটর কাজ করবে না
Docker কন্টেইনার, ক্লাউড VM, Raspberry Pi	মেমরি কম, GUI ইনস্টল করা নেই
`crontab -e`, `git commit`, `visudo`, `systemctl edit`	এই কমান্ডগুলো নিজেই একটা টার্মিনাল এডিটর খুলে দেয়
দ্রুত একটা লাইন বদলাতে হবে	টার্মিনাল থেকে হাত না সরিয়েই কাজ শেষ

লিনাক্স জগতে তিনটি টার্মিনাল এডিটর সবচেয়ে বেশি চলে:

এডিটর	শেখা কতটা সহজ	শক্তি	কাদের জন্য
nano	খুব সহজ, স্ক্রিনেই শর্টকাট লেখা থাকে	মাঝারি	নতুনরা, দ্রুত ছোট এডিট
vim (অধ্যায় ১৪)	কঠিন (mode-ভিত্তিক)	খুব বেশি	সিস্টেম অ্যাডমিন, প্রোগ্রামার
emacs	কঠিন	বিশাল (প্রায় একটা OS!)	যারা সব কিছু এক জায়গায় চায়

এই অধ্যায়ে আমরা nano পুরোপুরি শিখব। এটাই তোমার প্রথম এডিটর হওয়া উচিত। পরের অধ্যায়ে vim।

> 💡 **টিপস:** Ubuntu ও Debian-এ nano আগে থেকেই ইনস্টল করা থাকে এবং ডিফল্ট এডিটর হিসেবে সেট থাকে। Fedora 33 থেকে nano-ই ডিফল্ট এডিটর। RHEL 9 / Rocky / AlmaLinux-এর minimal ইনস্টলে অনেক সময় শুধু `vi` থাকে, nano আলাদা করে ইনস্টল করতে হয়।

১৩.২ nano কী? একটু ইতিহাস

nano হলো **GNU Project**-এর একটি ছোট, সহজ, ফ্রি টার্মিনাল টেক্সট এডিটর। ১৯৯৯ সালে এটা তৈরি হয় **Pico** নামের এক এডিটরের ফ্রি বিকল্প হিসেবে। Pico ছিল Pine ইমেইল প্রোগ্রামের অংশ, কিন্তু এর লাইসেন্স পুরোপুরি ফ্রি ছিল না। তাই শুরুতে nano-র নাম ছিল "TIP" (TIP Isn't Pico), পরে নাম হয় nano। মজার ব্যাপার, মাপের এককে pico (10^{-12}) এর চেয়ে nano (10^{-9}) বড়। নামটাও একটা রসিকতা।

বর্তমান সংস্করণ (distro অনুযায়ী):

Distro	nano ভার্সন (প্রায়)
Ubuntu 24.04 LTS	7.2
Debian 12	7.2
Fedora 40/41	8.x
RHEL 9 / Rocky 9	5.6.1

নিজের ভার্সন দেখতে:

```
$ nano --version
 GNU nano, version 7.2
```

```
(C) 2023 the Free Software Foundation and various contributors
Compiled options: --disable-libmagic --enable-utf8
```

> 📝 **মনে রাখো:** পুরোনো ভার্সনে (যেমন RHEL 9-এর 5.6) কিছু শর্টকাট আলাদা বা অনুপস্থিত থাকতে পারে। কোনো শর্টকাট কাজ না করলে nano-র ভেতরে `^G` চেপে হেল্প দেখো, ওটাই তোমার ভার্সনের আসল তালিকা।

ইনস্টল না থাকলে

```
# Ubuntu / Debian
$ sudo apt install nano

# Fedora / RHEL 9 / Rocky / AlmaLinux
$ sudo dnf install nano

# Arch
$ sudo pacman -S nano
```

প্যাকেজ ম্যানেজার নিয়ে বিস্তারিত অধ্যায় ৩০-এ।

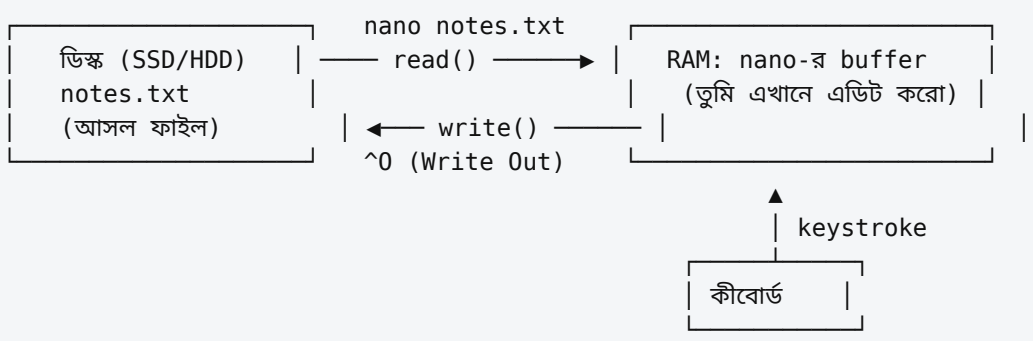
১৩.৩ ভেতরে কীভাবে কাজ করে (Under the Hood)

nano ব্যবহারের আগে এটা বুঝে নেওয়া ভালো যে তুমি কী টাইপ করলে আসলে কী ঘটে। এতে পরে অনেক "অদ্ভুত" আচরণ সহজে বুঝবে।

Buffer: ফাইলের একটা কাজের কপি

তুমি যখন `nano notes.txt` চালাও, nano ডিস্ক থেকে ফাইলটা পড়ে মেমরিতে (RAM) একটা কপি রাখে। এই কপিকে বলে **buffer** (বাংলায়: অস্থায়ী কাজের জায়গা)। তুমি যা টাইপ করো সব এই buffer-এ বদলায়, ডিস্কের আসল ফাইলে না। সেভ (`^O`) করলে তবেই buffer-এর লেখা ডিস্কে লেখা হয়।

উপমা: পরীক্ষার খাতায় লেখার আগে তুমি রাফ খাতায় লেখো। রাফ খাতা = buffer, আসল খাতা = ডিস্কের ফাইল। রাফে যত খুশি কাটাকাটি করো, আসল খাতায় তুলবে কিনা সেটা তোমার সিদ্ধান্ত।



এজন্যই সেভ না করে বের হয়ে গেলে, বা বিদ্যুৎ চলে গেলে, পরিবর্তন হারিয়ে যায়।

টার্মিনালের সাথে কথা বলা

nano একটা **TUI** (Text User Interface) প্রোগ্রাম। এটা `ncurses` নামের একটা লাইব্রেরি ব্যবহার করে পুরো টার্মিনাল স্ক্রিনটা নিজের দখলে নেয়, কার্সর যেখানে খুশি নিয়ে যায়, রং দেয়। টার্মিনাল তখন "raw mode"-এ চলে: তুমি `Ctrl+C` চাপলেও সেটা শেলের কাছে গিয়ে প্রসেস বন্ধ করে না, বরং nano নিজে সেটা পড়ে নিজের কাজে (cursor position দেখাতে) লাগায়।

```
তুমি Ctrl+O চাপলে
      │
      ▼
কীবোর্ড ──► টার্মিনাল এমুলেটর ──► বাইট 0x0F (ASCII control char)
                                          │
                                          ▼
                                   nano (ncurses) পড়ে
                                          │
                                          ▼
                          "Write Out" কাজ চালায়, স্ক্রিন আঁকে
```

`Ctrl` চেপে কোনো অক্ষর চাপলে টার্মিনাল একটা **control character** পাঠায় (যেমন Ctrl+O = বাইট 15)। এ কারণেই nano-র শর্টকাটগুলো টার্মিনালের ধরন যাই হোক, SSH-এর ওপারেও, কাজ করে।

Alt কী আর Esc

`Alt` (nano-র ভাষায় **Meta**) চেপে কোনো কী চাপলে বেশিরভাগ টার্মিনাল আসলে দুটো বাইট পাঠায়: আগে `Esc`, পরে অক্ষরটা। তাই যেখানে `Alt` কাজ করে না (কিছু টার্মিনাল বা Mac), সেখানে `Esc` একবার চেপে ছেড়ে দিয়ে তারপর অক্ষরটা চাপলেও একই কাজ হয়।

ফাইল সেভ: ভেতরে কী হয়

`^O` চাপলে nano ফাইলটা খুলে (open), buffer-এর লেখা লেখে (write), তারপর বন্ধ করে। ফাইলের মালিকানা ও পারমিশন সাধারণত আগের মতোই থাকে। কিন্তু যদি তোমার সেই ফাইলে লেখার পারমিশন না থাকে (অধ্যায় ১৬), তাহলে এই error দেখবে:

```
[ Error writing /etc/hosts: Permission denied ]
```

এর সমাধান পরে ১৩.১৪ অংশে দেখব।

১৩.৪ nano খোলা

Syntax

```
nano [OPTIONS] [[+LINE[,COLUMN]] FILE]...
```

উদাহরণ

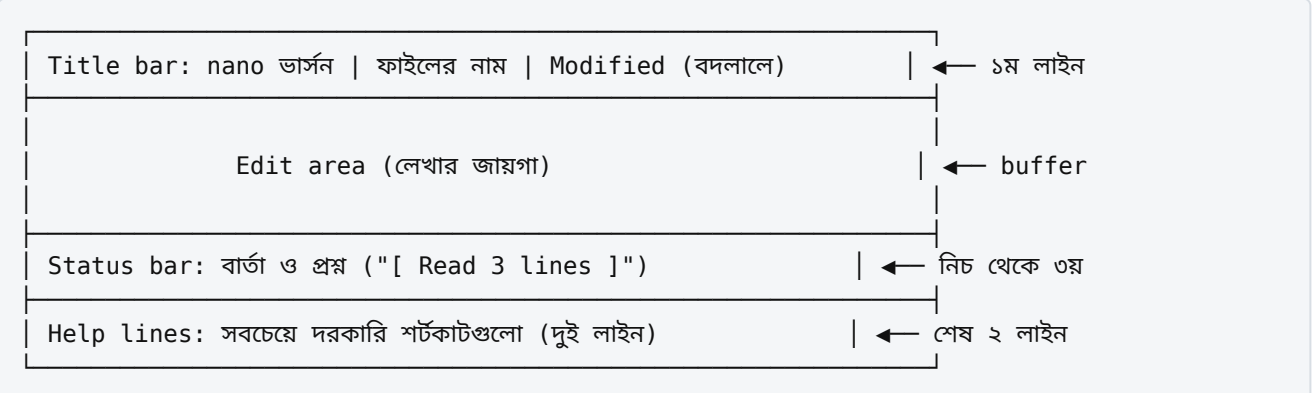
```
$ nano                  # নাম ছাড়া নতুন খালি buffer খোলে
$ nano notes.txt        # notes.txt খোলে; না থাকলে নতুন তৈরি হবে (সেভ করলে)
$ nano +25 script.sh    # ২৫ নম্বর লাইনে কার্সর রেখে খোলে
$ nano +25,10 app.py    # ২৫ নম্বর লাইনের ১০ নম্বর কলামে
$ nano +-1 log.txt      # শেষ লাইনে (নেগেটিভ সংখ্যা = শেষ থেকে গোনা)
$ nano a.txt b.txt      # দুটো ফাইল দুটো buffer-এ খোলে
$ nano -v /etc/fstab    # read-only (শুধু দেখার জন্য) মোডে
```

> 💡 **টিপস:** কোনো error message বলছে "line 42-এ সমস্যা"? সরাসরি `nano +42 file` দিয়ে খোলো, খুঁজতে হবে না।

nano-র স্ক্রিন চেনা

`nano notes.txt` চালালে এরকম দেখবে:

```
  GNU nano 7.2                      notes.txt
আজকের কাজ:
১. লিনাক্স অধ্যায় ১৩ পড়া
২. nano অনুশীলন করা
▉




                            [ Read 3 lines ]
^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line
```



- **Title bar**: ফাইলে কোনো পরিবর্তন করলে ডান পাশে `Modified` লেখা আসে। এটা দেখেই বুঝবে সেভ করা বাকি।
- **Status bar**: nano এখানে তোমাকে বার্তা দেয় বা প্রশ্ন করে ("File Name to Write:")।
- **Help lines**: স্ক্রিনেই শর্টকাট লেখা, nano-কে সহজ বলার মূল কারণ এটাই।

`^` আর `M-` চিহ্নের মানে

এটাই nano-র সবচেয়ে গুরুত্বপূর্ণ নোটেশন, ভালো করে মুখস্থ করো:

চিহ্ন	মানে	উদাহরণ	কীভাবে চাপবে
`^`	**Ctrl** কী	`^O`	Ctrl চেপে ধরে O
`M-`	**Meta** = **Alt** কী (বা Esc)	`M-U`	Alt চেপে ধরে U, অথবা Esc ছেড়ে তারপর U
`Sh-`	Shift	`Sh-M-C`	Shift+Alt+C

> 📝 **মনে রাখো:** অক্ষর বড় হাতের দেখালেও Shift চাপতে হয় না। `^O` মানে Ctrl+o, Ctrl+Shift+O না। একইভাবে `M-U` মানে Alt+u।

> ⚠️ **সতর্কতা:** কিছু টার্মিনাল এমুলেটর (যেমন GNOME Terminal) নিজেই কিছু Alt শর্টকাট ধরে ফেলে (যেমন Alt+F মেনু খোলে)। তখন `Esc` তারপর অক্ষর চাপো, অথবা টার্মিনালের Preferences-এ মেনু-শর্টকাট বন্ধ করো।

১৩.৫ লেখা, সেভ করা আর বের হওয়া (সবচেয়ে জরুরি তিনটি কাজ)

লেখা

nano-তে কোনো "mode" নেই (vim-এর মতো না)। খোলার পরই সরাসরি টাইপ করা যায়। অ্যারো কী দিয়ে কার্সর নড়াও, Backspace/Delete দিয়ে মোছো, Enter দিয়ে নতুন লাইন। একদম Notepad-এর মতো।

সেভ: `^O` (Write Out)

`Ctrl+O` চাপলে status bar-এ প্রশ্ন আসবে:

```
File Name to Write: notes.txt
^G Help             M-D DOS Format      M-A Append          M-B Backup File
^C Cancel           M-M Mac Format      M-P Prepend         ^T Browse
```

- নাম ঠিক থাকলে **Enter** চাপো। নিচে আসবে `[ Wrote 3 lines ]`।
- অন্য নামে সেভ করতে (Save As) নামটা মুছে নতুন নাম লিখে Enter।

- বাতিল করতে `^C`।

এই প্রম্পটের ভেতরের অপশনগুলো:

কী	কাজ
`M-D`	DOS/Windows ফরম্যাটে (CRLF লাইন-এন্ডিং) সেভ
`M-M`	পুরোনো Mac ফরম্যাটে (CR) সেভ
`M-A`	ফাইলের শেষে buffer জুড়ে দেয় (Append)
`M-P`	ফাইলের শুরুতে জুড়ে দেয় (Prepend)
`M-B`	সেভের আগে পুরোনো ফাইলের `filename~` ব্যাকআপ রাখে
`^T`	ফাইল ব্রাউজার খুলে জায়গা বাছাই

`^S` (Save): nano 4.0 থেকে `Ctrl+S` চাপলে কোনো প্রশ্ন ছাড়াই বর্তমান নামে সেভ হয়ে যায়। নামহীন নতুন buffer হলে নাম জিজ্ঞেস করবে।

> ⚠️ **সতর্কতা:** অনেক টার্মিনালে `Ctrl+S` হলো "XOFF" (ফ্লো কন্ট্রোল), যা স্ক্রিন জমিয়ে দেয়। nano চালু থাকলে nano নিজে এটা সামলায়, কিন্তু শেলে ভুল করে `Ctrl+S` চাপলে টার্মিনাল জমে গেছে মনে হবে। তখন `Ctrl+Q` চাপলে আবার চালু হবে। বিস্তারিত অধ্যায় ৬-এ।

বের হওয়া: `^X` (Exit)

- কোনো পরিবর্তন না থাকলে সাথে সাথে বের হয়ে শেলে ফিরবে।
- পরিবর্তন থাকলে জিজ্ঞেস করবে:

```
Save modified buffer?
 Y Yes
 N No           ^C Cancel
```

উত্তর	ফলাফল
`Y`	ফাইলের নাম জিজ্ঞেস করবে, Enter দিলে সেভ করে বের হবে
`N`	সেভ **না** করে বের হবে (সব পরিবর্তন বাতিল)
`^C`	বের হওয়া বাতিল, আবার এডিটরে ফিরে যাবে

> 💡 **টিপস:** দ্রুততম "সেভ করে বের হও" ক্রম: `Ctrl+X` → `Y` → `Enter`। এটা আঙুলে বসিয়ে ফেলো।

বাস্তব উদাহরণ: প্রথম ফাইল

```
$ nano hello.txt
# লেখো: Hello Linux!
# Ctrl+O, Enter, Ctrl+X
$ cat hello.txt
Hello Linux!
```

দুর্ঘটনায় বন্ধ হলে: `.save` ফাইল

তুমি এডিট করছ আর SSH কানেকশন কেটে গেল বা টার্মিনাল বন্ধ হয়ে গেল। nano তখন একটা signal (SIGHUP বা SIGTERM, অধ্যায় ২৪) পায়, আর মরার আগে অসেভড buffer বাঁচিয়ে রাখে `notes.txt.save` নামে (নামহীন buffer হলে `nano.save`)।

```
$ ls
notes.txt  notes.txt.save
$ diff notes.txt notes.txt.save    # কী পরিবর্তন হারিয়েছিল দেখো
```

দরকারি অংশ নিয়ে `.save` ফাইলটা মুছে দাও।

১৩.৬ কার্সর চলাফেরা (Navigation)

অ্যারো কী, Home, End, PageUp, PageDown সবই কাজ করে। কিন্তু হাত কীবোর্ডের মাঝখানে রেখে চলাফেরা করতে nano-র নিজস্ব শর্টকাটও আছে:

শর্টকাট	বিকল্প কী	কাজ
`^F`	→	এক অক্ষর সামনে (Forward)
`^B`	←	এক অক্ষর পেছনে (Back)
`^P`	↑	আগের লাইন (Previous)
`^N`	↓	পরের লাইন (Next)
`^A`	Home	লাইনের শুরুতে
`^E`	End	লাইনের শেষে
`^Space`	Ctrl+→	পরের শব্দে
`M-Space`	Ctrl+←	আগের শব্দে
`^Y`	PageUp	এক পাতা ওপরে
`^V`	PageDown	এক পাতা নিচে
`M-\`	Ctrl+Home	ফাইলের একদম শুরুতে
`M-/`	Ctrl+End	ফাইলের একদম শেষে

শর্টকাট	বিকল্প কী	কাজ
`^/` বা `^_` বা `M-G`	—	নির্দিষ্ট লাইন (ও কলাম) নম্বরে যাও
`M-]`	—	মিলে যাওয়া বন্ধনীতে (`(` ↔ `)`, `{` ↔ `}`) লাফ
`M-(` / `M-)`	—	আগের / পরের প্যারাগ্রাফে (বা কোড ব্লকে)
`M--` / `M-+`	—	কার্সর না নড়িয়ে স্ক্রিন এক লাইন ওপরে / নিচে স্ক্রল
`^L`	—	কার্সরের লাইনকে স্ক্রিনের মাঝে/ওপরে/নিচে আনে (বারবার চাপলে ঘোরে)

লাইন নম্বরে যাওয়া (`^/`)

`Ctrl+/` চাপলে প্রশ্ন আসে:

```
Enter line number, column number: 120,5
```

`120` দিলে ১২০ নম্বর লাইনে, `120,5` দিলে সেই লাইনের ৫ নম্বর কলামে। পুরোনো ভার্সনে এই শর্টকাট দেখাত `^_` হিসেবে; `Ctrl+/` আর `Ctrl+_` আসলে টার্মিনালে একই বাইট পাঠায়, তাই দুটোই চলে।

এখন কোথায় আছি? (`^C` Location)

`Ctrl+C` চাপলে nano প্রোগ্রাম বন্ধ **হয় না**, বরং status bar-এ কার্সরের অবস্থান দেখায়:

```
[ line 12/40 (30%), col 8/25 (32%), char 210/980 (21%) ]
```

> 📝 **মনে রাখো:** শেলে `Ctrl+C` মানে "প্রোগ্রাম থামাও", কিন্তু nano-র ভেতরে মানে "আমি কোথায়?" আর কোনো প্রশ্নের প্রম্পটে থাকলে মানে "বাতিল (Cancel)"।

১৩.৭ মোছা (Deletion)

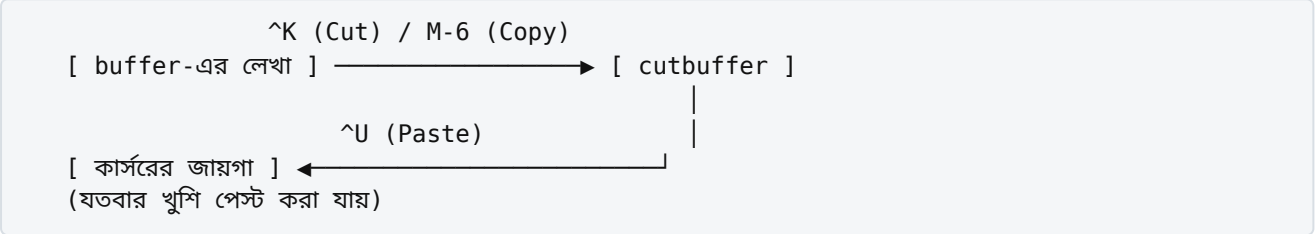
শর্টকাট	বিকল্প	কাজ
`^H`	Backspace	কার্সরের বাঁয়ের অক্ষর মোছে
`^D`	Delete	কার্সরের নিচের অক্ষর মোছে
`M-Bsp` (Alt+Backspace)	—	বাঁয়ের পুরো শব্দ মোছে
`^Del` (Ctrl+Delete)	—	ডানের পুরো শব্দ মোছে
`^K`	—	পুরো লাইন কেটে নেয় (cut, পরে পেস্ট করা যায়)
`M-T`	—	কার্সর থেকে ফাইলের শেষ পর্যন্ত কাটে

১৩.৮ কাট, কপি, পেস্ট আর মার্ক (Mark)

nano-র কাট-পেস্ট সিস্টেম একটু আলাদা, কিন্তু বুঝে ফেললে খুব দ্রুত।

Cutbuffer কী?

nano-র নিজস্ব একটা অস্থায়ী জায়গা আছে, নাম **cutbuffer**। `^K` দিয়ে কাটা বা `M-6` দিয়ে কপি করা লেখা এখানে যায়, আর `^U` দিয়ে এখান থেকে পেস্ট হয়। এটা সিস্টেমের ক্লিপবোর্ড **না**, তাই অন্য অ্যাপে পেস্ট হবে না।



পুরো লাইন কাট ও পেস্ট

- `^K` চাপো: বর্তমান লাইন কেটে cutbuffer-এ যাবে।
- **পরপর** কয়েকবার `^K` চাপলে সব লাইন একসাথে জমা হয় (মাঝে অন্য কিছু করলে নতুন করে শুরু হবে)।
- কার্সর নতুন জায়গায় নিয়ে `^U` চাপো: লাইনগুলো সেখানে বসবে।

উদাহরণ: লাইন ৩ ও ৪ কে ফাইলের শেষে সরানো

```
১. লাইন ৩-এ যাও
২. ^K ^K   (দুটো লাইন কাটা হলো)
৩. M-/     (ফাইলের শেষে)
৪. ^U      (পেস্ট)
```

একটা লাইন ডুপ্লিকেট (কপি) করা: `^K` তারপর সাথে সাথে `^U` (লাইন ফিরে এল), তারপর যেখানে দরকার গিয়ে আবার `^U`। অথবা সরাসরি `M-6` (কপি, লাইন জায়গায় থাকে) তারপর `^U`।

Mark দিয়ে নির্দিষ্ট অংশ বাছাই

পুরো লাইন না, একটা নির্দিষ্ট অংশ কাটতে/কপি করতে চাইলে আগে **mark** বসাতে হয় (অর্থাৎ নির্বাচন শুরু করা):

ধাপ	শর্টকাট	কী হয়
১	`M-A` বা `^6`	কার্সরের জায়গায় mark বসে, status bar-এ `[ Mark Set ]`
২	অ্যারো কী দিয়ে নড়াও	mark থেকে কার্সর পর্যন্ত অংশ হাইলাইট হয়
৩ক	`^K`	নির্বাচিত অংশ কাটো
৩খ	`M-6`	নির্বাচিত অংশ কপি করো
৪	নতুন জায়গায় `^U`	পেস্ট

আবার `M-A` চাপলে mark তুলে নেওয়া হয় (`[ Mark Unset ]`)।

> 💡 **টিপস:** Shift চেপে অ্যারো কী চাপলেও (Shift+→ ইত্যাদি) mark বসে আর নির্বাচন হয়, ঠিক সাধারণ এডিটরের মতো।

সিস্টেম ক্লিপবোর্ড ব্যবহার

nano-র বাইরে থেকে (যেমন ব্রাউজার থেকে) লেখা আনতে বা বাইরে নিতে **টার্মিনালের** কপি-পেস্ট ব্যবহার করো:

কাজ	বেশিরভাগ লিনাক্স টার্মিনালে
কপি	`Ctrl+Shift+C` (আগে মাউস দিয়ে সিলেক্ট করো)
পেস্ট	`Ctrl+Shift+V`
X11 middle-click পেস্ট	মাউস দিয়ে সিলেক্ট করো, তারপর মাউসের মাঝের বাটন চাপো

> ⚠️ **সতর্কতা:** nano-তে mouse support (`M-M`) চালু থাকলে মাউস দিয়ে সিলেক্ট করা টার্মিনালের সিলেকশন হয় না, nano-র mark হয়। তখন টার্মিনালে সিলেক্ট করতে Shift চেপে ধরে মাউস টানো।

১৩.৯ Undo আর Redo

শর্টকাট	কাজ
`M-U`	Undo, শেষ কাজটা ফেরত (বারবার চাপা যায়)
`M-E`	Redo, ফেরত নেওয়া কাজ আবার করো

> ⚠️ **সতর্কতা:** অন্য এডিটরের অভ্যাসে `Ctrl+Z` চাপবে না। nano-তে `^Z` মানে nano-কে **suspend** করা (পেছনে পাঠিয়ে শেলে ফেরা)। ভয় পেও না, শেলে `fg` লিখে Enter দিলেই nano ফিরে আসবে, কিছু হারায়নি। জব কন্ট্রোল বিস্তারিত অধ্যায় ২৫-এ। (nano 6.0-র আগের ভার্সনে `^Z` কাজ করত শুধু `-z` / `set suspendable` দিলে।)

> 📝 **মনে রাখো:** nano 8.0-তে একটা নতুন অপশন এসেছে `--modernbindings` (`-/`), যেটা দিলে `^Z` = Undo, `^Y` = Redo, `^C` = Copy, `^V` = Paste, `^F` = Find, `^Q` = Quit ইত্যাদি সাধারণ এডিটরের মতো কী হয়। এটা Fedora-র নতুন nano-তে আছে, কিন্তু Ubuntu 24.04 / Debian 12-এর nano 7.2-তে নেই।

১৩.১০ খোঁজা (Search)

সামনে খোঁজা: `^W` (Where Is)

`Ctrl+W` চাপো, শব্দ লেখো, Enter:

```
Search: server
^G Help        M-C Case Sens  M-B Backwards  M-J Justify    ^R Replace     ^/ Go To
Line
^C Cancel      M-R Reg.exp.   ^P Older       ^N Newer       ^T Go To Dir   ^Y First
Line
```

কার্সর প্রথম মিলে যাওয়া জায়গায় চলে যাবে। শেষে পৌঁছে আবার শুরু থেকে খোঁজে (wrap around), আর তখন status bar-এ লেখে `[ Search Wrapped ]`। কিছু না পেলে: `[ "server" not found ]`।

খোঁজা চালিয়ে যাওয়া

শর্টকাট	কাজ
`M-W`	পরের মিল (Next), একই শব্দ আবার খোঁজে
`M-Q`	আগের মিল (Previous)
`^Q`	পেছনের দিকে খোঁজা শুরু (Where Was, nano 5+)

Search প্রম্পটের ভেতরের টগল

কী	কাজ
`M-C`	Case sensitive চালু/বন্ধ (`Server` আর `server` আলাদা ধরবে কিনা)
`M-R`	রেগুলার এক্সপ্রেশন (regex) চালু/বন্ধ
`M-B`	উল্টো দিকে (Backwards) খোঁজা
`^P` / `^N` (বা ↑ ↓)	আগে খোঁজা শব্দের হিস্ট্রি
`^Y` / `^V`	সরাসরি প্রথম / শেষ লাইনে যাও

টগল চালু থাকলে প্রম্পটে লেখা বদলে যায়, যেমন `Search [Case Sensitive] [Regexp]:`।

Regex দিয়ে খোঁজা

`M-R` চালু করে extended regular expression লেখা যায়:

প্যাটার্ন	কী খুঁজবে
`^#`	`#` দিয়ে শুরু হওয়া লাইন (কমেন্ট)
`^$`	খালি লাইন

প্যাটার্ন	কী খুঁজবে
`[0-9]{3}`	পরপর তিনটি অঙ্ক
`error	fail`
`\.conf$`	`.conf` দিয়ে শেষ হওয়া লাইন

রেগুলার এক্সপ্রেশনের পুরো আলোচনা অধ্যায় ২০-এ।

১৩.১১ প্রতিস্থাপন (Search and Replace)

`^\` বা `M-R` (Replace)

ধাপগুলো:

```
১. Ctrl+\ চাপো
   Search (to replace): localhost
২. Enter, তারপর
   Replace with: 127.0.0.1
৩. Enter, এবার প্রতিটি মিলের জন্য প্রশ্ন:
   Replace this instance?
    Y Yes          A All
    N No           ^C Cancel
```

উত্তর	কাজ
`Y`	এইটা বদলাও, পরেরটায় যাও
`N`	এইটা বাদ দাও, পরেরটায় যাও
`A`	বাকি সবগুলো একসাথে বদলাও
`^C`	বাকিগুলো বাদ, থামো

শেষে জানাবে: `[ Replaced 5 occurrences ]`।

> 💡 **টিপস:** শুধু ফাইলের একটা অংশে replace করতে চাও? আগে `M-A` দিয়ে সেই অংশ mark করো, তারপর `^\`। replace তখন শুধু নির্বাচিত অংশেই হবে।

Regex দিয়ে replace ও back-reference

Search প্রম্পটে `M-R` চালু করলে regex-এর group ধরা যায় আর replace-এ `\1`, `\2` দিয়ে ব্যবহার করা যায়।

উদাহরণ: `2026-11-12` তারিখকে `12/11/2026` বানানো:

```
Search [Regexp] (to replace): ([0-9]{4})-([0-9]{2})-([0-9]{2})
Replace with: \3/\2/\1
```

পুরো ফাইলে একসাথে অনেক বড় replace-এর কাজ সাধারণত `sed` দিয়ে করা হয় (অধ্যায় ২১)।

১৩.১২ লাইন নম্বর, Syntax Highlighting ও অন্যান্য প্রদর্শন

লাইন নম্বর

- এডিটরের ভেতরে টগল: `M-N`
- খোলার সময়: `nano -l file` বা `nano --linenumbers file`

```
 GNU nano 7.2                    backup.sh
1 #!/bin/bash
2 # দৈনিক ব্যাকআপ স্ক্রিপ্ট
3 SRC="/home/rahim/docs"
4 DEST="/mnt/backup"
5 rsync -av "$SRC" "$DEST"
```

Syntax Highlighting (রঙিন কোড)

nano ফাইলের নাম বা ভেতরের প্রথম লাইন (যেমন `#!/bin/bash`) দেখে ভাষা চিনে কোডে রং দেয়: কমেন্ট এক রঙে, string আরেক রঙে, keyword আরেক রঙে। এই নিয়মগুলো থাকে **syntax definition** ফাইলে:

```
$ ls /usr/share/nano/
asm.nanorc    css.nanorc     html.nanorc    makefile.nanorc  python.nanorc  sh.nanorc
c.nanorc      default.nanorc java.nanorc    markdown.nanorc  ruby.nanorc    xml.nanorc
...
```

ভেতরে কীভাবে কাজ করে:

```
nano script.py
     │
     ▼
/usr/share/nano/*.nanorc থেকে সব "syntax" নিয়ম পড়া হয়েছে (nanorc-এর include দিয়ে)
     │
     ▼
python.nanorc:  syntax python "\.py$"  header "^#!.*python"
     │                      ▲ ফাইলের নাম মিলেছে
     ▼
color brightcyan "\<(def|class|import|return)\>"   ──► keyword রঙিন
color green      "['"].*['"]"                      ──► string রঙিন
color cyan       "#.*$"                            ──► কমেন্ট রঙিন
```

কাজ	কীভাবে
রং চালু/বন্ধ টগল	`M-Y`
জোর করে নির্দিষ্ট syntax	`nano -Y sh myscript` (এক্সটেনশন না থাকলেও shell হিসেবে রং দেবে)
রং একদম বন্ধ করে খোলা	`nano -Y none file`

📝 **মনে রাখো:** Ubuntu ও Debian-এর `/etc/nanorc`-এ আগে থেকেই `include "/usr/share/nano/*.nanorc"` লাইন চালু থাকে, তাই রং এমনিই আসে। কিছু distro-তে এটা কমেন্ট করা থাকে, তখন নিজের `~/.nanorc`-এ যোগ করতে হয় (নিচে দেখো)।

আরও প্রদর্শন টগল

শর্টকাট	কাজ
`M-N`	লাইন নম্বর দেখাও/লুকাও
`M-P`	whitespace (ট্যাব ও স্পেস) চিহ্নসহ দেখাও
`M-S`	Soft wrap: লম্বা লাইন স্ক্রিনের প্রস্থে ভেঙে দেখাও (ফাইলে কিছু বদলায় না)
`M-L`	Hard wrap: টাইপ করতে করতে লম্বা লাইন আসলেই ভেঙে নতুন লাইন বানাও
`M-C`	সবসময় কার্সরের অবস্থান status bar-এ দেখাও
`M-X`	নিচের help lines লুকাও/দেখাও (লেখার জায়গা বাড়ে)
`M-Y`	Syntax রং চালু/বন্ধ
`M-I`	Auto-indent চালু/বন্ধ
`M-O`	Tab চাপলে ট্যাবের বদলে স্পেস বসাও
`M-M`	মাউস সাপোর্ট চালু/বন্ধ
`M-K`	`^K` চাপলে পুরো লাইন না কেটে কার্সর থেকে লাইনের শেষ পর্যন্ত কাটো
`M-Z`	"zap": মার্ক করা অংশ Backspace/Delete চাপলে cutbuffer-এ না নিয়েই মুছে ফেলো

💡 **টিপস:** লম্বা কনফিগ লাইন (যেমন `GRUB_CMDLINE_LINUX=...`) স্ক্রিনের বাইরে চলে গেলে nano লাইনের শেষে `>` চিহ্ন দেখায়। `M-S` দিয়ে soft wrap চালু করলে পুরো লাইন দেখা যাবে। Hard wrap (`M-L`) কনফিগ ফাইলে **ব্যবহার করবে না**, এটা আসলেই লাইন ভেঙে দেয় আর কনফিগ নষ্ট করতে পারে। nano 4.0 থেকে hard wrap ডিফল্টে বন্ধ থাকে।

১৩.১৩ আরও শক্তিশালী ফিচার

ইনডেন্ট ও কমেন্ট

শর্টকাট	কাজ
`M-}` বা Tab (mark থাকলে)	লাইন/নির্বাচিত অংশ ডানে ইনডেন্ট
`M-{` বা Shift+Tab	বাঁয়ে সরাও (unindent)
`M-3` (`M-#`)	লাইন/নির্বাচিত অংশ কমেন্ট/আনকমেন্ট টগল (ভাষা অনুযায়ী `#`, `//` ইত্যাদি)

কনফিগ ফাইলে কোনো সেটিং সাময়িক বন্ধ রাখতে `M-3` দারুণ কাজের।

অন্য ফাইল ঢোকানো: `^R` (Read File)

`Ctrl+R` চাপলে একটা ফাইলের নাম চাইবে। দিলে সেই ফাইলের পুরো লেখা কার্সরের জায়গায় ঢুকে যাবে।

```
File to insert [from ./]: header.txt
^G Help        ^T Browse      M-F New Buffer   ^X Execute Command
^C Cancel      M-N No Conversion
```

এখানে `M-F` চাপলে ফাইলটা নতুন আলাদা buffer-এ খুলবে (ঢোকানো হবে না)।

কমান্ডের আউটপুট ঢোকানো ও বাইরের কমান্ড চালানো: `^T` (Execute)

`Ctrl+T` চাপলে "Command to execute:" প্রম্পট আসে। সেখানে যেকোনো শেল কমান্ড লিখলে তার আউটপুট কার্সরের জায়গায় বসে।

```
Command to execute: date
```

→ ফাইলে বসে যাবে: `Thu Nov 12 10:30:00 AM +06 2026`

আরও মজার: কমান্ডের শুরুতে `|` দিলে buffer (বা mark করা অংশ) সেই কমান্ডের ভেতর দিয়ে **filter** হয়ে আসে:

```
Command to execute: | sort        # নির্বাচিত লাইনগুলো সাজিয়ে দাও
Command to execute: | sort -u     # সাজাও আর ডুপ্লিকেট বাদ দাও
```

(পাইপের ধারণা অধ্যায় ১৯-এ, `sort` অধ্যায় ২৩-এ।)

এই প্রম্পটের ভেতরে আরও কিছু টুল আছে (ভার্সন ভেদে): `^S` Spell check (`spell` বা `hunspell` ইনস্টল থাকলে), `^Y` Linter, `^O` Formatter, `^J` Full Justify। `F12` চাপলেও সরাসরি spell check শুরু হয়।

Justify: `^J`

প্যারাগ্রাফের লাইনগুলোকে সমান প্রস্থে সাজিয়ে দেয় (লম্বা লাইন ভেঙে, ছোট লাইন জুড়ে)। সাধারণ লেখা বা ইমেইলের জন্য। কোডে চাপলে `M-U` দিয়ে ফেরত নাও।

শব্দ গোনা: `M-D`

```
[ 342 words, 41 lines, 2105 characters ]
```

(mark থাকলে শুধু নির্বাচিত অংশের হিসাব।)

Verbatim input: `M-V`

`M-V` চাপার পর যে কী চাপবে সেটা "যেমন আছে" ফাইলে বসবে। যেমন Makefile-এ সত্যিকারের ট্যাব অক্ষর দরকার, অথচ তুমি tabstospaces চালু রেখেছ, তখন `M-V` তারপর `Tab`।

Anchor (nano 7+)

শর্টকাট	কাজ
`M-Ins`	বর্তমান লাইনে anchor (বুকমার্ক) বসাও/তোলো
`M-PgUp` / `M-PgDn`	আগের / পরের anchor-এ লাফ দাও

বড় ফাইলে দুই জায়গার মধ্যে বারবার যাওয়া-আসায় সুবিধা।

একাধিক ফাইল (Multiple Buffers)

```
$ nano index.html style.css app.js
```

তিনটি ফাইল তিনটি আলাদা buffer-এ খুলবে, title bar-এ দেখাবে কোনটায় আছ।

শর্টকাট	কাজ
`M-.` বা `M->`	পরের buffer
`M-,` বা `M-<`	আগের buffer
`^R` তারপর `M-F`	নতুন ফাইল নতুন buffer-এ খোলো
`^X`	শুধু বর্তমান buffer বন্ধ করো (শেষটা বন্ধ হলে nano বন্ধ)

এক ফাইল থেকে `^K` / `M-6` করে আরেক buffer-এ গিয়ে `^U` করলে পেস্ট হয়, কারণ cutbuffer সব buffer-এ এক।

ফাইল ব্রাউজার

`^O` বা `^R` প্রম্পটে `^T` চাপলে ডিরেক্টরির ফাইলগুলো তালিকায় দেখায়, অ্যারো দিয়ে বেছে Enter দাও। `^W` দিয়ে তালিকায় খোঁজাও যায়।

Read-only ও Restricted মোড

```
$ nano -v /etc/passwd     # দেখো, কিন্তু ভুল করেও বদলানো যাবে না
$ nano -R notes.txt       # restricted: অন্য ফাইল খোলা/সেভ/কমান্ড চালানো বন্ধ
```

শুধু পড়ার জন্য অবশ্য `less` আরো ভালো (অধ্যায় ১০)।

১৩.১৪ sudo দিয়ে সিস্টেম ফাইল এডিট (sudoedit)

`/etc/hosts`, `/etc/fstab`, `/etc/ssh/sshd_config` এর মালিক root। সাধারণ ইউজার পড়তে পারে, লিখতে পারে না। দুইভাবে এডিট করা যায়।

পদ্ধতি ১: `sudo nano` (সহজ, প্রচলিত)

```
$ sudo nano /etc/hosts
[sudo] password for rahim:
```

এখানে **পুরো nano প্রোগ্রামটাই root হিসেবে চলে।** কাজ হয়, কিন্তু কিছু ঝুঁকি আছে:

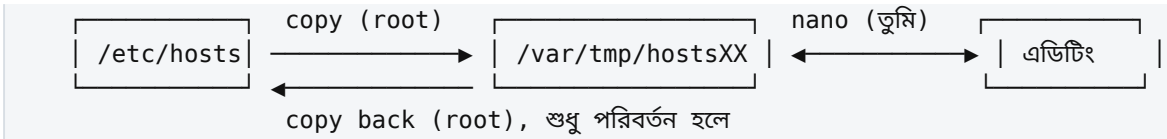
- nano-র ভেতর থেকে `^T` দিয়ে যে কমান্ড চালাবে সেটাও root হিসেবে চলবে।
- `^R` / `^O` দিয়ে যেকোনো ফাইল পড়া বা ওভাররাইট করা যাবে।
- nano-র কোনো বাগ থাকলে তা root ক্ষমতায় চলবে।
- কোন ইউজারের `~/.nanorc` পড়া হবে সেটা distro-র `sudo` কনফিগের উপর নির্ভর করে।

পদ্ধতি ২: `sudoedit` বা `sudo -e` (নিরাপদ, প্রস্তাবিত)

```
$ sudoedit /etc/hosts
# অথবা
$ sudo -e /etc/hosts
```

ভেতরে কী হয়:

```
১. sudo (root হিসেবে) /etc/hosts কপি করে → /var/tmp/hostsXXXXXX.  (তোমার মালিকানায়)
২. তোমার এডিটর (nano) চলে **তোমার নিজের** ইউজার হিসেবে, ঐ টেম্প কপিতে
৩. তুমি সেভ করে বের হলে
৪. sudo দেখে ফাইল বদলেছে কিনা → বদলালে root হিসেবে আসল /etc/hosts-এ কপি করে দেয়
৫. টেম্প ফাইল মুছে দেয়
```



সুবিধা:

- এডিটর root হিসেবে চলে না, তাই `^T` বা অন্য ফাইলে সেভ ইত্যাদি ঝুঁকি নেই।
- তোমার নিজের `~/.nanorc` সেটিং (রং, লাইন নম্বর) কাজ করে।
- `/etc/sudoers`-এ শুধু নির্দিষ্ট ফাইল এডিটের অনুমতি দেওয়া যায় (যেমন `rahim ALL = sudoedit /etc/hosts`)।

কোন এডিটর খুলবে সেটা sudoedit ঠিক করে এই ক্রমে: আগে `SUDO_EDITOR`, তারপর `VISUAL`, তারপর `EDITOR` ভেরিয়েবল দেখে; কোনোটাই না থাকলে sudo-র ডিফল্ট এডিটর (Ubuntu/Debian-এ সাধারণত `editor` alternatives-এর মাধ্যমে nano, অন্য distro-তে প্রায়ই `vi`)।

```
$ SUDO_EDITOR=nano sudoedit /etc/hosts
```

> ⚠️ **সতর্কতা:** `/etc/sudoers` ফাইল কখনো সরাসরি `sudo nano /etc/sudoers` দিয়ে এডিট করবে না। ভুল সিনট্যাক্স থাকলে sudo-ই আর কাজ করবে না, আর root ক্ষমতা হারাবে। সবসময় `sudo visudo` ব্যবহার করো, এটা সেভের আগে সিনট্যাক্স যাচাই করে। (`visudo` নামে "vi" থাকলেও এটা `EDITOR` / `SUDO_EDITOR` মেনে nano খুলতে পারে।) বিস্তারিত অধ্যায় ১৮-এ।

> ⚠️ **সতর্কতা:** systemd সার্ভিসের unit ফাইল সরাসরি এডিট না করে `sudo systemctl edit nginx` চালাও। এটা একটা override ফাইল তোমার এডিটরে খোলে (অধ্যায় ২৬)।

> 💡 **টিপস:** গুরুত্বপূর্ণ কনফিগ বদলানোর আগে ব্যাকআপ রাখো:
>
> ```
> $ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
> ```
>
> অথবা nano-কে `-B` দিয়ে চালাও, সেভের সময় সে নিজেই `sshd_config~` ব্যাকআপ রাখবে।

১৩.১৫ EDITOR ও VISUAL ভেরিয়েবল: ডিফল্ট এডিটর ঠিক করা

অনেক কমান্ড নিজে থেকে একটা এডিটর খোলে: `crontab -e`, `git commit`, `sudoedit`, `visudo`, `systemctl edit`, `less`-এ `v` চাপা, `fc` ইত্যাদি। কোন এডিটর খুলবে সেটা তারা জানে **environment variable** দেখে (অধ্যায় ৪৪):

ভেরিয়েবল	অর্থ
`VISUAL`	পূর্ণ-স্ক্রিন এডিটর (আধুনিক প্রোগ্রামগুলো আগে এটাই দেখে)
`EDITOR`	পুরোনো রীতির "line editor" ভেরিয়েবল; `VISUAL` না থাকলে এটা দেখা হয়

দুটোতেই nano সেট করে দেওয়া নিরাপদ।

বর্তমান সেটিং দেখা

```
$ echo "$EDITOR" "$VISUAL"

$                       # খালি মানে সেট করা নেই, প্রোগ্রাম তার নিজের ডিফল্ট নেবে
```

শুধু এই সেশনের জন্য

```
$ export EDITOR=nano
$ export VISUAL=nano
$ crontab -e            # এখন nano-তে খুলবে
```

স্থায়ীভাবে (প্রতিবার লগইনে)

`~/.bashrc` এর শেষে যোগ করো:

```
$ nano ~/.bashrc
# শেষে লেখো:
export EDITOR=nano
export VISUAL=nano
# সেভ করে বের হয়ে:
$ source ~/.bashrc
```

zsh হলে `~/.zshrc`-এ।

Debian/Ubuntu-র alternatives সিস্টেম

Debian পরিবারে `/usr/bin/editor` নামে একটা সিস্টেম-ব্যাপী ডিফল্ট এডিটর লিংক আছে। পরিবর্তন করতে:

```
$ sudo update-alternatives --config editor
There are 4 choices for the alternative editor (providing /usr/bin/editor).

  Selection    Path                Priority   Status
------------------------------------------------------------
* 0            /bin/nano            40        auto mode
  1            /bin/ed             -100       manual mode
  2            /bin/nano            40        manual mode
  3            /usr/bin/vim.basic   30        manual mode
  4            /usr/bin/vim.tiny    15        manual mode

Press <enter> to keep the current choice[*], or type selection number:
```

এছাড়া Ubuntu-তে `select-editor` কমান্ড আছে, যেটা ইউজার-ভিত্তিক পছন্দ `~/.selected_editor` ফাইলে রাখে (মূলত `crontab -e`, `sensible-editor` এটা মানে)। প্রথমবার `crontab -e` চালালে Ubuntu এটাই জিজ্ঞেস করে।

git-এর জন্য আলাদা

```
$ git config --global core.editor nano
```

vim খুলে গেলে আটকে গেছ?

ভুলক্রমে vim খুলে গেলে আর বের হতে না পারলে: `Esc` চাপো, তারপর `:q!` লিখে Enter (সেভ ছাড়া বের হওয়া)। এরপর উপরের মতো EDITOR ঠিক করে নাও। vim নিয়ে বিস্তারিত অধ্যায় ১৪-এ।

১৩.১৬ `~/.nanorc` দিয়ে nano সাজানো

nano চালু হওয়ার সময় কনফিগারেশন পড়ে এই ক্রমে:

```
১. /etc/nanorc                    ← সিস্টেম-ব্যাপী (সব ইউজারের জন্য, root এডিট করে)
২. ~/.nanorc   অথবা
   ~/.config/nano/nanorc          ← তোমার নিজস্ব (পরে পড়া হয়, তাই আগেরটা ওভাররাইড করে)
৩. কমান্ড-লাইন অপশন (nano -l ...) ← সবার শেষে, সবচেয়ে বেশি অগ্রাধিকার
```

(`~/.nanorc` থাকলে সেটাই পড়া হয়; না থাকলে `~/.config/nano/nanorc`। `nano -I` দিলে কোনো nanorc-ই পড়া হয় না।)

ফাইলের নিয়ম সহজ: প্রতি লাইনে একটা নির্দেশ, `#` দিয়ে কমেন্ট।

```
set   <option>          # কোনো সেটিং চালু
set   <option> <value>  # মানসহ সেটিং
unset <option>          # বন্ধ
include "<path>"        # অন্য nanorc ফাইল (যেমন syntax) যোগ
bind  <key> <function> <menu>   # শর্টকাট বদলানো
unbind <key> <menu>             # শর্টকাট সরানো
```

একটা ভালো শুরু-কনফিগ

```
$ nano ~/.nanorc
```

```
## ---------- প্রদর্শন ----------
set linenumbers          # বাঁয়ে লাইন নম্বর
set constantshow         # status bar-এ সবসময় কার্সর অবস্থান
set softwrap             # লম্বা লাইন স্ক্রিনে ভেঙে দেখাও
set atblanks             # softwrap শব্দের মাঝে না ভেঙে ফাঁকা জায়গায় ভাঙো
set indicator            # ডানে স্ক্রলবার-এর মতো অবস্থান নির্দেশক (nano 5+)
```

```
set minibar               # title bar-এর বদলে নিচে ছোট তথ্য-বার (nano 5.5+)
# set nohelp              # নিচের দুই লাইন help লুকাতে চাইলে

## ---------- লেখা ও কোড ----------
set autoindent            # নতুন লাইন আগের লাইনের ইনডেন্ট মেনে শুরু
set tabsize 4             # ট্যাবের প্রস্থ ৪ কলাম
set tabstospaces          # Tab চাপলে স্পেস বসাও (Python/YAML-এ কাজের)
set smarthome             # Home চাপলে প্রথমে লেখার শুরুতে, আবার চাপলে কলাম ১-এ
set trimblanks            # লাইন ভাঙলে শেষের ফাঁকা স্পেস মুছে দাও

## ---------- আচরণ ----------
set mouse                 # মাউস দিয়ে ক্লিক/সিলেক্ট
set zap                   # mark করা অংশ Delete করলে সরাসরি মুছে দাও
set historylog            # search/replace হিস্ট্রি সেভ রাখো
set positionlog           # ফাইল আবার খুললে শেষ কার্সর অবস্থানে খোলো
set locking               # vim-এর মতো lock ফাইল, দুইজন একসাথে একই ফাইল এডিট করলে সতর্ক
করবে

## ---------- ব্যাকআপ ----------
set backup                          # সেভের সময় পুরোনো সংস্করণ রাখো
set backupdir "~/.cache/nano-backups"   # ব্যাকআপ এই ফোল্ডারে (আগে ফোল্ডার বানাতে হবে)

## ---------- রং ----------
set titlecolor bold,white,blue
set statuscolor bold,white,green
set numbercolor cyan
set selectedcolor black,yellow

## ---------- syntax highlighting ----------
include "/usr/share/nano/*.nanorc"
# include "/usr/share/nano/extra/*.nanorc"    # Debian/Ubuntu-তে অতিরিক্ত কিছু ভাষা
```

ব্যাকআপ ফোল্ডার বানাও:

```
$ mkdir -p ~/.cache/nano-backups
```

এখন যেকোনো ফাইল খুললেই লাইন নম্বর, রং, অটো-ইনডেন্ট আসবে।

> ⚠️ **সতর্কতা:** কনফিগে ভুল অপশন থাকলে nano চালুর সময় এরকম error দেখাবে আর Enter চাইবে:
>
> ```
> Error in /home/rahim/.nanorc on line 7: Unknown option: "linenumber"
> ```
>
> লাইন নম্বর দেখে বানান ঠিক করো। যে অপশন তোমার ভার্সনে নেই (যেমন RHEL 9-এর nano 5.6-এ `minibar` নেই) সেটা মুছে দাও। নিজের ভার্সনের সব অপশন দেখতে `man nanorc`।

> 📝 **মনে রাখো:** পুরোনো গাইডে `set nowrap` দেখবে। nano 4.0 থেকে হার্ড র‍্যাপ ডিফল্টেই বন্ধ, তাই এটা আর লাগে না (নতুন ভার্সনে এটা obsolete, warning দিতে পারে)। উল্টোটা চাইলে `set breaklonglines`।

শর্টকাট বদলানো (`bind`)

ধরো তুমি চাও `Ctrl+Z` = undo আর `Ctrl+Y` = redo (অন্য এডিটরের অভ্যাস):

```
bind ^Z undo main
bind ^Y redo main
bind ^F whereis main
bind ^G findnext main
```

এখানে `main` মানে মূল এডিটিং এলাকা। অন্য মেনু: `search`, `replace`, `writeout`, `insert`, `help`, `all` ইত্যাদি। সব function-এর নাম `man nanorc`-এ "REBINDING KEYS" অংশে আছে।

নিজের syntax রং বানানো (সংক্ষেপে)

```
syntax "mylog" "\.log$"
color red    "ERROR.*"
color yellow "WARN.*"
color green  "INFO"
```

এখন যেকোনো `.log` ফাইলে ERROR লাল, WARN হলুদ দেখাবে।

সাধারণ কমান্ড-লাইন অপশন

একই সেটিং প্রায় সবই কমান্ড লাইনে অপশন হিসেবেও দেওয়া যায়:

অপশন (ছোট)	অপশন (লম্বা)	কাজ
`-l`	`--linenumbers`	লাইন নম্বর
`-m`	`--mouse`	মাউস সাপোর্ট
`-i`	`--autoindent`	অটো ইনডেন্ট
`-E`	`--tabstospaces`	ট্যাবকে স্পেসে রূপান্তর
`-T N`	`--tabsize=N`	ট্যাবের প্রস্থ
`-S`	`--softwrap`	soft wrap
`-c`	`--constantshow`	কার্সর অবস্থান সবসময়
`-B`	`--backup`	সেভের সময় `file~` ব্যাকআপ
`-C DIR`	`--backupdir=DIR`	ব্যাকআপ কোথায়
`-v`	`--view`	শুধু দেখা (read-only)
`-R`	`--restricted`	সীমিত মোড
`-Y NAME`	`--syntax=NAME`	নির্দিষ্ট syntax রং
`-x`	`--nohelp`	নিচের help lines লুকাও
`-w`	`--nowrap`	হার্ড র‍্যাপ বন্ধ (এখন ডিফল্ট)

অপশন (ছোট)	অপশন (লম্বা)	কাজ
`-k`	`--cutfromcursor`	`^K` কার্সর থেকে লাইনের শেষ পর্যন্ত কাটবে
`-I`	`--ignorercfiles`	কোনো nanorc পড়বে না
`-L`	`--nonewlines`	ফাইলের শেষে স্বয়ংক্রিয় newline যোগ করবে না
`-/`	`--modernbindings`	আধুনিক শর্টকাট (শুধু nano 8.0+)

উদাহরণ:

```
$ nano -liE -T 2 config.yaml    # লাইন নম্বর, অটো-ইনডেন্ট, ২-স্পেস ট্যাব
```

১৩.১৭ GUI বিকল্প: গ্রাফিক্যাল টেক্সট এডিটর

ডেস্কটপে কাজ করলে (অধ্যায় ৪) গ্রাফিক্যাল এডিটরও আরামদায়ক।

এডিটর	ডেস্কটপ	কমান্ড	বৈশিষ্ট্য
GNOME Text Editor	GNOME (Ubuntu 22.10+, Fedora)	`gnome-text-editor`	Ubuntu-তে মেনুতে নাম "Text Editor"; gedit-এর উত্তরসূরি
gedit	GNOME (পুরোনো ডিফল্ট)	`gedit`	এখনো `sudo apt install gedit` দিয়ে পাওয়া যায়, প্লাগইন আছে
Kate	KDE Plasma	`kate`	খুব শক্তিশালী: ট্যাব, স্প্লিট ভিউ, বিল্ট-ইন টার্মিনাল, LSP
KWrite	KDE	`kwrite`	Kate-এর হালকা সংস্করণ
Mousepad	XFCE	`mousepad`	হালকা, দ্রুত
Xed	Linux Mint (Cinnamon)	`xed`	gedit-এর মতো
VS Code	যেকোনো	`code`	প্রোগ্রামারদের জন্য পূর্ণ IDE-এর মতো

টার্মিনাল থেকে GUI এডিটর খোলা:

```
$ gnome-text-editor notes.txt &     # & দিলে টার্মিনাল আটকে থাকবে না
$ kate notes.txt &
```

GUI এডিটরে সিস্টেম ফাইল: `sudo gedit` করবে না

> ⚠️ **সতর্কতা:** `sudo gedit /etc/hosts` বা `sudo kate ...` চালানো খারাপ অভ্যাস। পুরো গ্রাফিক্যাল অ্যাপ root হিসেবে চললে তোমার হোম ডিরেক্টরিতে root-মালিকানার কনফিগ ফাইল তৈরি হতে পারে (পরে

> অদ্ভুত সমস্যা), আর Wayland-এ অনেক ক্ষেত্রে root GUI অ্যাপ চালুই হয় না। Kate তো সরাসরি root হিসেবে চলতে অস্বীকার করে।

নিরাপদ উপায়:

উপায়	কীভাবে
sudoedit + GUI এডিটর	`SUDO_EDITOR="gnome-text-editor --standalone" sudoedit /etc/hosts` (এডিটর বন্ধ না হওয়া পর্যন্ত অপেক্ষা করতে হবে, তাই standalone/নতুন উইন্ডো মোড দরকার)
GNOME `admin://`	`gnome-text-editor admin:///etc/hosts` বা Files অ্যাপে admin:// দিয়ে; সেভের সময় polkit পাসওয়ার্ড চাইবে
Kate/KWrite	সাধারণ ইউজার হিসেবে খোলো `kate /etc/hosts`; সেভের সময় নিজেই polkit দিয়ে পাসওয়ার্ড চাইবে
সবচেয়ে সহজ	টার্মিনালে `sudoedit /etc/hosts` (nano)

> 💡 **টিপস:** সার্ভারে কাজ করতে হলে GUI থাকবেই না, তাই GUI এডিটর থাকলেও nano-তে হাত পাকানো জরুরি।

১৩.১৮ nano-র সম্পূর্ণ শর্টকাট টেবিল

নিচের তালিকা nano 7.x অনুযায়ী (Ubuntu 24.04 / Debian 12)। পুরোনো ভার্সনে কিছু কম থাকতে পারে; নিশ্চিত হতে `^G`।

ফাইল ও প্রোগ্রাম

শর্টকাট	ফাংশন কী	কাজ
`^G`	F1	Help (হেল্প পাতা; ভেতরে `^X` দিয়ে বের হও)
`^X`	F2	Exit / বর্তমান buffer বন্ধ
`^O`	F3	Write Out (সেভ, নাম জিজ্ঞেস করে)
`^S`	—	Save (প্রশ্ন ছাড়া সেভ)
`^R`	F5, Insert	Read File (অন্য ফাইল ঢোকাও)
`^T`	—	Execute (বাইরের কমান্ড চালাও / filter)
`^Z`	—	Suspend (শেলে ফেরো; `fg` দিয়ে ফেরত)
`M-.` / `M->`	—	পরের buffer
`M-,` / `M-<`	—	আগের buffer

সম্পাদনা

শর্টকাট	ফাংশন কী	কাজ
`^K`	F9	Cut: লাইন বা mark করা অংশ কাটো
`^U`	F10	Paste (Uncut)
`M-6` / `M-^`	—	Copy: লাইন বা mark করা অংশ কপি
`M-A` / `^6`	—	Mark বসাও / তোলো
`M-T`	—	কার্সর থেকে ফাইলের শেষ পর্যন্ত কাটো
`M-U`	—	Undo
`M-E`	—	Redo
`^D`	Delete	ডানের অক্ষর মোছো
`^H`	Backspace	বাঁয়ের অক্ষর মোছো
`M-Bsp`	—	বাঁয়ের শব্দ মোছো
`^Del`	—	ডানের শব্দ মোছো
`^J`	F4	Justify (প্যারাগ্রাফ সাজাও)
`M-J`	—	পুরো ফাইল Justify
`M-}` / Tab	—	ইনডেন্ট
`M-{` / Sh-Tab	—	আনইনডেন্ট
`M-3`	—	কমেন্ট/আনকমেন্ট
`M-V`	—	Verbatim input
`M-D`	—	শব্দ, লাইন, অক্ষর গোনা
`F12`	—	Spell check

খোঁজা ও প্রতিস্থাপন

শর্টকাট	ফাংশন কী	কাজ
`^W`	F6	Where Is (সামনে খোঁজো)
`^Q`	—	Where Was (পেছনে খোঁজো)
`M-W`	—	পরের মিল
`M-Q`	—	আগের মিল
`^\` / `M-R`	—	Replace
(প্রম্পটে) `M-C`	—	Case sensitive টগল
(প্রম্পটে) `M-R`	—	Regex টগল
(প্রম্পটে) `M-B`	—	Backwards টগল

চলাফেরা

শর্টকাট	বিকল্প	কাজ
`^F` / `^B`	→ / ←	এক অক্ষর সামনে / পেছনে
`^N` / `^P`	↓ / ↑	পরের / আগের লাইন
`^Space` / `M-Space`	Ctrl+→ / Ctrl+←	পরের / আগের শব্দ
`^A` / `^E`	Home / End	লাইনের শুরু / শেষ
`^Y` / `^V`	PgUp / PgDn (F7 / F8)	এক পাতা ওপরে / নিচে
`M-\` / `M-/`	Ctrl+Home / Ctrl+End	ফাইলের শুরু / শেষ
`^/` / `^_` / `M-G`	—	লাইন নম্বরে যাও
`^C`	F11	কার্সরের অবস্থান দেখাও
`M-]`	—	মিলে যাওয়া বন্ধনী
`M-(` / `M-)`	—	আগের / পরের প্যারাগ্রাফ বা ব্লক
`M--` / `M-+`	—	স্ক্রিন ওপরে / নিচে স্ক্রল
`^L`	—	কার্সর লাইন মাঝে/ওপরে/নিচে
`M-Ins`	—	Anchor বসাও/তোলো
`M-PgUp` / `M-PgDn`	—	আগের / পরের anchor

টগল (চালু/বন্ধ)

শর্টকাট	কাজ
`M-N`	লাইন নম্বর
`M-S`	Soft wrap
`M-L`	Hard wrap
`M-P`	Whitespace দেখাও
`M-Y`	Syntax রং
`M-X`	Help lines
`M-C`	সবসময় কার্সর অবস্থান
`M-I`	Auto-indent
`M-O`	Tab → space
`M-M`	মাউস
`M-K`	Cut from cursor
`M-Z`	Zap

প্রম্পটের সাধারণ কী

শর্টকাট	কাজ
`Enter`	নিশ্চিত করো
`^C`	বাতিল (Cancel)
`↑` / `↓`	আগের লেখা হিস্ট্রি
`Tab`	ফাইলের নাম অটো-কমপ্লিট (ফাইলের নাম প্রম্পটে)
`Y` / `N` / `A`	হ্যাঁ / না / সবগুলো

১৩.১৯ সাধারণ সমস্যা ও সমাধান

সমস্যা	কারণ	সমাধান
`[ Error writing ...: Permission denied ]`	ফাইলে লেখার অনুমতি নেই	`^X` → `N` করে বের হয়ে `sudoedit file`। বা অন্য নামে (যেমন `~/file.new`) সেভ করে পরে `sudo cp`
`Directory '...' does not exist`	ভুল পাথে নতুন ফাইল	আগে `mkdir -p` দিয়ে ডিরেক্টরি বানাও
`File '...' is being edited (by rahim with nano ...)`	`set locking`-এ অন্য কেউ ফাইলটা খোলা রেখেছে, বা আগের nano ক্র্যাশ করেছে	অন্য সেশন বন্ধ করো, নিশ্চিত হলে `.file.swp` lock ফাইল মুছে দাও
হঠাৎ nano উধাও, শেলে `[1]+ Stopped nano`	ভুলে `^Z` চেপেছ	`fg`
Alt শর্টকাট কাজ করছে না	টার্মিনাল Alt ধরে নিচ্ছে	`Esc` তারপর অক্ষর
পেস্ট করা কোডের ইনডেন্ট সিঁড়ির মতো বেড়ে যাচ্ছে	auto-indent চালু	পেস্টের আগে `M-I` দিয়ে বন্ধ করো, পরে আবার চালু
Windows থেকে আনা ফাইলে `^M` বা অদ্ভুত আচরণ	CRLF লাইন-এন্ডিং	nano খোলার সময় নিজেই চেনে ও `[ Read 10 lines (Converted from DOS format) ]` দেখায়; সেভ প্রম্পটে `M-D` না চাপলে Unix ফরম্যাটে সেভ হবে
ফাইলের শেষে নতুন লাইন যোগ হয়ে গেল	nano ডিফল্টে শেষে newline রাখে (Unix রীতি)	দরকার না হলে `nano -L`

হাতে-কলমে অনুশীলন

ল্যাব ১: প্রথম ফাইল তৈরি ও সেভ

```
$ mkdir -p ~/nano-lab && cd ~/nano-lab
$ nano shopping.txt
```

১. নিচের লেখা টাইপ করো:

```
চাল
ডাল
তেল
লবণ
চিনি
```

২. `^O` → Enter দিয়ে সেভ করো, status bar-এ `[ Wrote 5 lines ]` দেখো। ৩. `^X` দিয়ে বের হও। ৪. যাচাই:

```
$ cat shopping.txt
চাল
ডাল
তেল
লবণ
চিনি
```

ল্যাব ২: কাট, পেস্ট, undo

১. `nano shopping.txt` ২. "তেল" লাইনে যাও, `^K` চাপো (কাটা হলো)। ৩. `M-/` দিয়ে শেষে যাও, `^U` (পেস্ট)। ৪. `M-U` চেপে দেখো পেস্ট ফেরত গেল, `M-E` চেপে আবার এল। ৫. "চাল" লাইনে গিয়ে `M-6` (কপি), তারপর `^U` দুবার, দুটো নতুন "চাল" লাইন হলো। ৬. `^X` → `N` (সেভ না করে বের হও)। `cat` দিয়ে দেখো ফাইল আগের মতোই আছে, বুঝলে buffer আর ডিস্কের পার্থক্য।

ল্যাব ৩: খোঁজা ও প্রতিস্থাপন

```
$ printf 'server=localhost\nport=8080\ndb_host=localhost\nbackup=localhost\n' >
app.conf
$ nano app.conf
```

১. `^W` → `localhost` → Enter। তারপর `M-W` চেপে পরের মিলগুলোতে যাও। ২. `^\` → `localhost` → Enter → `127.0.0.1` → Enter। ৩. প্রথমটায় `Y`, দ্বিতীয়টায় `N`, তারপর `A`। ৪. `^S` দিয়ে সেভ, `^X`।

```
$ cat app.conf
server=127.0.0.1
port=8080
```

```
db_host=localhost
backup=127.0.0.1
```

ল্যাব ৪: লাইন নম্বর, go-to ও কমেন্ট

```
$ seq 1 100 > numbers.txt
$ nano -l +50 numbers.txt
```

১. কার্সর ৫০ নম্বর লাইনে আছে কিনা দেখো, `^C` চেপে নিশ্চিত হও। ২. `^/` → `75` → Enter। ৩. `M-N` দিয়ে লাইন নম্বর বন্ধ, আবার চালু করো। ৪. `M-A` চেপে mark দাও, ↓ চারবার, তারপর `M-3`, পাঁচটা লাইন কমেন্ট হলো (`#` লাগল)। আবার `M-3` চেপে ফেরত। ৫. সেভ না করে বের হও।

ল্যাব ৫: Execute ও filter

১. `nano fruits.txt`, লেখো: `mango`, `apple`, `banana`, `apple`, `cherry` (আলাদা লাইনে)। ২. `M-\` দিয়ে শুরুতে, `M-A` mark, `M-/` শেষে যাও। ৩. `^T` → `| sort -u` → Enter। দেখো লাইনগুলো সাজানো আর ডুপ্লিকেট বাদ। ৪. শেষ লাইনে গিয়ে `^T` → `date` → Enter, আজকের তারিখ বসে গেল।

ল্যাব ৬: নিজের `~/.nanorc`

১. `nano ~/.nanorc` খোলো, এই কয়টা লাইন লেখো:

```
set linenumbers
set autoindent
set tabsize 4
set tabstospaces
set mouse
include "/usr/share/nano/*.nanorc"
```

২. সেভ করে বের হও। ৩. একটা স্ক্রিপ্ট খোলো:

```
$ nano test.sh
```

লেখো:

```
#!/bin/bash
# এটা কমেন্ট
name="Linux"
echo "Hello $name"
```

সেভ করে বের হয়ে আবার খোলো, রং আর লাইন নম্বর দেখো।

ল্যাব ৭: EDITOR ও sudoedit

```
$ echo 'export EDITOR=nano' >> ~/.bashrc
$ echo 'export VISUAL=nano' >> ~/.bashrc
$ source ~/.bashrc
$ echo $EDITOR
nano
$ sudoedit /etc/hosts
```

১. শেষে একটা লাইন যোগ করো: `127.0.0.1 mytest.local` ২. সেভ করে বের হও। ৩. যাচাই:

```
$ ping -c 1 mytest.local
PING mytest.local (127.0.0.1) 56(84) bytes of data.
64 bytes from localhost (127.0.0.1): icmp_seq=1 ttl=64 time=0.040 ms
```

৪. কাজ শেষে আবার `sudoedit /etc/hosts` দিয়ে লাইনটা মুছে দাও (`^K`), সেভ করো।

ল্যাব ৮: crash recovery দেখা

১. একটা টার্মিনালে `nano crash.txt` , কিছু লেখো, **সেভ করো না।** ২. আরেকটা টার্মিনাল থেকে:

```
$ pkill -HUP nano
```

৩. প্রথম টার্মিনালে দেখবে nano বন্ধ হয়ে গেছে।

```
$ ls ~/nano-lab
crash.txt.save  ...
$ cat crash.txt.save
```

লেখা বেঁচে আছে!

সারসংক্ষেপ

nano হলো লিনাক্সের সবচেয়ে সহজ টার্মিনাল এডিটর। এটা ফাইলকে মেমরিতে একটা buffer হিসেবে খোলে, সেভ না করা পর্যন্ত ডিস্কের ফাইল বদলায় না। `^` = Ctrl, `M-` = Alt (বা Esc)। সিস্টেম ফাইল নিরাপদে এডিট করতে `sudoedit` , ডিফল্ট এডিটর ঠিক করতে `EDITOR` / `VISUAL` , আর নিজের পছন্দমতো সাজাতে `~/.nanorc` ।

দ্রুত রেফারেন্স

কাজ	কমান্ড / শর্টকাট
ফাইল খোলা	`nano file`
নির্দিষ্ট লাইনে খোলা	`nano +N file`

কাজ	কমান্ড / শর্টকাট
লাইন নম্বরসহ খোলা	`nano -l file`
শুধু পড়া	`nano -v file`
সেভ (নাম জিজ্ঞেসসহ)	`^O`
সেভ (সরাসরি)	`^S`
বের হওয়া	`^X`
হেল্প	`^G`
খোঁজা / পরেরটা / আগেরটা	`^W` / `M-W` / `M-Q`
প্রতিস্থাপন	`^\`
লাইন কাটো / পেস্ট	`^K` / `^U`
Mark / কপি	`M-A` / `M-6`
Undo / Redo	`M-U` / `M-E`
লাইনে যাও	`^/` (বা `^_`)
অবস্থান দেখো	`^C`
ফাইলের শুরু / শেষ	`M-\` / `M-/`
লাইন নম্বর টগল	`M-N`
কমেন্ট টগল	`M-3`
কমান্ড চালাও / filter	`^T`
অন্য ফাইল ঢোকাও	`^R`
নিরাপদে সিস্টেম ফাইল এডিট	`sudoedit file` / `sudo -e file`
sudoers এডিট	`sudo visudo`
ডিফল্ট এডিটর	`export EDITOR=nano VISUAL=nano`
সিস্টেম ডিফল্ট (Debian/Ubuntu)	`sudo update-alternatives --config editor`
ব্যক্তিগত কনফিগ	`~/.nanorc` বা `~/.config/nano/nanorc`
কনফিগ ম্যানুয়াল	`man nano`, `man nanorc`

নিজেকে যাচাই করো

১. nano-তে `^O` আর `M-U` লেখা থাকলে আসলে কোন কী চাপতে হবে?

২. "buffer" কী? সেভ না করে nano বন্ধ করলে ডিস্কের ফাইলের কী হয়?

৩. ফাইলের ৪টি লাইন এক জায়গা থেকে কেটে অন্য জায়গায় নেওয়ার ধাপ লেখো।

৪. শুধু একটা শব্দের অংশ (পুরো লাইন না) কপি করতে কী করবে?

৫. `sudo nano /etc/hosts` আর `sudoedit /etc/hosts` এর মধ্যে পার্থক্য কী? কোনটা নিরাপদ এবং কেন?

৬. `crontab -e` চালালে vim খুলছে, কিন্তু তুমি nano চাও। কী করবে (স্থায়ীভাবে)?

৭. nano-তে ভুল করে `Ctrl+Z` চাপার পর nano চলে গেছে। কী হয়েছে, আর কীভাবে ফেরত আনবে?

৮. প্রতিবার nano খুললেই লাইন নম্বর আর ৪-স্পেস ট্যাব চাইলে কোথায় কী লিখবে?

৯. ফাইলে সব `http://` কে `https://` বানাতে nano-তে কী করবে?

১০. GNOME ডেস্কটপে `/etc/fstab` গ্রাফিক্যালি এডিট করতে `sudo gedit` কেন ব্যবহার করা উচিত নয়, বিকল্প কী?

▼ উত্তর দেখো

১. `^O` = `Ctrl+O` (Write Out / সেভ)। `M-U` = `Alt+U` (Undo); Alt কাজ না করলে `Esc` চেপে ছেড়ে তারপর `U`। Shift চাপতে হয় না।

২. buffer হলো RAM-এ রাখা ফাইলের কাজের কপি, এডিট সব এখানেই হয়। সেভ না করে বের হলে (`^X` → `N`) ডিস্কের ফাইল একদম আগের মতো থাকে, পরিবর্তন হারিয়ে যায়। (দুর্ঘটনায় বন্ধ হলে nano `file.save` বানিয়ে রাখে।)

৩. প্রথম লাইনে কার্সর রাখো → `^K` চারবার পরপর (চারটি লাইন cutbuffer-এ জমা হবে) → নতুন জায়গায় কার্সর নাও → `^U`।

৪. অংশের শুরুতে কার্সর রেখে `M-A` (বা `^6`) দিয়ে mark বসাও, অ্যারো দিয়ে শেষ পর্যন্ত নির্বাচন করো, `M-6` দিয়ে কপি করো, তারপর যেখানে দরকার `^U`।

৫. `sudo nano` পুরো এডিটরটাকে root হিসেবে চালায়, তাই এর ভেতর থেকে চালানো কমান্ড (`^T`), অন্য ফাইল পড়া/লেখা সবই root ক্ষমতায় হয়। `sudoedit` ফাইলের একটা টেম্প কপি বানিয়ে এডিটরটা তোমার সাধারণ ইউজার হিসেবে চালায়, সেভের পর শুধু ফাইলটা root হিসেবে আসল জায়গায় কপি করে। তাই `sudoedit` বেশি নিরাপদ, আর তোমার নিজের `~/.nanorc`-ও কাজ করে।

৬. `~/.bashrc`-এ `export EDITOR=nano` আর `export VISUAL=nano` যোগ করে `source ~/.bashrc`। Ubuntu-তে `select-editor` চালিয়ে nano বাছাই করলেও `crontab -e`-র জন্য কাজ হয়। সিস্টেম-ব্যাপী: `sudo update-alternatives --config editor`।

৭. `^Z` nano-কে suspend করেছে (পেছনে থামিয়ে রেখেছে), কিছু হারায়নি। শেলে `fg` লিখে Enter দিলে nano ঠিক আগের অবস্থায় ফিরে আসবে।

৮. `~/.nanorc` (বা `~/.config/nano/nanorc`) ফাইলে:

```
set linenumbers
set tabsize 4
set tabstospaces
```

৯. `^\` চাপো → `http://` → Enter → `https://` → Enter → প্রথম প্রশ্নে `A` (সবগুলো)। শুধু কিছু বদলাতে চাইলে প্রতিটায় `Y` / `N`। (এখানে regex দরকার নেই।)

১০. root হিসেবে GUI অ্যাপ চালালে হোম ডিরেক্টরিতে root-মালিকানার কনফিগ ফাইল তৈরি হতে পারে আর Wayland-এ প্রায়ই চালুই হয় না, তাছাড়া পুরো বড় GUI অ্যাপকে root ক্ষমতা দেওয়া নিরাপত্তা-ঝুঁকি। বিকল্প: `gnome-text-editor admin:///etc/fstab` (সেভের সময় polkit পাসওয়ার্ড চায়), অথবা টার্মিনালে `sudoedit /etc/fstab`।

অধ্যায় ১৪: Vim — সম্পূর্ণ গাইড

> "Vim থেকে বের হতে না পেরে কম্পিউটার রিস্টার্ট দিয়েছি" — এটা ইন্টারনেটের সবচেয়ে পুরোনো রসিকতাগুলোর একটা। এই অধ্যায় শেষ করলে তুমি শুধু বের হতেই পারবে না, Vim-এর "ভাষা"য় কথা বলতে শিখবে, আর বুঝবে কেন লক্ষ লক্ষ প্রোগ্রামার আর সিস্টেম অ্যাডমিন এই এডিটর ছাড়তে চায় না।

এই অধ্যায়ে যা শিখবে

- vi, Vim আর Neovim-এর ইতিহাস ও পার্থক্য; কোন ডিস্ট্রোতে কোনটা আসে আর কীভাবে ইনস্টল করবে
- Vim কেন "modal" এডিটর, আর পাঁচটা মূল mode: Normal, Insert, Visual, Command-line, Replace
- Vim থেকে নিরাপদে বের হওয়া: `:q`, `:q!`, `:w`, `:wq`, `:x`, `ZZ`, `ZQ`
- কার্সর চালানো: `h j k l`, শব্দ ধরে (`w b e`), লাইনের ভেতরে (`0 ^ $ f t`), ফাইল জুড়ে (`gg G nG % { }`), স্ক্রিন স্ক্রল (`Ctrl+f/b/d/u`)
- এডিট করা: `i a o O I A`, `x`, `dd`, `yy`, `p P`, `u`, `Ctrl+r`, `.`, `r`, `c`, `cw`, `ciw` ইত্যাদি
- Vim-এর আসল শক্তি: **operator + count + motion** ব্যাকরণ, আর **text objects** (`iw`, `ap`, `i"`, `a(`, `it` ...)
- সার্চ (`/ ? n N * #`) আর খুঁজে-বদলানো (`:%s/old/new/gc`), `:g` কমান্ড
- Visual mode, line-visual আর block-visual (একসাথে অনেক লাইনে লেখা)
- Registers (Vim-এর অনেকগুলো ক্লিপবোর্ড), সিস্টেম ক্লিপবোর্ড
- Macros (`q`) দিয়ে পুনরাবৃত্তিমূলক কাজ অটোমেট করা
- Marks আর jumplist দিয়ে ফাইলের ভেতরে লাফানো
- একাধিক ফাইল: buffers, windows (split), tabs
- `~/.vimrc` দিয়ে নিজের মতো Vim সাজানো, প্লাগইনের ধারণা
- swap file, crash recovery, undo tree, `vimtutor` আর `:help` ব্যবহার
- সব শর্টকাটের একটা বিশাল রেফারেন্স টেবিল

১৪.১ কেন Vim শিখবে?

আগের অধ্যায়ে (অধ্যায় ১৩) আমরা `nano` শিখেছি — সহজ, নিচে শর্টকাট লেখা থাকে, খুললেই টাইপ করা যায়। তাহলে Vim কেন?

1. **সব জায়গায় আছে।** প্রায় প্রতিটা লিনাক্স/ইউনিক্স সিস্টেমে কোনো না কোনো `vi` থাকে — ছোট্ট রাউটার, Docker কন্টেইনার, রেসকিউ মোড, পুরোনো সার্ভার। POSIX স্ট্যান্ডার্ডও `vi`-কে আবশ্যিক ইউটিলিটি

হিসেবে ধরে। `nano` না থাকলেও `vi` প্রায় সবসময় পাবে।

2. **হাত কিবোর্ড থেকে সরাতে হয় না।** মাউস, অ্যারো কি, Home/End — কিছুই লাগে না। হাত "home row"-তে (`asdf jkl;`) রেখেই সব কাজ।
3. **এডিটিং একটা ভাষা।** `d2w` মানে "delete 2 words", `ci"` মানে "change inside quotes"। কয়েকটা "শব্দ" শিখলে হাজারটা কম্বিনেশন নিজে থেকেই পারবে।
4. **রিমোট সার্ভারে আদর্শ।** SSH দিয়ে ধীর নেটওয়ার্কে কাজ করলেও Vim হালকা আর দ্রুত (অধ্যায় ৩৮)।
5. **অন্য টুলেও Vim-এর শর্টকাট।** `less` , `man` , `more` -এ `/` দিয়ে সার্চ, `j/k` দিয়ে চলা; bash-এ `set -o vi` ; VS Code, JetBrains, এমনকি ব্রাউজারেও Vim-emulation প্লাগইন আছে।

> 💡 **টিপস:** Vim শেখার প্রথম সপ্তাহ একটু ধীর লাগবে — সাইকেল চালানো শেখার মতো। কিন্তু একবার "মাসল মেমোরি" তৈরি হলে আর ফিরে যেতে ইচ্ছা করবে না। লক্ষ্য রাখো: প্রথম দিন শুধু খোলা-লেখা-সেভ-বের হওয়া; তারপর প্রতিদিন ২-৩টা নতুন কমান্ড।

১৪.২ ইতিহাস: ed → ex → vi → Vim → Neovim

```
1971  ed      (Ken Thompson, Bell Labs)  — line editor, এক লাইন করে দেখে
  |
1976  ex      (Bill Joy, UC Berkeley)    — ed-এর উন্নত রূপ
  |
1976-79 vi    (Bill Joy)                 — ex-এর "visual" mode, পুরো স্ক্রিন দেখায়
  |
1991  Vim     (Bram Moolenaar, Amiga)    — "Vi IMproved": undo tree, syntax highlight,
  |                                          split window, plugin, scripting
  |
2014  Neovim  (community fork)           — Lua দিয়ে কনফিগ, built-in LSP, async
```

- **`ed`** : তখনকার টার্মিনাল ছিল কাগজে প্রিন্ট করা টেলিটাইপ, স্ক্রিন ছিল না। তাই এডিটর এক লাইন করে কাজ করত। `:s/old/new/` , `:g/pattern/` — এই কমান্ডগুলো আজও Vim-এ আছে, এগুলো `ed` থেকেই এসেছে।
- **`vi`** : স্ক্রিনওয়ালা টার্মিনাল (ADM-3A) আসার পর Bill Joy `ex` -এ একটা "visual" mode বানালেন — সেটাই `vi` । মজার তথ্য: ADM-3A কিবোর্ডে অ্যারো চিহ্ন আঁকা ছিল `h j k l` কি-র ওপর, আর Escape কি ছিল এখনকার Tab-এর জায়গায়। তাই Vim-এ `hjkl` দিয়ে চলা আর Esc-এর এত ব্যবহার।
- **Vim**: ডাচ প্রোগ্রামার Bram Moolenaar ১৯৯১ সালে Amiga কম্পিউটারের জন্য প্রথম প্রকাশ করেন (প্রথমে নাম ছিল "Vi IMitation", পরে "Vi IMproved")। Vim চ্যারিটিওয়্যার — ব্যবহারকারীদের উগান্ডার শিশুদের জন্য দান করতে উৎসাহিত করা হতো। ২০২৩ সালের আগস্টে Bram মারা যান; এখন একটা কমিউনিটি টিম Vim রক্ষণাবেক্ষণ করে। Vim 9.0 (২০২২) এনেছে নতুন দ্রুত স্ক্রিপ্ট ভাষা Vim9 script; Vim 9.1 বের হয় ২০২৪-এর জানুয়ারিতে।
- **Neovim**: ২০১৪ সালে Vim-এর কোডবেস আধুনিক করার লক্ষ্যে fork। কনফিগ Lua-তে লেখা যায় (`~/.config/nvim/init.lua`), built-in LSP client (কোড কমপ্লিশন, এরর দেখানো), Tree-sitter,

async job। কমান্ড `nvim`। Normal-mode কিবাইন্ডিং প্রায় হুবহু Vim-এর মতো — তাই এই অধ্যায়ের ৯৫% Neovim-এও খাটে।

১৪.২.১ কোন ডিস্ট্রোতে কী থাকে

ডিস্ট্রো	ডিফল্টে কী আছে	পূর্ণ Vim ইনস্টল
Ubuntu 24.04 / Debian 12	`vim-tiny` (কমান্ড `vi`; অনেক ফিচার বাদ)	`sudo apt install vim`
Ubuntu/Debian (ক্লিপবোর্ড সাপোর্টসহ)	—	`sudo apt install vim-gtk3`
Fedora / RHEL 9 / Rocky / Alma	`vim-minimal` (কমান্ড `vi`)	`sudo dnf install vim-enhanced`
Arch	—	`sudo pacman -S vim` (বা `gvim` ক্লিপবোর্ডের জন্য)
Neovim	—	`sudo apt install neovim` / `sudo dnf install neovim`

```
$ vim --version | head -3
VIM - Vi IMproved 9.1 (2024 Jan 02, compiled ...)
Included patches: 1-16
Modified by team+vim@tracker.debian.org
```

> ⚠️ **সতর্কতা:** Ubuntu-তে ডিফল্ট `vi` আসলে `vim.tiny` — এতে syntax highlighting নেই, অনেক ফিচার বন্ধ। নতুনদের প্রায়ই মনে হয় "Vim এত কুৎসিত কেন, arrow key-তে A B C D লেখা হয় কেন!" সমাধান: পূর্ণ `vim` প্যাকেজ ইনস্টল করো।

`vim --version`-এর আউটপুটে `+clipboard` / `-clipboard`, `+python3` ইত্যাদি দেখে বুঝবে কোন ফিচার কম্পাইল করা আছে (`+` মানে আছে, `-` মানে নেই)।

১৪.২.২ Vim-কে ডিফল্ট এডিটর বানানো

`git commit`, `crontab -e`, `visudo`, `sudoedit` — এরা `EDITOR` / `VISUAL` এনভায়রনমেন্ট ভ্যারিয়েবল দেখে এডিটর খোলে।

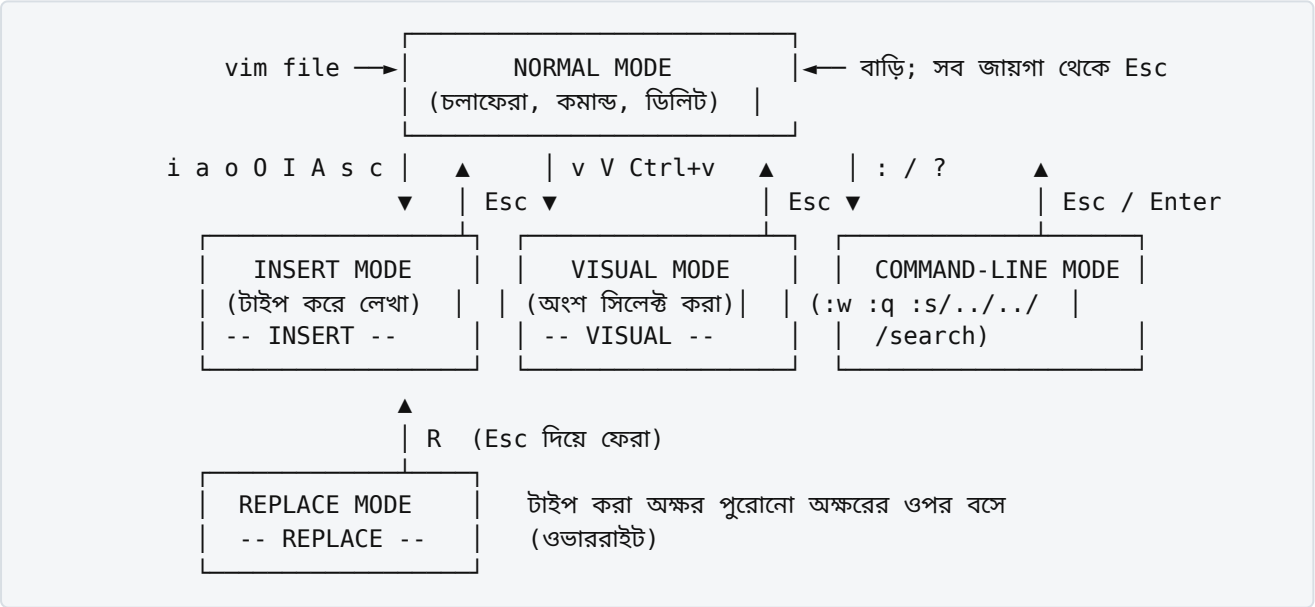
```
# ~/.bashrc-এ যোগ করো (বিস্তারিত: অধ্যায় ৪৪)
export EDITOR=vim
export VISUAL=vim

# Debian/Ubuntu-তে সিস্টেম-জুড়ে ডিফল্ট বদলানো
sudo update-alternatives --config editor
```

১৪.৩ Vim-এর মূল দর্শন: Modal Editing

সাধারণ এডিটরে (Notepad, nano) যে কি চাপো সেটাই লেখা হয়। কমান্ড দিতে হলে `Ctrl` / `Alt` চাপতে হয়। Vim ভিন্ন: এর আলাদা আলাদা **mode** (অবস্থা) আছে, আর একই কি ভিন্ন mode-এ ভিন্ন কাজ করে।

উপমা: একজন চিত্রশিল্পীর কথা ভাবো। বেশিরভাগ সময় সে ছবির দিকে তাকিয়ে চিন্তা করে, মাপজোখ করে, কোথায় তুলি দেবে ঠিক করে (এটা **Normal mode**)। তারপর কয়েক সেকেন্ডের জন্য তুলি তুলে রং লাগায় (এটা **Insert mode**)। আবার তুলি নামিয়ে দেখে। প্রোগ্রামাররাও তাই করে — লেখার চেয়ে কোড **পড়া, চলাফেরা আর বদলানো**তেই বেশি সময় যায়। তাই Vim-এর "বাড়ি" হলো Normal mode, যেখানে প্রতিটা কি একটা শক্তিশালী কমান্ড।

১৪.৩.১ mode-গুলোর মানচিত্র



Mode	কীভাবে ঢুকবে	স্ট্যাটাস লাইনে	কাজ
Normal	Vim খুললেই; অন্য যেকোনো mode থেকে `Esc`	(কিছু লেখা থাকে না)	চলাফেরা, ডিলিট, কপি, পেস্ট, কমান্ড
Insert	`i`, `a`, `o`, `O`, `I`, `A`, `s`, `S`, `c…`	`-- INSERT --`	সাধারণ টাইপিং
Visual	`v` (অক্ষর), `V` (লাইন), `Ctrl+v` (ব্লক)	`-- VISUAL --` / `-- VISUAL LINE --` / `-- VISUAL BLOCK --`	টেক্সট সিলেক্ট করে তার ওপর কাজ
Command-line	`:` (Ex কমান্ড), `/` বা `?` (সার্চ)	নিচে `:` বা `/` দেখায়	সেভ, বের হওয়া, replace, সেটিং
Replace	`R`	`-- REPLACE --`	ওভাররাইট করে লেখা

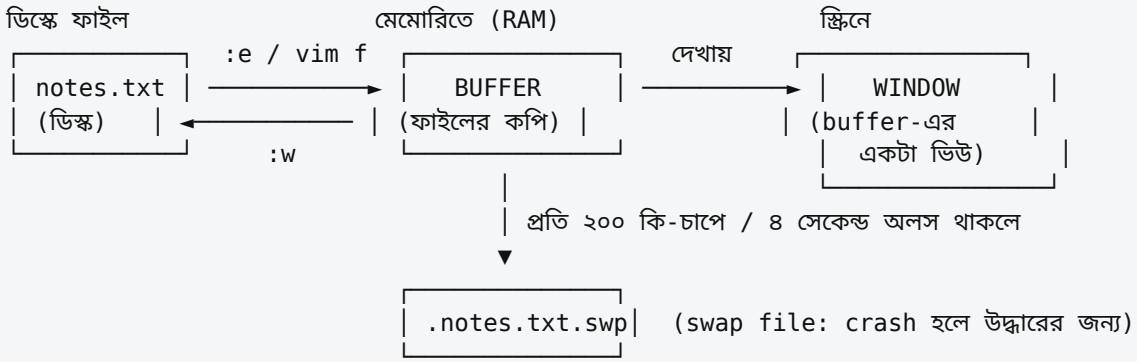
> 📝 **মনে রাখো:** দ্বিধায় পড়লে `Esc` **চাপো** (দুই-তিনবার চাপলেও ক্ষতি নেই)। তাহলে নিশ্চিতভাবে Normal

> mode-এ ফিরবে। সেখান থেকে সব শুরু।

আরও কিছু "ছোট" mode আছে যেগুলো জানলে ভালো:

- **Operator-pending mode:** `d` চাপার পর Vim অপেক্ষা করছে "কী ডিলিট করব?" — এই মুহূর্তের অবস্থা।
- **Insert Normal mode:** Insert mode-এ `Ctrl+o` চাপলে একটা মাত্র Normal কমান্ড চালিয়ে আবার Insert-এ ফেরে।
- **Select mode:** `gh` দিয়ে; Windows-এর মতো সিলেকশন (সিলেক্ট করে টাইপ করলে বদলে যায়)। খুব কম ব্যবহার হয়।
- **Ex mode:** `Q` বা `gQ` চাপলে ঢোকে — পুরোনো `ex` এডিটরের মতো। ভুল করে ঢুকলে `:vi` লিখে Enter দিলে বের হবে।

১৪.৩.২ ভেতরে কীভাবে কাজ করে: buffer, window, file



- তুমি যখন `vim notes.txt` চালাও, Vim ফাইলটা পড়ে মেমোরিতে একটা **buffer**-এ রাখে। তুমি buffer এডিট করছ, ডিস্কের ফাইল নয়।
- `:w` (write) চাপলে buffer-এর লেখা ডিস্কে লেখা হয়। তাই সেভ না করে `:q!` দিলে ডিস্কের ফাইল অক্ষত থাকে।
- একটা **window** হলো buffer-এর ওপর একটা জানালা। একই buffer দুটো window-এ দেখা যায়; আবার একটা window-এ পালাক্রমে অনেক buffer দেখা যায়।
- **tab page** হলো কয়েকটা window-এর একটা লেআউট।
- **swap file** (`.notes.txt.swp`): Vim নিয়মিত তোমার পরিবর্তন এখানে লেখে (`updatecount` ডিফল্ট ২০০ অক্ষর, `updatetime` ডিফল্ট ৪০০০ ms)। কম্পিউটার হঠাৎ বন্ধ হলে এখান থেকে কাজ উদ্ধার করা যায় (১৪.১৬ অংশে বিস্তারিত)।

১৪.৪ প্রথম দেখা: খোলা, লেখা, সেভ, বের হওয়া

১৪.৪.১ Vim খোলার উপায়

```
vim                     # খালি buffer
vim notes.txt           # ফাইল খোলা (না থাকলে সেভ করার সময় তৈরি হবে)
vim +25 app.py          # খুলেই ২৫ নম্বর লাইনে
vim + app.log           # খুলেই শেষ লাইনে
vim +/ERROR app.log     # খুলেই প্রথম "ERROR"-এ
vim -R /etc/fstab       # read-only (একই কাজ: view /etc/fstab)
vim -p a.txt b.txt      # প্রতিটা ফাইল আলাদা tab-এ
vim -o a.txt b.txt      # ওপরে-নিচে split
vim -O a.txt b.txt      # পাশাপাশি split
vimdiff old.conf new.conf   # দুটো ফাইলের পার্থক্য রঙ করে দেখায় (= vim -d)
vim -u NONE file        # কোনো .vimrc/প্লাগইন ছাড়া (ট্রাবলশুটিংয়ে কাজে লাগে)
command | vim -         # অন্য কমান্ডের আউটপুট Vim-এ খোলা
```

১৪.৪.২ স্ক্রিনের চেহারা

```
$ vim hello.txt
┌──────────────────────────────────────────────┐
│Hello Linux                                   │ ← লেখা
│~                                             │ ← "~" মানে ফাইলের শেষের পরের
│~                                             │    ফাঁকা জায়গা (এখানে লাইন নেই)
│~                                             │
│"hello.txt" 1L, 12B                1,1   All  │ ← ফাইলের নাম, লাইন, বাইট;
└──────────────────────────────────────────────┘   কার্সর লাইন,কলাম; অবস্থান
```

১৪.৪.৩ প্রথম ফাইল — ধাপে ধাপে

1. `vim hello.txt` — Normal mode-এ খুলল।
2. `i` চাপো — নিচে `-- INSERT --` দেখাবে।
3. লেখো: `Hello Linux`
4. `Esc` চাপো — Normal mode-এ ফিরলে।
5. `:w` লিখে `Enter` — সেভ হলো। নিচে দেখাবে: `"hello.txt" [New] 1L, 12B written`
6. `:q` লিখে `Enter` — বের হলে।

১৪.৪.৪ বের হওয়ার সব উপায়

কমান্ড	কাজ
`:q`	বের হও (পরিবর্তন সেভ না থাকলে Vim আটকাবে: `E37: No write since last change`)
`:q!`	সেভ না করে **জোর করে** বের হও — সব পরিবর্তন বাতিল
`:w`	সেভ করো (বের হবে না)

কমান্ড	কাজ
`:wq`	সেভ করো ও বের হও (পরিবর্তন না থাকলেও লেখে, ফাইলের সময় বদলায়)
`:x`	পরিবর্তন থাকলে তবেই সেভ করে, তারপর বের হয়
`ZZ`	Normal mode-এ; `:x`-এর সমান
`ZQ`	Normal mode-এ; `:q!`-এর সমান
`:qa` / `:qa!`	সব window/buffer বন্ধ / জোর করে সব বন্ধ
`:wa` / `:wqa` / `:xa`	সব ফাইল সেভ / সব সেভ করে বের হও
`:w newname.txt`	অন্য নামে একটা কপি সেভ (বর্তমান buffer আগের নামেই থাকে)
`:saveas newname.txt`	অন্য নামে সেভ এবং এখন থেকে নতুন নামে কাজ
`:w !sudo tee % > /dev/null`	root-এর ফাইল সাধারণ ইউজার হিসেবে খুলে ফেললে sudo দিয়ে সেভ (শুধু Vim-এ; Neovim-এ কাজ করে না)
`:e!`	শেষ সেভ করা অবস্থায় ফিরে যাও (সব পরিবর্তন ফেলে)

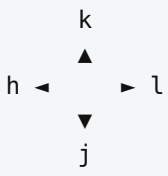
⚠️ **সতর্কতা:** ভুল করে `Ctrl+s` চাপলে টার্মিনাল "জমে যায়" (এটা Vim না, টার্মিনালের XOFF flow control)। ভয় পেও না — `Ctrl+q` চাপো, আবার চলবে। আর `Ctrl+z` চাপলে Vim সাসপেন্ড হয়ে শেলে ফিরে যায় (বন্ধ হয় না!); `fg` লিখে ফিরে আসো (অধ্যায় ২৫)।

💡 **টিপস:** root-এর ফাইল এডিট করার সবচেয়ে নিরাপদ উপায় `sudoedit /etc/hosts` (বা `sudo -e`)। এতে Vim তোমার নিজের ইউজার হিসেবে চলে, তোমার `.vimrc` কাজ করে, আর শেষে sudo ফাইলটা কপি করে বসায় (অধ্যায় ১৮)।

১৪.৫ চলাফেরা (Movement / Motion)

Normal mode-এ কার্সর সরানোর কমান্ডগুলোকে **motion** বলে। এগুলো শুধু চলার জন্য না — পরে দেখবে এগুলো operator-এর সাথে জুড়ে "কতটুকু ডিলিট/কপি করব" বোঝায়। তাই motion ভালো করে শেখা মানে অর্ধেক Vim শেখা।

১৪.৫.১ মৌলিক: h j k l



কি	দিক	মনে রাখার কৌশল
`h`	বামে	সবচেয়ে বামের কি
`j`	নিচে	`j`-এর লেজ নিচে ঝুলে আছে
`k`	ওপরে	
`l`	ডানে	সবচেয়ে ডানের কি

যেকোনো motion-এর আগে একটা সংখ্যা (**count**) দিলে ততবার চলে: `5j` = ৫ লাইন নিচে, `3l` = ৩ অক্ষর ডানে। arrow key-ও কাজ করে, কিন্তু `hjkl` অভ্যাস করলে হাত নড়াতে হয় না।

> 💡 **টিপস:** `:set relativenumber` দিলে প্রতিটা লাইনের পাশে বর্তমান লাইন থেকে দূরত্ব দেখায় — তখন `7j`, `12k` টাইপ করা খুব সহজ।

১৪.৫.২ শব্দ ধরে চলা

ধরো লাইনটা: `user.name = "rahim-khan";`

কি	কাজ
`w`	পরের **word**-এর শুরুতে
`b`	আগের (বা বর্তমান) word-এর শুরুতে (back)
`e`	বর্তমান/পরের word-এর শেষে (end)
`ge`	আগের word-এর শেষে
`W` `B` `E` `gE`	একই, কিন্তু **WORD** ধরে

word বনাম WORD: ছোট `word` হলো অক্ষর/সংখ্যা/`_` এর ধারাবাহিকতা, অথবা অন্য চিহ্নের ধারাবাহিকতা — যেকোনো একটা। বড় `WORD` হলো স্পেস দিয়ে আলাদা করা যেকোনো কিছু।

```
লাইন:  user.name = "rahim-khan";

word (w) হিসেবে টুকরা:  user | . | name | = | " | rahim | - | khan | ";    → ৯টা
WORD (W) হিসেবে টুকরা:  user.name | = | "rahim-khan";                      → ৩টা
```

অর্থাৎ লাইনের শুরু থেকে `w` চাপলে ৮ বারে শেষ টুকরায় পৌঁছাবে, `W` চাপলে মাত্র ২ বারে। কোড এডিট করার সময় `W` / `B` / `E` দিয়ে অনেক দ্রুত চলা যায়।

১৪.৫.৩ লাইনের ভেতরে

কি	কাজ
`0`	লাইনের একদম প্রথম কলামে

কি	কাজ
`^`	লাইনের প্রথম **অ-ফাঁকা** অক্ষরে (indentation পেরিয়ে)
`$`	লাইনের শেষ অক্ষরে
`g_`	লাইনের শেষ অ-ফাঁকা অক্ষরে
`f{c}`	ডানে প্রথম `{c}` অক্ষরে (find); যেমন `f"`
`F{c}`	বামে প্রথম `{c}` অক্ষরে
`t{c}`	ডানে `{c}`-এর ঠিক আগের অক্ষরে (till)
`T{c}`	বামে `{c}`-এর ঠিক পরের অক্ষরে
`;`	শেষ `f/F/t/T` একই দিকে আবার
`,`	শেষ `f/F/t/T` উল্টো দিকে
`{n}|`	`n` নম্বর কলামে (যেমন `20|`)

১৪.৫.৪ ফাইল জুড়ে

কি	কাজ
`gg`	ফাইলের প্রথম লাইনে
`G`	ফাইলের শেষ লাইনে
`{n}G` বা `{n}gg` বা `:{n}`	`n` নম্বর লাইনে (যেমন `42G`, `:42`)
`{n}%`	ফাইলের `n` শতাংশে (`50%` = মাঝখানে)
`%`	মিলে যাওয়া বন্ধনীতে লাফ: `(` ↔ `)`, `[` ↔ `]`, `{` ↔ `}`
`}`	পরের প্যারাগ্রাফে (পরের ফাঁকা লাইন)
`{`	আগের প্যারাগ্রাফে
`)` / `(`	পরের / আগের বাক্যে
`H`	স্ক্রিনের ওপরের (High) লাইনে
`M`	স্ক্রিনের মাঝের (Middle) লাইনে
`L`	স্ক্রিনের নিচের (Low) লাইনে
`gd`	কার্সরের নিচের শব্দের local definition-এ (go to declaration)
`gf`	কার্সরের নিচের লেখা ফাইলের নাম হলে সেই ফাইল খোলো

১৪.৫.৫ স্ক্রিন স্ক্রল

কি	কাজ
`Ctrl+f`	এক পাতা নিচে (forward)
`Ctrl+b`	এক পাতা ওপরে (backward)

কি	কাজ
`Ctrl+d`	আধা পাতা নিচে (down)
`Ctrl+u`	আধা পাতা ওপরে (up)
`Ctrl+e`	স্ক্রিন এক লাইন নিচে সরাও (কার্সর জায়গায় থাকে যতক্ষণ সম্ভব)
`Ctrl+y`	স্ক্রিন এক লাইন ওপরে সরাও
`zz`	বর্তমান লাইন স্ক্রিনের মাঝখানে আনো
`zt`	বর্তমান লাইন স্ক্রিনের ওপরে (top)
`zb`	বর্তমান লাইন স্ক্রিনের নিচে (bottom)

> 📝 **মনে রাখো:** `zz`-এর সাথে `ZZ` গুলিয়ো না! ছোট `zz` স্ক্রিন সাজায়, বড় `ZZ` সেভ করে বের হয়ে যায়।

১৪.৬ এডিটিং: লেখা, মোছা, কপি, পেস্ট

১৪.৬.১ Insert mode-এ ঢোকার উপায়

```
        I           i a               A
        ▼           ▼ ▼               ▼
    ····int count = 0;·····················
        ▲           ▲
        ^ (প্রথম অ-ফাঁকা)  কার্সর এখানে ('=')

  O  → ওপরে নতুন লাইন খুলে লেখা
  o  → নিচে নতুন লাইন খুলে লেখা
```

কি	কোথায় লেখা শুরু
`i`	কার্সরের **আগে** (insert)
`a`	কার্সরের **পরে** (append)
`I`	লাইনের প্রথম অ-ফাঁকা অক্ষরের আগে
`A`	লাইনের একদম শেষে
`o`	বর্তমান লাইনের **নিচে** নতুন লাইন খুলে
`O`	বর্তমান লাইনের **ওপরে** নতুন লাইন খুলে
`gi`	শেষবার যেখানে Insert mode ছেড়েছিলে সেখানে
`s`	কার্সরের অক্ষর মুছে লেখা শুরু (substitute)
`S` বা `cc`	পুরো লাইন মুছে (indentation রেখে) লেখা শুরু
`C`	কার্সর থেকে লাইনের শেষ পর্যন্ত মুছে লেখা শুরু (= `c$`)

১৪.৬.২ Insert mode-এর ভেতরের শর্টকাট

কি	কাজ
`Esc` বা `Ctrl+[`	Normal mode-এ ফেরো
`Ctrl+c`	Normal mode-এ ফেরো (কিছু ক্ষেত্রে abbreviation/autocmd ট্রিগার করে না)
`Ctrl+h`	বামের এক অক্ষর মোছো (Backspace-এর মতো)
`Ctrl+w`	বামের এক word মোছো
`Ctrl+u`	লাইনের শুরু পর্যন্ত (এই insert-এ লেখা অংশ) মোছো
`Ctrl+t` / `Ctrl+d`	লাইনের indent এক ধাপ বাড়াও / কমাও
`Ctrl+n` / `Ctrl+p`	auto-complete: খোলা buffer-এর শব্দ থেকে পরের/আগের প্রস্তাব
`Ctrl+r {reg}`	register-এর লেখা পেস্ট করো (যেমন `Ctrl+r "`)
`Ctrl+o {cmd}`	একটা Normal কমান্ড চালাও, তারপর Insert-এ ফেরো (যেমন `Ctrl+o zz`)
`Ctrl+v {key}`	কি-টা আক্ষরিক বসাও (যেমন `Ctrl+v Tab` = আসল Tab, expandtab থাকলেও)
`Ctrl+a`	শেষবার যা টাইপ করেছিলে তা আবার বসাও

> 💡 **টিপস:** অনেকে Caps Lock কি-কে Esc বানিয়ে নেয় (GNOME Tweaks → Keyboard → Additional Layout Options, অথবা `setxkbmap -option caps:escape`)। Esc আঙুলের অনেক কাছে চলে আসে।

১৪.৬.৩ মোছা (delete)

কি	কাজ
`x`	কার্সরের অক্ষর মোছো
`X`	কার্সরের বামের অক্ষর মোছো
`dd`	পুরো লাইন মোছো
`3dd`	৩ লাইন মোছো
`D`	কার্সর থেকে লাইনের শেষ পর্যন্ত মোছো (= `d$`)
`dw`	পরের word-এর শুরু পর্যন্ত মোছো
`J`	নিচের লাইন এই লাইনের সাথে জোড়া দাও (মাঝে একটা স্পেস দিয়ে)
`gJ`	জোড়া দাও, কিন্তু স্পেস যোগ না করে

> 📝 **মনে রাখো:** Vim-এ "delete" আসলে **cut** — মোছা লেখা একটা register-এ জমা থাকে, `p` চাপলেই ফিরে আসে। তাই `ddp` = এই লাইন আর নিচের লাইন অদলবদল; `xp` = দুটো অক্ষর অদলবদল (typo ঠিক

> করার দারুণ কৌশল: `teh` → `the`)।

১৪.৬.৪ কপি (yank) ও পেস্ট (put)

Vim-এ কপিকে বলে **yank** (`y`), কারণ `c` আগেই change-এর জন্য নেওয়া।

কি	কাজ
`yy` বা `Y`	পুরো লাইন কপি
`5yy`	৫ লাইন কপি
`yw`	একটা word কপি
`y$`	কার্সর থেকে লাইনের শেষ পর্যন্ত কপি
`p`	কার্সরের **পরে** পেস্ট (লাইন হলে নিচের লাইনে)
`P`	কার্সরের **আগে** পেস্ট (লাইন হলে ওপরের লাইনে)
`gp` / `gP`	`p` / `P`-এর মতো, কিন্তু কার্সর পেস্ট করা লেখার পরে চলে যায়
`]p`	পেস্ট করে বর্তমান লাইনের indentation-এর সাথে মিলিয়ে নেয়

> ⚠️ **সতর্কতা:** Vim-এ `Y` ঐতিহাসিকভাবে `yy`-এর সমান (পুরো লাইন)। কিন্তু **Neovim**-এ ডিফল্টে `Y` = `y$` (লাইনের শেষ পর্যন্ত), যাতে `D` আর `C`-এর সাথে সামঞ্জস্য থাকে।

১৪.৬.৫ বদলানো (change, replace)

কি	কাজ
`r{c}`	কার্সরের একটা অক্ষর `{c}` দিয়ে বদলাও (Insert mode-এ না ঢুকেই)
`5rx`	৫টা অক্ষর `x` দিয়ে বদলাও
`R`	Replace mode: যা টাইপ করবে পুরোনো লেখার ওপরে বসবে; Backspace দিলে পুরোনোটা ফিরে আসে
`cw`	word-এর বাকি অংশ মুছে Insert mode (বিশেষ নিয়ম: `cw` আসলে `ce`-এর মতো আচরণ করে, শেষের স্পেস মোছে না)
`ciw`	কার্সর word-এর যেখানেই থাকুক, পুরো word বদলাও
`cc`	পুরো লাইন বদলাও
`c$` বা `C`	লাইনের শেষ পর্যন্ত বদলাও
`~`	কার্সরের অক্ষরের case উল্টাও (a ↔ A) এবং এগোও
`g~iw` / `gUiw` / `guiw`	word-এর case উল্টাও / বড় হাতের / ছোট হাতের করো
`gUU` / `guu`	পুরো লাইন বড় / ছোট হাতের

কি	কাজ
`Ctrl+a` / `Ctrl+x`	কার্সরের (বা ডানের) সংখ্যা ১ বাড়াও / কমাও; `10 Ctrl+a` = ১০ বাড়াও
`>>` / `<<`	লাইনের indent বাড়াও / কমাও (`shiftwidth` অনুযায়ী)
`==`	লাইন স্বয়ংক্রিয়ভাবে indent করো
`gg=G`	পুরো ফাইল re-indent করো

১৪.৬.৬ Undo, Redo আর জাদুর ডট (`.`)

কি	কাজ
`u`	শেষ পরিবর্তন undo
`5u`	৫টা পরিবর্তন undo
`Ctrl+r`	redo (undo-কে উল্টানো)
`U`	শেষ যে লাইন বদলেছ তার সব পরিবর্তন একসাথে উল্টাও (নিজেও একটা পরিবর্তন হিসেবে গণ্য হয়)
`.`	**শেষ পরিবর্তনটা আবার করো**
`:earlier 10m` / `:later 10m`	১০ মিনিট আগের / পরের অবস্থায় যাও (সময় ধরে undo!)
`:earlier 3f`	৩টা সেভ আগের অবস্থায়
`g-` / `g+`	undo tree-তে সময়ক্রমে পেছনে / সামনে

একটা "পরিবর্তন" কী? Insert mode-এ ঢোকা থেকে বের হওয়া পর্যন্ত যা টাইপ করেছ — পুরোটাই একটা পরিবর্তন। তাই `u` চাপলে পুরো টাইপ করা অংশ একবারে চলে যায়।

`.` (dot) কমান্ড Vim-এর সবচেয়ে কাজের কি। উদাহরণ: প্রতিটা লাইনের শেষে `;` বসাতে চাও।

```
A;<Esc>     ← প্রথম লাইনে: শেষে গিয়ে ; লিখে বের হলে (এটা একটা "পরিবর্তন")
j.          ← নিচে গিয়ে . চাপো → একই কাজ
j.          ← আবার...
```

ভেতরে কীভাবে কাজ করে — undo tree: বেশিরভাগ এডিটরে undo একটা সরল লাইন; undo করে নতুন কিছু লিখলে পুরোনো "ভবিষ্যৎ" হারিয়ে যায়। Vim প্রতিটা পরিবর্তন একটা **গাছ (tree)**-এ রাখে, তাই কোনো অবস্থা হারায় না — `g-` / `g+` বা `:undolist` দিয়ে যেকোনো শাখায় যাওয়া যায়।

```
        (শুরু)
          |
          1  "Hello"
          |
          2  "Hello World"
         / \
  u করে  3   4   ← u করে নতুন লেখা = নতুন শাখা; শাখা 3 হারায়নি
```

```
 "Hello   "Hello Linux"
  Vim"
```

`.vimrc`-এ `set undofile` দিলে undo history ফাইল বন্ধ করার পরেও থাকে (১৪.১৫ দেখো)।

১৪.৭ Vim-এর ব্যাকরণ: Operator + Count + Motion

এটাই Vim-এর আসল রহস্য। প্রতিটা এডিট কমান্ড একটা ছোট বাক্য:

```
   [count]  operator  [count]  motion / text-object
      |        |          |           |
      2        d          3           w        → "২ বার ৩টা word ডিলিট" = ৬টা word
               |                      |
          ক্রিয়া (verb)          কতটুকু (noun)
```

উপমা: ইংরেজি বাক্যের মতো — **verb + noun**। "delete word" = `dw`, "change inside quotes" = `ci"`, "yank to end of line" = `y$`। ১০টা verb আর ২০টা noun শিখলে ২০০টা কম্বিনেশন বিনা মুখস্থে পারবে। আর নতুন একটা motion শিখলে সেটা সব operator-এর সাথে স্বয়ংক্রিয়ভাবে কাজ করবে।

১৪.৭.১ Operator (ক্রিয়া)

Operator	কাজ
`d`	delete (cut)
`c`	change (মুছে Insert mode)
`y`	yank (copy)
`>` / `<`	indent ডানে / বামে
`=`	auto-indent
`gu` / `gU` / `g~`	lowercase / uppercase / case উল্টানো
`gq`	লেখা নির্দিষ্ট প্রস্থে (`textwidth`) মুড়িয়ে সাজাও
`!`	বাইরের কমান্ড দিয়ে filter করো (যেমন `!ip sort`)
`zf`	fold তৈরি করো

নিয়ম: operator দুবার লিখলে মানে "পুরো বর্তমান লাইন" — `dd`, `cc`, `yy`, `>>`, `==`, `gUU` (বা `gUgU`)।

১৪.৭.২ উদাহরণ টেবিল

কমান্ড	বাংলায় পড়ো	কাজ
`dw`	delete word	পরের word-এর শুরু পর্যন্ত মোছো
`d3w` / `3dw`	delete 3 words	৩টা word মোছো

কমান্ড	বাংলায় পড়ো	কাজ
`d$`	delete to end	লাইনের শেষ পর্যন্ত মোছো
`d0`	delete to start	লাইনের শুরু পর্যন্ত মোছো
`dG`	delete to end of file	এখান থেকে ফাইলের শেষ পর্যন্ত সব লাইন
`dgg`	delete to top	ফাইলের শুরু পর্যন্ত সব লাইন
`dj`	delete down	এই লাইন আর নিচের লাইন
`d}`	delete paragraph forward	প্যারাগ্রাফের শেষ পর্যন্ত
`dt)`	delete till `)`	`)`-এর আগ পর্যন্ত মোছো
`df,`	delete find `,`	`,` সহ মোছো
`d/foo`	delete until search	"foo" পাওয়া পর্যন্ত মোছো
`c$`	change to end	লাইনের বাকিটা নতুন করে লেখো
`yG`	yank to end	এখান থেকে শেষ পর্যন্ত কপি
`>}`	indent paragraph	প্যারাগ্রাফ indent
`gUw`	uppercase word	word বড় হাতের
`=G`	indent to end	এখান থেকে শেষ পর্যন্ত re-indent

inclusive বনাম exclusive motion: `dw`-তে পরের word-এর প্রথম অক্ষর মোছে না (exclusive), কিন্তু `de`-তে word-এর শেষ অক্ষরও মোছে (inclusive)। `t` exclusive-এর মতো ("আগ পর্যন্ত"), `f` inclusive ("সহ")। `j` / `k` / `G` / `gg` হলো **linewise** — পুরো লাইন ধরে কাজ করে।

১৪.৭.৩ Text Objects — "ভেতরে" আর "চারপাশসহ"

Motion কার্সর থেকে কোথাও পর্যন্ত মাপে। কিন্তু অনেক সময় কার্সর একটা শব্দ বা বন্ধনীর মাঝখানে থাকে, আর তুমি **পুরো জিনিসটা** ধরতে চাও। এর জন্য **text object** — এগুলো শুধু operator-এর পরে (বা Visual mode-এ) ব্যবহার হয়, একা চলার জন্য না।

- `i` = **inner** (ভেতরেরটুকু, চারপাশের স্পেস/বন্ধনী ছাড়া)
- `a` = **a / around** (চারপাশের স্পেস বা বন্ধনীসহ)

Text object	কী বোঝায়
`iw` / `aw`	একটা word / word + পাশের স্পেস
`iW` / `aW`	একটা WORD / WORD + স্পেস
`is` / `as`	একটা বাক্য (sentence)
`ip` / `ap`	একটা প্যারাগ্রাফ / প্যারাগ্রাফ + পরের ফাঁকা লাইন
`i"` / `a"`	`"…"`-এর ভেতর / quote-সহ
`i'` / `a'`	`'…'`-এর ভেতর / quote-সহ

Text object	কী বোঝায়
`i`` / `a``	backtick-এর ভেতর / সহ
`i(` `ib` / `a(` `ab`	`( )`-এর ভেতর / বন্ধনীসহ (`i)` একই)
`i{` `iB` / `a{` `aB`	`{ }`-এর ভেতর / সহ
`i[` / `a[`	`[ ]`-এর ভেতর / সহ
`i<` / `a<`	`< >`-এর ভেতর / সহ
`it` / `at`	HTML/XML tag-এর ভেতর / tag-সহ

```
লাইন:   print("Hello, World")      কার্সর 'W'-এর ওপর
                    ▲
ci"   → print("|")                   quote-এর ভেতরটা মুছে লেখা শুরু
ca"   → print(||)                    quote-সহ মুছে
ci(   → print(||)                    বন্ধনীর ভেতরটা মুছে
da(   → print|                       বন্ধনীসহ মুছে
diw   → print("Hello, |")            শুধু "World" শব্দটা মুছে
```

```
<div><p>Old text</p></div>      কার্সর "text"-এর ওপর
cit  → <div><p>|</p></div>
dat  → <div>|</div>
```

> 💡 **টিপস:** `ci"` কমান্ডটা কার্সর quote-এর বাইরে থাকলেও কাজ করে — Vim লাইনে ডানদিকে প্রথম quote জোড়া খুঁজে নেয়। বন্ধনী (`ci(`, `ci{`)-এর জন্য অবশ্য কার্সরকে বন্ধনীর ভেতরে (বা বন্ধনী চিহ্নের ওপরে) থাকতে হয়, নইলে কিছুই হয় না।

কেন text object এত শক্তিশালী? `.` দিয়ে পুনরাবৃত্তি করলে বোঝা যায়। `ciw` + নতুন শব্দ + `Esc` করে, তারপর অন্য শব্দে গিয়ে শুধু `.` চাপো — কার্সর শব্দের যেখানেই থাকুক, পুরো শব্দটা বদলে যাবে। `cw` দিয়ে এটা হতো না, কারণ `cw` কার্সর থেকে শুরু করে।

১৪.৮ সার্চ: খোঁজা

১৪.৮.১ মৌলিক সার্চ

কি	কাজ
`/pattern` + Enter	সামনের দিকে খোঁজো
`?pattern` + Enter	পেছনের দিকে খোঁজো
`n`	একই দিকে পরেরটায়
`N`	উল্টো দিকে পরেরটায়

কি	কাজ
`*`	কার্সরের নিচের **পুরো শব্দ** সামনে খোঁজো (`\<word\>`)
`#`	একই, পেছনের দিকে
`g*` / `g#`	শব্দের অংশ হিসেবেও মেলাবে (word boundary ছাড়া)
`/` + Enter	শেষ সার্চ আবার
`/` তারপর `↑` / `↓`	সার্চ হিস্ট্রি
`:noh` (`:nohlsearch`)	হাইলাইট সাময়িকভাবে বন্ধ
`gn`	পরের মিল সিলেক্ট করো (যেমন `cgn` + নতুন লেখা, তারপর `.` দিয়ে একটা একটা করে বদলানো)

ফাইলের শেষে পৌঁছালে সার্চ আবার শুরু থেকে চলে (`wrapscan` অপশন), নিচে লেখা দেখায়: `search hit BOTTOM, continuing at TOP`।

১৪.৮.২ সার্চের অপশন

```
:set hlsearch      " সব মিল হাইলাইট
:set incsearch     " টাইপ করতে করতে খোঁজা
:set ignorecase    " ছোট-বড় হাতের পার্থক্য উপেক্ষা
:set smartcase     " pattern-এ বড় হাতের অক্ষর থাকলে তবেই case-sensitive
```

একবারের জন্য: `/\cerror` = case-insensitive, `/\Cerror` = case-sensitive।

১৪.৮.৩ Vim-এর regex — সংক্ষেপে

Vim-এর pattern রেগুলার এক্সপ্রেশন (অধ্যায় ২০) — তবে নিজস্ব "magic" নিয়মে। ডিফল্টে `.` `*` `[]` `^` `$` বিশেষ অর্থ রাখে, কিন্তু `+` `?` `|` `()` `{}`-এর আগে `\` দিতে হয়।

Pattern	মানে
`^` / `$`	লাইনের শুরু / শেষ
`.`	যেকোনো একটা অক্ষর
`*`	আগেরটা ০ বা বেশিবার
`\+` / `\=` বা `\?`	১ বা বেশিবার / ০ বা ১ বার
`\{2,4}`	২ থেকে ৪ বার
`\<` / `\>`	শব্দের শুরু / শেষ (word boundary)
`\(…\)`	group (replace-এ `\1` দিয়ে ব্যবহার)
`|`	অথবা
`\d` `\s` `\w`	অঙ্ক / ফাঁকা / word অক্ষর
`\v`	**very magic**: এরপর `+ ?

```
/\<the\>                  " শুধু আলাদা শব্দ "the", "there" না
/^\s*$                    " ফাঁকা (বা শুধু স্পেসওয়ালা) লাইন
/\v(error|fail)\d+        " very magic: error বা fail, তারপর এক বা বেশি অঙ্ক
/\(error\|fail\)\d\+      " উপরেরটাই, very magic ছাড়া
```

১৪.৯ খুঁজে-বদলানো: `:substitute`

১৪.৯.১ Syntax

```
:[range]s/pattern/replacement/[flags]
   │        │          │          │
   │        │          │          └─ g c i I n ...
   │        │          └─ নতুন লেখা
   │        └─ কী খুঁজবে
   └─ কোন লাইনগুলোতে (না দিলে শুধু বর্তমান লাইন)
```

১৪.৯.২ Range

Range	মানে
(কিছু না)	শুধু বর্তমান লাইন
`%`	পুরো ফাইল (= `1,$`)
`5,20`	৫ থেকে ২০ নম্বর লাইন
`.,$`	বর্তমান লাইন থেকে শেষ পর্যন্ত (`.` = বর্তমান, `$` = শেষ)
`.,+5`	বর্তমান থেকে পরের ৫ লাইন
`'<,'>`	Visual mode-এ সিলেক্ট করা লাইনগুলো (`V` দিয়ে সিলেক্ট করে `:` চাপলে নিজে থেকে বসে)
`'a,'b`	mark `a` থেকে mark `b` পর্যন্ত
`/start/,/end/`	"start" পাওয়া লাইন থেকে "end" পাওয়া লাইন পর্যন্ত

১৪.৯.৩ Flags

Flag	কাজ
`g`	লাইনের **সব** মিল বদলাও (না দিলে প্রতি লাইনের শুধু প্রথমটা)
`c`	প্রতিটা বদলানোর আগে জিজ্ঞেস করো (confirm)
`i` / `I`	case-insensitive / case-sensitive
`n`	বদলাবে না, শুধু গুনে বলবে কতগুলো মিল আছে
`e`	মিল না পেলে এরর দেখাবে না
`&`	আগের flag-গুলো রাখো

`c` flag দিলে প্রতিটা মিলে Vim জিজ্ঞেস করে:

```
replace with Linux (y/n/a/q/l/^E/^Y)?
```

`y` = এটা বদলাও, `n` = এটা বাদ, `a` = এটা ও বাকি সব বদলাও, `q` = থামো, `l` = এটা বদলে থামো (last), `Ctrl+e` / `Ctrl+y` = স্ক্রল।

১৪.৯.৪ উদাহরণ

```
:s/cat/dog/             " বর্তমান লাইনের প্রথম cat → dog
:s/cat/dog/g            " বর্তমান লাইনের সব cat → dog
:%s/cat/dog/g           " পুরো ফাইলে সব cat → dog
:%s/cat/dog/gc          " পুরো ফাইলে, প্রতিটায় জিজ্ঞেস করে
:%s/\<cat\>/dog/g       " শুধু আলাদা শব্দ cat ("category" বাদ)
:5,10s/^/# /            " ৫–১০ লাইনের শুরুতে "# " বসাও (comment out)
:%s/\s\+$//e            " সব লাইনের শেষের অপ্রয়োজনীয় স্পেস মুছে ফেলো
:%s/foo//gn             " কতবার foo আছে গুনে বলো
:%s#/usr/local#/opt#g   " "/" ভরা path-এ অন্য delimiter (#) ব্যবহার
:%s/\(\w\+\) \(\w\+\)/\2 \1/   " প্রতি লাইনের প্রথম দুই শব্দ অদলবদল
:%s/name/\U&/g          " name → NAME (& = পুরো মিল, \U = বাকিটা বড় হাতের)
:%s/,/\r/g              " প্রতিটা কমার জায়গায় নতুন লাইন (\r = newline বসাও)
```

> ⚠️ **সতর্কতা:** replacement অংশে নতুন লাইন বসাতে `\r` লিখবে, `\n` না। replacement-এ `\n` লিখলে Vim একটা NUL অক্ষর বসায় (স্ক্রিনে `^@` দেখায়)। তবে **pattern (খোঁজার)** অংশে লাইনের শেষ মেলাতে `\n` ঠিক আছে।

replacement-এর বিশেষ চিহ্ন: `&` বা `\0` = পুরো মিল, `\1` … `\9` = group, `\u` / `\l` = পরের অক্ষর বড়/ছোট, `\U` / `\L` = বাকি সব বড়/ছোট, `\E` = থামো, `~` = আগের replacement।

দ্রুত পুনরাবৃত্তি: `:s` চালানোর পর অন্য লাইনে গিয়ে `&` চাপলে একই substitute (flag ছাড়া) আবার চলে; `g&` = শেষ substitute পুরো ফাইলে আগের flag সহ।

১৪.৯.৫ `:g` — global কমান্ড

`ed` -যুগের এই কমান্ড অসম্ভব শক্তিশালী: "যেসব লাইনে pattern মেলে, তাদের প্রতিটায় এই কমান্ড চালাও"।

```
:g/DEBUG/d              " যেসব লাইনে DEBUG আছে সব মুছে ফেলো
:g!/ERROR/d             " যেসব লাইনে ERROR নেই সব মুছো (= :v/ERROR/d)
:g/^\s*$/d              " সব ফাঁকা লাইন মুছো
:g/TODO/p               " TODO-ওয়ালা লাইনগুলো দেখাও (grep-এর মতো; g/re/p — এখান থেকেই
"grep" নাম!)
:g/^#/m$                " comment লাইনগুলো ফাইলের শেষে সরাও
:g/pattern/normal A;    " মেলা প্রতিটা লাইনের শেষে ; বসাও
:g/^/m0                 " পুরো ফাইল উল্টো ক্রমে সাজাও
```

> 📝 **মনে রাখো:** `grep` কমান্ডের নামই এসেছে `ed`-এর `g/re/p` থেকে — **g**lobal / **r**egular **e**xpression / **p**rint।

১৪.৯.৬ আরও কিছু Ex কমান্ড

কমান্ড	কাজ
`:5,10d`	৫–১০ লাইন মোছো
`:5,10y`	৫–১০ লাইন কপি
`:5,10m 20`	৫–১০ লাইন ২০ নম্বরের পরে সরাও (move)
`:5,10t 20` বা `:5,10co 20`	৫–১০ লাইন ২০-এর পরে কপি করো
`:sort` / `:sort u` / `:sort n` / `:sort!`	লাইন সাজাও / ডুপ্লিকেট বাদ দিয়ে / সংখ্যা হিসেবে / উল্টো
`:r file.txt`	অন্য ফাইলের লেখা কার্সরের নিচে ঢোকাও
`:r !date`	কমান্ডের আউটপুট ঢোকাও
`:!ls -l`	শেল কমান্ড চালাও (Vim থেকে না বেরিয়ে)
`:%!sort`	পুরো ফাইল `sort` দিয়ে filter (বাইরের কমান্ড)
`:%!jq .`	JSON সুন্দর করে সাজাও (jq ইনস্টল থাকলে)
`:sh` বা `:terminal`	শেল খোলো / Vim-এর ভেতরে টার্মিনাল window (Vim 8.1+)
`@:`	শেষ Ex কমান্ড আবার চালাও
`q:`	Ex কমান্ড হিস্ট্রি একটা window-এ দেখাও (এডিট করে Enter দিয়ে চালানো যায়; `:q` দিয়ে বন্ধ)

Command-line mode-এ টাইপ করার সময়: `Tab` = কমপ্লিশন, `Ctrl+d` = সম্ভাব্য সব কমপ্লিশন দেখাও, `↑` / `↓` = হিস্ট্রি, `Ctrl+r Ctrl+w` = কার্সরের নিচের শব্দ বসাও, `Ctrl+r "` = register বসাও।

১৪.১০ Visual Mode: দেখে দেখে সিলেক্ট করা

Operator + motion না ভেবে আগে চোখে দেখে সিলেক্ট করতে চাইলে Visual mode। নতুনদের জন্য এটা সবচেয়ে স্বাভাবিক লাগে।

কি	Mode	কী সিলেক্ট হয়
`v`	character-wise visual	অক্ষর ধরে (মাউস দিয়ে টেনে সিলেক্ট করার মতো)
`V`	line-wise visual	পুরো লাইন ধরে
`Ctrl+v`	block-wise visual	আয়তক্ষেত্র (কলাম) ধরে

সিলেক্ট করার পর যেকোনো motion বা text object দিয়ে সিলেকশন বাড়ানো যায় (`w`, `}`, `G`, `iw`, `ip`, `i"` ...), তারপর একটা operator চাপো:

Visual mode-এ কি	কাজ
`d` বা `x`	সিলেকশন মোছো
`y`	কপি
`c`	মুছে Insert mode
`p`	সিলেকশনের জায়গায় পেস্ট (পুরোনো লেখা register-এ চলে যায়)
`>` / `<`	indent বাড়াও / কমাও
`=`	auto-indent
`u` / `U` / `~`	ছোট হাতের / বড় হাতের / case উল্টাও
`J`	লাইনগুলো জোড়া দাও
`r{c}`	সিলেকশনের সব অক্ষর `{c}` দিয়ে বদলাও
`o`	সিলেকশনের অন্য প্রান্তে কার্সর নিয়ে যাও (দুই দিকেই বাড়ানো যায়)
`O`	block mode-এ একই লাইনের অন্য কোণে যাও
`:`	সিলেক্ট করা লাইনগুলোর ওপর Ex কমান্ড — নিচে `:'<,'>` বসে যায়
`gv`	(Normal mode থেকে) শেষ সিলেকশন আবার সিলেক্ট করো
`g Ctrl+a`	সিলেক্ট করা লাইনগুলোর সংখ্যা ক্রমবর্ধমানভাবে বাড়াও (1,1,1 → 2,3,4)
`Esc`	Visual mode বাতিল

১৪.১০.১ Block mode — একসাথে অনেক লাইনে লেখা

এটা Vim-এর একটা "জাদু" ফিচার। ধরো ৪টা লাইনের শুরুতে `#` বসিয়ে comment করতে চাও:

```
আগে:              ধাপ:                                  পরে:
echo one           1. প্রথম লাইনের শুরুতে কার্সর (0)      # echo one
echo two           2. Ctrl+v চাপো                          # echo two
echo three         3. 3j চাপো (৪ লাইন জুড়ে কলাম সিলেক্ট)  # echo three
echo four          4. I চাপো, "# " লেখো                    # echo four
                   5. Esc চাপো → সব লাইনে একসাথে বসে যায়
```

> 📝 **মনে রাখো:** `I` চাপার পর লেখাটা শুধু প্রথম লাইনে দেখা যায় — ঘাবড়ে যেও না! `Esc` চাপার পরেই বাকি লাইনগুলোতে বসে।

আরও কৌশল:

- **comment সরানো:** `Ctrl+v` দিয়ে প্রথম দুই কলাম (`# `) সিলেক্ট করো (`Ctrl+v`, `3j`, `l`), তারপর `d`।

- **সব লাইনের শেষে লেখা:** `Ctrl+v`, `3j`, `$`, `A`, লেখো, `Esc` — লাইনগুলো ভিন্ন দৈর্ঘ্যের হলেও প্রতিটার শেষে বসবে।
- **একটা কলাম বদলানো:** `Ctrl+v` দিয়ে কলাম সিলেক্ট, `c`, নতুন লেখা, `Esc`।
- **কলাম কপি-পেস্ট:** block সিলেক্ট করে `y`, অন্য জায়গায় গিয়ে `p` — আয়তক্ষেত্র হিসেবেই বসে।

১৪.১১ Registers — Vim-এর অনেকগুলো ক্লিপবোর্ড

সাধারণ এডিটরে একটা ক্লিপবোর্ড থাকে: নতুন কিছু কপি করলে পুরোনোটা হারায়। Vim-এ অনেকগুলো "বাক্স" (**register**) আছে, প্রতিটার একটা নাম।

উপমা: অফিসের টেবিলে অনেকগুলো লেবেল লাগানো ড্রয়ার। `"a` ড্রয়ারে একটা জিনিস রাখলে, `"b`-তে আরেকটা — কোনোটা অন্যটাকে মুছে দেয় না।

ব্যবহারের নিয়ম: operator-এর আগে `"` + register-এর নাম।

```
"ayy     ← বর্তমান লাইন register a-তে কপি
"byiw    ← কার্সরের শব্দ register b-তে কপি
"ap      ← register a থেকে পেস্ট
"Ayy     ← বড় হাতের A: register a-তে **যোগ (append)** করো, মুছে না দিয়ে
:reg     ← সব register-এ কী আছে দেখাও (:registers, বা :reg a b নির্দিষ্টগুলো)
```

```
$ vim notes.txt     (তারপর :reg)
Type Name Content
  l  ""   Hello Linux^J
  l  "0   Hello Linux^J
  c  "a   rahim
  c  "-   foo
  c  ":   wq
  c  "%   notes.txt
```

(`l` = linewise, `c` = characterwise, `b` = blockwise; `^J` মানে লাইনের শেষ।)

১৪.১১.১ Register-এর ধরন

Register	নাম	কী থাকে
`""`	unnamed	শেষ delete/change/yank — সাধারণ `p` এখান থেকেই নেয়
`"0`	yank register	শেষ **yank** (কপি)। delete এটাকে নষ্ট করে না!
`"1` – `"9`	numbered	delete-এর ইতিহাস: `"1` = শেষ বড় (লাইনজুড়ে) delete, আগেরগুলো `"2`, `"3` ... তে সরে যায়
`"-`	small delete	এক লাইনের কম মোছা (যেমন `dw`, `x`)
`"a` – `"z`	named	তুমি নিজে যা রাখবে; `"A` – `"Z` দিয়ে append

Register	নাম	কী থাকে
`"+`	system clipboard	OS-এর ক্লিপবোর্ড (Ctrl+c/Ctrl+v যেটা ব্যবহার করে)
`"*`	primary selection	X11-এ মাউস দিয়ে সিলেক্ট করা লেখা (মাঝের বাটনে পেস্ট হয়)
`"_`	black hole	এখানে পাঠালে কোথাও জমা হয় না — "সত্যিকারের delete"
`".`	(read-only)	শেষবার Insert mode-এ যা লিখেছ
`":`	(read-only)	শেষ Ex কমান্ড
`"%`	(read-only)	বর্তমান ফাইলের নাম
`"#`		alternate (আগের) ফাইলের নাম
`"/`		শেষ সার্চ pattern
`"=`	expression	হিসাব করা মান; যেমন Insert mode-এ `Ctrl+r =5*8 Enter` → `40`

১৪.১১.২ একটা সাধারণ সমস্যা আর তার সমাধান

সমস্যা: একটা শব্দ কপি করলে (`yiw`), তারপর অন্য জায়গায় গিয়ে একটা পুরোনো শব্দ মুছে (`diw`) পেস্ট (`p`) করলে — কপি করা শব্দ না, মোছা শব্দটা পেস্ট হলো! কারণ `diw` unnamed register (`""`) বদলে দিয়েছে।

সমাধান: `"0p` — yank register-এ কপি করা শব্দ অক্ষত আছে। অথবা মোছার সময় black hole ব্যবহার করো: `"_diw` , তারপর `p` ।

১৪.১১.৩ সিস্টেম ক্লিপবোর্ড

```
"+yy      " লাইন কপি করে সিস্টেম ক্লিপবোর্ডে (ব্রাউজারে Ctrl+v দিয়ে পেস্ট করা যাবে)
"+p       " ব্রাউজার থেকে Ctrl+c করা লেখা Vim-এ পেস্ট
:set clipboard=unnamedplus   " সব y/d/p সরাসরি সিস্টেম ক্লিপবোর্ড ব্যবহার করবে
```

> ⚠️ **সতর্কতা:** এর জন্য Vim `+clipboard` সহ কম্পাইল করা থাকতে হবে। চেক করো: `vim --version | grep clipboard` । `-clipboard` দেখালে Ubuntu/Debian-এ `vim-gtk3` , Fedora/RHEL-এ `vim-X11` (কমান্ড `vimx`) ইনস্টল করো। Neovim বাইরের টুল ব্যবহার করে — Wayland-এ `wl-clipboard` , X11-এ `xclip` বা `xsel` ইনস্টল থাকতে হয় (`:checkhealth` দিয়ে দেখা যায়)। SSH-এ রিমোট সার্ভারে সিস্টেম ক্লিপবোর্ড সাধারণত কাজ করে না।

টার্মিনালের পেস্ট (`Ctrl+Shift+v`) দিয়েও Vim-এ লেখা ঢোকানো যায়। আধুনিক Vim (8.0+) "bracketed paste" বোঝে, তাই Insert mode-এ পেস্ট করলে auto-indent এলোমেলো হয় না। খুব পুরোনো Vim-এ এই সমস্যা হলে আগে `:set paste` , পেস্ট শেষে `:set nopaste` ।

১৪.১২ Macros — কাজ রেকর্ড করে বারবার চালানো

একই ধরনের এডিট ৫০টা লাইনে করতে হবে? একবার করে **রেকর্ড** করো, তারপর বাকিগুলোতে **প্লে** করো। macro আসলে একটা register-এ রাখা কি-চাপের তালিকা।

কি	কাজ
`q{a-z}`	register `{a-z}`-তে রেকর্ডিং শুরু (নিচে `recording @a` দেখায়)
`q`	রেকর্ডিং থামাও
`@a`	macro `a` চালাও
`@@`	শেষ চালানো macro আবার চালাও
`10@a`	macro `a` ১০ বার চালাও
`:5,20normal @a`	৫–২০ প্রতিটা লাইনে macro চালাও
`:'<,'>normal @a`	Visual-এ সিলেক্ট করা প্রতিটা লাইনে
`q{A-Z}`	বিদ্যমান macro-র শেষে আরও যোগ করে রেকর্ড

১৪.১২.১ উদাহরণ: CSV থেকে SQL

```
আগে:                    লক্ষ্য:
rahim, 25               INSERT INTO users VALUES ('rahim', 25);
karim, 30               INSERT INTO users VALUES ('karim', 30);
salma, 28               INSERT INTO users VALUES ('salma', 28);
```

প্রথম লাইনে কার্সর রেখে:

```
qa                              ← রেকর্ড শুরু (register a)
0                               ← লাইনের শুরু
iINSERT INTO users VALUES ('<Esc>  ← শুরুতে লেখা
f,                              ← কমা খোঁজো
i'<Esc>                         ← কমার আগে ' বসাও
A);<Esc>                        ← শেষে ); বসাও
j                               ← পরের লাইনে যাও (পরের বারের জন্য তৈরি)
q                               ← রেকর্ড থামাও
2@a                             ← বাকি ২ লাইনে চালাও
```

ফলাফল: `INSERT INTO users VALUES ('rahim', 25);` — কমার পরের স্পেস মূল লেখায় যেমন ছিল তেমনই থাকে, macro সেটা ছোঁয় না।

ভালো macro লেখার নিয়ম:

1. **শুরুতে কার্সর একটা নির্দিষ্ট জায়গায় নাও** (`0` বা `^`), যাতে লাইনে কার্সর যেখানেই থাকুক macro একই কাজ করে।
2. **`hjkl` /count-এর চেয়ে "অর্থপূর্ণ" motion ব্যবহার করো** (`f`, `w`, `/pattern`), কারণ লাইনের দৈর্ঘ্য ভিন্ন হতে পারে।

3. **শেষে পরের লাইনে যাও** (`j`), যাতে `@a` বারবার চালানো যায়।
4. macro মাঝপথে ব্যর্থ হলে (যেমন `f,` কমা পেল না) Vim বাকি পুনরাবৃত্তি থামিয়ে দেয় — তাই `100@a` নিরাপদে দেওয়া যায়, শেষ লাইনে পৌঁছালে এমনিতেই থামবে।

macro এডিট করা: macro তো একটা register। `:let @a = 'iHello ^['` লিখে বা `"ap` দিয়ে লেখা হিসেবে পেস্ট করে, ঠিক করে, তারপর `0"ay$` দিয়ে আবার register-এ ফিরিয়ে রাখা যায়। (`:let` -এ Esc বসাতে `Ctrl+v Esc` টাইপ করতে হয়, স্ক্রিনে `^[` দেখায়।)

১৪.১৩ Marks ও Jumps — ফাইলে বুকমার্ক

উপমা: বইয়ের পাতায় আঙুল রেখে অন্য পাতায় যাওয়া — তারপর এক ঝটকায় আগের পাতায় ফেরা।

কি	কাজ
`m{a-z}`	বর্তমান অবস্থানে লোকাল mark (শুধু এই ফাইলে); যেমন `ma`
`m{A-Z}`	গ্লোবাল mark — ফাইলের নামসহ মনে রাখে; অন্য ফাইল থেকেও `'A` দিয়ে সেখানে ফেরা যায়
`'a`	mark `a` -এর **লাইনে** (প্রথম অ-ফাঁকা অক্ষরে)
`` `a ``	mark `a` -এর **ঠিক অবস্থানে** (লাইন + কলাম)
`` `` `` / `''`	শেষ লাফের আগের অবস্থানে ফেরো (বারবার চাপলে দুই জায়গার মধ্যে আসা-যাওয়া)
`` `. `` / `'.`	শেষ পরিবর্তন যেখানে করেছ
`` `^ ``	শেষবার Insert mode যেখানে ছেড়েছ
`` `[ `` / `` `] ``	শেষ পরিবর্তিত/কপি করা লেখার শুরু / শেষ
`` `< `` / `` `> ``	শেষ Visual সিলেকশনের শুরু / শেষ
`` `" ``	গতবার ফাইল বন্ধ করার সময় কার্সর যেখানে ছিল
`:marks`	সব mark দেখাও
`:delmarks a` / `:delm!`	mark `a` মোছো / সব লোকাল mark মোছো

mark operator-এর সাথেও কাজ করে: `d'a` = এখান থেকে mark `a` -এর লাইন পর্যন্ত মোছো; `y`a` = ঠিক mark পর্যন্ত কপি।

১৪.১৩.১ Jumplist ও Changelist

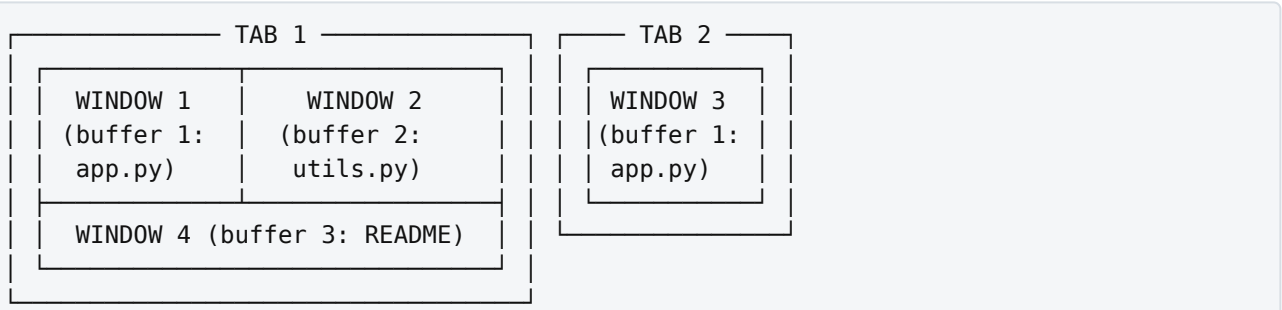
Vim প্রতিটা "বড় লাফ" (`G` , `gg` , `/search` , `n` , `%` , `{` , `}` , mark-এ যাওয়া, অন্য ফাইল খোলা) একটা তালিকায় মনে রাখে — ব্রাউজারের Back/Forward বাটনের মতো।

কি	কাজ
`Ctrl+o`	jumplist-এ পেছনে (আগের অবস্থান)
`Ctrl+i` বা `Tab`	jumplist-এ সামনে

কি	কাজ
`:jumps`	jumplist দেখাও
`g;`	আগের পরিবর্তনের জায়গায় (changelist)
`g,`	পরের পরিবর্তনের জায়গায়
`:changes`	changelist দেখাও

> 📝 **মনে রাখো:** `j` / `k` / `w` এর মতো ছোট নড়াচড়া jumplist-এ যায় না — শুধু "লাফ"। তাই `Ctrl+o` দিয়ে কোড ঘুরে দেখার পর আগের জায়গায় ফেরা খুব আরামদায়ক।

১৪.১৪ একাধিক ফাইল: Buffers, Windows ও Tabs



- **Buffer** = মেমোরিতে খোলা একটা ফাইল। দেখা না গেলেও খোলা থাকতে পারে ("hidden")।
- **Window** = buffer দেখার একটা জানালা (viewport)।
- **Tab** = কয়েকটা window-এর একটা লেআউট — ব্রাউজারের ট্যাবের মতো "একটা ফাইল" না, বরং একটা ওয়ার্কস্পেস।

> 💡 **টিপস:** অন্য এডিটর থেকে আসা মানুষ প্রায়ই "এক ফাইল = এক tab" ভাবে ব্যবহার করে। Vim-এর স্বাভাবিক পদ্ধতি হলো buffer-এর মধ্যে আসা-যাওয়া (`:b`, `Ctrl+^`); tab ব্যবহার করো আলাদা কাজের আলাদা লেআউটের জন্য।

১৪.১৪.১ Buffers

কমান্ড	কাজ
`:e file.txt`	ফাইল খোলো (নতুন buffer)
`:ls` (`:buffers`)	সব buffer-এর তালিকা
`:bn` / `:bp`	পরের / আগের buffer

কমান্ড	কাজ
`:b 3` / `:b app`	৩ নম্বর buffer / নামের অংশ দিয়ে (Tab দিয়ে কমপ্লিট)
`Ctrl+^` (`Ctrl+6`)	আগের (alternate) buffer-এ আসা-যাওয়া
`:bd` / `:bd 3`	buffer বন্ধ (delete from list)
`:bufdo cmd`	সব buffer-এ কমান্ড চালাও (যেমন `:bufdo %s/a/b/ge | update`)
`:set hidden`	সেভ না করেও অন্য buffer-এ যেতে দাও (Neovim-এ ডিফল্টে চালু)

```
:ls
  1 %a   "app.py"                    line 12
  2 #h + "utils.py"                  line 40
  3      "README.md"                 line 1
```

চিহ্নের মানে: `%` = বর্তমান window-এর buffer, `#` = alternate, `a` = active (দেখা যাচ্ছে), `h` = hidden, `+` = সেভ না করা পরিবর্তন আছে।

Argument list: `vim *.py` দিয়ে খুললে ফাইলগুলো "arglist"-এ থাকে: `:args` (তালিকা), `:next` / `:prev`, `:wn` (সেভ করে পরেরটায়), `:argdo %s/old/new/ge | update` (সব ফাইলে replace করে সেভ)।

১৪.১৪.২ Windows (Split)

কমান্ড / কি	কাজ
`:sp [file]` / `Ctrl+w s`	আড়াআড়ি ভাগ (ওপরে-নিচে)
`:vsp [file]` / `Ctrl+w v`	খাড়া ভাগ (পাশাপাশি)
`:new` / `:vnew`	নতুন খালি buffer-সহ split
`Ctrl+w h/j/k/l`	বাম/নিচ/ওপর/ডান window-এ যাও
`Ctrl+w w`	পরের window-এ (চক্রাকারে)
`Ctrl+w p`	আগের (শেষ ব্যবহৃত) window-এ
`Ctrl+w q` / `Ctrl+w c`	window বন্ধ (`:q` / `:close`)
`Ctrl+w o`	শুধু এই window রাখো, বাকি বন্ধ (`:only`)
`Ctrl+w =`	সব window সমান আকারে
`Ctrl+w _` / `Ctrl+w |`	উচ্চতা / প্রস্থ সর্বোচ্চ করো
`Ctrl+w +` / `Ctrl+w -`	উচ্চতা বাড়াও / কমাও (`10 Ctrl+w +` = ১০ লাইন)
`Ctrl+w >` / `Ctrl+w <`	প্রস্থ বাড়াও / কমাও
`Ctrl+w H/J/K/L`	window-টাকে বাম/নিচ/ওপর/ডান প্রান্তে সরাও
`Ctrl+w r` / `Ctrl+w x`	window ঘোরাও / পাশেরটার সাথে অদলবদল
`Ctrl+w T`	window-টাকে নতুন tab-এ সরাও

কমান্ড / কি	কাজ
`:resize 15` / `:vertical resize 80`	নির্দিষ্ট উচ্চতা / প্রস্থ
`:windo cmd`	সব window-এ কমান্ড

১৪.১৪.৩ Tabs

কমান্ড / কি	কাজ
`:tabnew [file]` / `:tabe file`	নতুন tab
`gt` / `gT`	পরের / আগের tab
`{n}gt`	`n` নম্বর tab-এ (যেমন `3gt`)
`:tabclose` / `:tabonly`	এই tab বন্ধ / শুধু এই tab রাখো
`:tabs`	সব tab ও তাদের window-এর তালিকা
`:tabm 0` / `:tabm`	tab-টাকে প্রথমে / শেষে সরাও
`:tabdo cmd`	সব tab-এ কমান্ড
`vim -p a b c`	প্রতিটা ফাইল আলাদা tab-এ খুলে শুরু

১৪.১৪.৪ ফাইল এক্সপ্লোরার ও diff

- `:Explore` (`:Ex`), `:Sexplore`, `:Vexplore` — Vim-এর বিল্ট-ইন **netrw** ফাইল ব্রাউজার। `vim .` চালালেও এটা খোলে। Enter দিয়ে খোলো, `-` দিয়ে প্যারেন্ট ডিরেক্টরিতে, `%` নতুন ফাইল, `d` নতুন ডিরেক্টরি।
- `vimdiff a b` — পার্থক্য রঙ করে দেখায়। `]c` / `[c` = পরের/আগের পার্থক্য, `do` = অন্য ফাইল থেকে পরিবর্তন আনো (diff obtain), `dp` = এই পরিবর্তন অন্য ফাইলে পাঠাও (diff put), `:diffupdate` = আবার হিসাব।

১৪.১৫ `.vimrc` — নিজের মতো Vim সাজানো

Vim চালু হওয়ার সময় একটা কনফিগ ফাইল পড়ে:

এডিটর	ইউজার কনফিগ	সিস্টেম কনফিগ
Vim	`~/.vimrc` বা `~/.vim/vimrc`	`/etc/vim/vimrc` (Debian/Ubuntu), `/etc/vimrc` (Fedora/RHEL)
Neovim	`~/.config/nvim/init.vim` বা `~/.config/nvim/init.lua`	—

> 📝 **মনে রাখো:** তোমার কোনো `~/.vimrc` না থাকলে Vim 8+ একটা `defaults.vim` লোড করে (কিছু ভালো ডিফল্ট: `incsearch`, `showcmd`, `scrolloff=5`, ফাইল আবার খুললে আগের জায়গায় কার্সর ইত্যাদি)। কিন্তু **নিজের `.vimrc` তৈরি করলেই `defaults.vim` আর লোড হয় না।** সেগুলো রাখতে চাইলে `.vimrc`-এর শুরুতে লেখো:
>
> ```
> unlet! skip_defaults_vim
> source $VIMRUNTIME/defaults.vim
> ```

১৪.১৫.১ অপশন সেট করার নিয়ম

```
:set number          " চালু করো (boolean অপশন)
:set nonumber        " বন্ধ করো ("no" উপসর্গ)
:set number!         " উল্টাও (toggle)
:set number?         " এখন কী অবস্থা দেখাও
:set tabstop=4       " মান দাও
:set tabstop&        " ডিফল্টে ফেরাও
:verbose set tabstop?   " কোন ফাইল এই মান সেট করেছে দেখাও
:setlocal ...        " শুধু বর্তমান buffer/window-এর জন্য
```

Vim-এর ভেতরে `:set` দিলে শুধু এই সেশনে থাকে; স্থায়ী করতে `.vimrc`-এ লেখো (সামনে `:` লাগে না)। `.vimrc`-এ comment শুরু হয় `"` দিয়ে।

১৪.১৫.২ একটা ভালো শুরুর `.vimrc`

```
" ===== ~/.vimrc : নতুনদের জন্য ভালো শুরু =====
" Vim-এর ভালো ডিফল্টগুলো রাখো
unlet! skip_defaults_vim
source $VIMRUNTIME/defaults.vim

" --- দেখা ---
syntax on                   " syntax highlighting
set number                  " লাইন নম্বর
set relativenumber          " আপেক্ষিক লাইন নম্বর (5j, 12k সহজ হয়)
set cursorline              " বর্তমান লাইন হাইলাইট
set showcmd                 " টাইপ করা অসম্পূর্ণ কমান্ড নিচে দেখাও
set showmatch               " বন্ধনী লিখলে জোড়াটা ক্ষণিক দেখাও
set laststatus=2            " সবসময় স্ট্যাটাস লাইন
set scrolloff=5             " কার্সরের ওপরে-নিচে অন্তত ৫ লাইন দেখাও
set wildmenu                " : কমান্ডে Tab চাপলে মেনু
set list listchars=tab:»\ ,trail:·   " Tab আর শেষের স্পেস দেখাও

" --- indentation ---
filetype plugin indent on   " ফাইলের ধরন অনুযায়ী প্লাগইন ও indent
set tabstop=4               " Tab অক্ষর কত কলাম দেখাবে
set shiftwidth=4            " >> << ও auto-indent কত কলাম
set softtabstop=4           " Tab/Backspace চাপলে কত কলাম
set expandtab               " Tab চাপলে স্পেস বসাও
set autoindent              " নতুন লাইনে আগের লাইনের indent
```

```
" --- সার্চ ---
set hlsearch incsearch ignorecase smartcase

" --- আচরণ ---
set hidden                  " সেভ ছাড়াই buffer বদলানো যাবে
set mouse=a                 " মাউস চালু (scroll, ক্লিক, split resize)
set clipboard=unnamedplus   " সিস্টেম ক্লিপবোর্ড (+clipboard লাগবে)
set undofile                " ফাইল বন্ধ করার পরেও undo history
set undodir=~/.vim/undo//   " undo ফাইল এক জায়গায় (ডিরেক্টরি আগে বানাতে হবে)
set encoding=utf-8          " বাংলা সহ সব ইউনিকোড ঠিকঠাক

" --- নিজের শর্টকাট (mapping) ---
" Space কি হবে <leader>
let mapleader = " "
nnoremap <leader>w :w<CR>
nnoremap <leader>q :q<CR>
nnoremap <leader><space> :nohlsearch<CR>
" Insert mode-এ দ্রুত jk টাইপ করলে = Esc
inoremap jk <Esc>
" Ctrl+h/j/k/l দিয়ে split window বদলানো
nnoremap <C-h> <C-w>h
nnoremap <C-j> <C-w>j
nnoremap <C-k> <C-w>k
nnoremap <C-l> <C-w>l

" --- ফাইলের ধরন অনুযায়ী ---
augroup my_filetypes
  autocmd!
  autocmd FileType yaml,html,javascript setlocal tabstop=2 shiftwidth=2 softtabstop=2
  autocmd FileType make setlocal noexpandtab
augroup END
```

> ⚠️ **সতর্কতা:** `:set` লাইনের শেষে `" comment` দেওয়া চলে, কিন্তু map কমান্ডের (`nnoremap`, `inoremap` ...) একই লাইনে `"` comment দিলে সেটা mapping-এরই অংশ হয়ে যায়! তাই ওপরে map-এর comment আলাদা লাইনে লেখা হয়েছে। আর `undodir` ব্যবহারের আগে `mkdir -p ~/.vim/undo` চালাও।

Mapping-এর নিয়ম:

কমান্ড	কোন mode-এ
`nnoremap`	Normal
`inoremap`	Insert
`xnoremap` / `vnoremap`	Visual (vnoremap Select mode-এও)
`cnoremap`	Command-line
`onoremap`	Operator-pending

সবসময় `noremap` (non-recursive) ব্যবহার করো — `map` / `nmap` অন্য mapping-এর ভেতরে আবার mapping ট্রিগার করতে পারে, অপ্রত্যাশিত লুপ হয়। `:map`, `:nmap` লিখলে বর্তমান mapping-গুলো দেখায়।

কি-র নাম: `<CR>` = Enter, `<Esc>`, `<Space>`, `<Tab>`, `<C-x>` = Ctrl+x, `<A-x>` / `<M-x>` = Alt+x, `<leader>`।

`.vimrc` এডিট করে Vim বন্ধ না করেই নতুন সেটিং চালু করতে: `:source ~/.vimrc` (বা `:so %` যদি সেটাই খোলা থাকে)।

১৪.১৫.৩ প্লাগইন

Vim 8+ এর **native package** ব্যবস্থা — কোনো প্লাগইন ম্যানেজার লাগে না:

```
mkdir -p ~/.vim/pack/plugins/start
cd ~/.vim/pack/plugins/start
git clone https://github.com/tpope/vim-surround.git
git clone https://github.com/tpope/vim-commentary.git
# Vim আবার চালু করলেই লোড হবে; help ট্যাগ বানাতে Vim-এ :helptags ALL
```

`start/`-এর প্লাগইন স্বয়ংক্রিয়ভাবে লোড হয়; `opt/`-এরগুলো `:packadd name` দিয়ে দরকারমতো।

জনপ্রিয় প্লাগইন ম্যানেজার: **vim-plug** (Vim ও Neovim), Neovim-এ **lazy.nvim**। জনপ্রিয় প্লাগইন:

প্লাগইন	কাজ
vim-surround	`cs"'` = চারপাশের `"` কে `'` বানাও; `ysiw)` = শব্দের চারপাশে `()`
vim-commentary	`gcc` = লাইন comment/uncomment, `gcap` = প্যারাগ্রাফ (Neovim 0.10+-এ এটা বিল্ট-ইন)
fzf.vim	fuzzy ফাইল/লেখা খোঁজা
NERDTree	সাইডবার ফাইল ট্রি
vim-fugitive	Vim-এর ভেতর থেকে git

> 💡 **টিপস:** প্রথমেই ৩০টা প্লাগইন ইনস্টল করো না। আগে খালি Vim ভালো করে শেখো — রিমোট সার্ভারে তোমার প্লাগইন থাকবে না, কিন্তু বিল্ট-ইন দক্ষতা সব জায়গায় কাজে লাগবে।

১৪.১৫.৪ আরও কিছু বিল্ট-ইন ফিচার

Folding (ভাঁজ করা):

কি	কাজ
`zf{motion}`	fold তৈরি (যেমন `zfap`; `foldmethod=manual` হলে)
`zo` / `zc` / `za`	fold খোলো / বন্ধ করো / উল্টাও
`zR` / `zM`	সব fold খোলো / সব বন্ধ করো
`zd`	fold মুছে ফেলো (লেখা না)
`:set foldmethod=indent`	indentation অনুযায়ী স্বয়ংক্রিয় fold (`syntax`, `marker` ও আছে)

Spell check: `:set spell spelllang=en_us` চালু করলে ভুল বানান লাল আন্ডারলাইন হয়; `]s` / `[s` = পরের/আগের ভুল, `z=` = সংশোধনের প্রস্তাব, `zg` = শব্দটা অভিধানে যোগ, `zw` = ভুল হিসেবে চিহ্নিত।

Completion (Insert mode-এ): `Ctrl+n` / `Ctrl+p` = শব্দ, `Ctrl+x Ctrl+f` = ফাইলের পাথ, `Ctrl+x Ctrl+l` = পুরো লাইন, `Ctrl+x Ctrl+o` = omni (ভাষা-ভিত্তিক), `Ctrl+x Ctrl+k` = অভিধান (`dictionary` সেট থাকলে)।

Abbreviation: `:iabbrev teh the` — `teh` লিখে স্পেস দিলেই `the` হয়ে যায়।

১৪.১৬ Swap File, Crash Recovery আর Backup

ধরো Vim-এ কাজ করছিলে, SSH কেটে গেল বা বিদ্যুৎ চলে গেল। আবার ফাইলটা খুললে দেখবে:

```
$ vim notes.txt
E325: ATTENTION
Found a swap file by the name ".notes.txt.swp"
          owned by: rahim   dated: Tue Nov 11 10:42:07 2026
         file name: ~rahim/notes.txt
          modified: YES
         user name: rahim   host name: laptop
        process ID: 4123
While opening file "notes.txt"
...
Swap file ".notes.txt.swp" already exists!
[O]pen Read-Only, (E)dit anyway, (R)ecover, (D)elete it, (Q)uit, (A)bort:
```

পছন্দ	কখন বেছে নেবে
`O` (Open Read-Only)	শুধু দেখতে চাও, এডিট করবে না
`E` (Edit anyway)	swap উপেক্ষা করে এডিট — বিপজ্জনক, অন্য কেউ এডিট করলে একজনের কাজ হারাবে
`R` (Recover)	crash হয়েছিল, সেভ না করা কাজ ফিরিয়ে আনো
`D` (Delete it)	swap-এর দরকার নেই (আগেই উদ্ধার করেছ বা পরিবর্তন অপ্রয়োজনীয়)
`Q` / `A`	বের হও / বাতিল

নিরাপদ উদ্ধারের ধাপ:

1. `R` চাপো → Vim swap থেকে লেখা ফিরিয়ে আনে।
2. দেখে নাও সব ঠিক আছে কিনা; চাইলে `:w notes.recovered.txt` দিয়ে আলাদা সেভ করে `diff` দিয়ে মিলিয়ে দেখো।
3. `:w` দিয়ে সেভ করো।
4. Vim থেকে বেরিয়ে swap file মুছে ফেলো: `rm .notes.txt.swp` (নইলে পরের বার আবার একই সতর্কবার্তা আসবে)। অথবা আবার খুলে `D` চাপো।

> ⚠️ **সতর্কতা:** "process ID: 4123" এর পাশে `(STILL RUNNING)` লেখা থাকলে মানে অন্য কোনো টার্মিনালে (বা tmux-এ) ফাইলটা এখনো খোলা! তখন `E` চেপো না — সেই Vim-এ গিয়ে কাজ করো।

অন্যান্য কমান্ড: `vim -r` = সব উদ্ধারযোগ্য swap file-এর তালিকা; `vim -r notes.txt` = সরাসরি উদ্ধার; `:recover`। swap file কোথায় রাখবে: `set directory=~/.vim/swap//` (শেষে `//` দিলে পুরো পাথ দিয়ে নাম হয়, ভিন্ন ডিরেক্টরির একই নামের ফাইলে সংঘর্ষ হয় না)। Neovim ডিফল্টে swap রাখে `~/.local/state/nvim/swap/`-এ।

১৪.১৭ সাহায্য পাওয়া: `vimtutor` আর `:help`

১৪.১৭.১ vimtutor — সবার আগে এটা করো

```
$ vimtutor
```

এটা একটা ইন্টারঅ্যাকটিভ টিউটোরিয়াল (প্রায় ২৫–৩০ মিনিট)। একটা অস্থায়ী কপি খুলে প্রতিটা পাঠে তুমি নিজে হাতে কমান্ড চালিয়ে শেখো — আসল ফাইল নষ্ট হওয়ার ভয় নেই। সাধারণত পূর্ণ `vim` প্যাকেজের সাথে আসে। কয়েকটা ভাষায় অনুবাদও আছে (`vimtutor fr`, `vimtutor de` ইত্যাদি)। Neovim-এ এর সমতুল্য হলো `:Tutor`।

> 💡 **টিপস:** প্রথম সপ্তাহে প্রতিদিন একবার `vimtutor` করো। তৃতীয়-চতুর্থ দিনে খেয়াল করবে আঙুল নিজে থেকেই সঠিক কি চাপছে।

১৪.১৭.২ বিল্ট-ইন help

Vim-এর ডকুমেন্টেশন অসাধারণ বিস্তারিত, আর পুরোটা Vim-এর ভেতরেই আছে।

কমান্ড	কী দেখায়
`:help` বা `F1`	মূল help পাতা
`:help dd`	Normal mode কমান্ড `dd`
`:help i_CTRL-W`	Insert mode-এ `Ctrl+w` (`i_` উপসর্গ)
`:help v_o`	Visual mode-এ `o` (`v_` উপসর্গ)
`:help c_CTRL-R`	Command-line mode-এ `Ctrl+r` (`c_` উপসর্গ)
`:help :s`	Ex কমান্ড `:substitute`
`:help 'tabstop'`	অপশন (একক উদ্ধৃতিতে)
`:help text-objects`	বিষয় অনুযায়ী

কমান্ড	কী দেখায়
`:help user-manual`	পুরো ইউজার ম্যানুয়াল (বইয়ের মতো পড়া যায়)
`:helpgrep pattern`	সব help-এ খোঁজো (`:cnext` দিয়ে পরেরটা)
`Ctrl+]`	help-এর ভেতরে লিংকে (`
`Ctrl+o` / `Ctrl+t`	লিংক থেকে ফিরে আসো
`:help` তারপর `Ctrl+d`	টাইপ করা অংশের সাথে মেলা সব help টপিক দেখাও

১৪.১৮ Neovim — দ্রুত তুলনা

বিষয়	Vim	Neovim
কমান্ড	`vim`	`nvim`
কনফিগ	`~/.vimrc`	`~/.config/nvim/init.vim` বা `init.lua`
স্ক্রিপ্ট ভাষা	Vimscript, Vim9 script	Vimscript + Lua (Vim9 script নেই)
ডিফল্ট	কম পরিবর্তিত (পুরোনো সামঞ্জস্য)	আধুনিক ডিফল্ট: `hidden`, `hlsearch`, `incsearch`, `autoindent`, `mouse=nvi`, `Y` = `y$` ইত্যাদি
LSP (কোড বুদ্ধিমত্তা)	প্লাগইন লাগে	বিল্ট-ইন LSP client
ক্লিপবোর্ড	`+clipboard` বিল্ড লাগে	বাইরের টুল (`wl-copy`, `xclip`)
স্বাস্থ্য পরীক্ষা	—	`:checkhealth`
টিউটোরিয়াল	`vimtutor`	`:Tutor`

এই অধ্যায়ের সব মৌলিক কমান্ড — mode, motion, operator, text object, register, macro, mark, split — দুটোতেই একই। তাই যেটা দিয়েই শুরু করো, দক্ষতা অন্যটাতে সরাসরি কাজে লাগবে।

> 💡 **টিপস:** bash-এও Vim-এর কিবাইন্ডিং পাওয়া যায়: `set -o vi` দিলে কমান্ড লাইনে `Esc` চেপে Normal mode-এ গিয়ে `b`, `w`, `dw`, `cw` চালানো যায় (অধ্যায় ৬)। আর কমান্ড লাইনে লম্বা কমান্ড লিখতে লিখতে `Ctrl+x Ctrl+e` চাপলে সেটা `$EDITOR`-এ (Vim) খোলে, সেভ করে বের হলে চলে।

১৪.১৯ সব শর্টকাটের বিশাল টেবিল

এই টেবিলটা প্রিন্ট করে মনিটরের পাশে রাখতে পারো। (সব কমান্ড Normal mode-এ, আলাদা উল্লেখ না থাকলে। পুরো বইয়ের সব শর্টকাট একসাথে পাবে পরিশিষ্ট ক-তে।)

ক) Mode বদলানো ও বের হওয়া

কি	কাজ
`Esc` / `Ctrl+[`	Normal mode-এ ফেরো
`i` `a` `I` `A` `o` `O`	Insert mode (কার্সরের আগে / পরে / লাইনের শুরু / শেষ / নিচে নতুন লাইন / ওপরে নতুন লাইন)
`s` / `S`	অক্ষর / লাইন মুছে Insert
`R`	Replace mode
`v` / `V` / `Ctrl+v`	Visual: অক্ষর / লাইন / ব্লক
`:`	Command-line mode
`:w` / `:q` / `:wq` / `:x` / `:q!`	সেভ / বের হও / সেভ করে বের / দরকার হলে সেভ করে বের / সেভ না করে বের
`ZZ` / `ZQ`	`:x` / `:q!`
`:qa!` / `:wqa`	সব বন্ধ (জোর করে) / সব সেভ করে বন্ধ
`:e!`	শেষ সেভে ফিরে যাও

খ) চলাফেরা

কি	কাজ
`h` `j` `k` `l`	বাম, নিচ, ওপর, ডান
`w` `b` `e` `ge`	পরের word শুরু, আগের word শুরু, word শেষ, আগের word শেষ
`W` `B` `E` `gE`	একই, WORD ধরে
`0` `^` `$` `g_`	লাইনের শুরু, প্রথম অক্ষর, শেষ, শেষ অ-ফাঁকা অক্ষর
`f{c}` `F{c}` `t{c}` `T{c}`	লাইনে অক্ষর খোঁজা (find/till)
`;` `,`	শেষ f/t একই দিকে / উল্টো দিকে
`gg` `G` `{n}G` `:{n}`	ফাইলের শুরু, শেষ, `n` নম্বর লাইন
`{n}%`	ফাইলের `n` % অবস্থানে
`%`	মিলে যাওয়া বন্ধনী
`{` `}`	আগের / পরের প্যারাগ্রাফ
`(` `)`	আগের / পরের বাক্য
`H` `M` `L`	স্ক্রিনের ওপর, মাঝ, নিচ
`Ctrl+f` `Ctrl+b`	এক পাতা নিচে / ওপরে
`Ctrl+d` `Ctrl+u`	আধা পাতা নিচে / ওপরে
`Ctrl+e` `Ctrl+y`	স্ক্রিন এক লাইন নিচে / ওপরে

কি	কাজ
`zz` `zt` `zb`	বর্তমান লাইন স্ক্রিনের মাঝে / ওপরে / নিচে
`gd` `gf`	local definition / কার্সরের নিচের ফাইলের নাম খোলো
`Ctrl+o` `Ctrl+i`	jumplist পেছনে / সামনে
`g;` `g,`	আগের / পরের পরিবর্তনের জায়গা

গ) এডিটিং

কি	কাজ
`x` `X`	কার্সরের / বামের অক্ষর মোছো
`dd` `D`	লাইন মোছো / লাইনের শেষ পর্যন্ত মোছো
`yy` (`Y`)	লাইন কপি
`p` `P`	পরে / আগে পেস্ট
`gp` `gP` `]p`	পেস্ট করে কার্সর পরে নাও / indent মিলিয়ে পেস্ট
`r{c}`	একটা অক্ষর বদলাও
`cw` `ciw` `cc` `C`	word বদলাও / পুরো word / পুরো লাইন / লাইনের শেষ পর্যন্ত
`J` `gJ`	লাইন জোড়া (স্পেসসহ / ছাড়া)
`u` `Ctrl+r` `U`	undo / redo / লাইনের সব পরিবর্তন উল্টাও
`.`	শেষ পরিবর্তন আবার
`~` `g~` `gu` `gU`	case উল্টাও / উল্টাও (operator) / ছোট হাতের / বড় হাতের
`>>` `<<` `==`	indent বাড়াও / কমাও / auto
`gg=G`	পুরো ফাইল re-indent
`Ctrl+a` `Ctrl+x`	সংখ্যা বাড়াও / কমাও
`xp` `ddp`	দুই অক্ষর / দুই লাইন অদলবদল
`:earlier 5m` `:later 5m`	সময় ধরে undo / redo

ঘ) Operator ও Text object

Operator		Text object	
`d`	delete	`iw` `aw`	word
`c`	change	`iW` `aW`	WORD
`y`	yank	`is` `as`	বাক্য
`>` `<`	indent	`ip` `ap`	প্যারাগ্রাফ
`=`	auto-indent	`i"` `a"` `i'` `a'`	quote

Operator		Text object	
`gu` `gU` `g~`	case	`i(` `a(` (`ib` `ab`)	`( )`
`gq`	format/wrap	`i{` `a{` (`iB` `aB`)	`{ }`
`!`	বাইরের কমান্ডে filter	`i[` `a[` `i<` `a<`	`[ ]`, `< >`
`zf`	fold	`it` `at`	HTML/XML tag

ঙ) Insert mode

কি	কাজ
`Ctrl+w` `Ctrl+u` `Ctrl+h`	word / লাইনের শুরু পর্যন্ত / এক অক্ষর মোছো
`Ctrl+t` `Ctrl+d`	indent বাড়াও / কমাও
`Ctrl+n` `Ctrl+p`	শব্দ auto-complete
`Ctrl+x Ctrl+f` `Ctrl+x Ctrl+l`	ফাইলের পাথ / পুরো লাইন complete
`Ctrl+r {reg}`	register পেস্ট
`Ctrl+r =`	হিসাব করে ফল বসাও
`Ctrl+o {cmd}`	একটা Normal কমান্ড
`Ctrl+v {key}`	আক্ষরিক অক্ষর
`Ctrl+a`	শেষবার লেখা লেখা আবার বসাও

চ) সার্চ ও Replace

কি	কাজ
`/pat` `?pat`	সামনে / পেছনে খোঁজো
`n` `N`	পরেরটা / উল্টো দিকে
`*` `#`	কার্সরের শব্দ সামনে / পেছনে
`gn` `cgn`	পরের মিল সিলেক্ট / বদলাও (তারপর `.`)
`:noh`	হাইলাইট বন্ধ
`:s/a/b/`	বর্তমান লাইনে প্রথমটা
`:%s/a/b/g`	পুরো ফাইলে সব
`:%s/a/b/gc`	জিজ্ঞেস করে করে
`:%s/a//gn`	শুধু গোনো
`&` `g&`	শেষ `:s` এই লাইনে / পুরো ফাইলে আবার
`:g/pat/d` `:v/pat/d`	মেলা / না-মেলা লাইন মোছো

ছ) Visual mode

কি	কাজ
`o` `O`	সিলেকশনের অন্য প্রান্ত / অন্য কোণ (block)
`gv`	শেষ সিলেকশন আবার
`I` `A` (block)	সব লাইনে আগে / পরে লেখা
`$` (block)	প্রতিটা লাইনের শেষ পর্যন্ত বাড়াও
`d` `y` `c` `p` `r` `J` `>` `<` `=` `u` `U` `~`	মোছো, কপি, বদলাও, পেস্ট, অক্ষর বদলাও, জোড়া, indent, case
`g Ctrl+a`	সংখ্যা ক্রমবর্ধমানভাবে বাড়াও
`:`	সিলেক্ট করা লাইনে Ex কমান্ড (`:'<,'>`)

জ) Registers, Macros, Marks

কি	কাজ
`"ayy` `"ap`	register a-তে কপি / থেকে পেস্ট
`"Ayy`	register a-তে যোগ
`"0p`	শেষ কপি করা লেখা পেস্ট
`"+y` `"+p`	সিস্টেম ক্লিপবোর্ডে কপি / থেকে পেস্ট
`"_d`	কোথাও জমা না রেখে মোছো
`:reg`	সব register দেখাও
`qa` … `q`	macro a রেকর্ড
`@a` `@@` `10@a`	macro চালাও / আবার / ১০ বার
`:%normal @a`	প্রতিটা লাইনে macro
`ma` `mA`	লোকাল / গ্লোবাল mark
`'a` `` `a ``	mark-এর লাইনে / ঠিক অবস্থানে
``` `` ``` ` '' `	শেষ লাফের আগের জায়গা
``` `. ```	শেষ পরিবর্তনের জায়গা
`:marks`	সব mark

ঝ) Buffers, Windows, Tabs

কি	কাজ
`:e file` `:ls` `:bn` `:bp` `:b N` `:bd`	buffer খোলো / তালিকা / পরের / আগের / নম্বরে / বন্ধ
`Ctrl+^`	আগের ফাইলে আসা-যাওয়া

কি	কাজ
`:sp` `:vsp` (`Ctrl+w s` `Ctrl+w v`)	আড়াআড়ি / খাড়া split
`Ctrl+w h/j/k/l` `Ctrl+w w`	window বদলাও
`Ctrl+w q` `Ctrl+w o` `Ctrl+w =`	বন্ধ / শুধু এটা / সমান আকার
`Ctrl+w +/-/</>` `Ctrl+w _` `Ctrl+w |`	আকার বদলাও / সর্বোচ্চ উচ্চতা / সর্বোচ্চ প্রস্থ
`Ctrl+w H/J/K/L` `Ctrl+w T`	window সরাও / নতুন tab-এ পাঠাও
`:tabnew` `gt` `gT` `{n}gt` `:tabclose`	tab খোলো / পরের / আগের / নম্বরে / বন্ধ
`:Ex`	ফাইল এক্সপ্লোরার
`]c` `[c` `do` `dp`	vimdiff: পরের/আগের পার্থক্য, আনো / পাঠাও

ঞ) বিবিধ

কি	কাজ
`:!cmd` `:r !cmd` `:%!cmd`	শেল কমান্ড / আউটপুট ঢোকাও / filter
`:r file`	ফাইলের লেখা ঢোকাও
`:sort u`	সাজাও, ডুপ্লিকেট বাদ
`:set nu!` `:set rnu!` `:set paste`	লাইন নম্বর / আপেক্ষিক নম্বর উল্টাও / paste mode
`:syntax on`	syntax highlight
`za` `zR` `zM`	fold উল্টাও / সব খোলো / সব বন্ধ
`]s` `z=` `zg`	পরের ভুল বানান / প্রস্তাব / অভিধানে যোগ
`q:` `q/`	Ex / সার্চ হিস্ট্রি window
`Ctrl+g`	ফাইলের নাম, মোট লাইন, অবস্থান দেখাও
`g Ctrl+g`	শব্দ, অক্ষর, বাইট গোনো
`ga`	কার্সরের অক্ষরের ASCII/Unicode মান
`:help x` `Ctrl+]` `Ctrl+o`	help / লিংকে যাও / ফেরো

হাতে-কলমে অনুশীলন

ল্যাব ১: অনুশীলনের ফাইল তৈরি

```
mkdir -p ~/vimlab && cd ~/vimlab
cat > practice.txt <<'TXT'
The quick brown fox jumps over the lazy dog.
Linux is free and open source.
teh cat sat on teh mat.
apple
```

```
banana
apple
cherry
name = "old value"
print("Hello, World")
rahim, 25
karim, 30
salma, 28
DEBUG: starting
ERROR: disk full
DEBUG: retry
TXT
vim practice.txt
```

ল্যাব ২: চলাফেরা (কিছু এডিট না করে)

1. `G` দিয়ে শেষ লাইনে যাও, `gg` দিয়ে প্রথমে ফেরো।
2. `:9` দিয়ে `print(...)` লাইনে যাও। `f(` চাপো, তারপর `%` চাপো — কার্সর `)`-এ যাবে।
3. প্রথম লাইনে গিয়ে `w` কয়েকবার, তারপর `b` কয়েকবার, `e`, `$`, `0`, `^` চেষ্টা করো।
4. `fo` চাপো, তারপর `;` `;` — প্রতিটা `o`-তে কার্সর থামবে।
5. `Ctrl+g` চাপো — নিচে ফাইলের নাম ও মোট লাইন দেখো।

ল্যাব ৩: এডিটিং ও dot

1. ৩ নম্বর লাইনে যাও (`3G`)। `/teh` + Enter দিয়ে প্রথম `teh`-এ যাও।
2. `cw` চাপো, `the` লেখো, `Esc`।
3. `n` চাপো (পরের `teh`), তারপর `.` — বদলে যাবে।
4. লাইন ১৩ (`DEBUG: starting`) এ গিয়ে `dd` চাপো, তারপর `u` দিয়ে ফিরিয়ে আনো, `Ctrl+r` দিয়ে আবার মোছো, আবার `u`।
5. লাইন ২-এর শেষে `Yes!` যোগ করো: `2G`, `A`, `Yes!`, `Esc`।

ল্যাব ৪: text object

1. `name = "old value"` লাইনে যাও, কার্সর `name`-এর ওপর রাখো।
2. `ci"` চাপো — quote-এর ভেতরটা মুছবে (কার্সর quote-এর বাইরে হলেও!)। `new value` লেখো, `Esc`।
3. `print("Hello, World")` লাইনে `W`-এর ওপর কার্সর রেখে `diw` → `World` মুছবে; `u`।
4. কার্সর বন্ধনীর ভেতরে রেখে `ci(` চাপো, `"Bangladesh"` লেখো, `Esc`।
5. `apple` লাইনে গিয়ে `dap` চাপো — কী হলো দেখো (ফাইলে কোনো ফাঁকা লাইন নেই, তাই পুরো ফাইলটাই একটা প্যারাগ্রাফ — সব মুছে গেল!), তারপর `u`।

ল্যাব ৫: সার্চ ও replace

1. `:%s/apple/mango/gc` চালাও; প্রথমটায় `y`, দ্বিতীয়টায় `n` দাও।

2. `:%s/DEBUG//gn` — কতগুলো DEBUG আছে গুনে দেখো (বদলাবে না)।
3. `:g/DEBUG/d` — সব DEBUG লাইন মুছে ফেলো। `u` দিয়ে ফিরিয়ে আনো।
4. লাইন ৪–৭ (`apple`, `banana` ...) `V` দিয়ে সিলেক্ট করো, তারপর `:sort u` (নিচে `:'<,'>sort u` দেখাবে)।

ল্যাব ৬: block mode

1. `rahim, 25` লাইনের শুরুতে কার্সর নাও (`0`)।
2. `Ctrl+v`, `2j` চাপো (৩ লাইনের প্রথম কলাম সিলেক্ট)।
3. `I`, `- ` লেখো, `Esc` — তিন লাইনের শুরুতেই `- ` বসবে।
4. আবার তিন লাইন `Ctrl+v` দিয়ে সিলেক্ট করে `$`, `A`, `years` লেখো, `Esc`।
5. `u` `u` দিয়ে দুটোই undo করো (পরের ল্যাবের জন্য)।

ল্যাব ৭: macro

`rahim, 25` থেকে শুরু করে তিন লাইনকে `INSERT INTO users VALUES ('rahim', 25);` বানাও — ১৪.১২.১ অংশের ধাপগুলো অনুসরণ করো (`qa` ... `q`, তারপর `2@a`)।

ল্যাব ৮: register

1. লাইন ১-এ গিয়ে `"ayy` (register a-তে কপি)।
2. লাইন ২-এ গিয়ে `"byy`।
3. শেষ লাইনে গিয়ে (`G`) `"ap` আর `"bp`।
4. `:reg a b` দিয়ে দেখো।
5. একটা শব্দ `yiw` দিয়ে কপি করে অন্য শব্দে `diw` করো, তারপর `p` আর `"0p`-এর পার্থক্য দেখো।

ল্যাব ৯: split ও buffer

1. `:vsp ~/.bashrc` — পাশাপাশি দুটো ফাইল।
2. `Ctrl+w l`, `Ctrl+w h` দিয়ে বদলাও।
3. `:ls` দেখো, `Ctrl+^` চেষ্টা করো।
4. `Ctrl+w o` দিয়ে শুধু একটা window রাখো, `:bd` দিয়ে `.bashrc` buffer বন্ধ করো।

ল্যাব ১০: নিজের `.vimrc`

1. `vim ~/.vimrc` খোলো, ১৪.১৫.২ অংশের কনফিগ থেকে যেগুলো পছন্দ সেগুলো লেখো।
2. `:w` তারপর `:so %`।
3. `practice.txt` আবার খোলো — লাইন নম্বর, হাইলাইট ইত্যাদি দেখো।
4. শেষে `:wq` দিয়ে বের হয়ে টার্মিনালে `vimtutor` চালাও এবং পুরোটা শেষ করো।

সারসংক্ষেপ

- Vim হলো **modal** এডিটর: Normal (বাড়ি), Insert, Visual, Command-line, Replace। সন্দেহ হলেই `Esc`।
- vi (১৯৭৬, Bill Joy) → Vim (১৯৯১, Bram Moolenaar) → Neovim (২০১৪)। মূল কিবাইন্ডিং তিনটাতেই প্রায় এক।
- Vim মেমোরিতে **buffer** এডিট করে; `:w` না করা পর্যন্ত ডিস্ক বদলায় না। crash হলে **swap file** থেকে উদ্ধার।
- এডিটিং একটা ভাষা: **operator + count + motion/text object** (`d3w`, `ci"`, `yap`, `gUiw`) আর `.` দিয়ে পুনরাবৃত্তি।
- registers অনেকগুলো ক্লিপবোর্ড; macros হলো রেকর্ড করা কি-চাপ; marks হলো বুকমার্ক।

কাজ	কমান্ড
খোলা / নির্দিষ্ট লাইনে	`vim file`, `vim +N file`
Insert mode	`i a o O I A`
সেভ / বের হওয়া	`:w`, `:q`, `:wq`, `:x`, `ZZ`, `:q!`
চলা	`hjkl`, `w b e`, `0 ^ $`, `gg G`, `%`, `{ }`, `Ctrl+f/b/d/u`
মোছা / কপি / পেস্ট	`x`, `dd`, `D`, `yy`, `p`, `P`
undo / redo / repeat	`u`, `Ctrl+r`, `.`
বদলানো	`r`, `R`, `cw`, `ciw`, `cc`, `C`
text objects	`iw aw`, `i" a"`, `i( a(`, `i{ a{`, `ip ap`, `it at`
সার্চ	`/pat`, `?pat`, `n N`, `* #`
replace	`:%s/old/new/gc`
global	`:g/pat/d`, `:v/pat/d`
visual	`v`, `V`, `Ctrl+v` → `I` / `A`
register	`"ayy`, `"ap`, `"0p`, `"+y`, `:reg`
macro	`qa … q`, `@a`, `@@`
mark	`ma`, `'a`, `` `a ``, `` `` ``
buffer / window / tab	`:ls :bn :b N`, `:sp :vsp Ctrl+w`, `:tabnew gt gT`
কনফিগ	`~/.vimrc`, `:set opt`, `:so %`
শেখা	`vimtutor`, `:help topic`

নিজেকে যাচাই করো

1. Vim-কে "modal" এডিটর বলা হয় কেন? Normal mode-কে "বাড়ি" বলা হয় কেন?
2. `:wq`, `:x` আর `:q!`-এর পার্থক্য কী? `ZZ` কোনটার সমান?
3. `dw`, `diw` আর `daw`-এর পার্থক্য কী? কার্সর শব্দের মাঝখানে থাকলে কোনটা পুরো শব্দ মোছে?
4. ফাইলের সব `foo` কে `bar` দিয়ে বদলাতে চাও, কিন্তু `food` শব্দটা অক্ষত রাখতে চাও, এবং প্রতিটা বদলানোর আগে নিশ্চিত হতে চাও — কমান্ড কী হবে?
5. একটা শব্দ `yiw` দিয়ে কপি করলে, তারপর অন্য শব্দ `diw` করে `p` চাপলে কপি করা শব্দটা আসে না কেন? সমাধান কী?
6. ১০টা লাইনের শুরুতে `// ` বসানোর দ্রুততম দুটো উপায় লেখো।
7. ভুল করে `Ctrl+s` চাপার পর Vim সাড়া দিচ্ছে না। কী হয়েছে, আর কীভাবে ঠিক করবে?
8. নিজের `~/.vimrc` বানানোর পর হঠাৎ কিছু সুবিধা (যেমন incsearch, ফাইলে আগের অবস্থানে ফেরা) চলে গেল কেন?
9. `E325: ATTENTION` বার্তা কেন আসে? `(STILL RUNNING)` লেখা থাকলে কী করবে?
10. একটা macro-কে ফাইলের ৫ থেকে ৫০ নম্বর প্রতিটা লাইনে কীভাবে চালাবে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. কারণ একই কি ভিন্ন mode-এ ভিন্ন কাজ করে — Normal mode-এ `d` মোছার কমান্ড, Insert mode-এ `d` অক্ষর। প্রোগ্রামাররা লেখার চেয়ে বেশি সময় পড়া, চলা আর বদলানোয় কাটায়; Normal mode-এ প্রতিটা কি একটা কমান্ড, তাই সেখানেই বেশিরভাগ সময় থাকা হয় এবং প্রতিটা কাজ শেষে `Esc` দিয়ে সেখানে ফেরা হয়।
2. `:wq` সবসময় ফাইল লেখে তারপর বের হয় (পরিবর্তন না থাকলেও; ফাইলের modification time বদলায়)। `:x` শুধু পরিবর্তন থাকলে লেখে, তারপর বের হয়। `:q!` কিছু না লিখে, সব পরিবর্তন ফেলে বের হয়। `ZZ` = `:x`।
3. `dw` কার্সর থেকে পরের word-এর শুরু পর্যন্ত মোছে (শব্দের মাঝে থাকলে শব্দের বাকি অংশ আর পরের স্পেস)। `diw` কার্সর যেখানেই থাকুক পুরো শব্দ মোছে, পাশের স্পেস রেখে। `daw` পুরো শব্দ আর পাশের একটা স্পেস মোছে। মাঝখান থেকে পুরো শব্দ মোছে `diw` ও `daw`।
4. `:%s/\<foo\>/bar/gc` — `\<` `\>` word boundary, `g` লাইনের সব মিল, `c` প্রতিবার জিজ্ঞেস।
5. `diw`-ও একটা "cut"; সেটা unnamed register (`""`) বদলে দেয়, তাই `p` মোছা শব্দটা পেস্ট করে। সমাধান: `"0p` (yank register-এ শেষ কপি থাকে, delete সেটাকে বদলায় না), অথবা মোছার সময় `"_diw` (black hole)।
6. (ক) প্রথম লাইনের শুরুতে `Ctrl+v`, `9j`, `I`, `// `, `Esc`। (খ) `:N,M s#^#// #` (যেমন `:1,10s#^#// #`), বা ১০ লাইন `V` দিয়ে সিলেক্ট করে `:s/^/\/\/ /`। (আরেক উপায়: `:1,10normal I// `।)
7. `Ctrl+s` টার্মিনালের XOFF flow control — আউটপুট থামিয়ে দেয়, Vim এর জন্য দায়ী না। `Ctrl+q` (XON) চাপলে আবার চলবে।
8. কোনো ইউজার vimrc না থাকলে Vim `defaults.vim` লোড করে। `~/.vimrc` বানালে সেটা আর লোড হয় না। সমাধান: `.vimrc`-এর শুরুতে `unlet! skip_defaults_vim` আর `source`
</details>

`$VIMRUNTIME/defaults.vim`।

9. ফাইলটার জন্য আগে থেকেই একটা swap file আছে — হয় আগের Vim crash/kill হয়েছিল, নয়তো অন্য কোথাও ফাইলটা এখনো খোলা। `(STILL RUNNING)` মানে অন্য Vim প্রসেস এখনো চলছে — `E` দিয়ে এডিট না করে `O` / `Q` বেছে সেই Vim-এ গিয়ে কাজ করো, নইলে দুজনের পরিবর্তন একে অন্যকে মুছে দেবে।
10. `:5,50normal @a` (macro যে register-এ আছে সেই নাম দিয়ে)। অথবা ৫ নম্বর লাইনে গিয়ে, macro-র শেষে `j` থাকলে, `46@a`।

অধ্যায় ১৫: ইউজার ও গ্রুপ ম্যানেজমেন্ট

> "লিনাক্স একটা বড় অ্যাপার্টমেন্ট বিল্ডিং। প্রত্যেক ভাড়াটিয়ার নিজের ফ্ল্যাট আর নিজের চাবি আছে। দারোয়ান (kernel) নাম দেখে না, দেখে ফ্ল্যাট নম্বর। আর বিল্ডিংয়ের মালিক (root) সব দরজা খুলতে পারে।"

এই অধ্যায়ে যা শিখবে

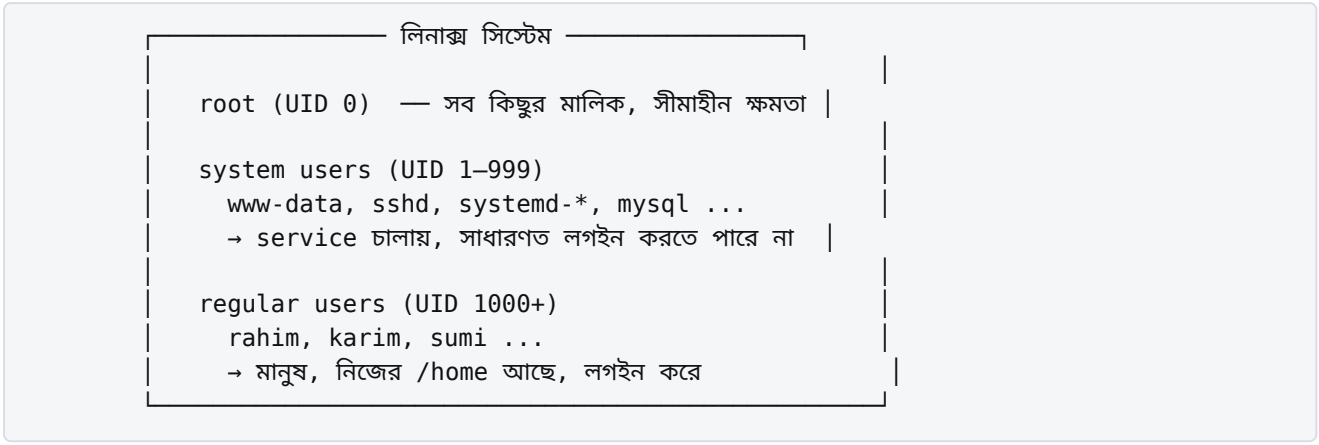
- লিনাক্স কেন **multi-user** সিস্টেম, আর এর ফলে নিরাপত্তা কীভাবে তৈরি হয়
- **UID** ও **GID** কী, kernel কেন নাম না দেখে শুধু নম্বর দেখে
- **system user** আর **regular user**-এর পার্থক্য, কোন নম্বর-রেঞ্জ কার জন্য
- `/etc/passwd`, `/etc/shadow`, `/etc/group`, `/etc/gshadow` ফাইলের প্রতিটি ফিল্ড
- ইউজার তৈরি, পরিবর্তন ও মুছে ফেলা: `useradd`, `adduser`, `usermod`, `userdel`
- পাসওয়ার্ড ও তার মেয়াদ: `passwd`, `chage`, `chpasswd`
- গ্রুপ ম্যানেজমেন্ট: `groupadd`, `groupmod`, `groupdel`, `gpasswd`, `newgrp`
- কে আমি, কে লগইন করে আছে: `id`, `groups`, `whoami`, `who`, `w`, `last`, `lastlog`
- `su` দিয়ে ইউজার বদলানো, **login shell** বনাম non-login shell
- `/etc/skel` দিয়ে নতুন ইউজারের ডিফল্ট ফাইল সাজানো
- `nologin` দিয়ে এমন অ্যাকাউন্ট বানানো যেটা দিয়ে লগইন করা যায় না
- ভেতরে কী ঘটে: login → PAM → NSS → kernel credential

১৫.১ multi-user ধারণা — লিনাক্স কেন একসাথে অনেকের

Windows-এর পুরোনো দিনের কম্পিউটার মূলত একজন মানুষের জন্য বানানো ছিল। লিনাক্সের পূর্বপুরুষ Unix জন্মেছিল ১৯৭০-এর দশকে, যখন একটা বড় কম্পিউটারে একসাথে অনেক মানুষ terminal দিয়ে কাজ করত। তাই শুরু থেকেই লিনাক্স **multi-user** (একসাথে বহু ব্যবহারকারী) এবং **multi-tasking** (একসাথে বহু কাজ) সিস্টেম।

আজ তোমার ল্যাপটপে হয়তো তুমি একাই মানুষ, কিন্তু তবুও সিস্টেমে ডজনখানেক "ইউজার" আছে: `root`, `www-data`, `systemd-network`, `messagebus`, `nobody` ইত্যাদি। এরা মানুষ না, এরা বিভিন্ন **service** (পেছনে চলা প্রোগ্রাম)-এর পরিচয়।

কেন এমন? কারণ **least privilege** (ন্যূনতম ক্ষমতা) নীতি: প্রত্যেক প্রোগ্রাম শুধু ততটুকু ক্ষমতা পাবে যতটুকু তার দরকার। ধরো ওয়েব সার্ভার `nginx` হ্যাক হলো। সেটা যদি `root` হিসেবে চলত, হ্যাকার পুরো সিস্টেম পেয়ে যেত। কিন্তু সেটা চলে `www-data` (Debian/Ubuntu) বা `nginx` (Fedora/RHEL) ইউজার হিসেবে, তাই হ্যাকার শুধু ওই ইউজারের সীমিত ফাইলেই হাত দিতে পারবে।



উপমা: অ্যাপার্টমেন্ট বিল্ডিং। `root` = বাড়িওয়ালা, মাস্টার-কী আছে। system user = লিফটম্যান, গার্ড, ক্লিনার — প্রত্যেকে নিজের কাজের ঘরের চাবি পায়, কারো ফ্ল্যাটে ঢুকতে পারে না। regular user = ভাড়াটিয়া, নিজের ফ্ল্যাট (`/home/rahim`) আছে। **group** = একই ফ্লোরের ভাড়াটিয়ারা, যারা একটা কমন ছাদ বা স্টোররুম শেয়ার করে।

> 📝 **মনে রাখো:** লিনাক্সে প্রতিটি file ও প্রতিটি process-এর একজন মালিক (user) এবং একটি গ্রুপ থাকে। ইউজার-গ্রুপ না বুঝলে পরের অধ্যায়ের ফাইল পারমিশন বোঝা অসম্ভব।

১৫.২ UID ও GID — kernel নাম চেনে না, নম্বর চেনে

UID (User ID) হলো প্রতিটি ইউজারের একটি পূর্ণসংখ্যা পরিচয়-নম্বর। **GID (Group ID)** হলো প্রতিটি গ্রুপের নম্বর।

এখানে সবচেয়ে গুরুত্বপূর্ণ কথা: **kernel "rahim" নামটা জানে না।** kernel-এর কাছে rahim মানে শুধু `1000` । ডিস্কে প্রতিটি ফাইলের inode-এ (দেখো অধ্যায় ১১) মালিকের নাম নয়, UID আর GID নম্বর লেখা থাকে। প্রতিটি process-এর ভেতরেও kernel UID/GID নম্বর রাখে।

তাহলে `ls -l` চালালে নাম দেখি কেন? কারণ `ls` নম্বরটা নিয়ে `/etc/passwd` (বা অন্য ডাটাবেস) খুঁজে নামে রূপান্তর করে দেখায়। নাম মানুষের সুবিধার জন্য, নম্বর মেশিনের জন্য।

প্রমাণ দেখতে `ls -n` চালাও, এটা নাম না দেখিয়ে নম্বর দেখায়:

```
$ ls -l report.txt
-rw-r--r-- 1 rahim rahim 120 Nov 12 10:00 report.txt
$ ls -n report.txt
-rw-r--r-- 1 1000 1000 120 Nov 12 10:00 report.txt
```

> 💡 **টিপস:** কোনো ইউজার মুছে ফেললে তার ফাইলগুলো `ls -l`-এ নামের জায়গায় শুধু নম্বর দেখায় (যেমন `1003`)। কারণ নম্বরটা আছে, কিন্তু সেই নম্বরের নাম আর ডাটাবেসে নেই। এমন ফাইলকে বলে **orphan file**। খুঁজতে: `sudo find / -nouser -o -nogroup` (অধ্যায় ১২)।

UID-এর রেঞ্জ

রেঞ্জ	কার জন্য	মন্তব্য
`0`	`root`	সুপারইউজার। নাম নয়, **UID 0** হওয়াটাই ক্ষমতার উৎস
`1–99`	distro-র স্থির (static) system অ্যাকাউন্ট	Debian-এ প্যাকেজ base-passwd এগুলো ঠিক করে
`100–999` (Debian/Ubuntu) / `201–999` (Fedora/RHEL)	ডাইনামিক system user	`useradd -r` এখান থেকে নম্বর দেয়
`1000–60000`	regular (মানুষ) ইউজার	প্রথম ইউজার সাধারণত `1000` পায়
`61184–65519`	systemd DynamicUser	service চালু থাকাকালীন অস্থায়ী ইউজার
`65534`	`nobody` / `nogroup` (Fedora/RHEL-এ গ্রুপের নাম `nobody`)	"কেউ না" — সবচেয়ে কম ক্ষমতার পরিচয়

এই সীমাগুলো আসে `/etc/login.defs` ফাইল থেকে:

```
$ grep -E '^(UID|GID|SYS_UID|SYS_GID)_(MIN|MAX)' /etc/login.defs
UID_MIN                  1000
UID_MAX                 60000
GID_MIN                  1000
GID_MAX                 60000
```

(Debian/Ubuntu-তে `SYS_UID_MIN` / `SYS_UID_MAX` লাইনগুলো সাধারণত comment করা থাকে, ডিফল্ট মান ব্যবহার হয়।)

> ⚠️ **সতর্কতা:** UID 0 মানেই root, নাম যাই হোক। কেউ যদি `/etc/passwd`-এ `hacker:x:0:0:...` লাইন ঢুকিয়ে দেয়, সেই `hacker` পূর্ণ root ক্ষমতা পাবে। তাই `awk -F: '$3==0' /etc/passwd` দিয়ে মাঝেমধ্যে চেক করো — শুধু `root` আসা উচিত।

Primary group আর Supplementary group

প্রতিটি ইউজারের ঠিক **একটি primary group** (প্রধান গ্রুপ) থাকে, যার GID `/etc/passwd`-এ লেখা থাকে। নতুন ফাইল বানালে ফাইলের গ্রুপ হয় এই primary group।

এছাড়া একজন ইউজার অনেকগুলো **supplementary group** (অতিরিক্ত গ্রুপ)-এর সদস্য হতে পারে, যেগুলো `/etc/group`-এ লেখা থাকে। যেমন `sudo` গ্রুপে থাকলে sudo চালানো যায়, `docker` গ্রুপে থাকলে docker চালানো যায়।

আধুনিক distro-গুলো **User Private Group (UPG)** পদ্ধতি মানে: প্রতিটি নতুন ইউজারের জন্য তার নিজের নামে একটা গ্রুপ তৈরি হয় (rahim ইউজারের primary group `rahim`)। এতে প্রত্যেকের ফাইল ডিফল্টভাবে নিজের গ্রুপে থাকে, অন্যদের সাথে অনিচ্ছায় শেয়ার হয় না।

১৫.৩ system user বনাম regular user

বৈশিষ্ট্য	system user	regular user
উদাহরণ	`www-data`, `sshd`, `mysql`, `systemd-resolve`	`rahim`, `sumi`
UID	১–৯৯৯	১০০০+
কে বানায়	প্যাকেজ ইনস্টলের সময় (`useradd -r`), বা `systemd-sysusers`	অ্যাডমিন (`useradd` / `adduser`) বা ইনস্টলার
home directory	প্রায়ই নেই, বা `/var/lib/...`, `/nonexistent`	`/home/<নাম>`
shell	`/usr/sbin/nologin` বা `/bin/false`	`/bin/bash`, `/bin/zsh` ইত্যাদি
পাসওয়ার্ড	নেই (shadow-এ `*` বা `!`)	থাকে
password aging	প্রযোজ্য নয়	প্রযোজ্য
উদ্দেশ্য	service-কে আলাদা ও সীমিত পরিচয় দেওয়া	মানুষের লগইন ও কাজ

> 📝 **মনে রাখো:** system user ও regular user-এর মধ্যে kernel-এর কাছে কোনো পার্থক্য নেই — দুটোই শুধু একটা নম্বর। পার্থক্যটা নিয়ম (convention) আর সেটিংসের: UID রেঞ্জ, shell, home। লগইন স্ক্রিন সাধারণত UID < 1000 ইউজারদের তালিকায় দেখায় না।

১৫.৪ ইউজার ডাটাবেস — চারটি গুরুত্বপূর্ণ ফাইল

ইউজার ও গ্রুপের তথ্য সাধারণ টেক্সট ফাইলে থাকে। এটাই Unix দর্শন: "সব কিছু টেক্সট, যাতে মানুষ পড়তে পারে আর টুল দিয়ে প্রসেস করা যায়।"

ফাইল	কী থাকে	পারমিশন (সাধারণত)	কে পড়তে পারে
`/etc/passwd`	ইউজারের মৌলিক তথ্য	`-rw-r--r--` root:root	সবাই

ফাইল	কী থাকে	পারমিশন (সাধারণত)	কে পড়তে পারে
`/etc/shadow`	পাসওয়ার্ড hash ও মেয়াদ	`-rw-r-----` root:shadow (Debian) / `----------` root:root (RHEL)	শুধু root (ও shadow গ্রুপ)
`/etc/group`	গ্রুপ ও সদস্য	`-rw-r--r--`	সবাই
`/etc/gshadow`	গ্রুপ পাসওয়ার্ড ও অ্যাডমিন	`/etc/shadow`-এর মতো	শুধু root

কেন দুই ভাগ? `/etc/passwd` সবাইকে পড়তে দিতেই হয়, কারণ `ls` বা `ps`-এর মতো সাধারণ প্রোগ্রাম UID থেকে নাম বের করে। আগেকার Unix-এ পাসওয়ার্ডের hash-ও এখানে থাকত — ফলে যে কেউ hash কপি করে অফলাইনে crack করার চেষ্টা করতে পারত। তাই hash সরিয়ে নেওয়া হলো শুধু-root-পড়তে-পারে এমন `/etc/shadow`-এ। একে বলে **shadow password** পদ্ধতি।

```
   /etc/passwd  (সবাই পড়ে)          /etc/shadow (শুধু root)
  ┌──────────────────────────┐      ┌──────────────────────────────┐
  │ rahim:x:1000:1000:...    │─────▶│ rahim:$y$j9T$...:20769:...   │
  └──────────────────────────┘  "x" └──────────────────────────────┘
                                মানে "পাসওয়ার্ড shadow-এ আছে"
```

`/etc/passwd` — ফিল্ড-বাই-ফিল্ড

```
$ grep rahim /etc/passwd
rahim:x:1000:1000:Rahim Uddin,,,:/home/rahim:/bin/bash
```

৭টি ফিল্ড, `:` (colon) দিয়ে আলাদা:

```
rahim : x : 1000 : 1000 : Rahim Uddin,,, : /home/rahim : /bin/bash
  |     |    |      |          |                |            |
  1     2    3      4          5                6            7
```

#	ফিল্ড	উদাহরণ	ব্যাখ্যা
১	username	`rahim`	লগইন নাম। ছোট হাতের অক্ষর, সংখ্যা, `-`, `_`; সংখ্যা দিয়ে শুরু না করাই ভালো; সর্বোচ্চ ৩২ অক্ষর
২	password	`x`	`x` মানে আসল hash `/etc/shadow`-এ। খালি থাকলে পাসওয়ার্ড ছাড়াই লগইন — বিপজ্জনক!
৩	UID	`1000`	ইউজার নম্বর
৪	GID	`1000`	**primary group**-এর নম্বর
৫	GECOS	`Rahim Uddin,,,`	মন্তব্য: পুরো নাম, রুম নম্বর, অফিস ফোন, বাসার ফোন — কমা দিয়ে আলাদা। নামটা ঐতিহাসিক (GE-র একটা OS থেকে)
৬	home	`/home/rahim`	লগইনের পর যেখানে শুরু হবে, `~` এটাকেই বোঝায়

#	ফিল্ড	উদাহরণ	ব্যাখ্যা
৭	shell	`/bin/bash`	লগইনের পর যে প্রোগ্রাম চলবে। `nologin` হলে লগইন বন্ধ

কয়েকটা system user দেখো:

```
$ head -5 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

লক্ষ্য করো `root`-এর home `/home/root` নয়, `/root`। কারণ `/home` আলাদা পার্টিশনে থাকতে পারে, আর সেটা mount না হলেও root যেন লগইন করে ঠিক করতে পারে।

`/etc/shadow` — ফিল্ড-বাই-ফিল্ড

```
$ sudo grep rahim /etc/shadow
rahim:$y$j9T$Vh3k...দীর্ঘ_hash...$Qm1:20769:0:99999:7:::
```

৯টি ফিল্ড:

```
rahim:$y$j9T$...:20769:0:99999:7:::
  1      2         3   4   5   6 7 8 9
```

#	ফিল্ড	উদাহরণ	ব্যাখ্যা
১	username	`rahim`	`/etc/passwd`-এর সাথে মেলে
২	password hash	`$y$j9T$...`	এনক্রিপ্ট নয়, **hash** (একমুখী)। বিশেষ মানগুলো নিচে
৩	last change	`20769`	শেষ পাসওয়ার্ড বদলের তারিখ, **১ জানুয়ারি ১৯৭০ থেকে কত দিন**। `0` মানে পরের লগইনে বদলাতেই হবে
৪	minimum	`0`	বদলানোর পর কমপক্ষে কত দিন আবার বদলানো যাবে না
৫	maximum	`99999`	কত দিন পর পাসওয়ার্ড মেয়াদোত্তীর্ণ হবে (`99999` ≈ কখনো না)
৬	warning	`7`	মেয়াদ শেষের কত দিন আগে সতর্ক করবে
৭	inactive	(খালি)	মেয়াদ শেষের পর কত দিন grace period, তারপর অ্যাকাউন্ট বন্ধ
৮	expire	(খালি)	পুরো **অ্যাকাউন্টের** মেয়াদ শেষের তারিখ (১৯৭০ থেকে দিন)
৯	reserved	(খালি)	ভবিষ্যতের জন্য সংরক্ষিত

Hash-এর শুরুর অংশ বলে দেয় কোন অ্যালগরিদম:

prefix	অ্যালগরিদম	কোথায় ডিফল্ট
`$y$`	yescrypt	Ubuntu 22.04+, Debian 11+, Fedora 35+
`$6$`	SHA-512 crypt	RHEL 8/9, পুরোনো distro
`$5$`	SHA-256 crypt	কম ব্যবহৃত
`$1$`	MD5 crypt	পুরোনো, অনিরাপদ

বিশেষ মান:

hash ফিল্ডে	মানে
`*`	কখনো পাসওয়ার্ড সেট হয়নি, পাসওয়ার্ডে লগইন অসম্ভব (system user)
`!` বা `!!`	লক / পাসওয়ার্ড সেট নেই (`!!` সাধারণত Fedora/RHEL-এ নতুন ইউজারের)
`!$y$...`	hash-এর আগে `!` = অ্যাকাউন্ট **লক** করা (`passwd -l`), hash অক্ষত আছে
খালি	পাসওয়ার্ড ছাড়াই লগইন — কখনো রেখো না

তারিখ সংখ্যাটা মানুষের ভাষায় দেখতে:

```
$ date -d "1970-01-01 +20769 days" +%F
2026-11-12
```

hash কীভাবে কাজ করে? পাসওয়ার্ড কখনো সংরক্ষিত হয় না। তুমি লগইনের সময় যা টাইপ করো, সিস্টেম সেটাকে একই **salt** (প্রতি ইউজারের র‍্যান্ডম অংশ, `$y$j9T$` এর পরের অংশ) সহ hash করে, তারপর সংরক্ষিত hash-এর সাথে মেলায়। মিললে ঢুকতে দেয়। salt থাকায় দুজনের একই পাসওয়ার্ড হলেও hash আলাদা হয়।

`/etc/group` — ফিল্ড-বাই-ফিল্ড

```
$ grep -E '^(sudo|developers):' /etc/group
sudo:x:27:rahim
developers:x:1050:rahim,sumi,karim
```

৪টি ফিল্ড:

#	ফিল্ড	উদাহরণ	ব্যাখ্যা
১	group name	`developers`	গ্রুপের নাম
২	password	`x`	`x` = gshadow-এ আছে (প্রায় কখনো ব্যবহৃত হয় না)
৩	GID	`1050`	গ্রুপ নম্বর
৪	members	`rahim,sumi,karim`	**supplementary** সদস্যদের তালিকা, কমা দিয়ে, স্পেস ছাড়া

> ⚠️ **সতর্কতা:** যে গ্রুপ কারো **primary** group, সেই ইউজারের নাম সাধারণত চতুর্থ ফিল্ডে লেখা থাকে না। যেমন `rahim:x:1000:` — শেষ ফিল্ড খালি, তবুও rahim এই গ্রুপের সদস্য (কারণ `/etc/passwd`-এ

> তার GID 1000)। তাই কারা সদস্য জানতে শুধু `/etc/group` দেখলে ভুল হবে।

`/etc/gshadow` — ফিল্ড-বাই-ফিল্ড

```
$ sudo grep developers /etc/gshadow
developers:!:rahim:rahim,sumi,karim
```

#	ফিল্ড	ব্যাখ্যা
১	group name	গ্রুপের নাম
২	encrypted password	গ্রুপ পাসওয়ার্ড (`newgrp`-এর জন্য); `!` বা `*` = নেই
৩	administrators	গ্রুপ অ্যাডমিন — এরা `gpasswd` দিয়ে সদস্য যোগ/বাদ দিতে পারে, root না হয়েও
৪	members	সদস্য তালিকা (`/etc/group`-এর সাথে মিল রাখতে হয়)

নিরাপদে এই ফাইলগুলো এডিট করা

⚠️ কখনো `nano /etc/passwd` দিয়ে সরাসরি এডিট করো না। একটা colon ভুল হলে কেউ লগইন করতে পারবে না, এমনকি root-ও না। পরিবর্তে:

কমান্ড	কাজ
`sudo vipw`	`/etc/passwd` লক করে এডিট করে, সেভের পর syntax চেক করে
`sudo vipw -s`	`/etc/shadow` এডিট
`sudo vigr`	`/etc/group` এডিট
`sudo vigr -s`	`/etc/gshadow` এডিট
`sudo pwck`	passwd/shadow-এর অসঙ্গতি খোঁজে (`-r` = শুধু পড়ে, ঠিক করে না)
`sudo grpck`	group/gshadow-এর অসঙ্গতি খোঁজে

তবে ৯৯% ক্ষেত্রে নিচের `useradd` / `usermod`-জাতীয় কমান্ডই ব্যবহার করা উচিত।

`getent` — সঠিক উপায়ে খোঁজা

বড় প্রতিষ্ঠানে ইউজার শুধু `/etc/passwd`-এ থাকে না; LDAP, Active Directory (SSSD দিয়ে) বা systemd-homed থেকেও আসতে পারে। `grep /etc/passwd` তখন তাদের খুঁজে পাবে না। `getent` সব উৎস একসাথে খোঁজে:

```
$ getent passwd rahim
rahim:x:1000:1000:Rahim Uddin,,,:/home/rahim:/bin/bash
$ getent group sudo
sudo:x:27:rahim
$ getent passwd 0
root:x:0:0:root:/root:/bin/bash
```

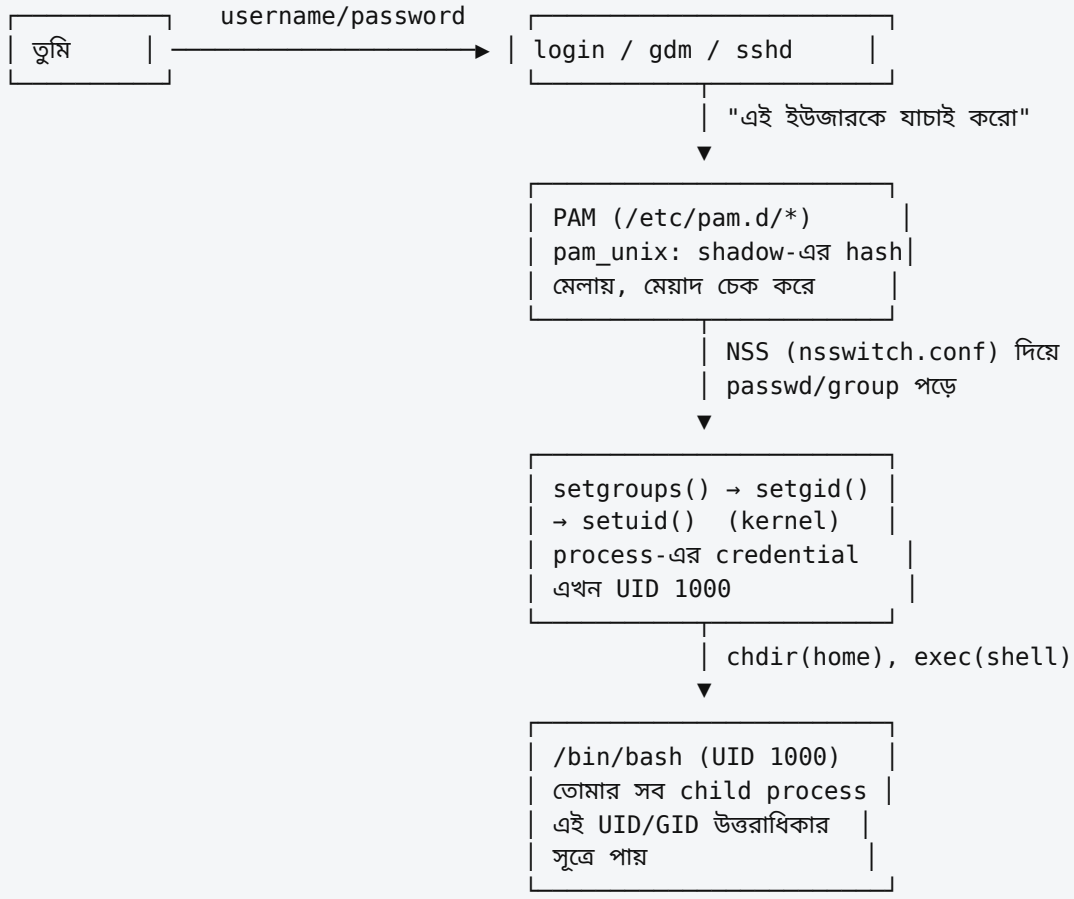
কোন উৎস কোন ক্রমে খোঁজা হবে তা ঠিক করে `/etc/nsswitch.conf`:

```
$ grep -E '^(passwd|group|shadow)' /etc/nsswitch.conf
passwd:         files systemd
group:          files systemd
shadow:         files systemd
```

(`files` = `/etc/passwd` ইত্যাদি; Fedora/RHEL-এ `sss` ও দেখা যায়।)

১৫.৫ ভেতরে কী ঘটে — login থেকে process পর্যন্ত

তুমি যখন লগইন করো, পেছনে এই ঘটনাগুলো ঘটে:



- **PAM (Pluggable Authentication Modules)** — প্রমাণীকরণের প্লাগইন-ব্যবস্থা। পাসওয়ার্ড, ফিঙ্গারপ্রিন্ট, 2FA — সব PAM মডিউল দিয়ে যুক্ত হয়।
- **NSS (Name Service Switch)** — নাম ↔ নম্বর রূপান্তরের ব্যবস্থা।
- **credential** — kernel প্রতিটি process-এর সাথে UID, GID, supplementary group-এর তালিকা রাখে। ফাইল খোলার সময় kernel এই নম্বরগুলো ফাইলের মালিকের নম্বরের সাথে মিলিয়ে পারমিশন দেয় (অধ্যায় ১৬)।

এর একটা গুরুত্বপূর্ণ ফল: **গ্রুপের সদস্যপদ লগইনের সময় একবারই process-এ বসে।** তাই তোমাকে নতুন গ্রুপে যোগ করলে, পুরোনো শেলে সেটা কাজ করবে না — আবার লগইন (বা `newgrp`) করতে হবে। এটা নতুনদের সবচেয়ে সাধারণ বিভ্রান্তি।

process-এর credential সরাসরি দেখতে:

```
$ grep -E '^(Uid|Gid|Groups)' /proc/self/status
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
Groups: 4 24 27 1000
```

চারটি কলাম হলো real, effective, saved, filesystem UID — এদের পার্থক্য অধ্যায় ১৭-এ setuid প্রসঙ্গে আসবে।

১৫.৬ আমি কে? — `whoami`, `id`, `groups`

`whoami`

বর্তমান **effective** ইউজারের নাম দেখায়।

```
$ whoami
rahim
$ sudo whoami
root
```

`id`

UID, GID ও সব গ্রুপ একসাথে দেখায়। সবচেয়ে কাজের কমান্ড।

Syntax: `id [OPTION]... [USER]`

```
$ id
uid=1000(rahim) gid=1000(rahim)
groups=1000(rahim),4(adm),24(cdrom),27(sudo),46(plugdev)
$ id sumi
uid=1001(sumi) gid=1001(sumi) groups=1001(sumi),1050(developers)
```

অপশন	কাজ
`-u`	শুধু UID
`-g`	শুধু primary GID
`-G`	সব গ্রুপের GID
`-n`	নম্বরের বদলে নাম (`-u`, `-g`, `-G`-এর সাথে)

অপশন	কাজ
`-r`	effective-এর বদলে real ID

```
$ id -u
1000
$ id -un
rahim
$ id -Gn
rahim adm cdrom sudo plugdev
```

> 💡 **টিপস:** স্ক্রিপ্টে root কিনা চেক করতে: `if [ "$(id -u)" -ne 0 ]; then echo "root হয়ে চালাও"; exit 1; fi`

> 📝 **মনে রাখো:** `id` (ইউজার নাম ছাড়া) **বর্তমান process**-এর গ্রুপ দেখায়, আর `id rahim` **ডাটাবেস** থেকে দেখায়। নতুন গ্রুপে যোগ হওয়ার পর `id rahim`-এ গ্রুপটা দেখাবে কিন্তু শুধু `id`-তে দেখাবে না — যতক্ষণ না আবার লগইন করো।

`groups`

```
$ groups
rahim adm cdrom sudo plugdev
$ groups sumi
sumi : sumi developers
```

১৫.৭ ইউজার তৈরি — `useradd` ও `adduser`

দুটো কমান্ড আছে, আর distro-ভেদে তাদের আচরণ আলাদা — এটা নতুনদের অনেক বিভ্রান্ত করে।

	`useradd`	`adduser`
ধরন	low-level, shadow-utils-এর অংশ, সব distro-তে আছে	Debian/Ubuntu-তে একটা বন্ধুত্বপূর্ণ interactive Perl স্ক্রিপ্ট
প্রশ্ন করে?	না, সব অপশন দিয়ে বলতে হয়	হ্যাঁ, পাসওয়ার্ড, পুরো নাম জিজ্ঞেস করে
Fedora/RHEL-এ	একই	`adduser` শুধু `useradd`-এর symlink
স্ক্রিপ্টে	✅ ব্যবহার করো (portable)	❌ এড়িয়ে চলো

`adduser` (Debian/Ubuntu)

```
$ sudo adduser sumi
info: Adding user `sumi' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `sumi' (1001) ...
info: Adding new user `sumi' (1001) with group `sumi (1001)' ...
info: Creating home directory `/home/sumi' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for sumi
Enter the new value, or press ENTER for the default
    Full Name []: Sumaiya Akter
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
```

(পুরোনো ভার্সনে `info:` লেখা থাকে না, বাকিটা একই।) এক কমান্ডেই home, গ্রুপ, পাসওয়ার্ড — সব হয়ে গেল। ডেস্কটপ ইউজারের জন্য সবচেয়ে সহজ।

`adduser` দিয়ে কাউকে বিদ্যমান গ্রুপে যোগ করা যায়: `sudo adduser sumi developers`।

`useradd`

Syntax: `useradd [OPTIONS] LOGIN`

অপশন	লম্বা রূপ	কাজ
`-m`	`--create-home`	home directory বানাও (+ `/etc/skel` কপি)
`-M`	`--no-create-home`	home বানিও না
`-d DIR`	`--home-dir`	home-এর পাথ নির্দিষ্ট করো
`-s SHELL`	`--shell`	login shell
`-u UID`	`--uid`	নির্দিষ্ট UID
`-g GROUP`	`--gid`	primary group (নাম বা GID; গ্রুপটা আগে থেকে থাকতে হবে)
`-G G1,G2`	`--groups`	supplementary group-এর তালিকা
`-c "TEXT"`	`--comment`	GECOS / পুরো নাম
`-e YYYY-MM-DD`	`--expiredate`	অ্যাকাউন্টের মেয়াদ শেষের তারিখ
`-f DAYS`	`--inactive`	পাসওয়ার্ড মেয়াদ শেষের পর কত দিনে অ্যাকাউন্ট নিষ্ক্রিয়
`-r`	`--system`	system user (system UID রেঞ্জ, aging নেই, ডিফল্টে home নেই)
`-k DIR`	`--skel`	`/etc/skel`-এর বদলে অন্য skeleton (`-m`-এর সাথে)

অপশন	লম্বা রূপ	কাজ
`-N`	`--no-user-group`	নিজের নামে গ্রুপ বানিও না
`-U`	`--user-group`	নিজের নামে গ্রুপ বানাও
`-o`	`--non-unique`	ডুপ্লিকেট UID অনুমতি দাও (`-u`-এর সাথে; সাধারণত এড়াও)
`-D`	`--defaults`	ডিফল্ট মান দেখাও/বদলাও

> ⚠️ **সতর্কতা (distro পার্থক্য):** Debian/Ubuntu-তে `useradd` শুধু নাম দিলে **home বানায় না** এবং shell ডিফল্ট হতে পারে `/bin/sh`। Fedora/RHEL-এ `/etc/login.defs`-এ `CREATE_HOME yes` থাকায় home বানায় আর shell `/bin/bash`। তাই portable হতে সবসময় `-m -s /bin/bash` স্পষ্ট করে লেখো।

বাস্তব উদাহরণ — একজন ডেভেলপার যোগ করা:

```
$ sudo useradd -m -s /bin/bash -c "Karim Hasan" -G developers,docker karim
$ sudo passwd karim
New password:
Retype new password:
passwd: password updated successfully
$ id karim
uid=1002(karim) gid=1002(karim) groups=1002(karim),1050(developers),998(docker)
$ ls -la /home/karim
total 20
drwxr-x--- 2 karim karim 4096 Nov 12 10:30 .
drwxr-xr-x 5 root  root  4096 Nov 12 10:30 ..
-rw-r--r-- 1 karim karim  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 karim karim 3771 Mar 31  2024 .bashrc
-rw-r--r-- 1 karim karim  807 Mar 31  2024 .profile
```

(home-এর পারমিশন `HOME_MODE` / `UMASK` সেটিংসের উপর নির্ভর করে: Ubuntu 21.04+-এ `750`, অনেক distro-তে `700`।)

> 📝 **মনে রাখো:** `useradd` **পাসওয়ার্ড সেট করে না**। নতুন অ্যাকাউন্ট তখন লক অবস্থায় থাকে (`!` বা `!!`)। লগইন করাতে চাইলে পরে `passwd` চালাতে হবে।

একটা service-এর জন্য system user:

```
$ sudo useradd -r -s /usr/sbin/nologin -d /var/lib/myapp -M myapp
$ getent passwd myapp
myapp:x:998:998::/var/lib/myapp:/usr/sbin/nologin
```

অস্থায়ী (কন্ট্রাক্টর) অ্যাকাউন্ট — ২০২৬-১২-৩১ এ মেয়াদ শেষ:

```
$ sudo useradd -m -s /bin/bash -e 2026-12-31 tempdev
```

useradd-এর ডিফল্ট কোথা থেকে আসে

`useradd` তিন জায়গা থেকে ডিফল্ট নেয়:

```
  /etc/login.defs       → UID/GID রেঞ্জ, CREATE_HOME, UMASK/HOME_MODE,
                          PASS_MAX_DAYS, ENCRYPT_METHOD, USERGROUPS_ENAB
  /etc/default/useradd  → SHELL, HOME (base dir), SKEL, INACTIVE, EXPIRE
  /etc/skel/            → নতুন home-এ কপি হওয়া ফাইল
```

```
$ useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/sh
SKEL=/etc/skel
CREATE_MAIL_SPOOL=no
LOG_INIT=yes
```

ডিফল্ট shell bash করতে: `sudo useradd -D -s /bin/bash` (এটা `/etc/default/useradd` বদলায়)।

১৫.৮ `/etc/skel` — নতুন ইউজারের "welcome kit"

`skel` = skeleton (কঙ্কাল)। `useradd -m` বা `adduser` নতুন home বানানোর সময় `/etc/skel`-এর সব কিছু (লুকানো ফাইলসহ) কপি করে দেয়, আর মালিকানা নতুন ইউজারকে দেয়।

উপমা: হোটেলে নতুন অতিথি এলে রুমে আগে থেকেই তোয়ালে, সাবান, পানির বোতল রাখা থাকে। `/etc/skel` হলো সেই স্ট্যান্ডার্ড সেটআপ।

```
$ ls -la /etc/skel
total 20
drwxr-xr-x   2 root root 4096 Apr 23  2024 .
drwxr-xr-x 130 root root 4096 Nov 12 10:30 ..
-rw-r--r--   1 root root  220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root root 3771 Mar 31  2024 .bashrc
-rw-r--r--   1 root root  807 Mar 31  2024 .profile
```

কোম্পানির সব নতুন কর্মীর জন্য একটা নিয়মাবলি ফাইল আর alias রাখতে চাও:

```
$ sudo tee /etc/skel/README-company.txt > /dev/null <<'EOF'
স্বাগতম! প্রজেক্ট ফাইল রাখো ~/projects-এ।
EOF
```

```
$ sudo mkdir /etc/skel/projects
$ echo "alias ll='ls -alF'" | sudo tee -a /etc/skel/.bashrc
```

এখন থেকে নতুন প্রতিটি ইউজার এগুলো পাবে।

> ⚠️ **সতর্কতা:** `/etc/skel` বদলালে **পুরোনো** ইউজারদের home-এ কোনো প্রভাব পড়ে না — কপি হয় শুধু তৈরির মুহূর্তে। আর এখানে কখনো গোপন তথ্য (key, পাসওয়ার্ড) রেখো না, কারণ সবাই কপি পাবে।

`.bashrc`, `.profile` আসলে কী করে, তা বিস্তারিত দেখবে অধ্যায় ৪৪-এ।

১৫.৯ পাসওয়ার্ড — `passwd` ও `chpasswd`

`passwd`

নিজের বা (root হলে) অন্যের পাসওয়ার্ড বদলায়, আর অ্যাকাউন্ট লক/আনলক করে।

Syntax: `passwd [OPTIONS] [USER]`

```
$ passwd
Changing password for rahim.
Current password:
New password:
Retype new password:
passwd: password updated successfully
```

টাইপ করার সময় কিছুই দেখা যায় না (কোনো `*`-ও না) — এটা স্বাভাবিক, পাশ থেকে কেউ যেন দৈর্ঘ্যও বুঝতে না পারে।

অপশন	কাজ
(নাম ছাড়া)	নিজের পাসওয়ার্ড বদলাও (পুরোনোটা লাগবে)
`USER` (root হিসেবে)	অন্যের পাসওয়ার্ড সেট করো (পুরোনোটা লাগে না)
`-l`	অ্যাকাউন্ট লক (hash-এর আগে `!` বসায়)
`-u`	আনলক
`-d`	পাসওয়ার্ড মুছে দাও (খালি পাসওয়ার্ড — বিপজ্জনক)
`-e`	পাসওয়ার্ড এখনই মেয়াদোত্তীর্ণ — পরের লগইনে বদলাতে বাধ্য
`-S`	পাসওয়ার্ডের অবস্থা দেখাও
`-n/-x/-w/-i DAYS`	min/max/warn/inactive দিন (chage-এর মতো)
`--stdin`	stdin থেকে পাসওয়ার্ড পড়ো — **শুধু Fedora/RHEL**, Debian/Ubuntu-তে নেই

```
$ sudo passwd -S sumi
sumi P 2026-11-12 0 99999 7 -1
```

স্ট্যাটাস কোড: `P` = usable password, `L` = locked, `NP` = no password। পরের সংখ্যাগুলো: শেষ বদল, min, max, warn, inactive।

নতুন কর্মীকে প্রথম লগইনে পাসওয়ার্ড বদলাতে বাধ্য করো:

```
$ sudo passwd -e sumi
passwd: password changed.
```

পরের লগইনে sumi দেখবে: `You are required to change your password immediately (administrator enforced).`

> ⚠️ **সতর্কতা:** `passwd -l` শুধু **পাসওয়ার্ড** লগইন বন্ধ করে। SSH key দিয়ে ইউজার তখনও ঢুকতে পারে! পুরো অ্যাকাউন্ট বন্ধ করতে লক করার সাথে সাথে `sudo usermod -e 1 USER` (মেয়াদ ১৯৭০-এ শেষ) অথবা shell `nologin` করো।

`chpasswd` — অনেক পাসওয়ার্ড একসাথে

`user:password` ফর্ম্যাটে stdin থেকে পড়ে। সব distro-তে কাজ করে, স্ক্রিপ্টের জন্য আদর্শ।

```
$ echo 'sumi:N3w-Str0ng#Pass' | sudo chpasswd
$ sudo chpasswd < users.txt      # প্রতি লাইনে user:password
```

> 💡 **টিপস:** কমান্ড লাইনে সরাসরি পাসওয়ার্ড লিখলে সেটা shell history-তে থেকে যায় (অধ্যায় ৬)। bash-এ কমান্ডের আগে একটা স্পেস দিলে (যদি `HISTCONTROL`-এ `ignorespace` থাকে, Ubuntu-তে ডিফল্ট `ignoreboth`) সেটা history-তে যায় না।

১৫.১০ পাসওয়ার্ডের মেয়াদ — `chage`

`chage` = **ch**ange **age**। `/etc/shadow`-এর ফিল্ড ৩–৮ সহজে বদলায়।

Syntax: `chage [OPTIONS] USER`

অপশন	shadow ফিল্ড	কাজ
`-l`	—	বর্তমান সেটিংস দেখাও (সাধারণ ইউজারও নিজেরটা দেখতে পারে)
`-d DATE`	৩	শেষ বদলের তারিখ সেট করো; `-d 0` = পরের লগইনে বদলাতে বাধ্য

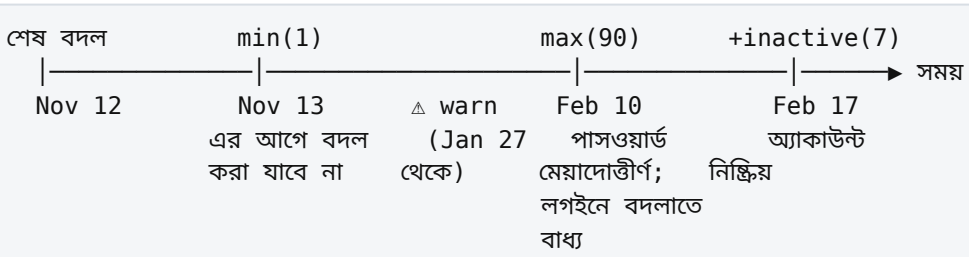
অপশন	shadow ফিল্ড	কাজ
`-m DAYS`	৪	বদলের মাঝে ন্যূনতম দিন
`-M DAYS`	৫	সর্বোচ্চ দিন (এরপর মেয়াদ শেষ); `-1` = সীমা তুলে দাও
`-W DAYS`	৬	কত দিন আগে সতর্ক করবে
`-I DAYS`	৭	মেয়াদ শেষের পর কত দিন grace, তারপর লক; `-1` = বন্ধ
`-E DATE`	৮	অ্যাকাউন্টের মেয়াদ (YYYY-MM-DD); `-1` = কখনো না

```
$ sudo chage -l sumi
Last password change                                    : Nov 12, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
```

কোম্পানির নীতি: প্রতি ৯০ দিনে পাসওয়ার্ড বদলাতে হবে, ১ দিনের আগে আবার বদলানো যাবে না, ১৪ দিন আগে সতর্কতা, মেয়াদ পেরোনোর ৭ দিন পর লক:

```
$ sudo chage -M 90 -m 1 -W 14 -I 7 sumi
$ sudo chage -l sumi
Last password change                                    : Nov 12, 2026
Password expires                                        : Feb 10, 2027
Password inactive                                       : Feb 17, 2027
Account expires                                         : never
Minimum number of days between password change          : 1
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 14
```

সময়রেখায় বোঝো:



নতুন সব ইউজারের জন্য ডিফল্ট বদলাতে `/etc/login.defs`-এ `PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE` বদলাও (শুধু ভবিষ্যতের ইউজারের উপর প্রভাব ফেলে)।

> 💡 **টিপস:** আধুনিক নিরাপত্তা নির্দেশিকা (যেমন NIST SP 800-63B) বলে, ঘন ঘন জোর করে পাসওয়ার্ড বদলানোর চেয়ে লম্বা, অনন্য পাসওয়ার্ড + 2FA বেশি কার্যকর। তবে অনেক প্রতিষ্ঠানের compliance নিয়মে aging লাগে, তাই `chage` জানা জরুরি।

১৫.১১ ইউজার পরিবর্তন — `usermod`

Syntax: `usermod [OPTIONS] LOGIN`

অপশন	কাজ
`-aG G1,G2`	supplementary group-এ **যোগ** করো (append) — সবচেয়ে বেশি ব্যবহৃত
`-G G1,G2`	supplementary group-এর তালিকা **প্রতিস্থাপন** করো (বাকিগুলো থেকে বাদ!)
`-g GROUP`	primary group বদলাও
`-l NEWNAME`	লগইন নাম বদলাও (home বা গ্রুপের নাম বদলায় না)
`-d DIR -m`	home পাথ বদলাও এবং পুরোনো ফাইল নতুন জায়গায় সরাও
`-s SHELL`	login shell বদলাও
`-c "TEXT"`	GECOS / পুরো নাম
`-u UID`	UID বদলাও (home-এর ফাইলের মালিকানাও আপডেট হয়)
`-L`	লক করো (`passwd -l` -এর মতো)
`-U`	আনলক
`-e DATE`	অ্যাকাউন্টের মেয়াদ; `-e ""` = মেয়াদ তুলে দাও
`-rG G`	নির্দিষ্ট গ্রুপ থেকে বাদ দাও (নতুন shadow-utils, ৪.১৩+; Ubuntu 24.04-এ আছে, পুরোনোতে `gpasswd -d` ব্যবহার করো)

সবচেয়ে বিখ্যাত ভুল: `-a` ভুলে যাওয়া

```
$ id rahim
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),4(adm),27(sudo)
$ sudo usermod -G docker rahim          # ❌ -a নেই!
$ id rahim
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),998(docker)
```

rahim এখন `sudo` গ্রুপ থেকে বাদ পড়ে গেছে! যদি এটাই একমাত্র অ্যাডমিন হয়, আর root লক থাকে (Ubuntu-তে ডিফল্ট), তাহলে তুমি নিজের সিস্টেম থেকে অ্যাডমিন ক্ষমতা হারালে। ঠিক করতে recovery mode লাগবে।

সঠিক উপায়:

```
$ sudo usermod -aG docker rahim         # ✅ append
$ id rahim
```

```
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),4(adm),27(sudo),998(docker)
```

> ⚠️ **সতর্কতা: `-G` কখনো `-a` ছাড়া ব্যবহার কোরো না**, যদি না ইচ্ছা করেই পুরো তালিকা বদলাতে চাও। মনে রাখার কৌশল: "**a**G = **a**dd to **G**roup"।

যোগ করার পর rahim-কে **লগআউট করে আবার লগইন** করতে হবে (ভেতরে কেন — ১৫.৫ দেখো)। তাৎক্ষণিক পরীক্ষা করতে `newgrp docker` (১৫.১৩)।

অন্যান্য উদাহরণ

```
# লগইন নাম বদল + home ও গ্রুপের নামও মিলিয়ে বদল
$ sudo usermod -l rahim2 rahim
$ sudo usermod -d /home/rahim2 -m rahim2
$ sudo groupmod -n rahim2 rahim

# shell zsh করা (zsh ইনস্টল থাকতে হবে)
$ sudo usermod -s /usr/bin/zsh sumi

# অ্যাকাউন্ট আজ থেকে বন্ধ (লক + মেয়াদ শেষ)
$ sudo usermod -L -e 1 karim
```

> ⚠️ **সতর্কতা:** যে ইউজার লগইন অবস্থায় আছে বা যার process চলছে, তার নাম/UID/home বদলানো যায় না: `usermod: user rahim is currently used by process 1234`। অন্য অ্যাডমিন অ্যাকাউন্ট থেকে করো।

১৫.১২ ইউজার মুছে ফেলা — `userdel`

Syntax: `userdel [OPTIONS] LOGIN`

অপশন	কাজ
(কিছু না)	শুধু অ্যাকাউন্ট মোছে; home ও ফাইল থেকে যায়
`-r`	home directory ও mail spool-ও মোছে
`-f`	লগইন থাকলেও জোর করে মোছে (বিপজ্জনক)

```
$ sudo userdel -r tempdev
userdel: tempdev mail spool (/var/mail/tempdev) not found
```

(mail spool না থাকলে এই বার্তা আসে — সমস্যা নয়।)

Debian/Ubuntu-তে বন্ধুত্বপূর্ণ বিকল্প `deluser`:

```
$ sudo deluser --remove-home tempdev
$ sudo deluser --backup --remove-home tempdev   # মোছার আগে home-এর ব্যাকআপ
$ sudo deluser sumi developers                   # শুধু গ্রুপ থেকে বাদ
```

> ⚠️ **সতর্কতা:** `-r` শুধু home আর mail spool মোছে। `/tmp`, `/var/www` বা অন্য কোথাও থাকা ঐ ইউজারের ফাইল থেকে যায় orphan হয়ে। পরে নতুন ইউজার একই UID পেলে সে ঐ ফাইলগুলোর মালিক হয়ে যাবে! মোছার আগে: `sudo find / -xdev -user tempdev 2>/dev/null`। আর প্রতিষ্ঠানে প্রায়ই মোছার বদলে প্রথমে **লক** করা ভালো — অডিটের জন্য।

১৫.১৩ গ্রুপ ম্যানেজমেন্ট

উপমা: গ্রুপ হলো অফিসের বিভাগ। "Accounts" বিভাগের সবাই একাউন্টস রুমে ঢুকতে পারে। নতুন কর্মী আসলে তাকে বিভাগে যোগ করো — প্রতিটি ফাইলের পারমিশন আলাদা করে দিতে হয় না।

`groupadd`

অপশন	কাজ
`-g GID`	নির্দিষ্ট GID
`-r`	system group (system GID রেঞ্জ)
`-f`	গ্রুপ আগে থেকে থাকলে error না দিয়ে সফলভাবে বেরিয়ে যাও

```
$ sudo groupadd developers
$ sudo groupadd -g 1050 designers
$ getent group developers
developers:x:1051:
```

`groupmod`

অপশন	কাজ
`-n NEWNAME`	নাম বদলাও
`-g GID`	GID বদলাও (⚠️ বিদ্যমান ফাইলের GID বদলায় না — সেগুলো orphan হবে)

```
$ sudo groupmod -n devteam developers
```

`groupdel`

```
$ sudo groupdel designers
$ sudo groupdel sumi
```

```
groupdel: cannot remove the primary group of user 'sumi'
```

কোনো ইউজারের **primary group** মোছা যায় না — আগে সেই ইউজারকে মুছতে হবে বা তার primary group বদলাতে হবে।

`gpasswd` — গ্রুপের সদস্য সামলানো

অপশন	কাজ
`-a USER GROUP`	সদস্য যোগ
`-d USER GROUP`	সদস্য বাদ
`-M u1,u2 GROUP`	পুরো সদস্য তালিকা সেট (প্রতিস্থাপন)
`-A u1,u2 GROUP`	গ্রুপ অ্যাডমিন সেট (gshadow-এর ৩ নম্বর ফিল্ড)
`-r GROUP`	গ্রুপ পাসওয়ার্ড মোছো
`GROUP` (শুধু)	গ্রুপ পাসওয়ার্ড সেট

```
$ sudo gpasswd -a sumi devteam
Adding user sumi to group devteam
$ sudo gpasswd -d sumi devteam
Removing user sumi from group devteam
$ sudo gpasswd -A rahim devteam      # rahim এখন sudo ছাড়াই সদস্য যোগ করতে পারবে
$ gpasswd -a karim devteam           # rahim হিসেবে, sudo ছাড়া
Adding user karim to group devteam
```

> 💡 **টিপস:** কাউকে গ্রুপ থেকে **বাদ** দেওয়ার সবচেয়ে নিরাপদ ও portable উপায় `gpasswd -d USER GROUP` — বাকি গ্রুপে হাত দেয় না।

`newgrp` ও `sg` — লগআউট ছাড়াই নতুন গ্রুপ

```
$ sudo usermod -aG docker rahim
$ id -Gn
rahim adm sudo                       # docker এখনো নেই
$ newgrp docker                      # নতুন subshell, primary group = docker
$ id -Gn
docker adm sudo rahim
$ exit                               # আগের শেলে ফিরে যাও
```

`newgrp` একটা নতুন shell চালু করে, যার primary group হয় নির্দিষ্ট গ্রুপটি। শুধু একটা কমান্ড চালাতে চাইলে `sg`:

```
$ sg docker -c "docker ps"
```

গুরুত্বপূর্ণ প্রচলিত গ্রুপ

গ্রুপ	Debian/Ubuntu	Fedora/RHEL	সদস্য হলে যা পারে
অ্যাডমিন	`sudo`	`wheel`	`sudo` দিয়ে root ক্ষমতা (অধ্যায় ১৮)
`adm`	✅	✅	`/var/log`-এর অনেক লগ পড়া
`systemd-journal`	✅	✅	সব journal লগ পড়া (অধ্যায় ২৮)
`docker`	প্যাকেজ বানায়	প্যাকেজ বানায়	docker চালানো (কার্যত root-এর সমান!)
`dialout`	✅	✅	serial port (`/dev/ttyUSB0`, Arduino)
`video`, `audio`, `render`	✅	✅	ক্যামেরা/GPU/সাউন্ড ডিভাইস
`plugdev`	✅	—	কিছু removable ডিভাইস
`lpadmin`	✅	—	প্রিন্টার পরিচালনা
`libvirt`	✅	✅	ভার্চুয়াল মেশিন পরিচালনা (অধ্যায় ৪৮)

> ⚠️ **সতর্কতা:** `docker` বা `libvirt` গ্রুপে কাউকে যোগ করা মানে কার্যত তাকে root দেওয়া, কারণ সে container-এ host-এর `/` mount করে যা খুশি করতে পারে। বিশ্বস্ত মানুষ ছাড়া দিও না।

১৫.১৪ কে লগইন আছে, কে এসেছিল — `who`, `w`, `users`, `last`, `lastlog`

`who`

এই মুহূর্তে লগইন করা ইউজারদের তালিকা (তথ্য আসে `/var/run/utmp` থেকে; নতুন কিছু distro-তে systemd-logind থেকে)।

```
$ who
rahim    seat0        2026-11-12 09:01 (login screen)
rahim    tty2         2026-11-12 09:01 (tty2)
karim    pts/1        2026-11-12 10:12 (192.168.1.25)
```

অপশন	কাজ
`-b`	শেষ boot-এর সময়
`-r`	বর্তমান runlevel
`-q`	শুধু নাম ও মোট সংখ্যা
`-H`	কলামের শিরোনাম দেখাও
`-a`	সব তথ্য

অপশন	কাজ
`am i` / `-m`	শুধু বর্তমান terminal-এর ইউজার

```
$ who -b
         system boot  2026-11-10 07:11
```

`tty` = সরাসরি কনসোল (Ctrl+Alt+F3 ইত্যাদি), `pts/N` = pseudo-terminal (GUI terminal window বা SSH)।

`w`

`who`-এর চেয়ে বেশি: প্রত্যেকে **কী করছে** তাও দেখায়।

```
$ w
 10:15:32 up 2 days,  3:04,  2 users,  load average: 0.15, 0.10, 0.05
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU  WHAT
rahim    tty2     -                09:01    1:14m  2:10   0.05s /usr/libexec/gnome-
session-binary
karim    pts/1    192.168.1.25     10:12    0.00s  0.04s  0.00s vim deploy.sh
```

কলাম	মানে
প্রথম লাইন	বর্তমান সময়, কতক্ষণ চালু (`uptime`-এর মতো), ইউজার সংখ্যা, load average (অধ্যায় ৪৬)
`FROM`	কোথা থেকে (SSH হলে IP)
`LOGIN@`	কখন লগইন করেছে
`IDLE`	কতক্ষণ কিছু টাইপ করেনি
`JCPU`	ঐ terminal-এর সব process মিলে CPU সময়
`PCPU`	বর্তমান process-এর CPU সময়
`WHAT`	এখন কী চালাচ্ছে

`w -h` = শিরোনাম ছাড়া, `w USER` = শুধু ঐ ইউজার। `users` কমান্ড শুধু নামগুলো এক লাইনে দেখায়।

`last`

অতীতের লগইন/লগআউট ও reboot-এর ইতিহাস (`/var/log/wtmp` থেকে)।

```
$ last -n 5
karim    pts/1        192.168.1.25     Thu Nov 12 10:12   still logged in
rahim    tty2         tty2             Thu Nov 12 09:01   still logged in
reboot   system boot  6.8.0-48-generic Tue Nov 10 07:11   still running
sumi     pts/0        10.0.0.7         Mon Nov  9 18:20 - 19:45  (01:25)
rahim    tty2         tty2             Mon Nov  9 08:55 - down   (10:12)

wtmp begins Sun Nov  1 00:00:02 2026
```

অপশন	কাজ
`-n N` / `-N`	শেষ N টি এন্ট্রি
`USER`	শুধু ঐ ইউজার
`reboot`	শুধু reboot-এর ইতিহাস
`-x`	shutdown ও runlevel পরিবর্তনও
`-F`	পূর্ণ তারিখ-সময়
`-i`	hostname-এর বদলে IP
`-s` / `-t`	নির্দিষ্ট সময়ের পর/আগে (যেমন `-s yesterday`)

ব্যর্থ লগইনের চেষ্টা দেখতে `sudo lastb` (`/var/log/btmp` থেকে) — SSH brute-force আক্রমণ ধরতে খুব কাজের:

```
$ sudo lastb -n 3
admin    ssh:notty    45.83.12.9       Thu Nov 12 03:14 - 03:14  (00:00)
root     ssh:notty    45.83.12.9       Thu Nov 12 03:14 - 03:14  (00:00)
test     ssh:notty    103.21.4.77      Thu Nov 12 02:58 - 02:58  (00:00)
```

`lastlog`

প্রতিটি ইউজারের **সর্বশেষ** লগইন একনজরে:

```
$ lastlog
Username         Port     From             Latest
root                                       **Never logged in**
daemon                                     **Never logged in**
rahim            tty2                      Thu Nov 12 09:01:44 +0600 2026
karim            pts/1    192.168.1.25     Thu Nov 12 10:12:03 +0600 2026
```

`lastlog -u karim` = শুধু karim, `lastlog -b 90` = যারা ৯০ দিনের বেশি লগইন করেনি (অব্যবহৃত অ্যাকাউন্ট খুঁজতে দারুণ)।

> 📝 **মনে রাখো (distro পার্থক্য):** পুরোনো utmp/wtmp ফরম্যাট ২০৩৮ সালের সময়-সীমা (Y2038) সমস্যায় ভোগে। তাই কিছু নতুন distro (যেমন Debian 13, Ubuntu 25.04+, openSUSE) `wtmpdb` ও `lastlog2` ব্যবহার শুরু করেছে; সেখানে `last` কমান্ড `wtmpdb last` দিয়ে চলে আর `lastlog`-এর জায়গায় `lastlog2` থাকতে পারে। RHEL 9-এ `lastlog` আছে, ভবিষ্যতে বাদ পড়ার কথা। systemd-ভিত্তিক বিকল্প: `loginctl list-sessions` আর `journalctl -u systemd-logind`।

১৫.১৫ ইউজার বদলানো — `su`

`su` = **s**ubstitute **u**ser (বা switch user)। অন্য ইউজার হিসেবে নতুন shell খোলে — **লক্ষ্য ইউজারের পাসওয়ার্ড** লাগে (root থেকে চালালে লাগে না)।

Syntax: `su [OPTIONS] [-] [USER [ARG...]]`

অপশন	কাজ
`su`	root হও, কিন্তু বর্তমান environment রাখো (non-login shell)
`su -` / `su -l` / `su --login`	root হও, **login shell** হিসেবে — পরিষ্কার environment
`su - USER`	ঐ ইউজারের login shell
`-c "CMD"`	শুধু একটা কমান্ড চালিয়ে ফিরে এসো
`-s SHELL`	নির্দিষ্ট shell ব্যবহার করো
`-p` / `-m`	environment সংরক্ষণ করো

```
$ su - sumi
Password:
sumi@laptop:~$ pwd
/home/sumi
sumi@laptop:~$ exit
logout
$ su -c "id" sumi
Password:
uid=1001(sumi) gid=1001(sumi) groups=1001(sumi),1051(devteam)
```

`su` বনাম `su -` — পার্থক্যটা গুরুত্বপূর্ণ

```
su root  (dash ছাড়া)                su - root  (dash সহ)
┌──────────────────────────┐         ┌──────────────────────────┐
│ UID = 0                  │         │ UID = 0                  │
│ PWD = যেখানে ছিলে       │         │ PWD = /root              │
│ PATH = rahim-এর PATH     │         │ PATH = root-এর PATH      │
│ HOME = (distro-ভেদে)     │         │ HOME = /root             │
│ .profile পড়ে না         │         │ /etc/profile, ~/.profile │
│                          │         │ পড়ে (login shell)       │
└──────────────────────────┘         └──────────────────────────┘
     ↑ মিশ্র environment                  ↑ সত্যিকারের root সেশন
```

dash ছাড়া `su` করলে পুরোনো ইউজারের `PATH` থেকে যেতে পারে — ফলে `/usr/sbin`-এর কমান্ড না-ও পাওয়া যেতে পারে, বা আরও খারাপ, কেউ নিজের `PATH`-এ ফাঁদ পেতে রাখলে root সেই ফাঁদের প্রোগ্রাম চালাতে পারে। **সবসময় `su -` ব্যবহার করো।**

> ⚠️ **সতর্কতা (Ubuntu):** Ubuntu-তে ডিফল্টভাবে root-এর কোনো পাসওয়ার্ড সেট নেই (লক), তাই `su -` কাজ করে না: `su: Authentication failure`। Ubuntu-তে `sudo -i` ব্যবহার করো। Fedora

> ইনস্টলারে root পাসওয়ার্ড সেট করার অপশন থাকে, RHEL-এও। `sudo` বনাম `su` এর পূর্ণ তুলনা অধ্যায় ১৮-এ।

> 💡 **টিপস:** `nologin` shell-ওয়ালা service user হিসেবে কমান্ড চালাতে: `sudo -u www-data whoami` বা `sudo su -s /bin/bash www-data`। শুধু `su www-data` চালালে nologin বার্তা দিয়ে বেরিয়ে যাবে।

১৫.১৬ login shell, non-login shell ও `/etc/shells`

login shell কী?

যে shell তুমি **লগইন করার সাথে সাথে** পাও — tty-তে লগইন, SSH, `su -`, `bash -l` — সেটাই **login shell**। আর GUI-তে terminal window খুললে বা লগইন অবস্থায় `bash` টাইপ করলে যেটা পাও, সেটা সাধারণত **non-login interactive shell**।

পার্থক্য হলো কোন কনফিগ ফাইল পড়া হয়:

```
login shell (bash)                    non-login interactive (bash)
──────────────────                    ────────────────────────────
/etc/profile                           /etc/bash.bashrc (Debian) বা
  └─ /etc/profile.d/*.sh                 /etc/bashrc (Fedora, ~/.bashrc থেকে)
তারপর প্রথম যেটা পায়:               ~/.bashrc
  ~/.bash_profile
  ~/.bash_login
  ~/.profile   (Debian/Ubuntu-তে এটা
                নিজেই ~/.bashrc ডাকে)
```

চেনার উপায়:

```
$ echo $0
-bash            # শুরুতে '-' মানে login shell
$ shopt -q login_shell && echo login || echo non-login
login
```

এই ফাইলগুলোর বিস্তারিত ও কাস্টমাইজেশন অধ্যায় ৪৪-এ।

`/etc/shells` ও `chsh`

`/etc/shells` হলো বৈধ login shell-এর তালিকা। `chsh` দিয়ে সাধারণ ইউজার শুধু এই তালিকার shell বেছে নিতে পারে।

```
$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
$ chsh -s /usr/bin/zsh          # নিজের shell zsh করো
Password:
$ getent passwd rahim | cut -d: -f7
/usr/bin/zsh
```

(`zsh` ইনস্টল করলে সেটা `/etc/shells`-এ যোগ হয়।) পরিবর্তন কার্যকর হবে পরের লগইনে। Fedora/RHEL-এ `chsh` আলাদা প্যাকেজ (`util-linux-user`) থেকে আসে; না থাকলে `sudo usermod -s` ব্যবহার করো।

GECOS (পুরো নাম ইত্যাদি) বদলাতে `chfn`: `chfn -f "Rahim Uddin"`।

১৫.১৭ `nologin` — লগইন বন্ধ কিন্তু অ্যাকাউন্ট সক্রিয়

service user-দের অ্যাকাউন্ট দরকার (ফাইলের মালিক হতে, process চালাতে), কিন্তু কেউ যেন ঐ অ্যাকাউন্ট দিয়ে লগইন করে shell না পায়। সমাধান: shell হিসেবে `/usr/sbin/nologin` দাও।

```
$ grep www-data /etc/passwd
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
$ sudo su - www-data
This account is currently not available.
```

`nologin` একটা ছোট প্রোগ্রাম যেটা ঐ বার্তা দেখিয়ে non-zero exit code দিয়ে বেরিয়ে যায়। বার্তা কাস্টমাইজ করতে `/etc/nologin.txt` ফাইল বানাও।

	`/usr/sbin/nologin`	`/bin/false`	`/etc/nologin` **ফাইল**
কী	shell হিসেবে সেট করা প্রোগ্রাম	কিছু না করে exit 1	একটা ফাইল, থাকলেই প্রভাব
প্রভাব	শুধু ঐ ইউজার	শুধু ঐ ইউজার	**root ছাড়া সবার** লগইন বন্ধ
বার্তা	ভদ্র বার্তা দেখায় ও লগ করে	চুপচাপ বেরিয়ে যায়	ফাইলের লেখা দেখায়
ব্যবহার	service account (পছন্দনীয়)	পুরোনো রীতি	মেইনটেন্যান্সের সময়

```
$ echo "সার্ভার মেইনটেন্যান্সে আছে, রাত ১২টায় ফিরব।" | sudo tee /etc/nologin
$ sudo rm /etc/nologin       # কাজ শেষে অবশ্যই মুছো!
```

(systemd সিস্টেমে বুটের সময় `/run/nologin` ফাইলও তৈরি হয় যাতে বুট পুরো না হওয়া পর্যন্ত লগইন না হয়।)

> ⚠️ **সতর্কতা:** `nologin` shell থাকলেও SSH-এর কিছু ফিচার (যেমন port forwarding `ssh -N`) কাজ করতে পারে, কারণ সেগুলো shell চালায় না। শক্তভাবে আটকাতে `sshd_config`-এ `AllowUsers` / `DenyUsers` ব্যবহার করো (অধ্যায় ৩৮)।

১৫.১৮ সংশ্লিষ্ট কীবোর্ড শর্টকাট

শর্টকাট / কমান্ড	কাজ
`Ctrl + D`	বর্তমান shell থেকে বেরোও (logout / `su` থেকে ফেরা) — খালি লাইনে
`exit` / `logout`	shell থেকে বেরোও (`logout` শুধু login shell-এ কাজ করে)
`Ctrl + Alt + F3` … `F6`	টেক্সট কনসোল (tty)-এ যাও, আলাদা ইউজারে লগইন করো
`Ctrl + Alt + F1` বা `F2`	GUI-তে ফিরে যাও (distro-ভেদে; Ubuntu/Fedora-তে সাধারণত F1 বা F2)
`Super + L`	স্ক্রিন লক (GNOME)
GNOME: Power মেনু → Switch User	লগআউট না করে অন্য ইউজার হিসেবে ঢোকো
পাসওয়ার্ড প্রম্পটে `Ctrl + U`	ভুল টাইপ করা পুরো পাসওয়ার্ড মুছে আবার লেখো
পাসওয়ার্ড প্রম্পটে `Ctrl + C`	বাতিল করো
`!!` (যেমন `sudo !!`)	আগের কমান্ড sudo দিয়ে আবার চালাও (permission denied পেলে)

হাতে-কলমে অনুশীলন

> ⚠️ এগুলো একটা VM বা টেস্ট মেশিনে করো (অধ্যায় ৩)। কাজ শেষে ল্যাব ৭-এ সব পরিষ্কার করা আছে।

ল্যাব ১: নিজেকে চেনো

```
$ whoami
$ id
$ groups
$ getent passwd "$USER"
$ grep -E '^(Uid|Gid|Groups)' /proc/self/status
```

প্রশ্ন: তোমার UID কত? primary group কোনটি? তুমি কি `sudo` বা `wheel` গ্রুপে আছো?

ল্যাব ২: সিস্টেমের ইউজার গোনো

```
$ awk -F: '$3 >= 1000 && $3 < 60000 {print $1, $3, $7}' /etc/passwd   # মানুষ ইউজার
$ awk -F: '$3 < 1000 {print $1}' /etc/passwd | wc -l                   # system user
```

```
সংখ্যা
$ awk -F: '$3 == 0 {print $1}' /etc/passwd                              # UID 0 কারা (শুধু
root থাকা উচিত)
$ grep -c nologin /etc/passwd
```

ল্যাব ৩: একটা টিম সেটআপ

```
$ sudo groupadd labteam
$ sudo useradd -m -s /bin/bash -c "Lab User One" -G labteam labuser1
$ sudo useradd -m -s /bin/bash -c "Lab User Two" labuser2
$ echo 'labuser1:Lab#Pass2026' | sudo chpasswd
$ echo 'labuser2:Lab#Pass2026' | sudo chpasswd
$ sudo usermod -aG labteam labuser2
$ getent group labteam
labteam:x:1052:labuser1,labuser2
$ id labuser2
```

ল্যাব ৪: `-a` ভুলের প্রভাব নিজের চোখে দেখো

```
$ sudo groupadd labextra
$ sudo usermod -G labextra labuser2      # -a নেই
$ id labuser2                            # labteam উধাও!
$ sudo usermod -aG labteam labuser2      # ঠিক করো
$ id labuser2
```

ল্যাব ৫: পাসওয়ার্ড নীতি ও লক

```
$ sudo chage -M 60 -W 10 -I 5 labuser1
$ sudo chage -l labuser1
$ sudo passwd -e labuser1                # প্রথম লগইনে বদলাতে বাধ্য
$ su - labuser1                          # পাসওয়ার্ড বদলাতে বলবে, বদলে exit করো
$ sudo passwd -l labuser2
$ sudo passwd -S labuser2                # L দেখাবে
$ sudo grep labuser2 /etc/shadow         # hash-এর আগে '!' দেখো
$ sudo passwd -u labuser2
```

ল্যাব ৬: skel ও nologin

```
$ echo "Welcome to the lab!" | sudo tee /etc/skel/WELCOME.txt
$ sudo useradd -m -s /bin/bash labuser3
$ sudo ls -la /home/labuser3             # WELCOME.txt আছে?
$ sudo usermod -s /usr/sbin/nologin labuser3
$ sudo su - labuser3                     # "This account is currently not available."
$ sudo -u labuser3 whoami                # তবুও এভাবে কমান্ড চালানো যায়
$ who; w; last -n 5
```

ল্যাব ৭: পরিষ্কার করো

```
$ sudo userdel -r labuser1
$ sudo userdel -r labuser2
$ sudo userdel -r labuser3
```

```
$ sudo groupdel labteam
$ sudo groupdel labextra
$ sudo rm /etc/skel/WELCOME.txt
$ sudo pwck -r && sudo grpck -r          # সব ঠিক আছে কিনা
```

সারসংক্ষেপ

কমান্ড / ফাইল	কাজ
`/etc/passwd`	`name:x:UID:GID:GECOS:home:shell` — সবাই পড়ে
`/etc/shadow`	`name:hash:last:min:max:warn:inactive:expire:` — শুধু root
`/etc/group`	`name:x:GID:members`
`/etc/gshadow`	`name:password:admins:members`
`/etc/login.defs`, `/etc/default/useradd`	নতুন ইউজারের ডিফল্ট
`/etc/skel/`	নতুন home-এ কপি হওয়া ফাইল
`getent passwd USER`	সব উৎস থেকে ইউজার খোঁজো
`id`, `id -un`, `groups`	পরিচয় ও গ্রুপ দেখো
`useradd -m -s /bin/bash -G g1 USER`	ইউজার বানাও (portable)
`adduser USER`	interactive ইউজার তৈরি (Debian/Ubuntu)
`useradd -r -s /usr/sbin/nologin USER`	system user
`passwd [USER]`	পাসওয়ার্ড বদলাও
`passwd -l / -u / -e / -S`	লক / আনলক / মেয়াদোত্তীর্ণ / অবস্থা
`chpasswd`	stdin থেকে `user:pass`
`chage -l / -M / -m / -W / -I / -E / -d 0`	পাসওয়ার্ড ও অ্যাকাউন্টের মেয়াদ
`usermod -aG GROUP USER`	গ্রুপে **যোগ** (কখনো `-a` বাদ দিও না)
`usermod -l / -d -m / -s / -L / -U / -e`	নাম / home / shell / লক / আনলক / মেয়াদ
`userdel -r USER`	ইউজার ও home মোছো
`groupadd / groupmod -n / groupdel`	গ্রুপ তৈরি / নাম বদল / মোছা
`gpasswd -a / -d / -A`	সদস্য যোগ / বাদ / অ্যাডমিন
`newgrp G`, `sg G -c CMD`	লগআউট ছাড়া নতুন গ্রুপ
`who`, `w`, `users`	এখন কে লগইন আছে
`last`, `lastb`, `lastlog`	লগইন ইতিহাস, ব্যর্থ চেষ্টা, শেষ লগইন

কমান্ড / ফাইল	কাজ
`su - USER`, `su -c CMD USER`	ইউজার বদলাও (login shell)
`chsh -s SHELL`, `/etc/shells`	login shell বদলাও
`vipw`, `vigr`, `pwck`, `grpck`	নিরাপদে এডিট ও যাচাই
`/usr/sbin/nologin`, `/etc/nologin`	ইউজারের / সবার লগইন বন্ধ

শর্টকাট	কাজ
`Ctrl + D`	shell / su থেকে বেরোও
`Ctrl + Alt + F3–F6`	টেক্সট কনসোল
`Super + L`	স্ক্রিন লক
পাসওয়ার্ড প্রম্পটে `Ctrl + U`	টাইপ করা মুছে ফেলো

নিজেকে যাচাই করো

1. kernel ফাইলের মালিক হিসেবে কী সংরক্ষণ করে — নাম না নম্বর? `ls -l` তাহলে নাম দেখায় কীভাবে?
2. `/etc/passwd` সবাই পড়তে পারে, তাহলে পাসওয়ার্ড নিরাপদ থাকে কীভাবে?
3. `/etc/shadow`-এ hash-এর জায়গায় `!$y$...` দেখলে কী বুঝবে? আর `*` দেখলে?
4. `sudo usermod -G docker rahim` আর `sudo usermod -aG docker rahim`-এর পার্থক্য কী?
5. তোমাকে `docker` গ্রুপে যোগ করা হয়েছে, কিন্তু `id`-তে দেখাচ্ছে না। কেন, আর সমাধান কী?
6. Debian-এ `sudo useradd sumi` চালানোর পর sumi লগইন করতে পারছে না এবং তার home নেই। কেন?
7. নতুন কর্মী যেন প্রথম লগইনেই পাসওয়ার্ড বদলায় — দুটো উপায় বলো।
8. `su` আর `su -`-এর মূল পার্থক্য কী? কোনটা ব্যবহার করা উচিত?
9. `/usr/sbin/nologin` shell আর `/etc/nologin` ফাইলের পার্থক্য কী?
10. কোনো ইউজারের primary group কেন `groupdel` দিয়ে মোছা যায় না?

▼ **উত্তর দেখো**

1. **নম্বর** (UID ও GID), inode-এ। `ls -l` NSS (`/etc/nsswitch.conf` অনুযায়ী `/etc/passwd`, `/etc/group` ইত্যাদি) থেকে নম্বরকে নামে রূপান্তর করে দেখায়। `ls -n` আসল নম্বর দেখায়।
2. `/etc/passwd`-এ পাসওয়ার্ডের জায়গায় শুধু `x` থাকে। আসল hash থাকে `/etc/shadow`-এ, যা শুধু root (Debian-এ shadow গ্রুপও) পড়তে পারে। আর সেখানেও পাসওয়ার্ড নয়, salt-সহ একমুখী hash থাকে।
3. `!` প্রিফিক্স মানে অ্যাকাউন্ট **লক** (`passwd -l` বা `usermod -L`), hash অক্ষত, `passwd -u` দিয়ে খোলা যাবে। `*` মানে কখনো পাসওয়ার্ড সেট হয়নি — পাসওয়ার্ড দিয়ে লগইন অসম্ভব (সাধারণত system user)।

4. `-G` একা supplementary group-এর **পুরো তালিকা প্রতিস্থাপন** করে — rahim অন্য সব গ্রুপ (যেমন sudo) থেকে বাদ পড়বে। `-aG` শুধু **যোগ** করে, বাকিগুলো অক্ষত থাকে।
5. গ্রুপের তালিকা লগইনের সময় process-এর credential-এ বসে; পুরোনো shell পুরোনো তালিকা নিয়েই চলছে। লগআউট করে আবার লগইন করো, অথবা সাময়িকভাবে `newgrp docker` / `sg docker -c "..."`। `id rahim` (নাম দিয়ে) ডাটাবেস দেখে, তাই সেখানে দেখাবে।
6. Debian/Ubuntu-তে `useradd` ডিফল্টভাবে home বানায় না (`-m` লাগে) এবং পাসওয়ার্ড সেট করে না (অ্যাকাউন্ট লক থাকে)। সমাধান: `useradd -m -s /bin/bash sumi` তারপর `passwd sumi`, অথবা `adduser sumi`।
7. `sudo passwd -e USER` অথবা `sudo chage -d 0 USER`।
8. `su` (dash ছাড়া) বর্তমান environment (PATH, PWD ইত্যাদি) রেখে দেয়, non-login shell। `su -` পূর্ণ login shell দেয় — লক্ষ্য ইউজারের home, PATH ও profile ফাইল। নিরাপত্তা ও সঠিকতার জন্য `su -` ব্যবহার করা উচিত (Ubuntu-তে root-এর জন্য `sudo -i`)।
9. `/usr/sbin/nologin` হলো নির্দিষ্ট একজন ইউজারের login shell — শুধু তার লগইন বন্ধ হয়। `/etc/nologin` ফাইল থাকলে **root ছাড়া সবার** লগইন বন্ধ হয়ে যায়, ফাইলের লেখা বার্তা হিসেবে দেখায় — মেইনটেন্যান্সের সময় কাজে লাগে।
10. কারণ `/etc/passwd`-এ ঐ ইউজারের GID ফিল্ড সেই গ্রুপকে নির্দেশ করে — মুছলে ইউজারের primary group অস্তিত্বহীন হয়ে যাবে। আগে ইউজার মুছতে হবে (`userdel`) বা `usermod -g` দিয়ে অন্য primary group দিতে হবে।

অধ্যায় ১৬: ফাইল পারমিশন

এই অধ্যায়ে যা শিখবে

- লিনাক্সে পারমিশন (permission, মানে "অনুমতি") কেন দরকার এবং এটা কোন সমস্যার সমাধান করে
- তিন ধরনের মানুষ: **user (owner)**, **group**, **others**
- তিন ধরনের অনুমতি: **r (read)**, **w (write)**, **x (execute)**, আর **ফাইল ও ডিরেক্টরিতে এদের মানে কেন আলাদা**
- `ls -l` আউটপুটের পারমিশন স্ট্রিং (`-rwxr-xr--`) অক্ষর ধরে ধরে পড়া
- `chmod` দিয়ে পারমিশন বদলানো: **symbolic** পদ্ধতি (`u+x` , `g-w` , `o=r` , `a+x`) আর **octal** পদ্ধতি (`755` , `644` , `600` , `700`)
- octal সংখ্যা মাথায় হিসাব করা: হিসাবের টেবিলসহ
- `chmod -R` , বড় হাতের `X` -এর কৌশল, `find` দিয়ে নিরাপদে recursive পারমিশন দেওয়া
- `chown` ও `chgrp` দিয়ে মালিক আর গ্রুপ বদলানো
- `umask` কীভাবে নতুন ফাইল/ডিরেক্টরির ডিফল্ট পারমিশন ঠিক করে (bitwise হিসাব, শুধু বিয়োগ না)
- **kernel ভেতরে কোন ক্রমে permission check করে**
- সাধারণ ভুল, আর `chmod 777` কেন খারাপ অভ্যাস

১৬.১ পারমিশন কেন দরকার?

একটা অ্যাপার্টমেন্ট বিল্ডিংয়ের কথা ভাবো। প্রত্যেক ফ্ল্যাটের নিজের চাবি আছে। ছাদের চাবি শুধু বিল্ডিংয়ের বাসিন্দাদের কাছে থাকে। আর নিচের নোটিশ বোর্ড পড়তে পারে যে কেউ, কিন্তু নোটিশ লাগাতে পারে শুধু ম্যানেজার।

লিনাক্স শুরু থেকেই **multi-user** (একসাথে অনেক ইউজার চালাতে পারে এমন) সিস্টেম। একটা সার্ভারে একসাথে অনেক মানুষ লগইন করতে পারে, আর অনেক service (যেমন web server, database) আলাদা আলাদা ইউজার হিসেবে চলে। কোনো নিয়ম না থাকলে:

- যেকোনো ইউজার অন্যের ব্যক্তিগত ফাইল পড়ে ফেলতে পারত
- একটা হ্যাক হওয়া web server পুরো সিস্টেমের ফাইল মুছে দিতে পারত
- ভুল করে কেউ `/etc/passwd` এডিট করে সিস্টেম নষ্ট করে ফেলতে পারত

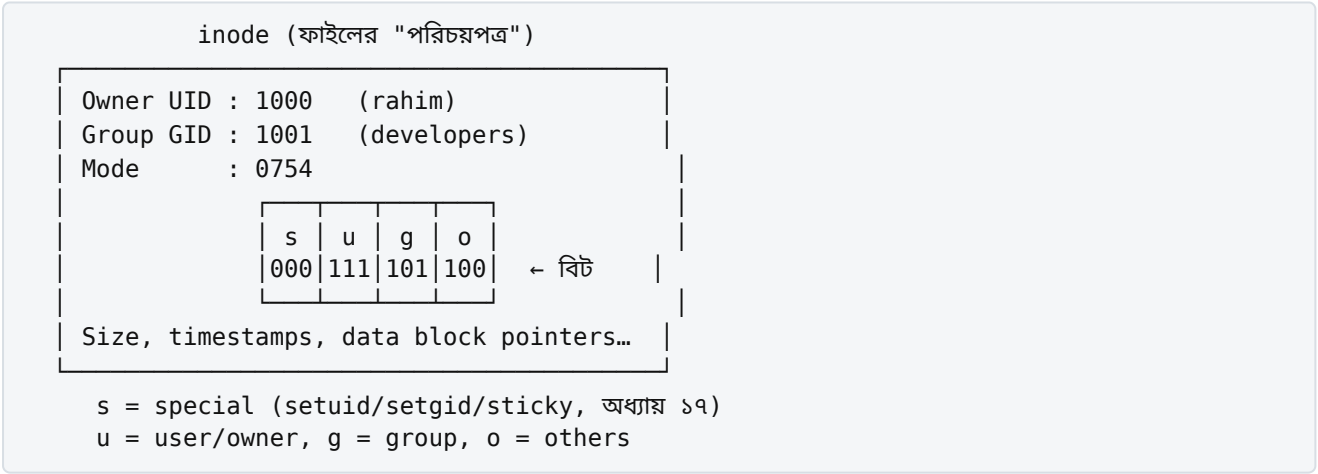
পারমিশন সিস্টেম এই "কে কোন ফাইলের সাথে কী করতে পারবে" প্রশ্নের উত্তর দেয়। এটাকে বলে **DAC (Discretionary Access Control)**। মানে ফাইলের মালিক নিজের ইচ্ছামতো (discretion) ঠিক করে অন্যরা কতটুকু অ্যাক্সেস পাবে।

> 📝 **মনে রাখো:** লিনাক্সে "সবকিছুই ফাইল", তাই এই পারমিশন সিস্টেম শুধু ডকুমেন্ট না, ডিরেক্টরি, ডিভাইস (`/dev/sda`), socket ইত্যাদি সবকিছুর উপর কাজ করে। ইউজার ও গ্রুপ কীভাবে তৈরি হয় সেটা অধ্যায় ১৫-এ দেখেছ। এই অধ্যায় সেটার উপরেই দাঁড়িয়ে আছে।

১৬.২ তিনটি প্রশ্ন: কে, কী, কোথায়

প্রতিটি ফাইলের **inode**-এর ভেতরে (inode নিয়ে বিস্তারিত অধ্যায় ১১-এ আছে) তিনটি তথ্য রাখা থাকে:

1. **Owner UID**: ফাইলের মালিক কোন ইউজার (একটা সংখ্যা, যেমন `1000`)
2. **Group GID**: ফাইলটা কোন গ্রুপের (একটা সংখ্যা, যেমন `1000`)
3. **Mode bits**: ১২টা বিট, যেখানে আসল অনুমতিগুলো লেখা থাকে (৯টা সাধারণ rwx বিট + ৩টা বিশেষ বিট)



> 💡 **টিপস:** kernel নাম চেনে না, শুধু সংখ্যা (UID/GID) চেনে। `ls -l` যে নাম দেখায়, সেটা `/etc/passwd` আর `/etc/group` থেকে UID/GID খুঁজে বের করা নাম। কোনো UID-এর ইউজার মুছে দিলে `ls -l` নামের জায়গায় শুধু সংখ্যা দেখাবে (যেমন `1005`)।

তিন শ্রেণির মানুষ

শ্রেণি	অক্ষর	কারা
User (owner)	`u`	ফাইলের মালিক। সাধারণত যে ফাইলটা তৈরি করেছে
Group	`g`	ফাইলের গ্রুপের সদস্যরা (primary বা supplementary, দুই ধরনের সদস্যই)
Others	`o`	বাকি সবাই, মানে যারা owner না, গ্রুপেরও সদস্য না
All	`a`	উপরের তিনটা একসাথে (`ugo`)

> ⚠️ **সতর্কতা:** `o` মানে **others**, **owner না**! নতুনরা প্রায়ই `o` দেখে owner ভাবে আর `chmod o+w` দিয়ে পুরো দুনিয়াকে write পারমিশন দিয়ে ফেলে। owner হলো `u` (user)।

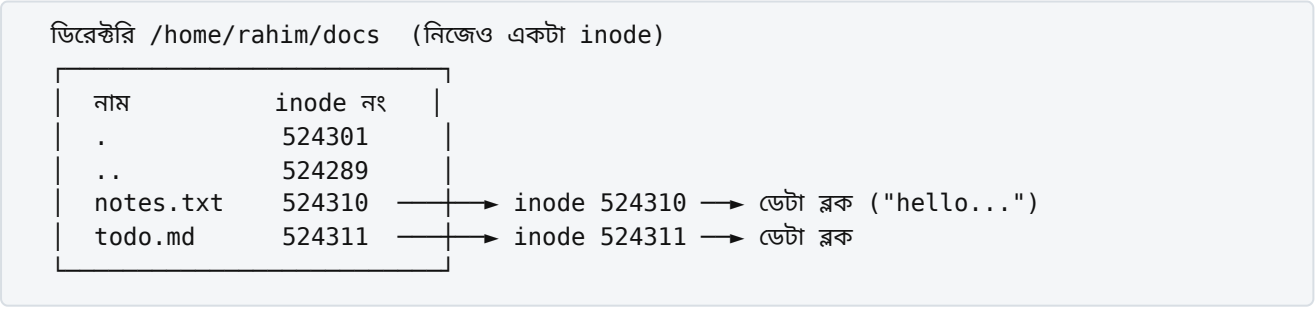
তিন ধরনের অনুমতি

অক্ষর	নাম	octal মান
`r`	read (পড়া)	4
`w`	write (লেখা)	2
`x`	execute (চালানো / ঢোকা)	1
`-`	অনুমতি নেই	0

১৬.৩ rwx: ফাইল বনাম ডিরেক্টরি (সবচেয়ে গুরুত্বপূর্ণ অংশ)

নতুনরা সবচেয়ে বেশি এখানেই আটকে যায়। একই অক্ষর `r`, `w`, `x` ফাইলের জন্য এক জিনিস বোঝায় আর ডিরেক্টরির জন্য আরেক জিনিস। কারণটা বুঝতে হলে মনে রাখো:

> 📝 **মনে রাখো:** একটা **ডিরেক্টরি আসলে একটা বিশেষ ফাইল, যার ভেতরে শুধু একটা তালিকা থাকে**: "নাম → inode নম্বর"। ফাইলের নাম ফাইলের ভেতরে না, **ডিরেক্টরির ভেতরে** থাকে।



এবার উপমাটা: ডিরেক্টরি হলো একটা **ঘর**, আর তার ভেতরের তালিকাটা হলো দরজায় লাগানো **নামের বোর্ড**।

ফাইলের জন্য

পারমিশন	মানে	উদাহরণ কমান্ড
`r`	ফাইলের **কনটেন্ট পড়া** যাবে	`cat`, `less`, `cp` (সোর্স হিসেবে)
`w`	ফাইলের **কনটেন্ট বদলানো** যাবে (লেখা, খালি করা, যোগ করা)	`echo "x" > file`, `nano file`
`x`	ফাইলটাকে **প্রোগ্রাম হিসেবে চালানো** যাবে	`./script.sh`, `./myprogram`

ডিরেক্টরির জন্য

পারমিশন	মানে	উপমা
`r`	ভেতরের **নামগুলোর তালিকা দেখা** যাবে (`ls`)	দরজার কাচ দিয়ে নামের বোর্ড পড়া
`w`	ভেতরে ফাইল **তৈরি, মুছে ফেলা, rename** করা যাবে (তালিকায় নাম যোগ/বাদ)	বোর্ডে নাম লেখা/মোছা
`x`	ডিরেক্টরিতে **ঢোকা** (`cd`) আর ভেতরের জিনিসে **পৌঁছানো** (traverse/search) যাবে	ঘরের দরজা দিয়ে ঢোকার চাবি

কয়েকটা চমকে দেওয়ার মতো, কিন্তু যুক্তিসঙ্গত ঘটনা

১. ফাইল মোছার জন্য ফাইলের পারমিশন লাগে না, লাগে ডিরেক্টরির `w` + `x`। মোছা মানে ডিরেক্টরির তালিকা থেকে নাম বাদ দেওয়া। তাই তুমি যদি একটা ডিরেক্টরিতে `w` + `x` পাও, তাহলে তার ভেতরের **অন্যের মালিকানাধীন read-only ফাইলও মুছে ফেলতে পারো**। (এটা ঠেকাতে sticky bit আছে, `/tmp`-এ যেটা ব্যবহার হয়। দেখো অধ্যায় ১৭।)

```
$ ls -l report.txt
-r--r--r-- 1 root root 120 Nov 12 10:00 report.txt
$ rm report.txt
rm: remove write-protected regular file 'report.txt'? y
$ ls report.txt
ls: cannot access 'report.txt': No such file or directory
```

`rm` শুধু ভদ্রতা করে জিজ্ঞেস করেছে। kernel আসলে অনুমতি দিয়েছে কারণ ডিরেক্টরিটা তোমার, আর তাতে `w` + `x` আছে।

২. `r` আছে কিন্তু `x` নেই এমন ডিরেক্টরি: নামের তালিকা দেখা যায়, কিন্তু ফাইলের বিস্তারিত (size, permission) বা কনটেন্ট দেখা যায় না।

```
$ chmod 644 secretdir      # r আছে, x নেই
$ \ls secretdir            # \ দিয়ে alias এড়ানো, শুধু নাম
a.txt  b.txt
$ ls -l secretdir
ls: cannot access 'secretdir/a.txt': Permission denied
ls: cannot access 'secretdir/b.txt': Permission denied
total 0
-????????? ? ? ? ?            ? a.txt
-????????? ? ? ? ?            ? b.txt
$ cat secretdir/a.txt
cat: secretdir/a.txt: Permission denied
```

(Ubuntu-তে `ls` আসলে `ls --color=auto`-এর alias। রঙ ঠিক করতে সে প্রতিটি ফাইলের ধরন জানতে চায়, তাই সাধারণ `ls`-এও কখনো কখনো "Permission denied" লাইন দেখতে পারো। নামগুলো কিন্তু ঠিকই দেখায়, কারণ তালিকা পড়তে শুধু `r` লাগে।)

৩. `x` আছে কিন্তু `r` নেই এমন ডিরেক্টরি: তালিকা দেখা যায় না, কিন্তু **নাম জানা থাকলে** ভেতরের ফাইলে সরাসরি পৌঁছানো যায়। অন্ধকার ঘরের মতো: আলো নেই, কিন্তু কোন তাকে কী আছে জানলে হাত বাড়িয়ে নিতে পারবে।

```
$ chmod 711 dropbox        # owner সব, বাকিরা শুধু x
# অন্য ইউজার হিসেবে:
$ ls /home/rahim/dropbox
ls: cannot open directory '/home/rahim/dropbox': Permission denied
$ cat /home/rahim/dropbox/known-name.txt
এই ফাইলের লেখা দেখা যাচ্ছে!
```

এজন্যই অনেক সিস্টেমে home ডিরেক্টরি `711` বা `750` রাখা হয়।

৪. `w` আছে কিন্তু `x` নেই এমন ডিরেক্টরি: কার্যত কিছুই লেখা যায় না! নতুন ফাইল বানাতে বা মুছতে ভেতরে "ঢুকতে" হয়, মানে `x` লাগে। তাই ডিরেক্টরিতে `w` প্রায় সবসময় `x`-এর সাথে জোড়া বাঁধে।

৫. path-এর প্রতিটি ধাপে `x` লাগে। `/home/rahim/docs/notes.txt` পড়তে হলে `/`, `/home`, `/home/rahim`, `/home/rahim/docs` সবগুলোতে `x` থাকতে হবে, আর শেষে `notes.txt`-এ `r`। মাঝের একটা দরজাও বন্ধ থাকলে ভেতরে পৌঁছানো যাবে না, ফাইলটা `777` হলেও না।

```
$ namei -l /home/rahim/docs/notes.txt
f: /home/rahim/docs/notes.txt
drwxr-xr-x root  root  /
drwxr-xr-x root  root  home
drwxr-x--- rahim rahim rahim
drwxr-xr-x rahim rahim docs
-rw-r--r-- rahim rahim notes.txt
```

> 💡 **টিপস:** "Permission denied" পেলে কিন্তু ফাইলের পারমিশন ঠিক মনে হলে `namei -l <path>` চালাও। এটা path-এর প্রতিটি অংশের পারমিশন একসাথে দেখায়। উপরের উদাহরণে `/home/rahim` হলো `drwxr-x---`, তাই অন্য ইউজার (others) `notes.txt`-এ পৌঁছাতে পারবে না।

৬. স্ক্রিপ্ট চালাতে `r` + `x` দুটোই লাগে, binary-তে শুধু `x` যথেষ্ট। কম্পাইল করা প্রোগ্রাম (যেমন `/usr/bin/ls`) kernel নিজেই মেমোরিতে লোড করে, তাই শুধু `x` লাগে। কিন্তু `#!/bin/bash` দিয়ে শুরু হওয়া স্ক্রিপ্ট আসলে `bash` নামের প্রোগ্রাম **পড়ে** চালায়। তাই তোমার ঐ ফাইলটা পড়ার অনুমতি (`r`) লাগে।

```
$ chmod 111 hello.sh
$ ./hello.sh
/bin/bash: ./hello.sh: Permission denied
$ chmod 555 hello.sh
$ ./hello.sh
Hello!
```

এক নজরে সারণি

	ফাইলে	ডিরেক্টরিতে
`r`	কনটেন্ট পড়া	নামের তালিকা দেখা (`ls`)
`w`	কনটেন্ট বদলানো	ভেতরে তৈরি/মোছা/rename (সাথে `x` লাগে)
`x`	প্রোগ্রাম হিসেবে চালানো	ভেতরে ঢোকা ও পৌঁছানো (`cd`, path traverse)

১৬.৪ `ls -l`-এর পারমিশন স্ট্রিং পড়া

```
$ ls -l
total 16
-rwxr-xr-- 1 rahim developers 2048 Nov 12 09:15 deploy.sh
drwxr-x--- 2 rahim rahim      4096 Nov 12 09:20 private
lrwxrwxrwx 1 rahim rahim        11 Nov 12 09:21 latest -> deploy.sh
-rw-r--r--. 1 root root        512 Nov 12 09:30 config.conf
-rw-rw-r--+ 1 rahim rahim      100 Nov 12 09:31 shared.txt
```

প্রথম লাইনটা ভেঙে দেখি:

```
-  rwx  r-x  r--   1  rahim  developers  2048  Nov 12 09:15  deploy.sh
|   |    |    |    |    |         |         |        |             |
|   |    |    |    |    |         |         |        |             └─ নাম
|   |    |    |    |    |         |         |        └─ শেষ পরিবর্তনের সময় (mtime)
|   |    |    |    |    |         |         └─ সাইজ (bytes)
|   |    |    |    |    |         └─ গ্রুপ
|   |    |    |    |    └─ owner (মালিক)
|   |    |    |    └─ hard link সংখ্যা
|   |    |    └─ others: শুধু read
|   |    └─ group: read + execute
|   └─ user/owner: read + write + execute
└─ ফাইলের ধরন
```

প্রথম অক্ষর: ফাইলের ধরন

অক্ষর	ধরন
`-`	সাধারণ ফাইল (regular file)
`d`	ডিরেক্টরি
`l`	symbolic link (soft link)
`c`	character device (যেমন `/dev/tty`)
`b`	block device (যেমন `/dev/sda`)
`s`	socket
`p`	named pipe (FIFO)

শেষের অতিরিক্ত অক্ষর (১১তম)

অক্ষর	মানে
(কিছু নেই)	বাড়তি কিছু নেই
`.`	SELinux security context আছে (Fedora/RHEL-এ প্রায় সব ফাইলে দেখবে)
`+`	ACL (Access Control List) আছে। বিস্তারিত অধ্যায় ১৭

> 📝 **মনে রাখো:** symbolic link সবসময় `lrwxrwxrwx` দেখায়। লিনাক্সে link-এর নিজের পারমিশন ব্যবহার হয় না। আসল check হয় link যে ফাইলকে নির্দেশ করে (target) তার পারমিশনে।

অন্য জায়গায় যেখানে `s`, `S`, `t`, `T` দেখবে

কখনো `x`-এর জায়গায় `s` বা `t` দেখবে, যেমন `/usr/bin/passwd`-এ `-rwsr-xr-x` বা `/tmp`-এ `drwxrwxrwt`। এগুলো setuid/setgid/sticky বিশেষ বিট, যেগুলো অধ্যায় ১৭-এ বিস্তারিত আছে।

`stat` দিয়ে সংখ্যায় দেখা

```
$ stat deploy.sh
  File: deploy.sh
  Size: 2048       Blocks: 8          IO Block: 4096   regular file
Device: 8,2        Inode: 1835021     Links: 1
Access: (0754/-rwxr-xr--)  Uid: ( 1000/   rahim)   Gid: ( 1001/developers)
...

$ stat -c '%a %A %U:%G %n' deploy.sh private
754 -rwxr-xr-- rahim:developers deploy.sh
750 drwxr-x--- rahim:rahim private
```

`stat -c` ফরম্যাট	মানে
`%a`	octal পারমিশন (যেমন `754`)
`%A`	মানুষের পড়ার মতো স্ট্রিং (`-rwxr-xr--`)
`%U` / `%u`	owner নাম / UID
`%G` / `%g`	গ্রুপ নাম / GID
`%n`	ফাইলের নাম

`ls -n` দিয়ে নামের বদলে সংখ্যা (UID/GID) দেখতে পারো, আর `ls -ld dir` দিয়ে ডিরেক্টরির ভেতরের জিনিস না, **ডিরেক্টরিটার নিজের** পারমিশন দেখতে পারো।

```
$ ls -ld /tmp /etc
drwxr-xr-x 140 root root 12288 Nov 12 08:00 /etc
drwxrwxrwt  20 root root  4096 Nov 12 10:12 /tmp
```

১৬.৫ octal: পারমিশনকে সংখ্যায় লেখা

প্রতিটি শ্রেণির (u, g, o) তিনটা বিট আছে, তাই প্রতিটি শ্রেণি একটা ০ থেকে ৭-এর সংখ্যা দিয়ে লেখা যায়। ৮-ভিত্তিক বলে একে **octal** বলে।

```
r   w   x
4 + 2 + 1  = 7
|   |   └── বিট ০ (মান 1)
|   └────── বিট ১ (মান 2)
└────────── বিট ২ (মান 4)
```

সম্পূর্ণ ক্যালকুলেশন টেবিল

octal	binary	স্ট্রিং	হিসাব	মানে
`0`	`000`	`---`	0	কিছুই না
`1`	`001`	`--x`	1	শুধু execute
`2`	`010`	`-w-`	2	শুধু write (বিরল)
`3`	`011`	`-wx`	2+1	write + execute (বিরল)
`4`	`100`	`r--`	4	শুধু read
`5`	`101`	`r-x`	4+1	read + execute
`6`	`110`	`rw-`	4+2	read + write
`7`	`111`	`rwx`	4+2+1	সব

সবচেয়ে বেশি ব্যবহৃত মোডগুলো

octal	স্ট্রিং	কোথায় ব্যবহার
`755`	`rwxr-xr-x`	প্রোগ্রাম, স্ক্রিপ্ট, সাধারণ ডিরেক্টরি: সবাই চালাতে/ঢুকতে পারে, শুধু owner বদলাতে পারে
`644`	`rw-r--r--`	সাধারণ ফাইল (ডকুমেন্ট, কনফিগ, ওয়েব পেজ): সবাই পড়তে পারে, owner লিখতে পারে
`600`	`rw-------`	গোপন ফাইল: SSH private key, পাসওয়ার্ড ফাইল, `.env`
`700`	`rwx------`	ব্যক্তিগত ডিরেক্টরি বা স্ক্রিপ্ট: `~/.ssh`
`750`	`rwxr-x---`	টিমের ডিরেক্টরি: গ্রুপ পড়তে/ঢুকতে পারে, বাইরের কেউ না
`640`	`rw-r-----`	গ্রুপ পড়তে পারে এমন কনফিগ (যেমন লগ ফাইল)
`775`	`rwxrwxr-x`	শেয়ার করা টিম ডিরেক্টরি: গ্রুপও লিখতে পারে
`664`	`rw-rw-r--`	টিমের সাধারণ ফাইল
`400`	`r--------`	শুধু owner পড়তে পারে, এমনকি সে নিজেও লিখতে পারে না (যেমন AWS `.pem` key)

octal	স্ট্রিং	কোথায় ব্যবহার
`711`	`rwx--x--x`	home ডিরেক্টরি: বাকিরা নাম জানলে ঢুকতে পারে, তালিকা দেখতে পারে না
`777`	`rwxrwxrwx`	সবার সব অনুমতি: ⚠️ প্রায় সবসময় ভুল (১৬.১২ দেখো)

স্ট্রিং থেকে octal: ধাপে ধাপে

`rwxr-x---` কে সংখ্যায় আনো:

```
rwx   r-x   ---
4+2+1 4+0+1 0+0+0
 7     5     0     →  750
```

octal থেকে স্ট্রিং

`640` কে স্ট্রিংয়ে আনো:

```
6 = 4+2   → rw-
4 = 4     → r--
0         → ---
            → rw-r-----
```

> 💡 **টিপস:** শুধু তিনটা সংখ্যা মনে রাখো: **r=4, w=2, x=1**। ৭ = সব, ৬ = rw, ৫ = rx, ৪ = r। বাকিগুলো খুব কম লাগে।

চার অঙ্কের octal

কখনো `0755` বা `4755` দেখবে। প্রথম অঙ্কটা বিশেষ বিটগুলোর জন্য (setuid=4, setgid=2, sticky=1)। শুধু তিন অঙ্ক লিখলে প্রথম অঙ্ক `0` ধরা হয় (তবে ডিরেক্টরিতে GNU `chmod` তিন অঙ্কের মোডে আগের setuid/setgid বিট মুছে দেয় না; বিস্তারিত অধ্যায় ১৭-এ)।

১৬.৬ `chmod`: পারমিশন বদলানো

chmod = **ch**ange **mod**e। ফাইলের mode bits বদলায়।

> 📝 **মনে রাখো:** একটা ফাইলের পারমিশন বদলাতে পারে **শুধু ঐ ফাইলের owner, অথবা root** (বা `CAP_FOWNER` capability থাকা প্রসেস)। গ্রুপের সদস্য হলেও, এমনকি ফাইলে `w` থাকলেও, তুমি অন্যের ফাইলের পারমিশন বদলাতে পারবে না।

Syntax:

```
chmod [OPTIONS] MODE FILE...
chmod [OPTIONS] --reference=RFILE FILE...
```

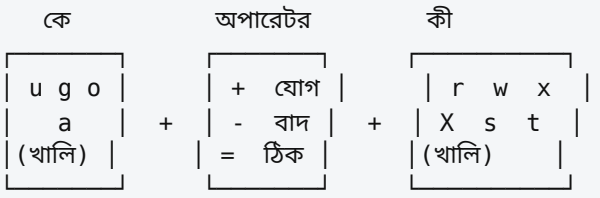
MODE দুইভাবে লেখা যায়: **symbolic** বা **octal**।

গুরুত্বপূর্ণ অপশন

অপশন	মানে
`-R`, `--recursive`	ডিরেক্টরি ও তার ভেতরের সবকিছুতে প্রয়োগ
`-v`, `--verbose`	প্রতিটি ফাইলের জন্য কী করল জানায়
`-c`, `--changes`	শুধু যেটা আসলে বদলেছে সেটা জানায়
`-f`, `--silent`, `--quiet`	বেশিরভাগ error message চুপ রাখে
`--reference=RFILE`	অন্য একটা ফাইলের পারমিশন কপি করে বসায়
`--preserve-root`	`/`-এর উপর recursive কাজ করতে দেয় না (GNU chmod-এ `-R` এর সাথে ডিফল্ট চালু)

Symbolic পদ্ধতি

গঠন: **`[কে][অপারেটর][কী]`**



অপারেটর	মানে
`+`	উল্লেখিত বিট **যোগ** করো, বাকি সব যেমন আছে থাকুক
`-`	উল্লেখিত বিট **বাদ** দাও, বাকি সব যেমন আছে থাকুক
`=`	**ঠিক এটাই** বসাও, ঐ শ্রেণির বাকি বিট মুছে দাও

উদাহরণ (ধরো শুরুতে ফাইলটা `rw-r--r--`, মানে `644`):

কমান্ড	ফল	ব্যাখ্যা
`chmod u+x file`	`rwxr--r--` (744)	owner-কে execute দাও
`chmod g+w file`	`rw-rw-r--` (664)	গ্রুপকে write দাও
`chmod g-w file`	`rw-r--r--` (644)	গ্রুপের write বাদ (আগে না থাকলে কিছুই হয় না)
`chmod o=r file`	`rw-r--r--` (644)	others-এর ঠিক শুধু r থাকবে
`chmod o= file`	`rw-r-----` (640)	others-এর সব বাদ (= এর পরে খালি)

কমান্ড	ফল	ব্যাখ্যা
`chmod a+x file`	`rwxr-xr-x` (755)	সবাইকে execute
`chmod +x file`	`rwxr-xr-x` (755)*	`a` উহ্য, কিন্তু umask মেনে চলে (*নিচে দেখো)
`chmod u=rwx,g=rx,o= file`	`rwxr-x---` (750)	কমা দিয়ে একসাথে কয়েকটা
`chmod ug+rw,o-rwx file`	`rw-rw----` (660)	একাধিক শ্রেণি একসাথে
`chmod go=u file`	`rw-rw-rw-` (666)	গ্রুপ ও others-কে owner-এর মতো অনুমতি কপি

> ⚠️ **সতর্কতা:** `chmod +x file` (কে বলা নেই) আর `chmod a+x file`-এর মধ্যে সূক্ষ্ম পার্থক্য আছে। কে বলা না থাকলে GNU `chmod` **umask-এ যে বিটগুলো বন্ধ আছে সেগুলো সেট করে না**। যেমন umask `022` হলে `chmod +w file` শুধু owner-কে write দেবে, গ্রুপ/others-কে না। কিন্তু `chmod a+w` umask উপেক্ষা করে সবাইকে দেবে। `+x`-এর ক্ষেত্রে সাধারণ umask-এ (`022` / `002`) দুটোর ফল একই হয়।

বড় হাতের `X`: স্মার্ট execute

`X` মানে: "execute দাও **শুধু যদি** এটা ডিরেক্টরি হয়, **অথবা** কারো কাছে আগে থেকেই execute থাকে"। recursive কাজে এটা অসাধারণ কাজের:

```
$ chmod -R u=rwX,go=rX project/
```

এতে ডিরেক্টরিগুলো `755` হবে (ঢোকা যাবে), সাধারণ টেক্সট ফাইলগুলো `644` হবে (ভুলে executable হবে না), আর আগে থেকে executable স্ক্রিপ্টগুলো `755` থাকবে।

Octal পদ্ধতি

```
$ chmod 755 deploy.sh
$ chmod 644 index.html
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 700 ~/.ssh
$ ls -l deploy.sh index.html
-rwxr-xr-x 1 rahim rahim 2048 Nov 12 09:15 deploy.sh
-rw-r--r-- 1 rahim rahim  900 Nov 12 09:40 index.html
```

Symbolic বনাম Octal: কখন কোনটা?

পরিস্থিতি	ভালো পছন্দ	কারণ
শুধু একটা বিট যোগ/বাদ করতে চাও	symbolic (`u+x`)	বাকি বিট অক্ষত থাকে
পুরো পারমিশন নির্দিষ্ট একটা অবস্থায় আনতে চাও	octal (`644`)	একবারে পরিষ্কার, ভুলের সুযোগ কম
স্ক্রিপ্ট বা ডকুমেন্টেশনে লিখছ	octal	সংক্ষিপ্ত, সবাই চেনে

পরিস্থিতি	ভালো পছন্দ	কারণ
recursive, ফাইল আর ডিরেক্টরিতে আলাদা আচরণ চাও	symbolic + `X`	octal-এ এটা সম্ভব না

`-v` ও `-c` দিয়ে দেখা কী হলো

```
$ chmod -v 640 notes.txt
mode of 'notes.txt' changed from 0644 (rw-r--r--) to 0640 (rw-r-----)

$ chmod -c 640 notes.txt
$                       # আগেই 640 ছিল, তাই -c কিছু বলল না
```

`--reference` দিয়ে কপি

```
$ chmod --reference=good.conf broken.conf
$ stat -c '%a %n' good.conf broken.conf
640 good.conf
640 broken.conf
```

১৬.৭ `chmod -R` ও নিরাপদ recursive পারমিশন

```
$ chmod -R 755 mysite/
```

এটা `mysite/`-এর ভেতরের **সব** ফাইল আর ডিরেক্টরিকে `755` বানাবে। সমস্যাটা হলো, এতে সব টেক্সট ফাইল, ছবি, CSS **executable** হয়ে যাবে, যেটা অপ্রয়োজনীয় আর নিরাপত্তার দিক থেকে খারাপ।

উল্টো দিকে `chmod -R 644 mysite/` আরও খারাপ: ডিরেক্টরিগুলো থেকে `x` চলে যাবে, ফলে কেউ (তুমিও) আর ভেতরে ঢুকতে পারবে না!

```
$ chmod -R 644 mysite/
$ ls mysite/
ls: cannot access 'mysite/css': Permission denied
...
$ cd mysite/css
bash: cd: mysite/css: Permission denied
```

সঠিক পদ্ধতি ১: বড় হাতের `X`

```
$ chmod -R u=rwX,go=rX mysite/
```

সঠিক পদ্ধতি ২: `find` দিয়ে আলাদা আলাদা

```
# শুধু ডিরেক্টরিগুলো 755
$ find mysite/ -type d -exec chmod 755 {} +
```

```
# শুধু ফাইলগুলো 644
$ find mysite/ -type f -exec chmod 644 {} +

# শুধু .sh ফাইলগুলো executable
$ find mysite/ -type f -name '*.sh' -exec chmod 755 {} +
```

`{} +` মানে `find` একসাথে অনেকগুলো ফাইলের নাম একটা `chmod`-এ পাঠায় (অনেক দ্রুত)। `find` নিয়ে বিস্তারিত অধ্যায় ১২-এ।

> ⚠️ **সতর্কতা:** `sudo chmod -R ... /` বা ভুল করে স্পেস দেওয়া path (`sudo chmod -R 777 /var/www`, খেয়াল করো `/` এর পরে স্পেস!) পুরো সিস্টেম নষ্ট করে দিতে পারে। `sudo`, `ssh` সহ অনেক প্রোগ্রাম ভুল পারমিশন দেখলে কাজ করতে অস্বীকার করে। GNU chmod `-R` এর সাথে ডিফল্টভাবে `--preserve-root` চালু রাখে, তাই ঠিক `/` তে আটকাবে, কিন্তু `/usr` বা `/etc`-এর মতো অন্য গুরুত্বপূর্ণ ডিরেক্টরি রক্ষা পাবে না। recursive কমান্ডের আগে সবসময় path দুইবার দেখো।

কোনগুলো ভুল পারমিশনে আছে খুঁজে বের করা

```
# সবাই লিখতে পারে এমন ফাইল (world-writable)
$ find /var/www -type f -perm -o=w

# ঠিক 777 পারমিশনের সব জিনিস
$ find . -perm 777

# যেকোনো execute বিট আছে এমন ফাইল
$ find . -type f -perm /111
```

`find -perm` রূপ	মানে
`-perm 644`	পারমিশন **হুবহু** 644
`-perm -644`	**অন্তত** এই বিটগুলো সব সেট আছে
`-perm /111`	এই বিটগুলোর **যেকোনো একটা** সেট আছে

১৬.৮ `chown`: মালিক বদলানো

chown = **ch**ange **own**er। ফাইলের owner এবং/অথবা গ্রুপ বদলায়।

> 📝 **মনে রাখো:** লিনাক্সে **owner বদলাতে পারে শুধু root** (বা `CAP_CHOWN` capability)। সাধারণ ইউজার নিজের ফাইলও অন্য কাউকে "দিয়ে দিতে" পারে না। কেন? কারণ তাহলে কেউ নিজের বিশাল ফাইল অন্যের নামে করে তার disk quota ভরিয়ে দিতে পারত, বা দায় অন্যের ঘাড়ে চাপাতে পারত।

Syntax:

```
chown [OPTIONS] [OWNER][:[GROUP]] FILE...
```

লেখার ধরন	মানে
`chown rahim file`	শুধু owner বদলাও
`chown rahim:developers file`	owner ও গ্রুপ দুটোই বদলাও
`chown :developers file`	শুধু গ্রুপ বদলাও (`chgrp` -এর মতো)
`chown rahim: file`	owner = rahim, গ্রুপ = rahim-এর **login (primary) গ্রুপ**
`chown 1000:1001 file`	নামের বদলে সংখ্যা (UID:GID) দিয়েও হয়

> 💡 **টিপস:** পুরোনো স্ক্রিপ্টে `chown rahim.developers` (ডট দিয়ে) দেখতে পারো। এটা পুরোনো রীতি। GNU coreutils এখনো সাধারণত মেনে নেয়, কিন্তু ইউজারনেমে ডট থাকলে বিভ্রান্তি হয় আর POSIX-এ এটা নেই। সবসময় কোলন (`:`) ব্যবহার করো।

গুরুত্বপূর্ণ অপশন

অপশন	মানে
`-R` , `--recursive`	ডিরেক্টরির ভেতরের সবকিছুতে
`-v` , `-c`	`chmod` -এর মতোই: verbose / শুধু পরিবর্তন জানাও
`-h` , `--no-dereference`	symlink-এর target না, symlink-টার নিজের মালিকানা বদলাও
`--reference=RFILE`	অন্য ফাইলের owner:group কপি করো
`--from=CUR_OWNER:CUR_GROUP`	শুধু সেই ফাইলগুলো বদলাও যাদের বর্তমান owner/group এটা
`--preserve-root`	`/` -এর উপর recursive কাজ আটকায়

বাস্তব উদাহরণ

```
# ওয়েব সাইটের ফাইল web server ইউজারকে দেওয়া (Debian/Ubuntu-তে www-data)
$ sudo chown -R www-data:www-data /var/www/mysite

# RHEL/Fedora-তে Apache ইউজার হলো apache
$ sudo chown -R apache:apache /var/www/html/mysite

# root হিসেবে কপি করা ফাইল আবার নিজের নামে ফেরত আনা
$ sudo chown -R "$USER": ~/Downloads/project

# শুধু 'olduser'-এর ফাইলগুলো 'newuser'-কে দেওয়া
$ sudo chown -R --from=olduser newuser /srv/data
```

```
$ sudo chown -v rahim:developers deploy.sh
changed ownership of 'deploy.sh' from root:root to rahim:developers
```

সাধারণ ইউজার চেষ্টা করলে:

```
$ chown karim myfile.txt
chown: changing ownership of 'myfile.txt': Operation not permitted
```

> ⚠️ **সতর্কতা:** সাধারণ ফাইলে `chown` চালালে kernel নিরাপত্তার জন্য **setuid ও setgid বিট মুছে দেয়** (executable হলে)। কারণটা অধ্যায় ১৭-এ বোঝা যাবে। তাই setuid বিট দিতে চাইলে আগে `chown`, তারপর `chmod`।

১৬.৯ `chgrp`: গ্রুপ বদলানো

chgrp = **ch**ange **gr**ou**p**। শুধু গ্রুপ বদলায়।

```
chgrp [OPTIONS] GROUP FILE...
```

অপশনগুলো `chown`-এর মতোই (`-R`, `-v`, `-c`, `-h`, `--reference`)।

`chown`-এর সাথে বড় পার্থক্য: **সাধারণ ইউজারও `chgrp` ব্যবহার করতে পারে**, তবে দুইটা শর্তে:

1. সে ফাইলের **owner**
2. নতুন গ্রুপটার সে **নিজে সদস্য**

```
$ id
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),27(sudo),1001(developers)

$ chgrp developers report.txt        # কাজ করবে: rahim developers-এর সদস্য
$ ls -l report.txt
-rw-r--r-- 1 rahim developers 300 Nov 12 11:00 report.txt

$ chgrp docker report.txt            # কাজ করবে না: rahim docker-এর সদস্য না
chgrp: changing group of 'report.txt': Operation not permitted
```

> 💡 **টিপস:** নতুন গ্রুপে যোগ হওয়ার পর (`sudo usermod -aG developers rahim`) সেটা কার্যকর হয় **নতুন লগইন সেশনে**। পুরোনো টার্মিনালে `id` চালালে নতুন গ্রুপ দেখাবে না। log out করে আবার login করো, অথবা সাময়িকভাবে `newgrp developers` চালাও (এটা নতুন একটা শেল খোলে)। এটা অধ্যায় ১৫-এ বিস্তারিত আছে।

টিম শেয়ারিং ডিরেক্টরির একটা সাধারণ রেসিপি

```
$ sudo mkdir /srv/team
$ sudo chgrp developers /srv/team
$ sudo chmod 2775 /srv/team     # 2 = setgid, ভেতরের নতুন ফাইল developers গ্রুপ পাবে
$ ls -ld /srv/team
drwxrwsr-x 2 root developers 4096 Nov 12 11:10 /srv/team
```

এখানে `2`-এর মানে (setgid) অধ্যায় ১৭-এ শিখবে। আপাতত জানো, এতে টিমের সবার তৈরি ফাইল স্বয়ংক্রিয়ভাবে `developers` গ্রুপের হবে।

১৬.১০ `umask`: নতুন ফাইলের ডিফল্ট পারমিশন কে ঠিক করে?

তুমি `touch a.txt` বা `mkdir dir` চালালে নতুন ফাইল/ডিরেক্টরি কোন পারমিশন নিয়ে জন্মায়? উত্তর আসে দুইটা জিনিস থেকে:

1. **প্রোগ্রামটা কী চেয়েছে** (requested mode)। বেশিরভাগ প্রোগ্রাম সাধারণ ফাইলের জন্য `666` (`rw-rw-rw-`) আর ডিরেক্টরির জন্য `777` (`rwxrwxrwx`) চায়।
2. **প্রসেসের umask**: একটা "ছাঁকনি" (mask), যেটা কোন কোন বিট **বন্ধ** করতে হবে তা বলে।

উপমা: প্রোগ্রাম একটা পূর্ণ কেক (৬৬৬ বা ৭৭৭) দিতে চায়, আর umask হলো একটা ছুরি, যেটা কয়েকটা টুকরো কেটে সরিয়ে রাখে।

> 📝 **মনে রাখো:** নতুন ফাইলে execute বিট সাধারণত থাকে না কারণ `touch`, `nano`, শেলের `>` ইত্যাদি **নিজেরাই 666 চায়, 777 না**। umask কখনো কোনো বিট **যোগ** করে না, শুধু **বাদ** দেয়। তাই নতুন টেক্সট ফাইল কখনো আপনা থেকে executable হয় না, স্ক্রিপ্টের জন্য তোমাকে `chmod +x` দিতে হয়।

আসল সূত্র (bitwise AND NOT)

```
final_mode = requested_mode  AND  (NOT umask)
```

উদাহরণ, umask = `022`:

```
ফাইল:
  requested   666  = 110 110 110
  NOT umask   ~022 = 111 101 101
  ──────────────────────────────── AND
  final       644  = 110 100 100   → rw-r--r--

ডিরেক্টরি:
  requested   777  = 111 111 111
  NOT umask   ~022 = 111 101 101
```

```
                                    ──────────────────────────── AND
final       755  = 111 101 101   → rwxr-xr-x
```

"বিয়োগ করলেই তো হয়?" না!

অনেক টিউটোরিয়াল বলে "666 থেকে umask বিয়োগ করো"। সাধারণ umask (`022`, `002`, `077`)-এ এটা মিলে যায়, কিন্তু সবসময় না। umask `033` দিয়ে দেখো:

```
ভুল পদ্ধতি (বিয়োগ):   666 - 033 = 633  → rw--wx-wx   ✗ (ফাইলে x চলে এল!)
সঠিক (AND NOT):
  666   = 110 110 110
  ~033  = 111 100 100
  AND   = 110 100 100  = 644  → rw-r--r--          ✓
```

```
$ umask 033
$ touch test.txt
$ stat -c '%a %A' test.txt
644 -rw-r--r--
```

> 💡 **টিপস:** সহজ মনে রাখার উপায়: umask-এর যে বিটগুলো ১, final-এ সেই বিটগুলো **অবশ্যই ০** হবে। বাকি বিট প্রোগ্রাম যা চেয়েছে তাই।

umask টেবিল

umask	নতুন ফাইল (666 থেকে)	নতুন ডিরেক্টরি (777 থেকে)	কখন ব্যবহার
`000`	`666` rw-rw-rw-	`777` rwxrwxrwx	⚠️ প্রায় কখনো না
`002`	`664` rw-rw-r--	`775` rwxrwxr-x	ব্যক্তিগত গ্রুপ (user private group) থাকা সাধারণ ইউজার, টিম কাজ
`022`	`644` rw-r--r--	`755` rwxr-xr-x	root ও সার্ভিসগুলোর প্রচলিত ডিফল্ট
`027`	`640` rw-r-----	`750` rwxr-x---	সার্ভারে নিরাপদ পছন্দ: others কিছুই পাবে না
`077`	`600` rw-------	`700` rwx------	সর্বোচ্চ গোপনীয়তা: শুধু নিজে

`umask` কমান্ড

```
$ umask            # বর্তমান মান, octal-এ (প্রথম 0 = বিশেষ বিট)
0002

$ umask -S         # symbolic-এ: কোন বিটগুলো "অনুমোদিত"
u=rwx,g=rwx,o=rx

$ umask 027        # এই শেলের জন্য বদলাও
$ umask u=rwx,g=rx,o=     # symbolic ভাবেও সেট করা যায় (= 027)
```

```
$ umask
0027
```

> ⚠️ **সতর্কতা:** `umask -S`-এর আউটপুট উল্টো দিক থেকে ভাবায়: এটা দেখায় কোন অনুমতিগুলো **থাকতে পারবে**, আর octal `umask` দেখায় কোনগুলো **কেটে ফেলা হবে**।

umask কোথা থেকে আসে? (distro-ভেদে)

umask আসলে প্রতিটি **প্রসেসের** একটা বৈশিষ্ট্য। child প্রসেস parent থেকে umask উত্তরাধিকার সূত্রে পায় (inherit করে)। তাই শেলে `umask` বদলালে ঐ শেল থেকে চালানো সব প্রোগ্রাম নতুন মান পায়, কিন্তু অন্য টার্মিনাল বা পুরো সিস্টেম না।

```
  login (pam_umask / login.defs)
        │ umask = 0002
        ▼
     bash  ── ~/.bashrc-এ "umask 027" ──►  umask = 0027
        │
        ├──► vim        (0027 পায়)
        └──► python     (0027 পায়)
```

জায়গা	কাজ
`/etc/login.defs`-এর `UMASK` লাইন	Debian/Ubuntu/RHEL-এ সিস্টেম-ব্যাপী ডিফল্ট (সাধারণত `022`)। `USERGROUPS_ENAB yes` থাকলে ইউজারের নাম আর primary group-এর নাম একই হলে (এবং UID = GID হলে) গ্রুপ বিটগুলো owner-এর মতো ধরা হয়, ফলে কার্যত `002` হয়
`pam_umask` মডিউল	Debian/Ubuntu-তে লগইনের সময় উপরের মান প্রয়োগ করে
`/etc/profile`, `/etc/bashrc` (RHEL/Fedora)	পুরোনো RHEL/Fedora-তে এখানে শর্তসাপেক্ষে `002` / `022` সেট হয়
`~/.bashrc`, `~/.profile`	ইউজারের নিজের পছন্দ (সবশেষে চলে, তাই সবার উপরে জেতে)
systemd ইউনিট ফাইলের `UMask=`	সার্ভিসের জন্য। ডিফল্ট `0022` (অধ্যায় ২৬)

তাই Ubuntu 24.04 বা Fedora-তে সাধারণ ইউজার হিসেবে `umask` চালালে সাধারণত `0002` দেখবে, আর `sudo -i` দিয়ে root হলে `0022`। ঠিক কী দেখবে সেটা নিজের মেশিনে `umask` চালিয়েই নিশ্চিত হও।

স্থায়ীভাবে বদলাতে `~/.bashrc`-এর শেষে লিখে দাও (অধ্যায় ৪৪):

```
$ echo 'umask 027' >> ~/.bashrc
$ source ~/.bashrc
```

umask-এর সীমাবদ্ধতা

- প্রোগ্রাম নিজে যদি কম পারমিশন চায়, umask তার চেয়ে বেশি দেবে না। যেমন `ssh-keygen` private key সবসময় `600` দিয়ে বানায়, তোমার umask `000` হলেও।
- umask শুধু **নতুন তৈরি** জিনিসে প্রভাব ফেলে। পুরোনো ফাইলের পারমিশন বদলায় না, আর `chmod`-এর উপর umask প্রয়োগ হয় না (শুধু "কে" না লেখা symbolic mode বাদে, ১৬.৬ দেখো)।
- `cp` নতুন ফাইল তৈরির সময় সোর্সের পারমিশন চায়, তারপর umask প্রয়োগ হয়। `cp -p` দিলে সোর্সের mode, owner, timestamp যতটা সম্ভব হুবহু রাখে।
- ডিরেক্টরিতে **default ACL** থাকলে নতুন ফাইলের জন্য umask-এর বদলে ACL নিয়ম প্রযোজ্য হয় (অধ্যায় ১৭)।

১৬.১১ Under the hood: kernel কীভাবে permission check করে

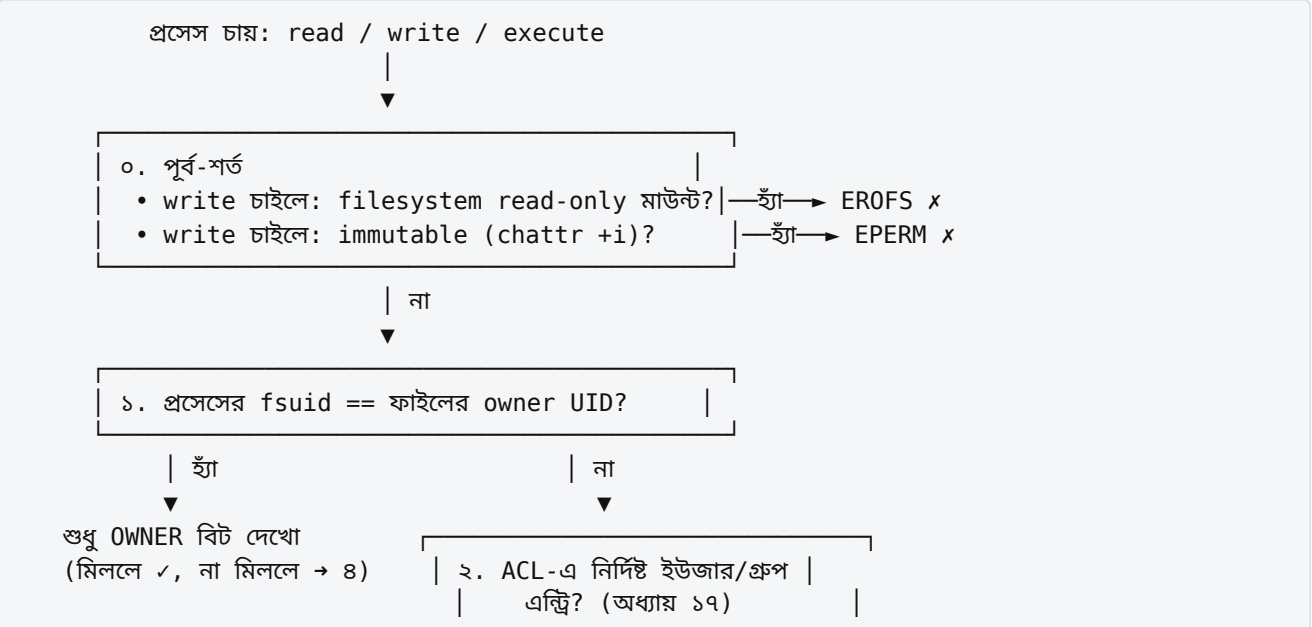
তুমি `cat /home/karim/notes.txt` চালালে আসলে কী হয়? `cat` প্রোগ্রাম kernel-কে একটা **system call** (kernel-এর কাছে সেবা চাওয়ার নির্দিষ্ট দরজা) পাঠায়: `openat("/home/karim/notes.txt", O_RDONLY)`। kernel তখন একজন দারোয়ানের মতো কাজ করে।

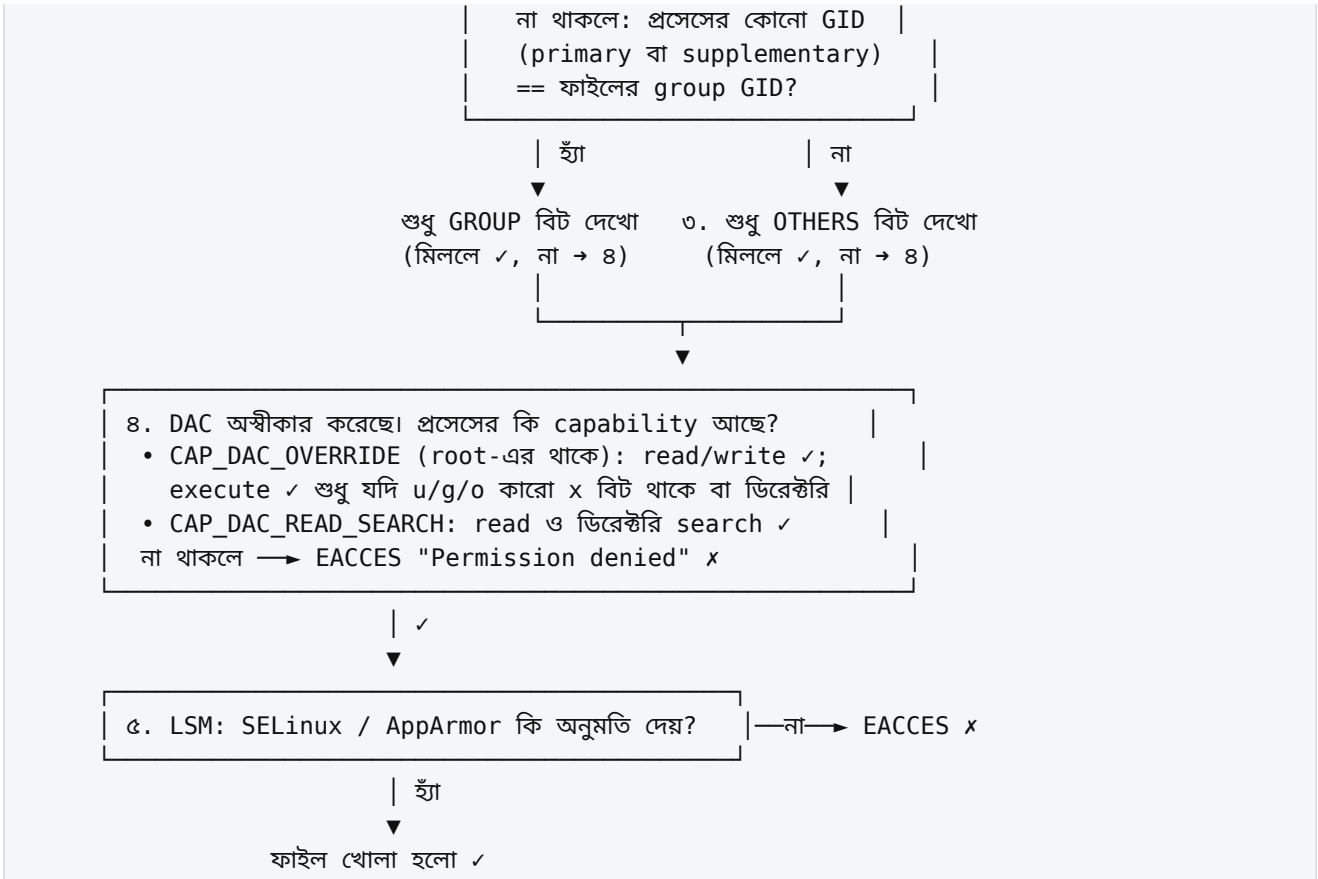
ধাপ ১: path ধরে হাঁটা (path walk)

kernel `/` থেকে শুরু করে প্রতিটি ডিরেক্টরিতে **search (`x`) পারমিশন** check করে, তারপর পরের অংশের নাম খোঁজে। যেকোনো ধাপে `x` না থাকলে সেখানেই `EACCES` (Permission denied)।

ধাপ ২: শেষ ফাইলের উপর check

প্রতিটি check-এ kernel প্রসেসের **filesystem UID ও GID** (সাধারণত effective UID/GID-এর সমান; প্রসেস ও UID নিয়ে অধ্যায় ২৪) ব্যবহার করে। ক্রমটা এরকম:





তিনটা খুব গুরুত্বপূর্ণ শিক্ষা

১. প্রথম মিলটাই চূড়ান্ত, নিচে নামে না (no fallthrough)। তুমি owner হলে kernel **শুধু owner বিট** দেখে, group বা others বিট কখনো দেখে না, এমনকি সেখানে বেশি অনুমতি থাকলেও। মজার পরীক্ষা:

```
$ echo "secret" > weird.txt
$ chmod 077 weird.txt          # ----rwxrwx
$ ls -l weird.txt
----rwxrwx 1 rahim rahim 7 Nov 12 12:00 weird.txt
$ cat weird.txt
cat: weird.txt: Permission denied
```

তুমি (owner) পড়তে পারছ না, কিন্তু অন্য যে কেউ পারবে! তবে তুমি owner, তাই `chmod 644 weird.txt` দিয়ে নিজেকে আবার অনুমতি দিতে পারো। পারমিশন বদলানোর অধিকার ফাইলের rwx বিটের উপর নির্ভর করে না, শুধু মালিকানার উপর।

এভাবে কাউকে "বাদ দেওয়া" সম্ভব: যেমন `chmod 604 file` আর ফাইলের গ্রুপ `interns` হলে, interns গ্রুপের সদস্যরা পড়তে পারবে না, অথচ বাইরের সবাই পারবে। (বাস্তবে এভাবে না করে ACL ব্যবহার করা ভালো।)

২. root প্রায় সব পারে, কিন্তু সব না। root-এর `CAP_DAC_OVERRIDE` থাকায় `000` পারমিশনের ফাইলও পড়তে/লিখতে পারে। কিন্তু:

- কোনো `x` বিট নেই এমন ফাইল root-ও **চালাতে** পারে না (ভুল করে টেক্সট ফাইল চালানো ঠেকাতে)
- read-only মাউন্ট বা `chattr +i` (immutable) ফাইলে root-ও লিখতে পারে না (অধ্যায় ১৭)

- SELinux/AppArmor root-কেও আটকাতে পারে

```
$ sudo cat /etc/shadow | head -1      # /etc/shadow সাধারণত 640 বা 000, তবু root পড়ে
root:*:19700:0:99999:7:::
$ chmod 644 notscript.txt
$ sudo ./notscript.txt
sudo: ./notscript.txt: command not found
```

(এখানে `sudo` আগেই দেখে ফেলে যে কোনো `x` বিট নেই, তাই চালায় না। শেলে root হিসেবে সরাসরি চালালে "Permission denied" আসবে।)

৩. পারমিশন check হয় ফাইল খোলার সময়, প্রতিবার পড়ার সময় না। একটা প্রসেস ফাইল খুলে ফেলার পর (file descriptor পেয়ে গেলে) তুমি `chmod 000` দিলেও ঐ প্রসেস পড়া চালিয়ে যেতে পারবে। নতুন `open()` থেকেই নতুন নিয়ম কাজ করবে। তাই কোনো গোপন ফাইলের পারমিশন ভুলে খোলা ছিল কিনা সন্দেহ হলে শুধু `chmod` না, সংশ্লিষ্ট প্রসেসও restart করো।

নিজের পরিচয় দেখা

kernel তোমাকে কীভাবে দেখে, সেটা জানতে:

```
$ id
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),4(adm),27(sudo),1001(developers)
```

ফাইলের group যদি এই তালিকার যেকোনো একটা হয়, তাহলে তুমি group বিট পাবে।

কোনো কাজ করতে পারবে কিনা আগে থেকে পরীক্ষা

```
$ test -r /etc/shadow && echo "পড়া যাবে" || echo "পড়া যাবে না"
পড়া যাবে না
$ [ -w ~/notes.txt ] && echo "লেখা যাবে"
লেখা যাবে
$ [ -x /usr/bin/ls ] && echo "চালানো যাবে"
চালানো যাবে
```

শেল স্ক্রিপ্টে এই test-গুলো খুব কাজের (অধ্যায় ৪১)।

১৬.১২ `chmod 777` কেন খারাপ?

ইন্টারনেটে "Permission denied? `chmod 777` দাও!" ধরনের পরামর্শ প্রচুর দেখবে। এটা ঘরের তালা খুলতে না পেরে **দরজাটাই খুলে রাস্তায় ফেলে দেওয়ার** মতো। কাজ হয়ে যায়, কিন্তু চোরেরও কাজ হয়ে যায়।

কেন খারাপ

1. **যে কেউ ফাইল বদলাতে পারে।** সিস্টেমের যেকোনো ইউজার বা যেকোনো হ্যাক হওয়া service (যেমন একটা দুর্বল PHP প্লাগিন) ফাইলের ভেতরে ক্ষতিকর কোড বসিয়ে দিতে পারে।
2. **যে কেউ ফাইল চালাতে পারে।** কনফিগ বা ডেটা ফাইল executable হওয়ার কোনো দরকার নেই।
3. **ডিরেক্টরিতে 777 হলে যে কেউ ভেতরের যেকোনো ফাইল মুছতে বা বদলে দিতে পারে** (sticky bit না থাকলে)। আক্রমণকারী তোমার `config.php` মুছে নিজেরটা রেখে দিতে পারে।
4. **একটা স্ক্রিপ্ট 777 হলে এবং root সেটা cron-এ চালালে** (অধ্যায় ২৯), যেকোনো সাধারণ ইউজার সেটা এডিট করে root হয়ে যেতে পারে। এটাকে বলে **privilege escalation**।
5. **অনেক প্রোগ্রাম ইচ্ছে করে কাজ বন্ধ করে দেয়।** যেমন OpenSSH দেখে যে private key বা `~/.ssh` অন্যরা পড়তে/লিখতে পারে, তাহলে সেটা ব্যবহার করতে অস্বীকার করে:

```
$ chmod 777 ~/.ssh/id_ed25519
$ ssh server
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0777 for '/home/rahim/.ssh/id_ed25519' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
```

6. **আসল সমস্যাটা লুকিয়ে ফেলে।** "Permission denied"-এর আসল কারণ সাধারণত ভুল owner বা ভুল group। 777 দিয়ে উপসর্গ চাপা দিলে আসল ভুলটা থেকেই যায়।

777-এর বদলে সঠিক সমাধান

পরিস্থিতি	ভুল সমাধান	সঠিক সমাধান
web server আপলোড ফোল্ডারে লিখতে পারছে না	`chmod -R 777 uploads/`	`sudo chown -R www-data:www-data uploads/` তারপর `chmod 755` (বা গ্রুপ দিয়ে `775`)
টিমের সবাই একটা ফোল্ডারে কাজ করবে	`chmod 777 /srv/team`	একটা গ্রুপ বানাও, `chgrp`, `chmod 2775` (অধ্যায় ১৫ ও ১৭)
স্ক্রিপ্ট চলছে না	`chmod 777 script.sh`	`chmod u+x script.sh` (বা `755`)
নির্দিষ্ট একজন ইউজারকে অ্যাক্সেস দরকার	`chmod 777`	ACL: `setfacl -m u:karim:rw file` (অধ্যায় ১৭)
root-এর তৈরি ফাইল নিজে এডিট করতে পারছ না	`sudo chmod 777 file`	`sudo chown $USER: file`

> 💡 **টিপস:** "Permission denied" পেলে এই তিনটা প্রশ্ন করো: (১) **আমি কে?** `id` (২) **ফাইলটা কার?** `ls -l` / `stat` (৩) **পথে কোনো দরজা বন্ধ?** `namei -l path`। প্রায় সব সমস্যার উত্তর এখানেই পাবে।

১৬.১৩ সাধারণ ভুল ও সমাধান

ভুল	কী হয়	সমাধান
`o` কে owner ভাবা (`chmod o+w`)	পুরো দুনিয়া লিখতে পারে	owner = `u`
`chmod -R 644 dir/`	ডিরেক্টরিতে `x` চলে যায়, ভেতরে ঢোকা যায় না	`chmod -R u=rwX,go=rX dir/` বা `find`
`chmod -R 755` সব ফাইলে	সব ফাইল অপ্রয়োজনে executable	`find -type f -exec chmod 644 {} +`
ফাইলে `777` দিয়েও পড়া যায় না	মাঝের কোনো ডিরেক্টরিতে `x` নেই	`namei -l` দিয়ে খুঁজে বের করো
`sudo` দিয়ে ফাইল বানিয়ে পরে নিজে এডিট করতে না পারা	ফাইল root-এর	কাজের ফাইল `sudo` ছাড়া বানাও, বা `sudo chown`
গ্রুপে যোগ হলাম, তবু অ্যাক্সেস নেই	পুরোনো সেশনে নতুন গ্রুপ লোড হয়নি	log out/in, বা `newgrp`
`~/.ssh` বা key-তে ঢিলেঢালা পারমিশন	SSH key ব্যবহার করে না	`chmod 700 ~/.ssh`, `chmod 600 ~/.ssh/id_*`, `chmod 644 ~/.ssh/*.pub`
USB/Windows (NTFS/FAT) ড্রাইভে `chmod` কাজ করে না	এসব filesystem লিনাক্স পারমিশন রাখে না	মাউন্টের সময় `uid=`, `gid=`, `umask=` অপশন দাও (অধ্যায় ৩২)
`x` আছে তবু স্ক্রিপ্ট চলে না	filesystem `noexec` দিয়ে মাউন্ট (যেমন অনেক সময় `/tmp`), বা `r` নেই	`mount
owner হয়েও নিজের ফাইল পড়া যাচ্ছে না	owner বিটে `r` নেই (no fallthrough)	`chmod u+r file`
`chmod` বলছে "Operation not permitted"	তুমি owner নও, বা ফাইল immutable	`sudo`, বা `lsattr` দিয়ে `i` attribute দেখো

GUI থেকে পারমিশন

GNOME Files (Nautilus), Dolphin ইত্যাদি ফাইল ম্যানেজারে ফাইলের উপর right-click করে **Properties** → **Permissions** ট্যাবে গেলে owner/group/others-এর জন্য ড্রপডাউন পাবে, আর স্ক্রিপ্টের জন্য "Allow executing file as program" ধরনের চেকবক্স। ভেতরে এরা একই `chmod` / `chown` system call ব্যবহার করে। তবে recursive বা নির্দিষ্ট octal কাজের জন্য টার্মিনালই দ্রুত ও নির্ভরযোগ্য।

১৬.১৪ পারমিশন নিয়ে কাজের সময় দরকারি কীবোর্ড শর্টকাট

পারমিশনের নিজস্ব কোনো শর্টকাট নেই, তবে bash-এর এই শর্টকাটগুলো `chmod` / `chown` কাজ অনেক দ্রুত করে (বিস্তারিত অধ্যায় ৬):

শর্টকাট	কাজ	পারমিশন-কাজে ব্যবহার
`Tab`	ফাইল/ডিরেক্টরির নাম auto-complete	লম্বা path ভুল ছাড়া লেখা
`Tab` `Tab`	সম্ভাব্য সব মিল দেখাও	কোন ফাইলে chmod দেবে দেখে নেওয়া
`Alt` + `.` (বা `Esc` তারপর `.`)	আগের কমান্ডের শেষ argument বসাও	`ls -l file` তারপর `chmod 600` + `Alt+.`
`!$`	আগের কমান্ডের শেষ argument (history expansion)	`chmod +x deploy.sh` তারপর `ls -l !$`
`sudo !!`	আগের পুরো কমান্ডটা sudo দিয়ে আবার চালাও	"Operation not permitted" পেলে
`Ctrl` + `R`	history-তে উল্টো খোঁজা	আগে লেখা লম্বা `find ... -exec chmod` কমান্ড খুঁজে বের করা
`Ctrl` + `A` / `Ctrl` + `E`	লাইনের শুরু / শেষে যাও	কমান্ডের শুরুতে `sudo` বসানো
`Ctrl` + `C`	চলমান কমান্ড বাতিল	ভুল path-এ চলা `chmod -R` থামানো (যতটুকু হয়ে গেছে তা ফিরবে না!)

হাতে-কলমে অনুশীলন

সব কাজ নিজের home-এ একটা খেলার ডিরেক্টরিতে করো, যাতে কিছু নষ্ট না হয়।

ল্যাব ১: পারমিশন স্ট্রিং পড়া

```
$ mkdir -p ~/permlab && cd ~/permlab
$ touch file1.txt
$ mkdir dir1
$ ls -l
$ stat -c '%a %A %n' file1.txt dir1
$ umask
```

প্রশ্ন: `file1.txt` আর `dir1`-এর পারমিশন কত এল? তোমার umask দিয়ে হিসাব করে মিলিয়ে দেখো (umask `0002` হলে `664` / `775`, `0022` হলে `644` / `755`)।

ল্যাব ২: symbolic ও octal

```
$ chmod u+x file1.txt          ; stat -c '%a %A' file1.txt
$ chmod g-w,o= file1.txt       ; stat -c '%a %A' file1.txt
$ chmod a+r file1.txt          ; stat -c '%a %A' file1.txt
$ chmod 600 file1.txt          ; stat -c '%a %A' file1.txt
$ chmod u=rwx,g=rx,o=r file1.txt ; stat -c '%a %A' file1.txt
```

প্রতিটি কমান্ড চালানোর **আগে** খাতায় অনুমান লেখো, তারপর মিলিয়ে দেখো। শেষটা `754` হওয়া উচিত।

ল্যাব ৩: ডিরেক্টরির r, w, x আলাদা করে বোঝা

```
$ mkdir box && echo "hello" > box/a.txt

$ chmod 300 box     # -wx: তালিকা দেখা যায় না, কিন্তু নাম জানলে পৌঁছানো যায়
$ ls box            # Permission denied
$ cat box/a.txt     # কাজ করবে!  (a.txt নিজে 664/644)

$ chmod 500 box     # r-x: দেখা ও পড়া যায়, তৈরি/মোছা যায় না
$ ls box
$ touch box/b.txt   # Permission denied
$ rm box/a.txt      # Permission denied

$ chmod 600 box     # rw-: x নেই, তাই কার্যত কিছুই হয় না
$ cd box            # Permission denied
$ touch box/c.txt   # Permission denied

$ chmod 755 box     # আবার স্বাভাবিক
```

ল্যাব ৪: স্ক্রিপ্টে r ও x

```
$ printf '#!/bin/bash\necho "চলছে!"\n' > hi.sh
$ ./hi.sh           # Permission denied (x নেই)
$ chmod u+x hi.sh && ./hi.sh
$ chmod u-r hi.sh && ./hi.sh   # আবার Permission denied: bash পড়তে পারছে না
$ chmod 755 hi.sh
```

ল্যাব ৫: umask-এর প্রভাব

```
$ ( umask 077; touch secret.txt; mkdir secretdir )
$ ( umask 033; touch odd.txt )
$ stat -c '%a %n' secret.txt secretdir odd.txt
600 secret.txt
700 secretdir
644 odd.txt
$ umask             # বাইরের শেলের umask বদলায়নি, কারণ ( ) একটা subshell
```

ল্যাব ৬: no-fallthrough পরীক্ষা

```
$ echo "test" > trap.txt
$ chmod 077 trap.txt
$ cat trap.txt      # Permission denied, অথচ group/others পারবে
$ chmod u+r trap.txt && cat trap.txt
```

ল্যাব ৭: নিরাপদ recursive

```
$ mkdir -p site/css site/js && touch site/index.html site/css/a.css site/js/run.sh
$ chmod -R 777 site            # ইচ্ছে করে "খারাপ" অবস্থা
$ find site -perm -o=w         # কোনগুলো world-writable?
```

```
$ chmod -R u=rwX,go=rX site
$ find site -exec stat -c '%a %n' {} +   # সব ফাইল এখনো 755! কেন? নিচে দেখো
$ find site -type d -exec chmod 755 {} +
$ find site -type f -exec chmod 644 {} +
$ find site -name '*.sh' -exec chmod 755 {} +
$ find site -exec stat -c '%a %n' {} +
```

লক্ষ্য করো: `chmod -R 777`-এর পরে `X` দিলে সব ফাইলেই আগে থেকে `x` ছিল, তাই `X` সবগুলোকে `755` রেখে দেয়। "খারাপ" অবস্থা থেকে মেরামতের সময় তাই `find -type f` পদ্ধতিই নিরাপদ।

ল্যাব ৮: chgrp ও chown (sudo লাগবে)

```
$ id
$ sudo groupadd labgroup
$ sudo usermod -aG labgroup "$USER"
$ newgrp labgroup               # নতুন শেল, এখন labgroup সক্রিয়
$ chgrp labgroup file1.txt && ls -l file1.txt
$ exit                          # newgrp শেল থেকে বেরোও
$ sudo chown root: file1.txt && ls -l file1.txt
$ chmod 644 file1.txt           # Operation not permitted, তুমি আর owner নও
$ sudo chown "$USER": file1.txt
```

পরিষ্কার করা

```
$ cd ~ && rm -rf ~/permlab
$ sudo gpasswd -d "$USER" labgroup && sudo groupdel labgroup
```

সারসংক্ষেপ

কমান্ড / ধারণা	কাজ	উদাহরণ
`ls -l` / `ls -ld dir`	পারমিশন স্ট্রিং দেখা	`ls -ld /tmp`
`stat -c '%a %A %U:%G %n'`	octal ও স্ট্রিং একসাথে	`stat -c '%a' file`
`namei -l path`	path-এর প্রতিটি ধাপের পারমিশন	`namei -l /var/www/html/index.html`
`id`	নিজের UID, GID, গ্রুপ	`id`
`chmod MODE file`	পারমিশন বদলানো (owner/root)	`chmod 644 f`, `chmod u+x f`
`chmod -R u=rwX,go=rX dir`	নিরাপদ recursive	
`chmod --reference=A B`	A-এর পারমিশন B-তে	
`chown user:group file`	মালিক বদলানো (শুধু root)	`sudo chown -R www-data: /var/www`
`chgrp group file`	গ্রুপ বদলানো (owner + সদস্য হলে)	`chgrp developers f`

কমান্ড / ধারণা	কাজ	উদাহরণ
`umask` / `umask -S` / `umask 027`	ডিফল্ট পারমিশনের মাস্ক দেখা/সেট	
`find -perm`	পারমিশন দিয়ে ফাইল খোঁজা	`find . -perm -o=w`
`test -r/-w/-x`	অনুমতি আছে কিনা পরীক্ষা	`[ -x f ] && ./f`

মনে রাখার সূত্র	
r=4, w=2, x=1	`7=rwx 6=rw- 5=r-x 4=r-- 0=---`
ফাইল / ডিরেক্টরি	`644` / `755` সাধারণ; `600` / `700` গোপন
umask	`final = requested AND NOT umask` (666 ফাইল, 777 ডিরেক্টরি)
kernel check	owner → (ACL) group → others → capability → LSM; প্রথম মিলই চূড়ান্ত
ডিরেক্টরি	`r` = তালিকা, `w` = তৈরি/মোছা (+x), `x` = ঢোকা/পৌঁছানো
ফাইল মোছা	ফাইলের না, **ডিরেক্টরির** `w` + `x` লাগে
777	প্রায় সবসময় ভুল; owner/group/ACL ঠিক করো

নিজেকে যাচাই করো

1. `drwxr-x--x` পারমিশনের একটা ডিরেক্টরির octal মান কত? others এখানে কী করতে পারবে আর কী পারবে না?
2. তোমার কাছে একটা ফাইলের পারমিশন `-r--r--r--` আর owner root। তবুও তুমি সেটা মুছে ফেলতে পারলে। কীভাবে সম্ভব?
3. umask `027` হলে `touch` দিয়ে বানানো ফাইল আর `mkdir` দিয়ে বানানো ডিরেক্টরির পারমিশন কত হবে?
4. umask `033` হলে নতুন ফাইলের পারমিশন কত? "666 − 033 = 633" কেন ভুল?
5. `chmod -R 644 project/` চালানোর পর আর `cd project/src` করা যাচ্ছে না কেন? সঠিক কমান্ড কী হতো?
6. একটা ফাইল `----rw-rw-` আর তুমি তার owner, আবার ফাইলের গ্রুপেরও সদস্য। তুমি কি এটা পড়তে পারবে? কেন?
7. সাধারণ ইউজার `rahim` কি `chown karim myfile` চালাতে পারবে? `chgrp developers myfile` পারবে কোন শর্তে?
8. একটা bash স্ক্রিপ্টে `--x--x--x` (111) পারমিশন দিলে সেটা চলে না, কিন্তু `/usr/bin/ls`-এ শুধু `x` থাকলেই চলে। কেন?
9. root কি `chmod 644` করা (কারো `x` নেই) একটা ফাইল execute করতে পারবে? `000` পারমিশনের ফাইল পড়তে পারবে?
10. web server আপলোড ফোল্ডারে লিখতে পারছে না। কেউ `chmod -R 777 uploads` বলল। কেন এটা খারাপ, আর ভালো সমাধান কী?

▼ উত্তর দেখো

1. `rwx`=7, `r-x`=5, `--x`=1, তাই **751**। others ডিরেক্টরিতে ঢুকতে পারবে আর নাম জানা থাকলে ভেতরের ফাইলে পৌঁছাতে পারবে (`x`), কিন্তু `ls` দিয়ে তালিকা দেখতে পারবে না (`r` নেই), ফাইল তৈরি/মুছতে পারবে না (`w` নেই)।

2. ফাইল মোছা মানে **ডিরেক্টরির** তালিকা থেকে নাম বাদ দেওয়া। ফাইলটা যে ডিরেক্টরিতে আছে সেখানে তোমার `w` + `x` আছে, তাই মুছতে পেরেছ। ফাইলের নিজের পারমিশন এখানে অপ্রাসঙ্গিক (ডিরেক্টরিতে sticky bit থাকলে আলাদা, অধ্যায় ১৭)।

3. ফাইল: `666 AND NOT 027` = **640** (`rw-r-----`)। ডিরেক্টরি: `777 AND NOT 027` = **750** (`rwxr-x---`)।

4. **644**। umask বিয়োগ না, bitwise mask: umask-এর যে বিট ১, সেটা final-এ ০ হবে। `666` = `110 110 110`, `~033` = `111 100 100`, AND = `110 100 100` = 644। বিয়োগ করলে 633 (`rw--wx-wx`) আসে, যেটায় execute বিট চলে আসে। অথচ umask কখনো বিট যোগ করতে পারে না।

5. `644`-এ কারো `x` নেই, আর ডিরেক্টরিতে ঢুকতে (`cd`) ও ভেতরে পৌঁছাতে `x` লাগে। সঠিক: `chmod -R u=rwX,go=rX project/` অথবা `find project -type d -exec chmod 755 {} +` আর `find project -type f -exec chmod 644 {} +`।

6. **না।** তুমি owner, তাই kernel শুধু owner বিট (`---`) দেখে এবং থেমে যায়; group বিটে নামে না (no fallthrough)। তবে তুমি owner বলে `chmod u+r` দিয়ে নিজেকে অনুমতি দিতে পারো।

7. `chown karim myfile` **পারবে না**: owner বদলাতে root (`CAP_CHOWN`) লাগে। `chgrp developers myfile` পারবে যদি সে ফাইলের owner হয় **এবং** নিজে `developers` গ্রুপের সদস্য হয়।

8. কম্পাইল করা binary kernel নিজে মেমোরিতে লোড করে চালায়, তাই শুধু `x` যথেষ্ট। স্ক্রিপ্টের ক্ষেত্রে kernel `#!` লাইন দেখে interpreter (`/bin/bash`) চালায়, আর bash তোমার হয়ে ফাইলটা **পড়ে**। তাই `r` লাগে।

9. `644` ফাইল root-ও execute করতে পারবে না: `CAP_DAC_OVERRIDE` execute অনুমতি দেয় শুধু যদি u/g/o কারো অন্তত একটা `x` বিট থাকে। কিন্তু `000` ফাইল root **পড়তে ও লিখতে পারবে** (read-only মাউন্ট, immutable attribute বা SELinux/AppArmor বাধা না দিলে)।

10. 777 হলে সিস্টেমের যেকোনো ইউজার/প্রসেস সেখানে ফাইল বদলাতে, মুছতে, ক্ষতিকর কোড রাখতে পারে, আর ফাইলগুলো অপ্রয়োজনে executable হয়। আসল সমস্যা (ভুল owner) চাপা পড়ে যায়। ভালো সমাধান: `sudo chown -R www-data:www-data uploads` (RHEL/Fedora-তে `apache`), তারপর ডিরেক্টরি `755`, ফাইল `644`; টিমেরও লেখার দরকার হলে গ্রুপ দিয়ে `775` / `2775` বা ACL।

অধ্যায় ১৭: বিশেষ পারমিশন, ACL ও ফাইল অ্যাট্রিবিউট

অধ্যায় ১৬-এ আমরা `rwx`, owner/group/others আর `chmod`, `chown`, `umask` শিখেছি। ওই ৯টি বিট দিয়ে বেশিরভাগ কাজ চলে যায়। তবু কিছু প্রশ্নের উত্তর ওখানে মেলে না:

- সাধারণ ইউজার `passwd` চালিয়ে নিজের পাসওয়ার্ড বদলায়। অথচ পাসওয়ার্ড থাকে `/etc/shadow`-এ, যেটায় শুধু root লিখতে পারে। তাহলে কাজটা হয় কীভাবে?
- `/tmp`-এ সবাই লিখতে পারে। তাহলে আমি কি অন্যের ফাইল মুছে দিতে পারি?
- একটা ফাইল শুধু একজন বাড়তি ইউজারকে পড়তে দিতে চাই, অথচ নতুন group বানাতে চাই না। উপায় কী?
- এমন ফাইল কি বানানো যায় যেটা root-ও ভুল করে মুছতে পারবে না?

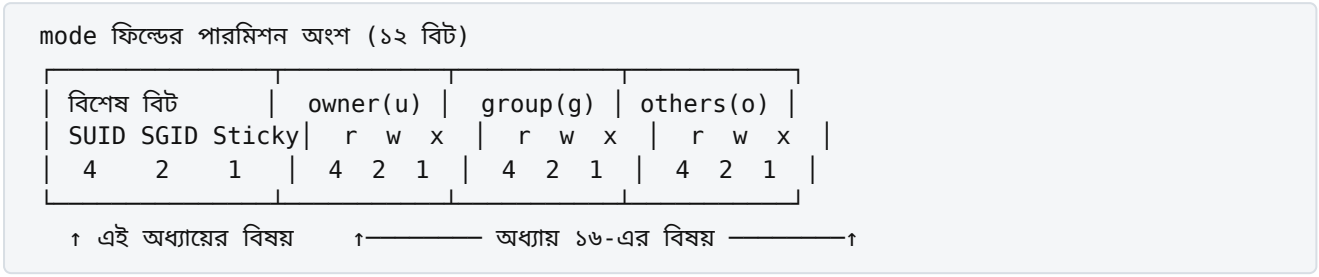
এই অধ্যায়ে এসবের উত্তর পাবে।

এই অধ্যায়ে যা শিখবে

- তিনটি বিশেষ পারমিশন বিট: **SUID**, **SGID** আর **sticky bit**। এগুলো কী, কেন আছে, ভেতরে কীভাবে কাজ করে
- Real UID আর Effective UID-এর পার্থক্য (SUID বোঝার মূল চাবি)
- `passwd` কীভাবে `/etc/shadow` বদলায়
- ফাইল আর ডিরেক্টরির ওপর SGID-এর দুই রকম আচরণ
- `/tmp`-এ sticky bit কেন থাকে
- 4/2/1 octal দিয়ে চার অঙ্কের মোড (`4755`, `2775`, `1777`)
- `ls -l`-এ `s`, `S`, `t`, `T` অক্ষরের মানে
- সিস্টেমের সব SUID/SGID ফাইল খোঁজা, আর এগুলোর নিরাপত্তা ঝুঁকি
- **ACL** (Access Control List): `getfacl`, `setfacl -m/-x/-b/-k/-d/-R`, mask, default ACL
- **ফাইল অ্যাট্রিবিউট**: `chattr` আর `lsattr` (`+i` immutable, `+a` append-only ইত্যাদি)
- **Linux capabilities**: `getcap`, `setcap`। root-এর ক্ষমতাকে ছোট ছোট টুকরো করে দেওয়ার পদ্ধতি

১৭.১ আগে একটু ঝালিয়ে নিই: সাধারণ পারমিশন কোথায় থাকে

প্রতিটি ফাইলের metadata থাকে তার **inode**-এ (অধ্যায় ১১)। সেখানে `mode` নামে একটা ১৬-বিটের ঘর আছে। এর ৪টি বিট ফাইলের ধরন বলে (regular file, directory, symlink ইত্যাদি), বাকি ১২টি পারমিশনের জন্য:



অধ্যায় ১৬-এ আমরা শুধু ডানের ৯টি বিট দেখেছি (যেমন `755`)। এখন সবচেয়ে বাঁয়ের ৩টি বিটের পালা। এগুলো নিয়েই পুরো মোড চার অঙ্কের হয়, যেমন `4755`।

> 📝 **মনে রাখো:** `chmod 755` লেখা আর `chmod 0755` লেখা একই কথা। প্রথম অঙ্ক না দিলে ধরে নেওয়া হয় সেটা `0`, মানে কোনো বিশেষ বিট নেই। (ডিরেক্টরির ক্ষেত্রে একটা ব্যতিক্রম আছে, ১৭.৬-এ দেখবে।)

১৭.২ Real UID আর Effective UID: SUID বোঝার আগে

লিনাক্সে প্রতিটি process (অধ্যায় ২৪) কয়েকটা পরিচয় সাথে নিয়ে চলে:

পরিচয়	মানে
Real UID (RUID)	যে ইউজার আসলে process-টা চালু করেছে
Effective UID (EUID)	পারমিশন চেক করার সময় kernel যে UID দেখে
Saved set-user-ID	EUID-এর পুরোনো মান জমা রাখার জায়গা, যাতে প্রোগ্রাম চাইলে পরে আবার ফিরে যেতে পারে

একইভাবে group-এর জন্যও Real GID, Effective GID আর saved GID আছে।

সাধারণত RUID আর EUID একই থাকে। তুমি (`nishat`, UID 1000) `cat` চালালে process-এর RUID=1000, EUID=1000। kernel কোনো ফাইল খোলার অনুমতি দেবে কি না, সেটা ঠিক করে **EUID** দেখে, RUID নয়।

উপমা: তুমি একটা অফিসে গেছ। গেটে তোমার নিজের নাম লেখা হলো (Real UID)। কিন্তু রিসেপশন তোমার গলায় একটা "স্টাফ" পাস ঝুলিয়ে দিল (Effective UID)। ভেতরের দরজাগুলো খোলে পাস দেখে, খাতার নাম দেখে না।

SUID বিটের পুরো খেলাটা এই EUID বদলানো নিয়েই।

১৭.৩ SUID (Set User ID)

SUID কী

কোনো executable ফাইলে SUID বিট দেওয়া থাকলে, ফাইলটা যে-ই চালাক, process-এর **Effective UID হয়ে যায় ফাইলের owner-এর UID**।

- octal মান: **4** (যেমন `4755`)
- symbolic: `u+s`
- `ls -l`-এ owner-এর execute ঘরে `x`-এর জায়গায় **`s`** দেখায়

সবচেয়ে বিখ্যাত উদাহরণ: `passwd`

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
```

(সাইজ আর তারিখ তোমার মেশিনে আলাদা হতে পারে।) owner-এর ঘরে `rws` দেখো। ওই `s` মানে SUID চালু আছে, আর execute বিটও আছে।

এবার `/etc/shadow` দেখো:

```
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1450 Oct  2 10:12 /etc/shadow
```

Ubuntu/Debian-এ এটা `root:shadow`, মোড `640`। Fedora/RHEL-এ সাধারণত `----------` (`000`) আর `root:root`। দুই ক্ষেত্রেই সাধারণ ইউজার এখানে লিখতে পারে না।

ভেতরে কী ঘটে

```
nishat (UID 1000) টার্মিনালে লিখল:  $ passwd
          │
          ▼
┌──────────────────────────────────────────────────┐
│ shell fork() করে, তারপর execve("/usr/bin/passwd") │
└──────────────────────────────────────────────────┘
          │  kernel ফাইলের inode দেখে: SUID বিট আছে, owner = root (UID 0)
          ▼
┌──────────────────────────────────────────────────┐
│ নতুন process:  RUID = 1000 (nishat)          │
│                EUID = 0    (root)  ← বদলে গেল│
└──────────────────────────────────────────────────┘
          │
          ▼
passwd নিজের কোডে getuid() দিয়ে দেখে "কে আসলে চালাল?" → 1000
তাই শুধু nishat-এর পাসওয়ার্ড বদলাতে দেয় (অন্যেরটা নয়)
          │
          ▼
/etc/shadow খোলার চেষ্টা → kernel EUID=0 দেখে → অনুমতি দেয় ✔
```

এখানে দুটো জিনিস একসাথে কাজ করছে:

1. **kernel** EUID বদলে দেয়, তাই ফাইলটা খোলা যায়।
2. **প্রোগ্রাম নিজে** RUID দেখে ঠিক করে কাকে কী করতে দেবে। তাই `nishat` চাইলেও `passwd alice` চালিয়ে alice-এর পাসওয়ার্ড বদলাতে পারে না। ওই নিয়ম `passwd`-এর কোডে লেখা আছে, kernel-এ নয়।

এ থেকেই বোঝা যায় SUID প্রোগ্রাম কেন বিপজ্জনক। পুরো নিরাপত্তা নির্ভর করছে প্রোগ্রামটা কতটা সাবধানে লেখা তার ওপর। প্রোগ্রামে একটা বাগ মানেই কেউ root হয়ে যেতে পারে।

নিজের চোখে দেখো

একটা টার্মিনালে `passwd` চালাও, পাসওয়ার্ড চাওয়ার জায়গায় থেমে থাকো। আরেকটা টার্মিনালে:

```
$ ps -o pid,ruser,euser,comm -C passwd
    PID RUSER    EUSER    COMM
   4821 nishat   root     passwd
```

RUSER `nishat`, কিন্তু EUSER `root`। প্রথম টার্মিনালে `Ctrl+C` চেপে `passwd` বন্ধ করে দাও।

আরও কিছু চেনা SUID প্রোগ্রাম

প্রোগ্রাম	কেন SUID লাগে
`/usr/bin/passwd`	`/etc/shadow` বদলাতে
`/usr/bin/sudo`	ইউজারকে root বা অন্য ইউজার হিসেবে কমান্ড চালাতে দিতে (অধ্যায় ১৮)
`/usr/bin/su`	অন্য ইউজারে switch করতে
`/usr/bin/chsh`, `/usr/bin/chfn`	`/etc/passwd`-এ নিজের shell বা নাম বদলাতে
`/usr/bin/newgrp`, `/usr/bin/gpasswd`	group সংক্রান্ত কাজে
`/usr/bin/mount`, `/usr/bin/umount`	`/etc/fstab`-এ `user` অপশন দেওয়া ডিস্ক সাধারণ ইউজারকে মাউন্ট করতে দিতে
`/usr/bin/fusermount3`	ইউজার-লেভেলে FUSE ফাইলসিস্টেম মাউন্ট করতে
`/usr/lib/dbus-1.0/dbus-daemon-launch-helper`	D-Bus সার্ভিস চালু করতে

ঠিক কোন কোন ফাইল SUID, সেটা distro আর ইনস্টল করা প্যাকেজভেদে আলাদা হয়।

SUID সেট করা ও তোলা

```
$ chmod u+s myprog        # symbolic
$ chmod 4755 myprog       # octal: 4 = SUID, 755 = rwxr-xr-x
$ chmod u-s myprog        # SUID তুলে দাও
```

SUID-এর কিছু জরুরি নিয়ম

1. **শেল স্ক্রিপ্টে SUID কাজ করে না।** লিনাক্স kernel `#!` দিয়ে শুরু হওয়া স্ক্রিপ্টের (bash, python ইত্যাদি) SUID বিট উপেক্ষা করে। কারণ স্ক্রিপ্ট চালু হওয়া আর interpreter-এর ফাইলটা পড়ার মাঝে একটা race

condition থাকে, তার ওপর environment variable দিয়েও interpreter-কে প্রভাবিত করা যায়। SUID শুধু compiled binary-তে (ELF) কাজ করে।

2. **ফাইলে কেউ লিখলে SUID চলে যায়।** root ছাড়া অন্য কেউ SUID ফাইলে write করলে kernel নিজে থেকেই SUID (আর group-execute থাকলে SGID) বিট মুছে দেয়। কেউ যাতে একটা SUID-root ফাইলের ভেতরে নিজের কোড বসাতে না পারে, সেজন্য এই ব্যবস্থা।
3. `chown` **করলেও SUID/SGID মুছে যায়।** owner বদলালে আগের SUID আর অর্থবহ থাকে না, তাই `chown` বিটটা পরিষ্কার করে দেয়। দরকার হলে `chown`-এর পরে আবার `chmod u+s` দাও।
4. `nosuid` **মাউন্ট অপশন।** কোনো ফাইলসিস্টেম `nosuid` দিয়ে মাউন্ট করা থাকলে (অধ্যায় ৩২) সেখানে SUID/SGID কাজ করে না। USB ড্রাইভ, `/tmp`, `/home`-এর মতো জায়গার জন্য এটা ভালো অভ্যাস।
5. **ডিরেক্টরিতে SUID-এর কোনো মানে নেই।** লিনাক্সে ডিরেক্টরিতে SUID বিট দিলে কোনো কাজ হয় না।

> ⚠️ **সতর্কতা:** নিজের বানানো কোনো প্রোগ্রামে বা সাধারণ টুলে (`vim`, `find`, `python3`, `bash`, `cp`) কখনো SUID-root দেবে না। এতে সিস্টেমের প্রত্যেক ইউজার সরাসরি root হয়ে যেতে পারবে। ১৭.৮-এ উদাহরণসহ দেখবে।

১৭.৪ SGID (Set Group ID)

SGID-এর আচরণ দুই রকম: ফাইলে এক রকম, ডিরেক্টরিতে আরেক রকম।

- octal মান: **2** (যেমন `2755`)
- symbolic: `g+s`
- `ls -l`-এ group-এর execute ঘরে `s` দেখায়

(ক) ফাইলে SGID

executable ফাইলে SGID থাকলে process-এর **Effective GID হয়ে যায় ফাইলের group**। SUID যা UID-এর জন্য করে, SGID সেটাই GID-এর জন্য করে।

উদাহরণ, `wall` (সব টার্মিনালে মেসেজ পাঠানোর কমান্ড):

```
$ ls -l /usr/bin/wall
-rwxr-sr-x 1 root tty 22912 Apr  9  2024 /usr/bin/wall
```

group-এর ঘরে `r-s`। টার্মিনাল ডিভাইসগুলো (`/dev/pts/*`) `tty` group-এর, আর ওই group-এর সেখানে write পারমিশন আছে:

```
$ ls -l /dev/pts/0
crw--w---- 1 nishat tty 136, 0 Nov 12 09:30 /dev/pts/0
```

তাই `wall` চালালে সে `tty` group-এর সদস্যের মতো আচরণ করে আর সবার টার্মিনালে লিখতে পারে, কিন্তু root হয় না। পুরো root ক্ষমতার বদলে শুধু একটা group-এর ক্ষমতা পায়। এটা **least privilege** (যতটুকু দরকার ঠিক ততটুকু ক্ষমতা) নীতির একটা ভালো উদাহরণ।

Debian/Ubuntu-তে `crontab` কমান্ডও এভাবে চলে:

```
$ ls -l /usr/bin/crontab
-rwxr-sr-x 1 root crontab 39664 Mar 31  2024 /usr/bin/crontab
```

এটা `crontab` group হয়ে `/var/spool/cron/crontabs/`-এ ইউজারের crontab ফাইল লেখে (অধ্যায় ২৯)। Fedora/RHEL-এ `crontab` এর বদলে SUID-root।

(খ) ডিরেক্টরিতে SGID: টিমের শেয়ার্ড ফোল্ডারের রহস্য

এটাই SGID-এর সবচেয়ে কাজের ব্যবহার।

সমস্যা: `/srv/project` ফোল্ডারে `dev` group-এর সবাই কাজ করে। alice একটা ফাইল বানালে লিনাক্সের সাধারণ নিয়মে ফাইলের group হয় alice-এর **primary group** (Ubuntu-তে সাধারণত `alice`)। ফলে bob (dev group-এর সদস্য) ফাইলটা group পারমিশন দিয়ে এডিট করতে পারে না।

সমাধান: ডিরেক্টরিতে SGID দাও। তখন:

1. ওই ডিরেক্টরিতে নতুন বানানো সব ফাইল আর সাব-ডিরেক্টরি **ডিরেক্টরির group** পায়, যে বানিয়েছে তার primary group নয়।
2. নতুন সাব-ডিরেক্টরিগুলো নিজেরাও SGID বিট পায়, তাই নিয়মটা গভীরে নামতে থাকে।

```
SGID ছাড়া:                          SGID সহ (chmod g+s /srv/project):
/srv/project  (group: dev)           /srv/project  (group: dev, drwxrwsr-x)
   └── alice.txt  group: alice ✗        └── alice.txt  group: dev ✔
   └── bob.txt    group: bob   ✗        └── bob.txt    group: dev ✔
                                        └── docs/      group: dev, SGID ✔
```

ধাপে ধাপে (root হিসেবে বা `sudo` দিয়ে):

```
$ sudo groupadd dev
$ sudo usermod -aG dev alice
$ sudo usermod -aG dev bob
$ sudo mkdir -p /srv/project
$ sudo chown root:dev /srv/project
$ sudo chmod 2775 /srv/project
$ ls -ld /srv/project
drwxrwsr-x 2 root dev 4096 Nov 12 10:00 /srv/project
```

এখন alice ফাইল বানালে:

```
$ touch /srv/project/plan.txt
$ ls -l /srv/project/plan.txt
```

```
-rw-rw-r-- 1 alice dev 0 Nov 12 10:05 /srv/project/plan.txt
```

group `dev`। bob-ও ফাইলটা এডিট করতে পারবে, তবে শর্ত হলো alice-এর `umask` `002` হতে হবে, যাতে ফাইলে group-write থাকে। Ubuntu-তে সাধারণ ইউজারের ডিফল্ট umask `002`, আর Fedora/RHEL-এও আধুনিক সংস্করণে ইউজারের নিজস্ব private group থাকলে `002`।

> 💡 **টিপস:** SGID ঠিক করে ফাইলের **group কোনটা হবে**। কিন্তু group-এর **পারমিশন কী হবে** সেটা ঠিক করে umask। ভরসাযোগ্যভাবে টিম-শেয়ারিং চাইলে SGID-এর সাথে default ACL (১৭.৯) ব্যবহার করো। ACL umask-এর ওপর নির্ভর করে না।

> 📝 **মনে রাখো:** group-এর সদস্যপদ বদলালে (`usermod -aG`) সেটা চালু হতে ইউজারকে log out করে আবার login করতে হয়, অথবা `newgrp dev` চালাতে হয়। `id` কমান্ড দিয়ে চেক করো (অধ্যায় ১৫)।

১৭.৫ Sticky Bit

সমস্যাটা

`/tmp` এমন জায়গা যেখানে সব ইউজার আর সব প্রোগ্রাম অস্থায়ী ফাইল রাখে। তাই এখানে সবার লেখার অনুমতি থাকতে হয় (`rwx` for others)। কিন্তু অধ্যায় ১৬-এর নিয়ম মনে করো: **কোনো ফাইল মোছা বা rename করার অনুমতি নির্ভর করে ডিরেক্টরির write পারমিশনের ওপর, ফাইলের পারমিশনের ওপর নয়।** তাহলে তো যে কেউ যে কারও ফাইল মুছে দিতে পারত!

সমাধান: sticky bit

ডিরেক্টরিতে sticky bit থাকলে সেখানকার কোনো ফাইল **মুছতে বা rename করতে** পারে শুধু:

- ফাইলের owner, অথবা
- ডিরেক্টরির owner, অথবা
- root (আরও নিখুঁতভাবে বললে, `CAP_FOWNER` ক্ষমতা আছে এমন process)

```
$ ls -ld /tmp
drwxrwxrwt 18 root root 4096 Nov 12 10:10 /tmp
```

others-এর ঘরে শেষ অক্ষর `t`। মোড `1777`।

- octal মান: **1**
- symbolic: `o+t` বা শুধু `+t`
- `ls -l`-এ others-এর execute ঘরে **`t`**

উপমা: অফিসের কমন ফ্রিজ। সবাই খাবার রাখতে পারে (write)। কিন্তু নিয়ম আছে, নিজের টিফিন বাক্স ছাড়া অন্যেরটা ফেলে দেওয়া যাবে না। ফ্রিজের মালিক (ডিরেক্টরির owner) আর অফিস ম্যানেজার (root) অবশ্য সব পরিষ্কার করতে পারে।

পরীক্ষা

```
alice$ echo "secret" > /tmp/alice.txt
alice$ chmod 666 /tmp/alice.txt          # সবাই লিখতে পারে এমন ফাইল

bob$ echo "hacked" >> /tmp/alice.txt      # ফাইলে লেখা গেল (ফাইলের পারমিশন 666)
bob$ rm /tmp/alice.txt
rm: cannot remove '/tmp/alice.txt': Operation not permitted
```

এখানে খেয়াল করার মতো একটা ব্যাপার আছে। sticky bit **ফাইলের ভেতরের লেখা রক্ষা করে না**, শুধু মোছা আর rename আটকায়। কন্টেন্ট রক্ষা হয় ফাইলের নিজের পারমিশন দিয়ে। আবার আধুনিক কার্নেলে `fs.protected_regular` চালু থাকলে (systemd-ভিত্তিক distro-তে সাধারণত থাকে) উপরের `>>` লাইনটাও sticky world-writable ডিরেক্টরিতে আটকে যেতে পারে। নিচে দেখো।

কার্নেলের বাড়তি সুরক্ষা

sticky world-writable ডিরেক্টরিতে (যেমন `/tmp`) কিছু পুরোনো আক্রমণ ঠেকাতে কার্নেলের কয়েকটা sysctl আছে:

```
$ sysctl fs.protected_symlinks fs.protected_hardlinks fs.protected_regular
fs.protected_symlinks = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
```

sysctl	কী করে
`fs.protected_symlinks`	sticky world-writable ডিরেক্টরিতে অন্য কারও বানানো symlink অনুসরণ করতে দেয় না (কিছু শর্ত সাপেক্ষে)
`fs.protected_hardlinks`	যে ফাইল তোমার নয় আর যেটায় তোমার read/write অনুমতি নেই, তার hard link বানাতে দেয় না
`fs.protected_regular`	sticky ডিরেক্টরিতে অন্যের আগে থেকে থাকা ফাইল `O_CREAT` দিয়ে খোলা আটকায়

মানগুলো distro-ভেদে আলাদা হতে পারে। বিস্তারিত অধ্যায় ৪৫ আর অধ্যায় ৪৭-এ।

ফাইলে sticky bit?

একেবারে পুরোনো UNIX-এ executable ফাইলে sticky bit দিলে প্রোগ্রামটা swap-এ "লেগে" (sticky) থাকত, তাই এই নাম। **আধুনিক লিনাক্স ফাইলের sticky bit পুরোপুরি উপেক্ষা করে।** এর ব্যবহার এখন শুধু ডিরেক্টরিতে।

```
$ chmod +t /srv/shared        # sticky দাও
$ chmod 1777 /srv/shared      # /tmp-এর মতো: সবাই লিখবে, কিন্তু নিজেরটা ছাড়া মুছবে না
$ chmod -t /srv/shared        # তুলে দাও
```

`/var/tmp`, আর কখনো কখনো `/dev/shm` আর `/dev/mqueue`-ও sticky বিট নিয়ে থাকে।

১৭.৬ চার অঙ্কের octal মোড: 4/2/1

তিনটা বিশেষ বিটের মানও rwx-এর মতো 4/2/1:

বিট	মান	symbolic	কোথায় দেখায়
SUID	4	`u+s`	owner-এর x ঘরে `s` / `S`
SGID	2	`g+s`	group-এর x ঘরে `s` / `S`
Sticky	1	`o+t` / `+t`	others-এর x ঘরে `t` / `T`

প্রথম অঙ্কটা এই মানগুলোর যোগফল:

মোড	মানে	বাস্তব উদাহরণ
`0755`	কোনো বিশেষ বিট নেই	সাধারণ প্রোগ্রাম
`4755`	SUID + rwxr-xr-x	`/usr/bin/passwd`
`2755`	SGID + rwxr-xr-x	`/usr/bin/wall`
`2775`	SGID + rwxrwxr-x	টিমের শেয়ার্ড ডিরেক্টরি
`1777`	sticky + rwxrwxrwx	`/tmp`
`3775`	SGID + sticky (2+1)	টিম ফোল্ডার যেখানে একে অন্যের ফাইল মুছতে পারবে না
`6755`	SUID + SGID (4+2)	দুটোই দরকার এমন বিরল প্রোগ্রাম

```
        chmod 2775 /srv/project
               | |||
               | |||└─ others = 5 = r-x
               | ||└── group  = 7 = rwx
               | |└─── owner  = 7 = rwx
               | └──── বিশেষ  = 2 = SGID
               └─────
```

`stat` দিয়ে চার অঙ্কেই মোড দেখা যায়:

```
$ stat -c '%a %A %U:%G %n' /usr/bin/passwd /tmp /srv/project
4755 -rwsr-xr-x root:root /usr/bin/passwd
1777 drwxrwxrwt root:root /tmp
2775 drwxrwsr-x root:dev /srv/project
```

ডিরেক্টরিতে numeric chmod-এর একটা ফাঁদ

GNU `chmod` (লিনাক্সে যেটা থাকে) ডিরেক্টরির SUID/SGID বিট রক্ষা করে, যদি না তুমি স্পষ্ট করে সেগুলো বদলাতে বলো। তাই:

```
$ ls -ld /srv/project
drwxrwsr-x 2 root dev 4096 Nov 12 10:00 /srv/project
$ sudo chmod 755 /srv/project
$ ls -ld /srv/project
drwxr-sr-x 2 root dev 4096 Nov 12 10:00 /srv/project      # SGID রয়ে গেছে!
```

SGID তুলতে চাইলে symbolic মোড সবচেয়ে নিরাপদ:

```
$ sudo chmod g-s /srv/project
```

numeric দিয়ে করতে চাইলে GNU coreutils-এর ডকুমেন্টেশন অনুযায়ী বাড়তি একটা শূন্য লাগে, যেমন `chmod 00755`। sticky বিট এই নিয়মের বাইরে, `chmod 755` দিলে সেটা উঠে যায়।

১৭.৭ `s` বনাম `S`, `t` বনাম `T`: বড় হাতের অক্ষর মানে সমস্যা

`ls -l` একটা ঘরে দুটো তথ্য দেখায়, execute বিট আছে কি না আর বিশেষ বিট আছে কি না:

ঘর	execute আছে	execute নেই
owner, SUID সহ	`s`	`S`
group, SGID সহ	`s`	`S`
others, sticky সহ	`t`	`T`
বিশেষ বিট নেই	`x`	`-`

```
$ touch demo
$ chmod 4644 demo
$ ls -l demo
-rwSr--r-- 1 nishat nishat 0 Nov 12 10:20 demo
```

বড় হাতের `S` মানে SUID দেওয়া আছে, কিন্তু owner-এর execute বিট নেই। ফাইলটা চালানোই যায় না, তাই SUID এখানে অর্থহীন। এটা সাধারণত ভুলের লক্ষণ।

```
$ chmod u+x demo
$ ls -l demo
-rwsr--r-- 1 nishat nishat 0 Nov 12 10:20 demo
```

একইভাবে `T` মানে sticky দেওয়া আছে, কিন্তু others-এর execute নেই। ডিরেক্টরিতে execute না থাকলে others ভেতরে ঢুকতেই পারে না, তাই এটাও প্রায় সবসময় ভুল।

একটা ব্যতিক্রম: **ডিরেক্টরিতে `S` (SGID, group execute ছাড়া)** দেখলে সেটা পুরো অর্থহীন নয়। নতুন ফাইলের group inherit করার নিয়ম তখনও খাটে। তবে group-এর সদস্যরা ভেতরে ঢুকতে পারবে না, তাই বাস্তবে এর ব্যবহার নেই। আর ফাইলে group-execute ছাড়া SGID দিলে কিছু পুরোনো সিস্টেমে সেটা mandatory locking বোঝাত। আধুনিক লিনাক্স কার্নেল (5.15 থেকে) mandatory locking সমর্থন বন্ধ করে দিয়েছে।

> 💡 **টিপস:** ছোট হাতের `s` / `t` = স্বাভাবিক। বড় হাতের `S` / `T` = "কোথাও গড়বড় আছে, চেক করো।"

১৭.৮ SUID/SGID ফাইল খোঁজা ও নিরাপত্তা ঝুঁকি

কেন খুঁজবে

প্রতিটি SUID-root প্রোগ্রাম সম্ভাব্য একটা দরজা। হ্যাকার সাধারণ ইউজার হিসেবে সিস্টেমে ঢুকলে প্রথম যে কাজগুলো করে, তার একটা হলো SUID ফাইলের তালিকা বের করা (**privilege escalation**, মানে ক্ষমতা বাড়ানোর চেষ্টা)। তাই সিস্টেম অ্যাডমিন হিসেবে তোমারও জানা থাকা দরকার কোন কোন ফাইল SUID, আর তালিকায় হঠাৎ নতুন কিছু যোগ হলো কি না।

`find` দিয়ে খোঁজা

`find`-এর `-perm` অপশন (অধ্যায় ১২) দিয়ে কাজটা হয়:

এক্সপ্রেশন	মানে
`-perm -4000`	SUID বিট **আছে** (অন্য বিট যা-ই থাকুক)
`-perm -2000`	SGID বিট আছে
`-perm -1000`	sticky বিট আছে
`-perm /6000`	SUID **অথবা** SGID, যেকোনো একটা আছে
`-perm -6000`	SUID **এবং** SGID দুটোই আছে
`-perm 4755`	মোড **হুবহু** 4755 (`-` বা `/` ছাড়া মানে exact match)

```
$ sudo find / -xdev -type f -perm -4000 2>/dev/null
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/umount
/usr/bin/fusermount3
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

```
/usr/lib/openssh/ssh-keysign
...
```

- `-xdev`: অন্য ফাইলসিস্টেমে (যেমন `/proc`, নেটওয়ার্ক ড্রাইভ) ঢুকবে না। প্রতিটি পার্টিশন আলাদা করে চেক করতে চাইলে এটা বাদ দাও।
- `2>/dev/null`: "Permission denied" জাতীয় error লুকায় (অধ্যায় ১৯)।

বিস্তারিত তালিকা দেখতে:

```
$ sudo find / -xdev -type f -perm /6000 -exec ls -l {} + 2>/dev/null
-rwxr-sr-x 1 root shadow   31312 Apr  5  2024 /usr/bin/chage
-rwsr-xr-x 1 root root     72792 May 30  2024 /usr/bin/chfn
-rwxr-sr-x 1 root crontab  39664 Mar 31  2024 /usr/bin/crontab
...
```

sticky ডিরেক্টরিগুলো দেখতে:

```
$ find / -xdev -type d -perm -1000 2>/dev/null
/tmp
/var/tmp
...
```

একটা baseline রাখো

ভালো অভ্যাস হলো একবার তালিকা বানিয়ে রাখা, তারপর মাঝে মাঝে মিলিয়ে দেখা:

```
$ sudo find / -xdev -type f -perm /6000 2>/dev/null | sort > ~/suid-baseline.txt
# কয়েক দিন পর:
$ sudo find / -xdev -type f -perm /6000 2>/dev/null | sort | diff ~/suid-baseline.txt -
> /home/bob/.cache/bash
```

`>` দিয়ে শুরু হওয়া লাইনগুলো নতুন। কারও home ডিরেক্টরিতে SUID-root `bash` পাওয়া মানে প্রায় নিশ্চিতভাবে সিস্টেম আক্রান্ত হয়েছে। প্যাকেজ আপডেটের পরেও তালিকা বদলাতে পারে, সেটা স্বাভাবিক।

> 💡 **টিপস:** `rkhunter`, `aide`, `lynis`-এর মতো টুল এই কাজ স্বয়ংক্রিয়ভাবে করে (অধ্যায় ৪৫)। কোনো ফাইল কোন প্যাকেজের, সেটা দেখতে Debian/Ubuntu-তে `dpkg -S /usr/bin/passwd`, Fedora/RHEL-এ `rpm -qf /usr/bin/passwd` (অধ্যায় ৩০)। কোনো প্যাকেজেরই নয় এমন SUID ফাইল সন্দেহজনক।

ঝুঁকি কোথায়: একটা বাস্তব উদাহরণ

ধরো কোনো অ্যাডমিন "সুবিধার জন্য" `find`-এ SUID দিয়ে দিয়েছে:

```
$ ls -l /usr/bin/find
```

```
-rwsr-xr-x 1 root root 224848 Apr  8  2024 /usr/bin/find
```

`find`-এর `-exec` অপশন যেকোনো কমান্ড চালাতে পারে। আর সেই কমান্ড চলবে EUID=0 নিয়ে:

```
nishat$ find . -exec /bin/sh -p \; -quit
# whoami
root
```

`-p` অপশন না দিলে `bash` আর `dash` বুঝতে পারে যে RUID আর EUID আলাদা, আর নিজে থেকে ক্ষমতা ছেড়ে দেয়। `-p` (privileged mode) সেই সুরক্ষা বন্ধ করে। এই একটা ভুলেই যেকোনো ইউজার root।

এ রকম শত শত প্রোগ্রাম আছে (`vim`, `less`, `python3`, `perl`, `awk`, `tar`, `cp`, `env` ...) যেগুলোতে SUID থাকলে root পাওয়া যায়। নিরাপত্তা গবেষকরা এগুলোর তালিকা রাখেন। সবচেয়ে পরিচিত তালিকা হলো **GTFOBins** প্রজেক্ট।

ঝুঁকি কমানোর নিয়ম

1. যে প্রোগ্রাম ব্যবহার করো না, তার SUID তুলে দাও। (তবে প্যাকেজ আপডেট হলে বিটটা আবার ফিরে আসতে পারে। Debian/Ubuntu-তে স্থায়ীভাবে বদলাতে `dpkg-statoverride` ব্যবহার করা হয়।)
2. সাধারণ টুলে কখনো SUID দেবে না। কাউকে নির্দিষ্ট একটা কমান্ড root হিসেবে চালাতে দিতে চাইলে `sudo`-র নিয়ম লেখো (অধ্যায় ১৮)।
3. পুরো root-এর বদলে শুধু একটা ক্ষমতা দরকার হলে **capabilities** ব্যবহার করো (১৭.১১)।
4. ইউজাররা লিখতে পারে এমন পার্টিশন (`/home`, `/tmp`, USB) `nosuid` দিয়ে মাউন্ট করো।
5. SUID ফাইলের তালিকা নিয়মিত মিলিয়ে দেখো।

১৭.৯ ACL (Access Control List): owner/group/others-এর বাইরে

সমস্যা

`report.txt` ফাইলটা তোমার (`nishat`), group `nishat`, মোড `640`। এখন:

- alice-কে পড়তে আর লিখতে দিতে চাও,
- bob-কে শুধু পড়তে দিতে চাও,
- আর কাউকে কিছুই না।

সাধারণ পারমিশনে মাত্র তিনটা ঘর (owner, একটা group, others)। এই কাজের জন্য নতুন group বানাতে হতো, আর প্রতিটি ফাইলের জন্য আলাদা মিশ্রণ চাইলে group-এর সংখ্যা বাড়তেই থাকত। এর সমাধান **ACL**: প্রতিটি ফাইলে যত খুশি ইউজার আর group-এর জন্য আলাদা আলাদা নিয়ম।

উপমা: সাধারণ পারমিশন যেন বাড়ির তিনটা চাবি: একটা মালিকের, একটা পরিবারের, একটা বাকি সবার। ACL হলো দারোয়ানের হাতে একটা লিস্ট: "রহিম ভাই ঢুকতে পারবে, করিম শুধু বসার ঘর পর্যন্ত।"

ভেতরে কোথায় থাকে

ACL inode-এর ১২টি বিটে আঁটে না। তাই এটা থাকে ফাইলের **extended attribute (xattr)**-এ, `system.posix_acl_access` আর (ডিরেক্টরির জন্য) `system.posix_acl_default` নামে। ext4, XFS, Btrfs, আর আধুনিক distro-র বেশিরভাগ ফাইলসিস্টেমে ACL ডিফল্টভাবে চালু থাকে। FAT/exFAT-এ ACL নেই।

`getfacl` / `setfacl` আসে `acl` প্যাকেজে। সাধারণত আগে থেকেই ইনস্টল থাকে, না থাকলে:

```
$ sudo apt install acl        # Debian/Ubuntu
$ sudo dnf install acl        # Fedora/RHEL
```

ACL entry-র ধরন

entry	লেখার ধরন	মানে
owner	`user::rw-`	ফাইলের owner (সাধারণ owner বিটের সমান)
named user	`user:alice:rw-`	নির্দিষ্ট একজন ইউজার
owning group	`group::r--`	ফাইলের group
named group	`group:dev:r-x`	নির্দিষ্ট একটা group
mask	`mask::rw-`	named user, named group আর owning group সর্বোচ্চ কতটা পাবে তার সীমা
others	`other::---`	বাকি সবাই

`getfacl`: ACL দেখা

ACL ছাড়া একটা ফাইল:

```
$ getfacl report.txt
# file: report.txt
# owner: nishat
# group: nishat
user::rw-
group::r--
other::---
```

এটা আসলে সাধারণ `640` পারমিশনকেই ACL-এর ভাষায় দেখানো।

`setfacl -m`: entry যোগ বা বদল (modify)

```
$ setfacl -m u:alice:rw report.txt
$ setfacl -m u:bob:r report.txt
$ getfacl report.txt
# file: report.txt
# owner: nishat
# group: nishat
user::rw-
```

```
user:alice:rw-
user:bob:r--
group::r--
mask::rw-
other::---
```

`u` = user, `g` = group, `m` = mask, `o` = other (পুরো শব্দ `user:`, `group:` লিখলেও চলে)। একসাথে একাধিক entry দিতে কমা দাও:

```
$ setfacl -m u:alice:rw,u:bob:r,g:dev:r report.txt
```

এখন `ls -l` দেখো:

```
$ ls -l report.txt
-rw-rw----+ 1 nishat nishat 120 Nov 12 11:00 report.txt
```

দুটো জিনিস খেয়াল করো:

1. শেষে `+` চিহ্ন। মানে এই ফাইলে ACL আছে, শুধু `ls -l` দেখে পুরো পারমিশন বোঝা যাবে না।
2. group-এর ঘরে এখন `rw-`, যদিও `group::` entry `r--`। **ACL থাকলে `ls -l`-এর group ঘর mask দেখায়**, owning group-এর পারমিশন নয়। এটা নতুনদের সবচেয়ে বড় বিভ্রান্তি।

mask: ACL-এর ছাঁকনি

mask হলো একটা উচ্চসীমা। named user, named group আর owning group, এদের প্রত্যেকের **কার্যকর (effective) পারমিশন = তার entry ∩ mask** (দুটোতেই যে বিট আছে, শুধু সেটুকু)। owner আর others-এর ওপর mask-এর প্রভাব নেই।

```
          entry       mask       effective
  alice:  rw-    ∩    r--    =   r--
  dev:    rwx    ∩    r--    =   r--
  owner:  rw-    (mask প্রযোজ্য নয়)  = rw-
  other:  ---    (mask প্রযোজ্য নয়)  = ---
```

```
$ setfacl -m m::r report.txt
$ getfacl report.txt
# file: report.txt
# owner: nishat
# group: nishat
user::rw-
user:alice:rw-                  #effective:r--
user:bob:r--
group::r--
mask::r--
other::---
```

`#effective:` মন্তব্য বলছে alice-এর entry-তে `rw-` লেখা থাকলেও সে এখন শুধু পড়তে পারবে।

mask নিয়ে তিনটা নিয়ম মনে রাখো:

- `setfacl -m` দিয়ে কিছু বদলালে mask **নিজে থেকে আবার হিসাব হয়**। সব named user, named group আর owning group-এর পারমিশন মিলিয়ে (union) নতুন mask বসে। তাই উপরে mask ম্যানুয়ালি `r` করার পর আবার কোনো `-m` চালালে mask আবার বড় হয়ে যেতে পারে। এটা চাইলে `-n` (`--no-mask`) অপশন দাও।
- ACL থাকা ফাইলে **`chmod g=...` দিলে mask বদলায়**, owning group-এর entry নয়। যেমন `chmod g-w report.txt` সব named entry থেকে write কার্যকরভাবে কেড়ে নেয়।
- তাই ACL থাকা ফাইলে `chmod 600` চালালে mask `---` হয়ে যায়, আর সব named user-এর প্রবেশ কার্যকরভাবে বন্ধ হয়ে যায়।

> ⚠️ **সতর্কতা:** ACL থাকা ফাইলে অভ্যাসবশে `chmod 644` বা `chmod 600` চালিয়ে দিলে ACL-এর পুরো হিসাব চুপচাপ বদলে যেতে পারে। ফাইলের শেষে `+` দেখলে আগে `getfacl` দেখো।

kernel কীভাবে ACL চেক করে

```
process ফাইল খুলতে চাইল
       │
       ▼
EUID = owner? ──হ্যাঁ──► user:: entry ব্যবহার (mask নয়)
       │না
       ▼
কোনো user:NAME: entry মেলে? ──হ্যাঁ──► সেই entry ∩ mask
       │না
       ▼
process-এর কোনো group owning group বা কোনো group:NAME:-এর সাথে মেলে?
       │ হ্যাঁ ──► মেলা entry-গুলোর যেকোনো একটা (∩ mask) অনুমতি দিলে ✔, নইলে ✘
       │না
       ▼
other:: entry ব্যবহার
```

প্রথম যে ধাপ মেলে, সিদ্ধান্ত সেখানেই হয়, পরের ধাপে যায় না। তাই কারও `user:` entry `---` হলে, তার group-এর অনুমতি থাকলেও সে ঢুকতে পারবে না।

`setfacl -x`: নির্দিষ্ট entry মুছে ফেলা

```
$ setfacl -x u:bob report.txt
$ setfacl -x g:dev report.txt
```

`-x`-এ পারমিশন লিখতে হয় না, শুধু কার entry সেটা।

`setfacl -b`: সব ACL মুছে ফেলা

```
$ setfacl -b report.txt
$ ls -l report.txt
-rw-r-----  1 nishat nishat 120 Nov 12 11:00 report.txt
```

`+` চলে গেছে। ফাইল আবার সাধারণ পারমিশনে ফিরেছে। (`-b` = remove all extended ACL entries; owner/group/other থেকে যায়।)

Default ACL (`-d`): ডিরেক্টরিতে উত্তরাধিকার

ডিরেক্টরিতে **default ACL** দিলে ভেতরে নতুন বানানো প্রতিটি ফাইল আর সাব-ডিরেক্টরি সেই ACL নিজের ACL হিসেবে পায়। SGID যেমন group inherit করায়, default ACL তেমনি পারমিশন inherit করায়।

```
$ sudo setfacl -d -m g:dev:rwX /srv/project
$ sudo setfacl -m g:dev:rwX /srv/project       # ডিরেক্টরির নিজের জন্যও
$ getfacl /srv/project
# file: srv/project
# owner: root
# group: dev
# flags: -s-
user::rwx
group::rwx
group:dev:rwx
mask::rwx
other::r-x
default:user::rwx
default:group::rwx
default:group:dev:rwx
default:mask::rwx
default:other::r-x
```

- `default:` দিয়ে শুরু হওয়া লাইনগুলো default ACL।
- `# flags: -s-` মানে ডিরেক্টরিতে SGID আছে (তিনটা অক্ষর SUID, SGID, sticky ক্রমে)।
- getfacl লেখার সময় path-এর শুরুর `/` বাদ দেয়, তাই `srv/project` দেখাচ্ছে।
- **বড় হাতের `X`**: execute দেবে শুধু ডিরেক্টরিতে, অথবা কোনো ফাইলে আগে থেকেই কারও execute থাকলে। তাই সাধারণ টেক্সট ফাইল ভুল করে executable হয়ে যায় না।

এখন যে কেউ এখানে ফাইল বানালে:

```
alice$ touch /srv/project/notes.txt
alice$ getfacl /srv/project/notes.txt
# file: srv/project/notes.txt
# owner: alice
# group: dev
user::rw-
group::rwx                      #effective:rw-
group:dev:rwx                   #effective:rw-
```

```
mask::rw-
other::r--
```

`touch` ফাইলটা `rw-` দিয়ে বানানোর অনুরোধ করে (mode 666), তাই default ACL-এর `x`, mask দিয়ে ছেঁকে বাদ পড়েছে। মূল কথা: **default ACL থাকলে umask ব্যবহার হয় না**। পারমিশন আসে default ACL আর প্রোগ্রাম যে মোড চেয়েছে, এই দুটো থেকে।

default ACL মুছতে:

```
$ sudo setfacl -k /srv/project       # শুধু default ACL মুছবে
```

`-R`: রিকার্সিভ

আগে থেকে থাকা সব ফাইলে একসাথে ACL দিতে:

```
$ sudo setfacl -R -m g:dev:rwX /srv/project
```

`-R` শুধু **এখন যা আছে** তাতে ACL দেয়। ভবিষ্যতের ফাইলের জন্য `-d` লাগবে। তাই দুটো সাধারণত একসাথে চলে।

ACL-এর ব্যাকআপ ও রিস্টোর

```
$ getfacl -R /srv/project > project-acl.txt       # পুরো ট্রির ACL টেক্সট ফাইলে
$ sudo setfacl --restore=project-acl.txt          # হুবহু ফিরিয়ে আনো
```

একটা ফাইলের ACL আরেকটায় কপি করতে:

```
$ getfacl source.txt | setfacl --set-file=- target.txt
```

ফাইল কপি করলে ACL কি থাকে?

কমান্ড	ACL রাখে?
`cp file new`	না, সাধারণত হারায়
`cp -a` বা `cp --preserve=all`	হ্যাঁ
`mv` (একই ফাইলসিস্টেমে)	হ্যাঁ (inode একই থাকে)
`tar`	`--acls` দিলে (অধ্যায় ৩৪)
`rsync`	`-A` দিলে (অধ্যায় ৩৫)

`setfacl`-এর অপশন সারণি

অপশন	কাজ
`-m spec`	entry যোগ বা বদল
`-x spec`	নির্দিষ্ট entry মুছে ফেলা
`-b`	সব extended ACL মুছে ফেলা
`-k`	শুধু default ACL মুছে ফেলা
`-d`	পরের `-m` / `-x` default ACL-এ প্রযোজ্য হবে
`-R`	রিকার্সিভ
`-n`	mask নতুন করে হিসাব করবে না
`--set=spec`	পুরো ACL বদলে দেবে (আগেরটা মুছে)
`--restore=file`	`getfacl -R`-এর আউটপুট থেকে ফিরিয়ে আনবে
`--test`	আসলে বদলাবে না, ফলাফল কী হবে শুধু দেখাবে

> 💡 **টিপস:** কোন ফাইলগুলোতে ACL আছে খুঁজতে `getfacl -R -s -p /srv 2>/dev/null | grep '^# file'`। এখানে `-s` (skip base) শুধু ACL থাকা ফাইল দেখায়।

১৭.১০ ফাইল অ্যাট্রিবিউট: `chattr` ও `lsattr`

পারমিশন আর ACL বলে "**কে** কী করতে পারবে"। ফাইল অ্যাট্রিবিউট বলে "**কেউই** কী করতে পারবে না"। root-ও না, যতক্ষণ না অ্যাট্রিবিউটটা তুলে নেওয়া হয়।

এগুলো ফাইলসিস্টেম-লেভেলের ফ্ল্যাগ। মূলত ext2/3/4-এর জন্য তৈরি। XFS আর Btrfs-ও কিছু অ্যাট্রিবিউট (যেমন `i`, `a`) সমর্থন করে, তবে সবগুলো নয়। টুল দুটো আসে `e2fsprogs` প্যাকেজে, যেটা প্রায় সব distro-তে ইনস্টল থাকে।

`lsattr`: অ্যাট্রিবিউট দেখা

```
$ lsattr notes.txt
--------------e------- notes.txt
```

ext4-এ `e` (extents) প্রায় সব ফাইলেই দেখবে। এটা ফাইলসিস্টেম নিজে বসায়, এ নিয়ে মাথা ঘামানোর দরকার নেই। ড্যাশের সংখ্যা `e2fsprogs`-এর version-ভেদে আলাদা হতে পারে।

অপশন	কাজ
`lsattr file`	ফাইলের অ্যাট্রিবিউট

অপশন	কাজ
`lsattr -d dir`	ডিরেক্টরির নিজের অ্যাট্রিবিউট (ভেতরের ফাইলের নয়)
`lsattr -R dir`	রিকার্সিভ
`lsattr -a`	লুকানো (`.` দিয়ে শুরু) ফাইলও
`lsattr -l`	অক্ষরের বদলে পুরো নাম দেখাও

`chattr`: অ্যাট্রিবিউট বসানো ও তোলা

```
chattr [+|-|=]অ্যাট্রিবিউট ফাইল...
```

`+` যোগ করে, `-` তুলে নেয়, `=` হুবহু ওইটুকু বসায়। `-R` দিলে রিকার্সিভ। `i` আর `a` বসানো বা তোলার জন্য root লাগে (আরও নিখুঁতভাবে, `CAP_LINUX_IMMUTABLE` ক্ষমতা)।

`+i`: Immutable (অপরিবর্তনীয়)

```
$ sudo chattr +i important.conf
$ lsattr important.conf
----i---------e------- important.conf

$ sudo rm important.conf
rm: cannot remove 'important.conf': Operation not permitted
$ sudo sh -c 'echo test >> important.conf'
sh: 1: cannot create important.conf: Operation not permitted
$ sudo mv important.conf x.conf
mv: cannot move 'important.conf' to 'x.conf': Operation not permitted
$ sudo chmod 777 important.conf
chmod: changing permissions of 'important.conf': Operation not permitted
```

immutable ফাইলে:

- লেখা যায় না, মোছা যায় না, rename করা যায় না
- hard link বানানো যায় না
- পারমিশন, owner, ACL বদলানো যায় না
- root-ও পারে না, যতক্ষণ না `chattr -i` দেওয়া হয়

ডিরেক্টরিতে `+i` দিলে ভেতরে নতুন ফাইল বানানো, মোছা বা rename করা যায় না। তবে ভেতরের আগে থেকে থাকা ফাইলের কন্টেন্ট বদলানো যায় (ফাইলগুলোর নিজের `+i` না থাকলে)।

তুলে নিতে:

```
$ sudo chattr -i important.conf
```

কোথায় কাজে লাগে: গুরুত্বপূর্ণ কনফিগ ফাইল (যেমন হাতে লেখা `/etc/resolv.conf`, যেটা অন্য কোনো সার্ভিস যেন বদলে না দেয়), ভুল করে `rm -rf` থেকে কিছু বাঁচানো।

> ⚠️ **সতর্কতা:** হ্যাকাররাও `chattr +i` ব্যবহার করে, যাতে তাদের বসানো ফাইল (যেমন `~/.ssh/authorized_keys`-এ ঢোকানো key) অ্যাডমিন মুছতে না পারে। কোনো ফাইল root হয়েও মুছতে না পারলে প্রথমে `lsattr` দেখো। আর মনে রেখো, immutable ফাইল প্যাকেজ আপডেটকেও আটকায়। `/etc`-এর কোনো ফাইলে `+i` দিয়ে ভুলে গেলে পরে `apt` / `dnf` আপডেট ব্যর্থ হতে পারে।

`+a`: Append-only (শুধু শেষে যোগ করা যাবে)

```
$ sudo chattr +a audit.log
$ echo "new entry" | sudo tee -a audit.log      # >> (append) কাজ করে
new entry
$ echo "overwrite" | sudo tee audit.log          # > (overwrite) কাজ করে না
tee: audit.log: Operation not permitted
$ sudo rm audit.log
rm: cannot remove 'audit.log': Operation not permitted
```

আগের লেখা মুছে ফেলা বা বদলানো যায় না, শুধু শেষে নতুন লাইন যোগ করা যায়। **লগ ফাইলের জন্য আদর্শ।** কোনো আক্রমণকারী root পেলেও নিজের চিহ্ন মুছতে পারবে না, যদি না আগে `chattr -a` চালায়। তবে লগ rotate করার টুলের (logrotate, অধ্যায় ২৮) সাথে এটা ঝামেলা করতে পারে, তাই বুঝে ব্যবহার করো।

অন্য অ্যাট্রিবিউটগুলো

অ্যাট্রিবিউট	নাম	কাজ	মন্তব্য
`i`	immutable	কোনো পরিবর্তন নয়	root লাগে
`a`	append only	শুধু শেষে যোগ	root লাগে
`A`	no atime	access time আপডেট হবে না	disk I/O কমায়
`d`	no dump	পুরোনো `dump` ব্যাকআপ টুল এই ফাইল বাদ দেবে	
`S`	synchronous	প্রতিটি লেখা সাথে সাথে ডিস্কে যাবে	ধীর, কিন্তু নিরাপদ
`C`	no copy-on-write	Btrfs-এ CoW বন্ধ করে	VM image ও ডাটাবেস ফাইলে কাজে লাগে; খালি ফাইল বা ডিরেক্টরিতে দিতে হয়
`c`	compressed	ফাইলটা কম্প্রেস করে রাখবে	Btrfs-এ কাজ করে, ext4-এ নয়
`e`	extents	ফাইল extents ব্যবহার করে	ext4 নিজে বসায়; হাতে বদলানোর দরকার নেই
`s`, `u`	secure delete, undelete		**ext4 এগুলো মানে না**। ম্যান পেজেও এটা লেখা আছে, এদের ওপর ভরসা কোরো না

১৭.১১ Linux Capabilities: root-এর ক্ষমতা টুকরো টুকরো করা

সমস্যা

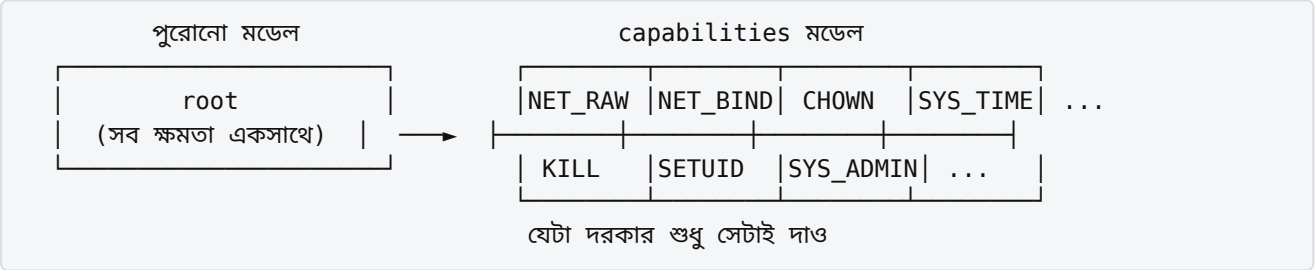
পুরোনো UNIX মডেলে দুই রকম process: root (UID 0, সব পারে) আর বাকিরা (প্রায় কিছুই পারে না বিশেষ)। কিন্তু অনেক প্রোগ্রামের দরকার মাত্র **একটা** বিশেষ ক্ষমতা:

- `ping`-এর দরকার raw network socket খোলা
- একটা ওয়েব সার্ভারের দরকার ১০২৪-এর নিচের পোর্ট (৮০, ৪৪৩) bind করা

এর জন্য পুরো root (SUID-root) দেওয়া মানে একটা ছোট কাজের জন্য গোটা বাড়ির চাবি দিয়ে দেওয়া।

সমাধান

লিনাক্স কার্নেল root-এর ক্ষমতাকে ৪০টার মতো আলাদা **capability**-তে ভাগ করেছে। এখন একটা process-কে শুধু দরকারি ক্ষমতাটুকু দেওয়া যায়।



কিছু গুরুত্বপূর্ণ capability

capability	কী অনুমতি দেয়
`CAP_NET_BIND_SERVICE`	১০২৪-এর নিচের পোর্টে bind করা
`CAP_NET_RAW`	raw socket (যেমন `ping`, packet capture)
`CAP_NET_ADMIN`	নেটওয়ার্ক কনফিগার (IP, routing, firewall)
`CAP_CHOWN`	যেকোনো ফাইলের owner বদলানো
`CAP_DAC_OVERRIDE`	ফাইলের read/write/execute পারমিশন চেক এড়িয়ে যাওয়া
`CAP_DAC_READ_SEARCH`	যেকোনো ফাইল পড়া আর ডিরেক্টরি ঘাঁটা
`CAP_FOWNER`	owner না হয়েও owner-এর মতো কিছু কাজ (যেমন sticky ডিরেক্টরিতে মোছা)
`CAP_KILL`	অন্যের process-এ signal পাঠানো
`CAP_SETUID` / `CAP_SETGID`	যেকোনো UID/GID ধারণ করা
`CAP_LINUX_IMMUTABLE`	`chattr +i` / `+a` বসানো ও তোলা
`CAP_SYS_TIME`	সিস্টেম ঘড়ি বদলানো
`CAP_SYS_ADMIN`	বিশাল একটা ঝুড়ি: mount, অনেক admin কাজ। কার্যত প্রায় root

পুরো তালিকা: `man 7 capabilities`।

file capability-র তিনটা ফ্ল্যাগ

ফাইলে capability বসানোর সময় তিনটা ফ্ল্যাগের কথা আসে:

ফ্ল্যাগ	নাম	মানে
`p`	Permitted	process এই ক্ষমতা পেতে পারে
`e`	Effective	চালু হওয়ার সাথে সাথে ক্ষমতাটা সক্রিয় থাকবে
`i`	Inheritable	exec-এর পরেও কিছু শর্তে উত্তরাধিকার হবে

সাধারণ প্রোগ্রামের জন্য প্রায় সবসময় `=ep` (বা `+ep`) লেখা হয়।

ACL-এর মতো file capability-ও extended attribute-এ (`security.capability`) থাকে। ফাইলে লিখলে সেটা মুছে যায়, আর `nosuid` মাউন্টে file capability-ও কাজ করে না।

`getcap`: capability দেখা

```
$ getcap /usr/bin/ping
/usr/bin/ping cap_net_raw=ep
```

কিছু distro-তে (যেমন Debian/Ubuntu) `ping`-এ এভাবে `cap_net_raw` দেওয়া থাকে। Fedora-র মতো কিছু distro-তে এর দরকার হয় না, কারণ সেখানে `net.ipv4.ping_group_range` sysctl দিয়ে সাধারণ ইউজারকে ICMP "ping socket" খোলার অনুমতি দেওয়া থাকে। তাই তোমার সিস্টেমে আউটপুট খালি হলেও অবাক হয়ো না।

পুরো সিস্টেমে capability-যুক্ত ফাইল খুঁজতে:

```
$ getcap -r / 2>/dev/null
/usr/bin/ping cap_net_raw=ep
/usr/bin/mtr-packet cap_net_raw=ep
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper
cap_net_bind_service,cap_net_admin,cap_sys_nice=ep
```

(তালিকা ইনস্টল করা প্যাকেজের ওপর নির্ভর করবে।) SUID তালিকার মতো এটাও নিরাপত্তা অডিটে দেখা উচিত।

`setcap`: capability বসানো

`setcap` আসে Debian/Ubuntu-তে `libcap2-bin`, Fedora/RHEL-এ `libcap` প্যাকেজে।

উদাহরণ: একটা প্রোগ্রামকে root ছাড়াই পোর্ট ৮০-তে চালাতে চাই। `setcap` symlink-এ কাজ করে না, আসল ফাইলে করতে হয়। আর সিস্টেমের `python3`-এ এটা দিলে সব python স্ক্রিপ্টই ক্ষমতাটা পেয়ে যেত, তাই একটা কপিতে পরীক্ষা করি:

```
$ cp "$(readlink -f /usr/bin/python3)" ~/mypython
$ ~/mypython -m http.server 80
PermissionError: [Errno 13] Permission denied

$ sudo setcap cap_net_bind_service=+ep ~/mypython
$ getcap ~/mypython
/home/nishat/mypython cap_net_bind_service=ep
$ ~/mypython -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

`Ctrl+C` দিয়ে সার্ভার বন্ধ করো। তারপর capability তুলে দাও আর কপিটা মুছে ফেলো:

```
$ sudo setcap -r ~/mypython
$ rm ~/mypython
```

কমান্ড	কাজ
`getcap file`	ফাইলের capability দেখাও
`getcap -r dir`	রিকার্সিভ খোঁজো
`setcap cap_x=+ep file`	capability বসাও
`setcap cap_a,cap_b=+ep file`	একাধিক একসাথে
`setcap -r file`	সব capability মুছে দাও
`capsh --print`	বর্তমান shell-এর capability দেখাও
`grep Cap /proc/$$/status`	process-এর capability (hex আকারে)
`capsh --decode=HEX`	hex মানটা নামে রূপান্তর

```
$ grep CapEff /proc/$$/status
CapEff: 0000000000000000
```

সাধারণ ইউজারের shell-এ effective capability শূন্য।

> ⚠️ **সতর্কতা:** capability-ও বিপজ্জনক হতে পারে। `cap_setuid` দেওয়া `python3`, `cap_dac_override` দেওয়া এডিটর, বা `cap_sys_admin` দেওয়া যেকোনো কিছু মানে কার্যত root। SUID-এর মতো এখানেও একই নিয়ম: সাধারণ interpreter বা টুলে capability দেবে না।

> 💡 **টিপস:** systemd সার্ভিসে `AmbientCapabilities=CAP_NET_BIND_SERVICE` লিখে ফাইলে হাত না দিয়েই সার্ভিসকে capability দেওয়া যায় (অধ্যায় ২৬)। আর শুধু নিচের পোর্ট খোলার জন্য আরেক উপায় হলো `net.ipv4.ip_unprivileged_port_start` sysctl। container-এর জগতে (Docker, Podman) capability-ই মূল নিরাপত্তা হাতিয়ার (অধ্যায় ৪৮)।

১৭.১২ কোনটা কখন: তুলনা

প্রয়োজন	ব্যবহার করো
সাধারণ ইউজার নির্দিষ্ট একটা প্রোগ্রাম দিয়ে root-এর ফাইল বদলাবে	SUID (শুধু ভালোভাবে লেখা, বিশ্বস্ত binary-তে)
প্রোগ্রাম শুধু একটা group-এর ক্ষমতা পাবে	SGID (ফাইলে)
টিম ফোল্ডারের সব ফাইল একই group পাবে	SGID (ডিরেক্টরিতে)
সবাই লিখবে, কিন্তু অন্যের ফাইল মুছবে না	sticky bit
নির্দিষ্ট কয়েকজনকে আলাদা আলাদা অনুমতি	ACL
নতুন ফাইলেও স্বয়ংক্রিয়ভাবে একই অনুমতি	default ACL
কেউই (root-ও) যেন বদলাতে না পারে	`chattr +i`
লগে শুধু যোগ করা যাবে	`chattr +a`
প্রোগ্রামের শুধু একটা root-ক্ষমতা দরকার	capabilities
ইউজার নির্দিষ্ট কমান্ড root হিসেবে চালাবে	`sudo` (অধ্যায় ১৮)

প্রাসঙ্গিক কীবোর্ড ও নোটেশন শর্টকাট

এই অধ্যায়ের নিজস্ব কোনো কীবোর্ড শর্টকাট নেই, তবে কাজের গতি বাড়ায় এমন কয়েকটা জিনিস:

শর্টকাট	কাজ
`Tab`	`setfacl`, `chattr`-এর পরে ফাইলের নাম auto-complete
`Alt+.`	আগের কমান্ডের শেষ argument (যেমন একই ফাইলে `chattr` তারপর `lsattr`) (অধ্যায় ৬)
`sudo !!`	শেষ কমান্ডটা sudo দিয়ে আবার চালাও (যখন "Operation not permitted" আসে)
`u+s` / `g+s` / `+t`	SUID / SGID / sticky-র symbolic রূপ
`u+s,g+s`	কমা দিয়ে একসাথে একাধিক পরিবর্তন
`X` (বড় হাতের)	শুধু ডিরেক্টরি বা আগে থেকেই executable ফাইলে execute
`u:`, `g:`, `m:`, `o:`	setfacl-এ `user:`, `group:`, `mask:`, `other:`-এর সংক্ষিপ্ত রূপ
`d:`	setfacl spec-এর শুরুতে দিলে `-d`-এর সমান (যেমন `setfacl -m d:u:alice:rw dir`)

হাতে-কলমে অনুশীলন

একটা VM বা টেস্ট মেশিনে করো। `sudo` লাগবে। (sudo নিয়ে বিস্তারিত অধ্যায় ১৮-এ।)

ল্যাব ১: SUID দেখা

```
$ ls -l /usr/bin/passwd /usr/bin/sudo
$ stat -c '%a %n' /usr/bin/passwd
```

1. owner ঘরে `s` আছে কি না দেখো, octal মান `4755` কি না দেখো।
2. একটা টার্মিনালে `passwd` চালিয়ে থেমে থাকো, অন্যটায় `ps -o pid,ruser,euser,comm -C passwd` দিয়ে RUSER আর EUSER মেলাও। তারপর `Ctrl+C`।

ল্যাব ২: `s` আর `S`

```
$ cd ~ && touch lab2
$ chmod 4644 lab2 && ls -l lab2        # rwS দেখবে
$ chmod u+x lab2 && ls -l lab2         # rws দেখবে
$ chmod 2644 lab2 && ls -l lab2        # group ঘরে S
$ rm lab2
```

ল্যাব ৩: টিম ফোল্ডার (SGID + sticky)

```
$ sudo groupadd labteam
$ sudo useradd -m -G labteam ali
$ sudo useradd -m -G labteam bela
$ sudo mkdir /srv/labteam
$ sudo chown root:labteam /srv/labteam
$ sudo chmod 3775 /srv/labteam
$ ls -ld /srv/labteam
drwxrwsr-t 2 root labteam 4096 Nov 12 12:00 /srv/labteam
```

```
$ sudo -u ali touch /srv/labteam/ali.txt
$ ls -l /srv/labteam
-rw-r--r-- 1 ali labteam 0 Nov 12 12:01 ali.txt
```

group `labteam` হয়েছে (SGID-এর কাজ)। এখন bela মোছার চেষ্টা করুক:

```
$ sudo -u bela rm /srv/labteam/ali.txt
rm: cannot remove '/srv/labteam/ali.txt': Operation not permitted
```

এটা sticky-র কাজ। (`sudo -u` দিয়ে চালানো কমান্ডে umask সাধারণত root-এর `022` থেকে আসে, তাই ফাইলে group-write নেই। পরের ল্যাবে ACL দিয়ে এটা ঠিক করব।)

ল্যাব ৪: ACL

আগে sticky বিটটা তুলে দাও। কারণ sticky আর group-writable ডিরেক্টরিতে `fs.protected_regular = 2` থাকলে kernel অন্যের ফাইল `>>` দিয়ে খোলাও আটকে দেয় (১৭.৫ দেখো), তখন ACL-এর প্রভাব আলাদা করে বোঝা যাবে না।

```
$ sudo chmod -t /srv/labteam
$ sudo setfacl -m g:labteam:rwX /srv/labteam
$ sudo setfacl -d -m g:labteam:rwX /srv/labteam
$ getfacl /srv/labteam
$ sudo -u ali touch /srv/labteam/shared.txt
$ getfacl /srv/labteam/shared.txt        # group:labteam:rw- দেখবে
$ sudo -u bela sh -c 'echo hi >> /srv/labteam/shared.txt' && cat
/srv/labteam/shared.txt
hi
```

এখন mask নিয়ে খেলো:

```
$ sudo setfacl -m m::r /srv/labteam/shared.txt
$ getfacl /srv/labteam/shared.txt        # #effective:r-- দেখবে
$ sudo -u bela sh -c 'echo again >> /srv/labteam/shared.txt'
sh: 1: cannot create /srv/labteam/shared.txt: Permission denied
```

ব্যাকআপ নিয়ে সব মুছে আবার ফিরিয়ে আনো:

```
$ getfacl -R /srv/labteam > /tmp/labacl.txt
$ sudo setfacl -R -b /srv/labteam
$ ls -l /srv/labteam                     # + নেই
$ cd / && sudo setfacl --restore=/tmp/labacl.txt
$ ls -l /srv/labteam                     # + ফিরে এসেছে
```

(`--restore` ফাইলের relative path ব্যবহার করে, তাই `/` থেকে চালাতে হয়।)

ল্যাব ৫: chattr

```
$ echo "do not touch" > ~/lab5.txt
$ sudo chattr +i ~/lab5.txt
$ lsattr ~/lab5.txt
$ rm -f ~/lab5.txt                       # Operation not permitted
$ sudo chattr -i ~/lab5.txt
$ sudo chattr +a ~/lab5.txt
$ echo "line 2" >> ~/lab5.txt            # কাজ করবে
$ echo "replace" > ~/lab5.txt            # Operation not permitted
$ sudo chattr -a ~/lab5.txt && rm ~/lab5.txt
```

ল্যাব ৬: SUID ও capability অডিট

```
$ sudo find / -xdev -type f -perm /6000 2>/dev/null | sort > ~/suid-baseline.txt
$ wc -l ~/suid-baseline.txt
$ getcap -r / 2>/dev/null
```

তালিকার তিনটা ফাইল বেছে নাও, আর `dpkg -S` বা `rpm -qf` দিয়ে দেখো কোন প্যাকেজের। প্রতিটি কেন SUID/SGID, সেটা ব্যাখ্যা করার চেষ্টা করো।

পরিষ্কার করা

```
$ sudo rm -rf /srv/labteam
$ sudo userdel -r ali; sudo userdel -r bela
$ sudo groupdel labteam
```

সারসংক্ষেপ

কমান্ড / নোটেশন	কাজ
`chmod u+s f` / `chmod 4755 f`	SUID বসাও
`chmod g+s d` / `chmod 2775 d`	SGID বসাও
`chmod +t d` / `chmod 1777 d`	sticky বসাও
`chmod u-s,g-s f`	SUID/SGID তুলে দাও (ডিরেক্টরিতে এটাই নিরাপদ পথ)
`s` / `S`	SUID বা SGID, execute সহ / execute ছাড়া
`t` / `T`	sticky, others-execute সহ / ছাড়া
`stat -c '%a %A %n' f`	চার অঙ্কের মোড দেখাও
`find / -xdev -type f -perm -4000`	SUID ফাইল খোঁজো
`find / -xdev -type f -perm /6000`	SUID বা SGID খোঁজো
`getfacl f`	ACL দেখাও
`setfacl -m u:alice:rw f`	ইউজারকে অনুমতি দাও
`setfacl -m g:dev:rX f`	group-কে অনুমতি দাও
`setfacl -m m::r f`	mask বদলাও
`setfacl -x u:alice f`	entry মুছে দাও
`setfacl -b f`	সব ACL মুছে দাও
`setfacl -d -m g:dev:rwX d`	default ACL (inherit)
`setfacl -k d`	default ACL মুছে দাও
`setfacl -R ...`	রিকার্সিভ
`getfacl -R d > bk` / `setfacl --restore=bk`	ACL ব্যাকআপ ও রিস্টোর
`lsattr f` / `lsattr -d d`	অ্যাট্রিবিউট দেখাও
`chattr +i f` / `chattr -i f`	immutable চালু / বন্ধ
`chattr +a f`	append-only
`getcap f` / `getcap -r /`	capability দেখাও / খোঁজো
`setcap cap_net_bind_service=+ep f`	capability বসাও

কমান্ড / নোটেশন	কাজ
`setcap -r f`	capability মুছে দাও
`capsh --print`	বর্তমান capability

নিজেকে যাচাই করো

1. সাধারণ ইউজার `/etc/shadow`-এ লিখতে পারে না, তবু `passwd` দিয়ে পাসওয়ার্ড বদলাতে পারে কেন? এখানে Real UID আর Effective UID-এর ভূমিকা কী?
2. `-rwSr--r--`-এর বড় হাতের `S` কী বোঝায়? এটা কি কাজের?
3. একটা ডিরেক্টরিতে SGID দিলে নতুন ফাইল আর নতুন সাব-ডিরেক্টরির ওপর কী প্রভাব পড়ে?
4. `/tmp`-এর মোড কত, আর ওখানে sticky bit না থাকলে কী সমস্যা হতো?
5. `chmod 6755` আর `chmod 3775`-এ কোন কোন বিশেষ বিট আছে?
6. একটা bash স্ক্রিপ্টে `chmod u+s` দিলে কি সেটা root হিসেবে চলবে? কেন?
7. একটা ফাইলের `ls -l` আউটপুট `-rw-rw----+`। শেষের `+` কী বোঝায়, আর group ঘরের `rw-` আসলে কী দেখাচ্ছে?
8. ACL থাকা একটা ফাইলে `user:alice:rw-` আর `mask::r--`। alice কী করতে পারবে?
9. `setfacl -x`, `-b` আর `-k`-এর পার্থক্য কী?
10. root হয়েও একটা ফাইল মোছা যাচ্ছে না, "Operation not permitted" আসছে। প্রথমে কী চেক করবে? আর ওয়েব সার্ভারকে পোর্ট ৮০ দিতে SUID-root-এর চেয়ে ভালো উপায় কী?

▼ উত্তর দেখো

1. `/usr/bin/passwd`-এ SUID বিট আছে আর owner root। তাই চালালে process-এর Effective UID হয় 0 (root), যদিও Real UID থাকে ইউজারের নিজের। kernel ফাইল খোলার অনুমতি দেয় EUID দেখে, তাই `/etc/shadow` খোলা যায়। `passwd` প্রোগ্রাম নিজে Real UID দেখে নিশ্চিত করে যে ইউজার শুধু নিজের পাসওয়ার্ডই বদলাচ্ছে।
2. SUID বিট দেওয়া আছে কিন্তু owner-এর execute বিট নেই। ফাইলটা চালানো যায় না, তাই SUID অর্থহীন। সাধারণত এটা ভুলের লক্ষণ।
3. নতুন ফাইল আর সাব-ডিরেক্টরি ডিরেক্টরির group পায় (যে বানিয়েছে তার primary group নয়)। নতুন সাব-ডিরেক্টরিগুলো নিজেরাও SGID বিট পায়।
4. `1777` (`drwxrwxrwt`)। sticky না থাকলে ডিরেক্টরিতে write পারমিশন থাকার কারণে যেকোনো ইউজার অন্য যেকোনো ইউজারের ফাইল মুছে বা rename করে ফেলতে পারত।
5. `6755` = SUID + SGID (4+2), সাথে `rwxr-xr-x`। `3775` = SGID + sticky (2+1), সাথে `rwxrwxr-x`।
6. না। লিনাক্স kernel `#!` দিয়ে শুরু হওয়া interpreted স্ক্রিপ্টের SUID উপেক্ষা করে। race condition আর environment variable দিয়ে interpreter-কে প্রভাবিত করার ঝুঁকির কারণে। এ রকম দরকারে `sudo`-র নিয়ম লেখো।

7. `+` মানে ফাইলে extended ACL আছে। ACL থাকলে `ls -l`-এর group ঘর ACL **mask** দেখায়, owning group-এর পারমিশন নয়। আসল ছবি দেখতে `getfacl` চালাও।
8. শুধু পড়তে পারবে। effective পারমিশন = entry ∩ mask = `rw-` ∩ `r--` = `r--`।
9. `-x` একটা নির্দিষ্ট entry মুছে দেয় (যেমন `-x u:alice`)। `-b` সব extended ACL entry মুছে দেয় (default সহ), শুধু সাধারণ owner/group/other থাকে। `-k` শুধু default ACL মুছে দেয়।
10. প্রথমে `lsattr` দিয়ে দেখো ফাইলে `i` (immutable) বা `a` (append-only) অ্যাট্রিবিউট আছে কি না। থাকলে `chattr -i` / `-a` দিয়ে তুলে নাও। (এ ছাড়া ফাইলসিস্টেম read-only মাউন্ট করা কি না, সেটাও দেখতে পারো।) পোর্ট ৮০-র জন্য SUID-root না দিয়ে শুধু `cap_net_bind_service` capability দাও (`setcap cap_net_bind_service=+ep binary`), অথবা systemd সার্ভিসে `AmbientCapabilities=CAP_NET_BIND_SERVICE`।

অধ্যায় ১৮: root, sudo ও প্রশাসনিক ক্ষমতা

এই অধ্যায়ে যা শিখবে

- root ইউজার কে, তার ক্ষমতা কতটা, আর সারাক্ষণ root হয়ে কাজ করা কেন বিপজ্জনক
- ভেতরে কীভাবে "ক্ষমতা" যাচাই হয়: UID 0, setuid বিট আর PAM
- `su`, `su -`, `sudo`, `sudo -i`, `sudo -s`, এদের মধ্যে আসল পার্থক্য
- `sudo`-র গুরুত্বপূর্ণ অপশন: `-u`, `-l`, `-v`, `-k`, `-e`, `-E`, `-n`, `-b`
- `/etc/sudoers` ফাইলের সিনট্যাক্স, Alias, `Defaults`, Tag (`NOPASSWD` ইত্যাদি)
- `visudo` দিয়ে নিরাপদে এডিট করা, আর `/etc/sudoers.d/` ডিরেক্টরি ব্যবহার
- `sudo` গ্রুপ (Debian/Ubuntu) আর `wheel` গ্রুপ (Fedora/RHEL)
- কোনো ইউজারকে শুধু নির্দিষ্ট কয়েকটা কমান্ডের অনুমতি দেওয়া, এবং তার ফাঁদগুলো
- `sudo -l` দিয়ে নিজের অনুমতি দেখা, sudo-র লগ কোথায় থাকে আর কীভাবে পড়তে হয়
- পাসওয়ার্ড মনে রাখার সময় (timestamp timeout) কীভাবে কাজ করে
- `pkexec` ও polkit কী, GUI-তে পাসওয়ার্ড চাওয়ার পেছনের ব্যবস্থা
- Least privilege (ন্যূনতম ক্ষমতা) নীতি, আর বাস্তবে সেটা কীভাবে মানতে হয়

১৮.১ root কে?

লিনাক্সে প্রতিটি ইউজারের একটা সংখ্যা থাকে, যাকে বলে **UID (User ID)**। অধ্যায় ১৫-এ এটা দেখেছ। এদের মধ্যে একজন বিশেষ: যার UID **0**। প্রথা অনুযায়ী তার নাম `root`।

```
$ id root
uid=0(root) gid=0(root) groups=0(root)
```

root হলো সিস্টেমের superuser (সর্বোচ্চ ক্ষমতার ইউজার)। কার্নেল যখন কোনো কাজের অনুমতি যাচাই করে, যেমন একটা ফাইল খোলা, অন্য ইউজারের process মেরে ফেলা বা নেটওয়ার্ক পোর্ট ৮০ খোলা, তখন সে প্রথমে দেখে: যে process অনুরোধ করছে তার effective UID কি 0? যদি হয়, তাহলে প্রায় সব পারমিশন চেক বাদ দেওয়া হয়।

> 📝 **মনে রাখো:** কার্নেল নাম দেখে না, সংখ্যা দেখে। কোনো ইউজারের নাম `root` না হয়েও UID 0 হলে সে-ও পুরো root। এজন্য নিরাপত্তা অডিটে `awk -F: '$3==0' /etc/passwd` চালিয়ে দেখা হয় UID 0 আর কারো আছে কি না।

উপমা: অফিসের মাস্টার চাবি

একটা অফিস ভবন ভাবো। সাধারণ কর্মচারীর কাছে শুধু নিজের রুমের চাবি থাকে। ম্যানেজারের কাছে তার বিভাগের কয়েকটা রুমের চাবি থাকে। আর কেয়ারটেকারের কাছে থাকে **মাস্টার চাবি**, যা দিয়ে সব দরজা খোলে, সার্ভার রুম, ভল্ট, এমনকি বিদ্যুতের মেইন সুইচ বোর্ডও।

root হলো সেই মাস্টার চাবি। মাস্টার চাবি দরকারি, কিন্তু কেউ সারাদিন সেটা পকেটে নিয়ে ঘোরে না। হারালে বা ভুল দরজা খুললে বিপদ। `sudo` হলো সেই ব্যবস্থা যেখানে তুমি রেজিস্টার খাতায় সই করে কিছুক্ষণের জন্য মাস্টার চাবি নাও, কাজ শেষে ফেরত দাও, আর খাতায় লেখা থাকে কে কখন কোন দরজা খুলেছিল।

root আসলে কী কী পারে

কাজ	সাধারণ ইউজার	root
অন্যের ফাইল পড়া/মোছা (পারমিশন না থাকলে)	না	হ্যাঁ
`/etc`, `/usr`, `/boot`-এ লেখা	না	হ্যাঁ
সফটওয়্যার ইনস্টল/আনইনস্টল	না	হ্যাঁ
অন্য ইউজারের process `kill` করা	না	হ্যাঁ
১০২৪-এর নিচের পোর্ট খোলা (যেমন ৮০, ৪৪৩)	না	হ্যাঁ
ডিস্ক ফরম্যাট, পার্টিশন বদলানো	না	হ্যাঁ
কার্নেল মডিউল লোড করা	না	হ্যাঁ
যেকোনো ইউজারের পাসওয়ার্ড বদলানো	না	হ্যাঁ
ফায়ারওয়াল নিয়ম বদলানো	না	হ্যাঁ

root-ও কি সব পারে?

প্রায় সব, কিন্তু পুরোপুরি না। আধুনিক লিনাক্সে root-এর ক্ষমতা ছোট ছোট টুকরোয় ভাগ করা, এগুলোকে বলে **capabilities** (যেমন `CAP_NET_BIND_SERVICE`, `CAP_SYS_ADMIN`)। root-এর সবগুলো থাকে, কিন্তু কন্টেইনারের ভেতরের "root"-এর অনেকগুলো থাকে না (অধ্যায় ৪৮)। এছাড়া SELinux/AppArmor-এর মতো MAC (Mandatory Access Control) ব্যবস্থা root-কেও আটকাতে পারে (অধ্যায় ৪৫)। আর `chattr +i` দেওয়া ফাইল root-ও সরাসরি মুছতে পারে না, আগে অ্যাট্রিবিউট সরাতে হয় (অধ্যায় ১৭)।

১৮.২ সারাক্ষণ root হয়ে থাকা কেন বিপজ্জনক

নতুনরা প্রায়ই ভাবে: "বারবার পাসওয়ার্ড দেওয়া ঝামেলা, root হয়েই সব কাজ করি।" এটা সবচেয়ে বড় ভুলগুলোর একটা। কারণগুলো দেখো:

১. কোনো "আনডু" নেই। লিনাক্সে `rm` করা ফাইল রিসাইকেল বিনে যায় না। একটা স্পেসের ভুল যথেষ্ট:

```
# rm -rf / home/rahim/old-project     ← ভুল: "/" আর "home" এর মাঝে স্পেস
```

সাধারণ ইউজার হিসেবে এই কমান্ড হাজার হাজার "Permission denied" দিয়ে থেমে যেত, ক্ষতি হতো শুধু নিজের ফাইলের। root হিসেবে এটা পুরো সিস্টেম মুছে ফেলার চেষ্টা করবে। (আধুনিক GNU `rm` সরাসরি `rm -rf /` আটকায় `--preserve-root` ডিফল্ট থাকায়, কিন্তু `rm -rf /*` আটকায় না।)

২. ম্যালওয়্যার আর বাগের ক্ষতি সীমাহীন হয়। ব্রাউজার বা কোনো স্ক্রিপ্ট যদি root হিসেবে চলে আর তাতে নিরাপত্তা ত্রুটি থাকে, আক্রমণকারী সাথে সাথে পুরো সিস্টেমের মালিক হয়ে যায়। সাধারণ ইউজার হিসেবে চললে ক্ষতি সেই ইউজারের ফাইলের মধ্যে আটকে থাকে।

৩. কোনো জবাবদিহি থাকে না। পাঁচজন অ্যাডমিন যদি একই root পাসওয়ার্ড শেয়ার করে, লগে শুধু লেখা থাকবে "root এটা করেছে"। কে করেছে বোঝার উপায় নেই। `sudo` প্রতিটি কমান্ডের সাথে আসল ইউজারের নাম লগে লিখে রাখে।

৪. পাসওয়ার্ড শেয়ার করা মানেই ঝুঁকি। একজন কর্মী চাকরি ছাড়লে root পাসওয়ার্ড বদলাতে হয় আর বাকি সবাইকে নতুনটা জানাতে হয়। sudo-তে শুধু ওই ইউজারকে গ্রুপ থেকে সরালেই হয়।

৫. ক্লান্ত মাথা ভুল করে। রাত দুটোয় সার্ভার ঠিক করার সময় প্রম্পটে `#` দেখেও খেয়াল না করে ভুল টার্মিনালে কমান্ড চালানো খুব সাধারণ ঘটনা।

> ⚠️ **সতর্কতা:** প্রম্পটের শেষ চিহ্নটা খেয়াল করো। `$` মানে সাধারণ ইউজার, `#` মানে root। এই বইয়ে উদাহরণে `$` দেখলে সাধারণ ইউজার, `#` দেখলে root shell বোঝাবে।

Ubuntu কেন root অ্যাকাউন্ট "লক" রাখে

Ubuntu ইনস্টল করার সময় root-এর পাসওয়ার্ড চাওয়া হয় না। root অ্যাকাউন্ট আছে, কিন্তু তার পাসওয়ার্ড **লক** করা। ফলে কেউ সরাসরি root হিসেবে লগইন করতে পারে না। প্রথম যে ইউজার তৈরি হয়, তাকে `sudo` গ্রুপে রাখা হয়।

```
$ sudo passwd -S root
root L 2024-04-23 0 99999 7 -1
```

এখানে দ্বিতীয় কলামের `L` মানে Locked। (`P` হলে usable password আছে, `NP` হলে কোনো পাসওয়ার্ড নেই।)

বিভিন্ন distro-র অবস্থা:

Distro	root পাসওয়ার্ড	প্রথম ইউজারের অ্যাডমিন গ্রুপ
Ubuntu 24.04	ডিফল্টে লক	`sudo`

Distro	root পাসওয়ার্ড	প্রথম ইউজারের অ্যাডমিন গ্রুপ
Debian 12	ইনস্টলার জিজ্ঞেস করে। root পাসওয়ার্ড দিলে প্রথম ইউজার `sudo` গ্রুপে যায় না, এমনকি `sudo` প্যাকেজই নাও থাকতে পারে। ফাঁকা রাখলে root লক হয় আর প্রথম ইউজার `sudo` গ্রুপে যায়	`sudo`
Fedora	ইনস্টলারে root ডিফল্টে লক। ইউজার তৈরির সময় "Make this user administrator" টিক দিলে `wheel` গ্রুপে যায়	`wheel`
RHEL 9 / Rocky / Alma	ইনস্টলারে root পাসওয়ার্ড সেট করা যায় বা লক রাখা যায়। ডিফল্টে root-এর পাসওয়ার্ড দিয়ে SSH লগইন বন্ধ	`wheel`

১৮.৩ ভেতরে কীভাবে কাজ করে: UID, setuid আর PAM

`sudo` বা `su` কীভাবে তোমাকে root বানায়, এটা বুঝলে বাকি সব সহজ হয়ে যায়।

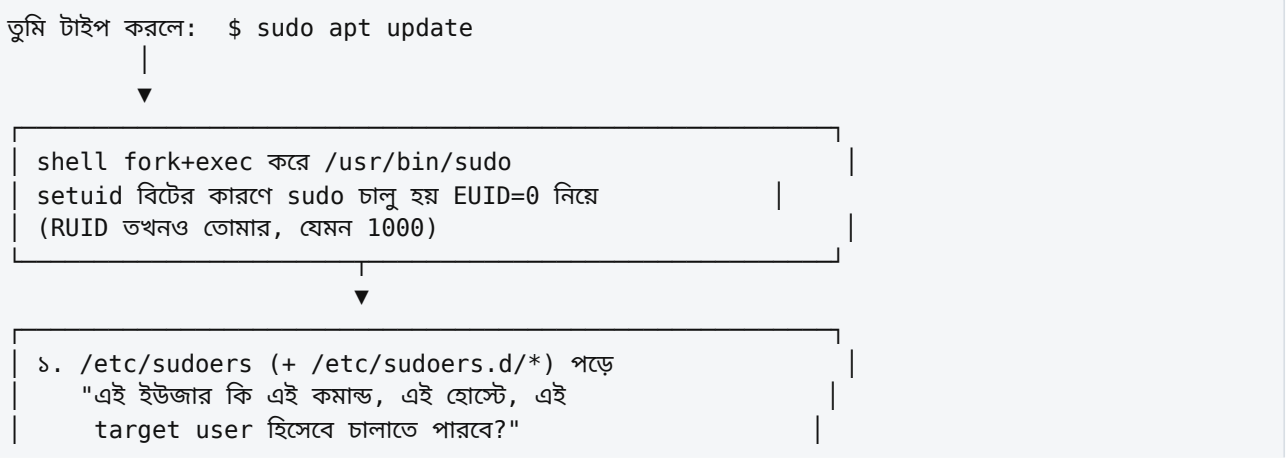
প্রতিটি process-এর সাথে কার্নেল কয়েকটা পরিচয় রাখে:

- **Real UID (RUID):** আসলে কে process টা চালু করেছে
- **Effective UID (EUID):** পারমিশন যাচাইয়ে কার পরিচয় ব্যবহার হবে
- **Saved UID:** পরে ফিরে যাওয়ার জন্য সংরক্ষিত পরিচয়

সাধারণত তিনটাই এক। এখন প্রশ্ন: সাধারণ ইউজার (UID 1000) চালানো একটা process কীভাবে UID 0 হয়? উত্তর **setuid বিট** (অধ্যায় ১৭)।

```
$ ls -l /usr/bin/sudo /usr/bin/su
-rwsr-xr-x 1 root root 277936 Apr  8  2024 /usr/bin/sudo
-rwsr-xr-x 1 root root  55680 Apr  9  2024 /usr/bin/su
```

`rws`-এর `s` হলো setuid। এর মানে: যে-ই এই প্রোগ্রাম চালাক, প্রোগ্রামটা চলবে ফাইলের **মালিকের** (root) effective UID নিয়ে। তাই `sudo` চালু হওয়ার মুহূর্তেই তার EUID 0। কিন্তু sudo বোকা না। root ক্ষমতা হাতে পেয়েও সে নিজে নিয়ম মেনে যাচাই করে তুমি অনুমতি পাওয়ার যোগ্য কি না।



```
│    না হলে → অস্বীকার + লগে লেখা                          │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│ ২. timestamp আছে ও মেয়াদ আছে? (গত ১৫ মিনিটে পাসওয়ার্ড?) │
│    না থাকলে → PAM (/etc/pam.d/sudo) দিয়ে তোমার          │
│    নিজের পাসওয়ার্ড যাচাই                                │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│ ৩. লগ লেখে: কে, কোন tty, কোন ডিরেক্টরি, কোন কমান্ড      │
│ ৪. environment পরিষ্কার করে (env_reset, secure_path)    │
│ ৫. setuid(0)/setgid(0) করে RUID-ও 0 বানায়              │
│ ৬. exec করে /usr/bin/apt update                          │
└──────────────────────────────────────────────────────────┘
```

PAM (Pluggable Authentication Modules) হলো লিনাক্সের প্রমাণীকরণের একটা প্লাগইন ব্যবস্থা। `sudo`, `su`, `login`, `sshd` নিজেরা পাসওয়ার্ড যাচাইয়ের কোড লেখে না, তারা PAM-কে জিজ্ঞেস করে। PAM কনফিগারেশন থাকে `/etc/pam.d/`-এ। এজন্যই ফিঙ্গারপ্রিন্ট বা দুই-ধাপের যাচাই (2FA) সহজে sudo-তে যোগ করা যায়, sudo-র কোড না বদলেই।

> 💡 **টিপস:** `/etc/sudoers` ফাইলের পারমিশন `0440` (`-r--r-----`) আর মালিক root। মানে সাধারণ ইউজার এটা পড়তেও পারে না। sudo setuid root হয়ে চলে বলেই নিজে ফাইলটা পড়তে পারে।

১৮.৪ `su` — ইউজার বদলানো

`su` মানে **substitute user** (অনেকে বলে "switch user")। এটা তোমাকে অন্য ইউজারে বদলে দেয়, আর **সেই target ইউজারের পাসওয়ার্ড** চায়।

সিনট্যাক্স:

```
su [options] [-] [user [argument...]]
```

user না দিলে ধরে নেওয়া হয় `root`।

অপশন	কাজ
`-`, `-l`, `--login`	পূর্ণ login shell: environment রিসেট, target-এর home-এ চলে যায়, তার profile ফাইল পড়ে
`-c "command"`	shell না খুলে শুধু একটা কমান্ড চালিয়ে ফিরে আসে
`-s /path/shell`	নির্দিষ্ট shell ব্যবহার করে
`-m`, `-p`	environment অপরিবর্তিত রাখে (`--preserve-environment`)
`-g group`	primary group বদলায় (শুধু root ব্যবহার করতে পারে)

`su` বনাম `su -` : আসল পার্থক্য

এটা ইন্টারভিউয়ের প্রিয় প্রশ্ন। পার্থক্য হলো **environment** (পরিবেশ চলক, যেমন `PATH`, `HOME`, `USER`) আর **বর্তমান ডিরেক্টরি**।

```
$ pwd
/home/rahim/projects
$ su
Password:
# pwd
/home/rahim/projects          ← একই ডিরেক্টরিতে রয়ে গেছ
# echo $HOME
/root                         ← util-linux su, HOME আর SHELL বদলায়
# echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin   ← distro ও কনফিগ অনুযায়ী ভিন্ন হতে পারে
# exit
$ su -
Password:
# pwd
/root                         ← root-এর home-এ চলে গেছ
# echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

বিষয়	`su`	`su -`
ডিরেক্টরি	যেখানে ছিলে সেখানেই	target-এর home (`/root`)
Shell ধরন	non-login shell	login shell
পড়ে	`~/.bashrc`	`/etc/profile`, `~/.bash_profile` বা `~/.profile` (আর সেখান থেকে সাধারণত `.bashrc`)
পুরনো environment	বেশির ভাগ থেকে যায়	প্রায় সব মুছে নতুন করে সেট হয়
নিরাপদ?	কম (তোমার `PATH`-এর কোনো ক্ষতিকর ফোল্ডার থেকে যেতে পারে)	বেশি

> 📝 **মনে রাখো:** root হতে চাইলে সবসময় `su -` ব্যবহার করো, শুধু `su` নয়। শুধু `su`-তে তোমার নিজের environment (যেমন অদ্ভুত `PATH` বা alias) root shell-এ চলে আসতে পারে, যা বিভ্রান্তি আর নিরাপত্তা ঝুঁকি তৈরি করে।

অন্য ইউজার হওয়া আর একটা কমান্ড চালানো

```
$ su - karim                      # karim হও (karim-এর পাসওয়ার্ড লাগবে)
$ su -c "systemctl restart nginx"  # root হিসেবে একটা কমান্ড চালাও, তারপর ফিরে এসো
Password:
$
```

Ubuntu-তে `su` কেন কাজ করে না

```
$ su -
Password:
su: Authentication failure
```

কারণ root-এর পাসওয়ার্ড লক। `su` সবসময় **target**-এর পাসওয়ার্ড চায়, আর root-এর তো কোনো ব্যবহারযোগ্য পাসওয়ার্ডই নেই। সমাধান: `sudo -i` ব্যবহার করো (নিচে দেখো)। root-এর পাসওয়ার্ড সেট করা (`sudo passwd root`) সম্ভব, কিন্তু সাধারণত দরকার নেই, বরং অপ্রয়োজনীয় ঝুঁকি।

root shell থেকে বের হতে `exit` লেখো বা `Ctrl+D` চাপো।

১৮.৫ `sudo` — একটা কমান্ড, অল্প সময়ের জন্য ক্ষমতা

`sudo` মানে "**su**bstitute user **do**" (আগে বলা হতো "superuser do")। এটা **একটা কমান্ড** অন্য ইউজার (ডিফল্টে root) হিসেবে চালায়, আর চায় **তোমার নিজের পাসওয়ার্ড**।

`su` আর `sudo`-র মূল দর্শনগত পার্থক্য

বিষয়	`su`	`sudo`
কার পাসওয়ার্ড চায়	target ইউজারের (root-এর)	তোমার নিজের
কতক্ষণ ক্ষমতা	যতক্ষণ shell খোলা	শুধু ওই একটা কমান্ড (+ কিছুক্ষণ পাসওয়ার্ড মনে রাখে)
কে পারবে তার নিয়ন্ত্রণ	যে পাসওয়ার্ড জানে, সে-ই	`/etc/sudoers` ফাইলে সূক্ষ্ম নিয়ম
নির্দিষ্ট কমান্ডে সীমাবদ্ধ করা	সম্ভব না	সম্ভব
লগ	"কেউ root হয়েছে"	প্রতিটি কমান্ড, আসল ইউজারের নামসহ
পাসওয়ার্ড শেয়ার	করতে হয়	করতে হয় না

প্রথম sudo

```
$ apt update
Reading package lists... Done
E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)
E: Unable to lock directory /var/lib/apt/lists/
$ sudo apt update
[sudo] password for rahim:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
...
```

> 💡 **টিপস:** পাসওয়ার্ড টাইপ করার সময় স্ক্রিনে কিছু দেখা যায় না, এমনকি তারকা চিহ্নও না। এটা স্বাভাবিক, কিবোর্ড নষ্ট না। টাইপ করে Enter চাপো। (কিছু distro যেমন Linux Mint `pwfeedback` চালু রাখে, তখন

> `*` দেখায়।)

প্রথমবার sudo চালালে একটা "lecture" (উপদেশ) দেখায়:

```
We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.
```

তিনটা কথাই সত্যি, বিশেষ করে দ্বিতীয়টা।

অনুমতি না থাকলে

```
$ sudo apt update
[sudo] password for karim:
karim is not in the sudoers file.
```

(পুরনো sudo ভার্সনে এর সাথে "This incident will be reported." লেখা থাকত।) এই চেষ্টাটা লগে রেকর্ড হয়।

`sudo`-র গুরুত্বপূর্ণ অপশন

সিনট্যাক্স:

```
sudo [options] command [arguments]
```

অপশন	কাজ
`-u user`	root-এর বদলে নির্দিষ্ট ইউজার হিসেবে চালাও
`-g group`	নির্দিষ্ট গ্রুপকে primary group করে চালাও
`-i`	target ইউজারের **login shell** খোলো (অথবা login shell-এর মাধ্যমে কমান্ড চালাও)
`-s`	`$SHELL`-এ থাকা shell খোলো (non-login)
`-l`	তুমি কী কী চালাতে পারো তার তালিকা (`-ll` দিলে বিস্তারিত)
`-l command`	নির্দিষ্ট কমান্ডটা অনুমোদিত কি না দেখাও (পূর্ণ path দেখায়)
`-U user -l`	অন্য ইউজারের অনুমতি দেখাও (এর জন্য নিজের যথেষ্ট অধিকার লাগে)
`-v`	কোনো কমান্ড না চালিয়ে timestamp নবায়ন করো (পাসওয়ার্ড আবার মনে রাখাও)
`-k`	বর্তমান session-এর timestamp বাতিল করো (পরের বার আবার পাসওয়ার্ড চাইবে)
`-K`	ইউজারের সব timestamp পুরোপুরি মুছে ফেলো
`-e file`	ফাইল নিরাপদে এডিট করো (`sudoedit`-এর মতো)
`-E`	তোমার environment রেখে দাও (sudoers-এ অনুমতি থাকলে)

অপশন	কাজ
`--preserve-env=VAR1,VAR2`	শুধু নির্দিষ্ট চলক রেখে দাও
`-n`	non-interactive: পাসওয়ার্ড লাগলে না চেয়ে ভুল হিসেবে বের হয়ে যাও (স্ক্রিপ্টে কাজের)
`-b`	কমান্ড background-এ চালাও
`-H`	`HOME` চলক target-এর home-এ সেট করো (আধুনিক sudo-তে এটা সাধারণত ডিফল্ট)
`-S`	পাসওয়ার্ড stdin থেকে পড়ো
`-V`	sudo-র ভার্সন দেখাও (root হিসেবে চালালে কনফিগ তথ্যও)

উদাহরণ:

```
$ sudo -u postgres psql                 # postgres ইউজার হিসেবে psql চালাও
$ sudo -u www-data touch /var/www/html/test.txt
$ sudo -n true && echo "পাসওয়ার্ড ছাড়াই sudo চলবে" || echo "পাসওয়ার্ড লাগবে"
$ sudo -v                               # এখনই পাসওয়ার্ড দিয়ে রাখো, লম্বা স্ক্রিপ্টের আগে কাজের
$ sudo -k                               # কম্পিউটার ছেড়ে ওঠার আগে "চাবি ফেরত দাও"
```

`sudo !!` — আগের কমান্ডটা sudo দিয়ে

কমান্ড চালিয়ে "Permission denied" পেলে পুরোটা আবার টাইপ করতে হবে না:

```
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ sudo !!
sudo cat /etc/shadow
[sudo] password for rahim:
root:*:19836:0:99999:7:::
...
```

`!!` হলো bash-এর history expansion, মানে "আগের পুরো কমান্ড" (অধ্যায় ৬)।

১৮.৬ `sudo -i`, `sudo -s`, `sudo su -`: root shell পাওয়ার নানা উপায়

কখনো কখনো অনেকগুলো প্রশাসনিক কমান্ড পরপর চালাতে হয়। তখন প্রতিবার `sudo` না লিখে একটা root shell নেওয়া যায়।

```
$ pwd
/home/rahim/projects

$ sudo -s
# pwd
/home/rahim/projects        ← ডিরেক্টরি বদলায়নি
# echo $HOME
```

```
/root                       ← আধুনিক sudo HOME বদলে দেয়
# exit

$ sudo -i
# pwd
/root                       ← root-এর home-এ
# exit
```

কমান্ড	কার পাসওয়ার্ড	shell-এর ধরন	ডিরেক্টরি	environment
`su`	root-এর	non-login	একই থাকে	অনেকটা তোমারটাই
`su -`	root-এর	login	`/root`	root-এর পূর্ণ login environment
`sudo -s`	তোমার	non-login (`$SHELL`)	একই থাকে	sudo রিসেট করা environment, root-এর `.bashrc` পড়ে
`sudo -i`	তোমার	login (root-এর shell, `/etc/passwd` থেকে)	`/root`	root-এর পূর্ণ login environment
`sudo su -`	তোমার (sudo-র জন্য)	login	`/root`	`su -`-এর মতো
`sudo bash`	তোমার	non-login	একই থাকে	`sudo -s`-এর কাছাকাছি

> 💡 **টিপস:** `sudo su -` পুরনো অভ্যাস, কাজ করে, কিন্তু এখানে দুটো setuid প্রোগ্রাম একটার ভেতর আরেকটা চলে। একই কাজ `sudo -i` সরাসরি করে, এটাই পরিষ্কার উপায়।

কোনটা কখন?

- একটা কমান্ড: `sudo command` (সবচেয়ে ভালো)
- পরপর অনেক অ্যাডমিন কাজ, পরিষ্কার environment চাই: `sudo -i`
- বর্তমান ডিরেক্টরিতে থেকে কয়েকটা কমান্ড: `sudo -s`
- কাজ শেষ হলেই `exit`। root shell খোলা রেখে উঠে যেও না।

একটা সাধারণ ফাঁদ: sudo আর redirection

```
$ sudo echo "nameserver 1.1.1.1" > /etc/resolv.conf
bash: /etc/resolv.conf: Permission denied
```

কেন? কারণ `>` (redirection) করে **তোমার shell**, sudo নয়। shell প্রথমে সাধারণ ইউজার হিসেবে ফাইলটা লেখার জন্য খুলতে চায়, ব্যর্থ হয়। sudo শুধু `echo` কে root বানায়, যার কোনো দরকারই ছিল না।

```
  তোমার shell (rahim)                        sudo → echo (root)
┌──────────────────────────┐
```

```
| > /etc/resolv.conf খোলো     |  ← এখানেই ব্যর্থ, rahim লিখতে পারে না
└──────────────────────────────┘
```

সমাধান:

```
$ echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf       # ওভাররাইট
$ echo "nameserver 8.8.8.8" | sudo tee -a /etc/resolv.conf    # যোগ (append)
$ sudo sh -c 'echo "nameserver 1.1.1.1" > /etc/resolv.conf'   # পুরো কমান্ড root shell-এ
```

`tee` সম্পর্কে বিস্তারিত অধ্যায় ১৯-এ।

`sudoedit` / `sudo -e` — root ফাইল নিরাপদে এডিট করা

`sudo vim /etc/hosts` চালালে পুরো এডিটরটাই root হিসেবে চলে। vim-এর ভেতর থেকে `:!bash` দিয়ে root shell পাওয়া যায়, প্লাগইনও root হিসেবে চলে। নিরাপদ উপায়:

```
$ sudoedit /etc/hosts
```

ভেতরে যা ঘটে:

1. sudo ফাইলটার একটা অস্থায়ী কপি বানায়, তোমার মালিকানায়
2. তোমার এডিটর (`SUDO_EDITOR`, `VISUAL` বা `EDITOR` চলক অনুযায়ী) **তোমার** নামে চলে
3. সেভ করে বের হলে sudo কপিটা আসল জায়গায় ফেরত লেখে

এডিটর কখনো root হয় না। নিজের পছন্দের এডিটর সেট করতে: `export SUDO_EDITOR=nano` (স্থায়ী করতে `.bashrc`-এ, অধ্যায় ৪৪)।

১৮.৭ কে sudo পাবে: `sudo` আর `wheel` গ্রুপ

প্রতিটি ইউজারের জন্য আলাদা নিয়ম না লিখে, distro-গুলো একটা গ্রুপকে পূর্ণ অ্যাডমিন অধিকার দিয়ে রাখে:

Distro পরিবার	গ্রুপ	`/etc/sudoers` -এর লাইন
Debian, Ubuntu, Mint	`sudo`	`%sudo ALL=(ALL:ALL) ALL`
Fedora, RHEL, Rocky, Alma, CentOS	`wheel`	`%wheel ALL=(ALL) ALL`
Arch	`wheel` (লাইনটা ডিফল্টে comment করা, নিজে চালু করতে হয়)	`# %wheel ALL=(ALL:ALL) ALL`

"wheel" নামটা এসেছে পুরনো ইউনিক্স যুগের "big wheel" (গুরুত্বপূর্ণ ব্যক্তি) কথা থেকে।

কাউকে অ্যাডমিন বানানো

```
# Debian/Ubuntu
$ sudo usermod -aG sudo karim

# Fedora/RHEL
$ sudo usermod -aG wheel karim

# অথবা gpasswd দিয়ে
$ sudo gpasswd -a karim sudo
Adding user karim to group sudo
```

> ⚠️ **সতর্কতা:** `usermod` -এ `-a` (append) দিতে ভুলবে না। শুধু `-G sudo` দিলে karim-এর বাকি সব secondary group মুছে গিয়ে শুধু `sudo` থাকবে। বিস্তারিত অধ্যায় ১৫-এ।

গ্রুপ পরিবর্তন সাথে সাথে চলমান session-এ প্রযোজ্য হয় না। karim-কে লগআউট করে আবার লগইন করতে হবে (বা SSH session নতুন করে খুলতে হবে)। যাচাই:

```
$ id karim
uid=1001(karim) gid=1001(karim) groups=1001(karim),27(sudo)
$ groups karim
karim : karim sudo
```

কারো অ্যাডমিন অধিকার কেড়ে নেওয়া

```
$ sudo gpasswd -d karim sudo
Removing user karim from group sudo
# অথবা
$ sudo deluser karim sudo        # Debian/Ubuntu
```

> 📝 **মনে রাখো:** অধিকার কেড়ে নেওয়ার পর karim-এর আগে থেকে খোলা root shell (যদি থাকে) বন্ধ হয় না। গুরুতর ক্ষেত্রে তার চলমান session শেষ করে দাও (`sudo loginctl terminate-user karim`)।

১৮.৮ `/etc/sudoers` ফাইলের গঠন

এবার মূল নিয়ম-বইটা খুলি। Ubuntu 24.04-এ ডিফল্ট ফাইলটা (comment বাদে) মোটামুটি এরকম:

```
Defaults        env_reset
Defaults        mail_badpass
Defaults
secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"
Defaults        use_pty
```

```
# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

@includedir /etc/sudoers.d
```

RHEL 9 / Fedora-তে মূল লাইনগুলো:

```
root    ALL=(ALL)       ALL
%wheel  ALL=(ALL)       ALL
## %wheel        ALL=(ALL)       NOPASSWD: ALL
#includedir /etc/sudoers.d
```

> ⚠️ **সতর্কতা:** `#includedir`-এর শুরুতে `#` থাকলেও এটা comment **না**! এটা পুরনো সিনট্যাক্সের একটা নির্দেশ। sudo 1.9.1 থেকে নতুন রূপ `@includedir` চালু হয়েছে, দুটোই কাজ করে। ভুল করে এই লাইন মুছবে না।

একটা নিয়মের লাইন পড়া (User Specification)

```
%sudo     ALL   =   (ALL : ALL)       ALL
  |         |          |     |          |
  |         |          |     |          └─ কোন কমান্ড চালাতে পারবে
  |         |          |     └────────────── কোন গ্রুপ হিসেবে চালাতে পারবে (Runas group)
  |         |          └──────────────────── কোন ইউজার হিসেবে চালাতে পারবে (Runas user)
  |         └─────────────────────────────── কোন হোস্টে (মেশিনে) এই নিয়ম প্রযোজ্য
  └───────────────────────────────────────── কে (ইউজার, বা % দিলে গ্রুপ)
```

সাধারণ রূপ:

```
WHO   WHERE = (AS_USER:AS_GROUP)  [TAG:]  COMMANDS
```

পড়ো এভাবে: "**sudo গ্রুপের সদস্যরা, যেকোনো হোস্টে, যেকোনো ইউজার ও যেকোনো গ্রুপ হিসেবে, যেকোনো কমান্ড চালাতে পারবে।**"

অংশগুলো বিস্তারিত:

অংশ	কী লেখা যায়	উদাহরণ
WHO	ইউজারনেম, `%group`, `#UID`, `%#GID`, User_Alias	`rahim`, `%devops`, `#1005`
WHERE	হোস্টনেম, IP, নেটওয়ার্ক, Host_Alias, `ALL`	`web01`, `192.168.1.0/24`, `ALL`
AS_USER	ইউজারনেম, Runas_Alias, `ALL`	`(root)`, `(postgres)`, `(ALL)`

অংশ	কী লেখা যায়	উদাহরণ
AS_GROUP	গ্রুপনেম, `ALL` (ঐচ্ছিক)	`(ALL:ALL)`, `(root:adm)`
TAG	`NOPASSWD:`, `PASSWD:`, `NOEXEC:`, `EXEC:`, `SETENV:`, `NOSETENV:` ইত্যাদি	`NOPASSWD:`
COMMANDS	**পূর্ণ path**সহ কমান্ড, Cmnd_Alias, `ALL`	`/usr/bin/systemctl restart nginx`

`(AS_USER)` একদম না লিখলে ডিফল্ট ধরা হয় `(root)`।

WHERE বা হোস্টের অংশ কেন? বড় প্রতিষ্ঠানে একই `sudoers` ফাইল শত শত সার্ভারে কপি করা হয় (Ansible বা অন্য কোনো টুল দিয়ে)। তখন এক লাইনেই বলা যায় "rahim শুধু web সার্ভারগুলোতে nginx রিস্টার্ট করতে পারবে"। একক মেশিনে সাধারণত `ALL` লেখা হয়।

উদাহরণ নিয়ম

```
# rahim পূর্ণ অ্যাডমিন
rahim   ALL=(ALL:ALL) ALL

# karim শুধু nginx রিস্টার্ট ও রিলোড করতে পারবে
karim   ALL=(root) /usr/bin/systemctl restart nginx, /usr/bin/systemctl reload nginx

# dbteam গ্রুপ শুধু postgres ইউজার হিসেবে যেকোনো কমান্ড
%dbteam ALL=(postgres) ALL

# backup ইউজার পাসওয়ার্ড ছাড়া rsync চালাতে পারবে
backup  ALL=(root) NOPASSWD: /usr/bin/rsync

# deploy ইউজার: একটা কমান্ডে পাসওয়ার্ড লাগবে না, অন্যটায় লাগবে
deploy  ALL=(root) NOPASSWD: /usr/bin/systemctl restart myapp, PASSWD: /usr/bin/apt
```

শেষ লাইনে দেখো, একবার Tag দিলে সেটা পরের সব কমান্ডে প্রযোজ্য থাকে যতক্ষণ না উল্টো Tag আসে।

> 📝 **মনে রাখো:** কমান্ড সবসময় **পূর্ণ path** দিয়ে লিখতে হয়। `systemctl` না, `/usr/bin/systemctl`। path জানতে `command -v systemctl` বা `which systemctl` চালাও (অধ্যায় ১২)। নইলে কেউ নিজের ফোল্ডারে `systemctl` নামে একটা ক্ষতিকর স্ক্রিপ্ট রেখে দিতে পারত।

কমান্ডে আর্গুমেন্টের নিয়ম

লেখা	মানে
`/usr/bin/systemctl`	যেকোনো আর্গুমেন্টসহ systemctl (বা কোনো আর্গুমেন্ট ছাড়া)
`/usr/bin/systemctl restart nginx`	ঠিক এই আর্গুমেন্টগুলো, অন্য কিছু না
`/usr/bin/systemctl ""`	শুধু আর্গুমেন্ট ছাড়া

লেখা	মানে
`/usr/bin/systemctl restart *`	`restart` এর পর যেকোনো কিছু (wildcard, বিপজ্জনক হতে পারে)
`!/usr/bin/su`	এই কমান্ড বাদ (negation, নিচে ফাঁদ দেখো)
`sudoedit /etc/nginx/nginx.conf`	এই ফাইলটা `sudoedit` দিয়ে এডিট করতে পারবে

Alias — নিয়মকে সংগঠিত করা

অনেক ইউজার আর অনেক কমান্ড থাকলে Alias দিয়ে নাম দেওয়া যায়। Alias-এর নাম **বড় হাতের অক্ষরে** (A-Z, 0-9, _) হতে হয়।

```
# কারা
User_Alias   WEBADMINS = rahim, karim, %webteam

# কোন মেশিনে
Host_Alias   WEBSERVERS = web01, web02, 10.0.5.0/24

# কার হিসেবে
Runas_Alias  APPUSERS = www-data, nginx

# কোন কমান্ড
Cmnd_Alias   WEBCTL = /usr/bin/systemctl restart nginx, \
                      /usr/bin/systemctl reload nginx, \
                      /usr/bin/systemctl status nginx
Cmnd_Alias   PKG    = /usr/bin/apt, /usr/bin/apt-get, /usr/bin/dpkg

# নিয়ম
WEBADMINS  WEBSERVERS = (root) WEBCTL
WEBADMINS  WEBSERVERS = (APPUSERS) NOPASSWD: ALL
```

লম্বা লাইন ভাঙতে শেষে `\` দেওয়া যায়।

`Defaults` — sudo-র আচরণ বদলানো

`Defaults` লাইন দিয়ে sudo-র সেটিংস বদলানো হয়। এগুলো সবার জন্য, বা নির্দিষ্ট ইউজার/হোস্ট/কমান্ডের জন্য দেওয়া যায়:

```
Defaults              env_reset                # সবার জন্য
Defaults:rahim        timestamp_timeout=30     # শুধু rahim-এর জন্য
Defaults:%devops      !lecture                 # devops গ্রুপের জন্য lecture বন্ধ
Defaults@web01        log_output               # শুধু web01 হোস্টে
Defaults>postgres     !requiretty              # postgres হিসেবে চালালে
Defaults!/usr/bin/apt !log_output              # নির্দিষ্ট কমান্ডের জন্য
```

গুরুত্বপূর্ণ `Defaults` অপশন (সবগুলো `man sudoers`-এ আছে):

অপশন	কাজ
`env_reset`	environment পরিষ্কার করে শুধু নিরাপদ কিছু চলক রাখে (ডিফল্টে চালু)

অপশন	কাজ
`env_keep += "VAR"`	নির্দিষ্ট চলক রেখে দাও (যেমন `http_proxy`)
`secure_path="..."`	sudo কমান্ডের জন্য নির্দিষ্ট নিরাপদ `PATH`
`timestamp_timeout=N`	কত মিনিট পাসওয়ার্ড মনে রাখবে (নিচে বিস্তারিত)
`timestamp_type=tty`	timestamp কীভাবে আলাদা রাখবে: `tty`, `ppid`, `global`, `kernel`
`passwd_tries=N`	ভুল পাসওয়ার্ডের সর্বোচ্চ চেষ্টা (ডিফল্ট ৩)
`passwd_timeout=N`	পাসওয়ার্ড প্রম্পট কত মিনিট অপেক্ষা করবে (০ মানে অসীম)
`badpass_message="..."`	ভুল পাসওয়ার্ডে দেখানো বার্তা
`insults`	ভুল পাসওয়ার্ডে মজার অপমানজনক বার্তা (মজা, তবে পেশাদার সার্ভারে নয়)
`pwfeedback`	পাসওয়ার্ড টাইপের সময় `*` দেখায়
`lecture=always/once/never`	lecture কখন দেখাবে
`logfile="/var/log/sudo.log"`	syslog ছাড়াও আলাদা লগ ফাইলে লেখো
`log_input`, `log_output`	পুরো session-এর ইনপুট/আউটপুট রেকর্ড করো (`sudoreplay` দিয়ে দেখা যায়)
`use_pty`	কমান্ড আলাদা pseudo-terminal-এ চালাও (নিরাপত্তার জন্য, Ubuntu-তে চালু)
`requiretty`	শুধু আসল terminal থেকে sudo চলবে (কিছু পুরনো RHEL কনফিগে ছিল)
`mail_badpass`	ভুল পাসওয়ার্ড দিলে `mailto`-তে মেইল পাঠাও
`editor=/usr/bin/nano`	`visudo` কোন এডিটর ব্যবহার করবে
`rootpw`	ইউজারের নিজের বদলে root-এর পাসওয়ার্ড চাও (openSUSE-এর মতো `targetpw` ও আছে)

বুলিয়ান অপশন বন্ধ করতে আগে `!` দাও: `Defaults !lecture`।

নিয়মের ক্রম: শেষ মিলটাই জেতে

একজন ইউজারের সাথে একাধিক লাইন মিলে গেলে, **ফাইলে সবার শেষে যে লাইন মেলে**, সেটা কার্যকর। এজন্য নির্দিষ্ট নিয়মগুলো সাধারণ নিয়মের **পরে** লিখতে হয়।

```
%sudo   ALL=(ALL:ALL) ALL
rahim   ALL=(ALL) NOPASSWD: /usr/bin/apt        ← rahim sudo গ্রুপে, এটা পরে আছে, তাই apt-এ পাসওয়ার্ড লাগবে না
```

উল্টো ক্রমে লিখলে `%sudo` লাইন পরে মিলত, আর rahim-কে apt-এর জন্যও পাসওয়ার্ড দিতে হতো।

`/etc/sudoers.d/`-এর ফাইলগুলো `@includedir` লাইনের জায়গায় **নামের বর্ণানুক্রমে** (lexical order) পড়া হয়। তাই `10-admins` আগে, `90-overrides` পরে। এজন্য ফাইলের নামের আগে সংখ্যা দেওয়া একটা ভালো অভ্যাস।

১৮.৯ `visudo` — sudoers এডিটের একমাত্র নিরাপদ উপায়

> ⚠️ **সতর্কতা:** `/etc/sudoers` কখনো সরাসরি `nano /etc/sudoers` বা `vim` দিয়ে এডিট করবে না। একটা টাইপো করলে sudo পুরো ফাইলটা পড়তেই অস্বীকার করবে, আর **কেউই** আর sudo চালাতে পারবে না। Ubuntu-তে root লক থাকায় তুমি নিজেকে সিস্টেম থেকে পুরোপুরি তালাবদ্ধ করে ফেলবে।

ভুল sudoers হলে যা দেখবে:

```
$ sudo ls
>>> /etc/sudoers: syntax error near line 25 <<<
sudo: parse error in /etc/sudoers near line 25
sudo: no valid sudoers sources found, quitting
```

`visudo` এই বিপদ ঠেকায়:

```
 $ sudo visudo
      │
      ▼
 ┌──────────────────────────────────────────────┐
 │ ১. /etc/sudoers লক করে (একসাথে দুজন        │
 │    এডিট করতে পারবে না)                      │
 │ ২. একটা অস্থায়ী কপি বানায় (/etc/sudoers.tmp)│
 │ ৩. এডিটরে খোলে                              │
 └──────────────────────┬───────────────────────┘
                        ▼  তুমি সেভ করে বের হলে
 ┌──────────────────────────────────────────────┐
 │ ৪. সিনট্যাক্স যাচাই                         │
 │    ঠিক আছে → আসল ফাইলে বসিয়ে দেয়          │
 │    ভুল → "What now?" জিজ্ঞেস করে            │
 └──────────────────────────────────────────────┘
```

ভুল থাকলে:

```
$ sudo visudo
/etc/sudoers:25:20: syntax error
karim ALL=(root) systemctl restart nginx
                  ^~~~~~~~
What now?
```

এখানে তিনটা বিকল্প:

কী চাপবে	মানে
`e`	আবার এডিটরে ফিরে গিয়ে ঠিক করো (**এটাই সাধারণত করা উচিত**)
`x`	কিছু সেভ না করে বের হও, আগের ফাইল অক্ষত থাকে
`Q`	ভুলসহই সেভ করো (**বিপজ্জনক, কখনো চাপবে না**)

উপরের ভুলটা হলো `systemctl`-এর পূর্ণ path না দেওয়া।

`visudo`-র অপশন

কমান্ড	কাজ
`sudo visudo`	`/etc/sudoers` এডিট
`sudo visudo -f /etc/sudoers.d/webteam`	নির্দিষ্ট ফাইল এডিট (নতুন হলে তৈরি)
`sudo visudo -c`	সব sudoers ফাইলের সিনট্যাক্স শুধু যাচাই করো, এডিট না
`sudo visudo -cf /path/file`	একটা নির্দিষ্ট ফাইল যাচাই করো
`sudo visudo -s`	কঠোর (strict) যাচাই, যেমন অব্যবহৃত বা অসংজ্ঞায়িত Alias ধরা

```
$ sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/webteam: parsed OK
```

কোন এডিটর খুলবে?

Ubuntu/Debian-এ `visudo` সাধারণত **nano** খোলে (ডিফল্ট `editor` বিকল্প অনুযায়ী)। Fedora/RHEL-এ খোলে **vi**। বদলাতে:

```
$ sudo EDITOR=nano visudo           # একবারের জন্য
$ sudo update-alternatives --config editor   # Debian/Ubuntu-তে সিস্টেম ডিফল্ট বদলাও
```

nano-র শর্টকাট অধ্যায় ১৩-এ, vim-এর অধ্যায় ১৪-এ।

> 💡 **টিপস:** sudoers বদলানোর সময় একটা **আলাদা টার্মিনালে** আগে থেকে `sudo -i` দিয়ে root shell খোলা রাখো। কিছু ভুল হলে সেই shell থেকে ঠিক করতে পারবে। নতুন নিয়ম কাজ করছে কি না অন্য টার্মিনাল থেকে পরীক্ষা করো, তারপরই root shell বন্ধ করো।

সব শেষ হয়ে গেলে: তালাবদ্ধ হয়ে গেলে কী করবে

যদি sudoers ভেঙে যায় আর root shell খোলা না থাকে:

1. **Ubuntu desktop-এ:** `pkexec visudo` চেষ্টা করো (polkit sudo-র উপর নির্ভর করে না, নিচে দেখো)।
2. **Recovery mode:** রিবুট করে GRUB মেনুতে "Advanced options → recovery mode → root" নাও, তারপর `mount -o remount,rw /` আর `visudo`।
3. **Live USB:** লাইভ ইউএসবি থেকে বুট করে ডিস্ক মাউন্ট করে ফাইল ঠিক করো (অধ্যায় ৩২)।
4. **ক্লাউড VM:** প্রোভাইডারের serial console বা রেসকিউ মোড।

বুট প্রসেস আর recovery নিয়ে অধ্যায় ২৭-এ বিস্তারিত আছে।

১৮.১০ `/etc/sudoers.d/` — নিজের নিয়ম আলাদা ফাইলে

মূল `/etc/sudoers` ফাইল প্যাকেজ আপডেটের সময় বদলাতে পারে (Debian/Ubuntu তখন জিজ্ঞেস করে কোনটা রাখবে)। নিজের নিয়ম এতে লিখলে conflict হয়। সমাধান: `/etc/sudoers.d/`-এ আলাদা ছোট ফাইল।

সুবিধা:

- প্রতিটি দলের জন্য আলাদা ফাইল, সহজে যোগ/মোছা যায়
- Ansible, Puppet-এর মতো টুল একটা ফাইল ফেলে দিলেই হয়
- মূল ফাইল অপরিবর্তিত থাকে

```
$ sudo visudo -f /etc/sudoers.d/10-webteam
```

ভেতরে লেখো:

```
# Web team: nginx নিয়ন্ত্রণ
Cmnd_Alias NGINXCTL = /usr/bin/systemctl restart nginx, \
                      /usr/bin/systemctl reload nginx, \
                      /usr/bin/systemctl status nginx, \
                      /usr/sbin/nginx -t
%webteam ALL=(root) NGINXCTL
```

`sudoers.d`-এর অলিখিত নিয়ম (এগুলো না জানলে ফাইল নিঃশব্দে উপেক্ষিত হয়)

নিয়ম	বিস্তারিত
ফাইলের নামে `.` থাকা চলবে না	`webteam.conf` **পড়া হবে না**! `webteam` বা `10-webteam` লেখো
`~` দিয়ে শেষ হলে উপেক্ষিত	`webteam~` (এডিটরের ব্যাকআপ) পড়া হয় না
পারমিশন `0440`	`visudo -f` এটা নিজেই ঠিক রাখে। হাতে তৈরি করলে `sudo chmod 0440 file`
মালিক root	`root:root`
বর্ণানুক্রমে পড়া হয়	পরের ফাইলের নিয়ম আগের ফাইলকে override করতে পারে (শেষ মিল জেতে)

> ⚠️ **সতর্কতা:** `.conf` এক্সটেনশন দেওয়ার ভুলটা খুব সাধারণ। তুমি নিয়ম লিখলে, `visudo -c` হয়তো ঠিক বলল, কিন্তু sudo সেটা কখনো পড়ছেই না। `sudo -l -U ইউজার` দিয়ে সবসময় যাচাই করো নিয়মটা সত্যিই কার্যকর হয়েছে কি না।

স্ক্রিপ্ট থেকে নিরাপদে ফাইল তৈরি করার প্যাটার্ন:

```
$ echo 'backup ALL=(root) NOPASSWD: /usr/bin/rsync' > /tmp/backup-sudo
$ sudo visudo -cf /tmp/backup-sudo && \
  sudo install -m 0440 -o root -g root /tmp/backup-sudo /etc/sudoers.d/20-backup
/tmp/backup-sudo: parsed OK
```

আগে যাচাই, তারপর ঠিক পারমিশনসহ বসানো।

১৮.১১ `NOPASSWD` — পাসওয়ার্ড ছাড়া sudo

কিছু ক্ষেত্রে পাসওয়ার্ড চাওয়া অসম্ভব বা অর্থহীন:

- cron job বা স্ক্রিপ্ট যা রাতে নিজে নিজে চলে
- CI/CD পাইপলাইন (deploy ইউজার)
- ক্লাউড VM যেখানে লগইন শুধু SSH key দিয়ে, ইউজারের কোনো পাসওয়ার্ডই নেই (যেমন AWS-এর `ubuntu` ইউজার, যা `/etc/sudoers.d/90-cloud-init-users`-এ `NOPASSWD:ALL` পায়)

```
# ভালো: সীমিত কমান্ডে NOPASSWD
deploy  ALL=(root) NOPASSWD: /usr/bin/systemctl restart myapp

# ঝুঁকিপূর্ণ: সবকিছুতে NOPASSWD
deploy  ALL=(ALL) NOPASSWD: ALL
```

> ⚠️ **সতর্কতা:** `NOPASSWD: ALL` মানে ওই ইউজারের অ্যাকাউন্ট যে দখল করবে (চুরি যাওয়া SSH key, ব্রাউজারের ত্রুটি, ক্ষতিকর npm প্যাকেজ), সে কোনো বাধা ছাড়াই root। পাসওয়ার্ড হলো শেষ একটা দেয়াল। পারলে NOPASSWD শুধু নির্দিষ্ট কমান্ডে দাও।

১৮.১২ নির্দিষ্ট কমান্ডের অনুমতি, এবং তার ফাঁদ

"karim শুধু লগ দেখতে পারবে" শুনতে সহজ। কিন্তু এখানেই সবচেয়ে বেশি নিরাপত্তা ছিদ্র তৈরি হয়। কারণ অনেক প্রোগ্রামের ভেতর থেকে **shell escape** (shell চালু করা) সম্ভব।

ফাঁদ ১: যেসব প্রোগ্রাম shell দেয়

```
karim ALL=(root) /usr/bin/less /var/log/syslog
```

নিরীহ মনে হয়। কিন্তু `less`-এর ভেতরে `!bash` টাইপ করলে একটা shell খোলে, আর সেটা root! একই সমস্যা:

প্রোগ্রাম	কীভাবে root shell পাওয়া যায়
`less`, `more`, `man`	`!sh`

প্রোগ্রাম	কীভাবে root shell পাওয়া যায়
`vi`, `vim`	`:!sh` বা `:shell`
`find`	`-exec /bin/sh \;`
`awk`	`'BEGIN {system("/bin/sh")}'`
`tar`	`--checkpoint-action=exec=/bin/sh`
`python`, `perl`, `ruby`	সরাসরি কোড চালানো
`cp`, `tee`, `dd`	যেকোনো ফাইলে লেখা, যেমন `/etc/sudoers` বা `/etc/shadow` ওভাররাইট
`chmod`, `chown`	যেকোনো ফাইলের পারমিশন বদলানো
`apt`, `dnf`	ক্ষতিকর প্যাকেজ ইনস্টল বা hook চালানো
`systemctl` (কোনো সীমা ছাড়া)	নতুন service ফাইল দিয়ে যেকোনো কমান্ড
`bash`, `sh`, `su`	সরাসরি shell

এই ধরনের প্রোগ্রামের একটা বিখ্যাত তালিকা হলো **GTFOBins** (gtfobins.github.io)। sudo-তে কোনো কমান্ড দেওয়ার আগে সেখানে দেখে নেওয়া ভালো অভ্যাস।

নিরাপদ বিকল্প:

- লগ পড়ার জন্য: ইউজারকে `adm` (Debian/Ubuntu) গ্রুপে দাও, তাহলে sudo ছাড়াই `/var/log/syslog` পড়তে পারবে। `systemd-journal` গ্রুপে দিলে পুরো journal পড়তে পারবে।
- ফাইল এডিটের জন্য: `sudoedit` অনুমতি দাও (`karim ALL=(root) sudoedit /etc/nginx/nginx.conf`)।
- `NOEXEC:` Tag ব্যবহার করো, যা কমান্ডকে আরেকটা প্রোগ্রাম exec করতে বাধা দেয় (`karim ALL=(root) NOEXEC: /usr/bin/less /var/log/syslog`)। তবে এটা শুধু dynamically linked প্রোগ্রামে কাজ করে আর শতভাগ নিশ্চয়তা দেয় না।

ফাঁদ ২: wildcard

```
karim ALL=(root) /usr/bin/cat /var/log/*
```

karim চালাতে পারে: `sudo cat /var/log/../../etc/shadow`। `*` path-এর `/` এবং `..`-ও মিলিয়ে নেয়। আরও: `sudo cat /var/log/syslog /etc/shadow`, কারণ `*` স্পেসসহ বাকি সব আর্গুমেন্ট মেলায়।

ফাঁদ ৩: negation (`!`) দিয়ে নিষেধ

```
rahim ALL=(ALL) ALL, !/usr/bin/su, !/bin/bash
```

এটা কাজ করে না। rahim `cp /bin/bash /tmp/mybash` করে `sudo /tmp/mybash` চালাবে। "সব দাও, শুধু এগুলো বাদ" ধরনের নিয়ম মৌলিকভাবে ভাঙা যায়। সবসময় উল্টোটা করো: **শুধু যা দরকার তা-ই দাও** (allowlist)।

ফাঁদ ৪: ইউজার-লেখযোগ্য স্ক্রিপ্ট

```
deploy ALL=(root) NOPASSWD: /home/deploy/restart.sh
```

`deploy` ইউজার নিজেই `restart.sh`-এর ভেতরটা বদলাতে পারে, তাই কার্যত সে root। sudo-তে দেওয়া যেকোনো স্ক্রিপ্ট (এবং তার ডিরেক্টরি) **root-এর মালিকানায়** আর অন্যদের জন্য লেখার অযোগ্য হতে হবে, যেমন `/usr/local/sbin/restart-myapp` মোড `0755`, মালিক `root:root`।

একটা ভালো নির্দিষ্ট-কমান্ড উদাহরণ

```
# /etc/sudoers.d/30-support
Cmnd_Alias SUPPORT_CMDS = /usr/bin/systemctl status *, \
                          /usr/bin/systemctl restart myapp, \
                          /usr/bin/journalctl -u myapp *
%support ALL=(root) SUPPORT_CMDS
```

এখানেও `journalctl` ডিফল্টে pager হিসেবে `less` খোলে, আর `systemctl status`-ও। sudo-র অধীনে আধুনিক systemd "secure mode"-এ less চালায় (shell escape বন্ধ), তবুও সতর্কতার জন্য `--no-pager` যোগ করা ভালো। আর `systemctl status *`-এর wildcard বাড়তি আর্গুমেন্ট মিলায়, তাই নীতির দিক থেকে যত নির্দিষ্ট সম্ভব লেখো। নিরাপত্তা হলো খুঁটিনাটির খেলা।

১৮.১৩ `sudo -l` — আমি কী কী পারি?

```
$ sudo -l
[sudo] password for karim:
Matching Defaults entries for karim on web01:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bi
n,
    use_pty

User karim may run the following commands on web01:
    (root) /usr/bin/systemctl restart nginx, /usr/bin/systemctl reload nginx
```

পূর্ণ অ্যাডমিনের ক্ষেত্রে:

```
User rahim may run the following commands on web01:
    (ALL : ALL) ALL
```

কোনো অনুমতি না থাকলে:

```
Sorry, user guest may not run sudo on web01.
```

আরও কিছু রূপ:

```
$ sudo -ll                                  # লম্বা, বিস্তারিত ফরম্যাট
$ sudo -l /usr/bin/systemctl restart nginx  # নির্দিষ্ট কমান্ড অনুমোদিত?
/usr/bin/systemctl restart nginx
$ sudo -l -U karim                          # অ্যাডমিন হিসেবে অন্যের অনুমতি দেখো
```

নির্দিষ্ট কমান্ড অনুমোদিত হলে তার পূর্ণ path প্রিন্ট করে আর exit status 0 দেয়, না হলে কিছু প্রিন্ট না করে exit status 1।

> 💡 **টিপস:** নতুন সার্ভারে লগইন করে সবার আগে `sudo -l` চালাও, জেনে নাও তোমার কতটুকু ক্ষমতা আছে। নিরাপত্তা পরীক্ষকরাও (pentester) এটা সবার আগে চালায়, কারণ ভুল sudoers হলো privilege escalation-এর সবচেয়ে সাধারণ পথ।

১৮.১৪ timestamp timeout — পাসওয়ার্ড কতক্ষণ মনে থাকে

একবার পাসওয়ার্ড দিলে sudo কিছু সময়ের জন্য আর পাসওয়ার্ড চায় না। ডিফল্ট **১৫ মিনিট** (Ubuntu, Debian, Fedora, RHEL সবখানে, যদি না distro বদলে দেয়)।

ভেতরে কীভাবে: sudo একটা timestamp রেকর্ড রাখে `/run/sudo/ts/<username>` ফাইলে (কিছু পুরনো সিস্টেমে `/var/run/sudo/ts/` বা `/var/lib/sudo/ts/`)। `/run` হলো tmpfs (RAM-এ), তাই রিবুটে মুছে যায়।

ডিফল্টে timestamp প্রতিটি **terminal (tty) অনুযায়ী আলাদা** (`timestamp_type=tty`)। মানে একটা টার্মিনালে পাসওয়ার্ড দিলে পাশের নতুন টার্মিনাল ট্যাবে আবার চাইবে। এটা ইচ্ছাকৃত: একটা টার্মিনালে ক্ষতিকর কোনো প্রক্রিয়া চলতে থাকলে সে তোমার অন্য টার্মিনালের "খোলা দরজা" ব্যবহার করতে পারবে না।

```
টার্মিনাল ১ (pts/0)            টার্মিনাল ২ (pts/1)
$ sudo apt update                  $ sudo ls /root
[sudo] password: ****              [sudo] password:    ← আবার চাইছে, আলাদা tty
...
$ sudo systemctl restart x
(চাইল না, ১৫ মিনিটের মধ্যে)
```

বদলানো

```
Defaults timestamp_timeout=5        # ৫ মিনিট
Defaults timestamp_timeout=0        # প্রতিবার পাসওয়ার্ড চাও (সবচেয়ে কড়া)
Defaults timestamp_timeout=-1       # কখনো মেয়াদ শেষ হবে না (সুপারিশ করা হয় না)
Defaults:rahim timestamp_timeout=30 # শুধু rahim-এর জন্য
Defaults timestamp_type=global      # সব terminal-এ একটাই timestamp
```

হাতে নিয়ন্ত্রণ:

কমান্ড	কাজ
`sudo -v`	পাসওয়ার্ড দিয়ে timestamp নবায়ন (আবার ১৫ মিনিট)
`sudo -k`	এই session-এর timestamp বাতিল
`sudo -K`	এই ইউজারের সব timestamp মুছে ফেলো
`sudo -n true`	পাসওয়ার্ড ছাড়া এখনই sudo চলবে কি না যাচাই (exit status দিয়ে)

> 💡 **টিপস:** কম্পিউটার রেখে চা খেতে উঠলে `sudo -k` চালাও (আর স্ক্রিন লক করো: GNOME-এ `Super+L`)। লম্বা ইনস্টল স্ক্রিপ্ট চালানোর আগে `sudo -v` দিয়ে রাখলে মাঝপথে পাসওয়ার্ড চেয়ে আটকে থাকবে না।

১৮.১৫ sudo লগ — কে কী করেছে

sudo প্রতিটি সফল ও ব্যর্থ চেষ্টা syslog-এ পাঠায় (`authpriv` facility)। কোথায় জমা হয়:

Distro	লগ ফাইল
Debian, Ubuntu	`/var/log/auth.log`
Fedora, RHEL, Rocky	`/var/log/secure`
সব systemd সিস্টেম	systemd journal (`journalctl`)

> 📝 **মনে রাখো:** Fedora-র নতুন ভার্সনে ডিফল্টে rsyslog ইনস্টল নাও থাকতে পারে, তখন `/var/log/secure` থাকবে না, শুধু journal। লগ ব্যবস্থা নিয়ে পুরোটা অধ্যায় ২৮-এ।

লগ দেখা

```
$ sudo grep sudo /var/log/auth.log | tail -3
2026-11-12T10:15:01.482913+06:00 web01 sudo:    rahim : TTY=pts/0 ; PWD=/home/rahim ;
USER=root ; COMMAND=/usr/bin/apt update
2026-11-12T10:15:01.486120+06:00 web01 sudo: pam_unix(sudo:session): session opened for
user root(uid=0) by rahim(uid=1000)
2026-11-12T10:15:09.031277+06:00 web01 sudo: pam_unix(sudo:session): session closed for
user root
```

(Ubuntu 24.04 থেকে `auth.log`-এ এই ISO-8601 সময় ফরম্যাট, পুরনো সংস্করণে `Nov 12 10:15:01` ধরনের ছিল।)

একটা লাইন ভেঙে দেখি:

অংশ	মানে
`rahim`	আসল ইউজার, যে sudo চালিয়েছে
`TTY=pts/0`	কোন terminal থেকে
`PWD=/home/rahim`	তখন কোন ডিরেক্টরিতে ছিল
`USER=root`	কার হিসেবে কমান্ড চলেছে
`COMMAND=/usr/bin/apt update`	ঠিক কোন কমান্ড, পূর্ণ path সহ

ব্যর্থ চেষ্টা:

```
2026-11-12T10:20:44.112233+06:00 web01 sudo:    karim : user NOT in sudoers ; TTY=pts/1
; PWD=/home/karim ; USER=root ; COMMAND=/usr/bin/cat /etc/shadow
2026-11-12T10:21:02.554411+06:00 web01 sudo:    karim : 3 incorrect password attempts ;
TTY=pts/1 ; PWD=/home/karim ; USER=root ; COMMAND=/usr/bin/apt update
2026-11-12T10:22:13.998877+06:00 web01 sudo:    karim : command not allowed ; TTY=pts/1
; PWD=/home/karim ; USER=root ; COMMAND=/usr/bin/bash
```

journal থেকে:

```
$ journalctl -t sudo --since today          # sudo-র নিজের বার্তা
$ journalctl _COMM=sudo -n 20               # sudo process থেকে আসা শেষ ২০টি
$ journalctl -t sudo -g "NOT in sudoers"    # নির্দিষ্ট প্যাটার্ন খোঁজা (-g = grep)
```

পুরো session রেকর্ড: `log_output` ও `sudoreplay`

শুধু কমান্ড না, স্ক্রিনে কী হয়েছিল তাও রেকর্ড করা যায় (অডিট-কঠোর পরিবেশে, যেমন ব্যাংকিং):

```
Defaults log_input, log_output
Defaults iolog_dir=/var/log/sudo-io
```

তারপর:

```
$ sudo sudoreplay -l                # রেকর্ড করা session-এর তালিকা
$ sudo sudoreplay 000001            # ভিডিওর মতো ঠিক আগের মতো আবার দেখো
```

> ⚠️ **সতর্কতা:** `log_input` চালু করলে টাইপ করা **পাসওয়ার্ডও** (যেমন sudo session-এর ভেতর `mysql -p`) রেকর্ড হতে পারে। সংবেদনশীল তথ্যের লগ সাবধানে সুরক্ষিত রাখতে হবে।

মনে রাখো: `sudo -i` বা `sudo -s` দিয়ে root shell নিলে, লগে শুধু লেখা থাকবে `COMMAND=/bin/bash`। এর ভেতরে কী কী চালানো হয়েছে sudo-র সাধারণ লগে আসবে না (log_output না থাকলে)। এটা root shell এড়িয়ে আলাদা আলাদা `sudo command` চালানোর আরেকটা কারণ।

১৮.১৬ environment ও sudo: কেন আমার `PATH` কাজ করছে না?

```
$ ls ~/bin
mytool
$ mytool
Hello!
$ sudo mytool
sudo: mytool: command not found
```

কারণ sudo তোমার `PATH` না নিয়ে `secure_path` ব্যবহার করে। এটা ইচ্ছাকৃত নিরাপত্তা: কেউ তোমার `PATH`-এর শুরুতে `/tmp/evil` ঢুকিয়ে `ls` নামে ম্যালওয়্যার রাখলে, sudo সেটা চালাত না।

সমাধান:

```
$ sudo ~/bin/mytool                  # পূর্ণ path দাও
$ sudo "$(command -v mytool)"        # path খুঁজে দাও
```

একইভাবে `env_reset`-এর কারণে বেশির ভাগ চলক মুছে যায়। প্রক্সির মতো কিছু রাখতে হলে:

```
$ sudo --preserve-env=http_proxy,https_proxy apt update
```

অথবা sudoers-এ `Defaults env_keep += "http_proxy https_proxy no_proxy"`। পুরো environment রাখার `sudo -E` সম্ভব হলে এড়িয়ে চলো, ক্ষতিকর চলক (যেমন `LD_PRELOAD` বাদে অন্য অনেক কিছু) ঢুকে যেতে পারে। (sudo নিজে কিছু বিপজ্জনক চলক যেমন `LD_PRELOAD` সবসময় সরিয়ে ফেলে।)

sudo কমান্ডের ভেতর কিছু চলক নিজে থেকেই সেট হয়, স্ক্রিপ্টে কাজের:

চলক	মান
`SUDO_USER`	আসল ইউজারের নাম
`SUDO_UID`, `SUDO_GID`	আসল ইউজারের UID/GID
`SUDO_COMMAND`	যে কমান্ড চালানো হয়েছে

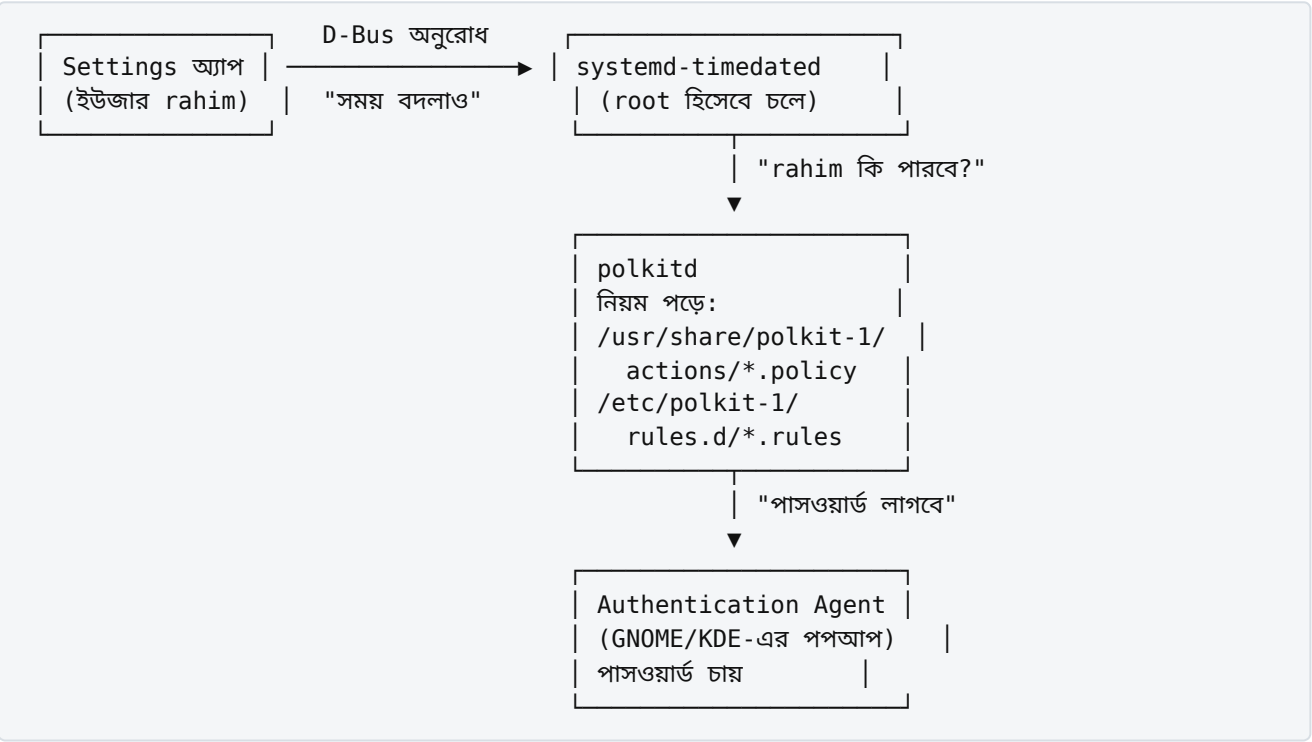
```
$ sudo bash -c 'echo "আসলে চালিয়েছে: $SUDO_USER, এখন আমি: $(whoami)"'
আসলে চালিয়েছে: rahim, এখন আমি: root
```

১৮.১৭ polkit ও `pkexec` — GUI-র প্রশাসনিক ক্ষমতা

desktop-এ সফটওয়্যার আপডেট, Wi-Fi কনফিগ বা তারিখ-সময় বদলাতে গেলে একটা ছোট উইন্ডো এসে পাসওয়ার্ড চায়। এর পেছনে sudo নেই, আছে **polkit** (আগের নাম PolicyKit)।

কেন আলাদা ব্যবস্থা?

sudo-র মডেল: "পুরো একটা প্রোগ্রাম root হিসেবে চালাও"। GUI-তে এটা খারাপ, কারণ পুরো Settings অ্যাপকে root বানালে বিশাল কোড root ক্ষমতা পায়। polkit-এর মডেল ভিন্ন: GUI অ্যাপ সাধারণ ইউজার হিসেবেই চলে। সে একটা ছোট, বিশেষায়িত root **service** (যেমন `systemd-timedated`, `NetworkManager`, `packagekitd`)-কে D-Bus দিয়ে অনুরোধ পাঠায়: "সময় বদলাও"। service জিজ্ঞেস করে polkit-কে: "এই ইউজার কি `org.freedesktop.timedate1.set-time` কাজটা করতে পারবে?"



polkit-এর মূল অংশ:

অংশ	কাজ
Action	একটা নির্দিষ্ট কাজের নাম, যেমন `org.freedesktop.login1.reboot`। সংজ্ঞা `/usr/share/polkit-1/actions/*.policy` (XML) ফাইলে
Rules	কে কোন action করতে পারবে, JavaScript-এ লেখা। থাকে `/etc/polkit-1/rules.d/` (অ্যাডমিনের) আর `/usr/share/polkit-1/rules.d/` (প্যাকেজের)
polkitd	daemon যে সিদ্ধান্ত নেয়
Authentication agent	পাসওয়ার্ড চাওয়ার পপআপ (GUI) বা টেক্সট এজেন্ট

উত্তর তিন রকম হতে পারে: `yes` (সরাসরি অনুমতি), `no` (নিষেধ), `auth_self` (নিজের পাসওয়ার্ড দাও) বা `auth_admin` (অ্যাডমিনের পাসওয়ার্ড লাগবে)। এগুলোর `_keep` সংস্করণও আছে, যা কিছুক্ষণ মনে রাখে।

polkit "অ্যাডমিন" কাকে ধরে? Ubuntu-তে `sudo` গ্রুপ, Fedora/RHEL-এ `wheel` গ্রুপের সদস্যকে।

একটা polkit নিয়ম উদাহরণ

`devops` গ্রুপকে পাসওয়ার্ড ছাড়া service রিস্টার্ট করতে দেওয়া:

```
// /etc/polkit-1/rules.d/50-devops-units.rules
polkit.addRule(function(action, subject) {
    if (action.id == "org.freedesktop.systemd1.manage-units" &&
        subject.isInGroup("devops")) {
        return polkit.Result.YES;
    }
});
```

(Debian 12-এর polkit 122 সংস্করণে এই JavaScript `.rules` ফরম্যাট সমর্থিত। খুব পুরনো Debian/Ubuntu-তে `.pkla` ফরম্যাট ছিল।)

polkit কমান্ড

কমান্ড	কাজ
`pkexec command`	polkit দিয়ে যাচাই করে command root হিসেবে চালাও
`pkexec --user karim command`	karim হিসেবে চালাও
`pkaction`	সব নিবন্ধিত action-এর তালিকা
`pkaction --action-id ID --verbose`	একটা action-এর বিস্তারিত (কে পারবে, ডিফল্ট কী)
`pkcheck`	একটা process কোনো action-এর অনুমতি পায় কি না যাচাই
`pkttyagent`	টেক্সট টার্মিনালের জন্য authentication agent

```
$ pkaction --action-id org.freedesktop.login1.reboot --verbose
org.freedesktop.login1.reboot:
  description:       Reboot the system
  message:           Authentication is required to reboot the system.
  vendor:            The systemd Project
  vendor_url:        https://systemd.io
  icon:
  implicit any:      auth_admin_keep
  implicit inactive: auth_admin_keep
  implicit active:   yes
```

`implicit active: yes` মানে যে ইউজার সামনে বসে লগইন করা (active session), সে পাসওয়ার্ড ছাড়াই রিবুট করতে পারে। SSH দিয়ে ঢুকলে (any) অ্যাডমিন পাসওয়ার্ড লাগবে।

`pkexec`

```
$ pkexec visudo
==== AUTHENTICATING FOR org.freedesktop.policykit.exec ====
Authentication is needed to run `/usr/sbin/visudo' as the super user
Authenticating as: Rahim Uddin,,, (rahim)
```

```
Password:
==== AUTHENTICATION COMPLETE ====
```

desktop-এ থাকলে পপআপ আসবে। `pkexec` -ও sudo-র মতো setuid root প্রোগ্রাম, কিন্তু sudoers পড়ে না, polkit পড়ে। এজন্য sudoers ভেঙে গেলে এটা উদ্ধারের উপায় হতে পারে।

> ⚠️ **সতর্কতা:** ২০২২ সালে `pkexec` -এ "PwnKit" (CVE-2021-4034) নামে একটা গুরুতর ত্রুটি পাওয়া গিয়েছিল, যা দিয়ে যেকোনো সাধারণ ইউজার root হতে পারত। ১২ বছর ধরে ত্রুটিটা ছিল। এখন সব distro-তে ঠিক করা, কিন্তু এটা দেখায় setuid প্রোগ্রাম কতটা স্পর্শকাতর। সার্ভারে GUI না থাকলে অনেকে polkit প্যাকেজই রাখে না বা `pkexec` -এর setuid বিট সরিয়ে দেয়। সিস্টেম আপডেট রাখো (অধ্যায় ৩০)।

আরও কিছু বিকল্প (জেনে রাখো)

- **`doas`** : OpenBSD থেকে আসা খুব ছোট sudo-বিকল্প। কনফিগ `/etc/doas.conf` , যেমন `permit persist :wheel` । Debian/Ubuntu-তে `opendoas` প্যাকেজ হিসেবে পাওয়া যায়, ডিফল্ট নয়।
- **`run0`** : systemd 256 থেকে আসা নতুন টুল, setuid ছাড়াই polkit দিয়ে কাজ করে (Fedora 40+ এ পাওয়া যায়, Ubuntu 24.04-এর systemd 255-এ নেই)।
- **`sudo-rs`** : Rust-এ নতুন করে লেখা sudo, মেমরি-নিরাপদ। Ubuntu 25.10 থেকে ডিফল্ট হিসেবে আসছে। ব্যবহারকারীর জন্য কমান্ড প্রায় একই।

১৮.১৮ Least Privilege — ন্যূনতম ক্ষমতার নীতি

Principle of Least Privilege (PoLP): প্রতিটি ইউজার, প্রোগ্রাম আর process ঠিক ততটুকু ক্ষমতা পাবে যতটুকু তার কাজের জন্য দরকার, তার এক বিন্দু বেশি নয়, আর ঠিক যতক্ষণ দরকার ততক্ষণই।

উপমা: হোটেলের কী-কার্ড। অতিথির কার্ড শুধু তার রুম আর জিম খোলে, তাও চেক-আউট পর্যন্ত। ক্লিনারের কার্ড তার ফ্লোরের রুমগুলো খোলে, শুধু শিফটের সময়। কারো কার্ডই ভল্ট খোলে না, শুধু ম্যানেজারেরটা খোলে।

বাস্তবে কীভাবে মানবে

ক্ষেত্র	ভুল পদ্ধতি	সঠিক পদ্ধতি
দৈনন্দিন কাজ	root হয়ে লগইন	সাধারণ ইউজার, দরকারে `sudo command`
দলের সদস্য	সবাইকে `sudo` গ্রুপে	নির্দিষ্ট কাজের জন্য `Cmnd_Alias`
লগ পড়া	sudo দিয়ে `less`	`adm` বা `systemd-journal` গ্রুপ
ওয়েব অ্যাপ	root হিসেবে চলে	নিজস্ব ইউজার (`www-data`), systemd-এর `User=`
পোর্ট ৮০	অ্যাপ root হিসেবে	`CAP_NET_BIND_SERVICE` বা reverse proxy (nginx)
deploy স্ক্রিপ্ট	`NOPASSWD: ALL`	`NOPASSWD:` শুধু নির্দিষ্ট root-মালিকানাধীন স্ক্রিপ্টে

ক্ষেত্র	ভুল পদ্ধতি	সঠিক পদ্ধতি
SSH	root লগইন চালু	`PermitRootLogin no`, key দিয়ে সাধারণ ইউজার (অধ্যায় ৩৮)
ফাইল এডিট	`sudo vim`	`sudoedit`
অস্থায়ী অধিকার	গ্রুপে যোগ করে ভুলে যাওয়া	কাজ শেষে সরানো, নিয়মিত অডিট
root shell	সারাদিন খোলা	কাজ শেষে সাথে সাথে `exit`

নিয়মিত অডিট চেকলিস্ট

```
$ getent group sudo wheel                # কারা অ্যাডমিন গ্রুপে আছে
$ awk -F: '$3==0 {print $1}' /etc/passwd # UID 0 আর কে আছে (শুধু root থাকা উচিত)
$ sudo grep -rn NOPASSWD /etc/sudoers /etc/sudoers.d/   # পাসওয়ার্ডহীন নিয়ম
$ sudo visudo -c                          # সব ফাইল ঠিক আছে?
$ ls -l /etc/sudoers.d/                   # অচেনা ফাইল আছে কি?
$ sudo find / -xdev -perm -4000 -type f 2>/dev/null   # setuid প্রোগ্রামের তালিকা
```

পুরো নিরাপত্তা চেকলিস্ট আর হার্ডেনিং অধ্যায় ৪৫-এ।

১৮.১৯ কীবোর্ড শর্টকাট ও দ্রুত কৌশল

শর্টকাট / কৌশল	কাজ
`sudo !!`	আগের কমান্ডটা sudo দিয়ে আবার চালাও
`Ctrl+A` তারপর `sudo ` টাইপ	বর্তমান লাইনের শুরুতে গিয়ে sudo যোগ করো
`Alt+.` (বা `Esc` তারপর `.`)	আগের কমান্ডের শেষ আর্গুমেন্ট বসাও, যেমন `sudo nano ` + `Alt+.`
`Ctrl+D` বা `exit`	root shell (বা `su`) থেকে বের হও
`Ctrl+C` (পাসওয়ার্ড প্রম্পটে)	sudo বাতিল করো
`Ctrl+U` (পাসওয়ার্ড প্রম্পটে)	এ পর্যন্ত টাইপ করা পাসওয়ার্ড মুছে নতুন করে শুরু করো
`Esc` `Esc` (zsh + Oh My Zsh-এর `sudo` প্লাগইন)	লাইনের শুরুতে `sudo` যোগ/বাদ
`Super+L` (GNOME), `Super+L` বা `Ctrl+Alt+L` (KDE)	উঠে যাওয়ার আগে স্ক্রিন লক করো
`visudo`-তে `e` / `x` / `Q`	ভুল হলে: আবার এডিট / সেভ না করে বের হও / (বিপজ্জনক) জোর করে সেভ
nano-তে (visudo): `Ctrl+O` তারপর `Enter`, `Ctrl+X`	সেভ, বের হওয়া
vi-তে (visudo): `Esc` `:wq` `Enter` / `:q!`	সেভ করে বের হওয়া / সেভ না করে বের হওয়া

> 💡 **টিপস:** bash-এ নিজের "লাইনের শুরুতে sudo" শর্টকাট বানাতে পারো। `~/.bashrc`-এ যোগ করো: `bind '"\e\e": "\C-asudo \C-e"'`। তারপর `Esc` দুবার চাপলে লাইনের শুরুতে `sudo` বসে যাবে। (কিছু টার্মিনালে দুটো `Esc` অন্য কাজে লাগে, পরীক্ষা করে দেখো।) bash কাস্টমাইজেশন বিস্তারিত অধ্যায় ৪৪-এ।

হাতে-কলমে অনুশীলন

> ⚠️ **সতর্কতা:** এই ল্যাব একটা **ভার্চুয়াল মেশিন** বা পরীক্ষামূলক সিস্টেমে করো (অধ্যায় ৩)। শুরুর আগে একটা আলাদা টার্মিনালে `sudo -i` দিয়ে root shell খুলে রাখো, বিপদে কাজে লাগবে। উদাহরণ Ubuntu/Debian-এর, Fedora/RHEL-এ `sudo` গ্রুপের জায়গায় `wheel` ব্যবহার করো।

ল্যাব ১: নিজের পরিচয় ও ক্ষমতা জানো

```
$ whoami
$ id
$ sudo -l
$ sudo whoami
root
$ sudo id
uid=0(root) gid=0(root) groups=0(root)
$ ls -l /usr/bin/sudo /etc/sudoers
```

খেয়াল করো sudo-তে `s` বিট আর sudoers-এর `0440` পারমিশন।

ল্যাব ২: `su`, `sudo -s`, `sudo -i` তুলনা

```
$ cd /tmp
$ sudo -s
# pwd ; echo $HOME ; echo $SUDO_USER
# exit
$ sudo -i
# pwd ; echo $HOME ; echo $SUDO_USER
# exit
```

দুই ক্ষেত্রে `pwd`-এর ফল লিখে রাখো। এবার root-এর পাসওয়ার্ড লক আছে কি না দেখো:

```
$ sudo passwd -S root
$ su -
```

ল্যাব ৩: পরীক্ষামূলক ইউজার তৈরি

```
$ sudo useradd -m -s /bin/bash tester
$ sudo passwd tester
New password:
Retype new password:
passwd: password updated successfully
$ su - tester
Password:
tester$ sudo ls /root
[sudo] password for tester:
tester is not in the sudoers file.
tester$ exit
```

লগে চেষ্টাটা খুঁজে দেখো:

```
$ sudo grep tester /var/log/auth.log | tail -2     # Fedora/RHEL: /var/log/secure
$ journalctl -t sudo -n 5
```

ল্যাব ৪: নির্দিষ্ট কমান্ডের অনুমতি দাও

```
$ sudo visudo -f /etc/sudoers.d/50-tester
```

ভেতরে লেখো (path মিলিয়ে নিতে আগে `command -v systemctl` চালাও):

```
tester ALL=(root) /usr/bin/systemctl status cron, /usr/bin/systemctl restart cron
```

(Fedora/RHEL-এ service-এর নাম `crond`।) সেভ করে বের হও, তারপর:

```
$ sudo visudo -c
$ ls -l /etc/sudoers.d/50-tester
-r--r----- 1 root root 77 Nov 12 11:02 /etc/sudoers.d/50-tester
$ sudo -l -U tester
...
User tester may run the following commands on web01:
    (root) /usr/bin/systemctl status cron, /usr/bin/systemctl restart cron
```

tester হিসেবে পরীক্ষা করো:

```
$ su - tester
tester$ sudo systemctl restart cron        # কাজ করবে
tester$ sudo systemctl stop cron           # অনুমতি নেই
Sorry, user tester is not allowed to execute '/usr/bin/systemctl stop cron' as root on
web01.
tester$ exit
```

ল্যাব ৫: `.conf` ফাঁদ নিজের চোখে দেখো

```
$ sudo visudo -f /etc/sudoers.d/60-tester.conf
```

ভেতরে লেখো `tester ALL=(root) /usr/bin/whoami`, সেভ করো। তারপর:

```
$ sudo -l -U tester
```

`whoami` তালিকায় **নেই**, কারণ নামে `.` থাকায় ফাইলটা উপেক্ষিত। এবার নাম ঠিক করো:

```
$ sudo mv /etc/sudoers.d/60-tester.conf /etc/sudoers.d/60-tester
$ sudo -l -U tester
```

এখন আছে।

ল্যাব ৬: ভুল সিনট্যাক্স ও visudo-র সুরক্ষা

```
$ sudo visudo -f /etc/sudoers.d/70-broken
```

ইচ্ছা করে ভুল লেখো: `tester ALL=(root) whoami` (পূর্ণ path ছাড়া)। সেভ করলে `What now?` আসবে। `x` চাপো। নিশ্চিত হও ফাইলটা তৈরি হয়নি:

```
$ ls /etc/sudoers.d/
```

ল্যাব ৭: timestamp নিয়ে খেলা

```
$ sudo -k
$ sudo -n true && echo "পাসওয়ার্ড লাগবে না" || echo "পাসওয়ার্ড লাগবে"
পাসওয়ার্ড লাগবে
$ sudo -v
[sudo] password for rahim:
$ sudo -n true && echo "পাসওয়ার্ড লাগবে না" || echo "পাসওয়ার্ড লাগবে"
পাসওয়ার্ড লাগবে না
```

নতুন একটা টার্মিনাল ট্যাব খুলে আবার `sudo -n true` চালাও, tty আলাদা বলে পাসওয়ার্ড লাগবে।

এবার `/etc/sudoers.d/80-timeout` ফাইলে (visudo দিয়ে) লেখো `Defaults:tester timestamp_timeout=0`, তারপর tester হিসেবে পরপর দুবার `sudo systemctl status cron` চালাও। দুবারই পাসওয়ার্ড চাইবে।

ল্যাব ৮: redirection ফাঁদ

```
$ sudo echo "test" > /etc/lab-test.txt
bash: /etc/lab-test.txt: Permission denied
$ echo "test" | sudo tee /etc/lab-test.txt
```

```
test
$ cat /etc/lab-test.txt
test
$ sudo rm /etc/lab-test.txt
```

ল্যাব ৯: polkit অন্বেষণ

```
$ pkaction | wc -l
$ pkaction | grep -i reboot
$ pkaction --action-id org.freedesktop.login1.reboot --verbose
$ pkexec whoami
root
```

ল্যাব ১০: পরিষ্কার করো

```
$ sudo rm /etc/sudoers.d/50-tester /etc/sudoers.d/60-tester /etc/sudoers.d/80-timeout
$ sudo visudo -c
$ sudo userdel -r tester
```

সারসংক্ষেপ

কমান্ড / ফাইল	কাজ
`id`, `whoami`	বর্তমান পরিচয় (UID 0 = root)
`su - [user]`	target-এর পাসওয়ার্ড দিয়ে পূর্ণ login shell-এ ইউজার বদল
`su -c "cmd"`	অন্য ইউজার হিসেবে একটা কমান্ড
`sudo cmd`	নিজের পাসওয়ার্ড দিয়ে root হিসেবে একটা কমান্ড
`sudo -u user cmd`	নির্দিষ্ট ইউজার হিসেবে
`sudo -i`	root-এর login shell (`/root`-এ)
`sudo -s`	root shell, বর্তমান ডিরেক্টরিতে
`sudo !!`	আগের কমান্ড sudo দিয়ে
`sudo -l`, `sudo -l -U user`	অনুমতির তালিকা
`sudo -v` / `-k` / `-K`	timestamp নবায়ন / বাতিল / সব মোছা
`sudo -n`	পাসওয়ার্ড না চেয়ে চেষ্টা (স্ক্রিপ্টে)
`sudoedit file` / `sudo -e`	নিরাপদে root-এর ফাইল এডিট
`cmd | sudo tee file`	root-এর ফাইলে লেখা
`sudo visudo`	`/etc/sudoers` নিরাপদে এডিট
`sudo visudo -f /etc/sudoers.d/name`	ড্রপ-ইন ফাইল এডিট (নামে `.` নয়)

কমান্ড / ফাইল	কাজ
`sudo visudo -c`	সিনট্যাক্স যাচাই
`usermod -aG sudo/wheel user`	কাউকে অ্যাডমিন বানানো
`gpasswd -d user sudo`	অ্যাডমিন অধিকার সরানো
`user HOST=(RUNAS) [TAG:] CMDS`	sudoers নিয়মের গঠন
`NOPASSWD:`	পাসওয়ার্ড ছাড়া (সীমিত কমান্ডে)
`Defaults timestamp_timeout=N`	পাসওয়ার্ড মনে রাখার সময় (ডিফল্ট ১৫ মিনিট)
`/var/log/auth.log` / `/var/log/secure`	sudo লগ (Debian-পরিবার / RHEL-পরিবার)
`journalctl -t sudo`	journal থেকে sudo লগ
`sudoreplay`	রেকর্ড করা sudo session আবার দেখা
`pkexec cmd`	polkit দিয়ে root হিসেবে চালানো
`pkaction --action-id ID --verbose`	polkit action-এর নিয়ম দেখা
`/etc/polkit-1/rules.d/`	অ্যাডমিনের polkit নিয়ম

নিজেকে যাচাই করো

1. `su` আর `sudo` কার পাসওয়ার্ড চায়? এই পার্থক্য নিরাপত্তায় কী সুবিধা দেয়?
2. `su` আর `su -`-এর মধ্যে দুটো ব্যবহারিক পার্থক্য বলো।
3. `sudo -i` আর `sudo -s` চালানোর পর `pwd` আলাদা হয় কেন?
4. এই লাইনটা সহজ বাংলায় পড়ো: `%ops web01=(postgres) NOPASSWD: /usr/bin/psql`
5. `/etc/sudoers.d/admins.conf` নামে ফাইল তৈরি করে নিয়ম লিখলে, কিন্তু কাজ করছে না। কেন?
6. `sudo echo hi > /etc/test` কেন "Permission denied" দেয়? দুটো সমাধান লেখো।
7. কোনো ইউজারকে শুধু `sudo vim /etc/hosts` চালানোর অনুমতি দেওয়া কেন বিপজ্জনক? ভালো বিকল্প কী?
8. sudo-র ডিফল্ট timestamp timeout কত, আর প্রতিবার পাসওয়ার্ড চাওয়ার জন্য কী লিখবে?
9. Ubuntu আর RHEL-এ sudo-র লগ কোথায় পাবে? journal থেকে দেখার কমান্ড কী?
10. polkit আর sudo-র কাজের ধরনে মূল পার্থক্য কী? GUI-তে polkit কেন ভালো?

▼ উত্তর দেখো

1. `su` চায় **target ইউজারের** (সাধারণত root-এর) পাসওয়ার্ড, `sudo` চায় **নিজের** পাসওয়ার্ড। ফলে root পাসওয়ার্ড কাউকে জানাতে হয় না, প্রত্যেকের কাজ তার নিজের নামে লগ হয়, আর কারো অধিকার সরাতে শুধু তাকে গ্রুপ/sudoers থেকে সরালেই হয়।

2. (ক) `su -` target-এর home ডিরেক্টরিতে নিয়ে যায়, `su` বর্তমান ডিরেক্টরিতেই রাখে। (খ) `su -` login shell, environment রিসেট করে target-এর profile পড়ে; `su`-তে তোমার অনেক চলক

(যেমন `PATH`) থেকে যায়।

3. `sudo -i` login shell চালায়, যা target ইউজারের home (`/root`)-এ চলে যায়। `sudo -s` non-login shell, তাই বর্তমান ডিরেক্টরি বদলায় না।

4. "`ops` গ্রুপের সদস্যরা, শুধু `web01` নামের মেশিনে, `postgres` ইউজার হিসেবে, পাসওয়ার্ড ছাড়াই, `/usr/bin/psql` কমান্ড (যেকোনো আর্গুমেন্টসহ) চালাতে পারবে।"

5. `/etc/sudoers.d/`-এ যেসব ফাইলের নামে `.` থাকে (বা যা `~` দিয়ে শেষ হয়), sudo সেগুলো উপেক্ষা করে। নাম বদলে `admins` বা `10-admins` রাখো, পারমিশন `0440` নিশ্চিত করো, আর `sudo -l -U user` দিয়ে যাচাই করো।

6. redirection (`>`) করে তোমার নিজের shell, sudo নয়। sudo শুধু `echo`-কে root বানায়। সমাধান: `echo hi | sudo tee /etc/test` অথবা `sudo sh -c 'echo hi > /etc/test'`।

7. vim-এর ভেতর থেকে `:!bash` বা `:shell` দিয়ে root shell পাওয়া যায়, আবার `:e /etc/shadow` দিয়ে যেকোনো ফাইল খোলা যায়। ফলে কার্যত পূর্ণ root। ভালো বিকল্প: `tester ALL=(root) sudoedit /etc/hosts`, তাতে এডিটর ইউজারের নিজের নামে চলে।

8. ১৫ মিনিট। প্রতিবার চাওয়ার জন্য `Defaults timestamp_timeout=0` (visudo দিয়ে)। হাতে বাতিল করতে `sudo -k`।

9. Debian/Ubuntu: `/var/log/auth.log`; Fedora/RHEL: `/var/log/secure` (rsyslog থাকলে)। journal থেকে: `journalctl -t sudo` (বা `journalctl _COMM=sudo`)।

10. sudo পুরো একটা প্রোগ্রামকে root হিসেবে চালায়। polkit-এ অ্যাপ সাধারণ ইউজার হিসেবেই থাকে, শুধু নির্দিষ্ট একটা "action" (যেমন সময় বদলানো) একটা ছোট root service-কে D-Bus দিয়ে করতে বলে, আর polkit ঠিক করে সেটা অনুমোদিত কি না। GUI-তে বিশাল অ্যাপকে root বানাতে হয় না বলে এটা নিরাপদ, আর নিয়ম action ধরে সূক্ষ্মভাবে দেওয়া যায়।

অধ্যায় ১৯: ইনপুট/আউটপুট রিডাইরেকশন ও পাইপ

এই অধ্যায়ে যা শিখবে

- প্রতিটি প্রোগ্রামের তিনটি "জন্মগত" চ্যানেল: **stdin (0)**, **stdout (1)**, **stderr (2)** — এবং **file descriptor** আসলে কী
- `>`, `>>`, `<`, `2>`, `2>&1`, `&>`, `&>>`, `>|`, `<>` — প্রতিটি রিডাইরেকশন অপারেটর কী করে এবং কেন ক্রম (order) গুরুত্বপূর্ণ
- `/dev/null` — লিনাক্সের "ব্ল্যাক হোল", আর তার ভাইবোন `/dev/zero`, `/dev/stdin`, `/dev/fd/N`
- **here document** (`<<EOF`, `<<-EOF`, `<<'EOF'`) ও **here string** (`<<<`)
- **pipe** (`|`) — ভেতরে kernel কীভাবে pipe buffer দিয়ে দুটি প্রসেসকে জোড়া দেয়, `SIGPIPE`, `PIPESTATUS`, `pipefail`
- `tee` — আউটপুট একসাথে স্ক্রিনে ও ফাইলে, এবং `sudo tee` কৌশল
- কমান্ড চেইনিং: `;`, `&&`, `||`
- **subshell** `( )` বনাম **group** `{ }`
- **command substitution** `$( )` ও backtick
- **process substitution** `<( )` ও `>( )`
- **named pipe** (`mkfifo`)
- **exit status** `$?` — কোন সংখ্যার কী মানে
- `exec` দিয়ে স্থায়ী রিডাইরেকশন ও নিজের file descriptor খোলা

১৯.১ বড় ছবি: লিনাক্সের "সবকিছুই ফাইল" দর্শন

লিনাক্সের (আসলে Unix-এর) সবচেয়ে সুন্দর ধারণাগুলোর একটি হলো: **ছোট ছোট প্রোগ্রাম, প্রত্যেকে একটা কাজ ভালোভাবে করে, আর তাদের জোড়া লাগিয়ে বড় কাজ করা যায়।** এই "জোড়া লাগানোর আঠা" হলো রিডাইরেকশন ও পাইপ।

একটা উপমা দিই। ধরো তোমার বাসার পানির লাইন। প্রতিটি কল (প্রোগ্রাম) এর একটা **ইনলেট** (পানি ঢোকে), একটা **আউটলেট** (পরিষ্কার পানি বের হয়) আর একটা **ড্রেন** (ময়লা পানি বের হয়)। প্লাম্বার (shell) চাইলে:

- আউটলেটের পাইপ বালতিতে (ফাইলে) ঘুরিয়ে দিতে পারে — **রিডাইরেকশন**
- এক কলের আউটলেট অন্য কলের ইনলেটে জুড়ে দিতে পারে — **পাইপ**
- ড্রেনের পানি নর্দমায় ফেলে দিতে পারে — **`/dev/null`**

প্রোগ্রাম নিজে জানেই না তার পানি কোথায় যাচ্ছে! সে শুধু "আউটলেটে লেখো" জানে। এটাই রিডাইরেকশনের জাদু — **প্রোগ্রাম না বদলিয়েই তার ইনপুট/আউটপুট বদলানো।**

> 📝 মনে রাখো: রিডাইরেকশন ও পাইপ সাজায় **shell** (bash), প্রোগ্রাম নয়। প্রোগ্রাম চালু হওয়ার *আগেই* shell সব "পাইপলাইন" ফিট করে রাখে।

১৯.২ File Descriptor: প্রোগ্রামের হাতে থাকা টিকিট

File descriptor কী?

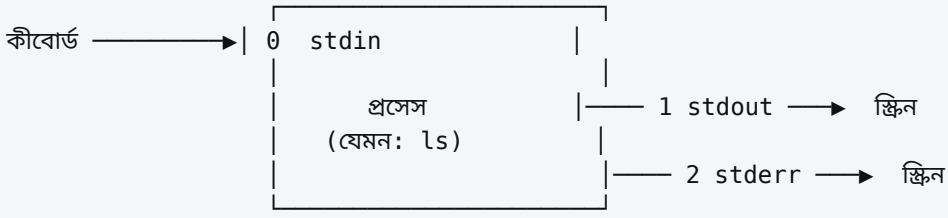
File descriptor (FD) হলো একটা ছোট পূর্ণসংখ্যা (0, 1, 2, 3 ...) যা দিয়ে একটা প্রসেস kernel-কে বলে "এই খোলা ফাইলটায় পড়ো/লেখো।" যখন কোনো প্রোগ্রাম ফাইল খোলে (`open()` system call), kernel একটা নম্বর ফেরত দেয় — ঠিক যেমন ব্যাংকে টোকেন নম্বর পাও। পরে শুধু টোকেন দেখালেই চলে।

এখানে "ফাইল" মানে শুধু ডিস্কের ফাইল না — টার্মিনাল, pipe, socket, device সবকিছুই FD দিয়ে পড়া-লেখা হয়। এজন্যই বলা হয় লিনাক্সে "everything is a file"।

তিনটি স্ট্যান্ডার্ড স্ট্রিম

প্রতিটি প্রসেস জন্ম থেকেই তিনটি FD খোলা অবস্থায় পায়:

FD	নাম	পূর্ণ রূপ	ডিফল্টভাবে কোথায় যুক্ত	কাজ
0	stdin	standard input	কীবোর্ড (টার্মিনাল)	প্রোগ্রাম এখান থেকে ইনপুট পড়ে
1	stdout	standard output	স্ক্রিন (টার্মিনাল)	স্বাভাবিক ফলাফল এখানে লেখে
2	stderr	standard error	স্ক্রিন (টার্মিনাল)	এরর ও ডায়াগনস্টিক মেসেজ এখানে লেখে



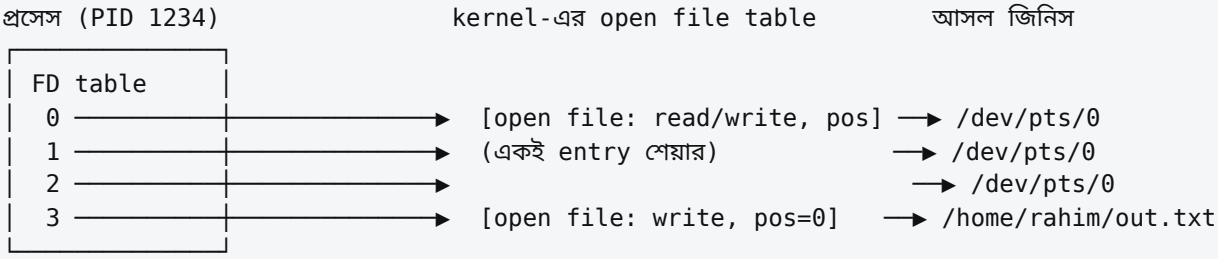
কেন stdout আর stderr আলাদা? কল্পনা করো তুমি `ls` এর আউটপুট ফাইলে রাখছ। যদি এরর মেসেজও একই জায়গায় যেত, তাহলে তোমার ডেটা ফাইলে "No such file or directory" জাতীয় আবর্জনা মিশে যেত। আলাদা চ্যানেল থাকায় ডেটা এক জায়গায়, সমস্যার খবর আরেক জায়গায় — দুটো আলাদাভাবে সামলানো যায়।

Under the hood: FD table

kernel প্রতিটি প্রসেসের জন্য একটা **file descriptor table** রাখে। তুমি নিজেই দেখতে পারো:

```
$ ls -l /proc/$$/fd
total 0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 0 -> /dev/pts/0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 1 -> /dev/pts/0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 2 -> /dev/pts/0
lrwx------ 1 rahim rahim 64 Nov 12 10:00 255 -> /dev/pts/0
```

এখানে `$$` মানে বর্তমান shell-এর PID। দেখো, 0, 1, 2 তিনটাই `/dev/pts/0` — অর্থাৎ তোমার টার্মিনাল। (255 হলো bash-এর নিজের ব্যবহারের জন্য রাখা একটা FD।) `/proc` সম্পর্কে বিস্তারিত অধ্যায় ৪৭-এ।



রিডাইরেকশন মানে আসলে এই টেবিলের কোনো সারির "তীর" অন্য দিকে ঘুরিয়ে দেওয়া। shell এটা করে `fork()` এর পর আর `exec()` এর আগে, `open()` ও `dup2()` system call দিয়ে। যেমন `ls > out.txt` চালালে shell মোটামুটি এই কাজ করে:

```
1. fork()                      → shell-এর একটা কপি (child) তৈরি
2. child-এ: fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC)   → ধরো fd=3
3. child-এ: dup2(3, 1)         → FD 1 এখন out.txt-কে দেখায়
4. child-এ: close(3)
5. child-এ: exec("ls")         → ls চালু হয়, সে ভাবে স্বাভাবিকভাবেই FD 1-এ লিখছে
```

`ls` জানেই না যে তার stdout এখন একটা ফাইল! (fork/exec সম্পর্কে আরও জানতে অধ্যায় ২৪ দেখো।)

১৯.৩ আউটপুট রিডাইরেকশন: `>` ও `>>`

`>` — ফাইলে লেখো (ওভাররাইট)

Syntax: `command > file` (আসলে এটা `command 1> file`-এর সংক্ষিপ্ত রূপ)

```
$ echo "হ্যালো লিনাক্স" > greeting.txt
$ cat greeting.txt
হ্যালো লিনাক্স
```

- ফাইল না থাকলে **তৈরি হবে**।
- ফাইল থাকলে **আগের সব লেখা মুছে যাবে** (truncate), তারপর নতুন লেখা বসবে।

```
$ echo "প্রথম লাইন" > note.txt
$ echo "দ্বিতীয় লাইন" > note.txt
```

```
$ cat note.txt
দ্বিতীয় লাইন
```

প্রথম লাইন হারিয়ে গেল!

> ⚠️ সতর্কতা: `>` কোনো প্রশ্ন না করেই ফাইল খালি করে দেয়। ভুলে `> important.conf` লিখলে পুরো কনফিগ শেষ। আর ফাইল খালি হয় **কমান্ড চালু হওয়ার আগেই** — তাই কমান্ড ভুল হলেও ক্ষতি হয়ে যায়।

সবচেয়ে কুখ্যাত ভুল: একই ফাইল পড়া ও লেখা

```
$ sort names.txt > names.txt      # ❌ ভুল!
$ cat names.txt
$                                  # ফাইল খালি!
```

কেন? shell প্রথমে `names.txt` কে `>` দিয়ে খোলে (তখনই খালি হয়ে যায়), তারপর `sort` চালু হয়ে খালি ফাইল পড়ে। সমাধান:

```
$ sort names.txt > names.sorted && mv names.sorted names.txt
$ sort -o names.txt names.txt      # sort-এর নিজস্ব -o অপশন নিরাপদ
```

`noclobber` — দুর্ঘটনা থেকে বাঁচার সুইচ

```
$ set -o noclobber          # চালু করো
$ echo "test" > note.txt
bash: note.txt: cannot overwrite existing file
$ echo "test" >| note.txt   # জেনেশুনে ওভাররাইট করতে চাইলে >|
$ set +o noclobber          # বন্ধ করো
```

অপারেটর	কাজ
`set -o noclobber` (বা `set -C`)	`>` দিয়ে বিদ্যমান ফাইল ওভাররাইট নিষেধ
`>|`	`>`
`set +o noclobber`	আবার স্বাভাবিক অবস্থায় ফেরা

> 💡 টিপস: `~/.bashrc`-এ `set -o noclobber` যোগ করে রাখলে অনেক দুর্ঘটনা এড়ানো যায় (অধ্যায় ৪৪)।

`>>` — ফাইলের শেষে যোগ করো (append)

```
$ echo "সকাল ৯টা: সার্ভার চালু" >> diary.log
$ echo "সকাল ১০টা: ব্যাকআপ শেষ" >> diary.log
$ cat diary.log
```

```
সকাল ৯টা: সার্ভার চালু
সকাল ১০টা: ব্যাকআপ শেষ
```

`>>` ফাইল খোলে `O_APPEND` ফ্ল্যাগ দিয়ে — kernel প্রতিটি write-এর আগে লেখার জায়গা ফাইলের শেষে নিয়ে যায়। তাই একাধিক প্রসেস একই লগ ফাইলে `>>` দিয়ে লিখলেও একে অপরের লেখা ওভাররাইট করে না।

খালি ফাইল তৈরি বা ফাইল খালি করার কৌশল

```
$ > empty.txt          # ফাইল তৈরি বা খালি (bash-এ কাজ করে)
$ : > app.log          # ':' কিছু করে না, শুধু রিডাইরেকশন হয় — বেশি portable
$ truncate -s 0 app.log   # আরেক উপায়
```

> 💡 টিপস: চলমান সার্ভারের বড় লগ ফাইল `rm` করলে ডিস্ক স্পেস ফেরত আসে না (প্রসেস তখনো ফাইলটা খোলা রেখেছে — অধ্যায় ১১ দেখো)। বরং `: > app.log` দিয়ে খালি করো।

১৯.৪ ইনপুট রিডাইরেকশন: `<`

Syntax: `command < file` (= `command 0< file`)

stdin কীবোর্ডের বদলে ফাইল থেকে আসে।

```
$ wc -l < /etc/passwd
42
$ wc -l /etc/passwd
42 /etc/passwd
```

পার্থক্য লক্ষ করো: দ্বিতীয় ক্ষেত্রে `wc` নিজে ফাইল খুলেছে তাই নাম জানে; প্রথম ক্ষেত্রে shell ফাইল খুলে দিয়েছে, `wc` শুধু একটা "ধারা" পেয়েছে — নাম জানে না। স্ক্রিপ্টে শুধু সংখ্যা দরকার হলে `<` কাজের।

```
$ tr 'a-z' 'A-Z' < notes.txt      # tr নিজে ফাইল নাম নেয় না, তাই < লাগবেই
```

`<>` — পড়া ও লেখা দুটোর জন্য খোলা

```
$ exec 3<> /tmp/data.txt     # FD 3 পড়া-লেখার জন্য খোলা (না থাকলে তৈরি হয়)
```

এটা কম ব্যবহৃত, তবে device বা socket নিয়ে কাজে লাগে (১৯.১৫-তে আরও)।

১৯.৫ এরর রিডাইরেকশন: `2>`, `2>&1`, `&>`

`2>` — শুধু এরর আলাদা করো

```
$ ls /etc/hostname /nai
ls: cannot access '/nai': No such file or directory
/etc/hostname

$ ls /etc/hostname /nai 2> errors.txt
/etc/hostname
$ cat errors.txt
ls: cannot access '/nai': No such file or directory
```

stdout স্ক্রিনে রইল, stderr ফাইলে গেল। দুটো আলাদা ফাইলেও পাঠানো যায়:

```
$ ls /etc/hostname /nai > ok.txt 2> err.txt
```

`2>>` দিয়ে এরর append করা যায়।

`2>&1` — stderr-কে stdout যেখানে যাচ্ছে সেখানে পাঠাও

`2>&1` পড়ো এভাবে: "FD 2-কে FD 1-এর **বর্তমান** গন্তব্যের কপি বানাও।" `&` না দিলে (`2>1`) shell ভাবত "1" নামের একটা ফাইল!

```
$ ls /etc/hostname /nai > all.txt 2>&1
$ cat all.txt
ls: cannot access '/nai': No such file or directory
/etc/hostname
```

⚠️ ক্রম (order) কেন গুরুত্বপূর্ণ

shell রিডাইরেকশনগুলো **বাম থেকে ডানে** একটার পর একটা প্রয়োগ করে। এটাই সবচেয়ে বেশি বিভ্রান্তি সৃষ্টি করে।

ঠিক: `cmd > file 2>&1`

```
ধাপ ০ (শুরু):     1 → টার্মিনাল     2 → টার্মিনাল
ধাপ ১ (> file):   1 → file          2 → টার্মিনাল
ধাপ ২ (2>&1):     1 → file          2 → file        ✅ দুটোই ফাইলে
```

ভুল (প্রত্যাশামতো কাজ করে না): `cmd 2>&1 > file`

```
ধাপ ০ (শুরু):     1 → টার্মিনাল     2 → টার্মিনাল
ধাপ ১ (2>&1):     1 → টার্মিনাল     2 → টার্মিনাল    (2 কপি করল 1-এর তখনকার গন্তব্য)
ধাপ ২ (> file):   1 → file          2 → টার্মিনাল   ❌ এরর এখনো স্ক্রিনে
```

`2>&1` কোনো স্থায়ী "বাঁধন" না — এটা ঠিক ওই মুহূর্তের একটা কপি (`dup2(1, 2)`)।

> 📝 মনে রাখো: "আগে ঠিকানা, তারপর জোড়া" — `> file 2>&1`।

`&>` ও `&>>` — bash-এর শর্টকাট

```
$ cmd &> all.txt       # = cmd > all.txt 2>&1
$ cmd &>> all.txt      # = cmd >> all.txt 2>&1   (bash 4+)
```

> ⚠️ সতর্কতা: `&>` হলো bash/zsh-এর সুবিধা, POSIX `sh`-এ নেই। Debian/Ubuntu-তে `/bin/sh` আসলে `dash` — সেখানে `cmd &> file` মানে হয় "`cmd` কে background-এ চালাও (`&`), আর `> file` দিয়ে একটা খালি ফাইল বানাও"! `#!/bin/sh` স্ক্রিপ্টে সবসময় `> file 2>&1` লেখো।

`1>&2` বা `>&2` — নিজের মেসেজ stderr-এ পাঠানো

স্ক্রিপ্টে এরর মেসেজ সঠিক চ্যানেলে দাও:

```
echo "ত্রুটি: কনফিগ ফাইল পাওয়া যায়নি" >&2
```

এতে কেউ তোমার স্ক্রিপ্টের আউটপুট ফাইলে রাখলেও এরর মেসেজ স্ক্রিনে দেখবে।

stdout ও stderr অদলবদল (swap)

একটা অতিরিক্ত FD 3 ধার করে:

```
$ cmd 3>&1 1>&2 2>&3 3>&-
```

ধাপে ধাপে: 3 = stdout-এর কপি; 1 = stderr-এর জায়গায়; 2 = 3 (আগের stdout); তারপর 3 বন্ধ (`3>&-`)। এখন শুধু এররগুলো পাইপে পাঠানো যায়:

```
$ find / -name '*.conf' 3>&1 1>&2 2>&3 3>&- | grep -c 'Permission denied'
```

সব রিডাইরেকশন অপারেটর একনজরে

অপারেটর	অর্থ	উদাহরণ
`> f` / `1> f`	stdout → f (ওভাররাইট)	`ls > list.txt`
`>> f`	stdout → f (append)	`date >> log.txt`
`< f`	stdin ← f	`wc -l < f`
`2> f`	stderr → f	`cmd 2> err.txt`
`2>> f`	stderr → f (append)	`cmd 2>> err.log`

অপারেটর	অর্থ	উদাহরণ
`2>&1`	stderr → stdout যেখানে আছে	`cmd > f 2>&1`
`>&2` / `1>&2`	stdout → stderr যেখানে আছে	`echo err >&2`
`&> f`	stdout+stderr → f (bash)	`cmd &> all.txt`
`&>> f`	stdout+stderr → f append (bash)	`cmd &>> all.log`
`>`	f`	noclobber উপেক্ষা করে ওভাররাইট
`<> f`	পড়া-লেখার জন্য খোলা	`exec 3<> f`
`n>&-`	FD n বন্ধ করো	`exec 3>&-`
`n>&m`	FD n = FD m-এর কপি	`3>&1`
`	&`	`2>&1

> 💡 টিপস: রিডাইরেকশন কমান্ড লাইনের যেকোনো জায়গায় লেখা যায় — `> out.txt echo hi` ও কাজ করে। তবে পড়ার সুবিধার জন্য শেষে রাখো।

১৯.৬ `/dev/null` ও তার ভাইবোন

`/dev/null` — লিনাক্সের ব্ল্যাক হোল

`/dev/null` একটা **character device** — এতে যা লেখো সব হারিয়ে যায় (kernel ফেলে দেয়), আর পড়তে গেলে সাথে সাথে EOF (ফাইলের শেষ) পাও।

```
$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Nov 12 09:00 /dev/null
```

প্রথম অক্ষর `c` = character device; `1, 3` হলো major/minor device নম্বর। সবাই পড়তে-লিখতে পারে (`rw-rw-rw-`)।

ব্যবহার:

```
$ find / -name passwd 2>/dev/null          # শুধু Permission denied এরর লুকাও
/etc/passwd
/usr/bin/passwd
...

$ cp bigfile /backup/ > /dev/null 2>&1      # কোনো আউটপুটই চাই না
$ cmd &> /dev/null                          # bash-এ একই

$ if grep -q root /etc/passwd; then echo আছে; fi     # -q থাকলে /dev/null লাগে না
আছে
```

```
$ cat /dev/null > app.log                    # ফাইল খালি করার আরেক উপায়
$ some_cmd < /dev/null                       # কোনো ইনপুট দিও না (প্রোগ্রাম অপেক্ষায় আটকাবে না)
```

> ⚠️ সতর্কতা: `2>/dev/null` দিয়ে সব এরর ঢেকে দেওয়া অভ্যাস করো না। সমস্যা হলে তুমি কিছুই জানবে না। শুধু যেগুলো জানা ও নিরীহ (যেমন `find` -এর Permission denied) সেগুলোই লুকাও।

অন্যান্য বিশেষ ফাইল

ফাইল	কাজ
`/dev/null`	লিখলে হারিয়ে যায়; পড়লে খালি (EOF)
`/dev/zero`	পড়লে অসীম শূন্য বাইট (`\0`) দেয়
`/dev/random` , `/dev/urandom`	এলোমেলো বাইট দেয়
`/dev/full`	লিখতে গেলে "No space left on device" এরর দেয় (টেস্টের জন্য)
`/dev/stdin` , `/dev/stdout` , `/dev/stderr`	বর্তমান প্রসেসের FD 0/1/2 (আসলে `/proc/self/fd/0..2` এর symlink)
`/dev/fd/N`	বর্তমান প্রসেসের FD N
`/dev/tty`	বর্তমান কন্ট্রোলিং টার্মিনাল (রিডাইরেকশন থাকলেও সরাসরি টার্মিনালে লেখে/পড়ে)

```
$ head -c 16 /dev/urandom | od -An -tx1
 3f a1 07 9c 5e 22 d0 18 b4 66 0e 7a c9 31 f2 4d
$ dd if=/dev/zero of=test.img bs=1M count=10
10+0 records in
10+0 records out
10485760 bytes (10 MB, 10 MiB) copied, 0.0123 s, 852 MB/s
$ echo "এটা সরাসরি টার্মিনালে" > /dev/tty
```

`/dev/stdin` দিয়ে এমন প্রোগ্রামকে stdin দেওয়া যায় যারা শুধু ফাইলনাম চায়:

```
$ echo "hello" | diff /dev/stdin <(echo "hello")
$
```

১৯.৭ Here Document (`<<`)

কী ও কেন

ধরো স্ক্রিপ্টের ভেতর থেকে একটা কমান্ডকে কয়েক লাইনের ইনপুট দিতে চাও — আলাদা ফাইল না বানিয়ে। এটাই **here document** (heredoc): "ইনপুট এখানেই, এই ডকুমেন্টে।"

Syntax:

```
command <<DELIMITER
লাইন ১
লাইন ২
DELIMITER
```

`DELIMITER` যেকোনো শব্দ হতে পারে (প্রথাগতভাবে `EOF`, `END`)। শেষের delimiter **একা একটা লাইনে, শুরুতে বা শেষে কোনো স্পেস ছাড়া** থাকতে হবে।

```
$ cat <<EOF
নাম: $USER
হোম: $HOME
আজ: $(date +%A)
EOF
নাম: rahim
হোম: /home/rahim
আজ: Thursday
```

দেখো, ভেতরের ভেরিয়েবল ও `$( )` expand হয়েছে।

Heredoc দিয়ে ফাইল তৈরি

```
$ cat > config.ini <<EOF
[server]
host = localhost
port = 8080
EOF
$ cat config.ini
[server]
host = localhost
port = 8080
```

`<<'EOF'` — expansion বন্ধ (আক্ষরিক লেখা)

delimiter-কে quote করলে (`'EOF'`, `"EOF"` বা `\EOF`) ভেতরে কিছুই expand হয় না। স্ক্রিপ্টের ভেতর থেকে আরেকটা স্ক্রিপ্ট লিখতে এটা অপরিহার্য:

```
$ cat > hello.sh <<'EOF'
#!/bin/bash
echo "আমার নাম $USER"
EOF
$ cat hello.sh
#!/bin/bash
echo "আমার নাম $USER"
```

quote না করলে `$USER` তখনই তোমার নাম দিয়ে বদলে যেত।

`<<-EOF` — শুরুর ট্যাব মুছে দাও

`<<-` ব্যবহার করলে প্রতিটি লাইনের (এবং শেষ delimiter-এর) **শুরুর TAB অক্ষর** মুছে যায় — ফলে স্ক্রিপ্টে সুন্দর ইনডেন্ট রাখা যায়:

```
if true; then
    cat <<-EOF
        এই লাইনের শুরুর ট্যাব মুছে যাবে
    EOF
fi
```

> ⚠️ সতর্কতা: `<<-` শুধু **TAB** মোছে, **space** নয়। অনেক এডিটর ট্যাবকে স্পেসে বদলে দেয় — তখন delimiter চিনতে না পেরে স্ক্রিপ্ট ভেঙে যায় ("here-document delimited by end-of-file" warning)।

Heredoc-এর বাস্তব ব্যবহার

```
# root হিসেবে ফাইল লেখা (sudo tee — ১৯.১০ দেখো)
$ sudo tee /etc/motd > /dev/null <<'EOF'
স্বাগতম! এই সার্ভারে সব কার্যকলাপ লগ হয়।
EOF

# রিমোট সার্ভারে একাধিক কমান্ড
$ ssh user@server <<'EOF'
uptime
df -h /
EOF

# ডেটাবেস কোয়েরি
$ mysql -u root -p mydb <<EOF
SELECT COUNT(*) FROM users;
EOF
```

Under the hood

bash heredoc-এর লেখাটা একটা অস্থায়ী জায়গায় রাখে এবং সেটার রিড-এন্ড কমান্ডের stdin (FD 0) হিসেবে দেয়। bash 5.1 থেকে লেখা ছোট হলে (pipe buffer-এর মধ্যে ধরে গেলে) pipe ব্যবহার করে, বড় হলে temp file — তোমার দিক থেকে কোনো পার্থক্য নেই, কমান্ড শুধু stdin থেকে পড়ে।

১৯.৮ Here String (`<<<`)

একটা মাত্র স্ট্রিং stdin হিসেবে দিতে চাইলে heredoc বাড়াবাড়ি। তখন **here string**:

Syntax: `command <<< "string"`

```
$ wc -w <<< "আমি লিনাক্স শিখছি"
3
$ tr 'a-z' 'A-Z' <<< "hello world"
HELLO WORLD
$ bc <<< "2^10"
1024
$ read first rest <<< "রহিম করিম যদু"
$ echo "$first"
রহিম
```

here string বনাম `echo ... |`:

```
$ echo "hello" | read x; echo "x=$x"
x=
$ read x <<< "hello"; echo "x=$x"
x=hello
```

পাইপে `read` একটা subshell-এ চলে (১৯.৯ দেখো), তাই ভেরিয়েবল হারিয়ে যায়। here string-এ `read` বর্তমান shell-এই চলে।

> 📝 মনে রাখো: here string শেষে একটা newline (`\n`) যোগ করে দেয়। `wc -c <<< "abc"` দেবে `4`, `3` নয়।

> ⚠️ সতর্কতা: `<<<` bash/zsh/ksh-এ আছে, POSIX `sh` / `dash`-এ নেই।

১৯.৯ পাইপ (`|`) — প্রোগ্রাম জোড়া লাগানোর আঠা

কী করে

Syntax: `cmd1 | cmd2 | cmd3 ...`

`cmd1`-এর **stdout** সরাসরি `cmd2`-এর **stdin**-এ চলে যায় — মাঝে কোনো ফাইল নেই।

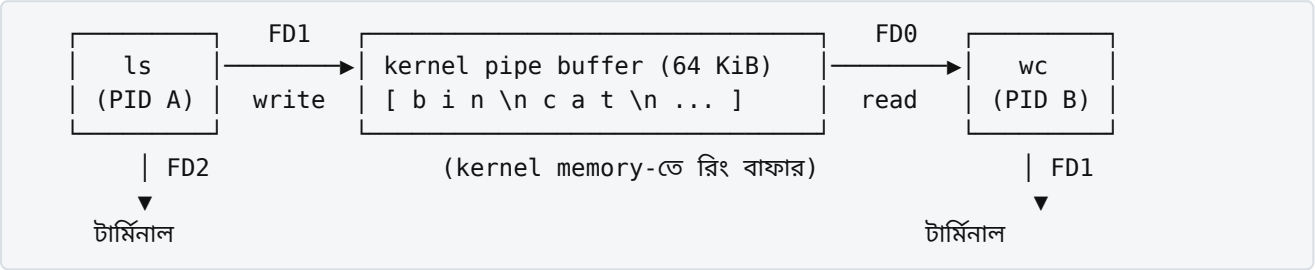
```
$ ls /usr/bin | wc -l
1873
$ cat /etc/passwd | cut -d: -f1 | sort | head -3
_apt
backup
bin
```

উপমা: কারখানার কনভেয়র বেল্ট। প্রথম শ্রমিক জিনিস কাটে, বেল্টে রাখে; দ্বিতীয়জন সাথে সাথে তুলে রং করে; তৃতীয়জন প্যাক করে। সবাই **একসাথে** কাজ করছে — কেউ পুরো ব্যাচ শেষ হওয়ার অপেক্ষায় বসে নেই।

Under the hood: kernel pipe buffer

যখন তুমি `ls | wc -l` লেখো, bash এই কাজগুলো করে:

```
1. pipe()   → kernel একটা pipe বানায়, দুটো FD দেয়: fd[0]=read end, fd[1]=write end
2. fork()   → child A (ls-এর জন্য):  dup2(fd[1], 1)  — stdout = pipe-এর write end
                                     বাকি pipe FD বন্ধ, তারপর exec("ls")
3. fork()   → child B (wc-এর জন্য):  dup2(fd[0], 0)  — stdin = pipe-এর read end
                                     বাকি pipe FD বন্ধ, তারপর exec("wc")
4. parent shell নিজের কপির দুই প্রান্তই বন্ধ করে, তারপর দুই child-এর জন্য অপেক্ষা (wait)
```



গুরুত্বপূর্ণ বিষয়গুলো:

1. **pipe buffer kernel memory-তে থাকে**, ডিস্কে না। Linux-এ ডিফল্ট সাইজ **65536 বাইট (64 KiB)**। (non-root একটা pipe-কে সর্বোচ্চ `/proc/sys/fs/pipe-max-size` পর্যন্ত বড় করতে পারে, সাধারণত 1 MiB।)

   ```
   $ cat /proc/sys/fs/pipe-max-size
   1048576
   ```

2. **Flow control (প্রবাহ নিয়ন্ত্রণ):**

   - buffer **ভর্তি** হলে লেখক (`ls`) `write()`-এ **block** হয়ে ঘুমায় — যতক্ষণ না পাঠক কিছু পড়ে জায়গা খালি করে।
   - buffer **খালি** হলে পাঠক (`wc`) `read()`-এ block হয়ে অপেক্ষা করে।
   - লেখক সব write end বন্ধ করলে (প্রসেস শেষ), পাঠক `read()`-এ **0 (EOF)** পায় — বুঝে যায় ডেটা শেষ।

   এভাবেই দ্রুত প্রোডিউসার ধীর কনজিউমারকে মেমরি দিয়ে ডুবিয়ে দিতে পারে না।

3. **সব কমান্ড একসাথে (concurrently) চলে।** `cmd1 | cmd2` মানে cmd1 শেষ হলে cmd2 চালু — এমন না। দুটো একই সময়ে আলাদা প্রসেস হিসেবে চলে। প্রমাণ:

   ```
   $ sleep 3 | sleep 3       # মোট ~3 সেকেন্ড লাগে, 6 না
   ```

4. **Atomic write:** `PIPE_BUF` (Linux-এ 4096 বাইট) বা তার কম আকারের একটা write কখনো অন্য লেখকের write-এর সাথে মিশে যায় না।

5. **SIGPIPE:** পাঠক আগে চলে গেলে (read end সব বন্ধ) লেখক পরের `write()`-এ **SIGPIPE** সিগন্যাল পায় এবং সাধারণত চুপচাপ মারা যায়। এজন্যই `yes | head -3` চিরকাল চলে না:

   ```
   $ yes | head -3
   y
   y
   y
   ```

   `head` তিন লাইন পড়ে বেরিয়ে যায় → `yes` লিখতে গিয়ে SIGPIPE পায় → শেষ। এজন্যই `cat huge.log | head` দ্রুত শেষ হয়, পুরো ফাইল পড়ে না।

শুধু stdout পাইপে যায়, stderr না

```
$ ls /etc/hostname /nai | wc -l
ls: cannot access '/nai': No such file or directory
1
```

এরর সরাসরি স্ক্রিনে এল, পাইপে গেল না। stderr-ও পাঠাতে চাইলে:

```
$ ls /etc/hostname /nai 2>&1 | wc -l
2
$ ls /etc/hostname /nai |& wc -l       # bash শর্টকাট
2
```

পাইপলাইনের exit status: `PIPESTATUS` ও `pipefail`

ডিফল্টভাবে একটা পাইপলাইনের exit status হলো **শেষ কমান্ডের** exit status।

```
$ false | true
$ echo $?
0
```

`false` ব্যর্থ হলো, কিন্তু আমরা জানতে পারলাম না! সমাধান দুটো:

```
$ false | true | false
$ echo "${PIPESTATUS[@]}"        # প্রতিটি কমান্ডের status (bash array)
1 0 1

$ set -o pipefail                 # কোনো একটা ব্যর্থ হলেই পুরো পাইপলাইন ব্যর্থ
$ false | true
$ echo $?
1
```

> ⚠️ সতর্কতা: `PIPESTATUS` পরের যেকোনো কমান্ড চালালেই (এমনকি `echo $?`) বদলে যায়। দরকার হলে সাথে সাথে কপি করো: `st=("${PIPESTATUS[@]}")`।

> 💡 টিপস: সিরিয়াস স্ক্রিপ্ট শুরু করো `set -euo pipefail` দিয়ে (অধ্যায় ৪৩)।

পাইপের প্রতিটি অংশ আলাদা প্রসেস (subshell সমস্যা)

bash-এ পাইপলাইনের প্রতিটি কমান্ড আলাদা subshell-এ চলে — শেষটাও। তাই এটা কাজ করে না:

```
$ count=0
$ cat names.txt | while read -r line; do count=$((count+1)); done
$ echo $count
0
```

`while` লুপটা subshell-এ চলেছে; তার `count` আলাদা কপি, শেষে হারিয়ে গেছে। সমাধান:

```
$ count=0
$ while read -r line; do count=$((count+1)); done < names.txt
$ echo $count
5
```

অথবা process substitution (১৯.১৪): `done < <(some_command)`। অথবা স্ক্রিপ্টে `shopt -s lastpipe` (শুধু যখন job control বন্ধ — যেমন non-interactive স্ক্রিপ্টে — তখন শেষ কমান্ড বর্তমান shell-এ চলে)।

UUOC — "Useless Use of Cat"

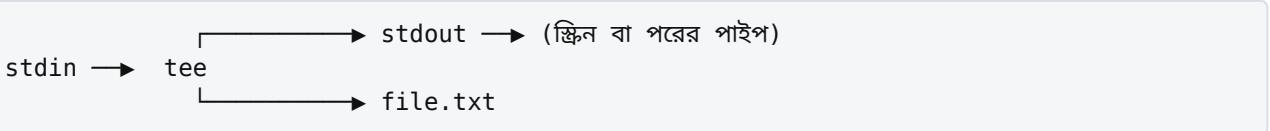
```
$ cat file | grep error     # কাজ করে, কিন্তু অপ্রয়োজনীয় একটা প্রসেস
$ grep error file           # ভালো
$ grep error < file         # এটাও ভালো
```

ছোট কাজে পার্থক্য নগণ্য, তবে অভ্যাসটা ভালো রাখা উচিত। পড়ার সুবিধার জন্য `cat` দিয়ে শুরু করলে সেটা অপরাধ না।

১৯.১০ `tee` — T-আকৃতির পাইপ

কী করে

প্লাম্বিংয়ের **T-জয়েন্ট** দেখেছ? একটা পাইপের পানি দুই দিকে ভাগ হয়। `tee` ঠিক তাই: stdin থেকে পড়ে, **একসাথে** stdout-এ ও এক বা একাধিক ফাইলে লেখে।



Syntax: `command | tee [OPTION]... [FILE]...`

অপশন	কাজ
`-a`, `--append`	ফাইল ওভাররাইট না করে শেষে যোগ করো
`-i`, `--ignore-interrupts`	Ctrl+C (SIGINT) উপেক্ষা করো

```
$ ls /etc | tee etc-list.txt | wc -l
etc-list.txt তৈরি হলো, আর স্ক্রিনে:
231
```

(এখানে `tee`-র stdout পাইপে গেছে, তাই স্ক্রিনে শুধু সংখ্যা। লিস্টটা ফাইলে আছে।)

```
$ ping -c 3 google.com | tee ping.log          # দেখোও, রাখোও
$ make 2>&1 | tee build.log                     # বিল্ডের সব আউটপুট দেখো ও লগ রাখো
$ echo "নতুন লাইন" | tee -a log1.txt log2.txt   # একসাথে দুই ফাইলে append
```

`sudo tee` — সবচেয়ে কাজের কৌশল

```
$ sudo echo "nameserver 1.1.1.1" > /etc/resolv.conf
bash: /etc/resolv.conf: Permission denied
```

কেন? `sudo` শুধু `echo`-কে root বানিয়েছে। কিন্তু `>` রিডাইরেকশন করছে **তোমার নিজের shell** — যে সাধারণ ইউজার! ফাইল খোলার কাজটা sudo-র বাইরে।

সমাধান: ফাইলে লেখার কাজটা root প্রসেসকে দাও:

```
$ echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf > /dev/null
$ echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf > /dev/null
```

`> /dev/null` দিলাম যাতে `tee` লেখাটা আবার স্ক্রিনে না দেখায়। বিকল্প: `sudo sh -c 'echo ... > /etc/file'`। (sudo বিস্তারিত: অধ্যায় ১৮।)

১৯.১১ কমান্ড চেইনিং: `;`, `&&`, `||`

Exit status আগে বুঝে নাও

প্রতিটি কমান্ড শেষে একটা সংখ্যা ফেরত দেয় — **exit status**। `0` মানে সফল, অন্য যেকোনো সংখ্যা (1–255) মানে কোনো না কোনো ব্যর্থতা। (বিস্তারিত ১৯.১৭-তে।)

`;` — শর্তহীন ক্রম

```
$ cd /tmp ; ls ; echo "শেষ"
```

আগেরটা সফল হোক বা না হোক, পরেরটা চলবে। নতুন লাইনে লেখা আর `;` দেওয়া একই।

`&&` — "সফল হলে তবেই"

```
$ mkdir project && cd project && git init
```

`mkdir` ব্যর্থ হলে `cd` চলবে না; `cd` ব্যর্থ হলে `git init` চলবে না। এটা নিরাপত্তার জন্য খুব গুরুত্বপূর্ণ:

```
$ cd /var/tmp/cache ; rm -rf *      # ❌ cd ব্যর্থ হলে বর্তমান ফোল্ডারেই সব মুছবে!
$ cd /var/tmp/cache && rm -rf *     # ✅ cd ব্যর্থ হলে rm চলবেই না
```

```
$ sudo apt update && sudo apt upgrade -y
```

`||` — "ব্যর্থ হলে তবেই"

```
$ ping -c1 -W2 8.8.8.8 > /dev/null || echo "ইন্টারনেট নেই!"
$ mkdir /data 2>/dev/null || echo "ফোল্ডার তৈরি করা যায়নি"
$ grep -q "^rahim:" /etc/passwd || sudo useradd rahim
```

`&& ... ||` — if-else-এর সংক্ষেপ (সাবধানে)

```
$ [ -f /etc/hostname ] && echo "আছে" || echo "নেই"
আছে
```

> ⚠️ সতর্কতা: `A && B || C` আসল if-else **না**। যদি `A` সফল হয় কিন্তু `B` ব্যর্থ হয়, তাহলে `C`-ও চলবে! যেমন `true && false || echo "এটাও চলল"`। জটিল ক্ষেত্রে আসল `if` ব্যবহার করো (অধ্যায় ৪১)।

অগ্রাধিকার (precedence)

`&&` ও `||` সমান অগ্রাধিকার, বাম থেকে ডানে মূল্যায়ন হয়। `;` ও `&` এদের চেয়ে দুর্বল, আর `|` সবচেয়ে শক্তিশালী।

```
$ false && echo A ; echo B
B
```

`echo B` সবসময় চলে কারণ `;` সবকিছুকে আলাদা করে।

অপারেটর	কখন পরের কমান্ড চলে	উদাহরণ
`;`	সবসময়	`date ; uptime`
`&&`	আগেরটা সফল (status 0) হলে	`make && make install`
`` ` ``		`` ` ``
`&`	আগেরটা background-এ পাঠিয়ে সাথে সাথে	`sleep 60 & echo চলছে`
`` ` ``	`` ` ``	একসাথে, stdout→stdin জোড়া

(`&` ও job control: অধ্যায় ২৫।)

১৯.১২ Subshell `( )` বনাম Group `{ }`

দুটোই কয়েকটা কমান্ডকে এক ইউনিট বানায় — যাতে একসাথে রিডাইরেক্ট বা চেইন করা যায়। পার্থক্য হলো **কোথায় চলে।**

`( )` — subshell: আলাদা প্রসেসে

`( )`-এর ভেতরের কমান্ডগুলো একটা **child shell** (fork করা কপি)-এ চলে। ভেতরের পরিবর্তন (`cd`, ভেরিয়েবল, `export`, `umask`) বাইরে আসে না।

```
$ pwd
/home/rahim
$ (cd /tmp && ls | head -2)
systemd-private-abc
tmp.XyZ
$ pwd
/home/rahim           # বাইরে কিছু বদলায়নি
```

```
$ x=1; (x=99; echo "ভেতরে: $x"); echo "বাইরে: $x"
ভেতরে: 99
বাইরে: 1
```

উপমা: subshell হলো ফটোকপি করা খাতায় লেখা — আসল খাতা অক্ষত থাকে।

ব্যবহার:

```
$ (cd /var/www && tar czf - html) > /backup/site.tar.gz
$ (date; uptime; df -h /) > report.txt       # তিনটির আউটপুট এক ফাইলে
```

`{ }` — group: বর্তমান shell-এই

```
$ { date; uptime; } > report.txt
```

এখানে নতুন প্রসেস তৈরি হয় না (দ্রুততর), আর ভেতরের পরিবর্তন বাইরে থেকে যায়:

```
$ x=1; { x=99; }; echo $x
99
$ { cd /tmp; }; pwd
/tmp
```

> ⚠️ সতর্কতা — syntax খুব কড়া:
>
> - `{` এর পরে **একটা স্পেস** লাগবেই (`{` হলো একটা reserved word, অপারেটর না)।
> - `}` এর আগে `;` **বা newline** লাগবেই।
> - `{date; uptime}` ❌ ভুল; `{ date; uptime; }` ✅ সঠিক।
> - `( )` এ এসব লাগে না: `(date;uptime)` ঠিক আছে।

একটা কাজের প্যাটার্ন: ব্যর্থ হলে মেসেজ দিয়ে বেরিয়ে যাও

```
cd /important/dir || { echo "ফোল্ডারে যাওয়া যায়নি" >&2; exit 1; }
```

এখানে `( )` দিলে `exit 1` শুধু subshell থেকে বের হতো, স্ক্রিপ্ট চলতেই থাকত! তাই `{ }` দরকার।

বিষয়	`( ... )` subshell	`{ ...; }` group
নতুন প্রসেস	হ্যাঁ (fork)	না
`cd` /ভেরিয়েবল বাইরে টেকে	না	হ্যাঁ
`exit`	শুধু subshell শেষ	পুরো shell/স্ক্রিপ্ট শেষ
গতি	একটু ধীর	দ্রুত
syntax	স্পেস/`;` ঐচ্ছিক	`{` স্পেস ও শেষে `;` বাধ্যতামূলক

> 📝 মনে রাখো: `$BASHPID` দিয়ে দেখো কোন প্রসেসে আছ — `echo $BASHPID; (echo $BASHPID)` দুটো আলাদা সংখ্যা দেবে। (`$$` কিন্তু subshell-এও মূল shell-এর PID দেখায়।)

১৯.১৩ Command Substitution: `$( )` ও backtick

কী করে

একটা কমান্ডের **আউটপুট** নিয়ে আরেকটা কমান্ডের **আর্গুমেন্ট** বা ভেরিয়েবলের মান হিসেবে বসিয়ে দেয়।

```
$ echo "আজ $(date +%d-%m-%Y)"
আজ 12-11-2026
```

```
$ today=$(date +%F)
$ echo $today
2026-11-12
$ mkdir backup-$(date +%F)
$ ls
backup-2026-11-12
$ echo "এই ফোল্ডারে $(ls | wc -l) টি ফাইল আছে"
এই ফোল্ডারে 7 টি ফাইল আছে
$ kill $(pgrep -f myscript.sh)
```

Under the hood

shell একটা subshell fork করে, তার stdout একটা pipe-এ জুড়ে দেয়, পুরো আউটপুট পড়ে নেয়, **শেষের সব newline মুছে ফেলে**, তারপর সেই লেখা কমান্ড লাইনে বসায়।

```
$ x=$(printf 'hello\n\n\n'); echo "[$x]"
[hello]
```

Backtick `` `...` `` — পুরনো রূপ

```
$ echo "আজ `date +%A`"
আজ Thursday
```

কাজ একই, কিন্তু `$( )` ভালো কারণ:

1. **Nesting সহজ:** `$(dirname $(which python3))` বনাম `` `dirname \`which python3\` `` (ব্যাকস্ল্যাশ লাগে)
2. **পড়তে সহজ:** backtick ও single quote (`'`) গুলিয়ে যায়
3. ভেতরে ব্যাকস্ল্যাশের নিয়ম সরল

```
$ echo $(basename $(dirname /usr/local/bin/tool))
local
```

Quote করার গুরুত্ব

```
$ files=$(ls)
$ echo $files          # quote ছাড়া: newline হারিয়ে সব এক লাইনে
a.txt b.txt c.txt
$ echo "$files"        # quote সহ: আসল রূপ রক্ষা পায়
a.txt
b.txt
c.txt
```

> ⚠️ সতর্কতা: quote ছাড়া `$( )` এর ফলাফল স্পেস দিয়ে ভেঙে যায় (word splitting) এবং `*` এর মতো wildcard expand হয় (globbing)। প্রায় সবসময় `"$(...)"` লেখো।

> 💡 টিপস: `$(< file)` হলো `$(cat file)`-এর দ্রুত bash রূপ — নতুন `cat` প্রসেস চালায় না: `content=$(< /etc/hostname)`।

> 📝 মনে রাখো: `$( )` (কমান্ড চালাও) আর `$(( ))` (গাণিতিক হিসাব) আলাদা: `echo $((5 + 3))` → `8`।

১৯.১৪ Process Substitution: `<( )` ও `>( )`

সমস্যা

কিছু কমান্ড শুধু **ফাইলনাম** চায়, stdin না — যেমন `diff`, `comm`, `paste` দুটো ফাইল তুলনা করে। দুটো কমান্ডের আউটপুট তুলনা করতে চাইলে? temp file বানানো ঝামেলা।

```
$ ls dir1 > /tmp/a; ls dir2 > /tmp/b; diff /tmp/a /tmp/b; rm /tmp/a /tmp/b   # ঝামেলা
```

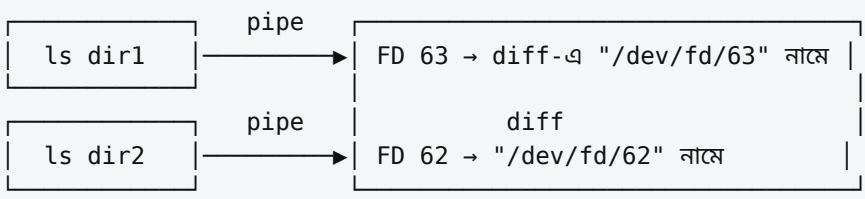
সমাধান: `<(command)`

```
$ diff <(ls dir1) <(ls dir2)
3c3
< report.pdf
---
> report-final.pdf
```

`<(cmd)` জায়গায় shell একটা **ফাইলনাম** বসিয়ে দেয়, যেটা পড়লে `cmd`-এর আউটপুট পাওয়া যায়:

```
$ echo <(true)
/dev/fd/63
$ ls -l <(true)
lr-x------ 1 rahim rahim 64 Nov 12 10:30 /dev/fd/63 -> pipe:[123456]
```

Under the hood



shell একটা pipe বানায়, `cmd`-কে background-এ চালিয়ে তার stdout pipe-এ জোড়ে, আর pipe-এর read end-কে child-এ একটা FD (যেমন 63) হিসেবে খোলা রাখে। `/dev/fd/63` পাথটা খুললে সেই pipe-ই পাওয়া যায়। `diff` ভাবে সে সাধারণ ফাইল খুলছে।

> 📝 মনে রাখো: এটা pipe, আসল ফাইল না — তাই শুধু **একবার শুরু থেকে শেষ পর্যন্ত** পড়া যায়। যে প্রোগ্রাম ফাইলে আগে-পিছে যায় (seek করে), সে ঠিকমতো কাজ নাও করতে পারে।

আরও উদাহরণ

```
# দুটো সার্ভারের প্যাকেজ তালিকা তুলনা
$ diff <(ssh web1 'dpkg -l | awk "{print \$2}"') <(ssh web2 'dpkg -l | awk "{print
\$2}"')

# শুধু প্রথম ফাইলে আছে এমন লাইন (comm-এর জন্য sorted ইনপুট লাগে)
$ comm -23 <(sort list1.txt) <(sort list2.txt)

# subshell সমস্যা ছাড়া while loop (১৯.৯ মনে করো)
$ count=0
$ while read -r line; do count=$((count+1)); done < <(ls /etc)
$ echo $count
231
```

শেষেরটায় খেয়াল করো: `< <(...)` — প্রথম `<` হলো ইনপুট রিডাইরেকশন, তারপর একটা **স্পেস**, তারপর `<(...)`। স্পেস ছাড়া `<<(` লিখলে heredoc ভেবে ভুল হবে।

`>(command)` — আউটপুট দিকে

```
$ ls -l | tee >(grep '^d' > dirs.txt) >(grep '^-' > files.txt) > /dev/null
```

একটা আউটপুট একসাথে দুটো ফিল্টারে পাঠালাম — একটা ডিরেক্টরি আলাদা করছে, আরেকটা সাধারণ ফাইল।

```
$ tar czf >(ssh backup-server 'cat > site.tar.gz') /var/www
```

> ⚠️ সতর্কতা: process substitution bash/zsh/ksh-এর বৈশিষ্ট্য, POSIX `sh` / `dash`-এ নেই। `#!/bin/sh` স্ক্রিপ্টে ব্যবহার করলে `Syntax error: "(" unexpected` দেখবে।

১৯.১৫ Named Pipe (FIFO) — `mkfifo`

সাধারণ pipe বনাম named pipe

`|` দিয়ে তৈরি pipe-এর কোনো নাম নেই (**anonymous pipe**) — শুধু একই shell থেকে শুরু হওয়া সম্পর্কিত প্রসেসগুলো ব্যবহার করতে পারে। **Named pipe** বা **FIFO** (First In, First Out) হলো ফাইলসিস্টেমে একটা **নাম** সহ pipe — যেকোনো দুটো অসম্পর্কিত প্রসেস, এমনকি আলাদা টার্মিনাল থেকে, এটা দিয়ে কথা বলতে পারে।

উপমা: anonymous pipe হলো দুই ভাইয়ের মধ্যে হাতে হাতে চিঠি দেওয়া; named pipe হলো বাড়ির সামনে একটা ডাকবাক্স — যে-কেউ চিঠি রাখতে পারে, যে-কেউ নিতে পারে।

Syntax: `mkfifo [-m MODE] NAME...`

অপশন	কাজ
`-m MODE`, `--mode=MODE`	পারমিশন সেট করো (যেমন `-m 600`)

```
$ mkfifo /tmp/mypipe
$ ls -l /tmp/mypipe
prw-r--r-- 1 rahim rahim 0 Nov 12 10:40 /tmp/mypipe
```

প্রথম অক্ষর `p` = pipe। সাইজ সবসময় 0 — কারণ ডেটা ডিস্কে থাকে না, kernel-এর pipe buffer-এ থাকে। ফাইলসিস্টেমে শুধু "নাম" আর inode আছে।

হাতে-কলমে: দুই টার্মিনালে

টার্মিনাল ১:

```
$ cat /tmp/mypipe
```

(এখানে আটকে থাকবে — লেখক আসার অপেক্ষায়)

টার্মিনাল ২:

```
$ echo "হ্যালো অন্য টার্মিনাল!" > /tmp/mypipe
```

টার্মিনাল ১-এ এখন দেখাবে:

```
হ্যালো অন্য টার্মিনাল!
$
```

Blocking আচরণ

FIFO খোলার সময়:

- **পড়ার জন্য** খুললে — কেউ লেখার জন্য না খোলা পর্যন্ত `open()` block হয়।
- **লেখার জন্য** খুললে — কেউ পড়ার জন্য না খোলা পর্যন্ত block হয়।

দুই পক্ষ "দেখা না হওয়া পর্যন্ত" কেউ এগোয় না — একে বলে rendezvous।

বাস্তব ব্যবহার

```
# একটা প্রসেসের লগ লাইভ আরেকটা প্রসেসে প্রসেস করা
$ mkfifo /tmp/logpipe
$ grep --line-buffered ERROR < /tmp/logpipe > errors.txt &
$ ./my_app > /tmp/logpipe

# একটা কমান্ডের আউটপুট একসাথে কম্প্রেস ও চেকসাম
$ mkfifo /tmp/p
$ sha256sum < /tmp/p > checksum.txt &
$ tar cf - mydir | tee /tmp/p | gzip > mydir.tar.gz
$ wait
$ rm /tmp/p
```

FIFO সাধারণ ফাইলের মতোই `rm` দিয়ে মোছা যায়। (`mknod name p` ও একই কাজ করে, তবে `mkfifo` পছন্দনীয়।)

| | Anonymous pipe (`|`) | Named pipe (FIFO) | |--|----------------------|-------------------| | তৈরি | shell স্বয়ংক্রিয়ভাবে | `mkfifo` | | ফাইলসিস্টেমে নাম | নেই | আছে (`p` টাইপ) | | কারা ব্যবহার করে | সম্পর্কিত প্রসেস (একই পাইপলাইন) | যেকোনো প্রসেস, পারমিশন থাকলে | | আয়ু | প্রসেস শেষ হলে শেষ | `rm` না করা পর্যন্ত নাম থাকে | | ডেটা কোথায় | kernel buffer | kernel buffer (ডিস্কে না) |

১৯.১৬ `exec` দিয়ে স্থায়ী রিডাইরেকশন ও নিজস্ব FD

কোনো কমান্ড ছাড়া `exec` এর সাথে রিডাইরেকশন দিলে সেটা **বর্তমান shell-এর** জন্য স্থায়ীভাবে প্রযোজ্য হয়।

```
#!/bin/bash
# স্ক্রিপ্টের বাকি সব আউটপুট ও এরর লগ ফাইলে
exec > /var/log/myscript.log 2>&1
echo "এটা লগ ফাইলে যাবে"
ls /nai          # এররও লগে যাবে
```

স্ক্রিনেও দেখতে চাইলে process substitution আর tee মিলিয়ে:

```
exec > >(tee -a /var/log/myscript.log) 2>&1
```

নিজের FD খোলা ও বন্ধ করা

FD 3–9 তুমি নিজের কাজে ব্যবহার করতে পারো (bash 10 ও তার বেশি নম্বরও সমর্থন করে, তবে shell নিজেও সেগুলো ব্যবহার করে, তাই সাবধানে)।

```
$ exec 3> /tmp/out.txt        # FD 3 লেখার জন্য খোলা
$ echo "প্রথম" >&3
$ echo "দ্বিতীয়" >&3
$ exec 3>&-                   # FD 3 বন্ধ
$ cat /tmp/out.txt
প্রথম
দ্বিতীয়

$ exec 4< /etc/hostname       # FD 4 পড়ার জন্য খোলা
$ read -r line <&4
$ echo "$line"
myserver
$ exec 4<&-
```

bash 4.1+ তে স্বয়ংক্রিয় FD নম্বর নেওয়া যায়:

```
$ exec {myfd}> /tmp/auto.txt
$ echo $myfd
10
$ echo "hi" >&$myfd
$ exec {myfd}>&-
```

মজার কৌশল: bash-এর `/dev/tcp`

bash নিজে (kernel না) `/dev/tcp/HOST/PORT` পাথকে বিশেষভাবে চেনে এবং একটা TCP সংযোগ খোলে:

```
$ exec 3<> /dev/tcp/example.com/80
$ printf 'HEAD / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' >&3
$ head -1 <&3
HTTP/1.1 200 OK
$ exec 3<&-
```

> 📝 মনে রাখো: `/dev/tcp` আসলে ডিস্কে নেই (`ls /dev/tcp` এরর দেবে); এটা শুধু bash-এর রিডাইরেকশনের ভেতরে কাজ করে। পোর্ট খোলা কিনা দ্রুত দেখতে কাজের: `timeout 2 bash -c '</dev/tcp/10.0.0.5/22' && echo open`।

১৯.১৭ Exit Status `$?`

কী

প্রতিটি কমান্ড শেষ হওয়ার সময় kernel-এর মাধ্যমে parent-কে একটা 0–255 সংখ্যা জানায়। bash সর্বশেষ কমান্ডের এই সংখ্যা `$?` ভেরিয়েবলে রাখে।

```
$ ls /etc > /dev/null
$ echo $?
```

```
0
$ ls /nai
ls: cannot access '/nai': No such file or directory
$ echo $?
2
```

> 📝 মনে রাখো: সাধারণ যুক্তির উল্টো — **0 = সফল (সত্য)**, অশূন্য = ব্যর্থ (মিথ্যা)। কারণ সফল হওয়ার একটাই উপায়, কিন্তু ব্যর্থ হওয়ার অনেক কারণ থাকতে পারে, আর অশূন্য সংখ্যাগুলো কারণ বলে দেয়।

প্রচলিত exit code

কোড	অর্থ
`0`	সফল
`1`	সাধারণ এরর (যেমন `grep` কিছু না পেলে)
`2`	ভুল ব্যবহার / গুরুতর সমস্যা (bash builtin-এর ভুল syntax; `ls`-এ গুরুতর সমস্যা; `grep`-এ এরর)
`126`	কমান্ড পাওয়া গেছে কিন্তু চালানো যায় না (execute পারমিশন নেই)
`127`	কমান্ড পাওয়া যায়নি (command not found)
`128+N`	সিগন্যাল N দিয়ে মারা গেছে
`130`	= 128+2, Ctrl+C (SIGINT)
`137`	= 128+9, SIGKILL (যেমন OOM killer বা `kill -9`)
`143`	= 128+15, SIGTERM

```
$ nosuchcommand
nosuchcommand: command not found
$ echo $?
127
$ sleep 100
^C
$ echo $?
130
```

> ⚠️ সতর্কতা: প্রতিটি প্রোগ্রাম নিজের অর্থ ঠিক করে। যেমন `grep`: `0` = মিলেছে, `1` = মেলেনি, `2` = এরর। `diff`: `0` = একই, `1` = পার্থক্য আছে, `2` = সমস্যা। সন্দেহ হলে `man` পেজের "EXIT STATUS" অংশ পড়ো।

`$?` একবারই বাঁচে

```
$ false
$ echo $?
1
```

```
$ echo $?
0          # আগের echo সফল হয়েছিল!
```

দরকার হলে সাথে সাথে সংরক্ষণ করো: `cmd; rc=$?`।

নিজের exit status দেওয়া

```
#!/bin/bash
if [ ! -f "$1" ]; then
    echo "ফাইল নেই: $1" >&2
    exit 1
fi
echo "ঠিক আছে"
exit 0
```

`exit` এ 255-এর বেশি দিলে 256 দিয়ে ভাগশেষ নেওয়া হয় (`exit 256` → `0`!)। ফাংশন থেকে `return N` একইভাবে কাজ করে (অধ্যায় ৪২)।

`$?` না দেখে সরাসরি কমান্ড পরীক্ষা

```
# কম ভালো
grep -q error app.log
if [ $? -eq 0 ]; then echo "এরর আছে"; fi

# ভালো
if grep -q error app.log; then echo "এরর আছে"; fi
```

`if` নিজেই কমান্ডের exit status দেখে।

`!` — status উল্টানো

```
$ ! grep -q rahim /etc/passwd && echo "ইউজার নেই"
```

১৯.১৮ সব একসাথে: বাস্তব পাইপলাইন

```
# সবচেয়ে বেশি মেমরি ব্যবহারকারী ৫টি প্রসেস
$ ps aux --sort=-%mem | head -6

# লগে কোন IP সবচেয়ে বেশি রিকোয়েস্ট পাঠিয়েছে
$ awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -5
   1523 203.0.113.7
    842 198.51.100.23
    ...

# কোন ফোল্ডার সবচেয়ে বেশি জায়গা নিচ্ছে
$ du -sh /var/* 2>/dev/null | sort -rh | head -5
```

```
# ব্যাকআপ: সফল হলে লগে লেখো, ব্যর্থ হলে এরর লগে
$ tar czf /backup/home-$(date +%F).tar.gz /home 2>/tmp/tar.err \
    && echo "$(date): ব্যাকআপ সফল" >> /var/log/backup.log \
    || { echo "$(date): ব্যাকআপ ব্যর্থ" >&2; cat /tmp/tar.err >&2; }
```

(`sort` , `uniq` , `awk` এর বিস্তারিত পরের অধ্যায়গুলোতে — অধ্যায় ২০, ২২, ২৩।)

> 💡 টিপস: লম্বা পাইপলাইন ধাপে ধাপে বানাও। প্রথম কমান্ড চালাও, আউটপুট দেখো; তারপর `↑` চেপে `| পরের_কমান্ড` যোগ করো। প্রতিটি ধাপে দেখো ডেটা কেমন দেখাচ্ছে।

> 💡 টিপস: লাইন শেষে `|` , `&&` বা `||` থাকলে bash নিজেই বোঝে কমান্ড শেষ হয়নি — `\` ছাড়াই পরের লাইনে লেখা যায়।

১৯.১৯ সম্পর্কিত কীবোর্ড শর্টকাট

শর্টকাট	কাজ	এই অধ্যায়ে প্রাসঙ্গিকতা
`Ctrl+D`	stdin-এ EOF পাঠাও (লাইনের শুরুতে)	`cat > file` বা `wc` চালিয়ে হাতে টাইপ শেষ করতে
`Ctrl+C`	foreground প্রসেস/পাইপলাইনে SIGINT	আটকে থাকা পাইপ বা FIFO রিডার থামাতে (exit 130)
`Ctrl+Z`	foreground পাইপলাইন সাময়িক বন্ধ (SIGTSTP)	পরে `bg` / `fg`
`Ctrl+\`	SIGQUIT পাঠাও	Ctrl+C কাজ না করলে
`Ctrl+S` / `Ctrl+Q`	টার্মিনাল আউটপুট থামাও / আবার চালাও	দ্রুত চলা আউটপুট থামিয়ে পড়তে (ভুলে Ctrl+S চাপলে টার্মিনাল "জমে" যায় — Ctrl+Q চাপো)
`Alt+.`	আগের কমান্ডের শেষ আর্গুমেন্ট বসাও	`ls > out.txt` এর পর `cat` + `Alt+.` → `out.txt`
`Ctrl+X` `Ctrl+E`	বর্তমান কমান্ড এডিটরে খোলো	লম্বা পাইপলাইন আরামে লিখতে
`↑` / `Ctrl+P`	আগের কমান্ড	পাইপলাইন ধাপে ধাপে বাড়াতে
`Ctrl+L`	স্ক্রিন পরিষ্কার	
`Shift+\`	`	` টাইপ (US কীবোর্ড)

পূর্ণ তালিকা: অধ্যায় ৬ ও পরিশিষ্ট ক।

```
$ cat > shopping.txt
চাল
```

```
ডাল
তেল
^D
$ cat shopping.txt
চাল
ডাল
তেল
```

(`^D` মানে Ctrl+D চেপেছ — স্ক্রিনে সাধারণত দেখা যায় না।)

হাতে-কলমে অনুশীলন

একটা অনুশীলন ফোল্ডার বানাও:

```
$ mkdir -p ~/lab19 && cd ~/lab19
```

ল্যাব ১: stdout ও stderr আলাদা করা

```
$ ls /etc/hostname /nai > out.txt 2> err.txt
$ cat out.txt
/etc/hostname
$ cat err.txt
ls: cannot access '/nai': No such file or directory
```

এখন ক্রমের পার্থক্য নিজে দেখো:

```
$ ls /etc/hostname /nai > both1.txt 2>&1
$ ls /etc/hostname /nai 2>&1 > both2.txt
ls: cannot access '/nai': No such file or directory
$ wc -l both1.txt both2.txt
 2 both1.txt
 1 both2.txt
 3 total
```

ল্যাব ২: ওভাররাইট বনাম append এবং noclobber

```
$ echo one > f.txt; echo two > f.txt; cat f.txt
two
$ echo three >> f.txt; cat f.txt
two
three
$ set -o noclobber
$ echo four > f.txt
bash: f.txt: cannot overwrite existing file
$ echo four >| f.txt; cat f.txt
four
$ set +o noclobber
```

ল্যাব ৩: FD টেবিল নিজের চোখে

```
$ ls -l /proc/$$/fd
$ exec 3> lab.txt
$ ls -l /proc/$$/fd | grep ' 3 '
l-wx------ 1 rahim rahim 64 Nov 12 11:00 3 -> /home/rahim/lab19/lab.txt
$ echo "FD 3 দিয়ে লেখা" >&3
$ exec 3>&-
$ cat lab.txt
FD 3 দিয়ে লেখা
```

ল্যাব ৪: pipe concurrency ও SIGPIPE

```
$ time (sleep 2 | sleep 2)
real    0m2.00s
...
$ yes "লিনাক্স" | head -n 3
লিনাক্স
লিনাক্স
লিনাক্স
$ yes | head -1; echo "${PIPESTATUS[@]}"
y
141 0
```

`141` = 128 + 13 (SIGPIPE) — `yes` SIGPIPE-এ মারা গেছে।

ল্যাব ৫: pipefail

```
$ ls /nai | wc -l; echo "status=$?"
ls: cannot access '/nai': No such file or directory
0
status=0
$ set -o pipefail
$ ls /nai | wc -l; echo "status=$?"
ls: cannot access '/nai': No such file or directory
0
status=2
$ set +o pipefail
```

ল্যাব ৬: heredoc দিয়ে স্ক্রিপ্ট তৈরি

```
$ cat > greet.sh <<'EOF'
#!/bin/bash
name=${1:-বন্ধু}
echo "স্বাগতম, $name!"
EOF
$ chmod +x greet.sh
$ ./greet.sh রহিম
স্বাগতম, রহিম!
```

এখন `'EOF'` এর বদলে `EOF` (quote ছাড়া) দিয়ে আবার বানাও এবং `cat greet.sh` করে পার্থক্য দেখো।

ল্যাব ৭: tee ও sudo tee

```
$ ls /etc | tee etc.txt | wc -l
$ head -3 etc.txt
$ echo "127.0.0.1 mylab.local" | sudo tee -a /etc/hosts > /dev/null
$ grep mylab /etc/hosts
127.0.0.1 mylab.local
```

(পরে মুছে দিতে: `sudo sed -i '/mylab.local/d' /etc/hosts` — sed: অধ্যায় ২১।)

ল্যাব ৮: subshell বনাম group

```
$ cd ~/lab19
$ (cd /tmp; pwd); pwd
/tmp
/home/rahim/lab19
$ { cd /tmp; pwd; }; pwd
/tmp
/tmp
$ cd ~/lab19
```

ল্যাব ৯: command ও process substitution

```
$ echo "এখানে $(ls | wc -l) টি আইটেম, আজ $(date +%A)"
$ mkdir a b; touch a/{1,2,3}.txt b/{2,3,4}.txt
$ diff <(ls a) <(ls b)
1d0
< 1.txt
3a3
> 4.txt
$ comm -12 <(ls a) <(ls b)
2.txt
3.txt
```

ল্যাব ১০: named pipe দুই টার্মিনালে

টার্মিনাল ১:

```
$ mkfifo ~/lab19/chat
$ while true; do cat ~/lab19/chat; done
```

টার্মিনাল ২:

```
$ echo "কেমন আছ?" > ~/lab19/chat
$ date > ~/lab19/chat
```

টার্মিনাল ১-এ মেসেজ আসবে। শেষে টার্মিনাল ১-এ `Ctrl+C`, তারপর `rm ~/lab19/chat`।

ল্যাব ১১: exit status শিকার

```
$ true; echo $?
0
$ false; echo $?
1
$ grep -q root /etc/passwd; echo $?
0
$ grep -q nobodyxyz /etc/passwd; echo $?
1
$ /etc/passwd; echo $?
bash: /etc/passwd: Permission denied
126
$ nocmd; echo $?
nocmd: command not found
127
```

চ্যালেঞ্জ

একটা এক-লাইনের কমান্ড লেখো যা: `/etc` এর নিচে `.conf` ফাইল খুঁজবে, Permission denied এরর লুকাবে, তালিকাটা `conf-list.txt`-এ রাখবে **এবং** স্ক্রিনে মোট সংখ্যা দেখাবে, আর `find` ব্যর্থ হলেও তুমি তা জানতে পারবে।

▼ একটা সম্ভাব্য সমাধান

```
$ set -o pipefail
$ find /etc -name '*.conf' 2>/dev/null | tee conf-list.txt | wc -l || echo "কিছু সমস্যা হয়েছে" >&2
```

লক্ষ করো: Permission denied থাকলে `find` নিজেই অশূন্য status দেয়, তাই `pipefail` চালু থাকলে এখানে "কিছু সমস্যা হয়েছে" দেখা যেতে পারে — এটাই প্রমাণ যে `2>/dev/null` এরর লুকালেও exit status লুকায় না।

সারসংক্ষেপ

বিষয়	Syntax	এক কথায়
stdout ফাইলে	`cmd > f`	ওভাররাইট
stdout append	`cmd >> f`	শেষে যোগ
stdin ফাইল থেকে	`cmd < f`	ইনপুট বদল
stderr ফাইলে	`cmd 2> f`	শুধু এরর

বিষয়	Syntax	এক কথায়
দুটোই এক ফাইলে	`cmd > f 2>&1` / `cmd &> f`	ক্রম জরুরি
দুটোই append	`cmd >> f 2>&1` / `cmd &>> f`	
নিঃশব্দ	`cmd > /dev/null 2>&1`	সব ফেলে দাও
মেসেজ stderr-এ	`echo msg >&2`	স্ক্রিপ্টে এরর
noclobber	`set -o noclobber`, `>`	`
here document	`cmd <<EOF ... EOF`	বহু-লাইন ইনপুট
আক্ষরিক heredoc	`cmd <<'EOF'`	expansion বন্ধ
ট্যাব-মোছা heredoc	`cmd <<-EOF`	শুধু TAB
here string	`cmd <<< "text"`	এক-লাইন ইনপুট
পাইপ	`a	b`	
stderr সহ পাইপ	`a 2>&1	b` / a
পাইপ status	`${PIPESTATUS[@]}`, `set -o pipefail`	
T-জয়েন্ট	`cmd	tee [-a] f`	
root ফাইলে লেখা	`echo x	sudo tee f > /dev/null`	
ক্রম	`a ; b`	সবসময়
সফল হলে	`a && b`	
ব্যর্থ হলে	`a || b`	
subshell	`( a; b )`	আলাদা প্রসেস
group	`{ a; b; }`	একই shell
command substitution	`"$(cmd)"`, `` `cmd` ``	আউটপুট বসাও
process substitution	`<(cmd)`, `>(cmd)`	আউটপুটকে ফাইলনাম বানাও
named pipe	`mkfifo name`	নামওয়ালা pipe
স্থায়ী রিডাইরেকশন	`exec > log 2>&1`	পুরো স্ক্রিপ্টে
নিজের FD	`exec 3> f`, `>&3`, `exec 3>&-`	
exit status	`$?`	0 = সফল
EOF পাঠানো	`Ctrl+D`	

নিজেকে যাচাই করো

1. stdin, stdout ও stderr এর FD নম্বর কত, এবং stdout ও stderr আলাদা রাখার যুক্তি কী?
2. `cmd > file 2>&1` আর `cmd 2>&1 > file` এর মধ্যে পার্থক্য কী? কেন?

3. `sort data.txt > data.txt` চালানোর পর `data.txt` খালি কেন হয়?
4. `sudo echo "x" > /etc/somefile` কেন "Permission denied" দেয়, আর সমাধান কী?
5. `cmd1 | cmd2` তে buffer ভর্তি হলে কী ঘটে? আর `cmd2` আগে বেরিয়ে গেলে `cmd1`-এর কী হয়?
6. `false | true; echo $?` কী দেখাবে? কীভাবে `false`-এর ব্যর্থতা ধরা যায়?
7. `cat <<EOF` আর `cat <<'EOF'` এর পার্থক্য কী?
8. `( cd /tmp )` আর `{ cd /tmp; }` চালানোর পর তুমি কোন ফোল্ডারে থাকবে?
9. `diff <(ls a) <(ls b)` তে `diff` আসলে কী ফাইলনাম পায়?
10. exit status `127`, `126` ও `130` এর মানে কী?

▼ **উত্তর দেখো**

1. stdin = 0, stdout = 1, stderr = 2। আলাদা রাখলে ডেটা (stdout) ফাইলে বা পাইপে পাঠানোর সময় এরর মেসেজ তাতে মিশে যায় না, আর এরর আলাদাভাবে দেখা/লগ করা যায়।
2. প্রথমটায় আগে stdout ফাইলে যায়, তারপর stderr-কে stdout-এর (এখন ফাইলের) কপি বানানো হয় — দুটোই ফাইলে। দ্বিতীয়টায় stderr আগে stdout-এর তখনকার গন্তব্য (টার্মিনাল) কপি করে, পরে শুধু stdout ফাইলে যায় — এরর টার্মিনালেই থাকে। কারণ রিডাইরেকশন বাম থেকে ডানে প্রয়োগ হয় এবং `2>&1` একটা মুহূর্তের কপি (`dup2`)।
3. shell `sort` চালু করার আগেই `>` দিয়ে `data.txt` খোলে ও truncate করে; তাই `sort` খালি ফাইল পড়ে। সমাধান: `sort -o data.txt data.txt` বা অন্য ফাইলে লিখে `mv`।
4. `sudo` শুধু `echo`-কে root হিসেবে চালায়, কিন্তু `>` রিডাইরেকশনটা করে সাধারণ ইউজারের shell। সমাধান: `echo "x" | sudo tee /etc/somefile > /dev/null` (বা `-a` দিয়ে append), অথবা `sudo sh -c 'echo x > /etc/somefile'`।
5. buffer (Linux-এ ডিফল্ট 64 KiB) ভর্তি হলে `cmd1` `write()`-এ block হয়ে অপেক্ষা করে যতক্ষণ না `cmd2` পড়ে জায়গা খালি করে। `cmd2` বেরিয়ে গেলে (read end বন্ধ) `cmd1` পরের write-এ SIGPIPE পায় এবং সাধারণত শেষ হয়ে যায় (status 141)।
6. `0` দেখাবে, কারণ পাইপলাইনের status হলো শেষ কমান্ডের। ধরতে: `echo "${PIPESTATUS[@]}"` (দেখাবে `1 0`) অথবা আগে `set -o pipefail`।
7. quote ছাড়া `EOF`-এ ভেতরের `$VAR`, `$(cmd)`, `` `cmd` `` ইত্যাদি expand হয়; `'EOF'` (quoted) হলে সব আক্ষরিকভাবে থাকে।
8. `( cd /tmp )` এর পর আগের ফোল্ডারেই থাকবে (subshell-এর পরিবর্তন বাইরে আসে না); `{ cd /tmp; }` এর পর `/tmp`-এ থাকবে (একই shell)।
9. `/dev/fd/63` ও `/dev/fd/62` জাতীয় পাথ — যা আসলে pipe-এর read end; পড়লে `ls a` ও `ls b` এর আউটপুট পাওয়া যায়।
10. `127` = কমান্ড পাওয়া যায়নি; `126` = পাওয়া গেছে কিন্তু চালানো যায় না (execute পারমিশন নেই বা ডিরেক্টরি); `130` = 128+2, অর্থাৎ SIGINT (Ctrl+C) দিয়ে থামানো হয়েছে।

অধ্যায় ২০: grep ও রেগুলার এক্সপ্রেশন

> "লাখ লাখ লাইনের লগ ফাইল থেকে একটা লাইন খুঁজে বের করতে হবে? মানুষের লাগবে কয়েক ঘণ্টা, `grep`-এর লাগবে এক সেকেন্ডেরও কম।"

এই অধ্যায়ে যা শিখবে

- `grep` কী, নামটা কোথা থেকে এল, আর ভেতরে এটা কীভাবে কাজ করে
- `grep`-এর সব দরকারি অপশন: `-i -v -n -c -l -L -r -R -w -x -o -A -B -C -E -F -P --color --include --exclude` এবং আরও কিছু
- রেগুলার এক্সপ্রেশন (regex) কী, কেন দরকার, আর তার সব মেটাক্যারেক্টার: `. * + ? ^ $ [] [^] {n,m} () | \b \d`
- BRE, ERE আর PCRE — তিন ধরনের regex-এর পার্থক্য
- POSIX character class (`[[:digit:]]`, `[[:alpha:]]` ইত্যাদি)
- greedy matching কী, আর কীভাবে তার ফাঁদ এড়াবে
- `egrep` ও `fgrep` কেন এখন "পুরোনো" (obsolescent)
- আধুনিক, অতি-দ্রুত বিকল্প **ripgrep (`rg`)**
- বাস্তব কাজ: লগ থেকে IP ঠিকানা বের করা, ফাইল থেকে email খোঁজা, কোডবেসে খোঁজা, স্ক্রিপ্টে `grep` ব্যবহার

২০.১ grep কী, আর কেন এটা এত গুরুত্বপূর্ণ

`grep` হলো লিনাক্সের **টেক্সট খোঁজার যন্ত্র**। তুমি একটা প্যাটার্ন (pattern — খোঁজার নকশা) দাও, আর `grep` এক বা একাধিক ফাইলের (অথবা পাইপ দিয়ে আসা ইনপুটের) প্রতিটা লাইন পড়ে দেখে — যে লাইনে প্যাটার্নটা মেলে, শুধু সেই লাইনগুলো স্ক্রিনে ছাপে।

উপমা: ধরো তোমার সামনে একটা মোটা টেলিফোন ডিরেক্টরি। তোমাকে বের করতে হবে কোন কোন লোকের নামে "রহমান" আছে। তুমি প্রতিটা পাতার প্রতিটা লাইনে আঙুল রেখে দেখবে "রহমান" আছে কি না, থাকলে সেই লাইনে হাইলাইটার দিয়ে দাগ দেবে। `grep` ঠিক এই কাজটাই করে — কিন্তু সেকেন্ডে কয়েকশো মেগাবাইট গতিতে।

নামটা কোথা থেকে এল?

১৯৭৩ সালে Ken Thompson ইউনিক্সের জন্য `grep` লেখেন। সেই সময়ের লাইন-এডিটর `ed`-এ একটা কমান্ড ছিল:

```
g/re/p
```

মানে **g**lobal — পুরো ফাইলে, **re** — একটা regular expression খোঁজো, **p**rint — মেলা লাইন ছাপো। এই কমান্ডটাকেই আলাদা প্রোগ্রাম বানানো হলো, নাম হলো `grep`। আজও প্রোগ্রামাররা "grep করা" কথাটা ক্রিয়াপদ হিসেবে ব্যবহার করে — যেমন "লগটা একটু grep করে দেখো তো।"

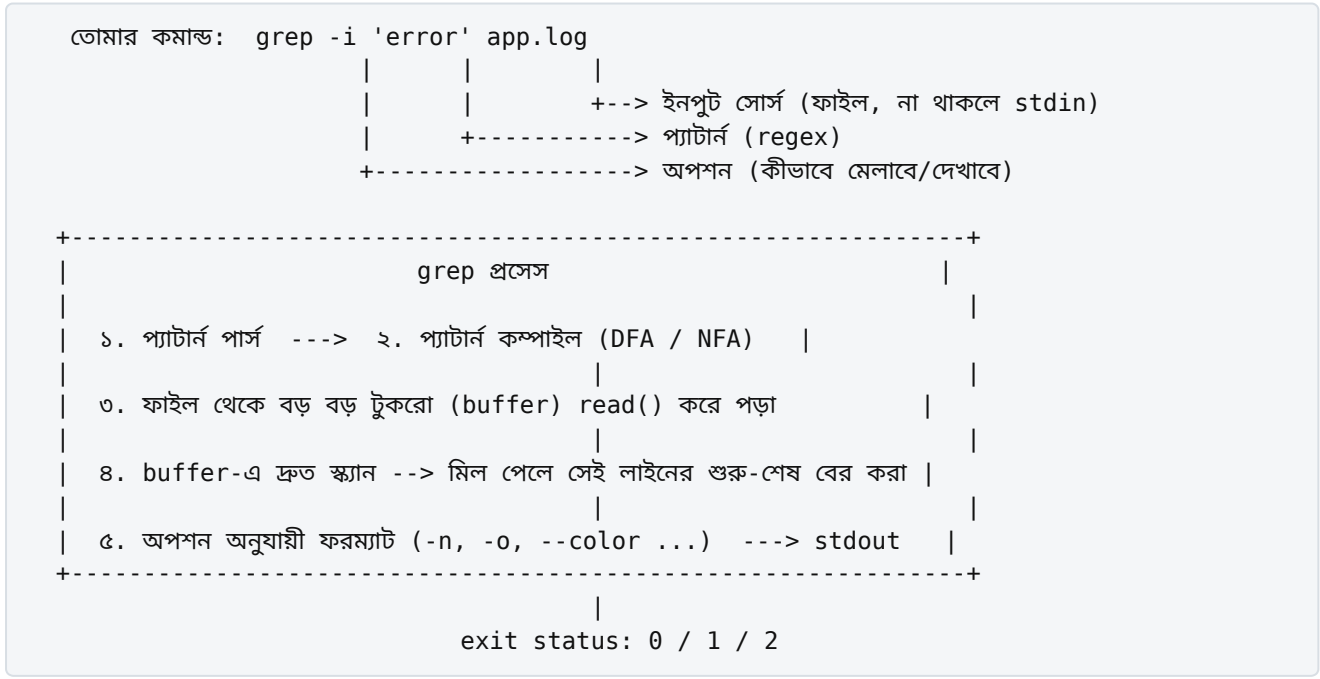
কোথায় কোথায় লাগে?

কাজ	উদাহরণ
লগ ফাইলে error খোঁজা	`grep -i error /var/log/syslog`
কনফিগ ফাইলে একটা সেটিং খোঁজা	`grep Port /etc/ssh/sshd_config`
কোডবেসে কোনো ফাংশন কোথায় ব্যবহার হয়েছে	`grep -rn "login(" src/`
অন্য কমান্ডের আউটপুট ছাঁকা	`ps aux | grep nginx`
স্ক্রিপ্টে শর্ত যাচাই	`if grep -q "^bob:" /etc/passwd; then ...`
ডেটা বের করা (IP, email, ফোন নম্বর)	`grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' access.log`

> 📝 **মনে রাখো:** `grep` ফাইল **খোঁজে না**, ফাইলের **ভেতরের লেখা** খোঁজে। নাম দিয়ে ফাইল খুঁজতে `find` বা `locate` লাগে — সেটা অধ্যায় ১২-এ আছে।

২০.২ ভেতরে কীভাবে কাজ করে (Under the hood)

`grep` যখন চলে, তখন মোটামুটি এই ঘটনাগুলো ঘটে:



কিছু গুরুত্বপূর্ণ ব্যাপার:

1. **প্যাটার্ন কম্পাইল হয়।** তুমি যে regex দাও, `grep` সেটাকে একটা ছোট "যন্ত্রে" (state machine — অবস্থা-যন্ত্র) রূপান্তর করে। GNU grep যতটা সম্ভব **DFA** (Deterministic Finite Automaton) ব্যবহার করে — এটা প্রতিটা অক্ষর একবারই দেখে, তাই ভীষণ দ্রুত। কিন্তু back-reference (`\1`)-এর মতো ফিচার DFA দিয়ে হয় না, তখন সে ধীর **backtracking** ইঞ্জিনে চলে যায়।
2. **সাধারণ শব্দ হলে আরও দ্রুত।** প্যাটার্নে যদি কোনো স্থির শব্দ থাকে (যেমন `error`), GNU grep Boyer–Moore ধাঁচের অ্যালগরিদম দিয়ে আগে শুধু সেই শব্দটা খোঁজে — এই অ্যালগরিদম অনেক অক্ষর "লাফিয়ে" যেতে পারে, সব অক্ষর পড়তেও হয় না।
3. **লাইন ভাগ করে না আগে।** grep প্রথমে পুরো buffer-এ মিল খোঁজে, মিল পেলে তবেই আশেপাশের newline খুঁজে লাইনটা আলাদা করে। এ কারণেই বড় ফাইলেও grep এত দ্রুত।
4. **exit status (প্রস্থান কোড)** দিয়ে ফলাফল জানায় — স্ক্রিপ্টে এটা খুব কাজের:

exit status	মানে
`0`	অন্তত একটা লাইন মিলেছে
`1`	কোনো লাইন মেলেনি
`2`	কোনো error হয়েছে (ফাইল নেই, ভুল regex ইত্যাদি)

```
$ grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
$ echo $?
0
$ grep zzzz /etc/passwd
$ echo $?
1
$ grep root /nai/emon/file
grep: /nai/emon/file: No such file or directory
$ echo $?
2
```

> 💡 **টিপস:** `-q` দিলে কোনো ফাইলে error হলেও (যেমন একটা ফাইল নেই) অন্য কোথাও মিল পেলে GNU grep `0`-ই দেয়। শুধু "মিলেছে কি না" জানতে চাইলে `-q` (quiet) ব্যবহার করো — কিছুই ছাপবে না, প্রথম মিল পেলেই থেমে যাবে।

২০.৩ অনুশীলনের জন্য নমুনা ফাইল

এই অধ্যায়ের বেশিরভাগ উদাহরণ নিচের ফাইলটার উপর। চলো আগে বানিয়ে নিই (heredoc কীভাবে কাজ করে, সেটা অধ্যায় ১৯-এ আছে):

```
$ mkdir -p ~/grep-lab && cd ~/grep-lab
$ cat > notes.txt << 'EOF'
```

```
Linux is powerful
linux is free
I love LINUX
Windows is also an OS
macOS is based on Unix

The kernel is the core
error: disk full
Error: permission denied
warning: low memory
EOF
```

লাইন নম্বরসহ দেখি (ষষ্ঠ লাইনটা ফাঁকা, খেয়াল রাখো):

```
$ cat -n notes.txt
     1  Linux is powerful
     2  linux is free
     3  I love LINUX
     4  Windows is also an OS
     5  macOS is based on Unix
     6
     7  The kernel is the core
     8  error: disk full
     9  Error: permission denied
    10  warning: low memory
```

২০.৪ grep-এর মৌলিক ব্যবহার

Syntax

```
grep [OPTIONS] PATTERN [FILE...]
grep [OPTIONS] -e PATTERN ... [FILE...]
grep [OPTIONS] -f PATTERN_FILE ... [FILE...]
```

- `PATTERN` — যা খুঁজবে (সাধারণ শব্দ বা regex)
- `FILE` — কোথায় খুঁজবে। কোনো ফাইল না দিলে grep **stdin** (কীবোর্ড বা পাইপ) থেকে পড়ে।

সবচেয়ে সহজ উদাহরণ

```
$ grep linux notes.txt
linux is free
```

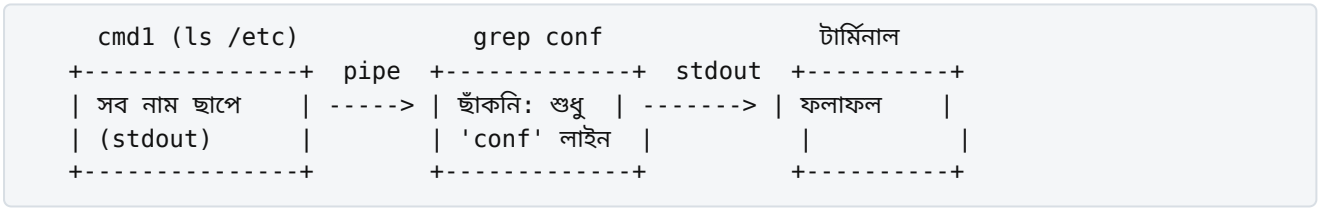
খেয়াল করো — শুধু দ্বিতীয় লাইন এসেছে। কারণ grep ডিফল্টভাবে **case-sensitive** (বড় হাত-ছোট হাতের অক্ষর আলাদা ধরে)। `Linux` আর `LINUX` তার কাছে আলাদা শব্দ।

পাইপ দিয়ে grep

grep সবচেয়ে বেশি ব্যবহার হয় অন্য কমান্ডের আউটপুট ছাঁকতে:

```
$ ls /etc | grep conf
adduser.conf
ca-certificates.conf
debconf.conf
...
```

```
$ history | grep ssh
  112  ssh user@192.168.1.20
  145  sudo systemctl restart ssh
```



প্যাটার্ন সবসময় single quote-এ রাখো

```
$ grep 'error: disk' notes.txt
error: disk full
```

> ⚠️ **সতর্কতা:** প্যাটার্নে space, `*`, `?`, `[`, `$`, `|` ইত্যাদি থাকলে সেটা অবশ্যই `'single quote'`-এ রাখো। নইলে **shell** (শেল) grep-এর আগেই এগুলোকে নিজের মতো ব্যাখ্যা করে ফেলবে। যেমন `grep a* file` লিখলে শেল `a*`-কে বর্তমান ফোল্ডারের `a` দিয়ে শুরু হওয়া ফাইলের নামে বদলে দিতে পারে (globbing) — grep তখন ভুল জিনিস খুঁজবে। Double quote-এর ভেতরে `$` আর `` ` `` তখনও শেল ব্যাখ্যা করে, তাই regex-এর জন্য single quote সবচেয়ে নিরাপদ।

২০.৫ grep-এর অপশন — একটা একটা করে

সব গুরুত্বপূর্ণ অপশনের টেবিল

অপশন	লম্বা রূপ	কাজ
`-i`	`--ignore-case`	বড়/ছোট হাতের অক্ষরের পার্থক্য উপেক্ষা
`-v`	`--invert-match`	যে লাইন **মেলে না** সেগুলো দেখাও
`-n`	`--line-number`	লাইন নম্বর দেখাও
`-c`	`--count`	কতগুলো লাইন মিলেছে শুধু সেই সংখ্যা

অপশন	লম্বা রূপ	কাজ
`-l`	`--files-with-matches`	শুধু যে ফাইলে মিল আছে তার নাম
`-L`	`--files-without-match`	শুধু যে ফাইলে মিল **নেই** তার নাম
`-r`	`--recursive`	ফোল্ডারের ভেতরে সব ফাইলে (সাবফোল্ডারসহ) খোঁজো
`-R`	`--dereference-recursive`	`-r`-এর মতো, কিন্তু সব symlink অনুসরণ করে
`-w`	`--word-regexp`	শুধু পুরো শব্দ মেলাও
`-x`	`--line-regexp`	পুরো লাইন হুবহু মিললে তবেই
`-o`	`--only-matching`	পুরো লাইন না, শুধু মেলা অংশটুকু
`-A N`	`--after-context=N`	মেলা লাইনের **পরের** N লাইনও দেখাও
`-B N`	`--before-context=N`	মেলা লাইনের **আগের** N লাইনও দেখাও
`-C N`	`--context=N`	আগে-পরে দুই দিকেই N লাইন
`-E`	`--extended-regexp`	Extended regex (ERE)
`-F`	`--fixed-strings`	প্যাটার্নকে regex না, সাধারণ লেখা হিসেবে ধরো
`-P`	`--perl-regexp`	Perl-compatible regex (PCRE)
`-G`	`--basic-regexp`	Basic regex (BRE) — এটাই ডিফল্ট
`-e PAT`	`--regexp=PAT`	প্যাটার্ন দাও (একাধিকবার দেওয়া যায়)
`-f FILE`	`--file=FILE`	প্যাটার্নগুলো একটা ফাইল থেকে পড়ো (প্রতি লাইনে একটা)
`-h`	`--no-filename`	আউটপুটে ফাইলের নাম দেখিও না
`-H`	`--with-filename`	সবসময় ফাইলের নাম দেখাও
`-m N`	`--max-count=N`	N-টা মেলা লাইন পেলেই থামো
`-q`	`--quiet`, `--silent`	কিছু ছাপবে না, শুধু exit status
`-s`	`--no-messages`	"ফাইল নেই"-জাতীয় error message লুকাও
`-b`	`--byte-offset`	ফাইলের শুরু থেকে কত বাইট পরে লাইনটা, দেখাও
`-a`	`--text`	binary ফাইলকেও টেক্সট হিসেবে পড়ো
`-I`	—	binary ফাইল বাদ দাও
`-Z`	`--null`	ফাইলের নামের পরে newline-এর বদলে NUL বাইট
`-z`	`--null-data`	ইনপুটের "লাইন" NUL দিয়ে আলাদা ধরো
—	`--color[=WHEN]`	মিল রঙিন করো (`never`, `always`, `auto`)
—	`--include=GLOB`	শুধু এই নামের ফাইলে খোঁজো (`-r`-এর সাথে)
—	`--exclude=GLOB`	এই নামের ফাইল বাদ দাও
—	`--exclude-dir=GLOB`	এই নামের ফোল্ডার বাদ দাও
—	`--line-buffered`	প্রতি লাইন সাথে সাথে আউটপুট দাও (পাইপে লাইভ লগের জন্য)

এবার প্রতিটা বিস্তারিত দেখি।

`-i` — case উপেক্ষা

```
$ grep -i linux notes.txt
Linux is powerful
linux is free
I love LINUX
```

`-n` — লাইন নম্বর

```
$ grep -n -i linux notes.txt
1:Linux is powerful
2:linux is free
3:I love LINUX
```

অপশনগুলো একসাথে জুড়ে লেখা যায়: `grep -ni linux notes.txt` একই কাজ করে।

> 💡 **টিপস:** লাইন নম্বর জানলে সরাসরি এডিটরে সেই লাইনে যেতে পারো: `vim +3 notes.txt` বা `nano +3 notes.txt`। (অধ্যায় ১৩, অধ্যায় ১৪)

`-v` — উল্টো খোঁজা (invert)

যে লাইনে `linux` (যেকোনো case) **নেই**, সেগুলো:

```
$ grep -vi linux notes.txt
Windows is also an OS
macOS is based on Unix

The kernel is the core
error: disk full
Error: permission denied
warning: low memory
```

ফাঁকা লাইনটাও এসেছে, কারণ ওতে `linux` নেই।

বাস্তব ব্যবহার — কনফিগ ফাইল থেকে comment (`#`) আর ফাঁকা লাইন বাদ দিয়ে শুধু আসল সেটিং দেখা:

```
$ grep -v '^#' /etc/ssh/sshd_config | grep -v '^$'
Include /etc/ssh/sshd_config.d/*.conf
KbdInteractiveAuthentication no
UsePAM yes
...
```

(একটা কমান্ডেই: `grep -Ev '^(#|$)' /etc/ssh/sshd_config` — `-E` আর `|` নিচে শিখবে।)

`-c` — গুনে দেখা

```
$ grep -ci linux notes.txt
3
$ grep -vci linux notes.txt
7
```

> ⚠️ **সতর্কতা:** `-c` **লাইন** গোনে, মিলের সংখ্যা না। একটা লাইনে `linux` তিনবার থাকলেও সেটা `1` গোনা হবে। মোট কতবার মিলেছে জানতে: `grep -o linux file | wc -l`।

একাধিক ফাইলে `-c` দিলে প্রতিটা ফাইলের আলাদা হিসাব আসে (যার মধ্যে মিল নেই, সেটা `0`)।

`-w` — পুরো শব্দ মেলানো

এটা খুবই গুরুত্বপূর্ণ, কারণ সাধারণ grep শব্দের **ভেতরের** অংশও মিলিয়ে ফেলে:

```
$ grep -n 'is' notes.txt
1:Linux is powerful
2:linux is free
4:Windows is also an OS
5:macOS is based on Unix
7:The kernel is the core
8:error: disk full
9:Error: permission denied
```

৮ নম্বর লাইনে `is` কোথায়? → d**is**k! ৯ নম্বরে → perm**is**sion! এবার `-w` দিয়ে:

```
$ grep -nw 'is' notes.txt
1:Linux is powerful
2:linux is free
4:Windows is also an OS
5:macOS is based on Unix
7:The kernel is the core
```

`-w`-এর নিয়ম: মেলা অংশের আগে ও পরে হয় লাইনের শুরু/শেষ থাকবে, নয়তো এমন অক্ষর যেটা "word character" না। word character মানে অক্ষর, সংখ্যা আর underscore (`_`)।

`-x` — পুরো লাইন হুবহু

```
$ grep -x 'linux is free' notes.txt
linux is free
$ grep -x 'linux' notes.txt
$
```

দ্বিতীয়টায় কিছুই আসেনি, কারণ কোনো লাইন **শুধু** `linux` না। `-x` অনেকটা প্যাটার্নের দুই পাশে `^` আর `$` বসানোর মতো (নিচে দেখবে)।

`-o` — শুধু মেলা অংশ

```
$ grep -oi 'linux' notes.txt
Linux
linux
LINUX
```

ডেটা বের করার (extract) কাজে `-o` সবচেয়ে দরকারি — IP, email, URL বের করা এটা দিয়েই হয়। একই লাইনে একাধিক মিল থাকলে প্রতিটা আলাদা লাইনে আসে।

`-A`, `-B`, `-C` — আশেপাশের লাইন (context)

লগ পড়ার সময় শুধু error লাইনটা যথেষ্ট না — তার আগে-পরে কী হয়েছিল সেটাও জানা দরকার।

```
$ grep -A 1 kernel notes.txt
The kernel is the core
error: disk full

$ grep -B 1 kernel notes.txt

The kernel is the core

$ grep -n -C 1 kernel notes.txt
6-
7:The kernel is the core
8-error: disk full
```

খেয়াল করো: `-n`-এর সাথে **মেলা লাইনের** নম্বরের পরে `:` আর **context লাইনের** পরে `-` বসে। এভাবে চোখে পড়ে কোনটা আসল মিল।

একাধিক মিলের মাঝে ফাঁক থাকলে grep `--` দিয়ে দলগুলো আলাদা করে:

```
$ grep -A 1 -i '^error' notes.txt
error: disk full
Error: permission denied
warning: low memory
```

প্রথমটায় কোনো `--` নেই, কারণ ৮, ৯, ১০ নম্বর লাইন পরপর — দলগুলো জোড়া লেগে গেছে। এবার দূরের দুটো মিল:

```
$ grep -A1 -e Windows -e kernel notes.txt
Windows is also an OS
macOS is based on Unix
--
The kernel is the core
error: disk full
```

`-l` ও `-L` — শুধু ফাইলের নাম

একটা ছোট প্রজেক্ট ফোল্ডার বানাই:

```
$ mkdir -p project/notes
$ printf 'import os\n\n# TODO: fix login\nprint("hello")\n' > project/app.py
$ echo 'TODO: write docs' > project/README.md
$ echo 'debug = true' > project/config.ini
$ echo 'buy milk' > project/notes/todo.txt
```

```
$ grep -rl TODO project
project/app.py
project/README.md

$ grep -rL TODO project
project/config.ini
project/notes/todo.txt
```

(ফাইলের ক্রম তোমার মেশিনে আলাদা হতে পারে — grep ডিরেক্টরি যে ক্রমে পড়ে, সেই ক্রমে দেখায়।)

`-r` ও `-R` — ফোল্ডারের ভেতরে খোঁজা

```
$ grep -rn TODO project
project/app.py:3:# TODO: fix login
project/README.md:1:TODO: write docs
```

একাধিক ফাইলে খুঁজলে grep নিজে থেকেই প্রতিটা লাইনের আগে ফাইলের নাম বসায় (`file:line:text`)।

`-r` বনাম `-R` পার্থক্য:

	`-r`	`-R`
কমান্ড লাইনে সরাসরি দেওয়া symlink	অনুসরণ করে	অনুসরণ করে
ফোল্ডারের ভেতরে পাওয়া symlink	**অনুসরণ করে না**	অনুসরণ করে
ঝুঁকি	কম	symlink লুপ বা বিশাল জায়গায় ঢুকে যেতে পারে

symlink কী, সেটা অধ্যায় ১১-এ আছে।

> 💡 **টিপস:** `grep -r pattern` লিখে কোনো ফোল্ডার না দিলে GNU grep বর্তমান ফোল্ডার (`.`) ধরে নেয়।

`--include`, `--exclude`, `--exclude-dir` — কোন ফাইলে খুঁজবে

```
$ grep -rn --include='*.py' TODO project
project/app.py:3:# TODO: fix login
```

```
$ grep -rn --exclude='*.md' TODO project
project/app.py:3:# TODO: fix login

$ grep -rn --exclude-dir=notes buy project
$
```

বাস্তব ডেভেলপার জীবনে সবচেয়ে কাজের লাইন:

```
$ grep -rn --exclude-dir={.git,node_modules,vendor} 'function login' .
```

এখানে `{.git,node_modules,vendor}` হলো **bash-এর brace expansion** — শেল এটাকে তিনটা আলাদা `--exclude-dir=...` অপশনে ভেঙে দেয়। একাধিক এক্সটেনশন: `--include='*.php' --include='*.js'`।

> ⚠️ **সতর্কতা:** `--include`-এর glob-টাও quote-এ রাখো (`'*.py'`), নইলে বর্তমান ফোল্ডারে `.py` ফাইল থাকলে শেল সেটা আগেই expand করে দেবে।

`-E`, `-F`, `-P`, `-G` — regex-এর ধরন বাছাই

এগুলো নিচে regex অংশে বিস্তারিত আছে। সংক্ষেপে:

```
$ grep -E 'error|warning' notes.txt          # ERE: | মানে "অথবা"
error: disk full
warning: low memory

$ grep -F 'a.b*c' file.txt                   # হুবহু "a.b*c" লেখাটাই খুঁজবে
$ grep -P '\d+' file.txt                     # Perl-ধাঁচের \d (সংখ্যা)
```

`-F` শুধু নিরাপদই না, দ্রুতও — কারণ regex ইঞ্জিন লাগে না। ডট, তারা, ব্র্যাকেটওয়ালা লেখা (যেমন IP `192.168.1.1` বা `array[0]`) হুবহু খুঁজতে `-F` দাও।

`-e` ও `-f` — একাধিক প্যাটার্ন

```
$ grep -e error -e warning notes.txt
error: disk full
warning: low memory
```

প্যাটার্ন যদি `-` দিয়ে শুরু হয়, তাহলে grep সেটাকে অপশন ভেবে ভুল করবে। সমাধান `-e` বা `--`:

```
$ grep -e '-v' file.txt
$ grep -- '-v' file.txt
```

অনেকগুলো প্যাটার্ন একটা ফাইলে রেখে:

```
$ printf 'error\nwarning\n' > patterns.txt
$ grep -f patterns.txt notes.txt
error: disk full
warning: low memory
```

`-h`, `-H`, `-m`, `-b`, `-q`, `-s`

```
$ grep -h TODO project/app.py project/README.md     # ফাইলের নাম ছাড়া
# TODO: fix login
TODO: write docs

$ grep -H kernel notes.txt                          # এক ফাইলেও নাম দেখাও
notes.txt:The kernel is the core

$ grep -m 1 -i linux notes.txt                      # প্রথম মিলেই থামো
Linux is powerful

$ grep -b kernel notes.txt                          # বাইট অফসেট
91:The kernel is the core

$ grep -q kernel notes.txt && echo "পাওয়া গেছে"
পাওয়া গেছে

$ grep -s TODO nai.txt notes.txt                     # nai.txt না থাকলেও চুপ
$
```

`--color` — রঙিন হাইলাইট

```
$ grep --color=auto kernel notes.txt
```

- `auto` — আউটপুট টার্মিনালে গেলে রঙ, পাইপ/ফাইলে গেলে রঙ না
- `always` — সবসময় রঙ (escape code সহ)
- `never` — কখনো না

Ubuntu ও Debian-এর ডিফল্ট `~/.bashrc`-এ `alias grep='grep --color=auto'` আগে থেকেই থাকে, তাই রঙ এমনিতেই দেখো। Fedora/RHEL-এ `/etc/profile.d/colorgrep.sh` একই কাজ করে।

রঙসহ আউটপুট `less`-এ দেখতে:

```
$ grep --color=always -rn TODO . | less -R
```

রঙ বদলাতে `GREP_COLORS` এনভায়রনমেন্ট ভেরিয়েবল (যেমন `export GREP_COLORS='mt=01;32'` — মিল সবুজ, বোল্ড)। পুরোনো `GREP_COLOR` ভেরিয়েবল এখন obsolete; নতুন grep এটা দেখলে warning দেয়। এনভায়রনমেন্ট ভেরিয়েবল নিয়ে বিস্তারিত অধ্যায় ৪৪-এ।

Binary ফাইল নিয়ে

```
$ grep -r ELF /usr/bin/ls
grep: /usr/bin/ls: binary file matches
```

grep দেখল ফাইলটা binary (ভেতরে NUL বাইট আছে), তাই আস্ত লাইন না ছেপে শুধু জানিয়ে দিল। বিকল্প:

- `-a` — জোর করে টেক্সট ধরে ছাপো (টার্মিনাল আবর্জনায় ভরে যেতে পারে)
- `-I` — binary ফাইল একেবারে বাদ দাও (কোড খোঁজার সময় কাজের)
- binary থেকে পড়ার মতো লেখা বের করতে `strings /usr/bin/ls | grep GLIBC`

`-Z` / `-z` — অদ্ভুত ফাইলনাম সামলানো

ফাইলের নামে space থাকলে `grep -l ... | xargs rm` ভেঙে যায়। NUL দিয়ে আলাদা করলে নিরাপদ:

```
$ grep -rlZ TODO project | xargs -0 ls -l
```

`xargs` নিয়ে বিস্তারিত অধ্যায় ২৩-এ।

`--line-buffered` — লাইভ লগ

```
$ tail -f /var/log/nginx/access.log | grep --line-buffered ' 500 ' | tee errors.txt
```

পাইপে লেখার সময় grep আউটপুট জমিয়ে (buffer) একবারে পাঠায়; `--line-buffered` দিলে প্রতিটা লাইন সাথে সাথে পাঠায়, ফলে লাইভ দেখা যায়।

২০.৬ রেগুলার এক্সপ্রেশন (Regex) — খোঁজার ভাষা

Regex কী?

Regular expression (রেগুলার এক্সপ্রেশন, সংক্ষেপে regex বা regexp) হলো একটা ছোট্ট "ভাষা", যেটা দিয়ে **লেখার নকশা** বর্ণনা করা হয়। "ঠিক এই শব্দটা খোঁজো" না বলে তুমি বলতে পারো:

- "যে লাইন `error` দিয়ে **শুরু**"
- "যেকোনো **তিন অঙ্কের সংখ্যা**"
- "`@` চিহ্নের দুই পাশে কিছু অক্ষর, তারপর ডট আর ডোমেইন" — মানে email

উপমা: ধরো তুমি পুলিশকে চোরের বর্ণনা দিচ্ছ — "লম্বা, কালো শার্ট, বাম হাতে ঘড়ি।" তুমি নাম জানো না, কিন্তু **বৈশিষ্ট্য** জানো। regex হলো লেখার জন্য এমন একটা বর্ণনা — নির্দিষ্ট লেখা না, লেখার **ধরন**।

Regex শুধু grep-এ না — `sed` (অধ্যায় ২১), `awk` (অধ্যায় ২২), `vim`, `less`, Python, JavaScript, PHP — প্রায় সব জায়গায় ব্যবহার হয়। একবার শিখলে সারাজীবন কাজে লাগবে।

দুই ধরনের অক্ষর

1. **Literal (আক্ষরিক)** — নিজেকেই বোঝায়। `a` মানে `a`, `cat` মানে `c-a-t`।
2. **Metacharacter (মেটাক্যারেক্টার)** — বিশেষ ক্ষমতাওয়ালা অক্ষর, যেমন `.` মানে "যেকোনো একটা অক্ষর"।

Regex ইঞ্জিন কীভাবে মেলায়

```
প্যাটার্ন:  c.t            লেখা:  "the cat sat on a cot"

 অবস্থান 0:  t  -> 'c' চাই, 't' পেলাম ..... ব্যর্থ, এক ঘর এগোও
 অবস্থান 1:  h  -> ব্যর্থ
 অবস্থান 2:  e  -> ব্যর্থ
 অবস্থান 3:  ' '-> ব্যর্থ
 অবস্থান 4:  c  -> 'c' মিলল
              a  -> '.' যেকোনো অক্ষর, মিলল
              t  -> 't' মিলল ......................... মিল! "cat"
 (grep -o হলে এখান থেকে আবার খোঁজা শুরু, পরে "cot"-ও পাবে)
```

ইঞ্জিন বাম থেকে ডানে প্রতিটা অবস্থান থেকে মেলানোর চেষ্টা করে; প্রথম যেখানে মেলে, সেটাই নেয় — একে বলে **leftmost match** (সবচেয়ে বামের মিল)।

২০.৭ মেটাক্যারেক্টার — এক এক করে

নিচের উদাহরণগুলোতে ERE (`grep -E`) ব্যবহার করব, কারণ এতে backslash কম লাগে। BRE-তে কী আলাদা, সেটা পরের সেকশনে।

সংক্ষিপ্ত তালিকা

মেটাক্যারেক্টার	মানে	উদাহরণ	কী মেলে
`.`	যেকোনো একটা অক্ষর (newline বাদে)	`c.t`	cat, cot, c9t, c t
`*`	আগেরটা **০ বা তার বেশি** বার	`ab*c`	ac, abc, abbbc
`+`	আগেরটা **১ বা তার বেশি** বার	`ab+c`	abc, abbc (ac না)
`?`	আগেরটা **০ বা ১** বার (ঐচ্ছিক)	`colou?r`	color, colour
`^`	লাইনের **শুরু**	`^error`	"error" দিয়ে শুরু লাইন
`$`	লাইনের **শেষ**	`full$`	"full" দিয়ে শেষ লাইন
`[abc]`	ব্র্যাকেটের যেকোনো **একটা** অক্ষর	`gr[ae]y`	gray, grey

মেটাক্যারেক্টার	মানে	উদাহরণ	কী মেলে
`[a-z]`	রেঞ্জ	`[0-9]`	যেকোনো একটা অঙ্ক
`[^abc]`	এগুলো **ছাড়া** যেকোনো একটা	`[^0-9]`	অঙ্ক নয় এমন অক্ষর
`{n}`	ঠিক n বার	`[0-9]{4}`	2026
`{n,}`	অন্তত n বার	`a{2,}`	aa, aaa, …
`{n,m}`	n থেকে m বার	`[0-9]{1,3}`	7, 42, 255
`{,m}`	সর্বোচ্চ m বার (GNU)	`x{,2}`	(ফাঁকা), x, xx
`( )`	গ্রুপ (দল)	`(ab)+`	ab, abab
`\`	পরের মেটাক্যারেক্টারকে সাধারণ বানাও (escape)	`\.`	আসল ডট
`\b`	শব্দের সীমানা (word boundary)	`\bis\b`	আলাদা শব্দ "is"
`\B`	শব্দের সীমানা **নয়**	`\Bis\B`	d**is**k-এর is
`\<` `\>`	শব্দের শুরু / শেষ (GNU)	`\<un`	un দিয়ে শুরু শব্দ
`\w`	word character (অক্ষর, অঙ্ক, `_`) (GNU)	`\w+`	hello_123
`\W`	word character নয়		
`\s` / `\S`	whitespace / whitespace নয় (GNU)	`\s+`	স্পেস, ট্যাব
`\d`	অঙ্ক — **শুধু `-P` মোডে**	`\d{3}`	123
`\1`, `\2`	back-reference: আগের গ্রুপ যা মিলেছে, হুবহু তাই আবার	`(a)\1`	aa

আরেকটা জরুরি মেটাক্যারেক্টার হলো **পাইপ চিহ্ন** — মানে "অথবা" (alternation)। ERE-তে `cat|dog` মানে cat অথবা dog (টেবিলের ভেতরে পাইপ লেখা ঝামেলা, তাই এখানে আলাদা করে বললাম)।

`.` — যেকোনো একটা অক্ষর

```
$ echo -e "cat\ncut\ncart\nct" | grep 'c.t'
cat
cut
```

`cart`-এ `c` আর `t`-এর মাঝে দুটো অক্ষর, আর `ct`-তে কোনো অক্ষরই নেই — তাই মেলেনি।

> ⚠️ **সতর্কতা:** আসল ডট খুঁজতে `\.` লিখতে হবে। `grep '192.168.1.1'` লিখলে `192x168y1z1`-ও মিলে যাবে! নিরাপদ পথ: `grep -F '192.168.1.1'` অথবা `grep '192\.168\.1\.1'`।

`^` ও `$` — anchor (নোঙর)

anchor কোনো অক্ষর মেলায় না, **অবস্থান** মেলায়।

```
$ grep -i '^error' notes.txt
error: disk full
Error: permission denied

$ grep 'ory$' notes.txt
warning: low memory

$ grep -c '^$' notes.txt        # ফাঁকা লাইন গোনা
1

$ grep -n '^.*$' notes.txt | head -2
1:Linux is powerful
2:linux is free
```

📝 **মনে রাখো:** `^` ব্র্যাকেটের **বাইরে** মানে "লাইনের শুরু", কিন্তু `[^...]`-এর **ভেতরে প্রথমে** থাকলে মানে "এগুলো ছাড়া"। একই চিহ্ন, জায়গাভেদে দুই মানে।

⚠️ **সতর্কতা (Windows ফাইল):** Windows-এ বানানো ফাইলের লাইনের শেষে অদৃশ্য `\r` (carriage return) থাকে। তখন `grep 'full$'` মেলে না, কারণ শেষ অক্ষর `l` না, `\r`। সমাধান: `dos2unix file` অথবা `grep $'full\r$'`। `cat -A file` দিলে লাইনের শেষে `^M$` দেখলেই বুঝবে।

`[ ]` — character class (অক্ষরের দল)

```
$ echo -e "gray\ngrey\ngroy" | grep 'gr[ae]y'
gray
grey

$ echo -e "file1\nfileA\nfile_" | grep 'file[0-9]'
file1

$ echo -e "file1\nfileA\nfile_" | grep 'file[^0-9]'
fileA
file_
```

ব্র্যাকেটের ভেতরের নিয়ম:

লেখা	মানে
`[abc]`	a, b, বা c
`[a-z]`, `[A-Z]`, `[0-9]`	রেঞ্জ
`[a-zA-Z0-9_]`	একাধিক রেঞ্জ একসাথে
`[^abc]`	a, b, c বাদে যেকোনো
`[]abc]`	`]`-কে সাধারণ অক্ষর রাখতে **প্রথমে** বসাও
`[abc-]` বা `[-abc]`	`-`-কে সাধারণ রাখতে শুরুতে বা শেষে বসাও

লেখা	মানে
`[.*]`	ব্র্যাকেটের ভেতরে `.` আর `*` সাধারণ অক্ষর! escape লাগে না

> ⚠️ **সতর্কতা (locale):** কিছু locale-এ `[a-z]` রেঞ্জ শুধু ছোট হাতের অক্ষরে সীমাবদ্ধ না-ও থাকতে পারে (পুরোনো সিস্টেমে এতে বড় হাতের অক্ষরও ঢুকে যেত)। আধুনিক GNU grep সাধারণত ঠিকঠাক আচরণ করে, তবু নিশ্চিত থাকতে POSIX class (`[[:lower:]]`) অথবা `LC_ALL=C grep ...` ব্যবহার করো।

`*`, `+`, `?` — quantifier (কতবার)

quantifier সবসময় তার **ঠিক আগের** জিনিসটার উপর কাজ করে — একটা অক্ষর, একটা `[...]`, বা একটা `(...)` গ্রুপ।

```
$ echo -e "ac\nabc\nabbbc" | grep -E 'ab*c'
ac
abc
abbbc

$ echo -e "ac\nabc\nabbbc" | grep -E 'ab+c'
abc
abbbc

$ echo -e "color\ncolour\ncolouur" | grep -E '^colou?r$'
color
colour
```

> ⚠️ **সতর্কতা — শুরুর সবচেয়ে কমন ভুল:** shell-এর glob-এ `*` মানে "যেকোনো কিছু", কিন্তু regex-এ `*` মানে "**আগেরটা** ০ বা বেশিবার"। regex-এ "যেকোনো কিছু" হলো `.*`। আর `grep 'a*'` প্রতিটা লাইনেই মিলবে — কারণ ০টা `a`-ও তো মিল!

`{n,m}` — নির্দিষ্ট সংখ্যক বার

```
$ echo -e "7\n42\n255\n1000" | grep -E '^[0-9]{1,3}$'
7
42
255

$ echo -e "01712345678\n0171234\n02812345678" | grep -E '^01[3-9][0-9]{8}$'
01712345678
```

শেষেরটা বাংলাদেশি মোবাইল নম্বর যাচাই: `01`, তারপর ৩–৯ এর একটা অঙ্ক, তারপর ঠিক ৮টা অঙ্ক = মোট ১১ অঙ্ক।

`( )` ও `|` — গ্রুপ ও বিকল্প

```
$ grep -E 'error|warning' notes.txt
error: disk full
warning: low memory

$ echo -e "ha\nhaha\nhahaha\nhah" | grep -Ex '(ha)+'
ha
haha
hahaha
```

`|`-এর অগ্রাধিকার সবচেয়ে কম। `^error|warning$` মানে "(`^error`) অথবা (`warning$`)", **না** যে "^(error|warning)$"। সীমা ঠিক করতে বন্ধনী দাও:

```
$ grep -Ei '^(error|warning):' notes.txt
error: disk full
Error: permission denied
warning: low memory
```

Back-reference `\1` — "আগে যা পেয়েছি, আবার সেটাই"

```
$ echo -e "the the cat\nthe cat\nis is" | grep -E '\b(\w+) \1\b'
the the cat
is is
```

এটা ডুপ্লিকেট শব্দ ধরে — লেখা প্রুফরিড করার জন্য দারুণ। `(\w+)` যা মেলাবে, `\1` ঠিক সেটাকেই আবার চায়।

> 📝 **মনে রাখো:** POSIX-এ back-reference শুধু BRE-তে আছে; GNU grep ERE-তেও দেয়। Back-reference থাকলে grep দ্রুত DFA বাদ দিয়ে ধীর ইঞ্জিনে চলে, বড় ফাইলে স্লো লাগতে পারে।

`\b`, `\<`, `\>` — শব্দের সীমানা

```
$ grep -n '\bis\b' notes.txt      # grep -w 'is' এর মতোই ফল
1:Linux is powerful
2:linux is free
4:Windows is also an OS
5:macOS is based on Unix
7:The kernel is the core

$ grep -o '\<[Ww][a-z]*' notes.txt
Windows
warning
```

`\b` শব্দের শুরু বা শেষ দুটোই বোঝায়; `\<` শুধু শুরু, `\>` শুধু শেষ।

`\d` — অঙ্ক (শুধু PCRE)

```
$ echo "Order 4521 shipped" | grep -oP '\d+'
4521

$ echo "Order 4521 shipped" | grep -oE '[0-9]+'
4521
```

> ⚠️ **সতর্কতা:** `\d` POSIX regex-এর অংশ **না**। `-P` ছাড়া `grep '\d'` লিখলে GNU grep সেটাকে সাধারণ `d` অক্ষর হিসেবে মেলায় — Ubuntu 24.04-এর grep 3.11-এ কোনো error বা warning ছাড়াই! মানে `echo xd | grep '\d'` মিলে যাবে, অথচ কোনো সংখ্যাই নেই। এটা নিঃশব্দ ভুল, তাই সবচেয়ে বিপজ্জনক। BRE/ERE-তে অঙ্কের জন্য `[0-9]` বা `[[:digit:]]` লেখো।

২০.৮ POSIX character class

`[[:name:]]` রূপের এই class-গুলো locale-নিরাপদ এবং সব POSIX টুলে (grep, sed, awk, tr) কাজ করে।

Class	মানে	সমতুল্য (ASCII-তে)
`[[:digit:]]`	অঙ্ক	`[0-9]`
`[[:alpha:]]`	বর্ণ	`[A-Za-z]`
`[[:alnum:]]`	বর্ণ বা অঙ্ক	`[A-Za-z0-9]`
`[[:lower:]]`	ছোট হাতের	`[a-z]`
`[[:upper:]]`	বড় হাতের	`[A-Z]`
`[[:space:]]`	সব whitespace (space, tab, newline, CR, FF, VT)	`[ \t\n\r\f\v]`
`[[:blank:]]`	শুধু space আর tab	`[ \t]`
`[[:punct:]]`	যতিচিহ্ন	`[!-/:-@[-`{-~]`
`[[:xdigit:]]`	hex অঙ্ক	`[0-9A-Fa-f]`
`[[:print:]]`	ছাপার যোগ্য (space সহ)	
`[[:graph:]]`	ছাপার যোগ্য (space ছাড়া)	
`[[:cntrl:]]`	control character	

ডাবল ব্র্যাকেট কেন? বাইরের `[ ]` হলো সাধারণ ব্র্যাকেট এক্সপ্রেশন, ভেতরের `[:digit:]` হলো তার একটা সদস্য। তাই অন্যগুলোর সাথে মেশানো যায়: `[[:digit:]_-]` মানে "অঙ্ক, underscore বা হাইফেন"।

```
$ echo -e "abc\nabc123\n   \n" | grep -c '^[[:space:]]*$'
2
```

```
$ echo "MAC: 3c:22:fb:9a:01:ee" | grep -oE '([[:xdigit:]]{2}:){5}[[:xdigit:]]{2}'
3c:22:fb:9a:01:ee
```

(প্রথম উদাহরণে `echo -e` শেষে নিজের একটা newline যোগ করে, তাই `" "` লাইন আর একটা পুরো ফাঁকা লাইন — মোট ২টা।)

> ⚠️ **সতর্কতা:** `grep '[:digit:]'` (একক ব্র্যাকেট) ভুল। GNU grep তখন error দেয়:
>
> ```
> $ grep '[:digit:]' notes.txt
> grep: character class syntax is [[:space:]], not [:space:]
> ```

২০.৯ BRE বনাম ERE বনাম PCRE

ইতিহাসের কারণে regex-এর তিনটা প্রধান "উপভাষা" (dialect) আছে:

- **BRE (Basic Regular Expression)** — সবচেয়ে পুরোনো। grep-এর ডিফল্ট (`grep` বা `grep -G`)। এখানে `+ ? | ( ) { }` সাধারণ অক্ষর; বিশেষ ক্ষমতা পেতে সামনে `\` লাগে।
- **ERE (Extended Regular Expression)** — `grep -E`। এখানে ওই চিহ্নগুলো সরাসরি বিশেষ; সাধারণ অক্ষর হিসেবে চাইলে `\` লাগে। পড়তে অনেক সহজ।
- **PCRE (Perl Compatible Regular Expression)** — `grep -P`। Perl ভাষা থেকে আসা সবচেয়ে শক্তিশালী উপভাষা; Python, JavaScript, PHP-র regex-ও প্রায় এমন। `\d`, lazy quantifier, lookahead/lookbehind এখানে আছে।

তুলনামূলক টেবিল

ফিচার	BRE (`grep`)	ERE (`grep -E`)	PCRE (`grep -P`)
যেকোনো অক্ষর	`.`	`.`	`.`
০ বা বেশি	`*`	`*`	`*`
১ বা বেশি	`\+` (GNU)	`+`	`+`
০ বা ১	`\?` (GNU)	`?`	`?`
অথবা (alternation)	টেবিলের নিচে দেখো	টেবিলের নিচে দেখো	টেবিলের নিচে দেখো
গ্রুপ	`\( \)`	`( )`	`( )`
পুনরাবৃত্তি	`\{n,m\}`	`{n,m}`	`{n,m}`
back-reference	`\1`	`\1` (GNU)	`\1`
`\b \w \s`	আছে (GNU)	আছে (GNU)	আছে
`\d` (অঙ্ক)	নেই	নেই	আছে

ফিচার	BRE (`grep`)	ERE (`grep -E`)	PCRE (`grep -P`)
lazy `*?` `+?`	নেই	নেই	আছে
lookahead `(?=...)` / lookbehind `(?<=...)`	নেই	নেই	আছে
`\K` (এর আগের অংশ ফলাফল থেকে বাদ)	নেই	নেই	আছে
case-insensitive অংশ `(?i)`	নেই	নেই	আছে

অথবা-চিহ্ন: BRE-তে `a\|b` (GNU এক্সটেনশন), ERE ও PCRE-তে সরাসরি `a|b`।

একই প্যাটার্ন তিনভাবে — "`ab` এক বা একাধিকবার, তারপর `c` বা `d`":

```
$ echo "ababd" | grep    '\(ab\)\+\(c\|d\)'      # BRE
ababd
$ echo "ababd" | grep -E '(ab)+(c|d)'            # ERE
ababd
$ echo "ababd" | grep -P '(ab)+(c|d)'            # PCRE
ababd
```

BRE-তে backslash-এর জঙ্গল দেখেছ? এ কারণেই বেশিরভাগ মানুষ `grep -E` ব্যবহার করে।

```
       regex-এর শক্তি ও জটিলতা
কম  <----------------------------------->  বেশি

grep -F      grep (BRE)     grep -E (ERE)     grep -P (PCRE)
হুবহু লেখা    পুরোনো, \ বেশি   পরিষ্কার, জনপ্রিয়    সবচেয়ে শক্তিশালী
সবচেয়ে দ্রুত    দ্রুত (DFA)     দ্রুত (DFA)        backtracking, কখনো ধীর
```

PCRE-র বিশেষ ক্ষমতা — উদাহরণ

Lookbehind + `\K` দিয়ে শুধু মান বের করা:

```
$ echo "user=rahim id=42" | grep -oP 'user=\K\w+'
rahim

$ echo "user=rahim id=42" | grep -oP '(?<=id=)\d+'
42
```

Lookahead — পরে কিছু আছে এমন অংশ:

```
$ echo "price: 500taka 20usd" | grep -oP '\d+(?=taka)'
500
```

Case-insensitive একটা অংশ:

```
$ echo "LINUX rocks" | grep -P '(?i)linux'
LINUX rocks
```

> 💡 **টিপস:** `-P` GNU grep-এর একটা ঐচ্ছিক ফিচার — PCRE লাইব্রেরি নিয়ে কম্পাইল করা থাকতে হয়। Ubuntu, Debian, Fedora, RHEL-এর grep-এ এটা আছে, কিন্তু macOS বা Alpine-এর (BusyBox) grep-এ নেই। পোর্টেবল স্ক্রিপ্টে তাই ERE-কেই প্রাধান্য দাও।

> ⚠️ **সতর্কতা:** PCRE backtracking ইঞ্জিন। `(a+)+$` -এর মতো প্যাটার্ন লম্বা লেখায় অত্যধিক সময় নিতে পারে (catastrophic backtracking)। GNU grep তখন সীমা পেরোলে `exceeded PCRE's backtracking limit` জাতীয় error দেয়। নেস্টেড quantifier এড়িয়ে চলো।

২০.১০ Greedy matching — "লোভী" মিল

regex-এর `*`, `+`, `?`, `{n,m}` ডিফল্টভাবে **greedy** (লোভী) — যতটা সম্ভব **বেশি** অক্ষর গিলে ফেলে, তবে পুরো প্যাটার্ন যেন মেলে সেটা নিশ্চিত রেখে।

উপমা: বুফেতে একজন লোভী মানুষ প্লেট যতটা ধরে ততটা খাবার তোলে; শুধু শেষ আইটেম রাখার জায়গা না থাকলে একটু ফেরত দেয়।

```
$ echo '<b>bold</b> and <i>italic</i>' | grep -o '<.*>'
<b>bold</b> and <i>italic</i>
```

আমরা চেয়েছিলাম `<b>`, `</b>` ইত্যাদি আলাদা ট্যাগ, পেলাম পুরো লাইন! কারণ:

```
  <b>bold</b> and <i>italic</i>
  ^                          ^
  '<' এখানে মিলল            '.*' লাইনের শেষ পর্যন্ত গিলে নিল,
                             তারপর '>' পেতে একটু একটু পিছিয়ে (backtrack)
                             সবচেয়ে শেষের '>'-এ থামল
```

সমাধান ১ — negated class (POSIX, সব জায়গায় চলে, দ্রুত):

```
$ echo '<b>bold</b> and <i>italic</i>' | grep -o '<[^>]*>'
<b>
</b>
<i>
</i>
```

`[^>]*` মানে "`>` ছাড়া যেকোনো কিছু" — তাই প্রথম `>`-এর আগেই থামতে বাধ্য।

সমাধান ২ — lazy quantifier (শুধু PCRE):

```
$ echo '<b>bold</b> and <i>italic</i>' | grep -oP '<.*?>'
<b>
```

```
</b>
<i>
</i>
```

`*?` মানে "যত **কম** সম্ভব"। PCRE-তে `+?`, `??`, `{n,m}?`-ও আছে।

> 📝 **মনে রাখো:** POSIX (BRE/ERE) ইঞ্জিন "leftmost-longest" নিয়মে চলে — সবচেয়ে বামের মিল, এবং সেখান থেকে **সবচেয়ে লম্বা** মিল। তাই BRE/ERE-তে lazy বলে কিছু নেই; negated class-ই ভরসা।

২০.১১ egrep ও fgrep — পুরোনো নাম

আগে তিনটা আলাদা প্রোগ্রাম ছিল: `grep` (BRE), `egrep` (ERE), `fgrep` (fixed string)। এখন সব কাজ এক `grep` দিয়েই হয়:

পুরোনো	আধুনিক সমতুল্য
`egrep '(ab)+'`	`grep -E '(ab)+'`
`fgrep 'a.b'`	`grep -F 'a.b'`

GNU grep ৩.৮ (২০২২) থেকে এগুলোকে আনুষ্ঠানিকভাবে obsolescent (বিলুপ্তির পথে) ঘোষণা করা হয়েছে। আসল GNU সংস্করণের `egrep` চালালে এমন warning আসে:

```
$ egrep 'error|warning' notes.txt
egrep: warning: egrep is obsolescent; using grep -E
error: disk full
warning: low memory
```

Debian ও Ubuntu (Debian 12, Ubuntu 24.04) নিজেদের প্যাকেজে `egrep` / `fgrep`-কে ছোট্ট wrapper স্ক্রিপ্ট বানিয়ে রেখেছে, যেটা warning ছাড়াই `grep -E` / `grep -F` চালায় — তাই সেখানে warning না-ও দেখতে পারো। তবু অন্য distro-তে বা ভবিষ্যতে এগুলো সরে যেতে পারে, তাই পুরোনো স্ক্রিপ্টে `egrep` থাকলে `grep -E`-এ বদলে নাও।

grep পরিবারের অন্য সদস্য

কমান্ড	কাজ
`zgrep`	`.gz` কম্প্রেস করা ফাইলে খোঁজে (যেমন `/var/log/syslog.2.gz`) — খোলা ছাড়াই
`xzgrep`, `bzgrep`, `zstdgrep`	`.xz`, `.bz2`, `.zst` ফাইলের জন্য (প্যাকেজ ইনস্টল থাকলে)
`pgrep`	নাম দিয়ে প্রসেস খোঁজে (অধ্যায় ২৪)
`journalctl -g PATTERN`	systemd journal-এ regex দিয়ে খোঁজে (অধ্যায় ২৮)

কমান্ড	কাজ
`git grep`	git রিপোজিটরিতে ট্র্যাক করা ফাইলে খোঁজে

```
$ zgrep -i 'failed' /var/log/auth.log.2.gz
```

২০.১২ ripgrep (`rg`) — আধুনিক, তুফান-গতির grep

ripgrep হলো Rust ভাষায় লেখা একটা নতুন টুল (কমান্ড `rg`)। কোড খোঁজার জন্য এটা এখন ডেভেলপারদের প্রিয়। VS Code-এর "Search in files" ভেতরে ripgrep-ই ব্যবহার করে।

কেন দ্রুত ও সুবিধাজনক?

- **ডিফল্টভাবে recursive** — `-r` লাগে না।
- **`.gitignore` মানে** — `node_modules`, build ফোল্ডার ইত্যাদি নিজেই বাদ দেয়।
- **লুকানো ফাইল (`.` দিয়ে শুরু) আর binary ফাইল ডিফল্টভাবে বাদ।**
- **একাধিক CPU core** ব্যবহার করে অনেক ফাইল একসাথে (parallel) পড়ে।
- টার্মিনালে লাইন নম্বর, রঙ, ফাইল অনুযায়ী গ্রুপ — সব ডিফল্ট।
- regex ইঞ্জিন Rust-এর `regex` crate — backtracking ছাড়া, তাই catastrophic backtracking হয় না।

ইনস্টলেশন

```
$ sudo apt install ripgrep          # Ubuntu / Debian
$ sudo dnf install ripgrep          # Fedora (RHEL 9-এ EPEL রিপো চালু করে)
```

প্যাকেজ ম্যানেজমেন্ট নিয়ে বিস্তারিত অধ্যায় ৩০-এ।

ব্যবহার

```
$ rg TODO project
project/app.py
3:# TODO: fix login

project/README.md
1:TODO: write docs
```

(ফাইলের ক্রম বদলাতে পারে, কারণ rg একসাথে অনেক ফাইল পড়ে।)

গুরুত্বপূর্ণ অপশন

অপশন	কাজ	grep-এর সমতুল্য
`-i`	case উপেক্ষা	`-i`
`-S` / `--smart-case`	প্যাটার্ন সব ছোট হাতের হলে case উপেক্ষা, বড় হাতের থাকলে না	—
`-w`	পুরো শব্দ	`-w`
`-F`	হুবহু লেখা	`-F`
`-v`	উল্টো	`-v`
`-l`	শুধু ফাইলের নাম	`-l`
`-c`	ফাইলপ্রতি মেলা লাইনের সংখ্যা	`-c`
`-o`	শুধু মেলা অংশ	`-o`
`-A/-B/-C N`	context	একই
`-t TYPE`	শুধু এই ধরনের ফাইল (`-t py` , `-t js`)	`--include`
`-T TYPE`	এই ধরন বাদ	`--exclude`
`-g GLOB`	glob দিয়ে বাছাই; `!` দিয়ে বাদ (`-g '!*.min.js'`)	`--include/--exclude`
`--hidden`	লুকানো ফাইলেও খোঁজো	—
`--no-ignore`	`.gitignore` উপেক্ষা করো	—
`-u` , `-uu` , `-uuu`	ধাপে ধাপে ফিল্টার বন্ধ (ignore → hidden → binary)	—
`-P`	PCRE2 ইঞ্জিন (lookaround ইত্যাদির জন্য)	`-P`
`-r TEXT`	আউটপুটে মিলের জায়গায় TEXT বসিয়ে **দেখাও** (ফাইল বদলায় না)	—
`--files`	কোন কোন ফাইলে খুঁজবে তার তালিকা	—
`--type-list`	সব ফাইল-টাইপের তালিকা	—

```
$ rg -t py TODO                    # শুধু Python ফাইলে
$ rg -S 'Login'                    # বড় হাতের আছে, তাই case-sensitive
$ rg -g '!*.test.js' 'fetch\('     # test ফাইল বাদ
$ rg -uuu 'password'               # কিছুই বাদ না, grep -r এর মতো
```

⚠️ **সতর্কতা:** rg-এর `-r` মানে **replace**, grep-এর মতো recursive **না**! আর rg-এর regex উপভাষা ERE/PCRE-এর কাছাকাছি (`\d` চলে, `+ ? |` সরাসরি), কিন্তু back-reference আর lookaround শুধু `-P` দিলে চলে।

💡 **টিপস:** সার্ভারে কিছু ইনস্টল করার অনুমতি না থাকলে সবখানে আছে এমন `grep` -ই ভরসা। তাই grep ভালো করে শেখো, rg বোনাস।

২০.১৩ বাস্তব উদাহরণ

১. লগ থেকে IP ঠিকানা বের করা

নমুনা ওয়েব লগ:

```
$ cat > server.log << 'EOF'
2026-11-01 10:15:01 192.168.1.10 GET /index.html 200
2026-11-01 10:15:07 10.0.0.5 POST /login 401
2026-11-01 10:16:22 192.168.1.10 GET /about.html 200
2026-11-01 10:17:45 203.0.113.7 POST /login 401
2026-11-01 10:18:03 203.0.113.7 POST /login 401
2026-11-01 10:18:09 999.300.1.2 GET /test 404
EOF
```

সহজ IP প্যাটার্ন: চারটা ১–৩ অঙ্কের সংখ্যা, মাঝে ডট।

```
$ grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' server.log
192.168.1.10
10.0.0.5
192.168.1.10
203.0.113.7
203.0.113.7
999.300.1.2
```

প্যাটার্নটা ভেঙে দেখি:

```
  \b  ( [0-9]{1,3}  \. ) {3}   [0-9]{1,3}  \b
  |   |  |          |    |     |           |
  |   |  ১-৩ অঙ্ক   আসল  এই দলটা  শেষ অংশ     শব্দের
  |   |             ডট   ৩ বার    ১-৩ অঙ্ক    সীমানা
  |   গ্রুপ শুরু
  শব্দের সীমানা (যাতে 1192.168... এর মাঝখান থেকে না ধরে)
```

কিন্তু `999.300.1.2` বৈধ IP না (প্রতিটা অংশ সর্বোচ্চ 255)! **কড়া (strict) প্যাটার্ন:**

```
$ grep -oE '\b((25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])\.){3}(25[0-5]|2[0-4][0-
9]|1[0-9]{2}|[1-9]?[0-9])\b' server.log
192.168.1.10
10.0.0.5
192.168.1.10
203.0.113.7
203.0.113.7
```

একটা অংশ (octet) এভাবে ভাঙা হয়েছে:

বিকল্প	কী মেলে
`25[0-5]`	250–255
`2[0-4][0-9]`	200–249
`1[0-9]{2}`	100–199
`[1-9]?[0-9]`	0–99

কোন IP কতবার এসেছে (পাইপলাইন, `sort` / `uniq` নিয়ে বিস্তারিত অধ্যায় ২৩):

```
$ grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' server.log | sort | uniq -c | sort -rn
      2 203.0.113.7
      2 192.168.1.10
      1 999.300.1.2
      1 10.0.0.5
```

শুধু ব্যর্থ লগইন (401) করা IP:

```
$ grep ' 401$' server.log | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort -u
10.0.0.5
203.0.113.7
```

SSH-এ brute-force আক্রমণ খোঁজা (Ubuntu/Debian-এ `/var/log/auth.log`, Fedora/RHEL-এ `/var/log/secure` বা `journalctl`):

```
$ sudo grep 'Failed password' /var/log/auth.log | grep -oE 'from ([0-9]{1,3}\.){3}[0-9]
{1,3}' | awk '{print $2}' | sort | uniq -c | sort -rn | head
     57 203.0.113.50
     12 198.51.100.23
```

```
$ sudo journalctl -u ssh --since today | grep -i 'failed password'     # systemd সিস্টেমে
```

(Ubuntu-তে সার্ভিসের নাম `ssh`, Fedora/RHEL-এ `sshd`। লগ নিয়ে পুরো আলোচনা অধ্যায় ২৮-এ, নিরাপত্তা অধ্যায় ৪৫-এ।)

২. ফাইল থেকে email ঠিকানা বের করা

```
$ cat > contacts.txt << 'EOF'
Contact rahim@example.com for sales.
Support: support.team@mail.example.org
Invalid: karim@ and @nodomain.com
Fatema: fatema_99@gmail.com, backup fatema.b@yahoo.co.uk
EOF

$ grep -oE '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' contacts.txt
rahim@example.com
support.team@mail.example.org
```

```
fatema_99@gmail.com
fatema.b@yahoo.co.uk
```

```
  [A-Za-z0-9._%+-]+   @   [A-Za-z0-9.-]+   \.   [A-Za-z]{2,}
  |                   |   |                |    |
  username অংশ        @   ডোমেইন           ডট   TLD (com, org, uk ...)
  (১ বা বেশি অক্ষর)       (১ বা বেশি)           কমপক্ষে ২ অক্ষর
```

`karim@` (ডোমেইন নেই) আর `@nodomain.com` (username নেই) বাদ পড়েছে। খেয়াল করো — `[A-Za-z0-9._%+-]`-এ `-` শেষে রাখা হয়েছে, যাতে রেঞ্জ না বোঝায়।

> 📝 **মনে রাখো:** পুরোপুরি নিখুঁত email regex (RFC 5322) বিশাল আর অব্যবহারিক। বাস্তবে উপরের মতো "যথেষ্ট ভালো" প্যাটার্নই চলে।

কোন ডোমেইন কতবার:

```
$ grep -oE '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' contacts.txt | cut -d@ -f2
| sort | uniq -c
      1 example.com
      1 gmail.com
      1 mail.example.org
      1 yahoo.co.uk
```

৩. আরও কাজের রেসিপি

```
# সিস্টেমে কোন কোন ইউজারের লগইন শেল bash
$ grep 'bash$' /etc/passwd

# UID 1000 বা তার বেশি (সাধারণ ইউজার) — awk দিয়ে আরও সহজ, অধ্যায় ২২
$ grep -E '^[^:]+:[^:]*:[0-9]{4,}:' /etc/passwd

# ফাঁকা ও comment লাইন বাদে আসল কনফিগ
$ grep -Ev '^[[:space:]]*(#|$)' /etc/nginx/nginx.conf

# কোডবেসে কোনো ফাংশন কোথায় কল হয়েছে, লাইন নম্বরসহ
$ grep -rnw --include='*.php' 'sendMail' app/

# URL বের করা
$ grep -oE 'https?://[^[:space:]"<>]+' page.html

# তারিখ (YYYY-MM-DD) বের করা
$ grep -oE '\b[0-9]{4}-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])\b' server.log | sort -u
2026-11-01

# কোন প্রসেস চলছে — grep নিজেকে যেন না দেখায়
$ ps aux | grep '[n]ginx'
```

শেষের কৌশলটা মজার: `ps aux | grep nginx` চালালে `grep nginx` প্রসেসটাও তালিকায় আসে। কিন্তু `[n]ginx` প্যাটার্নটা "nginx" লেখা মেলায়, অথচ grep-এর নিজের কমান্ড লাইনে লেখা আছে `[n]ginx` —

যেটা "nginx" না! তাই grep নিজেকে বাদ দেয়। (অবশ্য সবচেয়ে ভালো উপায় `pgrep -a nginx`।)

৪. স্ক্রিপ্টে grep

```
#!/usr/bin/env bash
if grep -q "^${USER}:" /etc/passwd; then
    echo "ইউজার $USER লোকাল /etc/passwd-এ আছে"
else
    echo "নেই (হয়তো LDAP/SSSD ইউজার)"
fi

# কনফিগে লাইন না থাকলেই যোগ করো (idempotent)
grep -qxF 'vm.swappiness=10' /etc/sysctl.conf || echo 'vm.swappiness=10' | sudo tee -a
/etc/sysctl.conf
```

`-qxF` = চুপচাপ, পুরো লাইন, হুবহু — স্ক্রিপ্টের সবচেয়ে কাজের কম্বো। স্ক্রিপ্টিং বিস্তারিত অধ্যায় ৪১ থেকে।

২০.১৪ পারফরম্যান্স টিপস

টিপ	কেন
হুবহু লেখা খুঁজলে `-F`	regex ইঞ্জিন লাগে না
`LC_ALL=C grep ...`	UTF-8 মাল্টিবাইট হিসাব বাদ, কখনো অনেক দ্রুত (তবে বাংলা/ইউনিকোড প্যাটার্নে ব্যবহার কোরো না)
শুধু অস্তিত্ব দেখতে `-q` বা `-m 1`	প্রথম মিলেই থামে
`--exclude-dir`, `--include` দিয়ে জায়গা ছোট করো	কম ফাইল পড়া
বড় কোডবেসে `rg`	parallel, `.gitignore` মানে
back-reference ও নেস্টেড quantifier এড়াও	ধীর ইঞ্জিনে নিয়ে যায়
`cat file | grep x` না লিখে `grep x file`	অপ্রয়োজনীয় প্রসেস ("useless use of cat")

২০.১৫ সম্পর্কিত কীবোর্ড শর্টকাট

grep-এর নিজের কোনো শর্টকাট নেই, কিন্তু grep নিয়ে কাজ করার সময় এগুলো লাগবেই:

শর্টকাট	কোথায়	কাজ
`Ctrl + C`	টার্মিনাল	চলমান grep থামাও (যেমন বিশাল ফোল্ডারে `-r`)

শর্টকাট	কোথায়	কাজ
`Ctrl + D`	টার্মিনাল	ফাইল না দিলে grep stdin-এ অপেক্ষা করে — ইনপুট শেষ জানাতে
`Ctrl + R`	bash	আগের grep কমান্ড history থেকে খোঁজো (অধ্যায় ৬)
`Ctrl + Z`	টার্মিনাল	লম্বা grep সাময়িক থামাও (`fg` দিয়ে ফেরত)
`Alt + .`	bash	আগের কমান্ডের শেষ argument (যেমন ফাইলের নাম) বসাও
`/pattern`	`less` , `man` , `vim`	সামনের দিকে regex খোঁজো
`?pattern`	`less` , `man` , `vim`	পেছনের দিকে খোঁজো
`n` / `N`	`less` , `vim`	পরের / আগের মিল
`&pattern`	`less`	শুধু মেলা লাইনগুলো দেখাও (less-এর ভেতরের grep!)
`-i` তারপর Enter	`less`	খোঁজায় case উপেক্ষা চালু/বন্ধ
`Esc u`	`less`	হাইলাইট বন্ধ/চালু
`Ctrl + Shift + F`	টার্মিনাল এমুলেটর (GNOME Terminal ইত্যাদি)	স্ক্রিনের লেখায় খোঁজো

> 💡 **টিপস:** grep আটকে আছে মনে হচ্ছে, কিছুই ছাপছে না? সম্ভবত ফাইলের নাম দিতে ভুলে গেছ — grep তখন কীবোর্ড থেকে ইনপুটের অপেক্ষায়। `Ctrl + C` চাপো।

হাতে-কলমে অনুশীলন

ল্যাব ১: মৌলিক খোঁজা

```
$ cd ~/grep-lab
```

1. `notes.txt` -এ case উপেক্ষা করে `error` আছে এমন লাইন লাইন নম্বরসহ দেখাও।

   ```
   $ grep -ni error notes.txt
   8:error: disk full
   9:Error: permission denied
   ```

2. কতগুলো লাইনে `is` আলাদা শব্দ হিসেবে আছে গোনো → `grep -cw is notes.txt` → `5`
3. ফাঁকা লাইন ছাড়া সব লাইন দেখাও → `grep -v '^$' notes.txt`
4. `kernel` লাইনের আগে-পরে ১ লাইনসহ দেখাও → `grep -C1 kernel notes.txt`

ল্যাব ২: regex অনুশীলন

```
$ cat > words.txt << 'EOF'
apple
banana
cherry
date
fig
grape
kiwi
level
radar
EOF
```

1. স্বরবর্ণ দিয়ে শুরু শব্দ: `grep -E '^[aeiou]' words.txt` → `apple`
2. ঠিক ৪ অক্ষরের শব্দ: `grep -Ex '.{4}' words.txt` → `date`, `kiwi`
3. একই অক্ষর পরপর দুবার: `grep -E '(.)\1' words.txt` → `apple`, `cherry`
4. `a` দিয়ে শেষ: `grep 'a$' words.txt` → `banana`
5. ৫ অক্ষরের palindrome (উল্টো পড়লেও এক):

   ```
   $ grep -Ex '(.)(.).\2\1' words.txt
   level
   radar
   ```

ল্যাব ৩: প্রজেক্টে খোঁজা

1. `project` ফোল্ডারে `TODO` আছে এমন ফাইলের নাম → `grep -rl TODO project`
2. শুধু `.py` ফাইলে, লাইন নম্বরসহ → `grep -rn --include='*.py' TODO project`
3. `notes` ফোল্ডার বাদ দিয়ে সব ফাইলে `milk` → `grep -r --exclude-dir=notes milk project` (কিছুই আসবে না)
4. rg ইনস্টল থাকলে একই কাজ: `rg -t py TODO project`

ল্যাব ৪: ডেটা বের করা

1. `server.log` থেকে সব IP, ইউনিক → `grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' server.log | sort -u`
2. `404` স্ট্যাটাসের লাইন → `grep ' 404$' server.log`
3. `contacts.txt` থেকে শুধু gmail ঠিকানা → `grep -oE '[A-Za-z0-9._%+-]+@gmail\.com' contacts.txt` → `fatema_99@gmail.com`
4. PCRE দিয়ে `user=`-এর পরের মান: `echo "id=7 user=karim" | grep -oP 'user=\K\w+'` → `karim`

ল্যাব ৫ (চ্যালেঞ্জ): নিজের সিস্টেমে

1. `/etc/passwd`-এ কতজন ইউজারের শেল `nologin` → `grep -c 'nologin$' /etc/passwd`
2. `/etc` ফোল্ডারে (সাবফোল্ডারসহ) কোন কোন ফাইলে `PermitRootLogin` আছে → `sudo grep -rl PermitRootLogin /etc 2>/dev/null`
3. আজকের লগে `error` শব্দ কতবার (Ubuntu) → `sudo grep -oi error /var/log/syslog | wc -l` (সিস্টেমে `/var/log/syslog` না থাকলে `journalctl -b | grep -ci error`)

সারসংক্ষেপ

grep দ্রুত রেফারেন্স

কমান্ড	কাজ
`grep pat file`	pat আছে এমন লাইন
`grep -i pat file`	case উপেক্ষা
`grep -v pat file`	pat নেই এমন লাইন
`grep -n pat file`	লাইন নম্বরসহ
`grep -c pat file`	মেলা লাইনের সংখ্যা
`grep -w pat file`	পুরো শব্দ
`grep -x pat file`	পুরো লাইন
`grep -o pat file`	শুধু মেলা অংশ
`grep -l / -L pat files`	মিল আছে / নেই এমন ফাইলের নাম
`grep -rn pat dir`	ফোল্ডারে recursive, লাইন নম্বরসহ
`grep -R pat dir`	recursive, সব symlink অনুসরণ
`grep -A/-B/-C N pat file`	পরের/আগের/দুই দিকের N লাইন
`grep -E 'colou?r'`	ERE — `+ ? ( ) { }` ও অথবা-চিহ্ন সরাসরি
`grep -F 'a.b'`	হুবহু লেখা
`grep -P '\d+'`	PCRE
`grep -e p1 -e p2` / `-f file`	একাধিক প্যাটার্ন
`grep -q pat file`	চুপচাপ, শুধু exit status
`grep -m N`	N মিলের পর থামো
`--include / --exclude / --exclude-dir`	ফাইল/ফোল্ডার বাছাই
`--color=auto`	মিল রঙিন

কমান্ড	কাজ
`rg pat`	ripgrep — recursive, .gitignore মানে

Regex দ্রুত রেফারেন্স

চিহ্ন	মানে
`.`	যেকোনো একটা অক্ষর
`*` / `+` / `?`	০+ / ১+ / ০ বা ১ বার
`{n}` `{n,}` `{n,m}`	নির্দিষ্ট সংখ্যক বার
`^` / `$`	লাইনের শুরু / শেষ
`[abc]` / `[^abc]` / `[a-z]`	দল / বাদে / রেঞ্জ
`( )`	গ্রুপ
`\b` `\<` `\>`	শব্দের সীমানা
`\w` `\s`	word char / whitespace (GNU)
`\d`	অঙ্ক (শুধু `-P`)
`\1`	back-reference
`*?` `+?`	lazy (শুধু `-P`)
`(?=..)` `(?<=..)` `\K`	lookahead / lookbehind / keep-out (শুধু `-P`)
`[[:digit:]]` `[[:alpha:]]` `[[:space:]]` …	POSIX class

অথবা (alternation): ERE ও PCRE-তে `a|b`, BRE-তে `a\|b`।

নিজেকে যাচাই করো

1. `grep` নামটা কোথা থেকে এসেছে?
2. `grep -c error app.log` আর `grep -o error app.log | wc -l` — এ দুটোর ফল কখন আলাদা হবে?
3. `grep 'is' notes.txt` লিখলে `disk` শব্দওয়ালা লাইন কেন আসে? কীভাবে ঠেকাবে?
4. `grep -r` আর `grep -R` -এর পার্থক্য কী?
5. BRE-তে "এক বা একাধিক" কীভাবে লেখো, আর ERE-তে কীভাবে?
6. `grep '\d+' file` কেন আশানুরূপ কাজ করে না? দুইটা সঠিক বিকল্প লেখো।
7. `echo '"a" and "b"' | grep -o '".*"'` কী ছাপবে, এবং শুধু `"a"` আর `"b"` আলাদা পেতে কী করবে?
8. grep-এর exit status `0`, `1`, `2` মানে কী? স্ক্রিপ্টে কোন অপশন দিয়ে শুধু status ব্যবহার করবে?
9. `192.168.1.1` হুবহু খুঁজতে `grep 192.168.1.1` লেখা কেন ঝুঁকিপূর্ণ?

10. ripgrep ডিফল্টভাবে কোন কোন ফাইল বাদ দেয়, আর rg-এর `-r` অপশনের মানে কী?

<details>
<summary>▼ উত্তর দেখো</summary>

1. পুরোনো লাইন-এডিটর `ed`-এর কমান্ড `g/re/p` থেকে — **g**lobal ভাবে একটা **r**egular **e**xpression খুঁজে মেলা লাইন **p**rint করো।
2. যখন একই লাইনে `error` একাধিকবার থাকে। `-c` মেলা **লাইন** গোনে, আর `-o ... | wc -l` প্রতিটা **মিল** আলাদা গোনে।
3. কারণ grep শব্দের ভেতরের অংশও মেলায় — d**is**k-এ `is` আছে। ঠেকাতে `grep -w 'is'` বা `grep '\bis\b'`।
4. দুটোই recursive। `-r` ফোল্ডারের ভেতরে পাওয়া symlink অনুসরণ করে না (শুধু কমান্ড লাইনে দেওয়াগুলো করে); `-R` সব symlink অনুসরণ করে।
5. BRE-তে `\+` (GNU এক্সটেনশন) বা পোর্টেবলভাবে `aa*`; ERE-তে সরাসরি `+`।
6. `\d` POSIX BRE/ERE-র অংশ না, আর BRE-তে `+` সাধারণ অক্ষর। বিকল্প: `grep -E '[0-9]+' file` অথবা `grep -P '\d+' file` (বা `grep -E '[[:digit:]]+'`)।
7. greedy মিলের কারণে পুরো `"a" and "b"` ছাপবে। আলাদা পেতে `grep -o '"[^"]*"'` অথবা `grep -oP '".*?"'`।
8. `0` = মিল পাওয়া গেছে, `1` = মিল নেই, `2` = error। `-q` দিয়ে।
9. কারণ `.` মানে যেকোনো অক্ষর — `192x168y1z1` বা `1921681171`-এর মতো লেখাও মিলে যেতে পারে। নিরাপদ: `grep -F '192.168.1.1'` বা ডট escape করে `'192\.168\.1\.1'` (আর পুরো IP হিসেবে চাইলে `-w`)।
10. `.gitignore` / `.ignore`-এ থাকা ফাইল, লুকানো (`.` দিয়ে শুরু) ফাইল ও ফোল্ডার, আর binary ফাইল। rg-এর `-r` মানে **replace** — আউটপুটে মিলের জায়গায় অন্য লেখা বসিয়ে দেখানো (ফাইল বদলায় না); recursive না, কারণ rg এমনিতেই recursive।

</details>

অধ্যায় ২১: sed — স্ট্রিম এডিটর

এই অধ্যায়ে যা শিখবে

- `sed` কী, কেন একে "stream editor" বলে, আর কোথায় এটা `nano` / `vim`-এর চেয়ে বেশি কাজের
- `sed` ভেতরে কীভাবে কাজ করে: **pattern space**, **hold space** আর প্রতিটি লাইনের **cycle** (ডায়াগ্রামসহ)
- সবচেয়ে বেশি ব্যবহৃত কমান্ড `s///` (substitute) আর তার flag: `g`, `i` / `I`, `p`, নম্বর (`2`, `3g`), `w`
- delimiter বদলানো (`s|...|...|`, `s#...#...#`), যাতে path নিয়ে কাজ সহজ হয়
- গুরুত্বপূর্ণ অপশন: `-n`, `-e`, `-f`, `-E` / `-r`, `-i`, `-i.bak`, `-s`, `-z`, `--debug`
- address: লাইন নম্বর, range, regex, `$`, `!` (উল্টো), `first~step`, `addr,+N`
- কমান্ড: `d`, `p`, `a`, `i`, `c`, `y`, `q`, `Q`, `=`, `n`, `N`, `r`, `w`
- `&` আর back-reference `\1` … `\9`, সাথে GNU-র `\U`, `\L` কেস-বদল
- hold space পরিচিতি: `h`, `H`, `g`, `G`, `x`
- একাধিক কমান্ড একসাথে চালানো, script ফাইল লেখা
- বাস্তব রেসিপি: কনফিগ ফাইল এডিট, ফাঁকা লাইন মোছা, কমেন্ট বাদ দেওয়া, লাইন জোড়া লাগানো, আর অনেক কিছু
- GNU sed বনাম BSD/macOS sed-এর পার্থক্য আর সাধারণ ভুলগুলো

২১.১ sed কী এবং কেন?

`sed` শব্দটা এসেছে **s**tream **ed**itor থেকে। ১৯৭৩-৭৪ সালে Bell Labs-এ Lee McMahon এটা তৈরি করেন, পুরনো লাইন-এডিটর `ed`-এর কমান্ডগুলোকে ভিত্তি করে। এজন্যই `ed`, `vi` / `vim` আর `sed`-এর substitute কমান্ড দেখতে প্রায় একই রকম: `s/old/new/`।

Stream মানে "স্রোত" — ডেটা যা এক দিক থেকে ঢুকে অন্য দিক দিয়ে বেরিয়ে যায়। `sed` একটা ফাইল খুলে তোমাকে এডিট করতে দেয় না (যেমন `nano` বা `vim` দেয়)। বরং এটা ইনপুট থেকে **একটা একটা লাইন** পড়ে, তোমার দেওয়া নির্দেশ অনুযায়ী লাইনটা বদলায়, তারপর output-এ ছেড়ে দেয়। কোনো মানুষ বসে বসে কী-বোর্ড টিপছে না — সব স্বয়ংক্রিয়।

> 💡 **উপমা:** একটা কারখানার conveyor belt কল্পনা করো। বেল্টের ওপর দিয়ে একটা একটা বাক্স (লাইন) আসছে। বেল্টের মাঝখানে একজন কর্মী (sed) দাঁড়িয়ে আছে, হাতে একটা নির্দেশের কাগজ (sed script)। সে প্রতিটা বাক্স তুলে নেয়, কাগজ দেখে লেবেল বদলায়, কোনোটা ফেলে দেয়, কোনোটার পাশে নতুন বাক্স রাখে, তারপর বাক্সটা আবার বেল্টে ছেড়ে দেয়। সে কখনো পেছনে ফিরে আগের বাক্স ধরতে পারে না (যদি না আলাদা করে একটা তাকে — hold space-এ — রেখে দেয়)।

কেন sed শিখবে?

পরিস্থিতি	sed ছাড়া	sed দিয়ে
৫০টা কনফিগ ফাইলে `old.server.com` বদলে `new.server.com` করা	৫০ বার ফাইল খুলে এডিট	এক লাইনের কমান্ড
2 GB লগ ফাইল থেকে ১০০০–১০২০ নম্বর লাইন দেখা	এডিটরে খুললে মেমরি শেষ	`sed -n '1000,1020p' big.log` — মুহূর্তে
স্ক্রিপ্টের ভেতর থেকে কনফিগ বদলানো (automation)	মানুষ লাগবে	পুরোপুরি স্বয়ংক্রিয়
pipe-এর মাঝখানে টেক্সট রূপান্তর	অসম্ভব/কঠিন	`cmd

`sed` প্রায় সব লিনাক্স সিস্টেমে আগে থেকেই থাকে (এমনকি ছোট container image বা rescue shell-এও, সাধারণত BusyBox-এর অংশ হিসেবে)। তাই সার্ভারে কাজ করতে গেলে এটা জানা প্রায় বাধ্যতামূলক।

> 📝 **মনে রাখো:** `grep` লাইন **খোঁজে**, `sed` লাইন **বদলায়**, আর `awk` লাইনকে কলামে ভেঙে **হিসাব করে**। `grep` আর regex আমরা অধ্যায় ২০-এ শিখেছি, `awk` আসছে অধ্যায় ২২-এ।

কোন sed তোমার কাছে আছে?

লিনাক্সে প্রায় সবসময় **GNU sed** থাকে। macOS আর FreeBSD-তে থাকে **BSD sed**, যেটার কিছু আচরণ আলাদা (বিশেষ করে `-i`)। দেখে নাও:

```
$ sed --version
sed (GNU sed) 4.9
Packaged by Debian
Copyright (C) 2022 Free Software Foundation, Inc.
...
```

Ubuntu 24.04 ও Debian 12-এ GNU sed 4.9, RHEL 9-এ 4.8। এই অধ্যায়ের সব উদাহরণ GNU sed ধরে লেখা; যেখানে কিছু শুধু GNU-তে চলে, সেখানে **(GNU)** লিখে দেওয়া আছে।

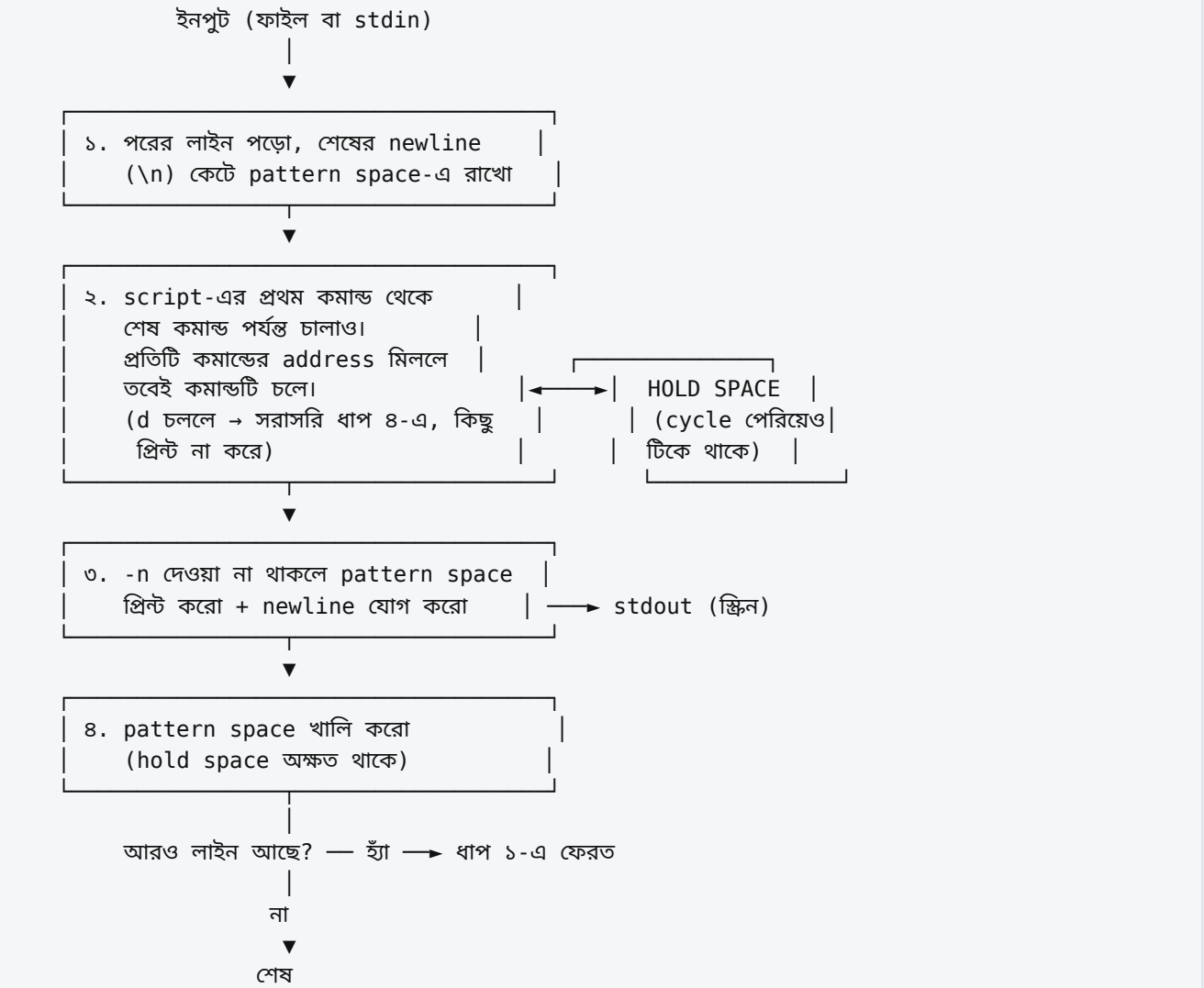
২১.২ ভেতরে কীভাবে কাজ করে — pattern space ও cycle

`sed` বুঝতে হলে দুটো "বাক্স" বা বাফার চিনতে হবে:

1. **Pattern space** — কাজের টেবিল। বর্তমান লাইনটা এখানে রাখা হয়, আর সব কমান্ড (যেমন `s`, `d`, `p`) এর ওপরই কাজ করে।
2. **Hold space** — পাশের তাক। শুরুতে খালি। এখানে তুমি ইচ্ছা করে কিছু জমিয়ে রাখতে পারো, এবং এটা এক লাইন থেকে আরেক লাইনে **টিকে থাকে**। (বিস্তারিত ২১.১১-এ)

এক cycle-এ কী ঘটে

প্রতিটি ইনপুট লাইনের জন্য sed নিচের চক্র (cycle) চালায়:



এই ডায়াগ্রাম থেকে কয়েকটা গুরুত্বপূর্ণ সত্য বেরিয়ে আসে:

- **sed ডিফল্টভাবে সব লাইন প্রিন্ট করে** (ধাপ ৩)। তাই `sed 's/a/b/' file` চালালে যে লাইনে কিছু বদলায়নি সেগুলোও দেখা যায়।
- `-n` **দিলে স্বয়ংক্রিয় প্রিন্ট বন্ধ হয়।** তখন শুধু তুমি `p` দিয়ে যা প্রিন্ট করতে বলবে তাই আসবে।
- `d` **মানে "এই লাইন ফেলে দাও এবং পরের cycle শুরু করো"** — `d`-এর পরের কমান্ডগুলো ওই লাইনের জন্য চলেই না।
- **মূল ফাইল বদলায় না** (যদি না `-i` দাও)। output যায় stdout-এ।
- **sed একবারে পুরো ফাইল মেমরিতে তোলে না**, একটা লাইন রাখে। তাই ১০০ GB ফাইলেও অল্প মেমরিতে চলে।

একটা উদাহরণ দিয়ে দেখি। ফাইল `fruits.txt`:

```
$ cat fruits.txt
apple
banana
cherry
```

```
$ sed 's/a/A/' fruits.txt
Apple
bAnana
cherry
```

ভেতরে যা হলো:

```
cycle 1: pattern space = "apple"   → s/a/A/ → "Apple"   → প্রিন্ট
cycle 2: pattern space = "banana"  → s/a/A/ → "bAnana"  → প্রিন্ট (শুধু প্রথম a!)
cycle 3: pattern space = "cherry"  → s/a/A/ → মিলল না   → "cherry" প্রিন্ট
```

GNU `--debug` দিয়ে ভেতরটা দেখো (GNU 4.6+)

```
$ printf 'apple\nbanana\n' | sed --debug 's/a/A/'
SED PROGRAM:
  s/a/A/
INPUT:   'STDIN' line 1
PATTERN: apple
COMMAND: s/a/A/
PATTERN: Apple
END-OF-CYCLE:
Apple
INPUT:   'STDIN' line 2
PATTERN: banana
COMMAND: s/a/A/
PATTERN: bAnana
END-OF-CYCLE:
bAnana
```

> 💡 **টিপস:** জটিল sed কমান্ড কেন কাজ করছে না বুঝতে না পারলে `--debug` সবচেয়ে ভালো বন্ধু। এটা প্রতিটি ধাপে pattern space-এ কী আছে দেখিয়ে দেয়।

২১.৩ sed-এর মৌলিক syntax

```
sed [অপশন] 'script' [ফাইল...]
sed [অপশন] -e 'script1' -e 'script2' [ফাইল...]
sed [অপশন] -f script.sed [ফাইল...]
```

script-এর প্রতিটি নির্দেশের গঠন:

```
[address[,address]][!]command[arguments]
   |                |    |
   |                |    └── কী করবে (s, d, p, a ...)
   |                └─────── address উল্টো করো (মিলেনি এমন লাইনে চালাও)
   └──────────────────────── কোন লাইনে করবে (না দিলে = সব লাইনে)
```

উদাহরণ:

কমান্ড	মানে
`sed 'd' f`	সব লাইন মুছো (address নেই → সব লাইন)
`sed '3d' f`	৩ নম্বর লাইন মুছো
`sed '/error/d' f`	যে লাইনে `error` আছে তা মুছো
`sed '/error/!d' f`	যে লাইনে `error` **নেই** তা মুছো (অর্থাৎ `grep error`-এর মতো)
`sed '2,5s/x/y/' f`	২ থেকে ৫ নম্বর লাইনে x→y

ইনপুট তিনভাবে দিতে পারো:

```
$ sed 's/hi/bye/' file.txt              # ফাইল থেকে
$ echo "hi there" | sed 's/hi/bye/'     # pipe থেকে
bye there
$ sed 's/hi/bye/' < file.txt            # redirection থেকে
```

> ⚠️ **সতর্কতা:** script সবসময় **single quote** (`'...'`) দিয়ে ঘিরে দাও। double quote দিলে shell আগে `$`, `` ` ``, `\`, `!` ইত্যাদি নিজে ব্যাখ্যা করে ফেলবে, ফলে `$` (শেষ লাইন) বা `\1` গোলমাল হবে। শুধু যখন shell variable ঢোকাতেই হবে তখন double quote ব্যবহার করো (২১.১৪ দেখো)।

২১.৪ গুরুত্বপূর্ণ অপশন

অপশন	লম্বা রূপ	কাজ
`-n`	`--quiet`, `--silent`	স্বয়ংক্রিয় প্রিন্ট বন্ধ; শুধু `p` দিয়ে যা চাইবে তা আসবে
`-e script`	`--expression=script`	একটা script যোগ করো; একাধিক `-e` দেওয়া যায়
`-f file`	`--file=file`	script একটা ফাইল থেকে পড়ো
`-E` / `-r`	`--regexp-extended`	Extended Regex (ERE) ব্যবহার করো; `-E` POSIX মানসম্মত, `-r` পুরনো GNU রূপ
`-i[SUFFIX]`	`--in-place[=SUFFIX]`	ফাইলকে সরাসরি বদলাও; SUFFIX দিলে ব্যাকআপ রাখো
`-s`	`--separate`	একাধিক ফাইলকে আলাদা আলাদা ধরো (`$`, লাইন নম্বর প্রতি ফাইলে নতুন করে)

অপশন	লম্বা রূপ	কাজ
`-z`	`--null-data`	লাইনের বিভাজক newline না ধরে NUL (`\0`) ধরো (GNU)
`-u`	`--unbuffered`	output বাফার না করে সাথে সাথে পাঠাও (`tail -f`-এর সাথে কাজে লাগে)
`-l N`	`--line-length=N`	`l` কমান্ডের লাইন-মোড়ানোর দৈর্ঘ্য (GNU)
	`--follow-symlinks`	`-i`-এর সময় symlink অনুসরণ করে আসল ফাইল বদলাও (GNU)
	`--posix`	GNU extension বন্ধ করে খাঁটি POSIX আচরণ
	`--debug`	প্রতিটি ধাপ দেখাও (GNU 4.6+)

`-n` আর `p` — জুটি

`p` মানে pattern space প্রিন্ট করো। `-n` ছাড়া `p` দিলে মিলে যাওয়া লাইন **দুবার** আসে:

```
$ sed '2p' fruits.txt
apple
banana
banana
cherry

$ sed -n '2p' fruits.txt
banana
```

প্রথম ক্ষেত্রে `banana` একবার `p` থেকে, একবার স্বয়ংক্রিয় প্রিন্ট থেকে। তাই নিয়ম: **নির্দিষ্ট লাইন দেখতে চাইলে সবসময় `-n` + `p`।**

`-e` — একাধিক script

```
$ sed -e 's/apple/APPLE/' -e 's/cherry/CHERRY/' fruits.txt
APPLE
banana
CHERRY
```

`-E` — Extended regex

Basic regex (BRE)-এ group-এর জন্য `\( \)`, repetition-এর জন্য `\{2,3\}` লিখতে হয়। `-E` দিলে backslash ছাড়াই লেখা যায়:

```
$ echo "2024-06-15" | sed 's/\([0-9]\{4\}\)-\([0-9]\{2\}\)-\([0-9]\{2\}\)/\3\/\2\/\1/'
15/06/2024

$ echo "2024-06-15" | sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3\/\2\/\1/'
15/06/2024
```

দুটোর ফল একই, কিন্তু দ্বিতীয়টা পড়া অনেক সহজ। BRE বনাম ERE-র পুরো পার্থক্য অধ্যায় ২০-এ আছে; সংক্ষেপে:

জিনিস	BRE (ডিফল্ট)	ERE (`-E`)
group	`\( \)`	`( )`
একাধিকবার	`\{n,m\}`	`{n,m}`
এক বা বেশি	`\+` (GNU)	`+`
শূন্য বা এক	`\?` (GNU)	`?`
বিকল্প	`|` (GNU)	`

`-i` — ফাইল সরাসরি বদলানো (in-place)

ডিফল্টে sed শুধু স্ক্রিনে দেখায়। ফাইলেই পরিবর্তন রাখতে চাইলে `-i`:

```
$ sed -i 's/banana/mango/' fruits.txt
$ cat fruits.txt
apple
mango
cherry
```

ব্যাকআপ রেখে বদলাতে চাইলে `-i`-এর **ঠিক পরেই, ফাঁকা না রেখে** suffix দাও:

```
$ sed -i.bak 's/mango/banana/' fruits.txt
$ ls fruits.txt*
fruits.txt  fruits.txt.bak
```

এখন `fruits.txt.bak`-এ আগের (mango-ওয়ালা) সংস্করণ আছে।

`-i` ভেতরে কীভাবে কাজ করে?

অনেকে ভাবে `-i` ফাইলের ভেতরে লিখে দেয়। আসলে তা নয়:

```
১. sed একই ডিরেক্টরিতে একটা temporary ফাইল বানায় (যেমন ./sedAb12Cd)
২. মূল ফাইল পড়ে, প্রক্রিয়া করে, output temp ফাইলে লেখে
৩. suffix দেওয়া থাকলে মূল ফাইলকে rename করে  fruits.txt → fruits.txt.bak
৪. temp ফাইলকে rename করে মূল নামে আনে  sedAb12Cd → fruits.txt
```

এর কয়েকটা গুরুত্বপূর্ণ ফল আছে (inode-এর ধারণা অধ্যায় ১১-এ):

- **ফাইলের inode নম্বর বদলে যায়।** নতুন ফাইল মানে নতুন inode।
- **হার্ড লিংক ভেঙে যায়।** অন্য নামের হার্ড লিংক পুরনো কনটেন্টই দেখাবে।
- **symlink-এর ওপর `-i` চালালে symlink-টা সাধারণ ফাইলে পরিণত হয়**, আসল টার্গেট অপরিবর্তিত থাকে। আসল ফাইল বদলাতে চাইলে `--follow-symlinks` দাও।
- ডিরেক্টরিতে **লেখার পারমিশন** লাগে (temp ফাইল বানানোর জন্য), শুধু ফাইলে লেখার পারমিশন যথেষ্ট নয়।
- GNU sed মালিকানা ও পারমিশন যতটা পারে আগের মতো রাখার চেষ্টা করে।

```
$ ls -i fruits.txt
1835012 fruits.txt
$ sed -i 's/a/a/' fruits.txt
$ ls -i fruits.txt
1835019 fruits.txt          # inode বদলে গেছে!
```

⚠️ **সতর্কতা — `-i` আর অন্য অপশন একসাথে:** `sed -in 's/x/y/' file` লিখলে GNU sed `n`-কে **suffix** হিসেবে নেয়! ফলে `filen` নামে ব্যাকআপ তৈরি হয় আর `-n` কাজই করে না। তাই `-i`-কে সবসময় আলাদা রাখো: `sed -n -i ...` অথবা `sed -i -n ...`।

⚠️ **সতর্কতা — macOS/BSD:** BSD sed-এ `-i`-এর পর suffix **বাধ্যতামূলক আলাদা argument**। ব্যাকআপ না চাইলে খালি string দিতে হয়: `sed -i '' 's/x/y/' file`। GNU-তে `sed -i '' ...` লিখলে `''`-কে script ধরে ভুল হবে। দুই সিস্টেমেই চলবে এমন নিরাপদ রূপ: `sed -i.bak 's/x/y/' file && rm file.bak`।

💡 **টিপস:** `-i` চালানোর আগে **সবসময় প্রথমে `-i` ছাড়া চালিয়ে output দেখে নাও।** সন্তুষ্ট হলে তারপর `-i` যোগ করো। আরও নিরাপদ: `sed 's/x/y/' file | diff file -` দিয়ে কী কী বদলাবে দেখে নাও।

`-s` — একাধিক ফাইল আলাদাভাবে

একাধিক ফাইল দিলে sed ডিফল্টে সবগুলোকে **একটা লম্বা stream** হিসেবে দেখে:

```
$ printf 'a1\na2\n' > a.txt; printf 'b1\nb2\n' > b.txt
$ sed -n '1p' a.txt b.txt
a1
$ sed -n '$p' a.txt b.txt
b2
$ sed -s -n '1p' a.txt b.txt
a1
b1
```

`-s` দিলে প্রতিটি ফাইলের প্রথম/শেষ লাইন আলাদাভাবে ধরা হয়। (`-i` দিলে স্বয়ংক্রিয়ভাবে `-s` আচরণ চালু হয়।)

`-z` — পুরো ফাইলকে এক লাইন হিসেবে (GNU)

`-z` দিলে sed লাইন ভাঙে NUL (`\0`) অক্ষরে। সাধারণ টেক্সট ফাইলে NUL থাকে না, তাই পুরো ফাইল একবারে pattern space-এ আসে — এবং `\n` দিয়ে newline ধরা যায়:

```
$ printf 'one\ntwo\nthree\n' | sed -z 's/\n/, /g'
one, two, three,
```

`find -print0` আর `xargs -0`-এর সাথেও এটা কাজে লাগে।

২১.৫ s/// — substitute কমান্ড (sed-এর রাজা)

sed-এর ৯০% কাজ এই একটা কমান্ড দিয়েই হয়।

```
s/REGEX/REPLACEMENT/FLAGS
|  |       |           |
|  |       |           └─ g, p, i, নম্বর, w file ...
|  |       └──────────── কী দিয়ে বদলাবে (& , \1 ব্যবহার করা যায়)
|  └──────────────────── কী খুঁজবে (regular expression)
└─────────────────────── substitute
```

```
$ echo "I love cats" | sed 's/cats/dogs/'
I love dogs
```

Flags

Flag	কাজ	উদাহরণ
(কিছু না)	প্রতি লাইনে **প্রথম** মিলটা বদলাও	`s/a/A/`
`g`	লাইনের **সব** মিল বদলাও (global)	`s/a/A/g`
`N` (১–৫১২)	শুধু **N-তম** মিল বদলাও	`s/a/A/2`
`Ng`	N-তম এবং তার পরের সব মিল (GNU)	`s/a/A/2g`
`p`	বদল হলে pattern space প্রিন্ট করো	`sed -n 's/a/A/p'`
`i` বা `I`	বড়/ছোট হাতের অক্ষর উপেক্ষা (GNU)	`s/hello/hi/I`
`w file`	বদল হওয়া লাইন ফাইলে লেখো	`s/a/A/w out.txt`
`m` বা `M`	multiline mode — `^` `$` প্রতিটি লাইনের শুরু/শেষ ধরে (GNU)	`s/^/> /Mg`
`e`	ফলাফলকে shell কমান্ড হিসেবে চালাও, output দিয়ে বদলাও (GNU, সাবধান!)	

উদাহরণগুলো দেখি:

```
$ echo "banana" | sed 's/a/A/'
bAnana
$ echo "banana" | sed 's/a/A/g'
bAnAnA
$ echo "banana" | sed 's/a/A/2'
banAna
$ echo "banana" | sed 's/a/A/2g'
banAnA
```

```
$ echo "Hello HELLO hello" | sed 's/hello/hi/Ig'
hi hi hi
```

`p` flag আর `-n` — "grep + বদল"

শুধু যে লাইনগুলোতে বদল হয়েছে সেগুলো দেখাতে:

```
$ cat config.txt
port=80
host=localhost
port_ssl=443
$ sed -n 's/port/PORT/p' config.txt
PORT=80
PORT_ssl=443
```

`w` flag

```
$ sed 's/port/PORT/w changed.txt' config.txt
PORT=80
host=localhost
PORT_ssl=443
$ cat changed.txt
PORT=80
PORT_ssl=443
```

> 📝 **মনে রাখো:** flag-এর ক্রম গুরুত্বপূর্ণ নয় (`gI` আর `Ig` একই), তবে `w file` সবসময় **শেষে** থাকতে হবে, কারণ এর পরের সবকিছুকে ফাইলের নাম ধরা হয়।

Delimiter বদলানো

`/` আসলে বাধ্যতামূলক নয়। `s`-এর ঠিক পরের অক্ষরটাই delimiter হয়ে যায়। এটা দারুণ কাজে আসে যখন খোঁজার জিনিসেই `/` থাকে, যেমন ফাইলের path।

`/` দিয়ে লিখলে প্রতিটি `/` escape করতে হয় — এটাকে মজা করে "leaning toothpick syndrome" বলে:

```
$ echo "/usr/local/bin" | sed 's/\/usr\/local\/bin/\/opt\/bin/'
/opt/bin
```

অন্য delimiter দিলে অনেক পরিষ্কার:

```
$ echo "/usr/local/bin" | sed 's|/usr/local/bin|/opt/bin|'
/opt/bin
$ echo "/usr/local/bin" | sed 's#/usr/local/bin#/opt/bin#'
/opt/bin
$ echo "/usr/local/bin" | sed 's:/usr/local:/opt:'
/opt/bin
```

নিয়ম:

- delimiter যেকোনো একক অক্ষর হতে পারে (newline আর backslash বাদে)। `|`, `#`, `:`, `,`, `@` সবচেয়ে জনপ্রিয়।
- যে অক্ষর delimiter বানাবে, সেটা pattern/replacement-এর ভেতরে লাগলে `\` দিয়ে escape করতে হবে।
- regex **address**-এও delimiter বদলানো যায়, তবে প্রথমটার আগে `\` দিতে হয়: `\|/usr/bin|d` (২১.৬ দেখো)।

Replacement-এর বিশেষ অক্ষর

replacement অংশ regex নয়, তবে এখানে কয়েকটা অক্ষরের বিশেষ মানে আছে:

অক্ষর	মানে
`&`	পুরো মিলে যাওয়া অংশ
`\1` … `\9`	১ম…৯ম group (`\( \)` বা `-E`-তে `( )`)
`\n`	newline (GNU)
`\t`	tab (GNU)
`\&`	আক্ষরিক `&`
`\\`	আক্ষরিক `\`
`\U`	এর পর থেকে সব বড় হাতের (GNU)
`\L`	এর পর থেকে সব ছোট হাতের (GNU)
`\u`	পরের একটা অক্ষর বড় হাতের (GNU)
`\l`	পরের একটা অক্ষর ছোট হাতের (GNU)
`\E`	`\U` / `\L` বন্ধ করো (GNU)

এগুলোর বিস্তারিত পরের সেকশনে।

২১.৬ `&` ও back-reference `\1`

`&` — পুরো মিল

`&` মানে "যা খুঁজে পেয়েছি ঠিক সেটা"। মিলটাকে মুছে না ফেলে তার চারপাশে কিছু যোগ করতে চাইলে এটা কাজে লাগে:

```
$ echo "price 250 taka" | sed 's/[0-9]\+/[&]/'
price [250] taka
```

```
$ echo "hello world" | sed 's/[a-z]*/"&"/g'
"hello" "world"
```

আক্ষরিক `&` চাইলে escape করো:

```
$ echo "Tom and Jerry" | sed 's/ and / \& /'
Tom & Jerry
```

> ⚠️ **সতর্কতা:** URL-এ প্রায়ই `&` থাকে (`?a=1&b=2`)। replacement-এ URL বসাতে গেলে `&` escape না করলে sed সেখানে মিলে যাওয়া অংশ বসিয়ে দেবে — খুব প্রচলিত ভুল।

`\1` … `\9` — group ধরা

regex-এর একটা অংশকে group-এ রাখলে (`\(...\)` BRE-তে, `(...)` `-E`-তে), replacement-এ সেটাকে নম্বর দিয়ে ফেরত আনা যায়। বাম দিক থেকে প্রথম খোলা বন্ধনী = `\1`, দ্বিতীয় = `\2`, ইত্যাদি।

```
  echo "Rahim Uddin" | sed -E 's/(\w+) (\w+)/\2, \1/'
                                ──────  ──────
                                  \1      \2
  ফল:  Uddin, Rahim
```

```
$ echo "Rahim Uddin" | sed -E 's/(\w+) (\w+)/\2, \1/'
Uddin, Rahim
```

আরও উদাহরণ:

```
# key=value কে value=key করো
$ echo "name=karim" | sed -E 's/([^=]+)=(.*)/\2=\1/'
karim=name

# একই শব্দ পরপর দুবার (the the) খুঁজে একবার করো — regex-এর ভেতরেও \1 চলে
$ echo "this is the the book" | sed -E 's/\b(\w+) \1\b/\1/g'
this is the book

# ফোন নম্বর ফরম্যাট
$ echo "01712345678" | sed -E 's/^(0[0-9]{4})([0-9]{6})$/\1-\2/'
01712-345678
```

> 📝 **মনে রাখো:** `\w`, `\b`, `\s` GNU extension; POSIX-নিরাপদ রূপ হলো `[[:alnum:]_]`, `[[:space:]]` ইত্যাদি। BSD sed-এ `\w` নাও চলতে পারে।

কেস বদলানো: `\U`, `\L`, `\u`, `\l`, `\E` (GNU)

```
$ echo "hello world" | sed 's/.*/\U&/'
HELLO WORLD
$ echo "hello world" | sed -E 's/\b(.)/\u\1/g'
Hello World
$ echo "MY_VARIABLE" | sed 's/.*/\L&/'
my_variable
$ echo "name: karim" | sed -E 's/(name: )(.*)/\1\U\2\E!/'
name: KARIM!
```

২১.৭ Address — কোন লাইনে কাজ হবে

address না দিলে কমান্ড **সব লাইনে** চলে। address দিয়ে নির্দিষ্ট লাইন বেছে নেওয়া যায়। অনুশীলনের জন্য একটা ফাইল বানাই:

```
$ seq 1 10 > nums.txt
$ cat nums.txt
1
2
3
...
10
```

Address-এর ধরন

Address	মানে	উদাহরণ
`N`	N নম্বর লাইন	`sed -n '3p'`
`$`	শেষ লাইন	`sed -n '$p'`
`/regex/`	regex মেলে এমন লাইন	`sed -n '/error/p'`
`\%regex%`	অন্য delimiter দিয়ে regex (`%` যেকোনো অক্ষর)	`sed -n '|/usr`
`/regex/I`	কেস উপেক্ষা করে regex (GNU)	`sed -n '/error/Ip'`
`A,B`	A থেকে B পর্যন্ত range	`sed -n '3,6p'`
`A,$`	A থেকে শেষ পর্যন্ত	`sed -n '8,$p'`
`/re1/,/re2/`	re1 মেলা লাইন থেকে re2 মেলা লাইন পর্যন্ত	`sed -n '/BEGIN/,/END/p'`
`A,+N`	A আর তার পরের N লাইন (GNU)	`sed -n '/error/,+2p'`
`A,~N`	A থেকে পরের N-এর গুণিতক লাইন পর্যন্ত (GNU)	`sed -n '5,~4p'` → ৫–৮
`first~step`	first থেকে শুরু করে প্রতি step-তম লাইন (GNU)	`sed -n '1~2p'` → বিজোড় লাইন
`0,/regex/`	একদম প্রথম লাইন থেকেই regex খোঁজা শুরু (GNU)	নিচে দেখো

Address	মানে	উদাহরণ
`addr!`	address-এর **উল্টো**	`sed '/^#/!d'`

উদাহরণ:

```
$ sed -n '3p' nums.txt
3
$ sed -n '$p' nums.txt
10
$ sed -n '4,6p' nums.txt
4
5
6
$ sed -n '1~3p' nums.txt
1
4
7
10
$ sed -n '0~5p' nums.txt
5
10
$ sed '2,9d' nums.txt
1
10
$ sed -n '2,9!p' nums.txt
1
10
```

Range কীভাবে কাজ করে (ভেতরের কথা)

range (`A,B`) একটা "সুইচ"-এর মতো:

```
লাইন আসছে ──►  [সুইচ বন্ধ]  ── A মিলল? ── হ্যাঁ ──► [সুইচ চালু] (এই লাইনও ধরা)
                                                          │
                          পরের লাইনগুলো ধরা হচ্ছে ◄──────┘
                                                          │
                            B মিলল? ── হ্যাঁ ──► এই লাইন ধরা, তারপর [সুইচ বন্ধ]
                                                          │
                          আবার A খোঁজা শুরু (range বারবার চালু হতে পারে)
```

মজার কিন্তু গুরুত্বপূর্ণ নিয়মগুলো:

1. **end address (B) শুরুর লাইনে পরীক্ষা করা হয় না।** তাই `/x/,/x/` লিখলে একই লাইনে শুরু আর শেষ হয় না; পরের `x`-ওয়ালা লাইন পর্যন্ত যায়।
2. **B যদি লাইন নম্বর হয় এবং সেটা A-র চেয়ে ছোট বা সমান হয়**, তাহলে শুধু A-র একটা লাইনই ধরা হয়। যেমন `sed -n '5,2p'` শুধু লাইন ৫ দেখাবে।
3. **B কখনো না মিললে** range ফাইলের শেষ পর্যন্ত চলে।
4. regex range **একাধিকবার** চালু হতে পারে — ফাইলে যতবার BEGIN...END জোড়া আছে, সবগুলো ধরা পড়বে।

```
$ cat block.txt
intro
BEGIN
  data 1
END
middle
BEGIN
  data 2
END
$ sed -n '/BEGIN/,/END/p' block.txt
BEGIN
  data 1
END
BEGIN
  data 2
END
```

`0,/regex/` — প্রথম মিলটাই শুধু (GNU)

ধরো, ফাইলের **শুধু প্রথম** `foo` বদলাতে চাও (প্রতিটি লাইনের প্রথম নয়, পুরো ফাইলের প্রথম)।

```
$ printf 'foo\nfoo\nfoo\n' | sed '0,/foo/s//bar/'
bar
foo
foo
```

`1,/foo/` লিখলে কাজ হতো না যদি প্রথম লাইনেই foo থাকে, কারণ নিয়ম ১ অনুযায়ী end address লাইন ১-এ পরীক্ষাই হয় না — range পরের foo পর্যন্ত চলে যেত। `0,` এই সমস্যার সমাধান। আর লক্ষ করো `s//bar/` — **খালি regex মানে "শেষবার ব্যবহৃত regex"**, অর্থাৎ এখানে `foo`।

Negation `!`

```
# কমেন্ট না এমন সব লাইন মুছো (অর্থাৎ শুধু কমেন্ট দেখাও)
$ sed '/^#/!d' file

# শেষ লাইন ছাড়া সব লাইনের শেষে কমা বসাও
$ printf 'a\nb\nc\n' | sed '$!s/$/,/'
a,
b,
c
```

> 💡 **টিপস:** shell-এ `!` history expansion ঘটায় (interactive bash-এ, double quote-এর ভেতরে)। single quote-এর ভেতরে লিখলে এই সমস্যা হয় না — আরও একটা কারণ single quote ব্যবহারের।

২১.৮ মৌলিক কমান্ডগুলো

`d` — delete

pattern space মুছে পরের cycle শুরু করে (স্বয়ংক্রিয় প্রিন্ট হয় না)।

```
$ sed '1d' nums.txt            # প্রথম লাইন বাদ (header বাদ দেওয়ার কৌশল)
$ sed '$d' nums.txt            # শেষ লাইন বাদ
$ sed '/^$/d' file             # ফাঁকা লাইন বাদ
$ sed '/^#/d' file             # # দিয়ে শুরু লাইন বাদ
```

`p` — print

```
$ sed -n '/root/p' /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

এটা `grep root /etc/passwd`-এর মতোই।

`=` — লাইন নম্বর প্রিন্ট

```
$ sed -n '/bash$/=' /etc/passwd
1
34
$ sed -n '$=' /etc/passwd          # মোট লাইন সংখ্যা (wc -l এর মতো)
45
```

`a`, `i`, `c` — append, insert, change

কমান্ড	কাজ
`a text`	লাইনের **পরে** নতুন লাইন যোগ
`i text`	লাইনের **আগে** নতুন লাইন যোগ
`c text`	লাইনটাকে **পুরো বদলে** text বসাও

GNU sed এক লাইনে লেখার সুবিধা দেয়:

```
$ printf 'one\ntwo\nthree\n' | sed '2a NEW'
one
two
NEW
three
$ printf 'one\ntwo\nthree\n' | sed '2i NEW'
one
NEW
two
three
$ printf 'one\ntwo\nthree\n' | sed '2c NEW'
```

```
one
NEW
three
$ printf 'one\ntwo\nthree\n' | sed '$a THE END'
one
two
three
THE END
```

POSIX/BSD-তে `a`, `i`, `c`-এর পর `\` আর newline দিতে হয়:

```
sed '2a\
NEW' file
```

একাধিক লাইন যোগ করতে (GNU):

```
$ sed '1a line A\nline B' file      # GNU: text-এ \n চলে
```

> 📝 **মনে রাখো:**
>
> - `a` দিয়ে যোগ করা টেক্সট pattern space-এ যায় না; এটা একটা queue-তে থাকে এবং cycle-এর শেষে (বা পরের লাইন পড়ার সময়) output-এ যায়। তাই পরের কোনো `s` কমান্ড এই টেক্সট বদলাতে পারে না।
> - `i` এর টেক্সট সাথে সাথে output-এ চলে যায়।
> - `a`-এর পর text-এর শুরুর ফাঁকা জায়গা GNU sed কেটে ফেলে। শুরুতে স্পেস রাখতে চাইলে `a\` দিয়ে লেখো: `sed '2a\ indented'`।
> - **range-এর সাথে `c`:** GNU sed-এ `2,4c TEXT` পুরো range মুছে **একবার** TEXT বসায় (range-এর শেষে)। কিন্তু `!` থাকলে প্রতিটি লাইনে বসায়।

```
$ seq 5 | sed '2,4c ---removed---'
1
---removed---
5
```

`y` — অক্ষর অনুবাদ (transliterate)

`y/abc/xyz/` মানে a→x, b→y, c→z। দুই পাশে **সমান সংখ্যক** অক্ষর থাকতে হবে। এটা regex নয়, একটা একটা অক্ষর মেলায় — অনেকটা `tr`-এর মতো (অধ্যায় ২৩)।

```
$ echo "hello" | sed 'y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/'
HELLO
$ echo "a-b-c" | sed 'y/-/_/'
a_b_c
```

> ⚠️ **সতর্কতা:** `y`-তে range (`a-z`) চলে না — `y/a-z/A-Z/` শুধু a→A, -→-, z→Z করবে। সব অক্ষর লিখে দিতে হয়।

`q` ও `Q` — quit

`q` বর্তমান লাইন প্রিন্ট করে বেরিয়ে যায়; `Q` (GNU) প্রিন্ট না করেই বেরোয়। বিশাল ফাইলে দ্রুত কাজ শেষ করার জন্য এটা দারুণ — বাকি ফাইল আর পড়া হয় না।

```
$ sed '3q' nums.txt          # head -n 3 এর মতো
1
2
3
$ sed '3Q' nums.txt
1
2
$ sed -n '1000000{p;q}' huge.log   # ১০ লাখতম লাইন দেখে সাথে সাথে বেরোও
```

GNU-তে exit code-ও দেওয়া যায়: `sed '/ERROR/q 5' file; echo $?` — ERROR পেলে exit code 5। স্ক্রিপ্টে কাজে লাগে।

`l` — অদৃশ্য অক্ষর দেখাও

```
$ printf 'a\tb  \r\n' | sed -n 'l'
a\tb  \r$
```

tab (`\t`), Windows-এর carriage return (`\r`) আর লাইনের শেষ (`$`) দেখা যাচ্ছে — trailing space বা CRLF সমস্যা ধরতে চমৎকার।

`r` ও `w` — ফাইল পড়া ও লেখা

```
$ sed '/<!-- footer -->/r footer.html' page.html   # মিলে যাওয়া লাইনের পরে footer.html ঢোকাও
$ sed -n '/ERROR/w errors.log' app.log             # ERROR লাইনগুলো errors.log-এ লেখো
```

`n` ও `N` — পরের লাইন

কমান্ড	কাজ
`n`	(`-n` না থাকলে) বর্তমান pattern space প্রিন্ট করে, তারপর পরের লাইন দিয়ে **প্রতিস্থাপন** করে
`N`	পরের লাইনকে `\n` দিয়ে জুড়ে pattern space-এ **যোগ** করে (এখন pattern space-এ দুই লাইন)
`P`	pattern space-এর প্রথম `\n` পর্যন্ত প্রিন্ট
`D`	pattern space-এর প্রথম `\n` পর্যন্ত মুছে, নতুন লাইন না পড়ে cycle আবার শুরু

```
# প্রতি জোড় লাইন মুছো: n দিয়ে বিজোড়টা প্রিন্ট, পরেরটা নিয়ে d
$ seq 6 | sed 'n;d'
1
3
5

# দুই লাইন করে জুড়ে দাও
$ seq 6 | sed 'N;s/\n/ /'
1 2
3 4
5 6
```

> 📝 **মনে রাখো:** GNU sed-এ শেষ লাইনে `N` চললে (পরের লাইন নেই) pattern space প্রিন্ট করে বেরিয়ে যায়। POSIX অনুযায়ী প্রিন্ট না করেই বেরোনোর কথা — তাই বহনযোগ্য স্ক্রিপ্টে `$!N` লেখা নিরাপদ।

`b`, `t`, `T` ও label — শাখা (একটু অ্যাডভান্সড)

sed-এ ছোট্ট "goto" আছে:

কমান্ড	কাজ
`:label`	একটা জায়গার নাম দাও
`b label`	label-এ লাফ দাও (label না দিলে script-এর শেষে)
`t label`	শেষ ইনপুট পড়ার পর কোনো `s` সফল হলে লাফ দাও
`T label`	কোনো `s` সফল **না** হলে লাফ দাও (GNU)

বিখ্যাত উদাহরণ — পুরো ফাইলের সব লাইন এক লাইনে জোড়া:

```
$ seq 5 | sed ':a;N;$!ba;s/\n/,/g'
1,2,3,4,5
```

ভেঙে দেখি:

```
:a        → এখানে label "a"
N         → পরের লাইন জুড়ে দাও
$!ba      → শেষ লাইন না হলে a-তে ফেরত যাও (লুপ)
s/\n/,/g  → শেষে সব newline কে কমা বানাও
```

(GNU-তে আজকাল সহজ রূপ: `sed -z 's/\n/,/g'`, তবে শেষ newline-ও কমা হয়ে যাবে।)

সংখ্যায় হাজারের কমা বসানো (`t` দিয়ে লুপ):

```
$ echo "1234567" | sed -E ':a;s/([0-9])([0-9]{3})($|,)/\1,\2\3/;ta'
1,234,567
```

২১.৯ একাধিক কমান্ড একসাথে

চারটা উপায় আছে:

```
# ১. সেমিকোলন
$ sed 's/a/A/;s/b/B/' file

# ২. একাধিক -e
$ sed -e 's/a/A/' -e 's/b/B/' file

# ৩. quote-এর ভেতরে newline
$ sed '
s/a/A/
s/b/B/
' file

# ৪. script ফাইল
$ sed -f fix.sed file
```

`{ }` — একটা address-এ কয়েকটা কমান্ড

```
# যে লাইনে "server" আছে, শুধু সেখানে দুটো বদল করো
$ sed '/server/{s/http/https/;s/80/443/}' config

# ৫ নম্বর লাইন প্রিন্ট করে বেরিয়ে যাও
$ sed -n '5{p;q}' file
```

address-এর ভেতরে address (nested) — শুধু `[database]` সেকশনের ভেতরে `port` বদলানো:

```
$ cat app.ini
[web]
port=80
[database]
port=3306
[cache]
port=6379
$ sed '/^\[database\]/,/^\[/{s/^port=.*/port=5432/}' app.ini
[web]
port=80
[database]
port=5432
[cache]
port=6379
```

sed script ফাইল

জটিল কাজ হলে ফাইলে লেখো — কমেন্টও দেওয়া যায় (`#`):

```
$ cat cleanup.sed
# কমেন্ট লাইন মুছো
```

```
/^[[:space:]]*#/d
# ফাঁকা লাইন মুছো
/^[[:space:]]*$/d
# লাইনের শেষের অপ্রয়োজনীয় স্পেস কাটো
s/[[:space:]]*$//

$ sed -f cleanup.sed /etc/ssh/sshd_config
```

ফাইলটাকে সরাসরি চালানোর যোগ্যও বানানো যায়, প্রথম লাইনে shebang দিয়ে (shebang ও স্ক্রিপ্ট অধ্যায় ৪১-এ):

```
#!/usr/bin/sed -f
s/foo/bar/g
```

```
$ chmod +x my.sed
$ ./my.sed input.txt
```

> ⚠️ **সতর্কতা:** `a`, `i`, `c`, `r`, `w`, `:label` — এদের পর সেমিকোলন দিয়ে আরেকটা কমান্ড লিখলে সেটা text বা ফাইলের নামের অংশ হয়ে যায়। যেমন `sed '2a hello; 3d'` লাইন ৩ মুছবে না, বরং `hello; 3d` টেক্সট যোগ করবে। এদের পর নতুন `-e` বা newline ব্যবহার করো: `sed -e '2a hello' -e '3d'`।

২১.১০ রেগুলার এক্সপ্রেশন: sed-এ যা বিশেষভাবে জানতে হবে

regex-এর পুরো আলোচনা অধ্যায় ২০-এ। sed-এর জন্য বাড়তি কথা:

- **sed সবসময় সবচেয়ে বাম দিকের, সবচেয়ে লম্বা (greedy) মিল নেয়।** `.*` যতটা সম্ভব খেয়ে ফেলে।

```
$ echo '<b>bold</b> and <i>italic</i>' | sed 's/<.*>//g'

$ echo '<b>bold</b> and <i>italic</i>' | sed 's/<[^>]*>//g'
bold and italic
```

প্রথমটা পুরো লাইন মুছে দিল কারণ `<.*>` প্রথম `<` থেকে শেষ `>` পর্যন্ত ধরেছে। sed-এ non-greedy `.*?` **নেই**; তার বদলে "নেগেটিভ ক্লাস" `[^>]*` ব্যবহার করো।

- **pattern space-এ শেষের newline থাকে না**, তাই `$` লাইনের শেষ বোঝায় এবং `\n` দিয়ে সাধারণ লাইনে কিছু মেলে না (যদি না `N` দিয়ে লাইন জোড়া হয়)।
- **খালি regex** `//` মানে শেষ ব্যবহৃত regex: `sed '/error/s//ERROR/g'`।
- character class: `[[:digit:]]`, `[[:alpha:]]`, `[[:space:]]`, `[[:upper:]]` — locale-নিরাপদ।

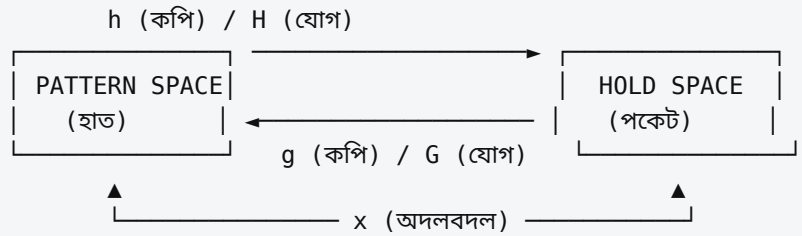
> 💡 **টিপস:** বিশেষ অক্ষর (`.` , `*` , `[` , `]` , `^` , `$` , `\` , `/`) আক্ষরিকভাবে খুঁজতে চাইলে `\` দাও। যেমন IP-তে `.` খুঁজতে: `s/192\.168\.1\.1/10.0.0.1/` — `\.` না দিলে `.` যেকোনো অক্ষর মেলাবে, ফলে `192x168` ও মিলে যেত।

২১.১১ Hold space পরিচিতি

এতক্ষণ আমরা শুধু pattern space ব্যবহার করেছি, যা প্রতিটি cycle শেষে খালি হয়ে যায়। কিন্তু কখনো আগের লাইনের তথ্য মনে রাখা দরকার হয় — তখন **hold space**।

> 💡 **উপমা:** pattern space হলো তোমার হাত, hold space হলো তোমার পকেট। প্রতিটা বাক্স হাতে নিয়ে কাজ করো, কাজ শেষে হাত খালি হয়ে যায়। কিন্তু পকেটে যা রাখো তা পরের বাক্সের সময়েও থাকে।

কমান্ড	মনে রাখার কৌশল	কাজ
`h`	**h**old	pattern → hold (hold-এর পুরনো জিনিস মুছে **কপি**)
`H`	**H**old append	pattern → hold-এ **যোগ** (মাঝে `\n` দিয়ে)
`g`	**g**et	hold → pattern (pattern-এর জিনিস মুছে কপি)
`G`	**G**et append	hold → pattern-এ **যোগ** (মাঝে `\n`)
`x`	e**x**change	pattern আর hold **অদলবদল**



উদাহরণ ১: প্রতি লাইনের পর ফাঁকা লাইন (double spacing)

hold space শুরুতে খালি। `G` প্রতিটি লাইনের শেষে `\n` + (খালি hold) যোগ করে:

```
$ printf 'a\nb\nc\n' | sed G
a

b

c
```

উদাহরণ ২: ফাইল উল্টো করা (`tac`-এর মতো)

```
$ printf 'one\ntwo\nthree\n' | sed -n '1!G;h;$p'
three
two
one
```

ধাপে ধাপে:

```
লাইন  কমান্ড        PATTERN                 HOLD
 1    (1!G বাদ)      one                       ""
      h               one                      one
 2    G               two\none                 one
      h               two\none                 two\none
 3    G               three\ntwo\none          two\none
      h               three\ntwo\none          three\ntwo\none
      $p              → প্রিন্ট: three / two / one
```

উদাহরণ ৩: কোনো লাইনের আগের লাইন দেখাও

```
$ printf 'user=ali\nstatus=FAIL\nuser=sumi\nstatus=OK\n' | sed -n '/FAIL/{x;p;x;p};h'
user=ali
status=FAIL
```

এখানে প্রতিটি লাইন `h` দিয়ে পকেটে রাখা হচ্ছে; FAIL পেলে `x` দিয়ে আগের লাইন বের করে প্রিন্ট, আবার `x` দিয়ে বর্তমানটা ফিরিয়ে প্রিন্ট।

> 📝 **মনে রাখো:** hold space শক্তিশালী, কিন্তু জটিল যুক্তির কাজ হলে `awk` (অধ্যায় ২২) সাধারণত বেশি পরিষ্কার। sed-এ ৩–৪ লাইনের বেশি hold-space কৌশল লাগলে awk ভেবে দেখো।

২১.১২ বাস্তব রেসিপি — রোজকার কাজের sed

ক. ফাঁকা লাইন ও স্পেস পরিষ্কার

```
sed '/^$/d' file                    # সম্পূর্ণ ফাঁকা লাইন মুছো
sed '/^[[:space:]]*$/d' file        # শুধু স্পেস/ট্যাব আছে এমন লাইনও মুছো
sed 's/^[[:space:]]*//' file        # লাইনের শুরুর স্পেস কাটো (ltrim)
sed 's/[[:space:]]*$//' file        # লাইনের শেষের স্পেস কাটো (rtrim)
sed 's/^[[:space:]]*//;s/[[:space:]]*$//' file   # দুই দিকেই
sed 's/  */ /g' file                # একাধিক স্পেসকে এক স্পেস
sed '/^$/N;/^\n$/D' file            # পরপর অনেক ফাঁকা লাইনকে একটায় নামাও (cat -s এর মতো)
```

খ. কমেন্ট বাদ দিয়ে আসল কনফিগ দেখা

সার্ভারের কনফিগ ফাইলে শত শত লাইন কমেন্ট থাকে। আসল সেটিং দেখতে:

```
$ sed -e '/^[[:space:]]*#/d' -e '/^[[:space:]]*$/d' /etc/ssh/sshd_config
Include /etc/ssh/sshd_config.d/*.conf
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding yes
PrintMotd no
AcceptEnv LANG LC_*
Subsystem       sftp    /usr/lib/openssh/sftp-server
```

(output Ubuntu 24.04-এর ডিফল্ট ফাইলের মতো; তোমার সিস্টেমে ভিন্ন হতে পারে।)

গ. কনফিগ ফাইল এডিট করা

একটা সেটিং বদলানো (কমেন্ট করা থাকুক বা না থাকুক):

```
# "#PermitRootLogin prohibit-password" বা "PermitRootLogin yes" — যা-ই থাকুক →
"PermitRootLogin no"
$ sudo sed -i.bak -E 's/^#?[[:space:]]*PermitRootLogin[[:space:]].*/PermitRootLogin
no/' /etc/ssh/sshd_config
$ grep -n '^PermitRootLogin' /etc/ssh/sshd_config
33:PermitRootLogin no
```

তারপর অবশ্যই কনফিগ যাচাই করে সার্ভিস reload করো (SSH নিয়ে বিস্তারিত অধ্যায় ৩৮, systemd অধ্যায় ২৬-এ):

```
$ sudo sshd -t && sudo systemctl reload ssh      # RHEL/Fedora-তে সার্ভিসের নাম sshd
```

একটা লাইন কমেন্ট করা / আন-কমেন্ট করা:

```
sed -i '/^Listen 8080/s/^/# /' httpd.conf      # কমেন্ট করো
sed -i 's/^#[[:space:]]*\(Listen 8080\)/\1/' httpd.conf   # কমেন্ট সরাও
```

key=value মান বদলানো:

```
$ cat .env
APP_ENV=local
APP_DEBUG=true
DB_HOST=127.0.0.1
$ sed -i 's/^APP_DEBUG=.*/APP_DEBUG=false/' .env
$ sed -i 's|^DB_HOST=.*|DB_HOST=db.internal|' .env
$ cat .env
APP_ENV=local
APP_DEBUG=false
DB_HOST=db.internal
```

লাইন না থাকলে যোগ করো, থাকলে বদলাও (idempotent — বারবার চালালেও ফল একই):

```
grep -q '^MAX_CONN=' app.conf \
  && sed -i 's/^MAX_CONN=.*/MAX_CONN=200/' app.conf \
  || echo 'MAX_CONN=200' >> app.conf
```

নির্দিষ্ট লাইনের পরে নতুন সেটিং যোগ:

```
sed -i '/^\[mysqld\]/a max_connections = 500' /etc/mysql/my.cnf
```

ঘ. লাইন বের করা (বড় ফাইল থেকে)

```
sed -n '100,200p' big.log           # ১০০–২০০ লাইন
sed -n '100,200p;201q' big.log      # একই, কিন্তু ২০১-এ থেমে যায় → অনেক দ্রুত
sed -n '/10:00:00/,/10:05:00/p' app.log   # দুটো সময়ের মাঝের লগ
sed -n '$p' file                    # শেষ লাইন (tail -n 1)
sed '1d' data.csv                   # CSV-এর header বাদ
```

ঙ. টেক্সট রূপান্তর

```
sed 's/\r$//' winfile.txt                    # Windows CRLF → Linux LF (dos2unix-এর
মতো)
sed 's/$/\r/' unixfile.txt                   # উল্টো: LF → CRLF (GNU)
sed 's/^/    /' code.txt                      # প্রতি লাইনের শুরুতে ৪ স্পেস (indent)
sed 's/^/> /' email.txt                      # ইমেইলে quote করা
sed = file | sed 'N;s/\n/\t/'                # লাইন নম্বর বসাও (cat -n এর মতো)
sed 's/,/\t/g' data.csv                      # CSV → TSV (সাধারণ CSV-র জন্য)
sed -E 's/([0-9]{1,3}\.){3}[0-9]{1,3}/x.x.x.x/g' access.log   # IP লুকাও
```

চ. একসাথে অনেক ফাইলে বদল

```
# প্রজেক্টের সব .php ফাইলে পুরনো ডোমেইন বদলাও (আগে দেখো, তারপর -i)
$ grep -rl 'old-domain.com' --include='*.php' . | xargs sed -i 's/old-domain\.com/new-
domain.com/g'

# ফাইলের নামে স্পেস থাকলে নিরাপদ রূপ
$ grep -rlZ 'old-domain.com' --include='*.php' . | xargs -0 sed -i 's/old-
domain\.com/new-domain.com/g'

# find দিয়ে
$ find . -name '*.conf' -exec sed -i.bak 's/foo/bar/g' {} +
```

(`xargs` আর `find -exec` বিস্তারিত অধ্যায় ১২ ও অধ্যায় ২৩-এ।)

> ⚠️ **সতর্কতা:** অনেক ফাইলে একসাথে `-i` চালানো খুব শক্তিশালী — এবং খুব বিপজ্জনক। ভুল regex দিলে শত ফাইল একসাথে নষ্ট হবে। git রিপোজিটরিতে থাকলে আগে `git status` পরিষ্কার রাখো যাতে `git`

> `diff` দিয়ে দেখতে পারো, আর `git checkout -- .` দিয়ে ফেরত যেতে পারো। না হলে `-i.bak` ব্যবহার করো।

ছ. pipe-এর মাঝে sed

```
$ ip -4 addr show | sed -n 's/.*inet \([0-9.]*\).*/\1/p'
127.0.0.1
192.168.1.105

$ echo "$PATH" | sed 's/:/\n/g'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
...

$ tail -f /var/log/syslog | sed -u 's/error/\x1b[31mERROR\x1b[0m/Ig'   # লাইভ লগে error লাল রঙে
```

(শেষটায় `-u` দিয়েছি যাতে output সাথে সাথে দেখা যায়; `\x1b` GNU-র escape।)

২১.১৩ Shell variable ব্যবহার করা

স্ক্রিপ্টে প্রায়ই মান variable থেকে আসে। তখন double quote লাগে:

```
$ name="Karim"
$ echo "Hello USER" | sed "s/USER/$name/"
Hello Karim
```

সমস্যা হয় যখন variable-এ delimiter বা বিশেষ অক্ষর থাকে:

```
$ dir="/var/www/html"
$ echo "root=X" | sed "s/X/$dir/"
sed: -e expression #1, char 9: unknown option to `s'
$ echo "root=X" | sed "s|X|$dir|"
root=/var/www/html
```

> ⚠️ **সতর্কতা:** variable-এ `&`, `\` বা delimiter থাকলে ফল ভুল হবে বা error আসবে। ইউজারের দেওয়া অজানা টেক্সট replacement-এ বসাতে হলে আগে escape করো:
>
> ```
> safe=$(printf '%s' "$value" | sed 's/[&/\]/\\&/g')
> sed "s/PLACEHOLDER/$safe/" template.txt
> ```

single আর double quote মিশিয়েও লেখা যায় — শুধু variable অংশ double quote-এ:

```
$ sed 's/^port=.*/port='"$PORT"'/' app.conf
```

২১.১৪ GNU sed বনাম BSD (macOS) sed

বিষয়	GNU sed (Linux)	BSD sed (macOS/FreeBSD)
ব্যাকআপ ছাড়া in-place	`sed -i 's/a/b/' f`	`sed -i '' 's/a/b/' f`
ERE	`-E` বা `-r`	`-E`
`\n`, `\t` replacement-এ	চলে	সাধারণত চলে না
`\U \L \u \l`	চলে	চলে না
`I` flag / `/re/I`	চলে	চলে না (পুরনো সংস্করণে)
`a text` এক লাইনে	চলে	`a\` + newline লাগে
`0,/re/`, `first~step`, `addr,+N`	চলে	চলে না
`Q`, `T`, `e`, `--debug`, `-z`	চলে	চলে না

> 💡 **টিপস:** macOS-এ GNU sed চাইলে `brew install gnu-sed` করো; এটা `gsed` নামে আসে। পোর্টেবল স্ক্রিপ্ট লিখতে চাইলে Linux-এ `sed --posix` দিয়ে পরীক্ষা করে নিতে পারো।

২১.১৫ সাধারণ ভুল ও সমাধান

ভুল	কী হয়	ঠিক রূপ
`sed 's/a/b/' f > f`	shell আগে `f` খালি করে দেয় → **ফাইল শূন্য!**	`sed -i 's/a/b/' f`
`sed -in ...`	`n` suffix হয়ে যায়	`sed -n -i ...`
`-n` ছাড়া `p`	লাইন দুবার আসে	`sed -n '...p'`
`g` ভুলে যাওয়া	প্রতি লাইনে শুধু প্রথম মিল বদলায়	`s/x/y/g`
path-এ `/`	`unknown option to 's'`	`s
`.` escape না করা	`1.2` এর জায়গায় `1x2`-ও মেলে	`1\.2`
`.*` greedy	অপ্রত্যাশিতভাবে বেশি মুছে যায়	`[^X]*`
BRE-তে `+`, `(`	আক্ষরিক অক্ষর হিসেবে ধরা হয়	`-E` দাও বা `\+`, `\(`
double quote-এ `$`	shell variable ভেবে বদলে দেয়	single quote
CRLF ফাইল	`$` দিয়ে লাইনের শেষ মেলে না	আগে `s/\r$//`

প্রথম ভুলটা সবচেয়ে বিপজ্জনক, তাই আবার বলি:

```
sed 's/a/b/' file > file
                  ──────
                     |
shell কমান্ড চালানোর আগেই redirection সেট করে → file কে 0 বাইট করে ফেলে
→ তারপর sed খালি ফাইল পড়ে → কিছুই লেখে না → সব ডেটা হারালে!
```

(redirection কীভাবে কাজ করে তা অধ্যায় ১৯-এ আছে।)

২১.১৬ সম্পর্কিত কীবোর্ড শর্টকাট

sed নিজে কোনো interactive প্রোগ্রাম নয়, তাই এর নিজস্ব শর্টকাট নেই। তবে sed নিয়ে কাজ করার সময় এই টার্মিনাল শর্টকাটগুলো বারবার লাগে (পুরো তালিকা অধ্যায় ৬-এ):

শর্টকাট	কাজ	sed-এর সাথে কোথায় কাজে লাগে
`Ctrl + D`	stdin শেষ (EOF) পাঠাও	ফাইল ছাড়া `sed 's/a/b/'` চালিয়ে হাতে টাইপ করে পরীক্ষা শেষে
`Ctrl + C`	চলমান কমান্ড থামাও	ভুলে ফাইল না দিলে sed stdin-এর অপেক্ষায় আটকে থাকে — তখন বেরোতে
`↑` / `Ctrl + P`	আগের কমান্ড	`-i` ছাড়া পরীক্ষা করে, ↑ চেপে `-i` যোগ করো
`Ctrl + R`	history-তে খোঁজা	আগে লেখা জটিল sed খুঁজে বের করা
`Alt + .`	আগের কমান্ডের শেষ argument	`sed ... file` এর পর `cat Alt+.`
`Ctrl + A` / `Ctrl + E`	লাইনের শুরু / শেষে যাও	লম্বা sed কমান্ড এডিট
`Ctrl + X Ctrl + E`	কমান্ডটা এডিটরে খোলো (bash)	বহু-লাইনের sed script লিখতে
`Ctrl + W`	আগের শব্দ মুছো	
`!!`	আগের পুরো কমান্ড	`sudo !!` — পারমিশন না থাকায় ব্যর্থ হলে
`^old^new`	আগের কমান্ডে old→new বদলে চালাও	নিজেই এক ছোট্ট sed! যেমন `^s/a/s/b`

> 💡 **টিপস:** Vim-এর `:%s/old/new/g` আর sed-এর `s/old/new/g` একই বংশের। sed শিখলে vim-এর search-replace (অধ্যায় ১৪) এমনিতেই শেখা হয়ে যায়, উল্টোটাও সত্যি।

হাতে-কলমে অনুশীলন

একটা আলাদা ফোল্ডারে কাজ করো যাতে কিছু নষ্ট না হয়।

ল্যাব ১: প্রস্তুতি

```
$ mkdir -p ~/sed-lab && cd ~/sed-lab
$ cat > students.txt << 'EOF'
# Student list 2024
id,name,dept,cgpa

1,rahim uddin,cse,3.75
2,karim ahmed,eee,3.20
3,sumi akter,cse,3.90

4,nila das,bba,2.95
5,tanvir hasan,cse,3.10
EOF
```

ল্যাব ২: দেখা ও বের করা

1. শুধু ৩ নম্বর লাইন দেখাও: `sed -n '3p' students.txt`
2. মোট লাইন গোনো: `sed -n '$=' students.txt` → `9`
3. `cse` ছাত্রদের দেখাও: `sed -n '/,cse,/p' students.txt`
4. কমেন্ট ও ফাঁকা লাইন বাদ দাও: `sed '/^#/d;/^$/d' students.txt`
5. header-সহ প্রথম তিনজন: `sed -n '2,6p' students.txt` — দেখো ফাঁকা লাইনও এসেছে; এবার `sed -n '2,6{/^$/!p}' students.txt` চালাও।

ল্যাব ৩: substitute

1. `cse` কে `CSE` করো: `sed 's/,cse,/,CSE,/' students.txt`
2. নামের প্রতিটি শব্দের প্রথম অক্ষর বড় হাতের করো:

   ```
   $ sed -E '/^[0-9]/s/,([a-z]+) ([a-z]+),/,\u\1 \u\2,/' students.txt
   ```

   প্রত্যাশিত: `1,Rahim Uddin,cse,3.75` ইত্যাদি।
3. কমা বদলে `|` দাও: `sed 's/,/ | /g' students.txt`
4. শুধু দ্বিতীয় কমা বদলাও: `sed 's/,/;/2' students.txt` — কী হলো ব্যাখ্যা করো।
5. cgpa-কে বন্ধনীতে রাখো `&` দিয়ে: `sed -E 's/[0-9]\.[0-9]{2}$/(&)/' students.txt`
6. নাম আর dept-এর জায়গা বদলাও:

   ```
   $ sed -E 's/^([0-9]+),([^,]+),([^,]+),/\1,\3,\2,/' students.txt
   ```

ল্যাব ৪: a, i, c, d, y, q

```
$ sed '/^id,/i # ---- data start ----' students.txt
$ sed '$a 6,new student,mat,3.00' students.txt
$ sed '/nila/c 4,REMOVED,-,-' students.txt
```

```
$ sed '/^[0-9]/y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/' students.txt
$ sed '/sumi/q' students.txt        # sumi-তে পৌঁছে থামো
```

ল্যাব ৫: in-place ও ব্যাকআপ

```
$ cp students.txt work.txt
$ ls -i work.txt                                    # inode নোট করো
$ sed -i.orig '/^$/d;/^#/d' work.txt
$ ls -i work.txt work.txt.orig                       # inode বদলেছে? .orig-এ কী আছে?
$ diff work.txt.orig work.txt
```

ল্যাব ৬: কনফিগ ফাইল

```
$ cat > app.ini << 'EOF'
[server]
#port = 8080
host = 0.0.0.0
[database]
port = 3306
user = root
EOF
```

1. `[server]` সেকশনের কমেন্ট করা port আন-কমেন্ট করে 9090 বানাও:

   ```
   $ sed -E '/^\[server\]/,/^\[/{s/^#?port = .*/port = 9090/}' app.ini
   ```

2. শুধু `[database]` সেকশনের `user` বদলে `app` করো (নিজে লেখো)।
3. `[database]`-এর পরে `password = secret` যোগ করো।
4. সব ঠিক থাকলে `-i.bak` দিয়ে ফাইলে লেখো।

ল্যাব ৭: hold space

```
$ seq 5 | sed -n '1!G;h;$p'          # উল্টো
$ seq 5 | sed G                       # double space
$ seq 5 | sed ':a;N;$!ba;s/\n/+/g'    # 1+2+3+4+5
$ seq 5 | sed ':a;N;$!ba;s/\n/+/g' | bc   # যোগফল 15
```

চ্যালেঞ্জ

- `/etc/passwd` থেকে শুধু ইউজারনেম আর shell দেখাও, `:` দিয়ে: (ইঙ্গিত: `sed -E 's/^([^:]*):.*:([^:]*)$/\1:\2/'`)
- একটা HTML ফাইল থেকে সব ট্যাগ মুছে শুধু লেখা রাখো।
- একটা লগ ফাইলে ঘণ্টা `HH:MM:SS` সময়কে `[HH:MM]` রূপে দেখাও।

সারসংক্ষেপ

কমান্ড/অপশন	কাজ
`sed 's/a/b/' f`	প্রতি লাইনে প্রথম a→b
`sed 's/a/b/g' f`	সব a→b
`sed 's/a/b/2' f`	দ্বিতীয় a→b
`sed 's/a/b/I' f`	কেস উপেক্ষা (GNU)
`sed -n 's/a/b/p' f`	শুধু বদলানো লাইন দেখাও
`sed 's	/x
`sed -n '5p' f`	৫ নম্বর লাইন
`sed -n '5,10p' f`	৫–১০ লাইন
`sed -n '$p' f`	শেষ লাইন
`sed -n '/re/p' f`	regex মেলা লাইন
`sed '/re/d' f`	regex মেলা লাইন মুছো
`sed '/re/!d' f`	regex না মেলা লাইন মুছো
`sed '/^$/d' f`	ফাঁকা লাইন মুছো
`sed '3a text' f`	৩-এর পরে লাইন যোগ
`sed '3i text' f`	৩-এর আগে লাইন যোগ
`sed '3c text' f`	৩ নম্বর লাইন বদলাও
`sed 'y/abc/xyz/' f`	অক্ষর অনুবাদ
`sed '10q' f`	১০ লাইন দেখিয়ে থামো
`sed -n '$=' f`	লাইন সংখ্যা
`sed -E '...'`	extended regex
`sed -e '..' -e '..'`	একাধিক কমান্ড
`sed -f x.sed f`	script ফাইল থেকে
`sed -i '...' f`	ফাইলেই বদল
`sed -i.bak '...' f`	ব্যাকআপ রেখে বদল
`&`	পুরো মিল
`\1` .. `\9`	group
`\U \L \u \l \E`	কেস বদল (GNU)
`h H g G x`	hold space কমান্ড
`n N P D`	পরের লাইন নিয়ে কাজ

কমান্ড/অপশন	কাজ
`:a ... ba, t`	লুপ/শাখা
`--debug`	ধাপে ধাপে দেখো (GNU)

নিজেকে যাচাই করো

1. sed-এর cycle-এ pattern space-এর কী হয়? `-n` দিলে কোন ধাপটা বন্ধ হয়?
2. `sed '2p' file` চালালে দ্বিতীয় লাইন দুবার আসে কেন?
3. `s/a/A/`, `s/a/A/g`, `s/a/A/3` আর `s/a/A/3g`-এর পার্থক্য কী?
4. `/usr/bin` কে `/opt/bin` বানানোর সবচেয়ে পরিষ্কার sed কমান্ড লেখো।
5. `sed 's/x/y/' file > file` কেন বিপজ্জনক? সঠিক উপায় কী?
6. `sed -i` চালালে ফাইলের inode বদলায় কেন?
7. `John Smith` কে `Smith, John` বানাতে কোন কমান্ড দেবে?
8. শুধু ১০ থেকে ২০ নম্বর লাইন মুছতে, আর শুধু ১০ থেকে ২০ নম্বর লাইন **রাখতে** কী লিখবে?
9. `a`, `i` আর `c`-এর পার্থক্য কী?
10. hold space কী, আর `sed -n '1!G;h;$p'` কী করে?

▼ উত্তর দেখো

1. প্রতিটি cycle-এ একটা লাইন (শেষের newline ছাড়া) pattern space-এ পড়া হয়, script-এর কমান্ডগুলো এর ওপর চলে, তারপর pattern space প্রিন্ট হয় এবং খালি করা হয়। `-n` দিলে **স্বয়ংক্রিয় প্রিন্টের ধাপ** বন্ধ হয়।
2. একবার `p` কমান্ড প্রিন্ট করে, আরেকবার cycle-শেষের স্বয়ংক্রিয় প্রিন্ট। সমাধান: `sed -n '2p' file`।
3. `s/a/A/` প্রতি লাইনের প্রথম a; `g` সব a; `3` শুধু তৃতীয় a; `3g` তৃতীয় ও তার পরের সব a (GNU)।
4. `sed 's|/usr/bin|/opt/bin|'` (বা অন্য কোনো delimiter যেমন `#`)।
5. shell redirection `> file` কমান্ড চালানোর আগেই ফাইলকে শূন্য করে দেয়, ফলে sed খালি ফাইল পড়ে এবং ডেটা হারায়। সঠিক: `sed -i 's/x/y/' file` বা অন্য ফাইলে লিখে `mv`।
6. কারণ `-i` আসলে একটা temporary ফাইলে নতুন কনটেন্ট লিখে তারপর rename করে মূল নামে আনে — নতুন ফাইল মানে নতুন inode। এজন্য হার্ড লিংক ভেঙে যায়।
7. `echo "John Smith" | sed -E 's/(\w+) (\w+)/\2, \1/'`
8. মুছতে: `sed '10,20d' file`; রাখতে: `sed -n '10,20p' file` (অথবা `sed '10,20!d' file`)।
9. `a` লাইনের পরে, `i` লাইনের আগে নতুন লাইন যোগ করে; `c` লাইনটাকেই নতুন টেক্সট দিয়ে প্রতিস্থাপন করে।

10. hold space একটা সহায়ক বাফার যা cycle পেরিয়েও টিকে থাকে — আগের লাইন মনে রাখতে ব্যবহার হয়। `1!G;h;$p` প্রতিটি নতুন লাইনের নিচে জমানো লাইনগুলো জুড়ে (`G`), পুরোটা আবার hold-এ রাখে (`h`), শেষ লাইনে প্রিন্ট করে — ফলে ফাইল উল্টো ক্রমে দেখায় (`tac`-এর মতো)।

অধ্যায় ২২: awk — টেক্সট প্রসেসিং ভাষা

এই অধ্যায়ে যা শিখবে

- `awk` আসলে কী, নামটা কোথা থেকে এলো, আর এটা কেন শুধু "কমান্ড" না, একটা পুরো **প্রোগ্রামিং ভাষা**
- awk ভেতরে কীভাবে কাজ করে: **record** (লাইন) আর **field** (কলাম)-এর ধারণা, আর awk-এর "অদৃশ্য লুপ"
- awk প্রোগ্রামের মূল গঠন: `pattern { action }`
- বিল্ট-ইন ভেরিয়েবল: `$0`, `$1`, `$NF`, `NR`, `NF`, `FNR`, `FS`, `OFS`, `RS`, `ORS`, `FILENAME`
- `-F` দিয়ে ফিল্ড আলাদা করার চিহ্ন (separator) বদলানো, `-v` দিয়ে বাইরে থেকে ভেরিয়েবল পাঠানো
- `BEGIN` আর `END` ব্লক: শুরুর আগে আর শেষের পরে কাজ
- শর্ত, তুলনা, লজিক্যাল অপারেটর, regex match (`~` ও `!~`), range pattern
- ভেরিয়েবল, string আর number-এর পার্থক্য, গাণিতিক হিসাব (যোগফল, গড়, সর্বোচ্চ, সর্বনিম্ন)
- `print` বনাম `printf`, সুন্দর টেবিল আকারে আউটপুট
- `if/else`, `for`, `while`, `do-while`, `next`, `exit`
- **associative array** দিয়ে গণনা ও গ্রুপ করা (awk-এর সবচেয়ে শক্তিশালী অস্ত্র)
- বিল্ট-ইন ফাংশন: `length`, `substr`, `index`, `split`, `sub`, `gsub`, `match`, `toupper`, `tolower`, `sprintf`, গাণিতিক ফাংশন
- নিজের ফাংশন লেখা, আউটপুট ফাইলে বা অন্য কমান্ডে পাঠানো, `getline`
- awk স্ক্রিপ্ট ফাইল (`-f`) আর shebang দিয়ে চালানো
- awk-এর বিভিন্ন রূপ: `gawk`, `mawk`, `busybox awk` — কোন distro-তে কোনটা, আর পার্থক্য কী
- বাস্তব রেসিপি: ওয়েব সার্ভার লগ বিশ্লেষণ, CSV রিপোর্ট, `/etc/passwd` পড়া, ডুপ্লিকেট সরানো, আর অনেক one-liner

---

২২.১ awk কী, আর কেন শিখবে?

নামের ইতিহাস

`awk` তৈরি হয়েছিল ১৯৭৭ সালে, AT&T-এর বিখ্যাত **Bell Labs**-এ (যেখানে Unix আর C ভাষারও জন্ম)। তিনজন বিজ্ঞানী এটা বানিয়েছিলেন:

- **A**lfred **A**ho
- Peter **W**einberger
- Brian **K**ernighan

তিনজনের নামের প্রথম অক্ষর মিলে হয়েছে **AWK**। তাই এটা কোনো ইংরেজি শব্দ না, উচ্চারণ "অক" (pronounced like the bird *auk*)।

awk কী সমস্যার সমাধান করে?

আগের দুই অধ্যায়ে আমরা দেখেছি:

- অধ্যায় ২০-এর `grep` → কোন **লাইন** দরকার সেটা খুঁজে বের করে
- অধ্যায় ২১-এর `sed` → লাইনের ভেতরের **লেখা বদলায়** (find & replace)

কিন্তু ধরো তোমার কাছে এমন একটা ফাইল আছে:

```
name dept salary age
Rahim IT 50000 28
Karim HR 35000 35
Sumi IT 62000 31
Nila Sales 40000 26
Jamal Sales 45000 40
```

এখন প্রশ্ন:

- শুধু নাম আর বেতন দেখাও (১ম আর ৩য় কলাম)
- যাদের বেতন ৪৫০০০-এর বেশি, শুধু তাদের দেখাও
- সবার বেতনের যোগফল আর গড় কত?
- প্রতিটা ডিপার্টমেন্টে মোট বেতন কত?

`grep` আর `sed` এগুলোর জন্য বানানো না। এখানেই awk-এর রাজত্ব। awk ডেটাকে **টেবিল** হিসেবে দেখে: প্রতিটা লাইন একটা **row**, আর প্রতিটা শব্দ একটা **column**। তারপর তুমি সেই কলাম নিয়ে হিসাব, তুলনা, গণনা, ফরম্যাটিং সব করতে পারো।

> 💡 **টিপস:** সহজভাবে মনে রাখো — `grep` = খোঁজো, `sed` = বদলাও, `awk` = **বিশ্লেষণ করো ও রিপোর্ট বানাও**।

উপমা: অফিসের হিসাবরক্ষক

ভাবো তোমার অফিসে একজন হিসাবরক্ষক আছেন। তুমি তাঁকে একটা খাতা দিলে, যেখানে প্রতিটা লাইনে একজন কর্মচারীর তথ্য। তুমি একটা চিরকুটে নির্দেশ লিখে দিলে:

> "যার ডিপার্টমেন্ট IT, তার নাম আর বেতন লিখে দাও। শেষে মোট বেতন জানাও।"

হিসাবরক্ষক কী করবেন?

1. খাতা খোলার আগে একটা ফাঁকা কাগজ নেবেন, যোগফল = ০ লিখে রাখবেন (**BEGIN**)

2. খাতার প্রতিটা লাইন একটা একটা করে পড়বেন (**record**)
3. প্রতিটা লাইনকে কলামে ভাগ করবেন (**field**)
4. চেক করবেন শর্ত মিলছে কিনা (**pattern**)
5. মিললে নির্দেশমতো কাজ করবেন (**action**)
6. সব লাইন শেষ হলে চূড়ান্ত যোগফল জানাবেন (**END**)

awk ঠিক এই হিসাবরক্ষক। আর তোমার চিরকুটটা হলো **awk প্রোগ্রাম**।

awk কেন শিখবে?

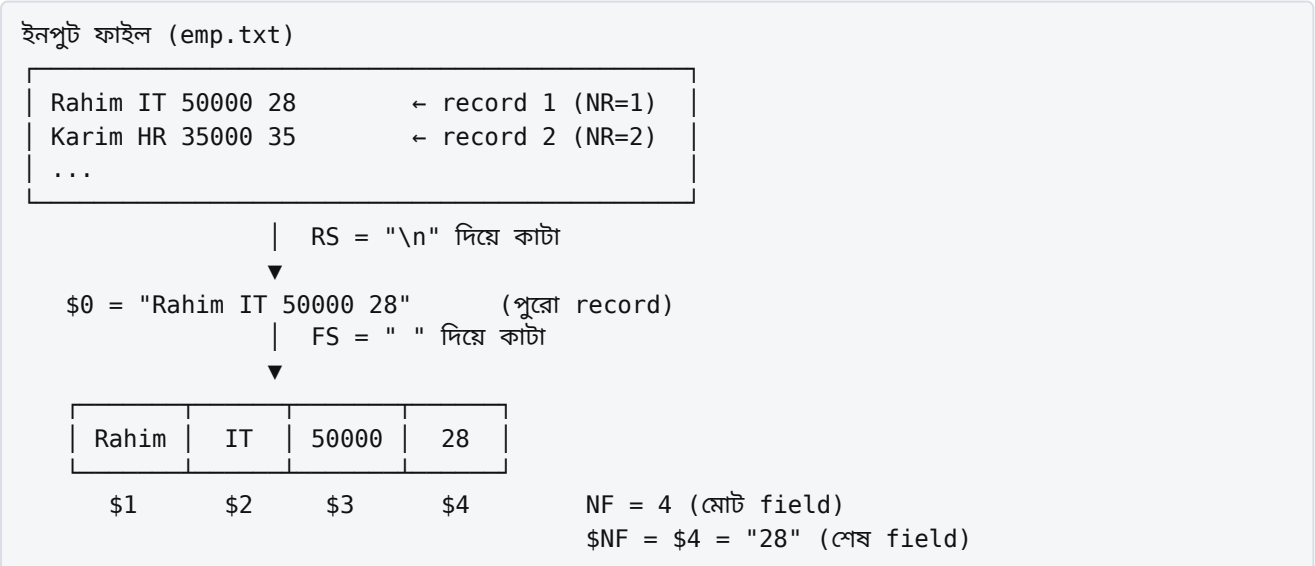
- লগ ফাইল বিশ্লেষণ: কোন IP সবচেয়ে বেশি রিকোয়েস্ট পাঠাচ্ছে, কয়টা 404 এরর হয়েছে
- কমান্ডের আউটপুট থেকে নির্দিষ্ট কলাম বের করা (`ps`, `df`, `ls -l`, `free` ইত্যাদি)
- CSV ফাইল থেকে রিপোর্ট তৈরি
- কনফিগারেশন ফাইল পার্স করা
- ছোট ছোট ডেটা প্রসেসিং, যেখানে Python স্ক্রিপ্ট লেখা বাড়াবাড়ি
- প্রায় সব লিনাক্স/ইউনিক্স সিস্টেমে awk আগে থেকেই ইনস্টল থাকে, কারণ এটা **POSIX** স্ট্যান্ডার্ডের অংশ

২২.২ ভেতরে কীভাবে কাজ করে — awk-এর আর্কিটেকচার

record আর field

awk ইনপুটকে দুই স্তরে ভাগ করে:

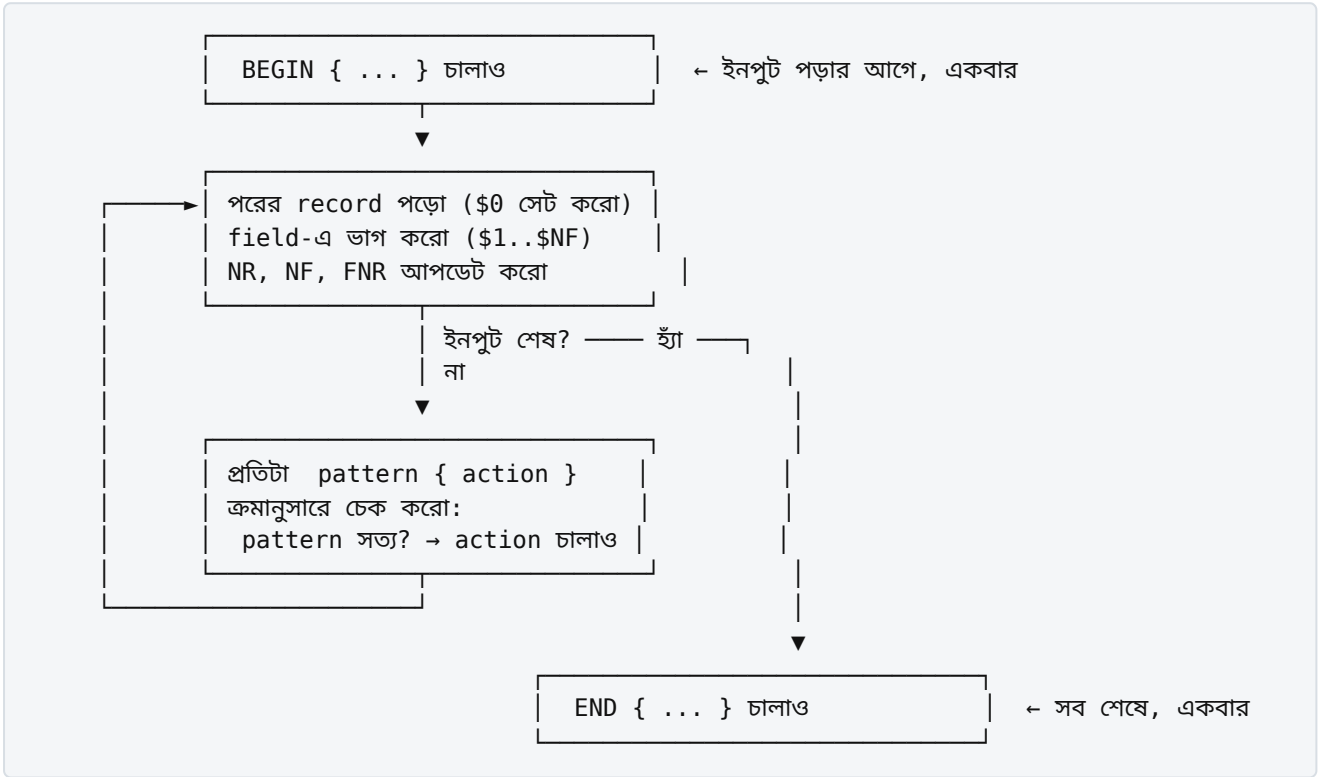
- **record** (রেকর্ড): ডিফল্টভাবে একটা **লাইন**। কোন চিহ্ন দেখে record আলাদা হবে, সেটা ঠিক করে `RS` (Record Separator), ডিফল্ট মান newline (`\n`)।
- **field** (ফিল্ড): record-এর ভেতরের একেকটা অংশ। ডিফল্টভাবে **space বা tab** দিয়ে আলাদা করা শব্দ। কোন চিহ্ন দেখে field আলাদা হবে, সেটা ঠিক করে `FS` (Field Separator)।



> 📝 **মনে রাখো:** ডিফল্ট `FS` হলো একটা space (`" "`), যার বিশেষ মানে আছে: **এক বা একাধিক space/tab একসাথে একটা separator**, আর লাইনের শুরু-শেষের ফাঁকা জায়গা উপেক্ষা করা হয়। তাই `" Rahim IT "` লাইনেও `$1` = `Rahim`, `$2` = `IT`। এটা `cut` কমান্ডের চেয়ে awk-কে অনেক সুবিধাজনক করে।

awk-এর "অদৃশ্য লুপ"

awk প্রোগ্রাম লেখার সময় তোমাকে "প্রতিটা লাইন পড়ো" এমন লুপ লিখতে হয় না। awk নিজেই এটা করে। ভেতরে যা ঘটে:



মানে একটা awk প্রোগ্রাম আসলে এমন কোডের সংক্ষিপ্ত রূপ:

```
BEGIN-এর কাজ
while (আরেকটা লাইন পড়া গেল) {
    লাইনকে field-এ ভাগ করো
    if (pattern1) action1
    if (pattern2) action2
    ...
}
END-এর কাজ
```

awk একটা interpreter

তুমি যখন `awk '{print $1}' file` লেখো:

1. shell কোটেশনের ভেতরের লেখাটা (প্রোগ্রাম) awk-কে argument হিসেবে দেয়
2. awk প্রোগ্রামটা **parse** করে (পড়ে বোঝে), ভেতরে একটা ছোট্ট কোড-রূপে (bytecode/syntax tree) রূপান্তর করে
3. তারপর ফাইলটা লাইন-লাইন **stream** হিসেবে পড়ে আর প্রতিটা লাইনে কোডটা চালায়

যেহেতু awk পুরো ফাইল memory-তে লোড করে না (একবারে একটা record), তাই কয়েক GB-এর লগ ফাইলও এটা অল্প RAM-এ প্রসেস করতে পারে। শুধু তুমি যদি array-তে ডেটা জমাও, তখন memory বাড়বে।

২২.৩ প্রথম awk কমান্ড — মৌলিক syntax

syntax

```
awk [অপশন] 'প্রোগ্রাম' [ফাইল...]
awk [অপশন] -f প্রোগ্রাম-ফাইল.awk [ফাইল...]
কমান্ড | awk 'প্রোগ্রাম'
```

ফাইল না দিলে awk **standard input** থেকে পড়ে, তাই pipe-এর সাথে চমৎকার কাজ করে (অধ্যায় ১৯ দেখো)।

গুরুত্বপূর্ণ অপশন

অপশন	কাজ	উদাহরণ
`-F sep`	field separator ঠিক করা (`FS` সেট করে)	`awk -F: '{print $1}' /etc/passwd`
`-v var=value`	প্রোগ্রাম শুরুর আগে ভেরিয়েবল সেট করা (BEGIN-এর আগেই)	`awk -v n=5 'NR<=n' file`
`-f file`	প্রোগ্রাম কোড ফাইল থেকে পড়া	`awk -f report.awk data.txt`
`--`	অপশন শেষ, এরপর সব ফাইল/argument	`awk -- '{print}' -weird-name`
`--version` (gawk) / `-W version` (mawk)	ভার্সন দেখানো	`awk --version`

শুধু gawk-এর কিছু দরকারি অপশন (পরে বিস্তারিত):

অপশন	কাজ
`-i inplace`	ফাইল সরাসরি এডিট করা (sed -i-এর মতো)
`--lint`	সন্দেহজনক কোডের জন্য সতর্কবার্তা দেখানো
`-p` / `--profile`	প্রোগ্রামের প্রোফাইল ও সুন্দর করে সাজানো কোড `awkprof.out` ফাইলে লেখা
`--posix`	শুধু POSIX স্ট্যান্ডার্ড ফিচার অনুমতি দেওয়া

অপশন	কাজ
`-e 'code'`	কমান্ড লাইনের কোড, `-f`-এর সাথে মেশানো যায়
`-k` / `--csv`	CSV মোড (gawk 5.3 থেকে)

একক কোটেশন কেন?

awk প্রোগ্রাম সবসময় **single quote** (`'...'`) দিয়ে মোড়াও। কারণ `$1`, `$0` এগুলো shell-এর কাছেও বিশেষ অর্থ বহন করে (shell-এর positional parameter)। double quote দিলে shell আগেই `$1`-কে নিজের মান দিয়ে বদলে ফেলবে, awk কখনো `$1` দেখবেই না।

```
$ awk "{print $1}" emp.txt      # ভুল! shell $1 কে খালি করে দেয় → {print }
$ awk '{print $1}' emp.txt      # সঠিক
```

> ⚠️ **সতর্কতা:** double quote-এর ভেতরে awk প্রোগ্রাম লিখলে `$1` shell খেয়ে ফেলে, ফলে awk পুরো লাইন (`print`) ছাপে। এই ভুল নতুনদের মধ্যে খুব সাধারণ।

অনুশীলনের জন্য নমুনা ফাইল

পুরো অধ্যায়ে আমরা এই তিনটা ফাইল ব্যবহার করব। নিজের কম্পিউটারে বানিয়ে নাও:

```
$ mkdir -p ~/awk-lab && cd ~/awk-lab
$ cat > emp.txt << 'EOF'
name dept salary age
Rahim IT 50000 28
Karim HR 35000 35
Sumi IT 62000 31
Nila Sales 40000 26
Jamal Sales 45000 40
EOF
```

```
$ cat > access.log << 'EOF'
192.168.1.10 - - [12/Nov/2026:10:01:22 +0600] "GET /index.html HTTP/1.1" 200 5120
192.168.1.11 - - [12/Nov/2026:10:01:25 +0600] "GET /about.html HTTP/1.1" 200 3200
192.168.1.10 - - [12/Nov/2026:10:02:01 +0600] "POST /login HTTP/1.1" 302 512
10.0.0.5 - - [12/Nov/2026:10:02:44 +0600] "GET /admin HTTP/1.1" 403 210
192.168.1.10 - - [12/Nov/2026:10:03:10 +0600] "GET /missing.png HTTP/1.1" 404 180
10.0.0.5 - - [12/Nov/2026:10:04:00 +0600] "GET /index.html HTTP/1.1" 200 5120
EOF
```

```
$ cat > sales.csv << 'EOF'
date,product,qty,price
2026-11-01,Pen,10,5
2026-11-01,Book,2,250
2026-11-02,Pen,5,5
2026-11-02,Bag,1,1200
```

```
2026-11-03,Book,3,250
EOF
```

প্রথম কমান্ড

```
$ awk '{print}' emp.txt
name dept salary age
Rahim IT 50000 28
Karim HR 35000 35
Sumi IT 62000 31
Nila Sales 40000 26
Jamal Sales 45000 40
```

এখানে কোনো pattern নেই, তাই action প্রতিটা লাইনে চলেছে। `print` একা লিখলে মানে `print $0` (পুরো লাইন)। অর্থাৎ এটা `cat`-এর মতো কাজ করল।

```
$ awk '{print $1, $3}' emp.txt
name salary
Rahim 50000
Karim 35000
Sumi 62000
Nila 40000
Jamal 45000
```

২২.৪ `pattern { action }` — awk-এর হৃদয়

প্রতিটা awk প্রোগ্রাম এক বা একাধিক **rule** (নিয়ম) দিয়ে তৈরি। প্রতিটা rule-এর গঠন:

```
pattern { action }
```

- **pattern**: কোন record-এ কাজ হবে তার শর্ত
- **action**: কী কাজ হবে (`{ }`-এর ভেতরে, একাধিক statement `;` বা newline দিয়ে আলাদা)

চারটা সম্ভাবনা:

লেখার ধরন	মানে	উদাহরণ
`pattern { action }`	শর্ত মিললে action	`'$2=="IT" {print $1}'`
`{ action }`	pattern নেই → প্রতিটা লাইনে action	`'{print $1}'`
`pattern`	action নেই → মিললে পুরো লাইন print	`'$3 > 45000'`
`pattern1, pattern2 { action }`	range: pattern1 থেকে pattern2 পর্যন্ত	`'NR==2, NR==4'`

pattern-এর ধরন

ধরন	উদাহরণ	মানে
regex	`/error/`	`$0` -এ "error" আছে এমন লাইন
তুলনা	`$3 > 45000`	৩য় field ৪৫০০০-এর বেশি
field-এ regex	`$2 ~ /^S/`	২য় field S দিয়ে শুরু
লজিক্যাল	`$2=="IT" && $4<30`	দুটোই সত্য
NR-ভিত্তিক	`NR > 1`	প্রথম লাইন (হেডার) বাদ
বিশেষ	`BEGIN`, `END`	শুরুতে / শেষে
range	`/start/, /stop/`	start থেকে stop পর্যন্ত সব লাইন
যেকোনো এক্সপ্রেশন	`NF`	NF শূন্য না হলে (মানে খালি লাইন না হলে) সত্য

```
$ awk '/Sales/' emp.txt
Nila Sales 40000 26
Jamal Sales 45000 40

$ awk '$2 == "IT" {print $1}' emp.txt
Rahim
Sumi
```

> 📝 **মনে রাখো:** awk-এ সত্য/মিথ্যার নিয়ম C ভাষার মতো: সংখ্যা `0` বা খালি string `""` হলে **মিথ্যা**, বাকি সব **সত্য**। তাই শুধু `NF` লেখা pattern খালি লাইন বাদ দেয় (খালি লাইনে NF = 0)।

```
$ printf 'a b\n\nc d\n' | awk 'NF'
a b
c d
```

২২.৫ বিল্ট-ইন ভেরিয়েবল

awk নিজে কিছু ভেরিয়েবল আপডেট করে বা তাদের মান দেখে আচরণ ঠিক করে।

ভেরিয়েবল	পূর্ণ নাম	মানে	ডিফল্ট
`$0`	—	পুরো বর্তমান record	—
`$1`, `$2` …	—	১ম, ২য়… field	—
`NF`	Number of Fields	বর্তমান record-এ কয়টা field	—
`$NF`	—	শেষ field	—
`NR`	Number of Records	এ পর্যন্ত মোট কয়টা record পড়া হয়েছে (সব ফাইল মিলে)	—

ভেরিয়েবল	পূর্ণ নাম	মানে	ডিফল্ট
`FNR`	File NR	বর্তমান **ফাইলে** কয়টা record	—
`FILENAME`	—	বর্তমান ইনপুট ফাইলের নাম	—
`FS`	Field Separator	ইনপুট field আলাদা করার চিহ্ন/regex	`" "`
`OFS`	Output FS	`print a, b`-তে কমার জায়গায় কী বসবে	`" "`
`RS`	Record Separator	ইনপুট record আলাদা করার চিহ্ন	`"\n"`
`ORS`	Output RS	প্রতিটা `print`-এর শেষে কী বসবে	`"\n"`
`OFMT`	Output Format	`print` দশমিক সংখ্যা কীভাবে দেখাবে	`"%.6g"`
`CONVFMT`	Conversion Format	সংখ্যা → string রূপান্তরের ফরম্যাট	`"%.6g"`
`SUBSEP`	Subscript Separator	বহু-মাত্রিক array-এর key জোড়ার চিহ্ন	`"\034"`
`RSTART`, `RLENGTH`	—	`match()` ফাংশন খুঁজে পাওয়া অংশের শুরু ও দৈর্ঘ্য	—
`ENVIRON`	—	environment variable-এর array	—
`ARGC`, `ARGV`	—	কমান্ড লাইন argument সংখ্যা ও তালিকা	—

`NR`, `NF`, `$NF` দেখা

```
$ awk '{print NR": "$1" ("NF" fields) last="$NF}' emp.txt
1: name (4 fields) last=age
2: Rahim (4 fields) last=28
3: Karim (4 fields) last=35
4: Sumi (4 fields) last=31
5: Nila (4 fields) last=26
6: Jamal (4 fields) last=40
```

এখানে একটা মজার জিনিস: `NR": "$1` — awk-এ দুটো জিনিস পাশাপাশি লিখলেই জোড়া লাগে (**concatenation**), কোনো `+` বা `.` লাগে না।

> 💡 **টিপস:** `$NF` মানে "NF-তম field"। `$` আসলে একটা **অপারেটর**, যেটা ডানের সংখ্যার field দেয়। তাই `$(NF-1)` মানে শেষের আগের field। বন্ধনী জরুরি: `$NF-1` মানে "শেষ field থেকে ১ বিয়োগ"!

```
$ echo "a b c d" | awk '{print $(NF-1)}'
c
$ echo "a b c 10" | awk '{print $NF-1}'
9
```

লাইন নম্বর দিয়ে কাজ

```
$ awk 'NR==3' emp.txt                    # শুধু ৩য় লাইন
Karim HR 35000 35
```

```
$ awk 'NR>1' emp.txt                    # হেডার বাদে সব
$ awk 'NR>=2 && NR<=4' emp.txt          # ২ থেকে ৪ নম্বর লাইন
$ awk 'END {print NR}' emp.txt          # মোট লাইন সংখ্যা (wc -l এর মতো)
6
```

`NR` বনাম `FNR` — একাধিক ফাইল

```
$ awk '{print FILENAME, FNR, NR}' emp.txt sales.csv | tail -2
sales.csv 5 11
sales.csv 6 12
```

`NR` দুই ফাইল মিলিয়ে গুনে চলেছে (১২), কিন্তু `FNR` প্রতিটা নতুন ফাইলে আবার ১ থেকে শুরু হয়। এই পার্থক্যটা দুই ফাইল মেলানোর (join) কৌশলে কাজে লাগে, যেটা আমরা ২২.১৭-এ দেখব।

২২.৬ Field Separator: `-F`, `FS`, `OFS`

`-F` দিয়ে ইনপুট separator

`/etc/passwd` ফাইলে field আলাদা হয় `:` দিয়ে (অধ্যায় ১৫ দেখো):

```
$ awk -F: '{print $1, $7}' /etc/passwd | head -3
root /bin/bash
daemon /usr/sbin/nologin
bin /usr/sbin/nologin
```

CSV ফাইলে `,`:

```
$ awk -F, '{print $2}' sales.csv
product
Pen
Book
Pen
Bag
Book
```

`-F` এর মান একটা **regex** ও হতে পারে (যদি একাধিক অক্ষর হয়):

```
$ echo "a1b22c333d" | awk -F'[0-9]+' '{print $1, $2, $3, $4}'
a b c d

$ echo "key = value" | awk -F' *= *' '{print $2}'
value
```

tab দিয়ে আলাদা ফাইলের জন্য:

```
$ awk -F'\t' '{print $2}' data.tsv
```

> ⚠️ **সতর্কতা:** `-F' '` (একটা space) দিলে সেটা ডিফল্ট আচরণ (একাধিক space/tab একসাথে)। কিন্তু ঠিক একটা space-কেই separator ধরতে চাইলে `-F'[ ]'` লেখো। তখন দুটো পাশাপাশি space-এর মাঝে একটা খালি field হবে।

একটা অক্ষর হলে সেটা আক্ষরিক (literal) ধরা হয়, তাই `-F'|'` বা `-F.` ঠিকঠাক কাজ করে, `|` বা `.` regex অর্থে ধরা হয় না। (ব্যতিক্রম: একটা space, যার বিশেষ অর্থ উপরে বলা হয়েছে।)

`BEGIN`-এ `FS` সেট করা

`-F:` আর `BEGIN {FS=":"}` একই কাজ করে:

```
$ awk 'BEGIN {FS=":"} {print $1}' /etc/passwd
```

> 📝 **মনে রাখো:** `FS` পরিবর্তন **পরের record থেকে** কার্যকর হয়। তাই প্রোগ্রামের মাঝখানে `{FS=":"}` লিখলে প্রথম লাইন পুরনো FS দিয়েই ভাগ হবে। এজন্য সবসময় `BEGIN`-এ বা `-F` দিয়ে সেট করো।

`OFS` — আউটপুট separator

`print $1, $2`-এর কমা আউটপুটে `OFS` হয়ে যায় (ডিফল্ট space):

```
$ awk -F: -v OFS=' | ' '{print $1, $3, $7}' /etc/passwd | head -2
root | 0 | /bin/bash
daemon | 1 | /usr/sbin/nologin
```

কমা না দিলে কী হয়?

```
$ awk '{print $1 $2}' emp.txt | head -2
namedept
RahimIT
```

কমা ছাড়া পাশাপাশি লিখলে **জোড়া লাগে** (কোনো ফাঁকা থাকে না)।

`$1=$1` কৌশল — পুরো লাইন নতুন OFS দিয়ে সাজানো

শুধু `OFS` বদলে `print` (মানে `print $0`) লিখলে কিছুই বদলায় না, কারণ `$0` আগের মতোই আছে। কিন্তু **কোনো field-এ মান assign করলে** awk পুরো `$0` নতুন `OFS` দিয়ে আবার তৈরি করে (rebuild):

```
$ awk 'BEGIN{OFS=","} {$1=$1; print}' emp.txt
name,dept,salary,age
```

```
Rahim,IT,50000,28
Karim,HR,35000,35
Sumi,IT,62000,31
Nila,Sales,40000,26
Jamal,Sales,45000,40
```

এক লাইনে স্পেস-আলাদা ফাইলকে CSV বানিয়ে ফেললাম! একই কৌশলে বাড়তি space পরিষ্কার করা যায়:

```
$ echo "   too    many   spaces  " | awk '{$1=$1; print}'
too many spaces
```

field বদলে দেওয়াও যায়:

```
$ echo "a:b:c" | awk -F: -v OFS='-' '{$2="X"; print}'
a-X-c
```

`RS` ও `ORS` — record separator

`RS` বদলালে "লাইন" ধারণাটাই বদলে যায়:

```
$ echo "a,b,c" | awk 'BEGIN{RS=","} {print NR, $0}'
1 a
2 b
3 c
```

(শেষ record হলো `"c\n"` — `echo`-র দেওয়া newline-টাও তার অংশ, তাই আউটপুটের শেষে একটা বাড়তি ফাঁকা লাইন দেখা যায়। এড়াতে `printf 'a,b,c'` ব্যবহার করো।)

paragraph mode: `RS=""` (খালি string) দিলে এক বা একাধিক **ফাঁকা লাইন** দিয়ে record আলাদা হয়, আর newline-ও field separator হিসেবে কাজ করে। ঠিকানার তালিকার মতো multi-line ডেটার জন্য দারুণ:

```
$ cat contacts.txt
Rahim
01711000000
Dhaka

Sumi
01811000000
Chattogram

$ awk 'BEGIN{RS=""} {print $1 " → " $3}' contacts.txt
Rahim → Dhaka
Sumi → Chattogram
```

`ORS` বদলে সব লাইন এক লাইনে জোড়া:

```
$ awk 'NR>1 {print $1}' ORS=', ' emp.txt; echo
Rahim, Karim, Sumi, Nila, Jamal,
```

> 💡 **টিপস:** উপরে `ORS=', '` ফাইলের নামের আগে লেখা হয়েছে। awk-এ ফাইল তালিকার মধ্যে `var=value` আকারে কিছু লিখলে সেটা ফাইল না, ভেরিয়েবল assignment হিসেবে ধরা হয়, এবং ঠিক সেই জায়গায় (পরের ফাইল পড়ার আগে) কার্যকর হয়। তবে `BEGIN`-এ এটা পাওয়া যায় না; `BEGIN`-এ দরকার হলে `-v` ব্যবহার করো।

২২.৭ `BEGIN` আর `END` ব্লক

- `BEGIN { ... }` চলে **কোনো ইনপুট পড়ার আগে**, একবার। কাজ: শিরোনাম ছাপা, ভেরিয়েবল initialize, `FS` / `OFS` সেট।
- `END { ... }` চলে **সব ইনপুট পড়া শেষে**, একবার। কাজ: যোগফল, গড়, সারসংক্ষেপ ছাপা।

```
$ awk 'BEGIN {print "=== কর্মচারী তালিকা ==="}
       NR>1  {print $1}
       END   {print "=== মোট", NR-1, "জন ==="}' emp.txt
=== কর্মচারী তালিকা ===
Rahim
Karim
Sumi
Nila
Jamal
=== মোট 5 জন ===
```

শুধু `BEGIN` থাকলে awk কোনো ফাইলই পড়ে না, তাই awk-কে **ক্যালকুলেটর** হিসেবে ব্যবহার করা যায়:

```
$ awk 'BEGIN {print 7/2, int(7/2), 7%3, 2^10, sqrt(16)}'
3.5 3 1 1024 4
```

shell-এর `$(( ))` শুধু পূর্ণসংখ্যা (integer) হিসাব পারে, কিন্তু awk দশমিকও পারে, তাই স্ক্রিপ্টে দশমিক হিসাবের জন্য এটা খুব জনপ্রিয়:

```
$ awk 'BEGIN {printf "%.2f\n", 100/3}'
33.33
```

> 📝 **মনে রাখো:** `END` ব্লকে `$0` আর field-গুলোতে সাধারণত **শেষ record**-এর মান থাকে (POSIX এটা নিশ্চিত করে, gawk/mawk মেনে চলে), আর `NR`-এ মোট record সংখ্যা। তাই `awk 'END{print $0}'` = ফাইলের শেষ লাইন (`tail -1`-এর মতো)।

২২.৮ তুলনা, লজিক্যাল অপারেটর ও regex match

তুলনা অপারেটর

অপারেটর	মানে	উদাহরণ
`==`	সমান	`$2 == "IT"`
`!=`	সমান না	`$2 != "HR"`
`<` `<=`	ছোট, ছোট বা সমান	`$4 < 30`
`>` `>=`	বড়, বড় বা সমান	`$3 >= 45000`
`~`	regex মেলে	`$1 ~ /^R/`
`!~`	regex মেলে না	`$1 !~ /^R/`
`in`	array-তে key আছে কিনা	`("IT" in total)`

লজিক্যাল অপারেটর

অপারেটর	মানে	উদাহরণ
`&&`	এবং (AND)	`$2=="IT" && $3>55000`
`||`	অথবা (OR)	`$2=="HR" || $2=="Sales"`
`!`	না (NOT)	`!/^#/`

```
$ awk 'NR>1 && $4<30 {print $1, $4}' emp.txt
Rahim 28
Nila 26

$ awk '$2=="HR" || $2=="Sales" {print $1, $2}' emp.txt
Karim HR
Nila Sales
Jamal Sales
```

হেডারের ফাঁদ — string বনাম number তুলনা

```
$ awk '$3 > 45000' emp.txt
name dept salary age
Rahim IT 50000 28
Sumi IT 62000 31
```

হেডার লাইনটাও এসেছে! কেন? কারণ `$3` = `"salary"`, যেটা সংখ্যা না। awk-এর নিয়ম:

- দুই পাশ সংখ্যা (বা সংখ্যার মতো দেখতে ইনপুট — যাকে বলে **strnum**) হলে → **সংখ্যা হিসেবে** তুলনা
- কোনো একপাশ সংখ্যা না হলে → **string (অভিধান-ক্রম) হিসেবে** তুলনা

`"salary"` আর `"45000"` string হিসেবে তুলনা করলে `s` অক্ষরটা `4`-এর পরে আসে, তাই `"salary" > "45000"` সত্য। সমাধান: হেডার বাদ দাও।

```
$ awk 'NR>1 && $3 > 45000' emp.txt
Rahim IT 50000 28
Sumi IT 62000 31
```

আরেকটা উদাহরণ — প্রোগ্রামের ভেতরে কোটেশনে লেখা `"10"` সবসময় string:

```
$ awk 'BEGIN {print ("10" < "9")}'
1
$ awk 'BEGIN {print (10 < 9)}'
0
```

string হিসেবে `"10"` আগে আসে (কারণ `1` < `9`), সংখ্যা হিসেবে ১০ বড়।

> 💡 **টিপস:** কোনো মানকে জোর করে সংখ্যা বানাতে `+0` যোগ করো (`$3+0`), আর জোর করে string বানাতে খালি string জোড়া দাও (`$3 ""`)।

regex match: `~` আর `!~`

`/regex/` একা লিখলে সেটা `$0 ~ /regex/`-এর সংক্ষিপ্ত রূপ। নির্দিষ্ট field-এ খুঁজতে `~` ব্যবহার করো:

```
$ awk '$1 ~ /^[KS]/' emp.txt          # নাম K বা S দিয়ে শুরু
Karim HR 35000 35
Sumi IT 62000 31

$ awk '$9 ~ /^4/ {print $1, $7, $9}' access.log   # 4xx এরর
10.0.0.5 /admin 403
192.168.1.10 /missing.png 404

$ awk '!/^#/ && NF' /etc/ssh/sshd_config   # কমেন্ট ও খালি লাইন বাদ
```

awk **ERE** (Extended Regular Expression) ব্যবহার করে, তাই `+`, `?`, `|`, `( )`, `{n,m}` ব্যাকস্ল্যাশ ছাড়াই কাজ করে। regex-এর বিস্তারিত অধ্যায় ২০-এ আছে।

regex উদাহরণ	মানে
`/^ERROR/`	লাইনের শুরুতে ERROR
`/\.log$/`	`.log` দিয়ে শেষ
`/fail(ed)?/`	fail বা failed
`/(GET|POST)/`	GET অথবা POST
`/[0-9]{3}/`	পরপর তিনটা অঙ্ক (gawk, আধুনিক mawk)

> ⚠️ **সতর্কতা:** `{n,m}` interval POSIX-এর অংশ, gawk 4.0+ ডিফল্টে সমর্থন করে। কিন্তু Ubuntu/Debian-এর পুরনো mawk (1.3.3) এটা বুঝত না। Ubuntu 24.04 / Debian 12-এর mawk 1.3.4 সাধারণত সমর্থন করে, তবু পোর্টেবল স্ক্রিপ্টে `[0-9][0-9][0-9]` লেখা নিরাপদ।

case-insensitive খোঁজা:

```
$ awk 'tolower($0) ~ /error/' app.log      # সব awk-এ চলে
$ gawk 'BEGIN{IGNORECASE=1} /error/' app.log   # শুধু gawk
```

dynamic regex — ভেরিয়েবল থেকে pattern:

```
$ awk -v pat="^Sa" '$2 ~ pat {print $1}' emp.txt
Nila
Jamal
```

range pattern

`pattern1, pattern2` মানে pattern1 মিলে যে লাইনে, সেখান থেকে pattern2 মেলা পর্যন্ত সব লাইন (দুটোই অন্তর্ভুক্ত):

```
$ awk 'NR==2, NR==4' emp.txt
Rahim IT 50000 28
Karim HR 35000 35
Sumi IT 62000 31

$ awk '/BEGIN CERT/, /END CERT/' bundle.pem      # সার্টিফিকেট ব্লক বের করা
```

২২.৯ ভেরিয়েবল ও গাণিতিক হিসাব

ভেরিয়েবলের নিয়ম

- ঘোষণা (declare) করতে হয় না, প্রথম ব্যবহারেই তৈরি হয়
- নতুন ভেরিয়েবলের মান: সংখ্যা হিসেবে `0`, string হিসেবে `""` — তাই `sum += $3` লেখার আগে `sum=0` লেখা লাগে না
- টাইপ নেই: একই ভেরিয়েবল প্রসঙ্গ বুঝে সংখ্যা বা string
- নামে অক্ষর, অঙ্ক, `_` চলে; অঙ্ক দিয়ে শুরু হতে পারে না; case-sensitive (`Sum` আর `sum` আলাদা)

গাণিতিক অপারেটর

অপারেটর	কাজ	উদাহরণ	ফল
`+ - * /`	যোগ, বিয়োগ, গুণ, ভাগ	`7/2`	`3.5`
`%`	ভাগশেষ	`7%3`	`1`
`^`	ঘাত (power)	`2^10`	`1024`
`++ --`	১ বাড়ানো/কমানো	`count++`	—
`+= -= *= /= %= ^=`	হিসাব করে assign	`sum += $3`	—

> 📝 **মনে রাখো:** awk-এ সব সংখ্যা ভেতরে **floating point (double)**। তাই `7/2` = `3.5`, পূর্ণসংখ্যা লাগলে `int()` ব্যবহার করো। `**` (ঘাত) gawk-এ চলে, কিন্তু পোর্টেবল হলো `^`।

যোগফল ও গড়

```
$ awk 'NR>1 {s += $3} END {print "Total:", s, "Avg:", s/(NR-1)}' emp.txt
Total: 232000 Avg: 46400
```

নিরাপদ গড় (ফাঁকা ফাইলে শূন্য দিয়ে ভাগ এড়াতে):

```
$ awk 'NR>1 {s+=$3; n++} END {if (n>0) print s/n; else print "no data"}' emp.txt
46400
```

সর্বোচ্চ ও সর্বনিম্ন

```
$ awk 'NR>1 { if (max=="" || $3>max) {max=$3; who=$1} } END {print who, max}' emp.txt
Sumi 62000
```

এখানে `max==""` চেক করা হয়েছে যাতে প্রথম মানটা দিয়ে শুরু হয়। (শুধু `max=0` থেকে শুরু করলে সব মান ঋণাত্মক হলে ভুল ফল আসত।)

`-v` দিয়ে shell থেকে মান পাঠানো

```
$ limit=40000
$ awk -v limit="$limit" 'NR>1 && $3 > limit {print $1}' emp.txt
Rahim
Sumi
Jamal
```

> ⚠️ **সতর্কতা:** shell ভেরিয়েবল কখনো awk প্রোগ্রামের কোটেশন ভেঙে ঢোকাবে না (`awk '$3 > '$limit''`)। এটা ভঙ্গুর এবং নিরাপত্তাঝুঁকি (code injection)। সবসময় `-v` ব্যবহার করো।

> environment variable পড়তে `ENVIRON["HOME"]` -ও ব্যবহার করা যায়।

```
$ awk 'BEGIN {print ENVIRON["HOME"]}'
/home/rahim
```

২২.১০ আউটপুট: `print` বনাম `printf`

`print`

- কমা দিয়ে আলাদা আইটেমের মাঝে `OFS` বসায়, শেষে `ORS` (newline) বসায়
- দশমিক সংখ্যা `OFMT` (`%.6g`) অনুযায়ী দেখায়

```
$ awk 'BEGIN {print 1/3}'
0.333333
```

`printf` — নিয়ন্ত্রিত ফরম্যাটিং

C ভাষার `printf`-এর মতো। **নিজে থেকে newline দেয় না**, `\n` লিখতে হয়।

```
printf "ফরম্যাট", মান১, মান২, ...
```

ফরম্যাট	মানে	উদাহরণ	ফল
`%s`	string	`printf "%s", "hi"`	`hi`
`%d` / `%i`	পূর্ণসংখ্যা	`printf "%d", 3.9`	`3`
`%f`	দশমিক	`printf "%.2f", 3.14159`	`3.14`
`%e`	বৈজ্ঞানিক রূপ	`printf "%e", 12345`	`1.234500e+04`
`%g`	%f/%e যেটা ছোট	`printf "%g", 0.5`	`0.5`
`%x` / `%o`	হেক্সাডেসিমাল / অক্টাল	`printf "%x", 255`	`ff`
`%c`	একটা অক্ষর	`printf "%c", 65`	`A`
`%%`	আক্ষরিক `%`	`printf "50%%"`	`50%`
`%10s`	১০ ঘরে ডানে সাজানো	—	`" hello"`
`%-10s`	১০ ঘরে বামে সাজানো	—	`"hello "`
`%05d`	শূন্য দিয়ে ভরা	`printf "%05d", 42`	`00042`
`%+d`	চিহ্ন সহ	`printf "%+d", 5`	`+5`
`%*d`	প্রস্থ argument থেকে	`printf "%*d", 6, 42`	`" 42"`

escape sequence: `\n` (newline), `\t` (tab), `\\` (ব্যাকস্ল্যাশ), `\"` (কোটেশন)।

সুন্দর টেবিল রিপোর্ট

```
$ awk 'BEGIN {printf "%-8s %-6s %10s\n", "Name", "Dept", "Salary"
              printf "%-8s %-6s %10s\n", "----", "----", "------"}
       NR>1  {printf "%-8s %-6s %10.2f\n", $1, $2, $3}' emp.txt
Name     Dept       Salary
----     ----       ------
Rahim    IT       50000.00
Karim    HR       35000.00
Sumi     IT       62000.00
Nila     Sales    40000.00
Jamal    Sales    45000.00
```

> ⚠️ **সতর্কতা:** বাংলা বা অন্য multi-byte (UTF-8) লেখার ক্ষেত্রে `%-10s` প্রস্থ ঠিকমতো না-ও মিলতে পারে। gawk ইউটিএফ-৮ locale-এ অক্ষর ধরে গোনে, কিন্তু mawk **byte** ধরে গোনে। এছাড়া টার্মিনালে বাংলা যুক্তাক্ষরের প্রদর্শন-প্রস্থ আলাদা হয়। টেবিল সাজানোর জন্য ইংরেজি ডেটায় এটা নিখুঁত কাজ করে।

`sprintf` — ফরম্যাট করে ভেরিয়েবলে রাখা

```
$ awk 'BEGIN {s = sprintf("%05.1f%%", 7.25); print s}'
007.2%
```

(rounding C লাইব্রেরির `printf` অনুযায়ী হয়। 7.25 ঠিক মাঝখানে, আর glibc "round half to even" নিয়ম মানে — জোড় অঙ্কের দিকে — তাই ফল `007.2`, `007.3` না। টাকা-পয়সার হিসাবে এই খুঁটিনাটি মাথায় রেখো।)

২২.১১ নিয়ন্ত্রণ কাঠামো: `if/else`, লুপ

awk-এর কন্ট্রোল স্ট্রাকচার প্রায় হুবহু C-এর মতো।

`if / else if / else`

```
$ awk 'NR>1 {
         if ($3 >= 50000)      grade = "High"
         else if ($3 >= 40000) grade = "Mid"
         else                  grade = "Low"
         print $1, grade
       }' emp.txt
Rahim High
Karim Low
Sumi High
Nila Mid
Jamal Mid
```

ternary অপারেটর `? :`

ছোট if/else এক লাইনে:

```
$ awk 'NR>1 {print $1, ($4 >= 30 ? "senior" : "junior")}' emp.txt
Rahim junior
Karim senior
Sumi senior
Nila junior
Jamal senior
```

`for` লুপ

```
$ awk 'BEGIN {for (i=1; i<=5; i++) printf "%d ", i*i; print ""}'
1 4 9 16 25
```

প্রতিটা field-এ লুপ — খুব দরকারি কৌশল:

```
$ echo "10 20 30 40" | awk '{s=0; for (i=1; i<=NF; i++) s+=$i; print "Sum:", s}'
Sum: 100
```

লাইন উল্টো করে ছাপা (field-এর ক্রম উল্টানো):

```
$ echo "one two three" | awk '{for (i=NF; i>0; i--) printf "%s%s", $i, (i>1 ? " " :
"\n")}'
three two one
```

`while` ও `do-while`

```
$ awk 'BEGIN {i=1; while (i<=3) {print "line", i; i++}}'
line 1
line 2
line 3

$ awk 'BEGIN {i=10; do {print i; i++} while (i<5)}'
10
```

`do-while` শর্ত মিথ্যা হলেও অন্তত একবার চলে।

`break`, `continue`, `next`, `nextfile`, `exit`

statement	কাজ
`break`	ভেতরের লুপ থেকে বেরিয়ে যাওয়া
`continue`	লুপের পরের ঘুরে (iteration) চলে যাওয়া
`next`	এই record-এর বাকি rule বাদ দিয়ে **পরের record** পড়া

statement	কাজ
`nextfile`	বর্তমান ফাইলের বাকিটা বাদ দিয়ে **পরের ফাইলে** যাওয়া (gawk, mawk ও POSIX-2024)
`exit [code]`	ইনপুট পড়া বন্ধ; `END` থাকলে সেটা চলবে, তারপর exit code দিয়ে বের হবে

```
$ awk 'NR==1 {next} {print $1}' emp.txt       # হেডার বাদ দেওয়ার আরেক উপায়
Rahim
Karim
Sumi
Nila
Jamal

$ awk '{print} NR==3 {exit}' emp.txt          # প্রথম ৩ লাইন (head -3)
name dept salary age
Rahim IT 50000 28
Karim HR 35000 35
```

> 💡 **টিপস:** বিশাল ফাইলে শুধু প্রথম মিলটা দরকার হলে `exit` ব্যবহার করো। awk তখনই থেমে যাবে, বাকি GB-গুলো পড়বে না।

```
$ awk '/ERROR/ {print; exit}' huge.log
```

২২.১২ associative array — awk-এর গোপন অস্ত্র

array কী?

অন্য ভাষায় array-এর index হয় ০, ১, ২... সংখ্যা। awk-এর array হলো **associative** — index যেকোনো **string** হতে পারে। একে key-value store, dictionary বা hash map-ও বলে।

উপমা: একটা **হাজিরা খাতা**, যেখানে প্রতিটা পাতার মাথায় একজনের নাম লেখা। নতুন কেউ এলে নতুন পাতা খোলো, পুরনো কেউ এলে তার পাতায় দাগ বাড়াও।

```
count["192.168.1.10"] = 3
count["10.0.0.5"]     = 2
count["192.168.1.11"] = 1

  key (string)          value
┌──────────────────┬─────────┐
│ 192.168.1.10     │   3     │
│ 10.0.0.5         │   2     │
│ 192.168.1.11     │   1     │
└──────────────────┴─────────┘
 ভেতরে একটা hash table: key → hash → bucket → value
```

ভেতরে awk একটা **hash table** ব্যবহার করে, তাই key দিয়ে মান খোঁজা খুব দ্রুত (গড়ে সংখ্যা যত বড়ই হোক প্রায় সমান সময়)।

array-এর মৌলিক কাজ

কাজ	syntax
মান রাখা	`arr["key"] = value`
মান বাড়ানো	`arr[key]++` বা `arr[key] += n`
key আছে কিনা চেক	`if (key in arr)`
সব key-তে লুপ	`for (k in arr) print k, arr[k]`
একটা উপাদান মোছা	`delete arr[key]`
পুরো array মোছা	`delete arr` (gawk, mawk, POSIX-2024)
উপাদান সংখ্যা	`length(arr)` (gawk, mawk)
বহু-মাত্রিক	`arr[i, j] = v` (key = i SUBSEP j)

> ⚠️ **সতর্কতা:** `if (arr[key] == "")` দিয়ে চেক করলে awk ভুল করে ওই key **তৈরি** করে ফেলে (শুধু উল্লেখ করলেই উপাদান তৈরি হয়)। আছে কিনা জানতে সবসময় `(key in arr)` ব্যবহার করো, এটা নতুন উপাদান তৈরি করে না।

গণনা (counting)

IP অনুযায়ী রিকোয়েস্ট সংখ্যা:

```
$ awk '{count[$1]++} END {for (ip in count) print count[ip], ip}' access.log | sort -rn
3 192.168.1.10
2 10.0.0.5
1 192.168.1.11
```

HTTP status code অনুযায়ী:

```
$ awk '{c[$9]++} END {for (s in c) print s, c[s]}' access.log
302 1
200 3
404 1
403 1
```

> 📝 **মনে রাখো:** `for (k in arr)` লুপের ক্রম **নির্দিষ্ট না** (hash-এর ক্রমে আসে, awk ভেদে আলাদা)। সাজানো আউটপুট চাইলে `| sort` পাইপ করো, অথবা gawk-এ `PROCINFO["sorted_in"]` ব্যবহার করো (২২.১৬ দেখো)।

গ্রুপ করে যোগফল (GROUP BY)

ডিপার্টমেন্ট অনুযায়ী মোট বেতন — এটা SQL-এর `SELECT dept, SUM(salary) ... GROUP BY dept`-এর মতো:

```
$ awk 'NR>1 {total[$2] += $3; n[$2]++}
       END {for (d in total) printf "%-6s total=%-7d avg=%.0f\n", d, total[d],
total[d]/n[d]}' emp.txt
IT     total=112000  avg=56000
Sales  total=85000   avg=42500
HR     total=35000   avg=35000
```

(ক্রম তোমার মেশিনে আলাদা হতে পারে।)

ডুপ্লিকেট সরানো — বিখ্যাত `!seen[$0]++`

```
$ printf 'apple\nbanana\napple\ncherry\nbanana\n' | awk '!seen[$0]++'
apple
banana
cherry
```

কীভাবে কাজ করে? ধাপে ধাপে:

```
লাইন     seen[$0] এর আগের মান   !মান (pattern)   ছাপা?   পরে seen[$0]
apple    0                      1 (সত্য)          হ্যাঁ      1
banana   0                      1 (সত্য)          হ্যাঁ      1
apple    1                      0 (মিথ্যা)        না       2
cherry   0                      1 (সত্য)          হ্যাঁ      1
banana   1                      0 (মিথ্যা)        না       2
```

`seen[$0]++` আগে পুরনো মান দেয়, তারপর বাড়ায় (post-increment)। প্রথমবার মান ০, `!0` = সত্য, তাই ছাপে। এর বড় সুবিধা: `sort | uniq`-এর মতো ক্রম বদলায় না, আর পাশাপাশি না থাকলেও ডুপ্লিকেট ধরে। নির্দিষ্ট কলাম অনুযায়ী ডুপ্লিকেট: `awk '!seen[$1]++'`।

`split()` দিয়ে array তৈরি

```
$ awk 'BEGIN {n = split("2026-11-12", d, "-"); print n, d[1], d[2], d[3]}'
3 2026 11 12
```

বহু-মাত্রিক array

```
$ awk 'NR>1 {sum[$2, ($4>=30 ? "30+" : "<30")] += $3}
       END {for (k in sum) {split(k, p, SUBSEP); print p[1], p[2], sum[k]}}' emp.txt
```

এটা ডিপার্টমেন্ট × বয়সগ্রুপ অনুযায়ী বেতন যোগ করে। `SUBSEP` চিহ্ন দিয়ে দুই key জোড়া লাগে, তাই `split(k, p, SUBSEP)` দিয়ে আবার আলাদা করা হয়েছে।

২২.১৩ বিল্ট-ইন ফাংশন

string ফাংশন

ফাংশন	কাজ	উদাহরণ	ফল
`length(s)`	দৈর্ঘ্য (s না দিলে `$0`)	`length("Linux")`	`5`
`substr(s, m, n)`	m অবস্থান থেকে n অক্ষর (১ থেকে গোনা)	`substr("Hello", 2, 3)`	`ell`
`index(s, t)`	s-এ t কোথায় (না পেলে 0)	`index("Hello", "l")`	`3`
`split(s, a, sep)`	s ভেঙে array a-তে, অংশ সংখ্যা ফেরত	`split("a:b", x, ":")`	`2`
`sub(re, repl, t)`	t-তে প্রথম মিল বদলানো (t না দিলে `$0`)	`sub(/a/, "X")`	পরিবর্তন সংখ্যা
`gsub(re, repl, t)`	সব মিল বদলানো	`gsub(/a/, "X")`	পরিবর্তন সংখ্যা
`match(s, re)`	regex কোথায় মেলে; `RSTART`, `RLENGTH` সেট করে	`match("ab12", /[0-9]+/)`	`3`
`toupper(s)`	বড় হাতের অক্ষরে	`toupper("abc")`	`ABC`
`tolower(s)`	ছোট হাতের অক্ষরে	`tolower("ABC")`	`abc`
`sprintf(fmt, ...)`	ফরম্যাট করা string ফেরত	`sprintf("%03d", 7)`	`007`

```
$ awk 'BEGIN {
    s = "Hello Linux World"
    print length(s)
    print substr(s, 7, 5)
    print toupper(s)
    n = split(s, a, " "); print n, a[3]
    print index(s, "Linux")
}'
17
Linux
HELLO LINUX WORLD
3 World
7
```

`sub` ও `gsub` বিস্তারিত

```
$ echo "aaa bbb aaa" | awk '{sub(/aaa/, "X"); print}'
X bbb aaa
```

```
$ echo "aaa bbb aaa" | awk '{n = gsub(/aaa/, "X"); print n, $0}'
2 X bbb X
```

- তৃতীয় argument না দিলে `$0` বদলায় (আর field-গুলো আবার ভাগ হয়)
- নির্দিষ্ট field বদলাতে: `gsub(/,/, "", $3)` → ৩য় field থেকে সব কমা মুছে ফেলবে (তখন `$0` নতুন `OFS` দিয়ে rebuild হয়)
- replacement-এ `&` মানে "যা মিলেছে":

```
$ echo "price 500" | awk '{gsub(/[0-9]+/, "[&]"); print}'
price [500]
```

আক্ষরিক `&` চাইলে `"\\&"` লেখো।

> 💡 **টিপস:** `sed 's/old/new/g'` আর `awk '{gsub(/old/,"new"); print}'` একই কাজ করে। কিন্তু awk দিয়ে **শুধু নির্দিষ্ট কলামে** বা **শর্তসাপেক্ষে** বদলানো সহজ: `awk '$2=="IT" {gsub(/0/, "9", $3)} {print}'`।

`match` দিয়ে টুকরো বের করা

```
$ echo "date: 2026-11-12" | awk '{ if (match($0, /[0-9]+-[0-9]+-[0-9]+/)) print RSTART, RLENGTH, substr($0, RSTART, RLENGTH) }'
7 10 2026-11-12
```

`length` দিয়ে লম্বা লাইন খোঁজা

```
$ awk 'length > 80' script.sh                # ৮০ অক্ষরের বেশি লম্বা লাইন
$ awk '{ if (length > max) {max = length; line = $0} } END {print max": "line}' file
```

গাণিতিক ফাংশন

ফাংশন	কাজ
`int(x)`	দশমিক অংশ কেটে ফেলা (০-এর দিকে)
`sqrt(x)`	বর্গমূল
`exp(x)`, `log(x)`	e^x, স্বাভাবিক লগ (ln)
`sin(x)`, `cos(x)`, `atan2(y, x)`	ত্রিকোণমিতি (radian)
`rand()`	০ থেকে ১-এর মধ্যে random সংখ্যা (০ ≤ r < ১)
`srand([seed])`	random-এর seed সেট করা (না দিলে সময় থেকে)

```
$ awk 'BEGIN {srand(); print int(rand()*6) + 1}'    # লুডুর ছক্কা: ১–৬
4
```

```
$ awk 'BEGIN {printf "%.4f\n", atan2(0, -1)}'        # π
3.1416
```

> ⚠️ **সতর্কতা:** `srand()` না ডাকলে প্রতিবার একই random ক্রম আসে। আর `srand()` সেকেন্ড-ভিত্তিক সময় নেয়, তাই এক সেকেন্ডে দুইবার চালালে একই ফল আসতে পারে। নিরাপত্তা-সংক্রান্ত (পাসওয়ার্ড ইত্যাদি) কাজে কখনো `rand()` ব্যবহার করবে না।

অন্যান্য ফাংশন

ফাংশন	কাজ
`system("cmd")`	shell কমান্ড চালানো, exit status ফেরত
`close("file/cmd")`	খোলা ফাইল বা pipe বন্ধ করা
`fflush()`	buffer খালি করে আউটপুট সঙ্গে সঙ্গে পাঠানো
`getline`	পরের লাইন পড়া (২২.১৫ দেখো)

শুধু gawk-এ (খুবই কাজের):

ফাংশন	কাজ
`gensub(re, repl, how, t)`	বদলানো string **ফেরত দেয়** (মূলটা বদলায় না), `\\1` দিয়ে group back-reference
`strftime(fmt, ts)`	timestamp → সুন্দর তারিখ
`systime()`	বর্তমান Unix timestamp
`mktime("YYYY MM DD HH MM SS")`	তারিখ → timestamp
`asort(a)`, `asorti(a)`	মান / key অনুযায়ী array সাজানো
`patsplit(s, a, re)`	regex যেটা **মেলে** সেগুলো দিয়ে ভাগ করা

```
$ echo "2026-11-12" | gawk '{print gensub(/([0-9]+)-([0-9]+)-([0-9]+)/, "\\3/\\2/\\1", "g")}'
12/11/2026

$ gawk 'BEGIN {print strftime("%Y-%m-%d %H:%M:%S", systime())}'
2026-11-12 14:30:05
```

২২.১৪ নিজের ফাংশন লেখা

```
function নাম(প্যারামিটার১, প্যারামিটার২,    লোকাল১, লোকাল২) {
    ...
```

```
    return মান
}
```

awk-এ আলাদা "local variable" ঘোষণার কিছু নেই। প্রচলিত কৌশল হলো: **বাড়তি প্যারামিটার** লোকাল ভেরিয়েবল হিসেবে ব্যবহার করা, আর সেগুলোর আগে কয়েকটা বাড়তি space দিয়ে বোঝানো যে এগুলো আসল argument না।

```
$ awk 'function max(a, b) { return a > b ? a : b }
       function repeat(s, n,    r, i) { r = ""; for (i=0; i<n; i++) r = r s; return r }
       BEGIN { print max(3, 9); print repeat("-", 20) }'
9
--------------------
```

> 📝 **মনে রাখো:**
>
> - ফাংশন ডাকার সময় নাম আর `(` -এর মাঝে **space দেবে না**: `max(3, 9)` ঠিক, `max (3, 9)` ভুল।
> - সাধারণ মান (scalar) **copy** হিসেবে যায় (by value), কিন্তু **array যায় reference হিসেবে** — ফাংশনের ভেতরে array বদলালে বাইরেও বদলায়।
> - লোকাল হিসেবে না রাখা ভেরিয়েবল **global**, পুরো প্রোগ্রামে দেখা যায়।

২২.১৫ আউটপুট redirection, pipe ও `getline`

ফাইলে লেখা

awk-এর ভেতর থেকেই আলাদা ফাইলে লেখা যায়:

```
$ awk 'NR>1 {print > ($2 ".txt")}' emp.txt
$ ls *.txt
emp.txt  HR.txt  IT.txt  Sales.txt
$ cat IT.txt
Rahim IT 50000 28
Sumi IT 62000 31
```

এক কমান্ডে ফাইলটা ডিপার্টমেন্ট অনুযায়ী ভাগ হয়ে গেল!

syntax	মানে
`print > "file"`	ফাইলে লেখা (প্রোগ্রাম চলাকালীন প্রথমবার খুলে খালি করে, তারপর একটানা যোগ করতে থাকে)
`print >> "file"`	ফাইলের শেষে যোগ করা
`print | "cmd"`	আউটপুট একটা কমান্ডে পাঠানো
`"cmd" | getline`	কমান্ডের আউটপুট পড়া

> 📝 **মনে রাখো:** shell-এর `>` প্রতিবার ফাইল খালি করে, কিন্তু awk-এর `>` ফাইলটা **একবার** খোলে (প্রথম লেখায়) আর প্রোগ্রাম শেষ না হওয়া পর্যন্ত খোলাই রাখে। তাই লুপে বারবার `print > "out"` লিখলেও সব লাইন থাকে।

> ⚠️ **সতর্কতা:** ফাইলের নাম এক্সপ্রেশন হলে বন্ধনী দাও: `print > ($2 ".txt")`। বন্ধনী ছাড়া কিছু awk ভুল বোঝে। আর হাজার হাজার আলাদা ফাইলে লিখলে "too many open files" এরর হতে পারে (বিশেষ করে mawk-এ); তখন প্রতিটা লেখার পর `close(fname)` ডাকো আর `>>` ব্যবহার করো।

কমান্ডে pipe

```
$ awk '{print $1}' access.log | sort | uniq -c       # বাইরে pipe
$ awk '{print $1 | "sort -u"}' access.log            # ভেতরে pipe
10.0.0.5
192.168.1.10
192.168.1.11
```

`getline`

`getline` দিয়ে awk-এর স্বাভাবিক লুপের বাইরে নিজে লাইন পড়া যায়:

রূপ	কাজ	কী সেট হয়
`getline`	মূল ইনপুটের পরের record	`$0`, `NF`, `NR`, `FNR`
`getline var`	পরের record var-এ	`var`, `NR`, `FNR`
`getline < "file"`	অন্য ফাইল থেকে পরের লাইন	`$0`, `NF`
`getline var < "file"`	অন্য ফাইল থেকে var-এ	`var`
`"cmd" | getline`	কমান্ডের আউটপুট	`$0`, `NF`, `NR`
`"cmd" | getline var`	কমান্ডের আউটপুট var-এ	`var`, `NR`

`getline` ফেরত দেয়: `1` (সফল), `0` (ফাইল শেষ), `-1` (এরর)।

```
$ awk 'BEGIN { "date +%Y" | getline yr; close("date +%Y"); print "এখন সাল:", yr }'
এখন সাল: 2026

$ awk 'BEGIN { while ((getline line < "/etc/hostname") > 0) print "Host:", line }'
Host: mypc
```

> ⚠️ **সতর্কতা:** `while (getline line < file)` লিখবে না — ফাইল না থাকলে `getline` -1 ফেরত দেয়, যেটা "সত্য", ফলে **অসীম লুপ**! সবসময় `> 0` দিয়ে তুলনা করো।

২২.১৬ awk স্ক্রিপ্ট ফাইল

কোড বড় হলে কমান্ড লাইনে লেখা কষ্টকর। তখন আলাদা `.awk` ফাইলে লেখো।

`-f` দিয়ে চালানো

`report.awk` ফাইল (অধ্যায় ১৩-এর nano বা অধ্যায় ১৪-এর vim দিয়ে বানাও):

```
# report.awk — ডিপার্টমেন্টভিত্তিক বেতন রিপোর্ট
# ব্যবহার: awk -f report.awk emp.txt

BEGIN {
    printf "%-8s %5s %10s %10s\n", "Dept", "Count", "Total", "Average"
    printf "%-8s %5s %10s %10s\n", "----", "-----", "-----", "-------"
}

NR == 1 { next }            # হেডার বাদ

{
    count[$2]++
    total[$2] += $3
    grand     += $3
}

END {
    for (d in count)
        printf "%-8s %5d %10d %10.1f\n", d, count[d], total[d], total[d] / count[d]
    printf "%-8s %5d %10d\n", "ALL", NR - 1, grand
}
```

```
$ awk -f report.awk emp.txt
Dept     Count      Total    Average
----     -----      -----    -------
IT           2     112000    56000.0
Sales        2      85000    42500.0
HR           1      35000    35000.0
ALL          5     232000
```

(ডিপার্টমেন্টের ক্রম মেশিনভেদে আলাদা হতে পারে।)

shebang দিয়ে সরাসরি executable

ফাইলের প্রথম লাইনে shebang দাও (অধ্যায় ৪১-এ shebang বিস্তারিত আছে):

```
#!/usr/bin/awk -f
NR == 1 { next }
{ s += $3 }
END { print "Total salary:", s }
```

```
$ chmod +x total.awk
$ ./total.awk emp.txt
Total salary: 232000
```

kernel shebang লাইন দেখে আসলে চালায় `/usr/bin/awk -f ./total.awk emp.txt`। তাই `-f` অবশ্যই লাগবে।

> 💡 **টিপস:** নির্দিষ্টভাবে gawk দরকার হলে `#!/usr/bin/gawk -f` লেখো। shebang-এ সাধারণত শুধু **একটা** argument নির্ভরযোগ্যভাবে যায়, তাই `#!/usr/bin/awk -F: -f` এমন লেখা কাজ করবে না; FS `BEGIN`-এ সেট করো।

shell স্ক্রিপ্টের ভেতরে awk

```
#!/bin/bash
threshold=${1:-80}
df -P | awk -v t="$threshold" 'NR>1 { gsub(/%/, "", $5); if ($5+0 > t) print "⚠️ ", $6, "পূর্ণ", $5 "%" }'
```

এটা যেসব ডিস্ক পার্টিশন threshold-এর বেশি ভরা, সেগুলো দেখায়। (`df -P` POSIX ফরম্যাটে আউটপুট দেয়, যাতে লম্বা ডিভাইস নামে লাইন ভেঙে না যায়।)

২২.১৭ awk-এর বিভিন্ন রূপ: gawk, mawk, busybox awk

"awk" আসলে একটা ভাষা/স্ট্যান্ডার্ডের নাম। এর বেশ কয়েকটা implementation আছে:

implementation	কে বানিয়েছে / বৈশিষ্ট্য	কোথায় ডিফল্ট
gawk (GNU awk)	GNU প্রজেক্ট; সবচেয়ে বেশি ফিচার, ভালো ডকুমেন্টেশন	Fedora, RHEL 9, Arch, openSUSE
mawk (Mike Brennan's awk)	হালকা ও খুব দ্রুত, ফিচার কম	Ubuntu, Debian (ডিফল্ট `awk`)
onetrue awk / nawk / BWK awk	Kernighan-এর মূল awk-এর বংশধর	macOS, FreeBSD
busybox awk	ছোট্ট, embedded সিস্টেমের জন্য	Alpine Linux, ছোট কন্টেইনার, রাউটার

তোমার `awk` আসলে কোনটা?

Debian/Ubuntu-তে `awk` একটা symlink, যেটা **alternatives** সিস্টেম দিয়ে চলে:

```
$ readlink -f "$(command -v awk)"
/usr/bin/mawk
```

```
$ ls -l /usr/bin/awk /etc/alternatives/awk
lrwxrwxrwx 1 root root 21 ... /usr/bin/awk -> /etc/alternatives/awk
lrwxrwxrwx 1 root root 13 ... /etc/alternatives/awk -> /usr/bin/mawk
```

ভার্সন দেখা:

```
$ gawk --version | head -1
GNU Awk 5.2.1, API 3.2, PMA Avon 8-g1, (GNU MPFR 4.2.1, GNU MP 6.3.0)

$ mawk -W version | head -1
mawk 1.3.4 20240123
```

(তোমার মেশিনে সংখ্যাগুলো আলাদা হতে পারে।)

Ubuntu/Debian-এ gawk ইনস্টল

```
$ sudo apt install gawk
```

ইনস্টল করার পর alternatives সিস্টেম সাধারণত gawk-কে (বেশি priority-র কারণে) `awk` বানিয়ে দেয়। নিজে বেছে নিতে:

```
$ sudo update-alternatives --config awk
```

Fedora/RHEL-এ gawk আগে থেকেই আছে: `/usr/bin/awk → gawk`। প্যাকেজ ম্যানেজমেন্টের বিস্তারিত অধ্যায় ৩০-এ।

কোন ফিচার কোথায় চলে?

ফিচার	POSIX awk	mawk	gawk
`pattern {action}`, `BEGIN/END`, array, printf	✅	✅	✅
`sub`, `gsub`, `match`, `split`, `substr`	✅	✅	✅
`length(array)`, `delete arr` (পুরো)	নতুন স্ট্যান্ডার্ডে	✅	✅
`nextfile`, `fflush()`	নতুন স্ট্যান্ডার্ডে	✅	✅
`gensub()`	❌	❌	✅
`strftime()`, `systime()`	❌	আধুনিক ভার্সনে আছে	✅
`asort()`, `asorti()`, `PROCINFO["sorted_in"]`	❌	❌	✅
`IGNORECASE`	❌	❌	✅
`-i inplace` (ফাইল এডিট)	❌	❌	✅
`FPAT` (CSV-র মতো কোটেড field)	❌	❌	✅
`--csv`	❌	❌	✅ (5.3+)

ফিচার	POSIX awk	mawk	gawk
`BEGINFILE` / `ENDFILE`	❌	❌	✅
সত্যিকারের array of arrays `a[i][j]`	❌	❌	✅
`@include "lib.awk"`	❌	❌	✅
`--lint`, `--profile`, ডিবাগার (`-D`)	❌	❌	✅

> 💡 **টিপস:** অন্যদের সাথে শেয়ার করা বা অনেক সার্ভারে চালানো স্ক্রিপ্টে POSIX ফিচারে থাকো। gawk-এর ফিচার দরকার হলে shebang বা কমান্ডে সরাসরি `gawk` লেখো, যাতে mawk-ওয়ালা মেশিনে চুপচাপ ভুল না হয়ে পরিষ্কার "command not found" দেখায়।

কিছু দরকারি gawk ফিচার

সাজানো array লুপ:

```
$ gawk 'NR>1 {t[$2]+=$3}
        END {PROCINFO["sorted_in"] = "@val_num_desc"; for (d in t) print d, t[d]}'
emp.txt
IT 112000
Sales 85000
HR 35000
```

`sorted_in`-এর মান: `"@ind_str_asc"` (key অনুযায়ী), `"@val_num_desc"` (মান অনুযায়ী বড় থেকে ছোট) ইত্যাদি।

ফাইল সরাসরি এডিট (in-place):

```
$ gawk -i inplace '{gsub(/http:/, "https:")} {print}' config.txt
```

> ⚠️ **সতর্কতা:** `-i inplace` আসল ফাইল বদলে দেয়, কোনো undo নেই। আগে `cp config.txt config.txt.bak` করে নাও, বা প্রথমে `-i` ছাড়া চালিয়ে আউটপুট দেখো। gawk-এ ব্যাকআপ রাখতে `-v inplace::suffix=.bak` দেওয়া যায় (gawk 5.0+; পুরনো ভার্সনে `-v INPLACE_SUFFIX=.bak`)।

কোটেড CSV (কমা ভেতরে থাকলে): সাধারণ `-F,` এখানে ভুল ভাগ করে:

```
$ echo 'Rahim,"Dhaka, Bangladesh",30' | awk -F, '{print $2}'
"Dhaka
$ echo 'Rahim,"Dhaka, Bangladesh",30' | gawk -v FPAT='([^,]*)|("[^"]*")' '{print $2}'
"Dhaka, Bangladesh"
$ echo 'Rahim,"Dhaka, Bangladesh",30' | gawk --csv '{print $2}'    # gawk 5.3+
Dhaka, Bangladesh
```

> 📝 **মনে রাখো:** Ubuntu 24.04 আর Debian 12-এর gawk ভার্সন 5.2.x, তাই সেখানে `--csv` নেই; `FPAT` ব্যবহার করো। Fedora-র নতুন রিলিজে gawk 5.3+ আছে। জটিল CSV (ভেতরে newline, escape করা কোটেশন) হলে Python-এর `csv` মডিউল বা `csvkit` / `mlr` (Miller) টুল বেশি নির্ভরযোগ্য।

গতি

mawk সাধারণত সহজ টেক্সট প্রসেসিংয়ে gawk-এর চেয়ে দ্রুত। বিশাল লগে গতি গুরুত্বপূর্ণ হলে তুলনা করে দেখো:

```
$ time mawk '{s+=$10} END{print s}' big.log
$ time gawk '{s+=$10} END{print s}' big.log
```

আরও একটা কৌশল: locale-এর কারণে regex ধীর হলে `LC_ALL=C awk ...` চালালে অনেক সময় গতি বাড়ে (তবে তখন বাংলার মতো UTF-8 অক্ষর byte হিসেবে গোনা হয়)।

২২.১৮ বাস্তব রেসিপি — লগ বিশ্লেষণ

ওয়েব সার্ভারের (Apache/Nginx) "combined" ফরম্যাটের access log-এ space দিয়ে ভাগ করলে field-গুলো এমন দাঁড়ায় (লগের বিস্তারিত অধ্যায় ২৮, ওয়েব সার্ভার অধ্যায় ৪০-এ):

```
192.168.1.10 - - [12/Nov/2026:10:01:22 +0600] "GET /index.html HTTP/1.1" 200 5120
     $1      $2 $3          $4              $5    $6       $7        $8      $9  $10
    (IP)               (তারিখ-সময়)             (মেথড)    (URL)             (status)
(bytes)
```

> 📝 **মনে রাখো:** combined ফরম্যাটে `$10`-এর পরে referrer আর user-agent থাকে, যেগুলোর ভেতরে space আছে, তাই সেগুলো অনেক field-এ ভেঙে যায়। কিন্তু `$1`–`$10` স্থির থাকে, তাই এগুলো নিশ্চিন্তে ব্যবহার করা যায়। bytes-এর জায়গায় `-` থাকলে awk সেটাকে ০ ধরে।

১. সবচেয়ে বেশি রিকোয়েস্ট করা IP (টপ ১০):

```
$ awk '{c[$1]++} END {for (ip in c) print c[ip], ip}' access.log | sort -rn | head -10
3 192.168.1.10
2 10.0.0.5
1 192.168.1.11
```

২. সবচেয়ে জনপ্রিয় URL:

```
$ awk '{u[$7]++} END {for (k in u) print u[k], k}' access.log | sort -rn | head -3
2 /index.html
```

```
1 /missing.png
1 /login
```

(সমান সংখ্যার লাইনগুলোর ক্রম আলাদা হতে পারে।)

৩. status code-এর শতাংশ:

```
$ awk '{c[$9]++} END {for (s in c) printf "%s %d (%.1f%%)\n", s, c[s], c[s]*100/NR}'
access.log | sort
200 3 (50.0%)
302 1 (16.7%)
403 1 (16.7%)
404 1 (16.7%)
```

৪. কোন URL-এ 404 হচ্ছে (ভাঙা লিংক খোঁজা):

```
$ awk '$9 == 404 {print $7}' access.log | sort | uniq -c | sort -rn
      1 /missing.png
```

৫. মোট ডেটা ট্রান্সফার:

```
$ awk '{b += $10} END {printf "Total: %d bytes (%.2f KB)\n", b, b/1024}' access.log
Total: 14342 bytes (14.01 KB)
```

৬. IP অনুযায়ী ব্যান্ডউইথ:

```
$ awk '{b[$1] += $10} END {for (ip in b) printf "%-15s %8d\n", ip, b[ip]}' access.log |
sort -k2 -rn
192.168.1.10        5812
10.0.0.5            5330
192.168.1.11        3200
```

৭. প্রতি ঘণ্টায় কয়টা রিকোয়েস্ট (ট্রাফিক প্যাটার্ন):

`$4` = `[12/Nov/2026:10:01:22`; `:` দিয়ে ভাঙলে ২য় অংশ = ঘণ্টা।

```
$ awk '{split($4, t, ":"); h[t[2]]++} END {for (x in h) print x":00", h[x]}' access.log
| sort
10:00 6
```

৮. HTTP মেথড গণনা (কোটেশন চিহ্ন সরিয়ে):

```
$ awk '{m = $6; gsub(/"/, "", m); c[m]++} END {for (k in c) print k, c[k]}' access.log
| sort
GET 5
POST 1
```

৯. সন্দেহজনক IP — নির্দিষ্ট সংখ্যার বেশি 4xx এরর:

```
$ awk '$9 ~ /^4/ {e[$1]++} END {for (ip in e) if (e[ip] >= 10) print ip, e[ip]}'
access.log
```

১০. auth log থেকে ব্যর্থ SSH লগইনের IP (Debian/Ubuntu-তে `/var/log/auth.log`, RHEL/Fedora-তে `/var/log/secure`; journal-ভিত্তিক সিস্টেমে `journalctl -u ssh` বা `-u sshd`):

```
$ sudo awk '/Failed password/ {for (i=1; i<=NF; i++) if ($i == "from") c[$(i+1)]++}
            END {for (ip in c) print c[ip], ip}' /var/log/auth.log | sort -rn | head
```

এখানে field-এর অবস্থান নির্দিষ্ট না (invalid user হলে শব্দ বেশি থাকে), তাই "from" শব্দ খুঁজে তার পরের field নেওয়া হয়েছে — এটা একটা দারুণ কৌশল। SSH নিরাপত্তা নিয়ে অধ্যায় ৩৮ ও অধ্যায় ৪৫ দেখো।

২২.১৯ বাস্তব রেসিপি — CSV ও রিপোর্ট

১. পণ্যভিত্তিক বিক্রি (পরিমাণ ও আয়):

```
$ awk -F, 'NR>1 {t[$2] += $3*$4; q[$2] += $3}
           END {printf "%-6s %4s %8s\n", "Item", "Qty", "Revenue"
                for (p in t) printf "%-6s %4d %8d\n", p, q[p], t[p]}' sales.csv
Item    Qty  Revenue
Pen      15       75
Bag       1     1200
Book      5     1250
```

২. তারিখভিত্তিক মোট আয়:

```
$ awk -F, 'NR>1 {r[$1] += $3*$4} END {for (d in r) print d, r[d]}' sales.csv | sort
2026-11-01 550
2026-11-02 1225
2026-11-03 750
```

৩. নতুন কলাম যোগ করা (total = qty × price):

```
$ awk -F, -v OFS=, 'NR==1 {print $0, "total"; next} {print $0, $3*$4}' sales.csv
date,product,qty,price,total
2026-11-01,Pen,10,5,50
2026-11-01,Book,2,250,500
2026-11-02,Pen,5,5,25
2026-11-02,Bag,1,1200,1200
2026-11-03,Book,3,250,750
```

৪. কলামের ক্রম বদলানো ও কিছু কলাম বাদ দেওয়া:

```
$ awk -F, -v OFS=, '{print $2, $1, $3}' sales.csv
```

৫. CSV → TSV, বা TSV → CSV:

```
$ awk 'BEGIN {FS=","; OFS="\t"} {$1=$1; print}' sales.csv > sales.tsv
```

৬. দুই ফাইল মেলানো (SQL JOIN-এর মতো) — বিখ্যাত `NR==FNR` কৌশল:

```
$ cat names.txt
101 Rahim
102 Sumi
$ cat pay.txt
101 50000
102 62000
103 1000

$ awk 'NR==FNR {name[$1] = $2; next} ($1 in name) {print $1, name[$1], $2}' names.txt
pay.txt
101 Rahim 50000
102 Sumi 62000
```

কীভাবে কাজ করে:

```
প্রথম ফাইল (names.txt) পড়ার সময়:  NR == FNR  (দুটোই 1, 2)
      → name[id] = নাম  তে জমা, next দিয়ে পরের লাইনে
দ্বিতীয় ফাইল (pay.txt) পড়ার সময়: NR (3,4,5) != FNR (1,2,3)
      → প্রথম rule বাদ; id যদি name-এ থাকে তবে ছাপো
```

> ⚠️ **সতর্কতা:** প্রথম ফাইলটা খালি হলে `NR==FNR` দ্বিতীয় ফাইলেও সত্য থাকবে আর ফল ভুল হবে। নিশ্চিত হতে চাইলে `FILENAME == ARGV[1]` দিয়ে চেক করো।

৭. Transpose (সারি ↔ কলাম):

```
$ printf '1 2 3\n4 5 6\n' | awk '{for (i=1; i<=NF; i++) a[NR,i] = $i; if (NF > mx) mx =
NF}
     END {for (i=1; i<=mx; i++) {r = ""; for (j=1; j<=NR; j++) r = r (j>1 ? " " : "")
a[j,i]; print r}}'
1 4
2 5
3 6
```

২২.২০ বাস্তব রেসিপি — সিস্টেম অ্যাডমিন one-liner

কাজ	কমান্ড
সাধারণ ইউজারদের নাম ও home	`awk -F: '$3>=1000 && $3<65534 {print $1, $6}' /etc/passwd`

কাজ	কমান্ড
UID 0 (root ক্ষমতার) অ্যাকাউন্ট	`awk -F: '$3==0 {print $1}' /etc/passwd`
কারা bash ব্যবহার করে	`awk -F: '$NF ~ /bash$/ {print $1}' /etc/passwd`
মেমোরি ব্যবহার শতাংশে	`free -m | awk '/^Mem:/ {printf "%.1f%%\n", $3*100/$2}'`
প্রোগ্রাম অনুযায়ী মোট RAM	`ps -eo rss,comm --no-headers | awk '{m[$2]+=$1} END {for (p in m) printf "%8.1f MB %s\n", m[p]/1024, p}' | sort -rn | head`
৮০%-এর বেশি ভরা ডিস্ক	`df -P | awk 'NR>1 && $5+0 > 80 {print $6, $5}'`
বড় ফাইল (১০০MB+)	`ls -l | awk '$5 > 100*1024*1024 {print $5, $NF}'`
ডিরেক্টরির মোট সাইজ (ls থেকে)	`ls -l | awk 'NR>1 {s+=$5} END {print s}'`
listening TCP পোর্ট	`ss -tln | awk 'NR>1 {print $4}'`
কোনো প্রসেসের PID	`ps aux | awk '$11 ~ /nginx/ {print $2}'`
কনফিগের কমেন্ট/খালি লাইন বাদ	`awk '!/^[[:space:]]*#/ && NF' file.conf`
ডুপ্লিকেট লাইন সরানো (ক্রম ঠিক রেখে)	`awk '!seen[$0]++' file`
নির্দিষ্ট লাইন (৫ থেকে ১০)	`awk 'NR>=5 && NR<=10' file`
মোট লাইন / শব্দ সংখ্যা	`awk '{w+=NF} END {print NR, w}' file`
ফাইল উল্টো ক্রমে (tac)	`awk '{a[NR]=$0} END {for (i=NR; i>0; i--) print a[i]}' file`
প্রতি ৩য় লাইন	`awk 'NR%3==0' file`
শব্দের ফ্রিকোয়েন্সি	`awk '{for (i=1;i<=NF;i++) w[tolower($i)]++} END {for (k in w) print w[k], k}' file | sort -rn`
কলামের যোগফল	`awk '{s+=$1} END {print s}' numbers.txt`
সংখ্যার গড়	`seq 1 10 | awk '{s+=$1} END {print s/NR}'` → `5.5`

💡 **টিপস:** `ps aux | grep nginx | awk '{print $2}'` লেখার দরকার নেই — `grep` নিজেও তালিকায় চলে আসে। `awk '/nginx/ && !/awk/ {print $2}'` লিখতে পারো, তবে সবচেয়ে ভালো সমাধান হলো `pgrep nginx` (অধ্যায় ২৪)। সঠিক টুল চেনাও এক্সপার্ট হওয়ার অংশ।

⚠️ **সতর্কতা:** `ls -l`-এর আউটপুট স্ক্রিপ্টে পার্স করা ঝুঁকিপূর্ণ (স্পেসওয়ালা ফাইলনাম, locale-ভেদে তারিখের ফরম্যাট বদলায়)। বাস্তব কাজে `find -printf` বা `stat` বেশি নির্ভরযোগ্য (অধ্যায় ১২)। উপরের টেবিলে এটা শুধু শেখার জন্য।

কখন awk না?

- **শুধু লাইন খোঁজা**: `grep` বেশি দ্রুত ও সহজ
- **শুধু একটা কলাম, নির্দিষ্ট এক-অক্ষরের separator**: `cut` যথেষ্ট (অধ্যায় ২৩)
- **সাধারণ find & replace**: `sed`
- **JSON**: `jq` ব্যবহার করো; awk দিয়ে JSON পার্স করা ভুলের রেসিপি
- **জটিল লজিক, শত লাইনের কোড, ডেটাবেস/নেটওয়ার্ক**: Python বা অন্য পূর্ণ ভাষা

২২.২১ সাধারণ ভুল ও ডিবাগিং

ভুল	সমস্যা	সমাধান
`awk "{print $1}"`	shell `$1` খেয়ে ফেলে	single quote: `'{print $1}'`
`awk '{print $1,$2}' file.csv`	CSV-তে FS দেওয়া হয়নি	`-F,` দাও
হেডার লাইনও হিসাবে ঢুকছে	string তুলনা/যোগে গোলমাল	`NR>1` বা `NR==1{next}`
`$NF-1`	শেষ field থেকে ১ বিয়োগ	`$(NF-1)`
`{FS=","} {print $1}`	প্রথম লাইন ভুল ভাগ	`BEGIN{FS=","}` বা `-F,`
`OFS` সেট করেও আউটপুট বদলায় না	`$0` rebuild হয়নি	`$1=$1`
`if (a[k] == "")`	ফাঁকা key তৈরি হয়ে যায়	`if (!(k in a))`
`func (x)`	নাম আর `(` মাঝে space	`func(x)`
`while (getline l < f)`	ফাইল না থাকলে অসীম লুপ	`while ((getline l < f) > 0)`
Windows ফাইলে শেষ field-এ অদ্ভুত আচরণ	লাইন শেষে `\r` (CRLF)	`{sub(/\r$/, "")}` আগে দাও, বা `dos2unix`
gawk ফাংশন mawk-এ এরর	`function never defined`	`gawk` সরাসরি ডাকো

ডিবাগিং কৌশল:

```
$ awk '{print NF": "$0}' file | head              # প্রতি লাইনে কয়টা field ভাগ হলো দেখো
$ awk -F, '{for (i=1;i<=NF;i++) print i" = ["$i"]"}' file | head    # প্রতিটা field বন্ধনীতে
$ cat -A file | head                               # লুকানো \r, tab (^I) দেখো
$ gawk --lint -f script.awk data                   # সন্দেহজনক কোডে সতর্কতা
$ gawk --pretty-print -f script.awk                # কোড সুন্দর করে awkprof.out-এ লেখে
```

হাতে-কলমে অনুশীলন

২২.৩-এ বানানো `~/awk-lab` ডিরেক্টরির `emp.txt`, `access.log`, `sales.csv` ব্যবহার করো।

ল্যাব ১: field চেনা

```
$ cd ~/awk-lab
$ awk '{print NF, $1, $NF}' emp.txt
$ awk '{print $(NF-1)}' emp.txt
$ awk -F: '{print NR, $1}' /etc/passwd | head -5
```

চেষ্টা করো: `/etc/passwd` থেকে প্রতিটা ইউজারের নাম আর shell `" uses "` দিয়ে জুড়ে দেখাও (যেমন `root uses /bin/bash`)।

ল্যাব ২: শর্ত দিয়ে ফিল্টার

1. শুধু IT ডিপার্টমেন্টের কর্মচারী দেখাও
2. যাদের বয়স ৩০ বা তার বেশি এবং বেতন ৪০০০০-এর বেশি, তাদের নাম দেখাও
3. যাদের নাম স্বরবর্ণ (A, E, I, O, U) দিয়ে শুরু না, তাদের দেখাও (হেডার বাদে)

```
$ awk '$2 == "IT"' emp.txt
$ awk 'NR>1 && $4 >= 30 && $3 > 40000 {print $1}' emp.txt
$ awk 'NR>1 && $1 !~ /^[AEIOU]/' emp.txt
```

ল্যাব ৩: হিসাব

1. মোট বেতন, গড় বেতন, সর্বোচ্চ ও সর্বনিম্ন বেতন একসাথে বের করো
2. প্রত্যেকের বেতনে ১০% বাড়িয়ে নতুন বেতন দেখাও

```
$ awk 'NR>1 {s+=$3; if (min=="" || $3<min) min=$3; if ($3>max) max=$3}
       END {printf "Total=%d Avg=%.2f Max=%d Min=%d\n", s, s/(NR-1), max, min}' emp.txt
Total=232000 Avg=46400.00 Max=62000 Min=35000

$ awk 'NR>1 {printf "%-6s %d -> %.0f\n", $1, $3, $3*1.10}' emp.txt
```

ল্যাব ৪: array দিয়ে গণনা

1. প্রতিটা ডিপার্টমেন্টে কতজন আছে গোনো
2. `access.log`-এ প্রতিটা IP কতবার এসেছে, বেশি থেকে কম ক্রমে সাজাও
3. `access.log` থেকে ইউনিক IP-গুলো (প্রথম আসার ক্রমে) দেখাও

```
$ awk 'NR>1 {c[$2]++} END {for (d in c) print d, c[d]}' emp.txt
$ awk '{c[$1]++} END {for (i in c) print c[i], i}' access.log | sort -rn
$ awk '!seen[$1]++ {print $1}' access.log
192.168.1.10
192.168.1.11
10.0.0.5
```

ল্যাব ৫: স্ক্রিপ্ট ফাইল

`salary-report.awk` নামে একটা স্ক্রিপ্ট লেখো, যেটা:

- `BEGIN`-এ একটা শিরোনাম ও কলাম-হেডার ছাপবে
- হেডার লাইন বাদ দেবে
- প্রতিটা কর্মচারীর নাম, ডিপার্টমেন্ট, বেতন `printf` দিয়ে সাজিয়ে দেখাবে, আর বেতন ৫০০০০ বা বেশি হলে পাশে `*` দেবে
- `END`-এ মোট কর্মচারী আর মোট বেতন দেখাবে

তারপর shebang (`#!/usr/bin/awk -f`) যোগ করে `chmod +x` দিয়ে সরাসরি চালাও: `./salary-report.awk emp.txt`।

ল্যাব ৬: বাস্তব সিস্টেম

```
$ df -hP | awk 'NR>1 {print $6, "→", $5}'                          # কোন পার্টিশন কত ভরা
$ ps -eo user --no-headers | awk '{c[$1]++} END {for (u in c) print c[u], u}' | sort -
rn   # কোন ইউজারের কত প্রসেস
$ awk -F: '{print $7}' /etc/passwd | sort | uniq -c | sort -rn  # কোন shell কতজন ব্যবহার
করে
```

ল্যাব ৭: চ্যালেঞ্জ

1. `sales.csv` থেকে সবচেয়ে বেশি আয়ের পণ্যটা (নাম ও আয়) একটা লাইনে দেখাও — `sort` ব্যবহার না করে, শুধু awk দিয়ে।
2. `emp.txt`-কে CSV-তে রূপান্তর করে `emp.csv` ফাইলে সেভ করো, তারপর সেই CSV থেকে আবার ডিপার্টমেন্টভিত্তিক গড় বেতন বের করো।
3. একটা awk কমান্ড লেখো যেটা `emp.txt`-কে ডিপার্টমেন্ট অনুযায়ী আলাদা ফাইলে (`IT.txt`, `HR.txt` ...) ভাগ করবে, এবং প্রতিটা ফাইলের প্রথম লাইনে হেডার রাখবে।

▼ **চ্যালেঞ্জের একটা সম্ভাব্য সমাধান**

```
# ১
$ awk -F, 'NR>1 {t[$2] += $3*$4}
           END {for (p in t) if (t[p] > best) {best = t[p]; name = p}; print name,
best}' sales.csv
Book 1250

# ২
$ awk -v OFS=, '{$1=$1; print}' emp.txt > emp.csv
$ awk -F, 'NR>1 {s[$2]+=$3; n[$2]++} END {for (d in s) print d, s[d]/n[d]}' emp.csv

# ৩
$ awk 'NR==1 {h = $0; next}
       { f = $2 ".txt"; if (!(f in done)) {print h > f; done[f] = 1}; print > f }'
emp.txt
```

সারসংক্ষেপ

awk = record-ভিত্তিক টেক্সট প্রসেসিং ভাষা। প্রতিটা লাইন (record) পড়ে, field-এ ভাগ করে, প্রতিটা `pattern { action }` চেক করে। `BEGIN` আগে, `END` পরে একবার চলে।

দ্রুত রেফারেন্স

জিনিস	মানে / উদাহরণ
`awk '{print $1}' f`	১ম কলাম
`awk -F: '...'`	`:` দিয়ে field ভাগ
`awk -F, -v OFS=, '...'`	CSV ইন/আউট
`awk -v x=5 '...'`	shell থেকে মান পাঠানো
`awk -f prog.awk f`	ফাইল থেকে প্রোগ্রাম
`$0`, `$1`, `$NF`, `$(NF-1)`	পুরো লাইন, ১ম, শেষ, শেষের আগের field
`NR`, `FNR`, `NF`	মোট লাইন নম্বর, ফাইলের লাইন নম্বর, field সংখ্যা
`FS`, `OFS`, `RS`, `ORS`	ইনপুট/আউটপুট field ও record separator
`FILENAME`, `ENVIRON`, `ARGV`	ফাইলের নাম, environment, argument
`BEGIN {}` / `END {}`	শুরুর আগে / শেষে
`/re/`, `$2 ~ /re/`, `$2 !~ /re/`	regex মেলানো
`NR>1`	হেডার বাদ
`NR==2, NR==4`	range
`{s+=$3} END {print s, s/NR}`	যোগফল ও গড়
`{c[$1]++} END {for (k in c) print k, c[k]}`	গণনা (group by)
`!seen[$0]++`	ডুপ্লিকেট সরানো
`NR==FNR {a[$1]=$2; next}`	দুই ফাইল join
`$1=$1`	নতুন OFS দিয়ে লাইন rebuild
`printf "%-10s %5.2f\n", a, b`	ফরম্যাট করা আউটপুট
`length`, `substr`, `index`, `split`	string ফাংশন
`sub`, `gsub`, `match`	regex দিয়ে বদলানো/খোঁজা
`toupper`, `tolower`, `sprintf`	case বদলানো, ফরম্যাট string
`int`, `sqrt`, `rand`, `srand`	গাণিতিক
`next`, `exit`, `break`, `continue`	প্রবাহ নিয়ন্ত্রণ

জিনিস	মানে / উদাহরণ
`print > "f"`, `>> "f"`, `| "cmd"`	আউটপুট redirection
`"cmd" | getline v`	কমান্ডের আউটপুট পড়া
`gawk -i inplace`, `gensub`, `strftime`, `PROCINFO["sorted_in"]`	শুধু gawk

শর্টকাট ও দ্রুত কৌশল

awk-এর নিজস্ব কোনো কীবোর্ড শর্টকাট নেই (এটা interactive প্রোগ্রাম না), কিন্তু টার্মিনালে awk কমান্ড লেখার সময় এগুলো খুব কাজে লাগে (অধ্যায় ৬-এ বিস্তারিত):

শর্টকাট / কৌশল	কাজ
`Ctrl + X` তারপর `Ctrl + E`	লম্বা awk কমান্ড `$EDITOR`-এ খুলে আরামে এডিট করা (bash)
`Ctrl + R`	আগে লেখা awk কমান্ড history থেকে খোঁজা
`Alt + .`	আগের কমান্ডের শেষ argument (যেমন ফাইলের নাম) বসানো
`Ctrl + C`	ভুল/অসীম লুপে আটকে যাওয়া awk থামানো
`Ctrl + D`	ফাইল ছাড়া চালানো awk-কে keyboard ইনপুট শেষ জানানো (EOF), তখন `END` চলে
`!awk`	শেষ যে কমান্ড `awk` দিয়ে শুরু, সেটা আবার চালানো
`^old^new`	আগের কমান্ডে `old` বদলে `new` দিয়ে চালানো (যেমন `^IT^HR`)
`alias`	প্রিয় one-liner-এর alias: `alias dedup="awk '!seen[\$0]++'"` (অধ্যায় ৪৪)

awk কোডের ভেতরে "শর্টকাট" (সংক্ষিপ্ত রূপ):

সংক্ষিপ্ত	পূর্ণ রূপ
`'NR>1'`	`'NR>1 {print $0}'`
`'/re/'`	`'$0 ~ /re/ {print $0}'`
`'{print}'`	`'{print $0}'`
`'1'`	সব লাইন ছাপো (সবসময় সত্য pattern)
`'NF'`	খালি লাইন বাদ
`'END{print NR}'`	`wc -l`
`'{$1=$1}1'`	বাড়তি space পরিষ্কার করে ছাপো

নিজেকে যাচাই করো

1. awk-এ "record" আর "field" কী? কোন দুটো ভেরিয়েবল এদের separator ঠিক করে?

2. `awk "{print $1}" file` কেন পুরো লাইন ছাপে?
3. `$NF` আর `$(NF-1)`-এর পার্থক্য কী? `$NF-1` লিখলে কী হয়?
4. `NR` আর `FNR` কখন আলাদা হয়? `NR==FNR` কৌশল কী কাজে লাগে?
5. `awk 'BEGIN{OFS=","} {print}' file` কেন separator বদলায় না? কীভাবে ঠিক করবে?
6. `awk '$3 > 45000' emp.txt` হেডার লাইনও দেখায় কেন?
7. `!seen[$0]++` কীভাবে ডুপ্লিকেট সরায়? `sort -u`-এর সাথে পার্থক্য কী?
8. `sub` আর `gsub`-এর পার্থক্য কী? দুটো কী ফেরত দেয়?
9. Ubuntu 24.04-এ `awk` কমান্ড ডিফল্টভাবে কোন implementation? `gensub()` না চললে কী করবে?
10. একটা কমান্ড লেখো যেটা `access.log` থেকে প্রতিটা status code কতবার এসেছে দেখায়।

▼ **উত্তর দেখো**

1. **record** = ইনপুটের একক (ডিফল্ট একটা লাইন), **field** = record-এর ভেতরের অংশ (ডিফল্ট space/tab-আলাদা শব্দ)। `RS` record separator, `FS` field separator ঠিক করে।
2. double quote-এর ভেতরে shell `$1`-কে নিজের positional parameter (সাধারণত খালি) দিয়ে বদলে দেয়, তাই awk পায় `{print }` — যেটা পুরো লাইন ছাপে। single quote ব্যবহার করতে হবে।
3. `$NF` = শেষ field, `$(NF-1)` = শেষের আগের field। `$NF-1` মানে "শেষ field-এর মান থেকে ১ বিয়োগ" (যেমন শেষ field 10 হলে 9)।
4. একাধিক ফাইল দিলে: `NR` সব ফাইল মিলে গুনতে থাকে, `FNR` প্রতি ফাইলে ১ থেকে শুরু হয়। প্রথম ফাইল পড়ার সময় শুধু `NR==FNR` সত্য, তাই এটা দিয়ে প্রথম ফাইলের ডেটা array-তে জমিয়ে দ্বিতীয় ফাইলের সাথে মেলানো (join/lookup) যায়।
5. `print` = `print $0`, আর `$0` আগের মতোই অপরিবর্তিত থাকে; কোনো field পরিবর্তন না হলে awk `$0` নতুন `OFS` দিয়ে rebuild করে না। ঠিক করতে: `{$1=$1; print}`।
6. হেডারে `$3` = `"salary"`, যেটা সংখ্যা না, তাই awk string হিসেবে তুলনা করে; `"salary" > "45000"` সত্য (s অক্ষর 4-এর পরে)। সমাধান: `NR>1 && $3 > 45000`।
7. `seen[$0]++` আগে পুরনো মান (প্রথমবার 0) দেয়, তারপর বাড়ায়। `!0` সত্য, তাই প্রথম দেখা লাইন ছাপে; পরের বার মান ≥১, `!` করলে মিথ্যা, তাই ছাপে না। `sort -u` ক্রম বদলে সাজিয়ে দেয়, কিন্তু awk মূল ক্রম ঠিক রাখে এবং পুরো ইনপুট sort করার দরকার পড়ে না (তবে প্রতিটা ইউনিক লাইন memory-তে রাখে)।
8. `sub` শুধু প্রথম মিল বদলায়, `gsub` সব মিল বদলায় (global)। দুটোই কতগুলো পরিবর্তন হয়েছে সেই সংখ্যা ফেরত দেয় (`sub`-এর ক্ষেত্রে 0 বা 1)। target না দিলে `$0` বদলায়।
9. **mawk** (Debian/Ubuntu-তে alternatives দিয়ে)। `gensub` শুধু gawk-এ আছে: `sudo apt install gawk` করে `gawk` নামে সরাসরি চালাও, অথবা `sub` / `gsub` / `match` / `substr` দিয়ে POSIX উপায়ে লেখো।
10. `awk '{c[$9]++} END {for (s in c) print s, c[s]}' access.log` (ক্রম চাইলে শেষে `| sort` যোগ করো)।

অধ্যায় ২৩: আরও টেক্সট টুল — cut, sort, uniq, tr, xargs ও অন্যান্য

এই অধ্যায়ে যা শিখবে

- Unix-এর "ছোট টুল, বড় কাজ" দর্শন, আর **filter** (ফিল্টার) মানে কী
- `cut` দিয়ে লাইনের নির্দিষ্ট কলাম বা অক্ষর কেটে বের করা
- `sort` দিয়ে সাজানো: সংখ্যা, উল্টো ক্রম, নির্দিষ্ট কলাম, মানুষের-পড়ার-মতো সাইজ (`1K`, `2G`), ভার্সন নম্বর
- `uniq` দিয়ে ডুপ্লিকেট সরানো ও গোনা, এবং **কেন আগে `sort` করতেই হয়**
- `wc` দিয়ে লাইন, শব্দ, বাইট গোনা
- `tr` দিয়ে অক্ষর বদলানো, মুছে ফেলা, পরপর একই অক্ষর চেপে এক করা (squeeze)
- `paste`, `join`, `comm` দিয়ে একাধিক ফাইল জোড়া লাগানো ও তুলনা করা
- `diff -u` দিয়ে পার্থক্য দেখা এবং `patch` দিয়ে সেই পার্থক্য প্রয়োগ করা; `cmp` দিয়ে বাইনারি তুলনা
- `xargs` দিয়ে এক কমান্ডের আউটপুটকে অন্য কমান্ডের **argument** বানানো (`-n`, `-I`, `-0`, `-P`)
- `tee`, `rev`, `shuf`, `fold`, `fmt`, `expand` / `unexpand`, `split`, `csplit`, `nl`, `column`
- `jq` দিয়ে JSON ডেটা পড়া ও ফিল্টার করা
- পাইপলাইন জুড়ে বাস্তব সমস্যার সমাধান — যেমন ওয়েব সার্ভার লগ থেকে **top 10 IP** বের করা

২৩.১ শুরুর আগে: Unix দর্শন ও "ফিল্টার"

২৩.১.১ এক কাজ, ভালোভাবে

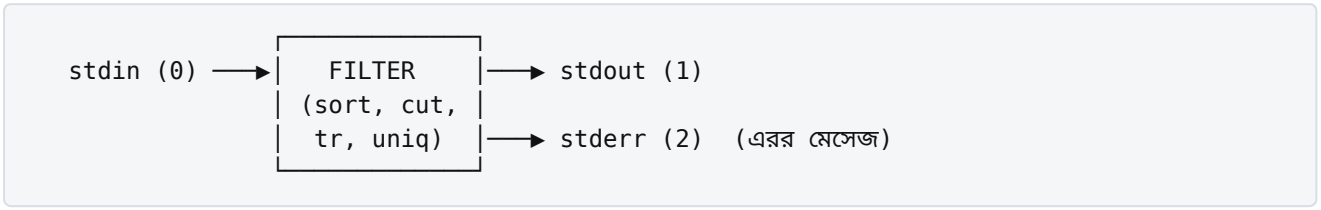
১৯৭০-এর দশকে Unix-এর নির্মাতারা একটা নীতি ঠিক করেছিলেন, যা আজও লিনাক্সের প্রাণ:

> প্রতিটি প্রোগ্রাম একটা কাজ করবে, এবং সেটা খুব ভালোভাবে করবে। প্রোগ্রামগুলো একসাথে কাজ করবে। আর সবাই টেক্সট দিয়ে কথা বলবে, কারণ টেক্সট হলো সবার বোঝার মতো ভাষা।

উপমা: একটা রান্নাঘরের কথা ভাবো। একজন সবজি কাটে, একজন ধোয়, একজন ভাজে, একজন প্লেটে সাজায়। কেউ সব কাজ একা করে না, কিন্তু একটা লাইনে দাঁড়িয়ে থাকা এই চারজন মিলে যে কোনো রান্না করে ফেলতে পারে। লিনাক্সের টেক্সট টুলগুলোও তাই — `cut` কাটে, `sort` সাজায়, `uniq` ডুপ্লিকেট বাছে, `wc` গোনে। আর এদের লাইনে দাঁড় করায় **পাইপ** (`|`)।

২৩.১.২ ফিল্টার কী?

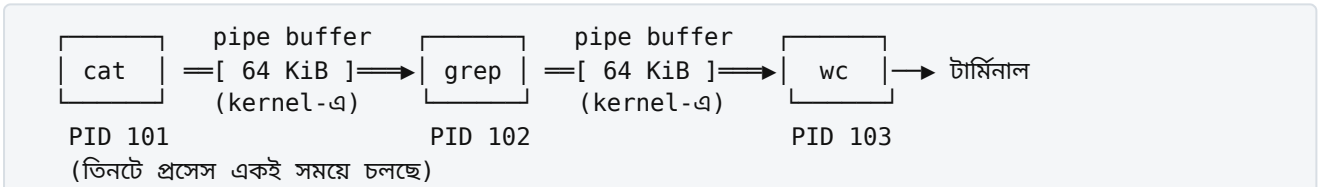
যে প্রোগ্রাম **stdin** (standard input — স্ট্যান্ডার্ড ইনপুট) থেকে ডেটা পড়ে, কিছু একটা পরিবর্তন করে, আর **stdout** (standard output) এ লিখে দেয় — তাকে **filter** বলে। এই অধ্যায়ের প্রায় সব টুলই ফিল্টার। এরা ফাইলের নাম দিলে ফাইল থেকে পড়ে, না দিলে stdin থেকে পড়ে।



stdin/stdout/stderr আর রিডাইরেকশন বিস্তারিতভাবে অধ্যায় ১৯-এ আছে। এখানে শুধু মনে রাখো: পাইপ একটা প্রোগ্রামের stdout-কে পরেরটার stdin-এর সাথে জুড়ে দেয়।

২৩.১.৩ ভেতরে কীভাবে কাজ করে: পাইপলাইন একসাথে চলে

অনেকে ভাবে `a | b | c` লিখলে আগে `a` পুরোটা শেষ হয়, তারপর `b` চলে। আসলে তা না। শেল (shell) তিনটে প্রসেসই **একসাথে** চালু করে, আর kernel তাদের মাঝে একটা করে **pipe buffer** (লিনাক্সে ডিফল্ট 64 KiB) তৈরি করে।



- বাফার ভরে গেলে লেখক প্রসেস (writer) অপেক্ষা করে।
- বাফার খালি থাকলে পাঠক প্রসেস (reader) অপেক্ষা করে।
- ফলে ১০ GB ফাইলও অল্প মেমরিতে প্রসেস করা যায় — ডেটা ধারার মতো বয়ে যায়।

তবে একটা ব্যতিক্রম আছে: **`sort`**। পুরো ইনপুট না দেখে সে প্রথম লাইনটাও বলতে পারে না (শেষ লাইনটাই হয়তো সবার আগে বসবে!)। তাই `sort` সব ইনপুট শেষ হওয়া পর্যন্ত অপেক্ষা করে। এ নিয়ে পরে আরও বলব।

২৩.১.৪ এই অধ্যায়ের নমুনা ফাইল

পুরো অধ্যায় জুড়ে আমরা কয়েকটা ছোট ফাইল ব্যবহার করব। চাইলে এখনই বানিয়ে ফেলো:

```
mkdir -p ~/textlab && cd ~/textlab

cat > employees.csv << 'EOF'
id,name,dept,salary,city
1,Rahim,IT,55000,Dhaka
2,Karim,HR,42000,Chittagong
3,Sumi,IT,61000,Dhaka
4,Nila,Sales,38000,Khulna
5,Jamal,HR,45000,Dhaka
```

```
EOF

cat > fruits.txt << 'EOF'
banana
apple
cherry
apple
banana
apple
EOF
```

> 📝 **মনে রাখো:** `<< 'EOF'` হলো **here-document** — শেলকে বলা হচ্ছে "EOF লেখা লাইন পর্যন্ত যা লিখছি, সেটা ইনপুট হিসেবে দাও"। বিস্তারিত অধ্যায় ১৯-এ।

২৩.২ `cut` — লাইন থেকে টুকরো কেটে নাও

২৩.২.১ কী করে

`cut` প্রতিটি লাইন থেকে নির্দিষ্ট **ফিল্ড** (কলাম), **অক্ষর** (character) বা **বাইট** কেটে বের করে। একটা কাঁচির মতো — প্রতিটা লাইনকে একই জায়গায় কাটে।

Syntax:

```
cut OPTION... [FILE]...
```

২৩.২.২ গুরুত্বপূর্ণ অপশন

অপশন	মানে	উদাহরণ
`-d DELIM`	ফিল্ড আলাদা করার চিহ্ন (delimiter); ডিফল্ট TAB	`-d ','`
`-f LIST`	কোন ফিল্ডগুলো নেবে	`-f 2`, `-f 1,3`, `-f 2-4`, `-f 3-`
`-c LIST`	কোন অক্ষরগুলো নেবে	`-c 1-5`
`-b LIST`	কোন বাইটগুলো নেবে	`-b 1-10`
`-s`	যে লাইনে delimiter নেই, সেটা বাদ দাও	`-s -d ':' -f1`
`--complement`	নির্বাচিত অংশ বাদ দিয়ে বাকিটা দেখাও (GNU)	`--complement -f 1`
`--output-delimiter=STR`	আউটপুটে অন্য delimiter বসাও (GNU)	`--output-delimiter='`

LIST লেখার নিয়ম: `N` (শুধু N নম্বর), `N-M` (N থেকে M), `N-` (N থেকে শেষ পর্যন্ত), `-M` (শুরু থেকে M পর্যন্ত), আর কমা দিয়ে একাধিক (`1,3,5`)।

২৩.২.৩ উদাহরণ

নাম ও বেতন বের করো:

```
$ cut -d ',' -f 2,4 employees.csv
name,salary
Rahim,55000
Karim,42000
Sumi,61000
Nila,38000
Jamal,45000
```

দ্বিতীয় কলাম থেকে শেষ পর্যন্ত:

```
$ cut -d ',' -f 2- employees.csv | head -2
name,dept,salary,city
Rahim,IT,55000,Dhaka
```

প্রথম কলাম (id) বাদ দিয়ে সব:

```
$ cut -d ',' --complement -f 1 employees.csv | head -2
name,dept,salary,city
Rahim,IT,55000,Dhaka
```

সিস্টেমের সব ইউজারনেম (`/etc/passwd`-এ delimiter হলো `:`):

```
$ cut -d ':' -f 1 /etc/passwd | head -3
root
daemon
bin
```

অক্ষর দিয়ে কাটা — প্রতিটি লাইনের প্রথম ৩টি অক্ষর:

```
$ cut -c 1-3 fruits.txt
ban
app
che
app
ban
app
```

আউটপুটের delimiter বদলানো:

```
$ cut -d ':' -f 1,7 --output-delimiter=' => ' /etc/passwd | head -2
root => /bin/bash
```

```
daemon => /usr/sbin/nologin
```

২৩.২.৪ `cut`-এর সীমাবদ্ধতা

> ⚠️ **সতর্কতা:** `cut`-এর delimiter **ঠিক একটা অক্ষর**। আর পরপর দুটো delimiter থাকলে মাঝখানে একটা **খালি ফিল্ড** ধরে। তাই একাধিক স্পেস দিয়ে আলাদা করা আউটপুট (যেমন `ls -l`, `ps aux`) `cut -d ' '` দিয়ে ঠিকমতো কাটা যায় না।

```
$ echo "a   b" | cut -d ' ' -f 2

$ echo "a   b" | awk '{print $2}'
b
```

প্রথম ক্ষেত্রে ২ নম্বর ফিল্ডটা খালি (প্রথম আর দ্বিতীয় স্পেসের মাঝের অংশ)। সমাধান দুটো:

1. `tr -s ' '` দিয়ে আগে স্পেস চেপে এক করে নাও (পরে দেখব), তারপর `cut`।
2. অথবা সরাসরি `awk` ব্যবহার করো (অধ্যায় ২২), যে whitespace-কে স্বাভাবিকভাবে সামলায়।

> ⚠️ **সতর্কতা:** `cut` উদ্ধৃতি চিহ্ন (quote) বোঝে না। `"Dhaka, Bangladesh"` এর মতো কমা-ওয়ালা ফিল্ড থাকলে CSV ভুলভাবে কাটবে। জটিল CSV-র জন্য `csvkit`, `mlr` (Miller) বা Python ব্যবহার করো।

> 📝 **মনে রাখো:** `-c` অক্ষর ধরে কাটে বলা হলেও GNU `cut`-এ বর্তমানে `-c` আর `-b` একইভাবে কাজ করে — অর্থাৎ বাইট ধরে। বাংলার মতো multi-byte (UTF-8) লেখায় `cut -c 1-3` একটা বাংলা অক্ষরকে মাঝখান থেকে কেটে ভাঙা অক্ষর দেখাতে পারে। বাংলা টেক্সটের জন্য `awk` বা `grep -o` নিরাপদ।

২৩.৩ `sort` — সবকিছু সাজাও

২৩.৩.১ কী করে

`sort` লাইনগুলোকে ক্রম অনুসারে সাজায়। ডিফল্টে সে পুরো লাইনকে টেক্সট হিসেবে, বর্তমান **locale** (ভাষা-অঞ্চলের নিয়ম) অনুযায়ী, ছোট থেকে বড় সাজায়।

Syntax:

```
sort [OPTION]... [FILE]...
```

২৩.৩.২ গুরুত্বপূর্ণ অপশন

অপশন	মানে
`-n`	সংখ্যা হিসেবে সাজাও (numeric)
`-r`	উল্টো ক্রম (reverse) — বড় থেকে ছোট
`-k POS1[,POS2]`	নির্দিষ্ট key/কলাম ধরে সাজাও
`-t SEP`	ফিল্ড আলাদা করার চিহ্ন (ডিফল্ট: blank থেকে non-blank-এ পরিবর্তন)
`-u`	একই রকম লাইন একবারই রাখো (unique)
`-h`	মানুষের পড়ার মতো সাইজ: `512`, `1K`, `2M`, `1G` (human-numeric)
`-V`	ভার্সন নম্বরের মতো সাজাও: `file2` < `file10`
`-f`	বড়/ছোট হাতের অক্ষর এক ধরো (fold case)
`-M`	মাসের নাম ধরে সাজাও: JAN < FEB < ... < DEC
`-b`	লাইনের শুরুর ফাঁকা জায়গা উপেক্ষা করো
`-s`	stable sort — সমান key হলে আগের ক্রম বজায় রাখো
`-o FILE`	আউটপুট সরাসরি ফাইলে লেখো (একই ফাইল হলেও নিরাপদ)
`-c`	শুধু চেক করো ফাইল সাজানো কিনা
`-R`	এলোমেলো সাজাও (random; একই key-র লাইন পাশাপাশি থাকে)
`-z`	লাইনের শেষ চিহ্ন newline না, NUL (`\0`)
`-T DIR`	অস্থায়ী ফাইল কোন ডিরেক্টরিতে রাখবে
`-S SIZE`	মেমরি বাফারের আকার, যেমন `-S 2G`
`--parallel=N`	N টা থ্রেডে সাজাও (GNU)

২৩.৩.৩ সাধারণ সাজানো

```
$ sort fruits.txt
apple
apple
apple
banana
banana
cherry
```

উল্টো:

```
$ sort -r fruits.txt
cherry
banana
banana
apple
```

```
apple
apple
```

ডুপ্লিকেট বাদ দিয়ে:

```
$ sort -u fruits.txt
apple
banana
cherry
```

২৩.৩.৪ টেক্সট বনাম সংখ্যা — `-n` কেন দরকার

```
$ printf '10\n9\n100\n2\n' | sort
10
100
2
9

$ printf '10\n9\n100\n2\n' | sort -n
2
9
10
100
```

কেন এমন হয়? `-n` ছাড়া `sort` অক্ষর ধরে তুলনা করে, ঠিক যেমন অভিধানে শব্দ সাজানো হয়। `"10"` আর `"9"` তুলনা করার সময় প্রথম অক্ষর `'1'` আর `'9'`; `'1'` ছোট, তাই `"10"` আগে বসে। `-n` দিলে সে পুরো জিনিসটাকে সংখ্যা হিসেবে পড়ে।

২৩.৩.৫ কলাম ধরে সাজানো: `-t` আর `-k`

বেতন অনুযায়ী বড় থেকে ছোট সাজাও (হেডার লাইন বাদ দিয়ে):

```
$ tail -n +2 employees.csv | sort -t ',' -k 4,4nr
3,Sumi,IT,61000,Dhaka
1,Rahim,IT,55000,Dhaka
5,Jamal,HR,45000,Dhaka
2,Karim,HR,42000,Chittagong
4,Nila,Sales,38000,Khulna
```

এখানে:

- `-t ','` — কমা দিয়ে কলাম আলাদা।
- `-k 4,4nr` — key শুরু ৪ নম্বর ফিল্ডে, **শেষও** ৪ নম্বর ফিল্ডে; আর এই key-র জন্য `n` (সংখ্যা) ও `r` (উল্টো)।
- `tail -n +2` — দ্বিতীয় লাইন থেকে শুরু, মানে হেডার বাদ।

> ⚠️ **সতর্কতা:** `-k 4` আর `-k 4,4` এক জিনিস না! `-k 4` মানে "৪ নম্বর ফিল্ড থেকে **লাইনের শেষ পর্যন্ত**" key। বেশিরভাগ সময় তুমি চাও শুধু একটা ফিল্ড — তাই সবসময় `-k N,N` লেখার অভ্যাস করো।

একাধিক key: আগে বিভাগ (dept) অনুযায়ী, একই বিভাগের মধ্যে বেতন বড় থেকে ছোট:

```
$ tail -n +2 employees.csv | sort -t ',' -k 3,3 -k 4,4nr
5,Jamal,HR,45000,Dhaka
2,Karim,HR,42000,Chittagong
3,Sumi,IT,61000,Dhaka
1,Rahim,IT,55000,Dhaka
4,Nila,Sales,38000,Khulna
```

ফিল্ডের ভেতরের অক্ষর: `-k 2.3` মানে ২ নম্বর ফিল্ডের ৩ নম্বর অক্ষর থেকে। যেমন `-k 1.5,1.8` দিয়ে `2026-11-12` এর মতো তারিখের মাসের অংশ ধরা যায়।

২৩.৩.৬ `-h` — সাইজ সাজানো

`du -h`, `ls -lh` এর আউটপুটে `4.0K`, `120M`, `2.1G` এর মতো সাইজ থাকে। সাধারণ `-n` এগুলো বোঝে না (`2.1G` কে 2.1 ধরবে)। `-h` এককগুলো বোঝে:

```
$ printf '1G\n512\n2M\n1K\n' | sort -h
512
1K
2M
1G
```

সবচেয়ে বড় ১০টা ফোল্ডার খোঁজার বিখ্যাত কমান্ড:

```
$ du -sh /var/* 2>/dev/null | sort -rh | head -5
2.3G    /var/lib
812M    /var/cache
240M    /var/log
12K     /var/tmp
4.0K    /var/mail
```

২৩.৩.৭ `-V` — ভার্সন সাজানো

```
$ printf 'file10\nfile2\nfile1\n' | sort
file1
file10
file2

$ printf 'file10\nfile2\nfile1\n' | sort -V
file1
file2
file10
```

```
$ printf '1.10.0\n1.2.3\n1.9.1\n' | sort -V
1.2.3
1.9.1
1.10.0
```

কার্নেল ভার্সন, প্যাকেজ ভার্সন, বা `img1.jpg ... img100.jpg` সাজাতে `-V` জাদুর মতো কাজ করে।

২৩.৩.৮ locale-এর প্রভাব: `LC_ALL=C`

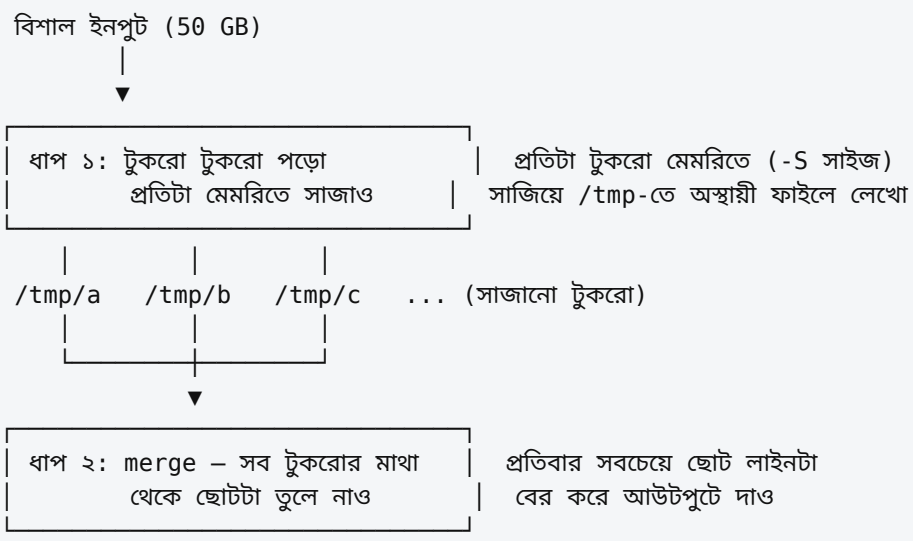
```
$ printf 'b\nA\na\nB\n' | sort
a
A
b
B

$ printf 'b\nA\na\nB\n' | LC_ALL=C sort
A
B
a
b
```

`en_US.UTF-8` এর মতো locale-এ `sort` "মানুষের মতো" সাজায় (বড়-ছোট হাতের অক্ষর পাশাপাশি)। `LC_ALL=C` দিলে সে শুধু **বাইটের মান** (ASCII কোড) তুলনা করে — `A` (65) < `B` (66) < `a` (97)।

> 💡 **টিপস:** বিশাল ফাইল সাজাতে `LC_ALL=C sort` অনেক গুণ দ্রুত, কারণ জটিল ভাষার নিয়ম প্রয়োগ করতে হয় না। আর স্ক্রিপ্টে `sort` আর `join` / `comm` একসাথে ব্যবহার করলে **দুই জায়গাতেই একই locale** দাও, নাহলে "file is not in sorted order" এরর আসতে পারে।

২৩.৩.৯ ভেতরে কীভাবে কাজ করে: external merge sort

ধরো তোমার কাছে ৫০ GB লগ ফাইল, অথচ RAM মাত্র ৮ GB। `sort` তবুও কাজ করে। কীভাবে?



```
                ▼
          সাজানো  আউটপুট
```

এটাকে **external merge sort** বলে। তাই:

- `/tmp` এ জায়গা কম থাকলে `sort -T /bigdisk/tmp` দাও।
- বেশি RAM দিয়ে দ্রুত করতে `-S 4G`।
- আগে থেকেই সাজানো কয়েকটা ফাইল মেলাতে `sort -m a b c` (শুধু merge ধাপ, অনেক দ্রুত)।

২৩.৩.১০ একই ফাইলে লেখার ফাঁদ

> ⚠️ **সতর্কতা:** `sort data.txt > data.txt` কখনো লিখবে না! শেল `>` দেখেই `sort` চালুর **আগে** ফাইলটা খালি (truncate) করে দেয়। ফলাফল: খালি ফাইল, ডেটা উধাও। সঠিক উপায়:
>
> ```
> sort -o data.txt data.txt
> ```
>
> `-o` দিলে `sort` পুরো ইনপুট পড়ে নেওয়ার পরে ফাইলে লেখে, তাই নিরাপদ।

২৩.৪ `uniq` — ডুপ্লিকেট সামলাও

২৩.৪.১ কী করে

`uniq` **পাশাপাশি** (adjacent) থাকা একই রকম লাইনগুলোকে একটা করে দেয়। গুনতে পারে, শুধু ডুপ্লিকেট বা শুধু অনন্য লাইন দেখাতে পারে।

Syntax:

```
uniq [OPTION]... [INPUT [OUTPUT]]
```

২৩.৪.২ গুরুত্বপূর্ণ অপশন

অপশন	মানে
`-c`	প্রতিটি লাইন কতবার এসেছে তা সামনে লেখো (count)
`-d`	শুধু সেই লাইনগুলো দেখাও যেগুলো একাধিকবার আছে (প্রতিটা একবার)
`-D`	ডুপ্লিকেট লাইনগুলো সবকটা দেখাও (GNU)
`-u`	শুধু সেই লাইন যেগুলো একবারই আছে
`-i`	বড়/ছোট হাতের অক্ষর উপেক্ষা করো
`-f N`	তুলনার সময় প্রথম N টা ফিল্ড বাদ দাও

অপশন	মানে
`-s N`	তুলনার সময় প্রথম N টা অক্ষর বাদ দাও
`-w N`	শুধু প্রথম N টা অক্ষর তুলনা করো

২৩.৪.৩ কেন আগে `sort` করতেই হবে

এটাই `uniq`-এর সবচেয়ে গুরুত্বপূর্ণ কথা। দেখো:

```
$ uniq fruits.txt
banana
apple
cherry
apple
banana
apple
```

কিছুই বাদ গেল না! কারণ `fruits.txt`-এ কোনো দুটো একই লাইন **পাশাপাশি** নেই।

```
ইনপুট          uniq যা দেখে (শুধু আগের লাইনের সাথে তুলনা)
──────          ──────────────────────────────────────────
banana   ──►  প্রথম লাইন → ছাপো
apple    ──►  আগেরটা banana, আলাদা → ছাপো
cherry   ──►  আগেরটা apple, আলাদা → ছাপো
apple    ──►  আগেরটা cherry, আলাদা → ছাপো   (প্রথম apple মনে নেই!)
banana   ──►  ...
```

ভেতরের কারণ: `uniq` শুধু **আগের একটা লাইন** মনে রাখে। এতে তার মেমরি লাগে প্রায় শূন্য, আর সে অসীম বড় ইনপুটও ধারার মতো (streaming) প্রসেস করতে পারে। সব লাইন মনে রাখতে গেলে বিশাল ফাইলে মেমরি ফুরিয়ে যেত। এই কাজটা ভাগ করে দেওয়া হয়েছে: `sort` একই লাইনগুলো পাশাপাশি এনে দেয়, `uniq` সেগুলো গোনে।

```
$ sort fruits.txt | uniq
apple
banana
cherry
```

> 💡 **টিপস:** শুধু ডুপ্লিকেট সরাতে চাইলে `sort -u` যথেষ্ট। কিন্তু **গুনতে** (`-c`), বা শুধু ডুপ্লিকেট/শুধু অনন্য দেখতে (`-d` / `-u`) হলে `sort | uniq` লাগবে।

> 💡 **টিপস:** ক্রম না বদলে (প্রথমবার যেখানে এসেছে সেখানেই রেখে) ডুপ্লিকেট সরাতে awk-এর বিখ্যাত ট্রিক: `awk '!seen[$0]++' file` (অধ্যায় ২২ দেখো)।

২৩.৪.৪ গোনা: `-c` — "ফ্রিকোয়েন্সি গণনা" প্যাটার্ন

```
$ sort fruits.txt | uniq -c
      3 apple
      2 banana
      1 cherry
```

সবচেয়ে বেশি থেকে কম অনুযায়ী সাজাতে আরেকবার `sort -rn`:

```
$ sort fruits.txt | uniq -c | sort -rn
      3 apple
      2 banana
      1 cherry
```

এই `sort | uniq -c | sort -rn` প্যাটার্নটা মুখস্থ করে ফেলো — লিনাক্স অ্যাডমিনের সবচেয়ে বেশি ব্যবহৃত পাইপলাইনগুলোর একটা।

বিভাগ অনুযায়ী কর্মী সংখ্যা:

```
$ tail -n +2 employees.csv | cut -d ',' -f 3 | sort | uniq -c
      2 HR
      2 IT
      1 Sales
```

২৩.৪.৫ `-d` আর `-u`

```
$ sort fruits.txt | uniq -d
apple
banana

$ sort fruits.txt | uniq -u
cherry
```

২৩.৪.৬ ফিল্ড বাদ দিয়ে তুলনা: `-f`

লগে প্রতিটা লাইনের শুরুতে টাইমস্ট্যাম্প থাকলে, টাইমস্ট্যাম্প বাদ দিয়ে একই মেসেজ পাশাপাশি এলে একবার দেখাও:

```
$ cat app.log
10:01 disk full
10:02 disk full
10:03 network down

$ uniq -f 1 app.log
10:01 disk full
10:03 network down
```

২৩.৫ `wc` — গোনো

২৩.৫.১ কী করে

`wc` (word count) লাইন, শব্দ, বাইট/অক্ষর গোনে।

অপশন	মানে
`-l`	লাইন সংখ্যা (আসলে newline অক্ষরের সংখ্যা)
`-w`	শব্দ সংখ্যা (whitespace দিয়ে আলাদা)
`-c`	বাইট সংখ্যা
`-m`	অক্ষর সংখ্যা (multi-byte UTF-8 সঠিকভাবে গোনে)
`-L`	সবচেয়ে লম্বা লাইনের দৈর্ঘ্য (GNU)

অপশন না দিলে লাইন, শব্দ, বাইট — এই তিনটে এই ক্রমে দেখায়।

২৩.৫.২ উদাহরণ

```
$ wc employees.csv
  6   6 147 employees.csv

$ wc -l employees.csv
6 employees.csv

$ wc -l < employees.csv
6
```

> 💡 **টিপস:** স্ক্রিপ্টে শুধু সংখ্যাটা চাইলে `wc -l < file` লেখো (ফাইলের নাম আসবে না)। `wc -l file` লিখলে সংখ্যার পাশে ফাইলের নামও আসে, যা আলাদা করতে বাড়তি `cut` লাগে।

একাধিক ফাইল দিলে শেষে মোট (total) দেখায়:

```
$ wc -l employees.csv fruits.txt
  6 employees.csv
  6 fruits.txt
 12 total
```

বাংলায় বাইট বনাম অক্ষর:

```
$ echo -n "লিনাক্স" | wc -c
21
$ echo -n "লিনাক্স" | wc -m
7
```

প্রতিটা বাংলা অক্ষর (codepoint) UTF-8-এ ৩ বাইট, তাই ৭টা codepoint = ২১ বাইট। (`-m` কাজ করতে UTF-8 locale লাগে।)

বাস্তব ব্যবহার:

```
ls | wc -l                          # এই ফোল্ডারে কতগুলো এন্ট্রি
grep -c "ERROR" app.log             # ERROR কতগুলো লাইনে (grep নিজেই গোনে)
ps aux --no-headers | wc -l         # কতগুলো প্রসেস চলছে
```

> ⚠️ **সতর্কতা:** `wc -l` আসলে newline গোনে। ফাইলের শেষ লাইনের পরে newline না থাকলে সেই লাইন গোনা হয় না — `printf 'a\nb' | wc -l` দেবে `1`।

২৩.৬ `tr` — অক্ষর অনুবাদ (translate)

২৩.৬.১ কী করে

`tr` অক্ষর ধরে কাজ করে: এক সেট অক্ষরকে আরেক সেটে বদলায়, মুছে ফেলে, বা পরপর একই অক্ষর চেপে এক করে।

Syntax:

```
tr [OPTION]... SET1 [SET2]
```

> 📝 **মনে রাখো:** `tr` **ফাইলের নাম নেয় না** — শুধু stdin থেকে পড়ে। তাই `tr 'a' 'b' file.txt` ভুল; লিখতে হবে `tr 'a' 'b' < file.txt` বা `cat file.txt | tr 'a' 'b'`।

অপশন	মানে
(কিছু না)	SET1-এর প্রতিটি অক্ষর SET2-এর একই অবস্থানের অক্ষরে বদলাও
`-d`	SET1-এর অক্ষরগুলো মুছে ফেলো (delete)
`-s`	পরপর একই অক্ষর (SET-এর) থাকলে একটা রাখো (squeeze)
`-c` / `-C`	SET1-এর **পরিপূরক** (complement) — SET1-এ নেই এমন সব অক্ষর
`-t`	SET1-কে কেটে SET2-এর দৈর্ঘ্যের সমান করো

অক্ষরের শ্রেণি (character class):

শ্রেণি	মানে
`[:lower:]`	ছোট হাতের অক্ষর

শ্রেণি	মানে
`[:upper:]`	বড় হাতের অক্ষর
`[:digit:]`	অঙ্ক 0-9
`[:alpha:]`	বর্ণ
`[:alnum:]`	বর্ণ + অঙ্ক
`[:space:]`	স্পেস, ট্যাব, newline ইত্যাদি
`[:blank:]`	শুধু স্পেস ও ট্যাব
`[:punct:]`	যতিচিহ্ন

এস্কেপ: `\n` (newline), `\t` (ট্যাব), `\r` (carriage return), `\\` (ব্যাকস্ল্যাশ)।

২৩.৬.২ বদলানো

```
$ echo "hello linux" | tr 'a-z' 'A-Z'
HELLO LINUX

$ echo "hello linux" | tr '[:lower:]' '[:upper:]'
HELLO LINUX

$ echo "a,b,c" | tr ',' '\n'
a
b
c

$ echo "PATH=/usr/bin:/bin" | tr ':' '\n'
PATH=/usr/bin
/bin
```

পরের উদাহরণটা দারুণ কাজের — `$PATH`-এর প্রতিটি ডিরেক্টরি আলাদা লাইনে:

```
echo "$PATH" | tr ':' '\n'
```

২৩.৬.৩ মুছে ফেলা: `-d`

```
$ echo "Phone: 017-1234-5678" | tr -d '-'
Phone: 01712345678

$ echo "abc123def456" | tr -d '[:digit:]'
abcdef
```

Windows ফাইল ঠিক করা: Windows-এ লাইনের শেষে `\r\n` থাকে, লিনাক্সে শুধু `\n`। Windows থেকে আসা স্ক্রিপ্ট চালালে `bad interpreter: /bin/bash^M` এর মতো এরর আসে। সমাধান:

```
tr -d '\r' < windows.txt > linux.txt
```

২৩.৬.৪ চেপে এক করা: `-s`

```
$ echo "a    b      c" | tr -s ' '
a b c
```

এবার `cut` দিয়ে একাধিক স্পেসের সমস্যা মেটানো যায়:

```
$ echo "a   b   c" | tr -s ' ' | cut -d ' ' -f 2
b
```

ফাঁকা লাইন চেপে ফেলা: `tr -s '\n'` (পরপর অনেকগুলো newline → একটা)।

২৩.৬.৫ পরিপূরক: `-c`

শুধু অক্ষর আর অঙ্ক রাখো, বাকি সব মুছে দাও (newline রেখে):

```
$ echo "Hello, World! 2026 :)" | tr -cd '[:alnum:]\n'
HelloWorld2026
```

লেখাকে প্রতি লাইনে একটা শব্দ করো (অক্ষর না এমন সব কিছু newline হয়ে যাবে, আর পরপর newline চেপে এক):

```
$ echo "Linux is fun, Linux is free." | tr -cs '[:alpha:]' '\n'
Linux
is
fun
Linux
is
free
```

২৩.৬.৬ ভেতরের কথা ও সীমাবদ্ধতা

`tr` আসলে একটা ২৫৬ ঘরের **lookup table** বানায় — প্রতিটি বাইটের জন্য "এটা কী হবে"। তারপর প্রতিটি বাইট টেবিল দেখে বদলায়। তাই এটা খুব দ্রুত।

> ⚠️ **সতর্কতা:** GNU `tr` বাইট ধরে কাজ করে, multi-byte অক্ষর (যেমন বাংলা) বোঝে না। বাংলা লেখায় `tr` দিয়ে অক্ষর বদলালে লেখা নষ্ট হতে পারে। সেক্ষেত্রে `sed` ব্যবহার করো (অধ্যায় ২১)।

> ⚠️ **সতর্কতা:** `tr` **শব্দ** বদলায় না, **অক্ষর** বদলায়। `echo "cat" | tr 'cat' 'dog'` এর মানে c→d, a→o, t→g; ফলে `dog` এসেছে কাকতালীয়ভাবে। `echo "act" | tr 'cat' 'dog'` দেবে `odg`। শব্দ বদলাতে `sed 's/cat/dog/g'`।

২৩.৭ `paste` — ফাইল পাশাপাশি জোড়া

২৩.৭.১ কী করে

`cat` ফাইলগুলোকে **উপর-নিচে** জোড়ে; `paste` জোড়ে **পাশাপাশি** — প্রথম ফাইলের লাইন ১ + দ্বিতীয় ফাইলের লাইন ১, ইত্যাদি। ডিফল্টে মাঝে TAB বসায়।

অপশন	মানে
`-d LIST`	delimiter (TAB-এর বদলে); একাধিক অক্ষর দিলে পালাক্রমে ব্যবহার করে
`-s`	serial — একটা ফাইলের সব লাইনকে এক লাইনে জোড়ো

২৩.৭.২ উদাহরণ

```
$ printf 'Rahim\nKarim\nSumi\n' > names.txt
$ printf '25\n30\n28\n' > ages.txt

$ paste names.txt ages.txt
Rahim   25
Karim   30
Sumi    28

$ paste -d ',' names.txt ages.txt
Rahim,25
Karim,30
Sumi,28
```

সব লাইন এক লাইনে (কমা দিয়ে):

```
$ paste -sd ',' names.txt
Rahim,Karim,Sumi
```

সংখ্যাগুলোর যোগফল — `paste` দিয়ে `+` বসিয়ে `bc` ক্যালকুলেটরে পাঠাও:

```
$ printf '10\n20\n30\n' | paste -sd '+' | bc
60
```

এক কলামকে তিন কলামে ভাগ (প্রতিটা `-` মানে stdin থেকে একটা লাইন):

```
$ seq 1 6 | paste - - -
1       2       3
4       5       6
```

২৩.৮ `join` — ডেটাবেসের মতো key ধরে মেলানো

২৩.৮.১ কী করে

`join` দুটো ফাইলকে একটা **সাধারণ key** (যেমন id) ধরে মেলায় — ঠিক SQL-এর `JOIN`-এর মতো। `paste` লাইন নম্বর ধরে জোড়ে; `join` key-এর মান ধরে।

> ⚠️ **সতর্কতা:** দুটো ফাইলই **join-এর key অনুযায়ী সাজানো** থাকতে হবে (একই locale-এ)। না হলে ভুল/অসম্পূর্ণ ফলাফল আর `join: file 1 is not in sorted order` সতর্কবার্তা আসে।

অপশন	মানে
`-t CHAR`	ফিল্ড delimiter
`-1 N`	প্রথম ফাইলের কোন ফিল্ড key
`-2 N`	দ্বিতীয় ফাইলের কোন ফিল্ড key
`-a 1` / `-a 2`	মেলেনি এমন লাইনও দেখাও সেই ফাইল থেকে (outer join)
`-v 1` / `-v 2`	**শুধু** মেলেনি এমন লাইন দেখাও
`-o FORMAT`	কোন কলামগুলো দেখাবে, যেমন `-o 1.2,2.2`
`-e STR`	খালি ঘরে কী বসবে (`-o` এর সাথে)
`-i`	বড়/ছোট হাতের অক্ষর উপেক্ষা
`--header`	প্রথম লাইনকে হেডার ধরো (GNU)

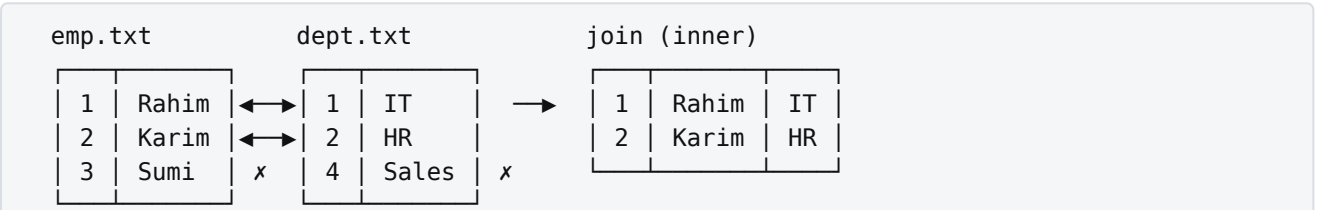
২৩.৮.২ উদাহরণ

```
$ cat emp.txt
1 Rahim
2 Karim
3 Sumi

$ cat dept.txt
1 IT
2 HR
4 Sales

$ join emp.txt dept.txt
1 Rahim IT
2 Karim HR
```

শুধু দুই ফাইলেই আছে এমন key (1, 2) এসেছে — এটা **inner join**।

```
$ join -a 1 emp.txt dept.txt
1 Rahim IT
2 Karim HR
```

```
3 Sumi

$ join -v 2 emp.txt dept.txt
4 Sales

$ join -o 1.2,2.2 emp.txt dept.txt
Rahim IT
Karim HR
```



ভেতরের কথা: যেহেতু দুটো ফাইলই সাজানো, `join` দুটোতে একটা করে "আঙুল" রেখে একসাথে এগোয় (merge join)। ছোট key-এর দিকের আঙুল এগোয়, সমান হলে মেলায়। এতে পুরো ফাইল মেমরিতে রাখতে হয় না — তাই সাজানোর শর্ত।

২৩.৯ `comm` — দুটো সাজানো তালিকার তুলনা

২৩.৯.১ কী করে

`comm` দুটো **সাজানো** ফাইল তুলনা করে তিনটে কলামে আউটপুট দেয়:

- কলাম ১: শুধু প্রথম ফাইলে আছে
- কলাম ২: শুধু দ্বিতীয় ফাইলে আছে
- কলাম ৩: দুটোতেই আছে

`-1`, `-2`, `-3` দিয়ে সেই কলাম **লুকানো** যায়।

```
$ printf 'apple\nbanana\ncherry\n' > list1.txt
$ printf 'banana\ncherry\ndate\n'   > list2.txt

$ comm list1.txt list2.txt
apple
        banana
        cherry
    date
```

(কলামগুলো TAB দিয়ে সরানো।)

কমান্ড	ফলাফল	সেট তত্ত্বে
`comm -12 a b`	দুটোতেই আছে	ছেদ (A ∩ B)
`comm -23 a b`	শুধু a-তে	A − B

কমান্ড	ফলাফল	সেট তত্ত্বে
`comm -13 a b`	শুধু b-তে	B − A

```
$ comm -12 list1.txt list2.txt
banana
cherry

$ comm -23 list1.txt list2.txt
apple
```

বাস্তব উদাহরণ: গতকাল আর আজকের ইনস্টল করা প্যাকেজের তালিকা তুলনা করে নতুন কী ইনস্টল হলো:

```
comm -13 <(sort packages-yesterday.txt) <(sort packages-today.txt)
```

`<( ... )` হলো **process substitution** — কমান্ডের আউটপুটকে একটা অস্থায়ী ফাইলের মতো ব্যবহার করে। ফলে আলাদা করে সাজানো ফাইল বানাতে হয় না।

২৩.১০ `diff` ও `patch` — পার্থক্য খোঁজা ও প্রয়োগ করা

২৩.১০.১ `diff` কী করে

`diff` দুটো ফাইল লাইন ধরে তুলনা করে বলে: প্রথমটাকে দ্বিতীয়টায় পরিণত করতে কী কী বদলাতে হবে। প্রোগ্রামাররা প্রতিদিন এটা ব্যবহার করে — `git diff` এই ধারণার উপরেই তৈরি।

```
$ cat old.conf
port=80
host=localhost
debug=true

$ cat new.conf
port=8080
host=localhost
debug=false
```

ডিফল্ট (normal) ফরম্যাট:

```
$ diff old.conf new.conf
1c1
< port=80
---
> port=8080
3c3
< debug=true
---
> debug=false
```

- `1c1` মানে প্রথম ফাইলের লাইন ১ **c**hange হয়ে দ্বিতীয় ফাইলের লাইন ১। অন্য অক্ষর: `a` (add), `d` (delete)।
- `<` প্রথম ফাইলের লাইন, `>` দ্বিতীয় ফাইলের।

২৩.১০.২ Unified ফরম্যাট: `diff -u`

এটাই সবচেয়ে জনপ্রিয় ফরম্যাট — পড়তে সহজ, আর `patch` ও `git` এটাই ব্যবহার করে:

```
$ diff -u old.conf new.conf
--- old.conf    2026-11-12 10:00:00.000000000 +0600
+++ new.conf    2026-11-12 10:05:00.000000000 +0600
@@ -1,3 +1,3 @@
-port=80
+port=8080
 host=localhost
-debug=true
+debug=false
```

পড়ার নিয়ম:

অংশ	মানে
`--- old.conf`	পুরনো (মূল) ফাইল
`+++ new.conf`	নতুন ফাইল
`@@ -1,3 +1,3 @@`	**hunk** (পরিবর্তনের খণ্ড): পুরনো ফাইলের লাইন ১ থেকে ৩টা লাইন, নতুন ফাইলের লাইন ১ থেকে ৩টা লাইন
`-` দিয়ে শুরু	মুছে ফেলা লাইন
`+` দিয়ে শুরু	যোগ করা লাইন
স্পেস দিয়ে শুরু	অপরিবর্তিত লাইন (context — প্রসঙ্গ; ডিফল্টে আশেপাশের ৩ লাইন)

২৩.১০.৩ `diff`-এর গুরুত্বপূর্ণ অপশন

অপশন	মানে
`-u`	unified ফরম্যাট (`-U N` দিলে N লাইন context)
`-r`	ডিরেক্টরি পুনরাবৃত্তভাবে (recursive) তুলনা
`-N`	একটা ফাইল না থাকলে খালি ধরে নাও (`-r` এর সাথে নতুন ফাইল patch-এ আনতে)
`-q`	শুধু বলো ফাইল আলাদা কিনা, বিস্তারিত না
`-s`	ফাইল একই হলেও জানাও
`-y`	পাশাপাশি (side-by-side) দেখাও; `-W N` দিয়ে প্রস্থ
`-i`	বড়/ছোট হাতের অক্ষর উপেক্ষা
`-w`	সব whitespace উপেক্ষা

অপশন	মানে
`-b`	whitespace-এর পরিমাণের পরিবর্তন উপেক্ষা
`-B`	ফাঁকা লাইনের পরিবর্তন উপেক্ষা
`--color`	রঙিন আউটপুট (GNU diffutils 3.4+)

পাশাপাশি দেখা:

```
$ diff -y -W 40 old.conf new.conf
port=80             |    port=8080
host=localhost           host=localhost
debug=true          |    debug=false
```

দুটো ফোল্ডারের কোন ফাইলগুলো আলাদা:

```
$ diff -rq project-v1/ project-v2/
Files project-v1/app.py and project-v2/app.py differ
Only in project-v2/: utils.py
```

Exit status (স্ক্রিপ্টে খুব কাজের):

কোড	মানে
`0`	ফাইল একই
`1`	ফাইল আলাদা
`2`	সমস্যা (যেমন ফাইল নেই)

```
if diff -q a.conf b.conf > /dev/null; then
    echo "কনফিগ একই আছে"
fi
```

২৩.১০.৪ `patch` — পার্থক্য প্রয়োগ করা

উপমা: `diff` হলো দর্জির মাপের চিরকুট — "হাতা ২ ইঞ্চি ছোট, কোমর ১ ইঞ্চি বড়"। `patch` হলো সেই দর্জি, যে চিরকুট দেখে আসল জামা বদলে দেয়। পুরো নতুন জামা পাঠানোর দরকার নেই, শুধু চিরকুট পাঠালেই হয়।

পুরনো দিনে ইন্টারনেট ধীর ছিল, তাই পুরো সোর্স কোড না পাঠিয়ে লোকে শুধু **patch ফাইল** পাঠাত। লিনাক্স কার্নেল এখনো mailing list-এ patch ইমেইল করে ডেভেলপ হয়।

```
# ১. পার্থক্য একটা patch ফাইলে রাখো
diff -u old.conf new.conf > fix.patch

# ২. অন্য কোথাও old.conf-এর কপিতে প্রয়োগ করো
patch old.conf < fix.patch
```

```
$ patch old.conf < fix.patch
patching file old.conf
$ cat old.conf
port=8080
host=localhost
debug=false
```

অপশন	মানে
`-pN`	patch-এর ফাইল পাথ থেকে শুরুর N টা অংশ (`/` পর্যন্ত) বাদ দাও
`-R`	উল্টো প্রয়োগ — patch ফিরিয়ে নাও (revert)
`--dry-run`	আসলে কিছু না বদলে দেখো প্রয়োগ হবে কিনা
`-b`	বদলানোর আগে `.orig` ব্যাকআপ রাখো
`-i FILE`	stdin-এর বদলে এই patch ফাইল পড়ো
`-d DIR`	আগে এই ডিরেক্টরিতে যাও

`-p1` কেন এত দেখা যায়? `git diff` বা ডিরেক্টরির `diff -ruN a/ b/` এর patch-এ পাথ থাকে এভাবে:

```
--- a/src/main.c
+++ b/src/main.c
```

প্রজেক্টের মূল ফোল্ডারে দাঁড়িয়ে প্রয়োগ করতে `a/` অংশটা বাদ দিতে হবে — `-p1` ঠিক সেটাই করে (একটা অংশ বাদ)। `-p0` মানে পাথ যেমন আছে তেমন।

```
cd project/
patch -p1 --dry-run < ../feature.patch   # আগে পরীক্ষা
patch -p1 < ../feature.patch             # তারপর আসল প্রয়োগ
patch -p1 -R < ../feature.patch          # দরকার হলে ফিরিয়ে নাও
```

> 💡 **টিপস:** সবসময় আগে `--dry-run` চালাও। ফাইল ইতিমধ্যে বদলে গিয়ে থাকলে patch "Hunk #1 FAILED" বলে, আর প্রয়োগ না হওয়া অংশ `.rej` (reject) ফাইলে রাখে। `patch` কিছুটা নমনীয়: লাইন নম্বর সামান্য সরে গেলে "offset" দিয়ে, আর context সামান্য না মিললে "fuzz" দিয়ে মিলিয়ে নেয়।

২৩.১০.৫ `cmp` — বাইট ধরে তুলনা

`diff` টেক্সট লাইন ধরে তুলনা করে; `cmp` বাইট ধরে। ছবি, zip, executable — যেকোনো **বাইনারি** ফাইলের জন্য `cmp` উপযুক্ত।

```
$ cmp a.bin b.bin
a.bin b.bin differ: byte 5, line 1

$ cmp -s a.bin b.bin && echo "একই" || echo "আলাদা"
আলাদা
```

অপশন	মানে
`-s`	চুপচাপ, শুধু exit status (0 = একই, 1 = আলাদা)
`-l`	সব আলাদা বাইটের অবস্থান ও মান (octal) দেখাও
`-n N`	শুধু প্রথম N বাইট তুলনা
`-b`	আলাদা বাইটগুলো অক্ষর হিসেবেও দেখাও

> 💡 **টিপস:** দুটো বড় ফাইল একই কিনা জানতে `cmp -s` প্রথম পার্থক্য পেলেই থেমে যায়, তাই দ্রুত। আর অন্য কম্পিউটারের ফাইলের সাথে তুলনায় হ্যাশ (`sha256sum`) মেলাও।

২৩.১১ `xargs` — আউটপুটকে argument বানাও

২৩.১১.১ সমস্যাটা কী?

পাইপ (`|`) একটা কমান্ডের আউটপুট পরের কমান্ডের **stdin**-এ পাঠায়। কিন্তু অনেক কমান্ড stdin পড়েই না — তারা চায় **argument** (কমান্ড লাইনে লেখা নাম)। যেমন `rm` , `mkdir` , `cp` , `chmod` ।

```
$ echo "a.txt b.txt" | rm
rm: missing operand
```

`rm` stdin-এর দিকে তাকায়ই না। এখানেই `xargs` আসে: সে stdin থেকে শব্দগুলো পড়ে, তারপর সেগুলোকে **argument হিসেবে জুড়ে** কমান্ডটা চালায়।

```
$ echo "a.txt b.txt" | xargs rm
# আসলে চলল:  rm a.txt b.txt
```

```
stdin: "a.txt\nb.txt\nc.txt"
          │
          ▼
    ┌──────────┐   argument বানিয়ে
    │ xargs rm │ ──────────────────►  rm a.txt b.txt c.txt
    └──────────┘   fork + exec
```

উপমা: পিয়ন একগাদা চিঠি (stdin) নিয়ে এল, কিন্তু বড়বাবু (rm) চিঠি নিজে খোলেন না — তিনি চান কেউ নামগুলো পড়ে তাঁর সামনে তালিকা রাখুক। `xargs` সেই সহকারী।

২৩.১১.২ ভেতরে কীভাবে কাজ করে

- `xargs` ডিফল্টে **whitespace** (স্পেস, ট্যাব, newline) দিয়ে ইনপুট ভাগ করে; উদ্ধৃতি চিহ্ন (`'` `"`) আর ব্যাকস্ল্যাশও বিশেষভাবে ধরে।
- কমান্ড না দিলে ডিফল্ট কমান্ড `echo` ।

- kernel-এর একটা সীমা আছে — একটা কমান্ডে মোট argument কত বড় হতে পারে (`ARG_MAX` ; দেখতে `getconf ARG_MAX` , সাধারণত 2 MB)। ১০ লাখ ফাইলের নাম একবারে দিলে `Argument list too long` এরর আসে। `xargs` বুদ্ধি করে তালিকাকে ভাগ করে কয়েকবার কমান্ড চালায়, যাতে এই সীমা পার না হয়।

```
$ rm *.log
bash: /usr/bin/rm: Argument list too long

$ find . -name '*.log' -print0 | xargs -0 rm      # কাজ করে
```

২৩.১১.৩ গুরুত্বপূর্ণ অপশন

অপশন	মানে
`-n N`	প্রতিবার সর্বোচ্চ N টা argument দিয়ে কমান্ড চালাও
`-I {}`	প্রতিটা ইনপুট **লাইনের** জন্য একবার চালাও, আর `{}` এর জায়গায় সেই লাইন বসাও
`-0`	ইনপুট NUL (`\0`) দিয়ে আলাদা (`find -print0` এর সাথে) — স্পেসওয়ালা নামের জন্য নিরাপদ
`-d DELIM`	নির্দিষ্ট delimiter দিয়ে ভাগ করো, যেমন `-d '\n'` (GNU)
`-P N`	একসাথে N টা কমান্ড সমান্তরালে (parallel) চালাও; `-P 0` = যত সম্ভব
`-t`	চালানোর আগে কমান্ডটা stderr-এ দেখাও (trace)
`-p`	প্রতিবার চালানোর আগে জিজ্ঞেস করো (prompt)
`-r`	ইনপুট খালি হলে কমান্ড চালাবে না (GNU `--no-run-if-empty`)
`-L N`	প্রতিবার N টা ইনপুট লাইন ব্যবহার করো
`-a FILE`	stdin-এর বদলে ফাইল থেকে পড়ো

২৩.১১.৪ `-n` — কতগুলো করে

```
$ echo "1 2 3 4 5" | xargs -n 2
1 2
3 4
5

$ echo "a b c" | xargs -n 1 echo "Item:"
Item: a
Item: b
Item: c
```

`-t` দিয়ে দেখো আসলে কী চলছে:

```
$ echo "a b c" | xargs -t -n 1 touch
touch a
touch b
touch c
```

২৩.১১.৫ `-I` — argument মাঝখানে বসানো

ডিফল্টে `xargs` argument কমান্ডের **শেষে** জোড়ে। কিন্তু `cp FILE /backup/` এর মতো ক্ষেত্রে ফাইলের নাম মাঝে বসাতে হয়:

```
$ ls *.txt | xargs -I {} cp {} /backup/{}.bak
```

```
$ printf 'dhaka\nkhulna\n' | xargs -I {} mkdir -p city_{}
$ ls -d city_*
city_dhaka  city_khulna
```

> 📝 **মনে রাখো:** `-I` দিলে প্রতিটি **লাইন** একটা argument (লাইনের ভেতরের স্পেস ভাঙে না), আর প্রতিটির জন্য কমান্ড **আলাদাভাবে** একবার চলে। `{}` প্রচলিত, তবে যেকোনো স্ট্রিং দেওয়া যায়, যেমন `-I %`।

২৩.১১.৬ `-0` — স্পেসওয়ালা ফাইলের নাম

```
$ touch "my file.txt"
$ ls *.txt | xargs rm
rm: cannot remove 'my': No such file or directory
rm: cannot remove 'file.txt': No such file or directory
```

`xargs` স্পেস দেখে নামটাকে দুই ভাগ করে ফেলেছে! সমাধান — NUL অক্ষর দিয়ে নাম আলাদা করা, কারণ লিনাক্সে ফাইলের নামে `/` আর NUL ছাড়া সব অক্ষর থাকতে পারে (এমনকি newline-ও):

```
find . -name '*.txt' -print0 | xargs -0 rm
```

> ⚠️ **সতর্কতা:** `ls | xargs ...` ফাইলের নামের ক্ষেত্রে ঝুঁকিপূর্ণ অভ্যাস। ফাইলের তালিকা পাঠাতে সবসময় `find ... -print0 | xargs -0` ব্যবহার করো। `find` নিয়ে বিস্তারিত অধ্যায় ১২-এ। আর অনেক সময় `find ... -exec cmd {} +` একই কাজ `xargs` ছাড়াই করে।

২৩.১১.৭ `-P` — সমান্তরাল কাজ

৪টা CPU core থাকলে ৪টা ছবি একসাথে কম্প্রেস করো:

```
find . -name '*.png' -print0 | xargs -0 -n 1 -P 4 optipng
```

২০টা সার্ভারে একসাথে ping:

```
cat servers.txt | xargs -I {} -P 10 ping -c 1 -W 1 {}
```

> 💡 **টিপস:** `-P` দিলে আউটপুট এলোমেলো ক্রমে আসতে পারে, কারণ যে আগে শেষ করে সে আগে লেখে। কতগুলো core আছে দেখতে `nproc`।

২৩.১১.৮ নিরাপত্তা অভ্যাস

> ⚠️ **সতর্কতা:** `xargs rm` এর মতো বিপজ্জনক কমান্ড চালানোর আগে `rm` এর জায়গায় `echo` বসিয়ে দেখে নাও কী চলবে:
>
> ```
> find /tmp -name '*.tmp' -mtime +7 -print0 | xargs -0 echo rm
> ```
>
> ঠিক থাকলে `echo` সরিয়ে আবার চালাও। আর খালি ইনপুটে কমান্ড না চলার জন্য GNU-তে `-r` দাও।

২৩.১২ `tee` — আউটপুট দুই দিকে পাঠাও

২৩.১২.১ কী করে

`tee` stdin থেকে পড়ে **একসাথে** stdout-এও লেখে আর এক বা একাধিক ফাইলেও লেখে। নামটা এসেছে প্লাম্বিং-এর **T-আকৃতির পাইপ জোড়া** থেকে — পানি এক দিক থেকে ঢুকে দুই দিকে যায়।

```
                      ┌──► stdout (পরের কমান্ড বা স্ক্রিন)
stdin ──► [ tee ] ────┤
                      └──► file.txt
```

অপশন	মানে
`-a`	ফাইলে নতুন করে না লিখে শেষে যোগ করো (append)
`-i`	Ctrl+C (SIGINT) উপেক্ষা করো

২৩.১২.২ উদাহরণ

```
$ ls /etc | tee etc-list.txt | wc -l
231
```

স্ক্রিনে সংখ্যা দেখলে, আর পুরো তালিকা `etc-list.txt`-তে সেভ হলো।

লম্বা কাজের লগ দেখতেও চাও, রাখতেও চাও:

```
sudo apt update 2>&1 | tee -a update.log
```

`2>&1` দিয়ে stderr-কেও stdout-এ মেশানো হলো, যাতে এরর মেসেজও লগে যায়।

২৩.১২.৩ বিখ্যাত `sudo tee` ট্রিক

```
$ sudo echo "nameserver 1.1.1.1" > /etc/resolv.conf
bash: /etc/resolv.conf: Permission denied
```

কেন ব্যর্থ? `sudo` শুধু `echo`-কে root বানায়। কিন্তু `>` রিডাইরেকশন করে **তোমার নিজের শেল**, যে সাধারণ ইউজার — ফাইল খোলার অনুমতি তার নেই। সমাধান:

```
echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf > /dev/null
echo "new line" | sudo tee -a /etc/hosts > /dev/null     # শেষে যোগ
```

এখন ফাইলটা খুলছে `tee`, যে `sudo` দিয়ে root হিসেবে চলছে। `> /dev/null` দেওয়া হলো যাতে স্ক্রিনে আবার না দেখায়। (`sudo` বিস্তারিত অধ্যায় ১৮-এ।)

> 💡 **টিপস:** Vim-এ root-এর ফাইল সাধারণ ইউজার হিসেবে খুলে সেভ করতে না পারলে `:w !sudo tee % > /dev/null` — একই কৌশল (অধ্যায় ১৪)।

২৩.১৩ ছোট কিন্তু কাজের টুল: `rev`, `shuf`, `fold`, `fmt`, `expand`, `nl`, `column`

২৩.১৩.১ `rev` — প্রতিটি লাইন উল্টো করো

```
$ echo "hello" | rev
olleh
```

কাজের ট্রিক: শেষ ফিল্ড বের করা। `cut` শুধু সামনে থেকে গোনে, কিন্তু উল্টে দিলে শেষটা সামনে চলে আসে:

```
$ echo "archive.tar.gz" | rev | cut -d '.' -f 1 | rev
gz

$ echo "/home/user/docs/report.pdf" | rev | cut -d '/' -f 1 | rev
report.pdf
```

> 📝 **মনে রাখো:** `rev` প্রতিটি লাইনের **অক্ষর** উল্টায়; লাইনগুলোর **ক্রম** উল্টাতে চাইলে `tac` (cat উল্টো বানান) — অধ্যায় ১০।

২৩.১৩.২ `shuf` — এলোমেলো করো

অপশন	মানে
`-n N`	শুধু N টা লাইন নাও
`-i LO-HI`	ফাইলের বদলে LO থেকে HI সংখ্যা এলোমেলো করো
`-e ARG...`	ফাইলের বদলে এই argumentগুলো এলোমেলো করো
`-r`	পুনরাবৃত্তি সহ (একই জিনিস বারবার আসতে পারে); `-n` এর সাথে
`-o FILE`	আউটপুট ফাইলে

```
$ shuf -n 1 names.txt                # লটারিতে একজন বিজয়ী
Karim

$ shuf -i 1-100 -n 5                 # ১-১০০ থেকে ৫টা আলাদা সংখ্যা
42
7
91
23
66

$ shuf -e rock paper scissors -n 1
paper

$ shuf -r -i 1-6 -n 3                # তিনবার ছক্কা ছোঁড়া
4
4
1
```

(আউটপুট প্রতিবার আলাদা হবে।) বড় ডেটাসেট থেকে ১০০০ লাইনের নমুনা: `shuf -n 1000 big.csv > sample.csv`।

২৩.১৩.৩ `fold` — লম্বা লাইন ভাঙো

`fold` লম্বা লাইনকে নির্দিষ্ট প্রস্থে ভেঙে দেয় (ডিফল্ট ৮০ কলাম)।

অপশন	মানে
`-w N`	N কলামে ভাঙো
`-s`	শব্দের মাঝখানে না ভেঙে স্পেসে ভাঙো
`-b`	কলামের বদলে বাইট গোনো

```
$ echo "Linux is a free and open source operating system kernel" | fold -w 20
Linux is a free and
open source operatin
g system kernel

$ echo "Linux is a free and open source operating system kernel" | fold -w 20 -s
Linux is a free and
```

```
open source
operating system
kernel
```

২৩.১৩.৪ `fmt` — অনুচ্ছেদ সুন্দর করে সাজাও

`fold` যান্ত্রিকভাবে কাটে; `fmt` একটু বুদ্ধিমান — অনুচ্ছেদের লাইনগুলো জোড়া দিয়ে আবার সমান দৈর্ঘ্যে সাজায় (ডিফল্ট সর্বোচ্চ প্রস্থ ৭৫)।

অপশন	মানে
`-w N`	সর্বোচ্চ প্রস্থ N
`-s`	শুধু লম্বা লাইন ভাঙো, ছোট লাইন জোড়া দিও না
`-u`	শব্দের মাঝে ১টা, বাক্যের পরে ২টা স্পেস (uniform spacing)

```
$ printf 'This is\na very\nshort\nline text.\n' | fmt -w 30
This is a very short line
text.
```

> 💡 **টিপস:** Vim-এ `gq` আর nano-তে justify এই ধরনের কাজই করে। ইমেইল বা README সাজাতে `fmt -w 72` জনপ্রিয়।

২৩.১৩.৫ `expand` ও `unexpand` — ট্যাব ↔ স্পেস

```
$ printf 'a\tb\n' | cat -A
a^Ib$
$ printf 'a\tb\n' | expand -t 4 | cat -A
a   b$
```

(`cat -A` অদৃশ্য অক্ষর দেখায়: `^I` = ট্যাব, `$` = লাইনের শেষ।)

কমান্ড	কাজ
`expand file`	ট্যাবকে স্পেসে বদলাও (ডিফল্ট ট্যাব স্টপ ৮)
`expand -t 4 file`	ট্যাব স্টপ ৪
`expand -i file`	শুধু লাইনের শুরুর ট্যাব বদলাও
`unexpand -a file`	স্পেসকে আবার ট্যাবে বদলাও (`-a` ছাড়া শুধু শুরুর blank)
`unexpand -t 4 file`	ট্যাব স্টপ ৪ ধরে

কেন দরকার? Python বা YAML-এ ট্যাব-স্পেস মেশালে এরর হয়; আর `Makefile`-এ রেসিপির লাইন ট্যাব দিয়েই শুরু হতে হয়।

২৩.১৩.৬ `nl` — লাইন নম্বর বসাও

```
$ nl fruits.txt | head -3
     1  banana
     2  apple
     3  cherry
```

`nl` ডিফল্টে ফাঁকা লাইনে নম্বর দেয় না; সব লাইনে দিতে `nl -ba`। (`cat -n` সব লাইনে দেয়।)

২৩.১৩.৭ `column` — টেবিল আকারে সাজাও

```
$ column -t -s ',' employees.csv
id  name   dept   salary  city
1   Rahim  IT     55000   Dhaka
2   Karim  HR     42000   Chittagong
3   Sumi   IT     61000   Dhaka
4   Nila   Sales  38000   Khulna
5   Jamal  HR     45000   Dhaka
```

`-t` টেবিল বানায়, `-s` ইনপুটের delimiter বলে। `mount | column -t` চালিয়ে দেখো — অগোছালো আউটপুট কেমন পরিষ্কার হয়ে যায়।

২৩.১৪ `split` ও `csplit` — বড় ফাইল টুকরো করো

২৩.১৪.১ `split` — আকার বা লাইন ধরে ভাগ

অপশন	মানে
`-l N`	প্রতি টুকরোয় N লাইন (ডিফল্ট ১০০০)
`-b SIZE`	প্রতি টুকরো SIZE বাইট (`100K`, `50M`, `1G`)
`-C SIZE`	সর্বোচ্চ SIZE, কিন্তু লাইনের মাঝখানে কাটবে না
`-n N`	মোট N টা সমান টুকরো; `-n l/N` দিলে লাইন না ভেঙে
`-d`	নামের শেষে অক্ষর (`aa`, `ab`) এর বদলে সংখ্যা (`00`, `01`)
`-a N`	suffix-এর দৈর্ঘ্য N
`--additional-suffix=.csv`	নামের শেষে বাড়তি এক্সটেনশন (GNU)

Syntax: `split [OPTION] [FILE [PREFIX]]` — PREFIX না দিলে টুকরোর নাম `xaa`, `xab`, ...

```
$ seq 1 2500 > numbers.txt
$ split -l 1000 -d --additional-suffix=.txt numbers.txt part_
$ ls part_*
part_00.txt  part_01.txt  part_02.txt
$ wc -l part_*
```

```
 1000 part_00.txt
 1000 part_01.txt
  500 part_02.txt
 2500 total
```

বড় ভিডিও বা ব্যাকআপকে ১ GB টুকরোয় ভাগ (যেমন FAT32 পেনড্রাইভে ৪ GB-এর বেশি ফাইল রাখা যায় না):

```
split -b 1G backup.tar.gz backup.tar.gz.part_
```

আবার জোড়া লাগাও — `cat` নামের অক্ষর-ক্রমে (alphabetical) ফাইল নেয়, আর `split`-এর নাম সেভাবেই সাজানো:

```
cat backup.tar.gz.part_* > backup.tar.gz
sha256sum backup.tar.gz        # মূলটার সাথে মিলিয়ে দেখো
```

২৩.১৪.২ `csplit` — প্যাটার্ন ধরে ভাগ (context split)

`split` অন্ধভাবে আকার ধরে কাটে; `csplit` লেখার **বিষয়বস্তু** দেখে কাটে — যেমন প্রতিটি "Chapter" শব্দ থেকে নতুন ফাইল।

```
$ cat book.txt
Intro text
Chapter 1
Linux basics
Chapter 2
Shell basics

$ csplit -z -f chap_ book.txt '/^Chapter/' '{*}'
11
23
23
$ ls chap_*
chap_00  chap_01  chap_02
$ cat chap_01
Chapter 1
Linux basics
```

ব্যাখ্যা:

- `'/^Chapter/'` — "Chapter" দিয়ে শুরু হওয়া লাইনের **আগে** কাটো।
- `'{*}'` — আগের প্যাটার্ন যতবার পাওয়া যায় ততবার পুনরাবৃত্তি করো।
- `-f chap_` — নামের prefix (ডিফল্ট `xx`)।
- `-z` — খালি টুকরো থাকলে মুছে দাও।
- আউটপুটের সংখ্যাগুলো প্রতিটি টুকরোর বাইট আকার (চুপ রাখতে `-s`)।

লাইন নম্বর ধরেও কাটা যায়: `csplit file 100 200` → ১–৯৯, ১০০–১৯৯, ২০০–শেষ।

২৩.১৫ `jq` — JSON-এর জন্য sed/awk

২৩.১৫.১ কেন jq?

আধুনিক দুনিয়ায় API, কনফিগ, ক্লাউড টুল (`kubectl`, `aws`, `docker inspect`) — সবাই **JSON** দেয়। JSON হলো নেস্টেড (একটার ভেতরে আরেকটা) গঠন; `grep` / `cut` দিয়ে এটা নির্ভরযোগ্যভাবে পড়া যায় না। `jq` JSON-এর গঠন বোঝে।

ইনস্টল (সাধারণত আগে থেকে থাকে না):

```
sudo apt install jq        # Ubuntu / Debian
sudo dnf install jq        # Fedora / RHEL 9 (RHEL-এ AppStream রিপোজিটরিতে)
```

প্যাকেজ ইনস্টল নিয়ে বিস্তারিত অধ্যায় ৩০-এ।

২৩.১৫.২ নমুনা JSON

```
cat > users.json << 'EOF'
[
  {"name": "Rahim", "age": 25, "city": "Dhaka",  "skills": ["linux", "python"]},
  {"name": "Karim", "age": 31, "city": "Khulna", "skills": ["bash"]},
  {"name": "Sumi",  "age": 28, "city": "Dhaka",  "skills": ["linux", "docker", "go"]}
]
EOF
```

২৩.১৫.৩ মূল ধারণা: filter

`jq` এর প্রতিটা এক্সপ্রেশন একটা **filter** — JSON নেয়, JSON দেয়। আর ভেতরে `|` দিয়ে filter জোড়া যায়, ঠিক শেলের পাইপের মতো।

Filter	মানে
`.`	পুরো ইনপুট যেমন আছে (সুন্দর করে সাজিয়ে, রঙিন)
`.name`	অবজেক্টের `name` কী-এর মান
`.a.b`	নেস্টেড — `a` এর ভেতরের `b`
`.[0]`	অ্যারের প্রথম উপাদান
`.[]`	অ্যারের প্রতিটি উপাদান আলাদাভাবে
`.[1:3]`	অ্যারের slice (১ ও ২ নম্বর)
`select(cond)`	শর্ত মিললে রাখো
`map(f)`	প্রতিটি উপাদানে f প্রয়োগ করে নতুন অ্যারে
`length`	অ্যারে/স্ট্রিং-এর দৈর্ঘ্য, অবজেক্টের কী-সংখ্যা
`keys`	অবজেক্টের সব কী (সাজানো)

Filter	মানে
`{a: .x}`	নতুন অবজেক্ট বানাও
`sort_by(.f)`, `group_by(.f)`	সাজানো / দল বাঁধা

অপশন	মানে
`-r`	raw — স্ট্রিং উদ্ধৃতি চিহ্ন ছাড়া দেখাও (শেলে ব্যবহারের জন্য জরুরি)
`-c`	compact — প্রতিটা ফলাফল এক লাইনে
`-s`	slurp — সব ইনপুট একটা অ্যারেতে নাও
`-n`	কোনো ইনপুট পড়বে না
`--arg NAME VAL`	শেলের মান jq ভেরিয়েবল `$NAME` হিসেবে দাও
`-e`	শেষ ফলাফল `false` / `null` হলে non-zero exit status

২৩.১৫.৪ উদাহরণ

```
$ jq '.[0].name' users.json
"Rahim"

$ jq -r '.[0].name' users.json
Rahim

$ jq -r '.[].name' users.json
Rahim
Karim
Sumi

$ jq 'length' users.json
3

$ jq -r '.[] | select(.city == "Dhaka") | .name' users.json
Rahim
Sumi

$ jq -r '.[] | select(.age > 26) | "\(.name) is \(.age)"' users.json
Karim is 31
Sumi is 28

$ jq -c '.[] | {name, skill_count: (.skills | length)}' users.json
{"name":"Rahim","skill_count":2}
{"name":"Karim","skill_count":1}
{"name":"Sumi","skill_count":3}

$ jq -c 'map(.name)' users.json
["Rahim","Karim","Sumi"]

$ jq '.[0] | keys' users.json
[
  "age",
  "city",
  "name",
```

```
  "skills"
]
```

`"\(.name)"` হলো **string interpolation** — স্ট্রিং-এর ভেতরে মান বসানো। আর `{name}` হলো `{name: .name}` এর সংক্ষিপ্ত রূপ।

যার `linux` স্কিল আছে:

```
$ jq -r '.[] | select(.skills | index("linux")) | .name' users.json
Rahim
Sumi
```

শেল ভেরিয়েবল নিরাপদে পাঠানো:

```
$ city=Khulna
$ jq -r --arg c "$city" '.[] | select(.city == $c) | .name' users.json
Karim
```

JSON সুন্দর করে দেখা বা বৈধ কিনা যাচাই:

```
jq . messy.json                          # সুন্দর করে দেখাও; ভুল JSON হলে parse error
curl -s https://api.github.com/repos/jqlang/jq | jq -r '.stargazers_count'
```

> 💡 **টিপস:** `jq` -এর আউটপুট শেলের অন্যান্য টুলে পাঠাতে প্রায় সবসময় `-r` লাগে। নাহলে প্রতিটা স্ট্রিং `"..."` উদ্ধৃতি সহ আসবে।

> 📝 **মনে রাখো:** YAML-এর জন্য একই রকম টুল `yq` আছে; আর CSV-র জন্য `mlr` (Miller)। তবে এগুলো আলাদা প্যাকেজ।

২৩.১৬ পাইপলাইন দিয়ে বাস্তব সমস্যার সমাধান

এবার আসল মজা — ছোট ছোট টুল জুড়ে বড় সমস্যা সমাধান। একজন লিনাক্স এক্সপার্ট আর নতুন ইউজারের সবচেয়ে বড় পার্থক্য এখানেই: এক্সপার্ট মাথায় পাইপলাইন সাজিয়ে ফেলে।

২৩.১৬.১ ওয়েব সার্ভার লগ থেকে top 10 IP

Nginx/Apache-এর access log-এর প্রতিটি লাইন দেখতে এরকম (combined format):

```
203.0.113.5 - - [12/Nov/2026:10:15:32 +0600] "GET /index.html HTTP/1.1" 200 5320 "-"
"Mozilla/5.0"
```

প্রথম ফিল্ড হলো ক্লায়েন্টের IP। কোন ১০টা IP সবচেয়ে বেশি রিকোয়েস্ট পাঠিয়েছে?

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10
```

```
$ awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10
   4521 203.0.113.5
   2210 198.51.100.23
   1876 192.0.2.44
    953 203.0.113.77
    611 198.51.100.8
    402 192.0.2.10
    388 203.0.113.91
    250 198.51.100.150
    199 192.0.2.201
    143 203.0.113.12
```

ধাপে ধাপে কী হচ্ছে:

```
access.log (লক্ষ লাইন)
    │
    ▼  awk '{print $1}'      প্রতিটি লাইনের প্রথম ফিল্ড (IP) বের করো
203.0.113.5
198.51.100.23
203.0.113.5
    │
    ▼  sort                   একই IP পাশাপাশি আনো (uniq-এর শর্ত!)
198.51.100.23
203.0.113.5
203.0.113.5
    │
    ▼  uniq -c                প্রতিটি IP গোনো
      1 198.51.100.23
      2 203.0.113.5
    │
    ▼  sort -rn               সংখ্যা ধরে বড় থেকে ছোট
      2 203.0.113.5
      1 198.51.100.23
    │
    ▼  head -10               প্রথম ১০টা
```

`awk`-এর বদলে `cut -d ' ' -f 1` দিলেও চলে, কারণ IP-র আগে কোনো স্পেস নেই।

> 💡 **টিপস:** লগ rotate হয়ে `.gz` ফাইলে চলে গেলে `zcat -f /var/log/nginx/access.log*` দিয়ে সব একসাথে পড়ো। Ubuntu/Debian-এ Nginx লগ `/var/log/nginx/`, Apache-এর `/var/log/apache2/`; RHEL/Fedora-তে Apache-এর `/var/log/httpd/`। লগ নিয়ে বিস্তারিত অধ্যায় ২৮-এ।

২৩.১৬.২ আরও কয়েকটা দরকারি রেসিপি

HTTP status code-এর হিসাব (combined format-এ ৯ নম্বর ফিল্ড):

```
$ awk '{print $9}' access.log | sort | uniq -c | sort -rn
  15230 200
    812 404
    301 304
     27 500
```

সবচেয়ে বেশি চাওয়া ১০টা URL (৭ নম্বর ফিল্ড):

```
awk '{print $7}' access.log | sort | uniq -c | sort -rn | head -10
```

কোন IP সবচেয়ে বেশি 404 পাচ্ছে (স্ক্যানার/বট ধরার জন্য):

```
awk '$9 == 404 {print $1}' access.log | sort | uniq -c | sort -rn | head
```

একটা লেখায় সবচেয়ে বেশি ব্যবহৃত ১০টা শব্দ:

```
tr -cs '[:alpha:]' '\n' < article.txt | tr '[:upper:]' '[:lower:]' \
  | sort | uniq -c | sort -rn | head -10
```

SSH-এ ব্যর্থ লগইনের চেষ্টা কোন IP থেকে বেশি (Debian/Ubuntu-তে `/var/log/auth.log`, RHEL/Fedora-তে `/var/log/secure`; journald-only সিস্টেমে `journalctl -u ssh` / `-u sshd`):

```
grep "Failed password" /var/log/auth.log \
  | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' \
  | sort | uniq -c | sort -rn | head
```

কোন shell কতজন ইউজার ব্যবহার করে:

```
$ cut -d ':' -f 7 /etc/passwd | sort | uniq -c | sort -rn
     18 /usr/sbin/nologin
      2 /bin/bash
      1 /bin/sync
```

বিভাগভিত্তিক মোট বেতন (`awk` দিয়ে যোগ, `sort` দিয়ে সাজানো):

```
$ tail -n +2 employees.csv | awk -F ',' '{sum[$3] += $4} END {for (d in sum) print
sum[d], d}' | sort -rn
116000 IT
87000 HR
38000 Sales
```

ফোল্ডারে কোন এক্সটেনশনের ফাইল কতগুলো:

```
find . -type f -name '*.*' | rev | cut -d '.' -f 1 | rev | sort | uniq -c | sort -rn
```

দুটো সার্ভারের ইনস্টল করা প্যাকেজের পার্থক্য (Debian/Ubuntu):

```
comm -3 <(ssh server1 "dpkg-query -W -f='\${Package}\n'" | sort) \
        <(ssh server2 "dpkg-query -W -f='\${Package}\n'" | sort)
```

৭ দিনের পুরনো `.log` ফাইল খুঁজে কম্প্রেস (সমান্তরালে):

```
find /var/log/myapp -name '*.log' -mtime +7 -print0 | xargs -0 -r -P 4 gzip
```

২৩.১৬.৩ পাইপলাইন ডিবাগ করার কৌশল

1. **এক এক করে বাড়াও।** প্রথমে শুধু প্রথম কমান্ড `| head` দিয়ে চালাও, আউটপুট ঠিক থাকলে পরের অংশ যোগ করো।
2. **মাঝখানে উঁকি দাও** `tee` দিয়ে: `... | tee /tmp/step2.txt | ...`।
3. **ছোট নমুনায় পরীক্ষা করো:** `head -1000 big.log | ...`।
4. **exit status দেখো:** bash-এ পাইপলাইনের প্রতিটি অংশের exit status `${PIPESTATUS[@]}` অ্যারেতে থাকে; আর স্ক্রিপ্টে `set -o pipefail` দিলে মাঝের কোনো কমান্ড ব্যর্থ হলে পুরো পাইপলাইন ব্যর্থ ধরা হয় (অধ্যায় ৪৩)।

```
$ false | true; echo "${PIPESTATUS[@]}"
1 0
```

২৩.১৭ প্রাসঙ্গিক কীবোর্ড শর্টকাট

এই টুলগুলোর নিজস্ব শর্টকাট নেই (এরা ইন্টারঅ্যাক্টিভ না), কিন্তু এদের সাথে কাজ করার সময় এই শর্টকাটগুলো বারবার লাগে:

শর্টকাট	কাজ
`Ctrl+D`	ইনপুট শেষ (EOF) — যেমন ফাইল ছাড়া `sort` চালিয়ে হাতে লাইন লিখে শেষে চাপো, তখন সে সাজিয়ে দেখাবে
`Ctrl+C`	চলমান পাইপলাইন থামাও (পাইপলাইনের সব প্রসেস SIGINT পায়)
`Ctrl+Z`	পাইপলাইন সাময়িক থামিয়ে পেছনে রাখো (`fg` দিয়ে ফেরত — অধ্যায় ২৫)
`Ctrl+S` / `Ctrl+Q`	টার্মিনাল আউটপুট থামাও / আবার চালু (ভুলে Ctrl+S চাপলে স্ক্রিন জমে গেছে মনে হয়!)
`Ctrl+V` তারপর `Tab`	কমান্ড লাইনে আসল TAB অক্ষর লেখো (যেমন `cut -d '<TAB>'` ; বিকল্প bash-এ `$'\t'`)
`Ctrl+R`	হিস্ট্রিতে আগে লেখা লম্বা পাইপলাইন খুঁজে বের করো
`Alt+.`	আগের কমান্ডের শেষ argument (যেমন ফাইলের নাম) বসাও

শর্টকাট	কাজ
`Ctrl+X Ctrl+E`	লম্বা পাইপলাইন এডিটরে (nano/vim) খুলে আরামে লেখো
`Ctrl+L`	স্ক্রিন পরিষ্কার
`q`	`less`-এ পাইপ করা আউটপুট (`...

TAB delimiter দেওয়ার উদাহরণ:

```
cut -d $'\t' -f 2 data.tsv
sort -t $'\t' -k 3,3n data.tsv
```

সব শর্টকাট এক জায়গায় পাবে পরিশিষ্ট ক-তে, আর টার্মিনাল শর্টকাটের বিস্তারিত অধ্যায় ৬-এ।

হাতে-কলমে অনুশীলন

ল্যাব ১: কর্মীদের ডেটা বিশ্লেষণ

`~/textlab/employees.csv` ব্যবহার করো।

1. শুধু নাম আর শহর দেখাও:

   ```
   cut -d ',' -f 2,5 employees.csv
   ```

2. হেডার বাদ দিয়ে বেতন অনুযায়ী ছোট থেকে বড় সাজাও:

   ```
   tail -n +2 employees.csv | sort -t ',' -k 4,4n
   ```

3. কোন শহরে কতজন কর্মী:

   ```
   tail -n +2 employees.csv | cut -d ',' -f 5 | sort | uniq -c | sort -rn
   ```

   প্রত্যাশিত: Dhaka 3, তারপর Chittagong 1, Khulna 1।

4. সর্বোচ্চ বেতনের মানুষের নাম (এক লাইনে):

   ```
   tail -n +2 employees.csv | sort -t ',' -k 4,4nr | head -1 | cut -d ',' -f 2
   ```

5. পুরো ফাইল টেবিল আকারে: `column -t -s ',' employees.csv`

ল্যাব ২: tr দিয়ে লেখা পরিষ্কার

```
echo "  Hello,,   WORLD!!   2026  " > messy.txt
tr -s ' ' < messy.txt                       # স্পেস চেপে এক
tr -d '[:punct:]' < messy.txt               # যতিচিহ্ন মুছো
```

```
tr '[:upper:]' '[:lower:]' < messy.txt      # ছোট হাতের
tr -cd '[:digit:]\n' < messy.txt            # শুধু সংখ্যা
```

ল্যাব ৩: diff আর patch

```
cd ~/textlab
printf 'port=80\nhost=localhost\ndebug=true\n' > old.conf
cp old.conf server.conf
printf 'port=8080\nhost=localhost\ndebug=false\n' > new.conf

diff -u old.conf new.conf > fix.patch
cat fix.patch
patch --dry-run server.conf < fix.patch     # আগে পরীক্ষা
patch -b server.conf < fix.patch            # প্রয়োগ, ব্যাকআপ server.conf.orig
diff server.conf new.conf && echo "একই হয়ে গেছে"
patch -R server.conf < fix.patch            # ফিরিয়ে নাও
cat server.conf                             # আবার port=80
```

ল্যাব ৪: xargs-এর বিভিন্ন রূপ

```
mkdir -p ~/textlab/xa && cd ~/textlab/xa
seq 1 5 | xargs -I {} touch "file {}.txt"   # নামে স্পেস!
ls
find . -name '*.txt' | xargs ls -l          # ভুল: স্পেসে ভেঙে যাবে
find . -name '*.txt' -print0 | xargs -0 ls -l   # ঠিক
find . -name '*.txt' -print0 | xargs -0 -n 2 echo   # দুটো করে
find . -name '*.txt' -print0 | xargs -0 echo rm     # শুকনো পরীক্ষা
find . -name '*.txt' -print0 | xargs -0 rm          # আসল মুছে ফেলা
```

ল্যাব ৫: comm দিয়ে সেট অপারেশন

```
cd ~/textlab
printf 'ali\nbabu\nchandan\ndipu\n' | sort > class_a.txt
printf 'babu\ndipu\nemon\n'          | sort > class_b.txt
comm -12 class_a.txt class_b.txt     # দুই ক্লাসেই: babu, dipu
comm -23 class_a.txt class_b.txt     # শুধু A: ali, chandan
comm -13 class_a.txt class_b.txt     # শুধু B: emon
```

ল্যাব ৬: নকল লগ বানিয়ে top 10 IP

```
cd ~/textlab
for i in $(seq 1 500); do
  echo "192.0.2.$(shuf -i 1-20 -n 1) - - [12/Nov/2026:10:00:00 +0600] \"GET /
HTTP/1.1\" 200 512 \"-\" \"curl\""
done > access.log

wc -l access.log                                                # 500
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10 # top 10
awk '{print $1}' access.log | sort -u | wc -l                   # মোট আলাদা IP
```

ল্যাব ৭: split, csplit ও jq

```
cd ~/textlab
seq 1 2500 > numbers.txt
split -l 1000 -d numbers.txt part_ && wc -l part_*
cat part_* | cmp -s - numbers.txt && echo "জোড়া লাগানো ঠিক আছে"

printf 'Intro\nChapter 1\nA\nChapter 2\nB\n' > book.txt
csplit -z -s -f chap_ book.txt '/^Chapter/' '{*}' && head chap_*

jq -r '.[] | select(.city == "Dhaka") | .name' users.json   # ২৩.১৫ এর ফাইল
```

`cmp -s - numbers.txt` এ `-` মানে stdin — পাইপ দিয়ে আসা জোড়া লাগানো ডেটা।

সারসংক্ষেপ

কমান্ড	কাজ	সবচেয়ে দরকারি রূপ
`cut`	কলাম/অক্ষর কাটা	`cut -d ',' -f 2,4 file`
`sort`	সাজানো	`sort -t ',' -k 3,3nr file`, `sort -h`, `sort -V`, `sort -u`
`uniq`	পাশাপাশি ডুপ্লিকেট সরানো/গোনা	`sort | uniq -c | sort -rn`
`wc`	লাইন/শব্দ/বাইট গোনা	`wc -l < file`
`tr`	অক্ষর বদলানো/মোছা/চাপা	`tr 'a-z' 'A-Z'`, `tr -d '\r'`, `tr -s ' '`
`paste`	পাশাপাশি জোড়া	`paste -d ',' a b`, `paste -sd '+'`
`join`	key ধরে মেলানো (সাজানো ইনপুট)	`join -t ',' -1 1 -2 1 a b`
`comm`	সাজানো তালিকার তুলনা	`comm -12 a b` (দুটোতেই)
`diff`	লাইন ধরে পার্থক্য	`diff -u old new > x.patch`, `diff -rq d1 d2`
`patch`	পার্থক্য প্রয়োগ	`patch -p1 --dry-run < x.patch`, `patch -R`
`cmp`	বাইট ধরে তুলনা	`cmp -s a b`
`xargs`	stdin → argument	`find ... -print0 | xargs -0 -P 4 cmd`, `xargs -I {} cmd {}`
`tee`	stdout + ফাইল	`cmd | tee -a log`, `| sudo tee /etc/file`
`rev`	লাইনের অক্ষর উল্টানো	`rev | cut -d. -f1 | rev`
`shuf`	এলোমেলো	`shuf -n 1 file`, `shuf -i 1-100 -n 5`
`fold`	লাইন ভাঙা	`fold -w 60 -s`

কমান্ড	কাজ	সবচেয়ে দরকারি রূপ
`fmt`	অনুচ্ছেদ সাজানো	`fmt -w 72`
`expand` / `unexpand`	ট্যাব ↔ স্পেস	`expand -t 4`, `unexpand -a`
`nl` / `column`	লাইন নম্বর / টেবিল	`nl -ba`, `column -t -s ','`
`split`	আকার/লাইন ধরে ভাগ	`split -l 1000 -d file part_`, `split -b 1G`
`csplit`	প্যাটার্ন ধরে ভাগ	`csplit -z -f p_ file '/^Chapter/' '{*}'`
`jq`	JSON প্রসেসিং	`jq -r '.[] | select(.x == "y") | .name'`

শর্টকাট / ট্রিক	কাজ
`Ctrl+D`	stdin শেষ (EOF)
`Ctrl+C`	পাইপলাইন থামাও
`Ctrl+V Tab` / `$'\t'`	আসল TAB অক্ষর
`Ctrl+X Ctrl+E`	লম্বা কমান্ড এডিটরে লেখো
`sort -o f f`	একই ফাইলে নিরাপদে সাজিয়ে রাখো
`LC_ALL=C sort`	দ্রুত, বাইট-ক্রমে সাজানো

নিজেকে যাচাই করো

1. `uniq` দিয়ে ডুপ্লিকেট গোনার আগে কেন `sort` করতে হয়? `uniq`-এর এই নকশার সুবিধা কী?
2. `sort -k 2` আর `sort -k 2,2` এর পার্থক্য কী?
3. `printf '10\n9\n100\n' | sort` এর আউটপুট কী হবে, আর কেন? সংখ্যা হিসেবে সাজাতে কী লাগবে?
4. `sort data.txt > data.txt` চালালে কী হয়? সঠিক উপায় কী?
5. `echo "a b" | cut -d ' ' -f 2` কেন খালি লাইন দেয়? দুটো সমাধান বলো।
6. `tr 'abc' 'xyz' file.txt` কেন কাজ করবে না?
7. `sudo echo "x" > /etc/somefile` কেন "Permission denied" দেয়, আর এর সমাধান কী?
8. স্পেসওয়ালা ফাইলের নাম নিরাপদে `xargs`-এ পাঠানোর উপায় কী?
9. `comm -12 a.txt b.txt` কী দেখায়? ইনপুট নিয়ে কোন শর্ত মানতে হয়?
10. git থেকে আসা একটা patch (`--- a/src/x.c`) প্রজেক্টের মূল ফোল্ডারে দাঁড়িয়ে প্রয়োগ করতে কোন অপশন লাগবে, আর প্রয়োগের আগে কী দিয়ে পরীক্ষা করবে?
11. access log থেকে top 10 IP বের করার পাইপলাইন লেখো, প্রতিটি ধাপের কাজসহ।
12. `jq '.name'` আর `jq -r '.name'` এর আউটপুটে পার্থক্য কী?

▼ উত্তর দেখো

1. `uniq` শুধু **পাশাপাশি** থাকা লাইন তুলনা করে — সে কেবল আগের একটা লাইন মনে রাখে। তাই একই লাইন ছড়িয়ে থাকলে ধরতে পারে না; `sort` সেগুলো পাশাপাশি এনে দেয়। সুবিধা: `uniq`-এর মেমরি প্রায় লাগে না, যত বড় ইনপুটই হোক streaming-এ কাজ করে।
2. `-k 2` মানে key ২ নম্বর ফিল্ড থেকে **লাইনের শেষ পর্যন্ত**; `-k 2,2` মানে key শুধু ২ নম্বর ফিল্ড। প্রায় সবসময় `-k N,N` চাই।
3. `10`, `100`, `9` — কারণ ডিফল্টে টেক্সট হিসেবে অক্ষর ধরে তুলনা হয় (`'1'` < `'9'`)। সংখ্যা হিসেবে সাজাতে `sort -n`।
4. শেল `sort` চালানোর আগেই `>` দিয়ে ফাইলটা খালি করে দেয়, ফলে ডেটা হারায়। সঠিক: `sort -o data.txt data.txt`।
5. `cut` প্রতিটি স্পেসকে delimiter ধরে, পরপর দুই স্পেসের মাঝে একটা খালি ফিল্ড হয় — ২ নম্বর ফিল্ড খালি। সমাধান: `tr -s ' ' | cut -d ' ' -f 2`, অথবা `awk '{print $2}'`।
6. `tr` ফাইলের নাম নেয় না, শুধু stdin পড়ে। লিখতে হবে `tr 'abc' 'xyz' < file.txt`।
7. `sudo` শুধু `echo`-কে root বানায়; `>` রিডাইরেকশন করে সাধারণ ইউজারের শেল, যার ফাইলে লেখার অনুমতি নেই। সমাধান: `echo "x" | sudo tee /etc/somefile > /dev/null` (যোগ করতে `tee -a`)।
8. `find ... -print0 | xargs -0 cmd` — নামগুলো NUL অক্ষর দিয়ে আলাদা হয়, যা ফাইলের নামে থাকতে পারে না।
9. যে লাইনগুলো দুই ফাইলেই আছে (ছেদ)। দুটো ফাইলই একই locale-এ **সাজানো** থাকতে হবে।
10. `patch -p1` (পাথ থেকে `a/` বা `b/` অংশ বাদ দেয়)। আগে `patch -p1 --dry-run < file.patch` দিয়ে পরীক্ষা।
11. `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10` — `awk` প্রতিটি লাইনের IP বের করে; `sort` একই IP পাশাপাশি আনে; `uniq -c` প্রতিটি IP গোনে; `sort -rn` গণনা অনুযায়ী বড় থেকে ছোট সাজায়; `head -10` প্রথম ১০টা দেখায়।
12. `jq '.name'` JSON স্ট্রিং হিসেবে উদ্ধৃতি সহ দেখায় (`"Rahim"`); `-r` (raw) উদ্ধৃতি ছাড়া দেখায় (`Rahim`), যা শেলের অন্য কমান্ডে ব্যবহারের উপযোগী।

অধ্যায় ২৪: প্রসেস ম্যানেজমেন্ট

তুমি যখন টার্মিনালে `ls` লিখে Enter চাপো, ব্রাউজারে একটা ট্যাব খোলো, বা ব্যাকগ্রাউন্ডে কোনো ওয়েব সার্ভার চলে, প্রতিটা ক্ষেত্রেই লিনাক্সের ভেতরে একটা করে **process** তৈরি হয়। process বুঝলে বোঝা যায় কম্পিউটার আসলে "কী করছে"। সার্ভার হঠাৎ ধীর হয়ে গেলে, কোনো প্রোগ্রাম আটকে গেলে বা মেমরি ফুরিয়ে গেলে প্রথম যে দক্ষতা কাজে লাগে, তা হলো প্রসেস ম্যানেজমেন্ট।

এই অধ্যায়ে যা শিখবে

- program, process আর thread-এর পার্থক্য
- PID, PPID কী, প্রসেস ট্রি কীভাবে গঠিত হয়, PID 1 কেন বিশেষ
- `fork()`, `exec()`, `wait()`, `exit()` দিয়ে প্রসেসের জন্ম থেকে মৃত্যু পর্যন্ত জীবনচক্র
- প্রসেসের state: `R`, `S`, `D`, `Z`, `T` ও আরও কয়েকটি
- zombie আর orphan প্রসেস কী, কেন হয়, কী করতে হয়
- `ps` কমান্ড: `aux`, `-ef`, `-o`, `--forest`, sorting
- `top` আর `htop`, সাথে প্রায় সব interactive শর্টকাট
- `pgrep`, `pkill`, `pidof` দিয়ে প্রসেস খোঁজা
- signals: SIGHUP, SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT ইত্যাদি
- `kill`, `killall` দিয়ে প্রসেস থামানো বা সিগন্যাল পাঠানো
- `nice`, `renice` দিয়ে priority বদলানো
- `/proc/PID` ফোল্ডারের ভেতরে কী আছে
- load average পড়া ও বোঝা

২৪.১ program, process আর thread

program: রেসিপির বই

program হলো ডিস্কে রাখা একটা ফাইল, যেমন `/usr/bin/ls` বা `/usr/bin/python3`। এর ভেতরে আছে মেশিন কোড (CPU যে নির্দেশ বোঝে)। ফাইলটা চুপচাপ পড়ে থাকে, কোনো কাজ করে না।

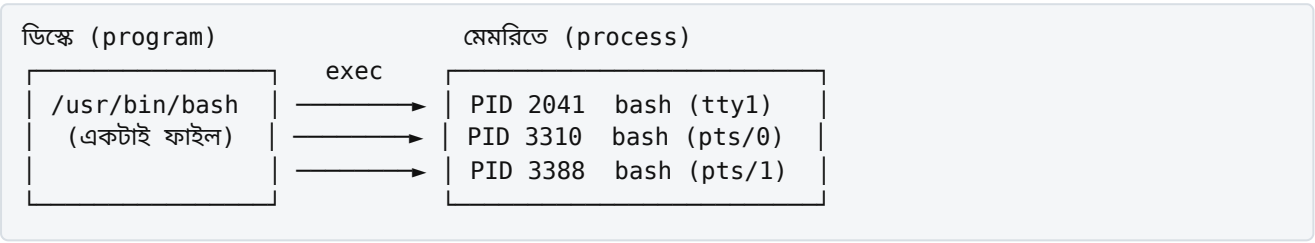
উপমা: রান্নার রেসিপির বই। বইটা তাকে থাকলে কোনো রান্না হয় না।

process: রান্না চলছে

process হলো চলমান program, মানে program-এর একটা জীবন্ত instance। kernel যখন program চালায়, তখন সে:

- মেমরিতে জায়গা দেয় (code, data, heap, stack)
- একটা ইউনিক নম্বর দেয়: **PID** (Process ID)
- খোলা ফাইলের তালিকা (file descriptor) রাখে
- কোন user চালাচ্ছে, কোন ডিরেক্টরিতে আছে, environment variable কী, এসব হিসাব রাখে
- CPU-তে কখন কতক্ষণ চলবে তা scheduler ঠিক করে

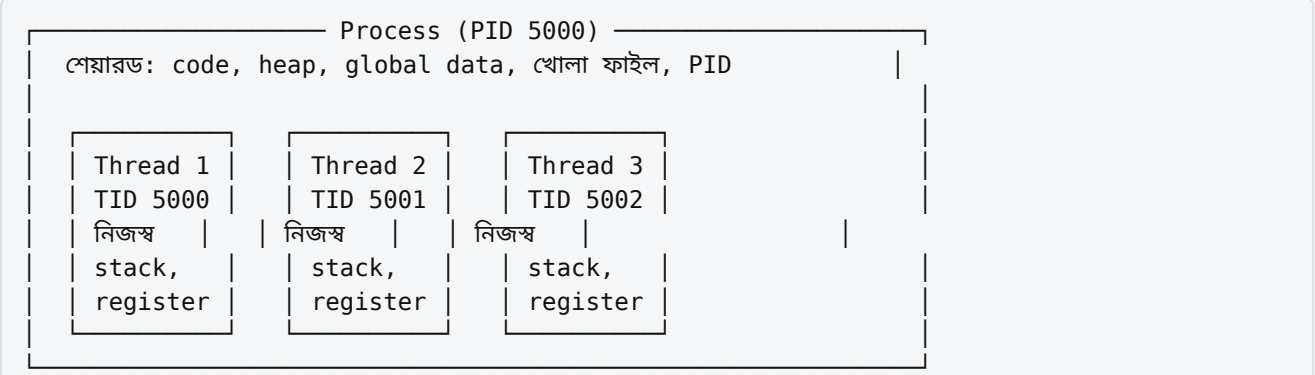
উপমা: রেসিপি দেখে রান্নাঘরে রান্না চলছে। একই রেসিপি দিয়ে তিনটা চুলায় তিনবার রান্না হতে পারে। ঠিক তেমনি একই `bash` program থেকে একসাথে দশটা আলাদা bash process চলতে পারে, প্রতিটার নিজস্ব PID আর নিজস্ব মেমরি।



thread: এক রান্নাঘরে কয়েকজন রাঁধুনি

একটা process-এর ভেতরে এক বা একাধিক **thread** (কাজের সুতো) থাকতে পারে। একই process-এর সব thread একই মেমরি, একই খোলা ফাইল শেয়ার করে, কিন্তু প্রত্যেকের নিজস্ব stack আর CPU register থাকে। তাই তারা একসাথে আলাদা আলাদা CPU core-এ চলতে পারে।

উপমা: একটা রান্নাঘরে (process) তিনজন রাঁধুনি (thread)। একজন সবজি কাটছে, একজন ভাত রাঁধছে, একজন মসলা বাটছে। ফ্রিজ, চুলা, হাঁড়ি সবাই শেয়ার করছে।



> 📝 **মনে রাখো:** লিনাক্স kernel-এর চোখে thread আর process প্রায় একই জিনিস। দুটোই একটা **task**, তৈরি হয় `clone()` system call দিয়ে। পার্থক্য শুধু কতটা জিনিস শেয়ার করা হলো। প্রথম thread-এর TID (Thread ID) আর process-এর PID একই হয়। `ps` -এ thread-কে **LWP** (Light Weight Process) বলে দেখানো হয়।

বিষয়	program	process	thread
কোথায় থাকে	ডিস্কে	মেমরিতে, চলমান	process-এর ভেতরে
পরিচয়	ফাইলের path	PID	TID (LWP)
মেমরি	নেই	নিজস্ব, আলাদা	process-এর সাথে শেয়ারড
একটা crash করলে	প্রযোজ্য নয়	অন্য process নিরাপদ	পুরো process মারা যেতে পারে
তৈরি হয়	compiler/installer দিয়ে	`fork()` + `exec()`	`clone()` (pthread লাইব্রেরি)

একটা process-এ কয়টা thread আছে দেখতে:

```
$ ps -o pid,nlwp,cmd -p 1
    PID NLWP CMD
      1    1 /sbin/init splash

$ ps -T -p $(pidof -s firefox)
    PID    SPID TTY          TIME CMD
   7123    7123 ?        00:01:12 firefox
   7123    7130 ?        00:00:00 gmain
   7123    7131 ?        00:00:03 gdbus
   ...
```

`NLWP` = Number of LWP, মানে thread সংখ্যা। `-T` দিলে প্রতিটা thread আলাদা লাইনে দেখায়, `SPID` হলো thread ID। সব process-এর সব thread একসাথে দেখতে `ps -eLf`।

২৪.২ PID ও PPID

প্রতিটা process-এর দুটো গুরুত্বপূর্ণ নম্বর থাকে:

- **PID** (Process ID): তার নিজের পরিচয় নম্বর।
- **PPID** (Parent Process ID): যে process তাকে জন্ম দিয়েছে, তার PID।

লিনাক্সে প্রতিটা process (PID 1 বাদে) অন্য কোনো process থেকে জন্মায়। তাই সবার একটা parent আছে।

```
$ echo $$          # বর্তমান শেলের PID
3310
$ echo $PPID       # বর্তমান শেলের parent-এর PID
3295
$ ps -o pid,ppid,cmd -p $$
    PID    PPID CMD
   3310    3295 bash
$ ps -o pid,ppid,cmd -p 3295
    PID    PPID CMD
   3295    1890 /usr/libexec/gnome-terminal-server
```

তাহলে বোঝা গেল: gnome-terminal (3295) একটা bash (3310) চালিয়েছে।

শেলের কয়েকটা বিশেষ variable:

variable	মানে
`$$`	বর্তমান শেলের PID
`$PPID`	বর্তমান শেলের parent-এর PID
`$!`	সবশেষে ব্যাকগ্রাউন্ডে (`&` দিয়ে) চালানো process-এর PID
`$BASHPID`	বর্তমান bash process-এর আসল PID (subshell-এর ভেতরে `$$` থেকে আলাদা হতে পারে)

```
$ sleep 100 &
[1] 4521
$ echo $!
4521
```

PID কীভাবে দেওয়া হয়

- kernel ক্রমানুসারে বাড়তে থাকা নম্বর দেয়। সর্বোচ্চ সীমায় পৌঁছালে আবার ছোট নম্বর থেকে শুরু করে (যেগুলো খালি)।
- সর্বোচ্চ সীমা থাকে `/proc/sys/kernel/pid_max`-এ। kernel-এর নিজস্ব ডিফল্ট `32768`, তবে Ubuntu 24.04, Fedora, RHEL 9-এর মতো আধুনিক 64-bit সিস্টেমে systemd এটা সাধারণত `4194304` করে দেয়।

```
$ cat /proc/sys/kernel/pid_max
4194304
```

বিশেষ PID	কে
0	idle task (ঐতিহাসিক নাম "swapper")। এটা kernel-এর ভেতরে থাকে, `ps`-এ দেখা যায় না।
1	init process, আধুনিক distro-তে **systemd**
2	`kthreadd`, সব kernel thread-এর parent

> ⚠️ **সতর্কতা:** PID স্থায়ী নয়। একটা process মরে গেলে কিছুক্ষণ পর তার PID অন্য নতুন process পেতে পারে। তাই স্ক্রিপ্টে পুরোনো PID ফাইলে লিখে রেখে অনেকক্ষণ পর `kill` করলে ভুল process মারা যেতে পারে।

২৪.৩ প্রসেস ট্রি ও PID 1

সব process মিলে একটা গাছ (tree) তৈরি হয়। গাছের গোড়ায় থাকে **PID 1**।

```
                    systemd (1)
```

```
   systemd-journald   sshd (812)        gdm (950)     kthreadd (2) ← kernel threads
                          │                │              ├── kworker/0:1
                    sshd: nishat (2101)  gnome-session      ├── ksoftirqd/0
                          │                │              └── ...
                      bash (2120)     gnome-terminal (3295)
                          │                │
                      vim (2210)        bash (3310)
                                           │
                                        pstree (3401)
```

PID 1 কেন বিশেষ

PID 1 হলো প্রথম user-space process। kernel বুট শেষে এটা চালায় (বিস্তারিত অধ্যায় ২৭)। আধুনিক Ubuntu, Debian, Fedora, RHEL সবখানে এটা **systemd**। এর দায়িত্ব:

1. বাকি সব সার্ভিস চালু করা (অধ্যায় ২৬)।
2. **অনাথ (orphan) process দত্তক নেওয়া** এবং তাদের মৃত্যুর পর পরিষ্কার করা (reap)।
3. এটা মরতে পারে না। PID 1 মারা গেলে kernel "Attempted to kill init!" বলে **kernel panic** করে।
4. kernel PID 1-কে বিশেষ সুরক্ষা দেয়: যে signal-এর জন্য PID 1 নিজে handler বসায়নি, সেগুলো তার কাছে পৌঁছায় না। তাই root হয়েও `kill -9 1` দিলে systemd মরে না।

```
$ ps -p 1 -o pid,comm
    PID COMMAND
      1 systemd
$ ls -l /sbin/init
lrwxrwxrwx 1 root root 22 ... /sbin/init -> ../lib/systemd/systemd
```

pstree: গাছটা চোখে দেখো

```
$ pstree
systemd─┬─ModemManager───3*[{ModemManager}]
        ├─NetworkManager───3*[{NetworkManager}]
        ├─cron
        ├─sshd───sshd───sshd───bash───pstree
        ├─systemd-journal
        ...
```

`3*[{ModemManager}]` মানে ModemManager-এর ভেতরে ৩টা thread (কোঁকড়া বন্ধনী `{}` = thread)। একই রকম অনেকগুলো child থাকলে `N*[...]` আকারে গুটিয়ে দেখায়।

অপশন	কাজ
`-p`	প্রতিটা নামের পাশে PID দেখায় (এতে গুটানো বন্ধ হয়)
`-a`	পুরো command line (arguments সহ) দেখায়
`-u`	user বদলালে বন্ধনীতে user নাম দেখায়
`-h`	বর্তমান process ও তার পূর্বপুরুষদের highlight করে

অপশন	কাজ
`-s PID`	ওই PID-এর parent-দের (ancestor) দেখায়
`-T`	thread লুকায় (নতুন psmisc ভার্সনে)
`-n`	নাম না, PID অনুযায়ী সাজায়
`-c`	একই রকম subtree গুটায় না
`USER` বা `PID`	শুধু ওই user বা ওই PID থেকে শুরু হওয়া গাছ

```
$ pstree -p $$
bash(3310)──pstree(3420)

$ pstree -s -p $$
systemd(1)──systemd(1720)──gnome-terminal-(3295)──bash(3310)──pstree(3421)
```

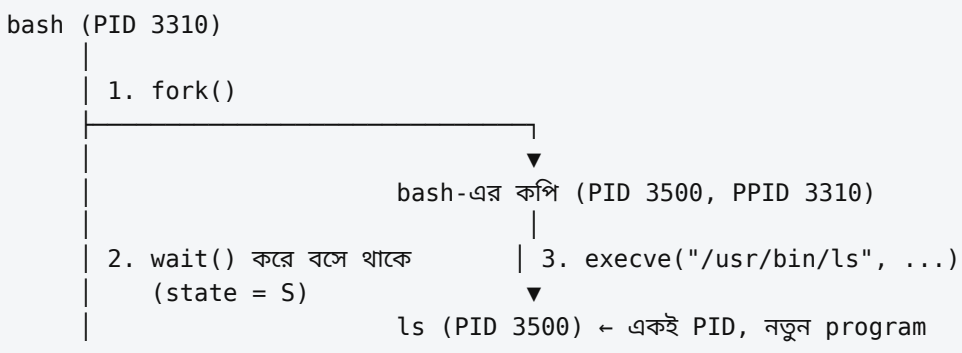
> 💡 **টিপস:** `pstree` আছে `psmisc` প্যাকেজে। মিনিমাল সার্ভারে না থাকলে `sudo apt install psmisc` (Debian/Ubuntu) বা `sudo dnf install psmisc` (Fedora/RHEL)। বিকল্প হিসেবে `ps -ef --forest` সব জায়গায় কাজ করে।

২৪.৪ process-এর জীবনচক্র: fork, exec, wait, exit

লিনাক্সে নতুন process তৈরির মূল কৌশল দুই ধাপের, যেটা Unix-এর একটা সুন্দর ডিজাইন:

1. **`fork()`**: parent নিজের একটা প্রায়-হুবহু কপি বানায়। নতুন child-এর PID আলাদা, PPID = parent-এর PID। বাকি সব (মেমরি, খোলা ফাইল, environment) কপি হয়।
2. **`exec()`** (আসলে `execve()`): child তার নিজের মেমরি মুছে নতুন program লোড করে। PID একই থাকে, কিন্তু ভেতরের code বদলে যায়।
3. **`exit()`**: child কাজ শেষে exit code (0 = সফল, অন্য সংখ্যা = সমস্যা) দিয়ে বের হয়ে যায়।
4. **`wait()`** (বা `waitpid()`): parent child-এর exit code সংগ্রহ করে। তখনই kernel child-এর শেষ চিহ্ন মুছে ফেলে।

উদাহরণ: শেলে `ls` লিখলে ভেতরে কী হয়



```
        │                                │
        │                                │ 4. ফাইলের তালিকা প্রিন্ট করে
        │                                │ 5. exit(0)
        │                                ▼
        │                        [zombie: শুধু exit code বাকি]
        │ 6. wait() exit code পায় ◄─────┘
        │    kernel child মুছে দেয়
        ▼
   আবার prompt দেখায়:  $
   echo $? → 0
```

উপমা: fork মানে নিজের ফটোকপি তৈরি করা। exec মানে সেই ফটোকপির ওপর পুরো নতুন লেখা ছাপিয়ে দেওয়া। wait মানে মা অপেক্ষা করছে সন্তান কাজ শেষ করে রিপোর্ট দেবে।

কেন দুই ধাপে?

fork আর exec-এর মাঝখানে child-এর হাতে একটা সুযোগ থাকে, নিজের পরিবেশ ঠিক করে নেওয়ার। শেল ঠিক এই ফাঁকেই redirection আর pipe সেট করে (অধ্যায় ১৯)। যেমন `ls > out.txt` চালালে child fork-এর পর stdout-কে `out.txt`-এ ঘুরিয়ে দেয়, তারপর exec করে। `ls` নিজে জানেই না যে তার output ফাইলে যাচ্ছে।

copy-on-write: কপি আসলে সস্তা

fork-এ পুরো মেমরি কপি করা খুব ব্যয়বহুল মনে হতে পারে। কিন্তু kernel **copy-on-write (COW)** ব্যবহার করে: শুরুতে parent আর child একই মেমরি page শেয়ার করে (read-only হিসেবে চিহ্নিত)। কেউ লিখতে গেলে তখনই শুধু ওই page কপি হয়। যেহেতু child সাধারণত সাথে সাথেই exec করে, বেশিরভাগ page কখনো কপিই হয় না।

builtin কমান্ডে fork হয় না

`cd`, `echo`, `export` এগুলো shell builtin। শেল নিজের ভেতরেই চালায়, নতুন process বানায় না। `cd` যদি আলাদা process হতো, তাহলে child-এর ডিরেক্টরি বদলাত, parent শেলের না, তাই কোনো লাভই হতো না।

```
$ type cd ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
```

`exec` কমান্ড: fork ছাড়া exec

শেলে `exec` builtin দিলে fork হয় না, বর্তমান শেলটাই নতুন program দিয়ে বদলে যায়:

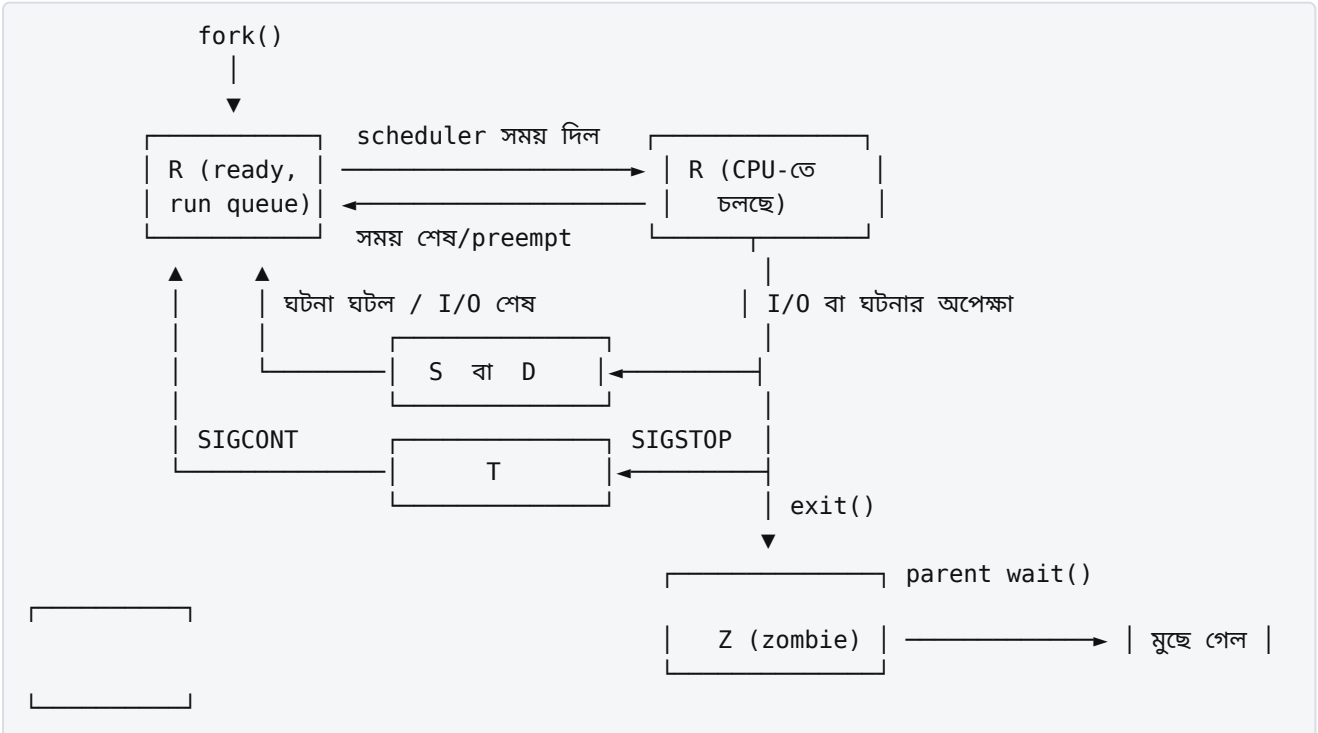
```
$ exec top     # top থেকে q চাপলে টার্মিনাল ট্যাবটাই বন্ধ হয়ে যাবে, কারণ শেল আর নেই
```

২৪.৫ প্রসেসের state (অবস্থা)

একটা process সবসময় CPU-তে চলে না। বেশিরভাগ সময় সে কোনো কিছুর জন্য অপেক্ষা করে। kernel প্রতিটা process-এর একটা state রাখে। `ps`-এর `STAT` কলাম আর `top`-এর `S` কলামে এটা দেখা যায়।

কোড	নাম	মানে	উপমা
`R`	Running / Runnable	CPU-তে চলছে, বা চলার জন্য লাইনে (run queue) দাঁড়িয়ে	কাউন্টারে সার্ভ হচ্ছে বা লাইনে আছে
`S`	Interruptible Sleep	কোনো ঘটনার জন্য অপেক্ষা (কীবোর্ড ইনপুট, নেটওয়ার্ক, টাইমার)। signal এলে জেগে ওঠে	ফোনের অপেক্ষায় ঘুমাচ্ছে, ফোন বাজলেই উঠবে
`D`	Uninterruptible Sleep	সাধারণত disk/NFS I/O শেষ হওয়ার অপেক্ষা। signal-এও জাগে না, এমনকি SIGKILL-এও না	অপারেশন টেবিলে, মাঝপথে থামানো যাবে না
`T`	Stopped	SIGSTOP বা Ctrl+Z (SIGTSTP) দিয়ে থামানো	pause বাটন চাপা ভিডিও
`t`	Tracing stop	debugger (যেমন `gdb`, `strace`) থামিয়ে রেখেছে	
`Z`	Zombie	মারা গেছে, কিন্তু parent এখনো `wait()` করেনি	মৃত্যু সনদ জমা হয়নি
`I`	Idle	অলস kernel thread (load average-এ গোনা হয় না)	
`X`	Dead	একেবারে শেষ মুহূর্ত, প্রায় কখনো দেখা যায় না	

state transition ডায়াগ্রাম



STAT কলামের অতিরিক্ত অক্ষর

`ps aux`-এ STAT প্রায়ই `Ss`, `R+`, `Sl`, `SN` এমন দেখায়। প্রথম অক্ষর state, বাকিগুলো বাড়তি তথ্য:

অক্ষর	মানে
`<`	উচ্চ priority (nice মান ঋণাত্মক)
`N`	নিম্ন priority (nice মান ধনাত্মক)
`L`	কিছু page মেমরিতে lock করা
`s`	session leader (যেমন লগইন শেল)
`l`	multi-threaded
`+`	foreground process group-এ আছে (টার্মিনালে সামনে চলছে)

উদাহরণ: `Ssl` = ঘুমাচ্ছে, session leader, multi-threaded। `R+` = চলছে, টার্মিনালের সামনের সারিতে।

> ⚠️ **সতর্কতা:** অনেকগুলো process দীর্ঘ সময় `D` state-এ থাকলে সাধারণত disk বা নেটওয়ার্ক স্টোরেজ (NFS) সমস্যা বোঝায়। এদের `kill -9` দিয়েও মারা যায় না, কারণ kernel-এর ভেতরের I/O কাজ শেষ না হলে signal ডেলিভারিই হয় না। আসল সমস্যা (ডিস্ক, NFS সার্ভার) ঠিক করতে হয়।

২৪.৬ zombie ও orphan প্রসেস

zombie (Z): মরেও মরেনি

child যখন `exit()` করে, kernel তার মেমরি, খোলা ফাইল সব ছেড়ে দেয়। কিন্তু process table-এ একটা ছোট্ট এন্ট্রি (PID আর exit status) রেখে দেয়, যাতে parent পরে `wait()` করে জানতে পারে child কীভাবে মারা গেছে। এই মাঝখানের অবস্থাই **zombie**। ps-এ নামের পাশে `<defunct>` লেখা থাকে।

```
$ ps aux | awk '$8 ~ /^Z/'
nishat    5123  0.0  0.0      0     0 pts/0    Z+   10:22   0:00 [myapp] <defunct>
```

- সাধারণত zombie কয়েক মিলিসেকেন্ড থাকে, তারপর parent wait() করলেই চলে যায়।
- parent প্রোগ্রামে বাগ থাকলে (wait না করলে) zombie জমতে থাকে।
- zombie কোনো CPU বা মেমরি খায় না, কিন্তু একটা PID আটকে রাখে। হাজার হাজার জমলে PID ফুরিয়ে নতুন process তৈরি বন্ধ হয়ে যেতে পারে।

zombie মারার উপায়? zombie তো আগেই মৃত, তাই `kill -9` কাজ করে না। উপায়:

1. parent-কে SIGCHLD পাঠাও, যদি সে সেটা পেয়ে wait() করে: `kill -s SIGCHLD <PPID>`

2. না হলে parent-কে মারো (বা restart করো)। তখন zombie গুলো orphan হয়ে PID 1-এর কাছে যায়, আর systemd সাথে সাথে wait() করে সাফ করে দেয়।

```
$ ps -o pid,ppid,stat,cmd -p 5123
    PID    PPID STAT CMD
   5123    5100 Z+   [myapp] <defunct>
$ kill 5100      # দায়ী parent-কে বন্ধ করো
```

orphan: অনাথ process

parent আগে মারা গেলে child হয় **orphan**। kernel তখন তাকে দত্তক দেয় PID 1-কে (বা কাছের কোনো "subreaper"-কে, যেমন user-এর `systemd --user` process)। orphan সম্পূর্ণ স্বাভাবিকভাবে চলতে থাকে, এতে কোনো ক্ষতি নেই। daemon (ব্যাকগ্রাউন্ড সার্ভিস) ইচ্ছা করেই এভাবে নিজেকে orphan বানাত।

```
$ bash -c 'sleep 300 & echo "child PID: $!"'
child PID: 6001
$ ps -o pid,ppid,cmd -p 6001
    PID    PPID CMD
   6001    1720 sleep 300        ← parent bash শেষ, এখন systemd --user (1720) দত্তক নিয়েছে
```

(ডেস্কটপে সাধারণত `systemd --user` দত্তক নেয়, সার্ভারে SSH সেশনে প্রায়ই PID 1 বা user manager।)

	zombie	orphan
জীবিত?	না, শুধু এন্ট্রি বাকি	হ্যাঁ, স্বাভাবিকভাবে চলছে
কারণ	parent wait() করেনি	parent আগেই মারা গেছে
state	`Z`	যেকোনো (R, S...)
ক্ষতিকর?	বেশি জমলে PID ফুরায়	না
সমাধান	parent ঠিক করো/মারো	দরকার নেই

২৪.৭ ps: প্রসেসের স্ন্যাপশট

`ps` (process status) কমান্ড চালানোর মুহূর্তের একটা ছবি দেয়। আসলে এটা `/proc` ফোল্ডার পড়ে সাজিয়ে দেখায়।

একটা মজার ইতিহাস আছে: `ps` তিন ধরনের অপশন স্টাইল বোঝে:

স্টাইল	উদাহরণ	উৎস
BSD	`ps aux` (ড্যাশ নেই)	BSD Unix
UNIX/POSIX	`ps -ef` (এক ড্যাশ)	System V Unix
GNU long	`ps --forest`, `--sort` (দুই ড্যাশ)	GNU

> ⚠️ **সতর্কতা:** `ps aux` আর `ps -aux` এক জিনিস নয়। `-aux` লিখলে ps সেটাকে "user x-এর process" হিসেবে বুঝতে পারে। সবসময় `ps aux` লেখো, ড্যাশ ছাড়া।

শুধু `ps`

```
$ ps
    PID TTY          TIME CMD
   3310 pts/0    00:00:00 bash
   3502 pts/0    00:00:00 ps
```

শুধু বর্তমান টার্মিনালের process দেখায়।

`ps aux`: সব process, BSD স্টাইল

- `a` = সব user-এর (টার্মিনাল আছে এমন) process
- `u` = user-কেন্দ্রিক বিস্তারিত কলাম
- `x` = টার্মিনাল ছাড়া process-ও (daemon)

```
$ ps aux | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1 167780 13120 ?        Ss   09:01   0:03 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    09:01   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    09:01   0:00
[pool_workqueue_release]
nishat      3310  0.0  0.0  11520  5300 pts/0    Ss   10:05   0:00 bash
```

কলাম	মানে
`USER`	কোন user-এর নামে চলছে
`PID`	process ID
`%CPU`	process-এর পুরো জীবনকাল ধরে গড় CPU ব্যবহার (তাৎক্ষণিক নয়!)
`%MEM`	মোট RAM-এর কত শতাংশ RSS
`VSZ`	virtual memory size (KiB), process যত মেমরি "চেয়ে রেখেছে"
`RSS`	resident set size (KiB), আসলে RAM-এ যতটুকু আছে
`TTY`	কোন টার্মিনাল। `?` মানে টার্মিনাল নেই (daemon)
`STAT`	state (উপরের টেবিল দেখো)
`START`	কখন শুরু হয়েছে
`TIME`	মোট কত CPU সময় খেয়েছে
`COMMAND`	কমান্ড। `[ ]` বন্ধনীতে থাকলে সেটা kernel thread

> 💡 **টিপস:** VSZ বড় দেখে ভয় পেও না। VSZ-এ শেয়ারড লাইব্রেরি আর এখনো ব্যবহার না করা মেমরিও গোনা হয়। আসল RAM ব্যবহারের ভালো ধারণা দেয় RSS।

`ps -ef` : সব process, UNIX স্টাইল

- `-e` = সব process
- `-f` = full format (PPID সহ)

```
$ ps -ef | head -4
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 09:01 ?        00:00:03 /sbin/init splash
root           2       0  0 09:01 ?        00:00:00 [kthreadd]
root           3       2  0 09:01 ?        00:00:00 [pool_workqueue_release]
```

কলাম	মানে
`UID`	user
`PPID`	parent PID
`C`	CPU ব্যবহারের হার (পূর্ণসংখ্যা)
`STIME`	শুরুর সময়

`ps -ef`-এ PPID থাকে, `ps aux`-এ থাকে %MEM আর RSS। তাই parent-child সম্পর্ক খুঁজতে `-ef`, মেমরি দেখতে `aux` সুবিধাজনক।

`-F` দিলে আরও বেশি তথ্য (RSS, কোন CPU-তে চলছে = `PSR`), আর `-l` দিলে long format (PRI, NI, WCHAN সহ)।

`--forest` : গাছ আকারে

```
$ ps -ef --forest | grep -A3 sshd
root         812       1  0 09:01 ?        00:00:00 sshd: /usr/sbin/sshd -D [listener]
0 of 10-100 startups
root        2101     812  0 10:00 ?        00:00:00  \_ sshd: nishat [priv]
nishat      2119    2101  0 10:00 ?        00:00:00      \_ sshd: nishat@pts/1
nishat      2120    2119  0 10:00 pts/1    00:00:00          \_ -bash
```

BSD স্টাইলে: `ps auxf` (শেষে `f` = forest)।

`-o` : নিজের পছন্দমতো কলাম

```
$ ps -eo pid,ppid,user,stat,ni,%cpu,%mem,rss,etime,cmd --sort=-%mem | head -5
    PID    PPID USER     STAT  NI %CPU %MEM   RSS     ELAPSED CMD
   7123    1720 nishat   Sl     0 12.1  9.8 1598320    02:13:44
/usr/lib/firefox/firefox
```

```
   2410    1720 nishat   Ssl    0  3.0  4.1 668012    03:01:10 /usr/bin/gnome-shell
   ...
```

কাজের কিছু কলাম নাম (`ps L` দিলে, ড্যাশ ছাড়া, সব কলাম নামের তালিকা পাবে; `-L` কিন্তু thread দেখানোর অপশন):

কলাম	মানে
`pid`, `ppid`, `pgid`, `sid`	process, parent, process group, session ID
`user`, `uid`, `group`	মালিক
`stat`, `state`	state (বিস্তারিত / শুধু এক অক্ষর)
`ni`, `pri`	nice মান, priority
`%cpu`, `%mem`, `rss`, `vsz`	রিসোর্স
`etime`, `etimes`	চালু হওয়ার পর কত সময় (ফরম্যাটেড / সেকেন্ডে)
`lstart`	শুরুর পুরো তারিখ-সময়
`time`	মোট CPU সময়
`tty`	টার্মিনাল
`comm`	শুধু program নাম
`args`, `cmd`	পুরো command line
`nlwp`	thread সংখ্যা
`wchan`	`S` / `D` state-এ থাকলে kernel-এর কোন ফাংশনে অপেক্ষা করছে
`psr`	এখন কোন CPU core-এ

হেডার বাদ দিতে কলাম নামের পরে `=` দাও। স্ক্রিপ্টে খুব কাজের:

```
$ ps -o pid=,etimes= -p 1
      1      12345
```

ফিল্টার করা

কমান্ড	কাজ
`ps -u nishat`	ওই user-এর process (effective user)
`ps -U root -u root`	real ও effective উভয় user root
`ps -p 1,2,3310`	নির্দিষ্ট PID গুলো
`ps -C nginx`	নির্দিষ্ট command নাম (exact)
`ps --ppid 812`	নির্দিষ্ট parent-এর child গুলো
`ps -t pts/0`	নির্দিষ্ট টার্মিনালের

কমান্ড	কাজ
`ps -eLf`	সব thread সহ

সাজানো (sort)

```
$ ps aux --sort=-%cpu | head -6     # সবচেয়ে বেশি CPU খাওয়া ৫টা
$ ps aux --sort=-rss | head -6      # সবচেয়ে বেশি RAM খাওয়া ৫টা
$ ps -eo pid,etime,cmd --sort=start_time | tail -5   # সবচেয়ে নতুন ৫টা
```

`-` মানে বড় থেকে ছোট, `+` বা কিছু না দিলে ছোট থেকে বড়।

`ps | grep`-এর পুরোনো ফাঁদ

```
$ ps aux | grep nginx
root     1010  0.0  0.1  55200  1500 ?   Ss  09:01  0:00 nginx: master process
/usr/sbin/nginx
www-data 1011  0.0  0.2  55900  5200 ?   S   09:01  0:00 nginx: worker process
nishat   3610  0.0  0.0   9200  2300 pts/0 S+ 10:30  0:00 grep --color=auto nginx
```

শেষ লাইনে `grep` নিজেই চলে এসেছে। সমাধান: `ps aux | grep '[n]ginx'` (regex `[n]ginx` "nginx"-এর সাথে মেলে, কিন্তু grep-এর নিজের command line-এ থাকা `[n]ginx` লেখার সাথে মেলে না)। আরও ভালো সমাধান হলো `pgrep` (নিচে দেখো)।

২৪.৮ top: লাইভ মনিটর

`ps` একটা ছবি, আর `top` হলো লাইভ ভিডিও। ডিফল্টে প্রতি ৩ সেকেন্ডে আপডেট হয়। এটা `procps` (Fedora/RHEL-এ `procps-ng`) প্যাকেজের অংশ, সব distro-তে থাকে।

```
$ top
top - 10:42:15 up  1:41,  2 users,  load average: 0.52, 0.61, 0.58
Tasks: 312 total,   1 running, 310 sleeping,   0 stopped,   1 zombie
%Cpu(s):  4.2 us,  1.3 sy,  0.0 ni, 93.9 id,  0.4 wa,  0.0 hi,  0.2 si,  0.0 st
MiB Mem :  15876.4 total,   6120.3 free,   5210.8 used,   4895.2 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.  10665.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   7123 nishat    20   0   11.2g   1.5g 210340 S  12.3   9.8  15:01.22 firefox
   2410 nishat    20   0 5512340 668012 120400 S   3.0   4.1   4:12.10 gnome-shell
      1 root      20   0  167780  13120   8400 S   0.0   0.1   0:03.11 systemd
```

হেডার অংশ লাইন ধরে ধরে

লাইন ১: বর্তমান সময়, কতক্ষণ চালু (`up 1:41`), কতজন user লগইন, আর **load average** (নিচে ২৪.১৪-এ বিস্তারিত)।

লাইন ২ (Tasks): মোট, running, sleeping, stopped, zombie সংখ্যা।

লাইন ৩ (%Cpu(s)): CPU সময় কোথায় যাচ্ছে:

ঘর	মানে
`us`	user space-এ সাধারণ প্রোগ্রাম
`sy`	kernel (system) কাজে
`ni`	nice বাড়ানো (কম priority) user প্রোগ্রামে
`id`	অলস (idle)
`wa`	I/O wait, CPU অলস কিন্তু ডিস্কের অপেক্ষায় আছে এমন কাজ আছে
`hi`	hardware interrupt
`si`	software interrupt
`st`	steal: VM হলে hypervisor অন্য VM-কে যে সময় দিয়েছে

> 💡 **টিপস:** `wa` বেশি হলে সমস্যা ডিস্কে, CPU-তে না। `st` বেশি হলে তোমার cloud VM-এর host overloaded।

লাইন ৪–৫ (Mem/Swap): লিনাক্স ফাঁকা RAM নষ্ট না করে ফাইল cache হিসেবে ব্যবহার করে (`buff/cache`)। তাই `free` কম দেখলে ঘাবড়াবে না। আসলে কত মেমরি নতুন প্রোগ্রামের জন্য পাওয়া যাবে, সেটা বলে **`avail Mem`**।

process টেবিলের কলাম

কলাম	মানে
`PR`	kernel-এর চোখে priority। সাধারণ process-এ `20 + NI`, real-time হলে `rt` বা ঋণাত্মক সংখ্যা
`NI`	nice মান (-20 থেকে 19)
`VIRT`	virtual memory (ps-এর VSZ)
`RES`	RAM-এ বাস্তবে আছে (ps-এর RSS)
`SHR`	RES-এর মধ্যে শেয়ারড অংশ
`S`	state
`%CPU`	**শেষ রিফ্রেশের পর** CPU ব্যবহার। মাল্টি-কোরে ১০০%-এর বেশি হতে পারে (৪ core পুরো খেলে ৪০০%)
`TIME+`	মোট CPU সময়, শতাংশ-সেকেন্ড পর্যন্ত

> 📝 **মনে রাখো:** `ps` -এর `%CPU` হলো জীবনভর গড়, `top` -এর `%CPU` হলো এই মুহূর্তের। তাই এখন কে CPU খাচ্ছে জানতে `top` বা `htop` দেখো।

top-এর interactive শর্টকাট (চলমান অবস্থায়)

top চলার সময় এই কী-গুলো চাপো (case-sensitive: `P` আর `p` আলাদা):

কী	কাজ
`h` বা `?`	help স্ক্রিন
`q`	বের হও
`Space` বা `Enter`	এখনই রিফ্রেশ
`d` বা `s`	রিফ্রেশ বিরতি বদলাও (যেমন `1` সেকেন্ড)
`P`	%CPU অনুযায়ী সাজাও (ডিফল্ট)
`M`	%MEM অনুযায়ী সাজাও
`T`	TIME+ অনুযায়ী সাজাও
`N`	PID অনুযায়ী সাজাও
`R`	সাজানোর ক্রম উল্টে দাও
`<` / `>`	sort কলাম বাঁয়ে / ডানে সরাও
`x`	sort কলাম highlight
`y`	running task highlight
`b`	bold/reverse highlight টগল (x/y-এর সাথে)
`z`	রঙিন মোড টগল
`Z`	রঙ কাস্টমাইজ
`1`	প্রতিটা CPU core আলাদা লাইনে দেখাও
`t`	CPU সারাংশের ধরন বদলাও (bar/লেখা/লুকানো)
`m`	মেমরি সারাংশের ধরন বদলাও
`l`	load average লাইন টগল
`E`	হেডারের মেমরি একক বদলাও (KiB → MiB → GiB…)
`e`	টেবিলের মেমরি একক বদলাও
`c`	শুধু নাম ↔ পুরো command line টগল
`V`	forest (গাছ) ভিউ
`H`	process ↔ thread ভিউ
`i`	idle (ঘুমন্ত) process লুকাও/দেখাও
`u` বা `U`	নির্দিষ্ট user-এর process দেখাও
`o` / `O`	কলামে ফিল্টার, যেমন `COMMAND=nginx` বা `%CPU>5.0`
`Ctrl+O`	বর্তমান ফিল্টার দেখাও

কী	কাজ
`=`	ফিল্টার মুছে দাও
`L`	লেখা খুঁজে বের করো (locate)
`&`	পরের মিল খোঁজো
`n` বা `#`	কয়টা process দেখাবে তা সীমিত করো
`f` বা `F`	fields manager: কলাম যোগ/বাদ/সাজানো
`k`	process-কে signal পাঠাও (PID চাইবে, তারপর signal, ডিফল্ট 15)
`r`	renice (PID, তারপর নতুন nice মান)
`W`	বর্তমান সেটিং `~/.config/procps/toprc`-এ সেভ করো
`A`	alternate display (৪টা উইন্ডো)

top-এর কমান্ড-লাইন অপশন

অপশন	কাজ
`-d 1`	১ সেকেন্ড পর পর রিফ্রেশ
`-n 5`	৫ বার রিফ্রেশ করে বেরিয়ে যাও
`-b`	batch mode: স্ক্রিনে না সাজিয়ে লেখা হিসেবে আউটপুট, ফাইলে সেভ বা স্ক্রিপ্টের জন্য
`-p PID[,PID]`	শুধু নির্দিষ্ট process
`-u user`	শুধু নির্দিষ্ট user
`-o FIELD`	ওই কলাম দিয়ে সাজাও, যেমন `-o %MEM`
`-H`	thread মোডে শুরু
`-c`	পুরো command line দেখিয়ে শুরু

```
$ top -b -n 1 -o %MEM | head -15 > snapshot.txt    # মেমরির একটা স্ন্যাপশট ফাইলে
$ top -p $(pgrep -d, nginx)                       # শুধু nginx-এর process গুলো
```

২৪.৯ htop: রঙিন, সহজ top

`htop` হলো top-এর আধুনিক, বন্ধুসুলভ সংস্করণ: রঙিন CPU/মেমরি বার, মাউস চলে, স্ক্রল করা যায়, গাছ দেখায়। ডিফল্টে সবসময় ইনস্টল থাকে না:

```
$ sudo apt install htop      # Debian/Ubuntu
$ sudo dnf install htop      # Fedora; RHEL 9-এ EPEL রিপো লাগে
```

```
  0[||||||               12.4%]   Tasks: 148, 612 thr, 210 kthr; 1 running
  1[|||||                 9.1%]   Load average: 0.52 0.61 0.58
```

```
  2[|||||||             18.0%]   Uptime: 01:41:22
  3[||                   4.0%]
  Mem[|||||||||||||||5.09G/15.5G]
  Swp[                    0K/4.00G]

    PID USER       PRI  NI  VIRT   RES   SHR S CPU% MEM%   TIME+  Command
   7123 nishat      20   0 11.2G 1.52G  205M S 12.3  9.8 15:01.22
/usr/lib/firefox/firefox
 F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice- F8Nice+ F9Kill F10Quit
```

বারের রঙের মানে (ডিফল্ট থিমে): CPU বারে সবুজ = সাধারণ user process, লাল = kernel, নীল = কম priority (nice)। মেমরি বারে সবুজ = ব্যবহৃত, নীল = buffer, হলুদ/কমলা = cache। সঠিক রঙ-তালিকা `F1` চাপলে দেখা যায়।

htop শর্টকাট

কী	কাজ
`F1` বা `h` বা `?`	help
`F2` বা `S`	setup: meter, কলাম, রঙ বদলাও
`F3` বা `/`	নাম খোঁজো (হাইলাইট করে)
`F4` বা `\`	ফিল্টার: শুধু মিলে যাওয়া process দেখাও
`F5` বা `t`	tree view টগল
`F6` বা `<` `>`	কোন কলাম দিয়ে সাজাবে বাছাই
`F7` বা `]`	nice কমাও = priority বাড়াও (root লাগে)
`F8` বা `[`	nice বাড়াও = priority কমাও
`F9` বা `k`	নির্বাচিত process-কে signal পাঠাও (তালিকা থেকে বাছো)
`F10` বা `q`	বের হও
`↑` `↓` `PgUp` `PgDn` `Home` `End`	তালিকায় চলাফেরা
`Space`	process ট্যাগ করো (একসাথে অনেকগুলোকে kill করতে)
`c`	process আর তার সব child ট্যাগ করো
`U`	সব ট্যাগ মুছে দাও
`u`	নির্দিষ্ট user-এর process দেখাও
`P`	CPU% অনুযায়ী সাজাও
`M`	MEM% অনুযায়ী সাজাও
`T`	TIME অনুযায়ী সাজাও
`I`	সাজানোর ক্রম উল্টাও
`+` / `-` (বা `*`)	tree view-তে শাখা খোলো/বন্ধ করো
`H`	user thread দেখাও/লুকাও

কী	কাজ
`K`	kernel thread দেখাও/লুকাও
`p`	program-এর পুরো path দেখাও/লুকাও
`F`	নির্বাচিত process-কে "follow" করো (সাজানো বদলালেও cursor তার সাথে থাকে)
`Z` (Shift+z)	আপডেট pause/resume
`Ctrl+L`	স্ক্রিন রিফ্রেশ
`e`	নির্বাচিত process-এর environment variable দেখাও
`l`	`lsof` দিয়ে খোলা ফাইল দেখাও (lsof ইনস্টল থাকতে হবে)
`s`	`strace` দিয়ে system call দেখাও (strace লাগবে)
`a`	CPU affinity সেট করো (কোন core-এ চলবে)
`w`	পুরো command line একাধিক লাইনে দেখাও
`Tab`	(htop 3.x) স্ক্রিন ট্যাব বদলাও, যেমন I/O ট্যাব

কমান্ড-লাইন অপশন: `htop -u nishat`, `htop -p 1234,5678`, `htop -t` (tree দিয়ে শুরু), `htop -d 10` (রিফ্রেশ বিরতি, **এক-দশমাংশ সেকেন্ড** এককে, তাই `-d 10` = ১ সেকেন্ড), `htop -s PERCENT_MEM` (ওই কলামে সাজিয়ে শুরু)।

> 💡 **টিপস:** আরও আধুনিক বিকল্প: `btop` (খুব সুন্দর গ্রাফ), `glances` (CPU, ডিস্ক, নেটওয়ার্ক সব এক স্ক্রিনে), `atop` (ইতিহাস রেকর্ড করে)। পারফরম্যান্স বিশ্লেষণ বিস্তারিত অধ্যায় ৪৬-এ।

২৪.১০ pgrep, pkill, pidof: নাম দিয়ে process খোঁজা

`ps | grep | awk` দিয়ে PID বের করার বদলে এই তিনটা কমান্ড সরাসরি কাজ করে। `pgrep` / `pkill` আছে `procps` প্যাকেজে, `pidof` আছে Debian 12-এ `sysvinit-utils`-এ, আর Fedora/RHEL ও নতুন Debian/Ubuntu-তে `procps` / `procps-ng`-এ। সব distro-তে ডিফল্টে থাকে।

pgrep

```
$ pgrep sshd
812
2101
2119

$ pgrep -l sshd          # PID + নাম
812 sshd
2101 sshd
2119 sshd
```

```
$ pgrep -a nginx         # PID + পুরো command line
1010 nginx: master process /usr/sbin/nginx
1011 nginx: worker process
```

অপশন	কাজ
`-l`	নামসহ দেখাও
`-a`	পুরো command line দেখাও
`-f`	শুধু নাম না, পুরো command line-এ খোঁজো (যেমন `pgrep -f "python3 app.py"`)
`-x`	নাম হুবহু মিলতে হবে (exact)
`-u user`	নির্দিষ্ট (effective) user-এর
`-U user`	নির্দিষ্ট real user-এর
`-P PPID`	নির্দিষ্ট parent-এর child
`-t pts/0`	নির্দিষ্ট টার্মিনালের
`-n`	সবচেয়ে নতুনটা
`-o`	সবচেয়ে পুরোনোটা
`-c`	শুধু সংখ্যা গোনো
`-d ,`	আউটপুটের মাঝে delimiter (ডিফল্ট newline)
`-v`	উল্টো, যারা মেলে না
`-i`	case-insensitive

> 📝 **মনে রাখো:** pattern একটা regex, আর ডিফল্টে শুধু process-এর নাম (`comm`, সর্বোচ্চ ১৫ অক্ষর) মেলায়। নাম ১৫ অক্ষরের বেশি হলে কেটে যায়, তখন `-f` লাগে। আবার `pgrep ssh` লিখলে `sshd`, `ssh-agent` সবই মিলবে। নির্দিষ্টটা চাইলে `-x` দাও।

pkill

pgrep-এর মতো খোঁজে, কিন্তু পেলে signal পাঠায় (ডিফল্ট SIGTERM)। অপশন প্রায় একই।

```
$ pkill -x firefox                # firefox-কে SIGTERM
$ pkill -HUP -x nginx             # nginx-কে SIGHUP (config reload)
$ pkill -u guest                  # guest user-এর সব process
$ pkill -9 -f "python3 worker.py"  # command line মিলিয়ে জোর করে মারো
$ pkill -e sleep                  # -e: কাকে মারল তা প্রিন্ট করো
sleep killed (pid 6001)
```

> ⚠️ **সতর্কতা:** `pkill` আগে `pgrep` দিয়ে পরীক্ষা করে নাও। `pkill -f python` দিলে সিস্টেমের সব python process মরবে, এমনকি দরকারি সার্ভিসও। প্রথমে `pgrep -af python` চালিয়ে দেখো কারা

> মিলছে।

pidof

শুধু program-এর হুবহু নাম দিয়ে PID দেয়। স্ক্রিপ্টে খুব সহজ:

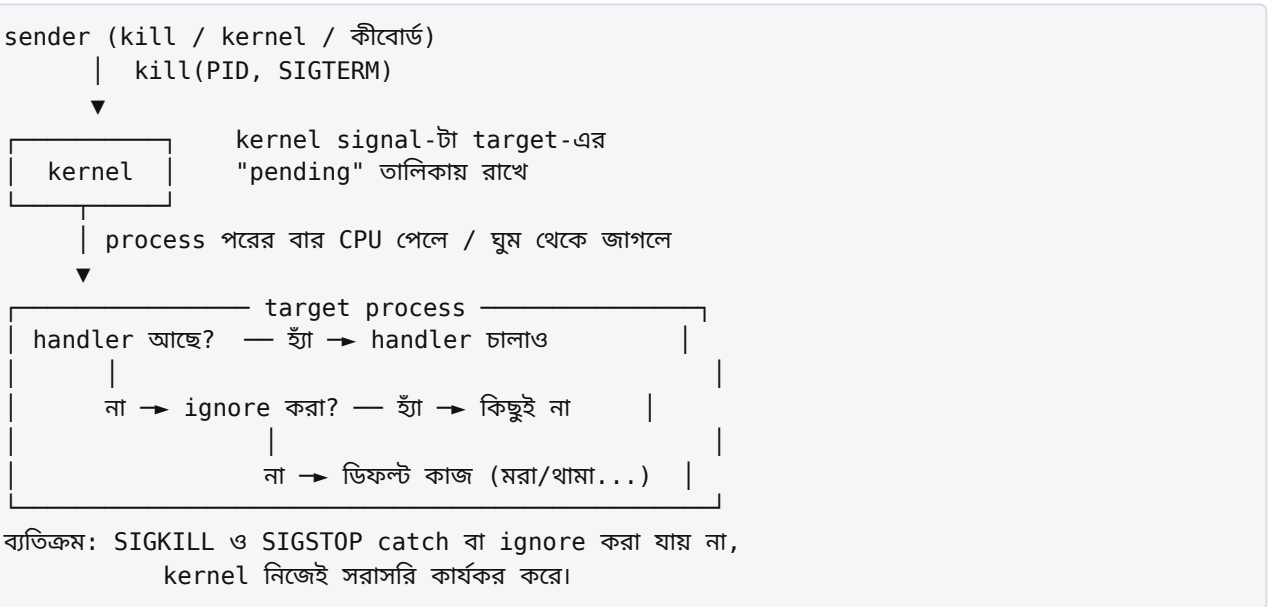
```
$ pidof systemd
1720 1
$ pidof -s nginx       # -s: শুধু একটা PID
1011
$ pidof -x backup.sh   # -x: শেল স্ক্রিপ্টও খোঁজো
4410
```

২৪.১১ signals: process-কে বার্তা পাঠানো

signal হলো kernel বা অন্য process থেকে একটা process-কে পাঠানো ছোট্ট সংকেত, একটা নম্বর মাত্র। process তিনভাবে সাড়া দিতে পারে:

1. **ডিফল্ট কাজ** (বেশিরভাগ ক্ষেত্রে মরে যাওয়া, কোনোটায় core dump, কোনোটায় থেমে যাওয়া)
2. **ignore** (উপেক্ষা)
3. **catch**: নিজের handler ফাংশন চালানো (যেমন ফাইল সেভ করে পরিষ্কারভাবে বের হওয়া)

উপমা: signal হলো দরজায় টোকা। SIGTERM মানে ভদ্রভাবে বলা "দয়া করে বেরিয়ে যান", মানুষটা জিনিসপত্র গুছিয়ে বের হয়। SIGKILL মানে পুলিশ এসে সরাসরি তুলে নিয়ে গেল, গোছানোর সুযোগ নেই।



গুরুত্বপূর্ণ signal টেবিল

(নম্বরগুলো x86 ও ARM লিনাক্সের। অন্য architecture-এ কিছু নম্বর আলাদা হতে পারে, তাই স্ক্রিপ্টে নম্বরের বদলে নাম লেখা নিরাপদ।)

নম্বর	নাম	ডিফল্ট কাজ	কখন/কেন
1	`SIGHUP`	মরে যাওয়া	"hang up": টার্মিনাল বন্ধ হলে। daemon-রা প্রথা অনুযায়ী এটা পেলে config reload করে (nginx, sshd)
2	`SIGINT`	মরে যাওয়া	কীবোর্ডে `Ctrl+C`
3	`SIGQUIT`	মরে যাওয়া + core dump	কীবোর্ডে `Ctrl+\`
6	`SIGABRT`	মরে যাওয়া + core dump	প্রোগ্রাম নিজে `abort()` করলে
9	`SIGKILL`	মরে যাওয়া	জোর করে হত্যা। **catch/ignore করা যায় না**
10	`SIGUSR1`	মরে যাওয়া	প্রোগ্রাম নিজের মতো অর্থ দেয় (যেমন `dd` এটা পেলে progress দেখায়)
11	`SIGSEGV`	মরে যাওয়া + core dump	ভুল মেমরি অ্যাক্সেস ("Segmentation fault")
12	`SIGUSR2`	মরে যাওয়া	প্রোগ্রাম-নির্ধারিত
13	`SIGPIPE`	মরে যাওয়া	পাইপের অপর প্রান্ত বন্ধ, তবু লেখার চেষ্টা
14	`SIGALRM`	মরে যাওয়া	টাইমার শেষ
15	`SIGTERM`	মরে যাওয়া	**ভদ্র অনুরোধ**: `kill`-এর ডিফল্ট। process গুছিয়ে বের হতে পারে
17	`SIGCHLD`	ignore	child মারা গেলে বা থামলে parent পায়
18	`SIGCONT`	থেমে থাকলে আবার চালু	`fg` / `bg` এটাই পাঠায়
19	`SIGSTOP`	থেমে যাওয়া	জোর করে pause। **catch/ignore করা যায় না**
20	`SIGTSTP`	থেমে যাওয়া	কীবোর্ডে `Ctrl+Z` (ভদ্র stop, catch করা যায়)
28	`SIGWINCH`	ignore	টার্মিনাল উইন্ডোর মাপ বদলেছে

সব signal দেখতে:

```
$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
...
$ kill -l 9
KILL
$ kill -l TERM
15
```

signal-এর সাথে জড়িত কীবোর্ড শর্টকাট

শর্টকাট	কী পাঠায়	ফল
`Ctrl+C`	SIGINT	foreground process বন্ধ
`Ctrl+Z`	SIGTSTP	foreground process থামিয়ে ব্যাকগ্রাউন্ডে রাখে (`fg` দিয়ে ফেরাও)
`Ctrl+\`	SIGQUIT	বন্ধ + core dump (Ctrl+C কাজ না করলে চেষ্টা করো)
`Ctrl+D`	কোনো signal নয়	ইনপুট শেষ (EOF) জানায়; শেলে দিলে লগআউট
`Ctrl+S` / `Ctrl+Q`	কোনো signal নয়	টার্মিনাল আউটপুট থামাও / চালু করো (flow control)

এই শর্টকাট গুলো আসলে টার্মিনাল ড্রাইভার (tty) signal-এ রূপান্তর করে। `stty -a` দিলে দেখা যায় কোন কী কোন কাজে বাঁধা (`intr = ^C; quit = ^\; susp = ^Z` ইত্যাদি)। job control (`fg` , `bg` , `jobs`) বিস্তারিত অধ্যায় ২৫-এ।

> 📝 **মনে রাখো:** একটা process signal-এ মারা গেলে শেলে তার exit code হয় **128 + signal নম্বর**। Ctrl+C-তে মরলে `$?` = 130, `kill -9` -এ মরলে 137। ডকারে "exit code 137" দেখলে বুঝবে SIGKILL (প্রায়ই OOM killer)।

২৪.১২ kill ও killall

kill: PID ধরে signal পাঠানো

নাম "kill" হলেও এর আসল কাজ **signal পাঠানো**। ডিফল্ট signal SIGTERM (15)।

```
kill [-signal] PID...
```

signal লেখার সব সমতুল্য রূপ:

```
$ kill 4521              # SIGTERM
$ kill -15 4521          # একই
$ kill -TERM 4521        # একই
$ kill -SIGTERM 4521     # একই
$ kill -s TERM 4521      # একই
```

ব্যবহার	কাজ
`kill PID`	ভদ্রভাবে বন্ধ হতে বলো
`kill -9 PID`	জোর করে মারো (শেষ উপায়)
`kill -HUP PID`	reload (daemon হলে)
`kill -STOP PID`	pause করো
`kill -CONT PID`	আবার চালু করো
`kill -0 PID`	কোনো signal যায় না, শুধু যাচাই: process আছে কি না এবং signal পাঠানোর অনুমতি আছে কি না (exit code 0 = হ্যাঁ)
`kill -l`	signal তালিকা
`kill -- -PGID`	পুরো process group-কে (ঋণাত্মক সংখ্যা = group)

```
$ sleep 500 &
[1] 7001
$ kill -0 7001 && echo "চলছে"
চলছে
$ kill 7001
[1]+  Terminated              sleep 500
$ kill -0 7001
bash: kill: (7001) - No such process
```

সঠিক পদ্ধতি: আগে TERM, পরে KILL

```
1. kill PID            → process কে গুছিয়ে বের হওয়ার সুযোগ দাও
2. কয়েক সেকেন্ড অপেক্ষা (sleep 5)
3. kill -0 PID          → এখনো আছে?
4. থাকলে: kill -9 PID  → শেষ অস্ত্র
```

systemd নিজেও সার্ভিস বন্ধ করার সময় ঠিক এটাই করে: আগে SIGTERM, তারপর ডিফল্টে ৯০ সেকেন্ড অপেক্ষা করে SIGKILL।

> ⚠️ **সতর্কতা:** অভ্যাসবশত `kill -9` দিও না। SIGKILL পেলে process temp ফাইল মুছতে পারে না, ডাটাবেস লেনদেন শেষ করতে পারে না, lock ফাইল রেখে যায়। ফলে ডেটা নষ্ট হতে পারে বা পরের বার প্রোগ্রাম

চালু হবে না।

> ⚠️ **সতর্কতা:** `kill -9 -1` মানে "আমার অনুমতি আছে এমন সব process মারো"। সাধারণ user দিলে তার নিজের সব process (ডেস্কটপ সেশন সহ) মরে যাবে, root দিলে প্রায় পুরো সিস্টেম। ভুলেও চালাবে না।

অনুমতি: সাধারণ user শুধু নিজের process-কে signal পাঠাতে পারে। অন্যের বা root-এর process-এ `Operation not permitted` আসবে, তখন `sudo kill ...` লাগে (অধ্যায় ১৮)।

> 📝 **মনে রাখো:** `kill` দুটো আছে: bash-এর builtin `kill` আর `/usr/bin/kill` (procps-ng/util-linux)। সাধারণত builtin চলে, যেটা `%1` এর মতো job নম্বরও বোঝে (`kill %1`)।

killall: নাম ধরে মারা

`killall` (`psmisc` প্যাকেজ) process-এর হুবহু নাম দিয়ে signal পাঠায়।

```
$ killall firefox
$ killall -HUP nginx
$ killall -u guest bash          # guest-এর সব bash
$ killall -i sleep               # প্রতিটার আগে জিজ্ঞেস করবে
Kill sleep(7101) ? (y/N) y
$ killall -w myapp               # সব মরা পর্যন্ত অপেক্ষা করো
$ killall -o 2h backup           # ২ ঘণ্টার বেশি পুরোনো backup process
```

অপশন	কাজ
`-s SIG` বা `-SIG`	কোন signal
`-i`	interactive, প্রতিবার নিশ্চিত হও
`-u user`	নির্দিষ্ট user-এর
`-w`	মরা পর্যন্ত অপেক্ষা
`-e`	exact: ১৫ অক্ষরের বেশি লম্বা নাম পুরো মিলাও
`-I`	case-insensitive
`-r`	নামকে regex হিসেবে ধরো
`-o TIME` / `-y TIME`	এর চেয়ে পুরোনো / নতুন (যেমন `30m`, `2h`, `1d`)
`-v`	কী হলো তা জানাও
`-l`	signal তালিকা

> ⚠️ **সতর্কতা:** Solaris-এর মতো কিছু পুরোনো Unix-এ `killall` মানে "সব process মারো"! লিনাক্সে

> নিরাপদ, কিন্তু অন্য Unix সার্ভারে অভ্যাসবশত চালিও না।

কোনটা কখন

প্রয়োজন	কমান্ড
নির্দিষ্ট একটা PID	`kill PID`
নামের সব instance	`killall name` বা `pkill -x name`
command line-এর অংশ মিলিয়ে	`pkill -f "pattern"`
সিস্টেম সার্ভিস	`sudo systemctl stop/restart/reload name` (অধ্যায় ২৬)
GUI-তে জমে যাওয়া উইন্ডো	`xkill` (X11-এ), বা System Monitor

২৪.১৩ nice, renice ও priority

CPU core সীমিত, process অনেক। কাকে কতটা সময় দেওয়া হবে, তা ঠিক করে kernel-এর **scheduler**। সাধারণ process-এর জন্য লিনাক্স ব্যবহার করে CFS (Completely Fair Scheduler), আর kernel 6.6 থেকে এর উত্তরসূরি **EEVDF** scheduler (Ubuntu 24.04-এর kernel 6.8, তাই সেখানে EEVDF)। দুটোতেই "nice" মান একই ভাবে কাজ করে।

nice মান কী

- পরিসর: **-20** (সবচেয়ে বেশি priority) থেকে **19** (সবচেয়ে কম)। ডিফল্ট **0**।
- নাম "nice" কারণ মান যত বেশি, process অন্যদের প্রতি তত "ভদ্র", তত বেশি জায়গা ছেড়ে দেয়।
- এটা কঠোর ক্রম না, ওজন (weight)। nice মান ১ বাড়লে মোটামুটি ~১০% কম CPU ভাগ পায়। ফাঁকা সিস্টেমে nice 19 process-ও পুরো CPU পাবে; প্রতিযোগিতা থাকলেই পার্থক্য বোঝা যায়।

উপমা: ব্যাংকের লাইন না, বরং একটা পিৎজা ভাগ করা। যার ওজন বেশি, সে বড় টুকরো পায়, কিন্তু কেউ একেবারে না খেয়ে থাকে না।

```
nice:   -20 ◄──────────────── 0 ────────────────► 19
        বেশি CPU ভাগ          ডিফল্ট           কম CPU ভাগ
        (শুধু root)                               (যে কেউ দিতে পারে)

top-এর PR:  0  ..........  20  ..........  39     (PR = 20 + NI)
ps -l এর PRI: 60 ........  80  ..........  99     (ps-এর হিসাব ভিন্ন স্কেলে)
```

কে কী পারে	
সাধারণ user	নিজের process-এর nice **বাড়াতে** পারে (priority কমাতে)

কে কী পারে	
সাধারণ user	nice **কমাতে** (priority বাড়াতে) পারে না, একবার বাড়ালে ফেরাতেও না (যদি না `/etc/security/limits.conf`-এ `nice` limit দেওয়া থাকে)
root	যেকোনো দিকে, -20 পর্যন্ত

nice: নতুন process নির্দিষ্ট priority-তে চালানো

```
$ nice                              # বর্তমান nice মান
0
$ nice tar czf backup.tar.gz /home/nishat/docs    # ডিফল্ট +10 বাড়িয়ে চালাবে
$ nice -n 15 ./heavy-report.sh                    # nice 15-এ
$ sudo nice -n -5 ./important-job                 # ঋণাত্মক: root লাগে
$ nice -n -5 ./job
nice: cannot set niceness: Permission denied
```

> 💡 **টিপস:** পুরোনো লেখায় `nice -10 cmd` দেখবে, যার মানে +10 (ড্যাশটা অপশন চিহ্ন, ঋণাত্মক নয়)! আর `nice --10` মানে -10। এই বিভ্রান্তি এড়াতে সবসময় `-n` লেখো।

renice: চলমান process-এর priority বদলানো

```
$ renice -n 10 -p 7123
7123 (process ID) old priority 0, new priority 10

$ sudo renice -n -5 -p 1010
1010 (process ID) old priority 0, new priority -5

$ sudo renice -n 5 -u guest          # guest-এর সব process
$ renice -n 19 -g 4500               # পুরো process group
```

অপশন	কাজ
`-n N`	নতুন nice মান (নতুন util-linux ভার্সনে `--relative` যোগ করলে মানটা বর্তমান nice-এর সাথে যোগ হয়; নিশ্চিত হতে `man renice` দেখো)
`-p PID`	process ধরে (ডিফল্ট)
`-u user`	user-এর সব process
`-g PGID`	process group ধরে

`top`-এ `r` আর `htop`-এ `F7` / `F8` দিয়েও একই কাজ করা যায়।

nice-এর বাইরে: real-time ও I/O priority

- **`chrt`** : real-time scheduling policy (SCHED_FIFO, SCHED_RR) দেয়। এরা সব সাধারণ (nice) process-এর আগে চলে। `chrt -p PID` দিয়ে দেখো, `sudo chrt -f 50 cmd` দিয়ে চালাও। ভুল ব্যবহারে সিস্টেম জমে যেতে পারে, সাবধান।
- **`ionice`** : ডিস্ক I/O-এর priority। `ionice -c 3 cmd` (idle class: ডিস্ক ফাঁকা থাকলেই শুধু I/O করবে)। ব্যাকআপের জন্য চমৎকার: `nice -n 19 ionice -c 3 rsync ...`।
- systemd সার্ভিসে `Nice=`, `CPUWeight=`, `IOWeight=` সেটিং দিয়ে আরও আধুনিক উপায়ে (cgroups) নিয়ন্ত্রণ করা যায় (অধ্যায় ২৬, অধ্যায় ৪৮)।

২৪.১৪ /proc/PID: process-এর ভেতরে উঁকি

`/proc` হলো একটা **virtual filesystem**: ডিস্কে নেই, kernel মেমরি থেকে তাৎক্ষণিক বানিয়ে দেয়। প্রতিটা process-এর জন্য একটা ফোল্ডার `/proc/<PID>`। `ps`, `top`, `htop` সবাই আসলে এখান থেকেই পড়ে। (`/proc` আর `/sys` পুরো বিস্তারিত অধ্যায় ৪৭-এ।)

```
$ ls /proc/$$
attr       cmdline  environ  limits     mounts      oom_score_adj  stat     task
cgroup     comm     exe      loginuid   ns          root           statm    wchan
...        cwd      fd       maps       oom_score   sched          status   ...
```

`/proc/self` সবসময় যে process পড়ছে, তার নিজের ফোল্ডারে নির্দেশ করে।

ফাইল/ফোল্ডার	কী আছে
`cmdline`	পুরো command line, arguments গুলো NUL (`\0`) দিয়ে আলাদা
`comm`	program-এর ছোট নাম (১৫ অক্ষর পর্যন্ত)
`status`	মানুষের পড়ার মতো সারাংশ: নাম, State, PPid, Uid, Threads, VmRSS...
`stat`	একই তথ্য এক লাইনে, স্ক্রিপ্ট/ps-এর জন্য
`statm`	মেমরির সংক্ষিপ্ত হিসাব (page এককে)
`exe`	আসল executable-এর symlink
`cwd`	বর্তমান ডিরেক্টরির symlink
`root`	process-এর root ডিরেক্টরি (chroot/কন্টেইনারে আলাদা)
`environ`	environment variable (NUL দিয়ে আলাদা)
`fd/`	খোলা প্রতিটা file descriptor-এর symlink (0=stdin, 1=stdout, 2=stderr)
`maps`	মেমরি ম্যাপ: কোন লাইব্রেরি কোথায় লোড
`smaps`, `smaps_rollup`	বিস্তারিত মেমরি (PSS, swap ইত্যাদি)
`limits`	resource limit (`ulimit` এর মান)

ফাইল/ফোল্ডার	কী আছে
`io`	কত বাইট পড়েছে/লিখেছে
`task/`	প্রতিটা thread-এর জন্য একটা উপ-ফোল্ডার
`cgroup`	কোন cgroup-এ (systemd সার্ভিস/কন্টেইনার) আছে
`ns/`	namespace গুলো (কন্টেইনারের ভিত্তি)
`oom_score`	মেমরি ফুরালে OOM killer একে মারার সম্ভাবনা (বেশি = আগে মরবে)
`oom_score_adj`	সেটা সমন্বয় করা যায় (-1000 থেকে 1000)
`wchan`	ঘুমিয়ে থাকলে kernel-এর কোন ফাংশনে অপেক্ষা করছে

কিছু কাজের উদাহরণ

```
$ grep -E '^(Name|State|PPid|Uid|Threads|VmRSS)' /proc/$$/status
Name:   bash
State:  S (sleeping)
PPid:   3295
Uid:    1000    1000    1000    1000
VmRSS:      5300 kB
Threads:        1

$ tr '\0' ' ' < /proc/1010/cmdline; echo
nginx: master process /usr/sbin/nginx

$ ls -l /proc/$$/cwd /proc/$$/exe
lrwxrwxrwx 1 nishat nishat 0 ... /proc/3310/cwd -> /home/nishat
lrwxrwxrwx 1 nishat nishat 0 ... /proc/3310/exe -> /usr/bin/bash

$ ls -l /proc/$$/fd
lrwx------ 1 nishat nishat 64 ... 0 -> /dev/pts/0
lrwx------ 1 nishat nishat 64 ... 1 -> /dev/pts/0
lrwx------ 1 nishat nishat 64 ... 2 -> /dev/pts/0
lrwx------ 1 nishat nishat 64 ... 255 -> /dev/pts/0

$ ls /proc/$(pidof -s firefox)/task | wc -l     # firefox-এর thread সংখ্যা
87
```

> 💡 **টিপস (জীবন বাঁচানো কৌশল):** ভুল করে একটা ফাইল মুছে ফেলেছ, কিন্তু কোনো process সেটা এখনো খুলে রেখেছে? ফাইলটা আসলে এখনো ডিস্কে আছে! `ls -l /proc/PID/fd` দিয়ে `(deleted)` লেখা লাইনটা খোঁজো, তারপর `cp /proc/PID/fd/4 ~/recovered.log` দিয়ে ফেরত আনো। (কারণ inode-এর link count শূন্য হলেও খোলা reference থাকলে ডেটা মোছা হয় না, দেখো অধ্যায় ১১)

> ⚠️ **সতর্কতা:** অন্য user-এর process-এর `environ`, `fd`, `maps` ইত্যাদি সাধারণ user পড়তে পারে না, এগুলোতে পাসওয়ার্ড বা টোকেন থাকতে পারে। root লাগে।

২৪.১৫ load average: সিস্টেম কতটা ব্যস্ত

`uptime`, `top`, `w` সবখানে তিনটা সংখ্যা দেখায়:

```
$ uptime
 10:42:15 up  1:41,  2 users,  load average: 0.52, 0.61, 0.58

$ cat /proc/loadavg
0.52 0.61 0.58 2/1310 7342
```

সংখ্যাগুলো কী

তিনটা সংখ্যা হলো গত **১ মিনিট, ৫ মিনিট, ১৫ মিনিটের** গড়। কিসের গড়? এমন task-এর সংখ্যার যারা:

- **R** state-এ (চলছে বা CPU-র লাইনে দাঁড়িয়ে), **অথবা**
- **D** state-এ (uninterruptible, সাধারণত disk I/O-র অপেক্ষায়)

লিনাক্সে D state-ও গোনা হয়, তাই load average শুধু CPU চাপ না, I/O চাপও দেখায়। এটা লিনাক্সের নিজস্ব বৈশিষ্ট্য (অন্য অনেক Unix শুধু CPU গোনে)। গড়গুলো সাধারণ গড় না, exponentially damped (সাম্প্রতিক মানের ওজন বেশি)।

`/proc/loadavg`-এর চতুর্থ ঘর `2/1310` = এই মুহূর্তে runnable task / মোট task (thread সহ), আর পঞ্চম ঘর `7342` = সবশেষে তৈরি হওয়া PID।

কীভাবে পড়বে: CPU সংখ্যার সাথে তুলনা

উপমা: একটা ব্রিজে লেন আছে (CPU core)। load হলো ব্রিজে বা ব্রিজের অপেক্ষায় থাকা গাড়ির সংখ্যা।

```
৪ core সিস্টেম (৪ লেনের ব্রিজ):

load 2.0  ‖🚗 🚗 ·  · ‖   ৫০% ব্যস্ত, আরামদায়ক
load 4.0  ‖🚗 🚗 🚗 🚗‖   পুরো ব্যবহার, কেউ অপেক্ষায় নেই
load 8.0  ‖🚗 🚗 🚗 🚗‖ 🚗🚗🚗🚗 ← ৪টা লাইনে দাঁড়িয়ে, দ্বিগুণ চাপ
```

```
$ nproc
4
```

load / core সংখ্যা	অর্থ
< 0.7	স্বস্তিতে আছে
প্রায় 1.0	পুরো ব্যবহার হচ্ছে, নজর রাখো
> 1.0 নিয়মিত	task অপেক্ষা করছে, সিস্টেম ধীর লাগবে
অনেক বেশি কিন্তু CPU `id` বেশি	D state-এর জন্য: ডিস্ক/NFS সমস্যা খোঁজো (`top`-এ `wa`, `ps`-এ `D`)

তিনটা সংখ্যা থেকে প্রবণতা (trend)

উদাহরণ	ব্যাখ্যা
`8.10, 3.20, 1.05`	চাপ সবে বাড়ছে (১ মিনিট > ১৫ মিনিট)
`0.40, 2.50, 6.80`	চাপ কমে আসছে
`4.00, 4.02, 3.98`	স্থির চাপ

উচ্চ load পেলে D state-এর process খুঁজতে:

```
$ ps -eo pid,stat,wchan:30,cmd | awk '$2 ~ /^D/'
```

> 📝 **মনে রাখো:** load average একটা "সংখ্যা", শতাংশ না। ১৬ core সার্ভারে load 10 মোটেও সমস্যা না, কিন্তু ১ core VM-এ load 10 মানে ভয়াবহ অবস্থা। সবসময় `nproc` -এর সাথে মিলিয়ে দেখো। আধুনিক kernel-এ আরও নিখুঁত মাপের জন্য PSI (Pressure Stall Information) আছে: `cat /proc/pressure/cpu` , বিস্তারিত অধ্যায় ৪৬-এ।

২৪.১৬ আরও কিছু কাজের টুল (সংক্ষেপে)

কমান্ড	কাজ
`pmap PID`	process-এর মেমরি ম্যাপ সুন্দর করে
`lsof -p PID`	process কোন কোন ফাইল/সকেট খুলে রেখেছে
`fuser -v /path`	কোন process এই ফাইল/ডিরেক্টরি ব্যবহার করছে (umount না হলে কাজের)
`strace -p PID`	process কী system call করছে লাইভ দেখো
`timeout 10s cmd`	১০ সেকেন্ড পর cmd-কে SIGTERM দাও
`watch -n 2 'ps ...'`	প্রতি ২ সেকেন্ডে কমান্ড আবার চালাও
`free -h`	মেমরির সারাংশ
`vmstat 1`	প্রতি সেকেন্ডে CPU, মেমরি, run queue (`r`) ও blocked (`b`) task
`systemd-cgls`	systemd-এর cgroup অনুযায়ী process গাছ
`loginctl` / `who`	কে লগইন করে আছে

হাতে-কলমে অনুশীলন

একটা টার্মিনাল খোলো (দুটো ট্যাব হলে আরও ভালো)। কোনো ধাপেই সিস্টেমের ক্ষতি হবে না।

ল্যাব ১: নিজের শেলের পরিচয়

```
$ echo "আমার PID: $$, parent: $PPID"
$ ps -o pid,ppid,stat,cmd -p $$
$ pstree -s -p $$
```

পর্যবেক্ষণ: তোমার শেল থেকে PID 1 পর্যন্ত পুরো বংশধারা দেখা গেল।

ল্যাব ২: fork দেখো

```
$ bash               # নতুন একটা শেল (child) চালাও
$ echo $$ $PPID      # PPID = আগের শেলের PID
$ ps --forest -o pid,ppid,cmd
$ exit               # আগের শেলে ফেরো
```

ল্যাব ৩: state পরিবর্তন নিজের চোখে

```
$ sleep 1000 &
[1] 8001
$ ps -o pid,stat,cmd -p 8001          # S (ঘুমাচ্ছে)
$ kill -STOP 8001
$ ps -o pid,stat,cmd -p 8001          # T (থেমে আছে)
$ kill -CONT 8001
$ ps -o pid,stat,cmd -p 8001          # আবার S
$ kill 8001                            # SIGTERM
```

ল্যাব ৪: একটা zombie বানাও ও সরাও

```
$ bash -c 'sleep 1 & exec sleep 120' &
[1] 8100
```

এখানে bash একটা `sleep 1` child বানিয়ে নিজেকে `exec` দিয়ে `sleep 120`-এ বদলে ফেলে। নতুন `sleep 120` জানে না যে তার একটা child আছে, তাই wait() করে না। ১ সেকেন্ড পর:

```
$ ps -o pid,ppid,stat,cmd --ppid 8100
    PID    PPID STAT CMD
   8101    8100 Z    [sleep] <defunct>
$ kill -9 8101                  # কিছুই হবে না, zombie আগেই মৃত
$ ps -o pid,stat -p 8101        # এখনো Z
$ kill 8100                     # parent মারো
$ ps -p 8101                    # zombie নেই, PID 1/subreaper সাফ করে দিয়েছে
```

ল্যাব ৫: SIGTERM বনাম SIGKILL

একটা স্ক্রিপ্ট বানাও যেটা SIGTERM catch করে (`trap` সম্পর্কে বিস্তারিত অধ্যায় ৪৩):

```
$ cat > ~/graceful.sh << 'EOF'
#!/bin/bash
trap 'echo "SIGTERM পেয়েছি, গুছিয়ে বের হচ্ছি..."; exit 0' TERM
echo "আমার PID: $$"
while true; do sleep 1; done
EOF
$ chmod +x ~/graceful.sh
$ ~/graceful.sh &
আমার PID: 8200
$ kill 8200
SIGTERM পেয়েছি, গুছিয়ে বের হচ্ছি...
$ ~/graceful.sh &
আমার PID: 8210
$ kill -9 8210                  # কোনো বার্তা নেই, সরাসরি মৃত্যু
[1]+  Killed                  ~/graceful.sh
```

(trap থাকা অবস্থায় bash ঘুমন্ত `sleep 1` শেষ হওয়ার পরেই handler চালায়, তাই বার্তা আসতে ১ সেকেন্ড পর্যন্ত লাগতে পারে।)

ল্যাব ৬: priority পরীক্ষা

দুটো CPU-খাদক চালাও, একই core-এ বেঁধে (`taskset -c 0`), একটার nice 19:

```
$ taskset -c 0 sh -c 'while :; do :; done' &
$ taskset -c 0 nice -n 19 sh -c 'while :; do :; done' &
$ top -o %CPU          # প্রথমটা ~৯৫%+, দ্বিতীয়টা কয়েক % মাত্র
$ pkill -f 'while :; do :; done'      # পরিষ্কার করো
```

ল্যাব ৭: top ও htop চর্চা

1. `top` চালাও। `1` চাপো (প্রতিটা core), `M` (মেমরি দিয়ে সাজাও), `c` (পুরো command), `V` (গাছ), `u` লিখে নিজের user, `q`।
2. `htop` চালাও। `F5` (গাছ), `F4` দিয়ে "bash" ফিল্টার, `F6` দিয়ে MEM% সাজাও, একটা `sleep` process-এ `F9` দিয়ে SIGTERM পাঠাও।

ল্যাব ৮: /proc অন্বেষণ

```
$ sleep 600 &
$ P=$!
$ cat /proc/$P/status | head -10
$ tr '\0' ' ' < /proc/$P/cmdline; echo
$ ls -l /proc/$P/cwd /proc/$P/exe
$ cat /proc/loadavg
$ kill $P
```

সারসংক্ষেপ

কমান্ড/শর্টকাট	কাজ
`echo $$` / `$PPID` / `$!`	নিজের PID / parent / শেষ background PID
`ps`	বর্তমান টার্মিনালের process
`ps aux`	সব process, CPU/MEM সহ
`ps -ef`	সব process, PPID সহ
`ps -ef --forest` / `ps auxf`	গাছ আকারে
`ps -eo pid,ppid,stat,%cpu,cmd --sort=-%cpu`	কাস্টম কলাম + সাজানো
`ps -eLf` / `ps -T -p PID`	thread দেখা
`pstree -p`	process গাছ, PID সহ
`top` / `htop`	লাইভ মনিটর
`top`: `P` `M` `T` `k` `r` `1` `c` `V` `q`	CPU / মেম / সময় সাজাও, kill, renice, core, cmd, গাছ, বের হও
`htop`: `F3` `F4` `F5` `F6` `F7` `F8` `F9` `F10`	খোঁজা, ফিল্টার, গাছ, সাজানো, nice-, nice+, kill, বের হও
`pgrep -a name`	নাম দিয়ে PID খোঁজা
`pkill [-SIG] name`	নাম দিয়ে signal
`pidof name`	হুবহু নামের PID
`kill PID`	SIGTERM (ভদ্র)
`kill -9 PID`	SIGKILL (জোর)
`kill -HUP PID`	reload
`kill -STOP` / `-CONT`	pause / resume
`kill -0 PID`	বেঁচে আছে কি না যাচাই
`kill -l`	signal তালিকা
`killall name`	নামের সব process
`nice -n 10 cmd`	কম priority-তে চালাও
`renice -n 5 -p PID`	চলমান process-এর priority বদলাও
`ionice -c 3 cmd`	idle I/O priority
`cat /proc/PID/status`	process-এর বিস্তারিত
`uptime` / `cat /proc/loadavg`	load average
`nproc`	CPU core সংখ্যা
`Ctrl+C`	SIGINT

কমান্ড/শর্টকাট	কাজ
`Ctrl+Z`	SIGTSTP (থামাও)
`Ctrl+\`	SIGQUIT
`Ctrl+D`	EOF (signal নয়)

নিজেকে যাচাই করো

1. program আর process-এর মধ্যে মূল পার্থক্য কী? একই program থেকে কয়টা process চলতে পারে?
2. একই process-এর দুটো thread কী শেয়ার করে আর কী আলাদা রাখে?
3. শেলে `ls` চালালে কোন কোন system call কোন ক্রমে ঘটে? `cd` কেন আলাদা process হিসেবে চলে না?
4. `S` আর `D` state-এর পার্থক্য কী? কোনটাকে `kill -9` দিয়েও সাথে সাথে মারা যায় না, এবং কেন?
5. একটা zombie process `kill -9` দিয়ে মরছে না। কেন, আর কী করবে?
6. `ps aux`-এর `%CPU` আর `top`-এর `%CPU`-এর পার্থক্য কী?
7. SIGTERM আর SIGKILL-এর পার্থক্য কী? কেন সবসময় আগে SIGTERM পাঠানো উচিত?
8. একটা সাধারণ user `nice -n -5 ./app` চালালে কী হবে? `renice -n 10` দেওয়ার পর কি সে আবার 0-তে ফিরিয়ে আনতে পারবে?
9. ৮ core সার্ভারে load average `12.5, 9.0, 4.2`। কী বুঝবে?
10. একটা প্রোগ্রাম "exit code 137" দিয়ে মারা গেছে। এর মানে কী?

▼ উত্তর দেখো

1. program হলো ডিস্কে রাখা নিষ্ক্রিয় executable ফাইল; process হলো সেই program-এর চলমান instance, যার নিজস্ব PID, মেমরি, খোলা ফাইল আছে। একই program থেকে যত খুশি process চলতে পারে (মেমরি ও PID সীমা পর্যন্ত)।
2. শেয়ার করে: code, heap, global data, খোলা file descriptor, PID। আলাদা: stack, CPU register, thread ID (TID)।
3. bash `fork()` করে নিজের কপি বানায় → child `execve("/usr/bin/ls")` করে → ls কাজ করে `exit()` করে → parent bash `wait()` দিয়ে exit code নেয়। `cd` আলাদা process হলে child-এর ডিরেক্টরি বদলাত, parent শেলের না; তাই এটা shell builtin।
4. `S` (interruptible sleep) signal পেলে জেগে ওঠে; `D` (uninterruptible) সাধারণত kernel-এর ভেতরে I/O সম্পূর্ণ হওয়ার অপেক্ষায় থাকে এবং signal-এ জাগে না। তাই `D`-তে থাকা process-এ SIGKILL পৌঁছায় না যতক্ষণ না I/O শেষ হয়।
5. zombie আগেই মৃত, শুধু process table-এ exit status-এর এন্ট্রি বাকি। মারার কিছু নেই। parent-কে `SIGCHLD` পাঠিয়ে wait() করাতে পারো, না হলে parent-কে বন্ধ/restart করো; তখন zombie PID 1 (বা subreaper)-এর কাছে গিয়ে সাফ হয়ে যায়।
6. `ps`-এর `%CPU` হলো process শুরু থেকে এখন পর্যন্ত গড়; `top`-এর `%CPU` হলো শেষ রিফ্রেশের পর থেকে, মানে এই মুহূর্তের ব্যবহার।

7. SIGTERM (15) একটা অনুরোধ, process catch করে পরিষ্কারভাবে (ফাইল সেভ, lock মুছে) বের হতে পারে। SIGKILL (9) kernel সরাসরি কার্যকর করে, catch/ignore করা যায় না, কোনো পরিষ্কারের সুযোগ নেই, ফলে ডেটা নষ্ট বা stale lock থাকতে পারে।
8. `nice: cannot set niceness: Permission denied` বার্তা আসবে (GNU nice তবু কমান্ডটা ডিফল্ট priority-তে চালিয়ে দেয়)। ঋণাত্মক nice শুধু root দিতে পারে। আর nice বাড়ানোর পর সাধারণ user সেটা কমিয়ে আগের অবস্থায় ফেরাতে পারে না (limits.conf-এ অনুমতি না থাকলে)।
9. ১ মিনিটের গড় 12.5 > 8 core, তাই এই মুহূর্তে task-রা অপেক্ষা করছে (overloaded)। ১৫ মিনিটে 4.2 থেকে বাড়তে বাড়তে 12.5, মানে চাপ সাম্প্রতিক এবং বাড়ছে। `top`-এ দেখো CPU (`us` / `sy`) নাকি I/O (`wa`, D state) এর কারণ।
10. 137 = 128 + 9, মানে process SIGKILL পেয়ে মারা গেছে। প্রায়ই এটা মেমরি ফুরিয়ে OOM killer-এর কাজ, বা কেউ `kill -9` দিয়েছে (`journalctl -k | grep -i oom` দিয়ে যাচাই করো)।

অধ্যায় ২৫: জব কন্ট্রোল, nohup, screen ও tmux

এই অধ্যায়ে যা শিখবে

- **foreground** আর **background** process কী, আর কেন এই দুটো আলাদা
- `&` দিয়ে কমান্ড background-এ চালানো, `$!` আর `wait` দিয়ে তাকে ধরা
- `Ctrl+Z` চাপলে ভেতরে কী হয় (SIGTSTP), `jobs`, `fg`, `bg` দিয়ে job নিয়ন্ত্রণ
- job specification: `%1`, `%+`, `%-`, `%%`, `%name`, `%?text`
- টার্মিনাল বন্ধ করলে process কেন মরে যায়: **SIGHUP**, session, process group, controlling terminal
- `disown`, `nohup`, `setsid` দিয়ে process-কে টার্মিনাল থেকে মুক্ত করা
- systemd-logind-এর `KillUserProcesses` আর `loginctl enable-linger`
- **GNU screen**: session, window, detach/attach, split, সব `Ctrl+A` শর্টকাট
- **tmux** পুরোটা: client-server architecture, session/window/pane, `Ctrl+B` prefix-এর সব শর্টকাট, split, resize, copy mode, `.tmux.conf`, scripting
- বাস্তব কাজের ধরন: SSH-এ লম্বা কাজ চালানো, সার্ভারে নিরাপদে deploy

---

২৫.১ গল্প দিয়ে শুরু: রান্নাঘরের চুলা

ধরো তুমি রান্নাঘরে একা। একটা চুলায় ভাত বসিয়েছ। তুমি দুইভাবে কাজ করতে পারো:

1. **চুলার সামনে দাঁড়িয়ে থাকো** যতক্ষণ না ভাত হয়। এই সময়ে আর কিছু করতে পারছ না। এটা **foreground**।
2. **ভাত বসিয়ে দিয়ে সবজি কাটতে চলে যাও।** ভাত নিজের মতো সেদ্ধ হচ্ছে, তুমি অন্য কাজ করছ। এটা **background**।

মাঝে মাঝে তুমি ভাতের হাঁড়িটা চুলা থেকে নামিয়ে পাশে রেখে দাও (**pause**, মানে থামিয়ে রাখা), পরে আবার চুলায় তোলো (**resume**)।

এখন ধরো তুমি রান্নাঘর থেকে বেরিয়ে বাসা থেকেই চলে গেলে, আর যাওয়ার সময় গ্যাসের মেইন সুইচ বন্ধ করে দিলে। সব চুলা নিভে গেল। টার্মিনাল বন্ধ করলে ঠিক এটাই হয়: টার্মিনাল হলো "মেইন সুইচ", আর তার সাথে যুক্ত সব process একটা **SIGHUP** সিগন্যাল পেয়ে মরে যায়।

এই অধ্যায়ে আমরা শিখব:

- কীভাবে একই টার্মিনালে অনেক "চুলা" (job) সামলাবে → **job control**
- কীভাবে একটা চুলা মেইন সুইচ থেকে আলাদা করবে → **nohup, disown, setsid**

- কীভাবে পুরো রান্নাঘরটাই এমনভাবে বানাবে যে তুমি বাইরে গেলেও সব চলতে থাকে, আর ফিরে এসে ঠিক যেখানে ছিলে সেখান থেকে শুরু করতে পারো → **screen ও tmux**

> 📝 **মনে রাখো:** process কী, PID কী, signal কী, `kill` কীভাবে কাজ করে — এগুলো অধ্যায় ২৪-এ বিস্তারিত আছে। এখানে আমরা সেই জ্ঞানকে টার্মিনাল আর shell-এর দৃষ্টিকোণ থেকে ব্যবহার করব।

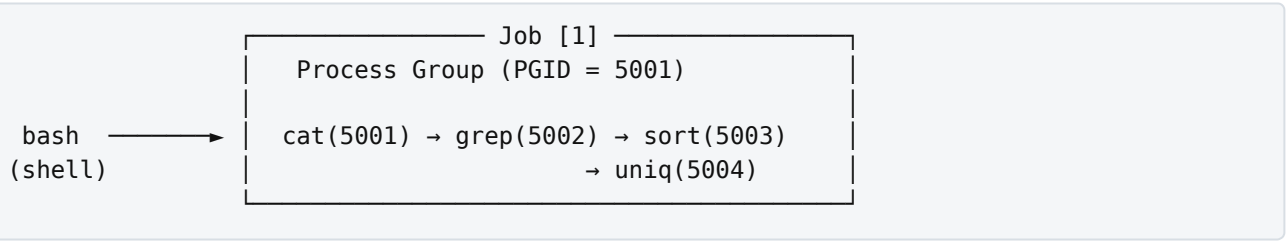
২৫.২ Job আর Process: পার্থক্য কী?

Process হলো kernel-এর ধারণা: চলমান একটা প্রোগ্রাম, যার একটা PID আছে।

Job হলো **shell-এর** ধারণা: তুমি একবার Enter চেপে যে কমান্ড লাইন চালাও, shell সেটাকে একটা job হিসেবে ধরে। একটা job-এ একাধিক process থাকতে পারে। যেমন:

```
$ cat access.log | grep 404 | sort | uniq -c
```

এখানে চারটা process (`cat`, `grep`, `sort`, `uniq`), কিন্তু **job একটাই**। shell এই চারটাকে একটা **process group**-এ রাখে, যাতে একসাথে থামানো, চালানো বা মারা যায়।



> 💡 **টিপস:** job number (`[1]`, `[2]`...) শুধু ওই shell-এর ভেতরে অর্থবহ। অন্য টার্মিনাল থেকে `%1` লিখলে কাজ করবে না; সেখানে PID ব্যবহার করতে হবে।

২৫.৩ ভেতরে কীভাবে কাজ করে: Session, Process Group, Controlling Terminal

job control বুঝতে হলে Unix-এর তিনটা স্তর বুঝতে হবে। একটু তাত্ত্বিক, কিন্তু এটা বুঝলে বাকি সব (SIGHUP, nohup, setsid, tmux) একদম পরিষ্কার হয়ে যাবে।

Session (সেশন)

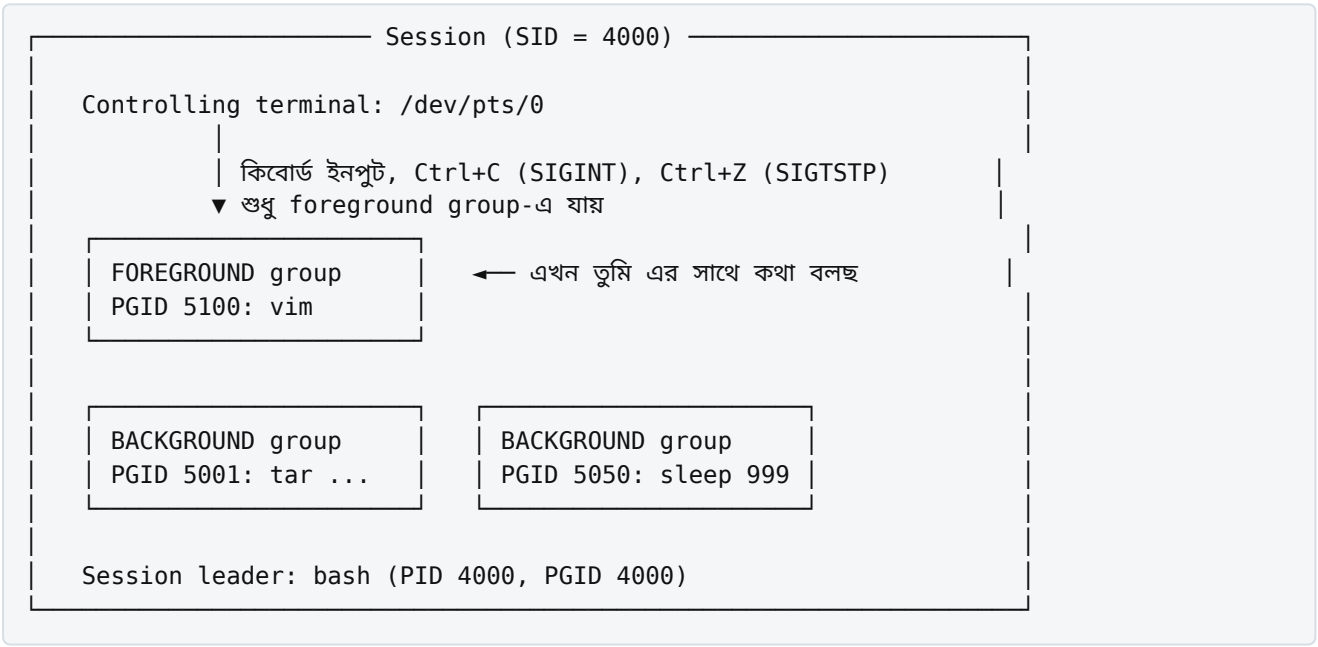
তুমি যখন টার্মিনাল খোলো বা SSH দিয়ে লগইন করো, তখন একটা **session** তৈরি হয়। সাধারণত তোমার shell (`bash`) হয় **session leader**। session-এর একটা ID থাকে (SID), যা session leader-এর PID-এর সমান।

Process Group (প্রসেস গ্রুপ)

session-এর ভেতরে প্রতিটা job একটা আলাদা **process group**। একটা সময়ে ঠিক **একটা** process group থাকে **foreground**-এ; বাকিগুলো **background**।

Controlling Terminal (নিয়ন্ত্রণকারী টার্মিনাল)

session-টা একটা টার্মিনাল ডিভাইসের সাথে বাঁধা থাকে, যেমন `/dev/pts/0`। এটাকে বলে **controlling terminal**। টার্মিনাল ঠিক করে কিবোর্ডের ইনপুট আর `Ctrl+C`, `Ctrl+Z`-এর মতো সিগন্যাল কোন process group পাবে: শুধু **foreground group**।



নিজের চোখে দেখো:

```
$ sleep 300 &
[1] 5050
$ ps -o pid,ppid,pgid,sid,tty,stat,cmd
    PID    PPID    PGID     SID TT       STAT CMD
   4000    3990    4000    4000 pts/0    Ss   bash
   5050    4000    5050    4000 pts/0    S    sleep 300
   5061    4000    5061    4000 pts/0    R+   ps -o pid,ppid,pgid,sid,tty,stat,cmd
```

লক্ষ করো:

- সবার `SID` একই (4000) → সবাই একই session-এ।
- প্রত্যেক job-এর নিজস্ব `PGID`।

- `STAT` কলামে `s` মানে session leader, `+` মানে foreground process group-এর সদস্য। `ps`-এর পাশে `+` আছে, `sleep`-এর পাশে নেই।

Background process টার্মিনাল থেকে পড়তে চাইলে কী হয়?

একটা background process যদি কিবোর্ড থেকে **ইনপুট পড়তে** চায়, kernel তাকে **SIGTTIN** সিগন্যাল পাঠিয়ে থামিয়ে দেয়। কারণ কিবোর্ড শুধু foreground-এর জন্য।

```
$ read name &
[1] 5200
[1]+  Stopped                 read name
```

লেখার ক্ষেত্রে ডিফল্টভাবে background process টার্মিনালে লিখতে পারে (তাই কখনো কখনো আউটপুট এলোমেলোভাবে prompt-এর মাঝে চলে আসে)। তবে `stty tostop` চালু করলে background-এর লেখার চেষ্টাতেও **SIGTTOU** পাঠিয়ে থামানো হয়।

```
$ stty tostop        # চালু
$ stty -tostop       # বন্ধ (ডিফল্ট)
```

২৫.৪ Foreground আর Background-এ কমান্ড চালানো

Foreground (ডিফল্ট)

```
$ sleep 10
```

10 সেকেন্ড prompt ফেরত আসবে না। shell অপেক্ষা করছে (`waitpid()` system call দিয়ে)।

`&` দিয়ে Background-এ চালানো

কমান্ডের শেষে `&` দিলে shell কমান্ড চালু করেই সাথে সাথে prompt ফেরত দেয়।

```
$ sleep 100 &
[1] 6123
$
```

- `[1]` হলো **job number**
- `6123` হলো ওই job-এর শেষ process-এর **PID** (pipeline হলে শেষ কমান্ডের PID)

কয়েকটা চালাও:

```
$ sleep 200 &
[2] 6130
```

```
$ sleep 300 &
[3] 6131
```

job শেষ হলে পরের বার Enter চাপলে shell জানায়:

```
$
[1]   Done                    sleep 100
```

> 💡 **টিপস:** shell সাধারণত পরের prompt দেখানোর ঠিক আগে job-এর অবস্থা জানায়। সাথে সাথে জানতে চাইলে `set -b` (বা `set -o notify`) চালু করো।

`$!` : শেষ background process-এর PID

```
$ gzip -k big.log &
[1] 6200
$ echo $!
6200
```

স্ক্রিপ্টে এটা খুব কাজের:

```
long_task &
pid=$!
echo "কাজ চলছে, PID = $pid"
wait "$pid"
echo "শেষ হয়েছে, exit status = $?"
```

`wait` : background job শেষ হওয়া পর্যন্ত অপেক্ষা

কমান্ড	কাজ
`wait`	সব background child শেষ হওয়া পর্যন্ত অপেক্ষা
`wait PID`	নির্দিষ্ট PID শেষ হওয়া পর্যন্ত অপেক্ষা, তার exit status ফেরত দেয়
`wait %2`	job 2-এর জন্য অপেক্ষা
`wait -n`	যেকোনো **একটা** job শেষ হলেই ফেরত আসে (bash 4.3+)

উদাহরণ — তিনটা ডাউনলোড সমান্তরালে:

```
curl -sO https://example.com/a.iso &
curl -sO https://example.com/b.iso &
curl -sO https://example.com/c.iso &
wait
echo "তিনটাই নামানো শেষ"
```

`&` ছাড়া `;` আর `&&`-এর পার্থক্য

লেখা	অর্থ
`a ; b`	a শেষ হলে b, a-এর ফলাফল যাই হোক
`a && b`	a সফল হলেই b
`a & b`	a background-এ, b সাথে সাথে foreground-এ
`(a; b) &`	পুরো গ্রুপটা একটা subshell-এ background-এ

> ⚠️ **সতর্কতা:** `&` দিয়ে চালালেও process তখনো এই টার্মিনালের session-এর সদস্য। টার্মিনাল বন্ধ করলে সাধারণত এটা মরে যাবে। নিচে ২৫.৭-এ দেখব কেন।

২৫.৫ `Ctrl+Z`, `jobs`, `fg`, `bg`

`Ctrl+Z`: চলমান কাজ থামানো (suspend)

ধরো তুমি একটা বড় ফাইল কপি শুরু করলে, কিন্তু `&` দিতে ভুলে গেছ:

```
$ cp ubuntu.iso /mnt/usb/
```

এখন prompt আটকে আছে। `Ctrl+Z` চাপো:

```
^Z
[1]+  Stopped                 cp ubuntu.iso /mnt/usb/
$
```

ভেতরে কী হলো?

1. তুমি `Ctrl+Z` চাপলে টার্মিনাল ড্রাইভার (kernel-এর tty layer) এটাকে সাধারণ অক্ষর হিসেবে না দেখে চিনে নেয়, কারণ এটা `susp` ক্যারেক্টার হিসেবে সেট করা (`stty -a` দিয়ে দেখো: `susp = ^Z`)।
2. kernel foreground process group-এর সব process-কে **SIGTSTP** পাঠায়।
3. process-গুলো **Stopped** অবস্থায় যায় (`ps`-এ `STAT` = `T`)। CPU ব্যবহার করে না, কিন্তু মেমরিতে থাকে।
4. shell দেখে তার child থেমে গেছে, তাই টার্মিনালের foreground নিজের কাছে ফিরিয়ে নিয়ে prompt দেখায়।

```
Ctrl+Z ──► tty driver ──► SIGTSTP ──► foreground group (cp)  ──► STOPPED (T)
                                              │
                                  bash টার্মিনাল ফেরত পায় ◄──┘
```

> 📝 **মনে রাখো:** `Ctrl+Z` কাজটা **মারে না**, শুধু থামায়। `Ctrl+C` (SIGINT) সাধারণত মেরে ফেলে। দুটো গুলিয়ে ফেলো না।

`jobs` : এই shell-এর সব job-এর তালিকা

```
$ sleep 500 &
[2] 6301
$ vim notes.txt
^Z
[3]+  Stopped                 vim notes.txt
$ jobs
[1]   Stopped                 cp ubuntu.iso /mnt/usb/
[2]   Running                 sleep 500 &
[3]+  Stopped                 vim notes.txt
```

- `+` = **current job** (সবচেয়ে সাম্প্রতিক; `fg` / `bg` অপশন ছাড়া দিলে এটাই ধরে)
- `-` = **previous job** (এর আগেরটা)

অপশন	কাজ
`jobs -l`	job number-এর সাথে PID-ও দেখায়
`jobs -p`	শুধু process group leader-এর PID দেখায় (স্ক্রিপ্টে কাজের)
`jobs -r`	শুধু running job
`jobs -s`	শুধু stopped job
`jobs -n`	শেষবার জানানোর পর যাদের অবস্থা বদলেছে

```
$ jobs -l
[1]  6250 Stopped                 cp ubuntu.iso /mnt/usb/
[2]  6301 Running                 sleep 500 &
[3]+ 6310 Stopped                 vim notes.txt
```

`bg` : থামানো job-কে background-এ চালু করা

```
$ bg %1
[1]  cp ubuntu.iso /mnt/usb/ &
$ jobs
[1]   Running                 cp ubuntu.iso /mnt/usb/ &
[2]-  Running                 sleep 500 &
[3]+  Stopped                 vim notes.txt
```

ভেতরে: shell ওই process group-কে **SIGCONT** পাঠায়, কিন্তু টার্মিনালের foreground নিজের কাছেই রাখে।

> 💡 **টিপস:** "ভুলে `&` দিইনি" সমস্যার ক্লাসিক সমাধান: `Ctrl+Z` তারপর `bg`। কাজটা আবার শুরু থেকে চালাতে হয় না।

`fg`: job-কে foreground-এ আনা

```
$ fg %3
vim notes.txt
```

vim আবার পর্দায় ফিরে আসবে, ঠিক যেখানে ছিলে। ভেতরে shell দুটো কাজ করে: `tcsetpgrp()` দিয়ে টার্মিনালের foreground group বদলে ওই job-কে দেয়, তারপর SIGCONT পাঠায় (job থেমে থাকলে)।

সবচেয়ে বাস্তব কাজের ধরন: vim ↔ shell

প্রোগ্রামাররা প্রতিদিন এটা করে:

```
$ vim app.py      # কোড লিখছ
^Z                # shell-এ এলে
$ python3 app.py  # চালিয়ে দেখলে
$ fg              # vim-এ ফেরত
```

> ⚠️ **সতর্কতা:** stopped job রেখে `exit` করলে bash সতর্ক করে:
>
> ```
> $ exit
> exit
> There are stopped jobs.
> ```
>
> আবার `exit` দিলে সে বেরিয়ে যাবে এবং stopped job-গুলো SIGHUP পেয়ে মারা যাবে। vim-এ unsaved কাজ থাকলে তা হারাবে (vim অবশ্য swap ফাইল রেখে যায়, অধ্যায় ১৪ দেখো)।

২৫.৬ Job Specification: `%`-এর পুরো ভাষা

`fg`, `bg`, `kill`, `wait`, `disown` সবাই এই নোটেশন বোঝে:

লেখা	অর্থ	উদাহরণ
`%n`	job number n	`fg %2`
`%+` বা `%%`	current job (যার পাশে `+`)	`bg %%`
`%-`	previous job (যার পাশে `-`)	`fg %-`
`%string`	যে job-এর কমান্ড `string` দিয়ে **শুরু**	`fg %vim`

লেখা	অর্থ	উদাহরণ
`%?string`	যে job-এর কমান্ডে কোথাও `string` আছে	`kill %?iso`

শর্টকাট:

- শুধু `fg` বা `bg` = `%+`
- শুধু `%1` লিখে Enter = `fg %1`
- `%1 &` লিখে Enter = `bg %1`

```
$ %2          # job 2 foreground-এ
$ %vim &      # vim নামের job background-এ (যদিও vim background-এ বসে থামবে)
```

> ⚠️ **সতর্কতা:** `%string` যদি একাধিক job-এর সাথে মেলে, bash ভুল দেখায়: `bash: fg: vim: ambiguous job spec`। তখন নম্বর ব্যবহার করো।

`kill` দিয়ে job মারা

```
$ kill %1             # SIGTERM পাঠায় job 1-এর পুরো process group-এ
$ kill -9 %2          # SIGKILL
$ kill -CONT %3       # থামানো job চালু (কিন্তু background-এ থাকবে)
$ kill -STOP %2       # Ctrl+Z-এর মতো, তবে SIGSTOP ধরা/উপেক্ষা করা যায় না
```

> 📝 **মনে রাখো:** `kill %1` হলো bash-এর **builtin** `kill`-এর সুবিধা। `/bin/kill %1` কাজ করবে না, কারণ বাইরের প্রোগ্রাম shell-এর job table জানে না।

stopped job-কে সিগন্যাল পাঠালে কী হয়? থেমে থাকা process বেশিরভাগ সিগন্যাল হাতে পায় চালু হওয়ার পরে। bash-এর builtin `kill` এটা জানে: stopped job-কে SIGTERM বা SIGHUP পাঠালে সে নিজে থেকেই সাথে একটা SIGCONT-ও পাঠায়, তাই job জেগে উঠে সিগন্যাল পেয়ে শেষ হয়। অন্য সিগন্যালে (যেমন `kill -INT %1`) job "Stopped"-ই থেকে যেতে পারে; তখন `kill -CONT %1` দাও, বা শেষ উপায় `kill -9 %1` (SIGKILL থামানো process-কেও সাথে সাথে মারে)।

২৫.৭ SIGHUP: টার্মিনাল বন্ধ করলে process কেন মরে?

"Hang up" শব্দটা কোথা থেকে এল

পুরনো যুগে মানুষ টেলিফোন লাইন আর modem দিয়ে কম্পিউটারে যুক্ত হতো। ফোন রেখে দিলে (hang up) লাইন কেটে যেত। তখন kernel ওই লাইনের session-কে জানাত: "ইউজার চলে গেছে।" সেই সিগন্যালের নাম

SIGHUP (signal hang up, নম্বর 1)। আজ modem নেই, কিন্তু নিয়মটা আছে: টার্মিনাল উইন্ডো বন্ধ করা বা SSH সংযোগ কেটে যাওয়া = "hang up"।

ধাপে ধাপে কী ঘটে

```
তুমি টার্মিনাল উইন্ডো বন্ধ করলে / SSH কেটে গেল
              │
              ▼
① pseudo-terminal-এর master side বন্ধ হলো (GNOME Terminal বা sshd সেটা ধরে রাখত)
              │
              ▼
② kernel controlling terminal-এর session leader-কে (bash) SIGHUP পাঠায়
              │
              ▼
③ bash SIGHUP পেয়ে বেরোনোর আগে তার job table-এর সব job-কে SIGHUP পাঠায়
   (stopped job-কে SIGCONT-ও পাঠায় যাতে তারা সিগন্যালটা হাতে পায়)
              │
              ▼
④ SIGHUP-এর ডিফল্ট কাজ = process শেষ করা  →  তোমার background কাজ মারা গেল
```

এছাড়া session leader মারা গেলে kernel নিজেও foreground process group-কে SIGHUP পাঠায়, আর কোনো process group "orphaned" হয়ে গেলে এবং তাতে stopped process থাকলে তাকে SIGHUP + SIGCONT পাঠায়।

bash-এর `huponexit` অপশন

ওপরের ধাপ ③ ঘটে যখন bash **SIGHUP পেয়ে** বেরোয়। কিন্তু তুমি যদি নিজে `exit` টাইপ করে বা `Ctrl+D` দিয়ে বেরোও, তখন কী? তখন bash ডিফল্টভাবে running background job-দের SIGHUP **পাঠায় না** (stopped job থাকলে আগে সতর্ক করে)। এটা নিয়ন্ত্রণ করে `huponexit`:

```
$ shopt huponexit
huponexit       off
```

`shopt -s huponexit` দিলে login shell থেকে `exit` করার সময়ও সব job-কে SIGHUP পাঠাবে।

> 💡 **টিপস:** এজন্যই দেখা যায় "`&` দিয়ে চালিয়ে `exit` দিলাম, কাজ চলছে, কিন্তু উইন্ডো ক্রস দিয়ে বন্ধ করলাম তো কাজ মরে গেল।" উইন্ডো বন্ধ করা = SIGHUP, `exit` = SIGHUP নয়। তবে এর ওপর ভরসা কোরো না; নিচের পদ্ধতিগুলো ব্যবহার করো।

systemd-logind-এর ভূমিকা: `KillUserProcesses`

আধুনিক distro-তে আরেকটা স্তর আছে। systemd-logind প্রতিটা লগইনকে একটা **scope** (cgroup)-এ রাখে। `/etc/systemd/logind.conf`-এ:

```
[Login]
KillUserProcesses=no
```

- `no` (Ubuntu, Debian, Fedora, RHEL-এ ডিফল্ট): লগআউটের পরও nohup/disown/tmux দিয়ে চালানো process বেঁচে থাকে।
- `yes` : লগআউট করলেই ওই session-এর cgroup-এর **সব** process মেরে ফেলা হয়, nohup দিলেও। এমনকি tmux-ও মরে যাবে!

`yes` থাকা সিস্টেমে কাজ বাঁচানোর উপায়:

```
$ loginctl enable-linger $USER                    # ইউজারের systemd instance লগআউটেও চালু থাকবে
$ systemd-run --user --scope tmux new -s work     # tmux-কে আলাদা scope-এ চালাও
```

> 📝 **মনে রাখো:** সত্যিকারের দীর্ঘমেয়াদি সার্ভিস (ওয়েব সার্ভার, ব্যাকগ্রাউন্ড worker) tmux বা nohup দিয়ে চালানো ঠিক নয়। ওগুলো systemd service বানাও, অধ্যায় ২৬ দেখো। tmux/nohup হলো এক-দুবারের লম্বা কাজের জন্য (বড় কপি, compile, migration, ডেটা প্রসেসিং)।

২৫.৮ `disown` : job table থেকে বাদ দেওয়া

`disown` bash-এর builtin। সে job-কে shell-এর job table থেকে সরিয়ে দেয়, ফলে টার্মিনাল বন্ধ হলে bash যখন সব job-কে SIGHUP পাঠায়, এই job সেই তালিকায় থাকে না।

```
$ ./long_backup.sh &
[1] 7001
$ disown %1
$ jobs
$                    # তালিকা খালি, কিন্তু process চলছে
$ ps -p 7001
    PID TTY          TIME CMD
   7001 pts/0    00:00:03 long_backup.sh
```

অপশন	কাজ
`disown`	current job (`%+`) সরায়
`disown %n`	নির্দিষ্ট job সরায়
`disown -a`	সব job সরায়
`disown -r`	শুধু running job সরায়
`disown -h %n`	job table-এ রাখে (`jobs` -এ দেখা যাবে), কিন্তু চিহ্ন দিয়ে রাখে যাতে shell তাকে SIGHUP না পাঠায়

সবচেয়ে কাজের রেসিপি: ভুলে nohup ছাড়া চালিয়ে ফেলেছ, এখন বাড়ি যেতে হবে:

```
$ ./import_data.sh          # foreground-এ চলছে
^Z
[1]+  Stopped                 ./import_data.sh
$ bg
[1]+ ./import_data.sh &
$ disown -h %1              # বা: disown %1
$ exit
```

⚠️ **সতর্কতা:** `disown`-এর পরেও process-এর stdout/stderr ওই টার্মিনালেই যুক্ত। টার্মিনাল চলে গেলে সে লিখতে চেষ্টা করলে write error (EIO) বা SIGPIPE পেতে পারে, আর কিছু প্রোগ্রাম তাতে মারা যায়। নিরাপদ পথ হলো শুরু থেকেই আউটপুট ফাইলে পাঠানো: `./import_data.sh > import.log 2>&1 &`

📝 **মনে রাখো:** `disown` kernel-এর SIGHUP বন্ধ করে না, শুধু **bash**-এর পাঠানো SIGHUP থেকে বাঁচায়। process ভিন্ন process group-এ background-এ থাকলে kernel সাধারণত সরাসরি তাকে hang-up পাঠায় না, তাই বাস্তবে এটাই যথেষ্ট হয়।

২৫.৯ `nohup`: SIGHUP উপেক্ষা করে চালানো

`nohup` (no hang up) একটা ছোট প্রোগ্রাম (coreutils-এর অংশ)। সে যা করে:

1. নিজের জন্য SIGHUP-কে **ignore** (SIG_IGN) সেট করে।
2. তারপর `exec` দিয়ে তোমার কমান্ডে রূপান্তরিত হয়। ignore করা সিগন্যাল `exec`-এর পরেও ignore থাকে, তাই তোমার প্রোগ্রাম SIGHUP পেলে কিছুই হয় না।
3. stdout যদি টার্মিনাল হয়, আউটপুট `nohup.out` ফাইলে পাঠায় (বর্তমান ডিরেক্টরিতে, লেখা না গেলে `$HOME/nohup.out`)।
4. stderr টার্মিনাল হলে সেটাকেও stdout-এ পাঠায়।
5. stdin টার্মিনাল হলে সেটাকে একটা অপাঠযোগ্য ফাইল থেকে redirect করে (ইনপুট ignore)।

Syntax

```
nohup COMMAND [ARGS...]
```

উদাহরণ

```
$ nohup ./train_model.py &
[1] 7200
$ nohup: ignoring input and appending output to 'nohup.out'
```

নিজে log ফাইল ঠিক করে দেওয়া (প্রস্তাবিত পদ্ধতি):

```
$ nohup ./train_model.py > train.log 2>&1 &
[1] 7210
$ tail -f train.log
Epoch 1/50 ... loss 0.84
Epoch 2/50 ... loss 0.71
```

এবার নিশ্চিন্তে SSH থেকে বেরিয়ে যাও। পরে ফিরে এসে:

```
$ pgrep -af train_model
7210 python3 ./train_model.py
$ tail train.log
```

> ⚠️ **সতর্কতা:**
>
> - `nohup` নিজে process-কে background-এ পাঠায় না। শেষে `&` দিতেই হবে।
> - `nohup` শুধু SIGHUP থেকে বাঁচায়। `Ctrl+C` (foreground-এ থাকলে), SIGTERM, SIGKILL, OOM killer বা `KillUserProcesses=yes` থেকে বাঁচায় না।
> - `nohup cd /tmp` বা `nohup alias` কাজ করে না। nohup একটা আলাদা প্রোগ্রাম, তাই shell builtin, alias বা function চালাতে পারে না। দরকার হলে: `nohup bash -c 'cmd1 && cmd2' > out.log 2>&1 &`

nohup বনাম disown

বিষয়	`nohup`	`disown`
কে	আলাদা প্রোগ্রাম (`/usr/bin/nohup`)	bash builtin
কখন ব্যবহার	কমান্ড **শুরুর আগে**	কমান্ড **চালু হয়ে যাওয়ার পরে**
কীভাবে বাঁচায়	process নিজেই SIGHUP ignore করে	bash তাকে SIGHUP পাঠানোর তালিকা থেকে বাদ দেয়
আউটপুট	স্বয়ংক্রিয়ভাবে `nohup.out`-এ	কোনো পরিবর্তন নেই
অন্য কেউ `kill -HUP` দিলে	বেঁচে যায়	মারা যায়

২৫.১০ `setsid`: সম্পূর্ণ নতুন session

`setsid` (util-linux প্যাকেজ) process-কে একদম **নতুন session**-এ চালায়। নতুন session মানে:

- নতুন SID, process নিজেই session leader

- **কোনো controlling terminal নেই**

টার্মিনালের সাথে কোনো সম্পর্কই নেই, তাই টার্মিনাল বন্ধ হলে তার কাছে SIGHUP যাওয়ার প্রশ্নই ওঠে না। এটা অনেকটা daemon-এর মতো।

```
আগে:  Session(bash, pts/0) ──► job: ./server
পরে:  Session(bash, pts/0)      Session(./server, tty = ?)   ← সম্পূর্ণ আলাদা
```

Syntax ও অপশন

```
setsid [options] program [arguments]
```

অপশন	কাজ
`-f`, `--fork`	সবসময় fork করে নতুন process-এ চালায় (shell সাথে সাথে prompt ফেরত পায়)
`-w`, `--wait`	প্রোগ্রাম শেষ হওয়া পর্যন্ত অপেক্ষা করে তার exit status ফেরত দেয়
`-c`, `--ctty`	বর্তমান টার্মিনালকে নতুন session-এর controlling terminal বানায়

উদাহরণ

```
$ setsid -f ./worker.sh > worker.log 2>&1 < /dev/null
$ ps -o pid,pgid,sid,tty,cmd -p $(pgrep -f worker.sh)
    PID    PGID     SID TT       CMD
   7400    7400    7400 ?        /bin/bash ./worker.sh
```

`TT` কলামে `?` মানে কোনো টার্মিনাল নেই, `SID` = নিজের PID।

> 💡 **টিপস:** GUI অ্যাপ টার্মিনাল থেকে খুলে টার্মিনাল বন্ধ করতে চাইলে: `setsid -f firefox >/dev/null 2>&1`

তিনটার তুলনা এক নজরে

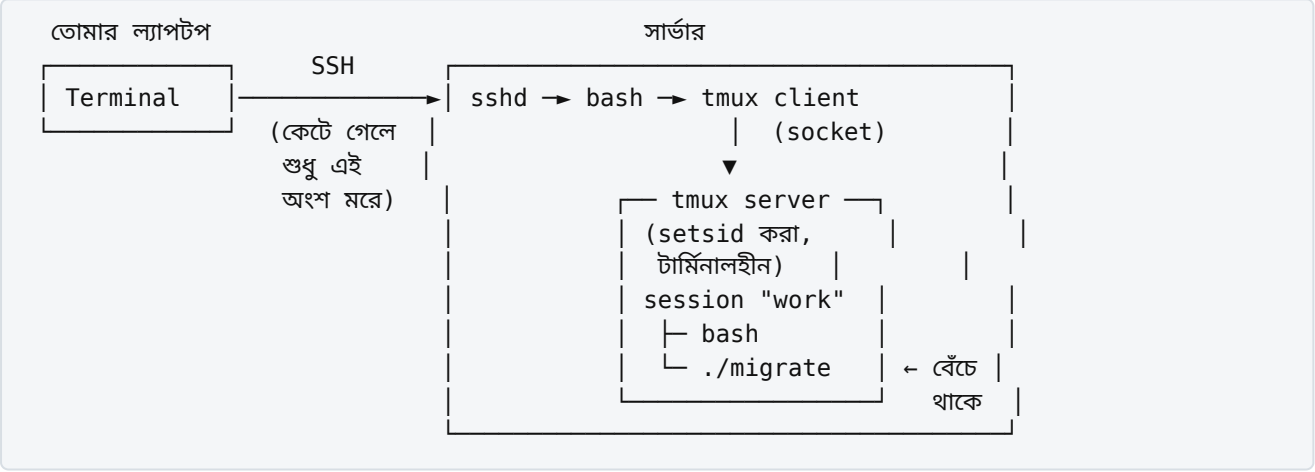
পদ্ধতি	কখন	টার্মিনাল থেকে বিচ্ছিন্নতার মাত্রা
`cmd &`	দ্রুত, টার্মিনাল খোলা থাকবে	নেই
`cmd & disown`	চালানোর পরে বাঁচাতে	মাঝারি
`nohup cmd &`	শুরুতেই জানো লম্বা চলবে	ভালো
`setsid -f cmd`	পুরোপুরি আলাদা করতে চাও	সম্পূর্ণ
`tmux` / `screen`	পরে ফিরে এসে আউটপুট দেখতে/ইন্টারঅ্যাক্ট করতে চাও	সম্পূর্ণ + ফিরে আসা যায়
`systemd-run` /service	স্থায়ী সার্ভিস	সম্পূর্ণ + পরিচালিত

একটা বড় সীমাবদ্ধতা: nohup/disown/setsid দিয়ে চালানো process-এর কাছে তুমি আর **ফিরে যেতে পারবে না**। তার পর্দা দেখা বা কিবোর্ড দিয়ে উত্তর দেওয়া সম্ভব নয়। এই সমস্যার সমাধানই হলো **terminal multiplexer**: screen আর tmux।

২৫.১১ Terminal Multiplexer: ধারণা

Multiplexer মানে একটা জিনিস দিয়ে অনেককে চালানো। terminal multiplexer একটা টার্মিনাল উইন্ডোর ভেতরে অনেকগুলো "ভার্চুয়াল টার্মিনাল" দেয়, আর সবচেয়ে বড় কথা: এগুলো তোমার টার্মিনাল থেকে **বিচ্ছিন্ন (detach)** হয়েও বেঁচে থাকে।

উপমা: টেলিভিশন আর কেবল অপারেটর। টিভি (তোমার টার্মিনাল) বন্ধ করলেও কেবল অপারেটরের কাছে চ্যানেল (tmux session) চলতেই থাকে। আবার টিভি খুললে (attach) দেখবে অনুষ্ঠান সামনে এগিয়ে গেছে। চাইলে অন্য ঘরের টিভিতেও (অন্য কম্পিউটার থেকে SSH) একই চ্যানেল দেখা যায়।



কেন কাজ করে: tmux/screen-এর server process নিজেকে আলাদা session-এ (setsid-এর মতো) নিয়ে যায় এবং তোমার প্রোগ্রামগুলোকে **নিজের তৈরি pseudo-terminal**-এ চালায়। তোমার আসল টার্মিনাল শুধু একটা "জানালা" যা দিয়ে তুমি ওই pseudo-terminal দেখছ। জানালা ভাঙলেও ঘর ঠিক থাকে।

ইনস্টলেশন

```
# Ubuntu / Debian
$ sudo apt install tmux screen

# Fedora
$ sudo dnf install tmux screen

# RHEL 9 / Rocky / Alma: tmux base repo-তে আছে; screen RHEL 8 থেকে বাদ পড়েছে, EPEL-এ পাওয়া যায়
$ sudo dnf install tmux
$ sudo dnf install epel-release && sudo dnf install screen
```

সংস্করণ দেখো:

```
$ tmux -V
tmux 3.4
$ screen --version
Screen version 4.09.01 (GNU) 20-Aug-23
```

> 💡 **টিপস:** নতুন কেউ হলে **tmux** শেখো; এটা সক্রিয়ভাবে উন্নত হচ্ছে, কনফিগার করা সহজ, scripting শক্তিশালী। তবে পুরনো সার্ভারে প্রায়ই শুধু screen থাকে, তাই screen-এর বেসিকও জানা দরকার।

২৫.১২ GNU screen

screen ১৯৮৭ সাল থেকে আছে। প্রায় সব পুরনো Unix/Linux সার্ভারে পাওয়া যায়।

screen-এর গঠন

```
screen session "deploy"
├── window 0: bash       (লগ দেখছ)
├── window 1: vim        (কনফিগ এডিট)
└── window 2: htop       (মনিটর)
```

screen-এ **session** আর **window** আছে। একটা window = একটা পূর্ণ-পর্দার ভার্চুয়াল টার্মিনাল। পর্দাকে ভাগ করা অংশকে screen বলে **region**।

কমান্ড লাইন থেকে screen

কমান্ড	কাজ
`screen`	নতুন নামহীন session শুরু
`screen -S deploy`	`deploy` নামে নতুন session
`screen -ls` (বা `-list`)	সব session-এর তালিকা
`screen -r`	একমাত্র detached session-এ ফেরত যাও
`screen -r deploy`	নির্দিষ্ট session-এ attach
`screen -d -r deploy`	অন্য কোথাও attached থাকলে সেখান থেকে detach করে এখানে attach
`screen -D -R deploy`	জোর করে detach + attach, session না থাকলে নতুন তৈরি করে
`screen -x deploy`	ইতিমধ্যে attached session-এ **একসাথে** attach (দুজন একই পর্দা দেখবে)
`screen -dmS job1 ./backup.sh`	detached অবস্থায় session চালু করে কমান্ড চালায়
`screen -S deploy -X quit`	বাইরে থেকে session বন্ধ
`screen -wipe`	মৃত (dead) session-এর আবর্জনা পরিষ্কার

```
$ screen -S deploy
# ... কাজ করো, তারপর Ctrl+A d
[detached from 8120.deploy]
$ screen -ls
There is a screen on:
        8120.deploy     (Detached)
1 Socket in /run/screen/S-rahim.
$ screen -r deploy
```

screen-এর শর্টকাট (prefix = `Ctrl+A`)

screen-এর সব কিবোর্ড কমান্ড শুরু হয় **`Ctrl+A`** দিয়ে (একে বলে command key বা prefix)। প্রথমে `Ctrl+A` চেপে ছেড়ে দাও, তারপর পরের কী চাপো। নিচের টেবিলে `C-a d` মানে "`Ctrl+A`, তারপর `d`"।

শর্টকাট	কাজ
`C-a ?`	সব key binding-এর সাহায্য তালিকা
`C-a d`	**detach** (session চলতে থাকবে)
`C-a c`	নতুন window তৈরি
`C-a n` / `C-a Space`	পরের window
`C-a p` / `C-a Backspace`	আগের window
`C-a 0` … `C-a 9`	নির্দিষ্ট নম্বরের window-এ যাও
`C-a C-a`	শেষ ব্যবহৃত window-এ ফেরত (দুই window-এর মধ্যে toggle)
`C-a "`	window-এর তালিকা থেকে বেছে নাও
`C-a w`	window তালিকা status line-এ দেখাও
`C-a '`	নাম/নম্বর লিখে window-এ যাও
`C-a A`	বর্তমান window-এর নাম (title) বদলাও
`C-a k`	বর্তমান window kill (নিশ্চিত করতে বলবে)
`C-a \`	সব window বন্ধ করে screen থেকে বেরোও (নিশ্চিত করতে বলবে)
`C-a S`	পর্দা ওপর-নিচে ভাগ (horizontal split)
`C-a |`	পর্দা বাম-ডানে ভাগ (vertical split, screen 4.1+)
`C-a Tab`	পরের region-এ যাও
`C-a X`	বর্তমান region বন্ধ
`C-a Q`	বর্তমান ছাড়া সব region বন্ধ
`C-a [` বা `C-a Esc`	copy/scrollback mode (vi-এর মতো চলাচল, `Space` দিয়ে শুরু-শেষ চিহ্নিত করে কপি)
`C-a ]`	কপি করা লেখা paste
`C-a H`	বর্তমান window-এর log ফাইল (`screenlog.N`) চালু/বন্ধ
`C-a M`	এই window-এ activity হলে জানাও (monitor)

শর্টকাট	কাজ
`C-a _`	এই window চুপ থাকলে (silence) জানাও
`C-a x`	session lock (পাসওয়ার্ড দিয়ে খুলতে হবে)
`C-a :`	screen-এর কমান্ড prompt (যেমন `:resize 10`)
`C-a a`	প্রোগ্রামের কাছে আসল `Ctrl+A` পাঠাও (bash-এ লাইনের শুরুতে যেতে)
`C-a t`	সময় ও লোড দেখাও
`C-a i`	বর্তমান window-এর তথ্য

> 💡 **টিপস:** split করার পর নতুন region-টা খালি থাকে। `C-a Tab` দিয়ে সেখানে গিয়ে `C-a c` দিয়ে নতুন window তৈরি করতে হয়, অথবা `C-a 1` দিয়ে কোনো বিদ্যমান window দেখাতে হয়।

> ⚠️ **সতর্কতা:** screen-এ split layout detach করলে হারিয়ে যায় (window-গুলো থাকে, ভাগ থাকে না)। tmux-এ pane-গুলো detach-এর পরেও অক্ষত থাকে।

`~/.screenrc` উদাহরণ

```
# স্বাগত বার্তা বন্ধ
startup_message off
# scrollback বড় করো
defscrollback 10000
# নিচে সবসময় window তালিকা দেখাও
hardstatus alwayslastline
hardstatus string '%{= kG}[%H] %{= kw}%-w%{= kW}[%n %t]%{-}%+w %=%{= kY}%c'
# visual bell
vbell on
```

২৫.১৩ tmux: পরিচিতি ও Architecture

tmux (terminal multiplexer) ২০০৭ সালে এসেছে, OpenBSD-তে ডিফল্ট, আর আধুনিক লিনাক্স ব্যবহারকারী ও DevOps ইঞ্জিনিয়ারদের প্রিয় টুল।

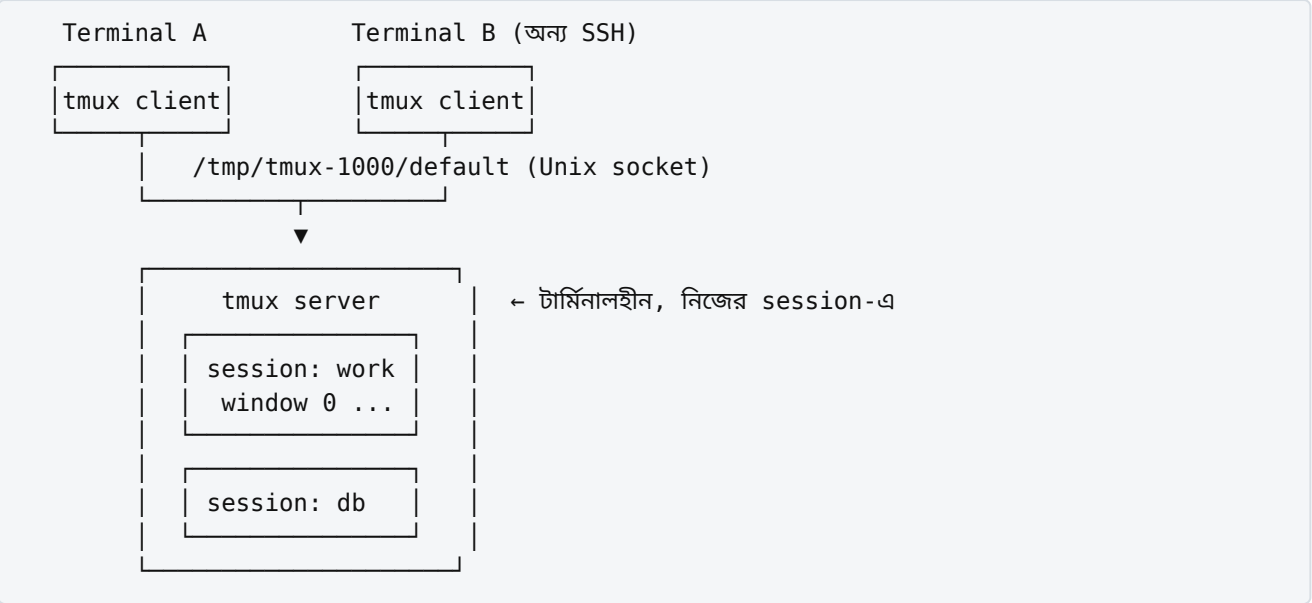
Client-Server মডেল

tmux চালু করলে আসলে দুটো জিনিস তৈরি হয়:

1. **tmux server**: একটা background process যে সব session, window, pane আর তাদের ভেতরের প্রোগ্রাম ধরে রাখে। প্রথম `tmux` কমান্ডেই এটা স্বয়ংক্রিয়ভাবে চালু হয়, আর শেষ session বন্ধ হলে নিজে থেকেই বন্ধ হয়।

2. **tmux client**: তোমার টার্মিনালে যা দেখছ। এটা শুধু server-এর সাথে কথা বলে আর পর্দা আঁকে।

দুজনে কথা বলে একটা **Unix domain socket** দিয়ে, যা থাকে `/tmp/tmux-UID/default`-এ (যেমন `/tmp/tmux-1000/default`)।



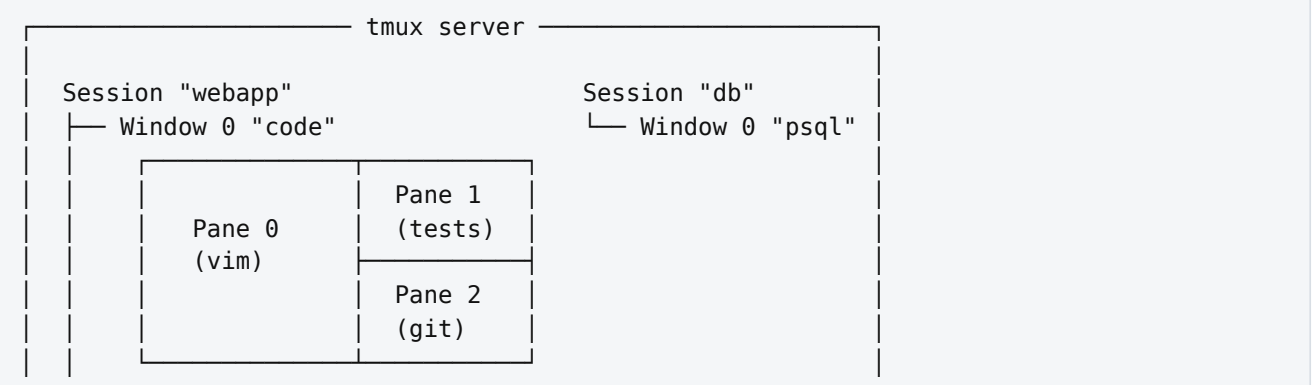
এজন্যই detach করলে কিছু হারায় না: শুধু client বেরিয়ে যায়, server আর তার ভেতরের সব কিছু চলতে থাকে।

```
$ ps -o pid,sid,tty,cmd -C tmux
    PID     SID TT       CMD
   9001    9001 ?        tmux new -s work       ← server (TT = ?)
   8990    4000 pts/0    tmux new -s work       ← client
```

Session → Window → Pane

tmux-এর তিন স্তরের ধারণা:

স্তর	কী	উপমা
Session	একটা প্রজেক্ট/কাজের পুরো সংগ্রহ; attach/detach হয় এই স্তরে	একটা ব্রাউজার উইন্ডো
Window	session-এর ভেতরে পূর্ণ-পর্দার একটা পাতা; নিচের status bar-এ তালিকা	ব্রাউজারের একটা ট্যাব
Pane	window-কে ভাগ করা আয়তাকার অংশ; প্রতিটায় আলাদা shell	ট্যাবের ভেতরে split view



```
│   ├── Window 1 "server"  (npm run dev)                    │
│   └── Window 2 "logs"    (tail -f)                        │
│                                                           │
└───────────────────────────────────────────────────────────┘

 status bar:  [webapp] 0:code* 1:server- 2:logs        "host" 14:32 12-Nov
```

status bar-এ `*` = বর্তমান window, `-` = আগের window, `Z` = zoom করা pane, `#` = activity হয়েছে।

২৫.১৪ tmux: কমান্ড লাইন থেকে ব্যবহার

কমান্ড	সংক্ষিপ্ত রূপ	কাজ
`tmux`		নতুন নামহীন session (নাম হবে 0, 1, ...)
`tmux new-session -s work`	`tmux new -s work`	`work` নামে session
`tmux new -s work -n editor`		প্রথম window-এর নাম `editor`
`tmux new -d -s job 'cmd'`		detached অবস্থায় session চালু করে কমান্ড চালাও
`tmux new -A -s work`		`work` থাকলে attach, না থাকলে তৈরি
`tmux list-sessions`	`tmux ls`	সব session-এর তালিকা
`tmux attach-session -t work`	`tmux a -t work`	attach
`tmux attach`	`tmux a`	সবচেয়ে সাম্প্রতিক session-এ attach
`tmux attach -d -t work`		অন্য client-দের detach করে attach
`tmux detach`		(tmux-এর ভেতর থেকে) detach
`tmux rename-session -t 0 work`		session-এর নাম বদল
`tmux kill-session -t work`		একটা session বন্ধ
`tmux kill-session -a`		বর্তমানটা ছাড়া সব session বন্ধ
`tmux kill-server`		সব session সহ server বন্ধ
`tmux new-window -n logs`	`tmux neww`	নতুন window
`tmux split-window -h`	`tmux splitw -h`	বাম-ডানে ভাগ
`tmux split-window -v`		ওপর-নিচে ভাগ
`tmux send-keys -t work:0 'ls' Enter`	`tmux send`	কোনো pane-এ কী-স্ট্রোক পাঠাও
`tmux list-keys`	`tmux lsk`	সব key binding
`tmux show-options -g`	`tmux show -g`	global option দেখাও
`tmux source-file ~/.tmux.conf`	`tmux source`	কনফিগ পুনরায় লোড

কমান্ড	সংক্ষিপ্ত রূপ	কাজ
`tmux capture-pane -p -t work`		pane-এর লেখা stdout-এ বের করো
`tmux has-session -t work`		session আছে কিনা (exit status 0/1)

Target নোটেশন (`-t`)

```
session:window.pane
```

যেমন `work:1.2` = session `work`, window 1, pane 2। `work:logs` = নাম দিয়ে window। শুধু `work` = session।

উদাহরণ:

```
$ tmux new -s work
# ... কাজ, তারপর Ctrl+B d
[detached (from session work)]
$ tmux ls
work: 3 windows (created Thu Nov 12 10:15:02 2026)
$ tmux a -t work
```

> 💡 **টিপস:** `tmux` কমান্ডের সব subcommand অস্পষ্ট না হলে সংক্ষেপে লেখা যায়: `tmux a`, `tmux ls`, `tmux new`, `tmux kill-ses -t x`।

২৫.১৫ tmux শর্টকাট: `Ctrl+B` prefix

tmux-এ সব শর্টকাটের আগে **prefix** চাপতে হয়: ডিফল্টভাবে **`Ctrl+B`**। পদ্ধতি:

1. `Ctrl` ধরে `B` চাপো
2. **দুটোই ছেড়ে দাও**
3. তারপর কমান্ড কী চাপো (যেমন `c`)

নিচে `C-b c` মানে ঠিক এটাই। `M-` মানে Alt (Meta) কী। `C-` মানে Ctrl।

> ⚠️ **সতর্কতা:** নতুনদের সবচেয়ে কমন ভুল হলো `Ctrl+B+c` একসাথে চেপে ধরা। `Ctrl+B` ছেড়ে দিয়ে **তারপর** `c` চাপো। আর `%` বা `"` টাইপ করতে Shift লাগে, সেটা স্বাভাবিকভাবে চাপবে।

Session-সংক্রান্ত

শর্টকাট	কাজ
`C-b d`	বর্তমান client **detach**
`C-b D`	কোন client detach করবে তা বেছে নাও
`C-b s`	session-এর গাছ (tree) থেকে বেছে নাও
`C-b $`	বর্তমান session-এর নাম বদলাও
`C-b (`	আগের session-এ যাও
`C-b )`	পরের session-এ যাও
`C-b L`	শেষ ব্যবহৃত session-এ যাও
`C-b C-z`	tmux client-কে suspend করো

Window-সংক্রান্ত

শর্টকাট	কাজ
`C-b c`	নতুন window
`C-b ,`	বর্তমান window-এর নাম বদলাও
`C-b &`	বর্তমান window kill (নিশ্চিত করতে বলবে)
`C-b n`	পরের window
`C-b p`	আগের window
`C-b l`	শেষ ব্যবহৃত window (toggle)
`C-b 0` … `C-b 9`	ওই নম্বরের window
`C-b '`	নম্বর লিখে window-এ যাও (১০-এর বেশি window হলে)
`C-b w`	সব session ও window-এর গাছ, preview সহ, বেছে নাও
`C-b f`	লেখা দিয়ে window খোঁজো
`C-b .`	window-কে অন্য নম্বরে সরাও
`C-b i`	বর্তমান window-এর তথ্য দেখাও

Pane-সংক্রান্ত

শর্টকাট	কাজ
`C-b %`	বাম-ডানে ভাগ (নতুন pane ডানে)
`C-b "`	ওপর-নিচে ভাগ (নতুন pane নিচে)
`C-b ←` `↑` `→` `↓`	তীর চিহ্নের দিকের pane-এ যাও

শর্টকাট	কাজ
`C-b o`	পরের pane-এ যাও
`C-b ;`	শেষ ব্যবহৃত pane-এ যাও
`C-b q`	pane নম্বর দেখাও (দেখানোর সময় নম্বর চাপলে সেখানে যাবে)
`C-b x`	বর্তমান pane kill (নিশ্চিত করতে বলবে)
`C-b z`	বর্তমান pane **zoom** করো (পূর্ণ পর্দা) / আবার চাপলে ফেরত
`C-b !`	বর্তমান pane-কে আলাদা নতুন window বানাও
`C-b {`	বর্তমান pane-কে আগের pane-এর সাথে অদলবদল
`C-b }`	বর্তমান pane-কে পরের pane-এর সাথে অদলবদল
`C-b C-o`	সব pane সামনের দিকে ঘোরাও (rotate)
`C-b M-o`	সব pane উল্টো দিকে ঘোরাও
`C-b Space`	পরের preset layout-এ যাও
`C-b M-1`	layout: even-horizontal (সব pane পাশাপাশি সমান)
`C-b M-2`	layout: even-vertical (সব pane ওপর-নিচে সমান)
`C-b M-3`	layout: main-horizontal (ওপরে একটা বড়, নিচে বাকিরা)
`C-b M-4`	layout: main-vertical (বামে একটা বড়, ডানে বাকিরা)
`C-b M-5`	layout: tiled (গ্রিড)
`C-b E`	pane-গুলো সমানভাবে ছড়িয়ে দাও (tmux 3.1+)
`C-b m`	বর্তমান pane চিহ্নিত (mark) করো
`C-b M`	চিহ্ন মুছে দাও

Pane Resize

শর্টকাট	কাজ
`C-b C-←/↑/→/↓`	ওই দিকে ১ ঘর করে আকার বদলাও (prefix-এর পর Ctrl ধরে তীর; বারবার চাপা যায়)
`C-b M-←/↑/→/↓`	ওই দিকে ৫ ঘর করে আকার বদলাও
`C-b :` তারপর `resize-pane -L 10`	বামে ১০ ঘর (`-R`, `-U`, `-D` অন্য দিক)
`C-b :` তারপর `resize-pane -Z`	zoom toggle (`C-b z`-এর সমান)

> 💡 **টিপস:** resize-এর কীগুলো **repeatable**: একবার prefix চাপার পর `repeat-time`-এর মধ্যে (ডিফল্ট 500 মিলিসেকেন্ড) বারবার তীর চাপতে পারো, প্রতিবার prefix লাগবে না।

> ⚠️ **সতর্কতা:** অনেক টার্মিনাল বা ডেস্কটপ (যেমন GNOME-এ `Ctrl+Alt+←`) কিছু Ctrl/Alt-তীর কম্বিনেশন নিজেরাই খেয়ে ফেলে। কাজ না করলে `resize-pane` কমান্ড বা mouse ব্যবহার করো।

Copy mode, buffer ও অন্যান্য

শর্টকাট	কাজ
`C-b [`	**copy mode**-এ ঢোকো (scroll করা, খোঁজা, কপি)
`C-b PageUp`	copy mode-এ ঢুকে এক পাতা ওপরে scroll
`C-b ]`	সবচেয়ে সাম্প্রতিক buffer paste
`C-b #`	সব paste buffer-এর তালিকা
`C-b =`	তালিকা থেকে buffer বেছে paste
`C-b -`	সবচেয়ে সাম্প্রতিক buffer মুছে দাও
`C-b :`	tmux কমান্ড prompt
`C-b ?`	**সব key binding-এর তালিকা** (q দিয়ে বেরোও)
`C-b t`	বড় ঘড়ি দেখাও (যেকোনো কী চাপলে বন্ধ)
`C-b r`	client-এর পর্দা নতুন করে আঁকো (redraw)
`C-b ~`	tmux-এর আগের বার্তাগুলো দেখাও
`C-b C-b`	প্রোগ্রামের কাছে আসল `Ctrl+B` পাঠাও

> 📝 **মনে রাখো:** কোনো শর্টকাট ভুলে গেলে শুধু `C-b ?` চাপো। এটাই tmux-এর বিল্ট-ইন চিটশিট।

Mouse

ডিফল্টভাবে mouse বন্ধ। `C-b :` চেপে লেখো `set -g mouse on`। তাহলে:

- pane-এ ক্লিক করে বেছে নেওয়া
- pane-এর সীমানা টেনে resize
- status bar-এ window-এ ক্লিক
- চাকা ঘুরিয়ে scroll (স্বয়ংক্রিয়ভাবে copy mode)

> 💡 **টিপস:** mouse চালু থাকলে টার্মিনালের সাধারণ মাউস-সিলেকশন tmux নিয়ে নেয়। টার্মিনালের নিজস্ব সিলেকশন চাইলে **Shift** চেপে ধরে টানো (বেশিরভাগ টার্মিনালে কাজ করে)।

২৫.১৬ tmux Copy Mode বিস্তারিত

pane-এর বাইরে চলে যাওয়া পুরনো আউটপুট দেখতে, খুঁজতে আর কপি করতে copy mode লাগে। ডিফল্টভাবে tmux প্রতিটা pane-এর ২০০০ লাইন মনে রাখে (`history-limit`)।

copy mode-এর কী-গুলো দুই ধরনের: **emacs** (ডিফল্ট) আর **vi**। `EDITOR` বা `VISUAL` environment variable-এ `vi` থাকলে tmux নিজে থেকেই vi ধরনে চলে। নিশ্চিত করতে `.tmux.conf`-এ লেখো `setw -g mode-keys vi`।

কাজ	vi mode	emacs mode
copy mode-এ ঢোকা	`C-b [`	`C-b [`
বেরোনো	`q`	`q` বা `Esc`
চলাচল	`h` `j` `k` `l` / তীর	তীর
এক পাতা ওপরে/নিচে	`C-b` / `C-f`	`PageUp` / `PageDown`
অর্ধ পাতা	`C-u` / `C-d`	`M-Up` / `M-Down`
ইতিহাসের একদম ওপরে/নিচে	`g` / `G`	`M-<` / `M->`
লাইনের শুরু/শেষ	`0` / `$`	`C-a` / `C-e`
পরের/আগের শব্দ	`w` / `b`	`M-f` / `M-b`
সামনে খোঁজা	`/`	`C-s`
পেছনে খোঁজা	`?`	`C-r`
পরের/আগের মিল	`n` / `N`	`n` / `N`
সিলেকশন শুরু	`Space`	`C-Space`
আয়তাকার (block) সিলেকশন	`v` (toggle)	`R`
কপি করে বেরোনো	`Enter`	`M-w`
সিলেকশন বাতিল	`Esc`	`C-g`

উদাহরণ (vi mode): একটা error লাইন কপি করে অন্য pane-এ paste

```
C-b [          → copy mode
?ERROR Enter   → পেছন দিকে "ERROR" খোঁজো
0              → লাইনের শুরু
Space          → সিলেকশন শুরু
$              → লাইনের শেষ পর্যন্ত
Enter          → কপি, copy mode থেকে বেরোও
C-b ;          → আগের pane-এ যাও
C-b ]          → paste
```

অনেকে vim-এর মতো `v` দিয়ে সিলেকশন আর `y` দিয়ে কপি পছন্দ করে। `.tmux.conf`-এ:

```
setw -g mode-keys vi
bind -T copy-mode-vi v send-keys -X begin-selection
bind -T copy-mode-vi y send-keys -X copy-selection-and-cancel
bind -T copy-mode-vi C-v send-keys -X rectangle-toggle
```

> 💡 **টিপস:** tmux-এর buffer আর সিস্টেমের clipboard আলাদা। tmux 3.2+ এ `set -g set-clipboard on` থাকলে এবং টার্মিনাল OSC 52 সমর্থন করলে কপি সিস্টেম clipboard-এ যায়। নইলে X11-এ `copy-pipe-and-cancel 'xclip -selection clipboard -in'` বা Wayland-এ `wl-copy` ব্যবহার করো।

pane-এর পুরো ইতিহাস ফাইলে সেভ করতে:

```
$ tmux capture-pane -p -S - -t work:0.0 > pane-history.txt
```

(`-S -` মানে ইতিহাসের একদম শুরু থেকে।)

২৫.১৭ `.tmux.conf` : কাস্টমাইজেশন

tmux চালু হওয়ার সময় কনফিগ পড়ে `~/.tmux.conf` থেকে (tmux 3.1+ এ `~/.config/tmux/tmux.conf`-ও কাজ করে)। সব ইউজারের জন্য `/etc/tmux.conf`।

> 📝 **মনে রাখো:** কনফিগ শুধু **server** চালু হওয়ার সময় পড়া হয়। বদলানোর পর হয় `tmux source-file ~/.tmux.conf` চালাও, নয়তো সব session বন্ধ করে (`tmux kill-server`) আবার শুরু করো। নতুন `tmux` উইন্ডো খুললেই কনফিগ পড়া হয় না যদি server আগে থেকে চলছে।

ধাপে ধাপে একটা ভালো কনফিগ

```
# ~/.tmux.conf

# ---------- prefix: Ctrl+B থেকে Ctrl+A (screen-এর মতো, হাতের কাছে) ----------
unbind C-b
set -g prefix C-a
bind C-a send-prefix          # C-a C-a চাপলে প্রোগ্রাম আসল Ctrl+A পাবে

# ---------- সাধারণ ----------
set -g mouse on                       # mouse চালু
set -g history-limit 50000            # বড় scrollback
set -g base-index 1                   # window নম্বর 1 থেকে (কিবোর্ডে 0 দূরে)
setw -g pane-base-index 1             # pane নম্বরও 1 থেকে
set -g renumber-windows on            # window বন্ধ হলে নম্বর আবার সাজাও
set -sg escape-time 10                # Esc-এর দেরি কমাও (vim ব্যবহারকারীদের জন্য জরুরি)
set -g default-terminal "tmux-256color"
```

```
set -ga terminal-overrides ",*256col*:Tc"   # true color
set -g focus-events on
setw -g mode-keys vi                  # copy mode-এ vi কী

# ---------- সহজ split: | আর -, বর্তমান ডিরেক্টরিতেই নতুন pane ----------
unbind '"'
unbind %
bind | split-window -h -c "#{pane_current_path}"
bind - split-window -v -c "#{pane_current_path}"
bind c new-window -c "#{pane_current_path}"

# ---------- vim-এর মতো pane চলাচল ----------
bind h select-pane -L
bind j select-pane -D
bind k select-pane -U
bind l select-pane -R

# ---------- resize (-r = repeatable) ----------
bind -r H resize-pane -L 5
bind -r J resize-pane -D 5
bind -r K resize-pane -U 5
bind -r L resize-pane -R 5

# ---------- কনফিগ reload ----------
bind r source-file ~/.tmux.conf \; display-message "কনফিগ reload হয়েছে"

# ---------- prefix ছাড়াই Alt+তীর দিয়ে pane বদল ----------
bind -n M-Left  select-pane -L
bind -n M-Right select-pane -R
bind -n M-Up    select-pane -U
bind -n M-Down  select-pane -D

# ---------- copy mode vim-এর মতো ----------
bind -T copy-mode-vi v send-keys -X begin-selection
bind -T copy-mode-vi y send-keys -X copy-selection-and-cancel

# ---------- status bar ----------
set -g status-position bottom
set -g status-interval 5
set -g status-style "bg=colour235,fg=colour250"
set -g status-left "#[fg=green,bold][#S] "
set -g status-left-length 30
set -g status-right "#[fg=yellow]#H #[fg=cyan]%Y-%m-%d %H:%M"
setw -g window-status-current-style "fg=black,bg=green,bold"
setw -g monitor-activity on
```

বুঝে নাও কয়েকটা জিনিস:

- `set -g` = global session option; `setw -g` (`set-window-option`) = window option; `set -s` = server option। tmux-এর নতুন সংস্করণে `set` নিজেই option-এর ধরন বুঝে নেয়, তবু এভাবে লেখা প্রচলিত।
- `bind KEY CMD` = prefix-এর পরে KEY চাপলে CMD; `bind -n` = prefix **ছাড়াই**; `bind -r` = repeatable; `bind -T copy-mode-vi` = নির্দিষ্ট key table-এ।

- `#S` = session নাম, `#H` = hostname, `#I` = window index, `#W` = window নাম, `#{pane_current_path}` = pane-এর বর্তমান ডিরেক্টরি। এগুলোকে বলে **format**।
- `bind ... \; ...` = একসাথে দুটো কমান্ড।

> ⚠️ **সতর্কতা:** prefix `C-a` করলে bash-এ লাইনের শুরুতে যাওয়ার `Ctrl+A` (অধ্যায় ৬) আর সরাসরি কাজ করবে না; তখন `C-a C-a` চাপতে হবে (ওপরের `send-prefix` binding-এর জন্য)। অনেকে এ কারণে `C-b` রেখে দেয় বা `C-Space` ব্যবহার করে।

Plugin (ঐচ্ছিক): TPM

TPM (Tmux Plugin Manager) দিয়ে plugin বসানো যায়, যেমন `tmux-resurrect` (রিবুটের পরেও session-এর layout ফিরিয়ে আনা)।

```
$ git clone https://github.com/tmux-plugins/tpm ~/.tmux/plugins/tpm
```

`.tmux.conf`-এর **একদম শেষে**:

```
set -g @plugin 'tmux-plugins/tpm'
set -g @plugin 'tmux-plugins/tmux-resurrect'
run '~/.tmux/plugins/tpm/tpm'
```

তারপর tmux-এর ভেতরে `C-b I` (বড় হাতের I) চাপলে plugin ইনস্টল হবে। tmux-resurrect-এ `C-b C-s` = সেভ, `C-b C-r` = রিস্টোর।

> 📝 **মনে রাখো:** রিবুট হলে tmux server মারা যায়, তাই ভেতরের সব প্রোগ্রামও। tmux রিবুট থেকে বাঁচায় না; resurrect শুধু layout আর কিছু প্রোগ্রাম আবার চালু করে।

২৫.১৮ tmux Scripting: এক কমান্ডে পুরো কাজের পরিবেশ

প্রতিদিন একই layout হাতে বানানোর বদলে একটা স্ক্রিপ্ট লেখো:

```
#!/usr/bin/env bash
# dev.sh — প্রজেক্টের tmux পরিবেশ তৈরি
SESSION=webapp
DIR=~/projects/webapp

# আগে থেকে থাকলে শুধু attach
if tmux has-session -t "$SESSION" 2>/dev/null; then
    exec tmux attach -t "$SESSION"
fi
```

```
tmux new-session  -d -s "$SESSION" -n code -c "$DIR"
tmux send-keys    -t "$SESSION:code" 'vim .' Enter
tmux split-window -h -t "$SESSION:code" -c "$DIR" -l 40%
tmux send-keys    -t "$SESSION:code.1" 'git status' Enter

tmux new-window   -t "$SESSION" -n server -c "$DIR"
tmux send-keys    -t "$SESSION:server" 'npm run dev' Enter

tmux new-window   -t "$SESSION" -n logs
tmux send-keys    -t "$SESSION:logs" 'journalctl -f' Enter

tmux select-window -t "$SESSION:code"
tmux select-pane   -t "$SESSION:code.0"
exec tmux attach -t "$SESSION"
```

> 📝 **মনে রাখো:** pane নম্বর `pane-base-index`-এর ওপর নির্ভর করে। ওপরের স্ক্রিপ্ট ডিফল্ট (০ থেকে শুরু) ধরে লেখা; `pane-base-index 1` থাকলে `.0` → `.1`, `.1` → `.2` করো। `-l 40%` শতাংশ হিসেবে আকার দেওয়া tmux 3.1+ এ কাজ করে।

অনেক সার্ভারে একসাথে টাইপ: `synchronize-panes`

চারটা pane-এ চারটা সার্ভারে SSH করে একই কমান্ড একসাথে চালাতে:

```
C-b :  setw synchronize-panes on
```

এখন যা টাইপ করবে সব pane-এ যাবে। বন্ধ করতে `setw synchronize-panes off`।

> ⚠️ **সতর্কতা:** synchronize চালু রেখে ভুলে `sudo reboot` বা `rm` টাইপ করলে **সব** সার্ভারে চলবে। কাজ শেষ হলেই বন্ধ করো।

Pair programming: একই session দুজন

একই ইউজার দুই টার্মিনাল থেকে `tmux a -t work` দিলে দুজনেই একই পর্দা দেখবে ও টাইপ করতে পারবে। আলাদা window দেখতে চাইলে **grouped session**:

```
$ tmux new -t work -s work-friend     # work-এর window শেয়ার করে, কিন্তু নিজের বর্তমান window
```

২৫.১৯ screen বনাম tmux

বিষয়	screen	tmux
ডিফল্ট prefix	`Ctrl+A`	`Ctrl+B`

বিষয়	screen	tmux
detach	`C-a d`	`C-b d`
নতুন window	`C-a c`	`C-b c`
পরের/আগের window	`C-a n` / `C-a p`	`C-b n` / `C-b p`
window তালিকা	`C-a "`	`C-b w`
window নাম বদল	`C-a A`	`C-b ,`
ওপর-নিচে split	`C-a S`	`C-b "`
বাম-ডানে split	`C-a |`	`C-b %`
region/pane বদল	`C-a Tab`	`C-b o` / তীর
copy mode	`C-a [`	`C-b [`
paste	`C-a ]`	`C-b ]`
সাহায্য	`C-a ?`	`C-b ?`
session তালিকা	`screen -ls`	`tmux ls`
attach	`screen -r name`	`tmux a -t name`
detach-এর পর split টেকে?	না	হ্যাঁ
scripting	সীমিত (`-X`)	খুব শক্তিশালী
serial console (`screen /dev/ttyUSB0 115200`)	হ্যাঁ	না
RHEL 9 base repo-তে	না (EPEL)	হ্যাঁ

> 💡 **টিপস:** screen-এর একটা অনন্য কাজ আছে: serial port-এ সংযোগ। রাউটার, সুইচ বা Raspberry Pi-এর serial console-এ ঢুকতে `screen /dev/ttyUSB0 115200` এখনো জনপ্রিয়। বেরোতে `C-a k`।

২৫.২০ বাস্তব পরিস্থিতি: কোন টুল কখন

পরিস্থিতি	সমাধান
vim থেকে এক মুহূর্তের জন্য shell-এ	`Ctrl+Z` … `fg`
`&` দিতে ভুলে গেছ	`Ctrl+Z` , `bg`
চালু কাজ রেখে লগআউট করতে হবে	`Ctrl+Z` , `bg` , `disown -h`
SSH-এ ২ ঘণ্টার database migration	`tmux new -s migrate` → চালাও → `C-b d`
সংযোগ কেটে যেতে পারে এমন দুর্বল Wi-Fi	সবসময় tmux-এর ভেতরে কাজ করো
স্ক্রিপ্ট থেকে একটা কাজ ছেড়ে দিতে, আউটপুট লগে	`nohup cmd > log 2>&1 &`
টার্মিনাল থেকে GUI অ্যাপ খুলে টার্মিনাল বন্ধ	`setsid -f app`

পরিস্থিতি	সমাধান
সবসময় চলবে, রিবুটেও ফিরে আসবে	systemd service (অধ্যায় ২৬)
নির্দিষ্ট সময়ে চালাতে হবে	cron/systemd timer (অধ্যায় ২৯)

> 💡 **টিপস:** SSH লগইনের সাথে সাথে tmux-এ ঢুকতে অনেকে `~/.bashrc`-এর শেষে লেখে:
>
> ```
> if [[ -n "$SSH_CONNECTION" && -z "$TMUX" && $- == *i* ]]; then
>     tmux new -A -s main
> fi
> ```
>
> `$TMUX` variable tmux-এর ভেতরে থাকলে সেট থাকে, তাই tmux-এর ভেতরে tmux (nesting) হবে না। `$- == *i*` নিশ্চিত করে শুধু interactive shell-এ চলবে (scp/rsync ভাঙবে না)।

> ⚠️ **সতর্কতা:** tmux-এর ভেতরে আবার tmux চালালে tmux সতর্ক করে: `sessions should be nested with care, unset $TMUX to force`। nested অবস্থায় ভেতরের tmux-এ prefix পাঠাতে `C-b C-b` (দুবার) চাপতে হয়।

হাতে-কলমে অনুশীলন

ল্যাব ১: job control

```
$ sleep 1000 &
$ sleep 2000 &
$ top                       # চালু হলে Ctrl+Z
$ jobs -l                   # তিনটা job, PID সহ
$ fg %top                   # top ফেরত আনো, তারপর আবার Ctrl+Z
$ bg %3                     # top-কে background-এ (দেখবে আবার Stopped হয়ে যাবে — কেন?)
$ kill %1 %2                # দুটো sleep মারো
$ jobs
$ fg                        # top-এ গিয়ে q দিয়ে বেরোও
```

প্রশ্ন: `bg` দেওয়ার পরেও top কেন stopped হলো? (উত্তর: top টার্মিনাল নিয়ন্ত্রণ করতে চায়/কিবোর্ড পড়তে চায়, background থেকে তা করলে kernel SIGTTIN/SIGTTOU দিয়ে থামিয়ে দেয়।)

ল্যাব ২: SIGHUP নিজের চোখে দেখা

1. একটা নতুন টার্মিনাল উইন্ডো খোলো, চালাও:

   ```
   $ sleep 600 &
   $ nohup sleep 700 > /dev/null 2>&1 &
   ```

```
$ sleep 800 & disown
$ setsid -f sleep 900
```

2. উইন্ডোটা **ক্রস চিহ্নে ক্লিক করে** বন্ধ করো।
3. অন্য টার্মিনালে দেখো:

```
$ pgrep -a sleep
2345 sleep 700
2350 sleep 800
2360 sleep 900
```

`sleep 600` মারা গেছে (SIGHUP), বাকি তিনটা বেঁচে আছে।

4. পরিষ্কার করো: `pkill -x sleep`

ল্যাব ৩: screen বেসিক

```
$ screen -S lab
$ top                          # window 0-এ top
C-a c                          # নতুন window
$ ping -c 100 1.1.1.1          # window 1-এ ping
C-a "                          # তালিকা থেকে window 0 বেছে নাও
C-a A                          # নাম দাও "monitor"
C-a d                          # detach
$ screen -ls
$ screen -r lab                # ফেরত, সব চলছে
C-a \                          # সব বন্ধ (y দিয়ে নিশ্চিত)
```

ল্যাব ৪: tmux সম্পূর্ণ চক্র

```
$ tmux new -s lab
C-b %                          # বাম-ডান ভাগ
C-b "                          # ডান অংশ ওপর-নিচে ভাগ
C-b ←                          # বামের pane-এ যাও
$ htop                         # (না থাকলে top)
C-b →                          # ডানের pane
$ watch -n1 date
C-b z                          # zoom, আবার C-b z দিয়ে ফেরত
C-b M-5                        # tiled layout
C-b c                          # নতুন window
C-b ,                          # নাম দাও "logs"
$ sudo journalctl -f
C-b 0                          # প্রথম window-এ ফেরত
C-b d                          # detach
$ tmux ls
lab: 2 windows (created ...)
$ tmux a -t lab                # সব ঠিক আছে, layout সহ
```

ল্যাব ৫: SSH কেটে গেলেও কাজ বাঁচানো

1. কোনো সার্ভার বা VM-এ SSH করো (না থাকলে `ssh localhost`, অধ্যায় ৩৮)।

2. `tmux new -s longjob`
3. চালাও: `for i in $(seq 1 300); do echo "step $i"; sleep 1; done`
4. সংযোগ জোর করে কাটো: SSH-এর escape sequence `Enter` তারপর `~.` চাপো।
5. আবার SSH করো, `tmux a -t longjob`। দেখবে লুপ চলছেই।

ল্যাব ৬: copy mode ও `.tmux.conf`

1. ওপরের ২৫.১৭-এর কনফিগ `~/.tmux.conf`-এ লেখো (অধ্যায় ১৩ বা ১৪ দিয়ে)।
2. `tmux source-file ~/.tmux.conf` (বা নতুন prefix-এর পর `r`)।
3. `seq 1 5000` চালাও, তারপর copy mode-এ ঢুকে `/2500` দিয়ে খোঁজো, `v` দিয়ে সিলেক্ট, `y` দিয়ে কপি, prompt-এ prefix + `]` দিয়ে paste করো।

ল্যাব ৭: `dev.sh` স্ক্রিপ্ট

২৫.১৮-এর স্ক্রিপ্টটা নিজের একটা ফোল্ডারের জন্য বদলে লেখো, `chmod +x dev.sh` দিয়ে চালাও। দুবার চালিয়ে দেখো দ্বিতীয়বার শুধু attach হয়।

সারসংক্ষেপ

Job control

কমান্ড/কী	কাজ
`cmd &`	background-এ চালাও
`$!`	শেষ background PID
`wait [id]`	শেষ হওয়া পর্যন্ত অপেক্ষা
`Ctrl+Z`	foreground job থামাও (SIGTSTP)
`Ctrl+C`	foreground job মারো (SIGINT)
`jobs -l`	job তালিকা PID সহ
`fg %n`	foreground-এ আনো
`bg %n`	background-এ চালু করো
`kill %n`	job-কে সিগন্যাল
`%+` `%-` `%n` `%str` `%?str`	job specification
`disown [-h] %n`	shell-এর SIGHUP থেকে মুক্ত
`nohup cmd > log 2>&1 &`	SIGHUP ignore করে চালাও
`setsid -f cmd`	নতুন session, টার্মিনাল ছাড়া
`shopt huponexit`	exit-এ SIGHUP পাঠাবে কিনা

কমান্ড/কী	কাজ
`loginctl enable-linger`	লগআউটের পরেও ইউজার সার্ভিস

screen (`C-a`)

কী	কাজ	কী	কাজ
`d`	detach	`c`	নতুন window
`n` / `p`	পরের/আগের	`"`	তালিকা
`A`	নাম বদল	`k`	window kill
`S` / `|`	split	`Tab`	region বদল
`[` / `]`	copy/paste	`?`	সাহায্য

tmux (`C-b`)

কী	কাজ	কী	কাজ
`d`	detach	`s`	session বাছাই
`$`	session নাম	`c`	নতুন window
`,`	window নাম	`&`	window kill
`n` / `p` / `l`	পরের/আগের/শেষ window	`w`	window tree
`%`	বাম-ডান split	`"`	ওপর-নিচে split
তীর / `o` / `;`	pane বদল	`x`	pane kill
`z`	zoom	`!`	pane → window
`{` / `}`	pane swap	`Space` / `M-1..5`	layout
`C-`তীর / `M-`তীর	resize ১/৫	`[` / `]`	copy / paste
`:`	কমান্ড prompt	`?`	সব শর্টকাট

tmux কমান্ড	কাজ
`tmux new -s NAME`	নতুন session
`tmux ls`	তালিকা
`tmux a -t NAME`	attach
`tmux new -A -s NAME`	থাকলে attach, নইলে তৈরি
`tmux kill-session -t NAME`	session বন্ধ
`tmux kill-server`	সব বন্ধ
`tmux source-file ~/.tmux.conf`	কনফিগ reload

নিজেকে যাচাই করো

1. job আর process-এর মধ্যে পার্থক্য কী? `ls | sort | head`-এ কয়টা job আর কয়টা process?
2. `Ctrl+Z` আর `Ctrl+C`-এর মধ্যে কোন সিগন্যাল যায়, আর ফলাফলে কী পার্থক্য?
3. একটা কমান্ড foreground-এ চালু করে ফেলেছ, এখন তাকে background-এ পাঠিয়ে লগআউট করতে চাও। ধাপগুলো লেখো।
4. টার্মিনাল উইন্ডো বন্ধ করলে background job মারা যায়, কিন্তু `exit` লিখে বেরোলে অনেক সময় বাঁচে। কেন?
5. `nohup` আর `disown`-এর মূল পার্থক্য কী? কোনটা কখন ব্যবহার করবে?
6. `setsid` দিয়ে চালানো process-এর `ps`-এ `TT` কলামে `?` কেন দেখায়?
7. `KillUserProcesses=yes` সেট থাকলে লগআউটের পর tmux session-এর কী হবে? কীভাবে বাঁচাবে?
8. tmux-এ detach করলে প্রোগ্রাম কেন চলতে থাকে? architecture দিয়ে ব্যাখ্যা করো।
9. tmux-এ session, window আর pane-এর সম্পর্ক কী? একটা করে তৈরির শর্টকাট বা কমান্ড লেখো।
10. `.tmux.conf` বদলানোর পর নতুন টার্মিনালে `tmux` খুললেও পরিবর্তন দেখা যাচ্ছে না কেন? সমাধান কী?

▼ উত্তর দেখো

1. process হলো kernel-এর চলমান প্রোগ্রাম (PID সহ); job হলো shell-এর দৃষ্টিতে একবারে চালানো একটা কমান্ড লাইন, যা একটা process group। `ls | sort | head`-এ **১টা job**, **৩টা process**।
2. `Ctrl+Z` → **SIGTSTP**, process থেমে থাকে (stopped), পরে `fg` / `bg` দিয়ে চালানো যায়। `Ctrl+C` → **SIGINT**, সাধারণত process শেষ হয়ে যায়।
3. `Ctrl+Z` → `bg` → `disown -h %1` (বা `disown %1`) → `exit`। (আউটপুট টার্মিনালে গেলে সমস্যা হতে পারে, তাই ভবিষ্যতে শুরুতেই redirect করো বা tmux ব্যবহার করো।)
4. উইন্ডো বন্ধ হলে kernel session leader (bash)-কে SIGHUP পাঠায়, আর bash SIGHUP পেয়ে বেরোনোর আগে সব job-কে SIGHUP পাঠায়। নিজে `exit` করলে bash running job-দের SIGHUP পাঠায় না, যদি না `huponexit` চালু থাকে।
5. `nohup` আলাদা প্রোগ্রাম; কমান্ড **শুরুর আগে** দিতে হয়, process নিজেই SIGHUP ignore করে এবং আউটপুট `nohup.out`-এ যায়। `disown` bash builtin; কমান্ড **চালুর পরে** দেওয়া যায়, শুধু bash-এর SIGHUP পাঠানোর তালিকা থেকে বাদ দেয়।
6. `setsid` নতুন session তৈরি করে যার কোনো controlling terminal নেই, তাই `TT` = `?`।
7. লগআউটের সাথে সাথে ওই session-এর scope-এর সব process মারা যাবে, tmux server-ও। বাঁচাতে `loginctl enable-linger $USER` চালু করো এবং `systemd-run --user --scope tmux new -s work` দিয়ে tmux-কে আলাদা scope-এ চালাও, অথবা অ্যাডমিন `KillUserProcesses=no` সেট করবে।
8. tmux client-server মডেলে চলে। server টার্মিনাল থেকে বিচ্ছিন্ন (নিজের session, `TT` = `?`) এবং প্রোগ্রামগুলোকে নিজের তৈরি pseudo-terminal-এ চালায়। detach-এ শুধু client বেরিয়ে যায়; server আর ভেতরের প্রোগ্রাম চলতে থাকে। আবার attach করলে নতুন client socket (`/tmp/tmux-UID/default`) দিয়ে সংযুক্ত হয়।

9. একটা session-এ অনেক window, একটা window-এ অনেক pane। session: `tmux new -s name`; window: `C-b c`; pane: `C-b %` বা `C-b "`।
10. tmux কনফিগ শুধু server চালু হওয়ার সময় পড়ে; server আগে থেকেই চলছে বলে নতুন client নতুন কনফিগ পড়ে না। সমাধান: `tmux source-file ~/.tmux.conf` বা `C-b :` এ `source-file ~/.tmux.conf`, অথবা `tmux kill-server` করে আবার শুরু।

অধ্যায় ২৬: systemd ও সার্ভিস ম্যানেজমেন্ট

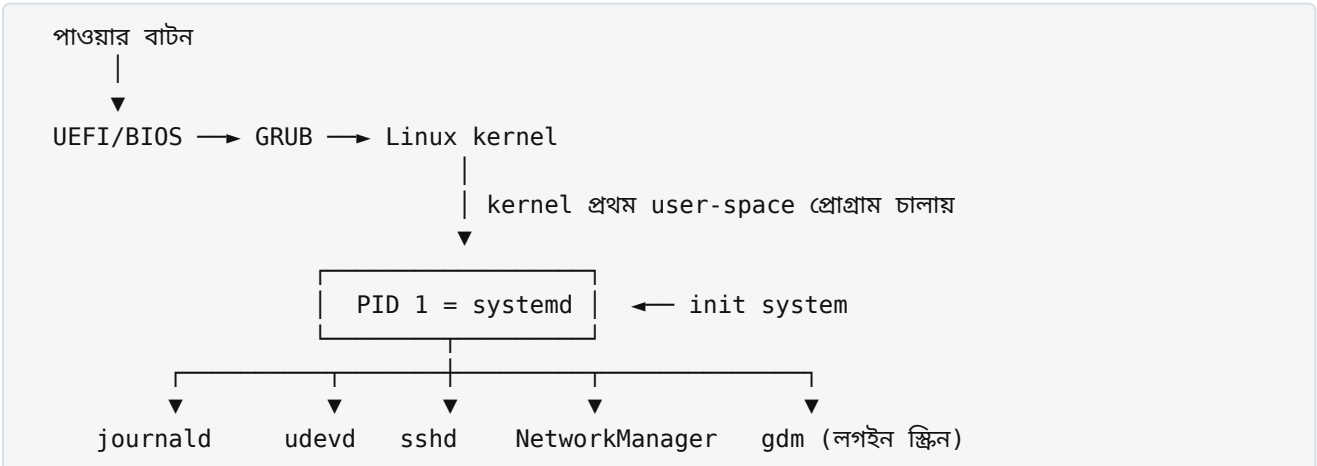
এই অধ্যায়ে যা শিখবে

- init system কী, আর কম্পিউটার চালু হওয়ার পর প্রথম process কেন এত গুরুত্বপূর্ণ
- পুরোনো SysV init থেকে লিনাক্স কেন systemd-তে সরে এল
- **unit** কী: service, socket, timer, target, mount সহ সব ধরনের unit
- `systemctl` দিয়ে সার্ভিস start, stop, restart, reload, status, enable, disable, mask করা
- target ও পুরোনো runlevel-এর মধ্যে সম্পর্ক
- unit ফাইল কোথায় থাকে (`/usr/lib/systemd/system` বনাম `/etc/systemd/system`) এবং ফাইলের গঠন
- নিজের একটি service unit লিখে চালানো
- `systemctl edit` দিয়ে override (drop-in) তৈরি করা
- dependency: `Wants`, `Requires`, `After`, `Before` ইত্যাদি
- `systemd-analyze` দিয়ে বুট সময় মাপা ও unit যাচাই করা
- `hostnamectl`, `timedatectl`, `localectl` দিয়ে সিস্টেম সেটিংস বদলানো
- user service (`systemctl --user`) ও নিরাপত্তা সেটিংস (sandboxing)

২৬.১ init system কী?

কম্পিউটার চালু করলে প্রথমে firmware (BIOS/UEFI), তারপর bootloader (GRUB), তারপর **kernel** লোড হয়। kernel হার্ডওয়্যার চিনে নেওয়ার পর একটা কাজই বাকি থাকে: user space-এ **প্রথম process** চালু করা। এই প্রথম process-এর PID (Process ID) সবসময় **১**। একেই বলে **init** (initialization, অর্থাৎ শুরু করার দায়িত্বে থাকা প্রোগ্রাম)।

বুটের পুরো ধাপগুলো বিস্তারিত দেখবে অধ্যায় ২৭-এ। এখানে শুধু PID 1-এর কাজ নিয়ে কথা বলব।



init-এর দায়িত্ব

1. **সার্ভিস চালু করা:** নেটওয়ার্ক, SSH, ওয়েব সার্ভার, লগিং, লগইন স্ক্রিন।
2. **সঠিক ক্রম রক্ষা করা:** নেটওয়ার্ক ওঠার আগে ওয়েব সার্ভার চালু হলে ঝামেলা হতে পারে।
3. **সার্ভিসের দেখাশোনা করা:** কোনো সার্ভিস crash করলে দরকার হলে আবার চালু করা।
4. **orphan process দত্তক নেওয়া:** কোনো process-এর parent মারা গেলে সেই "এতিম" process-টা PID 1-এর অধীনে চলে আসে। init তখন সেটার exit status সংগ্রহ (reap) করে, যাতে zombie process জমে না যায় (process নিয়ে বিস্তারিত অধ্যায় ২৪-এ)।
5. **শাটডাউন:** বন্ধ করার সময় সব সার্ভিস ঠিক ক্রমে থামানো, ফাইলসিস্টেম unmount করা।

> 💡 **উপমা:** init হলো একটা অফিসের ম্যানেজার। সকালে অফিস খুললে সে ঠিক করে কে আগে আসবে (দারোয়ান দরজা খুলবে, তারপর রিসেপশনিস্ট, তারপর বাকিরা), কেউ অসুস্থ হয়ে চলে গেলে বদলি পাঠায়, আর সন্ধ্যায় সবাইকে সঠিক ক্রমে ছুটি দিয়ে দরজায় তালা দেয়।

নিজের মেশিনে দেখো PID 1 কে:

```
$ ps -p 1 -o pid,comm
    PID COMMAND
      1 systemd

$ ls -l /sbin/init
lrwxrwxrwx 1 root root 20 Apr  8  2024 /sbin/init -> /lib/systemd/systemd
```

`/sbin/init` আসলে systemd-এর দিকে একটা symlink (symlink নিয়ে অধ্যায় ১১)।

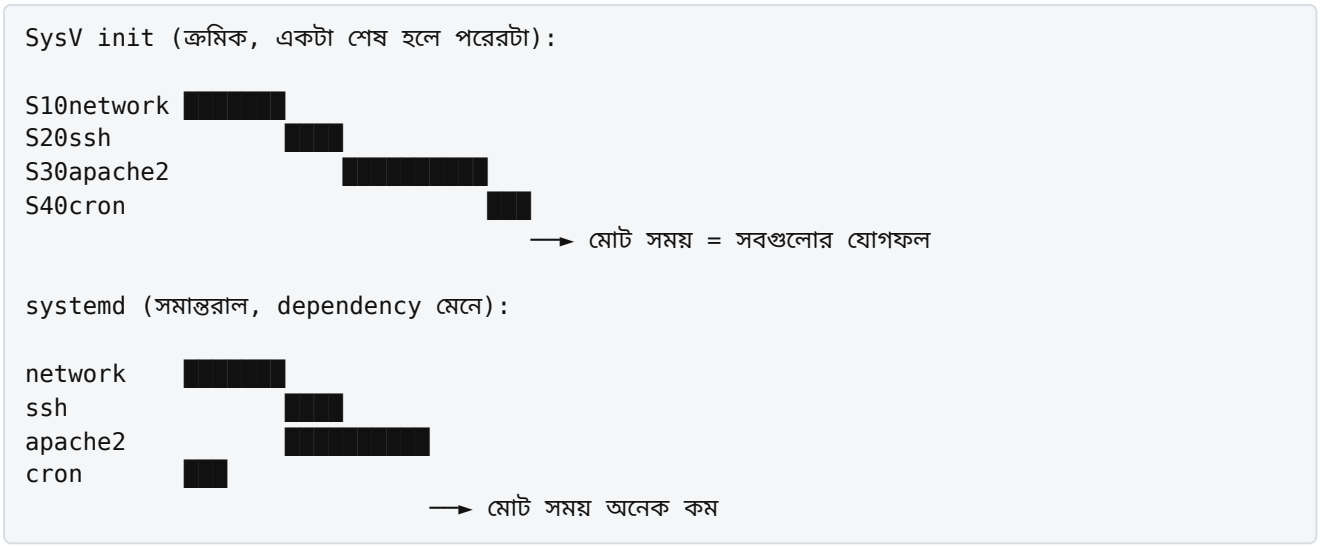
> 📝 **মনে রাখো:** PID 1 মারা গেলে kernel panic হয়। তাই init হতে হয় অত্যন্ত নির্ভরযোগ্য।

২৬.২ SysV init থেকে systemd: কেন এই পরিবর্তন?

পুরোনো দিন: SysV init

১৯৮০-এর দশকের UNIX System V থেকে আসা **SysV init** বহু বছর লিনাক্সে চলেছে। এর পদ্ধতি ছিল সহজ:

- প্রতিটি সার্ভিসের জন্য `/etc/init.d/` ফোল্ডারে একটা **shell script** থাকত, যেমন `/etc/init.d/apache2`।
- **runlevel** (০ থেকে ৬ পর্যন্ত সংখ্যা) দিয়ে সিস্টেমের অবস্থা বোঝানো হতো।
- `/etc/rc3.d/` ফোল্ডারে `S20apache2`, `S10network` এরকম নামের symlink থাকত। `S` মানে start, `K` মানে kill, আর সংখ্যা দিয়ে ক্রম ঠিক হতো।
- স্ক্রিপ্টগুলো **একটার পর একটা (sequentially)** চলত।



SysV init-এর সমস্যাগুলো

সমস্যা	ব্যাখ্যা
ধীর বুট	স্ক্রিপ্ট একটা একটা করে চলে, CPU-র একাধিক core কাজে লাগে না
সংখ্যা দিয়ে ক্রম	`S20` না `S25`? dependency আসলে কী তা স্পষ্ট লেখা থাকে না
দেখাশোনা নেই	সার্ভিস crash করলে init জানতেই পারে না, আবার চালু করে না
PID ফাইলের উপর নির্ভরতা	সার্ভিস কোন PID-এ চলছে তা জানতে `/var/run/*.pid` ফাইল লাগত, যা প্রায়ই ভুল/পুরোনো থাকত
প্রতিটি স্ক্রিপ্ট আলাদা	শত শত লাইনের shell script, প্রতিটি নিজের মতো লেখা, বাগে ভরা
child process ট্র্যাক করা কঠিন	সার্ভিস fork করে যে process বানায়, সেগুলো ধরে রাখার উপায় ছিল না

systemd-এর আগমন

২০১০ সালে Lennart Poettering ও Kay Sievers **systemd** তৈরি করেন। ২০১৫ সালের মধ্যে Fedora, RHEL 7, Debian 8, Ubuntu 15.04 সহ প্রায় সব বড় distro এটা গ্রহণ করে। (মাঝখানে Ubuntu কিছুদিন **Upstart** নামে নিজের init ব্যবহার করেছিল।)

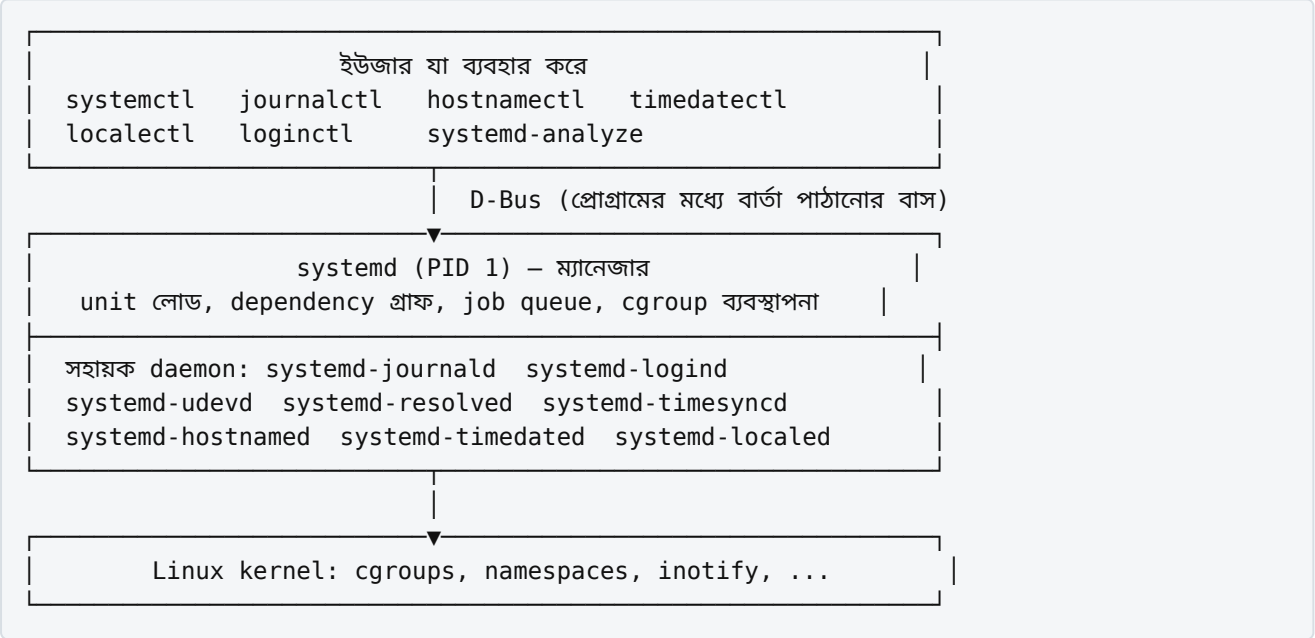
systemd যা নতুন আনল:

1. **Parallel start:** dependency না থাকলে সার্ভিসগুলো একসাথে চালু হয়।
2. **Declarative unit ফাইল:** script-এর বদলে সহজ `key=value` কনফিগারেশন। "কীভাবে করবে" না লিখে "কী চাই" লেখো।
3. **cgroups (control groups) দিয়ে ট্র্যাকিং:** প্রতিটি সার্ভিস নিজের cgroup-এ চলে। সার্ভিস যত child process-ই বানাক, সবগুলো ওই cgroup-এর ভেতরে থাকে, তাই stop করলে কেউ ফাঁকি দিয়ে থেকে যায় না।
4. **Socket ও on-demand activation:** কেউ সংযোগ করলে তবেই সার্ভিস চালু।
5. **স্বয়ংক্রিয় restart:** `Restart=on-failure` লিখে দিলেই crash হলে আবার চালু।

6. **একীভূত টুলসেট:** journald (লগ), logind (লগইন সেশন), networkd, resolved, timesyncd, hostnamed, timedated, localed ইত্যাদি।

> ⚠️ **সতর্কতা:** systemd নিয়ে লিনাক্স কমিউনিটিতে বিতর্ক আছে। সমালোচকরা বলেন এটা "একটা কাজ ভালোভাবে করো" এই UNIX দর্শন ভাঙে। Devuan, Alpine (OpenRC), Void (runit), Gentoo (OpenRC ডিফল্ট) এর মতো কিছু distro systemd ব্যবহার করে না। তবে Ubuntu, Debian, Fedora, RHEL, Arch, openSUSE সবগুলোতে systemd-ই ডিফল্ট, তাই চাকরির বাজারে এটা জানা আবশ্যক।

systemd-এর ভেতরের আর্কিটেকচার



তুমি যখন `systemctl restart nginx` লেখো, তখন `systemctl` নিজে কিছু চালু করে না। সে D-Bus দিয়ে PID 1-কে বার্তা পাঠায়: "nginx.service restart করার একটা **job** বানাও।" PID 1 সেটা job queue-তে রাখে, dependency দেখে, তারপর কাজটা করে।

২৬.৩ Unit: systemd-এর মূল ধারণা

systemd যা কিছু পরিচালনা করে, তার প্রতিটিকে বলে **unit**। unit-এর নামের শেষে যে extension থাকে, সেটা দেখে বোঝা যায় এটা কোন ধরনের unit।

Unit টাইপ	Extension	কী বোঝায়	উদাহরণ
Service	`.service`	একটা daemon বা প্রোগ্রাম চালানো	`ssh.service`, `nginx.service`
Socket	`.socket`	একটা নেটওয়ার্ক/IPC socket; সংযোগ এলে service চালু করে	`ssh.socket`, `docker.socket`
Target	`.target`	কয়েকটা unit-এর দল; একটা "অবস্থা" বা মাইলফলক	`multi-user.target`

Unit টাইপ	Extension	কী বোঝায়	উদাহরণ
Timer	`.timer`	নির্দিষ্ট সময়ে service চালানো (cron-এর বিকল্প)	`apt-daily.timer`
Mount	`.mount`	একটা ফাইলসিস্টেম mount পয়েন্ট	`home.mount`
Automount	`.automount`	প্রথম অ্যাক্সেসে mount হবে	`proc-sys-fs-binfmt_misc.automount`
Device	`.device`	udev থেকে আসা হার্ডওয়্যার ডিভাইস	`dev-sda.device`
Path	`.path`	কোনো ফাইল/ফোল্ডার বদলালে service চালু	`cups.path`
Swap	`.swap`	swap পার্টিশন/ফাইল	`swapfile.swap`
Slice	`.slice`	cgroup-এর রিসোর্স ভাগ (CPU/মেমরি সীমা)	`user.slice`, `system.slice`
Scope	`.scope`	বাইরে থেকে চালু হওয়া process-এর দল (যেমন লগইন সেশন)	`session-3.scope`

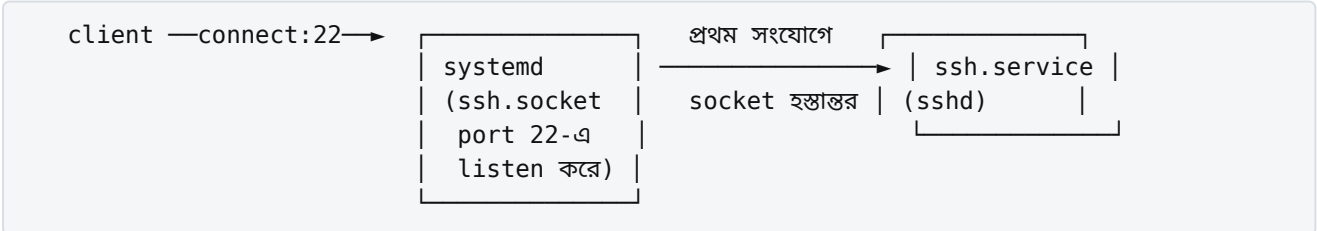
> 💡 **উপমা:** unit যেন একটা কোম্পানির বিভিন্ন কর্মী-পদ। service হলো যে আসল কাজ করে; socket হলো রিসেপশন ডেস্ক, যেখানে কেউ এলে সংশ্লিষ্ট কর্মীকে ডেকে আনা হয়; timer হলো এলার্ম ঘড়ি; target হলো একটা বিভাগ ("হিসাব বিভাগ পুরো চালু" মানে এর সব কর্মী হাজির); mount হলো গুদামঘরের দরজা খোলা।

Service unit

সবচেয়ে বেশি ব্যবহার হয়। একটা প্রোগ্রাম কীভাবে চালাতে হবে, কোন ইউজার হিসেবে, crash করলে কী করতে হবে, সব লেখা থাকে।

Socket unit ও socket activation

socket activation-এ systemd নিজে port-এ শুনতে (listen) থাকে। কোনো client সংযোগ করলে তবেই আসল service চালু করে socket-টা তার হাতে দিয়ে দেয়।



সুবিধা: বুট দ্রুত হয় (সার্ভিস আগে থেকে চালু থাকতে হয় না), মেমরি বাঁচে, আর যে সার্ভিসগুলো একে অপরের socket-এর উপর নির্ভর করে, সেগুলোও একসাথে চালু হতে পারে।

> 📝 **মনে রাখো:** Ubuntu 22.10 থেকে (এবং 24.04-এ) OpenSSH ডিফল্টভাবে `ssh.socket` দিয়ে socket activation ব্যবহার করে। তাই Ubuntu 24.04-এ SSH-এর port বদলাতে চাইলে শুধু `sshd_config` বদলালে হবে না; `ssh.socket`-এর `ListenStream` বা `daemon-reload` +

> `ssh.socket` restart লাগে। Debian 12 ও Fedora/RHEL-এ `ssh.service` / `sshd.service` সরাসরি চলে। SSH নিয়ে বিস্তারিত অধ্যায় ৩৮-এ।

Target unit

target নিজে কোনো কাজ করে না; এটা অন্য unit-গুলোকে দলবদ্ধ করে একটা "সিঙ্ক্রোনাইজেশন পয়েন্ট" বানায়। যেমন `network-online.target` মানে "নেটওয়ার্ক পুরোপুরি চালু হয়েছে।" পরে ২৬.৫-এ বিস্তারিত দেখব।

Timer unit

cron-এর আধুনিক বিকল্প। `backup.timer` ফাইলটি `backup.service`-কে সময়মতো চালু করে। বিস্তারিত অধ্যায় ২৯-এ।

Mount unit

`/etc/fstab`-এর প্রতিটি লাইন থেকে systemd বুটের সময় স্বয়ংক্রিয়ভাবে একটা `.mount` unit বানায় (`systemd-fstab-generator` নামের একটা generator এই কাজ করে)। mount unit-এর নাম হয় mount পয়েন্টের path থেকে: `/` বাদ দিয়ে `-` বসিয়ে। যেমন `/var/lib/data` → `var-lib-data.mount`।

```
$ systemd-escape -p --suffix=mount /var/lib/data
var-lib-data.mount
```

ডিস্ক ও mount নিয়ে বিস্তারিত অধ্যায় ৩২-এ।

Template unit (`@` চিহ্নওয়ালা)

কিছু unit-এর নামে `@` থাকে, যেমন `getty@.service`। এটা একটা ছাঁচ (template)। `@`-এর পরে যা লিখবে, তা হবে **instance**-এর নাম:

```
getty@tty1.service   → tty1-এ লগইন প্রম্পট
getty@tty2.service   → tty2-এ লগইন প্রম্পট
```

unit ফাইলের ভেতরে `%i` লিখলে সেখানে instance নামটা বসে যায়। এভাবে এক ফাইল দিয়ে অনেক instance চালানো যায়।

২৬.৪ systemctl: সার্ভিস নিয়ন্ত্রণের প্রধান অস্ত্র

`systemctl` হলো systemd-এর সাথে কথা বলার মূল কমান্ড।

সাধারণ syntax:

```
systemctl [OPTIONS] COMMAND [UNIT...]
```

unit-এর নামে `.service` না লিখলে systemctl ধরে নেয় সেটা service। অর্থাৎ `systemctl status ssh` আর `systemctl status ssh.service` একই।

> ⚠️ **সতর্কতা:** অবস্থা দেখার কমান্ড (`status` , `is-active` , `list-units`) সাধারণ ইউজার চালাতে পারে, কিন্তু অবস্থা বদলানোর কমান্ড (`start` , `stop` , `enable` ইত্যাদি) চালাতে `sudo` লাগে (অধ্যায় ১৮)। sudo ছাড়া চালালে desktop-এ polkit পাসওয়ার্ড চাইতে পারে।

> 📝 **মনে রাখো (distro-ভেদে নাম):** একই সফটওয়্যারের সার্ভিসের নাম distro-ভেদে আলাদা হয়।

সফটওয়্যার	Ubuntu/Debian	Fedora/RHEL
OpenSSH সার্ভার	`ssh`	`sshd`
Apache	`apache2`	`httpd`
cron	`cron`	`crond`
MySQL/MariaDB	`mysql` / `mariadb`	`mysqld` / `mariadb`

২৬.৪.১ status: অবস্থা দেখা

```
$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: active (running) since Tue 2026-11-10 09:14:02 +06; 2h 5min ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1043 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1051 (sshd)
      Tasks: 1 (limit: 9318)
     Memory: 4.2M (peak: 5.0M)
        CPU: 61ms
     CGroup: /system.slice/ssh.service
             └─1051 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 10 09:14:02 server1 systemd[1]: Starting ssh.service - OpenBSD Secure Shell
server...
Nov 10 09:14:02 server1 sshd[1051]: Server listening on 0.0.0.0 port 22.
Nov 10 09:14:02 server1 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

প্রতিটি লাইন কী বলছে:

লাইন	অর্থ
`●` (প্রথম লাইনের বিন্দু)	সবুজ = চলছে, সাদা = বন্ধ, লাল = failed
`Loaded:`	unit ফাইল কোথা থেকে লোড হলো; `enabled/disabled` = বুটে নিজে চালু হবে কি না; `preset` = distro-র ডিফল্ট পছন্দ
`Active:`	এখনকার অবস্থা ও কতক্ষণ ধরে
`TriggeredBy:`	কোন socket/timer এটাকে চালু করে
`Main PID:`	মূল process-এর PID
`Tasks`, `Memory`, `CPU`	cgroup থেকে পাওয়া রিসোর্স ব্যবহার
`CGroup:`	এই সার্ভিসের অধীনে থাকা সব process-এর গাছ
শেষের লাইনগুলো	journal থেকে সর্বশেষ ১০টা লগ লাইন

উপরের উদাহরণে `disabled` লেখা থাকলেও সার্ভিস চলছে, কারণ এটা `ssh.socket` দিয়ে চালু হয়েছে (Ubuntu 24.04-এর বৈশিষ্ট্য)।

Active অবস্থার সম্ভাব্য মান:

অবস্থা	অর্থ
`active (running)`	চলছে, process জীবিত
`active (exited)`	একবার চলে সফলভাবে শেষ (`oneshot` টাইপে স্বাভাবিক)
`active (waiting)`	চালু, কিন্তু কোনো ঘটনার অপেক্ষায় (যেমন timer)
`inactive (dead)`	বন্ধ
`activating` / `deactivating`	চালু/বন্ধ হওয়ার মাঝপথে
`failed`	ব্যর্থ হয়েছে (crash, exit code ≠0, timeout)

দরকারি অপশন:

```
$ systemctl status nginx -n 50          # শেষ ৫০টা লগ লাইন
$ systemctl status nginx --no-pager     # pager (less) ছাড়া সরাসরি আউটপুট
$ systemctl status nginx -l             # লম্বা লাইন না কেটে পুরোটা দেখাও
$ systemctl status 1051                 # PID দিয়েও খোঁজা যায় (কোন unit-এর process)
```

> 💡 **টিপস:** status-এর শুধু ১০ লাইন লগ যথেষ্ট না হলে `journalctl -u nginx` চালাও। `journalctl -u nginx -f` দিলে লাইভ লগ দেখাবে, `-b` দিলে শুধু এই বুটের। journal নিয়ে বিস্তারিত অধ্যায় ২৮-এ।

২৬.৪.২ start, stop, restart, reload

```
$ sudo systemctl start nginx       # এখনই চালু করো
$ sudo systemctl stop nginx        # এখনই বন্ধ করো
$ sudo systemctl restart nginx     # বন্ধ করে আবার চালু (process নতুন হবে)
$ sudo systemctl reload nginx      # process না থামিয়ে কনফিগ নতুন করে পড়ো
```

এই কমান্ডগুলো সফল হলে কিছুই প্রিন্ট করে না (UNIX-এর "নীরবতাই সাফল্য" নীতি)। ব্যর্থ হলে বার্তা দেখায়:

```
$ sudo systemctl start nginx
Job for nginx.service failed because the control process exited with error code.
See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.
```

restart বনাম reload:

```
restart:  [পুরোনো process] ──SIGTERM──► মৃত   ...   [নতুন process] চালু
          সংযোগগুলো কেটে যায়, কিছুক্ষণ সার্ভিস অনুপলব্ধ

reload:   [একই process] ◄──SIGHUP বা ExecReload কমান্ড── নতুন কনফিগ পড়ে
          চলমান সংযোগ টিকে থাকে, ডাউনটাইম প্রায় শূন্য
```

কমান্ড	কী করে
`restart`	সবসময় stop + start; বন্ধ থাকলে চালু করে
`reload`	unit ফাইলের `ExecReload=` চালায়; সব সার্ভিস এটা সমর্থন করে না
`try-restart`	চলমান থাকলে তবেই restart; বন্ধ থাকলে কিছু করে না
`reload-or-restart`	reload সমর্থন করলে reload, না হলে restart
`try-reload-or-restart`	চলমান থাকলেই, reload বা restart

⚠️ **সতর্কতা:** `systemctl reload` সার্ভিসের **কনফিগারেশন** (যেমন `/etc/nginx/nginx.conf`) আবার পড়ায়। এটা systemd-এর **unit ফাইল** আবার পড়ায় না। unit ফাইল বদলালে `systemctl daemon-reload` লাগে (২৬.৪.৮)। এই দুটো গুলিয়ে ফেলা নতুনদের সবচেয়ে সাধারণ ভুল।

💡 **টিপস:** প্রোডাকশনে কনফিগ বদলানোর পর আগে সিনট্যাক্স যাচাই করো, তারপর reload: `sudo nginx -t && sudo systemctl reload nginx`। `&&` মানে প্রথমটা সফল হলেই দ্বিতীয়টা চলবে।

একসাথে একাধিক unit:

```
$ sudo systemctl restart nginx php8.3-fpm
```

২৬.৪.৩ enable ও disable: বুটে চালু হবে কি না

`start` মানে **এখন** চালু। `enable` মানে **পরের বুট থেকে** স্বয়ংক্রিয়ভাবে চালু। দুটো সম্পূর্ণ আলাদা।

```
$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service →
/usr/lib/systemd/system/nginx.service.

$ sudo systemctl disable nginx
Removed "/etc/systemd/system/multi-user.target.wants/nginx.service".
```

ভেতরে কী হলো? `enable` আসলে শুধু একটা **symlink** বানায়। unit ফাইলের `[Install]` অংশে `WantedBy=multi-user.target` লেখা থাকে, তাই systemd `multi-user.target.wants/` ফোল্ডারে symlink রাখে। বুটের সময় systemd যখন `multi-user.target` চালু করে, তখন ওই `.wants` ফোল্ডারের সবাইকে চালু করে।

```
/etc/systemd/system/
└── multi-user.target.wants/
    ├── nginx.service  ──→ /usr/lib/systemd/system/nginx.service
    ├── cron.service   ──→ /usr/lib/systemd/system/cron.service
    └── ssh.service    ──→ ...
```

একসাথে enable ও start:

```
$ sudo systemctl enable --now nginx      # enable + start
$ sudo systemctl disable --now nginx     # disable + stop
```

> 💡 **টিপস:** নতুন সার্ভিস ইনস্টল করে প্রায় সবসময় `enable --now` ব্যবহার করবে। শুধু `start` দিলে রিবুটের পর সার্ভিস আর চলবে না, আর শুধু `enable` দিলে রিবুট পর্যন্ত চলবে না।

> 📝 **মনে রাখো:** Debian/Ubuntu-তে কোনো সার্ভিস প্যাকেজ (যেমন `apt install nginx`) ইনস্টল করলে সাধারণত সেটা নিজে থেকেই enable ও start হয়ে যায়। Fedora/RHEL-এ ডিফল্টভাবে হয় না; নিজে `enable --now` করতে হয়। এই পার্থক্য আসে **preset** নীতি থেকে (`/usr/lib/systemd/system-preset/`)।

২৬.৪.৪ is-active, is-enabled, is-failed: স্ক্রিপ্টের জন্য প্রশ্ন

এগুলো এক শব্দে উত্তর দেয় এবং exit code (০ = হ্যাঁ) দিয়ে ফল জানায়, তাই শেল স্ক্রিপ্টে খুব কাজের (অধ্যায় ৪১)।

```
$ systemctl is-active nginx
active
```

```
$ systemctl is-enabled nginx
enabled

$ systemctl is-failed nginx
active
```

`is-failed` failed না হলে বর্তমান অবস্থা প্রিন্ট করে এবং exit code ১ দেয়।

`is-enabled`-এর সম্ভাব্য উত্তর: `enabled`, `disabled`, `static` (কোনো `[Install]` অংশ নেই, অন্য unit-এর dependency হিসেবেই চলে), `masked`, `alias`, `indirect`, `generated`, `linked`।

স্ক্রিপ্টে ব্যবহার:

```
if systemctl is-active --quiet nginx; then
    echo "nginx চলছে"
else
    echo "nginx বন্ধ! চালু করছি..."
    sudo systemctl start nginx
fi
```

`--quiet` (বা `-q`) কিছু প্রিন্ট না করে শুধু exit code দেয়।

২৬.৪.৫ mask ও unmask: সম্পূর্ণ নিষিদ্ধ করা

`disable` করা সার্ভিস হাতে `start` করা যায়, বা অন্য কোনো সার্ভিস dependency হিসেবে তাকে চালু করে দিতে পারে। **mask** করলে কেউই আর চালু করতে পারবে না।

```
$ sudo systemctl mask apache2
Created symlink /etc/systemd/system/apache2.service → /dev/null.

$ sudo systemctl start apache2
Failed to start apache2.service: Unit apache2.service is masked.

$ sudo systemctl unmask apache2
Removed "/etc/systemd/system/apache2.service".
```

ভেতরে কী হয়: `/etc/systemd/system/`-এ unit-এর নামে `/dev/null`-এর দিকে একটা symlink তৈরি হয়। `/etc` অগ্রাধিকার পায় (২৬.৬ দেখো), তাই systemd আসল ফাইলের বদলে একটা ফাঁকা "কিছুই না" পায়।

অবস্থা	বুটে চালু হবে?	হাতে start করা যাবে?	অন্য unit চালু করাতে পারবে?
enabled	হ্যাঁ	হ্যাঁ	হ্যাঁ
disabled	না	হ্যাঁ	হ্যাঁ
masked	না	না	না

> 💡 **উপমা:** disable মানে কর্মীকে বলা "কাল থেকে নিজে আসার দরকার নেই, ডাকলে এসো।" mask মানে

> তার অফিস কার্ড বাতিল করে দেওয়া; ডাকলেও ঢুকতে পারবে না।

> 💡 **টিপস:** একই port-এ দুটো সার্ভিস (যেমন apache2 ও nginx দুটোই port 80) চললে সংঘর্ষ হয়। যেটা চাও না সেটাকে `mask --now` করে দাও: `sudo systemctl mask --now apache2`। আবার, ল্যাপটপে sleep বন্ধ রাখতে কেউ কেউ `sleep.target suspend.target hibernate.target hybrid-sleep.target` mask করে।

২৬.৪.৬ list-units ও list-unit-files: তালিকা দেখা

```
$ systemctl list-units --type=service
  UNIT                     LOAD   ACTIVE SUB     DESCRIPTION
  cron.service             loaded active running Regular background program processing
daemon
  nginx.service            loaded active running A high performance web server and a
reverse proxy server
  ssh.service              loaded active running OpenBSD Secure Shell server
  systemd-journald.service  loaded active running Journal Service
  ...

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.
42 loaded units listed. Pass --all to see loaded but inactive units, too.
```

`list-units` শুধু **মেমরিতে লোড হওয়া** unit দেখায় (ডিফল্টভাবে শুধু active বা failed)। আর `list-unit-files` ডিস্কে থাকা **সব** unit ফাইল ও তাদের enable অবস্থা দেখায়।

```
$ systemctl list-unit-files --type=service --state=enabled
UNIT FILE                 STATE   PRESET
cron.service              enabled enabled
nginx.service             enabled enabled
systemd-timesyncd.service enabled enabled
...
```

দরকারি অপশন:

অপশন	কাজ
`--type=service` (বা `-t service`)	নির্দিষ্ট ধরন: service, socket, timer, target, mount...
`--state=running`	নির্দিষ্ট অবস্থা: running, failed, exited, inactive, enabled...
`--all` (বা `-a`)	inactive unit সহ সব দেখাও
`--failed`	শুধু ব্যর্থ unit (`--state=failed`-এর সংক্ষেপ)
`--no-legend`	হেডার ও ফুটার বাদ (স্ক্রিপ্টের জন্য)
`--no-pager`	pager ছাড়া আউটপুট

অপশন	কাজ
প্যাটার্ন	`systemctl list-units 'ssh*'`

সবচেয়ে দরকারি কয়েকটা:

```
$ systemctl --failed                                    # কী কী ব্যর্থ হয়েছে?
  UNIT            LOAD   ACTIVE SUB    DESCRIPTION
● myapp.service   loaded failed failed My Python App

$ systemctl list-units --type=service --state=running # কী কী চলছে?
$ systemctl list-timers                                 # সব timer ও পরের চলার সময়
$ systemctl list-sockets                                # কোন socket কোন port-এ
```

শুধু `systemctl` লিখে Enter চাপলেও `list-units`-ই চলে।

> 💡 **টিপস:** সার্ভার লগইন করার পর প্রথম কমান্ড হিসেবে `systemctl --failed` চালানোর অভ্যাস করো। এক নজরে বুঝবে কিছু ভেঙেছে কি না। পুরো সিস্টেমের সারাংশ পেতে `systemctl status` (unit নাম ছাড়া) চালাও; উপরে `State: running` বা `State: degraded` দেখাবে।

২৬.৪.৭ আরও দরকারি systemctl কমান্ড

```
$ systemctl cat nginx                # unit ফাইল ও সব override একসাথে দেখাও
# /usr/lib/systemd/system/nginx.service
[Unit]
Description=A high performance web server and a reverse proxy server
...

$ systemctl show nginx -p MainPID -p ActiveState -p Restart
MainPID=1210
ActiveState=active
Restart=no

$ systemctl list-dependencies nginx   # কার উপর নির্ভরশীল (গাছ আকারে)
$ systemctl list-dependencies --reverse nginx   # কে nginx-এর উপর নির্ভরশীল

$ sudo systemctl kill -s SIGUSR1 nginx      # সার্ভিসের process-এ signal পাঠাও
$ sudo systemctl reset-failed               # failed অবস্থা মুছে দাও
```

কমান্ড	কাজ
`cat UNIT`	কার্যকর unit ফাইল (মূল + drop-in) দেখায়
`show UNIT`	সব property `Key=Value` আকারে (মেশিন-পাঠযোগ্য)
`show -p PROP UNIT`	নির্দিষ্ট property
`list-dependencies UNIT`	dependency গাছ
`kill UNIT`	cgroup-এর সব process-এ signal

কমান্ড	কাজ
`reset-failed [UNIT]`	failed অবস্থা ও restart কাউন্টার রিসেট
`edit UNIT`	override তৈরি (২৬.৮)
`revert UNIT`	সব override মুছে মূল অবস্থায় ফেরা
`daemon-reload`	unit ফাইল আবার পড়া
`get-default` / `set-default`	ডিফল্ট target
`isolate TARGET`	শুধু ওই target ও তার dependency রেখে বাকি সব থামাও
`poweroff`, `reboot`, `suspend`, `hibernate`	পাওয়ার ব্যবস্থাপনা

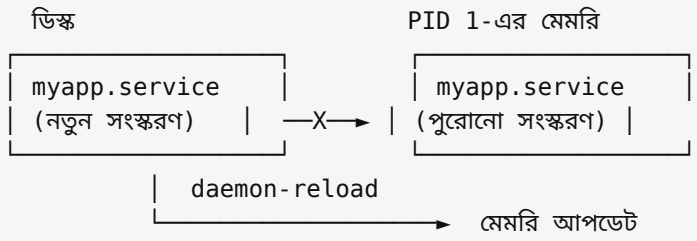
পাওয়ার কমান্ড: `shutdown`, `reboot`, `poweroff`, `halt` কমান্ডগুলো আধুনিক সিস্টেমে আসলে systemctl-এরই symlink।

```
$ sudo systemctl reboot
$ sudo systemctl poweroff
$ sudo shutdown -h +10 "১০ মিনিটে বন্ধ হবে"   # শিডিউল করা শাটডাউন
$ sudo shutdown -c                              # বাতিল
```

২৬.৪.৮ daemon-reload: unit ফাইল বদলানোর পর

systemd বুটের সময় সব unit ফাইল পড়ে মেমরিতে রাখে। তুমি পরে কোনো unit ফাইল নিজে বদলালে systemd তা জানে না, পুরোনো সংস্করণ নিয়েই কাজ চালায়।

```
$ sudo nano /etc/systemd/system/myapp.service      # কিছু বদলালে

$ sudo systemctl restart myapp
Warning: The unit file, source configuration file or drop-ins of myapp.service changed
on disk. Run 'systemctl daemon-reload' to reload units.

$ sudo systemctl daemon-reload                     # নতুন করে সব unit ফাইল পড়ো
$ sudo systemctl restart myapp                     # এবার নতুন সেটিং দিয়ে চলবে
```



> 📝 **মনে রাখো:** `daemon-reload` চলমান সার্ভিস restart করে না, শুধু definition আপডেট করে। নতুন সেটিং প্রয়োগ করতে সার্ভিসটা restart করতে হবে। আর `systemctl edit` ব্যবহার করলে daemon-reload নিজেই হয়ে যায়।

`daemon-reexec` আলাদা জিনিস: এটা systemd বাইনারিটাকেই নতুন করে চালায় (সাধারণত systemd প্যাকেজ আপগ্রেডের পর প্যাকেজ ম্যানেজার নিজেই করে)।

২৬.৫ Target ও Runlevel

runlevel কী ছিল

SysV init-এ সিস্টেমের অবস্থা বোঝাতে ০–৬ সংখ্যা ব্যবহার হতো। systemd সংখ্যার বদলে নামওয়ালা **target** ব্যবহার করে, তবে পুরোনো অভ্যাসের সাথে মিল রাখতে `runlevelN.target` নামের alias রেখেছে।

Runlevel	systemd target	অর্থ
0	`poweroff.target`	সিস্টেম বন্ধ
1	`rescue.target`	single-user মোড: শুধু root, নেটওয়ার্ক নেই, মেরামতের জন্য
2	`multi-user.target`	(Debian-এ পুরোনো দিনে পূর্ণ মোড ছিল)
3	`multi-user.target`	মাল্টি-ইউজার, নেটওয়ার্ক সহ, **GUI ছাড়া** (সার্ভারের সাধারণ মোড)
4	`multi-user.target`	ব্যবহারকারী-নির্ধারিত (কার্যত অব্যবহৃত)
5	`graphical.target`	multi-user + GUI লগইন স্ক্রিন (ডেস্কটপের সাধারণ মোড)
6	`reboot.target`	রিবুট
—	`emergency.target`	rescue-এর চেয়েও কম: শুধু root ফাইলসিস্টেম read-only, প্রায় কোনো সার্ভিস নেই

target-গুলো একে অপরের উপর দাঁড়িয়ে থাকে:

```
                graphical.target        (GUI: gdm/sddm)
                       │ Requires
                       ▼
                multi-user.target       (ssh, cron, nginx ...)
                       │ Requires
                       ▼
                 basic.target           (sockets, timers, paths, slices)
                       │
                       ▼
                sysinit.target          (mount, swap, udev, journald)
                       │
                       ▼
            local-fs.target + swap.target
```

এই ক্রম ও প্রতিটি ধাপের বিস্তারিত অধ্যায় ২৭-এ পাবে।

target নিয়ে কাজ

```
$ systemctl get-default
graphical.target
```

```
$ sudo systemctl set-default multi-user.target     # পরের বুট থেকে GUI ছাড়া
Removed "/etc/systemd/system/default.target".
Created symlink /etc/systemd/system/default.target → /usr/lib/systemd/system/multi-
user.target.

$ sudo systemctl isolate multi-user.target         # এখনই GUI বন্ধ করে টেক্সট মোডে
$ sudo systemctl isolate graphical.target          # এখনই GUI-তে ফেরা
$ sudo systemctl rescue                            # rescue মোডে যাও (isolate
rescue.target)
```

`default.target` আসলে একটা symlink; `set-default` শুধু সেটা বদলে দেয়।

> ⚠️ **সতর্কতা:** `isolate` ওই target-এর অংশ নয় এমন **সব** unit থামিয়ে দেয়। SSH দিয়ে দূরের সার্ভারে `isolate rescue.target` চালালে নেটওয়ার্ক বন্ধ হয়ে তোমার সংযোগ কেটে যাবে, আর ফিরে আসতে সার্ভারের কনসোলে সরাসরি যেতে হবে।

শুধু যে target-গুলোর unit ফাইলে `AllowIsolate=yes` আছে, সেগুলোকেই isolate করা যায়।

বুটের সময় এককালীন target বেছে নেওয়া: GRUB মেনুতে `e` চেপে kernel লাইনের (`linux ...`) শেষে লেখো `systemd.unit=rescue.target` (বা `emergency.target`), তারপর `Ctrl+X` বা `F10` চেপে বুট করো। পাসওয়ার্ড ভুলে গেলে বা fstab ভুলের কারণে বুট আটকে গেলে এটাই উদ্ধারের রাস্তা।

পুরোনো কমান্ডগুলোও এখনো কাজ করে:

```
$ runlevel
N 5
$ who -r
         run-level 5  2026-11-10 09:13
```

`N 5` মানে আগের runlevel নেই (N = none), এখন ৫।

২৬.৬ Unit ফাইল কোথায় থাকে

systemd তিনটি প্রধান জায়গা থেকে unit ফাইল পড়ে, এবং একই নামের ফাইল একাধিক জায়গায় থাকলে **অগ্রাধিকার (priority)** মেনে চলে।

অবস্থান	কে রাখে	অগ্রাধিকার
`/etc/systemd/system/`	**তুমি (অ্যাডমিন)**	সর্বোচ্চ
`/run/systemd/system/`	runtime-এ তৈরি (রিবুটে মুছে যায়)	মাঝারি
`/usr/lib/systemd/system/` (বা `/lib/systemd/system/`)	**প্যাকেজ ম্যানেজার** (apt/dnf)	সর্বনিম্ন

```
একই নামের unit খোঁজার ক্রম:

  /etc/systemd/system/nginx.service      ← থাকলে এটাই জিতবে
          │ না থাকলে
          ▼
  /run/systemd/system/nginx.service
          │ না থাকলে
          ▼
  /usr/lib/systemd/system/nginx.service  ← প্যাকেজের দেওয়া মূল ফাইল
```

> 📝 **মনে রাখো (/lib বনাম /usr/lib):** Ubuntu 24.04, Debian 12, Fedora ও RHEL 9 সবগুলোই "merged /usr" ব্যবহার করে, অর্থাৎ `/lib` আসলে `/usr/lib`-এর একটা symlink। তাই `/lib/systemd/system` আর `/usr/lib/systemd/system` একই ফোল্ডার। পুরোনো Debian/Ubuntu ডকুমেন্টেশনে `/lib/systemd/system` লেখা দেখবে; Fedora/RHEL-এ `/usr/lib/systemd/system`। FHS নিয়ে আরও জানতে অধ্যায় ৭।

> ⚠️ **সতর্কতা:** `/usr/lib/systemd/system/`-এর ফাইল **কখনো সরাসরি এডিট করবে না।** পরের প্যাকেজ আপডেটে তোমার পরিবর্তন মুছে যাবে। পরিবর্তন সবসময় `/etc/systemd/system/`-এ করো, সবচেয়ে ভালো উপায় `systemctl edit` (২৬.৮)।

user unit-এর অবস্থান (`systemctl --user`, ২৬.১১):

অবস্থান	ব্যাখ্যা
`~/.config/systemd/user/`	নিজের ইউজারের unit
`/etc/systemd/user/`	অ্যাডমিন দিয়েছে, সব ইউজারের জন্য
`/usr/lib/systemd/user/`	প্যাকেজের দেওয়া

পুরো তালিকা দেখতে চাইলে:

```
$ systemd-analyze unit-paths
/etc/systemd/system.control
/run/systemd/system.control
/run/systemd/transient
/run/systemd/generator.early
/etc/systemd/system
/etc/systemd/system.attached
/run/systemd/system
...
/usr/lib/systemd/system
/run/systemd/generator.late
```

২৬.৭ Unit ফাইলের গঠন

unit ফাইল হলো INI-ধরনের একটা টেক্সট ফাইল: `[Section]` হেডার, তার নিচে `Key=Value` লাইন। `#` বা `;` দিয়ে শুরু লাইন হলো মন্তব্য।

একটা বাস্তব উদাহরণ (Ubuntu-র `cron.service`, সামান্য সংক্ষেপিত):

```
[Unit]
Description=Regular background program processing daemon
Documentation=man:cron(8)
After=remote-fs.target nss-user-lookup.target

[Service]
EnvironmentFile=-/etc/default/cron
ExecStart=/usr/sbin/cron -f -P $EXTRA_OPTS
IgnoreSIGPIPE=false
KillMode=process
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

(`EnvironmentFile=`-এর আগে `-` মানে "ফাইলটা না থাকলেও ভুল ধরো না।")

```
┌──────────────────────────────────────────────────────┐
│ [Unit]     → সব unit টাইপে থাকে: পরিচিতি ও dependency  │
├──────────────────────────────────────────────────────┤
│ [Service]  → টাইপ-নির্দিষ্ট অংশ: কীভাবে চালাতে হবে      │
│ ([Socket], [Timer], [Mount], [Path] ... টাইপ অনুযায়ী)  │
├──────────────────────────────────────────────────────┤
│ [Install]  → enable করলে কোথায় যুক্ত হবে               │
└──────────────────────────────────────────────────────┘
```

[Unit] সেকশনের গুরুত্বপূর্ণ key

Key	কাজ
`Description=`	এক লাইনের বর্ণনা (status-এ দেখায়)
`Documentation=`	man পেজ বা URL
`Wants=`	দুর্বল dependency (২৬.৯)
`Requires=`	শক্ত dependency
`After=` / `Before=`	চালু হওয়ার ক্রম
`Conflicts=`	একসাথে চলতে পারবে না
`ConditionPathExists=`	শর্ত পূরণ না হলে নীরবে skip
`AssertPathExists=`	শর্ত পূরণ না হলে failed

Key	কাজ
`StartLimitIntervalSec=`, `StartLimitBurst=`	নির্দিষ্ট সময়ে সর্বোচ্চ কতবার start করা যাবে
`OnFailure=`	ব্যর্থ হলে অন্য কোন unit চালু হবে (যেমন নোটিফিকেশন)

[Service] সেকশনের গুরুত্বপূর্ণ key

Key	কাজ
`Type=`	সার্ভিস কীভাবে "চালু হয়েছে" জানায় (নিচের টেবিল)
`ExecStart=`	মূল কমান্ড (**absolute path** দিতে হবে বা PATH-এ থাকতে হবে)
`ExecStartPre=` / `ExecStartPost=`	আগে/পরে চালানোর কমান্ড
`ExecReload=`	`reload`-এ কী চলবে, যেমন `/bin/kill -HUP $MAINPID`
`ExecStop=`	থামানোর কমান্ড (না দিলে SIGTERM পাঠানো হয়)
`Restart=`	কখন আবার চালু হবে
`RestartSec=`	restart-এর আগে অপেক্ষা (ডিফল্ট 100ms)
`User=` / `Group=`	কোন ইউজার হিসেবে চলবে (ডিফল্ট root)
`WorkingDirectory=`	কোন ফোল্ডার থেকে চলবে
`Environment=`	এনভায়রনমেন্ট ভেরিয়েবল, যেমন `Environment="PORT=8080"`
`EnvironmentFile=`	ফাইল থেকে ভেরিয়েবল পড়া
`TimeoutStartSec=` / `TimeoutStopSec=`	কতক্ষণে চালু/বন্ধ না হলে ব্যর্থ ধরা হবে (ডিফল্ট সাধারণত 90s)
`KillMode=`	stop-এ কাদের মারা হবে: `control-group` (ডিফল্ট, সবাই), `mixed`, `process`, `none`
`StandardOutput=` / `StandardError=`	আউটপুট কোথায় যাবে (ডিফল্ট journal)

`Type=` এর মান:

Type	কখন ব্যবহার করবে	systemd কখন "চালু হয়েছে" ধরে
`simple`	ডিফল্ট; প্রোগ্রাম foreground-এ চলতে থাকে	fork করার সাথে সাথেই
`exec`	simple-এর মতো, তবে বেশি নির্ভুল	প্রোগ্রাম আসলে exec হলে (বাইনারি না থাকলে start-ই ব্যর্থ হবে)
`forking`	পুরোনো ধাঁচের daemon যারা নিজে background-এ চলে যায়	parent process exit করলে (প্রায়ই `PIDFile=` লাগে)
`oneshot`	একবার চলে শেষ হয় (স্ক্রিপ্ট, সেটআপ কাজ)	process শেষ হলে; সাথে `RemainAfterExit=yes` দিলে পরে active থাকে

Type	কখন ব্যবহার করবে	systemd কখন "চালু হয়েছে" ধরে
`notify`	প্রোগ্রাম নিজে `sd_notify("READY=1")` পাঠায়	READY বার্তা পেলে
`dbus`	D-Bus-এ নাম নেয়	`BusName=` পাওয়া গেলে
`idle`	simple-এর মতো, কিন্তু বাকি job শেষ হওয়া পর্যন্ত অপেক্ষা করে	কনসোল আউটপুট এলোমেলো না করার জন্য

> 💡 **টিপস:** নিজের লেখা প্রোগ্রাম (Python, Node.js, Go) সাধারণত foreground-এ চলে, তাই `Type=simple` বা `Type=exec` দাও। প্রোগ্রামের ভেতরে "daemonize" বা `&` দিয়ে background-এ পাঠাবে না; systemd নিজেই সেটা সামলায়।

`Restart=` এর মান:

মান	কখন restart
`no`	কখনো না (ডিফল্ট)
`on-failure`	exit code ≠0, signal-এ মৃত্যু, timeout, watchdog
`on-abnormal`	signal, timeout, watchdog (exit code ≠0 হলে নয়)
`on-abort`	শুধু অপ্রত্যাশিত signal-এ মৃত্যু
`on-success`	শুধু পরিষ্কারভাবে exit 0 হলে
`always`	যেভাবেই থামুক (হাতে `systemctl stop` বাদে)

> ⚠️ **সতর্কতা:** `Restart=always` দিলে আর প্রোগ্রামে কনফিগ ভুল থাকলে সেটা বারবার crash করবে। ডিফল্টভাবে ১০ সেকেন্ডে ৫ বারের বেশি start হলে systemd হাল ছেড়ে দিয়ে `failed` করে (বার্তা: `start request repeated too quickly`)। তখন ভুল ঠিক করে `systemctl reset-failed myapp` তারপর start করো।

[Install] সেকশন

Key	কাজ
`WantedBy=`	enable করলে কোন target-এর `.wants/` -এ symlink হবে (সবচেয়ে সাধারণ: `multi-user.target`)
`RequiredBy=`	`.requires/` -এ symlink
`Alias=`	বিকল্প নাম
`Also=`	এটা enable করলে আর কী কী enable হবে

`[Install]` অংশ শুধু `enable` / `disable` এর সময় পড়া হয়; সার্ভিস চলার সময় এর কোনো ভূমিকা নেই। যে unit-এ `[Install]` নেই, সেটা `static`; তাকে enable করা যায় না, অন্য unit dependency হিসেবে চালায়।

> 💡 **টিপস:** যেকোনো key-এর বিস্তারিত পেতে: `man systemd.unit` (সাধারণ ও [Unit]/[Install]), `man systemd.service` ([Service]), `man systemd.exec` (User, Environment, sandboxing), `man systemd.socket`, `man systemd.timer`, `man systemd.kill`, `man systemd.resource-control`। সব directive-এর সূচি: `man systemd.directives`।

Socket unit-এর একটা ছোট উদাহরণ

```
# /etc/systemd/system/echo.socket
[Unit]
Description=Echo socket

[Socket]
ListenStream=127.0.0.1:7777
Accept=yes

[Install]
WantedBy=sockets.target
```

`Accept=yes` মানে প্রতিটি সংযোগের জন্য আলাদা instance চালু হবে, তাই সাথে একটা template service লাগে:

```
# /etc/systemd/system/echo@.service
[Unit]
Description=Echo service

[Service]
ExecStart=/usr/bin/cat
StandardInput=socket
```

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now echo.socket
$ echo "hello systemd" | nc -q1 127.0.0.1 7777
hello systemd
```

(`nc` না থাকলে `sudo apt install netcat-openbsd`। Fedora-তে `nmap-ncat` প্যাকেজের `nc` ব্যবহার করো, যেখানে `-q1` অপশন নেই; সেখানে `--send-only` বা Ctrl+C দিয়ে বেরোও।)

২৬.৮ নিজের service unit লেখা: সম্পূর্ণ উদাহরণ

ধরো তুমি একটা ছোট Python ওয়েব অ্যাপ বানিয়েছ, চাও সেটা বুটে নিজে চালু হোক, crash করলে আবার উঠুক, আর root হিসেবে না চলুক।

ধাপ ১: প্রোগ্রাম তৈরি

```
$ sudo mkdir -p /opt/hello
$ sudo nano /opt/hello/app.py
```

```
#!/usr/bin/env python3
import http.server, os
port = int(os.environ.get("PORT", "8080"))
handler = http.server.SimpleHTTPRequestHandler
print(f"Starting on port {port}", flush=True)
http.server.ThreadingHTTPServer(("0.0.0.0", port), handler).serve_forever()
```

ধাপ ২: আলাদা system user তৈরি

সার্ভিস কখনো root হিসেবে চালানো উচিত না। একটা লগইন-অক্ষম সিস্টেম ইউজার বানাও (অধ্যায় ১৫):

```
$ sudo useradd --system --no-create-home --shell /usr/sbin/nologin hello
$ sudo chown -R hello:hello /opt/hello
```

ধাপ ৩: unit ফাইল লেখা

```
$ sudo nano /etc/systemd/system/hello.service
```

```
[Unit]
Description=Hello Python web app
Documentation=https://example.com/hello
After=network-online.target
Wants=network-online.target

[Service]
Type=exec
User=hello
Group=hello
WorkingDirectory=/opt/hello
Environment="PORT=8080"
ExecStart=/usr/bin/python3 /opt/hello/app.py
Restart=on-failure
RestartSec=5

# নিরাপত্তা (sandboxing)
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=yes

[Install]
WantedBy=multi-user.target
```

ধাপ ৪: যাচাই, লোড, চালু

```
$ sudo systemd-analyze verify /etc/systemd/system/hello.service
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now hello
Created symlink /etc/systemd/system/multi-user.target.wants/hello.service →
/etc/systemd/system/hello.service.

$ systemctl status hello --no-pager
● hello.service - Hello Python web app
     Loaded: loaded (/etc/systemd/system/hello.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-11-12 10:02:41 +06; 3s ago
   Main PID: 4821 (python3)
      Tasks: 1 (limit: 9318)
     Memory: 9.8M (peak: 10.1M)
        CPU: 52ms
     CGroup: /system.slice/hello.service
             └─4821 /usr/bin/python3 /opt/hello/app.py

Nov 12 10:02:41 server1 systemd[1]: Started hello.service - Hello Python web app.
Nov 12 10:02:41 server1 python3[4821]: Starting on port 8080

$ curl -sI http://localhost:8080 | head -1
HTTP/1.0 200 OK
```

`verify` কোনো সমস্যা না পেলে কিছুই প্রিন্ট করে না।

ধাপ ৫: auto-restart পরীক্ষা

```
$ sudo kill -9 4821
$ sleep 6; systemctl status hello --no-pager | head -3
● hello.service - Hello Python web app
     Loaded: loaded (/etc/systemd/system/hello.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-11-12 10:04:12 +06; 1s ago

$ journalctl -u hello -n 5 --no-pager
Nov 12 10:04:07 server1 systemd[1]: hello.service: Main process exited, code=killed,
status=9/KILL
Nov 12 10:04:07 server1 systemd[1]: hello.service: Failed with result 'signal'.
Nov 12 10:04:12 server1 systemd[1]: hello.service: Scheduled restart job, restart
counter is at 1.
Nov 12 10:04:12 server1 systemd[1]: Started hello.service - Hello Python web app.
```

process মারা যাওয়ার ৫ সেকেন্ড পর (`RestartSec=5`) systemd নিজে থেকেই নতুন PID-এ আবার চালু করেছে।

> 💡 **টিপস:** `print()` -এ `flush=True` দিয়েছি কারণ Python stdout pipe-এ গেলে buffer করে রাখে, ফলে লগ দেরিতে আসে। বিকল্প: `Environment="PYTHONUNBUFFERED=1"` ।

> 📝 **মনে রাখো:** `After=network-online.target` একা যথেষ্ট না; ক্রম ঠিক করে শুধু, target-কে টেনে আনে না। তাই সাথে `Wants=network-online.target` দিতে হয়। কেন, সেটা পরের অংশে।

একটা oneshot উদাহরণ

বুটে একবার কোনো স্ক্রিপ্ট চালাতে:

```
# /etc/systemd/system/boot-note.service
[Unit]
Description=Write boot time to a file

[Service]
Type=oneshot
ExecStart=/bin/sh -c 'date >> /var/log/boot-times.log'
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

> ⚠️ **সতর্কতা:** `ExecStart=` কোনো শেল না। `>>`, `|`, `&&`, `*` এর মতো শেল বৈশিষ্ট্য সরাসরি কাজ করে না। দরকার হলে উপরের মতো `/bin/sh -c '...'` দিয়ে মোড়াও, অথবা একটা আলাদা স্ক্রিপ্ট ফাইল বানাও। তবে `$VAR` ও `${VAR}` ভেরিয়েবল systemd নিজেই বদলে দেয়।

২৬.৯ Override: `systemctl edit` দিয়ে drop-in

ধরো nginx-এর unit-এ `Restart=` নেই, কিন্তু তুমি চাও crash করলে আবার চালু হোক। মূল ফাইল এডিট করা নিষেধ (আপডেটে মুছে যাবে)। সমাধান: **drop-in** ফাইল। এটা মূল ফাইলের উপরে শুধু তোমার পরিবর্তনগুলো "চাপিয়ে" দেয়।

```
$ sudo systemctl edit nginx
```

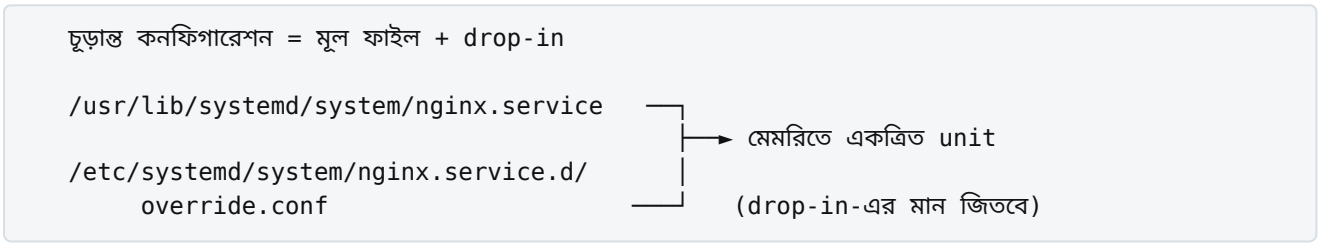
একটা এডিটর খুলবে (Ubuntu-তে সাধারণত nano; `SYSTEMD_EDITOR` বা `EDITOR` ভেরিয়েবল দিয়ে বদলানো যায়)। মাঝখানে লেখার জায়গা থাকবে, নিচে মন্তব্য হিসেবে মূল ফাইল:

```
### Editing /etc/systemd/system/nginx.service.d/override.conf
### Anything between here and the comment below will become the contents of the drop-in
file

[Service]
Restart=on-failure
RestartSec=3
```

```
### Edits below this comment will be discarded
...
```

সংরক্ষণ করে বেরোলে systemd নিজেই `/etc/systemd/system/nginx.service.d/override.conf` তৈরি করে এবং `daemon-reload` করে। এরপর শুধু restart:

```
$ sudo systemctl restart nginx
$ systemctl cat nginx
# /usr/lib/systemd/system/nginx.service
[Unit]
Description=A high performance web server and a reverse proxy server
...

# /etc/systemd/system/nginx.service.d/override.conf
[Service]
Restart=on-failure
RestartSec=3
```



list-ধরনের key রিসেট করা

কিছু key (যেমন `ExecStart=`, `Environment=`, `After=`) একাধিকবার লেখা যায়, আর প্রতিবার **যোগ** হয়। তাই `ExecStart=` বদলাতে হলে প্রথমে খালি `ExecStart=` দিয়ে পুরোনোটা মুছতে হয়:

```
[Service]
ExecStart=
ExecStart=/usr/sbin/nginx -g 'daemon on; master_process on; worker_processes 2;'
```

খালি লাইনটা না দিলে, `Type=simple` / `exec` সার্ভিসে দুটো `ExecStart` থাকার কারণে ভুল দেখাবে: `Service has more than one ExecStart= setting, which is only allowed for Type=oneshot services.`

অন্যান্য edit অপশন

কমান্ড	কাজ
`systemctl edit UNIT`	`override.conf` drop-in তৈরি/এডিট
`systemctl edit --full UNIT`	মূল ফাইলের একটা পূর্ণ কপি `/etc/systemd/system/`-এ বানিয়ে এডিট (প্যাকেজের ভবিষ্যৎ পরিবর্তন আর পাবে না)
`systemctl edit --drop-in=NAME UNIT`	ভিন্ন নামের drop-in ফাইল

কমান্ড	কাজ
`systemctl edit --force --full NEW.service`	একদম নতুন unit ফাইল তৈরি
`systemctl edit --runtime UNIT`	`/run`-এ অস্থায়ী override (রিবুটে মুছে যায়)
`systemctl revert UNIT`	`/etc` ও `/run`-এর সব override মুছে প্যাকেজের মূলে ফেরা
`systemd-delta`	সিস্টেমের সব override/masked/extended unit-এর তালিকা

```
$ systemd-delta --type=extended
[EXTENDED]   /usr/lib/systemd/system/nginx.service →
/etc/systemd/system/nginx.service.d/override.conf

1 overridden configuration files found.
```

> 💡 **টিপস:** drop-in ফোল্ডারে একাধিক `.conf` ফাইল থাকলে সেগুলো নামের বর্ণানুক্রমে পড়া হয়। তাই `10-restart.conf`, `20-limits.conf` এভাবে নাম দিলে ক্রম নিয়ন্ত্রণ সহজ।

২৬.১০ Dependency: Wants, Requires, After ও অন্যরা

systemd-এ দুটো সম্পূর্ণ আলাদা প্রশ্ন আছে, আর এই পার্থক্য বোঝাই এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ ধারণাগুলোর একটা:

1. **requirement (প্রয়োজন):** A চালু হলে B-কেও চালু করতে হবে কি? → `Wants=`, `Requires=`
2. **ordering (ক্রম):** দুটোই চালু হলে কে আগে? → `After=`, `Before=`

এ দুটো একে অপরের থেকে **স্বাধীন**।

```
শুধু Requires=B (After নেই):    A আর B একসাথে (সমান্তরালে) চালু হয়
                                   A ███
                                   B ███

Requires=B + After=B:              B পুরো চালু হওয়ার পর A
                                   B ███
                                   A    ███

শুধু After=B (Requires নেই):     B-কে কেউ চালু না করলে A একাই চলে;
                                   B-ও চালু হলে, তবে B আগে
```

> 💡 **উপমা:** বিয়ের অনুষ্ঠান। "বাবুর্চি লাগবেই" = Requires। "ফটোগ্রাফার থাকলে ভালো, না এলেও অনুষ্ঠান চলবে" = Wants। "রান্না শেষ হওয়ার পর খাবার পরিবেশন" = After। বাবুর্চি ডাকা আর কখন পরিবেশন হবে,

> দুটো আলাদা সিদ্ধান্ত।

requirement directive

Directive	অর্থ	B ব্যর্থ হলে A?	B থামানো হলে A?
`Wants=B`	B-কে চালু করার চেষ্টা করো	A চলবে	A চলবে
`Requires=B`	B অবশ্যই লাগবে	A চালু হবে না (যদি `After=B` -ও থাকে)	A-ও থামবে
`Requisite=B`	B আগে থেকেই চালু থাকতে হবে, নিজে চালু করবে না	A চালু হবে না	—
`BindsTo=B`	Requires-এর চেয়েও শক্ত; B হঠাৎ inactive হলেও A থামবে	A চালু হবে না	A-ও থামবে
`PartOf=B`	B stop/restart হলে A-ও stop/restart হবে (একমুখী, B-কে চালু করে না)	—	A-ও থামবে
`Conflicts=B`	A চালু হলে B থামবে, আর উল্টোটা	—	—

ordering directive

Directive	অর্থ
`After=B`	B চালু হওয়া শেষ হলে তবেই A চালু শুরু; বন্ধের সময় উল্টো ক্রম (A আগে বন্ধ)
`Before=B`	A চালু শেষ হলে তবেই B

> 📝 **মনে রাখো:** systemd-এর ডকুমেন্টেশন নিজেই বলে, বেশিরভাগ ক্ষেত্রে `Wants=` + `After=` ব্যবহার করাই সবচেয়ে ভালো। `Requires=` সিস্টেমকে ভঙ্গুর করে: একটা ছোট সার্ভিস ব্যর্থ হলে পুরো চেইন পড়ে যায়।

> 📝 **মনে রাখো:** বেশিরভাগ service unit-এ systemd স্বয়ংক্রিয়ভাবে কিছু dependency যোগ করে (`DefaultDependencies=yes`): যেমন `Requires=sysinit.target` , `After=sysinit.target basic.target` , `Conflicts=shutdown.target` , `Before=shutdown.target` । তাই সাধারণ সার্ভিসে এগুলো নিজে লিখতে হয় না।

network.target বনাম network-online.target

Target	অর্থ
`network.target`	নেটওয়ার্ক ম্যানেজমেন্ট সফটওয়্যার চালু হয়েছে; IP পাওয়া গেছে এমন নিশ্চয়তা **নেই**। মূলত শাটডাউনের সময় নেটওয়ার্ক শেষে বন্ধ হওয়ার জন্য
`network-online.target`	নেটওয়ার্ক কনফিগার হয়ে সত্যিই ব্যবহারযোগ্য (`NetworkManager-wait-online` বা `systemd-networkd-wait-online` দিয়ে নির্ধারিত)

তোমার অ্যাপ চালু হওয়ার সময়ই কোনো দূরের ডাটাবেসে সংযোগ করতে চাইলে `Wants=network-online.target` + `After=network-online.target` দাও।

dependency দেখা

```
$ systemctl list-dependencies hello
hello.service
● ├─system.slice
● ├─network-online.target
● │ └─NetworkManager-wait-online.service
● └─sysinit.target
●   ├─apparmor.service
●   ├─dev-hugepages.mount
...

$ systemctl show hello -p Wants -p After
Wants=network-online.target
After=network-online.target sysinit.target basic.target system.slice systemd-journald.socket
```

২৬.১১ User service: `systemctl --user`

systemd প্রতিটি লগইন করা ইউজারের জন্য একটা নিজস্ব systemd instance (`user@UID.service`) চালায়। এতে sudo ছাড়াই নিজের সার্ভিস চালানো যায়।

```
$ mkdir -p ~/.config/systemd/user
$ nano ~/.config/systemd/user/notes-sync.service
```

```
[Unit]
Description=Sync my notes folder

[Service]
Type=oneshot
ExecStart=/usr/bin/rsync -a %h/notes/ %h/backup/notes/
```

(`%h` = ইউজারের home ফোল্ডার। আরও specifier: `%u` = ইউজারনেম, `%i` = instance নাম, `%n` = পুরো unit নাম।)

```
$ systemctl --user daemon-reload
$ systemctl --user start notes-sync
$ systemctl --user status notes-sync
$ journalctl --user -u notes-sync
```

> ⚠️ **সতর্কতা:** ডিফল্টভাবে ইউজার লগআউট করলে তার user systemd instance (এবং সব user service) বন্ধ হয়ে যায়। সার্ভারে লগআউটের পরও চালু রাখতে **lingering** চালু করো:
>
> ```
> $ sudo loginctl enable-linger $USER
> $ loginctl show-user $USER -p Linger
> Linger=yes
> ```

> 💡 **টিপস:** user unit-এ `WantedBy=default.target` দাও (user instance-এ `multi-user.target` নেই)।

২৬.১২ রিসোর্স সীমা ও নিরাপত্তা (sandboxing)

প্রতিটি সার্ভিস নিজের cgroup-এ চলে বলে systemd সহজেই সীমা বসাতে পারে।

```
[Service]
MemoryMax=512M        # এর বেশি মেমরি চাইলে OOM kill
CPUQuota=50%          # একটা core-এর সর্বোচ্চ ৫০%
TasksMax=100          # সর্বোচ্চ ১০০ process/thread
LimitNOFILE=65536     # খোলা ফাইলের সীমা (ulimit -n)
```

লাইভ সীমা বসানো (unit ফাইল না বদলে):

```
$ sudo systemctl set-property hello MemoryMax=256M
```

এটা `/etc/systemd/system.control/hello.service.d/` এ একটা drop-in লিখে রাখে, তাই রিবুটের পরেও টিকে থাকে (`--runtime` দিলে টিকে থাকে না)।

cgroup অনুযায়ী লাইভ রিসোর্স ব্যবহার দেখতে:

```
$ systemd-cgtop          # top-এর মতো, কিন্তু সার্ভিস অনুযায়ী
$ systemd-cgls           # cgroup গাছ: কোন সার্ভিসের অধীনে কোন process
```

নিরাপত্তা directive (বিস্তারিত `man systemd.exec`):

Directive	কাজ
`NoNewPrivileges=yes`	process ও তার child-রা setuid ইত্যাদি দিয়ে নতুন ক্ষমতা পাবে না

Directive	কাজ
`PrivateTmp=yes`	নিজস্ব আলাদা `/tmp` ও `/var/tmp`
`ProtectSystem=strict`	পুরো ফাইলসিস্টেম read-only (শুধু `/dev`, `/proc`, `/sys` বাদে); লিখতে হলে `ReadWritePaths=`
`ProtectSystem=full`	`/usr`, `/boot`, `/etc` read-only
`ProtectHome=yes`	`/home`, `/root`, `/run/user` অদৃশ্য
`PrivateDevices=yes`	আসল হার্ডওয়্যার ডিভাইস অদৃশ্য
`DynamicUser=yes`	চলার সময় একটা অস্থায়ী ইউজার তৈরি করে চালায়
`StateDirectory=NAME`	`/var/lib/NAME` তৈরি করে সার্ভিসের ইউজারকে মালিক বানায়
`CapabilityBoundingSet=`	কোন Linux capability থাকবে

সার্ভিস কতটা সুরক্ষিত, তার একটা নম্বর পাবে:

```
$ systemd-analyze security hello
  NAME                        DESCRIPTION                                     EXPOSURE
✓ NoNewPrivileges=            Service processes cannot acquire new privileges
✓ PrivateTmp=                 Service has no access to other software's temporary
files
✗ PrivateDevices=             Service potentially has access to hardware devices
0.2
...
→ Overall exposure level for hello.service: 6.3 MEDIUM 😐
```

(নম্বর ০ = সবচেয়ে নিরাপদ, ১০ = সবচেয়ে অরক্ষিত। আসল সংখ্যা তোমার unit ও systemd সংস্করণভেদে আলাদা হবে।) নিরাপত্তা নিয়ে আরও অধ্যায় ৪৫-এ।

২৬.১৩ systemd-analyze: বুট বিশ্লেষণ ও যাচাই

```
$ systemd-analyze
Startup finished in 4.120s (firmware) + 2.310s (loader) + 1.902s (kernel) + 6.744s
(userspace) = 15.077s
graphical.target reached after 6.701s in userspace.
```

(`systemd-analyze` আর `systemd-analyze time` একই। firmware/loader অংশ শুধু UEFI সিস্টেমে দেখায়।)

কোন সার্ভিস সবচেয়ে বেশি সময় নিল:

```
$ systemd-analyze blame
3.211s NetworkManager-wait-online.service
1.104s snapd.service
```

```
 812ms dev-nvme0n1p2.device
 640ms systemd-journal-flush.service
 402ms apparmor.service
...
```

> ⚠️ **সতর্কতা:** `blame`-এ বেশি সময় মানেই বুট ধীর হওয়ার জন্য দায়ী, এমন নয়। অনেক সার্ভিস সমান্তরালে চলে, আর কেউ কেউ শুধু অপেক্ষা করে। আসল বাধা খুঁজতে `critical-chain` দেখো।

critical chain: যে ক্রমিক শিকলটা শেষ target পৌঁছাতে দেরি করিয়েছে:

```
$ systemd-analyze critical-chain
The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

graphical.target @6.701s
└─multi-user.target @6.700s
  └─snapd.seeded.service @5.420s +1.270s
    └─snapd.service @4.300s +1.104s
      └─basic.target @1.250s
        └─sockets.target @1.249s
...
```

অন্যান্য দরকারি subcommand:

কমান্ড	কাজ
`systemd-analyze time`	মোট বুট সময়
`systemd-analyze blame`	প্রতি unit-এর start সময় (বেশি থেকে কম)
`systemd-analyze critical-chain [UNIT]`	সময়-নির্ণায়ক শিকল
`systemd-analyze plot > boot.svg`	পুরো বুটের একটা SVG টাইমলাইন ছবি (ব্রাউজারে খোলো)
`systemd-analyze dot UNIT | dot -Tsvg > g.svg`	dependency গ্রাফ (graphviz লাগে)
`systemd-analyze verify FILE`	unit ফাইলের ভুল খোঁজো
`systemd-analyze security [UNIT]`	নিরাপত্তা স্কোর
`systemd-analyze calendar 'Mon *-*-* 09:00'`	timer-এর সময় এক্সপ্রেশন যাচাই (অধ্যায় ২৯)
`systemd-analyze timespan 1h30m`	সময়ের মান কীভাবে পড়া হবে
`systemd-analyze unit-paths`	unit ফাইল খোঁজার সব ফোল্ডার
`systemd-analyze cat-config systemd/journald.conf`	কনফিগ ফাইল ও তার drop-in একসাথে

```
$ systemd-analyze calendar 'Mon..Fri 09:00'
  Original form: Mon..Fri 09:00
Normalized form: Mon..Fri *-*-* 09:00:00
    Next elapse: Fri 2026-11-13 09:00:00 +06
```

```
       (in UTC): Fri 2026-11-13 03:00:00 UTC
       From now: 22h left
```

> 💡 **টিপস:** বুট ধীর হলে প্রায়ই অপরাধী হয় `NetworkManager-wait-online.service` বা `systemd-networkd-wait-online.service`। ডেস্কটপে কোনো সার্ভিস নেটওয়ার্কের জন্য অপেক্ষা না করলে এটা disable করা যায়, তবে সার্ভারে সাবধান: যে সার্ভিসগুলো `network-online.target`-এর উপর নির্ভর করে, তারা আগেভাগে চালু হয়ে ব্যর্থ হতে পারে।

২৬.১৪ hostnamectl, timedatectl, localectl

systemd কিছু সিস্টেম সেটিং বদলানোর জন্য ছোট ছোট daemon (hostnamed, timedated, localed) ও তাদের কমান্ড দেয়। এগুলো সঠিক কনফিগ ফাইলে লেখে এবং চলমান সিস্টেমকেও সাথে সাথে জানায়, তাই ফাইল নিজে এডিট করার চেয়ে নিরাপদ।

hostnamectl: কম্পিউটারের নাম

```
$ hostnamectl
 Static hostname: server1
       Icon name: computer-vm
         Chassis: vm 🖴
      Machine ID: 3f2a9c0d1e5b4c7a8f6e2d1c0b9a8f7e
         Boot ID: 9b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e
  Virtualization: kvm
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-48-generic
    Architecture: x86-64
```

hostname তিন ধরনের:

ধরন	অর্থ	কোথায় থাকে
static	স্থায়ী নাম, বুটে সেট হয়	`/etc/hostname`
transient	চলার সময় DHCP ইত্যাদি থেকে আসা অস্থায়ী নাম	kernel
pretty	মানুষের জন্য সুন্দর নাম, স্পেস/ইউনিকোড চলে	`/etc/machine-info`

```
$ sudo hostnamectl set-hostname web01
$ sudo hostnamectl set-hostname "নিশাতের ল্যাপটপ" --pretty
$ hostnamectl hostname
web01
```

> ⚠️ **সতর্কতা:** hostname বদলানোর পর `/etc/hosts` ফাইলে পুরোনো নামটা থাকলে (Ubuntu-তে সাধারণত `127.0.1.1 oldname` লাইন) সেটাও বদলে দাও, নইলে `sudo` চালাতে দেরি হবে ও `unable`

> `to resolve host` সতর্কবার্তা দেখাবে। নতুন নাম শেলের prompt-এ দেখতে নতুন টার্মিনাল খোলো।

timedatectl: সময়, তারিখ ও timezone

```
$ timedatectl
               Local time: Thu 2026-11-12 10:30:15 +06
           Universal time: Thu 2026-11-12 04:30:15 UTC
                 RTC time: Thu 2026-11-12 04:30:15
                Time zone: Asia/Dhaka (+06, +0600)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
```

কমান্ড	কাজ
`timedatectl list-timezones`	সব timezone (`grep Asia` দিয়ে ছাঁকো)
`sudo timedatectl set-timezone Asia/Dhaka`	timezone সেট
`sudo timedatectl set-ntp true`	NTP দিয়ে স্বয়ংক্রিয় সময় মেলানো চালু
`sudo timedatectl set-ntp false`	বন্ধ
`sudo timedatectl set-time '2026-11-12 10:30:00'`	হাতে সময় সেট (NTP বন্ধ থাকলেই কেবল কাজ করে)
`timedatectl timesync-status`	systemd-timesyncd-এর বিস্তারিত (শুধু timesyncd ব্যবহার হলে)
`sudo timedatectl set-local-rtc 0`	হার্ডওয়্যার ঘড়ি UTC-তে রাখো (প্রস্তাবিত)

```
$ timedatectl list-timezones | grep -i dhaka
Asia/Dhaka
```

timezone আসলে `/etc/localtime` নামের একটা symlink (`/usr/share/zoneinfo/Asia/Dhaka` -এর দিকে)।

> 📝 **মনে রাখো (distro পার্থক্য):** Ubuntu ও Debian ডিফল্টভাবে NTP-র জন্য `systemd-timesyncd` ব্যবহার করে; Fedora ও RHEL ব্যবহার করে `chronyd` (`chronyc tracking` দিয়ে দেখো)। `timedatectl set-ntp` দুটোর সাথেই কাজ করে।

> 💡 **টিপস:** Windows ও Linux dual-boot করলে ঘড়ি ৬ ঘণ্টা আগে-পিছে হয়ে যায়, কারণ Windows হার্ডওয়্যার ঘড়িকে local time ধরে আর Linux ধরে UTC। সবচেয়ে ভালো সমাধান Windows-কে UTC ব্যবহার করানো; সহজ সমাধান Linux-এ `sudo timedatectl set-local-rtc 1` (তবে systemd এতে সতর্কবার্তা দেয়)।

localectl: ভাষা ও কীবোর্ড

```
$ localectl
System Locale: LANG=en_US.UTF-8
    VC Keymap: us
   X11 Layout: us
```

কমান্ড	কাজ
`localectl list-locales`	উপলব্ধ (জেনারেট করা) locale
`sudo localectl set-locale LANG=bn_BD.UTF-8`	সিস্টেমের ভাষা
`localectl list-keymaps`	কনসোলের কীবোর্ড ম্যাপ
`sudo localectl set-keymap us`	কনসোল (VC) কীবোর্ড
`sudo localectl set-x11-keymap us,bd pc105 "" grp:alt_shift_toggle`	GUI কীবোর্ড: দুটি লেআউট, Alt+Shift দিয়ে বদল

> 📝 **মনে রাখো:** Debian/Ubuntu-তে কোনো locale `localectl list-locales`-এ না থাকলে আগে সেটা জেনারেট করতে হয়: `sudo locale-gen bn_BD.UTF-8` (বা `sudo dpkg-reconfigure locales`)। Fedora/RHEL-এ ভাষা প্যাক ইনস্টল করো: `sudo dnf install glibc-langpack-bn`। সেটিং থাকে Debian/Ubuntu-তে `/etc/default/locale`, Fedora/RHEL-এ `/etc/locale.conf`।

> 💡 **টিপস:** সার্ভারে সিস্টেম locale `en_US.UTF-8` বা `C.UTF-8` রাখাই ভালো; বাংলা locale দিলে error বার্তা বাংলায় আসে আর ইন্টারনেটে সমাধান খোঁজা কঠিন হয়। ডেস্কটপে নিজের পছন্দ মতো রাখো।

অন্যান্য একই ধরনের `*ctl` টুল: `loginctl` (লগইন সেশন), `networkctl` (systemd-networkd), `resolvectl` (DNS; অধ্যায় ৩৭), `journalctl` (অধ্যায় ২৮), `coredumpctl` (crash dump), `bootctl` (systemd-boot)।

২৬.১৫ সমস্যা সমাধানের কর্মপদ্ধতি

সার্ভিস চালু হচ্ছে না? এই ক্রমে দেখো:

```
১. systemctl status myapp           ← অবস্থা ও শেষ লগ
        │
২. journalctl -xeu myapp            ← পুরো লগ, ব্যাখ্যা সহ, শেষ থেকে
        │
৩. systemctl cat myapp              ← কার্যকর কনফিগ কী (override সহ)?
        │
৪. systemd-analyze verify ...       ← সিনট্যাক্স ভুল?
        │
```

```
৫. হাতে চালিয়ে দেখো:                    ← একই ইউজার হিসেবে
   sudo -u hello /usr/bin/python3 /opt/hello/app.py
        |
৬. ঠিক করে: daemon-reload → reset-failed → restart
```

সাধারণ ভুল ও তার মানে:

বার্তা / লক্ষণ	সম্ভাব্য কারণ
`Unit myapp.service not found.`	নামে ভুল, ফাইল ভুল ফোল্ডারে, বা `daemon-reload` হয়নি
`status=203/EXEC`	`ExecStart`-এর প্রোগ্রাম পাওয়া যায়নি বা executable না (`chmod +x`, absolute path দাও, স্ক্রিপ্টে shebang আছে কি না দেখো)
`status=217/USER`	`User=`-এ দেওয়া ইউজার নেই
`status=200/CHDIR`	`WorkingDirectory` নেই বা অ্যাক্সেস নেই
`status=226/NAMESPACE`	sandbox directive-এ দেওয়া path নেই
`Permission denied`	ফাইলের মালিকানা/পারমিশন (অধ্যায় ১৬), বা SELinux/AppArmor
`start request repeated too quickly`	বারবার crash; `reset-failed` করে আসল কারণ খোঁজো
active (exited) কিন্তু আসলে চলছে না	প্রোগ্রাম নিজে background-এ চলে গেছে অথচ `Type=simple` দেওয়া; `Type=forking` লাগবে বা foreground মোড দাও
timeout হয়ে failed	`Type=notify` কিন্তু প্রোগ্রাম READY পাঠায় না; বা `forking` কিন্তু parent exit করে না

২৬.১৬ কীবোর্ড শর্টকাট

`systemctl status`, `list-units`, `journalctl` এর আউটপুট লম্বা হলে **less** pager-এ খোলে। সেখানে এই কী-গুলো কাজে লাগে (less নিয়ে বিস্তারিত অধ্যায় ১০):

শর্টকাট	কাজ
`q`	pager থেকে বের হও
`Space` / `PageDown`	এক পাতা নিচে
`b` / `PageUp`	এক পাতা উপরে
`↓` / `j`, `↑` / `k`	এক লাইন নিচে/উপরে
`→` / `←`	লম্বা লাইনে ডানে/বামে স্ক্রল (systemctl লাইন কাটে না, ডানে সরায়)
`g` / `G`	একদম শুরুতে / শেষে
`/শব্দ`	নিচের দিকে খোঁজো; `n` পরেরটা, `N` আগেরটা
`?শব্দ`	উপরের দিকে খোঁজো
`Ctrl+C`	`journalctl -f` বা `status` এর লাইভ অনুসরণ থামাও

শর্টকাট	কাজ
`Tab`	unit নাম, subcommand auto-complete (bash-completion প্যাকেজ লাগে)

সিস্টেম-স্তরের শর্টকাট:

শর্টকাট	কাজ
`Ctrl+Alt+Del` (কনসোলে)	`ctrl-alt-del.target` চালু করে, ডিফল্টভাবে রিবুট; ২ সেকেন্ডের মধ্যে ৭ বার চাপলে তাৎক্ষণিক জোরপূর্বক রিবুট
`Ctrl+Alt+F1` … `F6`	ভিন্ন virtual terminal (প্রতিটির পিছনে `getty@ttyN.service`)
GRUB-এ `e`	বুট এন্ট্রি এডিট (যেমন `systemd.unit=rescue.target` যোগ)
GRUB এডিটে `Ctrl+X` / `F10`	এডিট করা এন্ট্রিতে বুট
`systemctl edit`-এ nano: `Ctrl+O`, `Enter`, `Ctrl+X`	সংরক্ষণ ও বের হওয়া (অধ্যায় ১৩)

> ⚠️ **সতর্কতা:** সার্ভারে কেউ যেন ভুল করে Ctrl+Alt+Del চেপে রিবুট না করে, সেজন্য অনেকে এটা mask করে: `sudo systemctl mask ctrl-alt-del.target`।

> 💡 **টিপস:** pager একেবারেই না চাইলে `--no-pager` দাও, অথবা `export SYSTEMD_PAGER=` সেট করো। স্থায়ীভাবে রাখতে `.bashrc`-এ লেখো (অধ্যায় ৪৪)। আর `systemctl`-কে বারবার লিখতে বিরক্ত লাগলে alias: `alias sc='sudo systemctl'`।

হাতে-কলমে অনুশীলন

> 💡 একটা VM বা পরীক্ষামূলক মেশিনে করো (অধ্যায় ৩), প্রোডাকশন সার্ভারে নয়।

ল্যাব ১: সিস্টেম ঘুরে দেখা

```
$ ps -p 1 -o comm=
$ systemctl --version | head -1
$ systemctl get-default
$ systemctl --failed
$ systemctl list-units --type=service --state=running --no-pager | wc -l
$ systemctl list-unit-files --state=enabled --no-pager
$ systemctl list-timers --no-pager
```

প্রশ্ন: কয়টা সার্ভিস চলছে? কোন timer সবার আগে চলবে?

ল্যাব ২: একটা সার্ভিসের পুরো জীবনচক্র

```
$ sudo apt install -y nginx          # Fedora/RHEL: sudo dnf install -y nginx
$ systemctl is-enabled nginx         # Ubuntu-তে enabled, Fedora-তে disabled
$ sudo systemctl enable --now nginx
$ curl -sI localhost | head -1
$ sudo systemctl stop nginx; systemctl is-active nginx
$ sudo systemctl mask nginx; sudo systemctl start nginx    # ব্যর্থ হবে
$ sudo systemctl unmask nginx; sudo systemctl start nginx
$ ls -l /etc/systemd/system/multi-user.target.wants/ | grep nginx
```

ল্যাব ৩: নিজের সার্ভিস

২৬.৮-এর `hello.service` পুরোটা তৈরি করো। তারপর:

1. `kill -9` দিয়ে মেরে দেখো আবার উঠে কি না।
2. `app.py`-তে ইচ্ছে করে সিনট্যাক্স ভুল ঢুকিয়ে restart করো; `journalctl -u hello` দেখে ভুল খোঁজো। `start request repeated too quickly` দেখা পর্যন্ত অপেক্ষা করো। ঠিক করে `reset-failed` ও `start`।
3. `systemd-analyze security hello` চালাও, তারপর `PrivateDevices=yes` যোগ করে আবার চালাও; স্কোর কমলো কি না দেখো।

ল্যাব ৪: override

```
$ sudo systemctl edit hello
```

ভেতরে লেখো:

```
[Service]
Environment="PORT=9090"
```

```
$ sudo systemctl restart hello
$ systemctl cat hello
$ curl -sI localhost:9090 | head -1
$ systemd-delta --type=extended
$ sudo systemctl revert hello && sudo systemctl restart hello
$ curl -sI localhost:8080 | head -1        # আবার পুরোনো port-এ
```

`Environment=` একাধিকবার দিলে পরেরটা একই ভেরিয়েবলের আগের মান বদলে দেয়, তাই এখানে খালি লাইন লাগেনি।

ল্যাব ৫: dependency পরীক্ষা

দুটো oneshot সার্ভিস বানাও: `a.service` ও `b.service`, প্রতিটি `ExecStart=/bin/sh -c 'echo X; sleep 3'`। `a`-তে প্রথমে শুধু `Wants=b.service` দিয়ে `sudo systemctl start a`, `journalctl -u`

`a -u b` দেখে লক্ষ করো দুটো একসাথে শুরু হলো। তারপর `After=b.service` যোগ করে (daemon-reload) আবার চালাও; এবার `b` শেষ হওয়ার পর `a` শুরু হবে।

ল্যাব ৬: বুট বিশ্লেষণ ও সিস্টেম সেটিং

```
$ systemd-analyze
$ systemd-analyze blame | head -5
$ systemd-analyze critical-chain
$ systemd-analyze plot > ~/boot.svg      # ফাইলটা ব্রাউজারে খোলো
$ hostnamectl
$ sudo hostnamectl set-hostname lab01 && hostnamectl hostname
$ timedatectl
$ sudo timedatectl set-timezone Asia/Dhaka
$ localectl
```

ল্যাব ৭: user service ও target

1. ২৬.১১-এর `notes-sync.service` তৈরি করে `systemctl --user` দিয়ে চালাও।
2. VM-এ `sudo systemctl isolate multi-user.target` চালিয়ে দেখো GUI বন্ধ হয় কি না, তারপর `sudo systemctl isolate graphical.target` (SSH-এ করবে না)।

সারসংক্ষেপ

কমান্ড	কাজ
`systemctl status UNIT`	অবস্থা, PID, মেমরি, শেষ লগ
`systemctl start / stop UNIT`	এখনই চালু / বন্ধ
`systemctl restart UNIT`	বন্ধ করে আবার চালু
`systemctl reload UNIT`	process না থামিয়ে কনফিগ পড়াও
`systemctl enable / disable UNIT`	বুটে চালু হবে / হবে না (symlink)
`systemctl enable --now UNIT`	enable + start
`systemctl is-active / is-enabled / is-failed UNIT`	এক-শব্দের উত্তর, স্ক্রিপ্টের জন্য
`systemctl mask / unmask UNIT`	সম্পূর্ণ নিষিদ্ধ / নিষেধ তোলা
`systemctl list-units [--type=service] [--state=…]`	লোড হওয়া unit
`systemctl list-unit-files`	ডিস্কের সব unit ও enable অবস্থা
`systemctl --failed`	ব্যর্থ unit
`systemctl cat / show UNIT`	unit ফাইল / property
`systemctl list-dependencies UNIT`	dependency গাছ
`systemctl daemon-reload`	unit ফাইল বদলানোর পর আবার পড়াও

কমান্ড	কাজ
`systemctl edit UNIT`	drop-in override
`systemctl edit --full UNIT`	পূর্ণ কপি এডিট
`systemctl revert UNIT`	override মুছে ফেলো
`systemctl reset-failed`	failed অবস্থা মুছে দাও
`systemctl get-default / set-default TARGET`	ডিফল্ট target
`systemctl isolate TARGET`	এখনই target বদলাও
`systemctl --user …`	নিজের user service
`systemd-analyze [blame / critical-chain / plot]`	বুট সময় বিশ্লেষণ
`systemd-analyze verify / security`	unit যাচাই / নিরাপত্তা স্কোর
`systemd-delta`	override-এর তালিকা
`systemd-cgtop`, `systemd-cgls`	cgroup অনুযায়ী রিসোর্স / process গাছ
`journalctl -u UNIT [-f]`	সার্ভিসের লগ
`hostnamectl [set-hostname NAME]`	hostname
`timedatectl [set-timezone TZ / set-ntp true]`	সময় ও timezone
`localectl [set-locale / set-keymap]`	ভাষা ও কীবোর্ড
`loginctl enable-linger USER`	লগআউটের পরও user service চালু

ফাইল/ফোল্ডার	ভূমিকা
`/usr/lib/systemd/system/` (= `/lib/...`)	প্যাকেজের unit (এডিট কোরো না)
`/etc/systemd/system/`	অ্যাডমিনের unit ও override (সর্বোচ্চ অগ্রাধিকার)
`/etc/systemd/system/UNIT.d/*.conf`	drop-in
`/etc/systemd/system/*.target.wants/`	enable করা unit-এর symlink
`~/.config/systemd/user/`	user unit

নিজেকে যাচাই করো

1. PID 1 কী, এবং কেন এটা মারা গেলে পুরো সিস্টেম পড়ে যায়?
2. SysV init-এর তুলনায় systemd বুট দ্রুত করে কোন দুটো প্রধান কৌশলে?
3. `systemctl start` আর `systemctl enable`-এর পার্থক্য কী? `enable` ভেতরে আসলে কী করে?
4. `reload`, `restart` আর `daemon-reload` তিনটা কখন ব্যবহার করবে?
5. `disable` আর `mask`-এর পার্থক্য কী? mask করলে ফাইলসিস্টেমে কী তৈরি হয়?
6. runlevel 3 ও 5-এর systemd সমতুল্য target কী? পরের বুট থেকে GUI বন্ধ রাখতে কোন কমান্ড দেবে?

7. `/usr/lib/systemd/system/nginx.service` ও `/etc/systemd/system/nginx.service` দুটোই থাকলে কোনটা কার্যকর হবে? প্যাকেজের ফাইল সরাসরি এডিট করা কেন উচিত নয়?
8. drop-in-এ `ExecStart` বদলাতে কেন আগে একটা খালি `ExecStart=` লিখতে হয়?
9. `Wants=B` দেওয়া আছে কিন্তু `After=B` নেই। A ও B কোন ক্রমে চালু হবে?
10. একটা সার্ভিস `status=203/EXEC` দিয়ে ব্যর্থ হচ্ছে। কী কী পরীক্ষা করবে?

▼ **উত্তর দেখো**

1. kernel যে প্রথম user-space process চালায়, সেটাই PID 1 (init, এখন systemd)। এটা বাকি সব সার্ভিস চালায়, orphan process দত্তক নেয় ও reap করে। PID 1 exit করলে kernel চালিয়ে যাওয়ার উপায় পায় না, তাই kernel panic হয়।
2. (ক) dependency না থাকলে সার্ভিসগুলো সমান্তরালে (parallel) চালানো, (খ) socket ও on-demand activation: socket আগে তৈরি করে রাখা, ফলে সার্ভিস প্রস্তুত হওয়ার আগেই অন্যরা সংযোগ করতে পারে এবং দরকার না হলে সার্ভিস চালুই হয় না।
3. `start` এখনই চালু করে কিন্তু রিবুটে টেকে না; `enable` বুটে চালু হওয়ার ব্যবস্থা করে কিন্তু এখন চালু করে না। `enable` `[Install]`-এর `WantedBy=` পড়ে সংশ্লিষ্ট target-এর `.wants/` ফোল্ডারে (যেমন `/etc/systemd/system/multi-user.target.wants/`) একটা symlink বানায়। দুটো একসাথে: `enable --now`।
4. `reload`: সার্ভিসের নিজের কনফিগ (যেমন nginx.conf) বদলালে, process না থামিয়ে। `restart`: process পুরো নতুন করে চালাতে (বাইনারি আপডেট, reload সমর্থন না করলে)। `daemon-reload`: systemd-এর unit ফাইল বা drop-in বদলালে, যাতে PID 1 নতুন definition পড়ে; এরপর সাধারণত restart-ও লাগে।
5. disable করা সার্ভিস হাতে বা অন্য unit-এর dependency হিসেবে চালু হতে পারে; masked সার্ভিস কোনোভাবেই চালু হয় না। mask করলে `/etc/systemd/system/UNIT` নামে `/dev/null`-এর দিকে একটা symlink তৈরি হয়।
6. runlevel 3 = `multi-user.target`, runlevel 5 = `graphical.target`। কমান্ড: `sudo systemctl set-default multi-user.target`।
7. `/etc/systemd/system/`-এরটা, কারণ এর অগ্রাধিকার সর্বোচ্চ। প্যাকেজের ফাইল এডিট করলে পরের প্যাকেজ আপডেটে পরিবর্তন মুছে যাবে; তাই `systemctl edit` দিয়ে drop-in বানাও।
8. `ExecStart=` একটা list-ধরনের key; প্রতিটি নতুন লাইন আগের মানের সাথে যোগ হয়। খালি `ExecStart=` আগের সব মান মুছে দেয়, তারপর নতুন মান বসে। না করলে simple/exec সার্ভিসে দুটো ExecStart হয়ে যায়, যা অনুমোদিত নয়।
9. কোনো নির্দিষ্ট ক্রম নেই, দুটো সমান্তরালে চালু হবে। `Wants` / `Requires` শুধু "চালু করো" বলে, ক্রম ঠিক করে `After` / `Before`।
10. `ExecStart`-এ দেওয়া path ঠিক ও absolute কি না, ফাইলটা আছে কি না, executable পারমিশন (`chmod +x`) আছে কি না, স্ক্রিপ্ট হলে প্রথম লাইনে সঠিক shebang (`#!/bin/bash`) আছে কি না, `User=`-এর ইউজারের ফাইলটা চালানোর অনুমতি আছে কি না, এবং ফাইলে Windows-এর CRLF লাইন-এন্ডিং আছে কি না। `journalctl -xeu UNIT`-এ বিস্তারিত দেখো।

অধ্যায় ২৭: লিনাক্স বুট প্রসেস — পাওয়ার বাটন থেকে লগইন পর্যন্ত

পাওয়ার বাটন চাপার পর মাত্র কয়েক সেকেন্ডে লগইন স্ক্রিন চলে আসে। কিন্তু এই কয়েক সেকেন্ডের ভেতরে অনেকগুলো প্রোগ্রাম পালা করে কাজ করে। প্রত্যেকে নিজের কাজটুকু সেরে পরের জনকে দায়িত্ব দেয়, রিলে দৌড়ের মতো। এই অধ্যায়ে সেই দৌড়টা শুরু থেকে শেষ পর্যন্ত দেখব।

বুট প্রসেস বোঝা কেন দরকার? যতদিন সব ঠিক চলে, ততদিন দরকার পড়ে না। কিন্তু যেদিন মেশিন বুট হয় না, কালো স্ক্রিনে `grub rescue>` বা `Give root password for maintenance` লেখা আসে, অথবা root পাসওয়ার্ড ভুলে যাও, সেদিন এই জ্ঞানই তোমাকে বাঁচাবে। বুট প্রসেস যে বোঝে, সে জানে কোন ধাপে সমস্যা হয়েছে, তাই কোথায় হাত দিতে হবে সেটাও জানে।

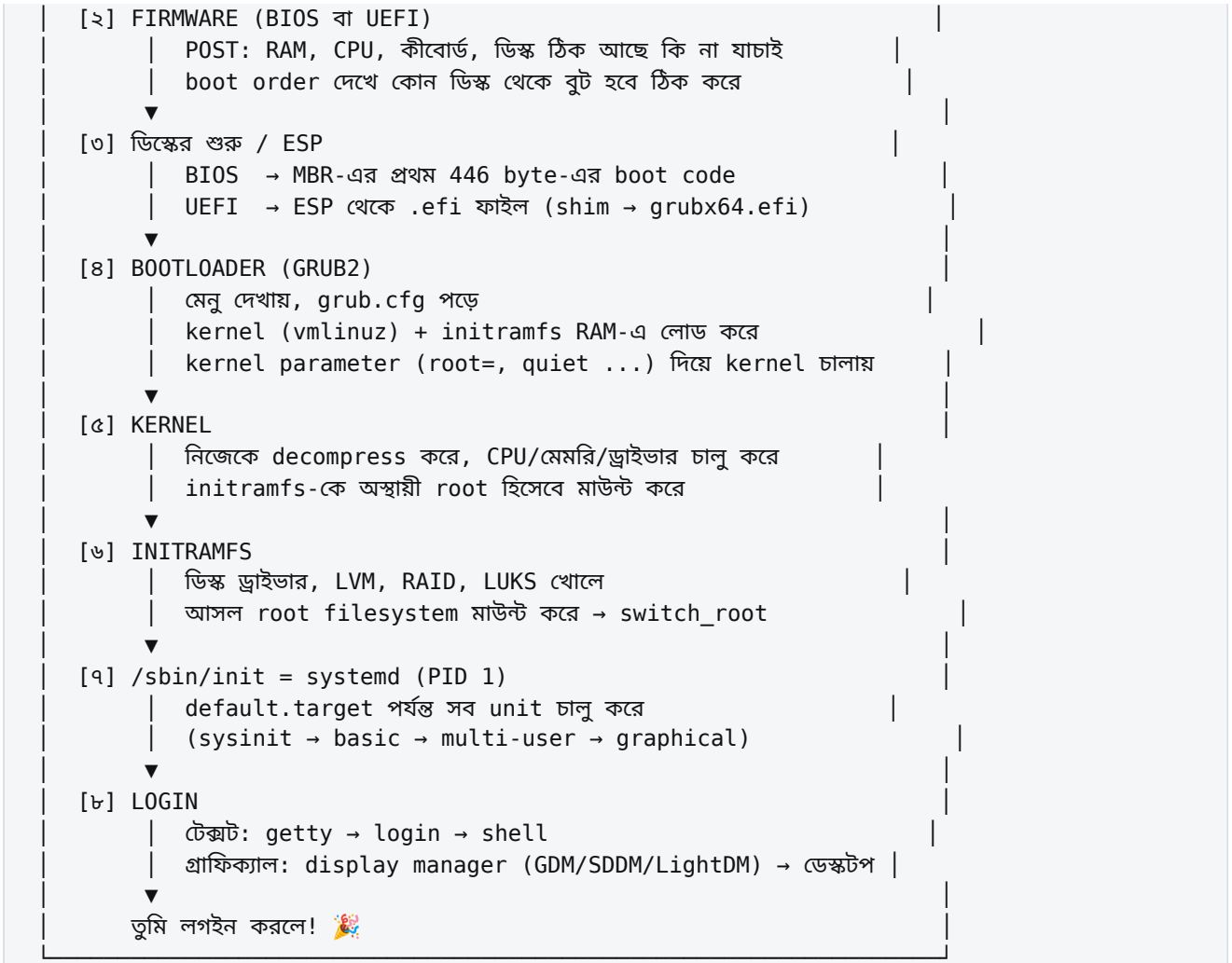
এই অধ্যায়ে যা শিখবে

- পাওয়ার বাটন থেকে লগইন প্রম্পট পর্যন্ত বুটের প্রতিটি ধাপ, একটা পূর্ণাঙ্গ ডায়াগ্রামসহ
- Firmware কী: BIOS আর UEFI-র পার্থক্য, POST কী করে
- ডিস্কের পার্টিশন টেবিল: MBR বনাম GPT, আর ESP (EFI System Partition) কেন লাগে
- Bootloader GRUB2: `grub.cfg`, `/etc/default/grub`, `update-grub` / `grub2-mkconfig`, `grubby`
- Kernel parameter কী, কীভাবে সাময়িক বা স্থায়ীভাবে বদলায়
- Kernel কীভাবে লোড হয়, আর initramfs কেন লাগে (`update-initramfs`, `dracut`)
- PID 1 হিসেবে systemd, targets, আর runlevel-এর সাথে তাদের সম্পর্ক
- getty, login আর display manager কীভাবে লগইন স্ক্রিন দেখায়
- `dmesg`, `journalctl -b`, `systemd-analyze` দিয়ে বুট পরীক্ষা করা
- Rescue mode আর emergency mode
- ভুলে যাওয়া root বা ইউজার পাসওয়ার্ড রিসেট করা (Ubuntu/Debian ও RHEL/Fedora দুই পদ্ধতিতেই)
- Secure Boot, shim, MOK কী এবং কেন
- বুট-সংক্রান্ত সব কীবোর্ড শর্টকাট

২৭.১ বড় ছবিটা আগে দেখি

পুরো বুট প্রসেসটা এক নজরে:

```
┌──────────────────────────────────────────────────────────────┐
│  [১] পাওয়ার বাটন চাপা                                      │
│       │  CPU চালু হয়, মাদারবোর্ডের ROM/flash থেকে firmware চলে  │
│       ▼                                                      │
```



> 💡 **উপমা:** বুট প্রসেস একটা স্কুল খোলার মতো। ভোরে দারোয়ান (firmware) গেট খুলে দেখে বিদ্যুৎ, পানি ঠিক আছে কি না (POST)। তারপর সে অফিস সহকারীকে (bootloader) ডাকে। সহকারী রুটিন খাতা (`grub.cfg`) দেখে প্রধান শিক্ষককে (kernel) ডেকে আনে, সাথে একটা ব্যাগভর্তি জরুরি জিনিসপত্র (initramfs) দেয়। প্রধান শিক্ষক ব্যাগ থেকে চাবি বের করে আসল ক্লাসরুমগুলো (root filesystem) খোলেন। তারপর ভাইস-প্রিন্সিপাল (systemd) সব শিক্ষককে (service) তাঁদের ক্লাসে পাঠান। সবশেষে গেটে রিসেপশনিস্ট (getty/display manager) বসে ছাত্রদের (ইউজার) নাম-পরিচয় যাচাই করে ভেতরে ঢুকতে দেয়।

প্রত্যেক ধাপের একটাই মূল কাজ: **পরের ধাপকে খুঁজে বের করে মেমরিতে লোড করা এবং তার হাতে নিয়ন্ত্রণ দেওয়া।** কোনো ধাপ ব্যর্থ হলে তার পরের ধাপগুলো আর শুরুই হয় না। তাই কোন স্ক্রিনে আটকে আছ সেটা দেখেই বোঝা যায় সমস্যা কোন ধাপে।

কোথায় আটকে আছে / কী দেখছ	সম্ভাব্য সমস্যার ধাপ
কিছুই দেখায় না, বিপ শব্দ হয়	হার্ডওয়্যার / POST
"No bootable device" / "Operating System not found"	firmware বুট ডিভাইস পাচ্ছে না, বা MBR/ESP নষ্ট

কোথায় আটকে আছে / কী দেখছ	সম্ভাব্য সমস্যার ধাপ
`grub rescue>` প্রম্পট	GRUB নিজের ফাইল (`/boot/grub`) খুঁজে পাচ্ছে না
`grub>` প্রম্পট (মেনু নেই)	`grub.cfg` নেই বা নষ্ট
"Kernel panic - not syncing: VFS: Unable to mount root fs"	kernel root খুঁজে পাচ্ছে না, initramfs বা `root=` প্যারামিটারে সমস্যা
`(initramfs)` প্রম্পট (BusyBox) বা dracut emergency shell	initramfs আসল root মাউন্ট করতে পারেনি
"You are in emergency mode"	systemd কোনো জরুরি মাউন্ট (প্রায়ই `/etc/fstab`-এর ভুল) করতে পারেনি
লগইন স্ক্রিন আসে না, কালো স্ক্রিন বা কার্সর	display manager বা গ্রাফিক্স ড্রাইভারের সমস্যা

এবার প্রতিটি ধাপ বিস্তারিত দেখি।

২৭.২ ধাপ ১: Firmware — BIOS ও UEFI

Firmware কী?

Firmware হলো মাদারবোর্ডের একটা flash চিপে স্থায়ীভাবে রাখা সফটওয়্যার। হার্ডওয়্যার আর সফটওয়্যারের মাঝামাঝি বলে এর নাম "firm"। পাওয়ার অন করলে CPU সবার আগে এটাকেই চালায়, কারণ তখনো RAM ফাঁকা আর ডিস্ক থেকে কিছু পড়া হয়নি।

এর দুটি প্রজন্ম আছে:

- **BIOS** (Basic Input/Output System): ১৯৮০-র দশকের IBM PC থেকে চলে আসছে। 16-bit real mode-এ চলে, এক MB মেমরির সীমার মধ্যে কাজ করে।
- **UEFI** (Unified Extensible Firmware Interface): আধুনিক বিকল্প। ২০১০-এর পর থেকে প্রায় সব কম্পিউটারে এটাই থাকে। 32/64-bit মোডে চলে, নিজেই FAT ফাইলসিস্টেম পড়তে পারে, মাউস-চালিত সেটআপ স্ক্রিন থাকে, Secure Boot সাপোর্ট করে।

> 📝 **মনে রাখো:** অনেকে এখনও UEFI সেটআপ স্ক্রিনকে "BIOS" বলে। আধুনিক মেশিনে "BIOS-এ ঢোকো" মানে আসলে UEFI সেটআপে ঢোকা। অনেক UEFI-তে **CSM** (Compatibility Support Module) নামে একটা "Legacy BIOS" মোডও থাকে, যেটা পুরনো BIOS-এর মতো আচরণ করে। নতুন অনেক মেশিন থেকে CSM সরিয়ে দেওয়া হচ্ছে।

POST — Power-On Self-Test

Firmware প্রথমে **POST** চালায়। মানে নিজে নিজে হার্ডওয়্যার পরীক্ষা:

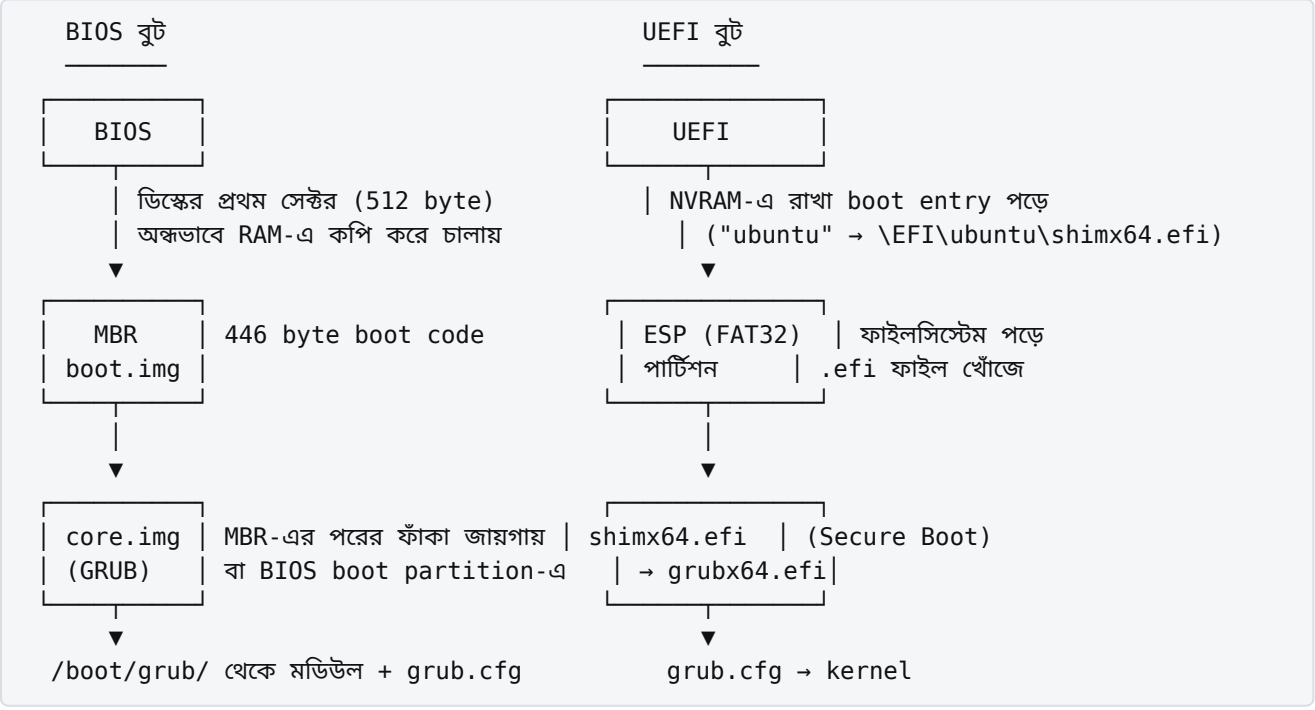
1. CPU রেজিস্টার ও firmware-এর নিজের অখণ্ডতা যাচাই
2. RAM শনাক্ত ও পরীক্ষা
3. ভিডিও কার্ড চালু (এ জন্যই স্ক্রিনে প্রথম লোগো আসে)
4. কীবোর্ড, স্টোরেজ কন্ট্রোলার, USB ইত্যাদি শনাক্ত

কোনো গুরুতর সমস্যা পেলে স্ক্রিন চালু হওয়ার আগেই নির্দিষ্ট প্যাটার্নে **বিপ কোড** বাজায়, বা মাদারবোর্ডের LED জ্বালায়। প্যাটার্নের মানে নির্মাতা-ভেদে আলাদা, তাই মাদারবোর্ডের ম্যানুয়াল দেখতে হয়।

POST সফল হলে firmware তার **boot order** (বুট ক্রম) দেখে: আগে USB, তারপর NVMe ডিস্ক ইত্যাদি। তালিকার প্রথম বুটযোগ্য ডিভাইসে যায়।

BIOS আর UEFI বুট করে কীভাবে?

এটাই আসল পার্থক্য:



BIOS ফাইলসিস্টেম চেনে না। সে শুধু ডিস্কের প্রথম 512 byte পড়ে চালিয়ে দেয়। ওই ছোট্ট জায়গায় পুরো bootloader আঁটে না, তাই GRUB কয়েক ধাপে নিজেকে লোড করে (boot.img → core.img → মডিউল)।

UEFI নিজেই FAT32 পড়তে পারে। ডিস্কের একটা বিশেষ পার্টিশন (ESP) থেকে সরাসরি একটা `.efi` প্রোগ্রাম ফাইল খুঁজে চালায়। কোন ফাইল চালাবে সেটা মাদারবোর্ডের **NVRAM**-এ (non-volatile RAM, বিদ্যুৎ গেলেও মনে রাখে) boot entry হিসেবে লেখা থাকে।

বৈশিষ্ট্য	BIOS (Legacy)	UEFI
চালু হওয়ার সময়	1980-এর দশক	2005-এর পর থেকে ব্যাপক
CPU মোড	16-bit real mode	32/64-bit
বুট পদ্ধতি	ডিস্কের প্রথম সেক্টর চালায়	ESP থেকে `.efi` ফাইল চালায়

বৈশিষ্ট্য	BIOS (Legacy)	UEFI
পার্টিশন টেবিল	সাধারণত MBR	সাধারণত GPT
ডিস্কের সীমা (MBR-এ)	2 TiB	কার্যত সীমাহীন (GPT)
Secure Boot	নেই	আছে
boot entry	নেই (শুধু ডিস্কের ক্রম)	NVRAM-এ নামসহ entry
সেটআপ ইন্টারফেস	টেক্সট, কীবোর্ড	প্রায়ই গ্রাফিক্যাল, মাউস চলে

তোমার সিস্টেম কোন মোডে বুট হয়েছে?

সবচেয়ে সহজ পরীক্ষা: `/sys/firmware/efi` ডিরেক্টরি আছে কি না।

```
$ [ -d /sys/firmware/efi ] && echo "UEFI" || echo "BIOS (Legacy)"
UEFI
```

UEFI হলে কত-bit তাও দেখা যায়:

```
$ cat /sys/firmware/efi/fw_platform_size
64
```

`efibootmgr` — UEFI boot entry দেখা ও বদলানো

UEFI-র NVRAM-এ রাখা boot entry লিনাক্স থেকেই দেখা আর বদলানো যায়।

```
$ efibootmgr
BootCurrent: 0001
Timeout: 1 seconds
BootOrder: 0001,0000,0002
Boot0000* Windows Boot Manager
Boot0001* ubuntu
Boot0002* UEFI: USB Flash Drive
```

`-v` দিলে প্রতিটি entry কোন পার্টিশনের কোন ফাইল চালায় তা দেখায়:

```
$ efibootmgr -v
Boot0001* ubuntu        HD(1,GPT,3f2a...,0x800,0x100000)/File(\EFI\ubuntu\shimx64.efi)
```

অপশন	কাজ
`-v`	বিস্তারিত (ফাইলের পাথসহ)
`-o 0001,0000`	boot order বদলাও
`-n 0002`	শুধু পরের একবার এই entry থেকে বুট হবে (BootNext)
`-b 0003 -B`	`0003` নম্বর entry মুছে ফেলো

অপশন	কাজ
`-c -d /dev/nvme0n1 -p 1 -L "নাম" -l '\EFI\x\y.efi'`	নতুন entry তৈরি

```
$ sudo efibootmgr -o 0001,0000,0002
```

> ⚠️ **সতর্কতা:** `efibootmgr` দিয়ে ভুল entry মুছে ফেললে মেশিন বুট নাও হতে পারে। তবে ESP-র ফাইলগুলো অক্ষত থাকলে সাধারণত firmware-এর বুট মেনু বা লাইভ USB দিয়ে আবার entry বানানো যায়। অনেক firmware entry না পেলে fallback পাথ `\EFI\BOOT\BOOTX64.EFI` চেষ্টা করে।

লিনাক্স থেকে সরাসরি পরের বুটে UEFI সেটআপ স্ক্রিনে যেতে চাইলে (কোন key চাপতে হয় খুঁজতে না হয়):

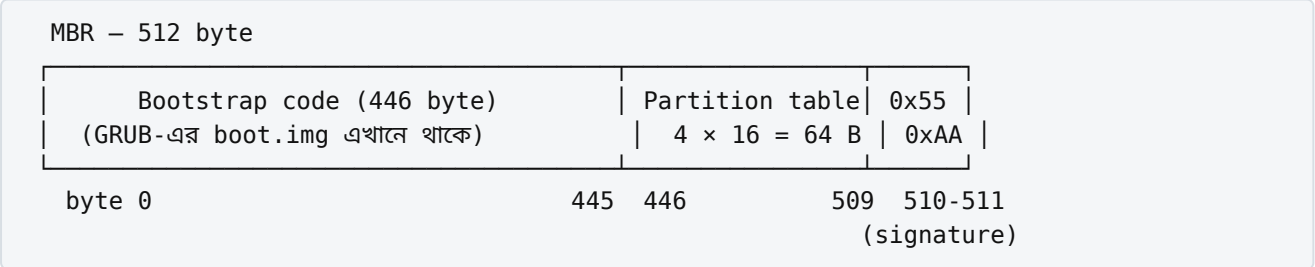
```
$ sudo systemctl reboot --firmware-setup
```

২৭.৩ ধাপ ২: ডিস্কের পার্টিশন টেবিল — MBR বনাম GPT

Firmware ডিস্কে পৌঁছানোর পর প্রথমে দেখে ডিস্কটা কীভাবে ভাগ করা আছে। এই ভাগের তালিকাকে বলে **partition table** (পার্টিশন টেবিল)। এর দুটি ফরম্যাট: MBR আর GPT। পার্টিশন তৈরি আর ফরম্যাটিংয়ের পুরো আলোচনা আছে অধ্যায় ৩২-এ। এখানে শুধু বুটের সাথে যেটুকু সম্পর্কিত, সেটা দেখব।

MBR (Master Boot Record)

ডিস্কের একদম প্রথম 512 byte (sector 0):

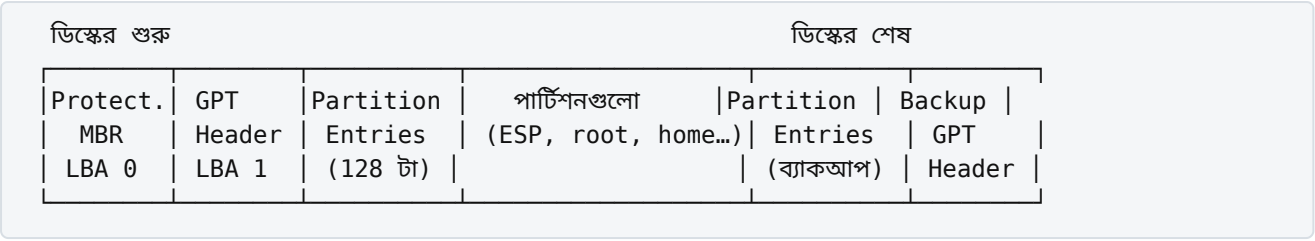


- **446 byte**: bootstrap code। BIOS এটাই চালায়।
- **64 byte**: পার্টিশন টেবিল, সর্বোচ্চ ৪টি **primary partition** (প্রতিটি 16 byte)। বেশি লাগলে একটাকে **extended partition** বানিয়ে তার ভেতরে **logical partition** তৈরি করতে হয়।
- **2 byte**: `0x55AA` signature। এটা না থাকলে BIOS ডিস্কটাকে বুটযোগ্য মনে করে না।

সীমাবদ্ধতা: পার্টিশনের শুরু আর আকার 32-bit সংখ্যায় রাখা হয়, তাই 512-byte সেক্টরে সর্বোচ্চ 2 TiB পর্যন্ত ব্যবহার করা যায়। পার্টিশন টেবিলের কোনো ব্যাকআপ কপিও নেই।

GPT (GUID Partition Table)

UEFI স্ট্যান্ডার্ডের অংশ, আধুনিক ডিস্কের ডিফল্ট:



- **Protective MBR**: পুরনো টুল যেন ডিস্কটাকে "খালি" ভেবে মুছে না ফেলে, তাই LBA 0-তে একটা নকল MBR রাখা থাকে, যেটা বলে "পুরো ডিস্ক একটাই পার্টিশন, টাইপ GPT"।
- ডিফল্টে **128টি পার্টিশন** রাখা যায়, extended/logical-এর ঝামেলা নেই।
- প্রতিটি পার্টিশনের একটা অনন্য **GUID** (PARTUUID) আর একটা টাইপ GUID থাকে।
- ডিস্কের শেষে **ব্যাকআপ header** থাকে, আর CRC32 checksum দিয়ে নষ্ট হয়েছে কি না যাচাই করা যায়।
- সীমা প্রায় 8 ZiB, বাস্তবে অসীম।

বৈশিষ্ট্য	MBR	GPT
সর্বোচ্চ ডিস্ক (512-byte সেক্টর)	2 TiB	~8 ZiB
পার্টিশন সংখ্যা	৪ primary (extended দিয়ে বেশি)	ডিফল্ট 128
ব্যাকআপ	নেই	ডিস্কের শেষে আছে
অখণ্ডতা যাচাই	নেই	CRC32
পার্টিশন পরিচয়	1-byte টাইপ কোড (যেমন `83`)	GUID
সাধারণত কার সাথে	BIOS	UEFI

তোমার ডিস্কে কোন টেবিল:

```
$ sudo fdisk -l /dev/nvme0n1 | grep -i "disklabel type"
Disklabel type: gpt
```

অথবা:

```
$ lsblk -o NAME,SIZE,PTTYPE,FSTYPE,MOUNTPOINTS
NAME          SIZE PTTYPE FSTYPE MOUNTPOINTS
nvme0n1     476.9G gpt
├─nvme0n1p1     1G gpt    vfat   /boot/efi
├─nvme0n1p2     2G gpt    ext4   /boot
└─nvme0n1p3 473.9G gpt    ext4   /
```

> 📝 **মনে রাখো:** BIOS + GPT একসাথেও চলে। সে ক্ষেত্রে GRUB-এর core.img রাখার জন্য ছোট্ট (~1 MiB) একটা **BIOS boot partition** লাগে, যার কোনো ফাইলসিস্টেম থাকে না (`parted`-এ flag `bios_grub`)।

> আবার UEFI + MBR-ও সম্ভব, তবে কম দেখা যায়।

ESP — EFI System Partition

UEFI বুটের হৃদয় হলো **ESP**। এটা একটা সাধারণ **FAT32** পার্টিশন, যার GPT টাইপ GUID `C12A7328-F81F-11D2-BA4B-00A0C93EC93B` (`fdisk`-এ টাইপ "EFI System")। UEFI firmware এই পার্টিশন পড়ে `.efi` ফাইল চালায়।

- আকার: সাধারণত 100 MiB থেকে 1 GiB। Ubuntu-র নতুন ইনস্টলার 1 GiB-এর মতো দেয়, আর ছোট পুরনো ESP-তে জায়গা ফুরিয়ে যাওয়ার সমস্যা এড়াতে বড় রাখাই নিরাপদ।
- লিনাক্সে মাউন্ট পয়েন্ট: Ubuntu, Debian, Fedora, RHEL-এ **`/boot/efi`**। কিছু সেটআপে (systemd-boot, Arch) `/efi` বা `/boot`।
- একই ডিস্কে Windows আর লিনাক্স থাকলে **একটাই ESP** দুজনে ভাগ করে ব্যবহার করে, প্রত্যেকে নিজের ফোল্ডারে।

```
$ sudo tree /boot/efi -L 3
/boot/efi
└── EFI
    ├── BOOT
    │   ├── BOOTX64.EFI        ← fallback (entry না থাকলে firmware এটা চালায়)
    │   ├── fbx64.efi
    │   └── mmx64.efi
    ├── Microsoft
    │   └── Boot
    └── ubuntu
        ├── grub.cfg           ← ছোট্ট stub, আসল grub.cfg-কে দেখিয়ে দেয়
        ├── grubx64.efi        ← GRUB নিজে
        ├── mmx64.efi          ← MOK Manager
        └── shimx64.efi        ← Secure Boot-এর প্রথম ধাপ
```

Fedora/RHEL-এ ফোল্ডারের নাম `EFI/fedora` বা `EFI/redhat`।

> 💡 **টিপস:** ESP-তে কিছু মুছবে না, যদি না ঠিক জানো কী করছ। Windows dual-boot-এ `EFI/Microsoft` মুছলে Windows আর বুট হবে না।

২৭.৪ ধাপ ৩: Bootloader — GRUB2

Bootloader কী আর কেন লাগে?

Bootloader হলো সেই প্রোগ্রাম যার কাজ OS-এর kernel খুঁজে মেমরিতে লোড করা আর চালানো। Firmware নিজে লিনাক্সের kernel চেনে না, ext4 বা Btrfs ফাইলসিস্টেমও পড়তে পারে না। Bootloader এই ফাঁকটা পূরণ করে:

- কোন OS বা কোন kernel ভার্সন বুট হবে তার মেনু দেখায়
- ext4, XFS, Btrfs, LVM, এমনকি এনক্রিপ্টেড `/boot` থেকেও kernel পড়তে পারে
- kernel-কে **parameter** (কমান্ড লাইন) দেয়
- initramfs ফাইলটাও মেমরিতে লোড করে দেয়

লিনাক্সের প্রধান bootloader হলো **GRUB2** (GRand Unified Bootloader, version 2)। Ubuntu, Debian, Fedora, RHEL সবাই এটা ব্যবহার করে। বিকল্প হিসেবে **systemd-boot** (সহজ, শুধু UEFI) আছে, Arch আর Pop!_OS-এ দেখা যায়। পুরনো **LILO** আর **GRUB Legacy** এখন অচল।

> 📝 **মনে রাখো:** Debian/Ubuntu-তে কমান্ডগুলোর নাম `grub-*` (যেমন `grub-install`, `grub-mkconfig`), আর Fedora/RHEL-এ `grub2-*` (যেমন `grub2-install`, `grub2-mkconfig`)। কাজ একই, শুধু নাম আলাদা।

GRUB-এর ফাইলগুলো কোথায়?

ফাইল/ডিরেক্টরি	কাজ	হাতে এডিট করবে?
`/etc/default/grub`	মূল সেটিংস (timeout, kernel parameter ইত্যাদি)	✅ হ্যাঁ, এটাই এডিট করার জায়গা
`/etc/grub.d/`	কিছু শেল স্ক্রিপ্ট, যারা মিলে `grub.cfg` বানায়	মাঝে মাঝে (কাস্টম entry-র জন্য `40_custom`)
`/boot/grub/grub.cfg` (Debian/Ubuntu)	GRUB আসলে যেটা পড়ে	❌ না, স্বয়ংক্রিয়ভাবে তৈরি
`/boot/grub2/grub.cfg` (Fedora/RHEL)	একই	❌ না
`/boot/loader/entries/*.conf` (Fedora/RHEL)	প্রতিটি kernel-এর আলাদা entry (BLS)	`grubby` দিয়ে বদলাও
`/boot/grub/` বা `/boot/grub2/`	GRUB-এর মডিউল, ফন্ট, থিম	❌ না

এর পেছনের নকশাটা এরকম:

```
/etc/default/grub  ─┐        (তোমার সেটিংস)
                    ├─►  grub-mkconfig / update-grub  ──►  /boot/grub/grub.cfg
/etc/grub.d/*      ─┘        (স্ক্রিপ্টগুলো চালায়)          (GRUB বুটের সময় পড়ে)
  00_header
  10_linux        ← /boot-এ যত kernel পায়, প্রত্যেকটার entry লেখে
  20_linux_xen
  30_os-prober    ← Windows বা অন্য OS খুঁজে entry লেখে
  40_custom       ← তোমার নিজের entry
```

`grub.cfg`-এর মাথাতেই লেখা থাকে `DO NOT EDIT THIS FILE`। কারণ পরের বার kernel আপডেট হলে এটা নতুন করে তৈরি হবে, আর তোমার হাতের পরিবর্তন মুছে যাবে।

`/etc/default/grub` — গুরুত্বপূর্ণ সেটিংস

একটা সাধারণ Ubuntu-র ফাইল:

```
$ cat /etc/default/grub
GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`( . /etc/os-release; echo ${NAME:-Ubuntu} ) 2>/dev/null || echo
Ubuntu`
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX=""
```

ভেরিয়েবল	মানে	উদাহরণ
`GRUB_DEFAULT`	কোন entry ডিফল্টে বুট হবে। সংখ্যা (০ থেকে শুরু), entry-র নাম, বা `saved`	`GRUB_DEFAULT=0`
`GRUB_SAVEDEFAULT`	`true` আর `GRUB_DEFAULT=saved` হলে শেষবার যেটা বেছেছিলে সেটাই মনে রাখে	`GRUB_SAVEDEFAULT=true`
`GRUB_TIMEOUT`	মেনু কত সেকেন্ড অপেক্ষা করবে। `-1` মানে অনির্দিষ্টকাল	`GRUB_TIMEOUT=5`
`GRUB_TIMEOUT_STYLE`	`menu` = মেনু দেখাও, `hidden` = লুকিয়ে রাখো, `countdown` = শুধু কাউন্টডাউন	`GRUB_TIMEOUT_STYLE=menu`
`GRUB_CMDLINE_LINUX`	**সব** entry-তে (recovery-সহ) যোগ হওয়া kernel parameter	`GRUB_CMDLINE_LINUX="ipv6.disable=1"`
`GRUB_CMDLINE_LINUX_DEFAULT`	শুধু সাধারণ entry-তে যোগ হয়, recovery-তে না (Debian/Ubuntu)	`"quiet splash"`
`GRUB_DISABLE_OS_PROBER`	`false` হলে Windows/অন্য OS খুঁজবে	`GRUB_DISABLE_OS_PROBER=false`
`GRUB_DISABLE_RECOVERY`	`true` হলে recovery entry তৈরি হবে না	
`GRUB_GFXMODE`	মেনুর রেজোলিউশন	`GRUB_GFXMODE=1920x1080`
`GRUB_ENABLE_BLSCFG`	Fedora/RHEL-এ BLS entry ব্যবহার হবে কি না	`true`

> 📝 **মনে রাখো:** GRUB 2.06 থেকে `os-prober` ডিফল্টে বন্ধ (নিরাপত্তার কারণে)। তাই নতুন Ubuntu/Debian-এ Windows মেনুতে না এলে `GRUB_DISABLE_OS_PROBER=false` যোগ করে `update-grub` চালাও (`os-prober` প্যাকেজ ইনস্টল থাকতে হবে)।

পরিবর্তন প্রয়োগ করা: `update-grub` ও `grub2-mkconfig`

`/etc/default/grub` এডিট করলে নিজে থেকে কিছু হয় না। `grub.cfg` নতুন করে তৈরি করতে হয়।

Debian / Ubuntu:

```
$ sudo nano /etc/default/grub
$ sudo update-grub
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-45-generic
Found initrd image: /boot/initrd.img-6.8.0-45-generic
Found linux image: /boot/vmlinuz-6.8.0-41-generic
Found initrd image: /boot/initrd.img-6.8.0-41-generic
Found Windows Boot Manager on /dev/nvme0n1p1@/EFI/Microsoft/Boot/bootmgfw.efi
Adding boot menu entry for UEFI Firmware Settings ...
done
```

`update-grub` আসলে একটা ছোট wrapper, যেটা ভেতরে চালায়:

```
$ sudo grub-mkconfig -o /boot/grub/grub.cfg
```

Fedora / RHEL 9:

```
$ sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

> ⚠️ **সতর্কতা (RHEL/Fedora + UEFI):** পুরনো গাইডে লেখা থাকে `grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg`। Fedora 34 আর RHEL 9 থেকে এটা **আর করবে না**। এখন ESP-তে থাকা `grub.cfg` একটা ছোট stub, যেটা `/boot/grub2/grub.cfg`-কে লোড করে। তুমি stub-টা ওভাররাইট করলে বুট ভেঙে যেতে পারে। সবসময় `-o /boot/grub2/grub.cfg` ব্যবহার করো।

Fedora/RHEL-এর BLS আর `grubby`

Fedora আর RHEL 8/9 **BLS** (Boot Loader Specification) ব্যবহার করে। এখানে প্রতিটি kernel-এর entry `grub.cfg`-এর ভেতরে না থেকে `/boot/loader/entries/`-এ আলাদা ফাইলে থাকে:

```
$ sudo ls /boot/loader/entries/
a1b2c3d4...-5.14.0-427.13.1.el9_4.x86_64.conf
a1b2c3d4...-0-rescue.conf

$ sudo cat /boot/loader/entries/a1b2c3d4...-5.14.0-427.13.1.el9_4.x86_64.conf
title Red Hat Enterprise Linux (5.14.0-427.13.1.el9_4.x86_64) 9.4 (Plow)
version 5.14.0-427.13.1.el9_4.x86_64
linux /vmlinuz-5.14.0-427.13.1.el9_4.x86_64
initrd /initramfs-5.14.0-427.13.1.el9_4.x86_64.img $tuned_initrd
options root=/dev/mapper/rhel-root ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M
resume=/dev/mapper/rhel-swap rd.lvm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet
grub_users $grub_users
grub_arg --unrestricted
grub_class rhel
```

এগুলো সামলানোর জন্য টুল হলো **`grubby`**:

কমান্ড	কাজ
`sudo grubby --default-kernel`	ডিফল্ট kernel কোনটা দেখাও
`sudo grubby --info=ALL`	সব entry-র বিস্তারিত
`sudo grubby --set-default=/boot/vmlinuz-<ver>`	ডিফল্ট kernel বদলাও
`sudo grubby --update-kernel=ALL --args="nomodeset"`	সব kernel-এ parameter যোগ করো
`sudo grubby --update-kernel=ALL --remove-args="quiet"`	parameter সরাও
`sudo grubby --update-kernel=DEFAULT --args="..."`	শুধু ডিফল্ট kernel-এ

```
$ sudo grubby --default-kernel
/boot/vmlinuz-5.14.0-427.13.1.el9_4.x86_64
```

> 💡 **টিপস:** RHEL 9-এ নতুন kernel ইনস্টলের সময়ও parameter বজায় রাখতে চাইলে `grubby --update-kernel=ALL` এর পাশাপাশি `/etc/default/grub` -এর `GRUB_CMDLINE_LINUX` -এও যোগ করে রাখো।

GRUB পুনরায় ইনস্টল: `grub-install`

`grub-mkconfig` শুধু কনফিগ ফাইল বানায়। আর `grub-install` GRUB-এর বাইনারিগুলো বুট লোকেশনে (MBR বা ESP) বসায়। সাধারণত এটা হাতে চালানোর দরকার হয় না। লাগে যখন Windows ইনস্টল করে GRUB মুছে গেছে, বা ডিস্ক বদলেছ।

```
# BIOS: ডিস্কে বসাও (পার্টিশনে না!)
$ sudo grub-install /dev/sda

# UEFI (Debian/Ubuntu):
$ sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-
id=ubuntu
```

> ⚠️ **সতর্কতা:** RHEL 9 / Fedora-তে UEFI + Secure Boot সিস্টেমে `grub2-install` চালানো উচিত না। সেখানে সাইন করা GRUB প্যাকেজ থেকে আসে, তাই মেরামতের জন্য `sudo dnf reinstall shim-x64 grub2-efi-x64` চালানো হয়।

GRUB মেনু ব্যবহার করা

Ubuntu-তে একটাই OS থাকলে মেনু লুকানো থাকে। দেখতে চাইলে বুটের সময়:

- **BIOS মেশিনে:** `Shift` চেপে ধরে রাখো
- **UEFI মেশিনে:** `Esc` বারবার চাপো (বেশিবার চাপলে `grub>` কমান্ড লাইনে চলে যেতে পারে, তখন `normal` লিখে Enter দিলে মেনুতে ফিরবে)

মেনুতে:

```
                     GNU GRUB  version 2.12

 ┌──────────────────────────────────────────────────────────────────────────┐
 │*Ubuntu                                                                   │
 │ Advanced options for Ubuntu                                              │
 │ Windows Boot Manager (on /dev/nvme0n1p1)                                 │
 │ UEFI Firmware Settings                                                   │
 └──────────────────────────────────────────────────────────────────────────┘

   Use the ↑ and ↓ keys to select which entry is highlighted.
   Press enter to boot the selected OS, `e' to edit the commands
   before booting or `c' for a command-line.
```

"Advanced options"-এর ভেতরে পুরনো kernel আর **recovery mode** entry থাকে। নতুন kernel-এ সমস্যা হলে এখান থেকে পুরনো kernel বেছে বুট করা যায়। এজন্যই প্যাকেজ ম্যানেজার আগের ১-২টি kernel রেখে দেয়।

একটা GRUB entry-র ভেতরে কী থাকে

মেনুতে কোনো entry-তে দাঁড়িয়ে `e` চাপলে এরকম দেখাবে:

```
setparams 'Ubuntu'

        recordfail
        load_video
        gfxmode $linux_gfx_mode
        insmod gzio
        insmod part_gpt
        insmod ext2
        search --no-floppy --fs-uuid --set=root 7e1c...-...
        linux   /boot/vmlinuz-6.8.0-45-generic root=UUID=7e1c...-... ro  quiet splash
$vt_handoff
        initrd  /boot/initrd.img-6.8.0-45-generic
```

লাইন	মানে
`insmod part_gpt`, `insmod ext2`	GPT আর ext2/3/4 পড়ার GRUB মডিউল লোড করো
`search ... --set=root UUID`	এই UUID-এর পার্টিশন খুঁজে GRUB-এর `root` বানাও
`linux /boot/vmlinuz-...`	এই kernel লোড করো, বাকিটা kernel parameter
`initrd /boot/initrd.img-...`	এই initramfs লোড করো

এখানে পরিবর্তন করে `Ctrl+x` বা `F10` চাপলে **শুধু এই একবারের জন্য** সেভাবে বুট হবে। পরের রিবুটে আগের মতো হয়ে যাবে। সমস্যা সমাধানে এটা খুব কাজের।

GRUB কমান্ড লাইন: হাতে বুট করা

মেনুতে `c` চাপলে, বা `grub.cfg` না পেলে, `grub>` প্রম্পট আসে। এখান থেকে হাতে বুট করা যায়:

```
grub> ls
(hd0) (hd0,gpt1) (hd0,gpt2) (hd0,gpt3)

grub> ls (hd0,gpt2)/
lost+found/ boot/ etc/ home/ ... vmlinuz initrd.img

grub> set root=(hd0,gpt2)
grub> linux /boot/vmlinuz-6.8.0-45-generic root=/dev/nvme0n1p2 ro
grub> initrd /boot/initrd.img-6.8.0-45-generic
grub> boot
```

GRUB-এ ডিস্ক `hd0` থেকে আর পার্টিশন ১ থেকে গোনা হয়। `Tab` চাপলে নাম auto-complete হয়।

যদি **`grub rescue>`** আসে, তার মানে GRUB নিজের মডিউলই খুঁজে পাচ্ছে না, তাই সেখানে কমান্ড খুব সীমিত। তখন সঠিক পার্টিশন খুঁজে normal মডিউল লোড করতে হয়:

```
grub rescue> ls
grub rescue> set prefix=(hd0,gpt2)/boot/grub
grub rescue> set root=(hd0,gpt2)
grub rescue> insmod normal
grub rescue> normal
```

তারপর বুট হলে `sudo grub-install ...` আর `sudo update-grub` চালিয়ে স্থায়ীভাবে ঠিক করো।

GRUB পাসওয়ার্ড — কেন?

একটু পরেই দেখবে, GRUB মেনুতে `e` চেপে যে কেউ root পাসওয়ার্ড রিসেট করতে পারে। সার্ভার রুম বা শেয়ার করা কম্পিউটারে এটা আটকাতে GRUB-এ পাসওয়ার্ড দেওয়া যায়:

- **RHEL/Fedora:** `sudo grub2-setpassword` (এটা `/boot/grub2/user.cfg` তৈরি করে)
- **Debian/Ubuntu:** `grub-mkpasswd-pbkdf2` দিয়ে হ্যাশ বানিয়ে `/etc/grub.d/40_custom`-এ `set superusers` ও `password_pbkdf2` লাইন যোগ করে `update-grub`

২৭.৫ Kernel Parameters — kernel-কে নির্দেশ দেওয়া

GRUB-এর `linux` লাইনে kernel ফাইলের পরে যা লেখা থাকে, সেগুলোই **kernel parameter** বা **kernel command line**। এগুলো kernel আর initramfs-কে বলে দেয় কোথায় root আছে, কতটা লগ দেখাবে, কোন ফিচার বন্ধ রাখবে ইত্যাদি। বর্তমান বুটে কী parameter ব্যবহার হয়েছে:

```
$ cat /proc/cmdline
BOOT_IMAGE=/boot/vmlinuz-6.8.0-45-generic root=UUID=7e1c...-... ro quiet splash
vt.handoff=7
```

সাধারণ ও কাজের parameter

Parameter	কাজ
`root=UUID=...` / `root=/dev/sda2`	root filesystem কোথায়
`ro` / `rw`	root প্রথমে read-only না read-write মাউন্ট হবে (সাধারণত `ro`, পরে systemd `rw` করে)
`quiet`	kernel-এর বেশিরভাগ বার্তা স্ক্রিনে দেখাবে না
`splash`	Plymouth-এর গ্রাফিক্যাল বুট স্ক্রিন দেখাও (Ubuntu); Fedora/RHEL-এ এর সমতুল্য `rhgb`
`loglevel=7`	কত বিস্তারিত kernel বার্তা কনসোলে দেখাবে (০–৭)
`nomodeset`	kernel-এর গ্রাফিক্স ড্রাইভার mode-setting বন্ধ — নতুন NVIDIA কার্ডে কালো স্ক্রিন হলে প্রাথমিক সমাধান
`systemd.unit=rescue.target`	rescue mode-এ বুট
`systemd.unit=emergency.target`	emergency mode-এ বুট
`single`, `s`, `S`, `1`	পুরনো single-user mode; systemd এগুলোকে `rescue.target` হিসেবে নেয়
`3` / `5`	systemd-এ যথাক্রমে `multi-user.target` / `graphical.target`
`init=/bin/bash`	systemd-এর বদলে সরাসরি bash চালাও (PID 1 = bash)
`rd.break`	(dracut, Fedora/RHEL) আসল root-এ যাওয়ার ঠিক আগে initramfs-এর শেলে থামো
`rd.lvm.lv=vg/lv`	(dracut) initramfs-এ এই LV চালু করো
`resume=UUID=...`	hibernate থেকে ফেরার swap পার্টিশন
`console=ttyS0,115200`	সিরিয়াল কনসোলে আউটপুট পাঠাও (ক্লাউড VM-এ সাধারণ)
`mem=4G`	kernel-কে ৪ GB-এর বেশি RAM ব্যবহার করতে দিও না
`selinux=0` / `enforcing=0`	SELinux বন্ধ / permissive (শুধু ট্রাবলশুটিংয়ে)
`systemd.log_level=debug`	systemd-এর বিস্তারিত লগ
`fsck.mode=force`	বুটে ফাইলসিস্টেম চেক বাধ্যতামূলক করো

সাময়িকভাবে বদলানো (একবারের জন্য)

1. GRUB মেনু খোলো (`Shift` / `Esc`)
2. entry-তে দাঁড়িয়ে `e`
3. `linux` লাইনের শেষে parameter যোগ করো, যেমন `nomodeset`
4. `Ctrl+x` বা `F10` চাপো

স্থায়ীভাবে বদলানো

Debian/Ubuntu:

```
$ sudo nano /etc/default/grub
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash nomodeset"
$ sudo update-grub
$ sudo reboot
$ cat /proc/cmdline       # যাচাই করো
```

Fedora/RHEL:

```
$ sudo grubby --update-kernel=ALL --args="nomodeset"
$ sudo reboot
```

> 💡 **টিপস:** বুট সমস্যা খুঁজতে `quiet` আর `splash` (বা `rhgb`) সরিয়ে বুট করো। তাহলে স্ক্রিনে সব বার্তা দেখা যাবে, কোথায় আটকাচ্ছে বোঝা যাবে।

> 📝 **মনে রাখো:** kernel যে parameter নিজে চেনে না, সেগুলো সে PID 1 (systemd)-কে দিয়ে দেয়। এভাবেই `systemd.unit=...`-এর মতো parameter systemd পর্যন্ত পৌঁছায়। `rd.*` parameter-গুলো initramfs (dracut) পড়ে।

২৭.৬ ধাপ ৪: Kernel লোড হওয়া

`/boot`-এ কী কী থাকে

```
$ ls -lh /boot
-rw-r--r-- 1 root root 287K Sep 20 10:12 config-6.8.0-45-generic
drwx------ 3 root root 4.0K Jan  1  1970 efi
drwxr-xr-x 5 root root 4.0K Oct  2 09:30 grub
lrwxrwxrwx 1 root root   28 Oct  2 09:29 initrd.img -> initrd.img-6.8.0-45-generic
-rw-r--r-- 1 root root  68M Oct  2 09:30 initrd.img-6.8.0-45-generic
-rw------- 1 root root 8.9M Sep 20 10:12 System.map-6.8.0-45-generic
lrwxrwxrwx 1 root root   25 Oct  2 09:29 vmlinuz -> vmlinuz-6.8.0-45-generic
-rw------- 1 root root  15M Sep 20 10:12 vmlinuz-6.8.0-45-generic
```

ফাইল	কী
`vmlinuz-<ver>`	কম্প্রেস করা kernel ইমেজ। নামের `z` মানে zipped (কম্প্রেসড)
`initrd.img-<ver>` (Debian/Ubuntu) / `initramfs-<ver>.img` (Fedora/RHEL)	initramfs আর্কাইভ

ফাইল	কী
`config-<ver>`	kernel কোন অপশন নিয়ে কম্পাইল হয়েছে
`System.map-<ver>`	kernel-এর ফাংশন/ভেরিয়েবলের ঠিকানার তালিকা (ডিবাগিংয়ে লাগে)
`grub/` বা `grub2/`	GRUB-এর ফাইল
`efi/`	ESP-র মাউন্ট পয়েন্ট

চলমান kernel-এর ভার্সন:

```
$ uname -r
6.8.0-45-generic
```

Kernel চালু হলে ভেতরে কী হয়

GRUB kernel আর initramfs দুটোকেই RAM-এ বসিয়ে kernel-এর শুরুর ঠিকানায় লাফ দেয়। এরপর kernel:

```
┌──────────────────────────────────────────────────────────┐
│ 1. নিজেকে decompress করে                                 │
│ 2. CPU-কে পূর্ণ 64-bit মোডে নেয়, page table / virtual memory │
│    চালু করে                                              │
│ 3. মেমরি ম্যানেজার, scheduler, interrupt চালু করে        │
│ 4. kernel-এর ভেতরে বিল্ট-ইন (built-in) ড্রাইভারগুলো শুরু করে, │
│    হার্ডওয়্যার খোঁজে (PCI, USB, ACPI)                   │
│ 5. initramfs আর্কাইভটা খুলে RAM-এ একটা ছোট ফাইলসিস্টেম বানায় │
│    (rootfs) এবং সেখানকার /init চালায়  ← PID 1 (অস্থায়ী) │
└──────────────────────────────────────────────────────────┘
```

এই সময় kernel যা যা বার্তা লেখে, সব জমা হয় **kernel ring buffer**-এ, যা পরে `dmesg` দিয়ে পড়া যায় (২৭.১০ অংশে)।

kernel আর হার্ডওয়্যারের সম্পর্ক কীভাবে কাজ করে তার বিস্তারিত আছে অধ্যায় ২ আর অধ্যায় ৪৭-এ।

> ⚠️ **"Kernel panic"** মানে kernel এমন সমস্যায় পড়েছে যেখান থেকে সে আর এগোতে পারছে না। বুটের সময় সবচেয়ে প্রচলিত বার্তা: `Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)`। এর মানে প্রায়ই initramfs নেই বা নষ্ট, অথবা `root=` ভুল। সমাধান: GRUB-এর "Advanced options" থেকে পুরনো kernel দিয়ে বুট করে initramfs নতুন করে বানাও।

২৭.৭ ধাপ ৫: initramfs — ছোট্ট উদ্ধারকারী ফাইলসিস্টেম

সমস্যাটা কী? মুরগি আগে না ডিম আগে

kernel-কে root filesystem মাউন্ট করতে হবে। কিন্তু:

- root যদি থাকে NVMe ডিস্কে, তাহলে NVMe ড্রাইভার লাগবে
- root যদি থাকে **LVM**-এ, তাহলে LVM টুল (`lvm`) চালাতে হবে
- root যদি **LUKS** দিয়ে এনক্রিপ্টেড হয়, তাহলে পাসওয়ার্ড চেয়ে ডিস্ক খুলতে হবে
- root যদি **RAID** বা নেটওয়ার্কে (NFS/iSCSI) থাকে, তাহলে আরও টুল লাগবে

ডিস্ট্রিবিউশনের সাধারণ kernel হাজার রকম হার্ডওয়্যারে চলে, তাই সব ড্রাইভার kernel-এর ভেতরে না রেখে আলাদা **module** (`.ko` ফাইল) হিসেবে রাখে, যেগুলো থাকে `/lib/modules/<ver>/`-এ। কিন্তু `/lib/modules` তো root filesystem-এর ভেতরে! root মাউন্ট করতে ড্রাইভার লাগে, আর ড্রাইভার আছে root-এর ভেতরে। এই চক্র ভাঙতেই **initramfs**।

> 💡 **উপমা:** তুমি বাসার চাবি বাসার ভেতরে রেখে বের হয়ে গেছ। initramfs হলো পকেটে রাখা একটা ছোট ব্যাগ, যাতে বাসার দরজা খোলার জন্য দরকারি চাবি আর যন্ত্রপাতি আছে। দরজা খোলা হয়ে গেলে ব্যাগটার আর দরকার নেই।

initramfs আসলে কী?

initramfs (initial RAM filesystem) একটা কম্প্রেস করা **cpio আর্কাইভ**। এর ভেতরে থাকে একটা মিনি লিনাক্স:

- `/init` স্ক্রিপ্ট বা প্রোগ্রাম (Fedora/RHEL-এ dracut-এর ভেতরে systemd নিজেই)
- দরকারি kernel module (স্টোরেজ, ফাইলসিস্টেম, এনক্রিপশন)
- ছোট টুল: `udev`, `lvm`, `cryptsetup`, `mdadm`, BusyBox (Debian/Ubuntu)
- `/etc/fstab`-এর root লাইন, কখনো কীবোর্ড লেআউট

> 📝 **মনে রাখো:** পুরনো দিনে এর নাম ছিল **initrd** (initial RAM disk), যেটা আসলে একটা ডিস্ক ইমেজ ছিল। এখন ভেতরে cpio আর্কাইভ, অর্থাৎ initramfs, কিন্তু Debian/Ubuntu ফাইলের নাম এখনো `initrd.img-*` রেখেছে।

initramfs-এর কাজের ধারা

```
kernel
  │ initramfs খুলে RAM-এ বসায়, /init চালায়
  ▼
┌──────────────────── initramfs (RAM-এ) ────────────────────┐
```

```
  │  udev চালু → হার্ডওয়্যার খোঁজে → দরকারি module লোড          │
  │  (দরকার হলে) RAID জোড়া লাগায় → LUKS পাসওয়ার্ড চায়         │
  │            → LVM volume চালু করে                              │
  │  root=UUID=... অনুযায়ী আসল root মাউন্ট করে                  │
  │     /sysroot (dracut) বা /root (initramfs-tools)-এ            │
  └──────────────────────────────┬──────────────────────────────────┘
                                 │ switch_root
                                 ▼
       আসল root filesystem হলো নতুন "/"
       initramfs-এর মেমরি মুক্ত হয়
       /sbin/init (= systemd) চালু হয়, PID 1 হিসেবে
```

`switch_root` নতুন root-কে `/` বানায়, পুরনো initramfs মুছে মেমরি খালি করে, আর `/sbin/init` চালায়। লক্ষ করো, নতুন প্রোগ্রামও PID 1 নম্বরটাই পায়, কারণ এটা `exec` করে চালানো হয়।

কে initramfs বানায়: `initramfs-tools` বনাম `dracut`

ডিস্ট্রো	টুল	কমান্ড	ফাইল
Debian 12, Ubuntu 24.04	initramfs-tools	`update-initramfs`	`/boot/initrd.img-<ver>`
Fedora, RHEL 8/9, Rocky, Alma	dracut	`dracut`	`/boot/initramfs-<ver>.img`

> 📝 **মনে রাখো:** Ubuntu 25.10 থেকে ডিফল্ট টুল dracut-এ বদলেছে। Ubuntu 24.04 LTS-এ এখনো initramfs-tools।

সাধারণত তোমাকে এগুলো হাতে চালাতে হয় না। kernel আপডেটের সময় প্যাকেজ ম্যানেজার নিজেই নতুন initramfs বানায়। হাতে লাগে যখন:

- `/etc/crypttab`, `/etc/fstab`-এর root, বা RAID কনফিগ বদলেছ
- নতুন স্টোরেজ ড্রাইভার লাগবে (যেমন VM অন্য হাইপারভাইজারে সরিয়েছ)
- কোনো module ব্ল্যাকলিস্ট করেছ (যেমন `nouveau`)
- initramfs নষ্ট হয়েছে বলে kernel panic হচ্ছে

`update-initramfs` (Debian/Ubuntu)

```
$ sudo update-initramfs -u
update-initramfs: Generating /boot/initrd.img-6.8.0-45-generic
```

অপশন	কাজ
`-u`	বিদ্যমান initramfs আপডেট (ডিফল্টে সর্বশেষ kernel-এর)
`-u -k all`	সব ইনস্টল করা kernel-এর জন্য আপডেট
`-c -k 6.8.0-45-generic`	নির্দিষ্ট kernel-এর জন্য নতুন তৈরি
`-d -k <ver>`	মুছে ফেলো

অপশন	কাজ
`-v`	বিস্তারিত আউটপুট

কনফিগ থাকে `/etc/initramfs-tools/`-এ। যেমন `/etc/initramfs-tools/modules` ফাইলে কোনো module-এর নাম লিখলে সেটা initramfs-এ যোগ হবে।

ভেতরে কী আছে দেখতে:

```
$ lsinitramfs /boot/initrd.img-$(uname -r) | grep -E 'nvme|ext4' | head
usr/lib/modules/6.8.0-45-generic/kernel/drivers/nvme/host/nvme.ko.zst
usr/lib/modules/6.8.0-45-generic/kernel/drivers/nvme/host/nvme-core.ko.zst
usr/lib/modules/6.8.0-45-generic/kernel/fs/ext4/ext4.ko.zst
```

`dracut` (Fedora/RHEL)

```
$ sudo dracut -f
```

অপশন/কমান্ড	কাজ
`dracut -f`	চলমান kernel-এর initramfs ওভাররাইট করে নতুন বানাও
`dracut -f --kver 5.14.0-427.13.1.el9_4.x86_64`	নির্দিষ্ট kernel-এর জন্য
`dracut -f --regenerate-all`	সব kernel-এর জন্য
`dracut --add-drivers "virtio_blk" -f`	নির্দিষ্ট ড্রাইভার যোগ করো
`dracut -H -f` / `--hostonly`	শুধু এই মেশিনের দরকারি জিনিস রাখো (RHEL-এ ডিফল্ট)
`dracut -N -f` / `--no-hostonly`	সাধারণ (generic), যেকোনো হার্ডওয়্যারে চলবে, আকারে বড়
`lsinitrd`	initramfs-এর বিষয়বস্তু দেখো
`lsinitrd -m`	কোন dracut module আছে দেখো

কনফিগ: `/etc/dracut.conf` আর `/etc/dracut.conf.d/*.conf`।

```
$ sudo lsinitrd | head -5
Image: /boot/initramfs-5.14.0-427.13.1.el9_4.x86_64.img: 38M
================================================================================
Early CPIO image
================================================================================
drwxr-xr-x   3 root     root            0 Oct  2 09:30 .
```

> ⚠️ **সতর্কতা:** initramfs বানানোর আগে চাইলে পুরনোটার কপি রেখে দাও (`sudo cp /boot/initrd.img-$(uname -r){,.bak}`)। নতুনটা ভুল হলে GRUB-এ `e` চেপে `initrd` লাইনে `.bak` যোগ করে বুট করা যাবে।

initramfs-এর শেলে আটকে গেলে

initramfs root খুঁজে না পেলে একটা জরুরি শেলে থামে:

- **Debian/Ubuntu:** BusyBox শেল, প্রম্পট `(initramfs)`। সাধারণ বার্তা: `ALERT! UUID=... does not exist. Dropping to a shell!`
- **Fedora/RHEL:** dracut emergency shell, প্রম্পট `dracut:/#`, আর `/run/initramfs/rdsosreport.txt`-এ রিপোর্ট লেখা থাকে।

এখানে `ls /dev/disk/by-uuid/`, `blkid`, `cat /proc/cmdline` দিয়ে দেখো ডিস্ক দেখা যাচ্ছে কি না, UUID মিলছে কি না। ফাইলসিস্টেম নষ্ট হলে BusyBox-এ `fsck -y /dev/sda2` চালিয়ে `exit` দিলে বুট চালু হতে পারে।

২৭.৮ ধাপ ৬: systemd — PID 1, সব প্রসেসের জনক

আসল root-এ পৌঁছে চালু হয় `/sbin/init`। আধুনিক সব প্রধান ডিস্ট্রোতে এটা systemd-এর একটা symlink:

```
$ ls -l /sbin/init
lrwxrwxrwx 1 root root 20 Aug  9 12:00 /sbin/init -> /lib/systemd/systemd

$ ps -p 1 -o pid,comm
    PID COMMAND
      1 systemd
```

PID 1 কেন বিশেষ?

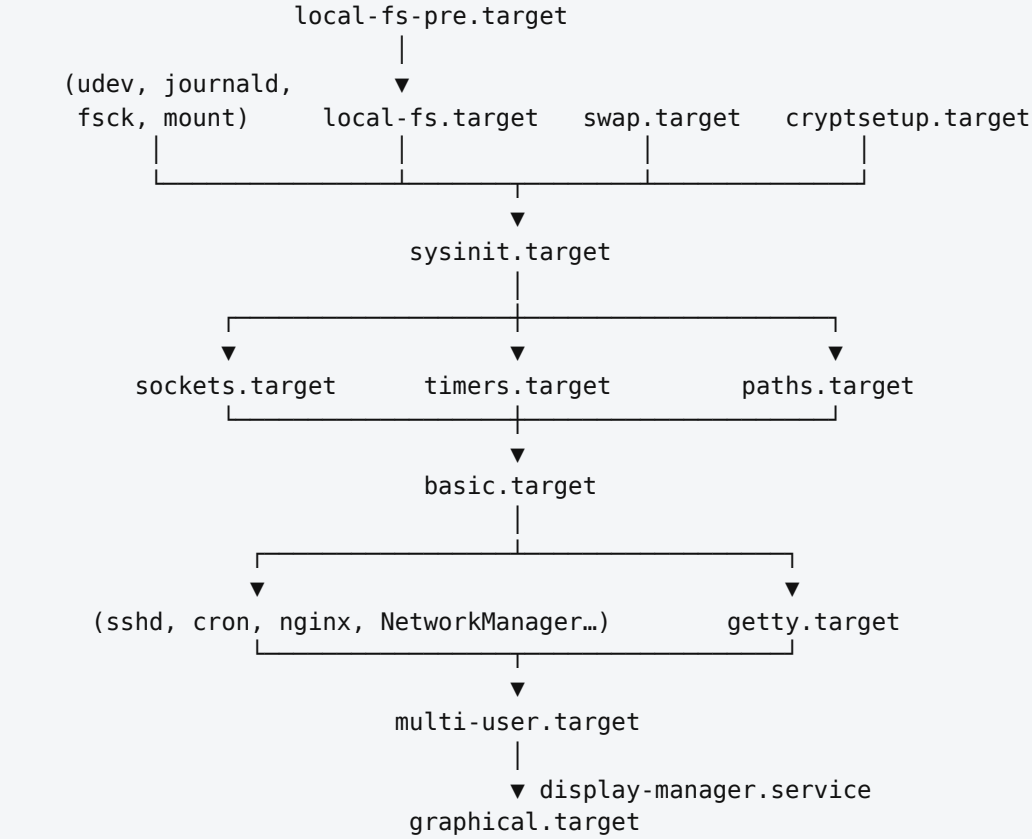
- এটাই kernel-এর চালানো প্রথম user-space প্রসেস। বাকি সব প্রসেস এর বংশধর (`pstree -p` চালিয়ে দেখো)।
- কোনো প্রসেসের প্যারেন্ট মরে গেলে সেই "অনাথ" প্রসেস PID 1-এর কাছে যায়, আর PID 1 পরে তার zombie পরিষ্কার করে।
- PID 1 মরে গেলে kernel panic হয়। তাই systemd খুব যত্ন করে লেখা।
- kernel PID 1-কে এমন সিগন্যাল থেকে রক্ষা করে যার হ্যান্ডলার সে বসায়নি, তাই `kill -9 1` কাজ করে না।

systemd, unit ফাইল আর `systemctl`-এর পুরো আলোচনা আছে অধ্যায় ২৬-এ। এখানে শুধু বুটের সাথে সম্পর্কিত অংশ।

systemd বুটে কী করে

systemd একটা **target** পর্যন্ত পৌঁছানোর চেষ্টা করে। target হলো অনেকগুলো unit-এর দল, একটা "অবস্থা" বা মাইলফলক। ডিফল্ট লক্ষ্য হলো `default.target`, যেটা সাধারণত `graphical.target` (ডেস্কটপ) বা `multi-user.target` (সার্ভার)-এর symlink।

systemd পুরো dependency গ্রাফ (`Wants=`, `Requires=`, `After=`) হিসাব করে, তারপর যেসব unit একে অন্যের ওপর নির্ভর করে না সেগুলো **একসাথে সমান্তরালে** চালায়। পুরনো SysV init একটার পর একটা স্ক্রিপ্ট চালাত, তাই systemd-এ বুট দ্রুত।



এই চিত্রটা `man bootup`-এর ডায়াগ্রামের সরল রূপ। পুরোটা দেখতে `man 7 bootup` পড়ো।

Targets আর পুরনো Runlevel

SysV init-এর যুগে সিস্টেমের অবস্থা বোঝাতে **runlevel** (০–৬) ব্যবহার হতো। systemd সামঞ্জস্যের জন্য `runlevelN.target` নামে alias রেখেছে:

Runlevel	systemd target	মানে
0	`poweroff.target`	বন্ধ
1, s, single	`rescue.target`	single-user, রক্ষণাবেক্ষণ
2, 3, 4	`multi-user.target`	মাল্টি-ইউজার, নেটওয়ার্ক, কিন্তু GUI নেই
5	`graphical.target`	মাল্টি-ইউজার + GUI
6	`reboot.target`	রিবুট
—	`emergency.target`	একদম ন্যূনতম শেল

```
$ systemctl get-default
graphical.target
```

```
$ sudo systemctl set-default multi-user.target     # পরের বুট থেকে GUI ছাড়া
Removed "/etc/systemd/system/default.target".
Created symlink /etc/systemd/system/default.target → /usr/lib/systemd/system/multi-
user.target.

$ sudo systemctl isolate multi-user.target          # এখনই GUI বন্ধ করে টেক্সট মোডে
$ sudo systemctl isolate graphical.target           # আবার GUI

$ runlevel                                          # পুরনো কমান্ড, এখনো কাজ করে
N 5
```

`runlevel`-এর আউটপুটে প্রথমটা আগের runlevel (`N` মানে নেই), দ্বিতীয়টা বর্তমান।

> ⚠️ **সতর্কতা:** `isolate` বর্তমান target-এ না থাকা সব unit বন্ধ করে দেয়। ডেস্কটপ থেকে `isolate multi-user.target` চালালে তোমার GUI সেশন, খোলা ফাইলসহ, সাথে সাথে বন্ধ হয়ে যাবে।

বুটে কত সময় লাগল: `systemd-analyze`

```
$ systemd-analyze
Startup finished in 6.512s (firmware) + 2.104s (loader) + 1.876s (kernel) + 3.951s
(initrd) + 8.230s (userspace) = 22.674s
graphical.target reached after 8.201s in userspace.
```

এই এক লাইনেই পুরো বুট প্রসেসের ধাপগুলো দেখা যাচ্ছে: firmware → loader (GRUB) → kernel → initrd → userspace (systemd)। (firmware ও loader-এর সময় শুধু তখনই দেখায় যখন মেশিন UEFI আর bootloader সেই সময়ের তথ্য systemd-কে জানায়, যেমন systemd-boot। অনেক GRUB সেটআপে এই দুটো অংশ থাকে না, শুধু kernel + initrd + userspace দেখায়।)

কমান্ড	কাজ
`systemd-analyze`	মোট বুট সময়, ধাপভিত্তিক
`systemd-analyze blame`	কোন unit কত সময় নিল, বেশি থেকে কম
`systemd-analyze critical-chain`	যে ধারাবাহিক শিকলটা বুটকে সবচেয়ে বেশি আটকে রেখেছে
`systemd-analyze critical-chain nginx.service`	নির্দিষ্ট unit পর্যন্ত শিকল
`systemd-analyze plot > boot.svg`	পুরো বুটের টাইমলাইন ছবি (ব্রাউজারে খোলো)
`systemd-analyze verify x.service`	unit ফাইলে ভুল আছে কি না

```
$ systemd-analyze blame | head -5
5.102s NetworkManager-wait-online.service
1.401s snapd.seeded.service
  812ms dev-nvme0n1p3.device
  640ms systemd-journal-flush.service
  402ms udisks2.service
```

```
$ systemd-analyze critical-chain
graphical.target @8.201s
└─multi-user.target @8.199s
  └─snapd.seeded.service @6.790s +1.401s
    └─basic.target @2.110s
      └─sockets.target @2.109s
...
```

`@` এর পরের সময় হলো unit কখন সক্রিয় হলো, `+` এর পরেরটা নিজে চালু হতে কত সময় নিল।

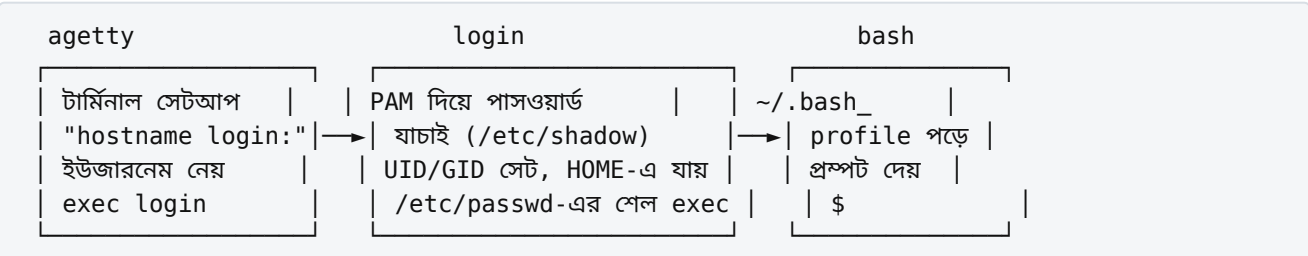
> 💡 **টিপস:** `blame`-এ শীর্ষে থাকা unit সবসময় বুট ধীর করে না, কারণ অনেক unit সমান্তরালে চলে। আসল বাধা খুঁজতে `critical-chain` দেখো। ডেস্কটপে `NetworkManager-wait-online.service` প্রায়ই অপ্রয়োজনীয়। কোনো service-এর সত্যিই নেটওয়ার্ক-অপেক্ষার দরকার না থাকলে সেটা `disable` করা যায়।

২৭.৯ ধাপ ৭: লগইন — getty ও display manager

টেক্সট লগইন: getty → login → shell

লিনাক্সে কয়েকটি **virtual console** (TTY) থাকে: `tty1`, `tty2` … `Ctrl+Alt+F1` … `F6` দিয়ে এগুলোর মধ্যে যাওয়া যায়। প্রতিটি কনসোলে লগইন প্রম্পট দেখায় **getty** (get teletype) প্রোগ্রাম, আধুনিক লিনাক্সে এর নাম **agetty**।

systemd এগুলো চালায় একটা টেমপ্লেট unit `getty@.service` দিয়ে: `getty@tty1.service`, `getty@tty2.service` ইত্যাদি। মজার ব্যাপার, systemd সব কনসোলে আগে থেকে getty চালায় না। তুমি `Ctrl+Alt+F3` চাপলে তখনই `getty@tty3.service` চালু হয় (`systemd-logind`-এর autovt ব্যবস্থা)। সিরিয়াল কনসোলের জন্য আছে `serial-getty@ttyS0.service`।



প্রতিটি ধাপ `exec` করে পরের প্রোগ্রামে বদলে যায়, তাই PID একই থাকে। তুমি shell থেকে `exit` করলে প্রসেস শেষ হয়, systemd সেটা দেখে আবার নতুন getty চালায় (unit-এ `Restart=always`)। এজন্যই লগআউটের পর আবার লগইন প্রম্পট আসে।

```
$ systemctl status getty@tty1.service
● getty@tty1.service - Getty on tty1
     Loaded: loaded (/usr/lib/systemd/system/getty@.service; enabled; preset: enabled)
```

```
     Active: active (running) since Mon 2026-11-09 09:12:03 +06; 2h ago
   Main PID: 1032 (agetty)
     CGroup: /system.slice/system-getty.slice/getty@tty1.service
             └─1032 /sbin/agetty -o "-p -- \\u" --noclear - linux
```

ইউজার, `/etc/passwd`, `/etc/shadow`-এর বিস্তারিত পাবে অধ্যায় ১৫-এ। লগইনের পর কোন কোন কনফিগ ফাইল পড়া হয় তা অধ্যায় ৪৪-এ।

গ্রাফিক্যাল লগইন: display manager

`graphical.target`-এ পৌঁছালে চালু হয় **display manager** (DM), যেটা গ্রাফিক্যাল লগইন স্ক্রিন দেখায়:

Display manager	সাধারণত কোথায়
GDM (GNOME Display Manager)	Ubuntu, Fedora Workstation, RHEL, Debian GNOME
SDDM	KDE Plasma (Kubuntu, Fedora KDE)
LightDM	Xfce, Linux Mint, Xubuntu

যেটাই ব্যবহার হোক, systemd তাকে `display-manager.service` নামে চেনে (একটা alias symlink):

```
$ ls -l /etc/systemd/system/display-manager.service
lrwxrwxrwx 1 root root 32 Jan 10  2026 /etc/systemd/system/display-manager.service ->
/usr/lib/systemd/system/gdm.service

$ systemctl status display-manager
```

DM পাসওয়ার্ড যাচাই করে (PAM দিয়ে), `systemd-logind`-এর কাছে একটা session নিবন্ধন করে, তারপর তোমার ডেস্কটপ (GNOME/KDE) চালায় Wayland বা X11-এ। বিস্তারিত অধ্যায় ৪-এ।

> 📝 **মনে রাখো:** Ubuntu-তে GDM-এর লগইন স্ক্রিন সাধারণত `tty1`-এ থাকে আর লগইনের পর তোমার ডেস্কটপ সেশন `tty2`-তে চলে (Fedora-তেও একই রকম)। তাই GUI জমে গেলে `Ctrl+Alt+F3` চেপে টেক্সট কনসোলে লগইন করে সমস্যা দেখতে পারো, তারপর `Ctrl+Alt+F2` (বা F1) চেপে GUI-তে ফেরো।

২৭.১০ বুট লগ পড়া: `dmesg` ও `journalctl -b`

`dmesg` — kernel-এর বার্তা

`dmesg` (display message) kernel ring buffer পড়ে। বুটের সময় হার্ডওয়্যার শনাক্ত, ড্রাইভার লোড, ডিস্ক এরর, USB লাগানো-খোলা সবই এখানে আসে।

```
$ sudo dmesg | head -5
[    0.000000] Linux version 6.8.0-45-generic (buildd@lcy02-amd64-075) (x86_64-linux-
```

```
gnu-gcc-13 ...) #45-Ubuntu SMP PREEMPT_DYNAMIC ...
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-45-generic
root=UUID=7e1c... ro quiet splash vt.handoff=7
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
```

বাঁদিকের `[ 0.000000]` হলো kernel চালু হওয়ার পর থেকে কত সেকেন্ড গেছে।

> 📝 **মনে রাখো:** Ubuntu ও Debian-এ `kernel.dmesg_restrict=1`, তাই সাধারণ ইউজার `dmesg` চালালে `dmesg: read kernel buffer failed: Operation not permitted` আসে। `sudo` লাগবে।

অপশন	কাজ
`-T` / `--ctime`	সময় মানুষের পড়ার মতো তারিখ-সময়ে দেখাও (suspend/resume হলে একটু ভুল হতে পারে)
`-H` / `--human`	রঙিন, pager-সহ, পড়তে সহজ
`-w` / `--follow`	নতুন বার্তা আসলে সাথে সাথে দেখাও (USB লাগিয়ে দেখো)
`-l err,warn` / `--level`	শুধু নির্দিষ্ট লেভেল (emerg, alert, crit, err, warn, notice, info, debug)
`-x` / `--decode`	facility ও level লেখায় দেখাও
`-k` / `--kernel`	শুধু kernel-এর বার্তা
`-c` / `--read-clear`	পড়ে বাফার খালি করো
`-C` / `--clear`	বাফার খালি করো

```
$ sudo dmesg -T -l err,warn
[Mon Nov  9 09:11:58 2026] ACPI BIOS Error (bug): Could not resolve symbol
[\_SB.PC00.LPCB.HEC.ECAV], AE_NOT_FOUND
[Mon Nov  9 09:12:00 2026] nvme nvme0: missing or invalid SUBNQN field.

$ sudo dmesg -w
[ 9310.551204] usb 3-2: new high-speed USB device number 5 using xhci_hcd
[ 9310.703118] usb-storage 3-2:1.0: USB Mass Storage device detected
[ 9311.731902] sd 0:0:0:0: [sdb] 60620800 512-byte logical blocks: (31.0 GB/28.9 GiB)
[ 9311.745320]  sdb: sdb1
```

কাজের কিছু খোঁজ:

```
$ sudo dmesg | grep -iE 'error|fail|firmware'   # সমস্যা
$ sudo dmesg | grep -i 'secure boot'            # Secure Boot চালু কি না
$ sudo dmesg | grep -i oom                       # মেমরি শেষ হয়ে প্রসেস মারা হয়েছে কি না
```

> ⚠️ **সতর্কতা:** ring buffer-এর আকার সীমিত (সাধারণত কয়েকশো KB থেকে কয়েক MB)। অনেকদিন চলা সিস্টেমে বুটের প্রথম দিকের বার্তা মুছে যেতে পারে। আর রিবুট করলে `dmesg`-এর পুরনো বার্তা চলে যায়।

> আগের বুটের kernel বার্তার জন্য `journalctl` লাগবে।

`journalctl -b` — পুরো বুটের লগ

systemd-এর journal kernel-এর বার্তা আর সব service-এর বার্তা একসাথে রাখে। বিস্তারিত অধ্যায় ২৮-এ। বুটের জন্য দরকারি অংশ:

কমান্ড	কাজ
`journalctl -b`	বর্তমান বুটের সব লগ
`journalctl -b -1`	আগের বুটের লগ (`-2` মানে তার আগের)
`journalctl --list-boots`	সব সংরক্ষিত বুটের তালিকা
`journalctl -k` / `--dmesg`	বর্তমান বুটের শুধু kernel বার্তা (dmesg-এর মতো)
`journalctl -k -b -1`	আগের বুটের kernel বার্তা
`journalctl -b -p err`	বর্তমান বুটের শুধু error ও তার চেয়ে গুরুতর
`journalctl -b -u ssh.service`	নির্দিষ্ট service-এর বুট লগ

```
$ journalctl --list-boots
IDX BOOT ID                          FIRST ENTRY                 LAST ENTRY
 -2 3c5e1a7f0b5d4c4f8a9d2e0c1b7a6f55 Sat 2026-11-07 08:01:12 +06 Sat 2026-11-07
23:40:02 +06
 -1 9a0d2e4c7b3f4a1e9d5c8b2a7f6e1d34 Sun 2026-11-08 08:30:44 +06 Sun 2026-11-08
22:15:10 +06
  0 f1e2d3c4b5a6478899aabbccddeeff00 Mon 2026-11-09 09:11:56 +06 Mon 2026-11-09
11:20:31 +06
```

> 💡 **টিপস:** "গতকাল মেশিন হঠাৎ বন্ধ হয়ে গেল কেন?" জানতে `journalctl -b -1 -e` চালাও (`-e` মানে শেষে চলে যাও)। আগের বুটের শেষ বার্তাগুলোতে কারণের সূত্র থাকে। এজন্য journal **persistent** হতে হবে, মানে `/var/log/journal/` ডিরেক্টরি থাকতে হবে। Ubuntu, Debian 12, Fedora, RHEL 9-এ ডিফল্টে এটা থাকে। না থাকলে `sudo mkdir -p /var/log/journal && sudo systemctl restart systemd-journald` চালাও।

২৭.১১ Rescue mode ও Emergency mode

কখনো সিস্টেম স্বাভাবিকভাবে বুট হয় না, বা কোনো ভুল কনফিগ ঠিক করতে ন্যূনতম পরিবেশ লাগে। এর জন্য systemd-এর দুটি বিশেষ target আছে।

	`rescue.target`	`emergency.target`
পুরনো নাম	single-user mode, runlevel 1	—

	`rescue.target`	`emergency.target`
কী কী মাউন্ট হয়	সব লোকাল ফাইলসিস্টেম (`/etc/fstab` থেকে)	শুধু root, তাও **read-only**
কোন service চলে	`sysinit.target` পর্যন্ত, মৌলিক service	প্রায় কিছুই না
নেটওয়ার্ক	না	না
কখন ব্যবহার	service বা কনফিগের সমস্যা, পাসওয়ার্ড বদল	`/etc/fstab` ভুল, ফাইলসিস্টেম নষ্ট, rescue-ও আটকে গেলে
kernel parameter	`systemd.unit=rescue.target` (বা `single`, `1`)	`systemd.unit=emergency.target` (বা `emergency`)
চলমান সিস্টেম থেকে	`sudo systemctl rescue`	`sudo systemctl emergency`

> 💡 **উপমা:** rescue mode হলো দোকানের শাটার অর্ধেক তুলে ভেতরে মেরামত করা: আলো-পাখা চলছে, কিন্তু খদ্দের ঢুকতে পারছে না। emergency mode হলো বিদ্যুৎ ছাড়া টর্চ হাতে ঢোকা: শুধু সবচেয়ে জরুরি কাজটুকু।

দুই মোডেই systemd `sulogin` চালায়, যেটা **root-এর পাসওয়ার্ড** চায়:

```
You are in emergency mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, "systemctl default" or "exit"
to boot into default mode.
Give root password for maintenance
(or press Control-D to continue):
```

এখানে:

```
# journalctl -xb                  # কী ভুল হয়েছে দেখো
# mount -o remount,rw /           # (emergency-তে) root লেখার যোগ্য করো
# nano /etc/fstab                 # ভুল লাইন ঠিক করো বা # দিয়ে কমেন্ট করো
# systemctl daemon-reload
# systemctl default               # স্বাভাবিক বুট চালিয়ে যাও (বা exit)
```

> ⚠️ **সতর্কতা (Ubuntu):** Ubuntu-তে ডিফল্টে root অ্যাকাউন্ট **লক** থাকে (পাসওয়ার্ড নেই, `sudo` ব্যবহার হয়)। তখন `sulogin` বলে `Cannot open access to console, the root account is locked.` আর Enter চাপলে আবার চেষ্টা করে। এ অবস্থায় rescue/emergency কাজে আসে না। Ubuntu-র GRUB-এর "Advanced options → (recovery mode)" entry এর বদলে একটা মেনু দেয় (`root — Drop to root shell prompt` সহ), যেখানে পাসওয়ার্ড লাগে না। সেটাও না চললে নিচের `init=/bin/bash` পদ্ধতি ব্যবহার করো।

> 📝 **মনে রাখো:** `/etc/fstab`-এ ভুল UUID থাকা emergency mode-এর সবচেয়ে প্রচলিত কারণ। বাইরের/USB ডিস্কের লাইনে `nofail` অপশন দিলে ডিস্ক না থাকলেও বুট আটকাবে না। fstab-এর বিস্তারিত অধ্যায় ৩২-এ।

Ubuntu-র recovery mode মেনু

GRUB → Advanced options for Ubuntu → `... (recovery mode)` বেছে নিলে:

```
Recovery Menu (filesystem state: read-only)
resume      Resume normal boot
clean       Try to make free space
dpkg        Repair broken packages
fsck        Check all file systems
grub        Update grub bootloader
network     Enable networking
root        Drop to root shell prompt
system-summary   System summary
```

`root` বেছে নিলে root শেল পাবে। ফাইলসিস্টেম read-only থাকে, তাই কিছু বদলাতে প্রথমে `mount -o remount,rw /` চালাতে হয়।

২৭.১২ ভুলে যাওয়া root (বা ইউজার) পাসওয়ার্ড রিসেট

শুরুতেই একটা সত্য জেনে রাখো: **যার কাছে মেশিনের শারীরিক অ্যাক্সেস (physical access) আছে, সে-ই আসলে root**, যদি না ডিস্ক এনক্রিপ্টেড থাকে বা GRUB/firmware-এ পাসওয়ার্ড থাকে। এই পদ্ধতিগুলো সেই সত্যেরই প্রমাণ। শুধু নিজের বা অনুমতিপ্রাপ্ত মেশিনে ব্যবহার করবে।

> 📝 **মনে রাখো:** ডিস্ক LUKS দিয়ে এনক্রিপ্টেড হলে এই পদ্ধতিতেও আগে ডিস্কের passphrase দিতে হবে। passphrase ছাড়া ডেটায় পৌঁছানো যাবে না, আর এটাই এনক্রিপশনের উদ্দেশ্য।

পদ্ধতি ক: `init=/bin/bash` (Ubuntu, Debian, প্রায় সব ডিস্ট্রো)

ধারণাটা হলো: kernel-কে বলব systemd-এর বদলে সরাসরি bash চালাও। তখন কোনো লগইন নেই, পাসওয়ার্ড যাচাই নেই, আর bash চলবে root হিসেবে, কারণ PID 1 সবসময় root।

1. রিবুট করো, GRUB মেনু আনো (`Shift` বা `Esc`)।
2. সাধারণ entry-তে দাঁড়িয়ে `e` চাপো।

3. `linux` দিয়ে শুরু হওয়া লাইনটা খুঁজে নাও। সেখানে `ro quiet splash $vt_handoff` অংশটা মুছে লেখো `rw init=/bin/bash`:

```
linux /boot/vmlinuz-6.8.0-45-generic root=UUID=7e1c... rw init=/bin/bash
```

4. `Ctrl+x` বা `F10` চাপো। কয়েক সেকেন্ডে root প্রম্পট আসবে:

```
root@(none):/#
```

5. root লেখার যোগ্য কি না নিশ্চিত হও (`rw` দিলে সাধারণত হয়ে যায়):

```
# mount -o remount,rw /
```

6. পাসওয়ার্ড বদলাও:

```
# passwd root            # root-এর জন্য
# passwd nishat          # বা নির্দিষ্ট ইউজারের জন্য (Ubuntu-তে এটাই বেশি দরকারি)
New password:
Retype new password:
passwd: password updated successfully
```

ইউজারনেম মনে না থাকলে: `ls /home` বা `grep -E ':[0-9]{4}:' /etc/passwd`।

7. ডিস্কে লেখা নিশ্চিত করে রিবুট করো:

```
# sync
# exec /sbin/init
```

`exec /sbin/init` bash-কে systemd দিয়ে প্রতিস্থাপন করে স্বাভাবিক বুট চালু করে। এখানে সাধারণ `reboot` কমান্ড কাজ নাও করতে পারে, কারণ systemd চলছে না। চাইলে `sync` এর পর `reboot -f` দিয়েও জোর করে রিবুট করা যায়।

> ⚠️ **সতর্কতা:** SELinux চালু থাকা সিস্টেমে (Fedora/RHEL) এই পদ্ধতিতে `/etc/shadow` ফাইলের SELinux লেবেল ভুল হয়ে যায়, ফলে পরে কেউ লগইন করতে পারবে না। তাই সেখানে পদ্ধতি খ ব্যবহার করো, বা পদ্ধতি ক-এর পর `touch /.autorelabel` করো।

পদ্ধতি খ: `rd.break` (RHEL 9, Fedora, Rocky, Alma)

dracut-এর `rd.break` initramfs-এর ভেতরে, আসল root-এ যাওয়ার ঠিক আগে একটা শেল দেয়। তখন আসল root `/sysroot`-এ read-only মাউন্ট থাকে।

1. রিবুট করো, GRUB মেনুতে entry-তে দাঁড়িয়ে `e` চাপো।

2. `linux` লাইনের শেষে যোগ করো `rd.break`। (ক্লাউড/VM-এ সিরিয়াল কনসোল থাকলে `console=` parameter-গুলো সরাতে হতে পারে, যাতে শেল স্ক্রিনেই আসে।)

3. `Ctrl+x` চাপো। প্রম্পট আসবে:

```
switch_root:/#
```

4. নিচের ধাপগুলো ক্রমানুসারে:

```
switch_root:/# mount -o remount,rw /sysroot
switch_root:/# chroot /sysroot
sh-5.1# passwd root
Changing password for user root.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
sh-5.1# touch /.autorelabel
sh-5.1# exit
switch_root:/# exit
```

ধাপ	কেন
`mount -o remount,rw /sysroot`	আসল root এখন read-only, লিখতে হলে rw করতে হবে
`chroot /sysroot`	আসল সিস্টেমকে "/" বানিয়ে তার `passwd` আর `/etc/shadow` ব্যবহার করা
`passwd root`	পাসওয়ার্ড বদল
`touch /.autorelabel`	পরের বুটে SELinux পুরো ফাইলসিস্টেমের লেবেল ঠিক করবে, `/etc/shadow`-সহ
`exit` দুবার	প্রথমটা chroot থেকে, দ্বিতীয়টা dracut শেল থেকে, বুট চালিয়ে যায়

> 💡 **টিপস:** `/.autorelabel`-এর কারণে পরের বুটে কয়েক মিনিট (বড় ডিস্কে আরও বেশি) সময় লাগবে আর মেশিন নিজে থেকে একবার রিবুট হবে। ভয় পেয়ো না, এটা স্বাভাবিক। দ্রুত বিকল্প: chroot-এর ভেতরে `touch` এর বদলে `load_policy -i` চালিয়ে তারপর `passwd` করো আর শেষে `restorecon -v /etc/shadow` চালাও।

> 📝 **মনে রাখো:** RHEL 9-এর অফিশিয়াল ডকুমেন্টেশনে এটাই "Resetting the root password" পদ্ধতি। RHCSA পরীক্ষার একটা পরিচিত কাজ এটা। RHEL-এর ইনস্টল মিডিয়ার "Troubleshooting → Rescue a Red Hat Enterprise Linux system" দিয়েও একই কাজ করা যায় (`chroot /mnt/sysroot`)।

পদ্ধতি গ: লাইভ USB দিয়ে

GRUB-ও যদি পাসওয়ার্ড-সুরক্ষিত বা নষ্ট হয়:

```
# লাইভ USB থেকে বুট করে টার্মিনালে:
$ lsblk                                    # root পার্টিশন খোঁজো, ধরো /dev/nvme0n1p3
$ sudo mount /dev/nvme0n1p3 /mnt
$ sudo chroot /mnt passwd nishat
$ sudo umount /mnt
$ reboot
```

(LVM হলে আগে `sudo vgchange -ay`, তারপর `/dev/mapper/...` মাউন্ট করো।)

এই "ফাঁক" কীভাবে বন্ধ করবে?

সুরক্ষা	কী আটকায়
GRUB পাসওয়ার্ড (`grub2-setpassword` / `password_pbkdf2`)	মেনুতে `e` বা `c` চেপে এডিট
UEFI/BIOS সেটআপ পাসওয়ার্ড + USB বুট বন্ধ	লাইভ USB দিয়ে বুট
পূর্ণ ডিস্ক এনক্রিপশন (LUKS)	ডিস্ক খুলে অন্য মেশিনে পড়া, chroot
Secure Boot	স্বাক্ষরবিহীন bootloader/kernel চালানো
শারীরিক নিরাপত্তা (তালাবদ্ধ সার্ভার রুম)	সবকিছু

আরও নিরাপত্তা-ব্যবস্থা পাবে অধ্যায় ৪৫-এ।

২৭.১৩ Secure Boot — শুধু বিশ্বস্ত কোড চলবে

সমস্যাটা কী

বুটের একদম শুরুতে, OS চালু হওয়ার আগেই, যদি কোনো ম্যালওয়্যার bootloader বা kernel বদলে দেয় (একে বলে **bootkit** বা **rootkit**), তাহলে OS-এর অ্যান্টিভাইরাস কিছুই ধরতে পারবে না, কারণ ম্যালওয়্যার তার আগেই চালু হয়ে গেছে।

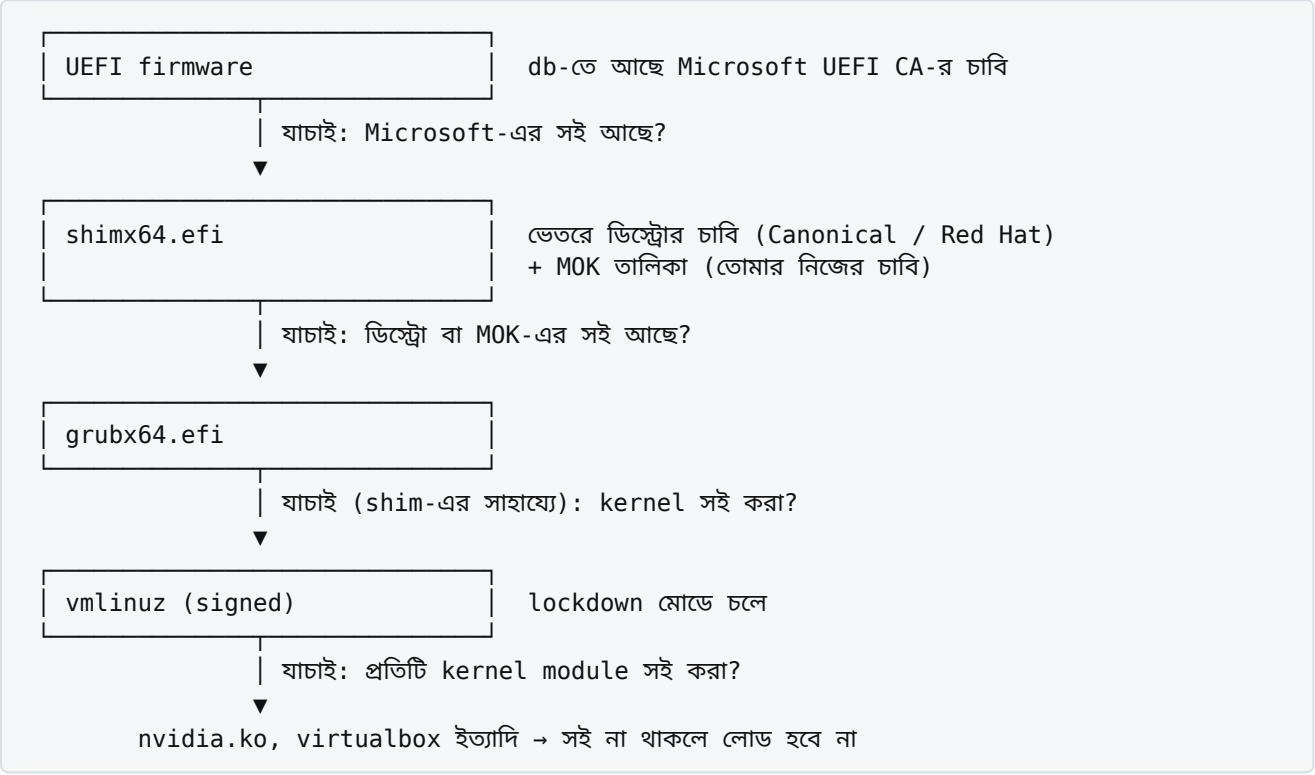
Secure Boot UEFI-র একটা ফিচার, যেটা প্রতিটি ধাপে পরের প্রোগ্রামের **ডিজিটাল স্বাক্ষর** (digital signature) যাচাই করে। স্বাক্ষর বিশ্বস্ত চাবির সাথে না মিললে চালাতে দেয় না। এভাবে তৈরি হয় একটা **chain of trust** (বিশ্বাসের শিকল)।

> 💡 **উপমা:** বিয়েবাড়ির গেটে দারোয়ান শুধু কার্ড দেখে ঢুকতে দেয়। কার্ডে বরের বাবার সই থাকতে হবে। ভেতরে ঢোকা অতিথি আবার নিজের সই করা পাস দিয়ে ড্রাইভারকে ঢোকাতে পারে। প্রত্যেকে পরের জনের জন্য দায়ী।

লিনাক্সে Secure Boot-এর শিকল

UEFI firmware-এ ডিফল্টে প্রায় সবসময় **Microsoft**-এর চাবি থাকে। লিনাক্স ডিস্ট্রোগুলো প্রতিবার নিজের GRUB Microsoft-কে দিয়ে সই করানোর বদলে একটা ছোট মধ্যস্থ প্রোগ্রাম **shim** একবার সই করিয়ে নেয়। shim-

এর ভেতরে ডিস্ট্রোর নিজের চাবি থাকে।



Secure Boot চালু আছে কি না দেখা

```
$ mokutil --sb-state
SecureBoot enabled

$ sudo dmesg | grep -i "secure boot"
[    0.000000] secureboot: Secure boot enabled
[    0.006542] Kernel is locked down from EFI Secure Boot mode; see man
kernel_lockdown.7
```

MOK — Machine Owner Key

ধরো তুমি NVIDIA-র প্রোপ্রাইটারি ড্রাইভার বা VirtualBox ইনস্টল করলে। এগুলোর kernel module তোমার মেশিনেই কম্পাইল হয় (**DKMS** দিয়ে), তাই ডিস্ট্রোর সই থাকে না। Secure Boot সেগুলো লোড হতে দেবে না:

```
$ sudo modprobe vboxdrv
modprobe: ERROR: could not insert 'vboxdrv': Key was rejected by service
```

সমাধান হলো নিজের একটা চাবি বানিয়ে module সই করা, আর সেই চাবিকে বিশ্বস্ত বলে নিবন্ধন করা। এই নিজের চাবিকেই বলে **MOK** (Machine Owner Key)।

Ubuntu-তে DKMS স্বয়ংক্রিয়ভাবে চাবি বানায় (`/var/lib/shim-signed/mok/MOK.der`) আর ড্রাইভার ইনস্টলের সময় একটা পাসওয়ার্ড চায়। হাতে করতে চাইলে:

```
$ sudo mokutil --import /var/lib/shim-signed/mok/MOK.der
input password:
input password again:
$ sudo reboot
```

রিবুটে একটা নীল স্ক্রিন **MOK Manager** (`mmx64.efi`) আসবে: `Enroll MOK` → `Continue` → `Yes` → একটু আগের পাসওয়ার্ড দাও → `Reboot` । এরপর চাবিটা বিশ্বস্ত হবে।

কমান্ড	কাজ
`mokutil --sb-state`	Secure Boot চালু কি না
`mokutil --list-enrolled`	নিবন্ধিত MOK চাবির তালিকা
`mokutil --import key.der`	নতুন চাবি নিবন্ধনের অনুরোধ (পরের বুটে নিশ্চিত করতে হয়)
`mokutil --delete key.der`	চাবি সরানোর অনুরোধ
`mokutil --disable-validation`	shim-এর যাচাই বন্ধ (প্রস্তাবিত না)

> ⚠️ **সতর্কতা:** MOK Manager-এর নীল স্ক্রিনে সময়সীমা থাকে (প্রায় ১০ সেকেন্ড)। কোনো key না চাপলে সেটা চলে যাবে আর চাবি নিবন্ধন হবে না। তখন আবার `mokutil --import` চালাতে হবে।

> 💡 **টিপস:** সমস্যা হলে অনেকে Secure Boot বন্ধ করে দেয়। এটা কাজ করে, কিন্তু নিরাপত্তা কমে। MOK নিবন্ধনই সঠিক পথ। Secure Boot চালু থাকলে kernel **lockdown** মোডে থাকে, যেখানে কিছু নিম্নস্তরের কাজ (যেমন `/dev/mem` -এ লেখা, স্বাক্ষরবিহীন kernel-এ `kexec`) নিষিদ্ধ।

২৭.১৪ বিপরীত দিক: Shutdown ও Reboot

বুটের উল্টো প্রক্রিয়াও systemd সামলায়। `systemctl poweroff` বা `reboot` চালালে systemd `shutdown.target` -এর দিকে যায়: সব service-কে `SIGTERM` পাঠায় (না থামলে নির্দিষ্ট সময় পর, ডিফল্ট ৯০ সেকেন্ড, `SIGKILL`), ফাইলসিস্টেম আনমাউন্ট করে, শেষে kernel-কে বন্ধ বা রিবুট করতে বলে।

কমান্ড	কাজ
`sudo systemctl reboot` বা `sudo reboot`	রিবুট
`sudo systemctl poweroff` বা `sudo poweroff`	বন্ধ
`sudo shutdown -h +10 "রক্ষণাবেক্ষণ"`	১০ মিনিট পর বন্ধ, সবাইকে বার্তা
`sudo shutdown -r now`	এখনই রিবুট
`sudo shutdown -c`	নির্ধারিত shutdown বাতিল

কমান্ড	কাজ
`sudo systemctl reboot --firmware-setup`	রিবুট করে UEFI সেটআপে যাও
`sudo systemctl kexec`	firmware/GRUB এড়িয়ে সরাসরি নতুন kernel-এ রিবুট (kexec-tools লাগে)

> 💡 **টিপস:** shutdown "A stop job is running for ..." বার্তায় অনেকক্ষণ আটকে থাকলে বুঝবে কোনো service থামতে দেরি করছে। পরের বুটে `journalctl -b -1 -e` দেখে কোনটা দেখে নাও।

২৭.১৫ বুট-সংক্রান্ত সব কীবোর্ড শর্টকাট

Firmware পর্যায় (নির্মাতা-ভেদে আলাদা)

কী	কাজ
`Del` / `F2`	UEFI/BIOS সেটআপে ঢোকা (বেশিরভাগ ডেস্কটপ মাদারবোর্ড ও অনেক ল্যাপটপ)
`F12` / `F11` / `F8` / `Esc`	এককালীন বুট মেনু (USB বেছে নিতে)
`F10`	কিছু HP-তে সেটআপ; অনেক সেটআপে "Save & Exit"
`F1`	কিছু Lenovo ThinkPad-এ সেটআপ (`Enter` চেপে interrupt মেনু)
`Option` (Alt) চেপে রাখা	Intel Mac-এ বুট ডিভাইস বেছে নেওয়া

GRUB পর্যায়

কী	কাজ
`Shift` চেপে রাখা	(BIOS) লুকানো GRUB মেনু দেখাও
`Esc`	(UEFI) লুকানো GRUB মেনু দেখাও; এডিট/কমান্ড লাইন থেকে মেনুতে ফেরো
`↑` / `↓`	entry বাছাই
`Enter`	বেছে নেওয়া entry বুট করো
`e`	entry এডিট (সাময়িক)
`Ctrl+x` বা `F10`	এডিট করা entry দিয়ে বুট
`c`	GRUB কমান্ড লাইন (`grub>`)
`Ctrl+c`	এডিট স্ক্রিন থেকে কমান্ড লাইনে
`Tab`	GRUB কমান্ড লাইনে auto-complete
`Ctrl+a` / `Ctrl+e`	এডিটরে লাইনের শুরু / শেষে

কী	কাজ
`Ctrl+k`	এডিটরে কার্সর থেকে লাইনের শেষ পর্যন্ত মুছে ফেলো

Kernel/systemd ও সেশন পর্যায়

কী	কাজ
`Esc` (Plymouth splash চলাকালীন)	গ্রাফিক্যাল বুট স্ক্রিন সরিয়ে টেক্সট বার্তা দেখাও
`Ctrl+Alt+F1` … `F6`	virtual console (TTY) বদলাও; GUI থেকে টেক্সট কনসোলে
`Alt+F1` … `F6` / `Alt+←` `Alt+→`	টেক্সট কনসোলে থাকা অবস্থায় অন্য TTY-তে
`Ctrl+Alt+Del`	(টেক্সট কনসোলে) `ctrl-alt-del.target` = রিবুট; ২ সেকেন্ডে ৭ বারের বেশি চাপলে জোর করে তাৎক্ষণিক রিবুট
`Ctrl+D`	sulogin প্রম্পটে স্বাভাবিক বুট চালিয়ে যাও / শেল থেকে বেরোও
`Alt+SysRq+R,E,I,S,U,B`	"Magic SysRq": জমে যাওয়া সিস্টেম নিরাপদে রিবুট (raw keyboard, SIGTERM, SIGKILL, Sync, read-only remount, reBoot)

> 📝 **মনে রাখো (SysRq):** কোন SysRq কাজ চালু থাকবে তা ঠিক করে `kernel.sysrq` মান। Ubuntu-তে ডিফল্ট মান `176` (= 16 sync + 32 remount read-only + 128 reboot), মানে শুধু `S`, `U`, `B` কাজ করে; `R`, `E`, `I` বন্ধ। Fedora/RHEL-এ ডিফল্ট `16` (শুধু sync)। দেখতে: `cat /proc/sys/kernel/sysrq`। ল্যাপটপে SysRq সাধারণত `PrtSc` কী, কখনো `Fn` লাগে।

হাতে-কলমে অনুশীলন

> ⚠️ **সতর্কতা:** নিচের ল্যাবগুলো করার জন্য সবচেয়ে ভালো জায়গা একটা ভার্চুয়াল মেশিন (VirtualBox, GNOME Boxes, virt-manager)। VM-এ snapshot নিয়ে রাখো, ভুল হলে ফিরিয়ে আনা যাবে। ল্যাব ৪–৬ কখনো প্রোডাকশন সার্ভারে করবে না।

ল্যাব ১: তোমার বুট পরিচিতি তৈরি করো

```
$ [ -d /sys/firmware/efi ] && echo UEFI || echo BIOS
$ lsblk -o NAME,SIZE,PTTYPE,FSTYPE,MOUNTPOINTS
$ findmnt /boot/efi
$ ls -lh /boot
$ uname -r
$ cat /proc/cmdline
```

```
$ ls -l /sbin/init
$ systemctl get-default
```

প্রতিটির উত্তর একটা খাতায় লেখো: firmware মোড, পার্টিশন টেবিল, ESP কোথায় মাউন্ট, kernel ভার্সন, root কোন UUID, ডিফল্ট target।

ল্যাব ২: বুট সময় বিশ্লেষণ

```
$ systemd-analyze
$ systemd-analyze blame | head -10
$ systemd-analyze critical-chain
$ systemd-analyze plot > ~/boot.svg
$ xdg-open ~/boot.svg         # ব্রাউজারে খুলবে
```

প্রশ্ন: কোন ধাপে (firmware/loader/kernel/initrd/userspace) সবচেয়ে বেশি সময় যাচ্ছে? critical chain-এ কোন unit সবচেয়ে বেশি `+` সময় নিচ্ছে?

ল্যাব ৩: বুট লগ খোঁজা

```
$ sudo dmesg -T | head -20
$ sudo dmesg -T -l err,warn
$ journalctl --list-boots
$ journalctl -b -p err --no-pager
$ journalctl -k -b -1 | tail -20      # আগের বুটের শেষ kernel বার্তা
```

একটা USB ড্রাইভ লাগিয়ে আরেকটা টার্মিনালে দেখো:

```
$ sudo dmesg -wT
```

ল্যাব ৪: GRUB মেনু দৃশ্যমান করা ও timeout বদলানো (Ubuntu/Debian)

```
$ sudo cp /etc/default/grub /etc/default/grub.bak
$ sudo nano /etc/default/grub
```

এই দুটো লাইন এভাবে করো:

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=10
```

তারপর:

```
$ sudo update-grub
$ grep -E 'timeout|style' /boot/grub/grub.cfg | head
$ sudo reboot
```

রিবুটে ১০ সেকেন্ড মেনু দেখা যাবে। Fedora/RHEL-এ শেষ ধাপে `sudo grub2-mkconfig -o /boot/grub2/grub.cfg`।

ল্যাব ৫: সাময়িক kernel parameter আর বিভিন্ন target

1. রিবুট করে GRUB মেনুতে `e` চাপো।
2. `linux` লাইন থেকে `quiet splash` (বা `rhgb quiet`) মুছে শেষে লেখো `systemd.unit=multi-user.target`।
3. `Ctrl+x`। এবার সব বুট বার্তা দেখবে আর GUI ছাড়া টেক্সট লগইন পাবে।
4. লগইন করে যাচাই করো:

```
$ cat /proc/cmdline
$ systemctl is-active graphical.target
inactive
$ sudo systemctl isolate graphical.target     # GUI-তে ফেরো
```

5. আবার রিবুট করো। এবার পরিবর্তন ছাড়াই স্বাভাবিক বুট হবে, কারণ GRUB-এ এডিট ছিল সাময়িক।

ল্যাব ৬: emergency mode-এ fstab মেরামত (শুধু VM-এ!)

1. snapshot নাও। তারপর `/etc/fstab`-এ ইচ্ছে করে একটা ভুল লাইন যোগ করো:

```
$ sudo cp /etc/fstab /etc/fstab.bak
$ echo 'UUID=00000000-dead-beef-0000-000000000000 /mnt/fake ext4 defaults 0 2' |
sudo tee -a /etc/fstab
$ sudo systemctl daemon-reload
$ sudo reboot
```

2. প্রায় ৯০ সেকেন্ড অপেক্ষার পর emergency mode আসবে। root পাসওয়ার্ড দাও (Ubuntu-তে root লক থাকলে recovery mode → root, বা `init=/bin/bash` ব্যবহার করো)।
3. মেরামত:

```
# journalctl -xb | grep -i fake
# mount -o remount,rw /
# sed -i '/dead-beef/d' /etc/fstab
# systemctl daemon-reload
# systemctl default
```

4. শিক্ষা: বাইরের ডিস্কের লাইনে `nofail` দিলে এমন হতো না।

ল্যাব ৭: ইউজারের পাসওয়ার্ড রিসেট (শুধু VM-এ)

- Ubuntu/Debian VM-এ ২৭.১২-এর পদ্ধতি ক (`rw init=/bin/bash`) দিয়ে তোমার ইউজারের পাসওয়ার্ড বদলাও।
- RHEL/Rocky/Alma/Fedora VM থাকলে পদ্ধতি খ (`rd.break`) দিয়ে root পাসওয়ার্ড বদলাও, আর খেয়াল করো `/.autorelabel`-এর কারণে পরের বুট কতক্ষণ নেয়।

ল্যাব ৮: initramfs অনুসন্ধান

```
# Ubuntu/Debian
$ lsinitramfs /boot/initrd.img-$(uname -r) | wc -l
$ lsinitramfs /boot/initrd.img-$(uname -r) | grep -E '/init$|busybox|cryptsetup|lvm'
$ sudo update-initramfs -u -v | tail -5

# Fedora/RHEL
$ sudo lsinitrd -m
$ sudo dracut -f -v 2>&1 | tail -5
```

প্রশ্ন: তোমার initramfs-এ কয়টি ফাইল আছে? `cryptsetup` আছে কি না, আর থাকলে কেন?

ল্যাব ৯: Secure Boot ও UEFI entry

```
$ mokutil --sb-state
$ efibootmgr -v
$ sudo mokutil --list-enrolled | grep -i "subject:"
```

খেয়াল করো: তোমার লিনাক্স entry কোন `.efi` ফাইল চালায়? সেটা `shimx64.efi` হলে বুঝবে Secure Boot-এর শিকল shim দিয়ে শুরু।

সারসংক্ষেপ

বুটের ধাপগুলো এক লাইনে:

Power → Firmware (POST) → MBR/ESP → GRUB2 → kernel → initramfs → systemd (PID 1) → default.target → getty/display manager → login → shell/desktop

দ্রুত রেফারেন্স

কাজ	কমান্ড / শর্টকাট
UEFI না BIOS	`[ -d /sys/firmware/efi ] && echo UEFI`
UEFI boot entry	`efibootmgr -v`
UEFI সেটআপে রিবুট	`sudo systemctl reboot --firmware-setup`

কাজ	কমান্ড / শর্টকাট
পার্টিশন টেবিলের ধরন	`lsblk -o NAME,PTTYPE` / `sudo fdisk -l`
GRUB সেটিংস	`/etc/default/grub`
grub.cfg পুনর্গঠন (Debian/Ubuntu)	`sudo update-grub`
grub.cfg পুনর্গঠন (Fedora/RHEL)	`sudo grub2-mkconfig -o /boot/grub2/grub.cfg`
kernel parameter (RHEL/Fedora)	`sudo grubby --update-kernel=ALL --args="..."`
ডিফল্ট kernel (RHEL/Fedora)	`sudo grubby --default-kernel`
GRUB পুনঃস্থাপন	`sudo grub-install ...`
বর্তমান kernel parameter	`cat /proc/cmdline`
kernel ভার্সন	`uname -r`
initramfs (Debian/Ubuntu)	`sudo update-initramfs -u [-k all]`, `lsinitramfs`
initramfs (Fedora/RHEL)	`sudo dracut -f [--regenerate-all]`, `lsinitrd`
ডিফল্ট target	`systemctl get-default` / `sudo systemctl set-default ...`
এখনই target বদল	`sudo systemctl isolate multi-user.target`
rescue / emergency	`systemd.unit=rescue.target` / `systemd.unit=emergency.target`
বুট সময়	`systemd-analyze`, `blame`, `critical-chain`, `plot`
kernel বার্তা	`sudo dmesg -T`, `-l err,warn`, `-w`
বুট লগ	`journalctl -b`, `-b -1`, `--list-boots`, `-k`
পাসওয়ার্ড রিসেট (Debian/Ubuntu)	GRUB `e` → `rw init=/bin/bash` → `passwd` → `exec /sbin/init`
পাসওয়ার্ড রিসেট (RHEL/Fedora)	GRUB `e` → `rd.break` → remount rw `/sysroot` → `chroot` → `passwd` → `touch /.autorelabel`
Secure Boot অবস্থা	`mokutil --sb-state`
MOK নিবন্ধন	`sudo mokutil --import key.der` → রিবুট → Enroll MOK
GRUB মেনু দেখাও	`Shift` (BIOS) / `Esc` (UEFI)
GRUB এডিট ও বুট	`e` → `Ctrl+x` বা `F10`
GRUB কমান্ড লাইন	`c`
TTY বদল	`Ctrl+Alt+F1` … `F6`

নিজেকে যাচাই করো

1. BIOS আর UEFI বুটের মূল পার্থক্য কী? MBR-এ bootloader-এর জন্য কত byte জায়গা থাকে?
2. MBR-এর তুলনায় GPT-র তিনটি সুবিধা বলো।
3. ESP কী, এর ফাইলসিস্টেম কী, আর Ubuntu/RHEL-এ এটা কোথায় মাউন্ট হয়?
4. `/boot/grub/grub.cfg` কেন সরাসরি এডিট করা উচিত না? স্থায়ীভাবে একটা kernel parameter যোগ করার সঠিক পদ্ধতি Ubuntu আর RHEL 9-এ কী?
5. initramfs কেন দরকার? এমন দুটি পরিস্থিতি বলো যেখানে initramfs ছাড়া বুট হবে না।
6. `rescue.target` আর `emergency.target`-এর পার্থক্য কী? `/etc/fstab`-এ ভুল UUID থাকলে সাধারণত কোনটায় গিয়ে পড়বে?
7. `systemd-analyze blame` আর `systemd-analyze critical-chain`-এর মধ্যে বুট ধীর হওয়ার কারণ খুঁজতে কোনটা বেশি নির্ভরযোগ্য, আর কেন?
8. RHEL 9-এ root পাসওয়ার্ড রিসেটে `touch /.autorelabel` কেন লাগে?
9. রিবুটের পর আগের বুটের kernel বার্তা `dmesg`-এ পাওয়া যায় না কেন? তাহলে কীভাবে দেখবে?
10. Secure Boot চালু থাকা অবস্থায় VirtualBox-এর module লোড করতে গিয়ে `Key was rejected by service` আসছে। সমস্যাটা কী আর সঠিক সমাধান কী?

▼ উত্তর দেখো

1. BIOS ফাইলসিস্টেম চেনে না। সে ডিস্কের প্রথম 512-byte সেক্টর (MBR) অন্ধভাবে চালায়। UEFI নিজেই FAT32 পড়ে ESP থেকে `.efi` ফাইল চালায়, আর কোন ফাইল চালাবে তা NVRAM-এর boot entry বলে দেয়। এছাড়া UEFI-তে Secure Boot আছে। MBR-এ bootstrap code-এর জন্য 446 byte থাকে, তাই GRUB-কে বাকি অংশ (core.img) অন্য জায়গায় রাখতে হয়।

2. যেকোনো তিনটি: (ক) 2 TiB-এর চেয়ে বড় ডিস্ক সাপোর্ট করে; (খ) ডিফল্টে 128টি পার্টিশন, extended/logical-এর দরকার নেই; (গ) ডিস্কের শেষে ব্যাকআপ header ও পার্টিশন টেবিল থাকে; (ঘ) CRC32 দিয়ে নষ্ট হওয়া ধরা যায়; (ঙ) প্রতিটি পার্টিশনের অনন্য GUID থাকে।

3. ESP (EFI System Partition) হলো একটা FAT32 পার্টিশন, যেখানে UEFI firmware বুটযোগ্য `.efi` ফাইল (shim, GRUB, Windows Boot Manager) খোঁজে। Ubuntu, Debian, Fedora, RHEL-এ এটা `/boot/efi`-তে মাউন্ট হয়।

4. কারণ এটা স্ক্রিপ্ট দিয়ে স্বয়ংক্রিয়ভাবে তৈরি হয়। পরের kernel আপডেট বা `update-grub`-এ হাতের পরিবর্তন মুছে যাবে। Ubuntu-তে: `/etc/default/grub`-এ `GRUB_CMDLINE_LINUX_DEFAULT` (বা `GRUB_CMDLINE_LINUX`) বদলে `sudo update-grub`। RHEL 9-এ: `sudo grubby --update-kernel=ALL --args="..."` (চাইলে `/etc/default/grub`-এর `GRUB_CMDLINE_LINUX`-এ যোগ করে `grub2-mkconfig -o /boot/grub2/grub.cfg`)।

5. ডিস্ট্রোর kernel বেশিরভাগ ড্রাইভার module হিসেবে রাখে, যেগুলো root filesystem-এর ভেতরে থাকে। root মাউন্টের আগে সেগুলো দরকার। initramfs সেই ড্রাইভার ও টুল আগেভাগে RAM-এ এনে

দেয়। পরিস্থিতি: root যখন LUKS-এনক্রিপ্টেড, LVM-এ, সফটওয়্যার RAID-এ, বা নেটওয়ার্কে (NFS/iSCSI) থাকে; অথবা স্টোরেজ ড্রাইভার kernel-এ বিল্ট-ইন না থাকে।

6. rescue mode সব লোকাল ফাইলসিস্টেম মাউন্ট করে আর মৌলিক service চালায় (single-user mode)। emergency mode শুধু root-কে read-only মাউন্ট করে আর প্রায় কিছুই চালায় না। `/etc/fstab`-এ ভুল UUID থাকলে মাউন্ট ব্যর্থ হয়ে `local-fs.target` পূরণ হয় না, তাই সাধারণত **emergency mode**-এ পড়বে।

7. `critical-chain`। `blame` শুধু বলে কোন unit কত সময় নিয়েছে, কিন্তু অনেক unit সমান্তরালে চলে, তাই বেশি সময় নেওয়া unit বুটকে আটকে নাও রাখতে পারে। `critical-chain` দেখায় সেই নির্ভরতার শিকল, যার জন্য target পৌঁছাতে আসলে দেরি হয়েছে।

8. `rd.break` + `chroot`-এ SELinux policy লোড হয় না। তাই `passwd` যে নতুন `/etc/shadow` লেখে, তার SELinux context ভুল থাকে। সেই অবস্থায় বুট করলে লগইন প্রসেস `/etc/shadow` পড়তে পারবে না, কেউ লগইন করতে পারবে না। `/.autorelabel` থাকলে পরের বুটে SELinux সব ফাইলের লেবেল ঠিক করে দেয়।

9. `dmesg` পড়ে kernel-এর ring buffer, যেটা RAM-এ থাকে আর রিবুটে মুছে যায়। আগের বুটের kernel বার্তা দেখতে `journalctl -k -b -1` চালাও। এর জন্য journal persistent হতে হবে (`/var/log/journal` থাকতে হবে)।

10. Secure Boot চালু থাকায় kernel শুধু বিশ্বস্ত চাবি দিয়ে সই করা module লোড করে। VirtualBox-এর module DKMS দিয়ে তোমার মেশিনে কম্পাইল হয়েছে, তাতে বিশ্বস্ত সই নেই। সঠিক সমাধান: নিজের MOK চাবি দিয়ে module সই করা আর সেই চাবি `sudo mokutil --import <key>.der` দিয়ে নিবন্ধন করা, তারপর রিবুটে MOK Manager-এ "Enroll MOK" নিশ্চিত করা। Secure Boot বন্ধ করলেও কাজ হবে, কিন্তু নিরাপত্তা কমে যায়।

অধ্যায় ২৮: লগ ও সিস্টেম মনিটরিং লগ

এই অধ্যায়ে যা শিখবে

- লগ (log) কী, কেন দরকার, আর লিনাক্সে লগ কে লেখে
- `/var/log` ডিরেক্টরির গুরুত্বপূর্ণ ফাইলগুলো: `syslog` / `messages`, `auth.log` / `secure`, `kern.log`, `dpkg.log`, `nginx` লগ ইত্যাদি, আর distro-ভেদে পার্থক্য
- লিনাক্স লগিং-এর পুরো আর্কিটেকচার: kernel ring buffer → systemd-journald → rsyslog → ফাইল
- syslog-এর **facility** আর **severity** কী, এবং rsyslog কনফিগারেশন কীভাবে লেখে
- `journalctl` — `-u`, `-f`, `-b`, `-p`, `--since`, `--until`, `-k`, `-o json` সহ সব দরকারি অপশন
- journal-কে persistent (রিবুটের পরও টিকে থাকা) করা এবং তার সাইজ নিয়ন্ত্রণ করা
- `logrotate` দিয়ে লগ ফাইল ঘুরিয়ে-কেটে-কম্প্রেস করা
- `logger` ও `systemd-cat` দিয়ে নিজের স্ক্রিপ্ট থেকে লগ লেখা
- `dmesg` দিয়ে kernel-এর বার্তা দেখা
- `last`, `lastb`, `lastlog` দিয়ে লগইন ইতিহাস দেখা
- লগ দেখে সমস্যা খোঁজার একটি ধাপে ধাপে পদ্ধতি (troubleshooting methodology)

২৮.১ লগ কী এবং কেন?

ধরো তুমি একটি বড় অফিস বিল্ডিং-এর ম্যানেজার। গেটে একজন দারোয়ান খাতা রাখে: কে কখন ঢুকল, কে বের হলো, কে ভুল কার্ড দেখাল। লিফটের টেকনিশিয়ান আলাদা খাতায় লেখে লিফট কখন আটকে গিয়েছিল। ক্যান্টিনের খাতায় থাকে কখন গ্যাস শেষ হলো। কোনো একদিন যদি শোনো "কাল রাতে কেউ ঢুকেছিল", তাহলে তুমি প্রথমে কী করবে? খাতাগুলো খুলবে।

লিনাক্সের **লগ (log)** ঠিক এই খাতাগুলো। Log মানে হলো সময়সহ ঘটনার লিখিত রেকর্ড। সিস্টেমে যা যা ঘটে — কেউ লগইন করল, কোনো সার্ভিস চালু বা ক্র্যাশ হলো, ডিস্কে error এল, নেটওয়ার্ক কার্ড ডাউন হলো, কেউ ভুল পাসওয়ার্ড দিল — প্রায় সবকিছুই কোথাও না কোথাও লেখা হয়।

লগ কেন এত জরুরি:

কারণ	উদাহরণ
ট্রাবলশুটিং (সমস্যা খোঁজা)	nginx চালু হচ্ছে না — লগে লেখা থাকবে কোন লাইনে কনফিগ ভুল
নিরাপত্তা (security)	কেউ রাতভর SSH-এ পাসওয়ার্ড অনুমান করছে কি না
অডিট (audit)	কে কখন `sudo` দিয়ে কী চালিয়েছে
পারফরম্যান্স	কোন সময়ে সার্ভার ধীর হয়েছিল, OOM killer কোন প্রসেস মেরেছে

কারণ	উদাহরণ
ইতিহাস	কোন প্যাকেজ কবে আপডেট হয়েছিল

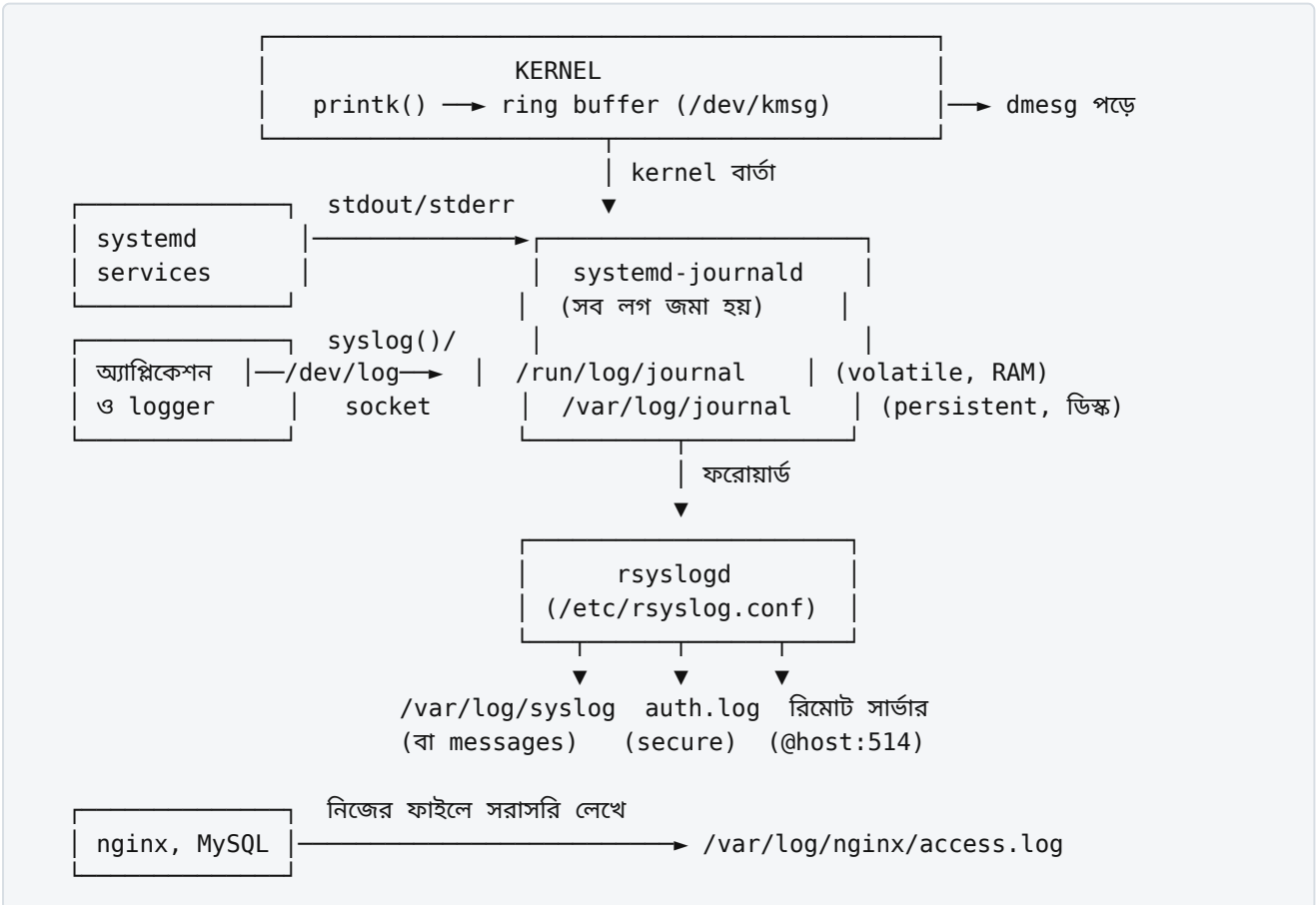
> 📝 **মনে রাখো:** লিনাক্সে সমস্যা হলে প্রথম প্রশ্ন সবসময় একটাই — **"লগ কী বলছে?"** অভিজ্ঞ অ্যাডমিনরা অনুমান করে না, লগ পড়ে।

২৮.২ লিনাক্স লগিং-এর আর্কিটেকচার — ভেতরে কী ঘটে

আধুনিক লিনাক্সে (Ubuntu 24.04, Debian 12, Fedora, RHEL 9) লগ লেখার দুটো প্রধান ব্যবস্থা পাশাপাশি চলে:

1. **systemd-journald** — systemd-এর অংশ, সব লগ একটা বাইনারি ডাটাবেসে (journal) রাখে। এর সাথে কথা বলার টুল `journalctl`।
2. **rsyslog** — পুরোনো ঐতিহ্যবাহী syslog ব্যবস্থার আধুনিক রূপ। লগকে সাধারণ টেক্সট ফাইলে (`/var/log/syslog` ইত্যাদি) লেখে।

আর এদের সবার নিচে আছে **kernel ring buffer** — kernel নিজের বার্তা এখানে রাখে।



কয়েকটি গুরুত্বপূর্ণ কথা এই ছবি থেকে:

- `/dev/log` একটি Unix socket (একই মেশিনের প্রসেসগুলোর মধ্যে কথা বলার "পাইপ")। কোনো প্রোগ্রাম C লাইব্রেরির `syslog()` ফাংশন ডাকলে বার্তা এই socket-এ যায়। systemd-ওয়ালা সিস্টেমে এই socket-টি আসলে journald ধরে রাখে।
- systemd যে সার্ভিস চালায়, তার `stdout` আর `stderr`-এ যা প্রিন্ট হয়, সেটাও স্বয়ংক্রিয়ভাবে journal-এ চলে যায়। তাই systemd সার্ভিসের জন্য আলাদা লগ ফাইল না থাকলেও `journalctl -u` দিয়ে লগ পাওয়া যায়।
- rsyslog journal থেকে বার্তা নেয় (Debian/Ubuntu-তে সাধারণত journald-এর syslog socket দিয়ে, RHEL/Fedora-তে `imjournal` মডিউল দিয়ে সরাসরি journal পড়ে) এবং নিয়ম অনুযায়ী টেক্সট ফাইলে ভাগ করে লেখে।
- কিছু প্রোগ্রাম (nginx, Apache, MySQL, PostgreSQL) syslog-কে পাশ কাটিয়ে **নিজেরাই** নিজেদের ফাইলে লেখে।

distro-ভেদে পার্থক্য

বিষয়	Ubuntu 24.04	Debian 12	Fedora (সাম্প্রতিক)	RHEL 9 / Rocky / Alma
rsyslog ডিফল্ট ইনস্টল	হ্যাঁ	**না** (শুধু journal)	**না**	হ্যাঁ
প্রধান টেক্সট লগ	`/var/log/syslog`	(rsyslog ইনস্টল করলে) `/var/log/syslog`	নেই, `journalctl`	`/var/log/messages`
অথেন্টিকেশন লগ	`/var/log/auth.log`	(rsyslog থাকলে) `auth.log`	journal	`/var/log/secure`
persistent journal	হ্যাঁ (`/var/log/journal` থাকে)	হ্যাঁ	হ্যাঁ	distro সংস্করণ/ইনস্টল ভেদে — নিজে চেক করো

> 💡 **টিপস:** Debian 12 বা Fedora-তে `/var/log/syslog` খুঁজে না পেলে ভয় পেয়ো না। সব লগ journal-এ আছে, `journalctl` দিয়ে দেখো। টেক্সট ফাইল চাইলে `sudo apt install rsyslog` (Debian) বা `sudo dnf install rsyslog` (Fedora)।

বুট প্রসেসের সময় journald কখন চালু হয়, সেটা অধ্যায় ২৭-এ আর systemd-এর সার্বিক ধারণা অধ্যায় ২৬-এ পাবে।

২৮.৩ `/var/log` — লগের বাড়ি

অধ্যায় ৭-এ দেখেছ যে `/var` মানে "variable" — যে ডেটা সময়ের সাথে বদলায়। লগ ঠিক তাই, তাই লগের বাড়ি `/var/log`।

```
$ ls /var/log
alternatives.log  apt        btmp           dpkg.log   kern.log   nginx     ubuntu-
advantage.log
auth.log          auth.log.1 cloud-init.log  faillog    journal    syslog
unattended-upgrades
boot.log          dist-upgrade  installer    lastlog    private    syslog.1  wtmp
```

গুরুত্বপূর্ণ লগ ফাইলের তালিকা

ফাইল (Debian/Ubuntu)	ফাইল (RHEL/Fedora)	কী থাকে
`/var/log/syslog`	`/var/log/messages`	সাধারণ সিস্টেম বার্তা — প্রায় সবকিছু
`/var/log/auth.log`	`/var/log/secure`	লগইন, SSH, `sudo`, `su`, PAM অথেন্টিকেশন
`/var/log/kern.log`	(`messages`-এ থাকে)	kernel বার্তা
`/var/log/dpkg.log`	`/var/log/dnf.log`, `dnf.rpm.log`	প্যাকেজ ইনস্টল/রিমুভ
`/var/log/apt/history.log`	`dnf history` কমান্ড	কোন apt কমান্ড কবে চালানো হয়েছিল
`/var/log/boot.log`	`/var/log/boot.log`	বুটের সময় সার্ভিস স্টার্টআপ বার্তা (সব সিস্টেমে থাকে না)
`/var/log/mail.log`	`/var/log/maillog`	মেইল সার্ভার (Postfix ইত্যাদি)
(`syslog`-এ থাকে)	`/var/log/cron`	cron জবের রেকর্ড
`/var/log/ufw.log`	—	ufw ফায়ারওয়ালের ব্লক করা প্যাকেট
—	`/var/log/audit/audit.log`	auditd ও SELinux denial (RHEL-এ খুব গুরুত্বপূর্ণ)
`/var/log/nginx/access.log`, `error.log`	একই	nginx ওয়েব সার্ভার
`/var/log/apache2/`	`/var/log/httpd/`	Apache ওয়েব সার্ভার
`/var/log/mysql/`	`/var/log/mysqld.log` বা `mariadb/`	ডাটাবেস
`/var/log/wtmp`	একই	সফল লগইন/লগআউট ও রিবুটের ইতিহাস (বাইনারি, `last` দিয়ে পড়ো)
`/var/log/btmp`	একই	ব্যর্থ লগইন চেষ্টা (বাইনারি, `lastb` দিয়ে পড়ো)
`/var/log/lastlog`	একই	প্রতি ইউজারের শেষ লগইন (বাইনারি, `lastlog` দিয়ে পড়ো)
`/var/log/journal/`	একই	systemd journal (বাইনারি, `journalctl` দিয়ে পড়ো)

> ⚠️ **সতর্কতা:** `wtmp`, `btmp`, `lastlog` আর `journal/`-এর ফাইলগুলো **বাইনারি**। এগুলো `cat` বা `less` দিয়ে খুললে অর্থহীন অক্ষর দেখবে, টার্মিনাল এলোমেলোও হয়ে যেতে পারে (তখন `reset` লিখে

> Enter চাপো)। এগুলো সবসময় নির্দিষ্ট কমান্ড দিয়ে পড়বে।

> 📝 **মনে রাখো:** কিছু নতুন distro সংস্করণে Y2038 সমস্যার কারণে `wtmp` / `lastlog` -এর বদলে `wtmpdb` ও `lastlog2` আসছে। কমান্ড না পেলে তোমার distro-র ডকুমেন্টেশন দেখো।

লগ পড়ার অনুমতি

বেশিরভাগ লগ সাধারণ ইউজার পড়তে পারে না, কারণ এতে গোপন তথ্য থাকে।

```
$ ls -l /var/log/auth.log
-rw-r----- 1 syslog adm 48213 Nov 12 10:15 /var/log/auth.log
$ cat /var/log/auth.log
cat: /var/log/auth.log: Permission denied
```

Ubuntu/Debian-এ **`adm`** গ্রুপের সদস্যরা বেশিরভাগ লগ পড়তে পারে। journal পড়ার জন্য **`systemd-journal`** গ্রুপ (Fedora/RHEL-এ `wheel` গ্রুপও সাধারণত পড়তে পারে)। উপায় দুটো: `sudo` ব্যবহার করো, অথবা নিজেকে গ্রুপে যোগ করো (অধ্যায় ১৫):

```
$ sudo usermod -aG adm,systemd-journal $USER
# তারপর লগআউট করে আবার লগইন করো
```

একটি syslog লাইনের গঠন

```
Nov 12 10:15:32 webserver sshd[2841]: Accepted publickey for rahim from 192.168.1.50
port 52144 ssh2
└──────────────┘ └─────┘ └──┘└──┘
└──────────────────────────────────────────────────────┘
    সময়          হোস্টনেম  প্রোগ্রাম PID                          বার্তা (message)
```

> 💡 **টিপস:** Ubuntu 24.04-এর rsyslog ডিফল্টভাবে উচ্চ-নির্ভুল ISO ফরম্যাটে সময় লেখে, যেমন `2026-11-12T10:15:32.123456+06:00`। পুরোনো সিস্টেমে উপরের মতো `Nov 12 10:15:32` দেখবে। দুটোরই অর্থ এক।

টেক্সট লগ পড়ার মৌলিক কমান্ড

এসব কমান্ড বিস্তারিত অধ্যায় ১০ আর অধ্যায় ২০-এ আছে; এখানে লগের প্রেক্ষিতে দেখো।

```
# শেষ ২০ লাইন
$ sudo tail -n 20 /var/log/syslog

# লাইভ দেখা (নতুন লাইন আসলেই দেখাবে) — থামাতে Ctrl+C
$ sudo tail -f /var/log/auth.log
```

```
# -F: ফাইল rotate হলেও (নাম বদলে নতুন ফাইল এলেও) ধরে রাখবে — লগের জন্য -f-এর চেয়ে ভালো
$ sudo tail -F /var/log/nginx/access.log

# less দিয়ে পড়া, শেষ থেকে শুরু
$ sudo less +G /var/log/syslog

# less-এ "follow" মোড (tail -f-এর মতো), Ctrl+C চাপলে সাধারণ less-এ ফেরে
$ sudo less +F /var/log/syslog

# খোঁজা
$ sudo grep -i "error" /var/log/syslog

# পুরোনো, কম্প্রেস করা লগে খোঁজা
$ sudo zgrep "Failed password" /var/log/auth.log.2.gz
$ sudo zcat /var/log/syslog.3.gz | less
```

একাধিক লগ একসাথে লাইভ দেখতে:

```
$ sudo tail -F /var/log/syslog /var/log/auth.log
==> /var/log/syslog <==
2026-11-12T10:20:01.512+06:00 web CRON[3011]: (root) CMD (command -v debian-sa1 >
/dev/null && debian-sa1 1 1)

==> /var/log/auth.log <==
2026-11-12T10:20:05.871+06:00 web sudo:    rahim : TTY=pts/0 ; PWD=/home/rahim ;
USER=root ; COMMAND=/usr/bin/apt update
```

২৮.৪ syslog প্রোটোকল: facility ও severity

syslog একটি পুরোনো (১৯৮০-র দশকের) কিন্তু এখনো সর্বব্যাপী মানদণ্ড, যা পরে RFC 5424-এ লিখিত হয়েছে। প্রতিটি syslog বার্তার সাথে দুটো ট্যাগ থাকে:

- **facility** — বার্তাটা **কোন বিভাগ** থেকে এসেছে (kernel? মেইল? অথেন্টিকেশন?)
- **severity** (বা priority/level) — বার্তাটা **কতটা গুরুতর**

উপমা: হাসপাতালের ইমার্জেন্সি। facility হলো রোগী কোন ওয়ার্ড থেকে এসেছে (হৃদরোগ, হাড়, শিশু), আর severity হলো রোগীর অবস্থা কতটা জরুরি (লাল, হলুদ, সবুজ)। rsyslog হলো রিসেপশনিস্ট, যে এই দুই ট্যাগ দেখে ঠিক করে রোগীকে কোথায় পাঠাবে।

Facility তালিকা

কোড	facility	কী জন্য
0	`kern`	kernel বার্তা
1	`user`	সাধারণ ইউজার-লেভেল প্রোগ্রাম (ডিফল্ট)
2	`mail`	মেইল সিস্টেম

কোড	facility	কী জন্য
3	`daemon`	সিস্টেম ডেমন (ব্যাকগ্রাউন্ড সার্ভিস)
4	`auth`	অথেন্টিকেশন/নিরাপত্তা
5	`syslog`	syslogd নিজের বার্তা
6	`lpr`	প্রিন্টার
7	`news`	Usenet news
8	`uucp`	UUCP (পুরোনো)
9	`cron`	cron ও at
10	`authpriv`	ব্যক্তিগত/গোপন অথেন্টিকেশন বার্তা
11	`ftp`	FTP সার্ভার
16–23	`local0` – `local7`	তোমার নিজের ব্যবহারের জন্য খালি রাখা

> 💡 **টিপস:** নিজের অ্যাপ বা স্ক্রিপ্টের লগ আলাদা ফাইলে পাঠাতে চাইলে `local0`–`local7` ব্যবহার করো। এগুলো ঠিক এই কাজের জন্যই আছে।

Severity তালিকা (সংখ্যা যত কম, তত গুরুতর)

কোড	নাম	অর্থ	উদাহরণ
0	`emerg`	সিস্টেম অচল	kernel panic
1	`alert`	এখনই ব্যবস্থা নিতে হবে	ডাটাবেস করাপ্ট
2	`crit`	গুরুতর অবস্থা	হার্ডওয়্যার ফেইলিওর
3	`err`	error	সার্ভিস চালু হতে পারেনি
4	`warning`	সতর্কতা	ডিস্ক ৯০% ভরা
5	`notice`	স্বাভাবিক কিন্তু উল্লেখযোগ্য	কনফিগ রিলোড হয়েছে
6	`info`	তথ্যমূলক	ইউজার লগইন করেছে
7	`debug`	ডিবাগ	ডেভেলপারদের খুঁটিনাটি

মনে রাখার ছড়া (ইংরেজিতে প্রথম অক্ষর): **"Every Alley Cat Eats Whiskas Not In Daytime"** → emerg, alert, crit, err, warning, notice, info, debug।

> 📝 **মনে রাখো:** "priority" শব্দটা দুইভাবে ব্যবহার হয়। rsyslog কনফিগে `facility.severity` জোড়াকে "selector" বা "priority" বলে; আর journalctl-এর `-p` অপশনে "priority" মানে শুধু severity।

২৮.৫ rsyslog — কনফিগারেশন ও রাউটিং

কনফিগ ফাইল কোথায়

```
/etc/rsyslog.conf          ← মূল ফাইল (মডিউল লোড, গ্লোবাল সেটিং)
/etc/rsyslog.d/*.conf      ← অতিরিক্ত নিয়ম; এখানে নিজের ফাইল রাখো
```

Ubuntu-তে আসল নিয়মগুলো থাকে `/etc/rsyslog.d/50-default.conf`-এ:

```
$ grep -v '^#' /etc/rsyslog.d/50-default.conf | grep -v '^$'
auth,authpriv.*                 /var/log/auth.log
*.*;auth,authpriv.none          -/var/log/syslog
kern.*                          -/var/log/kern.log
mail.*                          -/var/log/mail.log
mail.err                        /var/log/mail.err
*.emerg                         :omusrmsg:*
```

RHEL 9-এ `/etc/rsyslog.conf`-এর ভেতরেই:

```
*.info;mail.none;authpriv.none;cron.none    /var/log/messages
authpriv.*                                  /var/log/secure
mail.*                                      -/var/log/maillog
cron.*                                      /var/log/cron
*.emerg                                     :omusrmsg:*
local7.*                                    /var/log/boot.log
```

Selector-এর ব্যাকরণ

প্রতিটি নিয়মের দুটো অংশ: **বাঁয়ে selector (কোন বার্তা), ডানে action (কোথায় পাঠাবে)।**

লেখা	অর্থ
`kern.*`	kern facility-র সব severity
`mail.err`	mail facility-র `err` **এবং তার চেয়ে গুরুতর** (err, crit, alert, emerg)
`mail.=err`	**শুধু** `err` লেভেল
`mail.!err`	`err`-এর চেয়ে কম গুরুতর (warning থেকে debug)
`auth,authpriv.*`	একাধিক facility কমা দিয়ে
`*.*;auth,authpriv.none`	সবকিছু, **কিন্তু** auth/authpriv বাদে (`;` দিয়ে একাধিক selector, `none` মানে বাদ)
`*.emerg`	যেকোনো facility-র emergency

Action	অর্থ
`/var/log/file`	ফাইলে লেখো
`-/var/log/file`	ফাইলে লেখো, প্রতিবার sync করার দরকার নেই (পুরোনো সিনট্যাক্স; আধুনিক rsyslog-এ ডিফল্টই এমন)

Action	অর্থ
`:omusrmsg:*`	লগইন করা সব ইউজারের টার্মিনালে দেখাও
`@192.168.1.10:514`	রিমোট সার্ভারে **UDP** দিয়ে পাঠাও
`@@192.168.1.10:5144`	রিমোট সার্ভারে **TCP** দিয়ে পাঠাও
`stop`	এই বার্তা নিয়ে আর কিছু করো না (পরের নিয়মে যাবে না)

নিজের অ্যাপের জন্য আলাদা লগ ফাইল

```
$ sudo nano /etc/rsyslog.d/30-myapp.conf
```

```
# local3 facility-র সব বার্তা আলাদা ফাইলে, তারপর থামো
local3.*    /var/log/myapp.log
& stop
```

`& stop` মানে "আগের লাইনের মতো একই বার্তার জন্য — থামো", যাতে এগুলো `syslog`-এ দ্বিতীয়বার না যায়। (মনে রেখো, ফাইলের নাম `30-` দিয়ে শুরু করেছি যাতে `50-default.conf`-এর আগে পড়া হয়।)

কনফিগ ঠিক আছে কি না যাচাই করে রিস্টার্ট:

```
$ sudo rsyslogd -N1
rsyslogd: version 8.2312.0, config validation run (level 1), master config
/etc/rsyslog.conf
rsyslogd: End of config validation run. Bye.
$ sudo systemctl restart rsyslog
```

> ⚠️ **সতর্কতা:** Ubuntu-তে rsyslog `syslog` ইউজার হিসেবে চলে। নতুন লগ ফাইল `/var/log`-এ তৈরি করতে না পারলে কিছু লেখা হবে না। সমস্যা হলে আগে থেকে ফাইল বানিয়ে মালিকানা দাও: `sudo touch /var/log/myapp.log && sudo chown syslog:adm /var/log/myapp.log`।

আধুনিক RainerScript সিনট্যাক্স (সংক্ষেপে)

পুরোনো selector সিনট্যাক্সের পাশাপাশি rsyslog-এর নিজস্ব স্ক্রিপ্ট ভাষা আছে, যাতে প্রোগ্রামের নাম বা লেখার ভেতরের শব্দ দেখে ফিল্টার করা যায়:

```
if $programname == 'nginx' and $msg contains 'error' then {
    action(type="omfile" file="/var/log/nginx-errors-syslog.log")
    stop
}
```

সেন্ট্রালাইজড লগিং (একাধিক সার্ভারের লগ এক জায়গায়)

দশটা সার্ভার থাকলে দশ জায়গায় লগ খোঁজা কষ্টের, আর হ্যাকার সার্ভারে ঢুকে প্রথমে লগ মুছে দেয়। তাই লগ আরেকটা সার্ভারে পাঠানো ভালো অভ্যাস।

লগ গ্রহণকারী সার্ভারে (`/etc/rsyslog.d/10-receive.conf`):

```
module(load="imtcp")
input(type="imtcp" port="514")
```

লগ পাঠানো ক্লায়েন্টে (`/etc/rsyslog.d/90-forward.conf`):

```
*.*  @@logserver.example.com:514
```

তারপর দুই জায়গায় `sudo systemctl restart rsyslog`, আর সার্ভারে ফায়ারওয়ালে 514/tcp খুলতে হবে (অধ্যায় ৩৯)।

> 💡 **টিপস:** বড় প্রতিষ্ঠানে Graylog, Elasticsearch/OpenSearch + Kibana (ELK), বা Grafana Loki-এর মতো সিস্টেমে লগ জমা হয়, যেখানে ওয়েব ইন্টারফেসে খোঁজা ও গ্রাফ করা যায়। এগুলোর ভিত্তিও এই একই syslog/journal ধারণা।

২৮.৬ systemd journal ও `journalctl`

journal কেন আলাদা

পুরোনো টেক্সট লগের কিছু সীমাবদ্ধতা ছিল: প্রতিটা প্রোগ্রাম আলাদা ফরম্যাটে লেখে, কোন লাইন কোন সার্ভিসের সেটা নিশ্চিতভাবে বোঝা কঠিন, আর লেখাটা পরে চুপচাপ বদলে দেওয়া যায়। journald এগুলোর সমাধান দেয়:

- **স্ট্রাকচার্ড ডেটা**: প্রতিটা এন্ট্রির সাথে অনেক ফিল্ড থাকে — `_PID`, `_UID`, `_COMM` (কমান্ডের নাম), `_SYSTEMD_UNIT`, `_BOOT_ID`, `PRIORITY` ইত্যাদি। এই ফিল্ডগুলো (যেগুলো `_` দিয়ে শুরু) journald নিজে যোগ করে, তাই প্রোগ্রাম এগুলো জাল করতে পারে না।
- **ইনডেক্স করা**: লক্ষ লক্ষ লাইনের মধ্যেও নির্দিষ্ট সার্ভিস বা বুটের লগ দ্রুত বের করা যায়।
- **বুট অনুযায়ী ভাগ**: "আগের বুটে কী হয়েছিল" সহজে দেখা যায়।
- **শুরু থেকে সব**: initramfs আর kernel-এর প্রথম বার্তা থেকেই ধরে।
- **ঐচ্ছিক seal**: `--setup-keys` দিয়ে Forward Secure Sealing চালু করলে লগ বদলানো হয়েছে কি না যাচাই করা যায়।

উপমা: টেক্সট লগ যেন হাতে লেখা ডায়েরি — পড়তে সহজ কিন্তু খুঁজতে কষ্ট। journal যেন লাইব্রেরির কম্পিউটার ক্যাটালগ — বই (এন্ট্রি) লেখক, বিষয়, তারিখ দিয়ে মুহূর্তে খোঁজা যায়।

প্রাথমিক ব্যবহার

```
$ journalctl
```

কোনো অপশন ছাড়া পুরোনো থেকে নতুন — **সব** লগ দেখায়, `less` pager-এর ভেতরে। sudo ছাড়া চালালে (এবং তুমি `adm` / `systemd-journal` গ্রুপে না থাকলে) শুধু নিজের ইউজারের লগ দেখবে।

```
Nov 12 09:01:12 laptop kernel: Linux version 6.8.0-48-generic (buildd@lcy02-amd64-040)
...
Nov 12 09:01:12 laptop kernel: Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-48-generic
root=UUID=...
...
```

রঙ: `err` ও তার চেয়ে গুরুতর লাল ও মোটা, `warning` হলুদ/মোটা, `notice` মোটা — তাই চোখ বুলিয়েই সমস্যা ধরা যায়।

journalctl-এর গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-u UNIT`	নির্দিষ্ট systemd unit (সার্ভিস)-এর লগ; একাধিকবার দেওয়া যায়
`-f`	follow — লাইভ নতুন লগ (`tail -f`-এর মতো); থামাতে Ctrl+C
`-n N`	শেষ N লাইন (শুধু `-n` দিলে ১০)
`-e`	pager খুলে সরাসরি শেষে যাও
`-r`	উল্টো ক্রম — নতুনটা আগে
`-b`	বর্তমান বুট থেকে; `-b -1` আগের বুট, `-b -2` তার আগের
`--list-boots`	সব রেকর্ড করা বুটের তালিকা
`-k`	শুধু kernel বার্তা (`-b`-সহ বোঝায়), অনেকটা `dmesg`-এর মতো
`-p LEVEL`	এই severity ও তার চেয়ে গুরুতর; `-p warning..err` দিয়ে রেঞ্জ
`--since`, `-S`	এই সময় থেকে
`--until`, `-U`	এই সময় পর্যন্ত
`-t ID`	syslog identifier (ট্যাগ) দিয়ে ফিল্টার
`-g PATTERN`	বার্তার ভেতরে regex দিয়ে খোঁজা (`--grep`)
`-o FORMAT`	আউটপুট ফরম্যাট (নিচে টেবিল)
`-x`	যেখানে পাওয়া যায়, ব্যাখ্যামূলক অতিরিক্ত লেখা যোগ করো

অপশন	কাজ
`--no-pager`	pager ছাড়া সরাসরি প্রিন্ট (স্ক্রিপ্ট ও পাইপের জন্য)
`--user`	নিজের ইউজার সার্ভিসের লগ
`--user-unit UNIT`	নির্দিষ্ট ইউজার-unit-এর লগ
`-F FIELD`	একটা ফিল্ডে কী কী মান আছে তার তালিকা
`-N`	journal-এ থাকা সব ফিল্ডের নাম
`--disk-usage`	journal ডিস্কে কত জায়গা নিচ্ছে
`--vacuum-size=`, `--vacuum-time=`, `--vacuum-files=`	পুরোনো journal ফাইল মুছে জায়গা খালি করো
`--verify`	journal ফাইলের সঠিকতা যাচাই
`--flush`	RAM-এর (`/run`) journal ডিস্কে (`/var`) নিয়ে যাও
`--rotate`	এখনই নতুন journal ফাইল শুরু করো

`-u` — একটা সার্ভিসের লগ

```
$ journalctl -u ssh
Nov 12 09:01:20 server systemd[1]: Starting ssh.service - OpenBSD Secure Shell
server...
Nov 12 09:01:20 server sshd[812]: Server listening on 0.0.0.0 port 22.
Nov 12 09:01:20 server systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Nov 12 10:15:32 server sshd[2841]: Accepted publickey for rahim from 192.168.1.50 port
52144 ssh2
```

> 📝 **মনে রাখো:** Ubuntu/Debian-এ SSH সার্ভিসের নাম `ssh`, RHEL/Fedora-তে `sshd`। Apache-এর নাম Debian পরিবারে `apache2`, RHEL পরিবারে `httpd`।

একাধিক সার্ভিস একসাথে:

```
$ journalctl -u nginx -u php8.3-fpm --since today
```

সবচেয়ে বেশি ব্যবহৃত কম্বো — কোনো সার্ভিস ফেইল করলে:

```
$ systemctl status nginx          # সংক্ষিপ্ত অবস্থা + শেষ কয়েক লাইন লগ
$ journalctl -u nginx -n 50 --no-pager
$ journalctl -xeu nginx           # শেষে যাও, ব্যাখ্যাসহ
```

`-f` — লাইভ লগ

```
$ journalctl -f                   # সব লাইভ
$ journalctl -fu nginx            # শুধু nginx লাইভ
```

```
$ sudo journalctl -f -p err       # শুধু error লাইভ
```

একটা টার্মিনালে `journalctl -f` চালিয়ে আরেকটা টার্মিনালে সার্ভিস রিস্টার্ট করে দেখো — কী হচ্ছে চোখের সামনে দেখবে। ট্রাবলশুটিং-এ এটা সোনার মতো দামি কৌশল।

`-b` — বুট অনুযায়ী

```
$ journalctl --list-boots
IDX BOOT ID                          FIRST ENTRY                 LAST ENTRY
 -2 3f1c2a9e8b5d4c7a9e1f0b2c3d4e5f6a Sun 2026-11-10 08:12:01 +06 Sun 2026-11-10
23:40:12 +06
 -1 7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d Mon 2026-11-11 09:00:44 +06 Mon 2026-11-11
22:15:03 +06
  0 c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9 Wed 2026-11-12 09:01:12 +06 Wed 2026-11-12
10:45:30 +06

$ journalctl -b          # এই বুট
$ journalctl -b -1       # আগের বুট
$ journalctl -b -1 -e    # আগের বুটের একদম শেষ অংশ — "কম্পিউটার হঠাৎ বন্ধ হলো কেন?"
```

> ⚠️ **সতর্কতা:** journal persistent না হলে (`/run/log/journal` -এ থাকলে) প্রতিটা রিবুটে আগের লগ হারিয়ে যায়, আর `--list-boots` -এ শুধু একটা বুট দেখাবে। নিচে ২৮.৭-এ persistent করার পদ্ধতি দেখো।

`-p` — severity দিয়ে ফিল্টার

```
$ journalctl -p err -b                # এই বুটের সব error ও তার চেয়ে গুরুতর
$ journalctl -p 3 -b                  # একই কথা, সংখ্যা দিয়ে
$ journalctl -p warning..err          # শুধু warning থেকে err পর্যন্ত
```

নতুন সিস্টেম হাতে পেলে প্রথম স্বাস্থ্য-পরীক্ষা:

```
$ sudo journalctl -b -p err --no-pager
Nov 12 09:01:14 laptop kernel: ACPI BIOS Error (bug): Could not resolve symbol
[\_SB.PCI0.GPP0], AE_NOT_FOUND
Nov 12 09:01:25 laptop bluetoothd[901]: Failed to set mode: Failed (0x03)
```

> 💡 **টিপস:** প্রতিটা error মানেই সমস্যা নয়। ACPI BIOS error-এর মতো অনেক বার্তা বেশিরভাগ ল্যাপটপে দেখা যায় এবং ক্ষতিহীন। দেখতে হবে — যে সমস্যা নিয়ে তুমি চিন্তিত, তার **সময়ের সাথে** কোনো error মেলে কি না।

`--since` ও `--until` — সময়ের জানালা

```
$ journalctl --since "2026-11-12 09:00:00" --until "2026-11-12 09:30:00"
$ journalctl --since "2026-11-12"            # ওই দিনের মধ্যরাত থেকে
$ journalctl --since today
$ journalctl --since yesterday --until today
$ journalctl --since "1 hour ago"
$ journalctl --since "-15min"
$ journalctl -u nginx --since "10:00" --until "10:30"   # আজকের ১০টা থেকে সাড়ে ১০টা
```

গ্রহণযোগ্য সময়ের রূপ: `"YYYY-MM-DD HH:MM:SS"`, `"YYYY-MM-DD"`, `"HH:MM"`, `today`, `yesterday`, `tomorrow`, `now`, `"N hour ago"` / `"N min ago"`, অথবা `-1h`, `+30min`-এর মতো আপেক্ষিক রূপ। (বিস্তারিত: `man systemd.time`)

`-k` — kernel বার্তা

```
$ journalctl -k              # এই বুটের kernel বার্তা
$ journalctl -k -b -1        # আগের বুটের kernel বার্তা — dmesg যেটা পারে না!
$ journalctl -k -p warning
```

`dmesg` শুধু বর্তমান ring buffer দেখায়, কিন্তু persistent journal থাকলে `journalctl -k -b -1` দিয়ে আগের বুটের kernel বার্তাও পাবে।

ফিল্ড দিয়ে ম্যাচ করা

journalctl-এ `FIELD=value` লিখলেই ফিল্টার হয়:

```
$ journalctl _PID=2841
$ journalctl _UID=1000 --since today          # একজন ইউজারের সব প্রসেসের লগ
$ journalctl _COMM=sshd                        # কমান্ডের নাম দিয়ে
$ journalctl /usr/sbin/sshd                    # executable-এর পথ দিয়ে
$ journalctl _SYSTEMD_UNIT=cron.service
$ journalctl -t sudo                           # syslog ট্যাগ দিয়ে
```

নিয়ম: **ভিন্ন ফিল্ড** দিলে AND (দুটোই মিলতে হবে); **একই ফিল্ড** দুবার দিলে OR; মাঝে `+` দিলে দুই দিকের শর্তের OR।

```
$ journalctl _SYSTEMD_UNIT=nginx.service _PID=1234     # AND
$ journalctl _COMM=sshd _COMM=sudo                     # OR
$ journalctl _COMM=sshd + _SYSTEMD_UNIT=cron.service   # OR (দুই গ্রুপ)
```

কোন ফিল্ডে কী মান আছে জানতে:

```
$ journalctl -F _SYSTEMD_UNIT | head
cron.service
systemd-journald.service
ssh.service
```

```
nginx.service
...
```

`-g` — বার্তার ভেতরে খোঁজা

```
$ journalctl -u ssh -g "Failed password" --since today
$ journalctl -g "(?i)out of memory" -b
```

ডিফল্টভাবে প্যাটার্নে সব ছোট হাতের অক্ষর থাকলে খোঁজা case-insensitive হয়। `-g` কাজ না করলে (কিছু বিল্ডে PCRE2 ছাড়া কম্পাইল করা থাকে) পাইপ দিয়ে grep করো: `journalctl -u ssh --no-pager | grep "Failed password"`।

`-o` — আউটপুট ফরম্যাট

ফরম্যাট	কেমন দেখায় / কী কাজে
`short`	ডিফল্ট, syslog-এর মতো
`short-iso`	ISO 8601 সময় (`2026-11-12T10:15:32+06:00`)
`short-precise`	মাইক্রোসেকেন্ড পর্যন্ত সময়
`short-monotonic`	বুটের পর থেকে কত সেকেন্ড
`verbose`	এন্ট্রির **সব** ফিল্ড
`cat`	শুধু বার্তা, সময়/হোস্ট ছাড়া
`json`	প্রতি এন্ট্রি এক লাইনে JSON (মেশিনে প্রসেসের জন্য)
`json-pretty`	সাজানো, পড়ার উপযোগী JSON
`export`	বাইনারি-নিরাপদ এক্সপোর্ট ফরম্যাট (ব্যাকআপ/স্থানান্তর)

```
$ journalctl -u ssh -n 1 -o json-pretty
{
        "__REALTIME_TIMESTAMP" : "1762920932512345",
        "_HOSTNAME" : "server",
        "PRIORITY" : "6",
        "SYSLOG_IDENTIFIER" : "sshd",
        "_PID" : "2841",
        "_UID" : "0",
        "_COMM" : "sshd",
        "_EXE" : "/usr/sbin/sshd",
        "_SYSTEMD_UNIT" : "ssh.service",
        "_BOOT_ID" : "c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9",
        "MESSAGE" : "Accepted publickey for rahim from 192.168.1.50 port 52144 ssh2",
        ...
}
```

(আসল আউটপুটে আরও অনেক ফিল্ড থাকে।) JSON আউটপুট `jq` দিয়ে প্রসেস করা যায়:

```
$ journalctl -u ssh --since today -o json --no-pager | jq -r '.MESSAGE' | grep Accepted
```

নির্দিষ্ট ফিল্ডগুলোই দেখতে: `journalctl -o json-pretty --output-fields=MESSAGE,_PID -n 3`।

journalctl pager-এর কীবোর্ড শর্টকাট

journalctl ডিফল্টভাবে `less` ব্যবহার করে, তাই less-এর সব শর্টকাট কাজ করে:

কী	কাজ
`Space` / `f`	এক পাতা নিচে
`b`	এক পাতা উপরে
`↓` / `j`, `↑` / `k`	এক লাইন নিচে/উপরে
`G`	একদম শেষে (সবচেয়ে নতুন)
`g`	একদম শুরুতে
`/শব্দ`	নিচের দিকে খোঁজো
`?শব্দ`	উপরের দিকে খোঁজো
`n` / `N`	পরের / আগের মিল
`→` / `←`	লম্বা লাইন ডানে/বাঁয়ে স্ক্রল (journalctl লম্বা লাইন কাটে না, পাশে সরায়)
`F`	follow মোড (লাইভ); Ctrl+C দিয়ে ফেরো
`-S` তারপর Enter	লম্বা লাইন কাটা/মোড়ানো অদলবদল
`h`	less-এর সাহায্য
`q`	বের হও
`Ctrl+C`	`journalctl -f` বা `tail -f` থামাও

২৮.৭ journal-এর স্টোরেজ: volatile বনাম persistent

দুই জায়গা

```
/run/log/journal/<machine-id>/   ← volatile: RAM-এ (tmpfs), রিবুটে মুছে যায়
/var/log/journal/<machine-id>/   ← persistent: ডিস্কে, রিবুটের পরও থাকে
```

কোনটা ব্যবহার হবে সেটা ঠিক করে `/etc/systemd/journald.conf`-এর `Storage=` সেটিং:

মান	আচরণ
`volatile`	শুধু RAM-এ
`persistent`	ডিস্কে; দরকার হলে `/var/log/journal` নিজেই তৈরি করে

মান	আচরণ
`auto` (ডিফল্ট)	`/var/log/journal` ডিরেক্টরি **থাকলে** ডিস্কে, না থাকলে RAM-এ
`none`	কিছুই রাখে না (শুধু ফরোয়ার্ড করে)

বুটের শুরুতে `/var` হয়তো এখনো মাউন্ট হয়নি, তাই journald প্রথমে `/run`-এ লেখে, তারপর `systemd-journal-flush.service` সব ডিস্কে সরিয়ে নেয়।

তোমার সিস্টেম persistent কি না দেখো

```
$ ls -d /var/log/journal
/var/log/journal
$ journalctl --list-boots | wc -l
12
```

ডিরেক্টরি থাকলে আর একাধিক বুট দেখালে persistent।

persistent করার পদ্ধতি

পদ্ধতি ১ — ডিরেক্টরি বানানো (`Storage=auto` হলে যথেষ্ট):

```
$ sudo mkdir -p /var/log/journal
$ sudo systemd-tmpfiles --create --prefix /var/log/journal
$ sudo systemctl restart systemd-journald
```

`systemd-tmpfiles` ঠিক মালিকানা ও পারমিশন (গ্রুপ `systemd-journal`, ACL) বসিয়ে দেয়।

পদ্ধতি ২ — কনফিগে স্পষ্ট বলে দেওয়া। মূল ফাইল না বদলে drop-in ফাইল বানানো ভালো:

```
$ sudo mkdir -p /etc/systemd/journald.conf.d
$ sudo nano /etc/systemd/journald.conf.d/10-persistent.conf
```

```
[Journal]
Storage=persistent
SystemMaxUse=1G
MaxRetentionSec=1month
```

```
$ sudo systemctl restart systemd-journald
```

journald.conf-এর গুরুত্বপূর্ণ সেটিং

সেটিং	অর্থ
`Storage=`	উপরে বলা হয়েছে
`SystemMaxUse=`	`/var/log/journal` সর্বোচ্চ কত জায়গা নেবে (ডিফল্ট: ফাইলসিস্টেমের ১০%, সর্বোচ্চ ৪G)

সেটিং	অর্থ
`SystemKeepFree=`	ডিস্কে অন্তত কত জায়গা খালি রাখবে (ডিফল্ট ১৫%, সর্বোচ্চ ৪G)
`SystemMaxFileSize=`	একটা journal ফাইলের সর্বোচ্চ সাইজ, তারপর নতুন ফাইল
`RuntimeMaxUse=`	`/run`-এর (RAM) জন্য একই সীমা
`MaxRetentionSec=`	কত পুরোনো এন্ট্রি রাখবে (যেমন `2week`, `1month`)
`Compress=`	বড় এন্ট্রি কম্প্রেস করবে কি না (ডিফল্ট yes)
`ForwardToSyslog=`	syslog socket-এ ফরোয়ার্ড করবে কি না
`RateLimitIntervalSec=`, `RateLimitBurst=`	একটা সার্ভিস নির্দিষ্ট সময়ে কতগুলো বার্তা পাঠাতে পারবে; বেশি হলে বাকিগুলো ফেলে দেয়

> 💡 **টিপস:** লগে `Suppressed N messages from nginx.service` দেখলে বুঝবে rate limit-এ বার্তা বাদ গেছে। খুব বেশি লগ লেখা সার্ভিসের জন্য rate limit বাড়ানো যায়, কিন্তু আগে ভাবো — এত লগ আসছে কেন?

জায়গা পরিষ্কার করা

```
$ journalctl --disk-usage
Archived and active journals take up 1.2G in the file system.

$ sudo journalctl --vacuum-size=500M       # মোট ৫০০M-এ নামাও
$ sudo journalctl --vacuum-time=2weeks     # ২ সপ্তাহের পুরোনো মোছো
$ sudo journalctl --vacuum-files=5         # সর্বোচ্চ ৫টা archived ফাইল রাখো
Vacuuming done, freed 712.0M of archived journals from /var/log/journal/....
```

> ⚠️ **সতর্কতা:** vacuum শুধু **archived** (বন্ধ হয়ে যাওয়া) journal ফাইল মোছে, চালু (active) ফাইল নয়। সবকিছু মুছতে চাইলে আগে `sudo journalctl --rotate` চালাও, তারপর vacuum। আর `/var/log/journal`-এর ফাইল কখনো হাতে `rm` করবে না যখন journald চলছে।

journal-এ যারা ফাইল পড়ে — অন্য মেশিনের journal

ডেড সিস্টেমের ডিস্ক অন্য মেশিনে লাগিয়ে তার লগ পড়া যায়:

```
$ journalctl -D /mnt/broken/var/log/journal -b -1 -e
$ journalctl --file /path/to/system.journal
```

উদ্ধারের সময় এটা খুব কাজের (অধ্যায় ৪৬-এ ট্রাবলশুটিং দেখো)।

২৮.৮ `logger` ও `systemd-cat` — নিজে লগ লেখা

logger

`logger` কমান্ড দিয়ে টার্মিনাল বা শেল স্ক্রিপ্ট থেকে syslog-এ বার্তা পাঠানো যায়। বার্তা journal-এ যায়, আর rsyslog থাকলে টেক্সট ফাইলেও।

Syntax: `logger [অপশন] "বার্তা"`

অপশন	কাজ
`-p facility.level`	facility ও severity (ডিফল্ট `user.notice`)
`-t TAG`	ট্যাগ/identifier (ডিফল্ট: তোমার ইউজারনেম)
`-i`	প্রসেসের PID যোগ করো
`--id=PID`	নির্দিষ্ট PID যোগ করো
`-s`	বার্তা stderr-এও প্রিন্ট করো
`-f FILE`	ফাইলের প্রতিটা লাইন লগ করো
`-n HOST`	রিমোট syslog সার্ভারে পাঠাও
`-P PORT`	রিমোট পোর্ট (ডিফল্ট ৫১৪)
`-T` / `-d`	রিমোটে TCP / UDP ব্যবহার করো
`--journald[=FILE]`	journald-এ সরাসরি স্ট্রাকচার্ড এন্ট্রি পাঠাও (ফিল্ডসহ)

উদাহরণ:

```
$ logger "হ্যালো লগ, এটা আমার প্রথম বার্তা"
$ journalctl -n 1
Nov 12 10:40:02 laptop rahim[5120]: হ্যালো লগ, এটা আমার প্রথম বার্তা

$ logger -t backup -p local0.err "Backup failed: disk full"
$ journalctl -t backup -n 1
Nov 12 10:41:15 laptop backup[5133]: Backup failed: disk full

$ journalctl -t backup -p err      # severity দিয়েও খুঁজে পাবে
```

স্ক্রিপ্টে বাস্তব ব্যবহার (অধ্যায় ৪১-এ স্ক্রিপ্টিং শিখবে):

```
#!/bin/bash
TAG="nightly-backup"
logger -t "$TAG" -p local0.info "Backup started"
if rsync -a /home/ /mnt/backup/home/; then
    logger -t "$TAG" -p local0.info "Backup finished successfully"
else
    logger -t "$TAG" -p local0.err "Backup FAILED with exit code $?"
fi
```

এখন cron দিয়ে চালানো এই স্ক্রিপ্টের ফলাফল `journalctl -t nightly-backup` দিয়ে যেকোনো সময় দেখতে পাবে (অধ্যায় ২৯)।

একটা কমান্ডের পুরো আউটপুট লগে পাঠাতে:

```
$ df -h | logger -t diskreport
```

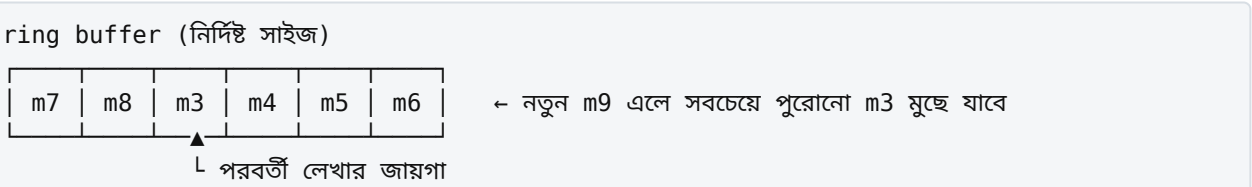
systemd-cat

`systemd-cat` কোনো কমান্ডের stdout/stderr সরাসরি journal-এ পাঠায়:

```
$ systemd-cat -t mytask -p info ls -l /etc/ssh
$ echo "manual note: server patched" | systemd-cat -t admin-note -p notice
$ journalctl -t admin-note
Nov 12 10:45:01 laptop admin-note[5201]: manual note: server patched
```

২৮.৯ `dmesg` — kernel-এর ডায়েরি

kernel বুট হওয়ার মুহূর্ত থেকেই বার্তা লেখে — কোন CPU পেল, কত RAM, কোন ডিস্ক, কোন ড্রাইভার লোড হলো, USB লাগানো হলো কি না। এগুলো যায় একটা নির্দিষ্ট সাইজের **ring buffer**-এ (মেমরিতে গোল চাকার মতো একটা জায়গা — ভরে গেলে সবচেয়ে পুরোনোটা মুছে নতুন লেখা হয়)। এই buffer পড়ার কমান্ড `dmesg` (display message)। ভেতরে এটা `/dev/kmsg` পড়ে।



এজন্য দীর্ঘদিন চলা সার্ভারে `dmesg`-এ বুটের শুরুর বার্তা নাও থাকতে পারে — তখন `journalctl -k -b` ব্যবহার করো।

```
$ dmesg
dmesg: read kernel buffer failed: Operation not permitted
$ sudo dmesg | head -5
[    0.000000] Linux version 6.8.0-48-generic (buildd@lcy02-amd64-040) ...
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-48-generic root=UUID=... ro
quiet splash
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
```

> 📝 **মনে রাখো:** Ubuntu ও Debian-এ `kernel.dmesg_restrict = 1` থাকে, তাই সাধারণ ইউজার `dmesg` চালাতে পারে না, `sudo` লাগে। বাঁ পাশের `[ 0.000000]` মানে বুটের পর কত সেকেন্ড পরে বার্তাটা

> এসেছে।

dmesg-এর অপশন

অপশন	কাজ
`-T`	মানুষের পড়ার মতো তারিখ-সময় (সাসপেন্ড/রিজিউমের পর সময় একটু ভুল হতে পারে)
`-H`	human-readable: রঙ, pager, আপেক্ষিক সময়
`-w`	follow — নতুন বার্তা আসলে দেখাবে (`--follow`)
`-l LEVEL`	নির্দিষ্ট লেভেল, যেমন `-l err,warn`
`-f FACILITY`	নির্দিষ্ট facility, যেমন `-f kern`
`-k`	শুধু kernel facility
`-x`	facility ও level লেখায় দেখাও
`-L`	রঙিন আউটপুট
`-c`	পড়ে buffer খালি করো
`-C`	না দেখিয়ে buffer খালি করো

```
$ sudo dmesg -T -l err,warn
[Wed Nov 12 09:01:14 2026] ACPI BIOS Error (bug): Could not resolve symbol
[\_SB.PCI0.GPP0], AE_NOT_FOUND
[Wed Nov 12 09:01:16 2026] usb 1-3: device descriptor read/64, error -71
```

USB ড্রাইভ লাগানোর সময় কী হয় দেখতে, একটা টার্মিনালে চালাও তারপর পেনড্রাইভ লাগাও:

```
$ sudo dmesg -Tw
[Wed Nov 12 10:50:21 2026] usb 2-1: new SuperSpeed USB device number 3 using xhci_hcd
[Wed Nov 12 10:50:21 2026] usb-storage 2-1:1.0: USB Mass Storage device detected
[Wed Nov 12 10:50:22 2026] sd 1:0:0:0: [sdb] 60555264 512-byte logical blocks: (31.0
GB/28.9 GiB)
[Wed Nov 12 10:50:22 2026]  sdb: sdb1
```

এখান থেকে জানলে নতুন ডিস্কের নাম `sdb` আর পার্টিশন `sdb1` — মাউন্ট করার জন্য ঠিক এই তথ্যই লাগে (অধ্যায় ৩২)।

dmesg-এ যে গুরুত্বপূর্ণ বার্তাগুলো চিনে রাখবে:

বার্তায় যা লেখা থাকে	মানে
`Out of memory: Killed process 1234 (java)`	RAM শেষ, OOM killer প্রসেস মেরেছে
`I/O error, dev sda, sector ...`	ডিস্কে পড়া/লেখার সমস্যা — ডিস্ক নষ্ট হচ্ছে হতে পারে
`EXT4-fs error` / `XFS ... Corruption`	ফাইলসিস্টেম করাপশন
`segfault at ... in libxyz.so`	কোনো প্রোগ্রাম ক্র্যাশ করেছে

বার্তায় যা লেখা থাকে	মানে
`link is not ready` / `Link is Up`	নেটওয়ার্ক কেবল খোলা/লাগানো
`temperature above threshold`	CPU অতিরিক্ত গরম
`audit: ... apparmor="DENIED"`	AppArmor কিছু ব্লক করেছে (অধ্যায় ৪৫)

kernel-এর ভেতরের গঠন ও লগ লেভেল (`/proc/sys/kernel/printk`) নিয়ে আরও জানতে অধ্যায় ৪৭।

২৮.১০ লগইন ইতিহাস: `last`, `lastb`, `lastlog`, `who`

```
$ last -n 5
rahim    pts/0        192.168.1.50     Wed Nov 12 10:15   still logged in
rahim    tty2         tty2             Wed Nov 12 09:02   still logged in
reboot   system boot  6.8.0-48-generic Wed Nov 12 09:01   still running
rahim    pts/1        192.168.1.50     Tue Nov 11 18:20 - 19:05  (00:45)
reboot   system boot  6.8.0-48-generic Tue Nov 11 09:00 - 22:15  (13:14)

wtmp begins Sat Nov  1 08:10:02 2026
```

কমান্ড	কী দেখায়	ফাইল
`last`	সফল লগইন, লগআউট, রিবুট	`/var/log/wtmp`
`last reboot`	শুধু রিবুটের ইতিহাস	`/var/log/wtmp`
`last -x`	shutdown ও runlevel পরিবর্তনও	`/var/log/wtmp`
`last rahim`	নির্দিষ্ট ইউজারের	`/var/log/wtmp`
`sudo lastb`	**ব্যর্থ** লগইন চেষ্টা	`/var/log/btmp`
`lastlog`	প্রতিটা ইউজারের সবশেষ লগইন	`/var/log/lastlog`
`who` / `w`	এই মুহূর্তে কে লগইন আছে	`/run/utmp`

```
$ sudo lastb -n 3
admin    ssh:notty    203.0.113.45     Wed Nov 12 03:14 - 03:14  (00:00)
root     ssh:notty    203.0.113.45     Wed Nov 12 03:14 - 03:14  (00:00)
test     ssh:notty    198.51.100.7     Wed Nov 12 02:58 - 02:58  (00:00)
```

ইন্টারনেটে খোলা যেকোনো SSH সার্ভারে এমন লাইন হাজারে হাজারে থাকবে — বট সারাক্ষণ চেষ্টা করছে। এজন্যই পাসওয়ার্ড লগইন বন্ধ করে key ব্যবহার করা উচিত (অধ্যায় ৩৮)।

২৮.১১ `logrotate` — লগ ফাইলের রক্ষণাবেক্ষণ

সমস্যা

লগ ফাইল শুধু বাড়ে, কখনো ছোট হয় না। একটা ব্যস্ত ওয়েব সার্ভারের `access.log` দিনে কয়েক GB হতে পারে। কিছু না করলে একদিন ডিস্ক ভরে যাবে, আর ডিস্ক ভরা মানে ডাটাবেস লিখতে পারবে না, সার্ভিস বন্ধ হবে।

সমাধান: rotation (ঘোরানো)

উপমা: অফিসের রেজিস্টার খাতা। মাস শেষে দারোয়ান খাতাটা বন্ধ করে আলমারিতে রাখে, নতুন খাতা শুরু করে। এক বছরের পুরোনো খাতা ফেলে দেওয়া হয়। `logrotate` ঠিক এটাই করে:

```
rotation-এর আগে:          rotation-এর পরে (rotate 4, compress, delaycompress):
syslog       (বড়)    ──►  syslog          (নতুন, খালি)
syslog.1                   syslog.1        (একটু আগের syslog)
syslog.2.gz                syslog.2.gz     (আগের syslog.1, এখন কম্প্রেসড)
syslog.3.gz                syslog.3.gz
syslog.4.gz                syslog.4.gz
                           (আগের syslog.4.gz মুছে গেছে)
```

logrotate কে চালায়

logrotate নিজে ডেমন নয়; এটা একবার চলে, কাজ করে, বের হয়ে যায়। আধুনিক distro-তে (Ubuntu 24.04, Debian 12, Fedora, RHEL 9) একটা systemd timer প্রতিদিন চালায়:

```
$ systemctl list-timers logrotate.timer
NEXT                         LEFT     LAST                         PASSED  UNIT
ACTIVATES
Thu 2026-11-13 00:00:00 +06 13h left Wed 2026-11-12 00:00:03 +06 10h ago
logrotate.timer logrotate.service
```

পুরোনো সিস্টেমে `/etc/cron.daily/logrotate` স্ক্রিপ্ট দিয়ে চলত। কোন ফাইল শেষবার কবে rotate হয়েছে তা মনে রাখে state ফাইলে: Debian/Ubuntu-তে `/var/lib/logrotate/status`, RHEL/Fedora-তে `/var/lib/logrotate/logrotate.status`।

কনফিগ ফাইল

```
/etc/logrotate.conf      ← গ্লোবাল ডিফল্ট + "include /etc/logrotate.d"
/etc/logrotate.d/        ← প্রতিটা প্যাকেজের নিজস্ব ফাইল (nginx, rsyslog, apt ...)
```

Ubuntu-র `/etc/logrotate.d/rsyslog` (সংক্ষেপে):

```
/var/log/syslog
/var/log/mail.log
/var/log/kern.log
/var/log/auth.log
```

```
{
        rotate 4
        weekly
        missingok
        notifempty
        compress
        delaycompress
        sharedscripts
        postrotate
                /usr/lib/rsyslog/rsyslog-rotate
        endscript
}
```

nginx-এর `/etc/logrotate.d/nginx` (Ubuntu, সংক্ষেপে):

```
/var/log/nginx/*.log {
        daily
        missingok
        rotate 14
        compress
        delaycompress
        notifempty
        create 0640 www-data adm
        sharedscripts
        postrotate
                invoke-rc.d nginx rotate >/dev/null 2>&1
        endscript
}
```

গুরুত্বপূর্ণ directive

directive	অর্থ
`daily` / `weekly` / `monthly` / `yearly`	কত ঘনঘন rotate
`rotate N`	কতগুলো পুরোনো কপি রাখবে (তারপর মুছবে); `rotate 0` মানে কোনো কপি নয়
`size 100M`	সাইজ ১০০M ছাড়ালে rotate (সময় যাই হোক)
`maxsize 500M`	সময় না হলেও এই সাইজ ছাড়ালে rotate
`minsize 1M`	সময় হলেও অন্তত এই সাইজ না হলে rotate নয়
`compress`	পুরোনো কপি gzip করো
`delaycompress`	সর্বশেষ পুরোনো কপি (`.1`) কম্প্রেস না করে পরের বারে করো
`missingok`	ফাইল না থাকলে error দেখিও না
`notifempty`	ফাইল খালি হলে rotate কোরো না
`create MODE OWNER GROUP`	rotate-এর পর নতুন খালি ফাইল এই পারমিশনে বানাও
`copytruncate`	ফাইলটা কপি করে আসল ফাইলটা শূন্য করে দাও (রিনেম না করে)

directive	অর্থ
`postrotate ... endscript`	rotate-এর পর এই কমান্ড চালাও
`prerotate ... endscript`	rotate-এর আগে চালাও
`sharedscripts`	একাধিক ফাইল থাকলেও postrotate একবারই চালাও
`dateext`	`.1`-এর বদলে তারিখ দিয়ে নাম, যেমন `app.log-20261112`
`maxage N`	N দিনের পুরোনো rotated ফাইল মুছে দাও
`olddir DIR`	পুরোনো কপি আলাদা ডিরেক্টরিতে রাখো
`su USER GROUP`	এই ইউজার/গ্রুপ হিসেবে rotate করো (ডিরেক্টরির পারমিশন সমস্যা এড়াতে)

ভেতরে কী ঘটে: postrotate ও copytruncate কেন লাগে

এটা লিনাক্সের একটা সূক্ষ্ম কিন্তু খুব গুরুত্বপূর্ণ বিষয়। nginx যখন `access.log` খোলে, kernel তাকে একটা **file descriptor** দেয়, যা আসলে ফাইলের নাম নয়, ফাইলের **inode**-কে নির্দেশ করে (অধ্যায় ১১)।

```
rotate-এর আগে:      nginx ──fd──► inode 5001  (নাম: access.log)

logrotate রিনেম করল: nginx ──fd──► inode 5001  (নাম এখন: access.log.1)
                     নতুন access.log = inode 5002 (খালি)

ফলাফল: nginx এখনো পুরোনো access.log.1-এ লিখছে! নতুন ফাইল খালিই থাকে।
```

সমাধান দুইটা:

1. **postrotate-এ সিগন্যাল পাঠানো** (ভালো পদ্ধতি): nginx-কে `USR1` সিগন্যাল (বা `rotate` / `reload` কমান্ড) পাঠালে সে লগ ফাইল নতুন করে খোলে, এবং নতুন inode 5002-তে লিখতে শুরু করে। এজন্য উপরের কনফিগে postrotate আছে। (সিগন্যাল নিয়ে অধ্যায় ২৪)
2. **copytruncate**: যে প্রোগ্রাম ফাইল নতুন করে খুলতে জানে না, তার জন্য। logrotate ফাইলের কপি বানায়, তারপর আসল ফাইলটা শূন্য করে দেয় — inode একই থাকে। অসুবিধা: কপি আর truncate-এর মাঝের মুহূর্তে লেখা কয়েকটা লাইন হারিয়ে যেতে পারে।

> ⚠️ **সতর্কতা:** লগ ফাইল বড় হয়ে গেলে চলমান সার্ভিসের লগ `rm` দিয়ে মুছবে না! ফাইলটা ডিরেক্টরি থেকে চলে যাবে, কিন্তু প্রোগ্রাম তখনো খোলা রাখায় ডিস্কের জায়গা খালি হবে না (`df` আগের মতোই ভরা দেখাবে)। নিরাপদ উপায়: `sudo truncate -s 0 /var/log/big.log` অথবা `: | sudo tee /var/log/big.log`। মুছে ফেলা কিন্তু খোলা ফাইল খুঁজতে: `sudo lsof +L1`।

নিজের অ্যাপের জন্য logrotate কনফিগ

```
$ sudo nano /etc/logrotate.d/myapp
```

```
/var/log/myapp/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0640 myapp adm
    dateext
    maxsize 200M
    sharedscripts
    postrotate
        systemctl reload myapp.service >/dev/null 2>&1 || true
    endscript
}
```

পরীক্ষা করা

```
# -d (debug): কী করবে দেখাবে, কিন্তু কিছু করবে না — সবসময় আগে এটা চালাও
$ sudo logrotate -d /etc/logrotate.d/myapp
reading config file /etc/logrotate.d/myapp
...
considering log /var/log/myapp/app.log
  log does not need rotating (log has already been rotated)

# -f (force): সময় না হলেও এখনই rotate করো
$ sudo logrotate -f /etc/logrotate.d/myapp

# -v (verbose): করার সময় বিস্তারিত দেখাও
$ sudo logrotate -vf /etc/logrotate.conf
```

> 📝 **মনে রাখো:** logrotate শুধু **টেক্সট লগ ফাইল** দেখে। systemd journal নিজের সাইজ নিজে সামলায় (`SystemMaxUse=` ইত্যাদি)। এই দুটো আলাদা ব্যবস্থা।

২৮.১২ ওয়েব সার্ভারের লগ: nginx উদাহরণ

nginx দুই ধরনের লগ লেখে:

- **access.log** — প্রতিটা HTTP রিকোয়েস্টের এক লাইন
- **error.log** — সমস্যা, কনফিগ error, upstream ব্যর্থতা

access.log-এর একটা লাইন (combined ফরম্যাট):

```
192.168.1.50 - - [12/Nov/2026:10:55:12 +0600] "GET /index.html HTTP/1.1" 200 615 "-"
"Mozilla/5.0 (X11; Linux x86_64)"
└──────┬──────┘     └──────────┬──────────┘ └──────────┬──────────┘ └┬┘ └┬┘ └┬┘
└──────────┬──────────┘
```

```
  ক্লায়েন্ট IP           সময়                        রিকোয়েস্ট       স্ট্যাটাস বাইট রেফারার     ব্রাউজার
(user agent)
```

অধ্যায় ২২ ও অধ্যায় ২৩-এর টুল দিয়ে বিশ্লেষণ:

```
# সবচেয়ে বেশি রিকোয়েস্ট পাঠানো ১০টা IP
$ sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head
   4521 203.0.113.10
    880 192.168.1.50
    ...

# স্ট্যাটাস কোড অনুযায়ী গণনা
$ sudo awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
  15230 200
    412 404
     37 500

# সব 5xx error
$ sudo awk '$9 ~ /^5/' /var/log/nginx/access.log | tail

# সবচেয়ে বেশি 404 পাওয়া URL
$ sudo awk '$9 == 404 {print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
| head

# error.log লাইভ
$ sudo tail -F /var/log/nginx/error.log
2026/11/12 10:58:03 [error] 1290#1290: *77 connect() failed (111: Connection refused)
while connecting to upstream, client: 192.168.1.50, server: example.com, request: "GET
/api HTTP/1.1", upstream: "http://127.0.0.1:8000/api"
```

শেষের লাইনটা পড়লেই বোঝা যায় — nginx ঠিক আছে, কিন্তু পেছনের অ্যাপ (পোর্ট ৮০০০) চলছে না। লগ ঠিক এভাবেই সমস্যার দিক দেখায়। ওয়েব সার্ভার সেটআপ বিস্তারিত অধ্যায় ৪০-এ।

> 💡 **টিপস:** টার্মিনালে লগ ঘাঁটার জন্য `lnav` (Log File Navigator) চমৎকার একটা টুল — রঙ, সময় অনুযায়ী একাধিক ফাইল মেলানো, SQL দিয়ে কুয়েরি। `sudo apt install lnav` / `sudo dnf install lnav` দিয়ে ইনস্টল করে `sudo lnav /var/log/nginx/` চালিয়ে দেখো।

২৮.১৩ লগ দিয়ে সমস্যা খোঁজার পদ্ধতি

অভিজ্ঞ অ্যাডমিনরা একটা নির্দিষ্ট ক্রমে চিন্তা করে। এটাকে মুখস্থ করে ফেলো:

```
১. লক্ষণ স্পষ্ট করো ──► "কী কাজ করছে না? কখন থেকে? কার জন্য?"
        │
২. সময় চিহ্নিত করো ──► সমস্যা প্রথম কখন হয়েছে → --since/--until
        │
৩. সঠিক লগ বাছো     ──► কোন সার্ভিস? → journalctl -u / নিজস্ব লগ ফাইল
        │
৪. গুরুতর আগে       ──► -p err, grep -i "error|fail|denied|refused|timeout"
```

```
          │
৫. প্রথম error খোঁজো ──► শেষেরটা নয়! প্রথমটাই সাধারণত মূল কারণ
          │
৬. প্রসঙ্গ পড়ো       ──► error-এর আগের-পরের লাইন (grep -B5 -A5)
          │
৭. নিচের স্তরে যাও   ──► kernel (dmesg), ডিস্ক (df -h), মেমরি (OOM), নেটওয়ার্ক
          │
৮. পুনরুৎপাদন করো    ──► journalctl -f চালিয়ে সমস্যাটা আবার ঘটাও
          │
৯. ঠিক করো ও যাচাই করো ──► লগে আর error আসছে না নিশ্চিত হও
```

> 📝 **মনে রাখো:** একটা মূল সমস্যা প্রায়ই অনেকগুলো পরবর্তী error তৈরি করে (cascade)। যেমন ডিস্ক ভরে গেলে ডাটাবেস error দেয়, তারপর অ্যাপ error দেয়, তারপর nginx 502 দেয়। শেষের 502 দেখে nginx ঘাঁটলে সময় নষ্ট হবে — **সবচেয়ে আগের error**-টা খোঁজো।

সাধারণ পরিস্থিতি ও কোথায় দেখবে

সমস্যা	প্রথমে কী চালাবে
সার্ভিস চালু হচ্ছে না	`systemctl status X`, `journalctl -xeu X`
কম্পিউটার হঠাৎ রিবুট/বন্ধ হয়েছে	`journalctl -b -1 -e`, `last -x reboot`
প্রসেস রহস্যজনকভাবে মরে গেছে	`journalctl -k -g "Out of memory"`, `dmesg -T | grep -i oom`
কেউ লগইন করেছে কি না	`/var/log/auth.log` বা `secure`, `last`, `lastb`, `journalctl -u ssh`
`sudo` দিয়ে কে কী চালিয়েছে	`journalctl _COMM=sudo` বা `grep sudo /var/log/auth.log`
ডিস্ক বা USB সমস্যা	`dmesg -T -l err,warn`, `journalctl -k -p err`
প্যাকেজ আপডেটের পর কিছু ভেঙেছে	`/var/log/apt/history.log`, `/var/log/dpkg.log`, `dnf history`
ওয়েবসাইট 502/500 দেখাচ্ছে	nginx `error.log`, তারপর অ্যাপের লগ
RHEL-এ পারমিশন ঠিক থাকলেও "Permission denied"	`/var/log/audit/audit.log`, `ausearch -m AVC -ts recent` (SELinux)
cron জব চলেনি	`journalctl -u cron` (Debian/Ubuntu) বা `-u crond` (RHEL), `/var/log/cron`
ডিস্ক ভরে যাচ্ছে	`sudo du -sh /var/log/* | sort -h`, `journalctl --disk-usage`

উদাহরণ ১: SSH brute-force শনাক্ত

```
$ sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -
c | sort -rn | head -5
   1532 203.0.113.45
```

```
    410 198.51.100.7
     12 192.168.1.77
```

(`$(NF-3)` মানে শেষ থেকে চার নম্বর ফিল্ড, যেখানে "from IP port N ssh2" ধাঁচের লাইনে IP থাকে।) RHEL-এ একই কাজ `/var/log/secure`-এ, আর journal দিয়ে:

```
$ sudo journalctl -u ssh --since today -g "Failed password" --no-pager | wc -l
```

উদাহরণ ২: সার্ভিস স্টার্ট হচ্ছে না

```
$ sudo systemctl restart nginx
Job for nginx.service failed because the control process exited with error code.
See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.

$ sudo journalctl -xeu nginx --no-pager | tail -5
Nov 12 11:05:10 web nginx[3301]: nginx: [emerg] unknown directive "servr_name" in
/etc/nginx/sites-enabled/default:23
Nov 12 11:05:10 web nginx[3301]: nginx: configuration file /etc/nginx/nginx.conf test
failed
Nov 12 11:05:10 web systemd[1]: nginx.service: Control process exited, code=exited,
status=1/FAILURE
Nov 12 11:05:10 web systemd[1]: nginx.service: Failed with result 'exit-code'.
```

লগ সরাসরি বলে দিচ্ছে: ফাইল `default`, লাইন ২৩, বানান ভুল `servr_name`।

উদাহরণ ৩: আগের রাতে সার্ভার কেন রিবুট হলো

```
$ last -x reboot shutdown | head -3
$ journalctl -b -1 -n 30 --no-pager
```

শেষ লাইনগুলোয় যদি স্বাভাবিক shutdown বার্তা থাকে (`Reached target ... Power-Off` / `System Reboot`), তাহলে কেউ বা কিছু ইচ্ছাকৃত রিবুট করেছে (`unattended-upgrades` স্বয়ংক্রিয় রিবুটও করতে পারে)। কোনো shutdown বার্তা ছাড়াই লগ হঠাৎ থেমে গেলে — বিদ্যুৎ বিভ্রাট, kernel panic, বা হার্ডওয়্যার সমস্যা।

উদাহরণ ৪: একটা সময়ের জানালায় সব কিছু

"১০:৪০ থেকে ১০:৪৫-এর মধ্যে সাইট ডাউন ছিল" — তখন সব উৎস একসাথে দেখো:

```
$ sudo journalctl --since "10:40" --until "10:45" -p warning --no-pager
```

হাতে-কলমে অনুশীলন

> ⚠️ **সতর্কতা:** এই ল্যাবগুলো নিজের ল্যাপটপ বা ভার্চুয়াল মেশিনে করো, প্রোডাকশন সার্ভারে নয়।

ল্যাব ১: লগ ফাইলের সাথে পরিচিতি

1. `ls -lh /var/log` চালাও। কোন ফাইলগুলো সবচেয়ে বড়?
2. তোমার distro-তে প্রধান টেক্সট লগ কোনটা — `syslog` না `messages`? নাকি কোনোটাই নেই?
3. `sudo tail -n 20 /var/log/auth.log` (বা `/var/log/secure`) চালাও।
4. আরেকটা টার্মিনালে `sudo ls` চালাও, তারপর আবার auth লগের শেষে দেখো — তোমার sudo কমান্ড রেকর্ড হয়েছে?

ল্যাব ২: journalctl অনুশীলন

```
$ journalctl --list-boots
$ journalctl -b -p err --no-pager
$ journalctl -u ssh --since today        # RHEL/Fedora: -u sshd
$ journalctl -k -n 20
$ journalctl --disk-usage
$ journalctl -n 1 -o json-pretty
$ journalctl -F _SYSTEMD_UNIT | wc -l
```

প্রতিটার আউটপুট দেখে নিজেকে বলো এটা কী দেখাচ্ছে।

ল্যাব ৩: লাইভ লগ দেখা

1. টার্মিনাল-১: `journalctl -f`
2. টার্মিনাল-২: `logger -t lab "Live test message"`
3. টার্মিনাল-১-এ বার্তাটা সাথে সাথে দেখা গেল? Ctrl+C দিয়ে থামাও।
4. এবার টার্মিনাল-১-এ `journalctl -f -p err` চালিয়ে টার্মিনাল-২-এ `logger -p user.err -t lab "an error"` আর `logger -p user.info -t lab "just info"` চালাও। কোনটা দেখা গেল, কেন?

ল্যাব ৪: নিজের অ্যাপের জন্য লগিং (Ubuntu/RHEL, যেখানে rsyslog আছে)

1. `/etc/rsyslog.d/30-lab.conf` বানাও:

   ```
   local5.*    /var/log/lab.log
   & stop
   ```

2. `sudo rsyslogd -N1` দিয়ে যাচাই, তারপর `sudo systemctl restart rsyslog`।
3. `logger -p local5.warning -t labapp "first lab entry"`

4. `sudo cat /var/log/lab.log` — বার্তাটা আছে? `/var/log/syslog`-এ নেই তো (`& stop`-এর কারণে)?
5. ফাইল না বানানো গেলে উপরের সতর্কতা অনুযায়ী `touch` ও `chown syslog:adm` করো।

ল্যাব ৫: logrotate পরীক্ষা

1. `/etc/logrotate.d/lab` বানাও:

   ```
   /var/log/lab.log {
       rotate 3
       daily
       compress
       missingok
       notifempty
       create 0640 syslog adm
       postrotate
           /usr/lib/rsyslog/rsyslog-rotate
       endscript
   }
   ```

   (RHEL-এ postrotate-এর ভেতরে `systemctl kill -s HUP rsyslog.service` দাও এবং ইউজার `root` রাখো।)
2. `sudo logrotate -d /etc/logrotate.d/lab` — কী করবে পড়ো।
3. কয়েকবার `logger -p local5.info "line"` চালিয়ে `sudo logrotate -f /etc/logrotate.d/lab` চালাও।
4. `ls -l /var/log/lab.log*` — `lab.log.1.gz` তৈরি হয়েছে? নতুন বার্তা `lab.log`-এ যাচ্ছে?

ল্যাব ৬: dmesg ও হার্ডওয়্যার

1. `sudo dmesg -T | tail` চালাও।
2. `sudo dmesg -Tw` চালু রেখে একটা পেনড্রাইভ লাগাও ও খোলো। ডিভাইসের নাম (`sdb` ইত্যাদি) বের করো।
3. `sudo dmesg -T -l err,warn` — কোনো সতর্কতা আছে? সেগুলোর মানে ইন্টারনেটে খুঁজে দেখো।

ল্যাব ৭: journal persistent করা

1. `ls -d /var/log/journal` — আছে কি?
2. না থাকলে ২৮.৭-এর পদ্ধতিতে persistent করো, তারপর রিবুট করে `journalctl --list-boots` দেখো।
3. `/etc/systemd/journald.conf.d/size.conf`-এ `SystemMaxUse=300M` দিয়ে `systemd-journald` রিস্টার্ট করো।

ল্যাব ৮: ছোট গোয়েন্দা কাজ

নিচের প্রশ্নগুলোর উত্তর শুধু লগ দেখে বের করো:

- এই কম্পিউটার শেষবার কখন রিবুট হয়েছিল?
- আজ কতবার `sudo` চালানো হয়েছে? (`journalctl _COMM=sudo --since today | grep -c COMMAND`)
- সবশেষ কোন প্যাকেজ ইনস্টল/আপডেট হয়েছে? (`tail /var/log/apt/history.log` বা `dnf history | head`)
- বর্তমান বুটে কোন সার্ভিস ফেইল করেছে? (`systemctl --failed` এবং `journalctl -b -p err`)

সারসংক্ষেপ

কাজ	কমান্ড
সাধারণ সিস্টেম লগ	`/var/log/syslog` (Debian/Ubuntu), `/var/log/messages` (RHEL)
অথেন্টিকেশন লগ	`/var/log/auth.log` (Debian/Ubuntu), `/var/log/secure` (RHEL)
লগের শেষ অংশ / লাইভ	`tail -n 50 FILE`, `tail -F FILE`, `less +F FILE`
কম্প্রেসড লগে খোঁজা	`zgrep PATTERN FILE.gz`, `zcat FILE.gz`
সব journal	`journalctl`
একটা সার্ভিস	`journalctl -u NAME`
লাইভ	`journalctl -f`, `journalctl -fu NAME`
এই বুট / আগের বুট	`journalctl -b`, `journalctl -b -1`, `journalctl --list-boots`
শুধু error	`journalctl -p err -b`
সময়ের জানালা	`journalctl --since "..." --until "..."`
kernel বার্তা	`journalctl -k`, `sudo dmesg -T`
শেষে + ব্যাখ্যা	`journalctl -xeu NAME`
JSON আউটপুট	`journalctl -o json` / `-o json-pretty`
ফিল্ড দিয়ে	`journalctl _PID=N`, `_UID=N`, `_COMM=X`, `-t TAG`
বার্তায় খোঁজা	`journalctl -g PATTERN`
journal সাইজ / পরিষ্কার	`journalctl --disk-usage`, `--vacuum-size=500M`, `--vacuum-time=2weeks`
persistent journal	`mkdir -p /var/log/journal` বা `Storage=persistent`
নিজে লগ লেখা	`logger -t TAG -p local0.err "msg"`, `systemd-cat -t TAG CMD`
kernel buffer লাইভ	`sudo dmesg -Tw`
লগইন ইতিহাস	`last`, `sudo lastb`, `lastlog`, `last reboot`

কাজ	কমান্ড
rsyslog কনফিগ যাচাই	`sudo rsyslogd -N1`
logrotate পরীক্ষা / জোর করে	`sudo logrotate -d FILE`, `sudo logrotate -f FILE`
মুছে ফেলা কিন্তু খোলা ফাইল	`sudo lsof +L1`

শর্টকাট (less / journalctl pager)	কাজ
`G` / `g`	শেষে / শুরুতে
`/`, `?`, `n`, `N`	খোঁজা ও পরের/আগের মিল
`F`	লাইভ follow মোড
`←` / `→`	লম্বা লাইন পাশে স্ক্রল
`q`	বের হওয়া
`Ctrl+C`	`tail -f` / `journalctl -f` / `dmesg -w` থামানো

নিজেকে যাচাই করো

1. Ubuntu-তে SSH লগইনের রেকর্ড কোন ফাইলে থাকে, আর RHEL 9-এ কোন ফাইলে?
2. syslog-এর severity লেভেলগুলো গুরুতর থেকে কম গুরুতর ক্রমে বলো। `err`-এর সংখ্যাসূচক মান কত?
3. rsyslog কনফিগে `mail.err` আর `mail.=err`-এর পার্থক্য কী?
4. আগের বুটের শুধু kernel বার্তা দেখার কমান্ড কী? এটা `dmesg` দিয়ে কেন সম্ভব নয়?
5. `Storage=auto` থাকলে journal কখন ডিস্কে আর কখন RAM-এ থাকে?
6. গতকাল সন্ধ্যা ৬টা থেকে ৭টার মধ্যে `nginx` সার্ভিসের লগ দেখার কমান্ড লেখো।
7. logrotate-এ `postrotate` স্ক্রিপ্টে সার্ভিসকে সিগন্যাল বা reload কেন পাঠানো হয়? `copytruncate` কখন ব্যবহার করবে?
8. একটা বড় লগ ফাইল `rm` দিয়ে মুছে দেওয়ার পরেও `df -h` জায়গা খালি দেখাচ্ছে না কেন? সমাধান কী?
9. একটা শেল স্ক্রিপ্ট থেকে `local2` facility আর `warning` severity দিয়ে "Disk almost full" বার্তা পাঠানোর কমান্ড কী?
10. কোনো প্রসেস হঠাৎ মরে গেছে, সন্দেহ RAM শেষ হয়ে গিয়েছিল। কোন কমান্ড দিয়ে নিশ্চিত হবে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. Ubuntu/Debian: `/var/log/auth.log`; RHEL/Fedora-পরিবার (rsyslog থাকলে): `/var/log/secure`। দুই জায়গাতেই `journalctl -u ssh` (বা `-u sshd`) দিয়েও দেখা যায়।
2. emerg (0), alert (1), crit (2), err (3), warning (4), notice (5), info (6), debug (7)। `err`-এর মান ৩।
3. `mail.err` মানে mail facility-র `err` এবং তার চেয়ে গুরুতর সব (crit, alert, emerg); `mail.=err` মানে শুধু ঠিক `err` লেভেল।
</details>

4. `journalctl -k -b -1`। `dmesg` শুধু বর্তমানে চলা kernel-এর মেমরির ring buffer পড়ে, রিবুটে সেটা মুছে যায়; অন্যদিকে persistent journal আগের বুটের kernel বার্তা ডিস্কে রেখে দেয়।
5. `/var/log/journal` ডিরেক্টরি থাকলে ডিস্কে (persistent), না থাকলে `/run/log/journal`-এ RAM-এ (volatile), যা রিবুটে মুছে যায়।
6. `journalctl -u nginx --since "yesterday 18:00" --until "yesterday 19:00"` অথবা নির্দিষ্ট তারিখ দিয়ে `--since "2026-11-11 18:00" --until "2026-11-11 19:00"`।
7. সার্ভিস ফাইলটা খোলা রাখে file descriptor (অর্থাৎ inode) দিয়ে, নাম দিয়ে নয়; রিনেমের পরও সে পুরোনো ফাইলে লিখতে থাকে। সিগন্যাল/reload পেলে সে নতুন ফাইল খোলে। যে প্রোগ্রাম লগ ফাইল নতুন করে খুলতে পারে না, তার জন্য `copytruncate` — তবে এতে অল্প কিছু লাইন হারানোর ঝুঁকি থাকে।
8. প্রোগ্রামটা ফাইল তখনো খোলা রেখেছে, তাই inode ও ডেটা ব্লক মুক্ত হয়নি। সমাধান: সার্ভিস রিস্টার্ট/reload করা; ভবিষ্যতে `rm`-এর বদলে `truncate -s 0 FILE` ব্যবহার করা। এমন ফাইল খুঁজতে `sudo lsof +L1`।
9. `logger -p local2.warning -t myscript "Disk almost full"`
10. `journalctl -k -g "Out of memory"` (বা `-b -1` যোগ করে আগের বুটে), অথবা `sudo dmesg -T | grep -i -E "out of memory|oom"`। সেখানে `Killed process PID (নাম)` লেখা থাকবে।

অধ্যায় ২৯: কাজ শিডিউল করা — cron, at ও systemd timers

> "যে কাজ দুইবার হাতে করেছ, তৃতীয়বার সেটা কম্পিউটারকে দিয়ে করাও।" — সিস্টেম অ্যাডমিনদের পুরোনো কথা

ভাবো তোমার একটা সার্ভার আছে। প্রতি রাত ২টায় ডাটাবেসের ব্যাকআপ নিতে হবে, প্রতি সপ্তাহে পুরোনো লগ মুছতে হবে, প্রতি ৫ মিনিটে ওয়েবসাইট চালু আছে কিনা দেখতে হবে। তুমি কি প্রতি রাতে ২টায় অ্যালার্ম দিয়ে উঠে কমান্ড চালাবে? অবশ্যই না। লিনাক্সে এই কাজগুলো **শিডিউল** (schedule — নির্দিষ্ট সময়ে স্বয়ংক্রিয়ভাবে চালানো) করে রাখা যায়। এই অধ্যায়ে আমরা শিখব সেই তিনটি প্রধান হাতিয়ার: **cron**, **at** এবং **systemd timers**।

এই অধ্যায়ে যা শিখবে

- শিডিউলিং কেন দরকার, আর লিনাক্সে কোন কাজের জন্য কোন টুল
- cron daemon কী, ভেতরে কীভাবে প্রতি মিনিটে কাজ খোঁজে
- `crontab -e`, `-l`, `-r`, `-u` — ইউজার crontab ম্যানেজ করা
- cron-এর ৫ ফিল্ডের সিনট্যাক্স — ডায়াগ্রাম, অপারেটর (`*` `,` `-` `/`) ও প্রচুর উদাহরণ
- বিশেষ স্ট্রিং: `@reboot`, `@daily`, `@hourly` ইত্যাদি
- সিস্টেম crontab: `/etc/crontab`, `/etc/cron.d/`, `/etc/cron.daily/` ইত্যাদি
- cron-এর সবচেয়ে বড় ফাঁদ: environment/PATH সমস্যা, `%` চিহ্ন, আউটপুট ও লগ
- anacron — যে কম্পিউটার সবসময় চালু থাকে না তার জন্য
- `at`, `atq`, `atrm`, `batch` — একবারের কাজ শিডিউল করা
- systemd timer — `OnCalendar`, `Persistent`, `OnBootSec` সহ সম্পূর্ণ উদাহরণ
- cron বনাম systemd timer — কখন কোনটা বেছে নেবে
- কে cron/at ব্যবহার করতে পারবে — `cron.allow` / `cron.deny`

২৯.১ শিডিউলিং কেন? বড় ছবিটা

দৈনন্দিন উপমা দিয়ে শুরু করি। তোমার মোবাইলে তিন ধরনের অ্যালার্ম/রিমাইন্ডার থাকে:

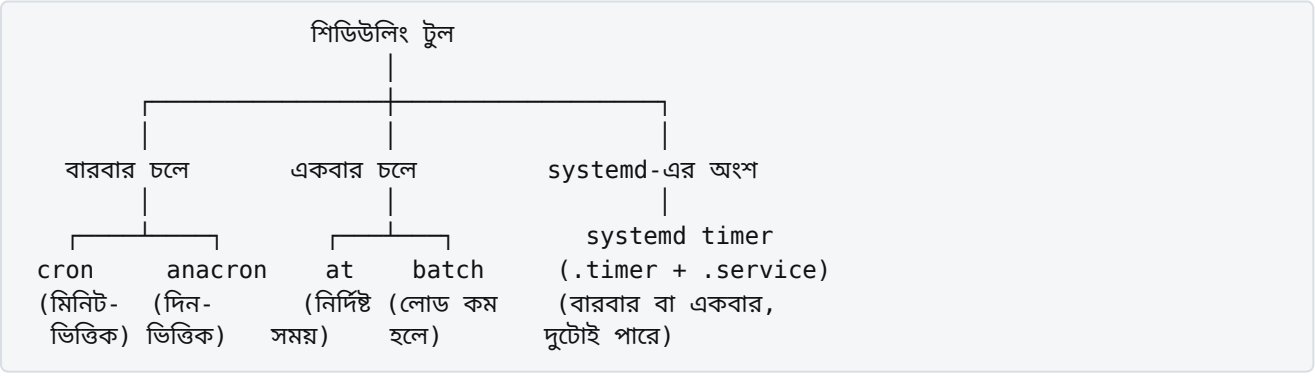
মোবাইলের জিনিস	লিনাক্সে সমতুল্য	কাজ
প্রতিদিন সকাল ৭টার অ্যালার্ম (repeat)	**cron** / **systemd timer**	বারবার, নির্দিষ্ট সময়ে

মোবাইলের জিনিস	লিনাক্সে সমতুল্য	কাজ
"আজ বিকেল ৫টায় মাকে ফোন করো" রিমাইন্ডার (একবার)	**at**	একবারই, নির্দিষ্ট সময়ে
"ফোন চার্জে থাকলে আপডেট করো"	**batch**	একবার, যখন সিস্টেম ফ্রি থাকবে
"ফোন বন্ধ ছিল, অ্যালার্ম মিস হয়েছে — চালু হলে জানাও"	**anacron** / `Persistent=true`	মিস হওয়া কাজ পরে চালানো

লিনাক্সে শিডিউলিং-এর কিছু সাধারণ ব্যবহার:

- ব্যাকআপ নেওয়া (অধ্যায় ৩৫)
- লগ রোটেট ও পরিষ্কার করা (অধ্যায় ২৮-এর logrotate নিজেই cron/timer দিয়ে চলে)
- SSL সার্টিফিকেট রিনিউ করা (certbot)
- প্যাকেজ আপডেট চেক করা
- রিপোর্ট বানিয়ে ইমেইল করা
- মনিটরিং স্ক্রিপ্ট চালানো

টুলগুলোর পরিবার



২৯.২ cron daemon — ভেতরে কীভাবে কাজ করে

daemon কী?

daemon (ডিমন) হলো এমন একটা প্রোগ্রাম যা ব্যাকগ্রাউন্ডে সবসময় চলতে থাকে, কোনো টার্মিনাল ছাড়াই (অধ্যায় ২৪ ও অধ্যায় ২৬ দেখো)। cron হলো এমনই একটা daemon। নাম এসেছে গ্রিক শব্দ *chronos* (সময়) থেকে।

উপমা: cron হলো অফিসের একজন অত্যন্ত নিয়মনিষ্ঠ দারোয়ান। তার হাতে একটা খাতা (crontab)। প্রতি মিনিটে সে ঘড়ি দেখে, খাতা খুলে মিলিয়ে দেখে — "এই মিনিটে কারো কোনো কাজ আছে?" থাকলে সে কাজটা করিয়ে দেয়, তারপর আবার পরের মিনিটের জন্য অপেক্ষা করে।

distro-ভেদে নাম

Distro	প্যাকেজ	সার্ভিসের নাম	ইউজার crontab কোথায় থাকে
Ubuntu 24.04 / Debian 12	`cron`	`cron.service`	`/var/spool/cron/crontabs/<user>`
Fedora / RHEL 9	`cronie`	`crond.service`	`/var/spool/cron/<user>`

সার্ভিস চলছে কিনা দেখো:

```
$ systemctl status cron          # Ubuntu/Debian
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-11-12 09:01:12 +06; 3h ago
       Docs: man:cron(8)
   Main PID: 812 (cron)
```

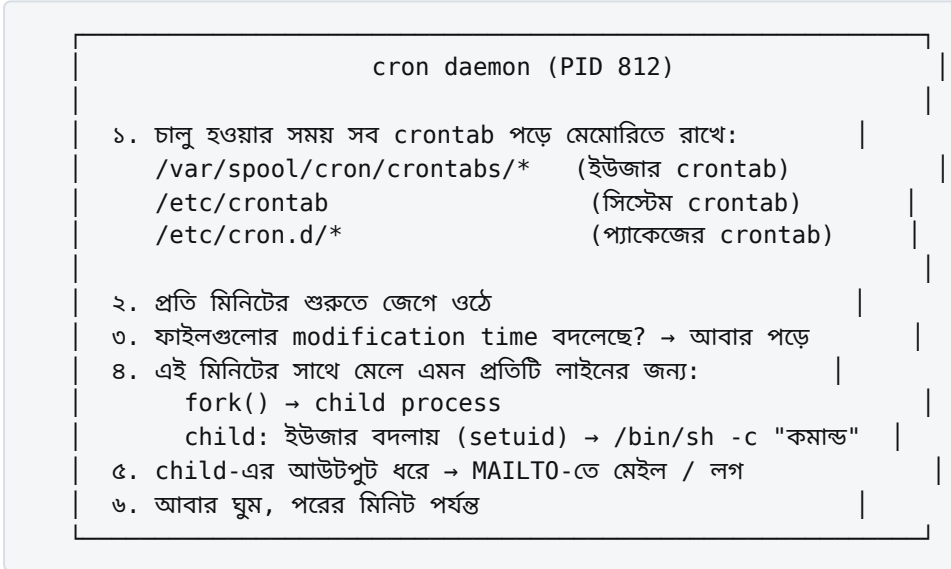
```
$ systemctl status crond         # Fedora/RHEL
```

না থাকলে ইনস্টল করো (অধ্যায় ৩০):

```
$ sudo apt install cron                      # Ubuntu/Debian
$ sudo dnf install cronie                    # Fedora/RHEL
$ sudo systemctl enable --now crond          # Fedora/RHEL-এ চালু করো
```

> 📝 মনে রাখো: কিছু মিনিমাল কন্টেইনার ইমেজ বা ক্লাউড ইমেজে cron ইনস্টল করা থাকে না। Fedora-র নতুন ভার্সনগুলো অনেক কাজ systemd timer দিয়ে করে, তাই সেখানেও cronie আলাদা করে লাগতে পারে।

ভেতরের কাজের ধারা (under the hood)



গুরুত্বপূর্ণ বিষয়গুলো:

- cron-এর সবচেয়ে ছোট একক হলো **১ মিনিট**। সেকেন্ড-ভিত্তিক শিডিউল cron পারে না (systemd timer পারে)।
- cron নিজে কমান্ড চালায় না — প্রতিটা কাজের জন্য একটা নতুন child process বানিয়ে `/bin/sh` দিয়ে চালায়। তাই কমান্ডটা **sh** সিনট্যাক্সে হতে হবে, bash-এর বিশেষ জিনিস (যেমন `[[ ]]`) সরাসরি নাও চলতে পারে। (Ubuntu/Debian-এ `/bin/sh` আসলে `dash`।)
- crontab ফাইল বদলালে cron রিস্টার্ট করতে হয় না — সে নিজেই প্রতি মিনিটে চেক করে।
- cron কাজ শেষ হওয়ার জন্য অপেক্ষা করে না। আগের রান শেষ না হতেই পরের মিনিটে আবার শুরু হতে পারে — এটা একটা বড় ফাঁদ (২৯.১০-এ `flock` দেখো)।

২৯.৩ crontab কমান্ড — নিজের কাজের খাতা

crontab শব্দের দুটো অর্থ: (১) একটা ফাইল যাতে শিডিউল লেখা থাকে (cron table), (২) সেই ফাইল এডিট করার কমান্ড।

> ⚠️ সতর্কতা: `/var/spool/cron/...` ফাইল কখনো সরাসরি এডিট করবে না। সবসময় `crontab` কমান্ড ব্যবহার করো। কারণ এটি সিনট্যাক্স চেক করে, সঠিক পারমিশন দেয়, আর cron-কে জানায়।

সিনট্যাক্স

```
crontab [-u user] -e | -l | -r | -i
crontab [-u user] file
```

অপশন	কাজ
`-e`	নিজের crontab এডিটরে খোলে (না থাকলে নতুন বানায়)
`-l`	crontab-এর বিষয়বস্তু দেখায় (list)
`-r`	পুরো crontab **মুছে ফেলে** (remove) — কোনো প্রশ্ন ছাড়াই!
`-i`	`-r`-এর সাথে ব্যবহার করলে মোছার আগে নিশ্চিত হতে চায়
`-u user`	অন্য ইউজারের crontab (শুধু root/sudo)
`file`	একটা ফাইল থেকে crontab ইনস্টল করে — **পুরোনোটা পুরো বদলে যায়**
`-`	stdin থেকে crontab পড়ে

প্রথমবার `crontab -e`

Ubuntu/Debian-এ প্রথমবার চালালে এডিটর বাছতে বলবে:

```
$ crontab -e
no crontab for nishat - using an empty one
```

```
Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.basic
  3. /usr/bin/vim.tiny
  4. /bin/ed

Choose 1-4 [1]: 1
```

নতুন হলে `nano` বেছে নাও (অধ্যায় ১৩)। পরে বদলাতে `select-editor` চালাও, অথবা `EDITOR` ভ্যারিয়েবল সেট করো:

```
$ EDITOR=vim crontab -e
```

Fedora/RHEL-এ ডিফল্ট এডিটর সাধারণত `vi` (RHEL 9) বা `EDITOR` ভ্যারিয়েবল অনুযায়ী (Fedora-য় ডিফল্ট `nano`)।

এডিটরে একটা লাইন লেখো, সেভ করে বের হও। তখন দেখবে:

```
crontab: installing new crontab
```

ভুল সিনট্যাক্স হলে:

```
"/tmp/crontab.Xb3kQe/crontab":23: bad minute
errors in crontab file, can't install.
Do you want to retry the same edit? (y/n)
```

`crontab -l` — দেখো

```
$ crontab -l
# m h  dom mon dow   command
0 2 * * * /home/nishat/bin/backup.sh
```

crontab না থাকলে:

```
$ crontab -l
no crontab for nishat
```

`crontab -r` — ভয়ংকর অপশন

```
$ crontab -r
$ crontab -l
no crontab for nishat
```

কোনো প্রশ্ন নেই, কোনো undo নেই! আর কীবোর্ডে `-e` আর `-r` পাশাপাশি — একটু হাত ফসকালেই সব শেষ।

> 💡 টিপস: নিয়মিত ব্যাকআপ রাখো: `crontab -l > ~/crontab.backup`। ফেরত আনতে: `crontab ~/crontab.backup`। আর নিরাপদ থাকতে মুছতে চাইলে `crontab -ri` ব্যবহার করো:
>
> ```
> $ crontab -ri
> crontab: really delete nishat's crontab? (y/n) n
> ```

অন্য ইউজারের crontab

```
$ sudo crontab -u www-data -l
$ sudo crontab -u www-data -e
```

ফাইল থেকে ইনস্টল ও স্ক্রিপ্টে লাইন যোগ করা

```
$ crontab mycron.txt                    # পুরোনো crontab মুছে এটা বসে যায়
$ (crontab -l 2>/dev/null; echo "*/5 * * * * /usr/local/bin/check.sh") | crontab -
```

দ্বিতীয় কমান্ডটা পুরোনো লাইন রেখে একটা নতুন লাইন যোগ করে — স্ক্রিপ্টে অটোমেশনের জন্য খুব কাজের (অধ্যায় ১৯-এর পাইপ ব্যবহার করে)।

২৯.৪ cron-এর ৫ ফিল্ড সিনট্যাক্স — সবচেয়ে গুরুত্বপূর্ণ অংশ

প্রতিটি crontab লাইনে ৫টি সময়-ফিল্ড, তারপর কমান্ড:

```
┌──────────────── মিনিট (minute)        0 - 59
│ ┌────────────── ঘণ্টা (hour)          0 - 23   (২৪ ঘণ্টার হিসাব)
│ │ ┌──────────── মাসের দিন (day of month) 1 - 31
│ │ │ ┌────────── মাস (month)           1 - 12  অথবা jan - dec
│ │ │ │ ┌──────── সপ্তাহের দিন (day of week) 0 - 7  (0 ও 7 দুটোই রবিবার) অথবা sun - sat
│ │ │ │ │
│ │ │ │ │
* * * * *  কমান্ড যা চালাতে হবে
```

মনে রাখার কৌশল: **"মি-ঘ-দি-মা-স"** — মিনিট, ঘণ্টা, দিন, মাস, সপ্তাহ। ছোট থেকে বড়, শুধু শেষেরটা (সপ্তাহের দিন) আলাদা।

সপ্তাহের দিনের সংখ্যা

সংখ্যা	0	1	2	3	4	5	6	7
দিন	রবি (sun)	সোম (mon)	মঙ্গল (tue)	বুধ (wed)	বৃহঃ (thu)	শুক্র (fri)	শনি (sat)	রবি (sun)

অপারেটর

চিহ্ন	অর্থ	উদাহরণ	ব্যাখ্যা
`*`	প্রতিটি মান	`*` ঘণ্টায়	প্রতি ঘণ্টা
`,`	তালিকা	`1,15,30`	১, ১৫ আর ৩০ তারিখে
`-`	পরিসর (range)	`1-5` সপ্তাহে	সোম থেকে শুক্র
`/`	ধাপ (step)	`*/10` মিনিটে	প্রতি ১০ মিনিট (0,10,20,...)
মিশ্র	একসাথে	`0-30/10`	0, 10, 20, 30 মিনিটে
নাম	মাস/দিনের নাম	`jan`, `mon`	তিন অক্ষর, case-insensitive

> 📝 মনে রাখো: নাম (যেমন `mon`) range-এ ব্যবহার করা নিয়ে cron বাস্তবায়নে পার্থক্য আছে — Debian-এর cron-এর man page বলে নামের range বা list চলবে না। নিরাপদে থাকতে range-এ সংখ্যা ব্যবহার করো: `1-5`, `mon-fri` নয়।

`*/10` কীভাবে কাজ করে: `*` মানে 0-59, `/10` মানে সেখান থেকে প্রতি ১০টা পরপর নাও → 0, 10, 20, 30, 40, 50। তাই `*/7` মিনিটে মানে 0, 7, 14, ..., 56 — তারপর আবার 0। অর্থাৎ ঘণ্টার শেষে 56 থেকে 0-তে মাত্র ৪ মিনিটের ফাঁক! `/` ধাপ প্রতি ঘণ্টায় নতুন করে গোনা শুরু হয়, একটানা না।

উদাহরণ টেবিল (মুখস্থ না, বুঝে নাও)

crontab লাইন	কখন চলবে
`* * * * *`	প্রতি মিনিটে
`*/5 * * * *`	প্রতি ৫ মিনিটে
`0 * * * *`	প্রতি ঘণ্টার শুরুতে (১:০০, ২:০০ ...)
`30 * * * *`	প্রতি ঘণ্টার ৩০ মিনিটে
`0 2 * * *`	প্রতিদিন রাত ২:০০টায়
`30 14 * * *`	প্রতিদিন দুপুর ২:৩০-এ
`0 9 * * 1-5`	সোম–শুক্র সকাল ৯টায়
`0 9 * * 0,6`	শনি ও রবিবার সকাল ৯টায়
`0 0 1 * *`	প্রতি মাসের ১ তারিখ মধ্যরাতে
`0 0 1 1 *`	প্রতি বছর ১ জানুয়ারি মধ্যরাতে
`15 10 * * 5`	প্রতি শুক্রবার সকাল ১০:১৫-এ
`0 */6 * * *`	প্রতি ৬ ঘণ্টায় (০, ৬, ১২, ১৮টায়)
`0 9-17 * * 1-5`	অফিস টাইমে (৯টা–৫টা) প্রতি ঘণ্টায়, কর্মদিবসে

crontab লাইন	কখন চলবে
`*/15 9-17 * * 1-5`	অফিস টাইমে প্রতি ১৫ মিনিটে
`0 0 * * 0`	প্রতি রবিবার মধ্যরাতে
`0 3 1,15 * *`	প্রতি মাসের ১ ও ১৫ তারিখ রাত ৩টায়
`0 0 1 */3 *`	প্রতি তিন মাসে একবার (জানু, এপ্রিল, জুলাই, অক্টো ১ তারিখ)
`5 4 * * sun`	প্রতি রবিবার ভোর ৪:০৫-এ
`0 22 * * 1-5`	কর্মদিবসে রাত ১০টায়

ফাঁদ: দিন আর সপ্তাহের দিন একসাথে (OR লজিক)

ধরো লিখলে:

```
0 9 13 * 5    /usr/local/bin/script.sh
```

তুমি হয়তো ভাবছ "১৩ তারিখ যদি শুক্রবার হয় তবে চলবে"। **ভুল!** যখন মাসের দিন (৩য় ফিল্ড) আর সপ্তাহের দিন (৫ম ফিল্ড) দুটোই `*` না, cron দুটোর মধ্যে **OR** করে: প্রতি মাসের ১৩ তারিখে **অথবা** প্রতি শুক্রবারে চলবে।

"মাসের শেষ শুক্রবার" বা "১৩ তারিখ শুক্রবার" এর মতো শর্ত দিতে স্ক্রিপ্টের ভেতরে চেক করো:

```
0 9 13 * *   [ "$(date +\%u)" -eq 5 ] && /usr/local/bin/script.sh
```

(এখানে `\%` কেন — পরের অংশে দেখো।)

`%` চিহ্নের ফাঁদ

crontab-এর কমান্ড অংশে `%` চিহ্নের বিশেষ অর্থ আছে: এটা **নতুন লাইন** হিসেবে গণ্য হয়, আর প্রথম `%`-এর পরের সব কিছু কমান্ডের stdin-এ চলে যায়। তাই `date +%Y-%m-%d` crontab-এ লিখলে কাজ করবে না।

```
# ভুল — কমান্ড ভেঙে যাবে
0 2 * * * tar czf /backup/home-$(date +%F).tar.gz /home

# ঠিক — % এর আগে ব্যাকস্ল্যাশ
0 2 * * * tar czf /backup/home-$(date +\%F).tar.gz /home
```

> 💡 টিপস: সবচেয়ে ভালো অভ্যাস হলো জটিল কমান্ড crontab-এ না লিখে একটা স্ক্রিপ্ট ফাইলে লেখা (অধ্যায় ৪১), আর crontab-এ শুধু স্ক্রিপ্টটা ডাকা। তাহলে `%`, quoting, PATH — সব ঝামেলা কমে যায়।

crontab ফাইলে আর কী লেখা যায়

```
# '#' দিয়ে শুরু হওয়া লাইন কমেন্ট
# ফাঁকা লাইনও চলে
```

```
# environment ভ্যারিয়েবল সেট করা (লাইনের শুরুতে NAME=value)
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=admin@example.com

# কাজ
0 2 * * * /home/nishat/bin/backup.sh
```

> ⚠️ সতর্কতা: crontab ফাইলের শেষ লাইনের পরে অবশ্যই একটা newline থাকতে হবে। না থাকলে কিছু cron বাস্তবায়ন শেষ লাইনটা উপেক্ষা করে (Debian cron "premature EOF" এরর দেয়)। `crontab -e` দিয়ে এডিট করলে সাধারণত এডিটর নিজেই newline দেয়।

> 📝 মনে রাখো: crontab-এর ভ্যারিয়েবল লাইনে shell expansion হয় না। `PATH=$PATH:/opt/bin` লিখলে `$PATH` আক্ষরিক টেক্সট হিসেবেই থাকবে। পুরো পাথ নিজে লিখে দাও।

টাইমজোন

cron সিস্টেমের লোকাল টাইমজোন অনুযায়ী চলে (`timedatectl` দিয়ে দেখো)। সার্ভার UTC-তে থাকলে "রাত ২টা" মানে UTC রাত ২টা — বাংলাদেশে তখন সকাল ৮টা!

```
$ timedatectl | grep "Time zone"
               Time zone: Asia/Dhaka (+06, +0600)
```

cronie (Fedora/RHEL) crontab-এর ভেতরে `CRON_TZ=Asia/Dhaka` ভ্যারিয়েবল সমর্থন করে, যা পরের লাইনগুলোর সময় সেই টাইমজোনে ধরে। Debian/Ubuntu-র cron-এ এটা নির্ভরযোগ্য নয় — সেখানে সিস্টেম টাইমজোনের উপর ভরসা করো, বা systemd timer ব্যবহার করো (যা `OnCalendar`-এ টাইমজোন নিতে পারে)।

ডেলাইট সেভিং টাইম (DST) আছে এমন দেশে ঘড়ি এগোনো/পেছানোর সময় কিছু কাজ একবার বাদ পড়তে বা দুবার চলতে পারে। বাংলাদেশে DST নেই, তবে বিদেশি সার্ভারে কাজ করলে মাথায় রেখো — DST-এর সময় (সাধারণত রাত ১টা–৩টা) জরুরি কাজ শিডিউল না করাই ভালো।

২৯.৫ বিশেষ স্ট্রিং — `@reboot`, `@daily` ইত্যাদি

৫টা ফিল্ডের বদলে কিছু শর্টকাট শব্দ ব্যবহার করা যায়:

স্ট্রিং	অর্থ	সমতুল্য
`@reboot`	cron daemon চালু হওয়ার সময় একবার (অর্থাৎ বুটে)	—
`@yearly`	বছরে একবার	`0 0 1 1 *`

স্ট্রিং	অর্থ	সমতুল্য
`@annually`	`@yearly`-এর মতোই	`0 0 1 1 *`
`@monthly`	মাসে একবার	`0 0 1 * *`
`@weekly`	সপ্তাহে একবার (রবিবার মধ্যরাত)	`0 0 * * 0`
`@daily`	দিনে একবার	`0 0 * * *`
`@midnight`	`@daily`-এর মতোই	`0 0 * * *`
`@hourly`	ঘণ্টায় একবার	`0 * * * *`

উদাহরণ:

```
@reboot   /home/nishat/bin/start-my-app.sh
@daily    /usr/local/bin/cleanup-tmp.sh
@hourly   /usr/local/bin/sync-data.sh
```

> ⚠️ সতর্কতা: `@reboot` আসলে "cron daemon চালু হলে" চলে, এটা নিশ্চিত করে না যে নেটওয়ার্ক বা ডাটাবেস তখন তৈরি। বুটে কোনো সার্ভিস চালাতে হলে systemd service (অধ্যায় ২৬) অনেক ভালো — সেখানে `After=network-online.target` দিয়ে নির্ভরতা বলা যায়। আর Debian cron-এ `cron` সার্ভিস রিস্টার্ট করলে `@reboot` কাজ আবার চলে না (এটা বুট শনাক্ত করে একটা ফাইল দিয়ে)।

২৯.৬ সিস্টেম crontab — `/etc/crontab`, `/etc/cron.d/` ও `cron.daily`

ইউজার crontab ছাড়াও সিস্টেম-লেভেলের শিডিউল থাকে। এগুলো root এডিট করে, আর এদের একটা বড় পার্থক্য আছে: **ইউজারের নাম লেখার একটা অতিরিক্ত ফিল্ড**।

```
ইউজার crontab (crontab -e):
  মিনিট ঘণ্টা দিন মাস সপ্তাহ        কমান্ড
  0     2     *   *   *              /home/nishat/backup.sh

সিস্টেম crontab (/etc/crontab, /etc/cron.d/*):
  মিনিট ঘণ্টা দিন মাস সপ্তাহ  ইউজার  কমান্ড
  0     2     *   *   *       root    /usr/local/bin/backup.sh
                              ^^^^
                              কোন ইউজার হিসেবে চলবে
```

পুরো কাঠামো এক নজরে

```
/etc/
├── crontab              ← প্রধান সিস্টেম crontab (ইউজার ফিল্ড সহ)
├── cron.d/              ← প্যাকেজগুলো নিজেদের crontab এখানে রাখে (ইউজার ফিল্ড সহ)
```

```
│   ├── e2scrub_all
│   ├── php               (php ইনস্টল থাকলে, সেশন পরিষ্কারের জন্য)
│   └── certbot           (কিছু ইনস্টলেশনে)
├── cron.hourly/          ← এখানে রাখা স্ক্রিপ্ট ঘণ্টায় একবার চলে
├── cron.daily/           ← দিনে একবার
│   ├── apt-compat
│   ├── dpkg
│   ├── logrotate         (অনেক সিস্টেমে এখন systemd timer দিয়ে চলে)
│   └── man-db
├── cron.weekly/          ← সপ্তাহে একবার
├── cron.monthly/         ← মাসে একবার
├── cron.allow / cron.deny  ← কে crontab ব্যবহার করতে পারবে
└── anacrontab            ← anacron-এর কনফিগ
```

`/etc/crontab` — Ubuntu/Debian

```
$ cat /etc/crontab
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report
/etc/cron.daily; }
47 6    * * 7   root    test -x /usr/sbin/anacron || { cd / && run-parts --report
/etc/cron.weekly; }
52 6    1 * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report
/etc/cron.monthly; }
```

(ভার্সন ভেদে `{ ...; }`-এর বদলে `( ... )` দেখতে পারো — অর্থ একই।)

লাইনগুলো ভেঙে বুঝি:

- **প্রথম লাইন**: প্রতি ঘণ্টার ১৭ মিনিটে `run-parts` দিয়ে `/etc/cron.hourly/`-এর সব স্ক্রিপ্ট চালায়।
- **বাকি তিন লাইন**: `test -x /usr/sbin/anacron ||` মানে — "যদি anacron ইনস্টল থাকে, তাহলে কিছু করো না (anacron নিজেই daily/weekly/monthly সামলাবে); না থাকলে এখানেই চালাও।" ২৯.৮-এ anacron বিস্তারিত দেখব।

কেন ১৭, ২৫, ৪৭, ৫২-এর মতো অদ্ভুত মিনিট? যাতে হাজার হাজার সার্ভার একই সাথে ঠিক ০ মিনিটে ভারী কাজ শুরু না করে, আর বিভিন্ন কাজ একে অপরের সাথে ধাক্কা না খায়।

`/etc/crontab` — Fedora/RHEL

RHEL 9-এ `/etc/crontab`-এ শুধু কমেন্ট আর ভ্যারিয়েবল থাকে, কোনো কাজ থাকে না। ঘণ্টার কাজটা আসে `/etc/cron.d/0hourly` থেকে:

```
$ cat /etc/cron.d/0hourly
# Run the hourly jobs
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
01 * * * * root run-parts /etc/cron.hourly
```

আর `/etc/cron.hourly/0anacron` স্ক্রিপ্টটি anacron চালায়, যা daily/weekly/monthly সামলায়।

`run-parts` কী?

`run-parts` একটা ছোট প্রোগ্রাম যা একটা ডিরেক্টরির ভেতরের সব executable ফাইল বর্ণানুক্রমে একে একে চালায়। তাই `/etc/cron.daily/`-এ একটা স্ক্রিপ্ট রেখে দিলেই সেটা দিনে একবার চলবে — কোনো সময় লিখতে হয় না।

```
$ run-parts --test /etc/cron.daily        # কী কী চলবে দেখায়, চালায় না (Debian)
/etc/cron.daily/apt-compat
/etc/cron.daily/dpkg
/etc/cron.daily/man-db
```

> ⚠️ সতর্কতা: Debian/Ubuntu-র `run-parts` ফাইলের নামে **ডট (`.`) থাকলে সেই ফাইল উপেক্ষা করে**! নাম শুধু অক্ষর, সংখ্যা, `_` আর `-` দিয়ে হতে হবে। তাই `/etc/cron.daily/backup.sh` কখনো চলবে না — নাম রাখো `backup`। এটা নতুনদের সবচেয়ে বেশি হওয়া ভুলগুলোর একটা। একই নিয়ম `/etc/cron.d/`-এর ফাইলের নামেও প্রযোজ্য। আর ফাইলটা অবশ্যই executable হতে হবে: `sudo chmod +x /etc/cron.daily/backup`।

`/etc/cron.d/` — নিজের সিস্টেম কাজ রাখার সেরা জায়গা

`/etc/crontab` সরাসরি এডিট করার চেয়ে `/etc/cron.d/`-এ আলাদা ফাইল বানানো ভালো — প্যাকেজ আপডেটে হারায় না, আর প্রতিটা কাজ আলাদা ফাইলে গোছানো থাকে।

```
$ sudo nano /etc/cron.d/db-backup
```

```
# প্রতিদিন রাত ২:৩০-এ postgres ইউজার হিসেবে ডাটাবেস ব্যাকআপ
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
MAILTO=admin@example.com

30 2 * * * postgres /usr/local/bin/pg-backup >> /var/log/pg-backup.log 2>&1
```

শর্ত:

- ফাইলের মালিক root হতে হবে, group/others লিখতে পারবে না (`sudo chmod 644 /etc/cron.d/db-backup`)
- ফাইলের নামে ডট নয় (Debian/Ubuntu)
- শেষে newline
- `/etc/cron.d/` -এর ফাইল executable হওয়ার দরকার নেই (এগুলো crontab, স্ক্রিপ্ট নয়)

কোনটা কখন ব্যবহার করবে

পরিস্থিতি	জায়গা
নিজের ব্যক্তিগত কাজ (ইউজার হিসেবে)	`crontab -e`
সিস্টেমের কাজ, নির্দিষ্ট সময় দরকার	`/etc/cron.d/<নাম>`
সিস্টেমের কাজ, "দিনে একবার" হলেই হলো	`/etc/cron.daily/<নাম>` (executable স্ক্রিপ্ট)
`/etc/crontab`	সাধারণত হাত দিও না

সব cron কাজ এক জায়গায় দেখা

```
$ sudo ls /var/spool/cron/crontabs/          # কোন কোন ইউজারের crontab আছে
(Ubuntu/Debian)
nishat  www-data
$ sudo ls /var/spool/cron/                   # Fedora/RHEL
$ ls /etc/cron.d/ /etc/cron.daily/
$ for u in $(cut -d: -f1 /etc/passwd); do sudo crontab -u "$u" -l 2>/dev/null | sed
"s/^/$u: /"; done
```

শেষ লাইনটা প্রতিটা ইউজারের crontab দেখায় — সার্ভার অডিটের সময় কাজের (অধ্যায় ৪৫)। আক্রমণকারীরা প্রায়ই cron-এ লুকিয়ে কাজ বসায়!

২৯.৭ cron-এর environment ও PATH সমস্যা, আউটপুট ও লগ

"টার্মিনালে চলে, cron-এ চলে না!" — এক নম্বর অভিযোগ

এটা লিনাক্সে সবচেয়ে বেশি জিজ্ঞাসা করা প্রশ্নগুলোর একটা। কারণ বোঝো:

```
তোমার টার্মিনাল (interactive login shell)          cron-এর কাজ
──────────────────────────────────────────          ──────────────────────
~/.bashrc, ~/.profile পড়া হয়েছে                    কিছুই পড়া হয় না
PATH=/usr/local/sbin:/usr/local/bin:                   PATH=/usr/bin:/bin
     /usr/sbin:/usr/bin:/sbin:/bin:                         (খুবই ছোট!)
     /snap/bin:/home/nishat/.local/bin
SHELL=/bin/bash                                        SHELL=/bin/sh (dash)
তোমার alias, function আছে                           কিছু নেই
```

```
   current directory = যেখানে আছো                    current directory = HOME
   DISPLAY, টার্মিনাল আছে                           কোনো টার্মিনাল নেই
   nvm/pyenv/conda সেট করা                            কিছুই সেট নেই
```

cron একদম ন্যূনতম environment দেয় — সাধারণত শুধু `SHELL=/bin/sh`, `PATH=/usr/bin:/bin`, `HOME`, `LOGNAME` (আর `USER`)। তাই `/usr/local/bin`, `/usr/sbin`, `~/.local/bin`, `/snap/bin`-এর কোনো কমান্ড পাওয়া যাবে না।

cron আসলে কী environment দেয় নিজের চোখে দেখো:

```
* * * * * env > /tmp/cron-env.txt
```

এক মিনিট পরে:

```
$ cat /tmp/cron-env.txt
HOME=/home/nishat
LOGNAME=nishat
PATH=/usr/bin:/bin
LANG=en_US.UTF-8
SHELL=/bin/sh
PWD=/home/nishat
```

(পরে লাইনটা মুছে দিও!)

সমাধানগুলো

১. কমান্ডের পুরো পাথ লেখো (সবচেয়ে নিরাপদ):

```
$ which mysqldump
/usr/bin/mysqldump
```

```
0 2 * * * /usr/bin/mysqldump mydb > /backup/mydb.sql
```

২. crontab-এর উপরে PATH সেট করো:

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

৩. bash লাগলে SHELL বদলাও:

```
SHELL=/bin/bash
```

৪. স্ক্রিপ্টের ভেতরেই environment ঠিক করো:

```
#!/bin/bash
export PATH=/usr/local/bin:/usr/bin:/bin
cd /home/nishat/myproject || exit 1
```

```
source /home/nishat/myproject/.env
./run-report
```

৫. রিলেটিভ পাথ এড়াও — cron-এ current directory হলো HOME, তাই `./data/file.txt` ভুল জায়গা খুঁজবে। সবসময় absolute path, অথবা স্ক্রিপ্টের শুরুতে `cd`।

৬. GUI কমান্ড (যেমন `notify-send`) cron থেকে চালাতে `DISPLAY` ও `DBUS_SESSION_BUS_ADDRESS` লাগে — ঝামেলার। ডেস্কটপ রিমাইন্ডারের জন্য systemd user timer সহজ।

cron-এর আউটপুট কোথায় যায়?

cron কাজের stdout বা stderr-এ যা আসে, cron সেটা **ইমেইল** করে পাঠায় crontab-এর মালিকের কাছে (বা `MAILTO`-তে)। কিন্তু আধুনিক ডেস্কটপ/সার্ভারে সাধারণত মেইল সার্ভার (MTA — Mail Transfer Agent, যেমন postfix) থাকে না, তাই আউটপুট হারিয়ে যায়। Ubuntu/Debian-এ লগে দেখবে:

```
CRON[4521]: (CRON) info (No MTA installed, discarding output)
```

এই লাইনটা দেখলে বুঝবে — কাজ চলেছে, কিছু বলেছে (হয়তো এরর!), কিন্তু তুমি সেটা দেখতে পাওনি।

আউটপুট নিজে লগ ফাইলে পাঠাও

রিডাইরেকশন (অধ্যায় ১৯) দিয়ে:

```
# stdout আর stderr দুটোই লগ ফাইলে যোগ (append) করো
0 2 * * * /usr/local/bin/backup >> /var/log/backup.log 2>&1

# শুধু এরর রাখো, সাধারণ আউটপুট ফেলে দাও
0 2 * * * /usr/local/bin/backup > /dev/null

# সব ফেলে দাও (মেইলও যাবে না) — সাবধানে, এরর লুকিয়ে যাবে
*/5 * * * * /usr/local/bin/ping-check > /dev/null 2>&1

# systemd journal-এ পাঠাও, ট্যাগ দিয়ে
0 2 * * * /usr/local/bin/backup 2>&1 | logger -t mybackup
```

> ⚠️ সতর্কতা: `2>&1 >> file` আর `>> file 2>&1` এক জিনিস না! ক্রম গুরুত্বপূর্ণ। সঠিক হলো `>> file 2>&1` — আগে stdout ফাইলে পাঠাও, তারপর stderr-কে stdout-এর জায়গায় পাঠাও।

শেষ উদাহরণের `logger` দিয়ে পাঠানো লগ দেখতে:

```
$ journalctl -t mybackup --since today
```

MAILTO

```
MAILTO=admin@example.com      # এই ঠিকানায় আউটপুট মেইল করো (MTA লাগবে)
MAILTO=""                     # কোনো মেইল পাঠাবে না
```

cron নিজে কী লগ করে

cron প্রতিটা কাজ শুরুর একটা লাইন লগ করে। কোথায় দেখবে:

Distro	কমান্ড/ফাইল
Ubuntu 24.04 / Debian 12	`journalctl -u cron` (আর rsyslog থাকলে `/var/log/syslog`)
Fedora / RHEL 9	`journalctl -u crond` অথবা `/var/log/cron`

```
$ journalctl -u cron --since "1 hour ago"
Nov 12 14:00:01 myhost CRON[5012]: pam_unix(cron:session): session opened for user
nishat(uid=1000) by nishat(uid=0)
Nov 12 14:00:01 myhost CRON[5013]: (nishat) CMD (/home/nishat/bin/backup.sh)
Nov 12 14:00:02 myhost CRON[5012]: pam_unix(cron:session): session closed for user
nishat
```

```
$ grep CRON /var/log/syslog | tail -5          # Ubuntu, rsyslog থাকলে
$ sudo tail -f /var/log/cron                   # RHEL/Fedora
```

> 📝 মনে রাখো: Debian 12-এ ডিফল্টে rsyslog ইনস্টল থাকে না, তাই `/var/log/syslog` নাও থাকতে পারে — journalctl ব্যবহার করো (অধ্যায় ২৮)।

cron ডিবাগ করার চেকলিস্ট

```
কাজ চলছে না?
   │
   ├─ ১. cron daemon চলছে?            → systemctl status cron / crond
   ├─ ২. crontab ইনস্টল হয়েছে?       → crontab -l
   ├─ ৩. cron চালানোর চেষ্টা করেছে?   → journalctl -u cron  (CMD লাইন আছে?)
   ├─ ৪. সময়ের ফিল্ড ঠিক?            → crontab.guru-তে মিলিয়ে দেখো, টাইমজোন দেখো
   ├─ ৫. PATH/পুরো পাথ?               → কমান্ডের absolute path দাও
   ├─ ৬. % এস্কেপ করা?                → \%
   ├─ ৭. স্ক্রিপ্ট executable?        → chmod +x, প্রথম লাইনে #!/bin/bash
   ├─ ৮. ফাইলের নামে ডট? (cron.d/daily) → নাম বদলাও
   ├─ ৯. আউটপুট/এরর কী বলছে?         → >> /tmp/debug.log 2>&1 যোগ করো
   └─ ১০. cron-এর মতো environment-এ টেস্ট করো:
         env -i HOME=$HOME PATH=/usr/bin:/bin /bin/sh -c '/path/to/script'
```

শেষ কমান্ডটা দারুণ কাজের: `env -i` সব environment মুছে ফেলে, তারপর শুধু cron-এর মতো ন্যূনতম ভ্যারিয়েবল দিয়ে স্ক্রিপ্ট চালায়। এখানে চললে cron-এও চলার সম্ভাবনা বেশি।

> 💡 টিপস: ৫ ফিল্ড মেলানো নিয়ে সন্দেহ হলে https://crontab.guru ওয়েবসাইটে লাইনটা বসিয়ে দেখো — সাধারণ ভাষায় বলে দেয় কখন চলবে।

২৯.৮ anacron — যে কম্পিউটার রাতে বন্ধ থাকে

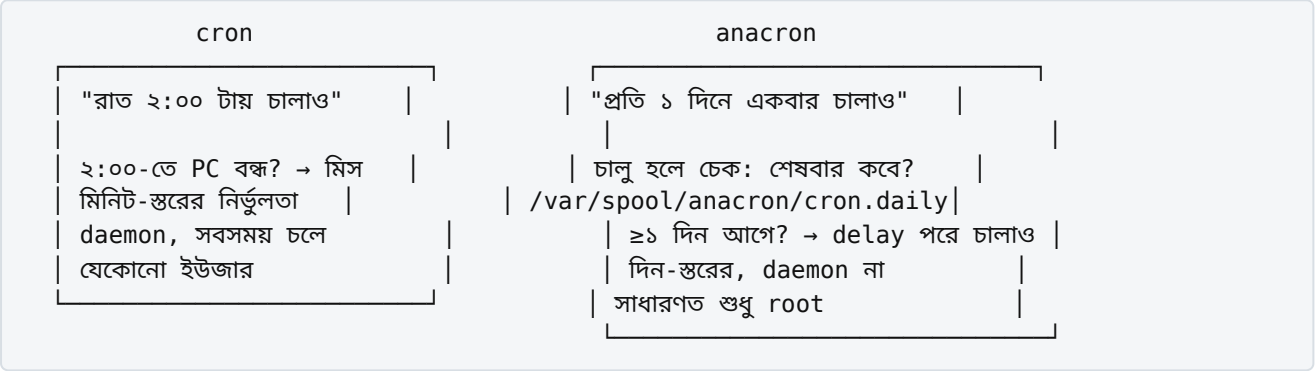
সমস্যা

cron ধরে নেয় কম্পিউটার ২৪ ঘণ্টা চালু। কিন্তু তোমার ল্যাপটপ রাতে বন্ধ থাকে। `/etc/cron.daily` যদি ভোর ৬:২৫-এ চলার কথা থাকে আর তুমি ল্যাপটপ চালু করো সকাল ৯টায় — cron সেই কাজ **কখনো চালাবে না**। মিস হওয়া মিনিট সে ফিরে দেখে না।

সমাধান: anacron

anacron (নাম এসেছে "anachronistic" থেকে — সময়ের সাথে না মেলা) সময় না, **পর্যায়কাল (period)** দিয়ে ভাবে: "শেষবার কবে চলেছিল? এক দিনের বেশি হয়ে গেলে এখন চালাও।"

উপমা: cron হলো ট্রেনের সময়সূচি — ৯:০০-এর ট্রেন ৯:০০-এই ছাড়বে, তুমি না থাকলে মিস। anacron হলো ওষুধ খাওয়ার নিয়ম — "দিনে একবার"; সকালে ভুলে গেলে দুপুরে মনে পড়লেই খেয়ে নাও।



anacron daemon না — এটা চালু হয়, কাজ চেক করে/চালায়, তারপর বেরিয়ে যায়। একে চালায় cron (RHEL-এ `/etc/cron.hourly/0anacron`), অথবা systemd (`anacron.timer`, Debian/Ubuntu-তে), অথবা বুটের সময়।

`/etc/anacrontab` — ফরম্যাট

```
period   delay   job-identifier   command
(দিন)    (মিনিট)  (নাম)            (কী চালাবে)
```

ফিল্ড	অর্থ
period	কত দিন পরপর; `@monthly` ও লেখা যায়

ফিল্ড	অর্থ
delay	চালানোর আগে কত মিনিট অপেক্ষা (বুটের সময় সব একসাথে চাপ না দিতে)
job-identifier	নাম; `/var/spool/anacron/<নাম>` ফাইলে শেষ চলার তারিখ রাখে
command	কমান্ড

Ubuntu/Debian:

```
$ cat /etc/anacrontab
SHELL=/bin/sh
HOME=/root
LOGNAME=root

# These replace cron's entries
1       5       cron.daily      run-parts --report /etc/cron.daily
7       10      cron.weekly     run-parts --report /etc/cron.weekly
@monthly        15      cron.monthly    run-parts --report /etc/cron.monthly
```

RHEL 9 / Fedora:

```
$ cat /etc/anacrontab
SHELL=/bin/sh
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
# the maximal random delay added to the base delay of the jobs
RANDOM_DELAY=45
# the jobs will be started during the following hours only
START_HOURS_RANGE=3-22

#period in days   delay in minutes   job-identifier   command
1       5       cron.daily              nice run-parts /etc/cron.daily
7       25      cron.weekly             nice run-parts /etc/cron.weekly
@monthly 45     cron.monthly            nice run-parts /etc/cron.monthly
```

এখানে `START_HOURS_RANGE=3-22` মানে শুধু ভোর ৩টা থেকে রাত ১০টার মধ্যে কাজ শুরু হবে, আর `RANDOM_DELAY=45` মানে delay-এর সাথে ০–৪৫ মিনিটের এলোমেলো দেরি যোগ হবে।

টাইমস্ট্যাম্প ফাইল:

```
$ sudo cat /var/spool/anacron/cron.daily
20261112
```

anacron কমান্ড

কমান্ড	কাজ
`anacron -T`	anacrontab-এর সিনট্যাক্স পরীক্ষা
`anacron -n`	delay উপেক্ষা করে এখনই (দরকারি কাজ) চালাও

কমান্ড	কাজ
`anacron -f`	সময় না হলেও জোর করে সব চালাও (force)
`anacron -u`	কিছু না চালিয়ে শুধু টাইমস্ট্যাম্প আজকের তারিখে আপডেট করো
`anacron -d`	foreground-এ চালাও, মেসেজ টার্মিনালে দেখাও (debug)
`anacron -s`	কাজগুলো একটার পর একটা (serialize) চালাও

```
$ sudo anacron -T && echo "সিনট্যাক্স ঠিক আছে"
সিনট্যাক্স ঠিক আছে
$ sudo anacron -fnd          # সব কাজ এখনই, foreground-এ
```

> 📝 মনে রাখো: anacron ইনস্টল না থাকলে (`test -x /usr/sbin/anacron` ব্যর্থ), Ubuntu/Debian-এর `/etc/crontab` নিজেই cron.daily ইত্যাদি চালায়। Ubuntu ডেস্কটপে anacron সাধারণত ইনস্টল থাকে; মিনিমাল সার্ভারে নাও থাকতে পারে: `sudo apt install anacron`। RHEL/Fedora-তে এটা `cronie-anacron` প্যাকেজে আসে।

২৯.৯ at, atq, atrm ও batch — একবারের কাজ

cron বারবারের জন্য। কিন্তু যদি বলো "আজ রাত ১১টায় একবার সার্ভার রিস্টার্ট করো" — তখন crontab-এ লাইন দিয়ে পরে মুছে ফেলা ঝামেলা। এর জন্য **at**।

ইনস্টল ও চালু করা

`at` সব জায়গায় ডিফল্টে থাকে না:

```
$ sudo apt install at                          # Ubuntu/Debian (atd নিজে চালু হয়)
$ sudo dnf install at && sudo systemctl enable --now atd    # Fedora/RHEL
$ systemctl status atd
```

ভেতরে: `at` কমান্ড তোমার কাজটা (সাথে তোমার বর্তমান environment ও current directory!) একটা স্ক্রিপ্ট হিসেবে `/var/spool/cron/atjobs/` (Debian) বা `/var/spool/at/` (RHEL)-এ রেখে দেয়। `atd` daemon সময় হলে সেটা চালায়।

> 📝 মনে রাখো: cron-এর বিপরীতে, `at` তোমার বর্তমান environment ভ্যারিয়েবল (PATH সহ) আর current directory **সংরক্ষণ করে**। তাই at-এ PATH সমস্যা অনেক কম। তবে টার্মিনাল থাকে না — আউটপুট মেইলে যায় (MTA থাকলে)।

at-এর ব্যবহার

```
$ at 23:00
warning: commands will be executed using /bin/sh
at> /usr/local/bin/backup
at> echo "done" >> /tmp/at-test.log
at> <EOT>                          ← এখানে Ctrl+D চাপো
job 3 at Thu Nov 12 23:00:00 2026
```

কমান্ড পাইপ বা ফাইল দিয়েও দেওয়া যায়:

```
$ echo "/usr/local/bin/backup" | at now + 30 minutes
job 4 at Thu Nov 12 15:42:00 2026

$ at -f /home/nishat/bin/task.sh 02:00 tomorrow
job 5 at Fri Nov 13 02:00:00 2026
```

সময় লেখার ধরন

`at` খুব সহজ ইংরেজি সময় বোঝে:

লেখা	অর্থ
`now + 5 minutes`	৫ মিনিট পর
`now + 2 hours`	২ ঘণ্টা পর
`now + 3 days`	৩ দিন পর (এখনকার সময়ে)
`now + 1 week`	১ সপ্তাহ পর
`14:30`	আজ ২:৩০-এ (পেরিয়ে গেলে কাল)
`2:30 PM`	একই, ১২ ঘণ্টার ফরম্যাটে
`noon` / `midnight`	দুপুর ১২টা / রাত ১২টা
`teatime`	বিকেল ৪টা
`tomorrow`	কাল এই সময়ে
`10:00 tomorrow`	কাল সকাল ১০টায়
`9am Monday`	পরের সোমবার সকাল ৯টায়
`10:00 2026-12-31`	নির্দিষ্ট তারিখে (ISO)
`10:00 12/31/2026`	নির্দিষ্ট তারিখে (MM/DD/YYYY)
`-t 202612311000`	`touch`-এর মতো `[[CC]YY]MMDDhhmm` ফরম্যাট

পুরো তালিকা: `/usr/share/doc/at/timespec` (Debian/Ubuntu)।

at-এর অপশন

অপশন	কাজ
`-f file`	stdin-এর বদলে ফাইল থেকে কমান্ড
`-l`	কিউর কাজ দেখাও (`atq` -এর সমান)
`-d job`	কাজ মুছো (`atrm` -এর সমান)
`-c job`	কাজের পুরো স্ক্রিপ্ট দেখাও (environment সহ)
`-m`	আউটপুট না থাকলেও শেষ হলে মেইল করো
`-M`	কখনো মেইল করবে না
`-q queue`	কিউ বাছো (a–z, A–Z; বড় অক্ষর = বেশি nice)
`-t time`	`[[CC]YY]MMDDhhmm[.ss]` ফরম্যাটে সময়

atq — কিউ দেখা

```
$ atq
3       Thu Nov 12 23:00:00 2026 a nishat
4       Thu Nov 12 15:42:00 2026 a nishat
5       Fri Nov 13 02:00:00 2026 a nishat
```

কলাম: job নম্বর, সময়, কিউ (`a`), ইউজার। root সবার কাজ দেখে।

কাজের ভেতরে কী আছে:

```
$ at -c 4 | tail -3
cd /home/nishat || {
     echo 'Execution directory inaccessible' >&2
     exit 1
}
/usr/local/bin/backup
```

atrm — কাজ বাতিল

```
$ atrm 4
$ atq
3       Thu Nov 12 23:00:00 2026 a nishat
5       Fri Nov 13 02:00:00 2026 a nishat
```

batch — সিস্টেম ফ্রি হলে চালাও

`batch` কোনো সময় নেয় না। কাজটা কিউতে রাখে, আর যখন সিস্টেমের **load average** একটা সীমার নিচে নামে (atd-এর ডিফল্ট সাধারণত 1.5; `atd -l` দিয়ে বদলানো যায়), তখন চালায়। load average কী — অধ্যায় ৪৬ দেখো।

```
$ batch
warning: commands will be executed using /bin/sh
at> tar czf /backup/big-archive.tar.gz /data
at> <EOT>
job 6 at Thu Nov 12 15:20:00 2026
$ atq
6       Thu Nov 12 15:20:00 2026 b nishat
```

লক্ষ্য করো কিউ `b` — batch-এর কিউ। ভারী কাজ (ভিডিও কনভার্ট, বড় কম্প্রেশন) যা জরুরি না, কিন্তু সিস্টেম ব্যস্ত থাকলে চালালে অন্যদের সমস্যা হবে — তার জন্য batch।

একটা বাস্তব উদাহরণ: নিরাপদ ফায়ারওয়াল পরীক্ষা

রিমোট সার্ভারে ফায়ারওয়াল নিয়ম বদলাচ্ছ (অধ্যায় ৩৯)? ভুল হলে নিজেকেই লক আউট করে ফেলবে। আগে একটা "সেফটি নেট" বসাও:

```
$ echo "ufw disable" | sudo at now + 10 minutes
job 7 at Thu Nov 12 15:30:00 2026
$ sudo ufw enable       # নতুন নিয়ম পরীক্ষা করো
# সব ঠিক থাকলে, সেফটি নেট বাতিল:
$ sudo atrm 7
```

ভুল হলে আর ঢুকতে না পারলেও ১০ মিনিট পর ফায়ারওয়াল নিজেই বন্ধ হবে।

at-এর কীবোর্ড শর্টকাট

শর্টকাট	কাজ
`Enter`	`at>` প্রম্পটে পরের কমান্ড লাইন
`Ctrl+D`	ইনপুট শেষ, কাজ কিউতে জমা (`<EOT>` দেখায়)
`Ctrl+C`	কিছু জমা না দিয়ে বাতিল

২৯.১০ কে cron/at ব্যবহার করতে পারবে — allow/deny ফাইল

ফাইল	নিয়ম
`/etc/cron.allow`	থাকলে **শুধু** এখানে লেখা ইউজাররা crontab ব্যবহার করতে পারবে
`/etc/cron.deny`	(allow না থাকলে) এখানে লেখা ইউজাররা পারবে না
`/etc/at.allow`	at-এর জন্য একই নিয়ম
`/etc/at.deny`	at-এর জন্য একই নিয়ম

নিয়মের ক্রম:

```
cron.allow আছে?
   ├── হ্যাঁ → শুধু তালিকাভুক্ত ইউজার পারবে (deny দেখা হয় না)
   └── না → cron.deny আছে?
             ├── হ্যাঁ → তালিকাভুক্ত ছাড়া সবাই পারবে
             └── না → distro-ভেদে: Debian-এ সবাই পারে, কিছু সিস্টেমে শুধু root
```

ফাইলে প্রতি লাইনে একটা ইউজারনেম। root সবসময় পারে। ব্লক করা ইউজার চেষ্টা করলে:

```
$ crontab -e
You (guest) are not allowed to use this program (crontab)
See crontab(1) for more information
```

> 📝 মনে রাখো: এই ফাইলগুলো শুধু crontab/at **কমান্ড** ব্যবহার আটকায়। `/etc/cron.d`-এ root যা রাখে সেটা চলবেই।

একই কাজ দুবার একসাথে না চলুক — `flock`

ধরো একটা কাজ প্রতি ৫ মিনিটে চলে, কিন্তু কোনো দিন সেটা ৭ মিনিট লাগল। তখন আগেরটা শেষ না হতেই নতুনটা শুরু হবে — দুটো একই ফাইলে লিখবে, ডেটা নষ্ট হবে। cron এটা আটকায় না। সমাধান `flock` (util-linux প্যাকেজে, প্রায় সব জায়গায় থাকে):

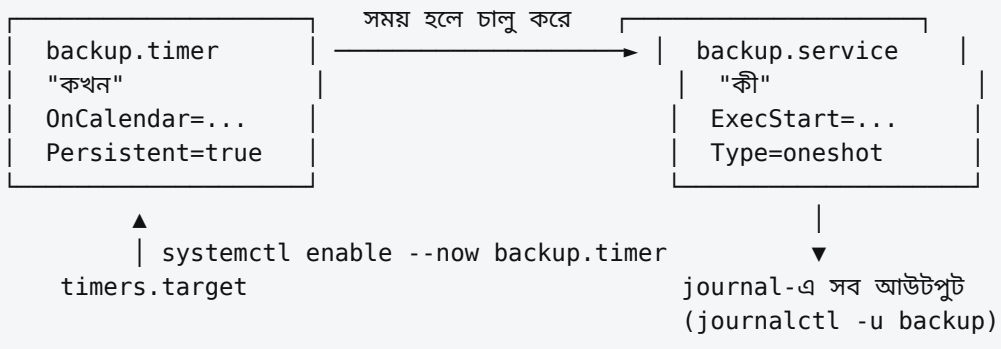
```
*/5 * * * * /usr/bin/flock -n /tmp/sync.lock /usr/local/bin/sync-data
```

`-n` (non-block) মানে: লক আগে থেকে নেওয়া থাকলে অপেক্ষা না করে সাথে সাথে বেরিয়ে যাও। (systemd timer-এ এই সমস্যা নেই — সার্ভিস আগে থেকে চললে timer নতুন করে শুরু করে না।)

২৯.১১ systemd timers — আধুনিক বিকল্প

ধারণা

systemd (অধ্যায় ২৬) শুধু সার্ভিস চালায় না, শিডিউলও করতে পারে। systemd timer দুটো unit ফাইল দিয়ে তৈরি:



উপমা: `.service` হলো রান্নার রেসিপি (কী রান্না হবে), `.timer` হলো রান্নাঘরের টাইমার (কখন চুলা জ্বলবে)। একই নামের হলে (`backup.timer` → `backup.service`) timer নিজে বুঝে নেয় কোন সার্ভিস চালাবে; আলাদা নাম হলে `Unit=` দিয়ে বলে দিতে হয়।

আসলে তোমার সিস্টেমে আগে থেকেই অনেক timer চলছে:

```
$ systemctl list-timers
NEXT                         LEFT LAST                              PASSED UNIT
ACTIVATES
Thu 2026-11-12 16:00:00 +06    24min Thu 2026-11-12 15:00:04 +06      35min ago
anacron.timer                anacron.service
Fri 2026-11-13 00:00:00 +06       8h Thu 2026-11-12 00:00:01 +06          15h ago
logrotate.timer              logrotate.service
Fri 2026-11-13 00:00:00 +06       8h Thu 2026-11-12 00:00:01 +06          15h ago dpkg-
db-backup.timer         dpkg-db-backup.service
Fri 2026-11-13 06:24:51 +06      14h Thu 2026-11-12 06:41:12 +06           8h ago apt-
daily-upgrade.timer      apt-daily-upgrade.service
Sun 2026-11-15 03:10:52 +06   2 days Sun 2026-11-08 03:10:07 +06       4 days ago
e2scrub_all.timer            e2scrub_all.service
Mon 2026-11-16 00:47:21 +06   3 days Mon 2026-11-09 00:15:33 +06       3 days ago
fstrim.timer                 fstrim.service

6 timers listed.
Pass --all to see loaded but inactive timers, too.
```

(তোমার আউটপুট ভিন্ন হবে।) `systemctl list-timers --all` নিষ্ক্রিয়গুলোও দেখায়।

সম্পূর্ণ উদাহরণ: প্রতিদিন রাত ২টার ব্যাকআপ

ধাপ ১: স্ক্রিপ্ট

```
$ sudo nano /usr/local/bin/mybackup
```

```
#!/bin/bash
set -euo pipefail
tar czf "/backup/etc-$(date +%F).tar.gz" /etc
echo "Backup finished at $(date)"
```

```
$ sudo chmod +x /usr/local/bin/mybackup
$ sudo mkdir -p /backup
```

(লক্ষ্য করো — এখানে `%` এস্কেপ করতে হয়নি, কারণ এটা crontab না!)

ধাপ ২: service unit

```
$ sudo nano /etc/systemd/system/mybackup.service
```

```
[Unit]
Description=Daily backup of /etc
```

```
[Service]
Type=oneshot
ExecStart=/usr/local/bin/mybackup
```

`Type=oneshot` মানে একবার চলে শেষ হয়ে যায় (দীর্ঘক্ষণ চলা সার্ভিস না)। এখানে `[Install]` সেকশন লাগে না, কারণ সার্ভিসটা timer চালু করবে, বুটে নয়।

ধাপ ৩: timer unit

```
$ sudo nano /etc/systemd/system/mybackup.timer
```

```
[Unit]
Description=Run mybackup daily at 02:00

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=10min

[Install]
WantedBy=timers.target
```

ধাপ ৪: চালু করো

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now mybackup.timer
Created symlink /etc/systemd/system/timers.target.wants/mybackup.timer →
/etc/systemd/system/mybackup.timer.
```

> ⚠️ সতর্কতা: enable করবে **`.timer`**, `.service` না! service enable করলে (যদি `[Install]` থাকত) সেটা বুটে চলত, শিডিউলে না।

ধাপ ৫: যাচাই

```
$ systemctl list-timers mybackup.timer
NEXT                         LEFT LAST PASSED UNIT           ACTIVATES
Fri 2026-11-13 02:04:37 +06   10h -    -      mybackup.timer mybackup.service

1 timers listed.
```

এখনই একবার পরীক্ষা করতে (timer-এর অপেক্ষা না করে):

```
$ sudo systemctl start mybackup.service
$ journalctl -u mybackup.service -n 5
Nov 12 15:40:11 myhost systemd[1]: Starting mybackup.service - Daily backup of /etc...
Nov 12 15:40:12 myhost mybackup[6120]: Backup finished at Thu Nov 12 15:40:12 +06 2026
```

```
Nov 12 15:40:12 myhost systemd[1]: mybackup.service: Deactivated successfully.
Nov 12 15:40:12 myhost systemd[1]: Finished mybackup.service - Daily backup of /etc.
```

দেখো — কোনো মেইল সার্ভার ছাড়াই আউটপুট সুন্দরভাবে journal-এ! এটাই timer-এর বড় সুবিধা।

`OnCalendar` সিনট্যাক্স

সাধারণ ফরম্যাট:

```
   DayOfWeek  Year-Month-Day  Hour:Minute:Second  [Timezone]
   Mon..Fri   *-*-*           09:00:00            Asia/Dhaka
   ─────────  ──────────────  ──────────────────
   (ঐচ্ছিক)    * = যেকোনো      সেকেন্ড বাদ দিলে :00
```

OnCalendar	অর্থ	cron সমতুল্য
`minutely`	প্রতি মিনিটে	`* * * * *`
`hourly`	প্রতি ঘণ্টার শুরুতে	`0 * * * *`
`daily`	প্রতিদিন মধ্যরাতে	`0 0 * * *`
`weekly`	সোমবার মধ্যরাতে (রবিবার না!)	`0 0 * * 1`
`monthly`	মাসের ১ তারিখ মধ্যরাতে	`0 0 1 * *`
`yearly` / `annually`	১ জানুয়ারি মধ্যরাতে	`0 0 1 1 *`
`quarterly`	জানু/এপ্রিল/জুলাই/অক্টো ১ তারিখে	`0 0 1 1,4,7,10 *`
`*-*-* 02:00:00`	প্রতিদিন রাত ২টা	`0 2 * * *`
`*:0/15`	প্রতি ১৫ মিনিটে	`*/15 * * * *`
`*:*:0/30`	প্রতি ৩০ সেকেন্ডে	(cron পারে না)
`Mon..Fri 09:00`	কর্মদিবসে সকাল ৯টায়	`0 9 * * 1-5`
`Sat,Sun 10:00`	সাপ্তাহিক ছুটিতে ১০টায়	`0 10 * * 0,6`
`*-*-01 03:00`	প্রতি মাসের ১ তারিখ রাত ৩টায়	`0 3 1 * *`
`*-*~01`	প্রতি মাসের **শেষ** দিনে (`~` = শেষ থেকে গোনা)	(cron পারে না)
`Fri *-*~07/1 18:00`	মাসের শেষ শুক্রবার সন্ধ্যা ৬টায়	(cron পারে না)
`*-*-* 02:00 Asia/Dhaka`	ঢাকার সময়ে রাত ২টা	—

ধাপ (`/`) আর পরিসর (`..`) ব্যবহার হয়; তালিকা কমা দিয়ে। একাধিক `OnCalendar=` লাইন দেওয়া যায় — যেকোনোটা মিললেই চলবে।

`systemd-analyze calendar` — লেখার আগে যাচাই

এটা অসাধারণ একটা টুল — বলে দেয় তোমার লেখা এক্সপ্রেশন কখন চলবে:

```
$ systemd-analyze calendar "Mon..Fri 09:00"
  Original form: Mon..Fri 09:00
Normalized form: Mon..Fri *-*-* 09:00:00
    Next elapse: Fri 2026-11-13 09:00:00 +06
       (in UTC): Fri 2026-11-13 03:00:00 UTC
       From now: 17h left
```

পরের কয়েকবার দেখতে:

```
$ systemd-analyze calendar --iterations=3 "*-*~01"
  Original form: *-*~01
Normalized form: *-*~01 00:00:00
    Next elapse: Mon 2026-11-30 00:00:00 +06
       (in UTC): Sun 2026-11-29 18:00:00 UTC
       From now: 2 weeks 3 days left
       Iter. #2: Thu 2026-12-31 00:00:00 +06
       (in UTC): Wed 2026-12-30 18:00:00 UTC
       From now: 1 month 18 days left
       Iter. #3: Sun 2027-01-31 00:00:00 +06
       (in UTC): Sat 2027-01-30 18:00:00 UTC
       From now: 2 months 18 days left
```

`[Timer]` সেকশনের গুরুত্বপূর্ণ অপশন

অপশন	কাজ	উদাহরণ
`OnCalendar=`	ক্যালেন্ডার/ঘড়ির সময় (wall-clock timer)	`OnCalendar=daily`
`Persistent=true`	মেশিন বন্ধ থাকায় মিস হলে, চালু হওয়ার পর একবার চালাও (anacron-এর মতো) — শুধু `OnCalendar`-এর সাথে কাজ করে	`Persistent=true`
`OnBootSec=`	বুটের এতক্ষণ পরে	`OnBootSec=5min`
`OnStartupSec=`	systemd (বা user manager) চালুর এতক্ষণ পরে	`OnStartupSec=10min`
`OnActiveSec=`	timer নিজে চালু হওয়ার এতক্ষণ পরে	`OnActiveSec=1h`
`OnUnitActiveSec=`	সার্ভিস শেষবার **চালু** হওয়ার এতক্ষণ পরে (পুনরাবৃত্তি)	`OnUnitActiveSec=15min`
`OnUnitInactiveSec=`	সার্ভিস শেষবার **শেষ** হওয়ার এতক্ষণ পরে	`OnUnitInactiveSec=30min`
`RandomizedDelaySec=`	০ থেকে এই সময়ের মধ্যে এলোমেলো দেরি (অনেক সার্ভার একসাথে চাপ না দিতে)	`RandomizedDelaySec=15min`
`AccuracySec=`	কতটা নির্ভুল হবে; ডিফল্ট 1min (systemd বিদ্যুৎ বাঁচাতে কাছাকাছি timer একসাথে চালায়)	`AccuracySec=1s`
`Unit=`	কোন unit চালাবে (ডিফল্ট: একই নামের .service)	`Unit=other.service`

অপশন	কাজ	উদাহরণ
`WakeSystem=true`	suspend থেকে মেশিন জাগিয়ে চালাও (হার্ডওয়্যার সমর্থন লাগে)	`WakeSystem=true`

`OnBootSec` আর `OnUnitActiveSec` মিলিয়ে "বুটের ৫ মিনিট পর একবার, তারপর প্রতি ১ ঘণ্টায়" — এমন **monotonic timer** (ঘড়ির সময় না, আপেক্ষিক সময়) বানানো যায়:

```
[Timer]
OnBootSec=5min
OnUnitActiveSec=1h
```

সময়ের একক: `us`, `ms`, `s` / `sec`, `m` / `min`, `h` / `hr`, `d` / `day`, `w` / `week`, `month`, `y` / `year` — যেমন `1h 30min`।

`Persistent=true` — ভেতরে কীভাবে কাজ করে

systemd প্রতিবার timer চলার সময় একটা স্ট্যাম্প ফাইল আপডেট করে: `/var/lib/systemd/timers/stamp-<নাম>.timer`। বুটের পর timer চালু হলে সে দেখে — "শেষবার চলা আর এখনকার মধ্যে কি কোনো নির্ধারিত সময় পার হয়ে গেছে?" হ্যাঁ হলে সাথে সাথে একবার চালায়। এক সপ্তাহ বন্ধ থাকলেও একবারই চালায়, সাতবার না।

```
$ ls /var/lib/systemd/timers/
stamp-apt-daily.timer  stamp-fstrim.timer  stamp-logrotate.timer  stamp-mybackup.timer
```

systemd-run — এক লাইনে অস্থায়ী timer (at-এর বিকল্প)

unit ফাইল না লিখেই:

```
$ sudo systemd-run --on-active=30min /usr/local/bin/mybackup
Running timer as unit: run-r8c1b2e3f.timer
Will run service as unit: run-r8c1b2e3f.service

$ sudo systemd-run --on-calendar="*-*-* 23:00" --unit=nightly-test /usr/bin/touch
/tmp/hello
Running timer as unit: nightly-test.timer
Will run service as unit: nightly-test.service
```

এগুলো অস্থায়ী (transient) — রিবুটে হারিয়ে যায়। বাতিল করতে: `sudo systemctl stop run-r8c1b2e3f.timer`।

ইউজার timer — root ছাড়া

নিজের ইউজারের জন্য timer রাখো `~/.config/systemd/user/`-এ, আর সব কমান্ডে `--user`:

```
$ mkdir -p ~/.config/systemd/user
$ nano ~/.config/systemd/user/remind.service
```

```
[Unit]
Description=Drink water reminder

[Service]
Type=oneshot
ExecStart=/usr/bin/notify-send "পানি খাও!"
```

```
$ nano ~/.config/systemd/user/remind.timer
```

```
[Unit]
Description=Every hour reminder

[Timer]
OnCalendar=hourly

[Install]
WantedBy=timers.target
```

```
$ systemctl --user daemon-reload
$ systemctl --user enable --now remind.timer
$ systemctl --user list-timers
```

ডেস্কটপ সেশনের ভেতরে চলে বলে `notify-send` এখানে কাজ করে (cron-এ যা ঝামেলার)।

> 📝 মনে রাখো: ইউজার timer শুধু তুমি লগইন থাকা অবস্থায় চলে। লগআউট থাকলেও (যেমন সার্ভারে) চালাতে চাইলে: `sudo loginctl enable-linger nishat`।

timer ম্যানেজমেন্ট কমান্ড

কমান্ড	কাজ
`systemctl list-timers`	সক্রিয় timer, পরের/শেষ চলার সময়
`systemctl list-timers --all`	নিষ্ক্রিয়গুলোও
`systemctl status mybackup.timer`	timer-এর অবস্থা, Trigger সময়
`systemctl enable --now X.timer`	চালু ও বুটে সক্রিয়
`systemctl disable --now X.timer`	বন্ধ ও নিষ্ক্রিয়
`systemctl start X.service`	এখনই একবার কাজটা চালাও
`systemctl cat X.timer`	unit ফাইল দেখাও
`systemctl edit X.timer`	override (drop-in) বানিয়ে বদলাও
`journalctl -u X.service`	কাজের আউটপুট/লগ

কমান্ড	কাজ
`systemd-analyze calendar "..."`	OnCalendar যাচাই
`systemd-analyze verify X.timer`	unit ফাইলে ভুল আছে কিনা দেখো
`systemctl daemon-reload`	unit ফাইল বদলানোর পর অবশ্যই

২৯.১২ cron বনাম systemd timer — তুলনা

বিষয়	cron	systemd timer
সেটআপ	এক লাইন — খুব সহজ	দুটো ফাইল — একটু বেশি লেখা
সর্বনিম্ন একক	১ মিনিট	মাইক্রোসেকেন্ড পর্যন্ত (ডিফল্ট নির্ভুলতা ১ মিনিট)
লগিং	মেইল (MTA লাগে) বা নিজে রিডাইরেক্ট	স্বয়ংক্রিয় journal-এ
মিস হওয়া কাজ	না (anacron লাগে)	`Persistent=true`
বুটের পরে আপেক্ষিক সময়	শুধু `@reboot`	`OnBootSec`, `OnUnitActiveSec`
একই কাজ দুবার একসাথে	আটকায় না (`flock` লাগে)	আটকায় (চলমান থাকলে আবার শুরু হয় না)
নির্ভরতা (যেমন নেটওয়ার্ক)	নেই	`After=`, `Requires=`
রিসোর্স লিমিট	নেই (`nice` দিয়ে কিছুটা)	`CPUQuota=`, `MemoryMax=`, `Nice=`, `IOSchedulingClass=`
নিরাপত্তা স্যান্ডবক্স	নেই	`ProtectSystem=`, `PrivateTmp=` ইত্যাদি
এখনই হাতে চালানো	কমান্ড কপি করে চালাও (environment ভিন্ন!)	`systemctl start X.service` (হুবহু একই environment)
পরের রান কখন	নিজে হিসাব করো	`systemctl list-timers`
এলোমেলো দেরি	নেই (cronie-র anacron-এ `RANDOM_DELAY`)	`RandomizedDelaySec=`
টাইমজোন প্রতি কাজে	cronie-তে `CRON_TZ`	`OnCalendar=... Asia/Dhaka`
পোর্টেবিলিটি	প্রায় সব Unix (BSD, macOS, পুরোনো লিনাক্স)	শুধু systemd-ভিত্তিক লিনাক্স
নতুনদের জন্য	সহজ	একটু শেখা লাগে

সিদ্ধান্ত নেওয়ার সহজ নিয়ম

```
একবারের কাজ?
   ├── হ্যাঁ → at   (বা systemd-run --on-active)
   └── না (বারবার)
```

```
          │
          ├── দ্রুত, সাধারণ, ব্যক্তিগত কাজ / পুরোনো বা non-systemd সিস্টেম?
          │       → cron
          │
          └── প্রোডাকশন সার্ভার, ভালো লগ চাই, মিস হলে চালাতে হবে,
              নির্ভরতা/রিসোর্স লিমিট দরকার, সেকেন্ড-স্তরের সময়?
                 → systemd timer
```

> 💡 টিপস: দুটোই শেখো। ইন্টারভিউ আর পুরোনো সার্ভারে cron সর্বত্র; আধুনিক distro-র নিজের কাজগুলো (logrotate, apt-daily, fstrim) বেশিরভাগ এখন timer দিয়ে চলে।

এক নজরে: একই কাজ তিনভাবে

"প্রতিদিন রাত ২টায় `/usr/local/bin/mybackup` চালাও":

```
# ১. ইউজার crontab (crontab -e)
0 2 * * * /usr/local/bin/mybackup >> /var/log/mybackup.log 2>&1

# ২. /etc/cron.d/mybackup  (ইউজার ফিল্ড সহ)
0 2 * * * root /usr/local/bin/mybackup >> /var/log/mybackup.log 2>&1

# ৩. systemd timer  (mybackup.timer)
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
```

আর "শুধু আজ রাত ২টায় একবার":

```
$ echo /usr/local/bin/mybackup | at 02:00
```

২৯.১৩ কাজের কীবোর্ড শর্টকাট

শিডিউলিং-এর নিজস্ব শর্টকাট কম, কিন্তু এডিটর আর প্রম্পটের শর্টকাট লাগবেই:

পরিস্থিতি	শর্টকাট	কাজ
`crontab -e` (nano)	`Ctrl+O`, তারপর `Enter`	সেভ
`crontab -e` (nano)	`Ctrl+X`	বের হও (সেভ করতে বলবে)
`crontab -e` (nano)	`Ctrl+K`	পুরো লাইন কাটো
`crontab -e` (nano)	`Alt+3`	লাইন কমেন্ট/আনকমেন্ট (নতুন nano-তে)
`crontab -e` (vim)	`i` → লেখো → `Esc` → `:wq`	লেখো ও সেভ করে বের হও
`crontab -e` (vim)	`:q!`	সেভ না করে বের হও (কিছুই বদলাবে না)

পরিস্থিতি	শর্টকাট	কাজ
`crontab -e` (vim)	`yy` `p`	লাইন কপি ও পেস্ট
`at>` প্রম্পট	`Ctrl+D`	কাজ জমা দাও
`at>` প্রম্পট	`Ctrl+C`	বাতিল
`journalctl -u ...`	`Shift+G` / `g`	শেষে / শুরুতে যাও
`journalctl -f` / `tail -f`	`Ctrl+C`	লাইভ দেখা বন্ধ
`systemctl list-timers`	`q`	pager থেকে বের হও

বিস্তারিত এডিটর শর্টকাট: অধ্যায় ১৩ ও অধ্যায় ১৪।

হাতে-কলমে অনুশীলন

ল্যাব ১: প্রথম cron কাজ

১. প্রতি মিনিটে সময় লিখে রাখা একটা কাজ বানাও:

```
$ crontab -e
```

যোগ করো:

```
* * * * * /usr/bin/date >> /tmp/cron-test.log 2>&1
```

২. সেভ করে বের হও, তারপর যাচাই:

```
$ crontab -l
* * * * * /usr/bin/date >> /tmp/cron-test.log 2>&1
```

৩. ২–৩ মিনিট অপেক্ষা করে দেখো:

```
$ cat /tmp/cron-test.log
Thu Nov 12 15:51:01 +06 2026
Thu Nov 12 15:52:01 +06 2026
Thu Nov 12 15:53:01 +06 2026
```

৪. লগে cron কী বলছে দেখো: `journalctl -u cron -n 5` (RHEL-এ `crond`)।

৫. এবার `*/2` করে দাও — প্রতি ২ মিনিটে চলছে কিনা দেখো।

ল্যাব ২: `%` আর environment-এর ফাঁদ নিজের চোখে দেখো

১. crontab-এ যোগ করো (ইচ্ছা করে ভুল):

```
* * * * * date +%H:%M >> /tmp/pct.log
```

২. এক মিনিট পর `cat /tmp/pct.log` — আশা অনুযায়ী আউটপুট আসবে না (কমান্ডটা `%`-এ ভেঙে গেছে)। এবার ঠিক করো:

```
* * * * * date +\%H:\%M >> /tmp/pct.log
```

৩. environment দেখো:

```
* * * * * env > /tmp/cron-env.txt
```

তারপর `diff <(sort /tmp/cron-env.txt) <(env | sort)` দিয়ে তোমার টার্মিনালের সাথে তুলনা করো। PATH-এর পার্থক্য লক্ষ্য করো।

৪. সব পরীক্ষার লাইন মুছে দাও (`crontab -e` দিয়ে, `-r` দিয়ে না!)।

ল্যাব ৩: crontab ব্যাকআপ ও রিস্টোর

```
$ crontab -l > ~/my-crontab.bak
$ crontab -ri
crontab: really delete nishat's crontab? (y/n) y
$ crontab -l
no crontab for nishat
$ crontab ~/my-crontab.bak
$ crontab -l                    # আগের সব ফিরে এসেছে
```

ল্যাব ৪: `/etc/cron.daily`-তে স্ক্রিপ্ট (ডট-এর ফাঁদ)

```
$ printf '#!/bin/sh\necho "daily ran $(date)" >> /tmp/daily.log\n' | sudo tee
/etc/cron.daily/mytest.sh
$ sudo chmod +x /etc/cron.daily/mytest.sh
$ run-parts --test /etc/cron.daily | grep mytest     # Ubuntu/Debian: কিছুই দেখাবে না!
$ sudo mv /etc/cron.daily/mytest.sh /etc/cron.daily/mytest
$ run-parts --test /etc/cron.daily | grep mytest
/etc/cron.daily/mytest
$ sudo run-parts --regex '^mytest$' /etc/cron.daily  # শুধু এটাকে এখনই চালাও (Debian)
$ cat /tmp/daily.log
$ sudo rm /etc/cron.daily/mytest                     # পরিষ্কার করো
```

ল্যাব ৫: at দিয়ে একবারের কাজ

```
$ echo 'echo "at worked at $(date)" > /tmp/at.log' | at now + 2 minutes
$ atq
$ at -c <job-নম্বর> | tail -3
# আরেকটা কাজ বানিয়ে বাতিল করো
$ echo 'touch /tmp/should-not-exist' | at now + 1 hour
$ atq
```

```
$ atrm <job-নম্বর>
$ atq
```

২ মিনিট পর `cat /tmp/at.log` দেখো।

ল্যাব ৬: systemd timer বানাও

২৯.১১-এর `mybackup` উদাহরণটা হুবহু তৈরি করো, কিন্তু `OnCalendar=*:0/2` দাও (প্রতি ২ মিনিটে, শুধু পরীক্ষার জন্য)। তারপর:

```
$ systemd-analyze calendar "*:0/2"
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now mybackup.timer
$ systemctl list-timers mybackup.timer
$ journalctl -u mybackup.service -f        # Ctrl+C দিয়ে বের হও
$ ls /backup
```

শেষে পরিষ্কার করো:

```
$ sudo systemctl disable --now mybackup.timer
$ sudo rm /etc/systemd/system/mybackup.{timer,service}
$ sudo systemctl daemon-reload
```

ল্যাব ৭: `Persistent=true` পরীক্ষা (VM-এ করো)

১. একটা timer বানাও `OnCalendar` দিয়ে এমন সময়ে যা কয়েক মিনিট পরে, আর `Persistent=true`। ২. সেই সময়ের আগেই VM বন্ধ করো, সময় পার হলে চালু করো। ৩. `journalctl -u <নাম>.service -b` দেখো — বুটের পরপরই কাজটা চলেছে। ৪. এবার `Persistent=false` দিয়ে একই পরীক্ষা করো, পার্থক্য দেখো।

ল্যাব ৮: চ্যালেঞ্জ

নিচের প্রতিটার জন্য cron লাইন **এবং** OnCalendar লেখো, তারপর `systemd-analyze calendar` দিয়ে যাচাই করো:

১. প্রতি কর্মদিবসে (সোম–শুক্র) বিকেল ৫:৩০ ২. প্রতি ঘণ্টার ১৫ ও ৪৫ মিনিটে ৩. প্রতি মাসের ১০ তারিখ সকাল ৮টা ৪. প্রতি ১০ মিনিটে, শুধু রাত ১০টা থেকে রাত ১১:৫০ পর্যন্ত

(উত্তর: ১. `30 17 * * 1-5` / `Mon..Fri 17:30`; ২. `15,45 * * * *` / `*:15,45`; ৩. `0 8 10 * *` / `*-*-10 08:00`; ৪. `*/10 22-23 * * *` / `22,23:0/10`)

সারসংক্ষেপ

কমান্ড / জিনিস	কাজ
`crontab -e`	নিজের crontab এডিট
`crontab -l`	crontab দেখো
`crontab -r` / `-ri`	crontab মুছো (`-i` = নিশ্চিত হয়ে)
`crontab -u user -e`	অন্য ইউজারের crontab (root)
`crontab file`	ফাইল থেকে crontab বসাও (পুরোনোটা বদলে যায়)
`মি ঘ দি মা স কমান্ড`	৫ ফিল্ড: 0-59, 0-23, 1-31, 1-12, 0-7
`*` `,` `-` `/`	সব, তালিকা, পরিসর, ধাপ
`@reboot @hourly @daily @weekly @monthly @yearly`	বিশেষ স্ট্রিং
`\%`	crontab-এ `%` এস্কেপ
`>> log 2>&1`	আউটপুট ও এরর লগে
`/etc/crontab`, `/etc/cron.d/`	সিস্টেম crontab (ইউজার ফিল্ড সহ)
`/etc/cron.{hourly,daily,weekly,monthly}/`	executable স্ক্রিপ্ট রাখো, নামে ডট নয়
`run-parts --test DIR`	কোন স্ক্রিপ্ট চলবে দেখো
`/etc/cron.allow`, `/etc/cron.deny`	কে crontab পারবে
`journalctl -u cron` / `crond`	cron-এর লগ
`/etc/anacrontab`	period delay id command
`anacron -T`, `anacron -fn`	সিনট্যাক্স চেক, জোর করে এখনই
`at TIME`, `at -f file TIME`	একবারের কাজ
`atq` / `at -l`	at কিউ
`at -c N`	কাজের বিষয়বস্তু
`atrm N` / `at -d N`	কাজ বাতিল
`batch`	লোড কম হলে চালাও
`flock -n lockfile cmd`	একসাথে দুবার চলা আটকাও
`X.timer` + `X.service`	systemd timer জোড়া
`OnCalendar=`, `Persistent=true`	ক্যালেন্ডার সময়, মিস হলে পরে চালাও
`OnBootSec=`, `OnUnitActiveSec=`	আপেক্ষিক (monotonic) সময়
`RandomizedDelaySec=`, `AccuracySec=`	এলোমেলো দেরি, নির্ভুলতা
`systemctl list-timers [--all]`	সব timer
`systemctl enable --now X.timer`	timer চালু

কমান্ড / জিনিস	কাজ
`systemd-analyze calendar "..."`	OnCalendar যাচাই
`systemd-run --on-active=30min cmd`	অস্থায়ী timer
`systemctl --user ...`, `loginctl enable-linger`	ইউজার timer

নিজেকে যাচাই করো

১. `*/20 8-18 * * 1-5` লাইনটা কখন চলবে?

২. ইউজার crontab আর `/etc/cron.d/`-এর ফাইলের ফরম্যাটে মূল পার্থক্য কী?

৩. একটা স্ক্রিপ্ট টার্মিনালে ঠিকঠাক চলে, কিন্তু cron-এ "command not found" দেয়। সম্ভাব্য কারণ ও দুটো সমাধান বলো।

৪. `0 2 * * * tar czf /backup/$(date +%F).tar.gz /home` লাইনে সমস্যা কী?

৫. Ubuntu-তে `/etc/cron.daily/cleanup.sh` নামে executable স্ক্রিপ্ট রাখলে, সেটা কি চলবে? কেন?

৬. `0 0 1 * 1` লাইনটা কখন চলবে — "মাসের ১ তারিখ যদি সোমবার হয়"?

৭. cron আর anacron-এর মূল পার্থক্য কী? ল্যাপটপে কোনটা দরকার?

৮. `at` দিয়ে ৪৫ মিনিট পরে একবার `/usr/local/bin/report` চালানোর কমান্ড লেখো, আর কিউ দেখা ও বাতিল করার কমান্ড বলো।

৯. systemd timer-এ `Persistent=true` কী করে? এটা কোন ধরনের timer-এর সাথে কাজ করে?

১০. systemd timer বানানোর পর কোন unit enable করতে হয়, আর timer-এর পরের রান কখন সেটা কীভাবে দেখবে?

▼ উত্তর দেখো

১. সোমবার থেকে শুক্রবার, সকাল ৮টা থেকে সন্ধ্যা ৬টার ঘণ্টাগুলোর (৮–১৮) প্রতিটিতে ০, ২০ ও ৪০ মিনিটে। অর্থাৎ শেষ রান ১৮:৪০-এ।

২. `/etc/cron.d/` (এবং `/etc/crontab`)-এর ফাইলে ৫টি সময়-ফিল্ডের পরে একটা অতিরিক্ত **ইউজার** ফিল্ড থাকে — কাজটা কোন ইউজার হিসেবে চলবে। ইউজার crontab-এ এই ফিল্ড নেই, কারণ মালিকের নামেই চলে। এছাড়া `/etc/cron.d/` ফাইল root-এর মালিকানায় ও নামে ডট ছাড়া হতে হয়।

৩. cron ন্যূনতম environment দেয় — `PATH` সাধারণত শুধু `/usr/bin:/bin`, `.bashrc`/`.profile` পড়ে না, shell `/bin/sh`। তাই `/usr/local/bin`, `~/.local/bin` বা `/usr/sbin`-এর কমান্ড পায়

না। সমাধান: (ক) কমান্ডের পুরো পাথ লেখো (`which` দিয়ে খুঁজে), (খ) crontab-এর উপরে `PATH=...` সেট করো, (গ) স্ক্রিপ্টের ভেতরে `export PATH=...` করো।

৪. crontab-এ `%` মানে নতুন লাইন; প্রথম `%`-এর পরের অংশ stdin হয়ে যায়, ফলে কমান্ড ভেঙে যায়। ঠিক করতে `date +\%F` লেখো, অথবা কমান্ডটা স্ক্রিপ্টে রাখো।

৫. না। Debian/Ubuntu-র `run-parts` ফাইলের নামে ডট (`.`) থাকলে সেটা উপেক্ষা করে। নাম `cleanup` রাখতে হবে। `run-parts --test /etc/cron.daily` দিয়ে যাচাই করা যায়।

৬. না। মাসের দিন আর সপ্তাহের দিন দুটোই নির্দিষ্ট হলে cron OR করে — এটা প্রতি মাসের ১ তারিখে **এবং** প্রতি সোমবার মধ্যরাতে চলবে। "দুটোই" শর্ত চাইলে কমান্ডের ভেতরে `date +\%u` দিয়ে চেক করতে হয় (বা systemd-এ `Mon *-*-01 00:00`)।

৭. cron নির্দিষ্ট সময়ে চালায়, মেশিন তখন বন্ধ থাকলে কাজ মিস হয়; মিনিট-স্তরের, daemon হিসেবে সবসময় চলে। anacron পর্যায়কাল (দিন) দিয়ে ভাবে — শেষবার কবে চলেছে মনে রাখে (`/var/spool/anacron/`), মিস হলে চালু হওয়ার পরে চালায়। ল্যাপটপ/ডেস্কটপে anacron (বা `Persistent=true` timer) দরকার, যাতে দৈনিক/সাপ্তাহিক কাজ বাদ না পড়ে।

৮. `echo /usr/local/bin/report | at now + 45 minutes` (বা `at now + 45 minutes` লিখে প্রম্পটে কমান্ড দিয়ে `Ctrl+D`)। কিউ দেখতে `atq` (বা `at -l`), বাতিল করতে `atrm <job-নম্বর>` (বা `at -d`)।

৯. মেশিন বন্ধ (বা timer নিষ্ক্রিয়) থাকার কারণে কোনো নির্ধারিত সময় মিস হলে, পরের বার timer চালু হওয়ার সাথে সাথে কাজটা একবার চালায়। শেষ চলার সময় `/var/lib/systemd/timers/stamp-*.timer`-এ রাখা হয়। এটা শুধু `OnCalendar=` (realtime/wall-clock) timer-এর সাথে কাজ করে।

১০. `.timer` unit enable করতে হয় (`sudo systemctl enable --now X.timer`), `.service` না। পরের রান দেখতে `systemctl list-timers` (বা `systemctl status X.timer`); আর লেখার আগে এক্সপ্রেশন যাচাই করতে `systemd-analyze calendar "..."` ।

অধ্যায় ৩০: প্যাকেজ ম্যানেজমেন্ট — সফটওয়্যার ইনস্টল

এই অধ্যায়ে যা শিখবে

- **package** (প্যাকেজ) আসলে কী, তার ভেতরে কী থাকে
- **dependency** (নির্ভরতা) কী, আর কেন এটা সামলানো কঠিন কাজ
- **repository** (রিপোজিটরি — সফটওয়্যারের অনলাইন গুদাম) কীভাবে কাজ করে
- low-level টুল (`dpkg`, `rpm`) আর high-level টুল (`apt`, `dnf`, `pacman`) এর পার্থক্য
- Debian/Ubuntu পরিবার: `apt` এর সব জরুরি সাবকমান্ড, `dpkg`, `sources.list`, deb822 ফরম্যাট, PPA, `apt-mark hold`
- RHEL/Fedora পরিবার: `dnf` / `yum`, `rpm`, `.repo` ফাইল
- Arch পরিবার: `pacman` ও AUR
- distro-নিরপেক্ষ প্যাকেজ: `snap`, `flatpak`, AppImage
- GPG key দিয়ে repository কীভাবে নিরাপদ রাখা হয়, আর পুরনো `apt-key` কেন বাদ দেওয়া হয়েছে
- প্যাকেজ সংক্রান্ত সাধারণ সমস্যা আর তার সমাধান

৩০.১ Windows বনাম Linux: সফটওয়্যার ইনস্টলের দুই দর্শন

Windows-এ সফটওয়্যার ইনস্টল করতে গেলে সাধারণত তুমি কী করো? Google-এ খোঁজো, কোনো ওয়েবসাইট থেকে `setup.exe` নামাও, "Next → Next → I Agree → Finish" চাপো। প্রতিটা সফটওয়্যার আলাদা জায়গা থেকে আসে, আলাদাভাবে আপডেট হয় (বা হয়ই না), আর কোনটা ভাইরাসযুক্ত সেটা বোঝার উপায় কম।

Linux-এর দর্শন একদম আলাদা। এখানে আছে **package manager** (প্যাকেজ ম্যানেজার — সফটওয়্যার ইনস্টল, আপডেট আর মুছে ফেলার কেন্দ্রীয় ব্যবস্থাপক)। তুমি শুধু বলো:

```
$ sudo apt install vlc
```

ব্যস। package manager নিজেই:

1. বিশ্বস্ত repository থেকে VLC খুঁজে বের করে
2. VLC চালাতে আর কী কী লাগবে (dependency) সেগুলো হিসাব করে
3. সবকিছু ডাউনলোড করে, ডিজিটাল সিগনেচার যাচাই করে
4. ঠিক জায়গায় ফাইলগুলো বসায়
5. একটা ডাটাবেসে লিখে রাখে কোন ফাইল কোন প্যাকেজের

পরে `sudo apt upgrade` দিলে সিস্টেমের **সব** সফটওয়্যার একসাথে আপডেট হয়ে যায় — কার্নেল থেকে ব্রাউজার পর্যন্ত।

> 💡 **উপমা:** Windows-এর পদ্ধতি হলো প্রতিটা জিনিস কিনতে আলাদা আলাদা দোকানে যাওয়া। Linux-এর package manager হলো একটা বিশ্বস্ত সুপারশপ, যেখানে একটা লিস্ট দিলে দোকানদার নিজেই সব জিনিস (আর সেগুলোর সাথে যা যা লাগে, যেমন টর্চের সাথে ব্যাটারি) প্যাক করে দেয়, আর মেয়াদ শেষ হলে নিজেই বদলে দেয়।

Android-এর Play Store বা iPhone-এর App Store আসলে এই Linux ধারণারই বংশধর — Debian-এর `apt` সেই ১৯৯৮ সাল থেকে এই কাজ করছে।

৩০.২ Package আসলে কী?

Package হলো একটা আর্কাইভ ফাইল (অনেকটা zip-এর মতো), যার ভেতরে থাকে:

অংশ	কী থাকে	উদাহরণ
প্রোগ্রামের ফাইল	compiled binary, library, কনফিগ, ডকুমেন্টেশন, man page	`/usr/bin/htop`, `/usr/share/man/man1/htop.1.gz`
metadata	নাম, version, লেখক, বর্ণনা, আকার, architecture	`Package: htop`, `Version: 3.3.0-4build1`
dependency তালিকা	এটা চালাতে আর কী কী প্যাকেজ লাগবে	`Depends: libc6, libncursesw6`
maintainer script	ইনস্টলের আগে/পরে চলার মতো ছোট স্ক্রিপ্ট	`preinst`, `postinst`, `prerm`, `postrm`
checksum	প্রতিটা ফাইলের হ্যাশ, যাতে নষ্ট হলে ধরা যায়	`md5sums`

Package ফাইলের ফরম্যাট

পরিবার	ফরম্যাট	এক্সটেনশন	উদাহরণ ফাইলনাম
Debian, Ubuntu, Linux Mint, Pop!_OS	deb	`.deb`	`htop_3.3.0-4build1_amd64.deb`
RHEL, Fedora, CentOS Stream, Rocky, AlmaLinux, openSUSE	RPM	`.rpm`	`htop-3.3.0-3.fc40.x86_64.rpm`
Arch, Manjaro, EndeavourOS	pacman package	`.pkg.tar.zst`	`htop-3.3.0-1-x86_64.pkg.tar.zst`

ফাইলনামেই অনেক তথ্য লুকিয়ে আছে:

```
htop_3.3.0-4build1_amd64.deb
│     │     │       │
│     │     │       └── architecture (amd64 = 64-bit Intel/AMD)
│     │     └────────── Debian/Ubuntu revision (প্যাকেজিং-এর সংস্করণ)
│     └──────────────── upstream version (মূল সফটওয়্যারের version)
└────────────────────── প্যাকেজের নাম
```

> 📝 **মনে রাখো:** Debian `amd64` বলে, RPM-জগৎ একই জিনিসকে `x86_64` বলে। ARM 64-bit কে Debian বলে `arm64`, RPM বলে `aarch64`। `noarch` / `all` মানে প্যাকেজটা যেকোনো architecture-এ চলবে (যেমন Python স্ক্রিপ্ট বা ডকুমেন্টেশন)।

ভেতরে উঁকি দেওয়া (under the hood)

একটা `.deb` ফাইল আসলে একটা `ar` আর্কাইভ, যার ভেতরে তিনটা জিনিস:

```
htop_3.3.0-4build1_amd64.deb   (ar archive)
├── debian-binary       → "2.0"  (ফরম্যাট version)
├── control.tar.zst     → metadata + maintainer scripts
└── data.tar.zst        → আসল ফাইলগুলো (/usr/bin/htop ...)
```

তুমি চাইলে নিজেই দেখতে পারো:

```
$ apt download htop          # শুধু .deb নামাবে, ইনস্টল করবে না
$ ar t htop_*.deb
debian-binary
control.tar.zst
data.tar.zst
$ dpkg-deb --info htop_*.deb      # metadata দেখাবে
$ dpkg-deb --contents htop_*.deb  # ভেতরের ফাইল তালিকা
```

`.rpm` ফাইলের ভেতরে থাকে একটা header (metadata) আর একটা cpio আর্কাইভ (payload)। `rpm2cpio file.rpm | cpio -idmv` দিয়ে ইনস্টল না করেই খোলা যায়।

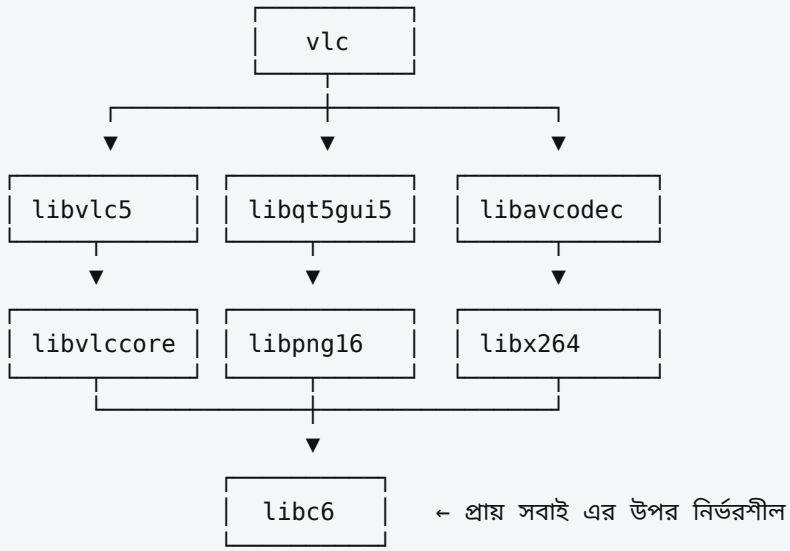
৩০.৩ Dependency — নির্ভরতার জাল

Dependency মানে একটা প্রোগ্রাম চলার জন্য অন্য যে প্রোগ্রাম বা library লাগে। Linux-এ প্রায় সব প্রোগ্রামই **shared library** (ভাগাভাগি করা লাইব্রেরি — একাধিক প্রোগ্রাম একসাথে ব্যবহার করে এমন কোডের ফাইল, যেমন `libc.so.6`) ব্যবহার করে।

কেন? ধরো ১০০টা প্রোগ্রাম PNG ছবি পড়তে পারে। প্রত্যেকে যদি PNG পড়ার কোড নিজের ভেতরে রাখে, তাহলে:

- ১০০ গুণ ডিস্ক আর RAM খরচ
- PNG লাইব্রেরিতে নিরাপত্তা ত্রুটি পাওয়া গেলে ১০০টা প্রোগ্রাম আলাদাভাবে আপডেট করতে হবে

shared library থাকলে একবার `libpng` আপডেট করলেই ১০০টা প্রোগ্রাম নিরাপদ।



Dependency-র ধরন (Debian পরিভাষা)

ফিল্ড	মানে
`Depends`	অবশ্যই লাগবে, না থাকলে ইনস্টল হবে না
`Pre-Depends`	ইনস্টল শুরুর **আগেই** লাগবে
`Recommends`	সাধারণত সাথে থাকা উচিত; `apt` ডিফল্টে এগুলোও ইনস্টল করে
`Suggests`	থাকলে ভালো, ডিফল্টে ইনস্টল হয় না
`Conflicts` / `Breaks`	এর সাথে একসাথে থাকতে পারবে না
`Provides`	"আমি অমুক ক্ষমতা দিই" — যেমন অনেকগুলো mail server সবাই `mail-transport-agent` provide করে

RPM-এর দিকেও একই ধারণা আছে: `Requires`, `Recommends`, `Suggests`, `Conflicts`, `Provides`, `Obsoletes`।

"Dependency Hell" — পুরনো দিনের দুঃস্বপ্ন

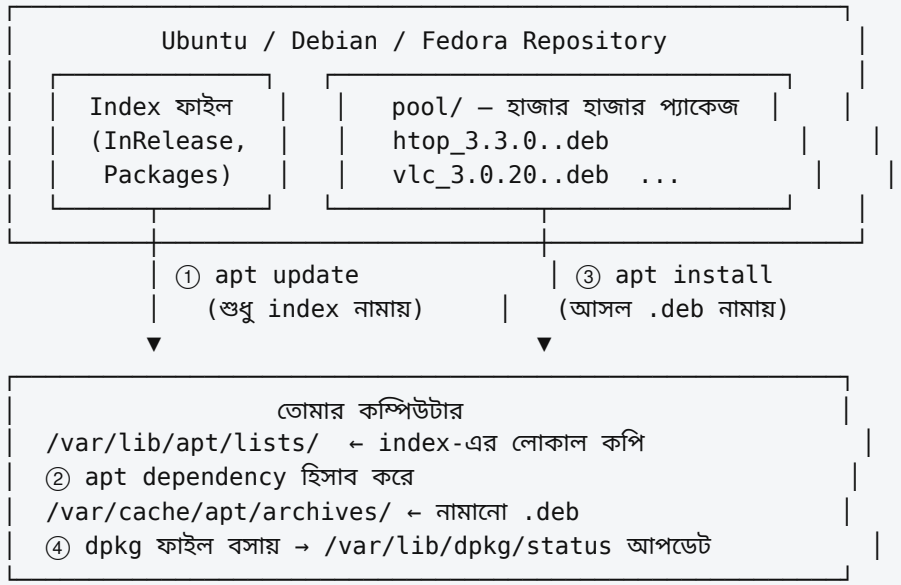
package manager আসার আগে, একটা প্রোগ্রাম ইনস্টল করতে গেলে বলত "libfoo লাগবে"। libfoo ইনস্টল করতে গেলে বলত "libbar 2.0 লাগবে"। কিন্তু অন্য একটা প্রোগ্রাম চায় libbar 1.0! এই প্যাঁচকে বলে **dependency hell**। আধুনিক high-level package manager একটা **dependency solver** (একধরনের গাণিতিক সমাধানকারী) চালিয়ে এই জট নিজেই খুলে দেয়। `dnf` ও openSUSE-র `zypper` এর জন্য `libsolv` নামের SAT-solver লাইব্রেরি ব্যবহার করে।

> ⚠️ **সতর্কতা:** ইন্টারনেট থেকে র‍্যান্ডম `.deb` / `.rpm` নামিয়ে ইনস্টল করলে dependency গড়মিল হতে পারে, আর সেটা অবিশ্বস্ত উৎস হলে ম্যালওয়্যারও হতে পারে। সম্ভব হলে সবসময় অফিসিয়াল repository ব্যবহার করো।

৩০.৪ Repository — সফটওয়্যারের গুদাম

Repository (সংক্ষেপে repo) হলো একটা সার্ভার (HTTP/HTTPS ওয়েবসাইটের মতো), যেখানে হাজার হাজার প্যাকেজ আর একটা **index** (সূচি) রাখা থাকে। index-এ লেখা থাকে কোন প্যাকেজের কোন version, কী dependency, আর checksum কত।

পৃথিবী জুড়ে এই repo-র অনেক কপি থাকে, যাদের বলে **mirror**। বাংলাদেশ থেকে কাছের mirror ব্যবহার করলে ডাউনলোড দ্রুত হয়।



> 📝 **মনে রাখো:** `apt update` কোনো সফটওয়্যার আপডেট করে **না** — এটা শুধু "দোকানের নতুন মূল্যতালিকা" নামায়। আসল আপডেট হয় `apt upgrade` দিয়ে। নতুনরা প্রায়ই এই দুইটা গুলিয়ে ফেলে।

Ubuntu-র repo component

Ubuntu-তে প্রতিটা repo চারটা ভাগে বিভক্ত:

component	কী থাকে	কে সাপোর্ট দেয়
`main`	free/open source, Canonical অফিসিয়ালি সাপোর্ট করে	Canonical
`universe`	free/open source, community maintain করে	Community

component	কী থাকে	কে সাপোর্ট দেয়
`restricted`	proprietary ড্রাইভার (যেমন কিছু NVIDIA ড্রাইভার)	Canonical
`multiverse`	কপিরাইট/লাইসেন্স সমস্যাযুক্ত সফটওয়্যার	কেউ না

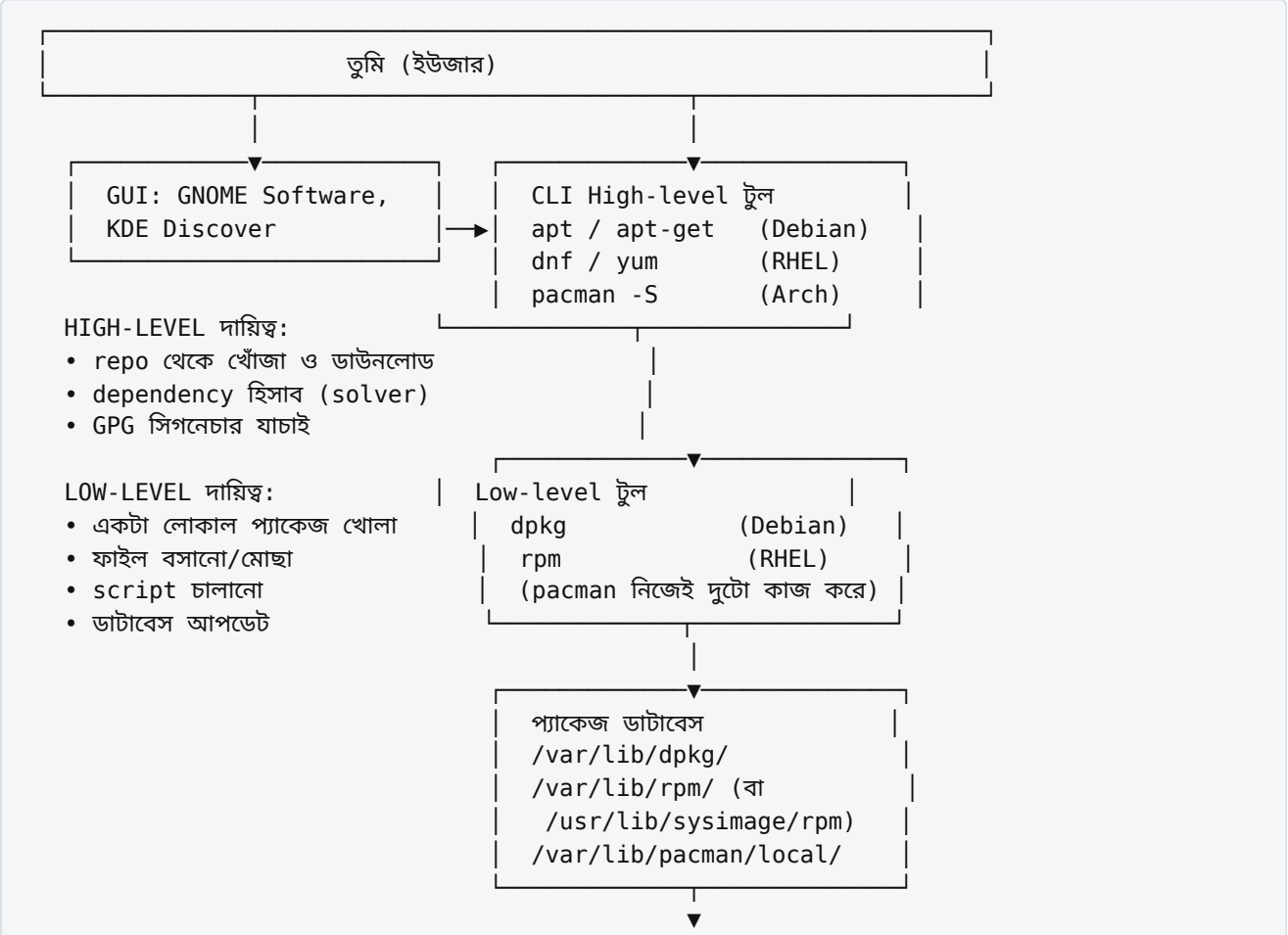
Debian 12-এ ভাগগুলো: `main`, `contrib`, `non-free`, আর Debian 12 থেকে নতুন `non-free-firmware` (Wi-Fi/GPU ফার্মওয়্যারের জন্য)।

আর **suite** (বা pocket) গুলো Ubuntu 24.04 (codename `noble`) এ এমন:

suite	মানে
`noble`	রিলিজের দিনের মূল প্যাকেজ
`noble-updates`	রিলিজের পরের বাগ ফিক্স
`noble-security`	নিরাপত্তা আপডেট
`noble-backports`	নতুন version, যা পুরনো রিলিজে আনা হয়েছে

৩০.৫ Low-level বনাম High-level টুল

এটা এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ architecture ধারণা। প্রতিটা distro-তে প্যাকেজ টুল দুই স্তরে কাজ করে:



```
                    ┌──────────────────────────┐
                    │ ফাইল সিস্টেম: /usr /etc ... │
                    └──────────────────────────┘
```

বৈশিষ্ট্য	Low-level (`dpkg` , `rpm`)	High-level (`apt` , `dnf`)
ইন্টারনেট থেকে নামাতে পারে?	না	হ্যাঁ
dependency নিজে সমাধান করে?	না, শুধু অভিযোগ করে	হ্যাঁ
কোন ফাইলে কাজ করে?	লোকাল `.deb` / `.rpm` ফাইল	repo-র প্যাকেজ নাম (লোকাল ফাইলও পারে)
কখন দরকার?	তদন্ত (কোন ফাইল কোন প্যাকেজের), ভাঙা অবস্থা মেরামত	দৈনন্দিন সব কাজ

> 💡 **উপমা:** `dpkg` হলো রাজমিস্ত্রি — ইট দিলে সে বসিয়ে দেবে, কিন্তু ইট কোথা থেকে আসবে, কত সিমেন্ট লাগবে সেটা জানে না। `apt` হলো ঠিকাদার — সে হিসাব করে, মালামাল কিনে আনে, তারপর রাজমিস্ত্রিকে দিয়ে কাজ করায়।

`apt install vlc` দিলে ভেতরে আসলে `apt` সব `.deb` নামিয়ে শেষে `dpkg` -কে ডেকে কাজ করায়। Fedora-তে `dnf` একইভাবে `rpm` লাইব্রেরি (librpm) ব্যবহার করে।

৩০.৬ Debian পরিবার: `apt`

Debian, Ubuntu, Linux Mint, Pop!_OS, Kali — সবখানে `apt` চলে।

`apt` , `apt-get` , `apt-cache` — পার্থক্য কী?

ইতিহাসে প্রথমে এসেছিল `apt-get` (ইনস্টল/আপডেট) আর `apt-cache` (খোঁজা/তথ্য)। ২০১৪ সালে (APT 1.0) এই দুটোর জনপ্রিয় কাজ এক করে আসে `apt` — রঙিন, প্রোগ্রেস বার সহ, মানুষের জন্য সুবিধাজনক।

টুল	কার জন্য	মন্তব্য
`apt`	মানুষ (interactive)	output ভবিষ্যতে বদলাতে পারে; স্ক্রিপ্টে ব্যবহার করলে সতর্কবার্তা দেয়
`apt-get`	স্ক্রিপ্ট, Dockerfile	output স্থির, পুরনো সব অপশন আছে
`apt-cache`	স্ক্রিপ্টে খোঁজা/তথ্য	`apt-cache policy` , `apt-cache depends` , `apt-cache rdepends`

```
$ apt list --installed | head -3

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
```

```
Listing...
accountsservice/noble,now 23.13.9-2ubuntu6 amd64 [installed,automatic]
```

> 💡 **টিপস:** শেল স্ক্রিপ্ট আর Dockerfile-এ `apt-get` লেখো, টার্মিনালে নিজে টাইপ করার সময় `apt`।

`apt update` — index রিফ্রেশ

```
$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [612 kB]
Fetched 864 kB in 2s (432 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

- `Hit` = আগের index এখনো চলতি, নতুন কিছু নেই
- `Get` = নতুন index নামানো হচ্ছে
- `Ign` = উপেক্ষা করা হয়েছে (সাধারণত অনুবাদ ফাইল না পেলে)
- `Err` = সমস্যা (নেটওয়ার্ক, ভুল URL, বা GPG key সমস্যা)

এই index জমা থাকে `/var/lib/apt/lists/` এ।

`apt upgrade` বনাম `apt full-upgrade`

```
$ apt list --upgradable
Listing... Done
curl/noble-updates,noble-security 8.5.0-2ubuntu10.6 amd64 [upgradable from: 8.5.0-
2ubuntu10.5]
libcurl4t64/noble-updates,noble-security 8.5.0-2ubuntu10.6 amd64 [upgradable from:
8.5.0-2ubuntu10.5]

$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  curl libcurl4t64
2 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 568 kB of archives.
After this operation, 0 B of additional disk space will be used.
Do you want to continue? [Y/n] y
```

কমান্ড	নতুন প্যাকেজ ইনস্টল করতে পারে?	পুরনো প্যাকেজ মুছতে পারে?	কখন ব্যবহার
`apt-get upgrade`	না	না	সবচেয়ে রক্ষণশীল
`apt upgrade`	হ্যাঁ (নতুন dependency লাগলে)	না	দৈনন্দিন আপডেট
`apt full-upgrade` (= `apt-get dist-upgrade`)	হ্যাঁ	হ্যাঁ, দরকার হলে	বড় পরিবর্তন, নতুন kernel series, রিলিজ আপগ্রেডের অংশ

> ⚠️ **সতর্কতা:** `full-upgrade` কোনো প্যাকেজ মুছে ফেলার প্রস্তাব দিলে `Y` চাপার আগে তালিকাটা পড়ো। "The following packages will be REMOVED" এর নিচে যদি গুরুত্বপূর্ণ কিছু (যেমন `ubuntu-desktop`) দেখো, থামো।

> 📝 **মনে রাখো:** "X packages have been kept back" লেখা আসলে মানে সেই প্যাকেজগুলোর আপগ্রেডে অন্য কিছু মুছতে বা বদলাতে হবে, অথবা Ubuntu-র **phased updates** (আপডেট ধাপে ধাপে কিছু ইউজারকে আগে দেওয়া) চলছে। phased update হলে অপেক্ষা করাই ভালো।

Ubuntu-র এক রিলিজ থেকে পরের রিলিজে (যেমন 22.04 → 24.04) যাওয়ার অফিসিয়াল টুল হলো `do-release-upgrade`, শুধু `full-upgrade` নয়।

একসাথে:

```
$ sudo apt update && sudo apt upgrade -y
```

`&&` মানে প্রথমটা সফল হলেই দ্বিতীয়টা চলবে (অধ্যায় ১৯ ও অধ্যায় ৪১ দেখো)। `-y` মানে সব প্রশ্নে স্বয়ংক্রিয়ভাবে "yes"।

`apt install` — ইনস্টল করা

```
$ sudo apt install htop
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  htop
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 171 kB of archives.
After this operation, 434 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 htop amd64 3.3.0-4build1 [171
kB]
Fetched 171 kB in 1s (210 kB/s)
Selecting previously unselected package htop.
(Reading database ... 198432 files and directories currently installed.)
Preparing to unpack .../htop_3.3.0-4build1_amd64.deb ...
```

```
Unpacking htop (3.3.0-4build1) ...
Setting up htop (3.3.0-4build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

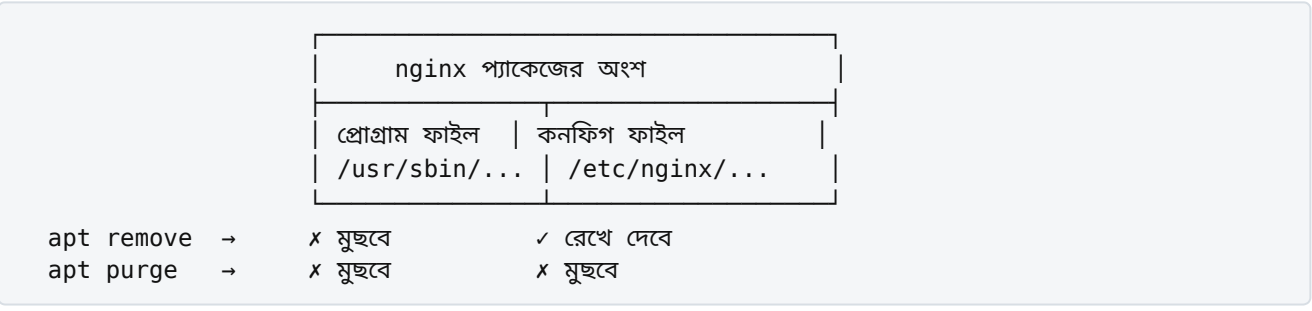
শেষের লাইনগুলো খেয়াল করো: `Unpacking` আর `Setting up` হলো `dpkg`-এর কাজ। `Processing triggers` মানে অন্য কোনো প্যাকেজ (এখানে `man-db`) বলেছিল "নতুন man page এলে আমাকে জানিও" — তাই man ডাটাবেস রিফ্রেশ হলো।

গুরুত্বপূর্ণ ব্যবহার:

```
$ sudo apt install nginx curl git           # একসাথে একাধিক
$ sudo apt install nginx=1.24.0-2ubuntu7    # নির্দিষ্ট version
$ sudo apt install -t noble-backports cockpit   # নির্দিষ্ট suite থেকে
$ sudo apt install ./google-chrome-stable_current_amd64.deb   # লোকাল .deb, dependency সহ!
$ sudo apt install --no-install-recommends ffmpeg  # Recommends বাদ দিয়ে, হালকা ইনস্টল
$ sudo apt reinstall htop                   # আবার ইনস্টল (নষ্ট ফাইল ঠিক করতে)
```

> 💡 **টিপস:** লোকাল `.deb` ইনস্টল করার সময় `./` দিতে ভুলো না। `apt install file.deb` লিখলে apt ভাববে এটা repo-তে থাকা প্যাকেজের নাম। `./file.deb` দিলে বুঝবে এটা ফাইল, আর `dpkg -i` এর মতো আটকে না গিয়ে dependency-ও নামিয়ে আনবে।

`apt remove`, `apt purge`, `apt autoremove`




```
$ sudo apt remove nginx      # প্রোগ্রাম যায়, /etc কনফিগ থাকে (পরে আবার ইনস্টলে কাজে লাগে)
$ sudo apt purge nginx       # কনফিগ সহ সব মুছে দেয়
$ sudo apt autoremove        # আর কারো দরকার নেই এমন "অনাথ" dependency মুছে দেয়
$ sudo apt autoremove --purge   # অনাথদের কনফিগ সহ
```

> ⚠️ **সতর্কতা:** `purge` তোমার **home ডিরেক্টরির** ফাইল (যেমন `~/.config/...`) মোছে না — শুধু প্যাকেজের নিজস্ব সিস্টেম কনফিগ (`/etc/...`) মোছে। আর ডাটাবেস সার্ভারের ক্ষেত্রে (যেমন `mysql-server`) purge প্রশ্ন করতে পারে ডেটা ডিরেক্টরিও মুছবে কিনা — সাবধানে উত্তর দাও।

autoremove কীভাবে জানে কোনটা "অনাথ"? apt প্রতিটা প্যাকেজকে দুই ভাগে চিহ্নিত রাখে:

- **manual** — তুমি নিজে নাম দিয়ে ইনস্টল করেছ

- **automatic** — অন্য প্যাকেজের dependency হিসেবে এসেছে

কোনো automatic প্যাকেজের উপর যখন আর কোনো manual প্যাকেজ নির্ভর করে না, তখন সে autoremove-এর শিকার। এই তথ্য থাকে `/var/lib/apt/extended_states` ফাইলে।

```
$ apt-mark showmanual | head -3
$ sudo apt-mark manual libfoo1    # "এটা আমার দরকার, মুছবে না"
$ sudo apt-mark auto libfoo1      # "এটা শুধু dependency"
```

> ⚠️ **সতর্কতা:** `autoremove` চালানোর আগে তালিকাটা দেখো। কখনো কখনো একটা meta-package (যেমন `ubuntu-desktop`) মুছে ফেললে পুরো ডেস্কটপের প্যাকেজগুলো "অনাথ" হয়ে যায়, আর autoremove সেগুলো মুছতে চাইবে!

`apt search`, `apt show`, `apt list`

```
$ apt search "image viewer"
Sorting... Done
Full Text Search... Done
eog/noble 45.3-1ubuntu1 amd64
  Eye of GNOME graphics viewer program

feh/noble 3.10.1-1build3 amd64
  imlib2 based image viewer
...

$ apt search --names-only '^python3-requests'   # শুধু নামে খোঁজো (regex)
```

```
$ apt show htop
Package: htop
Version: 3.3.0-4build1
Priority: optional
Section: utils
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Installed-Size: 434 kB
Depends: libc6 (>= 2.38), libncursesw6 (>= 6), libnl-3-200 (>= 3.2.7), libnl-genl-3-200
(>= 3.2.7), libtinfo6 (>= 6)
Suggests: lm-sensors, lsof, strace
Homepage: https://htop.dev/
Download-Size: 171 kB
APT-Sources: http://archive.ubuntu.com/ubuntu noble/main amd64 Packages
Description: interactive processes viewer
 Htop is an ncursed-based process viewer similar to top, but it
 allows one to scroll the list vertically and horizontally ...
```

```
$ apt list --installed             # ইনস্টল করা সব
$ apt list --upgradable            # যাদের আপডেট আছে
$ apt list --all-versions nginx    # repo-তে থাকা সব version
$ apt list 'linux-image-*'         # pattern দিয়ে
```

`apt policy` — কোন version কোথা থেকে আসবে

একই প্যাকেজ একাধিক repo-তে থাকতে পারে (যেমন Ubuntu-র নিজের repo আর একটা PPA)। apt কোনটা বেছে নেবে সেটা ঠিক করে **priority** (pin priority) দিয়ে।

```
$ apt policy nginx
nginx:
  Installed: (none)
  Candidate: 1.24.0-2ubuntu7.1
  Version table:
     1.24.0-2ubuntu7.1 500
        500 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages
        500 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages
     1.24.0-2ubuntu7 500
        500 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages
```

- `Installed` — এখন যা ইনস্টল আছে
- `Candidate` — `apt install` দিলে যা ইনস্টল হবে
- সংখ্যা `500` — সাধারণ repo-র ডিফল্ট priority; `100` = ইনস্টল করা version বা backports; `990` = target release

`apt policy` (নাম ছাড়া) দিলে সব repo আর তাদের priority দেখায়। কোনো repo-র priority বদলাতে চাইলে `/etc/apt/preferences.d/` এ ফাইল লেখো (একে বলে **APT pinning**):

```
# /etc/apt/preferences.d/firefox-mozilla
Package: firefox*
Pin: origin packages.mozilla.org
Pin-Priority: 1000
```

`man apt_preferences` এ পুরো নিয়ম আছে।

অন্যান্য দরকারি apt কমান্ড

কমান্ড	কাজ
`apt-cache depends pkg`	pkg কোন কোন প্যাকেজের উপর নির্ভর করে
`apt-cache rdepends pkg`	কে কে pkg-এর উপর নির্ভর করে (reverse dependency)
`apt download pkg`	শুধু `.deb` নামাও, ইনস্টল নয়
`apt source pkg`	সোর্স কোড নামাও (অধ্যায় ৩১)
`sudo apt clean`	`/var/cache/apt/archives/` এর সব নামানো `.deb` মুছে জায়গা খালি করো
`sudo apt autoclean`	শুধু যেগুলো আর repo-তে নেই সেগুলো মোছো
`sudo apt --fix-broken install` (`-f`)	ভাঙা dependency মেরামত

কমান্ড	কাজ
`apt changelog pkg`	প্যাকেজের পরিবর্তনের ইতিহাস
`sudo apt edit-sources`	সোর্স ফাইল এডিটরে খোলো

apt-এর কাজের লগ থাকে `/var/log/apt/history.log` (কী ইনস্টল/মুছেছে) আর `/var/log/apt/term.log` (পুরো টার্মিনাল output) এ — লগ সম্পর্কে বিস্তারিত অধ্যায় ২৮-এ।

```
$ grep -A3 "Start-Date" /var/log/apt/history.log | tail -4
Start-Date: 2026-11-12  10:14:02
Commandline: apt install htop
Requested-By: rahim (1000)
Install: htop:amd64 (3.3.0-4build1)
```

Ubuntu-র স্বয়ংক্রিয় নিরাপত্তা আপডেট

Ubuntu-তে ডিফল্টে `unattended-upgrades` প্যাকেজ চালু থাকে, যা প্রতিদিন নিরাপত্তা আপডেট নিজেই ইনস্টল করে। কনফিগ `/etc/apt/apt.conf.d/50unattended-upgrades` আর `20auto-upgrades`। Debian-এ `sudo apt install unattended-upgrades` দিয়ে, তারপর `sudo dpkg-reconfigure -plow unattended-upgrades` দিয়ে চালু করতে হয়। নিরাপত্তা নিয়ে আরও অধ্যায় ৪৫-এ।

> ⚠️ **সতর্কতা: lock error।** `E: Could not get lock /var/lib/dpkg/lock-frontend` দেখলে মানে অন্য কোনো apt/dpkg প্রক্রিয়া (প্রায়ই `unattended-upgrades` বা GUI Software Updater) চলছে। একসাথে দুটো প্যাকেজ টুল চললে ডাটাবেস নষ্ট হতে পারে, তাই এই lock। কয়েক মিনিট অপেক্ষা করো। lock ফাইল জোর করে `rm` করবে **না** — আগে `ps aux | grep -E 'apt|dpkg'` দিয়ে দেখো কে চলছে (অধ্যায় ২৪)।

৩০.৭ `dpkg` — Debian-এর low-level টুল

`dpkg` সরাসরি `.deb` ফাইল নিয়ে কাজ করে, আর `/var/lib/dpkg/` এর ডাটাবেস রক্ষা করে। এখানে থাকে:

- `/var/lib/dpkg/status` — প্রতিটা প্যাকেজের অবস্থা (একটা বিশাল টেক্সট ফাইল)
- `/var/lib/dpkg/info/*.list` — প্রতিটা প্যাকেজের ফাইল তালিকা
- `/var/lib/dpkg/info/*.postinst` ইত্যাদি — maintainer script

গুরুত্বপূর্ণ অপশন

অপশন	লম্বা রূপ	কাজ
`-i file.deb`	`--install`	লোকাল .deb ইনস্টল (dependency নিজে আনে না)

অপশন	লম্বা রূপ	কাজ
`-r pkg`	`--remove`	প্যাকেজ মোছো, কনফিগ রেখে
`-P pkg`	`--purge`	কনফিগ সহ মোছো
`-l [pattern]`	`--list`	ইনস্টল করা প্যাকেজের তালিকা ও অবস্থা
`-L pkg`	`--listfiles`	প্যাকেজটা কোন কোন ফাইল বসিয়েছে
`-S path`	`--search`	এই ফাইলটা কোন প্যাকেজের?
`-s pkg`	`--status`	প্যাকেজের বিস্তারিত অবস্থা
`-c file.deb`	`--contents`	ইনস্টল না করে .deb-এর ভেতরের ফাইল দেখো
`-I file.deb`	`--info`	.deb-এর metadata
`--configure -a`		আধা-ইনস্টল হওয়া সব প্যাকেজের কনফিগারেশন শেষ করো
`--get-selections`		সব প্যাকেজ ও তাদের নির্বাচিত অবস্থা

`dpkg -i` ও dependency সমস্যা

```
$ sudo dpkg -i some-app_1.0_amd64.deb
Selecting previously unselected package some-app.
(Reading database ... 198500 files and directories currently installed.)
Preparing to unpack some-app_1.0_amd64.deb ...
Unpacking some-app (1.0) ...
dpkg: dependency problems prevent configuration of some-app:
 some-app depends on libfoo2; however:
  Package libfoo2 is not installed.

dpkg: error processing package some-app (--install):
 dependency problems - leaving unconfigured
Errors were encountered while processing:
 some-app
```

দেখো — `dpkg` শুধু অভিযোগ করল, নিজে `libfoo2` আনল না। সমাধান:

```
$ sudo apt --fix-broken install
```

apt তখন হারানো dependency নামিয়ে আধা-ইনস্টল প্যাকেজের কাজ শেষ করবে (বা সেটা সম্ভব না হলে প্যাকেজটা মুছে দেওয়ার প্রস্তাব দেবে)। এজন্যই আধুনিক পরামর্শ: সরাসরি `sudo apt install ./some-app_1.0_amd64.deb` ব্যবহার করো।

`dpkg -l` — তালিকা আর অবস্থার কোড

```
$ dpkg -l 'nginx*'
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name           Version            Architecture Description
```

```
+++-==============-==================-============-================================
ii  nginx          1.24.0-2ubuntu7.1  amd64        small, powerful, scalable web/proxy
server
ii  nginx-common   1.24.0-2ubuntu7.1  all          small, powerful, scalable web/proxy
server - common files
rc  nginx-core     1.18.0-6ubuntu14   amd64        nginx web/proxy server (standard
version)
```

প্রথম দুই অক্ষর পড়তে শেখো:

কোড	মানে
`ii`	ঠিকঠাক ইনস্টল আছে (desired=install, status=installed)
`rc`	remove করা হয়েছে, কিন্তু কনফিগ ফাইল রয়ে গেছে (purge হয়নি)
`hi`	hold করা আছে, ইনস্টল অবস্থায়
`iU`	unpacked, কনফিগার হয়নি — অসম্পূর্ণ
`iF`	half-configured — কনফিগারেশনে ব্যর্থ
`un`	ইনস্টল নেই

> 💡 **টিপস:** সব `rc` অবস্থার প্যাকেজের অবশিষ্ট কনফিগ একবারে পরিষ্কার করতে:
>
> ```
> $ dpkg -l | awk '/^rc/ {print $2}' | xargs -r sudo dpkg -P
> ```
>
> (`awk` আর `xargs` শিখবে অধ্যায় ২২ ও অধ্যায় ২৩-এ।)

`dpkg -L` ও `dpkg -S` — তদন্তকারী কমান্ড

```
$ dpkg -L htop
/.
/usr
/usr/bin
/usr/bin/htop
/usr/share
/usr/share/applications
/usr/share/applications/htop.desktop
/usr/share/doc/htop/changelog.Debian.gz
/usr/share/man/man1/htop.1.gz
...

$ dpkg -S /usr/bin/ls
coreutils: /usr/bin/ls

$ dpkg -S bin/ssh
openssh-client: /usr/bin/ssh
openssh-client: /usr/bin/ssh-add
...
```

> 📝 **মনে রাখো:** `dpkg -S` শুধু **ইনস্টল করা** প্যাকেজে খোঁজে। যে ফাইল এখনো তোমার সিস্টেমে নেই, সেটা কোন প্যাকেজে আছে জানতে `apt-file` ব্যবহার করো:
>
> ```
> $ sudo apt install apt-file && sudo apt-file update
> $ apt-file search bin/ifconfig
> net-tools: /usr/sbin/ifconfig
> ```

Ubuntu-তে কোনো কমান্ড না থাকলে নিজেই বলে দেয় (এটা `command-not-found` প্যাকেজের কাজ):

```
$ tree
Command 'tree' not found, but can be installed with:
sudo apt install tree
```

`dpkg -r` ও মেরামত

```
$ sudo dpkg -r htop          # মোছো (কনফিগ থাকবে)
$ sudo dpkg -P htop          # পুরো মোছো
$ sudo dpkg --configure -a   # আপডেট মাঝপথে বন্ধ হলে (বিদ্যুৎ গেলে!) এটা আগে চালাও
$ sudo dpkg-reconfigure tzdata   # কোনো প্যাকেজের সেটআপ প্রশ্নগুলো আবার করো
```

> 💡 **টিপস:** আপডেটের মাঝে কম্পিউটার বন্ধ হয়ে গেলে, সবচেয়ে কাজের রেসিপি হলো:
>
> ```
> $ sudo dpkg --configure -a
> $ sudo apt --fix-broken install
> $ sudo apt update && sudo apt upgrade
> ```

ইনস্টল তালিকা ব্যাকআপ ও নতুন মেশিনে পুনরুদ্ধার

```
$ apt-mark showmanual > my-packages.txt      # পুরনো মেশিনে
$ xargs -a my-packages.txt sudo apt install -y   # নতুন মেশিনে (একই distro version)
```

৩০.৮ Debian পরিবারের repository কনফিগারেশন

পুরনো এক-লাইন ফরম্যাট: `sources.list`

Debian 12 এখনো ডিফল্টে `/etc/apt/sources.list` ব্যবহার করে:

```
deb http://deb.debian.org/debian bookworm main contrib non-free non-free-firmware
deb http://deb.debian.org/debian bookworm-updates main contrib non-free non-free-
firmware
deb http://security.debian.org/debian-security bookworm-security main contrib non-free
```

```
non-free-firmware
# deb-src http://deb.debian.org/debian bookworm main
```

প্রতিটা লাইনের গঠন:

```
deb  [options]  http://deb.debian.org/debian  bookworm  main contrib
|    |          |                             |         |
|    |          |                             |         └─ component
|    |          |                             └─────────── suite / codename
|    |          └─────────────────────────────────────── mirror URL
|    └──────────────────────────────────────────────────── যেমন [signed-
by=/path/key.gpg]
└──────────────────────────────────────────────────────── deb = binary, deb-src =
source
```

নতুন deb822 ফরম্যাট (Ubuntu 24.04+)

Ubuntu 24.04 থেকে `/etc/apt/sources.list` প্রায় খালি (শুধু একটা কমেন্ট), আর আসল কনফিগ থাকে `/etc/apt/sources.list.d/ubuntu.sources` এ, **deb822** ফরম্যাটে (key: value ধরনের, বেশি পাঠযোগ্য):

```
$ cat /etc/apt/sources.list.d/ubuntu.sources
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg

Types: deb
URIs: http://security.ubuntu.com/ubuntu/
Suites: noble-security
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

একটা ব্লক বন্ধ করতে চাইলে তাতে `Enabled: no` লাইন যোগ করো। Debian 13 (trixie)-ও deb822-এর দিকে যাচ্ছে।

`/etc/apt/sources.list.d/`

থার্ড-পার্টি repo (Docker, Google Chrome, VS Code ইত্যাদি) মূল ফাইল না ঘেঁটে এই ডিরেক্টরিতে আলাদা ফাইল রাখে — `.list` (এক-লাইন ফরম্যাট) বা `.sources` (deb822)। এতে একটা repo মুছতে শুধু একটা ফাইল মুছলেই হয়।

> 💡 **টিপস:** কাছের mirror বেছে নিতে Ubuntu-তে GUI-র "Software & Updates" → "Download from" ব্যবহার করতে পারো, অথবা ফাইলে `archive.ubuntu.com` এর জায়গায় দেশের mirror (যেমন

> `bd.archive.ubuntu.com`) বসাতে পারো — বসানোর আগে mirror-টা সক্রিয় কিনা ব্রাউজারে দেখে নিও। ফাইল বদলের পরে সবসময় `sudo apt update`।

PPA — Personal Package Archive (Ubuntu-only)

PPA হলো Launchpad (Canonical-এর প্ল্যাটফর্ম) এ কোনো ব্যক্তি বা দলের বানানো ছোট repo। অফিসিয়াল repo-তে যে নতুন version এখনো আসেনি, সেটা অনেক সময় PPA-তে পাওয়া যায়।

```
$ sudo apt install software-properties-common    # add-apt-repository এর জন্য (সাধারণত আগেই থাকে)
$ sudo add-apt-repository ppa:git-core/ppa
 The most current stable version of Git for Ubuntu.
 More info: https://launchpad.net/~git-core/+archive/ubuntu/ppa
Adding repository.
Press [ENTER] to continue or Ctrl-c to cancel.
...
$ sudo apt update
$ sudo apt install git
```

`add-apt-repository` নিজেই:

1. `/etc/apt/sources.list.d/` এ একটা ফাইল লেখে (24.04-এ `.sources` ফরম্যাটে)
2. PPA-র GPG key নামিয়ে বসায়
3. (সাম্প্রতিক version-এ) `apt update` চালায়

PPA সরাতে:

```
$ sudo add-apt-repository --remove ppa:git-core/ppa
$ sudo apt install ppa-purge && sudo ppa-purge ppa:git-core/ppa   # PPA-র প্যাকেজগুলোও অফিসিয়াল version-এ ফেরত নেয়
```

> ⚠️ **সতর্কতা:** PPA মানে কোনো ব্যক্তিকে তোমার কম্পিউটারে **root ক্ষমতা** দেওয়া — কারণ তার প্যাকেজের `postinst` স্ক্রিপ্ট root হিসেবে চলে। শুধু পরিচিত, বিশ্বস্ত PPA ব্যবহার করো। অনেক PPA যোগ করলে রিলিজ আপগ্রেডের সময় গোলমাল হয়। আর PPA শুধু Ubuntu-র জন্য — Debian-এ যোগ কোরো না।

`apt-mark hold` — আপডেট আটকে রাখা

ধরো তোমার একটা অ্যাপ নির্দিষ্ট version-এর `docker-ce` এ ঠিকমতো চলে, আর নতুন version-এ সমস্যা আছে। তখন সেটাকে "hold" করো:

```
$ sudo apt-mark hold docker-ce
docker-ce set on hold.

$ apt-mark showhold
docker-ce
```

```
$ sudo apt upgrade
...
The following packages have been kept back:
  docker-ce
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.

$ sudo apt-mark unhold docker-ce
Canceled hold on docker-ce.
```

`dpkg -l` এ hold করা প্যাকেজ `hi` দেখায়।

> ⚠️ **সতর্কতা:** hold করা প্যাকেজ নিরাপত্তা আপডেটও পাবে না। কাজ শেষ হলে unhold করতে ভুলো না। kernel hold করলে নিরাপত্তা ঝুঁকি বাড়ে।

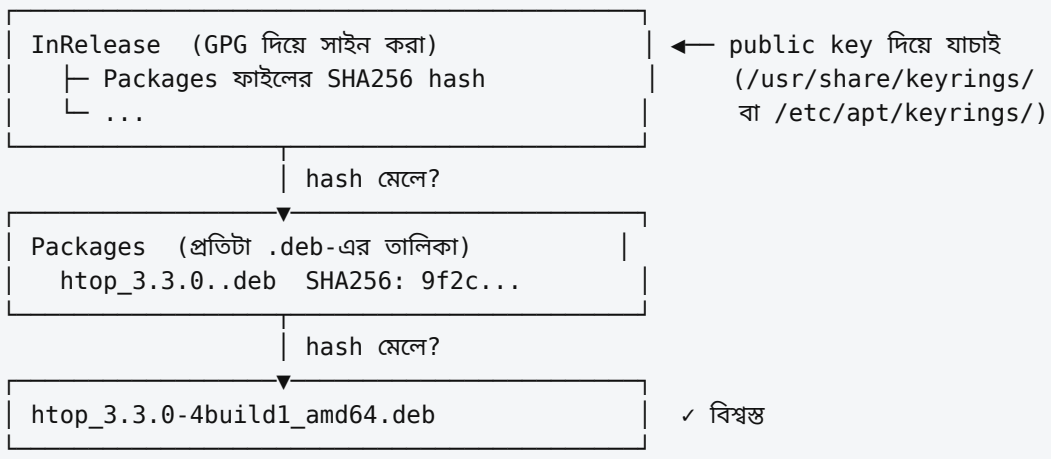
৩০.৯ GPG key দিয়ে repository নিরাপত্তা

ইন্টারনেট থেকে প্যাকেজ নামাচ্ছ, আর সেটা root হিসেবে চলছে। কেউ যদি mirror হ্যাক করে, বা মাঝপথে (man-in-the-middle) প্যাকেজ বদলে দেয়? এর সমাধান **public-key cryptography** (পাবলিক-কী ক্রিপ্টোগ্রাফি) — GPG (GNU Privacy Guard) দিয়ে।

> 💡 **উপমা:** repo-র মালিকের কাছে আছে একটা গোপন সিলমোহর (**private key**)। সে প্রতিটা তালিকায় সিল মারে। তোমার কাছে আছে সেই সিলের একটা নমুনা ছবি (**public key**)। নমুনার সাথে সিল মিললে বোঝো আসল; না মিললে জাল। নমুনা দেখে কেউ নতুন সিল বানাতে পারে না।

APT-এর বিশ্বাসের শিকল (chain of trust)

APT প্রতিটা প্যাকেজে আলাদা সিগনেচার দেখে না — এটা একটা শিকল বানায়:



একটা লিংক ভাঙলেই apt থেমে যাবে। এজন্যই HTTP (HTTPS নয়) mirror-ও নিরাপদ — বিষয়বস্তু বদলালে hash মিলবে না। তবে HTTPS গোপনীয়তা (কে কী নামাচ্ছে) দেয়।

key না মিললে এমন error আসে:

```
$ sudo apt update
...
W: GPG error: https://example.com/repo stable InRelease: The following signatures
couldn't be verified because the public key is not available: NO_PUBKEY
1234ABCD5678EF90
E: The repository 'https://example.com/repo stable InRelease' is not signed.
```

`apt-key` কেন বাদ দেওয়া হলো

আগে সবাই লিখত `wget -qO- https://.../key | sudo apt-key add -`। সমস্যা: এতে key যেত একটা **global** keyring-এ (`/etc/apt/trusted.gpg`), মানে সেই key দিয়ে **যেকোনো** repo-র প্যাকেজ সাইন করা যেত — এমনকি কেউ চাইলে Ubuntu-র অফিসিয়াল `openssl` এর জাল version-ও! তাই `apt-key` deprecated; Debian 12 / Ubuntu 22.04 থেকে সতর্কতা দেয়, আর নতুন APT (Ubuntu 24.04-এর APT 2.7 সহ) এ এটা ধাপে ধাপে সরিয়ে ফেলা হচ্ছে।

আধুনিক সঠিক পদ্ধতি: `signed-by`

এখন প্রতিটা থার্ড-পার্টি key আলাদা ফাইলে রাখা হয়, আর repo-র লাইনে বলে দেওয়া হয় "এই repo শুধু এই key দিয়ে যাচাই করো"।

উদাহরণ — Docker-এর অফিসিয়াল repo (Ubuntu):

```
# ১. key রাখার ডিরেক্টরি
$ sudo install -m 0755 -d /etc/apt/keyrings

# ২. key নামাও
$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc

# ৩. repo যোগ করো, signed-by দিয়ে
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
\
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# ৪. আপডেট ও ইনস্টল
$ sudo apt update
$ sudo apt install docker-ce
```

key যদি ASCII-armored (`-----BEGIN PGP PUBLIC KEY BLOCK-----`) হয়, `.asc` নামে রাখলে আধুনিক APT সরাসরি পড়তে পারে। পুরনো রীতিতে বাইনারি `.gpg` বানানো হয়:

```
$ curl -fsSL https://example.com/key.asc | sudo gpg --dearmor -o
/etc/apt/keyrings/example.gpg
```

deb822 ফরম্যাটে একই জিনিস:

```
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
```

> 📝 **মনে রাখো:** distro-র নিজস্ব key থাকে `/usr/share/keyrings/` এ (প্যাকেজ দিয়ে আসে)। তুমি নিজে যেগুলো যোগ করো সেগুলো `/etc/apt/keyrings/` এ রাখার রীতি।

> ⚠️ **সতর্কতা:** key-এর **fingerprint** সবসময় অফিসিয়াল ওয়েবসাইটের সাথে মিলিয়ে নাও:
>
> ```
> $ gpg --show-keys /etc/apt/keyrings/docker.asc
> pub   rsa4096 2017-02-22 [SCEA]
>       9DC858229FC7DD38854AE2D88D81803C0EBFCD88
> uid                      Docker Release (CE deb) <docker@docker.com>
> ```
>
> আর কখনো `[trusted=yes]` বা `--allow-unauthenticated` দিয়ে সিগনেচার যাচাই বন্ধ কোরো না।

৩০.১০ RHEL পরিবার: `dnf` ও `yum`

Fedora, RHEL, CentOS Stream, Rocky Linux, AlmaLinux, Oracle Linux — এখানে high-level টুল **`dnf`** (Dandified YUM)।

- **yum** (Yellowdog Updater, Modified) ছিল পুরনো টুল (RHEL 7 পর্যন্ত)।
- RHEL 8 থেকে `dnf` ডিফল্ট; `yum` কমান্ডটা এখন শুধু `dnf` -এর একটা symlink। তাই RHEL 9-এ `yum install` আর `dnf install` একই কাজ করে।
- Fedora 41 থেকে এসেছে **dnf5** — C++ এ নতুন করে লেখা, অনেক দ্রুত। বেশিরভাগ কমান্ড একই, তবে কিছু plugin কমান্ডের syntax বদলেছে (নিচে উল্লেখ করা আছে)।

দৈনন্দিন dnf কমান্ড

কাজ	কমান্ড	Debian-এর সমতুল্য
index রিফ্রেশ	`dnf makecache` (সাধারণত দরকার নেই)	`apt update`

কাজ	কমান্ড	Debian-এর সমতুল্য
আপডেট আছে কিনা দেখো	`dnf check-update`	`apt list --upgradable`
সব আপডেট	`sudo dnf upgrade` (`update` একই)	`apt update && apt upgrade`
ইনস্টল	`sudo dnf install pkg`	`apt install pkg`
মোছো	`sudo dnf remove pkg`	`apt remove pkg`
অনাথ dependency মোছো	`sudo dnf autoremove`	`apt autoremove`
খোঁজো	`dnf search word`	`apt search word`
তথ্য	`dnf info pkg`	`apt show pkg`
কোন প্যাকেজে ফাইলটা আছে	`dnf provides /path/or/cmd`	`apt-file search`
তালিকা	`dnf list installed`	`apt list --installed`
repo তালিকা	`dnf repolist`	`apt policy`
ইতিহাস	`dnf history`	`/var/log/apt/history.log`
ক্যাশ মোছো	`sudo dnf clean all`	`apt clean`
প্যাকেজ গ্রুপ	`sudo dnf group install "Development Tools"`	(meta-package যেমন `build-essential`)

> 📝 **মনে রাখো:** `dnf` মেটাডেটা নিজেই ক্যাশ থেকে মেয়াদ দেখে রিফ্রেশ করে, তাই `apt update` এর মতো আলাদা ধাপ সাধারণত লাগে না। আরেকটা পার্থক্য: dnf-এ `remove` করলে কনফিগ ফাইলের আলাদা purge ধারণা নেই — বদলানো কনফিগ ফাইল `.rpmsave` নামে রেখে দেয়।

ইনস্টল উদাহরণ

```
$ sudo dnf install htop
Updating and loading repositories:
Repositories loaded.
Package                 Arch    Version          Repository      Size
Installing:
 htop                   x86_64  3.3.0-3.fc41     fedora      471.6 KiB

Transaction Summary:
 Installing:         1 package

Total size of inbound packages is 205 KiB. Need to download 205 KiB.
After this operation, 472 KiB extra will be used (install 472 KiB, remove 0 B).
Is this ok [y/N]: y
[1/3] htop-0:3.3.0-3.fc41.x86_64       100% | 1.2 MiB/s | 205.3 KiB | 00m00s
...
Complete!
```

(output-এর চেহারা dnf4 আর dnf5-এ কিছুটা আলাদা; উপরেরটা Fedora 41-এর dnf5 ধাঁচের।)

লক্ষ করো: dnf ডিফল্টে `[y/N]` — মানে Enter চাপলে **না**। apt-এ `[Y/n]` — Enter মানে হ্যাঁ।

`dnf search`, `dnf info`, `dnf provides`

```
$ dnf search nginx
Matched fields: name, summary
 nginx.x86_64: A high performance web server and reverse proxy server
 ...

$ dnf info nginx
Name         : nginx
Epoch        : 2
Version      : 1.26.2
Release      : 1.fc41
Architecture : x86_64
Repository   : updates
Summary      : A high performance web server and reverse proxy server
License      : BSD-2-Clause
...

$ dnf provides /usr/bin/dig
bind-utils-32:9.18.28-1.fc41.x86_64 : Utilities for querying DNS name servers
Repo         : fedora
Matched From :
Filename     : /usr/bin/dig

$ dnf provides '*/ifconfig'      # wildcard দিয়ে
```

`dnf provides` এর বড় সুবিধা: ফাইলটা তোমার সিস্টেমে না থাকলেও repo-র ভেতরে খুঁজে দেয় (Debian-এ এর জন্য `apt-file` আলাদা লাগে)।

`dnf history` — টাইম মেশিন

dnf প্রতিটা transaction (লেনদেন — একবারে করা ইনস্টল/আপডেট) নম্বর দিয়ে মনে রাখে, আর চাইলে **undo** করা যায়। এটা dnf-এর একটা দারুণ বৈশিষ্ট্য।

```
$ sudo dnf history list
ID | Command line             | Date and time    | Action(s) | Altered
---+--------------------------+------------------+-----------+--------
 15| install htop             | 2026-11-12 10:20 | Install   |       1
 14| upgrade                  | 2026-11-11 21:03 | Upgrade   |      37
...

$ sudo dnf history info 15     # ১৫ নম্বরে কী হয়েছিল
$ sudo dnf history undo 15     # ১৫ নম্বর উল্টে দাও (htop মুছে যাবে)
$ sudo dnf history rollback 13 # ১৩ নম্বরের পরের সব পরিবর্তন উল্টে দাও
```

> ⚠️ **সতর্কতা:** undo/rollback কাজ করে যদি পুরনো version এখনো repo-তে থাকে। অনেক সময় repo থেকে পুরনো version সরিয়ে ফেলা হয়, তখন undo ব্যর্থ হয়।

অন্যান্য দরকারি dnf কমান্ড

```
$ sudo dnf reinstall pkg
$ sudo dnf downgrade pkg
$ sudo dnf install ./local-file.rpm        # লোকাল rpm, dependency সহ
$ dnf repoquery --requires pkg              # dependency তালিকা
$ dnf repoquery --whatrequires pkg          # reverse dependency
$ dnf list --upgrades                       # যাদের আপডেট আছে
$ sudo dnf upgrade --security               # শুধু নিরাপত্তা আপডেট (RHEL-এ বিশেষ উপযোগী)
$ dnf updateinfo list                       # কোন advisory আছে (dnf5-এ `dnf advisory
list`)
```

version আটকে রাখা (dnf-এ hold)

Debian-এর `apt-mark hold` এর সমতুল্য হলো **versionlock** plugin:

```
# dnf4 (RHEL 9)
$ sudo dnf install python3-dnf-plugin-versionlock
$ sudo dnf versionlock add docker-ce
$ sudo dnf versionlock list
$ sudo dnf versionlock delete docker-ce

# অথবা /etc/dnf/dnf.conf এ স্থায়ীভাবে বাদ দেওয়া:
# exclude=kernel*
```

dnf5 (Fedora 41+) এ versionlock মূল টুলেই আছে: `sudo dnf versionlock add pkg`।

dnf কনফিগ ও repo ফাইল

মূল কনফিগ: `/etc/dnf/dnf.conf`

```
[main]
gpgcheck=True
installonly_limit=3
clean_requirements_on_remove=True
best=True
skip_if_unavailable=False
# max_parallel_downloads=10
# fastestmirror=True
```

`installonly_limit=3` মানে সর্বোচ্চ ৩টা kernel version রাখবে (kernel আপডেট হলে পুরনোটা মোছে না, কারণ নতুনটা বুট না হলে ফেরত যাওয়া যায় — অধ্যায় ২৭ দেখো)।

repo-গুলো থাকে `/etc/yum.repos.d/*.repo` এ (নাম পুরনো yum-এর হলেও dnf এটাই পড়ে):

```
# /etc/yum.repos.d/docker-ce.repo
[docker-ce-stable]
name=Docker CE Stable - $basearch
baseurl=https://download.docker.com/linux/fedora/$releasever/$basearch/stable
enabled=1
gpgcheck=1
gpgkey=https://download.docker.com/linux/fedora/gpg
```

ফিল্ড	মানে
`[docker-ce-stable]`	repo ID (অনন্য নাম)
`baseurl` / `metalink` / `mirrorlist`	কোথা থেকে আনবে
`$releasever`, `$basearch`	ভেরিয়েবল — যেমন `41`, `x86_64`
`enabled=1`	চালু
`gpgcheck=1`	প্যাকেজের সিগনেচার যাচাই করবে
`gpgkey`	public key কোথায়

repo যোগ করা (`dnf-plugins-core` লাগে, যা সাধারণত আগেই থাকে):

```
# dnf4 (RHEL 9, Rocky 9, Alma 9)
$ sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
$ sudo dnf config-manager --set-disabled docker-ce-stable

# dnf5 (Fedora 41+)
$ sudo dnf config-manager addrepo --from-repofile=https://download.docker.com/linux/fedora/docker-ce.repo
$ sudo dnf config-manager setopt docker-ce-stable.enabled=0
```

এককালীন কোনো repo চালু/বন্ধ করে কমান্ড চালাতে:

```
$ sudo dnf --enablerepo=updates-testing upgrade pkg
$ sudo dnf --disablerepo='*' --enablerepo=baseos install pkg
```

RHEL-বিশেষ: subscription, EPEL, AppStream

- **RHEL** এ repo পেতে Red Hat subscription register করতে হয়: `sudo subscription-manager register`। (Rocky/Alma তে লাগে না।)
- RHEL 9-এ মূল দুটো repo: **BaseOS** (মূল OS) আর **AppStream** (অ্যাপ্লিকেশন, ভাষা, ডাটাবেস)।
- **EPEL** (Extra Packages for Enterprise Linux) — Fedora প্রকল্পের বানানো অতিরিক্ত প্যাকেজ। RHEL-এ প্রথমে CodeReady Builder (Rocky/Alma-তে `crb`) চালু করা লাগে:

  ```
  $ sudo dnf config-manager --set-enabled crb      # Rocky/Alma 9
  $ sudo dnf install epel-release
  ```

- Fedora-তে **RPM Fusion** হলো জনপ্রিয় থার্ড-পার্টি repo (মিডিয়া কোডেক, NVIDIA ড্রাইভার)।

dnf-এ GPG

RPM-জগতে **প্রতিটা প্যাকেজ ফাইল আলাদাভাবে সাইন করা** থাকে (repo metadata ছাড়াও)। `gpgcheck=1` থাকলে dnf ইনস্টলের আগে প্রতিটা `.rpm` এর সিগনেচার দেখে। নতুন repo-র key প্রথমবার ব্যবহারের সময় dnf তোমাকে জিজ্ঞেস করবে:

```
Importing GPG key 0x621E9F35:
 Userid     : "Docker Release (CE rpm) <docker@docker.com>"
 Fingerprint: 060A 61C5 1B55 8A7F 742B 77AA C52F EB6B 621E 9F35
 From       : https://download.docker.com/linux/fedora/gpg
Is this ok [y/N]:
```

fingerprint অফিসিয়াল সাইটের সাথে মিলিয়ে তারপর `y` দাও।

৩০.১১ `rpm` — RHEL-এর low-level টুল

`rpm` হলো Debian-এর `dpkg` এর সমতুল্য। সবচেয়ে বেশি ব্যবহার হয় **query** (`-q`) মোডে।

query অপশন

কমান্ড	কাজ	dpkg সমতুল্য
`rpm -qa`	সব ইনস্টল করা প্যাকেজ (query all)	`dpkg -l`
`rpm -q pkg`	প্যাকেজটা আছে কি না, কোন version	`dpkg -s pkg`
`rpm -qi pkg`	বিস্তারিত তথ্য (info)	`dpkg -s pkg`
`rpm -ql pkg`	কোন কোন ফাইল (list)	`dpkg -L pkg`
`rpm -qf /path`	ফাইলটা কোন প্যাকেজের (file)	`dpkg -S /path`
`rpm -qc pkg`	শুধু কনফিগ ফাইল	—
`rpm -qd pkg`	শুধু ডকুমেন্টেশন ফাইল	—
`rpm -qR pkg`	dependency (requires)	`apt-cache depends`
`rpm -q --changelog pkg`	changelog	`apt changelog`
`rpm -qpi file.rpm`	ইনস্টল না করা `.rpm` ফাইলের তথ্য (`p` = package file)	`dpkg -I file.deb`
`rpm -qpl file.rpm`	`.rpm` ফাইলের ভেতরের তালিকা	`dpkg -c file.deb`
`rpm -qa --last`	সাম্প্রতিক ইনস্টল আগে	—

```
$ rpm -qa | grep -i openssh
openssh-9.6p1-1.fc41.x86_64
openssh-clients-9.6p1-1.fc41.x86_64
openssh-server-9.6p1-1.fc41.x86_64
```

```
$ rpm -qf /usr/bin/ls
coreutils-9.5-8.fc41.x86_64

$ rpm -ql htop
/usr/bin/htop
/usr/share/applications/htop.desktop
/usr/share/doc/htop
/usr/share/doc/htop/README
/usr/share/man/man1/htop.1.gz
...

$ rpm -qi bash
Name        : bash
Version     : 5.2.32
Release     : 1.fc41
Architecture: x86_64
Install Date: Mon 04 Nov 2026 09:12:41 AM +06
Group       : Unspecified
Size        : 8592513
License     : GPL-3.0-or-later
Signature   : RSA/SHA256, ... Key ID ...
Source RPM  : bash-5.2.32-1.fc41.src.rpm
...
```

install/upgrade/erase

কমান্ড	কাজ
`sudo rpm -ivh file.rpm`	install, verbose, hash (#) প্রোগ্রেস বার
`sudo rpm -Uvh file.rpm`	upgrade (না থাকলে install)
`sudo rpm -Fvh file.rpm`	freshen — শুধু আগে থেকে থাকলে আপগ্রেড
`sudo rpm -e pkg`	erase (মোছো)
`rpm -V pkg`	verify — ইনস্টলের পর ফাইল বদলেছে কিনা
`rpm -K file.rpm` (`--checksig`)	`.rpm` ফাইলের সিগনেচার যাচাই
`sudo rpm --import key.asc`	GPG public key যোগ করো

```
$ rpm -V openssh-server
S.5....T.  c /etc/ssh/sshd_config
```

এখানে `S` = আকার বদলেছে, `5` = checksum বদলেছে, `T` = সময় বদলেছে, `c` = এটা কনফিগ ফাইল। মানে কেউ `sshd_config` এডিট করেছে — স্বাভাবিক। কিন্তু `/usr/bin/` এর কোনো binary এভাবে বদলালে সন্দেহজনক (হ্যাকিং-এর লক্ষণ হতে পারে)! Debian-এ সমতুল্য: `dpkg --verify pkg` বা `debsums` প্যাকেজ।

> ⚠️ **সতর্কতা:** `rpm -ivh` dependency নিজে আনে না, ঠিক `dpkg -i` এর মতো। আর `rpm -e --nodeps` দিয়ে জোর করে মুছলে সিস্টেম ভেঙে যেতে পারে। দৈনন্দিন কাজে `dnf install ./file.rpm`

> ব্যবহার করো।

RPM ডাটাবেস থাকে `/var/lib/rpm/` এ (Fedora 36+ এ আসলে `/usr/lib/sysimage/rpm/`, আর `/var/lib/rpm` একটা symlink)। RHEL 9 ও Fedora-তে এটা একটা SQLite ডাটাবেস।

৩০.১২ Arch পরিবার: `pacman` ও AUR

Arch Linux, Manjaro, EndeavourOS — এখানে `pacman` একাই low-level ও high-level দুই কাজ করে। Arch হলো **rolling release** (রোলিং রিলিজ — "Arch 24.04" জাতীয় নির্দিষ্ট version নেই; প্রতিনিয়ত সব সর্বশেষ version পায়)।

pacman-এর syntax প্রথমে অদ্ভুত লাগে: একটা বড় হাতের **operation** অক্ষর, তারপর ছোট হাতের **modifier**।

operation	মানে
`-S`	Sync — repo থেকে কাজ (ইনস্টল, খোঁজা)
`-R`	Remove
`-Q`	Query — লোকাল ডাটাবেস
`-U`	Upgrade — লোকাল প্যাকেজ ফাইল থেকে ইনস্টল
`-F`	Files — repo-র ফাইল তালিকায় খোঁজা

সবচেয়ে ব্যবহৃত pacman কমান্ড

কমান্ড	কাজ	apt সমতুল্য
`sudo pacman -Syu`	index রিফ্রেশ + পুরো সিস্টেম আপগ্রেড	`apt update && apt full-upgrade`
`sudo pacman -S pkg`	ইনস্টল	`apt install`
`sudo pacman -Syu pkg`	আপগ্রেড করে তারপর ইনস্টল (নিরাপদ পদ্ধতি)	—
`pacman -Ss word`	repo-তে খোঁজো	`apt search`
`pacman -Si pkg`	repo-র প্যাকেজের তথ্য	`apt show`
`sudo pacman -R pkg`	শুধু প্যাকেজটা মোছো	`apt remove`
`sudo pacman -Rs pkg`	সাথে অপ্রয়োজনীয় dependency-ও	`apt remove` + `autoremove`
`sudo pacman -Rns pkg`	সাথে কনফিগ ব্যাকআপও (`.pacsave`) না রেখে	`apt purge` + `autoremove`
`pacman -Q`	সব ইনস্টল করা প্যাকেজ	`apt list --installed`
`pacman -Qe`	যেগুলো নিজে (explicitly) ইনস্টল করেছ	`apt-mark showmanual`

কমান্ড	কাজ	apt সমতুল্য
`pacman -Qi pkg`	ইনস্টল করা প্যাকেজের তথ্য	`dpkg -s`
`pacman -Ql pkg`	ফাইল তালিকা	`dpkg -L`
`pacman -Qo /path`	ফাইলটা কোন প্যাকেজের (owns)	`dpkg -S`
`pacman -Qdt`	অনাথ dependency	—
`pacman -F cmd`	কোন প্যাকেজে আছে (আগে `sudo pacman -Fy`)	`apt-file search`
`sudo pacman -U file.pkg.tar.zst`	লোকাল প্যাকেজ ইনস্টল	`apt install ./file.deb`
`sudo pacman -Sc`	পুরনো ক্যাশ মোছো	`apt autoclean`

```
$ sudo pacman -Syu
:: Synchronizing package databases...
 core                  117.5 KiB   580 KiB/s 00:00
 extra                   7.7 MiB  4.21 MiB/s 00:02
:: Starting full system upgrade...
resolving dependencies...
looking for conflicting packages...

Packages (3) curl-8.11.0-1  linux-6.11.8.arch1-1  openssl-3.4.0-1

Total Download Size:   140.12 MiB
Total Installed Size:  145.87 MiB
Net Upgrade Size:        0.42 MiB

:: Proceed with installation? [Y/n]
```

সব অনাথ dependency একবারে মোছা:

```
$ sudo pacman -Rns $(pacman -Qdtq)
```

(`-q` = quiet, শুধু নাম; `$( )` হলো command substitution — অধ্যায় ৪১ দেখো।)

> ⚠️ **সতর্কতা: partial upgrade নিষিদ্ধ।** Arch-এ কখনো `sudo pacman -Sy pkg` (index রিফ্রেশ কিন্তু বাকি সিস্টেম আপগ্রেড না করে নতুন প্যাকেজ) কোরো না। এতে নতুন প্যাকেজ নতুন library চায়, কিন্তু বাকি সিস্টেম পুরনো library-তে — ফলে সিস্টেম ভাঙতে পারে। নিয়ম: হয় `-S pkg` (রিফ্রেশ ছাড়া), নয়তো `-Syu pkg`।

> 💡 **টিপস:** Arch-এ আপগ্রেডের আগে archlinux.org এর হোমপেজের "News" দেখে নাও — কখনো কখনো হাতে কিছু করতে বলা হয়। কনফিগ বদলালে pacman নতুন ভার্সন `.pacnew` নামে রাখে; `pacdiff` (pacman-contrib প্যাকেজ) দিয়ে মেলাও।

কনফিগ: `/etc/pacman.conf` (repo, `ParallelDownloads`, `IgnorePkg` = hold-এর সমতুল্য)। mirror তালিকা: `/etc/pacman.d/mirrorlist` (`reflector` টুল দিয়ে দ্রুততম mirror বাছাই করা যায়)। লগ: `/var/log/pacman.log`।

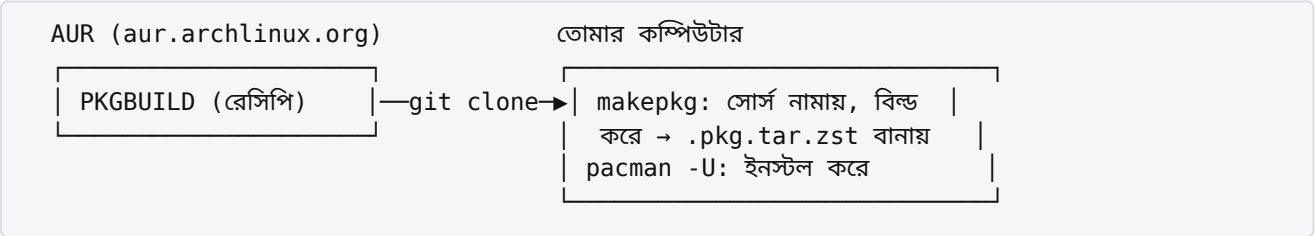
pacman-এ GPG

pacman প্রতিটা প্যাকেজের সিগনেচার যাচাই করে `pacman-key` দিয়ে পরিচালিত keyring (`/etc/pacman.d/gnupg/`) এর সাহায্যে। Arch-এর ডেভেলপারদের key আসে `archlinux-keyring` প্যাকেজে। অনেকদিন আপডেট না করলে "signature is unknown trust" error আসতে পারে, তখন:

```
$ sudo pacman -Sy archlinux-keyring && sudo pacman -Syu
```

AUR — Arch User Repository

AUR হলো কমিউনিটির একটা বিশাল সংগ্রহ, যেখানে কোনো বানানো প্যাকেজ থাকে না — থাকে শুধু **PKGBUILD** নামের একটা বিল্ড-রেসিপি (শেল স্ক্রিপ্ট), যা বলে সোর্স কোথা থেকে নামাতে হবে আর কীভাবে বিল্ড করতে হবে।



ম্যানুয়াল পদ্ধতি:

```
$ sudo pacman -S --needed base-devel git
$ git clone https://aur.archlinux.org/yay.git
$ cd yay
$ less PKGBUILD          # ← অবশ্যই পড়ো!
$ makepkg -si            # -s: dependency ইনস্টল, -i: বিল্ডের পর ইনস্টল
```

`makepkg` কে কখনো root হিসেবে চালাবে না — এটা নিজেই নিষেধ করে।

AUR helper (যেমন `yay`, `paru`) এই পুরো কাজ স্বয়ংক্রিয় করে, আর pacman-এর মতো syntax দেয়:

```
$ yay -S google-chrome     # AUR থেকে
$ yay -Syu                 # অফিসিয়াল repo + AUR দুটোই আপগ্রেড
$ paru -Ss visual-studio-code
```

> ⚠️ **সতর্কতা:** AUR **অফিসিয়ালি সাপোর্টেড নয়**, যে কেউ প্যাকেজ আপলোড করতে পারে। অতীতে ম্যালওয়্যারযুক্ত PKGBUILD পাওয়া গেছে। প্রতিবার PKGBUILD পড়ো — বিশেষ করে `source=` লাইন আর `curl` / `wget` দিয়ে অন্য কিছু নামানো হচ্ছে কিনা। ভোট সংখ্যা আর কমেন্ট দেখো। AUR helper অফিসিয়াল repo-তে নেই, এটা প্রথমবার হাতে বিল্ড করতে হয়।

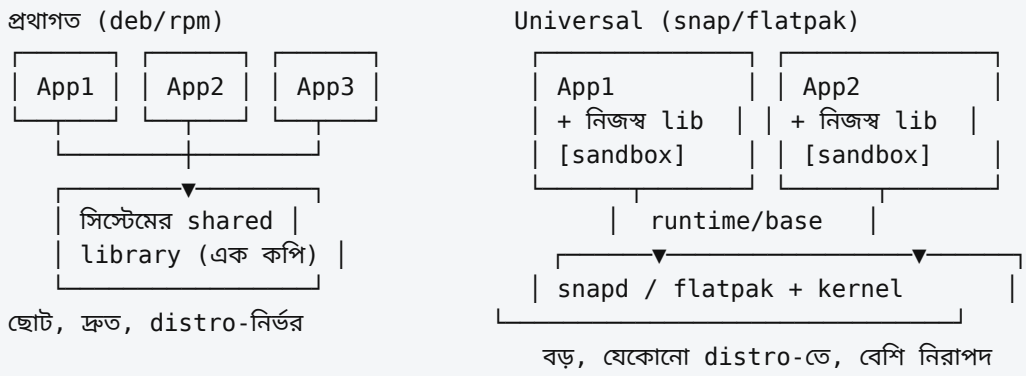
সোর্স থেকে বিল্ড সম্পর্কে বিস্তারিত অধ্যায় ৩১-এ।

অন্য distro-র package manager এক নজরে

distro	high-level	low-level
openSUSE	`zypper` (`zypper in`, `zypper up`, `zypper dup`)	`rpm`
Alpine (Docker-এ জনপ্রিয়)	`apk` (`apk add`, `apk del`, `apk update`)	`apk`
Gentoo	`emerge` (সোর্স থেকে বিল্ড)	—
Void	`xbps-install`	—
NixOS	`nix` (declarative, একদম আলাদা দর্শন)	—

৩০.১৩ Distro-নিরপেক্ষ প্যাকেজ: snap, flatpak, AppImage

উপরের সব প্যাকেজ নির্দিষ্ট distro-র জন্য বানানো। Ubuntu 24.04-এর `.deb` Fedora-তে চলবে না, এমনকি Ubuntu 20.04-এ ঠিকঠাক না-ও চলতে পারে (library version আলাদা)। ডেভেলপারের জন্য ১০টা distro-র জন্য আলাদা প্যাকেজ বানানো যন্ত্রণা। সমাধান: **universal package** — অ্যাপ নিজের সব dependency সাথে নিয়ে আসে, আর সিস্টেম থেকে কিছুটা আলাদা (sandbox — বালির বাক্সের মতো আলাদা ঘেরা জায়গা) পরিবেশে চলে।



snap (Canonical)

Ubuntu-তে ডিফল্টে চালু। ব্যাকগ্রাউন্ডে `snapd` ডেমন চলে। প্রতিটা snap একটা compressed SquashFS ইমেজ, যা `/snap/<name>/<revision>/` এ **mount** হয় (তাই `lsblk` বা `df` এ অনেক `loop` ডিভাইস দেখো — অধ্যায় ৩২)। Store একটাই: Snap Store (Canonical নিয়ন্ত্রিত)।

```
$ snap find vlc
Name  Version  Publisher   Notes  Summary
vlc   3.0.20   videolan✓   -      The ultimate media player

$ sudo snap install vlc
vlc 3.0.20 from VideoLAN✓ installed
```

```
$ snap list
Name     Version   Rev    Tracking         Publisher   Notes
core22   20240823  1621   latest/stable    canonical✓  base
firefox  132.0.1   5273   latest/stable/…  mozilla✓    -
vlc      3.0.20    3777   latest/stable    videolan✓   -

$ sudo snap refresh            # সব snap আপডেট (এমনিতে দিনে ৪ বার নিজেই চেক করে)
$ sudo snap remove vlc
$ snap info vlc                # চ্যানেল (stable/candidate/beta/edge) দেখাও
$ sudo snap install code --classic   # --classic = sandbox ছাড়া পূর্ণ অ্যাক্সেস (IDE-র দরকার
হয়)
$ sudo snap revert vlc         # আগের revision-এ ফেরত যাও
$ snap connections vlc         # কোন অনুমতি (interface) আছে
```

> 📝 **মনে রাখো:** Ubuntu 22.04 থেকে `sudo apt install firefox` দিলে আসলে একটা transitional `.deb` ইনস্টল হয়, যা snap থেকে Firefox ইনস্টল করে। Ubuntu-তে Chromium-ও একইভাবে snap।

flatpak

Fedora, Linux Mint সহ বেশিরভাগ ডেস্কটপ distro-তে জনপ্রিয়, কমিউনিটি-চালিত। মূল store **Flathub**। অ্যাপগুলো শেয়ার করা **runtime** (যেমন `org.gnome.Platform`) ব্যবহার করে, তাই প্রথম অ্যাপ বড় লাগলেও পরেরগুলো ছোট। sandbox হয় bubblewrap আর kernel namespace দিয়ে (অধ্যায় ৪৮)।

```
$ sudo apt install flatpak                  # Ubuntu/Debian (Fedora-তে আগেই থাকে)
$ flatpak remote-add --if-not-exists flathub
https://dl.flathub.org/repo/flathub.flatpakrepo

$ flatpak search gimp
Name   Description                Application ID     Version  Branch  Remotes
GIMP   Create images and edit...  org.gimp.GIMP      2.10.38  stable  flathub

$ flatpak install flathub org.gimp.GIMP
$ flatpak run org.gimp.GIMP
$ flatpak list
$ flatpak update
$ flatpak uninstall org.gimp.GIMP
$ flatpak uninstall --unused                # অব্যবহৃত runtime মোছো
$ flatpak info --show-permissions org.gimp.GIMP
```

> 💡 **টিপস:** flatpak অ্যাপের অনুমতি (কোন ফোল্ডার দেখতে পারবে ইত্যাদি) GUI দিয়ে বদলাতে **Flatseal** অ্যাপটা ব্যবহার করো (এটাও Flathub-এ আছে)। CLI-তে `flatpak override --user --filesystem=~/Documents org.gimp.GIMP`।

Fedora-তে বাইরের flatpak repo ডিফল্টে সীমিত থাকতে পারে; পুরো Flathub চালু করতে উপরের `remote-add` কমান্ড দাও।

AppImage

সবচেয়ে সরল: **একটা ফাইল = একটা অ্যাপ**। কোনো ইনস্টল নেই, কোনো ডেমন নেই, root লাগে না।

```
$ chmod +x Obsidian-1.7.5.AppImage    # execute পারমিশন দাও (অধ্যায় ১৬)
$ ./Obsidian-1.7.5.AppImage
```

ভেতরে এটা একটা ELF executable + SquashFS ইমেজ। চালালে নিজেকে FUSE (Filesystem in Userspace) দিয়ে অস্থায়ীভাবে mount করে চালায়।

> ⚠️ **সতর্কতা:** অনেক AppImage-এর পুরনো `libfuse2` লাগে। Ubuntu 24.04-এ: `sudo apt install libfuse2t64` (22.04-এ নাম ছিল `libfuse2`)। `fuse` প্যাকেজ ইনস্টল কোরো না — এতে Ubuntu-তে `fuse3` সরে যেতে পারে আর ডেস্কটপের অংশ ভাঙতে পারে।

AppImage-এর অসুবিধা: স্বয়ংক্রিয় আপডেট নেই (কিছু AppImage নিজে করে), sandbox নেই, মেনুতে আইকন আসে না (Gear Lever-এর মতো টুল এটা ঠিক করে)। আর যেকোনো সাইট থেকে নামানো ফাইল চালাচ্ছ — উৎস বিশ্বস্ত হতে হবে।

তুলনা

বৈশিষ্ট্য	deb/rpm	snap	flatpak	AppImage
কে বানিয়েছে	distro	Canonical	কমিউনিটি (freedesktop)	কমিউনিটি
store	distro repo	Snap Store (একটাই)	Flathub + যেকোনো remote	নেই (ওয়েবসাইট)
sandbox	না	হ্যাঁ (AppArmor)	হ্যাঁ (bubblewrap)	না
স্বয়ংক্রিয় আপডেট	distro নিয়মে	হ্যাঁ, ডিফল্টে	`flatpak update` / GUI	সাধারণত না
CLI/সার্ভার টুলের জন্য	সেরা	হ্যাঁ	মূলত GUI অ্যাপের জন্য	কিছুটা
ডিস্ক খরচ	সবচেয়ে কম	বেশি	বেশি (runtime শেয়ার হয়)	মাঝারি
root লাগে	হ্যাঁ	হ্যাঁ	না (`--user` দিয়ে)	না
প্রথম চালু হওয়ার গতি	দ্রুত	কিছুটা ধীর	দ্রুত	মাঝারি

> 💡 **কোনটা কখন?** সার্ভার, কমান্ড-লাইন টুল, সিস্টেম কম্পোনেন্ট → সবসময় distro-র deb/rpm। ডেস্কটপ GUI অ্যাপের নতুন version → flatpak (বা Ubuntu-তে snap)। একবার চালিয়ে দেখার জন্য বা পোর্টেবল অ্যাপ → AppImage।

ভাষা-নির্দিষ্ট package manager-ও আছে: Python-এর `pip`, Node-এর `npm`, Rust-এর `cargo`। এগুলো দিয়ে সিস্টেম-ব্যাপী (`sudo pip install`) ইনস্টল কোরো না — Ubuntu 24.04 / Debian 12-এ `pip`

`install` সিস্টেম Python-এ এখন "externally-managed-environment" error দেয় (PEP 668)। এর বদলে `python3 -m venv` দিয়ে virtual environment, বা `pipx` ব্যবহার করো।

৩০.১৪ GUI সফটওয়্যার সেন্টার

টার্মিনাল ছাড়াও:

- **Ubuntu 24.04:** "App Center" (snap ও deb দুটোই দেখায়)
- **GNOME (Fedora, Debian):** "GNOME Software"
- **KDE:** "Discover"

এগুলো ভেতরে **PackageKit** নামের একটা মধ্যস্থ স্তর ব্যবহার করে, যা apt/dnf/zypper এর সাথে কথা বলে। `pkcon` হলো তার CLI। মূল কথা: GUI আর CLI একই ডাটাবেস নিয়ে কাজ করে, তাই GUI-তে ইনস্টল করা জিনিস `apt list --installed` এ দেখা যাবে। ডেস্কটপ পরিবেশ নিয়ে আরও অধ্যায় ৪-এ।

৩০.১৫ সমস্যা সমাধান (troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	সমাধান
`E: Unable to locate package xyz`	`apt update` করোনি, বা নাম ভুল, বা `universe` বন্ধ	`sudo apt update`; `apt search xyz`; `sudo add-apt-repository universe`
`Could not get lock /var/lib/dpkg/lock-frontend`	অন্য apt/dpkg চলছে	অপেক্ষা করো; `ps aux | grep apt`
`dpkg was interrupted, you must manually run 'sudo dpkg --configure -a'`	আগের ইনস্টল মাঝপথে থেমেছিল	বার্তাতে যা বলছে তাই চালাও
`unmet dependencies` / `held broken packages`	ভুল repo/PPA, mixed রিলিজ	`sudo apt --fix-broken install`; `apt policy pkg` দিয়ে উৎস দেখো; সমস্যার PPA সরাও
`NO_PUBKEY`	repo-র GPG key নেই	অফিসিয়াল নির্দেশনা মতো `/etc/apt/keyrings/` এ key রাখো, `signed-by` দাও
`Release file ... is not valid yet`	কম্পিউটারের ঘড়ি ভুল	`timedatectl` দিয়ে সময় ঠিক করো
`404 Not Found` (পুরনো Ubuntu)	রিলিজের সাপোর্ট শেষ, repo `old-releases` এ সরেছে	রিলিজ আপগ্রেড করো

লক্ষণ	সম্ভাব্য কারণ	সমাধান
`/boot` ভরে গেছে	পুরনো kernel জমেছে	`sudo apt autoremove --purge`
dnf: `Error: GPG check FAILED`	key import হয়নি বা প্যাকেজ বদলানো	`rpm --import` ; উৎস যাচাই করো
pacman: `conflicting files`	ফাইল অন্য কোথাও থেকে বসানো	`pacman -Qo file` দিয়ে মালিক দেখো, Arch Wiki দেখো

> ⚠️ **সতর্কতা:** কোনো ফোরামে দেখা `sudo dpkg --force-all ...` বা `rpm --nodeps --force` জাতীয় কমান্ড না বুঝে চালাবে না। এগুলো ডাটাবেসকে মিথ্যা বলতে বাধ্য করে, পরে আরও বড় সমস্যা হয়।

হাতে-কলমে অনুশীলন

> নিচের ল্যাবগুলো Ubuntu 24.04 বা Debian 12-এর জন্য। Fedora থাকলে বন্ধনীর কমান্ড ব্যবহার করো। ভার্চুয়াল মেশিনে (অধ্যায় ৩) করাই সবচেয়ে নিরাপদ।

ল্যাব ১: প্রথম ইনস্টল

```
$ sudo apt update                       # (Fedora: dnf check-update)
$ apt list --upgradable
$ sudo apt upgrade -y                   # (sudo dnf upgrade -y)
$ apt show cowsay                       # (dnf info cowsay)
$ sudo apt install cowsay               # (sudo dnf install cowsay)
$ cowsay "Ami Linux shikhchi"
 ____________________
< Ami Linux shikhchi >
 --------------------
        \   ^__^
         \  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

Debian/Ubuntu-তে `cowsay` থাকে `/usr/games/` এ — PATH-এ না থাকলে `/usr/games/cowsay` লিখে চালাও।

ল্যাব ২: গোয়েন্দাগিরি

1. `ls` কমান্ড কোন প্যাকেজের? `dpkg -S $(which ls)` (Fedora: `rpm -qf $(which ls)`)
2. `cowsay` প্যাকেজ কোন কোন ফাইল বসিয়েছে? `dpkg -L cowsay` (`rpm -ql cowsay`)
3. মোট কতগুলো প্যাকেজ ইনস্টল আছে? `dpkg -l | grep -c '^ii'` (`rpm -qa | wc -l`)

4. `cowsay` এর dependency কী? `apt-cache depends cowsay` (`dnf repoquery --requires cowsay`)
5. সর্বশেষ কী ইনস্টল করেছ? `tail -20 /var/log/apt/history.log` (`dnf history list`)

ল্যাব ৩: remove বনাম purge

```
$ sudo apt install nginx
$ ls /etc/nginx/                      # কনফিগ আছে
$ sudo apt remove nginx
$ dpkg -l nginx nginx-common          # 'rc' অবস্থা দেখো
$ ls /etc/nginx/                      # এখনো আছে!
$ sudo apt purge nginx nginx-common
$ ls /etc/nginx/                      # এবার নেই (বা খালি)
$ sudo apt autoremove                 # অনাথ dependency
```

ল্যাব ৪: hold

```
$ sudo apt-mark hold cowsay
$ apt-mark showhold
$ dpkg -l cowsay                      # 'hi'
$ sudo apt-mark unhold cowsay
```

ল্যাব ৫: লোকাল .deb ও ভাঙা dependency

```
$ mkdir ~/pkglab && cd ~/pkglab
$ apt download htop
$ dpkg-deb --info htop_*.deb | grep Depends
$ dpkg-deb --contents htop_*.deb | head
$ sudo apt install ./htop_*.deb       # ./ লক্ষ করো
```

ল্যাব ৬: থার্ড-পার্টি repo নিরাপদে যোগ করা

Docker-এর অফিসিয়াল নির্দেশনা (docs.docker.com) খোলো, আর ৩০.৯ অংশের মতো `signed-by` সহ repo যোগ করো। তারপর:

```
$ apt policy docker-ce                # কোন repo থেকে আসছে দেখো
$ gpg --show-keys /etc/apt/keyrings/docker.asc   # fingerprint মিলাও
```

শেষে পরিষ্কার: `/etc/apt/sources.list.d/docker.list` ও key ফাইল মুছে `sudo apt update`।

ল্যাব ৭: universal package

```
$ snap list                           # Ubuntu-তে আগে থেকেই কী আছে
$ flatpak --version || sudo apt install flatpak
$ flatpak remote-add --if-not-exists flathub
https://dl.flathub.org/repo/flathub.flatpakrepo
```

```
$ flatpak search calculator
$ lsblk | grep loop                  # snap-এর mount দেখো
```

চ্যালেঞ্জ

একটা স্ক্রিপ্ট লেখো (অধ্যায় ৪১-এর পরে আবার ফিরে এসো) যা `/etc/os-release` পড়ে distro বুঝবে, আর সঠিক কমান্ড দিয়ে `curl git htop` ইনস্টল করবে — `apt-get`, `dnf`, বা `pacman` যেটা প্রযোজ্য।

সারসংক্ষেপ

তিন পরিবারের তুলনামূলক চিটশিট

কাজ	Debian/Ubuntu	Fedora/RHEL	Arch
index রিফ্রেশ	`sudo apt update`	`dnf makecache`	`sudo pacman -Sy` (একা নয়!)
সব আপগ্রেড	`sudo apt upgrade` / `full-upgrade`	`sudo dnf upgrade`	`sudo pacman -Syu`
ইনস্টল	`sudo apt install pkg`	`sudo dnf install pkg`	`sudo pacman -S pkg`
লোকাল ফাইল ইনস্টল	`sudo apt install ./f.deb`	`sudo dnf install ./f.rpm`	`sudo pacman -U f.pkg.tar.zst`
মোছো	`sudo apt remove pkg`	`sudo dnf remove pkg`	`sudo pacman -R pkg`
কনফিগ সহ মোছো	`sudo apt purge pkg`	(প্রযোজ্য নয়)	`sudo pacman -Rns pkg`
অনাথ মোছো	`sudo apt autoremove`	`sudo dnf autoremove`	`sudo pacman -Rns $(pacman -Qdtq)`
খোঁজো	`apt search word`	`dnf search word`	`pacman -Ss word`
তথ্য	`apt show pkg`	`dnf info pkg`	`pacman -Si pkg`
ইনস্টল তালিকা	`apt list --installed` / `dpkg -l`	`dnf list installed` / `rpm -qa`	`pacman -Q`
ফাইলের তালিকা	`dpkg -L pkg`	`rpm -ql pkg`	`pacman -Ql pkg`
ফাইলটা কার	`dpkg -S /path`	`rpm -qf /path`	`pacman -Qo /path`
না-থাকা ফাইল খোঁজো	`apt-file search f`	`dnf provides f`	`pacman -F f`
version উৎস	`apt policy pkg`	`dnf list --showduplicates pkg`	`pacman -Si pkg`
আপডেট আটকাও	`apt-mark hold pkg`	`dnf versionlock add pkg`	`IgnorePkg` in `pacman.conf`

কাজ	Debian/Ubuntu	Fedora/RHEL	Arch
ইতিহাস	`/var/log/apt/history.log`	`dnf history`	`/var/log/pacman.log`
ক্যাশ মোছো	`sudo apt clean`	`sudo dnf clean all`	`sudo pacman -Sc`
repo কনফিগ	`/etc/apt/sources.list(.d/)`	`/etc/yum.repos.d/*.repo`	`/etc/pacman.conf`
ভাঙা মেরামত	`dpkg --configure -a`; `apt -f install`	`dnf distro-sync`	`pacman -Syu`

universal package

কাজ	snap	flatpak	AppImage
খোঁজো	`snap find x`	`flatpak search x`	ওয়েবসাইট
ইনস্টল	`sudo snap install x`	`flatpak install flathub id`	`chmod +x f.AppImage`
চালাও	`x`	`flatpak run id`	`./f.AppImage`
তালিকা	`snap list`	`flatpak list`	—
আপডেট	`sudo snap refresh`	`flatpak update`	নতুন ফাইল নামাও
মোছো	`sudo snap remove x`	`flatpak uninstall id`	`rm f.AppImage`

কীবোর্ড শর্টকাট

প্যাকেজ ম্যানেজমেন্টের নিজস্ব কোনো শর্টকাট নেই, তবে কাজের সময় এগুলো লাগে (অধ্যায় ৬):

শর্টকাট	কাজ
`Tab` (দুইবার)	`apt install vl` + Tab Tab → প্যাকেজের নাম auto-complete (bash-completion থাকলে)
`Ctrl + C`	চলমান ডাউনলোড বাতিল (ইনস্টল-পর্বে চাপবে না!)
`Ctrl + R`	হিস্ট্রিতে আগের `apt install` কমান্ড খোঁজো
`!!` → `sudo !!`	`sudo` দিতে ভুলে গেলে আগের কমান্ড sudo দিয়ে আবার চালাও
`q`	`apt show` / `apt list` এর output pager-এ খুললে বের হও
`Y` / `Enter`	apt-এ নিশ্চিত করো (ডিফল্ট হ্যাঁ)
`y` + `Enter`	dnf-এ নিশ্চিত করো (ডিফল্ট না)

নিজেকে যাচাই করো

1. `apt update` আর `apt upgrade` এর মধ্যে পার্থক্য কী?

2. low-level আর high-level প্যাকেজ টুলের পার্থক্য একটা উপমা দিয়ে বোঝাও। Debian ও RHEL পরিবারে প্রত্যেক স্তরের একটা করে টুলের নাম বলো।
3. `apt remove` আর `apt purge` এর পার্থক্য কী? `dpkg -l` এ `rc` মানে কী?
4. `/usr/bin/curl` ফাইলটা কোন প্যাকেজের, Ubuntu, Fedora ও Arch-এ কীভাবে জানবে?
5. `apt-key add` কেন deprecated? এর বদলে কী করা উচিত?
6. `sudo dpkg -i app.deb` দিলে dependency error এলো। কী করবে? আর এই সমস্যা এড়ানোর ভালো উপায় কী ছিল?
7. Arch-এ `sudo pacman -Sy firefox` কেন বিপজ্জনক?
8. dnf-এ গতকালের একটা আপডেট উল্টাতে চাও। কোন কমান্ড?
9. snap আর flatpak-এর দুইটা মিল আর দুইটা অমিল বলো।
10. একটা প্যাকেজকে আপগ্রেড থেকে আটকে রাখতে Debian আর RHEL-এ কী করবে, আর এর ঝুঁকি কী?

▼ **উত্তর দেখো**

1. `apt update` শুধু repo থেকে প্যাকেজের সর্বশেষ **তালিকা (index)** নামিয়ে `/var/lib/apt/lists/` এ রাখে; কোনো সফটওয়্যার বদলায় না। `apt upgrade` সেই তালিকা অনুযায়ী ইনস্টল করা প্যাকেজগুলোকে নতুন version-এ আপডেট করে। তাই সাধারণত আগে update, তারপর upgrade।

2. low-level টুল (রাজমিস্ত্রি) একটা লোকাল প্যাকেজ ফাইল খুলে ফাইল বসায় ও ডাটাবেস আপডেট করে, কিন্তু ডাউনলোড বা dependency সমাধান করে না। high-level টুল (ঠিকাদার) repo থেকে খোঁজে, dependency হিসাব করে, সিগনেচার যাচাই করে, ডাউনলোড করে, তারপর low-level টুল দিয়ে ইনস্টল করায়। Debian: `dpkg` (low) ও `apt` (high); RHEL: `rpm` (low) ও `dnf` (high)।

3. `remove` প্রোগ্রামের ফাইল মোছে কিন্তু `/etc` এর কনফিগ ফাইল রেখে দেয়; `purge` কনফিগ সহ মোছে (home ডিরেক্টরির ব্যক্তিগত ফাইল কোনোটাই মোছে না)। `rc` মানে: desired = **r**emove, status = **c**onfig-files — প্যাকেজ মোছা হয়েছে, কনফিগ রয়ে গেছে।

4. Ubuntu: `dpkg -S /usr/bin/curl`; Fedora: `rpm -qf /usr/bin/curl`; Arch: `pacman -Qo /usr/bin/curl`।

5. `apt-key` key-গুলো একটা global keyring-এ রাখত, ফলে যেকোনো থার্ড-পার্টি key দিয়ে **যেকোনো** repo-র (এমনকি অফিসিয়াল) প্যাকেজ বিশ্বস্ত বলে মেনে নেওয়া হতো। এখন প্রতিটা key আলাদা ফাইলে `/etc/apt/keyrings/` এ রাখা হয়, আর repo-র লাইনে `signed-by=/etc/apt/keyrings/x.gpg` (বা deb822-এ `Signed-By:`) দিয়ে বলে দেওয়া হয় এই repo শুধু এই key দিয়ে যাচাই হবে।

6. `sudo apt --fix-broken install` (বা `sudo apt -f install`) চালাবে, যা হারানো dependency এনে কনফিগারেশন শেষ করবে। এড়ানোর ভালো উপায় ছিল শুরুতেই `sudo apt install ./app.deb` ব্যবহার করা।

7. `-Sy` index রিফ্রেশ করে কিন্তু বাকি সিস্টেম আপগ্রেড করে না। ফলে firefox-এর নতুন version নতুন library চাইতে পারে, অথচ বাকি সিস্টেম পুরনো library-তে — একে partial upgrade বলে, আর এটা Arch-এ সাপোর্টেড নয়, সিস্টেম ভাঙতে পারে। সঠিক: `sudo pacman -Syu firefox`।

8. `sudo dnf history list` দিয়ে transaction ID খুঁজে বের করো, তারপর `sudo dnf history undo <ID>`। (পুরনো version repo-তে থাকতে হবে।)

9. মিল: দুটোই distro-নিরপেক্ষ, অ্যাপের dependency সাথে নিয়ে আসে, sandbox-এ চালায়। অমিল: snap Canonical-এর, store একটাই (Snap Store) ও `snapd` ডেমন লাগে, SquashFS ইমেজ loop ডিভাইসে mount হয়, সার্ভার/CLI অ্যাপও বিতরণ করে; flatpak কমিউনিটি-চালিত, যেকোনো remote (মূলত Flathub) যোগ করা যায়, শেয়ার করা runtime ব্যবহার করে, মূলত ডেস্কটপ GUI অ্যাপের জন্য, root ছাড়া `--user` ইনস্টল সম্ভব।

10. Debian: `sudo apt-mark hold pkg` (ফেরত: `unhold`, দেখো: `showhold`)। RHEL: `sudo dnf versionlock add pkg` (dnf4-এ `python3-dnf-plugin-versionlock` লাগে), বা `dnf.conf` এ `exclude=`। ঝুঁকি: প্যাকেজটা নিরাপত্তা আপডেটও পাবে না; অনেকদিন hold থাকলে অন্য প্যাকেজের dependency আটকে যেতে পারে।

অধ্যায় ৩১: সোর্স কোড থেকে সফটওয়্যার বিল্ড করা

> আগের অধ্যায়ে (অধ্যায় ৩০) আমরা `apt` / `dnf` দিয়ে "রান্না করা খাবার" কিনে এনেছি। এই অধ্যায়ে আমরা নিজেরাই রান্না করব: কাঁচা উপকরণ (সোর্স কোড) থেকে চলার মতো প্রোগ্রাম বানাব। আর রান্নাঘরের ভেতরে আসলে কী কী হয়, সেটাও দেখব।

এই অধ্যায়ে যা শিখবে

- সোর্স কোড, binary, compile, link আসলে কী জিনিস
- compile-এর চার ধাপ: preprocess → compile → assemble → link (ডায়াগ্রামসহ)
- `gcc` দিয়ে প্রথম C প্রোগ্রাম (hello world) বানানো, প্রতিটা ধাপ আলাদা করে দেখা
- static library (`.a`) আর shared library (`.so`) এর পার্থক্য, আর দুটোই নিজে বানানো
- প্রোগ্রাম চালু হলে library কে খুঁজে দেয়: `ld.so` , `ldd` , `ldconfig` , `LD_LIBRARY_PATH`
- বিল্ড টুল ইনস্টল: `build-essential` (Debian/Ubuntu), `Development Tools` (Fedora/RHEL)
- tarball নামিয়ে ক্লাসিক `./configure && make && sudo make install`
- `Makefile` লেখার বেসিক
- `cmake` পরিচিতি
- `/usr/local` কেন ব্যবহার করা হয়
- `checkinstall` , `make uninstall` , `DESTDIR`
- কেন প্যাকেজ ম্যানেজারই প্রথম পছন্দ হওয়া উচিত

৩১.১ সোর্স কোড আর binary: আসল কথা

সোর্স কোড কী?

Source code মানে মানুষের পড়ার মতো করে লেখা প্রোগ্রাম, যেমন C, C++, Rust বা Go ভাষায় লেখা টেক্সট ফাইল। নিচেরটা একটা C সোর্স কোড:

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

এটা পড়ে তুমি বুঝতে পারছ এটা কী করবে। কিন্তু CPU কিছুই বোঝে না। CPU বোঝে শুধু **machine code**, মানে `0` আর `1` দিয়ে লেখা নির্দেশনা।

Binary কী?

Binary (বা executable, মানে চালানোর যোগ্য ফাইল) হলো সেই machine code-এর ফাইল। `/usr/bin/ls`, `/usr/bin/bash` সবই binary। লিনাক্সে এগুলো **ELF** (Executable and Linkable Format) নামের একটা নির্দিষ্ট ফরম্যাটে থাকে।

```
$ file /usr/bin/ls
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=..., for GNU/Linux
3.2.0, stripped
```

এই লাইনটা পরে আবার কাজে লাগবে। "dynamically linked" আর "interpreter /lib64/ld-linux-x86-64.so.2" অংশ দুটো খেয়াল রাখো।

Compile মানে কী?

Compile করা মানে সোর্স কোডকে machine code-এ অনুবাদ করা। যে প্রোগ্রাম অনুবাদ করে তাকে বলে **compiler**। লিনাক্সে সবচেয়ে বেশি চলে **GCC** (GNU Compiler Collection), আর তার পাশাপাশি **Clang**।

> 💡 **উপমা:** সোর্স কোড হলো বাংলায় লেখা রান্নার রেসিপি। CPU হলো এমন এক রাঁধুনি যে শুধু জাপানি বোঝে। compiler হলো অনুবাদক, যে পুরো রেসিপি জাপানিতে অনুবাদ করে দেয়। একবার অনুবাদ হয়ে গেলে রাঁধুনি যতবার খুশি সেটা দেখে রান্না করতে পারে, প্রতিবার অনুবাদক লাগে না।

কেন সোর্স থেকে বিল্ড করতে হয়?

সাধারণত দরকার পড়ে না, প্যাকেজ ম্যানেজারই যথেষ্ট। তবু কিছু ক্ষেত্রে লাগে:

পরিস্থিতি	উদাহরণ
distro-র রিপোজিটরিতে সফটওয়্যারটা নেই	ছোট কোনো GitHub প্রজেক্ট
রিপোজিটরির ভার্সন অনেক পুরনো	Debian stable-এ নতুন ফিচার দরকার
বিশেষ compile অপশন লাগবে	কোনো ফিচার চালু/বন্ধ করে বিল্ড, যেমন nginx-এ নির্দিষ্ট module
নিজের লেখা কোড	তুমি C/C++ ডেভেলপার
শেখা ও ডিবাগ করা	সফটওয়্যারটা ভেতরে কীভাবে কাজ করে বোঝা
প্যাচ লাগানো	একটা bug fix এখনো রিলিজ হয়নি

৩১.২ Compile-এর চার ধাপ: ভেতরে কী হয়

আমরা মুখে বলি "compile করলাম", কিন্তু আসলে `gcc` একটা নয়, চারটা আলাদা ধাপ চালায়:

```
hello.c   (তোমার লেখা সোর্স কোড)
   │
   │  ১. Preprocess  (cpp)      gcc -E
   │     #include ফাইল ঢোকানো, #define বদলানো, comment মুছে ফেলা
   ▼
hello.i   (বিশাল, বিস্তারিত C কোড)
   │
   │  ২. Compile     (cc1)      gcc -S
   │     C কোড → assembly ভাষা (CPU-নির্দিষ্ট, মানুষ পড়তে পারে)
   ▼
hello.s   (assembly)
   │
   │  ৩. Assemble    (as)       gcc -c
   │     assembly → machine code, কিন্তু অসম্পূর্ণ
   ▼
hello.o   (object file: printf কোথায় আছে এখনো জানে না)
   │
   │  ৪. Link        (ld)       gcc
   │     অনেক .o + library (libc) জোড়া লাগিয়ে চূড়ান্ত প্রোগ্রাম
   ▼
hello     (ELF executable, চালানোর যোগ্য)
```

১. Preprocessing (প্রি-প্রসেসিং)

`#` দিয়ে শুরু হওয়া লাইনগুলো (**directive**) এই ধাপে প্রসেস হয়:

- `#include <stdio.h>`: `/usr/include/stdio.h` ফাইলের পুরো লেখাটা ওই জায়গায় কপি করে বসিয়ে দেয়
- `#define PI 3.14`: কোডে যেখানে `PI` আছে সেখানে `3.14` বসায়
- `#ifdef ... #endif`: শর্ত অনুযায়ী কোডের অংশ রাখে বা বাদ দেয়
- comment মুছে ফেলে

২. Compilation (আসল অনুবাদ)

C কোড বিশ্লেষণ করে (syntax check, type check, optimization) তারপর **assembly language**-এ রূপান্তর করে। এখানেই তুমি syntax error-এর বার্তা পাও। `-02` এর মতো optimization অপশন এই ধাপে কাজ করে।

৩. Assembly (অ্যাসেম্বলি)

Assembler (`as`) assembly-কে machine code-এ রূপান্তর করে **object file** (`.o`) বানায়। এই ফাইলে তোমার কোডের machine code আছে, কিন্তু `printf` এর মতো বাইরের ফাংশনগুলো কোথায় আছে তা নেই। ওগুলো "unresolved symbol" হিসেবে চিহ্নিত থাকে।

৪. Linking (লিংকিং)

Linker (`ld`) সব object file আর library জোড়া লাগায়, unresolved symbol গুলোর ঠিকানা মিলিয়ে দেয় (`printf` → `libc` -এর ভেতরে), তারপর একটা পূর্ণ executable বানায়।

> 📝 **মনে রাখো:** "undefined reference to `xyz` " error আসে **link** ধাপে, মানে কোড ঠিক আছে কিন্তু ফাংশনটা কোন library-তে আছে linker সেটা পাচ্ছে না। "expected ';' before ..." এর মতো error আসে **compile** ধাপে। কোন ধাপের error সেটা চিনতে পারলে ঠিক করা অনেক সহজ হয়।

৩১.৩ বিল্ড টুল ইনস্টল করা

কোনো কিছু compile করার আগে compiler, linker, `make` ইত্যাদি লাগবে। প্রতিটা distro একটা "মেটা-প্যাকেজ" বা "গ্রুপ" দেয়, যেটা একবারে সব নামিয়ে দেয়।

Debian / Ubuntu: `build-essential`

```
$ sudo apt update
$ sudo apt install build-essential
```

`build-essential` নিজে কিছু না, এটা অন্য প্যাকেজের একটা তালিকা: `gcc` , `g++` , `make` , `libc6-dev` (C library-র header), `dpkg-dev` ।

Fedora / RHEL 9 / Rocky / AlmaLinux: `Development Tools` গ্রুপ

```
# Fedora 41 বা নতুন (dnf5)
$ sudo dnf group install development-tools

# RHEL 9 / Rocky 9 / AlmaLinux 9 এবং পুরনো Fedora (dnf4)
$ sudo dnf groupinstall "Development Tools"
```

> 💡 **টিপস:** গ্রুপের নাম/ID দেখতে `dnf group list` চালাও। RHEL-এ `dnf groupinstall` আর `dnf group install` দুটোই কাজ করে।

আরও যা প্রায়ই লাগে

টুল	কাজ	Debian/Ubuntu প্যাকেজ	Fedora/RHEL প্যাকেজ
`cmake`	আধুনিক বিল্ড সিস্টেম	`cmake`	`cmake`

টুল	কাজ	Debian/Ubuntu প্যাকেজ	Fedora/RHEL প্যাকেজ
`pkg-config`	library-র compile/link ফ্ল্যাগ বলে দেয়	`pkg-config` (বা `pkgconf`)	`pkgconf-pkg-config`
`autoconf`, `automake`, `libtool`	`configure` স্ক্রিপ্ট তৈরি করে	একই নাম	একই নাম
`git`	সোর্স ক্লোন করা	`git`	`git`
`ninja`	দ্রুত বিল্ড (cmake/meson-এর সাথে)	`ninja-build`	`ninja-build`
`meson`	আরেকটা আধুনিক বিল্ড সিস্টেম	`meson`	`meson`

যাচাই করো:

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
...
$ make --version
GNU Make 4.3
...
```

(ভার্সন নম্বর distro অনুযায়ী আলাদা হবে, যেমন Debian 12-এ gcc 12, RHEL 9-এ gcc 11।)

৩১.৪ `gcc` দিয়ে Hello World

প্রথম প্রোগ্রাম

একটা ফোল্ডার বানাও আর ফাইল লেখো (অধ্যায় ১৩-এর `nano` বা অধ্যায় ১৪-এর `vim` দিয়ে):

```
$ mkdir -p ~/build-lab/hello && cd ~/build-lab/hello
$ nano hello.c
```

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

compile করো আর চালাও:

```
$ gcc hello.c -o hello
$ ./hello
```

```
Hello, World!
```

> 📝 **মনে রাখো:** `./hello` লিখতে হয় কারণ বর্তমান ডিরেক্টরি (`.`) `PATH`-এ থাকে না (নিরাপত্তার কারণে)। শুধু `hello` লিখলে shell সেটা `PATH`-এর ডিরেক্টরিগুলোতে খুঁজবে। বিস্তারিত অধ্যায় ৪৪-এ।

`-o` না দিলে আউটপুট ফাইলের নাম হয় `a.out` (ঐতিহাসিক নাম)।

`gcc` সিনট্যাক্স

```
gcc [অপশন] সোর্স-ফাইল... [-o আউটপুট]
```

গুরুত্বপূর্ণ `gcc` অপশন

অপশন	অর্থ
`-o FILE`	আউটপুট ফাইলের নাম
`-E`	শুধু preprocess করে থামো (stdout-এ লেখে)
`-S`	compile পর্যন্ত, `.s` (assembly) বানাও
`-c`	assemble পর্যন্ত, `.o` বানাও, link কোরো না
`-Wall`	সাধারণ সব warning দেখাও (সবসময় ব্যবহার করো)
`-Wextra`	আরও কিছু বাড়তি warning
`-Werror`	warning-কেও error ধরো
`-g`	debug তথ্য রাখো (`gdb`-এর জন্য)
`-O0`, `-O1`, `-O2`, `-O3`, `-Os`	optimization লেভেল (`-O0` = কোনো optimization নেই, ডিফল্ট; `-Os` = সাইজ ছোট)
`-I DIR`	header (`.h`) খোঁজার জন্য বাড়তি ডিরেক্টরি
`-L DIR`	library খোঁজার জন্য বাড়তি ডিরেক্টরি (link সময়)
`-lNAME`	`libNAME.so` বা `libNAME.a` এর সাথে link করো (যেমন `-lm` = `libm`)
`-D NAME=VAL`	`#define NAME VAL` এর মতো কাজ
`-std=c17`	C স্ট্যান্ডার্ড নির্বাচন (`c99`, `c11`, `c17`, `gnu17` ...)
`-fPIC`	position-independent code (shared library-র জন্য)
`-shared`	shared library (`.so`) বানাও
`-static`	পুরো প্রোগ্রাম static link করো
`-v`	প্রতিটা ধাপে কী চালাচ্ছে দেখাও
`-save-temps`	`.i`, `.s`, `.o` মধ্যবর্তী ফাইলগুলো রেখে দাও

C++ এর জন্য একই অপশন দিয়ে `g++` ব্যবহার করো।

চার ধাপ নিজের চোখে দেখো

```
$ gcc -E hello.c -o hello.i      # ১. preprocess
$ wc -l hello.c hello.i
   6 hello.c
 795 hello.i
 801 total
```

মাত্র ৬ লাইন থেকে প্রায় ৮০০ লাইন! (সংখ্যাটা distro আর glibc ভার্সন অনুযায়ী আলাদা হবে।) পুরোটাই `stdio.h` আর তার ভেতরের অন্য header থেকে এসেছে।

```
$ gcc -S hello.i -o hello.s      # ২. compile → assembly
$ cat hello.s
        .file   "hello.c"
        .text
        .section        .rodata
.LC0:
        .string "Hello, World!"
        .text
        .globl  main
        .type   main, @function
main:
        ...
        call    puts@PLT
        ...
```

> 💡 **মজার তথ্য:** তুমি `printf` লিখেছ, কিন্তু assembly-তে `puts` এসেছে। কারণ শুধু একটা স্থির string আর শেষে `\n` থাকলে GCC নিজেই বুঝে নেয় যে `puts` দিয়েই একই কাজ হবে আর সেটা দ্রুত। compiler এরকম অনেক বুদ্ধি খাটায়।

```
$ gcc -c hello.s -o hello.o      # ৩. assemble → object file
$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

$ nm hello.o                     # object-এর symbol তালিকা
0000000000000000 T main
                 U puts
```

`nm` দেখাচ্ছে: `main` এখানে **T** (text, মানে কোড এই ফাইলেই আছে), আর `puts` **U** (undefined, অন্য কোথাও আছে)। linker এই `U` গুলো পূরণ করবে।

```
$ gcc hello.o -o hello           # ৪. link
$ ./hello
Hello, World!
```

এক কমান্ডে সব মধ্যবর্তী ফাইল রাখতে চাইলে: `gcc -save-temps hello.c -o hello`।

একাধিক ফাইলের প্রোগ্রাম

বাস্তবে প্রোগ্রাম অনেক ফাইলে ভাগ করা থাকে:

```
/* mathutil.h */
#ifndef MATHUTIL_H
#define MATHUTIL_H
int add(int a, int b);
int square(int x);
#endif
```

```
/* mathutil.c */
#include "mathutil.h"
int add(int a, int b) { return a + b; }
int square(int x)     { return x * x; }
```

```
/* main.c */
#include <stdio.h>
#include "mathutil.h"

int main(void) {
    printf("2 + 3 = %d\n", add(2, 3));
    printf("4^2  = %d\n", square(4));
    return 0;
}
```

```
$ gcc -Wall -c mathutil.c        # mathutil.o
$ gcc -Wall -c main.c            # main.o
$ gcc main.o mathutil.o -o calc  # link
$ ./calc
2 + 3 = 5
4^2  = 16
```

আলাদা করে `.o` বানানোর সুবিধা: শুধু `main.c` বদলালে শুধু সেটাই আবার compile করতে হয়, `mathutil.o` আগেরটাই চলে। বড় প্রজেক্টে (যেমন Linux kernel, যার হাজার হাজার ফাইল) এটা ঘণ্টার পর ঘণ্টা সময় বাঁচায়। এই "কোনটা আবার বিল্ড করতে হবে" হিসাবটাই `make` করে (৩১.৮-এ দেখব)।

> `<stdio.h>` (কোণ বন্ধনী) মানে সিস্টেম header, `/usr/include` এর মতো ডিরেক্টরিতে খোঁজে। `"mathutil.h"` (উদ্ধৃতি চিহ্ন) মানে আগে বর্তমান ডিরেক্টরিতে খোঁজে।

৩১.৫ Library: static বনাম shared

Library কী?

Library হলো আগে থেকে compile করা কিছু ফাংশনের সংগ্রহ, যেটা অনেক প্রোগ্রাম ব্যবহার করতে পারে। যেমন `printf`, `malloc`, `strlen` সব আছে **glibc** (GNU C Library)-তে, যার ফাইলের নাম `libc.so.6`। প্রায় প্রতিটা লিনাক্স প্রোগ্রাম এটা ব্যবহার করে।

লিনাক্সে library দুই রকম:

বৈশিষ্ট্য	Static library	Shared library
এক্সটেনশন	`.a` (archive)	`.so` (shared object)
উইন্ডোজে সমতুল্য	`.lib`	`.dll`
কখন জোড়া লাগে	**link time**-এ, প্রোগ্রামের ভেতরে কপি হয়ে যায়	**run time**-এ, প্রোগ্রাম চালুর সময় লোড হয়
binary সাইজ	বড়	ছোট
RAM ব্যবহার	প্রতিটা প্রোগ্রাম নিজের কপি রাখে	এক কপি RAM-এ, অনেক প্রোগ্রাম ভাগাভাগি করে
library আপডেট (যেমন security fix)	সব প্রোগ্রাম আবার link করতে হয়	শুধু `.so` বদলালেই সবাই নতুনটা পায়
নির্ভরতা	binary একা চলে	`.so` না থাকলে প্রোগ্রাম চালুই হয় না
উদাহরণ	`libc.a`	`libc.so.6`, `libssl.so.3`

> 💡 **উপমা: Static library** হলো একটা বইয়ের পাতা ফটোকপি করে নিজের খাতায় আঠা দিয়ে লাগিয়ে নেওয়া। খাতা ভারী হয়, কিন্তু লাইব্রেরি বন্ধ থাকলেও পড়তে পারবে। বইয়ে ভুল সংশোধন হলে তোমার কপিতে সেটা আসবে না। **Shared library** হলো খাতায় শুধু লিখে রাখা "পাবলিক লাইব্রেরির ৩ নম্বর তাকের বই, পৃষ্ঠা ৪২ দেখো"। খাতা হালকা, সবাই একই বই পড়ে, বই আপডেট হলে সবাই নতুনটা পায়। কিন্তু লাইব্রেরি থেকে বই সরিয়ে ফেললে তুমি আর পড়তে পারবে না।

এজন্যই লিনাক্স distro-গুলো প্রায় সবকিছু shared library দিয়ে বিল্ড করে: OpenSSL-এ একটা security bug ঠিক হলে একটা প্যাকেজ আপডেট করলেই শত শত প্রোগ্রাম নিরাপদ হয়ে যায়।

Static library বানানো

```
$ gcc -c mathutil.c -o mathutil.o
$ ar rcs libmathutil.a mathutil.o
$ ar t libmathutil.a              # ভেতরে কী আছে
mathutil.o
```

`ar` হলো archiver, অনেক `.o` কে একটা `.a` ফাইলে বাঁধে। `r` = যোগ/প্রতিস্থাপন, `c` = দরকার হলে তৈরি, `s` = symbol index লেখা।

```
$ gcc main.c -L. -lmathutil -o calc_static
$ ./calc_static
2 + 3 = 5
4^2  = 16
```

`-L.` মানে "বর্তমান ডিরেক্টরিতেও library খোঁজো", `-lmathutil` মানে `libmathutil` (সামনের `lib` আর পেছনের `.a` / `.so` linker নিজেই যোগ করে)।

> ⚠️ **সতর্কতা:** `-l` অপশন কমান্ডের **শেষের দিকে**, সোর্স/অবজেক্ট ফাইলের **পরে** দাও। GNU linker বাম থেকে ডানে পড়ে; library আগে দিলে তখনো কোনো unresolved symbol না থাকায় সে library টা থেকে কিছু নেয় না, আর পরে "undefined reference" error দেয়।

Shared library বানানো

```
$ gcc -fPIC -c mathutil.c -o mathutil.o
$ gcc -shared -Wl,-soname,libmathutil.so.1 -o libmathutil.so.1.0.0 mathutil.o
$ ln -s libmathutil.so.1.0.0 libmathutil.so.1
$ ln -s libmathutil.so.1 libmathutil.so
$ ls -l libmathutil.so*
lrwxrwxrwx 1 rahim rahim    16 ... libmathutil.so -> libmathutil.so.1
lrwxrwxrwx 1 rahim rahim    20 ... libmathutil.so.1 -> libmathutil.so.1.0.0
-rwxrwxr-x 1 rahim rahim 15560 ... libmathutil.so.1.0.0
```

এখানে কয়েকটা নতুন জিনিস:

- **`-fPIC`** (Position Independent Code): কোডটা মেমোরির যেকোনো ঠিকানায় লোড হলেও চলবে। shared library বিভিন্ন প্রোগ্রামে বিভিন্ন ঠিকানায় বসে, তাই এটা দরকার।
- **`-Wl,...`**: `gcc`-কে বলে "এই অপশনগুলো linker (`ld`) কে পাঠাও"।
- **soname**: library-র "অফিশিয়াল নাম", যেটা প্রোগ্রামের ভেতরে লেখা থাকে।
- symlink (অধ্যায় ১১) দিয়ে তিন স্তরের নাম:

```
libmathutil.so            ← "linker name": শুধু বিল্ডের সময় -lmathutil এটা খোঁজে
      │ (symlink)
      ▼
libmathutil.so.1          ← "soname": চালানোর সময় প্রোগ্রাম এটা চায়
      │ (symlink)               (major version 1 = ABI সামঞ্জস্য)
      ▼
libmathutil.so.1.0.0      ← "real name": আসল ফাইল (major.minor.patch)
```

major version (`.1`) বদলায় শুধু যখন library-তে এমন পরিবর্তন হয় যেটা পুরনো প্রোগ্রাম ভেঙে দেবে (**ABI**, Application Binary Interface, পরিবর্তন)। তাই `libssl.so.3` আর `libssl.so.1.1` একসাথে সিস্টেমে থাকতে পারে, আর পুরনো প্রোগ্রাম পুরনোটা, নতুন প্রোগ্রাম নতুনটা ব্যবহার করে।

এখন link করো:

```
$ gcc main.c -L. -lmathutil -o calc_shared
$ ./calc_shared
./calc_shared: error while loading shared libraries: libmathutil.so.1: cannot open
shared object file: No such file or directory
```

বিল্ড হয়েছে, কিন্তু চলছে না! কেন? কারণ `-L.` শুধু **বিল্ডের সময়** linker-কে বলেছিল কোথায় খুঁজতে হবে। চালানোর সময় library খোঁজে আরেকজন: **dynamic linker** `ld.so`। আর সে বর্তমান ডিরেক্টরিতে খোঁজে না। চলো তাকে চিনি।

> 📝 **মনে রাখো:** একই নামের `.so` আর `.a` দুটোই থাকলে `-l` ডিফল্টভাবে `.so` (shared) বেছে নেয়।

৩১.৬ Dynamic linker `ld.so`: প্রোগ্রাম চালুর সময় কী হয়

Under the hood

তুমি যখন `./calc_shared` চালাও:

```
তুমি: ./calc_shared
   │
   ▼
┌──────────────── Kernel (execve system call) ────────────────┐
│ ১. ফাইলটা ELF কিনা দেখে                                    │
│ ২. ELF header-এ "interpreter" লেখা পায়:                    │
│       /lib64/ld-linux-x86-64.so.2                           │
│ ৩. প্রোগ্রাম + এই interpreter মেমোরিতে লোড করে              │
│ ৪. নিয়ন্ত্রণ দেয় interpreter (ld.so)-কে, প্রোগ্রামকে না   │
└─────────────────────────┬───────────────────────────────────┘
                          ▼
┌──────────────── ld.so (dynamic linker/loader) ──────────────┐
│ ৫. প্রোগ্রামের "NEEDED" তালিকা পড়ে:                       │
│       libmathutil.so.1, libc.so.6                           │
│ ৬. প্রতিটা library খোঁজে (নিচের ক্রমে)                     │
│ ৭. খুঁজে পেলে মেমোরিতে map করে (mmap)                      │
│ ৮. symbol মেলায়: add → libmathutil-এর ঠিকানা,              │
│                   puts → libc-এর ঠিকানা                     │
│ ৯. নিয়ন্ত্রণ দেয় প্রোগ্রামের শুরুতে → main()              │
└─────────────────────────────────────────────────────────────┘
```

`ld.so` (বা `ld-linux.so`) নিজেও একটা shared library, আবার নিজেই চলতে পারে। man page: `man ld.so`। এর আসল পাথ distro ও আর্কিটেকচার অনুযায়ী বদলায়: x86_64-এ `/lib64/ld-linux-x86-64.so.2`, 64-bit ARM-এ `/lib/ld-linux-aarch64.so.1`।

প্রোগ্রাম কোন কোন library চায় দেখতে:

```
$ readelf -d calc_shared | grep NEEDED
 0x0000000000000001 (NEEDED)             Shared library: [libmathutil.so.1]
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
```

দেখো, এখানে **soname** (`libmathutil.so.1`) লেখা আছে, linker name বা real name না।

`ld.so` কোন ক্রমে library খোঁজে

1. binary-র ভেতরে লেখা **`DT_RPATH`** (শুধু যদি `DT_RUNPATH` না থাকে; পুরনো পদ্ধতি)
2. **`LD_LIBRARY_PATH`** environment variable-এর ডিরেক্টরিগুলো (setuid প্রোগ্রামে উপেক্ষা করা হয়)
3. binary-র ভেতরে লেখা **`DT_RUNPATH`**
4. **`/etc/ld.so.cache`** ফাইল (যেটা `ldconfig` বানায়)
5. ডিফল্ট ডিরেক্টরি: `/lib` , `/usr/lib` (64-bit সিস্টেমে distro অনুযায়ী `/lib64` , `/usr/lib64` বা `/usr/lib/x86_64-linux-gnu`)

> 💡 **উপমা:** `ld.so` হলো একজন কুরিয়ার। প্যাকেটে (প্রোগ্রামে) লেখা আছে "libmathutil.so.1 লাগবে"। সে আগে দেখে প্যাকেটের গায়ে কোনো বিশেষ ঠিকানা (RPATH) লেখা আছে কিনা, তারপর তোমার হাতে লেখা চিরকুট (`LD_LIBRARY_PATH`), তারপর তার নিজের ফোনবুক (`ld.so.cache`), শেষে পরিচিত গুদামগুলো (`/usr/lib`)। কোথাও না পেলে হাল ছেড়ে দেয়, আর সেটাই ওই "cannot open shared object file" error।

৩১.৭ `ldd` , `LD_LIBRARY_PATH` , `ldconfig`

`ldd` : কোন library লাগবে আর কোথায় পাওয়া গেল

কাজ: একটা প্রোগ্রাম বা library কোন shared library-র উপর নির্ভর করে, আর সেগুলো এখন কোথায় resolve হবে দেখায়।

সিনট্যাক্স: `ldd [অপশন] ফাইল...`

অপশন	অর্থ
`-v`	বিস্তারিত, symbol version তথ্যসহ
`-u`	যেসব library link হয়েছে কিন্তু আসলে ব্যবহার হয় না
`-d` / `-r`	data/function relocation চালিয়ে missing symbol রিপোর্ট করো

```
$ ldd /usr/bin/ls
        linux-vdso.so.1 (0x00007ffd5a1f2000)
        libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f8c3a5e0000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f8c3a200000)
```

```
        libpcre2-8.so.0 => /lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x00007f8c3a540000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f8c3a640000)
```

(Fedora/RHEL-এ পাথ হবে `/lib64/libc.so.6`।)

- `linux-vdso.so.1` : কোনো ফাইল না, kernel নিজেই প্রতিটা প্রসেসে এটা ঢুকিয়ে দেয় যাতে `gettimeofday` এর মতো কিছু system call দ্রুত হয় (অধ্যায় ৪৭)
- `=>` এর পরে আসল পাথ
- শেষেরটা `ld.so` নিজে

আমাদের ভাঙা প্রোগ্রাম:

```
$ ldd calc_shared
        linux-vdso.so.1 (0x00007ffc...)
        libmathutil.so.1 => not found
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f...)
        /lib64/ld-linux-x86-64.so.2 (0x00007f...)
```

`not found` দেখেই সমস্যা ধরা গেল। বাস্তবে কোনো প্রোগ্রাম "error while loading shared libraries" দিলে প্রথম কাজ: `ldd` চালাও।

> ⚠️ **সতর্কতা:** অবিশ্বস্ত (অচেনা জায়গা থেকে নামানো) binary-তে `ldd` চালাবে না। কিছু ক্ষেত্রে `ldd` প্রোগ্রামটাকে আসলেই চালিয়ে ফেলতে পারে (`man ldd` -এ এই সতর্কতা লেখা আছে)। নিরাপদ বিকল্প: `objdump -p FILE | grep NEEDED` বা `readelf -d FILE`।

`LD_LIBRARY_PATH` : সাময়িক সমাধান

```
$ LD_LIBRARY_PATH=. ./calc_shared
2 + 3 = 5
4^2  = 16

$ LD_LIBRARY_PATH=$PWD ldd calc_shared | grep mathutil
        libmathutil.so.1 => /home/rahim/build-lab/hello/libmathutil.so.1 (0x00007f...)
```

একাধিক ডিরেক্টরি `:` দিয়ে আলাদা করো: `LD_LIBRARY_PATH=/opt/foo/lib:/opt/bar/lib`।

> ⚠️ **সতর্কতা:** `LD_LIBRARY_PATH` স্থায়ীভাবে `.bashrc` -এ `export` করে রাখা খারাপ অভ্যাস। এতে সিস্টেমের সব প্রোগ্রাম আগে ওই ডিরেক্টরিতে খুঁজবে। ওখানে কোনো ভুল ভার্সনের `libssl.so.3` থাকলে অন্য প্রোগ্রামও ভেঙে যেতে পারে, ধীরও হয়। এটা টেস্ট আর ডিবাগের জন্য, অথবা একটা নির্দিষ্ট প্রোগ্রামের wrapper স্ক্রিপ্টের ভেতরে ব্যবহারের জন্য।

RPATH/RUNPATH দিয়ে binary-র ভেতরেই পাথ লিখে দেওয়া যায়:

```
$ gcc main.c -L. -lmathutil -Wl,-rpath,'$ORIGIN' -o calc_rpath
$ ./calc_rpath
2 + 3 = 5
4^2  = 16
$ readelf -d calc_rpath | grep -E 'RPATH|RUNPATH'
 0x000000000000001d (RUNPATH)            Library runpath: [$ORIGIN]
```

`$ORIGIN` মানে "binary যে ডিরেক্টরিতে আছে"। আধুনিক toolchain ডিফল্টভাবে `RUNPATH` লেখে। (single quote দিতে হয় যাতে shell `$ORIGIN`-কে variable ভেবে বদলে না ফেলে।)

`ldconfig` আর `/etc/ld.so.conf`: স্থায়ী সমাধান

প্রতিবার `ld.so` যদি পুরো ডিস্কের সব ডিরেক্টরি ঘেঁটে library খুঁজত, প্রোগ্রাম চালু হতে অনেক সময় লাগত। তাই একটা **cache** (`/etc/ld.so.cache`) থাকে, যেটা "soname → পাথ" এর তালিকা। এই cache বানায় **`ldconfig`**।

```
  /etc/ld.so.conf
     └── include /etc/ld.so.conf.d/*.conf
            ├── libc.conf              → /usr/local/lib   (Debian/Ubuntu)
            ├── x86_64-linux-gnu.conf  → /usr/local/lib/x86_64-linux-gnu, ...
            └── myapp.conf             → /opt/myapp/lib   (তুমি যোগ করবে)
                      │
                      ▼   sudo ldconfig  (ডিরেক্টরি স্ক্যান করে)
              /etc/ld.so.cache   (binary ফাইল, ld.so এটা পড়ে)
```

সিনট্যাক্স: `ldconfig [অপশন] [ডিরেক্টরি...]`

অপশন	অর্থ
(কোনো অপশন নেই)	cache নতুন করে বানাও, soname symlink আপডেট করো (root লাগে)
`-p`	cache-এ বর্তমানে কী আছে তা দেখাও
`-v`	কী করছে বিস্তারিত দেখাও
`-n DIR`	শুধু ওই ডিরেক্টরিতে symlink বানাও, cache ছুঁয়ো না
`-N`	cache আপডেট কোরো না (শুধু symlink)

```
$ ldconfig -p | grep libssl
        libssl.so.3 (libc6,x86-64) => /lib/x86_64-linux-gnu/libssl.so.3
$ ldconfig -p | head -1
1234 libs found in cache `/etc/ld.so.cache'
```

আমাদের library সিস্টেম-ওয়াইড ইনস্টল করি:

```
$ sudo cp libmathutil.so.1.0.0 /usr/local/lib/
$ sudo ldconfig
$ ls -l /usr/local/lib/libmathutil*
lrwxrwxrwx 1 root root    20 ... /usr/local/lib/libmathutil.so.1 ->
libmathutil.so.1.0.0
```

```
-rwxr-xr-x 1 root root 15560 ... /usr/local/lib/libmathutil.so.1.0.0
$ ./calc_shared
2 + 3 = 5
4^2  = 16
```

খেয়াল করো, `ldconfig` নিজেই soname symlink (`libmathutil.so.1`) বানিয়ে দিয়েছে।

> ⚠️ **Distro পার্থক্য:** Debian/Ubuntu-তে `/usr/local/lib` আগে থেকেই `/etc/ld.so.conf.d/libc.conf`-এ আছে। কিন্তু **Fedora/RHEL-এ `/usr/local/lib` বা `/usr/local/lib64` ডিফল্টভাবে খোঁজার তালিকায় নেই।** সেখানে নিজে যোগ করতে হবে:
>
> ```
> $ echo '/usr/local/lib' | sudo tee /etc/ld.so.conf.d/usr-local.conf
> $ echo '/usr/local/lib64' | sudo tee -a /etc/ld.so.conf.d/usr-local.conf
> $ sudo ldconfig
> ```

> 📝 **মনে রাখো:** নতুন shared library ইনস্টল করার পরে **`sudo ldconfig`** চালাও। প্যাকেজ ম্যানেজার এটা নিজে করে, কিন্তু `make install` অনেক সময় করে না।

Static link করা binary

```
$ gcc -static hello.c -o hello_static
$ ls -lh hello hello_static
-rwxrwxr-x 1 rahim rahim  16K ... hello
-rwxrwxr-x 1 rahim rahim 880K ... hello_static
$ ldd hello_static
        not a dynamic executable
$ file hello_static
hello_static: ELF 64-bit LSB executable, x86-64, ..., statically linked, ...
```

৫০ গুণের বেশি বড়, কারণ libc-এর দরকারি অংশ ভেতরে কপি হয়ে গেছে। (সাইজ ভার্সন অনুযায়ী আলাদা হবে।)

> ⚠️ **Distro পার্থক্য:** Fedora/RHEL-এ `-static` কাজ করতে `glibc-static` প্যাকেজ লাগে (`sudo dnf install glibc-static` ; RHEL 9-এ এটা CRB রিপোজিটরিতে থাকে)। না থাকলে "cannot find -lc" error আসে। Debian/Ubuntu-তে `libc6-dev`-এর সাথেই `libc.a` আসে।

৩১.৮ `make` আর `Makefile`: বিল্ড স্বয়ংক্রিয় করা

সমস্যা

১০০টা `.c` ফাইল থাকলে হাতে ১০০ বার `gcc -c` লিখবে? একটা ফাইল বদলালে কোনগুলো আবার compile করতে হবে মনে রাখবে? না, এটা করে **`make`**।

`make` একটা `Makefile` পড়ে, যেখানে লেখা থাকে কোন ফাইল কোন ফাইল থেকে তৈরি হয় (**dependency**) আর কীভাবে তৈরি হয় (**recipe**)। তারপর ফাইলের **modification time** (শেষ পরিবর্তনের সময়) মিলিয়ে দেখে: target-এর চেয়ে কোনো dependency নতুন হলে শুধু সেটাই আবার বিল্ড করে।

> 💡 **উপমা:** `Makefile` হলো রান্নাঘরে টাঙানো একটা চার্ট: "পোলাও বানাতে লাগবে সেদ্ধ চাল আর ভাজা পেঁয়াজ; সেদ্ধ চাল বানাতে লাগবে কাঁচা চাল"। `make` এমন এক বুদ্ধিমান সহকারী যে দেখে পেঁয়াজ আগেই ভাজা আছে কিনা। থাকলে আবার ভাজে না, শুধু যা দরকার তাই করে।

মৌলিক গঠন

```
target: dependency1 dependency2
    recipe-command
```

> ⚠️ **সবচেয়ে বিখ্যাত ভুল:** recipe লাইনের শুরুতে **TAB** অক্ষর লাগবে, স্পেস না। স্পেস দিলে error আসে: `Makefile:2: *** missing separator. Stop.` `vim`-এ `:set list` দিলে TAB দেখা যায় `^I` হিসেবে। `cat -A Makefile` দিয়েও দেখতে পারো।

আমাদের calc প্রজেক্টের Makefile

```
# Makefile
CC      = gcc
CFLAGS  = -Wall -Wextra -O2
LDFLAGS =
PREFIX ?= /usr/local

OBJS = main.o mathutil.o

calc: $(OBJS)
    $(CC) $(LDFLAGS) $^ -o $@

main.o: main.c mathutil.h
    $(CC) $(CFLAGS) -c $< -o $@

mathutil.o: mathutil.c mathutil.h
    $(CC) $(CFLAGS) -c $< -o $@

install: calc
```

```
    install -D -m 755 calc $(DESTDIR)$(PREFIX)/bin/calc

uninstall:
    rm -f $(DESTDIR)$(PREFIX)/bin/calc

clean:
    rm -f calc $(OBJS)

.PHONY: install uninstall clean
```

Makefile-এর উপাদান

উপাদান	অর্থ
`VAR = value`	variable সংজ্ঞা; ব্যবহার করার সময় মান বের হয় (recursive)
`VAR := value`	তখনই একবার মান বের করে রাখে (simple)
`VAR ?= value`	শুধু যদি আগে সেট না থাকে (কমান্ড লাইন বা environment থেকে বদলানো যায়)
`$(VAR)`	variable-এর মান
`$@`	বর্তমান target-এর নাম
`$<`	প্রথম dependency
`$^`	সব dependency (ডুপ্লিকেট বাদে)
`.PHONY`	এই target গুলো আসলে ফাইল না, শুধু কাজের নাম
`#`	comment
`%.o: %.c`	pattern rule: যেকোনো `.c` থেকে `.o`
`@command`	কমান্ডটা চালানোর আগে স্ক্রিনে দেখিও না

pattern rule দিয়ে আরও ছোট:

```
%.o: %.c mathutil.h
    $(CC) $(CFLAGS) -c $< -o $@
```

> 💡 **টিপস:** `make`-এর নিজের অনেক **implicit rule** আছে। কিছু না লিখলেও সে জানে `.c` থেকে `.o` কীভাবে বানাতে হয় (`$(CC) $(CPPFLAGS) $(CFLAGS) -c`)। `make -p` দিয়ে সব built-in rule দেখতে পারো।

`.PHONY` কেন? ধরো ডিরেক্টরিতে কোনোভাবে `clean` নামে একটা ফাইল চলে এলো। তখন `make clean` বলবে "`clean` is up to date" আর কিছু মুছবে না। `.PHONY` বলে দেয় এটা ফাইল না, সবসময় চালাও।

ব্যবহার

```
$ make
gcc -Wall -Wextra -O2 -c main.c -o main.o
gcc -Wall -Wextra -O2 -c mathutil.c -o mathutil.o
gcc  main.o mathutil.o -o calc

$ make
make: 'calc' is up to date.

$ touch main.c        # main.c "বদলালাম"
$ make
gcc -Wall -Wextra -O2 -c main.c -o main.o
gcc  main.o mathutil.o -o calc
```

শুধু `main.c` আবার compile হলো, `mathutil.c` না। এটাই `make`-এর জাদু।

```
             calc
            /    \
       main.o    mathutil.o
       /    \       /     \
  main.c  mathutil.h  mathutil.c
              (দুজনেই এটার উপর নির্ভর করে)

  main.c বদলালে → main.o → calc  আবার বিল্ড
  mathutil.h বদলালে → main.o + mathutil.o → calc  সব আবার বিল্ড
```

`make` অপশন

অপশন	অর্থ
`make`	প্রথম target বিল্ড করো (সাধারণত `all`)
`make TARGET`	নির্দিষ্ট target
`-j N`	একসাথে N টা কাজ চালাও (parallel build)
`-j$(nproc)`	যতগুলো CPU core আছে ততগুলো কাজ
`-n`	dry run: কী চালাবে শুধু দেখাও, চালিও না
`-B`	সব কিছু জোর করে আবার বিল্ড করো
`-C DIR`	আগে ওই ডিরেক্টরিতে গিয়ে তারপর make চালাও
`-f FILE`	অন্য নামের Makefile ব্যবহার করো
`-k`	error হলেও যতটা সম্ভব চালিয়ে যাও
`VAR=value`	variable বদলাও, যেমন `make CFLAGS=-g`
`V=1` বা `VERBOSE=1`	অনেক প্রজেক্টে (automake: `V=1`, cmake: `VERBOSE=1`) পুরো কমান্ড দেখায়

```
$ nproc
8
```

```
$ make -j$(nproc)
```

> 💡 **টিপস:** বড় প্রজেক্টে `-j$(nproc)` বিল্ড কয়েক গুণ দ্রুত করে। তবে RAM কম হলে (যেমন ২ GB) বেশি parallel job সিস্টেম জমিয়ে দিতে পারে; তখন `-j2` দাও।

৩১.৯ `/usr/local`: নিজের বিল্ড করা সফটওয়্যার কোথায় যাবে

অধ্যায় ৭-এ FHS দেখেছ। মূল নিয়ম:

```
/usr        ← প্যাকেজ ম্যানেজারের এলাকা (apt/dnf এর)। হাত দেবে না।
│   ├── bin, lib, include, share
│
/usr/local  ← স্থানীয় অ্যাডমিনের এলাকা (তুমি)। প্যাকেজ ম্যানেজার এখানে হাত দেয় না।
│   ├── bin        → এক্সিকিউটেবল (PATH-এ /usr/bin এর আগে থাকে)
│   ├── sbin
│   ├── lib        → library
│   ├── include    → header
│   ├── share      → man page, ডকুমেন্টেশন, ডেটা
│   ├── etc        → কনফিগ (কিছু প্রজেক্ট /usr/local/etc ব্যবহার করে)
│   └── src        → সোর্স কোড রাখার জায়গা (প্রথা)
│
/opt/APPNAME ← স্বয়ংসম্পূর্ণ তৃতীয়-পক্ষের সফটওয়্যার (একটা ফোল্ডারে সব)
│
~/.local    ← শুধু তোমার ইউজারের জন্য (root লাগে না), ~/.local/bin
```

কেন `/usr/local`?

1. **সংঘর্ষ এড়ানো:** তুমি `make install` দিয়ে `/usr/bin/python3` ওভাররাইট করলে পরের `apt upgrade` সেটা আবার বদলে দেবে, অথবা ডজন ডজন সিস্টেম টুল (যেগুলো distro-র python-এর উপর নির্ভর করে) ভেঙে যাবে।
2. **ট্র্যাকিং:** প্যাকেজ ম্যানেজার `/usr`-এর প্রতিটা ফাইলের হিসাব রাখে। সেখানে অচেনা ফাইল রাখলে বিভ্রান্তি তৈরি হয়।
3. **অগ্রাধিকার:** ডিফল্ট `PATH`-এ `/usr/local/bin` সাধারণত `/usr/bin`-এর **আগে** থাকে, তাই নিজের বিল্ড করা নতুন ভার্সন আগে চলে।

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

প্রায় সব `configure` স্ক্রিপ্ট আর CMake-এর ডিফল্ট install prefix হলো `/usr/local`। root ছাড়া ইনস্টল করতে চাইলে `--prefix=$HOME/.local` দাও।

> 💡 **টিপস:** নতুন কোনো প্রোগ্রাম ইনস্টলের পরেও shell পুরনো পাথ থেকে চালাচ্ছে মনে হলে `hash -r` চালাও। bash কমান্ডের পাথ মনে রেখে দেয় (cache)। কোনটা চলছে দেখতে `type -a calc` বা `which -`

> `a calc` (অধ্যায় ১২)।

৩১.১০ বাস্তব প্রজেক্ট: tarball থেকে GNU hello বিল্ড

GNU hello একটা ছোট প্রোগ্রাম যেটা GNU প্রজেক্ট বানিয়েছে, ঠিক এই উদ্দেশ্যে: GNU বিল্ড সিস্টেম কীভাবে কাজ করে তার উদাহরণ দেখানো।

ক্লাসিক তিন ধাপ, এক নজরে

```
hello-2.12.1.tar.gz
      │  tar xf       (অধ্যায় ৩৪)
      ▼
hello-2.12.1/   (configure, Makefile.in, src/, README, INSTALL ...)
      │
      │  ./configure      → তোমার সিস্টেম পরীক্ষা করে, Makefile তৈরি করে
      │                     (compiler আছে? কোন library আছে? কোথায় ইনস্টল?)
      ▼
Makefile, config.h
      │
      │  make             → compile + link (সোর্স ডিরেক্টরির ভেতরে)
      ▼
hello (binary)
      │
      │  sudo make install → ফাইলগুলো /usr/local/... তে কপি করে
      ▼
/usr/local/bin/hello, /usr/local/share/man/man1/hello.1, ...
```

ধাপ ১: সোর্স ডাউনলোড

```
$ mkdir -p ~/build-lab && cd ~/build-lab
$ wget https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
$ wget https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz.sig
```

ধাপ ২: যাচাই (সতর্ক অভ্যাস)

ইন্টারনেট থেকে নামানো কোড `sudo` দিয়ে ইনস্টল করবে, তাই সেটা আসল কিনা যাচাই করা জরুরি। GNU প্রজেক্ট GPG স্বাক্ষর দেয়:

```
$ gpg --verify hello-2.12.1.tar.gz.sig hello-2.12.1.tar.gz
gpg: Signature made ...
gpg: Can't check signature: No public key
```

"No public key" মানে রক্ষণাবেক্ষণকারীর public key তোমার কাছে নেই। GNU-র keyring (`https://ftp.gnu.org/gnu/gnu-keyring.gpg`) নামিয়ে import করে আবার যাচাই করতে পারো। অনেক প্রজেক্ট শুধু checksum দেয়; তখন:

```
$ sha256sum hello-2.12.1.tar.gz
```

আর ওয়েবসাইটে দেওয়া মানের সাথে মিলিয়ে দেখো।

> ⚠️ **সতর্কতা:** অচেনা ওয়েবসাইট বা ব্লগের লিংক থেকে সোর্স নামিয়ে `sudo make install` চালানো মানে সেই কোডকে তোমার পুরো সিস্টেমের মালিকানা দিয়ে দেওয়া। `Makefile`-এর `install` target-এ যেকোনো কমান্ড লেখা থাকতে পারে। সবসময় অফিশিয়াল সোর্স ব্যবহার করো।

ধাপ ৩: খোলা আর পড়া

```
$ tar xf hello-2.12.1.tar.gz
$ cd hello-2.12.1
$ ls
ABOUT-NLS  ChangeLog  INSTALL  Makefile.am  NEWS  README  build-aux  configure
contrib  doc  lib  m4  man  po  src  tests ...
```

> 📝 **মনে রাখো:** বিল্ড করার আগে `README` আর `INSTALL` ফাইল পড়বে। প্রতিটা প্রজেক্টের নিজস্ব নির্দেশনা, dependency তালিকা আর বিশেষ অপশন সেখানে লেখা থাকে।

ধাপ ৪: `./configure`

`configure` একটা বিশাল shell script (সাধারণত **Autoconf** দিয়ে তৈরি)। এটা তোমার সিস্টেম পরীক্ষা করে:

```
$ ./configure
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /usr/bin/mkdir -p
checking for gawk... no
checking for mawk... mawk
checking whether make sets $(MAKE)... yes
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
...
config.status: creating Makefile
config.status: creating config.h
```

এটা আসলে কী করে:

- ছোট ছোট টেস্ট প্রোগ্রাম লিখে compile করে দেখে (যেমন "এই সিস্টেমে `strnlen` ফাংশন আছে কি?")
- ফলাফল লেখে `config.h`-এ (`#define HAVE_STRNLEN 1`)
- `Makefile.in` টেমপ্লেট থেকে আসল `Makefile` বানায়, তোমার prefix ও অপশন বসিয়ে
- সব লগ রাখে `config.log`-এ

গুরুত্বপূর্ণ `configure` অপশন:

অপশন	অর্থ
`--help`	এই প্রজেক্টের সব অপশন দেখাও (সবচেয়ে কাজের)
`--prefix=DIR`	ইনস্টলের মূল ডিরেক্টরি (ডিফল্ট `/usr/local`)
`--bindir=`, `--libdir=`, `--sysconfdir=`	আলাদা আলাদা ডিরেক্টরি সেট করা
`--enable-FEATURE` / `--disable-FEATURE`	ঐচ্ছিক ফিচার চালু/বন্ধ
`--with-PACKAGE[=DIR]` / `--without-PACKAGE`	বাইরের কোনো library ব্যবহার করবে কি না
`CC=clang`	অন্য compiler
`CFLAGS="-O2 -g"`	compiler ফ্ল্যাগ

```
$ ./configure --prefix=$HOME/.local        # root ছাড়া, শুধু নিজের জন্য
$ ./configure --help | less
```

`configure` ব্যর্থ হলে (সবচেয়ে সাধারণ সমস্যা):

```
configure: error: Package requirements (libxml-2.0 >= 2.9) were not met:
No package 'libxml-2.0' found
```

মানে একটা library-র **development প্যাকেজ** নেই। চলার জন্য library (`libxml2`) আর বিল্ডের জন্য header + `.so` symlink + `.pc` ফাইল আলাদা প্যাকেজে থাকে:

distro	নামের ধরন	উদাহরণ
Debian/Ubuntu	`lib...-dev`	`sudo apt install libxml2-dev`
Fedora/RHEL	`...-devel`	`sudo dnf install libxml2-devel`

কোন প্যাকেজে কোন ফাইল আছে খুঁজতে: Ubuntu-তে `apt-file search libxml/parser.h` (আগে `sudo apt install apt-file && sudo apt-file update`), Fedora-তে `dnf provides '*/libxml/parser.h'` (অধ্যায় ৩০)।

> 💡 **টিপস:** distro-র প্যাকেজটা যে dependency দিয়ে বিল্ড হয়, সব একবারে নামাতে পারো:
>
> - Debian/Ubuntu: `sudo apt build-dep PACKAGE` (এর জন্য `deb-src` সোর্স চালু থাকতে হয়; Ubuntu 24.04-এ `/etc/apt/sources.list.d/ubuntu.sources` ফাইলে `Types: deb deb-src` করে `sudo apt update`)
> - Fedora/RHEL: `sudo dnf builddep PACKAGE` (`dnf-plugins-core` লাগে; Fedora 41+ এ dnf5-এ built-in)

`pkg-config` হলো সেই টুল যেটা `configure` library খুঁজতে ব্যবহার করে:

```
$ pkg-config --cflags --libs libxml-2.0
-I/usr/include/libxml2 -lxml2
$ pkg-config --modversion libxml-2.0
2.9.14
```

নিজের প্রোগ্রামেও: `gcc app.c $(pkg-config --cflags --libs libxml-2.0) -o app`।

ধাপ ৫: `make`

```
$ make -j$(nproc)
make  all-recursive
make[1]: Entering directory '/home/rahim/build-lab/hello-2.12.1'
...
  CC       src/hello.o
  CCLD     hello
...
$ ./hello
Hello, world!
$ ./hello --greeting="আসসালামু আলাইকুম"
আসসালামু আলাইকুম
```

অনেক প্রজেক্টে `make check` বা `make test` দিয়ে টেস্ট চালানো যায়:

```
$ make check
...
PASS: tests/greeting-1
PASS: tests/hello-1
...
# PASS:  ...
# FAIL:  0
```

ধাপ ৬: `sudo make install`

```
$ sudo make install
...
 /usr/bin/mkdir -p '/usr/local/bin'
  /usr/bin/install -c hello '/usr/local/bin'
 /usr/bin/mkdir -p '/usr/local/share/man/man1'
 /usr/bin/install -c -m 644 man/hello.1 '/usr/local/share/man/man1'
...
$ which hello
/usr/local/bin/hello
$ hello --version | head -1
hello (GNU Hello) 2.12.1
$ man hello
```

> 📝 **মনে রাখো:** শুধু `make install` ধাপটায় `sudo` লাগে (কারণ `/usr/local` root-এর)। `./configure` আর `make` কখনো `sudo` দিয়ে চালাবে না। তাহলে সোর্স ডিরেক্টরির সব ফাইলের মালিক

> root হয়ে যাবে, আর বিল্ড প্রসেসের কোনো ভুল পুরো সিস্টেমে ক্ষতি করতে পারবে (অধ্যায় ১৮)।

> 💡 **টিপস:** তিনটা এক লাইনে: `./configure && make -j$(nproc) && sudo make install`। `&&` মানে আগেরটা সফল হলেই পরেরটা চলবে (অধ্যায় ১৯)।

`git` থেকে নামানো সোর্সে `configure` না থাকলে

tarball-এ `configure` আগে থেকে তৈরি থাকে, কিন্তু git রিপোজিটরিতে সাধারণত শুধু `configure.ac` আর `Makefile.am` থাকে। তখন আগে `configure` তৈরি করতে হয়:

```
$ ./autogen.sh        # প্রজেক্ট দিলে, অথবা:
$ autoreconf -fi      # autoconf/automake/libtool লাগবে
$ ./configure
```

`make clean` আর `make distclean`

কমান্ড	কী মোছে
`make clean`	বিল্ড করা ফাইল (`.o`, binary)
`make distclean`	উপরেরটা + `configure`-এর তৈরি ফাইল (`Makefile`, `config.h`, `config.log`); tarball খোলার পরের অবস্থায় ফেরে

ভিন্ন `--prefix` বা অপশন দিয়ে আবার configure করতে চাইলে আগে `make distclean` করে নাও।

৩১.১১ আনইনস্টল: সবচেয়ে বড় দুর্বলতা

প্যাকেজ ম্যানেজার জানে কোন প্যাকেজ কোন ফাইল বসিয়েছে, তাই `apt remove` পরিষ্কার করে মুছে দেয়। কিন্তু `make install` কোনো হিসাব রাখে না। ফাইলগুলো শুধু কপি হয়ে যায়।

`make uninstall`

Automake দিয়ে বানানো প্রজেক্টে (GNU hello সহ) একটা `uninstall` target থাকে:

```
$ cd ~/build-lab/hello-2.12.1
$ sudo make uninstall
 ( cd '/usr/local/bin' && rm -f hello )
 ...
$ which hello
$
```

শর্ত আর সমস্যা:

- **একই সোর্স ডিরেক্টরি আর একই `configure` অপশন** লাগবে। সোর্স ফোল্ডার মুছে ফেললে বা ভিন্ন `--prefix` দিয়ে আবার configure করলে কাজ করবে না।
- **সব প্রজেক্টে এটা থাকে না।** হাতে লেখা Makefile-এ প্রায়ই নেই। থাকলে `make -n uninstall` দিয়ে আগে দেখে নাও কী মুছবে।
- নতুন ভার্সনের সোর্স দিয়ে পুরনো ভার্সন আনইনস্টল করলে কিছু ফাইল থেকে যেতে পারে।

> 💡 **টিপস:** সোর্স ডিরেক্টরিটা মুছবে না, `/usr/local/src`-এ রেখে দাও। পরে আনইনস্টলের সময় কাজে লাগবে।

`DESTDIR` দিয়ে আগে দেখে নেওয়া

প্রায় সব ভালো বিল্ড সিস্টেম `DESTDIR` বোঝে। এটা পুরো ইনস্টলটাকে একটা আলাদা ফোল্ডারের ভেতরে করে, প্যাকেজ বানানোর সময় distro-রা ঠিক এটাই করে:

```
$ make DESTDIR=/tmp/hello-stage install
$ find /tmp/hello-stage -type f
/tmp/hello-stage/usr/local/bin/hello
/tmp/hello-stage/usr/local/share/man/man1/hello.1
/tmp/hello-stage/usr/local/share/info/hello.info
/tmp/hello-stage/usr/local/share/locale/bn/LC_MESSAGES/hello.mo
...
```

এখানে `sudo` লাগেনি, আর তুমি জেনে গেলে ঠিক কোন কোন ফাইল ইনস্টল হবে। এই তালিকা সংরক্ষণ করে রাখলে পরে নিজেই মুছতে পারবে।

`checkinstall`: `make install` থেকে প্যাকেজ

`checkinstall` হলো `make install` কে ঘিরে চলা একটা টুল। এটা ইনস্টল কোন ফাইলগুলো বসাচ্ছে নজর রাখে, তারপর সেগুলো দিয়ে একটা `.deb` (বা `.rpm`) প্যাকেজ বানিয়ে প্যাকেজ ম্যানেজারের মাধ্যমে ইনস্টল করে। ফলে পরে `apt remove` দিয়ে পরিষ্কার মোছা যায়।

```
$ sudo apt install checkinstall          # Debian/Ubuntu
$ cd ~/build-lab/hello-2.12.1
$ ./configure && make
$ sudo checkinstall --pkgname=hello-local --pkgversion=2.12.1 --default
...
**********************************************************************
 Done. The new package has been installed and saved to
 /home/rahim/build-lab/hello-2.12.1/hello-local_2.12.1-1_amd64.deb

 You can remove it from your system anytime using:
      dpkg -r hello-local
**********************************************************************

$ dpkg -L hello-local | head
$ sudo apt remove hello-local            # পরিষ্কার আনইনস্টল
```

অপশন	অর্থ
`--pkgname=NAME`	প্যাকেজের নাম
`--pkgversion=VER`	ভার্সন
`--default`	সব প্রশ্নে ডিফল্ট উত্তর, প্রশ্ন না করে
`--install=no`	শুধু `.deb` বানাও, ইনস্টল কোরো না
`-D` / `-R`	Debian / RPM প্যাকেজ বানাও

> ⚠️ **সীমাবদ্ধতা:** `checkinstall` অনেক বছর ধরে সক্রিয়ভাবে উন্নত হচ্ছে না। এটা Ubuntu-র `universe` রিপোজিটরিতে আছে, কিন্তু **Fedora/RHEL-এ অফিশিয়াল রিপোজিটরিতে নেই**। জটিল ইনস্টল স্ক্রিপ্টে মাঝে মাঝে ভুল করে। এর বানানো প্যাকেজে সঠিক dependency তথ্যও থাকে না। নিজের মেশিনে ছোটখাটো জিনিসের জন্য ঠিক আছে; সার্ভারে বা অনেক মেশিনে বিতরণের জন্য আসল প্যাকেজ বানানো শেখো (Debian-এ `dpkg-buildpackage`, Fedora/RHEL-এ `rpmbuild` ও spec ফাইল)।

আরেকটা কৌশল: প্রতিটা প্রোগ্রাম আলাদা prefix-এ

```
$ ./configure --prefix=/opt/hello-2.12.1
$ make && sudo make install
$ sudo ln -s /opt/hello-2.12.1/bin/hello /usr/local/bin/hello
```

আনইনস্টল মানে শুধু `sudo rm -rf /opt/hello-2.12.1` আর symlink মোছা। একাধিক ভার্সন পাশাপাশিও রাখা যায়। (GNU Stow নামের টুল এই symlink ব্যবস্থাপনা স্বয়ংক্রিয় করে।)

৩১.১২ CMake পরিচিতি

Autotools (`configure`) পুরনো আর জটিল। নতুন অনেক C/C++ প্রজেক্ট (যেমন LLVM, KDE, OpenCV) **CMake** ব্যবহার করে। CMake নিজে বিল্ড করে না; সে একটা **build system generator**: `CMakeLists.txt` পড়ে `Makefile` (বা Ninja-র `build.ninja`) বানায়।

```
  CMakeLists.txt  ──cmake (configure)──►  build/Makefile বা build/build.ninja
                                                    │
                                       cmake --build (make/ninja)
                                                    ▼
                                                 binary
```

একটা সহজ `CMakeLists.txt`

আমাদের calc প্রজেক্টের জন্য:

```
cmake_minimum_required(VERSION 3.16)
project(calc VERSION 1.0 LANGUAGES C)

add_library(mathutil SHARED mathutil.c)
add_executable(calc main.c)
target_link_libraries(calc PRIVATE mathutil)

install(TARGETS calc mathutil)
```

বিল্ড করা ("out-of-source" বিল্ড)

```
$ cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info - done
...
-- Configuring done
-- Generating done
-- Build files have been written to: /home/rahim/build-lab/hello/build

$ cmake --build build -j$(nproc)
[ 25%] Building C object CMakeFiles/mathutil.dir/mathutil.c.o
[ 50%] Linking C shared library libmathutil.so
[ 75%] Building C object CMakeFiles/calc.dir/main.c.o
[100%] Linking C executable calc
[100%] Built target calc

$ ./build/calc
2 + 3 = 5
4^2  = 16

$ sudo cmake --install build
-- Install configuration: "Release"
-- Installing: /usr/local/bin/calc
-- Installing: /usr/local/lib/libmathutil.so
...
```

`-S` = source ডিরেক্টরি, `-B` = build ডিরেক্টরি। সব বিল্ড ফাইল `build/` এর ভেতরে থাকে, সোর্স পরিষ্কার থাকে। নতুন করে শুরু করতে শুধু `rm -rf build`।

> 📝 **মনে রাখো:** পুরনো টিউটোরিয়ালে দেখবে `mkdir build && cd build && cmake .. && make && sudo make install`। এটা একই কাজ করে। `cmake --build` আর `cmake --install` (CMake 3.15+) আধুনিক, generator-নিরপেক্ষ পদ্ধতি।

CMake অপশন / কমান্ড	অর্থ
`-S DIR` / `-B DIR`	সোর্স / বিল্ড ডিরেক্টরি
`-DCMAKE_INSTALL_PREFIX=/opt/x`	ইনস্টল prefix (ডিফল্ট `/usr/local`)
`-DCMAKE_BUILD_TYPE=Release`	`Debug`, `Release`, `RelWithDebInfo`, `MinSizeRel`

CMake অপশন / কমান্ড	অর্থ
`-DVAR=VALUE`	প্রজেক্টের কোনো অপশন সেট করা (যেমন `-DBUILD_TESTING=OFF`)
`-G Ninja`	Makefile-এর বদলে Ninja ব্যবহার
`cmake -LH build`	প্রজেক্টের সব cache variable ও তার ব্যাখ্যা দেখাও
`ccmake build`	টার্মিনালে মেনু দিয়ে অপশন বদলানো (`cmake-curses-gui` প্যাকেজ)
`cmake --build build --target clean`	clean
`cmake --build build -v`	পুরো কমান্ড দেখাও
`cmake --install build --prefix ~/.local`	ভিন্ন prefix-এ ইনস্টল

CMake-এ আনইনস্টল

CMake ডিফল্টভাবে `uninstall` target দেয় না। তবে `cmake --install` ইনস্টল করা ফাইলের তালিকা রাখে `build/install_manifest.txt`-এ:

```
$ cat build/install_manifest.txt
/usr/local/bin/calc
/usr/local/lib/libmathutil.so
$ xargs -d '\n' sudo rm -v < build/install_manifest.txt
```

(অধ্যায় ২৩-এ `xargs` দেখো।) এটা শুধু ফাইল মোছে, তৈরি হওয়া খালি ডিরেক্টরি না।

অন্যান্য বিল্ড সিস্টেম, এক নজরে

প্রজেক্টে থাকলে	বিল্ড সিস্টেম	সাধারণ কমান্ড
`configure` / `configure.ac`	Autotools	`./configure && make && sudo make install`
`CMakeLists.txt`	CMake	`cmake -S . -B build && cmake --build build && sudo cmake --install build`
`meson.build`	Meson	`meson setup build && meson compile -C build && sudo meson install -C build`
শুধু `Makefile`	হাতে লেখা make	`make && sudo make install` (README দেখো)
`Cargo.toml`	Rust / Cargo	`cargo build --release` বা `cargo install --path .`
`go.mod`	Go	`go build` বা `go install`

প্রজেক্টে থাকলে	বিল্ড সিস্টেম	সাধারণ কমান্ড
`pyproject.toml` / `setup.py`	Python	`pip install .` (virtualenv-এর ভেতরে)

৩১.১৩ কেন প্যাকেজ ম্যানেজারই প্রথম পছন্দ

এতকিছু শেখার পরে সবচেয়ে গুরুত্বপূর্ণ উপদেশ: **যদি প্যাকেজ ম্যানেজারে পাওয়া যায়, সেটাই ব্যবহার করো।** কারণ:

বিষয়	প্যাকেজ ম্যানেজার (`apt` / `dnf`)	সোর্স থেকে বিল্ড
Security আপডেট	`apt upgrade`-এ স্বয়ংক্রিয়	তোমাকে নিজে খবর রেখে আবার বিল্ড করতে হবে
Dependency	নিজে খুঁজে ইনস্টল করে	হাতে খুঁজে খুঁজে `-dev` প্যাকেজ ইনস্টল
আনইনস্টল	`apt remove`, পরিষ্কার	`make uninstall` থাকলে ভাগ্য ভালো
ফাইলের হিসাব	`dpkg -L` / `rpm -ql`	কোনো হিসাব নেই
সত্যতা যাচাই	GPG-স্বাক্ষরিত রিপোজিটরি, স্বয়ংক্রিয়	নিজে করতে হবে
টেস্টিং	distro আর হাজারো ইউজার পরীক্ষা করেছে	তুমিই প্রথম পরীক্ষক হতে পারো
distro-র সাথে মিল	distro-র নিয়ম মেনে কনফিগ, systemd unit, SELinux/AppArmor নীতি	এগুলো নিজে করতে হবে
সময়	কয়েক সেকেন্ড	কয়েক মিনিট থেকে কয়েক ঘণ্টা

সোর্সে যাওয়ার আগে বিকল্পগুলো দেখো

1. distro-র অফিশিয়াল রিপোজিটরি (`apt search`, `dnf search`)
2. Debian-এ **backports**, Fedora/RHEL-এ **EPEL** (RHEL/Rocky-র জন্য অতিরিক্ত প্যাকেজ)
3. প্রজেক্টের নিজস্ব অফিশিয়াল রিপোজিটরি (যেমন Docker, PostgreSQL, Node.js এর নিজস্ব apt/dnf repo)
4. Ubuntu PPA বা Fedora COPR (তৃতীয় পক্ষের, বিশ্বাসের প্রশ্ন আছে)
5. **Flatpak**, **Snap**, **AppImage** (মূলত ডেস্কটপ অ্যাপ)
6. **কন্টেইনার** (অধ্যায় ৪৮): সার্ভারে প্রায়ই সবচেয়ে পরিষ্কার উপায়
7. প্রজেক্টের দেওয়া pre-built binary
8. শেষ উপায়: সোর্স থেকে বিল্ড, আর তখন `/usr/local` বা `/opt` এ, `checkinstall` বা আসল প্যাকেজ দিয়ে

> ⚠️ **সতর্কতা:** distro-র মূল লাইব্রেরি (`glibc`, `openssl`) বা মূল টুল (`python3`, `gcc`) সোর্স থেকে বিল্ড করে `/usr`-এ বসানো সিস্টেম ভাঙার সবচেয়ে নিশ্চিত উপায়। glibc ভুলভাবে বদলালে প্রায় কোনো প্রোগ্রামই চলবে না, `sudo` বা `ls` ও না। নতুন Python দরকার হলে `./configure --`

> `prefix=/usr/local` দিয়ে বিল্ড করে **`sudo make altinstall`** ব্যবহার করো (CPython-এর README-তে এটা বলা আছে), যাতে `python3` নামটা ওভাররাইট না হয়ে `python3.13` এর মতো ভার্সন-নির্দিষ্ট নামে ইনস্টল হয়।

৩১.১৪ সাধারণ সমস্যা ও সমাধান

সমস্যা / error	কারণ	সমাধান
`gcc: command not found`	বিল্ড টুল নেই	`build-essential` / `Development Tools`
`fatal error: zlib.h: No such file or directory`	header নেই, মানে `-dev` / `-devel` প্যাকেজ নেই	`zlib1g-dev` / `zlib-devel`
`configure: error: ... not found`	dependency নেই	`config.log` পড়ো, তারপর `-dev` প্যাকেজ
`undefined reference to 'sqrt'`	link-এ library দেওয়া হয়নি	`-lm` যোগ করো (কমান্ডের শেষে)
`cannot find -lfoo`	link time-এ `libfoo.so` / `.a` নেই	`-dev` প্যাকেজ, বা `-L DIR`
`error while loading shared libraries: libfoo.so.1`	run time-এ `ld.so` খুঁজে পাচ্ছে না	`ldd` দিয়ে দেখো; `sudo ldconfig`, `/etc/ld.so.conf.d/`, বা `LD_LIBRARY_PATH`
`Makefile:N: *** missing separator.`	recipe-তে TAB-এর বদলে স্পেস	TAB দাও
`make: *** No targets specified and no makefile found.`	`Makefile` নেই	আগে `./configure` / `cmake` চালাও
`./configure: Permission denied`	execute permission নেই	`chmod +x configure` বা `sh configure`
বিল্ডের মাঝে সিস্টেম জমে যাওয়া / `Killed`	RAM শেষ (OOM killer)	`-j` কমাও, swap বাড়াও (অধ্যায় ৪৬)
নতুন ইনস্টল করলাম কিন্তু পুরনো ভার্সন চলছে	shell-এর hash cache / PATH ক্রম	`hash -r`, `type -a PROG`
`version 'GLIBC_2.38' not found`	নতুন glibc-ওয়ালা সিস্টেমে বিল্ড করা binary পুরনো সিস্টেমে চালানো	যে সিস্টেমে চালাবে সেখানেই (বা পুরনো glibc-এ) বিল্ড করো

> 💡 **টিপস:** লম্বা বিল্ডের আউটপুটে আসল error খুঁজে পাওয়া কঠিন। প্রথম `error:` লাইনটাই সাধারণত আসল কারণ, পরেরগুলো তার ফলাফল। লগ ফাইলে সংরক্ষণ করো: `make 2>&1 | tee build.log`,

> তারপর `grep -n 'error' build.log`। দীর্ঘ বিল্ড SSH-এ চালালে `tmux`-এর ভেতরে চালাও (অধ্যায় ২৫)।

৩১.১৫ কাজের কীবোর্ড শর্টকাট

এই অধ্যায়ের নিজস্ব কোনো শর্টকাট নেই, তবে বিল্ড করার সময় এগুলো সবচেয়ে বেশি কাজে লাগে (বিস্তারিত অধ্যায় ৬):

শর্টকাট	কাজ
`Ctrl+C`	চলমান `make` / `configure` থামাও
`Ctrl+Z` তারপর `bg`	বিল্ডকে background-এ পাঠাও
`Ctrl+R`	আগের লম্বা `./configure --prefix=...` কমান্ড খুঁজে বের করো
`Ctrl+L`	স্ক্রিন পরিষ্কার
`Shift+PageUp` / `Shift+PageDown`	অনেক টার্মিনালে আউটপুট উপরে-নিচে স্ক্রল
`Ctrl+Shift+F`	GNOME Terminal-এ আউটপুটে খোঁজা
`!!`	আগের কমান্ড আবার (যেমন `sudo !!`)
`Alt+.`	আগের কমান্ডের শেষ argument (যেমন tarball-এর নাম)
`Tab`	ফাইলের নাম auto-complete (`hello-2.12.1.tar.gz` টাইপ করতে হয় না)

হাতে-কলমে অনুশীলন

ল্যাব ১: চার ধাপ নিজের চোখে

```
$ sudo apt install build-essential          # বা: sudo dnf group install development-
tools
$ mkdir -p ~/build-lab/lab1 && cd ~/build-lab/lab1
$ printf '#include <stdio.h>\n#define NAME "Linux"\nint main(void){ printf("Hello,
%%s!\\n", NAME); return 0; }\n' > hi.c
$ cat hi.c
$ gcc -E hi.c | tail -5        # NAME কি "Linux" দিয়ে বদলে গেছে?
$ gcc -S hi.c && less hi.s      # assembly দেখো
$ gcc -c hi.c && nm hi.o        # T main, U printf
$ gcc hi.o -o hi && ./hi
Hello, Linux!
$ file hi && ldd hi
```

প্রশ্ন: `nm hi.o`-তে এবার `printf` এলো নাকি `puts`? কেন? (ইঙ্গিত: format string-এ `%s` আছে।)

ল্যাব ২: static ও shared library

1. ৩১.৪-এর `mathutil.h`, `mathutil.c`, `main.c` তিনটা ফাইল `~/build-lab/lab2`-এ বানাও।
2. `ar rcs` দিয়ে `libmathutil.a` বানিয়ে `calc_static` link করো। `ldd calc_static`-এ `libmathutil` আছে কিনা দেখো (থাকার কথা না, ওটা ভেতরে ঢুকে গেছে; libc কিন্তু থাকবে)।
3. ৩১.৫-এর মতো `libmathutil.so.1.0.0` আর symlink বানাও, `calc_shared` link করো।
4. `./calc_shared` চালিয়ে error দেখো, `ldd` দিয়ে `not found` নিশ্চিত করো।
5. `LD_LIBRARY_PATH=. ./calc_shared` দিয়ে চালাও।
6. এবার মজা: `mathutil.c`-তে `square` বদলে `return x * x * x;` করো, শুধু `.so` আবার বিল্ড করো (main আবার link না করে)। `LD_LIBRARY_PATH=. ./calc_shared` চালাও। ফল বদলেছে! `./calc_static` চালাও। বদলায়নি! shared আর static-এর পার্থক্য হাতেনাতে দেখলে।

ল্যাব ৩: নিজের Makefile

1. ল্যাব ২-এর ফোল্ডারে ৩১.৮-এর `Makefile` লেখো (TAB মনে রেখো)।
2. `make`, আবার `make` (up to date), `touch mathutil.h`, `make` চালিয়ে দেখো কোনগুলো আবার বিল্ড হলো।
3. `make -n clean` দিয়ে dry run দেখো, তারপর `make clean`।
4. `make install PREFIX=$HOME/.local` দিয়ে root ছাড়া ইনস্টল করো, `ls ~/.local/bin`।
5. `make DESTDIR=/tmp/stage install` দিয়ে দেখো ফাইল কোথায় যায়।

ল্যাব ৪: GNU hello, পুরো চক্র

```
$ cd ~/build-lab
$ wget https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
$ sha256sum hello-2.12.1.tar.gz
$ tar xf hello-2.12.1.tar.gz && cd hello-2.12.1
$ less README INSTALL
$ ./configure --help | less
$ ./configure --prefix=$HOME/.local
$ make -j$(nproc)
$ make check
$ make install                    # ~/.local, তাই sudo লাগবে না
$ ~/.local/bin/hello --greeting="শুভ সকাল"
$ make uninstall
$ ls ~/.local/bin/hello           # No such file or directory
```

বোনাস (Debian/Ubuntu): `make distclean`, তারপর ডিফল্ট prefix দিয়ে configure করে `sudo checkinstall --pkgname=hello-local --pkgversion=2.12.1 --default` চালাও। `dpkg -L hello-local` দিয়ে ফাইলের তালিকা দেখো, শেষে `sudo apt remove hello-local`।

ল্যাব ৫: CMake

ল্যাব ২-এর ফোল্ডারে ৩১.১২-এর `CMakeLists.txt` লেখো, `sudo apt install cmake` (বা `dnf`), তারপর:

```
$ cmake -S . -B build -DCMAKE_INSTALL_PREFIX=$HOME/.local
$ cmake --build build -j$(nproc)
$ ./build/calc
$ cmake --install build
$ cat build/install_manifest.txt
$ ldd ~/.local/bin/calc | grep mathutil
```

শেষের কমান্ডে কী দেখলে? CMake ইনস্টলের সময় RPATH মুছে দেয় (ডিফল্ট আচরণ), তাই `libmathutil.so not found` দেখাতে পারে, কারণ `~/.local/lib` লাইব্রেরি খোঁজার তালিকায় নেই। এটা ঠিক করার দুটো উপায় ভাবো: `LD_LIBRARY_PATH`, নাকি `-DCMAKE_INSTALL_RPATH=$HOME/.local/lib`? কোনটা ভালো, কেন?

শেষে পরিষ্কার: `xargs -d '\n' rm -v < build/install_manifest.txt`।

সারসংক্ষেপ

মূল ধারণা: সোর্স → **preprocess** (`.i`) → **compile** (`.s`) → **assemble** (`.o`) → **link** (executable)। shared library চালানোর সময় `ld.so` লোড করে; static library বিল্ডের সময় ভেতরে কপি হয়। নিজের বিল্ড করা জিনিস `/usr/local`-এ যায়, `/usr`-এ না। আর সম্ভব হলে প্যাকেজ ম্যানেজারই ব্যবহার করো।

কমান্ড	কাজ
`sudo apt install build-essential`	Debian/Ubuntu-তে বিল্ড টুল
`sudo dnf group install development-tools`	Fedora 41+ এ বিল্ড টুল (RHEL 9: `dnf groupinstall "Development Tools"`)
`gcc file.c -o prog`	compile + link
`gcc -Wall -g -O2`	warning, debug তথ্য, optimization
`gcc -E` / `-S` / `-c`	preprocess / assembly / object পর্যন্ত থামো
`gcc ... -L DIR -lNAME`	`libNAME` এর সাথে link
`ar rcs libX.a a.o b.o`	static library
`gcc -fPIC -shared -o libX.so a.c`	shared library
`nm file.o`	object-এর symbol তালিকা
`file prog`	ELF ধরন, static/dynamic
`readelf -d prog`	NEEDED library, RUNPATH

কমান্ড	কাজ
`ldd prog`	কোন `.so` লাগবে, কোথায় পাওয়া গেল
`LD_LIBRARY_PATH=DIR prog`	সাময়িকভাবে library ডিরেক্টরি যোগ
`sudo ldconfig`	library cache নতুন করে বানাও
`ldconfig -p`	cache-এ কী আছে
`/etc/ld.so.conf.d/*.conf`	স্থায়ী library ডিরেক্টরি
`pkg-config --cflags --libs NAME`	library-র compile/link ফ্ল্যাগ
`tar xf X.tar.gz`	tarball খোলা
`./configure --prefix=DIR`	সিস্টেম পরীক্ষা, Makefile তৈরি
`make -j$(nproc)`	parallel বিল্ড
`make check`	টেস্ট
`sudo make install`	ইনস্টল
`make DESTDIR=/tmp/x install`	আলাদা ফোল্ডারে ইনস্টল (দেখে নেওয়া/প্যাকেজিং)
`sudo make uninstall`	আনইনস্টল (থাকলে)
`make clean` / `make distclean`	বিল্ড ফাইল মোছা / configure-এর আগের অবস্থা
`sudo checkinstall`	`make install` থেকে `.deb` বানিয়ে ইনস্টল
`cmake -S . -B build`	CMake configure
`cmake --build build`	CMake বিল্ড
`sudo cmake --install build`	CMake ইনস্টল
`apt build-dep PKG` / `dnf builddep PKG`	বিল্ড dependency একবারে
`hash -r`	shell-এর কমান্ড পাথ cache মোছা

নিজেকে যাচাই করো

1. compile প্রক্রিয়ার চারটা ধাপের নাম ক্রমানুসারে বলো, আর প্রতিটা ধাপের আউটপুট ফাইলের এক্সটেনশন কী?
2. "undefined reference to `pow`" error কোন ধাপে আসে, আর সাধারণ সমাধান কী?
3. static library আর shared library-র দুটো সুবিধা ও দুটো অসুবিধা বলো। distro-রা কেন shared library পছন্দ করে?
4. একটা প্রোগ্রাম চালালে "error while loading shared libraries: libfoo.so.2: cannot open shared object file" আসছে। তুমি ধাপে ধাপে কী করবে?
5. `-L` অপশন আর `LD_LIBRARY_PATH` দুটোই "library ডিরেক্টরি" বলে দেয়। পার্থক্য কী?

6. `/usr/local/lib`-এ নতুন `.so` কপি করার পরে কোন কমান্ড চালাতে হয়? Fedora-তে বাড়তি কী করতে হতে পারে?
7. `./configure`, `make` আর `make install` আলাদা আলাদা কী কাজ করে? কোনটায় শুধু `sudo` লাগে, আর কেন বাকিগুলোয় দেওয়া উচিত না?
8. Makefile-এ "missing separator" error কেন আসে? `.PHONY` কেন ব্যবহার করা হয়?
9. নিজের বিল্ড করা সফটওয়্যার `/usr`-এ না রেখে `/usr/local`-এ রাখা হয় কেন?
10. `make install` দিয়ে বসানো সফটওয়্যার পরে পরিষ্কারভাবে মোছার তিনটা উপায় বলো।

▼ **উত্তর দেখো**

1. Preprocess (`.i`) → Compile (`.s`, assembly) → Assemble (`.o`, object file) → Link (executable, ডিফল্ট নাম `a.out`)।
2. **Link** ধাপে। `pow` আছে math library `libm`-এ, তাই কমান্ডের শেষে `-lm` যোগ করতে হয়: `gcc prog.c -o prog -lm`। কোড ঠিক আছে, শুধু linker ফাংশনটার সংজ্ঞা পাচ্ছে না।
3. Static: সুবিধা হলো binary একা চলে, dependency সমস্যা নেই, আর চালুর সময় খোঁজাখুঁজি নেই। অসুবিধা হলো binary বড় হয়, আর library-তে security fix এলে প্রতিটা প্রোগ্রাম আবার link করতে হয়। Shared: সুবিধা হলো binary ছোট, RAM-এ এক কপি ভাগাভাগি হয়, আর library আপডেট করলেই সব প্রোগ্রাম ঠিক হয়ে যায়। অসুবিধা হলো `.so` না থাকলে বা ভুল ভার্সন হলে প্রোগ্রাম চলে না। distro-রা shared পছন্দ করে মূলত security আপডেটের সুবিধার জন্য: একটা `libssl` প্যাকেজ আপডেট করলেই শত শত প্রোগ্রাম নিরাপদ হয়।
4. (ক) `ldd prog` চালিয়ে দেখো কোনটা `not found`। (খ) ফাইলটা সিস্টেমে আছে কিনা খোঁজো (`find / -name 'libfoo.so*' 2>/dev/null`, বা `apt-file search` / `dnf provides`)। (গ) না থাকলে প্যাকেজ ম্যানেজার দিয়ে সংশ্লিষ্ট library প্যাকেজ ইনস্টল করো। (ঘ) থাকলে কিন্তু অপ্রচলিত ডিরেক্টরিতে থাকলে সেই ডিরেক্টরি `/etc/ld.so.conf.d/` এর একটা `.conf` ফাইলে লিখে `sudo ldconfig` চালাও। শুধু টেস্টের জন্য হলে `LD_LIBRARY_PATH` ব্যবহার করো।
5. `-L` কাজ করে **বিল্ডের (link) সময়**, linker (`ld`)-এর জন্য। `LD_LIBRARY_PATH` কাজ করে **চালানোর সময়**, dynamic linker (`ld.so`)-এর জন্য। একটা দিয়ে অন্যটার কাজ হয় না, তাই `-L.` দিয়ে বিল্ড হওয়া প্রোগ্রামও চালানোর সময় library খুঁজে না পেতে পারে।
6. `sudo ldconfig`, যেটা `/etc/ld.so.cache` আপডেট করে আর soname symlink বানায়। Fedora/RHEL-এ `/usr/local/lib` (আর `/usr/local/lib64`) ডিফল্ট খোঁজার তালিকায় নেই, তাই আগে `/etc/ld.so.conf.d/usr-local.conf` ফাইলে এগুলো যোগ করতে হয়।
7. `./configure` সিস্টেম পরীক্ষা করে (compiler, library, header) আর তোমার অপশন অনুযায়ী `Makefile` ও `config.h` বানায়। `make` সোর্স compile আর link করে সোর্স ডিরেক্টরির ভেতরেই binary বানায়। `make install` সেগুলো prefix-এর (যেমন `/usr/local`) সঠিক জায়গায় কপি করে। শুধু `make install`-এ `sudo` লাগে, কারণ `/usr/local` root-এর মালিকানায়। বাকিগুলো `sudo` দিয়ে চালালে বিল্ড ফাইলের মালিক root হয়ে যায়, আর বিল্ড প্রসেসের কোনো ভুল বা ক্ষতিকর কোড পুরো সিস্টেমের ক্ষমতা পেয়ে যায়।

8. recipe লাইনের শুরুতে TAB না দিয়ে স্পেস দিলে "missing separator" আসে, কারণ `make` শুধু TAB দেখেই recipe চেনে। `.PHONY` বলে দেয় `clean` / `install` এর মতো target আসলে ফাইল না। নাহলে ওই নামে কোনো ফাইল থাকলে `make` ভাববে target "up to date" আর কাজটা চালাবে না।
9. `/usr` প্যাকেজ ম্যানেজারের এলাকা। সেখানে নিজের ফাইল রাখলে পরের আপডেটে ওভাররাইট হতে পারে, distro-র ফাইল ভেঙে যেতে পারে (যেমন সিস্টেম python বদলে ফেলা), আর কোন ফাইল কার সেই হিসাব গুলিয়ে যায়। `/usr/local` স্থানীয় অ্যাডমিনের জন্য সংরক্ষিত; প্যাকেজ ম্যানেজার সেখানে হাত দেয় না, আর `PATH`-এ এটা `/usr/bin`-এর আগে থাকে।
10. (ক) একই সোর্স ডিরেক্টরি ও একই configure অপশন রেখে `sudo make uninstall` (যদি Makefile দেয়)। (খ) শুরুতেই `checkinstall` দিয়ে ইনস্টল করা, তারপর `apt remove` / `dpkg -r`। (গ) প্রতিটা সফটওয়্যারকে আলাদা prefix-এ (`/opt/NAME-VERSION`) ইনস্টল করে পরে পুরো ফোল্ডার মুছে ফেলা। এছাড়া CMake-এ `install_manifest.txt` থেকে, অথবা `make DESTDIR=... install` দিয়ে আগে সংরক্ষণ করা তালিকা থেকে ফাইল মোছা যায়।

অধ্যায় ৩২: ডিস্ক, পার্টিশন, ফাইলসিস্টেম ও মাউন্ট

এই অধ্যায়ে যা শিখবে

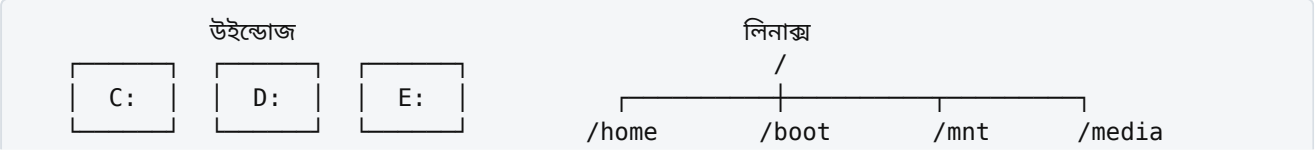
- লিনাক্স ডিস্ককে কীভাবে "দেখে" — block device কী, আর `sda`, `nvme0n1`, `vda`, `mmcblk0` নামগুলোর মানে কী
- ডিস্ক থেকে ফাইল পর্যন্ত পুরো স্তর (layer) — VFS, ফাইলসিস্টেম ড্রাইভার, page cache, block layer
- `lsblk`, `blkid`, `findmnt` দিয়ে কোন ডিস্কে কী আছে খুঁজে বের করা
- MBR আর GPT পার্টিশন টেবিলের পার্থক্য, ভেতরের গঠনসহ
- `fdisk`, `gdisk`, `parted` দিয়ে ধাপে ধাপে পার্টিশন তৈরি, মুছে ফেলা, টাইপ বদলানো
- `mkfs` দিয়ে ext4, XFS, vfat (FAT32), exFAT ফাইলসিস্টেম তৈরি
- mount point কী, `mount` / `umount` কীভাবে কাজ করে, "target is busy" সমস্যার সমাধান
- `/etc/fstab` ফাইলের প্রতিটি ফিল্ড, UUID কেন ব্যবহার করবে, `mount -a` ও `findmnt --verify`
- `df -h`, `du -sh`, `ncdu` দিয়ে ডিস্কের জায়গা মাপা — আর কেন `df` ও `du` আলাদা সংখ্যা দেখায়
- swap partition ও swapfile তৈরি, `swapon`, `swappiness`
- `fsck`, `e2fsck`, `xfs_repair`, `tune2fs` দিয়ে ফাইলসিস্টেম মেরামত ও টিউনিং
- USB পেনড্রাইভ নিরাপদে মাউন্ট ও আনমাউন্ট করা
- loop device ব্যবহার করে একদম ঝুঁকিহীনভাবে সব কিছু হাতে-কলমে চর্চা করা

৩২.১ শুরুর আগে একটা গল্প: আলমারি, তাক আর লাইব্রেরি

ধরো তুমি একটা নতুন বড় আলমারি কিনলে। এটা হলো তোমার **ডিস্ক** (হার্ডডিস্ক/SSD)। খালি আলমারিতে সরাসরি জিনিস রাখা যায়, কিন্তু সব এলোমেলো হয়ে যাবে। তাই তুমি:

1. আলমারিকে কয়েকটা **তাকে ভাগ করলে** — এটা হলো **পার্টিশন (partition)**।
2. প্রতিটা তাকে একটা **সাজানোর পদ্ধতি** ঠিক করলে — কোনটায় ফাইল-ফোল্ডার, কোনটায় বাক্স, প্রতিটার লেবেল আর একটা খাতায় তালিকা — এটা হলো **ফাইলসিস্টেম (filesystem)**, যেমন ext4 বা XFS।
3. তারপর বাড়ির কোনো ঘরের দরজার সাথে ওই তাকটাকে **জুড়ে দিলে**, যাতে "স্টোররুমে" ঢুকলেই ওই তাকের জিনিস পাওয়া যায় — এটা হলো **মাউন্ট (mount)**, আর ওই দরজাটা হলো **mount point**।

উইন্ডোজে প্রতিটা পার্টিশন আলাদা "ড্রাইভ" (C:, D:) হিসেবে আসে। লিনাক্সে কোনো ড্রাইভ লেটার নেই — পুরো সিস্টেম একটাই বিশাল গাছ (tree), যার গোড়া `/`। অন্য যেকোনো ডিস্ক বা পার্টিশন ওই গাছের কোনো একটা ডিরেক্টরিতে "জুড়ে" দেওয়া হয়। (ডিরেক্টরি গাছের বিস্তারিত দেখো অধ্যায় ৭।)

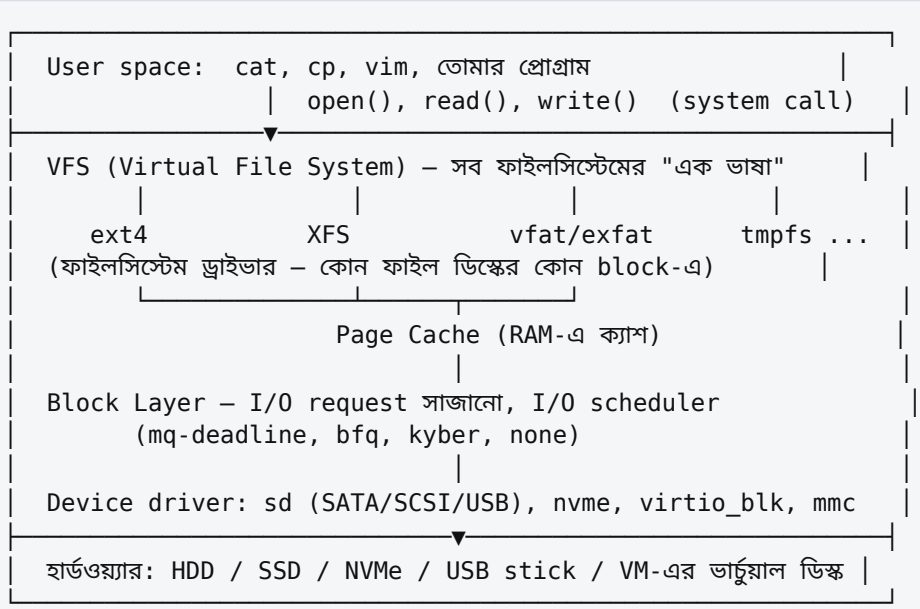


```
(প্রতিটা আলাদা গাছ)                    (sda3)      (sda1)      /data     /usb
                                                                 (sdb1)    (sdc1)
                                          একটাই গাছ, ডিস্কগুলো ডালে জোড়া
```

> 📝 **মনে রাখো:** ডিস্ক → পার্টিশন → ফাইলসিস্টেম → মাউন্ট। এই চারটা ধাপ মুখস্থ রাখলে এই অধ্যায়ের সবকিছু সহজ লাগবে।

৩২.২ ভেতরের গঠন: একটা `cat` কমান্ড ডিস্ক পর্যন্ত কীভাবে পৌঁছায়

তুমি যখন `cat /home/rahim/note.txt` চালাও, তখন ভেতরে অনেকগুলো স্তর পার হয়ে ডেটা আসে। অধ্যায় ২-এ আমরা kernel-এর সাধারণ গঠন দেখেছি; এখানে শুধু স্টোরেজ অংশটা জুম করে দেখি:



প্রতিটা স্তর কী করে:

- **VFS (Virtual File System):** kernel-এর ভেতরের একটা "অনুবাদক"। প্রোগ্রাম জানে না নিচে ext4 আছে নাকি পেনড্রাইভের FAT32 — সে শুধু `read()` ডাকে। VFS ঠিক ফাইলসিস্টেম ড্রাইভারের কাছে কাজটা পাঠিয়ে দেয়। এই কারণেই `cp` দিয়ে ext4 থেকে FAT32-এ কপি করা যায় কোনো বাড়তি ঝামেলা ছাড়া।
- **ফাইলসিস্টেম ড্রাইভার:** জানে ফাইলের নাম → inode → ডিস্কের কোন block-এ ডেটা। (inode-এর বিস্তারিত অধ্যায় ১১-এ।)
- **Page cache:** একবার পড়া ডেটা RAM-এ রেখে দেয়, পরের বার ডিস্কে না গিয়েই দেয়। লেখার সময়ও ডেটা আগে RAM-এ যায়, পরে ডিস্কে লেখা হয় ("dirty pages")। এজন্যই পেনড্রাইভ টেনে খোলার আগে `sync` / `umount` জরুরি।
- **Block layer:** ডিস্ককে দেখে কতগুলো নির্দিষ্ট মাপের ব্লকের (সাধারণত 512 বা 4096 byte sector) সারি হিসেবে। I/O request গুলো জোড়া লাগায়, ক্রম সাজায়।

- **Device driver:** আসল হার্ডওয়্যারের সাথে কথা বলে।

> 💡 **টিপস:** কোন ডিস্কে কোন I/O scheduler চলছে দেখতে: `cat /sys/block/sda/queue/scheduler` — বন্ধনীর ভেতরেরটা সক্রিয়, যেমন `[mq-deadline] none`। NVMe-তে সাধারণত `[none]`।

Block device আর character device

লিনাক্সে "সবকিছুই ফাইল", তাই ডিস্কও `/dev`-এর ভেতরে একটা ফাইল হিসেবে দেখা যায়।

- **Block device** (`b`): ডেটা নির্দিষ্ট মাপের ব্লকে পড়া-লেখা হয়, যেকোনো জায়গায় লাফিয়ে যাওয়া যায় (random access)। ডিস্ক, পার্টিশন, USB stick।
- **Character device** (`c`): ডেটা বাইটের স্রোতের মতো আসে। কীবোর্ড, টার্মিনাল (`/dev/tty`), `/dev/null`।

```
$ ls -l /dev/sda /dev/sda1 /dev/null
brw-rw---- 1 root disk 8, 0 Nov 12 09:10 /dev/sda
brw-rw---- 1 root disk 8, 1 Nov 12 09:10 /dev/sda1
crw-rw-rw- 1 root root 1, 3 Nov 12 09:10 /dev/null
```

প্রথম অক্ষর `b` মানে block, `c` মানে character। সাইজের জায়গায় যে `8, 0` দেখছ, তা হলো **major** ও **minor number**। major number বলে কোন ড্রাইভার (8 = `sd` ড্রাইভার), minor বলে কোন নির্দিষ্ট ডিভাইস (0 = পুরো ডিস্ক sda, 1 = প্রথম পার্টিশন)। `/dev`-এর এই ফাইলগুলো বানায় **udev** (systemd-udevd), হার্ডওয়্যার চিনতে পারার সাথে সাথে।

৩২.৩ Block device-এর নামকরণ: sda, nvme0n1, vda ...

নাম দেখে বুঝে নিতে পারবে কোন ধরনের ডিস্ক। এটা খুব কাজের দক্ষতা — ভুল ডিস্ক ফরম্যাট করা এড়ানোর প্রথম ধাপ।

নাম	কী ধরনের ডিভাইস	পার্টিশনের নাম
`sda`, `sdb`, `sdc` ...	SATA/SAS HDD-SSD, USB stick, SCSI — `sd` ড্রাইভার	`sda1`, `sda2`, `sdb1`
`nvme0n1`, `nvme1n1`	NVMe SSD (M.2)	`nvme0n1p1`, `nvme0n1p2`
`vda`, `vdb`	KVM/QEMU ভার্চুয়াল মেশিনের virtio ডিস্ক	`vda1`, `vda2`
`xvda`	Xen ভার্চুয়াল ডিস্ক (পুরোনো AWS EC2)	`xvda1`
`mmcblk0`	SD কার্ড / eMMC (Raspberry Pi, ল্যাপটপের কার্ড রিডার)	`mmcblk0p1`

নাম	কী ধরনের ডিভাইস	পার্টিশনের নাম
`sr0`	CD/DVD ড্রাইভ	—
`loop0`, `loop1`	loop device — একটা সাধারণ ফাইলকে ডিস্ক বানানো (snap প্যাকেজ এভাবেই মাউন্ট হয়)	`loop0p1`
`dm-0`, `dm-1`	device-mapper (LVM, LUKS এনক্রিপশন)	—
`md0`, `md127`	সফটওয়্যার RAID	`md0p1`
`zram0`	RAM-এর ভেতরে কম্প্রেসড swap (Fedora-তে ডিফল্ট)	—

`sd` নামের নিয়ম: প্রথম যে ডিস্ক পাওয়া যায় সেটা `sda`, পরেরটা `sdb`... `sdz`-এর পর `sdaa`। পার্টিশন নম্বর শুধু শেষে যোগ হয়।

NVMe নামের নিয়ম: `nvme0n1p2` ভাঙলে —

```
nvme 0  n1  p2
 |   |   |   └── partition 2
 |   |   └────── namespace 1 (একটা NVMe ড্রাইভকে কয়েকটা namespace-এ ভাগ করা যায়)
 |   └────────── controller 0 (প্রথম NVMe কন্ট্রোলার)
 └────────────── NVMe ডিভাইস
```

নাম যখন সংখ্যায় শেষ হয় (`nvme0n1`, `mmcblk0`, `loop0`), তখন পার্টিশনের আগে একটা `p` বসে, যাতে `nvme0n11` নিয়ে বিভ্রান্তি না হয়।

> ⚠️ **সতর্কতা:** `sda` / `sdb` নামগুলো **স্থায়ী নয়**। আজ যেটা `sdb`, কাল একটা USB লাগিয়ে বুট করলে সেটা `sdc` হয়ে যেতে পারে — কোন ডিস্ক আগে সাড়া দিল তার ওপর নির্ভর করে। তাই `/etc/fstab`-এ কখনো `/dev/sdb1` লিখবে না, UUID লিখবে (৩২.১০ দেখো)।

স্থায়ী নাম: `/dev/disk/by-*`

udev প্রতিটা ডিস্কের জন্য কিছু স্থায়ী symlink বানায় (symlink কী, দেখো অধ্যায় ১১):

```
$ ls -l /dev/disk/by-uuid/
lrwxrwxrwx 1 root root 10 Nov 12 09:10 3f2a9c1e-7b4d-4e8a-9d61-2c5b8e0f1a77 ->
../../sda2
lrwxrwxrwx 1 root root 10 Nov 12 09:10 A1B2-C3D4 -> ../../sda1
```

ডিরেক্টরি	কীসের ভিত্তিতে নাম
`/dev/disk/by-uuid/`	ফাইলসিস্টেমের UUID (mkfs করার সময় তৈরি হয়)
`/dev/disk/by-partuuid/`	পার্টিশনের UUID (GPT টেবিলে থাকে, ফরম্যাট করলেও বদলায় না)
`/dev/disk/by-label/`	ফাইলসিস্টেমের লেবেল (যেমন `DATA`)
`/dev/disk/by-id/`	হার্ডওয়্যারের মডেল + সিরিয়াল নম্বর

ডিরেক্টরি	কীসের ভিত্তিতে নাম
`/dev/disk/by-path/`	কোন পোর্ট/কন্ট্রোলারে লাগানো

৩২.৪ `lsblk` — কোন ডিস্কে কী আছে এক নজরে

`lsblk` (list block devices) হলো ডিস্কের কাজে তোমার প্রথম বন্ধু। এটা `/sys` থেকে তথ্য পড়ে, তাই সাধারণ ইউজার হিসেবেও চলে।

Syntax: `lsblk [options] [device...]`

```
$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0         7:0    0  74.3M  1 loop /snap/core22/1663
sda           8:0    1  14.9G  0 disk
└─sda1        8:1    1  14.9G  0 part /media/rahim/PENDRIVE
nvme0n1     259:0    0 476.9G  0 disk
├─nvme0n1p1 259:1    0     1G  0 part /boot/efi
├─nvme0n1p2 259:2    0     2G  0 part /boot
└─nvme0n1p3 259:3    0 473.9G  0 part /
```

কলামগুলোর মানে:

কলাম	মানে
`NAME`	ডিভাইসের নাম; গাছের মতো দেখায় কোন পার্টিশন কোন ডিস্কের
`MAJ:MIN`	major:minor number
`RM`	removable কিনা (1 = হ্যাঁ, যেমন USB)
`SIZE`	আকার
`RO`	read-only কিনা (1 = হ্যাঁ)
`TYPE`	`disk`, `part`, `loop`, `lvm`, `crypt`, `rom`, `raid1` ইত্যাদি
`MOUNTPOINTS`	কোথায় মাউন্ট করা (পুরোনো ভার্সনে কলামের নাম `MOUNTPOINT`)

গুরুত্বপূর্ণ অপশন:

অপশন	কাজ
`-f`	ফাইলসিস্টেম তথ্য: FSTYPE, LABEL, UUID, খালি জায়গা
`-o NAME,SIZE,FSTYPE,UUID`	নিজের পছন্দের কলাম
`-p`	পুরো পাথ দেখাবে (`/dev/sda1`)
`-d`	শুধু ডিস্ক, পার্টিশন বাদ
`-b`	সাইজ byte-এ

অপশন	কাজ
`-e 7`	major 7 (loop) বাদ দাও — snap থাকলে তালিকা পরিষ্কার হয়
`-S`	SCSI/SATA ডিভাইসের তথ্য (মডেল, ভেন্ডর, ট্রান্সপোর্ট)
`-J`	JSON আউটপুট (স্ক্রিপ্টে কাজে লাগে)

```
$ lsblk -f -e 7
NAME        FSTYPE FSVER LABEL    UUID                                 FSAVAIL FSUSE%
MOUNTPOINTS
sda
└─sda1      vfat   FAT32 PENDRIVE 5E3C-91AF                               9.1G    39%
/media/rahim/PENDRIVE
nvme0n1
├─nvme0n1p1 vfat   FAT32          A1B2-C3D4                             1G     1%
/boot/efi
├─nvme0n1p2 ext4   1.0            7c0e5d2a-1f3b-4a6e-8c9d-0b1a2e3f4d5c    1.6G    11%
/boot
└─nvme0n1p3 ext4   1.0            3f2a9c1e-7b4d-4e8a-9d61-2c5b8e0f1a77  380.2G    14% /
```

> 💡 **টিপস:** নতুন ডিস্ক বা USB লাগানোর **আগে** একবার `lsblk` চালাও, **পরে** আরেকবার। যেটা নতুন এসেছে, সেটাই তোমার ডিভাইস। এই অভ্যাস তোমাকে ভুল ডিস্ক মুছে ফেলা থেকে বাঁচাবে।

৩২.৫ `blkid` ও `findmnt` — UUID, টাইপ আর মাউন্টের খোঁজ

`blkid`

`blkid` ডিস্কের ভেতরে ফাইলসিস্টেমের "সিগনেচার" পড়ে টাইপ, UUID, লেবেল বলে দেয়।

```
$ sudo blkid
/dev/nvme0n1p1: UUID="A1B2-C3D4" BLOCK_SIZE="512" TYPE="vfat" PARTLABEL="EFI System
Partition" PARTUUID="9b3c1d2e-..."
/dev/nvme0n1p3: UUID="3f2a9c1e-7b4d-4e8a-9d61-2c5b8e0f1a77" BLOCK_SIZE="4096"
TYPE="ext4" PARTUUID="c4d5e6f7-..."
/dev/sda1: LABEL="PENDRIVE" UUID="5E3C-91AF" BLOCK_SIZE="512" TYPE="vfat"
PARTUUID="0a1b2c3d-01"
```

অপশন/ব্যবহার	কাজ
`blkid /dev/sdb1`	একটা নির্দিষ্ট ডিভাইস
`blkid -s UUID -o value /dev/sdb1`	শুধু UUID-এর মানটা (স্ক্রিপ্টে দারুণ)
`blkid -L DATA`	লেবেল দিয়ে ডিভাইস খোঁজা
`blkid -U <uuid>`	UUID দিয়ে ডিভাইস খোঁজা
`blkid -o list`	টেবিল আকারে

> 📝 **মনে রাখো:** দুই রকম UUID আছে। `UUID=` হলো **ফাইলসিস্টেমের** (আবার mkfs করলে বদলে যায়)। `PARTUUID=` হলো **পার্টিশনের** (GPT টেবিলে লেখা; MBR-এ এটা ডিস্ক-আইডি + পার্টিশন নম্বর, যেমন `0a1b2c3d-01`)। FAT ফাইলসিস্টেমের UUID আসলে ছোট একটা "volume serial" (`5E3C-91AF`)।

`findmnt`

কী কোথায় মাউন্ট আছে, গাছের মতো দেখায়:

```
$ findmnt /
TARGET SOURCE         FSTYPE OPTIONS
/      /dev/nvme0n1p3 ext4   rw,relatime,errors=remount-ro

$ findmnt -t ext4,vfat
TARGET     SOURCE         FSTYPE OPTIONS
/          /dev/nvme0n1p3 ext4   rw,relatime,errors=remount-ro
├─/boot    /dev/nvme0n1p2 ext4   rw,relatime
└─/boot/efi /dev/nvme0n1p1 vfat  rw,relatime,fmask=0077,dmask=0077,...
```

অপশন	কাজ
`findmnt`	সব মাউন্ট, গাছ আকারে
`findmnt /mnt/data`	নির্দিষ্ট mount point
`findmnt -t xfs`	নির্দিষ্ট ফাইলসিস্টেম টাইপ
`findmnt --verify`	`/etc/fstab`-এ ভুল আছে কিনা যাচাই (খুব গুরুত্বপূর্ণ!)
`findmnt -s`	`/etc/fstab`-এ যা লেখা আছে তা দেখায়
`findmnt -D`	`df`-এর মতো আউটপুট

৩২.৬ পার্টিশন টেবিল: MBR বনাম GPT

পার্টিশন বানানোর আগে ডিস্কে একটা "সূচিপত্র" লাগে — কোন পার্টিশন কোন sector থেকে কোথায় শেষ। এটাই **partition table** বা **disklabel**। দুই রকম আছে।

MBR (Master Boot Record) — পুরোনো পদ্ধতি

১৯৮৩ সাল থেকে চলছে (IBM PC DOS 2.0)। ডিস্কের একেবারে প্রথম 512 byte-এ থাকে:

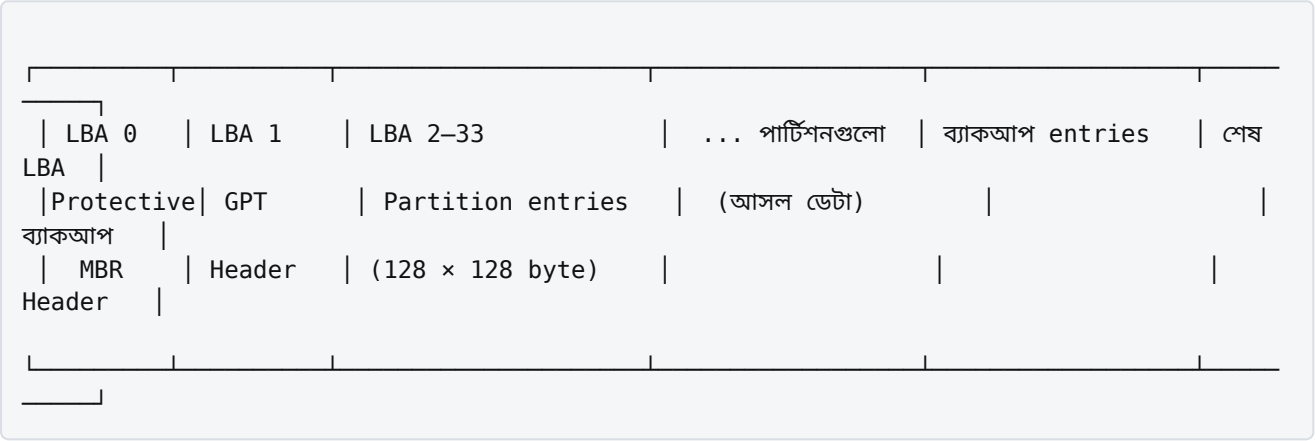
```
ডিস্কের sector 0 (512 byte)
┌──────────────────────────────┬──────────────────┬──────────┐
│  Bootstrap code (446 byte)   │ Partition table  │ 0x55AA   │
│  (BIOS এটা চালায়)           │ 4 × 16 = 64 byte │ 2 byte   │
└──────────────────────────────┴──────────────────┴──────────┘
                                ↑ সর্বোচ্চ ৪টা এন্ট্রি
```

MBR-এর সীমাবদ্ধতা:

- সর্বোচ্চ **৪টা primary** পার্টিশন। বেশি লাগলে ১টাকে **extended** বানিয়ে তার ভেতরে অনেকগুলো **logical** পার্টিশন (লিনাক্সে এরা `sda5` থেকে শুরু হয়)।
- sector-এর ঠিকানা 32-bit, তাই 512-byte sector-এ সর্বোচ্চ **2 TiB** ডিস্ক।
- টেবিলের একটাই কপি — নষ্ট হলে সব পার্টিশন হারানোর ঝুঁকি।
- কোনো checksum নেই।

GPT (GUID Partition Table) — আধুনিক পদ্ধতি

UEFI স্ট্যান্ডার্ডের অংশ। আজকের সব নতুন কম্পিউটারে এটাই ডিফল্ট।



- **Protective MBR:** পুরোনো টুল যেন ডিস্ককে "খালি" ভেবে মুছে না দেয়, তাই সামনে একটা নকল MBR থাকে যা বলে "পুরো ডিস্ক একটাই পার্টিশন, টাইপ 0xEE"।
- ডিফল্টে **১২৮টা** পার্টিশন, extended/logical-এর ঝামেলা নেই।
- 64-bit ঠিকানা — কার্যত সীমাহীন আকার (512-byte sector-এ ৮ ZiB)।
- **ডিস্কের শেষে ব্যাকআপ** হেডার ও টেবিল থাকে, আর **CRC32 checksum** দিয়ে নষ্ট হলে ধরা পড়ে।
- প্রতিটা পার্টিশনের একটা নিজস্ব GUID (PARTUUID), একটা টাইপ GUID ও একটা নাম (PARTLABEL) থাকে।

বিষয়	MBR	GPT
সর্বোচ্চ ডিস্ক আকার	2 TiB (512-byte sector)	কার্যত সীমাহীন
পার্টিশন সংখ্যা	৪ primary (extended দিয়ে বেশি)	ডিফল্ট ১২৮
ব্যাকআপ টেবিল	নেই	আছে (ডিস্কের শেষে)
Checksum	নেই	CRC32
বুট পদ্ধতি	Legacy BIOS	UEFI (BIOS+GRUB দিয়েও সম্ভব, BIOS boot partition লাগে)
`fdisk`-এ বানানোর কমান্ড	`o`	`g`

বিষয়	MBR	GPT
কবে ব্যবহার করবে	পুরোনো মেশিন, খুব পুরোনো ডিভাইসের সাথে সামঞ্জস্য	প্রায় সবসময়

> 💡 **টিপস:** কোন ডিস্কে কোন টেবিল আছে দেখতে: `sudo fdisk -l /dev/sda` চালিয়ে `Disklabel type: gpt` বা `dos` দেখো (`dos` মানে MBR)। অথবা `lsblk -o NAME,PTTYPE`।

বুট প্রসেসে MBR/GPT ও UEFI কীভাবে কাজ করে তা বিস্তারিত আছে অধ্যায় ২৭-এ।

সাধারণ পার্টিশন টাইপ

ব্যবহার	MBR টাইপ কোড	GPT (`gdisk` কোড)	`fdisk` -এ alias
Linux filesystem	`83`	`8300`	`linux`
Linux swap	`82`	`8200`	`swap`
Linux LVM	`8e`	`8e00`	`lvm`
Linux RAID	`fd`	`fd00`	`raid`
EFI System Partition	`ef`	`ef00`	`uefi`
BIOS boot (GPT+BIOS-এ GRUB-এর জন্য)	—	`ef02`	—
Windows NTFS/exFAT/FAT32 ডেটা	`07` / `0c`	`0700`	—

> 📝 **মনে রাখো:** পার্টিশনের "টাইপ" শুধু একটা লেবেল বা ইঙ্গিত। আসল ফাইলসিস্টেম কী হবে তা ঠিক হয় `mkfs`-এ। টাইপ `8300` রেখে তার ভেতরে XFS বানানো একদম স্বাভাবিক।

৩২.৭ পার্টিশন তৈরি — `fdisk`, `gdisk`, `parted`

> ⚠️ **সতর্কতা — পড়ো, তারপর এগোও:** পার্টিশন টুল ভুল ডিস্কে চালালে **সব ডেটা হারাতে পারো**, আর তা ফেরত আনা প্রায় অসম্ভব। নিয়ম তিনটা: (১) `lsblk` দিয়ে ডিস্কের নাম ও আকার মিলিয়ে নাও; (২) মাউন্ট করা ডিস্কে কাজ কোরো না; (৩) চর্চার জন্য loop device ব্যবহার করো (হাতে-কলমে অংশে দেখানো আছে)। `fdisk` আর `gdisk` তোমার পরিবর্তন `w` চাপার আগ পর্যন্ত ডিস্কে লেখে না — কিন্তু `parted` প্রতিটা কমান্ড **সাথে সাথেই** ডিস্কে লেখে!

`fdisk` — সবচেয়ে প্রচলিত

`util-linux` প্যাকেজের অংশ, সব distro-তে থাকে। আধুনিক `fdisk` MBR ও GPT দুটোই পারে।

শুধু দেখা:

```
$ sudo fdisk -l /dev/sdb
Disk /dev/sdb: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

কোনো `Disklabel type` লাইন না থাকলে ডিস্কটা একদম খালি।

ইন্টারঅ্যাকটিভ মোড: `sudo fdisk /dev/sdb`। ভেতরে এক অক্ষরের কমান্ড চলে:

কী	কাজ
`m`	সাহায্য (সব কমান্ডের তালিকা)
`p`	বর্তমান পার্টিশন টেবিল দেখাও
`g`	নতুন খালি **GPT** টেবিল বানাও
`o`	নতুন খালি **MBR (DOS)** টেবিল বানাও
`n`	নতুন পার্টিশন
`d`	পার্টিশন মুছে ফেলো
`t`	পার্টিশনের টাইপ বদলাও
`l`	সব টাইপের তালিকা
`F`	খালি (unpartitioned) জায়গা দেখাও
`v`	টেবিল যাচাই করো
`x`	expert মোড
`w`	পরিবর্তন ডিস্কে **লিখে** বের হও
`q`	কিছু না লিখে বের হও (ভুল হলে এটাই বাঁচার উপায়!)

ধাপে ধাপে উদাহরণ: একটা 20 GiB ডিস্ক `/dev/sdb`-তে GPT টেবিল, একটা 15 GiB ডেটা পার্টিশন আর একটা 2 GiB swap পার্টিশন।

```
$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.39.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x5c1d2e3f.

Command (m for help): g
Created a new GPT disklabel (GUID: 6A0B...-...).

Command (m for help): n
```

```
Partition number (1-128, default 1):              ← Enter (ডিফল্ট 1)
First sector (2048-41943006, default 2048):       ← Enter
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-41943006, default 41940991): +15G

Created a new partition 1 of type 'Linux filesystem' and of size 15 GiB.

Command (m for help): n
Partition number (2-128, default 2):
First sector (31459328-41943006, default 31459328):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (31459328-41943006, default 41940991):
+2G

Created a new partition 2 of type 'Linux filesystem' and of size 2 GiB.

Command (m for help): t
Partition number (1,2, default 2): 2
Partition type or alias (type L to list all): swap

Changed type of partition 'Linux filesystem' to 'Linux swap'.

Command (m for help): p
Disk /dev/sdb: 20 GiB, 21474836480 bytes, 41943040 sectors
...
Disklabel type: gpt
Disk identifier: 6A0B...

Device        Start      End  Sectors Size Type
/dev/sdb1      2048 31459327 31457280  15G Linux filesystem
/dev/sdb2  31459328 35653631  4194304   2G Linux swap

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

কয়েকটা জিনিস খেয়াল করো:

- **First sector 2048 কেন?** 2048 × 512 = 1 MiB। পার্টিশনকে 1 MiB সীমানায় শুরু করলে আধুনিক SSD ও 4K-sector ডিস্কে পারফরম্যান্স ভালো থাকে (**alignment**)। fdisk নিজেই এটা করে, তুমি শুধু Enter চাপো।
- `+15G` মানে "শুরু থেকে আরও 15 GiB"। `+500M`, `+1T` ইত্যাদিও চলে। শুধু Enter চাপলে বাকি সব জায়গা নেবে।
- বাকি ~3 GiB খালি থাকল — পরে কাজে লাগাতে পারবে।

fdisk-এর স্ক্রিপ্টেবল ভাই `sfdisk`: একটা লাইনে পুরো ডিস্ক পার্টিশন করা যায়, অটোমেশনে কাজে লাগে:

```
$ sudo sfdisk --dump /dev/sdb > sdb-table.txt     # টেবিলের ব্যাকআপ
$ sudo sfdisk /dev/sdb < sdb-table.txt            # ফেরত বসানো
```

> 💡 **টিপস:** কোনো ডিস্কে কাজ শুরুর আগে `sfdisk --dump` দিয়ে টেবিলের ব্যাকআপ রাখো। ভুলে পার্টিশন

> মুছে ফেললেও (ডেটা ওভাররাইট না হলে) টেবিল ফেরত বসিয়ে সব উদ্ধার করা যায়।

পার্টিশন টেবিল নতুন করে পড়ানো

`w` দেওয়ার পর kernel সাধারণত নতুন টেবিল নিজেই পড়ে নেয়। কিন্তু ডিস্কের কোনো পার্টিশন ব্যবহারে থাকলে বার্তা আসতে পারে: *"Re-reading the partition table failed.: Device or resource busy"*। তখন:

```
$ sudo partprobe /dev/sdb        # parted প্যাকেজের অংশ
$ sudo partx -u /dev/sdb         # util-linux-এর বিকল্প
```

এতেও না হলে রিবুট।

`gdisk` — GPT বিশেষজ্ঞ

`gdisk` (GPT fdisk) শুধু GPT-র জন্য, `fdisk`-এর মতোই ইন্টারঅ্যাকটিভ। Ubuntu/Debian-এ `sudo apt install gdisk`, Fedora/RHEL-এ `sudo dnf install gdisk`।

কী	কাজ
`?`	সাহায্য
`p`	টেবিল দেখাও
`n`	নতুন পার্টিশন
`d`	মুছে ফেলো
`t`	টাইপ বদলাও (৪-অক্ষরের হেক্স কোড)
`L`	টাইপ কোডের তালিকা
`c`	পার্টিশনের নাম (PARTLABEL) বদলাও
`o`	নতুন খালি GPT
`v`	যাচাই
`r`	recovery ও transformation মেনু (MBR→GPT রূপান্তরসহ)
`w`	লিখে বের হও
`q`	না লিখে বের হও

```
$ sudo gdisk /dev/sdb
GPT fdisk (gdisk) version 1.0.10
...
Command (? for help): n
Partition number (1-128, default 1):
First sector (34-41943006, default = 2048) or {+-}size{KMGTP}:
Last sector (2048-41943006, default = 41940991) or {+-}size{KMGTP}: +10G
Current type is 8300 (Linux filesystem)
Hex code or GUID (L to show codes, Enter = 8300): 8e00
Changed type of partition to 'Linux LVM'
```

```
Command (? for help): w

Final checks complete. About to write GPT data. THIS WILL OVERWRITE EXISTING
PARTITIONS!!

Do you want to proceed? (Y/N): Y
OK; writing new GUID partition table (GPT) to /dev/sdb.
The operation has completed successfully.
```

`gdisk` এর একটা বড় সুবিধা: এটা MBR ডিস্ক খুললে ডেটা না মুছেই GPT-তে রূপান্তরের প্রস্তাব দেয় (তবুও আগে ব্যাকআপ রাখবে)। আর `sgdisk` হলো এর স্ক্রিপ্টেবল রূপ, যেমন `sudo sgdisk -n 1:0:+10G -t 1:8300 /dev/sdb`।

`parted` — বড় ডিস্ক ও স্ক্রিপ্টিং

GNU `parted` MBR ও GPT দুটোই পারে, আর সরাসরি কমান্ড লাইন থেকে চালানো যায়। GUI টুল **GParted** এরই ওপর তৈরি।

> ⚠️ **সতর্কতা:** `parted`-এ কোনো "w" নেই — `mklabel`, `mkpart`, `rm` চাপামাত্রই ডিস্কে লেখা হয়ে যায়।

কমান্ড (parted-এর ভেতরে)	কাজ
`help`	সাহায্য
`print`	টেবিল দেখাও
`print free`	খালি জায়গাসহ দেখাও
`mklabel gpt` / `mklabel msdos`	নতুন টেবিল (msdos = MBR)
`mkpart NAME FSTYPE START END`	নতুন পার্টিশন (GPT-তে NAME, MBR-এ `primary` / `logical`)
`rm N`	N নম্বর পার্টিশন মোছো
`resizepart N END`	পার্টিশনের শেষ সীমা বদলাও (শুধু পার্টিশন, ফাইলসিস্টেম না!)
`set N esp on`	flag সেট (`esp`, `boot`, `lvm`, `raid`, `swap`)
`name N NAME`	GPT পার্টিশনের নাম
`unit MiB`	একক বদলাও (`s`, `MiB`, `GiB`, `%`)
`align-check optimal N`	alignment ঠিক আছে কিনা
`quit`	বের হও

এক লাইনে (non-interactive, `-s` = script mode):

```
$ sudo parted -s /dev/sdc mklabel gpt
$ sudo parted -s -a optimal /dev/sdc mkpart data ext4 1MiB 100%
$ sudo parted /dev/sdc print
Model: ATA VBOX HARDDISK (scsi)
```

```
Disk /dev/sdc: 10.7GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start   End     Size    File system  Name  Flags
 1      1049kB  10.7GB  10.7GB               data
```

`mkpart` -এ যে `ext4` লিখলাম, সেটা শুধু টাইপের ইঙ্গিত — parted ফাইলসিস্টেম **বানায় না**। `File system` কলাম খালি, কারণ এখনো mkfs হয়নি। `1MiB 100%` মানে 1 MiB থেকে শুরু করে ডিস্কের শেষ পর্যন্ত — alignment-এর জন্য সবচেয়ে নিরাপদ লেখা।

> 💡 **টিপস:** parted-এ `GB` (1000^3) আর `GiB` (1024^3) আলাদা। দ্বিধা এড়াতে সবসময় `MiB` / `GiB` বা `%` ব্যবহার করো।

কোন টুল কখন?

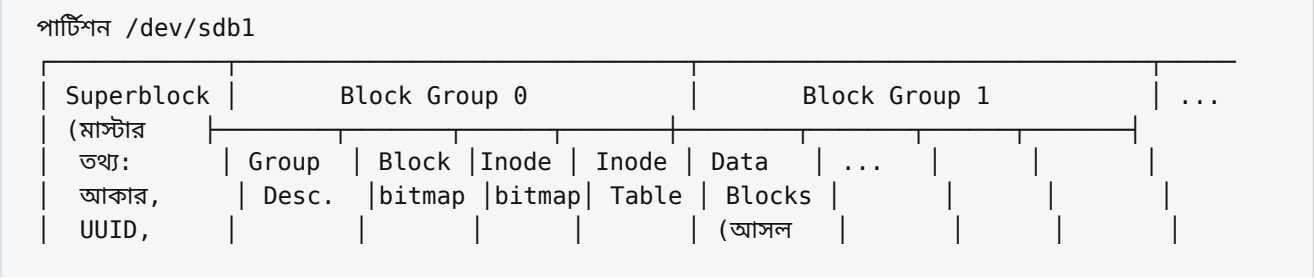
পরিস্থিতি	টুল
সাধারণ কাজ, MBR বা GPT	`fdisk`
শুধু GPT, টাইপ কোড/নাম নিয়ে সূক্ষ্ম কাজ, MBR→GPT রূপান্তর	`gdisk`
স্ক্রিপ্ট, পার্টিশন বড় করা, এক লাইনে কাজ	`parted`, `sfdisk`, `sgdisk`
মাউস দিয়ে, ডেস্কটপে	GParted, GNOME Disks
পুরোনো সিগনেচার মুছে ডিস্ক পরিষ্কার	`wipefs`

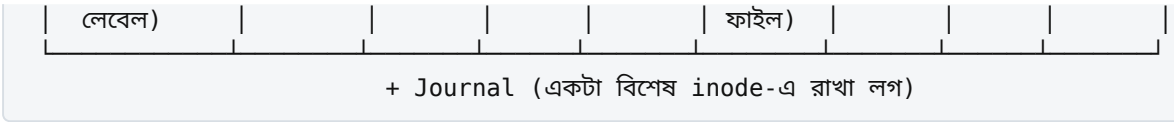
```
$ sudo wipefs /dev/sdb          # কী কী সিগনেচার আছে দেখাও (কিছু মোছে না)
$ sudo wipefs -a /dev/sdb       # সব সিগনেচার মুছে দাও — সাবধান!
```

৩২.৮ ফাইলসিস্টেম তৈরি — `mkfs`

পার্টিশন হলো ফাঁকা তাক। তাতে জিনিস রাখার আগে "সাজানোর পদ্ধতি" বসাতে হবে — এটাই **ফরম্যাট করা** বা **ফাইলসিস্টেম তৈরি**।

ফাইলসিস্টেম ভেতরে দেখতে কেমন (ext4 উদাহরণ)





- **Superblock:** পুরো ফাইলসিস্টেমের "আইডি কার্ড" — মোট আকার, block size, UUID, লেবেল, কতবার মাউন্ট হয়েছে। এত গুরুত্বপূর্ণ যে কয়েকটা block group-এ এর ব্যাকআপ কপি থাকে।
- **Bitmap:** কোন block/inode খালি, কোনটা ভরা — হ্যাঁ/না-এর লম্বা তালিকা।
- **Inode table:** প্রতিটা ফাইলের মেটাডেটা (মালিক, পারমিশন, সময়, কোন block-এ ডেটা)।
- **Journal:** বিদ্যুৎ চলে গেলে ফাইলসিস্টেম যাতে এলোমেলো না হয়। কোনো পরিবর্তন করার আগে ext4 সেটা জার্নালে লিখে রাখে ("এখন এই ফাইলের এই অংশ বদলাব")। ক্র্যাশের পর পুরো ডিস্ক না ঘেঁটে শুধু জার্নাল দেখে অসমাপ্ত কাজ শেষ বা বাতিল করে। উপমা: দোকানদার মাল বের করার আগে খাতায় লিখে রাখে — হঠাৎ ঝামেলা হলে খাতা দেখেই বোঝা যায় কোথায় থেমেছিল।

জনপ্রিয় ফাইলসিস্টেমের তুলনা

ফাইলসিস্টেম	কোথায় ডিফল্ট	শক্তি	দুর্বলতা/লক্ষ্য রাখো
ext4	Ubuntu, Debian	পরীক্ষিত, স্থিতিশীল, বড়-ছোট দুটোই করা যায় (shrink অফলাইনে)	inode সংখ্যা তৈরির সময় ঠিক হয়ে যায়
XFS	RHEL, Rocky, AlmaLinux, CentOS Stream	বড় ফাইল ও সমান্তরাল I/O-তে দ্রুত	**ছোট (shrink) করা যায় না**, শুধু বড় করা যায়
Btrfs	Fedora Workstation, openSUSE	snapshot, compression, subvolume, checksum	জটিল; RAID5/6 এখনো সুপারিশকৃত নয়
vfat (FAT32)	EFI পার্টিশন, পুরোনো পেনড্রাইভ	সব OS-এ চলে	একক ফাইল সর্বোচ্চ 4 GiB − 1 byte, পারমিশন নেই
exFAT	বড় পেনড্রাইভ, SD কার্ড	সব OS-এ চলে, 4 GiB-এর বেশি ফাইল	লিনাক্সে পারমিশন নেই, জার্নাল নেই
NTFS	উইন্ডোজ	উইন্ডোজের সাথে শেয়ার	লিনাক্সে `ntfs3` (kernel 5.15+) বা `ntfs-3g` ড্রাইভার লাগে
tmpfs	`/tmp` (Fedora, Debian 13), `/run`	RAM-এ থাকে, খুব দ্রুত	রিবুটে সব মুছে যায়

`mkfs` কীভাবে কাজ করে

`mkfs` আসলে একটা "সামনের দরজা"। `mkfs -t ext4 /dev/sdb1` লিখলে সে ভেতরে `mkfs.ext4` চালায়। দেখো কোনগুলো ইনস্টল আছে:

```
$ ls /usr/sbin/mkfs.*
/usr/sbin/mkfs.bfs  /usr/sbin/mkfs.ext2  /usr/sbin/mkfs.ext4  /usr/sbin/mkfs.minix
/usr/sbin/mkfs.btrfs /usr/sbin/mkfs.ext3 /usr/sbin/mkfs.fat   /usr/sbin/mkfs.vfat
/usr/sbin/mkfs.cramfs /usr/sbin/mkfs.exfat /usr/sbin/mkfs.msdos /usr/sbin/mkfs.xfs
```

কোনো একটা না থাকলে প্যাকেজ ইনস্টল করো (অধ্যায় ৩০):

ফাইলসিস্টেম	Debian/Ubuntu প্যাকেজ	Fedora/RHEL প্যাকেজ
ext4	`e2fsprogs`	`e2fsprogs`
XFS	`xfsprogs`	`xfsprogs`
vfat	`dosfstools`	`dosfstools`
exFAT	`exfatprogs`	`exfatprogs`
Btrfs	`btrfs-progs`	`btrfs-progs`
NTFS	`ntfs-3g`	`ntfs-3g` (RHEL-এ EPEL থেকে)

`mkfs.ext4`

```
$ sudo mkfs.ext4 -L DATA /dev/sdb1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 3932160 4k blocks and 983040 inodes
Filesystem UUID: 8d4f2b6a-3c1e-4f7a-9b2d-5e6f7a8b9c0d
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

আউটপুটে superblock-এর ব্যাকআপ কোথায় আছে লেখা — মূল superblock নষ্ট হলে এগুলো কাজে লাগে (৩২.১৪ দেখো)।

অপশন	কাজ
`-L LABEL`	লেবেল দাও (সর্বোচ্চ 16 অক্ষর)
`-m 1`	root-এর জন্য সংরক্ষিত জায়গা % (ডিফল্ট 5%)
`-b 4096`	block size (byte)
`-i 16384`	প্রতি কত byte-এ একটা inode (কম মান = বেশি inode, অনেক ছোট ফাইলের জন্য)
`-N 2000000`	সরাসরি inode সংখ্যা
`-T largefile`	ব্যবহার-ধরন অনুযায়ী প্রিসেট (`small` , `largefile` , `largefile4`)
`-U UUID`	নির্দিষ্ট UUID বসাও
`-n`	কিছু না বানিয়ে শুধু দেখাও কী করত (dry run)
`-F`	জোর করে (কিছু সতর্কতা উপেক্ষা)

> 💡 **টিপস:** 5% reserved জায়গা কেন? ডিস্ক পুরো ভরে গেলেও root যেন লগ লিখতে ও সিস্টেম ঠিক করতে পারে, আর ফ্র্যাগমেন্টেশন কম হয়। কিন্তু 4 TB-এর শুধু-ডেটা ডিস্কে 5% মানে 200 GB নষ্ট — সেখানে `-m 1` বা পরে `tune2fs -m 1` দাও। root (`/`) পার্টিশনে কমিয়ো না।

`mkfs.xfs`

```
$ sudo mkfs.xfs -L backup /dev/sdc1
meta-data=/dev/sdc1              isize=512    agcount=4, agsize=655296 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=1
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
data     =                       bsize=4096   blocks=2621184, imaxpct=25
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=16384, version=2
realtime =none                   extsz=4096   blocks=0, rtextents=0
```

অপশন	কাজ
`-L label`	লেবেল (সর্বোচ্চ **12** অক্ষর)
`-f`	আগে কোনো ফাইলসিস্টেম থাকলেও ওভাররাইট করো
`-b size=4096`	block size

`agcount=4` — XFS ডিস্ককে কয়েকটা **allocation group**-এ ভাগ করে, প্রতিটা স্বাধীনভাবে কাজ করতে পারে। এজন্যই অনেক CPU একসাথে লিখলে XFS দ্রুত।

> ⚠️ **সতর্কতা:** XFS ছোট করা যায় না। পরে জায়গা কমাতে হলে ব্যাকআপ → নতুন ফাইলসিস্টেম → রিস্টোর ছাড়া উপায় নেই। বড় করতে `xfs_growfs /mountpoint` (মাউন্ট অবস্থায়)।

`mkfs.vfat` ও `mkfs.exfat`

```
$ sudo mkfs.vfat -F 32 -n PENDRIVE /dev/sdd1
mkfs.fat 4.2 (2021-01-31)

$ sudo mkfs.exfat -n BIGUSB /dev/sdd1
exfatprogs version : 1.2.2
Creating exFAT filesystem(/dev/sdd1, cluster size=131072)
...
exFAT format complete!
```

অপশন (`mkfs.vfat`)	কাজ
`-F 32`	FAT32 (অন্যথায় আকার দেখে FAT12/16/32 ঠিক করে)
`-n LABEL`	লেবেল (সর্বোচ্চ 11 অক্ষর, সাধারণত বড় হাতের)

> ⚠️ **সতর্কতা:** `mkfs` চালালে পার্টিশনের আগের সব ডেটা কার্যত হারিয়ে যায়। চালানোর আগে আরেকবার `lsblk -f` দিয়ে নিশ্চিত হও — বিশেষ করে `/dev/sdb` (পুরো ডিস্ক) আর `/dev/sdb1` (পার্টিশন) গুলিয়ে ফেলো না।

৩২.৯ মাউন্ট করা — `mount` ও `umount`

mount point কী

Mount point হলো একটা সাধারণ, (সাধারণত) খালি ডিরেক্টরি, যেখানে একটা ফাইলসিস্টেমকে জুড়ে দেওয়া হয়। মাউন্টের পর ওই ডিরেক্টরিতে ঢুকলে তুমি আসলে ওই পার্টিশনের ভেতরে আছ।

```
মাউন্টের আগে:                          মাউন্টের পরে:  mount /dev/sdb1 /mnt/data
 /                                       /
 └── mnt/                                └── mnt/
     └── data/  (খালি, root                  └── data/  ← এখন sdb1-এর "গোড়া"
                ফাইলসিস্টেমের অংশ)              ├── photos/
                                                 │   ├── movies/
                                                 │   └── lost+found/
```

> 📝 **মনে রাখো:** মাউন্ট পয়েন্টে আগে থেকে কোনো ফাইল থাকলে সেগুলো **মুছে যায় না** — শুধু ঢাকা পড়ে (hidden)। আনমাউন্ট করলেই আবার দেখা যাবে। কখনো "ডিস্ক ভরা দেখাচ্ছে কিন্তু ফাইল পাচ্ছি না" সমস্যা হলে মনে রেখো, কোনো মাউন্টের নিচে লুকানো ফাইল থাকতে পারে।

প্রচলিত জায়গা:

ডিরেক্টরি	ব্যবহার
`/mnt`	অস্থায়ী, হাতে করা মাউন্ট (প্রশাসকের জন্য)
`/media/<user>/<label>`	Ubuntu/Debian-এ USB, CD-র অটো-মাউন্ট
`/run/media/<user>/<label>`	Fedora/RHEL-এ অটো-মাউন্ট
`/srv`, `/data`, `/var/lib/...`	স্থায়ী ডেটা ডিস্ক (নিজের পছন্দমতো)

ভেতরে কী হয়

`mount` কমান্ড kernel-কে `mount()` system call দিয়ে বলে: "এই block device-এ এই টাইপের ফাইলসিস্টেম আছে, একে এই ডিরেক্টরিতে বসাও।" kernel তখন ফাইলসিস্টেম ড্রাইভার দিয়ে superblock পড়ে, যাচাই করে, তারপর VFS-এর গাছে ওই ডিরেক্টরির জায়গায় নতুন গাছের গোড়াটা জুড়ে দেয়। সক্রিয় সব মাউন্টের তালিকা kernel রাখে, যা দেখা যায় `/proc/self/mounts` (= `/proc/mounts`) ফাইলে।

`mount` এর ব্যবহার

Syntax: `mount [-t type] [-o options] DEVICE DIRECTORY`

```
$ sudo mkdir -p /mnt/data
$ sudo mount /dev/sdb1 /mnt/data
$ findmnt /mnt/data
```

```
TARGET    SOURCE    FSTYPE OPTIONS
/mnt/data /dev/sdb1 ext4   rw,relatime
```

`-t` না দিলে `mount` নিজেই `blkid`-এর মতো সিগনেচার পড়ে টাইপ বুঝে নেয়। শুধু `mount` লিখলে সব মাউন্ট দেখায় (অনেক লম্বা, তাই `findmnt` ভালো)।

গুরুত্বপূর্ণ অপশন:

অপশন	কাজ
`-t ext4`	ফাইলসিস্টেম টাইপ নির্দিষ্ট করো
`-o ro`	শুধু-পড়া (read-only)
`-o rw`	পড়া-লেখা
`-o remount,rw /`	মাউন্ট না খুলেই অপশন বদলাও
`-o loop`	একটা ইমেজ ফাইল (যেমন `.iso`) মাউন্ট
`-o noexec`	এখান থেকে কোনো প্রোগ্রাম চালানো যাবে না
`-o nosuid`	setuid বিট কাজ করবে না (অধ্যায় ১৭)
`-o nodev`	device ফাইল কাজ করবে না
`-o noatime`	পড়ার সময় access time আপডেট হবে না (কম লেখা)
`-o uid=1000,gid=1000`	FAT/exFAT/NTFS-এ সব ফাইলের মালিক কে হবে
`-o umask=022`	FAT/exFAT-এ পারমিশন মাস্ক
`--bind /src /dst`	একটা ডিরেক্টরিকে আরেক জায়গায় "আয়না" হিসেবে দেখাও
`-a`	`/etc/fstab`-এর সব (যেগুলোতে `noauto` নেই) মাউন্ট করো
`-v`	বিস্তারিত বলো
`-L DATA` / `-U <uuid>`	লেবেল/UUID দিয়ে ডিভাইস বোঝাও

উদাহরণ:

```
# ISO ফাইলের ভেতর দেখা
$ sudo mkdir /mnt/iso
$ sudo mount -o loop,ro ubuntu-24.04.iso /mnt/iso
mount: /mnt/iso: WARNING: source write-protected, mounted read-only.

# সিস্টেম রিকভারির সময় root-কে লেখার উপযোগী করা
$ sudo mount -o remount,rw /

# UUID দিয়ে
$ sudo mount UUID=8d4f2b6a-3c1e-4f7a-9b2d-5e6f7a8b9c0d /mnt/data
```

> 📝 **মনে রাখো:** `relatime` (ডিফল্ট) হলো মাঝামাঝি সমাধান — access time তখনই আপডেট হয় যখন সেটা modify time-এর চেয়ে পুরোনো বা ২৪ ঘণ্টার বেশি পুরোনো। তাই বেশিরভাগ ক্ষেত্রে `noatime` না

> দিলেও চলে।

`umount` — খুলে ফেলা

বানান খেয়াল করো: `umount`, **`unmount` না**।

```
$ sudo umount /mnt/data          # mount point দিয়ে
$ sudo umount /dev/sdb1          # অথবা device দিয়ে
```

আনমাউন্ট করার সময় kernel page cache-এর সব অলেখা (dirty) ডেটা ডিস্কে লিখে দেয় — তাই এটা নিরাপদে ডিস্ক খোলার আসল ধাপ।

সবচেয়ে প্রচলিত ত্রুটি:

```
$ sudo umount /mnt/data
umount: /mnt/data: target is busy.
```

মানে কোনো প্রসেস এখনো ভেতরের কোনো ফাইল খোলা রেখেছে, বা কোনো শেলের current directory ওটার ভেতরে। খুঁজে বের করো কে:

```
$ sudo fuser -vm /mnt/data
                     USER        PID ACCESS COMMAND
/mnt/data:           root     kernel mount /mnt/data
                     rahim      4821 ..c.. bash
                     rahim      5102 F.... vlc

$ sudo lsof +f -- /mnt/data
COMMAND  PID  USER   FD   TYPE DEVICE  SIZE/OFF NODE NAME
bash    4821 rahim  cwd    DIR   8,17      4096    2 /mnt/data
vlc     5102 rahim   23r   REG   8,17 734003200   15 /mnt/data/movies/a.mkv
```

`c` = current directory, `F` = খোলা ফাইল। সমাধান: ওই টার্মিনালে `cd ~` করো, vlc বন্ধ করো, তারপর আবার `umount`। (প্রসেস বন্ধ করার বিস্তারিত অধ্যায় ২৪।)

অপশন	কাজ
`umount -l`	lazy: এখনই গাছ থেকে আলাদা করো, ব্যবহার শেষ হলে পুরোপুরি খোলো
`umount -f`	force (মূলত সাড়া না দেওয়া NFS-এর জন্য)
`umount -R /mnt/x`	ভেতরের সব নেস্টেড মাউন্টসহ (recursive)
`fuser -km /mnt/data`	ওই মাউন্ট ব্যবহারকারী সব প্রসেস **মেরে ফেলো** — সাবধান!

> ⚠️ **সতর্কতা:** `umount -l` দিয়ে খোলার পর USB টেনে খুলো না — ডেটা তখনো পুরো লেখা নাও হতে পারে। lazy umount জরুরি অবস্থার জন্য, রোজকার অভ্যাসের জন্য না।

৩২.১০ `/etc/fstab` — বুটের সময় স্বয়ংক্রিয় মাউন্ট

`mount` দিয়ে যা মাউন্ট করো, রিবুটের পর তা থাকে না। স্থায়ী করতে হলে `/etc/fstab` (filesystem table) ফাইলে লিখতে হয়। বুটের সময় systemd এই ফাইল পড়ে প্রতিটা লাইনের জন্য একটা `.mount` unit বানায় (`systemd-fstab-generator` দিয়ে) এবং মাউন্ট করে। (অধ্যায় ২৬ ও অধ্যায় ২৭ দেখো।)

```
$ cat /etc/fstab
# <file system>                             <mount point>  <type>  <options>
<dump> <pass>
UUID=3f2a9c1e-7b4d-4e8a-9d61-2c5b8e0f1a77  /              ext4    errors=remount-ro  0
1
UUID=A1B2-C3D4                             /boot/efi      vfat    umask=0077         0
1
/swap.img                                  none           swap    sw                 0
0
```

প্রতিটা লাইনে **৬টা ফিল্ড**, ফাঁকা জায়গা (space/tab) দিয়ে আলাদা:

```
UUID=8d4f...9c0d   /mnt/data   ext4   defaults,noatime   0   2
└──────┬──────┘   └────┬────┘  └─┬─┘  └──────┬──────┘  └┬┘ └┬┘
       │               │         │           │          │   └─ ৬. pass (fsck ক্রম)
       │               │         │           │          └──── ৫. dump
       │               │         │           └─────────────── ৪. options
       │               │         └─────────────────────────── ৩. type
       │               └───────────────────────────────────── ২. mount point
       └───────────────────────────────────────────────────── ১. device
```

১. Device (fs_spec): কোন ডিভাইস। লেখার উপায়:

ধরন	উদাহরণ	মন্তব্য
UUID	`UUID=8d4f2b6a-...`	**সবচেয়ে প্রচলিত ও নিরাপদ**
লেবেল	`LABEL=DATA`	পড়তে সহজ, কিন্তু দুই ডিস্কে একই লেবেল হলে সমস্যা
PARTUUID	`PARTUUID=c4d5e6f7-...`	ফরম্যাট করলেও বদলায় না
ডিভাইস পাথ	`/dev/sdb1`	**এড়িয়ে চলো** — নাম বদলাতে পারে
LVM	`/dev/mapper/vg0-data`	LVM নাম স্থায়ী, তাই চলে (অধ্যায় ৩৩)
নেটওয়ার্ক	`server:/export` (NFS), `//server/share` (CIFS)	

UUID কেন? আগেই বলেছি, `sdb` কাল `sdc` হয়ে যেতে পারে। তখন fstab-এ `/dev/sdb1` থাকলে হয় মাউন্ট ব্যর্থ, নয়তো ভুল ডিস্ক ভুল জায়গায় মাউন্ট হবে। UUID ফাইলসিস্টেমের ভেতরেই লেখা থাকে, তাই ডিস্ক যেকোনো পোর্টে লাগাও, একই থাকবে। উপমা: `sdb` হলো ক্লাসের রোল নম্বর (প্রতি বছর বদলায়), UUID হলো জন্ম নিবন্ধন নম্বর।

২. Mount point (fs_file): কোথায় মাউন্ট হবে। ডিরেক্টরিটা আগে থেকে থাকতে হবে। swap-এর জন্য `none`।

৩. Type (fs_vfstype): `ext4`, `xfs`, `vfat`, `exfat`, `btrfs`, `ntfs3`, `swap`, `nfs`, `tmpfs` বা `auto`।

৪. Options (fs_mntops): কমা দিয়ে আলাদা, ফাঁকা জায়গা ছাড়া। `mount -o`-এর সব অপশন চলে, সাথে এগুলো:

অপশন	মানে
`defaults`	`rw,suid,dev,exec,auto,nouser,async` এর সমষ্টি
`auto` / `noauto`	`mount -a` ও বুটে মাউন্ট হবে / হবে না
`user`	সাধারণ ইউজারও মাউন্ট করতে পারবে (সাথে নিজে `noexec,nosuid,nodev` যোগ করে)
`nofail`	ডিভাইস না পেলে বুট আটকাবে না — **বাইরের/ডেটা ডিস্কে অবশ্যই দাও**
`x-systemd.automount`	প্রথমবার কেউ ডিরেক্টরিতে ঢুকলে তখন মাউন্ট (বুট দ্রুত হয়)
`x-systemd.device-timeout=10s`	ডিভাইসের জন্য কতক্ষণ অপেক্ষা (ডিফল্ট 90 সেকেন্ড)
`_netdev`	নেটওয়ার্ক চালু হওয়ার পর মাউন্ট (iSCSI ইত্যাদিতে)
`errors=remount-ro`	(ext4) ত্রুটি পেলে read-only করে দাও
`sw`	swap লাইনে প্রচলিত (কার্যত `defaults`-এর মতো)

৫. Dump (fs_freq): পুরোনো `dump` ব্যাকআপ টুলের জন্য। আজকাল প্রায় সবসময় `0`।

৬. Pass (fs_passno): বুটের সময় `fsck` কোন ক্রমে চলবে।

মান	মানে
`0`	fsck চালাবে না (swap, XFS, Btrfs, নেটওয়ার্ক)
`1`	প্রথমে — শুধু root (`/`)
`2`	root-এর পরে — বাকি ext4 ও vfat পার্টিশন

> 💡 **টিপস:** XFS ও Btrfs-এ pass `0` দাও। এদের মেরামতের নিজস্ব পদ্ধতি আছে, বুট-টাইম fsck এদের জন্য কিছু করে না (`fsck.xfs` আসলে কিছুই করে না, শুধু `xfs_repair` ব্যবহারের পরামর্শ দেয়)।

ধাপে ধাপে: নতুন ডিস্ক fstab-এ যোগ করা

```
# ১. UUID বের করো
$ sudo blkid -s UUID -o value /dev/sdb1
8d4f2b6a-3c1e-4f7a-9b2d-5e6f7a8b9c0d
```

```
# ২. mount point বানাও
$ sudo mkdir -p /mnt/data

# ৩. fstab-এর ব্যাকআপ রাখো
$ sudo cp /etc/fstab /etc/fstab.bak

# ৪. লাইন যোগ করো (অথবা nano/vim দিয়ে)
$ echo 'UUID=8d4f2b6a-3c1e-4f7a-9b2d-5e6f7a8b9c0d /mnt/data ext4 defaults,nofail 0 2' |
sudo tee -a /etc/fstab

# ৫. যাচাই করো — রিবুটের আগে এটা বাধ্যতামূলক!
$ sudo findmnt --verify
Success, no errors or warnings detected

# ৬. systemd-কে নতুন fstab জানাও
$ sudo systemctl daemon-reload

# ৭. fstab-এর সব মাউন্ট করো
$ sudo mount -a
$ df -h /mnt/data
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb1        15G   24K   14G   1% /mnt/data
```

`mount -a` কোনো আউটপুট না দিলে মানে সফল। ভুল থাকলে এখানেই ত্রুটি দেখাবে — রিবুটের পরে না। (`tee` কেন লাগল, `sudo echo >>` কেন কাজ করে না, তা অধ্যায় ১৮ ও অধ্যায় ১৯-এ আছে।)

> ⚠️ **সতর্কতা:** fstab-এ ভুল (যেমন ভুল UUID) রেখে `nofail` ছাড়া রিবুট করলে সিস্টেম **emergency mode**-এ আটকে যেতে পারে: *"You are in emergency mode…"*। তখন root পাসওয়ার্ড দিয়ে ঢুকে `mount -o remount,rw /` করো, `/etc/fstab` ঠিক করো (বা `/etc/fstab.bak` ফেরত আনো), তারপর `systemctl daemon-reload` ও `reboot`। Ubuntu-তে root লক থাকলে GRUB মেনু থেকে recovery mode বা লাইভ USB লাগবে। তাই সবসময় `findmnt --verify` আর `mount -a` দিয়ে আগে পরীক্ষা করো।

৩২.১১ জায়গা মাপা — `df`, `du`, `ncdu`

`df` — ফাইলসিস্টেম কতটা ভরা (disk free)

`df` প্রতিটা মাউন্ট করা **ফাইলসিস্টেমের** superblock থেকে মোট/ব্যবহৃত/খালি হিসাব পড়ে — খুব দ্রুত।

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G  2.1M  1.6G   1% /run
/dev/nvme0n1p3  466G   63G  380G  15% /
tmpfs           7.8G     0  7.8G   0% /dev/shm
/dev/nvme0n1p2  2.0G  204M  1.6G  12% /boot
```

```
/dev/nvme0n1p1  1.1G  6.2M  1.1G   1% /boot/efi
/dev/sdb1        15G   24K   14G   1% /mnt/data
```

অপশন	কাজ
`-h`	মানুষের পড়ার মতো (K, M, G — 1024 ভিত্তিক)
`-H`	1000 ভিত্তিক (ডিস্ক বিক্রেতার হিসাব)
`-T`	ফাইলসিস্টেম টাইপ কলাম
`-i`	inode ব্যবহার (byte না)
`-x tmpfs -x devtmpfs`	এই টাইপগুলো বাদ দাও
`-t ext4`	শুধু এই টাইপ
`--total`	শেষে মোট যোগফল
`df -h /var`	`/var` যে ফাইলসিস্টেমে, শুধু সেটা

খেয়াল করো: `/mnt/data`-তে Size 15G, Used 24K, Avail 14G — যোগ মিলছে না! কারণ ওই 5% reserved জায়গা (root-এর জন্য) Avail-এ ধরা হয় না।

inode ফুরিয়ে যাওয়া: কখনো `df -h` বলে 40% খালি, কিন্তু ফাইল বানাতে গেলে *"No space left on device"*। তখন `df -i` দেখো — লাখ লাখ ছোট ফাইলে (যেমন session/cache ফাইল) inode শেষ হয়ে গেছে:

```
$ df -hi /
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/nvme0n1p3    30M   30M     0  100% /
```

`du` — কোন ফোল্ডার কতটা জায়গা নিচ্ছে (disk usage)

`du` ডিরেক্টরির ভেতরে ঢুকে প্রতিটা ফাইলের আকার যোগ করে — বড় ডিরেক্টরিতে সময় লাগে।

```
$ du -sh ~/Downloads
4.7G    /home/rahim/Downloads

$ sudo du -h -d 1 /var | sort -h
16K     /var/mail
4.0K    /var/opt
...
1.2G    /var/cache
3.8G    /var/log
5.9G    /var/lib
11G     /var
```

অপশন	কাজ
`-s`	শুধু মোট (summary)
`-h`	মানুষের পড়ার মতো
`-d 1` / `--max-depth=1`	এক স্তর গভীর পর্যন্ত

অপশন	কাজ
`-a`	ডিরেক্টরির সাথে ফাইলগুলোও দেখাও
`-x`	অন্য ফাইলসিস্টেমে ঢুকবে না (যেমন `/`-এ চালালে `/proc`, `/mnt` বাদ)
`-c`	শেষে মোট যোগফল
`--apparent-size`	ফাইলের "দেখানো" আকার, ডিস্কে আসলে কত নিয়েছে তা না
`--exclude='*.iso'`	প্যাটার্ন বাদ

সবচেয়ে কাজের কম্বো (sort-এর বিস্তারিত অধ্যায় ২৩):

```
$ du -sh * | sort -h | tail -5        # বর্তমান ফোল্ডারের সবচেয়ে বড় ৫টা
$ sudo du -xh / 2>/dev/null | sort -h | tail -20
```

`df` আর `du` আলাদা সংখ্যা দেখায় কেন?

এটা ইন্টারভিউ আর বাস্তবে খুব প্রচলিত প্রশ্ন।

```
  df: "ফাইলসিস্টেমের হিসাবখাতা (superblock) কী বলছে?"
  du: "আমি গাছে হেঁটে যতগুলো ফাইল দেখতে পাচ্ছি, তাদের যোগফল কত?"
```

কারণগুলো:

1. **মুছে ফেলা কিন্তু খোলা ফাইল:** কোনো প্রসেস (যেমন nginx) একটা 10 GB লগ খোলা রেখেছে, আর তুমি `rm` দিয়ে মুছলে। নাম চলে গেল (তাই `du` দেখে না), কিন্তু inode আর ডেটা থাকে যতক্ষণ প্রসেসটা ফাইল খোলা রাখে (তাই `df` দেখায় ভরা)। খুঁজতে: `sudo lsof +L1`। সমাধান: সার্ভিস রিস্টার্ট (অধ্যায় ২৬)।
2. **মাউন্টের নিচে লুকানো ফাইল** (৩২.৯-এ বলেছি)।
3. **Reserved block** আর ফাইলসিস্টেমের নিজের মেটাডেটা।
4. **Sparse file:** যে ফাইলের ভেতরে "ফাঁকা" অংশ ডিস্কে লেখা হয় না (VM ডিস্ক ইমেজ)। `ls -l` বড় দেখায়, `du` ছোট।

> 💡 **টিপস:** লগ ফাইল খালি করতে `rm` না করে `sudo truncate -s 0 /var/log/big.log` করো — তাহলে প্রসেস খোলা রাখলেও জায়গা সাথে সাথে খালি হবে।

`ncdu` — ইন্টারঅ্যাকটিভ du

`du`-এর রঙিন, মেনু-ভিত্তিক রূপ। কোথায় জায়গা গেল খুঁজতে সবচেয়ে আরামদায়ক।

```
$ sudo apt install ncdu        # Debian/Ubuntu
$ sudo dnf install ncdu        # Fedora (RHEL-এ EPEL থেকে)
$ ncdu ~
$ sudo ncdu -x /               # শুধু root ফাইলসিস্টেম
```

```
ncdu 1.19 ~ Use the arrow keys to navigate, press ? for help
--- /home/rahim ------------------------------------------------------------
    4.7 GiB [##########] /Downloads
    2.1 GiB [####      ] /.cache
  812.4 MiB [#         ] /Videos
  120.0 MiB [          ] /.local
 Total disk usage:   7.8 GiB  Apparent size:   7.6 GiB  Items: 48213
```

কী (`ncdu` -এর ভেতরে)	কাজ
`↑` `↓` বা `k` `j`	উপরে-নিচে
`Enter` বা `→` / `l`	ফোল্ডারে ঢোকো
`←` বা `h`	এক স্তর উপরে
`d`	নির্বাচিতটা মুছে ফেলো (নিশ্চিতকরণ চাইবে)
`s`	আকার অনুযায়ী সাজাও
`n`	নাম অনুযায়ী সাজাও
`C`	আইটেম সংখ্যা অনুযায়ী সাজাও
`a`	ডিস্কে আকার ↔ apparent size টগল
`g`	শতাংশ/গ্রাফ দেখানোর ধরন বদলাও
`e`	লুকানো/বাদ দেওয়া ফাইল দেখাও/লুকাও
`i`	নির্বাচিত আইটেমের তথ্য
`r`	আবার স্ক্যান করো
`?`	সাহায্য
`q`	বের হও

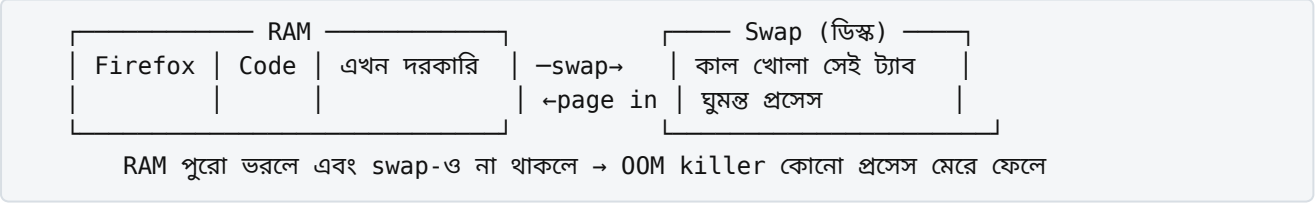
অপশন (কমান্ড লাইনে)	কাজ
`-x`	অন্য ফাইলসিস্টেমে যাবে না
`--exclude PATTERN`	বাদ দাও
`-o file` / `-f file`	স্ক্যান ফলাফল ফাইলে রাখো / পরে খোলো
`-r`	read-only, মোছা বন্ধ (নিরাপদ)

৩২.১২ Swap — RAM-এর "বাড়তি তাক"

Swap কী ও কেন

RAM ভরে গেলে kernel কম ব্যবহৃত মেমোরি পেজ (page, সাধারণত 4 KiB টুকরো) ডিস্কের একটা বিশেষ জায়গায় সরিয়ে রাখে — এটাই **swap**। পরে দরকার হলে আবার RAM-এ আনে।

উপমা: তোমার পড়ার টেবিল (RAM) ছোট। যে বইগুলো এখন পড়ছ না, সেগুলো পাশের তাকে (swap) রাখলে। দরকার হলে আবার আনতে সময় লাগে (ডিস্ক RAM-এর চেয়ে অনেক ধীর), কিন্তু অন্তত টেবিলে জায়গা ফুরিয়ে কাজ থেমে যায় না।



Swap কেন রাখবে:

- RAM ফুরিয়ে গেলে হঠাৎ প্রোগ্রাম মারা যাওয়া (OOM killer) ঠেকায়।
- অনেকদিন অব্যবহৃত মেমোরি সরিয়ে RAM-কে file cache-এর জন্য খালি করে।
- **Hibernate** (suspend to disk)-এর জন্য RAM-এর সমান বা বড় swap partition/file লাগে।

কত বড়? কঠিন নিয়ম নেই। সাধারণ পরামর্শ: RAM ≤ 2 GB হলে RAM-এর দ্বিগুণ; 2–8 GB হলে RAM-এর সমান; 8 GB-এর বেশি হলে 4–8 GB যথেষ্ট (hibernate চাইলে RAM-এর সমান বা বেশি)।

Distro-ভেদে ডিফল্ট:

Distro	ডিফল্ট swap
Ubuntu 24.04	`/swap.img` নামে একটা swapfile
Debian 12	ইনস্টলারে আলাদা swap partition
Fedora (33+)	**zram** — RAM-এর ভেতরেই কম্প্রেসড swap (`zram-generator`)
RHEL 9	LVM-এর ভেতরে swap logical volume

বর্তমান swap দেখা

```
$ swapon --show
NAME      TYPE  SIZE USED PRIO
/swap.img file    4G 512M   -2

$ free -h
               total        used        free      shared  buff/cache   available
Mem:            15Gi       6.2Gi       2.1Gi       410Mi       7.4Gi       8.9Gi
Swap:          4.0Gi       512Mi       3.5Gi
```

(`free` আর মেমোরি বিশ্লেষণের বিস্তারিত অধ্যায় ৪৬-এ।)

পদ্ধতি ১: Swap partition

ধরো ৩২.৭-এ বানানো `/dev/sdb2` (টাইপ Linux swap):

```
$ sudo mkswap -L myswap /dev/sdb2
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
LABEL=myswap, UUID=1e2d3c4b-5a69-4788-97a6-b5c4d3e2f100

$ sudo swapon /dev/sdb2
$ swapon --show
NAME      TYPE      SIZE USED PRIO
/swap.img file        4G 512M   -2
/dev/sdb2 partition   2G   0B   -3
```

স্থায়ী করতে `/etc/fstab`-এ:

```
UUID=1e2d3c4b-5a69-4788-97a6-b5c4d3e2f100  none  swap  defaults  0  0
```

পদ্ধতি ২: Swapfile (নতুন পার্টিশন ছাড়াই)

```
# ১. 2 GiB-এর ফাইল বানাও (দ্রুত পদ্ধতি)
$ sudo fallocate -l 2G /swapfile

#    fallocate সমর্থন না করলে বা swapon ত্রুটি দিলে, ধীর কিন্তু নিশ্চিত পদ্ধতি:
$ sudo dd if=/dev/zero of=/swapfile bs=1M count=2048 status=progress

# ২. শুধু root পড়তে-লিখতে পারবে — নিরাপত্তার জন্য জরুরি
$ sudo chmod 600 /swapfile

# ৩. swap হিসেবে প্রস্তুত করো
$ sudo mkswap /swapfile
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=7a6b5c4d-3e2f-4a1b-9c8d-7e6f5a4b3c2d

# ৪. চালু করো
$ sudo swapon /swapfile

# ৫. স্থায়ী করো
$ echo '/swapfile none swap defaults 0 0' | sudo tee -a /etc/fstab
```

> ⚠️ **সতর্কতা:** swapfile-এ RAM-এর যেকোনো কিছু (পাসওয়ার্ড, কী) লেখা হতে পারে। তাই `chmod 600` বাধ্যতামূলক — না দিলে `swapon` সতর্কবার্তা দেয়: *"insecure permissions 0644, fix with: chmod 0600 /swapfile"*।

> ⚠️ **সতর্কতা (Btrfs):** Btrfs-এ সাধারণ `fallocate` দিয়ে swapfile চলে না (copy-on-write ও compression বন্ধ করতে হয়)। btrfs-progs 6.1+ থাকলে লেখো: `sudo btrfs filesystem mkswapfile --size 2G /swap/swapfile`।

swap বন্ধ ও মুছে ফেলা:

```
$ sudo swapoff /swapfile        # swap-এ থাকা পেজগুলো RAM-এ ফেরত আনে, তারপর বন্ধ
$ sudo rm /swapfile             # তারপর fstab থেকেও লাইনটা মুছে দাও
$ sudo swapoff -a               # সব swap বন্ধ
$ sudo swapon -a                # fstab-এর সব swap চালু
```

কমান্ড/অপশন	কাজ
`mkswap DEV`	swap সিগনেচার লেখো
`mkswap -L LABEL`	লেবেলসহ
`swapon DEV`	চালু
`swapon -p 10 DEV`	priority (বেশি সংখ্যা আগে ব্যবহার হবে)
`swapon --show`	সক্রিয় swap
`swapon -a` / `swapoff -a`	fstab-এর সব চালু / সব বন্ধ
`cat /proc/swaps`	kernel-এর নিজের তালিকা

Swappiness — kernel কতটা "swap-প্রিয়"

`vm.swappiness` (0–200, ডিফল্ট **60**) বলে দেয় RAM-এর চাপ এলে kernel কতটা আগ্রহে অ্যাপের মেমোরি (anonymous page) swap-এ পাঠাবে, ফাইল cache ফেলে দেওয়ার তুলনায়।

- **কম মান (যেমন 10):** kernel যতটা পারে অ্যাপকে RAM-এ রাখে, বরং ফাইল cache ছাড়ে। ডেস্কটপ ও ডাটাবেস সার্ভারে প্রচলিত।
- **বেশি মান (100+):** swap নির্দ্বিধায় ব্যবহার। zram-এর মতো দ্রুত swap-এ যুক্তিসংগত (Fedora-র zram সেটআপ নিজেই এটা বাড়িয়ে দিতে পারে)।
- **0:** swap প্রায় এড়ায়, কিন্তু একেবারে বন্ধ করে না।

```
$ cat /proc/sys/vm/swappiness
60
$ sysctl vm.swappiness
vm.swappiness = 60

# এখনই বদলাও (রিবুটে হারাবে)
$ sudo sysctl vm.swappiness=10
vm.swappiness = 10

# স্থায়ী করো
$ echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swappiness.conf
$ sudo sysctl --system          # সব sysctl কনফিগ আবার লোড করো
```

> 📝 **মনে রাখো:** swappiness "RAM কত % ভরলে swap শুরু হবে" — এই প্রচলিত ধারণাটা **ভুল**। এটা একটা তুলনামূলক ওজন (anonymous memory বনাম file cache কোনটা আগে ছাড়া হবে)। `/proc/sys` ও sysctl নিয়ে আরও আছে অধ্যায় ৪৭-এ।

৩২.১৩ ফাইলসিস্টেম পরীক্ষা ও মেরামত — `fsck`, `e2fsck`, `xfs_repair`

হঠাৎ বিদ্যুৎ চলে যাওয়া, USB টেনে খোলা বা ডিস্কের সমস্যায় ফাইলসিস্টেমের মেটাডেটা (bitmap, inode, directory) অসঙ্গত হয়ে যেতে পারে। `fsck` (file system check) এগুলো খুঁজে ঠিক করে। উপমা: লাইব্রেরির ক্যাটালগ মিলিয়ে দেখা — যে বই তাকে আছে কিন্তু ক্যাটালগে নেই, বা ক্যাটালগে আছে কিন্তু তাকে নেই।

> ⚠️ **সবচেয়ে বড় সতর্কতা: মাউন্ট করা ফাইলসিস্টেমে কখনো মেরামতকারী fsck চালাবে না** — বাস্তবে ফাইলসিস্টেম আরও নষ্ট হয়ে যেতে পারে। আগে `umount`। root (`/`) পরীক্ষা করতে হলে বুটের সময়, রিকভারি মোডে বা লাইভ USB থেকে করো।

`fsck` — সামনের দরজা

`mkfs`-এর মতোই, `fsck` টাইপ দেখে ভেতরে `fsck.ext4`, `fsck.vfat` ইত্যাদি চালায়।

```
$ sudo umount /dev/sdb1
$ sudo fsck /dev/sdb1
fsck from util-linux 2.39.3
e2fsck 1.47.0 (5-Feb-2023)
DATA: clean, 11/983040 files, 87309/3932160 blocks
```

"clean" মানে শেষবার ঠিকঠাক আনমাউন্ট হয়েছিল, তাই পুরো পরীক্ষা হয়নি। জোর করে পুরো পরীক্ষা:

```
$ sudo fsck -f /dev/sdb1
fsck from util-linux 2.39.3
e2fsck 1.47.0 (5-Feb-2023)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
DATA: 11/983040 files (0.0% non-contiguous), 87309/3932160 blocks
```

ext4-এর এই ৫টা ধাপ: (১) প্রতিটা inode ও তার block, (২) ডিরেক্টরির গঠন, (৩) সব ডিরেক্টরি root থেকে পৌঁছানো যায় কিনা, (৪) প্রতিটা inode-এর link count ঠিক আছে কিনা, (৫) bitmap ও summary হিসাব মেলে কিনা। কোনো মালিকহীন ফাইল পেলে সেগুলো `lost+found` ডিরেক্টরিতে রেখে দেয় (inode নম্বর দিয়ে নাম)।

অপশন	কাজ
`-f`	"clean" হলেও জোর করে পরীক্ষা (e2fsck-এর অপশন, fsck পাস করে দেয়)
`-n`	কিছু বদলাবে না, শুধু রিপোর্ট — **সবচেয়ে নিরাপদ প্রথম পদক্ষেপ**
`-y`	সব প্রশ্নে স্বয়ংক্রিয়ভাবে "yes"
`-p` / `-a`	নিরাপদ সমস্যাগুলো প্রশ্ন ছাড়াই ঠিক করো (preen)

অপশন	কাজ
`-C 0`	progress bar (fsck-এর অপশন)
`-A`	fstab-এর সব (pass ক্রমে)
`-t ext4`	টাইপ নির্দিষ্ট
`-N`	কী চালাত তা দেখাও, চালাবে না

Exit code (স্ক্রিপ্টে কাজে লাগে; একাধিক হলে যোগ হয়):

কোড	মানে
`0`	কোনো ত্রুটি নেই
`1`	ত্রুটি পাওয়া গেছে ও ঠিক করা হয়েছে
`2`	ঠিক করা হয়েছে, রিবুট দরকার
`4`	ত্রুটি রয়ে গেছে, ঠিক হয়নি
`8`	অপারেশনাল ত্রুটি
`16`	ব্যবহারে ভুল (syntax)
`32`	ইউজার বাতিল করেছে
`128`	shared library ত্রুটি

`e2fsck` — ext2/3/4-এর আসল মেরামতকারী

```
$ sudo e2fsck -f -y /dev/sdb1      # জোর করে পরীক্ষা, সব ঠিক করো
$ sudo e2fsck -n /dev/sdb1         # শুধু দেখো
$ sudo e2fsck -c /dev/sdb1         # badblocks দিয়ে খারাপ sector খুঁজে চিহ্নিত করো (ধীর)
```

মূল superblock নষ্ট হলে (*"Bad magic number in super-block"*) — ব্যাকআপ superblock দিয়ে চেষ্টা:

```
$ sudo mke2fs -n /dev/sdb1          # -n: শুধু দেখায় ব্যাকআপ কোথায়, কিছুই বানায় না!
...
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, ...

$ sudo e2fsck -b 32768 /dev/sdb1
```

> ⚠️ **সতর্কতা:** `mke2fs -n`-এর `-n` ভুলে বাদ দিলে নতুন ফাইলসিস্টেম তৈরি হয়ে যাবে, সব ডেটা যাবে! আর আসল mkfs-এর সময় যে block size ব্যবহার হয়েছিল, সেটা না মিললে ব্যাকআপের অবস্থান ভুল দেখাবে। সন্দেহ থাকলে `sudo dumpe2fs /dev/sdb1 | grep -i superblock` চেষ্টা করো (যদি পড়তে পারে)।

`xfs_repair` — XFS-এর জন্য

```
$ sudo umount /dev/sdc1
$ sudo xfs_repair -n /dev/sdc1      # শুধু পরীক্ষা
$ sudo xfs_repair /dev/sdc1         # মেরামত
```

যদি বলে log-এ অসমাপ্ত পরিবর্তন আছে, প্রথমে একবার মাউন্ট-আনমাউন্ট করে log replay করাও। সেটাও না পারলে শেষ উপায় `xfs_repair -L` (log মুছে ফেলে — কিছু সাম্প্রতিক পরিবর্তন হারাতে পারে)।

বুটের সময় fsck

systemd-চালিত সিস্টেমে বুটে fsck চালায় `systemd-fsck@.service`, fstab-এর pass ফিল্ড মেনে। জোর করে পরের বুটে সব পরীক্ষা করাতে GRUB-এ kernel প্যারামিটার দাও `fsck.mode=force` (আর প্রশ্ন ছাড়া ঠিক করাতে `fsck.repair=yes`)। (অধ্যায় ২৭-এ kernel parameter দেওয়ার পদ্ধতি দেখো।)

```
$ journalctl -u 'systemd-fsck*' -b      # এই বুটের fsck লগ
```

৩২.১৪ `tune2fs` — ext4-এর সেটিং বদলানো (ফরম্যাট ছাড়াই)

`tune2fs` দিয়ে একটা বিদ্যমান ext2/3/4 ফাইলসিস্টেমের প্যারামিটার দেখা ও বদলানো যায়। বেশিরভাগ পরিবর্তন মাউন্ট অবস্থাতেই চলে।

```
$ sudo tune2fs -l /dev/sdb1 | head -20
tune2fs 1.47.0 (5-Feb-2023)
Filesystem volume name:   DATA
Last mounted on:          /mnt/data
Filesystem UUID:          8d4f2b6a-3c1e-4f7a-9b2d-5e6f7a8b9c0d
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype extent
64bit flex_bg sparse_super large_file huge_file dir_nlink extra_isize metadata_csum
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Inode count:              983040
Block count:              3932160
Reserved block count:     196608
...
Mount count:              3
Maximum mount count:      -1
```

`Maximum mount count: -1` মানে নির্দিষ্ট সংখ্যক মাউন্টের পর স্বয়ংক্রিয় fsck বন্ধ — আধুনিক e2fsprogs-এ এটাই ডিফল্ট (জার্নাল থাকায় দরকার কম)। `0xEF53` হলো ext ফাইলসিস্টেমের "magic number", যা দেখে `blkid` চিনতে পারে।

অপশন	কাজ
`-l`	সব তথ্য দেখাও (superblock)
`-L LABEL`	লেবেল বদলাও (`e2label /dev/sdb1 NEWLABEL` একই কাজ করে)
`-m 1`	reserved block % বদলাও
`-r COUNT`	reserved block সংখ্যা সরাসরি
`-c 30`	প্রতি ৩০ বার মাউন্টের পর fsck জোর করো
`-i 1m`	প্রতি ১ মাসে fsck (`d`, `w`, `m` একক)
`-C 0`	বর্তমান mount count শূন্য করো
`-e remount-ro`	ত্রুটিতে আচরণ (`continue`, `remount-ro`, `panic`)
`-U random`	নতুন random UUID (আনমাউন্ট অবস্থায়; ক্লোন করা ডিস্কে দুটো একই UUID হলে কাজে লাগে)
`-o acl`	ডিফল্ট মাউন্ট অপশন বসাও
`-O ^has_journal`	feature বন্ধ (`^` = বন্ধ) — আনমাউন্ট অবস্থায়
`-j`	ext2-তে জার্নাল যোগ করো (ext3 বানাও)

```
$ sudo tune2fs -m 1 /dev/sdb1
tune2fs 1.47.0 (5-Feb-2023)
Setting reserved blocks percentage to 1% (39321 blocks)

$ sudo tune2fs -L MYDATA /dev/sdb1
```

অন্য ফাইলসিস্টেমে লেবেল বদলানো:

ফাইলসিস্টেম	কমান্ড
ext4	`tune2fs -L NAME DEV` বা `e2label DEV NAME`
XFS	`xfs_admin -L NAME DEV` (আনমাউন্ট অবস্থায়)
vfat	`fatlabel DEV NAME`
exFAT	`exfatlabel DEV NAME`
Btrfs	`btrfs filesystem label MOUNTPOINT NAME`
swap	`swaplabel -L NAME DEV`

ফাইলসিস্টেম বড় করা (এক নজরে): পার্টিশন বড় করার পর (`parted resizepart` বা `growpart`), ext4-এ `sudo resize2fs /dev/sdb1` আর XFS-এ `sudo xfs_growfs /mnt/data`। LVM-এ এটা অনেক সহজ — অধ্যায় ৩৩ দেখো।

৩২.১৫ USB পেনড্রাইভ মাউন্ট করা

ডেস্কটপে (স্বয়ংক্রিয়)

GNOME/KDE-তে USB লাগালেই **udisks2** সার্ভিস সেটা মাউন্ট করে দেয় — Ubuntu/Debian-এ `/media/<user>/<LABEL>`, Fedora/RHEL-এ `/run/media/<user>/<LABEL>`। ফাইল ম্যানেজারে ⏏ (eject) চাপলে নিরাপদে খোলে। (অধ্যায় ৪)

টার্মিনালে, root ছাড়াই — `udisksctl`

```
$ udisksctl mount -b /dev/sdc1
Mounted /dev/sdc1 at /media/rahim/PENDRIVE

$ udisksctl unmount -b /dev/sdc1
Unmounted /dev/sdc1.

$ udisksctl power-off -b /dev/sdc       # পুরো ডিস্কের পাওয়ার বন্ধ — এখন টেনে খোলো
```

টার্মিনালে, হাতে-কলমে (সার্ভারে, GUI ছাড়া)

```
# ১. লাগানোর আগে ও পরে তুলনা করো
$ lsblk -o NAME,SIZE,RM,FSTYPE,LABEL,MOUNTPOINTS
NAME   SIZE RM FSTYPE LABEL    MOUNTPOINTS
sdc   14.9G  1
└─sdc1 14.9G  1 vfat   PENDRIVE

# ২. kernel কী দেখল (কোনো সমস্যা হলে এখানে ধরা পড়ে)
$ sudo dmesg | tail -5
[ 1834.210] usb 2-1: new SuperSpeed USB device number 3 using xhci_hcd
[ 1834.512] sd 6:0:0:0: [sdc] 31277232 512-byte logical blocks: (16.0 GB/14.9 GiB)
[ 1834.530]  sdc: sdc1
[ 1834.531] sd 6:0:0:0: [sdc] Attached SCSI removable disk

# ৩. মাউন্ট করো — FAT/exFAT-এ নিজের uid দাও, নাহলে সব ফাইল root-এর হবে
$ sudo mkdir -p /mnt/usb
$ sudo mount -o uid=$(id -u),gid=$(id -g),umask=022 /dev/sdc1 /mnt/usb

# ৪. কাজ শেষে
$ cd ~
$ sync                                  # page cache-এর সব ডেটা ডিস্কে লেখো
$ sudo umount /mnt/usb
$ sudo eject /dev/sdc                   # (ঐচ্ছিক)
```

> 📝 **মনে রাখো:** FAT32/exFAT/NTFS-এ লিনাক্সের মালিক-পারমিশন সংরক্ষণ হয় না, তাই মাউন্টের সময় `uid`, `gid`, `umask` (বা `fmask` / `dmask`) দিয়ে ঠিক করে দিতে হয়। ext4 USB-তে এগুলো লাগে না — সেখানে স্বাভাবিক পারমিশন চলে (অধ্যায় ১৬)।

NTFS (উইন্ডোজের ডিস্ক): kernel 5.15+-এ নিজস্ব `ntfs3` ড্রাইভার আছে: `sudo mount -t ntfs3 /dev/sdc1 /mnt/usb`। পুরোনো ও বহুল ব্যবহৃত বিকল্প `ntfs-3g` (FUSE-ভিত্তিক): `sudo mount -t ntfs-3g /dev/sdc1 /mnt/usb`। উইন্ডোজ "Fast Startup"/hibernate অবস্থায় বন্ধ হলে ডিস্ক dirty থাকে, শুধু read-only মাউন্ট হতে পারে — উইন্ডোজে পুরো Shut down করে আসো।

পেনড্রাইভ নতুন করে ফরম্যাট (সব মুছে):

```
$ sudo umount /dev/sdc1
$ sudo wipefs -a /dev/sdc
$ sudo parted -s /dev/sdc mklabel msdos mkpart primary 1MiB 100%
$ sudo mkfs.exfat -n MYUSB /dev/sdc1
```

(পুরোনো টিভি/গাড়ির স্টেরিওর জন্য MBR + FAT32 সবচেয়ে নিরাপদ। ISO থেকে বুটেবল USB বানানো `dd`-এর বিস্তারিত অধ্যায় ৩৫-এ।)

> ⚠️ **সতর্কতা:** "safely remove" ছাড়া USB টেনে খুললে শেষের কিছু লেখা হারাতে পারে, এমনকি ফাইলসিস্টেম নষ্ট হতে পারে — কারণ লেখা ডেটা তখনো RAM-এর page cache-এ বসে থাকতে পারে। সবসময় `umount` বা eject করো।

হাতে-কলমে অনুশীলন

এই ল্যাবে আমরা কোনো আসল ডিস্ক ছোঁব না। একটা সাধারণ ফাইলকে **loop device** বানিয়ে তার ওপর সব চর্চা করব — ভুল হলেও কোনো ক্ষতি নেই।

ল্যাব ১: একটা ভার্চুয়াল ডিস্ক বানাও

```
$ mkdir -p ~/disklab && cd ~/disklab
$ truncate -s 1G disk.img                # 1 GiB-এর sparse ফাইল
$ ls -lh disk.img; du -h disk.img
-rw-rw-r-- 1 rahim rahim 1.0G Nov 12 10:00 disk.img
0       disk.img                                  ← sparse: ডিস্কে এখনো কিছুই নেয়নি!

$ sudo losetup -fP --show disk.img       # -f: খালি loop খোঁজো, -P: পার্টিশন স্ক্যান
/dev/loop20
```

তোমার মেশিনে নম্বর আলাদা হতে পারে (`/dev/loop0`, `/dev/loop20`...) — নিচের কমান্ডে সেটাই বসাবে। এখানে আমরা ধরে নিচ্ছি `/dev/loop20`।

ল্যাব ২: GPT টেবিল ও দুটো পার্টিশন বানাও

```
$ sudo fdisk /dev/loop20
```

ভেতরে ক্রমে চাপো: `g` → `n` → Enter → Enter → `+600M` → `n` → Enter → Enter → Enter (বাকি সব) → `t` → `2` → `swap` → `p` → `w`।

```
$ lsblk /dev/loop20
NAME       MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop20       7:20   0    1G  0 loop
├─loop20p1 259:4    0  600M  0 part
└─loop20p2 259:5    0  423M  0 part
```

(`loop20p2` -এর আকার মোটামুটি 423M দেখাবে; GPT-র ব্যাকআপ টেবিলের জন্য শেষে কিছু জায়গা যায়।)

ল্যাব ৩: ফাইলসিস্টেম ও swap

```
$ sudo mkfs.ext4 -L LAB /dev/loop20p1
$ sudo mkswap -L LABSWAP /dev/loop20p2
$ lsblk -f /dev/loop20
```

FSTYPE কলামে `ext4` আর `swap` দেখা যাওয়া উচিত।

ল্যাব ৪: মাউন্ট, লেখা, লুকানো ফাইলের রহস্য

```
$ sudo mkdir -p /mnt/lab
$ echo "আমি নিচে লুকিয়ে আছি" | sudo tee /mnt/lab/hidden.txt
$ sudo mount /dev/loop20p1 /mnt/lab
$ ls /mnt/lab
lost+found                                  ← hidden.txt কোথায় গেল?
$ sudo cp /etc/hostname /mnt/lab/
$ df -h /mnt/lab
$ sudo umount /mnt/lab
$ ls /mnt/lab
hidden.txt                                  ← ফিরে এসেছে!
$ sudo rm /mnt/lab/hidden.txt
```

ল্যাব ৫: "target is busy" তৈরি করো ও সমাধান করো

```
$ sudo mount /dev/loop20p1 /mnt/lab
$ cd /mnt/lab
$ sudo umount /mnt/lab
umount: /mnt/lab: target is busy.
$ sudo fuser -vm /mnt/lab                   # দেখো তোমার bash-ই দোষী
$ cd ~
$ sudo umount /mnt/lab                      # এবার সফল
```

ল্যাব ৬: fstab দিয়ে স্থায়ী মাউন্ট (নিরাপদভাবে)

```
$ UUID=$(sudo blkid -s UUID -o value /dev/loop20p1); echo $UUID
$ sudo cp /etc/fstab /etc/fstab.bak
$ echo "UUID=$UUID /mnt/lab ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab
```

```
$ sudo findmnt --verify
$ sudo systemctl daemon-reload
$ sudo mount -a
$ findmnt /mnt/lab
```

> ⚠️ **গুরুত্বপূর্ণ:** loop device রিবুটের পর নিজে থেকে তৈরি হয় না, তাই ল্যাব শেষে এই লাইনটা অবশ্যই মুছে ফেলবে (ল্যাব ১০ দেখো)। `nofail` দিয়েছি বলে ভুলে গেলেও বুট আটকাবে না, তবে ৯০ সেকেন্ড পর্যন্ত অপেক্ষা করতে পারে।

ল্যাব ৭: swap চালু করো ও swappiness দেখো

```
$ sudo swapon /dev/loop20p2
$ swapon --show
$ free -h
$ cat /proc/sys/vm/swappiness
$ sudo swapoff /dev/loop20p2
```

ল্যাব ৮: tune2fs ও fsck

```
$ sudo umount /mnt/lab
$ sudo tune2fs -l /dev/loop20p1 | grep -E 'volume name|Reserved block count|Mount
count'
$ sudo tune2fs -m 1 -L LAB2 /dev/loop20p1
$ sudo e2fsck -f /dev/loop20p1
$ echo $?                                       # 0 হলে সব ঠিক
```

ল্যাব ৯: জায়গা খোঁজা

```
$ du -sh ~/* 2>/dev/null | sort -h | tail -5
$ df -hT -x tmpfs -x devtmpfs
$ df -i /
$ ncdu ~                                        # ইনস্টল থাকলে; q দিয়ে বের হও
```

ল্যাব ১০: পরিষ্কার করো

```
$ sudo umount /mnt/lab 2>/dev/null
$ sudo sed -i '\#/mnt/lab#d' /etc/fstab       # ল্যাবের লাইন মুছে দাও
$ sudo findmnt --verify
$ sudo systemctl daemon-reload
$ sudo losetup -d /dev/loop20                 # loop device খুলে দাও
$ losetup -a | grep disklab                   # কিছু না দেখালে সফল
$ rm -rf ~/disklab
```

(`sed`-এর `\#...#d` সিনট্যাক্স বুঝতে অধ্যায় ২১ দেখো।)

বোনাস চ্যালেঞ্জ: একই ল্যাব `parted -s` দিয়ে এক লাইনে করো, আর প্রথম পার্টিশনে ext4-এর বদলে XFS বানিয়ে `xfs_repair -n` চালাও।

সারসংক্ষেপ

কাজ	কমান্ড
সব ডিস্ক/পার্টিশন গাছ আকারে	`lsblk`, `lsblk -f`
UUID, টাইপ, লেবেল	`sudo blkid`, `blkid -s UUID -o value DEV`
কোথায় কী মাউন্ট	`findmnt`, `findmnt /path`
fstab যাচাই	`sudo findmnt --verify`
পার্টিশন টেবিল দেখা	`sudo fdisk -l /dev/sdX`
পার্টিশন (MBR/GPT)	`sudo fdisk /dev/sdX` → `g` / `o`, `n`, `t`, `p`, `w` / `q`
শুধু GPT	`sudo gdisk /dev/sdX`
স্ক্রিপ্টে পার্টিশন	`sudo parted -s DEV mklabel gpt mkpart NAME 1MiB 100%`
টেবিল ব্যাকআপ	`sudo sfdisk --dump DEV > file`
kernel-কে টেবিল পড়ানো	`sudo partprobe DEV`
সিগনেচার মোছা	`sudo wipefs -a DEV`
ফাইলসিস্টেম তৈরি	`mkfs.ext4 -L`, `mkfs.xfs -L`, `mkfs.vfat -F 32 -n`, `mkfs.exfat -n`
মাউন্ট	`sudo mount [-t T] [-o opts] DEV DIR`
অপশন বদলে আবার মাউন্ট	`sudo mount -o remount,rw /`
ISO মাউন্ট	`sudo mount -o loop,ro file.iso DIR`
fstab-এর সব মাউন্ট	`sudo systemctl daemon-reload && sudo mount -a`
আনমাউন্ট	`sudo umount DIR`
কে ব্যস্ত রেখেছে	`sudo fuser -vm DIR`, `sudo lsof +f -- DIR`
ফাইলসিস্টেমের জায়গা	`df -h`, `df -hT`, `df -i`
ফোল্ডারের আকার	`du -sh DIR`, `du -h -d 1 DIR | sort -h`
ইন্টারঅ্যাকটিভ	`ncdu -x DIR`
মুছে ফেলা কিন্তু খোলা ফাইল	`sudo lsof +L1`
swap তৈরি ও চালু	`mkswap DEV`, `swapon DEV`, `swapon --show`
swapfile	`fallocate -l 2G /swapfile; chmod 600; mkswap; swapon`
swappiness	`sysctl vm.swappiness=10`, `/etc/sysctl.d/99-swappiness.conf`
fsck	`sudo fsck -n DEV`, `sudo e2fsck -f -y DEV`, `sudo xfs_repair DEV`

কাজ	কমান্ড
ext4 টিউনিং	`tune2fs -l`, `tune2fs -L`, `tune2fs -m 1`
USB (root ছাড়া)	`udisksctl mount -b DEV`, `unmount`, `power-off`
loop device	`sudo losetup -fP --show img`, `losetup -d /dev/loopN`

শর্টকাট/কী এক নজরে:

টুল	মূল কী
`fdisk`	`m` সাহায্য, `p` দেখাও, `g` GPT, `o` MBR, `n` নতুন, `d` মোছো, `t` টাইপ, `l` টাইপ তালিকা, `F` খালি জায়গা, `w` লেখো, `q` বাতিল
`gdisk`	`?` সাহায্য, `p`, `n`, `d`, `t`, `L` কোড তালিকা, `c` নাম, `r` recovery, `w`, `q`
`parted`	`print`, `print free`, `mklabel`, `mkpart`, `rm`, `resizepart`, `set`, `unit`, `quit` (সব সাথে সাথে লেখা হয়!)
`ncdu`	`↑↓` / `jk`, `Enter`, `←`, `d` মোছো, `s` আকার, `n` নাম, `C` সংখ্যা, `a`, `g`, `e`, `i`, `r`, `?`, `q`

নিজেকে যাচাই করো

1. `nvme1n1p3` নামটা ভাঙলে কী কী তথ্য পাওয়া যায়? আর `sdb3`-এর সাথে নামকরণে পার্থক্য কোথায়?
2. MBR-এ সর্বোচ্চ কতগুলো primary পার্টিশন থাকতে পারে, এবং 512-byte sector-এ ডিস্কের সীমা কত? GPT কীভাবে টেবিল নষ্ট হওয়ার ঝুঁকি কমায়?
3. `fdisk` আর `parted`-এর মধ্যে ডিস্কে লেখার সময় নিয়ে সবচেয়ে বিপজ্জনক পার্থক্যটা কী?
4. `/etc/fstab`-এ `/dev/sdb1` না লিখে UUID লেখা কেন উচিত? `UUID` আর `PARTUUID`-এর পার্থক্য কী?
5. fstab-এর ষষ্ঠ ফিল্ডে root, একটা ext4 ডেটা পার্টিশন, একটা XFS পার্টিশন আর swap-এর জন্য কী মান দেবে?
6. `umount` করতে গেলে "target is busy" এলো। কোন দুটো কমান্ড দিয়ে কারণ খুঁজবে?
7. `df -h` বলছে `/` 100% ভরা, কিন্তু `du -sh /` অনেক কম দেখাচ্ছে। সম্ভাব্য দুটো কারণ ও সমাধান বলো।
8. একটা 2 GiB swapfile তৈরি করে স্থায়ীভাবে চালু করার সব ধাপ লেখো। `chmod 600` কেন জরুরি?
9. `vm.swappiness=10` মানে কি "RAM 90% ভরলে swap শুরু হবে"? ঠিক কী বোঝায়?
10. একটা মাউন্ট করা ext4 পার্টিশনে কেন `e2fsck -y` চালানো উচিত না, আর XFS-এর জন্য কোন টুল ব্যবহার করবে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. `nvme1n1p3` = দ্বিতীয় NVMe কন্ট্রোলার (`1`, গণনা 0 থেকে), namespace 1 (`n1`), পার্টিশন 3 (`p3`)। `sd` নামে ডিস্ক অক্ষর দিয়ে (`a`, `b`...), আর পার্টিশন নম্বর সরাসরি যোগ হয় (`sdb3`); NVMe-র নাম সংখ্যায় শেষ হয় বলে পার্টিশনের আগে `p` বসে।
</details>

2. ৪টা primary (বেশি লাগলে একটা extended-এর ভেতরে logical), সীমা 2 TiB। GPT ডিস্কের শেষে হেডার ও পার্টিশন টেবিলের ব্যাকআপ রাখে, আর CRC32 checksum দিয়ে নষ্ট হলে ধরে ফেলে, তাই ব্যাকআপ থেকে ফেরত আনা যায়।
3. `fdisk` (ও `gdisk`) সব পরিবর্তন মেমোরিতে রাখে, `w` চাপলে তবেই লেখে — `q` দিয়ে বাতিল করা যায়। `parted` প্রতিটা কমান্ড সাথে সাথে ডিস্কে লেখে, ফেরানোর সুযোগ নেই।
4. `sdX` নাম ডিস্ক শনাক্তের ক্রম অনুযায়ী বুটভেদে বদলাতে পারে; UUID ফাইলসিস্টেমের ভেতরে লেখা থাকে, তাই স্থির। `UUID` ফাইলসিস্টেমের (আবার mkfs করলে বদলায়), `PARTUUID` পার্টিশন টেবিলের এন্ট্রির (ফরম্যাট করলেও বদলায় না)।
5. root = `1`, ext4 ডেটা = `2`, XFS = `0`, swap = `0`।
6. `sudo fuser -vm /mount/point` এবং `sudo lsof +f -- /mount/point`। তারপর সেই প্রসেস বন্ধ করো বা শেলে `cd` দিয়ে বেরিয়ে আসো।
7. (ক) কোনো প্রসেস একটা মুছে ফেলা ফাইল খোলা রেখেছে — `sudo lsof +L1` দিয়ে খুঁজে সেই প্রসেস/সার্ভিস রিস্টার্ট করো। (খ) কোনো মাউন্ট পয়েন্টের নিচে ফাইল লুকিয়ে আছে — আনমাউন্ট করে দেখো (বা `mount --bind / /mnt/x` দিয়ে নিচের অংশ দেখো)। এছাড়া reserved block-ও কিছু পার্থক্য করে।
8. `sudo fallocate -l 2G /swapfile` (বা `dd`) → `sudo chmod 600 /swapfile` → `sudo mkswap /swapfile` → `sudo swapon /swapfile` → `/etc/fstab`-এ `/swapfile none swap defaults 0 0` যোগ করো → `swapon --show` দিয়ে যাচাই। swapfile-এ RAM-এর গোপন তথ্য (পাসওয়ার্ড, কী) লেখা হতে পারে, তাই অন্য কেউ যেন পড়তে না পারে।
9. না। swappiness হলো একটা তুলনামূলক ওজন — মেমোরির চাপ এলে kernel অ্যাপের মেমোরি (anonymous page) swap করবে নাকি ফাইল cache ফেলে দেবে, তার পছন্দ। কম মান মানে kernel অ্যাপের মেমোরিকে RAM-এ রাখতে বেশি চেষ্টা করে।
10. মাউন্ট থাকা অবস্থায় kernel একই সময়ে ফাইলসিস্টেমে লিখছে; fsck তার নিচ থেকে মেটাডেটা বদলালে দুজনের হিসাব সংঘর্ষে ফাইলসিস্টেম আরও নষ্ট হতে পারে। আগে `umount` করতে হয়। XFS-এর জন্য `xfs_repair` (আগে `xfs_repair -n` দিয়ে শুধু পরীক্ষা)।

অধ্যায় ৩৩: LVM ও RAID

> আগের অধ্যায়ে (অধ্যায় ৩২) তুমি শিখেছ ডিস্ক, পার্টিশন, ফাইলসিস্টেম আর মাউন্ট। সেখানে একটা সমস্যা চোখে পড়ার কথা: পার্টিশন একবার বানালে তার সাইজ বদলানো ঝামেলার, আর একটা ডিস্ক নষ্ট হলে তার সব ডেটা চলে যায়। এই অধ্যায়ের দুটো প্রযুক্তি ঠিক এই দুটো সমস্যার সমাধান করে। **LVM** দেয় নমনীয়তা (flexibility), আর **RAID** দেয় নির্ভরযোগ্যতা (redundancy) ও গতি।

এই অধ্যায়ে যা শিখবে

- সাধারণ পার্টিশনের সীমাবদ্ধতা কী, আর LVM কেন দরকার
- LVM-এর তিন স্তর: PV (Physical Volume), VG (Volume Group), LV (Logical Volume), আর PE/LE কী জিনিস
- ভেতরে কীভাবে কাজ করে: device-mapper, `/dev/mapper/`, LVM metadata
- `pvcreate`, `vgcreate`, `lvcreate` দিয়ে শূন্য থেকে LVM বানানো
- `pvs`, `vgs`, `lvs` এবং `*display` কমান্ড দিয়ে অবস্থা দেখা
- `lvextend -r`, `resize2fs`, `xfs_growfs` দিয়ে চলন্ত অবস্থায় জায়গা বাড়ানো
- `lvreduce` কেন বিপজ্জনক, আর কীভাবে নিরাপদে করতে হয়
- নতুন ডিস্ক যোগ (`vgextend`), ডেটা সরানো (`pvmove`), ডিস্ক বাদ দেওয়া (`vgreduce`)
- LVM snapshot: কীভাবে কাজ করে (Copy-on-Write), বানানো, ফিরিয়ে আনা (merge)
- RAID কী, RAID 0/1/5/6/10 এর তুলনা, ডায়াগ্রাম সহ
- Hardware RAID, Software RAID, Fake RAID এর পার্থক্য
- `mdadm` দিয়ে software RAID তৈরি, `/proc/mdstat` পড়া, কনফিগ সেভ করা
- নষ্ট ডিস্ক বদলানো (fail → remove → add → rebuild)
- RAID কেন ব্যাকআপ **নয়**
- LVM আর RAID একসাথে ব্যবহার

---

১. সমস্যাটা কী? সাধারণ পার্টিশনের সীমাবদ্ধতা

ধরো তুমি একটা সার্ভারে ৫০০ GB ডিস্ক নিয়ে এভাবে পার্টিশন করলে:

```
/dev/sda
┌──────────┬──────────────────┬──────────────────────────────┐
│ sda1     │ sda2             │ sda3                         │
│ /boot    │ /   (৫০ GB)      │ /var  (৪৪৯ GB)              │
│ 1 GB     │                  │                              │
└──────────┴──────────────────┴──────────────────────────────┘
```

ছয় মাস পরে `/` ভরে গেল, অথচ `/var`-এ ৩০০ GB খালি পড়ে আছে। এখন কী করবে?

- `/var` ছোট করে `/`-কে বড় করতে হলে পার্টিশনগুলোকে ডিস্কের ওপর **পাশাপাশি সরাতে** হবে। এটা ধীর, ঝুঁকিপূর্ণ, আর সাধারণত সার্ভার বন্ধ রেখে করতে হয়।
- নতুন একটা ডিস্ক লাগালে সেটা আলাদা `/dev/sdb` হয়। `/`-এর সাথে জোড়া লাগানো যায় না। বড়জোর অন্য কোথাও মাউন্ট করা যায়।

উপমা: সাধারণ পার্টিশন হলো ইটের দেয়াল দিয়ে ঘর ভাগ করা। একবার দেয়াল তুললে ঘর বড়-ছোট করতে দেয়াল ভাঙতে হয়। LVM হলো অফিসের নড়াচড়া-যোগ্য পার্টিশন বোর্ড (movable partition)। বোর্ড সরিয়ে যখন খুশি ঘরের মাপ বদলানো যায়, এমনকি পাশের বিল্ডিং ভাড়া নিয়ে জোড়াও দেওয়া যায়।

---

২. LVM কী এবং কেন

LVM (Logical Volume Manager) হলো kernel আর ফাইলসিস্টেমের মাঝখানে একটা স্তর (abstraction layer, অর্থাৎ বাস্তব জিনিসকে আড়াল করে একটা সুবিধাজনক রূপ দেখানো)। এটা একাধিক ফিজিক্যাল ডিস্ক বা পার্টিশনকে এক বড় "স্টোরেজ পুলে" জমা করে, তারপর সেই পুল থেকে ইচ্ছেমতো সাইজের "ভার্চুয়াল পার্টিশন" কেটে দেয়।

LVM-এর সুবিধা:

সুবিধা	ব্যাখ্যা
অনলাইন রিসাইজ	ফাইলসিস্টেম মাউন্ট থাকা অবস্থায় (সার্ভার চালু রেখে) জায়গা বাড়ানো যায়
একাধিক ডিস্ক জোড়া	৩টা ২ TB ডিস্ক মিলিয়ে একটা ৬ TB volume
Snapshot	কোনো মুহূর্তের "ছবি" রেখে দেওয়া, আপগ্রেডের আগে নিরাপত্তা
লাইভ মাইগ্রেশন	`pvmove` দিয়ে পুরনো ডিস্ক থেকে নতুন ডিস্কে ডেটা সরানো, সার্ভার চালু রেখেই
নাম দিয়ে চেনা	`/dev/sdb3` না বলে `/dev/vgdata/mysql`, পড়তে সহজ
Thin provisioning	বাস্তবে যত জায়গা আছে তার চেয়ে বেশি "প্রতিশ্রুতি" দেওয়া (অ্যাডভান্সড)

> 📝 **মনে রাখো:** Ubuntu Server ইনস্টলার, RHEL/Rocky/AlmaLinux ইনস্টলার — ডিফল্ট সেটআপেই LVM ব্যবহার করে। তাই তুমি নিজে LVM না বানালেও প্রোডাকশন সার্ভারে প্রায় নিশ্চিত দেখা হবে। Fedora Workstation (৩৩ সংস্করণ থেকে) অবশ্য ডিফল্টে Btrfs ব্যবহার করে, LVM নয়।

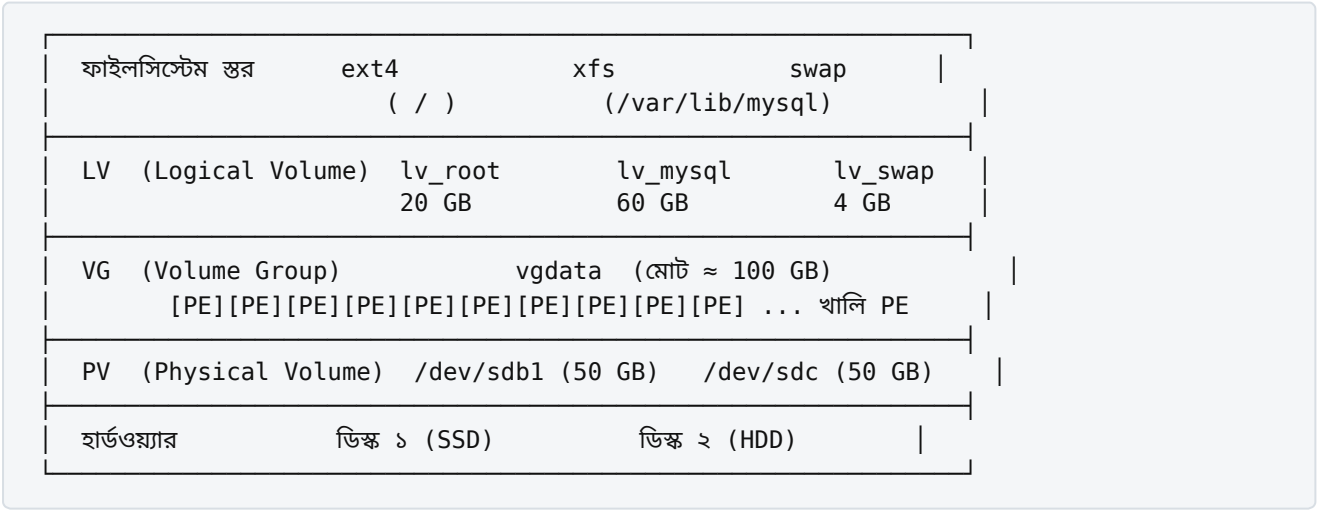
---

৩. LVM-এর তিন স্তর: PV, VG, LV

LVM বুঝতে তিনটা শব্দ মাথায় গেঁথে নাও:

স্তর	পুরো নাম	কী	উপমা
PV	Physical Volume	একটা ডিস্ক বা পার্টিশন, যাকে LVM-এর জন্য "চিহ্নিত" করা হয়েছে	আলাদা আলাদা জমির প্লট
VG	Volume Group	এক বা একাধিক PV মিলিয়ে বানানো স্টোরেজ পুল	সব প্লট মিলিয়ে একটা বড় খামার
LV	Logical Volume	VG থেকে কেটে নেওয়া অংশ, যার ওপর ফাইলসিস্টেম বসে	খামারের ভেতরে ভাগ করা ক্ষেত (ধান, পাট, সবজি)

স্তরের ডায়াগ্রাম

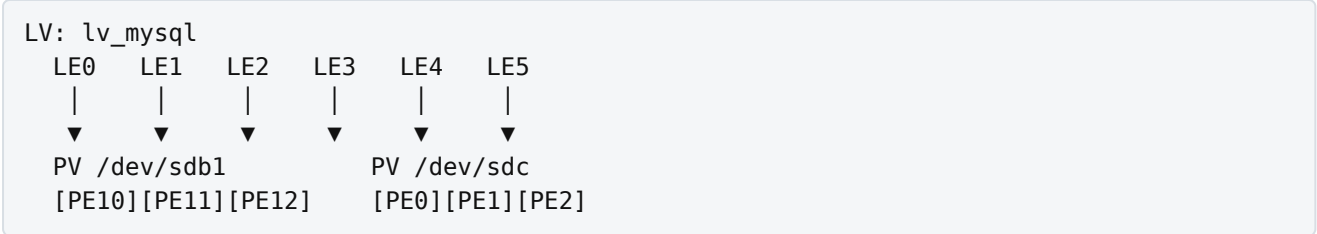


খেয়াল করো: `lv_mysql`-এর ডেটা আসলে দুটো ডিস্কেই ছড়িয়ে থাকতে পারে। ফাইলসিস্টেম সেটা জানেও না, জানার দরকারও নেই।

PE আর LE: ইট দিয়ে বানানো দেয়াল

VG তৈরি হলে LVM পুরো জায়গাটাকে সমান সাইজের ছোট ছোট টুকরোয় ভাগ করে। প্রতিটা টুকরোকে বলে **PE (Physical Extent)**। ডিফল্ট সাইজ **4 MiB**।

LV বানানো মানে আসলে কিছু PE বরাদ্দ (allocate) করা। LV-এর দিক থেকে প্রতিটা টুকরোকে বলে **LE (Logical Extent)**। প্রতিটা LE একটা PE-এর সাথে ম্যাপ করা থাকে।



তাই LV বড় করা মানে আরো কিছু খালি PE যোগ করে ম্যাপ লম্বা করা। কোনো ডেটা সরাতে হয় না। এ জন্যই কাজটা এত দ্রুত আর নিরাপদ।

> 💡 **টিপস:** LV-এর সাইজ সবসময় PE সাইজের গুণিতক হয়। তুমি 10.001 GB চাইলে LVM সেটাকে কাছের PE সীমায় গোল (round up) করে নেবে এবং একটা বার্তা দেখাবে।

---

৪. ভেতরে কীভাবে কাজ করে (Under the hood)

device-mapper

LVM নিজে ডেটা পড়ে-লেখে না। আসল কাজটা করে kernel-এর একটা অংশ, নাম **device-mapper** (dm)। device-mapper একটা "ম্যাপিং টেবিল" অনুযায়ী ভার্চুয়াল ব্লক ডিভাইসের অনুরোধ আসল ডিস্কে পাঠিয়ে দেয়।

```
অ্যাপ (mysql)  ──write──►  ফাইলসিস্টেম (xfs)
                                  │  "ব্লক 5000 লেখো"
                                  ▼
                        /dev/dm-1  (device-mapper ডিভাইস)
                                  │  ম্যাপিং টেবিল দেখে:
                                  │  "ব্লক 5000 = /dev/sdc-র অফসেট X"
                                  ▼
                              /dev/sdc  (আসল ডিস্ক)
```

User-space-এর LVM টুলগুলো (`lvcreate` ইত্যাদি) শুধু metadata আপডেট করে এবং kernel-কে নতুন ম্যাপিং টেবিল দেয়। এজন্য তুমি একই LV-কে তিনটা নামে দেখবে:

```
$ ls -l /dev/vgdata/lv_mysql /dev/mapper/vgdata-lv_mysql
lrwxrwxrwx 1 root root 7 Nov 12 10:15 /dev/mapper/vgdata-lv_mysql -> ../dm-1
lrwxrwxrwx 1 root root 7 Nov 12 10:15 /dev/vgdata/lv_mysql -> ../dm-1
```

তিনটাই আসলে একই `/dev/dm-1`। প্রথম দুটো symlink (সফট লিংক, দেখো অধ্যায় ১১)। `/etc/fstab`-এ `/dev/mapper/...` বা `/dev/vgname/lvname` লেখা নিরাপদ, কিন্তু `/dev/dm-1` লিখবে না, কারণ বুটভেদে নম্বর বদলাতে পারে।

> 📝 **মনে রাখো:** `/dev/mapper/` নামে VG আর LV-এর মাঝে একটা হাইফেন (`-`) থাকে। নামের ভেতরেই হাইফেন থাকলে সেটা দুটো হাইফেন হয়ে যায়। যেমন VG `my-vg`, LV `data` → `/dev/mapper/my--vg-data`।

dmsetup দিয়ে ম্যাপিং দেখা

```
$ sudo dmsetup ls
vgdata-lv_mysql (252:1)
vgdata-lv_root  (252:0)

$ sudo dmsetup table vgdata-lv_mysql
```

```
0 62914560 linear 8:17 41945088
62914560 62914560 linear 8:32 2048
```

এখানে দেখা যাচ্ছে LV-এর প্রথম অংশ ডিভাইস `8:17` (`sdb1`) থেকে, দ্বিতীয় অংশ `8:32` (`sdc`) থেকে আসছে। `linear` মানে টুকরোগুলো পরপর জোড়া।

LVM metadata কোথায় থাকে

প্রতিটা PV-এর শুরুর দিকে একটা ছোট জায়গায় LVM তার **metadata** (VG-এর নাম, কোন LV কোন PE ব্যবহার করে ইত্যাদি) লিখে রাখে। সব PV-তে metadata-র কপি থাকে। এছাড়া প্রতিবার পরিবর্তনের পর টেক্সট আকারে ব্যাকআপ রাখে:

- `/etc/lvm/backup/<VG-নাম>`: সর্বশেষ অবস্থা
- `/etc/lvm/archive/`: আগের অবস্থাগুলো

ভুল করে কোনো LV মুছে ফেললে `vgcfgrestore` দিয়ে metadata আগের অবস্থায় ফেরানো সম্ভব (যদি এর মধ্যে ওই জায়গায় নতুন কিছু লেখা না হয়ে থাকে)। এটা জরুরি অবস্থার টুল, রোজকার নয়।

```
$ sudo vgcfgrestore --list vgdata
```

---

৫. প্রস্তুতি: প্যাকেজ ও ডিস্ক চেনা

বেশিরভাগ সার্ভার distro-তে LVM টুল আগে থেকেই থাকে। না থাকলে:

```
# Ubuntu / Debian
$ sudo apt install lvm2

# Fedora / RHEL / Rocky / AlmaLinux
$ sudo dnf install lvm2
```

আগে দেখে নাও কোন ডিস্ক খালি আছে (অধ্যায় ৩২-এ `lsblk` শিখেছ):

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   40G  0 disk
├─sda1   8:1    0    1G  0 part /boot
└─sda2   8:2    0   39G  0 part /
sdb      8:16   0   10G  0 disk
sdc      8:32   0   10G  0 disk
sdd      8:48   0   10G  0 disk
```

`sdb`, `sdc`, `sdd` খালি। এগুলো নিয়েই আমরা কাজ করব।

> ⚠️ **সতর্কতা:** এই অধ্যায়ের প্রায় সব কমান্ড ডিস্কের ডেটা মুছে ফেলতে পারে। ডিভাইসের নাম তিনবার মিলিয়ে নাও। ভুল ডিস্কে `pvcreate` বা `mdadm --create` চালালে সেই ডিস্কের ডেটা হারাবে। শেখার সময় ভার্চুয়াল মেশিন বা loop device ব্যবহার করো (নিচের "হাতে-কলমে অনুশীলন" অংশে দেখানো আছে)।

পুরো ডিস্ক নাকি পার্টিশন?

PV হিসেবে পুরো ডিস্ক (`/dev/sdb`) বা পার্টিশন (`/dev/sdb1`) দুটোই চলে। অনেকে একটা পার্টিশন বানিয়ে তার type "Linux LVM" (GPT-তে `fdisk` -এর type `lvm` , MBR-এ `8e`) দেয়, যাতে অন্য টুল বা মানুষ দেখেই বোঝে এটা খালি ডিস্ক নয়। দুটো পদ্ধতিই সঠিক।

---

৬. ধাপ ১: PV বানানো — `pvcreate`

কী করে: একটা ডিস্ক/পার্টিশনকে LVM-এর জন্য তৈরি করে (শুরুতে LVM label ও metadata এলাকা লেখে)।

Syntax:

```
pvcreate [অপশন] ডিভাইস...
```

অপশন	কাজ
`-v`	বিস্তারিত বার্তা দেখায়
`-f`	জোর করে (আগের সিগনেচার থাকলেও) — খুব সাবধানে
`-y`	সব প্রশ্নে "হ্যাঁ" ধরে নেয়
`--dataalignment`	ডেটা শুরুর alignment ঠিক করে (সাধারণত ডিফল্টই ভালো)

উদাহরণ:

```
$ sudo pvcreate /dev/sdb /dev/sdc
  Physical volume "/dev/sdb" successfully created.
  Physical volume "/dev/sdc" successfully created.
```

ডিস্কে আগে কোনো ফাইলসিস্টেম বা পার্টিশন টেবিল থাকলে LVM জিজ্ঞেস করবে:

```
WARNING: ext4 signature detected on /dev/sdb at offset 1080. Wipe it? [y/n]:
```

এখানে থামো, ভাবো, তারপর উত্তর দাও। আগের সিগনেচার পরিষ্কার করতে `wipefs` টুলও আছে:

```
$ sudo wipefs -a /dev/sdb      # সব সিগনেচার মুছবে, সাবধান!
```

---

৭. ধাপ ২: VG বানানো — `vgcreate`

কী করে: এক বা একাধিক PV মিলিয়ে একটা Volume Group তৈরি করে।

Syntax:

```
vgcreate [অপশন] VG-নাম PV...
```

অপশন	কাজ
`-s`, `--physicalextentsize`	PE সাইজ ঠিক করে (ডিফল্ট 4M), যেমন `-s 16M`
`-v`	বিস্তারিত বার্তা

উদাহরণ:

```
$ sudo vgcreate vgdata /dev/sdb /dev/sdc
  Volume group "vgdata" successfully created
```

> 💡 **টিপস:** VG-এর নাম অর্থবহ রাখো: `vgdata`, `vg_db`, `vg_backup`। RHEL ইনস্টলার ডিফল্টে `rhel` (বা `rl`, `almalinux`) নাম দেয়, Ubuntu দেয় `ubuntu-vg`।

> 💡 **টিপস:** `vgcreate` চালানোর সময় ডিভাইসে PV না থাকলে আধুনিক LVM নিজে থেকেই `pvcreate` করে নেয়। তবে শেখার সময় ধাপগুলো আলাদা করে চালানো ভালো, বোঝা সহজ হয়।

---

৮. ধাপ ৩: LV বানানো — `lvcreate`

কী করে: VG থেকে জায়গা কেটে একটা Logical Volume তৈরি করে।

Syntax:

```
lvcreate [অপশন] -n LV-নাম VG-নাম
```

অপশন	কাজ
`-L সাইজ`	নির্দিষ্ট সাইজ: `-L 5G`, `-L 500M`, `-L 1T`
`-l সংখ্যা/শতাংশ`	PE সংখ্যা বা শতাংশ: `-l 100`, `-l 50%VG`, `-l 100%FREE`
`-n নাম`	LV-এর নাম
`-s`	snapshot বানায় (পরে দেখব)
`--type`	`linear` (ডিফল্ট), `striped`, `raid1`, `raid5`, `thin-pool` ইত্যাদি

অপশন	কাজ
`-i সংখ্যা`	stripe সংখ্যা (কয়টা PV-তে ছড়াবে)
`-m সংখ্যা`	mirror সংখ্যা (`--type raid1` এর সাথে)

উদাহরণ:

```
$ sudo lvcreate -L 5G -n lv_web vgdata
  Logical volume "lv_web" created.

$ sudo lvcreate -l 100%FREE -n lv_db vgdata
  Logical volume "lv_db" created.
```

দ্বিতীয় কমান্ডে VG-এর বাকি সব খালি জায়গা `lv_db` নিয়ে নিল।

> 💡 **টিপস:** শুরুতে সব জায়গা দিয়ে দিও না। VG-তে কিছু খালি জায়গা রেখে দাও। পরে যে LV-র দরকার পড়বে তাকে বাড়িয়ে দেওয়া যাবে, আর snapshot-এর জন্যও খালি জায়গা লাগে। LV বড় করা সহজ, ছোট করা কঠিন।

ফাইলসিস্টেম বানানো ও মাউন্ট

LV হলো একটা সাধারণ block device। এর ওপর যেকোনো ফাইলসিস্টেম বসানো যায়:

```
$ sudo mkfs.ext4 /dev/vgdata/lv_web
$ sudo mkfs.xfs  /dev/vgdata/lv_db

$ sudo mkdir -p /srv/web /srv/db
$ sudo mount /dev/vgdata/lv_web /srv/web
$ sudo mount /dev/vgdata/lv_db  /srv/db

$ df -h /srv/web /srv/db
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lv_web   4.9G   24K  4.6G   1% /srv/web
/dev/mapper/vgdata-lv_db     15G  140M   15G   1% /srv/db
```

রিবুটের পরও মাউন্ট রাখতে `/etc/fstab`-এ যোগ করো:

```
/dev/mapper/vgdata-lv_web  /srv/web  ext4  defaults  0 2
/dev/mapper/vgdata-lv_db   /srv/db   xfs   defaults  0 0
```

তারপর পরীক্ষা করো (ভুল থাকলে এখনই ধরা পড়বে, রিবুটে নয়):

```
$ sudo mount -a
$ sudo systemctl daemon-reload     # systemd-কে নতুন fstab জানাতে
```

> ⚠️ **সতর্কতা:** fstab-এ ভুল থাকলে সিস্টেম বুটে emergency mode-এ আটকে যেতে পারে। সবসময় `mount -a` বা `findmnt --verify` দিয়ে যাচাই করো।

---

৯. অবস্থা দেখা: `pvs`, `vgs`, `lvs`

এই তিনটা সংক্ষিপ্ত (এক লাইনে একটা) রিপোর্ট দেয়। দৈনন্দিন কাজে এগুলোই সবচেয়ে বেশি লাগে।

```
$ sudo pvs
  PV         VG     Fmt  Attr PSize   PFree
  /dev/sdb   vgdata lvm2 a--  <10.00g    0
  /dev/sdc   vgdata lvm2 a--  <10.00g    0
  /dev/sdd          lvm2 ---   10.00g 10.00g

$ sudo vgs
  VG     #PV #LV #SN Attr   VSize  VFree
  vgdata   2   2   0 wz--n- 19.99g    0

$ sudo lvs
  LV     VG     Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_db  vgdata -wi-ao---- 14.99g
  lv_web vgdata -wi-ao----  5.00g
```

কলামগুলোর মানে:

কলাম	মানে
`PSize` / `PFree`	PV-এর মোট সাইজ / খালি জায়গা
`#PV #LV #SN`	VG-তে কয়টা PV, কয়টা LV, কয়টা snapshot
`VSize` / `VFree`	VG-এর মোট / খালি জায়গা — **এটা দেখেই বোঝো আর কতটা বাড়ানো যাবে**
`LSize`	LV-এর সাইজ
`Origin`	snapshot হলে কোন LV-র snapshot
`Data%`	snapshot বা thin pool কতটা ভরেছে
`Cpy%Sync`	LVM RAID/mirror কতটা sync হয়েছে

`<10.00g` মানে "১০ GB-এর সামান্য কম" — metadata-র জন্য কিছু জায়গা গেছে।

Attr কলাম পড়া (lvs)

`-wi-ao----` প্রতিটা অক্ষরের একটা মানে আছে। প্রধানগুলো:

অবস্থান	উদাহরণ	মানে
১	`-` / `s` / `o` / `r` / `t` / `V`	সাধারণ / snapshot / origin / raid / thin pool / thin volume

অবস্থান	উদাহরণ	মানে
২	`w` / `r`	writeable / read-only
৩	`i` / `c`	allocation policy: inherited / contiguous
৫	`a`	active (চালু)
৬	`o`	open (মাউন্ট করা বা ব্যবহার হচ্ছে)

বিস্তারিত দেখা: `pvdisplay`, `vgdisplay`, `lvdisplay`

```
$ sudo vgdisplay vgdata
  --- Volume group ---
  VG Name               vgdata
  Format                lvm2
  VG Access             read/write
  VG Status             resizable
  Cur LV                2
  Open LV               2
  Cur PV                2
  VG Size               19.99 GiB
  PE Size               4.00 MiB
  Total PE              5118
  Alloc PE / Size       5118 / 19.99 GiB
  Free  PE / Size       0 / 0
  VG UUID               3Kp1fd-...
```

নিজের পছন্দের কলাম

```
$ sudo lvs -o lv_name,vg_name,lv_size,devices
  LV     VG     LSize  Devices
  lv_db  vgdata 14.99g /dev/sdb(1280)
  lv_db  vgdata 14.99g /dev/sdc(0)
  lv_web vgdata  5.00g /dev/sdb(0)
```

`-o devices` দেখায় কোন LV কোন ডিস্কের কোন PE থেকে শুরু। `lv_db` দুটো ডিস্কে ছড়িয়ে আছে। অন্য কাজের অপশন:

কমান্ড	কাজ
`lvs -a`	লুকানো (internal) LV-ও দেখায়, যেমন RAID-এর অংশ
`lvs --units m`	সাইজ MiB-তে দেখায়
`pvs -o +pv_used`	বাড়তি কলাম যোগ (`+` দিয়ে)
`pvscan`, `vgscan`, `lvscan`	সব ডিস্ক স্ক্যান করে LVM খোঁজে

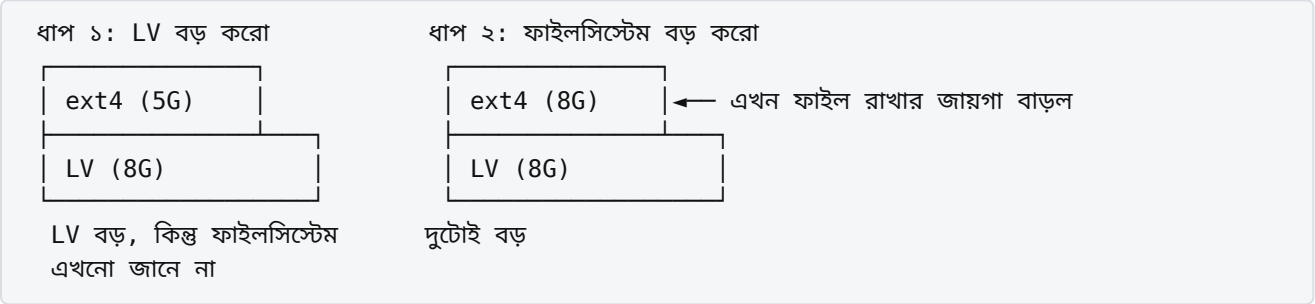
---

১০. জায়গা বাড়ানো: `lvextend`, `resize2fs`, `xfs_growfs`

এটাই LVM-এর সবচেয়ে বড় সুবিধা, আর বাস্তবে সবচেয়ে বেশি দরকার পড়ে। রাত ২টায় `/var` ভরে গেছে, সার্ভার বন্ধ না করেই বাড়াতে হবে।

দুই স্তরের কাজ

মনে রাখো, জায়গা বাড়াতে **দুটো** জিনিস বড় করতে হয়:



শুধু LV বড় করলে `df -h` আগের সাইজই দেখাবে। এটা নতুনদের সবচেয়ে সাধারণ বিভ্রান্তি।

`lvextend` এর syntax

```
lvextend [অপশন] LV-পাথ [PV...]
```

অপশন	কাজ
`-L +5G`	৫ GB **যোগ** করো (`+` চিহ্নটা খেয়াল করো)
`-L 20G`	মোট সাইজ **২০ GB** করো
`-l +100%FREE`	VG-এর সব খালি জায়গা যোগ করো
`-l +50%FREE`	খালি জায়গার অর্ধেক যোগ করো
`-r`, `--resizefs`	LV বাড়ানোর পর ফাইলসিস্টেমও বাড়িয়ে দাও (একসাথে দুই ধাপ)

> ⚠️ **সতর্কতা:** `-L +5G` আর `-L 5G` এর পার্থক্য বিশাল। প্রথমটা ৫ GB যোগ করে, দ্বিতীয়টা মোট সাইজ ৫ GB করে। `lvextend` -এ ছোট সাইজ দিলে LVM error দেবে (কারণ extend মানে শুধু বাড়ানো), কিন্তু `lvresize` -এ দিলে ছোট করে ফেলতে পারে।

সহজ পথ: `-r` দিয়ে একবারে

প্রথমে VG-তে খালি জায়গা আছে কিনা দেখো (`vgs` -এর `VFree`)। না থাকলে আগে নতুন ডিস্ক যোগ করতে হবে (পরের অংশে)।

```
$ sudo lvextend -r -L +3G /dev/vgdata/lv_web
  Size of logical volume vgdata/lv_web changed from 5.00 GiB (1280 extents) to 8.00 GiB
(2048 extents).
  Logical volume vgdata/lv_web successfully resized.
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/mapper/vgdata-lv_web is mounted on /srv/web; on-line resizing
required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/mapper/vgdata-lv_web is now 2097152 (4k) blocks long.

$ df -h /srv/web
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lv_web  7.8G   24K  7.4G   1% /srv/web
```

`-r` ভেতরে `fsadm` নামের একটা স্ক্রিপ্ট ডাকে, যেটা ফাইলসিস্টেমের ধরন দেখে ঠিক টুল (ext4 হলে `resize2fs`, XFS হলে `xfs_growfs`) চালায়। ফাইলসিস্টেম মাউন্ট থাকা অবস্থাতেই কাজ হয়।

হাতে-হাতে পথ: আলাদা দুই ধাপ

কখনো `-r` ভুলে গেলে, বা বুঝে শুনে আলাদা করে করতে চাইলে:

ext2/ext3/ext4 এর জন্য `resize2fs`:

```
$ sudo lvextend -L +2G /dev/vgdata/lv_web
$ sudo resize2fs /dev/vgdata/lv_web
```

`resize2fs` সাইজ না দিলে ডিভাইসের পুরো জায়গা পর্যন্ত বাড়ায়। এটা **ডিভাইসের পাথ** নেয়।

XFS এর জন্য `xfs_growfs`:

```
$ sudo lvextend -L +5G /dev/vgdata/lv_db
$ sudo xfs_growfs /srv/db
meta-data=/dev/mapper/vgdata-lv_db isize=512    agcount=4, agsize=982016 blks
...
data blocks changed from 3928064 to 5238784
```

> 📝 **মনে রাখো:** `xfs_growfs` **মাউন্ট পয়েন্ট** নেয় (যেমন `/srv/db`), আর XFS শুধু মাউন্ট থাকা অবস্থাতেই বড় করা যায়। ext4-এর `resize2fs` নেয় **ডিভাইস পাথ**, এবং মাউন্ট বা আনমাউন্ট দুই অবস্থাতেই বাড়াতে পারে।

ফাইলসিস্টেম	বড় করা	ছোট করা	টুল
ext4	মাউন্ট অবস্থায়ও পারে	শুধু আনমাউন্ট অবস্থায়	`resize2fs`
XFS	শুধু মাউন্ট অবস্থায়	**একেবারেই পারে না**	`xfs_growfs`

ফাইলসিস্টেম	বড় করা	ছোট করা	টুল
Btrfs	মাউন্ট অবস্থায়	মাউন্ট অবস্থায়	`btrfs filesystem resize`
swap	`swapoff` → `lvextend` → `mkswap` → `swapon`	একইভাবে	`mkswap`

বাস্তব ঘটনা: রুট (/) ভরে গেছে

Ubuntu Server ইনস্টলার অনেক সময় `ubuntu-lv`-কে ডিস্কের পুরো জায়গা দেয় না, বাকিটা VG-তে খালি রেখে দেয়। তখন অনেকে ভাবে "১০০ GB ডিস্কে মাত্র অল্প জায়গা কেন?" সমাধান:

```
$ sudo vgs
  VG        #PV #LV #SN Attr   VSize   VFree
  ubuntu-vg   1   1   0 wz--n- <98.00g  49.00g

$ sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
```

---

১১. নতুন ডিস্ক যোগ করা ও সরানো

VG বড় করা: `vgextend`

VG-তে জায়গা ফুরিয়েছে? নতুন ডিস্ক লাগাও, PV বানাও, VG-তে ঢোকাও:

```
$ sudo pvcreate /dev/sdd
  Physical volume "/dev/sdd" successfully created.

$ sudo vgextend vgdata /dev/sdd
  Volume group "vgdata" successfully extended

$ sudo vgs
  VG     #PV #LV #SN Attr   VSize  VFree
  vgdata   3   2   0 wz--n- 29.99g 10.00g
```

এখন `VFree` ১০ GB। যেকোনো LV-কে `lvextend` দিয়ে বাড়াও। ক্লাউডে (AWS EBS, Azure disk ইত্যাদি) এভাবেই নতুন ভলিউম জুড়ে জায়গা বাড়ানো হয়।

পুরনো ডিস্ক থেকে ডেটা সরানো: `pvmove`

ধরো `/dev/sdb` পুরনো হয়ে গেছে, SMART সতর্কতা দিচ্ছে। সার্ভার চালু রেখেই এর সব ডেটা অন্য PV-তে সরাতে পারো:

```
$ sudo pvmove /dev/sdb
  /dev/sdb: Moved: 12.45%
```

```
  /dev/sdb: Moved: 57.80%
  /dev/sdb: Moved: 100.00%
```

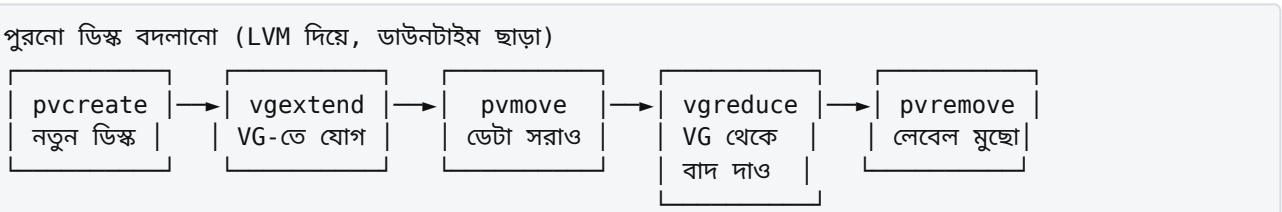
নির্দিষ্ট গন্তব্য দিতে চাইলে: `pvmove /dev/sdb /dev/sdd`। শর্ত: বাকি PV-গুলোতে যথেষ্ট খালি জায়গা থাকতে হবে।

> 💡 **টিপস:** `pvmove` বড় ডিস্কে ঘণ্টার পর ঘণ্টা লাগতে পারে। মাঝপথে বাধা পড়লে (রিবুট ইত্যাদি) শুধু `pvmove` আবার চালালে যেখানে থেমেছিল সেখান থেকে চলবে। এমন লম্বা কাজ `tmux`-এর ভেতরে চালাও (অধ্যায় ২৫)।

ডিস্ক বাদ দেওয়া: `vgreduce` ও `pvremove`

```
$ sudo vgreduce vgdata /dev/sdb
  Removed "/dev/sdb" from volume group "vgdata"

$ sudo pvremove /dev/sdb
  Labels on physical volume "/dev/sdb" successfully wiped.
```

এখন `/dev/sdb` খুলে ফেলা নিরাপদ। পুরো প্রক্রিয়া এক নজরে:



অন্যান্য দরকারি কমান্ড

কমান্ড	কাজ
`lvrename vgdata lv_web lv_www`	LV-এর নাম বদলানো
`vgrename vgdata vgprod`	VG-এর নাম বদলানো (রুট VG হলে fstab/grub ঠিক করতে হবে)
`lvremove /dev/vgdata/lv_old`	LV মুছে ফেলা (আগে umount করো)
`vgremove vgdata`	VG মুছে ফেলা (আগে সব LV মুছতে হবে)
`lvchange -an vgdata/lv_web`	LV নিষ্ক্রিয় (deactivate) করা
`vgchange -ay`	সব VG-এর সব LV সক্রিয় করা
`pvresize /dev/sdb`	ক্লাউডে ডিস্ক বড় করার পর PV-কে নতুন সাইজ জানানো

> 💡 **টিপস:** ক্লাউডে একটা ডিস্কের সাইজ বাড়ালে (যেমন ২০ GB → ৫০ GB) PV আপনা-আপনি বাড়ে না। ডিস্কে পার্টিশন থাকলে আগে পার্টিশন বড় করো (`growpart /dev/sdb 1`, `cloud-guest-utils` /

> `cloud-utils-growpart` প্যাকেজে), তারপর `pvresize`, তারপর `lvextend -r`।

---

১২. LV ছোট করা: `lvreduce` — সবচেয়ে বিপজ্জনক কাজ

বড় করা সহজ, কারণ নতুন জায়গা খালি। ছোট করা বিপজ্জনক, কারণ LV-এর শেষের অংশ কেটে ফেলা হয়। সেখানে যদি ফাইলের ডেটা থাকে, সেটা চিরতরে চলে যায়।

```
ভুল ক্রম (ডেটা নষ্ট!):
┌──────────────────────────────────────┐
│ ফাইলসিস্টেম ১০G (ডেটা শেষ পর্যন্ত) │
├──────────────────┬───────────────────┤
│ LV (৬G রাখলাম)   │ ✂ কেটে ফেলা      │ ← এখানকার ডেটা হারিয়ে গেল
└──────────────────┴───────────────────┘

সঠিক ক্রম:
ধাপ ১: ফাইলসিস্টেম ছোট করো (ডেটা সামনে সরে আসে)
ধাপ ২: তারপর LV ছোট করো
```

নিয়ম

1. **আগে ব্যাকআপ নাও।** সত্যি বলছি, এটা ঐচ্ছিক নয়।
2. **XFS ছোট করা যায় না।** XFS হলে একমাত্র পথ: ব্যাকআপ → LV মুছে নতুন ছোট LV বানাও → রিস্টোর।
3. ext4 ছোট করতে হলে **আনমাউন্ট** করতে হবে। রুট (`/`) ফাইলসিস্টেম হলে live USB থেকে বুট করতে হবে।
4. ফাইলসিস্টেমে যত ডেটা আছে তার চেয়ে ছোট করা যাবে না।

নিরাপদ উপায়: `-r` দিয়ে

`lvreduce -r` ঠিক ক্রমে (আগে ফাইলসিস্টেম, পরে LV) কাজ করে:

```
$ sudo umount /srv/web
$ sudo lvreduce -r -L 4G /dev/vgdata/lv_web
fsck from util-linux 2.39.3
/dev/mapper/vgdata-lv_web: 11/327680 files (0.0% non-contiguous), 42078/2097152 blocks
resize2fs 1.47.0 (5-Feb-2023)
Resizing the filesystem on /dev/mapper/vgdata-lv_web to 1048576 (4k) blocks.
  Size of logical volume vgdata/lv_web changed from 8.00 GiB (2048 extents) to 4.00 GiB
(1024 extents).
  Logical volume vgdata/lv_web successfully resized.
$ sudo mount /srv/web
```

হাতে-হাতে উপায় (ধারণা পরিষ্কার করার জন্য)

```
$ sudo umount /srv/web
$ sudo e2fsck -f /dev/vgdata/lv_web          # আগে বাধ্যতামূলক চেক
```

```
$ sudo resize2fs /dev/vgdata/lv_web 4G       # ১. ফাইলসিস্টেম ছোট
$ sudo lvreduce -L 4G /dev/vgdata/lv_web     # ২. LV ছোট (একই বা বড় সাইজ!)
$ sudo mount /srv/web
```

`lvreduce` -r ছাড়া চালালে সতর্কবার্তা দেখাবে:

```
  WARNING: Reducing active logical volume to 4.00 GiB.
  THIS MAY DESTROY YOUR DATA (filesystem etc.)
Do you really want to reduce vgdata/lv_web? [y/n]:
```

> ⚠️ **সতর্কতা:** হাতে-হাতে করলে GB আর GiB, রাউন্ডিং — এসবের গোলমালে LV ফাইলসিস্টেমের চেয়ে সামান্য ছোট হয়ে যেতে পারে, আর তাতেই ফাইলসিস্টেম ভেঙে যায়। নিরাপদ কৌশল: ফাইলসিস্টেমকে লক্ষ্যের চেয়ে একটু বেশি ছোট করো (যেমন 3.5G), LV-কে লক্ষ্য সাইজে (4G) আনো, তারপর `resize2fs` সাইজ ছাড়া চালিয়ে ফাইলসিস্টেমকে LV-এর পুরো সাইজে বাড়িয়ে নাও। অথবা সোজা `-r` ব্যবহার করো।

> 💡 **টিপস:** নতুন LVM সংস্করণে (2.03.17+, যা Ubuntu 24.04, Fedora, RHEL 9-এর নতুন আপডেটে আছে) `lvreduce` ডিফল্টে ফাইলসিস্টেম ভাঙতে পারে এমন কাজে বাধা দেয়, যদি না `-r` বা `--fs` অপশন দাও। এটা একটা ভালো নিরাপত্তা জাল, তবে এর ওপর পুরো ভরসা কোরো না।

---

১৩. LVM Snapshot

Snapshot কী

Snapshot হলো একটা LV-এর কোনো নির্দিষ্ট মুহূর্তের "জমে থাকা ছবি" (point-in-time copy)। বানাতে এক সেকেন্ডও লাগে না, কারণ শুরুতে কোনো ডেটা কপি হয় না।

উপমা: ধরো তুমি একটা খাতায় লিখছ। Snapshot নেওয়া মানে খাতার পাশে একটা খালি নোটপ্যাড রাখা। যখনই খাতার কোনো পাতা বদলাতে যাবে, বদলানোর **আগে** সেই পাতার পুরনো লেখাটা নোটপ্যাডে টুকে রাখবে। তাহলে খাতা + নোটপ্যাড মিলিয়ে সবসময় পুরনো অবস্থাটা ফিরে পাওয়া যায়। এই কৌশলের নাম **Copy-on-Write (CoW)**।

```
Snapshot নেওয়ার মুহূর্তে:
  Origin LV:  [A][B][C][D]
  Snapshot:   [ ][ ][ ][ ]   (খালি, শুধু "Origin দেখো" নির্দেশ)

পরে Origin-এ C বদলে X লেখা হলো:
  ১. আগে পুরনো C কপি হলো snapshot-এ
  ২. তারপর Origin-এ X লেখা হলো
  Origin LV:  [A][B][X][D]     ← বর্তমান অবস্থা
  Snapshot:   [ ][ ][C][ ]
```

```
Snapshot পড়লে: A, B (origin থেকে), C (snapshot থেকে), D (origin থেকে)
               = [A][B][C][D]  ← পুরনো অবস্থা অক্ষত!
```

তাই snapshot-এর জন্য যে জায়গা দাও, সেটা শুধু **পরিবর্তনগুলো** জমা রাখার জন্য। পুরো LV-এর সমান জায়গা লাগে না।

Snapshot বানানো

```
$ sudo lvcreate -s -L 1G -n lv_web_snap /dev/vgdata/lv_web
  Logical volume "lv_web_snap" created.

$ sudo lvs
  LV          VG     Attr       LSize Pool Origin Data%  Meta%
  lv_web      vgdata owi-aos--- 4.00g
  lv_web_snap vgdata swi-a-s--- 1.00g      lv_web 0.01
```

`lv_web`-এর Attr শুরু হলো `o` (origin) দিয়ে, `lv_web_snap` শুরু হলো `s` (snapshot) দিয়ে। `Data%` বলছে snapshot-এর জায়গা কতটা ভরেছে।

> ⚠️ **সতর্কতা:** Snapshot-এর জায়গা (এখানে 1G) ১০০% ভরে গেলে snapshot **অকেজো (invalid)** হয়ে যায় এবং আর ব্যবহার করা যায় না। পুরনো ধরনের snapshot তাই স্বল্পমেয়াদি কাজের জন্য। `lvs` দিয়ে নিয়মিত `Data%` দেখো, দরকারে `lvextend` দিয়ে snapshot বাড়াও। `/etc/lvm/lvm.conf`-এ `snapshot_autoextend_threshold` সেট করে স্বয়ংক্রিয় বাড়ানোও যায় (এর জন্য `lvm2-monitor`/dmeventd চালু থাকতে হয়)।

> ⚠️ **সতর্কতা:** Snapshot থাকা অবস্থায় origin-এ প্রতিটা নতুন লেখার আগে একটা বাড়তি কপি হয়, তাই লেখার গতি কমে যায়। কাজ শেষে snapshot মুছে ফেলো।

Snapshot দিয়ে কী করা যায়

১. ধারাবাহিক (consistent) ব্যাকআপ: চালু ডাটাবেসের ফাইল কপি করার সময় ফাইল বদলাতে থাকে। Snapshot নিয়ে সেটা read-only মাউন্ট করে কপি করলে সব ফাইল একই মুহূর্তের হয়।

```
$ sudo mkdir -p /mnt/snap
$ sudo mount -o ro /dev/vgdata/lv_web_snap /mnt/snap
$ sudo tar -czf /backup/web-$(date +%F).tar.gz -C /mnt/snap .
$ sudo umount /mnt/snap
$ sudo lvremove -y /dev/vgdata/lv_web_snap
  Logical volume "lv_web_snap" successfully removed.
```

> 📝 **মনে রাখো:** XFS snapshot মাউন্ট করতে `-o ro,nouuid` লাগে, কারণ snapshot আর origin-এর

> UUID একই, আর XFS একই UUID দুবার মাউন্ট করতে দেয় না।

২. আপগ্রেডের আগে নিরাপত্তা: বড় আপগ্রেডের আগে snapshot নাও। কিছু ভাঙলে আগের অবস্থায় ফিরে যাও।

Snapshot থেকে ফিরে যাওয়া: `lvconvert --merge`

```
$ sudo umount /srv/web
$ sudo lvconvert --merge /dev/vgdata/lv_web_snap
  Merging of volume vgdata/lv_web_snap started.
  vgdata/lv_web: Merged: 100.00%
$ sudo mount /srv/web
```

Merge হলে origin snapshot-এর মুহূর্তের অবস্থায় ফিরে যায়, আর snapshot নিজে মুছে যায়। Origin ব্যবহারে থাকলে (যেমন রুট ফাইলসিস্টেম) merge পরের বার LV সক্রিয় হওয়ার সময় (সাধারণত রিবুটে) হয়।

Thin provisioning ও thin snapshot (এক নজরে)

আধুনিক পদ্ধতি হলো **thin pool**। একটা pool বানিয়ে তার ভেতরে thin volume বানাও, যেগুলো যতটুকু লেখা হয় ততটুকুই জায়গা নেয়।

```
$ sudo lvcreate --type thin-pool -L 10G -n tpool vgdata
$ sudo lvcreate -V 50G --thinpool vgdata/tpool -n lv_thin
$ sudo lvcreate -s -n lv_thin_snap vgdata/lv_thin   # thin snapshot, সাইজ লাগে না
```

এখানে ১০ GB pool-এ ৫০ GB-এর ভলিউম! একে **over-provisioning** বলে। Thin snapshot অনেকগুলো রাখা যায় এবং গতি কম কমে। কিন্তু pool সত্যিই ভরে গেলে সব thin volume-এ লেখা আটকে যায়, তাই `lvs`-এ pool-এর `Data%` আর `Meta%` দুটোই নজরে রাখতে হয়।

বিষয়	পুরনো (thick) snapshot	Thin snapshot
সাইজ দিতে হয়	হ্যাঁ (`-L`)	না
ভরে গেলে	snapshot অকেজো	pool ভরে, সব thin volume আটকায়
অনেকগুলো রাখা	ধীর হয়ে যায়	তুলনামূলক কার্যকর
ব্যবহার	স্বল্পমেয়াদি ব্যাকআপ	দীর্ঘমেয়াদি, VM/কন্টেইনার

> ⚠️ **সতর্কতা:** Snapshot একই ডিস্কে থাকে। ডিস্ক নষ্ট হলে origin আর snapshot দুটোই যায়। তাই snapshot **ব্যাকআপ নয়**, এটা ব্যাকআপ নেওয়ার একটা হাতিয়ার। আসল ব্যাকআপ নিয়ে অধ্যায় ৩৫ দেখো।

---

১৪. RAID কী এবং কেন

RAID (Redundant Array of Independent Disks) মানে একাধিক ডিস্ককে একসাথে জুড়ে এমনভাবে ব্যবহার করা, যাতে নিচের এক বা একাধিক সুবিধা পাওয়া যায়:

- **Redundancy (অতিরিক্ত নিরাপত্তা):** একটা ডিস্ক নষ্ট হলেও ডেটা থাকে, সার্ভার চলতে থাকে।
- **Performance (গতি):** একসাথে কয়েকটা ডিস্ক থেকে পড়া/লেখা, তাই দ্রুত।
- **Capacity (ধারণক্ষমতা):** কয়েকটা ডিস্ক মিলিয়ে একটা বড় ডিস্ক।

উপমা: একটা গুরুত্বপূর্ণ চিঠির কথা ভাবো।

- **RAID 0** = চিঠিটা টুকরো করে দুইজন পিয়নকে দিলে, অর্ধেক অর্ধেক। দ্বিগুণ দ্রুত পৌঁছায়, কিন্তু একজন হারিয়ে গেলে পুরো চিঠি অর্থহীন।
- **RAID 1** = চিঠির দুটো ফটোকপি দুইজনকে দিলে। একজন হারালেও অন্যজনেরটা আছে।
- **RAID 5** = তিনজনকে চিঠির টুকরো দিলে, সাথে একটা "হিসাবের চিরকুট" (parity)। যেকোনো একজন হারালে বাকিদের টুকরো আর চিরকুট মিলিয়ে হারানো অংশ আবার বানানো যায়।

মূল তিনটা কৌশল

কৌশল	মানে
Striping	ডেটা ছোট টুকরোয় (chunk, সাধারণত 512 KiB — mdadm-এর ডিফল্ট) ভাগ করে ডিস্কগুলোতে পালাক্রমে লেখা
Mirroring	একই ডেটা একাধিক ডিস্কে হুবহু লেখা
Parity	ডেটা থেকে একটা গাণিতিক হিসাব (XOR) রাখা, যা দিয়ে হারানো ডেটা আবার বানানো যায়

Parity কীভাবে কাজ করে (XOR জাদু)

XOR (exclusive OR) এর নিয়ম: দুটো বিট একই হলে 0, আলাদা হলে 1। এর বিশেষ গুণ হলো, যেকোনো একটা মান হারালে বাকিগুলো XOR করলে সেটা ফিরে আসে।

```
ডিস্ক১ (D1) = 1011
ডিস্ক২ (D2) = 0110
Parity  (P) = D1 XOR D2 = 1101

ডিস্ক২ নষ্ট! ফেরত আনো:
D2 = D1 XOR P = 1011 XOR 1101 = 0110   ✔ ঠিক ফিরে এল
```

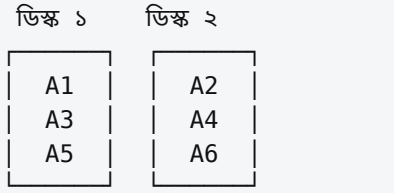
RAID 5 ঠিক এভাবেই একটা ডিস্কের ক্ষতি সামলায়। RAID 6 দুটো আলাদা parity (P ও Q, দ্বিতীয়টা ভিন্ন গণিতে — Reed-Solomon) রাখে, তাই দুটো ডিস্ক একসাথে নষ্ট হলেও টিকে থাকে।

---

১৫. RAID লেভেলগুলো — ডায়াগ্রাম সহ

নিচের ডায়াগ্রামে A1, A2, ... হলো ডেটার টুকরো (block/chunk), `P` ও `Q` হলো parity।

RAID 0 — Striping (গতি, নিরাপত্তা শূন্য)

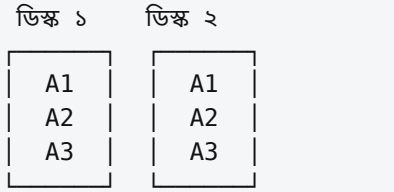


যেকোনো একটা নষ্ট → সব ডেটা শেষ

- কমপক্ষে ২টা ডিস্ক। ধারণক্ষমতা = সব ডিস্কের যোগফল।
- পড়া ও লেখা দুটোই দ্রুত।
- ব্যবহার: ভিডিও এডিটিং-এর অস্থায়ী (scratch) জায়গা, ক্যাশ — যেখানে ডেটা হারালে ক্ষতি নেই।

> ⚠️ **সতর্কতা:** RAID 0-তে নামেই "Redundant" কিন্তু কোনো redundancy নেই। ২টা ডিস্কে নষ্ট হওয়ার ঝুঁকি একটা ডিস্কের প্রায় দ্বিগুণ।

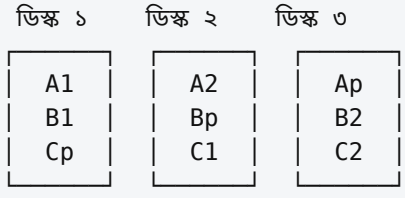
RAID 1 — Mirroring (নিরাপত্তা, সরল)



একটা নষ্ট → অন্যটা থেকে সব চলে

- কমপক্ষে ২টা ডিস্ক। ধারণক্ষমতা = একটা ডিস্কের সমান (৫০% কাজে লাগে)।
- পড়া দ্রুত হতে পারে (দুটো থেকে পড়া যায়), লেখা একটা ডিস্কের মতো।
- ব্যবহার: OS ডিস্ক, ছোট সার্ভার, boot ড্রাইভ।

RAID 5 — Striping + একক distributed parity



parity সব ডিস্কে ঘুরিয়ে রাখা হয় (distributed)
যেকোনো ১টা নষ্ট → টিকে থাকে

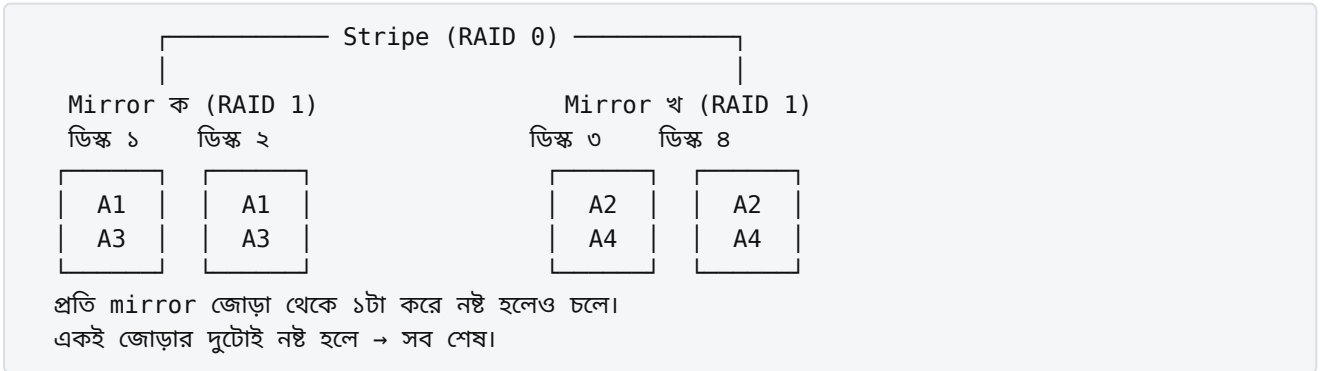
- কমপক্ষে ৩টা ডিস্ক। ধারণক্ষমতা = (N − 1) × ডিস্কের সাইজ।
- পড়া দ্রুত। ছোট লেখা ধীর, কারণ প্রতিটা লেখায় parity আবার হিসাব করতে হয় (read-modify-write, যাকে "write penalty" বলে)।
- Parity এক ডিস্কে না রেখে ঘুরিয়ে রাখা হয়, যাতে কোনো এক ডিস্কের ওপর চাপ না পড়ে (RAID 4-এ এক ডিস্কে রাখত, তাই সেটা bottleneck হতো)।

> ⚠️ **সতর্কতা:** বড় (কয়েক TB) HDD-তে RAID 5 এখন ঝুঁকিপূর্ণ ধরা হয়। একটা ডিস্ক নষ্ট হলে rebuild-এ অনেক ঘণ্টা থেকে দিন লাগে, এই সময় বাকি সব ডিস্ক পুরোটা পড়তে হয়। তখন আরেকটা ডিস্ক নষ্ট হলে বা একটা অপাঠযোগ্য সেক্টর (URE, Unrecoverable Read Error) পেলে ডেটা হারানোর ঝুঁকি থাকে। বড় অ্যারেতে RAID 6 বা RAID 10 পছন্দ করো।

RAID 6 — Striping + দ্বৈত parity

- কমপক্ষে ৪টা ডিস্ক। ধারণক্ষমতা = (N − 2) × ডিস্কের সাইজ।
- লেখা RAID 5-এর চেয়েও ধীর (দুটো parity হিসাব)।
- ব্যবহার: বড় ফাইল সার্ভার, আর্কাইভ, NAS।

RAID 10 (1+0) — Mirror-এর ওপর Stripe



- কমপক্ষে ৪টা ডিস্ক (জোড় সংখ্যা, প্রথাগত বিন্যাসে)। ধারণক্ষমতা = N / 2।
- পড়া-লেখা দুটোই দ্রুত, parity হিসাব নেই, rebuild দ্রুত (শুধু জোড়ার সঙ্গীর থেকে কপি)।
- ব্যবহার: ডাটাবেস সার্ভার, ভার্চুয়ালাইজেশন — যেখানে গতি আর নিরাপত্তা দুটোই চাই।

> 📝 **মনে রাখো:** Linux `mdadm`-এর RAID 10 একটা নিজস্ব ড্রাইভার, যা চাইলে ২ বা ৩টা ডিস্কেও চলে

> (`near`, `far`, `offset` বিন্যাস)। তবে প্রথাগত ধারণায় RAID 10 = ৪+ ডিস্ক।

তুলনা টেবিল

লেভেল	কৌশল	ন্যূনতম ডিস্ক	ব্যবহারযোগ্য জায়গা	কয়টা ডিস্ক নষ্ট সহ্য করে	পড়ার গতি	লেখার গতি	কোথায় ব্যবহার
RAID 0	Striping	২	N (১০০%)	০	খুব দ্রুত	খুব দ্রুত	অস্থায়ী/ক্যাশ ডেটা
RAID 1	Mirroring	২	1 ডিস্ক (৫০% যদি ২টা)	N − 1	দ্রুত	সাধারণ	OS, ছোট সার্ভার
RAID 5	Stripe + ১ parity	৩	N − 1	১	দ্রুত	ধীর (write penalty)	ছোট/মাঝারি ফাইল সার্ভার
RAID 6	Stripe + ২ parity	৪	N − 2	২	দ্রুত	আরো ধীর	বড় স্টোরেজ, আর্কাইভ
RAID 10	Mirror + Stripe	৪	N / 2	প্রতি জোড়ায় ১টা	খুব দ্রুত	দ্রুত	ডাটাবেস, VM

উদাহরণ হিসাব: ৪টা ৪ TB ডিস্ক থাকলে:

লেভেল	ব্যবহারযোগ্য জায়গা
RAID 0	১৬ TB
RAID 1 (৪টাই mirror)	৪ TB
RAID 5	১২ TB
RAID 6	৮ TB
RAID 10	৮ TB

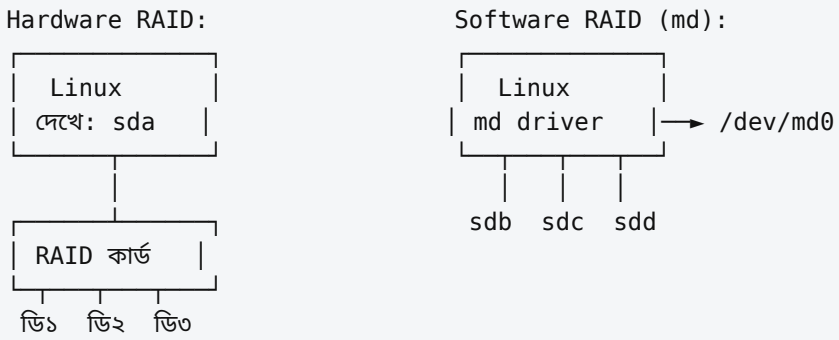
> 💡 **টিপস:** RAID-এ সব ডিস্ক একই সাইজের রাখো। ভিন্ন সাইজ হলে সবচেয়ে ছোট ডিস্কের সাইজ ধরে হিসাব হয়, বড় ডিস্কের বাড়তি জায়গা নষ্ট হয়।

---

১৬. Hardware RAID, Software RAID ও Fake RAID

ধরন	কে কাজটা করে	সুবিধা	অসুবিধা
Hardware RAID	আলাদা RAID কন্ট্রোলার কার্ড (নিজস্ব প্রসেসর, ক্যাশ, প্রায়ই ব্যাটারি)	CPU-তে চাপ নেই, OS শুধু একটা ডিস্ক দেখে, battery-backed cache-এ দ্রুত লেখা	দামি; কার্ড নষ্ট হলে প্রায়ই একই মডেলের কার্ড লাগে

ধরন	কে কাজটা করে	সুবিধা	অসুবিধা
Software RAID	Linux kernel (md driver), `mdadm` দিয়ে নিয়ন্ত্রণ	ফ্রি, যেকোনো Linux মেশিনে ডিস্ক সরিয়ে চালানো যায়, স্বচ্ছ	CPU কিছুটা ব্যবহার করে (আধুনিক CPU-তে নগণ্য)
Fake RAID (BIOS/firmware RAID)	মাদারবোর্ডের firmware + OS ড্রাইভার	Windows-এর সাথে dual boot-এ সুবিধা	আসলে software RAID-ই, কিন্তু বেশি জটিল; Linux-এ সাধারণত এড়িয়ে চলা ভালো



এই অধ্যায়ে আমরা Linux-এর software RAID শিখব, কারণ এটা সব জায়গায় একইভাবে কাজ করে।

> 📝 **মনে রাখো:** Hardware RAID থাকলে ডিস্কের স্বাস্থ্য দেখতে ভেন্ডরের টুল লাগে (যেমন `storcli`, `ssacli`, `perccli`)। `smartctl` সরাসরি নাও দেখতে পারে, বিশেষ অপশন লাগে (`-d megaraid,N` ইত্যাদি)।

---

১৭. `mdadm` দিয়ে Software RAID তৈরি

`md` ড্রাইভার ও `mdadm`

Linux kernel-এর **md (multiple devices)** ড্রাইভার RAID-এর আসল কাজ করে। `mdadm` (multiple devices admin) হলো user-space টুল যা দিয়ে আমরা RAID বানাই, দেখি, মেরামত করি। প্রতিটা সদস্য ডিস্কে একটা **superblock** (metadata, বর্তমান ডিফল্ট সংস্করণ 1.2) লেখা হয়, যেখানে লেখা থাকে এটা কোন অ্যারের অংশ, কোন অবস্থানে।

```
# Ubuntu / Debian
$ sudo apt install mdadm

# Fedora / RHEL
$ sudo dnf install mdadm
```

`mdadm` এর মোড ও প্রধান অপশন

```
mdadm [মোড] <raid-ডিভাইস> [অপশন] <সদস্য-ডিভাইস...>
```

অপশন	কাজ
`--create` / `-C`	নতুন অ্যারে বানানো
`--level=N` / `-l N`	RAID লেভেল: `0`, `1`, `5`, `6`, `10`
`--raid-devices=N` / `-n N`	সক্রিয় সদস্য ডিস্কের সংখ্যা
`--spare-devices=N` / `-x N`	কয়টা spare (অতিরিক্ত, অপেক্ষমাণ) ডিস্ক
`--chunk=SIZE` / `-c`	chunk সাইজ (KiB), যেমন `--chunk=256`
`--detail` / `-D`	অ্যারের বিস্তারিত তথ্য
`--examine` / `-E`	একটা সদস্য ডিস্কের superblock পড়া
`--assemble` / `-A`	আগের অ্যারে আবার জোড়া লাগানো
`--scan` / `-s`	কনফিগ/ডিস্ক স্ক্যান
`--fail` / `-f`	ডিস্ককে নষ্ট হিসেবে চিহ্নিত করা
`--remove` / `-r`	অ্যারে থেকে ডিস্ক বের করা
`--add` / `-a`	ডিস্ক যোগ করা
`--grow` / `-G`	অ্যারের আকার/লেভেল/সদস্য সংখ্যা বদলানো
`--stop` / `-S`	অ্যারে বন্ধ করা
`--zero-superblock`	ডিস্ক থেকে md superblock মুছে ফেলা
`--monitor` / `-F`	অ্যারে নজরে রেখে সমস্যা হলে মেইল/অ্যালার্ট দেওয়া

RAID 1 বানানো

```
$ sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
mdadm: Note: this array has metadata at the start and
    may not be suitable as a boot device.  If you plan to
    store '/boot' on this device please ensure that
    your boot-loader understands md/v1.x metadata, or use
    --metadata=0.90
Continue creating array? y
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

RAID 5 বানানো (৩ ডিস্ক + ১ spare)

```
$ sudo mdadm --create /dev/md1 --level=5 --raid-devices=3 \
       --spare-devices=1 /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Spare ডিস্ক চুপচাপ বসে থাকে। কোনো সক্রিয় ডিস্ক নষ্ট হলে md নিজে থেকে spare দিয়ে rebuild শুরু করে, মানুষের হাত লাগে না।

অন্য লেভেল

```
# RAID 0
$ sudo mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/sdb /dev/sdc

# RAID 6
$ sudo mdadm --create /dev/md0 --level=6 --raid-devices=4 /dev/sd[b-e]

# RAID 10
$ sudo mdadm --create /dev/md0 --level=10 --raid-devices=4 /dev/sd[b-e]
```

`/dev/sd[b-e]` হলো shell globbing, মানে `sdb sdc sdd sde` (অধ্যায় ৯)।

ফাইলসিস্টেম, মাউন্ট, fstab

RAID ডিভাইস তৈরির পরপরই ব্যবহার শুরু করা যায়, পটভূমিতে sync চলতে থাকলেও:

```
$ sudo mkfs.ext4 /dev/md0
$ sudo mkdir -p /srv/raid
$ sudo mount /dev/md0 /srv/raid

$ sudo blkid /dev/md0
/dev/md0: UUID="7c1f0a2e-5b3d-4e8a-9f21-0c6d8e4b1a77" BLOCK_SIZE="4096" TYPE="ext4"
```

fstab-এ UUID ব্যবহার করো (কারণ বুটে `/dev/md0` কখনো `/dev/md127` হয়ে যেতে পারে):

```
UUID=7c1f0a2e-5b3d-4e8a-9f21-0c6d8e4b1a77  /srv/raid  ext4  defaults,nofail  0 2
```

`nofail` দিলে অ্যারে কোনো কারণে না উঠলেও সিস্টেম বুট আটকে থাকবে না।

কনফিগারেশন সেভ করা (খুব জরুরি)

অ্যারের তথ্য `mdadm.conf`-এ লিখে রাখো এবং initramfs আপডেট করো, যাতে বুটের সময় অ্যারে ঠিক নামে জোড়া লাগে:

```
# Ubuntu / Debian — ফাইল: /etc/mdadm/mdadm.conf
$ sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
ARRAY /dev/md0 metadata=1.2 UUID=3a5c7e9f:1b2d4f6a:8c0e2a4b:6d8f0a2c
$ sudo update-initramfs -u

# Fedora / RHEL — ফাইল: /etc/mdadm.conf
$ sudo mdadm --detail --scan | sudo tee -a /etc/mdadm.conf
$ sudo dracut -f
```

> ⚠️ **সতর্কতা:** এই ধাপ বাদ দিলে রিবুটের পর অ্যারে প্রায়ই `/dev/md127` নামে উঠবে, আর fstab-এ `/dev/md0` লেখা থাকলে মাউন্ট ব্যর্থ হবে। `tee -a` দুবার চালালে ডুপ্লিকেট ARRAY লাইন হবে — ফাইলটা

> খুলে একবার দেখে নাও।

---

১৮. RAID-এর অবস্থা দেখা: `/proc/mdstat` ও `mdadm --detail`

`/proc/mdstat`

`/proc/mdstat` হলো kernel-এর তৈরি একটা ভার্চুয়াল ফাইল (অধ্যায় ৪৭), যেটা সব md অ্যারের বর্তমান অবস্থা দেখায়। RAID অ্যাডমিনের সবচেয়ে প্রিয় ফাইল।

```
$ cat /proc/mdstat
Personalities : [raid1] [raid6] [raid5] [raid4]
md0 : active raid1 sdc[1] sdb[0]
      10475520 blocks super 1.2 [2/2] [UU]
      [=======>.............]  resync = 38.4% (4023296/10475520) finish=0.5min
speed=201164K/sec

unused devices: <none>
```

লাইন ধরে পড়া:

অংশ	মানে
`Personalities`	kernel-এ কোন RAID লেভেলের ড্রাইভার লোড আছে
`md0 : active raid1`	অ্যারের নাম, চালু আছে, লেভেল raid1
`sdc[1] sdb[0]`	সদস্য ডিস্ক ও তাদের অবস্থান নম্বর
`10475520 blocks`	সাইজ (1 KiB ব্লকে)
`super 1.2`	superblock সংস্করণ
`[2/2]`	দরকার ২টা / চালু আছে ২টা
`[UU]`	প্রতিটা ডিস্কের অবস্থা: `U` = up (ঠিক আছে), `_` = নেই/নষ্ট
`resync` / `recovery`	sync বা rebuild চলছে, কত শতাংশ, আনুমানিক কত সময় বাকি

নষ্ট অবস্থার উদাহরণ:

```
md0 : active raid1 sdb[0] sdc[1](F)
      10475520 blocks super 1.2 [2/1] [U_]
```

`(F)` = failed, `[2/1]` = দুটোর জায়গায় একটা চলছে, `[U_]` = দ্বিতীয় ডিস্ক নেই। এই অবস্থাকে বলে **degraded**। অ্যারে চলছে, কিন্তু আর কোনো নিরাপত্তা নেই। এখনই ডিস্ক বদলাও!

অন্যান্য চিহ্ন: `(S)` = spare।

লাইভ নজর রাখতে:

```
$ watch -n 2 cat /proc/mdstat
```

(বের হতে `Ctrl+C`)

`mdadm --detail`

```
$ sudo mdadm --detail /dev/md0
/dev/md0:
           Version : 1.2
     Creation Time : Thu Nov 12 10:42:17 2026
        Raid Level : raid1
        Array Size : 10475520 (9.99 GiB 10.73 GB)
     Used Dev Size : 10475520 (9.99 GiB 10.73 GB)
      Raid Devices : 2
     Total Devices : 2
       Persistence : Superblock is persistent

             State : clean
    Active Devices : 2
   Working Devices : 2
    Failed Devices : 0
     Spare Devices : 0

              Name : server1:0  (local to host server1)
              UUID : 3a5c7e9f:1b2d4f6a:8c0e2a4b:6d8f0a2c
            Events : 17

    Number   Major   Minor   RaidDevice State
       0       8       16        0      active sync   /dev/sdb
       1       8       32        1      active sync   /dev/sdc
```

`State` লাইন দেখো: `clean` (সব ঠিক), `active`, `degraded` (সমস্যা), `resyncing`, `recovering`।

একটা ডিস্কের superblock দেখা

```
$ sudo mdadm --examine /dev/sdb
```

কোনো অপরিচিত ডিস্ক আগে RAID-এর অংশ ছিল কিনা জানতে এটা কাজে লাগে।

স্বয়ংক্রিয় নজরদারি

ডিস্ক নষ্ট হলে তুমি জানবে কী করে? `mdadm.conf`-এ একটা লাইন যোগ করো:

```
MAILADDR admin@example.com
```

Debian/Ubuntu ও RHEL-এ mdadm-এর monitor সার্ভিস (systemd unit-এর নাম distro-ভেদে `mdmonitor.service` বা `mdadm`-এর সাথে আসা unit) এটা পড়ে মেইল পাঠায় (মেইল পাঠানোর জন্য সিস্টেমে MTA/`sendmail` সেটআপ থাকতে হবে)। পরীক্ষামূলক অ্যালার্ট পাঠাতে:

```
$ sudo mdadm --monitor --scan --test --oneshot
```

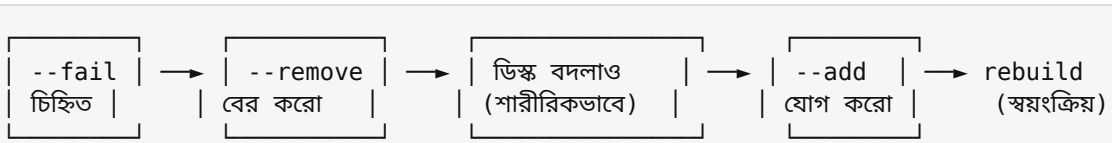
এছাড়া অনেক distro নিয়মিত (মাসে একবার ইত্যাদি) "scrub"/check চালায়, যা পুরো অ্যারে পড়ে লুকানো খারাপ সেক্টর খোঁজে। হাতে চালাতে:

```
$ echo check | sudo tee /sys/block/md0/md/sync_action
$ cat /sys/block/md0/md/mismatch_cnt
0
```

---

১৯. নষ্ট ডিস্ক বদলানো

এটাই RAID অ্যাডমিনের আসল পরীক্ষা। প্রক্রিয়াটা চার ধাপের:



ধাপ ১: কোন ডিস্ক নষ্ট খুঁজে বের করো

```
$ cat /proc/mdstat
md0 : active raid1 sdb[0] sdc[1](F)
      10475520 blocks super 1.2 [2/1] [U_]
```

`sdc` নষ্ট। কিন্তু সার্ভারের সামনে গিয়ে কোন ফিজিক্যাল ডিস্কটা `sdc`? সিরিয়াল নম্বর দেখো:

```
$ lsblk -o NAME,SIZE,SERIAL,MODEL
$ sudo smartctl -i /dev/sdc | grep -i serial
Serial Number:    WD-WX12A3456789
```

ডিস্কের গায়ের লেবেলে এই সিরিয়াল মিলিয়ে বের করবে।

> ⚠️ **সতর্কতা:** ভুল (ভালো) ডিস্ক খুলে ফেললে degraded RAID 1/5 পুরোপুরি বসে যাবে। সিরিয়াল না মিলিয়ে কখনো ডিস্ক খুলবে না।

ধাপ ২: fail ও remove

kernel নিজেই যদি `(F)` চিহ্ন দিয়ে থাকে তাহলে fail চিহ্নিত করা লাগবে না। নিজে বদলাতে চাইলে (যেমন SMART সতর্কতা দিচ্ছে কিন্তু এখনো নষ্ট হয়নি):

```
$ sudo mdadm /dev/md0 --fail /dev/sdc
mdadm: set /dev/sdc faulty in /dev/md0
```

```
$ sudo mdadm /dev/md0 --remove /dev/sdc
mdadm: hot removed /dev/sdc from /dev/md0
```

ধাপ ৩: শারীরিকভাবে ডিস্ক বদলাও

Hot-swap সাপোর্ট থাকলে চালু অবস্থায়, না থাকলে সার্ভার বন্ধ করে। নতুন ডিস্ক পুরনোটার সমান বা বড় হতে হবে।

যদি সদস্যগুলো পার্টিশন হয় (`/dev/sdb1` ইত্যাদি), নতুন ডিস্কে একই পার্টিশন টেবিল কপি করো:

```
# GPT ডিস্কের জন্য (gdisk প্যাকেজের sgdisk):
$ sudo sgdisk -R /dev/sdc /dev/sdb    # sdb থেকে sdc-তে টেবিল কপি (দিক খেয়াল করো!)
$ sudo sgdisk -G /dev/sdc             # নতুন ডিস্কে নতুন GUID

# MBR ডিস্কের জন্য:
$ sudo sfdisk -d /dev/sdb | sudo sfdisk /dev/sdc
```

ধাপ ৪: add — rebuild স্বয়ংক্রিয়

```
$ sudo mdadm /dev/md0 --add /dev/sdc
mdadm: added /dev/sdc

$ cat /proc/mdstat
md0 : active raid1 sdc[2] sdb[0]
      10475520 blocks super 1.2 [2/1] [U_]
      [===>.................]  recovery = 17.2% (1802240/10475520) finish=0.7min
speed=200249K/sec
```

`recovery` শেষ হয়ে `[UU]` দেখালে কাজ শেষ। এই সময়ে অ্যারে চলতে থাকে, তবে একটু ধীর।

> 💡 **টিপস:** Rebuild-এর গতি `/proc/sys/dev/raid/speed_limit_min` আর `speed_limit_max` (KiB/s) দিয়ে নিয়ন্ত্রিত। ব্যস্ত সার্ভারে ধীর করো, রাতে দ্রুত।

> 💡 **টিপস:** রুট বা `/boot` RAID 1-এ থাকলে নতুন ডিস্কেও bootloader বসাতে ভুলো না (BIOS-এ `grub-install /dev/sdc`, UEFI-তে EFI পার্টিশন আলাদাভাবে কপি/সেটআপ)। নইলে পরে প্রথম ডিস্ক নষ্ট হলে সার্ভার বুট হবে না।

অ্যারে বড় করা: `--grow`

RAID 5-এ চতুর্থ ডিস্ক যোগ করে জায়গা বাড়ানো:

```
$ sudo mdadm /dev/md1 --add /dev/sdf
$ sudo mdadm --grow /dev/md1 --raid-devices=4
```

```
$ cat /proc/mdstat          # "reshape" চলবে, অনেক সময় লাগতে পারে
$ sudo resize2fs /dev/md1   # reshape শেষে ফাইলসিস্টেম বাড়াও (ext4)
```

> ⚠️ **সতর্কতা:** Reshape হলো সবচেয়ে ঝুঁকিপূর্ণ RAID অপারেশনগুলোর একটা। শুরুর আগে ব্যাকআপ নাও।

অ্যারে পুরোপুরি মুছে ফেলা

```
$ sudo umount /srv/raid
$ sudo mdadm --stop /dev/md0
mdadm: stopped /dev/md0
$ sudo mdadm --zero-superblock /dev/sdb /dev/sdc
```

তারপর `mdadm.conf` থেকে ARRAY লাইন আর `/etc/fstab` থেকে এন্ট্রি মুছে initramfs আপডেট করো। `--zero-superblock` না করলে পরে ডিস্কগুলো অন্য কাজে লাগালে md পুরনো অ্যারে চিনে জোড়া লাগাতে চাইবে।

---

২০. RAID কেন ব্যাকআপ নয়

এই অংশটা এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ বাক্য:

> ⚠️ **সতর্কতা: RAID ব্যাকআপ নয়।** RAID রক্ষা করে শুধু **হার্ডওয়্যার (ডিস্ক) নষ্ট হওয়া থেকে**, আর সার্ভারকে চালু রাখে (availability)। ব্যাকআপ রক্ষা করে **ডেটা হারানো থেকে**, যেকোনো কারণেই হোক।

RAID 1-এ তুমি একটা ফাইল মুছলে, সেটা **দুটো ডিস্ক থেকেই সঙ্গে সঙ্গে** মুছে যায়। RAID ঠিক যা করার কথা তাই করছে: দুটো ডিস্ক হুবহু এক রাখছে, ভুলটাসহ।

বিপদ	RAID বাঁচায়?	ব্যাকআপ বাঁচায়?
একটা ডিস্ক নষ্ট	✅ হ্যাঁ	✅ হ্যাঁ (তবে রিস্টোরে সময় লাগে)
ভুল করে `rm -rf`	❌ না, সব ডিস্কে মুছে যায়	✅ হ্যাঁ
Ransomware/ভাইরাস ফাইল এনক্রিপ্ট করল	❌ না, এনক্রিপ্টেড ফাইল mirror হয়	✅ হ্যাঁ (অফলাইন/অপরিবর্তনীয় কপি থাকলে)
ফাইলসিস্টেম corruption, সফটওয়্যার বাগ	❌ না, corrupt ডেটাও কপি হয়	✅ হ্যাঁ
RAID কন্ট্রোলার বা PSU পুড়ে একাধিক ডিস্ক নষ্ট	❌ সম্ভবত না	✅ হ্যাঁ
আগুন, বন্যা, চুরি	❌ না, সব ডিস্ক এক জায়গায়	✅ হ্যাঁ (অন্য জায়গায় কপি থাকলে)
Rebuild-এর সময় আরেকটা ডিস্ক নষ্ট	❌ (RAID 5-এ)	✅ হ্যাঁ

উপমা: RAID হলো গাড়ির স্পেয়ার টায়ার। টায়ার পাংচার হলে চলা থামে না। কিন্তু গাড়িটাই দুর্ঘটনায় পড়লে বা চুরি হলে স্পেয়ার টায়ার কোনো কাজে আসে না। তার জন্য লাগে ইনশিওরেন্স — সেটাই ব্যাকআপ।

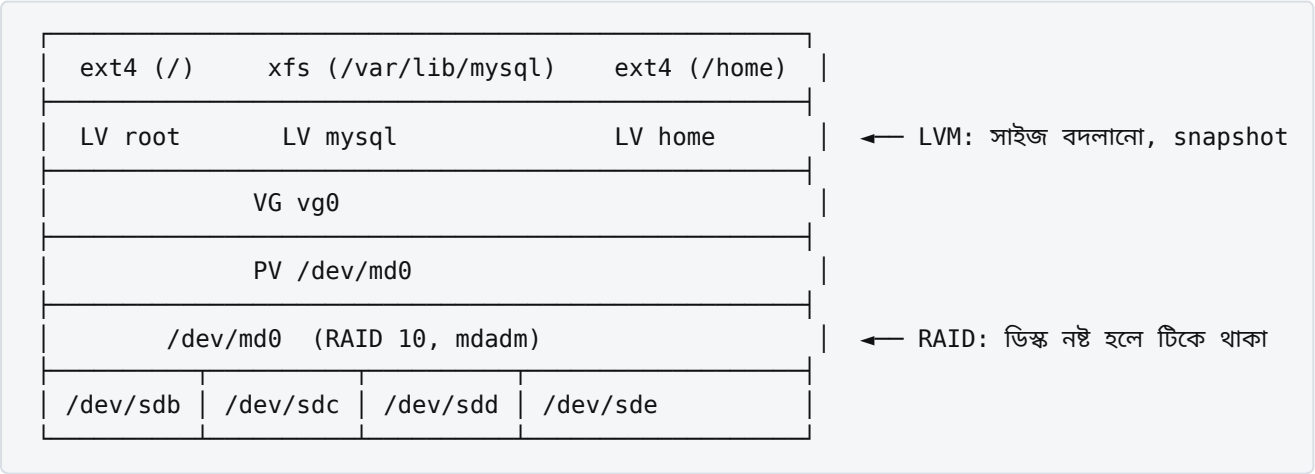
> 📝 **মনে রাখো — 3-2-1 নিয়ম:** ডেটার **৩টা** কপি, **২টা** ভিন্ন মাধ্যমে, **১টা** অন্য জায়গায় (offsite)। বিস্তারিত অধ্যায় ৩৫-এ।

একইভাবে, এক ব্যাচে কেনা একই মডেলের ডিস্কগুলো প্রায়ই কাছাকাছি সময়ে নষ্ট হয়। তাই RAID থাকলেও SMART নজরদারি (অধ্যায় ৪৬) আর নিয়মিত ব্যাকআপ — দুটোই লাগবে।

---

২১. LVM আর RAID একসাথে

বাস্তবে সার্ভারে সাধারণ বিন্যাস হলো: নিচে RAID (নিরাপত্তা), ওপরে LVM (নমনীয়তা)।




```
$ sudo mdadm --create /dev/md0 --level=10 --raid-devices=4 /dev/sd[b-e]
$ sudo pvcreate /dev/md0
$ sudo vgcreate vg0 /dev/md0
$ sudo lvcreate -L 30G -n mysql vg0
```

LVM-এর নিজস্ব RAID

LVM নিজেও md-এর একই kernel কোড ব্যবহার করে RAID বানাতে পারে, প্রতি-LV ভিত্তিতে:

```
$ sudo lvcreate --type raid1 -m 1 -L 5G -n lv_safe vgdata
$ sudo lvs -a -o name,copy_percent,devices vgdata
  LV                 Cpy%Sync Devices
  lv_safe            100.00   lv_safe_rimage_0(0),lv_safe_rimage_1(0)
  [lv_safe_rimage_0]          /dev/sdb(0)
  [lv_safe_rimage_1]          /dev/sdc(0)
  ...
```

`-m 1` মানে মূলের সাথে ১টা অতিরিক্ত কপি (মোট ২টা)। `--type raid5 -i 2` মানে ২টা ডেটা stripe + parity (মোট ৩টা PV লাগবে)। সুবিধা হলো এক VG-তে কোনো LV RAID, কোনোটা সাধারণ রাখা যায়। তবে বেশি প্রচলিত ও ডকুমেন্টেড পথ হলো mdadm + LVM।

> 📝 **মনে রাখো:** ZFS আর Btrfs হলো এমন ফাইলসিস্টেম যাদের ভেতরেই RAID ও volume management আছে, সাথে checksum দিয়ে নীরব ডেটা নষ্ট হওয়া (bit rot) ধরার ক্ষমতা। এগুলো আলাদা জগৎ; এখানে শুধু জেনে রাখো যে বিকল্প আছে।

---

২২. সমস্যা সমাধান (Troubleshooting) এক নজরে

লক্ষণ	সম্ভাব্য কারণ	কী করবে
`lvextend` করার পর `df` আগের সাইজ দেখাচ্ছে	ফাইলসিস্টেম বাড়ানো হয়নি	`resize2fs` (ext4) বা `xfs_growfs <mountpoint>` (XFS)
`Insufficient free space`	VG-তে খালি জায়গা নেই	`vgs` দেখো, নতুন PV এনে `vgextend`
রিবুটে অ্যারে `/dev/md127`	`mdadm.conf` নেই/পুরনো	`mdadm --detail --scan` দিয়ে conf আপডেট, initramfs আপডেট
বুটে emergency mode	fstab-এ ভুল ডিভাইস	রেসকিউ মোডে fstab ঠিক করো, `nofail` যোগ করো
`/proc/mdstat`-এ `[U_]`	ডিস্ক নষ্ট/বের হয়ে গেছে	`dmesg`, `smartctl` দেখো, ডিস্ক বদলাও
অ্যারে `inactive`	যথেষ্ট সদস্য পাওয়া যায়নি	`mdadm --examine` দিয়ে সদস্য দেখো, `mdadm --assemble --scan`
Snapshot `Invalid`	snapshot-এর জায়গা ভরে গেছে	ওই snapshot মুছে বড় সাইজে নতুন বানাও
`Device ... excluded by a filter`	`/etc/lvm/lvm.conf`-এর filter বা ডিস্কে পুরনো সিগনেচার	`wipefs` দিয়ে সিগনেচার দেখো, filter দেখো

---

২৩. সংশ্লিষ্ট কীবোর্ড শর্টকাট ও কৌশল

LVM/RAID নিজস্ব কোনো কীবোর্ড শর্টকাট নেই, কিন্তু এই কাজগুলোতে যেগুলো সবচেয়ে কাজে লাগে (বিস্তারিত অধ্যায় ৬):

শর্টকাট / কৌশল	কাজ
`Tab` / `Tab Tab`	`lv<Tab><Tab>` দিলে সব `lv*` কমান্ড দেখায়; bash-completion থাকলে VG/LV নামও পূরণ করে
`Ctrl+C`	`watch cat /proc/mdstat` বা `mdadm --monitor` থেকে বের হওয়া
`Ctrl+R`	আগের লম্বা `mdadm --create ...` কমান্ড খুঁজে বের করা (সাবধানে আবার চালাও!)

শর্টকাট / কৌশল	কাজ
`Alt+.`	আগের কমান্ডের শেষ argument বসানো (যেমন `/dev/vgdata/lv_web`)
`sudo lvm`	interactive LVM shell; ভেতরে `lvs`, `vgs` ইত্যাদি সরাসরি চালাও, `exit` দিয়ে বের হও
`Ctrl+B d` (tmux)	লম্বা `pvmove`/reshape চালু রেখে tmux থেকে detach (অধ্যায় ২৫)

হাতে-কলমে অনুশীলন

আসল ডিস্ক নষ্ট না করে শেখার জন্য আমরা **loop device** ব্যবহার করব। Loop device হলো একটা সাধারণ ফাইলকে ব্লক ডিভাইসের মতো দেখানো। ভার্চুয়াল মেশিনে করাই সবচেয়ে নিরাপদ।

> ⚠️ **সতর্কতা:** নিচে `losetup`-এর দেখানো আসল নাম (যেমন `/dev/loop10`) ব্যবহার করবে। Ubuntu-তে snap প্যাকেজগুলোও loop device ব্যবহার করে, তাই নম্বর আলাদা হবে। আমরা নামগুলো ভেরিয়েবলে রেখে নিচ্ছি যাতে ভুল না হয়।

ল্যাব ০: ভুয়া ডিস্ক বানানো

```
$ mkdir -p ~/lab && cd ~/lab
$ for i in 1 2 3 4; do truncate -s 1G disk$i.img; done
$ D1=$(sudo losetup -f --show disk1.img)
$ D2=$(sudo losetup -f --show disk2.img)
$ D3=$(sudo losetup -f --show disk3.img)
$ D4=$(sudo losetup -f --show disk4.img)
$ echo $D1 $D2 $D3 $D4
/dev/loop10 /dev/loop11 /dev/loop12 /dev/loop13
$ lsblk $D1 $D2 $D3 $D4
```

ল্যাব ১: LVM তৈরি

1. দুটো PV বানাও: `sudo pvcreate $D1 $D2`
2. VG বানাও: `sudo vgcreate vglab $D1 $D2`
3. `sudo vgs` দিয়ে দেখো `VSize` প্রায় 2G কিনা।
4. 500M-এর একটা LV: `sudo lvcreate -L 500M -n lvtest vglab`
5. ফাইলসিস্টেম ও মাউন্ট:

   ```
   $ sudo mkfs.ext4 /dev/vglab/lvtest
   $ sudo mkdir -p /mnt/lvtest
   $ sudo mount /dev/vglab/lvtest /mnt/lvtest
   $ echo "হ্যালো LVM" | sudo tee /mnt/lvtest/note.txt
   ```

6. `sudo lvs -o +devices` চালিয়ে দেখো LV কোন loop device-এ আছে।

ল্যাব ২: অনলাইন বড় করা

1. `df -h /mnt/lvtest` নোট করো।
2. `sudo lvextend -r -L +700M /dev/vglab/lvtest`
3. আবার `df -h /mnt/lvtest` — মাউন্ট খোলা না রেখেই কি বড় হলো? `note.txt` আছে তো?
4. চ্যালেঞ্জ: `-r` ছাড়া আরো 100M বাড়াও, `df` দেখো (বাড়েনি!), তারপর `resize2fs` চালিয়ে ঠিক করো।

ল্যাব ৩: নতুন ডিস্ক যোগ ও pvmove

1. `sudo pvcreate $D3 && sudo vgextend vglab $D3`
2. `sudo pvs` — নতুন PV-এর `PFree` দেখো।
3. `$D1` খালি করো: `sudo pvmove $D1`
4. `sudo pvs` দেখো — `$D1`-এর `PFree` এখন পুরো সাইজ?
5. বাদ দাও: `sudo vgreduce vglab $D1 && sudo pvremove $D1`
6. `cat /mnt/lvtest/note.txt` — ডেটা অক্ষত আছে, অথচ একটা "ডিস্ক" সরিয়ে ফেললে!

ল্যাব ৪: Snapshot ও merge

1. `sudo lvcreate -s -L 200M -n snap1 /dev/vglab/lvtest`
2. "দুর্ঘটনা" ঘটাও: `sudo rm /mnt/lvtest/note.txt`
3. `sudo lvs` — snapshot-এর `Data%` একটু বেড়েছে কি?
4. snapshot মাউন্ট করে দেখো ফাইল আছে কিনা:

```
$ sudo mkdir -p /mnt/snap
$ sudo mount -o ro /dev/vglab/snap1 /mnt/snap
$ cat /mnt/snap/note.txt
হ্যালো LVM
$ sudo umount /mnt/snap
```

5. পুরোটা ফিরিয়ে আনো:

```
$ sudo umount /mnt/lvtest
$ sudo lvconvert --merge /dev/vglab/snap1
$ sudo mount /dev/vglab/lvtest /mnt/lvtest
$ cat /mnt/lvtest/note.txt
```

ল্যাব ৫: lvreduce (নিরাপদ পথে)

1. `sudo umount /mnt/lvtest`
2. `sudo lvreduce -r -L 600M /dev/vglab/lvtest`
3. আবার মাউন্ট করে ফাইল দেখো।
4. ভেবে দেখো: XFS হলে এই ল্যাব কেন ব্যর্থ হতো?

ল্যাব ৬: LVM পরিষ্কার

```
$ sudo umount /mnt/lvtest
$ sudo lvremove -y vglab/lvtest
$ sudo vgremove vglab
$ sudo pvremove $D2 $D3
```

ল্যাব ৭: RAID 1 বানানো, নষ্ট করা, সারানো

1. বানাও:

   ```
   $ sudo mdadm --create /dev/md100 --level=1 --raid-devices=2 $D1 $D2
   ```

   (boot ডিভাইস সংক্রান্ত প্রশ্নে `y`)
2. `cat /proc/mdstat` — resync দেখো, `[UU]` হওয়া পর্যন্ত অপেক্ষা করো (`watch -n1 cat /proc/mdstat`, বের হতে `Ctrl+C`)।
3. ফাইলসিস্টেম ও ডেটা:

   ```
   $ sudo mkfs.ext4 /dev/md100
   $ sudo mkdir -p /mnt/raid && sudo mount /dev/md100 /mnt/raid
   $ echo "RAID টেস্ট" | sudo tee /mnt/raid/file.txt
   ```

4. ডিস্ক "নষ্ট" করো:

   ```
   $ sudo mdadm /dev/md100 --fail $D2
   $ cat /proc/mdstat        # [U_] আর (F) দেখো
   $ cat /mnt/raid/file.txt  # ডেটা এখনো আছে!
   ```

5. বদলাও:

   ```
   $ sudo mdadm /dev/md100 --remove $D2
   $ sudo mdadm /dev/md100 --add $D3
   $ watch -n1 cat /proc/mdstat   # recovery দেখো
   ```

6. `sudo mdadm --detail /dev/md100` — `State : clean`, সদস্য `$D1` আর `$D3`?

ল্যাব ৮: "RAID ব্যাকআপ নয়" নিজের চোখে

```
$ sudo rm /mnt/raid/file.txt
$ ls /mnt/raid
lost+found
```

দুটো ডিস্ক থাকা সত্ত্বেও ফাইল নেই। কোনো "অন্য কপি" থেকে ফেরত পাওয়ার উপায় নেই। মনে গেঁথে নাও।

ল্যাব ৯: সব পরিষ্কার

```
$ sudo umount /mnt/raid
$ sudo mdadm --stop /dev/md100
$ sudo mdadm --zero-superblock $D1 $D2 $D3
$ sudo losetup -d $D1 $D2 $D3 $D4
$ rm ~/lab/disk*.img
```

বোনাস চ্যালেঞ্জ

- চারটা loop device দিয়ে RAID 5 বানাও (৩টা সক্রিয় + ১টা spare)। একটা সক্রিয় ডিস্ক `--fail` করো এবং দেখো spare নিজে থেকে rebuild শুরু করে কিনা।
- RAID 1 অ্যারের ওপর PV → VG → LV বানিয়ে "LVM on RAID" সাজাও।

সারসংক্ষেপ

LVM দ্রুত রেফারেন্স

কাজ	কমান্ড
PV বানানো	`pvcreate /dev/sdb`
VG বানানো	`vgcreate vgdata /dev/sdb /dev/sdc`
LV বানানো (সাইজ)	`lvcreate -L 10G -n lvname vgdata`
LV বানানো (সব খালি জায়গা)	`lvcreate -l 100%FREE -n lvname vgdata`
সংক্ষিপ্ত অবস্থা	`pvs`, `vgs`, `lvs`
বিস্তারিত অবস্থা	`pvdisplay`, `vgdisplay`, `lvdisplay`
LV + ফাইলসিস্টেম বড় করা	`lvextend -r -L +5G /dev/vgdata/lvname`
শুধু ext4 বড় করা	`resize2fs /dev/vgdata/lvname`
শুধু XFS বড় করা	`xfs_growfs /mountpoint`
LV ছোট করা (ext4, umount করে)	`lvreduce -r -L 8G /dev/vgdata/lvname`
VG-তে ডিস্ক যোগ	`vgextend vgdata /dev/sdd`
PV থেকে ডেটা সরানো	`pvmove /dev/sdb`
VG থেকে ডিস্ক বাদ	`vgreduce vgdata /dev/sdb` → `pvremove /dev/sdb`
ক্লাউড ডিস্ক বড় হওয়ার পর	`growpart` → `pvresize` → `lvextend -r`
Snapshot বানানো	`lvcreate -s -L 1G -n snap /dev/vgdata/lvname`
Snapshot ফেরত আনা	`lvconvert --merge /dev/vgdata/snap`
LV মুছে ফেলা	`lvremove /dev/vgdata/lvname`

কাজ	কমান্ড
device-mapper দেখা	`dmsetup ls`, `dmsetup table`

RAID দ্রুত রেফারেন্স

কাজ	কমান্ড
RAID বানানো	`mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc`
অবস্থা দেখা	`cat /proc/mdstat`
বিস্তারিত	`mdadm --detail /dev/md0`
সদস্যের superblock	`mdadm --examine /dev/sdb`
কনফিগ সেভ (Debian/Ubuntu)	`mdadm --detail --scan >> /etc/mdadm/mdadm.conf` + `update-initramfs -u`
কনফিগ সেভ (RHEL/Fedora)	`mdadm --detail --scan >> /etc/mdadm.conf` + `dracut -f`
ডিস্ক নষ্ট চিহ্নিত	`mdadm /dev/md0 --fail /dev/sdc`
ডিস্ক বের করা	`mdadm /dev/md0 --remove /dev/sdc`
ডিস্ক যোগ	`mdadm /dev/md0 --add /dev/sdd`
অ্যারে বড় করা	`mdadm --grow /dev/md0 --raid-devices=4`
অ্যারে বন্ধ	`mdadm --stop /dev/md0`
সব অ্যারে জোড়া	`mdadm --assemble --scan`
superblock মোছা	`mdadm --zero-superblock /dev/sdb`
লাইভ নজর	`watch cat /proc/mdstat`

RAID লেভেল মুখস্থ সূত্র

লেভেল	ন্যূনতম	জায়গা	সহ্য করে
0	২	N	০
1	২	1	N−1
5	৩	N−1	১
6	৪	N−2	২
10	৪	N/2	প্রতি জোড়ায় ১

নিজেকে যাচাই করো

1. PV, VG আর LV এর সম্পর্ক একটা উপমা দিয়ে ব্যাখ্যা করো। PE কী?
2. `lvextend -L +5G /dev/vg/lv` চালানোর পর `df -h` আগের সাইজ দেখাচ্ছে। কেন? দুটো উপায়ে সমাধান লেখো।
3. `resize2fs` আর `xfs_growfs` কী argument নেয়, এবং কোনটা ফাইলসিস্টেম ছোটও করতে পারে?
4. `lvreduce` চালানোর সঠিক ক্রম কী, আর ভুল ক্রমে কী হয়?
5. LVM snapshot-এর জন্য origin LV-এর সমান জায়গা লাগে না কেন? Snapshot-এর জায়গা ভরে গেলে কী হয়?
6. ৬টা ২ TB ডিস্ক দিয়ে RAID 5, RAID 6 আর RAID 10-এ কত ব্যবহারযোগ্য জায়গা পাবে? প্রতিটা কয়টা ডিস্ক নষ্ট সহ্য করতে পারে?
7. `/proc/mdstat`-এ `[3/2] [UU_]` এর অর্থ কী? তুমি কী করবে?
8. RAID অ্যারে বানানোর পর `mdadm.conf` আপডেট আর initramfs পুনর্নির্মাণ কেন জরুরি?
9. "আমার সার্ভারে RAID 1 আছে, তাই ব্যাকআপ লাগবে না" — এই কথার তিনটা ভুল দেখাও।
10. সার্ভার চালু রেখে একটা পুরনো LVM ডিস্ক নতুন ডিস্ক দিয়ে বদলানোর ধাপগুলো লেখো।

▼ উত্তর দেখো

1. PV হলো আলাদা জমির প্লট (ডিস্ক/পার্টিশন), VG হলো সব প্লট মিলিয়ে একটা খামার (স্টোরেজ পুল), LV হলো খামারের ভেতরের আলাদা ক্ষেত (যার ওপর ফাইলসিস্টেম বসে)। PE (Physical Extent) হলো VG-এর জায়গার সমান মাপের ছোট টুকরো (ডিফল্ট 4 MiB); LV বানানো বা বাড়ানো মানে কিছু PE বরাদ্দ করা।

2. শুধু LV (block device) বড় হয়েছে, ওপরের ফাইলসিস্টেম এখনো পুরনো সাইজ জানে। সমাধান: (ক) ফাইলসিস্টেম আলাদাভাবে বাড়াও — ext4 হলে `resize2fs /dev/vg/lv`, XFS হলে `xfs_growfs /mountpoint`; (খ) পরের বার `lvextend -r` ব্যবহার করো যাতে একসাথে দুটোই হয়।

3. `resize2fs` নেয় ডিভাইস পাথ (এবং ঐচ্ছিক সাইজ), ext4 বড় করা মাউন্ট অবস্থাতেও যায়, ছোট করা শুধু আনমাউন্ট অবস্থায়। `xfs_growfs` নেয় মাউন্ট পয়েন্ট, শুধু বড় করতে পারে। তাই ছোট করতে পারে শুধু `resize2fs` (ext4)। XFS কখনো ছোট করা যায় না।

4. ব্যাকআপ → আনমাউন্ট → `e2fsck -f` → আগে ফাইলসিস্টেম ছোট (`resize2fs`) → তারপর LV ছোট (`lvreduce`) → মাউন্ট। অথবা সোজা `lvreduce -r`। ভুল ক্রমে (আগে LV ছোট) ফাইলসিস্টেমের শেষের অংশ কেটে যায়, ডেটা ও ফাইলসিস্টেম নষ্ট হয়।

5. Snapshot Copy-on-Write পদ্ধতিতে কাজ করে: শুরুতে কিছু কপি হয় না, শুধু origin-এর যে ব্লক বদলায় তার পুরনো সংস্করণ snapshot-এ রাখা হয়। তাই জায়গা লাগে শুধু পরিবর্তনের পরিমাণের সমান। জায়গা ১০০% ভরে গেলে (পুরনো ধরনের) snapshot invalid হয়ে যায়, আর ব্যবহার করা যায় না।

6. RAID 5: (6−1)×2 = ১০ TB, ১টা ডিস্ক নষ্ট সহ্য করে। RAID 6: (6−2)×2 = ৮ TB, যেকোনো ২টা। RAID 10: 6/2×2 = ৬ TB, প্রতিটা mirror জোড়া থেকে ১টা করে (সর্বোচ্চ ৩টা, তবে একই জোড়ার দুটো নষ্ট হলে সব শেষ)।

7. অ্যারেতে ৩টা ডিস্ক থাকার কথা, চলছে ২টা; তৃতীয় অবস্থানের ডিস্ক নেই বা নষ্ট — অ্যারে degraded। করণীয়: `mdadm --detail` আর `dmesg` / `smartctl` দিয়ে কোন ডিস্ক বুঝে নাও, সিরিয়াল মিলিয়ে বের করো, (দরকারে `--fail`) `--remove`, নতুন ডিস্ক লাগিয়ে `--add`, তারপর `/proc/mdstat`-এ recovery শেষ হওয়া পর্যন্ত নজর রাখো।

8. বুটের সময় (প্রায়ই initramfs-এর ভেতর থেকে) অ্যারে জোড়া লাগানো হয়। `mdadm.conf`-এ ARRAY লাইন না থাকলে বা initramfs-এ পুরনো কনফিগ থাকলে অ্যারে `/dev/md127`-এর মতো অন্য নামে উঠতে পারে বা না-ও উঠতে পারে, ফলে fstab-এর মাউন্ট ব্যর্থ হয়ে বুট আটকে যেতে পারে।

9. (ক) ভুল করে মুছে ফেলা ফাইল দুটো ডিস্ক থেকেই মুছে যায়; (খ) ransomware, ভাইরাস বা ফাইলসিস্টেম corruption-ও mirror হয়ে যায়; (গ) আগুন/চুরি/কন্ট্রোলার বা পাওয়ার সাপ্লাই নষ্টে একসাথে সব ডিস্ক যেতে পারে। RAID শুধু ডিস্ক নষ্ট হওয়া থেকে রক্ষা করে ও uptime দেয়; ব্যাকআপ (3-2-1 নিয়ম) আলাদা প্রয়োজন।

10. নতুন ডিস্ক লাগাও → `pvcreate /dev/sdNEW` → `vgextend vgname /dev/sdNEW` → `pvmove /dev/sdOLD` (ডেটা সরে যাবে, সার্ভার চালু থাকবে) → `vgreduce vgname /dev/sdOLD` → `pvremove /dev/sdOLD` → পুরনো ডিস্ক খুলে ফেলো।

অধ্যায় ৩৪: আর্কাইভ ও কম্প্রেশন

এই অধ্যায়ে যা শিখবে

- **Archive** (অনেক ফাইলকে এক ফাইলে বাঁধা) আর **Compression** (ফাইলের আকার ছোট করা) আসলে দুটো আলাদা কাজ, আর লিনাক্সে কেন এরা আলাদা টুল দিয়ে হয়
- কম্প্রেশন ভেতরে কীভাবে কাজ করে: repetition খোঁজা, dictionary, entropy, lossless বনাম lossy
- `tar` কমান্ড পুরোপুরি: `-c -x -t -v -f -z -j -J --zstd -C --exclude -a`, আর অপশনগুলো মনে রাখার কৌশল
- `gzip`, `gunzip`, `zcat`, `zless`, `zgrep`: সবচেয়ে প্রচলিত কম্প্রেসর
- `bzip2`, `xz`, `zstd`: কোনটা কখন, কোনটা দ্রুত, কোনটা বেশি ছোট করে
- গতি বনাম কম্প্রেশন অনুপাতের তুলনা টেবিল
- `zip` / `unzip` (Windows-এর সাথে আদান-প্রদানের জন্য), `7z`, `rar` / `unrar`
- `.tar.gz`, `.tgz`, `.tar.xz`, `.tar.zst`, `.deb`, `.jar` সহ এক্সটেনশন দেখে চেনা, আর `file` কমান্ড দিয়ে আসল ফরম্যাট বের করা
- পাইপ দিয়ে `tar`-কে `ssh`-এর ভেতর দিয়ে পাঠানো: ডিস্কে কোনো মধ্যবর্তী ফাইল না বানিয়ে এক সার্ভার থেকে আরেক সার্ভারে ফোল্ডার কপি
- multi-core কম্প্রেশন (`pigz`, `xz -T0`, `zstd -T0`), বড় আর্কাইভ ভাগ করা (`split`), আর tarbomb ও path traversal-এর মতো ঝুঁকি

৩৪.১ শুরুর কথা: আর্কাইভ আর কম্প্রেশন কেন দরকার?

ধরো তুমি বাসা বদলাচ্ছ। তোমার কাছে একশোটা বই, পঞ্চাশটা জামা, কিছু থালা-বাসন। এগুলো একটা একটা করে ট্রাকে তুললে বিশৃঙ্খল হবে, কিছু হারাবেও। তাই তুমি দুটো কাজ করবে:

১. **সব জিনিস কয়েকটা বাক্সে ভরবে।** জিনিসের সংখ্যা কমল না, শুধু সব এক জায়গায় গোছানো হলো। এটাই **archive** (আর্কাইভ)।

২. **জামাগুলো vacuum bag-এ ভরে বাতাস বের করে দেবে।** জামা একই থাকল, কিন্তু জায়গা কম লাগল। এটাই **compression** (কম্প্রেশন)।

কম্পিউটারেও ঠিক তাই:

কাজ	উদ্দেশ্য	লিনাক্সের টুল
Archive	অনেক ফাইল + ফোল্ডার + তাদের permission, owner, সময়, symlink সব একটি ফাইলে রাখা	`tar`, `cpio`, `ar`

কাজ	উদ্দেশ্য	লিনাক্সের টুল
Compression	একটি ডেটা-স্রোতের (stream) আকার ছোট করা	`gzip`, `bzip2`, `xz`, `zstd`, `lz4`
দুটোই একসাথে	Windows-ঘরানার ফরম্যাট	`zip`, `7z`, `rar`

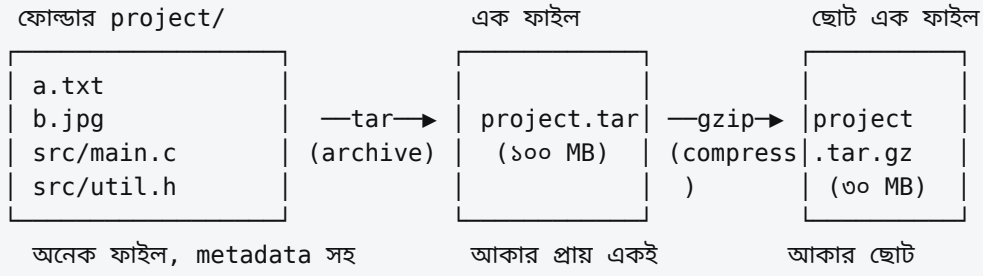
> 📝 **মনে রাখো:** `gzip`, `bzip2`, `xz`, `zstd` এরা **একটা ফাইলকে** কম্প্রেস করে। একটা ফোল্ডার এদের দিয়ে সরাসরি কম্প্রেস করা যায় না। ফোল্ডার কম্প্রেস করতে হলে আগে `tar` দিয়ে এক ফাইলে বাঁধো, তারপর কম্প্রেস করো। এ কারণেই লিনাক্সে `.tar.gz` এর মতো "দুই-স্তরের" এক্সটেনশন দেখা যায়।

কেন লিনাক্সে দুটো কাজ আলাদা?

এটা Unix দর্শনের একটা চমৎকার উদাহরণ: **"একটা প্রোগ্রাম একটা কাজই করবে, এবং ভালোভাবে করবে।"** (অধ্যায় ১-এ এই দর্শনের কথা বলা আছে।)

- `tar` জানে ফাইল সিস্টেমের খুঁটিনাটি: permission, owner, symlink, hard link, timestamp। কিন্তু কম্প্রেশনের কিছুই জানে না।
- `gzip` শুধু bytes-এর একটা স্রোত নেয় আর ছোট করে দেয়। সেটা ফাইল নাকি ফোল্ডার, তা জানে না, জানার দরকারও নেই।

এই আলাদা থাকার সুবিধা হলো যখন নতুন ও ভালো কম্প্রেসর (যেমন `zstd`) এল, তখন `tar` বদলাতে হলো না। শুধু `tar ... | zstd` লিখলেই হলো। পাইপ (অধ্যায় ১৯) দিয়ে দুটো টুল জোড়া লাগানো যায়।



> 💡 **টিপস:** Windows-এর `.zip` ফাইল দুটো কাজই একসাথে করে, আর প্রতিটা ফাইল **আলাদা আলাদা** কম্প্রেস করে। `.tar.gz` পুরো আর্কাইভকে **একসাথে** কম্প্রেস করে। এর ফলাফল নিয়ে একটু পরে কথা বলব ("solid" কম্প্রেশন, ৩৪.১০)।

৩৪.২ কম্প্রেশন ভেতরে কীভাবে কাজ করে (Under the Hood)

কম্প্রেশন কোনো জাদু নয়। এর মূল ধারণা হলো **ডেটার ভেতরের পুনরাবৃত্তি (redundancy) খুঁজে বের করে সংক্ষেপে লেখা।**

Lossless বনাম Lossy

ধরন	মানে	উদাহরণ
Lossless (ক্ষতিহীন)	ডিকম্প্রেস করলে হুবহু আগের ডেটা ফেরত আসে, একটা bit-ও বদলায় না	gzip, bzip2, xz, zstd, zip, 7z, PNG, FLAC
Lossy (ক্ষতিকর)	কিছু তথ্য ইচ্ছা করে ফেলে দেওয়া হয় যা মানুষ সহজে টের পায় না	JPEG, MP3, MP4/H.264

এই অধ্যায়ের সব টুল **lossless**। প্রোগ্রাম, কনফিগ ফাইল বা ডাটাবেস ব্যাকআপে একটা bit বদলে গেলেও সর্বনাশ, তাই এখানে lossless ছাড়া উপায় নেই।

ধারণা ১: Run-Length (একই জিনিস বারবার)

```
আসল:       AAAAAAAAAABBBBBCC        (১৭ অক্ষর)
সংক্ষেপে:  10A5B2C                  (৭ অক্ষর)
```

ধারণা ২: LZ77 (আগে যা দেখেছি তার রেফারেন্স)

এটাই `gzip`, `xz`, `zstd` সবার হৃদয়। কম্প্রেসর একটা "জানালা" (sliding window) ধরে রাখে, অর্থাৎ সাম্প্রতিক কিছু ডেটা মনে রাখে। নতুন কোনো অংশ আগে দেখা অংশের সাথে মিলে গেলে পুরোটা আবার না লিখে লেখে: **"পেছনে X ঘর গিয়ে Y অক্ষর কপি করো"।**

```
লেখা:  "ami bhat khai, tumi bhat khao"

            ┌── এখানে "bhat kha" আগেও এসেছে (১৬ অক্ষর পেছনে)
            ▼
"ami bhat khai, tumi " + <পেছনে ১৬, ৮ অক্ষর কপি> + "o"
```

জানালা যত বড়, তত দূরের পুনরাবৃত্তি ধরা যায়, আর তত ভালো কম্প্রেশন হয়, কিন্তু মেমোরি বেশি লাগে।

টুল	মূল অ্যালগরিদম	জানালা/ব্লক
gzip	DEFLATE = LZ77 + Huffman	৩২ KB জানালা
bzip2	Burrows-Wheeler Transform + Huffman	১০০–৯০০ KB ব্লক
xz	LZMA2	ডিফল্ট লেভেলে ৮ MB, সর্বোচ্চ লেভেলে ৬৪ MB dictionary
zstd	LZ77 + Finite State Entropy + Huffman	লেভেল অনুযায়ী বদলায়, `--long` দিলে অনেক বড়

ধারণা ৩: Entropy coding (ঘন ঘন আসা জিনিসের ছোট কোড)

Morse code-এর কথা ভাবো: ইংরেজিতে সবচেয়ে বেশি আসে `E`, তাই তার কোড সবচেয়ে ছোট (`.`)। কম আসা `Q`-এর কোড লম্বা (`--.-`)। **Huffman coding** ঠিক এভাবেই প্রতিটা byte-এর জন্য তার ব্যবহারের হার অনুযায়ী কোড বানায়।

কেন কিছু ফাইল কম্প্রেস হয় না?

`.jpg`, `.mp4`, `.mp3`, `.zip`, `.gz` এগুলো **আগে থেকেই কম্প্রেস করা**। এদের ভেতরে পুনরাবৃত্তি প্রায় নেই। এদের আবার কম্প্রেস করলে আকার প্রায় একই থাকে, কখনো একটু **বেড়েও** যায় (header-এর কারণে)। এলোমেলো (random) ডেটাও কম্প্রেস হয় না।

```
$ head -c 10M /dev/urandom > random.bin
$ gzip -k random.bin
$ ls -l random.bin random.bin.gz
-rw-r--r-- 1 rahim rahim 10485760 Nov 12 10:00 random.bin
-rw-r--r-- 1 rahim rahim 10488981 Nov 12 10:00 random.bin.gz
```

দেখো, `.gz` ফাইলটা আসলটার চেয়ে **বড়**!

> 📝 **মনে রাখো:** টেক্সট, লগ, সোর্স কোড, SQL dump, CSV দারুণ কম্প্রেস হয় (প্রায়ই ৫–১০ গুণ ছোট)। ছবি, ভিডিও, গান প্রায় হয়ই না। তাই ১০ GB ভিডিও ফোল্ডার `xz -9` দিয়ে ঘণ্টাখানেক কম্প্রেস করে লাভ নেই, শুধু `tar` করো।

Compression level

প্রায় সব টুলে `-1` থেকে `-9` লেভেল আছে:

- `-1`: দ্রুততম, কম ছোট করে (কম খোঁজে)
- `-9`: ধীরতম, সবচেয়ে ছোট করে (বেশি খোঁজে, বেশি মেমোরি নেয়)

একটা গুরুত্বপূর্ণ ব্যাপার: **ডিকম্প্রেশনের গতি প্রায় লেভেলের উপর নির্ভর করে না।** `-9` দিয়ে বানানো ফাইল খুলতেও প্রায় `-1`-এর মতোই সময় লাগে। এ কারণে যে ফাইল একবার বানিয়ে হাজার জনে ডাউনলোড করবে (যেমন সফটওয়্যার রিলিজ), সেটা উঁচু লেভেলে বানানোই লাভজনক।

৩৪.৩ `tar`: Tape Archiver

নামের ইতিহাস ও tar ফাইলের গঠন

`tar` মানে **T**ape **AR**chive। ১৯৭৯ সালের Unix-এ এটা বানানো হয়েছিল ম্যাগনেটিক টেপে ব্যাকআপ রাখার জন্য। টেপে random access নেই, শুধু শুরু থেকে শেষ পর্যন্ত পড়া যায়। তাই tar ফরম্যাটও একদম **সরলরৈখিক (sequential)**: ফাইলের পর ফাইল সাজানো।

```
project.tar এর ভেতরের গঠন (প্রতিটা ঘর ৫১২-byte ব্লকের গুণিতক)

┌──────────┬──────────────────┬──────────┬──────────────┬──────────┬──────────┐
│ Header   │ a.txt-এর ডেটা    │ Header   │ src/main.c   │ Header   │ দুটো     │
│ (a.txt)  │ (৫১২-এর গুণিতকে  │ (src/    │ -এর ডেটা    │ ...      │ খালি     │
│ ৫১২ byte │  padding করা)    │  main.c) │              │          │ ব্লক     │
└──────────┴──────────────────┴──────────┴──────────────┴──────────┴──────────┘
   │                                                                (শেষ চিহ্ন)
   ▼
Header-এ থাকে: ফাইলের নাম, mode (permission), uid/gid, owner/group নাম,
আকার, mtime (শেষ পরিবর্তনের সময়), ধরন (সাধারণ ফাইল/ডিরেক্টরি/symlink/
hard link/device), symlink-এর লক্ষ্য, checksum
```

এই গঠন থেকে কয়েকটা জরুরি সিদ্ধান্ত বের হয়:

1. **কোনো কেন্দ্রীয় সূচিপত্র (index) নেই।** তাই `.tar.gz`-এর ভেতরে কী আছে দেখতে (`tar -t`) পুরো ফাইলটা শুরু থেকে শেষ পর্যন্ত পড়তে ও ডিকম্প্রেস করতে হয়। ১০ GB আর্কাইভ থেকে একটা ছোট ফাইল বের করতেও পুরোটা পড়তে হতে পারে। `.zip`-এর শেষে একটা সূচিপত্র (central directory) থাকে, তাই `unzip -l` তাৎক্ষণিক।
2. **শেষে নতুন ফাইল যোগ করা সহজ** (`tar -r`), কিন্তু শুধু কম্প্রেস-না-করা `.tar`-এ।
3. **স্রোত হিসেবে কাজ করে।** আর্কাইভ বানাতে বানাতেই পাইপ দিয়ে অন্য দিকে পাঠানো যায়। এ কারণেই `tar | ssh` এত শক্তিশালী (৩৪.১২)।

আধুনিক লিনাক্সে (Ubuntu, Debian, Fedora, RHEL) `tar` মানে **GNU tar**। এটা POSIX `ustar` ও `pax` ফরম্যাট বোঝে, আর লম্বা ফাইলনাম, বড় ফাইল (৮ GB-এর বেশি) সবই সামলায়।

```
$ tar --version | head -1
tar (GNU tar) 1.35
```

(সংস্করণ নম্বর distro অনুযায়ী আলাদা হবে।)

tar-এর মূল syntax

```
tar [মোড] [অপশন...] -f আর্কাইভ_ফাইল [ফাইল/ফোল্ডার...]
```

মোড (প্রতিবার ঠিক একটা লাগবে):

অপশন	লম্বা রূপ	কাজ
`-c`	`--create`	নতুন আর্কাইভ বানাও
`-x`	`--extract` (`--get`)	আর্কাইভ থেকে ফাইল বের করো
`-t`	`--list`	ভেতরে কী আছে তার তালিকা দেখাও
`-r`	`--append`	শেষে ফাইল যোগ করো (শুধু কম্প্রেস-না-করা `.tar`)
`-u`	`--update`	শুধু নতুন/বদলানো ফাইল শেষে যোগ করো
`-d`	`--diff` / `--compare`	আর্কাইভ আর ডিস্কের ফাইলের পার্থক্য দেখাও
	`--delete`	আর্কাইভ থেকে ফাইল মুছে দাও (শুধু কম্প্রেস-না-করা `.tar`)
`-A`	`--concatenate`	একটা tar-এর শেষে আরেকটা tar জোড়া লাগাও

সাধারণ অপশন:

অপশন	লম্বা রূপ	কাজ
`-f FILE`	`--file=FILE`	কোন আর্কাইভ ফাইল নিয়ে কাজ। `-` দিলে stdin/stdout
`-v`	`--verbose`	প্রতিটা ফাইলের নাম দেখাও (`-vv` দিলে `ls -l` ধাঁচের বিস্তারিত)
`-z`	`--gzip`	gzip দিয়ে কম্প্রেস/ডিকম্প্রেস (`.tar.gz`)
`-j`	`--bzip2`	bzip2 দিয়ে (`.tar.bz2`)
`-J`	`--xz`	xz দিয়ে (`.tar.xz`)
	`--zstd`	zstd দিয়ে (`.tar.zst`), GNU tar 1.31+
`-a`	`--auto-compress`	আর্কাইভের এক্সটেনশন দেখে কম্প্রেসর ঠিক করো
`-I CMD`	`--use-compress-program=CMD`	নিজের পছন্দের কম্প্রেসর (যেমন `pigz`, `'zstd -19'`)
`-C DIR`	`--directory=DIR`	কাজ শুরুর আগে DIR-এ চলে যাও
	`--exclude=PATTERN`	মিলে যাওয়া ফাইল বাদ দাও
`-X FILE`	`--exclude-from=FILE`	FILE-এ লেখা প্যাটার্নগুলো বাদ দাও
	`--exclude-vcs`	`.git`, `.svn` ইত্যাদি বাদ দাও
`-T FILE`	`--files-from=FILE`	কোন ফাইলগুলো নেবে তা FILE থেকে পড়ো
`-p`	`--preserve-permissions`	extract-এর সময় permission হুবহু রাখো (root হলে ডিফল্ট)
	`--same-owner`	extract-এর সময় owner হুবহু রাখো (root হলে ডিফল্ট)
	`--numeric-owner`	নামের বদলে uid/gid সংখ্যা ব্যবহার করো
	`--strip-components=N`	extract-এর সময় path-এর প্রথম N অংশ বাদ দাও
`-k`	`--keep-old-files`	আগে থেকে থাকা ফাইল ওভাররাইট কোরো না (থাকলে error)
	`--skip-old-files`	আগে থেকে থাকা ফাইল চুপচাপ এড়িয়ে যাও
`-h`	`--dereference`	symlink-এর বদলে আসল ফাইলটা রাখো

অপশন	লম্বা রূপ	কাজ
`-P`	`--absolute-names`	শুরুর `/` মুছবে না (বিপজ্জনক, ৩৪.১৪ দেখো)
`-W`	`--verify`	লেখার পর যাচাই (কম্প্রেস-না-করা আর্কাইভে)
	`--wildcards`	extract/list-এ `*` ধরনের প্যাটার্ন বোঝো
`-g FILE`	`--listed-incremental=FILE`	incremental ব্যাকআপ (অধ্যায় ৩৫)
	`--xattrs`, `--acls`, `--selinux`	extended attribute, ACL, SELinux লেবেল রাখো
	`--checkpoint=N`	প্রতি N ব্লক পর পর অগ্রগতি জানাও

মনে রাখার কৌশল 🧠

`tar`-এর অপশন নতুনদের কাছে মন্ত্রের মতো লাগে। কিন্তু একটু ভাঙলেই সহজ:

```
tar  -c   z   v   f   backup.tar.gz   ফোল্ডার/
      |   |   |   |
      |   |   |   └── f = file   "এই নামের ফাইলে" (সবসময় শেষে, তার ঠিক পরেই নাম)
      |   |   └────── v = verbose "বলতে বলতে করো"
      |   └────────── z = gzip    "চেপে ছোট করো"
      └────────────── c = create  "বানাও"
```

তিনটা মোড, তিনটা প্রশ্ন:

অক্ষর	ইংরেজি	বাংলায় মনে রাখো
`c`	**C**reate	**"চাই বানাতে"**
`x`	e**X**tract	**"খুলে বের করো"** (x দেখতে খোলা কাঁচির মতো ✂)
`t`	lis**T** / **T**able	**"তালিকা দেখাও"**

কম্প্রেসরের অক্ষর:

অক্ষর	কম্প্রেসর	মনে রাখার উপায়
`z`	g**z**ip	gzip-এর মধ্যে z আছে
`j`	bzip2	"b" অক্ষর আগে থেকে ব্যস্ত (`-b` = blocking factor), তাই পাশের j
`J`	xz	বড় হাতের J: "বড়" কম্প্রেশন, xz সবচেয়ে বেশি ছোট করে
`--zstd`	zstd	এর জন্য কোনো এক-অক্ষরের অপশন নেই, পুরো নাম লেখো

জনপ্রিয় ইংরেজি কৌশল:

- `tar -czvf` → "**C**reate **Z**e **V**ile **F**ile"
- `tar -xzvf` → "e**X**tract **Z**e **V**ile **F**ile"
- `tar -tvf` → "**T**ell **V**erbose **F**ile" (কী আছে বলো)

> ⚠️ **সতর্কতা:** `-f` অপশনের **ঠিক পরেই** আর্কাইভের নাম দিতে হয়। `tar -cfv backup.tar dir/` লিখলে tar ভাববে আর্কাইভের নাম `v`! তখন `v` নামে একটা ফাইল তৈরি হবে আর `backup.tar` আর্কাইভের **ভেতরে** ঢুকে যাবে (যদি সেটা থাকে)। তাই নিয়ম: **`f` সবসময় অক্ষরগুচ্ছের শেষে।**

> 💡 **টিপস:** `tar`-এ ড্যাশ ছাড়াও লেখা যায়: `tar czvf x.tar.gz dir/`। এটা পুরোনো (BSD) স্টাইল, এখনো সব জায়গায় চলে। এই বইয়ে আমরা ড্যাশসহ লিখব, কারণ সেটা অন্য কমান্ডের সাথে সামঞ্জস্যপূর্ণ।

৩৪.৩.১ আর্কাইভ বানানো (`-c`)

প্রথমে অনুশীলনের জন্য একটা ফোল্ডার বানাই:

```
$ mkdir -p ~/lab34/project/{src,docs,logs}
$ cd ~/lab34
$ echo 'int main(){return 0;}' > project/src/main.c
$ echo '# My project' > project/docs/README.md
$ seq 1 100000 > project/logs/app.log
$ tree project
project
├── docs
│   └── README.md
├── logs
│   └── app.log
└── src
    └── main.c

4 directories, 3 files
```

কম্প্রেশন ছাড়া শুধু আর্কাইভ:

```
$ tar -cvf project.tar project/
project/
project/src/
project/src/main.c
project/docs/
project/docs/README.md
project/logs/
project/logs/app.log
```

gzip দিয়ে কম্প্রেস করে:

```
$ tar -czvf project.tar.gz project/
```

bzip2, xz, zstd দিয়ে:

```
$ tar -cjf project.tar.bz2 project/
$ tar -cJf project.tar.xz  project/
```

```
$ tar --zstd -cf project.tar.zst project/
```

আকার তুলনা করি:

```
$ ls -lh project.tar*
-rw-r--r-- 1 rahim rahim 600K Nov 12 10:05 project.tar
-rw-r--r-- 1 rahim rahim  48K Nov 12 10:06 project.tar.bz2
-rw-r--r-- 1 rahim rahim 122K Nov 12 10:05 project.tar.gz
-rw-r--r-- 1 rahim rahim  40K Nov 12 10:06 project.tar.xz
-rw-r--r-- 1 rahim rahim  75K Nov 12 10:06 project.tar.zst
```

(সংখ্যাগুলো আনুমানিক, তোমার মেশিনে একটু আলাদা হবে। `seq`-এর আউটপুট খুবই পুনরাবৃত্তিময়, তাই অনুপাত অনেক ভালো দেখাচ্ছে।)

`-a` দিয়ে এক্সটেনশন দেখে স্বয়ংক্রিয় বাছাই:

```
$ tar -caf project.tar.xz project/      # .xz দেখে xz ব্যবহার করবে
$ tar -caf project.tar.zst project/     # .zst দেখে zstd
```

নিজের পছন্দের কম্প্রেসর ও লেভেল (`-I`):

```
$ tar -I 'zstd -19 -T0' -cf project.tar.zst project/
$ tar -I pigz -cf project.tar.gz project/          # multi-core gzip
$ XZ_OPT='-9 -T0' tar -cJf project.tar.xz project/ # পরিবেশ ভেরিয়েবল দিয়ে xz-কে অপশন
```

> 📝 **মনে রাখো:** তুমি যে path দিয়ে আর্কাইভ বানাবে, ভেতরে সেই path-ই থাকবে। `tar -czf a.tar.gz /home/rahim/lab34/project` লিখলে ভেতরে থাকবে `home/rahim/lab34/project/...` (শুরুর `/` বাদ দিয়ে)। extract করলে এই পুরো ফোল্ডার-শিকল তৈরি হবে। পরিষ্কার আর্কাইভ চাইলে `-C` ব্যবহার করো (নিচে দেখো)।

৩৪.৩.২ শুরুর `/` সরিয়ে দেওয়ার বার্তা

```
$ tar -czf etc-backup.tar.gz /etc/hostname /etc/hosts
tar: Removing leading `/' from member names
```

এটা কোনো error না, **নিরাপত্তা ব্যবস্থা**। আর্কাইভে `/etc/hosts` হুবহু থাকলে অন্য কেউ সেটা extract করার সময় তার আসল `/etc/hosts` ওভাররাইট হয়ে যেত। GNU tar তাই `etc/hosts` হিসেবে রাখে, যাতে extract সবসময় **বর্তমান ফোল্ডারের ভেতরে** হয়।

৩৪.৩.৩ `-C`: কোন ফোল্ডার থেকে/কোন ফোল্ডারে

`-C DIR` মানে "কাজ শুরুর আগে `cd DIR` করো"।

বানানোর সময়: পুরো path-এর ঝামেলা ছাড়া আর্কাইভ:

```
$ tar -czf /tmp/project.tar.gz -C /home/rahim/lab34 project
$ tar -tzf /tmp/project.tar.gz | head -3
project/
project/src/
project/src/main.c
```

একাধিক `-C` দিয়ে আলাদা আলাদা জায়গা থেকে একসাথে নেওয়া যায়:

```
$ tar -czf mix.tar.gz -C /etc hostname -C /home/rahim/lab34 project
```

খোলার সময়: নির্দিষ্ট ফোল্ডারে extract:

```
$ mkdir -p /tmp/restore
$ tar -xzf project.tar.gz -C /tmp/restore
$ ls /tmp/restore
project
```

> ⚠️ **সতর্কতা:** `-C` -এর লক্ষ্য ফোল্ডার **আগে থেকে থাকতে হবে**। না থাকলে `tar: /tmp/restore: Cannot open: No such file or directory`। তাই আগে `mkdir -p` চালাও।

৩৪.৩.৪ ভেতরে কী আছে দেখা (`-t`)

আর্কাইভ খোলার আগে সবসময় একবার দেখে নাও। এটা ভালো অভ্যাস।

```
$ tar -tzf project.tar.gz
project/
project/src/
project/src/main.c
project/docs/
project/docs/README.md
project/logs/
project/logs/app.log
```

বিস্তারিত দেখতে `-v`:

```
$ tar -tvzf project.tar.gz
drwxr-xr-x rahim/rahim       0 2026-11-12 10:04 project/
drwxr-xr-x rahim/rahim       0 2026-11-12 10:04 project/src/
-rw-r--r-- rahim/rahim      22 2026-11-12 10:04 project/src/main.c
drwxr-xr-x rahim/rahim       0 2026-11-12 10:04 project/docs/
-rw-r--r-- rahim/rahim      13 2026-11-12 10:04 project/docs/README.md
drwxr-xr-x rahim/rahim       0 2026-11-12 10:04 project/logs/
-rw-r--r-- rahim/rahim  588895 2026-11-12 10:04 project/logs/app.log
```

কলামগুলো: permission (অধ্যায় ১৬), owner/group, আকার (byte), তারিখ-সময়, path।

নির্দিষ্ট প্যাটার্ন খুঁজতে:

```
$ tar -tzf project.tar.gz --wildcards '*.c'
project/src/main.c
$ tar -tzf big.tar.gz | grep -i readme
```

> 💡 **টিপস:** আধুনিক GNU tar **extract আর list করার সময়** ফাইলের শুরুর "magic bytes" দেখে নিজেই কম্প্রেশন চিনে ফেলে। তাই `tar -tf project.tar.xz` বা `tar -xf project.tar.zst` কোনো `-z/-J` ছাড়াই কাজ করে। কিন্তু **বানানোর সময়** (`-c`) কম্প্রেসর বলে দিতে হয় (`-z` , `-J` , `--zstd` বা `-a`)। শুধু stdin থেকে পড়ার সময় (`-f -`) স্বয়ংক্রিয় চেনা কাজ করে না, তখন অপশন লিখতে হয়।

৩৪.৩.৫ আর্কাইভ খোলা (`-x`)

```
$ cd /tmp/restore
$ tar -xzvf ~/lab34/project.tar.gz
project/
project/src/
project/src/main.c
...
```

শুধু একটা বা কয়েকটা ফাইল বের করা: ঠিক যে path `-t` -এ দেখায়, সেটা লেখো:

```
$ tar -xzf project.tar.gz project/docs/README.md
$ tar -xzf project.tar.gz --wildcards 'project/src/*.c'
```

`--strip-components` : path-এর শুরুর অংশ ফেলে দেওয়া। সফটওয়্যার ডাউনলোড করলে প্রায়ই `nodejs-v22.1.0-linux-x64/bin/node` ধরনের গঠন থাকে। শুধু ভেতরের জিনিস চাইলে:

```
$ tar -xzf project.tar.gz --strip-components=1 -C /tmp/flat
$ ls /tmp/flat
docs  logs  src
```

stdout-এ বের করা (`-O`): ডিস্কে না লিখে সরাসরি দেখা:

```
$ tar -xzOf project.tar.gz project/docs/README.md
# My project
```

ওভাররাইট নিয়ন্ত্রণ:

পরিস্থিতি	অপশন
ডিফল্ট	একই নামের ফাইল থাকলে ওভাররাইট করে
থাকলে error দাও, ছুঁয়ো না	`-k` / `--keep-old-files`
থাকলে চুপচাপ এড়িয়ে যাও	`--skip-old-files`
শুধু আর্কাইভের কপিটা নতুন হলে ওভাররাইট	`--keep-newer-files`

মালিকানা ও permission:

- সাধারণ ইউজার হিসেবে extract করলে সব ফাইলের মালিক **তুমি** হবে, আর permission থেকে তোমার `umask` (অধ্যায় ১৬) অনুযায়ী কিছু bit বাদ যাবে।
- `sudo tar -x...` করলে আর্কাইভের আসল owner ও permission হুবহু ফিরে আসে (`--same-owner` ও `-p` root-এর জন্য ডিফল্ট)। সিস্টেম ব্যাকআপ ফেরত আনার সময় এটা জরুরি।
- উল্টোটা চাইলে, অর্থাৎ root হয়েও owner হিসেবে নিজেকে রাখতে, `--no-same-owner` দাও।

৩৪.৩.৬ বাদ দেওয়া (`--exclude`)

```
$ tar -czf project.tar.gz --exclude='*.log' project/
$ tar -czf site.tar.gz \
      --exclude='node_modules' \
      --exclude='.git' \
      --exclude='*.tmp' \
      --exclude='project/cache/*' \
      project/
```

একটা ফাইলে প্যাটার্নের তালিকা রেখে:

```
$ cat exclude.txt
*.log
node_modules
.git
$ tar -czf project.tar.gz -X exclude.txt project/
```

অন্যান্য সুবিধাজনক বাদ দেওয়ার অপশন:

অপশন	কাজ
`--exclude-vcs`	`.git`, `.svn`, `.hg`, `CVS` ইত্যাদি বাদ
`--exclude-vcs-ignores`	`.gitignore` ফাইলে লেখা জিনিস বাদ
`--exclude-caches`	`CACHEDIR.TAG` ফাইলওয়ালা ফোল্ডারের ভেতরের জিনিস বাদ
`--exclude-tag=FILE`	যে ফোল্ডারে FILE আছে তার ভেতরের জিনিস বাদ
`--one-file-system`	অন্য মাউন্ট করা ফাইলসিস্টেমে ঢুকবে না (যেমন `/proc`)

> ⚠️ **সতর্কতা:** পুরোনো GNU tar-এ `--exclude` অপশনটা ফাইল/ফোল্ডারের নামের **আগে** লিখতে হয়। নতুন সংস্করণে পরে দিলেও সাধারণত কাজ করে, তবে সতর্কবার্তা দেয়। সবচেয়ে নিরাপদ অভ্যাস: **সব অপশন আগে, ফাইলের নাম শেষে।**

> 💡 **টিপস:** প্যাটার্ন সবসময় একক উদ্ধৃতিতে (`'*.log'`) রাখো। না রাখলে শেল নিজেই `*.log` বর্তমান ফোল্ডারের ফাইলনামে বদলে দেবে (globbing, অধ্যায় ৯), তখন tar ভুল প্যাটার্ন পাবে।

৩৪.৩.৭ ফাইলের তালিকা অন্য কমান্ড থেকে (`-T`)

শেষ ৭ দিনে বদলানো ফাইলগুলোর ব্যাকআপ (অধ্যায় ১২-এর `find`):

```
$ find project -type f -mtime -7 -print0 | tar --null -czf recent.tar.gz -T -
```

এখানে `-print0` আর `--null` ফাইলনামে স্পেস বা নতুন লাইন থাকলেও সমস্যা হতে দেয় না। `-T -` মানে "তালিকা stdin থেকে পড়ো"।

৩৪.৩.৮ যোগ, আপডেট, মোছা, তুলনা

```
$ tar -rf project.tar newfile.txt          # শেষে যোগ
$ tar -uf project.tar project/             # শুধু বদলানো ফাইল যোগ
$ tar --delete -f project.tar project/logs/app.log
$ tar -df project.tar                      # ডিস্কের সাথে তুলনা
project/src/main.c: Mod time differs
project/src/main.c: Size differs
```

> ⚠️ **সতর্কতা:** `-r`, `-u`, `--delete` **কম্প্রেস করা আর্কাইভে কাজ করে না।** `tar -rzf x.tar.gz` চালালে `tar: Cannot update compressed archives` error আসবে। কারণ কম্প্রেস করা স্রোতের মাঝখানে কিছু বসাতে বা মুছতে হলে পুরোটা আবার কম্প্রেস করতে হয়। তখন: ডিকম্প্রেস → বদলাও → আবার কম্প্রেস।

> 📝 **মনে রাখো:** `-u` পুরোনো কপি মোছে না, শুধু নতুনটা শেষে যোগ করে। extract করলে শেষেরটা (নতুনটা) আগেরটার উপর লেখা হয়, তাই ফলাফল ঠিক থাকে, কিন্তু আর্কাইভের আকার বাড়ে।

৩৪.৩.৯ অগ্রগতি দেখা

বড় আর্কাইভে কতটুকু হলো জানতে `pv` (Pipe Viewer, `sudo apt install pv` / `sudo dnf install pv`):

```
$ tar -cf - project/ | pv | gzip > project.tar.gz
 590KiB 0:00:00 [ 115MiB/s] [ <=>                     ]

$ pv project.tar.gz | tar -xzf -
 122KiB 0:00:00 [ 210MiB/s] [==================================>] 100%
```

অথবা tar-এর নিজস্ব checkpoint:

```
$ tar --checkpoint=1000 --checkpoint-action=dot -czf big.tar.gz bigdir/
..........
```

৩৪.৪ `gzip`: সবচেয়ে প্রচলিত কম্প্রেসর

`gzip` (GNU zip) ১৯৯২ সাল থেকে আছে। এটা দ্রুত, সব জায়গায় থাকে, ওয়েব সার্ভার এটা দিয়েই HTTP response কম্প্রেস করে (`Content-Encoding: gzip`), আর লিনাক্সের লগ রোটেশন (`/var/log/syslog.2.gz`) এটাই ব্যবহার করে।

syntax ও আচরণ

```
gzip [অপশন] ফাইল...
```

গুরুত্বপূর্ণ: ডিফল্টে `gzip` **মূল ফাইলটা মুছে** তার জায়গায় `.gz` ফাইল রাখে। অর্থাৎ ফাইল বদলে যায়, কপি থাকে না।

```
$ ls
notes.txt
$ gzip notes.txt
$ ls
notes.txt.gz
$ gunzip notes.txt.gz
$ ls
notes.txt
```

অপশনের টেবিল

অপশন	কাজ
`-k` / `--keep`	মূল ফাইল রেখে দাও (gzip 1.6+)
`-d` / `--decompress`	ডিকম্প্রেস (`gunzip`-এর সমান)
`-c` / `--stdout`	ফলাফল stdout-এ পাঠাও, মূল ফাইল অক্ষত থাকে
`-1` … `-9`	লেভেল; `-1` = `--fast`, `-9` = `--best`, ডিফল্ট `-6`
`-l` / `--list`	কম্প্রেসড/আসল আকার ও অনুপাত দেখাও
`-t` / `--test`	ফাইল ঠিক আছে কি না যাচাই
`-v` / `--verbose`	প্রতিটা ফাইলের অনুপাত দেখাও
`-r` / `--recursive`	ফোল্ডারের ভেতরের **প্রতিটা ফাইল আলাদা করে** কম্প্রেস
`-f` / `--force`	জোর করে (আগে থেকে `.gz` থাকলে বা symlink হলেও)
`-n` / `--no-name`	মূল নাম ও সময় header-এ রেখো না (reproducible build-এ কাজে লাগে)
`-N` / `--name`	ডিকম্প্রেসের সময় header-এর মূল নাম ও সময় ফেরত আনো

উদাহরণ

```
$ gzip -kv project/logs/app.log
project/logs/app.log:    78.9% -- created project/logs/app.log.gz

$ gzip -l project/logs/app.log.gz
         compressed        uncompressed  ratio uncompressed_name
             124185              588895  78.9% project/logs/app.log

$ gzip -t project/logs/app.log.gz && echo "ফাইল ঠিক আছে"
ফাইল ঠিক আছে
```

মূল ফাইল না ছুঁয়ে অন্য নামে বানানো:

```
$ gzip -c report.csv > /backup/report-2026-11-12.csv.gz
```

> ⚠️ **সতর্কতা:** `gzip -r folder/` ফোল্ডারটাকে একটা `.gz` বানায় **না**। এটা ভেতরের প্রতিটা ফাইলকে `a.txt.gz`, `b.txt.gz` এভাবে আলাদা করে কম্প্রেস করে আর মূলগুলো মুছে দেয়। পুরো ফোল্ডার এক ফাইলে চাইলে `tar -czf` ব্যবহার করো।

> 📝 **মনে রাখো:** `gzip -l`-এ দেখানো আসল আকার ৪ GB-এর বেশি ফাইলের জন্য ভুল হতে পারে, কারণ gzip-এর trailer-এ আকার মাত্র ৩২ bit-এ রাখা হয়। নিশ্চিত হতে `zcat file.gz | wc -c` চালাও।

`gunzip`, `zcat` ও z-পরিবার

কমান্ড	কাজ	সমতুল্য
`gunzip f.gz`	ডিকম্প্রেস করে `f` বানায়, `.gz` মুছে	`gzip -d`
`zcat f.gz`	ডিকম্প্রেস করে স্ক্রিনে দেখায়, ফাইল অক্ষত	`gzip -dc`
`zless f.gz`	`less` দিয়ে পাতায় পাতায় পড়া	
`zmore f.gz`	`more` দিয়ে পড়া	
`zgrep pattern f.gz`	খোলা ছাড়াই `grep`	
`zegrep`, `zfgrep`	`grep -E`, `grep -F`-এর z-সংস্করণ	
`zdiff a.gz b.gz`	দুটো কম্প্রেসড ফাইলের `diff`	
`zcmp`	`cmp`-এর z-সংস্করণ	
`zforce`	`.gz` ছাড়া নামের gzip ফাইলে `.gz` লাগায়	
`znew`	পুরোনো `.Z` ফাইলকে `.gz`-এ রূপান্তর	

লগ নিয়ে কাজের বাস্তব উদাহরণ (অধ্যায় ২৮):

```
$ ls /var/log/syslog*          # Ubuntu/Debian; RHEL/Fedora-তে /var/log/messages*
/var/log/syslog  /var/log/syslog.1  /var/log/syslog.2.gz  /var/log/syslog.3.gz

$ zgrep -i "error" /var/log/syslog.2.gz | head -3
$ zcat /var/log/syslog.*.gz | grep -c "CRON"
42
$ zless /var/log/syslog.2.gz
```

> 💡 **টিপস:** আধুনিক `less` (অনেক distro-তে `lesspipe`-এর মাধ্যমে) সরাসরি `less file.gz` দিয়েও খুলতে পারে। Ubuntu/Debian-এ `~/.bashrc`-এ সাধারণত `eval "$(lesspipe)"` লাইনটা থাকে।

Multi-core gzip: `pigz`

সাধারণ `gzip` একটা CPU core ব্যবহার করে। `pigz` (parallel implementation of gzip) সব core ব্যবহার করে, আর একই `.gz` ফরম্যাট বানায়:

```
$ sudo apt install pigz       # Debian/Ubuntu
$ sudo dnf install pigz       # Fedora/RHEL
$ pigz -k -p 8 bigfile.sql    # ৮টা thread
$ tar -I pigz -cf backup.tar.gz /data
```

৩৪.৫ `bzip2`

`bzip2` gzip-এর চেয়ে ভালো ছোট করে কিন্তু বেশ ধীর, বিশেষ করে ডিকম্প্রেশনে। এটা Burrows-Wheeler Transform (BWT) ব্যবহার করে। BWT ডেটা ব্লক ধরে এমনভাবে সাজায় যে একই রকম অক্ষর পাশাপাশি চলে আসে, তারপর সেটা সহজে কম্প্রেস হয়।

আজকাল নতুন কাজে `bzip2`-এর জায়গা প্রায় পুরোটাই `xz` আর `zstd` নিয়ে নিয়েছে। তবে পুরোনো আর্কাইভে অনেক `.bz2` পাবে, তাই চেনা দরকার।

অপশন	কাজ
`-k`	মূল ফাইল রাখো
`-d`	ডিকম্প্রেস (`bunzip2`)
`-c`	stdout-এ
`-1` … `-9`	**ব্লকের আকার** (১০০ KB … ৯০০ KB), ডিফল্ট `-9`
`-t`	যাচাই
`-v`	বিস্তারিত
`-f`	জোর করে

```
$ bzip2 -kv app.log
  app.log:  12.517:1,  0.639 bits/byte, 92.01% saved, 588895 in, 47048 out.
$ bunzip2 -k app.log.bz2
$ bzcat app.log.bz2 | tail -1
100000
```

সঙ্গী কমান্ড: `bunzip2`, `bzcat`, `bzless`, `bzmore`, `bzgrep`, `bzdiff`, `bzip2recover` (নষ্ট `.bz2` ফাইল থেকে অক্ষত ব্লকগুলো উদ্ধার করে)।

Multi-core সংস্করণ: `pbzip2` বা `lbzip2`।

৩৪.৬ `xz`: সর্বোচ্চ ছোট করার জন্য

`xz` LZMA2 অ্যালগরিদম ব্যবহার করে (7-Zip-এর মূল অ্যালগরিদমের উত্তরসূরি)। সাধারণ টুলগুলোর মধ্যে এটা সাধারণত **সবচেয়ে ছোট** ফাইল বানায়, কিন্তু কম্প্রেস করতে ধীর আর বেশি মেমোরি খায়। ডিকম্প্রেশন মোটামুটি দ্রুত (bzip2-এর চেয়ে অনেক দ্রুত)।

কোথায় দেখবে: লিনাক্স কার্নেল সোর্স (`linux-6.x.tar.xz`), অনেক সফটওয়্যার রিলিজ, Debian প্যাকেজের ভেতরের ডেটা, Fedora/RHEL-এর কার্নেল মডিউল (`.ko.xz`)।

অপশন	কাজ
`-k`	মূল ফাইল রাখো
`-d`	ডিকম্প্রেস (`unxz`)
`-c`	stdout-এ
`-0` … `-9`	লেভেল, ডিফল্ট `-6`
`-e` / `--extreme`	একই লেভেলে আরও একটু ছোট, আরও ধীর (যেমন `-9e`)
`-T N` / `--threads=N`	N thread; `-T0` = যতগুলো core আছে
`-l` / `--list`	আর্কাইভের তথ্য দেখাও
`-t`	যাচাই
`-v`	অগ্রগতি দেখাও
`-M LIMIT` / `--memlimit=`	মেমোরির সীমা (যেমন `-M 1GiB`)
`--format=lzma`	পুরোনো `.lzma` ফরম্যাট

```
$ xz -kv -T0 app.log
app.log (1/1)
  100 %      38.2 KiB / 575.1 KiB = 0.066

$ xz -l app.log.xz
Strms  Blocks   Compressed Uncompressed  Ratio  Check   Filename
    1       1     38.2 KiB    575.1 KiB  0.066  CRC64   app.log.xz
```

```
$ xzcat app.log.xz | head -2
1
2
```

সঙ্গী কমান্ড: `unxz`, `xzcat`, `xzless`, `xzmore`, `xzgrep`, `xzdiff`, আর পুরোনো `.lzma`-র জন্য `lzma`, `unlzma`, `lzcat`।

মেমোরির ব্যাপার: `xz -9` কম্প্রেস করতে প্রায় ৬৭৪ MiB, আর ডিকম্প্রেস করতে প্রায় ৬৫ MiB মেমোরি লাগে। `-6` (ডিফল্ট) কম্প্রেস করতে প্রায় ৯৪ MiB। থ্রেড যত, কম্প্রেশন-মেমোরিও প্রায় তত গুণ। ছোট VPS বা Raspberry Pi-তে `-9 -T0` চালালে মেমোরি শেষ হয়ে যেতে পারে।

> 📝 **মনে রাখো:** ২০২৪ সালের মার্চে **xz-utils backdoor (CVE-2024-3094)** ধরা পড়েছিল। কেউ কয়েক বছর ধরে বিশ্বাস অর্জন করে xz-এর 5.6.0 ও 5.6.1 সংস্করণে লুকানো কোড ঢুকিয়েছিল, যার লক্ষ্য ছিল নির্দিষ্ট পরিস্থিতিতে SSH সার্ভারে অনুপ্রবেশ। মূলত কিছু rolling ও পরীক্ষামূলক distro (Fedora Rawhide/40 beta, Debian unstable, Kali, Arch ইত্যাদি) আক্রান্ত হয়েছিল, স্থিতিশীল রিলিজ নয়। এটা একটা বিখ্যাত **supply-chain attack**। এর শিক্ষা নিয়ে অধ্যায় ৪৫-এ আরও আলোচনা আছে। বর্তমান সংস্করণগুলো নিরাপদ, তাই xz ব্যবহারে ভয়ের কিছু নেই, সিস্টেম আপডেট রাখলেই হলো।

৩৪.৭ `zstd`: আধুনিক সব-কাজের কম্প্রেসর

`zstd` (Zstandard) Facebook (Meta)-এর Yann Collet ২০১৬ সালে বানিয়েছেন। এর বিশেষত্ব: **খুব বিস্তৃত লেভেলের পরিসর।** নিচু লেভেলে এটা gzip-এর চেয়ে অনেক দ্রুত এবং প্রায়ই বেশি ছোট করে, আর উঁচু লেভেলে xz-এর কাছাকাছি অনুপাত দেয়। সবচেয়ে বড় কথা, **ডিকম্প্রেশন সব লেভেলেই খুব দ্রুত।**

কোথায় ব্যবহার হয়:

- Arch Linux প্যাকেজ (`.pkg.tar.zst`)
- Fedora-র RPM প্যাকেজের payload
- Ubuntu-র `.deb` প্যাকেজ (21.10 থেকে) ও initramfs
- Btrfs ফাইলসিস্টেমের স্বচ্ছ (transparent) কম্প্রেশন (`compress=zstd`)
- লিনাক্স কার্নেল ইমেজ ও মডিউল কম্প্রেশনের একটা বিকল্প

ইনস্টল

```
$ sudo apt install zstd       # Debian/Ubuntu (প্রায়ই আগে থেকেই থাকে)
$ sudo dnf install zstd       # Fedora/RHEL
```

অপশনের টেবিল

অপশন	কাজ
(কিছু না)	কম্প্রেস; **মূল ফাইল রেখে দেয়** (gzip-এর উল্টো)
`--rm`	কাজ শেষে মূল ফাইল মুছে দাও
`-d`	ডিকম্প্রেস (`unzstd`)
`-c`	stdout-এ
`-o FILE`	আউটপুট ফাইলের নাম
`-1` … `-19`	লেভেল, ডিফল্ট `-3`
`--ultra -20` … `-22`	আরও উঁচু লেভেল (প্রচুর মেমোরি লাগে)
`--fast=N`	ঋণাত্মক লেভেল: আরও দ্রুত, কম ছোট
`-T N`	N thread; `-T0` = সব core
`--long`	বড় জানালা (ডিফল্ট ১২৮ MB), বড় ফাইলে দূরের পুনরাবৃত্তি ধরে
`-l`	তথ্য দেখাও
`-t`	যাচাই
`-v` / `-q`	বেশি/কম বার্তা
`-b#`	নিজের মেশিনে # লেভেলে benchmark
`--train`	ছোট ছোট অনেক ফাইলের জন্য dictionary বানাও

উদাহরণ

```
$ zstd app.log
app.log              : 12.66%   (   575 KiB =>   72.8 KiB, app.log.zst)

$ ls app.log*
app.log  app.log.zst

$ zstd -19 -T0 app.log -o app19.zst
$ zstd -d app.log.zst -o restored.log
$ zstdcat app.log.zst | wc -l
100000
$ zstd -l app.log.zst
Frames  Skips  Compressed  Uncompressed  Ratio  Check  Filename
     1      0    72.8 KiB       575 KiB  7.902  XXH64  app.log.zst
```

নিজের মেশিনে লেভেল ১ থেকে ১৯ পর্যন্ত গতি বনাম অনুপাত মেপে দেখো:

```
$ zstd -b1 -e19 app.log
```

সঙ্গী কমান্ড: `unzstd`, `zstdcat`, `zstdless`, `zstdgrep`, `zstdmt` (= `zstd -T0`)।

> 💡 **টিপস:** নতুন কাজে কোন কম্প্রেসর নেবে বুঝতে না পারলে `zstd` **নাও।** দৈনন্দিন ব্যাকআপে `-3` (ডিফল্ট) বা `-T0`, আর একবার বানিয়ে বহুবার বিতরণ করবে এমন জিনিসে `-19`। শুধু যেখানে প্রাপকের মেশিনে zstd না-ও থাকতে পারে (খুব পুরোনো সার্ভার), সেখানে নিরাপদ বিকল্প `gzip`।

> ⚠️ **সতর্কতা:** `--long` দিয়ে বানানো বা `--ultra -20`-এর উপরের লেভেলের ফাইল ডিকম্প্রেস করতে বড় জানালা লাগে। প্রাপক যদি `zstd -d` চালিয়ে "Frame requires too much memory" error পায়, তাকে `zstd -d --long=31` বা `--memory=2GB` দিতে হতে পারে।

আরেকটা দ্রুতগামী: `lz4`

`lz4` (একই লেখকের) কম্প্রেশন অনুপাতে সবচেয়ে দুর্বল, কিন্তু গতিতে অবিশ্বাস্য, প্রায় মেমোরি কপির গতিতে। ব্যবহার হয় যেখানে CPU বাঁচানো বেশি জরুরি: zram (RAM-এ কম্প্রেস করা swap), কার্নেল/initramfs-এর দ্রুত বুট, ডাটাবেস।

```
$ lz4 app.log            # app.log.lz4 বানায়, মূল ফাইল রেখে দেয়
$ lz4 -d app.log.lz4 out.log
```

৩৪.৮ তুলনা টেবিল: গতি বনাম অনুপাত

নিচের টেবিলটা **আনুমানিক ও আপেক্ষিক**। আসল সংখ্যা ডেটার ধরন, CPU আর সংস্করণের উপর অনেক নির্ভর করে। ধারণা হিসেবে ধরো: প্রায় ১ GB সাধারণ টেক্সট/লগ/সোর্স কোড, একটা আধুনিক CPU core।

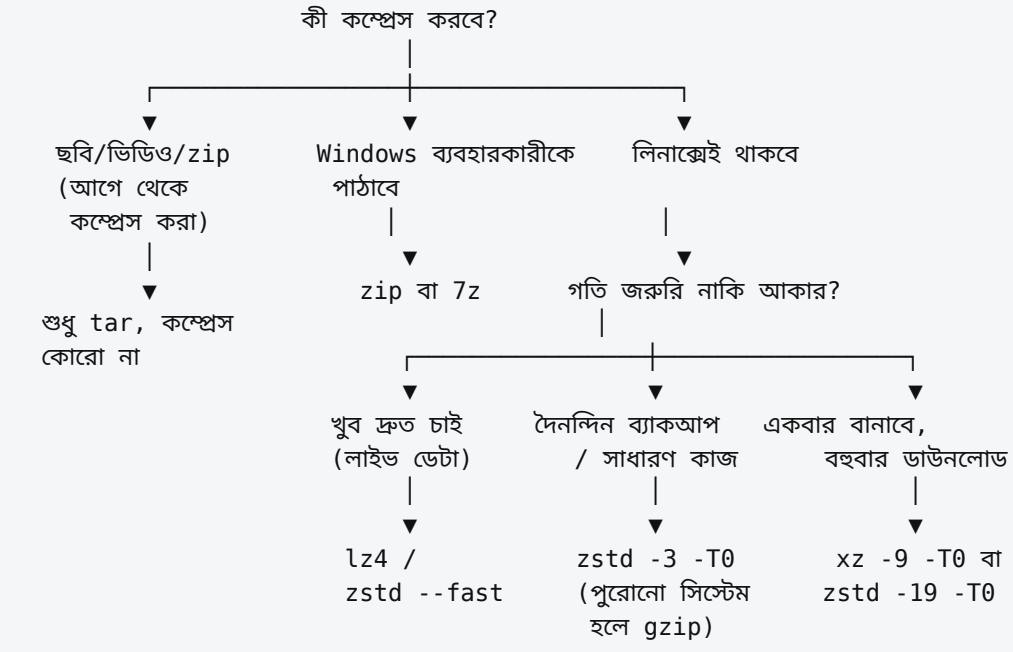
টুল ও লেভেল	কম্প্রেস গতি	ডিকম্প্রেস গতি	অনুপাত (বড় = ভালো)	মেমোরি	মন্তব্য
`lz4`	⚡⚡⚡⚡⚡ (কয়েকশো MB/s)	⚡⚡⚡⚡⚡ (GB/s)	★☆☆☆☆ (~২×)	খুব কম	গতিই সব
`zstd -1`	⚡⚡⚡⚡	⚡⚡⚡⚡⚡	★★☆☆☆ (~২.৮×)	কম	gzip-এর চেয়ে দ্রুত ও ভালো
`zstd -3` (ডিফল্ট)	⚡⚡⚡⚡	⚡⚡⚡⚡⚡	★★★☆☆ (~৩×)	কম	**সেরা সর্বজনীন বিকল্প**
`gzip -1`	⚡⚡⚡	⚡⚡⚡	★★☆☆☆ (~২.৭×)	খুব কম	
`gzip -6` (ডিফল্ট)	⚡⚡	⚡⚡⚡	★★☆☆☆ (~৩×)	খুব কম	সব জায়গায় চলে
`gzip -9`	⚡	⚡⚡⚡	★★☆☆☆ (-৬-এর চেয়ে সামান্য ভালো)	খুব কম	অতিরিক্ত সময়ের তুলনায় লাভ কম

টুল ও লেভেল	কম্প্রেস গতি	ডিকম্প্রেস গতি	অনুপাত (বড় = ভালো)	মেমোরি	মন্তব্য
`bzip2 -9` (ডিফল্ট)	⚡	🐢	★★★☆☆ (~৩.৫×)	কম	ডিকম্প্রেশন ধীর
`zstd -19`	🐢	⚡⚡⚡⚡⚡	★★★★☆ (~৩.৮×)	মাঝারি	বিতরণের জন্য চমৎকার
`xz -6` (ডিফল্ট)	🐢	⚡⚡	★★★★☆ (~৪×)	মাঝারি	
`xz -9e`	🐢🐢	⚡⚡	★★★★★	বেশি	সর্বোচ্চ ছোট

মোটামুটি নিয়ম (একই ধরনের ডেটায়):

```
অনুপাত (ছোট করার ক্ষমতা), কম থেকে বেশি:
  lz4  <  gzip  ≈  zstd(নিচু)  <  bzip2  <  zstd(উঁচু)  ≤  xz

কম্প্রেস গতি, দ্রুত থেকে ধীর:
  lz4  >  zstd(নিচু)  >  gzip  >  bzip2  >  zstd(উঁচু)  ≈  xz

ডিকম্প্রেস গতি, দ্রুত থেকে ধীর:
  lz4  >  zstd(যেকোনো লেভেল)  >  gzip  >  xz  >  bzip2
```

কখন কোনটা? সিদ্ধান্তের ছক



৩৪.৯ এক্সটেনশন চেনা ও আসল ফরম্যাট বের করা

এক্সটেনশনের তালিকা

এক্সটেনশন	কী	বানানো	খোলা
`.tar`	শুধু আর্কাইভ	`tar -cf`	`tar -xf`
`.tar.gz`, `.tgz`	tar + gzip	`tar -czf`	`tar -xzf` (বা `tar -xf`)
`.tar.bz2`, `.tbz2`, `.tbz`, `.tb2`	tar + bzip2	`tar -cjf`	`tar -xjf`
`.tar.xz`, `.txz`	tar + xz	`tar -cJf`	`tar -xJf`
`.tar.zst`, `.tzst`	tar + zstd	`tar --zstd -cf`	`tar --zstd -xf`
`.tar.lz4`	tar + lz4	`tar -I lz4 -cf`	`tar -I lz4 -xf`
`.tar.Z`, `.taz`	tar + পুরোনো `compress`		`tar -xZf`
`.tar.lzma`, `.tlz`	tar + lzma	`tar --lzma -cf`	`tar --lzma -xf`
`.gz`	একটা gzip-করা ফাইল	`gzip`	`gunzip`
`.bz2`	একটা bzip2 ফাইল	`bzip2`	`bunzip2`
`.xz`	একটা xz ফাইল	`xz`	`unxz`
`.zst`	একটা zstd ফাইল	`zstd`	`unzstd`
`.lz4`	একটা lz4 ফাইল	`lz4`	`lz4 -d`
`.Z`	পুরোনো Unix `compress`	`compress` (ncompress প্যাকেজ)	`uncompress` বা `gunzip`
`.zip`	zip আর্কাইভ	`zip -r`	`unzip`
`.7z`	7-Zip আর্কাইভ	`7z a`	`7z x`
`.rar`	RAR আর্কাইভ	`rar a`	`unrar x`
`.jar`, `.war`, `.apk`, `.docx`, `.xlsx`, `.odt`, `.epub`	ভেতরে আসলে zip		`unzip`
`.deb`	Debian প্যাকেজ (`ar` আর্কাইভ, ভেতরে tar)	`dpkg-deb`	`dpkg-deb -x`, `ar x`
`.rpm`	RPM প্যাকেজ (header + cpio payload)	`rpmbuild`	`rpm2cpio f.rpm | cpio -idmv`
`.cpio`	cpio আর্কাইভ (initramfs-এ ব্যবহৃত)	`cpio -o`	`cpio -idv`
`.iso`	CD/DVD ডিস্ক ইমেজ	`xorriso` / `genisoimage`	`mount -o loop` (অধ্যায় ৩২), `7z x`

এক্সটেনশন	কী	বানানো	খোলা
`.img`, `.img.xz`	ডিস্ক ইমেজ	`dd` (অধ্যায় ৩৫)	`xz -dc f.img.xz | dd ...`

> 💡 **টিপস:** `.docx` আসলে একটা zip! চেষ্টা করে দেখো: `unzip -l report.docx`। ভেতরে `word/document.xml` ইত্যাদি XML ফাইল পাবে।

এক্সটেনশন মিথ্যা বলতে পারে: `file` কমান্ড

লিনাক্সে ফাইলের নাম শুধু একটা লেবেল (অধ্যায় ১)। আসল ফরম্যাট বোঝা যায় ফাইলের শুরুর কয়েকটা byte, অর্থাৎ **magic number** দেখে। `file` কমান্ড ঠিক এটাই করে।

```
$ file project.tar.gz mystery.bin app.log.zst doc.docx
project.tar.gz: gzip compressed data, from Unix, original size modulo 2^32 614400
mystery.bin:    XZ compressed data, checksum CRC64
app.log.zst:    Zstandard compressed data (v0.8+), Dictionary ID: None
doc.docx:       Microsoft Word 2007+
```

ফরম্যাট	প্রথম byte-গুলো (hex)	ASCII-তে
gzip	`1f 8b`	
bzip2	`42 5a 68`	`BZh`
xz	`fd 37 7a 58 5a 00`	`.7zXZ.`
zstd	`28 b5 2f fd`	
zip	`50 4b 03 04`	`PK..` (আবিষ্কারক Phil Katz-এর নামের আদ্যক্ষর)
7z	`37 7a bc af 27 1c`	`7z....`
rar	`52 61 72 21 1a 07`	`Rar!..`
tar (ustar)	২৫৭ নম্বর byte থেকে	`ustar`

নিজে দেখো:

```
$ head -c 4 project.tar.gz | xxd
00000000: 1f8b 0800                                ....
$ head -c 4 app.log.zst | od -An -tx1
 28 b5 2f fd
```

কম্প্রেস করা tar-এর ভেতরে আসলে কী আছে দেখতে `-z` (look inside) অপশন:

```
$ file -z project.tar.gz
project.tar.gz: POSIX tar archive (GNU) (gzip compressed data, from Unix)
```

৩৪.১০ `zip` ও `unzip`: Windows-এর সাথে বন্ধুত্ব

`.zip` আর `.tar.gz`-এর গঠনগত পার্থক্য

```
.tar.gz  ("solid": পুরো স্রোত একসাথে কম্প্রেস)
┌──────────────────────────────────────────────────────┐
│ gzip( [hdr a][data a][hdr b][data b][hdr c][data c] ) │
└──────────────────────────────────────────────────────┘
  + একই ধরনের ফাইলগুলোর মধ্যের মিল কাজে লাগে → বেশি ছোট
  − একটা ফাইল বের করতে শুরু থেকে পড়তে হয়
  − মাঝখানে নষ্ট হলে তার পরের সবকিছু ঝুঁকিতে

.zip  (প্রতিটা ফাইল আলাদা কম্প্রেস, শেষে সূচিপত্র)
┌──────────┬──────────┬──────────┬──────────────────────┐
│[hdr a]   │[hdr b]   │[hdr c]   │ Central Directory    │
│deflate(a)│deflate(b)│deflate(c)│ (সব ফাইলের সূচিপত্র │
│          │          │          │  ও তাদের অবস্থান)   │
└──────────┴──────────┴──────────┴──────────────────────┘
  + সূচিপত্র শেষে → তালিকা তাৎক্ষণিক, যেকোনো ফাইল সরাসরি বের করা যায়
  + একটা ফাইল নষ্ট হলে বাকিগুলো সাধারণত টিকে যায়
  − ফাইলগুলোর মধ্যের মিল কাজে লাগে না → সাধারণত বড়
  − লিনাক্সের owner/group, ACL, কিছু বিশেষ ফাইল ঠিকমতো রাখে না
```

> 📝 **মনে রাখো:** লিনাক্স সার্ভারের ব্যাকআপ, যেখানে permission, owner, symlink সব অক্ষত রাখা জরুরি, সেখানে **tar** ব্যবহার করো। Windows/Mac ব্যবহারকারীকে ফাইল পাঠাতে **zip** ব্যবহার করো।

ইনস্টল

```
$ sudo apt install zip unzip     # Debian/Ubuntu
$ sudo dnf install zip unzip     # Fedora/RHEL
```

`zip`

```
zip [অপশন] আর্কাইভ.zip ফাইল...
```

অপশন	কাজ
`-r`	ফোল্ডারের ভেতরে recursive ঢোকো (**এটা না দিলে ফোল্ডারের ভেতরের ফাইল যাবে না!**)
`-0` ... `-9`	লেভেল; `-0` = শুধু রাখো, কম্প্রেস নয়; ডিফল্ট `-6`
`-e`	পাসওয়ার্ড দিয়ে এনক্রিপ্ট (প্রম্পটে জিজ্ঞেস করবে)
`-P pass`	কমান্ড লাইনেই পাসওয়ার্ড (**অনিরাপদ**, history-তে থাকে)
`-x PATTERN`	বাদ দাও
`-u`	শুধু নতুন/বদলানো ফাইল আপডেট

অপশন	কাজ
`-d`	আর্কাইভ থেকে মুছে দাও
`-m`	zip-এ ঢোকানোর পর মূল ফাইল মুছে দাও (move)
`-q`	চুপচাপ
`-y`	symlink-কে symlink হিসেবেই রাখো
`-j`	ফোল্ডারের path বাদ দিয়ে শুধু ফাইলের নাম রাখো
`-s SIZE`	আর্কাইভকে ভাগ করো (যেমন `-s 100m`)
`-T`	বানানোর পর যাচাই

```
$ zip -r project.zip project/
  adding: project/ (stored 0%)
  adding: project/src/ (stored 0%)
  adding: project/src/main.c (stored 0%)
  adding: project/docs/ (stored 0%)
  adding: project/docs/README.md (stored 0%)
  adding: project/logs/ (stored 0%)
  adding: project/logs/app.log (deflated 79%)

$ zip -r site.zip website/ -x '*.git*' -x '*/node_modules/*'
$ zip -e secret.zip passwords.txt
Enter password:
Verify password:
  adding: passwords.txt (deflated 12%)
```

> ⚠️ **সতর্কতা:** `zip -e`-এর ডিফল্ট এনক্রিপশন (ZipCrypto) **খুব দুর্বল**, সহজেই ভাঙা যায়। সত্যিকারের গোপন জিনিসের জন্য `7z -p -mhe=on` (AES-256) বা `gpg` ব্যবহার করো (অধ্যায় ৪৫)।

`unzip`

অপশন	কাজ
`-l`	তালিকা দেখাও
`-v`	বিস্তারিত তালিকা (অনুপাত, CRC সহ)
`-d DIR`	DIR-এ extract (না থাকলে বানিয়ে নেয়)
`-o`	জিজ্ঞেস না করে ওভাররাইট
`-n`	কখনো ওভাররাইট কোরো না
`-t`	যাচাই
`-q`	চুপচাপ
`-j`	ফোল্ডার কাঠামো বাদ দিয়ে সব ফাইল এক জায়গায়
`-p`	stdout-এ (পাইপের জন্য)

অপশন	কাজ
`-P pass`	পাসওয়ার্ড
`-x PATTERN`	বাদ দিয়ে extract
`-O CHARSET`	ফাইলনামের এনকোডিং (Windows থেকে আসা অ-ইংরেজি নামের জন্য, Debian/Ubuntu-র patched unzip-এ)

```
$ unzip -l project.zip
Archive:  project.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-11-12 10:04   project/
        0  2026-11-12 10:04   project/src/
       22  2026-11-12 10:04   project/src/main.c
        0  2026-11-12 10:04   project/docs/
       13  2026-11-12 10:04   project/docs/README.md
        0  2026-11-12 10:04   project/logs/
   588895  2026-11-12 10:04   project/logs/app.log
---------                     -------
   588930                     7 files

$ unzip project.zip -d /tmp/unz
$ unzip -o project.zip 'project/src/*'    # শুধু src অংশ, ওভাররাইট
$ unzip -p project.zip project/docs/README.md
# My project
```

ফাইল আগে থেকে থাকলে unzip জিজ্ঞেস করে:

```
replace project/src/main.c? [y]es, [n]o, [A]ll, [N]one, [r]ename:
```

সঙ্গী কমান্ড: `zipinfo` (বিস্তারিত তথ্য), `zipgrep` (zip-এর ভেতরে grep), `zipsplit`, `zipnote`।

০৪.১১ `7z` ও `rar`

`7z` : 7-Zip

7-Zip-এর ফরম্যাট `.7z` সাধারণত LZMA/LZMA2 ব্যবহার করে, "solid" কম্প্রেশন করে, আর **AES-256 এনক্রিপশন** দেয়, যেটা ফাইলের নামও লুকাতে পারে। এর বড় সুবিধা: একটা টুল দিয়েই প্রায় সব ফরম্যাট খোলা যায় (zip, tar, gz, xz, iso, rar-এর অনেক সংস্করণ, cab, msi...)।

ইনস্টল (distro অনুযায়ী আলাদা):

Distro	প্যাকেজ	কমান্ড
Ubuntu 24.04 / Debian 12	`p7zip-full` (পুরোনো ঐতিহ্যবাহী port)	`7z`, `7za`

Distro	প্যাকেজ	কমান্ড
Ubuntu 24.04 / Debian 12	`7zip` (মূল লেখকের অফিসিয়াল লিনাক্স সংস্করণ)	`7zz` (নতুন সংস্করণে `7z` নামেও)
Fedora	`7zip` (নতুন রিলিজে) বা `p7zip p7zip-plugins`	`7z` বা `7zz`
RHEL 9 / Rocky / Alma	EPEL repository থেকে `p7zip p7zip-plugins`	`7z`

> 📝 **মনে রাখো:** প্যাকেজের নাম ও কমান্ডের নাম সময়ের সাথে বদলাচ্ছে (`p7zip` থেকে অফিসিয়াল `7zip` -এর দিকে)। ইনস্টলের পর `which 7z 7za 7zz` চালিয়ে দেখো কোনটা আছে। নিচের উদাহরণে `7z` লিখেছি; তোমার সিস্টেমে শুধু `7zz` থাকলে সেটাই লেখো, অপশন একই।

7z-এর গঠন অন্যদের থেকে আলাদা: প্রথমে একটা **কমান্ড অক্ষর** (ড্যাশ ছাড়া), তারপর সুইচ (`-` দিয়ে)।

কমান্ড	কাজ
`a`	add: আর্কাইভ বানাও/যোগ করো
`x`	extract **পূর্ণ path সহ** (সাধারণত এটাই চাও)
`e`	extract **সব ফাইল এক ফোল্ডারে**, কাঠামো বাদ দিয়ে
`l`	তালিকা
`t`	যাচাই
`d`	মুছে দাও
`u`	আপডেট

সুইচ	কাজ
`-p`	পাসওয়ার্ড (শুধু `-p` দিলে প্রম্পটে জিজ্ঞেস করবে, যেটা নিরাপদ)
`-mhe=on`	ফাইলের নামও এনক্রিপ্ট করো (header encryption, শুধু `.7z` -এ)
`-mx=N`	লেভেল ০–৯ (`-mx=9` = ultra)
`-o DIR`	আউটপুট ফোল্ডার (**`-o` আর DIR-এর মাঝে কোনো স্পেস নেই**: `-o/tmp/out`)
`-t TYPE`	ফরম্যাট (`-tzip` , `-t7z` , `-ttar`)
`-v SIZE`	ভলিউমে ভাগ করো (`-v100m`)
`-x!PATTERN`	বাদ দাও
`-y`	সব প্রশ্নে হ্যাঁ

```
$ 7z a project.7z project/
$ 7z a -p -mhe=on -mx=9 secret.7z private/
Enter password:
Verify password:

$ 7z l project.7z
$ 7z x project.7z -o/tmp/out
$ 7z x anything.iso -o./iso-contents     # ISO-ও খোলা যায়
```

```
$ 7z t project.7z
Everything is Ok
```

> ⚠️ **সতর্কতা:** `.7z` ফরম্যাট লিনাক্সের owner/group ও পুরো permission তথ্য রাখে না। সার্ভার ব্যাকআপের জন্য তাই আগে `tar` করে, তারপর দরকার হলে সেই `.tar`-কে 7z করো: `tar -cf - dir | 7z a -si dir.tar.7z`।

`rar` ও `unrar`

RAR একটা **proprietary (মালিকানাধীন)** ফরম্যাট। Windows-এ WinRAR জনপ্রিয় বলে প্রায়ই `.rar` ফাইল পাবে।

কাজ	টুল	ইনস্টল
খোলা (অফিসিয়াল)	`unrar`	Ubuntu: `multiverse` repo থেকে `sudo apt install unrar`; Debian: `non-free` থেকে; Fedora: RPM Fusion (nonfree) থেকে
খোলা (মুক্ত বিকল্প)	`unar`, `lsar`	`sudo apt install unar` / `sudo dnf install unar`
খোলা (7-Zip দিয়ে)	`7z x f.rar`	প্লাগইন/সংস্করণ অনুযায়ী RAR5 সমর্থন ভিন্ন হতে পারে
বানানো	`rar`	শেয়ারওয়্যার, RARLAB থেকে; মুক্ত সফটওয়্যারে রার বানানোর কোনো উপায় নেই

```
$ unrar l movie.rar          # তালিকা
$ unrar x movie.rar          # পূর্ণ path সহ extract
$ unrar e movie.rar          # সব এক ফোল্ডারে
$ unrar t movie.rar          # যাচাই
$ unrar x movie.part1.rar    # বহু-খণ্ড আর্কাইভ: প্রথম খণ্ড দিলেই বাকিগুলো নিজে নেয়

$ unar movie.rar             # মুক্ত বিকল্প
$ lsar movie.rar
```

> 💡 **টিপস:** অন্যকে দেওয়ার জন্য নিজে কখনো `.rar` বানিয়ো না। মুক্ত ও সব জায়গায় খোলা যায় এমন ফরম্যাট বেছে নাও: `.zip`, `.7z`, বা লিনাক্স ব্যবহারকারীর জন্য `.tar.gz` / `.tar.zst`।

`zip` বনাম `7z` বনাম `rar` বনাম `tar.*`

বৈশিষ্ট্য	`.tar.gz` / `.tar.zst` / `.tar.xz`	`.zip`	`.7z`	`.rar`
লাইসেন্স	মুক্ত	মুক্ত	মুক্ত (LGPL)	মালিকানাধীন

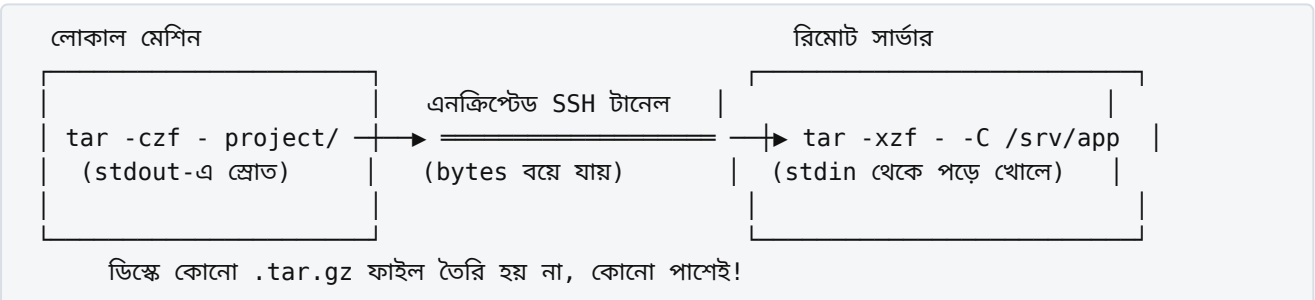
বৈশিষ্ট্য	`.tar.gz` / `.tar.zst` / `.tar.xz`	`.zip`	`.7z`	`.rar`
লিনাক্স permission/owner/ symlink	✅ পুরোপুরি	⚠️ আংশিক (permission ও symlink, owner নয়)	❌	❌
solid কম্প্রেশন	✅ সবসময়	❌	✅ (ডিফল্ট)	ঐচ্ছিক
দ্রুত তালিকা / random access	❌	✅	আংশিক	✅
শক্তিশালী এনক্রিপশন	❌ (gpg লাগাও)	⚠️ ঐচ্ছিক AES, ডিফল্ট দুর্বল	✅ AES-256	✅ AES
Windows-এ বিল্ট-ইন খোলা	Windows 11-এ সম্প্রতি এসেছে	✅	Windows 11-এ সম্প্রতি এসেছে	Windows 11-এ সম্প্রতি এসেছে
পাইপ/স্ট্রিম	✅ চমৎকার	⚠️ সীমিত	⚠️ সীমিত	❌

৩৪.১২ পাইপ দিয়ে `ssh`-এর ভেতর দিয়ে `tar`

এই অধ্যায়ের সবচেয়ে শক্তিশালী কৌশল। SSH নিয়ে বিস্তারিত অধ্যায় ৩৮-এ আছে, এখানে শুধু tar-এর সাথে এর জুটিটা দেখব।

মূল ধারণা

`tar -f -` মানে "আর্কাইভ ফাইলে না লিখে **stdout-এ** লেখো" (`-c`-এর সাথে) বা "**stdin থেকে** পড়ো" (`-x`-এর সাথে)। আর `ssh host 'কমান্ড'` রিমোট মেশিনে কমান্ড চালায়, আর আমাদের stdin সেখানে পাঠিয়ে দেয়। দুটো জুড়লে:



কেন `scp -r` বা `rsync`-এর বদলে এটা?

- **কোনো মধ্যবর্তী ফাইল লাগে না।** ১০০ GB ফোল্ডার পাঠাতে লোকালে ১০০ GB খালি জায়গা দরকার নেই।
- **হাজার হাজার ছোট ফাইলে দ্রুত।** `scp` প্রতিটা ফাইলের জন্য আলাদা আদান-প্রদান করে; tar সব এক স্রোতে পাঠায়।
- **permission, owner, symlink, timestamp অক্ষত থাকে।**
- পথে **কম্প্রেস** হয়, তাই ধীর নেটওয়ার্কে লাভ।

(বারবার সিঙ্ক করার জন্য অবশ্য `rsync` সেরা, কারণ শুধু পরিবর্তনটুকু পাঠায়। দেখো অধ্যায় ৩৫।)

১. লোকাল থেকে রিমোটে পাঠানো (push)

```
$ tar -czf - project/ | ssh rahim@server.example.com 'tar -xzf - -C /home/rahim/'
```

রিমোটে আগে ফোল্ডার বানিয়ে নিয়ে:

```
$ tar -czf - project/ | ssh rahim@server 'mkdir -p /srv/app && tar -xzf - -C /srv/app'
```

২. রিমোট থেকে লোকালে আনা (pull)

```
$ ssh rahim@server 'tar -czf - -C /var/www html' | tar -xzf - -C ~/site-copy/
```

৩. রিমোট ফোল্ডারের ব্যাকআপ সরাসরি লোকাল ফাইলে

```
$ ssh rahim@server 'tar -czf - /etc' > server-etc-$(date +%F).tar.gz
tar: Removing leading `/' from member names
```

(বার্তাটা রিমোট tar-এর stderr থেকে আসছে, ব্যাকআপ ঠিকই আছে।)

উল্টোটা, লোকাল ব্যাকআপ সরাসরি রিমোট ফাইলে:

```
$ tar -czf - ~/documents | ssh rahim@backup-server 'cat > /backups/docs.tar.gz'
```

৪. দ্রুত নেটওয়ার্কে কম্প্রেশন ছাড়া, বা zstd দিয়ে

LAN-এ (১ Gbps বা বেশি) gzip-এর একক-core গতিই প্রায়ই বাধা হয়ে দাঁড়ায়। তখন:

```
# কম্প্রেশন ছাড়া (দ্রুত LAN-এ)
$ tar -cf - bigdir/ | ssh server 'tar -xf - -C /data'

# zstd দিয়ে, সব core ব্যবহার করে (দুই প্রান্তেই zstd থাকতে হবে)
$ tar -cf - bigdir/ | zstd -T0 -3 | ssh server 'zstd -d | tar -xf - -C /data'

# অগ্রগতি দেখে
$ tar -cf - bigdir/ | pv | zstd -T0 | ssh server 'zstd -d | tar -xf - -C /data'
```

> 💡 **টিপস:** `ssh -C` দিয়েও SSH নিজেই কম্প্রেস করতে পারে (zlib/gzip-এর মতো)। কিন্তু `tar -z` আর `ssh -C` **দুটো একসাথে দিয়ো না**। ডেটা দুবার কম্প্রেস করলে লাভ নেই, শুধু CPU নষ্ট।

৫. root-এর ফাইল, owner অক্ষত রেখে

```
$ sudo tar -cpf - -C / etc/nginx | ssh admin@server 'sudo tar -xpf - -C /'
```

> ⚠️ **সতর্কতা:** রিমোটে `sudo` পাসওয়ার্ড চাইলে এটা ঝামেলা করবে, কারণ stdin ইতিমধ্যে tar-এর ডেটা বইছে। সমাধান: রিমোটে ওই কমান্ডের জন্য NOPASSWD sudo নিয়ম (অধ্যায় ১৮), অথবা আগে লোকাল ফাইলে বানিয়ে `scp` করে তারপর রিমোটে খোলা। আর **`-C /`-তে extract করার আগে দুবার ভাবো**, কারণ সেটা রিমোটের আসল সিস্টেম ফাইল ওভাররাইট করবে।

৬. একই মেশিনে এক ডিস্ক থেকে আরেক ডিস্কে

ssh ছাড়াও এই পাইপ কাজের:

```
$ sudo tar -cf - -C /old/disk . | sudo tar -xpf - -C /new/disk
```

এতে hard link, symlink, permission, owner সব অক্ষত থেকে কপি হয় (আজকাল `cp -a` বা `rsync -aHAX` দিয়েও করা যায়)।

৩৪.১৩ বড় আর্কাইভ ভাগ করা ও জোড়া লাগানো

FAT32 পেনড্রাইভে ৪ GB-এর বড় ফাইল রাখা যায় না, ইমেইলে বড় ফাইল পাঠানো যায় না। তখন `split` (অধ্যায় ২৩-এর কাছাকাছি টুল):

```
$ tar -czf - bigdir/ | split -b 1G -d - backup.tar.gz.part
$ ls
backup.tar.gz.part00  backup.tar.gz.part01  backup.tar.gz.part02

# জোড়া লাগিয়ে খোলা
$ cat backup.tar.gz.part* | tar -xzf -
```

`-b 1G` = প্রতি খণ্ড ১ GiB, `-d` = সংখ্যাযুক্ত প্রত্যয়, `-` = stdin থেকে পড়ো।

৩৪.১৪ নিরাপত্তা ও সাধারণ ভুল

Tarbomb: বোমার মতো ছড়িয়ে পড়া আর্কাইভ

কেউ যদি ফোল্ডারে না রেখে সরাসরি একশোটা ফাইল আর্কাইভ করে, তুমি সেটা তোমার home ফোল্ডারে extract করলে একশোটা ফাইল ছড়িয়ে পড়বে, তোমার নিজের ফাইলের সাথে মিশে যাবে, এমনকি একই নামের ফাইল

ওভাররাইট করবে।

প্রতিরোধ:

```
$ tar -tf unknown.tar.gz | head        # আগে দেখো
$ mkdir unknown && tar -xf unknown.tar.gz -C unknown   # আলাদা ফোল্ডারে খোলো
```

নিজে আর্কাইভ বানানোর সময়ও ভদ্রতা বজায় রাখো: সবকিছু এক মূল ফোল্ডারের ভেতরে রাখো (`project-1.0/...`)।

Path traversal ও অবিশ্বস্ত আর্কাইভ

বিদ্বেষপূর্ণ আর্কাইভে `../../.bashrc` ধরনের path থাকতে পারে, যাতে extract করলে লক্ষ্য ফোল্ডারের **বাইরে** ফাইল লেখা হয়। একে বলে **"Zip Slip"**।

- GNU tar ডিফল্টে শুরুর `/` আর `..` অংশ সরিয়ে দেয় (`tar: Removing leading '../' from member names`)। **`-P` বা `--absolute-names` দিলে এই সুরক্ষা বন্ধ হয়ে যায়**, তাই অবিশ্বস্ত আর্কাইভে কখনো দিয়ো না।
- symlink দিয়েও কৌশল করা যায় (আগে একটা symlink, পরে সেই পথে ফাইল)। অবিশ্বস্ত আর্কাইভ **কখনো `sudo` দিয়ে খুলো না**, আর আলাদা খালি ফোল্ডারে খোলো।
- `unzip`-এর আধুনিক সংস্করণ `../` ধরনের path আটকায় ও সতর্কবার্তা দেয়।

Zip bomb / decompression bomb

কয়েক KB-এর একটা ফাইল ডিকম্প্রেস করলে কয়েকশো GB হয়ে ডিস্ক ভরিয়ে দিতে পারে। অজানা ফাইল খোলার আগে আকার দেখে নাও (`gzip -l` , `xz -l` , `unzip -l` , `zstd -l`)।

সাধারণ ভুল ও সমাধান

ভুল	কী হয়	সঠিক
`tar -cvf dir/`	`dir/`-কে আর্কাইভের নাম ধরে, "Cowardly refusing to create an empty archive"	`tar -cvf dir.tar dir/`
`tar -cfv x.tar dir`	আর্কাইভের নাম হয় `v`	`tar -cvf x.tar dir`
`tar -xzf x.tar.xz`	`-z` বলে দেওয়ায় tar gzip চালায়, আর "not in gzip format" error আসে	`tar -xJf` বা শুধু `tar -xf` (তখন tar নিজেই চিনে নেয়)
`gzip folder/`	"is a directory -- ignored"	`tar -czf folder.tar.gz folder/`
`gzip file` চালিয়ে মূল ফাইল খোঁজা	মূল ফাইল মুছে গেছে	`gzip -k file`
`zip x.zip folder/`	শুধু খালি ফোল্ডার যায়	`zip -r x.zip folder/`

ভুল	কী হয়	সঠিক
`tar -czf backup.tar.gz .` চালানো ঠিক সেই ফোল্ডারে যেখানে backup.tar.gz তৈরি হচ্ছে	"file changed as we read it" সতর্কবার্তা, আর্কাইভ নিজের ভেতরে নিজেকে নিতে চায়	আর্কাইভ অন্য ফোল্ডারে বানাও, বা `--exclude=backup.tar.gz`
`-C` এর ফোল্ডার নেই	"Cannot open: No such file or directory"	আগে `mkdir -p`

> 📝 **মনে রাখো:** tar-এর exit code: `0` = সফল, `1` = কিছু ফাইল পড়ার সময় বদলে গেছে (যেমন চলমান লগ ফাইল; আর্কাইভ সাধারণত ব্যবহারযোগ্য), `2` = গুরুতর error। ব্যাকআপ স্ক্রিপ্টে (অধ্যায় ৪১) `$?` চেক করার সময় এটা কাজে লাগে।

ব্যাকআপ যাচাই করার অভ্যাস

যে ব্যাকআপ খুলে দেখা হয়নি, তা ব্যাকআপ নয়, শুধু আশা।

```
$ gzip -t backup.tar.gz && tar -tzf backup.tar.gz > /dev/null && echo "OK"
OK
$ sha256sum backup.tar.gz > backup.tar.gz.sha256
$ sha256sum -c backup.tar.gz.sha256
backup.tar.gz: OK
```

৩৪.১৫ GUI ও অন্যান্য টুকিটাকি

- **GUI:** GNOME-এ Files (Nautilus) অ্যাপে ফাইলে right-click করে "Compress..." ও "Extract Here" পাওয়া যায় (ভেতরে File Roller/Archive Manager বা GNOME-এর নিজস্ব autoar লাইব্রেরি)। KDE-তে **Ark**। (অধ্যায় ৪)
- **`cpio`:** আরেকটা পুরোনো আর্কাইভার, যেটা stdin থেকে ফাইলের নামের তালিকা নেয়। initramfs (অধ্যায় ২৭) ও RPM-এর ভেতরে এটা ব্যবহার হয়।

  ```
  $ find project | cpio -ov > project.cpio
  $ cpio -idv < project.cpio
  ```

- **`ar`:** `.deb` প্যাকেজ ও স্ট্যাটিক লাইব্রেরি (`.a`) এর ফরম্যাট।

  ```
  $ ar t some-package.deb
  debian-binary
  control.tar.zst
  data.tar.zst
  ```

(পুরোনো বা Debian-এর প্যাকেজে `control.tar.xz`, `data.tar.xz` দেখতে পারো।) প্যাকেজ নিয়ে বিস্তারিত অধ্যায় ৩০-এ।

- **ফাইলসিস্টেম-স্তরের কম্প্রেশন:** Btrfs-এ `compress=zstd` মাউন্ট অপশন দিলে সব ফাইল স্বচ্ছভাবে কম্প্রেস হয়, তোমাকে কিছুই করতে হয় না (অধ্যায় ৩২)।

কীবোর্ড শর্টকাট ও দ্রুত কৌশল

আর্কাইভ টুলগুলোর নিজস্ব কীবোর্ড শর্টকাট কম, তবে কাজের সময় এগুলো লাগবে:

শর্টকাট / কৌশল	কাজ
`Ctrl + C`	চলমান tar/gzip/xz বন্ধ করো (অর্ধেক তৈরি আর্কাইভ মুছে দিয়ো, ওটা অসম্পূর্ণ)
`Ctrl + Z`, তারপর `bg`	লম্বা কম্প্রেশন ব্যাকগ্রাউন্ডে পাঠাও (অধ্যায় ২৫)
`Tab`	ফাইলনাম সম্পূর্ণ করো; bash-completion থাকলে `tar -xf <Tab>` শুধু আর্কাইভ ফাইলই দেখায়
`Alt + .`	আগের কমান্ডের শেষ argument বসাও: `tar -tzf x.tar.gz` চালিয়ে তারপর `tar -xzf` লিখে `Alt+.` (অধ্যায় ৬)
`!$`	আগের কমান্ডের শেষ argument (একই কাজ, history expansion)
`^-t^-x`	আগের কমান্ডের প্রথম `-t`-কে `-x` দিয়ে বদলে চালাও: `tar -tzf a.tar.gz` → `tar -xzf a.tar.gz`। (শুধু `^t^x` লিখলে প্রথম মিল `tar`-এর `t` বদলে `xar` হয়ে যাবে!)
`zless` / `xzless`-এর ভেতরে `/শব্দ`, `n`, `q`	খোঁজা, পরের মিল, বের হওয়া (`less`-এর মতোই, অধ্যায় ১০)
unzip প্রম্পটে `y` / `n` / `A` / `N` / `r`	হ্যাঁ / না / সবগুলোতে হ্যাঁ / সবগুলোতে না / নতুন নাম
`7z`-এর প্রম্পটে `y` / `n` / `a` / `s` / `u` / `q`	হ্যাঁ / না / সব হ্যাঁ / সব এড়াও / অটো-রিনেম / বের হও

হাতে-কলমে অনুশীলন

ল্যাব ১: চার কম্প্রেসরের প্রতিযোগিতা

```
$ mkdir -p ~/lab34/race && cd ~/lab34/race
$ cp -r /usr/share/doc ./docs 2>/dev/null     # অনেক টেক্সট ফাইল
$ du -sh docs
$ tar -cf docs.tar docs
$ ls -lh docs.tar

$ time gzip  -k docs.tar
$ time bzip2 -k docs.tar
```

```
$ time xz    -k -T0 docs.tar
$ time zstd  -T0 docs.tar
$ time zstd  -19 -T0 docs.tar -o docs.tar.19.zst

$ ls -lhS docs.tar*
```

প্রশ্নের উত্তর খাতায় লেখো: কোনটা সবচেয়ে ছোট? কোনটা সবচেয়ে দ্রুত? এবার ডিকম্প্রেশনের সময় মাপো:

```
$ time zcat  docs.tar.gz  > /dev/null
$ time bzcat docs.tar.bz2 > /dev/null
$ time xzcat docs.tar.xz  > /dev/null
$ time zstdcat docs.tar.zst > /dev/null
```

`real` সময়ের তুলনা করে ৩৪.৮-এর টেবিলের সাথে মেলাও।

ল্যাব ২: tar-এর মোড ও `-C`

```
$ cd ~/lab34
$ tar -czf /tmp/p.tar.gz -C ~/lab34 project      # শুধু "project/" path নিয়ে
$ tar -tzf /tmp/p.tar.gz                          # যাচাই: path "project/..." দিয়ে শুরু?
$ mkdir -p /tmp/r1 /tmp/r2
$ tar -xzf /tmp/p.tar.gz -C /tmp/r1               # পুরোটা
$ tar -xzf /tmp/p.tar.gz -C /tmp/r2 --strip-components=1   # এক স্তর বাদ
$ ls /tmp/r1 /tmp/r2
$ tar -xzOf /tmp/p.tar.gz project/src/main.c      # স্ক্রিনে দেখাও
```

ল্যাব ৩: বাদ দেওয়া ও তালিকা থেকে নেওয়া

```
$ cd ~/lab34
$ mkdir -p project/node_modules/lib && touch project/node_modules/lib/x.js
project/tmp.tmp
$ tar -czf clean.tar.gz --exclude='node_modules' --exclude='*.tmp' project/
$ tar -tzf clean.tar.gz | grep -E 'node_modules|\.tmp' || echo "বাদ পড়েছে ✓"
$ find project -name '*.c' -print0 | tar --null -czf csrc.tar.gz -T -
$ tar -tzf csrc.tar.gz
```

ল্যাব ৪: gzip-পরিবার দিয়ে লগ বিশ্লেষণ

```
$ cd ~/lab34
$ for i in 1 2 3; do seq 1 5000 | sed "s/^/day$i line /" > log$i.txt; done
$ echo "day2 ERROR disk full" >> log2.txt
$ gzip log1.txt log2.txt log3.txt
$ ls
$ zgrep -l ERROR log*.gz          # কোন ফাইলে ERROR?
$ zcat log*.gz | wc -l            # মোট কত লাইন?
$ gzip -l log2.txt.gz
$ gunzip log2.txt.gz && ls
```

ল্যাব ৫: zip ও Windows-বন্ধু আর্কাইভ

```
$ zip -r share.zip project -x '*/node_modules/*'
$ unzip -l share.zip
$ mkdir /tmp/z && unzip -q share.zip -d /tmp/z && ls /tmp/z/project
$ zip -d share.zip 'project/tmp.tmp'    # একটা ফাইল মুছে দাও
$ unzip -t share.zip
```

ল্যাব ৬: পাইপ দিয়ে "রিমোট" কপি (নিজের মেশিনেই)

রিমোট সার্ভার না থাকলে `localhost`-এ SSH দিয়ে অনুশীলন করো (`openssh-server` চালু থাকতে হবে, অধ্যায় ৩৮), অথবা ssh ছাড়াই পাইপটা বোঝো:

```
# ssh ছাড়া: এক ফোল্ডার থেকে আরেক ফোল্ডারে স্রোত
$ mkdir -p /tmp/dest
$ tar -czf - -C ~/lab34 project | tar -xzf - -C /tmp/dest
$ diff -r ~/lab34/project /tmp/dest/project && echo "হুবহু এক ✓"

# ssh দিয়ে (যদি localhost-এ ssh চালু থাকে)
$ tar -czf - -C ~/lab34 project | ssh localhost 'mkdir -p /tmp/sshdest && tar -xzf - -C
/tmp/sshdest'
```

ল্যাব ৭: `file` দিয়ে ছদ্মবেশী ফাইল ধরা

```
$ cp ~/lab34/race/docs.tar.xz ~/lab34/photo.jpg
$ file ~/lab34/photo.jpg
/home/rahim/lab34/photo.jpg: XZ compressed data, checksum CRC64
$ tar -tf ~/lab34/photo.jpg | head -3     # নাম .jpg হলেও tar ঠিকই খোলে
```

ল্যাব ৮: ভাগ করা ব্যাকআপ ও যাচাই

```
$ cd ~/lab34/race
$ tar -cf - docs | zstd -T0 | split -b 5M -d - docs.tzst.part
$ ls -lh docs.tzst.part*
$ cat docs.tzst.part* | zstd -d | tar -tf - | wc -l
$ cat docs.tzst.part* | sha256sum
```

পরিষ্কার করতে: `rm -rf ~/lab34 /tmp/r1 /tmp/r2 /tmp/dest /tmp/z /tmp/p.tar.gz`

সারসংক্ষেপ

কাজ	কমান্ড
tar আর্কাইভ বানাও	`tar -cvf a.tar dir/`
tar.gz বানাও	`tar -czvf a.tar.gz dir/`

কাজ	কমান্ড
tar.bz2 / tar.xz / tar.zst	`tar -cjf` / `tar -cJf` / `tar --zstd -cf`
এক্সটেনশন দেখে স্বয়ংক্রিয়	`tar -caf a.tar.xz dir/`
নিজের কম্প্রেসর	`tar -I 'zstd -19 -T0' -cf a.tar.zst dir/`
তালিকা দেখাও	`tar -tvf a.tar.gz`
খোলো	`tar -xvf a.tar.gz`
নির্দিষ্ট ফোল্ডারে খোলো	`tar -xf a.tar.gz -C /path`
এক স্তর path বাদ	`tar -xf a.tar.gz --strip-components=1`
বাদ দিয়ে বানাও	`tar -czf a.tar.gz --exclude='*.log' dir/`
path ছোট রাখো	`tar -czf a.tar.gz -C /parent dir`
gzip / খোলা	`gzip -k f` / `gunzip f.gz` (বা `gzip -d`)
খোলা ছাড়া পড়া / খোঁজা	`zcat`, `zless`, `zgrep` (bz/xz/zstd-এরও আছে)
bzip2	`bzip2 -k f`, `bunzip2`, `bzcat`
xz	`xz -k -T0 f`, `unxz`, `xzcat`, `xz -l`
zstd	`zstd f` (মূল থাকে), `zstd -19 -T0`, `unzstd`, `zstdcat`
তথ্য দেখো	`gzip -l`, `xz -l`, `zstd -l`
যাচাই	`gzip -t`, `bzip2 -t`, `xz -t`, `zstd -t`, `unzip -t`, `7z t`
zip	`zip -r a.zip dir/`, `unzip -l a.zip`, `unzip a.zip -d dir`
7z	`7z a a.7z dir/`, `7z x a.7z -o/out`, `7z a -p -mhe=on s.7z dir/`
rar খোলা	`unrar x a.rar`, বা `unar a.rar`
আসল ফরম্যাট	`file f`, `file -z f`
রিমোটে পাঠাও	`tar -czf - dir | ssh host 'tar -xzf - -C /dst'`
রিমোট থেকে আনো	`ssh host 'tar -czf - -C /src dir' | tar -xzf -`
ভাগ করো / জোড়া দাও	`... | split -b 1G -d - a.part` / `cat a.part* | tar -xzf -`
অগ্রগতি	`tar -cf - dir | pv | gzip > a.tar.gz`
multi-core	`pigz`, `xz -T0`, `zstd -T0`, `pbzip2`

tar মনে রাখার সূত্র: `c` = বানাও, `x` = খোলো, `t` = তালিকা; `z` = gzip, `j` = bzip2, `J` = xz; `v` = বলতে বলতে; `f` = ফাইল (সবসময় শেষে, তার পরেই নাম)।

নিজেকে যাচাই করো

1. Archive আর compression-এর মধ্যে মূল পার্থক্য কী? লিনাক্সে কেন `gzip` দিয়ে সরাসরি একটা ফোল্ডার কম্প্রেস করা যায় না?
2. `tar -cfv backup.tar /home/rahim` চালালে কী ভুল হবে? সঠিক কমান্ড লেখো।
3. `/var/www/html` ফোল্ডারটাকে এমনভাবে `site.tar.gz` বানাও যাতে আর্কাইভের ভেতরের path `html/...` দিয়ে শুরু হয়, `var/www/html/...` দিয়ে নয়।
4. একটা `.tar.gz` আর্কাইভে `-r` দিয়ে নতুন ফাইল যোগ করা যায় না কেন?
5. `gzip report.txt` চালানোর পর `report.txt` আর পাওয়া যাচ্ছে না। কেন? মূল ফাইল রেখে কম্প্রেস করার দুটো উপায় বলো।
6. প্রতিদিন রাতে ৫০ GB ডাটাবেস ডাম্প কম্প্রেস করতে হবে, দ্রুত শেষ হওয়া দরকার, আর মাঝে মাঝে দ্রুত খুলতেও হবে। কোন টুল ও অপশন নেবে, এবং কেন?
7. একটা ফাইলের নাম `data.zip`, কিন্তু `unzip` বলছে এটা zip নয়। আসল ফরম্যাট কীভাবে বের করবে?
8. Windows ব্যবহারকারী বন্ধুকে একটা ফোল্ডার পাঠাবে, আর লিনাক্স সার্ভারের `/etc` ফোল্ডারের ব্যাকআপ নেবে। দুটোর জন্য কোন ফরম্যাট বেছে নেবে, এবং কেন?
9. লোকাল `~/photos` ফোল্ডারটা কোনো মধ্যবর্তী ফাইল না বানিয়ে `user@nas` সার্ভারের `/backup` ফোল্ডারে কপি করার এক লাইনের কমান্ড লেখো। ছবি বলে কম্প্রেস করা উচিত কি?
10. ইন্টারনেট থেকে ডাউনলোড করা অজানা `tool.tar.gz` নিরাপদে খোলার আগে কোন কোন ধাপ নেবে?

▼ উত্তর দেখো

1. Archive অনেক ফাইল ও তাদের metadata (permission, owner, সময়, symlink) এক ফাইলে বাঁধে, আকার প্রায় একই থাকে। Compression একটা byte-স্রোতের আকার ছোট করে। `gzip` শুধু একটা ফাইলের স্রোত বোঝে, ফোল্ডারের কাঠামো বোঝে না। Unix দর্শন অনুযায়ী এ কাজ `tar`-এর। তাই আগে `tar` দিয়ে বাঁধো, তারপর কম্প্রেস করো (`tar -czf`)।

2. `-f`-এর ঠিক পরে `v` আছে, তাই tar আর্কাইভের নাম ধরবে `v`। `backup.tar` আর `/home/rahim` দুটোকেই আর্কাইভ করার জিনিস ভাববে (`backup.tar` না থাকলে error দেবে)। সঠিক: `tar -cvf backup.tar /home/rahim`।

3. `tar -czf site.tar.gz -C /var/www html`

4. `.tar.gz` পুরো আর্কাইভকে একটা gzip স্রোত হিসেবে কম্প্রেস করে। শেষে কিছু যোগ করতে হলে কম্প্রেস করা স্রোতটা খুলে নতুন করে কম্প্রেস করতে হবে, যা `-r` করে না। তাই GNU tar "Cannot update compressed archives" দেখায়। সমাধান: `gunzip`, তারপর `tar -rf`, তারপর আবার `gzip`।

5. `gzip`-এর ডিফল্ট আচরণ হলো মূল ফাইল মুছে তার জায়গায় `report.txt.gz` রাখা। মূল রাখার উপায়: `gzip -k report.txt`, অথবা `gzip -c report.txt > report.txt.gz`। (`gunzip report.txt.gz` দিলে মূল ফাইল ফিরেও আসবে।)

6. `zstd -T0` (ডিফল্ট লেভেল ৩, বা ১–৩)। এটা multi-core, gzip-এর চেয়ে অনেক দ্রুত এবং প্রায়ই বেশি ছোট করে, আর ডিকম্প্রেশন খুবই দ্রুত। উদাহরণ: `mysqldump ... | zstd -T0 > db.sql.zst`। পুরোনো সিস্টেমে zstd না থাকলে `pigz` বিকল্প। `xz -9` / `bzip2` এখানে খুব ধীর হবে।

7. `file data.zip` চালাও। এটা নাম না দেখে ফাইলের শুরুর magic bytes দেখে আসল ফরম্যাট বলে দেবে (যেমন "gzip compressed data" বা "XZ compressed data")। দরকার হলে `head -c 8 data.zip | xxd` দিয়ে নিজেও দেখা যায়। তারপর সেই ফরম্যাটের সঠিক টুল দিয়ে খোলো।

8. Windows বন্ধুর জন্য `.zip` (Windows-এ বিল্ট-ইন খোলে), গোপনীয় হলে `.7z` AES দিয়ে। `/etc`-এর ব্যাকআপের জন্য `sudo tar -czpf` (বা `--zstd`)। কারণ tar owner, group, permission, symlink হুবহু রাখে, যা zip/7z পুরোপুরি রাখে না, আর সিস্টেম ফাইলে এগুলো অপরিহার্য।

9. `tar -cf - -C ~ photos | ssh user@nas 'tar -xf - -C /backup'`। JPEG ছবি আগে থেকেই কম্প্রেস করা, তাই `-z` দিলে লাভ প্রায় নেই, শুধু CPU খরচ হবে। দ্রুত নেটওয়ার্কে কম্প্রেশন ছাড়াই ভালো (`/backup` ফোল্ডার আগে থেকে থাকতে হবে, না থাকলে `mkdir -p /backup &&` আগে লাগাও)।

10. (ক) `file tool.tar.gz` দিয়ে ফরম্যাট নিশ্চিত করো; (খ) প্রকাশক checksum দিলে `sha256sum` দিয়ে মেলাও; (গ) `tar -tvzf tool.tar.gz | less` দিয়ে ভেতরে দেখো: tarbomb কিনা (সব এক ফোল্ডারে আছে কিনা), সন্দেহজনক `../` বা `/` path আছে কিনা, অস্বাভাবিক বড় আকার আছে কিনা; (ঘ) একটা নতুন খালি ফোল্ডারে সাধারণ ইউজার হিসেবে খোলো: `mkdir t && tar -xzf tool.tar.gz -C t`; (ঙ) **কখনো `sudo` দিয়ে বা `-P` দিয়ে খুলবে না।**

অধ্যায় ৩৫: ব্যাকআপ ও সিঙ্ক — rsync, dd ও কৌশল

> "দুই ধরনের মানুষ আছে — যারা ব্যাকআপ রাখে, আর যারা এখনো ডেটা হারায়নি।" — সিস্টেম অ্যাডমিনদের পুরনো প্রবাদ

এই অধ্যায়ে যা শিখবে

- ব্যাকআপ কেন দরকার, আর কোন কোন বিপদ থেকে ব্যাকআপ আমাদের বাঁচায়
- Full, Incremental ও Differential ব্যাকআপের পার্থক্য — কোনটা কখন
- বিখ্যাত **3-2-1 নিয়ম**, RPO/RTO, retention (কতদিন রাখব) ধারণা
- `rsync` — লিনাক্সের সবচেয়ে জনপ্রিয় সিঙ্ক টুল: `-a -v -z -P --delete --dry-run --exclude` সহ সব গুরুত্বপূর্ণ অপশন
- **trailing slash রহস্য** — `src` আর `src/` এর পার্থক্য, যেটা নতুনদের সবচেয়ে বেশি ভোগায়
- SSH দিয়ে রিমোট সার্ভারে rsync
- rsync-এর **delta algorithm** — ভেতরে কীভাবে শুধু পরিবর্তিত অংশটুকু পাঠায়
- `--link-dest` দিয়ে **hard-link snapshot ব্যাকআপ** — Time Machine-এর মতো
- `dd` — ডিস্ক ক্লোন, ISO থেকে বুটেবল USB, `bs`, `status=progress` এবং এর ভয়ংকর দিক
- cron দিয়ে স্বয়ংক্রিয় ব্যাকআপ স্ক্রিপ্ট (লকিং, লগ, রোটেশন সহ)
- আধুনিক ব্যাকআপ টুল **restic** ও **borg** — deduplication ও encryption
- **Restore পরীক্ষা** — যে ব্যাকআপ restore করে দেখা হয়নি, সেটা ব্যাকআপ না

৩৫.১ ব্যাকআপ কেন? — বিপদের তালিকা

ধরো তোমার ল্যাপটপে তিন বছরের ছবি, অফিসের প্রজেক্ট, থিসিস — সব আছে। একদিন সকালে দেখলে হার্ডডিস্ক আর চালু হচ্ছে না। এই মুহূর্তে একটাই প্রশ্ন: **ব্যাকআপ আছে?**

ডেটা হারানোর প্রধান কারণগুলো:

বিপদ	উদাহরণ	কোন ব্যাকআপ বাঁচায়
হার্ডওয়্যার নষ্ট	SSD/HDD মরে গেল	অন্য ডিস্কে কপি
মানুষের ভুল	`rm -rf` ভুল ডিরেক্টরিতে	পুরনো ভার্সন (snapshot)
সফটওয়্যার বাগ/করাপশন	আপডেটের পর ফাইলসিস্টেম ভাঙল	snapshot

বিপদ	উদাহরণ	কোন ব্যাকআপ বাঁচায়
Ransomware/হ্যাকিং	সব ফাইল এনক্রিপ্ট করে টাকা চাইছে	offline/immutable কপি
চুরি/আগুন/বন্যা	ল্যাপটপ চুরি, অফিসে আগুন	অন্য জায়গায় (offsite) কপি

> 📝 **মনে রাখো:** RAID ব্যাকআপ না! (অধ্যায় ৩৩) RAID ডিস্ক নষ্ট হলে সিস্টেম চালু রাখে, কিন্তু তুমি `rm` দিয়ে ফাইল মুছলে RAID সেটা সব ডিস্ক থেকেই সাথে সাথে মুছে দেয়। ঠিক তেমনি **সিঙ্ক (sync) ব্যাকআপ না** — মূল ফাইল নষ্ট হলে সিঙ্ক সেই নষ্ট ফাইলটাই কপি করে দেবে। ব্যাকআপ মানে **সময়ের পুরনো অবস্থা ফিরে পাওয়ার ক্ষমতা**।

Backup বনাম Sync বনাম Archive

- **Sync (সিঙ্ক):** দুই জায়গাকে হুবহু একই রকম রাখা (আয়না)। এখানে মুছলে ওখানেও মুছবে।
- **Backup (ব্যাকআপ):** নির্দিষ্ট সময়ের কপি রেখে দেওয়া, যাতে পরে সেই অবস্থায় ফেরা যায়। একাধিক ভার্সন থাকে।
- **Archive (আর্কাইভ):** পুরনো, আর লাগবে না এমন ডেটা দীর্ঘমেয়াদে আলাদা করে রেখে দেওয়া (অধ্যায় ৩৪ এ `tar` দেখেছ)।

৩৫.২ ব্যাকআপ কৌশল — Full, Incremental, Differential

উপমা দিয়ে শুরু করি। ধরো তুমি প্রতিদিন একটা ডায়েরি লেখো, আর সেটার কপি রাখতে চাও।

- **Full:** প্রতিদিন পুরো ডায়েরিটা ফটোকপি করো। নিরাপদ, কিন্তু কাগজ আর সময় অনেক লাগে।
- **Incremental:** রবিবার পুরো ফটোকপি, তারপর প্রতিদিন **শুধু আগের দিনের পর** যে পাতাগুলো নতুন লেখা হয়েছে সেগুলো।
- **Differential:** রবিবার পুরো ফটোকপি, তারপর প্রতিদিন **রবিবারের পর থেকে** যত পাতা বদলেছে সব।

Full Backup (পূর্ণ ব্যাকআপ)

প্রতিবার সব ডেটা কপি হয়।

- ✅ Restore সবচেয়ে সহজ — একটাই সেট লাগে।
- ❌ সময় ও জায়গা সবচেয়ে বেশি লাগে।

Incremental Backup (ক্রমবর্ধমান ব্যাকআপ)

শেষ ব্যাকআপের (full বা incremental যেটাই হোক) পর যা বদলেছে শুধু সেটুকু।

- ✅ প্রতিদিনের ব্যাকআপ খুব দ্রুত ও ছোট।

- ❌ Restore করতে full + **প্রতিটি** incremental ক্রমানুসারে লাগে। মাঝের একটা নষ্ট হলে পরেরগুলো অকেজো।

Differential Backup (পার্থক্য ব্যাকআপ)

শেষ **full** ব্যাকআপের পর যা বদলেছে সব।

- ✅ Restore করতে শুধু full + সর্বশেষ differential লাগে।
- ❌ দিন যত যায়, differential তত বড় হয়।

```
সপ্তাহের চিত্র (F = Full, I = Incremental, D = Differential)

দিন:         রবি    সোম    মঙ্গল    বুধ     বৃহ
              ----   ----   -----   ----    ----
Incremental: F      I1     I2      I3      I4
             |      |      |       |       |
             সব     সোম    মঙ্গল    বুধ     বৃহ-এর
                    এর     এর       এর      পরিবর্তন
                   পরিবর্তন শুধু

  বৃহস্পতিবার restore = F + I1 + I2 + I3 + I4   (৫টা সেট)

Differential:F      D1     D2      D3      D4
             |      |      |       |       |
             সব     রবি→   রবি→    রবি→    রবি→বৃহ
                    সোম    মঙ্গল    বুধ     (সবচেয়ে বড়)

  বৃহস্পতিবার restore = F + D4                  (২টা সেট)
```

বৈশিষ্ট্য	Full	Incremental	Differential
ব্যাকআপের সময়	সবচেয়ে বেশি	সবচেয়ে কম	মাঝামাঝি (বাড়তে থাকে)
জায়গা	সবচেয়ে বেশি	সবচেয়ে কম	মাঝামাঝি
Restore গতি	সবচেয়ে দ্রুত	সবচেয়ে ধীর	দ্রুত
Restore-এ কত সেট লাগে	১	full + সব incremental	full + শেষ differential

> 💡 **টিপস:** আধুনিক টুলগুলো (rsync `--link-dest`, restic, borg) এই দ্বন্দ্ব প্রায় মিটিয়ে দিয়েছে — **প্রতিটা ব্যাকআপ incremental-এর মতো ছোট আর দ্রুত, কিন্তু restore করার সময় full-এর মতো সম্পূর্ণ**। এগুলোকে বলে "incremental-forever" বা snapshot ব্যাকআপ। এই অধ্যায়ের পরের অংশে দেখবে।

৩-২-১ নিয়ম (The 3-2-1 Rule)

ব্যাকআপ জগতের সোনার নিয়ম:

```
┌──────────────────────────────────────────────────┐
│  3  টি কপি ডেটার   (১টা আসল + ২টা ব্যাকআপ)        │
│  2  টি আলাদা ধরনের মিডিয়ায় (যেমন SSD + বাহ্যিক HDD/NAS) │
│  1  টি কপি অন্য জায়গায় (offsite — ক্লাউড/অন্য বাড়ি)   │
└──────────────────────────────────────────────────┘

 [ল্যাপটপ: আসল] ──rsync──> [বাহ্যিক USB HDD]   (ঘরে)
        │
        └─────restic──────> [ক্লাউড/রিমোট সার্ভার] (offsite)
```

অনেকে এখন **3-2-1-1-0** বলে: আরও একটা কপি **offline বা immutable** (যেটা ransomware ছুঁতে পারবে না — যেমন খুলে রাখা USB ডিস্ক), আর **0** মানে restore পরীক্ষায় শূন্য ত্রুটি।

RPO, RTO ও Retention

- **RPO (Recovery Point Objective):** সর্বোচ্চ কতটা সময়ের ডেটা হারানো মেনে নিতে পারো? দিনে একবার ব্যাকআপ হলে RPO = ২৪ ঘণ্টা।
- **RTO (Recovery Time Objective):** দুর্ঘটনার পর কত দ্রুত আবার চালু করতে হবে? ১ ঘণ্টা? ১ দিন?
- **Retention (সংরক্ষণ নীতি):** কত পুরনো ব্যাকআপ রাখবে। জনপ্রিয় পদ্ধতি **GFS (Grandfather-Father-Son)**: ৭টা দৈনিক, ৪টা সাপ্তাহিক, ১২টা মাসিক।

কী ব্যাকআপ করবে, কী করবে না

ব্যাকআপ করো	সাধারণত বাদ দাও
`/home` (তোমার ফাইল)	`/proc`, `/sys`, `/dev`, `/run` (ভার্চুয়াল, অধ্যায় ৪৭)
`/etc` (কনফিগারেশন)	`/tmp`, `/var/tmp`
`/var/www`, `/srv` (সার্ভারের ডেটা)	`~/.cache`, `node_modules`, বিল্ড আউটপুট
ডেটাবেস **ডাম্প** (`mysqldump`, `pg_dump`)	চলমান ডেটাবেসের কাঁচা ফাইল (`/var/lib/mysql`) — অসংগত হতে পারে
ইনস্টল করা প্যাকেজের তালিকা	পুরো `/usr` (প্যাকেজ ম্যানেজার আবার দিতে পারে)

> ⚠️ **সতর্কতা:** চলমান ডেটাবেসের ফাইল সরাসরি কপি করলে অর্ধেক-লেখা অবস্থা কপি হতে পারে, যেটা restore করলে ডেটাবেস চালু নাও হতে পারে। আগে `mysqldump` / `pg_dump` দিয়ে ডাম্প নাও, অথবা LVM snapshot (অধ্যায় ৩৩) ব্যবহার করো।

প্যাকেজ তালিকা রাখার কৌশল (অধ্যায় ৩০):

```
# Debian/Ubuntu
$ dpkg --get-selections > ~/backup/packages.list
# Fedora/RHEL
$ rpm -qa > ~/backup/packages.list
```

৩৫.৩ rsync — পরিচিতি

`rsync` (remote sync) ১৯৯৬ সালে Andrew Tridgell ও Paul Mackerras তৈরি করেন। এটা ফাইল কপি করে, কিন্তু `cp`-এর চেয়ে অনেক বুদ্ধিমান:

- যেটা আগেই গন্তব্যে আছে এবং বদলায়নি, সেটা **আবার কপি করে না**।
- বড় ফাইলের সামান্য অংশ বদলালে **শুধু বদলানো অংশ** নেটওয়ার্কে পাঠায় (delta algorithm)।
- লোকাল ডিরেক্টরি থেকে লোকাল, বা SSH দিয়ে রিমোট মেশিনে — দুটোই পারে।
- পারমিশন, মালিকানা, সময়, symlink — সব ঠিকঠাক রাখতে পারে।

উপমা: `cp` হলো এমন কুরিয়ার যে প্রতিবার পুরো আলমারি তুলে নিয়ে যায়। `rsync` হলো বুদ্ধিমান কুরিয়ার — আগে ফোন করে জিজ্ঞেস করে "ওখানে কোন জিনিস আগে থেকেই আছে?", তারপর শুধু নতুন জিনিসগুলো নিয়ে যায়।

ইনস্টলেশন

বেশিরভাগ distro-তে আগে থেকেই থাকে। না থাকলে:

```
$ sudo apt install rsync        # Ubuntu/Debian
$ sudo dnf install rsync        # Fedora/RHEL 9
$ rsync --version
rsync  version 3.2.7  protocol version 31
...
```

> 📝 **মনে রাখো:** রিমোটে rsync চালাতে **দুই প্রান্তেই** `rsync` ইনস্টল থাকতে হবে। কারণ SSH দিয়ে ঢুকে রিমোট মেশিনেও একটা rsync প্রসেস চালু হয়, দুজন মিলে কাজ করে।

Syntax

```
rsync [OPTIONS] SOURCE... DESTINATION

লোকাল:      rsync [OPT] /src/path/   /dest/path/
পুশ (রিমোট): rsync [OPT] /src/path/   user@host:/dest/path/
পুল (রিমোট): rsync [OPT] user@host:/src/path/   /dest/path/
```

> ⚠️ **সতর্কতা:** `SOURCE` আর `DESTINATION` দুটোই রিমোট হতে পারে না (একটা অবশ্যই লোকাল)।

প্রথম উদাহরণ

```
$ mkdir -p ~/lab/src/docs ~/lab/dst
$ echo "hello" > ~/lab/src/a.txt
$ echo "world" > ~/lab/src/docs/b.txt
```

```
$ rsync -av ~/lab/src/ ~/lab/dst/
sending incremental file list
./
a.txt
docs/
docs/b.txt

sent 254 bytes  received 61 bytes  630.00 bytes/sec
total size is 12  speedup is 0.04
```

এবার আবার একই কমান্ড চালাও:

```
$ rsync -av ~/lab/src/ ~/lab/dst/
sending incremental file list

sent 125 bytes  received 12 bytes  274.00 bytes/sec
total size is 12  speedup is 0.09
```

কোনো ফাইল তালিকায় নেই — কারণ কিছুই বদলায়নি, তাই কিছুই কপি হয়নি। এটাই rsync-এর জাদু।

শেষ লাইনের মানে: `total size` = সোর্সের মোট আকার; `speedup` = মোট আকার ÷ (পাঠানো + গ্রহণ করা বাইট)। speedup যত বেশি, rsync তত বেশি ট্রান্সফার বাঁচিয়েছে। খুব ছোট ফাইলে metadata-র কারণে speedup ১-এর কম হতেই পারে।

৩৫.৪ rsync-এর গুরুত্বপূর্ণ অপশন

`-a` (archive) — আসলে কী?

`-a` হলো কয়েকটা অপশনের শর্টকাট: **`-rlptgoD`**

অপশন	পূর্ণ নাম	কাজ
`-r`	`--recursive`	সাব-ডিরেক্টরি সহ ভেতরে ঢুকে কপি
`-l`	`--links`	symlink-কে symlink হিসেবেই কপি (লক্ষ্য ফাইল না)
`-p`	`--perms`	পারমিশন ঠিক রাখা
`-t`	`--times`	modification time ঠিক রাখা (**খুব জরুরি**, কারণ পরের বার এটা দিয়েই বদল ধরা হয়)
`-g`	`--group`	গ্রুপ মালিকানা রাখা
`-o`	`--owner`	মালিক রাখা (শুধু root হলে কাজ করে)
`-D`	`--devices --specials`	device ফাইল ও বিশেষ ফাইল রাখা (root লাগে)

> 📝 **মনে রাখো:** `-a` তিনটা জিনিস **রাখে না**: hard link (`-H`), ACL (`-A`), extended attribute (`-X`)। পুরো সিস্টেম ব্যাকআপে সাধারণত `-aHAX` ব্যবহার করা হয়। ACL ও xattr কী, তা অধ্যায় ১৭ এ দেখেছ।

প্রায়ই ব্যবহৃত অপশনের টেবিল

অপশন	কাজ
`-v`, `--verbose`	কী কপি হচ্ছে দেখায় (`-vv` আরও বিস্তারিত)
`-z`, `--compress`	নেটওয়ার্কে পাঠানোর সময় কম্প্রেস করে (লোকালে লাভ নেই)
`-h`, `--human-readable`	সংখ্যা K/M/G আকারে দেখায়
`-P`	`--partial --progress` একসাথে
`--progress`	প্রতিটা ফাইলের অগ্রগতি দেখায়
`--info=progress2`	পুরো ট্রান্সফারের মোট অগ্রগতি দেখায়
`--partial`	মাঝপথে থেমে গেলে আধা-কপি ফাইল রেখে দেয়, পরে সেখান থেকে চলে
`-n`, `--dry-run`	আসলে কিছু না করে শুধু দেখায় কী করবে
`--delete`	সোর্সে নেই এমন ফাইল গন্তব্য থেকে মুছে দেয় (আয়না বানাতে)
`--delete-excluded`	exclude করা ফাইলও গন্তব্য থেকে মুছে দেয়
`--max-delete=N`	N-এর বেশি ফাইল মোছার দরকার হলে মোছা বন্ধ করে (সেফটি নেট)
`--exclude=PATTERN`	মিলে যাওয়া ফাইল বাদ দেয়
`--exclude-from=FILE`	ফাইল থেকে exclude তালিকা পড়ে
`--include=PATTERN`	exclude-এর ভেতরে কিছু ফাইল আবার যোগ করে
`-e "ssh -p 2222"`	রিমোট শেল ও তার অপশন নির্দিষ্ট করে
`-H`	hard link রাখে
`-A`	ACL রাখে (`-p` সহ)
`-X`	extended attribute রাখে
`-c`, `--checksum`	সময়+আকার না দেখে checksum দিয়ে তুলনা (ধীর, কিন্তু নিশ্চিত)
`-u`, `--update`	গন্তব্যের ফাইল বেশি নতুন হলে সেটা বাদ দেয়
`--ignore-existing`	গন্তব্যে আগে থেকে থাকা ফাইল ছোঁয় না
`-i`, `--itemize-changes`	প্রতিটা ফাইলের কী বদলেছে তার সাংকেতিক বর্ণনা দেয়
`--backup --backup-dir=DIR`	মুছে যাওয়া/ওভাররাইট হওয়া ফাইল DIR-এ সরিয়ে রাখে
`--link-dest=DIR`	অপরিবর্তিত ফাইল DIR থেকে hard link করে (snapshot)
`--bwlimit=RATE`	ব্যান্ডউইথ সীমা (যেমন `--bwlimit=5m` = ৫ MB/s)
`--remove-source-files`	সফলভাবে কপির পর সোর্স ফাইল মুছে দেয় (move-এর মতো; ডিরেক্টরি থেকে যায়)

অপশন	কাজ
`-x`, `--one-file-system`	অন্য মাউন্ট করা ফাইলসিস্টেমে ঢোকে না
`--numeric-ids`	ইউজার/গ্রুপ নামের বদলে UID/GID সংখ্যা রাখে (অন্য মেশিনে restore-এর জন্য ভালো)
`--stats`	শেষে বিস্তারিত পরিসংখ্যান দেখায়
`--log-file=FILE`	লগ ফাইলে লেখে
`--inplace`	টেম্প ফাইল না বানিয়ে সরাসরি গন্তব্য ফাইলে লেখে (বড় VM ইমেজে কাজে লাগে)

`-P` ও অগ্রগতি দেখা

```
$ rsync -avhP ~/Videos/movie.mkv /media/usb/
sending incremental file list
movie.mkv
        612.37M  43%   58.20MB/s    0:00:13
```

মাঝপথে `Ctrl+C` চাপলে `--partial` থাকার কারণে আধা ফাইলটা রয়ে যায়; আবার একই কমান্ড চালালে বাকিটা থেকে চলবে (বিশেষ করে রিমোট ট্রান্সফারে delta দিয়ে)।

পুরো ট্রান্সফারের মোট অগ্রগতি:

```
$ rsync -ah --info=progress2 ~/Pictures/ /media/usb/Pictures/
          3.21G  64%   95.11MB/s    0:00:32 (xfr#1840, to-chk=1012/2853)
```

`xfr#1840` = এখন পর্যন্ত ১৮৪০টা ফাইল ট্রান্সফার হয়েছে; `to-chk=1012/2853` = মোট ২৮৫৩টার মধ্যে ১০১২টা পরীক্ষা বাকি।

`--dry-run` — আগে দেখো, তারপর করো

```
$ rsync -avn --delete ~/lab/src/ ~/lab/dst/
sending incremental file list
deleting old-report.pdf
a.txt

sent 160 bytes  received 25 bytes  370.00 bytes/sec
total size is 12  speedup is 0.06 (DRY RUN)
```

শেষে `(DRY RUN)` লেখা দেখাচ্ছে যে আসলে কিছুই হয়নি।

> 💡 **টিপস:** `--delete` বা নতুন কোনো কমান্ড চালানোর আগে **সবসময়** `-n` দিয়ে একবার দেখে নাও। এটা অভ্যাস বানাও: আগে `-n`, আউটপুট পড়ো, তারপর `-n` সরিয়ে আসল কাজ।

`--delete` — আয়না তৈরি

ডিফল্টে rsync শুধু যোগ করে ও আপডেট করে, কিছু মোছে না। ফলে সোর্স থেকে কোনো ফাইল মুছলেও সেটা গন্তব্যে থেকে যায়। হুবহু আয়না চাইলে `--delete`:

```
$ rm ~/lab/src/a.txt
$ rsync -av --delete ~/lab/src/ ~/lab/dst/
sending incremental file list
deleting a.txt

sent 112 bytes  received 22 bytes  268.00 bytes/sec
total size is 6  speedup is 0.04
```

> ⚠️ **সতর্কতা:** `--delete` এর সাথে ভুল সোর্স দিলে সর্বনাশ। ধরো USB ডিস্কটা মাউন্ট হয়নি, আর `/media/usb/` তখন একটা খালি ডিরেক্টরি। তুমি যদি উল্টো দিকে `rsync -a --delete /media/usb/ ~/` চালাও — তোমার পুরো home খালি ডিরেক্টরির আয়না হয়ে যাবে, মানে **সব মুছে যাবে**। তাই স্ক্রিপ্টে আগে `mountpoint -q /media/usb` দিয়ে যাচাই করো, এবং `--max-delete` ব্যবহার করো।

মোছার সময় ভিন্নতা:

অপশন	কখন মোছে
`--delete`	rsync 3.x এ ডিফল্ট `--delete-during` (ট্রান্সফারের সাথে সাথে)
`--delete-before`	ট্রান্সফার শুরুর আগে
`--delete-after`	সব ট্রান্সফার শেষ হওয়ার পর (নিরাপদ, তবে বেশি মেমরি)
`--delete-delay`	ট্রান্সফারের সময় তালিকা বানায়, শেষে মোছে

`--exclude` ও `--include` — কী বাদ দেবে

```
# একটা প্যাটার্ন
$ rsync -av --exclude='*.log' ~/project/ /backup/project/

# একাধিক
$ rsync -av --exclude='node_modules/' --exclude='.git/' --exclude='*.tmp' \
      ~/project/ /backup/project/

# ব্রেস এক্সপ্যানশন দিয়ে (bash নিজেই ভেঙে দেয়)
$ rsync -av --exclude={'node_modules','.cache','*.tmp'} ~/ /backup/home/
```

ফাইল থেকে তালিকা:

```
$ cat ~/.rsync-exclude
.cache/
node_modules/
```

```
*.iso
Downloads/
.local/share/Trash/

$ rsync -av --exclude-from=$HOME/.rsync-exclude ~/ /backup/home/
```

প্যাটার্নের নিয়ম (খুব গুরুত্বপূর্ণ):

প্যাটার্ন	মানে
`*.log`	যেকোনো জায়গায় `.log` এ শেষ হওয়া ফাইল/ডিরেক্টরি
`cache/`	শেষে `/` মানে শুধু **ডিরেক্টরি** মিলবে, `cache` নামের ফাইল না
`/tmp`	শুরুতে `/` মানে ট্রান্সফারের **রুট** থেকে (সোর্সের ভেতরের শীর্ষ লেভেল), সিস্টেমের `/` না
`*`	`/` ছাড়া যেকোনো অক্ষর
`**`	`/` সহ যেকোনো কিছু (যেকোনো গভীরতা)

`include` আর `exclude` এর ক্রম গুরুত্বপূর্ণ — **প্রথম যে নিয়ম মেলে, সেটাই প্রযোজ্য**। শুধু `.jpg` ফাইল কপি করতে চাইলে:

```
$ rsync -av --include='*/' --include='*.jpg' --exclude='*' ~/Pictures/ /backup/jpgs/
```

ব্যাখ্যা: আগে সব ডিরেক্টরি ঢুকতে দাও (`*/`, নইলে ভেতরে যেতেই পারবে না), তারপর `.jpg` নাও, বাকি সব বাদ। খালি ডিরেক্টরি বাদ দিতে `-m` (`--prune-empty-dirs`) যোগ করো।

`-i` (itemize) — কী বদলেছে তার সংকেত

```
$ rsync -ai ~/lab/src/ ~/lab/dst/
>f+++++++++ new.txt
>f.st...... docs/b.txt
.d..t...... docs/
```

প্রথম অক্ষর `>` = ফাইল পাঠানো হচ্ছে, `c` = তৈরি হচ্ছে, `.` = কিছু পাঠানো হয়নি, `*deleting` = মোছা হচ্ছে। দ্বিতীয় অক্ষর `f` = ফাইল, `d` = ডিরেক্টরি, `L` = symlink। বাকিগুলো: `s` = size বদলেছে, `t` = time, `p` = permission, `o` = owner, `g` = group, `+++` = একেবারে নতুন।

৩৫.৫ Trailing Slash রহস্য — `src` বনাম `src/`

এটা rsync-এর সবচেয়ে বিখ্যাত ফাঁদ। **সোর্সের শেষে `/` থাকা না থাকায় ফল পুরো আলাদা।**

- `rsync -a src dest/` → `src` **ডিরেক্টরিটা নিজেই** dest-এর ভেতরে যায়: `dest/src/...`
- `rsync -a src/ dest/` → src-এর **ভেতরের জিনিসগুলো** dest-এ যায়: `dest/...`

উপমা: `src` মানে "বাক্সটা দাও"; `src/` মানে "বাক্সের ভেতরের জিনিসগুলো দাও"।

```
সোর্স:  photos/
        ├── a.jpg
        └── b.jpg

rsync -a photos  backup/        rsync -a photos/  backup/
        (স্ল্যাশ নেই)                  (স্ল্যাশ আছে)

backup/                          backup/
└── photos/                      ├── a.jpg
    ├── a.jpg                    └── b.jpg
    └── b.jpg
```

হাতে-কলমে দেখো:

```
$ mkdir -p /tmp/t/photos && touch /tmp/t/photos/{a,b}.jpg
$ rsync -a /tmp/t/photos  /tmp/t/out1/
$ rsync -a /tmp/t/photos/ /tmp/t/out2/
$ find /tmp/t/out1 /tmp/t/out2
/tmp/t/out1
/tmp/t/out1/photos
/tmp/t/out1/photos/a.jpg
/tmp/t/out1/photos/b.jpg
/tmp/t/out2
/tmp/t/out2/a.jpg
/tmp/t/out2/b.jpg
```

কিছু খুঁটিনাটি:

- **গন্তব্যের** শেষে স্ল্যাশ থাকুক বা না থাকুক, সাধারণত ফল একই (গন্তব্য না থাকলে rsync ডিরেক্টরি হিসেবে বানিয়ে নেয়, যখন সোর্স ডিরেক্টরি)। তবে একটা ফাইল কপির সময় `dest/` লিখলে rsync সেটাকে ডিরেক্টরি ধরে নেয়।
- `src/.` লিখলে `src/` এর মতোই আচরণ করে।
- `cp -r src dest` এর আচরণ আলাদা (dest থাকলে `dest/src`, না থাকলে `dest`) — rsync এর মতো স্ল্যাশ-নির্ভর না। তাই দুটোকে গুলিয়ে ফেলো না।

> 💡 **টিপস:** সহজ অভ্যাস — ডিরেক্টরি সিঙ্ক করতে **দুই পাশেই স্ল্যাশ দাও**: `rsync -a src/ dest/`। তাহলে মনে থাকবে "src-এর ভেতরটা dest-এর ভেতরে"।

> ⚠️ **সতর্কতা:** `--delete` এর সাথে স্ল্যাশ ভুল করা সবচেয়ে বিপজ্জনক। `rsync -a --delete photos backup/` আর `rsync -a --delete photos/ backup/` — দ্বিতীয়টা backup/ এর ভেতরের অন্য সব কিছু (যেগুলো photos-এ নেই) মুছে দেবে! প্রথমে `-n`।

`-R` (relative) — পূর্ণ পাথ রাখা

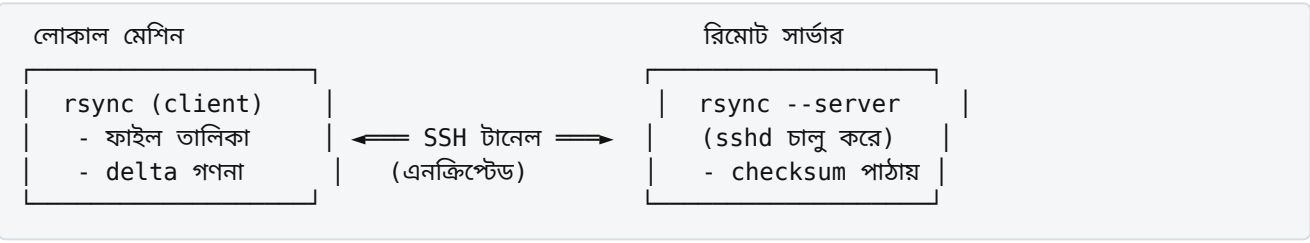
কখনো চাও গন্তব্যে পুরো পাথ কাঠামো থাকুক:

```
$ rsync -aR /etc/nginx /etc/ssh /backup/
# ফলাফল: /backup/etc/nginx/... এবং /backup/etc/ssh/...
```

পাথে `/./` দিলে তার আগের অংশ বাদ যায়: `rsync -aR /home/./rahim/docs /backup/` → `/backup/rahim/docs`।

৩৫.৬ SSH দিয়ে রিমোট rsync

rsync রিমোটে ডিফল্টভাবে **SSH** ব্যবহার করে (অধ্যায় ৩৮)। ফলে ডেটা এনক্রিপ্টেড থাকে, আলাদা কোনো সার্ভার চালাতে হয় না।



তুমি যখন `rsync -av dir/ user@host:/path/` চালাও, rsync ভেতরে ভেতরে প্রায় এরকম একটা কমান্ড চালায়: `ssh user@host rsync --server ...` — অর্থাৎ রিমোটে আরেকটা rsync চালু হয়, আর দুজন stdin/stdout পাইপ দিয়ে কথা বলে।

Push (লোকাল → রিমোট)

```
$ rsync -avzP ~/website/ deploy@203.0.113.10:/var/www/html/
deploy@203.0.113.10's password:
sending incremental file list
index.html
          4.21K 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=12/14)
css/style.css
          8.90K 100%    8.49MB/s    0:00:00 (xfr#2, to-chk=10/14)
...
```

Pull (রিমোট → লোকাল)

```
$ rsync -avzP backup@server:/var/backups/db/ ~/db-backups/
```

ভিন্ন পোর্ট বা কী-ফাইল

```
$ rsync -avz -e "ssh -p 2222 -i ~/.ssh/backup_key" ~/data/ user@host:/backup/data/
```

`~/.ssh/config` এ host সেট করা থাকলে আরও সহজ:

```
Host bkp
    HostName 203.0.113.10
    User backup
    Port 2222
    IdentityFile ~/.ssh/backup_key
```

```
$ rsync -avz ~/data/ bkp:/backup/data/
```

> 💡 **টিপস:** স্বয়ংক্রিয় (cron) ব্যাকআপের জন্য **পাসওয়ার্ডবিহীন SSH key** লাগবে (`ssh-keygen` , `ssh-copy-id` — অধ্যায় ৩৮)। নিরাপত্তার জন্য সার্ভারের `authorized_keys` এ এই key-কে সীমিত করতে পারো (`command=` , `from=` রেস্ট্রিকশন), অথবা rsync-এর সাথে আসা `rrsync` স্ক্রিপ্ট ব্যবহার করতে পারো যেটা key-কে একটা নির্দিষ্ট ডিরেক্টরিতে আটকে রাখে।

root ফাইলের ব্যাকআপ রিমোটে

রিমোটে মালিকানা রাখতে রিমোট প্রান্তেও root লাগে। প্রচলিত কৌশল:

```
$ sudo rsync -aHAX --numeric-ids -e ssh /etc/ root@bkp:/backup/host1/etc/
# অথবা রিমোটে sudo দিয়ে rsync চালানো:
$ rsync -a --rsync-path="sudo rsync" /data/ user@bkp:/backup/data/
```

দ্বিতীয়টার জন্য রিমোটে ওই ইউজারের `rsync` পাসওয়ার্ড ছাড়া sudo অনুমতি লাগবে (অধ্যায় ১৮)।

`-z` কখন দেবে?

- ধীর নেটওয়ার্ক (ইন্টারনেট) + টেক্সট/কোড/লগ ফাইল → `-z` লাভজনক।
- দ্রুত LAN, বা আগে থেকেই কম্প্রেসড ফাইল (`.jpg` , `.mp4` , `.zip` , `.gz`) → `-z` বরং CPU নষ্ট করে। rsync নিজেও কিছু পরিচিত extension আপনা-আপনি কম্প্রেস করা থেকে বাদ দেয়।
- লোকাল কপিতে `-z` এর কোনো মানে নেই।

rsync daemon (সংক্ষেপে)

SSH ছাড়াও rsync নিজস্ব সার্ভার মোডে (`rsync --daemon` , পোর্ট 873, কনফিগ `/etc/rsyncd.conf`) চলতে পারে। তখন ঠিকানা লেখা হয় `host::module` বা `rsync://host/module` । এটা এনক্রিপ্টেড না, তাই সাধারণত বিশ্বস্ত নেটওয়ার্কে বা পাবলিক মিরর সার্ভারে (যেমন Linux distro মিরর) ব্যবহার হয়। নতুনদের জন্য SSH-ই যথেষ্ট ও নিরাপদ।

৩৫.৭ ভেতরে কীভাবে কাজ করে — rsync-এর Delta Algorithm

এবার আসল মজার অংশ। ধরো সার্ভারে একটা ১ GB লগ ফাইল আছে, তোমার কাছেও তার পুরনো কপি আছে। নতুনটায় মাঝখানে কয়েক লাইন যোগ হয়েছে। rsync কীভাবে পুরো ১ GB না পাঠিয়ে শুধু ওই কয়েক লাইন পাঠায়, **অথচ দুই প্রান্তের কেউই অন্যের ফাইল দেখতে পায় না?**

ধাপ ০: Quick Check — আদৌ বদলেছে কি?

প্রতিটা ফাইলের জন্য rsync প্রথমে শুধু **আকার (size) আর modification time (mtime)** মেলায়। দুটোই এক হলে ধরে নেয় ফাইল বদলায়নি, এবং ছুঁয়েও দেখে না। এজন্যই rsync এত দ্রুত — লক্ষ লক্ষ ফাইলের মধ্যে বেশিরভাগ এই ধাপেই বাদ পড়ে। (`-c` দিলে এই শর্টকাট বাদ দিয়ে প্রতিটা ফাইলের পুরো checksum গণনা করে — নিশ্চিত, কিন্তু অনেক ধীর।)

এজন্যই `-t` (time রাখা, `-a` এর অংশ) এত জরুরি। `-t` ছাড়া কপি করলে গন্তব্যের সময় হবে কপির সময়, ফলে পরের বার সব ফাইল আবার "বদলানো" মনে হবে।

ধাপ ১–৪: বদলানো ফাইলের delta

ধরা যাক **A** = পাঠানেওয়ালা (sender, যার কাছে নতুন ফাইল), **B** = গ্রহণকারী (receiver, যার কাছে পুরনো ফাইল)।

```
B (পুরনো ফাইল আছে)                          A (নতুন ফাইল আছে)
──────────────────                           ──────────────────

১. পুরনো ফাইলকে সমান সাইজের ব্লকে ভাগ:
   [ব্লক0][ব্লক1][ব্লক2][ব্লক3]...
   প্রতিটার জন্য ২টা checksum:
    - দুর্বল, দ্রুত "rolling" checksum
    - শক্তিশালী hash (MD5/xxhash)
                  ──── checksum তালিকা ────►
                                             ২. নিজের নতুন ফাইলের উপর
                                                প্রতি বাইটে এক ঘর করে
                                                একটা "জানালা" সরায়:
                                                প্রতিটা অবস্থানে rolling
                                                checksum মেলে কি না দেখে।
                                                মিললে strong hash দিয়ে
                                                নিশ্চিত হয়।
                                             ৩. নির্দেশনা বানায়:
                                                "ব্লক0 নাও, তারপর এই নতুন
                                                 ২০০ বাইট, তারপর ব্লক1..."
                  ◄──── delta (নির্দেশ + নতুন ডেটা) ──
৪. নিজের পুরনো ব্লক + নতুন ডেটা দিয়ে
   একটা টেম্প ফাইলে নতুন ফাইল জোড়া লাগায়,
   পুরো ফাইলের checksum যাচাই করে,
   তারপর rename করে আসলটার জায়গায় বসায়।
```

উদাহরণ দিয়ে দেখি। পুরনো ফাইল (ব্লক সাইজ ৪ অক্ষর ধরে):

```
পুরনো:  AAAA BBBB CCCC DDDD
নতুন:   AAAA BBBB XY CCCC DDDD      (মাঝে "XY" ঢোকানো)
```

```
A যা পাঠায়:  [ব্লক0] [ব্লক1] "XY" [ব্লক2] [ব্লক3]
                ↑ শুধু রেফারেন্স     ↑ আসল ডেটা মাত্র ২ বাইট
```

Rolling checksum কেন দরকার?

যদি ব্লকগুলো শুধু নির্দিষ্ট জায়গায় (০, ৪, ৮...) মেলানো হতো, তাহলে মাঝে "XY" ঢোকার পর পরের সব ব্লক ২ ঘর সরে যেত, আর কিছুই মিলত না। তাই A **প্রতিটা বাইট অবস্থানে** মেলায়। কিন্তু প্রতিটা অবস্থানে নতুন করে hash গণনা করা খুব ধীর।

Rolling checksum এর বিশেষত্ব: জানালা এক বাইট সরালে আগের মান থেকে **বেরিয়ে যাওয়া বাইটটা বাদ দিয়ে আর ঢোকা বাইটটা যোগ করে** নতুন মান পাওয়া যায় — পুরো ব্লক আবার পড়তে হয় না। উপমা: ট্রেনের জানালা দিয়ে ৫টা বগি দেখছ; ট্রেন এক বগি এগোলে তুমি শুধু "একটা বগি গেল, একটা এল" হিসাব করো, পাঁচটা আবার গোনো না।

দুর্বল checksum মাঝে মাঝে ভুল করে মিলে যেতে পারে, তাই মিললে তখনই শক্তিশালী hash দিয়ে নিশ্চিত হওয়া হয়। এভাবে গতি আর নির্ভুলতা দুটোই পাওয়া যায়। (rsync 3.x এ শক্তিশালী hash হিসেবে আগে MD5 ছিল; 3.2 থেকে দুই প্রান্ত সমর্থন করলে xxhash-এর মতো দ্রুততর অ্যালগরিদম নিজেরাই বেছে নেয়।)

গুরুত্বপূর্ণ বাস্তব তথ্য

- **লোকাল কপিতে delta বন্ধ থাকে।** দুটো পাথই লোকাল হলে rsync ডিফল্টে `--whole-file` (`-W`) ব্যবহার করে, কারণ একই মেশিনে পুরো ফাইল পড়ে ব্লক তুলনা করার চেয়ে সরাসরি কপি দ্রুত। delta-র লাভ মূলত **নেটওয়ার্কে**। (তবে quick check লোকালেও কাজ করে, তাই অপরিবর্তিত ফাইল লোকালেও বাদ যায়।)
- ডিফল্টে rsync নতুন ফাইল একটা লুকানো টেম্প নামে (`.filename.XXXXXX`) লিখে শেষে rename করে। ফলে মাঝপথে থামলে পুরনো ফাইলটা অক্ষত থাকে। `--inplace` এই আচরণ বন্ধ করে।
- ব্লক সাইজ rsync ফাইলের আকার দেখে নিজেই ঠিক করে (প্রায় ফাইল সাইজের বর্গমূলের কাছাকাছি, নির্দিষ্ট সীমার মধ্যে)।
- **ফাইল তালিকা আদান-প্রদান:** শুরুতে sender পুরো ফাইল তালিকা (নাম, আকার, সময়, পারমিশন) পাঠায়; rsync 3.0 থেকে এটা **incremental** — তালিকা আসতে আসতেই কাজ শুরু হয়, তাই আউটপুটে লেখা থাকে `sending incremental file list`।

```
rsync-এর প্রসেস মডেল (রিমোট ক্ষেত্রে):

 Client                  Server
 ┌──────────┐            ┌──────────┐
 │  sender  │──ফাইল────►│ generator│ (checksum বানায় ও পাঠায়)
 │          │  তালিকা    │    │     │
 │          │◄─checksum──┤    ▼     │
 │          │──delta────►│ receiver │ (নতুন ফাইল জোড়া লাগায়)
 └──────────┘            └──────────┘
 (pull এর ক্ষেত্রে ভূমিকা উল্টে যায়)
```

৩৫.৮ Hard-link Snapshot ব্যাকআপ — `--link-dest`

এখন rsync দিয়ে এমন ব্যাকআপ বানাব যেখানে **প্রতিদিনের একটা আলাদা সম্পূর্ণ ফোল্ডার** থাকবে (যেকোনো দিনের অবস্থা দেখা যাবে), অথচ জায়গা লাগবে প্রায় incremental-এর মতো। এটাই macOS Time Machine এবং `rsnapshot`-এর মূল কৌশল।

আগে মনে করো: hard link কী?

অধ্যায় ১১ এ দেখেছ — ফাইলের আসল ডেটা থাকে **inode**-এ, আর ফাইলের নাম হলো ওই inode-এর দিকে একটা পয়েন্টার। একই inode-এর একাধিক নাম (hard link) থাকতে পারে। ডেটা ডিস্কে **একবারই** থাকে। সব নাম মুছলে তবেই ডেটা মুছে যায়।

`--link-dest` কী করে?

`--link-dest=PREV` দিলে rsync প্রতিটা ফাইলের জন্য দেখে: PREV ডিরেক্টরিতে কি একই ফাইল (একই আকার, সময়, পারমিশন, মালিক) আছে?

- **থাকলে:** কপি না করে নতুন snapshot-এ PREV-এর ফাইলের একটা **hard link** বানায় — কোনো নতুন জায়গা লাগে না।
- **না থাকলে (নতুন বা বদলানো):** সাধারণভাবে পূর্ণ কপি করে।

```
/backup/
├── 2026-11-10/          ├── 2026-11-11/          ├── 2026-11-12/
│   ├── a.txt ──┐        │   ├── a.txt ──┐        │   ├── a.txt ──┐
│   ├── b.txt ──┤|       │   ├── b.txt*  | (বদলেছে)  |   ├── b.txt* ──┼┐
│   └── c.txt ──┤||      │   └── c.txt ──┤ |       │   └── d.txt   ||(নতুন)
                |||                     | |           (c মোছা)    ||
 ডিস্কে inode: ▼▼▼                     ▼ ▼                       ▼▼
        [c-ডেটা][b-ডেটা পুরনো][a-ডেটা]  [b-ডেটা নতুন]  [d-ডেটা]

 a.txt: তিন snapshot-এ ৩টা নাম, কিন্তু ডিস্কে ডেটা একবার।
 প্রতিটা ফোল্ডার নিজেই একটা "সম্পূর্ণ" ব্যাকআপ — সরাসরি cp দিয়ে restore করা যায়।
```

পুরনো একটা snapshot ফোল্ডার মুছে দিলেও কোনো সমস্যা নেই — অন্য snapshot-এ যে ফাইলের link আছে, সেগুলোর ডেটা থেকে যায়। এটাই incremental-এর চেইন-ভাঙা সমস্যার সমাধান।

হাতে-কলমে

```
$ SRC=~/Documents/
$ DEST=/backup/snapshots
$ TODAY=$(date +%F)            # যেমন 2026-11-12

# প্রথম দিন: স্বাভাবিক পূর্ণ কপি
$ rsync -a --delete "$SRC" "$DEST/$TODAY/"

# পরের দিনগুলো: আগেরটার সাথে link
```

```
$ rsync -a --delete --link-dest="$DEST/latest" "$SRC" "$DEST/$TODAY/"
$ ln -sfn "$DEST/$TODAY" "$DEST/latest"     # latest সিমলিংক আপডেট
```

> 📝 **মনে রাখো:** `--link-dest` এর পাথ যদি আপেক্ষিক (relative) হয়, তাহলে সেটা **গন্তব্য ডিরেক্টরির সাপেক্ষে** ধরা হয়, তোমার বর্তমান ডিরেক্টরির না। বিভ্রান্তি এড়াতে সবসময় **পূর্ণ (absolute) পাথ** দাও। PREV ডিরেক্টরি না থাকলে rsync একটা সতর্কবার্তা দিয়ে স্বাভাবিক পূর্ণ কপি করে।

যাচাই — inode নম্বর ও link count দেখো:

```
$ ls -li /backup/snapshots/2026-11-11/a.txt /backup/snapshots/2026-11-12/a.txt
1835041 -rw-r--r-- 3 rahim rahim 512 Nov 10 09:14 /backup/snapshots/2026-11-11/a.txt
1835041 -rw-r--r-- 3 rahim rahim 512 Nov 10 09:14 /backup/snapshots/2026-11-12/a.txt
```

একই inode (`1835041`), link count `3` । আর `du` একসাথে চালালে hard link একবারই গোনে:

```
$ du -sh /backup/snapshots/*
2.1G    /backup/snapshots/2026-11-10
45M     /backup/snapshots/2026-11-11
12M     /backup/snapshots/2026-11-12
```

(প্রথমটা পূর্ণ সাইজ দেখাচ্ছে; বাকিগুলো শুধু নতুন ডেটা, কারণ `du` একই কমান্ডে আগে দেখা inode আর গোনে না।)

সীমাবদ্ধতা

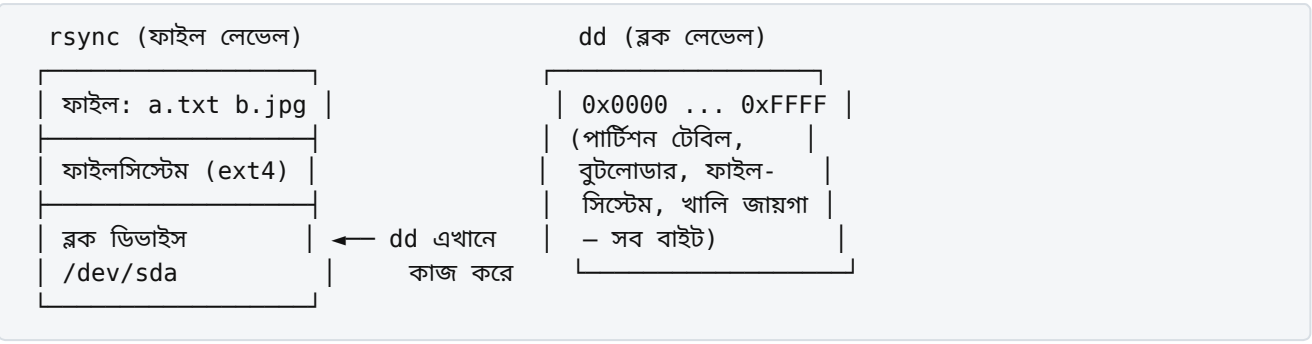
- গন্তব্য আর PREV **একই ফাইলসিস্টেমে** থাকতে হবে (hard link ফাইলসিস্টেম পেরোতে পারে না)।
- গন্তব্য ফাইলসিস্টেমকে hard link সমর্থন করতে হবে — ext4, XFS, Btrfs পারে; **FAT32/exFAT পারে না**। তাই ব্যাকআপ ডিস্ক ext4 এ ফরম্যাট করো (অধ্যায় ৩২)।
- একটা বড় ফাইলের এক বাইট বদলালেও পুরো ফাইল নতুন করে জায়গা নেয় (ফাইল-লেভেল dedup, ব্লক-লেভেল না)। VM ইমেজের মতো বড় ফাইলে restic/borg ভালো।
- snapshot-এর ফাইলগুলো hard link, তাই কোনো snapshot-এর ফাইল **সরাসরি এডিট করো না** — অন্য snapshot-এর একই ফাইলও বদলে যাবে! ব্যাকআপ ডিস্ক পড়ার সময় read-only মাউন্ট করাই ভালো।

> 💡 **টিপস:** এই কৌশলটা তৈরি অবস্থায় চাইলে `rsnapshot` প্যাকেজ আছে (`sudo apt install rsnapshot`) — `/etc/rsnapshot.conf` এ সোর্স আর retention লিখে দিলেই hourly/daily/weekly snapshot রোটেট করে।

৩৫.৯ `dd` — ব্লক লেভেলের কপি

`rsync` কাজ করে **ফাইল** নিয়ে। `dd` কাজ করে **কাঁচা বাইট** নিয়ে — সে জানেই না ফাইল কী, ফাইলসিস্টেম কী। ইনপুট থেকে বাইট পড়ে, আউটপুটে লেখে। এজন্য এটা দিয়ে পুরো ডিস্ক, পার্টিশন, বুট সেক্টর — সব হুবহু কপি করা যায়।

উপমা: rsync হলো লাইব্রেরিয়ান যে বই ধরে ধরে গুছিয়ে কপি করে। dd হলো ফটোকপি মেশিন যে পুরো তাক পাতা-পাতা ছবি তুলে নেয় — খালি পাতাও, ছেঁড়া পাতাও।



> ⚠️ **সতর্কতা:** `dd` এর ডাকনাম **"disk destroyer"**। এটা কোনো প্রশ্ন করে না, কোনো "Are you sure?" নেই। `of=` এ ভুল ডিস্ক লিখলে সেই ডিস্কের সব ডেটা মুহূর্তে চলে যাবে, ফেরানোর উপায় নেই। `if` (input) আর `of` (output) উল্টে ফেললেও একই সর্বনাশ।

Syntax ও অপশন

```
dd if=INPUT of=OUTPUT [bs=SIZE] [count=N] [status=progress] [conv=...] [oflag=...]
```

dd-এর অপশনগুলো অন্য কমান্ডের মতো `-x` না, বরং `key=value` আকারে।

অপশন	কাজ
`if=FILE`	input file — কোথা থেকে পড়বে (না দিলে stdin)
`of=FILE`	output file — কোথায় লিখবে (না দিলে stdout)
`bs=SIZE`	block size — একবারে কত বাইট পড়বে ও লিখবে (যেমন `4M`, `64K`; ডিফল্ট 512 বাইট)
`ibs=` / `obs=`	পড়া ও লেখার ব্লক সাইজ আলাদা করে
`count=N`	মাত্র N টা ব্লক কপি করে থামবে
`skip=N`	ইনপুটের শুরু থেকে N ব্লক বাদ দিয়ে পড়বে
`seek=N`	আউটপুটের শুরু থেকে N ব্লক বাদ দিয়ে লিখবে
`status=progress`	চলতে চলতে অগ্রগতি দেখাবে (GNU coreutils 8.24+)
`status=none`	কোনো পরিসংখ্যান দেখাবে না

অপশন	কাজ
`conv=fsync`	শেষে ডেটা আসলেই ডিস্কে লেখা নিশ্চিত করবে
`conv=noerror,sync`	পড়ার ত্রুটি হলে না থেমে চালিয়ে যাবে, খারাপ ব্লক শূন্য দিয়ে ভরবে
`conv=sparse`	আউটপুটে শূন্য ব্লক না লিখে জায়গা ফাঁকা রাখবে (sparse ফাইল)
`oflag=direct`	page cache এড়িয়ে সরাসরি লিখবে
`oflag=sync`	প্রতিটা ব্লক লেখার পর সিঙ্ক করবে (ধীর, তবে অগ্রগতি সঠিক দেখায়)
`iflag=fullblock`	পাইপ থেকে পড়ার সময় পুরো ব্লক ভরা পর্যন্ত অপেক্ষা করবে

সাইজ suffix: `K`=1024, `M`=1024², `G`=1024³; আর `KB`=1000, `MB`=1000² (দশমিক)।

`bs` কেন গুরুত্বপূর্ণ?

ডিফল্ট `bs=512` মানে একবারে মাত্র ৫১২ বাইট — প্রতিটা ছোট টুকরার জন্য একটা আলাদা `read()` ও `write()` system call। ৮ GB কপি করতে লাগবে প্রায় ১ কোটি ৬৮ লক্ষ system call! তাই অনেক ধীর। `bs=4M` দিলে একবারে ৪ MB, system call কমে প্রায় ২০৪৮টায়।

```
bs=512  : [.][.][.][.][.][.][.][.] ... কোটি বার যাতায়াত (ধীর)
bs=4M   : [████████][████████] ...   হাজার দুয়েক বার (দ্রুত)
```

সাধারণত `bs=1M` থেকে `bs=4M` ভালো কাজ করে। অনেক বড় (যেমন `1G`) দিলে মেমরি নষ্ট হয়, গতি আর বাড়ে না।

প্রথম কাজ: সঠিক ডিস্ক চেনা

dd চালানোর আগে **সবসময়** `lsblk` দিয়ে দেখো কোনটা কোন ডিস্ক (অধ্যায় ৩২):

```
$ lsblk -o NAME,SIZE,TYPE,MOUNTPOINTS,MODEL
NAME        SIZE TYPE MOUNTPOINTS MODEL
nvme0n1   476.9G disk             Samsung SSD 980
├─nvme0n1p1   1G part /boot/efi
└─nvme0n1p2 475.9G part /
sda        14.6G disk             SanDisk Ultra
└─sda1     14.6G part /media/rahim/USB
```

এখানে ১৪.৬G এর SanDisk USB হলো `/dev/sda`। সিস্টেম ডিস্ক `nvme0n1` — এটায় ভুলেও লেখা চলবে না।

> 💡 **টিপস:** USB লাগানোর আগে একবার আর পরে একবার `lsblk` চালাও। যেটা নতুন এসেছে, সেটাই তোমার USB। আরও নিশ্চিত হতে `ls -l /dev/disk/by-id/` দেখো — সেখানে মডেল নাম সহ স্থায়ী নাম থাকে, যেমন `usb-SanDisk_Ultra_...`। স্ক্রিপ্টে `/dev/sdX` এর বদলে এই by-id পাথ ব্যবহার করা নিরাপদ, কারণ রিবুটে `sda` / `sdb` এর ক্রম বদলে যেতে পারে।

কাজ ১: ISO থেকে বুটেবল USB বানানো

```
$ sudo umount /dev/sda1          # আগে মাউন্ট থাকলে খুলে নাও
$ sudo dd if=ubuntu-24.04-desktop-amd64.iso of=/dev/sda bs=4M status=progress
oflag=sync
5976883200 bytes (6.0 GB, 5.6 GiB) copied, 412 s, 14.5 MB/s
1425+1 records in
1425+1 records out
5978619904 bytes (6.0 GB, 5.6 GiB) copied, 412.8 s, 14.5 MB/s
$ sync
```

লক্ষ করো:

- `of=/dev/sda` — পুরো ডিস্ক, `/dev/sda1` **(পার্টিশন) না**। ISO-র ভেতরে নিজস্ব পার্টিশন টেবিল আছে।
- `1425+1 records in` — ১৪২৫টা পূর্ণ 8M ব্লক আর ১টা আংশিক ব্লক।
- কাজ শেষে `sync` চালাও, তারপর USB খোলো। `oflag=sync` বা `conv=fsync` ছাড়া dd শেষ হওয়ার পরও ডেটা কিছুক্ষণ মেমরির cache-এ থাকতে পারে।

> 📝 **মনে রাখো:** আধুনিক Linux ISO (Ubuntu, Fedora, Debian) "hybrid ISO", তাই সরাসরি dd করলেই বুট হয়। **Windows-এর ISO এভাবে কাজ করে না** — তার জন্য আলাদা টুল (যেমন WoeUSB, Ventoy) লাগে। আর GUI পছন্দ হলে GNOME Disks-এর "Restore Disk Image" বা Fedora Media Writer একই কাজ করে।

কাজ ২: পুরো ডিস্ক ক্লোন করা

```
# ডিস্ক থেকে ডিস্ক (sdb হুবহু sdc তে; sdc ≥ sdb হতে হবে)
$ sudo dd if=/dev/sdb of=/dev/sdc bs=4M status=progress conv=fsync

# ডিস্ক থেকে ইমেজ ফাইল (ব্যাকআপ)
$ sudo dd if=/dev/sdb of=/backup/sdb.img bs=4M status=progress

# ইমেজ কম্প্রেস করে রাখা (খালি জায়গা ভালো কম্প্রেস হয়)
$ sudo dd if=/dev/sdb bs=4M status=progress | gzip -c > /backup/sdb.img.gz

# Restore
$ gunzip -c /backup/sdb.img.gz | sudo dd of=/dev/sdb bs=4M status=progress
iflag=fullblock conv=fsync
```

> ⚠️ **সতর্কতা:**
>
> - **মাউন্ট করা, চলমান ফাইলসিস্টেম কখনো dd দিয়ে ক্লোন করো না** — কপির মাঝে ফাইল বদলাতে থাকবে, ফলে ক্লোন অসংগত (inconsistent) হবে। Live USB থেকে বুট করে ক্লোন করো।
> - গন্তব্য ডিস্ক সোর্সের চেয়ে ছোট হলে ক্লোন কাটা পড়বে।
> - ক্লোনের পর দুই ডিস্কেই একই UUID থাকে। দুটো একসাথে একই মেশিনে লাগালে মাউন্ট বিভ্রান্তি হতে পারে।

> - SSD-তে dd সব ব্লক লেখে (খালি জায়গাও), তাই সময় ও লেখার চক্র বেশি লাগে।

কাজ ৩: MBR/বুট সেক্টর ব্যাকআপ

```
# প্রথম ৫১২ বাইট (MBR: বুট কোড + পার্টিশন টেবিল)
$ sudo dd if=/dev/sda of=~/mbr.bin bs=512 count=1
1+0 records in
1+0 records out
512 bytes copied, 0.000312 s, 1.6 MB/s
```

GPT ডিস্কের পার্টিশন টেবিল ব্যাকআপের জন্য dd-এর চেয়ে `sgdisk --backup=table.bin /dev/sda` ভালো, আর MBR/GPT দুটোর জন্যই `sfdisk -d /dev/sda > table.txt` চলে।

কাজ ৪: টেস্ট ফাইল বানানো ও ডিস্ক মোছা

```
# ১ GB এর শূন্য-ভরা ফাইল (swap বা টেস্টের জন্য)
$ dd if=/dev/zero of=test.img bs=1M count=1024 status=progress

# র‍্যান্ডম ডেটার ফাইল
$ dd if=/dev/urandom of=random.bin bs=1M count=100

# পুরো ডিস্ক শূন্য দিয়ে মুছে ফেলা (বিক্রির আগে) — চূড়ান্ত বিপজ্জনক!
$ sudo dd if=/dev/zero of=/dev/sdX bs=4M status=progress
```

> 💡 **টিপস:** শুধু দ্রুত একটা বড় ফাইল দরকার হলে `fallocate -l 1G test.img` অনেক দ্রুত (ডেটা না লিখে শুধু জায়গা বরাদ্দ করে)। SSD নিরাপদে মুছতে dd-এর চেয়ে `blkdiscard` বা নির্মাতার secure-erase ভালো, কারণ SSD-র wear-leveling এর কারণে overwrite সব কোষে পৌঁছায় না।

চলমান dd এর অগ্রগতি দেখা

`status=progress` দিতে ভুলে গেলে অন্য টার্মিনাল থেকে `USR1` সিগন্যাল পাঠালে GNU dd তার বর্তমান পরিসংখ্যান ছাপে (অধ্যায় ২৪):

```
$ sudo kill -USR1 $(pgrep -x dd)
```

বা `progress` নামের ছোট টুল (`sudo apt install progress`) চালাও — cp, dd, rsync সবার অগ্রগতি দেখায়।

খারাপ ডিস্ক থেকে উদ্ধার

ত্রুটিপূর্ণ ডিস্ক থেকে ডেটা উদ্ধারে `dd conv=noerror,sync` ব্যবহার করা যায়, কিন্তু আসল কাজের টুল হলো **`ddrescue`** (প্যাকেজ: Ubuntu/Debian এ `gddrescue`, Fedora তে `ddrescue`)। এটা আগে ভালো

অংশগুলো দ্রুত কপি করে, তারপর খারাপ অংশে বারবার চেষ্টা করে, আর একটা map ফাইলে অগ্রগতি রাখে যাতে থেমে গেলে আবার চালানো যায়:

```
$ sudo ddrescue -d /dev/sdb /backup/rescue.img /backup/rescue.map
```

rsync বনাম dd বনাম tar — কখন কোনটা

পরিস্থিতি	টুল
ফোল্ডার নিয়মিত ব্যাকআপ/সিঙ্ক	`rsync`
রিমোট সার্ভারে ফাইল পাঠানো	`rsync` (বা `scp`)
একটা ফাইলে প্যাক করে পাঠানো/রাখা	`tar` (অধ্যায় ৩৪)
বুটেবল USB তৈরি	`dd`
পুরো ডিস্ক হুবহু ক্লোন (বুটলোডার সহ)	`dd` (বা Clonezilla, partclone)
নষ্ট হতে থাকা ডিস্ক থেকে উদ্ধার	`ddrescue`
এনক্রিপ্টেড, dedup করা, বহু-ভার্সন ব্যাকআপ	`restic` / `borg`

৩৫.১০ cron দিয়ে স্বয়ংক্রিয় ব্যাকআপ স্ক্রিপ্ট

হাতে ব্যাকআপ নিলে একদিন ভুলে যাবেই। তাই স্বয়ংক্রিয় করতে হবে। এখানে একটা বাস্তবসম্মত snapshot ব্যাকআপ স্ক্রিপ্ট লিখব, যেটা অধ্যায় ৪১–৪৩ এর শেল স্ক্রিপ্টিং আর অধ্যায় ২৯ এর cron ব্যবহার করে।

স্ক্রিপ্ট: `/usr/local/bin/snapshot-backup.sh`

```
#!/usr/bin/env bash
# snapshot-backup.sh — rsync --link-dest দিয়ে দৈনিক snapshot ব্যাকআপ
set -euo pipefail

# ---------- কনফিগারেশন ----------
SRC_DIRS=("/home" "/etc" "/var/www")       # কী ব্যাকআপ হবে
BACKUP_MNT="/mnt/backup"                    # ব্যাকআপ ডিস্কের মাউন্ট পয়েন্ট
DEST="$BACKUP_MNT/snapshots"
EXCLUDE_FILE="/etc/backup/exclude.txt"
KEEP_DAYS=14                                # কত দিনের snapshot রাখবে
LOG="/var/log/snapshot-backup.log"
LOCK="/run/snapshot-backup.lock"

TODAY=$(date +%F_%H%M)
NEW="$DEST/$TODAY"
LATEST="$DEST/latest"

log() { echo "[$(date '+%F %T')] $*" | tee -a "$LOG"; }

# ---------- একসাথে দুইবার চলা আটকাও ----------
exec 9>"$LOCK"
```

```
if ! flock -n 9; then
    log "আগের ব্যাকআপ এখনো চলছে, বাদ দিলাম।"
    exit 1
fi

# ---------- ব্যাকআপ ডিস্ক মাউন্ট আছে তো? ----------
if ! mountpoint -q "$BACKUP_MNT"; then
    log "ত্রুটি: $BACKUP_MNT মাউন্ট করা নেই!"
    exit 2
fi

mkdir -p "$DEST"
log "ব্যাকআপ শুরু: $NEW"

# আগের snapshot থাকলে link-dest দাও
LINK_OPT=()
if [[ -d "$LATEST" ]]; then
    LINK_OPT=(--link-dest="$(readlink -f "$LATEST")")
fi

# ---------- rsync ----------
rc=0
rsync -aHAX --numeric-ids --delete \
      --exclude-from="$EXCLUDE_FILE" \
      "${LINK_OPT[@]}" \
      -R "${SRC_DIRS[@]}" "$NEW/" >>"$LOG" 2>&1 || rc=$?

# rc 24 = কপির সময় কিছু ফাইল উধাও হয়েছে (চলমান সিস্টেমে স্বাভাবিক)
if [[ $rc -ne 0 && $rc -ne 24 ]]; then
    log "rsync ব্যর্থ (exit $rc)। অসম্পূর্ণ snapshot মুছে দিলাম।"
    rm -rf "$NEW"
    exit $rc
fi

touch "$NEW"          # snapshot ফোল্ডারের সময় = আজ (retention এর জন্য)
ln -sfn "$NEW" "$LATEST"
log "ব্যাকআপ সফল (exit $rc)।"

# ---------- পুরনো snapshot মোছা ----------
find "$DEST" -mindepth 1 -maxdepth 1 -type d -name '20*' \
     -mtime +"$KEEP_DAYS" -print -exec rm -rf {} + | tee -a "$LOG"

log "শেষ। ব্যবহৃত জায়গা: $(df -h --output=used,avail "$BACKUP_MNT" | tail -1)"
```

exclude ফাইল:

```
$ sudo mkdir -p /etc/backup
$ sudo tee /etc/backup/exclude.txt >/dev/null <<'X'
/home/*/.cache/
/home/*/.local/share/Trash/
/home/*/Downloads/
node_modules/
*.tmp
*.swp
X
```

> 📝 **মনে রাখো:** এখানে `-R` (relative) ব্যবহার করেছি, তাই exclude প্যাটার্নের শুরুর `/` পূর্ণ পাথের সাথে মেলে (যেমন `/home/*/.cache/`)। আর snapshot-এর ভেতরে কাঠামো হবে `2026-11-12_0230/home/...`, `.../etc/...`।

স্ক্রিপ্টের গুরুত্বপূর্ণ কৌশলগুলো:

কৌশল	কেন
`set -euo pipefail`	কোনো কমান্ড ব্যর্থ হলে স্ক্রিপ্ট থামবে, অসংজ্ঞায়িত ভ্যারিয়েবল ধরা পড়বে
`flock -n`	cron আগেরটা শেষ হওয়ার আগে আবার চালালে দুটো একসাথে চলবে না
`mountpoint -q`	ডিস্ক মাউন্ট না থাকলে root ফাইলসিস্টেমে ব্যাকআপ লিখে সেটা ভরিয়ে ফেলবে না
exit code 24 সহ্য করা	চলমান সিস্টেমে কপির সময় ফাইল মুছে গেলে rsync 24 দেয়, এটা আসল ব্যর্থতা না
ব্যর্থ হলে অসম্পূর্ণ snapshot মোছা	ভাঙা snapshot-কে `latest` বানানো এড়ানো
retention-এর জন্য `find -mtime`	পুরনো snapshot আপনা-আপনি মুছে জায়গা খালি রাখা

rsync-এর কিছু সাধারণ exit code:

কোড	মানে
0	সফল
1	syntax বা ব্যবহারের ভুল
11	ফাইল I/O ত্রুটি
12	rsync প্রোটোকল ডেটা স্ট্রিমে ত্রুটি
23	আংশিক ট্রান্সফার (কিছু ফাইলে ত্রুটি, যেমন পারমিশন)
24	আংশিক ট্রান্সফার — সোর্স ফাইল উধাও হয়েছে
30	ডেটা পাঠানো/গ্রহণে timeout
255	সাধারণত SSH সংযোগ ব্যর্থ (এটা ssh-এর কোড)

অনুমতি দাও ও পরীক্ষা করো

```
$ sudo chmod 750 /usr/local/bin/snapshot-backup.sh
$ sudo /usr/local/bin/snapshot-backup.sh
[2026-11-12 02:30:01] ব্যাকআপ শুরু: /mnt/backup/snapshots/2026-11-12_0230
[2026-11-12 02:34:17] ব্যাকআপ সফল (exit 0)।
[2026-11-12 02:34:18] শেষ। ব্যবহৃত জায়গা:   38G  420G
```

cron-এ বসাও

```
$ sudo crontab -e
```

```
# মিনিট ঘণ্টা দিন মাস সপ্তাহের-দিন  কমান্ড
30 2 * * *  /usr/local/bin/snapshot-backup.sh >/dev/null 2>&1
```

প্রতিদিন রাত ২:৩০-এ চলবে। স্ক্রিপ্ট নিজেই লগ লেখে, তাই cron-এর আউটপুট `/dev/null` এ ফেলেছি। কাজের পরিবেশে ব্যর্থ হলে ইমেইল বা নোটিফিকেশন পাঠানো উচিত (যেমন cron-এর `MAILTO=` সেট করে আউটপুট রেখে দেওয়া)।

> ⚠️ **সতর্কতা:** cron খুব সীমিত `PATH` আর এনভায়রনমেন্ট নিয়ে চলে। তাই স্ক্রিপ্টে পূর্ণ পাথ ব্যবহার করো, আর রিমোট ব্যাকআপে SSH key পাসফ্রেজবিহীন (বা ssh-agent ছাড়াই চলতে পারে এমন) হতে হবে।

systemd timer বিকল্প

আধুনিক সিস্টেমে systemd timer (অধ্যায় ২৬, অধ্যায় ২৯) এর সুবিধা: লগ `journalctl` এ জমা হয়, আর `Persistent=true` থাকলে মেশিন বন্ধ থাকায় বাদ পড়া রান পরে চালিয়ে নেয়।

`/etc/systemd/system/snapshot-backup.service`:

```
[Unit]
Description=Daily rsync snapshot backup

[Service]
Type=oneshot
ExecStart=/usr/local/bin/snapshot-backup.sh
Nice=10
IOSchedulingClass=idle
```

`/etc/systemd/system/snapshot-backup.timer`:

```
[Unit]
Description=Run snapshot backup daily

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true
RandomizedDelaySec=10min

[Install]
WantedBy=timers.target
```

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now snapshot-backup.timer
$ systemctl list-timers snapshot-backup.timer
NEXT                         LEFT     LAST  PASSED  UNIT                    ACTIVATES
Fri 2026-11-13 02:34:51 +06 14h left -     -       snapshot-backup.timer snapshot-
backup.service
$ journalctl -u snapshot-backup.service --since today
```

`Nice=10` ও `IOSchedulingClass=idle` ব্যাকআপকে কম অগ্রাধিকার দেয়, ফলে চলার সময় অন্য কাজ ধীর হয় না (অধ্যায় ৪৬)।

৩৫.১১ আধুনিক ব্যাকআপ টুল — restic ও borg

rsync snapshot চমৎকার, কিন্তু কিছু ঘাটতি আছে: এনক্রিপশন নেই, ফাইল-লেভেলের বাইরে deduplication নেই, ক্লাউডে সরাসরি পাঠানো যায় না। এই সমস্যার সমাধান **restic** আর **borg (BorgBackup)**।

ভেতরে কীভাবে কাজ করে — Content-defined chunking ও Deduplication

দুটো টুলই একই মূলনীতিতে চলে:

```
ফাইল ──► ছোট টুকরা (chunk) এ ভাগ ──► প্রতি chunk এর hash ──► রিপোজিটরিতে আছে?
         (বিষয়বস্তু দেখে সীমানা ঠিক          (SHA-256 ইত্যাদি)        ├─ হ্যাঁ: শুধু রেফারেন্স রাখো
          করে — rolling hash দিয়ে)                                      └─ না: কম্প্রেস + এনক্রিপ্ট করে রাখো

Snapshot = "কোন ফাইল কোন কোন chunk দিয়ে তৈরি" এর একটা তালিকা (tree)

রিপোজিটরি:
┌──────────────────────────────────────────┐
│ chunks: [c1][c2][c3][c4][c5][c6] ...     │ ← প্রতিটা অনন্য chunk একবারই
│ snapshot সোম: fileA=c1,c2  fileB=c3      │
│ snapshot মঙ্গল: fileA=c1,c2  fileB=c3,c6 │ ← নতুন শুধু c6
└──────────────────────────────────────────┘
```

- **Content-defined chunking:** ফাইলকে নির্দিষ্ট সাইজে না কেটে, বিষয়বস্তুর একটা rolling hash দেখে সীমানা ঠিক করা হয়। ফলে ফাইলের মাঝে কিছু ঢুকলেও শুধু আশেপাশের chunk বদলায়, বাকিগুলো আগের মতোই মেলে (rsync-এর রোলিং আইডিয়ার আত্মীয়)।
- **Deduplication:** একই chunk যত ফাইলে, যত snapshot-এ, এমনকি যত কম্পিউটারে (একই repo শেয়ার করলে) থাকুক — জমা থাকে একবার। ২০ GB VM ইমেজে ১০ MB বদলালে নতুন জায়গা লাগে মোটামুটি ওই বদলানো অংশটুকুই।
- **Encryption:** ডেটা তোমার মেশিনেই এনক্রিপ্ট হয়ে রিপোজিটরিতে যায়। তাই অবিশ্বস্ত ক্লাউড স্টোরেজেও রাখা নিরাপদ। পাসওয়ার্ড/কী হারালে ব্যাকআপও হারাবে!

restic

একটাই বাইনারি (Go ভাষায় লেখা), অনেক ধরনের ব্যাকএন্ড: লোকাল ডিরেক্টরি, SFTP, S3-সামঞ্জস্যপূর্ণ স্টোরেজ, Backblaze B2, Azure, Google Cloud, REST server, এবং rclone হয়ে আরও অনেক। এনক্রিপশন সবসময় চালু (বন্ধ করার উপায় নেই)।

```
$ sudo apt install restic          # Ubuntu/Debian
$ sudo dnf install restic          # Fedora (RHEL এ EPEL লাগে)

# ১. রিপোজিটরি তৈরি
```

```
$ restic init --repo /mnt/backup/restic-repo
enter password for new repository:
enter password again:
created restic repository 3a9f2c1b8e at /mnt/backup/restic-repo

Please note that knowledge of your password is required to access
the repository. Losing your password means that your data is
irrecoverably lost.
```

বারবার `--repo` আর পাসওয়ার্ড টাইপ না করতে এনভায়রনমেন্ট ভ্যারিয়েবল:

```
$ export RESTIC_REPOSITORY=/mnt/backup/restic-repo
$ export RESTIC_PASSWORD_FILE=~/.config/restic/password   # chmod 600 করে রাখো
```

```
# ২. ব্যাকআপ
$ restic backup ~/Documents ~/Pictures --exclude-file=~/.rsync-exclude
repository 3a9f2c1b opened (version 2, compression level auto)
...
Files:        1840 new,     0 changed,     0 unmodified
Dirs:          212 new,     0 changed,     0 unmodified
Added to the repository: 2.034 GiB (1.871 GiB stored)

processed 1840 files, 2.112 GiB in 0:41
snapshot 5c1e9d07 saved

# ৩. snapshot তালিকা
$ restic snapshots
ID        Time                 Host     Tags  Paths
------------------------------------------------------------------
5c1e9d07  2026-11-12 10:15:02  laptop         /home/rahim/Documents
                                              /home/rahim/Pictures
------------------------------------------------------------------
1 snapshots

# ৪. Restore
$ restic restore latest --target /tmp/restore-test
$ restic restore 5c1e9d07 --target /tmp/r --include /home/rahim/Documents/thesis.docx

# ৫. ব্রাউজ করার জন্য মাউন্ট (FUSE লাগে)
$ mkdir /tmp/rmnt && restic mount /tmp/rmnt

# ৬. পুরনো snapshot ছাঁটাই (retention)
$ restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 12 --prune

# ৭. রিপোজিটরির স্বাস্থ্য পরীক্ষা
$ restic check
$ restic check --read-data-subset=10%    # ১০% ডেটা আসলেই পড়ে যাচাই
```

রিমোট SFTP বা S3:

```
$ restic -r sftp:backup@bkp:/srv/restic-repo init
$ export AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=...
$ restic -r s3:s3.amazonaws.com/my-bucket/laptop init
```

> 📝 **মনে রাখো:** `forget` শুধু snapshot-এর রেকর্ড মোছে; আসল ডেটা chunk মুক্ত করে `prune`। তাই `forget --prune` একসাথে দাও, অথবা পরে আলাদা `restic prune` চালাও।

borg (BorgBackup)

Python ও C তে লেখা, খুব দক্ষ কম্প্রেশন ও deduplication। রিমোট ব্যাকআপ শুধু SSH দিয়ে (সার্ভারেও borg ইনস্টল থাকতে হয়)। নিচের কমান্ডগুলো borg 1.2/1.4 সিরিজের (বেশিরভাগ distro-তে এটাই আছে; borg 2.0 এর syntax আলাদা)।

```
$ sudo apt install borgbackup        # Ubuntu/Debian
$ sudo dnf install borgbackup        # Fedora (RHEL এ EPEL)

# ১. রিপোজিটরি তৈরি (repokey = কী রিপোজিটরিতে, পাসফ্রেজ দিয়ে সুরক্ষিত)
$ borg init --encryption=repokey /mnt/backup/borg-repo

# কী আলাদা করে ব্যাকআপ রাখো! (repo-র কী নষ্ট হলে সব শেষ)
$ borg key export /mnt/backup/borg-repo ~/borg-key-backup.txt

# ২. ব্যাকআপ (archive) তৈরি — {hostname} ও {now} আপনা-আপনি বসে যায়
$ borg create --stats --progress --compression zstd \
      /mnt/backup/borg-repo::'{hostname}-{now:%Y-%m-%d}' \
      ~/Documents ~/Pictures --exclude '*/.cache'
------------------------------------------------------------------------------
Repository: /mnt/backup/borg-repo
Archive name: laptop-2026-11-12
...
                       Original size      Compressed size    Deduplicated size
This archive:                2.11 GB              1.89 GB              1.87 GB
All archives:                2.11 GB              1.89 GB              1.87 GB
------------------------------------------------------------------------------

# ৩. তালিকা
$ borg list /mnt/backup/borg-repo
laptop-2026-11-12                    Thu, 2026-11-12 10:20:11 [9b3c...]

# ৪. Restore — borg extract বর্তমান ডিরেক্টরিতে বের করে!
$ mkdir /tmp/borg-restore && cd /tmp/borg-restore
$ borg extract /mnt/backup/borg-repo::laptop-2026-11-12
home/rahim/Documents/thesis.docx

# ৫. মাউন্ট করে ব্রাউজ
$ borg mount /mnt/backup/borg-repo::laptop-2026-11-12 /tmp/bmnt
$ borg umount /tmp/bmnt

# ৬. Retention ও জায়গা খালি করা
$ borg prune --keep-daily=7 --keep-weekly=4 --keep-monthly=12 /mnt/backup/borg-repo
$ borg compact /mnt/backup/borg-repo        # borg 1.2+ এ prune এর পর জায়গা আসলে খালি
করে

# ৭. যাচাই
$ borg check /mnt/backup/borg-repo
```

রিমোট: `borg init --encryption=repokey ssh://backup@bkp/./borg-repo`।

> ⚠️ **সতর্কতা:** `borg extract` আর্কাইভের ভেতরের পাথ (সামনের `/` ছাড়া) **বর্তমান ডিরেক্টরিতে** বের করে। `/` এ দাঁড়িয়ে extract চালালে তোমার চলমান ফাইল ওভাররাইট হয়ে যাবে। সবসময় একটা খালি ডিরেক্টরিতে `cd` করে নাও, প্রয়োজনে আগে `--dry-run --list` দাও।

তুলনা

বৈশিষ্ট্য	rsync snapshot	restic	borg
Deduplication	ফাইল-লেভেল (hard link)	chunk-লেভেল	chunk-লেভেল
Encryption	নেই	সবসময় চালু	ঐচ্ছিক (সুপারিশকৃত)
Compression	নেই (`-z` শুধু ট্রান্সফারে)	আছে (repo v2)	আছে (lz4/zstd/zlib/lzma)
ক্লাউড (S3, B2) সরাসরি	না	হ্যাঁ	না (SSH শুধু; ক্লাউডে BorgBase-এর মতো সেবা)
Restore সহজতা	সরাসরি `cp`, কোনো টুল লাগে না	`restic restore` / `mount`	`borg extract` / `mount`
রিমোট প্রান্তে কী লাগে	rsync + ssh	কিছু না (শুধু স্টোরেজ)	borg + ssh

GUI পছন্দ করলে: **Déjà Dup** (GNOME-এর ডিফল্ট ব্যাকআপ, ভেতরে duplicity/restic), **Vorta** (borg-এর GUI), **Timeshift** (সিস্টেম ফাইলের snapshot — rsync বা Btrfs মোডে; এটা মূলত সিস্টেম রোলব্যাকের জন্য, ব্যক্তিগত ফাইলের জন্য না)।

৩৫.১২ Restore পরীক্ষা — ব্যাকআপের আসল পরীক্ষা

> "যে ব্যাকআপ restore করে দেখা হয়নি, সেটা ব্যাকআপ না — সেটা একটা আশা।"

বহু প্রতিষ্ঠান বছরের পর বছর ব্যাকআপ নিয়েছে, তারপর দুর্ঘটনার দিন আবিষ্কার করেছে যে স্ক্রিপ্ট মাস ধরে নীরবে ব্যর্থ হচ্ছিল, বা ব্যাকআপে দরকারি ফোল্ডারটাই ছিল না, বা এনক্রিপশন পাসওয়ার্ড কেউ জানে না।

কী কী পরীক্ষা করবে

পরীক্ষা	কীভাবে
ব্যাকআপ আসলে চলছে তো?	লগ দেখো, শেষ snapshot-এর তারিখ দেখো (`ls -lt`, `restic snapshots`)
ব্যাকআপের অখণ্ডতা	`restic check --read-data`, `borg check --verify-data`

পরীক্ষা	কীভাবে
একটা ফাইল ফেরত আনা	এলোমেলো একটা ফাইল restore করে আসলটার সাথে মেলাও
পুরো ডিরেক্টরি ফেরত আনা	আলাদা জায়গায় restore করে `diff -r` বা `rsync -n` দিয়ে তুলনা
পুরো সিস্টেম	একটা ফাঁকা VM-এ (অধ্যায় ৪৮) সার্ভার পুরোটা নতুন করে দাঁড় করাও
ডেটাবেস	ডাম্পটা টেস্ট ডেটাবেসে import করে query চালাও
কী/পাসওয়ার্ড	এনক্রিপশন পাসওয়ার্ড আর কী ব্যাকআপ আলাদা নিরাপদ জায়গায় আছে কি না
সময় (RTO)	পুরো restore-এ কত সময় লাগল, সেটা লিখে রাখো

Restore যাচাইয়ের কৌশল

```
# ১. আলাদা জায়গায় restore
$ rsync -a /mnt/backup/snapshots/latest/home/rahim/Documents/ /tmp/verify/

# ২. checksum দিয়ে বিষয়বস্তু তুলনা (dry-run + itemize; শুধু পার্থক্য থাকা ফাইল ছাপে)
$ rsync -rcni ~/Documents/ /tmp/verify/
(কোনো ফাইলের লাইন না এলে সব মিলেছে)

# ৩. বা diff
$ diff -rq ~/Documents/ /tmp/verify/ && echo "সব ফাইল মিলেছে"
সব ফাইল মিলেছে

# ৪. sha256 তালিকা বানিয়ে পরে যাচাই
$ cd ~/Documents && find . -type f -exec sha256sum {} + > /tmp/sums.txt
$ cd /tmp/verify && sha256sum -c --quiet /tmp/sums.txt && echo OK
OK
```

dd ইমেজ যাচাই:

```
$ sudo sha256sum /dev/sdb /backup/sdb.img
# দুটো hash একই হলে ক্লোন নিখুঁত (ডিস্ক ও ইমেজ সমান আকারের হতে হবে)

# ইমেজের ভেতরের পার্টিশন দেখা ও মাউন্ট করে ফাইল পরীক্ষা
$ sudo losetup -fP --show /backup/sdb.img
/dev/loop0
$ sudo mount -o ro /dev/loop0p1 /mnt/check
$ ls /mnt/check
$ sudo umount /mnt/check && sudo losetup -d /dev/loop0
```

> 💡 **টিপস:** ক্যালেন্ডারে প্রতি মাসে একটা "restore day" রাখো। ছোট প্রতিষ্ঠানেও অন্তত ত্রৈমাসিক পুরো সিস্টেম restore অনুশীলন করা উচিত, আর একটা লিখিত **restore runbook** (ধাপে ধাপে নির্দেশনা) রাখা উচিত — যাতে তুমি না থাকলেও অন্য কেউ পারে।

ব্যাকআপ নিরাপত্তা

- ব্যাকআপে তোমার সব গোপন তথ্য থাকে, তাই ব্যাকআপ ডিরেক্টরির পারমিশন কঠোর রাখো (`chmod 700`) এবং offsite কপি এনক্রিপ্ট করো (অধ্যায় ৪৫)।
- **Ransomware সুরক্ষা:** যে মেশিন আক্রান্ত হবে, সে যদি ব্যাকআপ মুছতে পারে, ব্যাকআপও যাবে। সমাধান: ব্যাকআপ সার্ভার নিজে **pull** করুক (উৎস মেশিন ব্যাকআপে লিখতে পারবে, মুছতে পারবে না), append-only মোড (`borg serve --append-only`, restic `rest-server --append-only`), বা S3 Object Lock-এর মতো immutable স্টোরেজ, বা খুলে রাখা (offline) ডিস্ক।

হাতে-কলমে অনুশীলন

> ⚠️ এই ল্যাবগুলো শুধু `~/lab35` আর একটা লুপ ইমেজ ফাইল নিয়ে কাজ করে, কোনো আসল ডিস্ক ছোঁয় না।

ল্যাব ১: rsync-এর মূল আচরণ

```
$ mkdir -p ~/lab35/src/{docs,img,cache} ~/lab35/dst
$ for i in 1 2 3; do echo "file $i" > ~/lab35/src/docs/f$i.txt; done
$ head -c 1M /dev/urandom > ~/lab35/src/img/photo.jpg
$ echo junk > ~/lab35/src/cache/tmp.dat

# ক. dry-run দিয়ে দেখো
$ rsync -avhn ~/lab35/src/ ~/lab35/dst/
# খ. আসল কপি
$ rsync -avh ~/lab35/src/ ~/lab35/dst/
# গ. আবার চালাও — কিছু কপি হলো কি?
$ rsync -avh ~/lab35/src/ ~/lab35/dst/
# ঘ. একটা ফাইল বদলাও, itemize দেখো
$ echo "changed" >> ~/lab35/src/docs/f2.txt
$ rsync -ai ~/lab35/src/ ~/lab35/dst/
```

প্রশ্ন: (গ) ধাপে কোনো ফাইলের নাম এল না কেন? (ঘ) ধাপে `>f.st......` এর `s` আর `t` কী বোঝায়?

ল্যাব ২: trailing slash ও `--delete`

```
$ rsync -a ~/lab35/src  ~/lab35/slash-no/
$ rsync -a ~/lab35/src/ ~/lab35/slash-yes/
$ ls ~/lab35/slash-no ~/lab35/slash-yes

$ rm ~/lab35/src/docs/f3.txt
$ rsync -av --delete --exclude='cache/' -n ~/lab35/src/ ~/lab35/dst/
# 'deleting docs/f3.txt' দেখাচ্ছে? cache/ এর কী হলো?
$ rsync -av --delete --exclude='cache/' ~/lab35/src/ ~/lab35/dst/
```

লক্ষ করো: exclude করা `cache/` গন্তব্যে থেকে গেছে, কারণ exclude করা জিনিস `--delete` ও ছোঁয় না (যদি না `--delete-excluded` দাও)।

ল্যাব ৩: `--link-dest` snapshot

```
$ S=~/lab35/snaps
$ mkdir -p $S
$ rsync -a ~/lab35/src/ $S/day1/
$ echo "new file" > ~/lab35/src/docs/f4.txt
$ rsync -a --link-dest=$S/day1 ~/lab35/src/ $S/day2/
$ ls -li $S/day1/img/photo.jpg $S/day2/img/photo.jpg   # একই inode?
$ du -sh $S/day1 $S/day2                               # day2 কত ছোট?
$ rm -rf $S/day1
$ ls -l $S/day2/img/photo.jpg                          # এখনো আছে?
```

ল্যাব ৪: dd নিরাপদে (লুপ ডিভাইস দিয়ে)

```
$ dd if=/dev/zero of=~/lab35/disk.img bs=1M count=64 status=progress
$ mkfs.ext4 -q ~/lab35/disk.img
$ sudo mkdir -p /mnt/lab35 && sudo mount -o loop ~/lab35/disk.img /mnt/lab35
$ echo "important" | sudo tee /mnt/lab35/secret.txt
$ sudo umount /mnt/lab35

# ক্লোন ও যাচাই
$ dd if=~/lab35/disk.img of=~/lab35/clone.img bs=4M status=progress
$ sha256sum ~/lab35/disk.img ~/lab35/clone.img
$ sudo mount -o loop,ro ~/lab35/clone.img /mnt/lab35 && cat /mnt/lab35/secret.txt
$ sudo umount /mnt/lab35

# bs এর প্রভাব মাপো
$ time dd if=~/lab35/disk.img of=/dev/null bs=512
$ time dd if=~/lab35/disk.img of=/dev/null bs=4M
```

ল্যাব ৫: restic দিয়ে ব্যাকআপ ও restore পরীক্ষা

```
$ sudo apt install restic        # বা dnf
$ export RESTIC_REPOSITORY=~/lab35/restic-repo RESTIC_PASSWORD=lab-only-pass
$ restic init
$ restic backup ~/lab35/src
$ echo "v2" >> ~/lab35/src/docs/f1.txt
$ restic backup ~/lab35/src
$ restic snapshots
$ restic diff $(restic snapshots --json | python3 -c 'import
sys,json;s=json.load(sys.stdin);print(s[0]["short_id"],s[1]["short_id"])')
$ restic restore latest --target ~/lab35/restored
$ diff -r ~/lab35/src ~/lab35/restored$HOME/lab35/src && echo "Restore মিলেছে"
$ restic check
```

(শুধু ল্যাবে পাসওয়ার্ড এনভায়রনমেন্ট ভ্যারিয়েবলে রেখেছি; আসল কাজে `RESTIC_PASSWORD_FILE` ব্যবহার করো।)

ল্যাব ৬: নিজের স্ক্রিপ্ট

৩৫.১০ এর স্ক্রিপ্টটা কপি করে `SRC_DIRS=("$HOME/lab35/src")`, `BACKUP_MNT` হিসেবে একটা লুপ-মাউন্ট করা ইমেজ দিয়ে চালাও। তারপর:

1. দুইবার চালিয়ে দেখো দ্বিতীয় snapshot কত ছোট।
2. দুটো টার্মিনালে একসাথে চালিয়ে দেখো `flock` কাজ করে কি না।
3. ইমেজ আনমাউন্ট করে চালিয়ে দেখো `mountpoint` চেক ধরছে কি না।
4. `KEEP_DAYS=0` দিয়ে `touch -d '3 days ago'` করা একটা পুরনো snapshot মুছছে কি না দেখো।

শেষে পরিষ্কার: `rm -rf ~/lab35`।

সারসংক্ষেপ

rsync দ্রুত রেফারেন্স

কমান্ড	কাজ
`rsync -avh src/ dst/`	ডিরেক্টরির ভেতরটা কপি, বিস্তারিত দেখাও
`rsync -avhn ...`	dry-run — শুধু দেখাও
`rsync -avh --delete src/ dst/`	হুবহু আয়না
`rsync -avhP file host:/path/`	রিমোটে, অগ্রগতি সহ, থামলে আবার চালানো যায়
`rsync -avz -e "ssh -p 2222" src/ u@h:/d/`	ভিন্ন SSH পোর্টে, কম্প্রেস করে
`rsync -a --exclude='*.log' --exclude-from=F ...`	কিছু বাদ দিয়ে
`rsync -aHAX --numeric-ids ...`	পূর্ণ সিস্টেম metadata সহ
`rsync -a --link-dest=/abs/prev src/ new/`	hard-link snapshot
`rsync -ah --info=progress2 ...`	মোট অগ্রগতি
`rsync -rcni a/ b/`	checksum দিয়ে পার্থক্য যাচাই (কিছুই বদলায় না)

dd দ্রুত রেফারেন্স

কমান্ড	কাজ
`lsblk`	**আগে** ডিস্ক চেনো
`sudo dd if=x.iso of=/dev/sdX bs=4M status=progress oflag=sync`	বুটেবল USB
`sudo dd if=/dev/sdX of=disk.img bs=4M status=progress`	ডিস্ক ইমেজ
`sudo dd if=/dev/sdX of=mbr.bin bs=512 count=1`	MBR ব্যাকআপ
`dd if=/dev/zero of=f bs=1M count=100`	১০০ MB ফাইল

কমান্ড	কাজ
`sudo kill -USR1 $(pgrep -x dd)`	চলমান dd এর অগ্রগতি
`sudo ddrescue -d /dev/sdX img map`	নষ্ট ডিস্ক থেকে উদ্ধার

restic / borg দ্রুত রেফারেন্স

restic	borg (1.x)	কাজ
`restic init`	`borg init -e repokey REPO`	রিপোজিটরি তৈরি
`restic backup PATH`	`borg create REPO::NAME PATH`	ব্যাকআপ
`restic snapshots`	`borg list REPO`	তালিকা
`restic restore ID --target DIR`	`cd DIR && borg extract REPO::NAME`	restore
`restic mount DIR`	`borg mount REPO::NAME DIR`	ব্রাউজ
`restic forget --keep-daily 7 --prune`	`borg prune --keep-daily=7 REPO` + `borg compact REPO`	retention
`restic check`	`borg check REPO`	যাচাই

দরকারি কীবোর্ড শর্টকাট ও কৌশল

শর্টকাট/কৌশল	কাজ
`Ctrl+C`	চলমান rsync/dd থামাও (rsync `--partial` থাকলে আধা ফাইল রাখে; ডিস্কে dd থামালে ডিস্ক অসম্পূর্ণ থাকে)
`Ctrl+Z` তারপর `bg`	চলমান ব্যাকআপ ব্যাকগ্রাউন্ডে পাঠাও (অধ্যায় ২৫)
`Ctrl+R` তারপর `rsync`	হিস্ট্রি থেকে আগের rsync কমান্ড খুঁজে বের করো (অধ্যায় ৬)
`Alt+.`	আগের কমান্ডের শেষ আর্গুমেন্ট (যেমন গন্তব্য পাথ) বসাও
`!!:s/ -n//`	আগের dry-run কমান্ডটাই `-n` ছাড়া আবার চালাও (যদি `-n` আলাদা অপশন হিসেবে লেখা থাকে; প্রথম মিলটাই বদলায়)
`tmux` / `screen`	লম্বা রিমোট ব্যাকআপ SSH কেটে গেলেও চলতে থাকবে (অধ্যায় ২৫); tmux থেকে বের হতে `Ctrl+b` তারপর `d`

মূল নীতি

- 3-2-1: ৩ কপি, ২ মিডিয়া, ১ offsite (+১ offline, ০ ত্রুটি)
- সিঙ্ক ≠ ব্যাকআপ; RAID ≠ ব্যাকআপ
- `src/` = ভেতরের জিনিস; `src` = ডিরেক্টরিটা নিজে
- `--delete` আর `dd` এর আগে সবসময় dry-run / `lsblk`

- Restore পরীক্ষা না করা ব্যাকআপ = ব্যাকআপ না

নিজেকে যাচাই করো

1. Incremental আর Differential ব্যাকআপের মধ্যে restore করার সময় কোনটায় কম সেট লাগে, এবং কেন?
2. 3-2-1 নিয়মটা ব্যাখ্যা করো। শুধু একটা বাহ্যিক USB ডিস্কে ব্যাকআপ রাখলে কোন অংশ পূরণ হয় না?
3. `rsync -a` কোন কোন অপশনের সমষ্টি? কোন তিনটা জিনিস এটা রাখে না?
4. `rsync -a photos /backup/` আর `rsync -a photos/ /backup/` চালালে `/backup` এর ভেতরে কী কাঠামো হবে?
5. rsync কীভাবে বোঝে কোন ফাইল বদলায়নি, যাতে সেগুলো আর ছোঁয় না? `-c` দিলে কী বদলায়?
6. delta algorithm-এ "rolling checksum" কেন দরকার? শুধু নির্দিষ্ট ব্লক অবস্থানে তুলনা করলে কী সমস্যা হতো?
7. দুটো লোকাল ডিরেক্টরির মধ্যে rsync করলে কি delta algorithm ব্যবহার হয়?
8. `--link-dest` দিয়ে বানানো snapshot-এর একটা পুরনো ফোল্ডার মুছলে নতুন snapshot-এর ফাইল কেন নষ্ট হয় না? এটা FAT32 ডিস্কে কেন চলবে না?
9. ISO লেখার সময় `of=/dev/sdb1` লিখলে কী ভুল হবে? `bs=4M` কেন দেওয়া হয়?
10. restic/borg-এর deduplication rsync snapshot-এর চেয়ে কোথায় ভালো? restic-এ `forget` আর `prune` এর পার্থক্য কী?

<details>
<summary>▼ উত্তর দেখো</summary>

1. **Differential**-এ কম লাগে: শুধু শেষ full + শেষ differential (২টা সেট)। কারণ প্রতিটা differential-এ full-এর পর থেকে সব পরিবর্তন জমা থাকে। Incremental-এ full + তার পরের **সব** incremental ক্রমানুসারে লাগে।
2. ডেটার ৩টা কপি (আসল + ২ ব্যাকআপ), ২টা ভিন্ন ধরনের মিডিয়ায়, ১টা অন্য ভৌগোলিক জায়গায়। শুধু একটা USB ডিস্ক থাকলে মোট কপি ২টা (৩ না), আর offsite কপি নেই — ঘরে আগুন বা চুরি হলে দুটোই যাবে।
3. `-rlptgoD` (recursive, links, perms, times, group, owner, devices+specials)। এটা hard link (`-H`), ACL (`-A`), extended attribute (`-X`) রাখে না।
4. প্রথমটায় `/backup/photos/a.jpg...` (ডিরেক্টরিটা নিজেই যায়); দ্বিতীয়টায় `/backup/a.jpg...` (শুধু ভেতরের জিনিস)।
5. ডিফল্ট "quick check": ফাইলের আকার আর modification time দুটো এক হলে ধরে নেয় বদলায়নি। `-c` দিলে প্রতিটা ফাইলের পুরো checksum গণনা করে তুলনা করে — বেশি নিশ্চিত, কিন্তু সব ফাইল পুরো পড়তে হয় বলে অনেক ধীর।
6. ফাইলের মাঝে কিছু বাইট যোগ বা বাদ হলে পরের সব ব্লক সরে যায়। নির্দিষ্ট অবস্থানে তুলনা করলে তখন আর কিছুই মিলত না, পুরো বাকি ফাইল পাঠাতে হতো। Rolling checksum প্রতিটা বাইট অবস্থানে দ্রুত (আগের মান থেকে এক বাইট বাদ, এক বাইট যোগ করে) গণনা করা যায়, ফলে সরে যাওয়া ব্লকও খুঁজে পাওয়া যায়।
</details>

7. না। দুটো পাথই লোকাল হলে rsync ডিফল্টে `--whole-file` ব্যবহার করে, কারণ লোকালে সরাসরি কপি দ্রুত। তবে quick check দিয়ে অপরিবর্তিত ফাইল বাদ দেওয়া লোকালেও হয়।
8. কারণ অপরিবর্তিত ফাইলগুলো একই inode-এর hard link; ডেটা মুছে যায় কেবল তখন যখন শেষ নামটাও মোছা হয়। পুরনো ফোল্ডার মুছলে শুধু link count কমে। FAT32/exFAT ফাইলসিস্টেম hard link সমর্থন করে না, আর link-dest ও গন্তব্য একই ফাইলসিস্টেমে থাকতে হয়।
9. `sdb1` একটা পার্টিশন; ISO-র নিজস্ব পার্টিশন টেবিল ও বুট কাঠামো পুরো ডিস্কের শুরু থেকে লিখতে হয়, তাই `of=/dev/sdb` দিতে হবে — নইলে USB বুট করবে না। `bs=4M` দিলে একবারে বড় অংশ পড়া-লেখা হয়, system call অনেক কমে যায়, ফলে ডিফল্ট ৫১২ বাইটের চেয়ে অনেক দ্রুত হয়।
10. restic/borg ফাইলকে বিষয়বস্তু-ভিত্তিক chunk-এ ভাগ করে dedup করে, তাই বড় ফাইলের সামান্য বদলে শুধু বদলানো chunk জমা হয়; rsync snapshot-এ এক বাইট বদলালেও পুরো ফাইল নতুন করে জায়গা নেয়। এছাড়া এনক্রিপশন ও কম্প্রেশন আছে। restic-এ `forget` শুধু snapshot-এর রেকর্ড মোছে; `prune` আর কোনো snapshot-এ ব্যবহৃত না হওয়া ডেটা আসলে মুছে জায়গা খালি করে।

অধ্যায় ৩৬: নেটওয়ার্কিং-এর মূলনীতি

> আজকের দিনে প্রায় কোনো কম্পিউটারই একা চলে না। তুমি ওয়েবসাইট খুলছ, `apt` দিয়ে সফটওয়্যার নামাচ্ছ, SSH দিয়ে সার্ভারে ঢুকছ — সবকিছুর পেছনে নেটওয়ার্ক। লিনাক্স সার্ভারের মূল কাজই নেটওয়ার্কে সার্ভিস দেওয়া। তাই নেটওয়ার্ক না বুঝলে লিনাক্স এক্সপার্ট হওয়া যায় না। এই অধ্যায়ে আমরা থিওরি আর লিনাক্সের কনফিগারেশন দুটোই দেখব। নেটওয়ার্ক কমান্ড দিয়ে ট্রাবলশুটিং আসবে পরের অধ্যায়ে (অধ্যায় ৩৭)।

এই অধ্যায়ে যা শিখবে

- নেটওয়ার্ক জিনিসটা কী, আর ডেটা কীভাবে এক কম্পিউটার থেকে আরেকটায় যায়
- OSI মডেল (৭ স্তর) আর TCP/IP মডেল (৪ স্তর), ডায়াগ্রাম সহ; encapsulation কী
- IP অ্যাড্রেস: IPv4 ও IPv6, কোনটা কেমন দেখতে, কোনটা কী কাজে লাগে
- subnet mask আর CIDR: হাতে হিসাব করে network, broadcast, host range বের করা
- private IP, public IP, loopback, link-local
- gateway আর routing table: প্যাকেট বাইরে যায় কীভাবে
- MAC অ্যাড্রেস আর ARP: IP থেকে হার্ডওয়্যার ঠিকানা খুঁজে বের করা
- DNS কীভাবে কাজ করে (resolution ডায়াগ্রাম), রেকর্ডের ধরন, TTL
- `/etc/hosts`, `/etc/nsswitch.conf`, `/etc/resolv.conf`, `systemd-resolved` আর `resolvectl`
- DHCP-র DORA প্রসেস আর lease
- port ও protocol: TCP বনাম UDP, 3-way handshake, well-known port-এর টেবিল
- NAT: বাসার এক রাউটারের পেছনে এতগুলো ডিভাইস ইন্টারনেট পায় কীভাবে
- লিনাক্সে নেটওয়ার্ক কনফিগ: NetworkManager (`nmcli`, `nmtui`), Netplan (Ubuntu), Debian-এর `/etc/network/interfaces`, RHEL/Fedora-র ধরন
- interface-এর নাম `enp0s3`, `wlp2s0` এরকম অদ্ভুত কেন হয়

৩৬.১ নেটওয়ার্ক কী? একটা ডাকঘরের গল্প

নেটওয়ার্ক হলো দুই বা তার বেশি ডিভাইস, যারা কোনো মাধ্যম (তার, Wi-Fi, ফাইবার) দিয়ে যুক্ত থেকে ডেটা আদান-প্রদান করে।

ভাবো তুমি ঢাকা থেকে চট্টগ্রামে বন্ধুকে চিঠি পাঠাচ্ছ:

চিঠির জগৎ	নেটওয়ার্কের জগৎ
চিঠির লেখা	ডেটা (ওয়েবপেজ, ইমেইল, ফাইল)
খাম	packet (প্যাকেট) — ডেটার টুকরো, সাথে হেডার
বাড়ির ঠিকানা	IP অ্যাড্রেস
ফ্ল্যাট নম্বর / কার কাছে	port নম্বর (কোন প্রোগ্রাম পাবে)
পাড়ার পিয়ন যে বাড়ি চেনে	MAC অ্যাড্রেস (লোকাল নেটওয়ার্কে হার্ডওয়্যার ঠিকানা)
পাড়ার ডাকঘর	gateway / router
ডাক বিভাগের রুট	routing
ফোনবুক (নাম থেকে ঠিকানা)	DNS
রেজিস্টার্ড চিঠি (প্রাপ্তিস্বীকার সহ)	TCP
সাধারণ পোস্টকার্ড (পৌঁছাল কি না জানা নেই)	UDP

পুরো অধ্যায়ে এই উপমাটা বারবার কাজে লাগবে।

নেটওয়ার্কের ধরন

ধরন	পুরো নাম	উদাহরণ
LAN	Local Area Network	বাসা বা অফিসের নেটওয়ার্ক
WLAN	Wireless LAN	Wi-Fi
WAN	Wide Area Network	শহর বা দেশজুড়ে নেটওয়ার্ক; ইন্টারনেট সবচেয়ে বড় WAN
VLAN	Virtual LAN	এক সুইচে সফটওয়্যার দিয়ে আলাদা আলাদা LAN
VPN	Virtual Private Network	ইন্টারনেটের ওপর এনক্রিপ্টেড "সুড়ঙ্গ"

মূল হার্ডওয়্যার

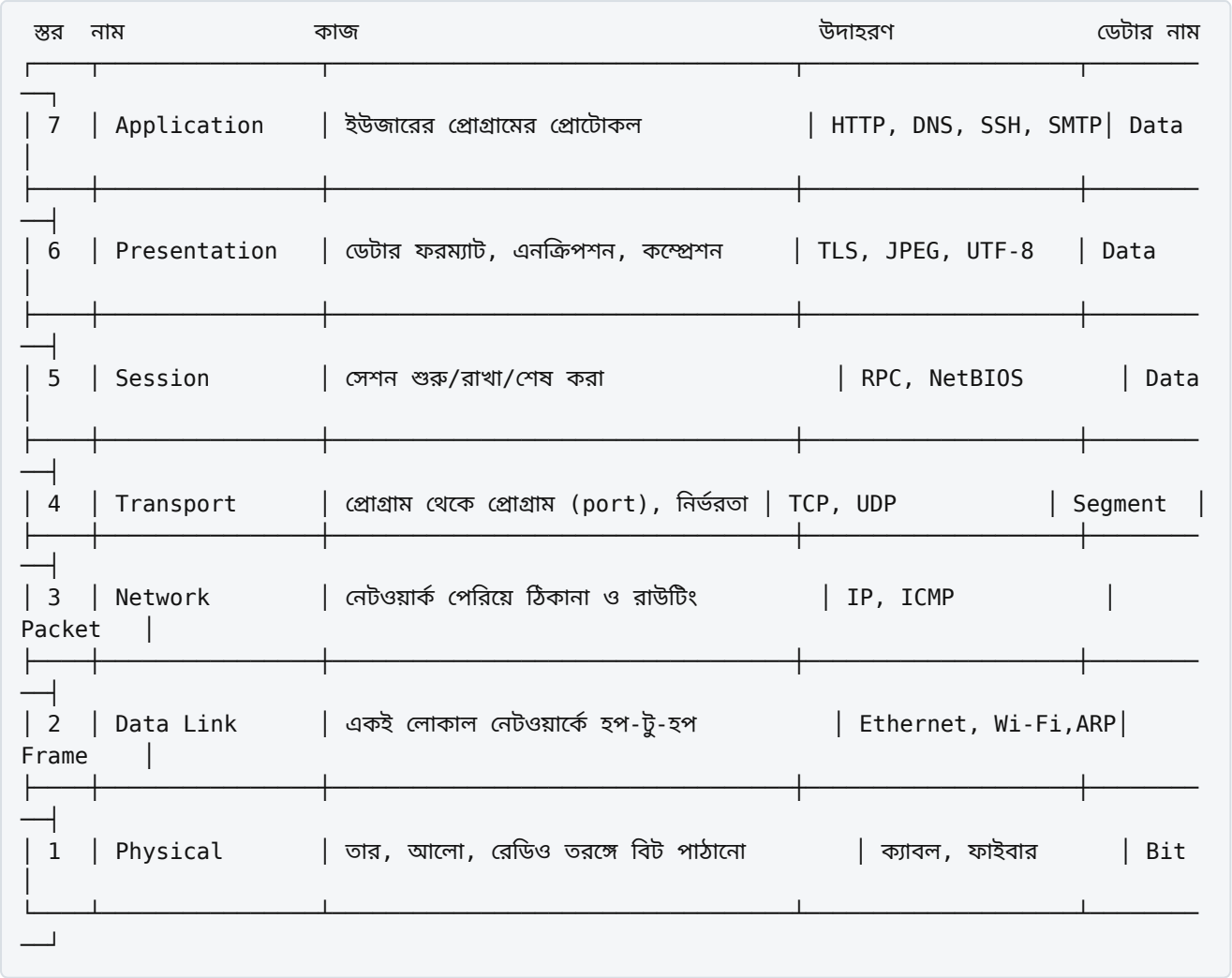
- **NIC (Network Interface Card)**: কম্পিউটারের নেটওয়ার্ক কার্ড, ইথারনেট বা Wi-Fi। লিনাক্সে এটা একটা **interface** হিসেবে দেখা যায় (যেমন `enp0s3`)।
- **Switch**: একই LAN-এর ডিভাইসগুলোকে জোড়ে। MAC অ্যাড্রেস দেখে ফ্রেম পাঠায় (Layer 2)।
- **Router**: আলাদা আলাদা নেটওয়ার্ককে জোড়ে। IP দেখে প্যাকেট পাঠায় (Layer 3)। তোমার বাসার "Wi-Fi রাউটার" আসলে router + switch + Wi-Fi access point + DHCP সার্ভার + NAT + ফায়ারওয়াল, সব এক বাক্সে।
- **Access Point**: Wi-Fi ডিভাইসকে তারের নেটওয়ার্কে যোগ করে।

৩৬.২ লেয়ার মডেল: OSI ও TCP/IP

নেটওয়ার্কিং অনেক জটিল, তাই কাজটাকে কয়েকটা **layer**-এ (স্তরে) ভাগ করা হয়েছে। প্রত্যেক স্তর নিজের কাজটুকু করে, নিচের স্তরের ওপর ভরসা রাখে, আর ওপরের স্তরকে সার্ভিস দেয়। এর ফলে Wi-Fi বদলে ইথারনেট লাগালে তোমার ব্রাউজার বদলাতে হয় না, কারণ সেটা অন্য স্তরের ব্যাপার।

OSI মডেল (৭ স্তর)

OSI (Open Systems Interconnection) একটা তাত্ত্বিক মডেল। বাস্তবে হুবহু এভাবে কিছু চলে না, কিন্তু সমস্যা নিয়ে কথা বলার সময় সবাই এর ভাষাই ব্যবহার করে ("এটা Layer 2 সমস্যা", "Layer 7 load balancer")।



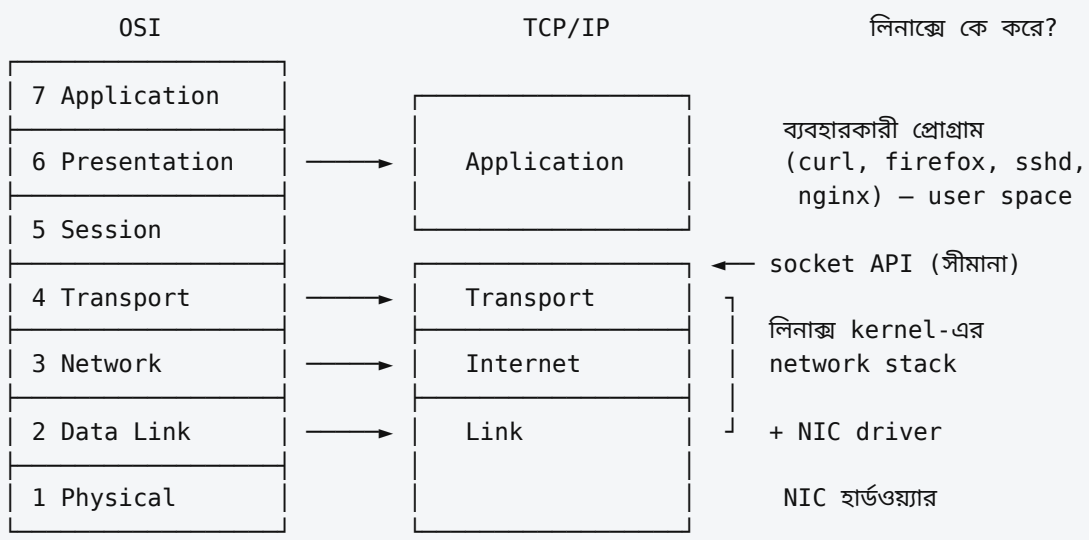
> 💡 **টিপস:** ওপর থেকে নিচে মনে রাখার ইংরেজি ছড়া: **"All People Seem To Need Data Processing"** (Application, Presentation, Session, Transport, Network, Data Link, Physical)।

> 📝 **মনে রাখো:** ARP নিয়ে বিতর্ক আছে। কেউ বলে Layer 2, কেউ বলে 2 আর 3-এর মাঝামাঝি, কারণ এটা IP (L3) থেকে MAC (L2) বের করে। ইন্টারভিউতে "Layer 2-তে কাজ করে, L3 আর L2-এর মধ্যে সংযোগ

> ঘটায়" বললেই যথেষ্ট।

TCP/IP মডেল (৪ স্তর): বাস্তবে যেটা চলে

ইন্টারনেট আসলে চলে TCP/IP মডেলে। OSI-র ৫, ৬, ৭ মিলে এখানে একটাই Application স্তর, আর ১ ও ২ মিলে Link স্তর।



এখানে লিনাক্সের আর্কিটেকচারের একটা জরুরি কথা লুকিয়ে আছে (অধ্যায় ২ মনে করো)। TCP, UDP, IP, routing, ফায়ারওয়াল (netfilter) সবই **kernel-এর ভেতরে** চলে। একটা প্রোগ্রাম (যেমন `curl`) শুধু **socket** খোলে। socket হলো kernel-এর দেওয়া একটা "ফোনের রিসিভার", যেটা দিয়ে প্রোগ্রাম system call-এর মাধ্যমে (`socket()`, `connect()`, `send()`, `recv()`) ডেটা দেয়-নেয়। বাকি সব kernel করে।

Encapsulation: খামের ভেতর খাম

ডেটা নিচে নামার সময় প্রত্যেক স্তর নিজের **header** জুড়ে দেয়, যেন একটা খামকে আরেকটা বড় খামে ভরা হচ্ছে। এর নাম **encapsulation**। অন্য প্রান্তে উল্টোটা হয়, মানে প্রত্যেক স্তর নিজের header খুলে ওপরে পাঠায় (**decapsulation**)।

```
Application:                          [ HTTP ডেটা: GET /index.html ]
Transport:                [TCP হেডার | HTTP ডেটা ]          ← src port 51514, dst port 80
Internet:       [IP হেডার | TCP হেডার | HTTP ডেটা ]          ← src IP, dst IP, TTL
Link:   [Eth হেডার | IP হেডার | TCP হেডার | HTTP ডেটা | FCS] ← src MAC, dst MAC
Physical:  0101100101110100101... (তারে বিদ্যুৎ / বাতাসে রেডিও তরঙ্গ)
```

> 📝 **মনে রাখো:** IP হেডারের ঠিকানা (উৎস ও গন্তব্য IP) পুরো যাত্রায় সাধারণত একই থাকে (NAT বাদে)। কিন্তু Ethernet হেডারের MAC অ্যাড্রেস **প্রতিটা হপে বদলায়**। চিঠির খামে বন্ধুর ঠিকানা একই থাকে, কিন্তু প্রতিটা ডাকঘর চিঠিটা পরের ডাকঘরের হাতে দেয়।

MTU (Maximum Transmission Unit): একটা ফ্রেমে সর্বোচ্চ কত বাইট IP প্যাকেট যেতে পারে। ইথারনেটে সাধারণত **1500 bytes**। এর চেয়ে বড় ডেটা হলে TCP তাকে ছোট ছোট segment-এ ভাগ করে। VPN বা PPPoE-তে MTU কম থাকে (যেমন 1492), তখন "কিছু সাইট খোলে, কিছু খোলে না" ধরনের অদ্ভুত সমস্যা দেখা দেয়।

৩৬.৩ IP অ্যাড্রেস

IP অ্যাড্রেস হলো নেটওয়ার্কে একটা interface-এর লজিক্যাল ঠিকানা। খেয়াল করো, ঠিকানাটা **interface**-এর, পুরো কম্পিউটারের নয়। একটা ল্যাপটপের Wi-Fi-র এক IP, ইথারনেটের আরেক IP, আর `lo`-র `127.0.0.1` থাকতে পারে।

IPv4

- ৩২ বিটের সংখ্যা, ৮ বিট করে ৪ ভাগে লেখা হয়, মাঝে ডট। এর নাম **dotted decimal**।
- প্রতি ভাগ (**octet**) 0 থেকে 255।
- মোট ঠিকানা $2^{32} \approx ৪৩০$ কোটি। পৃথিবীর জন্য এটা কম, তাই IPv4 ঠিকানা ফুরিয়ে গেছে। NAT আর IPv6 এসেছে এই কারণেই।

```
   192    .   168    .    1     .   10
11000000  10101000  00000001  00001010     ← আসলে কম্পিউটার এটাই দেখে (৩২ বিট)
```

বাইনারি দ্রুত ঝালিয়ে নাও

একটা octet-এর ৮টা বিটের মান (বাঁ থেকে ডানে):

```
বিট:   1    1    1    1    1    1    1    1
মান:  128   64   32   16    8    4    2    1      → সব যোগ = 255
```

উদাহরণ: `192 = 128 + 64` → `11000000`। `10 = 8 + 2` → `00001010`।

IP-এর দুই অংশ: network আর host

একটা IP অ্যাড্রেসের বাঁ দিকের কিছু বিট বলে **কোন নেটওয়ার্ক** (এলাকার পোস্ট কোড), আর বাকি ডান দিকের বিট বলে **সেই নেটওয়ার্কে কোন হোস্ট** (বাড়ির নম্বর)। কতগুলো বিট network-এর, সেটা বলে দেয় **subnet mask**।

```
IP:          192.168.1.10      11000000.10101000.00000001.00001010
Mask:        255.255.255.0     11111111.11111111.11111111.00000000
                               └──────── network (24 বিট) ────────┘└host┘
Network:     192.168.1.0       (IP AND Mask)
```

কম্পিউটার IP আর mask-এর **bitwise AND** করে network ঠিকানা বের করে। গন্তব্যের network যদি নিজের network-এর সমান হয়, তাহলে সরাসরি পাঠায়। না হলে gateway-কে দেয়। এটাই রাউটিংয়ের মূল সিদ্ধান্ত।

CIDR নোটেশন

mask লম্বা করে না লিখে শুধু **কতগুলো 1 বিট** সেটা `/` দিয়ে লেখা হয়। এর নাম **CIDR (Classless Inter-Domain Routing)**:

`192.168.1.10/24` মানে mask `255.255.255.0`।

CIDR	Subnet mask	মোট ঠিকানা	ব্যবহারযোগ্য হোস্ট	কোথায় দেখা যায়
/8	255.0.0.0	16,777,216	16,777,214	10.0.0.0/8
/12	255.240.0.0	1,048,576	1,048,574	172.16.0.0/12
/16	255.255.0.0	65,536	65,534	বড় অফিস, cloud VPC
/20	255.255.240.0	4,096	4,094	AWS ডিফল্ট subnet
/23	255.255.254.0	512	510	
/24	255.255.255.0	256	254	বাসার নেটওয়ার্ক
/25	255.255.255.128	128	126	
/26	255.255.255.192	64	62	
/27	255.255.255.224	32	30	
/28	255.255.255.240	16	14	ছোট সার্ভার ব্লক
/29	255.255.255.248	8	6	ISP-র দেওয়া ছোট পাবলিক ব্লক
/30	255.255.255.252	4	2	point-to-point লিংক
/31	255.255.255.254	2	2 (বিশেষ, RFC 3021)	point-to-point
/32	255.255.255.255	1	1 (একটাই হোস্ট)	ফায়ারওয়াল নিয়মে একটা IP

সূত্র: host বিট = `32 − prefix`, মোট ঠিকানা = `2^(host বিট)`, ব্যবহারযোগ্য = `মোট − 2`। প্রথম ঠিকানাটা **network address**, শেষটা **broadcast address**। এই দুটো কোনো হোস্টকে দেওয়া যায় না।

CIDR হিসাব, ধাপে ধাপে (ইন্টারভিউর প্রিয় প্রশ্ন)

প্রশ্ন: `192.168.1.130/26`। network, broadcast, প্রথম ও শেষ হোস্ট, আর হোস্টের সংখ্যা কত?

ধাপ ১: কোন octet-এ ভাগ হচ্ছে? /26 মানে প্রথম ৩ octet (24 বিট) পুরো network, আর চতুর্থ octet-এর প্রথম 2 বিট network-এর। তাই চতুর্থ octet-এর mask `11000000` = 192। পুরো mask `255.255.255.192`।

ধাপ ২: block size। `256 − 192 = 64`। অর্থাৎ চতুর্থ octet-এ subnet শুরু হয় 0, 64, 128, 192 থেকে।

ধাপ ৩: 130 কোন ব্লকে? 128 ≤ 130 < 192, তাই ব্লকটা 128।

```
Network address  : 192.168.1.128
প্রথম হোস্ট       : 192.168.1.129
শেষ হোস্ট         : 192.168.1.190
```

```
Broadcast         : 192.168.1.191      (পরের ব্লক 192 - 1)
হোস্ট সংখ্যা       : 2^6 - 2 = 62
```

আরেকটা: `10.20.37.200/20`। এখানে ভাগ হচ্ছে তৃতীয় octet-এ (16 বিট পুরো + তৃতীয় octet-এ 4 বিট)। তৃতীয় octet-এর mask `11110000` = 240, block size `256 - 240 = 16`। তৃতীয় octet-এ ব্লক শুরু 0, 16, 32, 48 থেকে, আর 37 পড়ছে 32-এর ব্লকে।

```
Network   : 10.20.32.0/20
প্রথম হোস্ট: 10.20.32.1
শেষ হোস্ট  : 10.20.47.254
Broadcast : 10.20.47.255
হোস্ট      : 2^12 - 2 = 4094
```

> 💡 **টিপস:** লিনাক্সে হিসাব মিলিয়ে দেখার জন্য `ipcalc` টুল আছে (Ubuntu/Debian: `sudo apt install ipcalc`, Fedora/RHEL: `sudo dnf install ipcalc`)। খেয়াল রাখো, দুই পরিবারের `ipcalc` দুটো **আলাদা প্রোগ্রাম**, তাই আউটপুটের চেহারাও আলাদা। Debian-এরটা বাইনারি সহ রঙিন টেবিল দেখায়।

```
$ ipcalc 192.168.1.130/26
Address:   192.168.1.130        11000000.10101000.00000001.10 000010
Netmask:   255.255.255.192 = 26 11111111.11111111.11111111.11 000000
Wildcard:  0.0.0.63             00000000.00000000.00000000.00 111111
=>
Network:   192.168.1.128/26     11000000.10101000.00000001.10 000000
HostMin:   192.168.1.129        11000000.10101000.00000001.10 000001
HostMax:   192.168.1.190        11000000.10101000.00000001.10 111110
Broadcast: 192.168.1.191        11000000.10101000.00000001.10 111111
Hosts/Net: 62                    Class C, Private Internet
```

পুরনো Class পদ্ধতি (শুধু ইতিহাস হিসেবে জানো)

১৯৯৩-এর আগে mask ঠিক হতো প্রথম octet দেখে: Class A (1–126, /8), Class B (128–191, /16), Class C (192–223, /24), Class D (224–239, multicast), Class E (240–255, পরীক্ষামূলক)। এখন সব জায়গায় CIDR চলে। তবু পুরনো বই আর ইন্টারভিউতে "Class C নেটওয়ার্ক" কথাটা শুনবে, মানে /24।

বিশেষ IPv4 রেঞ্জ: private, loopback ও অন্যান্য

রেঞ্জ	কী	ব্যাখ্যা
`10.0.0.0/8`	Private	বড় প্রতিষ্ঠান, cloud VPC
`172.16.0.0/12`	Private	172.16.0.0 থেকে 172.31.255.255। Docker ডিফল্টে `172.17.0.0/16` নেয়
`192.168.0.0/16`	Private	বাসার রাউটার (192.168.0.x / 192.168.1.x)

রেঞ্জ	কী	ব্যাখ্যা
`127.0.0.0/8`	Loopback	নিজেই নিজের সাথে কথা বলা। `127.0.0.1` = localhost
`169.254.0.0/16`	Link-local (APIPA)	DHCP না পেলে নিজে থেকে নেওয়া ঠিকানা। এটা দেখলে বুঝবে DHCP কাজ করছে না
`100.64.0.0/10`	CGNAT (Shared)	ISP-র নিজস্ব NAT। Tailscale-ও এই রেঞ্জ ব্যবহার করে
`0.0.0.0`	"সব interface" / "অজানা"	সার্ভার `0.0.0.0:80`-এ listen করা মানে সব interface-এ শোনা
`255.255.255.255`	Limited broadcast	লোকাল নেটওয়ার্কের সবাইকে (DHCP এটা ব্যবহার করে)
`224.0.0.0/4`	Multicast	একসাথে একদল হোস্টকে

Private IP (RFC 1918) ইন্টারনেটে রাউট হয় না। পৃথিবীর লাখ লাখ বাসায় একই `192.168.1.10` থাকতে পারে, কোনো সংঘর্ষ হয় না, কারণ ইন্টারনেটে যাওয়ার সময় রাউটার NAT করে নিজের **public IP** বসিয়ে দেয় (নিচে ৩৬.১১ অংশে NAT দেখো)।

> ⚠️ **সতর্কতা:** সার্ভারে কোনো সার্ভিস (যেমন MySQL) শুধু নিজের মেশিন থেকে ব্যবহার করবে, তাহলে তাকে `127.0.0.1`-এ bind করাও, `0.0.0.0`-এ নয়। `0.0.0.0` মানে পাবলিক interface-ও শুনবে, অর্থাৎ পুরো ইন্টারনেট তোমার ডাটাবেসের দরজায় টোকা দিতে পারবে।

IPv6

- ১২৮ বিট, তাই $2^{128} \approx 3.4 \times 10^{38}$টা ঠিকানা। পৃথিবীর প্রতিটা বালুকণাকে দিলেও শেষ হবে না।
- hexadecimal-এ লেখা হয়, ১৬ বিট করে ৮টা ভাগ, মাঝে `:` (colon)।

```
পূর্ণ রূপ:   2001:0db8:0000:0000:0000:ff00:0042:8329
সংক্ষেপ ১:   2001:db8:0:0:0:ff00:42:8329        ← প্রতি ভাগের শুরুর শূন্য বাদ
সংক্ষেপ ২:   2001:db8::ff00:42:8329             ← টানা শূন্যের ভাগগুলো "::" দিয়ে (পুরো ঠিকানায় একবারই)
```

> ⚠️ **সতর্কতা:** `::` পুরো ঠিকানায় **একবারই** ব্যবহার করা যায়। দুইবার দিলে বোঝা যায় না কোন ফাঁকে কতগুলো শূন্য ছিল।

IPv6-এ সাধারণত **/64** prefix ব্যবহার হয়: প্রথম ৬৪ বিট network, শেষ ৬৪ বিট interface ID। IPv6-এ **broadcast নেই**, তার জায়গায় multicast। আর **ARP-ও নেই**, তার কাজ করে **NDP (Neighbor Discovery Protocol)**, যেটা ICMPv6-এর ওপর চলে।

IPv6 রেঞ্জ	কী	IPv4-এ সমতুল্য
`::1/128`	Loopback	`127.0.0.1`

IPv6 রেঞ্জ	কী	IPv4-এ সমতুল্য
`::/128`	Unspecified	`0.0.0.0`
`fe80::/10`	Link-local। প্রত্যেক IPv6 interface-এ স্বয়ংক্রিয়ভাবে থাকে	`169.254.0.0/16`
`fc00::/7` (বাস্তবে `fd00::/8`)	Unique Local Address (ULA), প্রাইভেট	RFC 1918
`2000::/3`	Global Unicast, পাবলিক ইন্টারনেট	পাবলিক IP
`ff00::/8`	Multicast	`224.0.0.0/4`
`2001:db8::/32`	ডকুমেন্টেশন/উদাহরণের জন্য সংরক্ষিত	`192.0.2.0/24`

IPv6-এ ঠিকানা পাওয়ার দুইটা উপায়: **SLAAC** (রাউটারের Router Advertisement দেখে নিজেই ঠিকানা বানিয়ে নেওয়া) আর **DHCPv6**। লিনাক্সে সাধারণত privacy extension চালু থাকে, যেটা সময়ে সময়ে র‍্যান্ডম "temporary" ঠিকানা বানায় যাতে তোমাকে ট্র্যাক করা কঠিন হয়। `ip -6 addr` চালালে `temporary` লেখা ঠিকানা দেখবে।

URL-এ IPv6 ঠিকানা লিখতে হলে `[ ]` দিতে হয়: `http://[2001:db8::1]:8080/`।

নিজের IP দেখো

বিস্তারিত `ip` কমান্ড অধ্যায় ৩৭-এ। আপাতত এটুকু:

```
$ ip -brief address
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.1.10/24 fe80::a00:27ff:fe4e:66a1/64
wlp2s0           DOWN
```

```
$ ip address show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:4e:66:a1 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.10/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
       valid_lft 85934sec preferred_lft 85934sec
    inet6 fe80::a00:27ff:fe4e:66a1/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

লাইনগুলো পড়তে শেখো:

অংশ	মানে
`UP,LOWER_UP`	interface চালু, আর তার/সিগন্যাল সংযুক্ত (LOWER_UP = physical link আছে)
`mtu 1500`	সর্বোচ্চ প্যাকেট সাইজ
`link/ether 08:00:27:4e:66:a1`	MAC অ্যাড্রেস
`inet 192.168.1.10/24`	IPv4 ঠিকানা ও prefix
`brd 192.168.1.255`	broadcast ঠিকানা
`dynamic`	DHCP থেকে পাওয়া (static হলে এটা থাকে না)

অংশ	মানে
`valid_lft 85934sec`	DHCP lease আর কতক্ষণ বৈধ
`scope link`	শুধু এই লোকাল লিংকে বৈধ (link-local)

৩৬.৪ Gateway ও Routing

তোমার কম্পিউটার যখন কোনো প্যাকেট পাঠায়, kernel প্রথমে একটা প্রশ্ন করে: "গন্তব্য কি আমার নিজের নেটওয়ার্কে?"

- **হ্যাঁ** হলে সরাসরি পাঠায় (ARP দিয়ে গন্তব্যের MAC খুঁজে)।
- **না** হলে **default gateway**-কে দেয়। gateway মানে সাধারণত তোমার রাউটার (যেমন `192.168.1.1`), যে জানে বাইরের দুনিয়ায় কীভাবে যেতে হয়।

এই সিদ্ধান্তের জন্য kernel একটা **routing table** দেখে:

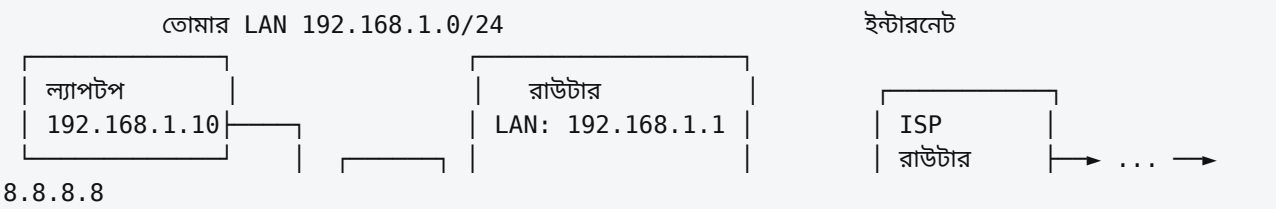
```
$ ip route
default via 192.168.1.1 dev enp0s3 proto dhcp src 192.168.1.10 metric 100
192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.10 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
```

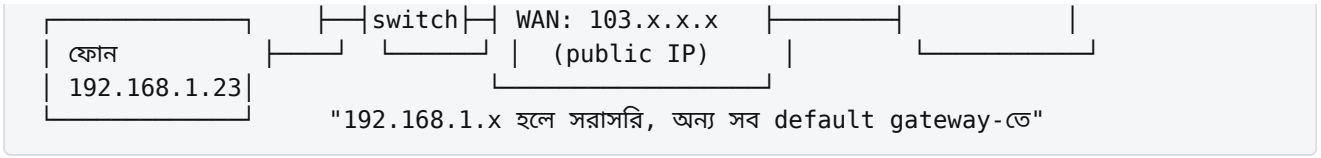
লাইন	মানে
`default via 192.168.1.1`	অন্য কোনো লাইনে না মিললে 192.168.1.1-কে দাও (default route = `0.0.0.0/0`)
`192.168.1.0/24 dev enp0s3 ... scope link`	এই নেটওয়ার্ক সরাসরি enp0s3-এ যুক্ত, gateway লাগবে না
`metric 100`	একই গন্তব্যে একাধিক রুট থাকলে যার metric কম, সে আগে বিবেচিত হয়
`proto dhcp` / `proto kernel`	রুটটা কে বসিয়েছে

Longest prefix match: একটা গন্তব্যের সাথে একাধিক লাইন মিললে **সবচেয়ে নির্দিষ্ট** (বড় prefix) লাইনটা জেতে। `/32` জেতে `/24`-এর ওপর, `/24` জেতে `/0` (default)-এর ওপর।

kernel কোন রুট বেছে নেবে, সেটা সরাসরি জিজ্ঞেস করা যায়:

```
$ ip route get 8.8.8.8
8.8.8.8 via 192.168.1.1 dev enp0s3 src 192.168.1.10 uid 1000
    cache
```





> 📝 **মনে রাখো:** gateway অবশ্যই **তোমার নিজের subnet-এর ভেতরে** হতে হবে। `192.168.1.10/24`-এর gateway `10.0.0.1` হতে পারে না, কারণ সেখানে পৌঁছাতেই তো gateway লাগবে।

লিনাক্স নিজেও রাউটার হতে পারে। তার জন্য **IP forwarding** চালু করতে হয় (কন্টেইনার, VPN সার্ভার আর NAT gateway-তে এটা লাগে):

```
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
$ sudo sysctl -w net.ipv4.ip_forward=1      # সাময়িক, রিবুটে হারাবে
```

স্থায়ী করতে `/etc/sysctl.d/99-forward.conf` ফাইলে `net.ipv4.ip_forward = 1` লেখো (বিস্তারিত অধ্যায় ৪৭)।

৩৬.৫ MAC অ্যাড্রেস ও ARP

MAC অ্যাড্রেস

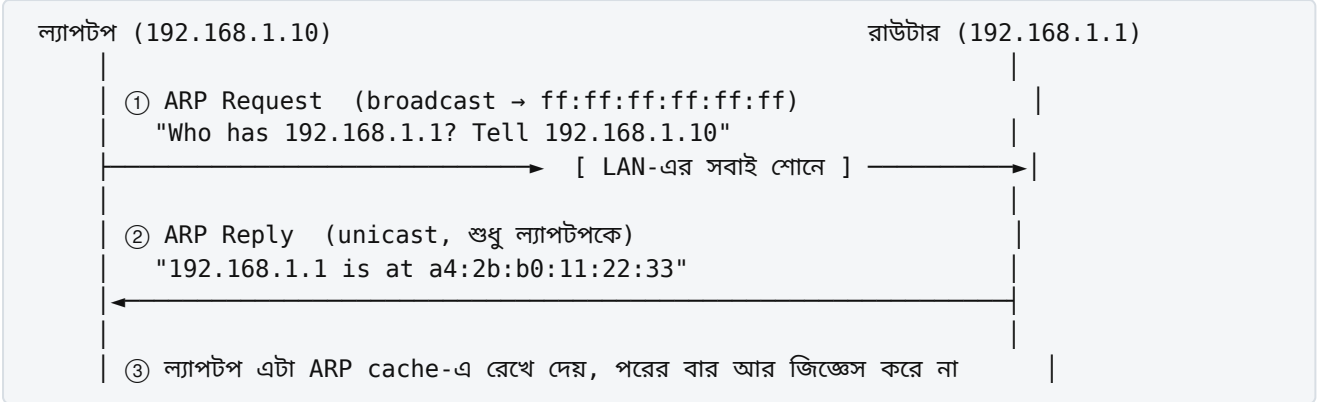
MAC (Media Access Control) অ্যাড্রেস হলো NIC-এর **হার্ডওয়্যার ঠিকানা**। এটা ৪৮ বিটের, ৬ জোড়া hex অঙ্কে লেখা হয়: `08:00:27:4e:66:a1`।

- প্রথম ৩ বাইট **OUI**, মানে প্রস্তুতকারকের পরিচয় (যেমন `08:00:27` = VirtualBox)।
- কারখানা থেকে NIC-এ বসানো থাকে, তবে সফটওয়্যার দিয়ে বদলানো যায়। NetworkManager Wi-Fi স্ক্যান করার সময় ডিফল্টে র‍্যান্ডম MAC ব্যবহার করে, প্রাইভেসির জন্য।
- MAC শুধু **লোকাল নেটওয়ার্কের ভেতরে** কাজে লাগে। রাউটার পার হলে প্যাকেটের MAC বদলে যায়।
- `ff:ff:ff:ff:ff:ff` মানে broadcast MAC, লোকাল নেটওয়ার্কের সবাই।

	IP অ্যাড্রেস	MAC অ্যাড্রেস
স্তর	Layer 3	Layer 2
দৈর্ঘ্য	32 বিট (v4) / 128 বিট (v6)	48 বিট
প্রকৃতি	লজিক্যাল, নেটওয়ার্ক বদলালে বদলায়	হার্ডওয়্যারের (সাধারণত স্থির)
পরিসর	পুরো ইন্টারনেটে রাউটযোগ্য	শুধু লোকাল segment-এ
উপমা	বাড়ির ঠিকানা	মানুষটার মুখ (পিয়ন চেনে)

ARP: "192.168.1.1 কার? MAC বলো!"

তোমার ল্যাপটপ জানে gateway-এর IP `192.168.1.1`। কিন্তু ইথারনেট ফ্রেম পাঠাতে লাগে **MAC**। IP থেকে MAC বের করার এই প্রোটোকলের নাম **ARP (Address Resolution Protocol)**।



ARP cache (লিনাক্সে একে **neighbor table** বলে) দেখো:

```
$ ip neigh
192.168.1.1 dev enp0s3 lladdr a4:2b:b0:11:22:33 REACHABLE
192.168.1.23 dev enp0s3 lladdr 3c:22:fb:aa:bb:cc STALE
fe80::a62b:b0ff:fe11:2233 dev enp0s3 lladdr a4:2b:b0:11:22:33 router REACHABLE
```

অবস্থা	মানে
`REACHABLE`	সম্প্রতি নিশ্চিত হওয়া গেছে, বৈধ
`STALE`	অনেকক্ষণ যাচাই হয়নি; পরের বার ব্যবহারের সময় যাচাই হবে
`FAILED`	উত্তর মেলেনি, হোস্টটা নেই বা বন্ধ
`PERMANENT`	হাতে বসানো স্ট্যাটিক এন্ট্রি

পুরনো কমান্ড `arp -n` (`net-tools` প্যাকেজের) এখনো অনেকে চালায়, কিন্তু আধুনিক distro-তে এটা ডিফল্টে ইনস্টল থাকে না।

> ⚠️ **সতর্কতা:** ARP-তে কোনো যাচাই নেই। যে কেউ মিথ্যা উত্তর দিয়ে বলতে পারে "gateway আমি"। এর নাম **ARP spoofing**, আর এটা দিয়ে man-in-the-middle আক্রমণ করা হয়। পাবলিক Wi-Fi-তে তাই HTTPS আর VPN জরুরি।

৩৬.৬ DNS: ইন্টারনেটের ফোনবুক

মানুষ `google.com` মনে রাখতে পারে, `142.250.193.46` নয়। **DNS (Domain Name System)** নাম থেকে IP (আর উল্টোটাও) বের করে দেয়। এটা একটা বিশাল, বিকেন্দ্রীভূত, স্তরভিত্তিক ডাটাবেস।

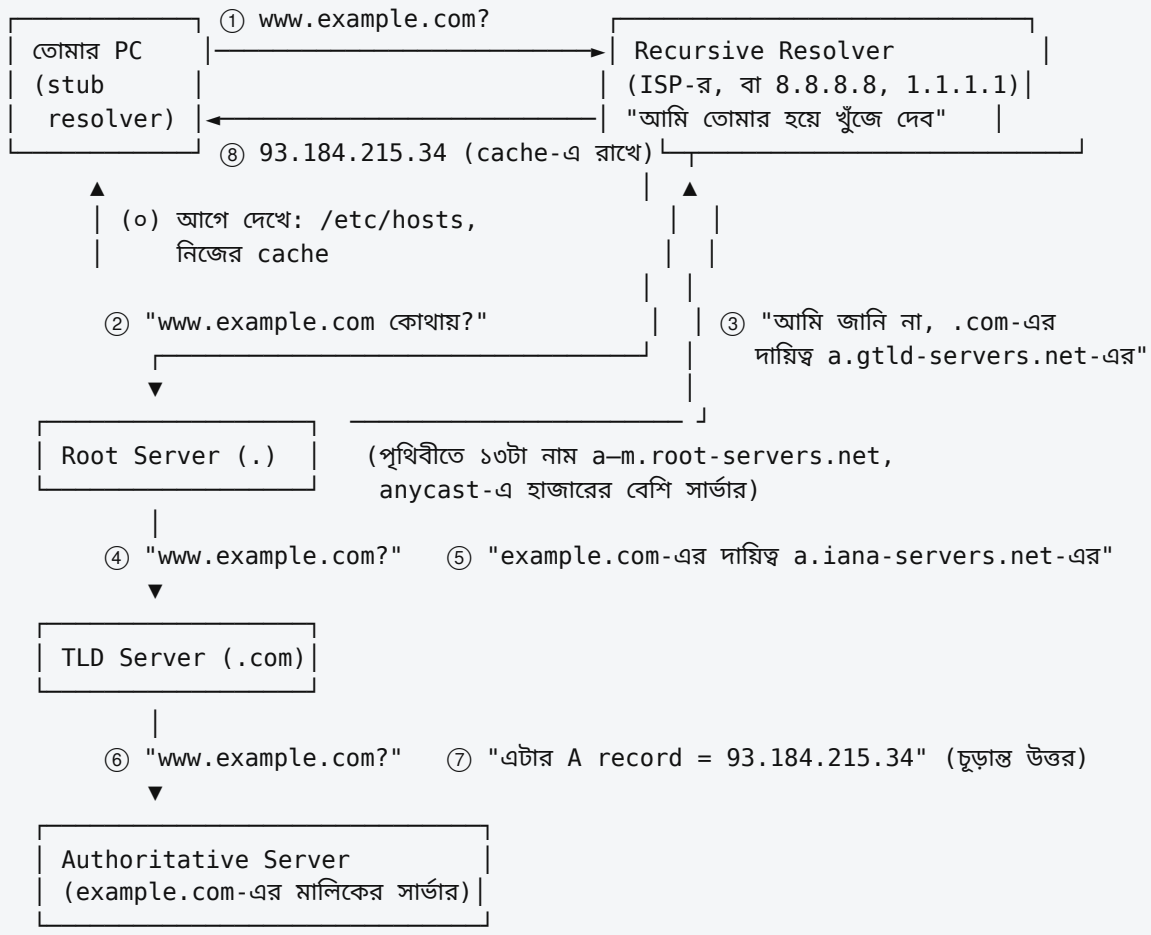
ডোমেইন নামের গঠন

নাম পড়তে হয় **ডান থেকে বাঁয়ে**:

```
            www . example . com .
             |       |       |  |
             |       |       |  └── root (".")  — শেষে একটা অদৃশ্য ডট থাকে
             |       |       └───── TLD (Top-Level Domain): com, org, bd, net
             |       └─────────────── second-level domain: example (কেউ কিনেছে)
             └─────────────────────── subdomain / hostname: www
```

DNS resolution: পর্দার পেছনে কী হয়

তুমি ব্রাউজারে `www.example.com` লিখলে:



> 📝 **মনে রাখো:** সংখ্যাগুলো ধাপের ক্রম দেখাচ্ছে, তবে ডায়াগ্রামে ② থেকে ⑦ সব প্রশ্ন resolver নিজে করে। তোমার PC শুধু ① প্রশ্ন করে আর ⑧ উত্তর পায়।

দুই ধরনের জিজ্ঞাসা আছে:

- **Recursive query**: তোমার PC resolver-কে বলে, "চূড়ান্ত উত্তর এনে দাও।" ①

- **Iterative query**: resolver root, TLD আর authoritative সার্ভারকে একে একে জিজ্ঞেস করে, আর প্রত্যেকে "পরের জনের কাছে যাও" বলে রেফার করে। ②–⑦

প্রতিটা উত্তরের সাথে একটা **TTL (Time To Live)** থাকে, মানে কত সেকেন্ড উত্তরটা cache-এ রাখা যাবে। এই কারণেই DNS বদলালে সবার কাছে নতুন মান পৌঁছাতে সময় লাগে। লোকে একে "propagation" বলে, আসলে পুরনো cache-এর মেয়াদ ফুরানোর অপেক্ষা।

DNS রেকর্ডের ধরন

রেকর্ড	কাজ	উদাহরণ
`A`	নাম থেকে IPv4	`example.com → 93.184.215.34`
`AAAA`	নাম থেকে IPv6	`example.com → 2606:2800:21f:cb07:6820:80da:af6b:8b2c`
`CNAME`	এক নামকে আরেক নামের "ডাকনাম" বানায়	`www.example.com → example.com`
`MX`	এই ডোমেইনের ইমেইল কোন সার্ভারে যাবে (priority সহ)	`10 mail.example.com`
`NS`	এই ডোমেইনের authoritative name server কারা	`ns1.example.com`
`TXT`	যেকোনো লেখা; SPF, DKIM, ডোমেইন যাচাইয়ে লাগে	`"v=spf1 include:_spf.google.com ~all"`
`PTR`	উল্টো দিকে, IP থেকে নাম (reverse DNS)	`34.215.184.93.in-addr.arpa → example.com`
`SOA`	zone-এর মূল তথ্য (serial, refresh ইত্যাদি)	
`SRV`	কোনো সার্ভিস কোন host:port-এ চলে	`_sip._tcp.example.com`
`CAA`	কোন CA এই ডোমেইনের SSL সার্টিফিকেট দিতে পারবে	`0 issue "letsencrypt.org"`

DNS সাধারণত **UDP port 53** ব্যবহার করে। উত্তর বড় হলে বা zone transfer হলে **TCP 53**। আধুনিক এনক্রিপ্টেড DNS-ও আছে: **DoT** (DNS over TLS, TCP 853) আর **DoH** (DNS over HTTPS, 443)।

`dig`, `nslookup`, `host` দিয়ে DNS জিজ্ঞাসা করা অধ্যায় ৩৭-এ বিস্তারিত আছে। একটা ঝলক:

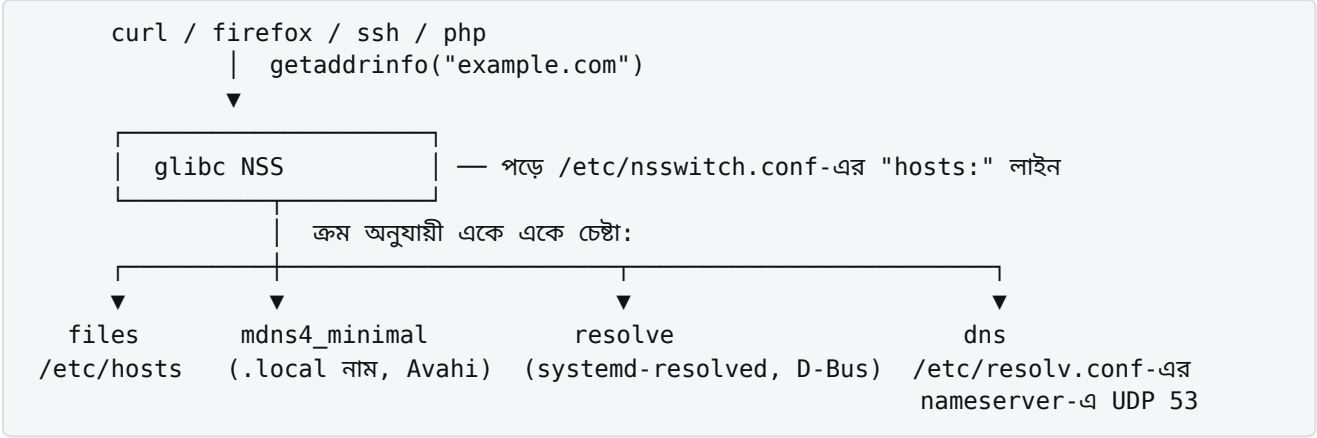
```
$ dig +short example.com A
93.184.215.34
$ getent hosts example.com
93.184.215.34   example.com
```

> 💡 **টিপস:** `dig` সরাসরি DNS সার্ভারকে জিজ্ঞেস করে, `/etc/hosts` দেখে না। অন্যদিকে `getent hosts` ঠিক সেই পথে যায় যে পথে তোমার প্রোগ্রামগুলো যায় (`/etc/nsswitch.conf` মেনে)। তাই

> "আমার অ্যাপ কোন IP পাচ্ছে?" জানতে `getent` বেশি সৎ উত্তর দেয়।

৩৬.৭ লিনাক্সে নাম কীভাবে resolve হয়: under the hood

একটা প্রোগ্রাম (যেমন `curl`) যখন `example.com`-এর IP চায়, সে নিজে DNS প্যাকেট বানায় না। সে glibc-এর ফাংশন `getaddrinfo()` ডাকে। তারপর:



`/etc/nsswitch.conf`

এই ফাইল ঠিক করে নাম খোঁজার **ক্রম**:

```
$ grep hosts /etc/nsswitch.conf
hosts:          files mdns4_minimal [NOTFOUND=return] dns
```

এটা Ubuntu ডেস্কটপের সাধারণ চেহারা। Fedora-তে সাধারণত `hosts: files myhostname resolve [!UNAVAIL=return] dns` ধরনের লাইন দেখবে (এখানে `resolve` মানে সরাসরি systemd-resolved)। `files` প্রথমে থাকে, তাই **`/etc/hosts` সবার আগে দেখা হয়**।

`/etc/hosts`

DNS আবিষ্কারের আগের যুগের হাতে লেখা ফোনবুক, কিন্তু এখনো খুব কাজের:

```
$ cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       mylaptop
192.168.1.50    devserver devserver.local
::1             localhost ip6-localhost ip6-loopback
```

ফরম্যাট: `IP প্রধান-নাম [ডাকনাম...]`। `#` দিয়ে শুরু লাইন মন্তব্য।

কাজে লাগে যখন:

- লোকাল ডেভেলপমেন্টে `myapp.test`-কে `127.0.0.1`-এ পাঠাতে চাও
- DNS ঠিক হওয়ার আগেই নতুন সার্ভার পরীক্ষা করতে চাও
- কোনো ডোমেইন ব্লক করতে চাও (`0.0.0.0 ads.example.com`)

```
$ echo "127.0.0.1  myapp.test" | sudo tee -a /etc/hosts
127.0.0.1  myapp.test
$ getent hosts myapp.test
127.0.0.1       myapp.test
```

> 📝 **মনে রাখো:** Debian/Ubuntu-তে নিজের hostname-এর জন্য `127.0.1.1` লাইনটা থাকে, যাতে নেটওয়ার্ক ছাড়াও hostname resolve হয়। এটা স্বাভাবিক, ভুল না।

`/etc/resolv.conf`

এখানে লেখা থাকে কোন DNS সার্ভারকে জিজ্ঞেস করা হবে:

```
nameserver 8.8.8.8
nameserver 1.1.1.1
search example.local
options timeout:2 attempts:2
```

নির্দেশ	মানে
`nameserver`	DNS সার্ভারের IP। glibc সর্বোচ্চ ৩টা ব্যবহার করে, ক্রমানুসারে
`search`	ছোট নামের পেছনে জুড়ে দেওয়ার ডোমেইন। `ping db` দিলে চেষ্টা করবে `db.example.local`
`options`	timeout, attempts, `ndots` ইত্যাদি

> ⚠️ **সতর্কতা:** আধুনিক সিস্টেমে **`/etc/resolv.conf` হাতে এডিট করবে না।** ফাইলটা NetworkManager বা systemd-resolved স্বয়ংক্রিয়ভাবে লেখে, তাই রিবুট বা নেটওয়ার্ক রিকানেক্টের পর তোমার পরিবর্তন মুছে যাবে। DNS বদলাতে হবে NetworkManager/Netplan দিয়ে (নিচে দেখানো আছে)। ফাইলটা কে নিয়ন্ত্রণ করছে, প্রথমে সেটা দেখো:

```
$ ls -l /etc/resolv.conf
lrwxrwxrwx 1 root root 39 Apr 23 10:12 /etc/resolv.conf -> ../run/systemd/resolve/stub-resolv.conf
```

এই symlink মানে ফাইলটা systemd-resolved-এর হাতে (Ubuntu এবং Fedora 33+-এর ডিফল্ট)। আর যদি সাধারণ ফাইল হয়, ওপরে `# Generated by NetworkManager` লেখা থাকে, তাহলে NetworkManager লিখছে (RHEL 9, Debian 12 ডেস্কটপের ডিফল্ট)।

systemd-resolved

systemd-resolved একটা লোকাল DNS সার্ভিস, যে:

- `127.0.0.53`-এ একটা **stub resolver** চালায়। প্রোগ্রামরা একে প্রশ্ন করে, সে আসল DNS সার্ভারে পাঠায়।
- উত্তর **cache** করে, ফলে দ্বিতীয়বার জিজ্ঞাসা দ্রুত হয়।
- **per-interface DNS** সামলায়। যেমন VPN চালু থাকলে `*.company.internal` যাবে VPN-এর DNS-এ, বাকি সব সাধারণ DNS-এ।
- DNSSEC আর DNS-over-TLS সাপোর্ট করে (ডিফল্টে সাধারণত বন্ধ থাকে)।

```
তোমার প্রোগ্রাম ──► 127.0.0.53:53 (systemd-resolved stub) ──► cache-এ আছে? ──হ্যাঁ──► উত্তর
                                                                    │
                                                                   না
                                                                    ▼
                                                  interface অনুযায়ী upstream DNS (যেমন 192.168.1.1)
```

তাই systemd-resolved চালু থাকলে `/etc/resolv.conf`-এ শুধু দেখবে:

```
nameserver 127.0.0.53
options edns0 trust-ad
search .
```

আসল upstream DNS সার্ভার দেখতে হলে `resolvectl` চালাও:

```
$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 2 (enp0s3)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.1.1
       DNS Servers: 192.168.1.1
```

`resolvectl` কমান্ড	কাজ
`resolvectl status`	প্রতিটা interface-এর DNS সার্ভার আর সেটিং
`resolvectl query example.com`	systemd-resolved-এর মাধ্যমে নাম খোঁজা
`resolvectl flush-caches`	DNS cache মুছে ফেলা
`resolvectl statistics`	cache hit/miss-এর পরিসংখ্যান
`resolvectl dns enp0s3 1.1.1.1`	সাময়িকভাবে একটা interface-এর DNS বদলানো (রিবুটে হারায়)

```
$ resolvectl query example.com
example.com: 93.184.215.34                     -- link: enp0s3
             2606:2800:21f:cb07:6820:80da:af6b:8b2c -- link: enp0s3
```

```
-- Information acquired via protocol DNS in 41.2ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: no
-- Data from: network
```

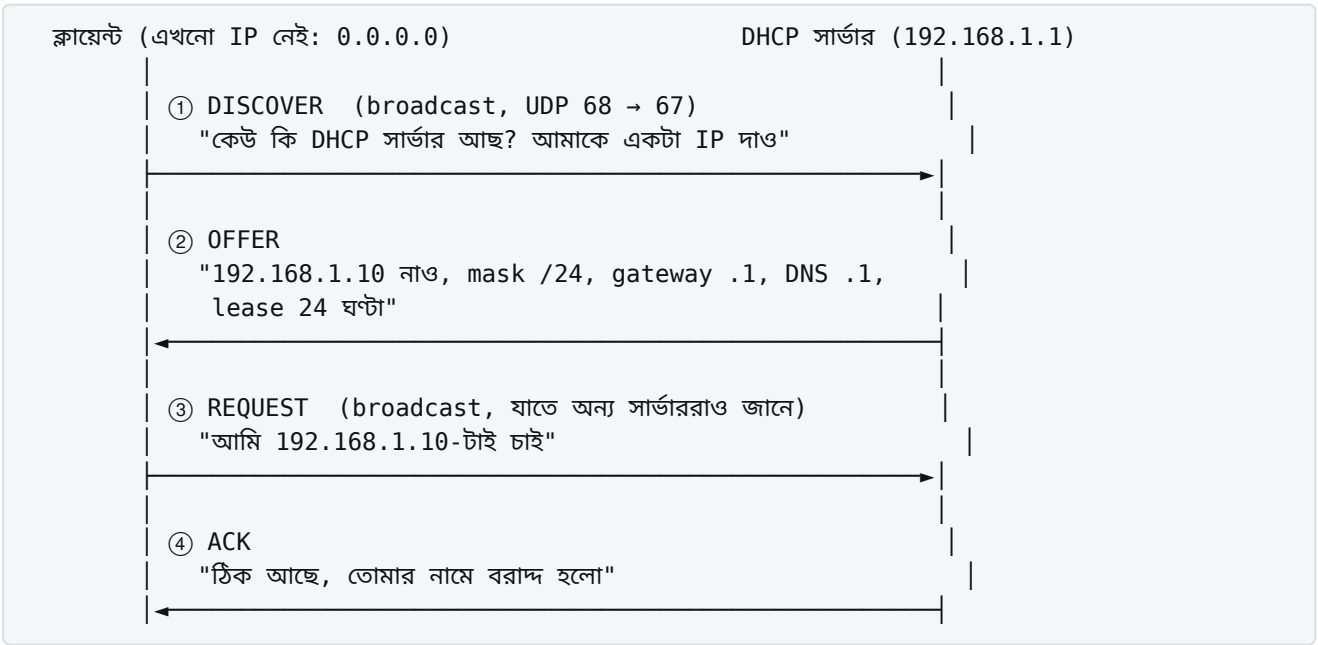
এর কনফিগ ফাইল `/etc/systemd/resolved.conf` (বা `/etc/systemd/resolved.conf.d/*.conf`), যেখানে `DNS=`, `FallbackDNS=`, `DNSSEC=`, `DNSOverTLS=` সেট করা যায়। পরিবর্তনের পর `sudo systemctl restart systemd-resolved` চালাও (অধ্যায় ২৬)।

Distro	ডিফল্ট DNS ব্যবস্থাপনা
Ubuntu 24.04 (ডেস্কটপ ও সার্ভার)	systemd-resolved (stub 127.0.0.53)
Fedora (33+)	systemd-resolved
RHEL 9 / Rocky / Alma	NetworkManager সরাসরি `/etc/resolv.conf` লেখে
Debian 12	ডেস্কটপে NetworkManager; সার্ভারে ifupdown/DHCP ক্লায়েন্ট সরাসরি লেখে; systemd-resolved আলাদা প্যাকেজ

৩৬.৮ DHCP: স্বয়ংক্রিয় ঠিকানা

প্রতিটা ডিভাইসে হাতে IP, mask, gateway আর DNS বসানো ঝামেলার কাজ। **DHCP (Dynamic Host Configuration Protocol)** এগুলো স্বয়ংক্রিয়ভাবে দিয়ে দেয়। বাসায় DHCP সার্ভার হলো তোমার রাউটার।

DORA প্রসেস



মনে রাখার উপায়: **D-O-R-A** = Discover, Offer, Request, Acknowledge।

- DHCP ব্যবহার করে **UDP**: সার্ভার port **67**, ক্লায়েন্ট port **68**।
- ঠিকানাটা চিরকালের নয়, **lease** (ইজারা) হিসেবে দেওয়া হয়। মেয়াদের অর্ধেক পার হলে (T1) ক্লায়েন্ট নবায়ন (renew) চায়।
- DHCP সার্ভার রাউটার পার হতে পারে না (broadcast রাউটার পার হয় না)। বড় নেটওয়ার্কে তাই **DHCP relay** লাগে।
- রাউটারে একটা নির্দিষ্ট MAC-কে সবসময় একই IP দেওয়া যায়, একে **DHCP reservation** বলে। প্রিন্টার বা হোম সার্ভারের জন্য এটা ভালো।

> 💡 **টিপস:** কোনো interface-এ `169.254.x.x` ঠিকানা দেখলে বুঝবে DHCP সার্ভার উত্তর দেয়নি। তখন তার, রাউটার বা Wi-Fi পাসওয়ার্ড পরীক্ষা করো।

লিনাক্সে DHCP ক্লায়েন্ট কে চালায়, সেটা distro-ভেদে আলাদা:

ব্যবস্থা	DHCP ক্লায়েন্ট
NetworkManager (Fedora, RHEL 9, Ubuntu ডেস্কটপ)	নিজস্ব internal ক্লায়েন্ট (ডিফল্ট)
systemd-networkd (Ubuntu সার্ভার, Netplan-এর মাধ্যমে)	নিজস্ব ক্লায়েন্ট
Debian 12 ifupdown	`dhclient` (isc-dhcp-client)

NetworkManager-এ lease নবায়ন করার সবচেয়ে সহজ উপায় হলো connection-টা আবার চালু করা:

```
$ nmcli connection up "Wired connection 1"
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/5)
```

DHCP থেকে কী কী পাওয়া গেছে দেখতে: `nmcli -f DHCP4 device show enp0s3`।

৩৬.৯ Port ও Protocol: TCP বনাম UDP

IP অ্যাড্রেস প্যাকেটকে সঠিক **কম্পিউটারে** পৌঁছে দেয়। কিন্তু একটা কম্পিউটারে একসাথে ওয়েব সার্ভার, SSH আর ডাটাবেস চলছে। প্যাকেটটা কার জন্য? সেটা বলে **port নম্বর** (০ থেকে ৬৫৫৩৫)। IP যদি অ্যাপার্টমেন্ট বিল্ডিংয়ের ঠিকানা হয়, port তাহলে ফ্ল্যাট নম্বর।

একটা নেটওয়ার্ক সংযোগকে চেনা যায় পাঁচটা জিনিস দিয়ে, একে **5-tuple** বলে:

```
(protocol, source IP, source port, destination IP, destination port)
   TCP     192.168.1.10   51514      93.184.215.34       443
```

ক্লায়েন্টের port সাধারণত kernel র‍্যান্ডমভাবে বেছে দেয়, একে **ephemeral port** বলে। লিনাক্সে এর ডিফল্ট রেঞ্জ:

```
$ cat /proc/sys/net/ipv4/ip_local_port_range
32768   60999
```

port রেঞ্জ	নাম	মানে
0–1023	Well-known / system ports	প্রচলিত সার্ভিস। লিনাক্সে এগুলোতে bind করতে root (বা `CAP_NET_BIND_SERVICE`) লাগে
1024–49151	Registered ports	IANA-তে নিবন্ধিত অ্যাপ (MySQL 3306 ইত্যাদি)
49152–65535	Dynamic / private	ephemeral (IANA-র হিসাবে; লিনাক্স নিজের রেঞ্জ ব্যবহার করে)

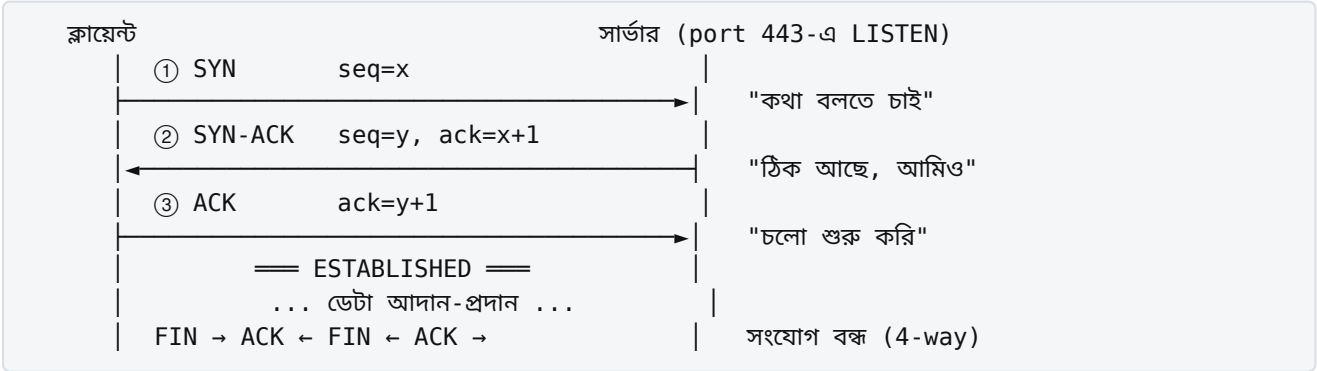
> 📝 **মনে রাখো:** কোন port কোন সার্ভিসের, তার তালিকা লিনাক্সে আছে `/etc/services` ফাইলে: `grep -w 22 /etc/services`। এই কারণেই কিছু টুল সংখ্যার বদলে `ssh` বা `https` নাম দেখায়।

TCP (Transmission Control Protocol)

TCP হলো রেজিস্টার্ড চিঠি বা ফোনকল। আগে সংযোগ স্থাপন, তারপর নিশ্চিত ডেলিভারি।

- **connection-oriented**: কথা শুরুর আগে "হ্যান্ডশেক"
- **reliable**: প্রতিটা segment-এর ACK (প্রাপ্তিস্বীকার) আসে; হারালে আবার পাঠায়
- **ordered**: sequence number দিয়ে ক্রম ঠিক রাখে
- **flow control ও congestion control**: নেটওয়ার্ক জ্যাম হলে গতি কমায়

3-way handshake (সংযোগ শুরু):



ট্রাবলশুটিংয়ে কাজে লাগে এমন TCP অবস্থা (`ss -tan` দিয়ে দেখা যায়, অধ্যায় ৩৭): `LISTEN` (সার্ভার অপেক্ষা করছে), `SYN-SENT`, `ESTABLISHED`, `TIME-WAIT` (বন্ধের পর কিছুক্ষণ অপেক্ষা), `CLOSE-WAIT` (অন্য পক্ষ বন্ধ করেছে কিন্তু তোমার অ্যাপ এখনো করেনি; অনেকগুলো জমে থাকলে বুঝবে অ্যাপে bug আছে)।

পোর্টে কেউ না শুনলে সার্ভার **RST** পাঠায়, তখন ক্লায়েন্টে দেখায় "Connection refused"। আর ফায়ারওয়াল যদি প্যাকেট চুপচাপ ফেলে দেয় (DROP), তখন হয় "Connection timed out"। এই দুইয়ের পার্থক্য বুঝলে অর্ধেক সমস্যা সেখানেই ধরা পড়ে।

UDP (User Datagram Protocol)

UDP হলো পোস্টকার্ড। ছুড়ে দাও, পৌঁছাল কি না কেউ জানাবে না।

- **connectionless**: হ্যান্ডশেক নেই
- **unreliable**: ACK নেই, পুনঃপ্রেরণ নেই, ক্রমের নিশ্চয়তা নেই
- **দ্রুত ও হালকা**: হেডার মাত্র 8 বাইট (TCP-র ন্যূনতম 20)

যেখানে গতি নির্ভরতার চেয়ে জরুরি, সেখানে UDP: DNS প্রশ্ন, ভিডিও কল, অনলাইন গেম, লাইভ স্ট্রিম, DHCP, NTP। একটা ভিডিও ফ্রেম হারালে আবার পাঠিয়ে লাভ নেই, ততক্ষণে পরের ফ্রেম চলে এসেছে। আধুনিক **HTTP/3 (QUIC)** UDP-র ওপর চলে, আর নির্ভরতার কাজটা নিজেই করে।

বৈশিষ্ট্য	TCP	UDP
সংযোগ	আছে (handshake)	নেই
নির্ভরযোগ্যতা	নিশ্চিত, ACK + retransmit	নেই
ক্রম	ঠিক থাকে	নিশ্চয়তা নেই
গতি/ওভারহেড	তুলনামূলক ধীর, ভারী	দ্রুত, হালকা
হেডার	ন্যূনতম 20 বাইট	8 বাইট
ব্যবহার	ওয়েব, SSH, ইমেইল, ডাটাবেস, ফাইল ট্রান্সফার	DNS, DHCP, VoIP, গেম, স্ট্রিমিং, NTP, QUIC
উপমা	রেজিস্টার্ড চিঠি / ফোনকল	পোস্টকার্ড / রেডিও সম্প্রচার

ICMP তৃতীয় একটা প্রোটোকল, যেটা Layer 3-এ থাকে। এতে কোনো port নেই, কাজ হলো ত্রুটি আর নিয়ন্ত্রণ বার্তা পাঠানো। `ping` (echo request/reply) আর `traceroute` ICMP দিয়ে চলে।

Well-known ports টেবিল (মুখস্থ করার মতো)

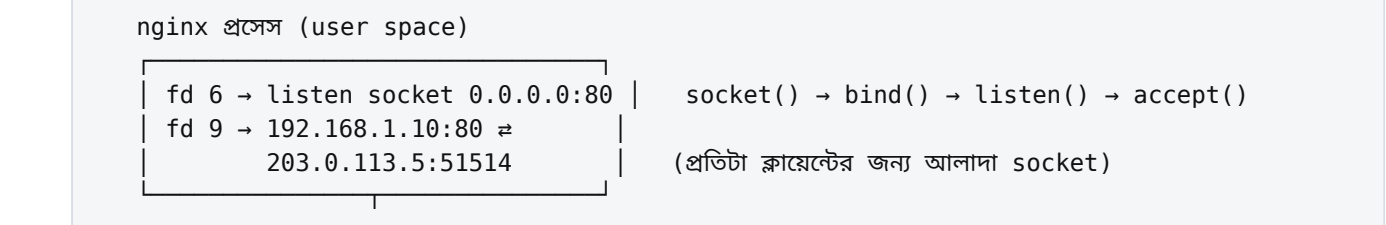
Port	Protocol	সার্ভিস	মন্তব্য
20, 21	TCP	FTP (ডেটা, কন্ট্রোল)	এনক্রিপশন নেই, পুরনো
22	TCP	SSH / SFTP / SCP	অধ্যায় ৩৮
23	TCP	Telnet	এনক্রিপশন নেই, ব্যবহার কোরো না
25	TCP	SMTP (সার্ভার থেকে সার্ভার মেইল)	অনেক ISP/cloud ব্লক করে রাখে
53	UDP/TCP	DNS	
67, 68	UDP	DHCP (সার্ভার, ক্লায়েন্ট)	
69	UDP	TFTP	নেটওয়ার্ক বুট (PXE)
80	TCP	HTTP	অধ্যায় ৪০
110	TCP	POP3	
123	UDP	NTP (সময় সিঙ্ক)	

Port	Protocol	সার্ভিস	মন্তব্য
143	TCP	IMAP	
161, 162	UDP	SNMP	নেটওয়ার্ক মনিটরিং
389 / 636	TCP	LDAP / LDAPS	
443	TCP (+UDP, HTTP/3)	HTTPS	
445	TCP	SMB (Windows ফাইল শেয়ার, Samba)	
465	TCP	SMTPS (implicit TLS)	
514	UDP	Syslog	অধ্যায় ২৮
587	TCP	SMTP submission (মেইল ক্লায়েন্ট থেকে)	
853	TCP	DNS over TLS	
993 / 995	TCP	IMAPS / POP3S	
2049	TCP	NFS	
3306	TCP	MySQL / MariaDB	
3389	TCP	RDP (Windows Remote Desktop)	
5432	TCP	PostgreSQL	
5900	TCP	VNC	
6379	TCP	Redis	
8080, 8443	TCP	বিকল্প HTTP/HTTPS (রীতি, নিয়ম নয়)	
9090	TCP	Cockpit (RHEL/Fedora ওয়েব কনসোল)	
27017	TCP	MongoDB	

> ⚠️ **সতর্কতা:** port নম্বর কেবল **রীতি**। কেউ চাইলে SSH 2222-তে বা ওয়েব সার্ভার 8000-এ চালাতে পারে। "port 80 খোলা মানেই ওয়েব সার্ভার" ধরে নিও না।

৩৬.১০ Socket: প্রোগ্রাম আর নেটওয়ার্কের সংযোগবিন্দু

socket হলো `IP:port`-এর একটা প্রান্ত, যেটা দিয়ে প্রোগ্রাম নেটওয়ার্কে কথা বলে। লিনাক্সে "সবকিছুই ফাইল" (অধ্যায় ৭), আর socket-ও প্রোগ্রামের কাছে একটা **file descriptor** হিসেবে দেখা দেয়।



```
═══════════════════╪═══════════════════  system call সীমানা
kernel             ▼
   ┌──────────────────────────────────┐
   │ TCP/IP stack: socket buffer,     │
   │ সংযোগের অবস্থা, retransmit       │
   │ netfilter (firewall, NAT)        │
   │ routing → NIC driver             │
   └──────────────┬───────────────────┘
                  ▼
              NIC (enp0s3)
```

একই মেশিনের দুই প্রোগ্রামের মধ্যে যোগাযোগের জন্য **Unix domain socket** আছে, যেটা ফাইলসিস্টেমে একটা ফাইল হিসেবে থাকে (যেমন `/run/php/php8.3-fpm.sock`, `/var/run/docker.sock`)। এটা TCP-র চেয়ে দ্রুত, কারণ নেটওয়ার্ক stack-এর ভেতর দিয়ে যায় না।

৩৬.১১ NAT (Network Address Translation)

সমস্যাটা কী

তোমার বাসায় ১০টা ডিভাইস, সবার private IP (`192.168.1.x`)। কিন্তু ISP তোমাকে **একটা** public IP দিয়েছে, আর private IP ইন্টারনেটে চলে না। তাহলে সবাই ইন্টারনেট পায় কীভাবে?

সমাধান: NAT

রাউটার বাইরে যাওয়া প্রতিটা প্যাকেটের **source IP (আর দরকারে port)** বদলে নিজের public IP বসিয়ে দেয়, আর মনে রাখে কোন সংযোগ কার। উত্তর ফিরে এলে উল্টো অনুবাদ করে ঠিক ডিভাইসে পাঠায়। একটা IP-র পেছনে অনেক ডিভাইস চালানোর এই কৌশলের নাম **PAT (Port Address Translation)** বা **NAT overload**। লিনাক্সে একে **masquerade** বলে।

```
  বাসার LAN (private)                 রাউটার (NAT)                      ইন্টারনেট

                                   ┌──────────────────┐
 ল্যাপটপ 192.168.1.10:51514 ─────► │ NAT টেবিল:       │ ─► 103.15.20.7:40001 ─► 93.184.215.34:443
                                   │ .10:51514 ⇄ :40001 │
 ফোন    192.168.1.23:42000 ─────► │ .23:42000 ⇄ :40002 │ ─► 103.15.20.7:40002 ─► 142.250.193.46:443
                                   └──────────────────┘
                                     WAN IP: 103.15.20.7
      উত্তর আসে 103.15.20.7:40002-এ → টেবিল দেখে → 192.168.1.23:42000-এ পাঠায়
```

> 📝 **মনে রাখো:** রাউটার এই সংযোগের তালিকা রাখে **connection tracking** টেবিলে। লিনাক্সে এটা kernel-এর **conntrack** (netfilter-এর অংশ)। Docker কন্টেইনার যে ইন্টারনেট পায়, সেটাও এই লিনাক্স NAT দিয়ে।

NAT-এর ধরন

ধরন	কী বদলায়	উদাহরণ
SNAT (Source NAT)	উৎসের IP	অফিসের সব PC একটা নির্দিষ্ট public IP দিয়ে বাইরে যায়
Masquerade	উৎসের IP, interface-এর বর্তমান IP দিয়ে (IP বদলালেও চলে)	বাসার রাউটার, Docker, লিনাক্স হটস্পট
DNAT (Destination NAT) / **Port forwarding**	গন্তব্যের IP/port	বাইরে থেকে `public-IP:8080`-এ এলে ভেতরের `192.168.1.50:80`-এ পাঠাও

NAT-এর একটা পার্শ্বফল আছে: বাইরে থেকে কেউ সরাসরি ভেতরের ডিভাইসে সংযোগ শুরু করতে পারে না, যদি না port forwarding করা থাকে। এটা কিছুটা নিরাপত্তা দেয়, তবে NAT ফায়ারওয়ালের বিকল্প নয়।

> ⚠️ **সতর্কতা:** অনেক ISP এখন **CGNAT** ব্যবহার করে। তোমার রাউটারের WAN IP হয়তো `100.64.x.x`, মানে ISP-র আরেকটা NAT-এর পেছনে তুমি। তখন রাউটারে port forwarding করলেও বাইরে থেকে ঢোকা যাবে না। এই অবস্থায় ISP-র কাছে public IP চাইতে হয়, অথবা Tailscale বা Cloudflare Tunnel জাতীয় সমাধান ব্যবহার করতে হয়।

লিনাক্সে NAT কীভাবে কনফিগ করতে হয় (`iptables -t nat ... MASQUERADE`, `nft`, `firewall-cmd --add-masquerade`), সেটা অধ্যায় ৩৯-এ দেখানো হবে।

৩৬.১২ Interface-এর নামকরণ: `enp0s3` কেন?

আগে লিনাক্সে interface-এর নাম হতো `eth0`, `eth1`, `wlan0`। সমস্যা ছিল, kernel যে ক্রমে কার্ড খুঁজে পেত, নাম সেই ক্রমে দেওয়া হতো। কোনো বুটে একটা কার্ডের নাম `eth0`, পরের বুটে সেটাই `eth1`, আর তখন ফায়ারওয়াল নিয়ম চলে যেত ভুল কার্ডে।

এর সমাধান **predictable network interface names**। systemd-udevd হার্ডওয়্যারের **অবস্থান** দেখে নাম দেয়, তাই প্রতি বুটে নাম একই থাকে।

```
e n p 0 s 3
└┘ └┘└─┘
 │   │  └── অবস্থান: PCI bus 0, slot 3
 │   └───── ধরন: en = Ethernet
 └─────────
```

উপসর্গ (ধরন)	মানে
`en`	Ethernet
`wl`	Wireless LAN (Wi-Fi)

উপসর্গ (ধরন)	মানে
`ww`	Wireless WAN (মোবাইল ডেটা মডেম)
`ib`	InfiniBand
`sl`	Serial line IP

অবস্থানের অংশ	মানে	উদাহরণ
`o<index>`	মাদারবোর্ডে বসানো (onboard), BIOS/firmware-এর দেওয়া ইনডেক্স	`eno1`
`s<slot>`	PCI Express hotplug স্লট নম্বর	`ens33` (VMware-এ খুব দেখা যায়)
`p<bus>s<slot>`	PCI bus আর slot অনুযায়ী	`enp0s3` (VirtualBox), `wlp2s0`
`x<MAC>`	MAC অ্যাড্রেস অনুযায়ী	`wlx00c0ca123456` (USB Wi-Fi)

অন্য পরিচিত নাম, যেগুলো হার্ডওয়্যার নয়, **ভার্চুয়াল interface**:

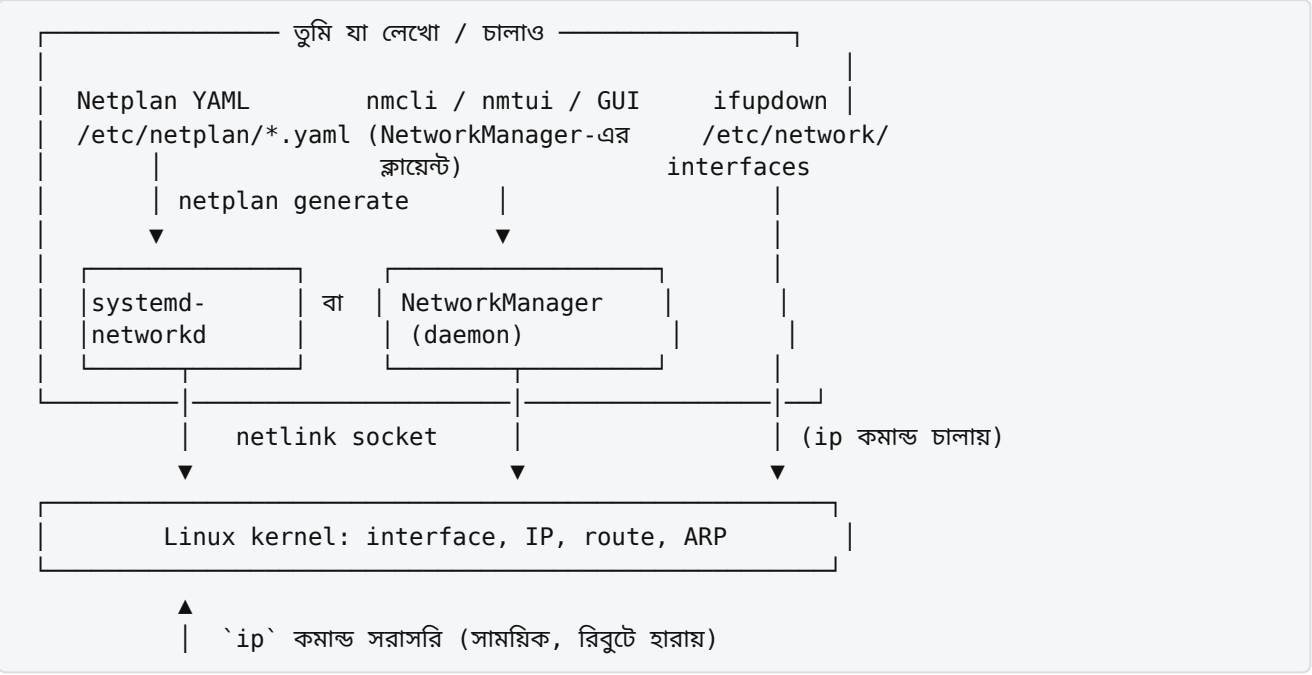
নাম	কী
`lo`	loopback, সবসময় থাকে
`docker0`	Docker-এর bridge
`veth...`	কন্টেইনারের ভার্চুয়াল তারের এক প্রান্ত (অধ্যায় ৪৮)
`virbr0`	libvirt/KVM-এর NAT bridge
`br0`	হাতে বানানো bridge
`bond0`	একাধিক NIC জোড়া দিয়ে বানানো bond
`tun0`, `wg0`	VPN (OpenVPN, WireGuard)
`enp0s3.100`	VLAN 100-এর sub-interface

```
$ ls /sys/class/net/
docker0  enp0s3  lo  wlp2s0
$ udevadm info /sys/class/net/enp0s3 | grep ID_NET_NAME
E: ID_NET_NAME_MAC=enx0800274e66a1
E: ID_NET_NAME_PATH=enp0s3
```

> 💡 **টিপস:** পুরনো `eth0` ধাঁচের নাম ফিরিয়ে আনা যায় kernel parameter `net.ifnames=0 biosdevname=0` দিয়ে (অধ্যায় ২৭), কিন্তু এর দরকার প্রায় কখনো পড়ে না। ক্লাউড VM-এ (AWS, GCP) এখনো `eth0` বা `ens5` জাতীয় নাম দেখবে, সেটা স্বাভাবিক।

৩৬.১৩ লিনাক্সে নেটওয়ার্ক কনফিগারেশন: বড় ছবি

লিনাক্সে নেটওয়ার্ক কনফিগ করার একাধিক "ব্যবস্থাপক" আছে, আর শুরুতে এটাই সবচেয়ে বিভ্রান্তিকর লাগে। চাবিকাঠি হলো এটা মনে রাখা: **আসল কাজ সবসময় kernel করে।** বাকিরা kernel-কে নির্দেশ দেওয়ার আলাদা আলাদা উপায় মাত্র।



Distro	ডিফল্ট ব্যবস্থা
Ubuntu 24.04 Desktop	Netplan → NetworkManager (renderer)
Ubuntu 24.04 Server	Netplan → systemd-networkd
Fedora (Workstation/Server)	NetworkManager
RHEL 9 / Rocky / Alma	NetworkManager (keyfile ফরম্যাট)
Debian 12 Desktop	NetworkManager
Debian 12 Server (মিনিমাল)	ifupdown (`/etc/network/interfaces`)

> ⚠️ **সতর্কতা:** `sudo ip addr add ...` দিয়ে করা পরিবর্তন **সাময়িক**, রিবুট দিলেই চলে যায়। পরীক্ষার জন্য ঠিক আছে, কিন্তু স্থায়ী কনফিগ সবসময় ওপরের কোনো একটা ব্যবস্থা দিয়ে করো। আর একই interface দুইটা ব্যবস্থাপককে দিও না (যেমন NetworkManager আর ifupdown একসাথে), তাহলে তারা একে অপরের কাজ উল্টে দেবে।

স্ট্যাটিক IP বসানোর আগে যা জেনে নেবে

তথ্য	উদাহরণ	কোথায় পাবে
interface-এর নাম	`enp0s3`	`ip -br link`

তথ্য	উদাহরণ	কোথায় পাবে
IP/prefix	`192.168.1.50/24`	নেটওয়ার্ক অ্যাডমিন; DHCP রেঞ্জের **বাইরের** একটা ঠিকানা
gateway	`192.168.1.1`	`ip route` এর `default via`
DNS	`1.1.1.1`, `8.8.8.8`	`resolvectl status` বা নিজের পছন্দ

> ⚠️ **সতর্কতা:** রিমোট সার্ভারে (SSH দিয়ে ঢুকে) নেটওয়ার্ক কনফিগ বদলানোর সময় একটা ভুলেই তুমি নিজেকে বাইরে আটকে ফেলতে পারো। সেক্ষেত্রে cloud console বা আউট-অফ-ব্যান্ড অ্যাক্সেস হাতের কাছে রাখো। Netplan-এ এর জন্য `netplan try` আছে, নিচে দেখো।

৩৬.১৪ NetworkManager: `nmcli` ও `nmtui`

NetworkManager একটা daemon (ব্যাকগ্রাউন্ড সার্ভিস)। এটা interface চালু করে, Wi-Fi-তে যুক্ত হয়, DHCP চালায়, VPN সামলায়, আর নেটওয়ার্ক বদলালে (তার খুলে Wi-Fi-তে গেলে) স্বয়ংক্রিয়ভাবে খাপ খাইয়ে নেয়। ডেস্কটপের ওপরের কোণার নেটওয়ার্ক আইকনটা আসলে এরই GUI ক্লায়েন্ট।

দুটো ধারণা আলাদা করে বুঝতে হবে:

- **Device**: আসল interface (`enp0s3`, `wlp2s0`)
- **Connection (profile)**: সেটিংসের একটা সংকলন (IP, DNS, Wi-Fi পাসওয়ার্ড), যেটা কোনো device-এ প্রয়োগ করা যায়। একটা device-এর অনেক profile থাকতে পারে (যেমন "অফিস স্ট্যাটিক", "বাসা DHCP"), তবে একসময়ে চালু থাকে একটাই।

profile সংরক্ষিত থাকে `/etc/NetworkManager/system-connections/` ফোল্ডারে, `.nmconnection` ফাইল হিসেবে (keyfile ফরম্যাট, ফাইলের অনুমতি 600, কারণ ভেতরে Wi-Fi পাসওয়ার্ড থাকে)। RHEL 9 থেকে এটাই ডিফল্ট। পুরনো `/etc/sysconfig/network-scripts/ifcfg-*` ফরম্যাট deprecated।

```
$ systemctl status NetworkManager --no-pager | head -3
● NetworkManager.service - Network Manager
     Loaded: loaded (/usr/lib/systemd/system/NetworkManager.service; enabled; preset:
enabled)
     Active: active (running) since Tue 2026-11-10 09:12:44 +06; 2h 3min ago
```

`nmcli`: কমান্ড লাইন ক্লায়েন্ট

Syntax: `nmcli [OPTIONS] OBJECT { COMMAND | help }`

OBJECT হতে পারে `general`, `networking`, `radio`, `connection`, `device`, `monitor`। প্রতিটা শব্দ সংক্ষেপেও লেখা যায়: `nmcli c` = `nmcli connection`, `nmcli d` = `nmcli device`, `nmcli c s` = `nmcli connection show`।

গুরুত্বপূর্ণ অপশন	কাজ
`-t`, `--terse`	স্ক্রিপ্টের জন্য সংক্ষিপ্ত `:`-বিভক্ত আউটপুট
`-f`, `--fields`	শুধু নির্দিষ্ট কলাম দেখাও
`-p`, `--pretty`	মানুষের পড়ার জন্য সাজানো আউটপুট
`-a`, `--ask`	দরকার হলে পাসওয়ার্ড জিজ্ঞেস করো
`-g`, `--get-values`	শুধু মানগুলো (`-t -f`-এর সংক্ষেপ)
`-c no`, `--colors no`	রং বন্ধ

অবস্থা দেখা

```
$ nmcli general status
STATE      CONNECTIVITY  WIFI-HW  WIFI     WWAN-HW  WWAN
connected  full          enabled  enabled  missing  enabled
```

```
$ nmcli device status
DEVICE   TYPE      STATE                   CONNECTION
enp0s3   ethernet  connected               Wired connection 1
wlp2s0   wifi      disconnected            --
docker0  bridge    connected (externally)  docker0
lo       loopback  connected (externally)  lo
```

```
$ nmcli connection show
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  3c9e1b52-7a0d-3f43-9a8e-2b8f5f1d6c11  ethernet  enp0s3
HomeWiFi            8b2a7e5c-1d44-4c1e-8f0b-6a1d2e3f4a5b  wifi      --
```

একটা profile-এর সব সেটিং দেখতে: `nmcli connection show "Wired connection 1"`। device-এর বর্তমান IP, gateway, DNS দেখতে: `nmcli device show enp0s3`।

```
$ nmcli -f IP4 device show enp0s3
IP4.ADDRESS[1]:                         192.168.1.10/24
IP4.GATEWAY:                            192.168.1.1
IP4.ROUTE[1]:                           dst = 192.168.1.0/24, nh = 0.0.0.0, mt = 100
IP4.ROUTE[2]:                           dst = 0.0.0.0/0, nh = 192.168.1.1, mt = 100
IP4.DNS[1]:                             192.168.1.1
```

Wi-Fi

```
$ nmcli radio wifi on
$ nmcli device wifi list
IN-USE  BSSID              SSID       MODE   CHAN  RATE        SIGNAL  BARS  SECURITY
        A4:2B:B0:11:22:33  HomeWiFi   Infra  36    540 Mbit/s  82      ▂▄▆█  WPA2
        C8:3A:35:44:55:66  Neighbor   Infra  6     130 Mbit/s  40      ▂▄__  WPA2 WPA3

$ nmcli device wifi connect HomeWiFi --ask
```

```
Password: ••••••••
Device 'wlp2s0' successfully activated with '8b2a7e5c-1d44-4c1e-8f0b-6a1d2e3f4a5b'.
```

> 💡 **টিপস:** `nmcli device wifi connect HomeWiFi password 'mypass'` লিখলেও কাজ হয়, কিন্তু তখন পাসওয়ার্ড shell history-তে থেকে যায় (অধ্যায় ৬)। তাই `--ask` বেশি নিরাপদ। হটস্পট বানাতে: `nmcli device wifi hotspot ssid MyHotspot password 'secret123'`। পাসওয়ার্ড QR কোডে দেখতে: `nmcli device wifi show-password`।

স্ট্যাটিক IP: বিদ্যমান profile বদলে

```
$ sudo nmcli connection modify "Wired connection 1" \
    ipv4.method manual \
    ipv4.addresses 192.168.1.50/24 \
    ipv4.gateway 192.168.1.1 \
    ipv4.dns "1.1.1.1 8.8.8.8" \
    ipv4.dns-search example.local
$ sudo nmcli connection up "Wired connection 1"
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/6)
```

> 📝 **মনে রাখো:** `modify` শুধু profile ফাইল বদলায়। চালু থাকা interface-এ সেটা প্রয়োগ হয় **`connection up`** (বা `device reapply`) চালালে। আর `ipv4.method manual` না দিয়ে শুধু ঠিকানা যোগ করলে DHCP-ও চলতে থাকবে, ফলে দুটো IP পাবে।

বিদ্যমান মানের সাথে যোগ বা বাদ দিতে `+` আর `-` উপসর্গ:

```
$ sudo nmcli connection modify "Wired connection 1" +ipv4.addresses 192.168.1.51/24
$ sudo nmcli connection modify "Wired connection 1" -ipv4.dns 8.8.8.8
```

আবার DHCP-তে ফেরত যেতে:

```
$ sudo nmcli connection modify "Wired connection 1" ipv4.method auto ipv4.addresses ""
ipv4.gateway "" ipv4.dns ""
$ sudo nmcli connection up "Wired connection 1"
```

DHCP চালু রেখে শুধু নিজের DNS ব্যবহার করতে চাইলে:

```
$ sudo nmcli connection modify "Wired connection 1" ipv4.ignore-auto-dns yes ipv4.dns
"1.1.1.1 9.9.9.9"
```

নতুন profile তৈরি

```
$ sudo nmcli connection add type ethernet ifname enp0s8 con-name lab-static \
    ipv4.method manual ipv4.addresses 10.0.0.5/24 ipv4.gateway 10.0.0.1 ipv4.dns
10.0.0.1 \
    connection.autoconnect yes
Connection 'lab-static' (5f6a...) successfully added.
```

`nmcli`-র দরকারি কমান্ডের তালিকা

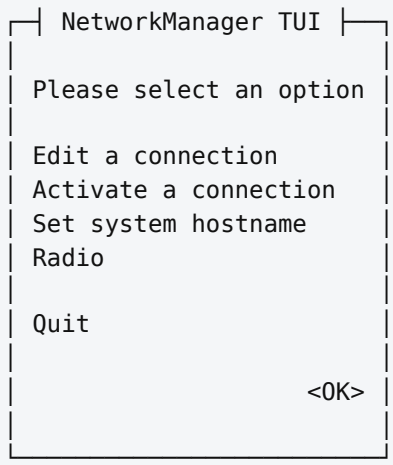
কমান্ড	কাজ
`nmcli general status`	সামগ্রিক অবস্থা
`nmcli general hostname NAME`	hostname বদলানো
`nmcli networking off` / `on`	পুরো নেটওয়ার্কিং বন্ধ/চালু
`nmcli radio wifi off` / `on`	Wi-Fi রেডিও বন্ধ/চালু
`nmcli device status`	সব device-এর অবস্থা
`nmcli device show IFACE`	device-এর বিস্তারিত (IP, DNS, gateway)
`nmcli device connect IFACE` / `disconnect IFACE`	device সংযুক্ত/বিচ্ছিন্ন
`nmcli device reapply IFACE`	profile-এর পরিবর্তন সংযোগ না কেটে প্রয়োগ
`nmcli device wifi list` / `wifi rescan`	Wi-Fi তালিকা / নতুন করে স্ক্যান
`nmcli device wifi connect SSID --ask`	Wi-Fi-তে যুক্ত হওয়া
`nmcli connection show [--active]`	সব (বা শুধু চালু) profile
`nmcli connection up NAME` / `down NAME`	profile চালু/বন্ধ
`nmcli connection add ...`	নতুন profile
`nmcli connection modify NAME prop val`	profile বদলানো
`nmcli connection edit NAME`	ইন্টারঅ্যাকটিভ এডিটর (`print`, `set`, `save`, `quit`)
`nmcli connection delete NAME`	profile মুছে ফেলা
`nmcli connection reload`	হাতে এডিট করা `.nmconnection` ফাইল পুনরায় পড়া
`nmcli connection import type openvpn file x.ovpn`	VPN কনফিগ ইমপোর্ট (প্লাগইন লাগে)
`nmcli monitor`	রিয়েল-টাইমে পরিবর্তন দেখা (Ctrl+C দিয়ে বের হও)

> 💡 **টিপস:** `nmcli`-তে **Tab completion** দারুণ কাজ করে। `nmcli connection modify "Wired connection 1" ipv4.` লিখে দুবার Tab চাপলে সব property-র নাম দেখাবে। প্রায় সব property-র ব্যাখ্যা আছে `man nm-settings`-এ।

`nmtui`: টার্মিনালের মেনু-ভিত্তিক ইন্টারফেস

কমান্ড মনে না থাকলে বা মাউস ছাড়া সার্ভারে কাজ করতে হলে **`nmtui`** (NetworkManager Text User Interface) খুব আরামের। Ubuntu-তে এটা `network-manager` প্যাকেজেই আসে, Fedora/RHEL-এ আলাদা প্যাকেজ `NetworkManager-tui`।

```
$ sudo nmtui
```



সরাসরি নির্দিষ্ট পাতায় যেতে: `nmtui edit`, `nmtui connect` (= Activate), `nmtui hostname`।

`nmtui` কীবোর্ড শর্টকাট:

কী	কাজ
`↑` `↓`	তালিকায় ওপরে-নিচে
`Tab` / `Shift+Tab`	পরের / আগের ঘর বা বোতাম
`Enter`	নির্বাচন, বোতাম চাপা, সাব-মেনু খোলা
`Space`	চেকবক্স `[X]` টগল করা
`Esc`	পেছনে যাওয়া / বাতিল
`←` `→`	ঘরের ভেতরে কার্সার, বা বোতামগুলোর মধ্যে চলাফেরা
`<Show>` বোতাম	IPv4/IPv6 কনফিগারেশনের বিস্তারিত ঘর দেখানো

স্ট্যাটিক IP বসানোর ধাপ: Edit a connection → profile বেছে `<Edit...>` → IPv4 CONFIGURATION-এর পাশে `<Automatic>` বদলে `<Manual>` → `<Show>` → Addresses-এ `192.168.1.50/24`, Gateway, DNS servers → নিচে `<OK>` → মূল মেনুতে "Activate a connection" থেকে profile-টা Deactivate করে আবার Activate।

৩৬.১৫ Netplan (Ubuntu)

Ubuntu ১৭.১০ থেকে নেটওয়ার্ক কনফিগ লেখা হয় **Netplan**-এর YAML ফাইলে। Netplan নিজে নেটওয়ার্ক চালায় না। সে YAML পড়ে **renderer**-এর (systemd-networkd বা NetworkManager) জন্য কনফিগ বানিয়ে দেয়। ফলে সব Ubuntu মেশিনে এক রকম ফাইল লিখলেই চলে, পেছনে যে-ই থাকুক।

```
/etc/netplan/*.yaml  ──netplan generate──►  /run/systemd/network/*.network
(renderer: networkd)
   (তুমি লেখো)                          বা  /run/NetworkManager/system-connections/
(renderer: NetworkManager)
                                                        │
                                          netplan apply → ব্যাকএন্ড রিস্টার্ট/রিলোড → kernel
```

ফাইলগুলো **বর্ণানুক্রমে** পড়া হয়, আর পরের ফাইলের মান আগের ফাইলকে ছাপিয়ে যায়। সাধারণ নাম: `01-netcfg.yaml`, `50-cloud-init.yaml`, `90-NM-<uuid>.yaml` (Ubuntu 23.10 থেকে ডেস্কটপে NetworkManager-এর profile-ও Netplan-এর YAML হিসেবে এখানে রাখা হয়)।

```
$ ls -l /etc/netplan/
-rw------- 1 root root 412 Apr 23 10:12 50-cloud-init.yaml
```

উদাহরণ ১: DHCP (সার্ভার)

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: true
```

উদাহরণ ২: স্ট্যাটিক IP

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.1.50/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
        search: [example.local]
```

> ⚠️ **সতর্কতা:** পুরনো টিউটোরিয়ালে `gateway4: 192.168.1.1` দেখবে। এটা **deprecated**, এখন চালালে সতর্কবার্তা দেয়। আধুনিক নিয়ম হলো `routes:` এর ভেতরে `to: default` আর `via:`।

> ⚠️ **সতর্কতা:** YAML-এ **indentation-ই সব**। ট্যাব দেবে না, শুধু স্পেস (সাধারণত ২টা)। একটা স্পেস কম-বেশি হলেই কনফিগ ভেঙে যাবে। এডিট করতে পারো `nano` (অধ্যায় ১৩) বা `vim` (অধ্যায় ১৪) দিয়ে।

উদাহরণ ৩: Wi-Fi

```
network:
  version: 2
  renderer: NetworkManager
  wifis:
    wlp2s0:
      dhcp4: true
      access-points:
        "HomeWiFi":
          password: "mypassword"
```

উদাহরণ ৪: ডেস্কটপে সবকিছু NetworkManager-এর হাতে ছেড়ে দেওয়া

```
network:
  version: 2
  renderer: NetworkManager
```

Netplan কমান্ড

কমান্ড	কাজ
`sudo netplan generate`	YAML থেকে ব্যাকএন্ড কনফিগ বানাও (প্রয়োগ করে না; সিনট্যাক্স পরীক্ষায় কাজে লাগে)
`sudo netplan try`	প্রয়োগ করে **১২০ সেকেন্ড** অপেক্ষা করে; Enter না চাপলে আগের অবস্থায় ফেরত যায়
`sudo netplan try --timeout 30`	অপেক্ষার সময় বদলানো
`sudo netplan apply`	স্থায়ীভাবে প্রয়োগ
`sudo netplan get`	মিলিত চূড়ান্ত কনফিগ দেখাও
`sudo netplan get ethernets.enp0s3`	নির্দিষ্ট অংশ দেখাও
`sudo netplan set ethernets.enp0s3.dhcp4=true`	ফাইল না খুলেই মান বদলাও
`sudo netplan status`	interface-গুলোর বর্তমান অবস্থা (Ubuntu 23.10+/Netplan 0.107+)
`sudo netplan --debug apply`	বিস্তারিত লগ সহ প্রয়োগ

```
$ sudo netplan try
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 117 seconds
Configuration accepted.
```

> 💡 **টিপস:** রিমোট সার্ভারে **সবসময়** `netplan try` ব্যবহার করো। ভুল কনফিগে SSH কেটে গেলে তুমি Enter চাপতে পারবে না, আর ২ মিনিট পর কনফিগ নিজে থেকেই আগের অবস্থায় ফিরে আসবে। এই একটা অভ্যাস অনেক রাত-জাগা থেকে বাঁচায়।

ফাইলের অনুমতি বেশি খোলা থাকলে `netplan` এরকম সতর্কবার্তা দেয়: `Permissions for /etc/netplan/01-netcfg.yaml are too open. Netplan configuration should NOT be accessible by others.` সমাধান: `sudo chmod 600 /etc/netplan/*.yaml` (অধ্যায় ১৬)।

> 📝 **মনে রাখো:** cloud VM-এ (AWS, Azure) `50-cloud-init.yaml` ফাইলটা **cloud-init** লেখে, আর প্রতি বুটে নতুন করে লিখে ফেলতে পারে। তাই নিজের পরিবর্তন আলাদা ফাইলে রাখো (যেমন `90-custom.yaml`), অথবা `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` ফাইলে `network: {config: disabled}` লিখে cloud-init-এর নেটওয়ার্ক কনফিগ বন্ধ করো।

systemd-networkd নিয়ে এক নজর

Netplan-এর renderer `networkd` হলে আসল কাজ করে **systemd-networkd**। এর অবস্থা দেখার টুল `networkctl`:

```
$ networkctl
IDX LINK   TYPE     OPERATIONAL SETUP
  1 lo     loopback carrier     unmanaged
  2 enp0s3 ether    routable    configured

2 links listed.
$ networkctl status enp0s3
```

Netplan ছাড়াও systemd-networkd সরাসরি ব্যবহার করা যায়, `/etc/systemd/network/*.network` ফাইল লিখে (Arch-এ যেমন হয়)।

৩৬.১৬ Debian-এর ifupdown আর RHEL/Fedora-র ধরন

Debian 12: `/etc/network/interfaces`

মিনিমাল Debian সার্ভারে নেটওয়ার্ক চালায় **ifupdown**:

```
# /etc/network/interfaces
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

# DHCP
allow-hotplug enp0s3
iface enp0s3 inet dhcp

# অথবা স্ট্যাটিক:
# auto enp0s3
# iface enp0s3 inet static
#     address 192.168.1.50/24
#     gateway 192.168.1.1
#     dns-nameservers 1.1.1.1 8.8.8.8     # resolvconf প্যাকেজ থাকলে তবেই কাজ করে
```

কমান্ড	কাজ
`sudo ifup enp0s3`	interface চালু করো (ফাইলের কনফিগ অনুযায়ী)
`sudo ifdown enp0s3`	বন্ধ করো
`sudo systemctl restart networking`	সব নতুন করে প্রয়োগ (রিমোটে সাবধানে!)

> 📝 **মনে রাখো:** `resolvconf` প্যাকেজ না থাকলে `dns-nameservers` লাইন কাজ করে না। তখন `/etc/resolv.conf` হাতেই লিখতে হয় (ifupdown + স্ট্যাটিক IP হলে এটা বৈধ, কারণ এই ক্ষেত্রে ফাইলটা আর কেউ লেখে না)। NetworkManager যদি এই ফাইলে উল্লিখিত কোনো interface দেখে, সেটাকে সে নিজে নিয়ন্ত্রণ করে না।

RHEL 9 / Fedora

এখানে সবকিছু **NetworkManager**। ওপরের `nmcli` / `nmtui` ছাড়াও ওয়েব-ভিত্তিক **Cockpit** (`https://server:9090`) দিয়ে GUI-তে নেটওয়ার্ক কনফিগ করা যায়।

```
$ ls /etc/NetworkManager/system-connections/
enp1s0.nmconnection
$ sudo cat /etc/NetworkManager/system-connections/enp1s0.nmconnection
[connection]
id=enp1s0
uuid=0f3b7c1e-....
type=ethernet
interface-name=enp1s0
```

```
[ipv4]
method=manual
address1=192.168.1.50/24,192.168.1.1
dns=1.1.1.1;8.8.8.8;

[ipv6]
method=auto
```

এই ফাইল হাতে এডিট করলে পরে `sudo nmcli connection reload` আর `sudo nmcli connection up enp1s0` চালাতে হয়।

৩৬.১৭ Hostname

hostname হলো নেটওয়ার্কে কম্পিউটারের নাম। systemd-ভিত্তিক সব distro-তে এটা সামলানোর টুল `hostnamectl`:

```
$ hostnamectl
 Static hostname: mylaptop
       Icon name: computer-laptop
         Chassis: laptop 💻
      Machine ID: 3f2b...
         Boot ID: 9a1c...
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
    Architecture: x86-64

$ sudo hostnamectl set-hostname web01
$ hostname
web01
```

স্থায়ী hostname জমা থাকে `/etc/hostname` ফাইলে। নাম বদলানোর পর `/etc/hosts`-এর `127.0.1.1` লাইনটাও (থাকলে) নতুন নামে বদলে দাও, নইলে `sudo` চালাতে গেলে "unable to resolve host" বার্তা দেখাবে।

কমান্ড	দেখায়
`hostname`	ছোট নাম
`hostname -f`	FQDN (Fully Qualified Domain Name), যেমন `web01.example.com`
`hostname -I`	সব interface-এর IP (loopback আর link-local বাদে)

৩৬.১৮ একটা ওয়েবপেজ খোলার পূর্ণ যাত্রা (সব একসাথে)

এখন পর্যন্ত যা শিখেছ, সব একটা গল্পে বেঁধে ফেলা যাক। তুমি টার্মিনালে `curl https://example.com` চালালে:

```
১. curl → getaddrinfo("example.com")
       → nsswitch: /etc/hosts-এ নেই → systemd-resolved (127.0.0.53)
       → cache-এ নেই → upstream DNS 192.168.1.1-এ UDP:53 → … → উত্তর 93.184.215.34
[DNS]

২. curl → socket() + connect(93.184.215.34:443)
[socket]

৩. kernel: routing table দেখে → 192.168.1.0/24-এ নয় → default via 192.168.1.1
[routing]

৪. kernel: 192.168.1.1-এর MAC কী? → ARP cache-এ না থাকলে ARP request
[ARP]

৫. kernel: ephemeral port (যেমন 51514) বেছে TCP SYN পাঠায়
           [Eth: আমার MAC→রাউটারের MAC][IP: 192.168.1.10→93.184.215.34][TCP 51514→443
SYN]   [encapsulation]

৬. রাউটার: NAT → উৎস বদলে 103.15.20.7:40001 → ISP → ইন্টারনেট → গন্তব্য
[NAT]

৭. 3-way handshake সম্পূর্ণ → TLS handshake (এনক্রিপশন) → HTTP GET /
[TCP, TLS]

৮. উত্তর ফেরে উল্টো পথে → রাউটার NAT টেবিল দেখে → 192.168.1.10:51514 → kernel → curl
[decapsulation]
```

এই পুরো ছবিটা মাথায় থাকলে যেকোনো নেটওয়ার্ক সমস্যায় তুমি ধাপে ধাপে খুঁজতে পারবে: "নাম resolve হচ্ছে? gateway-তে পৌঁছাচ্ছে? port খোলা?" ঠিক এই পদ্ধতিটাই আমরা অধ্যায় ৩৭-এ কমান্ড দিয়ে চর্চা করব।

হাতে-কলমে অনুশীলন

> ⚠️ **সতর্কতা:** এই ল্যাবগুলো একটা VM-এ (VirtualBox, VMware, KVM) বা নিজের ডেস্কটপে করো, প্রোডাকশন সার্ভারে নয়। স্ট্যাটিক IP বসানোর ল্যাব রিমোট সার্ভারে করবে না।

ল্যাব ১: নিজের নেটওয়ার্কের পরিচয় বের করো

```
$ ip -br link                  # ১. interface-এর নাম আর MAC
$ ip -br addr                  # ২. প্রতিটার IP/prefix
$ ip route                     # ৩. default gateway কে?
$ ip route get 1.1.1.1         # ৪. বাইরে যেতে কোন পথ?
$ ip neigh                     # ৫. gateway-এর MAC কী?
$ resolvectl status 2>/dev/null || cat /etc/resolv.conf     # ৬. DNS সার্ভার কে?
$ hostnamectl                  # ৭. hostname
```

একটা কাগজে লিখে রাখো: interface, IP, prefix, network ঠিকানা (নিজে হিসাব করো), broadcast, gateway, gateway-এর MAC, DNS।

ল্যাব ২: CIDR হিসাব অনুশীলন

হাতে হিসাব করো, তারপর `ipcalc` দিয়ে মিলিয়ে নাও:

1. `172.16.45.77/22`
2. `10.1.1.1/30`
3. `192.168.100.200/27`

```
$ sudo apt install ipcalc      # Fedora/RHEL: sudo dnf install ipcalc
$ ipcalc 172.16.45.77/22
```

▼ উত্তর দেখো

1. `/22` → তৃতীয় octet-এ mask 252, block 4। 45 পড়ছে 44-এর ব্লকে → Network `172.16.44.0`, Broadcast `172.16.47.255`, হোস্ট `172.16.44.1` – `172.16.47.254`, মোট 1022টা।
2. `/30` → block 4। 1 পড়ছে 0-র ব্লকে → Network `10.1.1.0`, Broadcast `10.1.1.3`, হোস্ট `.1` ও `.2`, মোট 2টা।
3. `/27` → block 32। 200 পড়ছে 192-এর ব্লকে → Network `192.168.100.192`, Broadcast `192.168.100.223`, হোস্ট `.193` – `.222`, মোট 30টা।

ল্যাব ৩: `/etc/hosts` দিয়ে নাম ঠিক করে দেওয়া

```
$ getent hosts mytest.local            # কিছুই আসবে না
$ echo "127.0.0.1  mytest.local" | sudo tee -a /etc/hosts
$ getent hosts mytest.local
127.0.0.1       mytest.local
$ ping -c 2 mytest.local               # 127.0.0.1 থেকে উত্তর আসবে
$ dig +short mytest.local              # কিছু আসবে না! কারণ dig /etc/hosts দেখে না
```

শেষে লাইনটা মুছে ফেলো: `sudo sed -i '/mytest.local/d' /etc/hosts` (অধ্যায় ২১)।

ল্যাব ৪: DNS cache আর systemd-resolved (Ubuntu/Fedora)

```
$ ls -l /etc/resolv.conf               # কে নিয়ন্ত্রণ করে?
$ resolvectl query debian.org          # প্রথমবার: "acquired via protocol DNS in XXms"
$ resolvectl query debian.org          # দ্বিতীয়বার: "Data from: cache"
$ resolvectl statistics                # Cache hits বাড়ল
$ sudo resolvectl flush-caches
$ resolvectl statistics                # Current Cache Size: 0
```

ল্যাব ৫: nmcli দিয়ে স্ট্যাটিক IP, তারপর আবার DHCP

```
$ nmcli connection show --active       # profile-এর নাম জেনে নাও; ধরা যাক "Wired
connection 1"
```

```
$ CON="Wired connection 1"
$ sudo nmcli connection modify "$CON" ipv4.method manual \
      ipv4.addresses 192.168.1.250/24 ipv4.gateway 192.168.1.1 ipv4.dns 1.1.1.1
$ sudo nmcli connection up "$CON"
$ ip -br addr show                      # নতুন IP দেখো (নিজের নেটওয়ার্ক অনুযায়ী ঠিকানা বদলে
নিও)
$ ping -c 2 1.1.1.1 && ping -c 2 debian.org

# আগের অবস্থায় ফেরত:
$ sudo nmcli connection modify "$CON" ipv4.method auto ipv4.addresses "" ipv4.gateway
"" ipv4.dns ""
$ sudo nmcli connection up "$CON"
```

ল্যাব ৬: nmtui ঘুরে দেখো

`sudo nmtui` চালাও। শুধু Tab, তীর-চিহ্ন, Space, Enter আর Esc দিয়ে একটা profile খোলো, IPv4 অংশে `<Show>` দেখো, তারপর কোনো পরিবর্তন না করে `<Cancel>` দিয়ে বের হয়ে এসো। "Set system hostname" মেনুটাও দেখে নাও।

ল্যাব ৭ (Ubuntu সার্ভার VM): netplan try

```
$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml   # ফাইলের নাম তোমার
সিস্টেমে আলাদা হতে পারে
$ sudo netplan get
$ sudo netplan try --timeout 30
# Enter চাপবে না → ৩০ সেকেন্ড পর নিজে থেকেই আগের অবস্থায় ফেরে, সেটা লক্ষ করো
```

তারপর ইচ্ছা করে YAML-এ indentation ভুল করো আর `sudo netplan generate` চালাও। কেমন error বার্তা আসে দেখো, তারপর ব্যাকআপ থেকে ফাইলটা ফিরিয়ে আনো।

ল্যাব ৮: port ও service চিনে নাও

```
$ grep -wE '22|53|80|443|3306' /etc/services
$ sudo ss -tulpn                        # কোন প্রোগ্রাম কোন port-এ LISTEN করছে (বিস্তারিত অধ্যায়
৩৭-এ)
$ cat /proc/sys/net/ipv4/ip_local_port_range
```

সারসংক্ষেপ

ধারণার সারাংশ:

বিষয়	এক লাইনে
OSI	৭ স্তর: Physical, Data Link, Network, Transport, Session, Presentation, Application
TCP/IP	৪ স্তর: Link, Internet, Transport, Application; TCP/IP stack kernel-এ চলে

বিষয়	এক লাইনে
IPv4	৩২ বিট, `a.b.c.d`, ঠিকানা ফুরিয়ে গেছে
IPv6	১২৮ বিট, hex, `::` সংক্ষেপ, ARP-র জায়গায় NDP
CIDR	`/n` = mask-এর 1 বিটের সংখ্যা; হোস্ট = 2^(32−n) − 2
Private IP	10/8, 172.16/12, 192.168/16
Gateway	নিজের subnet-এর বাইরের সব প্যাকেট যায় এখানে
MAC/ARP	লোকাল হার্ডওয়্যার ঠিকানা / IP থেকে MAC বের করা
DNS	নাম থেকে IP; root → TLD → authoritative; TTL অনুযায়ী cache
DHCP	DORA; UDP 67/68; lease
TCP / UDP	নির্ভরযোগ্য ও সংযোগভিত্তিক / দ্রুত ও সংযোগহীন
NAT	private থেকে public ঠিকানায় অনুবাদ; SNAT, masquerade, DNAT

কমান্ড ও ফাইলের দ্রুত রেফারেন্স:

কমান্ড / ফাইল	কাজ
`ip -br addr`, `ip addr show IFACE`	IP দেখা
`ip route`, `ip route get IP`	routing table, পথ যাচাই
`ip neigh`	ARP/neighbor cache
`ipcalc NET/PREFIX`	subnet হিসাব
`getent hosts NAME`	প্রোগ্রামের মতো করে নাম resolve
`/etc/hosts`	লোকাল নাম থেকে IP
`/etc/nsswitch.conf`	resolve করার ক্রম
`/etc/resolv.conf`	DNS সার্ভার (হাতে এডিট নয়)
`/etc/services`	port থেকে service-এর নাম
`resolvectl status / query / flush-caches`	systemd-resolved
`nmcli general status`	NetworkManager-এর অবস্থা
`nmcli device status / show`	device-এর তালিকা ও বিস্তারিত
`nmcli connection show / up / down`	profile দেখা, চালু, বন্ধ
`nmcli connection modify NAME ipv4.method manual ipv4.addresses ...`	স্ট্যাটিক IP
`nmcli device wifi list / connect SSID --ask`	Wi-Fi
`nmtui`	মেনু-ভিত্তিক কনফিগ
`/etc/netplan/*.yaml`	Ubuntu-র কনফিগ

কমান্ড / ফাইল	কাজ
`netplan try / apply / get / status`	Netplan প্রয়োগ ও পরীক্ষা
`networkctl`	systemd-networkd-এর অবস্থা
`/etc/network/interfaces`, `ifup` / `ifdown`	Debian ifupdown
`/etc/NetworkManager/system-connections/`	NM profile (RHEL 9/Fedora)
`hostnamectl set-hostname NAME`	hostname বদলানো
`sysctl net.ipv4.ip_forward`	IP forwarding-এর অবস্থা

শর্টকাট:

প্রসঙ্গ	কী	কাজ
`nmtui`	`Tab` / `Shift+Tab`	পরের / আগের ঘর
`nmtui`	`↑` `↓` `←` `→`	চলাফেরা
`nmtui`	`Space`	চেকবক্স টগল
`nmtui`	`Enter`	নির্বাচন / বোতাম
`nmtui`	`Esc`	পেছনে / বাতিল
`nmcli`	`Tab Tab`	object, command, property-র নাম অটো-কমপ্লিট
`nmcli monitor`	`Ctrl+C`	মনিটরিং বন্ধ
`netplan try`	`Enter`	নতুন কনফিগ গ্রহণ (না চাপলে টাইমআউটের পর আগের অবস্থায় ফেরে)
`netplan try`	`Ctrl+C`	সঙ্গে সঙ্গে আগের অবস্থায় ফেরত

নিজেকে যাচাই করো

1. OSI মডেলের কোন স্তরে TCP আর কোন স্তরে IP কাজ করে? কোন স্তরের ডেটাকে "frame" বলে?
2. `192.168.10.77/28`-এর network address, broadcast address আর ব্যবহারযোগ্য হোস্টের সংখ্যা কত?
3. নিচের কোনগুলো private IP? `172.20.5.1`, `172.32.0.1`, `10.255.0.9`, `192.169.1.1`, `100.64.3.3`
4. একটা প্যাকেট এক রাউটার পার হলে IP অ্যাড্রেস আর MAC অ্যাড্রেসের মধ্যে কোনটা বদলায় (NAT না থাকলে)?
5. কোনো ল্যাপটপের IP `169.254.12.7` দেখাচ্ছে। এর মানে কী?
6. `dig` দিয়ে নাম পাওয়া যাচ্ছে না, কিন্তু `ping` সেই নামেই চলছে। এর সম্ভাব্য কারণ কী?
7. Ubuntu-তে `/etc/resolv.conf`-এ `nameserver 127.0.0.53` লেখা। আসল DNS সার্ভার কোনটা, সেটা কোন কমান্ডে দেখবে? আর এই ফাইল হাতে বদলানো ঠিক নয় কেন?
8. DHCP-র চারটা ধাপের নাম লেখো। DHCP কোন protocol আর কোন port ব্যবহার করে?

9. "Connection refused" আর "Connection timed out"-এর মধ্যে পার্থক্য কী?
10. রিমোট Ubuntu সার্ভারে Netplan-এর কনফিগ বদলানোর সময় `netplan apply`-এর বদলে কোন কমান্ড ব্যবহার করা উচিত, আর কেন?
11. `nmcli connection modify` দিয়ে IP বদলানোর পরেও interface-এ পুরনো IP রয়ে গেছে কেন?
12. `wlp3s0` নামের প্রতিটা অংশের মানে কী?

▼ **উত্তর দেখো**

1. TCP চলে Layer 4 (Transport)-এ, IP চলে Layer 3 (Network)-এ। Layer 2 (Data Link)-এর ডেটাকে frame বলে।
2. `/28` মানে block size 16। 77 পড়ছে 64-এর ব্লকে (64–79)। Network `192.168.10.64`, Broadcast `192.168.10.79`, হোস্ট `.65` – `.78`, মোট `2^4 − 2 = 14` টা।
3. `172.20.5.1` (172.16–172.31 রেঞ্জের মধ্যে) আর `10.255.0.9` private। `172.32.0.1` আর `192.169.1.1` public। `100.64.3.3` হলো CGNAT shared address space: RFC 1918 private নয়, তবে ইন্টারনেটেও রাউট হয় না (ISP-র ভেতরে ব্যবহার হয়)।
4. MAC বদলায় (প্রতিটা হপে নতুন Ethernet হেডার)। IP একই থাকে।
5. এটা link-local (APIPA) ঠিকানা, মানে DHCP সার্ভার থেকে কোনো উত্তর আসেনি। তার/Wi-Fi সংযোগ বা রাউটারের DHCP পরীক্ষা করো।
6. নামটা সম্ভবত `/etc/hosts`-এ আছে (বা mDNS-এর `.local` নাম)। `ping` glibc/NSS-এর মাধ্যমে `/etc/hosts` দেখে, কিন্তু `dig` সরাসরি DNS সার্ভারকে জিজ্ঞেস করে, তাই `/etc/hosts` উপেক্ষা করে।
7. `resolvectl status` (বা `resolvectl dns`)। `127.0.0.53` হলো systemd-resolved-এর লোকাল stub। ফাইলটা একটা symlink, যেটা systemd-resolved/NetworkManager স্বয়ংক্রিয়ভাবে লেখে, তাই হাতে বদলালে পরের রিকানেক্ট বা রিবুটে পরিবর্তন মুছে যাবে।
8. Discover, Offer, Request, Acknowledge (DORA)। protocol UDP: সার্ভার port 67, ক্লায়েন্ট port 68।
9. Refused: প্যাকেট গন্তব্যে পৌঁছেছে কিন্তু ওই port-এ কেউ listen করছে না, তাই সার্ভার RST পাঠিয়েছে। Timed out: কোনো উত্তরই আসেনি। সাধারণত ফায়ারওয়াল প্যাকেট DROP করছে, অথবা হোস্ট/রুটে সমস্যা।
10. `sudo netplan try`। এটা কনফিগ প্রয়োগ করে নিশ্চিতকরণের জন্য অপেক্ষা করে (ডিফল্ট ১২০ সেকেন্ড)। ভুলের কারণে SSH কেটে গেলে তুমি Enter চাপতে পারবে না, তাই কনফিগ নিজে থেকেই আগের অবস্থায় ফেরত যাবে আর তুমি সার্ভার থেকে আটকে পড়বে না।
11. `modify` শুধু সংরক্ষিত profile বদলায়। চালু থাকা interface-এ প্রয়োগ করতে `nmcli connection up NAME` (বা `nmcli device reapply IFACE`) চালাতে হয়। আরেকটা সম্ভাবনা হলো `ipv4.method` তখনো `auto` রয়ে গেছে, ফলে DHCP-র ঠিকানাও থাকছে।
12. `wl` = Wireless LAN (Wi-Fi), `p3` = PCI bus 3, `s0` = slot 0। নামটা হার্ডওয়্যারের অবস্থান থেকে তৈরি, তাই প্রতি বুটে একই থাকে (predictable naming)।

অধ্যায় ৩৭: নেটওয়ার্ক কমান্ড ও ট্রাবলশুটিং

> আগের অধ্যায়ে (অধ্যায় ৩৬) আমরা শিখেছি নেটওয়ার্ক *কীভাবে কাজ করে*: IP address, subnet, gateway, DNS, port, TCP/UDP, OSI/TCP-IP মডেল। এই অধ্যায়ে সেই জ্ঞানকে হাতে নেব। এবার টার্মিনালে বসে নেটওয়ার্ক **দেখব, মাপব, পরীক্ষা করব আর ঠিক করব**।

এই অধ্যায়ে যা শিখবে

- `ip` কমান্ড দিয়ে interface, IP address, route আর ARP/neighbor টেবিল দেখা ও বদলানো (`ip addr`, `ip link`, `ip route`, `ip neigh`)
- পুরনো `ifconfig`, `route`, `netstat`, `arp` কোন নতুন কমান্ডে বদলে গেছে, তার পুরো তুলনা টেবিল
- `hostname` আর `hostnamectl` দিয়ে কম্পিউটারের নাম দেখা ও বদলানো
- `ping` দিয়ে কোনো হোস্ট বেঁচে আছে কি না যাচাই করা, আর ভেতরে ICMP কীভাবে কাজ করে
- `traceroute`, `tracepath` আর `mtr` দিয়ে প্যাকেটের পুরো রাস্তা দেখা, আর TTL-এর কৌশলটা বোঝা
- `ss -tulpn` আর `lsof -i` দিয়ে কোন প্রোগ্রাম কোন port খুলে রেখেছে তা বের করা
- `dig`, `nslookup`, `host` দিয়ে DNS প্রশ্ন করা
- `curl` (`-I -L -o -O -X -H -d -v -w` সহ) আর `wget` দিয়ে HTTP টেস্ট করা আর ফাইল নামানো
- `nc` (netcat) দিয়ে port টেস্ট, ছোট চ্যাট সার্ভার আর ফাইল পাঠানো
- `tcpdump` দিয়ে নেটওয়ার্কের প্যাকেট ধরা ও পড়া
- `nmap` পরিচিতি (শুধু নিজের সিস্টেমে, আইনি সীমার মধ্যে)
- নেটওয়ার্ক সমস্যা ধাপে ধাপে সমাধানের একটা পদ্ধতি, যা পেশাদার সিস্টেম অ্যাডমিনরা ব্যবহার করেন

৩৭.১ শুরুর আগে: টুলবক্স সাজিয়ে নাও

সব টুল সব distro-তে আগে থেকে ইনস্টল থাকে না। নিচের টেবিল থেকে দেখে নাও কোন প্যাকেজে কোন কমান্ড আছে:

কমান্ড	Ubuntu/Debian প্যাকেজ	Fedora/RHEL প্যাকেজ
`ip`, `ss`	`iproute2` (সাধারণত আগে থেকেই থাকে)	`iproute` (আগে থেকেই থাকে)
`ifconfig`, `route`, `netstat`, `arp`	`net-tools`	`net-tools`

কমান্ড	Ubuntu/Debian প্যাকেজ	Fedora/RHEL প্যাকেজ
`ping`, `tracepath`	`iputils-ping`, `iputils-tracepath`	`iputils`
`traceroute`	`traceroute`	`traceroute`
`mtr`	`mtr` (GUI-সহ) বা `mtr-tiny` (শুধু টার্মিনাল)	`mtr`
`dig`, `nslookup`, `host`	`dnsutils` বা `bind9-dnsutils`	`bind-utils`
`curl`	`curl`	`curl`
`wget`	`wget`	`wget` (Fedora 40 থেকে আসলে `wget2`)
`nc`	`netcat-openbsd`	`nmap-ncat` (এটা `ncat`, `nc` নামেও চলে)
`tcpdump`	`tcpdump`	`tcpdump`
`nmap`	`nmap`	`nmap`
`lsof`	`lsof`	`lsof`

একবারে সব ইনস্টল করে নাও:

```
# Ubuntu / Debian
$ sudo apt update
$ sudo apt install iproute2 net-tools iputils-ping iputils-tracepath traceroute mtr-
tiny \
      dnsutils curl wget netcat-openbsd tcpdump nmap lsof

# Fedora / RHEL 9
$ sudo dnf install iproute net-tools iputils traceroute mtr bind-utils curl wget \
      nmap-ncat tcpdump nmap lsof
```

> 📝 **মনে রাখো:** প্যাকেজ ম্যানেজার নিয়ে বিস্তারিত আছে অধ্যায় ৩০-এ।

কোন টুল কোন লেয়ারে কাজ করে

নেটওয়ার্ক সমস্যা ধরতে হলে আগে জানতে হবে কোন টুল কোন "তলায়" কাজ করে। একটা বহুতল ভবন ভাবো: নিচতলায় তার আর সিগন্যাল, ওপরতলায় ওয়েবসাইট। যে তলায় সমস্যা, সেই তলার যন্ত্র দিয়ে মাপতে হয়।

```
┌──────────────────────────────────────────────────────────┐
│ Application (HTTP, DNS, SSH...)                          │
│   curl, wget, dig, nslookup, host, nc (টেক্সট পাঠানো)          │
├──────────────────────────────────────────────────────────┤
│ Transport (TCP / UDP, port)                              │
│   ss, netstat, lsof -i, nc -zv, nmap                     │
├──────────────────────────────────────────────────────────┤
```

```
| Network / Internet (IP, ICMP, routing)                          |
|   ip addr, ip route, ping, traceroute, tracepath, mtr           |
├─────────────────────────────────────────────────────────────────┤
| Link (Ethernet, Wi-Fi, MAC, ARP)                                |
|   ip link, ip neigh, ethtool, iw                                |
└─────────────────────────────────────────────────────────────────┘
```

tcpdump সব লেয়ারের প্যাকেট একসাথে দেখতে পারে

৩৭.২ `ip` — আধুনিক লিনাক্সের নেটওয়ার্ক "সুইস আর্মি নাইফ"

`ip` কেন এল? ভেতরে কীভাবে কাজ করে?

আগে লিনাক্সে নেটওয়ার্ক দেখতে `ifconfig`, `route`, `arp` আলাদা আলাদা কমান্ড ছিল (`net-tools` প্যাকেজ)। এগুলো কার্নেলের সাথে পুরনো `ioctl()` ইন্টারফেস দিয়ে কথা বলত। এগুলোর ২০০১ সালের পর থেকে প্রায় কোনো উন্নতি হয়নি, তাই অনেক কিছু দেখাতে পারে না। যেমন একটা interface-এ একাধিক IP থাকলে `ifconfig` ঠিকমতো দেখায় না, আর policy routing বা network namespace-এর মতো নতুন ফিচার সে চেনেই না।

`ip` কমান্ড (প্যাকেজ `iproute2`) কার্নেলের সাথে কথা বলে **Netlink socket** দিয়ে। Netlink হলো user space আর kernel-এর মধ্যে নেটওয়ার্ক তথ্য আদান-প্রদানের আধুনিক চ্যানেল।

```
┌──────────────┐                          ┌──────────────────────────────┐
│  ip / ss     │ ── Netlink socket ──►    │  Linux Kernel                │
│ (iproute2)   │ ◄── উত্তর (মেসেজ) ──    │  - interface টেবিল           │
└──────────────┘                          │  - IP address টেবিল          │
                                          │  - routing টেবিল (FIB)       │
┌──────────────┐                          │  - neighbor (ARP) টেবিল      │
│ ifconfig/route│ ── পুরনো ioctl() ──►    │  - socket টেবিল              │
│ (net-tools)  │                          └──────────────────────────────┘
└──────────────┘
```

> 💡 **উপমা:** কার্নেল হলো একটা ব্যাংক। `ifconfig` পুরনো কাউন্টারে লাইনে দাঁড়িয়ে কাগজে ফর্ম জমা দেয়। `ip` মোবাইল অ্যাপ দিয়ে একই ব্যাংকের সব হিসাব একসাথে, বিস্তারিতভাবে দেখে।

`ip` কমান্ডের গঠন

```
ip [OPTIONS] OBJECT COMMAND [ARGUMENTS]
```

OBJECT	সংক্ষেপ	কী দেখায়/বদলায়
`address`	`addr`, `a`	interface-এর IP address
`link`	`l`	interface নিজে (নেটওয়ার্ক কার্ড): up/down, MAC, MTU
`route`	`r`	routing টেবিল: কোন প্যাকেট কোন পথে যাবে

OBJECT	সংক্ষেপ	কী দেখায়/বদলায়
`neighbour`	`neigh`, `n`	ARP/NDP টেবিল: IP থেকে MAC-এর ম্যাপ
`netns`		network namespace (কন্টেইনারে কাজে লাগে, অধ্যায় ৪৮)
`maddress`	`maddr`	multicast address

`ip`-এর গুরুত্বপূর্ণ সাধারণ অপশন:

অপশন	কাজ
`-br` / `-brief`	এক লাইনে সংক্ষেপে দেখায় (সবচেয়ে কাজের)
`-c` / `-color`	রঙিন আউটপুট
`-4` / `-6`	শুধু IPv4 / শুধু IPv6
`-s` / `-stats`	পরিসংখ্যান (কত প্যাকেট গেল-এল, error)
`-d` / `-details`	আরও বিস্তারিত
`-j` / `-json`	JSON আউটপুট (স্ক্রিপ্টে কাজে লাগে)
`-p` / `-pretty`	JSON সুন্দর করে সাজানো

> 💡 **টিপস:** `ip`-এর সব কমান্ড সংক্ষেপে লেখা যায়। `ip address show` = `ip addr show` = `ip a`। যেকোনো জায়গায় `help` লিখলে সাহায্য পাবে: `ip addr help`, `ip route help`।

৩৭.২.১ `ip addr` — IP address দেখা ও বসানো

```
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 3c:7c:3f:1a:2b:4c brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.105/24 brd 192.168.1.255 scope global dynamic noprefixroute enp3s0
       valid_lft 86142sec preferred_lft 86142sec
    inet6 fe80::3e7c:3fff:fe1a:2b4c/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: wlp2s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group
default qlen 1000
    link/ether a4:c3:f0:11:22:33 brd ff:ff:ff:ff:ff:ff
```

আউটপুট লাইন ধরে ধরে বোঝো:

অংশ	মানে
`2: enp3s0`	interface নম্বর ২, নাম `enp3s0`
`<...UP,LOWER_UP>`	`UP` = সফটওয়্যারে চালু; `LOWER_UP` = তার লাগানো/সিগন্যাল আছে (physical link)
`NO-CARRIER`	তার নেই বা Wi-Fi যুক্ত নয়
`mtu 1500`	এক প্যাকেটে সর্বোচ্চ কত byte যাবে
`state UP/DOWN`	interface-এর অবস্থা
`link/ether 3c:7c:...`	MAC address (হার্ডওয়্যার ঠিকানা)
`inet 192.168.1.105/24`	IPv4 address আর subnet (`/24` = `255.255.255.0`)
`brd 192.168.1.255`	broadcast address
`scope global`	বাইরের নেটওয়ার্কে ব্যবহারযোগ্য; `scope link` = শুধু লোকাল লিংকে; `scope host` = শুধু এই কম্পিউটারে
`dynamic`	DHCP থেকে পাওয়া IP
`valid_lft 86142sec`	DHCP lease আর কত সেকেন্ড বৈধ
`inet6 fe80::...`	IPv6 link-local address

interface-এর নাম এমন অদ্ভুত কেন? (`enp3s0` , `wlp2s0`)

আগে নাম ছিল `eth0` , `wlan0` । কিন্তু একাধিক কার্ড থাকলে বুটের সময় কোনটা আগে চিনবে তার ওপর নাম বদলে যেত। এখন systemd/udev **Predictable Network Interface Names** ব্যবহার করে। নামটা হার্ডওয়্যারের অবস্থান থেকে তৈরি হয়:

নাম	মানে
`en`	Ethernet
`wl`	Wireless LAN (Wi-Fi)
`ww`	WWAN (মোবাইল ব্রডব্যান্ড)
`p3s0`	PCI bus 3, slot 0
`o1` (`eno1`)	মাদারবোর্ডে বসানো (onboard) ডিভাইস নম্বর ১
`x3c7c3f...` (`enx...`)	MAC address থেকে নাম (সাধারণত USB adapter)

ক্লাউড VM-এ প্রায়ই `eth0` বা `ens5` দেখবে।

সংক্ষিপ্ত আর রঙিন ভিউ

প্রতিদিনের কাজে এটাই সবচেয়ে বেশি লাগবে:

```
$ ip -br -c addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp3s0           UP             192.168.1.105/24 fe80::3e7c:3fff:fe1a:2b4c/64
wlp2s0           DOWN
```

শুধু একটা interface বা শুধু IPv4:

```
$ ip addr show dev enp3s0
$ ip -4 addr
$ ip -4 -br addr show enp3s0
enp3s0           UP             192.168.1.105/24
```

IP যোগ করা ও মোছা (অস্থায়ী)

```
# একটা অতিরিক্ত IP বসাও (একটা interface-এ একাধিক IP থাকতে পারে)
$ sudo ip addr add 192.168.1.200/24 dev enp3s0

# মুছে ফেলো
$ sudo ip addr del 192.168.1.200/24 dev enp3s0

# interface-এর সব IP মুছে ফেলো (সাবধান!)
$ sudo ip addr flush dev enp3s0
```

> ⚠️ **সতর্কতা:** `ip` দিয়ে করা সব পরিবর্তন **অস্থায়ী**। রিবুট করলে বা NetworkManager নতুন DHCP lease নিলে সব মুছে যায়। স্থায়ী কনফিগারেশন করতে হয় distro-র টুল দিয়ে: Ubuntu-তে **Netplan** (`/etc/netplan/*.yaml`), Fedora/RHEL-এ **NetworkManager** (`nmcli`), Debian সার্ভারে `/etc/network/interfaces`। এগুলো অধ্যায় ৩৬-এ আছে।

> ⚠️ **সতর্কতা:** রিমোট সার্ভারে SSH দিয়ে ঢুকে যে interface দিয়ে ঢুকেছ সেটার IP মুছলে বা interface down করলে সংযোগ সঙ্গে সঙ্গে কেটে যাবে, আর ফিরে ঢুকতে পারবে না। তখন cloud console বা সার্ভারের সামনে গিয়ে ঠিক করতে হবে।

৩৭.২.২ `ip link` — নেটওয়ার্ক কার্ড নিজেকে নিয়ন্ত্রণ

`ip addr` দেখায় ঠিকানা (লেয়ার ৩), আর `ip link` দেখায় কার্ডের অবস্থা (লেয়ার ২)।

```
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode
DEFAULT group default qlen 1000
    link/ether 3c:7c:3f:1a:2b:4c brd ff:ff:ff:ff:ff:ff

$ ip -br link
```

```
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
enp3s0           UP             3c:7c:3f:1a:2b:4c <BROADCAST,MULTICAST,UP,LOWER_UP>
wlp2s0           DOWN           a4:c3:f0:11:22:33 <NO-CARRIER,BROADCAST,MULTICAST,UP>
```

গুরুত্বপূর্ণ কাজগুলো:

```
# interface বন্ধ ও চালু
$ sudo ip link set dev enp3s0 down
$ sudo ip link set dev enp3s0 up

# MTU বদলানো
$ sudo ip link set dev enp3s0 mtu 9000

# MAC address বদলানো (আগে down করতে হয়)
$ sudo ip link set dev enp3s0 down
$ sudo ip link set dev enp3s0 address 02:11:22:33:44:55
$ sudo ip link set dev enp3s0 up

# promiscuous mode (সব প্যাকেট গ্রহণ, tcpdump নিজেই চালু করে)
$ sudo ip link set dev enp3s0 promisc on
```

পরিসংখ্যান: প্যাকেট হারাচ্ছে কি না

```
$ ip -s link show dev enp3s0
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode
DEFAULT group default qlen 1000
    link/ether 3c:7c:3f:1a:2b:4c brd ff:ff:ff:ff:ff:ff
    RX:  bytes  packets errors dropped  missed   mcast
     987654321   812345      0      12       0    1543
    TX:  bytes  packets errors dropped carrier collsns
     123456789   456789      0       0       0       0
```

`RX` = গ্রহণ (receive), `TX` = পাঠানো (transmit)। `errors` বা `dropped` সংখ্যা যদি দ্রুত বাড়তে থাকে তাহলে তার, পোর্ট, ড্রাইভার বা ডুপ্লেক্স সেটিংয়ে সমস্যা আছে। `-s -s` (দুবার) দিলে আরও বিস্তারিত error ভাগ দেখায়।

> 💡 **টিপস:** তারের স্পিড আর ডুপ্লেক্স দেখতে `sudo ethtool enp3s0` চালাও (`Speed: 1000Mb/s`, `Duplex: Full`, `Link detected: yes`)। Wi-Fi-র সিগন্যাল দেখতে `iw dev wlp2s0 link`।

ভার্চুয়াল interface (পরিচিতি)

`ip link` দিয়ে সফটওয়্যারের তৈরি interface-ও বানানো যায়। Docker, VPN আর VM এগুলো নিজে থেকেই বানায়:

```
$ sudo ip link add br0 type bridge        # একটা ভার্চুয়াল সুইচ
$ sudo ip link add veth0 type veth peer name veth1   # দুই মাথার ভার্চুয়াল তার
$ sudo ip link delete br0
```

তুমি হয়তো `docker0`, `virbr0`, `tun0`, `wg0`, `vethXXXX` নামের interface দেখবে। এগুলো যথাক্রমে Docker bridge, libvirt bridge, OpenVPN, WireGuard আর কন্টেইনারের veth। বিস্তারিত অধ্যায় ৪৮-এ।

৩৭.২.৩ `ip route` — প্যাকেট কোন রাস্তায় যাবে

ভেতরে কীভাবে কাজ করে

প্রতিটা প্যাকেট পাঠানোর আগে কার্নেল **routing table** দেখে। নিয়মটা হলো **longest prefix match**: যে route গন্তব্যের সাথে সবচেয়ে নির্দিষ্টভাবে মেলে (সবচেয়ে বড় `/` সংখ্যা), সেটাই জেতে। কোনোটা না মিললে `default` route ব্যবহার হয়।

> 💡 **উপমা:** পোস্ট অফিসের কর্মী চিঠির ঠিকানা দেখে। "ঢাকা, মিরপুর-১০" এর জন্য আলাদা বাক্স থাকলে সেখানে রাখে, না থাকলে "ঢাকা" বাক্সে। কিছুই না মিললে "অন্যান্য, কেন্দ্রীয় অফিসে পাঠাও" বাক্সে রাখে। এই শেষ বাক্সটাই `default` route, আর কেন্দ্রীয় অফিস হলো gateway (রাউটার)।

```
$ ip route show
default via 192.168.1.1 dev enp3s0 proto dhcp src 192.168.1.105 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.1.0/24 dev enp3s0 proto kernel scope link src 192.168.1.105 metric 100
```

অংশ	মানে
`default via 192.168.1.1`	অচেনা সব গন্তব্যের জন্য রাউটার `192.168.1.1`-এ পাঠাও
`dev enp3s0`	এই interface দিয়ে বের হবে
`proto dhcp`	route-টা DHCP থেকে এসেছে; `proto kernel` = IP বসানোর সময় কার্নেল নিজে বানিয়েছে; `proto static` = হাতে বসানো
`scope link`	গন্তব্য সরাসরি এই লিংকে আছে, gateway লাগবে না
`src 192.168.1.105`	এই route দিয়ে যাওয়ার সময় কোন source IP বসবে
`metric 100`	একই গন্তব্যের একাধিক route থাকলে কম metric জেতে (যেমন তার আর Wi-Fi দুটোই থাকলে)
`linkdown`	interface-এর লিংক নেই

একটা নির্দিষ্ট গন্তব্যে কোন পথ নেবে?

এটা ট্রাবলশুটিংয়ের সোনার কমান্ড। কার্নেলকে সরাসরি জিজ্ঞেস করো:

```
$ ip route get 8.8.8.8
8.8.8.8 via 192.168.1.1 dev enp3s0 src 192.168.1.105 uid 1000
    cache

$ ip route get 192.168.1.50
```

```
192.168.1.50 dev enp3s0 src 192.168.1.105 uid 1000
    cache
```

প্রথমটায় `via` আছে, মানে রাউটার হয়ে যাবে। দ্বিতীয়টায় `via` নেই, মানে সরাসরি একই LAN-এ।

route যোগ/মোছা (অস্থায়ী)

```
# default gateway বসানো
$ sudo ip route add default via 192.168.1.1 dev enp3s0

# একটা নির্দিষ্ট নেটওয়ার্কের জন্য আলাদা রাউটার
$ sudo ip route add 10.10.0.0/16 via 192.168.1.254

# বদলানো (না থাকলে যোগ করে)
$ sudo ip route replace default via 192.168.1.2

# মোছা
$ sudo ip route del 10.10.0.0/16
```

> 📝 **মনে রাখো:** লিনাক্সে একাধিক routing table থাকতে পারে (`ip route show table all`) আর `ip rule` দিয়ে "কোন প্যাকেট কোন টেবিল দেখবে" ঠিক করা যায়। একে **policy routing** বলে। VPN আর multi-ISP সার্ভারে এটা কাজে লাগে। সাধারণ কাজে শুধু `main` টেবিলই দরকার।

৩৭.২.৪ `ip neigh` — ARP/Neighbor টেবিল

ভেতরে কীভাবে কাজ করে

একই LAN-এ প্যাকেট আসলে পৌঁছায় **MAC address** দিয়ে, IP দিয়ে নয়। তাই `192.168.1.1`-এ পাঠাতে হলে কম্পিউটারকে আগে জানতে হয় ওই IP-র MAC কত। এর জন্য সে পুরো LAN-এ চিৎকার করে জিজ্ঞেস করে (ARP broadcast): *"192.168.1.1 কার? জানাও!"* রাউটার উত্তর দেয়: *"আমার, আমার MAC 08:00:27:aa:bb:cc"*। কার্নেল উত্তরটা কিছুক্ষণ মনে রাখে। এই মনে রাখার খাতাটাই neighbor table। IPv6-এ একই কাজ করে **NDP** (Neighbor Discovery Protocol)।

```
তোমার PC                                    রাউটার (192.168.1.1)
   │  ARP Request (broadcast ff:ff:ff:ff:ff:ff)  │
   │  "Who has 192.168.1.1? Tell 192.168.1.105"  │
   │ ──────────────────────────────────────────► │
   │                                             │
   │  ARP Reply (unicast)                        │
   │  "192.168.1.1 is at 08:00:27:aa:bb:cc"      │
   │ ◄────────────────────────────────────────── │
   │                                             │
   └─► neighbor table-এ লিখে রাখে
```

```
$ ip neigh show
192.168.1.1 dev enp3s0 lladdr 08:00:27:aa:bb:cc REACHABLE
192.168.1.20 dev enp3s0 lladdr 5c:e9:1e:44:55:66 STALE
```

```
192.168.1.77 dev enp3s0 FAILED
fe80::1 dev enp3s0 lladdr 08:00:27:aa:bb:cc router STALE
```

অবস্থা (state)	মানে
`REACHABLE`	সম্প্রতি নিশ্চিত হয়েছে যে বেঁচে আছে
`STALE`	MAC জানা, তবে কিছুক্ষণ যোগাযোগ হয়নি; পরের বার ব্যবহারের সময় আবার যাচাই হবে
`DELAY` / `PROBE`	যাচাই চলছে
`INCOMPLETE`	ARP প্রশ্ন পাঠানো হয়েছে, উত্তর আসেনি এখনো
`FAILED`	উত্তর আসেনি, ওই IP-তে কেউ নেই (বা সাড়া দিচ্ছে না)
`PERMANENT`	হাতে বসানো স্থায়ী এন্ট্রি

```
# পুরো টেবিল খালি করা (সমস্যা হলে নতুন করে ARP শিখবে)
$ sudo ip neigh flush dev enp3s0

# স্থায়ী এন্ট্রি বসানো
$ sudo ip neigh add 192.168.1.50 lladdr 00:11:22:33:44:55 dev enp3s0 nud permanent

# মোছা
$ sudo ip neigh del 192.168.1.50 dev enp3s0
```

> 💡 **টিপস:** gateway-কে ping করতে পারছ না? `ip neigh` দেখো। gateway-এর পাশে `FAILED` বা `INCOMPLETE` মানে লেয়ার ২-তেই সমস্যা: তার, সুইচ, Wi-Fi বা ভুল subnet। `REACHABLE` থাকলে লিংক ঠিক আছে, সমস্যা ওপরের কোথাও।

৩৭.২.৫ `ip monitor` — লাইভ পরিবর্তন দেখা

```
$ ip monitor
[LINK]2: enp3s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN
group default
[ROUTE]Deleted default via 192.168.1.1 dev enp3s0 proto dhcp metric 100
...
```

তার খুলে ফেললে বা Wi-Fi কেটে গেলে সঙ্গে সঙ্গে এখানে দেখা যায়। থামাতে `Ctrl+C`।

৩৭.৩ পুরনো বনাম নতুন: `ifconfig`, `route`, `netstat`, `arp`

পুরনো টিউটোরিয়াল, Stack Overflow উত্তর আর পুরনো সার্ভারে তুমি `net-tools`-এর কমান্ড প্রচুর দেখবে। এগুলো এখন **deprecated** (পরিত্যক্ত ঘোষিত)। Ubuntu, Fedora, RHEL ৯-এর নতুন ইনস্টলে সাধারণত এগুলো থাকে না। তবে চিনে রাখা জরুরি, কারণ অনেক পুরনো স্ক্রিপ্ট আর বইয়ে এগুলোই আছে।

```
$ ifconfig
enp3s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.105  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::3e7c:3fff:fe1a:2b4c  prefixlen 64  scopeid 0x20<link>
        ether 3c:7c:3f:1a:2b:4c  txqueuelen 1000  (Ethernet)
        RX packets 812345  bytes 987654321 (987.6 MB)
        TX packets 456789  bytes 123456789 (123.4 MB)

$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.1.1     0.0.0.0         UG    100    0        0 enp3s0
192.168.1.0     0.0.0.0         255.255.255.0   U     100    0        0 enp3s0
```

বদল টেবিল (মুখস্থ রাখার মতো)

পুরনো (`net-tools`)	নতুন (`iproute2`)	কাজ
`ifconfig`	`ip addr` / `ip -br a`	IP address দেখা
`ifconfig -a`	`ip link`	সব interface (down সহ)
`ifconfig eth0 up`	`ip link set eth0 up`	interface চালু
`ifconfig eth0 down`	`ip link set eth0 down`	interface বন্ধ
`ifconfig eth0 192.168.1.5 netmask 255.255.255.0`	`ip addr add 192.168.1.5/24 dev eth0`	IP বসানো
`ifconfig eth0 mtu 9000`	`ip link set eth0 mtu 9000`	MTU
`ifconfig eth0 hw ether 02:..`	`ip link set eth0 address 02:..`	MAC বদল
`route -n` / `netstat -rn`	`ip route`	routing table
`route add default gw 192.168.1.1`	`ip route add default via 192.168.1.1`	default gateway
`route del -net 10.0.0.0/8`	`ip route del 10.0.0.0/8`	route মোছা
`arp -n`	`ip neigh`	ARP টেবিল
`arp -d 192.168.1.5`	`ip neigh del 192.168.1.5 dev eth0`	ARP এন্ট্রি মোছা
`netstat -tulpn`	`ss -tulpn`	listening port
`netstat -an`	`ss -an`	সব socket
`netstat -s`	`ss -s` (সারাংশ) / `nstat`	পরিসংখ্যান
`netstat -i`	`ip -s link`	interface পরিসংখ্যান
`netstat -g`	`ip maddr`	multicast group
`iwconfig`	`iw`	Wi-Fi

পুরনো (net-tools)	নতুন (iproute2)	কাজ
`nameif`	`ip link set eth0 name lan0`	interface-এর নাম বদল

> 📝 **মনে রাখো:** `ifconfig` subnet দেখায় `netmask 255.255.255.0` আকারে, আর `ip` দেখায় CIDR আকারে `/24`। দুটো একই জিনিস। হিসাবটা অধ্যায় ৩৬-এ আছে।

৩৭.৪ `hostname` ও `hostnamectl` — কম্পিউটারের নাম

প্রতিটা কম্পিউটারের একটা নাম থাকে। prompt-এ `nishat@mylaptop:~$`-এর `mylaptop` অংশটা সেই নাম।

```
$ hostname
mylaptop

$ hostname -I          # সব IP (loopback বাদে), শুধু সংখ্যা
192.168.1.105 172.17.0.1

$ hostname -f          # FQDN (পূর্ণ ডোমেইন নাম), যদি কনফিগার করা থাকে
mylaptop.home.lan

$ hostname -s          # ছোট নাম (প্রথম ডট পর্যন্ত)
mylaptop
```

অপশন	কাজ
`-I` (বড় হাতের i)	সব interface-এর IP
`-i`	hostname resolve করে যে IP পাওয়া যায় (প্রায়ই `127.0.1.1`, তাই বিভ্রান্তিকর)
`-f`	Fully Qualified Domain Name
`-s`	short name
`-d`	শুধু domain অংশ

নাম স্থায়ীভাবে বদলানো

আধুনিক systemd-ভিত্তিক সব distro-তে:

```
$ hostnamectl
 Static hostname: mylaptop
       Icon name: computer-laptop
         Chassis: laptop
      Machine ID: 4f2a...
         Boot ID: 9c1e...
Operating System: Ubuntu 24.04.1 LTS
```

```
          Kernel: Linux 6.8.0-45-generic
    Architecture: x86-64

$ sudo hostnamectl set-hostname webserver01
$ hostname
webserver01
```

`hostnamectl set-hostname` নামটা `/etc/hostname` ফাইলে লেখে, তাই রিবুটের পরেও থাকে। (`sudo hostname newname` শুধু অস্থায়ীভাবে বদলায়।)

> ⚠️ **সতর্কতা:** নাম বদলানোর পর `/etc/hosts` ফাইলেও পুরনো নাম বদলে দাও (Debian/Ubuntu-তে সাধারণত `127.0.1.1 mylaptop` লাইন থাকে)। না দিলে `sudo` চালানোর সময় `unable to resolve host webserver01` সতর্কবার্তা আসবে।

```
$ cat /etc/hosts
127.0.0.1   localhost
127.0.1.1   webserver01
::1         localhost ip6-localhost ip6-loopback
```

> 📝 **মনে রাখো:** `/etc/hosts` হলো ছোট্ট লোকাল ফোনবুক। নাম থেকে IP খোঁজার সময় সাধারণত এটা DNS-এর **আগে** দেখা হয় (ক্রম ঠিক করে `/etc/nsswitch.conf`-এর `hosts:` লাইন)। টেস্টের জন্য এখানে লিখে দিলে কোনো ডোমেইনকে নিজের পছন্দের IP-তে পাঠানো যায়।

৩৭.৫ `ping` — "তুমি কি বেঁচে আছ?"

ভেতরে কীভাবে কাজ করে

`ping` **ICMP** (Internet Control Message Protocol) ব্যবহার করে। সে গন্তব্যে একটা **Echo Request** প্যাকেট পাঠায়, আর গন্তব্য (যদি বেঁচে থাকে আর firewall আটকে না রাখে) একটা **Echo Reply** ফেরত পাঠায়। যাওয়া-আসার সময়টাকে বলে **RTT** (Round Trip Time)।

> 💡 **উপমা:** সাবমেরিনের সোনার। শব্দ পাঠাও, প্রতিধ্বনি ফিরে এলে বুঝবে সামনে কিছু আছে, আর কত সময়ে ফিরল তা দিয়ে দূরত্ব আন্দাজ করবে।

```
তোমার PC                                    8.8.8.8
  | ICMP Echo Request (type 8) seq=1 ──────────► |
  | ◄────────── ICMP Echo Reply (type 0) seq=1  |
  |        RTT = পাঠানো থেকে উত্তর পাওয়া পর্যন্ত সময়   |
```

সিনট্যাক্স ও ব্যবহার

```
ping [OPTIONS] HOST
```

লিনাক্সে `ping` থামানো পর্যন্ত চলতেই থাকে (Windows-এর মতো ৪টায় থামে না)। থামাতে `Ctrl+C`।

```
$ ping google.com
PING google.com (142.250.193.110) 56(84) bytes of data.
64 bytes from del12s08-in-f14.1e100.net (142.250.193.110): icmp_seq=1 ttl=117 time=24.6
ms
64 bytes from del12s08-in-f14.1e100.net (142.250.193.110): icmp_seq=2 ttl=117 time=23.9
ms
64 bytes from del12s08-in-f14.1e100.net (142.250.193.110): icmp_seq=3 ttl=117 time=25.1
ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 23.912/24.533/25.104/0.487 ms
```

আউটপুট পড়তে শেখো:

অংশ	মানে
`(142.250.193.110)`	DNS থেকে পাওয়া IP। নাম থেকে IP পেলে বোঝা যায় DNS কাজ করছে
`56(84) bytes`	৫৬ byte ডেটা; IP header (২০) আর ICMP header (৮) যোগ করে ৮৪
`icmp_seq=1`	ক্রমিক নম্বর। মাঝে নম্বর বাদ পড়লে প্যাকেট হারিয়েছে
`ttl=117`	ফেরত প্যাকেটের অবশিষ্ট TTL
`time=24.6 ms`	RTT
`0% packet loss`	কোনো প্যাকেট হারায়নি
`mdev`	সময়ের ওঠানামা (jitter-এর মতো); কম হলে সংযোগ স্থির

গুরুত্বপূর্ণ অপশন

অপশন	কাজ	উদাহরণ
`-c N`	N টা প্যাকেট পাঠিয়ে থামবে	`ping -c 4 8.8.8.8`
`-i SEC`	দুই প্যাকেটের মাঝে বিরতি (০.২ সেকেন্ডের কম দিতে root লাগে)	`ping -i 0.5 host`
`-W SEC`	প্রতিটা উত্তরের জন্য কতক্ষণ অপেক্ষা	`ping -c1 -W2 host`
`-w SEC`	পুরো ping মোট কত সেকেন্ড চলবে (deadline)	`ping -w 10 host`
`-s BYTES`	প্যাকেটের ডেটা সাইজ	`ping -s 1472 host`
`-t TTL`	পাঠানো প্যাকেটের TTL	`ping -t 5 host`

অপশন	কাজ	উদাহরণ
`-4` / `-6`	জোর করে IPv4 / IPv6	`ping -6 google.com`
`-q`	শুধু শেষের সারাংশ	`ping -c 10 -q host`
`-n`	IP-কে নামে উল্টো রূপান্তর (reverse DNS) করবে না	`ping -n 8.8.8.8`
`-I IFACE`	নির্দিষ্ট interface দিয়ে পাঠাও	`ping -I wlp2s0 8.8.8.8`
`-M do`	fragment করতে দেবে না (MTU পরীক্ষা)	`ping -M do -s 1472 host`
`-D`	প্রতিটা লাইনের আগে timestamp	`ping -D host`
`-a`	উত্তর এলে বিপ শব্দ	`ping -a host`
`-f`	flood ping (root লাগে, শুধু নিজের নেটওয়ার্কে টেস্টে)	`sudo ping -f -c 1000 192.168.1.1`

সাধারণ ফলাফল ও তার মানে

```
$ ping -c 2 192.168.1.77
PING 192.168.1.77 (192.168.1.77) 56(84) bytes of data.
From 192.168.1.105 icmp_seq=1 Destination Host Unreachable
From 192.168.1.105 icmp_seq=2 Destination Host Unreachable

--- 192.168.1.77 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1024ms
```

ফলাফল	সম্ভাব্য কারণ
`Destination Host Unreachable` (নিজের IP থেকে)	একই LAN-এ ওই IP-র কাছ থেকে ARP উত্তর আসেনি; হোস্ট বন্ধ বা নেই
কিছুই আসে না, শুধু শেষে `100% packet loss`	firewall ICMP আটকাচ্ছে, বা পথে কোথাও প্যাকেট হারাচ্ছে
`Network is unreachable`	কোনো route নেই (default gateway নেই?) → `ip route` দেখো
`ping: google.com: Temporary failure in name resolution`	DNS কাজ করছে না → IP দিয়ে ping করে দেখো
`ping: google.com: Name or service not known`	নামটাই পাওয়া যায়নি (বানান ভুল বা DNS-এ নেই)
`Request timeout` / বেশি `time` আর `packet loss`	নেটওয়ার্ক ব্যস্ত বা দুর্বল সিগন্যাল

⚠️ **সতর্কতা:** ping-এর উত্তর না আসা মানেই হোস্ট মৃত নয়। অনেক সার্ভার আর ক্লাউড (যেমন AWS-এর ডিফল্ট security group) ICMP আটকে রাখে। তখন `nc -zv host 443` বা `curl` দিয়ে আসল সার্ভিসটা পরীক্ষা করো।

> 💡 **টিপস:** ping চলার সময় `Ctrl+\` চাপলে ping না থামিয়েই এখন পর্যন্ত পরিসংখ্যান দেখায়।

> 📝 **মনে রাখো:** আধুনিক লিনাক্সে সাধারণ ইউজার `ping` চালাতে পারে, কারণ কার্নেল `net.ipv4.ping_group_range` দিয়ে unprivileged ICMP socket অনুমতি দেয় (বা পুরনো সিস্টেমে `ping`-এ `cap_net_raw` capability দেওয়া থাকে)। capability নিয়ে আছে অধ্যায় ১৭ আর অধ্যায় ৪৫-এ।

MTU সমস্যা ধরার কৌশল

Ethernet-এর MTU ১৫০০। ICMP+IP header ২৮ byte, তাই সর্বোচ্চ ডেটা ১৫০০ − ২৮ = ১৪৭২।

```
$ ping -c 2 -M do -s 1472 8.8.8.8      # কাজ করা উচিত
$ ping -c 2 -M do -s 1473 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 1473(1501) bytes of data.
ping: local error: message too long, mtu=1500
```

VPN বা PPPoE সংযোগে MTU কম থাকে (যেমন ১৪৯২ বা আরও কম)। তখন "কিছু ওয়েবসাইট খোলে, কিছু আটকে থাকে"-জাতীয় অদ্ভুত সমস্যা হয়। এই কৌশলে সঠিক MTU বের করা যায়।

৩৭.৬ `traceroute`, `tracepath` ও `mtr` — রাস্তার প্রতিটা মোড় দেখা

TTL-এর চতুর কৌশল

প্রতিটা IP প্যাকেটে একটা **TTL** (Time To Live) সংখ্যা থাকে। প্রতিটা রাউটার প্যাকেটটা পার করার সময় TTL থেকে ১ কমায়। TTL ০ হয়ে গেলে রাউটার প্যাকেটটা ফেলে দেয় আর প্রেরককে একটা ICMP **"Time Exceeded"** বার্তা পাঠায়, যাতে রাউটারের নিজের IP থাকে। (এই ব্যবস্থা আছে যাতে ভুল route-এ প্যাকেট অনন্তকাল ঘুরতে না থাকে।)

`traceroute` এই নিয়মের সুবিধা নেয়:

```
  TTL=1 পাঠাও → ১ম রাউটার TTL শূন্য করে → "Time Exceeded" ফেরত → ১ম hop জানা গেল
  TTL=2 পাঠাও → ১ম পার, ২য় রাউটারে শূন্য → "Time Exceeded" → ২য় hop জানা গেল
  TTL=3 ...
  ... গন্তব্যে পৌঁছালে সে ভিন্ন উত্তর দেয় (Port Unreachable / Echo Reply / SYN-ACK)

  তোমার PC ──► [রাউটার ১] ──► [ISP রাউটার] ──► [ব্যাকবোন] ──► ... ──► গন্তব্য
                TTL 1-এ মরে      TTL 2-এ মরে      TTL 3-এ মরে
```

> 💡 **উপমা:** একটা বল ছুঁড়ছ যেটা ১ বার হাত বদলের পর থেমে যায়। যার হাতে থামল, সে চিৎকার করে নিজের

> নাম বলে। এরপর ২ বার হাত বদলের বল ছোঁড়ো... এভাবে লাইনের প্রত্যেকের নাম জেনে যাও।

`traceroute`

```
$ traceroute google.com
traceroute to google.com (142.250.193.110), 30 hops max, 60 byte packets
 1  _gateway (192.168.1.1)  1.123 ms  1.045 ms  0.998 ms
 2  10.20.0.1 (10.20.0.1)  4.512 ms  4.498 ms  4.470 ms
 3  103.12.44.1 (103.12.44.1)  6.210 ms  6.188 ms  6.150 ms
 4  * * *
 5  72.14.218.10 (72.14.218.10)  22.877 ms  22.840 ms  22.812 ms
 6  del12s08-in-f14.1e100.net (142.250.193.110)  24.301 ms  24.255 ms  24.230 ms
```

- প্রতিটা লাইন একটা **hop** (রাউটার)। প্রতিটায় ৩টা প্রোব পাঠানো হয়, তাই ৩টা সময়।
- `* * *` মানে ওই রাউটার উত্তর দেয়নি। এটা সবসময় সমস্যা নয়। অনেক রাউটার "Time Exceeded" পাঠায় না বা কম অগ্রাধিকার দেয়। পরের hop-গুলো ঠিক থাকলে চিন্তা নেই।
- কোনো hop থেকে শুরু করে **শেষ পর্যন্ত** সব `* * *` হলে সমস্যা সম্ভবত সেখান থেকেই।

লিনাক্সের `traceroute` ডিফল্টভাবে **UDP** প্যাকেট পাঠায় (port 33434 থেকে শুরু)। কিছু firewall UDP আটকায়, তাই অন্য পদ্ধতিও আছে:

অপশন	কাজ
`-n`	DNS নাম খুঁজবে না (অনেক দ্রুত)
`-I`	UDP-র বদলে ICMP Echo (Windows-এর `tracert`-এর মতো; root লাগতে পারে)
`-T`	TCP SYN প্রোব (firewall পার হতে সবচেয়ে ভালো; root লাগে)
`-p PORT`	port নির্ধারণ (যেমন `-T -p 443`)
`-m N`	সর্বোচ্চ hop (ডিফল্ট ৩০)
`-q N`	প্রতি hop-এ কতটা প্রোব (ডিফল্ট ৩)
`-w SEC`	উত্তরের অপেক্ষা
`-4` / `-6`	IPv4 / IPv6

```
$ sudo traceroute -n -T -p 443 example.com
```

`tracepath` — root ছাড়া, MTU সহ

`tracepath` প্রায় `traceroute`-এর মতো, কিন্তু root লাগে না এবং পথের **PMTU** (Path MTU) দেখায়।

```
$ tracepath -n 8.8.8.8
 1?: [LOCALHOST]                      pmtu 1500
 1:  192.168.1.1                                           1.201ms
 1:  192.168.1.1                                           1.087ms
 2:  10.20.0.1                                             4.660ms
```

```
 3:  103.12.44.1                                           6.302ms asymm  4
 4:  no reply
 5:  8.8.8.8                                              21.904ms reached
     Resume: pmtu 1500 hops 5 back 5
```

`asymm 4` মানে ফেরার পথে hop সংখ্যা ভিন্ন (যাওয়া-আসার রাস্তা আলাদা)।

`mtr` — `ping` + `traceroute` একসাথে, লাইভ

`mtr` (My Traceroute) একটানা প্রতিটা hop-কে ping করতে থাকে আর প্রতিটার packet loss ও সময় লাইভ দেখায়। ISP-কে "আপনার নেটওয়ার্কের কোথায় প্যাকেট হারাচ্ছে" প্রমাণ দেখাতে এটাই সবচেয়ে কাজের।

```
$ mtr google.com            # ইন্টারঅ্যাকটিভ, লাইভ আপডেট (q চেপে বের হও)

$ mtr -rwn -c 100 8.8.8.8   # ১০০টা প্রোব পাঠিয়ে রিপোর্ট
Start: 2026-11-12T10:15:02+0600
HOST: mylaptop                     Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 192.168.1.1                 0.0%   100    1.1   1.2   0.9   3.4   0.3
  2.|-- 10.20.0.1                   0.0%   100    4.5   4.8   4.1  12.0   0.9
  3.|-- 103.12.44.1                35.0%   100    6.3   6.4   6.0   9.8   0.5
  4.|-- 72.14.218.10                0.0%   100   22.9  23.0  22.5  27.1   0.6
  5.|-- 8.8.8.8                     0.0%   100   21.9  22.1  21.6  25.0   0.4
```

কলাম	মানে
`Loss%`	এই hop-এ কত শতাংশ প্রোবের উত্তর আসেনি
`Snt`	কতটা পাঠানো হয়েছে
`Last/Avg/Best/Wrst`	শেষ/গড়/সেরা/সবচেয়ে খারাপ RTT (ms)
`StDev`	ওঠানামা

> 📝 **মনে রাখো (খুব গুরুত্বপূর্ণ):** ওপরে hop ৩-এ `35% Loss`, কিন্তু hop ৪ আর ৫-এ `0%`। তার মানে **আসল ক্ষতি নেই**। hop ৩-এর রাউটার শুধু নিজের কাছে আসা প্রোবের উত্তর দিতে কৃপণতা করছে (ICMP rate-limiting), কিন্তু অন্যদের ট্রাফিক ঠিকঠাক পার করছে। **আসল packet loss তখনই, যখন loss একটা hop থেকে শুরু হয়ে শেষ পর্যন্ত সব hop-এ থাকে।**

`mtr` অপশন	কাজ
`-r` / `--report`	রিপোর্ট মোড (শেষে একবার প্রিন্ট)
`-w` / `--report-wide`	পূর্ণ hostname সহ চওড়া রিপোর্ট
`-c N`	কতটা প্রোব
`-n`	DNS বন্ধ
`-b`	নাম ও IP দুটোই দেখাও
`-T` / `-u`	TCP / UDP প্রোব (ডিফল্ট ICMP)

`mtr` অপশন	কাজ
`-P PORT`	TCP/UDP port
`-4` / `-6`	IPv4 / IPv6

`mtr`-এর ইন্টারঅ্যাকটিভ কীবোর্ড শর্টকাট

কী	কাজ
`q`	বের হও
`p`	থামাও (pause)
`Space`	আবার চালাও (resume)
`r`	পরিসংখ্যান রিসেট
`n`	DNS নাম দেখানো চালু/বন্ধ
`d`	ডিসপ্লে মোড বদলাও (টেবিল/গ্রাফ)
`j`	latency আর jitter কলামের মধ্যে বদলাও
`o`	কলামের ক্রম বদলাও
`h` / `?`	সাহায্য

৩৭.৭ `ss` — কোন port খোলা, কে কার সাথে কথা বলছে

socket কী? ভেতরে কীভাবে কাজ করে

প্রোগ্রাম নেটওয়ার্কে কথা বলে **socket** দিয়ে। socket হলো কার্নেলের ভেতরে একটা যোগাযোগের "প্লাগ পয়েন্ট"। প্রতিটা TCP সংযোগ চারটা জিনিস দিয়ে আলাদা করে চেনা যায়: **(local IP, local port, remote IP, remote port)**।

একটা ওয়েব সার্ভার port 80-এ **listen** করে, মানে দরজা খুলে বসে থাকে। কেউ সংযোগ করলে কার্নেল একটা নতুন **ESTABLISHED** socket তৈরি করে।

> 💡 **উপমা:** একটা অফিসের অভ্যর্থনা ডেস্ক (LISTEN socket) সবসময় খোলা থাকে। অতিথি এলে অভ্যর্থনাকারী তাকে একটা আলাদা মিটিং রুমে (ESTABLISHED socket) বসিয়ে দেয়, আর নিজে আবার পরের অতিথির অপেক্ষায় থাকে।

`ss` (socket statistics) কার্নেলের socket টেবিল সরাসরি Netlink দিয়ে পড়ে। পুরনো `netstat` `/proc/net/tcp` ফাইল পার্স করত, তাই হাজার হাজার সংযোগ থাকলে খুব ধীর হতো। `ss` সেখানে অনেক দ্রুত।

সবচেয়ে বিখ্যাত কমান্ড: `ss -tulpn`

```
$ sudo ss -tulpn
Netid State  Recv-Q Send-Q  Local Address:Port   Peer Address:Port Process
udp   UNCONN 0      0       127.0.0.53%lo:53          0.0.0.0:*     users:(("systemd-
resolve",pid=612,fd=13))
udp   UNCONN 0      0             0.0.0.0:68          0.0.0.0:*     users:
(("NetworkManager",pid=801,fd=27))
tcp   LISTEN 0      4096    127.0.0.53%lo:53          0.0.0.0:*     users:(("systemd-
resolve",pid=612,fd=14))
tcp   LISTEN 0      128           0.0.0.0:22          0.0.0.0:*     users:
(("sshd",pid=1023,fd=3))
tcp   LISTEN 0      511           0.0.0.0:80          0.0.0.0:*     users:
(("nginx",pid=1240,fd=6))
tcp   LISTEN 0      151         127.0.0.1:3306        0.0.0.0:*     users:
(("mysqld",pid=1310,fd=21))
tcp   LISTEN 0      128              [::]:22             [::]:*     users:
(("sshd",pid=1023,fd=4))
```

অপশনগুলো মনে রাখার সহজ উপায় **"টিউলিপ-এন"** (tulpn):

অপশন	মানে
`-t`	TCP socket
`-u`	UDP socket
`-l`	শুধু listening (যারা অপেক্ষা করছে)
`-p`	কোন process (নাম, PID)। অন্য ইউজারের process দেখতে `sudo` লাগে
`-n`	numeric: port নম্বর আর IP দেখাও, নাম নয় (`:22` , `:ssh` নয়)

Local Address কলাম পড়তে শেখো (নিরাপত্তার জন্য খুব জরুরি):

Local Address	মানে
`0.0.0.0:80`	সব IPv4 interface-এ খোলা, বাইরের যে কেউ সংযোগ করতে পারবে (firewall না থাকলে)
`[::]:22` বা `*:22`	সব IPv6 (আর প্রায়ই IPv4-ও) interface-এ খোলা
`127.0.0.1:3306`	শুধু এই কম্পিউটার থেকেই সংযোগ করা যাবে। ডেটাবেসের জন্য এটাই নিরাপদ
`127.0.0.53%lo:53`	শুধু loopback-এ (systemd-resolved-এর DNS stub)
`192.168.1.105:8080`	শুধু ওই নির্দিষ্ট IP-তে

> ⚠️ **সতর্কতা:** ডেটাবেস (MySQL 3306, PostgreSQL 5432, Redis 6379, MongoDB 27017) যদি `0.0.0.0`-এ listen করে আর firewall না থাকে, তাহলে পুরো ইন্টারনেট সেটায় ঢোকার চেষ্টা করতে পারবে। সার্ভার সেটআপের পর প্রথম কাজ `sudo ss -tulpn` দেখা। firewall নিয়ে অধ্যায় ৩৯, নিরাপত্তা নিয়ে অধ্যায় ৪৫।

`Recv-Q` / `Send-Q` : LISTEN socket-এ `Send-Q` হলো backlog-এর সর্বোচ্চ সীমা, আর `Recv-Q` হলো এখন কতটা সংযোগ accept হওয়ার অপেক্ষায়। ESTABLISHED socket-এ এগুলো হলো এখনো না পড়া/না পাঠানো byte। এগুলো বারবার বড় দেখালে অ্যাপ্লিকেশন বা নেটওয়ার্ক ধীর।

`ss` -এর অন্য অপশন

অপশন	কাজ
`-a`	সব socket (listening + সংযুক্ত)
`-x`	Unix domain socket (একই মেশিনের প্রোগ্রামের মধ্যে যোগাযোগ)
`-w`	raw socket
`-4` / `-6`	শুধু IPv4 / IPv6
`-s`	সারাংশ পরিসংখ্যান
`-e`	বর্ধিত তথ্য (uid, inode)
`-o`	টাইমার (keepalive, retransmit)
`-i`	TCP-র ভেতরের তথ্য (rtt, cwnd, retrans)
`-m`	socket-এর মেমরি ব্যবহার
`-K`	মেলানো socket জোর করে বন্ধ (root, কার্নেল সাপোর্ট লাগে)
`-H`	হেডার লাইন বাদ (স্ক্রিপ্টের জন্য)

```
$ ss -s
Total: 812
TCP:   34 (estab 12, closed 6, orphaned 0, timewait 5)

Transport Total     IP        IPv6
RAW       1         0         1
UDP       9         6         3
TCP       28        21        7
INET      38        27        11
FRAG      0         0         0
```

ফিল্টার: `ss` -এর আসল শক্তি

```
# শুধু ESTABLISHED TCP সংযোগ
$ ss -tn state established
Recv-Q Send-Q   Local Address:Port      Peer Address:Port
0      0        192.168.1.105:51432    142.250.193.110:443
0      0        192.168.1.105:22         192.168.1.20:60122

# কে আমার port 22-তে সংযুক্ত?
$ ss -tn state established '( sport = :22 )'

# আমি কোন কোন সার্ভারের 443 port-এ সংযুক্ত?
$ ss -tn '( dport = :443 )'
```

```
# port 8080-এ কে listen করছে?
$ sudo ss -tlnp 'sport = :8080'

# একটা নির্দিষ্ট IP-র সাথে সব সংযোগ
$ ss -tn dst 192.168.1.20

# TIME-WAIT অবস্থার সংযোগ গোনা
$ ss -tan state time-wait | wc -l
```

TCP-র অবস্থা (state) নাম: `established`, `syn-sent`, `syn-recv`, `fin-wait-1`, `fin-wait-2`, `time-wait`, `closed`, `close-wait`, `last-ack`, `listening`, `closing`। এছাড়া কিছু দলগত নামও আছে: `connected` (listening আর closed বাদে সব), `synchronized`, `bucket`, `big`।

> 💡 **টিপস:** অনেক `CLOSE-WAIT` জমে থাকলে বুঝবে অ্যাপ্লিকেশন নিজের দিক থেকে সংযোগ বন্ধ করতে ভুলে যাচ্ছে (কোডে bug)। অনেক `SYN-RECV` থাকলে সম্ভবত SYN flood আক্রমণ চলছে।

`netstat` (পুরনো) — চিনে রাখো

```
$ sudo netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
1023/sshd
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
1240/nginx: master
```

অপশনগুলো `ss`-এর মতোই। এছাড়া `netstat -r` (route), `netstat -i` (interface), `netstat -s` (প্রোটোকল পরিসংখ্যান)।

৩৭.৮ `lsof -i` — নেটওয়ার্ক ব্যবহারকারী ফাইল খোঁজা

লিনাক্সে "সবকিছুই ফাইল"। তাই একটা নেটওয়ার্ক socket-ও প্রসেসের একটা খোলা file descriptor। `lsof` (LiSt Open Files) সব খোলা ফাইলের তালিকা দেয়, আর `-i` দিলে শুধু নেটওয়ার্কেরগুলো। (অধ্যায় ২৪-এ process আর file descriptor নিয়ে আরও আছে।)

```
$ sudo lsof -i :80
COMMAND  PID     USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
nginx   1240     root    6u  IPv4  23456      0t0  TCP *:http (LISTEN)
nginx   1241 www-data    6u  IPv4  23456      0t0  TCP *:http (LISTEN)
```

কমান্ড	কাজ
`lsof -i`	সব নেটওয়ার্ক সংযোগ

কমান্ড	কাজ
`lsof -i :22`	port 22 ব্যবহার করছে যারা
`lsof -i tcp` / `lsof -i udp`	শুধু TCP / UDP
`lsof -i tcp:443`	TCP port 443
`lsof -i @192.168.1.20`	এই হোস্টের সাথে সংযোগ
`lsof -i -n -P`	`-n` = IP নাম বানাবে না, `-P` = port নাম বানাবে না (দ্রুত)
`lsof -iTCP -sTCP:LISTEN`	শুধু listening TCP
`lsof -i -a -p 1240`	PID 1240-এর নেটওয়ার্ক সংযোগ (`-a` = AND)
`lsof -i -a -u www-data`	একজন ইউজারের নেটওয়ার্ক সংযোগ
`lsof -t -i :8080`	শুধু PID প্রিন্ট (`-t` = terse)

বাস্তব উদাহরণ: "port 3000 already in use" এরর এল। কে ধরে রেখেছে, আর তাকে বন্ধ করো:

```
$ lsof -t -i :3000
48211
$ kill 48211          # বা এক লাইনে: kill $(lsof -t -i :3000)
```

> ⚠️ **সতর্কতা:** `sudo` ছাড়া `lsof` শুধু তোমার নিজের প্রসেস দেখায়। ফলে "কিছুই নেই" দেখালেও আসলে অন্য ইউজারের প্রসেস port দখল করে থাকতে পারে।

> 💡 **টিপস:** আরেকটা ছোট টুল `fuser`: `sudo fuser 80/tcp` port 80 ব্যবহারকারী PID দেখায়, আর `sudo fuser -k 80/tcp` তাদের মেরে ফেলে (সাবধানে!)।

৩৭.৯ DNS টুল: `dig`, `nslookup`, `host`

DNS হলো ইন্টারনেটের ফোনবুক: `google.com` নাম থেকে IP বের করে। কীভাবে কাজ করে (resolver, root, TLD, authoritative server, record-এর ধরন) তা অধ্যায় ৩৬-এ আছে। এখানে আমরা প্রশ্ন করব।

আগে জানো: তোমার সিস্টেম কোন DNS সার্ভারকে জিজ্ঞেস করে

```
$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
nameserver 127.0.0.53
options edns0 trust-ad
search home.lan
```

Ubuntu আর Fedora-তে **systemd-resolved** চলে। সে `127.0.0.53`-এ একটা লোকাল DNS stub চালায়, যা উত্তর cache করে আর আসল DNS সার্ভারে প্রশ্ন পাঠায়। আসল সার্ভার দেখতে:

```
$ resolvectl status
Global
       Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
resolv.conf mode: stub

Link 2 (enp3s0)
    Current Scopes: DNS
  Current DNS Server: 192.168.1.1
         DNS Servers: 192.168.1.1 8.8.8.8
          DNS Domain: home.lan
```

```
 প্রোগ্রাম (curl, firefox)
        │ getaddrinfo("google.com")
        ▼
 glibc → /etc/nsswitch.conf ("hosts: files ... dns")
        │ ১) /etc/hosts দেখে
        │ ২) না পেলে /etc/resolv.conf-এর nameserver-কে জিজ্ঞেস করে
        ▼
 systemd-resolved (127.0.0.53)  ── cache-এ থাকলে সাথে সাথে উত্তর
        │
        ▼
 রাউটার / ISP / 8.8.8.8 (recursive resolver)
        │
        ▼
 root (.) → TLD (.com) → authoritative (google.com-এর নিজের সার্ভার)
```

```
$ resolvectl query google.com          # resolved দিয়ে প্রশ্ন
$ resolvectl flush-caches              # DNS cache খালি করো
$ resolvectl statistics                # cache hit/miss
```

> 📝 **মনে রাখো:** `dig`, `nslookup` আর `host` সরাসরি DNS সার্ভারকে প্রশ্ন করে, `/etc/hosts` দেখে না। আর সাধারণ প্রোগ্রাম (`ping`, `curl`) glibc হয়ে যায়, তাই `/etc/hosts`-ও দেখে। "dig ঠিক উত্তর দিচ্ছে কিন্তু ব্রাউজার ভুল জায়গায় যাচ্ছে" হলে `/etc/hosts` দেখো। glibc-র মতো করে নাম খুঁজতে: `getent hosts google.com`।

`dig` — পেশাদারের DNS টুল

```
$ dig google.com

; <<>> DiG 9.18.28-0ubuntu0.24.04.1-Ubuntu <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 31415
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
```

```
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;google.com.                    IN      A

;; ANSWER SECTION:
google.com.             187     IN      A       142.250.193.110

;; Query time: 3 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Thu Nov 12 10:20:11 +06 2026
;; MSG SIZE  rcvd: 55
```

অংশ	মানে
`status: NOERROR`	সফল। `NXDOMAIN` = ডোমেইনই নেই; `SERVFAIL` = সার্ভার উত্তর দিতে পারেনি (প্রায়ই DNSSEC বা authoritative সার্ভারের সমস্যা); `REFUSED` = সার্ভার প্রশ্ন নিতে রাজি নয়
`flags: qr rd ra`	`qr` = এটা উত্তর; `rd` = recursion চাওয়া হয়েছে; `ra` = recursion পাওয়া যায়; `aa` = authoritative উত্তর
`QUESTION SECTION`	কী জিজ্ঞেস করা হয়েছিল (`A` record)
`ANSWER SECTION`	উত্তর: নাম, **TTL (187 সেকেন্ড, এতক্ষণ cache থাকবে)**, class `IN`, type `A`, মান
`Query time`	কত সময় লাগল (cache থেকে এলে খুব কম)
`SERVER`	কোন DNS সার্ভার উত্তর দিল

`dig`-এর সবচেয়ে কাজের রূপগুলো

```
$ dig +short google.com                # শুধু উত্তর
142.250.193.110

$ dig google.com AAAA +short           # IPv6 ঠিকানা
2404:6800:4002:82d::200e

$ dig gmail.com MX +short              # মেইল সার্ভার (সংখ্যা = অগ্রাধিকার, কম = আগে)
5 gmail-smtp-in.l.google.com.
10 alt1.gmail-smtp-in.l.google.com.

$ dig google.com NS +short             # ডোমেইনের name server
$ dig google.com TXT +short            # TXT (SPF, ডোমেইন যাচাই)
$ dig www.github.com CNAME +short      # অন্য নামের ছদ্মনাম
$ dig google.com SOA +short            # zone-এর মূল তথ্য
$ dig google.com ANY                   # অনেক সার্ভার এখন ANY-র পূর্ণ উত্তর দেয় না (RFC 8482)

$ dig @8.8.8.8 example.com             # নির্দিষ্ট DNS সার্ভারকে প্রশ্ন (Google)
$ dig @1.1.1.1 example.com +short      # Cloudflare

$ dig -x 8.8.8.8 +short                # reverse lookup: IP → নাম (PTR)
dns.google.

$ dig +noall +answer google.com        # শুধু ANSWER সেকশন, পূর্ণ লাইনে
google.com.             187     IN      A       142.250.193.110
```

```
$ dig +trace example.com                # root থেকে শুরু করে প্রতিটা ধাপ নিজে হেঁটে দেখাও
```

`dig` অপশন	কাজ
`@SERVER`	কোন DNS সার্ভারকে জিজ্ঞেস করবে
`TYPE` (`A`, `AAAA`, `MX`, `NS`, `TXT`, `CNAME`, `SOA`, `PTR`, `SRV`, `CAA`)	record-এর ধরন
`-x IP`	reverse lookup
`+short`	সংক্ষিপ্ত উত্তর
`+noall +answer`	শুধু উত্তর সেকশন
`+trace`	root থেকে পুরো পথ
`+tcp`	UDP-র বদলে TCP
`+dnssec`	DNSSEC record চাও
`-4` / `-6`	IPv4 / IPv6 দিয়ে প্রশ্ন পাঠাও
`-p PORT`	ভিন্ন port
`+norecurse`	recursion চাইবে না

> 💡 **টিপস:** নতুন ডোমেইন বা DNS পরিবর্তন করেছ কিন্তু কাজ করছে না? `dig @8.8.8.8 yourdomain.com` আর `dig @1.1.1.1 yourdomain.com` দুটো চালিয়ে মেলাও, তারপর `dig yourdomain.com NS +short` দিয়ে authoritative সার্ভার বের করে সরাসরি তাকে জিজ্ঞেস করো (`dig @ns1.example-dns.com yourdomain.com`)। authoritative সার্ভার নতুন উত্তর দিলে বাকিরা পুরনো TTL শেষ হলেই আপডেট পাবে।

`nslookup` — পুরনো, সব OS-এ পাওয়া যায়

Windows-এও আছে, তাই অনেকে চেনে।

```
$ nslookup google.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.193.110
Name:   google.com
Address: 2404:6800:4002:82d::200e

$ nslookup -type=MX gmail.com
$ nslookup google.com 8.8.8.8           # নির্দিষ্ট সার্ভার (শেষে দাও)
```

"Non-authoritative answer" মানে উত্তর এসেছে cache বা recursive resolver থেকে, ডোমেইনের নিজের সার্ভার থেকে নয়। এটা স্বাভাবিক।

ইন্টারঅ্যাকটিভ মোডও আছে: শুধু `nslookup` লিখে Enter দাও, তারপর `set type=MX`, `server 8.8.8.8`, ডোমেইন নাম লেখো; বের হতে `exit`।

`host` — সবচেয়ে সহজ, পরিষ্কার উত্তর

```
$ host google.com
google.com has address 142.250.193.110
google.com has IPv6 address 2404:6800:4002:82d::200e
google.com mail is handled by 10 smtp.google.com.

$ host 8.8.8.8
8.8.8.8.in-addr.arpa domain name pointer dns.google.

$ host -t NS google.com
$ host -t TXT google.com
$ host google.com 1.1.1.1
$ host -a google.com           # বিস্তারিত (dig-এর মতো)
```

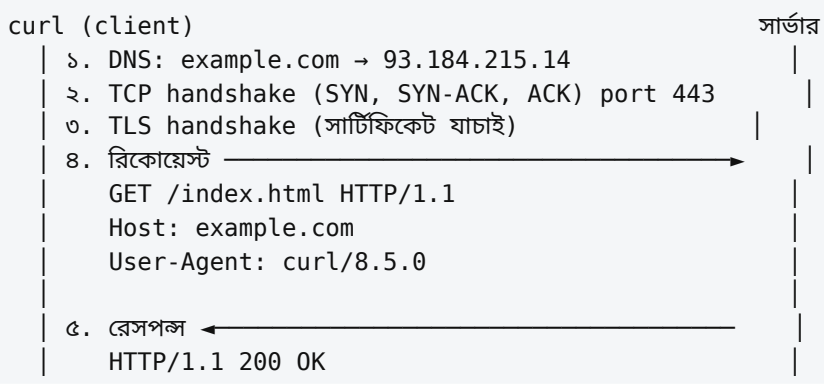
তিনটার তুলনা

বৈশিষ্ট্য	`dig`	`nslookup`	`host`
আউটপুট	বিস্তারিত, কাঁচা DNS	মাঝামাঝি	সংক্ষিপ্ত, ইংরেজি বাক্যে
স্ক্রিপ্টে ব্যবহার	চমৎকার (`+short`)	ভালো নয়	ভালো
`+trace` / DNSSEC	হ্যাঁ	না	সীমিত
কখন ব্যবহার করবে	ট্রাবলশুটিং, পেশাদার কাজ	Windows-এর অভ্যাস থাকলে	দ্রুত এক নজরে

৩৭.১০ `curl` — HTTP-র ছুরি-কাঁচি

`curl` (Client URL) টার্মিনাল থেকে URL-এ রিকোয়েস্ট পাঠায়। HTTP, HTTPS, FTP, SFTP, SCP সহ বহু প্রোটোকল সাপোর্ট করে। API টেস্ট, ওয়েবসাইট চেক, ফাইল নামানো, সার্ভারের হেডার দেখা: ডেভেলপার আর DevOps-এর প্রতিদিনের সঙ্গী।

একটা HTTP রিকোয়েস্ট-রেসপন্স কেমন দেখতে



```
    |    Content-Type: text/html                          |
    |    (ফাঁকা লাইন)                                    |
    |    <html>...body...</html>                          |
```

বেসিক

```
$ curl https://example.com                # body টার্মিনালে প্রিন্ট
$ curl -s https://api.ipify.org           # -s = silent (progress bar নেই); তোমার পাবলিক
IP
103.12.44.87
```

`-I` — শুধু হেডার দেখো (HEAD রিকোয়েস্ট)

```
$ curl -I https://example.com
HTTP/2 200
content-type: text/html
etag: "84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
last-modified: Mon, 13 Jan 2025 20:11:20 GMT
cache-control: max-age=2357
date: Thu, 12 Nov 2026 04:20:11 GMT
content-length: 1256
```

প্রথম লাইনের **status code** সবচেয়ে গুরুত্বপূর্ণ:

কোড	মানে
`200`	OK, সফল
`201`	Created (নতুন কিছু তৈরি হয়েছে, POST-এর পরে)
`204`	No Content (সফল, কিন্তু body নেই)
`301` / `308`	স্থায়ীভাবে অন্য জায়গায় সরে গেছে
`302` / `307`	অস্থায়ী redirect
`304`	Not Modified (cache-এরটাই চলবে)
`400`	Bad Request (রিকোয়েস্ট ভুল)
`401`	Unauthorized (লগইন/টোকেন লাগবে)
`403`	Forbidden (অনুমতি নেই)
`404`	Not Found
`405`	Method Not Allowed
`429`	Too Many Requests (rate limit)
`500`	Internal Server Error (সার্ভারের কোডে সমস্যা)
`502`	Bad Gateway (proxy-র পেছনের অ্যাপ সাড়া দেয়নি)
`503`	Service Unavailable

কোড	মানে
`504`	Gateway Timeout

> 📝 **মনে রাখো:** `-I` আসলে HEAD মেথড পাঠায়। কিছু সার্ভার HEAD-এ ঠিকমতো উত্তর দেয় না। তখন `curl -s -D - -o /dev/null URL` চালাও: GET পাঠাবে, হেডার (`-D -` = stdout-এ dump) দেখাবে আর body ফেলে দেবে। অথবা `-i` (ছোট হাতের) দিলে হেডার আর body দুটোই দেখায়।

`-L` — redirect অনুসরণ করো

```
$ curl -I http://github.com
HTTP/1.1 301 Moved Permanently
Content-Length: 0
Location: https://github.com/

$ curl -sIL http://github.com | grep -i -E '^(HTTP|location)'
HTTP/1.1 301 Moved Permanently
Location: https://github.com/
HTTP/2 200
```

ডিফল্টভাবে `curl` redirect অনুসরণ করে না। `-L` দিলে `Location` হেডার ধরে শেষ পর্যন্ত যায় (সর্বোচ্চ সংখ্যা `--max-redirs`)।

`-o` আর `-O` — ফাইলে সংরক্ষণ

```
$ curl -o ubuntu.iso https://releases.ubuntu.com/24.04/ubuntu-24.04.1-desktop-amd64.iso
$ curl -O https://example.com/files/report.pdf      # সার্ভারের নামেই (report.pdf) সেভ
$ curl -LO https://github.com/user/repo/releases/download/v1.0/tool.tar.gz   # redirect
+ সেভ
$ curl -C - -O https://example.com/big.iso          # থেমে যাওয়া ডাউনলোড আবার শুরু
(resume)
$ curl -o /dev/null -s -w "%{http_code}\n" https://example.com   # body ফেলে শুধু status
200
```

`-X`, `-H`, `-d` — API নিয়ে কাজ

```
# -X: HTTP method (GET, POST, PUT, PATCH, DELETE)
$ curl -X DELETE https://api.example.com/users/42

# -H: হেডার যোগ করো (একাধিকবার দেওয়া যায়)
$ curl -H "Accept: application/json" \
       -H "Authorization: Bearer eyJhbGciOi..." \
       https://api.example.com/me

# -d: body-তে ডেটা পাঠাও। -d দিলে method নিজে থেকেই POST হয়,
#     আর Content-Type হয় application/x-www-form-urlencoded
$ curl -d "name=Rahim&age=25" https://httpbin.org/post
```

```
# JSON পাঠানো (Content-Type নিজে বলে দিতে হবে)
$ curl -X POST https://httpbin.org/post \
       -H "Content-Type: application/json" \
       -d '{"name":"Rahim","age":25}'

# ফাইল থেকে body পড়ো (@ চিহ্ন)
$ curl -H "Content-Type: application/json" -d @user.json https://httpbin.org/post

# curl 7.82+ এ শর্টকাট: --json নিজেই Content-Type ও Accept হেডার বসায় আর POST করে
$ curl --json '{"name":"Rahim"}' https://httpbin.org/post

# PUT দিয়ে আপডেট
$ curl -X PUT -H "Content-Type: application/json" -d '{"age":26}'
https://api.example.com/users/42

# ফর্মে ফাইল আপলোড (multipart/form-data)
$ curl -F "photo=@/home/rahim/pic.jpg" -F "title=Holiday" https://httpbin.org/post
```

⚠️ **সতর্কতা:** `-X POST`-এর সাথে `-d` দিলে `-X` আসলে অপ্রয়োজনীয়, তবে ক্ষতি নেই। কিন্তু `-X POST`-এর সাথে `-L` দিলে redirect-এর পরেও POST-ই পাঠাবে, যা প্রায়ই ভুল ফল দেয়। redirect-এ সঠিক আচরণ চাইলে `-X` বাদ দিয়ে শুধু `-d` ব্যবহার করো।

⚠️ **সতর্কতা:** কমান্ড লাইনে পাসওয়ার্ড বা টোকেন লিখলে সেটা shell history-তে (`~/.bash_history`) আর `ps` আউটপুটে অন্যরা দেখতে পারে। লাইনের শুরুতে একটা স্পেস দিলে (যদি `HISTCONTROL=ignorespace` থাকে) history-তে যায় না, অথবা `-H @headers.txt` দিয়ে ফাইল থেকে হেডার পড়াও (curl 7.55+)। বিস্তারিত অধ্যায় ৬ আর অধ্যায় ৪৪-এ।

`-v` আর `-w` — ভেতরে কী হচ্ছে দেখো

```
$ curl -v https://example.com -o /dev/null
*   Trying 93.184.215.14:443...
* Connected to example.com (93.184.215.14) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
...
* Server certificate:
*  subject: C=US; ST=California; L=Los Angeles; O=Internet Corporation for Assigned
Names and Numbers; CN=*.example.com
*  expire date: Jan 15 23:59:59 2027 GMT
> GET / HTTP/2
> Host: example.com
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/2 200
< content-type: text/html
...
```

`*` = curl-এর নিজের তথ্য, `>` = পাঠানো রিকোয়েস্ট, `<` = পাওয়া রেসপন্স। সার্টিফিকেটের মেয়াদ আর TLS সমস্যা এখানেই ধরা পড়ে।

`-w` (write-out) দিয়ে সময় মাপো, কোন ধাপে দেরি হচ্ছে বোঝো:

```
$ curl -o /dev/null -s -w "DNS: %{time_namelookup}s\nConnect: %{time_connect}s\nTLS: %
{time_appconnect}s\nFirstByte: %{time_starttransfer}s\nTotal: %{time_total}s\nCode: %
{http_code}\n" https://example.com
DNS: 0.004512s
Connect: 0.152330s
TLS: 0.318842s
FirstByte: 0.471220s
Total: 0.471580s
Code: 200
```

এই সময়গুলো **শুরু থেকে জমা হিসাবে** গোনা। DNS বেশি হলে DNS সমস্যা; Connect বেশি হলে নেটওয়ার্ক বা দূরত্ব; FirstByte আর TLS-এর পার্থক্য বেশি হলে সার্ভারের অ্যাপ ধীর।

`curl`-এর গুরুত্বপূর্ণ অপশন — সম্পূর্ণ টেবিল

অপশন	পূর্ণ নাম	কাজ
`-I`	`--head`	শুধু হেডার (HEAD method)
`-i`	`--include`	হেডার + body
`-L`	`--location`	redirect অনুসরণ
`-o FILE`	`--output`	নির্দিষ্ট ফাইলে সেভ
`-O`	`--remote-name`	URL-এর নামে সেভ
`-X METHOD`	`--request`	HTTP method
`-H "K: V"`	`--header`	হেডার যোগ
`-d DATA`	`--data`	body ডেটা (POST)
`--data-urlencode`		ডেটা URL-encode করে পাঠাও
`--json DATA`		JSON পাঠাও (7.82+)
`-F`	`--form`	multipart ফর্ম/ফাইল আপলোড
`-u user:pass`	`--user`	Basic authentication
`-v`	`--verbose`	বিস্তারিত ডিবাগ
`-s`	`--silent`	progress/error চুপ
`-S`	`--show-error`	`-s`-এর সাথে দিলে শুধু error দেখাবে
`-f`	`--fail`	HTTP 4xx/5xx এ কোনো body না দেখিয়ে non-zero exit code (স্ক্রিপ্টে দরকারি)
`-k`	`--insecure`	TLS সার্টিফিকেট যাচাই করবে না (শুধু টেস্টে!)

অপশন	পূর্ণ নাম	কাজ
`-w FORMAT`	`--write-out`	শেষে নিজের মতো তথ্য প্রিন্ট
`-C -`	`--continue-at`	ডাউনলোড resume
`-A AGENT`	`--user-agent`	User-Agent বদলাও
`-e URL`	`--referer`	Referer হেডার
`-b` / `-c`	`--cookie` / `--cookie-jar`	cookie পাঠাও / সেভ করো
`-x PROXY`	`--proxy`	proxy দিয়ে যাও
`-m SEC`	`--max-time`	মোট সময়সীমা
`--connect-timeout SEC`		সংযোগ স্থাপনের সময়সীমা
`--retry N`		ব্যর্থ হলে N বার চেষ্টা
`-4` / `-6`		IPv4 / IPv6
`--resolve host:port:IP`		DNS বাদ দিয়ে নির্দিষ্ট IP-তে পাঠাও (নতুন সার্ভার টেস্টে দারুণ)
`-Z`	`--parallel`	একাধিক URL একসাথে (7.66+)

```
# DNS না বদলেই নতুন সার্ভারে ওয়েবসাইট টেস্ট
$ curl --resolve example.com:443:203.0.113.10 https://example.com -I

# স্ক্রিপ্টে: সাইট ডাউন হলে exit code non-zero
$ curl -fsS -m 10 https://example.com/health > /dev/null && echo "UP" || echo "DOWN"
UP
```

৩৭.১১ `wget` — বিশ্বস্ত ডাউনলোডার

`curl` সব কাজের ছুরি, আর `wget` বিশেষজ্ঞ ডাউনলোডার। এর বিশেষত্ব হলো ডিফল্টে ফাইলে সেভ করা, নেটওয়ার্ক কাটলে নিজে আবার চেষ্টা করা, আর পুরো ওয়েবসাইট recursive ভাবে নামাতে পারা।

```
$ wget https://example.com/file.zip
--2026-11-12 10:30:01--  https://example.com/file.zip
Resolving example.com (example.com)... 93.184.215.14
Connecting to example.com (example.com)|93.184.215.14|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10485760 (10M) [application/zip]
Saving to: ‘file.zip’

file.zip            100%[===================>]  10.00M  2.31MB/s    in 4.3s

2026-11-12 10:30:06 (2.31 MB/s) - ‘file.zip’ saved [10485760/10485760]
```

অপশন	কাজ
`-O FILE`	নির্দিষ্ট নামে সেভ (`-O -` = stdout-এ) — **খেয়াল করো: curl-এর `-O` -এর মানে আলাদা!**
`-P DIR`	কোন ফোল্ডারে সেভ
`-c`	আধা-নামানো ফাইল resume
`-b`	ব্যাকগ্রাউন্ডে চালাও (লগ `wget-log` -এ)
`-q`	চুপচাপ
`-i FILE`	ফাইলে লেখা URL-গুলো একে একে নামাও
`-t N`	কতবার চেষ্টা (`0` = অসীম)
`-T SEC`	timeout
`--limit-rate=500k`	স্পিড সীমিত করো
`-r`	recursive
`-l N`	recursive-এর গভীরতা
`-np`	parent ডিরেক্টরিতে উঠবে না
`-k`	ডাউনলোড করা পেজের লিংক লোকাল করে দাও
`-p`	পেজ দেখাতে যা লাগে (ছবি, CSS) সব নামাও
`-m` / `--mirror`	পুরো সাইট mirror (`-r -N -l inf --no-remove-listing`)
`--spider`	ডাউনলোড না করে শুধু আছে কি না দেখো
`--user` / `--password`	লগইন
`--no-check-certificate`	TLS যাচাই বাদ (শুধু টেস্টে)

```
$ wget -c https://releases.ubuntu.com/24.04/ubuntu-24.04.1-desktop-amd64.iso
$ wget -P ~/Downloads -i links.txt
$ wget --mirror -k -p -np https://docs.example.com/guide/
$ wget --spider https://example.com/file.zip && echo "ফাইল আছে"
$ wget -qO- https://api.ipify.org; echo     # ফাইলে না রেখে টার্মিনালে দেখাও
```

> 📝 **মনে রাখো:** Fedora 40 থেকে `wget` কমান্ডটা আসলে **wget2** (নতুন সংস্করণ, একাধিক সংযোগে দ্রুত)। প্রায় সব অপশন একই, তবে আউটপুট দেখতে কিছুটা আলাদা হতে পারে।

`curl` বনাম `wget`

বিষয়	`curl`	`wget`
ডিফল্ট আউটপুট	stdout (টার্মিনাল)	ফাইল
redirect	`-L` দিতে হয়	নিজেই অনুসরণ করে
recursive/mirror	না	হ্যাঁ

বিষয়	`curl`	`wget`
API (POST/PUT/হেডার)	খুব শক্তিশালী	সীমিত
প্রোটোকল	অনেক (HTTP, FTP, SFTP, SMTP, ...)	মূলত HTTP/HTTPS/FTP
লাইব্রেরি	`libcurl` (অসংখ্য প্রোগ্রাম ব্যবহার করে)	না

৩৭.১২ `nc` (netcat) — নেটওয়ার্কের "TCP/IP সুইস আর্মি নাইফ"

`nc` যেকোনো TCP বা UDP port-এ কাঁচা সংযোগ খোলে আর কীবোর্ড থেকে লেখা পাঠায়, বা কোনো port-এ শুনে বসে থাকে। কোনো প্রোটোকল নিজে বোঝে না, শুধু byte আদান-প্রদান করে। তাই যেকোনো সার্ভিস হাতে-কলমে পরীক্ষা করা যায়।

> 📝 **মনে রাখো (ভ্যারিয়েন্টের ঝামেলা):** তিন রকম netcat আছে, অপশনেও কিছু পার্থক্য:
>
> - **OpenBSD netcat** (`netcat-openbsd`): Ubuntu/Debian-এর ডিফল্ট। listen করতে `nc -l 1234` ।
> - **ncat** (Nmap প্রকল্পের, `nmap-ncat`): Fedora/RHEL-এর ডিফল্ট, `nc` নামেও চলে। `nc -l 1234` আর `nc -l -p 1234` দুটোই চলে।
> - **traditional netcat** (`netcat-traditional`): পুরনো; listen করতে `nc -l -p 1234` ।
>
> সন্দেহ হলে `nc -h` বা `man nc` দেখো।

অপশন	কাজ
`-l`	listen মোড (সার্ভার হয়ে বসে থাকো)
`-p PORT`	source/listen port (variant ভেদে)
`-z`	কোনো ডেটা না পাঠিয়ে শুধু port খোলা কি না দেখো (scan; খুব পুরনো ncat সংস্করণে নেই, তখন `nmap` ব্যবহার করো)
`-v`	বিস্তারিত (সফল/ব্যর্থ জানাবে)
`-u`	TCP-র বদলে UDP
`-w SEC`	timeout
`-n`	DNS lookup করবে না
`-k`	একটা সংযোগ শেষ হলেও listen চালিয়ে যাও (OpenBSD ও ncat)
`-N`	stdin শেষ হলে (EOF) সংযোগ বন্ধ করো (OpenBSD)
`-4` / `-6`	IPv4 / IPv6

১. port খোলা কি না (সবচেয়ে বেশি ব্যবহৃত)

```
$ nc -zv 192.168.1.10 22
Connection to 192.168.1.10 22 port [tcp/ssh] succeeded!

$ nc -zv 192.168.1.10 3306
nc: connect to 192.168.1.10 port 3306 (tcp) failed: Connection refused

$ nc -zv -w 3 example.com 8080
nc: connect to example.com port 8080 (tcp) timed out: Operation now in progress

$ nc -zv 192.168.1.10 20-25        # একটা রেঞ্জ (OpenBSD)
```

"refused" আর "timed out"-এর পার্থক্য বোঝা খুব জরুরি:

ফলাফল	ভেতরে কী হয়েছে	মানে
`succeeded`	SYN → SYN-ACK পাওয়া গেছে	port খোলা, কেউ listen করছে
`Connection refused`	SYN → **RST** ফেরত এসেছে	হোস্টে পৌঁছানো গেছে, কিন্তু ওই port-এ কেউ listen করছে না (সার্ভিস বন্ধ, বা শুধু `127.0.0.1`-এ listen করছে)
`timed out`	SYN → কোনো উত্তর নেই	পথে firewall প্যাকেট চুপচাপ ফেলে দিচ্ছে (DROP), বা হোস্টই নেই
`No route to host`	ICMP host unreachable, বা firewall REJECT করছে	route/ARP সমস্যা বা firewall reject

২. দুই কম্পিউটারের মধ্যে চ্যাট

```
# কম্পিউটার A (সার্ভার, IP 192.168.1.10):
$ nc -l 5000

# কম্পিউটার B (ক্লায়েন্ট):
$ nc 192.168.1.10 5000
হ্যালো!            ← B-তে লিখলে A-তে দেখা যাবে, উল্টোটাও
```

বের হতে `Ctrl+C` (বা ক্লায়েন্টে `Ctrl+D` দিয়ে EOF পাঠাও)।

৩. ফাইল পাঠানো

```
# গ্রহণকারী (A):
$ nc -l 5000 > received.tar.gz

# প্রেরক (B):
$ nc -N 192.168.1.10 5000 < backup.tar.gz      # -N: শেষ হলে বন্ধ (OpenBSD)
```

> ⚠️ **সতর্কতা:** `nc` সব কিছু **এনক্রিপশন ছাড়া** পাঠায়। বিশ্বস্ত লোকাল নেটওয়ার্কের বাইরে গোপন কিছু পাঠাতে `scp` / `rsync` over SSH ব্যবহার করো (অধ্যায় ৩৮, অধ্যায় ৩৫)। কাজ শেষে listener বন্ধ করতে ভুলো না।

৪. হাতে HTTP বা SMTP কথা বলা

```
$ printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' | nc
example.com 80
HTTP/1.1 200 OK
Content-Type: text/html
...
```

এভাবে বোঝা যায় HTTP আসলে শুধু সাধারণ টেক্সট লাইন। (HTTPS-এর জন্য TLS লাগে, তখন `openssl s_client -connect example.com:443` ব্যবহার করো।)

৫. একটা সাধারণ সার্ভার বানিয়ে firewall টেস্ট

নতুন একটা port firewall-এ খুলেছ, কিন্তু অ্যাপ এখনো বসাওনি। তাহলে খোলা ঠিক হলো কি না কীভাবে বুঝবে?

```
# সার্ভারে:
$ nc -lk 8080

# অন্য মেশিন থেকে:
$ nc -zv server-ip 8080     # succeeded হলে firewall ঠিক আছে
```

UDP টেস্ট: সার্ভারে `nc -lu 5353`, ক্লায়েন্টে `echo hi | nc -u -w1 server-ip 5353`। (UDP-তে কোনো handshake নেই, তাই `-zv` প্রায় সবসময় "succeeded" বলে। আসল ডেটা গেল কি না দেখে নিশ্চিত হও।)

৩৭.১৩ `tcpdump` — প্যাকেট নিজের চোখে দেখা

বাকি সব টুল যখন বলে "কিছু ঠিক নেই" কিন্তু কেন তা বলে না, তখন `tcpdump` তারের ভেতর দিয়ে যাওয়া আসল প্যাকেট দেখায়। এটা হলো নেটওয়ার্কের **এক্স-রে**।

ভেতরে কীভাবে কাজ করে

```
নেটওয়ার্ক কার্ড ──► কার্নেল নেটওয়ার্ক স্ট্যাক ──► অ্যাপ্লিকেশন
                          │
                          │  AF_PACKET socket (প্রতিটা প্যাকেটের কপি)
                          ▼
                    BPF ফিল্টার (কার্নেলের ভেতরে চলে, যেমন "port 80")
                          │  শুধু মিলে যাওয়া প্যাকেট
                          ▼
                    libpcap ──► tcpdump (প্রিন্ট বা .pcap ফাইলে লেখা)
```

ফিল্টারটা (BPF, Berkeley Packet Filter) কার্নেলের ভেতরেই চলে, তাই অপ্রয়োজনীয় প্যাকেট user space পর্যন্ত আসে না। ফলে ব্যস্ত সার্ভারেও tcpdump চালানো যায়। কাঁচা প্যাকেট পড়তে root (`CAP_NET_RAW`) লাগে।

বেসিক ব্যবহার

```
$ sudo tcpdump -D                  # কোন কোন interface আছে
1.enp3s0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
...

$ sudo tcpdump -i enp3s0 -nn -c 5 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp3s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:40:01.123456 IP 192.168.1.105 > 8.8.8.8: ICMP echo request, id 7, seq 1, length 64
10:40:01.145678 IP 8.8.8.8 > 192.168.1.105: ICMP echo reply, id 7, seq 1, length 64
...
5 packets captured
5 packets received by filter
0 packets dropped by kernel
```

(অন্য টার্মিনালে `ping 8.8.8.8` চালালে এগুলো দেখা যাবে।)

অপশন	কাজ
`-i IFACE`	কোন interface (`-i any` = সব)
`-D`	interface তালিকা
`-n`	IP-কে নামে বদলাবে না
`-nn`	IP আর port কোনোটাই নামে বদলাবে না (সবচেয়ে দ্রুত, সবচেয়ে প্রচলিত)
`-c N`	N টা প্যাকেট ধরে থামো
`-v` / `-vv` / `-vvv`	বেশি বিস্তারিত (TTL, IP id, checksum)
`-A`	প্যাকেটের ভেতরের ডেটা ASCII-তে (HTTP দেখতে কাজে লাগে)
`-X`	hex + ASCII দুটোই
`-e`	Ethernet (MAC) header দেখাও
`-w FILE.pcap`	প্যাকেট ফাইলে সেভ (পরে Wireshark-এ খোলা যায়)
`-r FILE.pcap`	সেভ করা ফাইল পড়ো
`-s N`	প্রতিটা প্যাকেটের কত byte ধরবে (ডিফল্ট 262144, মানে পুরোটা)
`-q`	সংক্ষিপ্ত আউটপুট
`-t`	timestamp দেখাবে না
`-l`	line-buffered (পাইপ দিয়ে `grep`-এ পাঠালে লাগে)

ফিল্টার (BPF expression)

ফিল্টার	মানে
`host 192.168.1.20`	এই হোস্টের যাওয়া বা আসা

ফিল্টার	মানে
`src host 10.0.0.5` / `dst host 10.0.0.5`	শুধু উৎস / শুধু গন্তব্য
`net 192.168.1.0/24`	পুরো subnet
`port 443`	port 443 (উৎস বা গন্তব্য)
`src port 53` / `dst port 80`	নির্দিষ্ট দিকের port
`portrange 8000-8100`	port রেঞ্জ
`tcp` / `udp` / `icmp` / `arp`	প্রোটোকল
`and` / `or` / `not` (`&&` `	
`'tcp[tcpflags] & tcp-syn != 0'`	শুধু SYN flag থাকা প্যাকেট

```
# DNS প্রশ্ন আর উত্তর দেখো
$ sudo tcpdump -i any -nn port 53
10:41:10.001 IP 192.168.1.105.48211 > 192.168.1.1.53: 31415+ A? example.com. (29)
10:41:10.019 IP 192.168.1.1.53 > 192.168.1.105.48211: 31415 1/0/0 A 93.184.215.14 (45)

# একটা হোস্টের সাথে HTTP ট্রাফিক, SSH বাদ দিয়ে
$ sudo tcpdump -i enp3s0 -nn 'host 192.168.1.20 and not port 22'

# HTTP (এনক্রিপ্ট না হওয়া) রিকোয়েস্টের লেখা পড়ো
$ sudo tcpdump -i any -nn -A 'tcp port 80'

# পরে বিশ্লেষণের জন্য ফাইলে রাখো
$ sudo tcpdump -i enp3s0 -nn -w /tmp/capture.pcap 'port 443'
$ tcpdump -nn -r /tmp/capture.pcap | head
```

> ⚠️ **সতর্কতা:** রিমোট সার্ভারে SSH দিয়ে ঢুকে ফিল্টার ছাড়া `tcpdump` চালালে সে নিজের SSH ট্রাফিকও ধরবে, যা আবার আউটপুট হয়ে SSH দিয়ে আসবে… এক অনন্ত লুপে স্ক্রিন ভরে যাবে। সবসময় `not port 22` যোগ করো।

> ⚠️ **সতর্কতা (আইনি):** অন্যের নেটওয়ার্ক বা অন্যের ট্রাফিক অনুমতি ছাড়া capture করা বেশিরভাগ দেশে অপরাধ। বাংলাদেশে ডিজিটাল/সাইবার নিরাপত্তা আইনেও এর শাস্তি আছে। শুধু নিজের মেশিন বা যেখানে লিখিত অনুমতি আছে সেখানে ব্যবহার করো। `.pcap` ফাইলে পাসওয়ার্ড বা ব্যক্তিগত তথ্য থাকতে পারে, তাই সাবধানে রাখো।

TCP flag পড়া: 3-way handshake নিজের চোখে

```
$ sudo tcpdump -i any -nn 'tcp port 80 and host 93.184.215.14'
10:42:00.100 IP 192.168.1.105.51500 > 93.184.215.14.80: Flags [S], seq 1000, win 64240,
length 0
10:42:00.250 IP 93.184.215.14.80 > 192.168.1.105.51500: Flags [S.], seq 5000, ack 1001,
win 65535, length 0
```

```
10:42:00.251 IP 192.168.1.105.51500 > 93.184.215.14.80: Flags [.], ack 5001, win 502,
length 0
10:42:00.252 IP 192.168.1.105.51500 > 93.184.215.14.80: Flags [P.], seq 1001:1076, ack
5001, length 75: HTTP: GET / HTTP/1.1
...
10:42:00.600 IP 192.168.1.105.51500 > 93.184.215.14.80: Flags [F.], seq 1076, ack 6300,
length 0
```

tcpdump flag	মানে
`[S]`	SYN: "সংযোগ করতে চাই"
`[S.]`	SYN-ACK: "ঠিক আছে, আমিও রাজি" (`.` = ACK)
`[.]`	শুধু ACK
`[P.]`	PUSH + ACK: ডেটা পাঠাচ্ছে
`[F.]`	FIN: "আমার কথা শেষ"
`[R]` / `[R.]`	RST: "সংযোগ বাতিল / এখানে কেউ নেই"

> 💡 **টিপস:** `[S]` যাচ্ছে কিন্তু কোনো উত্তর নেই = firewall DROP বা হোস্ট নেই। `[S]` যাচ্ছে আর `[R.]` আসছে = port বন্ধ (refused)। বারবার একই `[S]` পাঠাচ্ছে = retransmission, উত্তর হারিয়ে যাচ্ছে।

> 💡 **টিপস:** GUI-তে আরও আরামে বিশ্লেষণ করতে চাইলে **Wireshark** ইনস্টল করো (`sudo apt install wireshark`), আর `tcpdump -w` দিয়ে বানানো `.pcap` ফাইলটা খোলো। শুধু টার্মিনাল থাকলে `tshark` আছে।

৩৭.১৪ `nmap` পরিচিতি — নিজের সিস্টেম স্ক্যান

`nmap` (Network Mapper) একটা শক্তিশালী নেটওয়ার্ক স্ক্যানার: কোন হোস্ট বেঁচে আছে, কোন port খোলা, কোন সার্ভিস ও কোন সংস্করণ চলছে।

> ⚠️ **সতর্কতা (খুব গুরুত্বপূর্ণ):** অনুমতি ছাড়া অন্যের সার্ভার, কলেজ/অফিস নেটওয়ার্ক বা ইন্টারনেটের কোনো আইপি স্ক্যান করা অনৈতিক, অনেক দেশে বেআইনি, আর ISP তোমার সংযোগ বন্ধ করে দিতে পারে। **শুধু নিজের মেশিন (`localhost`), নিজের বাসার ল্যাব, বা যেখানে লিখিত অনুমতি আছে সেখানে চালাও।** অনুশীলনের জন্য Nmap প্রকল্প নিজেই `scanme.nmap.org` নামের একটা সার্ভার রেখেছে, যেটা হালকাভাবে স্ক্যান করার অনুমতি দেওয়া আছে।

```
$ nmap localhost
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-11-12 10:50 +06
Nmap scan report for localhost (127.0.0.1)
```

```
Host is up (0.000080s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
631/tcp  open  ipp
3306/tcp open  mysql

Nmap done: 1 IP address (1 host up) scanned in 0.05 seconds
```

ডিফল্টভাবে `nmap` সবচেয়ে প্রচলিত ১০০০টা TCP port দেখে।

অপশন	কাজ
`-sT`	TCP connect scan (পূর্ণ সংযোগ; root ছাড়া এটাই ডিফল্ট)
`-sS`	SYN scan ("half-open"; root লাগে, root হলে এটাই ডিফল্ট)
`-sU`	UDP scan (ধীর, root লাগে)
`-sn`	শুধু কোন হোস্ট বেঁচে আছে (port scan নেই; আগে নাম ছিল `-sP`)
`-p 22,80,443`	নির্দিষ্ট port
`-p 1-1024` / `-p-`	রেঞ্জ / সব ৬৫৫৩৫ port
`-F`	দ্রুত: শুধু শীর্ষ ১০০টা port
`-sV`	সার্ভিসের সংস্করণ বের করো
`-O`	অপারেটিং সিস্টেম অনুমান (root)
`-A`	aggressive: `-sV -O -sC --traceroute` একসাথে
`-Pn`	হোস্ট বেঁচে আছে কি না যাচাই না করেই স্ক্যান
`-T0` … `-T5`	গতি (`-T4` দ্রুত; বেশি দ্রুত মানে বেশি শব্দ)
`-oN file` / `-oX file`	ফলাফল সাধারণ লেখায় / XML-এ সেভ

```
$ nmap -sV -p 22,80 localhost
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    nginx 1.24.0 (Ubuntu)

# নিজের বাসার নেটওয়ার্কে কোন কোন ডিভাইস চালু (শুধু নিজের নেটওয়ার্ক!)
$ sudo nmap -sn 192.168.1.0/24
Nmap scan report for _gateway (192.168.1.1)
Host is up (0.0011s latency).
MAC Address: 08:00:27:AA:BB:CC (Router Vendor)
...
```

port-এর অবস্থা বোঝো:

STATE	মানে
`open`	কেউ listen করছে

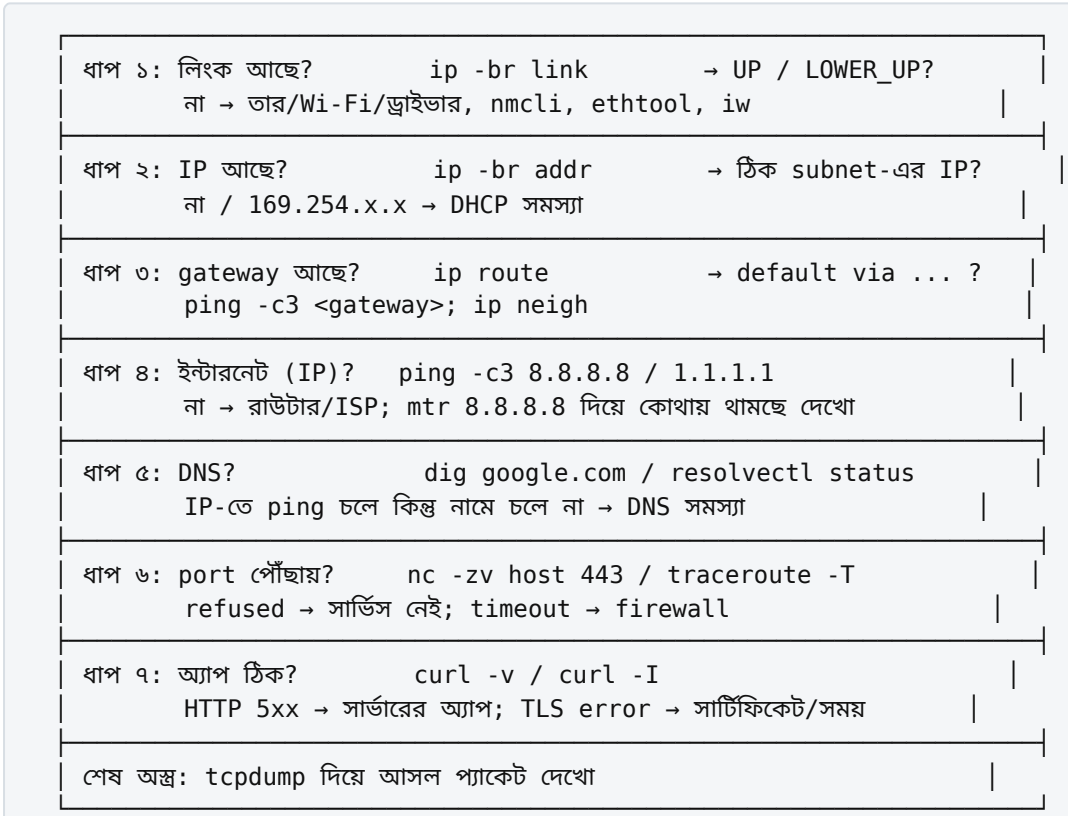
STATE	মানে
`closed`	পৌঁছানো গেছে, কেউ listen করছে না (RST এসেছে)
`filtered`	firewall উত্তর আটকে দিচ্ছে, nmap নিশ্চিত হতে পারছে না
`open|filtered`	UDP-তে প্রায়ই; খোলা নাকি filtered বোঝা যাচ্ছে না

> 💡 **টিপস:** নিজের সার্ভার বাইরে থেকে কেমন দেখায় তা জানতে **অন্য একটা মেশিন থেকে** নিজের সার্ভারকে `nmap` করো। `localhost` থেকে স্ক্যান করলে firewall পার হতে হয় না, তাই সব খোলা দেখায়, যা বাস্তব চিত্র নয়।

৩৭.১৫ নেটওয়ার্ক সমস্যা সমাধানের ধাপে ধাপে পদ্ধতি

"ইন্টারনেট চলছে না" খুব অস্পষ্ট একটা অভিযোগ। পেশাদাররা অনুমান করে এলোমেলো কমান্ড চালান না; **নিচ থেকে ওপরে** (bottom-up), একটা একটা লেয়ার পরীক্ষা করেন। যে ধাপে প্রথম ব্যর্থ হবে, সমস্যা সেখানেই বা তার ঠিক নিচে।

> 💡 **উপমা:** বাতি জ্বলছে না? আগে দেখো মেইন সুইচ চালু কি না, তারপর তার, তারপর হোল্ডার, শেষে বাল্ব। একেবারে নতুন বাল্ব কিনে আনা প্রথম কাজ নয়।



ধাপগুলো বিস্তারিত

ধাপ ১ — লিংক:

```
$ ip -br link
$ nmcli device status          # NetworkManager থাকলে (Desktop, Fedora, RHEL)
DEVICE   TYPE      STATE         CONNECTION
enp3s0   ethernet  connected     Wired connection 1
wlp2s0   wifi      disconnected  --
```

`NO-CARRIER` / `DOWN` হলে: তার লাগানো? Wi-Fi কানেক্টেড? `rfkill list` দেখো (Wi-Fi সফটওয়্যারে বা হার্ডওয়্যার সুইচে blocked কি না)। ড্রাইভার লোড হয়েছে কি না দেখতে `sudo dmesg | grep -i -E 'eth|wlan|firmware'` (অধ্যায় ৪৭)।

ধাপ ২ — IP:

```
$ ip -4 -br addr
enp3s0           UP             169.254.23.7/16
```

`169.254.x.x` (link-local / APIPA) বা কোনো IP না থাকা মানে DHCP সার্ভার থেকে IP পাওয়া যায়নি। NetworkManager-এ সংযোগ আবার চালু করো: `nmcli connection up "Wired connection 1"`, বা লগ দেখো: `journalctl -u NetworkManager -b --since "10 min ago"` (অধ্যায় ২৮)।

ধাপ ৩ — gateway:

```
$ ip route | grep default
default via 192.168.1.1 dev enp3s0 proto dhcp metric 100
$ ping -c 3 192.168.1.1
$ ip neigh show 192.168.1.1
```

default route নেই → `Network is unreachable` আসবে। gateway-এর ARP `FAILED` → লেয়ার ২ সমস্যা বা ভুল subnet।

ধাপ ৪ — ইন্টারনেট (IP দিয়ে, DNS ছাড়া):

```
$ ping -c 3 8.8.8.8
$ mtr -rwn -c 20 8.8.8.8
```

gateway পর্যন্ত ঠিক, কিন্তু এর পরে সব হারাচ্ছে → রাউটারের WAN/ISP সমস্যা।

ধাপ ৫ — DNS:

```
$ ping -c 1 google.com
ping: google.com: Temporary failure in name resolution
$ dig google.com +short                  # কিছু আসে না
$ dig @8.8.8.8 google.com +short         # এটায় আসে → লোকাল DNS সার্ভার সমস্যা
```

```
142.250.193.110
$ resolvectl status
```

`8.8.8.8` দিয়ে কাজ করে কিন্তু ডিফল্ট দিয়ে করে না → রাউটারের DNS বা resolved-এর কনফিগারেশন ঠিক করো (অস্থায়ীভাবে `resolvectl dns enp3s0 1.1.1.1 8.8.8.8`)।

ধাপ ৬ — নির্দিষ্ট সার্ভিসের port:

```
$ nc -zv -w 5 myserver.com 443
$ sudo traceroute -T -p 443 -n myserver.com
```

নিজে সার্ভার হলে উল্টো দিক থেকেও দেখো: সার্ভারে `sudo ss -tlnp | grep 443` দিয়ে সার্ভিস listen করছে কি না, আর কোন IP-তে (`127.0.0.1` না `0.0.0.0`)। তারপর firewall দেখো: `sudo ufw status` / `sudo firewall-cmd --list-all` / `sudo nft list ruleset` (অধ্যায় ৩৯)। ক্লাউডে থাকলে প্রোভাইডারের security group-ও দেখো।

ধাপ ৭ — অ্যাপ্লিকেশন:

```
$ curl -v https://myserver.com/
$ curl -o /dev/null -s -w "%{http_code} %{time_total}\n" https://myserver.com/
502 0.031
```

`502` → nginx চলছে কিন্তু পেছনের অ্যাপ (PHP-FPM, Node ইত্যাদি) সাড়া দিচ্ছে না: `systemctl status`, `journalctl -u` দেখো (অধ্যায় ২৬, অধ্যায় ৪০)। `certificate has expired` বা `not yet valid` → সার্টিফিকেট নবায়ন করো, অথবা সিস্টেমের ঘড়ি ভুল কি না দেখো (`timedatectl`)।

সাধারণ উপসর্গ → সম্ভাব্য কারণ

উপসর্গ	প্রথমে কী সন্দেহ করবে	কোন কমান্ড
IP দিয়ে চলে, নাম দিয়ে চলে না	DNS	`dig`, `resolvectl status`, `/etc/resolv.conf`
একটা নির্দিষ্ট সাইট ভুল জায়গায় যাচ্ছে	`/etc/hosts`, DNS cache	`getent hosts site`, `resolvectl flush-caches`
ছোট পেজ খোলে, বড় পেজ/ফাইল আটকে যায় (বিশেষ করে VPN-এ)	MTU	`ping -M do -s 1472`, `tracepath`
মাঝে মাঝে ধীর, প্যাকেট হারায়	Wi-Fi সিগন্যাল, তার, ISP	`mtr`, `ip -s link`, `iw dev wlp2s0 link`
"Connection refused"	সার্ভিস বন্ধ বা ভুল IP-তে listen করছে	`ss -tlnp`, `systemctl status`
"Connection timed out"	firewall DROP	`nc -zv`, `ufw` / `firewalld`, cloud security group

উপসর্গ	প্রথমে কী সন্দেহ করবে	কোন কমান্ড
"Address already in use"	অন্য প্রসেস port ধরে আছে	`ss -tlnp 'sport = :PORT'`, `lsof -i :PORT`
SSL/TLS error	সার্টিফিকেটের মেয়াদ, ভুল সময়, নাম না মেলা	`curl -v`, `timedatectl`, `openssl s_client`
হঠাৎ একটা নতুন হোস্টে যাওয়া যাচ্ছে না, বাকিগুলোয় যায়	route/ARP	`ip route get IP`, `ip neigh`

> 📝 **মনে রাখো:** যা পরীক্ষা করছ তা লিখে রাখো (কোন ধাপে কী ফল এল)। ISP, হোস্টিং সাপোর্ট বা সহকর্মীকে জানাতে গেলে `mtr -rwn` রিপোর্ট আর `curl -v` আউটপুট দিলে সমস্যা অনেক দ্রুত সমাধান হয়।

৩৭.১৬ কীবোর্ড শর্টকাট (এই অধ্যায়ের টুলগুলোর জন্য)

কী	কোথায়	কাজ
`Ctrl+C`	`ping`, `tcpdump`, `nc`, `ip monitor`, `traceroute`	থামাও (ping/tcpdump শেষে সারাংশ দেখায়)
`Ctrl+\`	`ping` (iputils)	না থামিয়ে এখন পর্যন্ত পরিসংখ্যান দেখাও
`Ctrl+D`	`nc` ক্লায়েন্ট, `nslookup` ইন্টারঅ্যাকটিভ	EOF পাঠাও / বের হও
`Ctrl+Z`	যেকোনো চলমান কমান্ড	সাময়িক থামিয়ে ব্যাকগ্রাউন্ডে রাখো (`fg` / `bg`, অধ্যায় ২৫)
`q`	`mtr`, `less`-এ খোলা আউটপুট	বের হও
`p` / `Space`	`mtr`	pause / resume
`r`	`mtr`	পরিসংখ্যান রিসেট
`n`	`mtr`	DNS নাম চালু/বন্ধ
`d`	`mtr`	ডিসপ্লে মোড বদল
`Ctrl+R`	shell	আগে চালানো লম্বা `curl` / `tcpdump` কমান্ড history থেকে খুঁজে বের করো (অধ্যায় ৬)
`Tab`	shell	`ip` সাব-কমান্ড আর interface নামের auto-complete (bash-completion থাকলে)

> 💡 **টিপস:** কোনো কমান্ডের আউটপুট লাইভ আপডেট দেখতে `watch` ব্যবহার করো: `watch -n 1 'ss -s'` বা `watch -n 2 'ip -s link show enp3s0'`। বের হতে `Ctrl+C`।

হাতে-কলমে অনুশীলন

> নিচের সব ল্যাব শুধু তোমার নিজের কম্পিউটার আর নিজের নেটওয়ার্কে করবে।

ল্যাব ১: নিজের নেটওয়ার্কের পরিচয়পত্র বানাও

1. `ip -br -c addr` চালাও। তোমার প্রধান interface-এর নাম, IP আর `/` সংখ্যা লিখে রাখো।
2. `ip -br link` দিয়ে MAC address বের করো।
3. `ip route | grep default` দিয়ে gateway বের করো।
4. `ip route get 1.1.1.1` চালাও; কোন interface আর কোন source IP দিয়ে যাবে মিলিয়ে দেখো।
5. `ping -c 2 <gateway>` চালিয়ে তারপর `ip neigh show` দেখো; gateway-এর MAC আর state লিখে রাখো।
6. `resolvectl status` থেকে আসল DNS সার্ভার বের করো।
7. `hostname`, `hostname -I` আর `hostnamectl` চালাও।
8. `curl -s https://api.ipify.org; echo` দিয়ে পাবলিক IP বের করো। লোকাল IP থেকে আলাদা কেন? (উত্তর: NAT, অধ্যায় ৩৬)

ল্যাব ২: রাস্তা মাপো

1. `ping -c 5 8.8.8.8` আর `ping -c 5 google.com`। গড় RTT তুলনা করো।
2. `tracepath -n 8.8.8.8` চালাও। কতটা hop? pmtu কত?
3. `mtr -rwn -c 30 1.1.1.1` চালাও। কোনো hop-এ loss আছে? সেটা কি শেষ পর্যন্ত টিকে আছে, নাকি শুধু মাঝখানে? আসল সমস্যা কি না সিদ্ধান্ত নাও।
4. `ping -c 2 -M do -s 1472 8.8.8.8` আর `-s 1473` দিয়ে MTU-র সীমা নিজে প্রমাণ করো।

ল্যাব ৩: একটা সার্ভার বানাও, খুঁজে বের করো, টেস্ট করো

1. টার্মিনাল ১-এ একটা ছোট ওয়েব সার্ভার চালাও:

```
$ mkdir -p ~/netlab && cd ~/netlab && echo "<h1>Hello Linux</h1>" > index.html
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

2. টার্মিনাল ২-এ খুঁজে বের করো: `ss -tlnp 'sport = :8000'` আর `lsof -i :8000`। কোন Local Address-এ listen করছে?
3. `curl -I http://localhost:8000/`, তারপর `curl -s -w "\n%{http_code}\n" http://localhost:8000/nope` চালাও। status code কী এল?
4. `nc -zv localhost 8000` (succeeded) আর `nc -zv localhost 8001` (refused) দেখো, পার্থক্য ব্যাখ্যা করো।

5. টার্মিনাল ৩-এ `sudo tcpdump -i lo -nn -A 'tcp port 8000'` চালিয়ে আবার `curl http://localhost:8000/` করো। `[S]`, `[S.]`, `[.]`, `[P.]` আর `GET / HTTP/1.1` লেখাটা খুঁজে বের করো।
6. সার্ভার বন্ধ করে (`Ctrl+C`) আবার `curl` চালাও। কী error এল? (`Connection refused`)

ল্যাব ৪: DNS গোয়েন্দা

1. `dig example.com` চালাও; status, TTL আর SERVER লিখে রাখো। দুবার চালাও। দ্বিতীয়বার `Query time` আর TTL কেন কম?
2. `dig gmail.com MX +short`, `dig google.com NS +short`, `dig -x 1.1.1.1 +short`।
3. `dig nosuchdomain-xyz-12345.com` চালাও; status কী? (`NXDOMAIN`)
4. `sudo` দিয়ে `/etc/hosts`-এ `127.0.0.1 mytest.local` লাইন যোগ করো। তারপর `getent hosts mytest.local` আর `dig mytest.local +short` চালাও। পার্থক্যের কারণ কী? কাজ শেষে লাইনটা মুছে দাও।
5. `sudo tcpdump -i any -nn port 53` চালু রেখে অন্য টার্মিনালে `dig @8.8.8.8 example.com` চালাও আর প্রশ্ন-উত্তর প্যাকেট দেখো।

ল্যাব ৫: netcat চ্যাট ও ফাইল ট্রান্সফার (একই মেশিনে দুই টার্মিনালে)

```
# টার্মিনাল ১
$ nc -l 5000
# টার্মিনাল ২
$ nc localhost 5000
```

দুই দিক থেকে লিখে দেখো। তারপর ফাইল পাঠাও:

```
# টার্মিনাল ১
$ nc -l 5001 > copy.txt
# টার্মিনাল ২
$ nc -N localhost 5001 < /etc/os-release
# যাচাই
$ diff /etc/os-release copy.txt && echo "একদম এক!"
```

(Fedora-র `ncat`-এ `-N` না চললে `--send-only` ব্যবহার করো।)

ল্যাব ৬: নিজের মেশিনের নিরাপত্তা অডিট

1. `sudo ss -tulpn` চালাও। প্রতিটা listening port-এর জন্য লেখো: কোন প্রোগ্রাম, `127.0.0.1` না `0.0.0.0`, তোমার দরকার কি না।
2. `nmap localhost` আর `sudo nmap -sV localhost` চালিয়ে ফল মেলাও।
3. দরকার নেই এমন সার্ভিস পেলে নোট করো। অধ্যায় ২৬-এ শিখবে কীভাবে `systemctl disable --now` দিয়ে বন্ধ করতে হয়, আর অধ্যায় ৪৫-এ পুরো হার্ডেনিং।

ল্যাব ৭: সমস্যা বানাও, সমাধান করো (VM বা নিজের ল্যাপটপে, রিমোট সার্ভারে নয়!)

1. DNS ভাঙো: `sudo resolvectl dns enp3s0 192.0.2.1` (192.0.2.0/24 ডকুমেন্টেশনের জন্য সংরক্ষিত, সেখানে কোনো DNS নেই)। এবার ৩৭.১৫-এর ধাপগুলো দিয়ে সমস্যা খুঁজে বের করো। ঠিক করতে: `sudo resolvectl revert enp3s0` (বা `nmcli connection up "<নাম>"`)।
2. route ভাঙো: `sudo ip route del default`। `ping 8.8.8.8` কী বলে? `ip route` দিয়ে কারণ বের করো। ঠিক করতে: `sudo ip route add default via <তোমার gateway>` বা NetworkManager দিয়ে সংযোগ আবার চালু করো।

সারসংক্ষেপ

কাজ	কমান্ড
IP দেখা (সংক্ষেপে, রঙিন)	`ip -br -c addr`
interface চালু/বন্ধ	`sudo ip link set dev IFACE up/down`
interface পরিসংখ্যান	`ip -s link show IFACE`
অস্থায়ী IP যোগ/মোছা	`sudo ip addr add/del 192.168.1.50/24 dev IFACE`
routing table	`ip route`
কোন পথে যাবে	`ip route get 8.8.8.8`
default gateway বসানো	`sudo ip route add default via GW`
ARP/neighbor টেবিল	`ip neigh`
লাইভ পরিবর্তন	`ip monitor`
hostname দেখা/বদলানো	`hostname -I`, `sudo hostnamectl set-hostname NAME`
হোস্ট বেঁচে আছে কি না	`ping -c 4 HOST`
MTU পরীক্ষা	`ping -M do -s 1472 HOST`
পথ দেখা	`traceroute -n HOST`, `tracepath HOST`
লাইভ পথ + loss	`mtr HOST`, `mtr -rwn -c 100 HOST`
listening port ও প্রসেস	`sudo ss -tulpn`
established সংযোগ	`ss -tn state established`
socket সারাংশ	`ss -s`
port কে ধরে আছে	`sudo lsof -i :PORT`, `lsof -t -i :PORT`
DNS প্রশ্ন	`dig NAME +short`, `dig @8.8.8.8 NAME`, `dig -x IP`
DNS-এর পুরো পথ	`dig +trace NAME`
সহজ DNS উত্তর	`host NAME`, `nslookup NAME`

কাজ	কমান্ড
সিস্টেম DNS	`resolvectl status`, `resolvectl flush-caches`, `getent hosts NAME`
HTTP হেডার	`curl -I URL`
redirect অনুসরণ	`curl -L URL`
ডাউনলোড	`curl -O URL`, `curl -o FILE URL`, `wget -c URL`
API রিকোয়েস্ট	`curl -X POST -H "Content-Type: application/json" -d '{..}' URL`
ডিবাগ	`curl -v URL`
status/সময় মাপা	`curl -o /dev/null -s -w "%{http_code} %{time_total}\n" URL`
পুরো সাইট mirror	`wget --mirror -k -p -np URL`
port খোলা কি না	`nc -zv HOST PORT`
port-এ শোনা	`nc -l PORT`
প্যাকেট ধরা	`sudo tcpdump -i any -nn port 53`
প্যাকেট ফাইলে	`sudo tcpdump -i IFACE -w file.pcap`
নিজের port স্ক্যান	`nmap localhost`, `sudo nmap -sV localhost`
LAN-এ কে আছে (নিজের)	`sudo nmap -sn 192.168.1.0/24`
ট্রাবলশুটিং ক্রম	link → IP → gateway → ইন্টারনেট IP → DNS → port → অ্যাপ

নিজেকে যাচাই করো

1. `ip` কমান্ড কেন `ifconfig`-এর চেয়ে ভালো? দুটো কার্নেলের সাথে কীভাবে কথা বলে?
2. `ip addr` আউটপুটে `UP` আর `LOWER_UP`-এর পার্থক্য কী? `NO-CARRIER` দেখলে কী বুঝবে?
3. `netstat -tulpn`, `route -n` আর `arp -n`-এর আধুনিক বিকল্প কী কী?
4. `ping google.com` বলছে `Temporary failure in name resolution`, কিন্তু `ping 8.8.8.8` কাজ করছে। সমস্যা কোথায়, আর পরের দুটো কমান্ড কী চালাবে?
5. `traceroute` ভেতরে কীভাবে প্রতিটা রাউটারের ঠিকানা জানে?
6. `mtr` রিপোর্টে hop ৪-এ ৫০% loss, কিন্তু শেষ hop-এ ০%। ইন্টারনেট কি খারাপ?
7. `ss -tulpn`-এ একটা লাইনে `127.0.0.1:5432` আর আরেকটায় `0.0.0.0:5432`। নিরাপত্তার দিক থেকে কোনটা কেমন?
8. `nc -zv server 443` চালালে "Connection refused" আর "timed out"-এর মানে কী আলাদা?
9. `curl`-এ `-I`, `-L`, `-o`, `-O`, `-X`, `-H`, `-d` প্রত্যেকটা কী করে? শুধু `-d` দিলে method কী হয়?
10. রিমোট সার্ভারে SSH দিয়ে ঢুকে `tcpdump` চালানোর সময় কোন ফিল্টার দেওয়া উচিত, আর কেন?

▼ উত্তর দেখো

1. `ifconfig` (net-tools) পুরনো `ioctl()` দিয়ে কার্নেলের সাথে কথা বলে আর বহু বছর ধরে প্রায় স্থবির; একাধিক IP, policy routing, namespace ঠিকমতো দেখাতে পারে না। `ip` (iproute2) আধুনিক **Netlink socket** দিয়ে কথা বলে, সব নতুন ফিচার সাপোর্ট করে, আর একটা কমান্ডেই address, link, route, neigh সব সামলায়।
2. `UP` মানে সফটওয়্যারে (অ্যাডমিন) interface চালু করা হয়েছে; `LOWER_UP` মানে physical লিংক আছে (তার লাগানো/Wi-Fi যুক্ত)। `NO-CARRIER` মানে interface চালু, কিন্তু তার বা সিগন্যাল নেই।
3. `netstat -tulpn` → `ss -tulpn`; `route -n` → `ip route`; `arp -n` → `ip neigh`।
4. IP-তে চলছে মানে লিংক, IP, gateway আর ইন্টারনেট ঠিক আছে; সমস্যা **DNS**-এ। পরের কমান্ড: `resolvectl status` (কোন DNS সার্ভার সেট আছে) আর `dig @8.8.8.8 google.com` (অন্য DNS দিয়ে কাজ করে কি না)। এছাড়া `/etc/resolv.conf` দেখা যায়।
5. TTL ১, ২, ৩... করে প্যাকেট পাঠায়। প্রতিটা রাউটার TTL ১ কমায়, আর ০ হলে প্যাকেট ফেলে দিয়ে প্রেরককে ICMP "Time Exceeded" পাঠায়, যার উৎস ঠিকানাই ওই রাউটারের IP। গন্তব্যে পৌঁছালে অন্য ধরনের উত্তর (Port Unreachable/Echo Reply/SYN-ACK) আসে, তখন থামে।
6. না। loss শেষ পর্যন্ত টিকে না থাকলে সেটা আসল ক্ষতি নয়; hop ৪-এর রাউটার শুধু নিজের কাছে আসা ICMP-র উত্তর কম দিচ্ছে (rate-limit), অন্যদের ট্রাফিক ঠিকঠাক পার করছে। আসল loss হলে সেই hop থেকে শেষ পর্যন্ত সব hop-এ loss দেখা যেত।
7. `127.0.0.1:5432` শুধু ওই মেশিন থেকেই সংযোগ নেয়, তাই নিরাপদ। `0.0.0.0:5432` সব interface-এ খোলা; firewall না থাকলে নেটওয়ার্ক বা ইন্টারনেট থেকে যে কেউ ডেটাবেসে ঢোকার চেষ্টা করতে পারে, তাই ঝুঁকিপূর্ণ।
8. **Refused**: হোস্টে পৌঁছানো গেছে আর সে RST ফেরত দিয়েছে, মানে ওই port-এ কোনো সার্ভিস listen করছে না (বা অন্য IP-তে করছে)। **Timed out**: কোনো উত্তরই আসেনি; সাধারণত firewall প্যাকেট DROP করছে, বা হোস্ট/রাস্তাই নেই।
9. `-I` শুধু হেডার (HEAD); `-L` redirect অনুসরণ; `-o FILE` নির্দিষ্ট নামে সেভ; `-O` URL-এর নামে সেভ; `-X` HTTP method ঠিক করে; `-H` হেডার যোগ করে; `-d` body-তে ডেটা পাঠায়। শুধু `-d` দিলে method নিজে থেকেই **POST** হয় (Content-Type `application/x-www-form-urlencoded`)।
10. `not port 22` দেওয়া উচিত। নইলে tcpdump নিজের SSH সেশনের প্যাকেটও ধরবে, সেই আউটপুট আবার SSH দিয়ে পাঠাতে নতুন প্যাকেট তৈরি হবে, আর এক অনন্ত চক্রে স্ক্রিন ভরে যাবে।

অধ্যায় ৩৮: SSH ও রিমোট অ্যাক্সেস

> একজন লিনাক্স অ্যাডমিনের দিনের বেশিরভাগ সময় কাটে এমন সব মেশিনে, যেগুলো সে কোনোদিন চোখেও দেখেনি। ডেটা সেন্টারে, ক্লাউডে, অন্য শহরে বা অন্য দেশে সার্ভারগুলো চলছে, আর সে নিজের ল্যাপটপে বসে সেগুলো চালাচ্ছে। এই কাজের দরজা হলো **SSH**।

এই অধ্যায়ে যা শিখবে

- SSH কী, কেন Telnet/FTP-এর বদলে SSH ব্যবহার করা হয়
- SSH কানেকশনের ভেতরে ধাপে ধাপে কী ঘটে: TCP handshake, key exchange, host key যাচাই, user authentication
- `ssh user@host` দিয়ে লগইন, রিমোট কমান্ড চালানো, ভিন্ন port ব্যবহার
- `known_hosts` কী, "REMOTE HOST IDENTIFICATION HAS CHANGED" সতর্কবার্তার মানে কী
- `ssh-keygen` দিয়ে ed25519 key তৈরি, passphrase, fingerprint
- `ssh-copy-id` আর `authorized_keys` দিয়ে পাসওয়ার্ড ছাড়া লগইন
- `~/.ssh/config` দিয়ে লম্বা কমান্ড ছোট করা
- `ssh-agent` ও `ssh-add` দিয়ে passphrase বারবার না লেখা
- সার্ভার সাইডে `sshd` ইনস্টল, `sshd_config` হার্ডেনিং (`PasswordAuthentication no`, `PermitRootLogin` ইত্যাদি)
- ফাইল আদান-প্রদান: `scp`, `sftp`, `rsync` over SSH
- Port forwarding: local (`-L`), remote (`-R`), dynamic/SOCKS (`-D`)
- Jump host / bastion (`-J`, `ProxyJump`)
- SSH escape sequences (`~.`, `~?` ইত্যাদি)
- পারমিশন সমস্যা আর সাধারণ ট্রাবলশুটিং (`ssh -v`)

৩৮.১ SSH কী এবং কেন?

SSH (Secure Shell) হলো একটি নেটওয়ার্ক প্রোটোকল, যা দিয়ে তুমি অন্য একটি কম্পিউটারে নিরাপদে লগইন করতে পারো, কমান্ড চালাতে পারো, ফাইল পাঠাতে পারো — আর পুরো আলাপটা থাকে **encrypted** (এনক্রিপ্টেড, মানে তালাবদ্ধ সংকেতে)।

আগে কী ছিল? Telnet-এর সমস্যা

SSH-এর আগে মানুষ `telnet`, `rlogin`, `rsh`, `ftp` ব্যবহার করত। এদের সবচেয়ে বড় সমস্যা: সবকিছু যেত **plain text**-এ। তুমি যে পাসওয়ার্ড টাইপ করছ, নেটওয়ার্কের মাঝখানে বসে থাকা যে কেউ (একই Wi-Fi-তে থাকা কেউ, ISP, বা কোনো হ্যাকড রাউটার) সেটা হুবহু পড়ে ফেলতে পারত।

উপমা: Telnet হলো পোস্টকার্ডে চিঠি পাঠানো — ডাকপিয়ন থেকে শুরু করে যে-ই হাতে পাক, পড়ে ফেলতে পারে। SSH হলো একটা তালাবদ্ধ বাক্সে চিঠি পাঠানো, যার চাবি শুধু তোমার আর প্রাপকের কাছে আছে — আর বাক্সের গায়ে সিল আছে, কেউ খুললে বা বদলালে ধরা পড়ে যাবে।

SSH তিনটা জিনিস নিশ্চিত করে:

বৈশিষ্ট্য	মানে	কীভাবে
Confidentiality (গোপনীয়তা)	মাঝখানে কেউ পড়তে পারবে না	Symmetric encryption (যেমন AES, ChaCha20)
Integrity (অখণ্ডতা)	মাঝখানে কেউ ডেটা বদলাতে পারবে না	MAC / AEAD (যেমন `chacha20-poly1305`, `aes-gcm`)
Authentication (পরিচয় যাচাই)	তুমি সত্যিই ঠিক সার্ভারে, আর সার্ভার নিশ্চিত যে তুমিই তুমি	Host key + user key/password

লিনাক্সে যে SSH সফটওয়্যারটা প্রায় সবাই ব্যবহার করে তার নাম **OpenSSH**। এর দুটো অংশ:

- **ক্লায়েন্ট** (`ssh`, `scp`, `sftp`, `ssh-keygen`, `ssh-agent` ...) — প্যাকেজ `openssh-client` (Debian/Ubuntu) বা `openssh-clients` (Fedora/RHEL)
- **সার্ভার** (`sshd` — "SSH daemon", যে প্রোগ্রামটা ব্যাকগ্রাউন্ডে বসে কানেকশনের অপেক্ষা করে) — প্যাকেজ `openssh-server`

ডিফল্টভাবে SSH চলে **TCP port 22**-এ।

```
$ ssh -V
OpenSSH_9.6p1 Ubuntu-3ubuntu13.5, OpenSSL 3.0.13 30 Jan 2024
```

> 📝 **মনে রাখো:** "SSH" দিয়ে প্রোটোকল বোঝায়, `ssh` হলো ক্লায়েন্ট কমান্ড, আর `sshd` হলো সার্ভার প্রোগ্রাম। নেটওয়ার্কের মূল ধারণা (IP, port, TCP) জানতে অধ্যায় ৩৬ দেখো।

৩৮.২ ভেতরে কীভাবে কাজ করে — SSH আর্কিটেকচার

SSH প্রোটোকল তিনটি স্তরে (layer) সাজানো:

```
┌──────────────────────────────────────────────────────────┐
│  Connection Layer   (RFC 4254)                           │
│  একটা কানেকশনের ভেতরে অনেকগুলো "channel":                  │
│  shell session, exec command, sftp, port forwarding...   │
├──────────────────────────────────────────────────────────┤
│  User Authentication Layer   (RFC 4252)                  │
│  "তুমি কে?" — publickey / password / keyboard-interactive  │
├──────────────────────────────────────────────────────────┤
│  Transport Layer   (RFC 4253)                            │
│  key exchange, server (host) authentication,             │
│  encryption, integrity, compression                      │
├──────────────────────────────────────────────────────────┤
│  TCP  (সাধারণত port 22)                                  │
└──────────────────────────────────────────────────────────┘
```

এখন দেখি `ssh rahim@server.example.com` লিখে Enter চাপলে ঠিক কী কী হয়।

ধাপ ১: TCP কানেকশন ও ভার্সন বিনিময়

ক্লায়েন্ট সার্ভারের port 22-এ TCP কানেকশন খোলে। তারপর দুই পক্ষ নিজেদের ভার্সন স্ট্রিং পাঠায়:

```
SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.5
```

তুমি নিজেও এটা দেখতে পারো:

```
$ nc server.example.com 22
SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.5
```

ধাপ ২: Algorithm negotiation

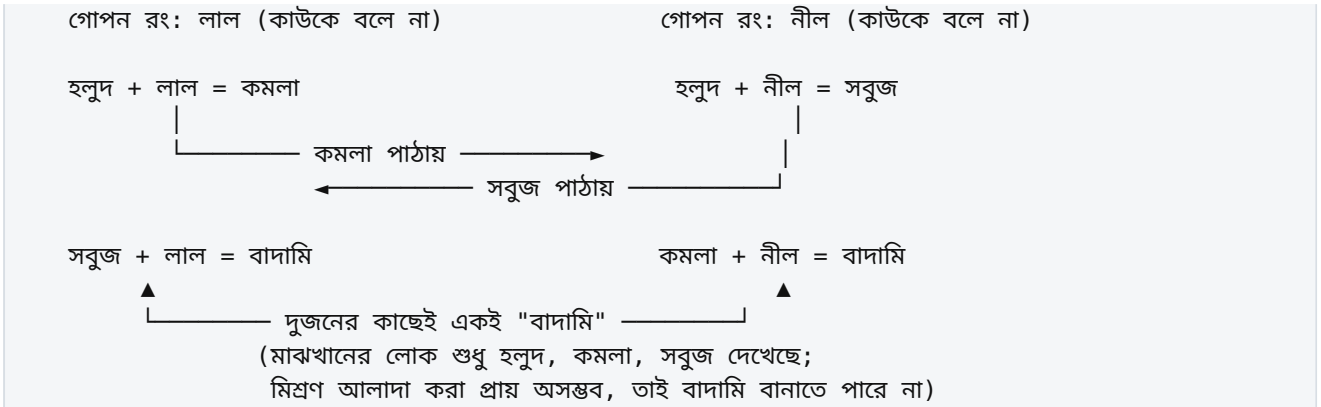
দুই পক্ষ নিজেদের সমর্থিত algorithm-এর তালিকা পাঠায় (key exchange, host key, cipher, MAC, compression)। দুই তালিকায় মিল আছে এমন সবচেয়ে পছন্দের algorithm বেছে নেওয়া হয়।

ধাপ ৩: Key Exchange (KEX) — অচেনা দুজনের গোপন চাবি বানানো

এটাই SSH-এর জাদু। ক্লায়েন্ট আর সার্ভার আগে কখনো কোনো গোপন কিছু ভাগাভাগি করেনি, অথচ খোলা নেটওয়ার্কে কথা বলেই তারা এমন একটা **shared secret** তৈরি করে ফেলে যেটা মাঝখানের কেউ জানতে পারে না। এর পদ্ধতির নাম **Diffie-Hellman key exchange** (আধুনিক রূপ: elliptic curve `curve25519-sha256`; নতুন OpenSSH-এ quantum-resistant hybrid `sntrup761x25519-sha512` বা `mlkem768x25519-sha256`)।

রঙের উপমা:

```
ক্লায়েন্ট                                          সার্ভার
──────────                                          ──────────
সবার জানা রং: হলুদ  ◄──────  public  ──────►  সবার জানা রং: হলুদ
```



এই "বাদামি" shared secret থেকে **session key** তৈরি হয়, আর বাকি পুরো কানেকশন সেই key দিয়ে symmetric encryption-এ চলে। প্রতিটা কানেকশনে নতুন session key — তাই আজকের key চুরি হলেও গতকালের রেকর্ড করা ট্রাফিক খোলা যায় না (একে **forward secrecy** বলে)।

ধাপ ৪: Server authentication — "এটা কি সত্যিই আমার সার্ভার?"

Key exchange-এর সময় সার্ভার তার **host key** (সার্ভারের নিজস্ব স্থায়ী key pair, থাকে `/etc/ssh/ssh_host_*_key`-এ) দিয়ে exchange-এর ডেটায় একটা **digital signature** দেয়। ক্লায়েন্ট সার্ভারের public host key দিয়ে সেটা যাচাই করে, আর মিলিয়ে দেখে `~/.ssh/known_hosts`-এ এই সার্ভারের যে key আগে সেভ করা আছে তার সাথে মেলে কিনা।

এটা না থাকলে **Man-in-the-Middle (MITM)** আক্রমণ সম্ভব হতো: কেউ মাঝখানে বসে নিজেকে সার্ভার হিসেবে উপস্থাপন করত, তুমি তার সাথে key exchange করতে, আর সে সব পড়ত।

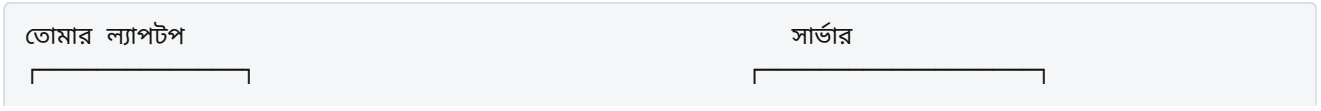
ধাপ ৫: User authentication — "তুমি কে?"

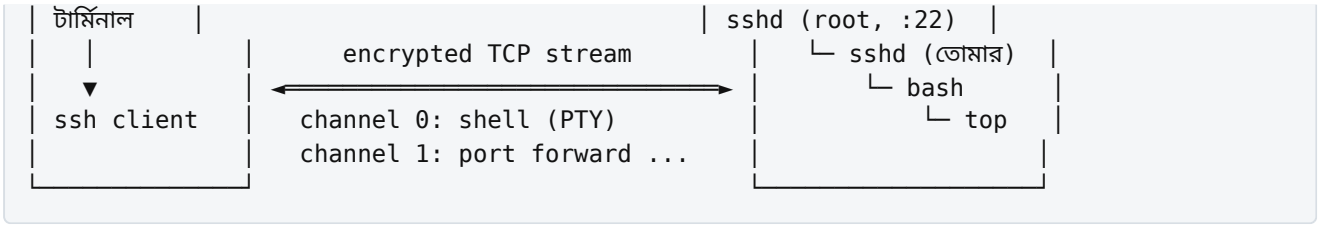
এবার encrypted চ্যানেলের ভেতরে ক্লায়েন্ট নিজের পরিচয় প্রমাণ করে। সাধারণ পদ্ধতি:

পদ্ধতি	কীভাবে	নিরাপত্তা
`publickey`	তোমার private key দিয়ে signature	সবচেয়ে ভালো (প্রস্তাবিত)
`password`	পাসওয়ার্ড পাঠানো (encrypted চ্যানেলে)	brute-force ঝুঁকি
`keyboard-interactive`	সার্ভার প্রশ্ন করে (পাসওয়ার্ড, OTP ইত্যাদি)	2FA-এর জন্য কাজে লাগে
`gssapi-with-mic`	Kerberos	এন্টারপ্রাইজ নেটওয়ার্কে

ধাপ ৬: Session — channel খোলা

সফল হলে ক্লায়েন্ট একটা **channel** খোলে, সার্ভারের `sshd` তোমার ইউজার হিসেবে একটা প্রসেস চালু করে (সাধারণত তোমার লগইন শেল, আর একটা pseudo-terminal বা **PTY**)। এখন তুমি যা টাইপ করো সেটা encrypted হয়ে যায়, শেল চালায়, output encrypted হয়ে ফেরত আসে।





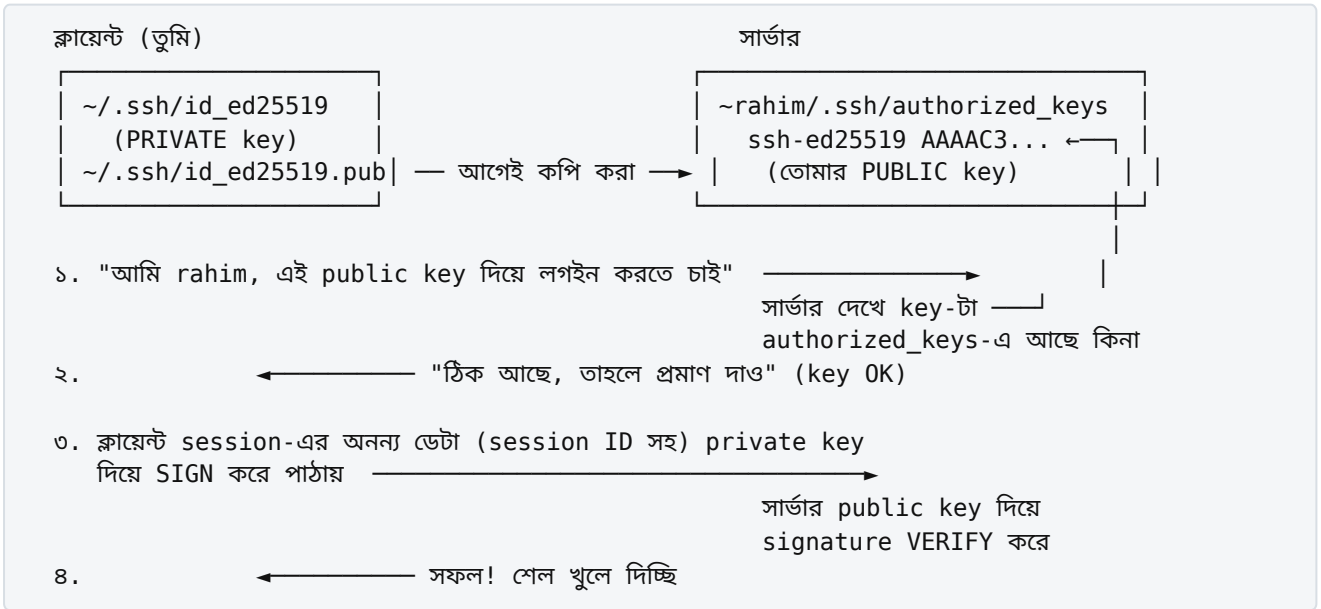
> 💡 **টিপস:** একটা SSH কানেকশনের ভেতর দিয়ে একসাথে অনেকগুলো channel যেতে পারে — এজন্যই একই কানেকশনে শেলও চলছে, আবার port forwarding-ও হচ্ছে।

Public key authentication বিস্তারিত

এটা বুঝলে SSH key-এর পুরো ব্যাপারটা পরিষ্কার হয়ে যাবে।

Key pair মানে দুটো গাণিতিকভাবে জোড়া লাগানো key:

- **Private key** (`~/.ssh/id_ed25519`) — শুধু তোমার কাছে থাকে, কাউকে দেবে না। এটা হলো **সিলমোহর**।
- **Public key** (`~/.ssh/id_ed25519.pub`) — যাকে খুশি দিতে পারো। এটা হলো সেই সিলমোহরের **ছাপের নমুনা** — যে কেউ মিলিয়ে দেখতে পারে কোনো ছাপ আসল সিলমোহর থেকে এসেছে কিনা, কিন্তু নমুনা থেকে সিলমোহর বানানো যায় না।



গুরুত্বপূর্ণ ব্যাপার: **private key কখনো নেটওয়ার্কে যায় না।** শুধু signature যায়, আর সেটা এই একটা session-এর ডেটার উপর, তাই চুরি করে পরে আবার ব্যবহার (replay) করা যায় না।

> 📝 **মনে রাখো:** দুই ধরনের key আছে, গুলিয়ে ফেলো না।
>
> - **Host key** — সার্ভারের পরিচয়। ক্লায়েন্ট সেভ করে রাখে `~/.ssh/known_hosts` -এ।

- **User key** — তোমার পরিচয়। সার্ভার সেভ করে রাখে `~/.ssh/authorized_keys`-এ।

৩৮.৩ প্রথম লগইন: `ssh user@host`

Syntax

```
ssh [options] [user@]host [command]
```

- `user` না দিলে তোমার লোকাল ইউজারনেম ধরে নেয়।
- `host` হতে পারে IP (`192.168.1.50`), hostname (`server1`), বা ডোমেইন (`server.example.com`)।
- `command` দিলে লগইন শেল না খুলে শুধু সেই কমান্ড চালিয়ে বেরিয়ে আসে।

সার্ভারে SSH চালু করা (যদি না থাকে)

যে মেশিনে লগইন করবে, সেখানে `sshd` চলতে হবে।

```
# Debian / Ubuntu
$ sudo apt install openssh-server
$ sudo systemctl enable --now ssh

# Fedora / RHEL / Rocky / Alma
$ sudo dnf install openssh-server
$ sudo systemctl enable --now sshd
```

> ⚠️ **সতর্কতা:** সার্ভিসের নাম distro-ভেদে আলাদা: Debian/Ubuntu-তে `ssh`, Fedora/RHEL-এ `sshd`। Ubuntu 22.10 থেকে (24.04 সহ) SSH ডিফল্টভাবে **socket activation** দিয়ে চলে — `ssh.socket` port-এ শোনে, কানেকশন এলে `ssh.service` চালু হয়। systemd-এর বিস্তারিত অধ্যায় ২৬-এ।

চলছে কিনা দেখো:

```
$ sudo ss -tlnp | grep :22
LISTEN 0  4096   0.0.0.0:22   0.0.0.0:*   users:(("systemd",pid=1,fd=xx))
LISTEN 0  4096      [::]:22      [::]:*   users:(("systemd",pid=1,fd=xx))
```

(socket activation-এ শোনে `systemd` নিজে; ক্লাসিক সেটআপে দেখাবে `sshd`।)

প্রথমবার কানেক্ট করা

```
$ ssh rahim@192.168.1.50
The authenticity of host '192.168.1.50 (192.168.1.50)' can't be established.
```

```
ED25519 key fingerprint is SHA256:Xq3b7mQ1u0dPq9yJmHkV2cGx8N6zVvE4tR1lKpW0aBc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.50' (ED25519) to the list of known hosts.
rahim@192.168.1.50's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-45-generic x86_64)
...
rahim@server1:~$
```

এই "authenticity can't be established" প্রশ্নটা হলো **TOFU — Trust On First Use**। প্রথমবার ক্লায়েন্ট সার্ভারকে চেনে না, তাই তোমাকে জিজ্ঞেস করছে। সত্যিকারের নিরাপদ পদ্ধতি হলো সার্ভারের কনসোলে বসে (বা অ্যাডমিনের কাছ থেকে) fingerprint মিলিয়ে নেওয়া:

```
# সার্ভারে চালাও
$ ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:Xq3b7mQ1u0dPq9yJmHkV2cGx8N6zVvE4tR1lKpW0aBc root@server1 (ED25519)
```

মিললে `yes` লেখো। প্রশ্নের উত্তরে সরাসরি fingerprint পেস্ট করলেও চলে — মিললে ক্লায়েন্ট নিজেই গ্রহণ করে।

বের হতে: `exit` লেখো বা `Ctrl+D` চাপো।

```
rahim@server1:~$ exit
logout
Connection to 192.168.1.50 closed.
```

গুরুত্বপূর্ণ `ssh` অপশন

অপশন	কাজ	উদাহরণ
`-p PORT`	ভিন্ন port (ছোট হাতের p)	`ssh -p 2222 rahim@host`
`-l USER`	ইউজার আলাদাভাবে দেওয়া	`ssh -l rahim host`
`-i FILE`	নির্দিষ্ট private key ব্যবহার	`ssh -i ~/.ssh/work_key host`
`-v`, `-vv`, `-vvv`	verbose — ডিবাগিং তথ্য	`ssh -v host`
`-t`	জোর করে PTY দেওয়া (interactive রিমোট কমান্ডের জন্য)	`ssh -t host top`
`-T`	PTY না দেওয়া	`ssh -T git@github.com`
`-C`	compression চালু (ধীর লিংকে কাজে লাগে)	`ssh -C host`
`-X` / `-Y`	X11 forwarding (রিমোট GUI অ্যাপ লোকালে দেখানো)	`ssh -X host xclock`
`-A`	agent forwarding (সাবধানে!)	`ssh -A host`
`-L`, `-R`, `-D`	port forwarding (নিচে বিস্তারিত)	
`-J`	jump host দিয়ে যাওয়া	`ssh -J bastion internal`
`-N`	কোনো রিমোট কমান্ড চালাবে না (শুধু forwarding)	`ssh -N -L ...`

অপশন	কাজ	উদাহরণ
`-f`	authentication-এর পর background-এ চলে যাবে	`ssh -fN -L ...`
`-o KEY=VALUE`	যেকোনো config অপশন কমান্ড লাইনে	`ssh -o ConnectTimeout=5 host`
`-F FILE`	ভিন্ন config ফাইল	`ssh -F ./cfg host`
`-q`	quiet — সতর্কবার্তা কম দেখাবে	`ssh -q host`
`-4` / `-6`	শুধু IPv4 / IPv6	`ssh -4 host`
`-G`	কোনো host-এর জন্য শেষ পর্যন্ত কী config প্রযোজ্য, সেটা দেখায় (কানেক্ট করে না)	`ssh -G web1`

রিমোট কমান্ড চালানো (লগইন না করেই)

```
$ ssh rahim@server1 'uptime'
 14:22:07 up 12 days,  3:41,  1 user,  load average: 0.08, 0.05, 0.01

$ ssh rahim@server1 'df -h / && free -h'

# লোকাল ফাইল রিমোটে পাইপ করা
$ cat notes.txt | ssh rahim@server1 'cat > /tmp/notes.txt'

# রিমোট output লোকালে প্রসেস করা
$ ssh rahim@server1 'cat /var/log/syslog' | grep -i error | tail -5
```

> ⚠️ **সতর্কতা:** কোটেশন খেয়াল করো। `ssh host "echo $HOME"` লিখলে `$HOME` তোমার **লোকাল** শেল আগেই expand করে ফেলবে (তোমার লোকাল home দেখাবে)। রিমোটের `$HOME` চাইলে single quote দাও: `ssh host 'echo $HOME'`। কোটেশনের নিয়ম জানতে অধ্যায় ৪১ দেখো।

`top`, `htop`, `vim` এর মতো interactive প্রোগ্রাম রিমোট কমান্ড হিসেবে চালাতে `-t` দরকার:

```
$ ssh -t rahim@server1 htop
$ ssh -t rahim@server1 'sudo systemctl restart nginx'   # sudo পাসওয়ার্ড চাইবে, তাই -t
```

৩৮.৪ `known_hosts` — সার্ভারের পরিচয়পত্রের খাতা

প্রথমবার `yes` বলার পর সার্ভারের public host key সেভ হয় `~/.ssh/known_hosts`-এ:

```
$ cat ~/.ssh/known_hosts
|1|kV3h...=|q8Zp...= ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGq...
```

শুরুর `|1|...` অংশটা হলো **hashed hostname** — Debian/Ubuntu-তে `HashKnownHosts yes` ডিফল্ট, যাতে কেউ ফাইলটা চুরি করলেও জানতে না পারে তুমি কোন কোন সার্ভারে যাও। Fedora/RHEL-এ সাধারণত hostname সরাসরি দেখা যায়।

পরের প্রতিবার কানেক্ট করার সময় ক্লায়েন্ট মিলিয়ে দেখে। না মিললে এই ভয়ংকর বার্তা:

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
...
Offending ED25519 key in /home/rahim/.ssh/known_hosts:3
Host key verification failed.
```

এর মানে কী? দুটোর একটা:

1. **নিরীহ কারণ:** সার্ভার রিইনস্টল হয়েছে, নতুন VM একই IP পেয়েছে, অ্যাডমিন host key বদলেছে।
2. **বিপজ্জনক কারণ:** কেউ MITM আক্রমণ করছে।

তাই অন্ধভাবে মুছে ফেলবে না — আগে নিশ্চিত হও কারণটা নিরীহ। নিশ্চিত হলে পুরোনো entry সরাও:

```
$ ssh-keygen -R 192.168.1.50
# Host 192.168.1.50 found: line 3
/home/rahim/.ssh/known_hosts updated.
Original contents retained as /home/rahim/.ssh/known_hosts.old
```

অন্য কাজের কমান্ড:

```
# known_hosts-এ কোনো host আছে কিনা খোঁজা (hashed হলেও কাজ করে)
$ ssh-keygen -F 192.168.1.50

# কানেক্ট না করেই সার্ভারের host key আনা
$ ssh-keyscan -t ed25519 192.168.1.50
192.168.1.50 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGq...
```

`StrictHostKeyChecking` অপশনটা নিয়ন্ত্রণ করে নতুন/বদলানো key-এর ক্ষেত্রে কী হবে:

মান	নতুন host	বদলানো key
`ask` (ডিফল্ট)	জিজ্ঞেস করে	কানেক্ট করে না
`accept-new`	চুপচাপ যোগ করে	কানেক্ট করে না
`yes`	কানেক্ট করে না	কানেক্ট করে না
`no` / `off`	চুপচাপ যোগ করে	সতর্ক করে, কিছু ঝুঁকিপূর্ণ ফিচার বন্ধ করে কানেক্ট করে

> 💡 **টিপস:** অটোমেশন স্ক্রিপ্টে `-o StrictHostKeyChecking=accept-new` ভালো আপস — নতুন সার্ভার যোগ করে, কিন্তু key বদলালে থামে। `StrictHostKeyChecking=no` আর `UserKnownHostsFile=/dev/null` একসাথে দেওয়া মানে MITM-এর দরজা পুরো খুলে দেওয়া; শুধু ল্যাবের ক্ষণস্থায়ী VM-এর জন্য।

৩৮.৫ SSH Key তৈরি: `ssh-keygen`

পাসওয়ার্ড দিয়ে লগইনের সমস্যা: দুর্বল পাসওয়ার্ড অনুমান করা যায়, ইন্টারনেটে খোলা যেকোনো SSH সার্ভারে দিনে হাজার হাজার bot পাসওয়ার্ড চেষ্টা করে। Key-based লগইন অনেক বেশি নিরাপদ এবং বেশি সুবিধাজনক।

কোন ধরনের key?

Type	মন্তব্য
ed25519	আধুনিক, দ্রুত, ছোট, নিরাপদ — **এটাই ব্যবহার করো**
rsa	পুরোনো সিস্টেমের সাথে compatible; ব্যবহার করলে কমপক্ষে 3072 বা 4096 bit
ecdsa	চলে, কিন্তু ed25519 বেশি পছন্দনীয়
ed25519-sk / ecdsa-sk	হার্ডওয়্যার security key (YubiKey ইত্যাদি)-তে বাঁধা
dsa	বাতিল; নতুন OpenSSH-এ সমর্থনই নেই

Key তৈরি

```
$ ssh-keygen -t ed25519 -C "rahim@laptop"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/rahim/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/rahim/.ssh/id_ed25519
Your public key has been saved in /home/rahim/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:9fT2nC8yW1aRkL0pM3xQe7HvB5uJdZ4sN6gO2iYtK1w rahim@laptop
The key's randomart image is:
+--[ED25519 256]--+
|        .o+=*o.  |
|       . ..=+=.  |
|        o *.B..  |
|       . O * =   |
|        S = = .  |
|         + + o   |
|        . = E .  |
|         o.B o   |
|          .=*B.  |
+----[SHA256]-----+
```

তৈরি হলো দুটো ফাইল:

```
$ ls -l ~/.ssh/
-rw------- 1 rahim rahim 411 Nov 12 10:01 id_ed25519       # private — কাউকে দেবে না!
-rw-r--r-- 1 rahim rahim  98 Nov 12 10:01 id_ed25519.pub   # public — নির্দ্বিধায় শেয়ার করো

$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJm8x... rahim@laptop
```

Public key-এর তিনটা অংশ: `type` `base64-key` `comment`। Comment শুধু চেনার সুবিধার জন্য, কাজে কোনো প্রভাব নেই।

Passphrase — দিতে হবে কেন?

Passphrase দিলে private key ফাইলটা নিজেই encrypted থাকে। ল্যাপটপ চুরি হলে বা ব্যাকআপ ফাঁস হলে চোর key ফাইল পেলেও passphrase ছাড়া ব্যবহার করতে পারবে না। বারবার লেখার ঝামেলা? সেটার সমাধান `ssh-agent` (৩৮.৮ অংশে দেখো)।

> 💡 **টিপস:** পাসওয়ার্ডবিহীন key শুধু অটোমেশনের জন্য (cron job, CI) — আর সেই key-কে `authorized_keys`-এ সীমাবদ্ধ করে দাও (৩৮.৬-এ দেখো)।

`ssh-keygen`-এর গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-t TYPE`	key type (`ed25519`, `rsa`, `ecdsa`, `ed25519-sk`)
`-b BITS`	key size (RSA-র জন্য, যেমন `-b 4096`)
`-C COMMENT`	comment
`-f FILE`	ফাইলের নাম/পাথ
`-N PASS`	passphrase কমান্ডেই (`-N ""` = খালি; স্ক্রিপ্টে)
`-a ROUNDS`	passphrase brute-force কঠিন করতে KDF rounds (ডিফল্ট 16)
`-p`	বিদ্যমান key-এর passphrase বদলানো
`-l`	fingerprint দেখানো
`-y`	private key থেকে public key আবার তৈরি করা
`-R HOST`	known_hosts থেকে host মুছে ফেলা
`-F HOST`	known_hosts-এ host খোঁজা
`-E md5`	পুরোনো MD5 ফরম্যাটে fingerprint

```
# কাজের জন্য আলাদা key, আলাদা নামে
$ ssh-keygen -t ed25519 -C "rahim@work" -f ~/.ssh/id_work
```

```
# passphrase বদলানো
$ ssh-keygen -p -f ~/.ssh/id_ed25519

# fingerprint দেখা
$ ssh-keygen -lf ~/.ssh/id_ed25519.pub
256 SHA256:9fT2nC8yW1aRkL0pM3xQe7HvB5uJdZ4sN6gO2iYtK1w rahim@laptop (ED25519)

# .pub ফাইল হারিয়ে গেলে
$ ssh-keygen -y -f ~/.ssh/id_ed25519 > ~/.ssh/id_ed25519.pub
```

৩৮.৬ সার্ভারে public key বসানো: `ssh-copy-id` ও `authorized_keys`

সহজ পথ: `ssh-copy-id`

```
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub rahim@192.168.1.50
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/home/rahim/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any
that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now
it is to install the new keys
rahim@192.168.1.50's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'rahim@192.168.1.50'"
and check to make sure that only the key(s) you wanted were added.
```

এটা একবার পাসওয়ার্ড দিয়ে লগইন করে, সার্ভারে `~/.ssh` না থাকলে তৈরি করে, তোমার public key `~/.ssh/authorized_keys`-এর শেষে যোগ করে, আর পারমিশন ঠিক করে দেয়। ভিন্ন port হলে: `ssh-copy-id -p 2222 rahim@host`।

এখন:

```
$ ssh rahim@192.168.1.50
Welcome to Ubuntu 24.04.1 LTS ...
rahim@server1:~$          # পাসওয়ার্ড চায়নি!
```

হাতে করে (ssh-copy-id না থাকলে)

```
$ cat ~/.ssh/id_ed25519.pub | ssh rahim@server1 \
  'mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600
~/.ssh/authorized_keys'
```

> ⚠️ **সতর্কতা:** `>>` (append) দিও, `>` না — নইলে আগের সব key মুছে যাবে এবং অন্যরা (বা তুমি নিজেই অন্য মেশিন থেকে) লক-আউট হবে। রিডাইরেকশন নিয়ে অধ্যায় ১৯ দেখো।

`authorized_keys` ফাইলের ফরম্যাট

প্রতি লাইনে একটা key:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJm8x... rahim@laptop
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKp2q... rahim@desktop
```

কোনো key সরিয়ে দিতে চাইলে (যেমন পুরোনো ল্যাপটপ হারিয়ে গেছে) শুধু সেই লাইনটা মুছে দাও।

Key-এর সামনে সীমাবদ্ধতা (options)

Key-এর আগে option বসিয়ে ক্ষমতা কমানো যায় — অটোমেশন key-এর জন্য দারুণ:

```
from="10.0.0.0/24",restrict,command="/usr/local/bin/backup.sh" ssh-ed25519 AAAA...
backup-bot
```

Option	কাজ
`from="pattern"`	শুধু এই IP/host থেকে key গ্রহণযোগ্য
`command="cmd"`	যা-ই চাওয়া হোক, শুধু এই কমান্ড চলবে
`restrict`	port/agent/X11 forwarding ও PTY সব বন্ধ
`no-port-forwarding`, `no-agent-forwarding`, `no-X11-forwarding`, `no-pty`	আলাদা আলাদা ভাবে বন্ধ
`expiry-time="YYYYMMDD"`	নির্দিষ্ট তারিখের পর key অচল
`permitopen="host:port"`	শুধু নির্দিষ্ট গন্তব্যে local forwarding

৩৮.৭ `~/.ssh/config` — শর্টকাটের রাজা

ধরো তুমি রোজ লেখো:

```
$ ssh -p 2222 -i ~/.ssh/id_work deploy@203.0.113.25
```

এর বদলে `~/.ssh/config` ফাইলে:

```
# ~/.ssh/config
```

```
Host web1
    HostName 203.0.113.25
    User deploy
    Port 2222
    IdentityFile ~/.ssh/id_work
    IdentitiesOnly yes

Host db1
    HostName 10.0.5.12
    User admin
    ProxyJump web1

Host github.com
    User git
    IdentityFile ~/.ssh/id_github

Host *.lab.local
    User student
    StrictHostKeyChecking accept-new

# সব host-এর জন্য ডিফল্ট — সবসময় শেষে রাখো
Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
    AddKeysToAgent yes
```

এখন শুধু:

```
$ ssh web1
$ scp report.pdf web1:/tmp/
$ rsync -av site/ web1:/var/www/
```

`scp`, `sftp`, `rsync`, `git` — সবাই এই config পড়ে।

কীভাবে পড়া হয় (খুব গুরুত্বপূর্ণ)

```
  ssh web1
     │
     ▼
  ১. কমান্ড লাইনের -o / -p / -i ইত্যাদি
  ২. ~/.ssh/config (উপর থেকে নিচে)
  ৩. /etc/ssh/ssh_config (+ /etc/ssh/ssh_config.d/*.conf)
     │
     ▼
  প্রতিটা অপশনের জন্য যে মান **প্রথমে** পাওয়া যায় সেটাই চূড়ান্ত
```

এজন্যই নির্দিষ্ট `Host` ব্লক উপরে আর `Host *` সবার শেষে রাখতে হয়। `Host *` উপরে রাখলে সেখানে দেওয়া `User` সব host-এর জন্য "প্রথম মান" হয়ে যাবে, নিচের ব্লকের `User` উপেক্ষিত হবে।

```
# কোনো host-এর চূড়ান্ত config দেখা
$ ssh -G web1 | grep -E '^(hostname|user|port|identityfile) '
hostname 203.0.113.25
user deploy
```

```
port 2222
identityfile ~/.ssh/id_work
```

দরকারি config অপশন

অপশন	কাজ
`Host`	alias/pattern (`*`, `?`, `!` চলে)
`HostName`	আসল host/IP
`User`, `Port`	ইউজার, port
`IdentityFile`	কোন private key
`IdentitiesOnly yes`	শুধু উল্লেখিত key চেষ্টা করবে (agent-এ অনেক key থাকলে "Too many authentication failures" এড়ায়)
`ProxyJump`	jump host
`LocalForward`, `RemoteForward`, `DynamicForward`	স্থায়ী port forwarding
`ServerAliveInterval`	এত সেকেন্ড পরপর keepalive পাঠায় (idle কানেকশন কেটে যাওয়া ঠেকায়)
`ForwardAgent`	agent forwarding
`AddKeysToAgent yes`	প্রথমবার ব্যবহারে key agent-এ যোগ
`ControlMaster`, `ControlPath`, `ControlPersist`	একই host-এ কানেকশন শেয়ার (multiplexing)
`Include`	অন্য config ফাইল যোগ করা

Multiplexing — দ্বিতীয় কানেকশন চোখের পলকে

```
Host *
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h:%p
    ControlPersist 10m
```

প্রথম `ssh web1` সাধারণভাবে কানেক্ট করে একটা socket রেখে দেয়। পরের `ssh web1`, `scp` ইত্যাদি সেই খোলা কানেকশনের ভেতরে নতুন channel খোলে — নতুন handshake বা authentication লাগে না। শেষ কানেকশন বন্ধ হওয়ার পর ১০ মিনিট পর্যন্ত master টিকে থাকে।

```
$ ssh -O check web1     # master চলছে কিনা
Master running (pid=41234)
$ ssh -O exit web1      # master বন্ধ করা
```

> 📝 **মনে রাখো:** `~/.ssh/config` ফাইলটা অন্যরা লিখতে পারলে ssh সেটা ব্যবহার করতে অস্বীকার করে ("Bad owner or permissions")। `chmod 600 ~/.ssh/config` দাও।

৩৮.৮ `ssh-agent` — passphrase একবার লেখো

Key-এ passphrase দিয়েছ (ভালো!), কিন্তু প্রতিবার `ssh` চালালে passphrase চাইছে (বিরক্তিকর!)। সমাধান **`ssh-agent`** — একটা ছোট ব্যাকগ্রাউন্ড প্রোগ্রাম যা তোমার decrypted private key মেমরিতে ধরে রাখে।

উপমা: অফিসের রিসেপশনিস্ট। সকালে একবার তাকে তোমার সিলমোহর দিয়ে দাও (passphrase দিয়ে খুলে)। সারাদিন কোনো কাগজে সিল লাগাতে হলে তুমি কাগজটা তার কাছে পাঠাও, সে সিল মেরে ফেরত দেয় — সিলমোহর কখনো তার টেবিল ছেড়ে যায় না।

```
ssh client ──"এই ডেটায় sign করে দাও"──► ssh-agent (মেমরিতে key)
     ▲                                          │
     └──────────── signature ◄──────────────────┘
        (যোগাযোগ হয় একটা UNIX socket দিয়ে: $SSH_AUTH_SOCK)
```

GNOME/KDE-এর মতো ডেস্কটপে সাধারণত লগইনের সাথে সাথেই একটা agent চালু থাকে (GNOME-এ `gcr-ssh-agent` বা পুরোনোতে `gnome-keyring`)। চেক করো:

```
$ echo $SSH_AUTH_SOCK
/run/user/1000/gcr/ssh
```

খালি হলে (যেমন সার্ভারে বা শুধু-টার্মিনাল সেশনে) নিজে চালু করো:

```
$ eval "$(ssh-agent -s)"
Agent pid 52811
```

`ssh-agent -s` কিছু শেল কমান্ড প্রিন্ট করে (`SSH_AUTH_SOCK=...; export SSH_AUTH_SOCK; ...`), আর `eval` সেগুলো বর্তমান শেলে চালিয়ে environment variable সেট করে দেয়।

`ssh-add`

কমান্ড	কাজ
`ssh-add`	ডিফল্ট key-গুলো (`~/.ssh/id_ed25519` ইত্যাদি) যোগ করে
`ssh-add ~/.ssh/id_work`	নির্দিষ্ট key যোগ
`ssh-add -t 3600`	১ ঘণ্টা পর key নিজে থেকে মুছে যাবে
`ssh-add -l`	লোড করা key-এর fingerprint তালিকা
`ssh-add -L`	লোড করা key-এর public key তালিকা
`ssh-add -d FILE`	একটা key সরানো
`ssh-add -D`	সব key সরানো
`ssh-add -x` / `-X`	agent লক / আনলক (পাসওয়ার্ড দিয়ে)

```
$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/rahim/.ssh/id_ed25519:
Identity added: /home/rahim/.ssh/id_ed25519 (rahim@laptop)

$ ssh-add -l
256 SHA256:9fT2nC8yW1aRkL0pM3xQe7HvB5uJdZ4sN6gO2iYtK1w rahim@laptop (ED25519)
```

এখন যত খুশি `ssh` চালাও, passphrase আর চাইবে না।

Agent forwarding (`-A`) — সুবিধা আর ঝুঁকি

`ssh -A server1` দিলে server1-এ বসে আরেকটা সার্ভারে (বা GitHub-এ) ssh করার সময় তোমার **লোকাল** agent ব্যবহার হয় — key সার্ভারে কপি করতে হয় না।

> ⚠️ **সতর্কতা:** server1-এর root (বা server1 হ্যাক করা কেউ) তোমার কানেকশন চলাকালীন তোমার forwarded agent socket ব্যবহার করে তোমার নামে অন্য সার্ভারে লগইন করতে পারে। অবিশ্বস্ত সার্ভারে কখনো `-A` দিও না। বেশিরভাগ ক্ষেত্রে jump host-এর জন্য `-J` (৩৮.১৫) agent forwarding-এর চেয়ে নিরাপদ বিকল্প।

৩৮.৯ সার্ভার সাইড: `sshd_config` ও হার্ডেনিং

সার্ভারের `sshd` কনফিগার হয় `/etc/ssh/sshd_config` দিয়ে (ক্লায়েন্টের `/etc/ssh/ssh_config`-এর সাথে গুলিয়ো না — নামে শুধু একটা `d`-এর পার্থক্য)।

ফাইলের গঠন — `Include` আর "প্রথম মান জেতে"

আধুনিক Ubuntu, Debian 12, Fedora, RHEL 9 — সবার `sshd_config`-এর একদম উপরের দিকে আছে:

```
Include /etc/ssh/sshd_config.d/*.conf
```

আর `sshd`-এর নিয়ম হলো: **বেশিরভাগ keyword-এর ক্ষেত্রে যে মান প্রথমে পাওয়া যায় সেটাই কার্যকর।** যেহেতু `Include` উপরে, তাই `sshd_config.d/`-এর ফাইলগুলো মূল ফাইলের চেয়ে অগ্রাধিকার পায়, আর সেখানে ফাইলগুলো বর্ণানুক্রমে পড়া হয়।

> ⚠️ **সতর্কতা (বাস্তবে অনেকেই এখানে আটকায়):** তুমি `sshd_config`-এ `PasswordAuthentication no` লিখলে, কিন্তু তবুও পাসওয়ার্ড দিয়ে লগইন হচ্ছে! কারণ প্রায়ই ক্লাউড ইমেজে `/etc/ssh/sshd_config.d/50-cloud-init.conf` ফাইলে `PasswordAuthentication yes` আছে — সেটা আগে পড়া হয় তাই জিতে যায়। সমাধান: নিজের সেটিং একটা কম নম্বরের ফাইলে রাখো, যেমন

> `/etc/ssh/sshd_config.d/00-hardening.conf`, অথবা সেই cloud-init ফাইলটা ঠিক করো। সবসময় `sshd -T` দিয়ে কার্যকর মান যাচাই করো।

হার্ডেনিং কনফিগারেশন — একটা ভালো উদাহরণ

```
# /etc/ssh/sshd_config.d/00-hardening.conf

# root সরাসরি লগইন করতে পারবে না (sudo ব্যবহার করো)
PermitRootLogin no

# শুধু key দিয়ে লগইন
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitEmptyPasswords no

# শুধু এই ইউজার/গ্রুপ লগইন করতে পারবে
AllowGroups sshusers

# brute-force কঠিন করা
MaxAuthTries 3
LoginGraceTime 30

# idle কানেকশন সামলানো (৩০০s × ২ = ~১০ মিনিট সাড়া না পেলে কেটে দেবে)
ClientAliveInterval 300
ClientAliveCountMax 2

# দরকার না থাকলে বন্ধ
X11Forwarding no
AllowAgentForwarding no
# AllowTcpForwarding no      # port forwarding লাগলে এটা বন্ধ করো না
```

গুরুত্বপূর্ণ `sshd_config` অপশন

অপশন	মান ও অর্থ
`Port`	শোনার port (ডিফল্ট 22)
`ListenAddress`	কোন IP-তে শুনবে
`PermitRootLogin`	`yes` / `prohibit-password` (root শুধু key দিয়ে; upstream ডিফল্ট) / `forced-commands-only` / `no`
`PasswordAuthentication`	`yes` / `no` — পাসওয়ার্ড লগইন
`KbdInteractiveAuthentication`	keyboard-interactive (পুরোনো নাম `ChallengeResponseAuthentication`)
`PubkeyAuthentication`	key লগইন (ডিফল্ট `yes`)
`AuthorizedKeysFile`	key ফাইলের পাথ (ডিফল্ট `.ssh/authorized_keys .ssh/authorized_keys2`)

অপশন	মান ও অর্থ
`AllowUsers` / `DenyUsers`	ইউজারের সাদা/কালো তালিকা (`rahim@10.0.0.*` ধরনের pattern চলে)
`AllowGroups` / `DenyGroups`	গ্রুপের সাদা/কালো তালিকা
`MaxAuthTries`	একটা কানেকশনে কতবার চেষ্টা
`MaxSessions`	একটা কানেকশনে কয়টা session (multiplexing)
`LoginGraceTime`	লগইন শেষ করার সময়সীমা
`ClientAliveInterval` / `ClientAliveCountMax`	সার্ভার থেকে keepalive
`X11Forwarding`	X11 forwarding
`AllowTcpForwarding`	port forwarding (`yes` / `no` / `local` / `remote`)
`GatewayPorts`	remote forward অন্য মেশিনের জন্য খোলা হবে কিনা
`AllowAgentForwarding`	agent forwarding
`StrictModes`	ইউজারের home/.ssh ফাইলের পারমিশন যাচাই (ডিফল্ট `yes` , রেখে দাও)
`UsePAM`	PAM ব্যবহার (Debian/Ubuntu/RHEL-এ `yes`)
`Banner`	লগইনের আগে দেখানো বার্তার ফাইল
`Match`	শর্তসাপেক্ষ সেটিং (নিচে দেখো)

`Match` দিয়ে নির্দিষ্ট ইউজার/গ্রুপ/IP-এর জন্য আলাদা নিয়ম:

```
# শুধু অফিস নেটওয়ার্ক থেকে পাসওয়ার্ড লগইন চলবে
Match Address 10.10.0.0/16
    PasswordAuthentication yes

# sftponly গ্রুপ শুধু ফাইল আদান-প্রদান করতে পারবে, শেল পাবে না
Match Group sftponly
    ChrootDirectory /srv/sftp/%u
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
```

> 📝 **মনে রাখো:** `Match` ব্লক পরের `Match` বা ফাইলের শেষ পর্যন্ত চলে। তাই `Match` সবসময় ফাইলের শেষের দিকে রাখো। `ChrootDirectory` -র পাথটা root-এর মালিকানায় থাকতে হবে এবং অন্য কেউ লিখতে পারবে না।

নিরাপদে পরিবর্তন প্রয়োগ — লক-আউট এড়ানোর নিয়ম

রিমোট সার্ভারের SSH কনফিগ ভুল করা মানে নিজের ঘরের চাবি ভেতরে রেখে দরজা বন্ধ করা। এই ধাপগুলো মেনে চলো:

```
# ১. key লগইন আগে থেকেই কাজ করছে কিনা নিশ্চিত হও (নতুন টার্মিনাল থেকে)
$ ssh rahim@server1 'echo key-login-OK'

# ২. সিনট্যাক্স পরীক্ষা (কিছু না দেখালে ঠিক আছে)
$ sudo sshd -t

# ৩. কার্যকর মানগুলো দেখো
$ sudo sshd -T | grep -Ei
'^(permitrootlogin|passwordauthentication|kbdinteractiveauthentication|port) '
port 22
permitrootlogin no
passwordauthentication no
kbdinteractiveauthentication no

# ৪. reload (চলমান কানেকশন কাটে না)
$ sudo systemctl reload ssh      # Debian/Ubuntu
$ sudo systemctl reload sshd     # Fedora/RHEL

# ৫. পুরোনো সেশন খোলা রেখেই, নতুন টার্মিনাল থেকে লগইন পরীক্ষা করো
```

> ⚠️ **সতর্কতা:** পরীক্ষা সফল না হওয়া পর্যন্ত বর্তমান SSH সেশন **বন্ধ করবে না**। `sshd` restart/reload করলেও চলমান সেশন টিকে থাকে — এটাই তোমার জীবনরক্ষাকারী দড়ি।

Port বদলানো — distro-ভেদে বিশেষ ধাপ

Port 22 থেকে অন্য port-এ নিলে bot-এর লগের আবর্জনা কমে (নিরাপত্তা তেমন বাড়ে না, তবে লগ পরিষ্কার থাকে)।

```
Port 2222
```

তারপর:

- **ফায়ারওয়াল:** নতুন port খোলো, পুরোনোটা বন্ধ করার আগে পরীক্ষা করো (অধ্যায় ৩৯)।

  ```
  $ sudo ufw allow 2222/tcp                                          # Ubuntu
  $ sudo firewall-cmd --permanent --add-port=2222/tcp && sudo firewall-cmd --reload
  # Fedora/RHEL
  ```

- **SELinux (Fedora/RHEL):** SELinux জানে sshd শুধু `ssh_port_t` টাইপের port-এ বসতে পারে। নতুন port যোগ না করলে sshd চালুই হবে না:

  ```
  $ sudo semanage port -a -t ssh_port_t -p tcp 2222
  $ sudo systemctl restart sshd
  ```

- **Ubuntu 24.04 (socket activation):** port শোনে `ssh.socket`। `sshd_config`-এর `Port` থেকে একটা systemd generator socket সেটিং তৈরি করে, তাই বদলানোর পর:

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart ssh.socket
```

> 💡 **টিপস:** আরও নিরাপত্তার জন্য fail2ban (বারবার ভুল চেষ্টা করা IP ব্লক করা), 2FA ইত্যাদি অধ্যায় ৪৫-এ আছে। SSH লগ দেখতে: `journalctl -u ssh` (Ubuntu) বা `journalctl -u sshd` (RHEL); ফাইলে `/var/log/auth.log` (Debian/Ubuntu) বা `/var/log/secure` (RHEL) — বিস্তারিত অধ্যায় ২৮-এ।

৩৮.১০ পারমিশন সমস্যা — "কেন key কাজ করছে না?"

Key ঠিকঠাক বসিয়েছ, তবুও পাসওয়ার্ড চাইছে বা "Permission denied (publickey)" দেখাচ্ছে? ৯০% ক্ষেত্রে কারণ **ফাইল পারমিশন**।

`sshd`-এর `StrictModes yes` (ডিফল্ট) নিয়ম: যদি অন্য কেউ তোমার `authorized_keys` বদলাতে পারে, তাহলে সে নিজের key ঢুকিয়ে তোমার অ্যাকাউন্টে ঢুকতে পারবে। তাই sshd এমন অবস্থায় key-টা বিশ্বাসই করে না।

```
সার্ভার সাইড (যার অ্যাকাউন্টে ঢুকছ):
  /home/rahim                    755 বা 750 বা 700  (group/other লিখতে পারবে না!)
  /home/rahim/.ssh               700  drwx------
  /home/rahim/.ssh/authorized_keys  600  -rw-------
  মালিক: rahim (root না!)

ক্লায়েন্ট সাইড:
  ~/.ssh                         700
  ~/.ssh/id_ed25519   (private)  600  -rw-------
  ~/.ssh/id_ed25519.pub          644
  ~/.ssh/config                  600
  ~/.ssh/known_hosts             644 বা 600
```

ঠিক করার কমান্ড:

```
# সার্ভারে
$ chmod go-w ~
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/authorized_keys
$ sudo chown -R rahim:rahim /home/rahim/.ssh     # root দিয়ে বানালে মালিক ভুল হয়

# ক্লায়েন্টে
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519 ~/.ssh/config
```

ক্লায়েন্টের private key বেশি খোলা থাকলে ssh নিজেই প্রত্যাখ্যান করে:

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
```

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '/home/rahim/.ssh/id_ed25519' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
```

পারমিশনের মূল ধারণা জানতে অধ্যায় ১৬ দেখো।

> 💡 **টিপস (Fedora/RHEL):** পারমিশন ঠিক থাকলেও key কাজ না করলে SELinux context দেখো — বিশেষ করে `.ssh` ফোল্ডার অন্য জায়গা থেকে কপি/move করলে। ঠিক করতে: `restorecon -Rv ~/.ssh`।

ট্রাবলশুটিং: `ssh -v`

সমস্যা হলে সবার আগে verbose মোড:

```
$ ssh -v rahim@server1
OpenSSH_9.6p1 Ubuntu-3ubuntu13.5, OpenSSL 3.0.13 30 Jan 2024
debug1: Reading configuration data /home/rahim/.ssh/config
debug1: Connecting to server1 [192.168.1.50] port 22.
debug1: Connection established.
...
debug1: Authentications that can continue: publickey,password
debug1: Offering public key: /home/rahim/.ssh/id_ed25519 ED25519 SHA256:9fT2... agent
debug1: Authentications that can continue: publickey,password
debug1: Next authentication method: password
```

"Offering public key" এর পর সার্ভার আবার "can continue" বললে মানে সার্ভার key-টা গ্রহণ করেনি — সার্ভারের পারমিশন বা `authorized_keys`-এর লেখা দেখো। সার্ভারের দিকের কারণ সার্ভারের লগে পাবে:

```
$ sudo journalctl -u ssh -n 20
... sshd[1234]: Authentication refused: bad ownership or modes for directory
/home/rahim/.ssh
```

Error	সম্ভাব্য কারণ
`Connection refused`	sshd চলছে না, বা ভুল port
`Connection timed out`	ফায়ারওয়াল ব্লক করছে, host বন্ধ, রাউটিং সমস্যা
`No route to host`	নেটওয়ার্ক/ফায়ারওয়াল (REJECT)
`Permission denied (publickey)`	key মেলেনি, পারমিশন সমস্যা, ভুল ইউজার, পাসওয়ার্ড লগইন বন্ধ
`Host key verification failed`	known_hosts-এর key মিলছে না
`Too many authentication failures`	agent-এ অনেক key; `IdentitiesOnly yes` + `IdentityFile` দাও
`Could not resolve hostname`	DNS সমস্যা বা ভুল নাম (অধ্যায় ৩৭)

৩৮.১১ ফাইল কপি: `scp`

`scp` (secure copy) হলো SSH-এর উপর দিয়ে `cp`।

Syntax

```
scp [options] SOURCE... DESTINATION
```

রিমোট পাথ লেখার নিয়ম: `user@host:path`। কোলন (`:`) না দিলে scp এটাকে লোকাল ফাইলের নাম ভাববে!

```
# লোকাল → রিমোট
$ scp report.pdf rahim@server1:/home/rahim/docs/
report.pdf                          100%  2048KB  10.5MB/s   00:00

# রিমোট → লোকাল (. = বর্তমান ডিরেক্টরি)
$ scp rahim@server1:/var/log/nginx/error.log .

# পুরো ফোল্ডার
$ scp -r ./website rahim@server1:/tmp/

# ভিন্ন port (বড় হাতের P!)
$ scp -P 2222 file.txt rahim@server1:~

# দুই রিমোটের মধ্যে, তোমার মেশিন দিয়ে ঘুরে
$ scp -3 server1:/data/a.tar server2:/backup/
```

অপশন	কাজ
`-r`	recursive (ফোল্ডার)
`-P PORT`	port (বড় হাতের, ssh-এর `-p`-র মতো না!)
`-p`	modification time ও mode রক্ষা করে
`-i FILE`	identity key
`-C`	compression
`-l KBIT`	bandwidth সীমা (Kbit/s)
`-3`	দুই রিমোটের মধ্যে কপি লোকাল দিয়ে ঘুরিয়ে
`-J HOST`	jump host
`-O`	পুরোনো SCP প্রোটোকল ব্যবহার
`-q`	progress bar বন্ধ

> 📝 **মনে রাখো:** OpenSSH 9.0 থেকে `scp` ভেতরে ভেতরে পুরোনো SCP প্রোটোকলের বদলে **SFTP প্রোটোকল** ব্যবহার করে (পুরোনোটার নিরাপত্তা ত্রুটি ছিল)। খুব পুরোনো সার্ভারে SFTP না থাকলে `-O` দাও।

> ⚠️ **সতর্কতা:** `scp` বারবার সিঙ্ক বা বিশাল ডেটার জন্য ভালো না — মাঝপথে কেটে গেলে আবার শুরু থেকে কপি করে। এ কাজে `rsync` (৩৮.১৩) ব্যবহার করো।

৩৮.১২ `sftp` — ইন্টারঅ্যাকটিভ ফাইল ট্রান্সফার

`sftp` হলো FTP-র মতো ইন্টারঅ্যাকটিভ সেশন, কিন্তু SSH-এর ভেতরে (FTP-র সাথে প্রোটোকলগত কোনো সম্পর্ক নেই)। সার্ভারে ফাইল ব্রাউজ করে দেখে তারপর নামাতে চাইলে সুবিধাজনক।

```
$ sftp rahim@server1
Connected to server1.
sftp> pwd
Remote working directory: /home/rahim
sftp> ls
docs   backup.tar.gz   notes.txt
sftp> lcd ~/Downloads
sftp> get backup.tar.gz
Fetching /home/rahim/backup.tar.gz to backup.tar.gz
backup.tar.gz                     100%   45MB  11.2MB/s   00:04
sftp> put photo.jpg docs/
Uploading photo.jpg to /home/rahim/docs/photo.jpg
sftp> bye
```

ভিন্ন port: `sftp -P 2222 rahim@server1` (এখানেও বড় হাতের P)।

sftp কমান্ড	কাজ
`ls`, `cd`, `pwd`, `mkdir`, `rm`, `rmdir`, `rename`	**রিমোটে** কাজ
`lls`, `lcd`, `lpwd`, `lmkdir`	**লোকালে** কাজ (শুরুতে `l` = local)
`get FILE`	রিমোট → লোকাল
`get -r DIR`	ফোল্ডার নামানো
`put FILE`	লোকাল → রিমোট
`put -r DIR`	ফোল্ডার তোলা
`reget` / `reput`	মাঝপথে থেমে যাওয়া ট্রান্সফার আবার চালু (resume)
`chmod`, `chown`	রিমোট পারমিশন
`df -h`	রিমোট ডিস্কের ফাঁকা জায়গা
`!cmd`	লোকাল শেলে কমান্ড
`help` বা `?`	সাহায্য
`bye` / `exit` / `quit`	বের হওয়া

স্ক্রিপ্টে ব্যবহারের জন্য batch মোড:

```
$ cat cmds.txt
cd /var/backups
get db-latest.sql.gz
$ sftp -b cmds.txt rahim@server1
```

> 💡 **টিপস:** GUI চাইলে বেশিরভাগ ফাইল ম্যানেজারে (GNOME Files, Dolphin) address bar-এ `sftp://rahim@server1/` লিখলেই রিমোট ফোল্ডার খুলে যায়। আর রিমোট ফোল্ডারকে লোকাল ফোল্ডারের মতো mount করতে `sshfs` আছে: `sshfs rahim@server1:/var/www ~/mnt/www` (খোলা: `fusermount3 -u ~/mnt/www`)।

৩৮.১৩ `rsync` over SSH — স্মার্ট সিঙ্ক

`rsync` শুধু **পরিবর্তিত অংশটুকু** পাঠায় — তাই দ্বিতীয়বার চালালে অনেক দ্রুত। রিমোট পাথ দিলে (`host:path`) ডিফল্টভাবেই SSH ব্যবহার করে। এখানে SSH-সংক্রান্ত দিকটা দেখাচ্ছি; rsync-এর পুরো গাইড অধ্যায় ৩৫-এ।

```
# লোকাল ফোল্ডার → সার্ভার
$ rsync -avz --progress ./website/ rahim@server1:/var/www/html/

# ভিন্ন port বা key — -e দিয়ে ssh কমান্ড বলে দাও
$ rsync -avz -e "ssh -p 2222 -i ~/.ssh/id_work" ./data/ deploy@web1:/srv/data/

# config-এ alias থাকলে আরও সহজ
$ rsync -avz ./data/ web1:/srv/data/

# আগে শুধু দেখো কী হবে (dry run), তারপর --delete দিয়ে হুবহু মিরর
$ rsync -avzn --delete ./website/ web1:/var/www/html/
$ rsync -avz  --delete ./website/ web1:/var/www/html/
```

অপশন	কাজ
`-a`	archive: recursive + পারমিশন, সময়, symlink ইত্যাদি রক্ষা
`-v`	verbose
`-z`	ট্রান্সফারের সময় compression
`-P`	`--partial --progress` (resume + progress)
`-n` / `--dry-run`	কিছু না করে শুধু দেখায়
`--delete`	গন্তব্যে যা উৎসে নেই তা মুছে ফেলে
`-e "ssh ..."`	কোন remote shell ও তার অপশন
`--exclude=PATTERN`	বাদ দেওয়া

> ⚠️ **সতর্কতা:** উৎসের শেষে `/` থাকা-না-থাকায় ফল আলাদা: `./website/` মানে "ফোল্ডারের **ভেতরের জিনিস**", `./website` মানে "ফোল্ডারটা **সহ**"। `--delete`-এর সাথে ভুল করলে গন্তব্যের ডেটা মুছে যেতে পারে — আগে `-n` দিয়ে দেখো।

> 📝 **মনে রাখো:** rsync-এর জন্য দুই মেশিনেই `rsync` ইনস্টল থাকতে হবে। নইলে "rsync: command not found" দেখাবে।

৩৮.১৪ Port Forwarding (SSH Tunneling)

SSH কানেকশনটাকে একটা **নিরাপদ সুড়ঙ্গ (tunnel)** হিসেবে ব্যবহার করে অন্য TCP ট্রাফিক পাঠানো যায়। এর তিন ধরন।

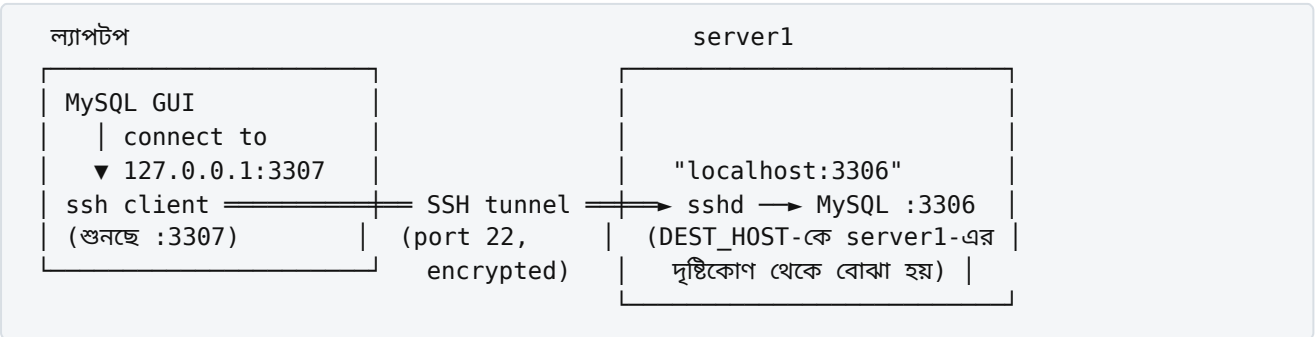
ক) Local forwarding: `-L` — "দূরের জিনিস কাছে আনো"

```
ssh -L [bind_address:]LOCAL_PORT:DEST_HOST:DEST_PORT user@ssh_server
```

উদাহরণ: server1-এ MySQL চলছে, কিন্তু শুধু `127.0.0.1:3306`-এ শোনে (বাইরে থেকে বন্ধ — নিরাপত্তার জন্য ভালো)। তুমি তোমার ল্যাপটপের GUI ক্লায়েন্ট দিয়ে সেটা ব্যবহার করতে চাও:

```
$ ssh -N -L 3307:localhost:3306 rahim@server1
```

এখন ল্যাপটপে `127.0.0.1:3307`-এ কানেক্ট করলে সেটা আসলে server1-এর MySQL-এ পৌঁছায়।



> 📝 **মনে রাখো:** `-L`-এ `DEST_HOST` টা **SSH সার্ভারের দৃষ্টিকোণ থেকে** বোঝা হয়। `localhost` মানে server1 নিজে। তুমি চাইলে server1-এর পাশের কোনো মেশিনও দিতে পারো: `-L 8080:10.0.5.20:80` — তখন server1 ঐ অভ্যন্তরীণ মেশিনে কানেক্ট করবে।

ডিফল্টভাবে লোকাল port শুধু `127.0.0.1`-এ খোলে (শুধু তোমার মেশিন ব্যবহার করতে পারে)। LAN-এর অন্যদের জন্য খুলতে চাইলে `-L 0.0.0.0:3307:localhost:3306` (সাবধানে!)।

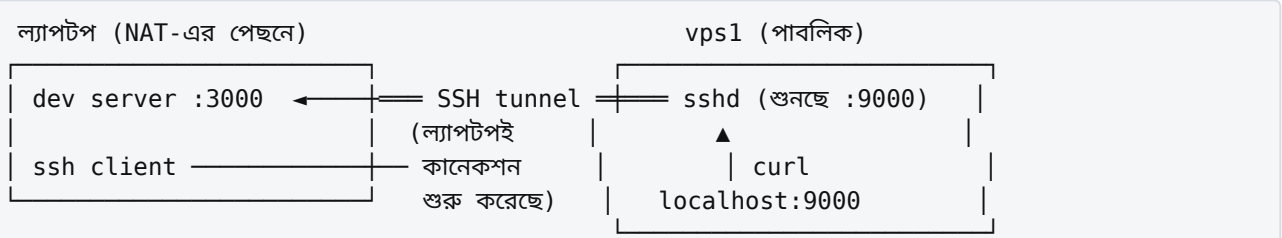
খ) Remote forwarding: `-R` — "কাছের জিনিস দূরে দেখাও"

```
ssh -R [bind_address:]REMOTE_PORT:DEST_HOST:DEST_PORT user@ssh_server
```

উদাহরণ: তোমার ল্যাপটপে `localhost:3000`-এ একটা ডেভেলপমেন্ট ওয়েবসাইট চলছে। একটা পাবলিক সার্ভার (vps1) এর মাধ্যমে বন্ধুকে দেখাতে চাও, অথচ তোমার ল্যাপটপ NAT/রাউটারের পেছনে, বাইরে থেকে সরাসরি পৌঁছানো যায় না:

```
$ ssh -N -R 9000:localhost:3000 rahim@vps1
```

এখন vps1-এ `localhost:9000`-এ গেলে সেটা তোমার ল্যাপটপের :3000-এ পৌঁছায়।



এখানে `DEST_HOST` বোঝা হয় **ক্লায়েন্টের (তোমার ল্যাপটপের) দৃষ্টিকোণ থেকে**।

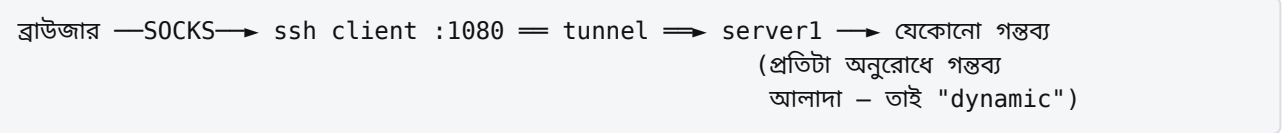
> ⚠️ **সতর্কতা:** ডিফল্টভাবে vps1-এর :9000 শুধু vps1-এর loopback-এ খোলে। পুরো ইন্টারনেটের জন্য খুলতে vps1-এর `sshd_config`-এ `GatewayPorts yes` (বা `clientspecified`) লাগবে — এবং এটা তোমার ল্যাপটপের সার্ভিস সারা দুনিয়ার কাছে উন্মুক্ত করে দেয়। ভেবেচিন্তে করো।

গ) Dynamic forwarding: `-D` — নিজের SOCKS proxy

```
$ ssh -N -D 1080 rahim@server1
```

এখন তোমার ল্যাপটপের `127.0.0.1:1080`-এ একটা **SOCKS5 proxy** চলছে। ব্রাউজারকে এই proxy ব্যবহার করতে বললে সব ওয়েব ট্রাফিক server1 হয়ে যাবে — যেন তুমি server1-এ বসে ব্রাউজ করছ। পাবলিক Wi-Fi-তে নিরাপদে ব্রাউজ করতে বা অফিসের ভেতরের ওয়েবসাইট দেখতে কাজে লাগে।

```
$ curl --socks5-hostname 127.0.0.1:1080 http://intranet.company.local/
```



তিন ধরনের তুলনা

	`-L` Local	`-R` Remote	`-D` Dynamic
port খোলে কোথায়	ক্লায়েন্টে	সার্ভারে	ক্লায়েন্টে
গন্তব্য	নির্দিষ্ট একটা (সার্ভারের দিক থেকে)	নির্দিষ্ট একটা (ক্লায়েন্টের দিক থেকে)	যেকোনো (SOCKS অনুযায়ী)
সাধারণ ব্যবহার	রিমোট DB/অ্যাডমিন প্যানেল	NAT-এর পেছনের সার্ভিস বাইরে দেখানো	proxy / নিরাপদ ব্রাউজিং

সাথে দরকারি অপশন

```
# -N: শেল খুলবে না, -f: background-এ চলে যাবে
$ ssh -fN -L 8080:localhost:80 rahim@server1

# পরে বন্ধ করতে
$ pgrep -af 'ssh -fN -L 8080'
48211 ssh -fN -L 8080:localhost:80 rahim@server1
$ kill 48211
```

`~/.ssh/config`-এ স্থায়ীভাবে:

```
Host dbtunnel
    HostName server1.example.com
    User rahim
    LocalForward 3307 localhost:3306
    ExitOnForwardFailure yes
```

তারপর শুধু `ssh -N dbtunnel`।

> 💡 **টিপস:** সার্ভারের `sshd_config`-এ `AllowTcpForwarding no` থাকলে কোনো forwarding কাজ করবে না। আর 1024-এর নিচের লোকাল port (যেমন 80) bind করতে root লাগে — তাই 8080, 3307 ধরনের উঁচু port ব্যবহার করো।

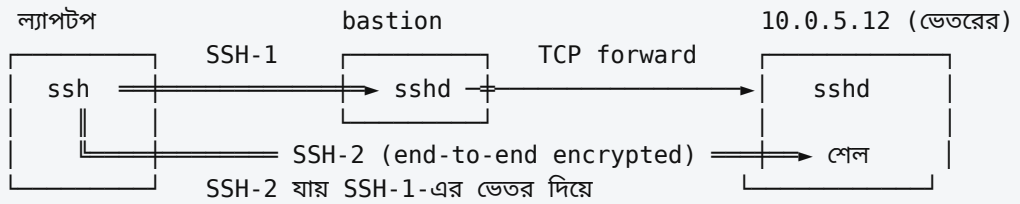
৩৮.১৫ Jump Host / Bastion: `-J`

বড় প্রতিষ্ঠানে অভ্যন্তরীণ সার্ভারগুলো ইন্টারনেট থেকে সরাসরি পৌঁছানো যায় না। একটাই প্রবেশদ্বার থাকে — **bastion** বা **jump host**। আগে সেখানে লগইন করে, তারপর ভেতরে যেতে হতো:

```
$ ssh rahim@bastion        # ধাপ ১
rahim@bastion:~$ ssh rahim@10.0.5.12    # ধাপ ২ — key বা agent bastion-এ লাগবে, ঝামেলা!
```

`-J` (ProxyJump) একবারে করে দেয়:

```
$ ssh -J rahim@bastion.example.com rahim@10.0.5.12

# একাধিক jump, কমা দিয়ে
$ ssh -J user@jump1,user@jump2 user@target

# scp/rsync-ও একইভাবে
$ scp -J bastion file.txt 10.0.5.12:/tmp/
$ rsync -av -e "ssh -J bastion" ./app/ 10.0.5.12:/srv/app/
```



কীভাবে কাজ করে: ক্লায়েন্ট প্রথমে bastion-এ কানেক্ট করে, bastion-কে বলে "10.0.5.12:22-এ একটা TCP সংযোগ জুড়ে দাও" (local forwarding-এর মতো)। তারপর ঐ পাইপের ভেতর দিয়ে **তোমার ল্যাপটপ থেকে সরাসরি** target-এর সাথে আলাদা SSH session চালায়। ফলাফল:

- Authentication দুটোই তোমার ল্যাপটপের key দিয়ে হয় — bastion-এ কোনো private key রাখতে হয় না।
- bastion শুধু encrypted বাইট পার করে, ভেতরের সেশন পড়তে পারে না।
- `-A` (agent forwarding)-এর ঝুঁকি নেই।

`~/.ssh/config`-এ:

```
Host bastion
    HostName bastion.example.com
    User rahim

Host 10.0.5.* app-*
    User rahim
    ProxyJump bastion
```

এখন `ssh 10.0.5.12` নিজে থেকেই bastion হয়ে যাবে। (bastion-এ `AllowTcpForwarding` চালু থাকতে হবে।)

> 💡 **টিপস:** পুরোনো পদ্ধতি `ProxyCommand ssh -W %h:%p bastion` — এটাও একই কাজ করে, পুরোনো config-এ দেখতে পাবে।

৩৮.১৬ SSH Escape Sequences — জমে যাওয়া সেশন থেকে মুক্তি

সার্ভার রিবুট হলো বা Wi-Fi কেটে গেল — তোমার টার্মিনাল জমে আছে, `Ctrl+C` কাজ করছে না (কারণ `Ctrl+C` রিমোটে পাঠানো হয়, আর রিমোট সাড়া দিচ্ছে না)। কী করবে?

Enter চাপো, তারপর `~` তারপর `.` — কানেকশন সাথে সাথে কেটে যাবে।

ssh ক্লায়েন্ট তোমার কীবোর্ড ইনপুটের দিকে নজর রাখে: **নতুন লাইনের একদম শুরুতে** `~` টাইপ করলে সেটা রিমোটে না পাঠিয়ে ক্লায়েন্টের নিজের জন্য একটা কমান্ড হিসেবে ধরে নেয়। এজন্যই আগে Enter চাপতে হয়।

Sequence	কাজ
`~.`	কানেকশন তৎক্ষণাৎ কেটে দাও (জমে যাওয়া সেশনের ওষুধ)
`~?`	সব escape sequence-এর তালিকা দেখাও
`~^Z` (`~` তারপর `Ctrl+Z`)	ssh নিজেকে suspend করো (লোকাল শেলে ফেরো; `fg` দিয়ে ফেরত)
`~&`	ssh-কে background-এ পাঠাও (forwarded কানেকশন শেষ হওয়ার অপেক্ষায় থাকলে)
`~#`	চলমান forwarded কানেকশনের তালিকা
`~C`	ssh কমান্ড লাইন খোলো — চলমান সেশনে নতুন forward যোগ করা যায় (যেমন `-L 8080:localhost:80`)
`~R`	rekey (নতুন session key) অনুরোধ
`~V` / `~v`	ক্লায়েন্টের log verbosity কমাও / বাড়াও
`~B`	রিমোটে BREAK পাঠাও
`~~`	আসল একটা `~` অক্ষর পাঠাও

```
rahim@server1:~$          ← (নেটওয়ার্ক কেটে গেছে, কিছুই সাড়া দিচ্ছে না)
[Enter] ~ .
Connection to server1 closed.
rahim@laptop:~$
```

নেস্টেড SSH (ssh-এর ভেতরে ssh): প্রতিটা `~` প্রথম ssh ক্লায়েন্ট খেয়ে ফেলে। ভেতরের সেশন কাটতে `~~.` দাও (প্রথম ক্লায়েন্ট `~~`-কে একটা `~` হিসেবে পাঠায়, দ্বিতীয় ক্লায়েন্ট তারপর `~.` পায়)।

> 📝 **মনে রাখো:** OpenSSH 9.2 থেকে `~C` কমান্ড লাইন ডিফল্টভাবে বন্ধ; ব্যবহার করতে ক্লায়েন্ট config-এ `EnableEscapeCommandline yes` দাও। escape character বদলাতে `-e` অপশন (`ssh -e '^' host`), পুরো বন্ধ করতে `-e none`।

৩৮.১৭ আরও কিছু দরকারি জিনিস

সংযোগ টিকিয়ে রাখা

NAT রাউটার বা ফায়ারওয়াল idle কানেকশন কিছুক্ষণ পর ভুলে যায়, ফলে সেশন জমে যায়। ক্লায়েন্ট সাইডে:

```
Host *
    ServerAliveInterval 60      # প্রতি ৬০ সেকেন্ডে encrypted "আছো?" বার্তা
    ServerAliveCountMax 3       # ৩ বার সাড়া না পেলে কানেকশন বন্ধ
```

দুর্বল/মোবাইল নেটওয়ার্কের জন্য `mosh` নামে আলাদা টুল আছে (SSH দিয়ে লগইন করে, তারপর UDP-তে চলে; নেটওয়ার্ক বদলালেও সেশন টেকে)। আর কানেকশন কাটলেও কাজ চালু রাখতে সার্ভারে `tmux` চালাও — অধ্যায় ২৫।

X11 forwarding — রিমোট GUI

```
$ ssh -X rahim@server1 gedit
```

রিমোটের GUI অ্যাপ তোমার স্ক্রিনে আসে। সার্ভারে `X11Forwarding yes` আর `xauth` প্যাকেজ লাগবে। `-Y` হলো "trusted" মোড — কম সীমাবদ্ধ, কম নিরাপদ।

Windows ও অন্যান্য

Windows 10/11-এ OpenSSH ক্লায়েন্ট বিল্ট-ইন — PowerShell-এ `ssh user@host` সরাসরি চলে। macOS-এও একই OpenSSH।

SSH certificate (অ্যাডভান্সড ঝলক)

হাজারো সার্ভারে প্রতিটার `authorized_keys`-এ key বসানো ঝামেলা। বড় প্রতিষ্ঠানে একটা **SSH CA (Certificate Authority)** key দিয়ে ইউজারের key-তে সময়সীমা-সহ সই করা হয় (`ssh-keygen -s ca_key -I id -n rahim -V +1d id_ed25519.pub`), আর সার্ভার শুধু CA-কে বিশ্বাস করে (`TrustedUserCAKeys`)। এতে key বিতরণ ও বাতিল দুটোই সহজ।

হাতে-কলমে অনুশীলন

> দুটো মেশিন লাগবে: একটা লিনাক্স VM (VirtualBox/KVM, অধ্যায় ৪৮) বা একটা সস্তা ক্লাউড VPS হলে চলবে। নিচে "ক্লায়েন্ট" = তোমার মেশিন, "সার্ভার" = VM, ধরে নিচ্ছি সার্ভারের IP `192.168.56.10`, ইউজার `student`।

ল্যাব ১: সার্ভার প্রস্তুত ও প্রথম লগইন

```
# সার্ভারে
$ sudo apt install openssh-server        # বা: sudo dnf install openssh-server
$ sudo systemctl enable --now ssh        # RHEL/Fedora-তে sshd
$ ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub   # fingerprint লিখে রাখো
$ ip -4 addr                             # IP দেখো

# ক্লায়েন্টে
$ ssh student@192.168.56.10              # fingerprint মিলিয়ে yes
$ ssh student@192.168.56.10 'hostname; uptime'
```

ল্যাব ২: Key-based লগইন

```
# ক্লায়েন্টে
$ ssh-keygen -t ed25519 -C "student-lab" -f ~/.ssh/id_lab
$ ssh-copy-id -i ~/.ssh/id_lab.pub student@192.168.56.10
$ ssh -i ~/.ssh/id_lab student@192.168.56.10     # passphrase চাইবে, পাসওয়ার্ড না
$ eval "$(ssh-agent -s)"; ssh-add ~/.ssh/id_lab
$ ssh -i ~/.ssh/id_lab student@192.168.56.10     # এবার কিছুই চাইবে না
```

ল্যাব ৩: config ফাইল

`~/.ssh/config`-এ লেখো:

```
Host lab
    HostName 192.168.56.10
    User student
    IdentityFile ~/.ssh/id_lab
    IdentitiesOnly yes
```

```
$ chmod 600 ~/.ssh/config
$ ssh lab
$ ssh -G lab | grep -E '^(user|hostname|identityfile) '
```

ল্যাব ৪: ইচ্ছাকৃত পারমিশন ভাঙা ও সারানো

```
$ ssh lab 'chmod 777 ~/.ssh'
$ ssh -v lab 2>&1 | grep -i -E 'offering|password'   # key প্রত্যাখ্যাত, পাসওয়ার্ড চাইবে
# পাসওয়ার্ড দিয়ে ঢুকে সার্ভারের লগ দেখো:
$ sudo journalctl -u ssh -n 5          # "bad ownership or modes" খোঁজো
$ chmod 700 ~/.ssh                     # সারাও, তারপর বের হয়ে আবার key-তে লগইন পরীক্ষা
```

ল্যাব ৫: হার্ডেনিং (পুরোনো সেশন খোলা রেখে!)

```
# সার্ভারে
$ echo -e "PermitRootLogin no\nPasswordAuthentication no\nKbdInteractiveAuthentication
no" \
```

```
  | sudo tee /etc/ssh/sshd_config.d/00-hardening.conf
$ sudo sshd -t && sudo sshd -T | grep -E '^(permitrootlogin|passwordauthentication) '
$ sudo systemctl reload ssh

# ক্লায়েন্টে, নতুন টার্মিনাল থেকে
$ ssh lab 'echo still-works'
$ ssh -o PubkeyAuthentication=no student@192.168.56.10
student@192.168.56.10: Permission denied (publickey).    # পাসওয়ার্ড লগইন বন্ধ, প্রমাণিত
```

ল্যাব ৬: ফাইল ট্রান্সফার

```
$ mkdir -p ~/demo && for i in 1 2 3; do echo "file $i" > ~/demo/f$i.txt; done
$ scp -r ~/demo lab:/tmp/
$ sftp lab          # ভেতরে: cd /tmp/demo, ls, get f1.txt, bye
$ echo "changed" >> ~/demo/f2.txt
$ rsync -avz ~/demo/ lab:/tmp/demo/    # লক্ষ করো, শুধু f2.txt যাচ্ছে
```

ল্যাব ৭: Tunnel

```
# সার্ভারে একটা ছোট ওয়েব সার্ভার, শুধু localhost-এ
$ python3 -m http.server 8000 --bind 127.0.0.1

# ক্লায়েন্টে
$ curl -m 3 http://192.168.56.10:8000/      # ব্যর্থ হবে — বাইরে থেকে বন্ধ
$ ssh -fN -L 9000:localhost:8000 lab
$ curl http://localhost:9000/               # কাজ করছে! টানেল দিয়ে
$ pkill -f 'ssh -fN -L 9000'
```

ল্যাব ৮: Escape sequence

```
$ ssh lab
student@server:~$          # এখন Enter, তারপর ~?  — তালিকা দেখো
student@server:~$          # Enter, তারপর ~.      — কানেকশন কেটে যাবে
```

সারসংক্ষেপ

কমান্ড / শর্টকাট	কাজ
`ssh user@host`	রিমোট লগইন
`ssh -p 2222 user@host`	ভিন্ন port
`ssh host 'cmd'` / `ssh -t host cmd`	রিমোট কমান্ড / interactive কমান্ড
`ssh -v host`	ডিবাগ
`ssh -G host`	চূড়ান্ত config দেখা
`ssh-keygen -t ed25519 -C "comment"`	নতুন key

কমান্ড / শর্টকাট	কাজ
`ssh-keygen -p -f key`	passphrase বদলানো
`ssh-keygen -lf file.pub`	fingerprint
`ssh-keygen -R host`	known_hosts থেকে host মুছে ফেলা
`ssh-keyscan host`	host key আনা
`ssh-copy-id -i key.pub user@host`	public key সার্ভারে বসানো
`eval "$(ssh-agent -s)"`	agent চালু
`ssh-add`, `ssh-add -l`, `ssh-add -D`	key যোগ / তালিকা / সব মোছা
`sudo sshd -t` / `sudo sshd -T`	সার্ভার config পরীক্ষা / কার্যকর মান
`systemctl reload ssh` / `sshd`	config প্রয়োগ (Ubuntu / RHEL)
`scp [-r] [-P port] src user@host:dst`	ফাইল কপি
`sftp [-P port] user@host`	ইন্টারঅ্যাকটিভ ট্রান্সফার
`rsync -avz -e "ssh -p 2222" src/ host:dst/`	সিঙ্ক over SSH
`ssh -L lport:dhost:dport host`	local forward
`ssh -R rport:dhost:dport host`	remote forward
`ssh -D 1080 host`	SOCKS proxy
`ssh -fN ...`	শুধু টানেল, background-এ
`ssh -J jump target`	jump host
`Enter ~.`	জমে যাওয়া সেশন কাটা
`Enter ~?`	escape তালিকা
`Enter ~^Z`	ssh suspend
`Enter ~#`	forwarded কানেকশন তালিকা
`Ctrl+D` / `exit`	লগআউট

পারমিশনের চাবি-সংখ্যা: `~/.ssh` = 700, `authorized_keys` = 600, private key = 600, `config` = 600, `.pub` = 644, home-এ group/other লিখতে পারবে না।

নিজেকে যাচাই করো

1. SSH-এ key exchange কী সমস্যার সমাধান করে? Private key কি কখনো নেটওয়ার্কে যায়?
2. `known_hosts` আর `authorized_keys` — কোনটা কোথায় থাকে, কার পরিচয় রাখে?
3. "REMOTE HOST IDENTIFICATION HAS CHANGED" দেখলে প্রথমে কী করবে, আর নিশ্চিত হওয়ার পর কোন কমান্ড চালাবে?

4. তুমি `/etc/ssh/sshd_config`-এ `PasswordAuthentication no` দিয়েছ, reload করেছ, তবুও পাসওয়ার্ড লগইন হচ্ছে। সম্ভাব্য কারণ কী, কীভাবে যাচাই করবে?
5. `ssh -p` আর `scp -P` — পার্থক্য কী? `scp -p` কী করে?
6. সার্ভারের PostgreSQL শুধু `127.0.0.1:5432`-এ শোনে। তোমার ল্যাপটপের :5433 থেকে সেটা ব্যবহার করার কমান্ড লেখো।
7. `-L`, `-R`, `-D`-এর মধ্যে কোনটায় port সার্ভারে খোলে?
8. Key ঠিকমতো বসানো, তবুও "Permission denied (publickey)"। কোন তিনটা জিনিস প্রথমে দেখবে?
9. নেটওয়ার্ক কেটে গেছে, SSH টার্মিনাল জমে আছে। কী চাপবে, আর কেন আগে Enter চাপতে হয়?
10. `ssh -A` দিয়ে bastion হয়ে যাওয়ার চেয়ে `ssh -J` কেন নিরাপদ?

▼ **উত্তর দেখো**

1. খোলা নেটওয়ার্কে আগে কোনো গোপন কিছু ভাগ না করেও দুই পক্ষ এমন একটা shared secret বানায় যা মাঝখানের কেউ জানতে পারে না। সেখান থেকে session key তৈরি হয়। না, private key কখনো নেটওয়ার্কে যায় না, শুধু তা দিয়ে বানানো signature যায়।
2. `known_hosts` থাকে **ক্লায়েন্টে** (`~/.ssh/known_hosts`), সার্ভারের host key রাখে, মানে সার্ভারের পরিচয়। `authorized_keys` থাকে **সার্ভারে**, যে ইউজারের অ্যাকাউন্টে ঢুকবে তার `~/.ssh/authorized_keys`-এ। এটা লগইন করার অনুমতিপ্রাপ্ত ইউজারদের public key রাখে।
3. প্রথমে নিশ্চিত হও কারণটা নিরীহ (সার্ভার রিইনস্টল, নতুন VM, অ্যাডমিন key বদলেছে)। সম্ভব হলে কনসোল থেকে নতুন fingerprint মিলিয়ে নাও। তারপর `ssh-keygen -R host` চালাও আর আবার কানেক্ট করে নতুন fingerprint যাচাই করো।
4. `sshd_config`-এর উপরের `Include /etc/ssh/sshd_config.d/*.conf` ফাইলগুলো আগে পড়া হয়, আর প্রথম মানই কার্যকর থাকে। যেমন `50-cloud-init.conf`-এ `PasswordAuthentication yes` থাকতে পারে। `sudo sshd -T | grep passwordauthentication` দিয়ে কার্যকর মান দেখো, `grep -r PasswordAuthentication /etc/ssh/` দিয়ে উৎস খোঁজো। সমাধান হলো `00-*.conf` ফাইলে সেটিং রাখা।
5. দুটোই port নির্দেশ করে, শুধু অক্ষরের কেস আলাদা: ssh-এ ছোট হাতের `-p`, scp/sftp-তে বড় হাতের `-P`। `scp -p` modification time ও mode রক্ষা করে।
6. `ssh -N -L 5433:localhost:5432 user@server`। এরপর ল্যাপটপে `127.0.0.1:5433`-এ কানেক্ট করো।
7. `-R`। Remote forwarding-এ port SSH সার্ভারে খোলে। `-L` আর `-D` খোলে ক্লায়েন্টে।
8. (ক) সার্ভারে `~/.ssh` 700 আর `authorized_keys` 600 কিনা, home ডিরেক্টরি group/other-writable কিনা, মালিক ঠিক আছে কিনা। (খ) `ssh -v` দিয়ে দেখো সঠিক key offer করা হচ্ছে কিনা আর ইউজারনেম ঠিক আছে কিনা। (গ) সার্ভারের লগ দেখো (`journalctl -u ssh` / `sshd`, `/var/log/auth.log` / `secure`)। RHEL-এ SELinux context-ও দেখো (`restorecon -Rv ~/.ssh`)।
9. `Enter`, তারপর `~`, তারপর `.`। ssh ক্লায়েন্ট escape character শুধু নতুন লাইনের শুরুতে চেনে, তাই আগে Enter দিতে হয়।

10. `-A` দিলে bastion-এর root বা আক্রমণকারী কানেকশন চলাকালীন তোমার agent socket ব্যবহার করে তোমার নামে অন্য সার্ভারে লগইন করতে পারে। `-J`-এ bastion শুধু TCP বাইট পার করে। ভেতরের SSH session তোমার ল্যাপটপ থেকে target পর্যন্ত end-to-end encrypted থাকে, আর authentication-ও তোমার ল্যাপটপেই হয়।

অধ্যায় ৩৯: ফায়ারওয়াল — ufw, firewalld, iptables ও nftables

এই অধ্যায়ে যা শিখবে

- ফায়ারওয়াল (firewall) আসলে কী, কেন লাগে, আর কোনটা ফায়ারওয়াল পারে না
- লিনাক্স কার্নেলের ভেতরের **netfilter** ফ্রেমওয়ার্ক: hooks, tables, chains — ডায়াগ্রাম সহ
- একটা প্যাকেট কম্পিউটারে ঢুকে কোন পথে কোথায় যায় (packet flow)
- **stateful** ফায়ারওয়াল আর **conntrack** (connection tracking) কীভাবে কাজ করে
- **iptables**: INPUT/OUTPUT/FORWARD chain, `-A -I -D -L -n -v`, ACCEPT/DROP/REJECT, রুল সেভ করা
- **nftables** (`nft`): আধুনিক সিনট্যাক্স, set, handle দিয়ে রুল মোছা, কনফিগ ফাইল
- **ufw** (Ubuntu/Debian): enable, allow, deny, limit, status numbered, delete, app profiles
- **firewalld** (Fedora/RHEL): zones, services, `--permanent`, `--reload`, rich rules
- **NAT** আর **port forwarding** — iptables, nftables, ufw ও firewalld চারটাতেই
- রিমোট সার্ভারে ফায়ারওয়াল চালু করতে গিয়ে **নিজেকে লক-আউট না করার** কৌশল
- কোন টুল কখন ব্যবহার করবে, আর কোন দুটো একসাথে চালানো বিপজ্জনক

৩৯.১ ফায়ারওয়াল কী এবং কেন?

একটা উপমা দিয়ে শুরু করি

ধরো তুমি একটা বড় অফিস বিল্ডিংয়ের সিকিউরিটি গার্ড। বিল্ডিংয়ে অনেকগুলো রুম — রুম ২২-এ IT টিম (SSH), রুম ৮০ আর ৪৪৩-এ রিসেপশন (ওয়েবসাইট), রুম ৩৩০৬-এ হিসাব বিভাগ (MySQL ডাটাবেস)। তোমার হাতে একটা **রুল বুক**:

1. "রিসেপশনে (৮০, ৪৪৩) যে কেউ আসতে পারবে।"
2. "IT রুমে (২২) শুধু অফিসের নিজের লোক (নির্দিষ্ট IP) আসতে পারবে।"
3. "হিসাব বিভাগে (৩৩০৬) বাইরের কেউ ঢুকবে না।"
4. "যে ভেতর থেকে বের হয়ে গিয়েছিল, সে ফিরে আসলে ঢুকতে দাও।"
5. "বাকি সবাইকে — চুপচাপ ফিরিয়ে দাও।"

প্রত্যেক আগন্তুককে তুমি রুল বুকের **উপর থেকে নিচে** মিলিয়ে দেখো; প্রথম যে রুলটা মিলে যায়, সেটাই সিদ্ধান্ত। কোনো রুল না মিললে শেষ নিয়ম (default policy) খাটে।

এই গার্ডই হলো **ফায়ারওয়াল**। আর আগন্তুকরা হলো **নেটওয়ার্ক প্যাকেট** (packet — নেটওয়ার্কে পাঠানো ডেটার ছোট ছোট টুকরো)।

সংজ্ঞা

ফায়ারওয়াল হলো এমন একটা ব্যবস্থা যা নেটওয়ার্ক ট্রাফিক (আসা-যাওয়া করা প্যাকেট) পরীক্ষা করে কিছু নিয়মের ভিত্তিতে সিদ্ধান্ত নেয় — প্যাকেটটা যেতে দেবে, ফেলে দেবে, নাকি ফেরত পাঠাবে।

প্যাকেটের যেসব তথ্য দেখে সিদ্ধান্ত নেওয়া হয় (নেটওয়ার্কিং-এর বেসিক জানতে অধ্যায় ৩৬ দেখো):

তথ্য	উদাহরণ
Source IP (কোথা থেকে এসেছে)	`203.0.113.5`
Destination IP (কোথায় যাবে)	`192.168.1.10`
Protocol	TCP, UDP, ICMP
Source/Destination port	`22`, `80`, `443`, `53`
Interface (কোন নেটওয়ার্ক কার্ড দিয়ে)	`eth0`, `wlan0`, `lo`
Connection state	নতুন কানেকশন, নাকি পুরনো কানেকশনের অংশ

কেন ফায়ারওয়াল লাগে?

- **Attack surface কমানো:** তোমার সার্ভারে হয়তো ভুল করে একটা ডাটাবেস বা ডিবাগ সার্ভার সব IP-তে (`0.0.0.0`) listen করছে। ফায়ারওয়াল থাকলে সেটা ইন্টারনেট থেকে দেখা যাবে না।
- **Brute-force কমানো:** SSH-এ সেকেন্ডে শত শত লগইন চেষ্টা আসে। rate-limit রুল দিয়ে সেটা ধীর করা যায়।
- **সেগমেন্টেশন:** ডাটাবেস সার্ভারে শুধু অ্যাপ সার্ভার থেকেই কানেকশন আসবে, আর কোথাও থেকে না।
- **রাউটার/গেটওয়ে বানানো:** লিনাক্স মেশিনকে রাউটার বানিয়ে NAT করে পুরো LAN-কে ইন্টারনেট দেওয়া যায়।

> 📝 **মনে রাখো:** ফায়ারওয়াল কোনো সার্ভিসের **ভেতরের** দুর্বলতা ঠিক করে না। পোর্ট ৮০ খোলা থাকলে ওয়েব অ্যাপের SQL injection ফায়ারওয়াল আটকাবে না। ফায়ারওয়াল হলো নিরাপত্তার একটা স্তর মাত্র — বাকি স্তরগুলো অধ্যায় ৪৫-এ।

ফায়ারওয়ালের ধরন

ধরন	কী দেখে	উদাহরণ
Packet filter (stateless)	প্রতিটা প্যাকেট আলাদাভাবে, শুধু হেডার	পুরনো ACL, সাধারণ রাউটার
Stateful firewall	প্যাকেট কোন কানেকশনের অংশ সেটাও মনে রাখে	netfilter + conntrack (লিনাক্স)

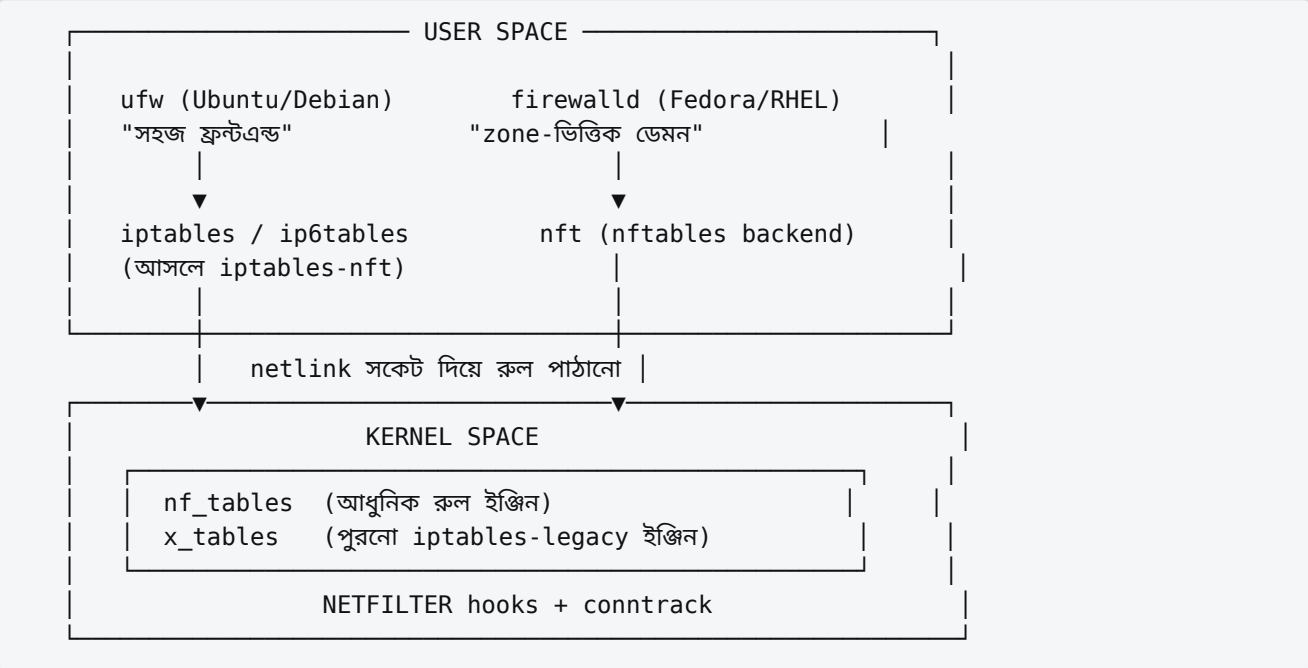
ধরন	কী দেখে	উদাহরণ
Application firewall / WAF	HTTP-এর ভেতরের ডেটা	ModSecurity, Cloudflare WAF
Host-based	নিজের মেশিনকে রক্ষা করে	ufw, firewalld
Network-based	পুরো নেটওয়ার্ককে রক্ষা করে	লিনাক্স রাউটার, pfSense, ক্লাউডের Security Group

লিনাক্সের ফায়ারওয়াল মূলত **stateful packet filter**, যেটা host-based আর network-based দুইভাবেই ব্যবহার করা যায়।

৩৯.২ বড় ছবি: টুলগুলোর পারস্পরিক সম্পর্ক

নতুনরা সবচেয়ে বেশি যেটায় গুলিয়ে ফেলে: "iptables, nftables, ufw, firewalld — এগুলো কি চারটা আলাদা ফায়ারওয়াল?"

না। আসল ফায়ারওয়াল একটাই — লিনাক্স **কার্নেলের ভেতরে** থাকা **netfilter**। বাকি সব হলো ইউজার-স্পেসের (user space — সাধারণ প্রোগ্রাম যেখানে চলে) টুল, যেগুলো কার্নেলকে রুল লিখে দেয়।



উপমা: netfilter হলো গাড়ির **ইঞ্জিন**। iptables আর nftables হলো দুই রকম **স্টিয়ারিং-গিয়ার** (ম্যানুয়াল কন্ট্রোল)। ufw আর firewalld হলো **অটোমেটিক গিয়ার** — সহজ, কিন্তু ভেতরে একই ইঞ্জিন চালায়।

আধুনিক ডিস্ট্রোতে কী ডিফল্ট?

ডিস্ট্রো	ডিফল্ট ফ্রন্টএন্ড	ব্যাকএন্ড
Ubuntu 24.04	ufw (ইনস্টলড, কিন্তু **disabled**)	iptables-nft → nf_tables
Debian 12	কিছুই চালু নেই (ufw/firewalld ইনস্টল করতে হয়)	nftables; `iptables` কমান্ড = iptables-nft
Fedora	firewalld (চালু)	nftables
RHEL 9 / Rocky / Alma	firewalld (চালু)	nftables; `iptables` = iptables-nft

চেক করো তোমার `iptables` কোন ব্যাকএন্ড:

```
$ iptables --version
iptables v1.8.10 (nf_tables)
```

`(nf_tables)` মানে iptables সিনট্যাক্সে লিখছ কিন্তু রুলগুলো আসলে nftables ইঞ্জিনে যাচ্ছে। `(legacy)` লেখা থাকলে পুরনো x_tables ইঞ্জিন।

> ⚠️ **সতর্কতা:** একই মেশিনে **ufw আর firewalld একসাথে চালু রাখবে না।** দুটোই নিজের মতো রুল লেখে, একে অপরের রুল মুছে দিতে পারে বা অপ্রত্যাশিত আচরণ হয়। একটা বেছে নাও, অন্যটা `systemctl disable --now` করো। একইভাবে ufw/firewalld চালু থাকা অবস্থায় হাতে iptables/nft রুল দিলে reload-এর সময় সেগুলো হারিয়ে যেতে পারে।

৩৯.৩ Netfilter আর্কিটেকচার — ভেতরে কী হয়

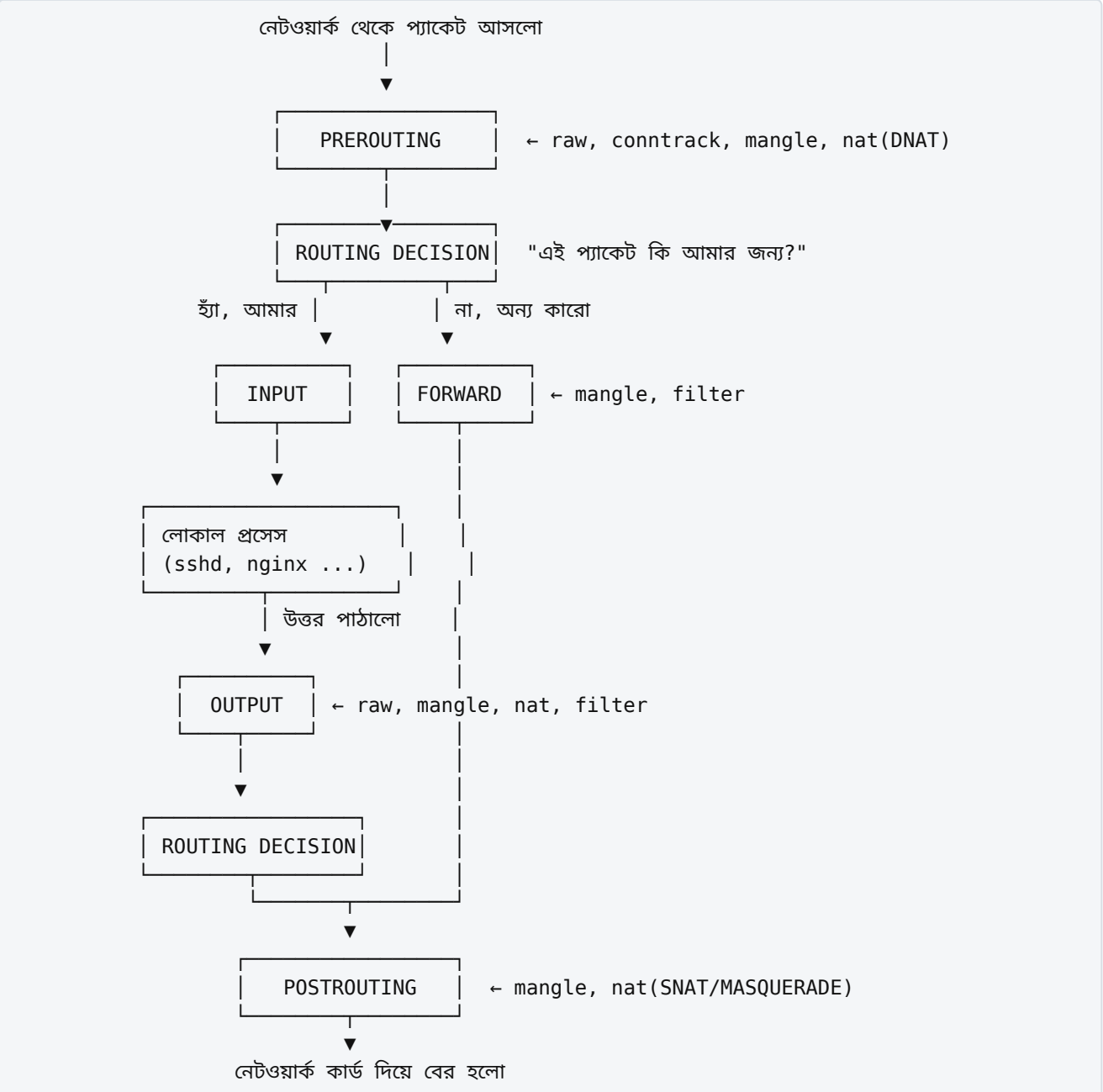
Hooks: প্যাকেটের পথে পাঁচটা চেকপোস্ট

কার্নেলের নেটওয়ার্ক স্ট্যাকে প্যাকেট যে পথ দিয়ে চলে, সেই পথে netfilter পাঁচটা জায়গায় "হুক" (hook — আঁকড়া, যেখানে নিজের কোড ঝুলিয়ে দেওয়া যায়) বসিয়েছে। প্রতিটা হুকে প্যাকেট থামে, রুল মিলিয়ে দেখা হয়, তারপর এগোয়।

Hook	কখন প্যাকেট এখানে আসে
PREROUTING	প্যাকেট নেটওয়ার্ক কার্ডে ঢোকার সাথে সাথে, routing সিদ্ধান্তের **আগে**
INPUT	routing ঠিক করেছে প্যাকেটটা **এই মেশিনের জন্যই** — লোকাল প্রসেসে পৌঁছানোর আগে
FORWARD	প্যাকেটটা অন্য কারো জন্য, এই মেশিন শুধু **পার করে দিচ্ছে** (রাউটার হিসেবে)
OUTPUT	এই মেশিনের কোনো লোকাল প্রসেস প্যাকেট **পাঠাচ্ছে**
POSTROUTING	প্যাকেট নেটওয়ার্ক কার্ড দিয়ে **বের হওয়ার ঠিক আগে**

(nftables-এ এর বাইরে `ingress` নামে আরও একটা আগের হুক আছে, যা ইন্টারফেস-স্তরে কাজ করে — অ্যাডভান্সড কাজে লাগে।)

Packet flow ডায়াগ্রাম



তিনটা পথ মনে রাখো:

1. **আমার কাছে আসা প্যাকেট:** PREROUTING → INPUT → লোকাল প্রসেস
2. **আমার পাঠানো প্যাকেট:** লোকাল প্রসেস → OUTPUT → POSTROUTING
3. **আমার ওপর দিয়ে পার হওয়া প্যাকেট:** PREROUTING → FORWARD → POSTROUTING

> 💡 **টিপস:** সাধারণ সার্ভার সুরক্ষায় ৯০% কাজ হয় **INPUT** chain-এ। FORWARD লাগে

> রাউটার/Docker/VM হোস্টে। PREROUTING/POSTROUTING লাগে NAT-এ।

Tables: কাজ অনুযায়ী রুলের ভাগ

iptables-এ রুলগুলো কাজের ধরন অনুযায়ী আলাদা **table**-এ রাখা হয়। প্রতিটা table কিছু নির্দিষ্ট হুকে বসে।

Table	কাজ	যে chain গুলো আছে
filter (ডিফল্ট)	প্যাকেট যেতে দেবে কি না	INPUT, FORWARD, OUTPUT
nat	ঠিকানা বদলানো (NAT)	PREROUTING, INPUT, OUTPUT, POSTROUTING
mangle	প্যাকেট হেডার বদলানো (TTL, TOS, mark)	পাঁচটাই
raw	conntrack থেকে বাদ দেওয়া (`NOTRACK`)	PREROUTING, OUTPUT
security	SELinux লেবেলিং	INPUT, FORWARD, OUTPUT

একই হুকে একাধিক table থাকলে ক্রম: **raw** → **(conntrack)** → **mangle** → **nat** → **filter**। (nat-এর ক্রম হুক অনুযায়ী একটু আলাদা — PREROUTING-এ DNAT filter-এর আগে, POSTROUTING-এ SNAT সবার শেষে।)

Chains আর Rules

- **Chain** = রুলের একটা ক্রমবদ্ধ তালিকা। বিল্ট-ইন chain (INPUT ইত্যাদি) হুকের সাথে যুক্ত; তুমি নিজেও user-defined chain বানাতে পারো (যেমন `SSH_RULES`) এবং সেখানে `-j` দিয়ে লাফ দিতে পারো।
- **Rule** = "যদি এই শর্ত মেলে (match), তাহলে এই কাজ করো (target)"।
- রুল **উপর থেকে নিচে** মেলানো হয়। প্রথম terminating target (ACCEPT/DROP/REJECT) পেলেই থামে।
- কোনো রুল না মিললে chain-এর **policy** খাটে (ACCEPT বা DROP)।

```
INPUT chain (policy: DROP)
┌───┬──────────────────────────────────────┬──────────┐
│ # │ match (শর্ত)                         │ target   │
├───┼──────────────────────────────────────┼──────────┤
│ 1 │ interface lo                         │ ACCEPT   │ ← প্যাকেট এখানে মিললে থামে
│ 2 │ state ESTABLISHED,RELATED            │ ACCEPT   │
│ 3 │ tcp dport 22                         │ ACCEPT   │
│ 4 │ tcp dport 80,443                     │ ACCEPT   │
└───┴──────────────────────────────────────┴──────────┘
  কিছুই মেলেনি? → policy DROP
```

Targets (কাজ)

Target	কী করে	ক্লায়েন্ট কী দেখে
`ACCEPT`	যেতে দাও	কানেকশন হয়

Target	কী করে	ক্লায়েন্ট কী দেখে
`DROP`	চুপচাপ ফেলে দাও, কোনো উত্তর না	timeout পর্যন্ত অপেক্ষা ("hang")
`REJECT`	ফেলে দাও এবং ICMP/TCP RST দিয়ে জানিয়ে দাও	সাথে সাথে "Connection refused"
`LOG`	কার্নেল লগে লিখে **পরের রুলে যাও** (non-terminating)	—
`RETURN`	এই chain ছেড়ে আগের chain-এ ফিরে যাও	—
`DNAT` / `SNAT` / `MASQUERADE` / `REDIRECT`	ঠিকানা বদলাও (nat table)	—
`<chain-name>`	user-defined chain-এ লাফ দাও	—

> 💡 **DROP না REJECT?** ইন্টারনেট-মুখী পোর্টে অনেকে DROP পছন্দ করে (স্ক্যানার ধীর হয়)। ভেতরের নেটওয়ার্কে REJECT ভালো — ভুল কনফিগ হলে ক্লায়েন্ট সাথে সাথে বুঝতে পারে, অযথা timeout-এ বসে থাকে না। দুটোই নিরাপত্তার দিক থেকে মোটামুটি সমান।

৩৯.৪ Stateful ফায়ারওয়াল ও conntrack

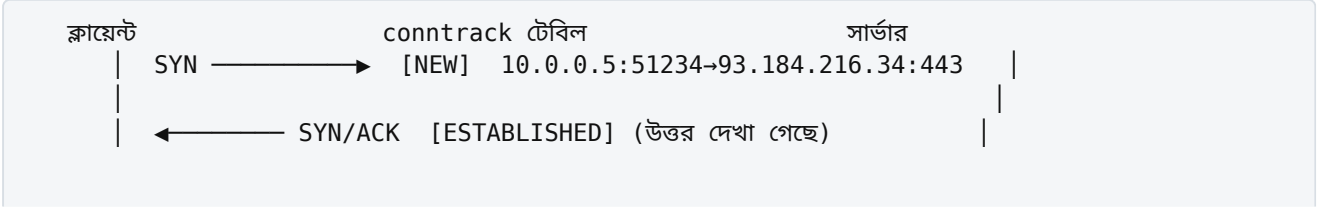
সমস্যাটা কী?

তুমি তোমার ল্যাপটপ থেকে `google.com`-এ HTTPS রিকোয়েস্ট পাঠালে। তোমার প্যাকেট যায় `তোমার IP:51234 → google:443`। উত্তর আসে `google:443 → তোমার IP:51234`। এখন যদি INPUT chain-এ নিয়ম থাকে "সব DROP, শুধু ২২ খোলা", তাহলে google-এর উত্তর তো পোর্ট 51234-এ আসছে — সেটা তো ফেলে দেওয়া হবে!

Stateless ফায়ারওয়ালে এর সমাধান ছিল হাজারটা high port খুলে রাখা — যা বিপজ্জনক। **Stateful** ফায়ারওয়াল মনে রাখে: "এই কানেকশনটা তো আমিই শুরু করেছিলাম, উত্তরটা আসতে দাও।"

conntrack কীভাবে কাজ করে

কার্নেলের **conntrack** (connection tracking) মডিউল প্রতিটা কানেকশনের একটা এন্ট্রি টেবিলে রাখে — উপমা: অফিসের গেটে "ভিজিটর রেজিস্টার"। যে বের হয়েছে তার নাম লেখা থাকে, ফিরে এলে রেজিস্টার দেখে ঢুকতে দেওয়া হয়।



```
      |  ACK, ডেটা ⇄  ⇄   [ESTABLISHED]                              |
      |  FIN ...           [কিছুক্ষণ পর এন্ট্রি মুছে যায়]          |
```

প্রতিটা প্যাকেট এই **state** গুলোর একটা পায়:

State	মানে
`NEW`	একটা নতুন কানেকশনের প্রথম প্যাকেট
`ESTABLISHED`	এমন কানেকশনের প্যাকেট যেখানে দুই দিক থেকেই ট্রাফিক দেখা গেছে
`RELATED`	নতুন, কিন্তু কোনো বিদ্যমান কানেকশনের সাথে সম্পর্কিত (যেমন ICMP error, FTP data channel)
`INVALID`	কোনো কানেকশনের সাথে মেলে না / ভাঙা প্যাকেট — সাধারণত DROP করা উচিত
`UNTRACKED`	raw table-এ `NOTRACK` করা প্যাকেট

> 📝 **মনে রাখো:** conntrack TCP ছাড়াও **UDP আর ICMP**-কে "কানেকশন" হিসেবে ট্র্যাক করে (যদিও এগুলো connectionless) — তাই DNS (UDP 53)-এর উত্তরও ESTABLISHED হিসেবে ঢোকে।

এ কারণেই প্রায় প্রতিটা ভালো ফায়ারওয়ালের প্রথম রুল:

```
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

এই এক লাইনের ফলে তুমি বাকি রুলে শুধু **NEW** কানেকশন নিয়ে চিন্তা করলেই হয়। পারফরম্যান্সও ভালো হয় — বেশিরভাগ প্যাকেট প্রথম রুলেই মিলে বেরিয়ে যায়।

conntrack টেবিল দেখা

`conntrack` টুল আলাদা প্যাকেজে আসে (`sudo apt install conntrack` বা `sudo dnf install conntrack-tools`):

```
$ sudo conntrack -L | head -3
tcp      6 431999 ESTABLISHED src=192.168.1.20 dst=192.168.1.10 sport=53122 dport=22
src=192.168.1.10 dst=192.168.1.20 sport=22 dport=53122 [ASSURED] mark=0 use=1
udp      17 25 src=192.168.1.10 dst=1.1.1.1 sport=41003 dport=53 src=1.1.1.1
dst=192.168.1.10 sport=53 dport=41003 mark=0 use=1
conntrack v1.4.7 (conntrack-tools): 2 flow entries have been shown.

$ sudo conntrack -C            # মোট এন্ট্রি সংখ্যা
2
$ cat /proc/sys/net/netfilter/nf_conntrack_max   # সর্বোচ্চ সীমা
262144
```

> ⚠️ **সতর্কতা:** খুব ব্যস্ত সার্ভারে conntrack টেবিল ভরে গেলে কার্নেল লগে `nf_conntrack: table full, dropping packet` দেখা যায় এবং নতুন কানেকশন ড্রপ হয়। তখন

> `net.netfilter.nf_conntrack_max` বাড়াতে হয় (sysctl নিয়ে অধ্যায় ৪৭)।

৩৯.৫ iptables — ক্লাসিক টুল

যদিও নতুন সিস্টেমে nftables-ই ভবিষ্যৎ, iptables এখনও সবচেয়ে বেশি টিউটোরিয়াল, স্ক্রিপ্ট, Docker, Kubernetes আর ইন্টারভিউতে দেখা যায়। তাই ভালো করে শেখা দরকার। সব iptables কমান্ড **root** হিসেবে চালাতে হয় (`sudo` , দেখো অধ্যায় ১৮)।

> 📝 IPv4-এর জন্য `iptables` , IPv6-এর জন্য **আলাদা** কমান্ড `ip6tables` (একই সিনট্যাক্স)। শুধু iptables সেট করে IPv6 খোলা রেখে দেওয়া একটা ক্লাসিক ভুল।

সিনট্যাক্স

```
iptables [-t table] COMMAND chain [match ...] [-j TARGET [target-options]]
```

`-t` না দিলে ডিফল্ট table `filter` ।

কমান্ড (কী করতে চাও)

অপশন	পূর্ণরূপ	কাজ
`-A chain`	`--append`	chain-এর **শেষে** রুল যোগ
`-I chain [n]`	`--insert`	n নম্বর অবস্থানে (ডিফল্ট ১ = সবার উপরে) রুল ঢোকাও
`-D chain rule` / `-D chain n`	`--delete`	রুল মোছো (হুবহু রুল লিখে, বা নম্বর দিয়ে)
`-R chain n rule`	`--replace`	n নম্বর রুল বদলাও
`-L [chain]`	`--list`	রুল দেখাও
`-S [chain]`	`--list-rules`	রুলগুলো কমান্ড আকারে দেখাও
`-F [chain]`	`--flush`	সব রুল মুছে ফেলো (policy বদলায় না!)
`-Z [chain]`	`--zero`	প্যাকেট/বাইট কাউন্টার শূন্য করো
`-P chain target`	`--policy`	chain-এর ডিফল্ট policy সেট
`-N chain`	`--new-chain`	নতুন user-defined chain
`-X [chain]`	`--delete-chain`	খালি user-defined chain মোছো
`-C chain rule`	`--check`	রুলটা আছে কি না যাচাই (স্ক্রিপ্টে কাজে লাগে)

দেখার অপশন

অপশন	কাজ
`-n`	IP/পোর্টকে নামে (DNS/service) রূপান্তর করবে না — **দ্রুত**, সবসময় ব্যবহার করো
`-v`	বিস্তারিত: interface, প্যাকেট ও বাইট কাউন্টার
`--line-numbers`	প্রতিটা রুলের নম্বর দেখাও (মোছার জন্য দরকার)
`-x`	কাউন্টার সঠিক সংখ্যায় (K/M ছাড়া)

ম্যাচ (শর্ত)

অপশন	মানে	উদাহরণ
`-p proto`	প্রোটোকল	`-p tcp`, `-p udp`, `-p icmp`
`-s addr`	source IP/নেটওয়ার্ক	`-s 192.168.1.0/24`
`-d addr`	destination	`-d 10.0.0.5`
`-i iface`	ইনকামিং ইন্টারফেস (INPUT/FORWARD/PREROUTING)	`-i eth0`
`-o iface`	আউটগোয়িং ইন্টারফেস (OUTPUT/FORWARD/POSTROUTING)	`-o eth1`
`--dport n`	destination port (`-p tcp/udp` লাগবে)	`--dport 22`
`--sport n`	source port	`--sport 53`
`-m multiport --dports a,b`	একাধিক পোর্ট	`--dports 80,443`
`-m conntrack --ctstate S`	কানেকশন state	`--ctstate NEW`
`-m limit --limit r`	rate limit	`--limit 5/min`
`-m comment --comment "..."`	রুলে মন্তব্য	`--comment "web"`
`!`	উল্টো (negation)	`! -s 10.0.0.0/8`

পোর্ট রেঞ্জ: `--dport 6000:6007`।

রুল দেখা

```
$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy DROP 12 packets, 720 bytes)
num   pkts bytes target  prot opt in   out  source          destination
1      120  9600 ACCEPT  all  --  lo   *    0.0.0.0/0       0.0.0.0/0
2     5340  412K ACCEPT  all  --  *    *    0.0.0.0/0       0.0.0.0/0     ctstate
```

```
RELATED,ESTABLISHED
3       14   840 ACCEPT  tcp  --  *    *    0.0.0.0/0       0.0.0.0/0     tcp dpt:22

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num   pkts bytes target  prot opt in   out  source          destination

Chain OUTPUT (policy ACCEPT 4210 packets, 501K bytes)
num   pkts bytes target  prot opt in   out  source          destination
```

কলামগুলো: `pkts/bytes` = এই রুল কতগুলো প্যাকেট মিলিয়েছে (ডিবাগিংয়ে সোনা!), `in/out` = ইন্টারফেস (`*` = যেকোনো), `opt` = IP fragment সম্পর্কিত ফ্ল্যাগ।

`-S` দিয়ে দেখলে কমান্ড আকারে পাবে — কপি করে অন্য সার্ভারে ব্যবহার করা যায়:

```
$ sudo iptables -S INPUT
-P INPUT DROP
-A INPUT -i lo -j ACCEPT
-A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
```

nat table দেখতে: `sudo iptables -t nat -L -n -v`।

রুল যোগ করা: -A বনাম -I

```
# শেষে যোগ (append)
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# সবার উপরে ঢোকাও (insert at 1)
$ sudo iptables -I INPUT -s 203.0.113.66 -j DROP

# ৩ নম্বর অবস্থানে ঢোকাও
$ sudo iptables -I INPUT 3 -p tcp --dport 443 -j ACCEPT
```

> ⚠️ **ক্রমই সব:** ধরো শেষে আছে `-A INPUT -j DROP` (সব ফেলে দাও)। এখন তুমি `-A INPUT -p tcp --dport 80 -j ACCEPT` দিলে সেটা DROP-এর **পরে** বসবে — কোনোদিন কাজ করবে না! এমন ক্ষেত্রে `-I` ব্যবহার করো। নিয়ম: নির্দিষ্ট (specific) রুল উপরে, সাধারণ (generic) রুল নিচে।

রুল মোছা

```
# পদ্ধতি ১: নম্বর দিয়ে
$ sudo iptables -L INPUT -n --line-numbers
num  target  prot opt source         destination
1    ACCEPT  all  --  0.0.0.0/0      0.0.0.0/0
2    ACCEPT  tcp  --  0.0.0.0/0      0.0.0.0/0     tcp dpt:80
3    ACCEPT  tcp  --  0.0.0.0/0      0.0.0.0/0     tcp dpt:22
$ sudo iptables -D INPUT 2
```

```
# পদ্ধতি ২: হুবহু রুল লিখে (-A এর জায়গায় -D)
$ sudo iptables -D INPUT -p tcp --dport 80 -j ACCEPT
```

> 💡 একটা রুল মুছলে নিচের রুলগুলোর নম্বর এক করে কমে যায়। একাধিক রুল নম্বর দিয়ে মুছতে চাইলে **বড় নম্বর থেকে ছোটর দিকে** মোছো।

Policy

```
$ sudo iptables -P INPUT DROP
$ sudo iptables -P FORWARD DROP
$ sudo iptables -P OUTPUT ACCEPT
```

Policy শুধু `ACCEPT` বা `DROP` হতে পারে (REJECT না)। শুধু বিল্ট-ইন chain-এর policy থাকে।

> ⚠️ **সবচেয়ে বিপজ্জনক কমান্ড কম্বো:** রিমোট সার্ভারে `iptables -P INPUT DROP` দিয়ে তারপর `iptables -F` দিলে — সব ACCEPT রুল মুছে গেল, policy DROP রয়ে গেল → তোমার SSH সাথে সাথে বন্ধ। **`-F` policy রিসেট করে না।** রিসেট করতে চাইলে আগে `-P INPUT ACCEPT`, তারপর `-F`।

ACCEPT, DROP, REJECT হাতে-কলমে

```
$ sudo iptables -A INPUT -p tcp --dport 8080 -j DROP
# অন্য মেশিন থেকে:
$ curl --max-time 5 http://192.168.1.10:8080
curl: (28) Connection timed out after 5001 milliseconds

# (ধরে নিচ্ছি 8080-এর রুলটা INPUT-এর ১ নম্বর রুল; --line-numbers দিয়ে মিলিয়ে নাও)
$ sudo iptables -R INPUT 1 -p tcp --dport 8080 -j REJECT --reject-with tcp-reset
$ curl http://192.168.1.10:8080
curl: (7) Failed to connect to 192.168.1.10 port 8080 after 2 ms: Connection refused
```

`--reject-with` এর মানগুলো: `icmp-port-unreachable` (ডিফল্ট), `icmp-host-unreachable`, `icmp-net-unreachable`, `icmp-admin-prohibited`, `tcp-reset` (শুধু TCP-তে)।

LOG দিয়ে ডিবাগ

```
$ sudo iptables -A INPUT -p tcp --dport 23 -j LOG --log-prefix "TELNET-TRY: " --log-
level 4
$ sudo iptables -A INPUT -p tcp --dport 23 -j DROP

$ sudo journalctl -k -g TELNET-TRY
Nov 12 10:15:01 srv kernel: TELNET-TRY: IN=eth0 OUT= MAC=... SRC=203.0.113.9
DST=192.168.1.10 LEN=60 ... PROTO=TCP SPT=40122 DPT=23 ... SYN
```

LOG non-terminating, তাই তার পরের DROP রুলটাও প্যাকেট পায়। লগ ফ্লাড এড়াতে `-m limit --limit 5/min` যোগ করো। (লগ দেখা নিয়ে বিস্তারিত অধ্যায় ২৮।)

একটা সম্পূর্ণ সার্ভার ফায়ারওয়াল (iptables)

```
#!/bin/bash
# web-server-firewall.sh — একটা সাধারণ ওয়েব সার্ভারের জন্য
set -e

# ১. আগে policy ACCEPT করে রুল পরিষ্কার (নিরাপদ রিসেট)
iptables -P INPUT ACCEPT
iptables -F
iptables -X

# ২. loopback সবসময় খোলা (অনেক প্রোগ্রাম 127.0.0.1-এ কথা বলে)
iptables -A INPUT -i lo -j ACCEPT

# ৩. পুরনো কানেকশনের উত্তর ঢুকতে দাও, ভাঙা প্যাকেট ফেলো
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -m conntrack --ctstate INVALID -j DROP

# ৪. ping (ICMP echo) — ডিবাগিংয়ে কাজে লাগে
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

# ৫. SSH শুধু অফিস নেটওয়ার্ক থেকে
iptables -A INPUT -p tcp -s 198.51.100.0/24 --dport 22 -m conntrack --ctstate NEW -j
ACCEPT

# ৬. ওয়েব সবার জন্য
iptables -A INPUT -p tcp -m multiport --dports 80,443 -m conntrack --ctstate NEW -j
ACCEPT

# ৭. বাকি সব — শেষে policy DROP
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
```

user-defined chain

রুল গুছিয়ে রাখতে:

```
$ sudo iptables -N SSH_GUARD
$ sudo iptables -A SSH_GUARD -s 198.51.100.0/24 -j ACCEPT
$ sudo iptables -A SSH_GUARD -j DROP
$ sudo iptables -A INPUT -p tcp --dport 22 -j SSH_GUARD     # INPUT থেকে লাফ
```

মুছতে: আগে রেফারেন্স (`-D INPUT ...`) মোছো, তারপর `-F SSH_GUARD` , তারপর `-X SSH_GUARD` ।

SSH brute-force rate limit (`recent` মডিউল)

```
# ৬০ সেকেন্ডে একই IP থেকে ৪টার বেশি নতুন SSH কানেকশন হলে ফেলে দাও
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --set --name
SSH
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --update --
seconds 60 --hitcount 4 --name SSH -j DROP
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -j ACCEPT
```

রুল স্থায়ী (persistent) করা

iptables রুল **মেমরিতে** থাকে — রিবুট দিলে হারিয়ে যায়। সেভ/রিস্টোর:

```
$ sudo iptables-save > rules.v4          # বর্তমান রুল ফাইলে
$ sudo iptables-restore < rules.v4       # ফাইল থেকে লোড (পুরনোগুলো প্রতিস্থাপন করে)
$ sudo iptables-restore --test < rules.v4   # শুধু সিনট্যাক্স চেক
```

`iptables-save` এর আউটপুট দেখতে এমন:

```
*filter
:INPUT DROP [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
-A INPUT -i lo -j ACCEPT
-A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
COMMIT
```

বুট-এ অটো লোড:

ডিস্ট্রো	প্যাকেজ	ফাইল	সেভ কমান্ড
Debian/Ubuntu	`iptables-persistent` (`netfilter-persistent` সার্ভিস)	`/etc/iptables/rules.v4`, `/etc/iptables/rules.v6`	`sudo netfilter-persistent save`
RHEL/Fedora	`iptables-services` (firewalld বন্ধ করতে হবে)	`/etc/sysconfig/iptables`, `/etc/sysconfig/ip6tables`	`sudo iptables-save > /etc/sysconfig/iptables`

> ⚠️ ufw বা firewalld চালু থাকলে iptables-persistent ইনস্টল করবে না — ঝামেলা বাড়বে। Ubuntu-তে iptables-persistent ইনস্টল করলে apt ufw সরিয়ে দিতে পারে (দুটো conflict করে)।

`iptables-apply` — লক-আউট থেকে বাঁচার বিল্ট-ইন উপায়

```
$ sudo iptables-apply /etc/iptables/rules.v4
Applying new ruleset... done.
Can you establish NEW connections to the machine? (y/N)
```

নতুন রুল লোড করে প্রশ্ন করে; নির্দিষ্ট সময়ে (ডিফল্ট ১০ সেকেন্ড, `-t` দিয়ে বদলানো যায়) `y` না দিলে পুরনো রুলে ফিরে যায়। নতুন SSH সেশন খুলে টেস্ট করে তারপর `y` দাও।

৩৯.৬ nftables — আধুনিক ফায়ারওয়াল

কেন nftables এলো?

iptables-এর কিছু সমস্যা ছিল:

- IPv4, IPv6, ARP, bridge-এর জন্য চারটা আলাদা টুল (`iptables`, `ip6tables`, `arptables`, `ebtables`) আর আলাদা কার্নেল কোড।
- একটা রুল বদলালেও পুরো টেবিল কার্নেলে নতুন করে পাঠাতে হতো — হাজার রুলে ধীর।
- ৫০০টা IP ব্লক করতে ৫০০টা রুল (ipset ছাড়া)।

nftables (কার্নেল ৩.১৩ থেকে, ২০১৪) সমাধান করে:

- একটা টুল `nft`, একটা সিনট্যাক্স — **`inet` family** দিয়ে IPv4 আর IPv6 একসাথে।
- কার্নেলের ভেতরে একটা ছোট **virtual machine** রুল চালায়; incremental (আংশিক) আপডেট।
- **set** আর **map** বিল্ট-ইন — হাজার IP এক রুলে।
- কোনো table/chain ডিফল্টে থাকে না — তুমি যা বানাবে তাই।

গঠন

```
ruleset
 └── table  <family> <name>         (family: ip, ip6, inet, arp, bridge, netdev)
      ├── chain <name> { type <filter|nat|route> hook <hook> priority <n>; policy
<accept|drop>; }
      │     ├── rule
      │     └── rule
      ├── set  <name> { type ipv4_addr; elements = {...} }
      └── map  ...
```

Base chain (হুকে যুক্ত) বনাম **regular chain** (শুধু `jump` / `goto` দিয়ে যাওয়া যায়)। **priority** ঠিক করে একই হুকে কোন chain আগে চলবে — ছোট সংখ্যা আগে। নাম দিয়েও লেখা যায়: `raw` (-300), `mangle` (-150), `dstnat` (-100), `filter` (0), `security` (50), `srcnat` (100)।

> ⚠️ **একটা গুরুত্বপূর্ণ ব্যাপার:** একই হুকে একাধিক base chain থাকলে, প্যাকেটকে **সবগুলো** chain পার হতে হয়। একটায় `accept` মানে শুধু ঐ chain থেকে ছাড়, অন্য chain-এ `drop` হলে প্যাকেট ড্রপ হবে। তাই firewalld/Docker-এর table থাকা অবস্থায় নিজের table যোগ করলে সবগুলোর রুল একসাথে খাটে।

কমান্ড-লাইন থেকে ধাপে ধাপে

```
# পুরো ruleset দেখো
$ sudo nft list ruleset

# table বানাও (inet = IPv4 + IPv6)
$ sudo nft add table inet myfw

# base chain বানাও — শেলের জন্য { } ও ; কোট করতে হয়
$ sudo nft add chain inet myfw input '{ type filter hook input priority filter; policy
accept; }'

# রুল যোগ
$ sudo nft add rule inet myfw input iif lo accept
$ sudo nft add rule inet myfw input ct state established,related accept
$ sudo nft add rule inet myfw input ct state invalid drop
$ sudo nft add rule inet myfw input tcp dport 22 accept
$ sudo nft add rule inet myfw input tcp dport { 80, 443 } accept
$ sudo nft add rule inet myfw input meta l4proto { icmp, ipv6-icmp } accept

# সব ঠিক থাকলে policy drop
$ sudo nft chain inet myfw input '{ policy drop; }'
```

> 💡 IPv6-এ **ICMPv6 ব্লক করবে না** — neighbor discovery (IPv6-এর ARP) ICMPv6 দিয়ে চলে; ব্লক করলে IPv6 নেটওয়ার্কিং-ই ভেঙে যায়।

দেখা ও handle দিয়ে মোছা

nftables-এ রুল মোছা হয় **handle** (প্রতিটা রুলের অনন্য আইডি) দিয়ে:

```
$ sudo nft -a list chain inet myfw input
table inet myfw {
    chain input { # handle 1
        type filter hook input priority filter; policy drop;
        iif "lo" accept # handle 2
        ct state established,related accept # handle 3
        ct state invalid drop # handle 4
        tcp dport 22 accept # handle 5
        tcp dport { 80, 443 } accept # handle 7
        meta l4proto { icmp, ipv6-icmp } accept # handle 9
    }
}

$ sudo nft delete rule inet myfw input handle 7
```

কাজ	কমান্ড
শেষে রুল	`nft add rule <family> <table> <chain> ...`
শুরুতে রুল	`nft insert rule <family> <table> <chain> ...`

কাজ	কমান্ড
নির্দিষ্ট রুলের পরে	`nft add rule ... <chain> position <handle> ...`
রুল বদলানো	`nft replace rule ... <chain> handle <n> ...`
রুল মোছা	`nft delete rule ... <chain> handle <n>`
chain খালি করা	`nft flush chain inet myfw input`
table মোছা	`nft delete table inet myfw`
সব মুছে ফেলা	`nft flush ruleset`
table তালিকা	`nft list tables`
handle সহ দেখা	`nft -a list ruleset`
IP-কে DNS নামে দেখাও	`nft -N list ruleset` (ডিফল্টে nft IP সংখ্যাতেই দেখায়)
ফাইল থেকে লোড	`nft -f /etc/nftables.conf`
শুধু চেক, লোড না	`nft -c -f file.nft`

> ⚠️ `nft flush ruleset` **সব** টেবিল মুছে দেয় — firewalld, Docker, libvirt-এর রুলও! সার্ভারে সাবধানে।

Sets — হাজার IP এক রুলে

```
$ sudo nft add set inet myfw blocklist '{ type ipv4_addr; flags interval; }'
$ sudo nft add element inet myfw blocklist '{ 203.0.113.0/24, 198.51.100.7 }'
$ sudo nft insert rule inet myfw input ip saddr @blocklist drop
$ sudo nft list set inet myfw blocklist
```

`flags interval` দিলে CIDR রেঞ্জ রাখা যায়। `flags timeout` দিয়ে এলিমেন্ট নির্দিষ্ট সময় পর নিজেই মুছে যায় — অটো-ব্যান সিস্টেম বানাতে দারুণ।

কনফিগ ফাইল — আসল ব্যবহার

বাস্তবে nftables রুল একটা ফাইলে লেখা হয় এবং একবারে (atomic) লোড হয় — অর্ধেক রুল লোড হয়ে আটকে থাকার ঝুঁকি নেই।

ডিস্ট্রো	ফাইল
Debian 12 / Ubuntu	`/etc/nftables.conf`
Fedora / RHEL 9	`/etc/sysconfig/nftables.conf` (যেটা `/etc/nftables/*.nft` include করে)

```
#!/usr/sbin/nft -f
# /etc/nftables.conf

flush ruleset
```

```
table inet filter {
    set ssh_allowed {
        type ipv4_addr
        flags interval
        elements = { 198.51.100.0/24, 192.168.1.0/24 }
    }

    chain input {
        type filter hook input priority filter; policy drop;

        iif "lo" accept
        ct state established,related accept
        ct state invalid drop

        meta l4proto { icmp, ipv6-icmp } accept

        ip saddr @ssh_allowed tcp dport 22 ct state new accept
        tcp dport { 80, 443 } ct state new accept

        # SSH rate limit: প্রতি IP মিনিটে ১০টার বেশি নতুন কানেকশন হলে ড্রপ
        tcp dport 22 ct state new meter ssh_meter { ip saddr limit rate over 10/minute
} drop

        limit rate 5/minute log prefix "nft-drop: "
        counter
    }

    chain forward {
        type filter hook forward priority filter; policy drop;
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
```

```
$ sudo nft -c -f /etc/nftables.conf        # আগে সিনট্যাক্স চেক
$ sudo nft -f /etc/nftables.conf           # লোড
$ sudo systemctl enable --now nftables     # বুটে অটো লোড
```

(নতুন nft ভার্সনে `meter`-এর জায়গায় dynamic set-এর সিনট্যাক্সও আছে; `meter` এখনও কাজ করে।)

> 📝 ফাইলের শুরুতে `flush ruleset` থাকলে প্রতিবার লোডে পরিষ্কার শুরু হয়। কিন্তু Docker/libvirt থাকলে তাদের রুলও মুছবে — তখন `flush ruleset`-এর বদলে শুধু নিজের table মুছে আবার বানাও (`table inet filter` আগে define করে তারপর `delete table inet filter`)।

iptables থেকে nftables-এ অনুবাদ

```
$ iptables-translate -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -j ACCEPT
nft 'add rule ip filter INPUT tcp dport 22 ct state new counter accept'
```

```
$ sudo iptables-save > old.v4
$ iptables-restore-translate -f old.v4 > new.nft
```

iptables বনাম nftables পাশাপাশি

কাজ	iptables	nftables
রুল দেখা	`iptables -L -n -v`	`nft list ruleset`
loopback	`-A INPUT -i lo -j ACCEPT`	`iif lo accept`
stateful	`-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`	`ct state established,related accept`
একাধিক পোর্ট	`-m multiport --dports 80,443`	`tcp dport { 80, 443 }`
IP থেকে	`-s 10.0.0.0/8`	`ip saddr 10.0.0.0/8`
ড্রপ	`-j DROP`	`drop`
রিজেক্ট	`-j REJECT`	`reject`
কাউন্টার	ডিফল্টে থাকে	`counter` লিখতে হয়
IPv4+IPv6	দুটো কমান্ড	`inet` family-তে একটা
মোছা	`-D INPUT 3`	`delete rule ... handle 5`
সেভ	`iptables-save`	`nft list ruleset > file`

৩৯.৭ ufw — Uncomplicated Firewall (Ubuntu/Debian)

নামেই পরিচয় — "জটিলতাহীন ফায়ারওয়াল"। ufw ভেতরে iptables (Ubuntu 24.04-এ iptables-nft) রুল লেখে, কিন্তু তোমাকে মানুষের ভাষার কাছাকাছি কমান্ড দেয়। ডেস্কটপ আর সাধারণ সার্ভারের জন্য সবচেয়ে সহজ।

```
# Ubuntu-তে আগে থেকেই থাকে; Debian-এ:
$ sudo apt install ufw
```

ডিফল্ট policy

ufw-এর ডিফল্ট: **incoming deny, outgoing allow, routed disabled** — মানে বাইরে থেকে কেউ ঢুকতে পারবে না, তুমি যেখানে খুশি যেতে পারবে। চাইলে স্পষ্টভাবে সেট করো:

```
$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
$ sudo ufw default allow outgoing
```

⚠️ চালু করার আগে সবচেয়ে জরুরি ধাপ

রিমোট সার্ভারে **আগে SSH allow করো, তারপর enable**:

```
$ sudo ufw allow OpenSSH          # বা: sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)

$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

SSH যদি অন্য পোর্টে চলে (ধরো 2222), তাহলে `sudo ufw allow 2222/tcp` — `OpenSSH` প্রোফাইল শুধু ২২ খোলে!

স্ট্যাটাস দেখা

```
$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
OpenSSH                    ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
OpenSSH (v6)               ALLOW       Anywhere (v6)
80/tcp (v6)                ALLOW       Anywhere (v6)

$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
...
```

`ufw enable` করার আগে (inactive অবস্থায়) যোগ করা রুলগুলো দেখতে: `sudo ufw show added`।

allow / deny / reject / limit — বিভিন্ন রূপ

```
# পোর্ট (TCP ও UDP দুটোই)
$ sudo ufw allow 53
# শুধু TCP
$ sudo ufw allow 443/tcp
# পোর্ট রেঞ্জ (প্রোটোকল বলতেই হবে)
$ sudo ufw allow 6000:6007/tcp
# /etc/services থেকে সার্ভিসের নাম
$ sudo ufw allow http
```

```
# নির্দিষ্ট IP থেকে সব
$ sudo ufw allow from 192.168.1.50
# নির্দিষ্ট নেটওয়ার্ক থেকে নির্দিষ্ট পোর্ট
$ sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp
# নির্দিষ্ট ইন্টারফেসে
$ sudo ufw allow in on eth1 to any port 3306 proto tcp
# ব্লক — চুপচাপ ফেলে দাও
$ sudo ufw deny from 203.0.113.66
# ব্লক — জানিয়ে দাও
$ sudo ufw reject 23/tcp
# আউটগোয়িং ব্লক
$ sudo ufw deny out 25/tcp
# rate limit (brute-force সুরক্ষা)
$ sudo ufw limit ssh
# রুলে মন্তব্য
$ sudo ufw allow 8080/tcp comment 'dev server'
```

`ufw limit` : একই IP থেকে **৩০ সেকেন্ডে ৬ বা তার বেশি** কানেকশন চেষ্টা হলে সেটা deny করে। SSH সার্ভারের জন্য চমৎকার।

ufw সিনট্যাক্স অংশ	মানে
`allow` / `deny` / `reject` / `limit`	কাজ
`in` / `out`	দিক (ডিফল্ট `in`)
`on <iface>`	ইন্টারফেস
`from <addr>`	source (`any` = যেকোনো)
`to <addr>`	destination (`any` = এই মেশিনের যেকোনো IP)
`port <n>`	পোর্ট
`proto tcp|udp`	প্রোটোকল
`comment '...'`	মন্তব্য

> ⚠️ **ক্রম এখানেও গুরুত্বপূর্ণ:** ufw রুলও প্রথম-মিল-জেতে নিয়মে চলে। আগে `allow 22` থাকলে পরে `deny from 203.0.113.66` যোগ করলে সেই IP তবুও ২২-এ ঢুকতে পারবে! ব্লক রুল উপরে বসাতে `insert` ব্যবহার করো:
>
> ```
> $ sudo ufw insert 1 deny from 203.0.113.66
> ```

রুল মোছা — numbered

```
$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
```

```
[ 2] 80/tcp                     ALLOW IN    Anywhere
[ 3] 8080/tcp                   ALLOW IN    Anywhere                   # dev server
[ 4] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 5] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 6] 8080/tcp (v6)              ALLOW IN    Anywhere (v6)              # dev server

$ sudo ufw delete 3
Deleting:
 allow 8080/tcp comment 'dev server'
Proceed with operation (y|n)? y
Rule deleted
```

লক্ষ করো, IPv6 রুল (`[6]`) এখনও আছে — নম্বর দিয়ে মুছলে শুধু ঐ একটাই মোছে। **রুলের নাম দিয়ে** মুছলে v4 আর v6 দুটোই একসাথে যায়:

```
$ sudo ufw delete allow 8080/tcp
Rule deleted
Rule deleted (v6)
```

অ্যাপ প্রোফাইল (application profiles)

প্যাকেজগুলো (openssh-server, nginx, apache2 ইত্যাদি) `/etc/ufw/applications.d/` -এ প্রোফাইল ফাইল রাখে, যাতে পোর্ট নম্বর মনে না রেখে নাম দিয়ে খোলা যায়।

```
$ sudo ufw app list
Available applications:
  Nginx Full
  Nginx HTTP
  Nginx HTTPS
  OpenSSH

$ sudo ufw app info 'Nginx Full'
Profile: Nginx Full
Title: Web Server (Nginx, HTTP + HTTPS)
Description: Small, but very powerful and efficient web server

Ports:
  80,443/tcp

$ sudo ufw allow 'Nginx Full'      # নামে স্পেস থাকলে কোট দাও
```

নিজের অ্যাপের প্রোফাইল বানাও — `/etc/ufw/applications.d/myapp` :

```
[MyApp]
title=My Node.js App
description=Internal API and admin panel
ports=3000/tcp|9229/tcp
```

```
$ sudo ufw app update MyApp
$ sudo ufw allow MyApp
```

(`ports=` এ একাধিক পোর্ট কমা দিয়ে একই প্রোটোকলে, যেমন `80,443/tcp` ; ভিন্ন প্রোটোকল `|` দিয়ে আলাদা, যেমন `53/tcp|53/udp` ।)

অন্যান্য দরকারি ufw কমান্ড

কমান্ড	কাজ
`ufw disable`	ফায়ারওয়াল বন্ধ (রুল থেকে যায়)
`ufw reload`	রুল আবার লোড
`ufw reset`	সব রুল মুছে ডিফল্টে ফেরো (ফায়ারওয়াল disable হয়; পুরনো রুল ফাইল ব্যাকআপ রাখে)
`ufw logging on|off|low|medium|high|full`	লগিং লেভেল
`ufw --dry-run allow 80`	আসলে না করে কী হতো দেখাও
`ufw show raw`	ভেতরের আসল iptables রুল দেখাও
`ufw show listening`	কোন পোর্টে প্রোগ্রাম listen করছে আর কোন রুল প্রযোজ্য
`ufw route allow in on eth1 out on eth0`	FORWARD (রাউটিং) ট্রাফিকের রুল

লগ যায় `/var/log/ufw.log` এ (বা `journalctl -k`), ব্লক হওয়া প্যাকেট `[UFW BLOCK]` প্রিফিক্স সহ:

```
$ sudo grep 'UFW BLOCK' /var/log/ufw.log | tail -1
Nov 12 11:02:44 srv kernel: [UFW BLOCK] IN=eth0 OUT= MAC=... SRC=45.155.205.3
DST=192.168.1.10 LEN=40 ... PROTO=TCP SPT=48211 DPT=3389 WINDOW=1024 RES=0x00 SYN
URGP=0
```

ufw-এর ফাইলগুলো

ফাইল	কী আছে
`/etc/default/ufw`	ডিফল্ট policy (`DEFAULT_FORWARD_POLICY` ইত্যাদি), `IPV6=yes`
`/etc/ufw/ufw.conf`	`ENABLED=yes` , লগিং লেভেল
`/etc/ufw/user.rules` , `user6.rules`	তোমার `ufw allow ...` রুলগুলো (হাতে এডিট করবে না)
`/etc/ufw/before.rules` , `after.rules`	ufw রুলের আগে/পরে চলা কাঁচা iptables রুল — NAT এখানে লেখা হয়
`/etc/ufw/sysctl.conf`	ufw লোডের সময় সেট হওয়া kernel প্যারামিটার
`/etc/ufw/applications.d/`	অ্যাপ প্রোফাইল

⚠️ Docker আর ufw — একটা কুখ্যাত ফাঁদ

Docker-এ `docker run -p 3306:3306 mysql` দিলে Docker নিজে iptables-এর `nat` table-এ DNAT রুল আর `FORWARD` chain-এ (তার `DOCKER` chain-এ) রুল বসায়। এই প্যাকেট **INPUT** chain-এ যায়ই না

(কারণ DNAT-এর পর এটা কন্টেইনারের দিকে forward হয়), তাই ufw-এর `deny 3306` **কাজ করে না** — পোর্টটা পুরো ইন্টারনেটে খোলা!

সমাধান:

- পোর্ট শুধু localhost-এ বাঁধো: `-p 127.0.0.1:3306:3306`
- অথবা Docker-এর `DOCKER-USER` chain-এ নিজের রুল দাও (Docker সেটা ছোঁয় না)।

(কন্টেইনার নিয়ে বিস্তারিত অধ্যায় ৪৮-এ।)

৩৯.৮ firewalld — zone-ভিত্তিক ডাইনামিক ফায়ারওয়াল (Fedora/RHEL)

firewalld একটা **ডেমন** (daemon — ব্যাকগ্রাউন্ডে চলা সার্ভিস) যা রুল সামলায় এবং D-Bus দিয়ে কমান্ড নেয়। এর কমান্ড-লাইন টুল `firewall-cmd`, GUI `firewall-config`। Fedora আর RHEL 9-এ এটা ডিফল্টে চালু, ব্যাকএন্ড nftables।

```
$ sudo systemctl status firewalld
$ sudo firewall-cmd --state
running
# Ubuntu/Debian-এ চাইলে: sudo apt install firewalld (তখন ufw disable করো)
```

মূল ধারণা ১: Zones

Zone = একটা বিশ্বাসের স্তর (trust level), সাথে সেই স্তরের রুলসেট। প্রতিটা **interface** বা **source IP** কোনো একটা zone-এর সাথে যুক্ত থাকে। উপমা: একটা বাড়িতে ড্রয়িং রুম (অতিথি আসতে পারে), বেডরুম (শুধু পরিবার), রান্নাঘর (শুধু রাঁধুনি) — দরজা দিয়ে কে এল সেই অনুযায়ী কোন রুমের নিয়ম খাটবে।

```
   eth0 (ইন্টারনেট) ──────► zone: public   → শুধু ssh, dhcpv6-client
   eth1 (অফিস LAN) ──────► zone: internal → ssh, samba, mdns ...
   source 10.8.0.0/24 ──► zone: trusted  → সবকিছু অনুমোদিত
```

প্যাকেট কোন zone পাবে: আগে **source** মেলানো হয়, না মিললে **interface**-এর zone, তাও না থাকলে **default zone**।

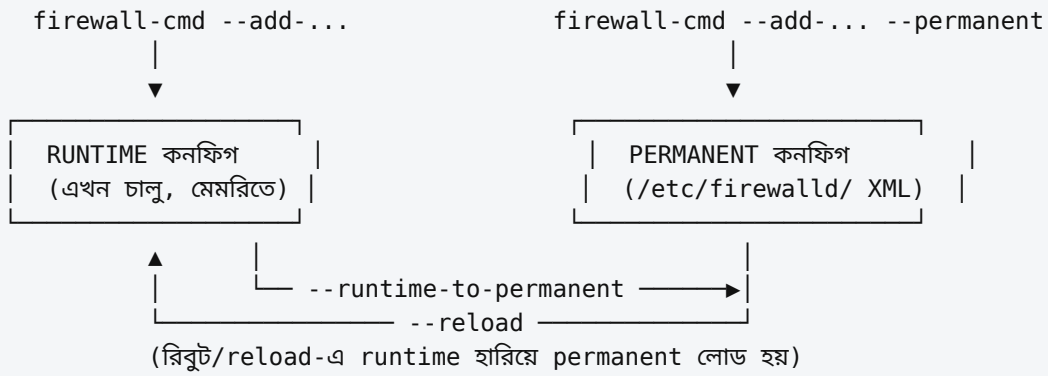
প্রি-ডিফাইন্ড zone	আচরণ (বিশ্বাস কম → বেশি)
`drop`	সব ইনকামিং চুপচাপ ড্রপ, কোনো উত্তর নেই; শুধু বাইরে যাওয়া চলে
`block`	সব ইনকামিং reject (icmp-host-prohibited)
`public`	পাবলিক নেটওয়ার্ক; শুধু বেছে নেওয়া সার্ভিস (RHEL ডিফল্ট zone)
`external`	রাউটারের বাইরের দিক; masquerading চালু

প্রি-ডিফাইন্ড zone	আচরণ (বিশ্বাস কম → বেশি)
`dmz`	DMZ-এর পাবলিক সার্ভার
`work`	অফিস নেটওয়ার্ক
`home`	বাসার নেটওয়ার্ক
`internal`	ভেতরের নেটওয়ার্ক
`trusted`	সব কানেকশন অনুমোদিত

> 📝 Fedora Workstation-এর ডিফল্ট zone `FedoraWorkstation` — এটা ১০২৫-৬৫৫৩৫ পর্যন্ত সব high port খোলা রাখে (ডেস্কটপের সুবিধার জন্য)। Fedora Server ও RHEL-এ `public`।

মূল ধারণা ২: Runtime বনাম Permanent

এটাই firewalld-এর সবচেয়ে বড় ধাঁধা:



- `--permanent` ছাড়া: সাথে সাথে কাজ করে, কিন্তু reload/রিবুটে হারিয়ে যায়।
- `--permanent` সহ: ফাইলে লেখা হয়, কিন্তু **এখনই কাজ করে না** — `--reload` লাগবে।

দুটো সাধারণ কৌশল:

```
# কৌশল ক: permanent লেখো, তারপর reload
$ sudo firewall-cmd --permanent --add-service=http
success
$ sudo firewall-cmd --reload
success

# কৌশল খ: runtime-এ টেস্ট করো, ঠিক থাকলে permanent বানাও (নিরাপদ!)
$ sudo firewall-cmd --add-service=http
$ ... টেস্ট ...
$ sudo firewall-cmd --runtime-to-permanent
```

> 💡 কৌশল খ লক-আউটের বিরুদ্ধে দারুণ: runtime-এ কিছু ভুল করলে `firewall-cmd --reload`

> দিলেই পুরনো permanent অবস্থায় ফিরে যাবে।

zone দেখা ও বদলানো

```
$ firewall-cmd --get-default-zone
public
$ firewall-cmd --get-zones
block dmz drop external home internal nm-shared public trusted work
$ firewall-cmd --get-active-zones
public
  interfaces: eth0

$ sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: eth0
  sources:
  services: cockpit dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:

$ sudo firewall-cmd --list-all --zone=internal    # অন্য zone
$ sudo firewall-cmd --list-all-zones              # সব zone

$ sudo firewall-cmd --set-default-zone=home       # ডিফল্ট বদলাও (runtime+permanent দুটোই)
$ sudo firewall-cmd --zone=internal --change-interface=eth1 --permanent
$ sudo firewall-cmd --zone=trusted --add-source=10.8.0.0/24 --permanent
$ sudo firewall-cmd --reload
```

> 📝 NetworkManager চালিত সিস্টেমে ইন্টারফেসের zone NetworkManager-এর connection প্রোফাইলেও থাকে (`nmcli connection modify eth1 connection.zone internal`) — দুই জায়গায় দুই রকম থাকলে বিভ্রান্তি হয়।

services ও ports

firewalld-এ **service** = নাম দেওয়া পোর্ট/প্রোটোকলের গুচ্ছ (ufw-এর অ্যাপ প্রোফাইলের মতো), XML ফাইলে সংজ্ঞায়িত।

```
$ firewall-cmd --get-services | tr ' ' '\n' | grep -E '^(http|https|ssh|mysql)$'
http
https
mysql
```

```
ssh

$ firewall-cmd --info-service=https
https
  ports: 443/tcp
  protocols:
  source-ports:
  modules:
  destination:
  includes:
  helpers:

$ sudo firewall-cmd --permanent --add-service={http,https}   # bash brace expansion
$ sudo firewall-cmd --permanent --remove-service=cockpit
$ sudo firewall-cmd --permanent --add-port=8080/tcp
$ sudo firewall-cmd --permanent --add-port=60000-61000/udp
$ sudo firewall-cmd --permanent --remove-port=8080/tcp
$ sudo firewall-cmd --reload
$ firewall-cmd --list-services
dhcpv6-client http https ssh
$ firewall-cmd --list-ports
60000-61000/udp
```

সার্ভিস ফাইলের জায়গা:

ডিরেক্টরি	কী
`/usr/lib/firewalld/`	প্যাকেজের দেওয়া ডিফল্ট zone/service (হাতে বদলাবে না)
`/etc/firewalld/`	তোমার কনফিগ — এখানকার ফাইল ডিফল্টকে override করে
`/etc/firewalld/firewalld.conf`	মূল সেটিং (`DefaultZone` , `FirewallBackend=nftables` , `LogDenied`)

নিজের সার্ভিস বানাও:

```
$ sudo firewall-cmd --permanent --new-service=myapp
$ sudo firewall-cmd --permanent --service=myapp --set-short="My App"
$ sudo firewall-cmd --permanent --service=myapp --add-port=3000/tcp
$ sudo firewall-cmd --reload
$ sudo firewall-cmd --permanent --add-service=myapp && sudo firewall-cmd --reload
```

Rich rules — সূক্ষ্ম নিয়ন্ত্রণ

শুধু "সার্ভিস খোলো" দিয়ে না হলে (যেমন নির্দিষ্ট IP থেকে, লগ সহ, rate limit সহ) **rich rule**:

```
# শুধু অফিস নেটওয়ার্ক থেকে SSH
$ sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source
address="198.51.100.0/24" service name="ssh" accept'

# একটা IP পুরো ব্লক
$ sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source
address="203.0.113.66" drop'

# MySQL শুধু অ্যাপ সার্ভার থেকে, লগ সহ
```

```
$ sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source
address="10.0.0.20" port port="3306" protocol="tcp" log prefix="mysql-ok " level="info"
accept'

# HTTP-তে মিনিটে ২৫টা নতুন কানেকশন
$ sudo firewall-cmd --permanent --add-rich-rule='rule service name="http" accept limit
value="25/m"'

$ sudo firewall-cmd --reload
$ firewall-cmd --list-rich-rules
```

মুছতে একই স্ট্রিং দিয়ে `--remove-rich-rule='...'`।

> ⚠️ ssh সার্ভিস zone-এ আগে থেকেই খোলা থাকলে শুধু rich rule যোগ করলে সীমাবদ্ধতা হবে না — `--remove-service=ssh` করতে হবে, নইলে সবাই সার্ভিস দিয়েই ঢুকে যাবে।

অন্যান্য দরকারি firewall-cmd অপশন

অপশন	কাজ
`--timeout=5m`	runtime রুল ৫ মিনিট পর নিজেই মুছে যাবে (`--permanent`-এর সাথে চলে না)
`--panic-on` / `--panic-off` / `--query-panic`	সব ট্রাফিক সাথে সাথে বন্ধ (আক্রমণের সময়) — রিমোট থেকে দিলে তুমিও কাটা পড়বে!
`--check-config`	permanent কনফিগ ফাইলের সিনট্যাক্স যাচাই
`--complete-reload`	পুরোপুরি reload, চলমান কানেকশনের state সহ মুছে যায়
`--query-service=ssh`	সার্ভিস চালু আছে কি না (`yes` / `no`)
`--set-log-denied=all`	ড্রপ হওয়া প্যাকেট লগ করো (`unicast`, `off` ইত্যাদি)
`--add-icmp-block=echo-request`	ping ব্লক
`--zone=X`	যেকোনো অপশনে নির্দিষ্ট zone

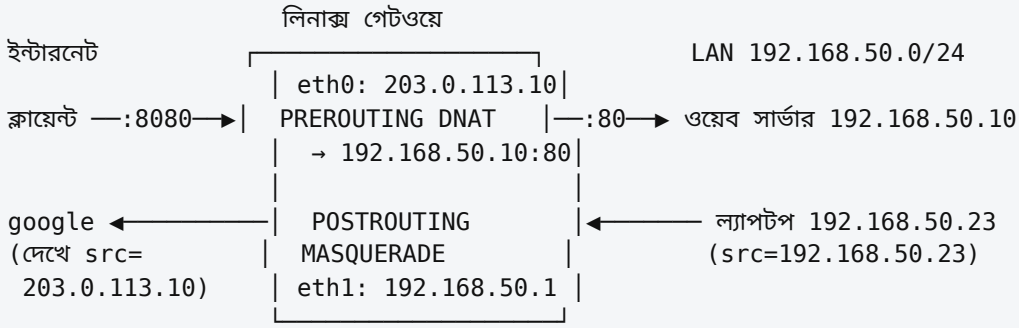
> 💡 firewalld-এর নতুন ভার্সনে zone-এর মধ্যে ট্রাফিক নিয়ন্ত্রণের জন্য **policy objects** (`--new-policy`, `--policy=...`) আছে; পুরনো `--direct` ইন্টারফেস deprecated। জটিল রাউটার সেটআপে policy ব্যবহার করো।

৩৯.৯ NAT ও Port Forwarding

NAT কী?

NAT (Network Address Translation) = প্যাকেটের IP ঠিকানা (এবং পোর্ট) পথের মাঝে বদলে দেওয়া। (ধারণাগত ভিত্তি অধ্যায় ৩৬-এ।) দুই ধরন:

- **SNAT / MASQUERADE** (source বদলানো, POSTROUTING-এ): ভেতরের প্রাইভেট IP-গুলোকে রাউটারের পাবলিক IP-র পেছনে লুকানো — বাসার Wi-Fi রাউটার ঠিক এটাই করে। MASQUERADE হলো SNAT যা ইন্টারফেসের বর্তমান IP নিজেই বের করে নেয় (DHCP/ডাইনামিক IP-তে সুবিধাজনক); IP স্থির হলে `SNAT --to-source` একটু দ্রুত।
- **DNAT** (destination বদলানো, PREROUTING-এ): "পোর্ট ফরওয়ার্ডিং" — বাইরের পোর্ট ৮০৮০-তে আসা ট্রাফিক ভেতরের কোনো মেশিনের ৮০-তে পাঠানো।



conntrack মনে রাখে কোন কানেকশনের ঠিকানা কী বদলানো হয়েছিল, তাই **উত্তরের প্যাকেট অটোমেটিক উল্টো অনুবাদ হয়** — তোমাকে উল্টো দিকের রুল লিখতে হয় না।

ধাপ ০: IP forwarding চালু করো

ডিফল্টে লিনাক্স অন্যের প্যাকেট forward করে না। রাউটার বানাতে:

```
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
$ sudo sysctl -w net.ipv4.ip_forward=1                # এখনই
$ echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-forward.conf
$ sudo sysctl --system                                # স্থায়ী
```

(IPv6-এর জন্য `net.ipv6.conf.all.forwarding = 1`।)

iptables দিয়ে

```
# LAN (eth1) কে ইন্টারনেট (eth0) দাও
sudo iptables -t nat -A POSTROUTING -s 192.168.50.0/24 -o eth0 -j MASQUERADE
sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
```

```
# পোর্ট ফরওয়ার্ড: বাইরের 8080 → ভেতরের 192.168.50.10:80
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8080 -j DNAT --to-destination
192.168.50.10:80
sudo iptables -A FORWARD -i eth0 -o eth1 -p tcp -d 192.168.50.10 --dport 80 -m
conntrack --ctstate NEW -j ACCEPT

# একই মেশিনে পোর্ট রিডাইরেক্ট: 80 → লোকাল 8080
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j REDIRECT --to-ports
8080
```

> 📝 লক্ষ করো, FORWARD রুলে destination port **80** (DNAT-এর পরের ঠিকানা), 8080 না — কারণ FORWARD chain DNAT-এর **পরে** আসে (ডায়াগ্রাম মনে করো)। আর গেটওয়ে মেশিন **নিজে** থেকে `curl localhost:8080` করলে সেটা PREROUTING দিয়ে যায় না (OUTPUT দিয়ে যায়), তাই টেস্ট করবে অন্য মেশিন থেকে।

nftables দিয়ে

```
table ip nat {
    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;
        iifname "eth0" tcp dport 8080 dnat to 192.168.50.10:80
    }
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        ip saddr 192.168.50.0/24 oifname "eth0" masquerade
    }
}

table inet filter {
    chain forward {
        type filter hook forward priority filter; policy drop;
        ct state established,related accept
        iifname "eth1" oifname "eth0" accept
        iifname "eth0" oifname "eth1" ip daddr 192.168.50.10 tcp dport 80 ct state new
accept
    }
}
```

`ct status dnat accept` লিখলে সব DNAT হওয়া কানেকশন একবারে অনুমোদন করা যায়।

ufw দিয়ে

ufw-এ NAT-এর জন্য সরাসরি কমান্ড নেই; ফাইল এডিট করতে হয়:

১. `/etc/default/ufw`-এ: `DEFAULT_FORWARD_POLICY="ACCEPT"` (অথবা DROP রেখে `ufw route allow` দিয়ে নির্দিষ্ট forward অনুমোদন)।

২. `/etc/ufw/sysctl.conf`-এ এই লাইন আনকমেন্ট করো: `net/ipv4/ip_forward=1`

৩. `/etc/ufw/before.rules`-এর **একদম শুরুতে** (`*filter` লাইনের আগে):

```
*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A PREROUTING -i eth0 -p tcp --dport 8080 -j DNAT --to-destination 192.168.50.10:80
-A POSTROUTING -s 192.168.50.0/24 -o eth0 -j MASQUERADE
COMMIT
```

৪. `sudo ufw disable && sudo ufw enable` (বা `sudo ufw reload`)।

firewalld দিয়ে

```
# masquerade (ইন্টারনেটের দিকের zone-এ)
$ sudo firewall-cmd --permanent --zone=external --change-interface=eth0
$ sudo firewall-cmd --permanent --zone=internal --change-interface=eth1
$ sudo firewall-cmd --permanent --zone=external --add-masquerade

# পোর্ট ফরওয়ার্ড অন্য হোস্টে
$ sudo firewall-cmd --permanent --zone=external \
    --add-forward-port=port=8080:proto=tcp:toport=80:toaddr=192.168.50.10

# একই হোস্টে পোর্ট বদল (toaddr ছাড়া)
$ sudo firewall-cmd --permanent --zone=public --add-forward-
port=port=80:proto=tcp:toport=8080

$ sudo firewall-cmd --reload
$ firewall-cmd --zone=external --list-forward-ports
port=8080:proto=tcp:toport=80:toaddr=192.168.50.10
```

masquerade চালু করলে firewalld IPv4 forwarding নিজেই চালু করে নেয়। ভেতরের zone থেকে বাইরের zone-এ forward অনুমতি নিয়ন্ত্রণে নতুন ভার্সনে policy object ব্যবহার হয়।

৩৯.১০ নিজেকে লক-আউট করো না! (রিমোট সার্ভারের সোনালি নিয়ম)

যেকোনো সিস্টেম অ্যাডমিনের জীবনে অন্তত একবার এটা ঘটে: SSH দিয়ে ক্লাউড সার্ভারে ঢুকে ফায়ারওয়াল চালু করলে — আর টার্মিনাল জমে গেল। সার্ভার ৫০০ কিলোমিটার দূরে, কিংবা ক্লাউডে। এখন?

প্রতিরোধের চেকলিস্ট

1. **আগে allow, পরে deny/enable।** SSH পোর্ট (যেটা আসলে ব্যবহার করছ — `ss -tlnp | grep sshd` দিয়ে দেখো) সবার আগে খোলো।
2. **ESTABLISHED,RELATED রুল রাখো।** এটা না থাকলে চলমান SSH সেশনের উত্তর প্যাকেটও আটকে যাবে।
3. **বর্তমান সেশন বন্ধ করবে না।** নতুন টার্মিনাল থেকে নতুন SSH কানেকশন খুলে টেস্ট করো; সফল হলে তবেই পুরনোটা ছাড়ো।

4. **টাইমার-ভিত্তিক রোলব্যাক রাখো** — রুল দেওয়ার আগে একটা "ডেড-ম্যানস সুইচ":

```
# ufw: ৫ মিনিট পর নিজেই disable হবে
$ echo "ufw disable" | sudo at now + 5 minutes      # at প্যাকেজ লাগবে
# অথবা systemd দিয়ে (at ছাড়াই):
$ sudo systemd-run --on-active=5min /usr/sbin/ufw disable

# iptables: আগে ব্যাকআপ, তারপর টাইমার
$ sudo iptables-save > /root/fw-backup.v4
$ sudo systemd-run --on-active=5min /bin/sh -c 'iptables-restore < /root/fw-backup.v4'

# nftables
$ sudo nft list ruleset > /root/nft-backup.nft
$ sudo systemd-run --on-active=5min /bin/sh -c 'nft flush ruleset; nft -f /root/nft-
backup.nft'

# firewalld: runtime-এ timeout দিয়ে টেস্ট, অথবা কৌশল খ (runtime → টেস্ট → runtime-to-
permanent)
```

সব ঠিক থাকলে টাইমার বাতিল করো: `atq` / `atrm <id>`, বা `systemctl list-timers` দেখে `sudo systemctl stop run-XXXX.timer`।

5. **`iptables-apply` ব্যবহার করো** (৩৯.৫ দেখো)।
6. **আউট-অফ-ব্যান্ড অ্যাক্সেস জেনে রাখো:** ক্লাউডের ওয়েব কনসোল (AWS EC2 Serial Console, DigitalOcean Droplet Console ইত্যাদি), ডেটা সেন্টারের IPMI/iDRAC/KVM। লক-আউট হলে এখান দিয়ে ঢুকে `ufw disable` বা `systemctl stop firewalld`।
7. **ক্লাউড ফায়ারওয়াল আলাদা স্তর:** AWS Security Group / GCP firewall rules হোস্টের বাইরে কাজ করে। হোস্টে পোর্ট খুললেও ক্লাউডের দিকে বন্ধ থাকলে কানেকশন আসবে না — দুই জায়গাতেই দেখো।
8. **`-F` এর আগে `-P ACCEPT`** — আবার মনে করিয়ে দিলাম।

> ⚠️ **সতর্কতা:** `firewall-cmd --panic-on`, `nft flush ruleset` (DROP policy-র chain রেখে), `iptables -P INPUT DROP` — এগুলো রিমোট থেকে দেওয়ার আগে তিনবার ভাবো।

৩৯.১১ ফায়ারওয়াল টেস্টিং ও ট্রাবলশুটিং

প্রশ্ন	কীভাবে দেখবে
সার্ভিস আদৌ চলছে ও কোন IP-তে listen করছে?	`sudo ss -tulnp` (বিস্তারিত অধ্যায় ৩৭)
বাইরে থেকে পোর্ট খোলা?	অন্য মেশিন থেকে `nc -zv 192.168.1.10 22` বা `nmap -p 22,80,443 192.168.1.10`
কোন রুল প্যাকেট ধরছে?	`iptables -L -n -v` এর কাউন্টার; nft-এ রুলে `counter` যোগ করো

প্রশ্ন	কীভাবে দেখবে
ড্রপ হওয়া প্যাকেট	LOG রুল / `ufw logging on` / `firewall-cmd --set-log-denied=all`, তারপর `journalctl -k -f`
সব রুল আসলে কী (সব টুলের)	`sudo nft list ruleset` — iptables-nft, ufw, firewalld, Docker সবার রুল এখানে দেখা যায়

```
$ nc -zv 192.168.1.10 22
Connection to 192.168.1.10 22 port [tcp/ssh] succeeded!
$ nc -zv -w 3 192.168.1.10 3306
nc: connect to 192.168.1.10 port 3306 (tcp) timed out: Operation now in progress
```

"timed out" = সম্ভবত DROP; "Connection refused" = REJECT অথবা পোর্টে কেউ listen করছে না।

> 💡 ট্রাবলশুটিং-এর দ্রুত পরীক্ষা: কয়েক সেকেন্ডের জন্য ফায়ারওয়াল বন্ধ করে দেখো সমস্যা যায় কি না (লোকাল মেশিনে, বা কনসোল অ্যাক্সেস থাকলে)। গেলে সমস্যা ফায়ারওয়ালে, না গেলে অন্য কোথাও (সার্ভিস, routing, ক্লাউড ফায়ারওয়াল)।

৩৯.১২ কোন টুল কখন?

পরিস্থিতি	সুপারিশ
Ubuntu ডেস্কটপ/সাধারণ সার্ভার	**ufw**
Fedora / RHEL / Rocky / Alma	**firewalld** (ডিফল্ট, বদলানোর দরকার নেই)
একাধিক ইন্টারফেস, ল্যাপটপ যা বিভিন্ন নেটওয়ার্কে যায়	**firewalld** (zone)
জটিল রাউটার/গেটওয়ে, হাজার IP-র blocklist, পূর্ণ নিয়ন্ত্রণ	**nftables** সরাসরি, ফাইলে
পুরনো স্ক্রিপ্ট, Docker/Kubernetes ভেতরের রুল বোঝা, ইন্টারভিউ	**iptables** জানতেই হবে

৩৯.১৩ কীবোর্ড শর্টকাট ও দ্রুত কাজের কৌশল

ফায়ারওয়াল টুলগুলোর নিজস্ব কীবোর্ড শর্টকাট নেই (এগুলো কমান্ড-লাইন টুল), তবে টার্মিনালের এই শর্টকাটগুলো ফায়ারওয়াল নিয়ে কাজ করার সময় খুব কাজে লাগে (সব শর্টকাট অধ্যায় ৬-এ):

শর্টকাট	কাজে লাগে যখন
`Ctrl + R` তারপর `iptables` টাইপ	আগের লম্বা iptables/firewall-cmd কমান্ড খুঁজে আবার চালাতে
`Alt + .`	আগের কমান্ডের শেষ argument বসাতে (যেমন একই IP বা ফাইল)
`Ctrl + A` / `Ctrl + E`	লম্বা রুলের শুরু/শেষে যেতে — `-A` কে `-D` বানাতে

শর্টকাট	কাজে লাগে যখন
`Ctrl + W`	এক শব্দ মুছতে
`!!`	আগের কমান্ড — `sudo !!` (sudo দিতে ভুলে গেলে)
`^old^new`	আগের কমান্ডে প্রথম `old` কে `new` দিয়ে বদলে চালাও — যেমন `^-A^-D` রুল মুছতে
`Ctrl + C`	ufw-এর `(y|n)` প্রশ্ন বা ঝুলে থাকা `nc` / `curl` বাতিল করতে
`watch -n1 'sudo iptables -L -n -v'`	কাউন্টার লাইভ দেখতে (`Ctrl + C` দিয়ে বের হও)

GUI টুল: Ubuntu-তে **gufw** (`sudo apt install gufw`), Fedora/RHEL-এ **firewall-config** — দুটোই মাউস দিয়ে রুল দেওয়ার জন্য; শেখার সময় ভেতরে কী রুল তৈরি হলো তা `nft list ruleset` দিয়ে মিলিয়ে দেখো।

হাতে-কলমে অনুশীলন

> ⚠️ এই ল্যাবগুলো একটা **লোকাল VM**-এ করো (VirtualBox/KVM — অধ্যায় ৪৮), প্রোডাকশন সার্ভারে না। দুটো VM থাকলে ভালো (একটা থেকে অন্যটা টেস্ট করবে)।

ল্যাব ১: ufw দিয়ে ওয়েব সার্ভার সুরক্ষা (Ubuntu)

```
# ১. একটা টেস্ট ওয়েব সার্ভার
$ sudo apt install -y nginx
$ curl -sI http://localhost | head -1
HTTP/1.1 200 OK

# ২. ফায়ারওয়ালের অবস্থা
$ sudo ufw status
Status: inactive

# ৩. ডিফল্ট আর SSH
$ sudo ufw default deny incoming
$ sudo ufw default allow outgoing
$ sudo ufw limit OpenSSH

# ৪. চালু
$ sudo ufw enable

# ৫. অন্য VM থেকে: curl --max-time 3 http://<VM-IP>  → timeout হওয়ার কথা
# ৬. HTTP খোলো
$ sudo ufw allow 'Nginx HTTP'
# ৭. আবার অন্য VM থেকে curl → এবার 200 OK
# ৮. দেখো ও মোছো
```

```
$ sudo ufw status numbered
$ sudo ufw delete allow 'Nginx HTTP'
```

ল্যাব ২: iptables-এর কাউন্টার দিয়ে প্যাকেট গোনা

```
$ sudo ufw disable                        # ল্যাবের জন্য ufw সরাও
$ sudo iptables-save > ~/before.v4        # ব্যাকআপ
$ sudo iptables -I INPUT -p icmp --icmp-type echo-request -j ACCEPT
$ sudo iptables -L INPUT -n -v --line-numbers | head -4
# অন্য VM থেকে: ping -c 5 <VM-IP>
$ sudo iptables -L INPUT -n -v --line-numbers | head -4
# প্রথম রুলের pkts কলাম ৫ বেড়েছে কি?
$ sudo iptables -R INPUT 1 -p icmp --icmp-type echo-request -j DROP
# আবার ping → এবার কোনো উত্তর নেই, কিন্তু কাউন্টার বাড়ছে
$ sudo iptables-restore < ~/before.v4     # আগের অবস্থায় ফেরো
```

ল্যাব ৩: nftables কনফিগ ফাইল লিখে লোড

১. ৩৯.৬-এর `/etc/nftables.conf` উদাহরণটা কপি করো, `ssh_allowed` set-এ নিজের নেটওয়ার্ক বসাও। ২. `sudo nft -c -f /etc/nftables.conf` — কোনো আউটপুট না এলে সিনট্যাক্স ঠিক। ৩. রোলব্যাক টাইমার: `sudo systemd-run --on-active=3min /usr/sbin/nft flush ruleset` ৪. `sudo nft -f /etc/nftables.conf` ৫. নতুন টার্মিনাল থেকে SSH করো। সফল? তাহলে টাইমার বন্ধ করো (`systemctl list-timers | grep run-` দেখে `systemctl stop`)। ৬. `sudo nft -a list ruleset` দিয়ে handle দেখো, ICMP রুলটা handle দিয়ে মুছে ping টেস্ট করো।

ল্যাব ৪: firewalld zone ও rich rule (Fedora/Rocky)

```
$ sudo firewall-cmd --get-active-zones
$ sudo firewall-cmd --add-service=http --timeout=2m        # ২ মিনিটের জন্য
$ firewall-cmd --list-services                             # http আছে
# ২ মিনিট পর আবার দেখো — http চলে গেছে
$ sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="
<অন্য VM IP>" port port="8000" protocol="tcp" accept'
$ sudo firewall-cmd --reload
$ python3 -m http.server 8000                               # টেস্ট সার্ভার
# অন্য VM থেকে curl → কাজ করে; তৃতীয় কোনো মেশিন থেকে → reject
```

ল্যাব ৫ (চ্যালেঞ্জ): দুই VM দিয়ে NAT গেটওয়ে

VM-A-তে দুটো নেটওয়ার্ক কার্ড (একটা NAT/ইন্টারনেট, একটা internal network), VM-B শুধু internal network-এ। VM-A-তে `ip_forward` চালু করে MASQUERADE দাও, VM-B-র default gateway VM-A দাও (`sudo ip route add default via <VM-A internal IP>`)। VM-B থেকে `ping 1.1.1.1` কাজ করলে তুমি একটা রাউটার বানিয়ে ফেলেছ! তারপর VM-A-র ৮০৮০ পোর্ট VM-B-র ৮০-তে DNAT করো।

সারসংক্ষেপ

লিনাক্সের আসল ফায়ারওয়াল কার্নেলের **netfilter**; iptables/nftables তাকে সরাসরি রুল দেয়, ufw/firewalld সহজ ফ্রন্টএন্ড। প্যাকেট **PREROUTING → INPUT/FORWARD → OUTPUT → POSTROUTING** হুক দিয়ে চলে; রুল উপর থেকে নিচে মেলে, প্রথম মিলই জেতে, না মিললে policy। **conntrack**-এর `ESTABLISHED,RELATED` রুল stateful ফায়ারওয়ালের ভিত্তি। রিমোটে সবসময় আগে SSH খোলো এবং রোলব্যাক টাইমার রাখো।

দ্রুত রেফারেন্স

কাজ	iptables	nftables	ufw	firewalld
রুল দেখা	`iptables -L -n -v --line-numbers`	`nft -a list ruleset`	`ufw status numbered`	`firewall-cmd --list-all`
পোর্ট খোলা	`iptables -A INPUT -p tcp --dport 80 -j ACCEPT`	`nft add rule inet f input tcp dport 80 accept`	`ufw allow 80/tcp`	`firewall-cmd --permanent --add-port=80/tcp`
সার্ভিস নামে	—	—	`ufw allow 'Nginx Full'`	`firewall-cmd --permanent --add-service=http`
IP ব্লক	`iptables -I INPUT -s IP -j DROP`	`nft insert rule inet f input ip saddr IP drop`	`ufw insert 1 deny from IP`	`--add-rich-rule='rule family="ipv4" source address="IP" drop'`
রুল মোছা	`iptables -D INPUT 3`	`nft delete rule inet f input handle 5`	`ufw delete 3`	`--remove-port=80/tcp`
ডিফল্ট policy	`iptables -P INPUT DROP`	`policy drop;`	`ufw default deny incoming`	zone বেছে নাও (`--set-default-zone`)
প্রয়োগ/লোড	তাৎক্ষণিক	`nft -f file`	তাৎক্ষণিক	`firewall-cmd --reload`
স্থায়ী করা	`iptables-save` / `netfilter-persistent save`	`systemctl enable nftables`	অটো	`--permanent` / `--runtime-to-permanent`
চালু/বন্ধ	—	`systemctl start/stop nftables`	`ufw enable/disable`	`systemctl start/stop firewalld`
NAT	`-t nat ... -j MASQUERADE / DNAT`	`masquerade` , `dnat to`	`/etc/ufw/before.rules`	`--add-masquerade` , `--add-forward-port`

অন্যান্য দরকারি কমান্ড	কাজ
`iptables --version`	ব্যাকএন্ড (nf_tables/legacy)
`iptables-apply file`	টাইমআউট-সহ নিরাপদ প্রয়োগ
`iptables-translate ...`	iptables → nft অনুবাদ
`conntrack -L` / `-C`	কানেকশন টেবিল
`ufw app list` / `app info X`	অ্যাপ প্রোফাইল
`firewall-cmd --get-active-zones`	কোন ইন্টারফেস কোন zone-এ
`firewall-cmd ... --timeout=5m`	সাময়িক রুল
`systemd-run --on-active=5min CMD`	লক-আউট রোলব্যাক টাইমার
`sysctl -w net.ipv4.ip_forward=1`	রাউটিং চালু

নিজেকে যাচাই করো

1. ufw, firewalld, iptables, nftables — এদের মধ্যে কোনটা আসলে প্যাকেট ফিল্টার করে? সম্পর্কটা এক লাইনে বলো।
2. একটা প্যাকেট এই মেশিনের nginx-এর জন্য এসেছে। কোন কোন হুক দিয়ে যাবে? আর যদি মেশিনটা রাউটার হয়ে প্যাকেট পার করে দেয়?
3. DROP আর REJECT-এর পার্থক্য কী? ক্লায়েন্টের দিক থেকে কী দেখা যায়?
4. `iptables -A INPUT -j DROP` থাকার পর `iptables -A INPUT -p tcp --dport 80 -j ACCEPT` দিলে পোর্ট ৮০ কাজ করবে না কেন? কীভাবে ঠিক করবে?
5. `ct state established,related accept` রুল না থাকলে কী সমস্যা হবে?
6. firewalld-এ `--permanent` দিয়ে সার্ভিস যোগ করার পরেও পোর্ট খুলছে না কেন? দুটো সমাধান বলো।
7. ufw-এ `ufw status numbered` থেকে নম্বর দিয়ে IPv4 রুল মুছলে IPv6 রুলের কী হয়?
8. Docker কন্টেইনারের পাবলিশ করা পোর্ট ufw-এর deny রুল মানে না কেন?
9. বাইরের 8080 পোর্টকে ভেতরের 192.168.50.10:80-তে পাঠাতে কোন হুকে কোন ধরনের NAT লাগবে? আর কোন kernel সেটিং চালু করতে হবে?
10. রিমোট সার্ভারে নতুন ফায়ারওয়াল রুল প্রয়োগের আগে লক-আউট এড়াতে অন্তত তিনটা কৌশল বলো।

<details>
<summary>▼ উত্তর দেখো</summary>

1. আসল ফিল্টারিং করে কার্নেলের **netfilter**। iptables ও nftables (`nft`) সরাসরি তাকে রুল দেয়; ufw (ভেতরে iptables) আর firewalld (ভেতরে nftables) হলো সহজ ফ্রন্টএন্ড।
2. লোকাল: **PREROUTING** → **INPUT** → nginx। রাউটার হিসেবে: **PREROUTING** → **FORWARD** → **POSTROUTING**।
3. DROP চুপচাপ ফেলে দেয় — ক্লায়েন্ট timeout পর্যন্ত অপেক্ষা করে। REJECT একটা ICMP error বা TCP RST পাঠায় — ক্লায়েন্ট সাথে সাথে "Connection refused" (বা unreachable) পায়।
</details>

4. রুল উপর থেকে নিচে মেলে; `-A` রুলটা DROP-এর পরে বসেছে, তাই প্যাকেট আগেই DROP-এ মিলে যায়। সমাধান: `iptables -I INPUT -p tcp --dport 80 -j ACCEPT` (বা DROP রুলের আগের নম্বরে `-I INPUT n`), অথবা শেষের DROP রুল বাদ দিয়ে policy DROP ব্যবহার।
5. তোমার নিজের শুরু করা কানেকশনের (DNS, apt update, curl) উত্তর প্যাকেট INPUT policy DROP-এ আটকে যাবে; চলমান SSH সেশনও ভেঙে যেতে পারে, কারণ সার্ভারে আসা প্রতিটা প্যাকেট আলাদা রুলে মেলাতে হবে।
6. `--permanent` শুধু ফাইলে লেখে, runtime-এ প্রয়োগ করে না। সমাধান: `firewall-cmd --reload`; অথবা `--permanent` ছাড়া একই কমান্ডটা আবার চালাও (runtime-এর জন্য)। বিকল্প হিসেবে runtime-এ যোগ করে `--runtime-to-permanent`।
7. IPv6 রুলটা থেকে যায় — নম্বর দিয়ে মুছলে শুধু ঐ নির্দিষ্ট রুল মোছে। দুটো একসাথে মুছতে রুলের নাম দিয়ে মোছো: `ufw delete allow 8080/tcp`।
8. Docker nat PREROUTING-এ DNAT করে প্যাকেট কন্টেইনারের দিকে **FORWARD** করে, তাই প্যাকেট INPUT chain-এ যায়ই না, যেখানে ufw-এর রুল থাকে। সমাধান: `-p 127.0.0.1:PORT:PORT`, অথবা `DOCKER-USER` chain-এ রুল।
9. **PREROUTING** হুকে **DNAT** (`--to-destination 192.168.50.10:80` / `dnat to`), সাথে FORWARD chain-এ অনুমতি, আর উত্তরের রাস্তা ঠিক রাখতে দরকার হলে POSTROUTING-এ MASQUERADE। kernel-এ `net.ipv4.ip_forward=1`।
10. যেকোনো তিনটা: (ক) enable/policy DROP-এর আগে SSH allow; (খ) ESTABLISHED,RELATED রুল রাখা; (গ) `systemd-run --on-active=5min` বা `at` দিয়ে রোলব্যাক টাইমার; (ঘ) `iptables-apply`; (ঙ) firewalld-এ runtime/`--timeout`-এ আগে টেস্ট; (চ) পুরনো সেশন খোলা রেখে নতুন সেশনে টেস্ট; (ছ) ক্লাউড/IPMI কনসোল অ্যাক্সেস প্রস্তুত রাখা; (জ) `-F` এর আগে `-P INPUT ACCEPT`।

অধ্যায় ৪০: সার্ভার সেটআপ — ওয়েব সার্ভার ও সাধারণ সার্ভিস

> এতক্ষণ তুমি লিনাক্সের খুঁটিনাটি শিখেছ: ফাইল, পারমিশন, প্রসেস, systemd, নেটওয়ার্ক, SSH, ফায়ারওয়াল। এই অধ্যায়ে সব একসাথে জোড়া লাগবে। একদম ফাঁকা একটা লিনাক্স সার্ভার নিয়ে আমরা সেটাকে এমন জায়গায় নিয়ে যাব যেখানে ইন্টারনেটে একটা ওয়েবসাইট HTTPS-সহ চলছে, পেছনে একটা ডেটাবেস আছে, আর অ্যাপটা systemd দেখাশোনা করছে।

এই অধ্যায়ে যা শিখবে

- নতুন লিনাক্স সার্ভার হাতে পেলে প্রথম ৩০ মিনিটে কী কী করতে হয় (চেকলিস্ট)
- ওয়েব সার্ভার আসলে কী, আর একটা HTTP request ভেতরে কীভাবে চলে
- nginx ইনস্টল, কনফিগ ফাইলের গঠন: `http`, `server` block, `location`, `root`, `index`, `try_files`
- nginx দিয়ে reverse proxy (Node/Python অ্যাপের সামনে বসানো)
- Apache (httpd) পরিচিতি: VirtualHost, মডিউল, `a2ensite` / `a2enmod`, nginx-এর সাথে তুলনা
- Let's Encrypt আর `certbot` দিয়ে বিনামূল্যে HTTPS, আর অটো-রিনিউ
- MySQL/MariaDB ইনস্টল, নিরাপদ করা, ডেটাবেস ও ইউজার তৈরি
- PostgreSQL ইনস্টল, `psql`, role আর `pg_hba.conf`
- PHP-FPM কী এবং nginx-এর সাথে কীভাবে যুক্ত হয়
- Node.js অ্যাপকে systemd service বানিয়ে চালানো
- nginx, Apache, ডেটাবেস আর অ্যাপের লগ কোথায় থাকে, কীভাবে পড়বে
- LAMP আর LEMP স্ট্যাকের ছবি ও তুলনা
- সাধারণ সমস্যা (502, 403, 404, পোর্ট দখল) ধরে ফেলার কৌশল

৪০.১ সার্ভার কী, সার্ভিস কী — ছোট্ট ভূমিকা

Server (সার্ভার) শব্দটার দুটো মানে আছে:

1. **হার্ডওয়্যার/মেশিন** অর্থে: একটা কম্পিউটার (প্রায়ই ক্লাউডের VPS), যেটা দিনরাত চালু থাকে আর অন্যদের সেবা দেয়।
2. **সফটওয়্যার** অর্থে: একটা প্রোগ্রাম যেটা কোনো পোর্টে কান পেতে (listen করে) বসে থাকে, request এলে উত্তর দেয়। যেমন nginx (ওয়েব সার্ভার), MySQL (ডেটাবেস সার্ভার), sshd (SSH সার্ভার)।

এই দ্বিতীয় ধরনের প্রোগ্রামগুলোকে লিনাক্সে **service** বা **daemon** বলে। এরা ব্যাকগ্রাউন্ডে চলে, কোনো টার্মিনালের সাথে বাঁধা থাকে না, আর systemd এদের চালু-বন্ধ-রিস্টার্ট করে (অধ্যায় ২৬)।

উপমা: একটা রেস্টুরেন্ট ভাবো। ভবনটা হলো মেশিন। দরজার রিসেপশনিস্ট হলো nginx — সে অতিথিকে (request) স্বাগত জানায়, ঠিক টেবিলে পাঠায়। রান্নাঘরের শেফ হলো তোমার অ্যাপ (PHP/Node)। আর স্টোররুম হলো ডেটাবেস। ম্যানেজার (systemd) খেয়াল রাখে কেউ অসুস্থ হয়ে পড়লে (crash) যেন আবার কাজে ফেরে।

একটা ওয়েব request-এর যাত্রা

ব্রাউজারে `https://example.com/products` লিখে Enter চাপলে কী হয়?

```
ব্রাউজার                                  সার্ভার (লিনাক্স)
─────────                                  ──────────────────
1. DNS: example.com -> 203.0.113.10
2. TCP connect 203.0.113.10:443  ──────>  kernel (ফায়ারওয়াল পার হলো)
3. TLS handshake (সার্টিফিকেট)   <──────>  nginx (port 443 এ listen করছে)
4. HTTP: GET /products           ──────>  nginx: কোন server block? কোন location?
                                              |
                                              ├── static ফাইল? -> ডিস্ক থেকে পাঠাও
                                              |
                                              └── ডাইনামিক? -> proxy_pass / fastcgi_pass
                                                        |
                                                        v
                                                  অ্যাপ (Node / PHP-FPM)
                                                        |
                                                        v
                                                  ডেটাবেস (MySQL / PostgreSQL)
5. HTTP 200 + HTML               <──────  উত্তর ফেরত একই পথে
```

DNS, TCP, পোর্টের ধারণা অধ্যায় ৩৬-এ আছে; ফায়ারওয়াল অধ্যায় ৩৯-এ।

৪০.২ নতুন সার্ভার প্রস্তুতির চেকলিস্ট

ক্লাউড প্রোভাইডার (DigitalOcean, AWS, Hetzner ইত্যাদি) থেকে নতুন Ubuntu 24.04 সার্ভার পেলে সাধারণত শুধু একটা IP আর root লগইন (বা SSH key) পাও। কোনো অ্যাপ বসানোর আগে নিচের কাজগুলো করো। প্রতিটি ধাপের বিস্তারিত অন্য অধ্যায়ে আছে, এখানে শুধু ক্রমটা মনে রাখার জন্য একসাথে দেওয়া হলো।

#	কাজ	কমান্ড (Ubuntu/Debian)	কেন	বিস্তারিত
1	সিস্টেম আপডেট	`apt update && apt full-upgrade -y`	পুরোনো প্যাকেজে জানা নিরাপত্তা-ছিদ্র থাকে	অধ্যায় ৩০
2	hostname ঠিক করা	`hostnamectl set-hostname web1`	লগ আর prompt-এ মেশিন চেনা যায়	অধ্যায় ৩৬
3	timezone	`timedatectl set-timezone Asia/Dhaka`	লগের সময় বোঝা সহজ	অধ্যায় ২৮

#	কাজ	কমান্ড (Ubuntu/Debian)	কেন	বিস্তারিত
4	সাধারণ ইউজার তৈরি	`adduser deploy`	সবসময় root হয়ে কাজ করা বিপজ্জনক	অধ্যায় ১৫
5	sudo দেওয়া	`usermod -aG sudo deploy` (RHEL: `wheel`)	দরকার হলে তবেই প্রশাসনিক ক্ষমতা	অধ্যায় ১৮
6	SSH key কপি	`ssh-copy-id deploy@IP` (নিজের পিসি থেকে)	পাসওয়ার্ডের চেয়ে অনেক নিরাপদ	অধ্যায় ৩৮
7	SSH শক্ত করা	`PermitRootLogin no`, `PasswordAuthentication no`	brute-force আক্রমণ বন্ধ	অধ্যায় ৩৮
8	ফায়ারওয়াল	`ufw allow OpenSSH && ufw enable`	শুধু দরকারি পোর্ট খোলা থাকবে	অধ্যায় ৩৯
9	অটো সিকিউরিটি আপডেট	`apt install unattended-upgrades`	ঘুমের মধ্যেও প্যাচ বসবে	অধ্যায় ৪৫
10	fail2ban (ঐচ্ছিক)	`apt install fail2ban`	বারবার ভুল লগইন করা IP ব্লক	অধ্যায় ৪৫
11	swap (ছোট RAM হলে)	`fallocate -l 2G /swapfile` ...	RAM শেষ হলে OOM-killer এড়াতে	অধ্যায় ৩২
12	সময় সিঙ্ক যাচাই	`timedatectl status`	TLS আর লগের জন্য সঠিক সময় জরুরি	—
13	ব্যাকআপ পরিকল্পনা	—	ডেটা হারালে কিছুই বাঁচে না	অধ্যায় ৩৫

Fedora/RHEL 9-এ সমতুল্য: `dnf upgrade -y`, `usermod -aG wheel deploy`, ফায়ারওয়াল `firewall-cmd`, আর অটো আপডেটের জন্য `dnf install dnf-automatic` তারপর `systemctl enable --now dnf-automatic.timer`।

> ⚠️ **সতর্কতা:** SSH-এ `PasswordAuthentication no` করার **আগে** নতুন একটা টার্মিনাল থেকে key দিয়ে লগইন হচ্ছে কিনা পরীক্ষা করো। পুরোনো সেশন খোলা রাখো। নাহলে নিজেকেই সার্ভারের বাইরে তালাবদ্ধ করে ফেলবে। ufw চালু করার আগে `ufw allow OpenSSH` দিতেও ভুলবে না।

> 💡 **টিপস:** চেকলিস্টের ধাপগুলো একটা স্ক্রিপ্টে লিখে রাখলে প্রতিটি নতুন সার্ভারে একই কাজ মিনিটে হয়ে যায়। স্ক্রিপ্ট লেখা শিখবে অধ্যায় ৪১ থেকে। বড় পরিসরে মানুষ Ansible-এর মতো টুল ব্যবহার করে।

সার্ভারে এখন কোন কোন পোর্টে কে listen করছে তা দেখার অভ্যাস করো:

```
$ sudo ss -tlnp
State   Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
```

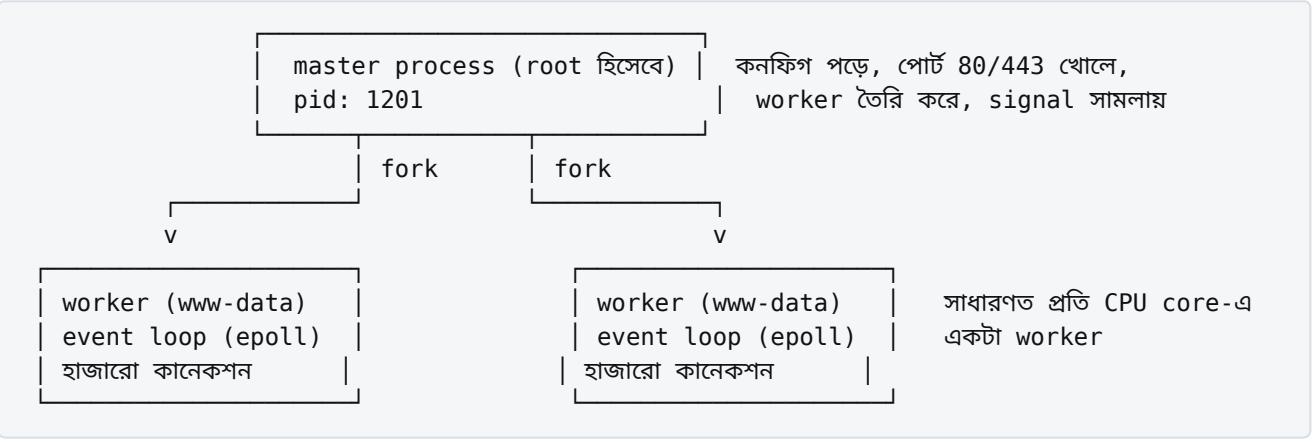
```
LISTEN  0       4096    0.0.0.0:22          0.0.0.0:*          users:
(("sshd",pid=812,fd=3))
LISTEN  0       4096    127.0.0.53%lo:53    0.0.0.0:*          users:(("systemd-
resolve",pid=530,fd=14))
```

এই অধ্যায় শেষে এখানে 80, 443, 3306/5432, 3000 ইত্যাদি দেখবে।

৪০.৩ nginx — আধুনিক ওয়েব সার্ভার

nginx (উচ্চারণ "engine-x") একটা ওয়েব সার্ভার, reverse proxy আর load balancer। এটা খুব কম মেমরিতে একসাথে হাজার হাজার কানেকশন সামলাতে পারে, তাই আজকের বেশিরভাগ ওয়েবসাইটের সামনে এটাই থাকে।

nginx ভেতরে কীভাবে কাজ করে



- **master process** root হিসেবে চলে, কারণ 1024-এর নিচের পোর্ট (80, 443) খুলতে বিশেষ ক্ষমতা লাগে। এটা নিজে কোনো request সামলায় না।
- **worker process** কম ক্ষমতার ইউজার হিসেবে চলে (Ubuntu/Debian-এ `www-data`, Fedora/RHEL-এ `nginx`)। ফলে কোনো worker হ্যাক হলেও আক্রমণকারী root পায় না।
- প্রতিটি worker একটা **event loop** চালায়। প্রতি কানেকশনের জন্য আলাদা thread না বানিয়ে kernel-এর `epoll` ব্যবস্থা দিয়ে জিজ্ঞেস করে, "কোন কানেকশনে এখন ডেটা এসেছে?" তারপর শুধু সেগুলোর কাজ করে। এটাকে **event-driven, non-blocking** আর্কিটেকচার বলে।

উপমা: পুরোনো ধাঁচের সার্ভার (যেমন Apache-এর prefork) প্রতিটি অতিথির জন্য একজন করে ওয়েটার রাখে, অতিথি মেনু পড়তে ১০ মিনিট নিলে ওয়েটারও ১০ মিনিট দাঁড়িয়ে থাকে। nginx-এর একজন ওয়েটার ঘুরে ঘুরে দেখে কোন টেবিল অর্ডার দিতে প্রস্তুত, শুধু সেখানেই যায়। তাই অল্প লোকে বেশি টেবিল সামলানো যায়।

`systemctl reload nginx` দিলে master নতুন কনফিগ পড়ে, নতুন worker চালু করে আর পুরোনো worker-দের হাতের কাজ শেষ করে বিদায় নিতে বলে। এজন্য reload-এ কোনো কানেকশন কাটে না।

ইনস্টল

```
# Ubuntu / Debian
sudo apt update
sudo apt install nginx

# Fedora / RHEL 9
sudo dnf install nginx
sudo systemctl enable --now nginx     # RHEL-এ নিজে থেকে চালু হয় না
```

Ubuntu/Debian-এ ইনস্টলের সাথে সাথেই service চালু ও enable হয়ে যায়। যাচাই করো:

```
$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-11-11 10:12:03 +06; 8s ago
   Main PID: 2311 (nginx)
      Tasks: 3 (limit: 2276)
     Memory: 2.6M
     CGroup: /system.slice/nginx.service
             ├─2311 "nginx: master process /usr/sbin/nginx -g daemon on; master_process
on;"
             ├─2312 "nginx: worker process"
             └─2313 "nginx: worker process"

$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Content-Type: text/html
...
```

ফায়ারওয়ালে পোর্ট খোলো:

```
# Ubuntu (ufw) — nginx প্যাকেজ নিজস্ব app profile দেয়
sudo ufw app list            # Nginx Full, Nginx HTTP, Nginx HTTPS
sudo ufw allow 'Nginx Full'  # 80 আর 443 দুটোই

# Fedora / RHEL (firewalld)
sudo firewall-cmd --permanent --add-service=http --add-service=https
sudo firewall-cmd --reload
```

এখন ব্রাউজারে `http://সার্ভারের-IP` খুললে "Welcome to nginx!" পাতা দেখবে।

nginx-এর ফাইল কোথায় থাকে

জিনিস	Ubuntu / Debian	Fedora / RHEL
মূল কনফিগ	`/etc/nginx/nginx.conf`	`/etc/nginx/nginx.conf`
সাইটের কনফিগ	`/etc/nginx/sites-available/` + `sites-enabled/` (symlink)	`/etc/nginx/conf.d/*.conf`

জিনিস	Ubuntu / Debian	Fedora / RHEL
অতিরিক্ত কনফিগ	`/etc/nginx/conf.d/*.conf`, `/etc/nginx/snippets/`	`/etc/nginx/default.d/`
ডিফল্ট ওয়েব রুট	`/var/www/html`	`/usr/share/nginx/html`
লগ	`/var/log/nginx/access.log`, `error.log`	একই
worker ইউজার	`www-data`	`nginx`

Debian-এর `sites-available` / `sites-enabled` ব্যবস্থাটা সহজ: সব সাইটের কনফিগ `sites-available`-এ রাখো, যেগুলো চালু চাও শুধু তাদের symlink (অধ্যায় ১১) `sites-enabled`-এ দাও। বন্ধ করতে symlink মুছে ফেলো, আসল ফাইল থেকে যায়।

```
sudo ln -s /etc/nginx/sites-available/mysite /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default      # ডিফল্ট সাইট বন্ধ
```

কনফিগের গঠন: context আর directive

nginx কনফিগ দুই রকম জিনিস দিয়ে তৈরি:

- **directive**: `নাম মান;` (শেষে সেমিকোলন বাধ্যতামূলক), যেমন `worker_processes auto;`
- **block / context**: `নাম { ... }`, যার ভেতরে আরও directive থাকে।

block-গুলো একটার ভেতরে আরেকটা, পেঁয়াজের খোসার মতো:

```
nginx.conf
┌──────────────────────────────────────────────────────────┐
│ main context: user, worker_processes, error_log, pid     │
│                                                          │
│ events { worker_connections 768; }                       │
│                                                          │
│ http {                                                   │
│   sendfile, gzip, log_format, include mime.types ...     │
│   include /etc/nginx/conf.d/*.conf;                      │
│   include /etc/nginx/sites-enabled/*;                    │
│                                                          │
│   ┌── server { listen 80; server_name a.com; ──────┐     │
│   │     root /var/www/a;                           │     │
│   │     location / { ... }                         │     │
│   │     location /api/ { proxy_pass ...; }         │     │
│   └────────────────────────────────────────────────┘     │
│   ┌── server { listen 80; server_name b.com; ──────┐     │
│   │     location / { ... }                         │     │
│   └────────────────────────────────────────────────┘     │
│ }                                                        │
└──────────────────────────────────────────────────────────┘
```

বাইরের block-এ লেখা directive ভেতরের block উত্তরাধিকারসূত্রে পায় (inherit করে), যদি না ভেতরে আবার নতুন করে লেখা হয়।

Ubuntu-র `/etc/nginx/nginx.conf`-এর গুরুত্বপূর্ণ অংশ (সংক্ষেপে):

```
user www-data;
worker_processes auto;          # CPU core অনুযায়ী worker সংখ্যা
pid /run/nginx.pid;

events {
    worker_connections 768;     # প্রতি worker সর্বোচ্চ কতগুলো কানেকশন
}

http {
    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;   # (নতুন সংস্করণে এটা main context-এ থাকতে পারে)

    gzip on;

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

> 📝 **মনে রাখো:** মূল `nginx.conf` সাধারণত হাত দেওয়ার দরকার হয় না। নিজের সাইটের জন্য আলাদা ফাইল বানাও (Ubuntu-তে `sites-available/`, RHEL-এ `conf.d/নাম.conf`)। প্যাকেজ আপগ্রেডের সময় তাতে ঝামেলা কম হয়।

প্রথম server block: একটা static সাইট

ধরো ডোমেইন `example.com`, ফাইল রাখব `/var/www/example`-এ।

```
sudo mkdir -p /var/www/example
echo '<h1>Hello from nginx</h1>' | sudo tee /var/www/example/index.html
sudo chown -R deploy:www-data /var/www/example
sudo chmod -R 755 /var/www/example
```

`/etc/nginx/sites-available/example` ফাইল:

```
server {
    listen 80;                          # IPv4 পোর্ট 80
    listen [::]:80;                     # IPv6 পোর্ট 80
    server_name example.com www.example.com;
```

```
    root /var/www/example;              # ফাইল কোথায় খুঁজবে
    index index.html index.htm;         # ডিরেক্টরি চাইলে কোন ফাইল দেখাবে

    access_log /var/log/nginx/example.access.log;
    error_log  /var/log/nginx/example.error.log;

    location / {
        try_files $uri $uri/ =404;      # ফাইল আছে? ডিরেক্টরি আছে? নাহলে 404
    }
}
```

চালু করো:

```
$ sudo ln -s /etc/nginx/sites-available/example /etc/nginx/sites-enabled/
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
$ sudo systemctl reload nginx
$ curl -H 'Host: example.com' http://localhost
<h1>Hello from nginx</h1>
```

`curl -H 'Host: ...'` দিয়ে ডোমেইন DNS-এ না থাকলেও নির্দিষ্ট server block পরীক্ষা করা যায়।

> ⚠️ **সতর্কতা:** কনফিগ বদলানোর পর **সবসময়** আগে `sudo nginx -t`, তারপর reload। ভুল কনফিগে `restart` দিলে nginx উঠবেই না, আর তোমার সব সাইট বন্ধ হয়ে যাবে। `reload` ভুল কনফিগ পেলে পুরোনো কনফিগেই চলতে থাকে, তবুও `nginx -t` আগে চালানোই নিয়ম।

nginx কোন server block বেছে নেয়?

একটা সার্ভারে দশটা সাইট থাকতে পারে, সবাই পোর্ট 80-তে। nginx ঠিক করে এভাবে:

1. যে IP:port-এ request এসেছে, সেটার সাথে মেলে এমন `listen` থাকা server block-গুলো আলাদা করে।
2. তাদের মধ্যে request-এর `Host` header-কে `server_name`-এর সাথে মেলায়: আগে হুবহু নাম, তারপর সামনে wildcard (`*.example.com`), তারপর পেছনে wildcard (`mail.*`), তারপর regex (`~^...`)।
3. কেউ না মিললে ওই পোর্টের **default server**-এ যায়: যেটায় `listen 80 default_server;` লেখা, না থাকলে কনফিগে প্রথম যেটা পাওয়া গেছে।

অচেনা Host বা সরাসরি IP দিয়ে আসা request ফেলে দিতে একটা "ক্যাচ-অল" রাখা ভালো অভ্যাস:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
```

```
    return 444;     # nginx-এর বিশেষ কোড: কোনো উত্তর না দিয়ে কানেকশন বন্ধ
}
```

location block — URL অনুযায়ী আচরণ

`location` বলে দেয় URL-এর কোন অংশে কী করতে হবে। এর কয়েকটা ধরন আছে:

সিনট্যাক্স	মানে	উদাহরণ
`location /path/`	prefix মিল (সবচেয়ে লম্বা মিলটা জেতে)	`location /images/`
`location = /path`	হুবহু মিল, পেলে সাথে সাথে থামে	`location = /favicon.ico`
`location ^~ /path/`	prefix মিল; এটা সবচেয়ে লম্বা prefix হলে regex আর দেখে না	`location ^~ /static/`
`location ~ regex`	regex, বড়-ছোট হাতের অক্ষর আলাদা ধরে	`location ~ \.php$`
`location ~* regex`	regex, বড়-ছোট হাতের অক্ষর একই ধরে	`location ~* \.(jpg|png)$`

nginx যেভাবে বেছে নেয়:

```
request URI: /static/logo.png
        │
        v
 ১. কোনো "= " হুবহু মিল আছে? ── হ্যাঁ ──> সেটাই, শেষ
        │ না
        v
 ২. সব prefix location থেকে সবচেয়ে লম্বা মিলটা মনে রাখো
        │
        ├── সেটায় "^~" আছে? ── হ্যাঁ ──> সেটাই, শেষ
        │ না
        v
 ৩. regex location গুলো কনফিগে যে ক্রমে লেখা সেই ক্রমে পরীক্ষা করো
        │
        ├── প্রথম যেটা মিলল ──> সেটাই, শেষ
        │ কোনোটা না
        v
 ৪. ধাপ ২-এর সবচেয়ে লম্বা prefix ব্যবহার করো
```

> 📝 **মনে রাখো:** regex location-এ **ক্রম** গুরুত্বপূর্ণ, prefix location-এ **দৈর্ঘ্য** গুরুত্বপূর্ণ। এই পার্থক্যটা না বুঝলে "আমার location কাজ করছে না কেন" প্রশ্নে অনেক সময় নষ্ট হয়। regex নিয়ে বিস্তারিত অধ্যায় ২০-এ।

root বনাম alias

দুটোই বলে ফাইল ডিস্কের কোথায় আছে, কিন্তু হিসাব আলাদা:

```
location /images/ {
    root /var/www/data;
}
# /images/cat.png  ->  /var/www/data/images/cat.png   (location অংশ যোগ হয়)

location /images/ {
    alias /var/www/pictures/;
}
# /images/cat.png  ->  /var/www/pictures/cat.png      (location অংশ বদলে যায়)
```

> ⚠️ **সতর্কতা:** `alias` ব্যবহার করলে location-এর শেষে `/` থাকলে alias-এর শেষেও `/` দাও। `location /img { alias /var/www/pictures/; }`-এর মতো অমিল থাকলে path traversal নিরাপত্তা-ছিদ্র তৈরি হতে পারে (যেমন `/img../` দিয়ে উপরের ডিরেক্টরিতে ঢোকা)।

try_files — ফাইল খোঁজার ক্রম

`try_files` বাম থেকে ডানে একেকটা অপশন চেষ্টা করে, প্রথম যেটা ডিস্কে পাওয়া যায় সেটা দেয়; শেষ অপশনটা হলো fallback (একটা URI বা `=কোড`)।

```
# সাধারণ static সাইট
try_files $uri $uri/ =404;

# React/Vue/Angular single-page app: সব অচেনা পথ index.html-এ
try_files $uri $uri/ /index.html;

# PHP ফ্রেমওয়ার্ক (Laravel, WordPress)
try_files $uri $uri/ /index.php?$query_string;
```

দরকারি আরও কিছু directive

```
server {
    ...
    client_max_body_size 20M;        # আপলোডের সর্বোচ্চ সাইজ (ডিফল্ট 1M; বেশি হলে 413 এরর)

    location = /old-page {
        return 301 /new-page;        # স্থায়ী রিডাইরেক্ট
    }

    location /admin/ {
        allow 203.0.113.5;           # শুধু এই IP ঢুকতে পারবে
        deny all;
    }

    location ~ /\. {
        deny all;                    # .git, .env জাতীয় লুকানো ফাইল লুকাও
    }

    location ~* \.(css|js|png|jpg|svg|woff2)$ {
        expires 30d;                 # ব্রাউজার ৩০ দিন ক্যাশ রাখবে
```

```
        access_log off;
    }

    error_page 404 /404.html;
}
```

nginx-এর ভ্যারিয়েবল (`$` দিয়ে শুরু) যেগুলো বারবার দেখবে:

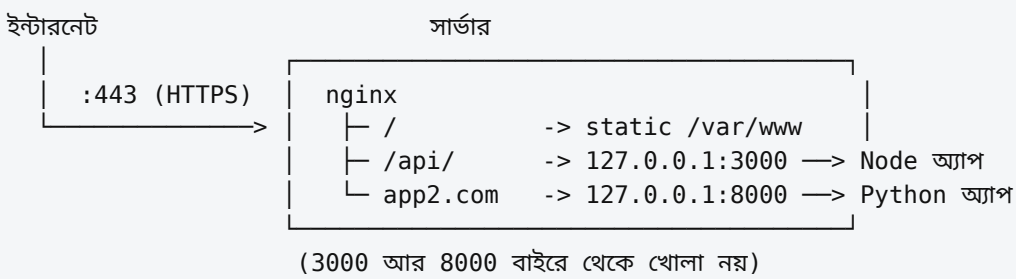
ভ্যারিয়েবল	মান
`$uri`	query string ছাড়া বর্তমান path, যেমন `/products/5`
`$request_uri`	ক্লায়েন্ট যা পাঠিয়েছে হুবহু, query সহ
`$args` / `$query_string`	`?` -এর পরের অংশ
`$host`	request-এর Host header (না থাকলে server_name)
`$remote_addr`	ক্লায়েন্টের IP
`$scheme`	`http` বা `https`
`$http_<নাম>`	যেকোনো request header, যেমন `$http_user_agent`

৪০.৪ nginx দিয়ে reverse proxy

Node.js, Python (Django/Flask/FastAPI), Go বা Java অ্যাপ সাধারণত নিজেই একটা ছোট HTTP সার্ভার চালায়, যেমন `127.0.0.1:3000` -এ। তাহলে সরাসরি পোর্ট 80-তে চালাই না কেন? কারণ সামনে nginx রাখলে:

- **TLS/HTTPS** nginx একাই সামলায়, অ্যাপকে সার্টিফিকেট নিয়ে ভাবতে হয় না।
- **static ফাইল** (ছবি, CSS) nginx অনেক দ্রুত দেয়, অ্যাপের CPU বাঁচে।
- একই সার্ভারে **একাধিক অ্যাপ** আলাদা ডোমেইন বা path-এ চালানো যায়।
- অ্যাপকে root হিসেবে চালাতে হয় না (1024-এর উপরের পোর্টে চলে)।
- ধীর ক্লায়েন্ট (খারাপ মোবাইল নেটওয়ার্ক) সামলানোর ভার nginx নেয়: সে পুরো উত্তর বাফারে নিয়ে ধীরে ধীরে পাঠায়, অ্যাপ ততক্ষণে পরের request ধরে।
- **load balancing**: একাধিক অ্যাপ instance-এ ট্রাফিক ভাগ করা।

উপমা: "proxy" মানে প্রতিনিধি। সাধারণ (forward) proxy ক্লায়েন্টের হয়ে বাইরে যায়। **reverse proxy** সার্ভারের হয়ে দরজায় দাঁড়ায়, ক্লায়েন্ট জানেই না ভেতরে আসলে কে কাজ করছে।



একটা সম্পূর্ণ reverse proxy কনফিগ

```
server {
    listen 80;
    listen [::]:80;
    server_name app.example.com;

    location / {
        proxy_pass http://127.0.0.1:3000;

        # আসল ক্লায়েন্টের তথ্য অ্যাপকে জানাও
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # WebSocket সাপোর্ট (Socket.IO ইত্যাদি)
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_read_timeout 60s;
    }
}
```

কেন এই header-গুলো? nginx-এর পেছনে থাকা অ্যাপের চোখে প্রতিটি request আসছে `127.0.0.1` থেকে (nginx থেকে)। `X-Real-IP` আর `X-Forwarded-For` না দিলে অ্যাপ আসল ব্যবহারকারীর IP জানতে পারবে না; `X-Forwarded-Proto` না দিলে অ্যাপ ভাববে request HTTP-তে এসেছে আর অনেক সময় অসীম রিডাইরেক্ট লুপ বানাবে।

> 💡 **টিপস:** WebSocket না থাকলে সব request-এ `Connection "upgrade"` পাঠানো আদর্শ নয়। প্রফেশনাল কনফিগে `http` context-এ একটা `map` বানানো হয়:
>
> ```
> map $http_upgrade $connection_upgrade {
>     default upgrade;
>     ''      close;
> }
> ```
>
> তারপর location-এ `proxy_set_header Connection $connection_upgrade;`।

proxy_pass-এর শেষ স্ল্যাশের রহস্য

এটা নতুনদের সবচেয়ে প্রিয় ফাঁদ:

```
location /api/ {
    proxy_pass http://127.0.0.1:3000;     # URI অংশ নেই
}
# /api/users  ->  অ্যাপ পায় /api/users
```

```
location /api/ {
    proxy_pass http://127.0.0.1:3000/;    # শেষে "/" (URI অংশ আছে)
}
# /api/users  ->  অ্যাপ পায় /users   (/api/ অংশ "/" দিয়ে বদলে গেল)
```

নিয়ম: `proxy_pass`-এ হোস্ট:পোর্টের পরে কোনো path (এমনকি একটা `/`) থাকলে location-এর মিলে যাওয়া অংশটা সেই path দিয়ে প্রতিস্থাপিত হয়। না থাকলে URI যেমন ছিল তেমনই যায়।

load balancing (upstream)

```
upstream app_backend {
    server 127.0.0.1:3000;
    server 127.0.0.1:3001;
    server 10.0.0.12:3000 backup;   # বাকিরা মরলে তবেই
}

server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app_backend;
        proxy_set_header Host $host;
    }
}
```

ডিফল্টে nginx পালাক্রমে (round-robin) পাঠায়। `least_conn;` লিখলে যার হাতে কম কানেকশন তাকে দেয়, `ip_hash;` লিখলে একই IP সবসময় একই সার্ভারে যায়।

> ⚠️ **সতর্কতা (RHEL/Fedora):** SELinux (অধ্যায় ৪৫) ডিফল্টে ওয়েব সার্ভারকে অন্য পোর্টে নেটওয়ার্ক কানেকশন খুলতে দেয় না। reverse proxy-তে **502 Bad Gateway** আর error.log-এ `connect() ... failed (13: Permission denied)` দেখলে চালাও:
>
> ```
> sudo setsebool -P httpd_can_network_connect 1
> ```
>
> ওয়েব রুট `/var/www`-এর বাইরে রাখলে ফাইলের SELinux লেবেলও ঠিক করতে হয় (`semanage fcontext` + `restorecon`)।

৪০.৫ Apache HTTP Server পরিচিতি

Apache HTTP Server ১৯৯৫ সাল থেকে চলছে, দীর্ঘদিন বিশ্বের এক নম্বর ওয়েব সার্ভার ছিল। এখনো শেয়ার্ড হোস্টিং, cPanel আর পুরোনো PHP প্রজেক্টে খুব প্রচলিত। প্যাকেজের নাম distro ভেদে আলাদা, এটা সবচেয়ে বড় বিভ্রান্তি:

জিনিস	Ubuntu / Debian	Fedora / RHEL
প্যাকেজ ও service	`apache2`	`httpd`
মূল কনফিগ	`/etc/apache2/apache2.conf`	`/etc/httpd/conf/httpd.conf`
সাইট কনফিগ	`/etc/apache2/sites-available/` + `sites-enabled/`	`/etc/httpd/conf.d/*.conf`
মডিউল	`/etc/apache2/mods-available/` + `mods-enabled/`	`/etc/httpd/conf.modules.d/`
পোর্ট	`/etc/apache2/ports.conf`	`httpd.conf`-এর `Listen`
লগ	`/var/log/apache2/`	`/var/log/httpd/`
কনফিগ পরীক্ষা	`sudo apache2ctl configtest`	`sudo apachectl configtest`
ইউজার	`www-data`	`apache`

```
# Ubuntu
sudo apt install apache2
sudo ufw allow 'Apache Full'

# RHEL / Fedora
sudo dnf install httpd
sudo systemctl enable --now httpd
```

> ⚠️ **সতর্কতা:** nginx আর Apache দুটোই ডিফল্টে পোর্ট 80 চায়। একসাথে ইনস্টল করলে যে পরে চালু হবে সে `Address already in use` এরর দিয়ে ব্যর্থ হবে। একটা রাখো, অথবা একটাকে অন্য পোর্টে সরাও (যেমন nginx সামনে 80-তে, Apache পেছনে 8080-এ)।

VirtualHost

nginx-এর `server` block-এর Apache সংস্করণ হলো `<VirtualHost>`। `/etc/apache2/sites-available/example.conf`:

```
<VirtualHost *:80>
    ServerName example.com
    ServerAlias www.example.com
    DocumentRoot /var/www/example

    <Directory /var/www/example>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/example_error.log
```

```
    CustomLog ${APACHE_LOG_DIR}/example_access.log combined
</VirtualHost>
```

- `Options -Indexes`: ফোল্ডারে index ফাইল না থাকলে ফাইলের তালিকা দেখাবে না।
- `AllowOverride All`: ডিরেক্টরির ভেতরে `.htaccess` ফাইল দিয়ে কনফিগ বদলানো যাবে (WordPress-এর জন্য দরকার)।
- `Require all granted`: সবাইকে ঢুকতে দাও।

Debian-এর সহায়ক কমান্ড

কমান্ড	কাজ
`sudo a2ensite example`	`sites-enabled`-এ symlink বানায় (সাইট চালু)
`sudo a2dissite 000-default`	সাইট বন্ধ
`sudo a2enmod rewrite`	মডিউল চালু (URL rewrite)
`sudo a2enmod proxy proxy_http headers ssl`	একসাথে কয়েকটা মডিউল
`sudo a2dismod autoindex`	মডিউল বন্ধ
`sudo a2enconf নাম` / `a2disconf`	`conf-available`-এর কনফিগ চালু/বন্ধ
`apache2ctl -S`	কোন VirtualHost কোন পোর্টে, তার তালিকা
`apache2ctl -M`	লোড হওয়া মডিউলের তালিকা

এরপর `sudo systemctl reload apache2`। RHEL-এ এসব helper নেই; সরাসরি `conf.d`-তে ফাইল রাখো বা মুছে ফেলো।

Apache-এর MPM — ভেতরের কাজের ধরন

Apache কীভাবে request সামলাবে তা ঠিক করে **MPM (Multi-Processing Module)**:

MPM	কৌশল	কখন
`prefork`	প্রতি কানেকশনে একটা আলাদা process, thread নেই	পুরোনো `mod_php` (যেটা thread-safe নয়)
`worker`	process-এর ভেতরে অনেক thread	মাঝামাঝি
`event`	`worker`-এর উন্নত রূপ; idle keep-alive কানেকশন আলাদা thread সামলায়	আধুনিক ডিফল্ট, PHP-FPM-এর সাথে

Ubuntu 24.04 আর RHEL 9-এ ডিফল্ট `event`। কিন্তু Ubuntu-তে `libapache2-mod-php` ইনস্টল করলে সেটা নির্ভরতা হিসেবে `prefork`-এ বদলে দেয়।

nginx বনাম Apache

বিষয়	nginx	Apache
আর্কিটেকচার	event-driven, অল্প worker	MPM (process/thread), event MPM-ও আছে
static ফাইল ও অনেক কানেকশন	খুব দ্রুত, কম মেমরি	ভালো, তবে বেশি মেমরি
PHP	সবসময় বাইরে (PHP-FPM)	`mod_php` (ভেতরে) বা PHP-FPM
`.htaccess`	নেই (সব কনফিগ কেন্দ্রে)	আছে, প্রতি ফোল্ডারে কনফিগ
কনফিগ ভাষা	সংক্ষিপ্ত block	XML-ঘেঁষা ট্যাগ
মডিউল	বেশিরভাগ কম্পাইল-টাইমে, কিছু dynamic	রানটাইমে চালু/বন্ধ
সাধারণ ব্যবহার	reverse proxy, আধুনিক স্ট্যাক	শেয়ার্ড হোস্টিং, পুরোনো অ্যাপ

`.htaccess` সুবিধাজনক কিন্তু ধীর: প্রতিটি request-এ Apache-কে পথের প্রতিটি ফোল্ডারে `.htaccess` খুঁজতে হয়। nginx এই নমনীয়তা ছেড়ে গতি নিয়েছে।

৪০.৬ HTTPS — Let's Encrypt ও certbot

HTTPS কেন, আর কীভাবে কাজ করে

HTTP-তে সব ডেটা খোলা চিঠির মতো যায়; মাঝপথে যে কেউ (একই Wi-Fi, ISP) পাসওয়ার্ড পড়তে পারে। **HTTPS** = HTTP + **TLS** (Transport Layer Security)। TLS দুটো কাজ করে:

1. **এনক্রিপশন:** ডেটা খামে ভরে সিলমোহর দেওয়া।
2. **পরিচয় যাচাই:** সার্ভার একটা **certificate** দেখায়, যেটা একটা বিশ্বস্ত **CA (Certificate Authority)** সই করেছে। ব্রাউজারে আগে থেকেই CA-দের তালিকা থাকে।

উপমা: সার্টিফিকেট হলো জাতীয় পরিচয়পত্র, CA হলো নির্বাচন কমিশন। যে কেউ বলতে পারে "আমি example.com", কিন্তু কমিশনের সই করা কার্ড দেখাতে পারে শুধু আসল মালিক।

```
ব্রাউজার                                    nginx
   |── ClientHello (সাপোর্টেড cipher) ──────────>|
   |<── ServerHello + certificate (public key) ─|
   |   ব্রাউজার যাচাই: CA-র সই ঠিক? ডোমেইন মেলে? মেয়াদ আছে?
   |── key exchange ──────────────────────────────>|
   |        দুই পক্ষ একই session key পায়         |
   |<═══════ এখন থেকে সব ডেটা এনক্রিপ্টেড ═══════>|
```

সার্ভারে দুটো ফাইল লাগে: **certificate** (পাবলিক, সবাইকে দেখানো হয়) আর **private key** (গোপন, কখনো শেয়ার করবে না)।

Let's Encrypt একটা অলাভজনক CA যারা বিনামূল্যে, স্বয়ংক্রিয়ভাবে সার্টিফিকেট দেয়। এরা **ACME** প্রোটোকলে যাচাই করে যে ডোমেইনটা সত্যিই তোমার নিয়ন্ত্রণে। সার্টিফিকেটের মেয়াদ বর্তমানে **৯০ দিন** (Let's Encrypt ভবিষ্যতে আরও কমানোর ঘোষণা দিয়েছে), তাই অটো-রিনিউ বাধ্যতামূলক। **certbot** হলো সবচেয়ে প্রচলিত ACME ক্লায়েন্ট।

পূর্বশর্ত

- ডোমেইনের DNS A (এবং IPv6 থাকলে AAAA) রেকর্ড তোমার সার্ভারের IP-তে নির্দেশ করছে (`dig +short example.com` দিয়ে যাচাই, অধ্যায় ৩৭)।
- পোর্ট 80 বাইরে থেকে খোলা (HTTP-01 যাচাই পোর্ট 80 দিয়েই হয়)।
- nginx-এ ওই ডোমেইনের `server_name` সহ server block আছে।

certbot ইনস্টল

```
# Ubuntu — certbot প্রকল্পের নিজস্ব সুপারিশ: snap
sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/bin/certbot

# অথবা distro-র প্যাকেজ (Ubuntu/Debian)
sudo apt install certbot python3-certbot-nginx     # Apache হলে python3-certbot-apache

# Fedora
sudo dnf install certbot python3-certbot-nginx
# RHEL 9: আগে EPEL রিপোজিটরি চালু করো (অধ্যায় ৩০ দেখো), তারপর একই কমান্ড
```

সার্টিফিকেট নেওয়া

```
$ sudo certbot --nginx -d example.com -d www.example.com
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address (used for urgent renewal and security notices)
 (Enter 'c' to cancel): admin@example.com
...
Requesting a certificate for example.com and www.example.com

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/example.com/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/example.com/privkey.pem
This certificate expires on 2027-02-09.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the
background.

Deploying certificate
Successfully deployed certificate for example.com to /etc/nginx/sites-enabled/example
Successfully deployed certificate for www.example.com to /etc/nginx/sites-
enabled/example
Congratulations! You have successfully enabled HTTPS on https://example.com and
https://www.example.com
```

`--nginx` plugin নিজেই তিনটা কাজ করে: যাচাই করে, সার্টিফিকেট আনে, আর তোমার server block-এ `listen 443 ssl` ও certificate-এর লাইন যোগ করে, সাথে HTTP থেকে HTTPS-এ রিডাইরেক্ট বসায়। কনফিগ শেষে দেখতে মোটামুটি এমন হয়:

```
server {
    server_name example.com www.example.com;
    root /var/www/example;
    ...
    listen [::]:443 ssl ipv6only=on; # managed by Certbot
    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem; # managed by
Certbot
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem; # managed by
Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot
}

server {
    if ($host = example.com) {
        return 301 https://$host$request_uri;
    } # managed by Certbot
    listen 80;
    server_name example.com www.example.com;
    return 404; # managed by Certbot
}
```

certbot-এর গুরুত্বপূর্ণ কমান্ড ও অপশন

কমান্ড / অপশন	কাজ
`certbot --nginx -d DOMAIN`	nginx-এর জন্য সার্টিফিকেট নেওয়া ও কনফিগ বসানো
`certbot --apache -d DOMAIN`	Apache-এর জন্য একই
`certbot certonly --nginx -d DOMAIN`	শুধু সার্টিফিকেট আনো, কনফিগে হাত দিও না
`certbot certonly --webroot -w /var/www/example -d DOMAIN`	চলমান সার্ভারের ওয়েব রুটে ফাইল রেখে যাচাই
`certbot certonly --standalone -d DOMAIN`	certbot নিজেই পোর্ট 80-তে সাময়িক সার্ভার চালায় (ওয়েব সার্ভার বন্ধ থাকতে হবে)
`certbot certonly --manual --preferred-challenges dns -d '*.example.com'`	DNS TXT রেকর্ড দিয়ে যাচাই; wildcard-এর জন্য DNS যাচাই বাধ্যতামূলক
`certbot certificates`	সব সার্টিফিকেট আর মেয়াদ দেখাও
`certbot renew`	যেগুলোর মেয়াদ শেষের কাছাকাছি সেগুলো নবায়ন
`certbot renew --dry-run`	পরীক্ষামূলক নবায়ন (staging সার্ভারে, আসল কিছু বদলায় না)
`certbot delete --cert-name DOMAIN`	সার্টিফিকেট মুছে ফেলা

কমান্ড / অপশন	কাজ
`--non-interactive --agree-tos -m EMAIL`	স্ক্রিপ্টে প্রশ্ন ছাড়া চালানো
`--redirect` / `--no-redirect`	HTTP→HTTPS রিডাইরেক্ট বসাবে কিনা
`--test-cert` (বা `--staging`)	পরীক্ষার জন্য staging CA (rate limit এড়াতে)

অটো-রিনিউ কীভাবে চলে

certbot ইনস্টলের সাথে একটা systemd timer (অধ্যায় ২৯) বা cron job আসে, যেটা দিনে দুবার `certbot renew` চালায়। মেয়াদ শেষ হওয়ার কাছাকাছি (সাধারণত ৩০ দিনের কম) থাকলে তবেই নবায়ন হয়।

```
$ systemctl list-timers | grep -i certbot
Thu 2026-11-12 03:41:00 +06  6h left  ...  snap.certbot.renew.timer
snap.certbot.renew.service
# apt/dnf প্যাকেজে timer-এর নাম certbot.timer (বা certbot-renew.timer)

$ sudo certbot renew --dry-run
...
Congratulations, all simulated renewals succeeded:
  /etc/letsencrypt/live/example.com/fullchain.pem (success)
```

`--nginx` plugin দিয়ে নেওয়া সার্টিফিকেট নবায়নের পর certbot নিজেই nginx reload করে। `certonly` / `webroot` দিয়ে নিলে reload করাতে hook লাগে:

```
sudo certbot renew --deploy-hook "systemctl reload nginx"
# স্থায়ীভাবে: /etc/letsencrypt/renewal-hooks/deploy/ ফোল্ডারে একটা executable স্ক্রিপ্ট রাখো
```

> ⚠️ **সতর্কতা:** Let's Encrypt-এর **rate limit** আছে (একই ডোমেইন-সেটের জন্য সপ্তাহে সীমিত সংখ্যক সার্টিফিকেট, বারবার ব্যর্থ যাচাইয়েরও সীমা)। শেখার সময় বা পরীক্ষায় `--test-cert` / `--dry-run` ব্যবহার করো, নাহলে কয়েক ঘণ্টা বা দিনের জন্য আটকে যেতে পারো।

> 📝 **মনে রাখো:** `/etc/letsencrypt/live/` ফোল্ডারের ফাইলগুলো আসলে `../archive/`-এর সর্বশেষ সংস্করণের symlink। কনফিগে সবসময় `live/` পথ ব্যবহার করো, তাহলে নবায়নের পর কিছু বদলাতে হবে না। `privkey.pem` শুধু root পড়তে পারে, এটা এমনই থাকা উচিত।

যাচাই:

```
$ curl -I https://example.com
HTTP/2 200
server: nginx/1.24.0 (Ubuntu)
...
$ echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null
| openssl x509 -noout -dates -issuer
notBefore=Nov 11 03:12:00 2026 GMT
```

```
notAfter=Feb  9 03:11:59 2027 GMT
issuer=C = US, O = Let's Encrypt, CN = R12
```

(issuer-এর `CN` মান Let's Encrypt-এর কোন intermediate সই করেছে তার উপর নির্ভর করে আলাদা হতে পারে।)

HTTPS চালু হলে চাইলে HSTS header যোগ করো, যাতে ব্রাউজার এরপর থেকে সবসময় HTTPS-এই আসে:

```
add_header Strict-Transport-Security "max-age=31536000" always;
```

> ⚠️ **সতর্কতা:** HSTS দেওয়ার পর HTTPS নষ্ট হলে দর্শকরা `max-age` শেষ না হওয়া পর্যন্ত HTTP-তেও ঢুকতে পারবে না। প্রথমে ছোট `max-age` (যেমন 300) দিয়ে পরীক্ষা করো।

৪০.৭ ডেটাবেস সার্ভার: MySQL / MariaDB

MySQL সবচেয়ে জনপ্রিয় ওপেন-সোর্স রিলেশনাল ডেটাবেস (এখন Oracle-এর মালিকানায়)। **MariaDB** হলো MySQL-এর মূল নির্মাতাদের তৈরি একটা fork; কমান্ড ও প্রোটোকল প্রায় একই, তাই বেশিরভাগ অ্যাপ দুটোতেই চলে। Debian 12 আর RHEL/Fedora-তে ডিফল্ট "MySQL" প্যাকেজ আসলে MariaDB; Ubuntu দুটোই দেয়।

ডেটাবেস সার্ভারের গঠন

```
অ্যাপ (PHP/Node)        mysql ক্লায়েন্ট (তুমি)
      | TCP 127.0.0.1:3306      | Unix socket /run/mysqld/mysqld.sock
      └──────────┬──────────────┘
                 v
       ┌──────────────────────────┐
       | mysqld / mariadbd        |  একটা process, অনেক thread
       |  ├─ কানেকশন ও যাচাই      |  (ইউজার@হোস্ট + পাসওয়ার্ড/প্লাগইন)
       |  ├─ SQL parser/optimizer |
       |  └─ storage engine InnoDB|
       └──────────┬───────────────┘
                  v
          /var/lib/mysql/   (ডেটা ফাইল — এটাই আসল সম্পদ)
```

ইনস্টল ও চালু

```
# Ubuntu 24.04 — MySQL 8.0
sudo apt install mysql-server           # service: mysql

# Ubuntu/Debian — MariaDB
sudo apt install mariadb-server         # service: mariadb

# Fedora / RHEL 9 — MariaDB
sudo dnf install mariadb-server
sudo systemctl enable --now mariadb
```

```
# RHEL 9 — MySQL
sudo dnf install mysql-server
sudo systemctl enable --now mysqld       # লক্ষ করো: service-এর নাম mysqld
```

```
$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
     Active: active (running) ...
$ sudo ss -tlnp | grep 3306
LISTEN 0  151  127.0.0.1:3306  0.0.0.0:*  users:(("mysqld",pid=4410,fd=23))
```

লক্ষ করো, Ubuntu-তে MySQL ডিফল্টে শুধু `127.0.0.1`-এ listen করে। মানে বাইরে থেকে কেউ ঢুকতে পারবে না। এটাই ভালো।

নিরাপদ করা: mysql_secure_installation

```
sudo mysql_secure_installation       # MySQL
sudo mariadb-secure-installation     # MariaDB (পুরোনো নাম mysql_secure_installation-ও কাজ করে)
```

এই স্ক্রিপ্ট প্রশ্ন করে করে যা করে:

প্রশ্ন	উত্তর	কারণ
VALIDATE PASSWORD component (MySQL)	ঐচ্ছিক	দুর্বল পাসওয়ার্ড আটকায়
Remove anonymous users?	Y	নাম ছাড়া লগইন বন্ধ
Disallow root login remotely?	Y	root শুধু লোকাল থেকে
Remove test database?	Y	সবার জন্য খোলা `test` ডেটাবেস মুছে ফেলো
Reload privilege tables?	Y	পরিবর্তন কার্যকর

> 📝 **মনে রাখো:** Ubuntu-র MySQL আর Debian/Ubuntu-র MariaDB-তে ডেটাবেসের root ইউজার পাসওয়ার্ড দিয়ে নয়, **unix socket authentication** দিয়ে লগইন করে: লিনাক্সের root (sudo) হলে ডেটাবেসের root-এ সরাসরি ঢোকা যায়। তাই `sudo mysql` লিখলেই হয়, পাসওয়ার্ড লাগে না। এটা নিরাপদ, বদলানোর দরকার নেই; অ্যাপের জন্য আলাদা ইউজার বানাও।

ডেটাবেস ও অ্যাপ ইউজার তৈরি

```
$ sudo mysql
mysql> CREATE DATABASE shopdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE USER 'shopapp'@'localhost' IDENTIFIED BY 'Str0ng-Pass!2026';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT ALL PRIVILEGES ON shopdb.* TO 'shopapp'@'localhost';
```

```
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW GRANTS FOR 'shopapp'@'localhost';
+---------------------------------------------------------------------+
| Grants for shopapp@localhost                                        |
+---------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `shopapp`@`localhost`                         |
| GRANT ALL PRIVILEGES ON `shopdb`.* TO `shopapp`@`localhost`         |
+---------------------------------------------------------------------+

mysql> exit
Bye
```

- `'shopapp'@'localhost'` : MySQL-এ ইউজার মানে **নাম + কোথা থেকে আসছে** জোড়া। `'shopapp'@'%'` মানে যেকোনো হোস্ট থেকে, যেটা দরকার না হলে দেবে না।
- `utf8mb4` : বাংলা আর ইমোজি সহ সব Unicode অক্ষর ঠিকভাবে রাখতে। (পুরোনো `utf8` আসলে ৩-বাইট সীমিত।)
- `GRANT` দিয়ে দেওয়া অধিকার সাথে সাথে কার্যকর হয়; `FLUSH PRIVILEGES` শুধু তখন লাগে যখন কেউ সরাসরি `mysql.user` টেবিল এডিট করে।
- **least privilege নীতি:** অ্যাপকে শুধু তার নিজের ডেটাবেসের অধিকার দাও, কখনো `*.*` নয়।

নতুন ইউজার দিয়ে পরীক্ষা:

```
$ mysql -u shopapp -p shopdb
Enter password:
mysql> SELECT CURRENT_USER(), DATABASE();
+-------------------+------------+
| CURRENT_USER()    | DATABASE() |
+-------------------+------------+
| shopapp@localhost | shopdb     |
+-------------------+------------+
```

mysql ক্লায়েন্টের দরকারি কমান্ড

কমান্ড	কাজ
`SHOW DATABASES;`	সব ডেটাবেস
`USE shopdb;`	ডেটাবেস বাছাই
`SHOW TABLES;`	টেবিলের তালিকা
`DESCRIBE orders;`	টেবিলের কলাম
`SELECT user, host FROM mysql.user;`	সব ইউজার
`SHOW PROCESSLIST;`	এখন কী query চলছে
`SELECT ... \G`	ফলাফল লম্বালম্বি (চওড়া টেবিল পড়তে সুবিধা)
`DROP USER 'x'@'localhost';`	ইউজার মুছে ফেলা
`ALTER USER 'x'@'localhost' IDENTIFIED BY 'নতুন';`	পাসওয়ার্ড বদল

কমান্ড	কাজ
`exit` / `\q` / `Ctrl+D`	বের হওয়া

শেল থেকে ব্যাকআপ ও রিস্টোর (অধ্যায় ৩৫):

```
mysqldump -u shopapp -p --single-transaction shopdb > shopdb-$(date +%F).sql
mysql -u shopapp -p shopdb < shopdb-2026-11-11.sql
```

কনফিগ ও দূর থেকে কানেকশন

	Ubuntu MySQL	Debian/Ubuntu MariaDB	RHEL/Fedora MariaDB
কনফিগ	`/etc/mysql/mysql.conf.d/mysqld.cnf`	`/etc/mysql/mariadb.conf.d/50-server.cnf`	`/etc/my.cnf.d/mariadb-server.cnf`
ডেটা	`/var/lib/mysql`	`/var/lib/mysql`	`/var/lib/mysql`
এরর লগ	`/var/log/mysql/error.log`	journald / `/var/log/mysql/`	`/var/log/mariadb/mariadb.log`

অন্য সার্ভার থেকে ঢুকতে হলে `bind-address = 127.0.0.1`-কে নির্দিষ্ট প্রাইভেট IP-তে বদলাও, `'user'@'10.0.0.%'` ধরনের ইউজার বানাও, আর ফায়ারওয়ালে শুধু সেই IP-কে 3306 খুলে দাও:

```
sudo ufw allow from 10.0.0.20 to any port 3306 proto tcp
```

> ⚠️ **সতর্কতা:** ডেটাবেস পোর্ট (3306, 5432) কখনো পুরো ইন্টারনেটে খুলবে না। ইন্টারনেটে বট সারাক্ষণ এই পোর্টগুলো স্ক্যান করে। দূর থেকে নিজে কাজ করতে চাইলে SSH tunnel ব্যবহার করো (অধ্যায় ৩৮): `ssh -L 3306:127.0.0.1:3306 deploy@server`।

৪০.৮ ডেটাবেস সার্ভার: PostgreSQL

PostgreSQL ("Postgres") আরেকটা শক্তিশালী ওপেন-সোর্স রিলেশনাল ডেটাবেস। SQL মান মেনে চলায়, জটিল query, JSON, এক্সটেনশন (যেমন PostGIS) ইত্যাদিতে এটা বিখ্যাত। Django, Rails এবং আধুনিক অনেক প্রজেক্টে এটাই প্রথম পছন্দ।

ইনস্টল

```
# Ubuntu 24.04 (PostgreSQL 16) / Debian 12 (PostgreSQL 15)
sudo apt install postgresql
# ইনস্টলের সাথেই একটা "cluster" তৈরি হয়ে চালু হয়ে যায়

# Fedora / RHEL 9
sudo dnf install postgresql-server
```

```
sudo postgresql-setup --initdb          # প্রথমবার ডেটা ডিরেক্টরি তৈরি — RHEL-এ বাধ্যতামূলক
sudo systemctl enable --now postgresql
```

```
$ sudo ss -tlnp | grep 5432
LISTEN 0  200  127.0.0.1:5432  0.0.0.0:*  users:(("postgres",pid=5120,fd=6))
```

Ubuntu/Debian-এ একাধিক সংস্করণ পাশাপাশি চালানো যায়; প্রতিটা হলো একটা **cluster**। তালিকা দেখো:

```
$ pg_lsclusters
Ver Cluster Port Status Owner    Data directory              Log file
16  main    5432 online postgres /var/lib/postgresql/16/main
/var/log/postgresql/postgresql-16-main.log
```

PostgreSQL-এর ভেতরের গঠন

MySQL-এর মতো একটা process-এ অনেক thread নয়; PostgreSQL **প্রতি কানেকশনে একটা আলাদা process** বানায়:

```
postgres (postmaster / main)  ── কানেকশন গ্রহণ করে
  ├── checkpointer, background writer, walwriter, autovacuum ... (রক্ষণাবেক্ষণ)
  ├── postgres: shopapp shopdb 127.0.0.1(51234) idle      <- ক্লায়েন্ট ১
  └── postgres: shopapp shopdb 127.0.0.1(51240) SELECT    <- ক্লায়েন্ট ২
```

`ps aux | grep postgres` চালালেই এগুলো দেখতে পাবে (অধ্যায় ২৪)। এজন্যই খুব বেশি কানেকশনের অ্যাপে PgBouncer-এর মতো connection pooler ব্যবহার হয়।

role আর লগইন

PostgreSQL-এ ইউজার আর গ্রুপ দুটোই **role**। ইনস্টলের সময় `postgres` নামে একটা লিনাক্স ইউজার আর একই নামের superuser role তৈরি হয়। ডিফল্ট লোকাল লগইন হয় **peer authentication** দিয়ে: লিনাক্স ইউজারের নাম আর ডেটাবেস role-এর নাম মিললে ঢুকতে দেয়।

```
$ sudo -u postgres psql
psql (16.4 (Ubuntu 16.4-0ubuntu0.24.04.2))
Type "help" for help.

postgres=# CREATE ROLE shopapp WITH LOGIN PASSWORD 'Str0ng-Pass!2026';
CREATE ROLE
postgres=# CREATE DATABASE shopdb OWNER shopapp;
CREATE DATABASE
postgres=# \l shopdb
                                    List of databases
  Name  |  Owner  | Encoding | ...
--------+---------+----------+----
 shopdb | shopapp | UTF8     | ...
postgres=# \q
```

শেল থেকেও একই কাজ করা যায়:

```
sudo -u postgres createuser --pwprompt shopapp
sudo -u postgres createdb -O shopapp shopdb
```

পাসওয়ার্ড দিয়ে TCP-তে পরীক্ষা (লক্ষ করো `-h 127.0.0.1`; এটা না দিলে socket আর peer auth চেষ্টা হবে):

```
$ psql -h 127.0.0.1 -U shopapp -d shopdb
Password for user shopapp:
shopdb=> SELECT current_user, version();
```

> 💡 **টিপস:** PostgreSQL 15 থেকে `public` schema-তে সাধারণ ইউজার ডিফল্টে টেবিল বানাতে পারে না। ডেটাবেসের `OWNER` অ্যাপ ইউজারকে বানালে (যেমন উপরে) এই সমস্যা হয় না।

psql-এর meta-command

কমান্ড	কাজ
`\l`	সব ডেটাবেস
`\c shopdb`	অন্য ডেটাবেসে যাও
`\dt`	টেবিলের তালিকা
`\d orders`	টেবিলের গঠন
`\du`	সব role
`\dn`	schema-র তালিকা
`\x`	লম্বালম্বি আউটপুট চালু/বন্ধ (MySQL-এর `\G`-এর মতো)
`\timing`	প্রতিটি query-র সময় দেখাও
`\e`	বর্তমান query `$EDITOR`-এ খোলো
`\i file.sql`	ফাইল থেকে SQL চালাও
`\?` / `\h`	meta-command / SQL সাহায্য
`\q` বা `Ctrl+D`	বের হও

কনফিগ: postgresql.conf ও pg_hba.conf

	Ubuntu/Debian	RHEL/Fedora
কনফিগ ফোল্ডার	`/etc/postgresql/16/main/`	`/var/lib/pgsql/data/`
ডেটা	`/var/lib/postgresql/16/main`	`/var/lib/pgsql/data`
লগ	`/var/log/postgresql/`	`/var/lib/pgsql/data/log/`

- **`postgresql.conf`**: সার্ভারের সেটিং। `listen_addresses = 'localhost'` (ডিফল্ট) মানে শুধু লোকাল। দূর থেকে চাইলে নির্দিষ্ট IP দাও।

- **`pg_hba.conf`** (host-based authentication): কে, কোন ডেটাবেসে, কোথা থেকে, কোন পদ্ধতিতে ঢুকতে পারবে। ওপর থেকে নিচে প্রথম মেলা লাইনটাই খাটে।

```
# TYPE  DATABASE  USER     ADDRESS         METHOD
local   all       postgres                 peer
local   all       all                      peer
host    all       all      127.0.0.1/32    scram-sha-256
host    all       all      ::1/128         scram-sha-256
host    shopdb    shopapp  10.0.0.20/32    scram-sha-256   # দূরের অ্যাপ সার্ভার
```

বদলানোর পর `sudo systemctl reload postgresql` (`listen_addresses` বদলালে restart লাগে)।

> ⚠️ **সতর্কতা:** `pg_hba.conf`-এ কখনো `trust` মেথড দূরের ঠিকানার জন্য দেবে না; `trust` মানে পাসওয়ার্ড ছাড়াই যে কেউ ঢুকতে পারবে।

ব্যাকআপ:

```
pg_dump -h 127.0.0.1 -U shopapp -Fc shopdb > shopdb.dump     # custom format
pg_restore -h 127.0.0.1 -U shopapp -d shopdb --clean shopdb.dump
```

MySQL বনাম PostgreSQL এক নজরে

বিষয়	MySQL / MariaDB	PostgreSQL
পোর্ট	3306	5432
অ্যাডমিন হিসেবে ঢোকা	`sudo mysql`	`sudo -u postgres psql`
ক্লায়েন্ট	`mysql` (MariaDB-তে `mariadb` ও)	`psql`
ডিফল্ট লোকাল auth	unix socket (Ubuntu/Debian)	peer
process মডেল	এক process, অনেক thread	প্রতি কানেকশনে এক process
ব্যাকআপ টুল	`mysqldump`	`pg_dump`, `pg_dumpall`
প্রচলিত সঙ্গী	PHP, WordPress, Laravel	Django, Rails, অ্যানালিটিক্স

৪০.৯ PHP-FPM — nginx-এর সাথে PHP চালানো

nginx নিজে PHP বোঝে না; সে শুধু ফাইল দিতে আর অন্যকে request পাঠাতে পারে। PHP চালায় **PHP-FPM** (FastCGI Process Manager): একটা আলাদা service, যেটা কয়েকটা PHP worker process প্রস্তুত রাখে আর **FastCGI** প্রোটোকলে nginx-এর কাছ থেকে কাজ নেয়।

```
nginx                    php-fpm (master, root)
  │  fastcgi_pass            ├── pool [www]  (ইউজার www-data)
  │  unix socket  ──────────> │     ├── worker 1  (index.php চালাচ্ছে)
```

```
  |  /run/php/php8.3-fpm.sock|     ├── worker 2  (idle)
  |                          |     └── worker 3  (idle)
  | <──── HTML output ──────  |
```

ইনস্টল

```
# Ubuntu 24.04 (PHP 8.3) — Debian 12-এ PHP 8.2, তখন নামে 8.2 বসাও
sudo apt install php-fpm php-mysql php-pgsql php-curl php-mbstring php-xml php-zip
systemctl status php8.3-fpm

# Fedora / RHEL 9
sudo dnf install php-fpm php-mysqlnd php-pgsql php-mbstring php-xml
sudo systemctl enable --now php-fpm
```

	Ubuntu/Debian	RHEL/Fedora
service	`php8.3-fpm`	`php-fpm`
pool কনফিগ	`/etc/php/8.3/fpm/pool.d/www.conf`	`/etc/php-fpm.d/www.conf`
php.ini	`/etc/php/8.3/fpm/php.ini`	`/etc/php.ini`
socket	`/run/php/php8.3-fpm.sock`	`/run/php-fpm/www.sock`
লগ	`/var/log/php8.3-fpm.log`	`/var/log/php-fpm/`

> 📝 **মনে রাখো:** RHEL/Fedora-তে PHP-FPM pool-এর ডিফল্ট ইউজার `apache`। nginx-এর সাথে চালালে `www.conf`-এ `user`/`group` আর socket-এর `listen.acl_users`-এ nginx ইউজার আছে কিনা মিলিয়ে নাও, নাহলে socket-এ permission denied (502) পাবে।

nginx-এ PHP সংযোগ (Ubuntu)

```
server {
    listen 80;
    server_name blog.example.com;
    root /var/www/blog;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;              # SCRIPT_FILENAME ইত্যাদি সেট করে
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

পরীক্ষা: `/var/www/blog/info.php` ফাইলে `<?php phpinfo();` লিখে ব্রাউজারে খোলো। কাজ করলে **সাথে সাথে ফাইলটা মুছে ফেলো**, কারণ এটা সার্ভারের অনেক গোপন তথ্য দেখায়।

pool টিউনিং

`www.conf`-এর গুরুত্বপূর্ণ লাইন:

```
pm = dynamic              ; static | dynamic | ondemand
pm.max_children = 5       ; একসাথে সর্বোচ্চ কতগুলো PHP worker
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
```

`pm.max_children` আন্দাজ করার সূত্র: (PHP-এর জন্য বরাদ্দ RAM) ÷ (একটা worker গড়ে কত MB খায়)। খুব বেশি দিলে RAM শেষ হয়ে swap/OOM, খুব কম দিলে request লাইনে দাঁড়িয়ে থাকে আর লগে `server reached pm.max_children setting` দেখায়।

বদলানোর পর:

```
sudo php-fpm8.3 -t            # কনফিগ পরীক্ষা (RHEL: sudo php-fpm -t)
sudo systemctl reload php8.3-fpm
```

একাধিক সাইট থাকলে প্রতিটি সাইটের জন্য আলাদা pool (আলাদা ইউজার ও socket) বানানো নিরাপদ: এক সাইট হ্যাক হলে অন্যটার ফাইল ছুঁতে পারবে না।

৪০.১০ Node.js অ্যাপকে systemd দিয়ে চালানো

টার্মিনালে `node server.js` চালিয়ে SSH বন্ধ করলে অ্যাপও মরে যায় (কারণ জানতে অধ্যায় ২৫)। `nohup` বা `tmux` কাজ চালানোর উপায়, কিন্তু প্রোডাকশনে ঠিক উপায় হলো **systemd service**: বুটে নিজে চালু, crash হলে নিজে রিস্টার্ট, লগ journald-এ জমা। unit ফাইলের পূর্ণ ব্যাখ্যা অধ্যায় ২৬-এ আছে; এখানে বাস্তব রেসিপি।

ধাপ ১: অ্যাপের জন্য আলাদা ইউজার

```
sudo useradd --system --create-home --home-dir /srv/myapp --shell /usr/sbin/nologin
myapp
```

`--system` আর `nologin` মানে এই ইউজার দিয়ে কেউ লগইন করতে পারবে না; শুধু অ্যাপ চালানোর জন্য।

ধাপ ২: Node ও অ্যাপ

```
# Ubuntu-র রিপোজিটরির nodejs পুরোনো হতে পারে; প্রোডাকশনে প্রায়ই NodeSource বা nvm থেকে LTS নেওয়া
হয়
```

```
sudo apt install nodejs npm
node --version
```

`/srv/myapp/server.js` (ছোট উদাহরণ):

```
const http = require('http');
const port = process.env.PORT || 3000;
http.createServer((req, res) => {
  res.end('Hello from Node, ip=' + (req.headers['x-real-ip'] || 'unknown') + '\n');
}).listen(port, '127.0.0.1', () => console.log(`listening on ${port}`));
```

লক্ষ করো `'127.0.0.1'`: অ্যাপ শুধু লোকালে শোনে, বাইরে থেকে পৌঁছানো যায় শুধু nginx দিয়ে।

ধাপ ৩: environment ফাইল (গোপন তথ্য)

`/etc/myapp.env`:

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgres://shopapp:Str0ng-Pass!2026@127.0.0.1:5432/shopdb
```

```
sudo chown root:myapp /etc/myapp.env
sudo chmod 640 /etc/myapp.env      # অন্যরা পাসওয়ার্ড পড়তে পারবে না
```

ধাপ ৪: unit ফাইল

`/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My Node.js app
After=network.target
# ডেটাবেস একই সার্ভারে থাকলে: After=network.target postgresql.service

[Service]
Type=simple
User=myapp
Group=myapp
WorkingDirectory=/srv/myapp
EnvironmentFile=/etc/myapp.env
ExecStart=/usr/bin/node /srv/myapp/server.js
Restart=on-failure
RestartSec=5

# অতিরিক্ত নিরাপত্তা (ঐচ্ছিক কিন্তু ভালো)
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full

[Install]
WantedBy=multi-user.target
```

লাইন	মানে
`After=network.target`	নেটওয়ার্ক প্রস্তুত হওয়ার পরে শুরু
`User=` / `Group=`	কোন ইউজার হিসেবে চলবে (root নয়!)
`WorkingDirectory=`	অ্যাপের কাজের ফোল্ডার
`EnvironmentFile=`	ফাইল থেকে environment ভ্যারিয়েবল
`ExecStart=`	পুরো পথসহ চালানোর কমান্ড
`Restart=on-failure`	crash (non-zero exit) হলে আবার চালু
`RestartSec=5`	রিস্টার্টের আগে ৫ সেকেন্ড অপেক্ষা
`NoNewPrivileges=true`	process নিজের ক্ষমতা বাড়াতে পারবে না
`ProtectSystem=full`	`/usr`, `/boot`, `/etc` read-only
`WantedBy=multi-user.target`	`enable` করলে বুটে চালু হবে

> ⚠️ **সতর্কতা:** `ExecStart`-এ সবসময় **পূর্ণ পথ** দাও (`which node` দিয়ে দেখে নাও)। nvm দিয়ে ইনস্টল করা node ইউজারের home-এ থাকে (`~/.nvm/versions/...`), systemd সেটা নিজে খুঁজে পায় না।

ধাপ ৫: চালু করো

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now myapp
Created symlink /etc/systemd/system/multi-user.target.wants/myapp.service →
/etc/systemd/system/myapp.service.
$ systemctl status myapp
● myapp.service - My Node.js app
     Loaded: loaded (/etc/systemd/system/myapp.service; enabled; preset: enabled)
     Active: active (running) since ...
   Main PID: 6021 (node)
$ curl http://127.0.0.1:3000
Hello from Node, ip=unknown
```

এখন ৪০.৪-এর reverse proxy কনফিগ দিয়ে nginx-এর পেছনে বসাও, তারপর certbot দিয়ে HTTPS। ব্যস, প্রোডাকশন-মানের ডিপ্লয়মেন্ট।

কোড আপডেটের পর:

```
sudo systemctl restart myapp
journalctl -u myapp -f        # লাইভ লগ দেখো
```

> 💡 **টিপস:** একই পদ্ধতি Python (`ExecStart=/srv/app/venv/bin/gunicorn -b 127.0.0.1:8000 app:app`), Go বা Java অ্যাপের জন্যও খাটে, শুধু `ExecStart` বদলাবে। Node

> জগতে PM2 নামে একটা জনপ্রিয় process manager আছে; সেটাও শেষ পর্যন্ত systemd দিয়েই বুটে চালু করা হয় (`pm2 startup`)। একটা সার্ভারে একটা-দুটো অ্যাপ হলে সরাসরি systemd সবচেয়ে সরল।

৪০.১১ লগ দেখা — সমস্যার প্রথম সূত্র

লগ সম্পর্কে সাধারণ জ্ঞান আর `journalctl`-এর পুরো ব্যাখ্যা অধ্যায় ২৮-এ। এখানে সার্ভিস-নির্দিষ্ট লগের মানচিত্র:

সার্ভিস	লগ ফাইল	journald
nginx	`/var/log/nginx/access.log`, `error.log`	`journalctl -u nginx` (শুধু start/stop বার্তা)
Apache (Debian)	`/var/log/apache2/access.log`, `error.log`	`journalctl -u apache2`
Apache (RHEL)	`/var/log/httpd/access_log`, `error_log`	`journalctl -u httpd`
MySQL (Ubuntu)	`/var/log/mysql/error.log`	`journalctl -u mysql`
MariaDB	distro-ভেদে ফাইল বা journald	`journalctl -u mariadb`
PostgreSQL	`/var/log/postgresql/postgresql-16-main.log` (Ubuntu)	`journalctl -u postgresql@16-main`
PHP-FPM	`/var/log/php8.3-fpm.log`	`journalctl -u php8.3-fpm`
তোমার Node অ্যাপ	(stdout/stderr)	`journalctl -u myapp`
certbot	`/var/log/letsencrypt/letsencrypt.log`	—

access log পড়া

```
$ sudo tail -n 2 /var/log/nginx/access.log
203.0.113.77 - - [11/Nov/2026:10:31:02 +0600] "GET /products HTTP/1.1" 200 5123
"https://google.com/" "Mozilla/5.0 ..."
198.51.100.9 - - [11/Nov/2026:10:31:05 +0600] "GET /.env HTTP/1.1" 404 162 "-"
"curl/8.5.0"
```

এটা **combined** ফরম্যাট:

```
203.0.113.77  -  -  [সময়]  "GET /products HTTP/1.1"  200  5123  "referer"  "user-agent"
   |                          |                        |     |
 ক্লায়েন্ট IP                method path protocol    status bytes
```

দ্বিতীয় লাইনটা খেয়াল করো: কেউ `.env` ফাইল খুঁজছে। এটা প্রতিটি পাবলিক সার্ভারে প্রতিদিন হয়; তাই লুকানো ফাইলে `deny all` দেওয়া জরুরি।

status code মনে রাখার সহজ নিয়ম: **2xx** সফল, **3xx** রিডাইরেক্ট, **4xx** ক্লায়েন্টের ভুল (404 নেই, 403 নিষেধ, 413 ফাইল বড়), **5xx** সার্ভারের ভুল (500 অ্যাপ ভেঙেছে, 502 পেছনের অ্যাপ সাড়া দেয়নি, 504 সময় শেষ)।

বাস্তব লগ-বিশ্লেষণ এক-লাইনার

অধ্যায় ২২ আর অধ্যায় ২৩-এর টুল এখানে কাজে আসে:

```
# লাইভ দেখো
sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log

# সবচেয়ে বেশি request পাঠানো ১০টা IP
sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head

# status code অনুযায়ী গণনা
sudo awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn

# শুধু 5xx এরর
sudo awk '$9 ~ /^5/' /var/log/nginx/access.log | tail

# সবচেয়ে বেশি চাওয়া path
sudo awk '{print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head

# আজকের অ্যাপ লগে "error" শব্দ
journalctl -u myapp --since today | grep -i error
```

পুরোনো লগ `logrotate` প্রতিদিন/সপ্তাহে ঘুরিয়ে `.1`, `.2.gz` নামে রাখে; gzip করা লগ পড়তে `zcat` / `zgrep` / `zless` ব্যবহার করো।

সাধারণ সমস্যা ও সমাধানের মানচিত্র

লক্ষণ	সম্ভাব্য কারণ	কোথায় দেখবে / কী করবে
ব্রাউজারে কানেক্টই হয় না	service বন্ধ, ফায়ারওয়াল, DNS ভুল	`systemctl status nginx`, `ss -tlnp`, `ufw status`, `dig`
nginx চালু হয় না	কনফিগ ভুল বা পোর্ট দখল	`nginx -t`, `journalctl -u nginx`, `ss -tlnp 'sport = :80'`
403 Forbidden	ফাইল/ফোল্ডারে পারমিশন নেই, index ফাইল নেই, SELinux	error.log; `namei -l /var/www/site/index.html`
404 Not Found	`root` ভুল, `try_files` ভুল	error.log-এ nginx কোন পথ খুঁজেছে দেখো
502 Bad Gateway	পেছনের অ্যাপ/PHP-FPM বন্ধ, ভুল পোর্ট/socket, SELinux	`systemctl status myapp`, `curl 127.0.0.1:3000`, error.log
504 Gateway Timeout	অ্যাপ খুব ধীর (প্রায়ই ধীর DB query)	অ্যাপ লগ, `proxy_read_timeout`

লক্ষণ	সম্ভাব্য কারণ	কোথায় দেখবে / কী করবে
413 Request Entity Too Large	আপলোড বড়	`client_max_body_size` ; PHP-তে `upload_max_filesize` , `post_max_size`
HTTPS সার্টিফিকেট মেয়াদোত্তীর্ণ	renew ব্যর্থ	`certbot renew --dry-run` , `/var/log/letsencrypt/`
অ্যাপ DB-তে ঢুকতে পারে না	ভুল পাসওয়ার্ড, ভুল host, `pg_hba.conf`	DB লগ, হাতে `mysql` / `psql` দিয়ে একই তথ্যে চেষ্টা

`namei -l` একটা চমৎকার কমান্ড: পথের প্রতিটি ধাপের পারমিশন দেখায়, যাতে বোঝা যায় কোন ফোল্ডারে worker ইউজার আটকে যাচ্ছে (অধ্যায় ১৬)।

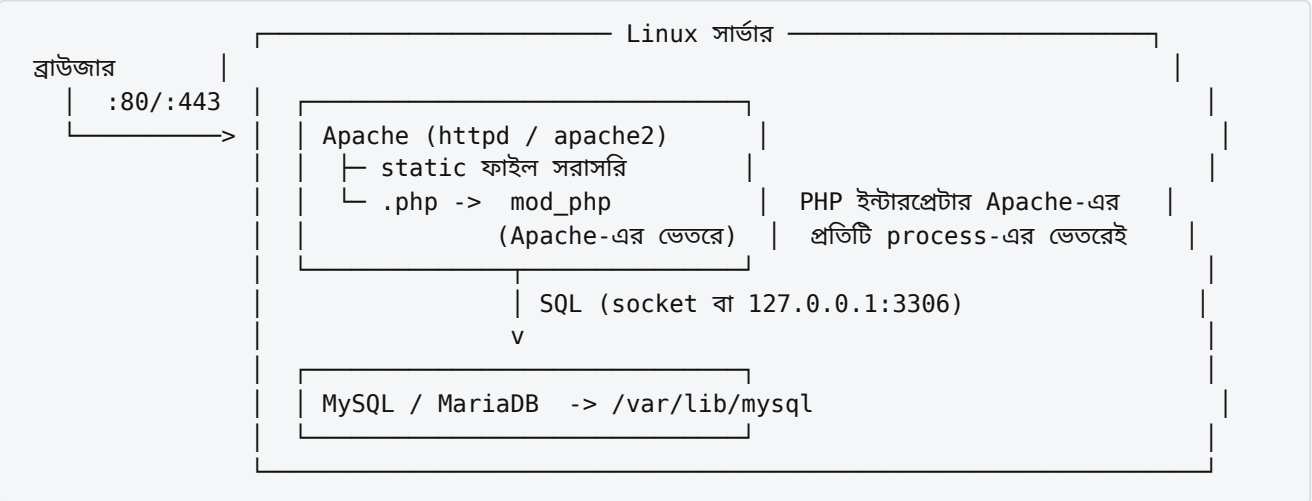
```
$ namei -l /var/www/example/index.html
f: /var/www/example/index.html
drwxr-xr-x root   root     /
drwxr-xr-x root   root     var
drwxr-xr-x root   root     www
drwxr-x--- deploy deploy   example      <- www-data এখানে আটকে যাবে!
-rw-r--r-- deploy deploy   index.html
```

৪০.১২ LAMP ও LEMP স্ট্যাক

"স্ট্যাক" মানে একসাথে কাজ করা সফটওয়্যারের সেট। নামগুলো আদ্যক্ষর:

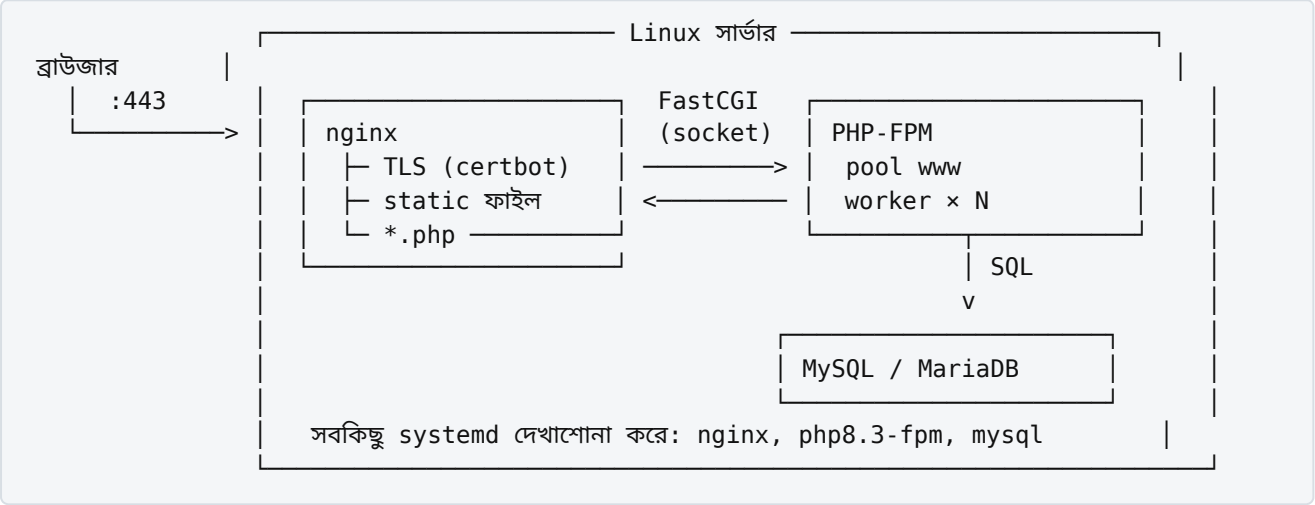
- **LAMP** = **L**inux + **A**pache + **M**ySQL/MariaDB + **P**HP (কখনো Perl/Python)
- **LEMP** = **L**inux + **E**nginx ("engine-x"-এর উচ্চারণ থেকে E) + **M**ySQL/MariaDB + **P**HP

LAMP — mod_php সহ



(আধুনিক LAMP-এ প্রায়ই mod_php-এর বদলে Apache-ও `proxy_fcgi` দিয়ে PHP-FPM ব্যবহার করে, event MPM-এর সাথে।)

LEMP — PHP-FPM সহ



অন্য জনপ্রিয় স্ট্যাক

স্ট্যাক	উপাদান	উদাহরণ
LAMP	Linux, Apache, MySQL, PHP	পুরোনো WordPress, cPanel হোস্টিং
LEMP	Linux, nginx, MySQL, PHP	আধুনিক WordPress, Laravel
MERN/MEAN	MongoDB, Express, React/Angular, Node	JavaScript-সর্বস্ব অ্যাপ
Node + Postgres	nginx → Node (systemd) → PostgreSQL	API সার্ভার
Django	nginx → gunicorn (systemd) → PostgreSQL	Python ওয়েব অ্যাপ

একটা LEMP স্ট্যাক এক নিঃশ্বাসে (Ubuntu 24.04)

```
sudo apt update
sudo apt install nginx mysql-server php-fpm php-mysql
sudo mysql_secure_installation
sudo ufw allow 'Nginx Full'
# তারপর: ডেটাবেস + ইউজার (৪০.৭), server block + PHP location (৪০.৯), certbot (৪০.৬)
```

> 💡 **টিপস:** এখনকার অনেক টিম এই একই স্ট্যাক কন্টেইনারে চালায় (nginx, php-fpm, mysql আলাদা container), যা অধ্যায় ৪৮-এ দেখবে। কিন্তু container-এর ভেতরেও ঠিক এই কনফিগ আর ধারণাই কাজ করে, তাই এখানে যা শিখলে তা সরাসরি কাজে লাগবে।

৪০.১৩ সংশ্লিষ্ট কীবোর্ড শর্টকাট

সার্ভার সেটআপে সবচেয়ে বেশি সময় যায় লগ দেখা, ডেটাবেস শেল আর কনফিগ এডিটে। সেগুলোর শর্টকাট:

`less` / `journalctl` (লগ পড়ার সময়)

কী	কাজ
`Space` / `b`	এক পাতা নিচে / উপরে
`G` / `g`	শেষে / শুরুতে
`/শব্দ` তারপর `n` / `N`	খোঁজা, পরের / আগের মিল
`F`	`tail -f`-এর মতো লাইভ অনুসরণ; থামাতে `Ctrl+C`
`q`	বের হওয়া

`tail -f`, চলমান কমান্ড

কী	কাজ
`Ctrl+C`	`tail -f` / `journalctl -f` থামানো
`Ctrl+S` / `Ctrl+Q`	টার্মিনাল আউটপুট সাময়িক থামানো / আবার চালু (দ্রুত চলা লগ ধরতে)

`mysql` ও `psql` শেল

কী	কাজ
`Ctrl+D`	শেল থেকে বের হওয়া (মুহূর্তে)
`Ctrl+C`	mysql: চলমান query বাতিল (৫.৭+/৮); psql: বর্তমান query বাতিল
`↑` / `↓`	আগের কমান্ড
`Ctrl+R`	হিস্ট্রিতে খোঁজা (readline সাপোর্ট থাকলে)
`Tab`	টেবিল/কলামের নাম auto-complete
`\c` (mysql)	অর্ধেক লেখা statement বাতিল
`\G` (mysql) / `\x` (psql)	লম্বালম্বি আউটপুট

কনফিগ এডিট — `nano`-র শর্টকাট অধ্যায় ১৩-এ আর `vim`-এর অধ্যায় ১৪-এ। দ্রুত মনে রাখার জন্য: nano-তে `Ctrl+O` সেভ, `Ctrl+X` বের, `Ctrl+W` খোঁজা; vim-এ `:w` সেভ, `:q` বের, `/` খোঁজা।

SSH সেশন (সার্ভারে কাজের সময়): ঝুলে যাওয়া SSH সেশন থেকে বের হতে নতুন লাইনে `~.` চাপো (অধ্যায় ৩৮)।

হাতে-কলমে অনুশীলন

একটা পরীক্ষামূলক Ubuntu 24.04 মেশিন নাও (VM, অধ্যায় ৩, বা সস্তা VPS)। ডোমেইন না থাকলে ল্যাব ১–৪ আর ৬–৭ লোকালি করা যায়; ল্যাব ৫-এর জন্য আসল ডোমেইন লাগবে।

ল্যাব ১: nginx static সাইট

1. `sudo apt install nginx` এবং `curl -I http://localhost` দিয়ে `200 OK` দেখো।
2. `/var/www/lab` বানিয়ে ভেতরে `index.html` রাখো।
3. `/etc/nginx/sites-available/lab` ফাইলে `server_name lab.local;` আর `root /var/www/lab;` দিয়ে server block লেখো।
4. symlink দাও, `sudo nginx -t`, `sudo systemctl reload nginx`।
5. `curl -H 'Host: lab.local' http://localhost` চালাও; নিজের HTML দেখা উচিত।
6. ইচ্ছে করে কনফিগে একটা `;` মুছে `nginx -t` চালাও, এরর বার্তা আর লাইন নম্বর পড়ো। ঠিক করো।

ল্যাব ২: location ও try_files পরীক্ষা

1. `/var/www/lab/docs/a.html` বানাও।
2. `location = /hello { return 200 "exact\n"; }` আর `location /hello { return 200 "prefix\n"; }` যোগ করো।
3. `curl -H 'Host: lab.local' localhost/hello` আর `.../hello/world` চালিয়ে কোনটা কোন উত্তর দেয় ব্যাখ্যা করো।
4. `location /` -এ `try_files $uri $uri/ /index.html;` দাও; `.../no-such-page` চাইলে কী আসে দেখো।

ল্যাব ৩: Node অ্যাপ systemd + reverse proxy

1. `myapp` system ইউজার বানাও, ৪০.১০-এর `server.js` রাখো।
2. unit ফাইল লেখো, `daemon-reload`, `enable --now`।
3. `sudo systemctl kill -s SIGKILL myapp` দিয়ে অ্যাপ "crash" করাও; ৫ সেকেন্ড পর `systemctl status myapp` দেখো, নতুন PID নিয়ে আবার চালু হয়েছে কিনা।
4. nginx-এ `app.local`-এর জন্য reverse proxy server block বানাও।
5. `curl -H 'Host: app.local' localhost` চালাও; আউটপুটে `ip=127.0.0.1` (X-Real-IP থেকে) দেখবে।
6. `sudo systemctl stop myapp` করে আবার curl করো: **502** পাবে। `/var/log/nginx/error.log`-এ `connect() failed (111: Connection refused)` খুঁজে বের করো। তারপর অ্যাপ আবার চালু করো।

ল্যাব ৪: ডেটাবেস

1. `sudo apt install mysql-server`, `sudo mysql_secure_installation`।
2. `labdb` ডেটাবেস আর `labuser@localhost` ইউজার বানাও, শুধু `labdb.*`-এ অধিকার দাও।
3. `mysql -u labuser -p labdb`-এ ঢুকে একটা টেবিল বানাও, দুটো row ঢোকাও।
4. `mysqldump` দিয়ে ব্যাকআপ নাও, টেবিল `DROP` করো, ব্যাকআপ থেকে ফেরত আনো।

5. একই কাজ PostgreSQL-এ করো: `sudo apt install postgresql`, `sudo -u postgres createuser --pwprompt labuser`, `createdb -O labuser labdb`, `psql -h 127.0.0.1 -U labuser labdb`।

ল্যাব ৫: HTTPS (আসল ডোমেইন লাগবে)

1. DNS-এ A রেকর্ড দাও, `dig +short` দিয়ে যাচাই।
2. `sudo certbot --nginx --test-cert -d তোমার.ডোমেইন` (staging) দিয়ে প্রথমে পরীক্ষা।
3. সফল হলে `--test-cert` ছাড়া আবার চালাও (প্রয়োজনে `--force-renewal`)।
4. `sudo certbot certificates` আর `sudo certbot renew --dry-run`।
5. `curl -I http://তোমার.ডোমেইন` দিয়ে 301 রিডাইরেক্ট দেখো।

ল্যাব ৬: LEMP-এ PHP

1. `sudo apt install php-fpm php-mysql`।
2. `lab` server block-এ ৪০.৯-এর `location ~ \.php$` যোগ করো।
3. `/var/www/lab/db.php` ফাইলে PDO দিয়ে `labdb`-তে কানেক্ট করে row গুলো প্রিন্ট করো।
4. `curl -H 'Host: lab.local' localhost/db.php` চালাও।
5. `sudo systemctl stop php8.3-fpm` করে 502 দেখো, error.log-এ socket-এর এরর পড়ো, আবার চালু করো।

ল্যাব ৭: লগ গোয়েন্দাগিরি

1. একটা লুপ দিয়ে কিছু ট্রাফিক বানাও: `for i in $(seq 50); do curl -s -o /dev/null -H 'Host: lab.local' localhost/; curl -s -o /dev/null -H 'Host: lab.local' localhost/nope; done`
2. ৪০.১১-এর awk কমান্ড দিয়ে status code-এর গণনা বের করো (200 আর 404 দুটোই ৫০ হওয়া উচিত)।
3. `journalctl -u myapp --since "10 min ago"` দিয়ে অ্যাপের রিস্টার্টের বার্তা খুঁজে বের করো।

সারসংক্ষেপ

কাজ	কমান্ড
nginx ইনস্টল	`sudo apt install nginx` / `sudo dnf install nginx`
nginx কনফিগ পরীক্ষা	`sudo nginx -t`
পুরো কনফিগ (include সহ) দেখা	`sudo nginx -T`
নতুন কনফিগ লোড	`sudo systemctl reload nginx`
সাইট চালু (Debian)	`sudo ln -s /etc/nginx/sites-available/X /etc/nginx/sites-enabled/`

কাজ	কমান্ড
Apache সাইট/মডিউল চালু	`sudo a2ensite X`, `sudo a2enmod rewrite`
Apache কনফিগ পরীক্ষা	`sudo apache2ctl configtest` / `sudo apachectl configtest`
HTTPS সার্টিফিকেট	`sudo certbot --nginx -d example.com`
সার্টিফিকেটের তালিকা	`sudo certbot certificates`
রিনিউ পরীক্ষা	`sudo certbot renew --dry-run`
MySQL/MariaDB ইনস্টল	`sudo apt install mysql-server` / `mariadb-server`
MySQL নিরাপদ করা	`sudo mysql_secure_installation`
MySQL অ্যাডমিন শেল	`sudo mysql`
MySQL ব্যাকআপ	`mysqldump -u U -p --single-transaction DB > f.sql`
PostgreSQL ইনস্টল	`sudo apt install postgresql` / `dnf install postgresql-server` + `postgresql-setup --initdb`
PostgreSQL অ্যাডমিন শেল	`sudo -u postgres psql`
Postgres ইউজার/ডেটাবেস	`createuser --pwprompt U`, `createdb -O U DB`
Postgres ব্যাকআপ	`pg_dump -Fc DB > f.dump`
PHP-FPM কনফিগ পরীক্ষা	`sudo php-fpm8.3 -t`
অ্যাপ service চালু	`sudo systemctl daemon-reload && sudo systemctl enable --now myapp`
অ্যাপের লাইভ লগ	`journalctl -u myapp -f`
ওয়েব লগ লাইভ	`sudo tail -f /var/log/nginx/access.log`
কে কোন পোর্টে	`sudo ss -tlnp`
HTTP header পরীক্ষা	`curl -I https://example.com`
নির্দিষ্ট Host পরীক্ষা	`curl -H 'Host: site' http://localhost`
পথের পারমিশন	`namei -l /path/to/file`
SELinux proxy অনুমতি (RHEL)	`sudo setsebool -P httpd_can_network_connect 1`

শর্টকাট	কাজ
`Ctrl+C`	`tail -f` / `journalctl -f` থামানো, psql/mysql-এ query বাতিল
`Ctrl+D`	mysql/psql থেকে বের হওয়া
`F` (less-এ)	লাইভ অনুসরণ
`/শব্দ`, `n`	less/journalctl-এ খোঁজা

শর্টকাট	কাজ
`\G` / `\x`	mysql / psql-এ লম্বালম্বি আউটপুট

নিজেকে যাচাই করো

1. nginx-এর master আর worker process-এর কাজ কী? worker কেন root হিসেবে চলে না?
2. কনফিগ বদলানোর পর `restart`-এর বদলে `nginx -t` আর `reload` কেন ব্যবহার করা উচিত?
3. `location /api/ { proxy_pass http://127.0.0.1:3000/; }` থাকলে `/api/users` request অ্যাপের কাছে কোন path হিসেবে পৌঁছাবে?
4. `root` আর `alias`-এর পার্থক্য উদাহরণ দিয়ে বলো।
5. reverse proxy-তে `X-Forwarded-Proto` header না দিলে কী সমস্যা হতে পারে?
6. Let's Encrypt সার্টিফিকেটের মেয়াদ কত, আর নবায়ন কে করে? নবায়ন ঠিকমতো চলবে কিনা কীভাবে পরীক্ষা করবে?
7. Ubuntu-তে MySQL ইনস্টলের পর `sudo mysql` লিখলে পাসওয়ার্ড ছাড়াই root হিসেবে ঢোকা যায় কেন?
8. PostgreSQL-এ `pg_hba.conf`-এর কাজ কী? `peer` আর `scram-sha-256` মেথডের পার্থক্য কী?
9. nginx `502 Bad Gateway` দিচ্ছে। কোন তিনটা জিনিস প্রথমে পরীক্ষা করবে?
10. LAMP আর LEMP স্ট্যাকে PHP চালানোর পদ্ধতিতে মূল পার্থক্য কী?

▼ উত্তর দেখো

1. **master** root হিসেবে চলে, কনফিগ পড়ে, 80/443-এর মতো privileged পোর্ট খোলে, worker তৈরি করে এবং reload/stop signal সামলায়। **worker** আসল request সামলায় event loop দিয়ে। worker কম ক্ষমতার ইউজার (`www-data` / `nginx`) হিসেবে চলে যাতে কোনো worker হ্যাক হলেও আক্রমণকারী root ক্ষমতা না পায় (least privilege)।

2. `nginx -t` আগে সিনট্যাক্স ও ফাইল পরীক্ষা করে, ফলে ভুল ধরা পড়ে সার্ভিস বন্ধ হওয়ার আগেই। `restart` পুরো process বন্ধ করে আবার চালায়: চলমান কানেকশন কাটে, আর কনফিগ ভুল হলে nginx আর উঠবেই না। `reload` নতুন worker দিয়ে ধীরে ধীরে বদলায়, কানেকশন কাটে না, আর ভুল কনফিগ পেলে পুরোনোটাতেই চলতে থাকে।

3. `/users`। `proxy_pass`-এ হোস্ট:পোর্টের পরে `/` (URI অংশ) আছে, তাই location-এর মিলে যাওয়া অংশ `/api/` সেটা দিয়ে প্রতিস্থাপিত হয়। শেষের `/` না থাকলে `/api/users`-ই যেত।

4. `root` হলে location-এর path যোগ হয়: `location /img/ { root /var/www/data; }` → `/img/a.png` = `/var/www/data/img/a.png`। `alias` হলে location-এর অংশ বদলে যায়: `location /img/ { alias /var/www/pics/; }` → `/img/a.png` = `/var/www/pics/a.png`।

5. অ্যাপ বুঝতে পারবে না ক্লায়েন্ট আসলে HTTPS দিয়ে এসেছে (nginx থেকে অ্যাপে কানেকশন সাধারণ HTTP)। ফলে অ্যাপ "HTTPS-এ যাও" রিডাইরেক্ট বারবার দিয়ে অসীম রিডাইরেক্ট লুপ তৈরি করতে পারে, অথবা ভুল `http://` লিংক বানাতে পারে, বা secure cookie ঠিকমতো সেট না-ও করতে পারে।

6. বর্তমানে ৯০ দিন। certbot ইনস্টলের সাথে আসা systemd timer (`snap.certbot.renew.timer` বা `certbot.timer`) বা cron দিনে দুবার `certbot renew` চালায় এবং মেয়াদ শেষের কাছাকাছি হলে নবায়ন করে। পরীক্ষা: `sudo certbot renew --dry-run` এবং `systemctl list-timers | grep certbot`।

7. Ubuntu-র MySQL-এ (এবং Debian/Ubuntu-র MariaDB-তে) ডেটাবেসের root ইউজার **unix socket authentication** ব্যবহার করে: কানেক্ট করা লিনাক্স ইউজার root হলে MySQL তাকে বিশ্বাস করে। `sudo` দিয়ে তুমি লিনাক্সের root, তাই পাসওয়ার্ড লাগে না। বাইরের কেউ এভাবে ঢুকতে পারে না, কারণ socket শুধু লোকাল।

8. `pg_hba.conf` ঠিক করে কোন ধরনের কানেকশন (local socket/host TCP), কোন ডেটাবেসে, কোন ইউজার, কোন ঠিকানা থেকে, কোন পদ্ধতিতে যাচাই হবে; ওপর থেকে প্রথম মেলা লাইন খাটে। `peer` : লিনাক্স ইউজারের নাম আর DB role-এর নাম মিললেই ঢুকতে দেয় (শুধু লোকাল socket-এ)। `scram-sha-256` : নিরাপদ challenge-response পদ্ধতিতে পাসওয়ার্ড যাচাই করে।

9. (ক) পেছনের অ্যাপ/PHP-FPM চলছে কিনা: `systemctl status myapp` বা `php8.3-fpm`। (খ) nginx যে ঠিকানায় পাঠাচ্ছে সেখানে সত্যিই কেউ শুনছে কিনা: `ss -tlnp` , `curl 127.0.0.1:3000` , socket-এর পথ মিলছে কিনা। (গ) `/var/log/nginx/error.log` পড়া: `Connection refused` , `No such file` (socket), বা `Permission denied` (socket পারমিশন বা RHEL-এ SELinux, তখন `httpd_can_network_connect`)।

10. LAMP-এ ঐতিহ্যগতভাবে PHP চলে Apache-এর **ভেতরে** `mod_php` মডিউল হিসেবে (প্রতিটি Apache process-এ PHP লোড থাকে, prefork MPM লাগে)। LEMP-এ nginx PHP চালাতে পারে না; PHP চলে **আলাদা service** PHP-FPM-এ, আর nginx FastCGI দিয়ে (সাধারণত unix socket) তাকে request পাঠায়। আধুনিক Apache-ও চাইলে PHP-FPM ব্যবহার করতে পারে।

অধ্যায় ৪১: শেল স্ক্রিপ্টিং — পর্ব ১ (ভিত্তি)

এতক্ষণ তুমি টার্মিনালে একটা একটা করে কমান্ড টাইপ করে কাজ করেছ। কিন্তু ধরো প্রতিদিন সকালে তোমাকে একই ১৫টা কমান্ড চালাতে হয়: লগ পরিষ্কার করা, ব্যাকআপ নেওয়া, ডিস্ক চেক করা। প্রতিদিন হাতে টাইপ করা ক্লান্তিকর, আর ভুল হওয়ার সম্ভাবনাও থাকে। এর সমাধান হলো **শেল স্ক্রিপ্ট**: কমান্ডগুলো একটা ফাইলে লিখে রাখো, তারপর একটা কমান্ডেই পুরোটা চালাও।

লিনাক্স অ্যাডমিন, DevOps ইঞ্জিনিয়ার, ডেভেলপার, সবাই প্রতিদিন শেল স্ক্রিপ্ট লেখে। এই অধ্যায় থেকে তুমিও শুরু করবে।

এই অধ্যায়ে যা শিখবে

- শেল স্ক্রিপ্ট কী, কেন লেখা হয়, আর kernel ও shell মিলে ভেতরে কীভাবে একটা স্ক্রিপ্ট চালায়
- shebang (`#!`) লাইন কী, কেন দরকার, `#!/bin/bash` আর `#!/usr/bin/env bash`-এর পার্থক্য
- `chmod +x` দিয়ে স্ক্রিপ্টকে executable করা, আর স্ক্রিপ্ট চালানোর চারটি উপায়: `./script.sh`, `bash script.sh`, `source script.sh`, `. script.sh`
- কমেন্ট লেখা (single-line, multi-line কৌশল)
- ভেরিয়েবল: তৈরি করা, `$var` ও `${var}`, ডিফল্ট মান, `readonly`, `unset`
- কোটিং: single quote (`'`), double quote (`"`), backslash (`\`), আর command substitution `$( )`
- `read` দিয়ে ইউজারের কাছ থেকে ইনপুট নেওয়া (`-p`, `-s`, `-t`, `-n`, `-r`, `-a`)
- positional parameters: `$0`, `$1` … `$9`, `${10}`, `$#`, `$@`, `$*`, `shift`, `set --`
- exit status, `$?`, `exit` কমান্ড, আর বিশেষ exit code (126, 127, 130 ইত্যাদি)
- গাণিতিক হিসাব: `$(( ))`, `(( ))`, `let`, `expr`, আর দশমিকের জন্য `bc`
- `test`, `[ ]`, `[[ ]]`: ফাইল, স্ট্রিং ও সংখ্যা তুলনার সব অপারেটর
- `if` / `elif` / `else`, `case`, আর `&&` ও `||` শর্টকাট
- স্ক্রিপ্ট ডিবাগ করা: `bash -n`, `bash -x`, `set -x`, `shellcheck`

৪১.১ শেল স্ক্রিপ্ট কী?

শেল স্ক্রিপ্ট (shell script) হলো একটা সাধারণ টেক্সট ফাইল, যার ভেতরে পরপর shell কমান্ড লেখা থাকে। shell (অর্থাৎ যে প্রোগ্রাম তোমার কমান্ড বুঝে কাজ করায়, যেমন `bash`; বিস্তারিত অধ্যায় ৫-এ) ফাইলটা উপর থেকে নিচে লাইন ধরে পড়ে আর প্রতিটা কমান্ড চালায়।

উপমা: রান্নার রেসিপির কথা ভাবো। "পেঁয়াজ কাটো, তেল গরম করো, পেঁয়াজ ভাজো..."। রেসিপি লিখে রাখলে যে কেউ যেকোনো দিন একই রান্না একইভাবে করতে পারে। শেল স্ক্রিপ্ট হলো কম্পিউটারের জন্য লেখা রেসিপি, আর

shell হলো রাঁধুনি।

কেন শেল স্ক্রিপ্ট শিখবে?

কারণ	উদাহরণ
অটোমেশন	প্রতিদিন রাত ২টায় ডাটাবেস ব্যাকআপ (অধ্যায় ২৯-এর cron দিয়ে)
পুনরাবৃত্তি এড়ানো	১০০টা সার্ভারে একই কনফিগারেশন
ভুল কমানো	হাতে টাইপ করলে টাইপো হয়, স্ক্রিপ্ট প্রতিবার একই কাজ করে
ডকুমেন্টেশন	স্ক্রিপ্ট নিজেই বলে দেয় কাজটা কীভাবে হয়
সব জায়গায় চলে	প্রায় প্রতিটা লিনাক্স সিস্টেমে bash আগে থেকেই থাকে, আলাদা কিছু ইনস্টল করতে হয় না

শেল স্ক্রিপ্ট বনাম অন্য প্রোগ্রামিং ভাষা

শেল স্ক্রিপ্ট সবচেয়ে ভালো কাজ করে যখন তোমার কাজ মূলত **অন্য প্রোগ্রাম চালানো আর তাদের জোড়া লাগানো**: ফাইল কপি, সার্ভিস রিস্টার্ট, লগ ফিল্টার। জটিল ডেটা স্ট্রাকচার, বড় হিসাব বা হাজার লাইনের লজিক থাকলে Python-এর মতো ভাষা ভালো।

> 💡 **টিপস:** একটা চলতি নিয়ম আছে: স্ক্রিপ্ট ১০০–২০০ লাইন ছাড়িয়ে গেলে আর তাতে জটিল ডেটা নিয়ে কাজ থাকলে Python-এ লেখার কথা ভাবো।

কোন shell-এর জন্য লিখছি?

লিনাক্সে কয়েকটা shell আছে: `sh` (POSIX shell), `bash`, `zsh`, `dash`, `fish`। এই বইয়ের সব স্ক্রিপ্ট **bash**-এর জন্য, কারণ Ubuntu, Debian, Fedora, RHEL সবখানেই bash ডিফল্ট ইউজার shell। তোমার bash ভার্সন দেখো:

```
$ bash --version
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
...
```

(Ubuntu 24.04-এ 5.2.x, Debian 12-এ 5.2.15, RHEL 9-এ 5.1.x।)

> ⚠️ **সতর্কতা:** Ubuntu ও Debian-এ `/bin/sh` আসলে **dash**-এর দিকে symlink, bash না। dash অনেক দ্রুত কিন্তু ছোট: `[[ ]]`, অ্যারে, `{1..10}` এর মতো bash-এর বাড়তি ফিচার (যাকে "bashism" বলে) এতে নেই। Fedora/RHEL-এ `/bin/sh` bash-এর দিকে যায়। তাই bash ফিচার ব্যবহার করলে স্ক্রিপ্ট অবশ্যই bash দিয়ে চালাবে, `sh` দিয়ে না।

```
$ ls -l /bin/sh          # Ubuntu 24.04
lrwxrwxrwx 1 root root 4 Mar 31  2024 /bin/sh -> dash
```

৪১.২ প্রথম স্ক্রিপ্ট: Hello World

একটা ডিরেক্টরি বানিয়ে প্রথম স্ক্রিপ্ট লেখো। এডিটর হিসেবে `nano` (অধ্যায় ১৩) বা `vim` (অধ্যায় ১৪) ব্যবহার করো।

```
$ mkdir -p ~/scripts && cd ~/scripts
$ nano hello.sh
```

ফাইলে লেখো:

```
#!/bin/bash
# আমার প্রথম স্ক্রিপ্ট
echo "Hello, World!"
echo "আজকের তারিখ: $(date +%F)"
echo "তুমি লগইন করেছ: $USER হিসেবে"
```

সেভ করে বের হও (nano-তে `Ctrl+O`, `Enter`, `Ctrl+X`)। এবার executable বানাও আর চালাও:

```
$ chmod +x hello.sh
$ ./hello.sh
Hello, World!
আজকের তারিখ: 2026-11-12
তুমি লগইন করেছ: rahim হিসেবে
```

অভিনন্দন! তুমি প্রথম শেল স্ক্রিপ্ট লিখে ফেলেছ। এখন প্রতিটা অংশ খুঁটিয়ে দেখি।

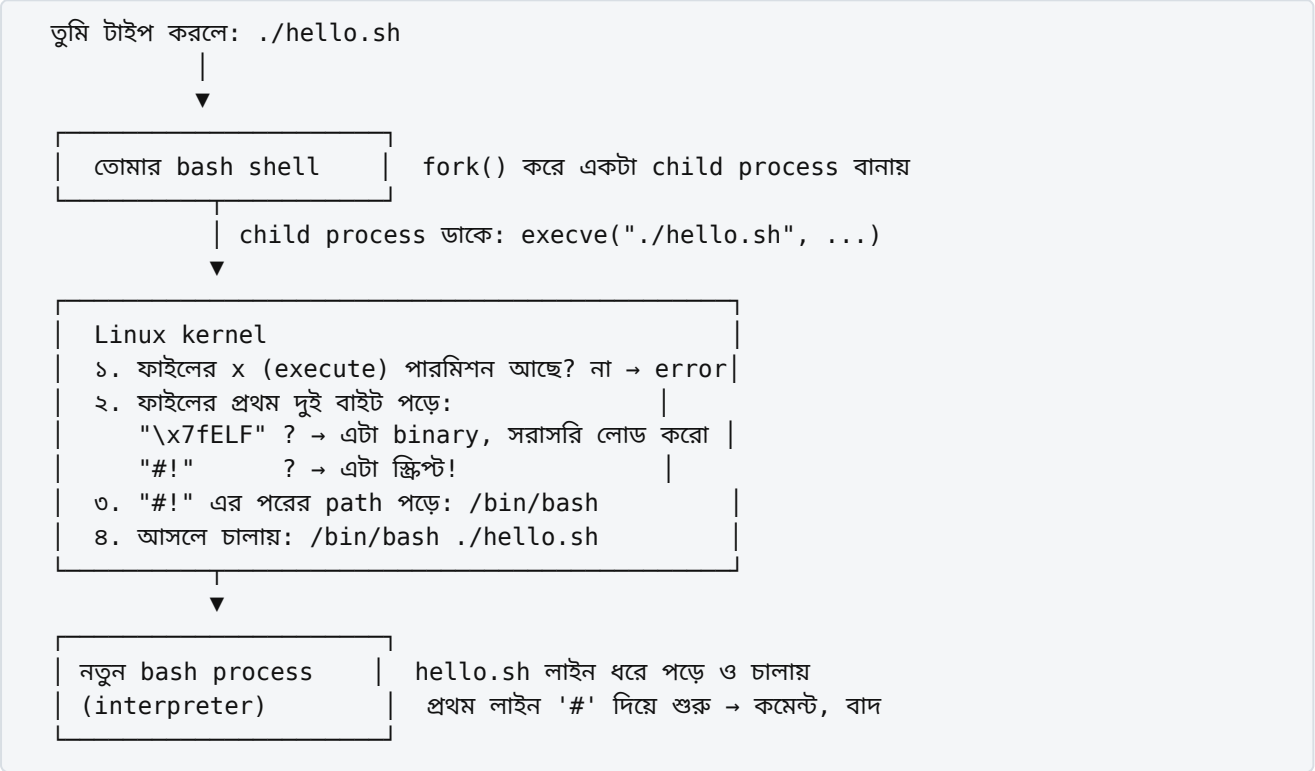
> 📝 **মনে রাখো:** `.sh` এক্সটেনশন বাধ্যতামূলক না। লিনাক্স ফাইলের নাম দেখে ঠিক করে না কোন প্রোগ্রাম চালাবে, সেটা ঠিক করে shebang লাইন। তবে `.sh` দিলে মানুষ ও এডিটর (syntax highlighting-এর জন্য) সহজে বোঝে। `/usr/local/bin`-এ রাখা কমান্ড-স্টাইল স্ক্রিপ্টে সাধারণত এক্সটেনশন দেওয়া হয় না।

৪১.৩ Shebang (`#!`) — স্ক্রিপ্টের প্রথম লাইন

স্ক্রিপ্টের একদম প্রথম লাইনের `#!/bin/bash` কে বলে **shebang** (sharp `#` + bang `!`)। এটা বলে দেয়: "এই ফাইলটা চালাতে `/bin/bash` প্রোগ্রাম ব্যবহার করো।"

ভেতরে কীভাবে কাজ করে (under the hood)

যখন তুমি `./hello.sh` টাইপ করো, তখন যা ঘটে:



অর্থাৎ shebang আসলে **kernel**-এর একটা ফিচার। kernel ফাইলের শুরুতে `#!` দেখলে বোঝে এটা interpreter দিয়ে চালাতে হবে। আর মজার ব্যাপার: bash-এর কাছে `#!/bin/bash` লাইনটা নিছক একটা কমেন্ট, কারণ এটা `#` দিয়ে শুরু।

বিভিন্ন shebang

Shebang	মানে
`#!/bin/bash`	সরাসরি `/bin/bash` দিয়ে চালাও (সবচেয়ে প্রচলিত)
`#!/usr/bin/env bash`	`PATH`-এ যেখানে প্রথম `bash` পাওয়া যায় সেটা দিয়ে চালাও (বেশি portable)
`#!/bin/sh`	POSIX shell দিয়ে চালাও (Ubuntu-তে dash), শুধু POSIX ফিচার ব্যবহার করবে
`#!/usr/bin/env python3`	Python স্ক্রিপ্ট
`#!/bin/bash -e`	bash চালাও `-e` অপশন সহ (error-এ থেমে যাবে)

`#!/usr/bin/env bash` কেন? কিছু সিস্টেমে (যেমন macOS-এ Homebrew-র নতুন bash, বা FreeBSD) bash `/bin/bash`-এ থাকে না, `/usr/local/bin/bash`-এ থাকে। `env` প্রোগ্রাম `PATH` খুঁজে bash বের করে, তাই স্ক্রিপ্ট বেশি জায়গায় চলে। লিনাক্সে দুটোই কাজ করে।

> ⚠️ **সতর্কতা:**

- shebang অবশ্যই **প্রথম লাইনের একদম শুরুতে** হতে হবে। আগে একটা ফাঁকা লাইন বা স্পেস থাকলেও kernel চিনবে না।
- Windows-এ লেখা ফাইলে লাইনের শেষে `\r\n` (CRLF) থাকে। তখন kernel `/bin/bash\r` খুঁজবে আর এরর দেবে: `/bin/bash^M: bad interpreter: No such file or directory`। সমাধান: `sed -i 's/\r$//' script.sh` বা `dos2unix script.sh`।
- লিনাক্সে `#!/usr/bin/env bash -e` এভাবে env-এর পরে একাধিক আর্গুমেন্ট দিলে কাজ করে না, কারণ kernel পুরো "bash -e" কে একটাই আর্গুমেন্ট হিসেবে পাঠায়। (নতুন coreutils-এ `env -S` দিয়ে এর সমাধান আছে: `#!/usr/bin/env -S bash -e`।)

shebang না দিলে কী হয়? `./script.sh` চালালে bash নিজে বুঝে নেয় এটা shell স্ক্রিপ্ট এবং সাধারণত `bash` দিয়েই চালায়, কিন্তু অন্য shell (যেমন zsh বা অন্য প্রোগ্রাম থেকে চালালে) ভিন্ন আচরণ করতে পারে। তাই **সবসময় shebang দেবে।**

৪১.৪ স্ক্রিপ্ট চালানোর উপায়

`chmod +x` — executable পারমিশন

লিনাক্সে কোনো ফাইল সরাসরি প্রোগ্রাম হিসেবে চালাতে তার **execute (x)** পারমিশন লাগে (পারমিশনের বিস্তারিত অধ্যায় ১৬-এ)।

```
$ ls -l hello.sh
-rw-rw-r-- 1 rahim rahim 120 Nov 12 10:00 hello.sh
$ ./hello.sh
bash: ./hello.sh: Permission denied
$ chmod +x hello.sh          # সবার জন্য x যোগ করো
$ ls -l hello.sh
-rwxrwxr-x 1 rahim rahim 120 Nov 12 10:00 hello.sh
```

কমান্ড	ফল
`chmod +x file`	owner, group, others সবাইকে execute পারমিশন (umask মেনে)
`chmod u+x file`	শুধু owner-কে execute পারমিশন
`chmod 755 file`	`rwxr-xr-x`: owner সব পারে, বাকিরা পড়তে ও চালাতে পারে
`chmod 700 file`	`rwx------`: শুধু তুমি (গোপন তথ্য থাকলে)

📝 **মনে রাখো:** স্ক্রিপ্ট `./` দিয়ে চালাতে **read (r)** আর **execute (x)** দুটোই লাগে, কারণ interpreter (bash) ফাইলটা পড়বে। শুধু x থাকলে চলবে না। কিন্তু binary প্রোগ্রাম চালাতে শুধু x-ই যথেষ্ট।

উপায় ১: `./script.sh` (path দিয়ে)

```
$ ./hello.sh
```

কেন `./` লাগে? তুমি শুধু `hello.sh` লিখলে shell সেটাকে কমান্ড ভেবে `PATH` ভেরিয়েবলের ডিরেক্টরিগুলোতে খোঁজে (`/usr/bin`, `/bin` ইত্যাদি)। বর্তমান ডিরেক্টরি (`.`) নিরাপত্তার কারণে `PATH`-এ থাকে না। কেউ যদি তোমার ডিরেক্টরিতে `ls` নামের ক্ষতিকর স্ক্রিপ্ট রেখে দেয়, আর `.` যদি PATH-এ থাকত, তুমি `ls` টাইপ করলেই সেটা চলে যেত! তাই `./hello.sh` মানে "বর্তমান ডিরেক্টরির hello.sh"।

```
$ hello.sh
hello.sh: command not found
$ ./hello.sh                    # relative path
$ ~/scripts/hello.sh            # absolute path, যেকোনো জায়গা থেকে
```

স্ক্রিপ্টকে সাধারণ কমান্ডের মতো যেকোনো জায়গা থেকে চালাতে চাইলে PATH-এ থাকা ডিরেক্টরিতে রাখো। Ubuntu/Debian/Fedora-তে `~/.local/bin` ডিরেক্টরি থাকলে লগইনের সময় সেটা PATH-এ যোগ হয় (Ubuntu-র `~/.profile` এটা করে):

```
$ mkdir -p ~/.local/bin
$ cp hello.sh ~/.local/bin/hello
$ hello                          # নতুন লগইন বা source ~/.profile এর পরে
Hello, World!
```

সবার জন্য হলে `sudo cp hello.sh /usr/local/bin/hello`। PATH নিয়ে বিস্তারিত অধ্যায় ৪৪-এ।

উপায় ২: `bash script.sh` (interpreter-কে সরাসরি)

```
$ bash hello.sh
```

এখানে তুমি নিজেই bash চালাচ্ছ আর ফাইলের নাম আর্গুমেন্ট হিসেবে দিচ্ছ। ফলে:

- **x পারমিশন লাগে না**, শুধু r লাগে।
- shebang উপেক্ষিত হয় (bash এটাকে কমেন্ট হিসেবে পড়ে)।
- ডিবাগের জন্য অপশন দেওয়া যায়: `bash -x hello.sh`।

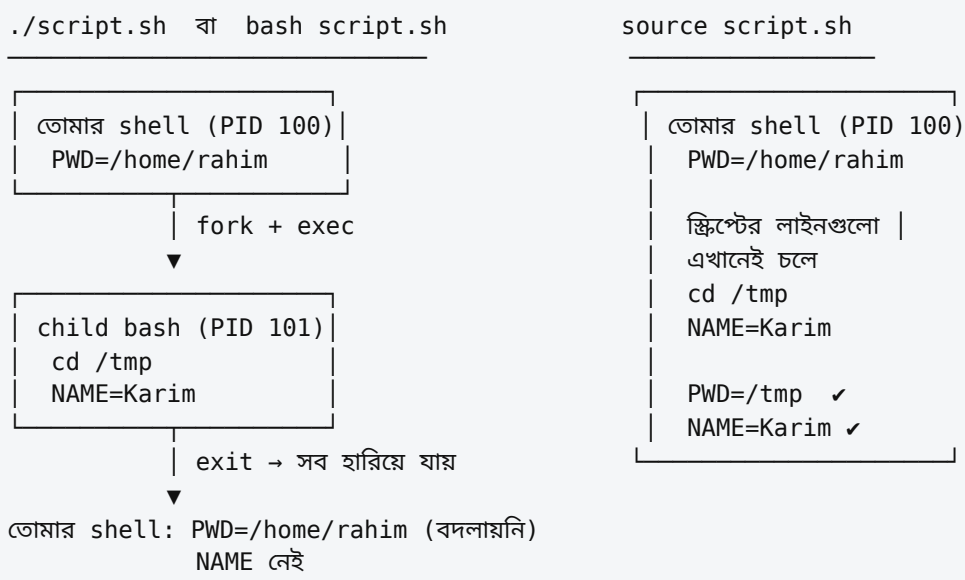
> ⚠️ **সতর্কতা:** `sh hello.sh` আর `bash hello.sh` এক না! Ubuntu-তে `sh` মানে dash, তাই bash-ফিচারওয়ালা স্ক্রিপ্ট `sh` দিয়ে চালালে `[[: not found` জাতীয় এরর পাবে।

উপায় ৩: `source script.sh` বা `. script.sh` (বর্তমান shell-এ)

```
$ source hello.sh
$ . hello.sh          # একই জিনিস; '.' হলো POSIX নাম, 'source' হলো bash-এর নাম
```

এখানে **নতুন process তৈরি হয় না**। স্ক্রিপ্টের কমান্ডগুলো তোমার বর্তমান shell-এই চলে। এর মানে স্ক্রিপ্ট যে ভেরিয়েবল সেট করে, `cd` করে, বা alias বানায়, সেগুলো স্ক্রিপ্ট শেষ হওয়ার পরেও তোমার shell-এ থেকে যায়।

ভেতরের পার্থক্য: subshell বনাম current shell



হাতে-কলমে দেখো:

```
$ cat > test_cd.sh << 'EOF'
#!/bin/bash
cd /tmp
MYVAR="hello"
echo "স্ক্রিপ্টের ভেতরে: $PWD"
EOF
$ chmod +x test_cd.sh
$ ./test_cd.sh
স্ক্রিপ্টের ভেতরে: /tmp
$ pwd; echo "MYVAR=$MYVAR"
/home/rahim/scripts
MYVAR=
$ source test_cd.sh
স্ক্রিপ্টের ভেতরে: /tmp
$ pwd; echo "MYVAR=$MYVAR"
/tmp
MYVAR=hello
```

কোনটা কখন?

উপায়	নতুন process?	x পারমিশন লাগে?	shebang মানে?	কখন ব্যবহার করবে
`./script.sh`	হ্যাঁ	হ্যাঁ (+ r)	হ্যাঁ	সাধারণ স্ক্রিপ্ট চালাতে (স্বাভাবিক উপায়)
`bash script.sh`	হ্যাঁ	না (শুধু r)	না	x নেই, বা ডিবাগ অপশন দিতে

উপায়	নতুন process?	x পারমিশন লাগে?	shebang মানে?	কখন ব্যবহার করবে
`source script.sh`	না	না (শুধু r)	না	`.bashrc` রিলোড, venv activate, env ফাইল লোড
`. script.sh`	না	না (শুধু r)	না	`source`-এর POSIX রূপ, `sh`/dash-এও চলে

> 💡 **টিপস:** `~/.bashrc` এডিট করার পর `source ~/.bashrc` চালাও যাতে নতুন টার্মিনাল না খুলেই পরিবর্তন কার্যকর হয়। Python-এর `source venv/bin/activate` ও একই কারণে source: এটা তোমার বর্তমান shell-এর PATH বদলায়।

> ⚠️ **সতর্কতা:** source করা স্ক্রিপ্টে `exit` থাকলে সেটা **তোমার টার্মিনালটাই বন্ধ করে দেবে**, কারণ স্ক্রিপ্ট তোমার shell-এই চলছে। source করার জন্য বানানো ফাইলে `exit`-এর বদলে `return` ব্যবহার করো।

৪১.৫ কমেন্ট

`#` চিহ্নের পর থেকে লাইনের শেষ পর্যন্ত সবকিছু **কমেন্ট**, অর্থাৎ shell এটা উপেক্ষা করে। কমেন্ট মানুষের জন্য: ছয় মাস পরে তুমি নিজেই ভুলে যাবে কোন লাইন কেন লিখেছিলে।

```
#!/bin/bash
# ============================================
# স্ক্রিপ্ট: backup.sh
# কাজ:      /etc ডিরেক্টরির ব্যাকআপ নেয়
# লেখক:     রহিম
# তারিখ:    2026-11-12
# ব্যবহার:  ./backup.sh [গন্তব্য]
# ============================================

DEST="/backup"      # লাইনের শেষেও কমেন্ট দেওয়া যায়

echo "Price: #1"    # কোটের ভেতরের # কমেন্ট না, এটা প্রিন্ট হবে
echo file#1         # শব্দের মাঝখানের # ও কমেন্ট না
```

```
$ ./demo.sh
Price: #1
file#1
```

> 📝 **মনে রাখো:** `#` তখনই কমেন্ট শুরু করে যখন সেটা **একটা শব্দের শুরুতে** থাকে (লাইনের শুরু বা স্পেসের পরে) এবং কোটের ভেতরে না থাকে।

একাধিক লাইনের কমেন্ট

bash-এ আলাদা multi-line কমেন্ট সিনট্যাক্স নেই। দুটো প্রচলিত কৌশল:

```
# কৌশল ১: প্রতিটা লাইনে # (সবচেয়ে পরিষ্কার, প্রস্তাবিত)
# লাইন এক
# লাইন দুই

# কৌশল ২: no-op কমান্ড ':' আর here-document
: << 'COMMENT'
এই পুরো ব্লক উপেক্ষিত হবে।
কোড সাময়িকভাবে বন্ধ রাখতে কাজে লাগে।
COMMENT
```

`:` হলো একটা built-in কমান্ড যা কিছুই করে না আর সবসময় সফল (exit status 0) হয়। here-document-এর লেখাটা তার stdin-এ যায় আর ফেলে দেওয়া হয়। `'COMMENT'` কোটে রাখার কারণ যাতে ভেতরের `$` বা `` ` `` expand না হয়।

> 💡 **টিপস:** vim-এ অনেকগুলো লাইন একসাথে কমেন্ট করতে `Ctrl+V` দিয়ে visual block সিলেক্ট করো, `I` চাপো, `#` লেখো, তারপর `Esc` (অধ্যায় ১৪)।

৪১.৬ ভেরিয়েবল

ভেরিয়েবল (variable) হলো একটা নাম দেওয়া বাক্স যাতে তুমি মান (value) রাখো আর পরে ব্যবহার করো।

উপমা: রান্নাঘরে লেবেল লাগানো কৌটা। কৌটার গায়ে লেখা "চিনি", ভেতরে চিনি। "চিনি" বললে সবাই বোঝে কোন কৌটা। ভেতরের জিনিস বদলাতে পারো, লেবেল একই থাকে।

ভেরিয়েবল তৈরি (assign)

```
name="Rahim"
age=25
city=Dhaka
greeting="Hello World"
empty=""
```

নিয়মগুলো (খুব গুরুত্বপূর্ণ):

1. `=` এর **দুই পাশে কোনো স্পেস থাকবে না।**
2. নামে অক্ষর (a-z, A-Z), সংখ্যা (0-9) ও `_` থাকতে পারে, কিন্তু সংখ্যা দিয়ে শুরু হবে না।
3. নাম case-sensitive: `name`, `Name`, `NAME` তিনটা আলাদা ভেরিয়েবল।
4. মানে স্পেস থাকলে কোট দিতে হবে।

```
$ name = "Rahim"
name: command not found        # bash ভেবেছে 'name' একটা কমান্ড, '=' আর '"Rahim"' তার
আর্গুমেন্ট
$ name= "Rahim"
Rahim: command not found       # name="" সেট করে "Rahim" কমান্ড চালানোর চেষ্টা!
$ greeting=Hello World
World: command not found       # greeting=Hello সেট করে World কমান্ড চালাতে চাইছে
$ greeting="Hello World"       # ✔ সঠিক
```

কেন এমন হয়? bash প্রতিটা লাইনকে স্পেস দিয়ে ভেঙে শব্দ বানায়, আর প্রথম শব্দকে কমান্ড ধরে। `name=Rahim` একটাই শব্দ আর তাতে `=` আছে, তাই bash চেনে এটা assignment। `VAR=value command` এই ফর্মটার আলাদা মানে আছে: শুধু ওই কমান্ডের জন্য একটা env ভেরিয়েবল সেট করা (যেমন `LANG=C sort file`)।

> 💡 **টিপস:** নাম দেওয়ার প্রচলিত রীতি: নিজের স্ক্রিপ্টের সাধারণ ভেরিয়েবলে ছোট হাতের অক্ষর (`file_count`), environment ও constant-এ বড় হাতের অক্ষর (`PATH`, `HOME`, `MAX_RETRY`)। নিজের ভেরিয়েবলের নাম `PATH`, `USER`, `HOME`, `IFS` দিলে সিস্টেমের ভেরিয়েবল নষ্ট হবে।

ভেরিয়েবলের মান পড়া: `$var` আর `${var}`

মান পেতে নামের আগে `$` দাও। এটাকে বলে **parameter expansion**: bash কমান্ড চালানোর আগে `$name` কে তার মান দিয়ে বদলে দেয়।

```
$ name="Rahim"
$ echo $name
Rahim
$ echo "আমার নাম $name"
আমার নাম Rahim
$ echo ${name}
Rahim
```

`${var}` কখন লাগে? যখন ভেরিয়েবলের নামের ঠিক পরে অক্ষর, সংখ্যা বা `_` থাকে:

```
$ fruit="apple"
$ echo "I have 5 $fruits"
I have 5                       # bash খুঁজেছে 'fruits' নামের ভেরিয়েবল, যা নেই
$ echo "I have 5 ${fruit}s"
I have 5 apples
$ file="report"
$ echo "$file_2026.txt"
.txt                           # 'file_2026' নামের ভেরিয়েবল খোঁজা হয়েছে
$ echo "${file}_2026.txt"
report_2026.txt
```

> 📝 **মনে রাখো:** সন্দেহ হলে `${var}` লেখো, কখনো ভুল হবে না।

ভেতরে কী হয়: bash কীভাবে একটা লাইন প্রসেস করে

```
ইনপুট লাইন:   echo "Hello $name" $(date +%Y) *.txt
      │
      ▼
১. Tokenize: শব্দ ও অপারেটরে ভাঙা (কোট মেনে)
      ▼
২. Brace expansion:      {a,b} → a b
      ▼
৩. Tilde expansion:      ~ → /home/rahim
      ▼
৪. Parameter expansion:  $name → Rahim
   Command substitution: $(date +%Y) → 2026
   Arithmetic expansion: $((2+3)) → 5
      ▼
৫. Word splitting:  কোটের বাইরের ফলাফল IFS (স্পেস/ট্যাব/নিউলাইন) দিয়ে ভাঙা
      ▼
৬. Pathname (glob) expansion: *.txt → a.txt b.txt
      ▼
৭. Quote removal: কোট চিহ্নগুলো সরানো
      ▼
৮. কমান্ড চালানো: echo  "Hello Rahim"  2026  a.txt  b.txt
```

ধাপ ৫ আর ৬ (word splitting ও globbing) হলো শেল স্ক্রিপ্টের বেশিরভাগ বাগের উৎস। double quote এই দুই ধাপ বন্ধ করে দেয়। তাই নিচের নিয়মটা মুখস্থ করো।

> ⚠️ **সতর্কতা: ভেরিয়েবল সবসময় double quote-এ রাখো: `"$var"`**
>
> ```
> $ file="my report.txt"
> $ touch "$file"
> $ rm $file                  # ✘ word splitting: rm 'my' 'report.txt'
> rm: cannot remove 'my': No such file or directory
> rm: cannot remove 'report.txt': No such file or directory
> $ rm "$file"                # ✔ একটাই আর্গুমেন্ট: 'my report.txt'
> ```
>
> আরও ভয়ংকর: `dir=""` থাকা অবস্থায় `rm -rf $dir/*` চালালে সেটা হয়ে যায় `rm -rf /*`! এজন্য `"${dir:?}"` ব্যবহার করা হয় (নিচে দেখো)।

ভেরিয়েবল মুছে ফেলা ও read-only করা

```
$ city="Dhaka"
$ unset city                # ভেরিয়েবল মুছে ফেলো ($ ছাড়া নাম দাও)
$ echo "[$city]"
[]

$ readonly PI=3.14159       # constant: আর বদলানো যাবে না
$ PI=3
bash: PI: readonly variable
```

```
$ unset PI
bash: unset: PI: cannot unset: readonly variable
```

`declare` দিয়েও ভেরিয়েবলের ধরন ঠিক করা যায়:

কমান্ড	কাজ
`declare -r NAME=val`	read-only (`readonly` এর মতো)
`declare -i num=5`	integer: assign করলে গাণিতিকভাবে হিসাব হয় (`num="2+3"` → 5)
`declare -l s="HeLLo"`	সবসময় ছোট হাতের অক্ষরে রাখে → `hello`
`declare -u s="hello"`	সবসময় বড় হাতের অক্ষরে রাখে → `HELLO`
`declare -x VAR=val`	export করা (`export VAR=val` এর মতো)
`declare -p var`	ভেরিয়েবলের ধরন ও মান দেখাও (ডিবাগে কাজের)

লোকাল বনাম environment ভেরিয়েবল (সংক্ষেপে)

সাধারণ ভেরিয়েবল শুধু বর্তমান shell-এ থাকে, child process (যেমন তোমার চালানো স্ক্রিপ্ট) দেখতে পায় না। `export` করলে child process-গুলো একটা কপি পায়।

```
$ color="blue"
$ bash -c 'echo "child দেখছে: [$color]"'
child দেখছে: []
$ export color
$ bash -c 'echo "child দেখছে: [$color]"'
child দেখছে: [blue]
```

`export` , `env` , `.bashrc` ও environment-এর পুরো বিস্তারিত অধ্যায় ৪৪-এ।

কাজের কিছু built-in ভেরিয়েবল

ভেরিয়েবল	মান
`$HOME`	তোমার home ডিরেক্টরি
`$USER`	বর্তমান ইউজারের নাম
`$PWD`	বর্তমান ডিরেক্টরি
`$HOSTNAME`	কম্পিউটারের নাম
`$RANDOM`	প্রতিবার 0–32767 এর মধ্যে একটা র‍্যান্ডম সংখ্যা
`$SECONDS`	shell/স্ক্রিপ্ট চালু হওয়ার পর কত সেকেন্ড গেছে
`$LINENO`	স্ক্রিপ্টের বর্তমান লাইন নম্বর (ডিবাগে কাজের)
`$BASH_VERSION`	bash-এর ভার্সন
`$$`	বর্তমান shell/স্ক্রিপ্টের PID

ভেরিয়েবল	মান
`$!`	সর্বশেষ background-এ পাঠানো process-এর PID
`$?`	শেষ কমান্ডের exit status (নিচে বিস্তারিত)

ডিফল্ট মান ও খালি ভেরিয়েবল সামলানো

parameter expansion-এর কিছু রূপ স্ক্রিপ্টে সবসময় লাগে:

সিনট্যাক্স	মানে
`${var:-default}`	var খালি বা unset হলে `default` ব্যবহার করো (var বদলায় না)
`${var:=default}`	var খালি বা unset হলে var-এ `default` বসাও এবং সেটা ব্যবহার করো
`${var:?message}`	var খালি বা unset হলে message দেখিয়ে স্ক্রিপ্ট থামাও
`${var:+other}`	var-এ মান থাকলে `other` ব্যবহার করো, না থাকলে কিছুই না
`${#var}`	var-এর মানের দৈর্ঘ্য (অক্ষর সংখ্যা)

```
$ unset dest
$ echo "ব্যাকআপ যাবে: ${dest:-/backup}"
ব্যাকআপ যাবে: /backup
$ name="Rahim"; echo ${#name}
5
$ dir=""
$ rm -rf "${dir:?dir ভেরিয়েবল খালি!}/"*
bash: dir: dir ভেরিয়েবল খালি!
```

(`:` বাদ দিলে, যেমন `${var-default}`, শুধু unset হলে ডিফল্ট নেয়; খালি স্ট্রিং থাকলে নেয় না।) substring, find-replace (`${var/a/b}`), কেস বদলানো ইত্যাদি আরও parameter expansion অধ্যায় ৪৩-এ।

৪১.৭ কোটিং: `'` বনাম `"` বনাম `\`

bash-এ কিছু অক্ষরের বিশেষ মানে আছে: `$`, `` ` ``, `\`, `*`, `?`, `[`, `~`, `#`, `&`, `;`, `|`, `<`, `>`, `(`, `)`, স্পেস ইত্যাদি। কোটিং দিয়ে তুমি bash-কে বলো "এই অক্ষরগুলোকে বিশেষ ভেবো না, সাধারণ লেখা ভাবো।"

উপমা: single quote হলো সিল করা খাম: ভেতরে যা লেখা, ঠিক তা-ই পৌঁছায়, কেউ খুলে কিছু বদলায় না। double quote হলো স্বচ্ছ খাম: বেশিরভাগ লেখা যেমন আছে তেমনই যায়, কিন্তু ডাকপিয়ন (bash) ভেতরের `$নাম` জায়গাগুলো দেখে আসল মান বসিয়ে দেয়।

Single quote `'...'` — সম্পূর্ণ আক্ষরিক (literal)

ভেতরের **কোনো কিছুই** expand হয় না: না `$`, না `` ` ``, না `\`।

```
$ name="Rahim"
$ echo 'Hello $name, today is $(date +%A)'
Hello $name, today is $(date +%A)
```

single quote-এর ভেতরে single quote রাখা যায় না, এমনকি `\'` দিয়েও না। কৌশল: কোট বন্ধ করো, escape করা `'` দাও, আবার খোলো:

```
$ echo 'It'\''s Rahim'\''s book'
It's Rahim's book
$ echo $'It\'s fine\tTab\nNewline'     # $'...' (ANSI-C quoting): \n, \t, \' বোঝে
It's fine       Tab
Newline
```

Double quote `"..."` — আংশিক, ভেরিয়েবল expand হয়

ভেতরে শুধু এগুলো বিশেষ থাকে: `$` (ভেরিয়েবল, `$( )`, `$(( ))`), `` ` `` (পুরোনো command substitution), `\` (শুধু `$` `` ` `` `"` `\` আর newline-এর আগে), আর interactive shell-এ `!` (history expansion)। বাকি সব (স্পেস, `*`, `?`, `~`, `#`, `|` ইত্যাদি) আক্ষরিক।

```
$ echo "Hello $name, today is $(date +%A)"
Hello Rahim, today is Thursday
$ echo "দাম: \$50, ফাইল: *.txt, \"কোট\""
দাম: $50, ফাইল: *.txt, "কোট"
$ echo "home: ~"
home: ~                 # tilde double quote-এ expand হয় না
```

Backslash `\` — একটা অক্ষর escape

কোটের বাইরে `\` তার পরের **একটা** অক্ষরের বিশেষ মানে মুছে দেয়।

```
$ echo Price: \$100 \* 2
Price: $100 * 2
$ touch my\ file.txt          # স্পেসটাও escape করা যায়
$ ls my\ file.txt
my file.txt
```

লাইনের শেষে `\` দিলে পরের লাইনটা একই কমান্ডের অংশ ধরা হয় (line continuation):

```
sudo apt install \
    curl \
    git \
    htop
```

> ⚠️ **সতর্কতা:** লাইন continuation-এর `\` এর পরে একটা স্পেসও থাকলে কাজ করবে না, কারণ তখন `\` স্পেসটাকে escape করে, newline-কে না।

Command substitution: `$(command)`

একটা কমান্ডের **output** কে অন্য কমান্ডে বা ভেরিয়েবলে বসাতে `$(...)` ব্যবহার করো।

```
$ today=$(date +%F)
$ echo "আজ $today"
আজ 2026-11-12
$ files=$(ls /etc | wc -l)
$ echo "/etc-এ $files টা এন্ট্রি আছে"
/etc-এ 214 টা এন্ট্রি আছে
$ echo "kernel: $(uname -r)"
kernel: 6.8.0-48-generic
```

পুরোনো স্টাইল ব্যাকটিক `` `date` `` একই কাজ করে, কিন্তু nest করা কঠিন আর পড়তে ঝামেলা। **সবসময় `$( )` ব্যবহার করো।**

```
echo "$(basename "$(pwd)")"     # $( ) সহজেই nest হয়, ভেতরের কোটও নিরাপদ
```

ভেতরে কী হয়: bash একটা **subshell** (child process) চালিয়ে কমান্ডটা চালায়, তার stdout একটা pipe দিয়ে ধরে, শেষের সব newline কেটে ফেলে, তারপর ফলটা জায়গামতো বসায়।

কোটিং সারণি

লেখা	`$var`	`$(cmd)`	`\`	`*` glob	স্পেসে ভাঙা	উদাহরণ ফল (`name=Rahim`)
কোট ছাড়া	expand	expand	escape	হ্যাঁ	হ্যাঁ	`echo $name` → `Rahim`
`"..."`	expand	expand	আংশিক	না	না	`echo "$name"` → `Rahim`
`'...'`	না	না	না	না	না	`echo '$name'` → `$name`
`$'...'`	না	না	`\n \t` বোঝে	না	না	`echo $'a\tb'` → `a b`

> 📝 **মনে রাখো (সোনালি নিয়ম):**
>
> - ভেরিয়েবল বা `$( )` থাকলে → **double quote**: `"$file"`, `"$(pwd)"`
> - পুরো লেখাটা আক্ষরিক রাখতে চাইলে (regex, `awk` / `sed` প্রোগ্রাম) → **single quote**: `awk '{print $1}'`
> - কোট দেবে না শুধু তখন, যখন তুমি *ইচ্ছা করে* word splitting বা globbing চাও।

৪১.৮ `echo` ও `printf` — আউটপুট দেখানো

স্ক্রিপ্টে আউটপুট দেখাতে `echo` আর `printf` লাগে।

`echo` অপশন	কাজ
`-n`	শেষে newline দেবে না
`-e`	`\n`, `\t`, `\e` ইত্যাদি escape বুঝবে
`-E`	escape বুঝবে না (bash-এ ডিফল্ট)

```
$ echo -n "নাম: "; echo "Rahim"
নাম: Rahim
$ echo -e "লাইন১\nলাইন২\tট্যাব"
লাইন১
লাইন২    ট্যাব
```

`printf` আরও নির্ভরযোগ্য ও ফরম্যাটিং-এ শক্তিশালী (C-এর printf-এর মতো)। এটা নিজে থেকে newline দেয় না:

```
$ printf "%s is %d years old\n" "Rahim" 25
Rahim is 25 years old
$ printf "%-10s|%5.2f\n" "Price" 3.14159
Price     | 3.14
$ printf "%05d\n" 42
00042
```

ফরম্যাট	মানে
`%s`	স্ট্রিং
`%d`	পূর্ণসংখ্যা
`%f`, `%.2f`	দশমিক সংখ্যা (দশমিকের পর ২ ঘর)
`%x`, `%o`	হেক্সাডেসিমাল, অক্টাল
`%-10s`	১০ ঘর জায়গায় বামে সাজানো
`%5d`	৫ ঘর জায়গায় ডানে সাজানো
`%%`	আক্ষরিক `%`

> 💡 **টিপস:** এরর মেসেজ stderr-এ পাঠানো উচিত, যাতে আউটপুট ফাইলে redirect করলেও এরর স্ক্রিনে দেখা যায়: `echo "Error: file not found" >&2` (redirection-এর বিস্তারিত অধ্যায় ১৯-এ)।

৪১.৯ `read` — ইউজারের কাছ থেকে ইনপুট নেওয়া

`read` একটা built-in কমান্ড যা stdin থেকে একটা লাইন পড়ে ভেরিয়েবলে রাখে।

Syntax:

```
read [options] [variable1 variable2 ...]
```

সবচেয়ে সহজ ব্যবহার

```
#!/bin/bash
echo "তোমার নাম কী?"
read name
echo "স্বাগতম, $name!"
```

```
$ ./greet.sh
তোমার নাম কী?
Karim
স্বাগতম, Karim!
```

ভেরিয়েবলের নাম না দিলে ইনপুট `REPLY` নামের ভেরিয়েবলে যায়:

```
$ read; echo "তুমি লিখেছ: $REPLY"
hello
তুমি লিখেছ: hello
```

গুরুত্বপূর্ণ অপশন

অপশন	কাজ	উদাহরণ
`-p "prompt"`	একই লাইনে প্রম্পট দেখাও	`read -p "নাম: " name`
`-s`	silent: টাইপ করা লেখা দেখাবে না (পাসওয়ার্ড)	`read -s pass`
`-t N`	N সেকেন্ডের মধ্যে ইনপুট না দিলে বাদ (exit status > 128)	`read -t 10 ans`
`-n N`	N টা অক্ষর পড়েই থেমে যাও, Enter লাগবে না	`read -n 1 key`
`-r`	raw: `\` কে escape হিসেবে ধরবে না (**প্রায় সবসময় দাও**)	`read -r line`
`-a arr`	শব্দগুলো একটা অ্যারেতে রাখো	`read -a colors`
`-d 'x'`	newline-এর বদলে `x` অক্ষর পর্যন্ত পড়ো	`read -d ':' part`
`-e`	readline চালু করো (ইনপুট দেওয়ার সময় Tab, arrow key কাজ করবে)	`read -e -p "path: " p`
`-i "text"`	`-e` -এর সাথে: ডিফল্ট লেখা আগে থেকে বসিয়ে রাখো	`read -e -i "/tmp" -p "dir: " d`

উদাহরণ

```
#!/bin/bash
read -rp "ইউজারনেম: " user
read -rsp "পাসওয়ার্ড: " pass
echo                                  # -s এর পরে newline হয় না, তাই নিজে দাও
```

```
echo "ইউজার $user, পাসওয়ার্ডের দৈর্ঘ্য ${#pass}"

read -n 1 -rp "চালিয়ে যাবে? (y/n) " ans
echo
echo "তুমি চেপেছ: $ans"

if read -t 5 -rp "৫ সেকেন্ডে শহরের নাম লেখো: " city; then
    echo "শহর: $city"
else
    echo
    echo "সময় শেষ! ডিফল্ট: Dhaka"
    city="Dhaka"
fi
```

```
$ ./login.sh
ইউজারনেম: rahim
পাসওয়ার্ড:
ইউজার rahim, পাসওয়ার্ডের দৈর্ঘ্য 8
চালিয়ে যাবে? (y/n) y
তুমি চেপেছ: y
৫ সেকেন্ডে শহরের নাম লেখো:
সময় শেষ! ডিফল্ট: Dhaka
```

একাধিক ভেরিয়েবলে পড়া ও IFS

`read` ইনপুটকে **IFS** (Internal Field Separator, ডিফল্ট স্পেস/ট্যাব/নিউলাইন) দিয়ে ভাঙে। প্রথম শব্দ প্রথম ভেরিয়েবলে, দ্বিতীয়টা দ্বিতীয়তে... আর **বাকি সব শেষ ভেরিয়েবলে** যায়।

```
$ read first last <<< "Rahim Uddin Ahmed"
$ echo "first=$first, last=$last"
first=Rahim, last=Uddin Ahmed

$ IFS=: read -r user pw uid gid rest <<< "root:x:0:0:root:/root:/bin/bash"
$ echo "user=$user uid=$uid"
user=root uid=0
```

শেষ উদাহরণে `IFS=:` শুধু ওই একটা `read` কমান্ডের জন্য সেট হয়েছে, পরে আগের মতো থাকে। `<<<` হলো "here-string": একটা স্ট্রিংকে stdin হিসেবে পাঠায় (অধ্যায় ১৯)।

> ⚠️ **সতর্কতা:**
>
> - `-r` না দিলে ইনপুটের `\` হারিয়ে যায়: `C:\new` পড়লে পাবে `C:new`। তাই **`read -r`** অভ্যাস করো।
> - `echo "a b" | read x y` কাজ করবে না যেমন আশা করো! পাইপের ডান দিকের কমান্ড একটা subshell-এ চলে, তাই `x`, `y` সেখানেই হারিয়ে যায়। এর বদলে here-string ব্যবহার করো: `read x y <<< "a b"`।
> - ফাইল লাইন ধরে পড়ার জন্য `while IFS= read -r line; do ... done < file` প্যাটার্নটা অধ্যায় ৪২-এ শিখবে।

৪১.১০ Positional Parameters — কমান্ড লাইন আর্গুমেন্ট

তুমি `cp a.txt b.txt` লিখলে `cp` প্রোগ্রাম `a.txt` আর `b.txt` আর্গুমেন্ট হিসেবে পায়। তোমার স্ক্রিপ্টও একইভাবে আর্গুমেন্ট নিতে পারে। এগুলো থাকে বিশেষ ভেরিয়েবলে, যাদের বলে **positional parameters** (অবস্থান অনুযায়ী নম্বর দেওয়া প্যারামিটার)।

```
  $ ./backup.sh   /etc    /backup    daily
        |          |         |          |
        $0         $1        $2         $3          $# = 3
```

ভেরিয়েবল	মান
`$0`	স্ক্রিপ্টের নাম/path, যেভাবে ডাকা হয়েছে (`./backup.sh`)
`$1` … `$9`	প্রথম থেকে নবম আর্গুমেন্ট
`${10}` , `${11}` …	দশম ও তার পরের আর্গুমেন্ট (`{}` **বাধ্যতামূলক**)
`$#`	মোট আর্গুমেন্ট সংখ্যা (`$0` বাদে)
`"$@"`	সব আর্গুমেন্ট, **প্রতিটা আলাদা শব্দ হিসেবে**
`"$*"`	সব আর্গুমেন্ট **একটাই স্ট্রিং** হিসেবে (IFS-এর প্রথম অক্ষর, সাধারণত স্পেস দিয়ে জোড়া)

> ⚠️ **সতর্কতা:** `$10` মানে `${1}0` , অর্থাৎ প্রথম আর্গুমেন্টের পরে একটা `0` ! দশম আর্গুমেন্টের জন্য `${10}` লেখো।

উদাহরণ

```
#!/bin/bash
# args.sh
echo "স্ক্রিপ্টের নাম: $0"
echo "প্রথম আর্গুমেন্ট: $1"
echo "দ্বিতীয় আর্গুমেন্ট: $2"
echo "মোট আর্গুমেন্ট: $#"
echo "সবগুলো: $@"
```

```
$ ./args.sh apple "red banana" cherry
স্ক্রিপ্টের নাম: ./args.sh
প্রথম আর্গুমেন্ট: apple
দ্বিতীয় আর্গুমেন্ট: red banana
মোট আর্গুমেন্ট: 3
সবগুলো: apple red banana cherry
```

খেয়াল করো, `"red banana"` কোটে থাকায় এটা একটাই আর্গুমেন্ট। কোট বাদ দিলে `$#` হতো 4।

`"$@"` বনাম `"$*"` বনাম `$@` — বড় পার্থক্য

এটা অনেকেই গুলিয়ে ফেলে, তাই পরীক্ষা করে দেখো:

```
#!/bin/bash
# compare.sh
count() { echo "  $# টা আর্গুমেন্ট পেলাম:"; printf '    [%s]\n' "$@"; }

echo '"$@" ব্যবহার করে:'; count "$@"
echo '"$*" ব্যবহার করে:'; count "$*"
echo '$@ (কোট ছাড়া):';   count $@
```

```
$ ./compare.sh "my file.txt" other.txt
"$@" ব্যবহার করে:
  2 টা আর্গুমেন্ট পেলাম:
    [my file.txt]
    [other.txt]
"$*" ব্যবহার করে:
  1 টা আর্গুমেন্ট পেলাম:
    [my file.txt other.txt]
$@ (কোট ছাড়া):
  3 টা আর্গুমেন্ট পেলাম:
    [my]
    [file.txt]
    [other.txt]
```

(এখানে `count` একটা function; function-এর বিস্তারিত অধ্যায় ৪২-এ।)

> 📝 **মনে রাখো:** আর্গুমেন্টগুলো অন্য কমান্ডে হুবহু পাঠাতে **সবসময়** `"$@"` ব্যবহার করো। `"$*"` কাজে লাগে শুধু যখন সবগুলোকে একটা বাক্য হিসেবে দেখাতে চাও, যেমন `echo "তুমি লিখেছ: $*"`।

`shift` — আর্গুমেন্টগুলো বাঁয়ে সরানো

`shift` প্রথম আর্গুমেন্ট (`$1`) ফেলে দিয়ে বাকিগুলোকে এক ঘর বাঁয়ে সরায়: পুরোনো `$2` হয় নতুন `$1`, আর `$#` এক কমে। `shift N` দিলে N ঘর সরায়।

```
  shift-এর আগে:   $1=-v   $2=file1   $3=file2     $#=3
                   ✘ ফেলে দেওয়া
  shift-এর পরে:   $1=file1  $2=file2               $#=2
```

```
#!/bin/bash
# shift_demo.sh
echo "শুরুতে: $# টা — $*"
mode=$1
shift
echo "mode=$mode, বাকি $# টা ফাইল: $*"
shift 2
echo "আরও ২ ঘর সরানোর পর: $# টা — $*"
```

```
$ ./shift_demo.sh copy a.txt b.txt c.txt
শুরুতে: 4 টা — copy a.txt b.txt c.txt
mode=copy, বাকি 3 টা ফাইল: a.txt b.txt c.txt
আরও ২ ঘর সরানোর পর: 1 টা — c.txt
```

`shift` সাধারণত loop-এর সাথে ব্যবহার হয় অপশন পার্স করতে (`while [ $# -gt 0 ]; do ... shift; done`); সেটা অধ্যায় ৪২-এ, আর `getopts` অধ্যায় ৪৩-এ।

> ⚠️ **সতর্কতা:** যতগুলো আর্গুমেন্ট আছে তার চেয়ে বেশি shift করলে (যেমন `$#=1` অবস্থায় `shift 2`) কিছুই সরে না আর `shift` non-zero exit status ফেরত দেয়।

`set --` দিয়ে positional parameters নিজে সেট করা

```
$ set -- one two "three four"
$ echo "$# : $1 | $3"
3 : one | three four
$ set --            # সব মুছে ফেলো
$ echo $#
0
```

অন্যান্য বিশেষ প্যারামিটার

প্যারামিটার	মান
`$?`	শেষ কমান্ডের exit status
`$$`	বর্তমান shell-এর PID (অস্থায়ী ফাইলের নাম ইউনিক করতে আগে ব্যবহার হতো; এখন `mktemp` ভালো)
`$!`	শেষ background (`&`) কমান্ডের PID
`$-`	বর্তমান shell-এ চালু থাকা অপশনগুলো (যেমন `himBHs`)
`$_`	আগের কমান্ডের শেষ আর্গুমেন্ট (interactive-এ খুব কাজের: `mkdir newdir && cd $_`)

৪১.১১ Exit Status ও `exit`

লিনাক্সে প্রতিটা কমান্ড শেষ হওয়ার সময় তার parent process-কে একটা ছোট সংখ্যা ফেরত দেয়: **exit status** (বা exit code, return code)। এটা 0 থেকে 255 এর মধ্যে।

- **0 = সফল** (success)
- **1–255 = কোনো সমস্যা হয়েছে** (failure); কোন সংখ্যা কোন সমস্যা, সেটা প্রোগ্রাম ঠিক করে।

উপমা: পরীক্ষার খাতায় শিক্ষক শুধু একটা চিহ্ন দেন। "0" মানে কোনো ভুল নেই। অন্য যেকোনো সংখ্যা মানে কোথাও গোলমাল, আর সংখ্যা দেখে বোঝা যায় কী ধরনের ভুল। অন্য ভাষায় সাধারণত 0 মানে false, কিন্তু shell-এ

উল্টো: **0 মানে true/সফল।** কারণ সফল হওয়ার একটাই উপায়, কিন্তু ব্যর্থ হওয়ার অনেক কারণ থাকতে পারে, তাই ব্যর্থতার জন্য অনেকগুলো সংখ্যা রাখা হয়েছে।

`$?` দিয়ে exit status দেখা

```
$ ls /etc/hostname
/etc/hostname
$ echo $?
0
$ ls /nonexistent
ls: cannot access '/nonexistent': No such file or directory
$ echo $?
2
$ grep -q root /etc/passwd; echo $?
0
$ grep -q xyz123 /etc/passwd; echo $?
1
$ true; echo $?
0
$ false; echo $?
1
```

> ⚠️ **সতর্কতা:** `$?` শুধু **ঠিক আগের** কমান্ডের status রাখে। মাঝে একটা `echo` চালালেই সেটা মুছে যায়:
>
> ```
> $ ls /nonexistent 2>/dev/null
> $ echo "চেক করছি"
> চেক করছি
> $ echo $?
> 0             # এটা echo-র status, ls-এর না!
> ```
>
> তাই দরকার হলে সাথে সাথে সেভ করো: `ls /x; status=$?`।

প্রচলিত exit code

কোড	মানে
`0`	সফল
`1`	সাধারণ এরর
`2`	ভুল ব্যবহার (built-in-এর ভুল আর্গুমেন্ট; `ls` / `grep` এর গুরুতর এরর)
`126`	কমান্ড পাওয়া গেছে কিন্তু চালানো যায়নি (x পারমিশন নেই, বা এটা ডিরেক্টরি)
`127`	কমান্ড পাওয়া যায়নি (`command not found`)
`128 + N`	signal N দিয়ে মারা গেছে
`130`	`Ctrl+C` (SIGINT = 2, তাই 128+2)
`137`	`kill -9` (SIGKILL = 9, 128+9); Docker-এ OOM kill হলেও এটা দেখবে

কোড	মানে
`143`	`kill` / SIGTERM (15, 128+15)

```
$ notacommand; echo $?
notacommand: command not found
127
$ /etc/passwd; echo $?
bash: /etc/passwd: Permission denied
126
$ sleep 100
^C
$ echo $?
130
```

signal নিয়ে বিস্তারিত অধ্যায় ২৪-এ।

`exit` — স্ক্রিপ্ট থেকে বের হওয়া

`exit N` স্ক্রিপ্ট সাথে সাথে বন্ধ করে আর N কে exit status হিসেবে ফেরত দেয়। শুধু `exit` লিখলে শেষ কমান্ডের status-ই ফেরত যায়। আর স্ক্রিপ্ট `exit` ছাড়াই শেষ হলে শেষ কমান্ডের status হয় স্ক্রিপ্টের status।

```
#!/bin/bash
# check_file.sh
if [ $# -ne 1 ]; then
    echo "ব্যবহার: $0 <ফাইল>" >&2
    exit 2
fi

if [ ! -f "$1" ]; then
    echo "Error: '$1' ফাইল পাওয়া যায়নি" >&2
    exit 1
fi

echo "'$1' পাওয়া গেছে, সাইজ $(stat -c %s "$1") বাইট"
exit 0
```

```
$ ./check_file.sh
ব্যবহার: ./check_file.sh <ফাইল>
$ echo $?
2
$ ./check_file.sh /etc/hostname
'/etc/hostname' পাওয়া গেছে, সাইজ 7 বাইট
$ echo $?
0
```

> 💡 **টিপস:** তোমার স্ক্রিপ্ট সবসময় অর্থপূর্ণ exit status দেবে। তাহলে অন্য স্ক্রিপ্ট, cron, systemd বা CI/CD (GitHub Actions, Jenkins) বুঝতে পারবে কাজ সফল হয়েছে কি না।

> ⚠️ **সতর্কতা:** exit status 8 বিটের, তাই `exit 256` আসলে 0 হয়ে যায় (256 % 256), আর `exit 300` হয় 44। 0–255 এর মধ্যে থাকো। আর নিজের কোডের জন্য 126+ এড়িয়ে চলো, কারণ সেগুলোর বিশেষ মানে আছে।

`set -e` সম্পর্কে এক নজর

স্ক্রিপ্টের শুরুতে `set -e` দিলে কোনো কমান্ড ব্যর্থ হলেই স্ক্রিপ্ট থেমে যায়। পেশাদার স্ক্রিপ্টে প্রায়ই দেখবে:

```
#!/bin/bash
set -euo pipefail
```

- `-e` : কমান্ড ব্যর্থ হলে থামো
- `-u` : সেট না করা ভেরিয়েবল ব্যবহার করলে এরর দাও
- `-o pipefail` : পাইপের যেকোনো অংশ ব্যর্থ হলে পুরো পাইপ ব্যর্থ ধরো

এদের অনেক সূক্ষ্ম ব্যতিক্রম আছে (যেমন `if`-এর শর্তে বা `&&` / `||` চেইনে থাকা কমান্ড ব্যর্থ হলে `-e` থামায় না)। বিস্তারিত ও `trap` দিয়ে এরর সামলানো অধ্যায় ৪৩-এ।

৪১.১২ গাণিতিক হিসাব

bash-এ সব ভেরিয়েবল আসলে **স্ট্রিং**। তাই এভাবে লিখলে যোগ হবে না:

```
$ a=5; b=3
$ echo $a+$b
5+3
```

গাণিতিক হিসাবের জন্য আলাদা সিনট্যাক্স লাগে।

`$(( ))` — arithmetic expansion (প্রস্তাবিত)

```
$ a=17; b=5
$ echo $(( a + b ))
22
$ echo $(( a - b )) $(( a * b )) $(( a / b )) $(( a % b ))
12 85 3 2
$ echo $(( 2 ** 10 ))
1024
$ result=$(( (a + b) * 2 ))
$ echo $result
44
```

`$(( ))` এর ভেতরে ভেরিয়েবলে `$` না দিলেও চলে (`a` ও `$a` দুটোই কাজ করে), আর স্পেস ইচ্ছামতো দেওয়া যায়।

> ⚠️ **সতর্কতা:** bash শুধু **পূর্ণসংখ্যা** (integer) নিয়ে হিসাব করে। ভাগফলের দশমিক অংশ কেটে ফেলা হয় (round হয় না):

```
$ echo $(( 7 / 2 ))
3
$ echo $(( 10 / 3 * 3 ))
9
$ echo $(( 1.5 + 1 ))
bash: 1.5 + 1 : syntax error: invalid arithmetic operator (error token is ".5 + 1 ")
$ echo $(( 5 / 0 ))
bash: 5 / 0 : division by 0 (error token is "0 ")
```

অপারেটর টেবিল

অপারেটর	কাজ	উদাহরণ (a=10)	ফল
`+ - * /`	যোগ, বিয়োগ, গুণ, ভাগ (integer)	`$((a / 3))`	3
`%`	ভাগশেষ (modulo)	`$((a % 3))`	1
`**`	ঘাত (power)	`$((a ** 2))`	100
`++ --`	এক বাড়াও / কমাও	`((a++))`	a হবে 11
`+= -= *= /= %=`	হিসাব করে বসাও	`((a += 5))`	a হবে 15
`== != < > <= >=`	তুলনা (সত্য হলে 1, মিথ্যা হলে 0)	`$((a > 5))`	1
`&& || !`	লজিক্যাল AND, OR, NOT	`$((a > 5 && a < 20))`	1
`& | ^ ~ << >>`	বিটওয়াইজ অপারেশন	`$((a << 1))`	20
`? :`	ternary (শর্ত ? সত্য : মিথ্যা)	`$((a > 5 ? 1 : 0))`	1

`(( ))` — হিসাব ও শর্ত (expansion ছাড়া)

`$` ছাড়া `(( ))` লিখলে এটা কোনো মান দেয় না, বরং একটা **কমান্ড** হিসেবে চলে। ফল non-zero হলে exit status 0 (সত্য), ফল 0 হলে exit status 1 (মিথ্যা)। তাই এটা ভেরিয়েবল বদলানো আর সংখ্যা তুলনা, দুই কাজেই ব্যবহার হয়:

```
$ count=0
$ (( count++ ))
$ (( count += 10 ))
$ echo $count
11
$ if (( count > 5 )); then echo "বড়"; fi
বড়
```

> ⚠️ **সতর্কতা:** `(( expression ))` এর ফল 0 হলে exit status 1 হয়। তাই `count=0; (( count++ ))` এর exit status 1 (কারণ post-increment পুরোনো মান 0 ফেরত দেয়)। `set -e` চালু থাকলে এই নিরীহ লাইনেই স্ক্রিপ্ট থেমে যাবে! সমাধান: `(( ++count ))` বা `count=$((count + 1))`।

`let`

`let` হলো `(( ))` এর পুরোনো রূপ। প্রতিটা expression একটা আর্গুমেন্ট, তাই স্পেস দিলে কোট লাগবে:

```
$ let x=5+3
$ let "y = x * 2"
$ let x++
$ echo $x $y
9 16
```

`expr` (পুরোনো, এড়িয়ে চলো)

`expr` একটা আলাদা প্রোগ্রাম (`/usr/bin/expr`), POSIX-এর শুরুর দিন থেকে আছে। এখনকার স্ক্রিপ্টে দরকার নেই, কিন্তু পুরোনো স্ক্রিপ্টে দেখবে:

```
$ expr 5 + 3
8
$ expr 5 \* 3          # * কে escape করতে হয়, না হলে glob হয়ে যাবে
15
$ expr 5+3             # অপারেটরের দুই পাশে স্পেস বাধ্যতামূলক
5+3
```

`bc` — দশমিক সংখ্যার হিসাব

দশমিক (floating point) হিসাবের জন্য `bc` (basic calculator) ব্যবহার করো। বেশিরভাগ distro-তে থাকে; না থাকলে `sudo apt install bc` বা `sudo dnf install bc`।

```
$ echo "10 / 3" | bc
3
$ echo "scale=4; 10 / 3" | bc
3.3333
$ echo "scale=2; 1.5 * 2.25" | bc
3.37
$ echo "10 / 3" | bc -l           # -l: math library, scale=20
3.33333333333333333333
$ echo "sqrt(2)" | bc -l
1.41421356237309504880
$ echo "5.5 > 3.2" | bc           # তুলনা: সত্য হলে 1
1
$ bc <<< "2^100"
1267650600228229401496703205376
```

`bc` জিনিস	মানে
`scale=N`	দশমিকের পর কত ঘর (ডিফল্ট 0)
`-l`	math library লোড, scale=20; `s()` sine, `c()` cosine, `a()` arctan, `l()` ln, `e()` exp
`sqrt(x)`	বর্গমূল
`^`	ঘাত (শুধু পূর্ণসংখ্যা ঘাত)
`ibase`, `obase`	ইনপুট/আউটপুট base (যেমন `obase=2; 10` → 1010)

> 📝 **মনে রাখো:** `bc` শুধু কেটে ফেলে (truncate), round করে না: `scale=2; 1.5*2.25` = 3.375, দেখায় 3.37। ঠিকমতো round করে দেখাতে `printf` ব্যবহার করো: `printf "%.2f\n" "$(echo "1.5*2.25" | bc -l)"` → `3.38` (printf হিসাব নিজের দশমিক নিয়মে round করে)। আরেক বিকল্প `awk 'BEGIN{print 10/3}'` (অধ্যায় ২২)।

সংখ্যা base ও leading zero-র ফাঁদ

```
$ echo $(( 2#1010 ))        # binary
10
$ echo $(( 16#ff ))         # hex (বা 0xff)
255
$ echo $(( 010 ))           # 0 দিয়ে শুরু = অক্টাল!
8
$ month=08
$ echo $(( month + 1 ))
bash: 08: value too great for base (error token is "08")
$ echo $(( 10#$month + 1 ))   # 10# দিয়ে জোর করে দশমিক
9
```

> ⚠️ **সতর্কতা:** `date +%m` বা `date +%H` থেকে আসা `08`, `09` হিসাব করতে গেলে এই এরর হয়। সবসময় `10#$var` লেখো।

কোনটা কখন

পদ্ধতি	দশমিক?	কখন
`$(( ))`	না	মান বের করা: `x=$((a+b))` (সবচেয়ে প্রচলিত)
`(( ))`	না	ভেরিয়েবল বদলানো বা সংখ্যার শর্ত: `((i++))`, `if ((a>b))`
`let`	না	পুরোনো স্ক্রিপ্টে
`expr`	না	খুব পুরোনো POSIX স্ক্রিপ্ট; নতুন কোডে না
`bc`	হ্যাঁ	দশমিক, বড় সংখ্যা, গাণিতিক ফাংশন

৪১.১৩ `test`, `[ ]` ও `[[ ]]` — শর্ত যাচাই

স্ক্রিপ্টকে সিদ্ধান্ত নিতে হয়: "ফাইলটা আছে কি?", "ইউজার কি 'yes' লিখেছে?", "সংখ্যাটা ১০০-র বেশি কি?"। এই প্রশ্নের উত্তর দেয় `test` কমান্ড।

`test` আর `[` আসলে একই জিনিস

এটা অনেকে জানে না: `[` একটা **কমান্ড**, কোনো বিশেষ সিনট্যাক্স না!

```
$ type test [ [[
test is a shell builtin
[ is a shell builtin
[[ is a shell keyword
$ ls -l /usr/bin/[
-rwxr-xr-x 1 root root 51464 Apr  5  2024 /usr/bin/[
```

`[` আর `test` হুবহু একই কাজ করে; পার্থক্য শুধু `[` এর শেষে একটা `]` আর্গুমেন্ট বাধ্যতামূলক। এগুলো কিছু প্রিন্ট করে না, শুধু exit status দেয়: শর্ত সত্য হলে 0, মিথ্যা হলে 1।

```
$ test 5 -gt 3; echo $?
0
$ [ 5 -gt 3 ]; echo $?
0
$ [ 5 -gt 10 ]; echo $?
1
```

এই কারণেই স্পেস বাধ্যতামূলক। `[` একটা কমান্ডের নাম, আর `5`, `-gt`, `3`, `]` তার আলাদা আলাদা আর্গুমেন্ট। কমান্ড আর আর্গুমেন্টের মাঝে স্পেস তো লাগবেই:

```
$ [5 -gt 3]
[5: command not found          # bash '[5' নামের কমান্ড খুঁজছে
$ [ 5 -gt 3]
bash: [: missing `]'           # '3]' একটা আর্গুমেন্ট, আলাদা ']' নেই
$ [ 5 -gt 3 ]                  # ✔
```

`[[ ]]` — bash-এর উন্নত সংস্করণ

`[[ ]]` একটা bash **keyword** (shell নিজেই এর ভেতরের লেখা বিশেষভাবে পার্স করে, কমান্ড হিসেবে না)। তাই:

- ভেতরে ভেরিয়েবলে word splitting ও globbing হয় না, ফলে কোট ভুলে গেলেও সাধারণত ভাঙে না
- `&&`, `||` সরাসরি ভেতরে লেখা যায়
- `<`, `>` দিয়ে স্ট্রিং তুলনায় escape লাগে না
- `==` দিয়ে glob প্যাটার্ন মেলানো (`[[ $f == *.txt ]]`)
- `=~` দিয়ে regex মেলানো

```
$ name=""
$ [ $name = "admin" ] && echo yes
bash: [: =: unary operator expected     # $name খালি → [ = admin ] হয়ে গেছে
$ [ "$name" = "admin" ] && echo yes     # ✔ কোট দিলে ঠিক আছে
$ [[ $name = "admin" ]] && echo yes     # ✔ [[ ]]-এ কোট ছাড়াও ঠিক
```

> 📝 **মনে রাখো:** bash স্ক্রিপ্টে (`#!/bin/bash`) **`[[ ]]` ব্যবহার করো**, এটা নিরাপদ ও শক্তিশালী। `#!/bin/sh` POSIX স্ক্রিপ্টে শুধু `[ ]` চলবে, আর তখন ভেরিয়েবল অবশ্যই কোটে রাখবে। সংখ্যার জন্য `(( ))` সবচেয়ে পরিষ্কার।

ফাইল টেস্ট অপারেটর

অপারেটর	সত্য হবে যদি...
`-e file`	ফাইল/ডিরেক্টরি যা-ই হোক, **আছে** (exists)
`-f file`	আছে এবং সাধারণ ফাইল (regular file)
`-d file`	আছে এবং ডিরেক্টরি
`-L file` বা `-h file`	আছে এবং symbolic link (অধ্যায় ১১)
`-b file`	block device (যেমন `/dev/sda`)
`-c file`	character device (যেমন `/dev/null`)
`-p file`	named pipe (FIFO)
`-S file`	socket
`-r file`	তোমার **পড়ার** পারমিশন আছে
`-w file`	তোমার **লেখার** পারমিশন আছে
`-x file`	তোমার **চালানোর** পারমিশন আছে (ডিরেক্টরির ক্ষেত্রে ঢোকার)
`-s file`	আছে এবং সাইজ 0-র বেশি (খালি না)
`-u file`	setuid বিট আছে (অধ্যায় ১৭)
`-g file`	setgid বিট আছে
`-k file`	sticky বিট আছে
`-O file`	তুমি ফাইলের owner
`-G file`	ফাইলের group তোমার effective group
`-N file`	শেষবার পড়ার পরে পরিবর্তিত হয়েছে
`f1 -nt f2`	f1, f2-এর চেয়ে নতুন (newer than, modification time)
`f1 -ot f2`	f1, f2-এর চেয়ে পুরোনো (older than)
`f1 -ef f2`	দুটো একই ফাইল (একই device ও inode, যেমন হার্ড লিংক)

```
$ [[ -f /etc/passwd ]] && echo "সাধারণ ফাইল"
সাধারণ ফাইল
$ [[ -d /etc ]] && echo "ডিরেক্টরি"
ডিরেক্টরি
$ [[ -w /etc/shadow ]] || echo "লেখার অনুমতি নেই"
লেখার অনুমতি নেই
$ [[ -e /nonexistent ]]; echo $?
1
```

> ⚠️ **সতর্কতা:** symlink-এর ক্ষেত্রে `-e`, `-f`, `-d` লিংকটা যে ফাইলকে দেখায় (target) তাকে পরীক্ষা করে। ভাঙা symlink (target নেই) এর ক্ষেত্রে `-e` মিথ্যা, কিন্তু `-L` সত্য। root হিসেবে চালালে `-r` / `-w` প্রায় সবসময় সত্য, কারণ root পারমিশন এড়িয়ে যেতে পারে।

স্ট্রিং টেস্ট অপারেটর

অপারেটর	সত্য হবে যদি...	কোথায়
`-z str`	স্ট্রিং খালি (zero length)	`[ ]` ও `[[ ]]`
`-n str`	স্ট্রিং খালি না (non-zero)	`[ ]` ও `[[ ]]`
`s1 = s2`	দুটো সমান	`[ ]` ও `[[ ]]`
`s1 == s2`	সমান (`[[ ]]`-এ ডান দিক glob প্যাটার্ন)	bash-এর `[ ]` ও `[[ ]]`
`s1 != s2`	সমান না	`[ ]` ও `[[ ]]`
`s1 < s2`	অভিধানের ক্রমে s1 আগে আসে	`[[ ]]` (বা `[ ]`-এ `\<`)
`s1 > s2`	অভিধানের ক্রমে s1 পরে আসে	`[[ ]]` (বা `[ ]`-এ `\>`)
`str =~ regex`	স্ট্রিং extended regex-এর সাথে মেলে	শুধু `[[ ]]`

```
$ answer="yes"
$ [[ $answer == "yes" ]] && echo "রাজি!"
রাজি!
$ [[ -z $undefined_var ]] && echo "খালি"
খালি
$ file="report.pdf"
$ [[ $file == *.pdf ]] && echo "PDF ফাইল"       # glob প্যাটার্ন (কোট দেবে না)
PDF ফাইল
$ [[ $file == "*.pdf" ]] && echo "মিলেছে" || echo "মেলেনি"   # কোট দিলে আক্ষরিক
মেলেনি
$ [[ "apple" < "banana" ]] && echo "apple আগে"
apple আগে
```

Regex মেলানো (`=~`): মিলে গেলে ব্র্যাকেট `( )` দিয়ে ধরা অংশগুলো `BASH_REMATCH` অ্যারেতে থাকে।

```
$ email="rahim@example.com"
$ if [[ $email =~ ^([a-z0-9._]+)@([a-z0-9.-]+)$ ]]; then
>     echo "ইউজার: ${BASH_REMATCH[1]}, ডোমেইন: ${BASH_REMATCH[2]}"
```

```
> fi
ইউজার: rahim, ডোমেইন: example.com
```

⚠️ **সতর্কতা:** `=~`-এর ডান দিকের regex **কোটে রাখবে না**। কোট দিলে সেটা আক্ষরিক স্ট্রিং হিসেবে মেলানো হয়। জটিল regex হলে আগে একটা ভেরিয়েবলে রাখো: `re='^[0-9]+$'; [[ $x =~ $re ]]`। regex-এর বিস্তারিত অধ্যায় ২০-এ।

⚠️ **সতর্কতা:** `[ ]`-এর ভেতরে `<` বা `>` escape না করলে সেটা redirection হয়ে যায়! `[ "a" > "b" ]` চালালে `b` নামে একটা খালি ফাইল তৈরি হবে আর শর্ত সবসময় সত্য দেখাবে।

সংখ্যা তুলনা অপারেটর

`[ ]` / `[[ ]]` অপারেটর	`(( ))` রূপ	মানে
`-eq`	`==`	সমান (equal)
`-ne`	`!=`	সমান না (not equal)
`-lt`	`<`	ছোট (less than)
`-le`	`<=`	ছোট বা সমান (less or equal)
`-gt`	`>`	বড় (greater than)
`-ge`	`>=`	বড় বা সমান (greater or equal)

```
$ age=20
$ [ "$age" -ge 18 ] && echo "প্রাপ্তবয়স্ক"
প্রাপ্তবয়স্ক
$ [[ $age -lt 13 ]] || echo "শিশু না"
শিশু না
$ (( age >= 18 && age < 60 )) && echo "কর্মক্ষম বয়স"
কর্মক্ষম বয়স
```

⚠️ **সতর্কতা: সংখ্যা তুলনায় `<`, `>`, `==` ব্যবহার কোরো না `[[ ]]`-এ!** `[[ ]]`-এ এগুলো **স্ট্রিং** তুলনা করে:

```
$ [[ 10 > 9 ]] && echo "বড়" || echo "ছোট"
ছোট           # স্ট্রিং হিসেবে "1" < "9", তাই "10" < "9"!
$ [[ 10 -gt 9 ]] && echo "বড়" || echo "ছোট"
বড়            # ✔
$ (( 10 > 9 )) && echo "বড়" || echo "ছোট"
বড়            # ✔
```

> আর `-eq`, `-lt` এর মতো অপারেটরে সংখ্যা ছাড়া অন্য কিছু দিলে `[ ]` এরর দেয়: `[: abc: integer expression expected`।

লজিক্যাল অপারেটর (শর্ত জোড়া লাগানো)

কাজ	`[ ]` এর ভেতরে	`[[ ]]` এর ভেতরে	বাইরে (কমান্ড স্তরে)
NOT	`[ ! -f x ]`	`[[ ! -f x ]]`	`! [ -f x ]`
AND	`-a` (পুরোনো, এড়াও)	`&&`	`[ … ] && [ … ]`
OR	`-o` (পুরোনো, এড়াও)	`||`	`[ … ] || [ … ]`
গ্রুপ	`\( … \)`	`( … )`	`{ …; }`

```
$ f=/etc/passwd
$ [[ -f $f && -r $f ]] && echo "ফাইলটা পড়া যাবে"
ফাইলটা পড়া যাবে
$ [ -f "$f" ] && [ -r "$f" ] && echo "POSIX স্টাইলেও একই"
POSIX স্টাইলেও একই
$ [[ ! -d $f ]] && echo "এটা ডিরেক্টরি না"
এটা ডিরেক্টরি না
$ x=15
$ [[ ( $x -gt 10 && $x -lt 20 ) || $x -eq 100 ]] && echo "রেঞ্জে আছে"
রেঞ্জে আছে
```

> 💡 **টিপস:** `[ ]`-এর ভেতরের `-a` ও `-o` POSIX-এ অপ্রচলিত (obsolescent) ঘোষণা করা হয়েছে, কারণ কিছু ইনপুটে ভুল পার্স হয়। POSIX স্ক্রিপ্টে আলাদা `[ ] && [ ]` লেখো।

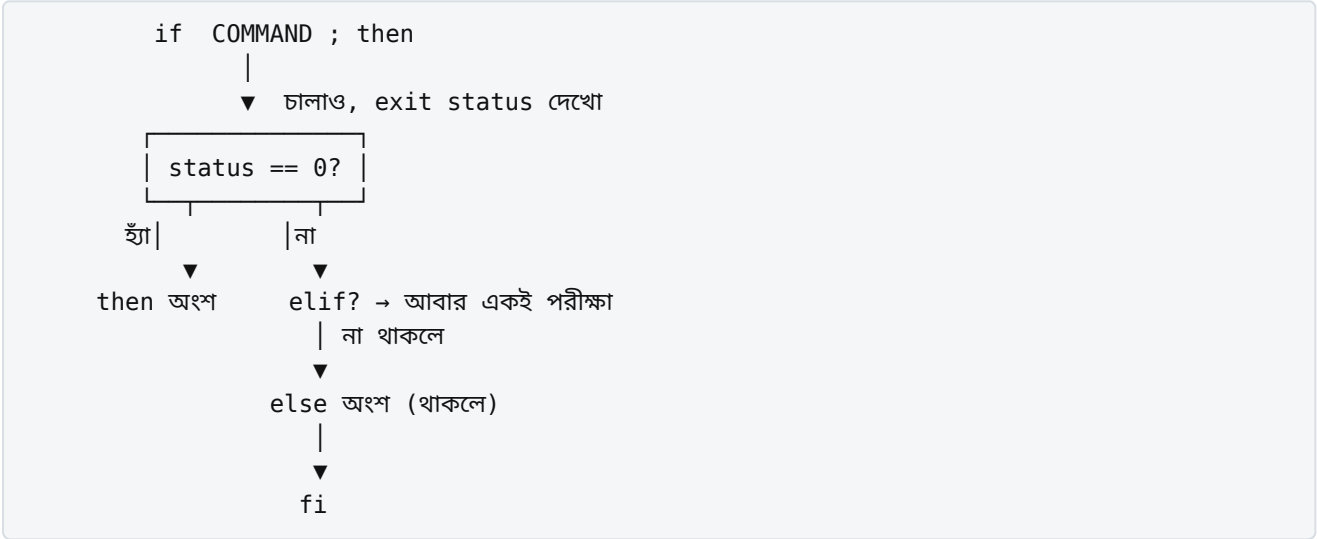
`[ ]` বনাম `[[ ]]` বনাম `(( ))` — তুলনা

বৈশিষ্ট্য	`[ ]` / `test`	`[[ ]]`	`(( ))`
ধরন	কমান্ড (builtin)	bash keyword	bash keyword
POSIX (`sh`/dash-এ চলে)	হ্যাঁ	না	না
ভেরিয়েবলে কোট লাগে?	**হ্যাঁ, অবশ্যই**	সাধারণত না	না
`&&` `||` ভেতরে	না	হ্যাঁ	হ্যাঁ
glob প্যাটার্ন (`==`)	না	হ্যাঁ	—
regex (`=~`)	না	হ্যাঁ	—
সেরা কাজ	portable স্ক্রিপ্ট	স্ট্রিং ও ফাইল টেস্ট	সংখ্যা তুলনা ও হিসাব

৪১.১৪ `if` / `elif` / `else`

`if` আসলে কী পরীক্ষা করে?

এটা bash-এর সবচেয়ে গুরুত্বপূর্ণ ধারণাগুলোর একটা: **`if` কোনো "শর্ত" পরীক্ষা করে না, একটা কমান্ড চালায় আর তার exit status দেখে।** exit status 0 হলে `then` অংশ চলে, অন্যথায় `else`। `[ ]`, `[[ ]]`, `(( ))` হলো exit status দেওয়া কমান্ড মাত্র। তাই যেকোনো কমান্ড `if`-এ বসানো যায়।



Syntax

```
if COMMANDS; then
    COMMANDS
elif COMMANDS; then
    COMMANDS
else
    COMMANDS
fi
```

- `fi` হলো `if` উল্টো করে লেখা, এটা ব্লক শেষ করে।
- `then` আলাদা লাইনে লিখলে `;` লাগে না। একই লাইনে লিখলে `;` লাগে।
- `elif` ও `else` ঐচ্ছিক; `elif` যতগুলো ইচ্ছা।
- `then`-এর পর অন্তত একটা কমান্ড থাকতে হবে; খালি রাখতে চাইলে `:` দাও।

উদাহরণ

১. সাধারণ if:

```
#!/bin/bash
if [[ -f /etc/os-release ]]; then
    source /etc/os-release
    echo "তোমার OS: $PRETTY_NAME"
fi
```

```
$ ./os.sh
তোমার OS: Ubuntu 24.04.1 LTS
```

২. if-else:

```
#!/bin/bash
read -rp "একটা সংখ্যা দাও: " n
if (( n % 2 == 0 )); then
    echo "$n জোড় সংখ্যা"
else
    echo "$n বিজোড় সংখ্যা"
fi
```

৩. if-elif-else (গ্রেড ক্যালকুলেটর):

```
#!/bin/bash
read -rp "নম্বর দাও (0-100): " marks

if ! [[ $marks =~ ^[0-9]+$ ]]; then
    echo "Error: শুধু সংখ্যা দাও" >&2
    exit 1
elif (( marks > 100 )); then
    echo "Error: 100-এর বেশি হতে পারে না" >&2
    exit 1
elif (( marks >= 80 )); then
    grade="A+"
elif (( marks >= 70 )); then
    grade="A"
elif (( marks >= 60 )); then
    grade="A-"
elif (( marks >= 50 )); then
    grade="B"
elif (( marks >= 40 )); then
    grade="C"
elif (( marks >= 33 )); then
    grade="D"
else
    grade="F"
fi
echo "তোমার গ্রেড: $grade"
```

```
$ ./grade.sh
নম্বর দাও (0-100): 75
তোমার গ্রেড: A
$ ./grade.sh
নম্বর দাও (0-100): abc
Error: শুধু সংখ্যা দাও
```

৪. সরাসরি কমান্ড দিয়ে if (`[ ]` ছাড়া):

```
if grep -q "^rahim:" /etc/passwd; then
    echo "rahim ইউজার আছে"
fi
```

```
if ping -c 1 -W 2 8.8.8.8 &> /dev/null; then
    echo "ইন্টারনেট চালু"
else
    echo "ইন্টারনেট নেই"
fi

if ! command -v docker &> /dev/null; then
    echo "docker ইনস্টল করা নেই"
fi

if mkdir /tmp/mylock 2>/dev/null; then
    echo "lock পেয়েছি"
fi
```

এখানে `grep -q` (quiet: কিছু প্রিন্ট করে না, শুধু status দেয়) একটা প্রচলিত কৌশল। `command -v` কোনো কমান্ড আছে কি না যাচাইয়ের নির্ভরযোগ্য উপায়।

> ⚠️ **সতর্কতা:** একটা খুব প্রচলিত ভুল:
>
> ```
> if [ grep -q root /etc/passwd ]; then    # ✘ ভুল!
> if grep -q root /etc/passwd; then        # ✔ সঠিক
> ```
>
> `[ ]` হলো টেস্ট করার কমান্ড; অন্য কমান্ডের সফলতা দেখতে সেটাকে `[ ]`-এ মোড়াবে না।

৫. Nested if (একটার ভেতরে আরেকটা):

```
if [[ -e $path ]]; then
    if [[ -d $path ]]; then
        echo "$path একটা ডিরেক্টরি"
    else
        echo "$path একটা ফাইল"
    fi
else
    echo "$path নেই"
fi
```

> 💡 **টিপস:** বেশি nesting কোড পড়া কঠিন করে। "guard clause" ব্যবহার করো: আগে ভুলের ক্ষেত্রগুলো চেক করে `exit` করো, তারপর মূল কাজ লেখো, যেমন উপরের গ্রেড স্ক্রিপ্টে।

এক লাইনে if

```
$ if [[ -d /tmp ]]; then echo "আছে"; else echo "নেই"; fi
আছে
```

৪১.১৫ `case` — একাধিক বিকল্প থেকে বাছাই

একটা ভেরিয়েবলকে অনেকগুলো সম্ভাব্য মানের সাথে মেলাতে হলে `if/elif` লম্বা ও অগোছালো হয়ে যায়। তখন `case` ব্যবহার করো।

উপমা: লিফটের বাটন প্যানেল। তুমি যে বাটন চাপবে (মান), লিফট সেই তলায় যাবে (সেই ব্লক চলবে)। কোনো বাটন না মিললে "ভুল ইনপুট" (`*`)।

Syntax

```
case WORD in
    PATTERN1)
        COMMANDS
        ;;
    PATTERN2 | PATTERN3)
        COMMANDS
        ;;
    *)
        DEFAULT COMMANDS
        ;;
esac
```

- `esac` হলো `case` উল্টো।
- প্রতিটা প্যাটার্ন `)` দিয়ে শেষ হয়; চাইলে শুরুতে `(` দেওয়া যায়: `(start)`।
- `;;` ব্লক শেষ করে আর `case` থেকে বের হয়ে যায়।
- প্যাটার্নগুলো **উপর থেকে নিচে** মেলানো হয়, প্রথম যেটা মেলে সেটাই চলে।
- প্যাটার্ন হলো **glob** (regex না): `*` যেকোনো কিছু, `?` একটা অক্ষর, `[abc]` এদের যেকোনো একটা, `|` মানে "অথবা"।

প্যাটার্ন টেবিল

প্যাটার্ন	মেলে
`start`	হুবহু "start"
`start | begin`	"start" অথবা "begin"
`[yY]`	"y" অথবা "Y"
`[yY][eE][sS]`	yes, YES, Yes, yEs ...
`*.txt`	.txt দিয়ে শেষ
`[0-9]`	একটা অঙ্ক
`?`	ঠিক একটা যেকোনো অক্ষর
`*`	যেকোনো কিছু (ডিফল্ট, সবশেষে রাখো)

উদাহরণ ১: Yes/No প্রশ্ন

```
#!/bin/bash
read -rp "ফাইলগুলো মুছে ফেলব? (yes/no) " ans
case $ans in
    [yY] | [yY][eE][sS])
        echo "মুছে ফেলছি..."
        ;;
    [nN] | [nN][oO])
        echo "বাতিল করা হলো"
        ;;
    *)
        echo "বুঝিনি, yes বা no লেখো" >&2
        exit 1
        ;;
esac
```

উদাহরণ ২: সার্ভিস কন্ট্রোল স্ক্রিপ্ট (পুরোনো init স্টাইল)

```
#!/bin/bash
# myapp.sh
case "$1" in
    start)
        echo "myapp চালু হচ্ছে..."
        ;;
    stop)
        echo "myapp বন্ধ হচ্ছে..."
        ;;
    restart)
        "$0" stop
        "$0" start
        ;;
    status)
        echo "myapp চলছে"
        ;;
    *)
        echo "ব্যবহার: $0 {start|stop|restart|status}" >&2
        exit 2
        ;;
esac
```

```
$ ./myapp.sh restart
myapp বন্ধ হচ্ছে...
myapp চালু হচ্ছে...
$ ./myapp.sh foo
ব্যবহার: ./myapp.sh {start|stop|restart|status}
```

এক সময় `/etc/init.d/`-এর সব সার্ভিস স্ক্রিপ্ট ঠিক এভাবেই লেখা হতো। এখন systemd এই কাজ করে (অধ্যায় ২৬)।

উদাহরণ ৩: ফাইলের ধরন অনুযায়ী কাজ

```
#!/bin/bash
file="$1"
case "$file" in
    *.tar.gz | *.tgz) tar -xzf "$file" ;;
    *.tar.bz2)        tar -xjf "$file" ;;
    *.tar.xz)         tar -xJf "$file" ;;
    *.zip)            unzip "$file" ;;
    *.gz)             gunzip "$file" ;;
    *)                echo "অচেনা ফরম্যাট: $file" >&2; exit 1 ;;
esac
```

খেয়াল করো, `*.tar.gz` কে `*.gz`-এর **আগে** রাখা হয়েছে; উল্টো রাখলে `.tar.gz` ফাইলও `*.gz`-এ মিলে যেত। আর্কাইভ কমান্ডগুলোর বিস্তারিত অধ্যায় ৩৪-এ।

`;;`, `;&` ও `;;&` — ব্লকের শেষ

টার্মিনেটর	আচরণ
`;;`	এই ব্লক চালিয়ে `case` থেকে বের হও (স্বাভাবিক)
`;&`	পরের ব্লকটাও চালাও, তার প্যাটার্ন না মিললেও (fall-through, bash 4+)
`;;&`	পরের প্যাটার্নগুলোও পরীক্ষা করতে থাকো, যেগুলো মেলে সেগুলোও চালাও (bash 4+)

```
#!/bin/bash
char="$1"
case "$char" in
    [0-9])     echo "অঙ্ক" ;;&
    [0-4])     echo "ছোট অঙ্ক (0-4)" ;;
    [a-z])     echo "ছোট হাতের অক্ষর" ;;
esac
```

```
$ ./chk.sh 3
অঙ্ক
ছোট অঙ্ক (0-4)
$ ./chk.sh 7
অঙ্ক
```

> ⚠️ **সতর্কতা:** `[a-z]` এর মতো রেঞ্জ locale-এর উপর নির্ভর করে কখনো কখনো বড় হাতের অক্ষরও মেলাতে পারে। নিশ্চিত হতে `[[:lower:]]`, `[[:upper:]]`, `[[:digit:]]` এর মতো POSIX character class ব্যবহার করো।

> 💡 **টিপস:** `case` এ `shopt -s nocasematch` চালু করলে বড়-ছোট হাতের পার্থক্য উপেক্ষা হয়, তখন `yes)` লিখলেই YES, Yes সব মিলবে। এটা `[[ == ]]`-এও প্রযোজ্য।

৪১.১৬ `&&` ও `||` — শর্টকাট শর্ত

এগুলো দুটো কমান্ডকে exit status দিয়ে জোড়া দেয়:

অপারেটর	মানে	উদাহরণ
`cmd1 && cmd2`	cmd1 **সফল** হলে তবেই cmd2 চালাও	`mkdir d && cd d`
`cmd1 || cmd2`	cmd1 **ব্যর্থ** হলে তবেই cmd2 চালাও	`cd d || exit 1`
`cmd1 ; cmd2`	যা-ই হোক, পরপর দুটোই চালাও	`cd /tmp; ls`
`! cmd`	cmd-এর exit status উল্টে দাও	`! grep -q x f`

উপমা: `&&` হলো "যদি পরীক্ষায় পাশ করো, *তাহলে* পার্টি"। `||` হলো "যদি বাস না পাও, *তাহলে* রিকশা নাও"।

এই আচরণের নাম **short-circuit evaluation**: `&&`-এর বাম দিক ব্যর্থ হলে ডান দিক আর দেখার দরকার নেই (ফল এমনিতেই ব্যর্থ), `||`-এর বাম দিক সফল হলে ডান দিক দেখার দরকার নেই।

```
$ mkdir -p ~/project && cd ~/project && echo "রেডি!"
রেডি!
$ cd /nonexistent || echo "ডিরেক্টরি পাওয়া যায়নি"
bash: cd: /nonexistent: No such file or directory
ডিরেক্টরি পাওয়া যায়নি
$ sudo apt update && sudo apt upgrade -y        # update সফল হলেই upgrade
$ [[ -f config.ini ]] || cp config.example.ini config.ini   # না থাকলে বানাও
```

স্ক্রিপ্টে প্রচলিত প্যাটার্ন

```
# প্রয়োজনীয় কমান্ড না থাকলে থামো
command -v curl > /dev/null || { echo "curl দরকার" >&2; exit 1; }

# root হিসেবে চালাতে হবে
[[ $EUID -eq 0 ]] || { echo "sudo দিয়ে চালাও" >&2; exit 1; }

# cd ব্যর্থ হলে আর এগিও না (খুব গুরুত্বপূর্ণ!)
cd /var/www/app || exit 1
rm -rf ./cache/*
```

শেষ উদাহরণটা ভাবো: `cd` ব্যর্থ হলে আর `|| exit` না থাকলে, `rm -rf ./cache/*` তোমার বর্তমান ডিরেক্টরিতে চলবে, ভুল জায়গার ফাইল মুছে যাবে!

`{ ...; }` হলো **command group**: কয়েকটা কমান্ডকে একটা ইউনিট হিসেবে ধরে। খেয়াল করো, `{` এর পরে স্পেস আর `}` এর আগে `;` (বা newline) বাধ্যতামূলক। `( ... )` ও গ্রুপ করে, কিন্তু সেটা subshell-এ চলে, তাই ভেতরের `exit` শুধু subshell থেকে বের হয়, স্ক্রিপ্ট থেকে না।

⚠️ `A && B || C` ফাঁদ

অনেকে `A && B || C` কে if-then-else ভাবে। কিন্তু এটা এক না:

```
$ [[ -f /etc/hostname ]] && grep -q nomatch /etc/hostname || echo "ফাইল নেই"
ফাইল নেই            # ✘ ভুল বার্তা! ফাইল আছে, কিন্তু grep ব্যর্থ হয়েছে
```

`C` চলে যদি `A` ব্যর্থ হয় **অথবা** `B` ব্যর্থ হয়। `B` সবসময় সফল হলে (যেমন `echo`) সমস্যা নেই, কিন্তু নিশ্চিত না হলে আসল `if` লেখো।

```
A && B || C

A সফল → B চালাও ─┬─ B সফল → শেষ (C চলবে না)
                  └─ B ব্যর্থ → C চালাও   ← অপ্রত্যাশিত!
A ব্যর্থ ──────────────────────→ C চালাও
```

অগ্রাধিকার (precedence)

`&&` আর `||` এর অগ্রাধিকার **সমান**, আর বাম থেকে ডানে হিসাব হয়। `;` আর newline এদের চেয়ে দুর্বল। তাই `a || b && c` মানে `(a || b) && c`। সন্দেহ থাকলে `{ }` দিয়ে গ্রুপ করো।

৪১.১৭ স্ক্রিপ্ট ডিবাগ করা

পদ্ধতি	কাজ
`bash -n script.sh`	শুধু syntax চেক, কিছু চালায় না
`bash -x script.sh`	প্রতিটা কমান্ড expand হওয়ার পর `+` দিয়ে দেখায়, তারপর চালায়
`bash -v script.sh`	প্রতিটা লাইন পড়ার সাথে সাথে হুবহু দেখায়
`set -x` / `set +x`	স্ক্রিপ্টের ভেতরে নির্দিষ্ট অংশে trace চালু/বন্ধ
`PS4='+ ${LINENO}: '`	trace লাইনের শুরুতে লাইন নম্বর দেখাও
`shellcheck script.sh`	স্ট্যাটিক বিশ্লেষণ: কোট ভুল, bashism, ফাঁদ ধরে দেয়

```
$ cat calc.sh
#!/bin/bash
a=5
b=$1
echo "যোগফল: $(( a + b ))"
$ bash -x calc.sh 3
+ a=5
+ b=3
+ echo 'যোগফল: 8'
যোগফল: 8
```

ShellCheck স্ক্রিপ্ট লেখার সবচেয়ে ভালো বন্ধু। ইনস্টল: `sudo apt install shellcheck` (Ubuntu/Debian) বা `sudo dnf install ShellCheck` (Fedora; RHEL-এ EPEL repo লাগে)।

```
$ cat bad.sh
#!/bin/bash
file=$1
rm $file
$ shellcheck bad.sh

In bad.sh line 3:
rm $file
   ^---^ SC2086 (info): Double quote to prevent globbing and word splitting.

Did you mean:
rm "$file"
...
```

> 💡 **টিপস:** VS Code-এ "ShellCheck" এক্সটেনশন বা vim-এ ALE প্লাগইন দিলে লেখার সময়েই ভুল দেখাবে।

৪১.১৮ সম্পর্কিত কীবোর্ড শর্টকাট

স্ক্রিপ্ট লেখা ও চালানোর সময় এই শর্টকাটগুলো কাজে লাগে (সব টার্মিনাল শর্টকাট অধ্যায় ৬-এ)।

শর্টকাট	কাজ
`Ctrl+C`	চলমান স্ক্রিপ্ট থামাও (SIGINT পাঠায়, exit status 130)
`Ctrl+D`	`read` অপেক্ষা করার সময় EOF পাঠাও (`read` ব্যর্থ হয়, status 1)
`Ctrl+Z`	স্ক্রিপ্ট সাময়িক থামাও (suspend), পরে `fg` দিয়ে ফেরাও (অধ্যায় ২৫)
`Ctrl+\`	SIGQUIT পাঠাও (`Ctrl+C` কাজ না করলে)
`Ctrl+X Ctrl+E`	বর্তমান কমান্ড লাইন `$EDITOR` -এ খোলো, বড় এক-লাইনার লিখতে সুবিধা
`Alt+.`	আগের কমান্ডের শেষ আর্গুমেন্ট বসাও (যেমন `nano s.sh` এর পর `chmod +x Alt+.`)
`!!`	আগের পুরো কমান্ড (যেমন `sudo !!`)
`!$`	আগের কমান্ডের শেষ আর্গুমেন্ট
`Ctrl+R`	হিস্ট্রিতে খোঁজো (যেমন আগে চালানো `./deploy.sh ...`)
`Tab`	ফাইলের নাম auto-complete (`./he` + Tab → `./hello.sh`)
`Ctrl+L`	স্ক্রিন পরিষ্কার
`Ctrl+S` / `Ctrl+Q`	টার্মিনাল আউটপুট ফ্রিজ / আবার চালু (ভুলে `Ctrl+S` চাপলে টার্মিনাল "আটকে" যায়, `Ctrl+Q` চাপো)

nano-তে: `Ctrl+O` সেভ, `Ctrl+X` বের হওয়া, `Alt+3` সিলেক্ট করা লাইন কমেন্ট/আনকমেন্ট। vim-এ: `:w` সেভ, `:!bash %` বর্তমান ফাইল চালাও, `:!shellcheck %` চেক করো।

৪১.১৯ একটা পূর্ণ উদাহরণ: সিস্টেম তথ্য স্ক্রিপ্ট

এই অধ্যায়ের প্রায় সব কিছু একসাথে:

```
#!/usr/bin/env bash
# sysinfo.sh — সিস্টেমের সংক্ষিপ্ত রিপোর্ট দেয়
# ব্যবহার: ./sysinfo.sh [disk-সতর্কতার সীমা %]   (ডিফল্ট 80)

readonly THRESHOLD="${1:-80}"

# ইনপুট যাচাই
if ! [[ $THRESHOLD =~ ^[0-9]+$ ]] || (( THRESHOLD > 100 )); then
    echo "Error: সীমা 0-100 এর মধ্যে একটা সংখ্যা হতে হবে" >&2
    exit 2
fi

host=$(hostname)
kernel=$(uname -r)
uptime_info=$(uptime -p)
mem_total=$(free -m | awk '/^Mem:/ {print $2}')
mem_used=$(free -m | awk '/^Mem:/ {print $3}')
mem_pct=$(echo "scale=1; $mem_used * 100 / $mem_total" | bc)
disk_pct=$(df --output=pcent / | tail -1 | tr -dc '0-9')

printf "%-12s: %s\n" "Host" "$host"
printf "%-12s: %s\n" "Kernel" "$kernel"
printf "%-12s: %s\n" "Uptime" "$uptime_info"
printf "%-12s: %s MB / %s MB (%s%%)\n" "Memory" "$mem_used" "$mem_total" "$mem_pct"
printf "%-12s: %s%%\n" "Disk (/)" "$disk_pct"

if (( disk_pct >= THRESHOLD )); then
    echo "⚠️  ডিস্ক ${disk_pct}% ভরা, সীমা ${THRESHOLD}%!" >&2
    exit 1
fi

echo "✔ সব ঠিক আছে"
```

```
$ chmod +x sysinfo.sh
$ ./sysinfo.sh
Host        : rahim-laptop
Kernel      : 6.8.0-48-generic
Uptime      : up 3 hours, 12 minutes
Memory      : 5230 MB / 15890 MB (32.9%)
Disk (/)    : 47%
✔ সব ঠিক আছে
$ ./sysinfo.sh 40; echo "exit=$?"
...
⚠️  ডিস্ক 47% ভরা, সীমা 40%!
exit=1
```

(`awk`, `tr`, `df` ইত্যাদির বিস্তারিত অধ্যায় ২২, ২৩, ৩২-এ।)

হাতে-কলমে অনুশীলন

ল্যাব ১: স্ক্রিপ্ট চালানোর চার উপায়

```
$ mkdir -p ~/lab41 && cd ~/lab41
$ printf '#!/bin/bash\necho "PID: $$, shell: $0"\nX=changed\n' > run.sh
$ cat run.sh
$ echo "আমার shell-এর PID: $$"
$ bash run.sh             # x ছাড়াই চলে, ভিন্ন PID
$ ./run.sh                # Permission denied
$ chmod +x run.sh && ./run.sh     # ভিন্ন PID
$ X=original; source run.sh; echo "X=$X"    # একই PID, X বদলে গেছে
```

লক্ষ্য করো: কোনটায় PID তোমার shell-এর সমান হলো, আর কোনটার পরে `X` বদলে গেল।

ল্যাব ২: কোটিং পরীক্ষা

```
$ name="Linux Book"
$ echo $name; echo "$name"; echo '$name'; echo "\$name = $name"
$ touch "a b.txt"
$ f="a b.txt"
$ ls $f        # কী এরর দেয়? কেন?
$ ls "$f"
$ echo *.txt "*.txt" '*.txt'
```

প্রতিটা লাইনের আউটপুট আগে অনুমান করো, তারপর চালিয়ে মিলিয়ে দেখো।

ল্যাব ৩: ক্যালকুলেটর

`calc.sh` লেখো যা তিনটা আর্গুমেন্ট নেয়: সংখ্যা, অপারেটর, সংখ্যা। শর্ত:

- আর্গুমেন্ট ৩টা না হলে ব্যবহারবিধি দেখিয়ে `exit 2`
- `case` দিয়ে `+ - x /` সামলাও (`*` shell-এ glob, তাই গুণের জন্য `x` নাও)
- ভাগের সময় `bc` দিয়ে দশমিকের পর ২ ঘর পর্যন্ত দেখাও, আর শূন্য দিয়ে ভাগ আটকাও

▼ একটা সমাধান

```
#!/bin/bash
if (( $# != 3 )); then
    echo "ব্যবহার: $0 <সংখ্যা> <+|-|x|/> <সংখ্যা>" >&2
    exit 2
fi
a=$1 op=$2 b=$3
case $op in
    +) echo $(( a + b )) ;;
    -) echo $(( a - b )) ;;
    x) echo $(( a * b )) ;;
    /)
        [[ $b -eq 0 ]] && { echo "Error: শূন্য দিয়ে ভাগ" >&2; exit 1; }
```

```
        echo "scale=2; $a / $b" | bc
        ;;
    *) echo "অচেনা অপারেটর: $op" >&2; exit 2 ;;
esac
```

```
$ ./calc.sh 10 / 3
3.33
$ ./calc.sh 7 x 6
42
$ ./calc.sh 5 / 0; echo $?
Error: শূন্য দিয়ে ভাগ
1
```

ল্যাব ৪: ফাইল পরীক্ষক

`fileinfo.sh <path>` লেখো, যা বলবে path-টা নেই / সাধারণ ফাইল / ডিরেক্টরি / symlink / device, আর তোমার r, w, x পারমিশন কোনগুলো আছে। পরীক্ষা করো: `/etc/passwd`, `/etc`, `/dev/null`, `/bin` (Ubuntu-তে এটা symlink), `/etc/shadow`।

ল্যাব ৫: ইন্টারঅ্যাকটিভ ব্যাকআপ

একটা স্ক্রিপ্ট লেখো যা:

1. `read -e -i "$HOME/Documents" -p` দিয়ে ব্যাকআপের সোর্স ডিরেক্টরি জিজ্ঞেস করবে
2. ডিরেক্টরি না থাকলে `exit 1`
3. `read -n 1` দিয়ে নিশ্চিত হবে (y/n)
4. `tar -czf "backup-$(date +%F).tar.gz" ...` দিয়ে আর্কাইভ বানাবে, আর `&&` / `||` দিয়ে সফলতা বা ব্যর্থতার বার্তা দেবে

ল্যাব ৬: ডিবাগ

```
$ cat > buggy.sh << 'EOF'
#!/bin/bash
count = 5
if [$count -gt 3]
then
  echo "big"
fi
EOF
$ bash -n buggy.sh
$ bash -x buggy.sh
$ shellcheck buggy.sh
```

ভুলগুলো খুঁজে ঠিক করো (ইঙ্গিত: দুটো ভুল আছে, দুটোই স্পেস সংক্রান্ত)।

সারসংক্ষেপ

বিষয়	সিনট্যাক্স / কমান্ড	মনে রাখার কথা
Shebang	`#!/bin/bash`, `#!/usr/bin/env bash`	প্রথম লাইনের একদম শুরুতে; kernel পড়ে
Executable	`chmod +x s.sh`	`./` দিয়ে চালাতে r+x লাগে
চালানো	`./s.sh`, `bash s.sh`	নতুন process, পরিবর্তন হারিয়ে যায়
বর্তমান shell-এ	`source s.sh`, `. s.sh`	পরিবর্তন থেকে যায়; `exit` টার্মিনাল বন্ধ করে
কমেন্ট	`# ...`, `: << 'X' ... X`	শব্দের শুরুতে, কোটের বাইরে
ভেরিয়েবল	`v=val`, `"$v"`, `"${v}x"`	`=` এর পাশে স্পেস না; সবসময় কোট
ডিফল্ট	`${v:-def}`, `${v:?msg}`	খালি/unset সামলানো
কোটিং	`'literal'`, `"expand $v"`, `\$`	single = কিছুই expand না
Command subst.	`$(cmd)`	ব্যাকটিকের বদলে
ইনপুট	`read -rp "msg: " v`	`-s` পাসওয়ার্ড, `-t` টাইমআউট, `-n` অক্ষর
আর্গুমেন্ট	`$0 $1..$9 ${10} $# "$@" "$*"`	সবসময় `"$@"`
সরানো	`shift`, `shift N`, `set -- a b`	`$1` ফেলে বাঁয়ে সরায়
Exit status	`$?`, `exit N`	0 = সফল; 126/127/130
হিসাব	`$((a+b))`, `((i++))`, `let`	শুধু integer
দশমিক	`echo "scale=2; 10/3" | bc`	`-l` math library
টেস্ট	`[ ]`, `[[ ]]`, `(( ))`	`[` একটা কমান্ড, স্পেস বাধ্যতামূলক
ফাইল টেস্ট	`-e -f -d -L -r -w -x -s -nt -ot`	
স্ট্রিং টেস্ট	`-z -n = == != < > =~`	`[[ ]]`-এ `<`, `>` স্ট্রিং তুলনা
সংখ্যা টেস্ট	`-eq -ne -lt -le -gt -ge`	বা `(( a < b ))`
শর্ত	`if …; then …; elif …; else …; fi`	if একটা কমান্ডের exit status দেখে
বাছাই	`case $v in pat) …;; *) …;; esac`	glob প্যাটার্ন, উপর থেকে নিচে
শর্টকাট	`a && b`, `a || b`, `! a`	`a && b || c` ≠if-else
ডিবাগ	`bash -n`, `bash -x`, `set -x`, `shellcheck`	
থামানো	`Ctrl+C` (130), `Ctrl+D` (EOF)	

নিজেকে যাচাই করো

1. `./script.sh` চালালে kernel কীভাবে বোঝে কোন interpreter দিয়ে ফাইলটা চালাতে হবে?
2. `./setenv.sh` চালানোর পর তার সেট করা ভেরিয়েবল তোমার টার্মিনালে নেই, কেন? কীভাবে চালালে থাকবে?

3. `name = "Rahim"` লিখলে কী হয় এবং কেন?
4. `msg='Hello $USER'` আর `msg="Hello $USER"` এর `echo "$msg"` আউটপুটে পার্থক্য কী?
5. `./s.sh a "b c" d` চালালে `$#`, `$2`, আর `"$*"` এর মান কী হবে?
6. একটা কমান্ড চালানোর পর `$?` তে 127 পেলে কী বুঝবে? 130 পেলে?
7. `echo $(( 7 / 2 ))` এর আউটপুট কী? 3.5 পেতে কী করবে?
8. `[[ 10 > 9 ]]` মিথ্যা হয় কেন? সঠিক উপায় দুটো লেখো।
9. `[ -e file ]` আর `[ -f file ]` এর পার্থক্য কী?
10. `cd /data && rm -rf old || echo "failed"` এ "failed" কখন কখন প্রিন্ট হবে?

▼ **উত্তর দেখো**

1. kernel `execve()` করার সময় ফাইলের প্রথম দুই বাইট পড়ে। `#!` পেলে সেই লাইনের বাকি অংশকে interpreter-এর path ধরে (যেমন `/bin/bash`) এবং আসলে `/bin/bash ./script.sh` চালায়। তার আগে ফাইলের x পারমিশন থাকতে হবে।
2. `./setenv.sh` একটা নতুন child process-এ চলে। child-এর ভেরিয়েবল শেষ হওয়ার সাথে সাথে হারিয়ে যায়, parent-এ ফেরত আসে না। `source setenv.sh` বা `. setenv.sh` দিয়ে চালালে বর্তমান shell-এই চলবে, তাই ভেরিয়েবল থেকে যাবে।
3. bash স্পেস দিয়ে লাইন ভেঙে প্রথম শব্দ `name` কে কমান্ড ভাবে, আর `=` ও `Rahim` তার আর্গুমেন্ট। ফল: `name: command not found`। assignment-এ `=` এর পাশে স্পেস থাকা চলবে না: `name="Rahim"`।
4. single quote-এ কিছু expand হয় না, তাই প্রথমটা দেখাবে `Hello $USER`। double quote-এ ভেরিয়েবল expand হয়, তাই দ্বিতীয়টা দেখাবে `Hello rahim` (তোমার ইউজারনেম)।
5. `$#` = 3, `$2` = `b c`, `"$*"` = `a b c d` (একটাই স্ট্রিং)।
6. 127 = কমান্ড পাওয়া যায়নি (টাইপো বা PATH-এ নেই)। 130 = 128+2, অর্থাৎ SIGINT (`Ctrl+C`) দিয়ে থামানো হয়েছে।
7. আউটপুট `3`, কারণ bash শুধু integer হিসাব করে আর দশমিক অংশ কেটে ফেলে। 3.5 পেতে: `echo "scale=1; 7/2" | bc` বা `echo "7/2" | bc -l`, অথবা `awk 'BEGIN{print 7/2}'`।
8. `[[ ]]`-এর ভেতরে `>` স্ট্রিং (অভিধানের ক্রম) তুলনা করে; "1" অক্ষর "9"-এর আগে আসে, তাই "10" < "9"। সঠিক: `[[ 10 -gt 9 ]]` বা `(( 10 > 9 ))`।
9. `-e` সত্য হয় যদি path-টা যেকোনো ধরনের হিসেবে থাকে (ফাইল, ডিরেক্টরি, device ইত্যাদি)। `-f` সত্য হয় শুধু সাধারণ ফাইল (regular file) হলে। ডিরেক্টরির জন্য `-e` সত্য, `-f` মিথ্যা।
10. দুই ক্ষেত্রে: (ক) `cd /data` ব্যর্থ হলে (তখন `rm` চলবেই না), অথবা (খ) `cd` সফল কিন্তু `rm -rf old` ব্যর্থ হলে। `A && B || C` তে A বা B যেকোনোটা ব্যর্থ হলেই C চলে।

অধ্যায় ৪২: শেল স্ক্রিপ্টিং — পর্ব ২ (লুপ, ফাংশন, অ্যারে)

অধ্যায় ৪১-এ তুমি স্ক্রিপ্টের ভিত্তি শিখেছ: shebang, variable, `read`, `if` / `case`, `test` / `[[ ]]`, exit status। কিন্তু এখন পর্যন্ত তোমার স্ক্রিপ্ট একটা কাজ **একবার** করে। আসল শক্তি আসে যখন স্ক্রিপ্ট একই কাজ **বারবার** করতে পারে (লুপ), কাজগুলোকে ছোট ছোট **নামওয়ালা টুকরো**তে ভাগ করতে পারে (ফাংশন), আর অনেকগুলো ডেটা **একসাথে** ধরে রাখতে পারে (অ্যারে)। এই অধ্যায়ে আমরা ঠিক এই তিনটে জিনিস, সাথে স্ট্রিং কাটাছেঁড়া, `select` মেনু আর `printf` শিখব।

এই অধ্যায়ে যা শিখবে

- `for` লুপের সব রূপ: লিস্ট, range `{1..10}`, `seq`, C-style `for ((...))`, ফাইল/glob-এর উপর, আর্গুমেন্টের উপর
- `while` আর `until` লুপ, অসীম লুপ, কাউন্টার
- ফাইল লাইন-বাই-লাইন নিরাপদে পড়া: `while IFS= read -r line` — আর কেন প্রতিটা অংশ দরকার
- পাইপ + `while` এর "হারিয়ে যাওয়া variable" ফাঁদ আর তার সমাধান
- `break`, `continue` (আর `break 2` এর মতো nested রূপ)
- ফাংশন: ডিফাইন করা, আর্গুমেন্ট পাঠানো, `return` বনাম `echo`, `local` variable, nameref
- Indexed array আর associative array (`declare -A`) — তৈরি, পড়া, যোগ, মোছা, লুপ
- `mapfile` / `readarray` দিয়ে ফাইল থেকে অ্যারে
- Parameter expansion দিয়ে স্ট্রিং অপারেশন: দৈর্ঘ্য, substring, replace, prefix/suffix কাটা, ডিফল্ট মান, case বদল
- `select` দিয়ে মেনু তৈরি
- `printf` দিয়ে সুন্দর, ফরম্যাট করা আউটপুট

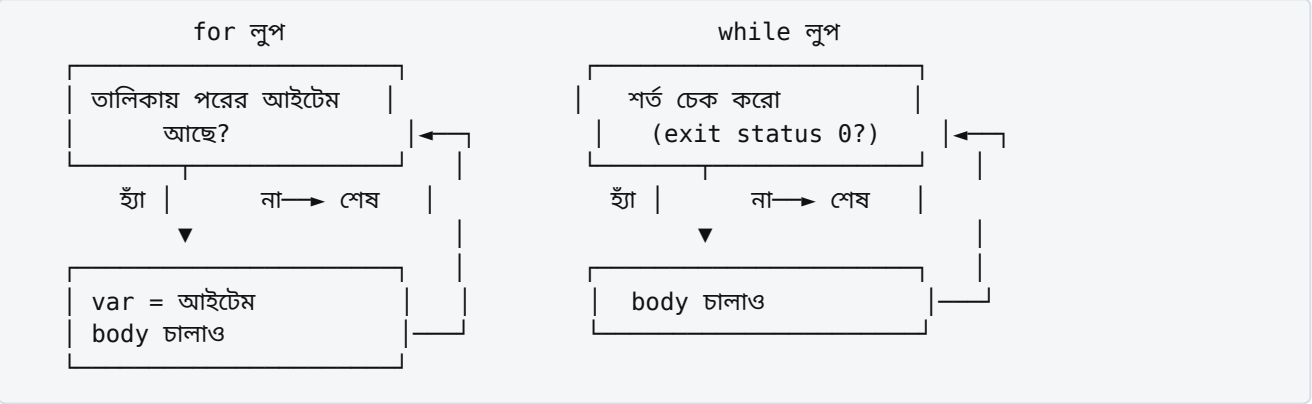
> 📝 **মনে রাখো:** এই অধ্যায়ের সবকিছু **bash** এর জন্য (Ubuntu 24.04-এ bash 5.2, Debian 12-এ 5.2, Fedora/RHEL 9-এ 5.1/5.2)। Array, `[[ ]]`, `{1..10}`, `declare -A` — এগুলো POSIX `sh` (যেমন Debian/Ubuntu-র `dash`) এ **নেই**। তাই স্ক্রিপ্টের শুরুতে অবশ্যই `#!/usr/bin/env bash` বা `#!/bin/bash` লিখবে, `#!/bin/sh` না।

---

১. লুপ কেন দরকার?

ধরো তোমাকে ১০০টা ছবির নাম বদলাতে হবে। হাতে ১০০ বার `mv` লেখা যায়, কিন্তু সেটা বোকামি। লুপ মানে: "এই কাজটা, এই তালিকার প্রতিটা জিনিসের জন্য, একবার করে করো।"

উপমা: তুমি একজন শিক্ষক, হাতে ৪০টা খাতা। তুমি প্রতিটা খাতা তোলো → নম্বর দাও → নামিয়ে রাখো → পরেরটা তোলো। খাতা শেষ হলে থামো। এটাই `for` লুপ। আর "যতক্ষণ চায়ের কাপে চা আছে, চুমুক দাও" — এটা `while` লুপ (শর্ত সত্য থাকা পর্যন্ত চলে)।



---

২. `for` লুপ

২.১ সাধারণ গঠন (list-based)

```
for VARIABLE in ITEM1 ITEM2 ITEM3 ...
do
    # প্রতিটা ITEM এর জন্য এখানকার কমান্ড চলবে
    echo "$VARIABLE"
done
```

এক লাইনে লিখতে চাইলে `;` দিয়ে আলাদা করো:

```
for VARIABLE in ITEM1 ITEM2; do echo "$VARIABLE"; done
```

উদাহরণ:

```
$ for fruit in আম কাঁঠাল লিচু; do echo "আমার প্রিয় ফল: $fruit"; done
আমার প্রিয় ফল: আম
আমার প্রিয় ফল: কাঁঠাল
আমার প্রিয় ফল: লিচু
```

ভেতরে কী হয়? bash প্রথমে `in` এর পরের অংশটা পুরো **expand** করে (variable, glob, brace, command substitution সব), তারপর ফলাফলকে শব্দে ভাগ করে (word splitting) একটা তালিকা বানায়। তারপর একে

একে প্রতিটা শব্দ `fruit` variable-এ বসিয়ে body চালায়। অর্থাৎ তালিকাটা **লুপ শুরুর আগেই একবারে** তৈরি হয়ে যায়।

> ⚠️ **সতর্কতা:** `for f in $(ls)` লিখবে না! `ls` এর আউটপুট word splitting-এর শিকার হয় — `my file.txt` নামের ফাইল দুটো আলাদা আইটেম `my` আর `file.txt` হয়ে যাবে। ফাইলের উপর লুপের জন্য সবসময় glob ব্যবহার করো: `for f in *`।

২.২ Range: `{1..10}` (brace expansion)

```
$ for i in {1..5}; do echo "সংখ্যা $i"; done
সংখ্যা 1
সংখ্যা 2
সংখ্যা 3
সংখ্যা 4
সংখ্যা 5
```

Brace expansion এর আরও রূপ:

লেখা	ফলাফল	মন্তব্য
`{1..5}`	`1 2 3 4 5`	১ থেকে ৫
`{5..1}`	`5 4 3 2 1`	উল্টো দিকে
`{0..20..5}`	`0 5 10 15 20`	step (ধাপ) ৫, bash 4+
`{01..10}`	`01 02 ... 10`	শূন্য দিয়ে প্যাড করা (zero padding)
`{a..e}`	`a b c d e`	অক্ষরের range
`{jan,feb,mar}`	`jan feb mar`	কমা দেওয়া তালিকা
`file{1..3}.txt`	`file1.txt file2.txt file3.txt`	prefix/suffix সহ

```
$ echo {0..20..5}
0 5 10 15 20
$ echo backup_{01..03}.tar
backup_01.tar backup_02.tar backup_03.tar
```

> ⚠️ **সতর্কতা:** Brace expansion হয় variable expansion-এর **আগে**। তাই `n=5; for i in {1..$n}` কাজ করবে **না** — আউটপুট আসবে আক্ষরিক `{1..5}` একটা শব্দ হিসেবে। variable দিয়ে range চাইলে `seq` বা C-style লুপ ব্যবহার করো (নিচে দেখো)।

```
$ n=3
$ for i in {1..$n}; do echo "$i"; done
{1..3}
```

২.৩ `seq` দিয়ে range

`seq` একটা আলাদা প্রোগ্রাম (coreutils), যা সংখ্যার ধারা প্রিন্ট করে:

কমান্ড	ফলাফল
`seq 5`	1 থেকে 5
`seq 2 5`	2 থেকে 5
`seq 0 10 50`	0, 10, 20 ... 50 (FIRST INCREMENT LAST)
`seq -w 1 10`	01, 02 ... 10 (সমান প্রস্থ)
`seq -s ' ' 5`	`1 2 3 4 5` (এক লাইনে, separator space)

```
$ n=3
$ for i in $(seq 1 "$n"); do echo "রাউন্ড $i"; done
রাউন্ড 1
রাউন্ড 2
রাউন্ড 3
```

২.৪ C-style `for ((...))`

C/Java/JavaScript জানলে এটা চেনা লাগবে। `(( ))` হলো bash এর **arithmetic context** — ভেতরে `$` ছাড়াই variable লেখা যায়।

```
for (( INIT; CONDITION; STEP )); do
    ...
done
```

```
$ for (( i=1; i<=5; i++ )); do echo -n "$i "; done; echo
1 2 3 4 5
$ n=10
$ for (( i=n; i>0; i-=3 )); do echo -n "$i "; done; echo
10 7 4 1
```

কখন কোনটা?

পরিস্থিতি	ভালো পছন্দ
স্থির range, ছোট	`{1..10}`
variable সীমা	`for ((i=1; i<=n; i++))`
দশমিক ধাপ (0.5, 1.0...)	`seq 0 0.5 2` (bash নিজে শুধু integer গণনা করে)
খুব বড় range (১ কোটি)	C-style — `{1..10000000}` পুরো তালিকা আগে মেমরিতে বানায়

২.৫ ফাইলের উপর লুপ (glob)

```
$ ls
a.txt  b.txt  notes.md  photo 1.jpg
$ for f in *.txt; do echo "টেক্সট ফাইল: $f"; done
টেক্সট ফাইল: a.txt
টেক্সট ফাইল: b.txt
$ for f in *.jpg; do echo "ছবি: $f"; done
ছবি: photo 1.jpg
```

লক্ষ্য করো, `photo 1.jpg` নামে space থাকা সত্ত্বেও এটা **একটা** আইটেম — কারণ glob-এর ফলাফল word splitting হয় না। কিন্তু body-র ভেতরে `$f` ব্যবহার করার সময় অবশ্যই `"$f"` (quote সহ) লিখবে, নাহলে সেখানে আবার ভেঙে যাবে।

বাস্তব উদাহরণ — সব `.jpeg` কে `.jpg` বানাও:

```
for f in *.jpeg; do
    mv -- "$f" "${f%.jpeg}.jpg"
done
```

(`${f%.jpeg}` মানে শেষ থেকে `.jpeg` কেটে ফেলো — এটা নিচে স্ট্রিং অংশে বিস্তারিত আছে। `--` মানে "এর পরে আর অপশন নেই", যাতে `-` দিয়ে শুরু হওয়া ফাইলনাম অপশন হিসেবে না পড়া হয়।)

> ⚠️ **সতর্কতা — কোনো ফাইল না মিললে কী হয়?** ডিফল্টভাবে bash glob-কে অপরিবর্তিত রেখে দেয়। যদি কোনো `.jpeg` না থাকে, তাহলে লুপ একবার চলবে `f='*.jpeg'` দিয়ে, আর `mv` error দেবে। সমাধান:
>
> ```
> shopt -s nullglob      # না মিললে glob খালি তালিকা হয়ে যাবে
> for f in *.jpeg; do ...; done
> ```
>
> অথবা লুপের ভেতরে `[[ -e $f ]] || continue` দিয়ে চেক করো।

সাবডিরেক্টরিসহ সব ফাইল (recursive) চাইলে:

```
shopt -s globstar      # ** কে recursive বানায় (bash 4+)
for f in **/*.log; do
    echo "$f"
done
```

২.৬ স্ক্রিপ্টের আর্গুমেন্টের উপর লুপ

```
#!/usr/bin/env bash
# args.sh
for arg in "$@"; do
    echo "আর্গুমেন্ট: [$arg]"
done
```

```
$ ./args.sh one "two words" three
আর্গুমেন্ট: [one]
আর্গুমেন্ট: [two words]
আর্গুমেন্ট: [three]
```

`"$@"` প্রতিটা আর্গুমেন্টকে আলাদা, অক্ষত শব্দ হিসেবে রাখে। শুধু `for arg; do` লিখলেও (in অংশ বাদ দিলে) bash ধরে নেয় `in "$@"`।

২.৭ কমান্ডের আউটপুটের উপর লুপ

```
for user in $(cut -d: -f1 /etc/passwd | head -3); do
    echo "ইউজার: $user"
done
```

```
ইউজার: root
ইউজার: daemon
ইউজার: bin
```

এটা তখনই নিরাপদ যখন আউটপুটে space বা `*` এর মতো glob অক্ষর নেই। নাহলে পরের অংশের `while read` পদ্ধতি ব্যবহার করো।

---

৩. `while` আর `until` লুপ

৩.১ `while` — শর্ত সত্য থাকা পর্যন্ত

```
while COMMAND; do
    ...
done
```

ভেতরে কী হয়? `while` আসলে কোনো "true/false" দেখে না — সে `COMMAND` চালায় আর তার **exit status** দেখে। exit status `0` (সফল) হলে body চালায়, অশূন্য হলে থামে। তাই `[ ]`, `[[ ]]`, `(( ))`, `grep -q`, `ping -c1` — যেকোনো কমান্ডই শর্ত হতে পারে।

```
count=1
while (( count <= 3 )); do
    echo "গণনা: $count"
    (( count++ ))
done
```

```
গণনা: 1
গণনা: 2
গণনা: 3
```

> 💡 **টিপস:** `(( count++ ))` এর একটা সূক্ষ্ম ফাঁদ আছে: যখন `count` এর পুরোনো মান `0`, তখন `(( count++ ))` এর ফলাফল 0, আর exit status হয় 1। `set -e` চালু থাকলে স্ক্রিপ্ট সেখানেই মারা যাবে! নিরাপদ বিকল্প: `(( ++count ))` বা `count=$((count + 1))`।

কমান্ডকে শর্ত বানানোর বাস্তব উদাহরণ — সার্ভার চালু না হওয়া পর্যন্ত অপেক্ষা:

```
while ! ping -c1 -W1 192.168.1.10 &>/dev/null; do
    echo "সার্ভার এখনো সাড়া দিচ্ছে না, ২ সেকেন্ড অপেক্ষা..."
    sleep 2
done
echo "সার্ভার চালু!"
```

৩.২ `until` — শর্ত সত্য না হওয়া পর্যন্ত

`until` হলো `while` এর উল্টো: কমান্ড ব্যর্থ (অশূন্য) থাকা পর্যন্ত চলে, সফল হলে থামে।

```
until ping -c1 -W1 192.168.1.10 &>/dev/null; do
    sleep 2
done
```

উপরের `while !` উদাহরণটা আর এটা হুবহু একই কাজ করে। যেটা পড়তে স্বাভাবিক লাগে সেটা লেখো: "যতক্ষণ X" → `while`, "X না হওয়া পর্যন্ত" → `until`।

```
n=1
until (( n > 3 )); do
    echo "n = $n"
    (( ++n ))
done
```

```
n = 1
n = 2
n = 3
```

৩.৩ অসীম লুপ (infinite loop)

```
while true; do
    date +%T
    sleep 1
done
```

`true` এমন একটা কমান্ড যা কিছুই করে না, শুধু exit status 0 দেয়। একই কাজ `while :; do` দিয়েও হয় (`:` হলো builtin "no-op" কমান্ড, সবসময় সফল)। এমন লুপ থেকে বের হতে হয় `break` দিয়ে, অথবা কীবোর্ডে **Ctrl+C** চেপে (যা প্রসেসকে SIGINT সিগন্যাল পাঠায় — বিস্তারিত অধ্যায় ২৪-এ)।

৩.৪ `break` আর `continue`

কীওয়ার্ড	কাজ
`break`	সবচেয়ে ভেতরের লুপ পুরোপুরি থামিয়ে `done` এর পরে চলে যায়
`break N`	N স্তর পর্যন্ত লুপ থেকে বের হয় (nested লুপে)
`continue`	এই রাউন্ডের বাকি অংশ বাদ দিয়ে পরের রাউন্ডে যায়
`continue N`	N-তম বাইরের লুপের পরের রাউন্ডে যায়

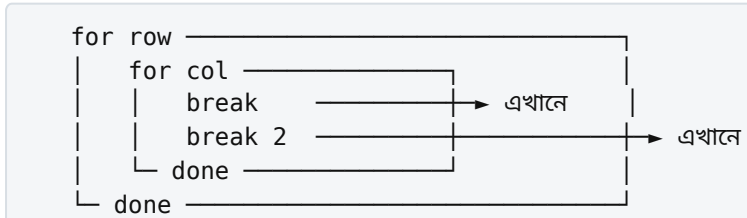
```
for i in {1..10}; do
    (( i % 2 == 0 )) && continue   # জোড় সংখ্যা বাদ
    (( i > 7 )) && break           # ৭ এর বেশি হলে থামো
    echo -n "$i "
done
echo
```

```
1 3 5 7
```

Nested লুপে `break 2`:

```
for row in {1..3}; do
    for col in {1..3}; do
        if (( row * col == 4 )); then
            echo "পাওয়া গেছে: row=$row col=$col"
            break 2          # দুটো লুপ থেকেই বের হও
        fi
    done
done
```

```
পাওয়া গেছে: row=2 col=2
```



৩.৫ লুপের আউটপুট রিডাইরেক্ট করা

পুরো লুপ একটা কমান্ডের মতো। তাই `done` এর পরে রিডাইরেকশন বা পাইপ দেওয়া যায় (অধ্যায় ১৯):

```
for i in {1..100}; do
    echo "line $i"
done > numbers.txt        # ১০০ বার ফাইল খোলার বদলে একবার খোলে — দ্রুত

for f in *.log; do
```

```
    wc -l < "$f"
done | sort -n | tail -1  # সবচেয়ে বড় লগের লাইনসংখ্যা
```

---

8. ফাইল লাইন-বাই-লাইন পড়া: `while IFS= read -r line`

এটা শেল স্ক্রিপ্টিং-এর সবচেয়ে গুরুত্বপূর্ণ "রেসিপি"গুলোর একটা। মুখস্থ করে ফেলো:

```
while IFS= read -r line; do
    echo "লাইন: $line"
done < input.txt
```

৪.১ প্রতিটা অংশ কেন দরকার?

অংশ	কী করে	বাদ দিলে কী সমস্যা
`read`	stdin থেকে **একটা লাইন** পড়ে variable-এ রাখে; ফাইল শেষ (EOF) হলে exit status 1 দেয় → লুপ থামে	—
`-r`	ব্যাকস্ল্যাশ `\` কে বিশেষ অক্ষর হিসেবে না দেখে আক্ষরিকভাবে রাখে	`C:\new` পড়লে `C:new` হয়ে যাবে; লাইনের শেষে `\` থাকলে পরের লাইন জোড়া লেগে যাবে
`IFS=`	শুধু এই `read` কমান্ডের জন্য IFS খালি করে	লাইনের শুরু আর শেষের space/tab কেটে ফেলবে (indentation হারাবে)
`< input.txt`	`done` এর পরে — পুরো লুপের stdin হিসেবে ফাইলটা দেয়	—

ভেতরে কী হয়? `< input.txt` লুপ শুরুর আগে ফাইলটা একবার খোলে, আর লুপের stdin (file descriptor 0) ওই ফাইলের দিকে নির্দেশ করে। প্রতিবার `read` চলে, সে ফাইলের "বর্তমান অবস্থান" থেকে newline পর্যন্ত পড়ে, আর অবস্থান এগিয়ে যায়। এজন্যই প্রতিটা রাউন্ডে পরের লাইন আসে।

```
input.txt:   [ লাইন১\n | লাইন২\n | লাইন৩\n ] EOF
                    ▲
   read #1 ─────────┘  অবস্থান সরে → লাইন২ এর শুরুতে
   read #2 ──────────────► লাইন২
   read #3 ─────────────────────► লাইন৩
   read #4 ─► EOF → exit status 1 → while থামে
```

৪.২ শেষ লাইনে newline না থাকলে

কিছু ফাইলের শেষ লাইনের পরে `\n` থাকে না (Windows-এ বানানো ফাইলে প্রায়ই)। তখন `read` শেষ লাইনটা variable-এ ঠিকই রাখে, কিন্তু exit status 1 দেয় — ফলে লুপ ওই লাইন প্রসেস না করেই থেমে যায়! সমাধান:

```
while IFS= read -r line || [[ -n $line ]]; do
    echo "$line"
```

```
done < input.txt
```

> 💡 **টিপস:** Windows-এর ফাইলে লাইনের শেষে `\r` (carriage return) থাকে, যা অদ্ভুত আউটপুট দেয়। `line=${line%$'\r'}` দিয়ে কেটে ফেলো, অথবা আগে `dos2unix file` চালাও।

৪.৩ লাইনকে ফিল্ডে ভাগ করে পড়া

`read` এ একাধিক variable দিলে, লাইনটা IFS অনুযায়ী ভাগ হয়; শেষ variable বাকি সবটুকু পায়।

```
$ echo "রহিম 25 ঢাকা বাংলাদেশ" | { read -r name age rest; echo "নাম=$name বয়স=$age
বাকি=$rest"; }
নাম=রহিম বয়স=25 বাকি=ঢাকা বাংলাদেশ
```

`/etc/passwd` পড়া (ফিল্ড `:` দিয়ে আলাদা):

```
while IFS=: read -r user _ uid gid _ home shell; do
    (( uid >= 1000 && uid < 65534 )) || continue
    printf '%-12s UID=%-6s %s\n' "$user" "$uid" "$shell"
done < /etc/passwd
```

```
nishat       UID=1000   /bin/bash
rahim        UID=1001   /bin/bash
```

(`_` একটা "ফেলে দেওয়ার" variable — যে ফিল্ড দরকার নেই সেটা এখানে রাখি।)

CSV পড়া:

```
# data.csv:  name,dept,salary
{
    read -r header                      # প্রথম লাইন (হেডার) বাদ
    while IFS=, read -r name dept salary; do
        echo "$name ($dept): $salary"
    done
} < data.csv
```

> ⚠️ **সতর্কতা:** এই পদ্ধতি শুধু "সরল" CSV-র জন্য। যদি কোনো ফিল্ডের ভেতরে কমা থাকে (`"Dhaka, BD"`), তাহলে ভুল হবে। জটিল CSV-র জন্য `awk` (অধ্যায় ২২), `python`-এর csv module, বা `mlr` / `csvkit` টুল ব্যবহার করো।

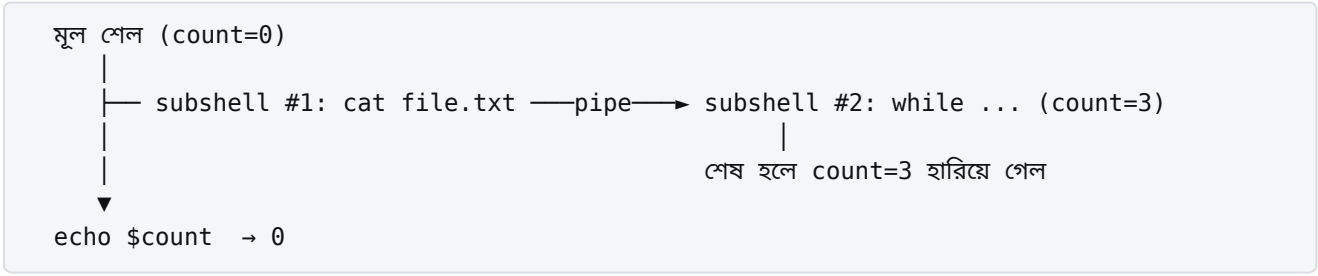
৪.৪ বড় ফাঁদ: পাইপ + while = subshell

```
count=0
cat file.txt | while IFS= read -r line; do
    (( ++count ))
```

```
done
echo "মোট লাইন: $count"
```

```
মোট লাইন: 0
```

কেন 0? bash এ পাইপের প্রতিটা অংশ আলাদা **subshell** (শেলের একটা কপি, child process) এ চলে। `while` লুপটা subshell-এ `count` বাড়িয়েছে, কিন্তু subshell শেষ হলে তার সব variable হারিয়ে যায়। মূল শেলের `count` তখনও 0।



সমাধান ১ — ফাইল সরাসরি রিডাইরেক্ট করো (সবচেয়ে ভালো, `cat` দরকারই নেই):

```
while IFS= read -r line; do (( ++count )); done < file.txt
```

সমাধান ২ — কমান্ডের আউটপুট হলে process substitution `< <(...)` :

```
count=0
while IFS= read -r line; do
    (( ++count ))
done < <(grep ERROR /var/log/syslog)
echo "ERROR লাইন: $count"
```

`<(cmd)` cmd এর আউটপুটকে একটা ফাইলের মতো (`/dev/fd/63` জাতীয় পাথ) দেখায়, তাই লুপ মূল শেলেই চলে।

সমাধান ৩ — `shopt -s lastpipe` : স্ক্রিপ্টে (যেখানে job control বন্ধ) এটা চালু করলে পাইপের শেষ অংশ মূল শেলে চলে।

৪.৫ লুপের ভেতরের কমান্ড stdin খেয়ে ফেলা

```
while IFS= read -r host; do
    ssh "$host" uptime          # ⚠️ শুধু প্রথম host এ চলবে!
done < hosts.txt
```

`ssh` নিজেও stdin পড়ে, তাই সে `hosts.txt` এর বাকি লাইনগুলো খেয়ে ফেলে। সমাধান: `ssh -n` (stdin কে `/dev/null` বানায়), অথবা ফাইলকে অন্য file descriptor দিয়ে পড়ো:

```
while IFS= read -r -u 3 host; do
    ssh "$host" uptime
```

```
done 3< hosts.txt
```

এখানে ফাইল খোলা হয়েছে fd 3-এ, `read -u 3` সেখান থেকে পড়ে, stdin অক্ষত থাকে।

৪.৬ `read` এর অন্যান্য দরকারি অপশন

অপশন	কাজ
`-r`	ব্যাকস্ল্যাশকে আক্ষরিক রাখে (প্রায় সবসময় দিও)
`-p "prompt"`	প্রম্প দেখায় (ইন্টারঅ্যাক্টিভ ইনপুটে)
`-s`	টাইপ করা অক্ষর দেখায় না (পাসওয়ার্ড)
`-t N`	N সেকেন্ডের মধ্যে ইনপুট না এলে ব্যর্থ হয়
`-n N`	N অক্ষর পড়েই থামে (Enter লাগে না)
`-d DELIM`	newline এর বদলে DELIM পর্যন্ত পড়ে (`-d ''` মানে NUL অক্ষর পর্যন্ত)
`-a ARR`	শব্দগুলো একটা indexed array-তে রাখে
`-u FD`	stdin না, FD নম্বর থেকে পড়ে

অদ্ভুত নামের ফাইল (নামে newline!) পর্যন্ত নিরাপদে পড়ার "পেশাদার" রূপ — `find -print0` এর সাথে (অধ্যায় ১২):

```
while IFS= read -r -d '' file; do
    echo "পাওয়া গেছে: $file"
done < <(find . -type f -name '*.conf' -print0)
```

---

৫. ফাংশন (Functions)

৫.১ ফাংশন কেন?

একই কোড পাঁচ জায়গায় কপি-পেস্ট করলে, একটা বাগ ঠিক করতে পাঁচ জায়গায় হাত দিতে হয়। ফাংশন হলো কোডের একটা **নামওয়ালা টুকরো**, যা যতবার খুশি নাম ধরে ডাকা যায়।

উপমা: রান্নার রেসিপি বইয়ে "পেঁয়াজ বেরেস্তা করার নিয়ম" একবার লেখা থাকে, আর বিরিয়ানি, হালিম, কোরমা — সব রেসিপিতে বলা থাকে "পৃষ্ঠা ১২ অনুযায়ী বেরেস্তা বানাও।" সেই পৃষ্ঠা ১২ টাই ফাংশন।

৫.২ ডিফাইন করা

দুটো সিনট্যাক্স আছে:

```
# রূপ ১ — POSIX, সব শেলে চলে (প্রস্তাবিত)
greet() {
    echo "আসসালামু আলাইকুম!"
```

```
}

# রূপ ২ — bash/ksh এর নিজস্ব
function greet {
    echo "আসসালামু আলাইকুম!"
}
```

ডাকা হয় একটা সাধারণ কমান্ডের মতো — বন্ধনী ছাড়া:

```
$ greet
আসসালামু আলাইকুম!
```

নিয়মগুলো:

- ফাংশন **ব্যবহারের আগে** ডিফাইন করতে হবে (bash উপর থেকে নিচে পড়ে)।
- `{` এর পরে space আর `}` এর আগে newline বা `;` লাগবে: `f() { echo hi; }`।
- ফাংশনের নাম যদি কোনো কমান্ডের নামের সাথে মিলে যায় (যেমন `ls`), ফাংশনটাই আগে চলবে। bash কমান্ড খোঁজার ক্রম: alias → keyword → **function** → builtin → `$PATH`-এর প্রোগ্রাম (অধ্যায় ৫)।

ভেতরে কী হয়? ফাংশন ডিফাইন করলে bash তার body মেমরিতে জমা রাখে — কোনো নতুন প্রসেস তৈরি হয় না। ডাকলে **একই শেল প্রসেসের** ভেতরেই চলে। তাই ফাংশন চালানো বাইরের স্ক্রিপ্ট চালানোর চেয়ে অনেক দ্রুত, আর ফাংশন মূল শেলের variable পড়তে ও বদলাতে পারে।

৫.৩ আর্গুমেন্ট পাঠানো

ফাংশনের ভেতরে `$1`, `$2`, ..., `$#`, `$@` — এগুলো **ফাংশনের** আর্গুমেন্ট বোঝায়, স্ক্রিপ্টের না। (`$0` অবশ্য তখনও স্ক্রিপ্টের নামই থাকে।)

```
greet() {
    local name=$1
    local time=${2:-দিন}          # দ্বিতীয় আর্গুমেন্ট না দিলে "দিন"
    echo "শুভ $time, $name! (মোট আর্গুমেন্ট: $#)"
}

greet "করিম" "সকাল"
greet "রহিমা"
```

```
শুভ সকাল, করিম! (মোট আর্গুমেন্ট: 2)
শুভ দিন, রহিমা! (মোট আর্গুমেন্ট: 1)
```

বিশেষ variable	ফাংশনের ভেতরে অর্থ
`$1` … `$9`, `${10}`	পজিশনাল আর্গুমেন্ট (১০ বা তার বেশি হলে `{}` লাগবে)
`$#`	আর্গুমেন্টের সংখ্যা
`"$@"`	সব আর্গুমেন্ট, আলাদা আলাদা শব্দ হিসেবে
`"$*"`	সব আর্গুমেন্ট একটা শব্দে জোড়া (IFS এর প্রথম অক্ষর দিয়ে)

বিশেষ variable	ফাংশনের ভেতরে অর্থ
`$0`	স্ক্রিপ্টের নাম (ফাংশনের না!)
`$FUNCNAME` / `${FUNCNAME[0]}`	বর্তমান ফাংশনের নাম
`shift`	`$1` বাদ দিয়ে সবাইকে এক ঘর বামে সরায়

আর্গুমেন্ট যাচাই করা একটা ভালো অভ্যাস:

```
backup_file() {
    if (( $# != 1 )); then
        echo "ব্যবহার: ${FUNCNAME[0]} <ফাইল>" >&2
        return 2
    fi
    cp -- "$1" "$1.bak.$(date +%F)"
}
```

৫.৪ `return` বনাম `echo` — সবচেয়ে বড় বিভ্রান্তি

অন্য প্রোগ্রামিং ভাষায় `return` দিয়ে যেকোনো মান (স্ট্রিং, সংখ্যা) ফেরত দেওয়া যায়। bash-এ **না**।

	`return N`	`echo` / `printf`
কী ফেরত দেয়	শুধু **exit status** (0–255 পূর্ণসংখ্যা)	যেকোনো **ডেটা** (টেক্সট) stdout-এ
কোথায় ধরা হয়	`$?`, অথবা `if func; then`	`result=$(func)` (command substitution)
উদ্দেশ্য	সফল/ব্যর্থ জানানো	ফলাফল দেওয়া

```
# ভুল ধারণা:
add() { return $(( $1 + $2 )); }
add 200 100
echo $?          # 44 !!  কারণ 300 % 256 = 44
```

```
# সঠিক — ডেটা echo দিয়ে:
add() { echo $(( $1 + $2 )); }
sum=$(add 200 100)
echo "যোগফল: $sum"
```

```
যোগফল: 300
```

```
# সঠিক — return দিয়ে সফল/ব্যর্থ:
is_even() {
    (( $1 % 2 == 0 ))     # এই কমান্ডের exit status-ই ফাংশনের exit status
}

if is_even 42; then echo "জোড়"; else echo "বিজোড়"; fi
```

```
জোড়
```

> 📝 **মনে রাখো:** `return` ছাড়া ফাংশন শেষ হলে, ফাংশনের exit status হয় **শেষ চালানো কমান্ডের** exit status। `return` (সংখ্যা ছাড়া) ও একই কাজ করে।

দুটো একসাথে: ডেটা stdout-এ, error মেসেজ stderr-এ (`>&2`), আর সফলতা exit status-এ:

```
get_ip() {
    local iface=$1 ip
    ip=$(ip -4 -o addr show dev "$iface" 2>/dev/null | awk '{print $4}' | cut -d/ -f1)
    if [[ -z $ip ]]; then
        echo "ত্রুটি: $iface এ কোনো IPv4 নেই" >&2
        return 1
    fi
    echo "$ip"
}

if myip=$(get_ip eth0); then
    echo "আমার IP: $myip"
fi
```

> ⚠️ **সতর্কতা:** `result=$(func)` এ ফাংশনটা **subshell**-এ চলে। তাই ফাংশন ভেতরে কোনো global variable বদলালে সেই পরিবর্তন বাইরে দেখা যাবে না। আর ফাংশনের সব stdout ফলাফলে ঢুকে যাবে — debug মেসেজ সবসময় `>&2` দিয়ে লিখবে।

৫.৫ `local` — variable-এর scope

bash-এ ফাংশনের ভেতরে বানানো variable ডিফল্টভাবে **global** — পুরো স্ক্রিপ্টে দেখা যায়, এবং বাইরের একই নামের variable-কে চুপচাপ বদলে ফেলে!

```
name="মূল"
change() {
    name="ফাংশনে বদলানো"
}
change
echo "$name"
```

```
ফাংশনে বদলানো
```

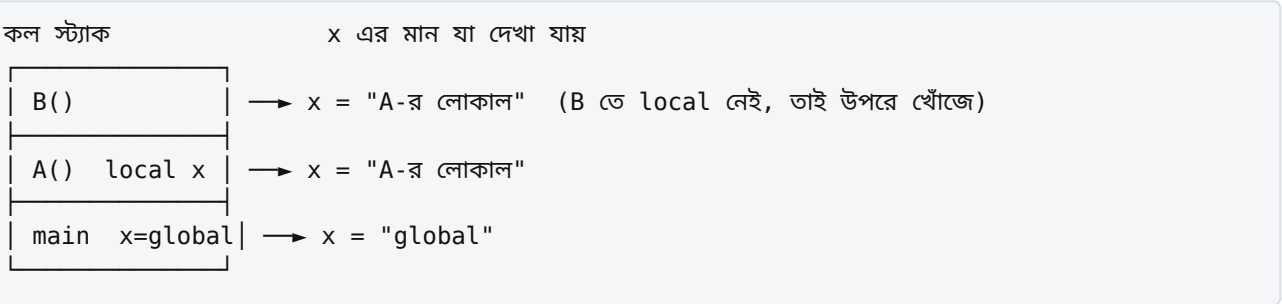
`local` দিলে variable শুধু ওই ফাংশনের (এবং সে যাদের ডাকে তাদের) মধ্যে সীমাবদ্ধ থাকে:

```
name="মূল"
safe_change() {
    local name="লোকাল"
    echo "ভেতরে: $name"
}
```

```
safe_change
echo "বাইরে: $name"
```

```
ভেতরে: লোকাল
বাইরে: মূল
```

ভেতরে কী হয় (dynamic scoping): bash একটা variable-এর স্তূপ (stack) রাখে। `local` ডাকলে পুরোনো মানটা আড়ালে সরিয়ে নতুন মান বসায়, আর ফাংশন শেষ হলে পুরোনোটা ফিরিয়ে আনে। একটা মজার ব্যাপার: bash **dynamic scoping** ব্যবহার করে, অর্থাৎ ফাংশন A তে `local x` থাকলে, A যে ফাংশন B কে ডাকে, সেই B ও A-র `x` দেখতে পায়।



> 💡 **টিপস:** অভ্যাস করো — ফাংশনের ভেতরের **সব** variable `local` দিয়ে ঘোষণা করবে। এতে অন্য অংশের সাথে নামের সংঘর্ষ হবে না।

> ⚠️ **সতর্কতা:** `local x=$(cmd)` লিখলে `$?` হবে `local` কমান্ডের status (প্রায় সবসময় 0), `cmd` এর না। error চেক করতে চাইলে দুই লাইনে লেখো:
>
> ```
> local out
> out=$(cmd) || return 1
> ```

`local` এর সাথে `declare` এর অপশনও চলে: `local -i n` (integer), `local -a arr` (array), `local -A map` (associative array), `local -r CONST=5` (read-only)।

৫.৬ রেফারেন্স দিয়ে variable বদলানো: `local -n` (nameref)

`$(...)` subshell এড়িয়ে ফাংশন দিয়ে সরাসরি কলারের variable-এ মান বসাতে চাইলে bash 4.3+ এর **nameref** ব্যবহার করো:

```
to_upper() {
    local -n _ref=$1         # _ref এখন কলারের variable-এর "ডাকনাম"
    _ref=${_ref^^}
}

city="dhaka"
```

```
to_upper city               # লক্ষ্য করো: $city না, শুধু নাম
echo "$city"
```

```
DHAKA
```

(ফাংশনের ভেতরের nameref-এর নাম `_ref` এর মতো অস্বাভাবিক রাখো — কলারের variable-এর নামও যদি `_ref` হয়, তাহলে circular reference হয়ে যাবে।)

৫.৭ ফাংশন ম্যানেজ করার কমান্ড

কমান্ড	কাজ
`declare -f`	সব ফাংশনের সম্পূর্ণ কোড দেখায়
`declare -f NAME`	একটা ফাংশনের কোড
`declare -F`	শুধু ফাংশনের নামের তালিকা
`type NAME`	NAME টা ফাংশন, builtin, alias নাকি ফাইল — বলে
`unset -f NAME`	ফাংশন মুছে ফেলে
`export -f NAME`	child bash প্রসেসেও ফাংশনটা পাঠায় (যেমন `bash -c`, `xargs ... bash -c`)

```
$ type greet
greet is a function
greet ()
{
    echo "আসসালামু আলাইকুম!"
}
```

৫.৮ রিকার্শন (recursion)

ফাংশন নিজেকে ডাকতে পারে:

```
factorial() {
    local n=$1
    if (( n <= 1 )); then
        echo 1
    else
        echo $(( n * $(factorial $((n - 1))) ))
    fi
}
factorial 5
```

```
120
```

কাজ করে, কিন্তু প্রতিটা `$(...)` একটা নতুন subshell (fork) — তাই ধীর। bash-এ গভীর রিকার্শন এড়িয়ে চলো; লুপ দিয়ে লেখা ভালো।

৫.৯ লাইব্রেরি বানানো: `source`

দরকারি ফাংশনগুলো একটা ফাইলে রেখে অন্য স্ক্রিপ্টে আনা যায়:

```
# lib.sh
log()  { printf '[%s] %s\n' "$(date +%T)" "$*"; }
die()  { echo "ত্রুটি: $*" >&2; exit 1; }
```

```
#!/usr/bin/env bash
# main.sh
source "$(dirname "$0")/lib.sh"     # অথবা:  . ./lib.sh
log "কাজ শুরু"
[[ -f config.txt ]] || die "config.txt নেই"
```

`source` ফাইলটাকে **বর্তমান শেলেই** চালায় (নতুন প্রসেসে না), তাই তার ফাংশন আর variable এখানে থেকে যায়। এই একই কৌশলে `~/.bashrc` লোড হয় — বিস্তারিত অধ্যায় ৪৪-এ।

---

৬. Indexed Array (সংখ্যা-ভিত্তিক অ্যারে)

৬.১ অ্যারে কী আর কেন?

সাধারণ variable এ একটা মান থাকে। অ্যারে হলো একটা variable যার মধ্যে **অনেকগুলো মান** থাকে, প্রতিটা একটা নম্বর (index) দিয়ে চিহ্নিত।

উপমা: ট্রেনের বগি। পুরো ট্রেনের একটা নাম (`servers`), আর প্রতিটা বগির একটা নম্বর (0, 1, 2 ...)। bash-এ নম্বর **0 থেকে** শুরু হয়।

```
servers ──► ┌─────────┬─────────┬─────────┬─────────┐
            │  web1   │  web2   │  db1    │ cache1  │
            └─────────┴─────────┴─────────┴─────────┘
  index:        0         1         2         3

  ${servers[2]}   → db1
  ${#servers[@]}  → 4   (মোট উপাদান)
  ${!servers[@]}  → 0 1 2 3   (সব index)
```

৬.২ তৈরি করা

```
fruits=(আম কাঁঠাল "কাঁচা আম" লিচু)    # space থাকলে quote
empty=()                                  # খালি অ্যারে
declare -a nums=(10 20 30)                # স্পষ্টভাবে ঘোষণা (ঐচ্ছিক)
days[0]="শনি"; days[1]="রবি"             # একটা একটা করে
files=( *.txt )                           # glob এর ফলাফল থেকে
sparse=([0]=a [5]=b [10]=c)               # নির্দিষ্ট index-এ
```

> ⚠️ **সতর্কতা:** `=` এর দুপাশে space দেবে না: `arr = (a b)` ভুল। আর অ্যারে বানাতে `arr=$(cmd)` লিখলে অ্যারে না, একটা সাধারণ স্ট্রিং হবে।

৬.৩ পড়া ও অপারেশন — পূর্ণ টেবিল

সিনট্যাক্স	অর্থ	উদাহরণ ফলাফল (`a=(x y z)`)
`${a[0]}`	index 0 এর মান	`x`
`${a[-1]}`	শেষ উপাদান (bash 4.3+)	`z`
`$a`	`${a[0]}` এর সমান (বিভ্রান্তিকর, এড়াও)	`x`
`"${a[@]}"`	সব উপাদান, প্রত্যেকটা আলাদা শব্দ	`x` `y` `z`
`"${a[*]}"`	সব উপাদান একটা শব্দে, IFS এর প্রথম অক্ষর দিয়ে জোড়া	`x y z`
`${#a[@]}`	উপাদানের সংখ্যা	`3`
`${#a[0]}`	index 0 এর স্ট্রিং-এর দৈর্ঘ্য	`1`
`${!a[@]}`	সব index	`0 1 2`
`"${a[@]:1:2}"`	slice: index 1 থেকে ২টা	`y z`
`a+=(w)`	শেষে যোগ (append)	`x y z w`
`a[1]=Y`	মান বদলানো	`x Y z`
`unset 'a[1]'`	একটা উপাদান মোছা (index ফাঁকা থাকে!)	index: `0 2`
`unset a`	পুরো অ্যারে মোছা	—
`a=("${a[@]}")`	ফাঁকা index গুলো সরিয়ে আবার 0,1,2... সাজানো	—

```
$ fruits=(আম কাঁঠাল "কাঁচা আম" লিচু)
$ echo "${fruits[2]}"
কাঁচা আম
$ echo "মোট: ${#fruits[@]}"
মোট: 4
$ fruits+=(জাম)
$ echo "${fruits[@]}"
আম কাঁঠাল কাঁচা আম লিচু জাম
$ unset 'fruits[1]'
$ echo "${!fruits[@]}"
0 2 3 4
```

লক্ষ্য করো — `unset` করার পরে index `1` এখন ফাঁকা; বাকিরা সরে আসেনি। bash অ্যারে আসলে **sparse** (ফাঁকা থাকতে পারে) — ভেতরে এটা একটা linked list এর মতো, যেখানে শুধু যেসব index-এ মান আছে সেগুলোই রাখা হয়। তাই "0 থেকে length-1 পর্যন্ত লুপ" সবসময় নিরাপদ না।

> 💡 **টিপস:** `unset fruits[1]` (quote ছাড়া) লিখলে `[1]` glob হিসেবে পড়া হতে পারে — যদি বর্তমান ডিরেক্টরিতে `fruits1` নামের ফাইল থাকে, অদ্ভুত বাগ হবে। তাই সবসময় `unset 'fruits[1]'`।

৬.৪ `"${a[@]}"` বনাম `${a[@]}` বনাম `"${a[*]}"`

```
a=("প্রথম আইটেম" "দ্বিতীয়")

for x in "${a[@]}"; do echo "[$x]"; done   # ✅ সঠিক
# [প্রথম আইটেম]
# [দ্বিতীয়]

for x in ${a[@]};   do echo "[$x]"; done   # ❌ quote নেই → ভেঙে যায়
# [প্রথম]
# [আইটেম]
# [দ্বিতীয়]

for x in "${a[*]}"; do echo "[$x]"; done   # এক শব্দে জোড়া
# [প্রথম আইটেম দ্বিতীয়]
```

> 📝 **মনে রাখো:** ৯৯% সময় তোমার দরকার `"${arr[@]}"` — quote সহ, `@` সহ।

৬.৫ অ্যারের উপর লুপ

```
servers=(web1 web2 db1)

# মান ধরে
for s in "${servers[@]}"; do
    echo "চেক করছি: $s"
done

# index ধরে (sparse অ্যারেতেও নিরাপদ)
for i in "${!servers[@]}"; do
    echo "$i → ${servers[i]}"
done
```

```
চেক করছি: web1
চেক করছি: web2
চেক করছি: db1
0 → web1
1 → web2
2 → db1
```

(`[ ]` এর ভেতরে index টা arithmetic context, তাই `${servers[i]}` এ `$i` না লিখলেও চলে।)

৬.৬ ফাইল বা কমান্ড থেকে অ্যারে: `mapfile` / `readarray`

`mapfile` (সমনাম `readarray`) একটা builtin যা stdin এর প্রতিটা লাইনকে একটা অ্যারে উপাদান বানায়। `while read` লুপের চেয়ে দ্রুত আর সহজ।

অপশন	কাজ
`-t`	প্রতিটা লাইনের শেষের newline কেটে ফেলে (প্রায় সবসময় দিও)
`-n N`	সর্বোচ্চ N লাইন পড়ে
`-s N`	প্রথম N লাইন বাদ দেয়
`-d DELIM`	newline এর বদলে DELIM দিয়ে ভাগ করে (bash 4.4+; `-d ''` মানে NUL)

```
$ mapfile -t lines < /etc/hostname
$ mapfile -t users < <(cut -d: -f1 /etc/passwd)
$ echo "প্রথম ইউজার: ${users[0]}, মোট: ${#users[@]}"
প্রথম ইউজার: root, মোট: 38
```

> ⚠️ **সতর্কতা:** `cmd | mapfile -t arr` কাজ করবে না — একই subshell সমস্যা (অংশ ৪.৪)। সবসময় `mapfile -t arr < <(cmd)`।

একটা স্ট্রিংকে ভাগ করে অ্যারে বানানো — `read -a`:

```
$ csv="ঢাকা,চট্টগ্রাম,সিলেট"
$ IFS=, read -r -a cities <<< "$csv"
$ echo "${cities[1]}"
চট্টগ্রাম
```

(`<<<` হলো "here-string" — একটা স্ট্রিংকে stdin হিসেবে দেয়।)

৬.৭ কিছু দরকারি অ্যারে কৌশল

```
# অ্যারেতে কোনো মান আছে কি না
contains() {
    local needle=$1; shift
    local item
    for item in "$@"; do
        [[ $item == "$needle" ]] && return 0
    done
    return 1
}
contains db1 "${servers[@]}" && echo "db1 আছে"

# অ্যারে সর্ট করা (প্রতিটা উপাদানে newline না থাকলে)
mapfile -t sorted < <(printf '%s\n' "${servers[@]}" | sort)

# অ্যারে জোড়া দিয়ে কমা-বিভক্ত স্ট্রিং
joined=$(IFS=,; echo "${servers[*]}")
```

```
echo "$joined"        # web1,web2,db1

# কমান্ডের আর্গুমেন্ট নিরাপদে তৈরি করা (খুব কাজের!)
opts=(-a -v --delete)
[[ $dry_run == yes ]] && opts+=(--dry-run)
rsync "${opts[@]}" src/ dest/
```

শেষ কৌশলটা গুরুত্বপূর্ণ: কমান্ডের অপশন একটা স্ট্রিং variable-এ রাখলে (`opts="-a --exclude 'my dir'"`) quote ঠিকমতো কাজ করে না। অ্যারেতে রাখলে প্রতিটা অপশন অক্ষত থাকে। (`rsync` এর বিস্তারিত অধ্যায় ৩৫-এ।)

---

৭. Associative Array (`declare -A`) — key-value অভিধান

৭.১ ধারণা

Indexed অ্যারেতে চাবি হলো সংখ্যা। Associative অ্যারেতে চাবি হলো **স্ট্রিং** — অনেকটা অভিধানের মতো: শব্দ (key) → অর্থ (value)। অন্য ভাষায় এর নাম dictionary, hash map, object।

```
capital ──►  ┌──────────────────┬──────────────────┐
             │  key             │  value           │
             ├──────────────────┼──────────────────┤
             │ "বাংলাদেশ"       │ "ঢাকা"           │
             │ "ভারত"          │ "নয়াদিল্লি"     │
             │ "জাপান"         │ "টোকিও"         │
             └──────────────────┴──────────────────┘
ভেতরে: hash table — key থেকে hash হিসাব করে সরাসরি ঘরে যায়, তাই খোঁজা দ্রুত
```

৭.২ তৈরি ও ব্যবহার

> ⚠️ **সতর্কতা:** Associative array **অবশ্যই** আগে `declare -A` দিয়ে ঘোষণা করতে হবে। না করলে bash এটাকে indexed array ভাববে, আর স্ট্রিং key কে arithmetic হিসেবে পড়বে (সব key → 0), ফলে সব মান একই ঘরে ঘষে যাবে। Associative array bash **4.0** থেকে আছে; macOS এর পুরোনো bash 3.2 তে নেই।

```
declare -A capital
capital[বাংলাদেশ]="ঢাকা"
capital[ভারত]="নয়াদিল্লি"
capital["দক্ষিণ কোরিয়া"]="সিউল"       # key তে space থাকলে quote

# অথবা একসাথে:
declare -A port=( [ssh]=22 [http]=80 [https]=443 [mysql]=3306 )
```

সিনট্যাক্স	অর্থ
`${port[ssh]}`	key `ssh` এর মান → `22`

সিনট্যাক্স	অর্থ
`"${port[@]}"`	সব value
`"${!port[@]}"`	সব key
`${#port[@]}`	key-value জোড়ার সংখ্যা
`port[dns]=53`	যোগ বা আপডেট
`unset 'port[mysql]'`	একটা key মোছা
`[[ -v port[ssh] ]]`	key আছে কি না (bash 4.3+)
`declare -p port`	পুরো অ্যারের গঠন দেখায় (debug-এর জন্য দারুণ)

```
$ echo "SSH পোর্ট: ${port[ssh]}"
SSH পোর্ট: 22
$ [[ -v port[ftp] ]] && echo "আছে" || echo "ftp নেই"
ftp নেই
$ for svc in "${!port[@]}"; do echo "$svc → ${port[$svc]}"; done
https → 443
ssh → 22
http → 80
mysql → 3306
```

> 📝 **মনে রাখো:** Associative array-র key গুলোর **কোনো নির্দিষ্ট ক্রম নেই** (hash table)। ক্রম দরকার হলে key গুলো সর্ট করো:
>
> ```
> for k in $(printf '%s\n' "${!port[@]}" | sort); do ...; done
> ```
>
> (key তে space থাকলে `mapfile -t keys < <(printf '%s\n' "${!port[@]}" | sort)` ব্যবহার করো।)

৭.৩ বাস্তব উদাহরণ: শব্দ গণনা

```
#!/usr/bin/env bash
declare -A count
while IFS= read -r line; do
    for word in $line; do                # এখানে ইচ্ছা করে word splitting
        word=${word,,}                   # ছোট হাতের অক্ষরে
        (( count[$word]++ ))
    done
done < story.txt

for w in "${!count[@]}"; do
    printf '%5d %s\n' "${count[$w]}" "$w"
done | sort -rn | head -5
```

```
   12 the
    8 linux
```

```
    5 is
    4 shell
    3 kernel
```

আরেকটা উদাহরণ — লগ থেকে প্রতি IP-র রিকোয়েস্ট গণনা:

```
declare -A hits
while read -r ip _; do
    (( hits[$ip]++ ))
done < /var/log/nginx/access.log

for ip in "${!hits[@]}"; do
    echo "${hits[$ip]} $ip"
done | sort -rn | head
```

---

৮. স্ট্রিং অপারেশন — Parameter Expansion

bash এর ভেতরেই স্ট্রিং কাটাছেঁড়ার শক্তিশালী ব্যবস্থা আছে, যাকে বলে **parameter expansion**। এগুলো `sed` / `cut` / `basename` এর মতো বাইরের প্রোগ্রাম চালায় না, তাই **অনেক দ্রুত** (কোনো নতুন প্রসেস fork হয় না)। লুপের ভেতরে হাজার বার চললে পার্থক্যটা বিশাল।

নিচের সব উদাহরণে ধরি:

```
path="/home/rahim/docs/report.final.pdf"
name="Bangladesh"
```

৮.১ দৈর্ঘ্য: `${#var}`

```
$ echo ${#name}
10
$ bn="বাংলা"; echo ${#bn}
5
```

(UTF-8 locale এ bash **অক্ষর** গোনে, byte না। "বাংলা" তে ব-া-ং-ল-া মোট ৫টা Unicode অক্ষর।)

৮.২ Substring: `${var:offset:length}`

লেখা	ফলাফল	ব্যাখ্যা
`${name:0:3}`	`Ban`	index 0 থেকে ৩ অক্ষর
`${name:3}`	`gladesh`	index 3 থেকে শেষ পর্যন্ত
`${name: -4}`	`desh`	শেষের ৪ অক্ষর (`:` **এর পরে space বাধ্যতামূলক**)
`${name: -4:2}`	`de`	শেষ থেকে ৪ এ গিয়ে ২ অক্ষর
`${name:2:-3}`	`nglad`	index 2 থেকে, শেষের ৩টা বাদ দিয়ে (bash 4.2+)

> ⚠️ **সতর্কতা:** `${name:-4}` (space ছাড়া) সম্পূর্ণ আলাদা জিনিস — এটা "default value" অপারেটর (অংশ ৮.৫)! name সেট থাকলে পুরো `Bangladesh` দেবে।

৮.৩ খোঁজা ও বদলানো: `${var/pattern/replacement}`

লেখা	কাজ	`s="banana"` এ ফলাফল
`${s/a/A}`	প্রথম মিল বদলাও	`bAnana`
`${s//a/A}`	**সব** মিল বদলাও	`bAnAnA`
`${s/#ba/BA}`	শুধু শুরুতে মিললে	`BAnana`
`${s/%na/NA}`	শুধু শেষে মিললে	`banaNA`
`${s//a/}`	সব `a` মুছে দাও	`bnn`
`${s//[an]/_}`	glob pattern ও চলে	`b___`

```
$ msg="Hello World Hello"
$ echo "${msg/Hello/Hi}"
Hi World Hello
$ echo "${msg//Hello/Hi}"
Hi World Hi
$ file="my song.mp3"
$ echo "${file// /_}"          # space কে underscore
my_song.mp3
```

> 📝 **মনে রাখো:** এখানে pattern হলো **glob** (`*`, `?`, `[...]`), regex না। Regex দরকার হলে `[[ $s =~ regex ]]` আর `BASH_REMATCH` (অধ্যায় ২০), বা `sed` (অধ্যায় ২১)।

৮.৪ শুরু/শেষ থেকে কাটা: `#`, `##`, `%`, `%%`

এগুলো সবচেয়ে কাজের, কিন্তু মনে রাখা কঠিন। একটা কৌশল: কীবোর্ডে `#` আছে `$` এর **বামে** (Shift+3), আর `%` আছে **ডানে** (Shift+5)। তাই `#` কাটে **বাম/শুরু** থেকে, `%` কাটে **ডান/শেষ** থেকে। একটা চিহ্ন = সবচেয়ে **ছোট** মিল, দুটো চিহ্ন = সবচেয়ে **বড়** মিল (greedy)।

লেখা	কাজ	`path="/home/rahim/docs/report.final.pdf"` এ ফলাফল
`${path#*/}`	শুরু থেকে সবচেয়ে ছোট `*/` মুছে	`home/rahim/docs/report.final.pdf`
`${path##*/}`	শুরু থেকে সবচেয়ে বড় `*/` মুছে → **ফাইলের নাম**	`report.final.pdf`

লেখা	কাজ	`path="/home/rahim/docs/report.final.pdf"` এ ফলাফল
`${path%/*}`	শেষ থেকে সবচেয়ে ছোট `/*` মুছে → **ডিরেক্টরি**	`/home/rahim/docs`
`${path%%/*}`	শেষ থেকে সবচেয়ে বড় `/*` মুছে	`` (খালি)
`${path%.*}`	শেষের এক্সটেনশন বাদ	`/home/rahim/docs/report.final`
`${path%%.*}`	প্রথম `.` থেকে সব বাদ	`/home/rahim/docs/report`
`${path##*.}`	শুধু এক্সটেনশন	`pdf`

ডায়াগ্রামে দেখো:

```
 path =  /home/rahim/docs/report.final.pdf
         ├──────────────────┤                    ${path##*/} এই অংশ মোছে → report.final.pdf
                            ├──────────────────┤  ${path%/*}  এই অংশ মোছে → /home/rahim/docs
                                               ├──┤  ${path%.*}  এই অংশ মোছে → .../report.final
                                  ├────────────┤  ${path%%.*} এই অংশ মোছে → .../report
         ├──────────────────────────────────┤     ${path##*.} এই অংশ মোছে → pdf
```

এগুলো দিয়েই `basename` আর `dirname` কমান্ডের কাজ বিনা খরচে হয়:

```
$ f="/var/log/nginx/access.log.1"
$ echo "নাম: ${f##*/}   ডিরেক্টরি: ${f%/*}"
নাম: access.log.1   ডিরেক্টরি: /var/log/nginx
```

বাস্তব উদাহরণ — সব `.png` কে `.webp` এ রূপান্তর:

```
for img in *.png; do
    cwebp -q 80 "$img" -o "${img%.png}.webp"
done
```

৮.৫ ডিফল্ট মান: `${var:-default}` পরিবার

লেখা	var সেট ও খালি নয়	var খালি বা unset	var-এর মান বদলায়?
`${var:-word}`	`$var`	`word`	না
`${var:=word}`	`$var`	`word`	**হ্যাঁ**, var=word হয়ে যায়
`${var:+word}`	`word`	খালি	না

লেখা	var সেট ও খালি নয়	var খালি বা unset	var-এর মান বদলায়?
`${var:?msg}`	`$var`	msg stderr-এ দেখিয়ে স্ক্রিপ্ট **বন্ধ**	—

`:` ছাড়া রূপগুলো (`${var-word}`, `${var=word}` ইত্যাদি) শুধু **unset** হলে কাজ করে; var সেট কিন্তু খালি (`var=""`) হলে করে না।

```
$ unset color
$ echo "রং: ${color:-নীল}"
রং: নীল
$ echo "এখনো: [$color]"
এখনো: []
$ echo "রং: ${color:=লাল}"
রং: লাল
$ echo "এখন: [$color]"
এখন: [লাল]
$ debug=1
$ echo "flags: ${debug:+--verbose}"
flags: --verbose
```

স্ক্রিপ্টে ব্যবহার:

```
#!/usr/bin/env bash
dir=${1:-.}                                  # আর্গুমেন্ট না দিলে বর্তমান ডিরেক্টরি
log_level=${LOG_LEVEL:-info}                 # environment variable না থাকলে info
: "${DB_HOST:?DB_HOST সেট করা নেই}"          # না থাকলে এখানেই থেমে যাবে
```

শেষ লাইনের `:` হলো no-op কমান্ড — শুধু expansion টা ঘটানোর জন্য, যাতে চেক হয়।

```
$ ./deploy.sh
./deploy.sh: line 4: DB_HOST: DB_HOST সেট করা নেই
```

৮.৬ অক্ষরের case বদল (bash 4+)

লেখা	কাজ	`s="hello World"`
`${s^}`	প্রথম অক্ষর বড় হাতের	`Hello World`
`${s^^}`	সব বড় হাতের	`HELLO WORLD`
`${s,}`	প্রথম অক্ষর ছোট হাতের	`hello World`
`${s,,}`	সব ছোট হাতের	`hello world`
`${s~~}`	সব অক্ষরের case উল্টো	`HELLO wORLD`

ইউজারের ইনপুট তুলনায় খুব কাজের: `[[ ${answer,,} == y* ]]` → "Y", "yes", "YES" সব মেলে।

৮.৭ আরও কিছু expansion

লেখা	কাজ
`${!var}`	indirect: var এর মান যে নামের variable, তার মান
`${!prefix*}`	`prefix` দিয়ে শুরু হওয়া সব variable-এর নাম
`${var@Q}`	মানটা shell-এ নিরাপদে পুনরায় ব্যবহারযোগ্য quote করা রূপে (bash 4.4+)

```
$ a=hello; ref=a
$ echo "${!ref}"
hello
$ echo "${!BASH_VERS*}"
BASH_VERSINFO BASH_VERSION
```

---

৯. `select` — মেনু তৈরি

`select` bash এর একটা বিশেষ লুপ, যা একটা নম্বরযুক্ত মেনু দেখায়, ইউজারের ইনপুট নেয়, আর বেছে নেওয়া আইটেম variable-এ রাখে।

```
select VAR in ITEM1 ITEM2 ...; do
    ...
done
```

কীভাবে কাজ করে:

1. তালিকাটা নম্বরসহ **stderr**-এ প্রিন্ট করে।
2. `PS3` variable এর মানকে prompt হিসেবে দেখায় (ডিফল্ট `#?`)।
3. ইউজার যা টাইপ করে সেটা `REPLY` variable-এ রাখে।
4. সঠিক নম্বর হলে VAR = ওই আইটেম; ভুল হলে VAR = খালি।
5. body চালায়, তারপর **আবার** prompt দেখায় — `break` না পাওয়া পর্যন্ত চলতেই থাকে।
6. ইউজার খালি Enter দিলে মেনু আবার প্রিন্ট করে; **Ctrl+D** (EOF) দিলে লুপ শেষ হয়।

```
#!/usr/bin/env bash
PS3="তোমার পছন্দের নম্বর দাও: "
options=("ডিস্ক ব্যবহার" "মেমরি" "আপটাইম" "বের হও")

select choice in "${options[@]}"; do
    case $choice in
        "ডিস্ক ব্যবহার") df -h / ;;
        "মেমরি")        free -h ;;
        "আপটাইম")       uptime -p ;;
        "বের হও")       echo "বিদায়!"; break ;;
        *)              echo "ভুল ইনপুট: '$REPLY' — আবার চেষ্টা করো" ;;
    esac
done
```

```
$ ./menu.sh
1) ডিস্ক ব্যবহার
2) মেমরি
3) আপটাইম
4) বের হও
তোমার পছন্দের নম্বর দাও: 3
up 2 hours, 14 minutes
তোমার পছন্দের নম্বর দাও: 9
ভুল ইনপুট: '9' — আবার চেষ্টা করো
তোমার পছন্দের নম্বর দাও: 4
বিদায়!
```

> 💡 **টিপস:** অনেকগুলো আইটেম হলে `select` এগুলো কলামে সাজায়, `COLUMNS` variable অনুযায়ী। আরও সুন্দর TUI মেনু চাইলে `whiptail` (Debian/Ubuntu তে সাধারণত থাকে) বা `dialog` প্যাকেজ দেখতে পারো।

---

১০. `printf` — নিয়ন্ত্রিত, সুন্দর আউটপুট

১০.১ কেন `echo` না?

`echo` সহজ, কিন্তু সমস্যা আছে: `echo -e`, `echo -n` এর আচরণ শেল ও সিস্টেমভেদে আলাদা (dash এর `echo` `-e` চেনে না; `\n` নিজেই interpret করে)। আর variable এর মান যদি `-n` হয়, `echo "$var"` সেটাকে অপশন ভাবে! `printf` সব জায়গায় একই রকম আচরণ করে (POSIX), আর কলাম সাজানো, দশমিক নিয়ন্ত্রণ — সব পারে।

১০.২ সিনট্যাক্স

```
printf FORMAT [ARGUMENTS...]
```

`printf` নিজে থেকে newline দেয় না — `\n` লিখতে হয়।

```
$ printf 'নাম: %s, বয়স: %d\n' "রহিম" 25
নাম: রহিম, বয়স: 25
```

১০.৩ Format specifier

specifier	অর্থ	উদাহরণ	ফলাফল
`%s`	স্ট্রিং	`printf '%s\n' hi`	`hi`
`%d` / `%i`	পূর্ণসংখ্যা	`printf '%d\n' 42`	`42`
`%f`	দশমিক (ডিফল্ট ৬ ঘর)	`printf '%f\n' 3.5`	`3.500000`

specifier	অর্থ	উদাহরণ	ফলাফল
`%.2f`	২ দশমিক ঘর	`printf '%.2f\n' 3.14159`	`3.14`
`%e`	বৈজ্ঞানিক নোটেশন	`printf '%e\n' 12345`	`1.234500e+04`
`%x` / `%X`	হেক্সাডেসিমাল	`printf '%x\n' 255`	`ff`
`%o`	অক্টাল	`printf '%o\n' 8`	`10`
`%c`	প্রথম অক্ষর	`printf '%c\n' hello`	`h`
`%b`	স্ট্রিং, তার ভেতরের `\n` `\t` interpret করে	`printf '%b\n' 'a\tb'`	`a b`
`%q`	shell-safe quote করা (bash)	`printf '%q\n' 'a b'`	`a\ b`
`%%`	আক্ষরিক `%`	`printf '50%%\n'`	`50%`

প্রস্থ (width) ও alignment:

লেখা	অর্থ	`printf '[...]\n'` ফলাফল
`%10s`	১০ ঘর জায়গা, ডানে সাজানো	`[ hello]`
`%-10s`	১০ ঘর, বামে সাজানো	`[hello ]`
`%05d`	৫ ঘর, সামনে শূন্য	`[00042]`
`%8.2f`	মোট ৮ ঘর, ২ দশমিক	`[ 3.14]`
`%.3s`	স্ট্রিং এর প্রথম ৩ অক্ষর	`[hel]`
`%*d`	প্রস্থ আর্গুমেন্ট থেকে নাও	`printf '[%*d]\n' 6 42` → `[ 42]`

Escape sequence (format এর ভেতরে): `\n` newline, `\t` tab, `\\` ব্যাকস্ল্যাশ, `\e` বা `\033` ESC (রং এর জন্য)।

১০.৪ Format পুনর্ব্যবহার — `printf` এর জাদু

আর্গুমেন্ট বেশি থাকলে `printf` format টা **বারবার** ব্যবহার করে যতক্ষণ আর্গুমেন্ট শেষ না হয়:

```
$ printf '%s\n' আম জাম লিচু
আম
জাম
লিচু
$ printf '%-8s %5s\n' নাম নম্বর করিম 85 রহিমা 92
নাম      নম্বর
করিম         85
রহিমা        92
```

এজন্যই অ্যারের প্রতিটা উপাদান আলাদা লাইনে প্রিন্ট করার সেরা উপায়: `printf '%s\n' "${arr[@]}"`।

> ⚠️ **সতর্কতা:** বাংলার মতো multi-byte লেখায় `%-8s` এর প্রস্থ গোনা কিছু সিস্টেমে byte ধরে হয় (bash এর builtin printf এ locale-ভেদে আচরণ ভিন্ন), আর যুক্তাক্ষর/কার চিহ্নের কারণে টার্মিনালে কলাম একটু বাঁকা

> দেখাতে পারে। পারফেক্ট টেবিল দরকার হলে ইংরেজি হেডার বা `column -t` ব্যবহার করো।

১০.৫ সুন্দর রিপোর্ট টেবিল

```
#!/usr/bin/env bash
printf '%-15s %10s %8s\n' "FILESYSTEM" "SIZE" "USE%"
printf '%-15s %10s %8s\n' "---------------" "----------" "--------"
df -h --output=source,size,pcent -x tmpfs -x devtmpfs | tail -n +2 |
while read -r fs size pct; do
    printf '%-15s %10s %8s\n' "$fs" "$size" "$pct"
done
```

```
FILESYSTEM            SIZE     USE%
---------------  ---------- --------
/dev/nvme0n1p2         468G      41%
/dev/nvme0n1p1         1.1G       1%
```

১০.৬ `printf -v` আর তারিখ

```
$ printf -v padded '%03d' 7          # আউটপুট প্রিন্ট না করে variable-এ রাখে (subshell লাগে না)
$ echo "$padded"
007
$ printf -v now '%(%Y-%m-%d %H:%M)T' -1   # bash 4.2+: -1 মানে "এখন"; date কমান্ড fork লাগে না
$ echo "$now"
2026-11-12 10:30
```

রঙিন আউটপুট:

```
RED=$'\e[31m'; GREEN=$'\e[32m'; RESET=$'\e[0m'
printf '%s[OK]%s ব্যাকআপ সফল\n' "$GREEN" "$RESET"
printf '%s[FAIL]%s ডিস্ক পূর্ণ\n' "$RED" "$RESET"
```

---

১১. সব একসাথে: একটা পূর্ণ উদাহরণ

নিচের স্ক্রিপ্টে লুপ, ফাংশন, দুই রকম অ্যারে, স্ট্রিং অপারেশন আর `printf` — সবকিছু আছে। এটা একটা ডিরেক্টরির ফাইলগুলোকে এক্সটেনশন অনুযায়ী গুনে সাইজসহ রিপোর্ট দেয়।

```
#!/usr/bin/env bash
# ext-report.sh — এক্সটেনশন অনুযায়ী ফাইলের সংখ্যা ও মোট সাইজ
set -uo pipefail
shopt -s nullglob dotglob

declare -A count size
```

```
human() {                           # বাইট → মানুষের পড়ার মতো
    local b=$1 units=(B K M G T) i=0
    while (( b >= 1024 && i < ${#units[@]} - 1 )); do
        (( b /= 1024, ++i ))
    done
    printf '%d%s' "$b" "${units[i]}"
}

scan() {
    local dir=${1:-.} f name ext bytes
    for f in "$dir"/*; do
        [[ -f $f ]] || continue
        name=${f##*/}
        if [[ $name == *.* && $name != .* ]]; then
            ext=${name##*.}; ext=${ext,,}
        else
            ext="(none)"
        fi
        bytes=$(stat -c %s -- "$f")
        (( count[$ext]++ ))
        (( size[$ext] += bytes ))
    done
}

scan "${1:-.}"

printf '%-10s %6s %8s\n' EXT FILES SIZE
for ext in "${!count[@]}"; do
    printf '%-10s %6d %8s\n' "$ext" "${count[$ext]}" "$(human "${size[$ext]}")"
done | sort -k2 -rn
```

```
$ ./ext-report.sh ~/Downloads
EXT         FILES     SIZE
pdf            23     148M
jpg            17      41M
zip             4     612M
(none)          2       3K
```

(`stat -c` হলো GNU coreutils এর অপশন — সব প্রধান লিনাক্স ডিস্ট্রোতে চলে।)

---

১২. সংশ্লিষ্ট কীবোর্ড শর্টকাট

লুপ আর ফাংশন প্রায়ই সরাসরি টার্মিনালে লিখে পরীক্ষা করা হয়। এই শর্টকাটগুলো তখন কাজে লাগে (পূর্ণ তালিকা অধ্যায় ৬-এ):

শর্টকাট	কাজ
Ctrl+C	চলমান অসীম লুপ বা কমান্ড থামাও (SIGINT)
Ctrl+D	`read` / `select` কে EOF পাঠাও — মেনু বা ইনপুট লুপ শেষ
Ctrl+Z	চলমান লুপ সাময়িক থামিয়ে ব্যাকগ্রাউন্ডে পাঠাও (পরে `fg`)

শর্টকাট	কাজ
Ctrl+X Ctrl+E	বর্তমান কমান্ড লাইন `$EDITOR` (nano/vim) এ খোলো — লম্বা লুপ আরাম করে লেখার জন্য; সেভ করে বের হলে চলে
Alt+Enter বা লাইনের শেষে `\` + Enter	একাধিক লাইনে কমান্ড লেখা (bash নিজেও `do` / `{` এর পরে `>` prompt দিয়ে পরের লাইন চায়)
↑ (Up)	আগের কমান্ড — মাল্টি-লাইন লুপ bash এক লাইনে (`;` দিয়ে) history-তে রাখে
Ctrl+R	history-তে আগে লেখা লুপ খোঁজো
Tab	ফাংশনের নাম, variable (`$` এর পরে), ফাইলনাম auto-complete

> 💡 **টিপস: Alt+Enter** সব টার্মিনালে কাজ করে না (অনেক টার্মিনাল এটাকে fullscreen বা অন্য কাজে ব্যবহার করে)। নিশ্চিত উপায় হলো `Ctrl+X Ctrl+E`। এটা কাজ করতে `export EDITOR=nano` বা `vim` সেট থাকা ভালো (অধ্যায় ১৩, অধ্যায় ১৪)।

হাতে-কলমে অনুশীলন

একটা অনুশীলন ফোল্ডার বানিয়ে নাও:

```
mkdir -p ~/lab42 && cd ~/lab42
```

ল্যাব ১: লুপের সব রূপ

```
for i in {1..5}; do echo -n "$i "; done; echo
for i in {10..0..2}; do echo -n "$i "; done; echo
n=4; for (( i=1; i<=n; i++ )); do echo -n "$((i*i)) "; done; echo
```

প্রত্যাশিত:

```
1 2 3 4 5
10 8 6 4 2 0
1 4 9 16
```

এবার `n=4; for i in {1..$n}; do echo $i; done` চালিয়ে দেখো কেন এটা কাজ করে না।

ল্যাব ২: ফাইল তৈরি ও নাম বদল

```
touch photo_{1..5}.JPEG "my pic.JPEG"
for f in *.JPEG; do
    new=${f%.JPEG}.jpg
    new=${new// /_}
    mv -v -- "$f" "$new"
```

```
done
ls
```

প্রত্যাশিত `ls`:

```
my_pic.jpg  photo_1.jpg  photo_2.jpg  photo_3.jpg  photo_4.jpg  photo_5.jpg
```

ল্যাব ৩: লাইন-বাই-লাইন পড়া ও subshell ফাঁদ

```
printf 'line one\n  indented\nback\\slash\n' > sample.txt

echo "--- ভুল উপায় (-r, IFS= ছাড়া) ---"
while read line; do echo "[$line]"; done < sample.txt
echo "--- সঠিক উপায় ---"
while IFS= read -r line; do echo "[$line]"; done < sample.txt
```

প্রত্যাশিত:

```
--- ভুল উপায় (-r, IFS= ছাড়া) ---
[line one]
[indented]
[backslash]
--- সঠিক উপায় ---
[line one]
[  indented]
[back\slash]
```

এবার subshell ফাঁদ:

```
c=0; cat sample.txt | while read -r l; do ((++c)); done; echo "pipe: $c"
c=0; while read -r l; do ((++c)); done < sample.txt;     echo "redirect: $c"
```

```
pipe: 0
redirect: 3
```

ল্যাব ৪: ফাংশন লাইব্রেরি

`mylib.sh` ফাইল বানাও:

```
cat > mylib.sh <<'EOF'
log()    { printf '[%(%H:%M:%S)T] %s\n' -1 "$*"; }
is_num() { [[ $1 =~ ^-?[0-9]+$ ]]; }
max() {
    local m=$1 x
    shift
    for x in "$@"; do (( x > m )) && m=$x; done
    echo "$m"
}
EOF
```

ব্যবহার করো:

```
source ./mylib.sh
log "শুরু"
is_num 42 && echo "42 সংখ্যা"
is_num abc || echo "abc সংখ্যা নয়"
echo "সর্বোচ্চ: $(max 3 17 9 42 8)"
type max | head -1
```

```
[10:30:05] শুরু
42 সংখ্যা
abc সংখ্যা নয়
সর্বোচ্চ: 42
max is a function
```

ল্যাব ৫: return এর ২৫৬ সীমা নিজে দেখো

```
f() { return 300; }; f; echo $?
```

```
44
```

ল্যাব ৬: অ্যারে ও associative array

```
langs=(bash python go)
langs+=(rust)
unset 'langs[1]'
declare -p langs
langs=("${langs[@]}")
declare -p langs

declare -A age=([রহিম]=30 [করিম]=25)
age[সালমা]=28
for p in "${!age[@]}"; do printf '%s → %d\n' "$p" "${age[$p]}"; done | sort
echo "মোট মানুষ: ${#age[@]}"
```

প্রত্যাশিত (associative অংশের ক্রম `sort` এর কারণে স্থির):

```
declare -a langs=([0]="bash" [2]="go" [3]="rust")
declare -a langs=([0]="bash" [1]="go" [2]="rust")
করিম → 25
রহিম → 30
সালমা → 28
মোট মানুষ: 3
```

ল্যাব ৭: স্ট্রিং অপারেশন কুইজ

প্রথমে নিজে উত্তর অনুমান করো, তারপর চালাও:

```
u="https://example.com/files/archive.tar.gz"
echo "${u#*://}"        # ?
echo "${u%%/*}"         # ?
echo "${u##*/}"         # ?
echo "${u%.*}"          # ?
echo "${u%%.*}"         # ?
echo "${#u}"            # ?
echo "${u:8:11}"        # ?
echo "${u/https/ftp}"   # ?
```

```
example.com/files/archive.tar.gz
https:
archive.tar.gz
https://example.com/files/archive.tar
https://example
40
example.com
ftp://example.com/files/archive.tar.gz
```

ল্যাব ৮: মিনি প্রজেক্ট — সার্ভার হেলথ মেনু

`health.sh` লেখো যাতে থাকবে: (১) `select` মেনু, (২) প্রতিটা অপশনের জন্য আলাদা ফাংশন, (৩) একটা associative array-তে চেক করার সার্ভিসের নাম→পোর্ট, (৪) `printf` দিয়ে টেবিল।

```
#!/usr/bin/env bash
declare -A svc=([ssh]=22 [http]=80 [https]=443)

check_ports() {
    local name state
    printf '%-8s %-6s %s\n' SERVICE PORT STATE
    for name in "${!svc[@]}"; do
        if ss -tln | grep -q ":${svc[$name]}\b"; then state=LISTEN; else state=closed;
fi
        printf '%-8s %-6s %s\n' "$name" "${svc[$name]}" "$state"
    done
}
show_disk() { df -h / | tail -1 | awk '{print "রুট ডিস্ক ব্যবহার: " $5}'; }
show_mem()  { free -h | awk '/^Mem:/ {print "ফ্রি মেমরি: " $7}'; }

PS3="> "
select opt in "পোর্ট চেক" "ডিস্ক" "মেমরি" "বের হও"; do
    case $REPLY in
        1) check_ports ;;
        2) show_disk ;;
        3) show_mem ;;
        4) break ;;
        *) echo "১-৪ এর মধ্যে বেছে নাও" ;;
    esac
done
```

চালাও: `chmod +x health.sh && ./health.sh`। (`ss` এর বিস্তারিত অধ্যায় ৩৭-এ।)

চ্যালেঞ্জ: একটা নতুন অপশন যোগ করো যা `/var/log` এর সবচেয়ে বড় ৩টা ফাইল দেখায় (`du` আর `sort` দিয়ে)।

সারসংক্ষেপ

বিষয়	সিনট্যাক্স / কমান্ড	মনে রাখার কথা
তালিকার উপর লুপ	`for x in a b c; do ...; done`	তালিকা লুপের আগে একবারে expand হয়
Range	`{1..10}`, `{0..100..5}`, `{01..10}`	variable চলে না
variable range	`for ((i=1; i<=n; i++))` / `seq 1 "$n"`	
ফাইলের উপর	`for f in *.txt` + `"$f"`	`shopt -s nullglob`
আর্গুমেন্টের উপর	`for a in "$@"`	
শর্ত সত্য থাকা পর্যন্ত	`while CMD; do ...; done`	exit status 0 = চলবে
শর্ত সত্য না হওয়া পর্যন্ত	`until CMD; do ...; done`	
অসীম লুপ	`while true` / `while :`	Ctrl+C
লাইন পড়া	`while IFS= read -r line; do ...; done < file`	`|| [[ -n $line ]]` শেষ লাইনের জন্য
কমান্ড আউটপুট পড়া	`done < <(cmd)`	পাইপ = subshell, variable হারায়
লুপ নিয়ন্ত্রণ	`break [N]`, `continue [N]`	
ফাংশন	`name() { ...; }`	ব্যবহারের আগে ডিফাইন
আর্গুমেন্ট	`$1 $# "$@"`, `shift`	`$0` স্ক্রিপ্টের নাম
status ফেরত	`return 0..255`	`$?`, `if func`
ডেটা ফেরত	`echo` + `x=$(func)`	subshell
লোকাল variable	`local x`	ডিফল্ট সব global
nameref	`local -n ref=$1`	bash 4.3+
Indexed array	`a=(x y)`, `${a[0]}`, `"${a[@]}"`, `${#a[@]}`, `${!a[@]}`, `a+=(z)`	`unset 'a[i]'` ফাঁকা রাখে
ফাইল → অ্যারে	`mapfile -t a < file`	
Associative array	`declare -A m; m[key]=val`	আগে declare বাধ্যতামূলক; ক্রম নেই
key আছে?	`[[ -v m[key] ]]`	
দৈর্ঘ্য	`${#var}`	

বিষয়	সিনট্যাক্স / কমান্ড	মনে রাখার কথা
Substring	`${var:0:3}`, `${var: -3}`	space!
Replace	`${var/a/b}`, `${var//a/b}`	glob pattern
শুরু কাটা	`${var#pat}`, `${var##pat}`	`##*/` = basename
শেষ কাটা	`${var%pat}`, `${var%%pat}`	`%.*` = এক্সটেনশন বাদ
ডিফল্ট	`${var:-def}`, `${var:=def}`, `${var:+alt}`, `${var:?err}`	
Case	`${var^^}`, `${var,,}`, `${var^}`	bash 4+
মেনু	`select x in ...; do ...; done`	`PS3`, `REPLY`, `break`
ফরম্যাট আউটপুট	`printf '%-10s %5.2f\n' ...`	নিজে newline দেয় না; format পুনর্ব্যবহার
variable এ printf	`printf -v var FMT ...`	
এডিটরে কমান্ড	**Ctrl+X Ctrl+E**	

নিজেকে যাচাই করো

1. `n=5; for i in {1..$n}; do echo $i; done` কী প্রিন্ট করবে, আর কেন? সঠিকভাবে ১ থেকে n পর্যন্ত লুপ কীভাবে লিখবে?
2. `while IFS= read -r line` — এখানে `IFS=` আর `-r` না দিলে কী কী সমস্যা হতে পারে?
3. নিচের কোড শেষে `total` কত দেখাবে, কেন, আর কীভাবে ঠিক করবে?

   ```
   total=0
   ls | while read -r f; do (( ++total )); done
   echo "$total"
   ```

4. একটা ফাংশন থেকে "ঢাকা" স্ট্রিংটা কীভাবে ফেরত নেবে? `return "ঢাকা"` কেন কাজ করবে না?
5. `local` না লিখলে ফাংশনের ভেতরের variable-এর কী হয়?
6. `declare -A` না দিয়ে `m[apple]=1; m[mango]=2; echo ${m[apple]}` চালালে কী আউটপুট আসবে, কেন?
7. `f="/srv/www/index.html.bak"` হলে `${f##*/}`, `${f%.*}`, `${f%%.*}`, `${f##*.}` — প্রত্যেকটার মান কী?
8. `${name:-4}` আর `${name: -4}` এর পার্থক্য কী?
9. `select` লুপে ইউজার মেনুর বাইরের নম্বর দিলে variable-এ কী থাকে? ইউজারের আসল টাইপ করা ইনপুট কোথায় পাবে?
10. `printf '%s-%s\n' a b c d e` এর আউটপুট কী হবে?

▼ **উত্তর দেখো**

1. আউটপুট হবে আক্ষরিক `{1..5}` (একবার)। কারণ brace expansion হয় variable expansion-এর **আগে** — তখন `$n` এখনো `5` হয়নি, তাই `{1..$n}` বৈধ range না, যেমন আছে তেমন থাকে। সঠিক: `for ((i=1; i<=n; i++))` অথবা `for i in $(seq 1 "$n")`।

2. `IFS=` না দিলে লাইনের শুরু ও শেষের space/tab কেটে যাবে (indentation নষ্ট)। `-r` না দিলে ব্যাকস্ল্যাশ escape হিসেবে পড়া হবে — `a\b` হবে `ab`, আর লাইনের শেষে `\` থাকলে পরের লাইনের সাথে জোড়া লাগবে।

3. `0` দেখাবে। পাইপের ডান অংশ (`while` লুপ) subshell-এ চলে, তার `total` পরিবর্তন মূল শেলে ফেরে না। ঠিক করার উপায়: `while read -r f; do ...; done < <(ls)` (process substitution), অথবা আরও ভালো — `ls` এর বদলে glob: `files=(*); total=${#files[@]}`।

4. `echo "ঢাকা"` দিয়ে stdout-এ লেখো, আর কলার ধরবে `city=$(func)` দিয়ে (অথবা nameref `local -n` দিয়ে)। `return` শুধু 0–255 পূর্ণসংখ্যা exit status নেয়; স্ট্রিং দিলে bash error দেয় ("numeric argument required")।

5. সেটা **global** হয়ে যায় — ফাংশন শেষ হওয়ার পরেও থাকে, আর বাইরে একই নামের variable থাকলে সেটাকে বদলে ফেলে।

6. আউটপুট `2`। `declare -A` ছাড়া `m` indexed array; `apple` আর `mango` arithmetic context-এ unset variable হিসেবে 0 ধরা হয়, তাই দুটোই `m[0]` — পরেরটা (`2`) আগেরটাকে মুছে দেয়।

7. `${f##*/}` → `index.html.bak`; `${f%.*}` → `/srv/www/index.html`; `${f%%.*}` → `/srv/www/index`; `${f##*.}` → `bak`।

8. `${name:-4}` হলো default-value অপারেটর: name খালি/unset হলে `4`, নাহলে name এর পুরো মান। `${name: -4}` (space সহ) হলো substring: name এর শেষ ৪টা অক্ষর।

9. variable খালি (empty string) হয়ে যায়। ইউজার আসলে যা টাইপ করেছে তা থাকে `REPLY` variable-এ।

10. format টা আর্গুমেন্ট শেষ না হওয়া পর্যন্ত পুনর্ব্যবহার হয়; শেষ রাউন্ডে অনুপস্থিত আর্গুমেন্ট খালি ধরা হয়:

    ```
    a-b
    c-d
    e-
    ```

অধ্যায় ৪৩: শেল স্ক্রিপ্টিং — পর্ব ৩ (অ্যাডভান্সড ও বাস্তব প্রজেক্ট)

আগের দুই পর্বে (অধ্যায় ৪১ ও অধ্যায় ৪২) তুমি variable, `if`, loop, function, array শিখেছ। এখন তুমি স্ক্রিপ্ট লিখতে পারো। কিন্তু "লিখতে পারা" আর "এমন স্ক্রিপ্ট লেখা যেটা রাত ৩টায় cron থেকে চলবে, ভুল হলে নিজেই থামবে, লগ রাখবে, দুইবার একসাথে চলবে না, আর পরের মানুষ পড়ে বুঝবে" — এ দুটো আলাদা জিনিস। এই অধ্যায় দ্বিতীয়টা শেখায়। এটাই একজন শখের স্ক্রিপ্ট-লেখক আর একজন প্রফেশনাল sysadmin/DevOps engineer-এর পার্থক্য।

এই অধ্যায়ে যা শিখবে

- `set -euo pipefail` — "strict mode" কী, প্রতিটি অংশ ভেতরে কীভাবে কাজ করে, আর এর ফাঁদগুলো কোথায়
- `trap` দিয়ে signal ধরা, cleanup করা, `EXIT` ও `ERR` trap
- ডিবাগিং: `bash -n`, `bash -x`, `set -x`, `PS4`, `BASH_XTRACEFD`
- `shellcheck` — স্ক্রিপ্টের "বানান-ব্যাকরণ পরীক্ষক"
- `getopts` দিয়ে প্রফেশনাল কমান্ড-লাইন অপশন পার্স করা
- lock ফাইল (`flock`, `mkdir` কৌশল) — একই স্ক্রিপ্ট দুইবার একসাথে না চলা
- লগিং ফাংশন, `logger` দিয়ে syslog/journald-এ লেখা
- রঙিন আউটপুট (ANSI escape code ও `tput`)
- exit code-এর কনভেনশন
- নিরাপদ temp ফাইল (`mktemp`), `readonly`, `local`, `main "$@"` প্যাটার্ন
- ভালো অভ্যাসের চেকলিস্ট ও একটি পুনর্ব্যবহারযোগ্য স্ক্রিপ্ট টেমপ্লেট
- ৫টি সম্পূর্ণ বাস্তব প্রজেক্ট: ব্যাকআপ, ডিস্ক অ্যালার্ট, লগ বিশ্লেষক, bulk ইউজার তৈরি, সার্ভিস হেলথ চেক — লাইন-বাই-লাইন ব্যাখ্যাসহ

৪৩.১ কেন "অ্যাডভান্সড" স্ক্রিপ্টিং দরকার?

ধরো তুমি একটা ছোট স্ক্রিপ্ট লিখলে:

```
#!/bin/bash
cd /var/www/old_site
rm -rf *
```

উদ্দেশ্য: পুরোনো সাইটের ফাইল মুছে ফেলা। কিন্তু যদি `/var/www/old_site` ফোল্ডারটা না থাকে? `cd` ব্যর্থ হবে, একটা error দেখাবে — কিন্তু bash **থামবে না**। পরের লাইন `rm -rf *` চলবে **বর্তমান ডিরেক্টরিতে** —

হয়তো সেটা `/` বা তোমার home! এরকম ঘটনায় বাস্তবে কোম্পানির পুরো সার্ভার মুছে গেছে।

> 📝 **মনে রাখো:** bash-এর ডিফল্ট আচরণ হলো "কোনো কমান্ড ফেল করলেও চুপচাপ পরের লাইনে চলে যাও।" ইন্টারঅ্যাকটিভ টার্মিনালে এটা সুবিধাজনক, কিন্তু স্ক্রিপ্টে এটা বিপজ্জনক।

উপমা: একজন রাঁধুনির কথা ভাবো যে রেসিপি অনুসরণ করছে। "ডিম ফাটাও" ধাপে দেখল ডিম নেই — কিন্তু সে থামল না, পরের ধাপে "ফাটানো ডিম কড়াইয়ে দাও" করতে গিয়ে খালি বাটি কড়াইয়ে ঢেলে দিল। অ্যাডভান্সড স্ক্রিপ্টিং মানে রাঁধুনিকে শেখানো: "কোনো ধাপ ব্যর্থ হলে থামো, রান্নাঘর গুছিয়ে রাখো (cleanup), আর খাতায় লিখে রাখো কী হয়েছিল (logging)।"

৪৩.২ Strict Mode: `set -euo pipefail`

প্রায় সব প্রফেশনাল bash স্ক্রিপ্টের শুরুতে তুমি এই লাইনটা দেখবে:

```
#!/usr/bin/env bash
set -euo pipefail
```

এটা আসলে তিনটা আলাদা সেটিং একসাথে: `-e`, `-u`, আর `-o pipefail`। একটা একটা করে দেখি।

৪৩.২.১ `set -e` (errexit) — ফেল করলেই থামো

`set -e` চালু থাকলে, কোনো "simple command" যদি non-zero exit status ফেরত দেয়, bash তৎক্ষণাৎ স্ক্রিপ্ট বন্ধ করে দেয়।

```
#!/bin/bash
set -e
echo "শুরু"
ls /nai_emon_folder
echo "এটা কখনো প্রিন্ট হবে না"
```

```
$ ./test.sh
শুরু
ls: cannot access '/nai_emon_folder': No such file or directory
$ echo $?
2
```

স্ক্রিপ্টটা `ls`-এর exit code (2) নিয়েই বের হয়ে গেল।

ভেতরে কীভাবে কাজ করে: প্রতিটি কমান্ড শেষ হলে bash তার exit status `$?`-তে রাখে। `errexit` অপশন চালু থাকলে bash প্রতিবার চেক করে — "এই কমান্ডটা কি এমন প্রসঙ্গে চলেছে যেখানে ফেল করা প্রত্যাশিত?" যদি না হয়, আর status ≠0, তাহলে `exit` করে।

কোন ক্ষেত্রে `set -e` কাজ করে না (খুব গুরুত্বপূর্ণ!):

প্রসঙ্গ	উদাহরণ	কেন থামে না
`if` / `while` / `until` -এর শর্তে	`if grep -q x file; then`	এখানে ফেল করাটা "প্রশ্নের উত্তর", error না
`&&` বা `||` তালিকায় (শেষ কমান্ড বাদে)	`false && echo hi`	bash ধরে নেয় তুমি নিজে ফেল সামলাচ্ছ
`!` দিয়ে উল্টানো কমান্ড	`! grep x file`	
pipeline-এর শেষটা বাদে বাকিগুলো	`false | true`	pipeline-এর status = শেষ কমান্ডের (যদি না `pipefail` থাকে)
ওপরের কোনো প্রসঙ্গে ডাকা function-এর ভেতরে	`if myfunc; then`	পুরো function-এর ভেতরে `-e` নিষ্ক্রিয় হয়ে যায়

> ⚠️ **সতর্কতা — `((count++))` ফাঁদ:** `count=0` থাকলে `((count++))` এর মান হয় 0 (post-increment পুরোনো মান দেয়), আর `(( ))` এর ভেতরের মান 0 হলে exit status হয় 1। ফলে `set -e` স্ক্রিপ্টটাকে চুপচাপ মেরে ফেলবে! নিরাপদ বিকল্প: `count=$((count + 1))` বা `((count += 1))` বা `((++count))`। (শেষ দুটোও মান 0 হলে ফেল করবে, তাই `count=$((count+1))` সবচেয়ে নিরাপদ।)

> ⚠️ **সতর্কতা — `local var=$(cmd)` ফাঁদ:** `local x=$(false)` এর exit status `local` -এর (যেটা সফল = 0), `false` -এর না। তাই `set -e` ধরবে না। সমাধান: আলাদা লাইনে লেখো:
>
> ```
> local x
> x=$(some_command)
> ```
>
> (shellcheck এটাকে SC2155 হিসেবে ধরিয়ে দেয়।)

যে কমান্ড ফেল করতে পারে এবং তুমি সেটা মেনে নিতে চাও, সেখানে স্পষ্টভাবে বলো:

```
rm -f /tmp/maybe_exists          # -f নিজেই ফেল না করার ব্যবস্থা
grep -q "pattern" file || true   # ফেল হলেও চলবে
if ! ping -c1 -W2 host >/dev/null; then
    echo "host down"
fi
```

৪৩.২.২ `set -u` (nounset) — অচেনা variable মানেই ভুল

```
#!/bin/bash
set -u
TARGET_DIR="/tmp/data"
rm -rf "$TARGTE_DIR/"*      # বানান ভুল!
```

`set -u` ছাড়া `$TARGTE_DIR` হয়ে যেত খালি স্ট্রিং, আর কমান্ডটা হতো `rm -rf /*` — সর্বনাশ! `set -u` থাকলে:

```
$ ./test.sh
./test.sh: line 4: TARGTE_DIR: unbound variable
```

ঐচ্ছিক variable-এর জন্য default দাও:

```
name="${1:-guest}"           # $1 না থাকলে "guest"
: "${CONFIG_FILE:=/etc/app.conf}"   # না থাকলে সেট করে দাও
if [[ -n "${DEBUG:-}" ]]; then ...   # সেট আছে কি না নিরাপদে চেক
```

> 💡 **টিপস:** bash 4.4-এর আগে `set -u` থাকলে খালি array-এ `"${arr[@]}"` লিখলে error দিত। Ubuntu 24.04 (bash 5.2), Debian 12 (bash 5.2), RHEL 9 (bash 5.1)-এ এই সমস্যা নেই।

৪৩.২.৩ `set -o pipefail` — pipeline-এর লুকানো ব্যর্থতা ধরো

ডিফল্টে একটা pipeline-এর exit status হলো **শুধু শেষ কমান্ডের** status:

```
$ cat /nai_file | sort
cat: /nai_file: No such file or directory
$ echo $?
0
```

`sort` সফল হয়েছে (খালি ইনপুট সাজাতে পেরেছে), তাই পুরো pipeline "সফল"! `pipefail` চালু করলে pipeline-এর status হয় **ডান দিক থেকে প্রথম non-zero status** (সব সফল হলে 0):

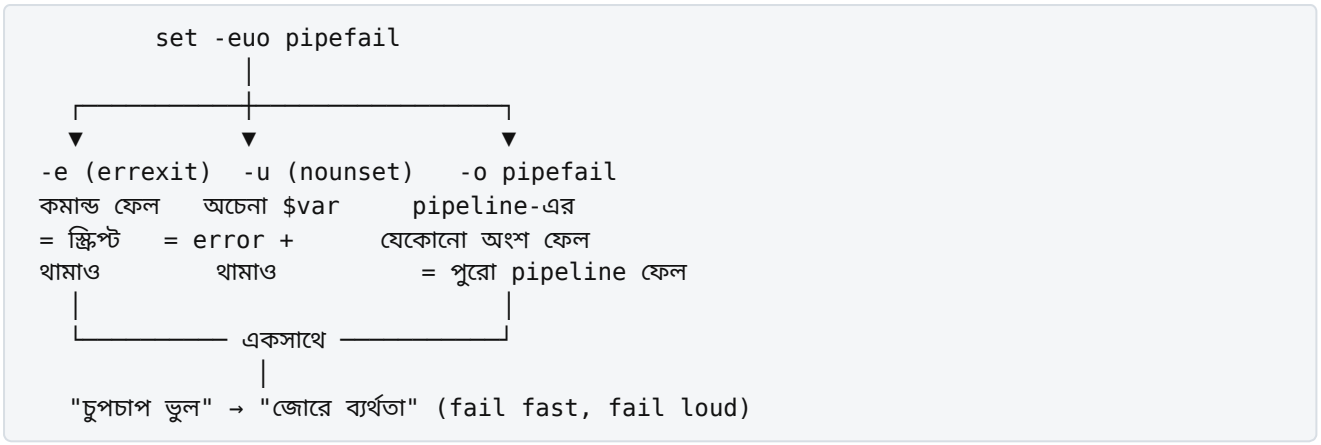
```
$ set -o pipefail
$ cat /nai_file | sort
cat: /nai_file: No such file or directory
$ echo $?
1
```

কোন অংশ ফেল করেছে জানতে `PIPESTATUS` array দেখো:

```
$ false | true | grep -q x </dev/null; echo "${PIPESTATUS[@]}"
1 0 1
```

> ⚠️ **সতর্কতা — SIGPIPE ফাঁদ:** `some_cmd | head -n1` — `head` প্রথম লাইন পড়েই বন্ধ হয়ে যায়, তখন `some_cmd` লিখতে গিয়ে SIGPIPE পায় এবং 141 (128+13) status দিয়ে মরে। `pipefail` থাকলে পুরো pipeline "ফেল"! যেমন `yes | head -n1` pipefail-এ 141 দেয়। এমন জায়গায় `|| true` দাও বা ডিজাইন বদলাও।

৪৩.২.৪ ভেতরের চিত্র



বর্তমান অপশন দেখতে:

```
$ echo $-
himBHs
$ set -o | grep -E 'errexit|nounset|pipefail'
errexit         off
nounset         off
pipefail        off
```

`$-` এ `e` বা `u` অক্ষর থাকলে সেগুলো চালু। কোনো অপশন বন্ধ করতে `+` দাও: `set +e` (সাময়িকভাবে errexit বন্ধ), পরে আবার `set -e`।

> 📝 **মনে রাখো:** strict mode কোনো জাদু না — এটা একটা "সেফটি নেট"। এর ফাঁকগুলো (if-এর ভেতর function, `local x=$(..)`, `((i++))`) জানা থাকলেই এটা সত্যিকার কাজে আসে।

আরেকটা কাজের অপশন হলো `IFS`। অনেক টেমপ্লেটে দেখবে:

```
IFS=$'\n\t'
```

এতে word splitting শুধু newline আর tab-এ হয়, space-এ না — ফাইলনামে space থাকলে কম বিপদ। তবে সবসময় variable quote করা (`"$var"`) এর চেয়েও বড় সুরক্ষা।

৪৩.৩ `trap` — signal ধরা ও cleanup

৪৩.৩.১ Signal কী?

Signal হলো kernel বা অন্য process থেকে একটা process-কে পাঠানো ছোট্ট "নোটিশ"। (অধ্যায় ২৪-এ বিস্তারিত আছে।) স্ক্রিপ্টের জন্য গুরুত্বপূর্ণগুলো:

Signal	নম্বর	কখন আসে	trap করা যায়?
`SIGHUP`	1	টার্মিনাল বন্ধ হলে	হ্যাঁ
`SIGINT`	2	`Ctrl+C` চাপলে	হ্যাঁ
`SIGQUIT`	3	`Ctrl+\` চাপলে	হ্যাঁ
`SIGKILL`	9	`kill -9`	**না**
`SIGTERM`	15	`kill` (ডিফল্ট), `systemctl stop`	হ্যাঁ
`SIGSTOP`	19	`kill -STOP`	**না**
`SIGTSTP`	20	`Ctrl+Z`	হ্যাঁ

এগুলো ছাড়াও bash-এর নিজস্ব কিছু "pseudo-signal" আছে:

Pseudo-signal	কখন চলে
`EXIT`	স্ক্রিপ্ট যেভাবেই শেষ হোক (স্বাভাবিক, `exit`, `set -e`, বা trap করা signal-এ)
`ERR`	কোনো কমান্ড non-zero দিলে (`set -e` যে নিয়মে থামে, প্রায় সেই নিয়মেই)
`DEBUG`	প্রতিটি কমান্ডের **আগে**
`RETURN`	function বা `source` করা ফাইল শেষ হলে

৪৩.৩.২ trap-এর syntax

```
trap 'কমান্ড' SIGNAL [SIGNAL ...]
trap - SIGNAL        # ডিফল্ট আচরণে ফেরাও
trap '' SIGNAL       # signal উপেক্ষা করো (ignore)
trap -p              # বর্তমান trap গুলো দেখাও
trap -l              # সব signal-এর নাম ও নম্বর
```

উপমা: trap হলো বাড়ির "ইমার্জেন্সি প্ল্যান"। আগুন লাগলে (signal) কী করবে — গ্যাস বন্ধ, দরজা খোলা — তা আগে থেকে লিখে রাখা।

৪৩.৩.৩ সবচেয়ে কাজের প্যাটার্ন: EXIT trap দিয়ে cleanup

```
#!/usr/bin/env bash
set -euo pipefail

tmpdir=$(mktemp -d)
cleanup() {
    rm -rf -- "$tmpdir"
    echo "temp ফোল্ডার মুছে ফেলা হয়েছে"
}
trap cleanup EXIT

echo "কাজ চলছে $tmpdir এ"
cp /etc/hosts "$tmpdir/"
```

```
false            # ইচ্ছাকৃত ব্যর্থতা — set -e স্ক্রিপ্ট থামাবে
echo "এটা আসবে না"
```

```
$ ./demo.sh
কাজ চলছে /tmp/tmp.k3Jd8sQw1x এ
temp ফোল্ডার মুছে ফেলা হয়েছে
```

স্ক্রিপ্ট ফেল করলেও cleanup চলেছে। এটাই EXIT trap-এর শক্তি।

৪৩.৩.৪ exit code রক্ষা করা ও signal-এ ঠিকমতো বের হওয়া

```
cleanup() {
    local rc=$?          # প্রথম লাইনেই exit code ধরে রাখো!
    rm -rf -- "$tmpdir"
    exit "$rc"
}
trap cleanup EXIT
trap 'exit 130' INT     # Ctrl+C → 130 (128+2)
trap 'exit 143' TERM    # kill → 143 (128+15)
```

INT/TERM trap-এ `exit` ডাকলে EXIT trap-ও চলবে, তাই cleanup একবারই লিখতে হয়।

> 💡 **টিপস:** bash-এ শুধু `trap cleanup EXIT` দিলেই সাধারণত `set -e` ব্যর্থতা ও স্বাভাবিক শেষ — দুটোতেই cleanup হয়। কিন্তু SIGINT/SIGTERM-এ bash কিছু ক্ষেত্রে EXIT trap চালায় না (অন্য shell-এ তো নয়ই), তাই INT/TERM আলাদাভাবে ধরে `exit` করা সবচেয়ে নির্ভরযোগ্য।

৪৩.৩.৫ ERR trap — কোথায় ফেল করল জানো

```
#!/usr/bin/env bash
set -eEuo pipefail     # -E (errtrace) — function-এর ভেতরেও ERR trap উত্তরাধিকার পাবে

on_error() {
    local rc=$? line=$1
    echo "ERROR: লাইন $line এ কমান্ড ফেল করেছে: '$BASH_COMMAND' (exit $rc)" >&2
}
trap 'on_error $LINENO' ERR

load_data() {
    cat /nai/file
}
load_data
```

```
$ ./err.sh
cat: /nai/file: No such file or directory
ERROR: লাইন 9 এ কমান্ড ফেল করেছে: 'cat /nai/file' (exit 1)
```

- `$LINENO` — বর্তমান লাইন নম্বর

- `$BASH_COMMAND` — এই মুহূর্তে চলা কমান্ডের লেখা
- `-E` (`set -o errtrace`) না দিলে function, command substitution ও subshell-এর ভেতরে ERR trap কাজ করে না।

> ⚠️ **সতর্কতা:** `SIGKILL` (`kill -9`) আর `SIGSTOP` কখনো trap করা যায় না। kernel সরাসরি process-কে মেরে ফেলে — কোনো cleanup হয় না। তাই গুরুত্বপূর্ণ জিনিস (যেমন lock) এমনভাবে ডিজাইন করো যেন process মরলে নিজে থেকেই মুক্ত হয় (দেখো ৪৩.৬, `flock`)।

৪৩.৩.৬ trap দিয়ে Ctrl+C উপেক্ষা

```
trap '' INT          # এখন Ctrl+C কাজ করবে না
critical_update
trap - INT           # আবার স্বাভাবিক
```

যেমন ডাটাবেস migration-এর মাঝখানে কেউ ভুলে Ctrl+C চাপলে যেন অর্ধেক কাজ না হয়।

৪৩.৪ ডিবাগিং — স্ক্রিপ্টের ভেতরে উঁকি দেওয়া

৪৩.৪.১ ডিবাগিং টুলের তালিকা

কমান্ড/অপশন	কী করে
`bash -n script.sh`	শুধু syntax পরীক্ষা, চালায় না (noexec)
`bash -v script.sh`	প্রতিটি লাইন পড়ার সাথে সাথে হুবহু প্রিন্ট করে (verbose)
`bash -x script.sh`	প্রতিটি কমান্ড expand হওয়ার পর `+` দিয়ে প্রিন্ট করে (xtrace)
`set -x` / `set +x`	স্ক্রিপ্টের নির্দিষ্ট অংশে trace চালু/বন্ধ
`PS4`	trace লাইনের শুরুর prefix (ডিফল্ট `+ `)
`BASH_XTRACEFD`	trace আউটপুট কোন file descriptor-এ যাবে
`trap '...' DEBUG`	প্রতিটি কমান্ডের আগে কিছু চালাও
`caller`	function কে কোথা থেকে ডেকেছে

৪৩.৪.২ `bash -n` — চালানোর আগেই syntax ভুল ধরো

```
$ cat bad.sh
#!/bin/bash
if [ -f /etc/hosts ]; then
    echo "আছে"
$ bash -n bad.sh
bad.sh: line 4: syntax error: unexpected end of file
```

`fi` ভুলে গেছ — চালানোর আগেই ধরা পড়ল।

৪৩.৪.৩ `bash -x` — প্রতিটি ধাপ দেখো

```
#!/bin/bash
name="world"
for i in 1 2; do
    echo "hello $name $i"
done
```

```
$ bash -x hello.sh
+ name=world
+ for i in 1 2
+ echo 'hello world 1'
hello world 1
+ for i in 1 2
+ echo 'hello world 2'
hello world 2
```

`+` দিয়ে শুরু লাইনগুলো trace — variable expand হওয়ার **পরের** আসল কমান্ড। এখানেই বোঝা যায় `$name` আসলে কী মান পেয়েছিল। nested subshell-এ `++`, `+++` দেখবে (গভীরতা অনুযায়ী `PS4`-এর প্রথম অক্ষর পুনরাবৃত্তি হয়)।

৪৩.৪.৪ নির্দিষ্ট অংশে trace

```
echo "এই অংশ স্বাভাবিক"
set -x
result=$(compute_something "$input")
set +x
echo "আবার স্বাভাবিক"
```

৪৩.৪.৫ `PS4` দিয়ে তথ্যবহুল trace

ডিফল্ট `+` থেকে কিছু বোঝা যায় না কোন ফাইলের কোন লাইন। `PS4` বদলাও:

```
export PS4='+ ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: '
```

```
$ bash -x deploy.sh
+ deploy.sh:5:main: target=/srv/app
+ deploy.sh:12:copy_files: cp -r build/. /srv/app
```

> ⚠️ **সতর্কতা:** `PS4` সিঙ্গেল কোটে লেখো, যাতে `$LINENO` ইত্যাদি প্রতিবার trace-এর সময় expand হয়, সেট করার সময় একবার না।

আরও সময়সহ:

```
export PS4='+ $(date +%T.%N | cut -c1-12) ${LINENO}: '
```

(প্রতিটি লাইনে `date` চলবে, তাই ধীর হবে — শুধু পারফরম্যান্স খুঁজতে ব্যবহার করো। bash 5-এ `$EPOCHREALTIME` আরও দ্রুত বিকল্প।)

৪৩.৪.৬ trace আলাদা ফাইলে পাঠানো

```
exec 5> /tmp/debug.log
BASH_XTRACEFD=5
set -x
```

এখন trace যাবে `/tmp/debug.log`-এ, আর স্ক্রিপ্টের সাধারণ stderr পরিষ্কার থাকবে।

৪৩.৪.৭ DEBUG mode সুইচ

প্রফেশনাল স্ক্রিপ্টে প্রায়ই এভাবে থাকে:

```
[[ "${DEBUG:-0}" == "1" ]] && set -x
```

```
$ DEBUG=1 ./myscript.sh
```

> ⚠️ **সতর্কতা:** `set -x` পাসওয়ার্ড, token সহ সব কিছু প্রিন্ট করে! গোপন তথ্য নিয়ে কাজের সময় `set +x` করে নাও, আর trace লগ কখনো শেয়ার করার আগে পরিষ্কার করো।

৪৩.৫ ShellCheck — স্ক্রিপ্টের "ব্যাকরণ শিক্ষক"

`shellcheck` হলো একটা static analysis টুল — স্ক্রিপ্ট না চালিয়েই পড়ে বলে দেয় কোথায় bug আছে বা হতে পারে। উপমা: Word-এর বানান-চেকারের মতো, কিন্তু শুধু বানান না — ব্যাকরণ আর অর্থের ভুলও ধরে।

ইনস্টল

```
sudo apt install shellcheck          # Ubuntu / Debian
sudo dnf install ShellCheck          # Fedora (RHEL 9-এ EPEL repo লাগবে)
```

ব্যবহার

```
$ cat example.sh
#!/bin/bash
files=$(ls *.txt)
for f in $files; do
    cp $f /backup/
```

```
done
cd /some/dir
echo "done"
```

```
$ shellcheck example.sh

In example.sh line 2:
files=$(ls *.txt)
       ^-------^ SC2012 (info): Use find instead of ls to better handle non-
alphanumeric filenames.

In example.sh line 4:
    cp $f /backup/
       ^-- SC2086 (info): Double quote to prevent globbing and word splitting.

Did you mean:
    cp "$f" /backup/

In example.sh line 6:
cd /some/dir
^----------^ SC2164 (warning): Use 'cd ... || exit' or 'cd ... || return' in case cd
fails.
```

(সঠিক আউটপুটের ফরম্যাট সংস্করণভেদে সামান্য আলাদা হতে পারে।)

সবচেয়ে পরিচিত সতর্কতা

কোড	মানে	সমাধান
SC2086	variable quote করা হয়নি	`"$var"`
SC2164	`cd` ফেল করলে কী হবে?	`cd dir || exit 1`
SC2155	`local x=$(cmd)` exit code লুকায়	আলাদা লাইনে declare ও assign
SC2034	variable তৈরি হয়েছে কিন্তু ব্যবহার হয়নি	মুছে ফেলো বা বানান ঠিক করো
SC2046	`$(...)` quote করা হয়নি	`"$(...)"`
SC2006	পুরোনো backtick `` `cmd` ``	`$(cmd)`
SC2002	অপ্রয়োজনীয় `cat` (useless use of cat)	`grep x file`
SC2068	`$@` quote করা হয়নি	`"$@"`
SC1090/SC1091	`source` করা ফাইল খুঁজে পাচ্ছে না	`# shellcheck source=path`

গুরুত্বপূর্ণ অপশন

অপশন	কাজ
`-s bash` / `-s sh`	কোন shell হিসেবে চেক করবে
`-e SC2086`	নির্দিষ্ট সতর্কতা বাদ দাও
`-S warning`	ন্যূনতম severity (error, warning, info, style)

অপশন	কাজ
`-x`	`source` করা ফাইলও অনুসরণ করো
`-f gcc` / `-f diff`	আউটপুট ফরম্যাট (diff ফরম্যাট দিয়ে auto-fix patch পাওয়া যায়)

নির্দিষ্ট লাইনে ইচ্ছাকৃতভাবে সতর্কতা বন্ধ করতে, ঠিক ওপরের লাইনে:

```
# shellcheck disable=SC2086
echo $intentionally_unquoted
```

> 💡 **টিপস:** VS Code-এ "ShellCheck" extension, আর Vim-এ ALE plugin লাগালে লেখার সময়ই লাল দাগ দেখাবে। CI pipeline-এ `shellcheck *.sh` যোগ করে দাও — খারাপ স্ক্রিপ্ট merge-ই হবে না।

৪৩.৬ `getopts` দিয়ে কমান্ড-লাইন অপশন পার্স

তুমি `ls -la`, `tar -czf file.tgz` দেখেছ। তোমার স্ক্রিপ্টও এমন অপশন নিতে পারে। bash-এর built-in `getopts` এই কাজ করে।

৪৩.৬.১ মূল ধারণা

```
while getopts "OPTSTRING" opt; do
    case "$opt" in
        ...
    esac
done
shift $((OPTIND - 1))
```

- **OPTSTRING:** কোন অক্ষরগুলো বৈধ অপশন। কোনো অক্ষরের পরে `:` থাকলে সেই অপশনের একটা মান (argument) লাগবে।
  - `"hvo:"` → `-h`, `-v` (মান ছাড়া), `-o VALUE` (মান সহ)
  - শুরুতে `:` (যেমন `":hvo:"`) → "silent mode": getopts নিজে error প্রিন্ট করবে না, তুমি সামলাবে।
- **`$opt`:** প্রতিবার লুপে পাওয়া অপশনের অক্ষর।
- **`$OPTARG`:** অপশনের মান (যেমন `-o file.txt` হলে `file.txt`)।
- **`$OPTIND`:** পরের যে argument প্রক্রিয়া করা হবে তার index। লুপ শেষে `shift $((OPTIND-1))` করলে অপশনগুলো সরে যায়, বাকি থাকে positional argument।

Silent mode-এ error হলে:

পরিস্থিতি	$opt	$OPTARG
অচেনা অপশন (`-z`)	`?`	`z`
মান লাগবে কিন্তু দেওয়া হয়নি (`-o` একা)	`:`	`o`

৪৩.৬.২ ভেতরে কীভাবে কাজ করে

```
 $ ./script.sh -v -o out.txt -n 5 file1 file2
                 |   |   |      |  |    |     |
 index:          1   2   3      4  5    6     7

 getopts call 1 → opt=v            OPTIND=2
 getopts call 2 → opt=o OPTARG=out.txt   OPTIND=4
 getopts call 3 → opt=n OPTARG=5         OPTIND=6
 getopts call 4 → "file1" অপশন না → return 1 (লুপ শেষ)
 shift $((6-1)) → $1=file1  $2=file2
```

getopts `-vn 5` এর মতো একত্রিত অপশন, আর `-n5` (ফাঁক ছাড়া মান) — দুটোই বোঝে। `--` পেলে অপশন পার্স থামিয়ে দেয়।

৪৩.৬.৩ সম্পূর্ণ উদাহরণ

```
#!/usr/bin/env bash
set -euo pipefail

usage() {
    cat <<EOF
ব্যবহার: ${0##*/} [-h] [-v] [-n COUNT] [-o OUTFILE] FILE...

  -h          এই সাহায্য দেখাও
  -v          বিস্তারিত (verbose) আউটপুট
  -n COUNT    কতগুলো লাইন (ডিফল্ট: 10)
  -o OUTFILE  ফলাফল কোন ফাইলে লেখা হবে
EOF
}

verbose=0
count=10
outfile=""

while getopts ":hvn:o:" opt; do
    case "$opt" in
        h) usage; exit 0 ;;
        v) verbose=1 ;;
        n) count="$OPTARG" ;;
        o) outfile="$OPTARG" ;;
        :) echo "ত্রুটি: -$OPTARG অপশনের জন্য মান দরকার" >&2; usage >&2; exit 2 ;;
        \?) echo "ত্রুটি: অচেনা অপশন -$OPTARG" >&2; usage >&2; exit 2 ;;
    esac
done
shift $((OPTIND - 1))
```

```
if [[ $# -eq 0 ]]; then
    echo "ত্রুটি: অন্তত একটা FILE দাও" >&2
    usage >&2
    exit 2
fi

if ! [[ "$count" =~ ^[0-9]+$ ]]; then
    echo "ত্রুটি: COUNT একটা সংখ্যা হতে হবে" >&2
    exit 2
fi

(( verbose )) && echo "count=$count outfile=${outfile:-stdout} files=$*"

for f in "$@"; do
    if [[ -n "$outfile" ]]; then
        head -n "$count" -- "$f" >> "$outfile"
    else
        head -n "$count" -- "$f"
    fi
done
```

```
$ ./myhead.sh -v -n 2 /etc/hostname /etc/os-release
count=2 outfile=stdout files=/etc/hostname /etc/os-release
myserver
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
$ ./myhead.sh -x
ত্রুটি: অচেনা অপশন -x
ব্যবহার: myhead.sh [-h] [-v] [-n COUNT] [-o OUTFILE] FILE...
...
$ ./myhead.sh -n
ত্রুটি: -n অপশনের জন্য মান দরকার
```

> 📝 **মনে রাখো:** `getopts` শুধু ছোট (single-letter) অপশন বোঝে, `--verbose` এর মতো লম্বা অপশন বোঝে না। লম্বা অপশন লাগলে হয় `while [[ $# -gt 0 ]]; do case "$1" in ... esac; shift; done` দিয়ে নিজে পার্স করো, নয়তো util-linux-এর বাহ্যিক `getopt` (শেষে `s` ছাড়া) কমান্ড ব্যবহার করো। দুটো নাম কাছাকাছি হলেও আলাদা জিনিস: `getopts` bash built-in, `getopt` আলাদা প্রোগ্রাম।

লম্বা অপশনের হাতে-লেখা পার্সার:

```
while [[ $# -gt 0 ]]; do
    case "$1" in
        -h|--help)    usage; exit 0 ;;
        -v|--verbose) verbose=1 ;;
        -n|--count)   count="${2:?--count এর মান দরকার}"; shift ;;
        --count=*)    count="${1#*=}" ;;
        --)           shift; break ;;
        -*)           echo "অচেনা অপশন: $1" >&2; exit 2 ;;
        *)            break ;;
    esac
```

```
    shift
done
```

`${2:?message}` — `$2` না থাকলে বা খালি হলে message দিয়ে error করে থামে।

৪৩.৭ Lock ফাইল — একই স্ক্রিপ্ট দুইবার একসাথে নয়

কেন দরকার?

ধরো ব্যাকআপ স্ক্রিপ্ট cron থেকে প্রতি ঘণ্টায় চলে। একদিন ডেটা বেশি হওয়ায় ব্যাকআপে ৯০ মিনিট লাগল। এর মধ্যে পরের ঘণ্টার কপিটাও শুরু হয়ে গেল — দুটো একই ফাইলে লিখছে, ডিস্ক IO দ্বিগুণ, ব্যাকআপ নষ্ট। সমাধান: **lock** — বাথরুমের দরজার ছিটকিনির মতো; ভেতরে কেউ থাকলে পরের জন অপেক্ষা করবে বা চলে যাবে।

৪৩.৭.১ ভুল পদ্ধতি (race condition)

```
# ❌ এটা করো না
if [[ -f /tmp/my.lock ]]; then exit 1; fi
touch /tmp/my.lock
```

"চেক করা" আর "তৈরি করা" — দুটো আলাদা ধাপ। দুটো process ঠিক একই মুহূর্তে চেক করলে দুজনেই দেখবে lock নেই, দুজনেই ঢুকে যাবে। এর নাম **race condition**। তার ওপর, স্ক্রিপ্ট `kill -9` হলে lock ফাইল চিরকাল থেকে যাবে (stale lock)।

৪৩.৭.২ সঠিক পদ্ধতি ১: `flock` (সবচেয়ে ভালো)

`flock` (util-linux প্যাকেজের অংশ, সব বড় distro-তে থাকে) kernel-এর advisory file lock ব্যবহার করে। lock ধরে থাকে একটা **খোলা file descriptor** — process মরে গেলে (এমনকি `kill -9`) kernel নিজে থেকে fd বন্ধ করে, ফলে lock মুক্ত হয়ে যায়। stale lock-এর ঝামেলা নেই।

```
#!/usr/bin/env bash
set -euo pipefail

LOCKFILE="/run/lock/mybackup.lock"    # বা /var/lock/... (সাধারণত /run/lock-এর symlink)

exec 9>"$LOCKFILE"                     # fd 9 খুলে lock ফাইলের সাথে যুক্ত করো
if ! flock -n 9; then                  # -n = অপেক্ষা না করে সাথে সাথে চেষ্টা
    echo "আরেকটা কপি ইতিমধ্যে চলছে, বের হচ্ছি।" >&2
    exit 1
fi

echo "lock পেয়েছি, কাজ শুরু (PID $$)"
sleep 30
```

```
$ ./lockdemo.sh &
[1] 41234
```

```
lock পেয়েছি, কাজ শুরু (PID 41234)
$ ./lockdemo.sh
আরেকটা কপি ইতিমধ্যে চলছে, বের হচ্ছি।
```

```
 Process A                    Kernel                  Process B
 exec 9>lockfile  ──────►  inode-এ fd খোলা
 flock -n 9       ──────►  LOCK দিলাম (A)
                                          ◄──────  flock -n 9
                           ব্যস্ত! fail   ──────►  exit 1
 (A মারা গেল / শেষ)
 kernel fd বন্ধ করে ──────►  LOCK মুক্ত
```

`flock`-এর গুরুত্বপূর্ণ অপশন:

অপশন	কাজ
`-n`, `--nonblock`	lock না পেলে অপেক্ষা না করে সাথে সাথে ফেল (exit 1)
`-w SEC`, `--timeout SEC`	সর্বোচ্চ এত সেকেন্ড অপেক্ষা
`-x`, `--exclusive`	exclusive lock (ডিফল্ট)
`-s`, `--shared`	shared lock (অনেক reader একসাথে)
`-u`, `--unlock`	lock ছেড়ে দাও
`-E N`, `--conflict-exit-code N`	`-n` / `-w` ফেল হলে কোন exit code

cron-এ এক লাইনেই ব্যবহার:

```
0 * * * * /usr/bin/flock -n /run/lock/backup.lock /usr/local/bin/backup.sh
```

এখানে `flock` নিজে lock নেয়, তারপর কমান্ডটা চালায়।

৪৩.৭.৩ সঠিক পদ্ধতি ২: `mkdir` (atomic)

`mkdir` একটাই atomic অপারেশন — ডিরেক্টরি হয় তৈরি হয় (সফল), নয়তো আগে থেকে থাকলে ফেল। চেক আর তৈরি একসাথে হয়, তাই race নেই।

```
LOCKDIR="/tmp/myscript.lock"
if mkdir "$LOCKDIR" 2>/dev/null; then
    trap 'rmdir "$LOCKDIR"' EXIT
    echo "$$" > "$LOCKDIR/pid"        # (rmdir-এর আগে এটাও মুছতে হবে: rm -rf)
else
    echo "চলছে (PID $(cat "$LOCKDIR/pid" 2>/dev/null))" >&2
    exit 1
fi
```

অসুবিধা: `kill -9` হলে lock থেকে যায়। তাই PID সংরক্ষণ করে `kill -0 PID` দিয়ে সেটা বেঁচে আছে কি না পরীক্ষা করার কোড লাগে। এজন্যই `flock` পছন্দনীয়।

> 💡 **টিপস:** systemd service হিসেবে চালালে systemd নিজেই একটা unit-এর একাধিক instance একসাথে চালায় না — আলাদা lock নাও লাগতে পারে। দেখো অধ্যায় ২৬ ও অধ্যায় ২৯।

৪৩.৮ লগিং ফাংশন

`echo` দিয়ে এলোমেলো মেসেজের চেয়ে একটা নিয়মিত লগ ফরম্যাট অনেক কাজের: সময়, লেভেল, মেসেজ।

```
LOG_FILE="${LOG_FILE:-/var/log/myscript.log}"

log() {
    local level="$1"; shift
    local ts
    ts=$(date '+%Y-%m-%d %H:%M:%S')
    printf '%s [%s] %s\n' "$ts" "$level" "$*" | tee -a "$LOG_FILE" >&2
}
info()  { log INFO  "$@"; }
warn()  { log WARN  "$@"; }
error() { log ERROR "$@"; }
die()   { error "$@"; exit 1; }
```

```
$ info "ব্যাকআপ শুরু"
2026-11-12 10:15:02 [INFO] ব্যাকআপ শুরু
$ die "ডিস্কে জায়গা নেই"
2026-11-12 10:15:03 [ERROR] ডিস্কে জায়গা নেই
```

ব্যাখ্যা:

- লগ **stderr-এ** (`>&2`) পাঠানো হচ্ছে — যাতে স্ক্রিপ্টের আসল আউটপুট (stdout) পাইপ করলে লগ মেশে না। এটা Unix-এর নীতি: "ডেটা stdout-এ, ডায়াগনস্টিক stderr-এ।"
- `tee -a` একসাথে ফাইলে append ও স্ক্রিনে দেখায়।
- `printf` ব্যবহার করা হয়েছে কারণ `echo`-এর আচরণ (যেমন `-e`, `-n`) shell ভেদে আলাদা।

লেভেল অনুযায়ী ফিল্টার

```
declare -A LEVELS=([DEBUG]=0 [INFO]=1 [WARN]=2 [ERROR]=3)
LOG_LEVEL="${LOG_LEVEL:-INFO}"

log() {
    local level="$1"; shift
    (( ${LEVELS[$level]} >= ${LEVELS[$LOG_LEVEL]} )) || return 0
    printf '%s [%-5s] %s\n' "$(date '+%F %T')" "$level" "$*" >&2
}
```

`LOG_LEVEL=DEBUG ./script.sh` চালালে সব দেখাবে, ডিফল্টে DEBUG লুকানো।

`logger` — syslog/journald-এ লেখা

```
logger -t mybackup -p user.err "ব্যাকআপ ফেল করেছে"
```

অপশন	কাজ
`-t TAG`	কোন প্রোগ্রামের নামে লগ হবে
`-p FACILITY.LEVEL`	priority, যেমন `user.info`, `local0.warning`, `user.err`
`-s`	একসাথে stderr-এও দেখাও
`-i`	PID যোগ করো

```
$ journalctl -t mybackup --since "10 min ago"
Nov 12 10:20:11 myserver mybackup[5123]: ব্যাকআপ ফেল করেছে
```

লগ নিয়ে বিস্তারিত অধ্যায় ২৮-এ।

পুরো স্ক্রিপ্টের আউটপুট একসাথে লগে পাঠানো

```
exec > >(tee -a "$LOG_FILE") 2>&1
```

এরপর থেকে সব stdout ও stderr স্ক্রিনেও আসবে, লগ ফাইলেও যাবে। (`>(...)` হলো process substitution — অধ্যায় ১৯ দেখো।)

৪৩.৯ রঙিন আউটপুট

ANSI escape code

টার্মিনাল কিছু বিশেষ অক্ষর-ক্রম (escape sequence) পেলে সেগুলো লেখা হিসেবে না দেখিয়ে রঙ বদলায়। গঠন: `ESC[<code>m`, যেখানে ESC = `\e` বা `\033`।

কোড	মানে	কোড	মানে
`0`	রিসেট (সব স্বাভাবিক)	`1`	বোল্ড
`2`	ফিকে (dim)	`4`	আন্ডারলাইন
`30`	কালো	`31`	লাল
`32`	সবুজ	`33`	হলুদ
`34`	নীল	`35`	ম্যাজেন্টা
`36`	সায়ান	`37`	সাদা
`40–47`	ব্যাকগ্রাউন্ড রঙ	`90–97`	উজ্জ্বল রঙ

```
printf '\e[31mলাল লেখা\e[0m স্বাভাবিক\n'
printf '\e[1;32mবোল্ড সবুজ\e[0m\n'
echo -e "\033[33mহলুদ\033[0m"
```

> ⚠️ **সতর্কতা:** শেষে `\e[0m` (রিসেট) না দিলে পরের সব লেখা, এমনকি তোমার prompt-ও, রঙিন থেকে যাবে।

`tput` — বেশি portable উপায়

`tput` terminfo ডাটাবেস দেখে বর্তমান টার্মিনালের জন্য সঠিক কোড দেয়:

```
RED=$(tput setaf 1); GREEN=$(tput setaf 2); YELLOW=$(tput setaf 3)
BOLD=$(tput bold); RESET=$(tput sgr0)
echo "${GREEN}সফল${RESET}"
```

`tput cols` (টার্মিনালের প্রস্থ), `tput lines`, `tput clear` ইত্যাদিও কাজে লাগে।

কখন রঙ দেবে না

আউটপুট যদি ফাইলে বা পাইপে যায়, রঙের কোড আবর্জনা হয়ে যায় (`^[[31m`)। তাই চেক করো stdout আসলেই টার্মিনাল কি না:

```
if [[ -t 1 && -z "${NO_COLOR:-}" ]]; then
    C_RED=$'\e[31m'; C_GRN=$'\e[32m'; C_YLW=$'\e[33m'; C_RST=$'\e[0m'
else
    C_RED=''; C_GRN=''; C_YLW=''; C_RST=''
fi

ok()   { printf '%s[ OK ]%s %s\n'   "$C_GRN" "$C_RST" "$*"; }
fail() { printf '%s[FAIL]%s %s\n'   "$C_RED" "$C_RST" "$*" >&2; }
```

- `[[ -t 1 ]]` — fd 1 (stdout) একটা টার্মিনাল হলে সত্য।
- `NO_COLOR` — একটা প্রচলিত কনভেনশন (no-color.org): এই environment variable সেট থাকলে প্রোগ্রাম রঙ দেয় না।
- `$'\e[31m'` — ANSI-C quoting; variable-এ আসল ESC অক্ষর ঢুকে যায়, তাই `printf` / `echo` যেকোনোটায় কাজ করে।

৪৩.১০ Exit code কনভেনশন

প্রতিটি process শেষে 0–255 এর মধ্যে একটা সংখ্যা ফেরত দেয়। **0 = সফল, বাকি সব = কোনো না কোনো ব্যর্থতা।** (উল্টো মনে হতে পারে, কিন্তু যুক্তি হলো: সফল হওয়ার একটাই উপায়, ব্যর্থ হওয়ার অনেক কারণ।)

কোড	প্রচলিত অর্থ
`0`	সফল
`1`	সাধারণ ত্রুটি
`2`	ভুল ব্যবহার (shell builtin-এর ভুল argument; অনেক টুল usage error-এ 2 দেয়)
`126`	কমান্ড পাওয়া গেছে কিন্তু চালানো যায়নি (execute permission নেই)
`127`	কমান্ড পাওয়া যায়নি (command not found)
`128+N`	signal N-এ মারা গেছে
`130`	`Ctrl+C` (128+2, SIGINT)
`137`	`kill -9` (128+9, SIGKILL) — Docker/OOM killer-এ প্রায়ই দেখা যায়
`143`	`kill` / SIGTERM (128+15)
`255`	সীমার বাইরে (যেমন `exit -1` → 255)

```
$ nai_emon_command; echo $?
nai_emon_command: command not found
127
$ exit 256   # আসলে 256 % 256 = 0 হয়ে যায়!
```

> ⚠️ **সতর্কতা:** exit code ৮ বিটের, তাই `exit 256` = `exit 0`, `exit 300` = `exit 44`। কখনো 255-এর বেশি দিও না।

BSD-এর `sysexits.h` আরও নির্দিষ্ট কিছু কোড প্রস্তাব করে (অনেক প্রোগ্রাম, যেমন mail server, এগুলো মানে):

কোড	নাম	অর্থ
64	`EX_USAGE`	কমান্ড ভুলভাবে ব্যবহার
65	`EX_DATAERR`	ইনপুট ডেটা ভুল
66	`EX_NOINPUT`	ইনপুট ফাইল নেই/পড়া যায় না
69	`EX_UNAVAILABLE`	সার্ভিস পাওয়া যাচ্ছে না
70	`EX_SOFTWARE`	ভেতরের সফটওয়্যার ত্রুটি
73	`EX_CANTCREAT`	আউটপুট ফাইল তৈরি করা যায়নি
75	`EX_TEMPFAIL`	সাময়িক ব্যর্থতা, পরে আবার চেষ্টা করো
77	`EX_NOPERM`	অনুমতি নেই
78	`EX_CONFIG`	কনফিগারেশন ভুল

নিজের স্ক্রিপ্টে নামযুক্ত constant ব্যবহার করো এবং `--help`-এ ডকুমেন্ট করো:

```
readonly E_OK=0 E_USAGE=2 E_LOCKED=3 E_DISKFULL=4
```

Monitoring টুল (Nagios/Icinga plugin) এর নিজস্ব কনভেনশন: `0=OK, 1=WARNING, 2=CRITICAL, 3=UNKNOWN`। হেলথ-চেক স্ক্রিপ্টে এটা প্রায়ই ব্যবহার হয় (প্রজেক্ট ৫ দেখো)।

৪৩.১১ আরও কিছু প্রফেশনাল কৌশল

৪৩.১১.১ নিরাপদ temp ফাইল: `mktemp`

```
# ❌ বিপজ্জনক: নাম অনুমান করা যায়, অন্য ইউজার symlink বসিয়ে আক্রমণ করতে পারে
tmp=/tmp/myscript.tmp

# ✅ নিরাপদ
tmp=$(mktemp)                           # /tmp/tmp.XXXXXXXXXX, permission 600
tmpdir=$(mktemp -d)                     # ডিরেক্টরি, permission 700
tmp=$(mktemp /tmp/backup.XXXXXX.log)    # নিজের টেমপ্লেট (অন্তত ৩টা X)
tmp=$(mktemp -t backup.XXXXXX)          # $TMPDIR বা /tmp-এর ভেতরে
```

৪৩.১১.২ `readonly` ও `local`

```
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_NAME="${0##*/}"

myfunc() {
    local count=0    # শুধু এই function-এর ভেতরে
}
```

`readonly` variable পরে ভুল করে বদলাতে গেলে error দেবে। `local` না দিলে function-এর variable পুরো স্ক্রিপ্টে global হয়ে যায় — অন্য জায়গার একই নামের variable নষ্ট করতে পারে।

৪৩.১১.৩ `main "$@"` প্যাটার্ন

```
main() {
    parse_args "$@"
    check_requirements
    do_work
}
main "$@"
```

সুবিধা:

1. সব function আগে define হয়, তারপর চলে — ক্রম নিয়ে ঝামেলা নেই।
2. স্ক্রিপ্টের "গল্প" `main` পড়েই বোঝা যায়।

3. ফাইল অর্ধেক ডাউনলোড হলে (`curl | bash` এর সময়) শেষ লাইন `main "$@"` না পৌঁছালে কিছুই চলবে না — অর্ধেক কাজের বিপদ নেই।

৪৩.১১.৪ প্রয়োজনীয় কমান্ড ও root চেক

```
require_cmd() {
    local c
    for c in "$@"; do
        command -v "$c" >/dev/null 2>&1 || die "প্রয়োজনীয় কমান্ড নেই: $c"
    done
}
require_cmd rsync tar gzip

require_root() {
    [[ $EUID -eq 0 ]] || die "এই স্ক্রিপ্ট root হিসেবে চালাতে হবে (sudo দাও)"
}
```

`command -v` হলো POSIX উপায়ে কমান্ড আছে কি না জানা (`which`-এর চেয়ে নির্ভরযোগ্য)। `$EUID` হলো effective user ID — root হলে 0। (অধ্যায় ১৮)

৪৩.১১.৫ Dry-run মোড

```
DRY_RUN=0
run() {
    if (( DRY_RUN )); then
        printf '[dry-run] %q ' "$@"; echo
    else
        "$@"
    fi
}
run rm -rf -- "$old_dir"
```

`-n` অপশনে `DRY_RUN=1` সেট করলে স্ক্রিপ্ট শুধু বলবে কী করত, আসলে কিছু করবে না। বিপজ্জনক অপারেশনের আগে এটা অমূল্য। `printf '%q'` argument-কে shell-quoted আকারে দেখায়।

৪৩.১১.৬ নিশ্চিতকরণ চাওয়া

```
confirm() {
    local reply
    read -r -p "${1:-নিশ্চিত?} [y/N] " reply
    [[ "$reply" =~ ^[Yy]([Ee][Ss])?$ ]]
}
confirm "সব লগ মুছে ফেলব?" || exit 0
```

৪৩.১১.৭ Retry লজিক

```
retry() {
    local max=$1 delay=$2 n=1; shift 2
```

```
    until "$@"; do
        (( n >= max )) && return 1
        warn "চেষ্টা $n ব্যর্থ, ${delay}s পরে আবার..."
        sleep "$delay"
        n=$((n + 1))
        delay=$((delay * 2))       # exponential backoff
    done
}
retry 5 2 curl -fsS https://example.com/health
```

৪৩.১১.৮ `timeout`

```
timeout 30 some_slow_command || warn "৩০ সেকেন্ডে শেষ হয়নি"
```

`timeout` (coreutils) কমান্ড নির্দিষ্ট সময়ের বেশি চললে SIGTERM পাঠায় এবং exit code 124 দেয়। `-k 5` দিলে তারপরও না মরলে ৫ সেকেন্ড পরে SIGKILL।

৪৩.১২ ভালো অভ্যাস চেকলিস্ট

প্রতিটি প্রোডাকশন স্ক্রিপ্ট লেখার পর এই তালিকা মিলিয়ে নাও:

#	চেক	কেন
১	`#!/usr/bin/env bash` বা `#!/bin/bash` shebang আছে	কোন interpreter চালাবে নিশ্চিত
২	`set -euo pipefail` আছে (আর এর ফাঁদগুলো জানো)	fail fast
৩	সব variable `"$quoted"`	space/glob থেকে সুরক্ষা
৪	`shellcheck` পরিষ্কার	লুকানো bug
৫	function-এ `local`	variable সংঘর্ষ নেই
৬	constant-এ `readonly`, UPPERCASE নাম; সাধারণ variable lowercase	পড়তে সহজ, environment variable-এর সাথে সংঘর্ষ কম
৭	`mktemp` + `trap cleanup EXIT`	আবর্জনা ফেলে যায় না
৮	`-h` / `--help` এবং usage মেসেজ	অন্যরা ব্যবহার করতে পারে
৯	ইনপুট যাচাই (সংখ্যা? ফাইল আছে?)	ভুল ইনপুটে বিপর্যয় নেই
১০	error মেসেজ stderr-এ, অর্থবহ exit code	অন্য টুল/cron বুঝতে পারে
১১	লগ: সময় + লেভেল	পরে তদন্ত সহজ
১২	cron/timer-এ চললে lock (`flock`)	দ্বৈত চালনা নেই

#	চেক	কেন
১৩	`rm -rf "$var/"` এর আগে `$var` খালি না হওয়া নিশ্চিত (`${var:?}`)	`rm -rf /` দুর্ঘটনা নেই
১৪	পরম পাথ (absolute path) বা `cd` এর ফল চেক	cron-এর ভিন্ন working directory
১৫	cron-এর জন্য `PATH` সেট করা	cron-এ PATH খুব ছোট থাকে
১৬	গোপন তথ্য (password) কোডে না, আলাদা ফাইলে `chmod 600`	নিরাপত্তা
১৭	idempotent — দুইবার চালালেও ক্ষতি নেই	পুনরায় চালানো নিরাপদ
১৮	dry-run মোড (বিপজ্জনক কাজে)	আগে দেখে নাও
১৯	`ls` এর আউটপুট পার্স না করে glob/ `find` ব্যবহার	অদ্ভুত ফাইলনামে ভাঙে না
২০	মন্তব্য: "কী" না, "কেন" লেখো	কোড নিজেই "কী" বলে

`${var:?}` কৌশল:

```
rm -rf -- "${BACKUP_DIR:?BACKUP_DIR খালি!}/old"
```

`BACKUP_DIR` খালি বা unset হলে কমান্ড চলার আগেই স্ক্রিপ্ট থামবে।

পুনর্ব্যবহারযোগ্য স্ক্রিপ্ট টেমপ্লেট

```
#!/usr/bin/env bash
#
# script-name.sh — এক লাইনে কী করে
# ব্যবহার: script-name.sh [-h] [-v] [-n] ARG
#
set -Eeuo pipefail

readonly SCRIPT_NAME="${0##*/}"
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

VERBOSE=0
DRY_RUN=0

# ---------- রঙ ও লগিং ----------
if [[ -t 2 && -z "${NO_COLOR:-}" ]]; then
    C_R=$'\e[31m' C_G=$'\e[32m' C_Y=$'\e[33m' C_0=$'\e[0m'
else
    C_R='' C_G='' C_Y='' C_0=''
fi
log()  { printf '%s %s\n' "$(date '+%F %T')" "$*" >&2; }
```

```
info() { log "${C_G}INFO${C_0}  $*"; }
warn() { log "${C_Y}WARN${C_0}  $*"; }
die()  { log "${C_R}ERROR${C_0} $*"; exit 1; }

# ---------- cleanup ও error ----------
TMPDIR_WORK=""
cleanup() {
    local rc=$?
    [[ -n "$TMPDIR_WORK" ]] && rm -rf -- "$TMPDIR_WORK"
    exit "$rc"
}
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM
trap 'warn "লাইন $LINENO এ ব্যর্থ: $BASH_COMMAND"' ERR

usage() {
    cat <<EOF
ব্যবহার: $SCRIPT_NAME [-h] [-v] [-n] ARG
  -h  সাহায্য
  -v  verbose
  -n  dry-run (শুধু দেখাও, কিছু করো না)
EOF
}

parse_args() {
    local opt
    while getopts ":hvn" opt; do
        case "$opt" in
            h) usage; exit 0 ;;
            v) VERBOSE=1 ;;
            n) DRY_RUN=1 ;;
            \?) usage >&2; die "অচেনা অপশন: -$OPTARG" ;;
        esac
    done
    shift $((OPTIND - 1))
    [[ $# -ge 1 ]] || { usage >&2; exit 2; }
    ARG="$1"
}

run() {
    if (( DRY_RUN )); then printf '[dry-run] %q ' "$@"; echo; else "$@"; fi
}

main() {
    parse_args "$@"
    (( VERBOSE )) && set -x
    TMPDIR_WORK=$(mktemp -d)
    info "শুরু: ARG=$ARG (dir: $SCRIPT_DIR)"
    # ... আসল কাজ ...
    info "শেষ"
}

main "$@"
```

নতুন স্ক্রিপ্ট লেখার সময় এই টেমপ্লেট কপি করে শুরু করলে প্রতিবার ভালো অভ্যাসগুলো আপনা-আপনি চলে আসবে।

৪৩.১৩ বাস্তব প্রজেক্ট ১: স্বয়ংক্রিয় ব্যাকআপ স্ক্রিপ্ট

লক্ষ্য: এক বা একাধিক ডিরেক্টরির সময়-চিহ্নিত (timestamped) `.tar.gz` ব্যাকআপ তৈরি করা, ব্যাকআপ ঠিক আছে কি না যাচাই করা, checksum রাখা, আর নির্দিষ্ট দিনের পুরোনো ব্যাকআপ মুছে ফেলা (rotation)। cron থেকে চালানোর উপযোগী — lock সহ।

```
  /etc  /home/ali  /var/www      (সোর্স)
        │
        ▼  tar -czf  (প্রথমে .part নামে)
  backup-web01-20261112-020001.tar.gz.part
        │  tar -tzf দিয়ে যাচাই
        ▼  mv (atomic rename)
  backup-web01-20261112-020001.tar.gz  +  .sha256
        │
        ▼  find -mtime +7 -delete
  ৭ দিনের পুরোনো ব্যাকআপ মুছে গেল
```

```
 1  #!/usr/bin/env bash
 2  # backup.sh — ডিরেক্টরির timestamped tar.gz ব্যাকআপ ও পুরোনো ব্যাকআপ মোছা
 3  set -Eeuo pipefail
 4
 5  readonly SCRIPT_NAME="${0##*/}"
 6  DEST="/var/backups/mybackup"
 7  KEEP_DAYS=7
 8  readonly LOCK="/run/lock/${SCRIPT_NAME%.sh}.lock"
 9
10  log() { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "${*:2}" >&2; }
11  die() { log ERROR "$*"; logger -t "$SCRIPT_NAME" -p user.err -- "$*" || true; exit
1; }
12
13  usage() {
14      cat <<EOF
15  ব্যবহার: $SCRIPT_NAME [-d DEST] [-k DAYS] SOURCE_DIR...
16    -d DEST   ব্যাকআপ কোথায় রাখা হবে (ডিফল্ট: $DEST)
17    -k DAYS   কত দিনের পুরোনো ব্যাকআপ রাখা হবে (ডিফল্ট: $KEEP_DAYS)
18  EOF
19  }
20
21  while getopts ":d:k:h" opt; do
22      case "$opt" in
23          d) DEST="$OPTARG" ;;
24          k) KEEP_DAYS="$OPTARG" ;;
25          h) usage; exit 0 ;;
26          :) die "-$OPTARG অপশনের মান দরকার" ;;
27          \?) die "অচেনা অপশন: -$OPTARG" ;;
28      esac
29  done
30  shift $((OPTIND - 1))
```

```
31  (( $# > 0 )) || { usage >&2; exit 2; }
32  SRC_DIRS=("$@")
33  [[ "$KEEP_DAYS" =~ ^[0-9]+$ ]] || die "DAYS একটা ধনাত্মক পূর্ণসংখ্যা হতে হবে"
34
35  exec 9>"$LOCK"
36  flock -n 9 || die "আরেকটি ব্যাকআপ ইতিমধ্যে চলছে"
37
38  for d in "${SRC_DIRS[@]}"; do
39      [[ -d "$d" ]] || die "সোর্স ডিরেক্টরি নেই: $d"
40  done
41  mkdir -p -- "$DEST"
42
43  ts=$(date +%Y%m%d-%H%M%S)
44  archive="$DEST/backup-$(hostname -s)-$ts.tar.gz"
45  partial="$archive.part"
46  trap 'rm -f -- "$partial"' EXIT
47
48  log INFO "ব্যাকআপ শুরু: ${SRC_DIRS[*]} -> $archive"
49  rc=0
50  tar -czf "$partial" "${SRC_DIRS[@]}" 2> >(grep -v 'Removing leading' >&2) || rc=$?
51  if (( rc > 1 )); then
52      die "tar ব্যর্থ (exit $rc)"
53  elif (( rc == 1 )); then
54      log WARN "কিছু ফাইল পড়ার সময় বদলেছে (tar exit 1) — ব্যাকআপ তবুও রাখা হলো"
55  fi
56
57  tar -tzf "$partial" >/dev/null || die "আর্কাইভ নষ্ট, যাচাই ব্যর্থ"
58  mv -- "$partial" "$archive"
59  ( cd "$DEST" && sha256sum -- "${archive##*/}" > "${archive##*/}.sha256" )
60
61  size=$(du -h -- "$archive" | cut -f1)
62  log INFO "সফল: $archive ($size)"
63
64  deleted=$(find "$DEST" -maxdepth 1 -type f \
65      \( -name 'backup-*.tar.gz' -o -name 'backup-*.tar.gz.sha256' \) \
66      -mtime +"$KEEP_DAYS" -print -delete | wc -l)
67  log INFO "রোটেশন: $deleted টি পুরোনো ফাইল মোছা হয়েছে (> $KEEP_DAYS দিন)"
68  logger -t "$SCRIPT_NAME" -p user.info -- "backup ok: $archive ($size)" || true
```

(লাইন নম্বরগুলো শুধু ব্যাখ্যার সুবিধার জন্য; আসল ফাইলে লিখবে না।)

লাইন-বাই-লাইন ব্যাখ্যা:

লাইন	ব্যাখ্যা
1	`env` দিয়ে PATH থেকে bash খোঁজা — bash যেখানেই ইনস্টল থাকুক কাজ করবে।
3	strict mode, সাথে `-E` যাতে function-এর ভেতরেও ERR trap উত্তরাধিকার পায়।
5	`${0##*/}` — `$0` থেকে শেষ `/` পর্যন্ত সব কেটে শুধু ফাইলের নাম (`basename`-এর দ্রুত বিকল্প)।
6–7	ডিফল্ট মান; অপশন দিয়ে বদলানো যাবে, তাই `readonly` না।
8	`${SCRIPT_NAME%.sh}` শেষের `.sh` কেটে দেয় → `/run/lock/backup.lock`। `/run` tmpfs, তাই reboot-এ এমনিতেই পরিষ্কার।

লাইন	ব্যাখ্যা
10	`"${*:2}"` — দ্বিতীয় argument থেকে বাকি সব একসাথে। লগ stderr-এ।
11	`die` স্ক্রিনে + syslog/journald-এ error লিখে বের হয়। `logger` না থাকলেও যেন স্ক্রিপ্ট আটকে না যায় তাই `|| true`। `--` মানে "এরপর আর অপশন না" — মেসেজ `-` দিয়ে শুরু হলেও নিরাপদ।
13–19	here-document দিয়ে help টেক্সট; `$DEST` এখানে expand হয়ে আসল ডিফল্ট দেখায়।
21–30	`getopts` পার্সিং (৪৩.৬-এ বিস্তারিত)।
31	অন্তত একটা সোর্স না দিলে usage দেখিয়ে exit 2 (usage error)।
32	বাকি সব argument একটা array-তে — space-যুক্ত পাথও নিরাপদ থাকে।
33	regex দিয়ে ইনপুট যাচাই; `-k abc` দিলে `find` পরে অদ্ভুত error দিত।
35–36	fd 9-এ lock ফাইল খুলে non-blocking `flock`। স্ক্রিপ্ট মরলে lock আপনা থেকে মুক্ত।
38–40	সব সোর্স আগে থেকেই আছে কি না যাচাই — অর্ধেক কাজ করে ফেল করার চেয়ে শুরুতেই ফেল করা ভালো।
41	গন্তব্য না থাকলে তৈরি; `-p` থাকলে আগে থেকে থাকলেও error দেয় না।
43–44	`hostname -s` (short hostname) + সময় দিয়ে অনন্য নাম; `%Y%m%d-%H%M%S` ফরম্যাট বর্ণানুক্রমে সাজালে সময়ানুক্রমেও সাজে।
45–46	প্রথমে `.part` নামে লেখা হয়; মাঝপথে ফেল করলে EXIT trap অসম্পূর্ণ ফাইলটা মুছে দেয়, তাই "আধা-ব্যাকআপ" কখনো আসল ব্যাকআপের মতো দেখায় না।
49–50	GNU tar-এর exit code: 0 = সফল, 1 = কিছু ফাইল পড়ার সময় বদলেছে, 2 = মারাত্মক ত্রুটি। `set -e` exit 1-এও থামাত, তাই `|| rc=$?` দিয়ে কোডটা ধরে নিজে সিদ্ধান্ত নিচ্ছি। `2> >(grep -v ...)` — absolute path দিলে tar যে "Removing leading `/' from member names" সতর্কতা দেয়, সেটা ফিল্টার করে বাকি error stderr-এ পাঠায়।
51–55	2 বা তার বেশি = মারাত্মক → die; 1 = সতর্কতা, ব্যাকআপ রাখা হয় (চালু সিস্টেমে লগ ফাইল বদলানো স্বাভাবিক)।
57	`-t` = list; পুরো আর্কাইভ পড়ে দেখে gzip/tar গঠন ঠিক আছে কি না। ব্যাকআপ যাচাই না করা মানে ব্যাকআপ না থাকা!
58	একই ফাইলসিস্টেমে `mv` হলো atomic rename — হয় পুরোনো নাম, নয় নতুন নাম; মাঝামাঝি অবস্থা নেই।
59	subshell `( ... )`-এ `cd` করা, যাতে মূল স্ক্রিপ্টের ডিরেক্টরি না বদলায়। checksum ফাইলে শুধু ফাইলের নাম থাকায় পরে `sha256sum -c` দিয়ে যেকোনো জায়গা থেকে যাচাই করা যায়।
61	`du -h` আউটপুট `1.2G<TAB>path`; `cut -f1` প্রথম ঘর (tab-separated)।
64–66	রোটেশন: শুধু এই ডিরেক্টরির (`-maxdepth 1`) নির্দিষ্ট প্যাটার্নের ফাইল, যেগুলো `KEEP_DAYS` দিনের বেশি পুরোনো (`-mtime +N`), সেগুলো প্রিন্ট ও মুছে; `wc -l` দিয়ে গুনছি। প্যাটার্ন নির্দিষ্ট রাখায় ভুল করে অন্য ফাইল মুছবে না।
68	সফলতাও syslog-এ — monitoring সিস্টেম "শেষ সফল ব্যাকআপ কখন" জানতে পারে।

ব্যবহার ও আউটপুট:

```
$ sudo ./backup.sh -k 14 /etc /var/www
2026-11-12 02:00:01 [INFO] ব্যাকআপ শুরু: /etc /var/www -> /var/backups/mybackup/backup-
web01-20261112-020001.tar.gz
2026-11-12 02:00:19 [INFO] সফল: /var/backups/mybackup/backup-web01-20261112-
020001.tar.gz (148M)
2026-11-12 02:00:19 [INFO] রোটেশন: 2 টি পুরোনো ফাইল মোছা হয়েছে (> 14 দিন)
$ cd /var/backups/mybackup && sha256sum -c backup-web01-20261112-020001.tar.gz.sha256
backup-web01-20261112-020001.tar.gz: OK
```

cron-এ প্রতিদিন রাত ২টায় (`sudo crontab -e`):

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
0 2 * * * /usr/local/bin/backup.sh -k 14 /etc /var/www >> /var/log/backup.log 2>&1
```

> 💡 **টিপস:** একই মেশিনে রাখা ব্যাকআপ আসলে পুরো ব্যাকআপ না — ডিস্ক নষ্ট হলে দুটোই যাবে। ৩-২-১ নিয়ম (৩ কপি, ২ ধরনের মিডিয়া, ১টা অন্য জায়গায়) আর `rsync` দিয়ে দূরে কপি করার কৌশল অধ্যায় ৩৫-এ আছে। আর্কাইভ নিয়ে বিস্তারিত অধ্যায় ৩৪-এ।

৪৩.১৪ বাস্তব প্রজেক্ট ২: ডিস্ক স্পেস অ্যালার্ট

লক্ষ্য: সব আসল ফাইলসিস্টেমের ব্যবহার দেখা; WARNING ও CRITICAL সীমা পার হলে রঙিন সতর্কতা, syslog-এ লেখা, ঐচ্ছিকভাবে ইমেইল পাঠানো; আর monitoring-বান্ধব exit code (0/1/2) দেওয়া।

```
 1  #!/usr/bin/env bash
 2  # disk-alert.sh — ফাইলসিস্টেম ব্যবহারের সতর্কতা
 3  set -euo pipefail
 4
 5  WARN=80
 6  CRIT=90
 7  MAIL_TO=""
 8
 9  if [[ -t 1 ]]; then
10      R=$'\e[31m' Y=$'\e[33m' G=$'\e[32m' N=$'\e[0m'
11  else
12      R='' Y='' G='' N=''
13  fi
14
15  while getopts ":w:c:m:h" opt; do
16      case "$opt" in
17          w) WARN="$OPTARG" ;;
18          c) CRIT="$OPTARG" ;;
19          m) MAIL_TO="$OPTARG" ;;
20          h) echo "ব্যবহার: ${0##*/} [-w WARN%] [-c CRIT%] [-m EMAIL]"; exit 0 ;;
21          *) echo "ভুল অপশন" >&2; exit 3 ;;
22      esac
23  done
24  [[ "$WARN" =~ ^[0-9]+$ && "$CRIT" =~ ^[0-9]+$ ]] || { echo "সীমা সংখ্যা হতে হবে" >&2;
exit 3; }
```

```
(( WARN < CRIT )) || { echo "WARN অবশ্যই CRIT এর চেয়ে ছোট" >&2; exit 3; }

status=0
report=""

while read -r pcent avail mount; do
    use="${pcent%\%}"
    if (( use >= CRIT )); then
        line="CRITICAL ${use}% ব্যবহৃত, খালি ${avail} — ${mount}"
        printf '%s%s%s\n' "$R" "$line" "$N"
        status=2
    elif (( use >= WARN )); then
        line="WARNING  ${use}% ব্যবহৃত, খালি ${avail} — ${mount}"
        printf '%s%s%s\n' "$Y" "$line" "$N"
        (( status < 1 )) && status=1
    else
        printf '%sOK       %s%% %s%s\n' "$G" "$use" "$mount" "$N"
        continue
    fi
    report+="$line"$'\n'
done < <(df -h --output=pcent,avail,target \
             -x tmpfs -x devtmpfs -x squashfs -x overlay -x efivarfs | tail -n +2)

if (( status > 0 )); then
    logger -t disk-alert -p user.warning -- "$(hostname -s): ${report//$'\n'/; }"
|| true
    if [[ -n "$MAIL_TO" ]] && command -v mail >/dev/null; then
        printf '%s\n' "$report" | mail -s "[disk-alert] $(hostname -s)" "$MAIL_TO"
    fi
fi
exit "$status"
```

লাইন-বাই-লাইন ব্যাখ্যা:

লাইন	ব্যাখ্যা
5–7	ডিফল্ট সীমা: ৮০% সতর্কতা, ৯০% সংকট।
9–13	stdout টার্মিনাল হলে তবেই রঙ; cron-এর লগে escape code যাবে না। একই লাইনে একাধিক assignment চলে।
15–23	অপশন পার্স। এখানে ভুলে exit 3 — Nagios কনভেনশনে 3 = UNKNOWN ("চেকটাই ঠিকমতো চলেনি")।
24–25	ইনপুট যাচাই — `-w 95 -c 90` দেওয়ার মতো যুক্তিহীন কনফিগারেশন আটকানো।
27–28	`status` সামগ্রিক ফল ধরে রাখে; `report` সমস্যাযুক্ত লাইনগুলো জমায়।

লাইন	ব্যাখ্যা
30	`read -r pcent avail mount` — প্রতিটি লাইন তিন ঘরে ভাগ। শেষ variable `mount` লাইনের বাকি সবটা পায়, তাই mount point-এ space থাকলেও ঠিক থাকে। `df` ডানদিকে সাজানোর জন্য যে শুরুর space দেয়, ডিফল্ট `IFS` সেটা বাদ দেয়।
31	`${pcent%\%}` — শেষের `%` চিহ্ন কেটে শুধু সংখ্যা (`87%` → `87`)।
32–35	সংকট হলে লাল রঙে লেখা ও status 2।
36–39	সতর্কতা হলে হলুদ; কিন্তু আগেই 2 হয়ে থাকলে সেটা কমিয়ে 1 করব না — তাই শর্ত।
40–42	ঠিক থাকলে সবুজ OK, আর `continue` দিয়ে report-এ না যোগ করে পরের লাইনে। `%%` হলো printf-এ আক্ষরিক `%`।
44	`+=` দিয়ে string-এ যোগ; `$'\n'` আসল newline অক্ষর।
45–46	**গুরুত্বপূর্ণ:** `df ... | while read` না লিখে `while ... done < <(df ...)` লেখা হয়েছে। pipe দিলে `while` লুপ একটা **subshell**-এ চলত, আর লুপের ভেতরে সেট করা `status` ও `report` লুপ শেষে হারিয়ে যেত! process substitution `<(...)` এ লুপ মূল shell-এই চলে। `--output=pcent,avail,target` শুধু দরকারি কলাম দেয় (GNU coreutils)। `-x` দিয়ে RAM-ভিত্তিক বা read-only ভার্চুয়াল ফাইলসিস্টেম বাদ — snap-এর `squashfs` সবসময় 100% দেখায়, ওগুলো মিথ্যা অ্যালার্ম দিত। `tail -n +2` হেডার লাইন বাদ দেয়।
49	`${report//$'\n'/; }` — সব newline কে `; ` দিয়ে প্রতিস্থাপন, যাতে syslog-এ এক লাইনে আসে।
50–51	ইমেইল ঠিকানা দেওয়া থাকলে ও `mail` কমান্ড (Debian/Ubuntu-তে `mailutils` বা `bsd-mailx`, Fedora/RHEL-এ `s-nail`) ইনস্টল থাকলে তবেই মেইল।
54	0/1/2 exit code — cron wrapper বা monitoring টুল এ দেখে সিদ্ধান্ত নিতে পারে।

```
$ ./disk-alert.sh -w 70 -c 85
OK       34% /
WARNING  78% ব্যবহৃত, খালি 22G — /var
OK       12% /boot
OK       5% /boot/efi
CRITICAL 91% ব্যবহৃত, খালি 9.1G — /home
$ echo $?
2
```

> 💡 **টিপস:** ডিস্ক "full" হওয়ার আরেকটা কারণ inode শেষ হয়ে যাওয়া (অনেক ছোট ফাইল)। একই স্ক্রিপ্টে `--output=ipcent,iavail,target` দিয়ে inode-ও চেক করতে পারো। inode সম্পর্কে অধ্যায় ১১ আর ডিস্ক সম্পর্কে অধ্যায় ৩২ দেখো।

৪৩.১৫ বাস্তব প্রজেক্ট ৩: ওয়েব সার্ভার লগ বিশ্লেষক

লক্ষ্য: Nginx/Apache-এর "combined" ফরম্যাটের access log পড়ে একটা রিপোর্ট বানানো: মোট request, সবচেয়ে বেশি request করা IP, জনপ্রিয় URL, HTTP status-এর বণ্টন, error rate, মোট ডেটা, আর সবচেয়ে বেশি 404। `.gz` করা পুরোনো লগও পড়বে।

একটা combined লগ লাইন ও তার ঘর (awk-এর ডিফল্ট whitespace বিভাজনে):

```
203.0.113.7 - - [12/Nov/2026:10:15:32 +0600] "GET /index.html HTTP/1.1" 200 5123 "-"
"Mozilla/5.0 ..."
    $1      $2 $3        $4            $5     $6      $7        $8     $9   $10   $11
$12...
   (IP)               (সময়)               (method) (URL)            (status)(bytes)
```

```
 1  #!/usr/bin/env bash
 2  # log-report.sh — combined ফরম্যাট access log বিশ্লেষণ
 3  set -euo pipefail
 4
 5  TOP=10
 6  usage() { echo "ব্যবহার: ${0##*/} [-n TOP] LOGFILE..."; }
 7
 8  while getopts ":n:h" opt; do
 9      case "$opt" in
10          n) TOP="$OPTARG" ;;
11          h) usage; exit 0 ;;
12          *) usage >&2; exit 2 ;;
13      esac
14  done
15  shift $((OPTIND - 1))
16  (( $# > 0 )) || { usage >&2; exit 2; }
17  [[ "$TOP" =~ ^[1-9][0-9]*$ ]] || { echo "TOP ধনাত্মক সংখ্যা হতে হবে" >&2; exit 2; }
18
19  for f in "$@"; do
20      [[ -r "$f" ]] || { echo "পড়া যাচ্ছে না: $f" >&2; exit 66; }
21  done
22
23  data=$(mktemp)
24  trap 'rm -f -- "$data"' EXIT
25  zcat -f -- "$@" > "$data"
26
27  section() { printf '\n=== %s ===\n' "$1"; }
28  rank() { sort | uniq -c | sort -rn | head -n "$TOP" || true; }
29
30  total=$(wc -l < "$data")
31  if (( total == 0 )); then echo "লগ খালি"; exit 0; fi
32
33  section "সারাংশ"
34  awk -v total="$total" '
35      { bytes += ($10 ~ /^[0-9]+$/) ? $10 : 0
36        if ($9 ~ /^4/) c4++
37        if ($9 ~ /^5/) c5++ }
38      END {
39        printf "মোট request : %d\n", total
40        printf "মোট ডেটা    : %.2f MB\n", bytes/1048576
```

```
41        printf "4xx হার     : %.2f%%\n", c4*100/total
42        printf "5xx হার     : %.2f%%\n", c5*100/total
43      }' "$data"
44  printf 'প্রথম entry : %s\n' "$(head -n1 "$data" | awk '{print substr($4,2)}')"
45  printf 'শেষ entry   : %s\n' "$(tail -n1 "$data" | awk '{print substr($4,2)}')"
46
47  section "শীর্ষ $TOP IP"
48  awk '{print $1}' "$data" | rank
49
50  section "শীর্ষ $TOP URL"
51  awk '{print $7}' "$data" | rank
52
53  section "HTTP status বণ্টন"
54  awk '{print $9}' "$data" | sort | uniq -c | sort -rn
55
56  section "শীর্ষ $TOP 404 URL"
57  awk '$9 == 404 {print $7}' "$data" | rank
58
59  section "প্রতি ঘণ্টায় request"
60  awk '{print substr($4, 14, 2)}' "$data" | sort | uniq -c
```

লাইন-বাই-লাইন ব্যাখ্যা:

লাইন	ব্যাখ্যা
5	ডিফল্টে শীর্ষ ১০টা দেখাবে।
8–17	অপশন পার্স ও যাচাই; regex `^[1-9][0-9]*$` শূন্য বা ঋণাত্মক আটকায়।
19–21	প্রতিটি ফাইল পড়া যায় কি না (`-r`)। না গেলে exit 66 (`EX_NOINPUT`)।
23–24	একটা নিরাপদ temp ফাইল, যেটা শেষে মুছে যাবে।
25	`zcat -f` — gzip ফাইল হলে খুলে দেয়, সাধারণ ফাইল হলে হুবহু দেয় (`-f` = force)। তাই `access.log` আর `access.log.2.gz` একসাথে দেওয়া যায়। সব ডেটা একবার temp ফাইলে রাখায় বারবার decompress করতে হয় না।
27	রিপোর্টের প্রতিটি অংশের শিরোনাম ছাপার ছোট্ট helper।
28	(নাম `rank`, কারণ `top` নামে আগেই একটা লিনাক্স কমান্ড আছে।) "গুনে সাজাও" প্যাটার্ন: `sort` (একই জিনিস পাশাপাশি আনে) → `uniq -c` (গোনে) → `sort -rn` (সংখ্যা অনুযায়ী বড় থেকে ছোট) → `head`। এটা লিনাক্সের সবচেয়ে বিখ্যাত one-liner প্যাটার্নগুলোর একটা (অধ্যায় ২৩)। এই function stdin থেকে পড়ে, তাই pipe-এর শেষে বসানো যায়। শেষের `|| true` জরুরি: অনেক আলাদা IP থাকলে `head` ১০ লাইন পড়েই বন্ধ হয়, আর তখনো লিখতে থাকা `sort` SIGPIPE পেয়ে 141 দেয় — `pipefail` + `set -e` পুরো রিপোর্ট মাঝপথে থামিয়ে দিত (৪৩.২.৩-এর ফাঁদ)।
30–31	`wc -l < file` — redirection দিলে আউটপুটে ফাইলের নাম আসে না, শুধু সংখ্যা। খালি লগে শূন্য দিয়ে ভাগ এড়াতে আগেই বের হই।

লাইন	ব্যাখ্যা
34	`-v total=...` দিয়ে shell variable awk-এ পাঠানো (অধ্যায় ২২)।
35	bytes ঘরে `-` থাকতে পারে (যেমন 304 response-এ), তাই সংখ্যা হলে তবেই যোগ।
36–37	status 4 বা 5 দিয়ে শুরু হলে গুনি।
38–43	`END` ব্লক সব লাইন পড়ার পর একবার চলে; `%%` আক্ষরিক `%`। `1048576` = 1024×1024।
44–45	`$4` হলো `[12/Nov/2026:10:15:32`; `substr($4,2)` প্রথম `[` বাদ দেয়।
48	প্রথম ঘর = ক্লায়েন্ট IP। কোনো একটা IP অস্বাভাবিক বেশি হলে সেটা bot বা আক্রমণ হতে পারে।
51	সপ্তম ঘর = অনুরোধ করা URL।
54	status বণ্টন — এখানে `head` নেই, সব status দেখাই।
57	awk-এর pattern `$9 == 404` — শুধু ওই লাইনগুলোর URL। ভাঙা লিংক বা স্ক্যানার (`/wp-login.php` খোঁজা bot) ধরা পড়ে।
60	`$4`-এর ১৪তম অক্ষর থেকে ২টা অক্ষর = ঘণ্টা (`[12/Nov/2026:` ১৩ অক্ষর)। ঘণ্টা অনুযায়ী ট্রাফিক বোঝা যায়।

```
$ ./log-report.sh -n 3 /var/log/nginx/access.log /var/log/nginx/access.log.2.gz

=== সারাংশ ===
মোট request : 48213
মোট ডেটা    : 912.47 MB
4xx হার     : 6.31%
5xx হার     : 0.42%
প্রথম entry : 10/Nov/2026:00:00:04
শেষ entry   : 12/Nov/2026:10:15:32

=== শীর্ষ 3 IP ===
   5120 198.51.100.23
   2301 203.0.113.7
    988 192.0.2.44

=== শীর্ষ 3 URL ===
  10544 /
   6120 /api/v1/status
   3300 /static/app.js

=== HTTP status বণ্টন ===
  41022 200
   3610 304
   2301 404
    981 301
    203 500
     96 403
```

```
=== শীর্ষ 3 404 URL ===
    812 /wp-login.php
    430 /.env
    101 /favicon.ico
...
```

> 📝 **মনে রাখো:** একাধিক ফাইল দিলে ক্রম গুরুত্বপূর্ণ — "প্রথম/শেষ entry" সঠিক পেতে পুরোনো ফাইল আগে দাও। আর `$4`-ভিত্তিক বিশ্লেষণ শুধু combined/common লগ ফরম্যাটে কাজ করে; কাস্টম `log_format` হলে ঘরের নম্বর বদলাবে।

> 💡 **টিপস:** `/.env`, `/wp-login.php`-এর মতো অনুরোধ মানে কেউ তোমার সার্ভার স্ক্যান করছে। এমন IP স্বয়ংক্রিয়ভাবে ব্লক করার জন্য fail2ban বা ফায়ারওয়াল (অধ্যায় ৩৯, অধ্যায় ৪৫) দেখো।

৪৩.১৬ বাস্তব প্রজেক্ট ৪: CSV থেকে একসাথে অনেক ইউজার তৈরি

লক্ষ্য: নতুন ৫০ জন কর্মী যোগ দিয়েছে। একটা CSV ফাইল থেকে সবার অ্যাকাউন্ট তৈরি করা, প্রয়োজনীয় গ্রুপ তৈরি করা, প্রত্যেককে একটা র‍্যান্ডম অস্থায়ী পাসওয়ার্ড দেওয়া যেটা প্রথম লগইনে বদলাতে বাধ্য করবে, আর পাসওয়ার্ডগুলো একটা সুরক্ষিত ফাইলে রাখা। ইতিমধ্যে থাকা ইউজার এড়িয়ে যাবে (idempotent)। dry-run মোডও থাকবে।

ইনপুট ফাইল `users.csv`:

```
# username,full name,groups (; দিয়ে আলাদা)
rahim,Rahim Uddin,developers;docker
karim,Karim Hasan,developers
nasrin,Nasrin Akter,hr
```

```
 1  #!/usr/bin/env bash
 2  # bulk-useradd.sh — CSV থেকে ইউজার তৈরি
 3  set -euo pipefail
 4
 5  DRY_RUN=0
 6  CREDS="/root/new-users-$(date +%F).csv"
 7
 8  log()  { printf '[%s] %s\n' "$1" "${*:2}" >&2; }
 9  run()  { if (( DRY_RUN )); then printf '[dry-run] %q ' "$@"; echo; else "$@"; fi; }
10
11  while getopts ":no:h" opt; do
12      case "$opt" in
13          n) DRY_RUN=1 ;;
14          o) CREDS="$OPTARG" ;;
15          h) echo "ব্যবহার: ${0##*/} [-n] [-o CREDS_FILE] USERS.csv"; exit 0 ;;
16          *) echo "ভুল অপশন" >&2; exit 2 ;;
17      esac
```

```
done
shift $((OPTIND - 1))
[[ $# -eq 1 ]] || { echo "ঠিক একটা CSV ফাইল দাও" >&2; exit 2; }
CSV="$1"
[[ -r "$CSV" ]] || { log ERROR "পড়া যাচ্ছে না: $CSV"; exit 66; }
[[ $EUID -eq 0 || $DRY_RUN -eq 1 ]] || { log ERROR "root হিসেবে চালাও"; exit 77; }
command -v openssl >/dev/null || { log ERROR "openssl লাগবে"; exit 69; }

if (( ! DRY_RUN )); then
    ( umask 077; : >> "$CREDS" )
fi

created=0 skipped=0 failed=0
lineno=0
while IFS=, read -r -u 3 user fullname groups || [[ -n "$user" ]]; do
    lineno=$((lineno + 1))
    groups="${groups%$'\r'}"
    [[ -z "$user" || "$user" == \#* ]] && continue

    if ! [[ "$user" =~ ^[a-z_][a-z0-9_-]{0,31}$ ]]; then
        log ERROR "লাইন $lineno: অবৈধ username '$user'"
        failed=$((failed + 1)); continue
    fi
    if id -u "$user" >/dev/null 2>&1; then
        log SKIP "$user ইতিমধ্যে আছে"
        skipped=$((skipped + 1)); continue
    fi

    opts=(-m -s /bin/bash -c "$fullname")
    if [[ -n "$groups" ]]; then
        IFS=';' read -r -a glist <<< "$groups"
        for g in "${glist[@]}"; do
            getent group "$g" >/dev/null || run groupadd "$g"
        done
        opts+=(-G "$(IFS=,; echo "${glist[*]}")")
    fi

    if ! run useradd "${opts[@]}" "$user"; then
        log ERROR "$user তৈরি ব্যর্থ"
        failed=$((failed + 1)); continue
    fi

    pass=$(openssl rand -base64 12)
    if (( DRY_RUN )); then
        echo "[dry-run] $user এর পাসওয়ার্ড সেট ও chage -d 0"
    else
        printf '%s:%s\n' "$user" "$pass" | chpasswd
        chage -d 0 "$user"
        printf '%s,%s\n' "$user" "$pass" >> "$CREDS"
    fi
    log OK "$user ($fullname) তৈরি হয়েছে"
    created=$((created + 1))
done 3< "$CSV"

log DONE "তৈরি: $created, এড়ানো: $skipped, ব্যর্থ: $failed"
(( DRY_RUN )) || log INFO "পাসওয়ার্ড ফাইল: $CREDS (শুধু root পড়তে পারে)"
(( failed == 0 ))
```

লাইন-বাই-লাইন ব্যাখ্যা:

লাইন	ব্যাখ্যা
6	পাসওয়ার্ড ফাইল root-এর home-এ, তারিখসহ নাম।
9	dry-run wrapper (৪৩.১১.৫)।
20–22	ঠিক একটা ইনপুট ফাইল, পড়ার যোগ্য।
23	`$EUID` 0 না হলে (root না) exit 77 (`EX_NOPERM`); তবে dry-run কেউ যেকোনো ইউজার হিসেবে চালাতে পারে।
24	পাসওয়ার্ড বানাতে `openssl` লাগবে; না থাকলে 69 (`EX_UNAVAILABLE`)।
26–28	subshell-এ `umask 077` দিয়ে ফাইল তৈরি → permission `600` (শুধু মালিক পড়তে/লিখতে পারে)। subshell হওয়ায় umask বাকি স্ক্রিপ্টে প্রভাব ফেলে না। `: >>` মানে "কিছু না লিখে ফাইলটা খোলো" (না থাকলে তৈরি হবে)। পাসওয়ার্ড লেখার **আগেই** permission ঠিক করা জরুরি। (অধ্যায় ১৬)
32	মূল লুপ। `IFS=,` শুধু এই `read` -এর জন্য বিভাজক কমা। `-u 3` = fd 3 থেকে পড়ো (লাইন 70-এ `3< "$CSV"`)। কেন stdin না? লুপের ভেতরের কোনো কমান্ড (যেমন `ssh` , বা ইন্টারঅ্যাকটিভ প্রোগ্রাম) stdin পড়ে ফেললে CSV-এর বাকি লাইন খেয়ে ফেলত — আলাদা fd ব্যবহার এই সূক্ষ্ম bug প্রতিরোধ করে। `\|\| [[ -n "$user" ]]` — ফাইলের শেষ লাইনে newline না থাকলে `read` ফেল করে কিন্তু ডেটা পড়ে ফেলে; এই শর্ত সেই শেষ লাইনটাও প্রক্রিয়া করে।
34	Windows-এ (Excel) বানানো CSV-তে লাইনের শেষে `\r` (CRLF) থাকে; শেষ ঘর থেকে সেটা কাটা। নাহলে গ্রুপের নাম হতো `hr\r` — অদৃশ্য কিন্তু মারাত্মক bug।
35	খালি লাইন আর `#` দিয়ে শুরু মন্তব্য বাদ।
37–40	username যাচাই: ছোট হাতের অক্ষর বা `_` দিয়ে শুরু, সর্বোচ্চ ৩২ অক্ষর — নিরাপদ, portable নাম। ভুল হলে গুনে পরের লাইনে।
41–44	`id -u` সফল মানে ইউজার আছে → এড়িয়ে যাও। তাই স্ক্রিপ্ট দুইবার চালালেও ক্ষতি নেই (idempotent)।
46	useradd-এর অপশন **array**-তে রাখা — `-c "Rahim Uddin"` -এর মতো space-যুক্ত মান নিরাপদে একটা argument থাকে। `-m` home ডিরেক্টরি তৈরি (Debian/Ubuntu-তে `useradd` ডিফল্টে home তৈরি করে না, RHEL/Fedora-তে করে — তাই স্পষ্ট করে দেওয়াই ভালো), `-s` login shell, `-c` comment/পূর্ণ নাম।
48	`IFS=';' read -a` — `developers;docker` কে array-তে ভাঙা। `<<<` হলো here-string।
49–51	`getent group` গ্রুপ খোঁজে (`/etc/group` + LDAP ইত্যাদি সব উৎসে)। না থাকলে তৈরি।
52	subshell-এ `IFS=,` সেট করে `"${glist[*]}"` → array-কে কমা দিয়ে জোড়া: `developers,docker` — `-G` এই ফরম্যাটই চায়।
55–58	`useradd` ব্যর্থ হলে পুরো স্ক্রিপ্ট না থামিয়ে (এটা `if` -এর শর্ত, তাই `set -e` থামায় না) এই ইউজার ব্যর্থ হিসেবে গুনে পরেরটায় যাই।

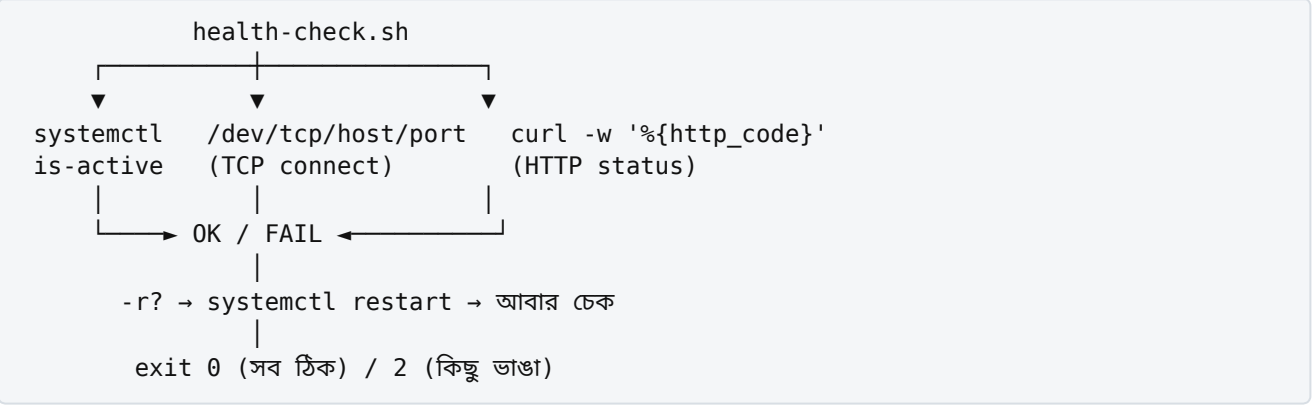
লাইন	ব্যাখ্যা
60	`openssl rand -base64 12` — ১২টা র‍্যান্ডম বাইট → ১৬ অক্ষরের পাসওয়ার্ড। (`tr -dc ... </dev/urandom | head -c 16` ও প্রচলিত, কিন্তু `pipefail`-এ `tr` SIGPIPE পেয়ে 141 দেয় আর স্ক্রিপ্ট মরে যায় — ৪৩.২.৩-এর ফাঁদ!)
64	`chpasswd` stdin থেকে `user:password` পড়ে পাসওয়ার্ড সেট করে। কমান্ড-লাইন argument-এ পাসওয়ার্ড দিলে সেটা `ps`-এ অন্যরা দেখতে পারত — pipe দিয়ে দেওয়ায় দেখা যায় না (`printf` bash builtin, আলাদা process হয় না)।
65	`chage -d 0` — "শেষ পাসওয়ার্ড বদলের তারিখ" ০ করে দেয়, ফলে প্রথম লগইনেই নতুন পাসওয়ার্ড দিতে বাধ্য।
66	পাসওয়ার্ড শুধু ৬০০-permission ফাইলে লেখা হয়, স্ক্রিনে বা লগে না।
70	`3< "$CSV"` — পুরো লুপের জন্য fd 3-এ CSV খোলা।
74	শেষ কমান্ডের status-ই স্ক্রিপ্টের exit code: কোনো ব্যর্থতা না থাকলে 0, থাকলে 1। (`set -e` শেষ কমান্ডের ক্ষেত্রে আলাদা কিছু করে না — exit code একই থাকে।)

```
$ sudo ./bulk-useradd.sh -n users.csv
[dry-run] groupadd developers
[dry-run] useradd -m -s /bin/bash -c Rahim\ Uddin -G developers,docker rahim
[dry-run] rahim এর পাসওয়ার্ড সেট ও chage -d 0
[OK] rahim (Rahim Uddin) তৈরি হয়েছে
...
$ sudo ./bulk-useradd.sh users.csv
[OK] rahim (Rahim Uddin) তৈরি হয়েছে
[OK] karim (Karim Hasan) তৈরি হয়েছে
[OK] nasrin (Nasrin Akter) তৈরি হয়েছে
[DONE] তৈরি: 3, এড়ানো: 0, ব্যর্থ: 0
[INFO] পাসওয়ার্ড ফাইল: /root/new-users-2026-11-12.csv (শুধু root পড়তে পারে)
$ sudo ./bulk-useradd.sh users.csv
[SKIP] rahim ইতিমধ্যে আছে
[SKIP] karim ইতিমধ্যে আছে
[SKIP] nasrin ইতিমধ্যে আছে
[DONE] তৈরি: 0, এড়ানো: 3, ব্যর্থ: 0
$ sudo chage -l rahim | head -n1
Last password change                                    : password must be changed
```

> ⚠️ **সতর্কতা:** dry-run-এ গ্রুপ আসলে তৈরি হয় না, তাই একই গ্রুপ একাধিক ইউজারের জন্য বারবার `[dry-run] groupadd` দেখাতে পারে — এটা স্বাভাবিক। পাসওয়ার্ড ফাইল ইউজারদের হাতে দেওয়ার পর `shred -u` দিয়ে মুছে ফেলো। প্রশাসনিক ক্ষমতা দিতে হলে Ubuntu/Debian-এ `sudo` গ্রুপ, Fedora/RHEL-এ `wheel` গ্রুপ। ইউজার-গ্রুপের বিস্তারিত অধ্যায় ১৫-এ।

৪৩.১৭ বাস্তব প্রজেক্ট ৫: সার্ভিস হেলথ চেক ও স্বয়ংক্রিয় রিস্টার্ট

লক্ষ্য: কয়েকটা systemd সার্ভিস চলছে কি না, কয়েকটা TCP পোর্ট খোলা কি না, আর কয়েকটা HTTP endpoint ঠিক response দিচ্ছে কি না পরীক্ষা করা। `-r` দিলে বন্ধ সার্ভিস রিস্টার্ট করার চেষ্টা। ফলাফল রঙিন টেবিলে, আর Nagios-ধাঁচের exit code।




```
#!/usr/bin/env bash
# health-check.sh — সার্ভিস, পোর্ট ও HTTP endpoint পরীক্ষা
set -uo pipefail

SERVICES=(nginx ssh cron)
PORTS=(localhost:22 localhost:80)
URLS=(http://localhost/ http://localhost/healthz)
AUTO_RESTART=0
CONF="/etc/health-check.conf"
[[ -r "$CONF" ]] && source "$CONF"

if [[ -t 1 ]]; then G=$'\e[32m' R=$'\e[31m' Y=$'\e[33m' N=$'\e[0m'; else G='' R=''
Y='' N=''; fi
ok()   { printf '  %s[ OK ]%s %s\n' "$G" "$N" "$*"; }
bad()  { printf '  %s[FAIL]%s %s\n' "$R" "$N" "$*"; problems=$((problems + 1)); }
note() { printf '  %s[INFO]%s %s\n' "$Y" "$N" "$*"; }

while getopts ":rh" opt; do
    case "$opt" in
        r) AUTO_RESTART=1 ;;
        h) echo "ব্যবহার: ${0##*/} [-r]   (-r = বন্ধ সার্ভিস রিস্টার্ট)"; exit 0 ;;
        *) echo "ভুল অপশন" >&2; exit 3 ;;
    esac
done

problems=0

check_service() {
    local svc=$1
    if systemctl is-active --quiet "$svc"; then
        ok "service $svc চলছে"
        return
    fi
    if (( AUTO_RESTART )); then
        note "service $svc বন্ধ — রিস্টার্ট করছি"
        systemctl restart "$svc" 2>/dev/null
```

```
36          sleep 3
37          if systemctl is-active --quiet "$svc"; then
38              ok "service $svc রিস্টার্টের পর চলছে"
39              logger -t health-check -- "restarted $svc" || true
40              return
41          fi
42      fi
43      bad "service $svc বন্ধ ($(systemctl is-active "$svc" 2>/dev/null))"
44  }
45
46  check_port() {
47      local host=${1%:*} port=${1##*:}
48      if timeout 3 bash -c ": > /dev/tcp/$host/$port" 2>/dev/null; then
49          ok "port $host:$port খোলা"
50      else
51          bad "port $host:$port এ সংযোগ হচ্ছে না"
52      fi
53  }
54
55  check_url() {
56      local url=$1 code time
57      read -r code time < <(curl -s -o /dev/null --max-time 5 \
58                               -w '%{http_code} %{time_total}' "$url")
59      if [[ "$code" =~ ^[23] ]]; then
60          ok "url $url → $code (${time}s)"
61      else
62          bad "url $url → ${code:-000}"
63      fi
64  }
65
66  echo "== Health check: $(hostname -s) @ $(date '+%F %T') =="
67  echo "Services:"; for s in "${SERVICES[@]}"; do check_service "$s"; done
68  echo "Ports:";    for p in "${PORTS[@]}";    do check_port "$p";    done
69  echo "HTTP:";     for u in "${URLS[@]}";     do check_url "$u";     done
70
71  if (( problems > 0 )); then
72      echo "${R}মোট সমস্যা: $problems${N}"
73      logger -t health-check -p user.err -- "$problems problem(s) on $(hostname -s)"
|| true
74      exit 2
75  fi
76  echo "${G}সব ঠিক আছে${N}"
77  exit 0
```

লাইন-বাই-লাইন ব্যাখ্যা:

লাইন	ব্যাখ্যা
3	লক্ষ্য করো: এখানে `-e` **নেই**, ইচ্ছাকৃতভাবে। হেলথ-চেকের কাজই হলো ব্যর্থতা খোঁজা — একটা চেক ফেল করলে পুরো স্ক্রিপ্ট থেমে গেলে বাকি চেক হবে না। strict mode অন্ধভাবে নয়, বুঝে ব্যবহার করো।
5–7	ডিফল্ট লক্ষ্য। Ubuntu/Debian-এ SSH সার্ভিসের নাম `ssh` (সাথে `sshd` alias), Fedora/RHEL-এ `sshd`; cron Fedora/RHEL-এ `crond`। নিজের সার্ভার অনুযায়ী বদলাও।

লাইন	ব্যাখ্যা
9–10	কনফিগ ফাইল থাকলে `source` করে variable গুলো ওভাররাইড করা — কোড না বদলে প্রতি সার্ভারে আলাদা তালিকা। ⚠️ `source` করা ফাইল কোড হিসেবে চলে, তাই এটা root-এর মালিকানায় ও অন্যদের অলেখ্য হতে হবে।
12–15	রঙ ও তিনটা আউটপুট helper। `bad` প্রতিবার `problems` বাড়ায় (global variable)।
17–23	`-r` থাকলে auto-restart চালু।
27–28	`local` দিয়ে function-এর নিজস্ব variable।
29	`systemctl is-active --quiet` কিছু প্রিন্ট করে না, শুধু exit code দেয়: সক্রিয় হলে 0। স্ক্রিপ্টে চেক করার আদর্শ উপায়।
33–42	auto-restart চালু থাকলে রিস্টার্ট, ৩ সেকেন্ড অপেক্ষা (সার্ভিস ওঠার সময়), আবার চেক; সফল হলে syslog-এ লিখে রাখা — পরে কেউ জানবে কেন সার্ভিস রিস্টার্ট হয়েছিল। (রিস্টার্টের জন্য root লাগে।)
43	ব্যর্থ — `systemctl is-active` এর লেখা অবস্থাও (`inactive` , `failed`) দেখানো হচ্ছে, সমস্যা বুঝতে সাহায্য করে।
47	`${1%:*}` শেষ `:` থেকে ডান দিক কেটে host, `${1##*:}` শেষ `:` পর্যন্ত বাঁ দিক কেটে port।
48	bash-এর বিশেষ ফিচার: `/dev/tcp/HOST/PORT` আসলে কোনো ফাইল না — এতে redirect করলে bash নিজেই TCP সংযোগ খোলে। আলাদা `bash -c` এ চালানো হয়েছে যাতে `timeout 3` পুরোটা মেরে ফেলতে পারে (ফায়ারওয়াল চুপচাপ প্যাকেট ফেলে দিলে সংযোগ অনেকক্ষণ ঝুলে থাকতে পারত)। `nc -z host port` ও বিকল্প, কিন্তু সব জায়গায় ইনস্টল থাকে না।
56–58	`curl -s` (চুপচাপ), `-o /dev/null` (body ফেলে দাও), `--max-time 5` (সর্বোচ্চ ৫ সেকেন্ড), `-w` দিয়ে শুধু status code ও মোট সময় প্রিন্ট। `read ... < <(...)` দিয়ে দুটো মান দুই variable-এ। সংযোগই না হলে curl `000` প্রিন্ট করে।
59	2xx বা 3xx = ঠিক আছে।
66–69	তিন ধরনের চেক ক্রমানুসারে।
71–77	সমস্যা থাকলে syslog-এ লিখে exit 2 (CRITICAL), না থাকলে 0 (OK)।

```
$ sudo ./health-check.sh -r
== Health check: web01 @ 2026-11-12 10:40:05 ==
Services:
  [ OK ] service nginx চলছে
  [ OK ] service ssh চলছে
```

```
  [INFO] service cron বন্ধ — রিস্টার্ট করছি
  [ OK ] service cron রিস্টার্টের পর চলছে
Ports:
  [ OK ] port localhost:22 খোলা
  [ OK ] port localhost:80 খোলা
HTTP:
  [ OK ] url http://localhost/ → 200 (0.003121s)
  [FAIL] url http://localhost/healthz → 404
মোট সমস্যা: 1
$ echo $?
2
```

প্রতি ৫ মিনিটে চালাতে (`sudo crontab -e`):

```
*/5 * * * * /usr/bin/flock -n /run/lock/health.lock /usr/local/bin/health-check.sh -r
>/dev/null 2>&1
```

> 💡 **টিপস:** systemd নিজেও সার্ভিস মরলে রিস্টার্ট করতে পারে — unit ফাইলে `Restart=on-failure`। তাই এই স্ক্রিপ্টের আসল মূল্য হলো "সার্ভিস চলছে কিন্তু কাজ করছে না" (HTTP 500, পোর্ট বন্ধ) ধরতে পারা। অধ্যায় ২৬ ও cron-এর বদলে systemd timer অধ্যায় ২৯-এ দেখো।

৪৩.১৮ সম্পর্কিত কীবোর্ড শর্টকাট

স্ক্রিপ্ট চালানো, থামানো ও ডিবাগ করার সময় যেগুলো লাগে (টার্মিনালের পূর্ণ তালিকা অধ্যায় ৬-এ, Vim-এর অধ্যায় ১৪-এ):

শর্টকাট	কাজ	স্ক্রিপ্টিং-এ প্রাসঙ্গিকতা
`Ctrl+C`	foreground process-কে SIGINT পাঠায়	`trap ... INT` ধরে; ডিফল্টে exit 130
`Ctrl+\`	SIGQUIT পাঠায়	`Ctrl+C` উপেক্ষা করা process থামাতে; core dump হতে পারে
`Ctrl+Z`	SIGTSTP — process সাময়িক থামায়	`fg` / `bg` দিয়ে ফেরানো (অধ্যায় ২৫)
`Ctrl+D`	ইনপুটের শেষ (EOF)	`read` বা here-doc ইনপুট শেষ করতে
`Ctrl+S` / `Ctrl+Q`	টার্মিনাল আউটপুট থামাও / আবার চালাও	`set -x` এর দ্রুত চলমান trace থামিয়ে পড়তে; ভুলে `Ctrl+S` চাপলে "টার্মিনাল জমে গেছে" মনে হয় — `Ctrl+Q` চাপো
`Ctrl+X Ctrl+E`	বর্তমান কমান্ড `$EDITOR`-এ খোলো	লম্বা one-liner এডিটরে লিখে চালাও
`Ctrl+R`	হিস্ট্রিতে পেছনে খোঁজা	আগে চালানো `bash -x ...` দ্রুত ফেরত আনা
`Alt+.`	আগের কমান্ডের শেষ argument বসাও	`vim script.sh` এর পরে `shellcheck` + `Alt+.`

শর্টকাট	কাজ	স্ক্রিপ্টিং-এ প্রাসঙ্গিকতা
Vim-এ `:!shellcheck %`	এডিটর না ছেড়ে বর্তমান ফাইল চেক	`%` = বর্তমান ফাইলের নাম
Vim-এ `:w !bash`	সেভ না করেই buffer bash-এ চালাও	দ্রুত পরীক্ষা

হাতে-কলমে অনুশীলন

একটা অনুশীলন ফোল্ডার বানাও: `mkdir -p ~/lab43 && cd ~/lab43`

ল্যাব ১ — strict mode-এর ফাঁদ নিজে দেখো

1. `trap1.sh` লেখো:

   ```
   #!/usr/bin/env bash
   set -euo pipefail
   count=0
   ((count++))
   echo "count = $count"
   ```

2. চালাও: `bash trap1.sh; echo "exit=$?"`। কিছু প্রিন্ট হলো না, exit=1 — কেন? (উত্তর ৪৩.২.১-এ)
3. `((count++))` বদলে `count=$((count + 1))` করো, আবার চালাও।
4. `echo $UNDEFINED_VAR` যোগ করে `set -u`-এর error দেখো।
5. `cat /nai | sort; echo ok` লিখে `pipefail` সহ ও ছাড়া (`set +o pipefail`) পার্থক্য দেখো।

ল্যাব ২ — trap ও cleanup

1. একটা স্ক্রিপ্ট লেখো যেটা `mktemp -d` দিয়ে ফোল্ডার বানায়, `trap cleanup EXIT` সেট করে, ফোল্ডারের নাম প্রিন্ট করে, তারপর `sleep 60`।
2. চালিয়ে `Ctrl+C` চাপো। `ls` দিয়ে দেখো temp ফোল্ডার আছে কি না।
3. আরেক টার্মিনাল থেকে `kill <PID>` (SIGTERM) দিয়ে চেষ্টা করো, তারপর `kill -9 <PID>` — পার্থক্যটা লক্ষ করো। কেন `-9`-এ cleanup হলো না?
4. `trap 'exit 130' INT` এবং `trap 'exit 143' TERM` যোগ করে আবার পরীক্ষা করো।

ল্যাব ৩ — ডিবাগিং

1. প্রজেক্ট ২-এর স্ক্রিপ্ট `disk-alert.sh` নামে সেভ করো।
2. `bash -n disk-alert.sh` — syntax ঠিক?
3. `PS4='+ ${LINENO}: ' bash -x disk-alert.sh 2>&1 | head -30` — প্রতিটি লাইন কীভাবে চলে দেখো।

4. ইচ্ছাকৃতভাবে `done < <(df ...)` কে `df ... | while read ...` এ বদলাও; `-w 1` দিয়ে চালিয়ে দেখো `exit` code কী আসে, এবং কেন সবসময় 0।

ল্যাব ৪ — shellcheck

1. `sudo apt install shellcheck` (বা `sudo dnf install ShellCheck`)।
2. ৪৩.৫-এর `example.sh` লিখে `shellcheck example.sh` চালাও।
3. প্রতিটি সতর্কতা ঠিক করো যতক্ষণ না কোনো আউটপুট আসে।
4. এই অধ্যায়ের পাঁচটা প্রজেক্ট স্ক্রিপ্টের ওপর `shellcheck` চালাও — কোনো সতর্কতা এলে বোঝার চেষ্টা করো সেটা আসল সমস্যা কি না।

ল্যাব ৫ — getopts ও lock

1. ৪৩.৬.৩-এর `myhead.sh` লেখো। `-h`, `-n 3`, `-x`, `-n` (মান ছাড়া) দিয়ে চালিয়ে আচরণ দেখো।
2. `-q` (quiet) নামে একটা নতুন অপশন যোগ করো।
3. ৪৩.৭.২-এর lock demo-তে lock পাথ `/tmp/lockdemo.lock` করে (sudo ছাড়া চালানোর জন্য) দুই টার্মিনাল থেকে একসাথে চালাও।
4. প্রথমটা `kill -9` করো, তারপর দ্বিতীয়টা আবার চালাও — lock নিজে থেকে মুক্ত হয়েছে কি না দেখো।

ল্যাব ৬ — নিজের প্রজেক্ট

1. প্রজেক্ট ৪ (`bulk-useradd.sh`) একটা **ভার্চুয়াল মেশিনে বা কন্টেইনারে** (নিজের আসল মেশিনে না!) `-n` দিয়ে প্রথমে চালাও, তারপর আসলে চালাও। শেষে তৈরি করা ইউজার মুছতে `userdel -r` দিয়ে একটা "rollback" স্ক্রিপ্ট নিজে লেখো।
2. ৪৩.১২-এর টেমপ্লেট থেকে শুরু করে একটা `cleanup-old-files.sh` লেখো: `-d DIR -a DAYS` নেবে, `-n` dry-run সমর্থন করবে, আর নির্দিষ্ট দিনের পুরোনো `*.log` ফাইল মুছবে। `shellcheck` পরিষ্কার হওয়া চাই।

সারসংক্ষেপ

বিষয়	কমান্ড / কৌশল	মনে রাখার কথা
Strict mode	`set -euo pipefail`	if-শর্ত, `&&` / `||`, `local x=$(..)`, `((i++))` ফাঁদ
ERR trap উত্তরাধিকার	`set -E` / `set -o errtrace`	function-এর ভেতরেও ERR trap
ঐচ্ছিক variable	`${var:-default}`, `${var:?msg}`	`set -u`-এর সাথে নিরাপদ
pipeline-এর অবস্থা	`${PIPESTATUS[@]}`	প্রতিটি অংশের exit code
Cleanup	`trap cleanup EXIT`	প্রথম লাইনে `local rc=$?`

বিষয়	কমান্ড / কৌশল	মনে রাখার কথা
Signal	`trap 'exit 130' INT`, `trap 'exit 143' TERM`	SIGKILL/SIGSTOP ধরা যায় না
Error এর অবস্থান	`trap '... $LINENO $BASH_COMMAND' ERR`	
Syntax চেক	`bash -n script.sh`	চালায় না
Trace	`bash -x`, `set -x` / `set +x`	গোপন তথ্য ফাঁস হতে পারে
Trace prefix	`PS4='+ ${BASH_SOURCE}:${LINENO}: '`	সিঙ্গেল কোটে
Trace আলাদা ফাইলে	`exec 5>f; BASH_XTRACEFD=5`	
Static analysis	`shellcheck script.sh`	`# shellcheck disable=SCxxxx`
অপশন পার্স	`while getopts ":ab:" opt` + `shift $((OPTIND-1))`	শুধু ছোট অপশন; `:` → মান নেই, `?` → অচেনা
Lock	`exec 9>lock; flock -n 9`	process মরলে আপনা থেকে মুক্ত
cron-এ lock	`flock -n /run/lock/x.lock cmd`	
Atomic lock	`mkdir lockdir`	stale lock সমস্যা
syslog	`logger -t TAG -p user.err "msg"`	`journalctl -t TAG`
সব আউটপুট লগে	`exec > >(tee -a log) 2>&1`	
রঙ	`$'\e[31m'...$'\e[0m'`, `tput setaf 1` / `tput sgr0`	`[[ -t 1 ]]`, `NO_COLOR`
Exit code	0 সফল, 1 সাধারণ, 2 usage, 126, 127, 128+N	সর্বোচ্চ 255
Temp ফাইল	`mktemp`, `mktemp -d`	permission 600/700
কমান্ড আছে?	`command -v cmd`	
root?	`[[ $EUID -eq 0 ]]`	
সময়সীমা	`timeout 30 cmd`	timeout হলে exit 124
subshell ফাঁদ	`while ...; done < <(cmd)`	`cmd | while` এ variable হারায়
লুপে নিরাপদ ইনপুট	`read -u 3 ...; done 3< file`	ভেতরের কমান্ড stdin খায় না

নিজেকে যাচাই করো

1. `set -e` চালু থাকা অবস্থায় `if grep -q foo file; then ...` লাইনে `grep` কিছু না পেলে স্ক্রিপ্ট থামে না কেন?
2. `set -o pipefail` ছাড়া `false | true` এর exit status কত? চালু থাকলে কত?
3. `count=0; ((count++))` `set -e`-যুক্ত স্ক্রিপ্টে কী সমস্যা তৈরি করে, আর এর নিরাপদ বিকল্প কী?
4. `trap cleanup EXIT` থাকা সত্ত্বেও `kill -9` দিলে cleanup হয় না কেন?

5. `PS4` সবসময় সিঙ্গেল কোটে লেখা উচিত কেন?
6. `getopts ":vo:"` এ প্রথম `:` আর `o`-এর পরের `:` — দুটোর মানে কী?
7. `[ -f lockfile ] || touch lockfile` দিয়ে lock করা ভুল কেন, আর `flock` কীভাবে সমস্যা দুটো সমাধান করে?
8. একটা স্ক্রিপ্টের exit code 137 এলে কী বোঝা যায়? 127 এলে?
9. প্রজেক্ট ২-এ `df ... | while read ...` না লিখে `while ...; done < <(df ...)` লেখা হয়েছে কেন?
10. রঙিন আউটপুট দেওয়ার আগে `[[ -t 1 ]]` চেক করা হয় কেন?

▼ উত্তর দেখো

1. `if`, `while`, `until`-এর শর্তে থাকা কমান্ডের ব্যর্থতাকে bash "প্রশ্নের উত্তর" হিসেবে ধরে, error হিসেবে না। তাই `errexit` সেখানে প্রযোজ্য নয়।
2. `pipefail` ছাড়া 0 (শুধু শেষ কমান্ড `true`-এর status)। `pipefail` সহ 1 — ডান দিক থেকে প্রথম non-zero status।
3. post-increment `count++` পুরোনো মান 0 ফেরত দেয়; `(( ))` এর মান 0 হলে exit status 1, ফলে `set -e` স্ক্রিপ্ট চুপচাপ থামিয়ে দেয়। নিরাপদ বিকল্প: `count=$((count + 1))`।
4. SIGKILL kernel সরাসরি কার্যকর করে; process-কে কোনো কোড চালানোর সুযোগ দেয় না, তাই কোনো trap (EXIT সহ) চলে না। SIGKILL ও SIGSTOP ধরা বা উপেক্ষা করা যায় না।
5. যাতে `$LINENO`, `$BASH_SOURCE`, `${FUNCNAME[0]}` ইত্যাদি সেট করার সময় একবার নয়, প্রতিটি trace লাইন ছাপার সময় নতুন করে expand হয়। ডাবল কোটে দিলে সবসময় একই (সেট করার সময়ের) লাইন নম্বর দেখাত।
6. প্রথম `:` = silent error mode: getopts নিজে error প্রিন্ট করে না; অচেনা অপশনে `opt='?'` আর মান না পেলে `opt=':'`, দুই ক্ষেত্রেই `OPTARG`-এ অপশনের অক্ষর। `o:` = `-o` অপশনের একটা মান লাগবে, যেটা `$OPTARG`-এ আসবে।
7. (ক) race condition: চেক আর তৈরি দুটো আলাদা ধাপ, তাই দুটো process একই সময়ে দুজনেই "lock নেই" দেখে ঢুকে যেতে পারে। (খ) stale lock: স্ক্রিপ্ট মারা গেলে ফাইল থেকে যায়। `flock` kernel-এর atomic file lock ব্যবহার করে (race নেই), আর lock একটা খোলা file descriptor-এর সাথে বাঁধা — process মরলে kernel fd বন্ধ করে lock মুক্ত করে দেয়।
8. 137 = 128 + 9 → SIGKILL দিয়ে মারা গেছে (যেমন `kill -9` বা OOM killer)। 127 = কমান্ড খুঁজে পাওয়া যায়নি (command not found)।
9. pipe-এর প্রতিটি অংশ (bash-এ ডিফল্টে শেষ অংশটাও) একটা subshell-এ চলে। তাই `while` লুপের ভেতরে বদলানো `status` / `report` লুপ শেষে মূল shell-এ হারিয়ে যেত। process substitution দিয়ে লুপ মূল shell-এ চলে, variable টিকে থাকে।
10. `-t 1` সত্য হয় যখন stdout একটা টার্মিনাল। আউটপুট ফাইলে, পাইপে বা cron-এর মেইলে গেলে ANSI escape code আবর্জনা (`^[[31m`) হিসেবে দেখা যেত এবং `grep` / `awk` দিয়ে প্রসেস করা কঠিন হতো।

অধ্যায় ৪৪: এনভায়রনমেন্ট, .bashrc, alias ও শেল কাস্টমাইজেশন

এই অধ্যায়ে যা শিখবে

- shell variable আর environment variable-এর পার্থক্য, আর `export` আসলে ভেতরে কী করে
- `env`, `printenv`, `set`, `unset`, `declare`, `readonly` দিয়ে variable দেখা ও নিয়ন্ত্রণ করা
- `PATH` দিয়ে শেল কীভাবে একটা কমান্ড খুঁজে বের করে, আর নিজের ফোল্ডার `PATH`-এ ঠিকভাবে যোগ করা
- গুরুত্বপূর্ণ variable: `HOME`, `USER`, `SHELL`, `PS1`, `LANG`, `EDITOR` এবং আরও কিছু
- login বনাম non-login shell, interactive বনাম non-interactive shell চেনা
- শেল চালু হলে কোন স্টার্টআপ ফাইল কোন ক্রমে পড়া হয় (`/etc/profile`, `~/.bash_profile`, `~/.profile`, `~/.bashrc`, `/etc/bash.bashrc`)
- `alias` ও `unalias` দিয়ে লম্বা কমান্ড ছোট করা
- `PS1` বদলে নিজের মতো রঙিন prompt বানানো
- zsh ও oh-my-zsh-এর পরিচিতি
- dotfiles কী, আর Git দিয়ে কীভাবে সেগুলো এক মেশিন থেকে আরেক মেশিনে নেওয়া যায়

৪৪.১ এনভায়রনমেন্ট কী, আর কেন এটা জানা দরকার

ধরো তুমি একটা নতুন অফিসে কাজ শুরু করলে। তোমার ডেস্কে একটা নোটবোর্ড আছে, তাতে লেখা: "তোমার নাম: রহিম", "প্রিন্টার: ৩ তলায়", "ভাষা: বাংলা", "জরুরি ফোন: ১০১"। অফিসে যে-ই তোমার হয়ে কোনো কাজ করে, সে আগে এই বোর্ডটা দেখে নেয়।

লিনাক্সে প্রতিটি process (চলমান প্রোগ্রাম, বিস্তারিত অধ্যায় ২৪-এ) এমনই একটা "নোটবোর্ড" নিয়ে চলে। একে বলে **environment** (এনভায়রনমেন্ট, পরিবেশ)। এটা আসলে কিছু `NAME=value` জোড়ার একটা তালিকা:

```
HOME=/home/rahim
USER=rahim
LANG=en_US.UTF-8
PATH=/usr/local/bin:/usr/bin:/bin
```

প্রোগ্রামগুলো এই তালিকা পড়ে ঠিক করে কীভাবে চলবে। যেমন:

- `ls` দেখে `LANG` / `LC_*`, তারপর ঠিক করে তারিখ কোন ফরম্যাটে দেখাবে
- `git commit` দেখে `EDITOR` বা `VISUAL`, তারপর ঠিক করে কোন এডিটর খুলবে

- শেল দেখে `PATH`, তারপর ঠিক করে `python3` লিখলে আসলে কোন ফাইলটা চালাবে

এই অধ্যায় শেষ করলে তুমি এই নোটবোর্ড পড়তে পারবে, লিখতে পারবে, আর প্রতিবার টার্মিনাল খুললে সেটা যেন নিজে থেকেই ঠিকমতো সাজানো থাকে, সেই ব্যবস্থাও করতে পারবে।

৪৪.২ Shell variable বনাম Environment variable

শেল স্ক্রিপ্টিং অধ্যায়ে (অধ্যায় ৪১) তুমি variable বানানো শিখেছ:

```
$ name="Rahim"
$ echo "$name"
Rahim
```

এটা একটা **shell variable** (শেল ভ্যারিয়েবল)। এটা শুধু এই শেলের নিজের মেমোরিতে থাকে। এই শেল থেকে নতুন কোনো প্রোগ্রাম (child process) চালালে সে এটা দেখতে পায় না।

Environment variable (এনভায়রনমেন্ট ভ্যারিয়েবল) হলো এমন variable যেটা child process-এর কাছেও পৌঁছায়।

হাতে-কলমে পার্থক্যটা দেখো

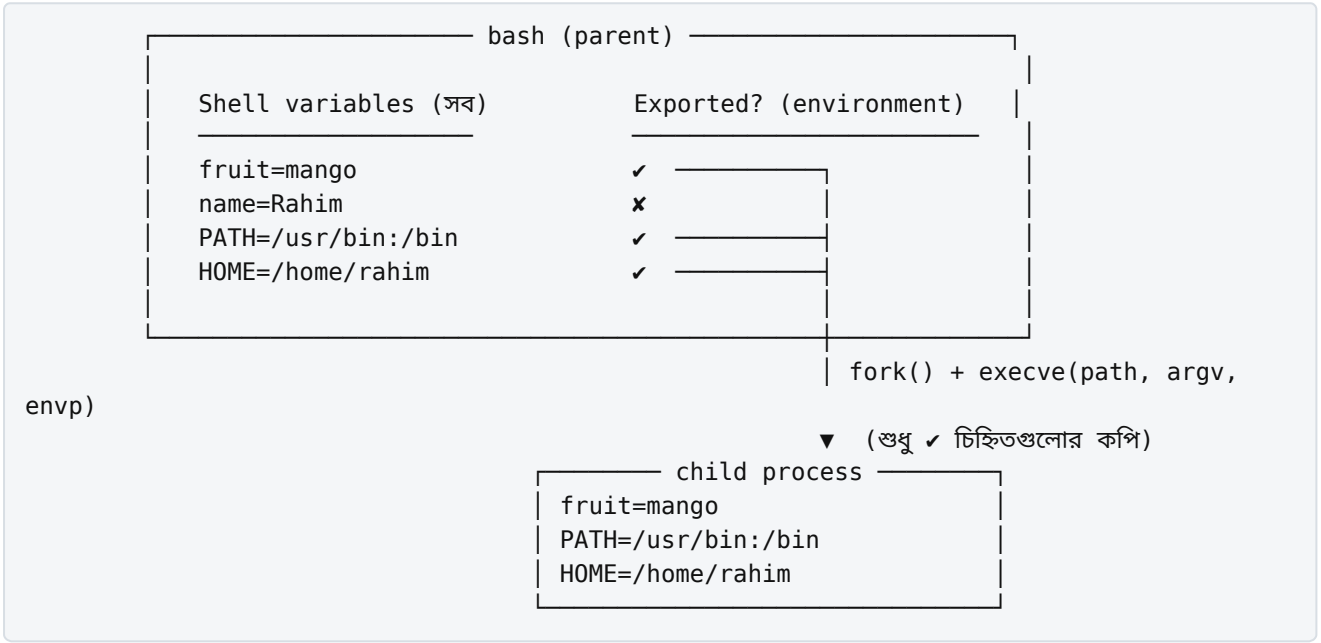
```
$ fruit="mango"          # শুধু shell variable
$ bash                   # নতুন child shell চালু করলাম
$ echo "fruit = $fruit"
fruit =
$ exit                   # child shell থেকে বের হলাম

$ export fruit           # এবার environment-এ তুলে দিলাম
$ bash
$ echo "fruit = $fruit"
fruit = mango
$ exit
```

প্রথমবার child shell কিছুই পায়নি। `export` করার পর পেয়েছে।

ভেতরে কী হচ্ছে (under the hood)

লিনাক্সে নতুন প্রোগ্রাম চালু হয় দুই ধাপে: `fork()` আর `execve()` system call দিয়ে (অধ্যায় ২ ও অধ্যায় ২৪ দেখো)। `execve()`-কে তিনটা জিনিস দিতে হয়: প্রোগ্রামের পাথ, argument তালিকা, আর **environment তালিকা** (`envp`)। Kernel এই তালিকাটা নতুন process-এর মেমোরিতে কপি করে দেয়।

শেল তার সব variable দুই ভাগে রাখে। যেগুলোর গায়ে "export" চিহ্ন আছে, শুধু সেগুলোই `envp` তালিকায় যায়।



> 📝 **মনে রাখো:** environment হলো **কপি**, শেয়ার না। child যদি নিজের `fruit` বদলে ফেলে, parent-এর `fruit` বদলায় না। তথ্য সবসময় parent থেকে child-এর দিকে যায়, উল্টো দিকে কখনো না।

এই কারণেই একটা স্ক্রিপ্টের ভেতরে `export PATH=...` লিখে স্ক্রিপ্ট চালালে তোমার বর্তমান টার্মিনালের `PATH` বদলায় না। স্ক্রিপ্টটা আলাদা child process-এ চলে, আর শেষ হলে তার environment-ও হারিয়ে যায়। বর্তমান শেলে প্রভাব ফেলতে চাইলে স্ক্রিপ্টটা `source` করতে হয় (৪৪.৮ অংশে দেখবে)।

উপমা দিলে: বাবা ছেলেকে বাজারের একটা লিস্টের ফটোকপি দিলেন। ছেলে নিজের কপিতে কাটাকাটি করলে বাবার আসল লিস্ট বদলায় না।

যেকোনো চলমান process-এর environment দেখা

প্রতিটি process-এর environment `/proc/<PID>/environ` ফাইলে দেখা যায় (অধ্যায় ৪৭)। এখানে entry-গুলো NUL (`\0`) ক্যারেক্টার দিয়ে আলাদা থাকে, তাই `tr` দিয়ে নতুন লাইনে বদলে নাও:

```
$ tr '\0' '\n' < /proc/$$/environ | head -5
SHELL=/bin/bash
LANG=en_US.UTF-8
HOME=/home/rahim
USER=rahim
LOGNAME=rahim
```

`$$` মানে বর্তমান শেলের PID।

> ⚠️ **সতর্কতা:** `/proc/PID/environ` দেখায় process **শুরু হওয়ার সময়** তার environment কেমন ছিল। পরে শেলের ভেতরে `export` করলে এই ফাইলে সেটা দেখা যাবে না। বর্তমান অবস্থা দেখতে `env` বা

> `printenv` ব্যবহার করো।

৪৪.৩ Variable দেখা ও পরিচালনার কমান্ড

`export` — variable-কে environment-এ পাঠানো

Syntax:

```
export NAME=value      # বানাও এবং export করো
export NAME            # আগে থেকে থাকা variable export করো
export -n NAME         # export চিহ্ন সরাও (variable থাকবে, শুধু shell variable হয়ে যাবে)
export -p              # সব exported variable দেখাও
export -f funcname     # একটা shell function export করো (bash-specific)
```

অপশন	কাজ
`-p`	সব exported variable `declare -x` ফরম্যাটে দেখায়
`-n`	export চিহ্ন তুলে দেয়
`-f`	variable না, function export করে (child bash পাবে)

```
$ export EDITOR=nano
$ export -p | grep EDITOR
declare -x EDITOR="nano"
$ export -n EDITOR
$ export -p | grep EDITOR
$                          # কিছু নেই — কিন্তু $EDITOR এখনো শেলে আছে
$ echo $EDITOR
nano
```

> 💡 **টিপস:** `=`-এর দুই পাশে কোনো স্পেস দেবে না। `export A = 5` লিখলে শেল ভাববে তুমি `A`, `=` আর `5` নামের তিনটা জিনিস export করতে চাইছ, আর error দেবে।

`env` — environment দেখা অথবা পরিবর্তিত environment-এ কমান্ড চালানো

`env` আসলে একটা আলাদা প্রোগ্রাম (`/usr/bin/env`, GNU coreutils-এর অংশ)। এর দুটো কাজ:

1. কোনো argument না দিলে বর্তমান environment প্রিন্ট করে
2. `env NAME=value command` দিলে **শুধু ওই কমান্ডের জন্য** environment বদলে কমান্ডটা চালায়

অপশন	কাজ
(কিছু না)	সব environment variable দেখায়

অপশন	কাজ
`-i` বা `-`	একদম খালি environment দিয়ে শুরু করে
`-u NAME`	ওই variable সরিয়ে দিয়ে কমান্ড চালায়
`-0`	প্রতিটা লাইন newline-এর বদলে NUL দিয়ে শেষ করে
`-C DIR`	কমান্ড চালানোর আগে DIR-এ যায় (coreutils 8.28+)
`-S "string"`	স্ট্রিংটাকে ভেঙে একাধিক argument বানায় (shebang-এ কাজে লাগে)

```
$ env | head -3
SHELL=/bin/bash
PWD=/home/rahim
LOGNAME=rahim

$ env LANG=C date               # শুধু এই date কমান্ড ইংরেজি "C" locale-এ চলবে
Thu Nov 12 10:15:42 AM +06 2026

$ env -i bash -c 'echo "PATH=[$PATH] HOME=[$HOME]"'
PATH=[/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games] HOME=[]
```

শেষ উদাহরণে খেয়াল করো: `env -i` দিয়ে environment খালি করলেও bash নিজে থেকে একটা ডিফল্ট `PATH` বসিয়ে নিয়েছে, কিন্তু `HOME` খালি। কোনো স্ক্রিপ্ট "পরিষ্কার" environment-এ ঠিকমতো চলে কিনা, সেটা পরীক্ষা করতে `env -i` খুব কাজের। (bash-এর এই ডিফল্ট `PATH`-এর মান কম্পাইলের সময় ঠিক হয়, তাই distro ভেদে একটু আলাদা হতে পারে।)

শেলেও একই কাজ `env` ছাড়া করা যায়:

```
$ LANG=C date          # কমান্ডের আগে NAME=value লিখলে শুধু ওই কমান্ডের জন্য প্রযোজ্য
```

স্ক্রিপ্টের প্রথম লাইনে `#!/usr/bin/env python3` লেখার রহস্যও এটাই: `env` তোমার `PATH`-এ `python3` খুঁজে সেটা চালায়। ফলে python যে মেশিনে যেখানেই ইনস্টল থাকুক, স্ক্রিপ্ট চলবে।

`printenv` — নির্দিষ্ট variable-এর মান

```
$ printenv HOME
/home/rahim
$ printenv HOME USER
/home/rahim
rahim
$ printenv NOT_EXIST; echo "exit code: $?"
exit code: 1
```

`printenv`-এর সুবিধা হলো variable না থাকলে exit code 1 দেয়, তাই স্ক্রিপ্টে পরীক্ষা করা সহজ। আর এটা **শুধু environment** দেখায়, export না করা shell variable দেখায় না:

```
$ secret=123
$ printenv secret; echo $?
```

```
1
$ echo $secret
123
```

`set` — সবকিছু দেখা, এবং শেলের অপশন বদলানো

argument ছাড়া `set` চালালে **সব** shell variable (exported আর non-exported দুটোই) **এবং সব function** দেখায়। তাই আউটপুট অনেক লম্বা হয়:

```
$ set | wc -l
2873
$ set | less
```

(সংখ্যাটা তোমার মেশিনে আলাদা হবে, কারণ bash-completion অনেক function লোড করে।)

`set`-এর আসল আরেকটা কাজ হলো শেলের আচরণ বদলানো (স্ক্রিপ্টিং অধ্যায়ে `set -euo pipefail` দেখেছ, অধ্যায় ৪৩):

কমান্ড	কাজ
`set -o`	সব অপশন কোনটা on, কোনটা off দেখায়
`set -o vi` / `set -o emacs`	কমান্ড লাইন এডিটিং vi বা emacs মোডে (অধ্যায় ৬)
`set -o noclobber` বা `set -C`	`>` দিয়ে আগে থাকা ফাইল overwrite আটকায়
`set -x`	প্রতিটা কমান্ড চালানোর আগে প্রিন্ট করে (ডিবাগ)
`set -u`	অসংজ্ঞায়িত variable ব্যবহার করলে error
`set +o NAME`	অপশন বন্ধ করে (`+` মানে বন্ধ, `-` মানে চালু)

`env` / `printenv` / `set` / `declare` — কোনটা কী দেখায়

কমান্ড	Exported variable	Shell-only variable	Function	ধরন
`env`	✔	✘	✘	বাইরের প্রোগ্রাম
`printenv`	✔	✘	✘	বাইরের প্রোগ্রাম
`export -p`	✔	✘	✘	শেল builtin
`set`	✔	✔	✔	শেল builtin
`declare -p`	✔	✔	✘	শেল builtin
`declare -x`	✔	✘	✘	শেল builtin

> 📝 **মনে রাখো:** `env` আর `printenv` বাইরের প্রোগ্রাম। শেল এগুলো চালানোর সময় যা পাঠায় শুধু তা-ই এরা দেখে, অর্থাৎ শুধু exported variable। `set` শেলের নিজের অংশ (builtin), তাই শেলের ভেতরের

> সবকিছু দেখতে পায়।

`unset` — variable বা function মুছে ফেলা

```
$ fruit=mango
$ unset fruit
$ echo "[$fruit]"
[]

$ greet() { echo hi; }
$ unset -f greet         # function মুছতে -f
$ unset -v fruit         # শুধু variable (ডিফল্ট আচরণও এটাই)
```

অপশন	কাজ
`-v`	variable মোছে
`-f`	function মোছে

> ⚠️ **সতর্কতা:** `unset PATH` করলে বর্তমান শেলে `ls`, `cat` কিছুই খুঁজে পাবে না ("command not found")। ভয় পাওয়ার কিছু নেই, টার্মিনাল বন্ধ করে আবার খোলো, বা `export PATH=/usr/local/bin:/usr/bin:/bin` টাইপ করো। Builtin (`cd`, `echo`, `export`) তখনও কাজ করবে, কারণ ওগুলোর জন্য PATH লাগে না।

`declare` ও `readonly`

```
$ declare -x COLOR=blue      # export করা variable বানানো (export COLOR=blue-এর সমান)
$ declare -r PI=3.14         # read-only
$ PI=3
bash: PI: readonly variable
$ readonly                   # সব readonly variable-এর তালিকা
$ declare -p HOME            # একটা variable-এর ধরন ও মান দেখা
declare -x HOME="/home/rahim"
```

`declare -p`-এর আউটপুটে `-x` মানে exported, `-r` মানে readonly, `-i` মানে integer, `-a` মানে indexed array, `-A` মানে associative array। readonly variable বর্তমান শেলে আর `unset` করা যায় না।

৪৪.৪ PATH — শেল কীভাবে কমান্ড খুঁজে পায়

সমস্যাটা কী

তুমি `ls` লিখলে। কিন্তু `ls` আসলে একটা ফাইল, `/usr/bin/ls`। তুমি তো পুরো পাথ লেখোনি। তাহলে শেল জানল কীভাবে?

উত্তর `PATH` variable:

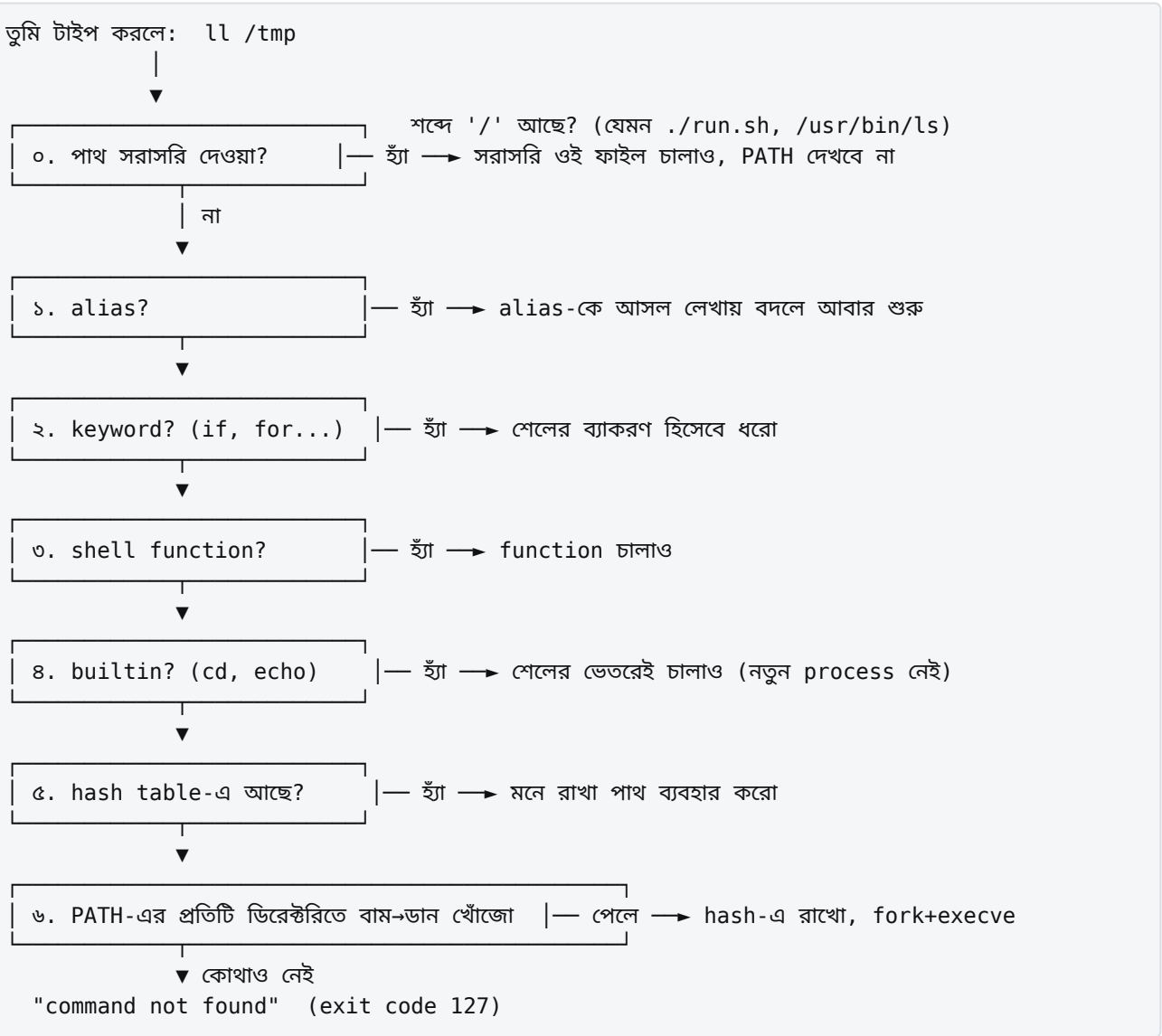
```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
```

এটা কোলন (`:`) দিয়ে আলাদা করা কিছু ডিরেক্টরির তালিকা। শেল **বাম থেকে ডানে** একটা একটা করে ডিরেক্টরিতে খোঁজে, আর **প্রথম যেখানে পায়** সেটাই চালায়।

উপমা: তুমি চাবি খুঁজছ। প্রথমে পকেটে, তারপর ব্যাগে, তারপর টেবিলের ড্রয়ারে। পকেটে পেয়ে গেলে ব্যাগ আর খোলো না। ব্যাগে আরেকটা চাবি থাকলেও সেটা কাজে লাগে না।

কমান্ড খোঁজার সম্পূর্ণ ক্রম

তুমি Enter চাপার পর bash নিচের ক্রমে খোঁজে:



> 💡 **টিপস:** Ubuntu/Debian-এ কমান্ড না পেলে "Command 'x' not found, but can be installed with: sudo apt install ..." বার্তা আসে। এটা `command-not-found` প্যাকেজের কাজ, যেটা bash-এর `command_not_found_handle` function ব্যবহার করে। Fedora-তে একই কাজ করে `PackageKit-command-not-found`।

`type`, `command -v`, `which`, `hash` — কমান্ড আসলে কোথা থেকে আসছে

```
$ type ls
ls is aliased to `ls --color=auto'
$ type cd
cd is a shell builtin
$ type -a echo
echo is a shell builtin
echo is /usr/bin/echo
echo is /bin/echo
$ type -t if
keyword
$ command -v python3
/usr/bin/python3
$ which python3
/usr/bin/python3
```

কমান্ড	কী বলে	মন্তব্য
`type NAME`	alias / keyword / function / builtin / file	সবচেয়ে নির্ভরযোগ্য (bash builtin)
`type -a NAME`	সব জায়গা যেখানে NAME পাওয়া যায়	PATH-এ একাধিক কপি ধরতে কাজে লাগে
`type -t NAME`	শুধু এক শব্দে ধরন	স্ক্রিপ্টে কাজে লাগে
`command -v NAME`	কোন পাথ চালানো হবে	POSIX, স্ক্রিপ্টে "আছে কি না" পরীক্ষায় সেরা
`which NAME`	PATH-এ ফাইলের পাথ	বাইরের প্রোগ্রাম, alias/builtin চেনে না
`hash`	এই সেশনে মনে রাখা পাথ ও কতবার চলেছে	
`hash -r`	মনে রাখা সব পাথ ভুলে যাও	

`type -a echo`-তে `/usr/bin/echo` আর `/bin/echo` দুটোই আসার কারণ: আধুনিক Ubuntu, Debian, Fedora, RHEL-এ `/bin` আসলে `/usr/bin`-এর symlink ("usr merge", অধ্যায় ৭ দেখো)।

Hash table: কেন মাঝে মাঝে "ভুল" প্রোগ্রাম চলে

প্রতিবার PATH-এর সব ডিরেক্টরি ঘাঁটলে সময় নষ্ট হয়। তাই bash একবার কোনো কমান্ড খুঁজে পেলে তার পাথ একটা hash table-এ মনে রাখে:

```
$ hash
hits    command
```

```
   3    /usr/bin/ls
   1    /usr/bin/git
```

ধরো তুমি `/usr/local/bin/node`-এ নতুন node ইনস্টল করলে, কিন্তু এই সেশনে আগে `/usr/bin/node` চালিয়েছিলে। bash পুরনো মনে রাখা পাথটা ব্যবহার করতে পারে। আর পুরনো ফাইলটা মুছে ফেললে bash নিজেই আবার খোঁজে। এ রকম গোলমাল হলে:

```
$ hash -r        # ভুলে যাও, নতুন করে খোঁজো
```

PATH-এ নিজের ডিরেক্টরি যোগ করা

ধরো তোমার নিজের স্ক্রিপ্টগুলো `~/bin`-এ রাখো, আর চাও যেকোনো জায়গা থেকে শুধু নাম লিখে চালাতে।

ধাপ ১: শুধু বর্তমান টার্মিনালে (সাময়িক)

```
$ mkdir -p ~/bin
$ export PATH="$HOME/bin:$PATH"     # শুরুতে যোগ করলাম
$ echo $PATH
/home/rahim/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:...
```

টার্মিনাল বন্ধ করলেই এটা হারিয়ে যাবে।

ধাপ ২: স্থায়ী করা

লাইনটা `~/.bashrc`-এর শেষে যোগ করো (কেন `.bashrc`, সেটা ৪৪.৭ অংশে বুঝবে):

```
$ echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc
$ source ~/.bashrc
```

শুরুতে না শেষে?

লেখার ধরন	ফল
`PATH="$HOME/bin:$PATH"`	তোমার ডিরেক্টরি আগে খোঁজা হবে। একই নামের সিস্টেম কমান্ডকে "ঢেকে" দেবে
`PATH="$PATH:$HOME/bin"`	সিস্টেম কমান্ড আগে। তোমারটা শুধু তখনই চলবে যখন সিস্টেমে ওই নামের কিছু নেই

> 💡 **টিপস:** Ubuntu/Debian-এর ডিফল্ট `~/.profile`-এ আগে থেকেই এই কোড আছে: `~/bin` বা `~/.local/bin` থাকলে PATH-এ যোগ করে। তাই ফোল্ডারটা বানিয়ে একবার logout/login করলেই কাজ হয়ে যায়। Fedora/RHEL-এর ডিফল্ট `~/.bashrc`-ও `~/.local/bin` আর `~/bin` যোগ করে। `pip install --user` আর `pipx` প্রোগ্রাম রাখে `~/.local/bin`-এ।

একই ডিরেক্টরি বারবার যোগ হওয়া ঠেকানো:

`.bashrc` প্রতিটা নতুন শেলে চলে। তাই child shell খুললে বা বারবার `source` করলে PATH-এ একই ডিরেক্টরি জমতে থাকে। নিরাপদ উপায়:

```
# ~/.bashrc-এ
case ":$PATH:" in
    *":$HOME/bin:"*) ;;                          # আগে থেকেই আছে, কিছু করো না
    *) export PATH="$HOME/bin:$PATH" ;;
esac
```

শুরু আর শেষে `:` বসানোর কৌশলের কারণে PATH-এর প্রথম বা শেষ এন্ট্রিও সঠিকভাবে মিলবে।

> ⚠️ **সতর্কতা: PATH-এ কখনো `.` (বর্তমান ডিরেক্টরি) রাখবে না।** কেউ `/tmp`-এ `ls` নামের একটা ক্ষতিকর স্ক্রিপ্ট রেখে দিলে, আর তুমি `/tmp`-এ গিয়ে `ls` লিখলে, সেটা চলবে। এ কারণেই লিনাক্সে নিজের ফোল্ডারের স্ক্রিপ্ট চালাতে `./script.sh` লিখতে হয়। খালি এন্ট্রিও (`::`, বা শুরু/শেষে `:`) বর্তমান ডিরেক্টরি বোঝায়, তাই `PATH=":$PATH"` জাতীয় ভুলও বিপজ্জনক।

> ⚠️ **সতর্কতা:** `export PATH=$HOME/bin` (পুরনো `$PATH` ছাড়া) লিখলে বাকি সব ডিরেক্টরি হারিয়ে যাবে। সবসময় `$PATH` রাখো।

সিস্টেম-জুড়ে PATH

- `/etc/environment`: Ubuntu/Debian-এ সিস্টেমের ডিফল্ট PATH এখানে থাকে। এটা **শেল স্ক্রিপ্ট নয়**। এটা পড়ে PAM-এর `pam_env` মডিউল লগইনের সময়। তাই এখানে শুধু `NAME="value"` লেখা যায়, `$PATH` বা `export` চলে না।
- `/etc/profile` আর `/etc/profile.d/*.sh`: সব ইউজারের login shell এগুলো চালায়। সব ইউজারের জন্য PATH যোগ করতে `/etc/profile.d/myapp.sh` নামে একটা ফাইল বানানোই প্রচলিত ও পরিষ্কার পদ্ধতি।
- `sudo`-এর PATH আলাদা: `/etc/sudoers`-এর `secure_path` সেটা ঠিক করে (অধ্যায় ১৮)। তাই `~/bin`-এর কমান্ড `sudo mycmd` দিয়ে চালালে "command not found" আসতে পারে।

৪৪.৫ গুরুত্বপূর্ণ Environment Variable-গুলো

Variable	কী রাখে	উদাহরণ মান	কে সেট করে
`HOME`	তোমার হোম ডিরেক্টরি	`/home/rahim`	`login`/PAM, `/etc/passwd` থেকে
`USER`	বর্তমান ইউজারনেম	`rahim`	login প্রক্রিয়া
`LOGNAME`	লগইন নাম (POSIX)	`rahim`	login প্রক্রিয়া

Variable	কী রাখে	উদাহরণ মান	কে সেট করে
`SHELL`	তোমার **ডিফল্ট লগইন শেল**	`/bin/bash`	`/etc/passwd`-এর শেষ ঘর
`PATH`	কমান্ড খোঁজার ডিরেক্টরি তালিকা	`/usr/local/bin:/usr/bin:/bin`	PAM, `/etc/profile`, `.profile`
`PWD`	বর্তমান ডিরেক্টরি	`/home/rahim/docs`	শেল (প্রতিটা `cd`-তে)
`OLDPWD`	আগের ডিরেক্টরি (`cd -` এটা ব্যবহার করে)	`/tmp`	শেল
`PS1`	প্রধান prompt-এর চেহারা	`\u@\h:\w\$`	`.bashrc`
`LANG`	ভাষা, অঞ্চল ও ক্যারেক্টার এনকোডিং	`en_US.UTF-8`, `bn_BD.UTF-8`	locale সেটিং
`LC_ALL`	সব `LC_*` ও `LANG`-কে ছাপিয়ে যায়	`C`	সাধারণত ফাঁকা
`EDITOR`	ডিফল্ট টেক্সট এডিটর	`nano`, `vim`	তুমি
`VISUAL`	পূর্ণ-স্ক্রিন এডিটর (থাকলে `EDITOR`-এর আগে দেখা হয়)	`vim`	তুমি
`PAGER`	লম্বা আউটপুটের pager	`less`	তুমি
`TERM`	টার্মিনালের ধরন (রঙ, কী-কোড)	`xterm-256color`	টার্মিনাল এমুলেটর
`HOSTNAME`	মেশিনের নাম	`myserver`	bash নিজে
`UID`	তোমার ইউজার আইডি (readonly)	`1000`	bash নিজে
`HISTSIZE` / `HISTFILE`	হিস্ট্রির আকার ও ফাইল (অধ্যায় ৬)	`1000` / `~/.bash_history`	`.bashrc`
`TZ`	টাইমজোন	`Asia/Dhaka`	ঐচ্ছিক
`XDG_CONFIG_HOME`	কনফিগ ফাইলের জায়গা (না থাকলে `~/.config` ধরা হয়)	`~/.config`	ঐচ্ছিক
`DISPLAY` / `WAYLAND_DISPLAY`	GUI কোন ডিসপ্লেতে দেখাবে	`:0` / `wayland-0`	ডেস্কটপ সেশন (অধ্যায় ৪)

`SHELL` নিয়ে একটা সাধারণ ভুল ধারণা

`$SHELL` **বর্তমানে চলমান শেল বলে না**। এটা বলে `/etc/passwd`-এ তোমার জন্য লেখা লগইন শেল কোনটা।

```
$ echo $SHELL
/bin/bash
$ zsh                  # এখন zsh-এর ভেতরে
% echo $SHELL
/bin/bash              # তবুও bash দেখাচ্ছে!
% echo $0
zsh                    # এটাই আসল চলমান শেল
% ps -p $$ -o comm=
zsh
```

ডিফল্ট লগইন শেল বদলাতে `chsh -s /usr/bin/zsh` (৪৪.১১ অংশ)।

`LANG` ও locale

`LANG` ঠিক করে প্রোগ্রাম কোন ভাষায় মেসেজ দেখাবে, তারিখ-সংখ্যা কীভাবে লিখবে, sort কোন ক্রমে হবে, আর ক্যারেক্টার এনকোডিং কী হবে। আরও সূক্ষ্ম নিয়ন্ত্রণের জন্য আলাদা `LC_*` variable আছে।

```
$ locale
LANG=en_US.UTF-8
LC_CTYPE="en_US.UTF-8"
LC_NUMERIC="en_US.UTF-8"
LC_TIME="en_US.UTF-8"
...
LC_ALL=
$ locale -a                # ইনস্টল করা সব locale
C
C.utf8
en_US.utf8
POSIX
```

অগ্রাধিকার: **`LC_ALL` > নির্দিষ্ট `LC_xxx` > `LANG`**।

Variable	কী নিয়ন্ত্রণ করে
`LC_CTYPE`	ক্যারেক্টারের ধরন ও এনকোডিং
`LC_NUMERIC`	দশমিক চিহ্ন (`.` না `,`)
`LC_TIME`	তারিখ ও সময়ের ফরম্যাট
`LC_COLLATE`	sort-এর ক্রম
`LC_MONETARY`	মুদ্রার ফরম্যাট
`LC_MESSAGES`	প্রোগ্রামের মেসেজের ভাষা

> 💡 **টিপস:** স্ক্রিপ্টে আউটপুট parse করার সময় `LC_ALL=C` ব্যবহার করো। এতে মেসেজ ইংরেজিতে আর ফরম্যাট সবসময় একই রকম থাকে, আর `sort` / `grep` দ্রুত চলে। যেমন `LC_ALL=C sort file.txt`।

সিস্টেমের locale সেট করা: Ubuntu/Debian-এ `sudo locale-gen bn_BD.UTF-8` দিয়ে locale তৈরি করো, তারপর `sudo update-locale LANG=en_US.UTF-8`। systemd-ভিত্তিক সব distro-তে (Fedora/RHEL সহ) `localectl set-locale LANG=en_US.UTF-8`। Fedora/RHEL-এ নতুন ভাষা লাগলে `glibc-langpack-bn` জাতীয় প্যাকেজ ইনস্টল করতে হয়।

`EDITOR` ও `VISUAL`

`git commit`, `crontab -e`, `visudo`, `sudoedit`, `less`-এ `v` চাপা — এরা সবাই এই variable দেখে কোন এডিটর খুলবে ঠিক করে।

```
$ export EDITOR=nano
$ export VISUAL=vim
```

ঐতিহাসিকভাবে `EDITOR` ছিল লাইন-এডিটরের (যেমন `ed`) জন্য, `VISUAL` পূর্ণ-স্ক্রিন এডিটরের জন্য। আজকাল বেশিরভাগ প্রোগ্রাম আগে `VISUAL` দেখে, না থাকলে `EDITOR`। দুটোতে একই মান দেওয়াই সহজ। Debian/Ubuntu-তে এগুলো না থাকলে `sensible-editor` ব্যবহার হয়, আর সিস্টেম-ডিফল্ট বদলানো যায় `sudo update-alternatives --config editor` দিয়ে, অথবা প্রথমবার `select-editor` জিজ্ঞেস করে।

৪৪.৬ শেলের চার রকম চরিত্র: Login / Non-login, Interactive / Non-interactive

স্টার্টআপ ফাইল বোঝার আগে জানতে হবে শেল কোন "মোডে" চালু হয়েছে। দুটো আলাদা প্রশ্ন আছে:

প্রশ্ন ১: Interactive (ইন্টারঅ্যাকটিভ) কি না? মানে, শেল কি তোমার সঙ্গে কথা বলছে (prompt দেখিয়ে কমান্ডের অপেক্ষা করছে), নাকি একটা স্ক্রিপ্ট চালিয়ে চুপচাপ শেষ হয়ে যাবে?

প্রশ্ন ২: Login (লগইন) কি না? মানে, এটা কি এই সেশনের প্রথম শেল, যেটা তোমার পরিচয় যাচাইয়ের (লগইন) ঠিক পরে চালু হয়েছে?

	Interactive	**Non-interactive**
Login	`ssh user@server`, tty-তে (Ctrl+Alt+F3) লগইন, `su - rahim`, `sudo -i`, `bash --login`	`bash -l script.sh`, `bash --login -c 'cmd'` (বিরল)
Non-login	ডেস্কটপে টার্মিনাল খোলা (GNOME Terminal-এর ডিফল্ট), টার্মিনালে `bash` টাইপ করা, `su rahim` (মাইনাস ছাড়া)	`bash script.sh`, `./script.sh`, cron job, `ssh server 'ls'`

> 📝 **মনে রাখো:** `ssh server 'ls'` (কমান্ড সহ ssh) একটা **non-login, non-interactive** শেল চালায়। তবে bash-এর একটা বিশেষ নিয়ম আছে: সে বুঝতে পারলে যে ssh থেকে চালু হয়েছে, তখন non-interactive হলেও `~/.bashrc` পড়ে। এ কারণেই Ubuntu/Debian-এর `~/.bashrc`-এর শুরুতে একটা পাহারা থাকে, যেটা non-interactive হলে বাকি ফাইল পড়া বন্ধ করে দেয় (৪৪.৭)।

নিজের শেল কোন মোডে, কীভাবে বুঝবে

```
# login কি না
$ shopt -q login_shell && echo "login" || echo "non-login"
non-login

# আরেক উপায়: $0-এর আগে '-' থাকলে login
$ echo $0
bash            # non-login
```

```
$ echo $0       # (ssh বা tty লগইনে)
-bash           # login

# interactive কি না: $- variable-এ 'i' অক্ষর আছে কি না
$ echo $-
himBHs
$ [[ $- == *i* ]] && echo "interactive"
interactive
```

`$0`-এর সামনের `-` চিহ্নটা আসে `login` বা `sshd` প্রোগ্রাম থেকে। এরা শেল চালানোর সময় ইচ্ছা করে argv[0]-এ `-bash` লিখে দেয়। শেল এটা দেখেই বোঝে সে login shell।

কেন এই ভাগ?

ইতিহাসটা বুঝলে কারণটা পরিষ্কার হয়। আগেকার দিনে তুমি একবার লগইন করতে, আর সেই একটা শেলের ভেতর থেকেই সারা দিন কাজ করতে। তাই:

- **একবারই যা করা দরকার** (environment variable সেট করা, যেমন PATH; লগইনের বার্তা দেখানো) তা login shell-এ করা হতো। child-রা তো environment উত্তরাধিকারসূত্রে পাবেই।
- **প্রতিটা শেলে যা দরকার, কিন্তু উত্তরাধিকারসূত্রে আসে না** (alias, function, prompt, শেল অপশন) তা প্রতিটা interactive শেলে আলাদা করে করতে হতো।

মনে রাখো: alias আর shell function `execve()` দিয়ে child-এর কাছে যায় না, কারণ ওগুলো environment-এর অংশ না। তাই প্রতিটা নতুন শেলে এগুলো আবার লোড করতে হয়।

৪৪.৭ স্টার্টআপ ফাইল: কোনটা কখন পড়া হয়

bash-এর নিয়ম (`man bash`-এর INVOCATION অংশ অনুযায়ী)

Interactive login shell (বা `--login` দিয়ে চালানো যেকোনো শেল):

1. `/etc/profile` পড়ে (থাকলে)
2. তারপর `~/.bash_profile`, `~/.bash_login`, `~/.profile` — এই তিনটার মধ্যে **প্রথম যেটা পাওয়া যায় শুধু সেটাই** পড়ে, বাকিগুলো উপেক্ষা করে
3. logout করার সময় `~/.bash_logout` পড়ে (আর থাকলে `/etc/bash.bash_logout`)

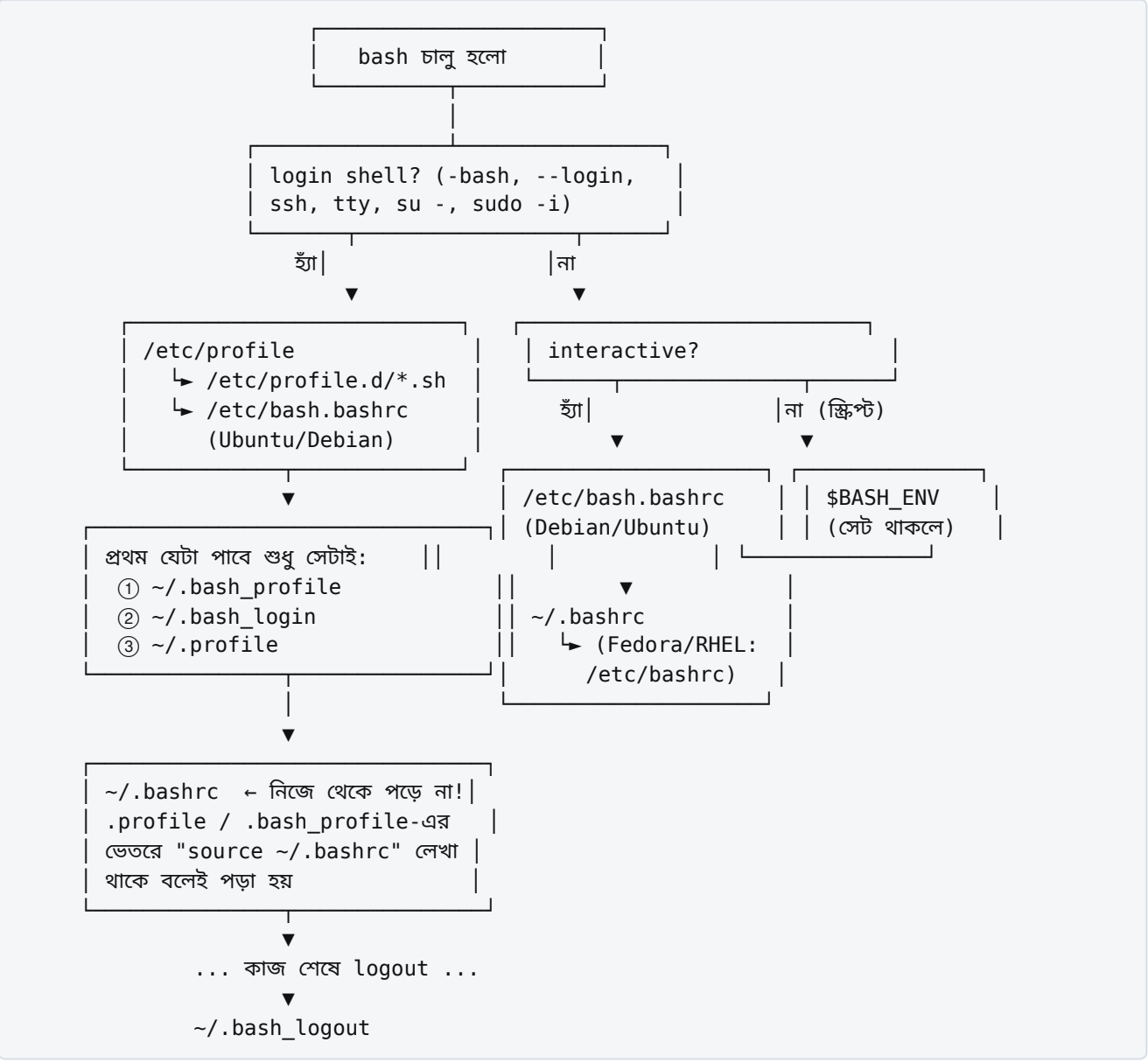
Interactive non-login shell:

1. সিস্টেম-ব্যাপী bashrc: Debian/Ubuntu-তে bash এমনভাবে কম্পাইল করা যে নিজেই `/etc/bash.bashrc` পড়ে। Fedora/RHEL-এ এটা নেই; সেখানে `~/.bashrc` নিজে `/etc/bashrc` source করে।
2. `~/.bashrc`

Non-interactive shell (স্ক্রিপ্ট):

- কোনো স্টার্টআপ ফাইল পড়ে না। শুধু `BASH_ENV` variable সেট থাকলে ওই ফাইলটা পড়ে।
- (ব্যতিক্রম: আগের অংশে বলা ssh-এর ক্ষেত্রে `~/.bashrc`।)

ডায়াগ্রাম



সবচেয়ে গুরুত্বপূর্ণ কথা: login shell নিজে থেকে `~/.bashrc` পড়ে না

এটাই সবচেয়ে বেশি বিভ্রান্তি তৈরি করে। নিয়ম অনুযায়ী login shell `.bashrc` পড়ে **না**। তবুও ssh-এ ঢুকলে তোমার alias কাজ করে, কারণ distro-গুলো ডিফল্ট ফাইলে নিচের কোড আগে থেকেই বসিয়ে রাখে।

Ubuntu/Debian `~/.profile` (সংক্ষেপে):

```
# if running bash
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
```

```
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi
if [ -d "$HOME/.local/bin" ] ; then
    PATH="$HOME/.local/bin:$PATH"
fi
```

Fedora/RHEL `~/.bash_profile`:

```
# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi
```

> ⚠️ **সতর্কতা: সবচেয়ে প্রচলিত ফাঁদ।** Ubuntu-তে কোনো টিউটোরিয়াল পড়ে তুমি একটা নতুন `~/.bash_profile` বানালে। এখন login shell `~/.bash_profile` পেয়ে গেছে, তাই `~/.profile` আর পড়ছে না। ফলে `.profile`-এর ভেতরের `.bashrc` লোড বন্ধ হয়ে গেল, আর ssh-এ তোমার সব alias আর রঙিন prompt উধাও! সমাধান: হয় `~/.bash_profile` মুছে দাও, নয়তো তার ভেতরে এই লাইন রাখো:
>
> ```
> [ -f ~/.profile ] && . ~/.profile
> ```

Ubuntu-র `~/.bashrc`-এর শুরুর পাহারা

```
# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;
esac
```

এর মানে: শেল interactive না হলে (`$-`-এ `i` নেই) এখানেই ফাইল পড়া থামাও। এটা না থাকলে `scp` বা `rsync` ভেঙে যেতে পারে। কারণ ওগুলো ssh দিয়ে non-interactive শেল চালায়, আর `.bashrc`-এর কোনো `echo "Welcome!"` তাদের ডেটা স্ট্রিমে ঢুকে প্রোটোকল নষ্ট করে দেয়।

> 💡 **টিপস:** তাই `.bashrc`-এ কিছু প্রিন্ট করতে চাইলে সেটা এই পাহারার **নিচে** লেখো। আর `export PATH=...` জাতীয় জিনিস যদি non-interactive ssh কমান্ডেও লাগে, তাহলে সেটা পাহারার **উপরে** রাখো।

কোন জিনিস কোন ফাইলে রাখবে — ব্যবহারিক নিয়ম

কী রাখতে চাও	কোথায় রাখবে	কারণ
`export PATH=...`, `export EDITOR=...`	`~/.profile` (বা `~/.bash_profile`), অথবা সরলতার জন্য `~/.bashrc`	একবার লাগে, child-রা পায়
`alias`, shell function	`~/.bashrc`	child-রা পায় না, প্রতিটা শেলে লাগে
`PS1`, `shopt`, `set -o`, completion	`~/.bashrc`	interactive শেলেরই বৈশিষ্ট্য
logout-এ স্ক্রিন পরিষ্কার, টেম্প মুছে ফেলা	`~/.bash_logout`	
সব ইউজারের জন্য environment	`/etc/profile.d/নাম.sh`	`/etc/profile` সরাসরি এডিট করলে আপডেটে ঝামেলা হয়
সব ইউজারের জন্য alias	`/etc/bash.bashrc` (Debian/Ubuntu) বা `/etc/bashrc` (Fedora/RHEL)	
GUI অ্যাপসহ সব জায়গার জন্য একটা variable	`~/.config/environment.d/*.conf` (systemd user session) বা `/etc/environment`	GUI-থেকে-চালানো অ্যাপ তোমার `.bashrc` পড়ে না

> 📝 **মনে রাখো:** ডেস্কটপ থেকে আইকনে ক্লিক করে চালানো অ্যাপ (যেমন VS Code) কোনো শেলের ভেতর দিয়ে চালু হয় না, তাই তোমার `~/.bashrc`-এর `export` সেটা পায় না। GNOME-এ লগইনের সময় সাধারণত `~/.profile` পড়া হয় (distro ও display manager ভেদে আলাদা), আর systemd ভিত্তিক সেশনে `~/.config/environment.d/` নির্ভরযোগ্যভাবে কাজ করে। ওই ফাইলে `NAME=value` লেখো, শেল সিনট্যাক্স নয়।

নিজে পরীক্ষা করে দেখো কোন ফাইল পড়া হলো

প্রতিটা ফাইলের শেষে একটা ট্রেস লাইন যোগ করো (পরীক্ষা শেষে মুছে দিও):

```
$ echo 'echo "[.profile পড়া হলো]"' >> ~/.profile
$ echo '[[ $- == *i* ]] && echo "[.bashrc পড়া হলো]"' >> ~/.bashrc

$ bash                    # non-login interactive
[.bashrc পড়া হলো]
$ exit
$ bash -l                 # login interactive
[.bashrc পড়া হলো]       # .profile-এর ভেতর থেকে source হয়েছে, তাই আগে
[.profile পড়া হলো]
$ exit
$ bash -c 'true'          # non-interactive: কিছুই না
$
```

`bash -l`-এ `.bashrc`-এর লাইন আগে এল কেন? কারণ `.profile`-এর ভেতরে `.bashrc` source হয় ফাইলের মাঝামাঝি, আর আমাদের ট্রেস লাইনটা আছে `.profile`-এর একদম শেষে।

আরেকটা কাজের উপায়: `bash -lixc exit 2>&1 | less` চালালে login shell চালু হওয়ার সময় যা যা execute হয় সবকিছু (`set -x` ট্রেস) দেখা যাবে।

`--norc`, `--noprofile`, `--rcfile`

অপশন	কাজ
`bash --norc`	`~/.bashrc` (ও সিস্টেম bashrc) পড়বে না। কনফিগে গোলমাল খুঁজতে কাজে লাগে
`bash --noprofile`	login shell হলেও `/etc/profile` আর `~/.bash_profile` … পড়বে না
`bash --rcfile FILE`	`~/.bashrc` -এর বদলে FILE পড়বে
`bash -l` / `bash --login`	login shell হিসেবে চালাও
`bash -i`	জোর করে interactive

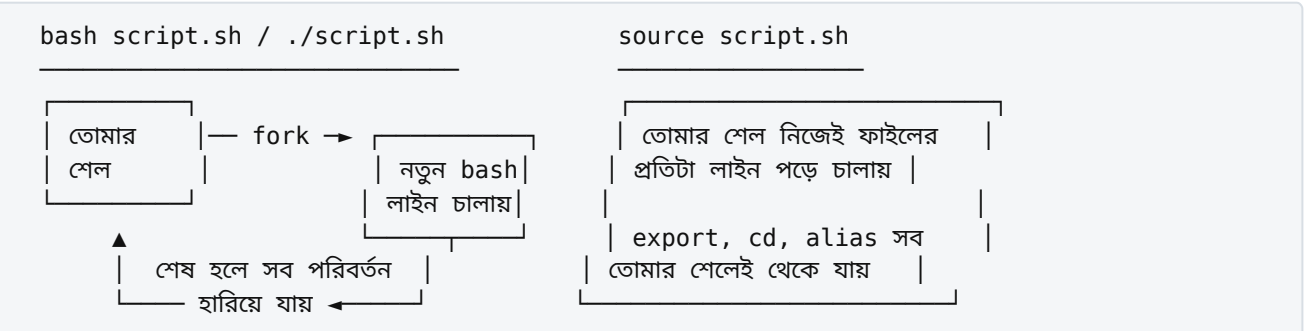
> 💡 **টিপস:** `.bashrc` -এ এমন ভুল করেছ যে টার্মিনাল খুললেই বন্ধ হয়ে যাচ্ছে? (যেমন `.bashrc` -এ `exit` লিখে ফেলেছ।) তাহলে tty-তে (Ctrl+Alt+F3) লগইন করো বা অন্য টার্মিনাল থেকে `bash --norc` চালাও, তারপর ফাইলটা ঠিক করো। Ubuntu-র মূল কপি `/etc/skel/.bashrc` -এ থাকে: `cp /etc/skel/.bashrc ~/` দিয়ে ডিফল্টে ফেরত যাওয়া যায়।

৪৪.৮ `source` (`.`) — পরিবর্তন সাথে সাথে চালু করা

`.bashrc` এডিট করার পর পরিবর্তন দেখতে নতুন টার্মিনাল খুলতে হয়। অথবা বর্তমান শেলেই ফাইলটা আবার পড়াও:

```
$ source ~/.bashrc
$ . ~/.bashrc          # একই জিনিস; '.' হলো POSIX নাম, 'source' bash-এর নাম
```

`source file` বনাম `bash file` বনাম `./file`:



একটা স্ক্রিপ্ট যদি `cd` করে বা variable সেট করে, আর তুমি চাও সেটা তোমার শেলে থেকে যাক (যেমন Python-এর `source venv/bin/activate`), তাহলে `source` করতে হবে।

৪৪.৯ alias — লম্বা কমান্ডের ডাকনাম

ধরো তোমার বন্ধুর নাম "মোহাম্মদ আব্দুর রহমান চৌধুরী", কিন্তু তুমি তাকে ডাকো "রাজু"। alias হলো কমান্ডের এমন একটা ডাকনাম।

alias তৈরি ও দেখা

```
$ alias ll='ls -alF'
$ ll
total 32
drwxr-x--- 5 rahim rahim 4096 Nov 12 10:20 ./
drwxr-xr-x 3 root  root  4096 Oct  1 09:00 ../
-rw-r--r-- 1 rahim rahim 3771 Oct  1 09:00 .bashrc
...

$ alias                 # সব alias দেখাও
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'

$ alias ll              # একটার মান দেখাও
alias ll='ls -alF'
```

উপরের তালিকাটা Ubuntu-র ডিফল্ট `~/.bashrc` থেকে আসে। Fedora/RHEL-এ `/etc/profile.d/colorls.sh` থেকে `ll='ls -l --color=auto'` আর root ইউজারের `/root/.bashrc`-এ `rm='rm -i'`, `cp='cp -i'`, `mv='mv -i'` থাকে।

Syntax-এর নিয়ম:

- `alias নাম='কমান্ড'` — `=`-এর দুই পাশে স্পেস দেবে না
- single quote (`'`) দাও, যাতে variable alias **ব্যবহারের সময়** expand হয়, alias **তৈরির সময়** না

```
$ alias here="echo $PWD"     # ভুল: এখনকার ডিরেক্টরি স্থির হয়ে গেল
$ alias here='echo $PWD'     # সঠিক: প্রতিবার চালানোর সময় দেখবে
```

alias স্থায়ী করা

শেলে লেখা alias টার্মিনাল বন্ধ করলে হারিয়ে যায়। স্থায়ী করতে `~/.bashrc`-এ লেখো। Ubuntu-র `.bashrc` আলাদা একটা ফাইল খোঁজে:

```
# Ubuntu ~/.bashrc-এ আগে থেকেই আছে:
if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi
```

তাই Ubuntu/Debian-এ নিজের alias `~/.bash_aliases`-এ রাখাই পরিষ্কার পদ্ধতি:

```
$ nano ~/.bash_aliases
$ source ~/.bashrc
```

কাজের কিছু alias

```
# ~/.bash_aliases

# নিরাপত্তা: overwrite/মোছার আগে জিজ্ঞেস করো
alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'

# চলাফেরা
alias ..='cd ..'
alias ...='cd ../..'

# তথ্য
alias df='df -h'
alias du1='du -h --max-depth=1'
alias free='free -h'
alias ports='ss -tulpn'
alias myip='curl -s ifconfig.me; echo'

# প্যাকেজ (Ubuntu/Debian)
alias update='sudo apt update && sudo apt upgrade'
# Fedora/RHEL-এর জন্য:
# alias update='sudo dnf upgrade --refresh'

# git
alias gs='git status'
alias gl='git log --oneline --graph --decorate -20'

# sudo-র পরেও alias যেন কাজ করে (শেষের স্পেসটা খেয়াল করো)
alias sudo='sudo '
```

শেষ লাইনটা একটা সূক্ষ্ম কৌশল। bash-এর নিয়ম: alias-এর মান যদি **স্পেস দিয়ে শেষ হয়**, তাহলে পরের শব্দটাও alias কি না দেখা হয়। নইলে `sudo ll` লিখলে "ll: command not found" আসে, কারণ `sudo` তোমার শেলের alias চেনে না।

alias এড়িয়ে আসল কমান্ড চালানো

```
$ \rm file.txt          # সামনে backslash: alias উপেক্ষা
$ command rm file.txt   # alias আর function দুটোই উপেক্ষা
$ /usr/bin/rm file.txt  # পুরো পাথ
$ 'rm' file.txt         # quote দিলেও alias expand হয় না
```

`unalias`

```
$ unalias ll            # একটা মুছে ফেলো
$ unalias -a            # বর্তমান শেলের সব alias মুছে ফেলো
```

এটা শুধু বর্তমান শেলে কাজ করে। স্থায়ীভাবে সরাতে `.bashrc` / `.bash_aliases` থেকে লাইনটা মুছে দাও।

alias-এর সীমাবদ্ধতা — কখন function লাগবে

alias হলো সাধারণ লেখা বদলে দেওয়া। এতে argument মাঝখানে বসানো যায় না:

```
$ alias mkcd='mkdir -p $1 && cd $1'    # কাজ করবে না! alias-এ $1 নেই
```

এ ক্ষেত্রে function বানাও (অধ্যায় ৪২):

```
# ~/.bashrc
mkcd() {
    mkdir -p -- "$1" && cd -- "$1"
}

# যেকোনো আর্কাইভ খোলা
extract() {
    case "$1" in
        *.tar.gz|*.tgz)   tar xzf "$1" ;;
        *.tar.xz)         tar xJf "$1" ;;
        *.tar.bz2)        tar xjf "$1" ;;
        *.zip)            unzip "$1" ;;
        *.gz)             gunzip "$1" ;;
        *) echo "চিনি না: $1" >&2; return 1 ;;
    esac
}
```

বৈশিষ্ট্য	alias	function
argument মাঝে বসানো	✘	✔ (`$1`, `$@`)
একাধিক লাইন, if/loop	কষ্টকর	✔
স্ক্রিপ্টে (non-interactive) কাজ করে	ডিফল্টে ✘	✔
child bash-এ পৌঁছায়	✘	`export -f` দিয়ে ✔
সহজ ডাকনাম	✔ সবচেয়ে সহজ	একটু বেশি লিখতে হয়

> 📝 **মনে রাখো:** স্ক্রিপ্টে alias ডিফল্টে expand হয় না (চালু করতে `shopt -s expand_aliases` লাগে)। স্ক্রিপ্টে alias-এর ওপর ভরসা না করে পূর্ণ কমান্ড বা function লেখো।

৪৪.১০ PS1 — নিজের মতো Prompt

তোমার টার্মিনালের `rahim@laptop:~/docs$` লেখাটাই prompt। এটা আসে `PS1` variable থেকে:

```
$ echo "$PS1"
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\
[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
```

দেখতে ভয়ের, কিন্তু টুকরো টুকরো করলে সহজ।

Prompt-এর বিশেষ escape কোড

কোড	মানে	উদাহরণ
`\u`	ইউজারনেম	`rahim`
`\h`	hostname-এর প্রথম অংশ (প্রথম `.` পর্যন্ত)	`laptop`
`\H`	পূর্ণ hostname	`laptop.home.lan`
`\w`	বর্তমান ডিরেক্টরি (হোমকে `~` দিয়ে)	`~/docs/notes`
`\W`	বর্তমান ডিরেক্টরির শুধু শেষ অংশ	`notes`
`\$`	সাধারণ ইউজারে `$`, root-এ `#`	`$`
`\d`	তারিখ	`Thu Nov 12`
`\t`	সময় ২৪-ঘণ্টা HH:MM:SS	`14:05:09`
`\T`	সময় ১২-ঘণ্টা HH:MM:SS	`02:05:09`
`\@`	১২-ঘণ্টা am/pm	`02:05 PM`
`\A`	২৪-ঘণ্টা HH:MM	`14:05`
`\D{format}`	`strftime` ফরম্যাটে সময়	`\D{%Y-%m-%d}` → `2026-11-12`
`\n`	নতুন লাইন	
`\s`	শেলের নাম	`bash`
`\v` / `\V`	bash version / পূর্ণ version	`5.2` / `5.2.21`
`\j`	ব্যাকগ্রাউন্ড job সংখ্যা (অধ্যায় ২৫)	`2`
`\!`	এই কমান্ডের history নম্বর	`501`
`\#`	এই সেশনের কমান্ড নম্বর	`12`
`\l`	টার্মিনাল ডিভাইসের নাম	`0` (pts/0 থেকে)
`\e` বা `\033`	ESC ক্যারেক্টার (রঙের কোড শুরু)	
`\[` ... `\]`	এর ভেতরের অংশ "অদৃশ্য", জায়গা নেয় না	রঙের কোডের চারপাশে **অবশ্যই**

ধাপে ধাপে prompt বানানো

```
$ PS1='\u@\h:\w\$ '
rahim@laptop:~/docs$

$ PS1='[\A] \W\$ '
[14:05] docs$

$ PS1='\u@\h \w\n\$ '      # দুই লাইনের prompt
rahim@laptop ~/docs/project/very/long/path
$
```

রঙ যোগ করা (ANSI escape code)

রঙের কোডের গঠন: `\e[` + সংখ্যা + `m`। একাধিক সংখ্যা `;` দিয়ে দেওয়া যায়।

কোড	মানে	কোড	মানে
`0`	সব রিসেট	`1`	মোটা (bold)
`4`	আন্ডারলাইন	`7`	উল্টো রঙ
`30`	কালো	`90`	উজ্জ্বল কালো (ধূসর)
`31`	লাল	`91`	উজ্জ্বল লাল
`32`	সবুজ	`92`	উজ্জ্বল সবুজ
`33`	হলুদ	`93`	উজ্জ্বল হলুদ
`34`	নীল	`94`	উজ্জ্বল নীল
`35`	বেগুনি (magenta)	`95`	উজ্জ্বল বেগুনি
`36`	সায়ান	`96`	উজ্জ্বল সায়ান
`37`	সাদা	`97`	উজ্জ্বল সাদা
`40`–`47`	পটভূমির রঙ	`38;5;N`	২৫৬ রঙের N নম্বর

```
PS1='\[\e[1;32m\]\u@\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '
#    └─ সবুজ bold ─┘ user@host └রিসেট┘  └─ নীল bold ─┘ dir └রিসেট┘
```

> ⚠️ **সতর্কতা: `\[` আর `\]` ভুলে যেও না।** bash prompt-এর দৈর্ঘ্য গুনে ঠিক করে লাইন কখন ভাঙবে। রঙের কোড স্ক্রিনে দেখা যায় না, কিন্তু `\[ \]` না দিলে bash এগুলোকেও অক্ষর হিসেবে গোনে। ফলে লম্বা কমান্ড লিখলে লেখা আগের লাইনের ওপর চেপে বসে, আর Ctrl+A/Ctrl+E-তে কার্সর ভুল জায়গায় যায়।

`tput` দিয়ে আরও পরিচ্ছন্ন রঙ

হাতে escape code না লিখে `tput` ব্যবহার করলে টার্মিনালের ধরন (`TERM`) অনুযায়ী ঠিক কোড পাওয়া যায়:

```
# ~/.bashrc
green=$(tput setaf 2); blue=$(tput setaf 4); bold=$(tput bold); reset=$(tput sgr0)
PS1='\[$bold$green\]\u@\h\[$reset\]:\[$bold$blue\]\w\[$reset\]\$ '
```

root হলে লাল prompt

```
if [ "$EUID" -eq 0 ]; then
    PS1='\[\e[1;31m\]\u@\h:\w#\[\e[0m\] '      # root: লাল, সতর্ক থাকো
else
    PS1='\[\e[1;32m\]\u@\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '
fi
```

আগের কমান্ড সফল কি না, prompt-এ দেখানো

```
# ~/.bashrc
__prompt() {
    local ec=$?                                # সবার আগে exit code ধরে রাখো
    local mark='\[\e[32m\]✔\[\e[0m\]'
    [ $ec -ne 0 ] && mark="\[\e[31m\]✘ $ec\[\e[0m\]"
    PS1="$mark \[\e[1;34m\]\w\[\e[0m\]\$ "
}
PROMPT_COMMAND=__prompt
```

```
✔ ~$ ls /nonexistent
ls: cannot access '/nonexistent': No such file or directory
✘ 2 ~$
```

`PROMPT_COMMAND`-এ যা থাকে, bash প্রতিবার prompt দেখানোর **ঠিক আগে** সেটা চালায়। (bash 5.1 থেকে `PROMPT_COMMAND` একটা array-ও হতে পারে।)

Git branch দেখানো

git-এর সাথে একটা স্ক্রিপ্ট আসে, যার `__git_ps1` function বর্তমান branch-এর নাম দেয়। Ubuntu/Debian-এ bash-completion চালু থাকলে এটা সাধারণত আপনা-আপনি লোড হয় (`/usr/lib/git-core/git-sh-prompt`)। Fedora/RHEL-এ থাকে `/usr/share/git-core/contrib/completion/git-prompt.sh`, সেটা source করতে হয়।

```
# ~/.bashrc (Fedora/RHEL-এ প্রথম লাইনটা লাগবে)
[ -f /usr/share/git-core/contrib/completion/git-prompt.sh ] && \
    . /usr/share/git-core/contrib/completion/git-prompt.sh
GIT_PS1_SHOWDIRTYSTATE=1          # অসংরক্ষিত পরিবর্তন থাকলে * দেখাবে
PS1='\u@\h:\w\[\e[33m\]$(__git_ps1 " (%s)")\[\e[0m\]\$ '
```

```
rahim@laptop:~/myproject (main *)$
```

খেয়াল করো, `$(__git_ps1 ...)` **single quote-এর ভেতরে** আছে, তাই এটা প্রতিবার prompt দেখানোর সময় চলে। double quote দিলে `.bashrc` পড়ার সময় একবারই চলত।

অন্য prompt variable-গুলো

Variable	কখন দেখায়	ডিফল্ট
`PS1`	প্রধান prompt	distro-ভেদে
`PS2`	কমান্ড অসম্পূর্ণ হলে পরের লাইনে (যেমন quote খোলা রেখে Enter)	`>`
`PS3`	`select` লুপের মেনু প্রশ্ন	`#?`
`PS4`	`set -x` ডিবাগ আউটপুটের প্রতিটা লাইনের শুরু	`+`

```
$ PS4='+ ${BASH_SOURCE}:${LINENO}: '   # set -x-এ ফাইল ও লাইন নম্বরও দেখাবে
```

> 💡 **টিপস:** একটা নিরীহ স্ক্রিপ্ট বা অনলাইন জেনারেটর দিয়ে (যেমন "bash prompt generator" লিখে খুঁজলে পাবে) PS1 বানিয়ে পরীক্ষা করতে পারো। আগে বর্তমান শেলে `PS1='...'` লিখে দেখো, পছন্দ হলে তবেই `.bashrc`-এ রাখো।

৪৪.১১ zsh ও oh-my-zsh পরিচিতি

zsh কী

zsh (Z shell) bash-এর মতোই আরেকটা শেল। বেশিরভাগ bash কমান্ড আর স্ক্রিপ্টিং সিনট্যাক্স এতে চলে, সাথে আছে বাড়তি সুবিধা। macOS Catalina (10.15) থেকে macOS-এর ডিফল্ট শেল zsh, আর Kali Linux-ও ২০২০.৪ থেকে zsh ব্যবহার করে। Ubuntu/Debian/Fedora/RHEL-এর ডিফল্ট এখনো bash।

zsh-এর বাড়তি সুবিধা:

- আরও স্মার্ট Tab completion: মেনু থেকে তীর দিয়ে বাছাই, অপশনের ব্যাখ্যা দেখা
- `**` দিয়ে recursive glob ডিফল্টে চালু (`ls **/*.txt`); bash-এ এর জন্য `shopt -s globstar` লাগে
- glob qualifier: `ls *(.)` শুধু সাধারণ ফাইল, `ls *(/)` শুধু ডিরেক্টরি, `ls *(m-1)` শেষ ১ দিনে বদলানো ফাইল
- ডিরেক্টরির নাম লিখে Enter দিলেই `cd` (`setopt autocd`)
- বানান ভুল ঠিক করার প্রস্তাব (`setopt correct`)
- একাধিক টার্মিনালে history ভাগ করা (`setopt share_history`)
- ডানপাশের prompt (`RPROMPT`)

> ⚠️ **সতর্কতা: কিছু পার্থক্য আছে।** zsh-এ array-এর index ১ থেকে শুরু (bash-এ ০ থেকে)। আর quote ছাড়া `$var` স্পেসে ভেঙে আলাদা শব্দ হয় না (word splitting হয় না)। তাই সার্ভারে চালানোর স্ক্রিপ্ট সবসময় `#!/bin/bash` দিয়ে লেখো। zsh শুধু নিজের interactive ব্যবহারের জন্য রাখো।

zsh ইনস্টল ও ডিফল্ট করা

```
# Ubuntu/Debian
$ sudo apt install zsh
# Fedora/RHEL
$ sudo dnf install zsh

$ zsh --version
zsh 5.9 (x86_64-ubuntu-linux-gnu)

$ chsh -s "$(command -v zsh)"      # ডিফল্ট লগইন শেল বদলাও (পাসওয়ার্ড চাইবে)
$ grep "^$USER:" /etc/passwd
rahim:x:1000:1000:Rahim,,,:/home/rahim:/usr/bin/zsh
```

পরিবর্তন কার্যকর হবে পরের লগইনে (logout/login করো)। `chsh` শুধু `/etc/shells`-এ তালিকাভুক্ত শেল গ্রহণ করে (`cat /etc/shells`)। Fedora/RHEL-এ `chsh` না থাকলে `sudo dnf install util-linux-user` দিয়ে ইনস্টল করো।

প্রথমবার zsh চালালে কোনো `~/.zshrc` না থাকলে `zsh-newuser-install` একটা মেনু দেখায়। `q` চাপলে কিছু না করে বের হবে, `0` চাপলে একটা খালি `~/.zshrc` বানাবে।

zsh-এর স্টার্টআপ ফাইল

zsh-এর নিয়ম bash-এর চেয়ে পরিষ্কার:

```
সব zsh (স্ক্রিপ্ট সহ)        login হলে       interactive হলে     login হলে      logout-এ
──────────────────────      ──────────      ─────────────────    ──────────     ─────────
/etc/zsh/zshenv      ──►  /etc/zsh/   ──►  /etc/zsh/zshrc  ──►  /etc/zsh/  ──►  ...
~/.zshenv                 zprofile          ~/.zshrc             zlogin         ~/.zlogout
                          ~/.zprofile                            ~/.zlogin
```

(সিস্টেম ফাইলগুলো Debian/Ubuntu-তে `/etc/zsh/` ডিরেক্টরিতে, Fedora/RHEL-এ সরাসরি `/etc/`-এ, যেমন `/etc/zshrc`।)

ফাইল	কখন	কী রাখবে
`~/.zshenv`	**সব** zsh-এ, এমনকি স্ক্রিপ্টেও	শুধু অপরিহার্য environment (কোনো আউটপুট নয়)
`~/.zprofile`	login shell	PATH ইত্যাদি (bash-এর `.profile`-এর মতো)
`~/.zshrc`	interactive shell	alias, prompt, plugin, `setopt`
`~/.zlogin`	login shell, `.zshrc`-এর পরে	বিরল

ফাইল	কখন	কী রাখবে
`~/.zlogout`	login shell থেকে বের হলে	পরিষ্কার করার কাজ

> 📝 **মনে রাখো:** zsh তোমার `~/.bashrc` বা `~/.bash_aliases` পড়ে না। bash থেকে zsh-এ গেলে alias আর PATH `~/.zshrc`-এ কপি করো, অথবা `~/.zshrc`-এ `source ~/.bash_aliases` লেখো (সাধারণ alias হলে দুই শেলেই চলবে)।

oh-my-zsh

oh-my-zsh হলো zsh-এর একটা জনপ্রিয় কমিউনিটি ফ্রেমওয়ার্ক (zsh-এর অংশ নয়, আলাদা প্রজেক্ট)। এতে শত শত plugin (alias আর completion-এর সংগ্রহ) আর অনেক theme (prompt-এর ডিজাইন) আছে।

ইনস্টল (আগে zsh, git আর curl লাগবে):

```
$ sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

> ⚠️ **সতর্কতা:** `curl ... | sh` বা `sh -c "$(curl ...)"` মানে ইন্টারনেটের একটা স্ক্রিপ্ট না দেখেই চালানো। বিশ্বস্ত উৎস হলেও ভালো অভ্যাস হলো আগে ডাউনলোড করে পড়ে নেওয়া:
>
> ```
> $ curl -fsSL -o install.sh
> https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh
> $ less install.sh
> $ sh install.sh
> ```

ইনস্টলার যা করে: `~/.oh-my-zsh`-এ রিপো clone করে, তোমার পুরনো `~/.zshrc` থাকলে সেটাকে `~/.zshrc.pre-oh-my-zsh` নামে ব্যাকআপ রাখে, নতুন একটা `~/.zshrc` বসায়, আর ডিফল্ট শেল zsh করার প্রস্তাব দেয়।

`~/.zshrc`-এর মূল অংশ:

```
export ZSH="$HOME/.oh-my-zsh"
ZSH_THEME="robbyrussell"        # ডিফল্ট থিম; "agnoster", "random" ইত্যাদি
plugins=(git sudo docker)       # স্পেস দিয়ে আলাদা, কমা নয়!
source $ZSH/oh-my-zsh.sh

# নিজের alias এই source লাইনের নিচে লেখো
alias ll='ls -alF'
```

কাজ	কমান্ড
থিম বদলানো	`~/.zshrc`-এ `ZSH_THEME="..."` বদলে `source ~/.zshrc`
থিমের তালিকা	`ls ~/.oh-my-zsh/themes/`

কাজ	কমান্ড
plugin-এর তালিকা	`ls ~/.oh-my-zsh/plugins/`
আপডেট	`omz update`
আনইনস্টল	`uninstall_oh_my_zsh`

জনপ্রিয় কিছু plugin:

Plugin	কী করে
`git`	`gst` (git status), `gco` (git checkout), `gp` (git push) ইত্যাদি অনেক alias
`sudo`	দুবার Esc চাপলে আগের/বর্তমান কমান্ডের সামনে `sudo` বসে যায়
`z`	যেসব ডিরেক্টরিতে বেশি যাও সেগুলোতে লাফ: `z proj`
`extract`	`x file.tar.gz` — যেকোনো আর্কাইভ খোলে
`zsh-autosuggestions`	history থেকে ধূসর রঙে পরামর্শ দেখায়, → চাপলে গ্রহণ (আলাদা ডাউনলোড লাগে)
`zsh-syntax-highlighting`	টাইপ করার সময় ঠিক কমান্ড সবুজ, ভুল লাল (আলাদা ডাউনলোড লাগে)

শেষের দুটো oh-my-zsh-এর সাথে আসে না, `custom/plugins`-এ clone করতে হয়:

```
$ git clone https://github.com/zsh-users/zsh-autosuggestions \
    ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-autosuggestions
$ git clone https://github.com/zsh-users/zsh-syntax-highlighting \
    ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting
# তারপর ~/.zshrc-এ: plugins=(git sudo zsh-autosuggestions zsh-syntax-highlighting)
```

(Ubuntu/Debian-এ `apt install zsh-autosuggestions zsh-syntax-highlighting` দিয়েও পাওয়া যায়। তখন oh-my-zsh plugin হিসেবে না দিয়ে `/usr/share/...` থেকে source করতে হয়।)

> 💡 **টিপস:** বেশি plugin দিলে টার্মিনাল খুলতে দেরি হয়। কতক্ষণ লাগছে মাপতে: `time zsh -i -c exit`। একই রকম জনপ্রিয় বিকল্প হলো **Starship** নামের prompt, যা bash, zsh, fish সব শেলে কাজ করে। আর **fish** শেল নিজেই ডিফল্টে autosuggestion আর রঙ দেয়, তবে এর স্ক্রিপ্টিং সিনট্যাক্স POSIX-এর মতো নয়।

৪৪.১২ dotfiles — তোমার কনফিগারেশনের সম্পদ

dotfiles কী

লিনাক্সে যে ফাইলের নাম `.` দিয়ে শুরু হয়, সেটা লুকানো থাকে (`ls`-এ দেখা যায় না, `ls -a`-তে দেখা যায়)। প্রোগ্রামগুলো তাদের কনফিগারেশন হোম ডিরেক্টরির এমন লুকানো ফাইলে রাখে। তাই এদের বলে **dotfiles**।

```
$ ls -a ~
.  ..  .bash_history  .bash_logout  .bashrc  .cache  .config  .gitconfig
.local  .profile  .ssh  .vimrc  Desktop  Documents  Downloads
```

ফাইল/ডিরেক্টরি	কিসের কনফিগ
`~/.bashrc`, `~/.profile`, `~/.bash_aliases`	bash
`~/.zshrc`	zsh
`~/.vimrc`	Vim (অধ্যায় ১৪)
`~/.nanorc`	nano (অধ্যায় ১৩)
`~/.gitconfig`	git
`~/.tmux.conf`	tmux (অধ্যায় ২৫)
`~/.inputrc`	readline: bash-এর কী-বাইন্ডিং ও completion আচরণ
`~/.ssh/config`	SSH হোস্টের শর্টকাট (অধ্যায় ৩৮)
`~/.config/`	XDG নিয়ম মেনে চলা আধুনিক অ্যাপের কনফিগ

নতুন ইউজারের dotfiles আসে কোথা থেকে? `/etc/skel/` থেকে। `useradd -m` বা `adduser` নতুন ইউজার বানানোর সময় এই ফোল্ডারের সবকিছু তার হোমে কপি করে দেয় (অধ্যায় ১৫)। তাই সব নতুন ইউজারকে একটা ডিফল্ট `.bashrc` দিতে চাইলে `/etc/skel/.bashrc` এডিট করো।

কেন dotfiles নিয়ে মাথা ঘামাবে

বছরের পর বছর তুমি নিজের মতো alias, prompt, vim সেটিং বানাবে। নতুন ল্যাপটপ বা নতুন সার্ভার পেলে সব আবার হাতে করা কষ্টকর। তাই অভিজ্ঞ লিনাক্স ইউজাররা dotfiles **Git রিপোজিটরিতে** রাখে, আর নতুন মেশিনে কয়েক মিনিটেই নিজের পরিচিত পরিবেশ ফিরিয়ে আনে। GitHub-এ "dotfiles" লিখে খুঁজলে হাজার হাজার মানুষের রিপো পাবে, যেখান থেকে শেখা যায়।

পদ্ধতি ১: ফোল্ডার + symlink (সবচেয়ে সহজ)

```
$ mkdir -p ~/dotfiles
$ mv ~/.bashrc ~/.bash_aliases ~/.vimrc ~/dotfiles/
$ ln -s ~/dotfiles/.bashrc       ~/.bashrc
$ ln -s ~/dotfiles/.bash_aliases ~/.bash_aliases
$ ln -s ~/dotfiles/.vimrc        ~/.vimrc
$ ls -l ~/.bashrc
lrwxrwxrwx 1 rahim rahim 27 Nov 12 11:00 /home/rahim/.bashrc ->
/home/rahim/dotfiles/.bashrc

$ cd ~/dotfiles && git init && git add . && git commit -m "my dotfiles"
```

symlink নিয়ে বিস্তারিত অধ্যায় ১১-এ। প্রোগ্রাম `~/.bashrc` খোলে, আর symlink তাকে `~/dotfiles/.bashrc`-এ নিয়ে যায়।

পদ্ধতি ২: GNU Stow (symlink স্বয়ংক্রিয়)

```
$ sudo apt install stow        # Fedora/RHEL: sudo dnf install stow (RHEL-এ EPEL লাগতে পারে)

# গঠন: প্রতিটা "প্যাকেজ" একটা ফোল্ডার, ভেতরে হোমের মতো একই কাঠামো
~/dotfiles/
├── bash/
│   ├── .bashrc
│   └── .bash_aliases
└── vim/
    └── .vimrc

$ cd ~/dotfiles
$ stow bash vim                # ~/.bashrc → dotfiles/bash/.bashrc ইত্যাদি symlink বানাবে
$ stow -D vim                  # vim-এর symlink সরাবে
```

`stow` ডিফল্টে বর্তমান ফোল্ডারের **parent** (এখানে `~`)-কে টার্গেট ধরে। হোমে একই নামের আসল ফাইল আগে থেকে থাকলে stow ঝুঁকি না নিয়ে conflict দেখিয়ে থেমে যায়। তখন আসল ফাইলটা সরিয়ে আবার চালাও।

পদ্ধতি ৩: bare git রিপো (কোনো symlink নেই)

```
$ git init --bare $HOME/.dotfiles
$ alias dot='git --git-dir=$HOME/.dotfiles --work-tree=$HOME'
$ dot config --local status.showUntrackedFiles no   # হোমের সব ফাইল untracked দেখাবে না
$ dot add ~/.bashrc ~/.vimrc
$ dot commit -m "add bashrc and vimrc"
$ dot remote add origin git@github.com:rahim/dotfiles.git
$ dot push -u origin main
```

এখানে তোমার হোম ডিরেক্টরিই git-এর work tree, তাই ফাইলগুলো নিজের জায়গাতেই থাকে। `dot` alias-টা `.bashrc`-এ রেখে দিও।

> ⚠️ **সতর্কতা: পাবলিক রিপোতে গোপন জিনিস রাখবে না।** `~/.ssh/id_*` (private key), `~/.aws/credentials`, `.netrc`, API token বা পাসওয়ার্ড যুক্ত `export` লাইন — এগুলো কখনো commit করবে না। গোপন জিনিস আলাদা একটা ফাইলে রাখো (যেমন `~/.bashrc.local`), সেটা git-এর বাইরে রাখো, আর `.bashrc` থেকে লোড করো:
>
> ```
> [ -f ~/.bashrc.local ] && . ~/.bashrc.local
> ```

৪৪.১৩ একটা পূর্ণাঙ্গ, পরিচ্ছন্ন `~/.bashrc` (নমুনা)

Ubuntu-র ডিফল্ট ফাইলের শেষে তুমি এ রকম একটা অংশ যোগ করতে পারো:

```
# ===== আমার কাস্টমাইজেশন =====

# ---- PATH (একবারই যোগ হবে) ----
for d in "$HOME/bin" "$HOME/.local/bin"; do
    [ -d "$d" ] || continue
    case ":$PATH:" in *":$d:"*) ;; *) PATH="$d:$PATH" ;; esac
done
export PATH

# ---- ডিফল্ট প্রোগ্রাম ----
export EDITOR=nano
export VISUAL=nano
export PAGER=less

# ---- History (বিস্তারিত অধ্যায় ৬) ----
HISTSIZE=10000
HISTFILESIZE=20000
HISTCONTROL=ignoreboth           # স্পেস দিয়ে শুরু ও পরপর একই কমান্ড বাদ
HISTTIMEFORMAT='%F %T '          # history-তে তারিখ-সময়
shopt -s histappend              # সব টার্মিনালের history যোগ হবে, overwrite নয়

# ---- শেলের আচরণ ----
shopt -s checkwinsize            # উইন্ডোর আকার বদলালে LINES/COLUMNS আপডেট
shopt -s globstar                # ** দিয়ে recursive glob
shopt -s cdspell                 # cd-তে ছোটখাটো বানান ভুল ঠিক করে
shopt -s autocd                  # ডিরেক্টরির নাম লিখলেই cd

# ---- Prompt ----
__prompt() {
    local ec=$?
    local c_ok='\[\e[32m\]' c_err='\[\e[31m\]' c_dir='\[\e[1;34m\]' r='\[\e[0m\]'
    local mark="${c_ok}✔${r}"
    [ $ec -ne 0 ] && mark="${c_err}✘${ec}${r}"
    PS1="${mark} \u@\h:${c_dir}\w${r}\n\$ "
}
PROMPT_COMMAND=__prompt

# ---- alias ও function ----
[ -f ~/.bash_aliases ] && . ~/.bash_aliases
mkcd() { mkdir -p -- "$1" && cd -- "$1"; }

# ---- গোপন বা মেশিন-নির্দিষ্ট সেটিং ----
[ -f ~/.bashrc.local ] && . ~/.bashrc.local
```

> 💡 **টিপস:** `.bashrc` এডিট করার আগে একটা কপি রাখো: `cp ~/.bashrc ~/.bashrc.bak`। আর এডিটের পর **পুরনো টার্মিনাল বন্ধ না করেই** নতুন একটা টার্মিনাল খুলে পরীক্ষা করো। কিছু ভাঙলে পুরনো টার্মিনাল থেকে ঠিক করতে পারবে।

`~/.inputrc` দিয়েও কিছু কাজের সেটিং করা যায় (এটা bash-এর লাইন-এডিটিং লাইব্রেরি readline পড়ে):

```
# ~/.inputrc
$include /etc/inputrc
set completion-ignore-case on        # Tab completion-এ ছোট-বড় হাতের অক্ষর এক ধরা
```

```
set show-all-if-ambiguous on          # একবার Tab-এই সব বিকল্প দেখাও
"\e[A": history-search-backward       # ↑: যা টাইপ করেছ তা দিয়ে শুরু হওয়া history
"\e[B": history-search-forward        # ↓
```

পরিবর্তন নতুন শেলে কার্যকর হবে, বা বর্তমান শেলে `bind -f ~/.inputrc` দিয়ে।

৪৪.১৪ এই অধ্যায়ের সাথে সম্পর্কিত কীবোর্ড শর্টকাট

শর্টকাট	কাজ	শেল
`Ctrl + Alt + E`	বর্তমান লাইনের alias, history ও variable expand করে দেখাও (চালায় না)	bash (readline `shell-expand-line`)
`Esc` তারপর `.` / `Alt + .`	আগের কমান্ডের শেষ argument বসাও (যেমন `mkdir dir` তারপর `cd Alt+.`)	bash, zsh
`Ctrl + X`, তারপর `Ctrl + E`	বর্তমান কমান্ড `$VISUAL` / `$EDITOR`-এ খুলে এডিট করো, সেভ করে বের হলে চলবে	bash
`Tab` (দুবার)	সম্ভাব্য সব completion দেখাও, যেমন `$HO` লিখে Tab দিলে `$HOME`	bash
`Esc` `Esc`	কমান্ডের সামনে `sudo` বসাও	zsh + oh-my-zsh `sudo` plugin
`→` বা `End`	ধূসর autosuggestion গ্রহণ	zsh-autosuggestions
`Ctrl + L`	স্ক্রিন পরিষ্কার (prompt নতুন করে আঁকে, PS1 পরীক্ষায় কাজে লাগে)	bash, zsh
`Ctrl + D`	খালি লাইনে চাপলে শেল থেকে বের হও (login shell হলে `~/.bash_logout` চলবে)	bash, zsh
`Ctrl + Alt + T`	নতুন টার্মিনাল খোলো (non-login interactive shell)	Ubuntu GNOME (অধ্যায় ৪)
`Ctrl + Alt + F3` … `F6`	টেক্সট কনসোল, যেখানে **login shell** চলে; `.bashrc` ভাঙলে উদ্ধার পেতে কাজে লাগে	বেশিরভাগ distro

`bind -P | grep -v 'not bound'` চালালে বর্তমানে bound সব readline শর্টকাট দেখা যাবে। শর্টকাটের পূর্ণ তালিকা আছে অধ্যায় ৬ আর পরিশিষ্ট ক-এ।

হাতে-কলমে অনুশীলন

ল্যাব ১: shell variable বনাম environment variable

```
$ city="Dhaka"
$ bash -c 'echo "child দেখছে: [$city]"'
child দেখছে: []
```

```
$ export city
$ bash -c 'echo "child দেখছে: [$city]"'
child দেখছে: [Dhaka]

# child বদলালে parent-এ প্রভাব পড়ে কি?
$ bash -c 'city="Chittagong"; echo "child-এ: $city"'
child-এ: Chittagong
$ echo "parent-এ: $city"
parent-এ: Dhaka

# শুধু একটা কমান্ডের জন্য variable
$ city=Sylhet bash -c 'echo $city'
Sylhet
$ echo $city
Dhaka
```

প্রশ্ন: শেষ দুটো আউটপুট আলাদা কেন? (উত্তর: `city=Sylhet command` লিখলে মানটা শুধু ওই কমান্ডের environment-এ যায়।)

ল্যাব ২: PATH-এ নিজের কমান্ড

```
$ mkdir -p ~/bin
$ cat > ~/bin/hello <<'EOF'
#!/bin/bash
echo "হ্যালো, $USER! আজ $(date +%A)"
EOF
$ chmod +x ~/bin/hello
$ hello
bash: hello: command not found       # PATH-এ ~/bin না থাকলে

$ export PATH="$HOME/bin:$PATH"
$ hello
হ্যালো, rahim! আজ Thursday
$ type hello
hello is /home/rahim/bin/hello
```

এবার PATH-এর ক্রমের প্রভাব দেখো:

```
$ cat > ~/bin/ls <<'EOF'
#!/bin/bash
echo "এটা আমার নকল ls!"
EOF
$ chmod +x ~/bin/ls
$ hash -r
$ \ls                        # alias এড়িয়ে
এটা আমার নকল ls!
$ type -a ls
ls is aliased to `ls --color=auto'
ls is /home/rahim/bin/ls
ls is /usr/bin/ls
ls is /bin/ls
$ rm ~/bin/ls && hash -r    # পরিষ্কার করো!
```

এই কারণেই PATH-এর শুরুতে কোন ডিরেক্টরি আছে, আর সেখানে কে লিখতে পারে, সেটা নিরাপত্তার জন্য গুরুত্বপূর্ণ (অধ্যায় ৪৫)।

ল্যাব ৩: login বনাম non-login

```
$ echo $0; shopt login_shell
bash
login_shell     off
$ bash -l
$ echo $0; shopt login_shell
bash
login_shell     on
$ exit
$ su - $USER                  # পাসওয়ার্ড দাও
$ echo $0
-bash
$ exit
```

(`bash -l`-এ `$0`-এর সামনে `-` থাকে না, কারণ argv[0] বদলানো হয়নি, শুধু `--login` অপশন দেওয়া হয়েছে। তাই `shopt login_shell` সবচেয়ে নির্ভরযোগ্য পরীক্ষা।)

ল্যাব ৪: স্থায়ী alias ও function

```
$ cp ~/.bashrc ~/.bashrc.bak
$ cat >> ~/.bash_aliases <<'EOF'
alias ..='cd ..'
alias ll='ls -alFh'
alias ports='ss -tulpn'
EOF
$ grep -q bash_aliases ~/.bashrc || echo '[ -f ~/.bash_aliases ] && . ~/.bash_aliases'
>> ~/.bashrc
$ echo 'mkcd() { mkdir -p -- "$1" && cd -- "$1"; }' >> ~/.bashrc
$ source ~/.bashrc
$ mkcd /tmp/test/a/b && pwd
/tmp/test/a/b
$ ..
$ pwd
/tmp/test/a
```

(Fedora/RHEL-এর `.bashrc` `~/.bash_aliases` খোঁজে না, তাই দ্বিতীয় লাইনের `grep` সেই লাইনটা যোগ করে দেবে।)

ল্যাব ৫: নিজের prompt

1. বর্তমান prompt সংরক্ষণ করো: `OLD_PS1="$PS1"`
2. চেষ্টা করো: `PS1='\[\e[1;35m\][\A]\[\e[0m\] \[\e[1;32m\]\u\[\e[0m\]:\W\$ '`
3. একটা লম্বা কমান্ড লিখে দেখো, লাইন ঠিকমতো ভাঙছে কি না

4. এবার ইচ্ছা করে `\[ \]` বাদ দিয়ে দেখো কী গোলমাল হয়: `PS1='\e[1;32m\u\e[0m\$ '`, তারপর অনেক লম্বা কিছু টাইপ করে Ctrl+A চাপো
5. ফেরত যাও: `PS1="$OLD_PS1"`
6. পছন্দেরটা `~/.bashrc`-এর শেষে লেখো

ল্যাব ৬: dotfiles রিপো

```
$ mkdir -p ~/dotfiles/bash
$ cp ~/.bashrc ~/.bash_aliases ~/dotfiles/bash/
$ cd ~/dotfiles && git init -q && git add . && git commit -qm "initial dotfiles"
$ git log --oneline
a1b2c3d initial dotfiles
```

চ্যালেঞ্জ: `stow` ইনস্টল করে আসল ফাইল দুটো সরাও (`mv ~/.bashrc ~/.bashrc.orig` ইত্যাদি), তারপর `cd ~/dotfiles && stow bash` চালাও। `ls -l ~/.bashrc` দিয়ে দেখো symlink হয়েছে কি না।

ল্যাব ৭ (ঐচ্ছিক): zsh ঘুরে দেখা

```
$ sudo apt install zsh        # বা: sudo dnf install zsh
$ zsh                         # প্রথমবার মেনু এলে 'q'
% echo $0
zsh
% ls /etc/**/*.conf | head    # recursive glob
% exit
```

শুধু ঘুরে দেখার জন্য ডিফল্ট শেল বদলানোর দরকার নেই।

সারসংক্ষেপ

কমান্ড / জিনিস	কাজ
`NAME=value`	shell variable (শুধু এই শেলে)
`export NAME=value`	environment variable (child process-ও পায়)
`export -n NAME`	export চিহ্ন তুলে দাও
`export -p` / `declare -x`	সব exported variable
`env`	environment দেখাও
`env -i cmd` / `env -u NAME cmd`	খালি / একটা variable বাদ দিয়ে environment-এ কমান্ড চালাও
`NAME=value cmd`	শুধু ওই কমান্ডের জন্য variable
`printenv NAME`	একটা environment variable-এর মান
`set`	সব variable ও function; `set -o` অপশন

কমান্ড / জিনিস	কাজ
`unset NAME` / `unset -f fn`	variable / function মুছে ফেলো
`declare -p NAME`, `readonly NAME`	variable-এর ধরন দেখাও / স্থির করো
`echo $PATH`	কমান্ড খোঁজার ডিরেক্টরি তালিকা
`export PATH="$HOME/bin:$PATH"`	PATH-এর শুরুতে যোগ
`type -a`, `command -v`, `which`	কমান্ড কোথা থেকে আসছে
`hash`, `hash -r`	মনে রাখা পাথ দেখাও / ভুলে যাও
`locale`, `localectl`	ভাষা/অঞ্চলের সেটিং
`shopt login_shell`, `echo $-`	login / interactive কি না
`/etc/profile` → `~/.bash_profile` বা `~/.bash_login` বা `~/.profile`	login shell (প্রথম পাওয়া একটা)
`/etc/bash.bashrc` → `~/.bashrc`	interactive non-login shell (Debian/Ubuntu)
`~/.bashrc` → `/etc/bashrc`	interactive non-login shell (Fedora/RHEL)
`~/.bash_logout`	login shell থেকে বের হলে
`source file` / `. file`	বর্তমান শেলেই ফাইল চালাও
`bash --norc`, `bash -l`	rc ছাড়া / login হিসেবে শেল
`alias name='cmd'`, `unalias name`	alias বানাও / মুছে ফেলো
`\cmd`, `command cmd`	alias এড়িয়ে আসল কমান্ড
`PS1`, `PS2`, `PS4`, `PROMPT_COMMAND`	prompt কাস্টমাইজেশন
`\u \h \w \W \$ \t \A \n \[ \]`	PS1-এর প্রধান escape কোড
`chsh -s /usr/bin/zsh`	ডিফল্ট লগইন শেল বদলাও
`~/.zshenv ~/.zprofile ~/.zshrc ~/.zlogin`	zsh-এর স্টার্টআপ ফাইল
`ZSH_THEME`, `plugins=(...)`, `omz update`	oh-my-zsh
`/etc/skel/`	নতুন ইউজারের ডিফল্ট dotfiles
`stow pkg`, bare git repo	dotfiles ব্যবস্থাপনা
`Ctrl+Alt+E`, `Ctrl+X Ctrl+E`, `Alt+.`	expand / এডিটরে কমান্ড / আগের argument

নিজেকে যাচাই করো

1. `x=5` লিখে তারপর `bash -c 'echo $x'` চালালে কী আসবে, আর কেন? এটাকে কাজ করাতে কী করতে হবে?
2. একটা স্ক্রিপ্টের ভেতরে `export PATH="$HOME/tools:$PATH"` লিখে `./setup.sh` চালালে তোমার টার্মিনালের PATH কেন বদলায় না? কীভাবে চালালে বদলাবে?

3. `env`, `printenv` আর `set`-এর আউটপুটের মূল পার্থক্য কী?
4. `PATH=/home/rahim/bin:/usr/bin` আর দুই ডিরেক্টরিতেই `backup` নামের প্রোগ্রাম থাকলে কোনটা চলবে? কীভাবে নিশ্চিত হবে?
5. Ubuntu-তে তুমি একটা নতুন `~/.bash_profile` বানালে, আর এখন ssh-এ লগইন করলে তোমার alias কাজ করছে না। কেন? সমাধান কী?
6. ডেস্কটপে GNOME Terminal খোলা, `ssh user@host`, আর `bash script.sh` — এই তিনটা কোন কোন ধরনের শেল (login/non-login, interactive/non-interactive)?
7. PS1-এ রঙের কোডের চারপাশে `\[` আর `\]` কেন দিতে হয়?
8. alias-এ argument মাঝখানে বসানো যায় না কেন, আর তার বিকল্প কী?
9. zsh-এ যেয়ে তোমার `~/.bashrc`-এর alias কাজ করছে না কেন?
10. dotfiles রিপো GitHub-এ পাবলিক রাখলে কোন সাবধানতা মানতে হবে?

▼ **উত্তর দেখো**

1. খালি লাইন আসবে। `x` একটা shell variable, export করা হয়নি, তাই child bash-এর environment-এ এটা যায় না। `export x` (বা `export x=5`) করলে আসবে `5`। অথবা শুধু ওই কমান্ডের জন্য `x=5 bash -c 'echo $x'`।

2. `./setup.sh` একটা আলাদা child process-এ চলে। সে নিজের environment কপিতে PATH বদলায়, আর শেষ হলে সেই কপি হারিয়ে যায়। environment কখনো child থেকে parent-এ যায় না। `source ./setup.sh` বা `. ./setup.sh` দিয়ে চালালে বর্তমান শেলেই চলবে, আর PATH বদলাবে।

3. `env` আর `printenv` বাইরের প্রোগ্রাম, তাই শুধু exported (environment) variable দেখায়। `printenv NAME` একটা নির্দিষ্ট variable-এর মান দেয়, আর না থাকলে exit code 1 দেয়। `set` শেলের builtin, তাই exported আর non-exported সব variable, সাথে সব shell function-ও দেখায়।

4. `/home/rahim/bin/backup` চলবে, কারণ PATH বাম থেকে ডানে খোঁজা হয় আর প্রথম যেটা পাওয়া যায় সেটাই চলে। `type -a backup` বা `command -v backup` দিয়ে নিশ্চিত হও। সম্প্রতি PATH বা ফাইল বদলালে `hash -r` করে নাও।

5. login shell `~/.bash_profile`, `~/.bash_login`, `~/.profile`-এর মধ্যে প্রথম যেটা পায় শুধু সেটাই পড়ে। নতুন `~/.bash_profile` থাকায় `~/.profile` আর পড়া হচ্ছে না, ফলে তার ভেতরের `. ~/.bashrc`-ও চলছে না। সমাধান: `~/.bash_profile` মুছে ফেলো, অথবা তার ভেতরে `[ -f ~/.profile ] && . ~/.profile` (বা সরাসরি `. ~/.bashrc`) যোগ করো।

6. GNOME Terminal: non-login, interactive (ডিফল্ট সেটিংয়ে)। `ssh user@host` (কমান্ড ছাড়া): login, interactive। `bash script.sh`: non-login, non-interactive (কোনো স্টার্টআপ ফাইল পড়ে না, শুধু `$BASH_ENV`)।

7. রঙের escape কোড স্ক্রিনে দেখা যায় না, কিন্তু bash prompt-এর দৈর্ঘ্য গুনে ঠিক করে কার্সর কোথায় আর লাইন কখন ভাঙবে। `\[ \]` দিয়ে জানিয়ে দিতে হয় যে এই অংশ জায়গা নেয় না। না দিলে লম্বা কমান্ডে লাইন ভাঙা আর কার্সর চলাচল এলোমেলো হয়ে যায়।

8. alias শুধু প্রথম শব্দটাকে একটা লেখা দিয়ে বদলে দেয়। এতে `$1` জাতীয় argument-এর ধারণা নেই, পরের শব্দগুলো শুধু শেষে জুড়ে যায়। মাঝখানে argument বসাতে বা যুক্তি (if/loop) লাগলে shell function ব্যবহার করো, যেমন `mkcd() { mkdir -p -- "$1" && cd -- "$1"; }`।

9. zsh নিজের ফাইল পড়ে (`~/.zshenv`, `~/.zprofile`, `~/.zshrc`, `~/.zlogin`), bash-এর `~/.bashrc` বা `~/.bash_aliases` পড়ে না। alias `~/.zshrc`-এ কপি করো, অথবা `~/.zshrc`-এ `source ~/.bash_aliases` লেখো।

10. গোপন তথ্য কখনো commit করবে না: SSH private key, ক্লাউড credential, API token, পাসওয়ার্ড যুক্ত `export` লাইন। এগুলো আলাদা ফাইলে রাখো (যেমন `~/.bashrc.local`), সেটা `.gitignore`-এ দাও বা রিপোর বাইরে রাখো, আর `[ -f ~/.bashrc.local ] && . ~/.bashrc.local` দিয়ে লোড করো। push করার আগে `git diff --cached` দেখে নাও।

অধ্যায় ৪৫: লিনাক্স নিরাপত্তা ও হার্ডেনিং

> "নিরাপত্তা কোনো প্রোডাক্ট না, এটা একটা প্রক্রিয়া।" — Bruce Schneier

এতদিন আমরা শিখেছি লিনাক্স দিয়ে কীভাবে কাজ করতে হয়। এই অধ্যায়ে শিখব কীভাবে সেই সিস্টেমকে **সুরক্ষিত** রাখতে হয়। একটা নতুন সার্ভার ইন্টারনেটে চালু করলে কয়েক মিনিটের মধ্যেই সারা দুনিয়ার bot তার SSH পোর্টে পাসওয়ার্ড অনুমান করা শুরু করে দেয়। তাই **hardening** (হার্ডেনিং — সিস্টেমকে "শক্ত" করা, মানে আক্রমণের সুযোগ যতটা পারা যায় কমিয়ে আনা) প্রতিটা লিনাক্স অ্যাডমিনের রোজকার কাজ।

এই অধ্যায়ে যা শিখবে

- নিরাপত্তার স্তর (defense in depth) ধারণা ও threat model
- সিস্টেম আপডেট রাখা এবং `unattended-upgrades` / `dnf-automatic` দিয়ে স্বয়ংক্রিয় সিকিউরিটি আপডেট
- শক্তিশালী পাসওয়ার্ড নীতি, পাসওয়ার্ডের মেয়াদ (`chage`) এবং PAM কীভাবে কাজ করে
- SSH সার্ভার হার্ডেনিং — key-based login, root login বন্ধ, ইত্যাদি
- `fail2ban` দিয়ে brute-force আক্রমণ ঠেকানো
- অপ্রয়োজনীয় সার্ভিস বন্ধ করা এবং খোলা পোর্ট অডিট (`ss`, `lsof`, `nmap`)
- DAC বনাম MAC — দুই ধরনের অ্যাক্সেস কন্ট্রোল
- SELinux: mode, `getenforce`, `setenforce`, context, `restorecon`, boolean, `audit2why`
- AppArmor: `aa-status`, profile, enforce/complain mode
- `auditd` দিয়ে কে কী করল তার হিসাব রাখা
- AIDE দিয়ে ফাইল ইন্টেগ্রিটি যাচাই, `rkhunter` দিয়ে rootkit খোঁজা
- `sha256sum` ও GPG দিয়ে ডাউনলোড করা ফাইল যাচাই
- Lynis দিয়ে পুরো সিস্টেমের নিরাপত্তা অডিট

৪৫.১ নিরাপত্তার মূল ধারণা

CIA ট্রায়াড

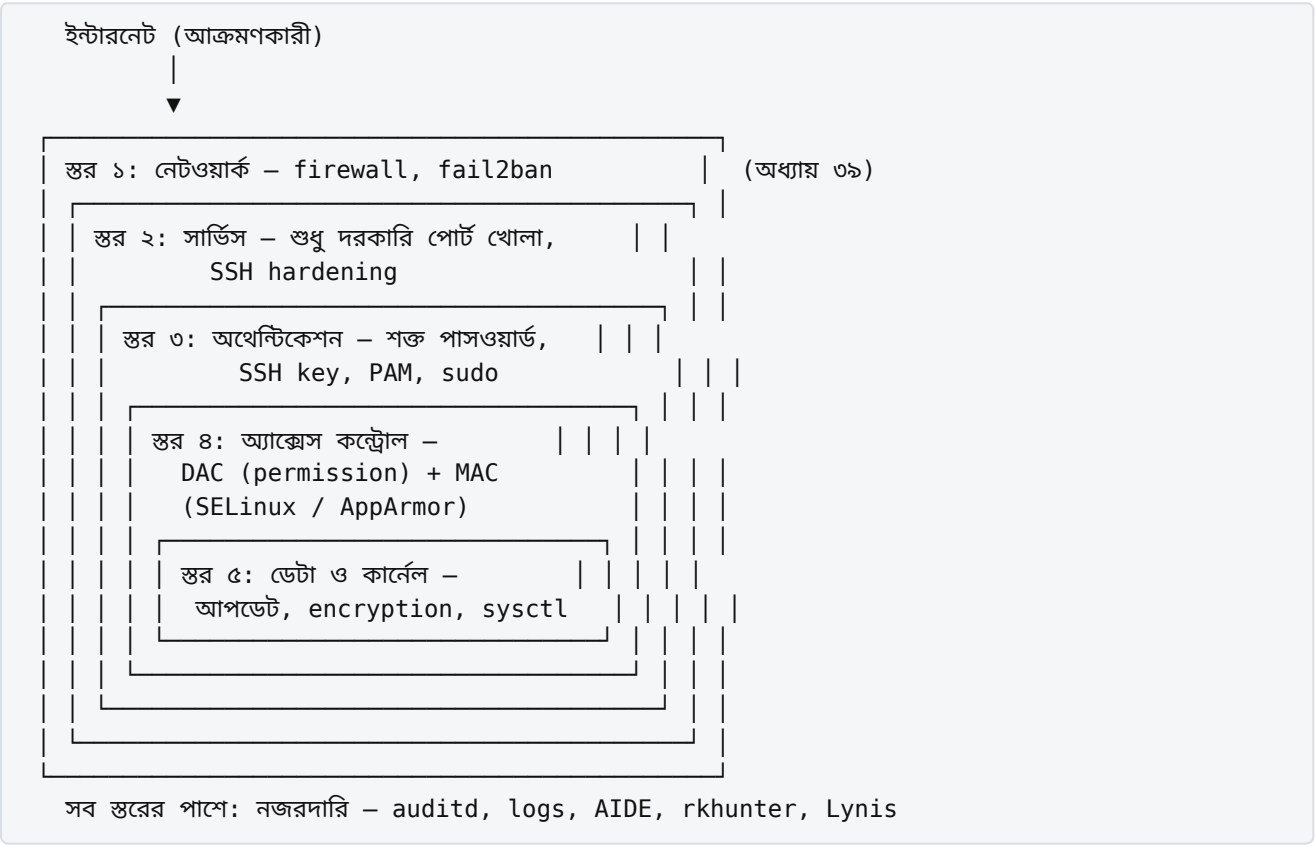
নিরাপত্তার তিনটা মূল লক্ষ্যকে বলা হয় **CIA triad**:

লক্ষ্য	মানে	উদাহরণ
Confidentiality (গোপনীয়তা)	যার দেখার অধিকার নেই, সে যেন দেখতে না পারে	`/etc/shadow` শুধু root পড়তে পারে

লক্ষ্য	মানে	উদাহরণ
Integrity (অখণ্ডতা)	ডেটা যেন অনুমতি ছাড়া বদলে না যায়	AIDE দিয়ে `/bin/ls` বদলেছে কি না দেখা
Availability (প্রাপ্যতা)	সার্ভিস যেন দরকারের সময় চালু থাকে	DDoS ঠেকানো, ব্যাকআপ রাখা

Defense in Depth — নিরাপত্তার স্তর

একটা পুরোনো দুর্গের কথা ভাবো। শুধু একটা দেয়াল থাকে না — বাইরে পরিখা (moat), তারপর উঁচু দেয়াল, তারপর ফটকে প্রহরী, ভেতরে আরেকটা দেয়াল, আর রাজার ঘরে তালা। শত্রু একটা স্তর ভাঙলেও পরের স্তরে আটকে যায়। লিনাক্স নিরাপত্তাও ঠিক এভাবে স্তরে স্তরে সাজানো:



> 📝 **মনে রাখো:** কোনো একটা টুল একা তোমাকে নিরাপদ রাখবে না। firewall আছে বলে SSH-এ দুর্বল পাসওয়ার্ড রাখা চলবে না, আবার SELinux আছে বলে আপডেট বাদ দেওয়া চলবে না।

Threat Model ও Attack Surface

- **Threat model** — "আমি কাকে থেকে, কী রক্ষা করছি?" একটা ব্যক্তিগত ল্যাপটপ আর একটা ব্যাংকের সার্ভারের threat model এক না।
- **Attack surface** (আক্রমণের পৃষ্ঠতল) — আক্রমণকারী যতগুলো দরজা দিয়ে ঢোকার চেষ্টা করতে পারে: খোলা পোর্ট, চালু সার্ভিস, ইউজার অ্যাকাউন্ট, ইনস্টল করা সফটওয়্যার। হার্ডেনিং-এর মূল কাজ হলো এই পৃষ্ঠতল **ছোট করা**।

- **Principle of least privilege** (ন্যূনতম অধিকারের নীতি) — প্রত্যেক ইউজার ও প্রোগ্রাম শুধু ততটুকু ক্ষমতা পাবে যতটুকু তার কাজের জন্য একান্ত দরকার। (sudo-র বিস্তারিত অধ্যায় ১৮-এ।)

৪৫.২ সিস্টেম আপডেট — প্রথম এবং সবচেয়ে গুরুত্বপূর্ণ ধাপ

বাস্তবে বেশিরভাগ সফল আক্রমণ হয় এমন দুর্বলতা (vulnerability) দিয়ে যার patch আগেই বের হয়ে গিয়েছিল, কিন্তু অ্যাডমিন আপডেট করেননি। প্রতিটা দুর্বলতার একটা আইডি থাকে, যাকে বলে **CVE** (Common Vulnerabilities and Exposures), যেমন `CVE-2024-3094` (xz backdoor)।

ম্যানুয়াল আপডেট

প্যাকেজ ম্যানেজারের বিস্তারিত অধ্যায় ৩০-এ আছে। এখানে নিরাপত্তার দিকটা দেখি:

```
# Debian / Ubuntu
$ sudo apt update && sudo apt upgrade

# Fedora / RHEL
$ sudo dnf upgrade --refresh

# RHEL/Fedora: শুধু সিকিউরিটি আপডেট দেখা ও নেওয়া
$ dnf updateinfo list --security
$ sudo dnf upgrade --security
```

কার্নেল বা `glibc`-এর মতো জিনিস আপডেট হলে রিবুট লাগতে পারে। কীভাবে বুঝবে?

```
# Debian / Ubuntu: এই ফাইলটা থাকলে রিবুট দরকার
$ cat /var/run/reboot-required
*** System restart required ***
$ cat /var/run/reboot-required.pkgs
linux-image-6.8.0-48-generic

# RHEL / Fedora (dnf-utils / dnf-plugins-core প্যাকেজ)
$ sudo needs-restarting -r
Core libraries or services have been updated since boot-up:
  * kernel
Reboot is required to fully utilize these updates.
```

> 💡 **টিপস:** Ubuntu-তে `needrestart` প্যাকেজ আপডেটের পর জানিয়ে দেয় কোন কোন সার্ভিস পুরোনো লাইব্রেরি নিয়ে চলছে এবং রিস্টার্ট দরকার।

unattended-upgrades (Debian / Ubuntu)

সার্ভারে প্রতিদিন নিজে লগইন করে আপডেট করা বাস্তবসম্মত না। `unattended-upgrades` প্যাকেজ প্রতিদিন নিজে থেকে সিকিউরিটি আপডেট ইনস্টল করে দেয়। Ubuntu-তে এটা সাধারণত আগে থেকেই ইনস্টল ও চালু

থাকে; Debian-এ নিজে ইনস্টল করতে হয়।

ভেতরে কীভাবে কাজ করে:

```
systemd timer: apt-daily.timer ──────► apt-daily.service
  (দিনে ২ বার, এলোমেলো সময়ে)          └─ package list আপডেট + ডাউনলোড
                                            (20auto-upgrades-এর সেটিং দেখে)

systemd timer: apt-daily-upgrade.timer ─► apt-daily-upgrade.service
  (সকাল ~৬টার দিকে)                       └─ unattended-upgrade চালায়
                                              └─ 50unattended-upgrades-এ
                                                 অনুমোদিত origin থেকে আপডেট
                                              └─ লগ: /var/log/unattended-upgrades/
```

ইনস্টল ও চালু করা:

```
$ sudo apt install unattended-upgrades apt-listchanges
$ sudo dpkg-reconfigure -plow unattended-upgrades
```

দ্বিতীয় কমান্ডটা একটা ডায়ালগ দেখায়, "Yes" দিলে `/etc/apt/apt.conf.d/20auto-upgrades` ফাইল তৈরি হয়:

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

এখানে `"1"` মানে "প্রতি ১ দিনে একবার"। `"0"` দিলে বন্ধ।

কোন উৎস (origin) থেকে আপডেট নেওয়া হবে, সেটা ঠিক হয় `/etc/apt/apt.conf.d/50unattended-upgrades` ফাইলে:

```
Unattended-Upgrade::Allowed-Origins {
        "${distro_id}:${distro_codename}";
        "${distro_id}:${distro_codename}-security";
        "${distro_id}ESMApps:${distro_codename}-apps-security";
        "${distro_id}ESM:${distro_codename}-infra-security";
//      "${distro_id}:${distro_codename}-updates";
};

// দরকার হলে নির্দিষ্ট প্যাকেজ বাদ দাও
Unattended-Upgrade::Package-Blacklist {
//      "linux-";
};

Unattended-Upgrade::Remove-Unused-Kernel-Packages "true";
Unattended-Upgrade::Remove-Unused-Dependencies "true";

// রিবুট লাগলে নিজে থেকে রিবুট (সার্ভারে সাবধানে!)
Unattended-Upgrade::Automatic-Reboot "false";
Unattended-Upgrade::Automatic-Reboot-Time "03:00";
```

```
// ইমেইলে রিপোর্ট (mail সিস্টেম কনফিগার থাকলে)
//Unattended-Upgrade::Mail "admin@example.com";
```

> 📝 **মনে রাখো:** `//` দিয়ে শুরু লাইন কমেন্ট। ডিফল্টভাবে শুধু `-security` চালু থাকে — এটাই নিরাপদ পছন্দ, কারণ সাধারণ ফিচার আপডেট মাঝে মাঝে জিনিস ভাঙতে পারে।

টেস্ট করা (আসলে কিছু ইনস্টল না করে দেখা কী হতো):

```
$ sudo unattended-upgrade --dry-run --debug
...
Allowed origins are: o=Ubuntu,a=noble, o=Ubuntu,a=noble-security, ...
Packages that will be upgraded: libssl3t64 openssl
...

$ systemctl list-timers 'apt-daily*'
NEXT                         LEFT     LAST                         PASSED  UNIT
ACTIVATES
Wed 2026-11-12 18:21:04 UTC 5h left  Wed 2026-11-12 06:40:11 UTC 6h ago  apt-
daily.timer         apt-daily.service
Thu 2026-11-13 06:12:33 UTC 17h left Wed 2026-11-12 06:05:02 UTC 7h ago  apt-daily-
upgrade.timer apt-daily-upgrade.service

$ sudo tail /var/log/unattended-upgrades/unattended-upgrades.log
```

dnf-automatic (Fedora / RHEL)

RHEL/Fedora-তে একই কাজ করে `dnf-automatic`:

```
$ sudo dnf install dnf-automatic
$ sudo vi /etc/dnf/automatic.conf
```

```
[commands]
# default = সব আপডেট, security = শুধু সিকিউরিটি
upgrade_type = security
download_updates = yes
apply_updates = yes
```

```
$ sudo systemctl enable --now dnf-automatic.timer
$ systemctl list-timers '*automatic*'
```

> ⚠️ **সতর্কতা:** Fedora 41 থেকে `dnf5` এসেছে, সেখানে প্যাকেজ ও টাইমারের নাম একটু ভিন্ন হতে পারে (`dnf5-plugin-automatic`)। নিশ্চিত হতে `systemctl list-timers '*automatic*'` দিয়ে আসল নাম দেখে নাও।

৪৫.৩ শক্তিশালী পাসওয়ার্ড ও PAM

ভালো পাসওয়ার্ড কেমন?

পাসওয়ার্ডের শক্তি মূলত তার **দৈর্ঘ্য** আর **অনিশ্চয়তা** (entropy)-র ওপর নির্ভর করে। `P@ssw0rd1` দেখতে জটিল মনে হলেও অভিধান-ভিত্তিক আক্রমণে মুহূর্তে ভেঙে যায়। অন্যদিকে `nodi-tara-cha-boi-megh` এর মতো চার-পাঁচটা এলোমেলো শব্দের **passphrase** মনে রাখা সহজ, কিন্তু ভাঙা অনেক কঠিন।

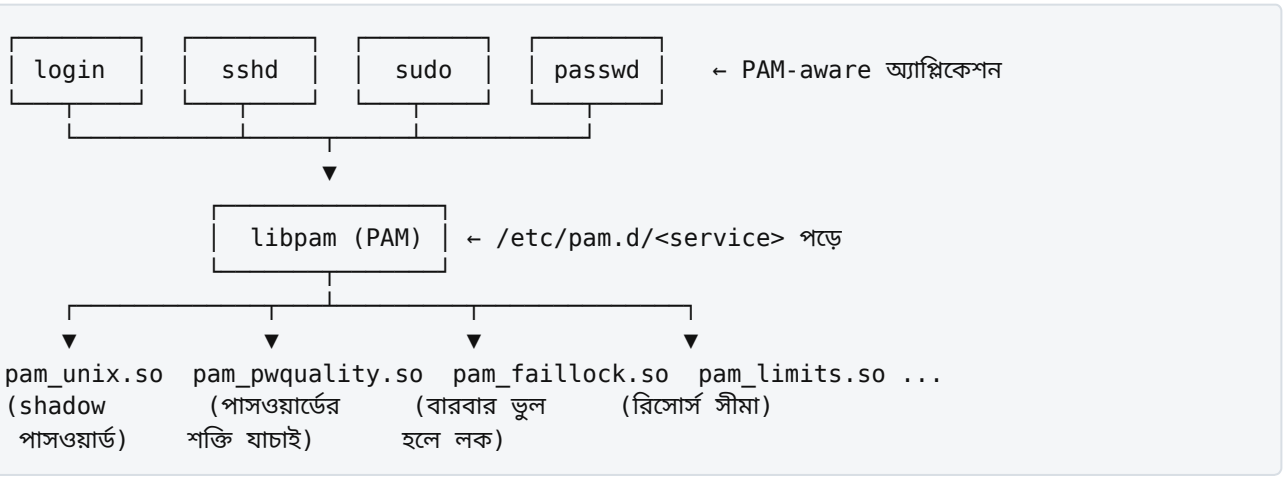
> 💡 **টিপস:** একটা password manager (KeePassXC, Bitwarden) ব্যবহার করো আর সার্ভারে যতটা সম্ভব পাসওয়ার্ডের বদলে SSH key দিয়ে লগইন করো (৪৫.৪ অংশ)।

পাসওয়ার্ড কোথায় থাকে, `/etc/shadow`-এর গঠন কী — এসব অধ্যায় ১৫-এ বিস্তারিত আছে। এখানে নীতি (policy) চাপানোর দিকটা দেখব।

PAM কী?

PAM (Pluggable Authentication Modules) হলো লিনাক্সের অথেন্টিকেশনের "সুইচবোর্ড"। `login`, `sshd`, `sudo`, `su`, `passwd` — এরা কেউ নিজে পাসওয়ার্ড যাচাই করে না। তারা PAM-কে জিজ্ঞেস করে, "এই ইউজারকে ঢুকতে দেব?" আর PAM কনফিগ ফাইল দেখে ঠিক করে কোন কোন মডিউল (প্লাগইন) দিয়ে যাচাই হবে।

উপমা: একটা অফিস ভবনের প্রবেশদ্বারের কথা ভাবো। প্রতিটা দরজা (প্রোগ্রাম) নিজের তালা বানায় না — সবাই একই নিরাপত্তা ডেস্ক (PAM) ব্যবহার করে। ডেস্কের নিয়মের খাতা (`/etc/pam.d/`) বদলালেই সব দরজার নিয়ম বদলে যায়, কোনো দরজা নতুন করে বানাতে হয় না।



PAM কনফিগ ফাইলের গঠন

প্রতিটা সার্ভিসের জন্য `/etc/pam.d/`-তে একটা ফাইল থাকে। প্রতিটা লাইনের গঠন:

```
<type>    <control>    <module>    [arguments]
```

চার ধরনের type (management group):

Type	কাজ
`auth`	তুমি কে, তা প্রমাণ করা (পাসওয়ার্ড, OTP)
`account`	অ্যাকাউন্ট ব্যবহারযোগ্য কি না (মেয়াদ শেষ? লক?)
`password`	পাসওয়ার্ড বদলানোর নিয়ম
`session`	লগইনের আগে-পরে কাজ (হোম ডিরেক্টরি মাউন্ট, লগ, limits)

Control flag:

Flag	মানে
`required`	পাস করতেই হবে; ফেল করলেও বাকি মডিউল চলে, শেষে ফল "ব্যর্থ"
`requisite`	পাস করতেই হবে; ফেল করলে সঙ্গে সঙ্গে থেমে যায়
`sufficient`	পাস করলে (এবং আগে কোনো required ফেল না করলে) সঙ্গে সঙ্গে সফল; ফেল করলে উপেক্ষা
`optional`	ফল সাধারণত গুরুত্ব পায় না (একমাত্র মডিউল হলে ছাড়া)
`include` / `substack`	অন্য একটা ফাইলের নিয়ম এখানে টেনে আনা

Ubuntu-তে উদাহরণ:

```
$ cat /etc/pam.d/sshd | grep -v '^#' | grep -v '^$'
@include common-auth
account    required     pam_nologin.so
@include common-account
session [success=ok ignore=ignore module_unknown=ignore default=bad] pam_selinux.so
close
session    required     pam_loginuid.so
...
@include common-password
```

> ⚠️ **সতর্কতা:** PAM ফাইলে একটা ভুল লাইন দিলে তুমি নিজেই সিস্টেম থেকে লকড আউট হয়ে যেতে পারো — sudo-ও কাজ করবে না! PAM ফাইল এডিট করার সময় **সবসময় একটা root শেল আলাদা খোলা রাখো**, আর অন্য টার্মিনালে নতুন লগইন টেস্ট করার পরেই পুরোনোটা বন্ধ করো। RHEL/Fedora-তে PAM ফাইল হাতে না বদলে `authselect` টুল ব্যবহার করা উচিত; Debian/Ubuntu-তে `pam-auth-update`।

pam_pwquality — পাসওয়ার্ডের শক্তি চাপানো

```
# Debian / Ubuntu (RHEL/Fedora-তে আগে থেকেই থাকে)
$ sudo apt install libpam-pwquality
```

কনফিগ ফাইল `/etc/security/pwquality.conf` (বা `/etc/security/pwquality.conf.d/*.conf`):

```
minlen = 14          # ন্যূনতম দৈর্ঘ্য
minclass = 3         # ছোট হাতের, বড় হাতের, সংখ্যা, চিহ্ন — অন্তত ৩ ধরনের
maxrepeat = 3        # একই অক্ষর পরপর ৩ বারের বেশি না
dictcheck = 1        # অভিধানের শব্দ নিষেধ
usercheck = 1        # ইউজারনেম থাকা চলবে না
difok = 5            # পুরোনো পাসওয়ার্ড থেকে অন্তত ৫টা অক্ষর আলাদা
enforce_for_root     # root নিজেও এই নিয়ম মানবে
```

`dcredit`, `ucredit`, `lcredit`, `ocredit` অপশনে ঋণাত্মক মান দিলে (যেমন `dcredit = -1`) মানে "অন্তত ১টা সংখ্যা থাকতেই হবে"।

```
$ passwd
Current password:
New password: abc123
BAD PASSWORD: The password is shorter than 14 characters
```

pam_faillock — বারবার ভুল পাসওয়ার্ড দিলে লক

কনফিগ: `/etc/security/faillock.conf`

```
deny = 5             # ৫ বার ভুল হলে লক
unlock_time = 900    # ৯০০ সেকেন্ড (১৫ মিনিট) পরে নিজে খুলবে
fail_interval = 900  # এই সময়ের মধ্যে ভুলগুলো গোনা হবে
# even_deny_root     # root-কেও লক করবে (সাবধান)
```

- RHEL 9 / Fedora: `sudo authselect enable-feature with-faillock`
- Ubuntu/Debian-এ ডিফল্টভাবে চালু থাকে না; `common-auth`-এ মডিউল যোগ করতে হয় (সাবধানে, ম্যান পেজ `man pam_faillock` দেখে)।

```
$ sudo faillock --user rahim
rahim:
When                Type  Source                                           Valid
2026-11-12 10:21:33 RHOST 203.0.113.50                                        V
2026-11-12 10:21:40 RHOST 203.0.113.50                                        V

$ sudo faillock --user rahim --reset      # লক খুলে দেওয়া
```

পাসওয়ার্ডের মেয়াদ — `chage`

```
$ sudo chage -l rahim
Last password change                                    : Nov 01, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
```

অপশন	কাজ
`-l`	মেয়াদের তথ্য দেখানো
`-M 90`	সর্বোচ্চ ৯০ দিন পর পাসওয়ার্ড বদলাতে হবে
`-m 1`	অন্তত ১ দিন পর আবার বদলানো যাবে
`-W 7`	মেয়াদ শেষের ৭ দিন আগে সতর্ক করবে
`-I 30`	মেয়াদ শেষের ৩০ দিন পর অ্যাকাউন্ট নিষ্ক্রিয়
`-E 2027-06-30`	ঐ তারিখে পুরো অ্যাকাউন্টের মেয়াদ শেষ
`-d 0`	পরের লগইনে পাসওয়ার্ড বদলাতে বাধ্য করা

নতুন ইউজারদের ডিফল্ট মান আসে `/etc/login.defs` থেকে (`PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`)।

> 💡 **টিপস:** আধুনিক নির্দেশিকা (যেমন NIST SP 800-63B) বলে, বারবার জোর করে পাসওয়ার্ড বদলানোর চেয়ে লম্বা, অনন্য পাসওয়ার্ড + MFA বেশি কার্যকর। তাই অন্ধভাবে `-M 30` বসিয়ে দিও না; তোমার প্রতিষ্ঠানের নীতি অনুযায়ী ঠিক করো।

অ্যাকাউন্ট পরিচ্ছন্নতা

```
# খালি পাসওয়ার্ডওয়ালা অ্যাকাউন্ট খোঁজা (ভয়ংকর!)
$ sudo awk -F: '($2 == "") {print $1}' /etc/shadow

# UID 0 (root ক্ষমতা) আছে কার কার — শুধু root থাকা উচিত
$ awk -F: '($3 == 0) {print $1}' /etc/passwd
root

# অব্যবহৃত অ্যাকাউন্ট লক করা
$ sudo usermod -L -e 1 olduser     # পাসওয়ার্ড লক + অ্যাকাউন্টের মেয়াদ শেষ
$ sudo passwd -S olduser
olduser L 2026-01-10 0 99999 7 -1
```

৪৫.৪ SSH হার্ডেনিং

SSH-এর মৌলিক ব্যবহার ও key তৈরি অধ্যায় ৩৮-এ আছে। এখানে শুধু **সার্ভার দিকটা শক্ত করা** দেখব।

কেন দরকার?

ইন্টারনেটে খোলা যেকোনো সার্ভারের লগ দেখো:

```
$ sudo journalctl -u ssh --since "1 hour ago" | grep -c "Failed password"
1874
```

এক ঘণ্টায় প্রায় দুই হাজার ব্যর্থ চেষ্টা! এগুলো স্বয়ংক্রিয় bot।

ধাপ ১: আগে key-based login চালু ও টেস্ট করো

```
# নিজের কম্পিউটার থেকে
$ ssh-keygen -t ed25519 -C "rahim@laptop"
$ ssh-copy-id rahim@server.example.com
$ ssh rahim@server.example.com      # পাসওয়ার্ড ছাড়াই ঢুকতে পারছ? তাহলে ঠিক আছে
```

> ⚠️ **সতর্কতা:** key দিয়ে লগইন **নিশ্চিত হওয়ার আগে** কখনো `PasswordAuthentication no` দিও না। আর কনফিগ বদলানোর সময় একটা SSH সেশন খোলা রেখে **দ্বিতীয় একটা টার্মিনাল থেকে** নতুন লগইন টেস্ট করো। এক সেশনে ভুল করলে পুরোনো সেশন দিয়ে ঠিক করতে পারবে।

ধাপ ২: drop-in কনফিগ ফাইল

মূল ফাইল `/etc/ssh/sshd_config` সরাসরি না বদলে আধুনিক distro-তে `/etc/ssh/sshd_config.d/` ডিরেক্টরিতে আলাদা `.conf` ফাইল রাখা ভালো। মূল ফাইলের একদম শুরুতে থাকে:

```
Include /etc/ssh/sshd_config.d/*.conf
```

> 📝 **মনে রাখো — sshd-এর "প্রথম মান জেতে" নিয়ম:** `sshd_config`-এ কোনো অপশন একাধিকবার থাকলে **প্রথমবার যে মান পাওয়া যায় সেটাই কার্যকর** (বেশিরভাগ অপশনের ক্ষেত্রে)। drop-in ফাইলগুলো বর্ণানুক্রমে পড়া হয়। Ubuntu cloud image-এ প্রায়ই `50-cloud-init.conf` ফাইলে `PasswordAuthentication yes` থাকে! তাই তোমার ফাইলের নাম `00-hardening.conf` রাখো যাতে সেটা আগে পড়া হয়।

```
$ sudo nano /etc/ssh/sshd_config.d/00-hardening.conf
```

```
# --- অথেন্টিকেশন ---
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no
AuthenticationMethods publickey

# --- শুধু নির্দিষ্ট ইউজার/গ্রুপ ---
AllowGroups sshusers

# --- brute-force কঠিন করা ---
MaxAuthTries 3
LoginGraceTime 30
MaxStartups 10:30:60
```

```
# --- অপ্রয়োজনীয় ফিচার বন্ধ ---
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no
PermitTunnel no

# --- নিষ্ক্রিয় সেশন কেটে দেওয়া (৩০০ সে × ২ = ~১০ মিনিট) ---
ClientAliveInterval 300
ClientAliveCountMax 2

# --- লগ ---
LogLevel VERBOSE
```

অপশনগুলোর ব্যাখ্যা:

অপশন	মান	কেন
`PermitRootLogin`	`no`	root নাম সবাই জানে — আক্রমণকারীর অর্ধেক কাজ সহজ। সাধারণ ইউজারে ঢুকে `sudo` ব্যবহার করো। (`prohibit-password` দিলে root শুধু key দিয়ে ঢুকতে পারবে)
`PasswordAuthentication`	`no`	পাসওয়ার্ড অনুমান করে ঢোকার পথ পুরো বন্ধ
`KbdInteractiveAuthentication`	`no`	keyboard-interactive (পুরোনো নাম `ChallengeResponseAuthentication`) পথেও পাসওয়ার্ড চাওয়া বন্ধ
`AllowUsers` / `AllowGroups`	ইউজার /গ্রুপ	তালিকার বাইরে কেউ ঢুকতে পারবে না
`MaxAuthTries`	`3`	এক সংযোগে সর্বোচ্চ ৩ বার চেষ্টা
`LoginGraceTime`	`30`	৩০ সেকেন্ডে লগইন শেষ না হলে সংযোগ কাটা
`MaxStartups`	`10:30:60`	অসম্পূর্ণ সংযোগ ১০টার বেশি হলে ৩০% হারে বাতিল, ৬০ হলে সব বাতিল
`X11Forwarding`	`no`	দরকার না হলে গ্রাফিক্স ফরওয়ার্ডিং বন্ধ
`ClientAliveInterval`	`300`	৫ মিনিট পরপর ক্লায়েন্ট বেঁচে আছে কি না দেখা

> ⚠️ **সতর্কতা:** `AllowGroups sshusers` দিলে আগে গ্রুপটা বানিয়ে নিজেকে যোগ করো: `sudo groupadd sshusers && sudo usermod -aG sshusers rahim` । নাহলে তুমি নিজেই আটকে যাবে।

ধাপ ৩: যাচাই করে রিলোড

```
# সিনট্যাক্স পরীক্ষা — কিছু না দেখালে মানে ঠিক আছে
$ sudo sshd -t

# আসলে কোন মান কার্যকর হচ্ছে (সব ফাইল মিলিয়ে)
$ sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthentication|maxauthtries'
permitrootlogin no
passwordauthentication no
```

```
maxauthtries 3

# রিলোড (চলমান সেশন কাটে না)
$ sudo systemctl reload ssh      # Debian / Ubuntu
$ sudo systemctl reload sshd     # RHEL / Fedora
```

> 📝 **মনে রাখো:** সার্ভিসের নাম Debian/Ubuntu-তে `ssh`, RHEL/Fedora-তে `sshd`।

পোর্ট বদলানো — দরকার আছে কি?

`Port 2222` দিলে লগের "শব্দ" (bot-এর চেষ্টা) অনেক কমে যায়, কিন্তু এটা আসল নিরাপত্তা না — এক `nmap` স্ক্যানেই নতুন পোর্ট ধরা পড়ে (একে বলে *security through obscurity*)। key-only লগইন আসল সুরক্ষা। তবু বদলাতে চাইলে তিনটা জায়গায় খেয়াল রাখো:

```
# ১. firewall-এ নতুন পোর্ট খোলো (পুরোনোটা বন্ধের আগে!)
$ sudo ufw allow 2222/tcp                                    # Ubuntu
$ sudo firewall-cmd --permanent --add-port=2222/tcp && sudo firewall-cmd --reload  #
RHEL

# ২. RHEL/Fedora-তে SELinux-কে জানাও যে sshd এই পোর্ট ব্যবহার করবে
$ sudo semanage port -a -t ssh_port_t -p tcp 2222

# ৩. Ubuntu 22.10+ (24.04 সহ) ডিফল্টভাবে ssh.socket (socket activation) ব্যবহার করে;
#    সেখানে পোর্ট বদলের পর:
$ sudo systemctl daemon-reload
$ sudo systemctl restart ssh.socket
```

SSH-এর জন্য দরকারি কীবোর্ড "শর্টকাট" — escape sequence

হার্ডেনিং-এর পর সেশন আটকে গেলে (যেমন নেটওয়ার্ক কেটে গেছে) এগুলো কাজে লাগে। এগুলো কাজ করে **নতুন লাইনের শুরুতে** (আগে `Enter` চাপো):

কী	কাজ
`Enter` তারপর `~.`	ঝুলে থাকা SSH সেশন জোর করে বন্ধ
`Enter` তারপর `~?`	সব escape sequence-এর তালিকা
`Enter` তারপর `~#`	চলমান forwarded সংযোগের তালিকা
`Enter` তারপর `~Ctrl+Z`	SSH সেশন background-এ পাঠানো
`Super+L` (GNOME/KDE)	ডেস্কটপ স্ক্রিন লক — কম্পিউটার ছেড়ে উঠলে অবশ্যই!

আরও এক ধাপ: 2FA

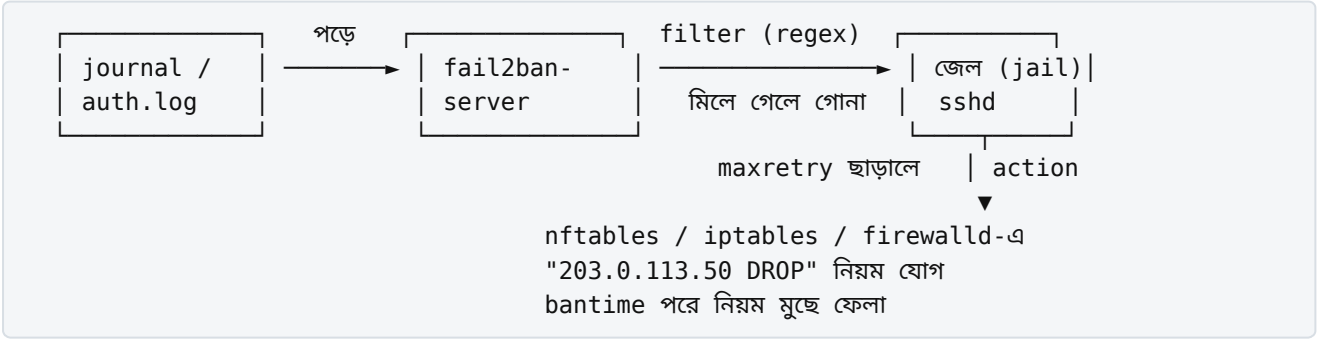
SSH-এ key-এর সাথে TOTP (Google Authenticator ধরনের) যোগ করা যায় `libpam-google-authenticator` (Debian/Ubuntu) বা `google-authenticator` (EPEL/Fedora) প্যাকেজ দিয়ে,

যেখানে `AuthenticationMethods publickey,keyboard-interactive` দিতে হয়। এটা PAM-এর একটা চমৎকার বাস্তব উদাহরণ।

৪৫.৫ fail2ban — brute-force আক্রমণকারীকে ব্যান

কী করে: লগ ফাইল পড়ে দেখে কোন IP বারবার ভুল লগইন করছে, তারপর firewall-এ সেই IP কিছু সময়ের জন্য ব্লক করে দেয়।

উপমা: পাড়ার দারোয়ান খাতায় লিখে রাখছে কে কতবার ভুল চাবি দিয়ে গেট খোলার চেষ্টা করল। ৫ বারের বেশি হলে তাকে এক ঘণ্টার জন্য গেটের সামনে থেকে সরিয়ে দেয়।



- **filter** — `/etc/fail2ban/filter.d/*.conf`, regex দিয়ে লগের কোন লাইন "ব্যর্থতা" তা চেনে
- **action** — `/etc/fail2ban/action.d/*.conf`, ব্যান কীভাবে করবে
- **jail** — filter + action + সংখ্যার সীমা মিলিয়ে একটা নিয়ম

ইনস্টল

```
$ sudo apt install fail2ban                    # Debian / Ubuntu
$ sudo dnf install epel-release && sudo dnf install fail2ban   # RHEL (EPEL থেকে)
$ sudo dnf install fail2ban                    # Fedora
```

কনফিগ

> ⚠️ **সতর্কতা:** `/etc/fail2ban/jail.conf` সরাসরি এডিট করো না — প্যাকেজ আপডেটে বদলে যেতে পারে। নিজের সেটিং রাখো `/etc/fail2ban/jail.local` বা `/etc/fail2ban/jail.d/*.local` ফাইলে।

```
$ sudo nano /etc/fail2ban/jail.local
```

```
[DEFAULT]
# নিজের IP কখনো ব্যান হবে না
ignoreip = 127.0.0.1/8 ::1 192.168.1.0/24
```

```
bantime  = 1h
findtime = 10m
maxretry = 5
# বারবার অপরাধীর ব্যানের সময় বাড়তে থাকবে
bantime.increment = true
# journald থেকে লগ পড়া (Debian 12-এ /var/log/auth.log ডিফল্টে নেই)
backend = systemd

[sshd]
enabled = true
port    = ssh
maxretry = 3
```

মানে: **findtime** (১০ মিনিট)-এর মধ্যে **maxretry** (৩) বার ব্যর্থ হলে **bantime** (১ ঘণ্টা) ব্যান।

```
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status
Status
|- Number of jail:      1
`- Jail list:   sshd

$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 2
|  |- Total failed:     147
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 4
   |- Total banned:     19
   `- Banned IP list:   203.0.113.50 198.51.100.7 ...
```

কমান্ড	কাজ
`fail2ban-client status`	সব jail-এর তালিকা
`fail2ban-client status sshd`	নির্দিষ্ট jail-এর অবস্থা
`fail2ban-client set sshd banip 1.2.3.4`	হাতে IP ব্যান
`fail2ban-client set sshd unbanip 1.2.3.4`	IP আনব্যান
`fail2ban-client unban --all`	সব আনব্যান
`fail2ban-client reload`	কনফিগ রিলোড
`fail2ban-regex <log> <filter>`	filter টেস্ট করা

> 💡 **টিপস:** nginx, Apache, Postfix ইত্যাদির জন্যও তৈরি filter আছে (`ls /etc/fail2ban/filter.d/`)। শুধু `jail.local`-এ `[nginx-http-auth]` ধরনের সেকশনে `enabled = true` দিলেই হয়।

> 📝 **মনে রাখো:** key-only SSH থাকলে fail2ban আসলে নিরাপত্তা খুব বেশি বাড়ায় না (পাসওয়ার্ড অনুমান তো কাজই করবে না), তবে লগের আবর্জনা ও সার্ভারের লোড কমায়।

৪৫.৬ অপ্রয়োজনীয় সার্ভিস বন্ধ করা

যে সার্ভিস চলছেই না, তাকে হ্যাক করা যায় না। তাই নিয়ম: **যা দরকার নেই, তা বন্ধ — সম্ভব হলে আনইনস্টল।** systemd-এর বিস্তারিত অধ্যায় ২৬-এ।

```
# এখন কোন সার্ভিস চলছে
$ systemctl list-units --type=service --state=running

# বুটে কোনগুলো চালু হবে
$ systemctl list-unit-files --type=service --state=enabled
UNIT FILE                    STATE   PRESET
avahi-daemon.service         enabled enabled
cups.service                 enabled enabled
ssh.service                  enabled enabled
...
```

সার্ভারে সাধারণত দরকার নেই এমন কিছু উদাহরণ:

সার্ভিস	কী করে	সার্ভারে লাগে?
`cups`	প্রিন্টার	প্রায়ই না
`avahi-daemon`	লোকাল নেটওয়ার্কে mDNS/Bonjour ঘোষণা	না
`bluetooth`	ব্লুটুথ	না
`rpcbind`	NFS-এর জন্য	NFS না থাকলে না
`telnet`, `rsh`, `vsftpd` (অনিরাপদ সেটআপে)	এনক্রিপশনহীন প্রোটোকল	কখনোই না

```
$ sudo systemctl disable --now cups.service cups.socket cups.path
$ sudo systemctl mask avahi-daemon.service avahi-daemon.socket
$ sudo apt purge telnetd         # একেবারে মুছে ফেলা
```

> 📝 **মনে রাখো:** `disable` = বুটে চালু হবে না, কিন্তু অন্য কেউ চাইলে চালু করতে পারে। `mask` = `/dev/null`-এর সাথে symlink করে দেয়, কেউই চালু করতে পারবে না। অনেক সার্ভিস `.socket` ইউনিট দিয়েও জেগে ওঠে — তাই socket-টাও বন্ধ করো।

৪৫.৭ খোলা পোর্ট অডিট

প্রতিটা খোলা (listening) পোর্ট একটা দরজা। কোনগুলো খোলা আছে আর কে খুলে রেখেছে, জানা জরুরি। (কমান্ডগুলোর পূর্ণ বিবরণ অধ্যায় ৩৭-এ।)

ভেতর থেকে: `ss`

```
$ sudo ss -tulpn
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
udp   UNCONN 0      0      127.0.0.53%lo:53         0.0.0.0:*     users:(("systemd-
resolve",pid=612,fd=13))
tcp   LISTEN 0      4096   0.0.0.0:22               0.0.0.0:*     users:
(("sshd",pid=901,fd=3))
tcp   LISTEN 0      511    0.0.0.0:80               0.0.0.0:*     users:
(("nginx",pid=1210,fd=6))
tcp   LISTEN 0      151    127.0.0.1:3306           0.0.0.0:*     users:
(("mysqld",pid=1033,fd=21))
tcp   LISTEN 0      4096   0.0.0.0:6379             0.0.0.0:*     users:(("redis-
server",pid=1101,fd=6))
```

অপশন	মানে
`-t`	TCP
`-u`	UDP
`-l`	শুধু listening
`-p`	কোন প্রসেস (root লাগে)
`-n`	নাম না বানিয়ে সংখ্যায় দেখানো

কীভাবে পড়বে:

- `127.0.0.1:3306` — শুধু নিজের মেশিন থেকে ঢোকা যায়। ভালো।
- `0.0.0.0:6379` বা `[::]:6379` — **সব নেটওয়ার্ক ইন্টারফেস** থেকে খোলা! Redis-এর মতো ডেটাবেস ইন্টারনেটে খোলা থাকা বড় বিপদ। এটা `bind 127.0.0.1` করে দিতে হবে।

```
# বিকল্প
$ sudo lsof -i -P -n | grep LISTEN
$ sudo lsof -i :6379          # নির্দিষ্ট পোর্ট কে ব্যবহার করছে
```

বাইরে থেকে: `nmap`

ভেতর থেকে যা দেখছ, বাইরে থেকে firewall পেরিয়ে আসলে কী দেখা যায় — সেটা বোঝার জন্য **অন্য একটা মেশিন থেকে** স্ক্যান করো:

```
$ sudo nmap -sS -sV -p- server.example.com
PORT   STATE SERVICE VERSION
```

```
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    nginx 1.24.0 (Ubuntu)
```

অপশন	মানে
`-sS`	SYN স্ক্যান (দ্রুত, root লাগে)
`-sV`	সার্ভিসের ভার্সন বের করা
`-p-`	১ থেকে ৬৫৫৩৫ সব পোর্ট
`-sU`	UDP স্ক্যান (ধীর)

> ⚠️ **সতর্কতা:** শুধু **নিজের** বা অনুমতি পাওয়া সিস্টেমে `nmap` চালাও। অন্যের সার্ভারে অনুমতি ছাড়া স্ক্যান অনেক দেশে আইনত অপরাধ হতে পারে এবং ক্লাউড প্রোভাইডারের নীতিভঙ্গ।

পোর্ট অডিটের সিদ্ধান্ত তালিকা: প্রতিটা listening পোর্টের জন্য জিজ্ঞেস করো —

1. এই সার্ভিসটা কি দরকার? না → বন্ধ/আনইনস্টল।
2. বাইরে থেকে দরকার? না → `127.0.0.1`-এ bind করো।
3. বাইরে থেকে দরকার, কিন্তু সবার জন্য না? → firewall-এ নির্দিষ্ট IP-কে অনুমতি (অধ্যায় ৩৯)।

৪৫.৮ DAC বনাম MAC — অ্যাক্সেস কন্ট্রোলের দুই দর্শন

DAC — Discretionary Access Control

এটা আমাদের চেনা `rwx` পারমিশন (অধ্যায় ১৬)। "Discretionary" মানে **ফাইলের মালিক নিজের ইচ্ছামতো** (at their discretion) ঠিক করে কে ঢুকবে। সমস্যা:

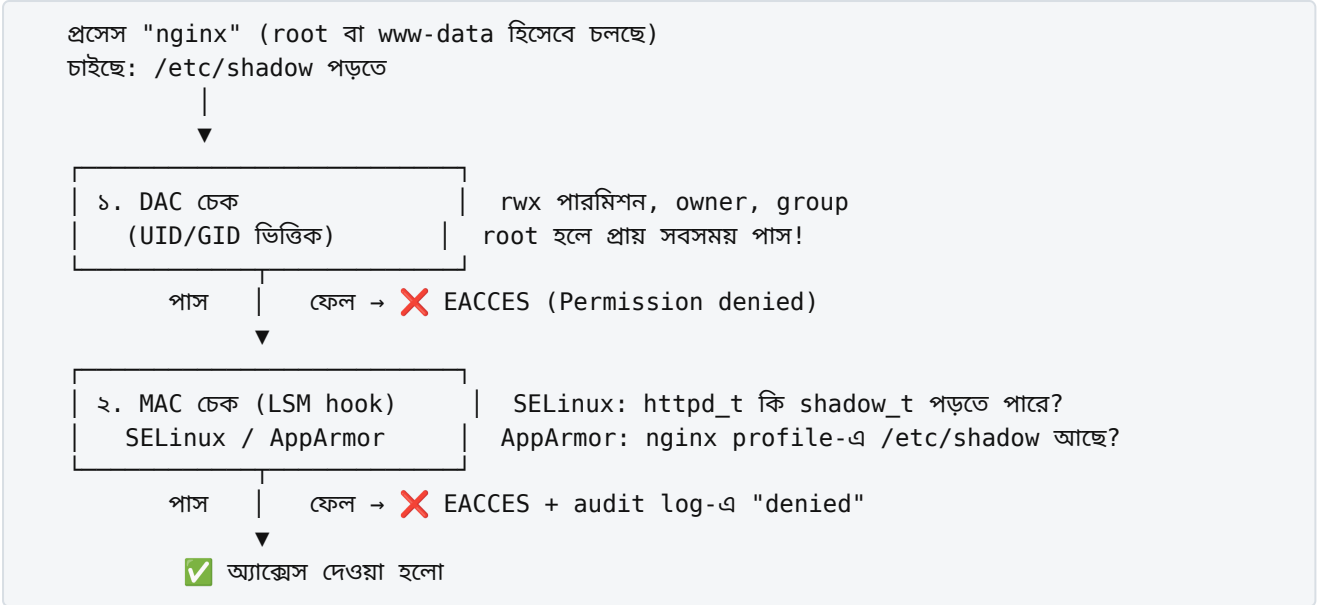
1. মালিক ভুল করে `chmod 777` দিলে সবাই সব পারে।
2. **root সবকিছুর ঊর্ধ্বে।** কোনো সার্ভিস (যেমন root হিসেবে চলা ওয়েব সার্ভার) হ্যাক হলে আক্রমণকারী পুরো সিস্টেম পায়।
3. একটা প্রোগ্রাম সেই ইউজারের **সব** ফাইল ছুঁতে পারে — তোমার ব্রাউজার হ্যাক হলে সে তোমার `~/.ssh/id_ed25519` পড়তে পারে, কারণ দুটোই তোমার।

MAC — Mandatory Access Control

এখানে **সিস্টেম-ব্যাপী নীতি (policy)** ঠিক করে কোন প্রোগ্রাম কী করতে পারবে — মালিক বা এমনকি root-ও সাধারণ উপায়ে তা এড়াতে পারে না। "Mandatory" মানে বাধ্যতামূলক।

উপমা: একটা হাসপাতাল। DAC হলো প্রতিটা রুমের চাবি — যার হাতে চাবি, সে ঢুকতে পারে, আর চাবির মালিক যাকে খুশি চাবি দিতে পারে। MAC হলো হাসপাতালের নিয়ম: "ক্যান্টিনের কর্মী (চাবি থাকুক বা না থাকুক)

অপারেশন থিয়েটারে ঢুকতে পারবে না।" এমনকি হাসপাতালের পরিচালকের দেওয়া চাবিও এই নিয়ম ভাঙতে পারে না।



> 📝 **মনে রাখো:** MAC **DAC-এর পরে** চেক হয় এবং শুধু **আরও সীমাবদ্ধ** করতে পারে, কখনো বাড়তি অনুমতি দেয় না। DAC "না" বললে MAC পর্যন্ত যাওয়াই হয় না।

ভেতরে কীভাবে: কার্নেলে আছে **LSM** (Linux Security Modules) framework — কার্নেলের গুরুত্বপূর্ণ জায়গাগুলোতে (ফাইল খোলা, সকেট বানানো, প্রসেস চালানো) "hook" বসানো আছে। SELinux আর AppArmor হলো এই hook-এ লাগানো দুটো ভিন্ন LSM। কোনটা চালু আছে দেখতে:

```
$ cat /sys/kernel/security/lsm
lockdown,capability,landlock,yama,apparmor,bpf          # Ubuntu
lockdown,capability,yama,selinux,bpf,landlock           # Fedora/RHEL (উদাহরণ)
```

বিষয়	SELinux	AppArmor
ডিফল্ট distro	RHEL, Fedora, CentOS Stream, Rocky, Alma	Ubuntu, Debian (10 থেকে), openSUSE (পুরোনো ভার্সন)
ভিত্তি	**label** (প্রতিটা ফাইল/প্রসেসের গায়ে context লেখা)	**path** (ফাইলের পাথ দিয়ে নিয়ম)
জটিলতা	বেশি, কিন্তু অনেক সূক্ষ্ম নিয়ন্ত্রণ	কম, শেখা সহজ
কভারেজ	প্রায় পুরো সিস্টেম	শুধু যেসব প্রোগ্রামের profile আছে
মূল টুল	`getenforce`, `semanage`, `restorecon`	`aa-status`, `aa-enforce`, `aa-complain`

৪৫.৯ SELinux

SELinux (Security-Enhanced Linux) মূলত আমেরিকার NSA তৈরি করে, ২০০৩ সালে কার্নেল ২.৬-এ যুক্ত হয়। RHEL/Fedora-তে এটা ডিফল্টে চালু থাকে।

> ⚠️ **সতর্কতা:** অনেক টিউটোরিয়ালে প্রথম লাইনেই লেখা থাকে "SELinux বন্ধ করে দিন"। **এটা কোরো না।** SELinux সমস্যা করলে সেটা প্রায় সবসময় একটা ভুল label বা বন্ধ boolean — এই অধ্যায় পড়ে তুমি ২ মিনিটে ঠিক করতে পারবে।

SELinux mode

Mode	কী হয়
Enforcing	নীতি চাপানো হয় — নিষিদ্ধ কাজ আটকানো হয় এবং লগ হয়
Permissive	কিছু আটকানো হয় না, শুধু "এটা আটকানো উচিত ছিল" লগ হয় — ডিবাগের জন্য
Disabled	SELinux একদম বন্ধ — কোনো label-ও রক্ষণাবেক্ষণ হয় না

```
$ getenforce
Enforcing

$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
```

সাময়িকভাবে mode বদলানো (রিবুটে আবার আগের মতো):

```
$ sudo setenforce 0      # Permissive
$ getenforce
Permissive
$ sudo setenforce 1      # Enforcing
```

`setenforce` শুধু Enforcing ⇄ Permissive-এর মধ্যে বদলাতে পারে, Disabled করতে পারে না।

স্থায়ীভাবে: `/etc/selinux/config` ফাইলে:

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

> 📝 **মনে রাখো:** RHEL 9-এ পুরোপুরি disable করতে হলে config ফাইলের বদলে কার্নেল প্যারামিটার ব্যবহারের পরামর্শ দেওয়া হয়: `sudo grubby --update-kernel ALL --args selinux=0`। কিন্তু আবার বলছি — disable কোরো না। আর disabled থেকে আবার চালু করলে পুরো ফাইলসিস্টেম relabel করতে হয় (`sudo touch /.autorelabel` তারপর রিবুট — বড় ডিস্কে অনেক সময় নেয়)।

> 💡 **টিপস:** পুরো সিস্টেম permissive না করে শুধু একটা সার্ভিসের domain permissive করা যায়: `sudo semanage permissive -a httpd_t` (সরাতে `-d`)। বাকি সিস্টেম সুরক্ষিত থাকে।

Security Context — SELinux-এর "লেবেল"

SELinux-এ প্রতিটা ফাইল, প্রসেস, পোর্টের গায়ে একটা লেবেল থাকে, যাকে বলে **context**। দেখতে `-Z` অপশন:

```
$ ls -Z /var/www/html/index.html
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html

$ ps -eZ | grep httpd
system_u:system_r:httpd_t:s0       1234 ?        00:00:00 httpd

$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Context-এর চারটা অংশ:

```
   system_u : object_r : httpd_sys_content_t : s0
   ────┬───   ────┬────   ─────────┬─────────   ──┬
       │          │                │              └─ level (MLS/MCS সংবেদনশীলতা স্তর)
       │          │                └─ TYPE  ★ সবচেয়ে গুরুত্বপূর্ণ
       │          └─ role
       └─ SELinux user (Linux user না!)
```

Type Enforcement — SELinux-এর মূল ধারণা: প্রসেসের type-কে বলে **domain** (যেমন `httpd_t`)। নীতিতে লেখা থাকে কোন domain কোন type-এর ফাইলে কী করতে পারবে:

```
allow httpd_t httpd_sys_content_t : file { read open getattr };
└─ httpd_t domain-এর প্রসেস httpd_sys_content_t type-এর ফাইল পড়তে পারবে
```

নীতিতে যা স্পষ্টভাবে `allow` নেই, তা নিষিদ্ধ (default deny)। তাই হ্যাক হওয়া Apache `/etc/shadow` (type `shadow_t`) বা `/home/rahim` (type `user_home_t`) পড়তে পারবে না — প্রসেসটা root হলেও।

```
$ ls -Z /etc/shadow
system_u:object_r:shadow_t:s0 /etc/shadow
```

সবচেয়ে সাধারণ SELinux সমস্যা: ভুল label

একটা ক্লাসিক দৃশ্য:

```
$ echo "Hello" > ~/index.html
$ sudo mv ~/index.html /var/www/html/
$ curl http://localhost/index.html
<title>403 Forbidden</title>
```

পারমিশন তো ঠিক আছে, তাহলে 403 কেন?

```
$ ls -Z /var/www/html/index.html
unconfined_u:object_r:user_home_t:s0 /var/www/html/index.html
```

কারণ: `mv` ফাইলের পুরোনো label (`user_home_t`) সাথে নিয়ে এসেছে। Apache `user_home_t` পড়তে পারে না। (`cp` করলে নতুন জায়গার ডিফল্ট label পেত।)

সমাধান — `restorecon` : নীতিতে লেখা "এই পাথের ডিফল্ট label" ফিরিয়ে দেয়।

```
$ sudo restorecon -v /var/www/html/index.html
Relabeled /var/www/html/index.html from unconfined_u:object_r:user_home_t:s0 to
unconfined_u:object_r:httpd_sys_content_t:s0

$ sudo restorecon -Rv /var/www/html     # পুরো ডিরেক্টরি, recursive
```

`restorecon` অপশন	মানে
`-R`	recursive
`-v`	কী বদলাল দেখানো
`-n`	কিছু না বদলে শুধু দেখানো (dry run)
`-F`	user/role/level সহ পুরো context জোর করে রিসেট

নিজের পাথের জন্য স্থায়ী label: `semanage fcontext`

ধরো ওয়েবসাইট রাখবে `/srv/mysite` -এ। এই পাথ নীতিতে নেই, তাই label হবে `var_t` বা `default_t` , Apache পড়তে পারবে না।

```
# ভুল উপায় (অস্থায়ী): chcon — পরের restorecon বা relabel-এ মুছে যাবে
$ sudo chcon -R -t httpd_sys_content_t /srv/mysite

# সঠিক উপায়: নীতিতে নিয়ম যোগ, তারপর প্রয়োগ
$ sudo semanage fcontext -a -t httpd_sys_content_t "/srv/mysite(/.*)?"
$ sudo restorecon -Rv /srv/mysite

# নিয়মটা দেখা
$ sudo semanage fcontext -l | grep mysite
/srv/mysite(/.*)?    all files    system_u:object_r:httpd_sys_content_t:s0
```

> 📝 **মনে রাখো:** `chcon` = অস্থায়ী, সরাসরি ফাইলে লেবেল লেখে। `semanage fcontext` + `restorecon` = স্থায়ী, নীতিতে নিয়ম রাখে। সবসময় দ্বিতীয়টা ব্যবহার করো।

> 💡 **টিপস:** `semanage` কমান্ড না পেলে প্যাকেজ ইনস্টল করো: `sudo dnf install policycoreutils-python-utils`। আর `setroubleshoot-server` ইনস্টল করলে `sealert` দিয়ে সহজ ভাষায় সমস্যা ও সমাধান পাবে।

SELinux Boolean — নীতির অন/অফ সুইচ

পুরো নীতি নতুন করে লেখা ছাড়াই কিছু সাধারণ আচরণ চালু/বন্ধ করার জন্য boolean আছে।

```
$ getsebool -a | grep httpd_can_network
httpd_can_network_connect --> off
httpd_can_network_connect_db --> off

# nginx/Apache যদি reverse proxy হিসেবে অন্য পোর্টে সংযোগ করতে চায়:
$ sudo setsebool -P httpd_can_network_connect on
```

`-P` না দিলে রিবুটে আগের মানে ফিরে যাবে। `semanage boolean -l` দিয়ে প্রতিটা boolean-এর বর্ণনা দেখা যায়।

পোর্ট label

```
$ sudo semanage port -l | grep http_port_t
http_port_t     tcp      80, 81, 443, 488, 8008, 8009, 8443, 9000

$ sudo semanage port -a -t http_port_t -p tcp 8088     # Apache-কে 8088 শোনার অনুমতি
```

SELinux ডিবাগিং: AVC, `ausearch`, `audit2why`, `audit2allow`

SELinux কিছু আটকালে একটা **AVC** (Access Vector Cache) "denied" বার্তা লগ হয় — `auditd` চালু থাকলে `/var/log/audit/audit.log`-এ।

```
$ sudo ausearch -m AVC -ts recent
type=AVC msg=audit(1731400000.123:456): avc:  denied  { read } for  pid=1234
  comm="httpd" name="index.html" dev="dm-0" ino=393220
  scontext=system_u:system_r:httpd_t:s0
  tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0
```

পড়ার কৌশল: **কে** (`scontext` = `httpd_t`) **কী করতে চেয়েছিল** (`{ read }`) **কার ওপর** (`tcontext` = `user_home_t`, `tclass=file`)।

`audit2why` — কেন আটকানো হলো, সহজ ভাষায় বলে:

```
$ sudo ausearch -m AVC -ts recent | audit2why
type=AVC msg=audit(...): avc:  denied  { read } for ... comm="httpd" ...

        Was caused by:
                Missing type enforcement (TE) allow rule.

                You can use audit2allow to generate a loadable module to allow this
access.
```

boolean দিয়ে ঠিক হওয়ার মতো সমস্যা হলে `audit2why` সরাসরি বলে দেয় কোন boolean চালু করতে হবে — যেমন `setsebool -P httpd_can_network_connect 1`।

`audit2allow` — denial থেকে নতুন নীতি মডিউল বানায়:

```
$ sudo ausearch -m AVC -ts recent | audit2allow -M mywebfix
******************* IMPORTANT ***********************
To make this policy package active, execute:

semodule -i mywebfix.pp

$ cat mywebfix.te        # আগে পড়ে দেখো কী অনুমতি দিচ্ছ!
$ sudo semodule -i mywebfix.pp
$ sudo semodule -l | grep mywebfix     # লোড হয়েছে কি না
$ sudo semodule -r mywebfix            # সরাতে
```

> ⚠️ **সতর্কতা:** `audit2allow` হলো শেষ উপায়। উপরের উদাহরণে আসল সমস্যা ছিল ভুল label — `audit2allow` চালালে Apache-কে সব ইউজারের হোম ফাইল পড়ার অনুমতি দিয়ে বসতে! সমস্যা সমাধানের ক্রম মনে রাখো:

```
SELinux denial হলো?
  │
  ├─► ১. label ঠিক আছে?          → না  → restorecon / semanage fcontext
  │
  ├─► ২. কোনো boolean আছে?       → হ্যাঁ → setsebool -P  (audit2why বলে দেবে)
  │
  ├─► ৩. পোর্ট label?            → না  → semanage port -a
  │
  └─► ৪. কিছুই না মিললে          → audit2allow দিয়ে ছোট, পড়ে-দেখা মডিউল
```

৪৫.১০ AppArmor

Ubuntu ও Debian-এ ডিফল্ট MAC হলো **AppArmor**। এখানে label-এর বদলে প্রতিটা প্রোগ্রামের জন্য একটা **profile** থাকে, যাতে পাথ দিয়ে লেখা থাকে সেই প্রোগ্রাম কোন ফাইলে কী করতে পারবে।

অবস্থা দেখা: `aa-status`

```
$ sudo aa-status
apparmor module is loaded.
121 profiles are loaded.
33 profiles are in enforce mode.
   /usr/bin/man
   /usr/sbin/cupsd
   /usr/sbin/tcpdump
   lsb_release
   nvidia_modprobe
   ...
5 profiles are in complain mode.
   ...
4 processes have profiles defined.
4 processes are in enforce mode.
   /usr/sbin/cupsd (1102)
   ...
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
```

(`apparmor_status` একই কমান্ডের পুরোনো নাম।)

দুই mode

Mode	কাজ
enforce	নিয়ম চাপানো হয়, নিষিদ্ধ কাজ আটকানো ও লগ
complain	কিছু আটকানো হয় না, শুধু লগ (SELinux-এর permissive-এর মতো)

Profile দেখতে কেমন

Profile থাকে `/etc/apparmor.d/`-তে, নাম সাধারণত প্রোগ্রামের পাথের `/` বদলে `.` দিয়ে, যেমন `/usr/sbin/tcpdump` → `usr.sbin.tcpdump`।

```
# /etc/apparmor.d/usr.sbin.tcpdump (সংক্ষেপিত)
abi <abi/4.0>,
include <tunables/global>

profile tcpdump /usr/sbin/tcpdump {
  include <abstractions/base>
  include <abstractions/nameservice>

  capability net_raw,
  capability setuid,
  capability setgid,

  network raw,
  network packet,

  /usr/sbin/tcpdump mr,
```

```
  /etc/ld.so.cache r,
  # pcap ফাইল লেখা যাবে
  /**.[pP][cC][aA][pP] rw,
}
```

পারমিশন অক্ষর: `r` পড়া, `w` লেখা, `a` append, `m` মেমরিতে executable ম্যাপ, `k` lock, `l` link, `ix` / `px` / `cx` / `ux` — অন্য প্রোগ্রাম চালানোর নিয়ম (inherit / আলাদা profile / child / unconfined)।

Profile ম্যানেজ করা

```
$ sudo apt install apparmor-utils apparmor-profiles   # aa-* টুল ও বাড়তি profile

$ sudo aa-complain /usr/sbin/tcpdump     # complain mode-এ
$ sudo aa-enforce  /usr/sbin/tcpdump     # enforce mode-এ
$ sudo aa-disable  /usr/sbin/tcpdump     # profile নিষ্ক্রিয়

# profile ফাইল হাতে বদলালে কার্নেলে আবার লোড
$ sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.tcpdump
$ sudo systemctl reload apparmor          # সব profile রিলোড
```

নতুন profile তৈরি

```
$ sudo aa-genprof /usr/local/bin/myapp
# এক টার্মিনালে aa-genprof চলবে, অন্য টার্মিনালে myapp ব্যবহার করো
# তারপর 'S' (Scan) চেপে লগ থেকে নিয়ম প্রস্তাব নাও, প্রতিটা Allow/Deny করো, শেষে 'F'

$ sudo aa-logprof     # পরে নতুন denial দেখে profile আপডেট
```

AppArmor denial দেখা

```
$ sudo journalctl -k | grep -i 'apparmor="DENIED"'
audit: type=1400 audit(1731400000.555:88): apparmor="DENIED" operation="open"
  profile="/usr/sbin/tcpdump" name="/etc/shadow" pid=4321 comm="tcpdump"
  requested_mask="r" denied_mask="r" fsuid=0 ouid=0
```

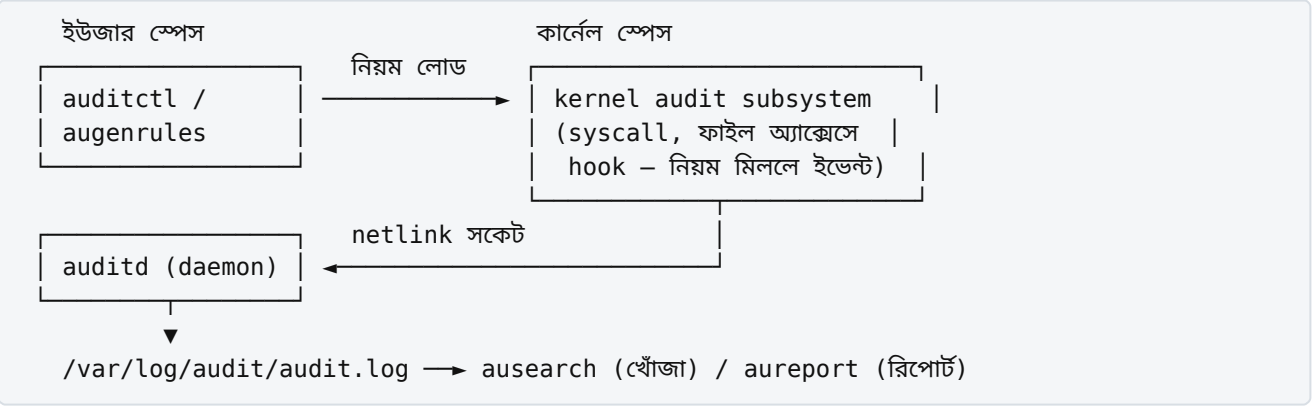
`auditd` চালু থাকলে `/var/log/audit/audit.log`-এও আসে।

> 📝 **মনে রাখো:** Ubuntu 24.04 থেকে AppArmor দিয়ে **unprivileged user namespace** সীমিত করা হয়েছে (`kernel.apparmor_restrict_unprivileged_userns=1`)। কিছু অ্যাপ (যেমন কিছু AppImage/ব্রাউজার sandbox) এ কারণে চলতে না পারলে সেই অ্যাপের জন্য profile দরকার হয়। পুরো ফিচার বন্ধ করে দেওয়া নিরাপদ সমাধান না।

৪৫.১১ auditd — কে, কখন, কী করল

MAC আটকায়, কিন্তু অনেক সময় দরকার **হিসাব রাখা**: কে `/etc/passwd` বদলাল? কে রাত ২টায় `sudo` চালাল? এর জন্য লিনাক্সের **Audit framework**।

উপমা: ব্যাংকের CCTV ক্যামেরা। এটা চোর আটকায় না, কিন্তু পরে কী ঘটেছিল তার অকাট্য প্রমাণ রাখে।



ইনস্টল

```
$ sudo apt install auditd audispd-plugins     # Debian / Ubuntu
$ sudo dnf install audit                      # RHEL / Fedora (সাধারণত আগে থেকেই আছে)
$ sudo systemctl enable --now auditd
```

> ⚠️ **সতর্কতা:** `auditd` সার্ভিস `systemctl restart/stop` দিয়ে বন্ধ করতে দেয় না (ইউনিট ফাইলে `RefuseManualStop=yes`) — যাতে আক্রমণকারী সহজে নজরদারি বন্ধ করতে না পারে। রিস্টার্ট দরকার হলে: `sudo service auditd restart`।

নিয়ম (rules) — দুই ধরন

১. ফাইল watch (`-w`):

```
$ sudo auditctl -w /etc/passwd -p wa -k identity
$ sudo auditctl -w /etc/sudoers -p wa -k sudoers_change
$ sudo auditctl -w /etc/ssh/sshd_config -p wa -k sshd_config
```

অংশ	মানে
`-w <path>`	এই ফাইল/ডিরেক্টরি নজরে রাখো
`-p rwxa`	কোন অ্যাক্সেস: `r` পড়া, `w` লেখা, `x` চালানো, `a` attribute বদল
`-k <key>`	পরে খোঁজার জন্য একটা নাম-ট্যাগ

২. syscall নিয়ম (`-a`):

```
# সাধারণ ইউজার (auid>=1000) কোনো ফাইল মুছলে লগ
$ sudo auditctl -a always,exit -F arch=b64 -S unlink,unlinkat,rename,renameat \
    -F auid>=1000 -F auid!=unset -k delete
```

`auid` মানে **audit UID** — লগইনের সময় যে ইউজার ছিল। কেউ `sudo su -` করে root হলেও auid আসল ইউজারটাই থাকে। তাই "কে আসলে করেছে" ধরা যায়।

```
$ sudo auditctl -l          # বর্তমান নিয়ম দেখা
-w /etc/passwd -p wa -k identity
...
$ sudo auditctl -s          # অবস্থা (enabled, lost ইভেন্ট ইত্যাদি)
$ sudo auditctl -D          # সব নিয়ম মুছে ফেলা (অস্থায়ী)
```

স্থায়ী নিয়ম

`auditctl` দিয়ে দেওয়া নিয়ম রিবুটে হারিয়ে যায়। স্থায়ী করতে `/etc/audit/rules.d/`-তে ফাইল বানাও:

```
$ sudo nano /etc/audit/rules.d/50-hardening.rules
```

```
-w /etc/passwd -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/sudoers -p wa -k sudoers_change
-w /etc/sudoers.d/ -p wa -k sudoers_change
-w /etc/ssh/sshd_config -p wa -k sshd_config
-w /var/log/lastlog -p wa -k logins
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root_cmd
```

```
$ sudo augenrules --load    # rules.d-র সব ফাইল জোড়া দিয়ে audit.rules বানিয়ে লোড
```

> 💡 **টিপস:** একদম শেষ নিয়ম হিসেবে `-e 2` দিলে নিয়মগুলো **immutable** হয়ে যায় — রিবুট ছাড়া কেউ (root-ও না) বদলাতে পারবে না। প্রোডাকশনের জন্য ভালো, কিন্তু টেস্টের সময় দিও না।

লগ খোঁজা ও রিপোর্ট

```
$ sudo ausearch -k identity -i
----
type=PROCTITLE msg=audit(11/12/2026 10:45:12.331:812) : proctitle=useradd hacker
type=PATH msg=audit(...) : item=0 name=/etc/passwd ...
type=SYSCALL msg=audit(...) : arch=x86_64 syscall=openat success=yes ...
  auid=rahim uid=root comm=useradd exe=/usr/sbin/useradd key=identity
```

এখান থেকে স্পষ্ট: `rahim` (auid) root হয়ে `useradd hacker` চালিয়েছে।

কমান্ড	কাজ
`ausearch -k <key>`	key দিয়ে খোঁজা
`ausearch -i`	সংখ্যা (UID, syscall) মানুষের পড়ার মতো করে দেখানো
`ausearch -ua 1000`	নির্দিষ্ট UID/auid-এর ইভেন্ট
`ausearch -m USER_LOGIN --success no`	ব্যর্থ লগইন
`ausearch -ts today` / `-ts recent`	আজকের / গত ১০ মিনিটের
`aureport --summary`	সারাংশ
`aureport -au`	অথেন্টিকেশন রিপোর্ট
`aureport -l --failed`	ব্যর্থ লগইন রিপোর্ট
`aureport -x --summary`	কোন প্রোগ্রাম কতবার চলেছে

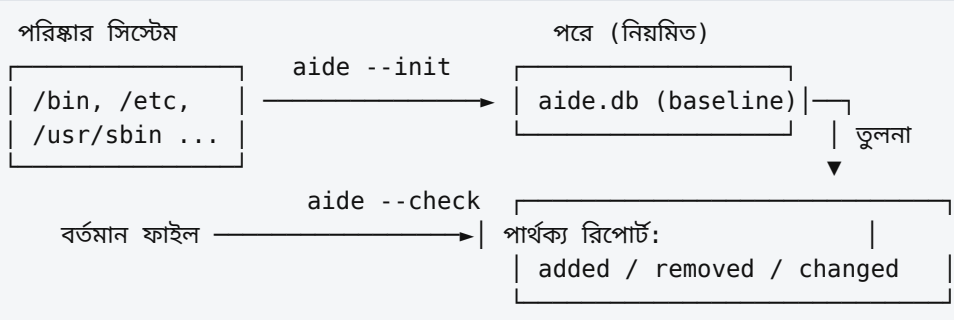
কনফিগ `/etc/audit/auditd.conf`-এ লগের আকার ও ঘোরানো (`max_log_file`, `num_logs`, `max_log_file_action`) ঠিক করা যায়। (সাধারণ লগ ব্যবস্থার জন্য দেখো অধ্যায় ২৮।)

৪৫.১২ AIDE — ফাইল ইন্টেগ্রিটি যাচাই

আক্রমণকারী সিস্টেমে ঢুকে প্রায়ই `/bin/ls`, `/usr/bin/ps`, `/usr/sbin/sshd`-এর মতো ফাইল বদলে দেয় (যাতে নিজের প্রসেস লুকানো যায় বা পাসওয়ার্ড চুরি করা যায়)। **AIDE** (Advanced Intrusion Detection Environment) এমন পরিবর্তন ধরে।

কীভাবে কাজ করে: পরিষ্কার অবস্থায় প্রতিটা গুরুত্বপূর্ণ ফাইলের "আঙুলের ছাপ" (hash, size, permission, owner, inode, mtime) একটা ডেটাবেসে রাখে। পরে আবার মিলিয়ে দেখে কী বদলেছে।

উপমা: বাড়ি থেকে বের হওয়ার আগে প্রতিটা ঘরের ছবি তুলে রাখলে, ফিরে এসে ছবির সাথে মিলিয়ে বোঝা যায় কিছু সরানো হয়েছে কি না।



RHEL / Fedora

```
$ sudo dnf install aide
$ sudo aide --init                # baseline তৈরি (কয়েক মিনিট লাগে)
AIDE initialized database at /var/lib/aide/aide.db.new.gz
```

```
$ sudo mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz

$ sudo aide --check
AIDE found NO differences between database and filesystem. Looks okay!!
```

Debian / Ubuntu

```
$ sudo apt install aide
$ sudo aideinit                     # ডেটাবেস বানায় ও /var/lib/aide/aide.db-তে কপি করে
$ sudo aide --config /etc/aide/aide.conf --check
```

Debian প্যাকেজ একটা দৈনিক চেক (cron/systemd timer `dailyaidecheck`) সাথে দেয়।

কিছু বদলালে

```
$ sudo aide --check
AIDE found differences between database and filesystem!!
Summary:
  Total number of entries:      145032
  Added entries:                0
  Removed entries:              0
  Changed entries:              1
---------------------------------------------------
Changed entries:
---------------------------------------------------
f   ...    .C... : /usr/bin/ls
---------------------------------------------------
Detailed information about changes:
---------------------------------------------------
File: /usr/bin/ls
  SHA512   : 3bQ1...                          | 9xZk...
```

বৈধ পরিবর্তনের (যেমন প্যাকেজ আপডেট) পর baseline হালনাগাদ:

```
$ sudo aide --update                # নতুন ডেটাবেস বানায় (aide.db.new.gz)
$ sudo mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
```

কী নজরে রাখবে তা ঠিক হয় `/etc/aide.conf` (RHEL) বা `/etc/aide/aide.conf` + `/etc/aide/aide.conf.d/` (Debian) ফাইলে।

> ⚠️ **সতর্কতা:** আক্রমণকারী root পেলে AIDE-এর ডেটাবেসও বদলে দিতে পারে! তাই baseline ডেটাবেসের একটা কপি **অফলাইনে / রিড-অনলি মিডিয়ায়** রাখো, আর আসল চেক সম্ভব হলে বিশ্বস্ত মিডিয়া থেকে চালাও।

> 📝 **মনে রাখো:** Debian/Ubuntu-তে প্যাকেজের ফাইল দ্রুত যাচাইয়ের জন্য `debsums` (`sudo debsums`

> `-c`) আর RHEL/Fedora-তে `rpm -Va` আছে — এরা প্যাকেজ ডেটাবেসের hash-এর সাথে মিলিয়ে দেখে।

৪৫.১৩ rkhunter — rootkit খোঁজা

Rootkit হলো এমন ম্যালওয়্যার যা root অ্যাক্সেস পাওয়ার পর নিজেকে লুকিয়ে রাখে — `ps`-এ নিজের প্রসেস দেখায় না, `ls`-এ নিজের ফাইল দেখায় না, কখনো কার্নেল মডিউল হিসেবে কার্নেলের ভেতরেই বসে থাকে।

rkhunter (Rootkit Hunter) যা পরীক্ষা করে:

- পরিচিত rootkit-এর ফাইল ও সিগনেচার
- গুরুত্বপূর্ণ কমান্ডের (`ls`, `ps`, `netstat`...) hash ও পারমিশন বদলেছে কি না
- লুকানো ফাইল, সন্দেহজনক কার্নেল মডিউল, অদ্ভুত খোলা পোর্ট
- SSH কনফিগ (যেমন root লগইন চালু কি না)

```
$ sudo apt install rkhunter        # Debian / Ubuntu
$ sudo dnf install rkhunter        # Fedora; RHEL-এ EPEL থেকে

$ sudo rkhunter --update           # সিগনেচার ডেটাবেস আপডেট
$ sudo rkhunter --propupd          # বর্তমান ফাইলগুলোর property baseline
$ sudo rkhunter --check --sk       # স্ক্যান (--sk = প্রতিটা ধাপে Enter চাপতে হবে না)
...
System checks summary
=====================
File properties checks...
    Files checked: 142
    Suspect files: 0

Rootkit checks...
    Rootkits checked : 498
    Possible rootkits: 0

Applications checks...
    All checks skipped

The system checks took: 1 minute and 52 seconds

All results have been written to the log file: /var/log/rkhunter.log
```

অপশন	কাজ
`--update`	ডেটা ফাইল আপডেট
`--propupd`	ফাইল property baseline আপডেট (সিস্টেম আপডেটের পর)
`-c`, `--check`	স্ক্যান চালানো
`--sk`, `--skip-keypress`	কী চাপার অপেক্ষা না করা
`--rwo`, `--report-warnings-only`	শুধু সতর্কতা দেখানো

অপশন	কাজ
`--cronjob`	cron-এর জন্য (রঙ ও কী-চাপা ছাড়া)

```
$ sudo rkhunter --check --sk --rwo
Warning: The SSH and rkhunter configuration options should be the same:
         SSH configuration option 'PermitRootLogin': no
         Rkhunter configuration option 'ALLOW_SSH_ROOT_USER': unset
```

> 💡 **টিপস:** rkhunter-এ অনেক **false positive** (ভুল সতর্কতা) আসে, বিশেষত প্যাকেজ আপডেটের পর। প্রতিটা সতর্কতা `/var/log/rkhunter.log`-এ পড়ে দেখো, আর বৈধ আপডেটের পর `--propupd` চালাও। কনফিগ: `/etc/rkhunter.conf` (Debian-এ `/etc/default/rkhunter`-এ দৈনিক চালানোর সেটিং)। বিকল্প টুল: `chkrootkit`।

> ⚠️ **সতর্কতা:** সত্যিকারের rootkit সন্দেহ হলে সংক্রমিত সিস্টেমের নিজের টুলের ওপর ভরসা করা যায় না — rootkit সেগুলোকেও ধোঁকা দিতে পারে। সবচেয়ে নিরাপদ পথ: সিস্টেম নেটওয়ার্ক থেকে আলাদা করো, live USB থেকে বুট করে ডিস্ক পরীক্ষা করো, ডেটা ব্যাকআপ নিয়ে **নতুন করে ইনস্টল** করো। পাসওয়ার্ড ও key সব বদলাও।

৪৫.১৪ ডাউনলোড যাচাই — checksum ও GPG

ধরো Ubuntu-র ISO ডাউনলোড করলে। দুটো প্রশ্ন:

1. **ফাইলটা কি নষ্ট হয়নি?** (ডাউনলোডে ভুল) → **checksum** দিয়ে যাচাই
2. **ফাইলটা কি সত্যিই Ubuntu-র, নাকি কেউ মাঝপথে বদলে দিয়েছে?** → **GPG signature** দিয়ে যাচাই

Hash / Checksum কী?

Hash function যেকোনো আকারের ডেটা থেকে একটা নির্দিষ্ট দৈর্ঘ্যের "আঙুলের ছাপ" বানায়। ফাইলের একটা বিটও বদলালে hash পুরো বদলে যায়।

```
$ echo "hello" | sha256sum
5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03  -
$ echo "hellp" | sha256sum
# সম্পূর্ণ ভিন্ন একটা মান আসবে — একটা অক্ষর বদলালেই
```

টুল	অ্যালগরিদম	এখন নিরাপদ?
`md5sum`	MD5 (128-bit)	❌ ভাঙা — শুধু দুর্ঘটনাজনিত নষ্ট ধরার জন্য
`sha1sum`	SHA-1 (160-bit)	❌ collision দেখানো হয়েছে

টুল	অ্যালগরিদম	এখন নিরাপদ?
`sha256sum`	SHA-256	✅
`sha512sum`	SHA-512	✅
`b2sum`	BLAKE2b	✅

`sha256sum` ব্যবহার

```
# একটা ফাইলের hash
$ sha256sum ubuntu-24.04.1-desktop-amd64.iso
c2e6f4dc37ac944e2ed507f87c6188dd4d3179bf4a3f9e110d3c88d1f3294bdc  ubuntu-24.04.1-
desktop-amd64.iso
# (উপরের hash উদাহরণমাত্র — আসল মান Ubuntu-র SHA256SUMS ফাইলে দেখো)

# প্রকাশকের দেওয়া SHA256SUMS ফাইল দিয়ে স্বয়ংক্রিয় মিলানো
$ sha256sum -c SHA256SUMS --ignore-missing
ubuntu-24.04.1-desktop-amd64.iso: OK
```

অপশন	কাজ
`-c FILE`	FILE-এ লেখা hash-এর সাথে মিলিয়ে দেখা
`--ignore-missing`	তালিকার যেসব ফাইল ডাউনলোড করোনি, সেগুলো উপেক্ষা
`--quiet`	শুধু ব্যর্থতা দেখানো
`--status`	কিছু না দেখিয়ে শুধু exit code (স্ক্রিপ্টের জন্য)

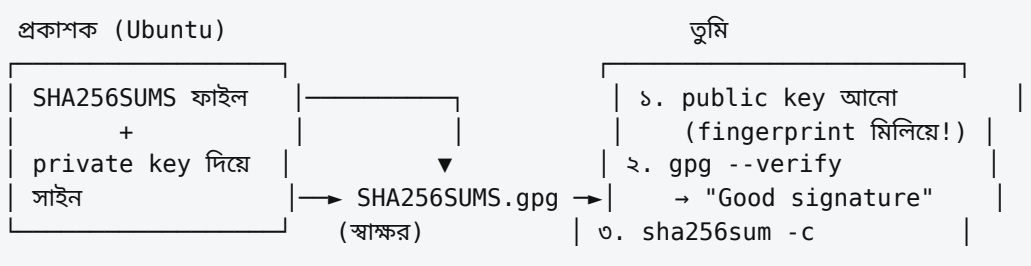
> ⚠️ **সতর্কতা:** checksum একা **যথেষ্ট না**। আক্রমণকারী যদি ওয়েবসাইট হ্যাক করে ISO বদলাতে পারে, তাহলে পাশের `SHA256SUMS` ফাইলটাও বদলে দিতে পারে! এজন্যই দরকার GPG signature।

GPG — ডিজিটাল স্বাক্ষর

GPG (GNU Privacy Guard) **public-key cryptography** ব্যবহার করে। প্রত্যেকের দুটো key:

- **Private key** — নিজের কাছে গোপন রাখো, এটা দিয়ে **সাইন** করো (আর ডিক্রিপ্ট)।
- **Public key** — সবাইকে দাও, এটা দিয়ে অন্যরা **স্বাক্ষর যাচাই** করে (আর তোমার জন্য এনক্রিপ্ট)।

উপমা: private key হলো তোমার হাতের সই করার ক্ষমতা, যা কেউ নকল করতে পারে না। public key হলো ব্যাংকে জমা রাখা তোমার সইয়ের নমুনা, যা দেখে যে কেউ মিলিয়ে নিতে পারে।



```
                                        │    → ISO: OK                │
                                        └─────────────────────────────┘

   বিশ্বাসের শৃঙ্খল: key-এর fingerprint → স্বাক্ষর → SHA256SUMS → ISO
```

ধাপে ধাপে ISO যাচাই (Ubuntu উদাহরণ)

```
# ১. তিনটা ফাইল একই ফোল্ডারে: ISO, SHA256SUMS, SHA256SUMS.gpg

# ২. স্বাক্ষর যাচাইয়ের চেষ্টা — প্রথমবার key নেই বলে কোন key লাগবে তা বলবে
$ gpg --keyid-format long --verify SHA256SUMS.gpg SHA256SUMS
gpg: Signature made ...
gpg:                using RSA key <KEY-ID>
gpg: Can't check signature: No public key

# ৩. ঐ key keyserver থেকে আনো
$ gpg --keyid-format long --keyserver hkps://keyserver.ubuntu.com --recv-keys <KEY-ID>

# ৪. key-এর fingerprint দেখো এবং প্রকাশকের অফিসিয়াল ডকুমেন্টেশনের সাথে মিলাও
$ gpg --fingerprint <KEY-ID>

# ৫. আবার যাচাই
$ gpg --keyid-format long --verify SHA256SUMS.gpg SHA256SUMS
gpg: Good signature from "Ubuntu CD Image Automatic Signing Key (2012)
<cdimage@ubuntu.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.

# ৬. এবার checksum
$ sha256sum -c SHA256SUMS --ignore-missing
ubuntu-24.04.1-desktop-amd64.iso: OK
```

> 📝 **মনে রাখো:** "**Good signature**" মানে ফাইলটা ঐ key দিয়েই সাইন করা। "WARNING: not certified" মানে তুমি নিজে এখনো ঐ key-কে "বিশ্বস্ত" হিসেবে চিহ্নিত করোনি — এটা স্বাভাবিক, **যদি তুমি fingerprint প্রকাশকের অফিসিয়াল সূত্রের সাথে মিলিয়ে থাকো**। কিন্তু "**BAD signature**" দেখলে ফাইলটা কখনো ব্যবহার কোরো না।

নিজের GPG key ও দৈনন্দিন ব্যবহার

```
$ gpg --full-generate-key                # নতুন key (ডিফল্ট ECC/ed25519 বা RSA বেছে নাও)
$ gpg --list-keys                        # public key তালিকা
$ gpg --list-secret-keys --keyid-format long
$ gpg --armor --export rahim@example.com > rahim.pub.asc   # public key শেয়ারের জন্য
$ gpg --import someone.pub.asc           # অন্যের public key যোগ

$ gpg --detach-sign --armor report.pdf   # report.pdf.asc স্বাক্ষর তৈরি
$ gpg --verify report.pdf.asc report.pdf  # যাচাই

$ gpg -e -r karim@example.com secret.txt  # karim-এর জন্য এনক্রিপ্ট → secret.txt.gpg
$ gpg -d secret.txt.gpg > secret.txt      # ডিক্রিপ্ট
$ gpg -c backup.tar                       # পাসওয়ার্ড দিয়ে symmetric এনক্রিপ্ট
```

```
$ gpg --gen-revoke rahim@example.com > revoke.asc   # key হারালে বাতিলের সার্টিফিকেট (আগেই বানিয়ে রাখো)
```

> 📝 **মনে রাখো:** প্যাকেজ ম্যানেজারও ঠিক এই কাজটাই প্রতিবার নিজে নিজে করে — `apt` রিপোজিটরির `Release` ফাইলের GPG স্বাক্ষর যাচাই করে (key থাকে `/etc/apt/keyrings/` বা `/usr/share/keyrings/`-এ), আর `dnf` / `rpm` প্রতিটা RPM-এর স্বাক্ষর (`gpgcheck=1`)। তাই কোনো রিপোতে `gpgcheck=0` বা `[trusted=yes]` দেখলে সতর্ক হও (অধ্যায় ৩০)।

৪৫.১৫ Lynis — পুরো সিস্টেমের নিরাপত্তা অডিট

এতক্ষণ যা শিখলাম, সব হাতে মিলিয়ে দেখা কষ্টসাধ্য। **Lynis** একটা ওপেন-সোর্স অডিট টুল যা শত শত পরীক্ষা চালিয়ে বলে দেয় কোথায় দুর্বলতা আছে আর কী করা উচিত। এটা নিজে কিছু বদলায় না — শুধু রিপোর্ট দেয়।

```
$ sudo apt install lynis          # Debian / Ubuntu
$ sudo dnf install lynis          # Fedora; RHEL-এ EPEL থেকে

$ sudo lynis audit system
[ Lynis 3.x ]
...
[+] Boot and services
------------------------------------
  - Service Manager                                           [ systemd ]
  - Checking UEFI boot                                        [ ENABLED ]
  - Checking Secure Boot                                      [ ENABLED ]
...
[+] SSH Support
------------------------------------
  - Checking running SSH daemon                               [ FOUND ]
    - OpenSSH option: PermitRootLogin                         [ OK ]
    - OpenSSH option: PasswordAuthentication                  [ OK ]
    - OpenSSH option: AllowTcpForwarding                      [ OK ]
...
  -[ Lynis 3.x Results ]-

  Warnings (1):
  ----------------------------
  ! Found one or more vulnerable packages. [PKGS-7392]
      https://cisofy.com/lynis/controls/PKGS-7392/

  Suggestions (34):
  ----------------------------
  * Install a PAM module for password strength testing like pam_cracklib or
pam_passwdqc [AUTH-9262]
  * Consider hardening SSH configuration [SSH-7408]
    - Details  : MaxSessions (10 --> 2)
  ...

  Lynis security scan details:
```

```
  Hardening index : 67 [#############       ]
  Tests performed : 262
  Plugins enabled : 0

  - Test and debug information      : /var/log/lynis.log
  - Report data                     : /var/log/lynis-report.dat
```

কীভাবে পড়বে:

- **Warnings** — জরুরি, আগে ঠিক করো।
- **Suggestions** — উন্নতির পরামর্শ; প্রতিটা অন্ধভাবে মানতে হবে না, তোমার সিস্টেমের প্রয়োজন বুঝে সিদ্ধান্ত নাও।
- **Hardening index** — ০–১০০, একটা মোটামুটি নির্দেশক। লক্ষ্য হলো সময়ের সাথে বাড়ানো, ১০০ পাওয়া না।
- বন্ধনীর কোড (যেমন `[SSH-7408]`) হলো test ID।

কমান্ড	কাজ
`lynis audit system`	পূর্ণ অডিট
`lynis audit system --quick`	মাঝে থামবে না
`lynis audit system --cronjob`	cron-এর জন্য (রঙ ও বিরতি ছাড়া)
`lynis show details SSH-7408`	একটা টেস্টের বিস্তারিত
`lynis show commands`	সব কমান্ড
`lynis show version`	ভার্সন
`lynis update info`	ভার্সন হালনাগাদ কি না

> 💡 **টিপস:** distro-র রিপোতে Lynis প্রায়ই পুরোনো ভার্সন থাকে। সর্বশেষ ভার্সনের জন্য CISOfy-র অফিসিয়াল রিপো বা GitHub ব্যবহার করা যায়। কাজের ধারা: অডিট → কয়েকটা জিনিস ঠিক করো → আবার অডিট → hardening index তুলনা।

৪৫.১৬ আরও কিছু হার্ডেনিং কৌশল (সংক্ষেপে)

এগুলোর বিস্তারিত অন্য অধ্যায়ে আছে, কিন্তু একটা হার্ডেনিং চেকলিস্টে অবশ্যই থাকা উচিত:

কার্নেল প্যারামিটার (sysctl) — `/etc/sysctl.d/99-hardening.conf` (অধ্যায় ৪৭):

```
net.ipv4.tcp_syncookies = 1              # SYN flood প্রতিরোধ
net.ipv4.conf.all.rp_filter = 1          # spoofed সোর্স IP বাদ
net.ipv4.conf.all.accept_redirects = 0   # ICMP redirect গ্রহণ না
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1
```

```
kernel.kptr_restrict = 2                  # কার্নেল pointer লুকানো
kernel.dmesg_restrict = 1                 # সাধারণ ইউজার dmesg পড়তে পারবে না
fs.protected_symlinks = 1
fs.protected_hardlinks = 1
```

```
$ sudo sysctl --system      # সব .conf ফাইল লোড
```

মাউন্ট অপশন — `/tmp` বা `/dev/shm`-এ `nodev,nosuid,noexec` দিলে সেখান থেকে প্রোগ্রাম চালানো বা SUID কাজে লাগানো কঠিন হয় (অধ্যায় ৩২)।

SUID ফাইল অডিট (অধ্যায় ১৭):

```
$ sudo find / -xdev -perm -4000 -type f 2>/dev/null
```

অন্যান্য:

- ফায়ারওয়াল ডিফল্ট "deny incoming" (অধ্যায় ৩৯)
- `sudo` কনফিগ সীমিত রাখা, `NOPASSWD` এড়ানো (অধ্যায় ১৮)
- ডিস্ক এনক্রিপশন (LUKS) — ল্যাপটপ চুরি হলে ডেটা রক্ষা
- UEFI Secure Boot ও BIOS/GRUB পাসওয়ার্ড — শারীরিক অ্যাক্সেস থেকে সুরক্ষা (অধ্যায় ২৭)
- নিয়মিত, পরীক্ষিত, অফলাইন ব্যাকআপ — ransomware-এর শেষ প্রতিরক্ষা (অধ্যায় ৩৫)
- কেন্দ্রীয় লগ সংগ্রহ — হ্যাকার লোকাল লগ মুছে দিলেও কপি থাকবে (অধ্যায় ২৮)
- প্রতিষ্ঠানে বড় পরিসরে: **CIS Benchmark** ও **OpenSCAP** (`oscap`) দিয়ে মানদণ্ড অনুযায়ী স্বয়ংক্রিয় যাচাই

নিরাপত্তা সংক্রান্ত দরকারি শর্টকাট

শর্টকাট / কৌশল	কাজ
`Super+L` (GNOME/KDE)	স্ক্রিন লক
`Ctrl+Alt+Delete`	systemd-এ ডিফল্টে রিবুট! সার্ভারে বন্ধ করতে: `sudo systemctl mask ctrl-alt-del.target`
`Enter` `~.`	আটকে যাওয়া SSH সেশন বন্ধ
কমান্ডের আগে একটা Space	`HISTCONTROL=ignorespace` থাকলে ঐ কমান্ড history-তে যায় না (পাসওয়ার্ডসহ কমান্ডের জন্য; অধ্যায় ৬)
`Ctrl+D` / `exit`	কাজ শেষে root শেল সঙ্গে সঙ্গে ছেড়ে দাও
`less`-এ `/denied` তারপর `n`	বড় লগে denial খোঁজা ও পরেরটায় যাওয়া
`journalctl -f` + `Ctrl+C`	লাইভ লগ দেখা ও বন্ধ

হাতে-কলমে অনুশীলন

> ⚠️ এই ল্যাবগুলো একটা **ভার্চুয়াল মেশিনে** (VirtualBox/KVM, অধ্যায় ৪৮) করো, যার snapshot নেওয়া আছে। আসল সার্ভারে প্রথমবার পরীক্ষা কোরো না।

ল্যাব ১: আপডেট স্বয়ংক্রিয় করা (Ubuntu/Debian)

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install -y unattended-upgrades
$ sudo dpkg-reconfigure -plow unattended-upgrades     # "Yes"
$ cat /etc/apt/apt.conf.d/20auto-upgrades
$ sudo unattended-upgrade --dry-run --debug 2>&1 | tail -20
$ systemctl list-timers 'apt-daily*'
$ ls /var/run/reboot-required 2>/dev/null && echo "রিবুট দরকার" || echo "রিবুট লাগবে না"
```

ল্যাব ২: পাসওয়ার্ড নীতি

```
$ sudo apt install -y libpam-pwquality
$ echo -e "minlen = 12\nminclass = 3" | sudo tee
/etc/security/pwquality.conf.d/lab.conf
$ sudo useradd -m testuser
$ sudo passwd testuser          # "abc" দিয়ে চেষ্টা — BAD PASSWORD আসবে (root হিসেবে শুধু
সতর্কতা দেখাবে)
$ sudo chage -M 90 -W 7 testuser
$ sudo chage -l testuser
```

> 💡 root হিসেবে `passwd` চালালে দুর্বল পাসওয়ার্ডে সতর্কতা দেখিয়েও মেনে নেয় (`enforce_for_root` না দিলে)। আসল প্রভাব দেখতে `su - testuser` হয়ে `passwd` চালাও।

ল্যাব ৩: SSH হার্ডেনিং

```
# ১. নিজের key আছে কি না নিশ্চিত হও (ক্লায়েন্ট থেকে ssh-copy-id আগে করো)
$ sudo groupadd sshusers
$ sudo usermod -aG sshusers $USER

# ২. কনফিগ
$ sudo tee /etc/ssh/sshd_config.d/00-hardening.conf >/dev/null <<'CONF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowGroups sshusers
CONF
```

```
# ৩. যাচাই ও রিলোড
$ sudo sshd -t && sudo sshd -T | grep -Ei 'permitroot|passwordauth|allowgroups'
$ sudo systemctl reload ssh

# ৪. পুরোনো সেশন খোলা রেখে নতুন টার্মিনাল থেকে লগইন টেস্ট করো
#    পাসওয়ার্ড দিয়ে চেষ্টা করলে: Permission denied (publickey).
```

ল্যাব ৪: fail2ban

```
$ sudo apt install -y fail2ban
$ sudo tee /etc/fail2ban/jail.local >/dev/null <<'CONF'
[DEFAULT]
bantime = 10m
findtime = 10m
maxretry = 3
backend = systemd

[sshd]
enabled = true
CONF
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status sshd
# অন্য একটা VM থেকে ইচ্ছে করে ৩–৪ বার ভুল ইউজারনেমে ssh করো, তারপর:
$ sudo fail2ban-client status sshd        # Banned IP list-এ ঐ IP
$ sudo fail2ban-client set sshd unbanip <ঐ-IP>
```

ল্যাব ৫: পোর্ট ও সার্ভিস অডিট

```
$ sudo ss -tulpn
$ systemctl list-unit-files --type=service --state=enabled
# প্রতিটা listening পোর্টের পাশে লিখে ফেলো: দরকার? বাইরে থেকে দরকার?
$ sudo systemctl disable --now cups.service cups.socket 2>/dev/null
$ sudo ss -tulpn          # আবার দেখো, কী কমল
```

ল্যাব ৬: SELinux (Fedora / RHEL / Rocky VM)

```
$ getenforce
$ sudo dnf install -y httpd policycoreutils-python-utils setroubleshoot-server
$ sudo systemctl enable --now httpd
$ echo "Hello SELinux" > ~/index.html
$ sudo mv ~/index.html /var/www/html/
$ curl -s localhost | head -3          # 403 Forbidden
$ ls -Z /var/www/html/index.html       # user_home_t
$ sudo ausearch -m AVC -ts recent | audit2why
$ sudo restorecon -v /var/www/html/index.html
$ curl -s localhost                    # Hello SELinux

# নিজের ডিরেক্টরি
$ sudo mkdir -p /srv/mysite && echo "srv site" | sudo tee /srv/mysite/index.html
$ ls -Z /srv/mysite/index.html
$ sudo semanage fcontext -a -t httpd_sys_content_t "/srv/mysite(/.*)?"
```

```
$ sudo restorecon -Rv /srv/mysite
$ getsebool httpd_can_network_connect
```

ল্যাব ৭: AppArmor (Ubuntu VM)

```
$ sudo aa-status | head -5
$ sudo apt install -y apparmor-utils
$ sudo aa-complain /usr/sbin/tcpdump 2>/dev/null || echo "tcpdump profile নেই"
$ sudo aa-status | grep -A3 complain
$ sudo aa-enforce /usr/sbin/tcpdump
```

ল্যাব ৮: auditd

```
$ sudo apt install -y auditd
$ echo "-w /etc/passwd -p wa -k identity" | sudo tee /etc/audit/rules.d/50-lab.rules
$ sudo augenrules --load
$ sudo auditctl -l
$ sudo useradd labuser2
$ sudo ausearch -k identity -i | tail -20     # auid তোমার নাম দেখাবে
$ sudo aureport --summary
```

ল্যাব ৯: AIDE, rkhunter, Lynis

```
$ sudo apt install -y aide rkhunter lynis
$ sudo aideinit                                  # সময় লাগবে
$ sudo touch /usr/local/bin/suspicious && sudo chmod 4755 /usr/local/bin/suspicious
$ sudo aide --config /etc/aide/aide.conf --check | head -30   # /usr/local নজরে থাকলে
নতুন SUID ফাইলটা "Added entries"-এ আসবে
$ sudo rm /usr/local/bin/suspicious

$ sudo rkhunter --update ; sudo rkhunter --propupd
$ sudo rkhunter --check --sk --rwo

$ sudo lynis audit system --quick | tail -30   # hardening index লিখে রাখো
```

ল্যাব ১০: ডাউনলোড যাচাই

```
$ mkdir ~/verify && cd ~/verify
$ echo "important data" > file.txt
$ sha256sum file.txt > SHA256SUMS
$ sha256sum -c SHA256SUMS                  # file.txt: OK
$ echo "tampered" >> file.txt
$ sha256sum -c SHA256SUMS                  # file.txt: FAILED

$ gpg --quick-generate-key "Lab User <lab@example.com>" default default 1y
$ gpg --detach-sign --armor SHA256SUMS
$ gpg --verify SHA256SUMS.asc SHA256SUMS   # Good signature
$ echo "x" >> SHA256SUMS
$ gpg --verify SHA256SUMS.asc SHA256SUMS   # BAD signature
```

সারসংক্ষেপ

বিষয়	মূল কমান্ড / ফাইল	এক লাইনে
আপডেট (Debian/Ubuntu)	`apt upgrade`, `unattended-upgrades`, `/etc/apt/apt.conf.d/50unattended-upgrades`	সিকিউরিটি আপডেট স্বয়ংক্রিয়
আপডেট (RHEL/Fedora)	`dnf upgrade --security`, `dnf-automatic.timer`, `needs-restarting -r`	একই কাজ
পাসওয়ার্ড নীতি	`/etc/security/pwquality.conf`, `chage -l/-M/-W`	শক্তি ও মেয়াদ
PAM	`/etc/pam.d/`, `auth/account/password/session`, `faillock`	অথেন্টিকেশনের সুইচবোর্ড
SSH	`/etc/ssh/sshd_config.d/00-*.conf`, `sshd -t`, `sshd -T`	key-only, root বন্ধ
fail2ban	`jail.local`, `fail2ban-client status sshd`, `unbanip`	brute-force ব্যান
সার্ভিস	`systemctl list-unit-files --state=enabled`, `disable --now`, `mask`	যা লাগে না তা বন্ধ
পোর্ট	`ss -tulpn`, `lsof -i`, `nmap -sV`	কোন দরজা খোলা
SELinux	`getenforce`, `sestatus`, `setenforce 0/1`, `ls -Z`, `restorecon -Rv`, `semanage fcontext/port/boolean`, `setsebool -P`, `ausearch -m AVC`, `audit2why`, `audit2allow`	label-ভিত্তিক MAC
AppArmor	`aa-status`, `aa-enforce`, `aa-complain`, `apparmor_parser -r`, `aa-genprof`, `aa-logprof`	path-ভিত্তিক MAC
auditd	`auditctl -w ... -p wa -k`, `/etc/audit/rules.d/`, `augenrules --load`, `ausearch -k -i`, `aureport`	কে কী করল
AIDE	`aide --init`, `aide --check`, `aide --update`, `aideinit`	ফাইল বদল ধরা
rkhunter	`rkhunter --update`, `--propupd`, `--check --sk --rwo`	rootkit খোঁজা
checksum	`sha256sum`, `sha256sum -c SHA256SUMS --ignore-missing`	ফাইল অক্ষত কি না
GPG	`gpg --verify`, `--recv-keys`, `--fingerprint`, `--detach-sign`, `-e -r`, `-c`	উৎস আসল কি না
Lynis	`lynis audit system`, `lynis show details <ID>`	সামগ্রিক অডিট

নিজেকে যাচাই করো

1. "Defense in depth" বলতে কী বোঝায়? অন্তত চারটা স্তরের নাম ও প্রতিটার একটা করে টুল বলো।
2. Ubuntu-তে `unattended-upgrades` কোন দুটো কনফিগ ফাইল ব্যবহার করে, আর প্রতিটার কাজ কী?
3. PAM-এ `required` আর `requisite`-এর মধ্যে পার্থক্য কী?
4. তুমি `/etc/ssh/sshd_config.d/99-my.conf`-এ `PasswordAuthentication no` লিখেছ, কিন্তু তবু পাসওয়ার্ড দিয়ে ঢোকা যাচ্ছে। সম্ভাব্য কারণ কী, আর কীভাবে নিশ্চিত হবে?
5. fail2ban-এ `findtime`, `maxretry`, `bantime` কীভাবে একসাথে কাজ করে?
6. `ss -tulpn` আউটপুটে `0.0.0.0:6379` আর `127.0.0.1:3306` — কোনটা বেশি চিন্তার এবং কেন?
7. DAC ও MAC-এর মূল পার্থক্য কী? root হিসেবে চলা একটা হ্যাক হওয়া প্রসেসের ক্ষেত্রে MAC কেন গুরুত্বপূর্ণ?
8. Apache একটা ফাইলে 403 দিচ্ছে, পারমিশন ঠিক আছে, SELinux enforcing। তুমি কোন ক্রমে সমস্যা খুঁজবে ও ঠিক করবে?
9. `chcon` আর `semanage fcontext` + `restorecon`-এর পার্থক্য কী?
10. শুধু `sha256sum` মিলে গেলেই কেন একটা ডাউনলোড নিরাপদ বলা যায় না? GPG কোন সমস্যার সমাধান করে?

<details>
<summary>▼ উত্তর দেখো</summary>

1. একাধিক স্তরে নিরাপত্তা সাজানো, যাতে একটা স্তর ভাঙলেও পরেরটা আটকায়। যেমন: নেটওয়ার্ক — firewall/fail2ban; সার্ভিস — অপ্রয়োজনীয় সার্ভিস বন্ধ, SSH hardening; অথেন্টিকেশন — PAM/pwquality/SSH key; অ্যাক্সেস কন্ট্রোল — permission + SELinux/AppArmor; ডেটা/কার্নেল — আপডেট, LUKS, sysctl; নজরদারি — auditd/AIDE/Lynis।
2. `/etc/apt/apt.conf.d/20auto-upgrades` — স্বয়ংক্রিয় চালু/বন্ধ ও কত দিন পরপর (`Update-Package-Lists`, `Unattended-Upgrade`); `/etc/apt/apt.conf.d/50unattended-upgrades` — কোন origin থেকে আপডেট নেবে, blacklist, স্বয়ংক্রিয় রিবুট, ইমেইল ইত্যাদি।
3. দুটোতেই মডিউল পাস করা বাধ্যতামূলক। `required` ফেল করলেও বাকি মডিউলগুলো চলে (আক্রমণকারী বুঝতে পারে না ঠিক কোথায় ফেল হলো), শেষে ফল ব্যর্থ। `requisite` ফেল করলে সঙ্গে সঙ্গে থেমে ব্যর্থতা ফেরত দেয়।
4. sshd-তে প্রথম পাওয়া মান কার্যকর হয়, আর drop-in ফাইলগুলো বর্ণানুক্রমে পড়া হয়। হয়তো আগের কোনো ফাইল (যেমন `50-cloud-init.conf`) বা মূল ফাইলের Include-এর আগের অংশে `PasswordAuthentication yes` আছে। `sudo sshd -T | grep passwordauthentication` দিয়ে কার্যকর মান দেখো, `grep -ri passwordauth /etc/ssh/` দিয়ে উৎস খোঁজো, নিজের ফাইলের নাম `00-hardening.conf` করো, তারপর `sshd -t` ও reload। আর `KbdInteractiveAuthentication no`-ও দিয়েছ কি না দেখো।
5. `findtime` সময়সীমার মধ্যে কোনো IP থেকে `maxretry` সংখ্যক ব্যর্থতা হলে সেই IP `bantime` সময়ের জন্য firewall-এ ব্লক হয়। যেমন ১০ মিনিটে ৩ বার → ১ ঘণ্টা ব্যান।
6. `0.0.0.0:6379` — Redis সব ইন্টারফেসে খোলা, firewall না থাকলে ইন্টারনেট থেকে ঢোকা যায়; এটা বেশি চিন্তার। `127.0.0.1:3306` শুধু লোকাল মেশিন থেকে অ্যাক্সেসযোগ্য।

</details>

7. DAC-এ ফাইলের মালিক নিজের ইচ্ছামতো অনুমতি দেয় এবং root প্রায় সব বাধা পেরোতে পারে। MAC-এ সিস্টেম-ব্যাপী নীতি ঠিক করে প্রোগ্রাম কী করতে পারবে, যা মালিক বা root সাধারণ উপায়ে এড়াতে পারে না। তাই root হিসেবে চলা হ্যাক হওয়া ওয়েব সার্ভারও তার domain-এর বাইরে (যেমন `/etc/shadow`) যেতে পারে না।
8. (ক) `ausearch -m AVC -ts recent` দিয়ে denial দেখা ও `audit2why` দিয়ে কারণ বোঝা; (খ) `ls -Z` দিয়ে label দেখা — ভুল হলে `restorecon -Rv` (অ-ডিফল্ট পাথে `semanage fcontext -a` তারপর `restorecon`); (গ) boolean দরকার হলে `setsebool -P`; (ঘ) অ-ডিফল্ট পোর্ট হলে `semanage port -a`; (ঙ) কিছুই না মিললে শেষ উপায় হিসেবে `audit2allow -M` দিয়ে মডিউল, আগে `.te` ফাইল পড়ে।
9. `chcon` সরাসরি ফাইলের label বদলায় — অস্থায়ী, পরের `restorecon` বা পূর্ণ relabel-এ মুছে যায়। `semanage fcontext` নীতিতে স্থায়ী নিয়ম যোগ করে, আর `restorecon` সেই নিয়ম অনুযায়ী label প্রয়োগ করে — এটাই সঠিক পদ্ধতি।
10. আক্রমণকারী ISO বদলাতে পারলে পাশের `SHA256SUMS` ফাইলটাও বদলে দিতে পারে; checksum শুধু ফাইল অক্ষত কি না বলে, উৎস আসল কি না বলে না। GPG স্বাক্ষর প্রমাণ করে যে `SHA256SUMS` প্রকাশকের private key দিয়েই সাইন করা (যদি তুমি public key-এর fingerprint অফিসিয়াল সূত্রের সাথে মিলিয়ে থাকো) — অর্থাৎ authenticity নিশ্চিত করে।

অধ্যায় ৪৬: পারফরম্যান্স মনিটরিং ও ট্রাবলশুটিং

> "সার্ভার স্লো হয়ে গেছে!" এই বাক্যটা একজন লিনাক্স অ্যাডমিন জীবনে হাজারবার শুনবে। নতুনরা এটা শুনে ঘাবড়ে যায়, আন্দাজে সার্ভার রিস্টার্ট করে। অভিজ্ঞরা একটা নির্দিষ্ট পদ্ধতি মেনে কয়েক মিনিটের মধ্যে সমস্যার উৎস খুঁজে বের করে। এই অধ্যায় তোমাকে সেই পদ্ধতিটা শেখাবে।

এই অধ্যায়ে যা শিখবে

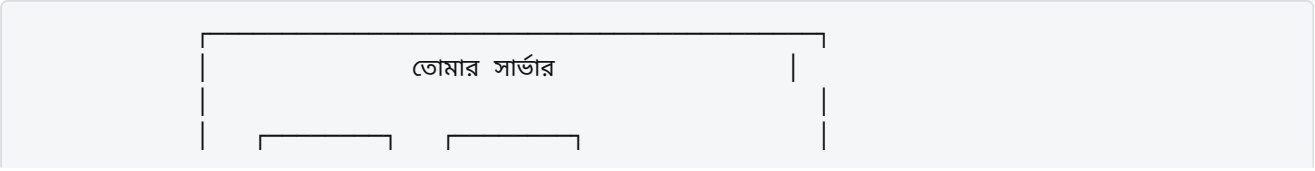
- পারফরম্যান্স সমস্যা খোঁজার বৈজ্ঞানিক উপায়: **USE পদ্ধতি** (Utilization, Saturation, Errors)
- `uptime` আর **load average** আসলে কী মাপে এবং সংখ্যাগুলো কীভাবে পড়তে হয়
- CPU দেখা: `top`, `mpstat`, `pidstat`, আর `us`, `sy`, `wa`, `st` এসবের মানে
- Memory বোঝা: `free -h`, **buff/cache** আর **available** এর রহস্য, `/proc/meminfo`, `vmstat`, swap, **OOM killer**
- Disk I/O: `iostat`, `iotop`, আর `%util` ও `await` এর মানে
- Network: `iftop`, `nethogs`, `sar -n DEV`
- **sysstat/sar** দিয়ে অতীতের পারফরম্যান্স ডেটা দেখা ("গত রাত ৩টায় কী হয়েছিল?")
- `lsof` দিয়ে খোলা ফাইল ও পোর্ট খোঁজা
- `strace` ও `ltrace` দিয়ে একটা প্রোগ্রামের ভেতরে উঁকি দেওয়া
- `perf` দিয়ে CPU প্রোফাইলিং-এর পরিচিতি
- আধুনিক ড্যাশবোর্ড টুল: `htop`, `btop`, `glances`
- **PSI** (Pressure Stall Information): আধুনিক কার্নেলের "চাপ মাপার যন্ত্র"
- "সার্ভার স্লো" সমস্যার ধাপে ধাপে ডায়াগনোসিস চেকলিস্ট
- সাধারণ সমস্যা ও সমাধান: disk full, high load, port already in use, permission denied

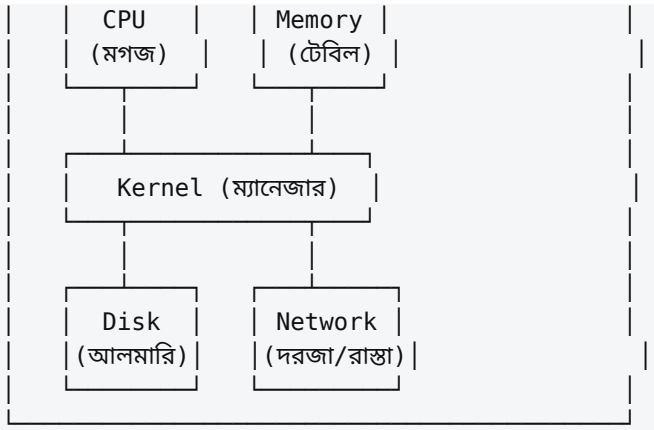
---

৪৬.১ পারফরম্যান্স ট্রাবলশুটিং কেন একটা "পদ্ধতি", আন্দাজ না

ধরো তুমি ডাক্তারের কাছে গেলে, বললে "শরীর খারাপ লাগছে"। ভালো ডাক্তার সাথে সাথে ওষুধ লিখে দেবে না। আগে জ্বর মাপবে, প্রেশার দেখবে, পালস গুনবে, দরকার হলে রক্ত পরীক্ষা করাবে। প্রতিটা অঙ্গ একটা একটা করে চেক করবে, তারপর রোগ ধরবে।

লিনাক্স সার্ভারের ক্ষেত্রেও একই কথা। কম্পিউটারের "অঙ্গ" মূলত চারটা:





একটা উপমা দিই। একটা রেস্টুরেন্টের রান্নাঘর ভাবো:

কম্পিউটারের অংশ	রান্নাঘরের উপমা	"স্লো" হলে কেমন দেখায়
CPU	রাঁধুনি (cook)	রাঁধুনি সবাই ব্যস্ত, অর্ডার জমে আছে
Memory (RAM)	রান্নার টেবিল (কাজের জায়গা)	টেবিল ভরা, বারবার জিনিস আলমারিতে তুলে রাখতে হচ্ছে
Disk	স্টোররুম/আলমারি	স্টোররুম থেকে মাল আনতে অনেক সময় লাগছে
Network	ওয়েটার আর দরজা	ওয়েটার কম, দরজায় ভিড়

গ্রাহক যখন বলে "খাবার আসতে দেরি হচ্ছে", তখন তুমি জানো না সমস্যা রাঁধুনিতে, টেবিলে, স্টোররুমে নাকি ওয়েটারে। তাই প্রতিটা চেক করতে হবে। এই চেক করার সবচেয়ে বিখ্যাত পদ্ধতির নাম **USE Method**।

৪৬.২ USE পদ্ধতি

USE পদ্ধতি তৈরি করেছেন পারফরম্যান্স বিশেষজ্ঞ **Brendan Gregg**। মূল কথা খুব সহজ:

> **প্রতিটা resource (সম্পদ) এর জন্য তিনটা জিনিস চেক করো: Utilization, Saturation, Errors।**

অক্ষর	মানে	সহজ বাংলায়	রান্নাঘরের উপমা
Utilization	resource কত শতাংশ সময় ব্যস্ত ছিল	কতটা কাজে লাগছে	রাঁধুনি দিনের কত % সময় রান্না করছে
Saturation	resource এর বাইরে কত কাজ অপেক্ষায় (queue)	কত কাজ লাইনে দাঁড়িয়ে	কতগুলো অর্ডার স্লিপ ঝুলছে, এখনো শুরু হয়নি
Errors	error কয়টা হচ্ছে	কাজ কতবার ভুল/ব্যর্থ হলো	কতগুলো ডিশ পুড়ে গেল, ফেরত এলো

📝 **মনে রাখো:** Utilization ১০০% হওয়া মানেই সমস্যা না। রাঁধুনি সারাক্ষণ ব্যস্ত, কিন্তু কোনো অর্ডার অপেক্ষায় নেই, তাহলে সব ঠিক আছে। আসল বিপদের সংকেত হলো **Saturation**, মানে যখন কাজ লাইনে দাঁড়িয়ে থাকে।

USE চেকলিস্ট: কোন resource এর জন্য কোন টুল

Resource	Utilization	Saturation	Errors
CPU	`mpstat -P ALL 1`, `top` এর `%Cpu`	load average > CPU সংখ্যা, `vmstat` এর `r` কলাম, `/proc/pressure/cpu`	`dmesg` (machine check error)
Memory	`free -h` (used vs available)	swap ব্যবহার, `vmstat` এর `si`/`so`, OOM, `/proc/pressure/memory`	`dmesg` এ "Out of memory", ECC error
Disk I/O	`iostat -x` এর `%util`	`iostat` এর `aqu-sz`, `await`, `vmstat` এর `b`	`dmesg` এ I/O error, `smartctl`
Disk space	`df -h`, `df -i`	(পূর্ণ হলে লেখাই যায় না)	"No space left on device"
Network	`sar -n DEV 1`, `iftop` (লিঙ্ক স্পিডের তুলনায়)	`ss -s`, drops, retransmits (`sar -n ETCP 1`)	`ip -s link` এর errors/dropped

এই টেবিলটা মাথায় রাখলে তুমি কখনো "কোথা থেকে শুরু করব" ভেবে আটকে থাকবে না। এখন প্রতিটা টুল বিস্তারিত দেখি।

৪৬.৩ `uptime` এবং Load Average: সিস্টেমের "জ্বর মাপার থার্মোমিটার"

`uptime` কী করে

সিস্টেম কতক্ষণ চালু আছে, কতজন ইউজার লগইন করে আছে আর load average কত, এক লাইনে দেখায়।

```
$ uptime
 14:32:10 up 12 days,  3:45,  2 users,  load average: 0.52, 0.78, 1.05
```

অংশ	মানে
`14:32:10`	এখনকার সময়
`up 12 days, 3:45`	১২ দিন ৩ ঘণ্টা ৪৫ মিনিট ধরে চালু (শেষ reboot এর পর থেকে)
`2 users`	লগইন করা session সংখ্যা
`load average: 0.52, 0.78, 1.05`	গত **১ মিনিট, ৫ মিনিট, ১৫ মিনিটের** গড় load

দরকারি অপশন:

অপশন	কাজ
`uptime -p`	সুন্দর করে দেখায়: `up 1 week, 5 days, 3 hours, 45 minutes`

অপশন	কাজ
`uptime -s`	কবে থেকে চালু: `2026-10-31 10:47:02`

Load average আসলে কী?

এটা লিনাক্সের সবচেয়ে ভুল বোঝা সংখ্যাগুলোর একটা। অনেকে ভাবে load মানে "CPU কত % ব্যবহার হচ্ছে"। **না।**

Load average = গড়ে কতগুলো task (process/thread) একই সময়ে চলছে বা চলার জন্য অপেক্ষা করছে।

লিনাক্সে এর মধ্যে দুই ধরনের task গোনা হয়:

1. **R (Running/Runnable)**: CPU তে চলছে, বা CPU পাওয়ার জন্য লাইনে দাঁড়িয়ে।
2. **D (Uninterruptible sleep)**: সাধারণত disk বা NFS এর মতো I/O শেষ হওয়ার অপেক্ষায় আটকে আছে।

⚠️ **সতর্কতা:** দ্বিতীয় পয়েন্টটা লিনাক্সের বিশেষত্ব। অন্য অনেক Unix (যেমন FreeBSD) শুধু CPU এর task গোনে। তাই লিনাক্সে load বেশি মানেই CPU ব্যস্ত না; হতে পারে disk স্লো, তাই অনেক process D state এ আটকে আছে। High load দেখলে সবসময় CPU **আর** I/O দুটোই চেক করবে।

উপমা: ব্যাংকের কাউন্টার

ধরো একটা ব্যাংকে ৪টা কাউন্টার (৪টা CPU core)।

```
load = 2.0  (৪ কাউন্টারে)        load = 4.0                load = 8.0
┌──┐┌──┐┌──┐┌──┐                ┌──┐┌──┐┌──┐┌──┐          ┌──┐┌──┐┌──┐┌──┐
│😀││😀│└──┘└──┘                │😀││😀││😀││😀│          │😀││😀││😀││😀│
└──┘└──┘                       └──┘└──┘└──┘└──┘          └──┘└──┘└──┘└──┘
২টা কাউন্টার খালি             সব কাউন্টার ব্যস্ত,      😐😐😐😐 লাইনে ৪ জন
(৫০% ব্যস্ত, আরামদায়ক)       কেউ লাইনে নেই (১০০%)      অপেক্ষা করছে (overload)
```

তাই load এর সংখ্যাটা **CPU core সংখ্যার সাথে তুলনা করে** পড়তে হয়:

```
$ nproc
4
```

load ÷ CPU সংখ্যা	অবস্থা
< 0.7	আরামদায়ক
≈ 1.0	পুরোপুরি ব্যস্ত, তবে লাইন নেই
> 1.0	কাজ লাইনে দাঁড়িয়ে (saturation)। দীর্ঘক্ষণ থাকলে তদন্ত করো
> 2.0 বা বেশি	স্পষ্ট overload, ইউজার স্লো টের পাবে

💡 **টিপস:** এগুলো শক্ত নিয়ম না, শুধু আন্দাজ। কিছু সার্ভার (যেমন batch job চালানো মেশিন) ইচ্ছা করেই সবসময় ১০০% চলে।

তিনটা সংখ্যার ট্রেন্ড পড়া

উদাহরণ	ব্যাখ্যা
`load average: 8.20, 4.10, 1.50`	load **বাড়ছে**। সমস্যা এই মাত্র শুরু হয়েছে, এখনই দেখো
`load average: 1.20, 4.50, 8.30`	load **কমছে**। সমস্যা আগে ছিল, এখন সেরে যাচ্ছে
`load average: 4.00, 4.00, 4.00`	স্থির load। এটাই এই সার্ভারের স্বাভাবিক অবস্থা হতে পারে

ভেতরে কীভাবে কাজ করে

Kernel প্রতি ৫ সেকেন্ডে R+D state এর task গোনে এবং একটা **exponentially-damped moving average** হিসাব করে। সহজ ভাষায়: নতুন মাপের গুরুত্ব বেশি, পুরনো মাপের প্রভাব ধীরে ধীরে মিলিয়ে যায়। এজন্য "১ মিনিটের গড়" আসলে ঠিক গত ৬০ সেকেন্ডের সরল গড় না, বরং ঝুঁকে থাকা একটা গড়। Kernel এই তিনটা সংখ্যা `/proc/loadavg` ফাইলে রাখে:

```
$ cat /proc/loadavg
0.52 0.78 1.05 2/1234 98765
```

শেষের `2/1234` মানে এই মুহূর্তে ২টা task runnable, মোট ১২৩৪টা task (thread) আছে। আর `98765` হলো সর্বশেষ তৈরি হওয়া PID। (`/proc` নিয়ে বিস্তারিত অধ্যায় ৪৭ এ।)

৪৬.৪ CPU মনিটরিং

CPU সময় কোথায় যায়: ভাগগুলো চেনো

যেকোনো CPU টুল (top, mpstat, vmstat, sar) CPU সময়কে কয়েকটা ভাগে দেখায়। এগুলো চিনলে অর্ধেক কাজ শেষ।

ঘর	পুরো নাম	মানে	বেশি হলে কী বুঝবে
`us`	user	সাধারণ প্রোগ্রামের নিজের কোড চালাতে সময়	অ্যাপ্লিকেশন নিজেই ভারী হিসাব করছে (যেমন PHP, Java, compression)
`sy`	system	Kernel এর কাজে সময় (system call, network stack, file system)	অনেক বেশি system call, context switch, বা kernel এর ঝামেলা
`ni`	nice	কম priority (nice মান বাড়ানো) process এর user সময়	এমনিতে সমস্যা না
`id`	idle	CPU অলস বসে ছিল	বেশি হলে ভালো
`wa`	iowait	CPU অলস, **এবং** কেউ disk I/O এর অপেক্ষায়	Disk স্লো বা overload। Disk এর দিকে তাকাও

ঘর	পুরো নাম	মানে	বেশি হলে কী বুঝবে
`hi`	hardware interrupt	হার্ডওয়্যার interrupt সামলাতে	NIC/disk এর অনেক interrupt
`si`	software interrupt	soft interrupt (প্রায়ই network packet প্রসেসিং)	অনেক network ট্রাফিক
`st`	steal	VM এর ক্ষেত্রে: hypervisor তোমার CPU সময় অন্য VM কে দিয়ে দিয়েছে	Cloud এর host overload, বা তোমার CPU credit শেষ (যেমন burstable instance)। তোমার সার্ভারের ভেতরে ঠিক করার কিছু নেই

📝 **মনে রাখো:** `wa` (iowait) আসলে "idle এর একটা ধরন"। CPU কাজ করছে না, শুধু বসে আছে কারণ disk উত্তর দেয়নি। তাই `wa` বেশি হলে CPU কেনার দরকার নেই, disk দেখো।

`top` : সবচেয়ে পরিচিত লাইভ মনিটর

`top` প্রায় সব লিনাক্সে (procps-ng প্যাকেজের অংশ হিসেবে) আগে থেকেই থাকে। Process ম্যানেজমেন্টের দিক থেকে `top` এর পরিচিতি অধ্যায় ২৪ এ আছে; এখানে আমরা পারফরম্যান্সের চোখে দেখব।

```
$ top
top - 14:35:02 up 12 days,  3:48,  2 users,  load average: 3.91, 2.15, 1.20
Tasks: 245 total,   3 running, 242 sleeping,   0 stopped,   0 zombie
%Cpu(s): 72.5 us,  8.1 sy,  0.0 ni, 12.3 id,  6.6 wa,  0.0 hi,  0.5 si,  0.0 st
MiB Mem :   7841.2 total,    312.4 free,   5120.8 used,   2408.0 buff/cache
MiB Swap:   2048.0 total,   1980.5 free,     67.5 used.   2720.4 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2314 www-data  20   0  812340 245120  18432 R  98.3   3.1  45:12.08 php-fpm8.3
   1877 mysql     20   0 2456780   1.2g  32100 S  65.1  15.7 120:33.41 mysqld
   3021 root      20   0   12345   4400   3500 R   2.0   0.1   0:00.15 top
```

উপরের "summary" অংশ লাইন ধরে

লাইন	কী বলে
১ম লাইন	`uptime` এর মতোই: সময়, uptime, ইউজার, load average
`Tasks:`	মোট process, কয়টা চলছে (R), ঘুমাচ্ছে (S), থেমে আছে (T), **zombie** (Z)
`%Cpu(s):`	উপরের টেবিলের ভাগগুলো (সব CPU এর গড়)
`MiB Mem:`	মোট, free, used, buff/cache (memory অংশে বিস্তারিত)
`MiB Swap:`	swap এর অবস্থা, আর শেষে `avail Mem` (নতুন কাজের জন্য আসলে কত memory পাওয়া যাবে)

Process টেবিলের কলাম

কলাম	মানে
`PID`	Process ID

কলাম	মানে
`USER`	কার process
`PR` / `NI`	Priority / nice মান (-20 সবচেয়ে বেশি priority, 19 সবচেয়ে কম)
`VIRT`	Virtual memory: process যত memory "ঠিকানা" রিজার্ভ করেছে (সব আসলে ব্যবহার হয় না)
`RES`	Resident: আসলে RAM এ যতটুকু আছে। **এটাই আসল memory ব্যবহার বোঝার সবচেয়ে কাজের কলাম**
`SHR`	Shared: অন্য process এর সাথে ভাগ করা (যেমন shared library)
`S`	State: R (running), S (sleeping), D (uninterruptible, প্রায়ই I/O), T (stopped), Z (zombie), I (idle kernel thread)
`%CPU`	এক CPU core এর তুলনায় শতাংশ। **multi-core এ ১০০% এর বেশি হতে পারে** (যেমন ৪ core ব্যবহার করলে ৪০০%)
`%MEM`	মোট RAM এর কত % (RES ভিত্তিক)
`TIME+`	এ পর্যন্ত মোট কত CPU সময় খেয়েছে
`COMMAND`	প্রোগ্রামের নাম

💡 **টিপস:** `%CPU` এর ১০০% এর বেশি দেখে ভয় পেয়ো না। `top` এ `Shift+I` চাপলে **Irix mode** বন্ধ হয়ে Solaris mode চালু হয়, তখন `%CPU` মোট CPU সংখ্যা দিয়ে ভাগ হয়ে দেখায় (সর্বোচ্চ ১০০%)।

`top` এর ভেতরের কীবোর্ড শর্টকাট (চলাকালীন চাপতে হয়)

কী	কাজ
`P`	CPU অনুযায়ী সাজাও (ডিফল্ট)
`M`	Memory (`%MEM`) অনুযায়ী সাজাও
`T`	`TIME+` অনুযায়ী সাজাও
`N`	PID অনুযায়ী সাজাও
`<` / `>`	সাজানোর কলাম বাঁয়ে/ডানে সরাও
`R`	সাজানোর ক্রম উল্টাও
`1`	প্রতিটা CPU core আলাদা লাইনে দেখাও (খুব কাজের!)
`t`	CPU summary লাইনের দেখানোর ধরন বদলাও (bar graph)
`m`	Memory summary এর দেখানোর ধরন বদলাও
`c`	পুরো command line দেখাও (যেমন `php artisan queue:work` বোঝা যাবে)
`H`	thread আলাদা করে দেখাও
`V`	tree/forest view (কে কার child)
`u`	নির্দিষ্ট ইউজারের process ফিল্টার করো

কী	কাজ
`o` / `O`	কলামের মান দিয়ে ফিল্টার, যেমন `%CPU>5.0`
`k`	process কে signal পাঠাও (kill)
`r`	renice (priority বদলাও)
`d` বা `s`	রিফ্রেশ বিরতি বদলাও (ডিফল্ট ৩ সেকেন্ড)
`f`	কোন কলাম দেখাবে বেছে নাও
`E` / `e`	summary / টেবিলে memory একক বদলাও (KiB → MiB → GiB ...)
`x` / `y`	সাজানো কলাম / running task হাইলাইট
`z`	রঙিন মোড
`W`	বর্তমান সেটিং `~/.config/procps/toprc` এ সেভ করো
`h` বা `?`	হেল্প
`q`	বের হও

`top` এর কমান্ড-লাইন অপশন

অপশন	কাজ	উদাহরণ
`-d SEC`	রিফ্রেশ বিরতি	`top -d 1`
`-n N`	N বার রিফ্রেশ করে বের হও	`top -n 1`
`-b`	batch mode (স্ক্রিপ্ট বা ফাইলে আউটপুট দেওয়ার জন্য)	`top -b -n 1 > snapshot.txt`
`-p PID`	শুধু নির্দিষ্ট PID দেখাও	`top -p 1877,2314`
`-u USER`	নির্দিষ্ট ইউজার	`top -u www-data`
`-o FIELD`	কোন কলামে সাজাবে	`top -o %MEM`
`-H`	thread দেখাও	`top -H -p 1877`

💡 **টিপস:** সমস্যার মুহূর্তের একটা "ছবি" সংরক্ষণ করতে: `top -b -n 1 -o %CPU | head -20 > /tmp/top-$(date +%F-%H%M).txt`। পরে টিকিটে বা টিমকে দেখাতে কাজে লাগবে।

`htop`: top এর রঙিন বড় ভাই

`htop` আলাদা প্যাকেজ (`sudo apt install htop` / `sudo dnf install htop`; RHEL এ EPEL লাগতে পারে)। মাউস চলে, scroll করা যায়, রঙিন bar দেখায়।

কী	কাজ
`F1` / `h`	হেল্প
`F2`	Setup (কলাম, রঙ, meter)
`F3` / `/`	নাম দিয়ে খোঁজা

কী	কাজ
`F4` / `\`	ফিল্টার
`F5` / `t`	tree view
`F6` / `>`	সাজানোর কলাম বেছে নাও
`F7` / `F8`	nice কমাও/বাড়াও (priority বাড়াও/কমাও)
`F9` / `k`	kill (signal বেছে নিতে পারবে)
`F10` / `q`	বের হও
`Space`	process ট্যাগ করো (একসাথে অনেকগুলো kill করতে)
`u`	ইউজার দিয়ে ফিল্টার
`H`	ইউজার thread লুকাও/দেখাও
`K`	kernel thread লুকাও/দেখাও
`p`	প্রোগ্রামের পুরো path দেখাও/লুকাও
`l`	ওই process এর খোলা ফাইল (`lsof` লাগবে)
`s`	ওই process এ `strace` লাগাও
`M` / `P` / `T`	memory / CPU / time অনুযায়ী সাজাও

`mpstat`: প্রতিটা CPU core আলাদা করে দেখা

`top` সব CPU এর গড় দেখায়। কিন্তু ধরো ৮ core এর মেশিনে একটা single-threaded প্রোগ্রাম একটা core কে ১০০% ব্যস্ত রেখেছে। গড় দেখাবে মাত্র ১২.৫%, তুমি ভাববে সব ঠিক। এই ফাঁদ ধরতে `mpstat`।

`mpstat` আসে **sysstat** প্যাকেজে:

```
# Debian/Ubuntu
$ sudo apt install sysstat
# Fedora/RHEL
$ sudo dnf install sysstat
```

Syntax: `mpstat [options] [interval [count]]`

অপশন	কাজ
`-P ALL`	সব CPU আলাদা লাইনে
`-P 0,2`	শুধু CPU 0 আর 2
`-u`	CPU ব্যবহার (ডিফল্ট)
`-I SUM` / `-I ALL`	interrupt পরিসংখ্যান
`interval count`	প্রতি interval সেকেন্ডে, count বার

```
$ mpstat -P ALL 1 1
Linux 6.8.0-45-generic (web01)  11/12/2026  _x86_64_  (4 CPU)

02:40:11 PM  CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest
%gnice   %idle
02:40:12 PM  all   26.00    0.00    2.25    0.25    0.00    0.25    0.00    0.00
0.00   71.25
02:40:12 PM    0  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
0.00    0.00
02:40:12 PM    1    2.00    0.00    3.00    1.00    0.00    1.00    0.00    0.00
0.00   93.00
02:40:12 PM    2    1.00    0.00    4.00    0.00    0.00    0.00    0.00    0.00
0.00   95.00
02:40:12 PM    3    1.00    0.00    2.00    0.00    0.00    0.00    0.00    0.00
0.00   97.00
```

দেখো, `all` এ ৭১% idle, কিন্তু CPU 0 পুরো ১০০% `%usr` এ। মানে একটা single-threaded কাজ একটা core কে আটকে রেখেছে। এই ধরনের প্রোগ্রাম আরও CPU দিলেও দ্রুত হবে না।

`pidstat`: কোন process কী খাচ্ছে, সময়ের সাথে

এটাও sysstat এর অংশ। `top` এর মতো স্ক্রিন মুছে ফেলে না, বরং লাইন ধরে লিখে যায়, তাই পরিবর্তন দেখতে সুবিধা।

অপশন	কাজ
`-u`	CPU (ডিফল্ট)
`-r`	memory ও page fault
`-d`	disk I/O (প্রতি process কত পড়ছে/লিখছে)
`-w`	context switch
`-t`	thread সহ
`-p PID`	নির্দিষ্ট process
`-C NAME`	নাম মিলিয়ে process

```
$ pidstat 1 3
02:41:01 PM   UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
02:41:02 PM    33      2314   97.00    1.00    0.00    0.00   98.00     0  php-fpm8.3
02:41:02 PM   999      1877   12.00    3.00    0.00    1.00   15.00     2  mysqld
```

`%wait` কলাম বলে process কতক্ষণ CPU পাওয়ার জন্য লাইনে অপেক্ষা করেছে। এটা বেশি মানে CPU saturation।

```
# কোন process disk এ সবচেয়ে বেশি লিখছে?
$ pidstat -d 1 5
```

৪৬.৫ Memory মনিটরিং

`free -h`: memory এর এক নজরে হিসাব

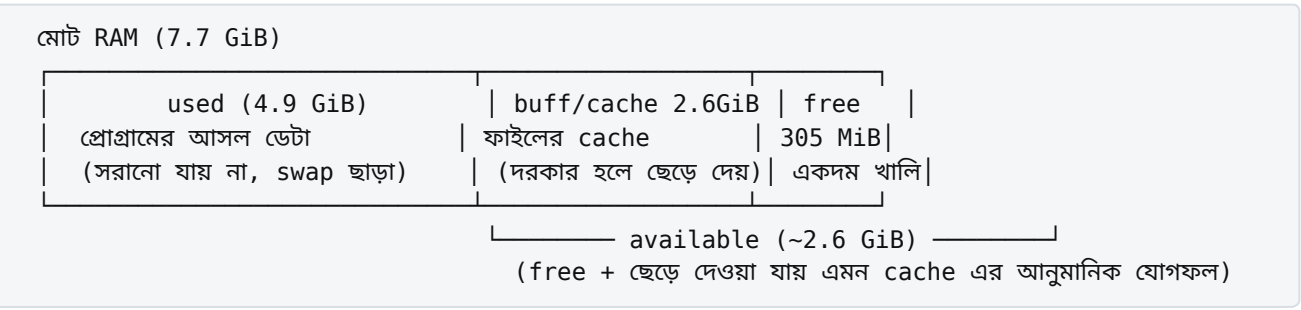
```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       4.9Gi       305Mi       180Mi       2.6Gi       2.6Gi
Swap:          2.0Gi        67Mi       1.9Gi
```

নতুনরা এখানে `free` কলামে মাত্র 305Mi দেখে আতঙ্কিত হয়: "RAM শেষ!"। আসলে না। এটা বোঝার জন্য প্রথমে জানতে হবে লিনাক্স RAM কে কীভাবে দেখে।

"Free RAM is wasted RAM": লিনাক্সের দর্শন

লিনাক্স মনে করে খালি পড়ে থাকা RAM হলো অপচয়। তাই যেটুকু RAM কোনো প্রোগ্রাম চাইছে না, সেটুকু দিয়ে kernel disk থেকে পড়া ফাইলগুলো মনে রাখে। একে বলে **page cache**। পরের বার একই ফাইল লাগলে disk এ না গিয়ে RAM থেকেই দিয়ে দেয়, যা কয়েকশো থেকে কয়েক হাজার গুণ দ্রুত।

উপমা: তোমার পড়ার টেবিল। যে বইগুলো আজ পড়েছ, সেগুলো তাকে তুলে না রেখে টেবিলেই খোলা রেখেছ, কারণ আবার লাগতে পারে। কিন্তু নতুন একটা বড় খাতা রাখার জায়গা লাগলে সাথে সাথে পুরনো বই সরিয়ে জায়গা করে দেবে। টেবিল "ভরা" দেখালেও আসলে জায়গা আছে।



প্রতিটা কলামের মানে

কলাম	মানে
`total`	মোট ব্যবহারযোগ্য RAM (kernel নিজের জন্য কিছু রেখে দেয়, তাই কেনা RAM এর চেয়ে একটু কম দেখায়)
`used`	প্রোগ্রামগুলো আসলে ব্যবহার করছে। হিসাব: `total - free - buff/cache` (procps-ng এর নতুন ভার্সনে `total - available`)
`free`	একদম অব্যবহৃত, কিছুই রাখা নেই
`shared`	মূলত `tmpfs` (যেমন `/dev/shm`, `/run`) ও shared memory
`buff/cache`	**buffers** (block device এর metadata এর cache) + **cache** (ফাইলের page cache ও reclaimable slab)
`available`	**নতুন প্রোগ্রাম swap ছাড়াই আনুমানিক কত memory পাবে**। এটাই আসল উত্তর "কত RAM বাকি আছে"

📝 **মনে রাখো:** "RAM কত বাকি?" এই প্রশ্নের উত্তর সবসময় `available` কলামে, `free` কলামে না।

⚠️ **সতর্কতা:** ইন্টারনেটে অনেক জায়গায় দেখবে `echo 3 > /proc/sys/vm/drop_caches` চালিয়ে "RAM পরিষ্কার" করতে বলছে। Production এ এটা করবে না। এতে cache মুছে যায়, ফলে পরের কিছুক্ষণ সব ফাইল আবার disk থেকে পড়তে হয় এবং সিস্টেম **আরও স্লো** হয়। এটা শুধু benchmark টেস্টের সময় কাজে লাগে।

`free` এর অপশন

অপশন	কাজ
`-h`	মানুষের পড়ার মতো একক (Mi, Gi)
`-m` / `-g`	MiB / GiB এ
`-w`	wide: `buffers` আর `cache` আলাদা কলামে
`-t`	নিচে RAM+swap এর মোট লাইন
`-s N`	প্রতি N সেকেন্ডে বারবার দেখাও
`-c N`	N বার দেখিয়ে থামো (`-s` এর সাথে)

```
$ free -h -s 2 -c 3      # প্রতি ২ সেকেন্ডে, ৩ বার
```

Swap: RAM এর "এক্সটেনশন কর্ড"

যখন RAM সত্যিই ফুরিয়ে আসে, kernel কম-ব্যবহৃত memory page গুলো disk এর একটা অংশে (swap partition বা swap file) সরিয়ে রাখে। একে **swap out** বলে। আবার দরকার হলে ফেরত আনে (**swap in**)।

- Swap **ব্যবহৃত** থাকা সমস্যা না। হয়তো অনেক আগে কিছু অলস page সরানো হয়েছিল, আর ফেরত আনার দরকার পড়েনি।
- সমস্যা হলো **সক্রিয় swapping**, মানে প্রতি সেকেন্ডে page আসা-যাওয়া। এটা `vmstat` এর `si` / `so` কলামে দেখা যায়। সক্রিয় swapping চলতে থাকলে সিস্টেম প্রায় থেমে যায়, একে বলে **thrashing**।

```
$ swapon --show
NAME      TYPE SIZE  USED PRIO
/swap.img file   2G 67.5M   -2

$ cat /proc/sys/vm/swappiness
60
```

`vm.swappiness` (0 থেকে 200, ডিফল্ট সাধারণত 60) বলে kernel কতটা আগ্রহ নিয়ে swap ব্যবহার করবে, cache ছাড়ার তুলনায়। Database সার্ভারে অনেকে এটা কমিয়ে 10 করে। (sysctl দিয়ে পরিবর্তন অধ্যায় ৪৭ এ।)

💡 **টিপস:** Fedora (33+) ডিফল্টে **zram** ব্যবহার করে: RAM এর ভেতরেই একটা compressed swap। disk swap এর চেয়ে অনেক দ্রুত। `swapon --show` এ `/dev/zram0` দেখলে বুঝবে এটাই।

`/proc/meminfo`: memory এর আসল উৎস

`free`, `top` সবাই আসলে এই ফাইল পড়ে হিসাব করে। এখানে kernel এর সব memory হিসাব আছে।

```
$ head -25 /proc/meminfo
MemTotal:        8029388 kB
MemFree:          312876 kB
MemAvailable:    2785432 kB
Buffers:          145220 kB
Cached:          2201344 kB
SwapCached:        12044 kB
Active:          4123456 kB
Inactive:        2654321 kB
...
SwapTotal:       2097148 kB
SwapFree:        2028012 kB
Dirty:              1840 kB
Writeback:             0 kB
AnonPages:       4402112 kB
Mapped:           512344 kB
Shmem:            184320 kB
...
Slab:             420112 kB
SReclaimable:     265432 kB
SUnreclaim:       154680 kB
```

গুরুত্বপূর্ণ লাইনগুলো:

ফিল্ড	মানে
`MemTotal`	মোট ব্যবহারযোগ্য RAM
`MemFree`	একদম খালি
`MemAvailable`	kernel এর অনুমান: swap ছাড়া নতুন কাজ কত পাবে (`free` এর `available`)
`Buffers` / `Cached`	block device buffer / ফাইলের page cache
`SwapCached`	যে page swap থেকে ফেরত এসেছে কিন্তু swap এ কপিও রয়ে গেছে
`Active` / `Inactive`	সম্প্রতি ব্যবহৃত / অনেকক্ষণ ব্যবহৃত হয়নি (Inactive আগে ছাড়া হয়)
`Dirty`	RAM এ বদলানো হয়েছে কিন্তু এখনো disk এ লেখা হয়নি
`Writeback`	এই মুহূর্তে disk এ লেখা হচ্ছে
`AnonPages`	কোনো ফাইলের সাথে যুক্ত না এমন memory (heap, stack): প্রোগ্রামের "নিজের" ডেটা
`Shmem`	shared memory ও tmpfs
`Slab`	kernel এর নিজের data structure এর cache
`SReclaimable` / `SUnreclaim`	slab এর যে অংশ ছাড়া যায় / যায় না
`HugePages_*`	huge page (বড় আকারের memory page, database এ ব্যবহৃত)

💡 **টিপস:** `SUnreclaim` যদি দিনে দিনে বাড়তে থাকে এবং কোনো process এর `RES` বাড়ছে না, তাহলে সন্দেহ করো kernel বা কোনো driver এ memory leak। `slabtop` কমান্ড দিয়ে দেখো কোন slab বাড়ছে।

`vmstat`: এক লাইনে পুরো সিস্টেমের নাড়ি

`vmstat` (virtual memory statistics) procps-ng এর অংশ, তাই প্রায় সবখানে থাকে। নাম memory হলেও এটা CPU, process, I/O সব দেখায়।

Syntax: `vmstat [options] [delay [count]]`

```
$ vmstat 1 5
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0  69120 312876 145220 2201344    0    0    12    45  210  380  8  2 89  1  0
 5  0  69120 310112 145220 2201500    0    0     0   120 1890 3200 72  8 14  6  0
 6  2  69120 305400 145224 2201600    0    0   800  2400 2100 3500 70  9 10 11  0
 4  1  69120 301200 145224 2201700    0    0   640  1900 1950 3300 71  8 13  8  0
 5  0  69120 300900 145228 2201800    0    0     0   300 1800 3100 73  7 18  2  0
```

⚠️ **সতর্কতা: প্রথম লাইনটা বাদ দাও।** প্রথম লাইন হলো boot থেকে এখন পর্যন্ত গড়, এখনকার অবস্থা না। দ্বিতীয় লাইন থেকে আসল লাইভ ডেটা।

কলাম	মানে	কখন সমস্যা
`r`	runnable process (চলছে বা CPU এর অপেক্ষায়)	CPU সংখ্যার চেয়ে স্থায়ীভাবে বেশি হলে CPU saturation
`b`	uninterruptible sleep (D state, প্রায়ই I/O অপেক্ষা)	নিয়মিত > 0 হলে I/O সমস্যা
`swpd`	ব্যবহৃত swap (KiB)	শুধু সংখ্যাটা চিন্তার না
`free` / `buff` / `cache`	memory (KiB)	
`si` / `so`	প্রতি সেকেন্ডে swap in / swap out	**নিয়মিত > 0 হলে memory সংকট**
`bi` / `bo`	block device থেকে পড়া / লেখা (block/s)	disk এর কাজের পরিমাণ
`in`	প্রতি সেকেন্ডে interrupt	
`cs`	প্রতি সেকেন্ডে context switch	হঠাৎ অনেক বেশি হলে অনেক thread একে অপরের সাথে কাড়াকাড়ি করছে
`us sy id wa st`	CPU ভাগ (৪৬.৪ এর টেবিল দেখো)	

📝 **মনে রাখো:** procps-ng এর নতুন ভার্সনে শেষে `gu` (guest) কলামও দেখতে পারো। কলাম কম-বেশি হলে ঘাবড়াবে না, হেডার দেখে পড়ো।

অপশন	কাজ
`-S M`	একক MB তে (`-S k`, `-S m`, `-S K`, `-S M`)
`-w`	wide (বড় সংখ্যা ঠিকমতো দেখাতে)
`-t`	প্রতি লাইনে timestamp
`-a`	buff/cache এর বদলে active/inactive
`-s`	memory ও event এর মোট পরিসংখ্যান, এক বার
`-d`	disk পরিসংখ্যান

```
$ vmstat -w -t -S M 2       # প্রতি ২ সেকেন্ড, MB তে, timestamp সহ
```

OOM Killer: যখন memory সত্যিই শেষ

কেন দরকার

লিনাক্স ডিফল্টে **overcommit** করে: প্রোগ্রাম যত memory চায়, তার চেয়ে বেশি "প্রতিশ্রুতি" দিয়ে দেয়, কারণ বেশিরভাগ প্রোগ্রাম যা চায় তার সবটা কখনো ব্যবহার করে না (উপমা: এয়ারলাইন ১০০ সিটের প্লেনে ১০৫টা টিকিট বিক্রি করে, কারণ কয়েকজন আসবেই না)। কিন্তু কোনোদিন সবাই একসাথে এসে গেলে? RAM আর swap দুটোই শেষ হলে kernel এর আর উপায় থাকে না। তখন সে **OOM (Out Of Memory) killer** চালায়: একটা process বেছে নিয়ে মেরে ফেলে, যাতে পুরো সিস্টেম না আটকায়।

কাকে মারবে, কীভাবে ঠিক করে

প্রতিটা process এর একটা `oom_score` থাকে (0 থেকে 1000 পর্যন্ত, মূলত কত memory ব্যবহার করছে তার ভিত্তিতে)। সবচেয়ে বেশি স্কোরের process মারা যায়।

```
$ cat /proc/1877/oom_score
682
$ cat /proc/1877/oom_score_adj
0
```

`oom_score_adj` (-1000 থেকে +1000) দিয়ে তুমি স্কোর বদলাতে পারো:

- `-1000` = কখনো মারবে না
- `+1000` = সবার আগে মারবে

```
# sshd কে OOM থেকে বাঁচাতে (root হিসেবে)
$ echo -500 | sudo tee /proc/$(pgrep -o sshd)/oom_score_adj
```

systemd সার্ভিসে স্থায়ীভাবে করতে unit ফাইলে `OOMScoreAdjust=-500` লেখো (অধ্যায় ২৬)।

OOM হয়েছে কিনা কীভাবে বুঝবে

হঠাৎ কোনো সার্ভিস "কোনো কারণ ছাড়াই" মারা গেছে? প্রথমে kernel log দেখো:

```
$ sudo dmesg -T | grep -i -E 'out of memory|killed process'
[Wed Nov 12 03:14:22 2026] Out of memory: Killed process 1877 (mysqld) total-
vm:2456780kB, anon-rss:1258000kB, file-rss:0kB, shmem-rss:0kB, UID:999 pgtables:3200kB
oom_score_adj:0

$ sudo journalctl -k --since "today" | grep -i oom
```

💡 **টিপস:** Ubuntu 22.04+ এবং Fedora 34+ এ **systemd-oomd** নামে একটা userspace OOM daemon ডিফল্টে চালু থাকে। এটা kernel এর OOM killer এর **আগেই**, memory চাপ (PSI) দেখে পুরো cgroup (যেমন একটা সার্ভিস বা ইউজার session) মেরে দিতে পারে। এর লগ দেখতে: `journalctl -u systemd-oomd`, অবস্থা দেখতে: `oomctl`।

⚠️ **সতর্কতা:** OOM বারবার হলে সমাধান "OOM killer বন্ধ করা" না। আসল কারণ খোঁজো: কোন process এর memory বাড়ছে (memory leak?), কনফিগারেশনে অতিরিক্ত worker দেওয়া আছে কিনা (যেমন php-fpm এর `pm.max_children` বা MySQL এর `innodb_buffer_pool_size` RAM এর তুলনায় বেশি), নাকি সত্যিই RAM বাড়ানো দরকার।

কোন process সবচেয়ে বেশি memory খাচ্ছে

```
$ ps aux --sort=-rss | head -6
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
mysql       1877 15.2 15.7 2456780 1258000 ?     Ssl  Oct31 120:33 /usr/sbin/mysqld
www-data    2314 25.1  3.1 812340 245120 ?       R    14:00  45:12 php-fpm: pool www
...
```

⚠️ **সতর্কতা:** অনেকগুলো process এর `RSS` যোগ করলে মোট RAM এর চেয়ে বেশি হতে পারে, কারণ shared library প্রতিটা process এর RSS এ গোনা হয়। প্রতিটা process এর "ন্যায্য ভাগ" জানতে **PSS** (Proportional Set Size) দেখো: `smem` টুল (`sudo apt install smem`) বা `/proc/PID/smaps_rollup` ফাইল।

```
$ sudo grep -E '^(Rss|Pss)' /proc/1877/smaps_rollup
Rss:             1258000 kB
Pss:             1240112 kB
```

৪৬.৬ Disk I/O মনিটরিং

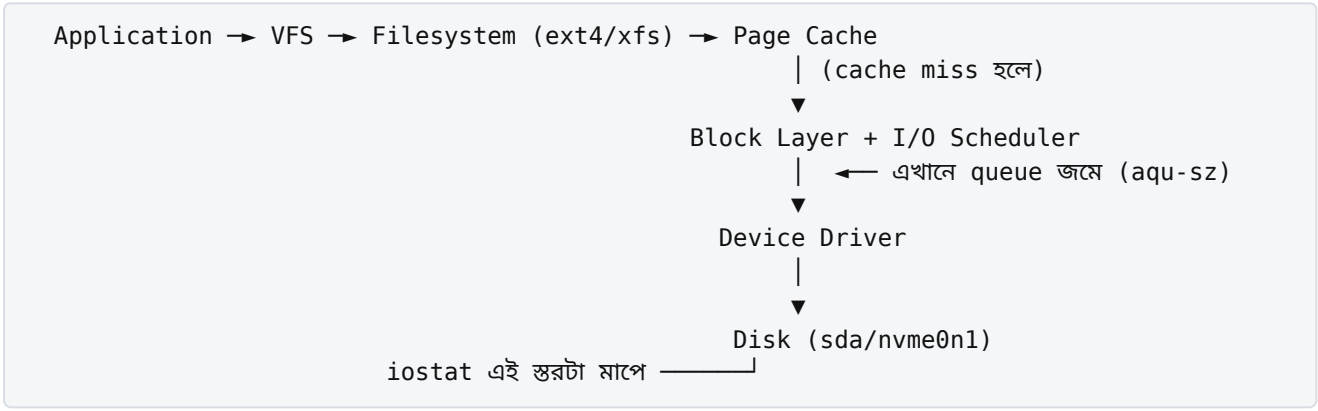
Disk কেন প্রায়ই "অপরাধী"

RAM এর তুলনায় disk অনেক ধীর। আন্দাজ (মোটা দাগে):

মাধ্যম	একটা random access এ সময় (আনুমানিক)
RAM	~100 ন্যানোসেকেন্ড

মাধ্যম	একটা random access এ সময় (আনুমানিক)
NVMe SSD	~১০-১০০ মাইক্রোসেকেন্ড
SATA SSD	~১০০ মাইক্রোসেকেন্ড
HDD (ঘূর্ণায়মান)	~৫-১০ মিলিসেকেন্ড

অর্থাৎ HDD, RAM এর চেয়ে প্রায় **লক্ষ গুণ** ধীর। তাই I/O তে সমস্যা হলে পুরো সিস্টেম ঝিমিয়ে পড়ে, load average বেড়ে যায় (D state এর কারণে), অথচ CPU অলস বসে থাকে (`wa` বেশি)।



`iostat` : প্রতিটা disk এর অবস্থা

sysstat প্যাকেজের অংশ। সবচেয়ে কাজের রূপ:

```
$ iostat -xz 1
```

অপশন	কাজ
`-x`	extended (বিস্তারিত) পরিসংখ্যান: **সবসময় এটা ব্যবহার করো**
`-z`	যে device এ কোনো কাজ হয়নি সেগুলো লুকাও
`-d`	শুধু device (CPU অংশ বাদ)
`-c`	শুধু CPU
`-h`	মানুষের পড়ার মতো
`-m` / `-k`	MB/s / kB/s এ
`-p sda`	sda এবং তার partition গুলো
`-t`	timestamp
`-N`	LVM/device-mapper এর নাম দেখাও (অধ্যায় ৩৩)

```
$ iostat -dxz 1 2
Linux 6.8.0-45-generic (web01)  11/12/2026  _x86_64_  (4 CPU)
...
Device      r/s     rkB/s   r_await rareq-sz   w/s     wkB/s   w_await wareq-sz  aqu-sz
%util
nvme0n1    12.00   480.00     0.35    40.00   85.00   9800.00    1.20   115.29    0.11
```

```
9.20
sda       210.00  8400.00    38.50    40.00  140.00  5600.00   95.10    40.00   21.40
99.60
```

(আসল আউটপুটে `rrqm/s`, `%rrqm`, `wrqm/s`, discard (`d/s`) ও flush (`f/s`) সম্পর্কিত আরও কলাম থাকে; এখানে বোঝার সুবিধার জন্য কিছু বাদ দেওয়া হয়েছে।)

কলাম	মানে
`r/s`, `w/s`	প্রতি সেকেন্ডে read/write request (IOPS)
`rkB/s`, `wkB/s`	প্রতি সেকেন্ডে কত kB পড়া/লেখা (throughput)
`r_await`, `w_await`	একটা request এর গড় সময় (ms), **queue তে অপেক্ষা + আসল কাজ মিলিয়ে**। ইউজার এই latency ই টের পায়
`rareq-sz`, `wareq-sz`	গড় request এর আকার (kB)
`aqu-sz`	গড় queue এর দৈর্ঘ্য (আগের ভার্সনে `avgqu-sz`)। **Saturation এর মাপ**
`%util`	device কত % সময় ব্যস্ত ছিল

উপরের উদাহরণে `sda` স্পষ্টভাবে সমস্যায়: `%util` প্রায় ১০০%, `aqu-sz` ২১, write এ ৯৫ ms অপেক্ষা। `nvme0n1` আরামে আছে।

⚠️ **সতর্কতা:** `%util` ১০০% মানেই disk "পুরো ভরা" না, বিশেষ করে SSD/NVMe আর RAID এর ক্ষেত্রে। এগুলো একসাথে অনেক request সমান্তরালে সামলাতে পারে। `%util` শুধু বলে "অন্তত একটা request চলছিল এতটা সময়"। তাই SSD তে `await` আর `aqu-sz` বেশি বিশ্বাসযোগ্য সংকেত। HDD (যেটা একবারে একটাই কাজ পারে) এর ক্ষেত্রে `%util` ভালো সংকেত।

💡 **টিপস:** সাধারণ নিয়ম (আন্দাজ): SSD তে `await` কয়েক ms এর বেশি, বা HDD তে ২০-৩০ ms এর বেশি স্থায়ীভাবে থাকলে তদন্ত করো।

`iotop`: কোন process disk কাঁপাচ্ছে

`iostat` বলে কোন **disk** ব্যস্ত, কিন্তু কে ব্যস্ত রাখছে বলে না। সেজন্য `iotop`। এটা root হিসেবে চালাতে হয়।

```
$ sudo apt install iotop        # Ubuntu তে iotop-c নামে C ভার্সনও আছে
$ sudo dnf install iotop        # Fedora/RHEL
$ sudo iotop -o
Total DISK READ:         8.20 M/s | Total DISK WRITE:         5.47 M/s
Current DISK READ:       8.20 M/s | Current DISK WRITE:       9.10 M/s
    TID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN     IO>    COMMAND
   1877 be/4 mysql       7.90 M/s    4.80 M/s  0.00 %  88.12 %  mysqld
   4410 be/4 root      300.00 K/s  600.00 K/s  0.00 %  10.05 %  rsync -a /var/www
/backup
```

অপশন	কাজ
`-o`	শুধু যারা এখন I/O করছে
`-P`	thread না, process দেখাও
`-a`	মুহূর্তের হার না, শুরু থেকে জমা মোট
`-b`	batch mode (স্ক্রিপ্ট/লগের জন্য)
`-n N`	N বার দেখিয়ে থামো
`-d SEC`	রিফ্রেশ বিরতি
`-u USER` / `-p PID`	ফিল্টার

চলাকালীন কী	কাজ
`o`	শুধু active দেখানো চালু/বন্ধ
`p`	process/thread মোড বদল
`a`	accumulated মোড বদল
`←` / `→`	সাজানোর কলাম বদল
`r`	সাজানোর ক্রম উল্টাও
`q`	বের হও

📝 **মনে রাখো:** `IO>` কলাম বলে ওই thread কত % সময় I/O এর অপেক্ষায় কাটিয়েছে। কিছু নতুন kernel এ এই হিসাবের জন্য `kernel.task_delayacct` sysctl চালু করতে হয় (`sudo sysctl kernel.task_delayacct=1`); iotop নিজেই প্রয়োজনে সতর্ক করে দেয়।

`iotop` না থাকলে বিকল্প: `pidstat -d 1` (sysstat)।

৪৬.৭ Network মনিটরিং

নেটওয়ার্কের মূলনীতি অধ্যায় ৩৬ এ আর `ss`, `ping`, `traceroute`, `tcpdump` এর মতো ট্রাবলশুটিং কমান্ড অধ্যায় ৩৭ এ আছে। এখানে শুধু **কতটা bandwidth কে খাচ্ছে** সেটা দেখব।

`iftop`: কোন কানেকশনে কত ট্রাফিক

`iftop` একটা interface এর উপর দিয়ে যাওয়া প্রতিটা কানেকশন (দুই host এর জোড়া) এর bandwidth লাইভ দেখায়। root লাগে।

```
$ sudo apt install iftop     # Fedora/RHEL: sudo dnf install iftop (RHEL এ EPEL)
$ sudo iftop -i eth0 -nP
                 12.5Mb          25.0Mb          37.5Mb          50.0Mb
└─────────────────┴─────────────────┴─────────────────┴─────────────────┘
10.0.0.5:443        => 203.0.113.45:51822      8.20Mb  6.10Mb  5.90Mb
```

```
                    <=                              210Kb   180Kb   170Kb
10.0.0.5:22         => 198.51.100.7:60211            12Kb    10Kb    9Kb
                    <=                              3.1Kb   2.8Kb   2.5Kb
--------------------------------------------------------------------------------
TX:             cum:   45.2MB   peak:  9.10Mb  rates:  8.21Mb  6.11Mb  5.91Mb
RX:                     1.6MB          250Kb            213Kb   183Kb   173Kb
TOTAL:                 46.8MB          9.35Mb          8.42Mb  6.29Mb  6.08Mb
```

ডানের তিনটা কলাম হলো গত **২ সেকেন্ড, ১০ সেকেন্ড, ৪০ সেকেন্ডের** গড় হার।

অপশন	কাজ
`-i IFACE`	কোন interface (নাম জানতে `ip link`)
`-n`	IP কে hostname এ রূপান্তর (DNS lookup) করো না (দ্রুত, এবং নিজেই ট্রাফিক তৈরি করে না)
`-N`	port নম্বরকে service নামে বদলিও না
`-P`	port দেখাও
`-B`	bit এর বদলে byte এ দেখাও
`-f FILTER`	pcap ফিল্টার, যেমন `-f 'port 443'`
`-t`	text mode (ncurses ছাড়া), `-s N` এর সাথে স্ক্রিপ্টে

চলাকালীন কী	কাজ
`n`	DNS resolve চালু/বন্ধ
`p`	port দেখানো চালু/বন্ধ
`P`	pause
`t`	এক লাইন/দুই লাইন/শুধু পাঠানো/শুধু আসা ঘুরিয়ে দেখাও
`1` / `2` / `3`	২s / ১০s / ৪০s কলাম দিয়ে সাজাও
`<` / `>`	source / destination নাম দিয়ে সাজাও
`l`	ফিল্টার করে দেখাও
`q`	বের হও

`nethogs`: কোন process কত bandwidth খাচ্ছে

`iftop` বলে কোন কানেকশন, `nethogs` বলে কোন প্রোগ্রাম। "কে ইন্টারনেট খেয়ে ফেলছে?" প্রশ্নের সরাসরি উত্তর।

```
$ sudo apt install nethogs     # Fedora: sudo dnf install nethogs
$ sudo nethogs eth0
NetHogs version 0.8.7

    PID USER     PROGRAM                          DEV        SENT      RECEIVED
   4410 root     rsync                            eth0     5210.4     12.1 KB/sec
   2314 www-data php-fpm: pool www                eth0      820.2     95.3 KB/sec
```

```
   1102 root     sshd: admin@pts/0                eth0        1.2      0.3 KB/sec
      ? root     unknown TCP                                  0.0      0.0 KB/sec
```

অপশন / কী	কাজ
`-d SEC`	রিফ্রেশ বিরতি
`-t`	trace mode (লাইন ধরে আউটপুট)
`-v N`	একক: 0=KB/s, 1=মোট KB, 2=মোট B, 3=মোট MB
চলাকালীন `m`	একক ঘোরাও (KB/s → KB → B → MB)
`s` / `r`	sent / received দিয়ে সাজাও
`q`	বের হও

দ্রুত interface পরিসংখ্যান

```
$ ip -s link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
    RX:  bytes  packets errors dropped  missed   mcast
    9876543210  8123456      0      12       0       0
    TX:  bytes  packets errors dropped carrier collsns
    5432109876  6012345      0       0       0       0
```

`errors` আর `dropped` বাড়তে থাকলে এটা USE এর **E** (Errors)। কেবল, NIC driver, বা buffer এর সমস্যা হতে পারে।

৪৬.৮ sysstat ও `sar`: পারফরম্যান্সের "ব্ল্যাক বক্স রেকর্ডার"

`top`, `iostat` সবই এই মুহূর্তের ছবি দেখায়। কিন্তু সমস্যা প্রায়ই রাত ৩টায় হয়, আর সকালে তুমি এসে দেখো সব স্বাভাবিক। তখন কী করবে?

উপমা: বিমানের **black box** সারাক্ষণ সব রেকর্ড করে, দুর্ঘটনার পর সেটা দেখে কারণ বোঝা যায়। লিনাক্সে সেই black box হলো **sysstat** এর `sadc` (system activity data collector), যা নিয়মিত বিরতিতে (ডিফল্ট ১০ মিনিট) সব পরিসংখ্যান ফাইলে সংরক্ষণ করে। আর `sar` (system activity reporter) সেই ফাইল পড়ে দেখায়।

```
   systemd timer (sysstat-collect.timer, প্রতি ১০ মিনিট)
           │
           ▼
      sa1 ──► sadc ──► /var/log/sysstat/saDD   (Debian/Ubuntu)
                       /var/log/sa/saDD        (Fedora/RHEL)
                            │      (DD = মাসের তারিখ, যেমন sa12)
                            ▼
                       sar ──► তোমার স্ক্রিনে রিপোর্ট
```

চালু করা

```
# Debian/Ubuntu
$ sudo apt install sysstat
$ sudo sed -i 's/^ENABLED="false"/ENABLED="true"/' /etc/default/sysstat
$ sudo systemctl enable --now sysstat

# Fedora/RHEL
$ sudo dnf install sysstat
$ sudo systemctl enable --now sysstat
```

📝 **মনে রাখো:** Debian/Ubuntu তে `/etc/default/sysstat` ফাইলের `ENABLED="true"` না করলে ডেটা জমা হয় না (Ubuntu র নতুন রিলিজে প্যাকেজ ইনস্টলের সময় এটা ঠিক করা থাকতে পারে, তবু একবার দেখে নাও)। কতদিনের ডেটা রাখবে তা ঠিক হয় `/etc/sysstat/sysstat` (Debian) বা `/etc/sysconfig/sysstat` (RHEL) ফাইলের `HISTORY` দিয়ে।

`sar` এর গুরুত্বপূর্ণ অপশন

অপশন	কী দেখায়
`-u`	CPU (ডিফল্ট); `-u ALL` সব ভাগ
`-P ALL`	প্রতিটা CPU আলাদা
`-q`	load average ও run queue (`runq-sz` , `ldavg-1` , ...)
`-r`	memory ব্যবহার
`-S`	swap ব্যবহার
`-W`	swapping এর হার (`pswpin/s` , `pswpout/s`)
`-B`	paging পরিসংখ্যান
`-b`	মোট I/O হার
`-d`	প্রতি block device (`-p` দিলে সুন্দর নাম)
`-n DEV`	প্রতি network interface এর ট্রাফিক
`-n EDEV`	network error
`-n TCP,ETCP`	TCP কানেকশন ও retransmit
`-n SOCK`	socket সংখ্যা
`-w`	context switch ও process তৈরির হার
`-A`	সব কিছু
`-f FILE`	নির্দিষ্ট ডেটা ফাইল পড়ো
`-s HH:MM:SS` / `-e HH:MM:SS`	শুরু / শেষ সময়
`interval count`	লাইভ মোড (ফাইল না পড়ে এখনই মাপো)

উদাহরণ

```
# লাইভ: প্রতি সেকেন্ডে ৩ বার network ট্রাফিক
$ sar -n DEV 1 3
02:50:01 PM     IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s
rxmcst/s   %ifutil
02:50:02 PM        lo     10.00     10.00      1.20      1.20      0.00      0.00
0.00      0.00
02:50:02 PM      eth0   1520.00   2810.00     95.30   5210.40      0.00      0.00
0.00      4.27
```

```
# আজকের সংরক্ষিত CPU ডেটা, রাত ২টা থেকে ৪টা
$ sar -u -s 02:00:00 -e 04:00:00
02:00:01 AM     CPU     %user     %nice   %system   %iowait    %steal     %idle
02:10:01 AM     all      5.12      0.00      1.20      0.40      0.00     93.28
02:20:01 AM     all      4.98      0.00      1.18      0.35      0.00     93.49
...
03:10:01 AM     all     12.40      0.00      6.80     62.10      0.00     18.70
03:20:01 AM     all     11.95      0.00      6.50     65.40      0.00     16.15
...
```

রাত ৩টার পর `%iowait` হঠাৎ ৬০% এর উপরে। এখন সন্দেহ: ওই সময় কোনো backup বা cron job চলে কিনা (অধ্যায় ২৯, অধ্যায় ৩৫)।

```
# গতকালের (মনে করো ১১ তারিখ) memory ডেটা
$ sar -r -f /var/log/sysstat/sa11          # Ubuntu/Debian
$ sar -r -f /var/log/sa/sa11               # RHEL/Fedora

# load ইতিহাস
$ sar -q
02:10:01 PM   runq-sz  plist-sz   ldavg-1   ldavg-5  ldavg-15   blocked
02:20:01 PM         2      1234      0.52      0.78      1.05         0
```

💡 **টিপস:** নতুন সার্ভার সেটআপ করলেই প্রথম দিন sysstat চালু করে দাও। সমস্যার দিন ডেটা না থাকলে আফসোস করা ছাড়া কিছু করার থাকে না।

৪৬.৯ PSI (Pressure Stall Information): আধুনিক "চাপ মাপার যন্ত্র"

Kernel 4.20 থেকে লিনাক্সে একটা দারুণ জিনিস আছে: **PSI**। Load average এর একটা দুর্বলতা হলো এটা CPU আর I/O মিশিয়ে ফেলে। PSI সরাসরি বলে: **গত ১০/৬০/৩০০ সেকেন্ডে কত % সময় task গুলো CPU, memory বা I/O এর অভাবে আটকে ছিল।** এটা খাঁটি **Saturation** এর মাপ।

```
$ cat /proc/pressure/cpu
some avg10=45.20 avg60=30.10 avg300=12.05 total=987654321
full avg10=0.00 avg60=0.00 avg300=0.00 total=0

$ cat /proc/pressure/memory
some avg10=0.00 avg60=0.00 avg300=0.00 total=12345
```

```
full avg10=0.00 avg60=0.00 avg300=0.00 total=6789

$ cat /proc/pressure/io
some avg10=12.50 avg60=8.30 avg300=3.10 total=55443322
full avg10=9.80 avg60=6.10 avg300=2.20 total=44332211
```

শব্দ	মানে
`some`	অন্তত **একটা** task ওই resource এর অভাবে আটকে ছিল এমন সময়ের %
`full`	**সব** non-idle task একসাথে আটকে ছিল (কেউ কাজ করতে পারেনি) এমন সময়ের %। সবচেয়ে খারাপ অবস্থা
`avg10/60/300`	গত ১০, ৬০, ৩০০ সেকেন্ডের গড়

উপরের উদাহরণে CPU তে ৪৫% সময় কেউ না কেউ অপেক্ষায় ছিল, আর I/O তে ৯.৮% সময় সবাই আটকে ছিল। Memory চাপ শূন্য। ব্যস, এক নজরে কোথায় সমস্যা বোঝা গেল। systemd-oomd এই memory PSI দেখেই সিদ্ধান্ত নেয়।

৪৬.১০ `lsof`: "কে কী খুলে রেখেছে?"

লিনাক্সে প্রায় সবকিছুই ফাইল: সাধারণ ফাইল, ডিরেক্টরি, device, pipe, এমনকি network socket। `lsof` (list open files) বলে কোন process কোন ফাইল/socket খুলে রেখেছে। ট্রাবলশুটিং এ এটা জাদুর কাঠি।

```
$ sudo apt install lsof    # সাধারণত আগে থেকেই থাকে; Fedora/RHEL: sudo dnf install lsof
```

⚠️ **সতর্কতা:** অন্যের process দেখতে `sudo` লাগবে, নইলে শুধু নিজেরগুলো দেখবে।

অপশন	কাজ	উদাহরণ
`-i`	network socket	`lsof -i`
`-i :PORT`	নির্দিষ্ট port	`lsof -i :80`
`-i TCP` / `-i UDP`	প্রোটোকল	`lsof -i TCP:443`
`-i @HOST`	নির্দিষ্ট host এর সাথে কানেকশন	`lsof -i @10.0.0.9`
`-p PID`	নির্দিষ্ট process এর সব খোলা ফাইল	`lsof -p 1877`
`-u USER`	ইউজারের খোলা ফাইল	`lsof -u www-data`
`-c NAME`	যে command এর নাম NAME দিয়ে শুরু	`lsof -c nginx`
`+D DIR`	ডিরেক্টরির ভেতরে (recursive) খোলা ফাইল	`lsof +D /var/log`
`+L1`	**মুছে ফেলা হয়েছে কিন্তু এখনো খোলা** ফাইল (link count < 1)	`lsof +L1`
`-n` / `-P`	IP কে নাম / port কে service নামে বদলিও না (দ্রুত)	`lsof -nP -i`
`-t`	শুধু PID দেখাও (স্ক্রিপ্টে কাজে লাগে)	`kill $(lsof -t -i :8080)`

অপশন	কাজ	উদাহরণ
`-a`	একাধিক শর্ত AND করো (ডিফল্টে OR)	`lsof -a -u www-data -i`
`-r SEC`	প্রতি SEC সেকেন্ডে আবার দেখাও	`lsof -r 2 -i :443`

```
$ sudo lsof -nP -i :80
COMMAND  PID     USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
nginx   1020     root    6u  IPv4  23456      0t0  TCP *:80 (LISTEN)
nginx   1021 www-data    6u  IPv4  23456      0t0  TCP *:80 (LISTEN)
nginx   1021 www-data   12u  IPv4  98765      0t0  TCP 10.0.0.5:80->203.0.113.45:51822
(ESTABLISHED)
```

কলাম	মানে
`FD`	File descriptor: `cwd` (current dir), `txt` (প্রোগ্রামের নিজের কোড), `mem` (memory-mapped ফাইল), সংখ্যা + `r` / `w` / `u` (read/write/দুটোই)
`TYPE`	`REG` (সাধারণ ফাইল), `DIR`, `CHR` (character device), `IPv4` / `IPv6`, `unix`, `FIFO`
`NAME`	ফাইলের path বা কানেকশন

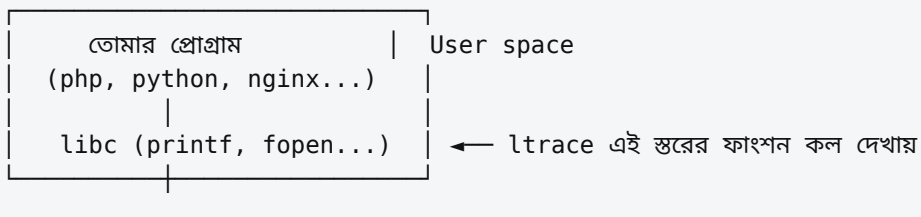
💡 **টিপস:** "ডিস্ক umount হচ্ছে না, `target is busy` বলছে"? `sudo lsof +D /mnt/usb` বা `sudo fuser -vm /mnt/usb` দিয়ে দেখো কে ওটা ব্যবহার করছে।

💡 **টিপস:** একটা process কতগুলো file descriptor খুলে রেখেছে: `ls /proc/1877/fd | wc -l`। সীমা দেখতে: `grep 'open files' /proc/1877/limits`। "Too many open files" error এ এটা কাজে লাগে।

৪৬.১১ `strace` ও `ltrace`: প্রোগ্রামের ভেতরে উঁকি

System call কী, আবার মনে করি

একটা প্রোগ্রাম নিজে সরাসরি disk, network বা স্ক্রিনে হাত দিতে পারে না। সব কাজ তাকে kernel এর কাছে "আবেদন" করে করাতে হয়। এই আবেদনগুলোকে বলে **system call** (syscall), যেমন `openat()` (ফাইল খোলো), `read()`, `write()`, `connect()`। (অধ্যায় ২ এ user space আর kernel space এর পার্থক্য দেখেছ।)



```
              │  system call (openat, read, write, connect...)
              │                ◄── strace এই সীমানাটা দেখায়
    ┌─────────▼──────────────────┐
    │          Kernel            │  Kernel space
    └────────────────────────────┘
```

উপমা: প্রোগ্রাম হলো অফিসের একজন কর্মচারী, kernel হলো ম্যানেজার। কর্মচারী প্রতিটা কাজের জন্য ম্যানেজারকে চিঠি লেখে। `strace` হলো সেই পিয়ন যে প্রতিটা চিঠির কপি তোমাকে দেখায়: কী চাওয়া হলো আর ম্যানেজার কী উত্তর দিল।

`strace`

```
$ sudo apt install strace     # Fedora/RHEL: sudo dnf install strace
```

অপশন	কাজ
`strace CMD`	নতুন একটা কমান্ড চালিয়ে trace করো
`-p PID`	চলমান process এ যুক্ত হও (root/একই ইউজার লাগে); বের হতে `Ctrl+C`
`-f`	child process ও thread ও trace করো
`-e trace=openat,read`	শুধু নির্দিষ্ট syscall
`-e trace=%file` / `%network` / `%process`	syscall এর গ্রুপ (ফাইল/নেটওয়ার্ক/process সম্পর্কিত)
`-c`	প্রতিটা trace না দেখিয়ে শেষে একটা **সারাংশ টেবিল** দাও
`-T`	প্রতিটা syscall এ কত সময় লাগল
`-t` / `-tt`	প্রতিটা লাইনে সময় / মাইক্রোসেকেন্ড সহ সময়
`-o FILE`	আউটপুট ফাইলে
`-s N`	string কত দৈর্ঘ্য পর্যন্ত দেখাবে (ডিফল্ট 32)
`-y`	file descriptor নম্বরের পাশে path দেখাও

উদাহরণ ১: প্রোগ্রাম কোন config ফাইল খুঁজছে?

```
$ strace -e trace=openat cat /etc/hostname
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
web01
+++ exited with 0 +++
```

প্রতিটা লাইনে syscall, তার argument, আর `=` এর পরে ফলাফল। ফলাফল `-1` হলে সাথে error নাম থাকে:

```
$ strace -e trace=openat myapp 2>&1 | grep ENOENT
openat(AT_FDCWD, "/etc/myapp/config.yml", O_RDONLY) = -1 ENOENT (No such file or
directory)
```

এখন জানলে, প্রোগ্রাম `/etc/myapp/config.yml` খুঁজছে কিন্তু পাচ্ছে না। কোনো ডকুমেন্টেশন না পড়েই সমস্যা ধরা পড়ল। একইভাবে `EACCES` মানে permission denied।

উদাহরণ ২: আটকে থাকা process কী করছে?

```
$ sudo strace -p 2314 -tt -T
14:55:01.120345 poll([{fd=9, events=POLLIN}], 1, -1
```

process টা `poll()` এ অপেক্ষা করছে, কোনো সময়সীমা (`-1`) ছাড়াই। `-y` দিয়ে দেখো fd 9 কী: হয়তো একটা database বা external API এর socket যেটা উত্তর দিচ্ছে না।

উদাহরণ ৩: সময় কোথায় যাচ্ছে (সারাংশ)

```
$ sudo strace -c -f -p 2314
^C
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 62.10    0.412300         206      2001           read
 20.40    0.135400          45      3005       512 openat
 10.20    0.067700          22      3002           newfstatat
  ...
------ ----------- ----------- --------- --------- ----------------
100.00    0.663900                 9120       530 total
```

`openat` এ ৫১২টা error! হয়তো প্রোগ্রাম অনেক অস্তিত্বহীন ফাইল খুঁজছে (যেমন ভুল include path)।

⚠️ **সতর্কতা:** `strace` লক্ষ্য process কে **অনেক ধীর** করে দেয় (কখনো কয়েকগুণ), কারণ প্রতিটা syscall এ process থামিয়ে তথ্য পড়তে হয়। ব্যস্ত production সার্ভারে অল্প সময়ের জন্য ব্যবহার করো।

`ltrace`

`ltrace` একই কাজ করে, কিন্তু **library function call** (যেমন `malloc`, `strlen`, `printf`) দেখায়, syscall না।

```
$ sudo apt install ltrace       # Fedora: sudo dnf install ltrace
$ ltrace -c ls > /dev/null
% time     seconds  usecs/call     calls      function
------ ----------- ----------- --------- --------------------
 25.10    0.001200          12       100 strlen
 ...
```

অপশন	কাজ
`-p PID`	চলমান process
`-c`	সারাংশ
`-S`	library call এর সাথে syscall ও দেখাও
`-e malloc+free`	নির্দিষ্ট ফাংশন
`-f`	child process

📝 **মনে রাখো:** আধুনিক distro তে অনেক binary বিশেষ নিরাপত্তা অপশন দিয়ে compile করা থাকে, ফলে `ltrace` সবসময় সব কল দেখাতে পারে না। দৈনন্দিন ট্রাবলশুটিং এ `strace` অনেক বেশি কাজের।

৪৬.১২ `perf` পরিচিতি: CPU কোন ফাংশনে সময় দিচ্ছে

`top` বলে "php-fpm ৯৮% CPU খাচ্ছে"। কিন্তু প্রোগ্রামের **কোন ফাংশনে**? এর উত্তর দেয় `perf`, লিনাক্স kernel এর সাথে আসা অফিসিয়াল profiling টুল। এটা CPU এর hardware counter আর kernel এর tracepoint ব্যবহার করে খুব কম overhead এ মাপে।

উপমা: `strace` হলো প্রতিটা চিঠি পড়া পিয়ন (ধীর)। `perf` হলো এমন একজন, যে প্রতি সেকেন্ডে কয়েকশো বার এক ঝলক উঁকি দিয়ে দেখে "এই মুহূর্তে কর্মচারী কী করছে"। হাজারটা ঝলক মিলিয়ে বোঝা যায় কোন কাজে সবচেয়ে বেশি সময় যায় (**sampling**)।

ইনস্টল:

```
# Ubuntu (চলমান kernel এর সাথে মিলিয়ে)
$ sudo apt install linux-tools-common linux-tools-$(uname -r)
# Debian 12
$ sudo apt install linux-perf
# Fedora/RHEL
$ sudo dnf install perf
```

কমান্ড	কাজ
`sudo perf top`	লাইভ: পুরো সিস্টেমে কোন ফাংশন সবচেয়ে বেশি CPU খাচ্ছে
`sudo perf top -p PID`	নির্দিষ্ট process
`perf stat CMD`	একটা কমান্ডের মোট পরিসংখ্যান (cycles, instructions, cache miss...)
`sudo perf record -g -p PID -- sleep 30`	৩০ সেকেন্ড ধরে call stack সহ রেকর্ড করো (`perf.data` ফাইলে)
`sudo perf report`	রেকর্ড করা ডেটা বিশ্লেষণ (interactive)
`perf list`	কোন কোন event মাপা যায়

```
$ perf stat gzip -k bigfile.log

 Performance counter stats for 'gzip -k bigfile.log':

          2,104.55 msec task-clock                #    0.998 CPUs utilized
                12      context-switches          #    5.702 /sec
     8,120,334,221      cycles                    #    3.858 GHz
    11,402,118,003      instructions              #    1.40  insn per cycle
       ...
       2.108331021 seconds time elapsed
```

```
$ sudo perf top
Samples: 42K of event 'cycles:P', 4000 Hz, Event count (approx.): 9876543210
Overhead  Shared Object            Symbol
  18.42%  php-fpm8.3               [.] execute_ex
   9.10%  libc.so.6                [.] __memmove_avx_unaligned_erms
   6.75%  [kernel]                 [k] copy_user_enhanced_fast_string
```

`[.]` মানে user space ফাংশন, `[k]` মানে kernel ফাংশন।

📝 **মনে রাখো:** `kernel.perf_event_paranoid` sysctl ঠিক করে সাধারণ ইউজার কতটা perf চালাতে পারবে। অনেক distro তে এটা কড়া সেট করা থাকে, তাই সিস্টেম-জুড়ে profiling এর জন্য `sudo` দিয়ে চালাও। VM এ hardware counter (cycles, instructions) সবসময় পাওয়া যায় না, তখন `perf top` নিজে software event (`cpu-clock`) ব্যবহার করে।

💡 **টিপস:** `perf record -g` এর ডেটা থেকে **Flame Graph** (Brendan Gregg এর তৈরি) বানানো যায়: একটা ছবি যেখানে চওড়া বাক্স মানে বেশি CPU সময়। গভীরে যেতে চাইলে পরবর্তী ধাপ হলো **eBPF** টুল (`bpftrace` , bcc-tools এর `execsnoop` , `opensnoop` , `biolatency`), যেগুলো production এ খুব কম খরচে kernel এর ভেতরের ঘটনা মাপে।

৪৬.১৩ সব-এক-জায়গায় ড্যাশবোর্ড: `btop` ও `glances`

`btop`

সুন্দর, রঙিন, মাউস-চালিত মনিটর: CPU (প্রতি core এর গ্রাফ সহ), memory, disk, network, process সব এক স্ক্রিনে।

```
$ sudo apt install btop        # Ubuntu 22.04+, Debian 12
$ sudo dnf install btop        # Fedora (RHEL এ EPEL)
$ btop
```

কী	কাজ
`Esc` / `m`	মেনু
`o`	অপশন/সেটিংস

কী	কাজ
`h`	হেল্প
`1` / `2` / `3` / `4`	CPU / memory / network / process বক্স দেখাও-লুকাও
`←` / `→`	process সাজানোর কলাম বদল
`r`	সাজানোর ক্রম উল্টাও
`e`	tree view
`f`	process ফিল্টার
`↑` / `↓` + `Enter`	process বেছে বিস্তারিত দেখো
`k` / `t` / `s`	kill (SIGKILL) / terminate (SIGTERM) / signal বেছে পাঠাও
`+` / `-`	রিফ্রেশ বিরতি বাড়াও/কমাও
`q`	বের হও

`glances`

Python এ লেখা, এক স্ক্রিনে প্রায় সবকিছু, সাথে নিজে থেকে **সতর্কবার্তা** (সবুজ/নীল/বেগুনি/লাল রঙে) দেখায়। Web বা API মোডেও চালানো যায়।

```
$ sudo apt install glances     # Fedora: sudo dnf install glances
$ glances
$ glances -w                   # web UI, ডিফল্ট পোর্ট 61208 (ব্রাউজারে http://IP:61208)
```

কী	কাজ
`a`	স্বয়ংক্রিয় সাজানো (যেটা সমস্যা করছে সেটা উপরে)
`c` / `m` / `i` / `p` / `t`	CPU / memory / I/O / নাম / সময় অনুযায়ী সাজাও
`1`	প্রতি core আলাদা
`d` / `n` / `s`	disk / network / sensor অংশ লুকাও-দেখাও
`f`	filesystem অংশ লুকাও-দেখাও
`h`	হেল্প
`q` / `Esc`	বের হও

⚠️ **সতর্কতা:** `glances -w` এ ডিফল্টে কোনো পাসওয়ার্ড নেই। ইন্টারনেটের সামনে খোলা রাখবে না; দরকার হলে `--password` ব্যবহার করো এবং ফায়ারওয়াল দিয়ে আটকাও (অধ্যায় ৩৯)।

কোন টুল কখন: তুলনা

টুল	সবচেয়ে ভালো কাজ	প্রায় সবখানে আছে?
`top`	দ্রুত এক নজর, যেকোনো সার্ভারে	হ্যাঁ

টুল	সবচেয়ে ভালো কাজ	প্রায় সবখানে আছে?
`htop`	ইন্টারঅ্যাকটিভ process ম্যানেজমেন্ট	প্রায়ই
`btop`	সুন্দর সার্বিক চিত্র	না, ইনস্টল লাগে
`glances`	সার্বিক চিত্র + সতর্কবার্তা + web	না
`vmstat`, `mpstat`, `iostat`, `pidstat`	নির্দিষ্ট resource এর নির্ভুল সংখ্যা	sysstat ইনস্টল লাগে (vmstat ছাড়া)
`sar`	অতীতের ডেটা	sysstat চালু থাকতে হবে

৪৬.১৪ "সার্ভার স্লো!" ধাপে ধাপে ডায়াগনোসিস চেকলিস্ট

এটাই এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ অংশ। Brendan Gregg এর বিখ্যাত **"৬০ সেকেন্ডের বিশ্লেষণ"** এর উপর ভিত্তি করে সাজানো। লগইন করেই এই ক্রমে চালাও:

```
┌──────────────────────────────────────────────────────────────┐
│ ধাপ ০: প্রশ্ন করো — "স্লো" মানে কী? কবে থেকে? সবার জন্য নাকি একজনের? │
│        সম্প্রতি কী বদলেছে (deploy, update, config)?          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
        ১. uptime ──► load বাড়ছে না কমছে? CPU সংখ্যার তুলনায় কত?
                              ▼
        ২. dmesg -T | tail ──► OOM? disk error? hardware error?
                              ▼
        ৩. vmstat 1 ──► r (CPU লাইন), b (I/O), si/so (swap), wa, st
                              ▼
        ৪. mpstat -P ALL 1 ──► একটা core কি একা আটকে?
                              ▼
        ৫. pidstat 1 ──► কোন process CPU খাচ্ছে?
                              ▼
        ৬. iostat -xz 1 ──► কোন disk ধীর? await, aqu-sz, %util
                              ▼
        ৭. free -h ──► available কত? swap চলছে?
                              ▼
        ৮. sar -n DEV 1 ──► network কি লিঙ্কের সীমায়?
                              ▼
        ৯. sar -n TCP,ETCP 1 ──► retransmit (নেটওয়ার্ক সমস্যা)?
                              ▼
        ১০. top / htop ──► সব মিলিয়ে নিশ্চিত হও
                              ▼
        ১১. df -h; df -i ──► disk/inode ভরা?
                              ▼
        সন্দেহজনক process পেলে ──► strace -p / lsof -p / perf top -p / লগ দেখো
```

ধাপ	কমান্ড	কী খুঁজবে
১	`uptime`	load এর ধারা (বাড়ছে/কমছে), `nproc` এর সাথে তুলনা
২	`sudo dmesg -T | tail -30`	"Out of memory", "I/O error", "segfault", "link down"
৩	`vmstat 1 5`	`r` > CPU সংখ্যা? `b` > 0? `si` / `so` > 0? `wa` বা `st` বেশি?

ধাপ	কমান্ড	কী খুঁজবে
৪	`mpstat -P ALL 1 3`	অসম CPU ব্যবহার (single-thread bottleneck)
৫	`pidstat 1 3`	কোন process CPU খাচ্ছে, `%wait` কত
৬	`iostat -xz 1 3`	`await`, `aqu-sz`, `%util`
৭	`free -h`	`available` কম? swap ব্যবহার বাড়ছে?
৮	`sar -n DEV 1 3`	`rxkB/s` / `txkB/s` লিঙ্ক স্পিডের কাছাকাছি?
৯	`sar -n TCP,ETCP 1 3`	`retrans/s` বেশি = নেটওয়ার্ক সমস্যা
১০	`top`	উপরের সব সন্দেহ এক জায়গায় মিলিয়ে দেখো
১১	`df -h; df -i`	১০০% ভরা filesystem
১২	`cat /proc/pressure/{cpu,memory,io}`	কোন resource এ চাপ
১৩	`journalctl -p err --since "1 hour ago"`	সাম্প্রতিক error (অধ্যায় ২৮)

ফলাফল দেখে সিদ্ধান্ত

যা দেখলে	সম্ভাব্য কারণ	পরের পদক্ষেপ
load বেশি, `us` বেশি, `id` কম	অ্যাপ্লিকেশন CPU খাচ্ছে	`pidstat` / `top` এ process ধরো, `perf top -p` দিয়ে কোন ফাংশন
load বেশি, `wa` বেশি, CPU অলস	Disk bottleneck	`iostat -x` এ disk, `iotop` এ process
`sy` অস্বাভাবিক বেশি	অতিরিক্ত syscall/context switch	`strace -c -p`, `vmstat` এর `cs`
`st` বেশি	Cloud host এ CPU চুরি হচ্ছে	ক্লাউড প্রোভাইডারের সাথে কথা বলো বা instance বদলাও
`si` / `so` নিয়মিত > 0, `available` কম	Memory সংকট	`ps aux --sort=-rss`, memory leak খোঁজো
`dmesg` এ OOM	Memory শেষ হয়ে process মারা গেছে	ওপরের OOM অংশ দেখো
সব resource স্বাভাবিক, তবু স্লো	বাইরের নির্ভরতা (database, API, DNS) বা lock	`strace -p` এ কোথায় অপেক্ষা করছে দেখো, অ্যাপের লগ দেখো

💡 **টিপস:** "সব স্বাভাবিক, তবু স্লো" খুব সাধারণ ঘটনা। তখন প্রায়ই সমস্যা অন্য সার্ভারে (ধীর database query, বাইরের API timeout) বা DNS এ। `strace -tt -T -p PID` দিয়ে দেখো কোন `connect()` বা `poll()` এ দীর্ঘ সময় যাচ্ছে।

⚠️ **সতর্কতা:** তদন্তের আগে reboot করবে না। Reboot করলে সমস্যা হয়তো সাময়িক চলে যাবে, কিন্তু সব প্রমাণ (চলমান process, memory এর অবস্থা) মুছে যাবে, আর সমস্যা আবার ফিরে আসবে। আগে উপরের কমান্ডগুলোর আউটপুট সংরক্ষণ করো, তারপর সিদ্ধান্ত নাও।

৪৬.১৫ সাধারণ সমস্যা ও সমাধান

সমস্যা ১: Disk full ("No space left on device")

লক্ষণ: ফাইল লেখা যাচ্ছে না, database বন্ধ হয়ে গেছে, লগইন করতেও সমস্যা।

ধাপ ১: কোন filesystem ভরা?

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        40G   40G     0 100% /
/dev/sda1       1.1G  6.1M  1.1G   1% /boot/efi
```

ধাপ ২: কোথায় জায়গা গেল? (`-x` দিয়ে অন্য filesystem এ যাওয়া বন্ধ রাখো)

```
$ sudo du -xh --max-depth=1 / 2>/dev/null | sort -h | tail -8
1.2G    /usr
2.1G    /home
33G     /var
40G     /

$ sudo du -xh --max-depth=1 /var | sort -h | tail -5
...
31G     /var/log
```

💡 **টিপস:** `ncdu` (`sudo apt install ncdu`) একটা ইন্টারঅ্যাকটিভ টুল, তীর চিহ্ন দিয়ে ডিরেক্টরিতে ঢুকে দেখা যায় কে কত জায়গা নিচ্ছে: `sudo ncdu -x /` । (`du` , `find -size` এর বিস্তারিত অধ্যায় ১২ ও অধ্যায় ৩২ এ।)

ধাপ ৩: সাধারণ অপরাধী ও সমাধান

```
# বড় ফাইল খোঁজো
$ sudo find / -xdev -type f -size +500M -exec ls -lh {} \; 2>/dev/null

# systemd journal কত জায়গা নিচ্ছে, কমাও
$ journalctl --disk-usage
$ sudo journalctl --vacuum-size=500M

# পুরনো প্যাকেজ cache
$ sudo apt clean            # Debian/Ubuntu
$ sudo dnf clean all        # Fedora/RHEL

# Docker থাকলে
$ docker system df
$ docker system prune       # অব্যবহৃত image/container মোছে, সাবধানে!
```

ধাপ ৪: ফাঁদ ১ — মুছে দিলাম, তবু জায়গা খালি হলো না!

তুমি একটা ১০ GB লগ ফাইল `rm` করলে, কিন্তু `df` এখনো ১০০% দেখাচ্ছে। কেন? কারণ কোনো process (যেমন nginx) ফাইলটা এখনো খুলে রেখেছে। লিনাক্সে ফাইল আসলে মোছা হয় তখনই যখন তার **সব নাম (link) মুছে যায় এবং কেউ আর খুলে রাখেনি** (অধ্যায় ১১)।

```
$ sudo lsof +L1
COMMAND  PID     USER   FD   TYPE DEVICE    SIZE/OFF NLINK  NODE NAME
nginx   1021 www-data    5w   REG    8,2 10737418240     0 13021
/var/log/nginx/access.log (deleted)
```

সমাধান: সার্ভিস reload/restart করো (`sudo systemctl reload nginx`), ফাইল ছেড়ে দেবে। ভবিষ্যতে চলমান লগ ফাইল খালি করতে `rm` না করে truncate করো:

```
$ sudo truncate -s 0 /var/log/nginx/access.log
# বা
$ : | sudo tee /var/log/nginx/access.log
```

আর স্থায়ী সমাধান হলো `logrotate` ঠিকমতো কনফিগার করা (অধ্যায় ২৮)।

ধাপ ৫: ফাঁদ ২ — জায়গা আছে, তবু "No space left"!

```
$ df -i
Filesystem      Inodes   IUsed IFree IUse% Mounted on
/dev/sda2      2621440 2621440     0  100% /
```

inode শেষ। প্রতিটা ফাইলের জন্য একটা inode লাগে। লক্ষ লক্ষ ছোট ফাইল (যেমন PHP session ফাইল, mail queue, cache) inode শেষ করে দিতে পারে। কোন ডিরেক্টরিতে সবচেয়ে বেশি ফাইল:

```
$ sudo du --inodes -x / 2>/dev/null | sort -n | tail -5
```

ফাঁদ ৩: ext4 ডিফল্টে ৫% জায়গা root এর জন্য রিজার্ভ রাখে, তাই সাধারণ ইউজার `Avail` 0 দেখলেও root হয়তো লিখতে পারে। বড় ডেটা disk এ এটা কমানো যায়: `sudo tune2fs -m 1 /dev/sdb1` (root filesystem এ করবে না)।

সমস্যা ২: High load average

```
$ uptime
 15:10:01 up 30 days, ... load average: 24.50, 18.20, 9.10
$ nproc
4
```

load ২৪, কিন্তু CPU মাত্র ৪টা। এখন বোঝো load টা **CPU** নাকি **I/O** থেকে:

```
# State অনুযায়ী process গোনো
$ ps -eo state= | sort | uniq -c
    230 S
     22 R
```

```
     19 D
      2 I

# D state (I/O তে আটকে) process গুলো কারা?
$ ps -eo pid,state,wchan:32,cmd | awk '$2=="D"'
  4410 D  io_schedule                  rsync -a /var/www /backup
  ...
```

- **R বেশি** → CPU সমস্যা: `top` / `pidstat` এ process খোঁজো, `perf top` দিয়ে কারণ। সমাধান: অপ্রয়োজনীয় process থামাও, `renice` দিয়ে priority কমাও (অধ্যায় ২৪), কোড optimize, বা CPU বাড়াও।
- **D বেশি** → I/O সমস্যা: `iostat -x` আর `iotop`। সমাধান: ভারী কাজ (backup, re-index) ব্যস্ত সময় থেকে সরাও, `ionice -c3 CMD` দিয়ে idle priority তে চালাও, দ্রুত disk (SSD) নাও। NFS mount সাড়া না দিলেও process D state এ আটকে load বাড়ায়।

```
# backup কে কম priority তে চালানো (CPU ও I/O দুটোতেই)
$ nice -n 19 ionice -c3 rsync -a /var/www /backup
```

সমস্যা ৩: Port already in use ("Address already in use")

```
$ sudo systemctl start nginx
Job for nginx.service failed ...
$ sudo journalctl -u nginx -n 5
... bind() to 0.0.0.0:80 failed (98: Address already in use)
```

কে port 80 দখল করে আছে?

```
$ sudo ss -tulpn | grep ':80 '
tcp   LISTEN 0  511   0.0.0.0:80   0.0.0.0:*   users:(("apache2",pid=889,fd=4))

# বিকল্প উপায়
$ sudo lsof -nP -i :80
$ sudo fuser -v 80/tcp
```

`ss` অপশন	মানে
`-t` / `-u`	TCP / UDP
`-l`	শুধু listening
`-p`	process দেখাও (root লাগে)
`-n`	নাম resolve করো না

সমাধান:

- ভুল করে অন্য সার্ভিস চালু আছে (যেমন apache2 আর nginx দুটোই): `sudo systemctl disable --now apache2`।

- পুরনো process আটকে আছে: আগে স্বাভাবিকভাবে বন্ধ করো (`kill PID`), না হলে শেষ অস্ত্র `kill -9 PID` । `sudo fuser -k 80/tcp` port ব্যবহারকারী সব process মেরে দেয়, সাবধানে।
- নিজের অ্যাপ অন্য port এ চালাও।

📝 **মনে রাখো:** ১০২৪ এর নিচের port (যেমন ৮০, ৪৪৩) এ সাধারণ ইউজার bind করতে পারে না: সেক্ষেত্রে error হবে "Permission denied", "Address already in use" না। এছাড়া প্রোগ্রাম বন্ধ করার পরপরই আবার চালালে কখনো পুরনো কানেকশন `TIME-WAIT` অবস্থায় থাকে; ভালোভাবে লেখা সার্ভার `SO_REUSEADDR` ব্যবহার করে এই সমস্যা এড়ায়।

সমস্যা ৪: Permission denied

এটা দেখতে সহজ কিন্তু কারণ অনেক রকম হতে পারে। ধাপে ধাপে চেক করো:

```
$ cat /var/www/app/storage/logs/app.log
cat: /var/www/app/storage/logs/app.log: Permission denied
```

ধাপ ১: আমি কে?

```
$ id
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),27(sudo)
```

ধাপ ২: পুরো path এর প্রতিটা ধাপের permission (একটা ডিরেক্টরিতে `x` না থাকলেই ভেতরে ঢোকা যায় না)

```
$ namei -l /var/www/app/storage/logs/app.log
f: /var/www/app/storage/logs/app.log
drwxr-xr-x root     root     /
drwxr-xr-x root     root     var
drwxr-xr-x root     root     www
drwxr-x--- www-data www-data app          ← এখানে আটকে গেছে! others এর কোনো অনুমতি নেই
drwxrwxr-x www-data www-data storage
drwxrwxr-x www-data www-data logs
-rw-r--r-- www-data www-data app.log
```

ধাপ ৩: সাধারণ কারণ ও সমাধান

কারণ	কীভাবে বুঝবে	সমাধান
সাধারণ permission/ownership	`ls -l` , `namei -l`	`chmod` / `chown` বা ইউজারকে গ্রুপে যোগ (অধ্যায় ১৬)
গ্রুপে যোগ করলাম, তবু হচ্ছে না	`id` এ নতুন গ্রুপ নেই	logout/login করো, বা `newgrp GROUP`
ACL	`ls -l` এ `+` চিহ্ন, `getfacl FILE`	`setfacl` (অধ্যায় ১৭)
immutable attribute	`lsattr FILE` এ `i`	`sudo chattr -i FILE` (কেন দেওয়া হয়েছিল আগে জানো)

কারণ	কীভাবে বুঝবে	সমাধান
script চলছে না	`ls -l` এ `x` নেই	`chmod +x script.sh`
`noexec` mount	`findmnt -no OPTIONS /tmp` এ `noexec`	অন্য জায়গা থেকে চালাও, বা `bash script.sh`
read-only filesystem	`findmnt` এ `ro`, error এ "Read-only file system"	`dmesg` দেখো (disk error এর কারণে kernel ro করে দিতে পারে)
SELinux (Fedora/RHEL)	permission ঠিক আছে তবু denied; `getenforce` = Enforcing; `sudo ausearch -m avc -ts recent`	`restorecon -Rv PATH`, সঠিক context/boolean (অধ্যায় ৪৫)
AppArmor (Ubuntu/Debian)	`sudo journalctl -k | grep -i apparmor` এ `DENIED`	profile ঠিক করো (অধ্যায় ৪৫)
১০২৪ এর নিচে port bind	সাধারণ ইউজার, port < 1024	root/systemd দিয়ে চালাও বা `CAP_NET_BIND_SERVICE`

⚠️ **সতর্কতা:** "Permission denied" এর সমাধান হিসেবে কখনো `chmod -R 777` করবে না। এটা সমস্যা "সমাধান" করে নিরাপত্তায় বিশাল ফুটো তৈরি করে। আর SELinux error এ `setenforce 0` স্থায়ী সমাধান না; কারণ খুঁজে ঠিক করো।

💡 **টিপস:** কোন ফাইলে আসলে permission আটকাচ্ছে বুঝতে না পারলে: `strace -f -e trace=%file CMD 2>&1 | grep EACCES`।

হাতে-কলমে অনুশীলন

> এই ল্যাবগুলো নিজের টেস্ট মেশিন বা VM এ করো, production সার্ভারে না। শুরুতে প্রয়োজনীয় টুল ইনস্টল করো:
>
> ```
> $ sudo apt install sysstat iotop htop btop lsof strace stress-ng   # Ubuntu/Debian
> $ sudo dnf install sysstat iotop htop btop lsof strace stress-ng   # Fedora
> ```

ল্যাব ১: CPU চাপ তৈরি করে ধরা

```
# টার্মিনাল ১: ২টা CPU core এ ৬০ সেকেন্ড চাপ দাও
$ stress-ng --cpu 2 --timeout 60s

# টার্মিনাল ২:
$ uptime                  # load কয়েক সেকেন্ড পরে বাড়তে থাকবে (১ মিনিটের মান)
$ mpstat -P ALL 1 3       # কোন ২টা core ১০০%?
$ pidstat 1 3             # stress-ng process দেখো
$ top                     # 1 চাপো (প্রতি core), P চাপো, তারপর q
```

প্রশ্ন: `top` এ stress-ng এর `%CPU` কত দেখাচ্ছে? `Shift+I` চাপলে কী বদলায়?

ল্যাব ২: Memory বোঝা

```
$ free -h                           # available নোট করো
$ cat /usr/bin/* > /dev/null 2>&1   # অনেক ফাইল পড়ো (cache ভরবে)
$ free -h                           # buff/cache বেড়েছে, available প্রায় একই আছে কি?
$ grep -E 'MemAvailable|^Cached|Dirty' /proc/meminfo
$ vmstat 1 5                        # si/so কলাম দেখো
```

প্রশ্ন: `free` কমে গেলেও `available` কেন প্রায় একই রইল?

ল্যাব ৩: Disk I/O চাপ

```
# টার্মিনাল ১: I/O চাপ (৬০ সেকেন্ড)
$ stress-ng --hdd 1 --timeout 60s

# টার্মিনাল ২:
$ iostat -dxz 1 5        # কোন disk এ w/s, w_await, %util বাড়ছে?
$ sudo iotop -o          # কোন process? (q দিয়ে বের হও)
$ vmstat 1 5             # wa আর b কলাম
$ cat /proc/pressure/io
```

ল্যাব ৪: মুছে ফেলা কিন্তু খোলা ফাইল

```
$ dd if=/dev/zero of=/tmp/big.bin bs=1M count=200     # ২০০ MB ফাইল
$ df -h /tmp
$ tail -f /tmp/big.bin > /dev/null &                   # ফাইলটা খুলে রাখো
$ rm /tmp/big.bin
$ df -h /tmp                                           # জায়গা ফেরত আসেনি!
$ lsof +L1 | grep big.bin                              # ধরা পড়ল
$ kill %1                                              # tail বন্ধ করো
$ df -h /tmp                                           # এখন জায়গা খালি
```

(নোট: কিছু সিস্টেমে `/tmp` হলো `tmpfs`, মানে RAM; তখন `df -h /tmp` ছাড়াও `free -h` এর `shared` এ পরিবর্তন দেখবে।)

ল্যাব ৫: Port in use

```
$ python3 -m http.server 8080 &         # port 8080 দখল করো
$ python3 -m http.server 8080           # দ্বিতীয়বার → "Address already in use"
$ ss -tlpn | grep 8080                  # কে দখল করেছে?
$ lsof -nP -i :8080
$ kill $(lsof -t -i :8080)              # মুক্ত করো
```

ল্যাব ৬: strace দিয়ে গোয়েন্দাগিরি

```
$ strace -e trace=openat ls /nonexistent 2>&1 | tail -3
$ strace -c ls -R /usr/share/doc > /dev/null      # সারাংশে কোন syscall সবচেয়ে বেশি?
$ strace -f -e trace=%network curl -s https://example.com -o /dev/null 2>&1 | grep
connect
```

প্রশ্ন: `curl` কোন IP তে `connect()` করল?

ল্যাব ৭: sar চালু করে ইতিহাস দেখা

```
$ sudo systemctl enable --now sysstat
$ systemctl list-timers | grep sysstat
# অন্তত ২০-৩০ মিনিট অপেক্ষা করো, তারপর:
$ sar -u
$ sar -r
$ sar -q
```

ল্যাব ৮: পুরো চেকলিস্ট একটা স্ক্রিপ্টে

নিচের স্ক্রিপ্টটা `~/perfcheck.sh` নামে সেভ করো, `chmod +x` দাও, চালাও। (শেল স্ক্রিপ্টিং অধ্যায় ৪১ থেকে।)

```
#!/usr/bin/env bash
# perfcheck.sh — সার্ভার স্লো হলে প্রথম ৬০ সেকেন্ডের তথ্য সংগ্রহ
out="/tmp/perfcheck-$(date +%F-%H%M%S).txt"
{
  echo "== uptime ==";           uptime; echo "CPUs: $(nproc)"
  echo "== dmesg (last 20) ==";  sudo dmesg -T | tail -20
  echo "== vmstat ==";           vmstat 1 5
  echo "== mpstat ==";           mpstat -P ALL 1 2
  echo "== pidstat ==";          pidstat 1 2
  echo "== iostat ==";           iostat -xz 1 2
  echo "== free ==";             free -h
  echo "== sar net ==";          sar -n DEV 1 2
  echo "== df ==";               df -h; df -i
  echo "== pressure ==";         grep . /proc/pressure/*
  echo "== top ==";              top -b -n 1 | head -25
} > "$out" 2>&1
echo "রিপোর্ট সেভ হয়েছে: $out"
```

সারসংক্ষেপ

মূল কমান্ড

কমান্ড	কাজ	USE এর কোন অংশ
`uptime`	uptime ও load average	CPU+I/O saturation

কমান্ড	কাজ	USE এর কোন অংশ
`cat /proc/loadavg`	load এর কাঁচা ডেটা	saturation
`top` / `htop` / `btop`	লাইভ সার্বিক চিত্র	সব
`mpstat -P ALL 1`	প্রতি core এর ব্যবহার	CPU utilization
`pidstat 1` / `-d` / `-r`	প্রতি process CPU / disk / memory	কে দায়ী
`free -h`	memory সারাংশ (`available` দেখো!)	memory utilization
`cat /proc/meminfo`	memory এর বিস্তারিত	memory
`vmstat 1`	r, b, si/so, wa এক লাইনে	CPU/mem/I/O saturation
`swapon --show`	swap ডিভাইস	memory
`dmesg -T | grep -i oom`	OOM ঘটনা	memory errors
`iostat -xz 1`	disk latency/queue/util	disk U+S
`iotop -o`	কোন process disk ব্যবহার করছে	disk
`iftop -i IFACE -nP`	কানেকশন ভিত্তিক bandwidth	network
`nethogs IFACE`	process ভিত্তিক bandwidth	network
`ip -s link`	interface errors/drops	network errors
`sar -u/-r/-q/-d/-n DEV`	ইতিহাস ও লাইভ পরিসংখ্যান	সব
`cat /proc/pressure/*`	PSI চাপ	saturation
`lsof -i :PORT` / `+L1` / `-p PID`	খোলা socket/ফাইল	ট্রাবলশুটিং
`ss -tulpn`	listening port ও process	port সমস্যা
`strace -p PID -c` / `-e trace=`	system call দেখা	গভীর তদন্ত
`ltrace`	library call দেখা	গভীর তদন্ত
`perf top` / `perf record -g` / `perf report`	CPU profiling	CPU কোথায় যাচ্ছে
`df -h` / `df -i` / `du -xh` / `ncdu`	disk জায়গা ও inode	disk space
`namei -l PATH`	path এর প্রতি ধাপের permission	permission denied
`nice` / `ionice -c3`	কম priority তে চালানো	load কমানো

`top` এর জরুরি শর্টকাট

কী	কাজ	কী	কাজ
`P`	CPU দিয়ে সাজাও	`1`	প্রতি core
`M`	memory দিয়ে সাজাও	`c`	পুরো command

কী	কাজ	কী	কাজ
`T`	সময় দিয়ে সাজাও	`H`	thread
`k`	kill	`V`	tree
`r`	renice	`u`	ইউজার ফিল্টার
`Shift+I`	Irix mode টগল	`W`	সেটিং সেভ
`h`	হেল্প	`q`	বের হও

`htop` / `btop` / `iotop` / `iftop` দ্রুত মনে রাখো

টুল	সার্চ/ফিল্টার	সাজানো	kill	tree	বের হওয়া
`htop`	`F3` / `F4`	`F6`	`F9`	`F5`	`F10` / `q`
`btop`	`f`	`←` `→`	`k`	`e`	`q`
`iotop`	—	`←` `→`	—	—	`q`
`iftop`	`l`	`1` `2` `3`	—	—	`q`

নিজেকে যাচাই করো

1. USE পদ্ধতির তিনটা অক্ষর কী বোঝায়? কোনটা সবচেয়ে বিপদের সংকেত এবং কেন?
2. একটা ৪ core সার্ভারে `load average: 6.10, 5.80, 2.30` দেখলে কী বুঝবে?
3. লিনাক্সে load average বেশি, অথচ `top` এ CPU ৮০% idle। এটা কীভাবে সম্ভব?
4. `free -h` এ `free` 200Mi কিন্তু `available` 5Gi। সিস্টেমে কি memory সংকট?
5. `vmstat` এর কোন কলাম দেখে বুঝবে সিস্টেম thrashing করছে?
6. একটা সার্ভিস হঠাৎ মারা গেছে, লগে কিছু নেই। OOM killer দায়ী কিনা কীভাবে নিশ্চিত হবে?
7. SSD তে `%util` ১০০% মানেই কি disk এর ক্ষমতা শেষ? কোন কলাম বেশি নির্ভরযোগ্য?
8. ১০ GB লগ ফাইল মুছে দেওয়ার পরও `df` জায়গা দেখাচ্ছে না। কারণ ও সমাধান কী?
9. `df -h` এ ৪০% খালি, তবু "No space left on device" আসছে। কী চেক করবে?
10. গত রাত ৩টায় সার্ভার স্লো ছিল, এখন সব ঠিক। কোন টুল দিয়ে তদন্ত করবে, আর সেটা কাজ করার পূর্বশর্ত কী?

<details>
<summary>▼ উত্তর দেখো</summary>

1. **U**tilization (resource কত % সময় ব্যস্ত), **S**aturation (কত কাজ লাইনে অপেক্ষায়), **E**rrors (কতগুলো error)। Saturation সবচেয়ে সরাসরি বিপদের সংকেত, কারণ এর মানে কাজ অপেক্ষা করছে, অর্থাৎ ইউজার দেরি টের পাচ্ছে; শুধু উচ্চ utilization মানেই সমস্যা না। (Errors উপেক্ষা করা চলবে না, কারণ সেগুলো প্রায়ই নীরবে পারফরম্যান্স নষ্ট করে।)
2. গত ১ মিনিটে গড়ে ~৬টা task চলছিল বা অপেক্ষায় ছিল, ৪ core এর চেয়ে বেশি, তাই saturation আছে। ১৫ মিনিটের মান (2.30) কম, অর্থাৎ load সম্প্রতি **বাড়ছে**। সমস্যা এই মাত্র শুরু, এখনই তদন্ত
</details>

করো।

3. লিনাক্সের load average এ D state (uninterruptible sleep, সাধারণত I/O এর অপেক্ষা) এর task ও গোনা হয়। ধীর disk বা আটকে যাওয়া NFS এর কারণে অনেক process D state এ থাকলে CPU অলস থাকলেও load বেশি হয়। `vmstat` এর `b`, `wa` আর `iostat -x` দেখো।
4. না। লিনাক্স খালি RAM দিয়ে ফাইল cache রাখে, যা দরকার হলে ছেড়ে দেয়। `available` (5Gi) বলে নতুন কাজের জন্য swap ছাড়াই আনুমানিক ৫ GiB পাওয়া যাবে। এটাই আসল মাপ।
5. `si` আর `so` (swap in/out প্রতি সেকেন্ড) নিয়মিত শূন্যের উপরে থাকলে; সাথে সাধারণত `wa` আর `b` বাড়ে।
6. `sudo dmesg -T | grep -i -E 'out of memory|killed process'` বা `journalctl -k | grep -i oom`। systemd-oomd থাকলে `journalctl -u systemd-oomd` ও দেখো। সার্ভিসের ক্ষেত্রে `systemctl status SERVICE` এ `oom-kill` result দেখাতে পারে।
7. না। SSD/NVMe এবং RAID একসাথে অনেক request সমান্তরালে সামলাতে পারে; `%util` শুধু বলে অন্তত একটা request চলছিল। `r_await` / `w_await` (latency) আর `aqu-sz` (queue) বেশি নির্ভরযোগ্য।
8. কোনো process ফাইলটা এখনো খুলে রেখেছে, তাই inode আর data block মুক্ত হয়নি। `sudo lsof +L1` দিয়ে process খুঁজে সার্ভিসটা reload/restart করো। ভবিষ্যতে চলমান লগ `rm` না করে `truncate -s 0` করো এবং logrotate ঠিক রাখো।
9. `df -i` দিয়ে inode। লক্ষ লক্ষ ছোট ফাইল inode শেষ করে দিতে পারে। `du --inodes -x /` দিয়ে কোন ডিরেক্টরিতে ফাইল বেশি খোঁজো। এছাড়া read-only mount বা ext4 এর root-reserved জায়গাও কারণ হতে পারে।
10. `sar` (sysstat), যেমন `sar -u -s 02:30:00 -e 03:30:00`, `sar -q`, `sar -r`, `sar -d`, দরকার হলে `-f /var/log/sysstat/saDD` (RHEL এ `/var/log/sa/saDD`)। পূর্বশর্ত: sysstat ইনস্টল ও ডেটা সংগ্রহ আগে থেকেই চালু থাকতে হবে (`systemctl enable --now sysstat`, Debian/Ubuntu তে `ENABLED="true"`)। সাথে ওই সময়ের `journalctl --since/--until` লগও দেখো।

অধ্যায় ৪৭: কার্নেল, মডিউল, /proc ও /sys

> "লিনাক্স" শব্দটা আসলে মূলত একটা জিনিসের নাম, সেটা হলো **kernel**। বাকি সব (shell, desktop, `ls`, `grep`) kernel-এর ওপরে বসানো সফটওয়্যার। এই অধ্যায়ে আমরা সেই kernel-এর একদম কাছে যাব: কোন kernel চলছে, কীভাবে নতুন driver লোড হয়, kernel নিজের ভেতরের খবর কোথায় রাখে, আর চলন্ত অবস্থাতেই kernel-এর আচরণ কীভাবে বদলানো যায়।

এই অধ্যায়ে যা শিখবে

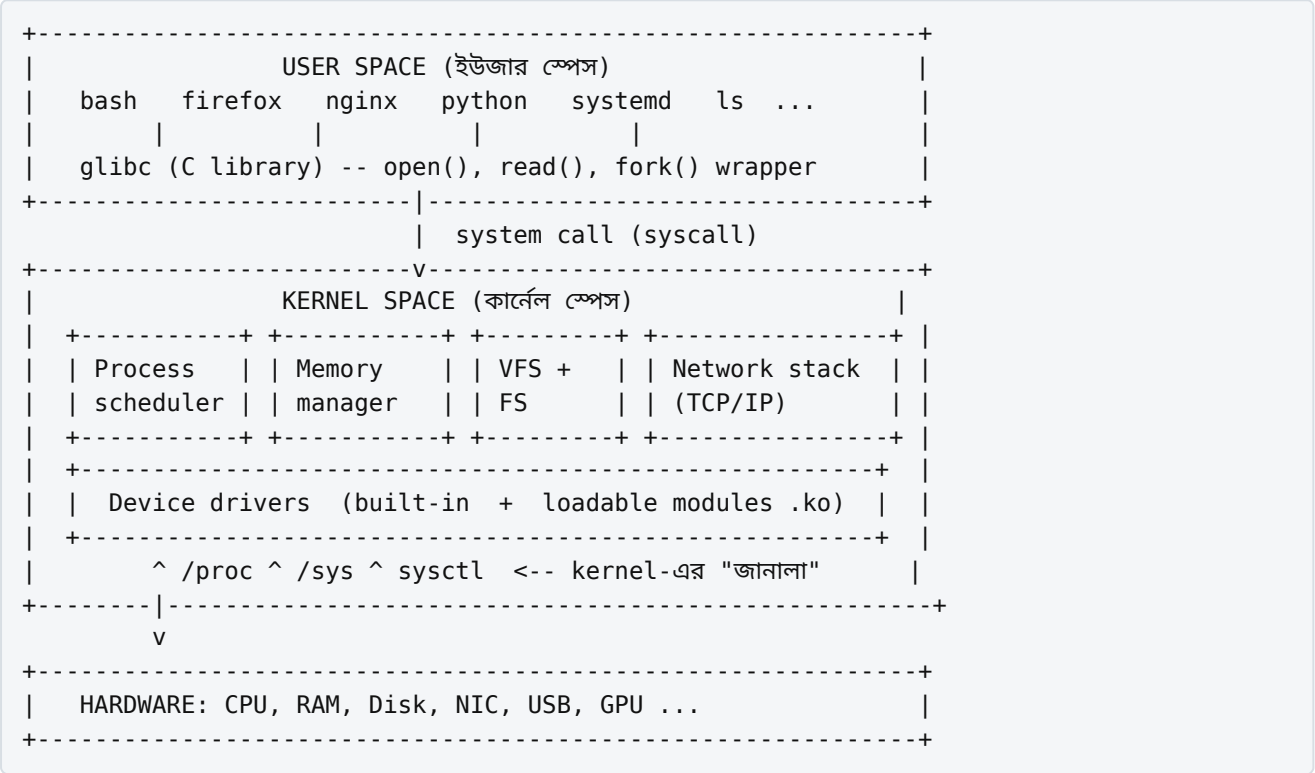
- kernel কী, এবং kernel version (`uname -r`) পড়ার নিয়ম ও নামকরণের মানে
- monolithic kernel আর loadable module-এর ধারণা
- module নিয়ে কাজ: `lsmod`, `modinfo`, `modprobe`, `insmod`, `rmmod`, `depmod`
- boot-এর সময় module লোড করা (`/etc/modules-load.d`), module option দেওয়া এবং blacklist করা
- `/proc` ফাইলসিস্টেম: `cpuinfo`, `meminfo`, প্রতিটি process-এর PID ডিরেক্টরি এবং আরও অনেক কিছু
- `/sys` (sysfs): ডিভাইস, driver ও kernel object-এর গাছ
- `sysctl` দিয়ে kernel-এর runtime প্যারামিটার দেখা ও বদলানো (`-a`, `-w`, `/etc/sysctl.d`)
- udev: ডিভাইস প্লাগ করলে কী ঘটে, আর নিজের udev rule কীভাবে লিখতে হয়
- হার্ডওয়্যারের তথ্য বের করা: `lscpu`, `lspci`, `lsusb`, `dmidecode`, `lshw` ইত্যাদি
- kernel আপডেট, একাধিক kernel রাখা, পুরোনো kernel-এ boot করা
- DKMS: kernel আপডেটের পরও third-party driver যেন কাজ করে
- kernel panic ও oops কী, কেন হয়, আর তখন কী করতে হয়

৪৭.১ kernel আসলে কী — এক নজরে

অধ্যায় ২-এ আমরা লিনাক্সের স্তরগুলো দেখেছি। এখানে শুধু মনে করিয়ে দিই।

Kernel (বাংলায় বলা যায় "কেন্দ্রীয় শাঁস") হলো সেই প্রোগ্রাম যেটা সবার আগে মেমরিতে লোড হয়, আর কম্পিউটার বন্ধ না হওয়া পর্যন্ত চলতে থাকে। হার্ডওয়্যারের ওপর সরাসরি নিয়ন্ত্রণ শুধু এর হাতেই থাকে।

উপমা: একটা বড় হোটেলের কথা ভাবো। অতিথিরা (process/অ্যাপ্লিকেশন) রান্নাঘরে, স্টোররুমে বা ইলেকট্রিক রুমে নিজেরা ঢুকতে পারে না। তারা রিসেপশনে (system call) গিয়ে অনুরোধ করে, আর হোটেলের স্টাফ (kernel) কাজটা করে দেয়। এতে কোনো অতিথি অন্যের রুমে ঢুকতে পারে না, আর কেউ ভুল করে পুরো হোটেলের বিদ্যুৎ বন্ধও করে দিতে পারে না।



এই অধ্যায়ের মূল কথা ডায়াগ্রামের নিচের দিকের ওই "জানালা"গুলো: `/proc`, `/sys` আর `sysctl`। এগুলো দিয়ে user space থেকে kernel-এর ভেতরে উঁকি দেওয়া যায়, কিছু ক্ষেত্রে বদলানোও যায়।

Monolithic, তবু modular

লিনাক্স kernel হলো **monolithic kernel**। মানে scheduler, memory manager, filesystem, network stack আর driver সব একই address space-এ, একই বড় প্রোগ্রাম হিসেবে চলে। এর উল্টোটা **microkernel** (যেমন MINIX, QNX), যেখানে driver আলাদা process হিসেবে চলে।

monolithic হওয়ার সুবিধা হলো গতি, কারণ ভেতরের অংশগুলো একে অপরকে সরাসরি function call করতে পারে। অসুবিধা হলো, পৃথিবীর সব হার্ডওয়্যারের driver যদি একটা ফাইলে ঢুকিয়ে দেওয়া হয়, kernel বিশাল হয়ে যাবে। এর সমাধান হলো **Loadable Kernel Module (LKM)**। দরকারি অংশ kernel-এর ভেতরে **built-in** থাকে, আর বাকি driver আলাদা `.ko` (kernel object) ফাইলে থাকে। দরকার হলে চলন্ত kernel-এ এগুলো জুড়ে দেওয়া যায় (load), আবার খুলেও ফেলা যায় (unload)।

> 📝 **মনে রাখো:** module লোড হলে সেটা kernel space-এ চলে, পূর্ণ ক্ষমতা নিয়ে। একটা খারাপ module পুরো সিস্টেম ক্র্যাশ করাতে পারে। এজন্যই module লোড/আনলোড করতে root লাগে।

৪৭.২ kernel version: `uname` ও নামকরণ

`uname` কমান্ড

`uname` (unix name) চলমান kernel আর মেশিনের মৌলিক তথ্য দেখায়।

Syntax: `uname [OPTION]`

অপশন	মানে
`-s`	kernel-এর নাম (`Linux`), অপশন না দিলেও এটাই দেখায়
`-n`	hostname (নেটওয়ার্কে মেশিনের নাম)
`-r`	kernel release, সবচেয়ে বেশি ব্যবহৃত
`-v`	kernel version string (build নম্বর ও তারিখ)
`-m`	মেশিন হার্ডওয়্যার আর্কিটেকচার (`x86_64`, `aarch64`)
`-o`	operating system (`GNU/Linux`)
`-a`	সব একসাথে

```
$ uname -r
6.8.0-45-generic

$ uname -m
x86_64

$ uname -a
Linux mypc 6.8.0-45-generic #45-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug 30 12:02:04 UTC 2024
x86_64 x86_64 x86_64 GNU/Linux
```

একই তথ্য অন্য জায়গা থেকেও পাওয়া যায়:

```
$ cat /proc/version
Linux version 6.8.0-45-generic (buildd@lcy02-amd64-075) (x86_64-linux-gnu-gcc-13 ...)
#45-Ubuntu SMP PREEMPT_DYNAMIC ...

$ cat /proc/sys/kernel/osrelease
6.8.0-45-generic

$ hostnamectl | grep -i kernel
          Kernel: Linux 6.8.0-45-generic
```

version নম্বর কীভাবে পড়বে

upstream (Linus Torvalds-এর kernel.org) kernel-এর version তিন ভাগে: `MAJOR.MINOR.PATCH`, যেমন `6.8.12`।

- **6**: major। এটা বাড়ার পেছনে বড় কোনো প্রযুক্তিগত কারণ থাকে না; minor নম্বর অনেক বড় হয়ে গেলে Linus এটা বাড়ান (3.19 থেকে 4.0, 4.20 থেকে 5.0, 5.19 থেকে 6.0)।
- **8**: minor। প্রতি ৯–১০ সপ্তাহে একটা নতুন minor release আসে, তাতে নতুন feature থাকে।
- **12**: patch বা stable release। শুধু bug fix ও security fix।

Distribution-গুলো এর পেছনে নিজেদের অংশ জুড়ে দেয়:

```
6 . 8 . 0 - 45 - generic          (Ubuntu 24.04)
|   |   |   |     |
|   |   |   |     +-- flavour: generic / lowlatency / aws / azure ...
|   |   |   +-------- Ubuntu-র ABI/build নম্বর (প্যাকেজ আপডেটে বাড়ে)
|   |   +------------ Ubuntu এখানে upstream patch নম্বর না বসিয়ে 0 রাখে
+---+---------------- upstream 6.8 সিরিজের ওপর ভিত্তি করে বানানো

6 . 1 . 0 - 25 - amd64            (Debian 12)
5 . 14 . 0 - 427.13.1.el9_4 . x86_64   (RHEL 9 / Rocky / Alma)
6 . 10 . 6 - 200.fc40 . x86_64      (Fedora 40)
```

> 💡 **টিপস:** RHEL 9-এর kernel `5.14` দেখে অনেকে ভাবে এটা "পুরোনো"। আসলে Red Hat পুরো RHEL 9-এর জীবনকাল ধরে 5.14 ভিত্তি ধরে রাখে, কিন্তু নতুন kernel থেকে হাজার হাজার fix আর feature **backport** করে (পেছনে টেনে আনে)। তাই শুধু নম্বর দেখে feature আছে কিনা বলা যায় না।

kernel-এর ধরন: mainline, stable, LTS

ধরন	মানে
mainline	Linus-এর সরাসরি ট্রি, `-rc1`, `-rc2` ... (release candidate)
stable	প্রকাশিত version-এর bug fix সিরিজ, অল্প কিছুদিন চলে
longterm (LTS)	কয়েক বছর fix পায়, যেমন 5.15, 6.1, 6.6, 6.12
distro kernel	distro নিজে patch করে, পরীক্ষা করে প্যাকেজ বানায়; সাধারণত এটাই ব্যবহার করা উচিত

kernel ফাইলগুলো কোথায় থাকে

```
$ ls /boot
config-6.8.0-45-generic      initrd.img-6.8.0-45-generic   vmlinuz-6.8.0-45-generic
config-6.8.0-41-generic      initrd.img-6.8.0-41-generic   vmlinuz-6.8.0-41-generic
System.map-6.8.0-45-generic  grub/  efi/ ...
```

ফাইল	কী
`vmlinuz-<ver>`	সংকুচিত (compressed) kernel image। "z" মানে compressed
`initrd.img-<ver>` (Debian/Ubuntu) / `initramfs-<ver>.img` (Fedora/RHEL)	প্রাথমিক RAM ডিস্ক: আসল root ফাইলসিস্টেম মাউন্ট করার আগে যে driver লাগে সেগুলো এখানে থাকে

ফাইল	কী
`config-<ver>`	এই kernel কোন অপশন নিয়ে compile হয়েছে
`System.map-<ver>`	kernel symbol-এর ঠিকানার তালিকা (debug-এর কাজে)
`/lib/modules/<ver>/`	এই kernel-এর সব module

কোনো feature built-in নাকি module, `config` ফাইল থেকে দেখা যায়:

```
$ grep -E 'CONFIG_EXT4_FS=|CONFIG_BTRFS_FS=' /boot/config-$(uname -r)
CONFIG_EXT4_FS=y
CONFIG_BTRFS_FS=m
```

`=y` মানে built-in (kernel-এর ভেতরেই আছে), `=m` মানে module হিসেবে বানানো, আর `# ... is not set` মানে একেবারেই নেই। boot ও kernel image-এর পুরো যাত্রা নিয়ে অধ্যায় ২৭-এ বিস্তারিত বলা আছে।

৪৭.৩ kernel module

module ফাইল কোথায় থাকে

```
/lib/modules/6.8.0-45-generic/
├── kernel/
│   ├── drivers/        <- net, usb, gpu, block ... driver
│   ├── fs/             <- btrfs, xfs, nfs ... filesystem
│   ├── net/            <- netfilter, bridge ... network feature
│   ├── sound/
│   └── crypto/
├── modules.dep         <- কোন module কোনটার ওপর নির্ভরশীল
├── modules.alias       <- হার্ডওয়্যার ID থেকে module-এর নাম
├── modules.builtin     <- যেগুলো built-in (লোড করার দরকার নেই)
└── updates/ , extra/   <- DKMS বা third-party module
```

ফাইলের extension distro অনুযায়ী আলাদা হয়: `.ko`, `.ko.xz` (RHEL/Fedora), অথবা `.ko.zst` (Ubuntu 24.04)। শেষের দুটো compressed। `modprobe` নিজেই এগুলো খুলে নেয়।

> 📝 **মনে রাখো:** আধুনিক distro-তে `/lib` আসলে `/usr/lib`-এর symlink ("merged /usr"), তাই `/usr/lib/modules` আর `/lib/modules` একই জায়গা।

`lsmod`: কোন module লোড আছে

`lsmod` আসলে `/proc/modules` ফাইলটাকে সুন্দর করে দেখায়।

```
$ lsmod | head -6
Module                  Size  Used by
```

```
snd_hda_intel          61440  4
snd_intel_dspcfg       36864  1 snd_hda_intel
snd_hda_codec         204800  2 snd_hda_codec_hdmi,snd_hda_intel
iwlwifi               598016  1 iwlmvm
cfg80211             1355776  3 iwlmvm,iwlwifi,mac80211
```

কলাম	মানে
`Module`	module-এর নাম
`Size`	মেমরিতে কত বাইট নিয়েছে
`Used by`	কতজন ব্যবহার করছে (সংখ্যা), আর কোন module-গুলো এর ওপর নির্ভরশীল

`Used by`-এর সংখ্যা ০ না হলে module-টা সাধারণত আনলোড করা যায় না।

```
$ lsmod | grep -i usb_storage
usb_storage            86016  1 uas
```

> 💡 **টিপস:** built-in অংশ `lsmod`-এ দেখা যায় না, কারণ সেগুলো আলাদা module হিসেবে লোডই হয় না। দেখতে চাইলে: `grep ext4 /lib/modules/$(uname -r)/modules.builtin`।

`modinfo`: module-এর পরিচয়পত্র

```
$ modinfo e1000e
filename:       /lib/modules/6.8.0-45-
generic/kernel/drivers/net/ethernet/intel/e1000e/e1000e.ko.zst
license:        GPL v2
description:    Intel(R) PRO/1000 Network Driver
author:         Intel Corporation, <linux.nics@intel.com>
alias:          pci:v00008086d0000550Esv*sd*bc*sc*i*
...
depends:
intree:         Y
name:           e1000e
vermagic:       6.8.0-45-generic SMP preempt mod_unload modversions
sig_id:         PKCS#7
signer:         Build time autogenerated kernel key
parm:           debug:Debug level (0=none,...,16=all) (int)
parm:           IntMode:Interrupt Mode (array of int)
```

ফিল্ড	গুরুত্ব
`filename`	ডিস্কে ফাইলটা কোথায়
`description`, `author`, `license`	পরিচয়। license GPL না হলে kernel "tainted" চিহ্নিত হয়
`alias`	কোন হার্ডওয়্যার ID দেখলে এই module লোড হবে
`depends`	আগে কোন module লোড থাকতে হবে
`vermagic`	কোন kernel-এর জন্য বানানো; না মিললে লোড হবে না

ফিল্ড	গুরুত্ব
`signer`, `sig_id`	digital signature (Secure Boot-এ গুরুত্বপূর্ণ)
`parm`	কোন কোন প্যারামিটার দেওয়া যায়

কাজের অপশন:

অপশন	কাজ
`-F field`	শুধু একটা ফিল্ড, যেমন `modinfo -F parm e1000e`
`-p`	শুধু প্যারামিটারগুলো
`-k <ver>`	অন্য kernel version-এর module দেখো

`modprobe`: বুদ্ধিমান লোডার

`modprobe` হলো module লোড/আনলোডের সঠিক উপায়। এটা `modules.dep` পড়ে আগে dependency লোড করে নেয়, তারপর আসল module। আর `/etc/modprobe.d/`-এর কনফিগও মেনে চলে।

```
$ sudo modprobe -v vfat
insmod /lib/modules/6.8.0-45-generic/kernel/fs/fat/fat.ko.zst
insmod /lib/modules/6.8.0-45-generic/kernel/fs/fat/vfat.ko.zst
```

দেখো, `vfat` চাওয়া হয়েছিল, কিন্তু `modprobe` নিজে থেকেই আগে `fat` লোড করেছে।

অপশন	কাজ
`modprobe <name>`	লোড করো (dependency সহ)
`modprobe <name> param=value`	প্যারামিটার দিয়ে লোড করো
`-r <name>`	আনলোড করো (আর অব্যবহৃত dependency-গুলোও)
`-v`	কী করছে দেখাও (verbose)
`-n`	dry run: আসলে কিছু করবে না (`-n -v` একসাথে খুব কাজের)
`--show-depends <name>`	কোন কোন ফাইল লোড হবে সেটা দেখাও
`-c`	সব কার্যকর কনফিগারেশন দেখাও
`-a`	একাধিক module একসাথে লোড করো

```
$ modprobe --show-depends vfat
insmod /lib/modules/6.8.0-45-generic/kernel/fs/fat/fat.ko.zst
insmod /lib/modules/6.8.0-45-generic/kernel/fs/fat/vfat.ko.zst

$ sudo modprobe -r vfat
$ lsmod | grep fat
$
```

`insmod`, `rmmod`, `depmod`: নিচু স্তরের টুল

কমান্ড	কাজ	সমস্যা
`insmod /path/file.ko`	নির্দিষ্ট ফাইল সরাসরি লোড করে	dependency দেখে না, পুরো path দিতে হয়
`rmmod <name>`	module সরিয়ে দেয়	dependency দেখে না; ব্যবহারে থাকলে ব্যর্থ হয়
`depmod -a`	`modules.dep` ও `modules.alias` আবার বানায়	নতুন module হাতে কপি করার পর চালাতে হয়

```
$ sudo rmmod snd_hda_intel
rmmod: ERROR: Module snd_hda_intel is in use
```

> ⚠️ **সতর্কতা:** `rmmod -f` (force) দিয়ে ব্যবহৃত module জোর করে সরালে সিস্টেম ক্র্যাশ করতে পারে। বেশিরভাগ distro kernel-এ force unload বন্ধও থাকে। সাধারণ কাজে সবসময় `modprobe` / `modprobe -r` ব্যবহার করো।

module স্বয়ংক্রিয়ভাবে কীভাবে লোড হয়

তুমি একটা USB Wi-Fi অ্যাডাপ্টার লাগালে, `modprobe` না চালালেও driver লোড হয়ে যায়। কীভাবে?

```
[USB ডিভাইস লাগানো হলো]
         |
         v
kernel ডিভাইসের ID পড়ে  --> "usb:v0BDAp8179..."  (modalias)
         |
         v
kernel একটা uevent পাঠায় (netlink দিয়ে) ---> systemd-udevd
         |
         v
udevd: "modprobe usb:v0BDAp8179..."  (alias দিয়ে)
         |
         v
modprobe modules.alias খোঁজে --> r8188eu / rtl8xxxu
         |
         v
module লোড হলো, driver ডিভাইসকে "bind" করল --> wlan0 তৈরি
```

যেকোনো ডিভাইসের modalias দেখা যায় `/sys`-এ:

```
$ cat /sys/class/net/enp3s0/device/modalias
pci:v00008086d000015BCsv000017AAsd00002292bc02sc00i00
```

boot-এর সময় module লোড করা: `/etc/modules-load.d`

কিছু module আছে যেগুলো কোনো হার্ডওয়্যার দেখে লোড হয় না, যেমন `br_netfilter` বা `overlay` (container-এর জন্য), কিংবা `loop`। প্রতিবার boot-এ এগুলো লোড করতে `/etc/modules-load.d/` ডিরেক্টরিতে একটা `.conf` ফাইল বানাও, প্রতি লাইনে একটা module-এর নাম:

```
$ sudo tee /etc/modules-load.d/k8s.conf <<'EOF'
# Kubernetes-এর জন্য দরকারি module
overlay
br_netfilter
EOF
```

boot-এর সময় `systemd-modules-load.service` এই ফাইলগুলো পড়ে। এখনই পরীক্ষা করতে চাইলে:

```
$ sudo systemctl restart systemd-modules-load.service
$ lsmod | grep -E 'overlay|br_netfilter'
br_netfilter           32768  0
overlay               212992  0
```

লোকেশন	কে ব্যবহার করে
`/etc/modules-load.d/*.conf`	administrator (তুমি)
`/usr/lib/modules-load.d/*.conf`	প্যাকেজ (হাত দিও না)
`/run/modules-load.d/*.conf`	runtime, অস্থায়ী
`/etc/modules`	Debian/Ubuntu-র পুরোনো পদ্ধতি, এখনও কাজ করে

module option: `/etc/modprobe.d`

module লোড হওয়ার সময় স্থায়ীভাবে কোনো প্যারামিটার দিতে চাইলে `/etc/modprobe.d/` ডিরেক্টরিতে লেখো:

```
# /etc/modprobe.d/sound.conf
options snd_hda_intel power_save=1
```

module এখন লোড থাকলে তার বর্তমান প্যারামিটার দেখা যায় `/sys/module` থেকে:

```
$ cat /sys/module/snd_hda_intel/parameters/power_save
1
```

`/etc/modprobe.d/`-এর ফাইলগুলোতে যেসব নির্দেশনা দেওয়া যায়:

নির্দেশনা	উদাহরণ	মানে
`options`	`options kvm_intel nested=1`	প্যারামিটার দাও
`alias`	`alias mynet e1000e`	অন্য নাম

নির্দেশনা	উদাহরণ	মানে
`blacklist`	`blacklist nouveau`	alias দিয়ে স্বয়ংক্রিয়ভাবে লোড হবে না
`install`	`install usb_storage /bin/false`	লোড চাইলে module-এর বদলে এই কমান্ড চলবে
`softdep`	`softdep nvme pre: crc32c`	আগে/পরে অন্য module লোড করো

blacklist: কোনো module যেন লোড না হয়

উদাহরণ: NVIDIA-র proprietary driver ইনস্টল করার সময় open-source `nouveau` driver আটকাতে হয়।

```
# /etc/modprobe.d/blacklist-nouveau.conf
blacklist nouveau
options nouveau modeset=0
```

এখানে একটা সূক্ষ্ম বিষয় বুঝে রাখো। `blacklist` শুধু **alias দিয়ে স্বয়ংক্রিয় লোড** বন্ধ করে। কেউ হাতে `modprobe nouveau` চালালে, বা অন্য কোনো module এর ওপর নির্ভরশীল হলে, এটা তবুও লোড হবে। পুরোপুরি আটকাতে হলে:

```
# /etc/modprobe.d/disable-usb-storage.conf
install usb_storage /bin/false
```

এটা নিরাপত্তার জন্য প্রচলিত কৌশল: সার্ভারে USB pen drive চলবে না (দেখো অধ্যায় ৪৫)।

গুরুত্বপূর্ণ: অনেক driver boot-এর একদম শুরুতে initramfs থেকে লোড হয়। তাই blacklist করার পর initramfs আবার বানাতে হয়:

```
# Debian / Ubuntu
$ sudo update-initramfs -u

# Fedora / RHEL
$ sudo dracut -f
```

আর সিস্টেম boot-ই না হলে GRUB-এ kernel command line-এ সাময়িকভাবে যোগ করা যায়: `modprobe.blacklist=nouveau`।

> 💡 **টিপস:** কোন blacklist কার্যকর আছে সেটা দেখতে: `modprobe -c | grep -i blacklist`।

৪৭.৪ `/proc`: kernel-এর জীবন্ত জানালা

`/proc` কী

`/proc` একটা **virtual filesystem** (procfs)। এর একটা ফাইলও ডিস্কে নেই। তুমি যখন `cat /proc/meminfo` চালাও, kernel ঠিক তখনই মেমরির বর্তমান অবস্থা হিসাব করে লেখা বানিয়ে তোমাকে দেয়।

উপমা: গাড়ির ড্যাশবোর্ড। স্পিডোমিটারের কাঁটা কোথাও "সংরক্ষিত" থাকে না; তুমি তাকালেই তখনকার গতি দেখায়।

```
$ mount | grep '^proc'
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)

$ ls -l /proc/meminfo
-r--r--r-- 1 root root 0 Nov 12 10:00 /proc/meminfo
```

দেখো, ফাইলের সাইজ **0**। তবু `cat` করলে অনেক লেখা আসে। কারণ লেখাটা পড়ার মুহূর্তে তৈরি হয়।

> 📝 **মনে রাখো:** "Everything is a file" দর্শনের সবচেয়ে সুন্দর উদাহরণ `/proc` আর `/sys`। kernel-এর তথ্য পড়তে আলাদা কোনো API লাগে না; `cat`, `grep`, `awk` দিয়েই কাজ হয়ে যায়।

`/proc/cpuinfo`

প্রতিটি logical CPU-র জন্য একটা করে ব্লক থাকে:

```
$ grep -m1 'model name' /proc/cpuinfo
model name      : Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz

$ grep -c ^processor /proc/cpuinfo
8

$ grep -m1 -o -w -E 'vmx|svm' /proc/cpuinfo
vmx
```

ফিল্ড	মানে
`processor`	logical CPU নম্বর (0 থেকে শুরু)
`model name`	CPU-র নাম
`cpu MHz`	এই মুহূর্তের clock speed (বদলাতে থাকে)
`cpu cores`	একটা physical chip-এ কয়টা core
`siblings`	একটা chip-এ কয়টা logical CPU (hyper-threading সহ)
`flags`	CPU কী কী feature সাপোর্ট করে (`vmx` / `svm` = virtualization, `aes`, `avx2` ...)

> 💡 **টিপস:** CPU সংখ্যা জানার সহজ উপায় `nproc`। আর `/proc/cpuinfo`-এর পুরো তথ্য সাজানো আকারে দেখায় `lscpu` (৪৭.৮ অংশে দেখবো)। ARM মেশিনে (Raspberry Pi, AWS Graviton) `cpuinfo`-র ফিল্ডগুলো অনেক আলাদা।

`/proc/meminfo`

```
$ head -8 /proc/meminfo
MemTotal:       16165604 kB
MemFree:         1203416 kB
MemAvailable:    9876540 kB
Buffers:          412300 kB
Cached:          7890120 kB
SwapCached:          120 kB
Active:          6123456 kB
Inactive:        5987654 kB
```

ফিল্ড	মানে
`MemTotal`	মোট ব্যবহারযোগ্য RAM
`MemFree`	একেবারে খালি RAM (এটা কম হলেও ভয়ের কিছু নেই)
`MemAvailable`	নতুন প্রোগ্রাম আসলে কতটা পেতে পারে (cache ছেড়ে দিয়ে); **আসল হিসাব এটাই**
`Buffers`, `Cached`	ডিস্কের ডেটার cache; দরকার হলে kernel ছেড়ে দেয়
`SwapTotal`, `SwapFree`	swap-এর অবস্থা
`Dirty`	ডিস্কে লেখার অপেক্ষায় থাকা ডেটা
`HugePages_Total`	hugepage (ডেটাবেসের জন্য বড় page)

`free -h` কমান্ড এই ফাইলটাই পড়ে। মেমরি বিশ্লেষণ নিয়ে বিস্তারিত আছে অধ্যায় ৪৬-এ।

`/proc`-এর আরও দরকারি ফাইল

ফাইল	কী দেখায়	সংশ্লিষ্ট কমান্ড
`/proc/version`	kernel version ও build তথ্য	`uname -a`
`/proc/cmdline`	boot-এর সময় kernel-কে দেওয়া প্যারামিটার	
`/proc/uptime`	চালু থাকার সেকেন্ড, idle সেকেন্ড	`uptime`
`/proc/loadavg`	১, ৫, ১৫ মিনিটের load average	`uptime`, `top`
`/proc/stat`	CPU সময়ের হিসাব, context switch	`vmstat`, `top`
`/proc/modules`	লোড হওয়া module	`lsmod`
`/proc/mounts`	মাউন্ট হওয়া ফাইলসিস্টেম (আসলে `self/mounts`-এর link)	`mount`, `findmnt`
`/proc/filesystems`	kernel কোন কোন filesystem চেনে	

ফাইল	কী দেখায়	সংশ্লিষ্ট কমান্ড
`/proc/partitions`	ডিস্ক পার্টিশন	`lsblk`
`/proc/swaps`	সক্রিয় swap	`swapon --show`
`/proc/interrupts`	কোন CPU-তে কোন ডিভাইসের কতগুলো interrupt এসেছে	
`/proc/devices`	character ও block ডিভাইসের major নম্বর	
`/proc/net/`	network-এর অবস্থা (`tcp` , `dev` , `route`)	`ss` , `ip`
`/proc/sys/`	পরিবর্তনযোগ্য kernel প্যারামিটার	`sysctl`
`/proc/sysrq-trigger`	SysRq কমান্ড পাঠানোর জায়গা (৪৭.১১ দেখো)	

```
$ cat /proc/cmdline
BOOT_IMAGE=/boot/vmlinuz-6.8.0-45-generic root=UUID=3f1c...e9 ro quiet splash
vt.handoff=7

$ cat /proc/loadavg
0.52 0.61 0.58 2/1043 245871
```

`/proc/loadavg` -এর চতুর্থ ঘরে `2/1043` মানে এখন ২টা চলছে (runnable), মোট ১০৪৩টা thread আছে। শেষ সংখ্যাটা সবচেয়ে সাম্প্রতিক তৈরি হওয়া PID।

PID ডিরেক্টরি: প্রতিটি process-এর খাতা

`/proc` -এ সংখ্যা নামের যেসব ডিরেক্টরি আছে, প্রতিটা একটা চলন্ত process (সংখ্যাটা তার PID)। process শেষ হলে ডিরেক্টরিটাও মিলিয়ে যায়।

```
$ ls -d /proc/[0-9]* | head -3
/proc/1
/proc/2
/proc/3

$ sleep 1000 &
[1] 24567
$ ls /proc/24567
attr  cgroup  cmdline  comm  cwd  environ  exe  fd  io  limits  maps  mountinfo  ns
oom_score  oom_score_adj  root  stat  statm  status  task ...
```

```
/proc/<PID>/
├── cmdline     কোন কমান্ড দিয়ে চালু হয়েছে (argument গুলো NUL দিয়ে আলাদা)
├── comm        process-এর ছোট নাম
├── status      মানুষের পড়ার মতো সারাংশ: State, PPid, Uid, VmRSS, Threads
├── stat        ps/top-এর জন্য এক লাইনের কাঁচা তথ্য
├── environ     environment variable (NUL দিয়ে আলাদা)
├── cwd  ->     বর্তমান ডিরেক্টরির symlink
├── exe  ->     আসল executable ফাইলের symlink
├── root ->     process-এর root ডিরেক্টরি (chroot/container-এ আলাদা)
├── fd/         খোলা প্রতিটি ফাইলের symlink (0=stdin, 1=stdout, 2=stderr)
├── maps        মেমরিতে কী কী ম্যাপ করা (library, heap, stack)
├── limits      resource limit (ulimit)
```

```
├── io           কত বাইট পড়েছে/লিখেছে
├── ns/          কোন namespace-এ আছে (container বোঝার জন্য, অধ্যায় ৪৮)
├── oom_score_adj  মেমরি ফুরালে একে মারার অগ্রাধিকার (-1000 থেকে 1000)
└── task/        প্রতিটি thread-এর আলাদা ডিরেক্টরি
```

বাস্তব উদাহরণ:

```
$ cat /proc/24567/cmdline | tr '\0' ' '; echo
sleep 1000

$ grep -E 'State|PPid|VmRSS|Threads' /proc/24567/status
State:  S (sleeping)
PPid:   23001
VmRSS:      1792 kB
Threads:        1

$ ls -l /proc/24567/cwd /proc/24567/exe
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:05 /proc/24567/cwd -> /home/rahim
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:05 /proc/24567/exe -> /usr/bin/sleep

$ ls -l /proc/24567/fd
lrwx------ 1 rahim rahim 64 Nov 12 10:05 0 -> /dev/pts/1
lrwx------ 1 rahim rahim 64 Nov 12 10:05 1 -> /dev/pts/1
lrwx------ 1 rahim rahim 64 Nov 12 10:05 2 -> /dev/pts/1
```

`/proc/self`: যে process `/proc` পড়ছে, তার নিজের ডিরেক্টরির link।

```
$ readlink /proc/self
24601          <- এটা ls/readlink process-এর নিজের PID, তাই প্রতিবার আলাদা
```

> 💡 **টিপস (জীবন বাঁচানো কৌশল):** কেউ ভুল করে একটা log ফাইল `rm` করেছে, কিন্তু process-টা এখনও সেটা খোলা রেখে লিখে যাচ্ছে। ডিস্কে জায়গাও খালি হচ্ছে না। ফাইলটা `/proc/<PID>/fd/`-এ `(deleted)` লেখা নিয়ে এখনও দেখা যাবে, আর সেখান থেকে কপি করে উদ্ধার করা যায়: `cp /proc/<PID>/fd/3 ~/recovered.log`।

> ⚠️ **সতর্কতা:** `/proc/<PID>/environ`-এ password বা API key থাকতে পারে। এজন্য অন্য user-এর process-এর `environ`, `fd`, `maps` শুধু root বা ওই user-ই পড়তে পারে।

process নিয়ে বাকি সব কিছু (`ps`, `kill`, signal) আছে অধ্যায় ২৪-এ।

৪৭.৫ `/sys` ও sysfs: ডিভাইসের সাজানো গাছ

`/proc` থাকতে `/sys` কেন

শুরুতে kernel সব তথ্য `/proc`-এ রাখত, ফলে সেখানে process, হার্ডওয়্যার, প্যারামিটার সব মিলে একটা জগাখিচুড়ি তৈরি হয়েছিল। kernel 2.6-এ **sysfs** এলো (`/sys`-এ মাউন্ট হয়)। এর কাজ শুধু একটা: kernel-এর ভেতরের **device model** (ডিভাইস, bus, driver, তাদের সম্পর্ক) একটা সুশৃঙ্খল ডিরেক্টরি-গাছ হিসেবে দেখানো।

নিয়ম: sysfs-এ সাধারণত **এক ফাইলে একটাই মান** থাকে। তাই script দিয়ে পড়া-লেখা খুব সহজ।

উপমা: `/proc` যদি হাসপাতালের জরুরি বিভাগের হোয়াইটবোর্ড হয় (কোন রোগী কোথায়, কী অবস্থা), তবে `/sys` হলো হাসপাতালের বিল্ডিং-এর নকশা: কোন ফ্লোরে কোন ওয়ার্ড, কোন ওয়ার্ডে কোন বেড, কোন ডাক্তার দায়িত্বে।

```
/sys
├── devices/    <- আসল গাছ: প্রতিটি ডিভাইস তার physical অবস্থান অনুযায়ী
│     └── pci0000:00/0000:00:1f.6/net/enp0s31f6 ...
├── bus/        <- bus অনুযায়ী (pci, usb, i2c ...), প্রতিটিতে devices/ ও drivers/
├── class/      <- কাজ অনুযায়ী (net, block, tty, power_supply, backlight ...)
│     └── net/enp0s31f6  --> ../../devices/... (symlink)
├── block/      <- ডিস্ক (sda, nvme0n1) — symlink
├── module/     <- লোড হওয়া module ও তাদের parameters/
├── firmware/   <- ACPI, EFI, DMI তথ্য
├── kernel/     <- kernel-এর কিছু সেটিং (mm/transparent_hugepage ...)
├── fs/         <- filesystem-ভিত্তিক তথ্য, cgroup (fs/cgroup)
└── power/      <- suspend/hibernate নিয়ন্ত্রণ
```

`/sys/class` আর `/sys/block`-এর বেশিরভাগ জিনিস আসলে `/sys/devices`-এর দিকে symlink। একই ডিভাইসকে "কাজ" দিয়ে খুঁজে পেতে এই ব্যবস্থা।

বাস্তব উদাহরণ

```
# নেটওয়ার্ক কার্ডের MAC address, অবস্থা, গতি
$ cat /sys/class/net/enp0s31f6/address
8c:16:45:aa:bb:cc
$ cat /sys/class/net/enp0s31f6/operstate
up
$ cat /sys/class/net/enp0s31f6/speed
1000

# ল্যাপটপের ব্যাটারি
$ cat /sys/class/power_supply/BAT0/capacity
87
$ cat /sys/class/power_supply/BAT0/status
Discharging

# ডিস্ক SSD নাকি HDD (0 = SSD, 1 = ঘূর্ণায়মান HDD)
$ cat /sys/block/nvme0n1/queue/rotational
0
```

```
# ডিস্কের I/O scheduler (বন্ধনীর ভেতরেরটা সক্রিয়)
$ cat /sys/block/sda/queue/scheduler
none [mq-deadline] kyber bfq

# তাপমাত্রা (মিলি-ডিগ্রি সেলসিয়াসে)
$ cat /sys/class/thermal/thermal_zone0/temp
48000

# কোন নেটওয়ার্ক কার্ড কোন driver ব্যবহার করছে
$ readlink /sys/class/net/enp0s31f6/device/driver
../../../../bus/pci/drivers/e1000e
```

`/sys`-এ লেখা

কিছু ফাইল লেখার যোগ্য, এবং লিখলে সঙ্গে সঙ্গে আচরণ বদলায়:

```
# স্ক্রিনের উজ্জ্বলতা (মান হার্ডওয়্যার অনুযায়ী আলাদা)
$ cat /sys/class/backlight/intel_backlight/max_brightness
19393
$ echo 9000 | sudo tee /sys/class/backlight/intel_backlight/brightness

# ডিস্কের scheduler বদলানো
$ echo bfq | sudo tee /sys/block/sda/queue/scheduler
```

> ⚠️ **সতর্কতা:** `sudo echo 5000 > /sys/...` কাজ করবে না! কারণ redirection (`>`) তোমার সাধারণ shell করে, `sudo` নয়, তাই "Permission denied" আসবে। সবসময় `echo ... | sudo tee /path` ব্যবহার করো (অধ্যায় ১৮ ও অধ্যায় ১৯ দেখো)।

> 📝 **মনে রাখো:** `/proc` ও `/sys`-এ লেখা পরিবর্তন **reboot হলেই হারিয়ে যায়**, কারণ এগুলো RAM-এ থাকা kernel-এর অবস্থা। স্থায়ী করতে `sysctl.d`, `modprobe.d`, বা udev rule ব্যবহার করতে হয়।

`/proc` বনাম `/sys` বনাম `/dev`

	`/proc`	`/sys`	`/dev`
filesystem	procfs	sysfs	devtmpfs
কী আছে	process আর kernel-এর পরিসংখ্যান, `/proc/sys` প্যারামিটার	ডিভাইস, driver, bus-এর গঠন ও attribute	ডিভাইস **ফাইল** (যেটা দিয়ে ডেটা পড়া/লেখা হয়)
উদাহরণ	`/proc/meminfo`	`/sys/block/sda/size`	`/dev/sda`
কে তৈরি করে	kernel	kernel	kernel (devtmpfs), নাম/পারমিশন ঠিক করে udev

---

৪৭.৬ `sysctl`: চলন্ত kernel-এর সেটিং বদলানো

ধারণা

`/proc/sys/` ডিরেক্টরির নিচে শত শত ফাইল আছে, যেগুলো kernel-এর **tunable** (সমন্বয়যোগ্য) প্যারামিটার। `sysctl` কমান্ড এগুলো পড়ে ও লেখে। নামের মধ্যে `/`-এর বদলে `.` ব্যবহার হয়:

```
/proc/sys/net/ipv4/ip_forward    <==>    net.ipv4.ip_forward
/proc/sys/vm/swappiness          <==>    vm.swappiness
```

উপসর্গ	বিষয়
`kernel.*`	kernel-এর সাধারণ আচরণ (hostname, panic, pid_max, sysrq)
`vm.*`	virtual memory (swap, dirty page, overcommit)
`net.*`	network stack (IPv4/IPv6, TCP, buffer)
`fs.*`	filesystem (file handle, inotify)
`dev.*`	ডিভাইস-নির্দিষ্ট

কমান্ড ও অপশন

কমান্ড	কাজ
`sysctl <name>`	একটা মান পড়ো
`sysctl -n <name>`	শুধু মানটা (নাম ছাড়া), script-এর জন্য
`sysctl -a`	সব প্যারামিটার দেখাও
`sudo sysctl -w <name>=<value>`	মান বদলাও (সাময়িক)
`sudo sysctl -p [file]`	ফাইল থেকে লোড করো (ফাইল না দিলে `/etc/sysctl.conf`)
`sudo sysctl --system`	সব কনফিগ ডিরেক্টরি থেকে নিয়ম মেনে লোড করো
`sysctl -a -r <regex>`	pattern দিয়ে খোঁজো

```
$ sysctl vm.swappiness
vm.swappiness = 60

$ sysctl -n net.ipv4.ip_forward
0

$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1

$ cat /proc/sys/net/ipv4/ip_forward      # একই জিনিস
1

$ sudo sysctl -a 2>/dev/null | wc -l
1100                  <- সংখ্যাটা kernel ও লোড হওয়া module অনুযায়ী আলাদা হবে
```

```
$ sysctl -a -r 'tcp_.*keepalive' 2>/dev/null
net.ipv4.tcp_keepalive_intvl = 75
net.ipv4.tcp_keepalive_probes = 9
net.ipv4.tcp_keepalive_time = 7200
```

> 💡 **টিপস:** `sysctl -a` সাধারণ user হিসেবে চালালে কিছু প্যারামিটার পড়ার অনুমতি না থাকায় "permission denied" বার্তা আসে। `2>/dev/null` দিয়ে সেগুলো লুকাও, অথবা `sudo` দাও।

স্থায়ী করা: `/etc/sysctl.d/`

`sysctl -w` দিয়ে করা পরিবর্তন reboot-এ হারায়। স্থায়ী করতে `/etc/sysctl.d/`-এ একটা ফাইল বানাও:

```
$ sudo tee /etc/sysctl.d/99-custom.conf <<'EOF'
# রাউটার/VPN/Docker-এর জন্য প্যাকেট ফরওয়ার্ডিং
net.ipv4.ip_forward = 1
# RAM বেশি থাকলে swap কম ব্যবহার করো
vm.swappiness = 10
EOF

$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-...conf ...
* Applying /etc/sysctl.d/99-custom.conf ...
net.ipv4.ip_forward = 1
vm.swappiness = 10
```

boot-এর সময় `systemd-sysctl.service` এই ফাইলগুলো পড়ে। পড়ার নিয়ম:

```
/usr/lib/sysctl.d/*.conf    (প্যাকেজের ডিফল্ট)
/run/sysctl.d/*.conf        (runtime)
/etc/sysctl.d/*.conf        (administrator — তোমার)
       |
       v  সব ফাইল নাম অনুযায়ী (lexical order) সাজানো হয়।
          একই নামের ফাইল থাকলে /etc জেতে /run ও /usr/lib-এর ওপর।
          একই প্যারামিটার একাধিক ফাইলে থাকলে, পরে পড়া ফাইলের মান টেকে।
       v
/etc/sysctl.conf            (পুরোনো ফাইল, --system সবশেষে এটাও পড়ে, থাকলে)
```

তাই ফাইলের নাম `99-...conf` দেওয়া হয়, যাতে এটা সবার শেষে পড়া হয় আর এর মানই কার্যকর থাকে।

> 📝 **মনে রাখো:** কিছু প্যারামিটার (যেমন `net.bridge.*`, `net.netfilter.*`) তখনই থাকে যখন সংশ্লিষ্ট module লোড থাকে। module লোড হওয়ার আগে sysctl চললে ওই সেটিং প্রয়োগ হবে না। সমাধান: module-টা `/etc/modules-load.d`-এ রাখো।

গুরুত্বপূর্ণ sysctl প্যারামিটার

প্যারামিটার	সাধারণ ডিফল্ট	কী করে / কখন বদলাও
`net.ipv4.ip_forward`	0	1 করলে মেশিন রাউটারের মতো প্যাকেট ফরওয়ার্ড করে (NAT, VPN, container)
`net.ipv6.conf.all.forwarding`	0	IPv6-এর জন্য একই
`vm.swappiness`	60	কত আগ্রহ নিয়ে swap ব্যবহার করবে (0–200); ডেটাবেস সার্ভারে প্রায়ই 1–10
`vm.dirty_ratio` / `vm.dirty_background_ratio`	20 / 10	RAM-এর কত শতাংশ "লেখার অপেক্ষায়" ডেটা জমবে
`vm.overcommit_memory`	0	0 = heuristic, 1 = সবসময় হ্যাঁ, 2 = কড়া হিসাব
`vm.max_map_count`	distro অনুযায়ী	Elasticsearch/OpenSearch কমপক্ষে 262144 চায়
`fs.file-max`	অনেক বড় (RAM অনুযায়ী)	সারা সিস্টেমে সর্বোচ্চ খোলা ফাইল
`fs.inotify.max_user_watches`	distro অনুযায়ী	VS Code, webpack ইত্যাদি "too many watchers" এরর দিলে বাড়াও
`net.core.somaxconn`	4096	listen queue-এর সর্বোচ্চ দৈর্ঘ্য (ব্যস্ত ওয়েব সার্ভার)
`net.ipv4.ip_local_port_range`	`32768 60999`	বাইরে যাওয়া connection-এর জন্য port পরিসর
`net.ipv4.tcp_syncookies`	1	SYN flood আক্রমণ থেকে রক্ষা
`net.ipv4.conf.all.rp_filter`	distro অনুযায়ী (০/১/২)	ভুয়া source IP-এর প্যাকেট বাদ দেওয়া
`net.ipv4.icmp_echo_ignore_all`	0	1 করলে `ping`-এর উত্তর দেবে না
`kernel.hostname`		hostname (স্থায়ী করতে `hostnamectl` ব্যবহার করো)
`kernel.pid_max`	4194304 (64-bit-এ সাধারণত)	সর্বোচ্চ PID সংখ্যা
`kernel.panic`	0	panic হলে কত সেকেন্ড পর নিজে reboot (0 = আটকে থাকবে)
`kernel.sysrq`	distro অনুযায়ী (Ubuntu 176, Fedora 16)	Magic SysRq কোন কোন কাজে অনুমোদিত (bitmask)
`kernel.dmesg_restrict`	Ubuntu-তে 1	1 হলে সাধারণ user `dmesg` পড়তে পারবে না
`kernel.kptr_restrict`	1	kernel পয়েন্টার ঠিকানা লুকানো (নিরাপত্তা)

⚠️ **সতর্কতা:** ইন্টারনেট থেকে "Linux performance tuning sysctl" লিখে পাওয়া লম্বা তালিকা অন্ধভাবে কপি কোরো না। আধুনিক kernel-এর ডিফল্টগুলো বেশিরভাগ ক্ষেত্রেই যথেষ্ট ভালো। প্রতিটা পরিবর্তন একটা

> একটা করে মাপো (দেখো অধ্যায় ৪৬), আর কেন বদলালে সেটা ফাইলে মন্তব্য করে লিখে রাখো।

নিরাপত্তা-সম্পর্কিত sysctl (যেমন `kernel.randomize_va_space`, `net.ipv4.conf.all.accept_redirects`) নিয়ে বিস্তারিত আছে অধ্যায় ৪৫-এ।

৪৭.৭ udev: ডিভাইস ম্যানেজার ও ডিভাইস নিয়ম

সমস্যাটা কী

তুমি একটা pen drive লাগালে সেটা `/dev/sdb` হয়ে যায়। পরের বার আগে আরেকটা লাগানো থাকলে এটাই হয়ে যায় `/dev/sdc`। তাহলে script লিখবে কীভাবে? আর `/dev/sdb` ফাইলটা কে বানায়, তার owner আর পারমিশন কে ঠিক করে? এই কাজগুলো করে **udev**, যেটা আধুনিক সিস্টেমে `systemd-udevd` নামের একটা daemon (পটভূমিতে চলা সার্ভিস) হিসেবে চলে।

উপমা: অফিসের রিসেপশনিস্ট। নতুন কেউ এলে (ডিভাইস প্লাগ) নিরাপত্তারক্ষী (kernel) রিসেপশনে খবর পাঠায়। রিসেপশনিস্ট (udev) নিয়মের খাতা (rules) দেখে ঠিক করে: তাকে কোন রুমে বসাতে হবে (`/dev`-এ নাম), কে কে তার সাথে দেখা করতে পারবে (owner/group/mode), কাকে খবর দিতে হবে (`RUN`)।

```
ডিভাইস প্লাগ/আনপ্লাগ
        |
        v
KERNEL: driver ডিভাইস চিনল ---> /sys-এ নতুন এন্ট্রি
        |                        ---> devtmpfs-এ /dev/sdb তৈরি (মৌলিক)
        | uevent (netlink socket দিয়ে) — ACTION=add, DEVPATH=..., SUBSYSTEM=block
        v
systemd-udevd
   | 1. /sys থেকে attribute পড়ে (vendor, model, serial ...)
   | 2. rules ফাইলগুলো ক্রমানুসারে মেলায়
   | 3. ফল: owner/group/mode বসায়, symlink বানায়
   |       (/dev/disk/by-id/..., /dev/disk/by-uuid/...)
   |       network interface-এর নাম দেয় (enp3s0), দরকার হলে RUN চালায়
   v
অন্যরা খবর পায়: systemd (.device unit), udisks2 (ডেস্কটপে auto-mount) ...
```

এজন্যই `/dev/disk/by-uuid/` বা `/dev/disk/by-id/` দিয়ে ডিস্ক খুঁজলে নাম কখনো বদলায় না (অধ্যায় ৩২-এ fstab-এ UUID ব্যবহারের কারণ এটাই)। আর তোমার নেটওয়ার্ক কার্ডের নাম `eth0` না হয়ে `enp3s0` হয় udev-এর **predictable network interface names** নিয়মের কারণে: `en` = ethernet, `p3` = PCI bus 3, `s0` = slot 0। Wi-Fi হলে শুরু হয় `wl` দিয়ে, যেমন `wlp2s0`।

rules ফাইল কোথায় থাকে

ডিরেক্টরি	কার জন্য
`/usr/lib/udev/rules.d/`	প্যাকেজের নিয়ম (এগুলো এডিট কোরো না)

ডিরেক্টরি	কার জন্য
`/etc/udev/rules.d/`	তোমার নিজের নিয়ম; একই নামের ফাইল থাকলে এটাই জেতে
`/run/udev/rules.d/`	runtime, অস্থায়ী

সব ফাইল নাম অনুযায়ী সাজিয়ে পড়া হয়। ফাইলের নাম `NN-description.rules` আকারে হয়, যেমন `99-myusb.rules`।

`udevadm`: udev-এর সব কাজের টুল

কমান্ড	কাজ
`udevadm info -n /dev/sdb`	একটা ডিভাইসের udev তথ্য
`udevadm info -q property -n /dev/sdb`	শুধু property (`ENV`) গুলো
`udevadm info -a -n /dev/sdb`	ডিভাইস আর তার সব parent-এর attribute; rule লেখার জন্য সবচেয়ে কাজের
`udevadm monitor`	লাইভ event দেখো (প্লাগ/আনপ্লাগ করে দেখো)
`udevadm monitor --udev --property`	udev event, property সহ
`sudo udevadm control --reload`	rule ফাইল আবার পড়াও (পুরোনো নাম `--reload-rules`)
`sudo udevadm trigger`	বিদ্যমান ডিভাইসের জন্য আবার event পাঠাও, যাতে নতুন rule প্রয়োগ হয়
`udevadm test /sys/class/...`	rule আসলে কীভাবে মেলে, simulation করে দেখাও
`udevadm settle`	সব event প্রক্রিয়া হওয়া পর্যন্ত অপেক্ষা করো (script-এ)

```
$ udevadm info -q property -n /dev/sdb | grep -E
'ID_VENDOR=|ID_MODEL=|ID_SERIAL_SHORT|ID_BUS'
ID_BUS=usb
ID_MODEL=Cruzer_Blade
ID_SERIAL_SHORT=4C530001230808113264
ID_VENDOR=SanDisk

$ udevadm monitor --udev
monitor will print the received events for:
UDEV - the event which udev sends out after rule processing

UDEV  [12345.678901] add      /devices/pci0000:00/0000:00:14.0/usb1/1-2 (usb)
UDEV  [12345.701234] add      /devices/.../block/sdb (block)
UDEV  [12345.712345] add      /devices/.../block/sdb/sdb1 (block)
^C
```

rule-এর ভাষা

প্রতিটি rule **এক লাইনে**, কমা দিয়ে আলাদা করা `key operator value` জোড়া। কিছু key **মেলানোর** জন্য (match), কিছু **বসানোর** জন্য (assign)।

operator	ধরন	মানে
`==`	match	সমান হলে
`!=`	match	সমান না হলে
`=`	assign	মান বসাও (আগেরটা মুছে)
`+=`	assign	তালিকায় যোগ করো
`-=`	assign	তালিকা থেকে সরাও
`:=`	assign	চূড়ান্ত মান বসাও; পরের rule আর বদলাতে পারবে না

key	ধরন	মানে
`ACTION`	match	`add`, `remove`, `change`, `bind` ...
`KERNEL`	match	kernel-এর দেওয়া নাম (`sd*`, `ttyUSB*`)
`SUBSYSTEM`	match	`block`, `net`, `tty`, `usb` ...
`ATTR{file}`	match	এই ডিভাইসের sysfs attribute
`ATTRS{file}`	match	এই ডিভাইস **বা তার যেকোনো parent**-এর attribute
`ENV{KEY}`	match/assign	property
`SYMLINK`	assign	`/dev`-এ অতিরিক্ত symlink
`OWNER`, `GROUP`, `MODE`	assign	মালিক, গ্রুপ, পারমিশন
`NAME`	assign	শুধু network interface-এর নাম বদলানো যায়
`TAG+="uaccess"`	assign	ডেস্কটপে বসে থাকা (logged-in) user-কে অ্যাক্সেস দেওয়া
`RUN+=`	assign	event শেষে একটা প্রোগ্রাম চালাও (পুরো path)

`%k` (kernel নাম), `%n` (kernel নম্বর) এর মতো substitution-ও ব্যবহার করা যায়।

উদাহরণ ১: Arduino/USB-serial-কে স্থায়ী নাম দেওয়া

প্রথমে vendor ও product ID খোঁজো:

```
$ lsusb | grep -i ftdi
Bus 001 Device 007: ID 0403:6001 Future Technology Devices International, Ltd FT232
Serial (UART) IC
```

তারপর rule লেখো:

```
# /etc/udev/rules.d/99-arduino.rules
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001",
SYMLINK+="arduino", MODE="0660", GROUP="dialout"
```

```
$ sudo udevadm control --reload
$ sudo udevadm trigger --subsystem-match=tty
```

```
$ ls -l /dev/arduino
lrwxrwxrwx 1 root root 7 Nov 12 11:00 /dev/arduino -> ttyUSB0
```

এখন ডিভাইস `ttyUSB0` হোক বা `ttyUSB3`, তোমার প্রোগ্রাম সবসময় `/dev/arduino` ব্যবহার করতে পারবে। আর তুমি `dialout` গ্রুপে থাকলে `sudo` ছাড়াই অ্যাক্সেস পাবে।

উদাহরণ ২: নির্দিষ্ট pen drive লাগালে log লেখা

```
# /etc/udev/rules.d/99-backup-disk.rules
ACTION=="add", SUBSYSTEM=="block", ENV{ID_SERIAL_SHORT}=="4C530001230808113264",
ENV{DEVTYPE}=="partition", RUN+="/usr/bin/logger 'backup disk %k plugged'"
```

> ⚠️ **সতর্কতা:** `RUN` দিয়ে চালানো প্রোগ্রাম খুব ছোট হতে হবে। udev অল্প সময় পর সেটাকে মেরে ফেলে, আর ভেতরে নেটওয়ার্ক বা mount ঠিকমতো কাজ নাও করতে পারে। লম্বা কাজের জন্য (যেমন ব্যাকআপ চালানো) `RUN` থেকে সরাসরি না চালিয়ে systemd service চালু করো: `ENV{SYSTEMD_WANTS}+="mybackup.service"` আর সাথে `TAG+="systemd"`।

> 💡 **টিপস:** rule কাজ না করলে, প্রথমে `udevadm info -a`-তে দেখো তুমি যে attribute মেলাচ্ছ সেটা আসলে আছে কিনা। মনে রাখবে, একটা rule-এ `ATTRS{}` শুধু **একটাই** parent ডিভাইস থেকে আসতে পারে, অনেকগুলো parent মিশিয়ে মেলানো যায় না। তারপর `udevadm test $(udevadm info -q path -n /dev/sdb)` চালাও।

৪৭.৮ হার্ডওয়্যারের তথ্য বের করা

নতুন সার্ভার হাতে পেলে, বা কোনো driver কাজ না করলে, সবার আগে জানতে হয় ভেতরে কী আছে। নিচের টুলগুলো এই তথ্য `/proc`, `/sys` ও firmware টেবিল থেকে পড়ে সাজিয়ে দেখায়।

কমান্ড	কী দেখায়	প্যাকেজ	root লাগে?
`lscpu`	CPU-র গঠন	util-linux (সব জায়গায় থাকে)	না
`lspci`	PCI/PCIe ডিভাইস (GPU, NIC, NVMe)	pciutils	বেশিরভাগ না; বিস্তারিত (`-vv`) দেখতে হ্যাঁ
`lsusb`	USB ডিভাইস	usbutils	না
`lsblk`	ডিস্ক ও পার্টিশন	util-linux	না
`dmidecode`	BIOS/মাদারবোর্ড/RAM স্লট/সিরিয়াল নম্বর	dmidecode	**হ্যাঁ**
`lshw`	সব হার্ডওয়্যারের পূর্ণ তালিকা	lshw	পূর্ণ তথ্যের জন্য হ্যাঁ

কমান্ড	কী দেখায়	প্যাকেজ	root লাগে?
`hwinfo`	বিস্তারিত (openSUSE-র টুল)	hwinfo	হ্যাঁ
`inxi`	সুন্দর সারাংশ, ফোরামে শেয়ার করার জন্য	inxi	না
`sensors`	তাপমাত্রা, ফ্যানের গতি	lm-sensors	না
`free`, `nproc`	RAM, CPU সংখ্যা	procps / coreutils	না

`lscpu`

```
$ lscpu
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Byte Order:             Little Endian
CPU(s):                   8
  On-line CPU(s) list:    0-7
Vendor ID:                GenuineIntel
  Model name:             Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz
    Thread(s) per core:   2
    Core(s) per socket:   4
    Socket(s):            1
Virtualization features:
  Virtualization:         VT-x
Caches (sum of all):
  L1d:                    128 KiB (4 instances)
  L2:                     1 MiB (4 instances)
  L3:                     8 MiB (1 instance)
NUMA:
  NUMA node(s):           1
...
```

হিসাবটা বুঝে নাও: মোট logical CPU = Socket × Core per socket × Thread per core = 1 × 4 × 2 = **8**।

অপশন	কাজ
`-e`	প্রতিটি CPU এক লাইনে (টেবিল আকারে)
`-J`	JSON আউটপুট
`--caches`	cache-এর বিস্তারিত

`lspci`

```
$ lspci
00:02.0 VGA compatible controller: Intel Corporation CometLake-U GT2 [UHD Graphics]
(rev 02)
00:14.3 Network controller: Intel Corporation Comet Lake PCH-LP CNVi WiFi
00:1f.6 Ethernet controller: Intel Corporation Ethernet Connection (10) I219-V
02:00.0 Non-Volatile memory controller: Samsung Electronics Co Ltd NVMe SSD Controller
SM981/PM981/PM983
```

```
$ lspci -k -s 00:1f.6
00:1f.6 Ethernet controller: Intel Corporation Ethernet Connection (10) I219-V
    Subsystem: Lenovo Device 2292
    Kernel driver in use: e1000e
    Kernel modules: e1000e
```

অপশন	কাজ
`-k`	কোন kernel driver ব্যবহার হচ্ছে, কোন module সম্ভব; **troubleshooting-এর সেরা অপশন**
`-v`, `-vv`	বিস্তারিত
`-nn`	নামের সাথে সংখ্যাসূচক vendor:device ID (`[8086:15bc]`)
`-s <slot>`	নির্দিষ্ট একটা ডিভাইস
`-t`	গাছের আকারে
`-d <vendor>:<device>`	ID দিয়ে খোঁজো

> 💡 **টিপস:** কোনো ডিভাইসের পাশে `Kernel driver in use` লাইনটা না থাকলে বুঝবে driver লোড হয়নি। তখন `lspci -nn` থেকে ID নিয়ে ইন্টারনেটে খোঁজো কোন module লাগবে। নামের ডেটাবেস পুরোনো হলে আপডেট করো: `sudo update-pciids`।

`lsusb`

```
$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 8087:0026 Intel Corp. AX201 Bluetooth
Bus 001 Device 004: ID 046d:c52b Logitech, Inc. Unifying Receiver
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub

$ lsusb -t
/:  Bus 001.Port 001: Dev 001, Class=root_hub, Driver=xhci_hcd/12p, 480M
    |__ Port 001: Dev 004, If 0, Class=Human Interface Device, Driver=usbhid, 12M
    ...
```

অপশন	কাজ
`-t`	গাছ আকারে, driver ও গতি সহ
`-v`	বিস্তারিত descriptor
`-d vendor:product`	নির্দিষ্ট ডিভাইস
`-s bus:dev`	bus/device নম্বর দিয়ে

`dmidecode`

এটা BIOS/UEFI firmware-এর **SMBIOS/DMI** টেবিল পড়ে। এখানে এমন সব তথ্য থাকে যা kernel নিজে জানে না: মেশিনের সিরিয়াল নম্বর, RAM কোন স্লটে কত GB, সর্বোচ্চ কত RAM লাগানো যাবে।

```
$ sudo dmidecode -s system-manufacturer
LENOVO
$ sudo dmidecode -s system-product-name
20U9CTO1WW
$ sudo dmidecode -s bios-version
N2WET36W (1.26 )

$ sudo dmidecode -t memory | grep -E 'Size|Locator|Speed|Type:' | head
    Size: 8 GB
    Locator: ChannelA-DIMM0
    Type: DDR4
    Speed: 2667 MT/s
    Size: 8 GB
    Locator: ChannelB-DIMM0
    ...
```

অপশন	কাজ
`-t <type>`	নির্দিষ্ট ধরন: `bios`, `system`, `baseboard`, `chassis`, `processor`, `memory`, `cache`, `slot`
`-s <keyword>`	একটা মান: `system-serial-number`, `system-uuid`, `bios-version`, `baseboard-product-name` …
`-q`	কম কথা (quiet)

> 📝 **মনে রাখো:** virtual machine-এ `dmidecode` hypervisor-এর বানানো তথ্য দেখায় (যেমন `QEMU`, `VMware`)। কোন hypervisor-এ চলছ সেটা জানার আরেকটা সহজ উপায় `systemd-detect-virt`।

`lshw`

```
$ sudo lshw -short
H/W path          Device      Class          Description
============================================================
                              system         20U9CTO1WW (LENOVO_MT_20U9)
/0                            bus            20U9CTO1WW
/0/0                          memory         128KiB BIOS
/0/4                          processor      Intel(R) Core(TM) i7-10510U
/0/2c                         memory         16GiB System Memory
/0/100/1f.6       enp0s31f6   network        Ethernet Connection (10) I219-V
/0/100/1d/0       /dev/nvme0  storage        Samsung SSD 970 EVO Plus 500GB
...
```

অপশন	কাজ
`-short`	সংক্ষিপ্ত টেবিল

অপশন	কাজ
`-class <cls>` (`-c`)	একটা শ্রেণি: `network`, `disk`, `memory`, `display`, `processor`
`-businfo`	bus ঠিকানা সহ
`-html`, `-json`, `-xml`	রিপোর্ট ফাইল বানানোর জন্য
`-sanitize`	সিরিয়াল নম্বর, IP ইত্যাদি লুকাও (শেয়ার করার আগে)

```
$ sudo lshw -class network -short
H/W path         Device      Class      Description
=====================================================
/0/100/14.3      wlp0s20f3   network    Comet Lake PCH-LP CNVi WiFi
/0/100/1f.6      enp0s31f6   network    Ethernet Connection (10) I219-V
```

অন্যান্য

```
$ inxi -Fxz          # সব কিছুর সারাংশ; -z ব্যক্তিগত তথ্য (MAC, IP) লুকায়
$ sensors            # প্রথমবার: sudo sensors-detect
$ lsblk -o NAME,SIZE,TYPE,MODEL,ROTA
$ cat /sys/class/dmi/id/product_name     # dmidecode-এর কিছু তথ্য root ছাড়াই
```

৪৭.৯ kernel আপডেট ও একাধিক kernel

আপডেটের সময় কী হয়

বেশিরভাগ প্যাকেজ আপডেট হলে পুরোনো version মুছে নতুনটা বসে। কিন্তু kernel আলাদা: **নতুন kernel পুরোনোটার পাশে আলাদাভাবে ইনস্টল হয়।** কারণ নতুন kernel যদি তোমার মেশিনে boot না করে, তুমি যেন পুরোনোটায় ফিরে যেতে পারো। এটা একটা নিরাপত্তা-জাল।

```
apt/dnf upgrade
    |
    +--> /boot/vmlinuz-<নতুন>              (নতুন kernel image)
    +--> /lib/modules/<নতুন>/              (নতুন module সেট)
    +--> initramfs তৈরি (update-initramfs / dracut)
    +--> DKMS module গুলো নতুন kernel-এর জন্য build (৪৭.১০)
    +--> bootloader এন্ট্রি আপডেট (update-grub / BLS entry)
    |
    v
পুরোনো kernel এখনও চলছে!  -->  reboot  -->  নতুন kernel চালু
```

> 📝 **মনে রাখো:** kernel আপডেটের পর **reboot না করা পর্যন্ত নতুন kernel চলে না।** `uname -r` দেখলে পুরোনো version-ই দেখাবে। Ubuntu-তে reboot দরকার হলে `/var/run/reboot-required` ফাইলটা তৈরি হয়, আর `needrestart` টুলও সতর্ক করে।

Debian / Ubuntu

```
# ইনস্টল করা kernel-এর তালিকা
$ dpkg --list 'linux-image*' | grep ^ii
ii  linux-image-6.8.0-41-generic  6.8.0-41.41  amd64  Signed kernel image generic
ii  linux-image-6.8.0-45-generic  6.8.0-45.45  amd64  Signed kernel image generic
ii  linux-image-generic           6.8.0-45.45  amd64  Generic Linux kernel image

# reboot দরকার কিনা
$ cat /var/run/reboot-required 2>/dev/null
*** System restart required ***

# পুরোনো অপ্রয়োজনীয় kernel সরানো (চলমান ও তার আগেরটা সাধারণত রেখে দেয়)
$ sudo apt autoremove --purge
```

- `linux-image-generic` (Ubuntu) বা `linux-image-amd64` (Debian) হলো **metapackage**। এটা নিজে কিছু না, শুধু "সর্বশেষ kernel" প্যাকেজের ওপর নির্ভর করে, তাই আপডেটে নতুন kernel টেনে আনে।
- Ubuntu LTS-এ নতুন হার্ডওয়্যারের জন্য **HWE (Hardware Enablement)** kernel পাওয়া যায়: `linux-generic-hwe-24.04`। Debian-এ নতুন kernel আসে `backports` repository থেকে।
- kernel আপডেট আপাতত আটকাতে: `sudo apt-mark hold linux-image-generic linux-headers-generic`।

Fedora / RHEL

```
$ rpm -q kernel
kernel-5.14.0-427.13.1.el9_4.x86_64
kernel-5.14.0-427.18.1.el9_4.x86_64

$ sudo grubby --default-kernel
/boot/vmlinuz-5.14.0-427.18.1.el9_4.x86_64

$ sudo grubby --info=ALL | grep -E '^(index|kernel|title)'

# নির্দিষ্ট kernel-কে ডিফল্ট করো
$ sudo grubby --set-default /boot/vmlinuz-5.14.0-427.13.1.el9_4.x86_64
```

- কতগুলো kernel রাখা হবে সেটা ঠিক করে `/etc/dnf/dnf.conf`-এর `installonly_limit` (ডিফল্ট `3`)। সংখ্যা পার হলে সবচেয়ে পুরোনোটা নিজে থেকেই মুছে যায়।
- Fedora/RHEL-এ bootloader এন্ট্রিগুলো `/boot/loader/entries/*.conf`-এ আলাদা ফাইল হিসেবে থাকে (BootLoaderSpec বা BLS), আর `grubby` দিয়ে সেগুলো সামলানো হয়।
- সব kernel-এ একটা boot প্যারামিটার যোগ করতে: `sudo grubby --update-kernel=ALL --args="mitigations=auto"`। Debian/Ubuntu-তে একই কাজ হয় `/etc/default/grub`-এর `GRUB_CMDLINE_LINUX_DEFAULT` এডিট করে, তারপর `sudo update-grub` চালিয়ে।

পুরোনো kernel দিয়ে boot করা

নতুন kernel-এ সমস্যা হলে (Wi-Fi কাজ করছে না, কালো স্ক্রিন):

1. reboot করো।
2. GRUB মেনু দেখাও। Ubuntu সাধারণত মেনু লুকিয়ে রাখে। UEFI মেশিনে boot শুরু হতেই **Esc** বারবার চাপো; পুরোনো BIOS মেশিনে **Shift** চেপে ধরে রাখো।
3. **Advanced options for Ubuntu** বেছে নাও, তারপর পুরোনো version-টা।
4. পুরোনোটায় সব ঠিক থাকলে সমস্যার রিপোর্ট করো, নতুন আপডেটের জন্য অপেক্ষা করো, অথবা পুরোনোটাকে ডিফল্ট বানাও।

reboot ছাড়া patch: livepatch

সার্ভার reboot করা সবসময় সম্ভব হয় না। তাই গুরুতর security fix চলন্ত kernel-এ সরাসরি বসানোর ব্যবস্থা আছে: Ubuntu-তে **Canonical Livepatch** (Ubuntu Pro দিয়ে, `sudo pro enable livepatch`), RHEL-এ **kpatch**। তবে এগুলো সাময়িক সমাধান; শেষ পর্যন্ত reboot করে নতুন kernel-এ যেতেই হবে।

> ⚠️ **সতর্কতা:** যে kernel-এ এখন চলছ (`uname -r`), সেটা কখনো মুছবে না। আর `/boot` পার্টিশন ছোট হলে অনেকগুলো kernel জমে সেটা ভরে যায়, তখন আপডেট মাঝপথে ব্যর্থ হয়। নিয়মিত `df -h /boot` দেখো।

৪৭.১০ DKMS: kernel বদলালেও driver টিকে থাকে

সমস্যা

kernel module একটা নির্দিষ্ট kernel version-এর জন্য compile করা হয় (মনে আছে `modinfo`-র `vermagic`?)। তুমি NVIDIA, VirtualBox বা কোনো Wi-Fi কার্ডের **out-of-tree** module (যেটা অফিশিয়াল kernel source-এর অংশ নয়) হাতে compile করে ইনস্টল করলে, পরের kernel আপডেটের পর সেটা আর লোড হবে না। নতুন kernel-এর `/lib/modules/<নতুন>/`-এ সেটা নেই-ই।

সমাধান: DKMS

DKMS (Dynamic Kernel Module Support) module-এর **source code** নিজের কাছে রেখে দেয়। যখনই নতুন kernel ইনস্টল হয়, প্যাকেজ ম্যানেজারের একটা hook DKMS-কে ডাকে, আর DKMS নতুন kernel-এর জন্য module-টা আবার compile করে ইনস্টল করে দেয়।

উপমা: দর্জির কাছে তোমার শার্টের মাপ আর কাপড়ের নকশা রাখা আছে। তুমি একটু মোটা হলে (kernel আপডেট) দর্জি নিজেই নতুন মাপে শার্ট সেলাই করে দেয়; তোমাকে আবার নকশা নিয়ে যেতে হয় না।

```
/usr/src/v4l2loopback-0.12.7/      <- source + dkms.conf
           |
           |  dkms add      (DKMS-এর খাতায় নাম তোলা)
           v
/var/lib/dkms/v4l2loopback/0.12.7/
           |
           |  dkms build -k <kernel>   (kernel headers লাগে)
           v
  .ko তৈরি
           |
           |  dkms install -k <kernel>
           v
/lib/modules/<kernel>/updates/dkms/v4l2loopback.ko   + depmod
           ^
           |  নতুন kernel ইনস্টল হলে: dkms autoinstall (স্বয়ংক্রিয়)
```

পূর্বশর্ত: kernel-এর **headers** ইনস্টল থাকতে হবে:

```
# Debian / Ubuntu
$ sudo apt install dkms linux-headers-$(uname -r)
# Fedora / RHEL (RHEL-এ dkms আসে EPEL থেকে)
$ sudo dnf install dkms kernel-devel kernel-headers
```

DKMS কমান্ড

কমান্ড	কাজ
`dkms status`	কোন module কোন kernel-এর জন্য কী অবস্থায় আছে
`sudo dkms add -m <name> -v <ver>`	`/usr/src/<name>-<ver>` থেকে নিবন্ধন
`sudo dkms build -m <name> -v <ver> [-k <kernel>]`	compile
`sudo dkms install -m <name> -v <ver> [-k <kernel>]`	ইনস্টল (দরকার হলে build-ও করে)
`sudo dkms remove -m <name> -v <ver> --all`	সব kernel থেকে সরাও
`sudo dkms autoinstall [-k <kernel>]`	সব নিবন্ধিত module একসাথে build ও install

বাস্তবে বেশিরভাগ সময় তুমি নিজে এগুলো চালাবে না; `*-dkms` নামের প্যাকেজ ইনস্টল করলেই সব হয়ে যায়:

```
$ sudo apt install v4l2loopback-dkms
...
Building for 6.8.0-45-generic
Building initial module for 6.8.0-45-generic
Done.

$ dkms status
v4l2loopback/0.12.7, 6.8.0-45-generic, x86_64: installed
```

একটা সাধারণ `dkms.conf` দেখতে এমন:

```
PACKAGE_NAME="hello"
PACKAGE_VERSION="1.0"
BUILT_MODULE_NAME[0]="hello"
DEST_MODULE_LOCATION[0]="/updates/dkms"
AUTOINSTALL="yes"
```

Secure Boot ও module signing

UEFI **Secure Boot** চালু থাকলে kernel শুধু বিশ্বস্ত key দিয়ে **signed** module লোড করে। না হলে এরর আসে:

```
$ sudo modprobe hello
modprobe: ERROR: could not insert 'hello': Key was rejected by service
```

```
$ mokutil --sb-state
SecureBoot enabled
```

Ubuntu-তে DKMS নিজের একটা key (MOK, Machine Owner Key) দিয়ে module sign করে। প্রথমবার সেই key-টা firmware-এ নিবন্ধন করতে হয়: `sudo mokutil --import /var/lib/shim-signed/mok/MOK.der` চালিয়ে একটা পাসওয়ার্ড দাও, তারপর reboot করলে নীল রঙের **MOK Manager** স্ক্রিনে "Enroll MOK" বেছে সেই পাসওয়ার্ড দাও। Fedora/RHEL-এ key-এর অবস্থান আলাদা (DKMS-এর কনফিগে উল্লেখ থাকে, সাধারণত `/var/lib/dkms/mok.pub`)।

> 💡 **টিপস:** kernel আপডেটের পর NVIDIA বা VirtualBox কাজ না করলে আগে `dkms status` দেখো। নতুন kernel-এর পাশে `installed` না থাকলে headers আছে কিনা দেখে নাও, তারপর `sudo dkms autoinstall`। build ব্যর্থ হলে log থাকে `/var/lib/dkms/<name>/<ver>/build/make.log`-এ।

সোর্স থেকে সফটওয়্যার বানানোর সাধারণ নিয়ম আছে অধ্যায় ৩১-এ।

৪৭.১১ kernel panic, oops ও জরুরি অবস্থা

oops বনাম panic

	Oops	Panic
কী	kernel একটা ভুল ধরেছে (যেমন NULL pointer), কিন্তু চালিয়ে যেতে পারছে	kernel এমন অবস্থায় যে আর নিরাপদে চালানো সম্ভব নয়
ফল	যে process ভুলটা ঘটিয়েছে সেটা মারা যায়; সিস্টেম অস্থির থাকতে পারে	সব থেমে যায়। স্ক্রিনে বার্তা, কীবোর্ডের LED জ্বলা-নেভা
কোথায় দেখবে	`dmesg`, `journalctl -k`	স্ক্রিনে (বা kdump/serial console থাকলে সেখানে)

উপমা: oops হলো রান্নাঘরে একটা হাঁড়ি পড়ে যাওয়া: পরিষ্কার করে কাজ চালানো যায়। panic হলো রান্নাঘরে আগুন: সবকিছু বন্ধ করে বেরিয়ে যেতে হয়, কারণ চালাতে থাকলে আরও ক্ষতি হবে (যেমন ডিস্কের ডেটা নষ্ট হয়ে যেতে পারে)।

সাধারণ panic বার্তা ও কারণ

```
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
```

এর মানে kernel root ফাইলসিস্টেম খুঁজে পায়নি। কারণ সাধারণত initramfs নেই বা নষ্ট, অথবা `root=` প্যারামিটার ভুল। সমাধান: GRUB থেকে পুরোনো kernel-এ boot করে initramfs আবার বানাও (`sudo update-initramfs -u -k <ver>` বা `sudo dracut -f --kver <ver>`)।

```
Kernel panic - not syncing: Attempted to kill init! exitcode=0x00000100
```

এর মানে PID 1 (systemd) মারা গেছে। কারণ হতে পারে নষ্ট root ফাইলসিস্টেম, ভাঙা glibc, বা ভুল `init=` প্যারামিটার।

অন্যান্য কারণের মধ্যে আছে নষ্ট RAM (`memtest86+` দিয়ে পরীক্ষা করো), অতিরিক্ত গরম হওয়া, buggy driver/module (বিশেষ করে proprietary ও out-of-tree), আর `vm.panic_on_oom=1` থাকা অবস্থায় মেমরি শেষ হয়ে যাওয়া।

tainted kernel

proprietary বা unsigned module লোড হলে kernel নিজেকে "tainted" (দাগী) চিহ্নিত করে। এতে চলায় কোনো সমস্যা হয় না; শুধু bug রিপোর্টে kernel ডেভেলপাররা বুঝতে পারেন যে অজানা কোড যুক্ত ছিল।

```
$ cat /proc/sys/kernel/tainted
0              <- 0 মানে পরিষ্কার

$ cat /proc/sys/kernel/tainted
12289          <- bitmask: 1 (P, proprietary) + 4096 (O, out-of-tree) + 8192 (E,
unsigned)
```

panic-এর পর তথ্য সংগ্রহ

panic-এর মুহূর্তে ডিস্কে লেখা নিরাপদ নয়, তাই বার্তা প্রায়ই log-এ পৌঁছায় না। উপায়গুলো:

উপায়	কীভাবে
আগের boot-এর kernel log	`journalctl -k -b -1` (persistent journal চালু থাকলে, দেখো অধ্যায় ২৮)
pstore	কিছু firmware panic-এর বার্তা সংরক্ষণ করে রাখে, পরের boot-এ `/sys/fs/pstore/` -এ দেখা যায়

উপায়	কীভাবে
kdump	panic হলেই আগে থেকে লোড করা একটা দ্বিতীয় "crash kernel" চালু হয়, সে পুরো মেমরি ডাম্প করে ডিস্কে লিখে রাখে। Ubuntu: `sudo apt install kdump-tools` (বা `linux-crashdump`); RHEL: `kexec-tools`, `kdumpctl status`। ডাম্প জমা হয় `/var/crash/`-এ, বিশ্লেষণ করা হয় `crash` টুল দিয়ে
serial/net console	সার্ভারে kernel বার্তা অন্য মেশিনে পাঠানো
স্ক্রিনের ছবি	বার্তার প্রথম কয়েক লাইন সবচেয়ে গুরুত্বপূর্ণ, ফোনে একটা ছবি তুলে রাখো

সার্ভারে panic হলে মেশিন আটকে না থেকে যেন নিজেই reboot করে:

```
$ echo 'kernel.panic = 10' | sudo tee /etc/sysctl.d/90-panic.conf
$ sudo sysctl --system
```

`dmesg`: kernel-এর রিং বাফার

kernel-এর নিজের বার্তা (driver লোড, ডিভাইস প্লাগ, এরর, oops) একটা **ring buffer**-এ জমা হয়: মেমরিতে একটা নির্দিষ্ট মাপের চক্রাকার জায়গা, ভরে গেলে সবচেয়ে পুরোনো বার্তা মুছে যায়।

কমান্ড	কাজ
`sudo dmesg`	সব বার্তা (Ubuntu-তে `dmesg_restrict=1`, তাই `sudo` লাগে)
`sudo dmesg -T`	মানুষের পড়ার মতো সময়
`sudo dmesg -H`	রঙিন, pager সহ
`sudo dmesg -w`	লাইভ দেখো (`tail -f`-এর মতো)
`sudo dmesg -l err,warn`	শুধু এরর ও warning
`journalctl -k`	journal থেকে kernel বার্তা (`-b -1` = আগের boot)

```
$ sudo dmesg -T | tail -3
[Tue Nov 12 11:02:15 2026] usb 1-2: new high-speed USB device number 8 using xhci_hcd
[Tue Nov 12 11:02:15 2026] usb-storage 1-2:1.0: USB Mass Storage device detected
[Tue Nov 12 11:02:16 2026] sd 2:0:0:0: [sdb] 30031872 512-byte logical blocks: (15.4
GB/14.3 GiB)
```

Magic SysRq: সিস্টেম জমে গেলে শেষ ভরসা

সিস্টেম পুরোপুরি জমে গেলে (মাউস নড়ে না, টার্মিনাল সাড়া দেয় না) সরাসরি পাওয়ার বাটন না চেপে, আগে kernel-কে সরাসরি কীবোর্ড থেকে নির্দেশ দেওয়া যায়। কম্বিনেশন হলো **Alt + SysRq (PrintScreen) + একটা অক্ষর**। কিছু ল্যাপটপে **Alt + Fn + PrtSc + অক্ষর** চাপতে হয়।

ক্লাসিক নিরাপদ reboot ক্রম হলো **R-E-I-S-U-B** (মনে রাখার বাক্য: "**R**eboot **E**ven **I**f **S**ystem **U**tterly **B**roken")। প্রতিটি অক্ষরের মাঝে কয়েক সেকেন্ড অপেক্ষা করো।

শর্টকাট	অক্ষর	কাজ
Alt+SysRq+**R**	R	কীবোর্ড X/Wayland থেকে kernel-এর নিয়ন্ত্রণে ফেরানো (raw mode)
Alt+SysRq+**E**	E	init ছাড়া সব process-কে SIGTERM পাঠানো
Alt+SysRq+**I**	I	init ছাড়া সব process-কে SIGKILL পাঠানো
Alt+SysRq+**S**	S	সব ফাইলসিস্টেম sync (ডেটা ডিস্কে লেখা)
Alt+SysRq+**U**	U	সব ফাইলসিস্টেম read-only হিসেবে আবার মাউন্ট
Alt+SysRq+**B**	B	সাথে সাথে reboot
Alt+SysRq+**O**	O	পাওয়ার বন্ধ
Alt+SysRq+**F**	F	OOM killer ডেকে সবচেয়ে বেশি মেমরি খাওয়া process মারা
Alt+SysRq+**H**	H	সাহায্য (কনসোলে/dmesg-এ তালিকা)

⚠️ **সতর্কতা:** সব অক্ষর ডিফল্টভাবে চালু থাকে না। কোনগুলো চালু সেটা ঠিক করে `kernel.sysrq` bitmask। Ubuntu-তে ডিফল্ট `176` (= 16 sync + 32 remount + 128 reboot/poweroff), তাই S, U, B, O কাজ করে, কিন্তু E, I, F করে না। সব চালু করতে: `sudo sysctl -w kernel.sysrq=1` (সাময়িক)। নিরাপত্তার কারণে পাবলিক বা শেয়ার্ড মেশিনে এটা সীমিত রাখাই ভালো।

একই কাজ টার্মিনাল থেকেও করা যায় (root হিসেবে):

```
$ echo s | sudo tee /proc/sysrq-trigger      # sync
$ echo c | sudo tee /proc/sysrq-trigger      # ইচ্ছে করে crash/panic! শুধু kdump পরীক্ষার জন্য, VM-এ
```

⚠️ **সতর্কতা:** `echo c > /proc/sysrq-trigger` সঙ্গে সঙ্গে kernel ক্র্যাশ করায়, আর সেভ না করা সব কাজ হারিয়ে যায়। এটা শুধু টেস্ট VM-এ kdump যাচাই করার জন্য।

GRUB-এর কীবোর্ড শর্টকাট (kernel সমস্যায় দরকার হয়)

কী	কাজ
Esc (UEFI) / **Shift** চেপে ধরা (BIOS)	লুকানো GRUB মেনু দেখানো (Ubuntu)
↑ / ↓	এন্ট্রি বাছাই
Enter	বাছাই করা এন্ট্রি boot
e	এন্ট্রি সাময়িকভাবে এডিট (যেমন `linux` লাইনে `modprobe.blacklist=nouveau` বা `systemd.unit=rescue.target` যোগ করা)
Ctrl+X বা **F10**	এডিট করা এন্ট্রি boot

কী	কাজ
c	GRUB কমান্ড লাইন
Esc (এডিট অবস্থায়)	পরিবর্তন বাতিল করে মেনুতে ফেরা

হাতে-কলমে অনুশীলন

> 💡 সব ল্যাব একটা virtual machine-এ করা সবচেয়ে নিরাপদ (দেখো অধ্যায় ৩ ও অধ্যায় ৪৮)।

ল্যাব ১: তোমার kernel-কে চেনো

```
$ uname -r
$ uname -m
$ cat /proc/cmdline
$ ls /boot
$ grep -E 'CONFIG_EXT4_FS=|CONFIG_XFS_FS=|CONFIG_BTRFS_FS=' /boot/config-$(uname -r)
```

প্রশ্ন: তোমার kernel-এ ext4 built-in নাকি module? `/lib/modules/$(uname -r)/modules.builtin`-এ `ext4` আছে কিনা মিলিয়ে দেখো।

ল্যাব ২: module লোড-আনলোড

```
# 1. dry run: কী কী লোড হবে?
$ sudo modprobe -n -v vfat

# 2. লোড করো এবং যাচাই করো
$ sudo modprobe -v vfat
$ lsmod | grep -E '^(vfat|fat) '

# 3. তথ্য দেখো
$ modinfo -F description vfat
$ modinfo -F depends vfat

# 4. আনলোড করো
$ sudo modprobe -r -v vfat
$ lsmod | grep fat || echo "fat গেল"
```

(যদি `vfat` ইতিমধ্যে ব্যবহারে থাকে, যেমন `/boot/efi` মাউন্ট করা থাকলে, তাহলে আনলোড হবে না। সেক্ষেত্রে `dummy` module দিয়ে চেষ্টা করো: `sudo modprobe dummy`, তারপর `ip link` দেখো, `dummy0` নামে নতুন ইন্টারফেস এসেছে কিনা।)

ল্যাব ৩: boot-এ module লোড + blacklist

```
$ echo dummy | sudo tee /etc/modules-load.d/lab.conf
$ sudo systemctl restart systemd-modules-load.service
$ lsmod | grep dummy

# এবার blacklist পরীক্ষা
$ sudo modprobe -r dummy
$ echo 'install dummy /bin/false' | sudo tee /etc/modprobe.d/lab-block.conf
$ sudo modprobe dummy
modprobe: ERROR: ../libkmod/libkmod-module.c:... could not insert 'dummy': Operation
not permitted
          <- বার্তার শব্দ kmod version অনুযায়ী একটু আলাদা হতে পারে

# পরিষ্কার করো
$ sudo rm /etc/modules-load.d/lab.conf /etc/modprobe.d/lab-block.conf
```

ল্যাব ৪: `/proc` দিয়ে process তদন্ত

```
$ sleep 600 &
$ PID=$!
$ cat /proc/$PID/cmdline | tr '\0' ' '; echo
$ grep -E 'Name|State|PPid|VmRSS' /proc/$PID/status
$ ls -l /proc/$PID/exe /proc/$PID/cwd
$ ls -l /proc/$PID/fd
$ cat /proc/$PID/limits | head -5
$ kill $PID
$ ls /proc/$PID 2>&1 | head -1       # এখন আর নেই
```

ল্যাব ৫: `/sys` থেকে হার্ডওয়্যার পড়া

```
$ ls /sys/class/net
$ for n in /sys/class/net/*; do echo "$(basename $n): $(cat $n/address) $(cat
$n/operstate)"; done
$ for d in /sys/block/*; do echo "$(basename $d) rotational=$(cat
$d/queue/rotational)"; done
$ cat /sys/class/power_supply/BAT*/capacity 2>/dev/null || echo "ব্যাটারি নেই (ডেস্কটপ/VM)"
```

ল্যাব ৬: sysctl সাময়িক ও স্থায়ী

```
$ sysctl net.ipv4.ip_forward
$ sudo sysctl -w net.ipv4.ip_forward=1
$ cat /proc/sys/net/ipv4/ip_forward

# স্থায়ী করো
$ echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-lab.conf
$ sudo sysctl --system | grep ip_forward

# reboot-এর পর আবার দেখো, তারপর পরিষ্কার করো
```

```
$ sudo rm /etc/sysctl.d/99-lab.conf
$ sudo sysctl -w net.ipv4.ip_forward=0
```

ল্যাব ৭: udev দেখা ও rule লেখা

```
# টার্মিনাল ১
$ udevadm monitor --udev
# এখন একটা pen drive লাগাও/খোলো, event দেখো; শেষে Ctrl+C

# তথ্য বের করো (তোমার ডিভাইসের নাম বসাও)
$ udevadm info -q property -n /dev/sdb | grep -E
'ID_VENDOR=|ID_MODEL=|ID_SERIAL_SHORT='

# rule: এই pen drive-এর জন্য /dev/mypendrive symlink
$ echo 'SUBSYSTEM=="block", ENV{ID_SERIAL_SHORT}=="<তোমার-সিরিয়াল>",
ENV{DEVTYPE}=="disk", SYMLINK+="mypendrive"' | sudo tee /etc/udev/rules.d/99-lab.rules
$ sudo udevadm control --reload
# pen drive খুলে আবার লাগাও
$ ls -l /dev/mypendrive
```

ল্যাব ৮: হার্ডওয়্যার রিপোর্ট

```
$ lscpu | grep -E 'Model name|Socket|Core|Thread|^CPU\(s\)'
$ lspci -k | grep -A3 -iE 'vga|network|ethernet'
$ lsusb -t
$ sudo dmidecode -s system-product-name
$ sudo dmidecode -t memory | grep -E '^\s+Size:'
$ sudo lshw -short -sanitize > ~/hardware.txt
```

চ্যালেঞ্জ: একটা script লেখো যেটা CPU মডেল, মোট RAM (`/proc/meminfo` থেকে), kernel version, আর প্রতিটি নেটওয়ার্ক কার্ডের driver (`/sys/class/net/*/device/driver`) এক পাতায় দেখায়। (অধ্যায় ৪১ দেখো।)

ল্যাব ৯ (অ্যাডভান্সড, VM-এ): নিজের প্রথম kernel module

```
$ sudo apt install build-essential linux-headers-$(uname -r)    # Fedora: sudo dnf
install gcc make kernel-devel
$ mkdir ~/hello-mod && cd ~/hello-mod
```

`hello.c`:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Rahim");
MODULE_DESCRIPTION("Amar prothom kernel module");

static int __init hello_init(void)
```

```
{
    pr_info("hello: kernel-e swagotom!\n");
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("hello: bidai!\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

`Makefile` (লাইনের শুরুতে **Tab** দিতে হবে, space দিলে চলবে না):

```
obj-m += hello.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

```
$ make
$ modinfo ./hello.ko
$ sudo insmod ./hello.ko
$ sudo dmesg | tail -1
[ 9876.543210] hello: kernel-e swagotom!
$ lsmod | grep hello
$ sudo rmmod hello
$ sudo dmesg | tail -1
[ 9890.123456] hello: bidai!
$ cat /proc/sys/kernel/tainted      # out-of-tree module, তাই tainted হয়েছে
```

> ⚠️ Secure Boot চালু থাকলে `insmod` "Key was rejected by service" এরর দেবে। পরীক্ষার VM-এ Secure Boot বন্ধ রাখো, অথবা ৪৭.১০-এর মতো MOK দিয়ে sign করো।

সারসংক্ষেপ

কাজ	কমান্ড / ফাইল
kernel version	`uname -r`, `uname -a`, `cat /proc/version`
boot প্যারামিটার	`cat /proc/cmdline`
built-in নাকি module	`grep CONFIG_X /boot/config-$(uname -r)`
লোড হওয়া module	`lsmod` (= `/proc/modules`)
module-এর তথ্য	`modinfo <name>`, `modinfo -F parm <name>`

কাজ	কমান্ড / ফাইল
লোড / আনলোড	`sudo modprobe <name>` / `sudo modprobe -r <name>`
dry run	`modprobe -n -v <name>`
module dependency map আবার বানানো	`sudo depmod -a`
boot-এ লোড	`/etc/modules-load.d/*.conf`
module option ও blacklist	`/etc/modprobe.d/*.conf` (`options` , `blacklist` , `install ... /bin/false`)
initramfs আবার বানানো	`sudo update-initramfs -u` / `sudo dracut -f`
CPU / মেমরি	`/proc/cpuinfo` , `/proc/meminfo` , `lscpu` , `free -h`
process-এর তথ্য	`/proc/<PID>/{status,cmdline,fd,exe,cwd,environ}`
ডিভাইসের গাছ	`/sys/class` , `/sys/block` , `/sys/devices` , `/sys/module`
kernel প্যারামিটার পড়া	`sysctl <name>` , `sysctl -a`
সাময়িক বদল	`sudo sysctl -w <name>=<val>`
স্থায়ী বদল	`/etc/sysctl.d/99-*.conf` + `sudo sysctl --system`
udev তথ্য / লাইভ event	`udevadm info -a -n /dev/X` , `udevadm monitor`
udev rule আবার পড়া	`sudo udevadm control --reload && sudo udevadm trigger`
হার্ডওয়্যার	`lspci -k` , `lsusb -t` , `sudo dmidecode -t memory` , `sudo lshw -short`
ইনস্টল করা kernel	`dpkg --list 'linux-image*'` / `rpm -q kernel`
ডিফল্ট kernel (RHEL/Fedora)	`sudo grubby --default-kernel` , `--set-default`
DKMS	`dkms status` , `sudo dkms autoinstall`
Secure Boot অবস্থা	`mokutil --sb-state`
kernel বার্তা	`sudo dmesg -T` , `journalctl -k -b -1`
tainted?	`cat /proc/sys/kernel/tainted`

শর্টকাট	কাজ
Alt+SysRq+R,E,I,S,U,B	জমে যাওয়া সিস্টেম নিরাপদে reboot (`kernel.sysrq` অনুমতি দিলে)
Alt+SysRq+F	OOM killer ডাকা
Esc / Shift (boot-এর সময়)	GRUB মেনু দেখানো
`e` তারপর Ctrl+X / F10	GRUB-এ kernel প্যারামিটার সাময়িক বদলে boot
Ctrl+C	`dmesg -w` বা `udevadm monitor` থামানো

নিজেকে যাচাই করো

1. `uname -r` দেখাচ্ছে `6.8.0-45-generic`। এর `6.8`, `45` আর `generic` অংশগুলো কী বোঝায়?
2. `modprobe` আর `insmod`-এর মধ্যে মূল পার্থক্য কী? সাধারণ কাজে কোনটা ব্যবহার করবে এবং কেন?
3. `/etc/modprobe.d/`-এ `blacklist usb_storage` লেখার পরও কেউ হাতে `modprobe usb_storage` চালালে কী হবে? পুরোপুরি আটকাতে কী লিখবে?
4. `/proc/meminfo` ফাইলের সাইজ ০ বাইট দেখায়, তবু `cat` করলে লেখা আসে কেন?
5. কোনো process ভুল করে মুছে ফেলা একটা log ফাইল এখনও খোলা রেখেছে। ফাইলটা কীভাবে উদ্ধার করবে?
6. `sudo echo 1 > /proc/sys/net/ipv4/ip_forward` কেন "Permission denied" দেয়? সঠিক উপায় দুটি বলো।
7. `sysctl -w` দিয়ে বদলানো মান reboot-এর পরও রাখতে কী করবে? ফাইলের নামের শুরুতে `99-` কেন দেওয়া হয়?
8. একটা USB-serial ডিভাইসকে সবসময় `/dev/gps` নামে পেতে চাও। কোন টুল দিয়ে attribute খুঁজবে, আর rule কোথায় রাখবে?
9. একটা নেটওয়ার্ক কার্ড কাজ করছে না। কোন কমান্ড দিয়ে দেখবে তার জন্য কোনো driver লোড হয়েছে কিনা?
10. kernel আপডেটের পর VirtualBox কাজ করছে না। DKMS-এর দিক থেকে কী কী পরীক্ষা করবে?

▼ উত্তর দেখো

1. `6.8` হলো upstream kernel সিরিজ (major 6, minor 8)। `45` হলো Ubuntu-র ABI/build নম্বর, যেটা distro-র প্রতিটি kernel প্যাকেজ আপডেটে বাড়ে। `generic` হলো flavour, অর্থাৎ সাধারণ ডেস্কটপ/সার্ভারের জন্য বানানো (অন্যগুলো হলো `lowlatency`, `aws` ইত্যাদি)।
2. `insmod` একটা নির্দিষ্ট `.ko` ফাইল সরাসরি লোড করে: পুরো path লাগে, dependency দেখে না, `/etc/modprobe.d` মানে না। `modprobe` নাম দিয়ে কাজ করে, `modules.dep` থেকে dependency আগে লোড করে, কনফিগ (options, blacklist) মানে, আর `-r` দিয়ে পরিচ্ছন্নভাবে আনলোড করে। তাই সাধারণ কাজে `modprobe` ব্যবহার করবে।
3. লোড হয়ে যাবে, কারণ `blacklist` শুধু alias দিয়ে স্বয়ংক্রিয় লোড বন্ধ করে। পুরোপুরি আটকাতে লেখো `install usb_storage /bin/false`, তারপর initramfs আবার বানাও।
4. `/proc` একটা virtual filesystem (procfs)। ডিস্কে কোনো ডেটা নেই; পড়ার মুহূর্তে kernel লেখাটা তৈরি করে দেয়, তাই আগে থেকে সাইজ জানা যায় না।
5. `ls -l /proc/<PID>/fd/` দেখো, `(deleted)` লেখা লিংকটা খুঁজে বের করো, তারপর `cp /proc/<PID>/fd/<N> ~/recovered.log` চালাও।
6. redirection (`>`) চালায় তোমার সাধারণ shell, `sudo` নয়, তাই ফাইল খোলার অনুমতি পাওয়া যায় না। সঠিক উপায়: `echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward`, অথবা `sudo sysctl -w net.ipv4.ip_forward=1`।
7. `/etc/sysctl.d/99-custom.conf`-এ `key = value` লিখে `sudo sysctl --system` চালাও। ফাইলগুলো নাম অনুযায়ী ক্রমে পড়া হয়, আর পরে পড়া মান টেকে; `99-` দিলে ফাইলটা সবার

শেষে পড়া হয়, ফলে অন্য কোনো ফাইল তোমার মান বদলে দিতে পারে না।

8. `lsusb` দিয়ে idVendor/idProduct, আর `udevadm info -a -n /dev/ttyUSB0` দিয়ে বিস্তারিত attribute খুঁজবে। rule রাখবে `/etc/udev/rules.d/99-gps.rules`-এ, যেমন `SUBSYSTEM=="tty", ATTRS{idVendor}=="xxxx", ATTRS{idProduct}=="yyyy", SYMLINK+="gps"`। তারপর `udevadm control --reload` ও `udevadm trigger`।
9. `lspci -k` (বা USB হলে `lsusb -t`) চালাও; `Kernel driver in use` লাইন না থাকলে driver লোড হয়নি। সাথে `sudo dmesg -T | grep -i <driver/firmware>` দেখো, আর `ip link` দিয়ে দেখো ইন্টারফেস তৈরি হয়েছে কিনা।
10. `dkms status` দেখো, নতুন kernel-এর জন্য `installed` আছে কিনা। নতুন kernel-এর headers ইনস্টল আছে কিনা যাচাই করো (`linux-headers-$(uname -r)` বা `kernel-devel`)। তারপর `sudo dkms autoinstall` চালাও। build ব্যর্থ হলে `make.log` দেখো। Secure Boot চালু থাকলে MOK enroll করা আছে কিনা দেখো (`mokutil --sb-state`, আর লোডের সময় "Key was rejected" এরর আসছে কিনা)।

অধ্যায় ৪৮: কন্টেইনার ও ভার্চুয়ালাইজেশন

এই অধ্যায়ে যা শিখবে

- Virtualization (ভার্চুয়ালাইজেশন) কী, কেন দরকার, আর Virtual Machine (VM) ও Container-এর মধ্যে আসল পার্থক্য কোথায়
- Hypervisor কী — Type 1 আর Type 2-এর পার্থক্য
- KVM, QEMU, libvirt কীভাবে একসাথে কাজ করে, আর `virsh` / `virt-install` দিয়ে VM চালানো
- কন্টেইনার আসলে কোনো "জাদু" নয় — লিনাক্স kernel-এর কয়েকটা ফিচার জোড়া লাগিয়ে বানানো: `chroot`, **namespaces**, **cgroups v2**, **OverlayFS**
- `unshare` দিয়ে হাতে-কলমে নিজের "মিনি কন্টেইনার" বানানো
- Docker: ইনস্টল, image ও container, `run -d -p -v --name`, `ps`, `logs`, `exec`, `stop`, `rm`, volume, network
- `Dockerfile` লেখা, image build করা, multi-stage build
- `docker compose` দিয়ে একাধিক কন্টেইনার একসাথে চালানো
- Podman — daemon ছাড়া কন্টেইনার, আর rootless container কীভাবে কাজ করে
- Kubernetes এক নজরে — কী, কেন, আর মূল ধারণাগুলো
- কন্টেইনার ও VM-সংক্রান্ত সব দরকারি কীবোর্ড শর্টকাট

৪৮.১ ভার্চুয়ালাইজেশন কেন দরকার?

ধরো তোমার একটা বিশাল বাড়ি আছে (একটা শক্তিশালী সার্ভার — ৬৪ কোর CPU, ২৫৬ GB RAM)। তুমি সেখানে একটা মাত্র ওয়েবসাইট চালাচ্ছ, যেটা CPU-র ৫% ব্যবহার করে। বাকি ৯৫% অলস পড়ে আছে। আবার, তোমার আরেকটা অ্যাপ লাগবে যেটা পুরনো Python ভার্সন চায়, আর প্রথম অ্যাপটা নতুন ভার্সন চায় — একই মেশিনে দুজনের ঝগড়া!

সমাধান দুটো:

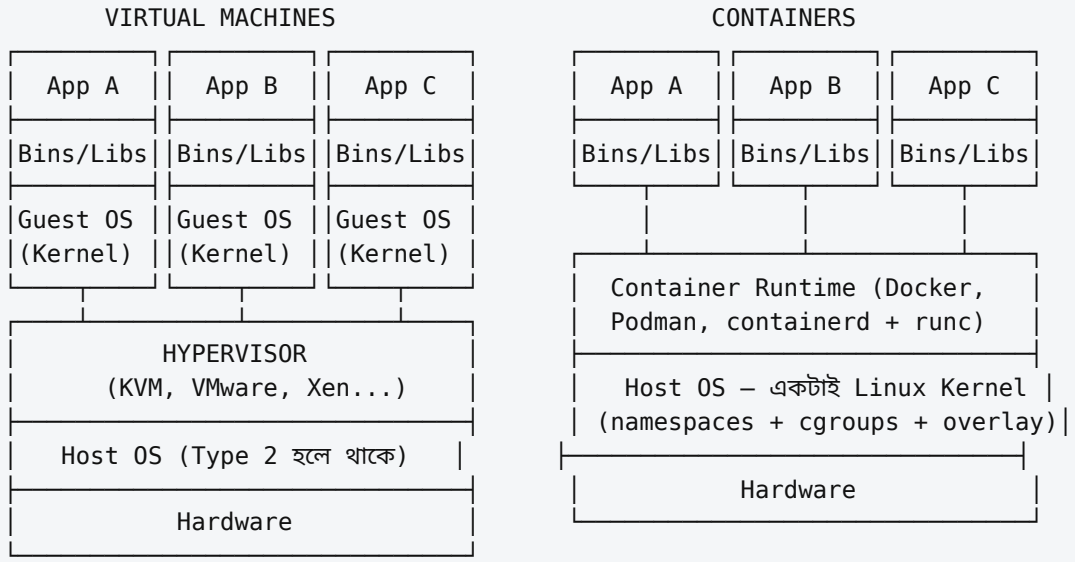
1. **Virtualization (ভার্চুয়ালাইজেশন):** বাড়িটাকে আলাদা আলাদা **ফ্ল্যাটে** ভাগ করো। প্রতিটা ফ্ল্যাটের নিজস্ব রান্নাঘর, বাথরুম, পানির লাইন — সম্পূর্ণ আলাদা সংসার। এটাই **Virtual Machine** — প্রতিটা VM-এর নিজের পুরো অপারেটিং সিস্টেম, নিজের kernel।
2. **Containerization (কন্টেইনারাইজেশন):** বাড়িটাকে **হোস্টেলের রুমে** ভাগ করো। প্রত্যেকের নিজের রুম, নিজের তালা — কিন্তু রান্নাঘর, পানির লাইন, বিদ্যুৎ সবাই শেয়ার করে। এটাই **Container** — সবাই হোস্টের একটাই kernel শেয়ার করে, শুধু চারপাশে দেয়াল তোলা।

> 📝 **মনে রাখো:** VM = আলাদা kernel সহ পুরো কম্পিউটারের নকল। Container = একই kernel, কিন্তু আলাদা করে রাখা (isolated) process।

সুবিধাগুলো এক নজরে

প্রয়োজন	ভার্চুয়ালাইজেশন/কন্টেইনার কীভাবে সাহায্য করে
হার্ডওয়্যারের পুরো ব্যবহার	এক মেশিনে অনেক VM/container চালানো যায়
আলাদা আলাদা পরিবেশ	প্রতিটা অ্যাপের নিজস্ব লাইব্রেরি ভার্সন
"আমার মেশিনে তো চলে!" সমস্যা	Container image-এ সব dependency বাঁধা থাকে, সব জায়গায় একইভাবে চলে
নিরাপদ পরীক্ষা	VM নষ্ট হলে মুছে আবার বানাও, হোস্টের কিছু হয় না
দ্রুত deploy ও scale	নতুন container সেকেন্ডে চালু হয়
ভিন্ন OS চালানো	লিনাক্সের ভেতরে Windows VM চালানো যায় (container দিয়ে যায় না)

৪৮.২ VM বনাম Container — আর্কিটেকচার ডায়াগ্রাম



বিষয়	Virtual Machine	Container
Kernel	প্রতিটা VM-এর নিজস্ব	হোস্টের kernel শেয়ার
আকার	কয়েক GB	কয়েক MB থেকে কয়েকশো MB
চালু হতে সময়	৩০ সেকেন্ড – কয়েক মিনিট (পুরো OS বুট)	মিলিসেকেন্ড – কয়েক সেকেন্ড (শুধু একটা process)
Isolation (বিচ্ছিন্নতা)	খুব শক্তিশালী (হার্ডওয়্যার-লেভেল)	তুলনামূলক দুর্বল (kernel শেয়ার, kernel bug হলে ঝুঁকি)
ভিন্ন OS	হ্যাঁ — Linux-এ Windows, BSD চলে	না — লিনাক্স হোস্টে শুধু লিনাক্স container

বিষয়	Virtual Machine	Container
Overhead	বেশি (প্রতিটার আলাদা RAM, kernel)	প্রায় শূন্য
এক সার্ভারে কতগুলো	ডজনখানেক	শত শত
ব্যবহার	ভিন্ন OS, শক্ত isolation, পুরনো সিস্টেম	microservices, CI/CD, অ্যাপ প্যাকেজিং

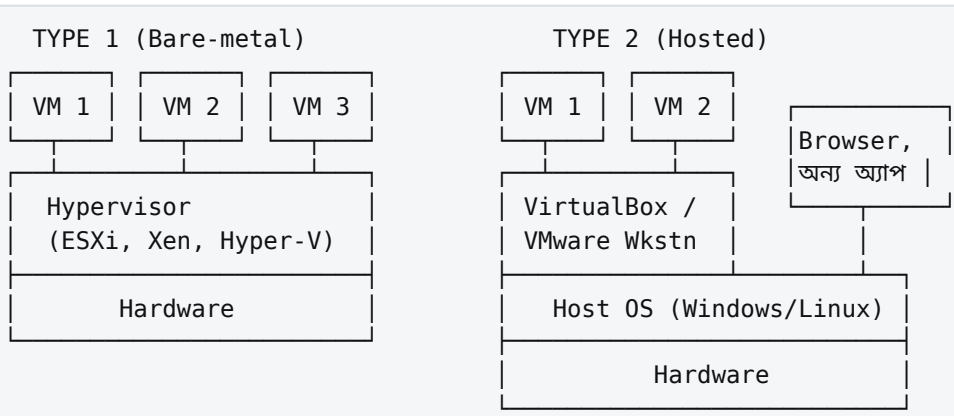
> 💡 **টিপস:** বাস্তবে দুটো একসাথে ব্যবহার হয়। AWS/GCP-তে তুমি একটা VM ভাড়া নাও, আর সেই VM-এর ভেতরে Docker container চালাও।

> ⚠️ **সতর্কতা:** Mac বা Windows-এ Docker Desktop আসলে গোপনে একটা ছোট লিনাক্স VM চালায়, তার ভেতরে container চলে — কারণ লিনাক্স container-এর জন্য লিনাক্স kernel লাগবেই।

৪৮.৩ Hypervisor — VM-এর পরিচালক

Hypervisor (হাইপারভাইজার, অন্য নাম VMM — Virtual Machine Monitor) হলো সেই সফটওয়্যার যে আসল হার্ডওয়্যারকে ভাগ করে প্রতিটা VM-কে মনে করায় "এই পুরো কম্পিউটারটা আমার"। উপমা: একজন বাড়িওয়ালা, যে প্রতিটা ফ্ল্যাটের ভাড়াটেকে তার ভাগের বিদ্যুৎ-পানি মেপে দেয় আর কেউ যেন অন্যের ফ্ল্যাটে ঢুকতে না পারে তা দেখে।

Type 1 বনাম Type 2



	Type 1 (Bare-metal)	Type 2 (Hosted)
কোথায় চলে	সরাসরি হার্ডওয়্যারের উপর	একটা সাধারণ OS-এর উপর অ্যাপ হিসেবে
উদাহরণ	VMware ESXi, Xen, Microsoft Hyper-V, **KVM**	Oracle VirtualBox, VMware Workstation, Parallels
পারফরম্যান্স	বেশি	তুলনামূলক কম
ব্যবহার	ডেটা সেন্টার, ক্লাউড	ডেস্কটপে শেখা/টেস্ট

KVM কোন টাইপ?

KVM (Kernel-based Virtual Machine) হলো লিনাক্স kernel-এর একটা মডিউল (`kvm.ko` আর `kvm_intel.ko` বা `kvm_amd.ko`)। এটা লোড হলে লিনাক্স kernel নিজেই hypervisor হয়ে যায়। যেহেতু এটা kernel-এর অংশ এবং সরাসরি হার্ডওয়্যার নিয়ন্ত্রণ করে, তাই KVM-কে সাধারণত **Type 1** ধরা হয় — যদিও লিনাক্স একসাথে সাধারণ OS হিসেবেও কাজ করে, তাই কেউ কেউ একে "হাইব্রিড" বলে। AWS (Nitro), Google Cloud — সবাই KVM-ভিত্তিক।

হার্ডওয়্যার সাপোর্ট — VT-x / AMD-V

আধুনিক CPU-তে virtualization-এর জন্য বিশেষ নির্দেশ (instruction) থাকে: Intel-এ **VT-x** (`vmx` flag), AMD-তে **AMD-V** (`svm` flag)। এগুলো ছাড়া KVM চলবে না।

```
$ grep -Ec '(vmx|svm)' /proc/cpuinfo
8
```

০-এর বেশি হলে CPU সাপোর্ট করে (সংখ্যাটা মূলত logical CPU-র সংখ্যা)। ০ হলে BIOS/UEFI সেটিংসে গিয়ে "Intel Virtualization Technology" বা "SVM Mode" চালু করো।

```
$ lsmod | grep kvm
kvm_intel             487424  0
kvm                  1404928  1 kvm_intel

$ ls -l /dev/kvm
crw-rw----+ 1 root kvm 10, 232 Nov 12 09:10 /dev/kvm
```

Ubuntu/Debian-এ সহজ চেকার:

```
$ sudo apt install cpu-checker
$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used
```

KVM + QEMU + libvirt — কে কী করে?

এই তিনটা নাম প্রায় সবসময় একসাথে আসে, তাই ভূমিকা পরিষ্কার করে নিই:

```
┌──────────────────────────────────────────────────────┐
│  ব্যবহারকারী:  virt-manager (GUI)   virsh (CLI)  virt-install │
└──────────────────────────┬───────────────────────────┘
                           │ (API কল)
┌──────────────────────────▼───────────────────────────┐
│  libvirtd / virtqemud  — ম্যানেজমেন্ট ডেমন (libvirt)          │
│  VM-এর XML কনফিগ, নেটওয়ার্ক, স্টোরেজ পুল সামলায়              │
└──────────────────────────┬───────────────────────────┘
                           │ (প্রতিটা VM-এর জন্য একটা process চালায়)
┌──────────────────────────▼──────────────────────────┐
```

```
┌──────────────────────────────────────────────────────────┐
│  qemu-system-x86_64  — USER SPACE                        │
│  ভার্চুয়াল ডিস্ক, নেটওয়ার্ক কার্ড, USB, ডিসপ্লে ইত্যাদি        │
│  ডিভাইস "নকল" (emulate) করে                               │
└──────────────────────────┬───────────────────────────────┘
                           │ ioctl(/dev/kvm)
┌──────────────────────────▼───────────────────────────────┐
│  KVM kernel module  — KERNEL SPACE                       │
│  CPU ও মেমরি virtualization (VT-x/AMD-V ব্যবহার করে)        │
└──────────────────────────┬───────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────┐
│              Hardware (CPU with VT-x / AMD-V)            │
└──────────────────────────────────────────────────────────┘
```

উপাদান	কাজ	উপমা
KVM	CPU আর RAM-কে নিরাপদে VM-এর হাতে তুলে দেয়, প্রায় native গতিতে	বাড়ির মূল কাঠামো আর বিদ্যুতের লাইন
QEMU	বাকি সব হার্ডওয়্যার (ডিস্ক, নেটওয়ার্ক কার্ড, মাউস, স্ক্রিন) emulate করে। KVM ছাড়াও একা চলতে পারে (তখন খুব ধীর, কিন্তু ARM-এর মতো অন্য আর্কিটেকচারও চালাতে পারে)	ফ্ল্যাটের আসবাবপত্র
libvirt	সব VM ম্যানেজ করার একটা কমন API ও ডেমন	বাড়ির ম্যানেজার অফিস
virsh	libvirt-এর কমান্ড-লাইন ক্লায়েন্ট	ম্যানেজারের সাথে কথা বলার ফোন
virt-manager	libvirt-এর GUI	ম্যানেজার অফিসের রিসেপশন ডেস্ক

প্রতিটা চলমান VM হোস্টে আসলে একটা সাধারণ লিনাক্স process মাত্র — `ps aux | grep qemu` দিলে দেখতে পাবে। VM-এর প্রতিটা vCPU সেই process-এর একটা thread।

KVM/libvirt ইনস্টল

```
# Ubuntu 24.04 / Debian 12
$ sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients virtinst virt-manager
$ sudo usermod -aG libvirt,kvm $USER     # এরপর logout/login করো

# Fedora
$ sudo dnf install @virtualization
$ sudo systemctl enable --now libvirtd

# RHEL 9 / Rocky / Alma
$ sudo dnf install qemu-kvm libvirt virt-install virt-viewer
$ sudo systemctl enable --now libvirtd
```

> 📝 **মনে রাখো:** নতুন libvirt "modular daemon" ব্যবহার করে (`virtqemud` , `virtnetworkd` ইত্যাদি)। কিছু distro-তে `libvirtd` একক ডেমন, কিছুতে modular। `virsh` দুটোর সাথেই একইভাবে

> কাজ করে।

`virsh` দুই রকম "connection"-এ কাজ করে: `qemu:///system` (root-লেভেল, সিস্টেমের সব VM — `sudo` বা `libvirt` গ্রুপ লাগে) আর `qemu:///session` (শুধু তোমার ইউজারের VM)। `virsh` কোনটায় যাচ্ছে না বুঝলে স্পষ্ট করে বলো: `virsh -c qemu:///system list --all`।

`qemu-img` — ভার্চুয়াল ডিস্ক

```
$ qemu-img create -f qcow2 ~/vms/test.qcow2 20G
Formatting '/home/rahim/vms/test.qcow2', fmt=qcow2 cluster_size=65536 extended_l2=off
compression_type=zlib size=21474836480 lazy_refcounts=off refcount_bits=16

$ qemu-img info ~/vms/test.qcow2
image: /home/rahim/vms/test.qcow2
file format: qcow2
virtual size: 20 GiB (21474836480 bytes)
disk size: 196 KiB
...
```

qcow2 (QEMU Copy-On-Write v2) ফরম্যাটের মজা হলো — ২০ GB "ভার্চুয়াল সাইজ" হলেও আসল ডিস্কে মাত্র ১৯৬ KB! ডেটা লেখার সাথে সাথে ফাইল বড় হয় (thin provisioning)। snapshot-ও সাপোর্ট করে। অন্য ফরম্যাট `raw` — দ্রুত, কিন্তু কোনো ফিচার নেই।

সরাসরি QEMU দিয়ে VM চালানো (libvirt ছাড়া — শেখার জন্য ভালো):

```
$ qemu-system-x86_64 -enable-kvm -m 2048 -smp 2 \
    -drive file=~/vms/test.qcow2,format=qcow2 \
    -cdrom ~/Downloads/ubuntu-24.04-live-server-amd64.iso -boot d
```

অপশন	অর্থ
`-enable-kvm` (বা `-accel kvm`)	KVM hardware acceleration ব্যবহার করো (না দিলে খুব ধীর)
`-m 2048`	2048 MB RAM
`-smp 2`	২টা vCPU
`-drive file=...`	ভার্চুয়াল হার্ডডিস্ক
`-cdrom file.iso`	ISO-কে CD ড্রাইভ হিসেবে
`-boot d`	CD থেকে বুট করো
`-nographic`	কোনো উইন্ডো না, সিরিয়াল কনসোল টার্মিনালেই

`virt-install` — libvirt দিয়ে নতুন VM

```
$ sudo virt-install \
    --name ubuntu-vm \
    --memory 2048 \
```

```
    --vcpus 2 \
    --disk size=20 \
    --cdrom ~/Downloads/ubuntu-24.04-live-server-amd64.iso \
    --os-variant ubuntu24.04 \
    --network network=default \
    --graphics vnc
```

`--os-variant`-এর সঠিক নাম জানতে: `osinfo-query os | grep -i ubuntu` (`libosinfo-bin` প্যাকেজ)। `--disk size=20` দিলে libvirt ডিফল্ট storage pool-এ (সাধারণত `/var/lib/libvirt/images/`) ২০ GB-র qcow2 ডিস্ক বানিয়ে নেয়।

`virsh` — VM নিয়ন্ত্রণের রিমোট কন্ট্রোল

```
$ sudo virsh list --all
 Id   Name        State
----------------------------
 1    ubuntu-vm   running
 -    debian-vm   shut off
```

কমান্ড	কাজ
`virsh list` / `virsh list --all`	চলমান / সব VM দেখাও
`virsh start NAME`	VM চালু করো
`virsh shutdown NAME`	ভদ্রভাবে বন্ধ (ACPI সিগন্যাল — যেন পাওয়ার বাটন চাপলে)
`virsh destroy NAME`	জোর করে বন্ধ (পাওয়ার কর্ড টানার মতো — ডিস্ক মোছে **না**)
`virsh reboot NAME`	রিবুট
`virsh suspend NAME` / `virsh resume NAME`	VM থামিয়ে রাখো (RAM-এ) / আবার চালাও
`virsh dominfo NAME`	VM-এর তথ্য (CPU, RAM, state)
`virsh domifaddr NAME`	VM-এর IP ঠিকানা
`virsh console NAME`	সিরিয়াল কনসোলে ঢোকো (বের হতে `Ctrl+]`)
`virsh autostart NAME`	হোস্ট বুট হলে VM-ও চালু হবে
`virsh edit NAME`	VM-এর XML কনফিগ এডিটরে খোলো
`virsh dumpxml NAME`	XML কনফিগ দেখাও
`virsh snapshot-create-as NAME snap1`	snapshot নাও
`virsh snapshot-list NAME`	snapshot তালিকা
`virsh snapshot-revert NAME snap1`	snapshot-এ ফিরে যাও
`virsh undefine NAME`	VM-এর সংজ্ঞা মুছে ফেলো (`--remove-all-storage` দিলে ডিস্কও)
`virsh net-list --all`	ভার্চুয়াল নেটওয়ার্ক তালিকা

কমান্ড	কাজ
`virsh pool-list --all`	storage pool তালিকা

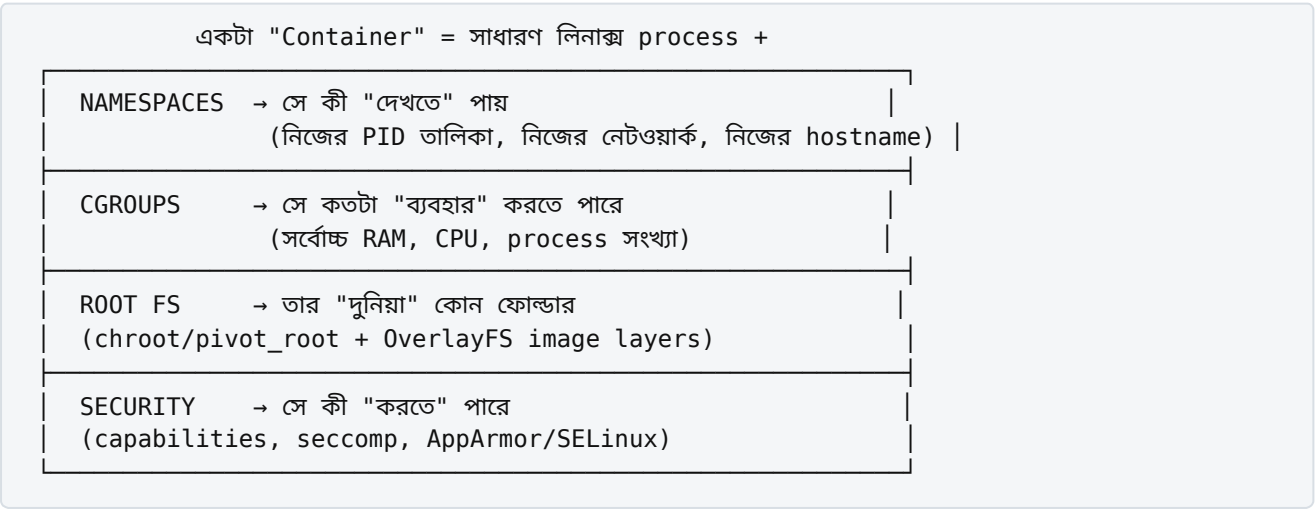
> ⚠️ **সতর্কতা:** `virsh destroy` নাম শুনে ভয় পেও না — এটা VM মুছে ফেলে না, শুধু হঠাৎ বন্ধ করে। কিন্তু `virsh undefine --remove-all-storage` সত্যিই সব মুছে ফেলে, ফিরিয়ে আনা যায় না।

libvirt-এর `default` নেটওয়ার্ক একটা NAT নেটওয়ার্ক — হোস্টে `virbr0` নামে একটা ব্রিজ ইন্টারফেস বানায় (সাধারণত `192.168.122.0/24`), আর `dnsmasq` দিয়ে VM-দের IP দেয়। VM বাইরের ইন্টারনেটে যেতে পারে, কিন্তু বাইরে থেকে সরাসরি VM-এ আসা যায় না (bridged নেটওয়ার্ক বানালে যায়)। নেটওয়ার্কিং-এর মূলনীতির জন্য দেখো অধ্যায় ৩৬।

> 💡 **টিপস:** GUI পছন্দ হলে `virt-manager` চালাও — ক্লিক করে VM বানানো, কনসোল দেখা, snapshot নেওয়া সব করা যায়। GNOME-এ সহজ বিকল্প **GNOME Boxes**।

৪৮.৪ কন্টেইনার আসলে কী দিয়ে তৈরি?

এখানেই সবচেয়ে গুরুত্বপূর্ণ কথাটা: **লিনাক্স kernel-এ "container" নামে কোনো জিনিস নেই।** Container হলো একটা সাধারণ process, যাকে kernel-এর কয়েকটা ফিচার দিয়ে ঘিরে রাখা হয়েছে:



হোস্টেলের উপমায়:

- **Namespace** = রুমের দেয়াল আর পর্দা — তুমি পাশের রুমের লোকদের দেখতে পাও না।
- **cgroup** = বিদ্যুতের মিটার আর ফিউজ — তুমি তোমার বরাদ্দের বেশি বিদ্যুৎ টানতে পারবে না।
- **chroot/OverlayFS** = তোমার রুমের আসবাব — তোমার জগৎ ওই রুমটুকু।
- **capabilities/seccomp** = হোস্টেলের নিয়মকানুন — তুমি মেইন সুইচ বন্ধ করতে পারবে না।

চলো একটা একটা করে দেখি।

৪৮.৪.১ chroot — সবচেয়ে পুরনো "কারাগার"

`chroot` (change root, ১৯৭৯ সালের Unix V7 থেকে আছে) একটা process-এর কাছে `/` (root ডিরেক্টরি) কোথায় সেটা বদলে দেয়। process মনে করে একটা ফোল্ডারই পুরো ফাইল সিস্টেম।

```
$ mkdir -p ~/jail
$ cp --parents /bin/bash /bin/ls ~/jail/
$ for lib in $(ldd /bin/bash /bin/ls | grep -o '/[^ ]*'); do
>     cp --parents "$lib" ~/jail/
> done
$ sudo chroot ~/jail /bin/bash
bash-5.2# ls /
bin  lib  lib64
bash-5.2# cd ..
bash-5.2# pwd
/
bash-5.2# exit
```

এখানে `ldd` দেখায় `bash` আর `ls` কোন কোন shared library-র উপর নির্ভর করে, আর `cp --parents` পুরো পাথসহ কপি করে। jail-এর ভেতরে `cd ..` দিয়েও বাইরে যাওয়া যায় না।

কিন্তু chroot একা যথেষ্ট নয়:

- jail-এর ভেতরের process এখনও `ps` চালালে (যদি `/proc` মাউন্ট করা থাকে) হোস্টের সব process দেখবে।
- নেটওয়ার্ক পুরোটাই হোস্টের।
- root ইউজার chroot থেকে কিছু কৌশলে বেরিয়ে আসতে পারে।
- কোনো রিসোর্স লিমিট নেই।

তাই আধুনিক container runtime `chroot`-এর বদলে নিরাপদ `pivot_root` system call ব্যবহার করে, আর তার সাথে যোগ করে namespaces ও cgroups।

৪৮.৪.২ Namespaces — কে কী দেখতে পাবে

Namespace হলো kernel-এর একটা ফিচার যা কোনো একটা গ্লোবাল রিসোর্সকে "মোড়ক" দিয়ে আলাদা করে — যাতে namespace-এর ভেতরের process-রা মনে করে রিসোর্সটা শুধু তাদেরই।

Namespace	`unshare` অপশন	কী আলাদা করে	উদাহরণ
mnt (Mount)	`--mount` / `-m`	মাউন্ট পয়েন্টের তালিকা	container-এর নিজের `/`
pid (Process ID)	`--pid` / `-p`	process ID নম্বর	container-এর প্রথম process-এর PID = 1

Namespace	`unshare` অপশন	কী আলাদা করে	উদাহরণ
net (Network)	`--net` / `-n`	নেটওয়ার্ক ইন্টারফেস, IP, routing table, firewall rule, port	container-এর নিজের `eth0`, নিজের port 80
uts (UNIX Time-Sharing)	`--uts` / `-u`	hostname ও domain name	container-এর আলাদা hostname
ipc (Inter-Process Comm.)	`--ipc` / `-i`	System V IPC, POSIX message queue, shared memory	এক container অন্যটার shared memory পায় না
user	`--user` / `-U`	UID/GID ম্যাপিং	ভেতরে root (UID 0), বাইরে সাধারণ ইউজার
cgroup	`--cgroup` / `-C`	cgroup ট্রি-র দৃশ্য	container নিজের cgroup-কেই root দেখে
time	`--time` / `-T`	`CLOCK_MONOTONIC` ও `CLOCK_BOOTTIME` অফসেট	container migration-এ uptime ঠিক রাখা

> 📝 **মনে রাখো:** `time` namespace-ও ঘড়ির "wall-clock" সময় (তারিখ/সময়) বদলাতে পারে না — শুধু monotonic আর boottime ঘড়ি।

প্রতিটা process কোন namespace-এ আছে তা `/proc/PID/ns/`-এ দেখা যায়:

```
$ ls -l /proc/$$/ns
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 mnt -> 'mnt:[4026531841]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 net -> 'net:[4026531840]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 time -> 'time:[4026531834]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 user -> 'user:[4026531837]'
lrwxrwxrwx 1 rahim rahim 0 Nov 12 10:00 uts -> 'uts:[4026531838]'
```

বন্ধনীর সংখ্যাটা namespace-এর পরিচয় (inode নম্বর)। দুটো process-এর সংখ্যা একই হলে তারা একই namespace-এ। (`$$` = বর্তমান shell-এর PID; `/proc` নিয়ে বিস্তারিত অধ্যায় ৪৭-এ।)

সিস্টেমের সব namespace দেখতে `lsns`:

```
$ sudo lsns -t net
        NS TYPE NPROCS   PID USER      NETNSID NSFS COMMAND
4026531840 net     312     1 root   unassigned      /sbin/init splash
4026532501 net       2  4120 root            0      nginx: master process nginx -g
daemon off;
```

Namespace-এর পেছনের system call

System call	কাজ
`clone()` / `clone3()`	নতুন process বানাও, চাইলে নতুন namespace-এ (`CLONE_NEWPID`, `CLONE_NEWNET` ইত্যাদি flag)
`unshare()`	বর্তমান process-কে নতুন namespace-এ সরিয়ে নাও
`setns()`	একটা বিদ্যমান namespace-এ ঢোকো (`docker exec` এটাই করে!)

`unshare` ডেমো — নিজের হাতে container বানাও

`unshare` কমান্ড (util-linux প্যাকেজ) নতুন namespace-এ একটা প্রোগ্রাম চালায়।

ধাপ ১ — UTS namespace (আলাদা hostname):

```
$ hostname
my-laptop
$ sudo unshare --uts bash
root@my-laptop:~# hostname container1
root@my-laptop:~# hostname
container1
root@my-laptop:~# exit
$ hostname
my-laptop          # হোস্টের hostname অক্ষত!
```

(prompt-এ পুরনো নাম দেখাচ্ছে কারণ prompt shell শুরুর সময় তৈরি; নতুন shell খুললে নতুন নাম দেখাবে।)

ধাপ ২ — PID namespace (নিজের process ট্রি):

```
$ sudo unshare --pid --fork --mount-proc bash
root@my-laptop:~# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   9184  5248 pts/2    S    10:05   0:00 bash
root           8  0.0  0.0  11320  4608 pts/2    R+   10:05   0:00 ps aux
root@my-laptop:~# exit
```

এখানে:

- `--pid` — নতুন PID namespace।
- `--fork` — দরকারি! `unshare` নিজে নতুন PID namespace-এ ঢোকে না, শুধু তার *child*-রা ঢোকে। তাই fork করে child হিসেবে bash চালাতে হয়, যে PID 1 পায়।
- `--mount-proc` — নতুন mount namespace বানিয়ে সেখানে নতুন `/proc` মাউন্ট করে। নাহলে `ps` পুরনো `/proc` পড়ে হোস্টের সব process দেখাত।

দেখো — container-এর ভেতরে bash হলো **PID 1**, আর মাত্র দুটো process দেখা যাচ্ছে! অথচ হোস্ট থেকে (অন্য টার্মিনালে) `ps aux | grep bash` দিলে এই bash-কে সাধারণ একটা বড় PID (যেমন 5123) হিসেবে দেখবে। **একই process, দুই জগতে দুই নম্বর।**

> 📝 **মনে রাখো:** PID 1 বিশেষ — সে মারা গেলে পুরো PID namespace-এর সব process kernel মেরে ফেলে। এজন্যই container-এর মূল process বন্ধ হলে container-ও বন্ধ হয়ে যায়।

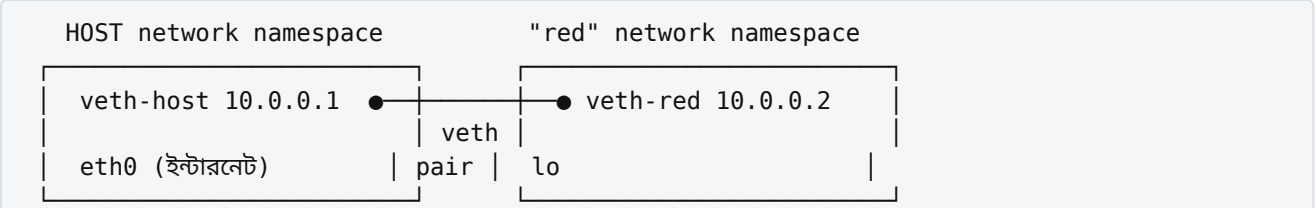
ধাপ ৩ — Network namespace:

```
$ sudo unshare --net bash
root@my-laptop:~# ip addr
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
root@my-laptop:~# ping -c1 8.8.8.8
ping: connect: Network is unreachable
root@my-laptop:~# exit
```

নতুন network namespace সম্পূর্ণ খালি — শুধু একটা বন্ধ (DOWN) loopback। কোনো ইন্টারনেট নেই। Docker এখানে একটা **veth pair** (ভার্চুয়াল ইথারনেট তার — এক মাথা container-এ, অন্য মাথা হোস্টের ব্রিজে) জুড়ে দেয়।

`ip netns` দিয়ে নাম দেওয়া network namespace বানানো ও জোড়া লাগানো যায়:

```
$ sudo ip netns add red
$ sudo ip link add veth-host type veth peer name veth-red
$ sudo ip link set veth-red netns red
$ sudo ip addr add 10.0.0.1/24 dev veth-host
$ sudo ip link set veth-host up
$ sudo ip netns exec red ip addr add 10.0.0.2/24 dev veth-red
$ sudo ip netns exec red ip link set veth-red up
$ sudo ip netns exec red ip link set lo up
$ ping -c2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.048 ms
$ sudo ip netns delete red        # পরিষ্কার করো (veth জোড়াও মুছে যায়)
```



এটাই হুবহু Docker-এর bridge নেটওয়ার্কের মূলনীতি — শুধু Docker মাঝখানে `docker0` নামে একটা ব্রিজ রাখে আর NAT rule যোগ করে।

ধাপ ৪ — সব একসাথে:

```
$ sudo unshare --uts --pid --net --mount --ipc --fork --mount-proc bash
root@my-laptop:~# hostname mini-container
```

```
root@my-laptop:~# ps aux | wc -l
3
```

অভিনন্দন — তুমি Docker ছাড়াই প্রায় একটা container বানিয়ে ফেলেছ! বাকি শুধু: আলাদা root ফাইলসিস্টেম, রিসোর্স লিমিট (cgroups), আর নিরাপত্তা নিয়ম।

ধাপ ৫ — User namespace (root ছাড়া "root"):

```
$ id
uid=1000(rahim) gid=1000(rahim) groups=1000(rahim),27(sudo)
$ unshare --user --map-root-user bash
root@my-laptop:~# id
uid=0(root) gid=0(root) groups=0(root),65534(nogroup)
root@my-laptop:~# touch /etc/hack
touch: cannot touch '/etc/hack': Permission denied
root@my-laptop:~# exit
```

`sudo` ছাড়াই ভেতরে তুমি "root"! কিন্তু এই root শুধু namespace-এর ভেতরে — বাইরের দুনিয়ায় সে এখনও UID 1000, তাই হোস্টের `/etc`-তে লিখতে পারে না। এটাই **rootless container**-এর ভিত্তি (৪৮.১০-এ বিস্তারিত)।

> ⚠️ **সতর্কতা (Ubuntu 23.10+/24.04):** Ubuntu AppArmor দিয়ে unprivileged user namespace সীমিত করে (`kernel.apparmor_restrict_unprivileged_userns = 1`)। উপরের `id` ডেমো চলবে, কিন্তু namespace-এর ভেতরে mount-এর মতো ক্ষমতা কাজ নাও করতে পারে। Podman/Docker rootless-এর জন্য প্যাকেজের সাথে AppArmor প্রোফাইল আসে, তাই সেগুলো ঠিকমতো চলে।

`nsenter` — চলমান container-এ ঢোকা

```
$ PID=$(docker inspect -f '{{.State.Pid}}' web)
$ sudo nsenter --target "$PID" --net ip addr
```

এটা `web` container-এর network namespace-এ ঢুকে **হোস্টের** `ip` কমান্ড চালায় — container image-এ `ip` টুল না থাকলেও! ডিবাগিংয়ে খুব কাজের। `docker exec` ভেতরে এই একই `setns()` কৌশল ব্যবহার করে।

৪৮.৪.৩ cgroups v2 — কে কতটা ব্যবহার করবে

Namespace ঠিক করে process কী **দেখবে**; কিন্তু একটা container যদি সার্ভারের সব RAM খেয়ে ফেলে? তখন দরকার **cgroups** (Control Groups) — process-দের গ্রুপে ভাগ করে প্রতিটা গ্রুপের রিসোর্স ব্যবহার **সীমিত, মাপা ও নিয়ন্ত্রণ** করার kernel ফিচার।

উপমা: হোস্টেলের প্রতিটা রুমে আলাদা বিদ্যুৎ মিটার আর ফিউজ। কেউ বেশি টানলে তার রুমের ফিউজ কাটবে, পুরো হোস্টেলের না।

v1 বনাম v2: পুরনো cgroups v1-এ প্রতিটা রিসোর্সের (cpu, memory...) আলাদা আলাদা ট্রি ছিল — জটিল ও বিশৃঙ্খল। **cgroups v2**-এ একটাই ইউনিফাইড ট্রি। Ubuntu 22.04+, Debian 11+, Fedora 31+, RHEL 9 — সবাই ডিফল্টে v2 ব্যবহার করে।

```
$ stat -fc %T /sys/fs/cgroup
cgroup2fs          # v2 (যদি "tmpfs" দেখায়, তাহলে v1 বা hybrid)

$ cat /sys/fs/cgroup/cgroup.controllers
cpuset cpu io memory hugetlb pids rdma misc
```

cgroup ট্রি আসলে `/sys/fs/cgroup`-এ মাউন্ট করা একটা ভার্চুয়াল ফাইলসিস্টেম। **ফোল্ডার বানানো = নতুন cgroup বানানো**, আর ফাইলে লেখা = লিমিট সেট করা।

```
/sys/fs/cgroup/                          ← root cgroup
├── cgroup.controllers                   ← কোন কোন controller আছে
├── cgroup.subtree_control               ← child-দের জন্য কোনগুলো চালু
├── system.slice/                        ← systemd সার্ভিসগুলো
│   ├── docker.service/
│   ├── ssh.service/
│   └── docker-4f3a...e1.scope/          ← একটা Docker container!
│       ├── cgroup.procs                 ← এই গ্রুপের PID তালিকা
│       ├── memory.max                   ← RAM লিমিট
│       ├── memory.current               ← এখন কত RAM ব্যবহার করছে
│       ├── cpu.max                      ← CPU লিমিট
│       └── pids.max                     ← সর্বোচ্চ process সংখ্যা
└── user.slice/                          ← লগইন করা ইউজারদের process
```

গুরুত্বপূর্ণ ফাইলগুলো:

ফাইল	অর্থ	উদাহরণ মান
`cgroup.procs`	এই cgroup-এর process-দের PID; PID লিখলে process এখানে সরে আসে	`4120`
`memory.max`	সর্বোচ্চ RAM; বেশি হলে OOM killer মারে	`512M` বা `max`
`memory.high`	"নরম" সীমা — পেরোলে kernel জোর করে মেমরি ফেরত নেয় (throttle)	`400M`
`memory.current`	বর্তমান ব্যবহার (শুধু পড়া যায়)	`104857600`
`memory.swap.max`	সর্বোচ্চ swap	`0`
`cpu.max`	`QUOTA PERIOD` (মাইক্রোসেকেন্ড): প্রতি PERIOD-এ সর্বোচ্চ QUOTA সময় CPU	`50000 100000` = আধা CPU
`cpu.weight`	CPU ভাগাভাগির অনুপাত (১–১০০০০, ডিফল্ট ১০০)	`200` = দ্বিগুণ ভাগ
`io.max`	ডিস্ক I/O লিমিট (device অনুযায়ী)	`8:0 rbps=1048576`
`pids.max`	সর্বোচ্চ process/thread সংখ্যা (fork bomb ঠেকায়)	`100`
`cpu.stat`, `memory.stat`	পরিসংখ্যান	—

হাতে-কলমে: নিজের cgroup বানাও

```
$ sudo mkdir /sys/fs/cgroup/demo
$ ls /sys/fs/cgroup/demo | head -5
cgroup.controllers
cgroup.events
cgroup.freeze
cgroup.kill
cgroup.max.depth

$ echo 100M | sudo tee /sys/fs/cgroup/demo/memory.max
100M
$ echo "20000 100000" | sudo tee /sys/fs/cgroup/demo/cpu.max     # ২০% CPU
20000 100000
$ echo 20 | sudo tee /sys/fs/cgroup/demo/pids.max
20

# বর্তমান shell-কে এই cgroup-এ ঢোকাও
$ echo $$ | sudo tee /sys/fs/cgroup/demo/cgroup.procs
$ cat /proc/$$/cgroup
0::/demo
```

এখন এই shell থেকে যা-ই চালাবে, সব মিলিয়ে ১০০ MB RAM, ২০% CPU, আর ২০টার বেশি process পাবে না। যেমন `yes > /dev/null` চালিয়ে অন্য টার্মিনালে `top` দেখলে দেখবে ওটা ~২০% CPU-তে আটকে আছে।

পরিষ্কার করতে: shell বন্ধ করো (cgroup খালি হবে), তারপর `sudo rmdir /sys/fs/cgroup/demo`। (খালি না হলে `rmdir` "Device or resource busy" দেবে।)

> 💡 **টিপস:** হাতে ফাইল না লিখে systemd দিয়েই একই কাজ করা যায় — যে কোনো কমান্ডকে লিমিটসহ চালাও:
>
> ```
> $ systemd-run --user --scope -p MemoryMax=200M -p CPUQuota=30% stress-ng --cpu 2
> ```
>
> আর পুরো cgroup ট্রি দেখতে `systemd-cgls`, লাইভ রিসোর্স ব্যবহার দেখতে `systemd-cgtop`। systemd নিয়ে বিস্তারিত অধ্যায় ২৬-এ।

Docker যখন `docker run --memory 512m --cpus 1.5` চালায়, ঠিক এই ফাইলগুলোতেই লেখে: `memory.max = 536870912` আর `cpu.max = 150000 100000`।

৪৮.৪.৪ OverlayFS — স্তরে স্তরে ফাইলসিস্টেম

একটা Ubuntu image ~৮০ MB। ১০০টা container চালালে কি ৮ GB ডিস্ক লাগবে? না! কারণ **OverlayFS** (union filesystem)।

উপমা: একটা ছাপানো বইয়ের উপর **ট্রেসিং পেপার** রাখো। তুমি ট্রেসিং পেপারে যা খুশি লেখো, কাটো — নিচের বই অক্ষত থাকে। উপর থেকে তাকালে দুটো মিলিয়ে একটা পাতা দেখায়। ১০০ জন ছাত্র একই বই শেয়ার করতে পারে,

প্রত্যেকের শুধু নিজের ট্রেসিং পেপার।

```
          merged  (container যা দেখে — একীভূত দৃশ্য)
┌──────────────────────────────────────────────────┐
│  /etc/nginx.conf(নতুন)  /app/data  /bin/sh  ...  │
└──────────────────────────────────────────────────┘
                    ▲  উপরের স্তর জেতে
┌──────────────────────────────────────────────────┐
│ upperdir  (লেখা যায়, container-এর নিজস্ব)       │ ← container layer
│   etc/nginx.conf (পরিবর্তিত কপি), app/data       │
├──────────────────────────────────────────────────┤
│ lowerdir 3 (read-only)  — COPY app/ ...          │ ┐
├──────────────────────────────────────────────────┤ │ image layers
│ lowerdir 2 (read-only)  — apt install nginx      │ │ (সব container
├──────────────────────────────────────────────────┤ │  শেয়ার করে)
│ lowerdir 1 (read-only)  — ubuntu base            │ ┘
└──────────────────────────────────────────────────┘
```

নিয়মগুলো:

- **পড়া:** ফাইল উপরের স্তরে থাকলে সেটা, নাহলে নিচের স্তর থেকে।
- **লেখা (Copy-on-Write):** নিচের স্তরের কোনো ফাইল বদলালে আগে পুরো ফাইলটা upperdir-এ কপি হয়, তারপর বদলায়। নিচেরটা কখনো বদলায় না।
- **মোছা:** নিচের স্তরের ফাইল মুছলে upperdir-এ একটা **whiteout** ফাইল (বিশেষ character device, 0/0) তৈরি হয়, যা ওই ফাইলকে "লুকিয়ে" রাখে।

নিজে OverlayFS মাউন্ট করো

```
$ mkdir -p ~/ov/{lower,upper,work,merged}
$ echo "আমি নিচের স্তরে" > ~/ov/lower/a.txt
$ echo "আমিও নিচে"     > ~/ov/lower/b.txt

$ sudo mount -t overlay overlay \
    -o lowerdir=$HOME/ov/lower,upperdir=$HOME/ov/upper,workdir=$HOME/ov/work \
    ~/ov/merged

$ ls ~/ov/merged
a.txt  b.txt

$ echo "বদলে দিলাম" | sudo tee ~/ov/merged/a.txt
$ sudo rm ~/ov/merged/b.txt
$ sudo touch ~/ov/merged/c.txt

$ cat ~/ov/lower/a.txt
আমি নিচের স্তরে                  # নিচের স্তর অক্ষত!
$ ls -l ~/ov/upper
-rw-r--r-- 1 rahim rahim    29 Nov 12 11:00 a.txt   # পরিবর্তিত কপি
c---------  1 root  root  0, 0 Nov 12 11:00 b.txt   # whiteout!
-rw-r--r-- 1 root  root     0 Nov 12 11:00 c.txt   # নতুন ফাইল

$ sudo umount ~/ov/merged
```

অপশন	কাজ
`lowerdir=`	read-only স্তর(গুলো); একাধিক হলে `:` দিয়ে, **বামেরটা সবচেয়ে উপরে**
`upperdir=`	লেখার স্তর
`workdir=`	kernel-এর অভ্যন্তরীণ কাজের জন্য খালি ডিরেক্টরি; upperdir-এর সাথে একই ফাইলসিস্টেমে হতে হবে

Docker-এর ক্লাসিক storage driver `overlay2` ঠিক এটাই করে। চলমান container-এর মাউন্ট দেখো:

```
$ mount | grep overlay
overlay on /var/lib/docker/overlay2/7c1e.../merged type overlay
(rw,relatime,lowerdir=/var/lib/docker/overlay2/l/ABC...:/var/lib/docker/overlay2/l/DEF.
..,upperdir=...,workdir=...)
```

> 📝 **মনে রাখো:** container মুছলে তার upperdir মুছে যায় — অর্থাৎ container-এর ভেতরে লেখা সব ডেটা হারিয়ে যায়! স্থায়ী ডেটার জন্য **volume** লাগে (৪৮.৬.৬-এ দেখব)।

> 💡 **টিপস:** নতুন Docker ভার্সনে (v29 থেকে নতুন ইনস্টলে) image রাখার দায়িত্ব containerd-এর "image store"-কে দেওয়া হয়েছে, তখন `docker info`-তে storage driver হিসেবে `overlayfs` (containerd snapshotter) দেখাতে পারে। নাম যা-ই হোক, ভেতরের কৌশল এই OverlayFS-ই।

৪৮.৪.৫ নিরাপত্তার স্তর — capabilities, seccomp, LSM

Namespace আর cgroup ছাড়াও runtime আরও কয়েকটা দেয়াল দেয়:

স্তর	কাজ
Capabilities	root-এর ক্ষমতাকে ~৪০টা টুকরোয় ভাগ করা (`CAP_NET_ADMIN`, `CAP_SYS_ADMIN`...)। Docker ডিফল্টে container-এর root-কে মাত্র ~১৪টা দেয়।
seccomp	কোন system call চালানো যাবে তার ফিল্টার। Docker-এর ডিফল্ট প্রোফাইল ৪০+ বিপজ্জনক syscall বন্ধ রাখে (যেমন `reboot`, kernel module লোড)।
AppArmor / SELinux	Mandatory Access Control — Ubuntu-তে AppArmor, Fedora/RHEL-এ SELinux। বিস্তারিত অধ্যায় ৪৫-এ।
read-only rootfs	`--read-only` দিলে container নিজের ফাইলসিস্টেমে লিখতে পারে না।

> ⚠️ **সতর্কতা:** `docker run --privileged` এই সব দেয়াল ভেঙে দেয় — container প্রায় হোস্টের root-এর সমান ক্ষমতা পায়। একান্ত দরকার না হলে কখনো ব্যবহার করবে না।

৪৮.৫ Container ইকোসিস্টেম — কে কোন কাজ করে

Image বনাম Container

	Image	Container
কী	read-only টেমপ্লেট (স্তরের সমষ্টি + মেটাডেটা)	image থেকে চালানো একটা জীবন্ত instance
উপমা	রান্নার রেসিপি / class	রান্না করা খাবার / object
অবস্থা	স্থির	চলে, থামে, মরে
সংখ্যা	একটা image	তা থেকে অসংখ্য container

Registry হলো image-এর লাইব্রেরি/অ্যাপ-স্টোর: Docker Hub (`docker.io`), GitHub (`ghcr.io`), Quay (`quay.io`), নিজস্ব প্রাইভেট registry।

Image-এর পুরো নাম: `registry/namespace/repository:tag`

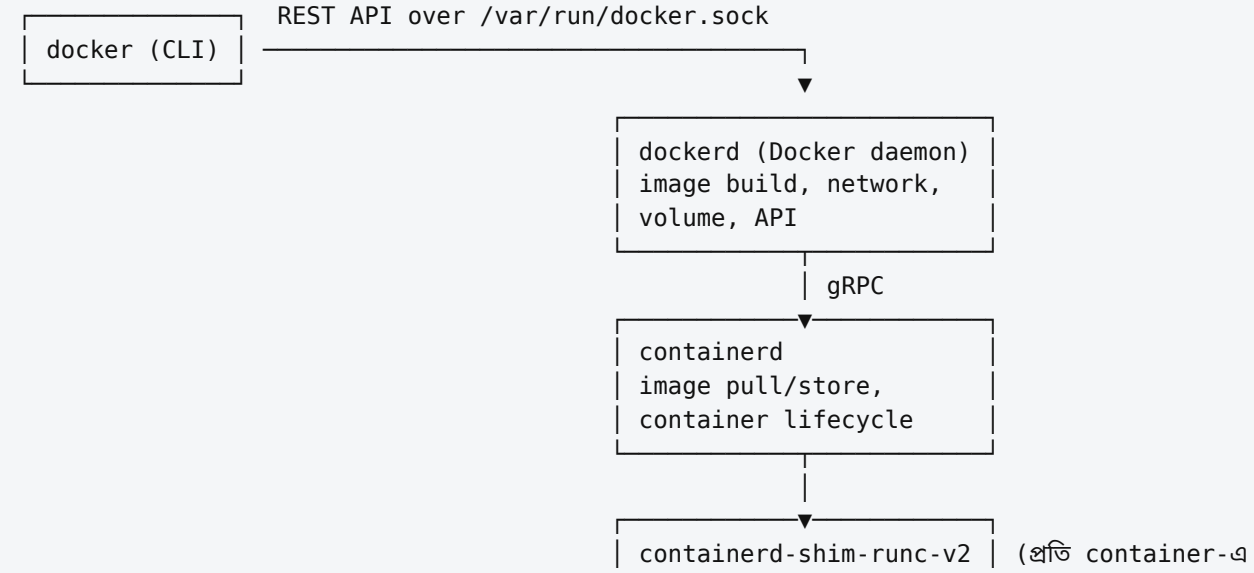
```
docker.io / library / nginx : 1.27-alpine
└registry┘ └namespace┘ └repo┘ └── tag ──┘
```

`nginx` লিখলে আসলে মানে `docker.io/library/nginx:latest` । (`latest` মানে "সবচেয়ে নতুন" নয় — শুধু একটা ডিফল্ট tag নাম! প্রোডাকশনে সবসময় নির্দিষ্ট ভার্সন tag দাও।)

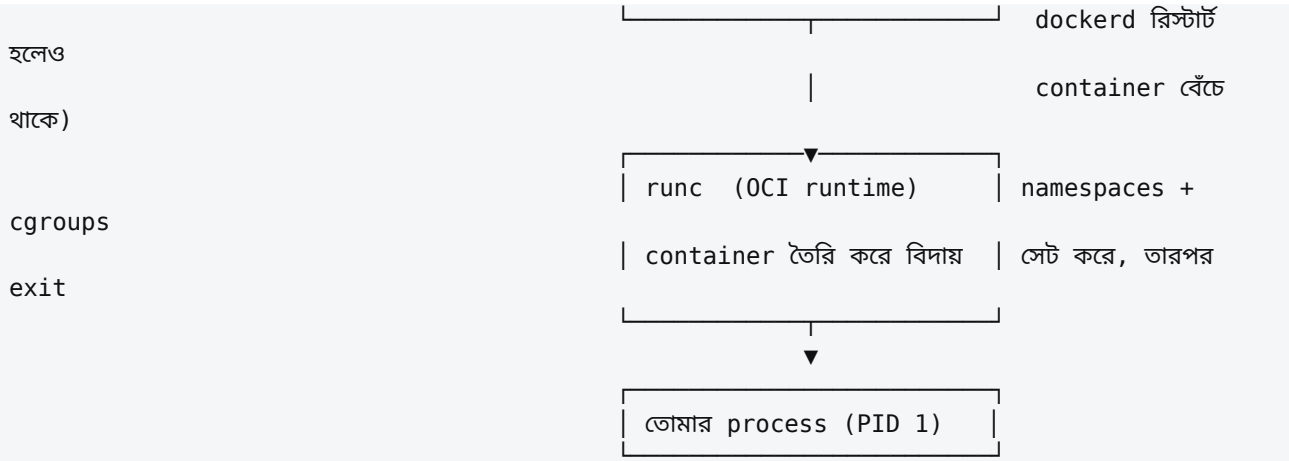
OCI — সবার জন্য এক নিয়ম

OCI (Open Container Initiative) image ফরম্যাট আর runtime-এর স্ট্যান্ডার্ড ঠিক করে। তাই Docker দিয়ে বানানো image Podman, Kubernetes সব জায়গায় চলে।

Docker-এর ভেতরের আর্কিটেকচার



একটা;



উপাদান	ভূমিকা
`docker`	শুধু একটা ক্লায়েন্ট — তুমি যা টাইপ করো API-তে পাঠায়
`dockerd`	মূল daemon, root হিসেবে চলে
`containerd`	container-এর জীবনচক্র সামলানোর "ইন্ডাস্ট্রি-স্ট্যান্ডার্ড" runtime (Kubernetes-ও সরাসরি এটা ব্যবহার করে)
`runc`	নিম্ন-স্তরের runtime — `clone()`, `unshare()`, cgroup ফাইলে লেখা, `pivot_root` — ঠিক যা আমরা উপরে হাতে করলাম

৪৮.৬ Docker — হাতে-কলমে

৪৮.৬.১ ইনস্টল

> ⚠️ **সতর্কতা:** Ubuntu-র নিজস্ব `docker.io` প্যাকেজ আর Docker-এর অফিশিয়াল `docker-ce` প্যাকেজ একসাথে রেখো না। আগে পুরনো/বিরোধী প্যাকেজ সরাও:
>
> ```
> $ for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do
> sudo apt-get remove -y $pkg; done
> ```

Ubuntu 24.04 (অফিশিয়াল Docker রিপোজিটরি থেকে):

```
$ sudo apt update
$ sudo apt install ca-certificates curl
$ sudo install -m 0755 -d /etc/apt/keyrings
$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
\
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" \
```

```
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt update
$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

Debian 12-এ একই ধাপ, শুধু URL-এ `ubuntu`-র বদলে `debian`।

Fedora:

```
$ sudo dnf install dnf-plugins-core
$ sudo dnf config-manager addrepo --from-
repofile=https://download.docker.com/linux/fedora/docker-ce.repo
$ sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
$ sudo systemctl enable --now docker
```

RHEL 9 / Rocky / Alma:

```
$ sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-
ce.repo
$ sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
$ sudo systemctl enable --now docker
```

(Fedora 41+-এর `dnf5`-এ `config-manager addrepo` সিনট্যাক্স, পুরনো `dnf4`-এ `--add-repo`। RHEL-এ Red Hat নিজে Docker-এর বদলে **Podman** দেয় — ৪৮.৯ দেখো।)

যাচাই:

```
$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
...
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

`hello-world` চালাতে ভেতরে যা ঘটল: CLI → dockerd-কে বলল; dockerd দেখল image লোকালে নেই → Docker Hub থেকে pull করল → container বানাল → container বার্তা ছাপিয়ে exit করল।

sudo ছাড়া docker:

```
$ sudo usermod -aG docker $USER
$ newgrp docker          # অথবা logout/login
$ docker ps
```

> ⚠️ **সতর্কতা:** `docker` গ্রুপের সদস্য হওয়া মানে কার্যত **root** হওয়া! কারণ `docker run -v /:/host -it ubuntu chroot /host` দিয়ে যে কেউ হোস্টের পুরো ফাইলসিস্টেমে root হয়ে ঢুকতে পারে। শুধু বিশ্বস্ত

> ইউজারকে এই গ্রুপে দাও, অথবা rootless Docker/Podman ব্যবহার করো।

৪৮.৬.২ Image নিয়ে কাজ

```
$ docker pull nginx:1.27
$ docker images            # বা: docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         1.27      3b25b682ea82   3 weeks ago    192MB
hello-world   latest    d2c94e258dcb   18 months ago  13.3kB

$ docker image history nginx:1.27      # কোন স্তর কীভাবে তৈরি
$ docker image inspect nginx:1.27      # সব মেটাডেটা (JSON)
$ docker rmi hello-world               # image মোছো
$ docker image prune                   # dangling (নামহীন) image মোছো
$ docker search postgres               # Docker Hub-এ খোঁজো
$ docker tag nginx:1.27 myrepo/nginx:v1  # নতুন নাম দাও (কপি না, শুধু লেবেল)
$ docker login && docker push myrepo/nginx:v1
```

৪৮.৬.৩ `docker run` — সবচেয়ে গুরুত্বপূর্ণ কমান্ড

Syntax: `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

```
$ docker run -d -p 8080:80 -v ~/site:/usr/share/nginx/html:ro --name web nginx:1.27
7f3c2a9b1d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a
```

এক লাইনে ভেঙে দেখি:

অংশ	অর্থ
`-d` (`--detach`)	ব্যাকগ্রাউন্ডে চালাও, container ID ছাপিয়ে টার্মিনাল ফেরত দাও
`-p 8080:80` (`--publish`)	**হোস্টের** port 8080 → **container-এর** port 80
`-v ~/site:/usr/share/nginx/html:ro` (`--volume`)	হোস্টের `~/site` ফোল্ডার container-এর ভেতরে ওই পাথে; `:ro` = read-only
`--name web`	container-এর নাম `web` (না দিলে Docker মজার র‍্যান্ডম নাম দেয়, যেমন `quirky_turing`)
`nginx:1.27`	কোন image থেকে

এখন ব্রাউজারে `http://localhost:8080` খোলো — nginx পেজ!

`docker run`-এর অন্যান্য দরকারি অপশন:

অপশন	অর্থ
`-it`	`-i` (stdin খোলা) + `-t` (pseudo-terminal) — ইন্টারঅ্যাক্টিভ shell-এর জন্য
`--rm`	container থামলেই নিজে মুছে যাবে

অপশন	অর্থ
`-e KEY=VALUE` (`--env`)	environment variable
`--env-file .env`	ফাইল থেকে env var
`-p 127.0.0.1:8080:80`	শুধু localhost-এ খোলো (বাইরে থেকে না)
`-P`	image-এর সব `EXPOSE` করা port র‍্যান্ডম হোস্ট port-এ
`--network NAME`	নির্দিষ্ট নেটওয়ার্কে যুক্ত করো
`--restart unless-stopped`	crash/রিবুটে আবার চালু হবে (`no`, `on-failure[:N]`, `always`, `unless-stopped`)
`--memory 512m` / `-m 512m`	RAM লিমিট (cgroup `memory.max`)
`--cpus 1.5`	CPU লিমিট (cgroup `cpu.max`)
`--pids-limit 100`	process সংখ্যা লিমিট
`-w /app` (`--workdir`)	ভেতরের working directory
`-u 1000:1000` (`--user`)	কোন UID:GID হিসেবে চালাবে
`--read-only`	rootfs read-only
`--hostname NAME`	container-এর hostname (UTS namespace)
`--entrypoint CMD`	image-এর ENTRYPOINT বদলাও
`--mount type=bind,src=...,dst=...`	`-v`-এর আরও স্পষ্ট, নতুন সিনট্যাক্স

```
# ইন্টারঅ্যাক্টিভ, কাজ শেষে মুছে যাবে
$ docker run -it --rm ubuntu:24.04 bash
root@a1b2c3d4e5f6:/# cat /etc/os-release | head -2
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
root@a1b2c3d4e5f6:/# exit

# ডেটাবেস — env var সহ
$ docker run -d --name db -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres:16
```

> 💡 **টিপস:** hostname-এ যে হিজিবিজি `a1b2c3d4e5f6` — ওটা container ID-র প্রথম ১২ অক্ষর। Docker UTS namespace দিয়ে এটা সেট করে।

৪৮.৬.৪ চলমান container দেখা ও নিয়ন্ত্রণ

```
$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS
PORTS                                     NAMES
7f3c2a9b1d4e   nginx:1.27    "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   web
```

```
$ docker ps -a          # থেমে থাকাগুলোও (Exited)
$ docker ps -q          # শুধু ID (স্ক্রিপ্টে কাজের)
```

কমান্ড	কাজ
`docker logs web`	container-এর stdout/stderr দেখাও
`docker logs -f web`	লাইভ ফলো করো (`tail -f` -এর মতো)
`docker logs --tail 50 --since 10m web`	শেষ ৫০ লাইন, গত ১০ মিনিটের
`docker exec -it web bash`	চলমান container-এর ভেতরে নতুন shell (bash না থাকলে `sh`)
`docker exec web nginx -t`	ভেতরে একটা কমান্ড চালাও
`docker stop web`	ভদ্রভাবে থামাও: SIGTERM, ১০ সেকেন্ড পর SIGKILL
`docker kill web`	সঙ্গে সঙ্গে SIGKILL
`docker start web` / `docker restart web`	আবার চালু / রিস্টার্ট
`docker pause web` / `unpause`	freeze (cgroup freezer)
`docker rm web`	থামা container মোছো
`docker rm -f web`	চলমান হলেও জোর করে থামিয়ে মোছো
`docker inspect web`	সব বিস্তারিত JSON
`docker inspect -f '{{.NetworkSettings.IPAddress}}' web`	নির্দিষ্ট ফিল্ড
`docker stats`	লাইভ CPU/RAM/নেটওয়ার্ক ব্যবহার (`top` -এর মতো)
`docker top web`	container-এর process তালিকা
`docker cp web:/etc/nginx/nginx.conf .`	container ↔ হোস্ট ফাইল কপি
`docker diff web`	upperdir-এ কী কী বদলেছে (A=যোগ, C=পরিবর্তন, D=মোছা)
`docker port web`	port ম্যাপিং
`docker attach web`	container-এর মূল process-এর stdin/stdout-এ যুক্ত হও
`docker rename web web2`	নাম বদলাও
`docker update --memory 1g web`	চলমান container-এর লিমিট বদলাও

```
$ docker logs --tail 3 web
172.17.0.1 - - [12/Nov/2026:11:20:01 +0000] "GET / HTTP/1.1" 200 615 "-" "curl/8.5.0"
"-"
...

$ docker stats --no-stream
CONTAINER ID   NAME    CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK
I/O   PIDS
```

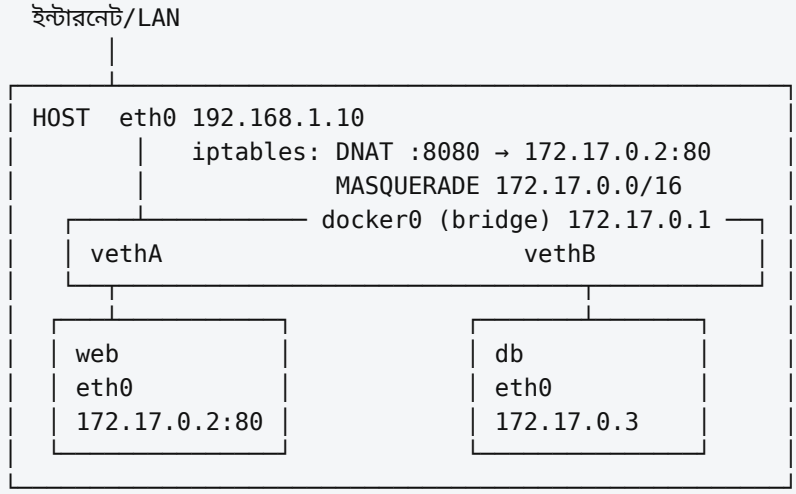
```
7f3c2a9b1d4e   web    0.00%     8.4MiB / 15.5GiB      0.05%     5.2kB / 2.1kB   0B /
8.19kB  9
```

> 📝 **মনে রাখো — container-এর জীবনচক্র:**
>
> ```
> docker create    docker start        docker stop/kill        docker rm
> ──────────► Created ──────► Running ──────────────────► Exited ──────► (মুছে গেল)
>                                │  ▲                        │
>                docker pause  ▼  │ unpause   docker start    │
>                              Paused                        └──► Running
> docker run = create + start
> ```

> 💡 **টিপস:** container ID পুরোটা লিখতে হয় না — যতটুকু লিখলে অনন্য হয় ততটুকুই যথেষ্ট: `docker stop 7f3`।

৪৮.৬.৫ Docker-এ নেটওয়ার্কিং

ডিফল্টে Docker হোস্টে `docker0` নামে একটা Linux bridge বানায় (সাধারণত `172.17.0.0/16`)। প্রতিটা container-এর network namespace থেকে একটা veth জোড়া এই ব্রিজে যুক্ত হয়, আর iptables/nftables-এ NAT (MASQUERADE) rule দিয়ে বাইরে যায়। `-p 8080:80` দিলে Docker একটা DNAT rule যোগ করে: হোস্টের 8080-এ আসা প্যাকেট → container-এর IP:80।



নেটওয়ার্ক ড্রাইভার	অর্থ
`bridge`	ডিফল্ট; একই হোস্টে container-রা নিজেদের মধ্যে কথা বলে
`host`	container আলাদা network namespace পায় না, হোস্টের নেটওয়ার্কই ব্যবহার করে (port ম্যাপিং লাগে না)
`none`	শুধু loopback, কোনো নেটওয়ার্ক নেই

নেটওয়ার্ক ড্রাইভার	অর্থ
`macvlan` / `ipvlan`	container LAN-এ নিজের আলাদা MAC/IP পায়
`overlay`	একাধিক হোস্ট জুড়ে (Swarm)

```
$ docker network create mynet
$ docker run -d --name db  --network mynet -e POSTGRES_PASSWORD=secret postgres:16
$ docker run -it --rm --network mynet postgres:16 psql -h db -U postgres
```

> 💡 **টিপস: নিজের বানানো** (user-defined) bridge নেটওয়ার্কে Docker একটা বিল্ট-ইন DNS দেয় — তাই container-রা একে অপরকে **নাম দিয়ে** খুঁজে পায় (`-h db`)। ডিফল্ট `bridge` নেটওয়ার্কে এই সুবিধা নেই। তাই সবসময় নিজের নেটওয়ার্ক বানাও।

> ⚠️ **সতর্কতা:** Docker নিজের iptables rule সরাসরি যোগ করে, যা অনেক সময় **ufw-কে পাশ কাটিয়ে যায়**। ufw-তে port বন্ধ থাকলেও `-p 5432:5432` দিলে ডেটাবেস ইন্টারনেটে খোলা হয়ে যেতে পারে! সমাধান: `-p 127.0.0.1:5432:5432` ব্যবহার করো, অথবা `DOCKER-USER` chain-এ rule দাও। ফায়ারওয়াল নিয়ে অধ্যায় ৩৯ দেখো।

৪৮.৬.৬ Volumes — স্থায়ী ডেটা

আগেই দেখেছি container মুছলে upperdir মুছে যায়। ডেটা বাঁচানোর তিনটা উপায়:

ধরন	সিনট্যাক্স	কোথায় থাকে	কখন
Named volume	`-v pgdata:/var/lib/postgresql/data`	`/var/lib/docker/volumes/pgdata/_data` — Docker সামলায়	ডেটাবেস, প্রোডাকশন ডেটা
Bind mount	`-v /home/rahim/site:/usr/share/nginx/html`	হোস্টের যেকোনো পাথ	ডেভেলপমেন্ট (কোড এডিট করে সাথে সাথে দেখা), কনফিগ ফাইল
tmpfs	`--tmpfs /tmp`	শুধু RAM-এ	অস্থায়ী গোপন ডেটা

`-v`-এর প্রথম অংশ `/` বা `./` দিয়ে শুরু হলে bind mount, নাহলে named volume।

```
$ docker volume create pgdata
$ docker volume ls
DRIVER    VOLUME NAME
local     pgdata
$ docker volume inspect pgdata
$ docker run -d --name db -e POSTGRES_PASSWORD=secret -v
pgdata:/var/lib/postgresql/data postgres:16
$ docker rm -f db           # container গেল...
```

```
$ docker run -d --name db -e POSTGRES_PASSWORD=secret -v
pgdata:/var/lib/postgresql/data postgres:16
                            # ...কিন্তু ডেটা আগের মতোই আছে!
$ docker volume rm pgdata   # volume মোছো (সাবধানে!)
$ docker volume prune       # অব্যবহৃত volume মোছো
```

> ⚠️ **সতর্কতা:** Fedora/RHEL-এ SELinux চালু থাকলে bind mount-এ "Permission denied" আসতে পারে। তখন `:z` (শেয়ার্ড) বা `:Z` (প্রাইভেট) সাফিক্স দাও: `-v ~/site:/usr/share/nginx/html:Z` — এটা ফোল্ডারের SELinux লেবেল ঠিক করে দেয়।

৪৮.৬.৭ পরিষ্কার-পরিচ্ছন্নতা

```
$ docker system df            # কে কতটা ডিস্ক খাচ্ছে
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          12        3         4.1GB     3.2GB (78%)
Containers      5         2         120MB     80MB (66%)
Local Volumes   4         2         1.2GB     300MB (25%)
Build Cache     40        0         900MB     900MB

$ docker container prune      # সব থামা container মোছো
$ docker system prune         # থামা container + অব্যবহৃত নেটওয়ার্ক + dangling image + build
cache
$ docker system prune -a --volumes   # ⚠️ সব অব্যবহৃত image ও volume-ও!
```

> ⚠️ **সতর্কতা:** `docker system prune -a --volumes` এক কমান্ডে তোমার ডেটাবেসের volume-ও মুছে দিতে পারে যদি সেই মুহূর্তে কোনো container সেটা ব্যবহার না করে। চালানোর আগে দুবার ভাবো।

৪৮.৭ Dockerfile — নিজের image বানাও

Dockerfile হলো image বানানোর রেসিপি — একটা সাধারণ টেক্সট ফাইল, যার প্রতিটা লাইন একটা নির্দেশ। `docker build` এটা উপর থেকে নিচে পড়ে, আর ফাইলসিস্টেম বদলায় এমন প্রতিটা নির্দেশ (`RUN`, `COPY`, `ADD`) একটা নতুন **স্তর (layer)** তৈরি করে।

একটা বাস্তব উদাহরণ — Python Flask অ্যাপ

প্রজেক্টের গঠন:

```
myapp/
├── app.py
├── requirements.txt
├── .dockerignore
└── Dockerfile
```

`app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "আসসালামু আলাইকুম, Docker থেকে!\n"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

`requirements.txt`:

```
flask==3.0.3
```

`Dockerfile`:

```
# ১. বেস image — কোথা থেকে শুরু
FROM python:3.12-slim

# ২. মেটাডেটা
LABEL maintainer="rahim@example.com"

# ৩. environment variable
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# ৪. ভেতরের কাজের ফোল্ডার (না থাকলে তৈরি হবে)
WORKDIR /app

# ৫. আগে শুধু dependency তালিকা কপি — ক্যাশের সুবিধার জন্য
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ৬. তারপর বাকি কোড
COPY . .

# ৭. root হিসেবে না চালিয়ে সাধারণ ইউজার হিসেবে
RUN useradd --create-home appuser
USER appuser

# ৮. ডকুমেন্টেশন: অ্যাপ এই port শোনে
EXPOSE 5000

# ৯. container চালু হলে কী চলবে
CMD ["python", "app.py"]
```

`.dockerignore` (build context থেকে এগুলো বাদ — `.gitignore`-এর মতো):

```
.git
__pycache__/
*.pyc
```

```
.env
venv/
```

Build ও চালানো:

```
$ cd myapp
$ docker build -t myapp:1.0 .
[+] Building 12.3s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => [1/5] FROM docker.io/library/python:3.12-slim
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
 ...
 => => naming to docker.io/library/myapp:1.0

$ docker run -d -p 5000:5000 --name myapp myapp:1.0
$ curl localhost:5000
আসসালামু আলাইকুম, Docker থেকে!
```

শেষের `.` হলো **build context** — এই ফোল্ডারের সব ফাইল (`.dockerignore` বাদে) builder-এর কাছে পাঠানো হয়। `COPY` শুধু এর ভেতরের ফাইলই কপি করতে পারে।

Dockerfile নির্দেশাবলী

নির্দেশ	কাজ	উদাহরণ
`FROM`	বেস image (প্রথম নির্দেশ হতে হবে, `ARG` ছাড়া)	`FROM debian:12-slim`
`RUN`	build-এর সময় কমান্ড চালাও, ফলাফল নতুন স্তর	`RUN apt-get update && apt-get install -y curl`
`COPY`	context থেকে image-এ ফাইল কপি	`COPY src/ /app/src/`
`ADD`	`COPY` -এর মতো, কিন্তু URL ও লোকাল tar স্বয়ংক্রিয় extract করে	`ADD app.tar.gz /opt/`
`WORKDIR`	পরের নির্দেশগুলোর working directory	`WORKDIR /app`
`ENV`	environment variable (build ও run দুই সময়েই)	`ENV NODE_ENV=production`
`ARG`	শুধু build-time variable	`ARG VERSION=1.0` → `docker build --build-arg VERSION=2.0`
`EXPOSE`	কোন port শোনে তার ডকুমেন্টেশন (নিজে port খোলে **না**)	`EXPOSE 80`
`USER`	কোন ইউজার হিসেবে চলবে	`USER 1000`
`VOLUME`	এই পাথ volume হবে বলে চিহ্নিত	`VOLUME /data`

নির্দেশ	কাজ	উদাহরণ
`CMD`	ডিফল্ট কমান্ড — `docker run IMAGE অন্য-কমান্ড` দিলে বদলে যায়	`CMD ["nginx", "-g", "daemon off;"]`
`ENTRYPOINT`	মূল executable — run-এর আর্গুমেন্ট এর পরে যোগ হয়	`ENTRYPOINT ["ping"]`
`HEALTHCHECK`	container সুস্থ কিনা যাচাইয়ের কমান্ড	`HEALTHCHECK CMD curl -f http://localhost/ || exit 1`
`LABEL`	মেটাডেটা	`LABEL version="1.0"`
`SHELL`	`RUN` -এর shell বদলাও	`SHELL ["/bin/bash", "-c"]`
`STOPSIGNAL`	`docker stop` কোন signal পাঠাবে	`STOPSIGNAL SIGQUIT`

CMD বনাম ENTRYPOINT

```
ENTRYPOINT ["ping", "-c", "3"]
CMD ["localhost"]
```

```
$ docker run pinger              # চলবে: ping -c 3 localhost
$ docker run pinger google.com   # চলবে: ping -c 3 google.com  (CMD বদলে গেল)
```

ENTRYPOINT = "কী চালাবে", CMD = "ডিফল্ট আর্গুমেন্ট"।

exec form বনাম shell form:

ফর্ম	লেখা	কী ঘটে
exec form ✅	`CMD ["python", "app.py"]`	python সরাসরি PID 1, `docker stop` -এর SIGTERM ঠিকমতো পায়
shell form	`CMD python app.py`	আসলে চলে `/bin/sh -c "python app.py"` — sh হয় PID 1, signal অনেক সময় অ্যাপে পৌঁছায় না, তাই `docker stop` ১০ সেকেন্ড অপেক্ষা করে SIGKILL দেয়

> 💡 **টিপস:** CMD/ENTRYPOINT-এ সবসময় exec form (JSON array) ব্যবহার করো।

Layer cache — কেন ক্রম গুরুত্বপূর্ণ

Docker প্রতিটা স্তর ক্যাশ করে। কোনো নির্দেশ বা তার ইনপুট ফাইল বদলালে **সেই স্তর ও তার পরের সব স্তর** আবার বানাতে হয়।

```
❌ খারাপ ক্রম                    ✅ ভালো ক্রম
COPY . .          ← কোড বদলালেই    COPY requirements.txt .
```

```
RUN pip install    ← আবার সব       RUN pip install    ← ক্যাশ থেকে (দ্রুত!)
                     install!      COPY . .           ← শুধু এটা নতুন
```

যা কম বদলায় সেটা উপরে, যা বেশি বদলায় সেটা নিচে।

Multi-stage build — ছোট image

Go বা Java-র মতো ভাষায় compiler লাগে build-এর সময়, কিন্তু চালানোর সময় না। Multi-stage build-এ এক stage-এ build করে শুধু ফলাফল পরের stage-এ নাও:

```
# Stage 1: build (বড় image, compiler সহ ~৮০০ MB)
FROM golang:1.23 AS builder
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/server .

# Stage 2: run (ছোট image)
FROM alpine:3.20
COPY --from=builder /out/server /usr/local/bin/server
USER nobody
ENTRYPOINT ["/usr/local/bin/server"]
```

চূড়ান্ত image-এ শুধু alpine (~৮ MB) + তোমার binary। compiler, সোর্স কোড কিছুই নেই — ছোট, দ্রুত, আর আক্রমণের সুযোগ কম।

ভালো অভ্যাস (Best practices)

1. ছোট বেস image: `-slim`, `alpine`, বা `distroless`।
2. নির্দিষ্ট ভার্সন tag: `FROM python:3.12-slim`, `latest` নয়।
3. `RUN` একত্র করো ও পরিষ্কার করো:

   ```
   RUN apt-get update && \
       apt-get install -y --no-install-recommends curl && \
       rm -rf /var/lib/apt/lists/*
   ```

   (আলাদা `RUN`-এ মুছলে লাভ নেই — আগের স্তরে ফাইল থেকেই যায়।)
4. root হিসেবে চালিও না — `USER` ব্যবহার করো।
5. পাসওয়ার্ড/কী image-এ রেখো না (`docker history`-তে দেখা যায়!) — runtime-এ env/secret দিয়ে দাও।
6. `.dockerignore` রাখো।
7. এক container = এক কাজ (একটা মূল process)।

Build-সংক্রান্ত কমান্ড

কমান্ড	কাজ
`docker build -t name:tag .`	build ও নাম দাও

কমান্ড	কাজ
`docker build -f Dockerfile.prod -t app .`	অন্য নামের Dockerfile
`docker build --no-cache -t app .`	ক্যাশ ছাড়া নতুন করে
`docker build --target builder -t app-dev .`	নির্দিষ্ট stage পর্যন্ত
`docker buildx build --platform linux/amd64,linux/arm64 -t user/app --push .`	একাধিক CPU আর্কিটেকচারের জন্য
`docker builder prune`	build cache মোছো
`docker commit web myimage:snap`	চলমান container থেকে image (শুধু পরীক্ষার জন্য; Dockerfile-ই সঠিক পথ)
`docker save -o app.tar app:1.0` / `docker load -i app.tar`	image ফাইলে রাখো / ফাইল থেকে আনো (ইন্টারনেট ছাড়া সরানো)

৪৮.৮ Docker Compose — অনেক container একসাথে

বাস্তব অ্যাপে থাকে ওয়েব সার্ভার + ডেটাবেস + ক্যাশ। প্রতিবার লম্বা `docker run` লেখা ক্লান্তিকর আর ভুলপ্রবণ। **Docker Compose** দিয়ে সবকিছু একটা YAML ফাইলে বর্ণনা করে এক কমান্ডে চালানো যায়।

> 📝 **মনে রাখো:** পুরনো `docker-compose` (হাইফেন সহ, Python-এ লেখা, v1) বাতিল হয়ে গেছে। এখন **`docker compose`** (স্পেস দিয়ে, v2, Docker CLI-এর plugin)। ফাইলের পছন্দের নাম `compose.yaml` (পুরনো `docker-compose.yml` -ও চলে)। ফাইলের শুরুতে `version: "3.8"` লাইন এখন আর লাগে না (obsolete)।

`compose.yaml`:

```
services:
  web:
    build: .                    # এই ফোল্ডারের Dockerfile থেকে build
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://app:secret@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
```

```
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 5s
      retries: 5

  cache:
    image: redis:7-alpine

volumes:
  pgdata:
```

Compose স্বয়ংক্রিয়ভাবে প্রজেক্টের জন্য একটা নেটওয়ার্ক বানায় (`myapp_default`), তাই `web` container `db` আর `cache` নামে অন্যদের খুঁজে পায়।

```
$ docker compose up -d
[+] Running 5/5
 ✔ Network myapp_default    Created
 ✔ Volume "myapp_pgdata"    Created
 ✔ Container myapp-db-1     Healthy
 ✔ Container myapp-cache-1  Started
 ✔ Container myapp-web-1    Started

$ docker compose ps
NAME            IMAGE           COMMAND                  SERVICE   CREATED
STATUS                    PORTS
myapp-cache-1   redis:7-alpine  "docker-entrypoint.s…"   cache     30 seconds ago   Up
29 seconds             6379/tcp
myapp-db-1      postgres:16     "docker-entrypoint.s…"   db        30 seconds ago   Up
29 seconds (healthy)   5432/tcp
myapp-web-1     myapp-web       "python app.py"          web       30 seconds ago   Up
23 seconds             0.0.0.0:5000->5000/tcp
```

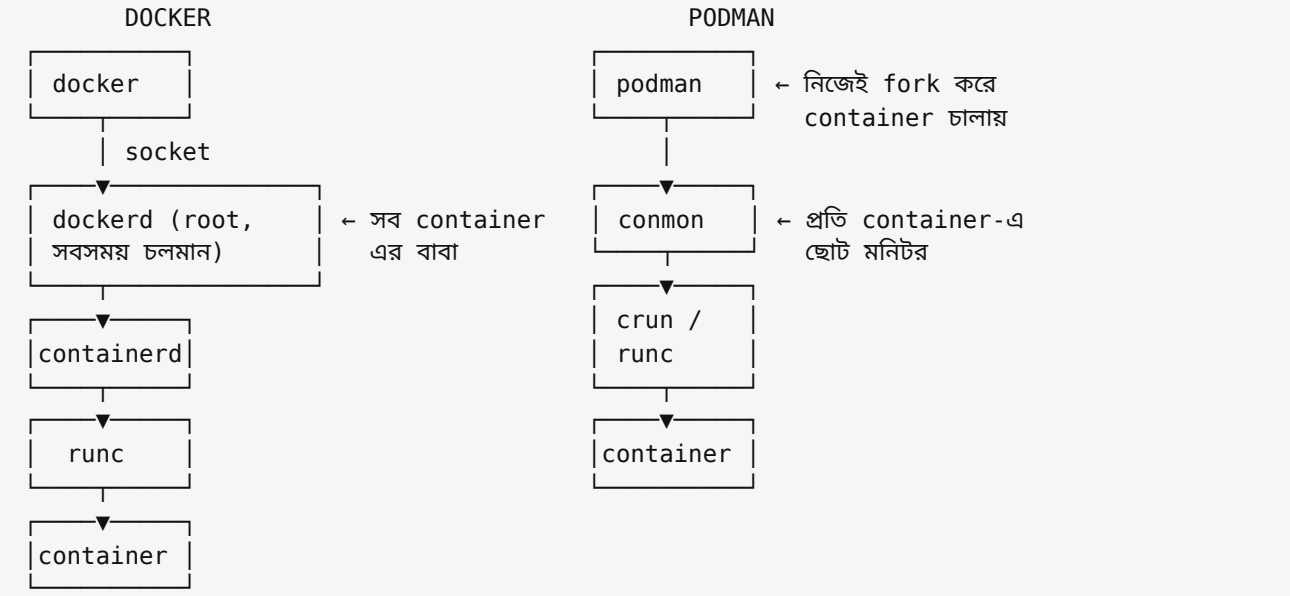
কমান্ড	কাজ
`docker compose up -d`	সব সার্ভিস ব্যাকগ্রাউন্ডে চালাও (দরকার হলে build/pull)
`docker compose up -d --build`	আগে image আবার build করো
`docker compose down`	container ও নেটওয়ার্ক মোছো (volume থাকে)
`docker compose down -v`	⚠️ volume-ও মোছো
`docker compose ps`	অবস্থা
`docker compose logs -f web`	একটা সার্ভিসের লগ ফলো
`docker compose exec db psql -U app appdb`	চলমান সার্ভিসে কমান্ড
`docker compose run --rm web python manage.py migrate`	এককালীন কাজের জন্য নতুন container
`docker compose stop` / `start` / `restart`	থামাও / চালাও / রিস্টার্ট
`docker compose pull`	নতুন image নামাও

কমান্ড	কাজ
`docker compose build`	শুধু build
`docker compose config`	চূড়ান্ত (variable বসানো) কনফিগ দেখাও — ভুল ধরতে দারুণ
`docker compose up -d --scale web=3`	একটা সার্ভিসের ৩টা কপি (তখন হোস্ট port ফিক্সড রাখা যায় না)

> 💡 **টিপস:** একই ফোল্ডারে `.env` ফাইল রাখলে compose সেখান থেকে variable পড়ে, আর YAML-এ `${POSTGRES_PASSWORD}` লিখে ব্যবহার করা যায়। পাসওয়ার্ড তখন YAML-এ লিখতে হয় না (`.env`-কে git-এ commit কোরো না)।

৪৮.৯ Podman — daemon ছাড়া container

Podman (Pod Manager) Red Hat-এর তৈরি, Docker-এর প্রায় হুবহু বিকল্প। RHEL 8/9 ও Fedora-তে এটাই ডিফল্ট container টুল।



বৈশিষ্ট্য	Docker	Podman
Daemon	`dockerd` সবসময় root হিসেবে চলে	কোনো daemon নেই
ডিফল্ট মোড	rootful	**rootless** (সাধারণ ইউজার হিসেবেই চলে)
Single point of failure	daemon মরলে ঝামেলা	নেই
Pod সাপোর্ট	নেই	আছে (Kubernetes-এর মতো)
systemd ইন্টিগ্রেশন	সীমিত	চমৎকার (Quadlet)

বৈশিষ্ট্য	Docker	Podman
CLI	`docker ...`	প্রায় হুবহু একই: `podman ...`
ডিফল্ট OCI runtime	`runc`	Fedora/RHEL-এ `crun` (C-তে লেখা, দ্রুত)

```
# ইনস্টল
$ sudo apt install podman        # Ubuntu/Debian
$ sudo dnf install podman        # Fedora/RHEL

# ব্যবহার — docker-এর মতোই, কিন্তু sudo ছাড়া!
$ podman run -d -p 8080:80 --name web docker.io/library/nginx:1.27
$ podman ps
CONTAINER ID  IMAGE                           COMMAND               CREATED
STATUS        PORTS                   NAMES
a3f1c2d4e5b6  docker.io/library/nginx:1.27    nginx -g daemon o...  5 seconds ago  Up 5
seconds  0.0.0.0:8080->80/tcp  web
$ podman logs web
$ podman exec -it web sh
$ podman stop web && podman rm web
```

> 💡 **টিপস:** পুরো image নাম (`docker.io/library/nginx`) লেখা ভালো অভ্যাস। ছোট নাম (`nginx`) দিলে Podman `/etc/containers/registries.conf`-এর তালিকা থেকে registry বাছাইয়ের জন্য প্রশ্ন করতে পারে।

> 💡 **টিপস:** অভ্যাস না বদলাতে চাইলে `alias docker=podman` — বেশিরভাগ কমান্ড একইভাবে চলবে। Compose ফাইলের জন্য আছে `podman compose` (ভেতরে `podman-compose` বা `docker-compose` টুল ব্যবহার করে)।

Podman-এর বাড়তি টুলভাই: **Buildah** (image build), **Skopeo** (registry-তে image কপি/inspect, image নামানো ছাড়াই)।

Pods

একটা **pod** হলো এমন কয়েকটা container-এর দল যারা একই network namespace শেয়ার করে (একে অপরকে `localhost` দিয়ে পায়) — ঠিক Kubernetes-এর pod-এর মতো।

```
$ podman pod create --name mypod -p 8080:80
$ podman run -d --pod mypod --name web docker.io/library/nginx:1.27
$ podman run -d --pod mypod --name sidecar docker.io/library/alpine sleep infinity
$ podman pod ps
$ podman generate kube mypod > mypod.yaml     # Kubernetes YAML বানাও!
$ podman kube play mypod.yaml                 # YAML থেকে চালাও
```

Quadlet — container-কে systemd সার্ভিস বানাও

পুরনো `podman generate systemd` এখন deprecated; নতুন পথ **Quadlet** (Podman 4.4+)। একটা `.container` ফাইল লেখো, systemd নিজেই সার্ভিস বানিয়ে নেবে:

`~/.config/containers/systemd/web.container` (rootless; সিস্টেম-ব্যাপী হলে `/etc/containers/systemd/`):

```
[Unit]
Description=My Nginx container

[Container]
Image=docker.io/library/nginx:1.27
PublishPort=8080:80
Volume=%h/site:/usr/share/nginx/html:ro,Z

[Service]
Restart=always

[Install]
WantedBy=default.target
```

```
$ systemctl --user daemon-reload
$ systemctl --user start web.service
$ systemctl --user status web.service
$ loginctl enable-linger $USER     # লগআউটের পরও ইউজার সার্ভিস চলুক
```

৪৮.১০ Rootless container — root ছাড়া container

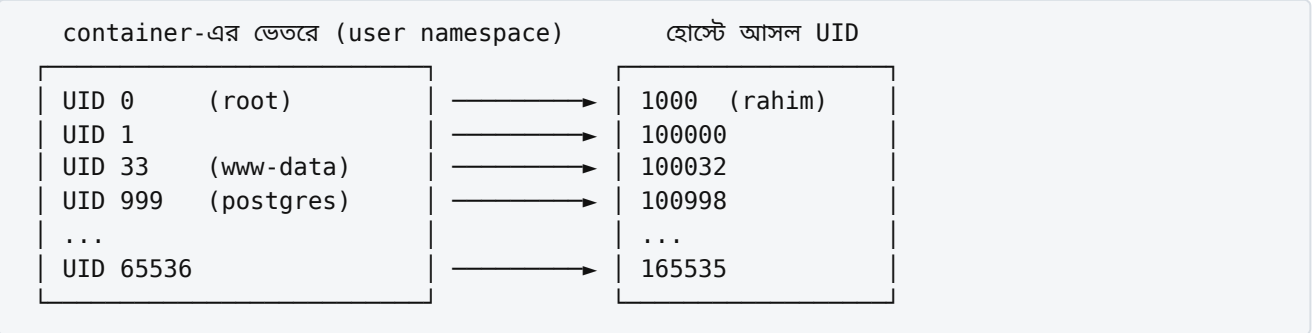
Rootless container মানে container runtime আর container — দুটোই সাধারণ ইউজার হিসেবে চলে, কোথাও আসল root লাগে না। কেউ container ভেঙে বেরিয়ে এলেও সে শুধু একজন সাধারণ ইউজার — পুরো সিস্টেম নয়।

কীভাবে কাজ করে — user namespace + subordinate ID

container-এর ভেতরে অনেক ইউজার লাগে (root=0, `www-data`=33, `postgres`=999...)। কিন্তু তুমি তো একজন মাত্র ইউজার (UID 1000)! সমাধান: সিস্টেম তোমাকে আরও ৬৫,৫৩৬টা "অধীনস্থ" (subordinate) UID বরাদ্দ দেয়:

```
$ cat /etc/subuid
rahim:100000:65536
$ cat /etc/subgid
rahim:100000:65536
```

মানে rahim ১০০০০০ থেকে ১৬৫৫৩৫ পর্যন্ত UID ব্যবহার করতে পারবে। (নতুন ইউজার `useradd` দিয়ে বানালে সাধারণত এটা স্বয়ংক্রিয় হয়; না থাকলে `sudo usermod --add-subuids 100000-165535 --add-subgids 100000-165535 rahim`।)




```
$ podman unshare cat /proc/self/uid_map
         0       1000          1
         1     100000      65536
```

(প্রথম সংখ্যা = ভেতরের UID, দ্বিতীয় = বাইরের UID, তৃতীয় = কতগুলো।) `newuidmap` / `newgidmap` নামের ছোট setuid প্রোগ্রাম (`uidmap` প্যাকেজ) এই ম্যাপিং লেখে।

```
$ podman run -d --name sleeper docker.io/library/alpine sleep 1000
$ podman exec sleeper id
uid=0(root) gid=0(root) groups=0(root),...
$ ps -o user,pid,cmd -C sleep
USER         PID CMD
rahim       8812 sleep 1000      # হোস্ট থেকে দেখলে — সাধারণ ইউজার!
```

Rootless-এর সীমাবদ্ধতা

সমস্যা	কারণ	সমাধান
১০২৪-এর নিচের port (৮০, ৪৪৩) bind করা যায় না	ওগুলো privileged port	`-p 8080:80` ব্যবহার করো, বা `sudo sysctl net.ipv4.ip_unprivileged_port_start=80`
নেটওয়ার্ক একটু ধীর	আসল bridge/veth বানাতে root লাগে, তাই user-space নেটওয়ার্ক স্ট্যাক ব্যবহার হয়	Podman 5+-এ ডিফল্ট `pasta` (আগে `slirp4netns`) যথেষ্ট দ্রুত
bind mount-এ ফাইলের মালিক অদ্ভুত (যেমন 100032)	UID ম্যাপিং	`podman unshare chown`, বা `--userns=keep-id`
cgroup লিমিট কাজ না করা	cgroups v2 + systemd delegation লাগে	আধুনিক distro-তে সাধারণত ঠিক থাকে
image ইউজারভেদে আলাদা	প্রতিটা ইউজারের নিজস্ব স্টোরেজ	rootless: `~/.local/share/containers/storage`, rootful: `/var/lib/containers/storage`

Docker rootless মোড: Docker-ও rootless চালানো যায়:

```
$ sudo apt install uidmap docker-ce-rootless-extras
$ dockerd-rootless-setuptool.sh install
$ systemctl --user enable --now docker
$ export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
$ docker run -d -p 8080:80 nginx
```

৪৮.১১ Kubernetes এক নজরে

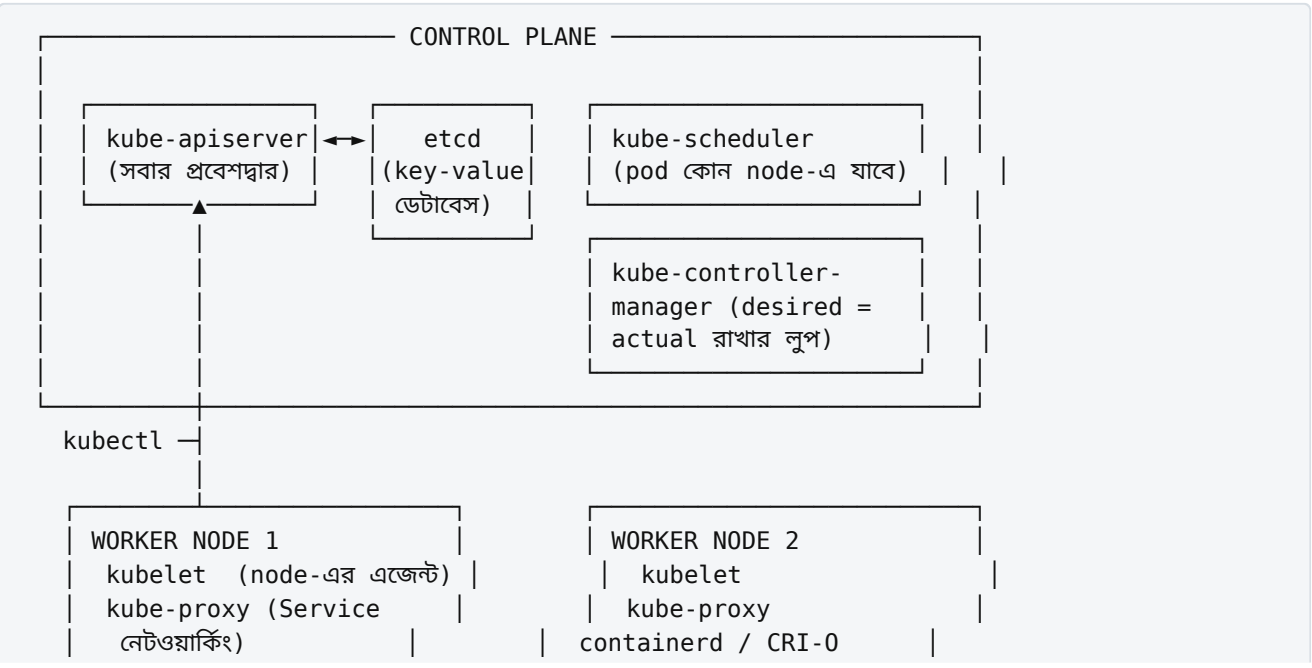
একটা সার্ভারে ১০টা container সামলানো সহজ। কিন্তু ৫০টা সার্ভারে ৫,০০০ container? কোনটা কোন সার্ভারে চলবে, কোনটা crash করলে কে আবার চালাবে, ট্রাফিক কীভাবে ভাগ হবে, নতুন ভার্সন কীভাবে downtime ছাড়া deploy হবে? এই কাজের নাম **container orchestration**, আর তার রাজা **Kubernetes** (সংক্ষেপে **K8s** — K আর s-এর মাঝে ৮টা অক্ষর)। Google-এর ভেতরের "Borg" সিস্টেমের অভিজ্ঞতা থেকে তৈরি, এখন CNCF-এর অধীনে।

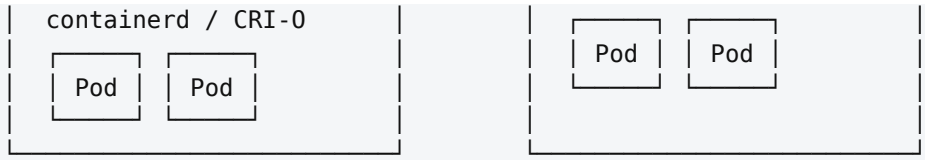
উপমা: Docker হলো একজন জাহাজের নাবিক যে কন্টেইনার লোড করে। Kubernetes হলো পুরো **বন্দরের কন্ট্রোল টাওয়ার** — কোন জাহাজে কোন কন্টেইনার যাবে, কোন জাহাজ ডুবলে মাল কোথায় সরবে, সব সে ঠিক করে। (গ্রিক ভাষায় Kubernetes মানেই "জাহাজের কাণ্ডারি"।)

মূল ধারণা: Declarative

তুমি Kubernetes-কে বলো **"আমি কী চাই"** (desired state) — যেমন "nginx-এর ৩টা কপি সবসময় চলুক"। Kubernetes ক্রমাগত দেখে আসল অবস্থা (actual state) কী, আর পার্থক্য থাকলে ঠিক করে (reconciliation loop)। একটা crash করলে নিজেই নতুন একটা চালায় — একে বলে **self-healing**।

আর্কিটেকচার





উপাদান	কাজ
kube-apiserver	সব যোগাযোগের কেন্দ্র; `kubectl` এর সাথেই কথা বলে
etcd	পুরো ক্লাস্টারের অবস্থা রাখা distributed key-value ডেটাবেস
kube-scheduler	নতুন pod-কে কোন node-এ বসাবে তা ঠিক করে (রিসোর্স দেখে)
kube-controller-manager	Deployment, ReplicaSet, Node ইত্যাদির controller লুপ চালায়
kubelet	প্রতিটা node-এ এজেন্ট — container runtime-কে বলে pod চালাতে
kube-proxy	Service-এর জন্য নেটওয়ার্ক rule (iptables/IPVS/nftables) বসায়
Container runtime	containerd বা CRI-O (CRI ইন্টারফেস দিয়ে) — Kubernetes 1.24 থেকে dockershim সরানো হয়েছে, কিন্তু Docker-এ বানানো image ঠিকই চলে

মূল অবজেক্টগুলো

অবজেক্ট	কী
Pod	সবচেয়ে ছোট একক — এক বা একাধিক container, একই IP ও নেটওয়ার্ক শেয়ার করে
ReplicaSet	নির্দিষ্ট সংখ্যক একই pod চালু রাখে
Deployment	ReplicaSet-কে সামলায়; rolling update ও rollback
Service	pod-গুলোর সামনে একটা স্থায়ী IP/DNS নাম ও load balancer (pod-এর IP বদলায়, Service-এর না)
Ingress	বাইরের HTTP/HTTPS ট্রাফিককে ডোমেইন/পাথ অনুযায়ী Service-এ পাঠায়
ConfigMap / Secret	কনফিগ ও গোপন তথ্য
PersistentVolume(Claim)	স্থায়ী স্টোরেজ
Namespace	ক্লাস্টারের ভেতরে লজিক্যাল ভাগ (dev, prod) — লিনাক্স kernel namespace-এর সাথে গুলিয়ে ফেলো না!
StatefulSet / DaemonSet / Job / CronJob	ডেটাবেস / প্রতিটা node-এ একটা করে / এককালীন কাজ / শিডিউলড কাজ

একটা Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
```

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            limits:
              memory: "128Mi"
              cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: NodePort
```

(`cpu: "500m"` = ৫০০ millicore = আধা CPU — ভেতরে সেই cgroup `cpu.max` !)

শেখার জন্য লোকাল ক্লাস্টার

টুল	বৈশিষ্ট্য
minikube	এক-node ক্লাস্টার, VM বা container-এ; শুরু করার জন্য সবচেয়ে সহজ
kind (Kubernetes IN Docker)	Docker container-কে node বানায়; CI-তে জনপ্রিয়
k3s	হালকা, প্রোডাকশন-যোগ্য; Raspberry Pi/edge-এ চলে
MicroK8s	Canonical-এর, `snap` দিয়ে Ubuntu-তে

`kubectl` — প্রাথমিক কমান্ড

```
$ minikube start
$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   1m    v1.31.0

$ kubectl apply -f web.yaml
deployment.apps/web created
```

```
service/web created

$ kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
web-7d9c6b5f8d-4xkqz   1/1     Running   0          20s
web-7d9c6b5f8d-9hjtl   1/1     Running   0          20s
web-7d9c6b5f8d-mzp2r   1/1     Running   0          20s

$ kubectl delete pod web-7d9c6b5f8d-4xkqz      # একটা মেরে দেখো...
$ kubectl get pods                              # ...নতুন একটা নিজে চলে এসেছে!
```

কমান্ড	কাজ
`kubectl get pods/deploy/svc/nodes`	তালিকা (`-o wide` বিস্তারিত, `-A` সব namespace)
`kubectl describe pod NAME`	বিস্তারিত + events (ডিবাগিংয়ের প্রথম ধাপ)
`kubectl logs -f POD`	লগ
`kubectl exec -it POD -- sh`	pod-এর ভেতরে shell
`kubectl apply -f file.yaml`	YAML প্রয়োগ (তৈরি বা আপডেট)
`kubectl delete -f file.yaml`	মুছে ফেলো
`kubectl create deployment web --image=nginx:1.27 --replicas=3`	YAML ছাড়া দ্রুত Deployment
`kubectl expose deployment web --port=80 --type=NodePort`	Service বানাও
`kubectl scale deployment web --replicas=5`	কপি বাড়াও/কমাও
`kubectl set image deployment/web nginx=nginx:1.28`	নতুন image — rolling update
`kubectl rollout status deployment/web` / `rollout undo deployment/web`	আপডেটের অবস্থা / আগের ভার্সনে ফেরো
`kubectl port-forward svc/web 8080:80`	লোকাল port থেকে Service-এ সুড়ঙ্গ

> 💡 **টিপস:** Kubernetes নিজেই একটা পুরো বই-সমান বিষয়। এই অধ্যায় শেষে লিনাক্স, নেটওয়ার্কিং আর Docker-এ পাকা হলে তবেই Kubernetes-এ ঝাঁপ দাও — তখন বুঝবে এর প্রতিটা অংশ আসলে এই বইয়ের কোনো না কোনো অধ্যায়ের ধারণার উপর দাঁড়িয়ে।

৪৮.১২ আরও কিছু প্রযুক্তি — নাম জেনে রাখো

প্রযুক্তি	কী
LXC / Incus (LXD)	"system container" — পুরো একটা OS (systemd সহ) container-এ চালায়, অনেকটা হালকা VM-এর মতো। Incus হলো LXD-এর কমিউনিটি fork।
systemd-nspawn	systemd-এর নিজস্ব হালকা container টুল; `machinectl` দিয়ে সামলানো

প্রযুক্তি	কী
Firecracker	AWS-এর "microVM" — VM-এর নিরাপত্তা, container-এর গতি (AWS Lambda এটা ব্যবহার করে)
gVisor / Kata Containers	container-কে বাড়তি sandbox/হালকা VM-এ মুড়ে শক্ত isolation
Vagrant	কোড দিয়ে VM বানানো ও সামলানোর টুল
Proxmox VE	KVM + LXC-ভিত্তিক ওপেন-সোর্স virtualization প্ল্যাটফর্ম (Debian-ভিত্তিক)
Flatpak / Snap	ডেস্কটপ অ্যাপ প্যাকেজিং — ভেতরে namespace আর sandbox-এর একই কৌশল (অধ্যায় ৩০)

৪৮.১৩ কীবোর্ড শর্টকাট

শর্টকাট	কোথায়	কাজ
`Ctrl+P` তারপর `Ctrl+Q`	`docker run -it` / `docker attach` / `podman attach`	container না থামিয়ে **detach** করো (বেরিয়ে আসো)
`Ctrl+C`	`docker run` (foreground, `-it` সহ)	মূল process-এ SIGINT — সাধারণত container থেমে যায়
`Ctrl+D` / `exit`	container-এর ভেতরের shell	shell থেকে বের হও (`docker exec` -এর shell হলে container চলতেই থাকে; `docker run -it` -এর মূল shell হলে container থেমে যায়)
`Ctrl+C`	`docker logs -f` , `docker stats` , `kubectl logs -f`	ফলো করা বন্ধ (container থামে না)
`Ctrl+]`	`virsh console`	VM-এর সিরিয়াল কনসোল থেকে বের হও
`Ctrl+A` তারপর `X`	`qemu-system-* -nographic`	QEMU বন্ধ করে বের হও
`Ctrl+A` তারপর `C`	`qemu -nographic`	QEMU monitor ↔ সিরিয়াল কনসোল টগল
`Ctrl+A` তারপর `H`	`qemu -nographic`	এই escape শর্টকাটগুলোর সাহায্য
`Ctrl+Alt+G`	QEMU GTK উইন্ডো	মাউস/কীবোর্ড grab ছাড়ো
`Ctrl+Alt+F`	QEMU GTK উইন্ডো	ফুলস্ক্রিন টগল
`Ctrl+Alt+2` / `Ctrl+Alt+1`	QEMU GTK উইন্ডো	QEMU monitor / গেস্ট ডিসপ্লে
বাম `Ctrl+Alt`	virt-manager / virt-viewer কনসোল	মাউস/কীবোর্ড grab ছাড়ো
ডান `Ctrl` (Host key)	VirtualBox	মাউস/কীবোর্ড grab ছাড়ো

শর্টকাট	কোথায়	কাজ
`Ctrl+L`	কোনো shell-এ	স্ক্রিন পরিষ্কার (অধ্যায় ৬)

> 💡 **টিপস:** detach key বদলাতে চাইলে: `docker run -it --detach-keys="ctrl-x" ...`, অথবা `~/.docker/config.json`-এ `"detachKeys": "ctrl-x"`। (`Ctrl+P` bash-এর হিস্ট্রি শর্টকাটের সাথে সংঘর্ষ করে বলে অনেকে বদলায়।)

হাতে-কলমে অনুশীলন

ল্যাব ১: Docker ছাড়া "container" বানাও (namespace + cgroup)

```
# ১. একটা Alpine root ফাইলসিস্টেম নামাও (মাত্র ~৩ MB)
$ mkdir -p ~/mycontainer && cd ~/mycontainer
$ curl -LO https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpine-
minirootfs-3.20.3-x86_64.tar.gz
$ mkdir rootfs && sudo tar -xzf alpine-minirootfs-3.20.3-x86_64.tar.gz -C rootfs

# ২. cgroup বানাও: ৫০ MB RAM, ২০টা process
$ sudo mkdir /sys/fs/cgroup/mybox
$ echo 50M | sudo tee /sys/fs/cgroup/mybox/memory.max
$ echo 20  | sudo tee /sys/fs/cgroup/mybox/pids.max

# ৩. shell-কে cgroup-এ ঢুকিয়ে, নতুন namespace-এ, alpine root-এ চালাও
$ sudo bash -c 'echo $$ > /sys/fs/cgroup/mybox/cgroup.procs &&
    exec unshare --uts --pid --net --mount --ipc --fork \
    chroot rootfs /bin/sh -c "mount -t proc proc /proc && hostname mybox && exec
/bin/sh"'

# ৪. ভেতরে পরীক্ষা করো
/ # hostname
mybox
/ # ps
PID   USER     TIME  COMMAND
    1 root      0:00 /bin/sh
    2 root      0:00 ps
/ # cat /etc/os-release | head -1
NAME="Alpine Linux"
/ # ip addr        # শুধু lo
/ # exit

# ৫. পরিষ্কার
$ sudo rmdir /sys/fs/cgroup/mybox
```

(Alpine-এর ভার্সন নম্বর বদলে থাকলে https://dl-cdn.alpinelinux.org/alpine/ থেকে সর্বশেষ minirootfs ফাইলের নাম দেখে নাও।) লক্ষ করো: তুমি হোস্টে চালিয়েও ভেতরে পেলে আলাদা OS (Alpine), আলাদা hostname, PID 1, নেটওয়ার্ক নেই, আর RAM/process সীমিত — এটাই container!

ল্যাব ২: Docker দিয়ে একটা স্ট্যাটিক ওয়েবসাইট

1. `mkdir ~/site && echo '<h1>আমার প্রথম Docker সাইট</h1>' > ~/site/index.html`
2. `docker run -d -p 8080:80 -v ~/site:/usr/share/nginx/html:ro --name web nginx:1.27`
3. `curl localhost:8080` — তোমার লেখা দেখাবে।
4. `index.html` এডিট করো, আবার `curl` করো — সাথে সাথে বদল (bind mount!)।
5. `docker logs web` দিয়ে access log দেখো।
6. `docker exec -it web sh` → ভেতরে `cat /etc/os-release`, `ps` (না থাকলে `ls /proc`) দেখো, `exit`।
7. হোস্ট থেকে `ps aux | grep nginx` — container-এর nginx হোস্টেও একটা process হিসেবে দেখা যাচ্ছে!
8. `docker inspect -f '{{.State.Pid}}' web` দিয়ে PID বের করে `sudo ls -l /proc/<PID>/ns` — নিজের shell-এর সাথে namespace নম্বর তুলনা করো।
9. `cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' web).scope/memory.current` — container-এর RAM ব্যবহার সরাসরি cgroup থেকে! (systemd cgroup driver-এ এই পাথ।)
10. `docker stop web && docker rm web`।

ল্যাব ৩: নিজের image বানাও ও compose দিয়ে চালাও

1. ৪৮.৭-এর Flask অ্যাপ (`app.py`, `requirements.txt`, `Dockerfile`, `.dockerignore`) বানাও।
2. `docker build -t myapp:1.0 .` — সময়টা খেয়াল করো।
3. `app.py`-এর বার্তা বদলে আবার build করো — দেখো `pip install` ধাপ `CACHED` দেখাচ্ছে।
4. `docker image history myapp:1.0` দিয়ে স্তরগুলো দেখো।
5. ৪৮.৮-এর `compose.yaml` বানিয়ে `docker compose up -d`।
6. `docker compose ps`, `docker compose logs -f web`।
7. `docker compose exec db psql -U app appdb -c '\l'`।
8. `docker compose down`, তারপর আবার `up -d` — ডেটাবেস ডেটা volume-এ টিকে আছে কিনা দেখো। শেষে `docker compose down -v`।

ল্যাব ৪: Podman rootless

1. `podman run -d --name sleeper docker.io/library/alpine sleep 1000`
2. `podman exec sleeper id` — ভেতরে root।
3. `ps -o user,pid,cmd -C sleep` — বাইরে তুমি।
4. `podman unshare cat /proc/self/uid_map` — ম্যাপিং দেখো।
5. `podman run -d -p 80:80 docker.io/library/nginx` চেষ্টা করো — কী error আসে? কেন? এবার `-p 8080:80` দিয়ে চালাও।
6. `podman rm -f -a` দিয়ে সব পরিষ্কার।

ল্যাব ৫: KVM VM (ঐচ্ছিক, হার্ডওয়্যার সাপোর্ট লাগবে)

1. `grep -Ec '(vmx|svm)' /proc/cpuinfo` ও `kvm-ok` দিয়ে যাচাই।
2. libvirt ইনস্টল করে নিজেকে `libvirt` গ্রুপে যোগ করো।
3. একটা ছোট ISO (যেমন Alpine "virtual") দিয়ে `virt-install` চালাও।
4. `virsh list --all`, `virsh dominfo`, `virsh domifaddr` চালাও।
5. `virsh snapshot-create-as VM clean` → VM-এ কিছু ফাইল মুছে → `virsh snapshot-revert VM clean`।
6. হোস্টে `ps aux | grep qemu` — VM আসলে একটা process!

সারসংক্ষেপ

বিষয়	মূল কমান্ড / ধারণা
VM বনাম Container	VM = আলাদা kernel (hypervisor); Container = শেয়ার্ড kernel (namespace + cgroup)
Hypervisor	Type 1 (ESXi, Xen, Hyper-V, KVM) / Type 2 (VirtualBox, VMware Workstation)
KVM সাপোর্ট	`grep -Ec '(vmx|svm)' /proc/cpuinfo`, `kvm-ok`, `lsmod | grep kvm`
QEMU	`qemu-img create -f qcow2 d.qcow2 20G`, `qemu-system-x86_64 -enable-kvm ...`
libvirt	`virt-install`, `virsh list --all/start/shutdown/destroy/console/snapshot-create-as/undefine`
chroot	`sudo chroot DIR /bin/bash`
Namespaces	`unshare --uts --pid --net --mount --ipc --user --fork --mount-proc`, `lsns`, `nsenter`, `ip netns`
cgroups v2	`/sys/fs/cgroup/`, `memory.max`, `cpu.max`, `pids.max`, `cgroup.procs`, `systemd-cgls`, `systemd-cgtop`, `systemd-run -p`
OverlayFS	`mount -t overlay overlay -o lowerdir=,upperdir=,workdir= MERGED`
Docker image	`pull`, `images`, `rmi`, `build -t`, `tag`, `push`, `image history`, `save/load`
Docker container	`run -d -p -v --name -e --rm -it`, `ps -a`, `logs -f`, `exec -it`, `stop`, `start`, `rm -f`, `inspect`, `stats`, `cp`
Docker volume/network	`volume create/ls/rm`, `network create`, `--network`
পরিষ্কার	`docker system df`, `docker system prune`
Dockerfile	`FROM RUN COPY WORKDIR ENV ARG EXPOSE USER CMD ENTRYPOINT`; multi-stage `COPY --from=`
Compose	`docker compose up -d / ps / logs -f / exec / down / down -v / config`

বিষয়	মূল কমান্ড / ধারণা
Podman	`podman run/ps/exec/pod create/generate kube/kube play`, Quadlet `.container`
Rootless	`/etc/subuid`, `/etc/subgid`, `podman unshare`, `dockerd-rootless-setuptool.sh install`
Kubernetes	`kubectl get/describe/logs/exec/apply/delete/scale/rollout`, minikube/kind/k3s
শর্টকাট	`Ctrl+P Ctrl+Q` detach, `Ctrl+]` virsh console থেকে বের, `Ctrl+A X` QEMU থেকে বের, `Ctrl+Alt+G` মাউস ছাড়ো

নিজেকে যাচাই করো

1. VM আর container-এর সবচেয়ে মৌলিক পার্থক্য কী? লিনাক্স হোস্টে কেন Windows container চালানো যায় না, অথচ Windows VM চালানো যায়?
2. Type 1 আর Type 2 hypervisor-এর পার্থক্য কী? KVM, QEMU আর libvirt — প্রত্যেকে কী দায়িত্ব পালন করে?
3. `virsh destroy` আর `virsh undefine --remove-all-storage`-এর মধ্যে পার্থক্য কী?
4. `sudo unshare --pid bash` চালালে `ps` কেন এখনও হোস্টের সব process দেখায়? ঠিক করতে কোন দুটো অপশন লাগবে, আর কেন?
5. cgroup-এ `cpu.max` ফাইলে `25000 100000` লিখলে কী হবে?
6. OverlayFS-এ নিচের স্তরের একটা ফাইল container মুছে ফেললে আসলে কী ঘটে?
7. `docker run -d -p 8080:80 -v data:/data --name app myimg` — প্রতিটা অংশ ব্যাখ্যা করো। `data` কি bind mount না named volume?
8. Dockerfile-এ `COPY requirements.txt .` আর `RUN pip install` কেন `COPY . .`-এর আগে রাখা হয়?
9. `CMD python app.py` না লিখে `CMD ["python", "app.py"]` লেখা কেন ভালো?
10. Rootless container-এ ভেতরের root (UID 0) হোস্টে কে? এটা কোন kernel ফিচার আর কোন ফাইলের উপর নির্ভর করে? আর কেন `docker` গ্রুপে ইউজার যোগ করা নিরাপত্তা-ঝুঁকি?

▼ উত্তর দেখো

1. VM-এর নিজস্ব পূর্ণ OS ও kernel থাকে, hypervisor হার্ডওয়্যার ভাগ করে দেয়; container হোস্টের kernel শেয়ার করে শুধু namespace/cgroup দিয়ে আলাদা করা process। Windows container-এর জন্য Windows kernel লাগে, যা লিনাক্স হোস্টে নেই; কিন্তু VM নিজের kernel নিজে বুট করে, তাই VM-এ Windows চলে।

2. Type 1 সরাসরি হার্ডওয়্যারে চলে (ESXi, Xen, Hyper-V, KVM); Type 2 একটা সাধারণ OS-এর উপর অ্যাপ হিসেবে চলে (VirtualBox, VMware Workstation)। KVM = kernel মডিউল, CPU/মেমরি virtualization (VT-x/AMD-V দিয়ে); QEMU = user-space-এ ডিস্ক, নেটওয়ার্ক কার্ড ইত্যাদি

ডিভাইস emulate করে; libvirt = সব VM সামলানোর API/ডেমন, যার ক্লায়েন্ট `virsh` / `virt-manager`।

3. `virsh destroy` শুধু VM-কে জোর করে বন্ধ করে (পাওয়ার কাটার মতো) — VM ও ডিস্ক থেকে যায়, আবার `virsh start` করা যায়। `virsh undefine --remove-all-storage` VM-এর সংজ্ঞা ও ডিস্ক ফাইল স্থায়ীভাবে মুছে দেয়।

4. (ক) `unshare` নিজে নতুন PID namespace-এ ঢোকে না, শুধু তার child-রা — তাই `--fork` লাগে। (খ) `ps` তথ্য পড়ে `/proc` থেকে, যা এখনও হোস্টের PID namespace-এর; তাই নতুন mount namespace-এ নতুন `/proc` মাউন্ট করতে `--mount-proc` লাগে।

5. প্রতি ১০০,০০০ µs (১০০ ms) সময়ে ওই cgroup-এর process-রা মিলে সর্বোচ্চ ২৫,০০০ µs (২৫ ms) CPU সময় পাবে — অর্থাৎ একটা CPU-র ২৫%।

6. নিচের (read-only) স্তর বদলায় না; upperdir-এ ওই নামে একটা **whiteout** ফাইল (character device 0/0) তৈরি হয়, যা merged দৃশ্যে ফাইলটাকে লুকিয়ে রাখে।

7. `-d` ব্যাকগ্রাউন্ডে; `-p 8080:80` হোস্টের 8080 → container-এর 80; `-v data:/data` — `data` `/` বা `./` দিয়ে শুরু হয়নি, তাই এটা **named volume**, container-এর `/data`-তে মাউন্ট; `--name app` container-এর নাম; `myimg` image।

8. Layer cache-এর জন্য। dependency তালিকা কম বদলায়; কোড বেশি বদলায়। আগে `requirements.txt` কপি করে install করলে, শুধু কোড বদলালে install স্তর ক্যাশ থেকে আসে — build অনেক দ্রুত হয়।

9. JSON (exec) form-এ `python` সরাসরি PID 1 হয় এবং `docker stop`-এর SIGTERM পায়, ভদ্রভাবে বন্ধ হতে পারে। shell form-এ `/bin/sh -c` PID 1 হয়, signal প্রায়ই অ্যাপে পৌঁছায় না, ফলে ১০ সেকেন্ড পর SIGKILL দিয়ে জোর করে মারা হয়।

10. ভেতরের UID 0 হোস্টে তোমার নিজের সাধারণ UID (যেমন 1000); বাকি UID-গুলো `/etc/subuid` / `/etc/subgid`-এ বরাদ্দ subordinate রেঞ্জে (যেমন 100000+) ম্যাপ হয়। এটা **user namespace**-এর উপর নির্ভর করে (`newuidmap` / `newgidmap` ম্যাপিং লেখে)। আর `docker` গ্রুপের সদস্য root হিসেবে চলা `dockerd`-কে যেকোনো আদেশ দিতে পারে, যেমন `docker run -v /:/host ...` দিয়ে হোস্টের পুরো ফাইলসিস্টেম root হিসেবে মাউন্ট করা — তাই কার্যত root-এর সমান।

পরিশিষ্ট খ: কমান্ড রেফারেন্স A-Z, শেখার রোডম্যাপ ও ইন্টারভিউ প্রস্তুতি

> এই পরিশিষ্টটা বইয়ের "ম্যাপ"। পুরো বই পড়ার পর কোনো কমান্ড ভুলে গেলে এখান থেকে খুঁজে নেবে, কোন অধ্যায়ে সেটা বিস্তারিত আছে সেখানে লাফ দেবে। নতুন শুরু করলে এখানকার ৯০ দিনের রোডম্যাপ ধরে এগোবে। চাকরির ইন্টারভিউর আগে এখানকার প্রশ্নোত্তরগুলো ঝালাই করে নেবে।

এই অধ্যায়ে যা শিখবে

- বইয়ে আসা প্রায় ২৫০টি কমান্ডের বর্ণানুক্রমিক (A-Z) রেফারেন্স টেবিল: কমান্ড, এক লাইনে কাজ, আর কোন অধ্যায়ে বিস্তারিত আছে (লিংকসহ)
- কাজ অনুযায়ী (task-based) কমান্ড খোঁজার "উল্টো সূচি"
- অচেনা কমান্ড নিজে নিজে শেখার পদ্ধতি: `man`, `--help`, `tldr`, `apropos`, `type`
- ৯০ দিনের সপ্তাহভিত্তিক শেখার রোডম্যাপ, প্রতি সপ্তাহের লক্ষ্য আর "পাস করার পরীক্ষা"
- ফ্রেশার থেকে এক্সপার্ট পর্যন্ত দক্ষতার ৬টি স্তর, আর প্রতিটি স্তরে কী জানা উচিত
- সার্টিফিকেশন: LFCS, RHCSA, RHCE, LPIC (১/২/৩), CompTIA Linux+ — কোনটা কার জন্য
- অনুশীলনের জায়গা: OverTheWire Bandit, home lab, অনলাইন স্যান্ডবক্স
- ৫৮টি লিনাক্স ইন্টারভিউ প্রশ্ন ও উত্তর (বেসিক → অ্যাডভান্সড → বাস্তব পরিস্থিতি)
- দরকারি বই, ওয়েবসাইট আর টুলের তালিকা

খ.১ এই পরিশিষ্ট কীভাবে ব্যবহার করবে

একটা অভিধান (dictionary) যেমন পুরোটা মুখস্থ করার জিনিস না, দরকারের সময় খোলার জিনিস — এই পরিশিষ্টও তাই। তিনভাবে ব্যবহার করতে পারো:

তুমি কে	কীভাবে ব্যবহার করবে
একদম নতুন	আগে নিচের **খ.৪ — ৯০ দিনের রোডম্যাপ** দেখো, সেই ক্রমে অধ্যায় পড়ো
বই পড়া শেষ, এখন কাজ করছ	**খ.২ — A-Z টেবিল** থেকে কমান্ড খুঁজে অধ্যায়ের লিংকে যাও
ইন্টারভিউ বা সার্টিফিকেশন সামনে	**খ.৬ (সার্টিফিকেশন)** আর **খ.৮ (ইন্টারভিউ)** পড়ো, তারপর "নিজেকে যাচাই করো"

> 💡 **টিপস:** Markdown viewer-এ (GitHub, VS Code preview, Obsidian) `Ctrl+F` চেপে কমান্ডের নাম লিখলেই টেবিলে সরাসরি পৌঁছে যাবে।

টেবিলের চিহ্নগুলো

চিহ্ন	মানে
🅓	Debian/Ubuntu পরিবারের নিজস্ব কমান্ড
🅡	RHEL/Fedora পরিবারের নিজস্ব কমান্ড
🧱	শেলের নিজের ভেতরের কমান্ড (builtin) — আলাদা প্রোগ্রাম নয়
⚠️	ভুল ব্যবহারে ডেটা হারাতে বা সিস্টেম নষ্ট হতে পারে
🕰️	পুরোনো (legacy) — নতুন সিস্টেমে বিকল্প আছে, কিন্তু জানা দরকার

খ.২ কমান্ড রেফারেন্স A-Z

> 📝 **মনে রাখো:** একটা কমান্ড কয়েকটা অধ্যায়ে আসতে পারে। টেবিলে মূল (যেখানে সবচেয়ে বিস্তারিত) অধ্যায়টা দেওয়া আছে।

A

কমান্ড	এক লাইনে কাজ	অধ্যায়
`alias` 🧱	লম্বা কমান্ডের ছোট ডাকনাম বানায় (`alias ll='ls -alF'`)	৪৪
`apropos`	কীওয়ার্ড দিয়ে man page খোঁজে (`man -k` -এর সমান)	৫, ১২
`apt` 🅓	প্যাকেজ ইনস্টল/আপডেট/রিমুভ করার ব্যবহারকারী-বান্ধব টুল	৩০
`apt-cache` 🅓	প্যাকেজের তথ্য, নির্ভরতা (dependency) খোঁজে	৩০
`apt-get` 🅓	`apt` -এর পুরোনো, স্ক্রিপ্টের জন্য স্থিতিশীল সংস্করণ	৩০
`at`	নির্দিষ্ট সময়ে একবার একটা কাজ চালায়	২৯
`atq` / `atrm`	`at` -এর অপেক্ষমাণ কাজের তালিকা দেখায় / মুছে ফেলে	২৯
`awk`	কলাম-ভিত্তিক টেক্সট প্রসেসিংয়ের পুরো একটা ভাষা	২২

B

কমান্ড	এক লাইনে কাজ	অধ্যায়
`basename`	পাথ থেকে শুধু ফাইলের নাম বের করে	৪৩
`bash`	Bourne Again Shell — লিনাক্সের সবচেয়ে প্রচলিত শেল	৫
`bg` 🧱	থামানো (stopped) জবকে ব্যাকগ্রাউন্ডে চালু করে	২৫
`blkid`	ব্লক ডিভাইসের UUID, ফাইলসিস্টেম টাইপ দেখায়	৩২

কমান্ড	এক লাইনে কাজ	অধ্যায়
`bunzip2` / `bzip2`	`.bz2` ফরম্যাটে কম্প্রেস/ডিকম্প্রেস করে	৩৪

C

কমান্ড	এক লাইনে কাজ	অধ্যায়
`cat`	ফাইলের পুরো লেখা দেখায় / ফাইল জোড়া লাগায় (concatenate)	১০
`cd` 🧱	ডিরেক্টরি বদলায় (`cd -` আগের জায়গায় ফেরত)	৮
`chage`	পাসওয়ার্ডের মেয়াদ (aging) দেখে ও বদলায়	১৫
`chattr`	ফাইলের বিশেষ অ্যাট্রিবিউট বসায় (যেমন `+i` = immutable)	১৭
`chgrp`	ফাইলের গ্রুপ-মালিক বদলায়	১৬
`chmod`	ফাইলের পারমিশন বদলায় (`chmod 755` , `chmod u+x`)	১৬
`chown`	ফাইলের মালিক (ও গ্রুপ) বদলায়	১৬
`chsh`	ইউজারের লগইন শেল বদলায়	১৫
`clear`	টার্মিনাল স্ক্রিন পরিষ্কার করে (শর্টকাট `Ctrl+L`)	৬
`cmake`	আধুনিক বিল্ড-সিস্টেম জেনারেটর	৩১
`column`	টেক্সটকে সুন্দর কলামে সাজায় (`column -t`)	২৩
`comm`	দুটো সাজানো (sorted) ফাইলের মিল-অমিল লাইন দেখায়	২৩
`cp`	ফাইল/ডিরেক্টরি কপি করে (`-r` , `-a` , `-i`)	৯
`crontab`	ইউজারের নিয়মিত (পুনরাবৃত্ত) কাজের তালিকা এডিট করে	২৯
`curl`	URL থেকে ডেটা আনে/পাঠায়, API টেস্টের প্রধান হাতিয়ার	৩৭
`cut`	প্রতিটি লাইনের নির্দিষ্ট কলাম/ক্যারেক্টার কেটে নেয়	২৩

D

কমান্ড	এক লাইনে কাজ	অধ্যায়
`date`	তারিখ-সময় দেখায়/ফরম্যাট করে	৫
`dd` ⚠️	ব্লক-লেভেলে কপি করে (ISO থেকে USB, ডিস্ক ক্লোন)	৩৫
`declare` 🧱	ভেরিয়েবলের ধরন বলে দেয় (`-a` অ্যারে, `-A` associative, `-i` integer)	৪২
`df`	ফাইলসিস্টেমে কতটা জায়গা খালি আছে দেখায় (`df -h`)	৩২
`diff`	দুটো ফাইলের পার্থক্য দেখায়	২৩
`dig`	DNS কোয়েরি করে বিস্তারিত উত্তর দেখায়	৩৭
`dirname`	পাথ থেকে ডিরেক্টরির অংশটা বের করে	৪৩
`dmesg`	কার্নেলের রিং-বাফার মেসেজ দেখায় (হার্ডওয়্যার, ড্রাইভার)	২৮, ৪৭

কমান্ড	এক লাইনে কাজ	অধ্যায়
`dnf` 🅁	Fedora/RHEL-এর প্যাকেজ ম্যানেজার (`yum` -এর উত্তরসূরি)	৩০
`docker`	কন্টেইনার বানানো, চালানো, ম্যানেজ করা	৪৮
`dpkg` 🅓	`.deb` প্যাকেজের নিচু-স্তরের (low-level) ম্যানেজার	৩০
`du`	ফাইল/ডিরেক্টরি কতটা জায়গা খাচ্ছে দেখায় (`du -sh *`)	৩২

E

কমান্ড	এক লাইনে কাজ	অধ্যায়
`echo` 🧱	লেখা বা ভেরিয়েবলের মান প্রিন্ট করে	৫
`egrep` 🕰️	`grep -E` -এর পুরোনো নাম (এখন ব্যবহার করলে সতর্কবার্তা দেয়)	২০
`env`	এনভায়রনমেন্ট ভেরিয়েবল দেখায় / পরিবর্তিত পরিবেশে কমান্ড চালায়	৪৪
`exit` 🧱	শেল বা স্ক্রিপ্ট থেকে বের হয় (exit code সহ)	৪১
`export` 🧱	ভেরিয়েবলকে child process-এর জন্য দৃশ্যমান করে	৪৪

F

কমান্ড	এক লাইনে কাজ	অধ্যায়
`fail2ban-client`	বারবার ভুল লগইন করা IP ব্লক করার সার্ভিস নিয়ন্ত্রণ করে	৪৫
`fdisk` ⚠️	ডিস্কে পার্টিশন বানায়/মোছে (MBR ও GPT দুটোই)	৩২
`fg` 🧱	ব্যাকগ্রাউন্ডের জবকে সামনে (foreground) আনে	২৫
`file`	ফাইলের আসল ধরন বলে (এক্সটেনশন না দেখে, ভেতরের "magic" দেখে)	১০
`find`	নাম, সাইজ, সময়, মালিক — যেকোনো শর্তে ফাইল খোঁজে	১২
`firewall-cmd` 🅁	firewalld-এর জোন, সার্ভিস, পোর্ট ম্যানেজ করে	৩৯
`flatpak`	distro-নিরপেক্ষ স্যান্ডবক্সড অ্যাপ ইনস্টল করে	৩০
`free`	RAM ও swap-এর ব্যবহার দেখায় (`free -h`)	৪৬
`fsck` ⚠️	ফাইলসিস্টেম পরীক্ষা ও মেরামত করে (unmount অবস্থায়)	৩২
`fuser`	কোন প্রসেস একটা ফাইল/পোর্ট ধরে রেখেছে দেখায়	৪৬

G

কমান্ড	এক লাইনে কাজ	অধ্যায়
`gcc`	C কম্পাইলার	৩১
`gdisk` ⚠️	GPT পার্টিশন টেবিল এডিটর	৩২

কমান্ড	এক লাইনে কাজ	অধ্যায়
`getenforce` / `setenforce` 🅁	SELinux-এর মোড দেখায় / সাময়িক বদলায়	৪৫
`getent`	`/etc/passwd`, `/etc/group`, `/etc/hosts` ইত্যাদি ডেটাবেস থেকে এন্ট্রি আনে	১৫
`getfacl` / `setfacl`	ফাইলের ACL দেখায় / বসায়	১৭
`getopts` 🧱	স্ক্রিপ্টে `-a -b value` ধরনের অপশন পার্স করে	৪৩
`grep`	প্যাটার্ন মেলে এমন লাইন খোঁজে (`-i`, `-r`, `-v`, `-n`, `-E`)	২০
`groupadd` / `groupdel` / `groupmod`	গ্রুপ বানায় / মোছে / বদলায়	১৫
`groups`	ইউজার কোন কোন গ্রুপে আছে দেখায়	১৫
`gunzip` / `gzip`	`.gz` ফরম্যাটে কম্প্রেস/ডিকম্প্রেস করে	৩৪

H

কমান্ড	এক লাইনে কাজ	অধ্যায়
`head`	ফাইলের প্রথম কয়েক লাইন দেখায় (ডিফল্ট ১০)	১০
`help` 🧱	bash builtin কমান্ডের সাহায্য দেখায় (`help cd`)	৫
`history` 🧱	আগে চালানো কমান্ডের তালিকা	৬
`host`	সহজ DNS লুকআপ	৩৭
`hostname`	মেশিনের নাম দেখায় (`hostname -I` = সব IP)	৩৬
`hostnamectl`	systemd দিয়ে স্থায়ীভাবে hostname দেখায়/বসায়	৩৬
`htop`	রঙিন, ইন্টারঅ্যাক্টিভ প্রসেস মনিটর	২৪

I

কমান্ড	এক লাইনে কাজ	অধ্যায়
`id`	ইউজারের UID, GID ও গ্রুপগুলো দেখায়	১৫
`ifconfig` 🕰️	নেটওয়ার্ক ইন্টারফেস দেখায় — এখন `ip` ব্যবহার করো	৩৭
`info`	GNU-র বিস্তারিত ডকুমেন্টেশন পড়ার ভিউয়ার	৫
`insmod` / `rmmod`	কার্নেল মডিউল সরাসরি লোড/আনলোড করে (নির্ভরতা দেখে না)	৪৭
`iostat`	ডিস্কের I/O পরিসংখ্যান দেখায় (`sysstat` প্যাকেজ)	৪৬
`iotop`	কোন প্রসেস সবচেয়ে বেশি ডিস্ক I/O করছে দেখায়	৪৬
`ip`	IP ঠিকানা, রাউট, লিংক — আধুনিক নেটওয়ার্ক টুল (`ip a`, `ip r`)	৩৭
`iptables` 🕰️	নেটফিল্টার ফায়ারওয়াল রুল (আধুনিক distro-তে nftables-এর ওপর চলে)	৩৯

J – K

কমান্ড	এক লাইনে কাজ	অধ্যায়
`jobs` 🧱	বর্তমান শেলের জবগুলোর তালিকা	২৫
`journalctl`	systemd journal-এর লগ পড়ে (`-u` , `-f` , `-b` , `--since`)	২৮
`kill`	প্রসেসকে সিগন্যাল পাঠায় (ডিফল্ট SIGTERM)	২৪
`killall`	নাম দিয়ে সব মিলে যাওয়া প্রসেসে সিগন্যাল পাঠায়	২৪

L

কমান্ড	এক লাইনে কাজ	অধ্যায়
`last`	সাম্প্রতিক লগইন/রিবুটের ইতিহাস	২৮
`lastb`	ব্যর্থ লগইনের ইতিহাস (root লাগে)	২৮
`ldd`	একটা প্রোগ্রাম কোন shared library-র ওপর নির্ভর করে দেখায়	৩১
`less`	বড় ফাইল পাতা-পাতা করে পড়ে (`/` দিয়ে খোঁজা, `q` দিয়ে বের হওয়া)	১০
`ln`	হার্ড লিংক বানায়; `ln -s` দিয়ে সফট (symbolic) লিংক	১১
`locate`	আগে বানানো ডেটাবেস থেকে খুব দ্রুত ফাইলের নাম খোঁজে	১২
`logger`	স্ক্রিপ্ট থেকে সিস্টেম লগে মেসেজ লেখে	২৮
`logrotate`	লগ ফাইল ঘুরিয়ে (rotate), কম্প্রেস করে, পুরোনো মুছে দেয়	২৮
`ls`	ডিরেক্টরির ভেতরের ফাইল দেখায় (`-l` , `-a` , `-h` , `-t` , `-R`)	৮
`lsattr`	`chattr` দিয়ে বসানো অ্যাট্রিবিউট দেখায়	১৭
`lsblk`	ডিস্ক ও পার্টিশনের গাছ (tree) দেখায়	৩২
`lscpu`	CPU-র বিস্তারিত তথ্য	৪৭
`lsmod`	লোড হওয়া কার্নেল মডিউলের তালিকা	৪৭
`lsof`	কোন প্রসেস কোন ফাইল/সকেট খুলে রেখেছে	৪৬
`lspci` / `lsusb`	PCI / USB ডিভাইসের তালিকা	৪৭
`lvcreate` / `lvextend` / `lvs`	LVM-এর logical volume বানায় / বড় করে / দেখায়	৩৩

M

কমান্ড	এক লাইনে কাজ	অধ্যায়
`make`	`Makefile` অনুযায়ী সোর্স কোড বিল্ড করে	৩১
`man`	যেকোনো কমান্ডের ম্যানুয়াল পাতা দেখায়	৫
`mdadm`	সফটওয়্যার RAID বানায় ও ম্যানেজ করে	৩৩

কমান্ড	এক লাইনে কাজ	অধ্যায়
`mkdir`	নতুন ডিরেক্টরি বানায় (`-p` দিয়ে মাঝের সবগুলোও)	৯
`mkfs` ⚠️	পার্টিশনে ফাইলসিস্টেম বানায় (`mkfs.ext4` , `mkfs.xfs`) — ভেতরের ডেটা মুছে যায়	৩২
`mkswap` / `swapon` / `swapoff`	swap এলাকা বানায় / চালু / বন্ধ করে	৩২
`modinfo`	একটা কার্নেল মডিউলের তথ্য দেখায়	৪৭
`modprobe`	নির্ভরতাসহ কার্নেল মডিউল লোড করে (`-r` দিয়ে সরায়)	৪৭
`more` 🕰️	`less` -এর পুরোনো, কম সুবিধার ভাই	১০
`mount` / `umount`	ফাইলসিস্টেমকে ডিরেক্টরি-গাছে জোড়া দেয় / খোলে	৩২
`mtr`	`ping` + `traceroute` একসাথে, লাইভ	৩৭
`mv`	ফাইল সরায় বা নাম বদলায়	৯

N

কমান্ড	এক লাইনে কাজ	অধ্যায়
`nano`	নতুনদের জন্য সহজ টার্মিনাল টেক্সট এডিটর	১৩
`nc` (netcat)	TCP/UDP সংযোগ খোলে — পোর্ট টেস্টের "সুইস নাইফ"	৩৭
`nft`	nftables — আধুনিক লিনাক্স ফায়ারওয়াল ফ্রেমওয়ার্ক	৩৯
`nginx`	ওয়েব সার্ভার (`nginx -t` দিয়ে কনফিগ পরীক্ষা)	৪০
`nice` / `renice`	প্রসেসের CPU অগ্রাধিকার (niceness, −20 থেকে 19) ঠিক করে	২৪
`nl`	লাইনে নম্বর বসিয়ে দেখায়	১০
`nmcli`	NetworkManager-কে কমান্ড লাইন থেকে নিয়ন্ত্রণ করে	৩৭
`nohup`	টার্মিনাল বন্ধ হলেও যেন কমান্ড না মরে (SIGHUP উপেক্ষা)	২৫
`nproc`	কতগুলো CPU core ব্যবহারযোগ্য দেখায়	৩১
`nslookup`	DNS লুকআপ (পুরোনো, কিন্তু সব OS-এ আছে)	৩৭

O – P

কমান্ড	এক লাইনে কাজ	অধ্যায়
`openssl`	সার্টিফিকেট, কী (key), TLS সংযোগ পরীক্ষা	৪০
`passwd`	পাসওয়ার্ড বদলায় / অ্যাকাউন্ট লক (`-l`) করে	১৫
`paste`	কয়েকটা ফাইলের লাইন পাশাপাশি জোড়া দেয়	২৩
`pgrep` / `pkill`	নাম দিয়ে প্রসেসের PID খোঁজে / সিগন্যাল পাঠায়	২৪

কমান্ড	এক লাইনে কাজ	অধ্যায়
`ping`	কোনো হোস্টে পৌঁছানো যায় কিনা (ICMP) পরীক্ষা করে	৩৭
`podman`	daemon ছাড়া, rootless কন্টেইনার চালায় (Docker-এর বিকল্প)	৪৮
`printenv`	এনভায়রনমেন্ট ভেরিয়েবল দেখায়	৪৪
`printf` 🧱	ফরম্যাট করে প্রিন্ট করে (`echo` -র চেয়ে নির্ভরযোগ্য)	৪১
`ps`	চলমান প্রসেসের স্ন্যাপশট (`ps aux` , `ps -ef`)	২৪
`pstree`	প্রসেসগুলোকে বাবা-সন্তান গাছ আকারে দেখায়	২৪
`pvcreate` / `pvs`	LVM-এর physical volume বানায় / দেখায়	৩৩
`pwd`	এখন কোন ডিরেক্টরিতে আছ দেখায়	৮

R

কমান্ড	এক লাইনে কাজ	অধ্যায়
`read` 🧱	ইউজার বা ফাইল থেকে ইনপুট পড়ে ভেরিয়েবলে রাখে	৪১
`realpath` / `readlink -f`	লিংক মিটিয়ে আসল পূর্ণ পাথ দেখায়	১১
`reboot` / `poweroff`	মেশিন রিস্টার্ট / বন্ধ করে (systemd-তে `systemctl reboot` -এর সমান)	২৭
`rm` ⚠️	ফাইল মোছে — কোনো Recycle Bin নেই! (`-r` , `-f` , `-i`)	৯
`rmdir`	খালি ডিরেক্টরি মোছে	৯
`rpm` 🅡	`.rpm` প্যাকেজের নিচু-স্তরের ম্যানেজার (`rpm -qa` , `rpm -qf`)	৩০
`rsync`	শুধু পরিবর্তিত অংশ কপি করে — ব্যাকআপ ও সিঙ্কের রাজা	৩৫

S

কমান্ড	এক লাইনে কাজ	অধ্যায়
`sar`	সময়ের সাথে CPU/মেমরি/I/O-র ইতিহাস (`sysstat`)	৪৬
`scp`	SSH দিয়ে ফাইল কপি করে	৩৮
`screen`	পুরোনো টার্মিনাল মাল্টিপ্লেক্সার (detach/attach)	২৫
`sed`	স্ট্রিম এডিটর — খোঁজো ও বদলাও (`sed 's/a/b/g'`)	২১
`seq`	সংখ্যার ধারা তৈরি করে (`seq 1 10`)	৪২
`set` 🧱	শেলের অপশন বসায় (`set -euo pipefail`)	৪৩
`sftp`	SSH-এর ওপর ইন্টারঅ্যাক্টিভ ফাইল ট্রান্সফার	৩৮
`shutdown`	নির্ধারিত সময়ে মেশিন বন্ধ/রিবুট (`shutdown -r +5`)	২৭
`sleep`	নির্দিষ্ট সময় অপেক্ষা করে	৪২
`snap` 🅓	Canonical-এর স্যান্ডবক্সড প্যাকেজ ম্যানেজার	৩০

কমান্ড	এক লাইনে কাজ	অধ্যায়
`sort`	লাইন সাজায় (`-n` সংখ্যা, `-r` উল্টো, `-k` কলাম, `-h` মানুষ-পাঠযোগ্য সাইজ)	২৩
`source` / `.` 🧱	ফাইলের কমান্ড বর্তমান শেলেই চালায়	৪৪
`ss`	সকেট/পোর্ট দেখায় (`ss -tulpn`) — `netstat` -এর আধুনিক বিকল্প	৩৭
`ssh`	নিরাপদে রিমোট মেশিনে লগইন করে	৩৮
`ssh-copy-id`	তোমার পাবলিক কী রিমোট সার্ভারে বসিয়ে দেয়	৩৮
`ssh-keygen`	SSH কী-জোড়া (key pair) বানায়	৩৮
`stat`	ফাইলের inode, সাইজ, তিন ধরনের সময়, পারমিশন দেখায়	১১
`strace`	প্রসেস কোন কোন system call করছে দেখায়	৪৬
`su`	অন্য ইউজার (ডিফল্ট root) হিসেবে শেল খোলে	১৮
`sudo`	অন্য ইউজার (সাধারণত root) হিসেবে একটা কমান্ড চালায়	১৮
`sysctl`	চলমান কার্নেলের প্যারামিটার দেখে/বদলায়	৪৭
`systemctl`	systemd সার্ভিস চালু/বন্ধ/enable/status	২৬
`systemd-analyze`	বুটে কোন ধাপে কত সময় লাগল দেখায় (`blame`)	২৭

T

কমান্ড	এক লাইনে কাজ	অধ্যায়
`tail`	ফাইলের শেষ কয়েক লাইন দেখায়; `-f` দিয়ে লাইভ অনুসরণ	১০
`tar`	অনেক ফাইলকে এক আর্কাইভে ভরে/খোলে (`-czvf` , `-xzvf`)	৩৪
`tcpdump`	নেটওয়ার্ক প্যাকেট ধরে দেখায়	৩৭
`tee`	ইনপুট একই সাথে স্ক্রিনে আর ফাইলে লেখে	১৯
`test` / `[ ]` / `[[ ]]` 🧱	শর্ত পরীক্ষা করে (ফাইল আছে কি, সংখ্যা বড় কি)	৪১
`timedatectl`	সিস্টেমের সময়, টাইমজোন, NTP সিঙ্ক দেখায়/বসায়	২৯
`tldr`	কমান্ডের সংক্ষিপ্ত, উদাহরণভিত্তিক সাহায্য (আলাদা ইনস্টল লাগে)	৫
`tmux`	আধুনিক টার্মিনাল মাল্টিপ্লেক্সার — সেশন, উইন্ডো, প্যান	২৫
`top`	লাইভ প্রসেস ও রিসোর্স মনিটর	২৪
`touch`	খালি ফাইল বানায় / টাইমস্ট্যাম্প হালনাগাদ করে	৯
`tr`	ক্যারেক্টার বদলায় বা মোছে (`tr 'a-z' 'A-Z'`)	২৩
`traceroute` / `tracepath`	প্যাকেট কোন কোন রাউটার হয়ে যাচ্ছে দেখায়	৩৭
`trap` 🧱	সিগন্যাল বা স্ক্রিপ্টের শেষে কী হবে নির্ধারণ করে	৪৩
`tree`	ডিরেক্টরির গাছ ছবির মতো দেখায় (আলাদা ইনস্টল লাগতে পারে)	৮
`type` 🧱	একটা নাম alias, builtin, function নাকি ফাইল — বলে দেয়	১২

U

কমান্ড	এক লাইনে কাজ	অধ্যায়
`ufw` 🅳	Uncomplicated Firewall — Ubuntu-র সহজ ফায়ারওয়াল	৩৯
`ulimit` 🧱	প্রসেসের রিসোর্স সীমা (খোলা ফাইল, প্রসেস সংখ্যা) দেখায়/বসায়	৪৬
`umask` 🧱	নতুন ফাইলের ডিফল্ট পারমিশন থেকে কোন বিট বাদ যাবে ঠিক করে	১৬
`uname`	কার্নেলের নাম, ভার্সন, আর্কিটেকচার (`uname -r`, `-a`)	৪৭
`uniq`	পাশাপাশি পুনরাবৃত্ত লাইন বাদ দেয়/গোনে (আগে `sort` লাগে)	২৩
`unshare`	নতুন namespace-এ প্রসেস চালায় (কন্টেইনারের ভিত্তি)	৪৮
`unzip` / `zip`	`.zip` আর্কাইভ খোলে / বানায়	৩৪
`update-grub` 🅳	GRUB কনফিগ নতুন করে তৈরি করে (RHEL-এ `grub2-mkconfig`)	২৭
`uptime`	কতক্ষণ চালু, কতজন ইউজার, load average	৪৬
`useradd` / `adduser`	নতুন ইউজার বানায় (`adduser` Debian-এ ইন্টারঅ্যাক্টিভ)	১৫
`userdel`	ইউজার মোছে (`-r` দিয়ে home সহ)	১৫
`usermod`	ইউজার বদলায় (`-aG` দিয়ে গ্রুপে যোগ)	১৫

V

কমান্ড	এক লাইনে কাজ	অধ্যায়
`vgcreate` / `vgextend` / `vgs`	LVM-এর volume group বানায় / বড় করে / দেখায়	৩৩
`vi` / `vim`	শক্তিশালী মোড-ভিত্তিক টেক্সট এডিটর	১৪
`virsh`	KVM/libvirt ভার্চুয়াল মেশিন ম্যানেজ করে	৪৮
`visudo`	sudoers ফাইল নিরাপদে (সিনট্যাক্স যাচাইসহ) এডিট করে	১৮
`vmstat`	মেমরি, swap, CPU, I/O-র সংক্ষিপ্ত পরিসংখ্যান	৪৬

W – Z

কমান্ড	এক লাইনে কাজ	অধ্যায়
`w`	কে লগইন আছে আর কী করছে	১৫
`watch`	একটা কমান্ড বারবার চালিয়ে আউটপুট দেখায় (`watch -n 2 df -h`)	২৪
`wc`	লাইন, শব্দ, বাইট গোনে (`wc -l`)	২৩
`wget`	URL থেকে ফাইল ডাউনলোড করে	৩৭
`whatis`	কমান্ডের এক লাইনের বর্ণনা দেখায়	৫
`whereis`	কমান্ডের binary, সোর্স ও man page-এর অবস্থান	১২

কমান্ড	এক লাইনে কাজ	অধ্যায়
`which`	`PATH`-এ কোন প্রোগ্রাম ফাইল চলবে দেখায়	১২
`who` / `whoami`	লগইন থাকা ইউজার / আমি কে	১৫
`xargs`	ইনপুট থেকে আর্গুমেন্ট বানিয়ে অন্য কমান্ড চালায়	২৩
`xz` / `unxz`	খুব বেশি কম্প্রেশনের `.xz` ফরম্যাট	৩৪
`yum` 🅁🕰️	RHEL 7-এর প্যাকেজ ম্যানেজার; RHEL 8+-এ এটা `dnf`-এর লিংক	৩০
`zcat` / `zgrep` / `zless`	`.gz` ফাইল না খুলেই পড়া/খোঁজা	৩৪
`zypper`	openSUSE-র প্যাকেজ ম্যানেজার	৩০

শেলের বিশেষ চিহ্ন (অক্ষর নয়, কিন্তু কমান্ডের মতোই জরুরি)

চিহ্ন	কাজ	অধ্যায়
`>` / `>>`	stdout ফাইলে লেখা (মুছে) / শেষে যোগ	১৯
`<`	ফাইল থেকে stdin	১৯
`2>` / `2>&1` / `&>`	stderr আলাদা ফাইলে / stderr-কে stdout-এ / দুটোই একসাথে	১৯
`|`	এক কমান্ডের আউটপুট আরেকটার ইনপুট (pipe)	১৯
`&&` / `||`	আগেরটা সফল হলে / ব্যর্থ হলে পরেরটা চালাও	৪১
`;`	পরপর চালাও, ফলাফল যা-ই হোক	৫
`&`	কমান্ড ব্যাকগ্রাউন্ডে চালাও	২৫
`$( )`	কমান্ডের আউটপুট বসাও (command substitution)	৪১
`~` / `.` / `..` / `-`	home / এই ডিরেক্টরি / ওপরের ডিরেক্টরি / আগের ডিরেক্টরি (`cd -`)	৮
`*` / `?` / `[ ]` / `{a,b}`	wildcard (globbing) ও brace expansion	৯
`!!` / `!$` / `!n`	আগের পুরো কমান্ড / আগের শেষ আর্গুমেন্ট / হিস্ট্রির n নম্বর	৬

খ.৩ কাজ অনুযায়ী কমান্ড খোঁজা (উল্টো সূচি)

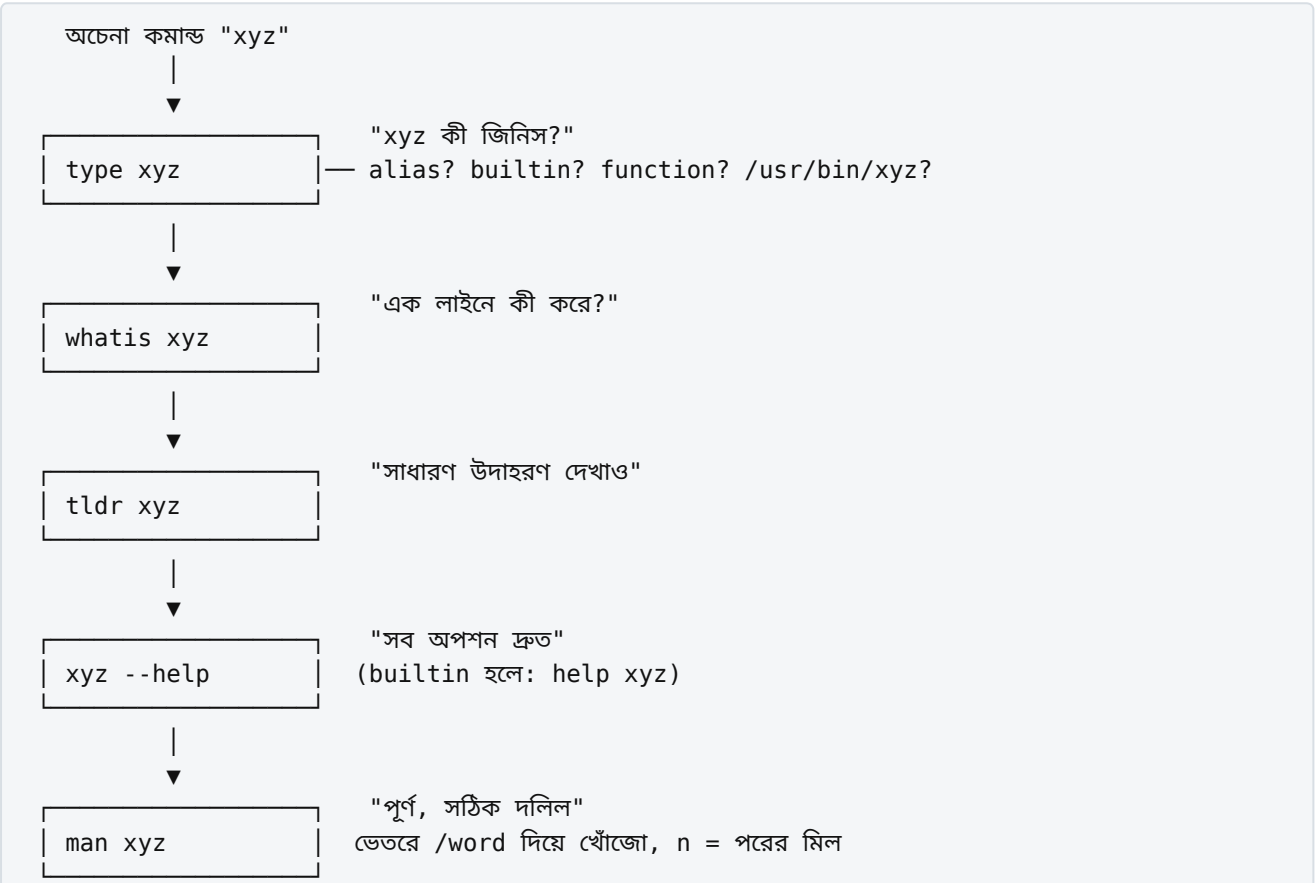
অনেক সময় তুমি কমান্ডের নাম জানো না, শুধু জানো কী করতে চাও। তখন এই টেবিল:

আমি চাই...	কমান্ড
আমি কোথায় আছি জানতে	`pwd`
ফাইলের তালিকা, লুকানো সহ, সাইজ সহ	`ls -lah`
বড় ফাইল আরামে পড়তে	`less file`
লগ লাইভ দেখতে	`tail -f file` বা `journalctl -f`

আমি চাই...	কমান্ড
নাম দিয়ে ফাইল খুঁজতে	`find / -name "*.conf" 2>/dev/null`
ফাইলের ভেতরে লেখা খুঁজতে	`grep -rn "text" /path`
কোন ডিরেক্টরি সবচেয়ে বেশি জায়গা খাচ্ছে	`du -sh * | sort -h`
ডিস্ক কতটা ভরা	`df -h`
RAM কতটা ব্যবহার হচ্ছে	`free -h`
কোন প্রসেস CPU খাচ্ছে	`top` বা `htop`
একটা প্রসেস বন্ধ করতে	`kill PID` → না মরলে `kill -9 PID`
কোন পোর্টে কোন প্রোগ্রাম শুনছে	`sudo ss -tulpn`
আমার IP ঠিকানা	`ip a`
ইন্টারনেট আছে কিনা	`ping -c 4 8.8.8.8`
DNS কাজ করছে কিনা	`dig example.com`
সার্ভিস চালু/বন্ধ/অবস্থা	`systemctl start|stop|status name`
সার্ভিস বুটে চালু রাখতে	`systemctl enable --now name`
সার্ভিসের লগ	`journalctl -u name`
সফটওয়্যার ইনস্টল	`sudo apt install pkg` / `sudo dnf install pkg`
ফোল্ডার কম্প্রেস করতে	`tar -czvf out.tar.gz folder/`
আর্কাইভ খুলতে	`tar -xvf file.tar.gz`
অন্য সার্ভারে লগইন	`ssh user@host`
অন্য সার্ভারে ফাইল পাঠাতে	`rsync -avz src/ user@host:/dst/`
প্রতিদিন রাত ২টায় কাজ চালাতে	`crontab -e` → `0 2 * * * /path/script.sh`
স্ক্রিপ্টকে চালানোর যোগ্য করতে	`chmod +x script.sh`
ইউজারকে sudo দিতে	`sudo usermod -aG sudo user` (RHEL-এ `wheel`)
কার্নেল ভার্সন	`uname -r`
distro ভার্সন	`cat /etc/os-release`
SSH-এ গিয়ে কাজ চালু রেখে বেরিয়ে আসতে	`tmux` → কাজ → `Ctrl+b` তারপর `d`

খ.৩.১ অচেনা কমান্ড নিজে শেখার পদ্ধতি

এক্সপার্ট মানে সব কমান্ড মুখস্থ থাকা মানুষ না — এক্সপার্ট মানে যে **দ্রুত খুঁজে বের করতে পারে**। এই পাঁচ ধাপের অভ্যাসটা তৈরি করো:




```
$ type ls
ls is aliased to `ls --color=auto'
$ type cd
cd is a shell builtin
$ whatis tar
tar (1)              - an archiving utility
$ man -k "copy files"
cp (1)               - copy files and directories
...
```

man page-এর সেকশন নম্বরগুলো মনে রাখো:

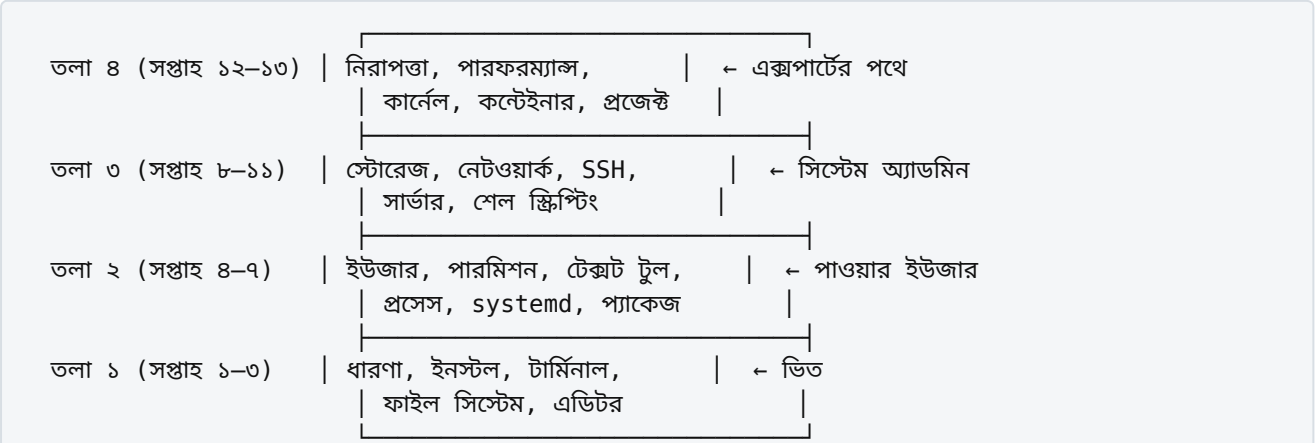
সেকশন	কী থাকে	উদাহরণ
1	সাধারণ ইউজার কমান্ড	`man 1 ls`
2	system call (কার্নেলের ফাংশন)	`man 2 open`
3	C লাইব্রেরি ফাংশন	`man 3 printf`
4	বিশেষ ফাইল (`/dev`)	`man 4 null`
5	কনফিগ ফাইলের ফরম্যাট	`man 5 crontab` , `man 5 fstab`
6	গেম	
7	বিবিধ, কনভেনশন	`man 7 regex` , `man 7 signal`
8	সিস্টেম অ্যাডমিন কমান্ড	`man 8 mount`

> 💡 **টিপস:** `man crontab` দেখায় কমান্ডটা (সেকশন ১), আর `man 5 crontab` দেখায় crontab ফাইলের লেখার নিয়ম। অনেকে দ্বিতীয়টার কথা জানেই না!

খ.৪ ৯০ দিনের শেখার রোডম্যাপ

রোডম্যাপের দর্শন: কেন এই ক্রম?

লিনাক্স শেখা একটা বাড়ি বানানোর মতো। ভিত (architecture, শেল, ফাইল সিস্টেম) না গেঁথে ছাদ (স্ক্রিপ্টিং, সার্ভার, কন্টেইনার) বানাতে গেলে সব ভেঙে পড়বে। তাই রোডম্যাপটা চার তলায় ভাগ করা:



দৈনিক রুটিন (প্রায় ১.৫–২ ঘণ্টা):

সময়	কী করবে
২০ মিনিট	অধ্যায় পড়া (শুধু পড়া না — পাশে টার্মিনাল খোলা রাখা)
৪০ মিনিট	অধ্যায়ের প্রতিটা উদাহরণ নিজে টাইপ করা (কপি-পেস্ট না!)
২০ মিনিট	"হাতে-কলমে অনুশীলন" ল্যাব
১০ মিনিট	নিজের নোটে আজকের ৫টা নতুন কমান্ড লেখা
সপ্তাহে ১ দিন	রিভিশন + OverTheWire Bandit-এর কয়েকটা লেভেল

> ⚠️ **সতর্কতা:** কপি-পেস্ট করে শেখা হয় না। আঙুলে কমান্ড টাইপ করলে "muscle memory" তৈরি হয়, যেটা ইন্টারভিউ বা প্রোডাকশন সার্ভারে আতঙ্কের মুহূর্তে কাজে আসে।

> 💡 **টিপস:** প্রথম দিন থেকেই লিনাক্সকে তোমার **প্রধান** কম্পিউটার বানাও (dual-boot বা অন্তত WSL2/VM)। যে ভাষা রোজ বলা হয়, সেটাই শেখা হয়।

সপ্তাহভিত্তিক পরিকল্পনা

সপ্তাহ	দিন	বিষয়	অধ্যায়	সপ্তাহ শেষের "পাস পরীক্ষা"
১	১–৭	লিনাক্স কী, আর্কিটেকচার, distro বাছাই, ইনস্টল, ডেস্কটপ	১, ২, ৩, ৪	VM-এ Ubuntu বা Fedora ইনস্টল করে আপডেট করতে পারো; kernel, shell, user space আঁকতে পারো
২	৮–১৪	টার্মিনাল, শেল, শর্টকাট, হিস্ট্রি, FHS, চলাফেরা	৫, ৬, ৭, ৮	মাউস ছাড়া ১৫ মিনিট কাজ; `/etc`, `/var`, `/usr`, `/proc` কী জন্য বলতে পারো
৩	১৫–২১	ফাইল ম্যানেজমেন্ট, ফাইল দেখা, inode/লিংক, খোঁজা, nano, Vim	৯–১৪	`vimtutor` শেষ; `find` দিয়ে ৭ দিনের পুরোনো ১MB-র বড় ফাইল খুঁজতে পারো
৪	২২–২৮	ইউজার, গ্রুপ, পারমিশন, বিশেষ পারমিশন, ACL, sudo	১৫–১৮	তিন ইউজারের একটা শেয়ার্ড ফোল্ডার বানাও যেখানে কেউ অন্যের ফাইল মুছতে পারবে না (SGID + sticky)
৫	২৯–৩৫	রিডাইরেকশন, পাইপ, grep, regex, sed	১৯–২১	একটা লগ ফাইল থেকে সব ERROR লাইন বের করে তারিখ বদলে নতুন ফাইলে রাখো — এক লাইনে
৬	৩৬–৪২	awk, cut/sort/uniq/tr/xargs	২২, ২৩	ওয়েব সার্ভার লগ থেকে "সবচেয়ে বেশি হিট করা ১০টা IP" বের করো
৭	৪৩–৪৯	প্রসেস, জব, tmux, systemd, বুট প্রসেস	২৪–২৭	নিজের একটা স্ক্রিপ্টকে systemd সার্ভিস বানাও যা ক্র্যাশ করলে নিজে রিস্টার্ট হয়
৮	৫০–৫৬	লগ, শিডিউলিং, প্যাকেজ ম্যানেজমেন্ট, সোর্স থেকে বিল্ড	২৮–৩১	প্রতি ঘণ্টায় চলা একটা cron জব আর একটা systemd timer — দুটোরই লগ `journalctl`-এ দেখাও
৯	৫৭–৬৩	ডিস্ক, পার্টিশন, মাউন্ট, LVM, RAID, আর্কাইভ, ব্যাকআপ	৩২–৩৫	VM-এ দুটো ভার্চুয়াল ডিস্ক যোগ করে LVM বানাও, `fstab`-এ বসাও, অনলাইনে বড় করো
১০	৬৪–৭০	নেটওয়ার্কিং মূলনীতি, নেটওয়ার্ক কমান্ড, SSH	৩৬–৩৮	দুটো VM-এর মধ্যে পাসওয়ার্ড ছাড়া (key) SSH; root লগইন বন্ধ
১১	৭১–৭৭	ফায়ারওয়াল, ওয়েব সার্ভার, শেল স্ক্রিপ্টিং ১ ও ২	৩৯–৪২	Nginx চালু, শুধু 22/80/443 খোলা; ফাংশন ও লুপসহ একটা মেনু-স্ক্রিপ্ট
১২	৭৮–৮৪	অ্যাডভান্সড স্ক্রিপ্টিং, এনভায়রনমেন্ট, নিরাপত্তা, পারফরম্যান্স	৪৩–৪৬	`set -euo pipefail` সহ একটা ব্যাকআপ স্ক্রিপ্ট; সার্ভার হার্ডেনিং চেকলিস্ট প্রয়োগ
১৩	৮৫–৯০	কার্নেল, `/proc`, `/sys`, কন্টেইনার, ভার্চুয়ালাইজেশন + ফাইনাল প্রজেক্ট	৪৭, ৪৮	নিচের "ফাইনাল প্রজেক্ট" শেষ করো

ফাইনাল প্রজেক্ট (দিন ৮৫–৯০)

শেষ সপ্তাহে একটা ছোট কিন্তু "আসল" সার্ভার নিজে দাঁড় করাও:

1. একটা VM-এ Ubuntu Server 24.04 বা Rocky/AlmaLinux 9 ইনস্টল করো।
2. নতুন ইউজার বানাও, sudo দাও, SSH key দিয়ে লগইন, password লগইন ও root লগইন বন্ধ (অধ্যায় ৩৮)।
3. ফায়ারওয়ালে শুধু 22, 80, 443 খোলা রাখো (অধ্যায় ৩৯)।

4. Nginx দিয়ে একটা স্ট্যাটিক ওয়েবসাইট সার্ভ করো (অধ্যায় ৪০)।
5. ওয়েবসাইটের ফোল্ডার একটা আলাদা LVM ভলিউমে রাখো (অধ্যায় ৩৩)।
6. প্রতি রাতে `rsync` দিয়ে ব্যাকআপ নেওয়ার স্ক্রিপ্ট, systemd timer দিয়ে চালাও (অধ্যায় ৩৫, ২৯)।
7. Nginx-এর access log থেকে প্রতিদিনের রিপোর্ট (টপ IP, টপ পেজ, 404 সংখ্যা) বানানোর awk স্ক্রিপ্ট (অধ্যায় ২২)।
8. একই সাইট একটা Docker/Podman কন্টেইনারেও চালাও (অধ্যায় ৪৮)।
9. পুরো কাজটা একটা `README.md`-তে লিখে GitHub-এ রাখো — এটাই তোমার পোর্টফোলিও।

> 📝 **মনে রাখো:** ৯০ দিন একটা লক্ষ্য, বাধ্যবাধকতা নয়। কোনো সপ্তাহ বেশি কঠিন লাগলে দুই সপ্তাহ নাও। কিন্তু **একদিনও পুরো বাদ দিয়ো না** — ১৫ মিনিট হলেও টার্মিনাল খোলো।

খ.৫ ফ্রেশার থেকে এক্সপার্ট: দক্ষতার ৬টি স্তর

নিজেকে কোন স্তরে আছ যাচাই করো। প্রতিটা স্তরের "আমি পারি" তালিকার সব ক'টা সত্যি হলে পরের স্তরে উঠেছ।

```
স্তর ৫ — এক্সপার্ট / আর্কিটেক্ট        ▲  "কেন" আর "ভেতরে কী হচ্ছে" জানো
স্তর ৪ — সিনিয়র সিসঅ্যাডমিন/DevOps  │
স্তর ৩ — জুনিয়র সিস্টেম অ্যাডমিন     │
স্তর ২ — পাওয়ার ইউজার                  │
স্তর ১ — সাধারণ ব্যবহারকারী             │
স্তর ০ — একদম নতুন (fresher)             │  "কী" টাইপ করতে হয় জানো
```

স্তর	নাম	তুমি যা পারো	মূল অধ্যায়	প্রায় সময়
০	ফ্রেশার	লিনাক্স ইনস্টল, GUI ব্যবহার, টার্মিনাল খোলা, `ls`, `cd`, `pwd`	১–৫	১ সপ্তাহ
১	ব্যবহারকারী	ফাইল বানানো/কপি/মোছা, nano/vim-এ এডিট, প্যাকেজ ইনস্টল, পারমিশন পড়া	৬–১৬, ৩০	১ মাস
২	পাওয়ার ইউজার	পাইপ দিয়ে কমান্ড জোড়া, grep/sed/awk, প্রসেস মারা, tmux, ছোট স্ক্রিপ্ট	১৭–২৫, ৪১, ৪৪	২–৩ মাস
৩	জুনিয়র অ্যাডমিন	systemd সার্ভিস, লগ পড়া, cron, ডিস্ক মাউন্ট, LVM, SSH, ফায়ারওয়াল, ওয়েব সার্ভার	২৬–৪০	৪–৬ মাস
৪	সিনিয়র অ্যাডমিন / DevOps	প্রোডাকশন ট্রাবলশুটিং, হার্ডেনিং, পারফরম্যান্স টিউনিং, অটোমেশন (Bash + Ansible), কন্টেইনার	৪২–৪৮	১–২ বছর
৫	এক্সপার্ট	system call, কার্নেল প্যারামিটার, `strace` / `perf` দিয়ে গভীর ডিবাগ, আর্কিটেকচার ডিজাইন, অন্যকে শেখানো	পুরো বই + বাস্তব অভিজ্ঞতা	৩–৫+ বছর

প্রতিটি স্তরের আত্ম-পরীক্ষা (এক প্রশ্নে):

স্তর	পরীক্ষা
০ → ১	"home ডিরেক্টরিতে `project/src` বানিয়ে তার ভেতরে একটা ফাইল লেখো" — GUI ছাড়া পারো?
১ → ২	"`/var/log`-এ সবচেয়ে বড় ৫টা ফাইল দেখাও" — এক লাইনে পারো?
২ → ৩	"Nginx বুটের পর চালু হচ্ছে না" — কোথা থেকে খোঁজা শুরু করবে জানো?
৩ → ৪	"সার্ভার হঠাৎ ধীর, load average 40" — CPU, মেমরি, I/O নাকি নেটওয়ার্ক — ৫ মিনিটে আলাদা করতে পারো?
৪ → ৫	"`ls` টাইপ করে Enter চাপার পর থেকে আউটপুট আসা পর্যন্ত কার্নেলে কী কী হয়?" — ব্যাখ্যা করতে পারো?

> 💡 **টিপস:** স্তর ৩-এ ওঠা মানেই তুমি জুনিয়র Linux/System Administrator, Cloud Support বা DevOps ইন্টার্নের চাকরির জন্য তৈরি। বাকি স্তর আসে কাজের অভিজ্ঞতা থেকে।

ক্যারিয়ারের পথ

```
                          ┌─► Linux System Administrator ──► Senior SysAdmin
                          │
লিনাক্স ভিত (এই বই) ──┼─► DevOps Engineer ──► SRE (Site Reliability Engineer)
                          │       (+ Git, CI/CD, Ansible, Docker, Kubernetes)
                          │
                          ├─► Cloud Engineer (AWS / Azure / GCP)
                          │
                          ├─► Security Engineer / Pentester
                          │       (+ নেটওয়ার্কিং গভীরে, Kali, CTF)
                          │
                          └─► Embedded / Kernel Developer
                                  (+ C, কম্পিউটার আর্কিটেকচার)
```

খ.৬ সার্টিফিকেশন গাইড

সার্টিফিকেশন কেন (এবং কখন না)?

সার্টিফিকেশন একটা **প্রমাণপত্র** — চাকরিদাতাকে বলে "এই মানুষটা অন্তত এই পরিমাণ জানে বলে যাচাই হয়েছে"। ফ্রেশারদের জন্য, যাদের কাজের অভিজ্ঞতা নেই, এটা CV-তে বড় পার্থক্য তৈরি করে। কিন্তু মনে রাখো:

- সার্টিফিকেট ছাড়া দক্ষতা কাজে লাগে; দক্ষতা ছাড়া সার্টিফিকেট ইন্টারভিউর প্রথম প্রশ্নেই ধরা পড়ে।
- **Performance-based** পরীক্ষা (LFCS, RHCSA, RHCE) — যেখানে আসল লিনাক্স মেশিনে কাজ করে দেখাতে হয় — চাকরির বাজারে বেশি মূল্য পায়, কারণ মুখস্থ করে পাস করা যায় না।
- **Multiple-choice** পরীক্ষা (LPIC, Linux+) — জ্ঞানের ব্যাপ্তি যাচাই করে, তুলনামূলক সহজলভ্য।

তুলনামূলক টেবিল

সার্টিফিকেশন	প্রতিষ্ঠান	পরীক্ষার ধরন	distro	সময়	কার জন্য
LFCS (Linux Foundation Certified System Administrator)	Linux Foundation	হাতে-কলমে (performance-based), অনলাইন প্রক্টরড	Ubuntu	২ ঘণ্টা	distro-নিরপেক্ষ অ্যাডমিন দক্ষতা প্রমাণ করতে চাও
RHCSA (EX200)	Red Hat	হাতে-কলমে	RHEL	প্রায় ২.৫–৩ ঘণ্টা	এন্টারপ্রাইজ/ব্যাংক/টেলকো-তে RHEL অ্যাডমিন চাকরি
RHCE (EX294)	Red Hat	হাতে-কলমে, Ansible দিয়ে অটোমেশন	RHEL	প্রায় ৪ ঘণ্টা	RHCSA-র পরের ধাপ; DevOps/অটোমেশন
LPIC-1 (101 + 102)	LPI	multiple-choice + fill-in-the-blank, দুটো পরীক্ষা	distro-নিরপেক্ষ (Debian ও RPM দুটোই)	প্রতিটা ৯০ মিনিট, ৬০ প্রশ্ন	তাত্ত্বিক ভিত্তি মজবুত করতে, প্রথম সার্টিফিকেট
LPIC-2 (201 + 202)	LPI	multiple-choice	distro-নিরপেক্ষ	প্রতিটা ৯০ মিনিট	উন্নত অ্যাডমিন: কার্নেল, নেটওয়ার্ক সার্ভিস, DNS, ওয়েব, মেইল
LPIC-3 (বিভিন্ন স্পেশালিটি)	LPI	multiple-choice	distro-নিরপেক্ষ	—	Mixed Environments, Security, Virtualization & Containerization, High Availability
CompTIA Linux+	CompTIA	multiple-choice + performance-based প্রশ্ন	distro-নিরপেক্ষ	৯০ মিনিট, সর্বোচ্চ ৯০ প্রশ্ন	যারা CompTIA পথ (A+, Network+, Security+) ধরে এগোচ্ছে

> ⚠️ **সতর্কতা:** পরীক্ষার ফি, সময়, প্রশ্নসংখ্যা, মেয়াদ (validity) আর exam code প্রতিষ্ঠানগুলো নিয়মিত বদলায় (যেমন CompTIA Linux+ XK0-005 থেকে নতুন XK0-006-এ যাচ্ছে, Red Hat পরীক্ষাগুলো RHEL-এর নতুন ভার্সনে হালনাগাদ হয়)। রেজিস্ট্রেশনের আগে **অবশ্যই অফিসিয়াল সাইট** দেখে নেবে: `training.linuxfoundation.org`, `redhat.com/en/services/training`, `lpi.org`, `comptia.org`।

প্রতিটি সার্টিফিকেশন বিস্তারিত

১. LFCS — Linux Foundation-এর পরীক্ষা। তোমাকে একটা টার্মিনাল দেওয়া হবে, আর কাজ দেওয়া হবে: "এই ইউজার বানাও, এই LVM বড় করো, এই সার্ভিস বুটে চালু রাখো"। স্কোর হয় তোমার মেশিনের শেষ অবস্থা দেখে। সিলেবাসের মূল এলাকা: অপারেশনস ডিপ্লয়মেন্ট, নেটওয়ার্কিং, স্টোরেজ, এসেনশিয়াল কমান্ড, ইউজার ও গ্রুপ

ম্যানেজমেন্ট। পরীক্ষার মেশিনে সাধারণত ইনস্টল থাকা `man` পেজ দেখা যায় — তাই খ.৩.১-এর "নিজে খোঁজার পদ্ধতি" এখানে সোনার মতো দামি।

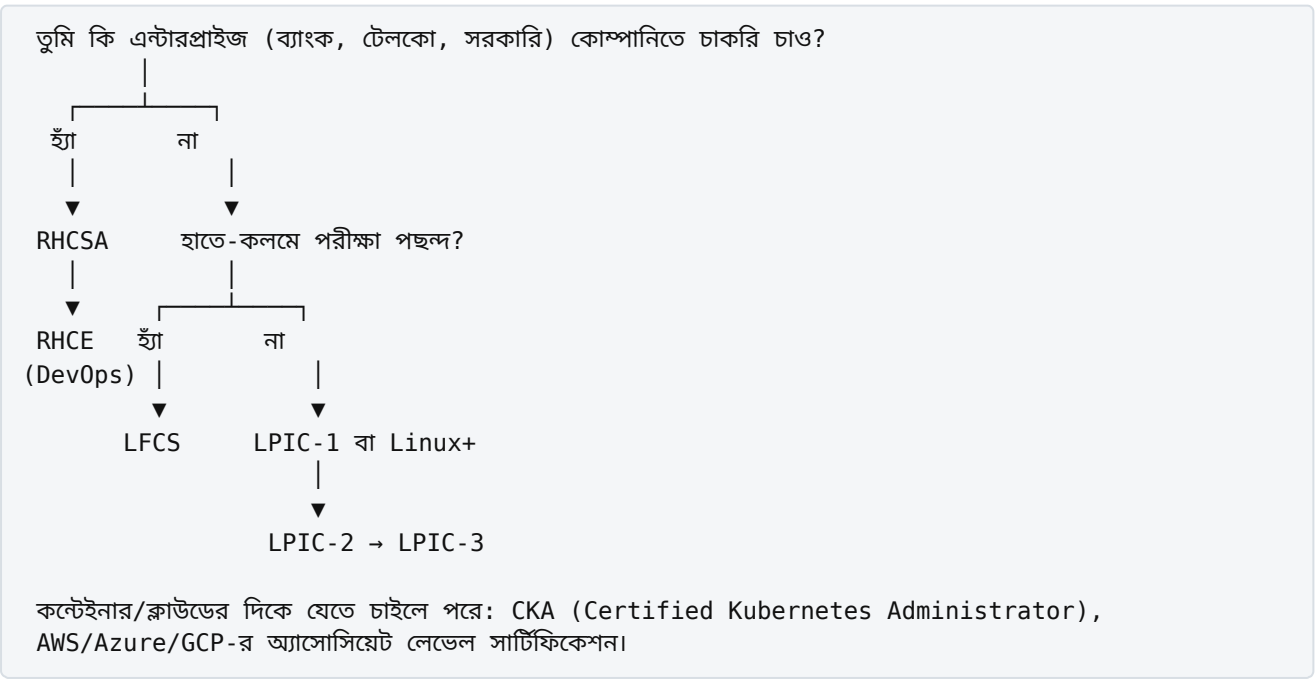
২. RHCSA (EX200) — লিনাক্স চাকরির বাজারে সবচেয়ে পরিচিত নাম। সিলেবাসে থাকে: বুট টার্গেট, root পাসওয়ার্ড রিসেট (বুট লোডার থেকে), LVM, স্টোরেজ, `fstab`, ইউজার/গ্রুপ, পারমিশন, SELinux (context ও boolean), firewalld, `dnf`, systemd, cron/at, কন্টেইনার (`podman`) ইত্যাদি। বইয়ের অধ্যায় ১৫–১৮, ২৪–৩৩, ৩৮, ৩৯, ৪৫, ৪৮ এর প্রায় পুরোটা ঢাকে।

৩. RHCE (EX294) — RHCSA পাস করা থাকতে হয়। আগে এটা ছিল সার্ভিস কনফিগারেশন; এখন (RHEL 8 থেকে) পুরোটাই **Ansible** দিয়ে অনেকগুলো মেশিন স্বয়ংক্রিয়ভাবে কনফিগার করা — playbook, role, variable, template, Ansible Vault।

৪. LPIC-1 — দুটো পরীক্ষা (101-500 ও 102-500) দুটোই পাস করলে সার্টিফিকেট। 101: সিস্টেম আর্কিটেকচার, ইনস্টলেশন ও প্যাকেজ ম্যানেজমেন্ট, GNU/Unix কমান্ড, ডিভাইস ও ফাইলসিস্টেম। 102: শেল ও স্ক্রিপ্টিং, ইউজার ইন্টারফেস, অ্যাডমিন টাস্ক, এসেনশিয়াল সিস্টেম সার্ভিস, নেটওয়ার্কিং, নিরাপত্তা। প্রশ্ন অপশন-স্তরের খুঁটিনাটি জিজ্ঞেস করে ("`tar`-এর কোন অপশন bzip2 ব্যবহার করে?") — উত্তর: `-j`।

৫. CompTIA Linux+ — একটা পরীক্ষা, multiple-choice-এর সাথে কিছু performance-based (সিমুলেশন) প্রশ্ন। সিস্টেম ম্যানেজমেন্ট, নিরাপত্তা, স্ক্রিপ্টিং/কন্টেইনার/অটোমেশন, ট্রাবলশুটিং — চার এলাকা। পাসিং স্কোর ৯০০-র মধ্যে ৭২০।

কোনটা আগে নেবে? — সিদ্ধান্ত গাছ



> 💡 **টিপস:** বাংলাদেশ ও ভারতের চাকরির বিজ্ঞাপনে "RHCSA/RHCE preferred" সবচেয়ে বেশি দেখা যায়। হাতে-কলমে প্র্যাকটিসের জন্য Red Hat-এর বিনামূল্যের **Red Hat Developer Subscription for**

> **Individuals** দিয়ে আসল RHEL ব্যবহার করা যায়, অথবা প্রায়-সমান (binary-compatible) **Rocky Linux** বা **AlmaLinux** দিয়ে অনুশীলন করো।

খ.৭ অনুশীলনের জায়গা

OverTheWire Bandit — খেলতে খেলতে শেখা

Bandit হলো একটা বিনামূল্যের অনলাইন "wargame"। প্রতিটা লেভেলে তুমি SSH দিয়ে একটা সার্ভারে ঢোকো, আর পরের লেভেলের পাসওয়ার্ড কোথাও লুকানো থাকে — কখনো একটা লুকানো ফাইলে, কখনো হাজার লাইনের মধ্যে একমাত্র অনন্য (unique) লাইনে, কখনো একটা নির্দিষ্ট পোর্টে। সেটা বের করতে তোমাকে এই বইয়ের কমান্ডগুলোই ব্যবহার করতে হবে।

শুরু করার নিয়ম (`overthewire.org/wargames/bandit/`):

```
$ ssh bandit0@bandit.labs.overthewire.org -p 2220
bandit0@bandit.labs.overthewire.org's password:      ← এখানে লেখো: bandit0
...
bandit0@bandit:~$ ls
readme
bandit0@bandit:~$ cat readme
```

Bandit-এর লেভেল আর এই বইয়ের অধ্যায়ের মোটামুটি মিল:

Bandit লেভেল	যা শিখবে	বইয়ের অধ্যায়
0–6	`ls -a` , অদ্ভুত নামের ফাইল (`-` , স্পেস), `find` দিয়ে সাইজ/মালিক ধরে খোঁজা	৮, ৯, ১০, ১২
7–12	`grep` , `sort | uniq -u` , `strings` , `base64 -d` , `tr` (ROT13), বারবার কম্প্রেস করা ফাইল খোলা (`xxd -r` , `gzip` , `bzip2` , `tar`)	১৯, ২০, ২৩, ৩৪
13–20	SSH private key, পোর্টে সংযোগ (`nc` , `openssl s_client`), পোর্ট স্ক্যান, `diff` , SUID বাইনারি	১৭, ৩৭, ৩৮
21–24	cron জব পড়ে বোঝা, স্ক্রিপ্ট লেখা, brute-force লুপ	২৯, ৪১–৪৩
25–33	শেল এস্কেপ, `more` / `vi` থেকে শেলে বেরোনো, Git রিপোজিটরি খোঁজা	১০, ১৪

> ⚠️ **সতর্কতা:** অনলাইনে Bandit-এর সব উত্তর পাওয়া যায়। দেখে নিলে খেলাটার পুরো লাভ নষ্ট। একটা লেভেলে ৩০ মিনিট আটকে থাকলে শুধু **ইঙ্গিত** খোঁজো, পুরো সমাধান না। আর প্রতি লেভেলের পাসওয়ার্ড নিজের নোটে লিখে রাখো — না হলে পরের দিন আবার শুরু থেকে আসতে হবে।

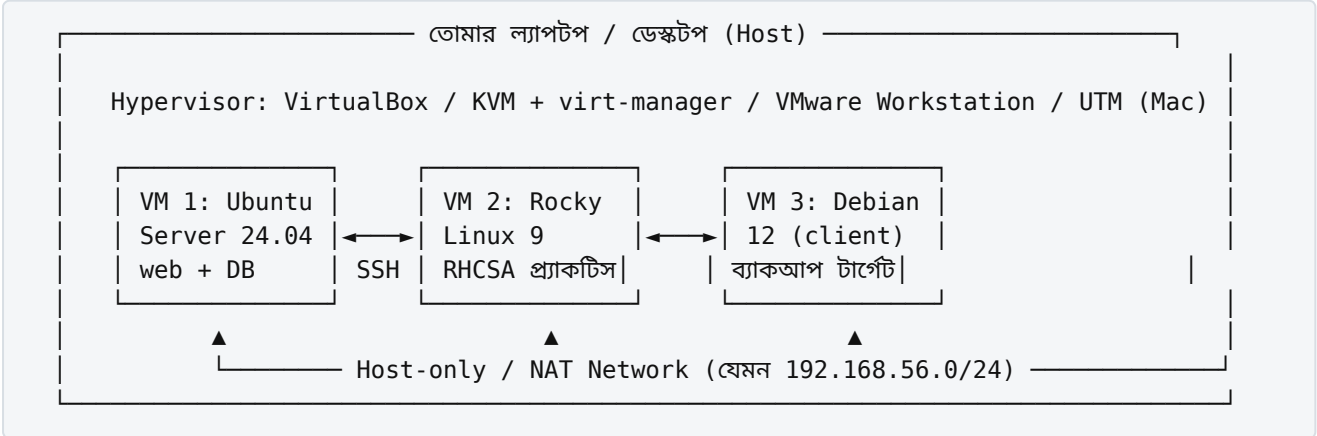
> 📝 **মনে রাখো:** Bandit শেষ হলে OverTheWire-এর পরের গেমগুলো: **Leviathan** (সহজ, বাইনারি), **Natas** (ওয়েব নিরাপত্তা), **Krypton** (ক্রিপ্টোগ্রাফি)।

আরও অনলাইন অনুশীলনের জায়গা

জায়গা	কী পাবে
SadServers (`sadservers.com`)	"ভাঙা" লিনাক্স সার্ভার ঠিক করার বাস্তব ট্রাবলশুটিং ধাঁধা — ইন্টারভিউ প্রস্তুতির জন্য দারুণ
Killercoda (`killercoda.com`)	ব্রাউজারেই লিনাক্স/Docker/Kubernetes প্লেগ্রাউন্ড ও গাইডেড সিনারিও
Linux Journey (`linuxjourney.com`)	সহজ ইংরেজিতে ধাপে ধাপে লিনাক্স পাঠ ও কুইজ
picoCTF, **TryHackMe**, **Hack The Box**	নিরাপত্তা-ভিত্তিক চ্যালেঞ্জ, যেখানে লিনাক্স দক্ষতা অপরিহার্য
Exercism (Bash track)	Bash স্ক্রিপ্টিংয়ের অনুশীলনী, মেন্টরের রিভিউসহ
cmdchallenge (`cmdchallenge.com`)	এক লাইনের কমান্ড দিয়ে সমস্যা সমাধানের চ্যালেঞ্জ

Home Lab — নিজের ঘরে লিনাক্স ল্যাব

একজন ডাক্তার যেমন ল্যাবে মডেলের ওপর অনুশীলন করে তারপর রোগীর গায়ে হাত দেয়, তেমনি তুমিও আগে নিজের ল্যাবে ভাঙবে-গড়বে, তারপর প্রোডাকশন সার্ভারে।



ল্যাবের ধরন	সুবিধা	অসুবিধা
VirtualBox	ফ্রি, Windows/macOS/Linux সবখানে চলে, snapshot	পারফরম্যান্স KVM-এর চেয়ে কম
KVM + virt-manager	লিনাক্স host-এ কার্নেল-নেটিভ, দ্রুত; প্রোডাকশন ক্লাউডেও এটাই	শুধু লিনাক্স host-এ

ল্যাবের ধরন	সুবিধা	অসুবিধা
Multipass (Canonical)	এক কমান্ডে Ubuntu VM: `multipass launch`	মূলত Ubuntu-কেন্দ্রিক
Vagrant	কোড দিয়ে VM বানানো (`Vagrantfile`), একই ল্যাব বারবার	শেখার একটা অতিরিক্ত ধাপ
WSL2 (Windows)	Windows-এর ভেতরেই আসল লিনাক্স কার্নেল	systemd/নেটওয়ার্কিংয়ে কিছু পার্থক্য; বুট প্রসেস শেখা যায় না
Docker/Podman কন্টেইনার	সেকেন্ডে নতুন পরিবেশ	কার্নেল শেয়ার হয় — বুট, ডিস্ক, কার্নেল মডিউল শেখার জন্য উপযুক্ত নয়
পুরোনো PC / Raspberry Pi	আসল হার্ডওয়্যার, ২৪/৭ হোম সার্ভার (Pi-hole, Nextcloud, মিডিয়া সার্ভার)	খরচ, জায়গা
ক্লাউড ফ্রি টায়ার (AWS, Oracle Cloud, GCP)	পাবলিক IP-সহ আসল সার্ভার	বিল নিয়ে সতর্ক থাকতে হয়

> ⚠️ **সতর্কতা:** VM-এ বড় কোনো পরীক্ষার আগে সবসময় **snapshot** নাও। `rm -rf` , `dd` , `fdisk` , `mkfs` , ফায়ারওয়াল নিয়ম — এসব ভুল হলে এক ক্লিকে আগের অবস্থায় ফিরতে পারবে। ক্লাউডে পরীক্ষা শেষে VM বন্ধ বা মুছে দিতে ভুলবে না।

হোম ল্যাবে করার মতো ১০টা প্রজেক্ট (সহজ → কঠিন):

1. দুটো VM-এর মধ্যে key-based SSH আর `rsync` ব্যাকআপ
2. Nginx + একটা স্ট্যাটিক সাইট + `ufw` / `firewalld`
3. LAMP/LEMP স্ট্যাক (Linux + Nginx + MariaDB + PHP) দিয়ে WordPress
4. Samba বা NFS দিয়ে ফাইল শেয়ার
5. Pi-hole দিয়ে ঘরের DNS ও বিজ্ঞাপন-ব্লকার
6. LVM + RAID 1 দিয়ে নিরাপদ স্টোরেজ, একটা ডিস্ক "নষ্ট" করে পুনরুদ্ধার
7. Prometheus + Grafana দিয়ে মনিটরিং ড্যাশবোর্ড
8. একটা লোকাল DNS সার্ভার (BIND বা dnsmasq)
9. Ansible দিয়ে তিনটে VM একসাথে কনফিগার
10. k3s বা kind দিয়ে ছোট Kubernetes ক্লাস্টার

খ.৮ লিনাক্স ইন্টারভিউ প্রশ্নোত্তর

ইন্টারভিউয়ে কীভাবে উত্তর দেবে

ইন্টারভিউয়ার শুধু সঠিক উত্তর শুনতে চান না — তিনি দেখতে চান তুমি **কীভাবে চিন্তা করো**। তাই প্রতিটা উত্তর তিন ধাপে সাজাও:

```
১. সংজ্ঞা (এক লাইনে) → ২. কেন/ভেতরে কীভাবে → ৩. বাস্তব উদাহরণ বা কমান্ড
```

না জানলে বানিয়ে বলবে না। বলো: "এটা আমি নিশ্চিত জানি না, তবে আমি `man` /ডকুমেন্টেশন দেখে এভাবে খুঁজে বের করতাম..." — এটা ভুল উত্তরের চেয়ে অনেক ভালো ছাপ ফেলে।

> 💡 **টিপস:** প্রতিটা প্রশ্নের উত্তর নিজে আগে মুখে বলো (বা খাতায় লেখো), তারপর "উত্তর দেখো" খোলো। শুধু পড়ে গেলে মনে হবে "এটা তো জানি" — কিন্তু ইন্টারভিউ রুমে মুখ দিয়ে বেরোবে না।

পর্ব ক: বেসিক (১–২০)

১. লিনাক্স কী? Linux আর GNU/Linux-এর পার্থক্য কী?

<details>
<summary>▼ উত্তর দেখো</summary>

কঠোর অর্থে **Linux** হলো শুধু একটা **kernel** — ১৯৯১ সালে Linus Torvalds শুরু করেছিলেন। আমরা যাকে "লিনাক্স অপারেটিং সিস্টেম" বলি, সেটা হলো এই kernel + GNU প্রজেক্টের টুল (bash, coreutils, gcc, glibc) + অন্যান্য সফটওয়্যার। তাই Free Software Foundation একে "GNU/Linux" বলতে বলে। একটা distro (Ubuntu, Fedora) এই সবকিছু একসাথে প্যাকেজ করে দেয়। বিস্তারিত: অধ্যায় ১।

</details>

২. Kernel-এর প্রধান কাজগুলো কী?

<details>
<summary>▼ উত্তর দেখো</summary>

Kernel হলো হার্ডওয়্যার আর প্রোগ্রামের মাঝখানের "ম্যানেজার"। চারটা প্রধান কাজ: (১) **প্রসেস ম্যানেজমেন্ট** ও scheduling — কে কখন CPU পাবে; (২) **মেমরি ম্যানেজমেন্ট** — virtual memory, paging; (৩) **ডিভাইস ড্রাইভার** — হার্ডওয়্যারের সাথে কথা বলা; (৪) **system call** ইন্টারফেস — প্রোগ্রাম যেন `open()`, `read()`, `fork()` দিয়ে kernel-এর সেবা চাইতে পারে। এছাড়া ফাইলসিস্টেম, নেটওয়ার্ক স্ট্যাক, নিরাপত্তাও kernel দেখে। লিনাক্স একটা **monolithic kernel** কিন্তু **loadable module** সমর্থন করে। অধ্যায় ২, ৪৭।

</details>

৩. Kernel space আর user space কী?

<details>
<summary>▼ উত্তর দেখো</summary>

CPU-র দুটো সুবিধা-স্তর (privilege level) থাকে। **Kernel space**-এ kernel চলে, সব হার্ডওয়্যার ও সব মেমরিতে পূর্ণ অধিকার নিয়ে। **User space**-এ সাধারণ প্রোগ্রাম চলে, সীমিত অধিকারে — সরাসরি হার্ডওয়্যার ছুঁতে পারে না। কিছু দরকার হলে **system call** দিয়ে kernel-কে অনুরোধ করে। এই আলাদা করার কারণে একটা প্রোগ্রাম ক্র্যাশ করলে পুরো সিস্টেম পড়ে যায় না।

</details>

৪. Shell কী? bash আর sh-এর পার্থক্য কী?

▼ উত্তর দেখো

Shell হলো একটা **command interpreter** — তুমি যা টাইপ করো সেটা পড়ে, বুঝে, প্রোগ্রাম চালায়। `sh` হলো POSIX স্ট্যান্ডার্ডের শেলের নাম; Debian/Ubuntu-তে `/bin/sh` আসলে `dash` (ছোট, দ্রুত), আর RHEL/Fedora-তে `/bin/sh` হলো `bash`-এর লিংক। `bash`-এ POSIX-এর বাইরেও অনেক সুবিধা আছে: অ্যারে, `[[ ]]`, brace expansion, `$(( ))` ইত্যাদি। তাই `#!/bin/sh` লেখা স্ক্রিপ্টে bash-এর বিশেষ সুবিধা ব্যবহার করলে Ubuntu-তে ভেঙে যেতে পারে। অধ্যায় ৫।

৫. Absolute path আর relative path-এর পার্থক্য কী?

▼ উত্তর দেখো

Absolute path সবসময় root `/` থেকে শুরু হয় — `/home/rahim/notes.txt` — যেকোনো জায়গা থেকে একই ফাইল বোঝায়। **Relative path** বর্তমান ডিরেক্টরি থেকে হিসাব হয় — `notes.txt`, `../docs/a.txt`। স্ক্রিপ্ট ও cron-এ absolute path নিরাপদ, কারণ সেখানে বর্তমান ডিরেক্টরি কী হবে নিশ্চিত না। অধ্যায় ৮।

৬. `/etc`, `/var`, `/tmp`, `/home`, `/usr`, `/opt` — কোনটা কী জন্য?

▼ উত্তর দেখো

ডিরেক্টরি	কী থাকে
`/etc`	সিস্টেম-ব্যাপী কনফিগারেশন ফাইল
`/var`	পরিবর্তনশীল ডেটা: লগ (`/var/log`), ক্যাশ, স্পুল, ডেটাবেস
`/tmp`	অস্থায়ী ফাইল; সবাই লিখতে পারে (sticky bit), রিবুটে মুছতে পারে
`/home`	সাধারণ ইউজারদের ব্যক্তিগত ডিরেক্টরি
`/usr`	ইনস্টল করা প্রোগ্রাম, লাইব্রেরি, ডকুমেন্টেশন (শুধু-পড়া ধরনের)
`/opt`	তৃতীয় পক্ষের বড় সফটওয়্যার প্যাকেজ

অধ্যায় ৭।

৭. inode কী?

▼ উত্তর দেখো

inode হলো ফাইলসিস্টেমের একটা ডেটা স্ট্রাকচার যেখানে ফাইলের **metadata** থাকে: সাইজ, মালিক (UID/GID), পারমিশন, টাইমস্ট্যাম্প, লিংক সংখ্যা, আর ডিস্কে ডেটা কোথায় আছে তার পয়েন্টার। মজার

ব্যাপার: inode-এ **ফাইলের নাম থাকে না** — নাম থাকে ডিরেক্টরিতে, যা আসলে "নাম → inode নম্বর" এর একটা তালিকা। `ls -i` বা `stat` দিয়ে দেখা যায়। অধ্যায় ১১।

৮. Hard link আর soft (symbolic) link-এর পার্থক্য কী?

▼ উত্তর দেখো

বিষয়	Hard link	Soft link
কী	একই inode-এর আরেকটা নাম	আলাদা ফাইল, যার ভেতরে টার্গেটের পাথ লেখা
inode নম্বর	মূল ফাইলের সমান	আলাদা
মূল ফাইল মুছলে	ডেটা থাকে (link count ০ না হওয়া পর্যন্ত)	লিংক ভেঙে যায় (dangling)
ডিরেক্টরিতে	সাধারণত অনুমতি নেই	করা যায়
অন্য ফাইলসিস্টেমে	যায় না	যায়

```
$ ln file.txt hard.txt
$ ln -s file.txt soft.txt
```

অধ্যায় ১১।

৯. `chmod 755` মানে কী? `rwx` ডিরেক্টরিতে কী বোঝায়?

▼ উত্তর দেখো

r=4, w=2, x=1। `755` = মালিক `7` (rwx), গ্রুপ `5` (r-x), অন্যরা `5` (r-x)। ফাইলে: r = পড়া, w = বদলানো, x = চালানো। **ডিরেক্টরিতে** অর্থ আলাদা: r = ভেতরের নামের তালিকা দেখা, w = ভেতরে ফাইল বানানো/মোছা/নাম বদলানো (x-সহ), x = ভেতরে ঢোকা (`cd`) ও ভেতরের ফাইল অ্যাক্সেস করা। তাই একটা ফাইল মুছতে ফাইলের পারমিশন না, **ডিরেক্টরির** w+x পারমিশন লাগে। অধ্যায় ১৬।

১০. umask কী? umask 022 হলে নতুন ফাইল ও ডিরেক্টরির পারমিশন কত হবে?

▼ উত্তর দেখো

umask হলো একটা "মাস্ক" যা নতুন ফাইলের ডিফল্ট পারমিশন থেকে বিট **সরিয়ে দেয়**। ফাইলের ভিত্তি `666`, ডিরেক্টরির `777`। umask `022` হলে ফাইল = `644` (rw-r--r--), ডিরেক্টরি = `755` (rwxr-xr-x)। (এটা বিয়োগ না, বিট-মাস্ক — umask `033` হলে ফাইল `644` হয়, `633` না।) অধ্যায় ১৬।

১১. root কে? `sudo` আর `su` -এর পার্থক্য কী?

▼ উত্তর দেখো

root হলো UID 0-এর সুপার ইউজার, যার ওপর প্রায় কোনো পারমিশন বাধা নেই। `su` (substitute user) অন্য ইউজার হিসেবে পুরো শেল খোলে, **টার্গেট ইউজারের** (root-এর) পাসওয়ার্ড চায়। `sudo` একটা কমান্ড অন্য ইউজার হিসেবে চালায়, **নিজের** পাসওয়ার্ড চায়, `/etc/sudoers` দিয়ে নিয়ন্ত্রিত, আর প্রতিটা ব্যবহার লগ হয়। তাই `sudo` বেশি নিরাপদ ও জবাবদিহিমূলক: root পাসওয়ার্ড কাউকে দিতে হয় না, নির্দিষ্ট কমান্ডের অনুমতি দেওয়া যায়। অধ্যায় ১৮।

১২. Process আর thread-এর পার্থক্য কী?

▼ উত্তর দেখো

Process হলো চলমান একটা প্রোগ্রাম, যার নিজস্ব আলাদা মেমরি-জগৎ (address space), নিজের PID, খোলা ফাইলের তালিকা। **Thread** একই প্রসেসের ভেতরে একাধিক কাজের ধারা, যারা মেমরি শেয়ার করে — তাই দ্রুত যোগাযোগ করে, কিন্তু একটা thread মেমরি নষ্ট করলে পুরো প্রসেস ভোগে। লিনাক্স kernel-এর চোখে দুটোই "task" — `clone()` system call দিয়ে বানানো, শুধু কতটা শেয়ার হবে সেটা আলাদা। অধ্যায় ২৪।

১৩. SIGTERM আর SIGKILL-এর পার্থক্য কী? কেন আগে `kill -9` না?

▼ উত্তর দেখো

`SIGTERM` (15) হলো ভদ্র অনুরোধ — প্রসেস এটা ধরে (catch করে) কাজ গুছিয়ে, ফাইল সেভ করে, temp ফাইল মুছে বের হতে পারে। `SIGKILL` (9) kernel সরাসরি প্রসেস মেরে ফেলে — ধরা বা উপেক্ষা করা যায় না, ফলে পরিষ্কারের সুযোগ নেই: অর্ধেক লেখা ফাইল, lock ফাইল থেকে যায়, ডেটা নষ্ট হতে পারে। তাই নিয়ম: আগে `kill PID`, কয়েক সেকেন্ড অপেক্ষা, তারপর দরকার হলে `kill -9 PID`। অধ্যায় ২৪।

১৪. `find` আর `locate`-এর পার্থক্য কী?

▼ উত্তর দেখো

`find` রিয়েল-টাইমে ডিরেক্টরি ঘুরে ঘুরে খোঁজে — ধীর, কিন্তু সবসময় হালনাগাদ, আর সাইজ/সময়/মালিক/পারমিশন দিয়ে খোঁজা যায়, `-exec` দিয়ে কাজও করা যায়। `locate` আগে থেকে বানানো ডেটাবেস (`updatedb`, সাধারণত দিনে একবার চলে) থেকে নাম খোঁজে — খুব দ্রুত, কিন্তু আজকে বানানো ফাইল নাও পেতে পারে। অধ্যায় ১২।

১৫. `>` আর `>>`, `2>&1` এর মানে কী?

▼ উত্তর দেখো

`>` stdout-কে ফাইলে পাঠায়, ফাইল আগে থাকলে **খালি করে** লেখে। `>>` ফাইলের শেষে **যোগ** করে। প্রতিটা প্রসেসের তিনটা ডিফল্ট file descriptor: 0 = stdin, 1 = stdout, 2 = stderr। `2>&1` মানে "stderr-কে সেখানে পাঠাও যেখানে stdout এখন যাচ্ছে"। ক্রম জরুরি: `cmd > out.log 2>&1` দুটোই ফাইলে পাঠায়,

কিন্তু `cmd 2>&1 > out.log` stderr-কে স্ক্রিনেই রেখে দেয়। bash-এ সংক্ষেপে `cmd &> out.log`। অধ্যায় ১৯।

১৬. Pipe (`|`) ভেতরে কীভাবে কাজ করে?

▼ উত্তর দেখো

শেল দুটো প্রসেস একসাথে চালু করে, আর kernel-এর একটা ছোট বাফার (pipe) বানিয়ে প্রথমটার stdout সেই বাফারের লেখার মুখে আর দ্বিতীয়টার stdin পড়ার মুখে জুড়ে দেয়। দুটো প্রসেস **একসাথে** চলে — বাফার ভরে গেলে লেখক অপেক্ষা করে, খালি থাকলে পাঠক। তাই `cat huge.log | grep ERROR | head` বিশাল ফাইলেও কম মেমরিতে চলে। মনে রাখো, pipe শুধু stdout পাঠায়, stderr না (`|&` দিয়ে দুটোই)। অধ্যায় ১৯।

১৭. Environment variable আর shell variable-এর পার্থক্য কী?

▼ উত্তর দেখো

Shell variable (`NAME=rahim`) শুধু বর্তমান শেলে থাকে। `export NAME` করলে সেটা **environment variable** হয় — এই শেল থেকে চালানো সব child process উত্তরাধিকার সূত্রে পায়। child প্রসেস বদলালে parent-এ প্রভাব পড়ে না। স্থায়ী করতে `~/.bashrc` বা `~/.profile`-এ লেখো। `PATH`, `HOME`, `USER`, `LANG` পরিচিত environment variable। অধ্যায় ৪৪।

১৮. `PATH` কী? কমান্ড টাইপ করলে শেল প্রোগ্রাম খোঁজে কীভাবে?

▼ উত্তর দেখো

শেল ক্রম অনুযায়ী দেখে: (১) alias, (২) shell keyword ও function, (৩) builtin, (৪) তারপর `PATH` ভেরিয়েবলে কোলন দিয়ে আলাদা করা ডিরেক্টরিগুলোতে বাম থেকে ডানে প্রথম যে চালানোর যোগ্য ফাইল মেলে। খুঁজে পাওয়া পাথ bash একটা hash table-এ মনে রাখে (`hash` কমান্ড)। বর্তমান ডিরেক্টরি নিরাপত্তার কারণে `PATH`-এ থাকে না — তাই `./script.sh` লিখতে হয়। `type -a cmd` দিয়ে সব সম্ভাবনা দেখা যায়।

১৯. `/etc/passwd` আর `/etc/shadow`-এ কী থাকে?

▼ উত্তর দেখো

`/etc/passwd` (সবাই পড়তে পারে): `username:x:UID:GID:comment:home:shell` — ৭টা ফিল্ড। `x` মানে পাসওয়ার্ড অন্য জায়গায়। `/etc/shadow` (শুধু root পড়তে পারে): পাসওয়ার্ডের **hash** (আধুনিক distro-তে সাধারণত yescrypt বা SHA-512) আর পাসওয়ার্ডের মেয়াদ সংক্রান্ত তথ্য। আলাদা রাখার কারণ: hash সবাই দেখলে offline-এ পাসওয়ার্ড ভাঙার চেষ্টা করা যেত। অধ্যায় ১৫।

২০. লিনাক্স বুটের পর কোন প্রসেসটা প্রথম চলে? তার PID কত?

▼ উত্তর দেখো

kernel নিজে চালু হয়ে প্রথম user space প্রসেস চালায় — **PID 1**, আধুনিক distro-তে সেটা **systemd** (`/sbin/init` আসলে systemd-এর লিংক)। বাকি সব প্রসেস এর বংশধর। PID 1 কখনো মারা গেলে kernel panic হয়। আর কোনো প্রসেসের parent মারা গেলে সেই "অনাথ" (orphan) প্রসেসকে PID 1 (বা কোনো subreaper) দত্তক নেয়। অধ্যায় ২৬, ২৭।

পর্ব খ: মাঝারি (২১–৪০)

২১. Zombie process আর orphan process কী?

▼ উত্তর দেখো

Zombie (`Z` অবস্থা): প্রসেস শেষ হয়ে গেছে, কিন্তু parent এখনো `wait()` দিয়ে তার exit status নেয়নি, তাই প্রসেস টেবিলে একটা এন্ট্রি রয়ে গেছে। এটা CPU/মেমরি খায় না, শুধু PID দখল করে। `kill` দিয়ে zombie মারা যায় না (সে তো মরেই আছে!) — parent-কে `SIGCHLD` পাঠাতে হয় বা parent-কে মারতে হয়, তখন init দত্তক নিয়ে পরিষ্কার করে। **Orphan**: parent আগে মারা গেছে কিন্তু child চলছে — একে PID 1 দত্তক নেয়, এটা স্বাভাবিক ও ক্ষতিকর না। অধ্যায় ২৪।

২২. Load average কী? ৪ core মেশিনে load 4.00 মানে কী?

▼ উত্তর দেখো

`uptime` / `top`-এ দেখা তিনটা সংখ্যা হলো গত ১, ৫ ও ১৫ মিনিটের গড় — কতগুলো প্রসেস চলছিল বা CPU-র জন্য অপেক্ষা করছিল (R অবস্থা), **আর লিনাক্সে** যারা uninterruptible sleep-এ (D অবস্থা, সাধারণত ডিস্ক I/O-র অপেক্ষায়) ছিল তারাও। ৪ core-এ 4.00 মানে মোটামুটি পুরো ক্ষমতায় চলছে; ৮.০০ মানে গড়ে প্রতি core-এর জন্য একজন অপেক্ষায়। load বেশি কিন্তু CPU ফাঁকা হলে সন্দেহ করো I/O সমস্যা। অধ্যায় ৪৬।

২৩. `systemctl start` আর `systemctl enable`-এর পার্থক্য কী?

▼ উত্তর দেখো

`start` সার্ভিস **এখনই** চালু করে, কিন্তু রিবুটের পর চালু হবে না। `enable` সার্ভিসকে বুটে চালু হওয়ার জন্য নিবন্ধন করে (unit-এর `[Install]` অনুযায়ী টার্গেটের `.wants/` ডিরেক্টরিতে symlink বানায়), কিন্তু এখন চালু করে না। দুটো একসাথে: `systemctl enable --now nginx`। উল্টোগুলো `stop` ও `disable`; আর `mask` দিয়ে এমনভাবে আটকানো যায় যে হাতেও চালু করা যাবে না। অধ্যায় ২৬।

২৪. লিনাক্স বুট প্রসেস ধাপে ধাপে বলো।

▼ উত্তর দেখো

```
পাওয়ার অন → UEFI/BIOS (POST, বুট ডিভাইস বাছাই)
  → বুটলোডার (GRUB2) → kernel + initramfs মেমরিতে লোড
  → kernel হার্ডওয়্যার চেনে, initramfs থেকে দরকারি ড্রাইভার লোড করে
  → আসল root ফাইলসিস্টেম মাউন্ট (switch_root)
  → PID 1 = systemd → default.target পর্যন্ত সব unit চালু
  → লগইন প্রম্পট (getty) বা গ্রাফিক্যাল লগইন (display manager)
```

অধ্যায় ২৭।

২৫. Runlevel আর systemd target-এর সম্পর্ক কী?

▼ উত্তর দেখো

পুরোনো SysV init-এ runlevel ছিল ০–৬। systemd-এ তাদের জায়গায় target: runlevel 0 ≈ `poweroff.target`, 1 ≈ `rescue.target`, 3 ≈ `multi-user.target` (টেক্সট মোড, সার্ভার), 5 ≈ `graphical.target`, 6 ≈ `reboot.target`। দেখো `systemctl get-default`, বদলাও `systemctl set-default multi-user.target`, এখনই যাও `systemctl isolate rescue.target`। অধ্যায় ২৭।

২৬. cron-এর ৫টা ফিল্ড কী? `*/15 9-17 * * 1-5` মানে কী?

▼ উত্তর দেখো

ক্রম: **মিনিট (0–59), ঘণ্টা (0–23), মাসের দিন (1–31), মাস (1–12), সপ্তাহের দিন (0–7, 0 ও 7 দুটোই রবিবার)**। উদাহরণটা মানে: সোমবার থেকে শুক্রবার, সকাল ৯টা থেকে বিকেল ৫টা ৪৫ পর্যন্ত, প্রতি ১৫ মিনিটে। মনে রাখো cron-এর পরিবেশ খুব সীমিত (ছোট `PATH`), তাই স্ক্রিপ্টে absolute path দাও আর আউটপুট লগে রিডাইরেক্ট করো। অধ্যায় ২৯।

২৭. SUID, SGID আর sticky bit কী? একটা করে উদাহরণ দাও।

▼ উত্তর দেখো

- **SUID** (4000, `s` মালিকের x-এর জায়গায়): ফাইলটা চালানোর সময় চালকের বদলে **ফাইলের মালিকের** অধিকারে চলে। উদাহরণ: `/usr/bin/passwd` — সাধারণ ইউজার নিজের পাসওয়ার্ড বদলাতে পারে, যদিও `/etc/shadow` শুধু root লিখতে পারে।
- **SGID** (2000): ফাইলে — গ্রুপের অধিকারে চলে; **ডিরেক্টরিতে** — ভেতরে বানানো নতুন ফাইল ডিরেক্টরির গ্রুপ পায় (টিম ফোল্ডারের জন্য দারুণ)।
- **Sticky bit** (1000, `t`): ডিরেক্টরিতে — ভেতরের ফাইল শুধু তার মালিক (বা ডিরেক্টরির মালিক/root) মুছতে পারে। উদাহরণ: `/tmp` (`drwxrwxrwt`)।

অধ্যায় ১৭।

২৮. ACL কেন দরকার হয়?

<details>
<summary>▼ উত্তর দেখো</summary>

সাধারণ পারমিশনে শুধু তিনটা শ্রেণি: মালিক, এক গ্রুপ, অন্য সবাই। যদি একটা ফাইল "রহিম পড়তে পারবে, করিম লিখতে পারবে, বাকি গ্রুপ কিছুই না" — এমন সূক্ষ্ম নিয়ম চাও, তখন **ACL** (Access Control List) লাগে: `setfacl -m u:karim:rw file`, দেখো `getfacl file`। ACL থাকলে `ls -l`-এ পারমিশনের শেষে `+` দেখা যায়। অধ্যায় ১৭।

</details>

২৯. LVM কী? এর সুবিধা কী?

<details>
<summary>▼ উত্তর দেখো</summary>

LVM (Logical Volume Manager) ডিস্ক আর ফাইলসিস্টেমের মাঝে একটা নমনীয় স্তর:

```
ফিজিক্যাল ডিস্ক/পার্টিশন → PV (Physical Volume)
        কয়েকটা PV মিলে → VG (Volume Group) = জায়গার একটা পুকুর
        VG থেকে কেটে নেওয়া → LV (Logical Volume) → এর ওপর ফাইলসিস্টেম
```

সুবিধা: সার্ভার না থামিয়ে ভলিউম বড় করা (`lvextend -r`), একাধিক ডিস্ক জুড়ে একটা ভলিউম, snapshot, নতুন ডিস্কে ডেটা সরানো (`pvmove`)। অধ্যায় ৩৩।

</details>

৩০. RAID 0, 1, 5, 6, 10 এর পার্থক্য কী?

<details>
<summary>▼ উত্তর দেখো</summary>

RAID	ন্যূনতম ডিস্ক	কৌশল	কতটা ডিস্ক নষ্ট সহ্য করে	ব্যবহারযোগ্য জায়গা
0	2	striping	একটাও না	১০০%
1	2	mirroring	n−1	১টা ডিস্কের সমান
5	3	striping + এক parity	১টা	(n−1) ডিস্ক
6	4	striping + দুই parity	২টা	(n−2) ডিস্ক
10	4	mirror-এর ওপর stripe	প্রতিটা mirror জোড়া থেকে ১টা	৫০%

আর সবচেয়ে জরুরি কথা: **RAID ব্যাকআপ নয়** — ভুল করে `rm` করলে সেটা সব ডিস্কেই মুছে যায়। অধ্যায় ৩৩।

</details>

৩১. `df` বলছে ডিস্ক ১০০% ভরা, কিন্তু `du` দিয়ে যোগ করলে অনেক কম আসে। কেন?

<details>
<summary>▼ উত্তর দেখো</summary>
</details>

সবচেয়ে সাধারণ কারণ: কোনো ফাইল **মুছে ফেলা হয়েছে কিন্তু কোনো প্রসেস এখনো খুলে রেখেছে** (যেমন বড় লগ ফাইল `rm` করা হয়েছে, অথচ অ্যাপ তাতে লিখেই যাচ্ছে)। নাম চলে গেছে, তাই `du` দেখে না; কিন্তু inode ও ডেটা ব্লক মুক্ত হয়নি, তাই `df` দেখে। খোঁজো: `sudo lsof +L1` বা `sudo lsof | grep deleted`। সমাধান: সংশ্লিষ্ট সার্ভিস রিস্টার্ট করো। আর ভবিষ্যতে চলমান লগ মুছবে না, খালি করবে: `: > file.log` বা `truncate -s 0 file.log`। অন্য কারণ হতে পারে: মাউন্ট পয়েন্টের **নিচে** লুকানো ফাইল, বা ext4-এর root-এর জন্য সংরক্ষিত ব্লক। অধ্যায় ৩২, ৪৬।

৩২. Swap কী? swap ব্যবহার হলেই কি সমস্যা?

▼ উত্তর দেখো

Swap হলো ডিস্কের একটা এলাকা যেখানে kernel কম-ব্যবহৃত মেমরি পেজ সরিয়ে রাখে, RAM ফাঁকা করতে। কিছুটা swap ব্যবহার স্বাভাবিক — kernel অলস পেজ সরিয়ে RAM ক্যাশের কাজে লাগায়। সমস্যা হলো যখন **ক্রমাগত** swap-in/swap-out হয় (`vmstat`-এ `si` / `so` কলাম বড়) — একে thrashing বলে, তখন সিস্টেম খুব ধীর হয়ে যায়। `vm.swappiness` (০–২০০, ডিফল্ট সাধারণত ৬০) দিয়ে kernel কতটা swap-প্রবণ হবে ঠিক করা যায়। অধ্যায় ৩২, ৪৬।

৩৩. `free -h`-এ "free" মেমরি খুব কম দেখাচ্ছে — সার্ভারে কি মেমরি শেষ?

▼ উত্তর দেখো

সম্ভবত না। লিনাক্স ফাঁকা RAM ফেলে না রেখে **page cache ও buffer** হিসেবে ডিস্কের ডেটা রাখে — "Unused RAM is wasted RAM"। প্রোগ্রামের দরকার হলে kernel এই ক্যাশ সঙ্গে সঙ্গে ছেড়ে দেয়। আসল সূচক হলো **`available`** কলাম — নতুন প্রোগ্রাম swap ছাড়াই কতটা পাবে তার অনুমান। অধ্যায় ৪৬।

৩৪. কোন পোর্টে কোন প্রসেস শুনছে কীভাবে দেখবে?

▼ উত্তর দেখো

```
$ sudo ss -tulpn
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
tcp   LISTEN 0      511          0.0.0.0:80         0.0.0.0:*     users:
(("nginx",pid=812,fd=6))
tcp   LISTEN 0      4096         0.0.0.0:22         0.0.0.0:*     users:
(("sshd",pid=701,fd=3))
```

অপশন: `-t` TCP, `-u` UDP, `-l` শুধু listening, `-p` প্রসেস (root লাগে), `-n` নাম না, সংখ্যা। বিকল্প: `sudo lsof -i :80`। পুরোনো সিস্টেমে `netstat -tulpn`। অধ্যায় ৩৭।

৩৫. `ping google.com` কাজ করছে না — কীভাবে ধাপে ধাপে খুঁজবে?

▼ উত্তর দেখো

নিচ থেকে ওপরে (OSI স্তর ধরে):

1. লিংক আছে? `ip link` (state UP?)
2. IP আছে? `ip a`
3. গেটওয়ে আছে ও পৌঁছানো যায়? `ip r`, তারপর `ping <gateway>`
4. বাইরের IP-তে যায়? `ping -c 3 8.8.8.8` — গেলে সমস্যা DNS-এ
5. DNS: `resolvectl status` বা `cat /etc/resolv.conf`, তারপর `dig google.com`
6. পথে কোথায় আটকাচ্ছে? `traceroute` / `mtr`
7. ফায়ারওয়াল ICMP আটকাচ্ছে কিনা — কিছু নেটওয়ার্ক `ping` ব্লক করে, তখন `curl -I https://google.com` দিয়ে দেখো।

অধ্যায় ৩৬, ৩৭।

৩৬. হোস্টনেম থেকে IP বের করার ক্রম কী? `/etc/hosts` কখন আগে দেখা হয়?

▼ উত্তর দেখো

ক্রমটা ঠিক করে `/etc/nsswitch.conf`-এর `hosts:` লাইন — সাধারণত `files` আগে (মানে `/etc/hosts`), তারপর `dns`; কিছু distro-তে মাঝে `mdns4_minimal`, `myhostname` বা `resolve` (systemd-resolved) ধরনের এন্ট্রিও থাকে। তাই `/etc/hosts`-এ কোনো এন্ট্রি থাকলে DNS জিজ্ঞেসই করা হয় না। মনে রাখো `dig` ও `nslookup` সরাসরি DNS সার্ভারকে জিজ্ঞেস করে, nsswitch মানে না; সিস্টেম যা দেখে সেটা দেখতে `getent hosts name` ব্যবহার করো। অধ্যায় ৩৬।

৩৭. SSH key-based authentication কীভাবে কাজ করে?

▼ উত্তর দেখো

তুমি `ssh-keygen` দিয়ে একজোড়া কী বানাও: **private key** (তোমার কাছে, গোপন) আর **public key** (সার্ভারের `~/.ssh/authorized_keys`-এ, `ssh-copy-id` দিয়ে)। লগইনের সময় সার্ভার একটা চ্যালেঞ্জ দেয়; তোমার ক্লায়েন্ট private key দিয়ে সেটায় **ডিজিটাল স্বাক্ষর** করে; সার্ভার public key দিয়ে যাচাই করে। private key কখনো নেটওয়ার্কে যায় না। পাসওয়ার্ডের চেয়ে নিরাপদ, কারণ brute-force প্রায় অসম্ভব। আধুনিক পছন্দ `ssh-keygen -t ed25519`। অধ্যায় ৩৮।

৩৮. `apt` বনাম `dpkg`, আর `dnf` বনাম `rpm` — পার্থক্য কী?

▼ উত্তর দেখো

`dpkg` ও `rpm` হলো নিচু-স্তরের টুল: একটা লোকাল প্যাকেজ ফাইল ইনস্টল/রিমুভ/কোয়েরি করে, কিন্তু **dependency নিজে ডাউনলোড করে না**। `apt` ও `dnf` হলো উঁচু-স্তরের টুল: রিপোজিটরি থেকে প্যাকেজ

খোঁজে, dependency হিসাব করে সব ডাউনলোড করে, তারপর ভেতরে `dpkg` / `rpm`-এর কাজটাই করে। দরকারি কোয়েরি: "এই ফাইল কোন প্যাকেজের?" → `dpkg -S /path` বা `rpm -qf /path`। অধ্যায় ৩০।

৩৯. `/proc` আর `/sys` কী? এগুলো কি ডিস্কে থাকে?

▼ উত্তর দেখো

না — এরা **ভার্চুয়াল ফাইলসিস্টেম**, মেমরিতে kernel পড়ার মুহূর্তে বানায়। `/proc` প্রসেস (`/proc/<PID>/`) আর kernel-এর তথ্য দেখায় (`/proc/cpuinfo`, `/proc/meminfo`), আর `/proc/sys/` দিয়ে kernel প্যারামিটার বদলানো যায় (`sysctl` এটাই ব্যবহার করে)। `/sys` (sysfs) ডিভাইস, ড্রাইভার আর kernel অবজেক্টকে সুশৃঙ্খল গাছ আকারে দেখায়। `ls -l /proc/cpuinfo` সাইজ ০ দেখায়, তবু `cat` করলে লেখা আসে — এটাই প্রমাণ। অধ্যায় ৪৭।

৪০. Soft limit আর hard limit (`ulimit`) কী?

▼ উত্তর দেখো

kernel প্রতিটা প্রসেসের রিসোর্সে (খোলা ফাইল, প্রসেস সংখ্যা, core dump সাইজ) সীমা বসায়। **Soft limit** হলো কার্যকর সীমা — ইউজার নিজে hard limit পর্যন্ত বাড়াতে পারে। **Hard limit** হলো ছাদ — শুধু root বাড়াতে পারে। দেখো `ulimit -Sn` / `ulimit -Hn` (খোলা ফাইল)। স্থায়ী করতে `/etc/security/limits.conf`; কিন্তু systemd সার্ভিসের জন্য unit ফাইলে `LimitNOFILE=` দিতে হয়, কারণ সার্ভিস PAM-এর limits.conf দিয়ে যায় না। অধ্যায় ৪৬।

পর্ব গ: অ্যাডভান্সড ও বাস্তব পরিস্থিতি (৪১–৫৮)

৪১. টার্মিনালে `ls -l` লিখে Enter চাপলে ভেতরে কী কী ঘটে?

▼ উত্তর দেখো

1. টার্মিনাল (tty/pty) কীবোর্ডের অক্ষরগুলো শেলকে দেয়; Enter-এ শেল লাইনটা পড়ে।
2. শেল লাইনটা ভাঙে (tokenize), expansion করে (variable, wildcard, alias — `ls` হয়তো `ls --color=auto`-এর alias)।
3. শেল `ls` খোঁজে: alias → function → builtin → `PATH` → `/usr/bin/ls`।
4. শেল `fork()` (আসলে `clone()`) করে নিজের একটা কপি বানায়।
5. child প্রসেস `execve("/usr/bin/ls", ...)` দিয়ে নিজেকে `ls` প্রোগ্রামে বদলে ফেলে; dynamic linker (`ld-linux`) দরকারি লাইব্রেরি (libc ইত্যাদি) লোড করে।
6. `ls` system call দিয়ে ডিরেক্টরি পড়ে (`openat`, `getdents64`), প্রতিটা ফাইলের তথ্য নেয় (`statx` / `lstat`), `write()` দিয়ে stdout-এ লেখে।
7. `ls` `exit()` করে; parent শেল `wait()` দিয়ে exit status নেয় (`$?`), তারপর আবার প্রম্পট দেখায়।

নিজে দেখো: `strace -f ls -l 2>&1 | less`। অধ্যায় ২, ২৪, ৪৬।

৪২. প্রোডাকশন সার্ভারের ডিস্ক ভরে গেছে। কী করবে?

▼ উত্তর দেখো

```
$ df -h                                          # কোন ফাইলসিস্টেম ভরা
$ df -i                                          # inode শেষ কিনা (ছোট ফাইলের ঢল)
$ sudo du -xh / --max-depth=1 | sort -h     # কোন ডিরেক্টরি বড় (-x = অন্য ফাইলসিস্টেমে যেও না)
$ sudo find / -xdev -type f -size +500M -exec ls -lh {} +
$ sudo lsof +L1                                  # মুছে ফেলা কিন্তু খোলা ফাইল
$ sudo journalctl --disk-usage
$ sudo journalctl --vacuum-size=500M
$ sudo apt clean   # বা sudo dnf clean all
```

সাধারণ অপরাধী: `/var/log`, পুরোনো kernel, প্যাকেজ ক্যাশ, Docker ইমেজ (`docker system df`, `docker system prune`), core dump, ব্যাকআপ ফাইল। তাড়াহুড়ায় চলমান লগ `rm` কোরো না — truncate করো। শেষে মূল কারণ ঠিক করো (logrotate কনফিগ, মনিটরিং অ্যালার্ট)। অধ্যায় ২৮, ৩২, ৪৬।

৪৩. সার্ভার হঠাৎ খুব ধীর। কীভাবে তদন্ত করবে?

▼ উত্তর দেখো

একটা পরিচিত "৬০ সেকেন্ডের চেকলিস্ট":

```
$ uptime            # load average বাড়ছে না কমছে
$ dmesg -T | tail   # OOM killer, ডিস্ক এরর, হার্ডওয়্যার সমস্যা
$ vmstat 1          # r (রান কিউ), si/so (swap), wa (I/O অপেক্ষা)
$ mpstat -P ALL 1   # কোনো একটা core কি ১০০%?
$ pidstat 1         # কোন প্রসেস CPU খাচ্ছে
$ iostat -xz 1      # ডিস্কের %util, await
$ free -h           # available মেমরি
$ sar -n DEV 1      # নেটওয়ার্ক থ্রুপুট
$ top / htop        # সারসংক্ষেপ
```

(`mpstat`, `pidstat`, `iostat`, `sar` আসে `sysstat` প্যাকেজে।) তারপর USE পদ্ধতি: প্রতিটা রিসোর্সের **U**tilization, **S**aturation, **E**rrors। অধ্যায় ৪৬।

৪৪. SSH দিয়ে সার্ভারে ঢুকতে পারছ না। কী কী দেখবে?

▼ উত্তর দেখো

ক্লায়েন্ট দিক থেকে: `ssh -v user@host` (বা `-vvv`) — কোথায় আটকাচ্ছে দেখায়। নেটওয়ার্ক: `ping host`, `nc -zv host 22`। "Connection refused" = পোর্টে কেউ শুনছে না (sshd বন্ধ বা অন্য পোর্টে); "timed out" = ফায়ারওয়াল বা রাউটিং; "Permission denied (publickey)" = কী সমস্যা। সার্ভার দিক

থেকে (কনসোল দিয়ে): `systemctl status ssh` (RHEL-এ `sshd`), `journalctl -u ssh`, `sudo sshd -t` (কনফিগ পরীক্ষা), ফায়ারওয়াল, `~/.ssh` (700) ও `authorized_keys` (600)-এর পারমিশন ও মালিক, home ডিরেক্টরি অন্যদের লেখার যোগ্য কিনা, SELinux context (`restorecon -Rv ~/.ssh`), fail2ban তোমার IP ব্লক করেছে কিনা। অধ্যায় ৩৮।

৪৫. "Too many open files" এরর কেন আসে, কীভাবে ঠিক করবে?

▼ উত্তর দেখো

প্রসেসের খোলা file descriptor (ফাইল, সকেট, pipe) তার `nofile` limit ছুঁয়ে ফেলেছে। প্রথমে দেখো লিক কিনা: `ls /proc/<PID>/fd | wc -l` সময়ের সাথে বাড়ছে কি? `cat /proc/<PID>/limits` দিয়ে বর্তমান সীমা দেখো। আসলেই বেশি দরকার হলে (ব্যস্ত ওয়েব/DB সার্ভার) systemd unit-এ `LimitNOFILE=65535` (`systemctl edit service` দিয়ে override), বা সাধারণ ইউজারের জন্য `/etc/security/limits.conf`। সিস্টেম-ব্যাপী সীমা: `fs.file-max`। অধ্যায় ৪৬, ৪৭।

৪৬. ডিস্কে জায়গা আছে, তবু "No space left on device" — কেন?

▼ উত্তর দেখো

খুব সম্ভবত **inode শেষ** — `df -i` দেখো। লাখো ছোট ফাইল (session ফাইল, mail queue, ক্যাশ) প্রতিটা একটা inode খায়; ext4-এ inode সংখ্যা ফাইলসিস্টেম বানানোর সময়েই ঠিক হয়ে যায়। খোঁজো কোন ডিরেক্টরিতে সবচেয়ে বেশি ফাইল: `sudo du --inodes -x / | sort -n | tail`। অন্য কারণ: quota শেষ, বা ext4-এর সংরক্ষিত ব্লক (সাধারণ ইউজারের জন্য)। অধ্যায় ১১, ৩২।

৪৭. স্ক্রিপ্টে `chmod +x` দেওয়া, তবু `./script.sh` চালালে "Permission denied" বা "bad interpreter" আসে। কারণ?

▼ উত্তর দেখো

সম্ভাব্য কারণ: (১) ফাইলসিস্টেম `noexec` অপশনে মাউন্ট করা (প্রায়ই `/tmp` বা USB) — `findmnt -T script.sh` দেখো; (২) ডিরেক্টরিতে x পারমিশন নেই; (৩) shebang ভুল — `#!/bin/bsh`; (৪) Windows-এ লেখা ফাইল, লাইনের শেষে `\r` (CRLF) — তখন এরর আসে `/bin/bash^M: bad interpreter`; ঠিক করো `sed -i 's/\r$//' script.sh` বা `dos2unix`; (৫) SELinux/AppArmor নিষেধ করছে। অধ্যায় ১৬, ৪১।

৪৮. OOM killer কী? কোন প্রসেস মারা হলো কীভাবে জানবে?

▼ উত্তর দেখো

মেমরি (RAM + swap) পুরো শেষ হলে kernel-এর **Out-Of-Memory killer** একটা প্রসেস বেছে নিয়ে মেরে ফেলে, যাতে পুরো সিস্টেম না আটকায়। বাছাই হয় প্রতিটা প্রসেসের `oom_score` দিয়ে (মূলত কতটা মেমরি

খাচ্ছে), যা `/proc/<PID>/oom_score_adj` দিয়ে প্রভাবিত করা যায় (−1000 = কখনো মারবে না)। খোঁজো: `dmesg -T | grep -i -E "killed process|out of memory"` বা `journalctl -k | grep -i oom`। systemd সার্ভিসে `MemoryMax=` দিয়ে আগেই সীমা বাঁধা যায়। অধ্যায় ৪৬, ৪৭।

৪৯. iptables, nftables, firewalld আর ufw — এদের সম্পর্ক কী?

▼ উত্তর দেখো

আসল প্যাকেট ফিল্টারিং করে kernel-এর **netfilter** ফ্রেমওয়ার্ক। `iptables` ছিল পুরোনো ইউজার-স্পেস টুল; `nftables` (`nft` কমান্ড) এর আধুনিক উত্তরসূরি, যা Debian 10+ ও RHEL 8+-এ ডিফল্ট — আজকের `iptables` কমান্ডও প্রায়ই `iptables-nft` ব্যাকএন্ড দিয়ে nftables-এই নিয়ম লেখে। `firewalld` (RHEL/Fedora) আর `ufw` (Ubuntu) হলো উঁচু-স্তরের সহজ সামনের-মুখ (frontend), যারা নিচে nftables/iptables নিয়ম বানায়। একসাথে একাধিক frontend ব্যবহার করবে না — নিয়ম জট পাকাবে।

```
ufw / firewalld   ← সহজ ইন্টারফেস
       │
nft / iptables    ← নিয়ম লেখার টুল
       │
netfilter (kernel) ← আসল ফিল্টার
```

অধ্যায় ৩৯।

৫০. SELinux আর AppArmor কী? "SELinux বন্ধ করে দাও" কেন খারাপ পরামর্শ?

▼ উত্তর দেখো

দুটোই **MAC** (Mandatory Access Control) — সাধারণ পারমিশনের (DAC) ওপর একটা বাড়তি নিরাপত্তা স্তর: কোনো প্রসেস root হিসেবে চললেও তার নীতির (policy) বাইরে কিছু করতে পারে না। SELinux (RHEL/Fedora) প্রতিটা ফাইল ও প্রসেসে **label/context** লাগায়; AppArmor (Ubuntu/Debian/SUSE) **পাথ-ভিত্তিক প্রোফাইল** ব্যবহার করে। কোনো সার্ভিস আক্রান্ত হলেও এরা ক্ষতি আটকে রাখে। বন্ধ না করে সমস্যা ঠিক করো: `ausearch -m avc -ts recent`, `restorecon -Rv /path`, `setsebool -P httpd_can_network_connect on`, `semanage port -a -t http_port_t -p tcp 8080`। সাময়িক পরীক্ষার জন্য বড়জোর `setenforce 0` (permissive)। অধ্যায় ৪৫।

৫১. কন্টেইনার আর ভার্চুয়াল মেশিনের পার্থক্য কী? কন্টেইনার লিনাক্সের কোন ফিচারের ওপর দাঁড়িয়ে?

▼ উত্তর দেখো

VM-এর প্রতিটার নিজের পুরো kernel থাকে, hypervisor ভার্চুয়াল হার্ডওয়্যার দেয় — শক্ত আলাদাকরণ, কিন্তু ভারী (GB, মিনিট)। কন্টেইনার হোস্টের **একই kernel শেয়ার করে**, শুধু আলাদা করা প্রসেস — হালকা (MB, সেকেন্ড)। ভিত্তি তিনটা kernel ফিচার: **namespaces** (কী দেখতে পাবে — PID, network, mount, UTS, IPC, user, cgroup), **cgroups** (কতটা ব্যবহার করতে পারবে — CPU, মেমরি, I/O), আর **union/overlay**

filesystem (স্তরযুক্ত ইমেজ)। এর ওপর seccomp, capabilities, SELinux/AppArmor নিরাপত্তা দেয়। অধ্যায় ৪৮।

৫২. `$?`, `$$`, `$!`, `$#`, `$0`, `$@` আর `$*` কী?

▼ উত্তর দেখো

ভেরিয়েবল	মানে
`$?`	শেষ কমান্ডের exit status (0 = সফল)
`$$`	বর্তমান শেলের PID
`$!`	সর্বশেষ ব্যাকগ্রাউন্ড প্রসেসের PID
`$#`	আর্গুমেন্টের সংখ্যা
`$0`	স্ক্রিপ্টের নাম
`"$@"`	প্রতিটা আর্গুমেন্ট আলাদা শব্দ হিসেবে — স্পেসওয়ালা আর্গুমেন্টও অক্ষত থাকে
`"$*"`	সব আর্গুমেন্ট জুড়ে **একটা** শব্দ (IFS-এর প্রথম অক্ষর দিয়ে)

প্রায় সবসময় `"$@"` (কোটসহ) ব্যবহার করবে। অধ্যায় ৪১, ৪২।

৫৩. `set -euo pipefail` কী করে? কেন স্ক্রিপ্টের শুরুতে দেওয়া হয়?

▼ উত্তর দেখো

- `-e`: কোনো কমান্ড ব্যর্থ হলে (non-zero exit) স্ক্রিপ্ট থেমে যায় (কিছু ব্যতিক্রম আছে — `if`-এর শর্ত, `&&` / `||`-এর বাম পাশ)।
- `-u`: সংজ্ঞায়িত-না-করা ভেরিয়েবল ব্যবহার করলে এরর। এটা `rm -rf "$DIR/"*` জাতীয় দুর্ঘটনা আটকায় যখন `DIR` খালি।
- `-o pipefail`: pipe-এর **যেকোনো** অংশ ব্যর্থ হলে পুরো pipe ব্যর্থ ধরা হয় (না হলে শুধু শেষ কমান্ডের status গোনা হয়)।

সাথে প্রায়ই `trap 'cleanup' EXIT` দেওয়া হয়। অধ্যায় ৪৩।

৫৪. ext4, XFS আর Btrfs — কোনটা কখন?

▼ উত্তর দেখো

ext4: Debian/Ubuntu-র ডিফল্ট, পরিপক্ক, বড় বা ছোট করা যায় (ছোট করতে unmount লাগে)। **XFS**: RHEL-এর ডিফল্ট, বড় ফাইল ও সমান্তরাল I/O-তে চমৎকার, অনলাইনে বড় করা যায় (`xfs_growfs`), কিন্তু **ছোট করা যায় না**। **Btrfs**: Fedora Workstation ও openSUSE-র ডিফল্ট — copy-on-write, snapshot,

subvolume, checksum, কম্প্রেশন। ইন্টারভিউয়ের ফাঁদ প্রশ্ন: "XFS ভলিউম ছোট করবে কীভাবে?" — উত্তর: সরাসরি পারবে না; ব্যাকআপ নিয়ে নতুন করে বানিয়ে restore। অধ্যায় ৩২।

৫৫. `/etc/fstab`-এর ৬টা ফিল্ড কী? ভুল fstab-এ কী হয়?

▼ উত্তর দেখো

```
UUID=3f2a...  /data  ext4  defaults,noatime  0  2
   (১)         (২)   (৩)        (৪)          (৫) (৬)
```

(১) ডিভাইস (UUID ভালো, কারণ `/dev/sdb1` নাম বদলাতে পারে), (২) মাউন্ট পয়েন্ট, (৩) ফাইলসিস্টেম টাইপ, (৪) মাউন্ট অপশন, (৫) dump (প্রায় সবসময় 0), (৬) fsck ক্রম (root = 1, অন্যরা 2, 0 = পরীক্ষা না)। ভুল এন্ট্রি থাকলে বুট emergency mode-এ আটকে যেতে পারে — তাই সম্পাদনার পর রিবুটের আগে অবশ্যই `sudo mount -a` (আর systemd-তে `sudo systemctl daemon-reload`) দিয়ে পরীক্ষা করো; দরকার হলে অপশনে `nofail` দাও। অধ্যায় ৩২।

৫৬. root পাসওয়ার্ড ভুলে গেছ — RHEL/Rocky-তে কীভাবে রিসেট করবে?

▼ উত্তর দেখো

1. রিবুট, GRUB মেনুতে `e` চেপে kernel লাইন এডিট করো।
2. `linux ...` লাইনের শেষে `rd.break` যোগ করো, `Ctrl+x` দিয়ে বুট।
3.
```
switch_root:/# mount -o remount,rw /sysroot
switch_root:/# chroot /sysroot
sh-5.1# passwd root
sh-5.1# touch /.autorelabel      # SELinux লেবেল ঠিক করার জন্য — বাদ দিলে লগইন ভাঙবে
sh-5.1# exit
switch_root:/# exit
```

(Ubuntu/Debian-এ সাধারণত GRUB-এ `init=/bin/bash` দিয়ে, তারপর `mount -o remount,rw /` আর `passwd`।) এটা RHCSA-র ক্লাসিক কাজ — আর এটা দেখায় কেন **শারীরিক অ্যাক্সেস = পূর্ণ অ্যাক্সেস**, তাই GRUB পাসওয়ার্ড ও ডিস্ক এনক্রিপশন জরুরি। অধ্যায় ২৭, ৪৫।

৫৭. Nice value কী? কোন প্রসেসকে কম অগ্রাধিকার দেবে কীভাবে?

▼ উত্তর দেখো

Nice value −20 (সর্বোচ্চ অগ্রাধিকার) থেকে 19 (সর্বনিম্ন), ডিফল্ট 0। নাম থেকেই বোঝা যায়: বেশি "nice" মানে অন্যদের প্রতি বেশি ভদ্র — কম CPU নেয়। সাধারণ ইউজার শুধু nice **বাড়াতে** পারে, কমাতে (অগ্রাধিকার

বাড়াতে) root লাগে। নতুন কমান্ড: `nice -n 10 tar -czf backup.tgz /data`; চলমান প্রসেস: `renice -n 15 -p 1234`। ডিস্কের জন্য একই রকম `ionice`। অধ্যায় ২৪।

৫৮. তুমি একটা সার্ভারের দায়িত্ব নিলে প্রথম দিনে কী কী দেখবে?

▼ উত্তর দেখো

একটা ভালো উত্তরে দেখাও তুমি গোছানো:

- **পরিচয়**: `cat /etc/os-release`, `uname -r`, `hostnamectl`, `uptime`
- **রিসোর্স**: `lscpu`, `free -h`, `lsblk`, `df -h`
- **কী চলছে**: `systemctl list-units --type=service --state=running`, `ss -tulpn`, `crontab -l` ও `/etc/cron.*`, `systemctl list-timers`
- **কে আসতে পারে**: `/etc/passwd`-এ লগইন শেলওয়ালা ইউজার, `sudo` গ্রুপ, `/etc/ssh/sshd_config`, `authorized_keys`
- **নিরাপত্তা**: ফায়ারওয়াল নিয়ম, SELinux/AppArmor অবস্থা, পেন্ডিং আপডেট
- **স্বাস্থ্য**: `journalctl -p err -b`, `dmesg -T`, `last`
- **ব্যাকআপ**: কোথায়, কত ঘনঘন, শেষ কবে **restore পরীক্ষা** হয়েছে
- আর সব কিছু ডকুমেন্টে লিখে রাখা।

অধ্যায় ২৬, ৪৫, ৪৬।

ইন্টারভিউয়ের দ্রুত-প্রশ্ন (এক লাইনের উত্তর)

ইন্টারভিউয়ের শুরুতে প্রায়ই "র‍্যাপিড ফায়ার" হয়:

প্রশ্ন	উত্তর
কার্নেল ভার্সন?	`uname -r`
লুকানো ফাইল কীভাবে বানায়?	নামের শুরুতে `.`
ফাইলের প্রথম ৫ লাইন?	`head -n 5 file`
লাইন গোনা?	`wc -l file`
বর্তমান শেল?	`echo $0` বা `ps -p $$`
ব্যাকগ্রাউন্ডে চালানো?	কমান্ডের শেষে `&`
ডিফল্ট SSH পোর্ট?	22
HTTP / HTTPS / DNS পোর্ট?	80 / 443 / 53
root-এর UID?	0
`/etc/shadow` কে পড়তে পারে?	শুধু root
শেষ কমান্ডের exit status?	`echo $?`

প্রশ্ন	উত্তর
ইউজারকে গ্রুপে যোগ (অন্য গ্রুপ না হারিয়ে)?	`usermod -aG group user`
ফাইলের মালিক বদল?	`chown user:group file`
সব চলমান প্রসেস?	`ps aux`
খোলা পোর্ট?	`ss -tulpn`
লগ লাইভ দেখা?	`tail -f` / `journalctl -f`
`tar`-এ gzip / bzip2 / xz অপশন?	`-z` / `-j` / `-J`
ফাইলের আসল টাইপ?	`file name`
ডিরেক্টরির সাইজ?	`du -sh dir`
সিস্টেম কতক্ষণ চালু?	`uptime`

খ.৯ দরকারি রিসোর্স

টার্মিনালের ভেতরের সাহায্য (সবসময় হাতের কাছে, ইন্টারনেট ছাড়াই)

টুল	কী দেয়	ইনস্টল
`man`	পূর্ণ ম্যানুয়াল	সাধারণত আগে থেকেই থাকে (minimal/কন্টেইনার ইমেজে নাও থাকতে পারে)
`info`	GNU টুলের আরও বিস্তারিত দলিল (যেমন `info coreutils`)	প্রায়ই আগে থেকে
`--help`	অপশনের সংক্ষিপ্ত তালিকা	কমান্ডের সাথেই
`help`	bash builtin-এর সাহায্য	bash-এর সাথেই
`tldr`	উদাহরণভিত্তিক ছোট সাহায্য	`sudo apt install tldr` / `sudo dnf install tldr` (বা `pip` / `npm` দিয়ে অন্য ক্লায়েন্ট)
`vimtutor`	৩০ মিনিটের ইন্টারঅ্যাক্টিভ Vim পাঠ	Vim-এর সাথে
`/usr/share/doc/`	প্যাকেজের নিজস্ব ডকুমেন্টেশন, উদাহরণ কনফিগ	প্যাকেজের সাথে

বই (ইংরেজি)

বই	লেখক	কেন পড়বে
The Linux Command Line	William Shotts	নতুনদের জন্য সেরা; লেখকের সাইট `linuxcommand.org`-এ বিনামূল্যে PDF
How Linux Works	Brian Ward	"ভেতরে কীভাবে কাজ করে" — এই বইয়ের দর্শনের সবচেয়ে কাছাকাছি

বই	লেখক	কেন পড়বে
UNIX and Linux System Administration Handbook	Evi Nemeth ও অন্যান্য	সিসঅ্যাডমিনের "বাইবেল"
Linux Bible	Christopher Negus	বিস্তৃত, RHEL-ঘেঁষা, RHCSA প্রস্তুতিতে সহায়ক
Classic Shell Scripting	Arnold Robbins ও Nelson Beebe	পোর্টেবল শেল স্ক্রিপ্টিং গভীরে
Systems Performance	Brendan Gregg	পারফরম্যান্স বিশ্লেষণের সর্বোচ্চ স্তর
The Linux Programming Interface	Michael Kerrisk	system call ও kernel-ইন্টারফেসের বিশ্বকোষ (এক্সপার্ট স্তর)

ওয়েবসাইট ও ডকুমেন্টেশন

সাইট	কী পাবে
ArchWiki (`wiki.archlinux.org`)	যেকোনো distro-র জন্য লিনাক্সের সবচেয়ে বিস্তারিত উইকি
Ubuntu Server Docs (`documentation.ubuntu.com/server`)	Ubuntu সার্ভারের অফিসিয়াল গাইড
Red Hat Documentation (`docs.redhat.com`)	RHEL-এর অফিসিয়াল ডকুমেন্টেশন — RHCSA-র জন্য সোনা
The Debian Administrator's Handbook (`debian-handbook.info`)	বিনামূল্যের পূর্ণ বই
kernel.org documentation (`docs.kernel.org`)	kernel-এর অফিসিয়াল দলিল
explainshell.com	যেকোনো কমান্ড লাইন পেস্ট করো, প্রতিটা অংশের ব্যাখ্যা পাবে
ShellCheck (`shellcheck.net` , বা `shellcheck` কমান্ড)	শেল স্ক্রিপ্টের ভুল ধরে দেয় — প্রতিটা স্ক্রিপ্টে চালাও
Bash Reference Manual (`gnu.org/software/bash/manual`)	bash-এর অফিসিয়াল ম্যানুয়াল
Greg's Wiki — BashFAQ ও BashPitfalls (`mywiki.wooledge.org`)	স্ক্রিপ্টিংয়ের সাধারণ ভুল ও সঠিক পদ্ধতি
Brendan Gregg's site (`brendangregg.com`)	পারফরম্যান্স টুলের ডায়াগ্রাম ও পদ্ধতি
Unix & Linux Stack Exchange (`unix.stackexchange.com`)	প্রশ্নোত্তর; প্রশ্ন করার আগে খুঁজে দেখো

কমিউনিটি

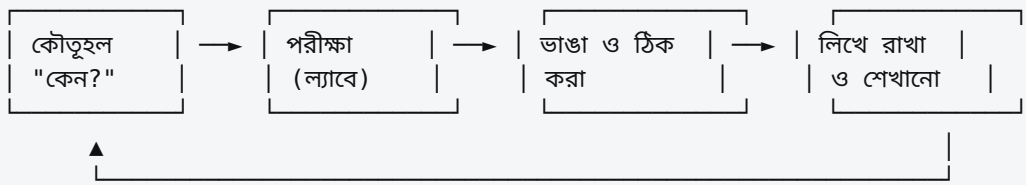
- স্থানীয় লিনাক্স ইউজার গ্রুপ (LUG) ও ওপেন সোর্স কমিউনিটি — বাংলাদেশে বিভিন্ন বিশ্ববিদ্যালয়ভিত্তিক ও অনলাইন গ্রুপ আছে
- Reddit: `r/linux4noobs` (নতুনদের), `r/linuxadmin` , `r/sysadmin`

- নিজের distro-র অফিসিয়াল ফোরাম (Ask Ubuntu, Fedora Discussion)

> 💡 **টিপস:** প্রশ্ন করার সময় সবসময় দাও: distro ও ভার্সন (`cat /etc/os-release`), ঠিক কী কমান্ড চালিয়েছ, **পুরো** এরর মেসেজ (স্ক্রিনশট না, টেক্সট), আর তুমি আগে কী কী চেষ্টা করেছ। ভালো প্রশ্ন অর্ধেক উত্তর।

> ⚠️ **সতর্কতা:** অনলাইনে পাওয়া কোনো কমান্ড — বিশেষ করে `curl ... | sudo bash`, `rm -rf`, `dd`, `chmod -R 777` — না বুঝে চালাবে না। আগে explainshell.com বা `man` দিয়ে প্রতিটা অংশ বুঝে নাও।

শেখার মানসিকতা: এক্সপার্টরা যা করে



1. **প্রতিটা এরর মেসেজ পুরোটা পড়ো।** বেশিরভাগ সময় সমাধান মেসেজেই লেখা থাকে।
2. **লগ আগে, অনুমান পরে।** `journalctl`, `/var/log`, `dmesg`।
3. **একবারে একটা পরিবর্তন।** পাঁচটা জিনিস একসাথে বদলালে কোনটায় ঠিক হলো জানবে না।
4. **কনফিগ বদলানোর আগে ব্যাকআপ:** `sudo cp file file.bak.$(date +%F)`।
5. **নিজের নোট রাখো** — একটা Git রিপোজিটরিতে Markdown ফাইল, যেখানে প্রতিটা সমাধান করা সমস্যা লিখে রাখবে। এটাই তোমার নিজের "বই"।
6. **অন্যকে শেখাও।** যা ব্যাখ্যা করতে পারো না, তা আসলে পুরো বোঝোনি।

হাতে-কলমে অনুশীলন

এই অনুশীলনগুলো পুরো বইয়ের একটা "মিনি পরীক্ষা"। প্রতিটায় বন্ধনীর মধ্যে অধ্যায় নম্বর দেওয়া আছে — আটকে গেলে সেখানে ফিরে যাও।

ল্যাব ১: নিজের কমান্ড-অভিধান বানাও (অধ্যায় ৫, ১২, ১৯, ২৩)

তোমার সিস্টেমে কতগুলো কমান্ড আছে, আর তাদের এক লাইনের বর্ণনা — এই পরিশিষ্টের মতোই একটা তালিকা নিজে বানাও।

```
$ compgen -c | sort -u | wc -l
2143
$ for c in ls cp grep awk ss; do whatis "$c" 2>/dev/null | head -n 1; done
ls (1)               - list directory contents
```

```
cp (1)               - copy files and directories
grep (1)             - print lines that match patterns
awk (1)              - pattern scanning and text processing language
ss (8)               - another utility to investigate sockets
$ ls /usr/bin | head -n 200 | xargs -I{} sh -c 'whatis {} 2>/dev/null | head -n 1' >
~/my-commands.txt
$ wc -l ~/my-commands.txt
```

(`compgen -c` হলো bash builtin, যা সব alias, builtin, function আর `PATH`-এর প্রোগ্রামের নাম দেখায়; সংখ্যাটা তোমার মেশিনে আলাদা হবে।)

চ্যালেঞ্জ: `~/my-commands.txt` থেকে শুধু সেকশন ৮ (অ্যাডমিন কমান্ড) এর তালিকা বের করো।

▼ সমাধান দেখো

```
$ grep '(8)' ~/my-commands.txt
```

ল্যাব ২: দক্ষতার স্তর যাচাই (অধ্যায় ৮–২৩)

প্রতিটা কাজ **এক লাইনের কমান্ডে** করো, কোনো সাহায্য ছাড়া। ১০টার মধ্যে ৮টা পারলে তুমি অন্তত স্তর ২-এ।

1. home ডিরেক্টরিতে `lab/a/b/c` ডিরেক্টরি একবারে বানাও।
2. `/etc` থেকে শুধু `.conf` দিয়ে শেষ হওয়া ফাইলের সংখ্যা গোনো (সরাসরি `/etc`-এর ভেতরে)।
3. `/etc/passwd` থেকে শুধু ইউজারনেমগুলো বর্ণানুক্রমে দেখাও।
4. যেসব ইউজারের শেল `/bin/bash` তাদের নাম দেখাও।
5. `/var/log`-এ সবচেয়ে বড় ৫টা জিনিস দেখাও।
6. `/etc`-এর ভেতরে "PermitRootLogin" লেখাটা কোন ফাইলে আছে খোঁজো।
7. গত ২৪ ঘণ্টায় home-এ বদলানো ফাইলগুলো খোঁজো।
8. একটা ফাইলের সব ছোট হাতের অক্ষর বড় হাতের করে নতুন ফাইলে রাখো।
9. সবচেয়ে বেশি মেমরি খাওয়া ৫টা প্রসেস দেখাও।
10. `~/lab` ডিরেক্টরিকে আজকের তারিখসহ নামে `.tar.gz` আর্কাইভ বানাও।

▼ সমাধান দেখো

```
1.  $ mkdir -p ~/lab/a/b/c
2.  $ ls /etc/*.conf | wc -l
3.  $ cut -d: -f1 /etc/passwd | sort
4.  $ awk -F: '$7 == "/bin/bash" {print $1}' /etc/passwd
5.  $ sudo du -sh /var/log/* 2>/dev/null | sort -h | tail -n 5
6.  $ sudo grep -rl "PermitRootLogin" /etc 2>/dev/null
7.  $ find ~ -type f -mtime -1
8.  $ tr 'a-z' 'A-Z' < input.txt > output.txt
9.  $ ps aux --sort=-%mem | head -n 6
10. $ tar -czvf lab-$(date +%F).tar.gz -C ~ lab
```

৯ নম্বরে `head -n 6` কারণ প্রথম লাইনটা হেডার।

ল্যাব ৩: সার্ভার স্বাস্থ্য-পরীক্ষার স্ক্রিপ্ট (অধ্যায় ২৪, ২৬, ৩২, ৩৭, ৪১–৪৩, ৪৬)

খ.৮-এর ৫৮ নম্বর প্রশ্নের উত্তরটাকে একটা স্ক্রিপ্টে রূপ দাও — যেটা চালালে সার্ভারের এক পাতার রিপোর্ট আসে।

```
#!/usr/bin/env bash
# health-check.sh — সার্ভারের দ্রুত স্বাস্থ্য রিপোর্ট
set -euo pipefail

section() { printf '\n===== %s =====\n' "$1"; }

section "পরিচয়"
. /etc/os-release
echo "Host   : $(hostname)"
echo "OS     : ${PRETTY_NAME}"
echo "Kernel : $(uname -r)"
echo "Uptime : $(uptime -p)"

section "লোড ও মেমরি"
cat /proc/loadavg
free -h

section "ডিস্ক (৮০%-এর বেশি ভরা হলে সতর্কতা)"
df -h --output=pcent,target -x tmpfs -x devtmpfs | tail -n +2 | while read -r pct mnt;
do
    use=${pct%\%}
    if (( use > 80 )); then
        echo "⚠️  ${mnt} ${pct} ভরা!"
    else
        echo "OK  ${mnt} ${pct}"
    fi
done

section "ব্যর্থ সার্ভিস"
systemctl --failed --no-legend || true

section "শুনছে এমন পোর্ট"
ss -tuln | awk 'NR>1 {print $1, $5}' | sort -u

section "সবচেয়ে বেশি CPU খাওয়া ৫টা প্রসেস"
ps -eo pid,user,%cpu,%mem,comm --sort=-%cpu | head -n 6
```

```
$ chmod +x health-check.sh
$ ./health-check.sh

===== পরিচয় =====
Host   : lab-ubuntu
OS     : Ubuntu 24.04.1 LTS
Kernel : 6.8.0-45-generic
Uptime : up 3 hours, 12 minutes
...
```

চ্যালেঞ্জ:

- `shellcheck health-check.sh` চালিয়ে কোনো সতর্কতা থাকলে ঠিক করো।
- স্ক্রিপ্টটা প্রতিদিন সকাল ৮টায় চালিয়ে আউটপুট `/var/log/health-$(date +%F).log`-এ রাখার cron বা systemd timer বানাও (অধ্যায় ২৯)।

ল্যাব ৪: OverTheWire Bandit — প্রথম ৫টা লেভেল

খ.৭-এর নির্দেশনা ধরে Bandit 0 থেকে 5 পর্যন্ত শেষ করো। প্রতিটা লেভেলের পর নিজের নোটে লেখো: (১) কী সমস্যা ছিল, (২) কোন কমান্ড দিয়ে সমাধান করলে, (৩) **কেন** সেটা কাজ করল।

> 💡 **টিপস:** Bandit লেভেল 1-এ ফাইলের নাম শুধু `-`। `cat -` লিখলে `cat` এটাকে "stdin থেকে পড়ো" ভাববে। ভাবো — কীভাবে বোঝাবে যে এটা একটা ফাইলের নাম? (ইঙ্গিত: path দিয়ে।)

ল্যাব ৫: নিজের ৯০ দিনের পরিকল্পনা লেখো

খ.৪-এর টেবিল কপি করে নিজের ক্যালেন্ডার অনুযায়ী তারিখ বসাও, আর প্রতি সপ্তাহের শেষে "পাস পরীক্ষা" করে ✅ দাও। এটা একটা Git রিপোজিটরিতে রাখো:

```
$ mkdir -p ~/linux-journey && cd ~/linux-journey
$ git init
$ nano PLAN.md          # রোডম্যাপ + তারিখ
$ nano NOTES.md         # প্রতিদিনের ৫টা নতুন কমান্ড
$ git add . && git commit -m "Day 1: start of my Linux journey"
```

সারসংক্ষেপ

এই পরিশিষ্টের মূল কথা

বিষয়	মূল কথা
কমান্ড রেফারেন্স	খ.২-এর A-Z টেবিল + খ.৩-এর কাজভিত্তিক সূচি; বিস্তারিত সংশ্লিষ্ট অধ্যায়ে
নিজে শেখা	`type` → `whatis` → `tldr` → `--help` → `man`
রোডম্যাপ	১৩ সপ্তাহ, চার তলা: ভিত → পাওয়ার ইউজার → অ্যাডমিন → এক্সপার্টের পথ
দক্ষতার স্তর	০ ফ্রেশার → ১ ব্যবহারকারী → ২ পাওয়ার ইউজার → ৩ জুনিয়র অ্যাডমিন → ৪ সিনিয়র/DevOps → ৫ এক্সপার্ট
সার্টিফিকেশন	হাতে-কলমে: LFCS, RHCSA, RHCE; multiple-choice: LPIC-1/2/3, Linux+
অনুশীলন	OverTheWire Bandit, SadServers, Killercoda, VM-ভিত্তিক home lab
ইন্টারভিউ	সংজ্ঞা → কেন/ভেতরে কীভাবে → উদাহরণ; না জানলে খোঁজার পদ্ধতি বলো

সবচেয়ে বেশি ব্যবহৃত ৩০টা কমান্ড — শেষবারের ঝালাই

দৈনন্দিন	টেক্সট	সিস্টেম	নেটওয়ার্ক ও অন্যান্য
`ls -lah`	`grep -rn`	`systemctl status`	`ip a`
`cd -`	`sed -i 's/a/b/g'`	`journalctl -xeu`	`ss -tulpn`
`cp -a`	`awk '{print $1}'`	`ps aux`	`ping -c 4`
`mv`	`sort | uniq -c`	`top` / `htop`	`curl -I`
`rm -i`	`cut -d: -f1`	`df -h` / `du -sh`	`ssh` / `rsync -avz`
`find -name`	`less` / `tail -f`	`free -h`	`tar -czvf` / `-xvf`
`chmod` / `chown`	`wc -l`	`sudo`	`man` / `tldr`
`mkdir -p`	`xargs`	`kill`	`crontab -e`

অপরিহার্য শর্টকাট (বিস্তারিত: পরিশিষ্ট ক)

শর্টকাট	কাজ
`Tab` / `Tab Tab`	অটো-কমপ্লিট / সব সম্ভাবনা দেখাও
`Ctrl+C`	চলমান কমান্ড থামাও (SIGINT)
`Ctrl+Z`	কমান্ড সাময়িক থামাও (SIGTSTP) — পরে `fg` / `bg`
`Ctrl+D`	ইনপুট শেষ (EOF) / শেল থেকে বের হও
`Ctrl+R`	হিস্ট্রিতে উল্টো খোঁজো
`Ctrl+A` / `Ctrl+E`	লাইনের শুরু / শেষে যাও
`Ctrl+U` / `Ctrl+K`	কার্সারের আগের / পরের অংশ কাটো
`Ctrl+W`	আগের শব্দ মোছো
`Ctrl+L`	স্ক্রিন পরিষ্কার
`Alt+.`	আগের কমান্ডের শেষ আর্গুমেন্ট বসাও
`!!`	আগের পুরো কমান্ড (যেমন `sudo !!`)
man/less-এ `/`, `n`, `q`	খোঁজো, পরের মিল, বের হও
Vim-এ `Esc` `:wq` / `:q!`	সেভ করে বের হও / সেভ ছাড়া বের হও
tmux-এ `Ctrl+b` তারপর `d`	সেশন detach

নিজেকে যাচাই করো

1. একটা কমান্ডের নাম জানো কিন্তু সেটা alias, builtin নাকি আলাদা প্রোগ্রাম — কীভাবে জানবে?
2. `man 5 passwd` আর `man passwd`-এর পার্থক্য কী?

3. ৯০ দিনের রোডম্যাপে স্টোরেজ (LVM) কেন নেটওয়ার্কিংয়ের আগে, আর স্ক্রিপ্টিং কেন টেক্সট টুলের পরে?
4. LFCS/RHCSA আর LPIC-1-এর পরীক্ষার ধরনে মূল পার্থক্য কী?
5. RHCE পরীক্ষা মূলত কোন টুলের ওপর?
6. OverTheWire Bandit-এ প্রথম লেভেলে কীভাবে ঢুকবে (পুরো কমান্ড)?
7. home lab-এ বুট প্রসেস আর kernel মডিউল শিখতে Docker কন্টেইনার কেন যথেষ্ট নয়?
8. `df` ১০০% দেখাচ্ছে কিন্তু `du`-এর যোগফল কম — সবচেয়ে সাধারণ কারণ ও খোঁজার কমান্ড কী?
9. `"$@"` আর `"$*"`-এর পার্থক্য কী?
10. কন্টেইনার আলাদাকরণ (isolation) kernel-এর কোন দুটো প্রধান ফিচারের ওপর দাঁড়িয়ে?

▼ **উত্তর দেখো**

1. `type name` (সব সম্ভাবনা দেখতে `type -a name`)।
2. `man passwd` (সেকশন ১) দেখায় `passwd` **কমান্ড**; `man 5 passwd` দেখায় `/etc/passwd` **ফাইলের ফরম্যাট**।
3. LVM-এর পরে ব্যাকআপ (`rsync`) আসে, আর রিমোট ব্যাকআপ/SSH-এর জন্য নেটওয়ার্কিং লাগে — তাই পরপর; স্ক্রিপ্টিং আসলে grep/sed/awk/প্রসেস/cron-কে জোড়া লাগানোর ভাষা, তাই ওগুলো আগে জানতে হয়।
4. LFCS/RHCSA **performance-based** — আসল মেশিনে কাজ করে দেখাতে হয়; LPIC-1 মূলত **multiple-choice ও fill-in-the-blank**।
5. **Ansible** (অটোমেশন)।
6. `ssh bandit0@bandit.labs.overthewire.org -p 2220`, পাসওয়ার্ড `bandit0`।
7. কন্টেইনার হোস্টের **kernel শেয়ার** করে, নিজে বুট হয় না (BIOS/GRUB/initramfs নেই), আর সাধারণত kernel মডিউল লোড করতে পারে না — এগুলো শিখতে পূর্ণ VM লাগে।
8. মুছে ফেলা কিন্তু কোনো প্রসেসের খুলে রাখা ফাইল; `sudo lsof +L1` (বা `lsof | grep deleted`), সমাধান — সংশ্লিষ্ট প্রসেস রিস্টার্ট।
9. `"$@"` প্রতিটা আর্গুমেন্টকে আলাদা শব্দ রাখে (স্পেসসহ অক্ষত); `"$*"` সবগুলো জুড়ে একটা শব্দ বানায়।
10. **namespaces** (কী দেখতে পাবে) আর **cgroups** (কতটা রিসোর্স পাবে)।

🎓 **শেষ কথা:** অভিনন্দন! তুমি পুরো বইটা পার করে এসেছ। মনে রাখো, লিনাক্স শেখা কখনো "শেষ" হয় না — এক্সপার্টরাও প্রতিদিন `man` খোলে। এই বই তোমাকে মানচিত্র দিয়েছে; এখন রাস্তায় হাঁটার পালা। একটা টার্মিনাল খোলো, একটা VM বানাও, কিছু ভাঙো, তারপর ঠিক করো। শুভকামনা! 🐧